

AV2 Bitstream & Decoding Process Specification



Final Deliverable, 28 May 2026

Version:

[v1.0.0](#)

Issue Tracking:

[GitHub](#)

Copyright 2026, [Alliance for Open Media](#)

Licensing information is available at <https://www.aomedia.org/license/>

The MATERIALS ARE PROVIDED “AS IS.” The Alliance for Open Media, its members, and its contributors expressly disclaim any warranties (express, implied, or otherwise), including implied warranties of merchantability, non-infringement, fitness for a particular purpose, or title, related to the materials. The entire risk as to implementing or otherwise using the materials is assumed by the implementer and user. IN NO EVENT WILL THE ALLIANCE FOR OPEN MEDIA, ITS MEMBERS, OR CONTRIBUTORS BE LIABLE TO ANY OTHER PARTY FOR LOST PROFITS OR ANY FORM OF INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS DELIVERABLE OR ITS GOVERNING AGREEMENT, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND WHETHER OR NOT THE OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Abstract

This document defines the bitstream format and decoding process for the [Alliance for Open Media](#) Video 2 (AV2) codec.

Table of Contents	Pages
1 Scope	17
2 Terms and definitions	18
3 Symbols	29
4 Conventions	41
4.1 General	41
4.2 Arithmetic operators	41
4.3 Ternary operator	41
4.4 Logical operators	42
4.5 Relational operators	42
4.6 Bitwise operators	42
4.7 Assignment	42
4.8 Mathematical functions	42
4.9 Method of describing bitstream syntax	44
4.10 Functions	46
4.11 Descriptors	46
4.11.1 General	46
4.11.2 f(n)	46
4.11.3 uvlc()	46
4.11.4 svlc()	47
4.11.5 le(n)	47
4.11.6 leb128()	47
4.11.7 su(n)	48
4.11.8 ns(n)	49
4.11.9 tu(mx)	49
4.11.10 rg(n)	50
4.11.11 L(n)	50
4.11.12 S()	50
4.11.13 NS(n)	50
5 Syntax structures	52
5.1 General	52

5.2	OBU syntax	52
5.2.1	General OBU syntax	52
5.2.2	OBU header syntax	54
5.2.3	Trailing bits syntax	54
5.2.4	Byte alignment syntax	55
5.3	Reserved OBU syntax	55
5.4	Sequence header OBU syntax	55
5.4.1	General sequence header OBU syntax	55
5.4.2	Sequence tile config syntax	59
5.4.3	Sequence partition config syntax	59
5.4.4	Sequence segment config syntax	60
5.4.5	Sequence intra config syntax	60
5.4.6	Sequence inter config syntax	60
5.4.7	Sequence screen content config syntax	63
5.4.8	Sequence transform quant entropy config syntax	64
5.4.9	Segment information syntax	66
5.4.10	Sequence filter config syntax	66
5.4.11	User defined QM syntax	67
5.4.12	Timing info syntax	69
5.4.13	Sequence decoder model info syntax	69
5.5	Temporal delimiter OBU syntax	69
5.6	Multi Stream Decoder Operation OBU syntax	69
5.7	Multi frame header OBU syntax	70
5.8	Layer config record OBU syntax	71
5.8.1	LCR global info syntax	71
5.8.2	LCR local info syntax	72
5.8.3	LCR aggregate info syntax	72
5.8.4	LCR sequence profile tier level information syntax	73
5.8.5	LCR global payload syntax	73
5.8.6	LCR xlayer info syntax	73
5.8.7	LCR rep info syntax	74
5.8.8	LCR embedded layer info syntax	74

5.8.9 LCR xlayer color info syntax	75
5.9 Atlas segment info OBU syntax	75
5.9.1 Atlas label segment info syntax	76
5.9.2 Atlas enhanced atlas info syntax	76
5.9.3 Atlas multistream info syntax	78
5.9.4 Atlas multistream with alpha info syntax	78
5.9.5 Atlas basic info syntax	79
5.10 Operating point set OBU syntax	80
5.11 Operating point payload syntax	80
5.11.1 Operating point set aggregate info syntax	82
5.11.2 Operating point set sequence profile tier level information syntax	82
5.11.3 Operating point set decoder model info syntax	82
5.11.4 Operating point set color info syntax	82
5.11.5 Operating point set mlayer info syntax	82
5.12 Buffer removal timing OBU syntax	83
5.13 Quantizer Matrix OBU syntax	83
5.14 Film grain OBU syntax	84
5.15 Content interpretation OBU syntax	85
5.16 Padding OBU syntax	86
5.17 Metadata OBU syntax	87
5.17.1 Metadata unit syntax	87
5.17.2 Metadata short OBU syntax	88
5.17.3 Metadata group OBU syntax	88
5.17.4 Metadata ITUT T35 syntax	90
5.17.5 Metadata high dynamic range content light level syntax	90
5.17.6 Metadata high dynamic range mastering display color volume syntax	90
5.17.7 Metadata timecode syntax	90
5.17.8 Metadata banding hints syntax	91
5.17.9 Metadata ICC profile syntax	92
5.17.10 Metadata scan type syntax	92
5.17.11 Metadata temporal point info syntax	92
5.17.12 Metadata decoded frame hash syntax	93

5.17.13 Metadata user data unregistered syntax	93
5.18 Frame header syntax	93
5.18.1 General frame header syntax	93
5.18.2 Frame header info syntax	94
5.18.3 Frame configuration structures	113
5.18.4 Frame size structures	115
5.18.5 Filtering structures	116
5.18.6 Quantization structures	118
5.18.7 Segmentation and tiling structures	120
5.18.8 Transform and coding mode structures	135
5.18.9 Global motion structures	137
5.18.10 Film grain structures	141
5.19 Tile group OBU syntax	144
5.20 Tile group payload syntax	145
5.20.1 General tile group payload syntax	145
5.20.2 Tile-level structures	146
5.20.3 Partition structures	148
5.20.4 Block decoding structures	161
5.20.5 Mode information structures	169
5.20.6 Transform and quantization structures	182
5.20.7 Motion vector and prediction structures	186
5.20.8 Coding tools structures	235
5.20.9 Helper functions	243
5.20.10 Filtering structures	244
6 Syntax structures semantics	255
6.1 General	255
6.2 OBU semantics	255
6.2.1 General OBU semantics	255
6.2.2 OBU header semantics	255
6.2.3 Trailing bits semantics	258
6.2.4 Byte alignment semantics	258
6.3 Reserved OBU semantics	258

6.4	Sequence header OBU semantics	258
6.4.1	General sequence header OBU semantics	258
6.4.2	Sequence tile config semantics	264
6.4.3	Sequence partition config semantics	264
6.4.4	Sequence segment config semantics	265
6.4.5	Sequence intra config semantics	266
6.4.6	Sequence inter config semantics	266
6.4.7	Sequence screen content config semantics	270
6.4.8	Sequence transform quant entropy config semantics	270
6.4.9	Segment information semantics	271
6.4.10	Sequence filter config semantics	271
6.4.11	User defined QM semantics	273
6.4.12	Timing info semantics	273
6.4.13	Sequence decoder model info semantics	274
6.5	Temporal delimiter OBU semantics	274
6.6	Multi Stream Decoder Operation OBU semantics	274
6.7	Multi frame header OBU semantics	276
6.8	Layer config record OBU semantics	276
6.8.1	General	277
6.8.2	LCR global info semantics	278
6.8.3	LCR local info semantics	280
6.8.4	LCR aggregate info semantics	281
6.8.5	LCR sequence profile tier level information semantics	281
6.8.6	LCR global payload semantics	282
6.8.7	LCR xlayer info semantics	282
6.8.8	LCR rep info semantics	284
6.8.9	LCR embedded layer info semantics	284
6.8.10	LCR xlayer color info semantics	287
6.9	Atlas segment info OBU semantics	288
6.9.1	General	288
6.9.2	Atlas label segment info semantics	290
6.9.3	Atlas enhanced atlas info semantics	290

6.9.4	Atlas multistream info semantics	292
6.9.5	Atlas multistream with alpha info semantics	292
6.9.6	Atlas basic info semantics	292
6.10	Operating point set OBU semantics	293
6.10.1	General	293
6.10.2	Operating point set OBU syntax elements	294
6.10.3	Operating point set aggregate info semantics	297
6.10.4	Operating point set sequence profile tier level information semantics	297
6.10.5	Operating point set decoder model info semantics	297
6.10.6	Operating point set color info semantics	298
6.10.7	Operating point set mlayer info semantics	300
6.11	Buffer removal timing OBU semantics	301
6.12	Quantizer Matrix OBU semantics	301
6.13	Film grain OBU semantics	302
6.14	Content interpretation OBU semantics	303
6.15	Padding OBU semantics	305
6.16	Metadata OBU semantics	305
6.16.1	Metadata unit semantics	305
6.16.2	Metadata short OBU semantics	305
6.16.3	Metadata group OBU semantics	306
6.16.4	Metadata ITUT T35 semantics	309
6.16.5	Metadata high dynamic range content light level semantics	309
6.16.6	Metadata high dynamic range mastering display color volume semantics	310
6.16.7	Metadata timecode semantics	312
6.16.8	Metadata banding hints semantics	313
6.16.9	Metadata ICC profile semantics	315
6.16.10	Metadata scan type semantics	315
6.16.11	Metadata temporal point info semantics	316
6.16.12	Metadata user data unregistered semantics	317
6.16.13	Metadata decoded frame hash semantics	317
6.17	Frame header OBU semantics	319
6.17.1	General frame header semantics	319

6.17.2	Frame header info semantics	319
6.17.3	Frame configuration structures	334
6.17.4	Frame size structures	335
6.17.5	Filtering structures	337
6.17.6	Quantization structures	337
6.17.7	Segmentation and tiling structures	339
6.17.8	Transform and coding mode structures	344
6.17.9	Global motion structures	345
6.17.10	Film grain structures	346
6.18	Tile group OBU semantics	349
6.19	Tile group payload semantics	349
6.19.1	General tile group payload semantics	349
6.19.2	Tile-level structures	350
6.19.3	Partition structures	351
6.19.4	Block decoding structures	354
6.19.5	Mode information structures	355
6.19.6	Transform and quantization structures	358
6.19.7	Motion vector and prediction structures	360
6.19.8	Coding tools structures	372
6.19.9	Filtering structures	373
7	Decoding process	375
7.1	General decoding process	375
7.2	Decode frame wrapup process	376
7.3	Ordering of OBUs	379
7.3.1	General	379
7.3.2	Coded multistream video sequence boundaries	380
7.3.3	Coded output frame unit	380
7.3.4	Coded non-output frame unit	382
7.3.5	Coded frame unit	383
7.3.6	Coded extended layer unit	383
7.3.7	Temporal unit	385
7.3.8	Availability of high level syntax OBUs	386

7.3.9 Availability of long-term reference frames	389
7.4 Random access decoding	390
7.4.1 General	390
7.4.2 Random access and use of long-term reference frames	391
7.4.3 Closed Random Access	391
7.4.4 Open Random Access	391
7.4.5 Random Access Switch	393
7.4.6 Multistream Random Access	394
7.5 Frame end update CDF process	395
7.6 Extended layer context management	395
7.7 Get ref frames process	397
7.8 Get past future cur ref lists process	400
7.9 Motion field estimation process	401
7.9.1 General	401
7.9.2 Fill trajectory motion vector gap process	406
7.9.3 Projection process	407
7.9.4 Get MV projection process	411
7.9.5 Get MV projection clamp process	411
7.9.6 Get sampled position process	412
7.9.7 Get block position no constraint process	412
7.9.8 Check block position process	413
7.10 Setup TIP motion field process	413
7.10.1 General	413
7.10.2 TIP temporal scale motion field process	414
7.10.3 TIP fill motion field holes process	415
7.10.4 TIP block average filter motion vector process	415
7.10.5 Fill temporal motion vectors sample gap process	416
7.10.6 Build TIP process	418
7.11 Motion vector context processes	419
7.11.1 General	419
7.11.2 Find mode context process	419
7.11.3 Scan point context process	420

7.11.4	Scan point warp context process	421
7.12	Motion vector prediction processes	422
7.12.1	General	422
7.12.2	Find MV stack process	422
7.12.3	Find warp samples process	444
7.13	Prediction processes	447
7.13.1	General	447
7.13.2	Intra prediction process	447
7.13.3	Inter prediction process	465
7.13.4	Palette prediction process	507
7.13.5	Predict chroma from luma process	508
7.13.6	MHCCP process	511
7.13.7	Derive multi param process	512
7.13.8	Gaussian elimination process	514
7.14	Reconstruction and dequantization	515
7.14.1	General	515
7.14.2	Dequantization functions	515
7.14.3	Reconstruct process	516
7.14.4	Dequantization process	517
7.14.5	Save dequant process	519
7.14.6	Get dequant process	519
7.15	Inverse transform process	520
7.15.1	General	520
7.15.2	1D transforms	520
7.15.3	Secondary transform process	522
7.15.4	2D inverse transform process	524
7.16	Deblocking filter for TIP process	528
7.17	Deblocking filter process	530
7.17.1	General	530
7.17.2	Edge deblocking filter process	531
7.17.3	Filter maximum width process	535
7.17.4	Filter size process	536

7.17.5 Adaptive filter strength process	536
7.17.6 Adaptive filter strength selection process	537
7.17.7 Sample filtering process	537
7.18 CDEF process	540
7.18.1 CDEF block process	540
7.18.2 CDEF direction process	542
7.18.3 CDEF filter process	544
7.19 CCSO process	545
7.19.1 Apply CCSO filter process	546
7.20 Loop restoration process	548
7.20.1 Loop restore block process	549
7.20.2 Get source sample process	552
7.20.3 Non-separable Wiener filter process	553
7.20.4 Pixel classified Wiener filter process	555
7.20.5 Apply GDF filter process	557
7.21 Output processes	560
7.21.1 Output process	560
7.21.2 Intermediate output preparation process	561
7.21.3 Output successive frames process	562
7.21.4 Output implicit output frame process	562
7.21.5 Flush implicit output frames process	563
7.21.6 Output frame buffers process	564
7.21.7 Film grain synthesis process	565
7.22 Motion field motion vector storage process	572
7.23 Reference frame update process	576
8 Parsing process	579
8.1 Parsing process for $f(n)$	579
8.2 Parsing process for symbol decoder	579
8.2.1 General	579
8.2.2 Initialization process for symbol decoder	579
8.2.3 Boolean decoding process	585
8.2.4 Exit process for symbol decoder	585

8.2.5	Parsing process for read_literal	587
8.2.6	Symbol decoding process	587
8.3	Parsing process for CDF encoded syntax elements	588
8.3.1	General	588
8.3.2	Cdf selection process	589
9	Additional tables	609
9.1	General	609
9.2	Conversion tables	609
9.3	Default CDF tables	623
9.4	Quantizer matrix tables	742
9.4.1	General	742
9.4.2	Derivation process (Informative)	743
9.4.3	Tables	743
9.5	Warp filter tables	841
9.6	1d transform tables	845
9.7	Secondary transform tables	849
9.8	Loop restoration tables	921
Annex A:	Profiles, levels, and tiers	1101
A.1	General	1101
A.2	Profiles	1101
A.3	Multi-sequence configurations	1104
A.4	Levels	1104
A.5	Decoder Conformance	1111
Annex B:	Length delimited bitstream format	1112
B.1	Overview	1112
B.2	Length delimited bitstream syntax	1112
B.3	Length delimited bitstream semantics	1112
Annex C:	Error resilience behavior (informative)	1113
C.1	General	1113
C.2	Definition of processable frames	1113
C.3	Recommendation for processable frames	1114
C.4	Encoder consequences of processable frames	1114

C.5 Decoder consequences of processable frames	1114
Annex D: Multistream composition process (informative)	1115
D.1 General	1115
D.2 Chroma format determination process	1116
D.3 Array initialization process	1117
D.3.1 Background color determination process	1117
D.4 Spatial mapping process	1118
D.5 Spatial mapping with alpha process	1119
D.5.1 Frame resampling process	1120
D.5.2 Monochrome frame resampling process	1121
Annex E: Decoder model	1122
E.1 General	1122
E.2 Operating point selection	1123
E.3 Decoder model definitions	1123
E.4 Operating modes	1126
E.4.1 Resource availability mode	1126
E.4.2 Decoding schedule mode	1127
E.4.3 Establishing bitstream conformance	1128
E.4.4 When timing information is not present in the bitstream	1128
E.5 Frame timing definitions	1128
E.5.1 Start of DFG bits arrival	1128
E.5.2 End of DFG bits arrival	1129
E.5.3 Scheduled removal times	1129
E.5.4 Removal times in decoding schedule mode	1129
E.5.5 Removal times in resource availability mode	1130
E.5.6 Frame decode timing	1131
E.5.7 Frame presentation timing	1131
E.6 Decoder model	1132
E.6.1 Decoder model structure	1132
E.6.2 Decoder model functions	1133
E.6.3 Decoder model error codes	1136
E.7 Bitstream conformance	1136

E.7.1 General	1136
E.7.2 Decoder buffer delay consistency across random access points (applies to decod	1137
E.7.3 Smoothing buffer overflow	1137
E.7.4 Smoothing buffer underflow	1137
E.7.5 Minimum decode time (applies to decoding schedule mode)	1137
E.7.6 Minimum presentation interval	1138
E.7.7 Decode deadline	1138
E.7.8 Level imposed constraints	1138
E.7.9 Decode Process constraints	1138
Annex F: Sub-bitstream extraction (informative)	1139
F.1 General	1139
F.2 Operating point usage	1139
F.2.1 General decoder operation	1139
F.2.2 Multistream bitstream decoder operation	1139
F.2.3 Singlestream bitstream decoder operation	1142
F.3 Sub-bitstream extraction processes	1143
F.3.1 Operating point selection and analysis process	1145
F.3.2 Sub-bitstream extraction process	1149
F.3.3 Preserved OBU types	1150
Annex G: Layer composition and Atlas usage examples (informative)	1151
G.1 General	1151
G.2 360-degree viewport-dependent streaming with subpictures	1151
G.2.1 Layer structure	1152
G.2.2 LCR configuration	1153
G.2.3 Atlas configuration	1154
G.2.4 Viewport-dependent streaming process	1155
G.2.5 Benefits for 360-degree streaming	1156
G.3 Subpicture composition example	1158
G.3.1 LCR configuration	1158
G.3.2 Atlas configuration	1159
G.3.3 Rendering and adaptive streaming	1159
G.4 Region-of-interest scalability example with encoder padding	1161

G.4.1 LCR configuration	1162
G.4.2 Atlas configuration	1162
G.4.3 Rendering scenarios	1163
G.5 Implementation considerations	1165
G.5.1 Decoder requirements	1165
G.5.2 Encoder recommendations	1166
G.5.3 Interoperability	1166
Index	1167
Terms defined by this specification	1167
References	1168
Normative References	1168
Informative References	1169

Introduction

This document specifies the bitstream format and decoding process for the Alliance for Open Media Video 2 (AV2) codec. It is intended to be read by implementers of AV2 decoders and encoders, by authors of container and transport formats that carry AV2 bitstreams, and by authors of conformance tests.

A conforming AV2 decoder is fully specified by the normative content of [§ 4 Conventions](#), [§ 5 Syntax structures](#), [§ 6 Syntax structures semantics](#), [§ 7 Decoding process](#), [§ 8 Parsing process](#), [Annex A: Profiles, levels, and tiers](#), and [Annex E: Decoder model](#). The informative annexes describe recommended behavior and illustrative use cases and are not required for conformance. Informative content is identified either by an annex title ending with "(informative)", by an introductory statement, or by paragraphs beginning with the word "Note" that are visually set apart from the surrounding text.

A first reading of this document is recommended in the following order:

1. [§ 2 Terms and definitions](#) and [§ 3 Symbols](#) to establish vocabulary. Defined terms appear as links throughout the document, and following a link navigates to the term's definition.
2. [§ 4 Conventions](#) to understand the mathematical operators, pseudocode style, and descriptor notation used in the syntax tables. Syntax element descriptors such as $f(n)$ and $L(n)$ are defined in [§ 8 Parsing process](#).
3. [§ 5 Syntax structures](#) alongside [§ 6 Syntax structures semantics](#). The syntax structures, presented as pseudocode, define the order in which bits are read. The semantics define the meaning of each syntax element and the variables it updates.
4. [§ 7 Decoding process](#) and [§ 8 Parsing process](#), which together describe how a conforming decoder transforms a sequence of OBUs into decoded frames.
5. [Annex A: Profiles, levels, and tiers](#) for conformance constraints and [Annex E: Decoder model](#) for the decoder model.

The informative annexes may be consulted as needed: [Annex C: Error resilience behavior \(informative\)](#) for decoding from non-key starting points, [Annex D: Multistream composition process \(informative\)](#) for composing decoded frames from a [multistream](#), [Annex F: Sub-bitstream extraction \(informative\)](#) for extracting sub-bitstreams based on operating points, and [Annex G: Layer composition and Atlas usage examples \(informative\)](#) for usage examples of the layer configuration record.

§ 1. Scope

This document specifies the Alliance for Open Media Video 2 (AV2) bitstream format and decoding process.

[Back to Table of Contents](#)

§ 2. Terms and definitions

For the purposes of this document, the following terms and definitions apply:

AC coefficient

Any transform coefficient whose frequency indices are non-zero in at least one dimension.

ADST

Asymmetric Discrete Sine Transform.

AOMedia

Alliance for Open Media.

Atlas

A virtual 2D image associated with the decoded layers of a bitstream. The atlas can provide information on how to interpret, render, and utilize all such layers, depending on the application.

Base layer

A layer with obu_mlayer_id and obu_tlayer_id values equal to 0.

BAWP

Block Adaptive Weighted Prediction modifies inter prediction samples with a linear equation based on a scaling factor and an offset. The model parameters are based on observations from surrounding samples in the decoded frame and reference frame or the OrderHints distance of the decoded frame and reference frame.

Bitstream

The sequence of bits generated by encoding a sequence of frames.

Bit string

An ordered string with limited number of bits. The leftmost bit is the most significant bit (MSB), the rightmost bit is the least significant bit (LSB).

Block

A square or rectangular region of samples.

Bridge frame

A non-output [inter frame](#) that produces a copy of a single [reference frame](#) at equal or reduced resolution for storage in the reference buffer. A bridge frame contains no coded residual data and all prediction is performed with zero motion vectors.

BRU

Backwards reference update allows an existing reference frame to be partially updated.

Byte

A string of 8 bits.

Byte alignment

One bit is byte aligned if the position of the bit is an integer multiple of eight from the position of the first bit in the bitstream.

CCSO

Cross Component Sample Offset filter designed to modify both luma and chroma samples based on luma brightness and brightness gradient.

CCTX

Cross Component Transform. A transform that jointly processes chroma components to exploit correlation between Cb and Cr.

CDEF

Constrained Directional Enhancement Filter designed to adaptively filter blocks based on identifying the direction.

CDF

Cumulative distribution function representing the probability times 32768 that a symbol has value less than or equal to a given level.

CFL

Chroma from Luma. An intra prediction tool that derives chroma samples from reconstructed luma samples.

Chroma

A sample value matrix or a single sample value of one of the two color difference signals.

NOTE: Symbols of chroma are U and V.

CLK

Closed Loop Key. A coded frame with obu_type equal to OBU_CLOSED_LOOP_KEY. See [closed random access](#).

Closed random access

The random access process that applies to an extended layer when the first coded frame unit of its coded extended layer unit has obu_type equal to OBU_CLOSED_LOOP_KEY. The process starts a new [coded video sequence](#) for the extended layer. See [§ 7.4.3 Closed Random Access](#).

Coded frame

The representation of one frame before the decoding process.

Coded multistream video sequence

A set of [coded video sequences](#) across two or more [extended layers](#) that satisfies the requirements specified in [§ 7.3 Ordering of OBUs](#).

Coded video sequence

A sequence of temporal units for an extended layer, starting at a [closed random access](#) point and continuing until the next closed random access point for that extended layer or the end of the bitstream. See [§ 7.3.6 Coded extended layer unit](#).

NOTE: When a decoder initiates decoding at an [open random access](#) point, the decoding process treats it as if it were the start of a new coded video sequence (see [§ 7.4.4 Open Random Access](#)), but during sequential decoding an open random access point does not start a new coded video sequence.

Component

One of the three sample value matrices (one luma matrix and two chroma matrices) or its single sample value.

Compound prediction

A type of inter prediction where sample values are computed by blending together predictions from two reference frames (the frames blended can be the same or different).

DC coefficient

A transform coefficient whose frequency indices are zero in all dimensions.

DCT

Discrete Cosine Transform.

DDT

Data Dependent Transform.

Decoded frame

The frame reconstructed out of the bitstream by the decoder.

Decoder

One embodiment of the decoding process.

Decoding process

The process that derives decoded frames from syntax elements, including any processing steps used prior to and for the film grain synthesis process.

Dequantization

The process in which transform coefficients are obtained by scaling the quantized coefficients.

Embedded layer

A set of OBUs with identical obu_xlayer_id and obu_mlayer_id values.

Encoder

One embodiment of the [encoding process](#).

Encoding process

A process not specified in this specification that generates the bitstream that conforms to the description provided in this document.

Enhancement layer

A layer with either obu_mlayer_id greater than 0 or obu_tlayer_id greater than 0.

EOB

End of a transform block. The scan position one past the last non-zero coefficient in a [transform block](#).

Extended layer

A set of OBUs with identical obu_xlayer_id values.

Frame

The representation of video signals in the spatial domain, composed of one luma sample matrix (Y) and zero or two chroma sample matrices (U and V).

Frame context

A set of probabilities used in the decoding process.

Frame header info

High level description of the frame to be decoded that is encoded without the use of arithmetic encoding.

FSC

Forward Skip Coding. A coding mode for a block that skips the regular coefficient coding process and uses special coefficient coding rules with a forward scan and first position.

GDF

Guided detail filter designed to selectively enhance details.

Global operating point set

An [OPS](#) OBU with obu_xlayer_id equal to GLOBAL_XLAYER_ID that describes operating points applicable to the entire [multistream](#) bitstream, potentially spanning multiple extended layers.

IBP

Intra bi-prediction blends two different intra predictions together for a single block.

Inter coding

Coding one block or frame using inter prediction.

Inter frame

A frame compressed by referencing previously decoded frames and that may use intra prediction or inter prediction.

Inter prediction

The process of deriving the [prediction value](#) for the current frame using previously decoded frames.

Intra coding

Coding one block or frame using [intra prediction](#).

Intra frame

A frame compressed using only [intra prediction](#) which can be independently decoded.

Intra prediction

The process of deriving the [prediction value](#) for the current sample using previously decoded sample values in the same decoded frame.

Inverse transform

The process in which a transform coefficient matrix is transformed into a spatial sample value matrix.

IST

Intra-inter Secondary Transforms. An additional transform applied to low-frequency coefficients after the primary transform to further decorrelate the residual signal.

Key frame

A coded frame with obu_type equal to OBU_CLOSED_LOOP_KEY or OBU_OPEN_LOOP_KEY.

Layer

A set of OBUs with identical obu_xlayer_id, obu_mlayer_id, and obu_tlayer_id values.

LCR

Layer Configuration Record.

Leading frame

A frame with obu_type equal to OBU_LEADING_TILE_GROUP, OBU_LEADING_SEF, or OBU_LEADING_TIP (i.e., IsRegular is equal to 0). Leading frames can follow an open random access point and may reference frames that precede the open random access point. See [§ 7.4.4 Open Random Access](#).

Level

A defined set of constraints on the values for the syntax elements and variables.

LF

Low frequency region of a transform block.

Local operating point set

An [OPS](#) OBU with obu_xlayer_id not equal to GLOBAL_XLAYER_ID that describes operating points applicable to a single extended layer sub-bitstream.

Long-term reference frame

A reference frame that has been assigned a long-term identifier via the long_term_id_plus_1 syntax element. Only [key frames](#) can be designated as long-term reference frames.

Deblocking filter

A filtering process applied to the reconstruction intended to reduce the visibility of block edges.

LSB

Least Significant Bit.

Luma

A sample value matrix or a single sample value representing the monochrome signal related to the primary colors.

NOTE: The symbol representing luma is Y.

MHCCP

Multi-hypothesis cross component prediction.

Mode info

Syntax elements sent for a block containing an indication of how a block is to be predicted during the decoding process.

Mode info block

A luma sample value block of size 4x4 or larger and its two corresponding chroma sample value blocks (if present).

Motion vector

A two-dimensional vector used for inter prediction which refers the current frame to the reference frame, the value of which provides the coordinate offsets from a location in the current frame to a location in the reference frame.

MRL

Multiple Reference Line. An intra prediction tool that allows using reference samples from lines beyond the immediately adjacent line for directional intra prediction modes.

MSB

Most Significant Bit.

MSDO

Multi Stream Decoder Operation.

Multistream

A bitstream that contains two or more distinct non-global values for the extended layer identifier.

OBU

All structures are packetized in "Open Bitstream Units" or OBUs. Each OBU has a header, which provides identifying information for the contained data (payload).

OLK

Open Loop Key. A coded frame with obu_type equal to OBU_OPEN_LOOP_KEY. See [open random access](#).

Open random access

The random access process that applies to an extended layer when the first coded frame unit of its coded extended layer unit has obu_type equal to OBU_OPEN_LOOP_KEY. During sequential decoding, the process does not start a new [coded video sequence](#) for the extended layer. However, when a decoder initiates decoding at the open random access point, the process is treated as if it were the start of a new coded video sequence for the extended layer. [Leading frames](#) that follow the OLK are discarded. See [§ 7.4.4 Open Random Access](#).

OPS

Operating Point Set.

Parity hiding

A coefficient coding technique that hides the parity of the [DC coefficient](#) level in the parity of the sum of coefficient levels in the same transform block, allowing the [DC coefficient](#) to be coded with reduced precision.

Parse

The procedure of getting the syntax element from the bitstream.

Picture

A frame (before content interpretation) produced by the decoding process.

The decoding process works exclusively with frames.

However, content interpretation metadata allows a decoded frame to be interpreted as a field.

The term picture can be used to emphasize that the text refers to the decoded frame and its associated information regardless of whether it is interpreted as a frame picture or a field picture.

Prediction

The implementation of the prediction process consisting of either inter or intra prediction.

Prediction process

The process of estimating the decoded sample value or data element using a predictor.

Prediction value

The value, which is the combination of the previously decoded sample values or data elements, used in the decoding process of the next sample value or data element.

Profile

A subset of syntax, semantics and algorithms defined in a part.

Quantization parameter

A variable used for scaling the quantized coefficients in the decoding process.

Quantized coefficient

A transform coefficient before dequantization.

RAS

Random Access Switch. A coded frame with obu_type equal to OBU_RAS_FRAME. See [random access switch](#).

Random access switch

The random access process that applies to an extended layer when the coded extended layer unit contains an OBU with obu_type equal to OBU_RAS_FRAME. The process does not start a new [coded video sequence](#). The RAS frame is inter-predicted using [long-term reference frames](#) identified by ref_long_term_id. See [§ 7.4.5 Random Access Switch](#).

Raster scan

Maps a two dimensional rectangular raster into a one dimensional raster, in which the entry of the one dimensional raster starts from the first row of the two dimensional raster, and the scanning then goes through the second row and the third row, and so on. Each raster row is scanned in left to right order.

Reconstruction

Obtaining the addition of the decoded residual and the corresponding [prediction values](#).

Reference

One of a set of tags, each of which is mapped to a reference frame.

Reference frame

A storage area for a previously decoded frame and associated information.

Regular frame

A frame with obu_type equal to OBU_OPEN_LOOP_KEY, OBU_REGULAR_TILE_GROUP, OBU_REGULAR_TIP, OBU_REGULAR_SEF, OBU_SWITCH, OBU_RAS_FRAME or OBU_BRIDGE_FRAME (i.e., IsRegular is equal to 1).

Reserved

A special syntax element value which may be used to extend this part in the future.

Residual

The differences between the reconstructed samples and the corresponding [prediction values](#).

Sample

The basic elements that compose the frame.

Sample value

The value of a sample. This is an integer from 0 to 255 (inclusive) for 8-bit frames, and from 0 to 1023 (inclusive) for 10-bit frames.

SDP

Semi-Decoupled Partitioning. A partitioning mode where chroma blocks can use a different partition structure than luma blocks.

SEF

Show Existing Frame. A coded frame with obu_type equal to OBU_REGULAR_SEF or OBU_LEADING_SEF.

Segmentation map

One 4-bit number per 4x4 block in the frame specifying the segment affiliation of that block. A segmentation map is stored for each reference frame to allow new frames to use a previously coded map.

Sequence

The highest level syntax structure of the coding bitstream, including one or several consecutive coded frames.

Singlestream

A bitstream that contains a single distinct non-global value for the extended layer identifier.

Sub-bitstream

A bitstream derived from another bitstream through the [sub-bitstream extraction process](#), containing only OBUs associated with selected layers as determined by operating point selection.

Sub-bitstream extraction process

A specified process that extracts a [sub-bitstream](#) from a bitstream by removing OBUs not associated with selected extended layers, embedded layers, and temporal layers. The layers to retain are determined by an operating point selection and analysis process which may involve one or more operating points from [OPS](#) OBUs (for example, a [global operating point set](#) and optionally one or more [local operating point sets](#) in a [multistream](#) bitstream). The output sub-bitstream contains only OBUs from the retained layers.

Superblock

The top level of the block tree within a tile. All superblocks within a frame are the same size and are square. The superblocks may be 256x256 luma samples, 128x128 luma samples, or 64x64 luma samples. A superblock may contain multiple blocks, which may themselves be further subpartitioned, forming the block tree.

Switch Frame

An inter frame that can be used as a point to switch between sequences. The intention is to allow a streaming use case where videos can be encoded in small chunks (say of 1 second duration), each starting with a switch frame. If the available bandwidth drops, the server can start sending chunks from a lower bitrate encoding instead. When this happens, the inter prediction uses the existing higher quality reference frames to decode the switch frame. This approach allows a bitrate switch without the cost of a full key frame.

Syntax element

An element of data represented in the bitstream.

TCQ

Trellis coded quantization adjusts the quantizer levels based on the parity of the decoded coefficients.

Temporal delimiter OBU

An indication that the following OBUs will have a different presentation/decoding time stamp from the one of the last frame prior to the temporal delimiter.

Temporal layer

A set of OBUs with identical obu_xlayer_id and obu_tlayer_id values.

Temporal unit

A Temporal unit consists of all the OBUs that are associated with a specific, distinct time instant. It consists of a temporal delimiter OBU, and all of the OBUs that follow, up to but not including the next temporal delimiter.

TG

Tile Group.

Tier

A specified category of level constraints imposed on the values of the syntax elements in the bitstream.

Tile

A rectangular region of the frame that can be decoded and encoded independently, although loop filtering across tile edges is still applied in some cases.

Tile Group

A group of one or more contiguous tiles in tile scan order, associated with a single frame and included in a single OBU with obu_type equal to OBU_REGULAR_TILE_GROUP or OBU_LEADING_TILE_GROUP.

TIP

Temporally interpolated prediction.

Transform block

A rectangular transform coefficient matrix, used as input to the inverse transform process.

Transform coefficient

A scalar value, considered to be in a frequency domain, contained in a transform block.

WAIP

Wide Angle Intra Prediction.

WHT

Walsh Hadamard Transform.

[↑ Back to Table of Contents](#)

§ 3. Symbols

The specification makes use of a number of constant integers. Constants that relate to the semantics of a particular [syntax element](#) are defined in [§ 6 Syntax structures semantics](#).

Additional constants are defined below:

Table 3.1: Additional constants used in the specification

Symbol name	Value	Description
ADST_ADST	3	Inverse transform rows with ADST and columns with ADST
ADST_DCT	1	Inverse transform rows with DCT and columns with ADST
ADST_FLIPADST	7	Inverse transform rows with FLIPADST and columns with ADST
AFFINE	2	Warp model is a general affine transform
ANGLE_STEP	3	Number of degrees of step-per-unit increase in AngleDeltaY or AngleDeltaUV.
BANK_REFS_PER_FRAME	9	Number of parameter banks for motion vectors
BAWP_SCALES_CTX_COUNT	3	Number of contexts for explicit_bawp
BLEND_WEIGHT_MAX	32	A blend weight used in smooth intra prediction
BLOCK_INVALID	29	Sentinel value to mark partition choices that are not allowed
BLOCK_SIZES	29	Number of different block sizes used
BLOCK_SIZE_GROUPS	4	Number of contexts when decoding y_mode
BR_CDF_SIZE	4	Number of values for coeff_br
CCSO_BAND_NUM	64	Maximum number of bands allowed in CCSO
CCSO_CONTEXT	4	Number of contexts when decoding ccso_blk
CCSO_INPUT_INTERVAL	3	Number of classes for CCSO
CCSO_LUMA_SIZE_LOG2	8	Base 2 logarithm of size of CCSO blocks (measured in luma samples)
CCTX_PREC_BITS	8	Precision bits used during cross component transform
CCTX_TYPES	7	Number of values for cctx_type
CDEF_ON_SKIP_TXFM_ADAPTIVE	2	Value indicating CDEF has a frame level enabled for whether it is used on skipped transform blocks
CDEF_ON_SKIP_TXFM_ALWAYS_ON	1	Value indicating CDEF is enabled on skipped transform blocks
CDEF_ON_SKIP_TXFM_DISABLED	0	Value indicating CDEF is disabled on skipped transform blocks
CDEF_STRENGTH_INDEX0_CTX	4	Number of contexts for cdef_index0
CFL_ALPHABET_SIZE	8	Number of values for cfl_alpha_u and cfl_alpha_v
CFL_ALPHA_CONTEXTS	6	Number of contexts for cfl_alpha_u and cfl_alpha_v

CFL_CONTEXTS	3	Number of contexts for is_cfl
CFL_JOINT_SIGNS	8	Number of values for cfl_alpha_signs
CHROMA_MODE_COUNT	8	Number of values for uv_mode
COEFF_BASE_PH_CONTEXTS	5	Number of contexts for coeff_base when the parity is hidden
COEFF_BASE_RANGE	3	Number of values for coeff_br (coeff_br extends the range of coeff_base)
COEFF_CDF_Q_CTXS	4	Number of selectable context types for the coeffs() syntax structure
COMPOUND_MODES	7	Number of values for compound_mode
COMPOUND_MODE_CONTEXTS	5	Number of contexts for compound_mode
COMPOUND_TYPES	2	Number of values for compound_type
COMP_GROUP_IDX_CONTEXTS	12	Number of contexts for comp_group_idx
COMP_INTER_CONTEXTS	5	Number of contexts for comp_mode
CWP_EQUAL	8	Value for CwpIdx that corresponds to equal weighting for two inter references
DBL_REG_DECIS_LEN	9	Length of Q_First array
DCT_ADST	2	Inverse transform rows with ADST and columns with DCT
DCT_DCT	0	Inverse transform rows with DCT and columns with DCT
DCT_FLIPADST	5	Inverse transform rows with FLIPADST and columns with DCT
DC_SIGN_CONTEXTS	3	Number of contexts for dc_sign
DC_SIGN_GROUPS	2	Number of groups of contexts for dc_sign (corresponding to whether the sign is hidden or not)
DECAY_DIST_CAP	6	Maximum distance that can use array Dist_Score_Lookup
DELTAWARP	3	Use delta warp motion compensation
DELTA_DCQUANT_BITS	5	Number of bits for base_y_dc_delta_q, base_uv_dc_delta_q, and base_uv_ac_delta_q
DELTA_DCQUANT_MAX	$(1 \ll (\text{DELTA_DCQUANT_BITS} - 2))$	Maximum value for BaseYDcDeltaQ and BaseUVDcDeltaQ
DELTA_DCQUANT_MIN	$(\text{DELTA_DCQUANT_MAX} - (1 \ll \text{DELTA_DCQUANT_BITS}) + 1)$	Minimum value for BaseYDcDeltaQ and BaseUVDcDeltaQ
DELTA_Q_SMALL	7	Value indicating alternative encoding of quantizer index delta values
DF_DELTA_SCALE	8	Scale factor for DfDeltaQ
DF_SHIFT	8	Shift used in deblocking filter
DIP_CTXS	3	Number of contexts for use_dip
DIRECTIONAL_MODES_COUNT	56	Number of directional intra modes
DISPLAY_ORDER_HINT_BITS	30	Number of order hint bits
DIST_WEIGHT_BITS	6	Scaling used in scoring reference frames
DIV_LUT_BITS	7	Number of fractional bits for lookup in divisor lookup table

DIV_LUT_NUM	129	Number of entries in divisor lookup table
DIV_LUT_PREC_BITS	9	Number of fractional bits of entries in divisor lookup table
DIV_PREC_BITS	14	Number of bits used in <code>get_division_scale_shift</code>
DIV_PREC_BITS_POW2	8	Number of regions used in <code>get_division_scale_shift</code>
DIV_SLOT_BITS	3	Base 2 logarithm of regions used in <code>get_division_scale_shift</code>
DRL_MODE_CONTEXTS	5	Number of contexts for <code>drl_mode</code>
EC_PROB_SHIFT	7	Number of bits to reduce CDF precision during arithmetic coding
EOB_PLANE_CTXS	3	Number of contexts for EOB -related syntax elements
EXTENDWARP	4	Use extended warp motion compensation
EXT_PARTITION_TYPES	10	Number of partition types
EXT_TX_SIZES	4	Number of size classes (each size class has a different choice of transform types)
EXT_WARP_PHASES	64	Number of phases for extended warp filtering
EXT_WARP_PHASES_LOG2	6	Base 2 logarithm of number of phases for extended warp filtering
EXT_WARP_ROUND_BITS	WARPEMODEL_PREC_BITS - EXT_WARP_PHASES_LOG2	Difference between bits used for the warp model and bits needed to specify the phase for extended warp filtering
EXT_WARP_TAPS	6	Number of taps in extended warp filtering
FILTER_BITS	7	Number of bits used in Wiener filter coefficients
FIRST_MODE_COUNT	13	Number of values coded via the first intra mode set
FLIPADST_ADST	8	Inverse transform rows with ADST and columns with FLIPADST
FLIPADST_DCT	4	Inverse transform rows with DCT and columns with FLIPADST
FLIPADST_FLIPADST	6	Inverse transform rows with FLIPADST and columns with FLIPADST
FSC_BSIZE_CONTEXTS	6	Number of block size groups in context for <code>fsc_mode</code>
FSC_MAX	32	Max width/height for blocks to use forward skip coding
FSC_MODES	2	Number of values of <code>fsc_mode</code>
FSC_MODE_CONTEXTS	4	Number of contexts for <code>fsc_mode</code>
FSC_TX_SIZE_CONTEXTS	3	Number of transform size context groups for forward skip coding
GDF_DIAG0	2	GDF first diagonal direction
GDF_DIAG1	3	GDF second diagonal direction
GDF_HOR	1	GDF horizontal direction
GDF_MIN_SIZE	128	Minimum size of GDF blocks when <code>gdf_unit_matches_sb_size</code> is equal to 0

GDF_VER	0	GDF vertical direction
GLOBAL_XLAYER_ID	31	Value for xlayer_id that indicates global scope
GM_ABS_ALPHA_BITS	9	Number of bits encoded for non-translational components of global motion models
GM_ABS_TRANS_BITS	14	Number of bits encoded for translational components of global motion models, if part of a ROTZOOM or AFFINE model
GM_ALPHA_MAX	$(1 \ll \text{GM_ABS_ALPHA_BITS}) - 1$	Maximum non-translational value
GM_ALPHA_MIN	$-\text{GM_ALPHA_MAX}$	Minimum non-translational value
GM_ALPHA_PREC_BITS	10	Number of fractional bits for sending non-translational warp model coefficients
GM_ALPHA_PREC_DIFF	$\text{WARPEDMODEL_PREC_BITS} - \text{GM_ALPHA_PREC_BITS}$	Difference between warped model and non-translational precision
GM_TRANS_MAX	$(1 \ll \text{GM_ABS_TRANS_BITS}) - 1$	Maximum translational value
GM_TRANS_MIN	$-\text{GM_TRANS_MAX}$	Minimum translational value
GM_TRANS_ONLY_PREC_DIFF	$\text{WARPEDMODEL_PREC_BITS} - 3$	Difference between warped model and motion vector precision
GM_TRANS_PREC_BITS	3	Number of fractional bits for sending translational warp model coefficients
GM_TRANS_PREC_DIFF	$\text{WARPEDMODEL_PREC_BITS} - \text{GM_TRANS_PREC_BITS}$	Difference between warped model and translational precision
H_ADST	13	Inverse transform rows with ADST and columns with identity
H_DCT	11	Inverse transform rows with DCT and columns with identity
H_FLIPADST	15	Inverse transform rows with FLIPADST and columns with identity
H_WEDGE_ANGLES	10	Number of wedge angles when wedge_angle_dir is equal to 0
IBC_BUFFER_SIZE	64	Size of buffer used in local intra block copy
IBC_BUFFER_SIZE_LOG2	6	Base 2 logarithm of size of buffer used in local intra block copy
IBC_NUM_BUFFERS	4	Number of buffers used in local intra block copy
IBP_WEIGHT_MAX	128	Sum of weights used in IBP
IBP_WEIGHT_SHIFT	7	Scaling shift for IBP process
IBP_WEIGHT_SIZE	$1 \ll \text{IBP_WEIGHT_SIZE_LOG2}$	Size of weights used in IBP
IBP_WEIGHT_SIZE_LOG2	4	Base 2 logarithm of size of weights used in IBP
IDENTITY	0	Warp model is just an identity transform
IDTX	9	Inverse transform rows with identity and columns with identity
IDTX_LEVEL_CONTEXTS	7	Number of contexts per transform size group for coeff_br_idtx
IDTX_SIGN_CONTEXTS	9	Number of contexts per transform size group for idtx_sign
IDTX_SIG_COEF_CONTEXTS	7	Number of contexts per transform size group for coeff_base_idtx
INT32MAX	$(1 \ll 31) - 1$	

		Largest number representable with 32-bit signed integer
INT32MIN	$-(1 \ll 31)$	Smallest number representable with 32-bit signed integer
INTERINTRA	1	Use inter intra motion compensation
INTERINTRA_MODES	4	Number of inter intra modes
INTERP_FILTERS	3	Number of values for <code>interp_filter</code>
INTERP_FILTER_CONTEXTS	16	Number of contexts for <code>interp_filter</code>
INTER_SDP_BSIZE_GROUP	4	Number of contexts for <code>region_type</code>
INTER_SDP_MAX_BLOCK_SIZE	64	Maximum size for switching partitioning scheme
INTRABC_CONTEXTS	3	Number of contexts for <code>use_intrabc</code>
INTRABC_DELAY_PIXELS	256	Number of horizontal luma samples before intra block copy can be used
INTRABC_DELAY_SB64	4	Number of 64 by 64 blocks before intra block copy can be used
INTRA_EDGE_KERNELS	3	Number of filter kernels for the intra edge filter
INTRA_EDGE_TAPS	5	Number of kernel taps for the intra edge filter
INTRA_MODES	13	Number of values for <code>y_mode</code>
INTRA_MODE_SETS	4	Number of values for <code>y_mode_set</code>
INTRA_REGION	0	Value for <code>region_type</code> that indicates intra coding
INTRA_TX_TYPES	7	Number of values for <code>intra_tx_type</code>
IST_4X4_HEIGHT	8	Height of matrix used in 4x4 secondary transform
IST_4X4_WIDTH	16	Width of matrix used in 4x4 secondary transform
IST_8X8_HEIGHT	32	Height of matrix used in 8x8 secondary transform
IST_8X8_HEIGHT_RED	20	Reduced height of matrix used in special case of 8x8 secondary transform
IST_8X8_WIDTH	48	Width of matrix used in 8x8 secondary transform
IST_DIR_SIZE	7	Number of directional groups in secondary transform kernels
IST_REDUCE_SET_SIZE_ADST_ADST	4	Number of different sets of secondary transforms for ADST
IST_SET_SIZE_4X4	14	Number of different sets of 4x4 secondary transforms
IST_SET_SIZE_8X8	11	Number of different sets of 8x8 secondary transforms
IS_INTER_CONTEXTS	4	Number of contexts for <code>is_inter</code>
JOINT_AMVD_SCALE_FACTOR_CNT	3	Number of values for <code>jmvd_scale_mode</code> when <code>use_amvd</code> is equal to 1
JOINT_NEWMV_SCALE_FACTOR_CNT	5	Number of values for <code>jmvd_scale_mode</code> when <code>use_amvd</code> is equal to 0
LEAST_SQUARES_SAMPLES_MAX	8	Largest number of samples used when computing a local warp

LEVEL_CONTEXTS	7	Number of contexts for coeff_br for high frequency luma coefficients
LEVEL_CONTEXTS_UV	4	Number of contexts for coeff_br for high frequency chroma coefficients
LF_BASE_SYMBOLS	6	Number of values for coeff_base for low frequency coefficients
LF_LEVEL_CONTEXTS	14	Number of contexts for coeff_br for low frequency luma coefficients
LF_NUM_BASE_LEVELS	$\text{LF_BASE_SYMBOLS} - 2$	Base level threshold for low frequency coefficients for deciding to read coeff_br
LF_SIG_COEF_CONTEXTS	$\text{LF_SIG_COEF_CONTEXTS_2D} + \text{LF_SIG_COEF_CONTEXTS_1D}$	Number of contexts for coeff_base for low frequency luma coefficients
LF_SIG_COEF_CONTEXTS_1D	12	Number of contexts for 1d luma transform class
LF_SIG_COEF_CONTEXTS_1D_UV	4	Number of contexts for 1d chroma transform class
LF_SIG_COEF_CONTEXTS_2D	21	Number of contexts for 2d luma transform class
LF_SIG_COEF_CONTEXTS_2D_UV	8	Number of contexts for 2d chroma transform class
LF_SIG_COEF_CONTEXTS_UV	$\text{LF_SIG_COEF_CONTEXTS_2D_UV} + \text{LF_SIG_COEF_CONTEXTS_1D_UV}$	Number of contexts for coeff_base for low frequency chroma coefficients
LOCALWARP	2	Use local warp motion compensation
LR_BANK_SIZE	4	Size of coefficient cache used for loop restoration
LS_MV_MAX	256	Largest motion vector difference to include in local warp computation
MASK_MASTER_SIZE	128	Size of MasterMask array
MAXQ_8_BITS	255	Maximum quantizer when bit depth is 8
MAXQ_10_BITS	$\text{MAXQ_8_BITS} + 2 * \text{MAXQ_OFFSET}$	Maximum quantizer when bit depth is 10
MAXQ_BITS	$\text{MAXQ_8_BITS} + 4 * \text{MAXQ_OFFSET}$	Maximum quantizer irrespective of the bit depth
MAXQ_OFFSET	24	Increase in allowed quantizer for each increase in bit depth
MAX_AMVD_INDEX	8	Number of values for amvd_index
MAX_ANGLE_DELTA	3	Maximum magnitude of AngleDeltaY and AngleDeltaUV
MAX_ATLAS_COLS	64	Maximum number of Atlas region columns
MAX_ATLAS_ROWS	64	Maximum number of Atlas region rows
MAX_BASE_BR_RANGE	$\text{COEFF_BASE_RANGE} + \text{NUM_BASE_LEVELS} + 1$	The maximum value for coeff_base and coeff_br combined
MAX_COL_TRUNCATED_UNARY_VAL	2	Maximum times col_mv_greater can be coded per motion vector
MAX_CWP_NUM	5	Number of values for CwpIdx
MAX_DBL_FLT_LEN	8	Maximum distance from edge for samples used in the deblocking filter
MAX_DR_PR_NUM	2	Used to limit the number of derived motion vector pruning operations
MAX_DR_STACK_SIZE	4	

		Maximum number of motion vectors in the derived stack
MAX_FILM_GRAIN	8	Maximum number of film grain configurations
MAX_FRAME_DISTANCE	31	Maximum distance when computing weighted prediction
MAX_LR_FLEX_SWITCHABLE_BITS	3	Maximum number of loop restoration tools to switch between
MAX_LS_BITS	26	Maximum bits in least squares calculations
MAX_MFH_NUM	16	Maximum number of multi-frame headers
MAX_NUM_ATLAS_SEGMENTS	256	Maximum number of Atlas segments
MAX_NUM_MLAYERS	8	Maximum number of embedded layers
MAX_NUM_TLAYERS	4	Maximum number of temporal layers
MAX_PR_NUM	16	Used to limit the number of motion vector pruning operations
MAX_REF_BV_STACK_SIZE	4	Maximum number of motion vectors in the stack for intra block copy
MAX_REF_MV_STACK_SIZE	6	Maximum number of motion vectors in the stack
MAX_RMB_SB_HITS	64	Maximum number of accesses to the bank of motion vectors per superblock
MAX_SEGMENTS	16	Number of segments allowed in segmentation map
MAX_SEQ_NUM	16	Maximum number of sequence headers
MAX_SIDE_TABLE	296	Length of Side_Thresholds array
MAX_TILE_AREA	4096 * 2304	Maximum area of a tile in units of luma samples
MAX_TILE_COLS	64	Maximum number of tile columns
MAX_TILE_ROWS	64	Maximum number of tile rows
MAX_TILE_WIDTH	4096	Maximum width of a tile in units of luma samples
MAX_WARP_REF_CANDIDATES	4	Maximum number of warp reference candidates
MAX_WARP_SB_HITS	64	Maximum number of accesses to the warp parameter bank per superblock
MFMV_STACK_SIZE	4	Stack size for motion field motion vectors
MHCCP_BITS	16	Number of bits used in MHCCP
MIXED_REGION	1	Value for region_type that indicates mixed intra coding and inter coding
MI_SIZE	4	Smallest size of a mode info block in luma samples
MI_SIZE_LOG2	2	Base 2 logarithm of smallest size of a mode info block
MODE_INDEX_COUNT	8	Number of values for y_mode_index
MODE_OFFSET_COUNT	6	Number of values for y_mode_offset
MOTION_MODES	5	Number of values for motion modes
MRL_INDEX_CONTEXTS	3	Number of contexts for mrl_index
MV_BORDER	128	Value used when clipping motion vectors
MV_CONTEXTS	2	Number of contexts for decoding motion vectors including one for intra block copy

MV_INTRABC_CONTEXT	1	Motion vector context used for intra block copy
MV_IN_USE_BITS	16	Number of bits for motion vectors (not including sign bit)
MV_JOINTS	4	Number of values for mv_joint
MV_LOW	$-(1 \ll MV_IN_USE_BITS)$	Exclusive lower bound on motion vectors
MV_REFINE_PREC_BITS	4	Number of bits for motion vectors from optical flow
MV_UPP	$(1 \ll MV_IN_USE_BITS)$	Exclusive upper bound on motion vectors
NON_DIRECTIONAL_MODES_COUNT	5	Number of non-directional intra modes
NUM_BASE_LEVELS	2	Number of quantizer base levels
NUM_CTX_COL_MV_GTX	2	Number of contexts for col_mv_greater
NUM_CTX_COL_MV_INDEX	4	Number of contexts for col_mv_index
NUM_CUSTOM_QMS	15	Maximum number of quantization matrices that can be present
NUM_PARA_COMBINATIONS	125	Number of adaptation rates
NUM_PARA_INTERVALS	3	Number of time intervals for computing adaptation rates
NUM_PC_WIENER_FILTERS	64	Number of filters in pixel-classified Wiener filtering
NUM_PC_WIENER_LUT_CLASSES	256	Number of classes in pixel-classified Wiener filtering
NUM_RECT_PARTS	2	Number of types of rectangle
NUM_REF_FRAMES	16	Number of frames that can be stored for future reference
NUM_REF_SAM_CFL	8	Number of samples used in chroma from luma prediction
NUM_UNEVEN_4WAY_PARTS	2	Number of uneven partition types
NUM_WEDGE_DIST	4	Number of distances for the wedge mask process
OPFL_GRAD_UNIT	16	Size of unit used in gradient computation
OPFL_GRAD_UNIT_LOG2	4	Base 2 logarithm of size of unit used in gradient computation
OPFL_MV_DELTA_LIMIT	$1 \ll MV_REFINE_PREC_BITS$	Maximum adjustment for motion vectors from optical flow
PALETTE_COLORS	8	Number of values for palette_color
PALETTE_COLOR_CONTEXTS	5	Number of values for color contexts
PALETTE_MAX_COLOR_CONTEXT_HASH	8	Number of mappings between color context hash and color context
PALETTE_NUM_NEIGHBORS	3	Number of neighbors considered within palette computation
PALETTE_ROW_FLAG_CONTEXTS	4	Number of values for identity row contexts
PALETTE_SIZES	7	Number of values for palette_size
PARTITION_CONTEXTS	64	Number of contexts when decoding partition syntax elements
PARTITION_STRUCTURE_NUM	2	Maximum number of partitions for a block (luma and chroma can have different partitions)

PC_WIENER_COEFFS	13	Number of coefficients in pixel-classified Wiener filtering
PC_WIENER_LAG	4	Number of lagging taps in pixel-classified Wiener filtering
PC_WIENER_LEAD	1	Number of leading taps in pixel-classified Wiener filtering
PC_WIENER_NUM_FEATURES	4	Number of features for pixel-classified Wiener filtering
PC_WIENER_PREC_BITS	7	Bit precision for pixel-classified Wiener filtering
PC_WIENER_PREC_FEATURE	14	Bit precision for pixel-classified features
PC_WIENER_TAPS	$PC_WIENER_COEFFS * 2 - 1$	Number of taps in pixel-classified Wiener filtering
PHTHRESH	4	Number of non-zero coefficients that will allow the parity to be hidden
PLANE_TYPES	2	Number of different plane types (luma or chroma)
PRIMARY_REF_CHOOSE	8	Value of <code>primary_ref_frame</code> , indicating that the primary reference frame is chosen automatically from the available reference frames
PRIMARY_REF_NONE	7	Value of <code>primary_ref_frame</code> , indicating that there is no primary reference frame
QUANT_TABLE_BITS	3	Number of bits to discard from quantizer before application
RECT_HORZ	0	Block is split with a horizontal cut
RECT_INVALID	2	Block cannot be split into rectangles
RECT_VERT	1	Block is split with a vertical cut
REFINEMV_CONTEXTS	24	Number of contexts for <code>use_refinermv</code>
REFMVS_LIMIT	$(1 \ll 11) - 1$	Largest reference MV component that can be saved
REFS_PER_FRAME	7	Number of reference frames that can be used for inter prediction
REF_CONTEXTS	3	Number of contexts for <code>single_ref</code>
REF_MV_BANK_SIZE	4	Size of the parameter bank for motion vectors
REF_SCALE_SHIFT	14	Number of bits of precision when scaling reference frames
RESTORATION_TILESIZE_MAX	512	Maximum size of a loop restoration tile
RESTORE_SWITCHABLE_TYPES	RESTORE_SWITCHABLE	Number of switchable loop restoration types
RESTRICTED_OH	-1	Sentinel order hint to mark restricted reference frames
ROTZOOM	1	Warp model is a rotation + symmetric zoom + translation
SCALE_SUBPEL_BITS	10	Number of bits of precision when computing inter prediction locations
SECOND_MODE_COUNT	16	Number of values for <code>y_second_mode</code>
SEGMENT_ID_CONTEXTS	3	Number of contexts for <code>segment_id</code>
SEGMENT_ID_PREDICTED_CONTEXTS	3	Number of contexts for <code>segment_id_predicted</code>

SEG_LVL_ALT_Q	0	Index for quantizer segment feature
SEG_LVL_GLOBALMV	2	Index for global mv feature
SEG_LVL_MAX	3	Number of segment features
SEG_LVL_SKIP	1	Index for skip segment feature
SELECT_INTEGER_MV	2	Value that indicates the force_integer_mv syntax element is coded
SELECT_SCREEN_CONTENT_TOOLS	2	Value that indicates the allow_screen_content_tools syntax element is coded
SIG_COEF_CONTEXTS	20	Number of contexts for coeff_base for luma
SIG_COEF_CONTEXTS_BOB	3	Number of contexts for coeff_base_bob
SIG_COEF_CONTEXTS_EOB	4	Number of contexts for coeff_base_eob
SIG_COEF_CONTEXTS_UV	12	Number of contexts for coeff_base for chroma
SIG_REF_DIFF_OFFSET_NUM	5	Maximum number of context samples to be used in determining the context index for coeff_base and coeff_base_eob.
SIMPLE	0	Use translation or global motion compensation
SINGLE_MODE_CONTEXTS	5	Number of contexts for single_mode
SKIP_CONTEXTS	6	Number of contexts for decoding skip
SKIP_MODE_CONTEXTS	3	Number of contexts for decoding skip_mode
SQUARE_SPLIT_CONTEXTS	8	Number of contexts for do_square_split syntax element
STX_TYPES	4	Number of secondary transform types
SUBPEL_BITS	4	Number of bits of precision when choosing an inter prediction filter kernel
SUBPEL_MASK	15	$(1 \ll \text{SUBPEL_BITS}) - 1$
TIP_CONTEXTS	3	Number of contexts for tip_mode
TIP_MFMV_STACK_SIZE	3	Stack size for motion field motion vectors related to TIP
TOTAL_ANGLE_DELTA_COUNT	7	Number of different angle deltas
TXB_SKIP_CONTEXTS	10	Number of contexts for all_zero per group
TXFM_SPLIT_GROUP	9	Number of groups of transform split types
TX_CLASS_2D	0	Transform class for transform types performing non-identity transforms in both directions
TX_CLASS_HORIZ	1	Transform class for transforms performing only a horizontal non-identity transform
TX_CLASS_VERT	2	Transform class for transforms performing only a vertical non-identity transform
TX_PARTITION_TYPE_NUM	7	Number of contexts for tx_partition_type
TX_PARTITION_TYPE_NUM_VERT_AND_HORIZ	14	Number of values (not equal to BLOCK_INVALID) in the output range of Size_To_Tx_Type_Group_Vert_And_Horz
TX_PARTITION_TYPE_NUM_VERT_OR_HORIZ	3	Number of values (not equal to BLOCK_INVALID) in the output range of Size_To_Tx_Type_Group_Vert_Or_Horz
TX_SET_TYPES_INTER	9	Number of inter transform set types

TX_SET_TYPES_INTRA	7	Number of intra transform set types
TX_SIZES	5	Number of square transform sizes
TX_SIZES_ALL	25	Number of transform sizes (including non-square sizes)
TX_TYPES	16	Number of inverse transform types
UV_INTRA_MODES_CFL_ALLOWED	14	Number of values for uv_mode when chroma from luma is allowed
UV_INTRA_MODES_CFL_NOT_ALLOWED	13	Number of values for uv_mode when chroma from luma is not allowed
UV_MODE_CONTEXTS	2	Number of contexts for uv_mode
V_ADST	12	Inverse transform rows with identity and columns with ADST
V_DCT	10	Inverse transform rows with identity and columns with DCT
V_FLIPADST	14	Inverse transform rows with identity and columns with FLIPADST
V_TXB_SKIP_CONTEXTS	12	Number of contexts for all_zero for the V plane
WAIP_WH_RATIO_2_THRES	61	Threshold used in WAIP
WAIP_WH_RATIO_4_THRES	73	Threshold used in WAIP
WAIP_WH_RATIO_8_THRES	82	Threshold used in WAIP
WAIP_WH_RATIO_16_THRES	86	Threshold used in WAIP
WARPEDDIFF_PREC_BITS	10	Number of extra bits of precision in warped filtering
WARPEDMODEL_PREC_BITS	16	Internal precision of warped motion models
WARPEDMODEL_TRANS_CLAMP	1 << 27	Clamping value used for translation components of warp
WARPEDPIXEL_PREC_SHIFTS	1 << 6	Number of phases used in warped filtering
WARPMV_MODE_CONTEXT	5	Number of contexts when decoding is_warp
WARP_CAUSAL_MODE_CTX	4	Number of contexts when decoding use_local_warp
WARP_DELTA_NUM_SYMBOLS_HIGH	8	Number of values for warp_delta_param_high
WARP_DELTA_NUM_SYMBOLS_LOW	8	Number of values for warp_delta_param_low
WARP_DELTA_STEP_BITS	10	Shift to apply to warp_delta_param
WARP_PARAM_BANK_SIZE	4	Size of the parameter bank for warp
WARP_PARAM_REDUCE_BITS	6	Rounding bitwidth for the parameters to the shear process
WEDGE_ANGLES	20	Number of angles for the wedge mask process
WEDGE_BLD_LUT_SIZE	32	Size of table lookup in the wedge mask process
WEDGE_BOUNDARY_SHARP	0	Value indicating a sharp boundary
WEDGE_BOUNDARY_SMOOTH	1	Value indicating a smooth boundary
WEDGE_BOUNDARY_TYPES	2	Number of different boundary types
WEDGE_TYPES	68	Number of types of wedge
WIENER_COEFFS	3	Number of Wiener filter coefficients to read
WIENER_NS_CHROMA_COEFFS	18	

		Number of chroma non-separable Wiener filter coefficients
WIENER_NS_CLASSES	16	Number of classes of non-separable Wiener filter coefficients
WIENER_NS_LUMA_COEFFS	16	Number of luma non-separable Wiener filter coefficients
WIENER_NS_PLANES	3	Number of planes of non-separable Wiener filter coefficients
WIENER_NS_PREC_BITS	7	Number of bits used in non-separable Wiener filter coefficients
WIENER_NS_SHORT_COEFFS	6	Number of short non-separable Wiener filter coefficients
WIENER_NS_TAPS_UV	12	Number of chroma non-separable Wiener filter taps
WIENER_NS_TAPS_Y	32	Number of luma non-separable Wiener filter taps
Y_MODE_CONTEXTS	3	Number of contexts for y_mode_index and y_second_mode

[↑ Back to Table of Contents](#)

§ 4. Conventions

§ 4.1. General

The mathematical operators and their precedence rules used to describe this specification are similar to those used in the C programming language. However, the operation of integer division with truncation is specifically defined.

In addition, a length 2 array used to hold a [motion vector](#) (indicated by the variable name ending with the letters Mv or Mvs) can be accessed using either array notation (e.g., Mv[0] and Mv[1]), or by just the name (e.g., Mv). The only operations defined when using the name are assignment and equality/inequality testing. Assignment of an array is represented using the notation A = B and is specified to mean the same as doing both the individual assignments A[0] = B[0] and A[1] = B[1]. Equality testing of 2 motion vectors is represented using the notation A == B and is specified to mean the same as (A[0] == B[0] && A[1] == B[1]). Inequality testing is defined as A != B and is specified to mean the same as (A[0] != B[0] || A[1] != B[1]).

If a process specifies something happens for $x = L..H$, where x is a variable name and L and H are expressions, it means that the variable takes all integer values starting at L and going up to (and including) H .

When a variable is said to be representable by a signed integer with x bits, it means that the variable is greater than or equal to $-(1 \ll (x-1))$, and that the variable is less than or equal to $(1 \ll (x-1))-1$.

The key words “must”, “must not”, “required”, “shall”, “shall not”, “should”, “should not”, “recommended”, “may”, and “optional” in this document are to be interpreted as described in [\[RFC2119\]](#).

Informative notes begin with the word “Note” and are set apart from the normative text with class="note", like this:

NOTE: This is an informative note.

§ 4.2. Arithmetic operators

+	Addition
-	Subtraction (as a binary operator) or negation (as a unary prefix operator)
*	Multiplication
/	Integer division with truncation of the result toward zero (for example, 7/4 and -7/-4 are truncated to 1, and -7/4 and 7/-4 are truncated to -1)
a % b	Remainder from division of a by b, where both a and b are positive integers
÷	Floating point (arithmetical) division
ceil(x)	The smallest integer that is greater than or equal to x
floor(x)	The largest integer that is less than or equal to x

§ 4.3. Ternary operator

cond ? a : b	a if cond is true, b if cond is false
--------------	---------------------------------------

§ 4.4. Logical operators

a && b	Logical AND operation between a and b
a b	Logical OR operation between a and b
!	Logical NOT operation

§ 4.5. Relational operators

>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
==	Equal to
!=	Not equal to

§ 4.6. Bitwise operators

&	AND operation
	OR operation
^	XOR operation
~	Negation operation
a >> b	Shift a in 2's complement binary integer representation format to the right by b bit positions. This operator is only used with b being a non-negative integer. Bits shifted into the MSBs as a result of the right shift have a value equal to the MSB of a prior to the shift operation.
a << b	Shift a in 2's complement binary integer representation format to the left by b bit positions. This operator is only used with b being a non-negative integer. Bits shifted into the LSBs as a result of the left shift have a value equal to 0.

§ 4.7. Assignment

=	Assignment operator
++	Increment (for example, x++ is equivalent to x = x + 1). When this operator is used for an array index, the variable value is obtained before the auto increment operation.
--	Decrement (for example, x-- is equivalent to x = x - 1). When this operator is used for an array index, the variable value is obtained before the auto decrement operation.
+=	Addition assignment operator (for example, x += 3 corresponds to x = x + 3)
-=	Subtraction assignment operator (for example, x -= 3 corresponds to x = x - 3)

§ 4.8. Mathematical functions

The following mathematical functions (Abs, Clip3, Clip1, Min, Max, Round2 and Round2Signed) are defined as follows:

$$\text{Abs}(x) = \begin{cases} x; & x \geq 0 \\ -x; & x < 0 \end{cases} \quad (1)$$

$$\text{Clip1}(x) = \text{Clip3}(0, 2^{\text{BitDepth}} - 1, x) \quad (2)$$

$$\text{Clip3}(x, y, z) = \begin{cases} x; & z < x \\ y; & z > y \\ z; & \text{otherwise} \end{cases} \quad (3)$$

$$\text{Min}(x, y) = \begin{cases} x; & x \leq y \\ y; & x > y \end{cases} \quad (4)$$

$$\text{Max}(x, y) = \begin{cases} x; & x \geq y \\ y; & x < y \end{cases} \quad (5)$$

$$\text{Round2}(x, n) = \lfloor \frac{x + 2^{n-1}}{2^n} \rfloor \quad (6)$$

$$\text{Round2Signed}(x, n) = \begin{cases} \text{Round2}(x, n); & x \geq 0 \\ -\text{Round2}(-x, n); & x < 0 \end{cases} \quad (7)$$

The definition of Round2 uses standard mathematical power and division operations, not integer operations. An equivalent definition using integer operations is:

```
Round2( x, n ) {
    if ( n == 0 )
        return x
    return (x + (1 << (n - 1)) ) >> n
}
```

The FloorLog2(x) function is defined to be the floor of the base 2 logarithm of the input x.

The input x will always be an integer, and will always be greater than or equal to 1.

This function extracts the location of the most significant bit ([MSB](#)) in x.

An equivalent definition (using the pseudo-code notation introduced in the following section) is:

```
FloorLog2( x ) {
    s = 0
    while ( x != 0 ) {
        x = x >> 1
        s++
    }
    return s - 1
}
```

The GetMsb(x) function is the same as FloorLog2, except that an input of 0 is also allowed.

The function is defined as follows:

```
GetMsb( x ) {
    if ( x==0 ) {
        return 0
    }
}
```

```

    }
    return FloorLog2( x )
}

```

The `CeilLog2(x)` function is defined to be the ceiling of the base 2 logarithm of the input `x` (when `x` is 0, it is defined to return 0).

The input `x` will always be an integer, and will always be greater than or equal to 0.

This function extracts the number of bits needed to code a value in the range 0 to `x-1`.

An equivalent definition (using the pseudo-code notation introduced in the following section) is:

```

CeilLog2( x ) {
    if ( x < 2 )
        return 0
    i = 1
    p = 2
    while ( p < x ) {
        i++
        p = p << 1
    }
    return i
}

```

§ 4.9. Method of describing bitstream syntax

The description style of the syntax is similar to the C programming language. [Syntax elements](#) in the [bitstream](#) are represented in bold type. Each syntax element is described by its name (using only lower case letters with underscore characters) and a descriptor for its method of coded representation. The [decoding process](#) behaves according to the value of the syntax element and to the values of previously decoded syntax elements. When a value of a syntax element is used in the syntax tables or the text, it appears in regular (i.e., not bold) type. If the value of a syntax element is being computed (e.g., being written with a default value instead of being coded in the bitstream), it also appears in regular type (e.g., `tile_size_minus_1`).

In some cases the syntax tables may use the values of other variables derived from syntax element values. Such variables appear in the syntax tables, or text, named by a mixture of lower case and upper case letters and without any underscore characters. Variables starting with an upper case letter are derived for the decoding of the current syntax structure and all dependent syntax structures. These variables may be used in the decoding process for later syntax structures. Variables starting with a lower case letter are only used within the process from which they are derived. (Single-character variables are allowed.)

Constant values appear in all upper case letters with underscore characters (e.g., `MI_SIZE`).

Constant lookup tables appear as words (with the first letter of each word in upper case, and remaining letters in lower case) separated with underscore characters (e.g., `Block_Width[...]`).

Hexadecimal notation, indicated by prefixing the hexadecimal number by `0x`, may be used when the number of bits is an integer multiple of 4. For example, `0x1a` represents a [bit string](#) `0001 1010`.

Binary notation is indicated by prefixing the binary number by `0b`. For example, `0b00011010` represents a bit string `0001 1010`. Binary numbers may include underscore characters to enhance readability. If present,

the underscore characters appear every 4 binary digits starting from the [LSB](#). For example, 0b11010 may also be written as 0b1_1010.

A value equal to 0 represents a FALSE condition in a test statement. The TRUE condition is represented by any value not equal to 0.

The following table lists examples of the syntax specification format. When **syntax_element** appears (in bold font), it specifies that this syntax element is parsed from the bitstream.

syntax_structure_name(parameter1, parameter2, ...) {	Descriptor
<i>/* A statement can be a syntax element with an associated descriptor or can be an expression used to specify its existence, type, and value, as in the following examples. */</i>	
syntax_element	f(1)
<i>/* A group of statements enclosed in brackets is a compound statement and is treated functionally as a single statement. */</i>	
{	
statement	
...	
}	
<i>/* A "while" structure specifies that the statement is to be evaluated repeatedly while the condition remains true. */</i>	
while (condition)	
statement	
<i>/* A "do .. while" structure executes the statement once and then tests the condition. It repeatedly evaluates the statement while the condition remains true. */</i>	
do	
statement	
while (condition)	
<i>/* An "if .. else" structure tests the condition first. If it is true, the primary statement is evaluated. Otherwise, the alternative statement is evaluated. If the alternative statement is unnecessary to be evaluated, the "else" and corresponding alternative statement can be omitted. */</i>	
if (condition)	
primary statement	
else	
alternative statement	
<i>/* A "for" structure evaluates the initial statement at the beginning, then tests the condition. If it is true, the primary and subsequent statements are evaluated until the condition becomes false. */</i>	
for (initial statement; condition; subsequent statement)	
primary statement	
<i>/* The return statement in a syntax structure specifies that the parsing of the syntax structure will be terminated without processing any additional information after this stage. When a value immediately follows a return statement, this value shall also be returned as the output of this syntax structure. */</i>	
return x	
}	

§ 4.10. Functions

[Bitstream](#) functions used for syntax description are specified in this section.

Other functions are included in the syntax tables. The convention is that a section is called `_syntax_` if it causes [syntax elements](#) to be read from the bitstream, either directly or indirectly through subprocesses. The remaining sections are called `_functions_`.

The specification of these functions makes use of a bitstream position indicator. This bitstream position indicator locates the position of the bit that is going to be read next.

get_position(): Return the value of the bitstream position indicator.

init_symbol(sz): Initialize the arithmetic decode process for the symbol [decoder](#) with a size of `sz` [bytes](#) as specified in [§ 8.2.2 Initialization process for symbol decoder](#).

exit_symbol(): Exit the arithmetic decode process as described in [§ 8.2.4 Exit process for symbol decoder](#) (this includes reading trailing bits).

When referring to a function, brackets are included only when introducing a parameter which is needed for the explanation.

§ 4.11. Descriptors

§ 4.11.1. General

The following descriptors specify the parsing of [syntax elements](#). Lower case descriptors specify syntax elements that are represented by an integer number of bits in the [bitstream](#); upper case descriptors specify syntax elements that are represented by arithmetic coding.

§ 4.11.2. f(n)

Unsigned `n`-bit number appearing directly in the [bitstream](#). The bits are read from highest to lowest. The parsing process specified in [§ 8.1 Parsing process for f\(n\)](#) is invoked, and the [syntax element](#) is set equal to the return value.

§ 4.11.3. uvlc()

Variable-length unsigned number appearing directly in the [bitstream](#). The parsing process for this descriptor is specified below:

uvlc() {	Descriptor
leadingZeros = 0	
while (1) {	
done	f(1)
if (done)	
break	
leadingZeros++	
}	
if (leadingZeros >= 32) {	
return (1 << 32) - 1	

}	
value	f(leadingZeros)
return value + (1 << leadingZeros) - 1	
}	

It is a requirement of bitstream conformance that `leadingZeros` is less than 32 when this function returns.

NOTE: This means that the largest value that can be returned by a `uvlc()` descriptor is $(1 \ll 32) - 2$.

§ 4.11.4. svlc()

Variable-length signed number appearing directly in the [bitstream](#). The parsing process for this descriptor is specified below:

svlc() {	Descriptor
value	uvlc()
half = (value + 1) >> 1	
if (value & 1) {	
return half	
} else {	
return -half	
}	
}	

§ 4.11.5. le(n)

Unsigned little-endian *n*-[byte](#) number appearing directly in the [bitstream](#). The parsing process for this descriptor is specified below:

le(n) {	Descriptor
t = 0	
for (i = 0; i < n; i++) {	
byte	f(8)
t += (byte << (i * 8))	
}	
return t	
}	

§ 4.11.6. leb128()

Unsigned integer represented by a variable number of little-endian [bytes](#).

NOTE: This [syntax element](#) will only be present when the [bitstream](#) position is byte aligned.

In this encoding, the most significant bit of each byte is equal to 1 to signal that more bytes should be read, or equal to 0 to signal the end of the encoding.

A variable `Leb128Bytes` is set equal to the number of bytes read during this process.

The parsing process for this descriptor is specified below:

<code>leb128() {</code>	Descriptor
<code>value = 0</code>	
<code>Leb128Bytes = 0</code>	
<code>for (i = 0; i < 8; i++) {</code>	
<code> leb128_byte</code>	<code>f(8)</code>
<code> value = ((leb128_byte & 0x7f) << (i*7))</code>	
<code> Leb128Bytes += 1</code>	
<code> if (!(leb128_byte & 0x80)) {</code>	
<code> break</code>	
<code> }</code>	
<code>}</code>	
<code>return value</code>	
<code>}</code>	

It is a requirement of bitstream conformance that the value returned from the `leb128` parsing process is less than or equal to $(1 \ll 32) - 1$.

`leb128_byte` contains 8 bits read from the bitstream. The bottom 7 bits are used to compute the variable `value`. The most significant bit is used to indicate that there are more bytes to be read.

It is a requirement of bitstream conformance that the most significant bit of `leb128_byte` is equal to 0 if `i` is equal to 7. (This ensures that this syntax descriptor never uses more than 8 bytes.)

NOTE: There are multiple ways of encoding the same value, depending on how many leading zero bits are encoded. There is no requirement that this syntax descriptor uses the most compressed representation. This can be useful for [encoder](#) implementations by allowing a fixed amount of space to be filled in later when the value becomes known.

NOTE: Only 5 bytes (providing 35 bits) are needed for this syntax descriptor because the bitstream conformance requirement limits the return value to 32 bits (7 bits in each of the first 4 bytes, and 4 bits in the 5th byte).

§ 4.11.7. `su(n)`

Signed integer converted from an `n`-bit unsigned integer in the [bitstream](#). (The unsigned integer corresponds to the bottom `n` bits of the signed integer.) The parsing process for this descriptor is specified below:

<code>su(n) {</code>	Descriptor
<code>value</code>	<code>f(n)</code>

signMask = 1 << (n - 1)	
if (value & signMask)	
value = value - 2 * signMask	
return value	
}	

§ 4.11.8. ns(n)

Unsigned encoded integer with maximum number of values n (i.e., output in range $0..n-1$).

This descriptor is similar to $f(\text{CeilLog2}(n))$, but reduces wastage incurred when encoding non-power of two value ranges by encoding 1 fewer bit for the lower part of the value range. For example, when n is equal to 5, the encodings are as follows (full binary encodings are also presented for comparison):

Table 4.1: Example encodings for ns(5)

Value	Full binary encoding	ns(n) encoding
0	000	00
1	001	01
2	010	10
3	011	110
4	100	111

The parsing process for this descriptor is specified as:

ns(n) {	Descriptor
w = FloorLog2(n) + 1	
m = (1 << w) - n	
v	$f(w - 1)$
if (v < m)	
return v	
extra_bit	$f(1)$
return (v << 1) - m + extra_bit	
}	

The abbreviation `ns` stands for `_non-symmetric_`. This encoding is non-symmetric because the values are not all coded with the same number of bits.

§ 4.11.9. tu(mx)

Integer in the range 0 to mx using truncated unary encoding (a series of zero or more 1s followed by a single 0, except that the final 0 is omitted if the maximum is reached).

The parsing process for this descriptor is specified below:

tu(mx) {	Descriptor
for (idx = 0; idx < mx; idx++) {	

tu_bit	f(1)
if (tu_bit == 0) {	
return idx	
}	
}	
return mx	
}	

§ 4.11.10. rg(n)

Integer with Rice-Golomb coding with parameter n (a fixed length coding of the n least significant bits preceded by a unary encoding of the most significant bits).

The parsing process for this descriptor is specified below:

rg(n) {	Descriptor
for (q = 0; q < 32; q++) {	
rg_bit	f(1)
if (rg_bit == 0) {	
remainder	f(n)
return (q << n) + remainder	
}	
}	
return -1	
}	

It is a requirement of bitstream conformance that this descriptor never returns a value less than 0.

§ 4.11.11. L(n)

Unsigned arithmetic encoded n-bit number encoded as n flags (a `_literal_`). The flags are read from highest to lowest. The [syntax element](#) is set equal to the return value of `read_literal( n )` (see [§ 8.2.5 Parsing process for read_literal](#) for a specification of this process).

§ 4.11.12. S()

An arithmetic encoded symbol coded from a small alphabet of at most 8 entries.

The symbol is decoded based on a context-sensitive [CDF](#) (see [§ 8.3 Parsing process for CDF encoded syntax elements](#) for the specification of this process).

§ 4.11.13. NS(n)

Unsigned arithmetic encoded integer with maximum number of values n (i.e., output in range 0..n-1).

This descriptor is the same as `ns(n)`, except the underlying bits are coded arithmetically.

The parsing process for this descriptor is specified as:

NS(n) {	Descriptor
w = FloorLog2(n) + 1	
m = (1 << w) - n	
v	L(w - 1)
if (v < m)	
return v	
extra_bit	L(1)
return (v << 1) - m + extra_bit	
}	

[↑ Back to Table of Contents](#)

§ 5. Syntax structures

§ 5.1. General

This section presents the syntax structures in a tabular form. The meaning of each of the syntax elements is presented in [§6 Syntax structures semantics](#).

§ 5.2. OBU syntax

§ 5.2.1. General OBU syntax

open_bitstream_unit(sz) {	Descriptor
obu_header()	
obuPayloadSize = sz - 1 - obu_header_extension_flag	
startPosition = get_position()	
load_xlayer_context(obu_xlayer_id)	
if (obu_type == OBU_SEQUENCE_HEADER) {	
sequence_header_obu()	
} else if (obu_type == OBU_TEMPORAL_DELIMITER) {	
FirstPictureInTU = 1	
temporal_delimiter_obu()	
} else if (obu_type == OBU_MSDO) {	
multistream_decoder_operation_obu()	
} else if (obu_type == OBU_MULTI_FRAME_HEADER) {	
multi_frame_header_obu()	
} else if (is_sef() is_tip_frame() obu_type == OBU_BRIDGE_FRAME) {	
frame_header(1)	
} else if (obu_type == OBU_METADATA_SHORT) {	
metadata_short_obu(obuPayloadSize)	
} else if (obu_type == OBU_METADATA_GROUP) {	
metadata_group_obu()	
} else if (is_tile_group()) {	
tile_group_obu(obuPayloadSize)	
} else if (obu_type == OBU_LAYER_CONFIGURATION_RECORD) {	
layer_config_record_obu()	
} else if (obu_type == OBU_ATLAS_SEGMENT) {	
atlas_segment_info_obu()	
} else if (obu_type == OBU_OPERATING_POINT_SET) {	
operating_point_set_obu()	
} else if (obu_type == OBU_BUFFER_REMOVAL_TIMING) {	
buffer_removal_timing_obu()	
} else if (obu_type == OBU_QUANTIZATION_MATRIX) {	
quantizer_matrix_obu()	

} else if (obu_type == OBU_FILM_GRAIN) {	
film_grain_obu()	
} else if (obu_type == OBU_CONTENT_INTERPRETATION) {	
content_interpretation_obu()	
} else if (obu_type == OBU_PADDING) {	
padding_obu()	
} else {	
reserved_obu()	
}	
usedArith = is_tile_group()	
currentPosition = get_position()	
parsedPayloadBits = currentPosition - startPosition	
remainingPayloadBits = obuPayloadSize * 8 - parsedPayloadBits	
if (obuPayloadSize > 0 && !usedArith) {	
if (is_extensible_obu()) {	
<i>// OBUs with extensible payloads</i>	
obu_extension_flag	f(1)
if (obu_extension_flag) {	
obu_extension_data(remainingPayloadBits - 1)	
} else {	
trailing_bits(remainingPayloadBits - 1)	
}	
} else {	
trailing_bits(remainingPayloadBits)	
}	
}	
save_xlayer_context(obu_xlayer_id)	
}	

where some helper functions used to identify collections of OBU types are specified as:

```
is_tip_frame() {
  return obu_type == OBU_LEADING_TIP || obu_type == OBU_REGULAR_TIP
}
```

```
is_sef() {
  return obu_type == OBU_LEADING_SEF || obu_type == OBU_REGULAR_SEF
}
```

```
is_tile_group() {
  return obu_type == OBU_LEADING_TILE_GROUP ||
    obu_type == OBU_REGULAR_TILE_GROUP ||
    obu_type == OBU_CLOSED_LOOP_KEY ||
    obu_type == OBU_OPEN_LOOP_KEY ||

```

```

        obu_type == OBU_SWITCH ||
        obu_type == OBU_RAS_FRAME
    }

```

```

is_extensible_obu() {
    return obu_type == OBU_SEQUENCE_HEADER ||
        obu_type == OBU_MULTI_FRAME_HEADER ||
        obu_type == OBU_LAYER_CONFIGURATION_RECORD ||
        obu_type == OBU_CONTENT_INTERPRETATION ||
        obu_type == OBU_OPERATING_POINT_SET ||
        obu_type == OBU_ATLAS_SEGMENT
}

```

obu_extension_data(sz) {	Descriptor
for (i = 0; i < sz; i++) {	
obu_extension_data_bit	f(1)
}	
}	

§ 5.2.2. OBU header syntax

obu_header() {	Descriptor
obu_header_extension_flag	f(1)
obu_type	f(5)
obu_tlayer_id	f(2)
if (obu_header_extension_flag == 1) {	
obu_mlayer_id	f(3)
obu_xlayer_id	f(5)
} else {	
obu_mlayer_id = 0	
obu_xlayer_id = (obu_type == OBU_MSDO obu_type == OBU_TEMPORAL_DELIMITER) ? GLOBAL_XLAYER_ID : 0	
}	
}	

§ 5.2.3. Trailing bits syntax

trailing_bits(nbBits) {	Descriptor
trailing_one_bit	f(1)
nbBits--	
while (nbBits > 0) {	
trailing_zero_bit	f(1)
nbBits--	
}	
}	

§ 5.2.4. Byte alignment syntax

byte_alignment() {	Descriptor
while (get_position() & 7) {	
zero_bit	f(1)
}	
}	

§ 5.3. Reserved OBU syntax

reserved_obu() {	Descriptor
}	

NOTE: Reserved OBUs do not have a defined syntax. The obu_type reserved values are [reserved](#) for future use by AOMedia. Decoders should ignore the entire [OBU](#) if they do not understand the obu_type. The last [byte](#) of the valid content of the payload data for this [OBU](#) type is considered to be the last [byte](#) that is not equal to zero. This rule is to prevent the dropping of valid bytes by systems that interpret trailing zero bytes as a continuation of the trailing bits in an [OBU](#). This implies that when any payload data is present for this [OBU](#) type, at least one byte of the payload data (including the trailing bit) shall not be equal to 0.

§ 5.4. Sequence header OBU syntax

§ 5.4.1. General sequence header OBU syntax

sequence_header_obu() {	Descriptor
seq_header_id	uvlc()
seq_profile_idc	f(5)
single_picture_header_flag	f(1)
seq_level_idx	f(5)
if (seq_level_idx > 3 && !single_picture_header_flag) {	
seq_tier	f(1)
} else {	
seq_tier = 0	
}	
chroma_format_idc	uvlc()
bit_depth_idc	uvlc()
set_chroma_format_and_bit_depth()	
if (single_picture_header_flag) {	
seq_lcr_id = 0	
still_picture = 1	
max_tlayer_id = 0	
max_mlayer_id = 0	
SeqMaxMlayerCnt = 1	

monotonic_output_order_flag = 1	
} else {	
seq_lcr_id	f(3)
still_picture	f(1)
max_tlayer_id	f(2)
max_mlayer_id	f(3)
if (max_mlayer_id > 0) {	
n = CeilLog2(max_mlayer_id + 1)	
seq_max_mlayer_cnt_minus_1	f(n)
SeqMaxMlayerCnt = seq_max_mlayer_cnt_minus_1 + 1	
} else {	
SeqMaxMlayerCnt = 1	
}	
monotonic_output_order_flag	f(1)
}	
frame_width_bits_minus_1	f(4)
frame_height_bits_minus_1	f(4)
n = frame_width_bits_minus_1 + 1	
max_frame_width_minus_1	f(n)
n = frame_height_bits_minus_1 + 1	
max_frame_height_minus_1	f(n)
seq_cropping_window_present_flag	f(1)
if (seq_cropping_window_present_flag) {	
seq_cropping_win_left_offset	uvlc()
seq_cropping_win_right_offset	uvlc()
seq_cropping_win_top_offset	uvlc()
seq_cropping_win_bottom_offset	uvlc()
} else {	
seq_cropping_win_left_offset = 0	
seq_cropping_win_right_offset = 0	
seq_cropping_win_top_offset = 0	
seq_cropping_win_bottom_offset = 0	
}	
if (single_picture_header_flag) {	
decoder_model_info_present_flag = 0	
} else {	
seq_initial_display_delay_present_flag	f(1)
if (seq_initial_display_delay_present_flag) {	
seq_initial_display_delay_minus_1	f(4)
}	
decoder_model_info_present_flag	f(1)

if (decoder_model_info_present_flag) {	
num_units_in_decoding_tick	f(32)
seq_decoder_model_info_present_flag	f(1)
if (seq_decoder_model_info_present_flag) {	
seq_decoder_model_info()	
}	
}	
for (mLayer = 0; mLayer < MAX_NUM_MLAYERS; mLayer++) {	
for (currTLayer = 0; currTLayer < MAX_NUM_TLAYERS; currTLayer++) {	
for (refTLayer = 0; refTLayer < MAX_NUM_TLAYERS; refTLayer++) {	
TLayerDependencyMap[mLayer][currTLayer][refTLayer] =	
refTLayer <= currTLayer && currTLayer <= max_tlayer_id && mLayer <= max_mlayer_id	
}	
}	
}	
for (currLayer = 0; currLayer < MAX_NUM_MLAYERS; currLayer++) {	
for (refLayer = 0; refLayer < MAX_NUM_MLAYERS; refLayer++) {	
MLayerDependencyMap[currLayer][refLayer] =	
refLayer <= currLayer && currLayer <= max_mlayer_id	
}	
}	
if (max_mlayer_id > 0) {	
mlayer_dependency_present_flag	f(1)
if (mlayer_dependency_present_flag) {	
for (currLayer = 1; currLayer <= max_mlayer_id; currLayer++) {	
for (refLayer = currLayer; refLayer >= 0; refLayer--) {	
mlayer_dependency_map	f(1)
MLayerDependencyMap[currLayer][refLayer] =	
mlayer_dependency_map	
}	
}	
}	
}	
if (max_tlayer_id > 0) {	
tlayer_dependency_present_flag	f(1)
if (tlayer_dependency_present_flag) {	
if (max_mlayer_id > 0)	
multi_tlayer_dependency_map_present_flag	f(1)
else	
multi_tlayer_dependency_map_present_flag = 0	

for (mLayer = 0; mLayer <= max_mlayer_id; mLayer++) {	
for (currTLayer = 1; currTLayer <= max_tlayer_id; currTLayer++) {	
for (refTLayer = currTLayer; refTLayer >= 0; refTLayer--) {	
if (multi_tlayer_dependency_map_present_flag > 0	
mLayer == 0) {	
tlayer_dependency_map	f(1)
TLayerDependencyMap[mLayer][currTLayer][refTLayer] =	
tlayer_dependency_map	
} else {	
TLayerDependencyMap[mLayer][currTLayer][refTLayer] =	
TLayerDependencyMap[0][currTLayer][refTLayer]	
}	
}	
}	
}	
}	
for (mlayerId = 0; mlayerId < MAX_NUM_MLAYERS; mlayerId++) {	
for (refMlayer = 0; refMlayer < MAX_NUM_MLAYERS; refMlayer++) {	
MlayerPresenceMap[mlayerId][refMlayer] = 0	
if (mlayerId == refMlayer	
MlayerDependencyMap[mlayerId][refMlayer]) {	
MlayerPresenceMap[mlayerId][refMlayer] = 1	
for (depMlayerId = 0; depMlayerId < refMlayer; depMlayerId++) {	
MlayerPresenceMap[mlayerId][depMlayerId] =	
MlayerPresenceMap[refMlayer][depMlayerId]	
}	
}	
}	
sequence_partition_config()	
sequence_segment_config()	
sequence_intra_config()	
sequence_inter_config()	
sequence_scc_config()	
sequence_transform_quant_entropy_config()	
sequence_filter_config()	
sequence_tile_config()	
film_grain_params_present	f(1)
save_sequence_header()	
}	

§ 5.4.2. Sequence tile config syntax

sequence_tile_config() {	Descriptor
seq_tile_info_present_flag	f(1)
if (seq_tile_info_present_flag) {	
allow_tile_info_change	f(1)
seqSbSize = get_seq_sb_size()	
(SeqSbRowStarts, SeqSbRows, SeqTileRows, SeqTileRowsLog2,	
SeqSbColStarts, SeqSbCols, SeqTileCols, SeqTileColsLog2,	
SeqUniformTileSpacingFlag, sbShift) = tile_params(	
max_frame_width_minus_1 + 1, max_frame_height_minus_1 + 1,	
seqSbSize, seqSbSize, 0)	
}	
}	

§ 5.4.3. Sequence partition config syntax

sequence_partition_config() {	Descriptor
use_256x256_superblock	f(1)
if (!use_256x256_superblock) {	
use_128x128_superblock	f(1)
}	
if (Monochrome) {	
enable_sdp = 0	
} else {	
enable_sdp	f(1)
}	
if (enable_sdp && !single_picture_header_flag) {	
enable_extended_sdp	f(1)
} else {	
enable_extended_sdp = 0	
}	
enable_ext_partitions	f(1)
if (enable_ext_partitions) {	
enable_uneven_4way_partitions	f(1)
} else {	
enable_uneven_4way_partitions = 0	
}	
reduce_pb_aspect_ratio	f(1)
if (reduce_pb_aspect_ratio) {	
max_pb_aspect_ratio_log2_minus_1	f(1)
MaxPbAspectRatio = 1 << (max_pb_aspect_ratio_log2_minus_1 + 1)	
} else {	

MaxPbAspectRatio = 8	
}	
}	

§ 5.4.4. Sequence segment config syntax

sequence_segment_config() {	Descriptor
enable_ext_seg	f(1)
MaxSegments = enable_ext_seg ? 16 : 8	
seq_seg_info_present_flag	f(1)
if (seq_seg_info_present_flag) {	
seq_allow_seg_info_change	f(1)
(SeqFeatureEnabled, SeqFeatureData) = seg_info(MaxSegments)	
}	
}	

§ 5.4.5. Sequence intra config syntax

sequence_intra_config() {	Descriptor
enable_dip	f(1)
enable_intra_edge_filter	f(1)
enable_mrls	f(1)
enable_cfl_intra	f(1)
if (Monochrome) {	
cfl_ds_filter_index = 0	
} else {	
cfl_ds_filter_index	f(2)
}	
enable_mhccp	f(1)
enable_ibp	f(1)
}	

§ 5.4.6. Sequence inter config syntax

sequence_inter_config() {	Descriptor
if (single_picture_header_flag) {	
for (i = 0; i < MOTION_MODES; i++) {	
seq_enabled_motion_modes[i] = 0	
}	
enable_six_param_warp_delta = 0	
enable_masked_compound = 0	
enable_ref_frame_mvs = 0	
reduced_ref_frame_mvs_mode = 0	
OrderHintBits = 0	

enable_opfl_refine = REFINE_NONE	
enable_refmvpbank	f(1)
disable_drl_reorder	f(1)
if (disable_drl_reorder) {	
DrlReorder = DRL_REORDER_DISABLED	
} else {	
constrain_drl_reorder	f(1)
DrlReorder = constrain_drl_reorder ?	
DRL_REORDER_CONSTRAINT : DRL_REORDER_ALWAYS	
}	
n = MAX_REF_BV_STACK_SIZE - 1	
seq_max_bvp_drl_bits_minus_1	ns(n)
allow_frame_max_bvp_drl_bits	f(1)
enable_bawp	f(1)
enable_mv_traj = 0	
enable_imp_msk_bld = 0	
NumRefFrames = 2	
long_term_frame_id_bits = 0	
} else {	
motionModeEnabled = 0	
for (mode = INTERINTRA; mode < MOTION_MODES; mode++) {	
seq_enabled_motion_modes[mode]	f(1)
motionModeEnabled = seq_enabled_motion_modes[mode]	
}	
if (motionModeEnabled) {	
seq_frame_motion_modes_present_flag	f(1)
} else {	
seq_frame_motion_modes_present_flag = 0	
}	
if (seq_enabled_motion_modes[DELTAWARP]) {	
enable_six_param_warp_delta	f(1)
} else {	
enable_six_param_warp_delta = 0	
}	
enable_masked_compound	f(1)
enable_ref_frame_mvs	f(1)
if (enable_ref_frame_mvs) {	
reduced_ref_frame_mvs_mode	f(1)
} else {	
reduced_ref_frame_mvs_mode = 0	
}	
order_hint_bits_minus_1	

	f(4)
OrderHintBits = order_hint_bits_minus_1 + 1	
enable_refmvbank	f(1)
disable_drl_reorder	f(1)
if (disable_drl_reorder) {	
DrlReorder = DRL_REORDER_DISABLED	
} else {	
constrain_drl_reorder	f(1)
DrlReorder = constrain_drl_reorder ? DRL_REORDER_CONSTRAINT :	
DRL_REORDER_ALWAYS	
}	
explicit_ref_frame_map	f(1)
explicit_num_ref_frames	f(1)
if (explicit_num_ref_frames) {	
num_ref_frames_minus_1	f(4)
NumRefFrames = num_ref_frames_minus_1 + 1	
} else {	
NumRefFrames = 8	
}	
ActiveNumRefFrames = Min(REFS_PER_FRAME, NumRefFrames)	
long_term_frame_id_bits	f(3)
n = MAX_REF_MV_STACK_SIZE - 1	
seq_max_drl_bits_minus_1	ns(n)
allow_frame_max_drl_bits	f(1)
n = MAX_REF_BV_STACK_SIZE - 1	
seq_max_bvp_drl_bits_minus_1	ns(n)
allow_frame_max_bvp_drl_bits	f(1)
num_same_ref_compound	f(2)
enable_tip	f(1)
if (enable_tip) {	
disable_tip_output	f(1)
EnableTipOutput = !disable_tip_output	
enable_tip_hole_fill	f(1)
} else {	
enable_tip_hole_fill = 0	
EnableTipOutput = 0	
}	
enable_mv_traj	f(1)
enable_bawp	f(1)
enable_cwp	f(1)
enable_imp_msk_bld	f(1)
enable_df_sub_pu	f(1)

if (EnableTipOutput && enable_df_sub_pu) {	
enable_tip_explicit_qp	f(1)
} else {	
enable_tip_explicit_qp = 0	
}	
enable_opfl_refine	f(2)
enable_refinemv	f(1)
if (enable_tip && (enable_opfl_refine != 0 enable_refinemv)) {	
enable_tip_refinemv	f(1)
} else {	
enable_tip_refinemv = 0	
}	
enable_bru	f(1)
enable_adaptive_mvd	f(1)
enable_mvd_sign_derive	f(1)
enable_flex_mvres	f(1)
if (single_picture_header_flag) {	
enable_global_motion = 0	
} else {	
enable_global_motion	f(1)
}	
enable_short_refresh_frame_flags	f(1)
}	
}	

§ 5.4.7. Sequence screen content config syntax

sequence_scc_config() {	Descriptor
if (single_picture_header_flag) {	
seq_force_screen_content_tools = SELECT_SCREEN_CONTENT_TOOLS	
seq_force_integer_mv = SELECT_INTEGER_MV	
} else {	
seq_choose_screen_content_tools	f(1)
if (seq_choose_screen_content_tools) {	
seq_force_screen_content_tools = SELECT_SCREEN_CONTENT_TOOLS	
} else {	
seq_force_screen_content_tools	f(1)
}	
if (seq_force_screen_content_tools > 0) {	
seq_choose_integer_mv	f(1)
if (seq_choose_integer_mv) {	
seq_force_integer_mv = SELECT_INTEGER_MV	

} else {	
seq_force_integer_mv	f(1)
}	
} else {	
seq_force_integer_mv = SELECT_INTEGER_MV	
}	
}	
}	

§ 5.4.8. Sequence transform quant entropy config syntax

sequence_transform_quant_entropy_config() {	Descriptor
enable_fsc	f(1)
if (enable_fsc) {	
enable_idtx_intra = 1	
} else {	
enable_idtx_intra	f(1)
}	
enable_intra_ist	f(1)
enable_inter_ist	f(1)
if (Monochrome) {	
enable_chroma_dctonly = 0	
} else {	
enable_chroma_dctonly	f(1)
}	
if (!single_picture_header_flag) {	
enable_inter_ddt	f(1)
}	
reduced_tx_part_set	f(1)
if (Monochrome) {	
enable_cctx = 0	
} else {	
enable_cctx	f(1)
}	
enable_tcq	f(1)
if (enable_tcq && !single_picture_header_flag) {	
choose_tcq_per_frame	f(1)
} else {	
choose_tcq_per_frame = 0	
}	
if (enable_tcq && !choose_tcq_per_frame) {	
enable_parity_hiding = 0	

} else {	
enable_parity_hiding	f(1)
}	
if (single_picture_header_flag) {	
enable_avg_cdf = 1	
avg_cdf_type = 1	
} else {	
enable_avg_cdf	f(1)
if (enable_avg_cdf) {	
avg_cdf_type	f(1)
}	
}	
if (Monochrome) {	
separate_uv_delta_q = 0	
} else {	
separate_uv_delta_q	f(1)
}	
BaseYDcDeltaQ = 0	
BaseUVDcDeltaQ = 0	
BaseUVAcDeltaQ = 0	
y_dc_delta_q_enabled = 0	
uv_dc_delta_q_enabled = 0	
uv_ac_delta_q_enabled = 0	
equal_ac_dc_q	f(1)
if (!equal_ac_dc_q) {	
base_y_dc_delta_q	f(5)
BaseYDcDeltaQ = DELTA_DCQUANT_MIN + base_y_dc_delta_q	
y_dc_delta_q_enabled	f(1)
}	
if (!Monochrome) {	
if (!equal_ac_dc_q) {	
base_uv_dc_delta_q	f(5)
BaseUVDcDeltaQ = DELTA_DCQUANT_MIN + base_uv_dc_delta_q	
uv_dc_delta_q_enabled	f(1)
}	
base_uv_ac_delta_q	f(5)
BaseUVAcDeltaQ = DELTA_DCQUANT_MIN + base_uv_ac_delta_q	
uv_ac_delta_q_enabled	f(1)
if (equal_ac_dc_q) {	
BaseUVDcDeltaQ = BaseUVAcDeltaQ	
}	

}	
}	

§ 5.4.9. Segment information syntax

seg_info(numSegments) {	Descriptor
for (i = 0; i < numSegments; i++) {	
for (j = 0; j < SEG_LVL_MAX; j++) {	
feature_enabled	f(1)
enabled[i][j] = feature_enabled	
clippedValue = 0	
if (feature_enabled == 1) {	
bitsToRead = Segmentation_Feature_Bits[j]	
limit = Segmentation_Feature_Max[j]	
if (Segmentation_Feature_Signed[j] == 1) {	
n = 1 + bitsToRead	
feature_value	su(n)
clippedValue = Clip3(-limit, limit, feature_value)	
} else {	
feature_value	f(bitsToRead)
clippedValue = Clip3(0, limit, feature_value)	
}	
}	
data[i][j] = clippedValue	
}	
}	
return (enabled, data)	
}	

§ 5.4.10. Sequence filter config syntax

sequence_filter_config() {	Descriptor
disable_loopfilters_across_tiles	f(1)
enable_cdef	f(1)
enable_gdf	f(1)
if (enable_gdf && get_seq_sb_size() == BLOCK_64X64) {	
gdf_unit_matches_sb_size	f(1)
} else {	
gdf_unit_matches_sb_size = 0	
}	
enable_restoration	f(1)
if (enable_restoration) {	
lr_tools_disable[0][RESTORE_PC_WIENER]	f(1)

lr_tools_disable[0] [RESTORE_WIENER_NONSEP]	f(1)
lr_tools_disable[1] [RESTORE_PC_WIENER] = 1	
lr_tools_uv_present	f(1)
if (lr_tools_uv_present) {	
lr_tools_disable[1] [RESTORE_WIENER_NONSEP]	f(1)
} else {	
lr_tools_disable[1] [RESTORE_WIENER_NONSEP] =	
lr_tools_disable[0] [RESTORE_WIENER_NONSEP]	
}	
}	
enable_ccso	f(1)
if (enable_ccso) {	
ccso_unit_matches_sb_size	f(1)
} else {	
ccso_unit_matches_sb_size = 0	
}	
if (single_picture_header_flag) {	
CdefOnSkipTxfm = CDEF_ON_SKIP_TXFM_ADAPTIVE	
} else {	
cdef_on_skip_txfm_always_on	f(1)
if (cdef_on_skip_txfm_always_on) {	
CdefOnSkipTxfm = CDEF_ON_SKIP_TXFM_ALWAYS_ON	
} else {	
cdef_on_skip_txfm_disabled	f(1)
CdefOnSkipTxfm = cdef_on_skip_txfm_disabled ?	
CDEF_ON_SKIP_TXFM_DISABLED : CDEF_ON_SKIP_TXFM_ADAPTIVE	
}	
}	
df_par_bits_minus_2	f(2)
}	

§ 5.4.11. User defined QM syntax

user_defined_qm(level, t, plane) {	Descriptor
txSz = Fundamental_Tx_Size[t]	
w = Tx_Width[txSz]	
h = Tx_Height[txSz]	
if (plane > 0) {	
qm_copy_from_previous_plane	f(1)
if (qm_copy_from_previous_plane) {	
for (i = 0; i < h; i++) {	
for (j = 0; j < w; j++) {	

UserQm[level][t][plane][i][j] =	
UserQm[level][t][plane - 1][i][j]	
}	
}	
return	
}	
}	
if (t == 0) {	
qm_8x8_is_symmetric	f(1)
} else if (t == 2) {	
qm_4x8_is_transpose_of_8x4	f(1)
if (qm_4x8_is_transpose_of_8x4) {	
for (i = 0; i < h; i++) {	
for (j = 0; j < w; j++) {	
UserQm[level][t][plane][i][j] =	
UserQm[level][1][plane][j][i]	
}	
}	
return	
}	
}	
scan = get_scan(txSz, TX_CLASS_2D)	
quant = 32	
coefRepeat = 0	
for (c = 0; c < w * h; c++) {	
pos = scan[c]	
(row, col) = get_tx_row_col(pos, txSz)	
if (t == 0 && qm_8x8_is_symmetric && col > row) {	
quant = UserQm[level][t][plane][col][row]	
UserQm[level][t][plane][row][col] = quant	
} else if (coefRepeat) {	
UserQm[level][t][plane][row][col] = quant	
} else {	
quant_delta	svlc()
quant2 = (quant + quant_delta) & 255	
if (quant2 == 0) {	
coefRepeat = 1	
} else {	
quant = quant2	
}	
UserQm[level][t][plane][row][col] = quant	

}	
}	
}	

where Fundamental_Tx_Size (which gives the order of quantization matrices) is specified as:

```
Fundamental_Tx_Size[ 3 ] = { TX_8X8, TX_8X4, TX_4X8 }
```

§ 5.4.12. Timing info syntax

timing_info() {	Descriptor
num_units_in_display_tick	f(32)
time_scale	f(32)
equal_picture_interval	f(1)
if (equal_picture_interval) {	
num_ticks_per_picture_minus_1	uvlc()
}	
}	

§ 5.4.13. Sequence decoder model info syntax

seq_decoder_model_info() {	Descriptor
decoder_buffer_delay	uvlc()
encoder_buffer_delay	uvlc()
low_delay_mode_flag	f(1)
}	

§ 5.5. Temporal delimiter OBU syntax

temporal_delimiter_obu() {	Descriptor
SeenFrameHeader = 0	
for (level = 0; level < 15; level++) {	
QmProtected[level] = 0	
}	
}	

NOTE: The temporal delimiter has an empty payload.

§ 5.6. Multi Stream Decoder Operation OBU syntax

multistream_decoder_operation_obu() {	Descriptor
num_streams_minus_2	f(3)
multistream_profile_idc	f(5)
multistream_level_idx	f(5)

multistream_tier	f(1)
multistream_even_allocation_flag	f(1)
if (!multistream_even_allocation_flag) {	
multistream_large_picture_idc	f(3)
}	
for (i = 0; i < num_streams_minus_2 + 2; i++) {	
sub_xlayer_id[i]	f(5)
sub_stream_max_profile[i]	f(5)
sub_stream_max_level[i]	f(5)
sub_stream_max_tier[i]	f(1)
}	
multistream_doh_constraint_flag	f(1)
}	

§ 5.7. Multi frame header OBU syntax

multi_frame_header_obu() {	Descriptor
mfh_seq_header_id	uvlc()
mfh_id_minus_1	uvlc()
mfhId = mfh_id_minus_1 + 1	
MfhSeqHeaderId[mfhId] = mfh_seq_header_id	
MfhTLayerId[mfhId] = obu_tlayer_id	
MfhMLayerId[mfhId] = obu_mlayer_id	
mfh_frame_size_present_flag[mfhId]	f(1)
if (mfh_frame_size_present_flag[mfhId]) {	
mfh_frame_width_bits_minus_1	f(4)
mfh_frame_height_bits_minus_1	f(4)
n = mfh_frame_width_bits_minus_1 + 1	
mfh_frame_width_minus_1[mfhId]	f(n)
n = mfh_frame_height_bits_minus_1 + 1	
mfh_frame_height_minus_1[mfhId]	f(n)
}	
mfh_deblocking_filter_update[mfhId]	f(1)
if (mfh_deblocking_filter_update[mfhId]) {	
for (i = 0; i < 4; i++) {	
mfh_apply_deblocking_filter[mfhId][i]	f(1)
}	
}	
mfh_seg_info_present_flag[mfhId]	f(1)
if (mfh_seg_info_present_flag[mfhId]) {	
mfh_ext_seg_flag[mfhId]	f(1)
mfh_allow_seg_info_change[mfhId]	f(1)

(MfhFeatureEnabled[mfhId], MfhFeatureData[mfhId]) =	
seg_info(mfh_ext_seg_flag[mfhId] ? 16 : 8)	
}	
}	

§ 5.8. Layer config record OBU syntax

layer_config_record_obu() {	Descriptor
if (obu_xlayer_id == GLOBAL_XLAYER_ID) {	
lcr_global_info()	
} else {	
lcr_local_info(obu_xlayer_id)	
}	
}	

§ 5.8.1. LCR global info syntax

lcr_global_info() {	Descriptor
lcr_global_config_record_id	f(3)
lcr_xlayer_map	f(31)
LcrMaxNumXLayerCount = 0	
for (i = 0; i < 31; i++) {	
if (lcr_xlayer_map & (1 << i)) {	
LcrXLayerID[LcrMaxNumXLayerCount] = i	
LcrMaxNumXLayerCount ++	
}	
}	
lcr_aggregate_info_present_flag	f(1)
lcr_seq_profile_tier_level_info_present_flag	f(1)
lcr_global_payload_present_flag	f(1)
lcr_dependent_xlayers_flag	f(1)
lcr_global_atlas_id_present_flag	f(1)
lcr_global_purpose_id	f(7)
lcr_doh_constraint_flag	f(1)
lcr_enforce_tile_alignment_flag	f(1)
if (lcr_global_atlas_id_present_flag) {	
lcr_global_atlas_id	f(3)
} else {	
lcr_global_reserved_zero_3bits	f(3)
}	
lcr_global_reserved_zero_5bits	f(5)
if (lcr_aggregate_info_present_flag) {	
lcr_aggregate_info()	

}	
if (lcr_seq_profile_tier_level_info_present_flag) {	
for (i = 0; i < LcrMaxNumXLayerCount; i++) {	
lcr_seq_profile_tier_level_info(LcrXLayerID[i])	
}	
}	
if (lcr_global_payload_present_flag) {	
for (i = 0; i < LcrMaxNumXLayerCount; i++) {	
lcr_data_size [i]	leb128()
lcr_global_payload(LcrXLayerID[i], lcr_data_size [i])	
}	
}	
}	

§ 5.8.2. LCR local info syntax

lcr_local_info(xlayerId) {	Descriptor
lcr_global_id[xlayerId]	f(3)
lcr_local_id[xlayerId]	f(3)
lcr_profile_tier_level_info_present_flag[xlayerId]	f(1)
lcr_local_atlas_id_present_flag[xlayerId]	f(1)
if (lcr_profile_tier_level_info_present_flag[xlayerId]) {	
lcr_seq_profile_tier_level_info(xlayerId)	
}	
if (lcr_local_atlas_id_present_flag[xlayerId]) {	
lcr_local_atlas_id[xlayerId]	f(3)
} else {	
lcr_local_reserved_zero_3bits[xlayerId]	f(3)
}	
lcr_local_reserved_zero_5bits[xlayerId]	f(5)
lcr_xlayer_info(0, xlayerId)	
}	

§ 5.8.3. LCR aggregate info syntax

lcr_aggregate_info() {	Descriptor
lcr_config_idc	f(6)
lcr_aggregate_level_idx	f(5)
lcr_max_tier_flag	f(1)
lcr_max_interop	f(4)
}	

§ 5.8.4. LCR sequence profile tier level information syntax

lcr_seq_profile_tier_level_info(i) {	Descriptor
lcr_seq_profile_idc[i]	f(5)
lcr_max_level_idx[i]	f(5)
lcr_tier_flag[i]	f(1)
lcr_max_mlayer_count[i]	f(3)
lsptli_reserved_2bits	f(2)
}	

§ 5.8.5. LCR global payload syntax

lcr_global_payload(n, sz) {	Descriptor
startPosition = get_position()	
if (lcr_dependent_xlayers_flag && n > 0) {	
lcr_num_dependent_xlayer_map[n]	f(n)
}	
lcr_xlayer_info(1 , n)	
currentPosition = get_position()	
parsedPayloadBits = currentPosition - startPosition	
RemainingLcrPayloadBits = sz * 8 - parsedPayloadBits	
for (j = 0; j < RemainingLcrPayloadBits; j++) {	
lcr_remaining_payload_bit	f(1)
}	
}	

§ 5.8.6. LCR xlayer info syntax

lcr_xlayer_info(isGlobal, xId) {	Descriptor
lcr_rep_info_present_flag[isGlobal][xId]	f(1)
lcr_xlayer_purpose_present_flag[isGlobal][xId]	f(1)
lcr_xlayer_color_info_present_flag[isGlobal][xId]	f(1)
lcr_embedded_layer_info_present_flag[isGlobal][xId]	f(1)
if (lcr_rep_info_present_flag[isGlobal][xId]) {	
lcr_rep_info(isGlobal, xId)	
}	
if(lcr_xlayer_purpose_present_flag[isGlobal][xId]) {	
lcr_xlayer_purpose_id[isGlobal][xId]	f(7)
}	
if(lcr_xlayer_color_info_present_flag[isGlobal][xId]) {	
lcr_xlayer_color_info(isGlobal, xId)	
}	
byte_alignment()	
if (lcr_embedded_layer_info_present_flag[isGlobal][xId]) {	

lcr_embedded_layer_info(isGlobal, xId)	
} else {	
if (isGlobal && lcr_global_atlas_id_present_flag) {	
lcr_xlayer_atlas_segment_id[xId]	f(8)
lcr_xlayer_priority_order[xId]	f(8)
lcr_xlayer_rendering_method[xId]	f(8)
}	
}	
}	

§ 5.8.7. LCR rep info syntax

lcr_rep_info(isGlobal, xId) {	Descriptor
lcr_max_pic_width[isGlobal][xId]	uvlc()
lcr_max_pic_height[isGlobal][xId]	uvlc()
lcr_format_info_present_flag[isGlobal][xId]	f(1)
lcr_cropping_window_present_flag[isGlobal][xId]	f(1)
if (lcr_format_info_present_flag[isGlobal][xId]) {	
lcr_bit_depth_idc[isGlobal][xId]	uvlc()
lcr_chroma_format_idc[isGlobal][xId]	uvlc()
}	
if (lcr_cropping_window_present_flag[isGlobal][xId]) {	
lcr_cropping_win_left_offset [isGlobal][xId]	uvlc()
lcr_cropping_win_right_offset[isGlobal][xId]	uvlc()
lcr_cropping_win_top_offset [isGlobal][xId]	uvlc()
lcr_cropping_win_bottom_offset[isGlobal][xId]	uvlc()
}	
}	

§ 5.8.8. LCR embedded layer info syntax

lcr_embedded_layer_info(isGlobal, xId) {	Descriptor
lcr_mlayer_map[isGlobal][xId]	f(8)
for (j = 0; j < 8; j++) {	
if (lcr_mlayer_map[isGlobal][xId] & (1 << j)) {	
n = MAX_NUM_TLAYERS	
lcr_tlayer_map[isGlobal][xId][j]	f(n)
atlasSegmentPresent = isGlobal ?	
lcr_global_atlas_id_present_flag :	
lcr_local_atlas_id_present_flag[xId]	
if (atlasSegmentPresent) {	
lcr_layer_atlas_segment_id[isGlobal][xId][j]	f(8)
lcr_priority_order[isGlobal][xId][j]	f(8)

lcr_rendering_method[isGlobal][xId][j]	f(8)
}	
lcr_layer_type[isGlobal][xId][j]	f(8)
if (lcr_layer_type[isGlobal][xId][j] == AUX_LAYER) {	
lcr_auxiliary_type[isGlobal][xId][j]	f(8)
}	
lcr_view_type[isGlobal][xId][j]	f(8)
if (lcr_view_type[isGlobal][xId][j] == VIEW_EXPLICIT) {	
lcr_view_id[isGlobal][xId][j]	f(8)
}	
if (j > 0) {	
lcr_dependent_layer_map[isGlobal][xId][j]	f(j)
}	
lcr_same_sh_max_resolution_flag[isGlobal][xId][j]	f(1)
if (!lcr_same_sh_max_resolution_flag[isGlobal][xId][j]) {	
lcr_max_expected_width[isGlobal][xId][j]	uvlc()
lcr_max_expected_height[isGlobal][xId][j]	uvlc()
}	
byte_alignment()	
}	
}	
}	

§ 5.8.9. LCR xlayer color info syntax

lcr_xlayer_color_info(isGlobal, xId) {	Descriptor
layer_color_description_idc[isGlobal][xId]	rg(2)
if (layer_color_description_idc[isGlobal][xId] == 0) {	
layer_color_primaries[isGlobal][xId]	f(8)
layer_transfer_characteristics[isGlobal][xId]	f(8)
layer_matrix_coefficients[isGlobal][xId]	f(8)
}	
layer_full_range_flag[isGlobal][xId]	f(1)
}	

§ 5.9. Atlas segment info OBU syntax

atlas_segment_info_obu() {	Descriptor
atlas_segment_id[obu_xlayer_id]	f(3)
xAId = atlas_segment_id[obu_xlayer_id]	
ats_atlas_segment_mode_idc[xAId]	uvlc()
if (ats_atlas_segment_mode_idc[xAId] == ENHANCED_ATLAS) {	
numSegments = ats_enhanced_atlas_info(xAId)	

} else if (ats_atlas_segment_mode_idc[xAId] == BASIC_ATLAS) {	
numSegments = ats_basic_info(xAId)	
} else if (ats_atlas_segment_mode_idc[xAId] == SINGLE_ATLAS) {	
numSegments = 1	
ats_nominal_width_minus_1[xAId]	uvlc()
ats_nominal_height_minus_1[xAId]	uvlc()
} else if (ats_atlas_segment_mode_idc[xAId] == MULTISTREAM_ATLAS) {	
numSegments = ats_multistream_info(obu_xlayer_id, xAId)	
} else if (ats_atlas_segment_mode_idc[xAId] ==	
MULTISTREAM_ALPHA_ATLAS) {	
numSegments = ats_multistream_with_alpha_info(obu_xlayer_id, xAId)	
}	
ats_label_segment_info(obu_xlayer_id, xAId, numSegments)	
}	

§ 5.9.1. Atlas label segment info syntax

ats_label_segment_info(xlayerId, xAId, numSegments) {	Descriptor
ats_signaled_atlas_segment_ids_flag[xlayerId][xAId]	f(1)
if (ats_signaled_atlas_segment_ids_flag[xlayerId][xAId]) {	
for (i = 0; i < numSegments; i++) {	
ats_atlas_segment_id[xlayerId][xAId][i]	f(8)
AtlasSegmentIDToIndex[xlayerId][xAId]	
[ats_atlas_segment_id[xlayerId][xAId][i]] = i	
AtlasSegmentIndexToID[xlayerId][xAId][i] =	
ats_atlas_segment_id[xlayerId][xAId][i]	
}	
} else {	
for (i = 0; i < numSegments; i++) {	
ats_atlas_segment_id[xlayerId][xAId][i] = i	
AtlasSegmentIDToIndex[xlayerId][xAId][i] = i	
AtlasSegmentIndexToID[xlayerId][xAId][i] = i	
}	
}	
}	

§ 5.9.2. Atlas enhanced atlas info syntax

ats_enhanced_atlas_info(xAId) {	Descriptor
ats_region_info(xAId)	
numSegments = ats_region_to_segment_mapping(xAId)	
return numSegments	
}	

§ 5.9.2.1. Atlas region info syntax

ats_region_info(xAId) {	Descriptor
ats_num_region_columns_minus_1[xAId]	uvlc()
ats_num_region_rows_minus_1[xAId]	uvlc()
ats_uniform_spacing_flag[xAId]	f(1)
AtlasWidth = 0	
AtlasHeight = 0	
if (!ats_uniform_spacing_flag[xAId]) {	
for (i = 0; i < ats_num_region_columns_minus_1[xAId] + 1;	
i++) {	
ats_column_width_minus_1[xAId][i]	uvlc()
AtlasWidth += (ats_column_width_minus_1[xAId][i] + 1)	
}	
for (i = 0; i < ats_num_region_rows_minus_1[xAId] + 1; i++) {	
ats_row_height_minus_1[xAId][i]	uvlc()
AtlasHeight += (ats_row_height_minus_1[xAId][i] + 1)	
}	
} else {	
ats_region_width_minus_1[xAId]	uvlc()
ats_region_height_minus_1[xAId]	uvlc()
AtlasWidth =	
(ats_region_width_minus_1[xAId] + 1) *	
(ats_num_region_columns_minus_1[xAId] + 1)	
AtlasHeight =	
(ats_region_height_minus_1[xAId] + 1) *	
(ats_num_region_rows_minus_1[xAId] + 1)	
}	
NumRegionsInAtlas[xAId] =	
(ats_num_region_columns_minus_1[xAId] + 1) *	
(ats_num_region_rows_minus_1[xAId] + 1)	
}	

§ 5.9.2.2. Atlas region to segment mapping syntax

ats_region_to_segment_mapping(xAId) {	Descriptor
ats_single_region_per_atlas_segment_flag[xAId]	f(1)
if (!ats_single_region_per_atlas_segment_flag[xAId]) {	
ats_num_atlas_segments_minus_1[xAId]	uvlc()
for (i = 0; i <= ats_num_atlas_segments_minus_1[xAId]; i++) {	
ats_top_left_region_column[xAId][i]	uvlc()
ats_top_left_region_row[xAId][i]	uvlc()
ats_bottom_right_region_column_off[xAId][i]	uvlc()

ats_bottom_right_region_row_off[xAId][i]	uvlc()
}	
} else {	
ats_num_atlas_segments_minus_1[xAId] =	
NumRegionsInAtlas[xAId] - 1	
}	
return ats_num_atlas_segments_minus_1[xAId] + 1	
}	

§ 5.9.3. Atlas multistream info syntax

ats_multistream_info(xlayerId, xAId) {	Descriptor
ats_msi_width[xlayerId][xAId]	uvlc()
ats_msi_height[xlayerId][xAId]	uvlc()
AtlasWidth = ats_msi_width[xlayerId][xAId]	
AtlasHeight = ats_msi_height[xlayerId][xAId]	
ats_msi_num_atlas_segments_minus_1[xlayerId][xAId]	uvlc()
ats_msi_background_info_present_flag[xlayerId][xAId]	f(1)
if (ats_msi_background_info_present_flag[xlayerId][xAId]) {	
ats_msi_background_red_value[xlayerId][xAId]	f(8)
ats_msi_background_green_value[xlayerId][xAId]	f(8)
ats_msi_background_blue_value[xlayerId][xAId]	f(8)
}	
for (i=0;i<=ats_msi_num_atlas_segments_minus_1[xlayerId][xAId];i++) {	
ats_msi_input_stream_id[xlayerId][xAId][i]	f(5)
ats_msi_segment_top_left_pos_x[xlayerId][xAId][i]	uvlc()
ats_msi_segment_top_left_pos_y[xlayerId][xAId][i]	uvlc()
ats_msi_segment_width[xlayerId][xAId][i]	uvlc()
ats_msi_segment_height[xlayerId][xAId][i]	uvlc()
}	
return ats_msi_num_atlas_segments_minus_1[xlayerId][xAId] + 1	
}	

§ 5.9.4. Atlas multistream with alpha info syntax

ats_multistream_with_alpha_info(xlayerId, xAId) {	Descriptor
ats_msi_width[xlayerId][xAId]	uvlc()
ats_msi_height[xlayerId][xAId]	uvlc()
AtlasWidth = ats_msi_width[xlayerId][xAId]	
AtlasHeight = ats_msi_height[xlayerId][xAId]	
ats_msi_num_atlas_segments_minus_1[xlayerId][xAId]	uvlc()
ats_msi_alpha_segments_present_flag[xlayerId][xAId]	f(1)
ats_msi_background_info_present_flag[xlayerId][xAId]	f(1)

if (ats_msi_background_info_present_flag[xlayerId][xAId]) {	
ats_msi_background_red_value[xlayerId][xAId]	f(8)
ats_msi_background_green_value[xlayerId][xAId]	f(8)
ats_msi_background_blue_value[xlayerId][xAId]	f(8)
}	
for (i=0;i<=ats_msi_num_atlas_segments_minus_1[xlayerId][xAId];i++) {	
ats_msi_input_stream_id[xlayerId][xAId][i]	f(5)
ats_msi_segment_top_left_pos_x[xlayerId][xAId][i]	uvlc()
ats_msi_segment_top_left_pos_y[xlayerId][xAId][i]	uvlc()
ats_msi_segment_width[xlayerId][xAId][i]	uvlc()
ats_msi_segment_height[xlayerId][xAId][i]	uvlc()
if (ats_msi_alpha_segments_present_flag[xlayerId][xAId] &&	
i != ats_msi_num_atlas_segments_minus_1[xlayerId][xAId]) {	
ats_msi_alpha_segment_flag[xlayerId][xAId][i]	f(1)
} else {	
ats_msi_alpha_segment_flag[xlayerId][xAId][i] = 0	
}	
}	
return ats_msi_num_atlas_segments_minus_1[xlayerId][xAId] + 1	
}	

§ 5.9.5. Atlas basic info syntax

ats_basic_info(xAId) {	Descriptor
ats_stream_id_present[xAId]	f(1)
ats_width[xAId]	uvlc()
ats_height[xAId]	uvlc()
ats_num_atlas_segments_minus_1[xAId]	uvlc()
AtlasWidth = ats_width[xAId]	
AtlasHeight = ats_height[xAId]	
for (i = 0; i <= ats_num_atlas_segments_minus_1[xAId]; i++) {	
if (ats_stream_id_present[xAId]) {	
ats_input_stream_id[xAId][i]	f(5)
}	
ats_segment_top_left_pos_x[xAId][i]	uvlc()
ats_segment_top_left_pos_y[xAId][i]	uvlc()
ats_segment_width[xAId][i]	uvlc()
ats_segment_height[xAId][i]	uvlc()
}	
return ats_num_atlas_segments_minus_1[xAId] + 1	
}	

§ 5.10. Operating point set OBU syntax

operating_point_set_obu() {	Descriptor
ops_reset_flag[obu_xlayer_id]	f(1)
ops_id[obu_xlayer_id]	f(4)
opsID = ops_id[obu_xlayer_id]	
ops_cnt[obu_xlayer_id][opsID]	f(3)
if (ops_cnt[obu_xlayer_id][opsID] > 0) {	
ops_priority[obu_xlayer_id][opsID]	f(4)
ops_intent[obu_xlayer_id][opsID]	f(7)
ops_intent_present_flag[obu_xlayer_id][opsID]	f(1)
ops_ptl_present_flag[obu_xlayer_id][opsID]	f(1)
ops_color_info_present_flag[obu_xlayer_id][opsID]	f(1)
if (obu_xlayer_id == GLOBAL_XLAYER_ID) {	
ops_mlayer_info_idc[opsID]	f(2)
} else {	
ops_reserved_2bits	f(2)
}	
for(i = 0; i < ops_cnt[obu_xlayer_id][opsID]; i++) {	
operating_point_payload(obu_xlayer_id, opsID, i)	
}	
}	
}	

§ 5.11. Operating point payload syntax

operating_point_payload(xId, opsID, i) {	Descriptor
ops_data_size[xId][opsID][i]	leb128()
startPos = get_position()	
if (ops_intent_present_flag[xId][opsID]) {	
ops_op_intent[xId][opsID][i]	f(7)
}	
if (ops_ptl_present_flag[xId][opsID]) {	
if (xId == GLOBAL_XLAYER_ID) {	
ops_aggregate_info(opsID, i)	
} else {	
ops_seq_profile_tier_level_info(xId, opsID, i, xId)	
}	
}	
if (ops_color_info_present_flag[xId][opsID]) {	
ops_color_info(opsID, i)	
}	
ops_decoder_model_info_for_this_op_present_flag[xId][opsID][i]	

	f(1)
if (ops_decoder_model_info_for_this_op_present_flag[xId][opsID][i]) {	
ops_decoder_model_info(opsID, i)	
}	
ops_initial_display_delay_present_flag[xId][opsID][i]	f(1)
if (ops_initial_display_delay_present_flag[xId][opsID][i]) {	
ops_initial_display_delay_minus_1[xId][opsID][i]	f(4)
}	
if (xId == GLOBAL_XLAYER_ID) {	
ops_xlayer_map[opsID][i]	f(31)
k = 0	
for (j = 0; j < 31; j++) {	
if (ops_xlayer_map[opsID][i] & (1 << j)) {	
OpsxLayerId[xId][opsID][i][k] = j	
k++	
if (ops_ptl_present_flag[xId][opsID]) {	
ops_seq_profile_tier_level_info(xId, opsID, i, j)	
}	
idc = ops_mlayer_info_idc[opsID]	
if (idc == 1) {	
ops_mlayer_info(xId, opsID, i, j)	
} else if (idc == 2) {	
ops_mlayer_explicit_info_flag[opsID][i][j]	f(1)
if (ops_mlayer_explicit_info_flag[opsID][i][j]) {	
ops_mlayer_info(xId, opsID, i, j)	
} else {	
ops_embedded_ops_id[opsID][i][j]	f(4)
ops_embedded_op_index[opsID][i][j]	f(3)
}	
}	
}	
XCount[xId][opsID][i] = k	
} else {	
XCount[xId][opsID][i] = 1	
OpsxLayerId[xId][opsID][i][0] = xId	
ops_mlayer_info(xId, opsID, i, xId)	
}	
byte_alignment()	
opsBytes = (get_position() - startPos) >> 3	
}	

§ 5.11.1. Operating point set aggregate info syntax

ops_aggregate_info(opsID, i) {	Descriptor
ops_config_idc[opsID][i]	f(6)
ops_aggregate_level_idx[opsID][i]	f(5)
ops_max_tier_flag[opsID][i]	f(1)
ops_max_interop[opsID][i]	f(4)
}	

§ 5.11.2. Operating point set sequence profile tier level information syntax

ops_seq_profile_tier_level_info(xId, opsID, i, j) {	Descriptor
ops_seq_profile_idc[xId][opsID][i][j]	f(5)
ops_level_idx[xId][opsID][i][j]	f(5)
ops_tier_flag[xId][opsID][i][j]	f(1)
ops_mlayer_count[xId][opsID][i][j]	f(3)
ops_ptl_reserved_2bits	f(2)
}	

§ 5.11.3. Operating point set decoder model info syntax

ops_decoder_model_info(opsID, i) {	Descriptor
ops_decoder_buffer_delay[obu_xlayer_id][opsID][i]	uvlc()
ops_encoder_buffer_delay[obu_xlayer_id][opsID][i]	uvlc()
ops_low_delay_mode_flag[obu_xlayer_id][opsID][i]	f(1)
}	

§ 5.11.4. Operating point set color info syntax

ops_color_info(opsID, i) {	Descriptor
ops_color_description_idc[obu_xlayer_id][opsID][i]	rg(2)
if (ops_color_description_idc[obu_xlayer_id][opsID][i] == 0) {	
ops_color_primaries[obu_xlayer_id][opsID][i]	f(8)
ops_transfer_characteristics[obu_xlayer_id][opsID][i]	f(8)
ops_matrix_coefficients[obu_xlayer_id][opsID][i]	f(8)
}	
ops_full_range_flag[obu_xlayer_id][opsID][i]	f(1)
}	

§ 5.11.5. Operating point set mlayer info syntax

ops_mlayer_info(obuXLId, opsID, opIndex, xLId) {	Descriptor
ops_mlayer_map[obuXLId][opsID][opIndex][xLId]	f(8)
mCount = 0	
for (j = 0; j < 8; j++) {	

if (ops_mlayer_map[obuXLid][opsID][opIndex][xLid] & (1 << j)) {	
ops_tlayer_map[obuXLid][opsID][opIndex][xLid][j]	f(4)
tCount = 0	
for (k = 0; k < 4; k++) {	
if (ops_tlayer_map[obuXLid][opsID][opIndex][xLid][j]	
& (1 << k)) {	
tCount++	
}	
}	
mCount++	
}	
}	
}	

§ 5.12. Buffer removal timing OBU syntax

buffer_removal_timing_obu() {	Descriptor
br_ops_dependent_flag	f(1)
if (br_ops_dependent_flag) {	
br_ops_id	f(4)
br_ops_cnt[br_ops_id]	f(3)
for (i = 0; i < br_ops_cnt[br_ops_id]; i++) {	
br_decoder_model_present_op_flag[br_ops_id][i]	f(1)
if (br_decoder_model_present_op_flag[br_ops_id][i]) {	
br_time_op[br_ops_id][i]	rg(4)
}	
}	
} else {	
br_time	rg(4)
}	
}	

§ 5.13. Quantizer Matrix OBU syntax

quantizer_matrix_obu() {	Descriptor
qm_bit_map	f(15)
qm_chroma_info_present_flag	f(1)
numPlanes = qm_chroma_info_present_flag ? 3 : 1	
if (qm_bit_map == 0){	
for (level = 0; level < NUM_CUSTOM_QMS; level++) {	
QmProtected[level] = 1	
QmNumPlanes[level] = numPlanes	
QmDataPresent[level] = 0	

QmMLayerId[level] = -1	
QmTLayerId[level] = -1	
}	
} else {	
for (level = 0; level < 15; level++) {	
if (qm_bit_map & (1 << level)) {	
QmSeen[level] = 1	
QmProtected[level] = 1	
QmNumPlanes[level] = numPlanes	
QmMLayerId[level] = obu_mlayer_id	
QmTLayerId[level] = obu_tlayer_id	
QmDataPresent[level] = 1	
qm_is_default_flag	f(1)
if (qm_is_default_flag) {	
QmDataPresent[level] = 0	
} else {	
for (t = 0; t < 3; t++){	
for (plane = 0; plane < numPlanes; plane++) {	
user_defined_qm(level, t, plane)	
}	
}	
}	
}	
}	
}	
}	

§ 5.14. Film grain OBU syntax

film_grain_obu() {	Descriptor
fgm_update_flags	f(8)
fgm_chroma_idc	uvlc()
if (fgm_chroma_idc == CHROMA_FORMAT_420) {	
subX = 1	
subY = 1	
} else if (fgm_chroma_idc == CHROMA_FORMAT_444) {	
subX = 0	
subY = 0	
} else if (fgm_chroma_idc == CHROMA_FORMAT_422) {	
subX = 1	
subY = 0	
} else if (fgm_chroma_idc == CHROMA_FORMAT_400) {	

subX = 1	
subY = 1	
}	
monochrome = fgm_chroma_idc == CHROMA_FORMAT_400	
for (i = 0; i < MAX_FILM_GRAIN; i++) {	
if (fgm_update_flags & (1 << i)) {	
FilmGrainPresent[i] = 1	
film_grain_model(monochrome, subX, subY)	
save_grain_model(i)	
FgmTLayerId[i] = obu_tlayer_id	
FgmMLayerId[i] = obu_mlayer_id	
FgmChromaIdc[i] = fgm_chroma_idc	
}	
}	
}	

§ 5.15. Content interpretation OBU syntax

content_interpretation_obu() {	Descriptor
ci_scan_type_idc	f(2)
ci_color_description_present_flag	f(1)
ci_chroma_sample_position_present_flag	f(1)
ci_aspect_ratio_info_present_flag	f(1)
ci_timing_info_present_flag	f(1)
ci_reserved_2bit	f(2)
ci_color_primaries = CP_UNSPECIFIED	
ci_transfer_characteristics = TC_UNSPECIFIED	
ci_matrix_coefficients = MC_UNSPECIFIED	
ci_full_range_flag = 0	
if (ci_color_description_present_flag) {	
ci_color_description_idc	rg(2)
if (ci_color_description_idc == 0) {	
ci_color_primaries	f(8)
ci_transfer_characteristics	f(8)
ci_matrix_coefficients	f(8)
}	
ci_full_range_flag	f(1)
}	
if (ci_chroma_sample_position_present_flag) {	
ci_chroma_sample_position_top	uvlc()
if (ci_scan_type_idc != 1) {	
ci_chroma_sample_position_bottom	uvlc()

} else {	
ci_chroma_sample_position_bottom = ci_chroma_sample_position_top	
}	
} else {	
ci_chroma_sample_position_top = CSP_UNSPECIFIED	
ci_chroma_sample_position_bottom = CSP_UNSPECIFIED	
}	
if (ci_aspect_ratio_info_present_flag) {	
ci_aspect_ratio_idc	f(8)
if (ci_aspect_ratio_idc == 255) {	
ci_sar_width	uvlc()
ci_sar_height	uvlc()
} else {	
ci_sar_width = Aspect_Ratio_Width[ci_aspect_ratio_idc]	
ci_sar_height = Aspect_Ratio_Height[ci_aspect_ratio_idc]	
}	
}	
if (ci_timing_info_present_flag) {	
timing_info()	
}	
}	

where the tables Aspect_Ratio_Width and Aspect_Ratio_Height are specified as:

```

Aspect_Ratio_Width[ 17 ] = { 0, 1, 12, 10, 16, 40, 24, 20,
                             32, 80, 18, 15, 64, 160, 4, 3, 2 }

Aspect_Ratio_Height[ 17 ] = { 0, 1, 11, 11, 11, 33, 11, 11,
                               11, 33, 11, 11, 33, 99, 3, 2, 1 }

```

§ 5.16. Padding OBU syntax

padding_obu() {	Descriptor
for (i = 0; i < obu_padding_length; i++) {	
obu_padding_byte	f(8)
}	
}	

NOTE: `obu_padding_length` is not coded in the [bitstream](#) but can be computed based on the OBU size minus the number of trailing bytes. In practice, though, since this is padding data meant to be skipped, decoders do not need to determine either that length or the number of trailing bytes. They can ignore the entire [OBU](#). The last byte of the valid content of the payload data for this [OBU](#) type is considered to be the last byte that is not equal to zero. This rule is to prevent the dropping of valid bytes by systems that interpret trailing zero bytes as a continuation of the trailing bits in an [OBU](#). This implies that when any payload data is present for this [OBU](#) type, at least one byte of the payload data (including the trailing bit) shall not be equal to 0.

NOTE: A padding OBU with an `obuPayloadSize` of 0 is legal. This means the OBU has `obu_padding_length` of 0 and will not contain any trailing bits. A padding OBU with an `obuPayloadSize` of 1 is legal. This means the OBU has `obu_padding_length` of 0 and does contain trailing bits. This is allowed so that any OBU can be converted into a padding OBU in-place.

§ 5.17. Metadata OBU syntax

This specification defines two distinct OBU types for carrying metadata:

- `OBU_METADATA_SHORT`: using metadata short OBU syntax, and
- `OBU_METADATA_GROUP`: using metadata group OBU syntax.

Both OBU types use the same `metadata_unit()` syntax element to carry the actual metadata payload. The `OBU_METADATA_SHORT` type provides a compact header structure, while `OBU_METADATA_GROUP` provides extended capabilities including the ability to carry multiple metadata units within a single OBU with additional signaling for application-specific handling, layer targeting, and priority.

§ 5.17.1. Metadata unit syntax

<code>metadata_unit(metadataPayloadSize) {</code>	Descriptor
<code>startPosition = get_position()</code>	
<code>if (metadata_type == METADATA_TYPE_ITUT_T35) {</code>	
<code> metadata_itut_t35(metadataPayloadSize)</code>	
<code>} else if (metadata_type == METADATA_TYPE_HDR_CLL) {</code>	
<code> metadata_hdr_cll()</code>	
<code>} else if (metadata_type == METADATA_TYPE_HDR_MDCV) {</code>	
<code> metadata_hdr_mdcv()</code>	
<code>} else if (metadata_type == METADATA_TYPE_TIMECODE) {</code>	
<code> metadata_timecode()</code>	
<code>} else if (metadata_type == METADATA_TYPE_BANDING_HINTS) {</code>	
<code> metadata_banding_hints()</code>	
<code>} else if (metadata_type == METADATA_TYPE_ICC_PROFILE) {</code>	
<code> metadata_icc_profile(metadataPayloadSize)</code>	
<code>} else if (metadata_type == METADATA_TYPE_SCAN_TYPE) {</code>	
<code> metadata_scan_type()</code>	
<code>} else if (metadata_type == METADATA_TYPE_TEMPORAL_POINT_INFO) {</code>	

metadata_temporal_point_info()	
} else if (metadata_type == METADATA_TYPE_DECODED_FRAME_HASH) {	
metadata_decoded_frame_hash()	
} else if (metadata_type == METADATA_TYPE_USER_DATA_UNREGISTERED) {	
metadata_user_data_unregistered(metadataPayloadSize)	
}	
currentPosition = get_position()	
parsedPayloadBits = currentPosition - startPosition	
remainingMuPayloadBits = metadataPayloadSize * 8 - parsedPayloadBits	
for (j = 0; j < remainingMuPayloadBits; j++) {	
metadata_unit_remaining_bit	f(1)
}	
}	

NOTE: The exact syntax of `metadata_unit` is not defined in this specification when `metadata_type` is equal to a value [reserved](#) for future use or a user private value. Decoders should ignore the `metadata_unit()` if they do not understand the `metadata_type`. For `OBU_METADATA_SHORT`, this means ignoring the entire [OBU](#). For `OBU_METADATA_GROUP`, decoders should skip only the unrecognized `metadata_unit()` and continue processing other metadata units within the same [OBU](#).

§ 5.17.2. Metadata short OBU syntax

metadata_short_obu(obuPayloadSize) {	Descriptor
metadata_is_suffix	f(1)
metadata_necessity_idc = 0	
metadata_application_id = 0	
muh_layer_idc	f(3)
muh_cancel_flag	f(1)
muh_persistence_idc	f(3)
muh_priority = 0	
metadata_type	leb128()
if (muh_cancel_flag) {	
return	
}	
metadataPayloadSize = obuPayloadSize - 2 - Leb128Bytes	
metadata_unit(metadataPayloadSize)	
}	

§ 5.17.3. Metadata group OBU syntax

metadata_group_obu() {	Descriptor
metadata_is_suffix	f(1)
metadata_necessity_idc	f(2)

metadata_application_id	f(5)
metadata_unit_cnt_minus_1	leb128()
for (i = 0; i <= metadata_unit_cnt_minus_1; i++) {	
metadata_type	leb128()
muh_header_size	f(7)
muh_cancel_flag	f(1)
headerRemainingBytes = muh_header_size	
if (!muh_cancel_flag) {	
muh_payload_size	leb128()
headerRemainingBytes -= Leb128Bytes	
muh_layer_idc	f(3)
muh_persistence_idc	f(3)
muh_priority	f(8)
muh_reserved_zero_2bits	f(2)
headerRemainingBytes -= 2	
if (muh_layer_idc == LAYER_VALUES) {	
if (obu_xlayer_id == GLOBAL_XLAYER_ID) {	
muh_xlayer_map	f(32)
headerRemainingBytes -= 4	
for (n = 0; n < 31; n++) {	
if (muh_xlayer_map & (0x1 << n)) {	
muh_mlayer_map	f(8)
headerRemainingBytes -= 1	
}	
}	
} else {	
muh_mlayer_map	f(8)
headerRemainingBytes -= 1	
}	
}	
}	
for (j = 0; j < headerRemainingBytes; j++) {	
muh_header_extension_byte	f(8)
}	
if (!muh_cancel_flag) {	
metadata_unit(muh_payload_size)	
}	
}	

§ 5.17.4. Metadata ITUT T35 syntax

metadata_itut_t35(metadataPayloadSize) {	Descriptor
itu_t_t35_country_code	f(8)
t35PayloadSize = metadataPayloadSize - 1	
if (itu_t_t35_country_code == 0xFF) {	
itu_t_t35_country_code_extension_byte	f(8)
t35PayloadSize--	
}	
itu_t_t35_payload_bytes	le(t35PayloadSize)
}	

NOTE: The exact syntax of `itu_t_t35_payload_bytes` is not defined in this specification. External specifications can define the syntax. Decoders should ignore the entire [OBU](#) if they do not understand it.

§ 5.17.5. Metadata high dynamic range content light level syntax

metadata_hdr_cll() {	Descriptor
max_cll	f(16)
max_fall	f(16)
}	

§ 5.17.6. Metadata high dynamic range mastering display color volume syntax

metadata_hdr_mdcv() {	Descriptor
for (i = 0; i < 3; i++) {	
primary_chromaticity_x[i]	f(16)
primary_chromaticity_y[i]	f(16)
}	
white_point_chromaticity_x	f(16)
white_point_chromaticity_y	f(16)
luminance_max	f(32)
luminance_min	f(32)
}	

§ 5.17.7. Metadata timecode syntax

metadata_timecode() {	Descriptor
counting_type	f(5)
full_timestamp_flag	f(1)
discontinuity_flag	f(1)
cnt_dropped_flag	f(1)
n_frames	f(9)

if (full_timestamp_flag) {	
seconds_value	f(6)
minutes_value	f(6)
hours_value	f(5)
} else {	
seconds_flag	f(1)
if (seconds_flag) {	
seconds_value	f(6)
minutes_flag	f(1)
if (minutes_flag) {	
minutes_value	f(6)
hours_flag	f(1)
if (hours_flag) {	
hours_value	f(5)
}	
}	
}	
time_offset_length	f(5)
if (time_offset_length > 0) {	
time_offset_value	f(time_offset_length)
}	
}	

§ 5.17.8. Metadata banding hints syntax

metadata_banding_hints() {	Descriptor
coding_banding_present_flag	f(1)
source_banding_present_flag	f(1)
if (coding_banding_present_flag) {	
banding_hints_flag	f(1)
if (banding_hints_flag) {	
three_color_components_flag	f(1)
numComponents = three_color_components_flag ? 3 : 1	
for (plane = 0; plane < numComponents; plane++) {	
banding_in_component_present_flag	f(1)
if (banding_in_component_present_flag) {	
max_band_width_minus_4	f(6)
max_band_step_minus_1	f(4)
}	
}	
}	
band_units_information_present_flag	f(1)

§ 5.17.12. Metadata decoded frame hash syntax

metadata_decoded_frame_hash() {	Descriptor
hash_type	f(4)
per_plane	f(1)
has_grain	f(1)
is_monochrome	f(1)
reserved	f(1)
if (per_plane) {	
numPlanes = is_monochrome ? 1 : 3	
for (i = 0; i < numPlanes; i++) {	
plane_hash[i]	le(16)
}	
} else {	
frame_hash	le(16)
}	
}	

§ 5.17.13. Metadata user data unregistered syntax

metadata_user_data_unregistered(metadataPayloadSize) {	Descriptor
uuid_iso_iec_11578	f(128)
for(i = 16; i < metadataPayloadSize; i++) {	
user_data_payload_byte	f(8)
}	
}	

§ 5.18. Frame header syntax

§ 5.18.1. General frame header syntax

frame_header(isFirst) {	Descriptor
if (isFirst) {	
SeenFrameHeader = 1	
CountFrameHeaderForLevelConstraint = 1	
FrameSymbolCount = 0	
startBitPos = get_position()	
frame_header_info()	
NumFrameHeaderBits = get_position() - startBitPos	
FirstPictureInTU = 0	
if (IsBridge) {	
NumTiles = TileCols * TileRows	
tg_start = 0	
tg_end = NumTiles - 1	

tile_group_payload(0)	
} else if (ShowExistingFrame	
TipFrameMode == TIP_FRAME_AS_OUTPUT	
bru_inactive) {	
decode_frame_wrapup()	
SeenFrameHeader = 0	
CountFrameHeaderForLevelConstraint = 0	
} else {	
TileNum = 0	
}	
} else {	
CountFrameHeaderForLevelConstraint = 0	
frame_header_copy()	
}	
}	

where the syntax structure frame_header_copy is defined as:

frame_header_copy() {	Descriptor
for (i = 0; i < NumFrameHeaderBits; i++) {	
header_bit[i]	f(1)
}	
}	

§ 5.18.2. Frame header info syntax

frame_header_info() {	Descriptor
keyFrame = obu_type == OBU_CLOSED_LOOP_KEY obu_type == OBU_OPEN_LOOP_KEY	
IsRegular = (obu_type == OBU_OPEN_LOOP_KEY	
obu_type == OBU_REGULAR_TILE_GROUP	
obu_type == OBU_REGULAR_TIP	
obu_type == OBU_REGULAR_SEF	
obu_type == OBU_SWITCH	
obu_type == OBU_RAS_FRAME	
obu_type == OBU_BRIDGE_FRAME)	
for (i = 0; i < NUM_CUSTOM_QMS; i++) {	
QmSeen[i] = 0	
}	
startCVS = obu_type == OBU_CLOSED_LOOP_KEY && FirstPictureInTU	
if (startCVS) {	
olkEncountered = 0	
for(i = 0; i < MAX_NUM_MLAYERS; i++) {	
olkRefresh[i] = 0	

}	
flush_implicit_output_frames(0)	
}	
if (OlkEncountered && IsRegular && FirstPictureInTU) {	
flush_implicit_output_frames(1)	
OlkEncountered = 0	
allowedFrames = 0	
for (i = 0; i < MAX_NUM_MLAYERS; i++) {	
allowedFrames = OlkRefresh[i]	
OlkRefresh[i] = 0	
}	
for (i = 0; i < NUM_REF_FRAMES; i++) {	
if ((allowedFrames & (1 << i)) == 0 && RefLongTermId[i] == -1)	
RefValid[i] = 0	
}	
}	
IsBridge = obu_type == OBU_BRIDGE_FRAME	
if (IsBridge) {	
cur_mfh_id = 0	
} else {	
cur_mfh_id	uvlc()
}	
if (cur_mfh_id == 0) {	
seq_header_id_in_frame_header	uvlc()
load_sequence_header(seq_header_id_in_frame_header)	
mfh_deblocking_filter_update[cur_mfh_id] = 0	
} else {	
load_sequence_header(MfhSeqHeaderId[cur_mfh_id])	
}	
if (keyFrame) {	
if (seq_lcr_id != 0) {	
activate_layer_configuration_record(seq_lcr_id)	
}	
}	
if (cur_mfh_id == 0 !mfh_frame_size_present_flag[cur_mfh_id]) {	
mfh_frame_width_minus_1[cur_mfh_id] = max_frame_width_minus_1	
mfh_frame_height_minus_1[cur_mfh_id] = max_frame_height_minus_1	
}	
if (keyFrame && FirstPictureInTU) {	
reset_qm()	
}	

if (IsBridge) {	
n = CeilLog2(NumRefFrames)	
bridge_frame_ref_idx	f(n)
}	
allFrames = (1 << NumRefFrames) - 1	
use_bru = 0	
bru_inactive = 0	
if (single_picture_header_flag) {	
ShowExistingFrame = 0	
FrameType = KEY_FRAME	
FrameIsIntra = 1	
immediate_output_frame = 1	
implicit_output_frame = 0	
} else {	
ShowExistingFrame = is_sef()	
if (ShowExistingFrame == 1) {	
n = CeilLog2(NumRefFrames)	
frame_to_show_map_idx	f(n)
derive_sef_order_hint	f(1)
if (derive_sef_order_hint == 0) {	
sef_order_hint	f(OrderHintBits)
OrderHintLsbs = sef_order_hint	
OrderHint = get_disp_order_hint()	
} else {	
OrderHint = RefOrderHint[frame_to_show_map_idx]	
}	
if (IsRegular && OlkEncountered && !FirstPictureInTU) {	
OlkTUOrderHint = derive_sef_order_hint ?	
RefOrderHint[frame_to_show_map_idx] :	
OrderHint	
}	
refresh_frame_flags = 0	
FrameType = RefFrameType[frame_to_show_map_idx]	
immediate_output_frame = 1	
film_grain_config()	
if (derive_sef_order_hint) {	
save_grain_params(frame_to_show_map_idx)	
}	
TipFrameMode = TIP_FRAME_DISABLED	
return	
}	

if (IsBridge) {	
FrameType = INTER_FRAME	
} else if (obu_type == OBU_SWITCH obu_type == OBU_RAS_FRAME) {	
restricted_prediction_switch	f(1)
FrameType = SWITCH_FRAME	
} else if (is_tip_frame()) {	
FrameType = INTER_FRAME	
} else if (obu_type == OBU_CLOSED_LOOP_KEY	
obu_type == OBU_OPEN_LOOP_KEY) {	
FrameType = KEY_FRAME	
} else {	
frame_is_inter	f(1)
FrameType = frame_is_inter ? INTER_FRAME : INTRA_ONLY_FRAME	
}	
LongTermId = -1	
if (FrameType == KEY_FRAME) {	
long_term_id_plus_1	f(long_term_frame_id_bits)
LongTermId = long_term_id_plus_1 - 1	
}	
num_key_ref_frames = 0	
if ((obu_type == OBU_RAS_FRAME obu_type == OBU_OPEN_LOOP_KEY) &&	
long_term_frame_id_bits != 0) {	
num_key_ref_frames	f(3)
for (i = 0; i < num_key_ref_frames; i++) {	
ref_long_term_id[i]	f(long_term_frame_id_bits)
}	
}	
if (FrameType == SWITCH_FRAME && restricted_prediction_switch) {	
for (i = 0; i < NUM_REF_FRAMES; i++) {	
if (MLayerPresenceMap[RefMLayerId[i]][obu_mlayer_id]) {	
if (is_frame_eligible_for_output(i)) {	
output_frame_buffers(i)	
}	
RefOrderHint[i] = RESTRICTED_OH	
}	
}	
}	
if (obu_type == OBU_RAS_FRAME	
(obu_type == OBU_SWITCH && restricted_prediction_switch)) {	
reset_qm()	

}	
FrameIsIntra = (FrameType == INTRA_ONLY_FRAME	
FrameType == KEY_FRAME)	
if (IsBridge obu_type == OBU_OPEN_LOOP_KEY) {	
immediate_output_frame = 0	
} else {	
immediate_output_frame	f(1)
}	
if (IsBridge immediate_output_frame monotonic_output_order_flag) {	
implicit_output_frame = 0	
} else {	
implicit_output_frame	f(1)
}	
}	
if (use_256x256_superblock) {	
SbSize = FrameIsIntra ? BLOCK_128X128 : BLOCK_256X256	
} else if (use_128x128_superblock) {	
SbSize = BLOCK_128X128	
} else {	
SbSize = BLOCK_64X64	
}	
if (FrameType == KEY_FRAME && immediate_output_frame) {	
for (i = 0; i < REFS_PER_FRAME; i++) {	
OrderHints[i] = 0	
}	
}	
disable_cross_frame_cdf_init = 0	
if (IsBridge) {	
primary_ref_frame = PRIMARY_REF_NONE	
OrderHintLsbs = RefOrderHintLsbs[bridge_frame_ref_idx]	
OrderHint = RefOrderHint[bridge_frame_ref_idx]	
} else {	
if (FrameType == SWITCH_FRAME) {	
frame_size_override_flag = 1	
} else if (single_picture_header_flag) {	
frame_size_override_flag = 0	
} else {	
frame_size_override_flag	f(1)
}	
order_hint	f(OrderHintBits)
OrderHintLsbs = order_hint	

OrderHint = get_disp_order_hint()	
if (FrameIsIntra FrameType == SWITCH_FRAME) {	
primary_ref_frame = PRIMARY_REF_NONE	
} else {	
signal_primary_ref_frame	f(1)
if (!is_tip_frame()) {	
disable_cross_frame_cdf_init	f(1)
}	
if (signal_primary_ref_frame) {	
primary_ref_frame	f(3)
} else {	
primary_ref_frame = PRIMARY_REF_CHOOSE	
}	
}	
FrameMvPrecision = MV_PRECISION_ONE_PEL	
MvPrecision = FrameMvPrecision	
allow_high_precision_mv = 0	
use_ref_frame_mvs = 0	
allow_intrabc = 0	
allow_global_intrabc = 0	
allow_local_intrabc = 0	
allow_high_precision_mv = 0	
allow_df_sub_pu = 0	
if (IsBridge) {	
bridge_frame_overwrite_flag	f(1)
}	
if (FrameType == KEY_FRAME) {	
if (obu_type == OBU_CLOSED_LOOP_KEY && max_mlayer_id == 0) {	
refresh_frame_flags = allFrames	
} else if (enable_short_refresh_frame_flags) {	
n = CeilLog2(NumRefFrames)	
frame_to_refresh	f(n)
refresh_frame_flags = 1 << frame_to_refresh	
} else {	
refresh_frame_flags	f(NumRefFrames)
}	
if (obu_type == OBU_CLOSED_LOOP_KEY && FirstPictureInTU) {	
for (i = 0; i < NumRefFrames; i++) {	
RefValid[i] = 0	
}	

}	
if (obu_type == OBU_CLOSED_LOOP_KEY) {	
olkEncountered = 0	
for(i = 0; i < MAX_NUM_MLAYERS; i++) {	
olkRefresh[i] = 0	
}	
}	
if (obu_type == OBU_OPEN_LOOP_KEY) {	
olkEncountered = 1	
olkRefresh[obu_mlayer_id] = refresh_frame_flags	
if (implicit_output_frame) {	
olkTUOrderHint = OrderHint	
}	
}	
} else if (IsBridge && !bridge_frame_overwrite_flag) {	
refresh_frame_flags = 1 << bridge_frame_ref_idx	
} else if (obu_type == OBU_RAS_FRAME && max_mlayer_id == 0) {	
refresh_frame_flags = 0	
for (i = 0; i < NumRefFrames; i++) {	
if (!RefValid[i] !long_term_id_in_use(RefLongTermId[i])) {	
refresh_frame_flags = (1 << i)	
}	
}	
} else if (FrameType == SWITCH_FRAME) {	
refresh_frame_flags	f(NumRefFrames)
} else if (enable_short_refresh_frame_flags &&	
FrameType != SWITCH_FRAME &&	
FrameType != KEY_FRAME) {	
has_refresh_frame_flags	f(1)
if (has_refresh_frame_flags) {	
n = CeilLog2(NumRefFrames)	
frame_to_refresh	f(n)
refresh_frame_flags = 1 << frame_to_refresh	
} else {	
refresh_frame_flags = 0	
}	
} else {	
refresh_frame_flags	f(NumRefFrames)
}	
AllowedFrames = -1	
if (IsRegular && olkEncountered && !FirstPictureInTU) {	

AllowedFrames = 0	
for (i = 0; i < MAX_NUM_MLAYERS; i++) {	
AllowedFrames = OlkRefresh[i]	
}	
OlkRefresh[obu_mlayer_id] = refresh_frame_flags	
if (immediate_output_frame implicit_output_frame) {	
OlkTUOrderHint = OrderHint	
}	
}	
if (FrameIsIntra) {	
frame_size()	
screen_content_params()	
intrabc_params()	
NumTotalRefs = 0	
TipFrameMode = TIP_FRAME_DISABLED	
} else {	
if (FrameType == SWITCH_FRAME IsBridge) {	
explicitRefFrameMap = 1	
} else if (explicit_ref_frame_map) {	
frame_explicit_ref_frame_map	f(1)
explicitRefFrameMap = frame_explicit_ref_frame_map	
} else {	
explicitRefFrameMap = 0	
}	
if (IsBridge) {	
NumTotalRefs = 1	
} else if (explicitRefFrameMap) {	
num_total_refs	f(3)
NumTotalRefs = num_total_refs	
} else {	
get_ref_frames(0)	
}	
for (i = 0; i < NumTotalRefs; i++) {	
if (IsBridge) {	
ref_frame_idx[i] = bridge_frame_ref_idx	
} else if (explicitRefFrameMap) {	
n = CeilLog2(NumRefFrames)	
ref_frame_idx[i]	f(n)
}	
}	
if (IsBridge) {	

frame_size_with_bridge()	
} else if (frame_size_override_flag && FrameType != SWITCH_FRAME) {	
frame_size_with_refs()	
} else {	
frame_size()	
}	
if (!explicitRefFrameMap) {	
get_ref_frames(1)	
}	
NumSameRefCompound = Min(num_same_ref_compound, NumTotalRefs)	
if (enable_bru && FrameType == INTER_FRAME && !is_tip_frame() &&	
!IsBridge) {	
use_bru	f(1)
if (use_bru) {	
n = CeilLog2(NumTotalRefs)	
bru_ref	f(n)
bru_inactive	f(1)
}	
}	
if (explicitRefFrameMap) {	
for (i = 0; i < NumTotalRefs; i++) {	
ScoresDistance[i] = get_relative_dist(OrderHint,	
RefOrderHint[ref_frame_idx[i]])	
}	
}	
get_past_future_cur_ref_lists()	
if (FrameType == SWITCH_FRAME !enable_ref_frame_mvs	
IsBridge bru_inactive) {	
use_ref_frame_mvs = 0	
} else {	
use_ref_frame_mvs	f(1)
}	
if (use_ref_frame_mvs && NumTotalRefs > 1 && SbSize != BLOCK_64X64) {	
tmvp_sample_step_minus_1	f(1)
ProjStep = tmvp_sample_step_minus_1 + 1	
} else {	
ProjStep = 1	
}	
for (i = 0; i < NumTotalRefs; i++) {	
FrameDistance[i] = get_relative_dist(OrderHint,	
RefOrderHint[ref_frame_idx[i]])	

if (RefOrderHint[ref_frame_idx[i]] == RESTRICTED_OH) {	
FrameDistance[i] = -FrameDistance[i]	
}	
}	
for (i = 0; i < NumTotalRefs; i++) {	
refFrame = i	
hint = RefOrderHint[ref_frame_idx[i]]	
OrderHints[refFrame] = hint	
}	
if (enable_tip &&	
(use_ref_frame_mvs && NumTotalRefs >= 2) &&	
!bru_inactive) {	
TipInterpFilter = EIGHTTAP_SHARP	
TipGlobalMv[0] = 0	
TipGlobalMv[1] = 0	
if (EnableTipOutput && is_tip_frame()) {	
TipFrameMode = TIP_FRAME_AS_OUTPUT	
} else {	
tip_frame_mode	f(1)
TipFrameMode = tip_frame_mode	
}	
frame_opfl_refine_type()	
if (TipFrameMode != TIP_FRAME_DISABLED &&	
enable_tip_hole_fill) {	
allow_tip_hole_fill	f(1)
} else {	
allow_tip_hole_fill = 0	
}	
usesEqualWeight = enable_tip_refinemv &&	
NumFutureRefs > 0 && NumPastRefs > 0 &&	
(opfl_refine_type != REFINES_NONE enable_refinemv)	
if (TipFrameMode == TIP_FRAME_DISABLED usesEqualWeight) {	
tip_global_wtd_index = 0	
} else {	
tip_global_wtd_index	f(3)
}	
if (TipFrameMode == TIP_FRAME_AS_OUTPUT) {	
tip_mv_zero	f(1)
if (!tip_mv_zero) {	
tip_mv_row	f(4)
tip_mv_col	f(4)

if (tip_mv_row != 0) {	
tip_mv_row_sign	f(1)
TipGlobalMv[0] = tip_mv_row_sign ?	
-tip_mv_row : tip_mv_row	
}	
if (tip_mv_col != 0) {	
tip_mv_col_sign	f(1)
TipGlobalMv[1] = tip_mv_col_sign ?	
-tip_mv_col : tip_mv_col	
}	
}	
tip_sharp	f(1)
if (tip_sharp) {	
TipInterpFilter = EIGHTTAP_SHARP	
} else {	
tip_regular	f(1)
TipInterpFilter = tip_regular ? EIGHTTAP: EIGHTTAP_SMOOTH	
}	
}	
} else {	
TipFrameMode = TIP_FRAME_DISABLED	
if (!bru_inactive && !IsBridge) {	
frame_opfl_refine_type()	
}	
}	
if (TipFrameMode != TIP_FRAME_AS_OUTPUT && !bru_inactive &&	
!IsBridge) {	
screen_content_params()	
intrabc_params()	
max_drl_bits_minus_1 = seq_max_drl_bits_minus_1	
if (allow_frame_max_drl_bits) {	
change_drl	f(1)
if (change_drl) {	
n = MAX_REF_MV_STACK_SIZE - 2	
max_drl_bits_minus_1	ns(n)
if (max_drl_bits_minus_1 >= seq_max_drl_bits_minus_1) {	
max_drl_bits_minus_1 += 1	
}	
}	
}	
}	
if (force_integer_mv) {	

FrameMvPrecision = MV_PRECISION_ONE_PEL	
UsePerBlockMvPrecision = 0	
} else {	
use_qtr_precision_mv	f(1)
if (use_qtr_precision_mv) {	
FrameMvPrecision = MV_PRECISION_QUARTER_PEL	
} else {	
allow_high_precision_mv	f(1)
FrameMvPrecision = allow_high_precision_mv ?	
MV_PRECISION_EIGHTH_PEL : MV_PRECISION_HALF_PEL	
}	
UsePerBlockMvPrecision = enable_flex_mvres	
}	
MvPrecision = FrameMvPrecision	
read_interpolation_filter()	
for (mode = INTERINTRA; mode < MOTION_MODES; mode++) {	
if (!seq_frame_motion_modes_present_flag) {	
frame_enabled_motion_modes[mode] =	
seq_enabled_motion_modes[mode]	
} else if (seq_enabled_motion_modes[mode]) {	
frame_enabled_motion_modes[mode]	f(1)
} else {	
frame_enabled_motion_modes[mode] = 0	
}	
}	
}	
if (TipFrameMode == TIP_FRAME_AS_OUTPUT) {	
if (enable_tip_explicit_qp) {	
quantization_params()	
}	
if (enable_df_sub_pu) {	
allow_df_sub_pu	f(1)
}	
if (allow_df_sub_pu) {	
apply_deblocking_filter_tip	f(1)
} else {	
apply_deblocking_filter_tip = 0	
}	
}	
if (TipFrameMode == TIP_FRAME_AS_OUTPUT bru_inactive IsBridge) {	

for (i = 0 ; i < 3; i++) {	
frame_filters_on[i] = 0	
}	
if (bru_inactive IsBridge) {	
if (IsBridge) {	
tile_info()	
refIdx = bridge_frame_ref_idx	
} else {	
refIdx = ref_frame_idx[bru_ref]	
}	
base_q_idx = RefBaseQIdx[refIdx]	
DeltaQUAc = RefDeltaQUAc[refIdx]	
DeltaQVAc = RefDeltaQVAc[refIdx]	
set_primary_ref_frame_and_ctx(0)	
} else if (apply_deblocking_filter_tip) {	
tile_info()	
}	
film_grain_config()	
if (bru_inactive IsBridge) {	
set_primary_ref_frame_and_ctx(1)	
}	
for (row = 0; row < MiRows; row++) {	
for (col = 0; col < MiCols; col++) {	
SegmentIds[row][col] = 0	
}	
}	
for (ref = 0; ref < REFS_PER_FRAME; ref++) {	
for (i = 0; i < 6; i++) {	
gm_params[ref][i] = Default_Warp_Params[i]	
}	
}	
} else {	
disable_cdf_update	f(1)
}	
if (bru_inactive IsBridge) {	
apply_deblocking_filter[0] = 0	
apply_deblocking_filter[1] = 0	
cdef_frame_enable = 0	
for (plane = 0; plane < NumPlanes; plane++) {	
ccso_planes[plane] = 0	
}	

FrameRestorationType[0] = RESTORE_NONE	
FrameRestorationType[1] = RESTORE_NONE	
FrameRestorationType[2] = RESTORE_NONE	
gdf_frame_enable = 0	
segmentation_enabled = 0	
for (i = 0; i < MAX_SEGMENTS; i++) {	
for (j = 0; j < SEG_LVL_MAX; j++) {	
FeatureEnabled[i][j] = 0	
FeatureData[i][j] = 0	
}	
}	
if (primary_ref_frame == PRIMARY_REF_NONE	
disable_cross_frame_cdf_init) {	
init_coeff_cdfs()	
}	
return	
}	
if (use_ref_frame_mvs == 1) {	
HasBothRefs = ClosestFuture != NONE && ClosestPast != NONE	
motion_field_estimation()	
if (TipFrameMode == TIP_FRAME_AS_OUTPUT) {	
if (!enable_tip_explicit_qp) {	
slot0 = ref_frame_idx[ClosestPast]	
slot1 = ref_frame_idx[ClosestFuture]	
base_q_idx = Round2(RefBaseQIdx[slot0] + RefBaseQIdx[slot1], 1)	
DeltaQUAc = Round2(RefDeltaQUAc[slot0] + RefDeltaQUAc[slot1], 1)	
DeltaQVAc = Round2(RefDeltaQVAc[slot0] + RefDeltaQVAc[slot1], 1)	
}	
set_primary_ref_frame_and_ctx(1)	
for (i = 0; i < MAX_SEGMENTS; i++) {	
for (j = 0; j < SEG_LVL_MAX; j++) {	
FeatureData[i][j] = 0	
FeatureEnabled[i][j] = 0	
}	
}	
for (row = 0; row < MiRows; row++) {	
for (col = 0; col < MiCols; col++) {	
PrevSegmentIds[row][col] = 0	
}	
}	
for (plane = 0; plane < 3; plane++) {	

ccso_planes[plane] = 0	
}	
if (primary_ref_frame == PRIMARY_REF_NONE	
disable_cross_frame_cdf_init) {	
init_coeff_cdfs()	
}	
}	
if (TipFrameMode == TIP_FRAME_DISABLED) {	
fill_tpl_mvs_sample_gap()	
}	
}	
if (TipFrameMode != TIP_FRAME_DISABLED) {	
setup_tip_motion_field()	
}	
if (TipFrameMode == TIP_FRAME_AS_OUTPUT) {	
return	
}	
tile_info()	
quantization_params()	
set_primary_ref_frame_and_ctx(1)	
segmentation_params()	
setup_qm_params()	
delta_q_params()	
if (primary_ref_frame == PRIMARY_REF_NONE	
disable_cross_frame_cdf_init) {	
init_coeff_cdfs()	
}	
if (DerivedPrimaryRefFrame != PRIMARY_REF_NONE) {	
load_previous_segment_ids()	
}	
CodedLossless = 1	
HasLosslessSegment = 0	
for (segmentId = 0; segmentId < MaxSegments; segmentId++) {	
qindex = get_qindex(1, segmentId)	
LosslessArray[segmentId] = qindex == 0 && delta_q_present == 0 &&	
DeltaQYDc + BaseYDcDeltaQ <= 0 &&	
DeltaQUdC + BaseUVdCDeltaQ <= 0 &&	
DeltaQVDc + BaseUVdCDeltaQ <= 0 &&	
DeltaQUAc + BaseUVAcDeltaQ <= 0 &&	
DeltaQVAc + BaseUVAcDeltaQ <= 0	
if (LosslessArray[segmentId]) {	

HasLosslessSegment = 1	
} else {	
CodedLossless = 0	
}	
if (using_qmatrix) {	
if (LosslessArray[segmentId]) {	
SegQMLLevel[0][segmentId] = 15	
SegQMLLevel[1][segmentId] = 15	
SegQMLLevel[2][segmentId] = 15	
} else {	
qmNum = pic_qm_num_minus_1 + 1	
qmIndexBits = CeilLog2(qmNum)	
qm_index	f(qmIndexBits)
SegQMLLevel[0][segmentId] = qm_y[qm_index]	
SegQMLLevel[1][segmentId] = qm_u[qm_index]	
SegQMLLevel[2][segmentId] = qm_v[qm_index]	
}	
}	
}	
if (CodedLossless) {	
allow_tcq = 0	
} else if (choose_tcq_per_frame) {	
allow_tcq	f(1)
} else {	
allow_tcq = enable_tcq	
}	
if (CodedLossless !enable_parity_hiding allow_tcq) {	
allow_parity_hiding = 0	
} else {	
allow_parity_hiding	f(1)
}	
deblocking_filter_params()	
gdf_params()	
cdef_params()	
lr_params()	
ccso_params()	
read_tx_mode()	
frame_reference_mode()	
skip_mode_params()	
if (!FrameIsIntra && enable_bawp) {	
allow_bawp	f(1)

} else {	
allow_bawp = 0	
}	
if (!FrameIsIntra && frame_enabled_motion_modes[DELTAWARP]) {	
allow_warpmv_mode	f(1)
} else {	
allow_warpmv_mode = 0	
}	
reduced_tx_set	f(2)
global_motion_params()	
film_grain_config()	
}	

where the function `reset_qm` is defined as:

```
reset_qm() {
    for ( level = 0; level < 15; level++ ) {
        if ( obu_type == OBU_SWITCH || obu_type == OBU_RAS_FRAME ) {
            needsReset = QmMLayerId[ level ] == -1 ||
                MLayerPresenceMap[QmMLayerId[ level ]][obu_mlayer_id]
        } else {
            needsReset = 1
        }
        if ( !QmProtected[ level ] && needsReset ) {
            QmDataPresent[ level ] = 0
            QmNumPlanes[ level ] = NumPlanes
            QmMLayerId[ level ] = -1
            QmTLayerId[ level ] = -1
        }
    }
}
```

where the function `get_disp_order_hint` is defined as:

```
get_disp_order_hint( ) {
    if ( obu_type == OBU_CLOSED_LOOP_KEY ||
        ( !is_sef() && FrameType == SWITCH_FRAME &&
          restricted_prediction_switch ) ) {
        return OrderHintLsbs
    }
    maxDisp = get_max_disp_order_hint( 1 )
    dispOrderHint = OrderHintLsbs
    offset = maxDisp - ((1 << OrderHintBits) >> 1) - OrderHintLsbs
    if ( offset >= 0 ) {
        dispOrderHint += ((offset >> OrderHintBits) + 1) << OrderHintBits
    }
    return dispOrderHint
}
```

where `get_max_disp_order_hint` (which returns the maximum order hint from certain frames) is defined as:

```
get_max_disp_order_hint( onlyShowable ) {
    maxDisp = 0
    for ( i = 0; i < NumRefFrames; i++ ) {
        if ( RefValid[i] &&
```

```

        TLayerDependencyMap[obu_mlayer_id][obu_tlayer_id][RefTLayerId[i]] &&
        MLayerDependencyMap[obu_mlayer_id][RefMLayerId[i]] &&
        ( !onlyShowable || RefImplicitOutputFrame[ i ] ||
          RefImmediateOutputFrame[ i ] ) ) {
            maxDisp = Max( maxDisp, RefOrderHint[i] )
        }
    }
    return maxDisp
}

```

It is a requirement of bitstream conformance that the value returned from `get_disp_order_hint` is less than $(1 \ll (\text{DISPLAY_ORDER_HINT_BITS} - 1))$.

The function `set_primary_ref_frame_and_ctx` is defined as:

```

set_primary_ref_frame_and_ctx( loadCdfs ) {
    (DerivedPrimaryRefFrame, derivedSecondaryRefFrame) =
        choose_primary_secondary_ref_frame()
    if ( primary_ref_frame == PRIMARY_REF_CHOOSE ) {
        primary_ref_frame = DerivedPrimaryRefFrame
    }
    if ( DerivedPrimaryRefFrame == PRIMARY_REF_NONE ||
        primary_ref_frame == PRIMARY_REF_NONE ) {
        primary_ref_frame = PRIMARY_REF_NONE
        DerivedPrimaryRefFrame = PRIMARY_REF_NONE
        disable_cross_frame_cdf_init = 1
    }
    if ( !loadCdfs ) {
        return
    }
    if ( primary_ref_frame == PRIMARY_REF_NONE ||
        disable_cross_frame_cdf_init ) {
        init_non_coeff_cdfs()
    } else {
        load_cdfs( ref_frame_idx[ primary_ref_frame ] )
        if ( TipFrameMode != TIP_FRAME_AS_OUTPUT ) {
            blendFrame = (primary_ref_frame == DerivedPrimaryRefFrame) ?
                derivedSecondaryRefFrame : DerivedPrimaryRefFrame
            if ( enable_avg_cdf && !avg_cdf_type &&
                blendFrame != PRIMARY_REF_NONE &&
                !bru_inactive ) {
                blend_cdfs( ref_frame_idx[ blendFrame ] )
            }
        }
    }
    if ( DerivedPrimaryRefFrame == PRIMARY_REF_NONE ) {
        setup_past_independence()
    } else {
        load_previous()
    }
}

```

The functions `choose_primary_secondary_ref_frame` and `is_ref_better` are defined as:

```

choose_primary_secondary_ref_frame() {
    if ( FrameIsIntra || FrameType == SWITCH_FRAME ) {
        return ( PRIMARY_REF_NONE, PRIMARY_REF_NONE )
    }
    primary = PRIMARY_REF_NONE
    primaryQpDiff = 512
    secondary = PRIMARY_REF_NONE
    secondaryQpDiff = 512
    primaryD = 0
    secondaryD = 0
    primaryRatio = 0
}

```

```

secondaryRatio = 0
for ( i = 0; i < NumTotalRefs; i++ ) {
    idx = ref_frame_idx[ i ]
    if ( RefFrameType[ idx ] == INTER_FRAME && first_slot_with_ref(idx) &&
        RefOrderHint[ idx ] != RESTRICTED_OH ) {
        q = RefBaseQIdx[ idx ]
        d = RefOrderHint[ idx ]
        dRatio = FloorLog2( RefFrameWidth[ idx ] * RefFrameHeight[ idx ] )
        qpDiff = Abs(q - base_q_idx)
        if ( (qpDiff < primaryQpDiff) ||
            (qpDiff == primaryQpDiff &&
              is_ref_better(d,primaryD,dRatio,primaryRatio)) ) {
            secondary = primary
            secondaryQpDiff = primaryQpDiff
            secondaryD = primaryD
            secondaryRatio = primaryRatio
            primary = i
            primaryQpDiff = qpDiff
            primaryD = d
            primaryRatio = dRatio
        } else if ((qpDiff < secondaryQpDiff) ||
                    (qpDiff == secondaryQpDiff &&
                      is_ref_better(d,secondaryD,dRatio,secondaryRatio))) {
            secondary = i
            secondaryQpDiff = qpDiff
            secondaryD = d
            secondaryRatio = dRatio
        }
    }
}
if ( signal_primary_ref_frame ) {
    if ( primary_ref_frame == PRIMARY_REF_NONE ) {
        primary = PRIMARY_REF_NONE
        secondary = PRIMARY_REF_NONE
    } else if ( primary_ref_frame != primary ) {
        if ( secondary == PRIMARY_REF_NONE ||
            secondary == primary_ref_frame ) {
            secondary = primary
        }
        primary = primary_ref_frame
    }
}
return (primary,secondary)
}

is_ref_better(refDisp, bestDisp, refRatio, bestRatio) {
    d0 = Abs(get_relative_dist(OrderHint,refDisp)) - (refRatio << 1)
    d1 = Abs(get_relative_dist(OrderHint,bestDisp)) - (bestRatio << 1)
    if (d0 < d1) {
        return 1
    }
    if (d0 == d1 && get_relative_dist(refDisp,bestDisp) > 0) {
        return 1
    }
    return 0
}

```

The function `long_term_id_in_use` (which determines if `longTermId` is present in the `ref_long_term_id` array) is defined as:

```

long_term_id_in_use( longTermId ) {
    for ( j = 0; j < num_key_ref_frames; j++ ) {
        if ( longTermId == ref_long_term_id[ j ] ) {
            return 1
        }
    }
}

```



```

    }
    return 0
}

```

§ 5.18.3. Frame configuration structures

§ 5.18.3.1. *Get relative distance function*

This function computes the distance between two order hints by sign extending the result of subtracting the values.

```

get_relative_dist( a, b ) {
    if ( a == RESTRICTED_OH && b == RESTRICTED_OH ) {
        return 0
    } else if ( a == RESTRICTED_OH ) {
        return 127
    } else if ( b == RESTRICTED_OH ) {
        return -127
    } else {
        return Clip3( -127, 127, a - b )
    }
}

```

§ 5.18.3.2. *Frame optical flow refine type syntax*

frame_opfl_refine_type() {	Descriptor
if (TipFrameMode == TIP_FRAME_AS_OUTPUT) {	
opfl_refine_type = (!enable_tip_refinemv	
enable_opfl_refine == REFINES_NONE) ?	
REFINES_NONE : REFINES_ALL	
} else if (enable_opfl_refine == REFINES_AUTO) {	
opfl_refine_type	f(1)
if (opfl_refine_type != REFINES_SWITCHABLE) {	
opfl_refine_all	f(1)
opfl_refine_type = opfl_refine_all ? REFINES_ALL : REFINES_NONE	
}	
} else {	
opfl_refine_type = enable_opfl_refine	
}	

§ 5.18.3.3. *Screen content params syntax*

screen_content_params() {	Descriptor
if (seq_force_screen_content_tools == SELECT_SCREEN_CONTENT_TOOLS) {	
allow_screen_content_tools	f(1)
} else {	
allow_screen_content_tools = seq_force_screen_content_tools	
}	

if (allow_screen_content_tools) {	
if (seq_force_integer_mv == SELECT_INTEGER_MV) {	
force_integer_mv	f(1)
} else {	
force_integer_mv = seq_force_integer_mv	
}	
} else {	
force_integer_mv = 0	
}	
}	

§ 5.18.3.4. Intra block copy params syntax

intrabc_params() {	Descriptor
allow_intrabc	f(1)
if (allow_intrabc) {	
if (FrameIsIntra) {	
allow_global_intrabc	f(1)
if (allow_global_intrabc) {	
allow_local_intrabc	f(1)
} else {	
allow_local_intrabc = 1	
}	
} else {	
allow_global_intrabc = 0	
allow_local_intrabc = 1	
}	
max_bvp_drl_bits_minus_1 = seq_max_bvp_drl_bits_minus_1	
if (allow_frame_max_bvp_drl_bits) {	
change_bvp_drl	f(1)
if (change_bvp_drl) {	
max_bvp_drl_bits_minus_1	ns(2)
if (max_bvp_drl_bits_minus_1 >=	
seq_max_bvp_drl_bits_minus_1) {	
max_bvp_drl_bits_minus_1 += 1	
}	
}	
}	
}	

§ 5.18.4. Frame size structures

§ 5.18.4.1. Frame size syntax

frame_size() {	Descriptor
if (frame_size_override_flag) {	
n = frame_width_bits_minus_1 + 1	
frame_width_minus_1	f(n)
n = frame_height_bits_minus_1 + 1	
frame_height_minus_1	f(n)
FrameWidth = frame_width_minus_1 + 1	
FrameHeight = frame_height_minus_1 + 1	
} else {	
FrameWidth = mfh_frame_width_minus_1[cur_mfh_id] + 1	
FrameHeight = mfh_frame_height_minus_1[cur_mfh_id] + 1	
}	
compute_image_size()	
}	

§ 5.18.4.2. Frame size with bridge syntax

frame_size_with_bridge() {	Descriptor
n = frame_width_bits_minus_1 + 1	
bridge_frame_width_minus_1	f(n)
n = frame_height_bits_minus_1 + 1	
bridge_frame_height_minus_1	f(n)
FrameWidth = Min(RefFrameWidth[bridge_frame_ref_idx],	
bridge_frame_width_minus_1 + 1)	
FrameHeight = Min(RefFrameHeight[bridge_frame_ref_idx],	
bridge_frame_height_minus_1 + 1)	
compute_image_size()	
}	

§ 5.18.4.3. Frame size with refs syntax

frame_size_with_refs() {	Descriptor
for (i = 0; i < NumTotalRefs; i++) {	
found_ref	f(1)
if (found_ref == 1) {	
FrameWidth = RefFrameWidth[ref_frame_idx[i]]	
FrameHeight = RefFrameHeight[ref_frame_idx[i]]	
break	
}	
}	

if (NumTotalRefs == 0 found_ref == 0) {	
frame_size()	
} else {	
compute_image_size()	
}	
}	

§ 5.18.4.4. Compute image size function

```
compute_image_size( ) {
    MiCols = 2 * ( ( FrameWidth + 7 ) >> 3 )
    MiRows = 2 * ( ( FrameHeight + 7 ) >> 3 )
    maxFrameWidth = max_frame_width_minus_1 + 1
    maxFrameHeight = max_frame_height_minus_1 + 1
    CropLeft = (seq_cropping_win_left_offset * FrameWidth) / maxFrameWidth
    cropRight = FrameWidth - ((seq_cropping_win_right_offset * FrameWidth) /
                               maxFrameWidth)
    CropTop = (seq_cropping_win_top_offset * FrameHeight) / maxFrameHeight
    cropBottom = FrameHeight - ((seq_cropping_win_bottom_offset * FrameHeight) /
                                 maxFrameHeight)
    CropWidth = cropRight - CropLeft
    CropHeight = cropBottom - CropTop
}
```

§ 5.18.5. Filtering structures

§ 5.18.5.1. Interpolation filter syntax

read_interpolation_filter() {	Descriptor
is_filter_switchable	f(1)
if (is_filter_switchable == 1) {	
interpolation_filter = SWITCHABLE	
} else {	
interpolation_filter	f(2)
}	
}	

§ 5.18.5.2. Deblocking filter params syntax

deblocking_filter_params() {	Descriptor
if (CodedLossless) {	
apply_deblocking_filter[0] = 0	
apply_deblocking_filter[1] = 0	
return	
}	
if (enable_df_sub_pu && FrameType == INTER_FRAME) {	
allow_df_sub_pu	f(1)
} else {	
allow_df_sub_pu = 0	

}	
if (mfh_deblocking_filter_update[cur_mfh_id]) {	
apply_deblocking_filter[0] = mfh_apply_deblocking_filter[cur_mfh_id][0]	
apply_deblocking_filter[1] = mfh_apply_deblocking_filter[cur_mfh_id][1]	
apply_deblocking_filter[2] = 0	
apply_deblocking_filter[3] = 0	
if (NumPlanes > 1) {	
if (apply_deblocking_filter[0] apply_deblocking_filter[1]) {	
apply_deblocking_filter[2] = mfh_apply_deblocking_filter[cur_mfh_id][2]	
apply_deblocking_filter[3] = mfh_apply_deblocking_filter[cur_mfh_id][3]	
}	
}	
} else {	
apply_deblocking_filter[0]	f(1)
apply_deblocking_filter[1]	f(1)
apply_deblocking_filter[2] = 0	
apply_deblocking_filter[3] = 0	
if (NumPlanes > 1) {	
if (apply_deblocking_filter[0] apply_deblocking_filter[1]) {	
apply_deblocking_filter[2]	f(1)
apply_deblocking_filter[3]	f(1)
}	
}	
}	
for (i = 0; i < 4; i++) {	
if (apply_deblocking_filter[i]) {	
df_delta_q_present[i]	f(1)
if (df_delta_q_present[i]) {	
dfParBits = df_par_bits_minus_2 + 2	
df_delta_q[i]	f(dfParBits)
DfDeltaQ[i] = df_delta_q[i] - (1 << (dfParBits - 1))	
} else {	
DfDeltaQ[i] = (i == 1) ? DfDeltaQ[0] : 0	
}	
} else {	
DfDeltaQ[i] = 0	
}	
}	
}	

§ 5.18.6. Quantization structures

§ 5.18.6.1. Quantization params syntax

quantization_params() {	Descriptor
n = BitDepth == 8 ? 8 : 9	
base_q_idx	f(n)
DeltaQYDc = 0	
DeltaQUdc = 0	
DeltaQUAc = 0	
DeltaQVDc = 0	
DeltaQVAc = 0	
if (TipFrameMode != TIP_FRAME_AS_OUTPUT && y_dc_delta_q_enabled) {	
DeltaQYDc = read_delta_q()	
}	
if (NumPlanes > 1 && (	
uv_ac_delta_q_enabled	
(TipFrameMode != TIP_FRAME_AS_OUTPUT && uv_dc_delta_q_enabled)	
)) {	
if (separate_uv_delta_q) {	
diff_uv_delta	f(1)
} else {	
diff_uv_delta = 0	
}	
if (TipFrameMode != TIP_FRAME_AS_OUTPUT && uv_dc_delta_q_enabled) {	
DeltaQUdc = read_delta_q()	
}	
if (uv_ac_delta_q_enabled) {	
DeltaQUAc = read_delta_q()	
}	
if (equal_ac_dc_q) {	
DeltaQUdc = DeltaQUAc	
}	
if (diff_uv_delta) {	
if (TipFrameMode != TIP_FRAME_AS_OUTPUT &&	
uv_dc_delta_q_enabled) {	
DeltaQVDc = read_delta_q()	
}	
if (uv_ac_delta_q_enabled) {	
DeltaQVAc = read_delta_q()	
}	
if (equal_ac_dc_q) {	

DeltaQVDc = DeltaQVAc	
}	
} else {	
DeltaQVDc = DeltaQUdc	
DeltaQVAc = DeltaQUAc	
}	
}	
}	

§ 5.18.6.2. Setup QM params syntax

setup_qm_params() {	Descriptor
using_qmatrix	f(1)
if (using_qmatrix) {	
if (segmentation_enabled) {	
pic_qm_num_minus_1	f(2)
} else {	
pic_qm_num_minus_1 = 0	
}	
qmNum = pic_qm_num_minus_1 + 1	
for (i = 0; i < qmNum; i++) {	
qm_y[i]	f(4)
if (NumPlanes > 1) {	
qm_uv_same_as_y	f(1)
if (qm_uv_same_as_y) {	
qm_u[i] = qm_y [i]	
qm_v[i] = qm_y [i]	
} else {	
qm_u[i]	f(4)
if (!separate_uv_delta_q) {	
qm_v[i] = qm_u[i]	
} else {	
qm_v[i]	f(4)
}	
}	
}	
}	
}	

§ 5.18.6.3. Delta quantizer syntax

read_delta_q() {	Descriptor
-------------------	------------

delta_coded	f(1)
if (delta_coded) {	
delta_q	su(7)
} else {	
delta_q = 0	
}	
return delta_q	
}	

§ 5.18.7. Segmentation and tiling structures

§ 5.18.7.1. Segmentation params syntax

segmentation_params() {	Descriptor
segmentation_enabled	f(1)
if (segmentation_enabled == 1) {	
if (cur_mfh_id > 0 && mfh_seg_info_present_flag[cur_mfh_id]) {	
haveSegParams = mfh_ext_seg_flag[cur_mfh_id] == enable_ext_seg	
allowChange = haveSegParams && mfh_allow_seg_info_change[cur_mfh_id]	
mfhId = cur_mfh_id	
} else if (seq_seg_info_present_flag) {	
haveSegParams = 1	
allowChange = seq_allow_seg_info_change	
mfhId = 0	
} else {	
haveSegParams = 0	
allowChange = 0	
}	
if (allowChange) {	
reuse_seg_info	f(1)
} else {	
reuse_seg_info = haveSegParams	
}	
if (reuse_seg_info) {	
for (i = 0; i < MAX_SEGMENTS; i++) {	
for (j = 0; j < SEG_LVL_MAX; j++) {	
if (mfhId == 0) {	
FeatureData[i][j] = SeqFeatureData[i][j]	
FeatureEnabled[i][j] = SeqFeatureEnabled[i][j]	
} else {	
FeatureData[i][j] =	
MfhFeatureData[mfhId][i][j]	
FeatureEnabled[i][j] =	

MfhFeatureEnabled[mfhId][i][j]	
}	
}	
}	
} else {	
(FeatureEnabled, FeatureData) = seg_info(MaxSegments)	
}	
if (DerivedPrimaryRefFrame == PRIMARY_REF_NONE) {	
segmentation_update_map = 1	
segmentation_temporal_update = 0	
} else {	
segmentation_update_map	f(1)
if (segmentation_update_map == 1 && FrameType != SWITCH_FRAME) {	
segmentation_temporal_update	f(1)
} else {	
segmentation_temporal_update = 0	
}	
}	
} else {	
for (i = 0; i < MAX_SEGMENTS; i++) {	
for (j = 0; j < SEG_LVL_MAX; j++) {	
FeatureEnabled[i][j] = 0	
FeatureData[i][j] = 0	
}	
}	
}	
SegIdPreSkip = 0	
LastActiveSegId = 0	
for (i = 0; i < MaxSegments; i++) {	
for (j = 0; j < SEG_LVL_MAX; j++) {	
if (FeatureEnabled[i][j]) {	
LastActiveSegId = i	
if (j >= SEG_LVL_SKIP) {	
SegIdPreSkip = 1	
}	
}	
}	
}	
}	

The constant lookup tables used in this syntax are defined as:

```
Segmentation_Feature_Bits[ SEG_LVL_MAX ] = { 9, 0, 0 }
Segmentation_Feature_Signed[ SEG_LVL_MAX ] = { 1, 0, 0 }
Segmentation_Feature_Max[ SEG_LVL_MAX ] = { MAXQ_BITS, 0, 0 }
```

5.18.7.2. Tile info syntax

tile_info () {	Descriptor
sb4x4 = Num_4x4_Blocks_Wide[SbSize]	
sbShift = Mi_Width_Log2[SbSize]	
sbCols = (MiCols + sb4x4 - 1) >> sbShift	
sbRows = (MiRows + sb4x4 - 1) >> sbShift	
if (IsBridge) {	
haveTileParams = 0	
} else {	
haveTileParams = seq_tile_info_present_flag	
}	
if (haveTileParams &&	
(SeqUniformTileSpacingFlag ? (	
uniform_eligible(SeqTileRowsLog2, sbRows) &&	
uniform_eligible(SeqTileColsLog2, sbCols)) :	
(SeqSbCols == sbCols && SeqSbRows == sbRows)) {	
if (allow_tile_info_change) {	
reuse_tile_info	f(1)
} else {	
reuse_tile_info = 1	
}	
} else {	
reuse_tile_info = 0	
}	
seqSbSize = get_seq_sb_size()	
if (reuse_tile_info) {	
(sbRowStarts, TileRows, TileRowsLog2, sbColStarts, TileCols,	
TileColsLog2, sbShift2) = reuse_tile_params(SeqUniformTileSpacingFlag,	
SeqSbRowStarts, SeqTileRows, SeqTileRowsLog2,	
SeqSbColStarts, SeqTileCols, SeqTileColsLog2, seqSbSize, SbSize)	
} else {	
(sbRowStarts, sbRows, TileRows, TileRowsLog2, sbColStarts, sbCols,	
TileCols, TileColsLog2, uniformSpacing, sbShift2) = tile_params(	
FrameWidth, FrameHeight, seqSbSize, SbSize, IsBridge)	
}	
for (i = 0; i < TileCols; i++) {	

MiColStarts[i] = sbColStarts[i] << sbShift2	
}	
for (i = 0; i < TileRows; i++) {	
MiRowStarts[i] = sbRowStarts[i] << sbShift2	
}	
MiColStarts[TileCols] = MiCols	
MiRowStarts[TileRows] = MiRows	
if ((TileCols > 1 TileRows > 1) && !IsBridge &&	
TipFrameMode != TIP_FRAME_AS_OUTPUT) {	
if (!enable_avg_cdf !avg_cdf_type) {	
n = TileRowsLog2 + TileColsLog2	
context_update_tile_id	f(n)
}	
tile_size_bytes_minus_1	f(2)
TileSizeBytes = tile_size_bytes_minus_1 + 1	
} else {	
context_update_tile_id = 0	
}	
}	

where `uniform_eligible` is specified as:

```
uniform_eligible( tileLog2, sbNum ) {
    tileNum = 1 << tileLog2
    tileWidth = (sbNum + tileNum - 1) >> tileLog2
    lastTileWidth = sbNum - (tileNum - 1) * tileWidth
    return tileWidth >= 1 && lastTileWidth >= 1
}
```

§ 5.18.7.3. Tile params syntax

tile_params(frameWidth, frameHeight, uniformSbSize, sbSize, isBridge) {	Descriptor
miCols = 2 * ((frameWidth + 7) >> 3)	
miRows = 2 * ((frameHeight + 7) >> 3)	
sb4x4 = Num_4x4_Blocks_Wide[sbSize]	
sbShift = Mi_Width_Log2[sbSize]	
sbCols = (miCols + sb4x4 - 1) >> sbShift	
sbRows = (miRows + sb4x4 - 1) >> sbShift	
if (seq_level_idx != 31) {	
maxTileWidthSb = (Tile_Width_Scaling_Factor[seq_tier][seq_level_idx] * MAX_TILE_WIDTH) >> (sbShift + 4)	
maxTileAreaSb = (Tile_Area_Scaling_Factor[seq_tier][seq_level_idx] * MAX_TILE_AREA) >> (2 * (sbShift + 2) + 2)	
} else {	

maxTileWidthSb = sbCols	
maxTileAreaSb = sbCols * sbRows	
}	
minLog2TileCols = tile_log2(maxTileWidthSb, sbCols)	
maxLog2TileCols = tile_log2(1, Min(sbCols, MAX_TILE_COLS))	
maxLog2TileRows = tile_log2(1, Min(sbRows, MAX_TILE_ROWS))	
minLog2Tiles = Max(minLog2TileCols,	
tile_log2(maxTileAreaSb, sbRows * sbCols))	
if (isBridge) {	
uniform_tile_spacing_flag = 1	
} else {	
uniform_tile_spacing_flag	f(1)
}	
if (uniform_tile_spacing_flag) {	
sbShift = Mi_Width_Log2[uniformSbSize]	
tileColsLog2 = minLog2TileCols	
if (!isBridge) {	
while (tileColsLog2 < maxLog2TileCols) {	
increment_tile_cols_log2	f(1)
if (increment_tile_cols_log2 == 1) {	
tileColsLog2 += 1	
} else {	
break	
}	
}	
}	
(sbColStarts, tileCols) = uniform_spacing(tileColsLog2, miCols,	
uniformSbSize)	
tileColsLog2 = tile_log2(1, tileCols)	
minLog2TileRows = Max(minLog2Tiles - tileColsLog2, 0)	
tileRowsLog2 = minLog2TileRows	
if (!isBridge) {	
while (tileRowsLog2 < maxLog2TileRows) {	
increment_tile_rows_log2	f(1)
if (increment_tile_rows_log2 == 1) {	
tileRowsLog2++	
} else {	
break	
}	
}	
}	

(sbRowStarts, tileRows) = uniform_spacing(tileRowsLog2, miRows,	
uniformSbSize)	
} else {	
widestTileSb = 1	
startSb = 0	
for (i = 0; startSb < sbCols; i++) {	
sbColStarts[i] = startSb	
n = Min(sbCols - startSb, maxTileWidthSb)	
width_in_sbs_minus_1	ns(n)
sizeSb = width_in_sbs_minus_1 + 1	
widestTileSb = Max(sizeSb, widestTileSb)	
startSb += sizeSb	
}	
tileCols = i	
tileColsLog2 = tile_log2(1, tileCols)	
if (minLog2Tiles > 0) {	
maxTileAreaSb = (sbRows * sbCols) >> (minLog2Tiles + 1)	
} else {	
maxTileAreaSb = sbRows * sbCols	
}	
maxTileHeightSb = Max(maxTileAreaSb / widestTileSb, 1)	
startSb = 0	
for (i = 0; startSb < sbRows; i++) {	
sbRowStarts[i] = startSb	
maxHeight = Min(sbRows - startSb, maxTileHeightSb)	
height_in_sbs_minus_1	ns(maxHeight)
sizeSb = height_in_sbs_minus_1 + 1	
startSb = startSb + sizeSb	
}	
tileRows = i	
}	
tileRowsLog2 = tile_log2(1, tileRows)	
return (sbRowStarts, sbRows, tileRows, tileRowsLog2, sbColStarts, sbCols,	
tileCols, tileColsLog2, uniform_tile_spacing_flag, sbShift)	
}	

[§](#) 5.18.7.4. Reuse tile params function

```

reuse_tile_params( uniformSpacing, sbRowStarts, tileRows, tileRowsLog2, sbColStarts, tileCols,
tileColsLog2, seqSbSize, sbSize ) {
  if ( uniformSpacing ) {
    sbShift = Mi_Width_Log2[ seqSbSize ]
    (unifSbColStarts, tileCols) = uniform_spacing( tileColsLog2, MiCols,
                                                    seqSbSize )
    (unifSbRowStarts, tileRows) = uniform_spacing( tileRowsLog2, MiRows,

```

```

seqSbSize )
    tileColsLog2 = tile_log2(1, tileCols)
    tileRowsLog2 = tile_log2(1, tileRows)
    return ( unifSbRowStarts, tileRows, tileRowsLog2, unifSbColStarts,
             tileCols, tileColsLog2, sbShift)
} else {
    sbShift = Mi_Width_Log2[ sbSize ]
    tileColsLog2 = tile_log2(1, tileCols)
    tileRowsLog2 = tile_log2(1, tileRows)
    return ( sbRowStarts, tileRows, tileRowsLog2, sbColStarts, tileCols,
             tileColsLog2, sbShift)
}
}

```

§ 5.18.7.5. Uniform spacing function

```

uniform_spacing( tileLog2, mis, sbSize ) {
    sb4x4 = Num_4x4_Blocks_Wide[ sbSize ]
    sbShift = Mi_Width_Log2[ sbSize ]
    sbs = ( mis + sb4x4 - 1 ) >> sbShift
    fullSbs = mis >> sbShift
    tileSb = fullSbs >> tileLog2
    if ( tileSb == 0 ) {
        extraSbs = sbs
    } else {
        extraSbs = fullSbs - (tileSb << tileLog2)
    }
    startSb = 0
    for ( i = 0; i < (1 << tileLog2) && startSb < sbs; i++) {
        sbStarts[ i ] = startSb
        startSb += tileSb
        if ( i < extraSbs ) {
            startSb += 1
        }
    }
    return (sbStarts, i)
}

```

§ 5.18.7.6. Get sequence superblock size function

```

get_seq_sb_size() {
    if ( use_256x256_superblock ) {
        return BLOCK_256X256
    } else if ( use_128x128_superblock ) {
        return BLOCK_128X128
    } else {
        return BLOCK_64X64
    }
}

```

§ 5.18.7.7. Tile size calculation function

tile_log2 returns the smallest value for k such that blkSize << k is greater than or equal to target.

```

tile_log2( blkSize, target ) {
    for ( k = 0; (blkSize << k) < target; k++ ) {
    }
    return k
}

```

§ 5.18.7.8. Quantizer index delta parameters syntax

delta_q_params() {	Descriptor
delta_q_res = 0	
delta_q_present = 0	
if (base_q_idx > 0) {	
delta_q_present	f(1)
}	
if (delta_q_present) {	
delta_q_res	f(2)
}	
}	

§ 5.18.7.9. GDF params syntax

gdf_params() {	Descriptor
if (CodedLossless !enable_gdf) {	
gdf_frame_enable = 0	
} else {	
if (single_picture_header_flag) {	
gdf_frame_enable = 1	
} else {	
gdf_frame_enable	f(1)
}	
if (!gdf_frame_enable) {	
return	
}	
gdfBlkSize = Max(Block_Width[SbSize],GDF_MIN_SIZE)	
if (gdf_unit_matches_sb_size) {	
gdfBlkSize = Block_Width[SbSize]	
} else if (SbSize == BLOCK_64X64) {	
a = 0	
for (i = 0; i < TileCols; i++) {	
a = a MiColStarts[i]	
}	
for (i = 0; i < TileRows; i++) {	
a = a MiRowStarts[i]	
}	
if (a & 16) {	
gdfBlkSize = 64	
}	
}	
GdfBlkSize = gdfBlkSize	

if (MiCols * MI_SIZE > gdfBlkSize	
MiRows * MI_SIZE > gdfBlkSize	
(disable_loopfilters_across_tiles &&	
(TileRows > 1 TileCols > 1))) {	
gdf_per_block	f(1)
} else {	
gdf_per_block = 0	
}	
gdf_pic_qc_idx	f(2)
gdf_pic_scale_idx	f(2)
GdfPixScale = 1 + gdf_pic_scale_idx	
}	
}	

§ 5.18.7.10. CDEF params syntax

cdef_params() {	Descriptor
if (CodedLossless	
!enable_cdef) {	
cdef_frame_enable = 0	
return	
}	
if (single_picture_header_flag) {	
cdef_frame_enable = 1	
} else {	
cdef_frame_enable	f(1)
}	
if (!cdef_frame_enable) {	
return	
}	
cdef_damping_minus_3	f(2)
CdefDamping = cdef_damping_minus_3 + 3	
cdef_strengths_minus_1	f(3)
CdefStrengths = cdef_strengths_minus_1 + 1	
if (CdefOnSkipTxfm == CDEF_ON_SKIP_TXFM_ADAPTIVE) {	
cdef_on_skip_txfm_frame_enable	f(1)
} else if (CdefOnSkipTxfm == CDEF_ON_SKIP_TXFM_ALWAYS_ON) {	
cdef_on_skip_txfm_frame_enable = 1	
} else {	
cdef_on_skip_txfm_frame_enable = 0	
}	
for (i = 0; i < CdefStrengths; i++) {	

cdef_y_pri_zero	f(1)
if (cdef_y_pri_zero) {	
cdef_y_pri_strength[i] = 0	
} else {	
cdef_y_pri_strength[i]	f(4)
}	
cdef_y_sec_strength[i]	f(2)
if (cdef_y_sec_strength[i] == 3) {	
cdef_y_sec_strength[i] += 1	
}	
if (NumPlanes > 1) {	
cdef_uv_pri_zero	f(1)
if (cdef_uv_pri_zero) {	
cdef_uv_pri_strength[i] = 0	
} else {	
cdef_uv_pri_strength[i]	f(4)
}	
cdef_uv_sec_strength[i]	f(2)
if (cdef_uv_sec_strength[i] == 3) {	
cdef_uv_sec_strength[i] += 1	
}	
}	
}	
}	

[§](#) **5.18.7.11.** *Loop restoration params syntax*

lr_params() {	Descriptor
if (CodedLossless !enable_restoration) {	
FrameRestorationType[0] = RESTORE_NONE	
FrameRestorationType[1] = RESTORE_NONE	
FrameRestorationType[2] = RESTORE_NONE	
UsesLr = 0	
for (i = 0; i < 3; i++) {	
frame_filters_on[i] = 0	
}	
return	
}	
usesLumaLr = 0	
usesChromaLr = 0	
for (plane = 0; plane < NumPlanes; plane++) {	
toolsCount = 1	

indexToTool[0] = RESTORE_NONE	
for (i = 1; i < RESTORE_SWITCHABLE_TYPES; i++) {	
if (!lr_tools_disable[plane > 0][i]) {	
indexToTool[toolsCount] = i	
toolsCount += 1	
}	
}	
indexToTool[toolsCount] = RESTORE_SWITCHABLE	
allowSwitchable = (toolsCount > 2)	
n = toolsCount + allowSwitchable	
tool_index	ns(n)
FrameRestorationType[plane] = indexToTool[tool_index]	
if (FrameRestorationType[plane] != RESTORE_NONE) {	
if (plane == 0) {	
usesLumaLr = 1	
} else {	
usesChromaLr = 1	
}	
}	
r = FrameRestorationType[plane]	
if (plane == 0) {	
NumFilterClasses = 1	
}	
frame_filters_on[plane] = 0	
temporal_pred_flag[plane] = 0	
if (r == RESTORE_WIENER_NONSEP r == RESTORE_SWITCHABLE) {	
frame_filters_on[plane]	f(1)
if (frame_filters_on[plane]) {	
numRefFrames = (FrameIsIntra FrameType == SWITCH_FRAME) ?	
0 : NumTotalRefs	
if (numRefFrames > 0) {	
temporal_pred_flag[plane]	f(1)
}	
if (temporal_pred_flag[plane] && numRefFrames > 1) {	
n = CeilLog2(numRefFrames)	
rst_ref_pic_idx	f(n)
} else {	
rst_ref_pic_idx = 0	
}	
if (temporal_pred_flag[plane]) {	
refIdx = ref_frame_idx[rst_ref_pic_idx]	

refPlane = plane	
if (plane > 0 && !RefFrameFiltersOn[refIdx][plane]) {	
refPlane = plane == 1 ? 2 : 1	
}	
if (plane == 0) {	
NumFilterClasses = RefNumFilterClasses[refIdx]	
}	
for (c = 0; c < WIENER_NS_CLASSES; c++) {	
for (i = 0; i < WIENER_NS_CHROMA_COEFFS; i++) {	
FrameLrWienerNs[plane][c][i] =	
RefFrameLrWienerNs[refIdx][refPlane][c][i]	
}	
}	
}	
}	
if (plane == 0 && frame_filters_on[0]) {	
if (temporal_pred_flag[plane]) {	
num_filter_classes_idx =	
Encode_Num_Filter_Classes[NumFilterClasses]	
} else {	
num_filter_classes_idx	f(3)
NumFilterClasses =	
Decode_Num_Filter_Classes[num_filter_classes_idx]	
}	
qindex = base_q_idx	
index = get_filter_set_index(qindex)	
SubclassLookup =	
Pc_Wiener_Sub_Classify2[index][num_filter_classes_idx]	
}	
}	
}	
UsesLr = usesLumaLr usesChromaLr	
LoopRestorationSize[0] = RESTORATION_TILESIZE_MAX >> 3	
LoopRestorationSize[1] = RESTORATION_TILESIZE_MAX >>	
(3 + Max(SubsamplingX, SubsamplingY))	
if (usesLumaLr) {	
lr_luma_use_half_size	f(1)
if (lr_luma_use_half_size) {	
shift = 1	
} else if (SbSize == BLOCK_256X256) {	
shift = 0	

} else {	
lr_luma_use_max_size	f(1)
if (lr_luma_use_max_size) {	
shift = 0	
} else if (SbSize == BLOCK_128X128) {	
shift = 2	
} else {	
lr_luma_use_quarter_size	f(1)
shift = lr_luma_use_quarter_size ? 2 : 3	
}	
}	
LoopRestorationSize[0] = RESTORATION_TILESIZE_MAX >> shift	
}	
if (usesChromaLr) {	
LoopRestorationSize[1] = RESTORATION_TILESIZE_MAX >>	
Max(SubsamplingX, SubsamplingY)	
lr_chroma_use_half_size	f(1)
if (lr_chroma_use_half_size) {	
shift = 1	
} else if (SbSize == BLOCK_256X256) {	
shift = 0	
} else {	
lr_chroma_use_max_size	f(1)
if (lr_chroma_use_max_size) {	
shift = 0	
} else if (SbSize == BLOCK_128X128) {	
shift = 2	
} else {	
lr_chroma_use_quarter_size	f(1)
shift = lr_chroma_use_quarter_size ? 2 : 3	
}	
}	
LoopRestorationSize[1] = LoopRestorationSize[1] >> shift	
}	
LoopRestorationSize[2] = LoopRestorationSize[1]	
for (plane = 0; plane < NumPlanes; plane++) {	
if (frame_filters_on[plane] && !temporal_pred_flag[plane]) {	
read_wienersns_filter(plane, 0, 0, 1)	
}	
}	
}	

where the function `get_filter_set_index` is defined as:

```
get_filter_set_index( base_qindex ) {
    if (base_qindex < 130) {
        return 0
    } else if (base_qindex < 190) {
        return 1
    } else if (base_qindex < 220) {
        return 2
    } else {
        return 3
    }
}
```

and the constant tables `Decode_Num_Filter_Classes` and `Encode_Num_Filter_Classes` are defined as:

```
Encode_Num_Filter_Classes[ 17 ] = {
    0, 0, 1, 2, 3, 0, 4, 0, 5, 0, 0, 0, 6, 0, 0, 0, 7
}

Decode_Num_Filter_Classes[ 8 ] = {
    1, 2, 3, 4, 6, 8, 12, 16
}
```

§ 5.18.7.12. CCSO params syntax

ccso_params() {	Descriptor
for (plane = 0; plane < NumPlanes; plane++) {	
ccso_planes[plane] = 0	
}	
if (CodedLossless !enable_ccso) {	
return	
}	
a = 0	
for (i = 0; i < TileCols; i++) {	
a = a MiColStarts[i]	
}	
for (i = 0; i < TileRows; i++) {	
a = a MiRowStarts[i]	
}	
if (ccso_unit_matches_sb_size) {	
CcsoLumaSizeLog2 = Mi_Width_Log2[SbSize] + MI_SIZE_LOG2	
} else if ((a & 63) == 0) {	
CcsoLumaSizeLog2 = 8	
} else if ((a & 31) == 0) {	
CcsoLumaSizeLog2 = 7	
} else {	
CcsoLumaSizeLog2 = 6	
}	

if (single_picture_header_flag) {	
ccso_frame_flag = 1	
} else {	
ccso_frame_flag	f(1)
}	
if (!ccso_frame_flag) {	
return	
}	
for (plane = 0; plane < NumPlanes; plane++) {	
ccso_planes[plane]	f(1)
if (ccso_planes[plane]) {	
if (FrameIsIntra FrameType == SWITCH_FRAME) {	
reuse_ccso[plane] = 0	
sb_reuse_ccso[plane] = 0	
} else {	
reuse_ccso[plane]	f(1)
sb_reuse_ccso[plane]	f(1)
}	
if (reuse_ccso[plane] sb_reuse_ccso[plane]) {	
n = CeilLog2(NumTotalRefs)	
ccso_ref_idx[plane]	f(n)
idx = ref_frame_idx[ccso_ref_idx[plane]]	
tmpCcsolumaSizeLog2 = CcsolumaSizeLog2	
load_ccso_params(idx, plane)	
CcsolumaSizeLog2 = tmpCcsolumaSizeLog2	
}	
}	
if (ccso_planes[plane] && !reuse_ccso[plane]) {	
ccso_bo_only[plane]	f(1)
ccso_scale_idx[plane]	f(2)
if (ccso_bo_only[plane]) {	
ccso_quant_idx[plane] = 0	
ccso_ext_filter[plane] = 0	
ccso_edge_clf[plane] = 0	
} else {	
ccso_quant_idx[plane]	f(2)
ccso_ext_filter[plane]	f(3)
quantStep = CCSO_Quant_Sz[ccso_scale_idx[plane]]	
[ccso_quant_idx[plane]]	
if (quantStep == 0) {	
ccso_edge_clf[plane] = 0	

} else {	
ccso_edge_clf[plane]	f(1)
}	
}	
n = 2 + ccso_bo_only[plane]	
ccso_max_band_log2[plane]	f(n)
maxEdgeInterval = CCSO_INPUT_INTERVAL - ccso_edge_clf[plane]	
if (ccso_bo_only[plane]) {	
maxEdgeInterval = 1	
}	
maxBand = 1 << ccso_max_band_log2[plane]	
for (d0 = 0; d0 < maxEdgeInterval; d0++) {	
for (d1 = 0; d1 < maxEdgeInterval; d1++) {	
for (band = 0; band < maxBand; band++) {	
ccso_offset_idx	tu(7)
offset = Ccso_Offset[ccso_offset_idx] *	
(ccso_scale_idx[plane] + 1)	
CcsoFilterOffset[plane][band][d0][d1] = offset	
}	
}	
}	
}	
}	

where Ccso_Offset is defined as:

```
Ccso_Offset[ 8 ] = {
    0, 1, -1, 3, -3, 7, -7, -10
}
```

§ 5.18.8. Transform and coding mode structures

§ 5.18.8.1. TX mode syntax

read_tx_mode() {	Descriptor
if (CodedLossless == 1) {	
TxMode = ONLY_4X4	
} else {	
tx_mode_select	f(1)
if (tx_mode_select) {	
TxMode = TX_MODE_SELECT	
} else {	
TxMode = TX_MODE_LARGEST	

}	
}	
}	

§ 5.18.8.2. Skip mode params syntax

skip_mode_params() {	Descriptor
if (FrameIsIntra FrameType == SWITCH_FRAME) {	
skipModeAllowed = 0	
} else {	
skipModeAllowed = 1	
SkipModeFrame[0] = 0	
SkipModeFrame[1] = NumTotalRefs > 1 ? 1 : 0	
if (NumTotalRefs > 1) {	
curToRef0 = Abs(get_relative_dist(OrderHint,	
RefOrderHint[ref_frame_idx[0]]))	
curToRef1 = Abs(get_relative_dist(OrderHint,	
RefOrderHint[ref_frame_idx[1]]))	
if (OrderHints[0] == RESTRICTED_OH) {	
curToRef0 = 0	
}	
if (OrderHints[1] == RESTRICTED_OH) {	
curToRef1 = 0	
}	
if (Abs(curToRef0 - curToRef1) > 1) {	
SkipModeFrame[1] = 0	
}	
}	
}	
if (skipModeAllowed) {	
skip_mode_present	f(1)
} else {	
skip_mode_present = 0	
}	
}	

§ 5.18.8.3. Frame reference mode syntax

frame_reference_mode() {	Descriptor
if (FrameIsIntra) {	
reference_select = 0	
} else {	
reference_select	f(1)

}	
}	

§ 5.18.9. Global motion structures

§ 5.18.9.1. Global motion params syntax

global_motion_params() {	Descriptor
for (ref = 0; ref < REFS_PER_FRAME; ref++) {	
GmType[ref] = IDENTITY	
for (i = 0; i < 6; i++) {	
gm_params[ref][i] = ((i % 3 == 2) ?	
1 << WARPEDMODEL_PREC_BITS : 0)	
}	
}	
if (FrameIsIntra !enable_global_motion) {	
return	
}	
use_global_motion	f(1)
if (!use_global_motion) {	
return	
}	
for (i = 0; i < 6; i++) {	
baseParams[i] = Default_Warp_Params[i]	
}	
baseDistance = 1	
if (FrameType == SWITCH_FRAME) {	
our_ref = NumTotalRefs	
} else {	
n = NumTotalRefs + 1	
our_ref	ns(n)
}	
if (our_ref != NumTotalRefs) {	
refIdx = ref_frame_idx[our_ref]	
if (RefNumTotalRefs[refIdx] > 0) {	
n = RefNumTotalRefs[refIdx]	
their_ref	ns(n)
for (i = 0; i < 6; i++) {	
baseParams[i] = SavedGmParams[refIdx][their_ref][i]	
}	
baseDistance = get_relative_dist(OrderHints[our_ref],	
SavedOrderHints[refIdx][their_ref])	
}	

}	
for (ref = 0; ref < NumTotalRefs; ref++) {	
dist = get_relative_dist(OrderHint,OrderHints[ref])	
if (dist == 0 OrderHints[ref] == RESTRICTED_OH) {	
for (i = 0; i < 6; i++) {	
gm_params[ref][i] = Default_Warp_Params[i]	
}	
GmType[ref] = IDENTITY	
} else {	
for (i = 0; i < 6; i++) {	
params = scale_warp_model(baseParams, baseDistance, dist)	
PrevGmParams[ref][i] = params[i]	
}	
is_global	f(1)
if (is_global) {	
is_rot_zoom	f(1)
if (is_rot_zoom) {	
type = ROTZOOM	
} else {	
type = AFFINE	
}	
} else {	
type = IDENTITY	
}	
GmType[ref] = type	
if (type >= ROTZOOM) {	
read_global_param(ref,2)	
read_global_param(ref,3)	
if (type == AFFINE) {	
read_global_param(ref,4)	
read_global_param(ref,5)	
} else {	
gm_params[ref][4] = -gm_params[ref][3]	
gm_params[ref][5] = gm_params[ref][2]	
}	
read_global_param(ref,0)	
read_global_param(ref,1)	
}	
}	
}	
}	

where Param_Shift, Param_Min, Param_Max, and scale_warp_model are defined as:

```
Param_Shift[ 6 ] = {
    GM_TRANS_PREC_DIFF,    GM_TRANS_PREC_DIFF,    GM_ALPHA_PREC_DIFF,
    GM_ALPHA_PREC_DIFF,    GM_ALPHA_PREC_DIFF,    GM_ALPHA_PREC_DIFF
}

Param_Min[ 6 ] = {
    GM_TRANS_MIN,          GM_TRANS_MIN,
    GM_ALPHA_MIN,          GM_ALPHA_MIN,
    GM_ALPHA_MIN,          GM_ALPHA_MIN
}

Param_Max[ 6 ] = {
    GM_TRANS_MAX,          GM_TRANS_MAX,
    GM_ALPHA_MAX,          GM_ALPHA_MAX,
    GM_ALPHA_MAX,          GM_ALPHA_MAX
}

scale_warp_model(baseParams, baseDistance, dist) {
    if ( baseDistance == 0 ) {
        return Default_Warp_Params
    }
    if ( baseDistance < 0 ) {
        baseDistance = -baseDistance
        dist = -dist
    }
    for ( i = 0; i < 6; i++ ) {
        center = Default_Warp_Params[ i ]
        limit = (1 << 22) - 1
        input = Clip3( -limit, limit, baseParams[ i ] - center )
        (divShift, divFactor) = resolve_divisor( baseDistance )
        scaled = Round2Signed( input * divFactor, divShift )
        output = Round2Signed( scaled * dist, Param_Shift[ i ] )
        output = Clip3( Param_Min[i], Param_Max[i], output ) << Param_Shift[i]
        params[ i ] = center + output
    }
    return params
}
```

§ 5.18.9.2. Global param syntax

read_global_param(ref, idx) {	Descriptor
precBits = GM_ALPHA_PREC_BITS	
mx = GM_ALPHA_MAX	
if (idx < 2) {	
precBits = GM_TRANS_PREC_BITS	
mx = GM_TRANS_MAX	
}	
precDiff = WARPEDMODEL_PREC_BITS - precBits	
round = (idx % 3) == 2 ? (1 << WARPEDMODEL_PREC_BITS) : 0	
sub = (idx % 3) == 2 ? (1 << precBits) : 0	
r = (PrevGmParams[ref][idx] >> precDiff) - sub	
gm_params[ref][idx] =	
(decode_signed_subexp_with_ref(-mx, mx + 1, r, 3) << precDiff) + round	
}	

NOTE: When `force_integer_mv` is equal to 1, some fractional bits are still read for the translation components. However, these fractional bits will be discarded during the Setup Global MV process.

§ 5.18.9.3. Decode signed subexp with ref syntax

<code>decode_signed_subexp_with_ref(low, high, r, k) {</code>	Descriptor
<code> x = decode_unsigned_subexp_with_ref(high - low, r - low, k)</code>	
<code> return x + low</code>	
<code>}</code>	

§ 5.18.9.4. Decode unsigned subexp with ref syntax

<code>decode_unsigned_subexp_with_ref(mx, r, k) {</code>	Descriptor
<code> v = decode_subexp(mx, k)</code>	
<code> if ((r << 1) <= mx) {</code>	
<code> return inverse_recenter(r, v)</code>	
<code> } else {</code>	
<code> return mx - 1 - inverse_recenter(mx - 1 - r, v)</code>	
<code> }</code>	
<code>}</code>	

§ 5.18.9.5. Decode subexp syntax

<code>decode_subexp(numSyms, k) {</code>	Descriptor
<code> i = 0</code>	
<code> mk = 0</code>	
<code> while (1) {</code>	
<code> b2 = i ? k + i - 1 : k</code>	
<code> a = 1 << b2</code>	
<code> if (numSyms <= mk + 3 * a) {</code>	
<code> n = numSyms - mk</code>	
<code> subexp_final_bits</code>	<code>ns(n)</code>
<code> return subexp_final_bits + mk</code>	
<code> } else {</code>	
<code> subexp_more_bits</code>	<code>f(1)</code>
<code> if (subexp_more_bits) {</code>	
<code> i++</code>	
<code> mk += a</code>	
<code> } else {</code>	
<code> subexp_bits</code>	<code>f(b2)</code>
<code> return subexp_bits + mk</code>	
<code> }</code>	
<code> }</code>	

}	
}	

§ 5.18.9.6. Inverse recenter function

```
inverse_recenter( r, v ) {
    if ( v > 2 * r ) {
        return v
    } else if ( v & 1 ) {
        return r - ((v + 1) >> 1)
    } else {
        return r + (v >> 1)
    }
}
```

§ 5.18.10. Film grain structures

§ 5.18.10.1. Film grain config syntax

film_grain_config() {	Descriptor
if (!film_grain_params_present (!immediate_output_frame && !implicit_output_frame)) {	
apply_grain = 0	
} else if (single_picture_header_flag) {	
apply_grain = 1	
} else {	
apply_grain	f(1)
}	
if (apply_grain) {	
fgm_id	f(3)
load_grain_model(fgm_id)	
grain_seed	f(16)
}	
}	

§ 5.18.10.2. Film grain model syntax

film_grain_model(monochrome, subX, subY) {	Descriptor
if (monochrome) {	
chroma_scaling_from_luma = 0	
} else {	
chroma_scaling_from_luma	f(1)
}	
num_y_points	f(4)
if (num_y_points > 0) {	
point_value_increment_bits_minus_1	f(3)
bitsIncr = point_value_increment_bits_minus_1 + 1	
point_scaling_bits_minus_5	f(2)

bitsScal = point_scaling_bits_minus_5 + 5	
}	
for (i = 0; i < num_y_points; i++) {	
point_y_value[i]	f(bitsIncr)
if (i > 0) {	
point_y_value[i] += point_y_value[i - 1]	
}	
point_y_scaling[i]	f(bitsScal)
}	
if (monochrome chroma_scaling_from_luma) {	
num_cb_points = 0	
num_cr_points = 0	
} else {	
num_cb_points	f(4)
if (num_cb_points > 0) {	
point_value_increment_bits_minus_1	f(3)
bitsIncr = point_value_increment_bits_minus_1 + 1	
point_scaling_bits_minus_5	f(2)
bitsScal = point_scaling_bits_minus_5 + 5	
}	
for (i = 0; i < num_cb_points; i++) {	
point_cb_value[i]	f(bitsIncr)
if (i > 0) {	
point_cb_value[i] += point_cb_value[i - 1]	
}	
point_cb_scaling[i]	f(bitsScal)
}	
num_cr_points	f(4)
if (num_cr_points > 0) {	
point_value_increment_bits_minus_1	f(3)
bitsIncr = point_value_increment_bits_minus_1 + 1	
point_scaling_bits_minus_5	f(2)
bitsScal = point_scaling_bits_minus_5 + 5	
}	
for (i = 0; i < num_cr_points; i++) {	
point_cr_value[i]	f(bitsIncr)
if (i > 0) {	
point_cr_value[i] += point_cr_value[i - 1]	
}	
point_cr_scaling[i]	f(bitsScal)
}	

}	
grain_scaling_minus_8	f(2)
ar_coeff_lag	f(2)
numPosLuma = 2 * ar_coeff_lag * (ar_coeff_lag + 1)	
if (num_y_points) {	
bits_per_ar_coeff_y_minus_5	f(2)
bitsCoef = bits_per_ar_coeff_y_minus_5 + 5	
numPosChroma = numPosLuma + 1	
for (i = 0; i < numPosLuma; i++) {	
ar_coeffs_y[i]	f(bitsCoef)
ar_coeffs_y[i] -= (1 << (bitsCoef - 1))	
}	
} else {	
numPosChroma = numPosLuma	
}	
if (chroma_scaling_from_luma num_cb_points) {	
bits_per_ar_coeff_cb_minus_5	f(2)
bitsCoef = bits_per_ar_coeff_cb_minus_5 + 5	
for (i = 0; i < numPosChroma; i++) {	
ar_coeffs_cb[i]	f(bitsCoef)
ar_coeffs_cb[i] -= (1 << (bitsCoef - 1))	
}	
}	
if (chroma_scaling_from_luma num_cr_points) {	
bits_per_ar_coeff_cr_minus_5	f(2)
bitsCoef = bits_per_ar_coeff_cr_minus_5 + 5	
for (i = 0; i < numPosChroma; i++) {	
ar_coeffs_cr[i]	f(bitsCoef)
ar_coeffs_cr[i] -= (1 << (bitsCoef - 1))	
}	
}	
ar_coeff_shift_minus_6	f(2)
grain_scale_shift	f(2)
if (num_cb_points) {	
cb_mult	f(8)
cb_luma_mult	f(8)
cb_offset	f(9)
}	
if (num_cr_points) {	
cr_mult	f(8)
cr_luma_mult	f(8)
cr_offset	

	f(9)
}	
overlap_flag	f(1)
clip_to_restricted_range	f(1)
if (clip_to_restricted_range) {	
fg_mc_identity	f(1)
} else {	
fg_mc_identity = 0	
}	
film_grain_block_size	f(1)
}	

§ 5.19. Tile group OBU syntax

tile_group_obu(sz) {	Descriptor
startBitPos = get_position()	
is_first_tile_group	f(1)
if (is_first_tile_group) {	
frame_header_present_flag = 1	
} else {	
frame_header_present_flag	f(1)
}	
if (frame_header_present_flag) {	
frame_header(is_first_tile_group)	
}	
if (bru_inactive) {	
headerBits = get_position() - startBitPos	
remainingBits = sz * 8 - headerBits	
trailing_bits(remainingBits)	
return	
}	
NumTiles = TileCols * TileRows	
tile_start_and_end_present_flag = 0	
if (NumTiles > 1) {	
tile_start_and_end_present_flag	f(1)
}	
if (NumTiles == 1 !tile_start_and_end_present_flag) {	
tg_start = 0	
tg_end = NumTiles - 1	
} else {	
tileBits = TileColsLog2 + TileRowsLog2	
tg_start	f(tileBits)

tg_end	f(tileBits)
}	
if (use_bru) {	
if (NumTiles > 1) {	
for (TileNum = tg_start; TileNum <= tg_end; TileNum++) {	
tileRow = TileNum / TileCols	
tileCol = TileNum % TileCols	
bru_tile_active	f(1)
BruTileActives[tileRow][tileCol] = bru_tile_active	
}	
} else {	
BruTileActives[0][0] = 1	
}	
}	
byte_alignment()	
endBitPos = get_position()	
headerBytes = (endBitPos - startBitPos) / 8	
sz -= headerBytes	
tile_group_payload(sz)	
}	

§ 5.20. Tile group payload syntax

§ 5.20.1. General tile group payload syntax

tile_group_payload(sz) {	Descriptor
for (TileNum = tg_start; TileNum <= tg_end; TileNum++) {	
tileRow = TileNum / TileCols	
tileCol = TileNum % TileCols	
lastTile = TileNum == tg_end	
if (lastTile) {	
tileSize = sz	
} else if (!IsBridge) {	
tile_size_minus_1	le(TileSizeBytes)
tileSize = tile_size_minus_1 + 1	
sz -= tileSize + TileSizeBytes	
}	
MiRowStart = MiRowStarts[tileRow]	
MiRowEnd = MiRowStarts[tileRow + 1]	
MiColStart = MiColStarts[tileCol]	
MiColEnd = MiColStarts[tileCol + 1]	
BruTileActive = use_bru ? BruTileActives[tileRow][tileCol] : 0	

align = Num_4x4_Blocks_High[SbSize]	
shift = Mi_Height_Log2[SbSize]	
for(r = MiRowStart; r < ((MiRowEnd + align - 1) >> shift) << shift;	
r++) {	
for(c = MiColStart; c < ((MiColEnd + align - 1) >> shift) << shift;	
c++) {	
IBCCoded[r][c] = 0	
}	
}	
CurrentQIndex = base_q_idx	
if (!IsBridge) {	
init_symbol(tileSize)	
}	
decode_tile()	
if (!IsBridge) {	
exit_symbol()	
}	
}	
if (tg_end == NumTiles - 1) {	
if (!IsBridge) {	
frame_end_update_cdf()	
}	
decode_frame_wrapup()	
SeenFrameHeader = 0	
}	
}	

§ 5.20.2. Tile-level structures

§ 5.20.2.1. Decode tile syntax

decode_tile() {	Descriptor
clear_above_context()	
for (plane = 0; plane < WIENER_NS_PLANES; plane++) {	
for (c = 0; c < WIENER_NS_CLASSES; c++) {	
for (i = 0; i < WIENER_NS_CHROMA_COEFFS; i++) {	
min = Wiener_Ns_Taps_Min[plane != 0][i]	
k = Wiener_Ns_Taps_K[plane != 0][i]	
RefLrWienerNs[plane][c][0][i] = min + ((1 << k) >> 1)	
}	
WienerNsPtr[plane][c] = 0	
WienerNsBankSize[plane][c] = 0	
}	

}	
sbSize4 = Num_4x4_Blocks_Wide[SbSize]	
for (r = MiRowStart; r < MiRowEnd; r += sbSize4) {	
clear_left_context()	
for (i = 0; i < IBC_NUM_BUFFERS; i++) {	
IBCBufferValid[i] = 0	
}	
IBCBufferCurRow = r >> (IBC_BUFFER_SIZE_LOG2 - MI_SIZE_LOG2)	
IBCBufferCurCol = 0	
for (c = MiColStart; c < MiColEnd; c += sbSize4) {	
reset_refmv_bank(r, c, sbSize4, r == MiRowStart)	
ReadDeltas = delta_q_present	
clear_cdef(r, c)	
clear_block_decoded_flags(r, c, sbSize4)	
if (IsBridge) {	
bru_mode = BRU_INACTIVE	
} else if (BruTileActive) {	
bru_mode	S()
} else {	
bru_mode = use_bru ? BRU_INACTIVE : BRU_ACTIVE	
}	
BruModes[r][c] = bru_mode	
RegionType = MIXED_REGION	
TreeType = SHARED_PART	
PlaneStart = 0	
PlaneEnd = NumPlanes	
decode_partition(r, c, SbSize, BLOCK_INVALID, 0, 1,	
enable_extended_sdp && !FrameIsIntra)	
}	
}	
}	

where Wiener_Ns_Taps_Min and Wiener_Ns_Taps_K are constant lookup tables specified as:

```

Wiener_Ns_Taps_Min[ 2 ][ 18 ] = {
  {-24, -24, -14 , -14, -16, -16, -8,  -8,  -8,  -8, -8, -8, -8, -8, -8, -8,
   -8, -8},
  {-24, -24, -14 , -14, -16, -16, -16, -16, -16, -16, -8, -8, -8, -8, -8, -8,
   -8, -8}
}

Wiener_Ns_Taps_K[ 2 ][ 18 ] = {
  {6, 6, 5, 5, 5, 5, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4},
  {6, 6, 5, 5, 5, 5, 5, 5, 5, 5, 4, 4, 4, 4, 4, 4, 4, 4}
}

```

§ 5.20.2.2. Reset reference motion vector bank function

```

reset_refmv_bank( r, c, sbSize4, topRow ) {
    WarpBankHits = 0
    RefMvBankHits = 0
    RefMvRemainHits = 0
    RefMvUnitHits = 0
    if ( FrameIsIntra || topRow ) {
        return
    }
    rowHits = 0
    candRow = r - 1
    candCol = c
    while ( candCol < MiCols && candCol < c + sbSize4 && rowHits < 4 ) {
        candCol2 = (candCol >> 1) << 1
        if ( IsInters[ candRow ][ candCol2 ] ) {
            rowHits++
            update_ref_mv_bank( RefFrames[ candRow ][ candCol2 ],
                                Mvs[ candRow ][ candCol2 ], CwpIdxs[ candRow ][ candCol2 ], 0)
            if ( MotionModes[ candRow ][ candCol2 ] >= LOCALWARP ) {
                update_warp_param_bank( RefFrames[ candRow ][ candCol2 ],
                                        WarpParams[ candRow ][ candCol2 ], 1)
            }
        }
        candSize = MiSizes[ 0 ][ candRow ][ candCol2 ]
        candCol += Num_4x4_Blocks_Wide[ candSize ]
    }
}

```

§ 5.20.2.3. Clear block decoded flags function

```

clear_block_decoded_flags( r, c, sbSize4 ) {
    for ( plane = 0; plane < NumPlanes; plane++ ) {
        subX = (plane > 0) ? SubsamplingX : 0
        subY = (plane > 0) ? SubsamplingY : 0
        sbWidth4 = ( MiColEnd - c ) >> subX
        sbHeight4 = ( MiRowEnd - r ) >> subY
        for ( y = -1; y <= ( sbSize4 >> subY ); y++ ) {
            for ( x = -1; x <= ( (2 * sbSize4) >> subX ); x++ ) {
                if ( y < 0 && x < sbWidth4 ) {
                    BlockDecoded[ plane ][ y ][ x ] = 1
                } else if ( x < 0 && y < sbHeight4 ) {
                    BlockDecoded[ plane ][ y ][ x ] = 1
                } else {
                    BlockDecoded[ plane ][ y ][ x ] = 0
                }
            }
        }
        BlockDecoded[ plane ][ sbSize4 >> subY ][ -1 ] = 0
    }
}

```

§ 5.20.3. Partition structures

§ 5.20.3.1. Decode partition syntax

decode_partition(r, c, bSize, parentSize, chromaOffset, hasChroma, extendedSdpAllowed) {	Descriptor
if (r >= MiRows c >= MiCols) {	
for (y = 0; y < Num_4x4_Blocks_High[bSize]; y++) {	
for (x = 0; x < Num_4x4_Blocks_Wide[bSize]; x++) {	
IBCCoded[r + y][x + c] = 1	
}	

}	
widthChunks = Max(1, Block_Width[bSize] >> 6)	
heightChunks = Max(1, Block_Height[bSize] >> 6)	
for (chunkY = 0; chunkY < heightChunks; chunkY++) {	
for (chunkX = 0; chunkX < widthChunks; chunkX++) {	
miRowChunk = r + (chunkY << 4)	
miColChunk = c + (chunkX << 4)	
update_ibc_buffers(miRowChunk, miColChunk)	
}	
}	
return	
}	
if (enable_sdp && TreeType == SHARED_PART &&	
bSize == BLOCK_64X64 && FrameIsIntra) {	
TreeType = LUMA_PART	
PlaneStart = 0	
PlaneEnd = 1	
decode_partition(r, c, BLOCK_64X64, parentSize, 0, 1, 0)	
TreeType = CHROMA_PART	
PlaneStart = 1	
PlaneEnd = NumPlanes	
decode_partition(r, c, BLOCK_64X64, parentSize, 0, 1, 0)	
TreeType = SHARED_PART	
PlaneStart = 0	
PlaneEnd = NumPlanes	
return	
}	
if (SbSize == bSize) {	
read_lr(r, c, SbSize)	
}	
AvailU = is_inside(r - 1, c)	
AvailL = is_inside(r, c - 1)	
num4x4wide = Num_4x4_Blocks_Wide[bSize]	
halfBlock4x4wide = num4x4wide >> 1	
num4x4high = Num_4x4_Blocks_High[bSize]	
halfBlock4x4high = num4x4high >> 1	
partition = read_partition(r, c, bSize, chromaOffset, hasChroma)	
subSize = Partition_Subsize[partition][bSize]	
usingSdp = 0	
if (bSize != SbSize && extendedSdpAllowed &&	
TreeType == SHARED_PART &&	

is_bsize_allowed_for_extended_sdp(bSize, partition) &&	
bru_mode == BRU_ACTIVE) {	
region_type	S()
if (region_type == INTRA_REGION) {	
TreeType = LUMA_PART	
RegionType = INTRA_REGION	
PlaneStart = 0	
PlaneEnd = 1	
usingSdp = 1	
}	
}	
extendedSdpAllowed = extendedSdpAllowed && Block_Width[subSize] > 4 &&	
Block_Height[subSize] > 4	
if (partition == PARTITION_HORZ_3 partition == PARTITION_VERT_3) {	
subSize2 = H_Partition_Midsize[bSize]	
extendedSdpAllowed = extendedSdpAllowed &&	
Block_Width[subSize2] > 4 &&	
Block_Height[subSize2] > 4	
}	
if (SbSize == BLOCK_128X128) {	
if (bSize == BLOCK_128X128) {	
AllowExtraIBCRange = partition == PARTITION_HORZ	
partition == PARTITION_SPLIT	
}	
} else {	
AllowExtraIBCRange = 0	
}	
if (FrameIsIntra) {	
if (TreeType == LUMA_PART && bSize == BLOCK_64X64) {	
TopLumaHorz = partition == PARTITION_HORZ	
partition == PARTITION_HORZ_3	
TopLumaVert = partition == PARTITION_VERT	
partition == PARTITION_VERT_3	
TopLumaUnevenHorz = partition == PARTITION_HORZ_4A	
partition == PARTITION_HORZ_4B	
TopLumaUnevenVert = partition == PARTITION_VERT_4A	
partition == PARTITION_VERT_4B	
ChromaFollowsLuma = (partition == PARTITION_NONE)	
TopLumaHorz TopLumaVert	
LumaPartitions[r][c] = partition	
}	

thisHorz = partition == PARTITION_HORZ	
partition == PARTITION_HORZ_3	
partition == PARTITION_HORZ_4A	
partition == PARTITION_HORZ_4B	
thisVert = partition == PARTITION_VERT	
partition == PARTITION_VERT_3	
partition == PARTITION_VERT_4A	
partition == PARTITION_VERT_4B	
if (TreeType == CHROMA_PART && bSize == BLOCK_64X64) {	
if (ChromaFollowsLuma	
partition == PARTITION_NONE	
(TopLumaHorz TopLumaUnevenHorz) && thisHorz	
(TopLumaVert TopLumaUnevenVert) && thisVert) {	
CflAllowedInSdp = 1	
} else {	
CflAllowedInSdp = 0	
}	
}	
if (TreeType == LUMA_PART && parentSize == BLOCK_64X64) {	
if (partition == PARTITION_NONE	
(TopLumaHorz && thisHorz)	
(TopLumaVert && thisVert)) {	
ChromaFollowsLuma = 0	
}	
}	
}	
if (!chromaOffset && hasChroma) {	
chromaOffset = is_chroma_offset_for_partition(partition, bSize)	
ChromaMiRow = r	
ChromaMiCol = c	
ChromaMiSize = bSize	
}	
if (partition == PARTITION_NONE) {	
HasChroma = hasChroma && NumPlanes > 1 && TreeType != LUMA_PART	
decode_block(r, c, subSize)	
} else if (partition == PARTITION_HORZ) {	
decode_partition(r, c, subSize, bSize, chromaOffset,	
hasChroma && !chromaOffset, extendedSdpAllowed)	
decode_partition(r + halfBlock4x4high, c, subSize, bSize, chromaOffset,	
hasChroma, extendedSdpAllowed)	
} else if (partition == PARTITION_VERT) {	

decode_partition(r, c, subSize, bSize, chromaOffset,	
hasChroma && !chromaOffset, extendedSdpAllowed)	
decode_partition(r, c + halfBlock4x4wide, subSize, bSize, chromaOffset,	
hasChroma, extendedSdpAllowed)	
} else if (partition == PARTITION_HORZ_3) {	
decode_partition(r, c, subSize, bSize, chromaOffset,	
hasChroma && !chromaOffset, extendedSdpAllowed)	
middleChroma = bSize == BLOCK_8X32 && hasChroma && SubsamplingX	
if (middleChroma) {	
ChromaMiRow = r + (halfBlock4x4high >> 1)	
ChromaMiCol = c	
ChromaMiSize = Partition_Subsize[PARTITION_HORZ][bSize]	
}	
decode_partition(r + (halfBlock4x4high >> 1), c,	
H_Partition_Midsize[bSize], bSize,	
chromaOffset middleChroma,	
hasChroma && !chromaOffset && !middleChroma,	
extendedSdpAllowed)	
decode_partition(r + (halfBlock4x4high >> 1), c + halfBlock4x4wide,	
H_Partition_Midsize[bSize], bSize,	
chromaOffset middleChroma,	
hasChroma && !chromaOffset, extendedSdpAllowed)	
decode_partition(r + 3 * (halfBlock4x4high >> 1), c,	
subSize, bSize, chromaOffset,	
hasChroma, extendedSdpAllowed)	
} else if (partition == PARTITION_HORZ_4A) {	
bSizeBig = Partition_Subsize[PARTITION_HORZ][bSize]	
bSizeMed = Partition_Subsize[PARTITION_HORZ][bSizeBig]	
decode_partition(r, c, subSize, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r + (num4x4high >> 3), c, bSizeMed, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r + 3 * (num4x4high >> 3), c, bSizeBig, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r + 7 * (num4x4high >> 3), c, subSize, bSize,	
chromaOffset, hasChroma, extendedSdpAllowed)	
} else if (partition == PARTITION_HORZ_4B) {	
bSizeBig = Partition_Subsize[PARTITION_HORZ][bSize]	

bsizeMed = Partition_Subsize[PARTITION_HORZ][bSizeBig]	
decode_partition(r, c, subSize, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r + (num4x4high >> 3), c, bSizeBig, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r + 5 * (num4x4high >> 3), c, bsizeMed, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r + 7 * (num4x4high >> 3), c, subSize, bSize,	
chromaOffset, hasChroma, extendedSdpAllowed)	
} else if (partition == PARTITION_VERT_4A) {	
bSizeBig = Partition_Subsize[PARTITION_VERT][bSize]	
bsizeMed = Partition_Subsize[PARTITION_VERT][bSizeBig]	
decode_partition(r, c, subSize, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r, c + (num4x4wide >> 3), bsizeMed, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r, c + 3 * (num4x4wide >> 3), bSizeBig, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r, c + 7 * (num4x4wide >> 3), subSize, bSize,	
chromaOffset, hasChroma, extendedSdpAllowed)	
} else if (partition == PARTITION_VERT_4B) {	
bSizeBig = Partition_Subsize[PARTITION_VERT][bSize]	
bsizeMed = Partition_Subsize[PARTITION_VERT][bSizeBig]	
decode_partition(r, c, subSize, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r, c + (num4x4wide >> 3), bSizeBig, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r, c + 5 * (num4x4wide >> 3), bsizeMed, bSize,	
chromaOffset, hasChroma && !chromaOffset,	
extendedSdpAllowed)	
decode_partition(r, c + 7 * (num4x4wide >> 3), subSize, bSize,	
chromaOffset, hasChroma, extendedSdpAllowed)	
} else if (partition == PARTITION_SPLIT) {	

decode_partition(r, c, subSize, bSize, 0,	
hasChroma, extendedSdpAllowed)	
decode_partition(r, c + halfBlock4x4wide, subSize, bSize, 0,	
hasChroma, extendedSdpAllowed)	
decode_partition(r + halfBlock4x4high, c, subSize, bSize, 0,	
hasChroma, extendedSdpAllowed)	
decode_partition(r + halfBlock4x4high, c + halfBlock4x4wide,	
subSize, bSize, 0, hasChroma, extendedSdpAllowed)	
} else {	
decode_partition(r, c, subSize, bSize, chromaOffset,	
hasChroma && !chromaOffset, extendedSdpAllowed)	
middleChroma = bSize == BLOCK_32X8 && hasChroma && SubsamplingY	
if (middleChroma) {	
ChromaMiRow = r	
ChromaMiCol = c + (halfBlock4x4wide >> 1)	
ChromaMiSize = Partition_Subsize[PARTITION_VERT][bSize]	
}	
decode_partition(r, c + (halfBlock4x4wide >> 1),	
H_Partition_Midsize[bSize], bSize,	
chromaOffset middleChroma,	
hasChroma && !chromaOffset && !middleChroma,	
extendedSdpAllowed)	
decode_partition(r + halfBlock4x4high, c + (halfBlock4x4wide >> 1),	
H_Partition_Midsize[bSize], bSize,	
chromaOffset middleChroma,	
hasChroma && !chromaOffset, extendedSdpAllowed)	
decode_partition(r, c + 3 * (halfBlock4x4wide >> 1),	
subSize, bSize, chromaOffset,	
hasChroma, extendedSdpAllowed)	
}	
if (FrameIsIntra && TreeType == LUMA_PART && bSize == BLOCK_64X64) {	
ChromaPartitionKnown[r][c] = ChromaFollowsLuma	
}	
if (usingSdp) {	
TreeType = CHROMA_PART	
HasChroma = 1	
PlaneStart = 1	
PlaneEnd = NumPlanes	
ChromaMiRow = r	
ChromaMiCol = c	
ChromaMiSize = bSize	

AvailU = is_inside(r - 1, c)	
AvailL = is_inside(r, c - 1)	
decode_block(r, c, bSize)	
TreeType = SHARED_PART	
PlaneStart = 0	
PlaneEnd = NumPlanes	
RegionType = MIXED_REGION	
}	
}	

The function `is_bsize_allowed_for_extended_sdp` is defined as:

```
is_bsize_allowed_for_extended_sdp(bSize, partition) {
    bw = Block_Width[ bSize ]
    bh = Block_Height[ bSize ]
    return bw <= INTER_SDP_MAX_BLOCK_SIZE && bh <= INTER_SDP_MAX_BLOCK_SIZE &&
           bw >= 8 && bh >= 8 &&
           partition < PARTITION_HORZ_4A && partition != PARTITION_NONE
}
```

[§](#) 5.20.3.2. Read partition syntax

```
Rect_Part_Table[ 2 ][ 2 ][ NUM_UNEVEN_4WAY_PARTS ][ NUM_RECT_PARTS ] = {
{
    {
        { PARTITION_HORZ, PARTITION_VERT },
        { PARTITION_HORZ, PARTITION_VERT },
    },
    {
        { PARTITION_HORZ, PARTITION_VERT },
        { PARTITION_HORZ, PARTITION_VERT },
    },
},
{
    {
        { PARTITION_HORZ_3, PARTITION_VERT_3 },
        { PARTITION_HORZ_3, PARTITION_VERT_3 },
    },
    {
        { PARTITION_HORZ_4A, PARTITION_VERT_4A },
        { PARTITION_HORZ_4B, PARTITION_VERT_4B },
    },
}
}
```

<code>read_partition(r, c, bSize, chromaOffset, hasChroma) {</code>	Descriptor
<code>(implied,partition) = partition_implied(r, c, bSize)</code>	
<code>(numAllowed, allowed) = init_allowed_partitions(r, c, bSize,</code>	
<code>chromaOffset, hasChroma)</code>	
<code>if (implied && allowed[partition]) {</code>	
<code>return partition</code>	
<code>}</code>	
<code>if (numAllowed == 1) {</code>	
<code>for (p = 0; p < EXT_PARTITION_TYPES; p++) {</code>	

if (allowed[p]) {	
return p	
}	
}	
}	
if (bru_mode != BRU_ACTIVE) {	
return PARTITION_NONE	
}	
if (allowed[PARTITION_NONE]) {	
do_split	S()
if (!do_split) {	
return PARTITION_NONE	
}	
}	
if (allowed[PARTITION_SPLIT]) {	
do_square_split	S()
if (do_square_split) {	
return PARTITION_SPLIT	
}	
}	
rectType = rect_type_implied_by_bsize(bSize)	
if (rectType == RECT_INVALID) {	
allowHorz = (allowed[PARTITION_HORZ]	
allowed[PARTITION_HORZ_3]	
allowed[PARTITION_HORZ_4A]	
allowed[PARTITION_HORZ_4B])	
allowVert = (allowed[PARTITION_VERT]	
allowed[PARTITION_VERT_3]	
allowed[PARTITION_VERT_4A]	
allowed[PARTITION_VERT_4B])	
if (!allowHorz) {	
rectType = RECT_VERT	
} else if (!allowVert) {	
rectType = RECT_HORZ	
}	
}	
if (rectType == RECT_INVALID) {	
rect_type	S()
rectType = rect_type	
}	
if (rectType == RECT_HORZ) {	

nonExtAllowed = allowed[PARTITION_HORZ]	
extAllowed3 = allowed[PARTITION_HORZ_3]	
extAllowed4 = allowed[PARTITION_HORZ_4A]	
allowed[PARTITION_HORZ_4B]	
} else {	
nonExtAllowed = allowed[PARTITION_VERT]	
extAllowed3 = allowed[PARTITION_VERT_3]	
extAllowed4 = allowed[PARTITION_VERT_4A]	
allowed[PARTITION_VERT_4B]	
}	
if (nonExtAllowed && (extAllowed3 extAllowed4)) {	
do_ext_partition	S()
} else {	
do_ext_partition = extAllowed3 extAllowed4	
}	
do_uneven_4way_partition = 0	
uneven_4way_partition_type = 0	
if (do_ext_partition) {	
if (extAllowed3 && extAllowed4) {	
do_uneven_4way_partition	S()
} else {	
do_uneven_4way_partition = extAllowed4	
}	
if (do_uneven_4way_partition) {	
uneven_4way_partition_type	L(1)
}	
}	
return Rect_Part_Table[do_ext_partition][do_uneven_4way_partition]	
[uneven_4way_partition_type][rectType]	
}	

where `init_allowed_partitions`, `is_partition_allowed`, `is_chroma_offset_for_partition`, `is_chroma_offset_for_subsize`, `check_chroma`, `block_coded`, `rect_type_implied_by_bsize`, `is_ext_partition_allowed`, `partition_implied_at_bo_undary`, `partition_implied`, and `is_uneven_4way_partition_allowed` are functions defined as:

```
block_coded(r,c) {
    return r < MiRows && c < MiCols
}
```

```
check_chroma(bSize) {
    if ( get_plane_residual_size( bSize, 1 ) == BLOCK_INVALID ) {
        return 0
    }
    return ( TreeType == LUMA_PART &&
```

```

        Block_Width[ bSize ] >= 64 &&
        Block_Height[ bSize ] >= 64 )
    }

```

```

is_chroma_offset_for_subsize( subSize ) {
    if ( SubsamplingY && Mi_Height_Log2[ subSize ] == 0 ) {
        return 1
    }
    if ( SubsamplingX && Mi_Width_Log2[ subSize ] == 0 ) {
        return 1
    }
    return 0
}

```

```

is_chroma_offset_for_partition( p, bSize ) {
    if ( is_chroma_offset_for_subsize( Partition_Subsize[ p ][ bSize ] ) ) {
        return 1
    }
    if ( p == PARTITION_HORZ_3 ) {
        middleChroma = bSize == BLOCK_8X32 && SubsamplingX
        if ( !middleChroma ) {
            if ( is_chroma_offset_for_subsize( H_Partition_Midsize[bSize] ) ) {
                return 1
            }
        }
    }
    return 0
}

```

```

is_partition_allowed(r,c,p,bSize,chromaOffset,hasChroma,numPlanes) {
    subSize = Partition_Subsize[ p ][ bSize ]
    if ( subSize == BLOCK_INVALID ) {
        return 0
    }
    if ( !FrameIsIntra && RegionType == MIXED_REGION && subSize == BLOCK_4X4 ) {
        return 0
    }
    rectType = rect_type_implied_by_bsize( bSize )
    if ( rectType == RECT_VERT &&
        (p == PARTITION_HORZ ||
         p == PARTITION_HORZ_3 ||
         p == PARTITION_HORZ_4A ||
         p == PARTITION_HORZ_4B) ) {
        return 0
    }
    if ( rectType == RECT_HORZ &&
        (p == PARTITION_VERT ||
         p == PARTITION_VERT_3 ||
         p == PARTITION_VERT_4A ||
         p == PARTITION_VERT_4B) ) {
        return 0
    }
    bw = Block_Width[ subSize ]
    bh = Block_Height[ subSize ]
    if ( bw > bh * MaxPbAspectRatio || bh > bw * MaxPbAspectRatio ) {
        if ( p == PARTITION_NONE ) {
            return 0
        }
        if ( bw >= bh * 8 || bh >= bw * 8 ) {
            return 0
        }
    }
    num4x4wide = Num_4x4_Blocks_Wide[ bSize ]
    num4x4high = Num_4x4_Blocks_High[ bSize ]
    halfBlock4x4wide = num4x4wide >> 1
    halfBlock4x4high = num4x4high >> 1
    if ( hasChroma && TreeType != CHROMA_PART ) {
        if ( !chromaOffset ) {

```

```

        chromaOffset = is_chroma_offset_for_partition( p, bSize )
    }
}
if ( (hasChroma && !chromaOffset && TreeType != LUMA_PART) ||
    check_chroma(bSize) ) {
    if ( get_plane_residual_size( subSize, 1 ) == BLOCK_INVALID ) {
        return 0
    }
}
if ( p == PARTITION_HORZ_3 ) {
    if ( !is_ext_partition_allowed( bSize, RECT_HORZ ) ) {
        return 0
    }
}
else if ( p == PARTITION_VERT_3 ) {
    if ( !is_ext_partition_allowed( bSize, RECT_VERT ) ) {
        return 0
    }
}
else if ( p == PARTITION_HORZ_4A || p == PARTITION_HORZ_4B ) {
    if ( !is_ext_partition_allowed( bSize, RECT_HORZ ) ||
        !is_uneven_4way_partition_allowed( bSize, RECT_HORZ ) ) {
        return 0
    }
}
else if ( p == PARTITION_VERT_4A || p == PARTITION_VERT_4B ) {
    if ( !is_ext_partition_allowed( bSize, RECT_VERT ) ||
        !is_uneven_4way_partition_allowed( bSize, RECT_VERT ) ) {
        return 0
    }
}
else if ( p == PARTITION_NONE ) {
    hasRows = ( r + halfBlock4x4high ) < MiRows
    hasCols = ( c + halfBlock4x4wide ) < MiCols
    if ( (TreeType != CHROMA_PART || bSize != BLOCK_8X8) &&
        (!hasRows || !hasCols) ) {
        return 0
    }
}
if ( hasChroma && TreeType != LUMA_PART && numPlanes > 1 ) {
    if ( chromaOffset ) {
        if ( p == PARTITION_HORZ ) {
            return block_coded( r + halfBlock4x4high, c )
        }
        else if ( p == PARTITION_VERT ) {
            return block_coded( r, c + halfBlock4x4wide )
        }
        else if ( p == PARTITION_HORZ_3 ) {
            return block_coded( r + 3 * (halfBlock4x4high >> 1), c )
        }
        else if ( p == PARTITION_VERT_3 ) {
            return block_coded( r, c + 3 * (halfBlock4x4wide >> 1) )
        }
        else if ( p == PARTITION_HORZ_4A || p == PARTITION_HORZ_4B ) {
            h4 = Num_4x4_Blocks_High[ subSize ]
            return block_coded( r + 7 * h4, c )
        }
        else if ( p == PARTITION_VERT_4A || p == PARTITION_VERT_4B ) {
            w4 = Num_4x4_Blocks_Wide[ subSize ]
            return block_coded( r, c + 7 * w4 )
        }
    }
}
return 1
}

```

```

init_allowed_partitions(r,c,bSize,chromaOffset,hasChroma) {
    numAllowed = 0
    for ( p = 0; p < EXT_PARTITION_TYPES; p++ ) {
        good = is_partition_allowed(r,c,p,bSize,chromaOffset,
                                   hasChroma,NumPlanes)

        numAllowed += good
        allowed[ p ] = good
    }
    if ( numAllowed == 0 ) {
        allowed[ PARTITION_NONE ] = 1
        numAllowed = 1
    }
}

```

```

    }
    return (numAllowed,allowed)
}

```

```

rect_type_implied_by_bsize(bSize) {
    if ( bSize == BLOCK_4X8 || bSize == BLOCK_64X128 ||
        bSize == BLOCK_128X256 || bSize == BLOCK_4X16 ) {
        return RECT_HORZ
    }
    if ( bSize == BLOCK_8X4 || bSize == BLOCK_128X64 ||
        bSize == BLOCK_256X128 || bSize == BLOCK_16X4 ) {
        return RECT_VERT
    }
    if ( TreeType == CHROMA_PART ) {
        if ( bSize == BLOCK_8X16 || bSize == BLOCK_8X32 ) {
            return RECT_HORZ
        }
        if ( bSize == BLOCK_16X8 || bSize == BLOCK_32X8 ) {
            return RECT_VERT
        }
    }
    return RECT_INVALID
}

```

```

is_ext_partition_allowed(bSize, rectType) {
    if ( !enable_ext_partitions ) {
        return 0
    }
    return TreeType != CHROMA_PART ||
        (rectType == RECT_HORZ &&
         Block_Height[ bSize ] > 16 && Block_Width[ bSize ] > 8) ||
        (rectType == RECT_VERT &&
         Block_Width[ bSize ] > 16 && Block_Height[ bSize ] > 8)
}

```

```

partition_implied_at_boundary(r, c, bSize) {
    numWide4x4 = Num_4x4_Blocks_Wide[ bSize ]
    numHigh4x4 = Num_4x4_Blocks_High[ bSize ]
    hasRows = ( r + (numHigh4x4 >> 1) ) < MiRows
    hasCols = ( c + (numWide4x4 >> 1) ) < MiCols
    if ( hasRows && hasCols ) {
        return (0, PARTITION_NONE)
    }
    impliedPartition = PARTITION_NONE
    if ( numWide4x4 == numHigh4x4 ) {
        impliedPartition = hasRows ? PARTITION_VERT : PARTITION_HORZ
    } else if ( numHigh4x4 > numWide4x4 ) {
        if ( !hasRows ) {
            impliedPartition = PARTITION_HORZ
        } else {
            subHasCols = ( c + (numWide4x4 >> 2) ) < MiCols
            if ( numWide4x4 >= 4 && !subHasCols ) {
                impliedPartition = PARTITION_HORZ
            }
        }
    } else {
        if ( !hasCols ) {
            impliedPartition = PARTITION_VERT
        } else {
            subHasRows = ( r + (numHigh4x4 >> 2) ) < MiRows
            if ( numHigh4x4 >= 4 && !subHasRows ) {
                impliedPartition = PARTITION_VERT
            }
        }
    }
}

```



```

    }
    return (impliedPartition != PARTITION_NONE, impliedPartition)
}

```

```

partition_implied(r, c, bSize) {
    if ( bSize == BLOCK_4X4 || bSize >= BLOCK_4X32 ) {
        return (1, PARTITION_NONE)
    }
    if ( TreeType == CHROMA_PART && bSize == BLOCK_8X8 ) {
        return (1, PARTITION_NONE)
    }
    if ( TreeType == CHROMA_PART && bSize == BLOCK_64X64 &&
        ChromaPartitionKnown[ r ][ c ] ) {
        return (1, LumaPartitions[ r ][ c ])
    }
    return partition_implied_at_boundary(r, c, bSize)
}

```

```

is_uneven_4way_partition_allowed(bSize, rectType) {
    if ( !enable_uneven_4way_partitions ) {
        return 0
    }
    return TreeType != CHROMA_PART ||
        (rectType == RECT_HORZ && Block_Height[ bSize ] == 64) ||
        (rectType == RECT_VERT && Block_Width[ bSize ] == 64)
}

```

§ 5.20.4. Block decoding structures

§ 5.20.4.1. Decode block syntax

decode_block(r, c, subSize) {	Descriptor
MiRow = r	
MiCol = c	
MiSize = subSize	
bw4 = Num_4x4_Blocks_Wide[subSize]	
bh4 = Num_4x4_Blocks_High[subSize]	
update_ibc_buffers(r, c)	
for (y = 0; y < bh4; y++) {	
for (x = 0; x < bw4; x++) {	
IBCCoded[r + y][x + c] = 1	
}	
}	
if (HasChroma) {	
AvailUChroma = is_inside(ChromaMiRow - 1, ChromaMiCol)	
AvailLChroma = is_inside(ChromaMiRow, ChromaMiCol - 1)	
} else {	
AvailUChroma = 0	
AvailLChroma = 0	
}	
NNum = 0	
NNumBuf = 0	

add_neighbor(r + bh4 - 1, c - 1)	
add_neighbor(r - 1, c + bw4 - 1)	
add_neighbor(r, c - 1)	
add_neighbor(r - 1, c)	
for (n = 0; n < NNumBuf; n++) {	
for (list = 0; list < 2; list++) {	
NRefFrame[n][list] =	
RefFrames[NPosBuf[n][0]][NPosBuf[n][1]][list]	
}	
NIntra[n] = !IsInters[NPosBuf[n][0]][NPosBuf[n][1]]	
NSingle[n] = !is_inter_ref_frame(NRefFrame[n][1])	
}	
mode_info()	
palette_tokens()	
if (TreeType != CHROMA_PART) {	
read_block_tx_size()	
}	
if (skip_flag) {	
reset_block_context(bw4, bh4)	
}	
isCompound = is_inter_ref_frame(RefFrame[1])	
for (y = 0; y < bh4; y++) {	
for (x = 0; x < bw4; x++) {	
if (PlaneStart == 0) {	
IntraJointModes[r + y][c + x] = IntraJointMode	
YModes [r + y][c + x] = YMode	
AngleDeltaYs[r + y][c + x] = AngleDeltaY	
for (refList = 0; refList < 2; refList++) {	
RefFrames[r + y][c + x][refList] = RefFrame[refList]	
}	
MiSizes[0][r + y][c + x] = MiSize	
w = bw4 * 4	
h = bh4 * 4	
TipSizes16x16[r + y][c + x] = enable_tip_refinemv ?	
(w == 256 && h == 256) :	
(w >= 16 && h >= 16)	
LeftMiSizes[0][r + y] = MiSize	
AboveMiSizes[0][c + x] = MiSize	
MiColStartGrid[r + y][c + x] = MiColStart	
MiRowStartGrid[r + y][c + x] = MiRowStart	
MiColEndGrid[r + y][c + x] = MiColEnd	

MiRowEndGrid[r + y][c + x] = MiRowEnd	
MiColBase[0][r + y][c + x] = MiCol	
MiRowBase[0][r + y][c + x] = MiRow	
if (is_inter) {	
if (!use_intrabc) {	
CompGroupIdxs[r + y][c + x] = comp_group_idx	
}	
InterpFilters[r + y][c + x] = interp_filter	
for (refList = 0; refList < 1 + isCompound; refList++) {	
Mvs[r + y][c + x][refList] = BlockMvs[refList]	
SubMvs[r + y][c + x][refList] = BlockMvs[refList]	
}	
}	
SubPuColBase[0][r + y][c + x] = c	
SubPuRowBase[0][r + y][c + x] = r	
SubPuSize[0][r + y][c + x] = Max_Tx_Size_Rect[MiSize]	
}	
}	
if (HasChroma) {	
uvSmooth = !is_inter && (UVMode == SMOOTH_PRED	
UVMode == SMOOTH_V_PRED UVMode == SMOOTH_H_PRED)	
for (y = 0; y < Num_4x4_Blocks_High[ChromaMiSize]; y++) {	
for (x = 0; x < Num_4x4_Blocks_Wide[ChromaMiSize]; x++) {	
MiSizes[1][ChromaMiRow + y][ChromaMiCol + x] =	
ChromaMiSize	
LeftMiSizes[1][ChromaMiRow + y] = ChromaMiSize	
AboveMiSizes[1][ChromaMiCol + x] = ChromaMiSize	
MiColBase[1][ChromaMiRow + y][ChromaMiCol + x] =	
ChromaMiCol	
MiRowBase[1][ChromaMiRow + y][ChromaMiCol + x] =	
ChromaMiRow	
UVSmooth[ChromaMiRow + y][ChromaMiCol + x] = uvSmooth	
UVCfls[ChromaMiRow + y][ChromaMiCol + x] =	
!is_inter && (UVMode == UV_CFL_PRED)	
SubPuColBase[1][ChromaMiRow + y][ChromaMiCol + x] =	
ChromaMiCol	
SubPuRowBase[1][ChromaMiRow + y][ChromaMiCol + x] =	
ChromaMiRow	
SubPuSize[1][ChromaMiRow + y][ChromaMiCol + x] =	
Max_Tx_Size_Rect[ChromaMiSize]	

RegionTypes[ChromaMiRow + y][ChromaMiCol + x] =	
RegionType	
ChromaSegmentIds[ChromaMiRow + y][ChromaMiCol + x] =	
segment_id	
ChromaQIndex[ChromaMiRow + y][ChromaMiCol + x] =	
CurrentQIndex	
}	
}	
}	
compute_prediction()	
residual()	
if (is_inter && motion_mode >= LOCALWARP) {	
update_warp_param_bank(RefFrame, LocalWarpParams, 0)	
}	
if (enable_refmvbank &&	
bru_mode == BRU_ACTIVE &&	
RefMvBankHits < MAX_RMB_SB_HITS) {	
if (is_inter) {	
update_ref_mv_bank(RefFrame, BlockMvs, CwpIdx, 1)	
} else {	
update_ref_mv_count()	
}	
}	
for (y = 0; y < bh4; y++) {	
for (x = 0; x < bw4; x++) {	
if (PlaneStart == 0) {	
for (refList = 0; refList < 1 + isCompound; refList++) {	
for (i = 0; i < 6; i++) {	
WarpParams[r + y][c + x][refList][i] =	
LocalWarpParams[refList][i]	
}	
}	
IsInters[r + y][c + x] = is_inter	
SkipModes[r + y][c + x] = skip_mode	
Skips[r + y][c + x] = skip_flag	
CwpIdxs[r + y][c + x] = CwpIdx	
FscModes[r + y][c + x] = fsc_mode	
UsesMrls[r + y][c + x] =	
(mrl_index > 0 ? (mrl_sec_index ? 2 : 1) : 0)	
UsesAmvds[r + y][c + x] = use_amvd	
UseDip[r + y][c + x] = use_dip	

UseMostProbablePrecisions[r + y][c + x] =	
use_most_probable_precision	
MvPrecisions[r + y][c + x] =	
use_intrabc ? FrameMvPrecision : MvPrecision	
MorphPreds[r + y][c + x] = use_intrabc && morph_pred	
SegmentIds[r + y][c + x] = segment_id	
PaletteSizes[r + y][c + x] = PaletteSizeY	
for (i = 0; i < PaletteSizeY; i++) {	
PaletteColors[r + y][c + x][i] =	
palette_colors_y[i]	
}	
MotionModes[r + y][c + x] = motion_mode	
LumaQIndex[r + y][c + x] = CurrentQIndex	
}	
}	
}	
if (PlaneStart == 0) {	
if (isCompound && opfl_allowed_for_refs(RefFrame) && use_optflow) {	
motion_field_motion_vector_storage(r, c, subSize, 1)	
} else if (isCompound && compound_type == COMPOUND_AVERAGE &&	
use_refinemv) {	
motion_field_motion_vector_storage(r, c, subSize, 2)	
} else if (RefFrame[0] == TIP_FRAME) {	
if (store_refined_mvs()) {	
motion_field_motion_vector_storage(r, c, subSize,	
LumaUseOptflowRefinement ? 1 : 2)	
} else {	
motion_field_motion_vector_storage(r, c, subSize, 0)	
}	
} else {	
motion_field_motion_vector_storage(r, c, subSize, 0)	
}	
}	
}	

where reset_block_context() is specified as:

```

reset_block_context( bw4, bh4 ) {
    for ( plane = 0; plane < 1 + 2 * HasChroma; plane++ ) {
        c = plane > 0 ? ChromaMiCol : MiCol
        r = plane > 0 ? ChromaMiRow : MiRow
        w4 = plane > 0 ? Num_4x4_Blocks_Wide[ ChromaMiSize ] : bw4
        h4 = plane > 0 ? Num_4x4_Blocks_High[ ChromaMiSize ] : bh4
        subX = plane > 0 ? SubsamplingX : 0
        subY = plane > 0 ? SubsamplingY : 0
    }
}

```

```

    for ( i = c >> subX; i < ( ( c + w4 ) >> subX ); i++) {
        AboveLevelContext[ plane ][ i ] = 0
        AboveDcContext[ plane ][ i ] = 0
    }
    for ( i = r >> subY; i < ( ( r + h4 ) >> subY ); i++) {
        LeftLevelContext[ plane ][ i ] = 0
        LeftDcContext[ plane ][ i ] = 0
    }
}
}
}

```

update_warp_param_bank is specified as:

```

update_warp_param_bank( refFrames , params, candFromSbAbove ) {
    isCompound = is_inter_ref_frame( refFrames[ 1 ] ) && !candFromSbAbove
    for ( refList = 0; refList < 1 + isCompound; refList++ ) {
        if ( WarpBankHits >= MAX_WARP_SB_HITS ) {
            return
        }
        WarpBankHits++
        ref = refFrames[ refList ]
        found = -1
        count = WarpBankSize[ ref ]
        start = WarpBankStart[ ref ]
        for ( i = 0; i < count; i++ ) {
            idx = (start + i) % WARP_PARAM_BANK_SIZE
            if ( params_equal( WarpBankParams[ ref ][ idx ],
                             params[ refList ] ) ) {
                found = i
                break
            }
        }
        if ( found >= 0 ) {
            for ( j = 0; j < 6; j++ ) {
                tmpParams[ j ] = WarpBankParams[ ref ][ idx ][ j ]
            }
            for ( i = found; i < count - 1; i++ ) {
                idx0 = (start + i) % WARP_PARAM_BANK_SIZE
                idx1 = (start + i + 1) % WARP_PARAM_BANK_SIZE
                for ( j = 0; j < 6; j++ ) {
                    WarpBankParams[ ref ][ idx0 ][ j ] =
                        WarpBankParams[ ref ][ idx1 ][ j ]
                }
            }
            tail = (start + count - 1) % WARP_PARAM_BANK_SIZE
            for ( j = 0; j < 6; j++ ) {
                WarpBankParams[ ref ][ tail ][ j ] = tmpParams[ j ]
            }
        } else {
            idx = (start + count) % WARP_PARAM_BANK_SIZE
            for ( j = 0; j < 6; j++ ) {
                WarpBankParams[ ref ][ idx ][ j ] = params[ refList ][ j ]
            }
            if ( count < WARP_PARAM_BANK_SIZE ) {
                WarpBankSize[ ref ] = count + 1
            } else {
                WarpBankStart[ ref ] = (start + 1) % WARP_PARAM_BANK_SIZE
            }
        }
    }
}
}
}

```

The function `params_equal` (which checks if the non-translational parts of two warps are equal) is defined as:

```
params_equal( paramsA , paramsB ) {
    for ( i = 2; i < 6; i++ ) {
        if ( paramsA[ i ] != paramsB[ i ] ) {
            return 0
        }
    }
    return 1
}
```

`update_ref_mv_bank` (which ensures the current parameters are at the tail of the appropriate bank of motion vectors) is specified as:

```
update_ref_mv_bank( refFrames, mvs, cwpIdx, fromWithinSb ) {
    if ( fromWithinSb ) {
        update_ref_mv_count( )
        if ( RefMvRemainHits == 0 || RefMvUnitHits >= 16 ) {
            return
        }
        RefMvRemainHits--
        RefMvUnitHits++
    }
    RefMvBankHits++
    r0 = refFrames[ 0 ]
    r1 = refFrames[ 1 ]
    isCompound = is_inter_ref_frame(r1)
    ref = get_rmb_list_index( refFrames )
    for ( i = 0; i < 6; i++ ) {
        p[ i ] = 0
    }
    p[ 0 ] = cwpIdx
    p[ 1 ] = isCompound ? r0 + (r1 + 1) * BANK_REFS_PER_FRAME : r0
    p[ 2 ] = mvs[ 0 ][ 0 ]
    p[ 3 ] = mvs[ 0 ][ 1 ]
    if ( isCompound ) {
        p[ 4 ] = mvs[ 1 ][ 0 ]
        p[ 5 ] = mvs[ 1 ][ 1 ]
    }
    found = -1
    count = RefMvBankSize[ ref ]
    start = RefMvBankStart[ ref ]
    for ( i = 0; i < count; i++ ) {
        idx = (start + i) % REF_MV_BANK_SIZE
        if ( rmb_params_equal(RefMvBankParams[ ref ][ idx ], p) ) {
            found = i
            break
        }
    }
    if ( found >= 0 ) {
        for ( i = 0; i < 6; i++ ) {
            tmpParams[ i ] = RefMvBankParams[ ref ][ idx ][ i ]
        }
        for ( i = found; i < count - 1; i++ ) {
            idx0 = (start + i) % REF_MV_BANK_SIZE
            idx1 = (start + i + 1) % REF_MV_BANK_SIZE
            for ( j = 0; j < 6; j++ ) {
                RefMvBankParams[ ref ][ idx0 ][ j ] =
                    RefMvBankParams[ ref ][ idx1 ][ j ]
            }
        }
        tail = (start + count - 1) % REF_MV_BANK_SIZE
        for ( j = 0; j < 6; j++ ) {
            RefMvBankParams[ ref ][ tail ][ j ] = tmpParams[ j ]
        }
    }
}
```

```

        return
    }
    idx = (start + count) % REF_MV_BANK_SIZE
    for ( j = 0; j < 6; j++ ) {
        RefMvBankParams[ ref ][ idx ][ j ] = p[ j ]
    }
    if ( count < REF_MV_BANK_SIZE ) {
        RefMvBankSize[ ref ] = count + 1
    } else {
        RefMvBankStart[ ref ] = (start + 1) % REF_MV_BANK_SIZE
    }
}

```

add_neighbor is specified as:

```

add_neighbor(nRow, nCol) {
    aboveSbBoundary = (MiRow >> Mi_Width_Log2[ SbSize ]) !=
        (nRow >> Mi_Width_Log2[ SbSize ])
    if ( NNum < 2 && is_inside(nRow,nCol) && !aboveSbBoundary ) {
        NPos[ NNum ][ 0 ] = nRow
        NPos[ NNum ][ 1 ] = nCol
        NNum += 1
    }
    if ( NNumBuf < 2 && is_inside(nRow,nCol) ) {
        NPosBuf[ NNumBuf ][ 0 ] = nRow
        NPosBuf[ NNumBuf ][ 1 ] = nCol
        NNumBuf += 1
    }
}

```

NOTE: NPos will only contain locations that are in the same superblock row as the current block. NPosBuf contains locations that may require buffered access to a different superblock row.

update_ref_mv_count is specified as:

```

update_ref_mv_count() {
    if ( TreeType != CHROMA_PART ) {
        sbSize4 = Num_4x4_Blocks_Wide[ SbSize ]
        unitSize4 = sbSize4 >> 3
        unitCount = Max( Num_4x4_Blocks_Wide[ MiSize ] / unitSize4 , 1 ) *
            Max( Num_4x4_Blocks_High[ MiSize ] / unitSize4 , 1 )
        if ( MiRow % sbSize4 == 0 && MiCol % sbSize4 == 0 ) {
            RefMvRemainHits = Max( unitCount , 4 )
            RefMvUnitHits = 0
        } else if ( MiRow % unitSize4 == 0 && MiCol % unitSize4 == 0 ) {
            RefMvRemainHits += unitCount
            RefMvUnitHits = 0
        }
    }
}

```

rmb_params_equal is specified as:

```

rmb_params_equal( paramsA, paramsB ) {
    for ( i = 1; i < 6; i++ ) {
        if ( paramsA[ i ] != paramsB[ i ] ) {
            return 0
        }
    }
    return 1
}

```


update_ibc_buffers is specified as:

```
update_ibc_buffers(miRow, miCol) {
    bufRow = miRow >> (IBC_BUFFER_SIZE_LOG2 - MI_SIZE_LOG2)
    bufCol = miCol >> (IBC_BUFFER_SIZE_LOG2 - MI_SIZE_LOG2)
    if ( bufRow != IBCBufferCurRow || bufCol != IBCBufferCurCol ) {
        blkIdx = ibc_buffer_index(IBCBufferCurRow, IBCBufferCurCol)
        IBCBufferRow[ blkIdx ] = IBCBufferCurRow
        IBCBufferCol[ blkIdx ] = IBCBufferCurCol
        IBCBufferValid[ blkIdx ] = 1
        if ( SbSize == BLOCK_64X64 ) {
            bruRow = IBCBufferCurRow << (IBC_BUFFER_SIZE_LOG2 - MI_SIZE_LOG2)
            bruCol = IBCBufferCurCol << (IBC_BUFFER_SIZE_LOG2 - MI_SIZE_LOG2)
            if ( BruModes[ bruRow ][ bruCol ] == BRU_INACTIVE ) {
                for ( i = 0; i < IBC_NUM_BUFFERS; i++ ) {
                    IBCBufferValid[ i ] = 0
                }
            }
        }
        IBCBufferCurRow = bufRow
        IBCBufferCurCol = bufCol
    }
}
```

NOTE: Calls to update_ibc_buffers are only needed for bitstream conformance checks. However, a decoder implementation may wish to use the same logic for updating a local cache of information available for intra block copy.

ibc_buffer_index is specified as:

```
ibc_buffer_index(row, col) {
    if ( SbSize == BLOCK_64X64 ) {
        return col & 3
    } else {
        return (col & 1) | ((row & 1) << 1)
    }
}
```

store_refined_mvs is specified as:

```
store_refined_mvs() {
    return Tip_Weighting_Factor[ tip_global_wtd_index ] == CWP_EQUAL &&
        enable_tip_refinemv && NumFutureRefs > 0 && NumPastRefs > 0
}
```

§ 5.20.5. Mode information structures

§ 5.20.5.1. Mode info syntax

mode_info() {	Descriptor
if (bru_mode != BRU_ACTIVE) {	
bru_mode_info()	
} else if (FrameIsIntra) {	
intra_frame_mode_info()	
} else {	
inter_frame_mode_info()	

}	
}	

§ 5.20.5.2. BRU mode info syntax

bru_mode_info() {	Descriptor
use_intrabc = 0	
skip_flag = 1	
segment_id = 0	
Lossless = LosslessArray[segment_id]	
skip_mode = 0	
is_inter = 1	
RefFrame[0] = IsBridge ? 0 : bru_ref	
RefFrame[1] = NONE	
mrl_index = 0	
use_dip = 0	
fsc_mode = 0	
use_dpcm_y = 0	
use_dpcm_uv = 0	
PaletteSizeY = 0	
MvPrecision = MV_PRECISION_ONE_PEL	
use_most_probable_precision = 0	
IntraJointMode = DC_PRED	
use_bawp = 0	
use_amvd = 0	
CwpIdx = CWP_EQUAL	
CurrentQIndex = base_q_idx	
use_optflow = 0	
use_refinemv = 0	
YMode = NEWMV	
motion_mode = SIMPLE	
BlockMvs[0][0] = 0	
BlockMvs[0][1] = 0	
interp_filter = EIGHTTAP_SHARP	
read_gdf()	
read_ccso()	
for (r = MiRow; r < MiRow + Num_4x4_Blocks_High[MiSize]; r++) {	
LeftSegPredContext[r] = 0	
}	
for (c = MiCol; c < MiCol + Num_4x4_Blocks_Wide[MiSize]; c++) {	
AboveSegPredContext[c] = 0	
}	

}	
---	--

§ 5.20.5.3. Intra frame mode info syntax

intra_frame_mode_info() {	Descriptor
skip_flag = 0	
if (SegIdPreSkip) {	
intra_segment_id()	
}	
skip_mode = 0	
use_most_probable_precision = 0	
MvPrecision = FrameMvPrecision	
CwpIdx = CWP_EQUAL	
motion_mode = SIMPLE	
if (allow_intrabc && TreeType != CHROMA_PART &&	
Block_Width[MiSize] <= 64 &&	
Block_Height[MiSize] <= 64 &&	
MiSize != BLOCK_64X64) {	
use_intrabc	S()
} else {	
use_intrabc = 0	
}	
if (use_intrabc) {	
read_skip()	
} else {	
skip_flag = 0	
}	
if (!SegIdPreSkip) {	
intra_segment_id()	
}	
if (TreeType != CHROMA_PART) {	
read_gdf()	
read_cdef()	
read_ccso()	
read_delta_qindex()	
}	
ReadDeltas = 0	
RefFrame[0] = INTRA_FRAME	
RefFrame[1] = NONE	
fsc_mode = 0	
if (use_intrabc) {	
is_inter = 1	

mrl_index = 0	
read_intrabc_info()	
} else {	
is_inter = 0	
PaletteSizeY = 0	
if (TreeType != CHROMA_PART) {	
read_intra_y_mode()	
} else {	
YMode = YModes[MiRow][MiCol]	
AngleDeltaY = AngleDeltaYs[MiRow][MiCol]	
PaletteSizeY = PaletteSizes[MiRow][MiCol]	
}	
if (HasChroma) {	
read_intra_uv_mode()	
if (UVMode == UV_CFL_PRED) {	
read_cfl_alphas()	
}	
}	
if (MiSize >= BLOCK_8X8 &&	
Block_Width[MiSize] <= 64 &&	
Block_Height[MiSize] <= 64 &&	
allow_screen_content_tools) {	
palette_mode_info()	
}	
if (TreeType != CHROMA_PART) {	
dip_mode_info()	
}	
}	
}	

§ 5.20.5.4. Read intra block copy syntax

read_intrabc_info() {	Descriptor
IntraJointMode = DC_PRED	
mrl_index = 0	
use_dip = 0	
fsc_mode = 0	
AngleDeltaY = 0	
use_bawp = 0	
use_amvd = 0	
warpmv_with_mvd = 0	
use_refinemv = 0	

DecidedAgainstRefinemv = 0	
use_dpcm_y = 0	
use_dpcm_uv = 0	
CwpIdx = CWP_EQUAL	
YMode = DC_PRED	
UVMode = DC_PRED	
motion_mode = SIMPLE	
compound_type = COMPOUND_AVERAGE	
PaletteSizeY = 0	
interp_filter = BILINEAR	
RefFrame[0] = INTRA_FRAME	
RefFrame[1] = NONE	
MvPrecision = force_integer_mv ? MV_PRECISION_ONE_PEL :	
MV_PRECISION_QUARTER_PEL	
use_most_probable_precision = 0	
DeriveWrl = 0	
IsAdaptiveMvd = 0	
find_mv_stack(0)	
m = max_bvp_drl_bits_minus_1 + 1	
intrabc_mode	S()
RefMvIdx = 0	
for (idx = 0; idx < m; idx++) {	
intrabc_drl_mode	L(1)
if (intrabc_drl_mode == 0) {	
RefMvIdx = idx	
break	
}	
RefMvIdx = idx + 1	
}	
if (intrabc_mode == 0 && !force_integer_mv) {	
intrabc_precision	S()
MvPrecision = intrabc_precision ? MV_PRECISION_QUARTER_PEL :	
MV_PRECISION_ONE_PEL	
}	
assign_mv(0)	
if (FrameIsIntra && allow_screen_content_tools && enable_bawp) {	
morph_pred	S()
} else {	
morph_pred = 0	
}	
}	

§ 5.20.5.5. Read intra Y mode syntax

read_intra_y_mode() {	Descriptor
if (Lossless) {	
use_dpcm_y	S()
} else {	
use_dpcm_y = 0	
}	
if (use_dpcm_y) {	
dpcm_mode_y	S()
AngleDeltaY = 0	
mrl_index = 0	
if (dpcm_mode_y) {	
YMode = H_PRED	
IntraJointMode = 50	
} else {	
YMode = V_PRED	
IntraJointMode = 22	
}	
if (allow_fsc_intra()) {	
fsc_mode	S()
}	
return	
}	
y_mode_set	S()
if (y_mode_set == 0) {	
y_mode_index	S()
modeIdx = y_mode_index	
if (y_mode_index == MODE_INDEX_COUNT - 1) {	
y_mode_offset	S()
modeIdx += y_mode_offset	
}	
} else {	
y_second_mode	L(4)
modeIdx = FIRST_MODE_COUNT + (y_mode_set - 1) * SECOND_MODE_COUNT +	
y_second_mode	
}	
modeDelta = get_intra_y_mode_set(modeIdx)	
IntraJointMode = modeDelta	
if (modeDelta < NON_DIRECTIONAL_MODES_COUNT) {	
YMode = Reordered_Y_Mode[modeDelta]	
AngleDeltaY = 0	

} else {	
modeDelta -= NON_DIRECTIONAL_MODES_COUNT	
YMode = Reordered_Y_Mode[modeDelta / TOTAL_ANGLE_DELTA_COUNT +	
NON_DIRECTIONAL_MODES_COUNT]	
AngleDeltaY = (modeDelta % TOTAL_ANGLE_DELTA_COUNT) - MAX_ANGLE_DELTA	
}	
if (TreeType != CHROMA_PART && allow_fsc_intra()) {	
fsc_mode	S()
}	
if (enable_mrls && is_directional_mode(YMode)) {	
mrl_index	S()
if (mrl_index > 0) {	
mrl_sec_index	S()
}	
} else {	
mrl_index = 0	
}	
}	

where Reordered_Y_Mode, Default_Mode_List_Y, get_intra_y_mode_set, get_joint_mode, and allow_fsc_intra are defined as:

```
Reordered_Y_Mode[ INTRA_MODES ] = {
    DC_PRED,    SMOOTH_PRED, SMOOTH_V_PRED, SMOOTH_H_PRED, PAETH_PRED,
    D45_PRED,   D67_PRED,   V_PRED,        D113_PRED,   D135_PRED,
    D157_PRED,  H_PRED,     D203_PRED
}
```

```
Default_Mode_List_Y[ DIRECTIONAL_MODES_COUNT ] = {
    17, 45, 3, 10, 24, 31, 38, 52,
    15, 19, 43, 47, 1, 5, 8, 12, 22, 26, 29, 33, 36, 40, 50, 54,
    16, 18, 44, 46, 2, 4, 9, 11, 23, 25, 30, 32, 37, 39, 51, 53,
    14, 20, 42, 48, 0, 6, 7, 13, 21, 27, 28, 34, 35, 41, 49, 55
}
```

```
get_joint_mode( dir ) {
    if ( dir ) {
        mvRow = MiRow - 1
        mvCol = MiCol + Num_4x4_Blocks_Wide[ MiSize ] - 1
    } else {
        mvCol = MiCol - 1
        mvRow = MiRow + Num_4x4_Blocks_High[ MiSize ] - 1
    }
    if ( is_inside( mvRow, mvCol ) ) {
        return IntraJointModes[ mvRow ][ mvCol ]
    }
    return DC_PRED
}
```

```
get_intra_y_mode_set( modeIdx ) {
    if ( modeIdx < NON_DIRECTIONAL_MODES_COUNT ) {
        return modeIdx
    }
    modeIdx -= NON_DIRECTIONAL_MODES_COUNT
    for ( i = 0; i < DIRECTIONAL_MODES_COUNT; i++ ) {
```

```

        isDirSelected[ i ] = 0
    }
    if ( MiSize >= BLOCK_8X8 ) {
        count = 0
        for ( dir = 0; dir < 2; dir++ ) {
            mode = get_joint_mode( dir )
            if ( mode >= NON_DIRECTIONAL_MODES_COUNT ) {
                mode -= NON_DIRECTIONAL_MODES_COUNT
                if ( count == 0 || mode != dirModes[ 0 ] ) {
                    if ( modeIdx == 0 ) {
                        return mode + NON_DIRECTIONAL_MODES_COUNT
                    }
                    modeIdx -= 1
                    isDirSelected[ mode ] = 1
                    dirModes[ count ] = mode
                    count += 1
                }
            }
        }
    }
    if ( Block_Width[ MiSize ] * Block_Height[ MiSize ] > 64 ) {
        for ( i = 1; i <= 4; i++ ) {
            for ( j = 0; j < count; j++ ) {
                for ( sgn = -1; sgn <= 1; sgn += 2 ) {
                    mode = dirModes[ j ] + i * sgn
                    if ( mode < 0 ) {
                        mode += DIRECTIONAL_MODES_COUNT
                    }
                    else if ( mode >= DIRECTIONAL_MODES_COUNT )
                        mode -= DIRECTIONAL_MODES_COUNT
                    if ( !isDirSelected[ mode ] ) {
                        if ( modeIdx == 0 ) {
                            return mode + NON_DIRECTIONAL_MODES_COUNT
                        }
                        modeIdx -= 1
                        isDirSelected[ mode ] = 1
                    }
                }
            }
        }
    }
    for ( i = 0; i < DIRECTIONAL_MODES_COUNT; i++ ) {
        mode = Default_Mode_List_Y[ i ]
        if ( !isDirSelected[ mode ] ) {
            if ( modeIdx == 0 ) {
                return mode + NON_DIRECTIONAL_MODES_COUNT
            }
            modeIdx -= 1
        }
    }
}

allow_fsc_intra( ) {
    w = Block_Width[ MiSize ]
    h = Block_Height[ MiSize ]
    return enable_idtx_intra && w <= FSC_MAX && h <= FSC_MAX
}

```

§ 5.20.5.6. Read intra UV mode syntax

read_intra_uv_mode() {	Descriptor
if (Lossless) {	
use_dpcm_uv	S()
} else {	
use_dpcm_uv = 0	

}	
if (use_dpcm_uv) {	
dpcm_mode_uv	S()
if (dpcm_mode_uv) {	
UVMode = H_PRED	
} else {	
UVMode = V_PRED	
}	
if (UVMode == YMode) {	
AngleDeltaUV = AngleDeltaY	
} else {	
AngleDeltaUV = 0	
}	
return	
}	
planeSz = get_plane_residual_size(ChromaMiSize, 1)	
if (!enable_cfl_intra) {	
cflAllowed = 0	
} else if (TreeType == CHROMA_PART && FrameIsIntra && !CflAllowedInSdp) {	
cflAllowed = 0	
} else if (Lossless) {	
cflAllowed = planeSz == BLOCK_4X4	
} else {	
cflAllowed = Block_Width[planeSz] <= 64 &&	
Block_Height[planeSz] <= 64	
}	
if (cflAllowed is_mhccp_allowed()) {	
is_cfl	S()
if (is_cfl) {	
AngleDeltaUV = 0	
UVMode = UV_CFL_PRED	
return	
}	
}	
uv_mode	S()
if (uv_mode == CHROMA_MODE_COUNT - 1) {	
uv_mode_idx	L(3)
uv_mode += uv_mode_idx	
}	
UVMode = get_intra_uv_mode_set(uv_mode)	
if (UVMode == YMode) {	

AngleDeltaUV = AngleDeltaY	
} else {	
AngleDeltaUV = 0	
}	
}	

where Default_Mode_List_Uv, get_intra_uv_mode_set, and is_mhccp_allowed are defined as:

```
Default_Mode_List_Uv[ UV_INTRA_MODES_CFL_NOT_ALLOWED ] = {
    DC_PRED, SMOOTH_PRED, SMOOTH_V_PRED, SMOOTH_H_PRED, PAETH_PRED,
    V_PRED,   H_PRED,   D45_PRED,   D135_PRED,
    D67_PRED, D113_PRED, D157_PRED, D203_PRED
}
```

```
get_intra_uv_mode_set( modeIdx ) {
    if ( is_directional_mode( YMode ) ) {
        if ( modeIdx == 0 ) {
            return YMode
        }
        modeIdx -= 1
    }
    for ( i = 0; i < UV_INTRA_MODES_CFL_NOT_ALLOWED; i++ ) {
        mode = Default_Mode_List_Uv[ i ]
        if ( mode != YMode || !is_directional_mode( YMode ) ) {
            if ( modeIdx == 0 ) {
                return mode
            }
            modeIdx -= 1
        }
    }
}
```

```
is_mhccp_allowed( ) {
    planeSz = get_plane_residual_size( ChromaMiSize, 1 )
    if ( !enable_mhccp ) {
        return 0
    } else if ( TreeType == CHROMA_PART && FrameIsIntra && !CflAllowedInSdp ) {
        return 0
    } else if ( Lossless ) {
        return planeSz == BLOCK_4X4
    } else {
        w = Block_Width[ planeSz ]
        h = Block_Height[ planeSz ]
        return ( w > 4 || h > 4 ) && w <= 32 && h <= 32
    }
}
```

§ 5.20.5.7. Intra segment ID syntax

intra_segment_id() {	Descriptor
if (TreeType == CHROMA_PART) {	
segment_id = SegmentIds[MiRow][MiCol]	
} else if (segmentation_enabled) {	
read_segment_id()	
} else {	
segment_id = 0	

}	
Lossless = LosslessArray[segment_id]	
}	

§ 5.20.5.8. Read segment ID syntax

read_segment_id() {	Descriptor
if (AvailU && AvailL) {	
prevUL = SegmentIds[MiRow - 1][MiCol - 1]	
} else {	
prevUL = -1	
}	
if (AvailU) {	
prevU = SegmentIds[MiRow - 1][MiCol]	
} else {	
prevU = -1	
}	
if (AvailL) {	
prevL = SegmentIds[MiRow][MiCol - 1]	
} else {	
prevL = -1	
}	
if (prevU == -1) {	
pred = (prevL == -1) ? 0 : prevL	
} else if (prevL == -1) {	
pred = prevU	
} else {	
pred = (prevUL == prevU) ? prevU : prevL	
}	
if (skip_flag && !HasLosslessSegment) {	
segment_id = pred	
} else {	
if (enable_ext_seg) {	
seg_id_ext_flag	S()
} else {	
seg_id_ext_flag = 0	
}	
segment_id	S()
if (seg_id_ext_flag) {	
segment_id += 8	
}	
segment_id = neg_deinterleave(segment_id, pred,	

LastActiveSegId + 1)	
}	
}	

where neg_deinterleave is a function defined as:

```
neg_deinterleave(diff, ref, max) {
    if ( !ref ) {
        return diff
    }
    if ( ref >= (max - 1) ) {
        return max - diff - 1
    }
    if ( 2 * ref < max ) {
        if ( diff <= 2 * ref ) {
            if ( diff & 1 ) {
                return ref + ((diff + 1) >> 1)
            } else {
                return ref - (diff >> 1)
            }
        }
        return diff
    } else {
        if ( diff <= 2 * (max - ref - 1) ) {
            if ( diff & 1 ) {
                return ref + ((diff + 1) >> 1)
            } else {
                return ref - (diff >> 1)
            }
        }
        return max - (diff + 1)
    }
}
```

§ 5.20.5.9. Skip mode syntax

read_skip_mode() {	Descriptor
if (seg_feature_active(SEG_LVL_SKIP)	
seg_feature_active(SEG_LVL_GLOBALMV)	
!skip_mode_present	
!is_comp_ref_allowed()	
RegionType == INTRA_REGION) {	
skip_mode = 0	
} else {	
skip_mode	S()
}	
}	

where is_comp_ref_allowed is a function that checks the block size as follows:

```
is_comp_ref_allowed( ) {
    w = Block_Width[ MiSize ]
    h = Block_Height[ MiSize ]
    return ( Min( w, h ) >= 8 ) || is_thin_4xn_nx4_block()
}
```

§ 5.20.5.10. Skip syntax

read_skip() {	Descriptor
if (SegIdPreSkip && seg_feature_active(SEG_LVL_SKIP)) {	
skip_flag = 1	
} else {	
skip_flag	S()
}	
}	

§ 5.20.5.11. Quantizer index delta syntax

read_delta_qindex() {	Descriptor
if (!(MiSize == SbSize && skip_flag) && ReadDeltas) {	
delta_q_abs	S()
if (delta_q_abs == DELTA_Q_SMALL) {	
delta_q_rem_bits	L(3)
delta_q_rem_bits++	
delta_q_abs_bits	L(delta_q_rem_bits)
delta_q_abs = delta_q_abs_bits + (1 << delta_q_rem_bits) +	
DELTA_Q_SMALL - 2	
}	
if (delta_q_abs) {	
delta_q_sign_bit	L(1)
reducedDeltaQIndex = delta_q_sign_bit ? -delta_q_abs : delta_q_abs	
CurrentQIndex = Clip3(1, MaxQ,	
CurrentQIndex + (reducedDeltaQIndex << delta_q_res))	
}	
}	
if (delta_q_present) {	
CurrentQIndex = Clip3(1, MaxQ, CurrentQIndex)	
}	
}	

§ 5.20.5.12. Segmentation feature active function

```

seg_feature_active_idx( idx, feature ) {
    return segmentation_enabled && FeatureEnabled[ idx ][ feature ]
}

seg_feature_active( feature ) {
    return seg_feature_active_idx( segment_id, feature )
}

```

§ 5.20.6. Transform and quantization structures

§ 5.20.6.1. TX size syntax

read_tx_size(allowSelect) {	Descriptor
if (Lossless) {	
if (MiSize == BLOCK_4X4	
(!is_inter && !fsc_mode)	
!allowSelect) {	
TxSize = TX_4X4	
} else {	
lossless_tx_size	S()
if (lossless_tx_size) {	
TxSize = find_tx_size(Min(32, Block_Width[MiSize]),	
Min(32, Block_Height[MiSize]))	
} else {	
TxSize = TX_4X4	
}	
}	
return 0	
}	
maxRectTxSize = Max_Tx_Size_Rect[MiSize]	
TxSize = maxRectTxSize	
if (MiSize > BLOCK_4X4 && allowSelect && TxMode == TX_MODE_SELECT) {	
widthChunks = Block_Width[MiSize] >> 6	
heightChunks = Block_Height[MiSize] >> 6	
if (widthChunks > 1 heightChunks > 1) {	
for (chunkY = 0; chunkY < heightChunks; chunkY++) {	
for (chunkX = 0; chunkX < widthChunks; chunkX++) {	
miRowChunk = MiRow + (chunkY << 4)	
miColChunk = MiCol + (chunkX << 4)	
set_tx_size(miRowChunk, miColChunk, 16, 16, 0, 0)	
}	
}	
} else {	
read_tx_partition(MiRow, MiCol, maxRectTxSize)	
}	
return 1	
}	
return 0	
}	

NOTE: The same transform partition is used for all chunks when `read_tx_size` is called.

§ 5.20.6.2. Block TX size syntax

<code>read_block_tx_size() {</code>	Descriptor
<code> bw4 = Num_4x4_Blocks_Wide[MiSize]</code>	
<code> bh4 = Num_4x4_Blocks_High[MiSize]</code>	
<code> if (TxMode == TX_MODE_SELECT &&</code>	
<code> MiSize > BLOCK_4X4 && is_inter &&</code>	
<code> !skip_flag && !Lossless) {</code>	
<code> maxTxSz = Max_Tx_Size_Rect[MiSize]</code>	
<code> txW4 = Tx_Width[maxTxSz] / MI_SIZE</code>	
<code> txH4 = Tx_Height[maxTxSz] / MI_SIZE</code>	
<code> for (row = MiRow; row < MiRow + bh4; row += txH4) {</code>	
<code> for (col = MiCol; col < MiCol + bw4; col += txW4) {</code>	
<code> read_tx_partition(row, col, maxTxSz)</code>	
<code> }</code>	
<code> }</code>	
<code> } else {</code>	
<code> if (read_tx_size(!skip_flag !is_inter) == 0) {</code>	
<code> for (row = MiRow; row < MiRow + bh4; row++) {</code>	
<code> for (col = MiCol; col < MiCol + bw4; col++) {</code>	
<code> LumaTxSizes[row][col] = TxSize</code>	
<code> LumaTxMiddle[row][col] = 0</code>	
<code> LumaTxScanOrder[row][col] = 0</code>	
<code> }</code>	
<code> }</code>	
<code> }</code>	
<code> }</code>	
<code>}</code>	

§ 5.20.6.3. Read TX partition syntax

<code>read_tx_partition(row, col, txSz) {</code>	Descriptor
<code> if (row >= MiRows col >= MiCols) {</code>	
<code> return</code>	
<code> }</code>	
<code> horzTxSz = find_tx_size(Tx_Width[txSz], Tx_Height[txSz] >> 1)</code>	
<code> vertTxSz = find_tx_size(Tx_Width[txSz] >> 1, Tx_Height[txSz])</code>	
<code> allowHorz = horzTxSz != TX_INVALID</code>	
<code> allowVert = vertTxSz != TX_INVALID</code>	
<code> txPartition = TX_PARTITION_NONE</code>	

if (Block_Width[MiSize] <= 64 && Block_Height[MiSize] <= 64) {	
tx_do_partition	S()
if (tx_do_partition) {	
if (allowHorz && allowVert) {	
tx_partition_type	S()
txPartition = tx_partition_type + 1	
} else if (Size_To_Tx_Type_Group_Vert_Or_Horz[MiSize] > 0) {	
if (reduced_tx_part_set) {	
tx_2or3_partition_type = 0	
} else {	
tx_2or3_partition_type	S()
}	
if (allowHorz) {	
txPartition = tx_2or3_partition_type ? TX_PARTITION_HORZ4 :	
TX_PARTITION_HORZ	
} else {	
txPartition = tx_2or3_partition_type ? TX_PARTITION_VERT4 :	
TX_PARTITION_VERT	
}	
} else {	
txPartition = allowHorz ? TX_PARTITION_HORZ : TX_PARTITION_VERT	
}	
}	
w4 = Tx_Width[txSz] / MI_SIZE	
h4 = Tx_Height[txSz] / MI_SIZE	
if (txPartition == TX_PARTITION_NONE) {	
TxSize = set_tx_size(row, col, h4 , w4, 0, 0)	
} else if (txPartition == TX_PARTITION_HORZ) {	
h4 = h4 >> 1	
set_tx_size(row, col, h4, w4, 0, 0)	
row += h4	
TxSize = set_tx_size(row, col, h4 , w4, 0, 0)	
} else if (txPartition == TX_PARTITION_VERT) {	
w4 = w4 >> 1	
set_tx_size(row, col, h4, w4, 0, 0)	
col += w4	
TxSize = set_tx_size(row, col, h4 , w4, 0, 0)	
} else if (txPartition == TX_PARTITION_HORZ4) {	
h4 = h4 >> 2	
set_tx_size(row, col, h4, w4, 0, 0)	

row += h4	
set_tx_size(row, col, h4, w4, 0, 0)	
row += h4	
set_tx_size(row, col, h4, w4, 0, 0)	
row += h4	
TxSize = set_tx_size(row, col, h4, w4, 0, 0)	
} else if (txPartition == TX_PARTITION_VERT4) {	
w4 = w4 >> 2	
set_tx_size(row, col, h4, w4, 0, 0)	
col += w4	
set_tx_size(row, col, h4, w4, 0, 0)	
col += w4	
set_tx_size(row, col, h4, w4, 0, 0)	
col += w4	
TxSize = set_tx_size(row, col, h4, w4, 0, 0)	
} else if (txPartition == TX_PARTITION_HORZ5) {	
h4 = h4 >> 2	
w4 = w4 >> 1	
set_tx_size(row, col, h4, w4, 0, 0)	
col += w4	
set_tx_size(row, col, h4, w4, 1, 0)	
col -= w4	
row += h4	
h4 = h4 << 1	
w4 = w4 << 1	
set_tx_size(row, col, h4, w4, 1, 0)	
row += h4	
h4 = h4 >> 1	
w4 = w4 >> 1	
set_tx_size(row, col, h4, w4, 1, 0)	
col += w4	
TxSize = set_tx_size(row, col, h4, w4, 1, 0)	
} else if (txPartition == TX_PARTITION_VERT5) {	
h4 = h4 >> 1	
w4 = w4 >> 2	
set_tx_size(row, col, h4, w4, 0, 1)	
row += h4	
set_tx_size(row, col, h4, w4, 1, 1)	
col += w4	
row -= h4	
h4 = h4 << 1	

w4 = w4 << 1	
set_tx_size(row, col, h4, w4, 1, 1)	
col += w4	
h4 = h4 >> 1	
w4 = w4 >> 1	
set_tx_size(row, col, h4, w4, 1, 1)	
row += h4	
TxSize = set_tx_size(row, col, h4, w4, 1, 1)	
} else { <i>// TX_PARTITION_SPLIT</i>	
w4 = w4 >> 1	
h4 = h4 >> 1	
set_tx_size(row, col + w4, h4, w4, 0, 0)	
set_tx_size(row, col, h4, w4, 0, 0)	
set_tx_size(row + h4, col, h4, w4, 0, 0)	
TxSize = set_tx_size(row + h4, col + w4, h4, w4, 0, 0)	
}	
}	

where the function `find_tx_size` finds the transform block size for the given dimensions and is defined as:

<pre>find_tx_size(w, h) { for (txSz = 0; txSz < TX_SIZES_ALL; txSz++) { if (Tx_Width[txSz] == w && Tx_Height[txSz] == h) { return txSz } } return TX_INVALID }</pre>

and the function `set_tx_size` saves the transform size as follows:

<pre>set_tx_size(row, col, h4, w4, mid, scanOrder) { subTxSz = find_tx_size(w4 << 2, h4 << 2) for (i = 0; i < h4; i++) { for (j = 0; j < w4; j++) { LumaTxSizes[row + i][col + j] = subTxSz LumaTxMiddle[row + i][col + j] = mid LumaTxScanOrder[row + i][col + j] = scanOrder } } return subTxSz }</pre>

§ 5.20.7. Motion vector and prediction structures

§ 5.20.7.1. Inter frame mode info syntax

<code>inter_frame_mode_info()</code> {	Descriptor
use_intrabc = 0	
skip_flag = 0	

inter_segment_id(1)	
read_skip_mode()	
read_is_inter()	
if (is_inter) {	
read_skip()	
} else {	
skip_flag = 0	
}	
if (!SegIdPreSkip) {	
inter_segment_id(0)	
}	
Lossless = LosslessArray[segment_id]	
if (TreeType != CHROMA_PART) {	
read_gdf()	
read_cdef()	
read_ccso()	
read_delta_qindex()	
}	
ReadDeltas = 0	
if (use_intrabc) {	
read_intrabc_info()	
} else if (is_inter) {	
inter_block_mode_info()	
} else {	
intra_block_mode_info()	
}	
}	

§ 5.20.7.2. Inter segment ID syntax

This is called before (preSkip equal to 1) and after (preSkip equal to 0) the skip_flag [syntax element](#) has been read.

inter_segment_id(preSkip) {	Descriptor
if (TreeType == CHROMA_PART) {	
segment_id = SegmentIds[MiRow][MiCol]	
} else if (segmentation_enabled) {	
predictedSegmentId = get_segment_id()	
if (segmentation_update_map) {	
if (preSkip && !SegIdPreSkip) {	
segment_id = 0	
return	
}	

if (!preSkip) {	
if (skip_flag) {	
seg_id_predicted = 0	
for (i = 0; i < Num_4x4_Blocks_Wide[MiSize]; i++) {	
AboveSegPredContext[MiCol + i] = seg_id_predicted	
}	
for (i = 0; i < Num_4x4_Blocks_High[MiSize]; i++) {	
LeftSegPredContext[MiRow + i] = seg_id_predicted	
}	
read_segment_id()	
return	
}	
}	
if (segmentation_temporal_update == 1) {	
seg_id_predicted	S()
if (seg_id_predicted) {	
segment_id = predictedSegmentId	
} else {	
read_segment_id()	
}	
for (i = 0; i < Num_4x4_Blocks_Wide[MiSize]; i++) {	
AboveSegPredContext[MiCol + i] = seg_id_predicted	
}	
for (i = 0; i < Num_4x4_Blocks_High[MiSize]; i++) {	
LeftSegPredContext[MiRow + i] = seg_id_predicted	
}	
} else {	
read_segment_id()	
}	
} else {	
segment_id = predictedSegmentId	
}	
} else {	
segment_id = 0	
}	
}	

[§](#) 5.20.7.3. Is inter syntax

read_is_inter() {	Descriptor
if (RegionType == INTRA_REGION) {	
is_inter = 0	

} else if (skip_mode) {	
is_inter = 1	
} else if (seg_feature_active (SEG_LVL_GLOBALMV)) {	
is_inter = 1	
} else if (TreeType == SHARED_PART && MiSize != ChromaMiSize) {	
is_inter = 1	
} else {	
is_inter	S()
}	
if (!is_inter && allow_intrabc &&	
Block_Width[MiSize] <= 64 &&	
Block_Height[MiSize] <= 64 &&	
MiSize != BLOCK_64X64 &&	
RegionType == MIXED_REGION) {	
use_intrabc	S()
if (use_intrabc) {	
is_inter = 1	
}	
} else {	
use_intrabc = 0	
}	
}	

§ 5.20.7.4. Get segment ID function

The predicted segment id is the smallest value found in the on-screen region of the [segmentation map](#) covered by the current [block](#).

```

get_segment_id( ) {
  bw4 = Num_4x4_Blocks_Wide[ MiSize ]
  bh4 = Num_4x4_Blocks_High[ MiSize ]
  xMis = Min( MiCols - MiCol, bw4 )
  yMis = Min( MiRows - MiRow, bh4 )
  seg = MAX_SEGMENTS - 1
  for ( y = 0; y < yMis; y++ ) {
    for ( x = 0; x < xMis; x++ ) {
      seg = Min( seg, PrevSegmentIds[ MiRow + y ][ MiCol + x ] )
    }
  }
  return seg
}

```

§ 5.20.7.5. Intra block mode info syntax

intra_block_mode_info() {	Descriptor
RefFrame[0] = INTRA_FRAME	
RefFrame[1] = NONE	
motion_mode = SIMPLE	

fsc_mode = 0	
use_most_probable_precision = 0	
MvPrecision = FrameMvPrecision	
CwpIdx = CWP_EQUAL	
PaletteSizeY = 0	
motion_mode = SIMPLE	
if (TreeType != CHROMA_PART) {	
read_intra_y_mode()	
} else {	
YMode = YModes[MiRow][MiCol]	
AngleDeltaY = AngleDeltaYs[MiRow][MiCol]	
PaletteSizeY = PaletteSizes[MiRow][MiCol]	
}	
if (HasChroma) {	
read_intra_uv_mode()	
if (UVMode == UV_CFL_PRED) {	
read_cfl_alphas()	
}	
}	
if (MiSize >= BLOCK_8X8 &&	
Block_Width[MiSize] <= 64 &&	
Block_Height[MiSize] <= 64 &&	
allow_screen_content_tools) {	
palette_mode_info()	
}	
if (TreeType != CHROMA_PART) {	
dip_mode_info()	
}	
}	

§ 5.20.7.6. Inter block mode info syntax

inter_block_mode_info() {	Descriptor
mrl_index = 0	
use_dip = 0	
fsc_mode = 0	
use_dpcm_y = 0	
use_dpcm_uv = 0	
PaletteSizeY = 0	
use_most_probable_precision = 0	
MvPrecision = FrameMvPrecision	
IntraJointMode = DC_PRED	

use_bawp = 0	
use_amvd = 0	
read_ref_frames()	
isCompound = is_inter_ref_frame(RefFrame[1])	
DeriveWrl = !skip_mode && !isCompound && RefFrame[0] != TIP_FRAME &&	
Block_Width[MiSize] >= 8 && Block_Height[MiSize] >= 8	
find_mode_ctx(isCompound)	
if (skip_mode) {	
YMode = NEAR_NEARMV	
use_optflow = 0	
} else if (seg_feature_active(SEG_LVL_SKIP)	
seg_feature_active(SEG_LVL_GLOBALMV)) {	
YMode = GLOBALMV	
use_optflow = 0	
} else if (isCompound) {	
if (RefFrame[0] == RefFrame[1]) {	
compound_mode_same_refs	S()
if (compound_mode_same_refs < 2) {	
YMode = NEAR_NEARMV + compound_mode_same_refs	
} else {	
YMode = NEAR_NEARMV + compound_mode_same_refs + 1	
}	
} else {	
is_joint	S()
if (is_joint) {	
YMode = JOINT_NEWMV	
} else {	
compound_mode_non_joint	S()
YMode = NEAR_NEARMV + compound_mode_non_joint	
}	
}	
if (opfl_refine_type == REFINESWITCHABLE &&	
opfl_allowed_for_refs(RefFrame) &&	
Block_Width[MiSize] >= 8 && Block_Height[MiSize] >= 8 &&	
YMode != GLOBAL_GLOBALMV) {	
use_optflow	S()
} else {	
use_optflow = 0	
}	
if (allow_amvd_mode(YMode)) {	
use_amvd	S()

}	
} else {	
use_optflow = 0	
if (RefFrame[0] == TIP_FRAME) {	
tip_pred_mode	S()
YMode = Tip_Pred_Index_To_Mode[tip_pred_mode]	
if (allow_amvd_mode(YMode)) {	
use_amvd	S()
}	
} else {	
if (allow_warpmv_mode &&	
Min(Block_Width[MiSize], Block_Height[MiSize]) >= 8) {	
is_warp	S()
} else {	
is_warp = 0	
}	
if (is_warp) {	
if (force_integer_mv) {	
warp_mv = 1	
} else {	
warp_mv	S()
}	
YMode = warp_mv ? WARPMV : WARP_NEWMV	
} else {	
single_mode	S()
YMode = NEARMV + single_mode	
if (allow_amvd_mode(YMode)) {	
use_amvd	S()
}	
if (allow_bawp && !is_scaled(RefFrame[0], 1) &&	
Min(Block_Width[MiSize], Block_Height[MiSize]) >= 8 &&	
FrameType != SWITCH_FRAME && YMode != GLOBALMV) {	
use_bawp	S()
if (use_bawp) {	
explicit_bawp	S()
if (explicit_bawp) {	
explicit_bawp_scale	S()
}	
} else {	
explicit_bawp = 0	
}	

if (use_bawp && HasChroma) {	
use_bawp_chroma	S()
}	
}	
}	
}	
}	
if (skip_mode) {	
find_mv_stack(isCompound)	
} else if (has_second_drl(YMode)) {	
r0 = RefFrame[0]	
r1 = RefFrame[1]	
RefFrame[0] = r0	
RefFrame[1] = NONE	
find_mv_stack(0)	
for (i = 0; i < MAX_REF_MV_STACK_SIZE; i++) {	
RefStack0Mvs[i] = RefStackMv[i][0]	
}	
RefFrame[0] = r1	
RefFrame[1] = NONE	
find_mv_stack(0)	
for (i = 0; i < MAX_REF_MV_STACK_SIZE; i++) {	
RefStack1Mvs[i] = RefStackMv[i][0]	
}	
RefFrame[0] = r0	
RefFrame[1] = r1	
} else {	
find_mv_stack(isCompound)	
}	
motion_mode = read_motion_mode(isCompound)	
RefWarpIdx = 0	
if (YMode == WARPMV motion_mode == DELTAWARP) {	
for (idx = 0; idx < MAX_WARP_REF_CANDIDATES - 1; idx++) {	
warp_idx	S()
if (warp_idx == 0) {	
RefWarpIdx = idx	
break	
}	
RefWarpIdx = idx + 1	
}	
}	

if (YMode == WARP_MV && RefWarpIdx < 2) {	
warpmv_with_mvd	S()
} else {	
warpmv_with_mvd = 0	
}	
if (is_joint_mvd_coding_mode(YMode)) {	
jmvd_scale_mode	S()
}	
RefMvIdx = 0	
if (has_newmv(YMode) has_nearmv(YMode)) {	
m = max_drl_bits_minus_1 + 1	
if (has_second_drl(YMode)) {	
RefMvIdx0 = read_drl_idx(0, m)	
start = (RefFrame[0] == RefFrame[1] && YMode == NEAR_NEARMV) ?	
RefMvIdx0 + 1 : 0	
RefMvIdx1 = read_drl_idx(start, m)	
} else {	
RefMvIdx = read_drl_idx(0, m)	
}	
}	
IsAdaptiveMvd = enable_adaptive_mvd && use_amvd	
if (IsAdaptiveMvd) {	
MvPrecision = FrameMvPrecision	
use_most_probable_precision = 1	
} else if (enable_flex_mvres && UsePerBlockMvPrecision &&	
has_newmv(YMode)) {	
use_most_probable_precision	S()
if (use_most_probable_precision) {	
MvPrecision = FrameMvPrecision	
} else {	
pb_mv_precision	S()
adjustedPrecision = Max(MV_PRECISION_ONE_PEL,	
FrameMvPrecision - 2) -	
pb_mv_precision	
if (adjustedPrecision <= MV_PRECISION_TWO_PEL) {	
MvPrecision = adjustedPrecision - 1	
} else {	
MvPrecision = adjustedPrecision	
}	
}	
} else {	

MvPrecision = FrameMvPrecision	
use_most_probable_precision = 1	
}	
assign_mv(isCompound)	
if (motion_mode == DELTAWARP) {	
read_warp_delta()	
}	
if (YMode == WARPMV) {	
read_interintra_mode(1)	
}	
read_refinemv(isCompound)	
read_compound_type(isCompound)	
CwpIdx = CWP_EQUAL	
if (enable_cwp) {	
if (isCompound && skip_mode) {	
CwpIdx = RefStackCwp[RefMvIdx]	
} else if (isCompound && !use_refinemv &&	
compound_type == COMPOUND_AVERAGE &&	
motion_mode == SIMPLE && !use_optflow) {	
if (YMode == NEAR_NEARMV (is_joint_mvd_coding_mode(YMode) &&	
jmvd_scale_mode==0)) {	
for (idx = 0; idx < MAX_CWP_NUM - 1; idx++) {	
cwp_idx	S()
if (cwp_idx == 0) {	
break	
}	
}	
CwpIdx = Cwp_Weighting_Factor[is_same_side()][idx]	
}	
}	
}	
if (isCompound && opfl_refine_type == REFINE_ALL &&	
compound_type == COMPOUND_AVERAGE &&	
YMode != GLOBAL_GLOBALMV &&	
!skip_mode &&	
CwpIdx == CWP_EQUAL &&	
opfl_allowed_for_refs(RefFrame) &&	
Block_Width[MiSize] >= 8 && Block_Height[MiSize] >= 8) {	
use_optflow = 1	
}	
if (skip_mode use_optflow use_refinemv DecidedAgainstRefinemv	

RefFrame[0] == TIP_FRAME) {	
interp_filter = EIGHTTAP_SHARP	
} else if (interpolation_filter == SWITCHABLE) {	
if (needs_interp_filter()) {	
interp_filter	S()
} else {	
interp_filter = EIGHTTAP	
}	
} else {	
interp_filter = interpolation_filter	
}	
}	

The function has_nearmv is defined as:

```
has_nearmv( mode ) {
    return (mode == NEARMV || mode == NEAR_NEARMV
           || mode == NEAR_NEWMV || mode == NEW_NEARMV)
}
```

The function has_newmv is defined as:

```
has_newmv( mode ) {
    return (mode == NEWMV ||
           mode == NEW_NEWMV ||
           mode == NEAR_NEWMV ||
           mode == NEW_NEARMV ||
           mode == WARP_NEWMV ||
           mode == JOINT_NEWMV
           )
}
```

The function needs_interp_filter is defined as:

```
needs_interp_filter( ) {
    large = (Min(Block_Width[ MiSize ], Block_Height[ MiSize ]) >= 8)
    if ( motion_mode >= LOCALWARP ) {
        return 0
    } else if ( large && YMode == GLOBALMV ) {
        return 0
    } else if ( large && YMode == GLOBAL_GLOBALMV ) {
        return 0
    } else {
        return 1
    }
}
```

The function is_inter_ref_frame is defined as:

```
is_inter_ref_frame(ref) {
    return ref != INTRA_FRAME && ref != NONE
}
```

The function `is_joint_mvd_coding_mode` is defined as:

```
is_joint_mvd_coding_mode(mode) {
    return mode == JOINT_NEWMV
}
```

The function `has_second_drl` is defined as:

```
has_second_drl(mode) {
    return (mode == NEAR_NEARMV || mode == NEAR_NEWMV) && !skip_mode &&
        !use_optflow
}
```

NOTE: Two reference lists can be used for NEAR_NEWMV, but only one for NEW_NEARMV.

The constant table `Cwp_Weighting_Factor` is defined as:

```
Cwp_Weighting_Factor[ 2 ][ MAX_CWP_NUM ] = {
    { 8, 12, 4, 10, 6 },
    { 8, 12, 4, 20, -4 }
}
```

The function `opfl_allowed_for_refs` is defined as:

```
opfl_allowed_for_refs( refFrames ) {
    if ( FrameType == SWITCH_FRAME ||
        is_scaled( refFrames[ 0 ], 1 ) ||
        is_scaled( refFrames[ 1 ], 1 ) ) {
        return 0
    }
    d0 = get_relative_dist( OrderHint, OrderHints[ refFrames[ 0 ] ] )
    d1 = get_relative_dist( OrderHint, OrderHints[ refFrames[ 1 ] ] )
    return (d0 <= 0) ^ (d1 <= 0)
}
```

The constant table `Tip_Pred_Index_To_Mode` is defined as:

```
Tip_Pred_Index_To_Mode[ 2 ] = {
    NEARMV,
    NEWMV
}
```

The function `allow_amvd_mode` is defined as:

```
allow_amvd_mode( mode ) {
    return enable_adaptive_mvd &&
        (mode == NEWMV ||
         mode == NEW_NEWMV ||
         mode == NEAR_NEWMV ||
         mode == NEW_NEARMV ||
         mode == JOINT_NEWMV)
}
```

[5.20.7.7. Read warp delta syntax](#)

```
read_warp_delta( ) {
```

Descriptor

for (i = 0; i < 6; i++) {	
params[i] = WarpParamStack[RefWarpIdx][i]	
}	
useSixParam = enable_six_param_warp_delta && RefWarpIdx == 1	
if (YMode == WARP_NEWMV && (useSixParam RefWarpIdx == 0)) {	
warp_delta_precision	S()
params[0] = 0	
params[1] = 0	
params[2] += read_warp_delta_param(2, warp_delta_precision)	
params[3] += read_warp_delta_param(3, warp_delta_precision)	
if (useSixParam) {	
params[4] += read_warp_delta_param(4, warp_delta_precision)	
params[5] += read_warp_delta_param(5, warp_delta_precision)	
} else {	
params[4] = -params[3]	
params[5] = params[2]	
}	
}	
LocalWarpParams[0] = reduce_warp_model(params)	
(LocalWarpParams[0][0], LocalWarpParams[0][1]) =	
get_warp_translation(LocalWarpParams[0], 0)	
}	

where the function read_warp_delta_param is specified as:

```

read_warp_delta_param( idx, highPrec ) {
    S() warp_delta_param_low;
    v = warp_delta_param_low
    if ( highPrec && v == WARP_DELTA_NUM_SYMBOLS_LOW - 1 ) {
        S() warp_delta_param_high;
        v += warp_delta_param_high
    }
    if ( v != 0 ) {
        S() warp_delta_param_sign;
        if ( warp_delta_param_sign ) {
            v = -v
        }
    }
    return v << ( WARP_DELTA_STEP_BITS + 1 - highPrec )
}

```

§ 5.20.7.8. Read drl_idx syntax

read_drl_idx(start,m) {	Descriptor
for (idx = start; idx < m; idx++) {	
drl_mode	S()
if (drl_mode == 0) {	
return idx	

}	
}	
return m	
}	

§ 5.20.7.9. DIP mode info syntax

dip_mode_info() {	Descriptor
use_dip = 0	
if (enable_dip &&	
YMode == DC_PRED && PaletteSizeY == 0 &&	
Block_Width[MiSize] > 4 && Block_Height[MiSize] > 4 &&	
Block_Width[MiSize] * Block_Height[MiSize] >= 128) {	
use_dip	S()
if (use_dip) {	
dip_transpose	L(1)
dip_mode	S()
}	
}	
}	

§ 5.20.7.10. Ref frames syntax

read_ref_frames() {	Descriptor
if (skip_mode) {	
(RefFrame[0], RefFrame[1]) = skip_mode_frames()	
return	
}	
bw4 = Num_4x4_Blocks_Wide[MiSize]	
bh4 = Num_4x4_Blocks_High[MiSize]	
if (TipFrameMode != TIP_FRAME_DISABLED &&	
!skip_mode && Min(bw4, bh4) >= 2 &&	
MiSize == ChromaMiSize) {	
tip_mode	S()
if (tip_mode) {	
RefFrame[0] = TIP_FRAME	
RefFrame[1] = NONE	
return	
}	
}	
if (seg_feature_active(SEG_LVL_SKIP)	
seg_feature_active(SEG_LVL_GLOBALMV)) {	
RefFrame[0] = SkipSegFrame	

RefFrame[1] = NONE	
} else {	
if (reference_select && is_comp_ref_allowed()) {	
comp_mode	S()
} else {	
comp_mode = SINGLE_REFERENCE	
}	
if (comp_mode == COMPOUND_REFERENCE) {	
read_compound_ref()	
} else {	
RefFrame[0] = read_single_ref()	
RefFrame[1] = NONE	
}	
}	
}	

where skip_mode_frames is specified as:

```

skip_mode_frames() {
    for ( n = 0; n < NNumBuf; n++ ) {
        if ( NRefFrame[ n ][ 0 ] == TIP_FRAME ) {
            return ( Min(ClosestPast, ClosestFuture),
                     Max(ClosestPast, ClosestFuture) )
        }
        if ( is_inter_ref_frame( NRefFrame[ n ][ 0 ] ) &&
              is_inter_ref_frame( NRefFrame[ n ][ 1 ] ) ) {
            return ( NRefFrame[ n ][ 0 ], NRefFrame[ n ][ 1 ] )
        }
        if ( is_inter_ref_frame( NRefFrame[ n ][ 0 ] ) ) {
            break
        }
    }
    return (SkipModeFrame[ 0 ], SkipModeFrame[ 1 ])
}

```

§ 5.20.7.11. Read compound ref syntax

read_compound_ref() {	Descriptor
RefFrame[0] = NumTotalRefs - 1	
RefFrame[1] = NumTotalRefs - 1	
nFound = 0	
for (ref = 0; ref < NumTotalRefs - 1 && nFound < 2; ref++) {	
if (nFound == 0 && ref == 2) {	
comp_ref = 1	
} else if (nFound == 0 &&	
ref + 1 >= NumSameRefCompound &&	
ref + 1 == NumTotalRefs - 1) {	
comp_ref = 1	
} else {	

comp_ref	S()
}	
if (comp_ref) {	
RefFrame[nFound] = ref	
nFound++	
if (ref < NumSameRefCompound) {	
ref--	
}	
}	
}	
}	

§ 5.20.7.12. Read single ref syntax

read_single_ref() {	Descriptor
for (ref = 0; ref < NumTotalRefs - 1; ref++) {	
single_ref	S()
if (single_ref) {	
return ref	
}	
}	
return NumTotalRefs - 1	
}	

§ 5.20.7.13. Assign MV syntax

assign_mv(isCompound) {	Descriptor
mvdRead[0] = 0	
mvdRead[1] = 0	
baseList = 0	
firstDist = 0	
secondDist = 0	
if (is_joint_mvd_coding_mode(YMode)) {	
firstDist = Abs(get_relative_dist(OrderHints[RefFrame[0]],	
OrderHint))	
secondDist = Abs(get_relative_dist(OrderHints[RefFrame[1]],	
OrderHint))	
restrict0 = OrderHints[RefFrame[0]] == RESTRICTED_OH	
restrict1 = OrderHints[RefFrame[1]] == RESTRICTED_OH	
if (firstDist < secondDist (!restrict0 && restrict1)) {	
baseList = 1	
(firstDist, secondDist) = (secondDist, firstDist)	
}	
}	

if (!is_same_side()) {	
secondDist = -secondDist	
}	
}	
for (i = 0; i < 1 + isCompound; i++) {	
if (use_intrabc) {	
compMode = intrabc_mode ? NEARMV : NEWMV	
} else {	
compMode = get_mode(i, baseList)	
}	
if (use_intrabc) {	
PredMvs[0] = RefStackMv[RefMvIdx][0]	
} else if (compMode == GLOBALMV) {	
PredMvs[i] = GlobalMvs[i]	
} else if (compMode == WARPMV) {	
PredMvs[0] = get_warp_motion_vector(	
WarpParamStack[RefWarpIdx],	
warpmv_with_mvd ? FrameMvPrecision :	
MV_PRECISION_EIGHTH_PEL)	
} else if (has_second_drl(YMode)) {	
if (i == 0) {	
PredMvs[i] = RefStack0Mvs[RefMvIdx0]	
} else {	
PredMvs[i] = RefStack1Mvs[RefMvIdx1]	
}	
} else {	
PredMvs[i] = RefStackMv[RefMvIdx][i]	
}	
if (compMode == NEWMV warpmv_with_mvd compMode == WARP_NEWMV) {	
if (!warpmv_with_mvd && MvPrecision < MV_PRECISION_HALF_PEL &&	
!IsAdaptiveMvd) {	
lower_mv_precision(MvPrecision, PredMvs[i])	
}	
diffMvs[i] = read_mv()	
mvdRead[i] = 1	
} else {	
for (comp = 0; comp < 2; comp++) {	
diffMvs[i][comp] = 0	
}	
}	
}	

shift = MV_PRECISION_EIGHTH_PEL - MvPrecision	
lastSign = 0	
numNonzero = 0	
for (i = 0; i < 1 + isCompound; i++) {	
if (mvdRead[i]) {	
for (comp = 0; comp < 2; comp++) {	
if (diffMvs[i][comp] != 0) {	
lastRef = i	
lastComp = comp	
lastSign += diffMvs[i][comp] >> shift	
numNonzero++	
}	
}	
}	
}	
}	
thresh = YMode == NEW_NEWMV ? 4 : 1	
allowed = is_mvd_sign_derive_allowed(isCompound) && numNonzero >= thresh	
for (i = 0; i < 1 + isCompound; i++) {	
if (mvdRead[i]) {	
for (comp = 0; comp < 2; comp++) {	
if (diffMvs[i][comp] != 0) {	
if (allowed && i == lastRef && comp == lastComp) {	
mv_sign = lastSign & 1	
} else {	
mv_sign	L(1)
}	
diffMvs[i][comp] = mv_sign ? -diffMvs[i][comp] :	
diffMvs[i][comp]	
}	
}	
}	
}	
}	
if (is_joint_mvd_coding_mode(YMode)) {	
projMv = get_mv_projection(diffMvs[baseList], secondDist, firstDist)	
if (use_amvd) {	
for (comp = 0; comp < 2; comp++) {	
if (jmvd_scale_mode == 1) {	
projMv[comp] = projMv[comp] * 2	
} else if (jmvd_scale_mode == 2) {	
projMv[comp] = projMv[comp] / 2	
}	

}	
} else if (jmv_d_scale_mode > 0) {	
comp = (jmv_d_scale_mode - 1) & 1	
if (jmv_d_scale_mode <= 2) {	
projMv[comp] = projMv[comp] * 2	
} else {	
projMv[comp] = projMv[comp] / 2	
}	
}	
for (comp = 0; comp < 2; comp++) {	
BlockMvs[baseList][comp] = mv_clamp_to_integer(	
PredMvs[baseList][comp] + diffMvs[baseList][comp])	
BlockMvs[1 - baseList][comp] = mv_clamp_to_integer(	
PredMvs[1 - baseList][comp] + projMv[comp])	
}	
} else {	
for (i = 0; i < 1 + isCompound; i++) {	
for (comp = 0; comp < 2; comp++) {	
BlockMvs[i][comp] = mv_clamp_to_integer(	
PredMvs[i][comp] + diffMvs[i][comp])	
}	
}	
}	
}	

where the function `is_same_side` is defined as:

```
is_same_side() {
    return ( FrameDistance[ RefFrame[ 0 ] ] < 0 &&
             FrameDistance[ RefFrame[ 1 ] ] < 0 ) ||
           ( FrameDistance[ RefFrame[ 0 ] ] > 0 &&
             FrameDistance[ RefFrame[ 1 ] ] > 0 )
}
```

and the function `is_mvd_sign_derive_allowed` is defined as:

```
is_mvd_sign_derive_allowed(isCompound) {
    if ( use_intra_bc ||
         !enable_mvd_sign_derive ||
         motion_mode != SIMPLE ||
         IsAdaptiveMvd || skip_mode ||
         allow_screen_content_tools ||
         FrameMvPrecision > MV_PRECISION_QUARTER_PEL ||
         MvPrecision >= MV_PRECISION_QUARTER_PEL ||
         has_nearmv(YMode) ) {
        return 0
    }
    if ( isCompound ) {
        return RefMvIdx == 0
    } else {
```

```

    return 1
}
}

```

and the function `lower_mv_precision` (which modifies the contents of the input motion vector to the target precision) is defined as:

```

lower_mv_precision( precision, candMv ) {
    bits = MV_PRECISION_EIGHTH_PEL - precision
    radix = 1 << bits
    for ( i = 0; i < 2; i++ ) {
        a = Abs( candMv[ i ] )
        aInt = Round2( a - 1, bits )
        if ( candMv[ i ] >= 0 ) {
            candMv[ i ] = aInt << bits
        } else {
            candMv[ i ] = (-aInt) << bits
        }
        if ((aInt << bits) != a) {
            candMv[ i ] = Clip3( MV_LOW + radix, MV_UPP - radix, candMv[ i ] )
        }
    }
}

```

and the function `mv_clamp_to_integer` (which adjusts a motion vector component to an integer location if it would have overflowed the allowed range) is defined as:

```

mv_clamp_to_integer( v ) {
    if ( v < MV_LOW + 1 ) {
        return MV_LOW + 8
    } else if ( v > MV_UPP - 1 ) {
        return MV_UPP - 8
    } else {
        return v
    }
}

```

[§](#) 5.20.7.14. Read motion mode syntax

<code>read_motion_mode(isCompound) {</code>	Descriptor
<code>motion_mode_allowed(isCompound)</code>	
<code>inter_intra = 0</code>	
<code>localAllowed = AllowedMotionModes[LOCALWARP] &&</code>	
<code>frame_enabled_motion_modes[LOCALWARP]</code>	
<code>if (YMode == WARPMV) {</code>	
<code>return DELTAWARP</code>	
<code>}</code>	
<code>if (YMode == WARP_NEWMV) {</code>	
<code>extendAllowed = AllowedMotionModes[EXTENDWARP] &&</code>	
<code>frame_enabled_motion_modes[EXTENDWARP]</code>	
<code>if (extendAllowed) {</code>	
<code>use_extend_warp</code>	S()
<code>if (use_extend_warp) {</code>	

return EXTENDWARP	
}	
}	
if (localAllowed) {	
use_local_warp	S()
if (use_local_warp) {	
return LOCALWARP	
}	
}	
return DELTAWARP	
}	
if (AllowedMotionModes[INTERINTRA] &&	
frame_enabled_motion_modes[INTERINTRA]) {	
read_interintra_mode(0)	
if (inter_intra) {	
return INTERINTRA	
}	
}	
if (localAllowed) {	
use_local_warp	S()
if (use_local_warp) {	
return LOCALWARP	
}	
}	
return SIMPLE	
}	

The function `motion_mode_allowed` works out the allowed motion modes as follows:

```

motion_mode_allowed(isCompound) {
    for ( i = 0; i < MOTION_MODES; i++ ) {
        AllowedMotionModes[ i ] = 0
    }
    if ( YMode == WARPMV ) {
        AllowedMotionModes[ DELTAWARP ] = 1
        return
    }
    if ( YMode == WARP_NEWMV ) {
        AllowedMotionModes[ LOCALWARP ] = WarpSampleFound[ 0 ]
        AllowedMotionModes[ EXTENDWARP ] = WarpSampleFound[ 0 ]
        AllowedMotionModes[ DELTAWARP ] = 1
        return
    }
    if ( skip_mode || RefFrame[ 0 ] == INTRA_FRAME || use_bawp ||
        RefFrame[ 0 ] == TIP_FRAME ||
        seg_feature_active(SEG_LVL_SKIP) ||
        seg_feature_active(SEG_LVL_GLOBALMV) ||
        ( isCompound && is_thin_4xn_nx4_block() ) ) {
        return
    }
    AllowedMotionModes[ INTERINTRA ] = (!isCompound &&

```

```

        MiSize >= BLOCK_8X8 &&
        Block_Width[ MiSize ] <= 64 &&
        Block_Height[ MiSize ] <= 64)
if ( RefFrame[ 0 ] == RefFrame[ 1 ] ) {
    return
}
if ( !force_integer_mv &&
    ( YMode == GLOBALMV || YMode == GLOBAL_GLOBALMV ) &&
    GmType[ RefFrame[ 0 ] ] > IDENTITY ) {
    return
}
if ( Min( Block_Width[ MiSize ], Block_Height[ MiSize ] ) < 8 ) {
    return
}
AllowedMotionModes[ LOCALWARP ] = !force_integer_mv && YMode == NEW_NEWMV &&
    !use_optflow &&
    opfl_refine_type != REFINES_ALL &&
    WarpSampleFound[ 0 ] &&
    WarpSampleFound[ 1 ]
}

```

where `is_scaled` is a function that determines whether a [reference frame](#) uses scaling and is specified as:

```

is_scaled( refFrame, checkRestricted ) {
    if ( checkRestricted && OrderHints[ refFrame ] == RESTRICTED_OH ) {
        return 1
    }
    refIdx = ref_frame_idx[ refFrame ]
    xScale = ( ( RefFrameWidth[ refIdx ] << REF_SCALE_SHIFT ) +
        ( FrameWidth / 2 ) ) / FrameWidth
    yScale = ( ( RefFrameHeight[ refIdx ] << REF_SCALE_SHIFT ) +
        ( FrameHeight / 2 ) ) / FrameHeight
    noScale = 1 << REF_SCALE_SHIFT
    return xScale != noScale || yScale != noScale
}

```

and `is_thin_4xn_nx4_block` is a function that tests the block size as follows:

```

is_thin_4xn_nx4_block( ) {
    w = Block_Width[ MiSize ]
    h = Block_Height[ MiSize ]
    return (w == 4 && h >= 16) || (h == 4 && w >= 16)
}

```

§ 5.20.7.15. Read inter intra syntax

<code>read_interintra_mode(isWarp) {</code>	Descriptor
<code>if (isWarp) {</code>	
<code>if (Block_Width[MiSize] <= 64 && Block_Height[MiSize] <= 64) {</code>	
<code> warp_inter_intra</code>	S()
<code> inter_intra = warp_inter_intra</code>	
<code>} else {</code>	
<code> inter_intra = 0</code>	
<code>}</code>	
<code>} else {</code>	
<code> inter_intra</code>	S()
<code>}</code>	

if (inter_intra) {	
interintra_mode	S()
RefFrame[1] = INTRA_FRAME	
AngleDeltaY = 0	
AngleDeltaUV = 0	
UVMode = DC_PRED	
if (Wedge_Bits[MiSize] == 0) {	
wedge_interintra = 0	
} else {	
wedge_interintra	S()
}	
if (wedge_interintra) {	
read_wedge_mode()	
wedge_sign = 0	
}	
}	

[§](#) 5.20.7.16. Read compound type syntax

read_compound_type(isCompound) {	Descriptor
comp_group_idx = 0	
if (skip_mode use_optflow	
(YMode == JOINT_NEWMV && use_amvd)	
(use_refinemv && is_switchable_refinemv())) {	
compound_type = COMPOUND_AVERAGE	
return	
}	
if (isCompound) {	
n = Wedge_Bits[MiSize]	
if (enable_masked_compound && !is_thin_4xn_nx4_block()) {	
comp_group_idx	S()
if (comp_group_idx != 0 && use_refinemv) {	
DecidedAgainstRefinemv = 1	
use_refinemv = 0	
}	
}	
if (comp_group_idx == 0) {	
compound_type = COMPOUND_AVERAGE	
} else {	
if (n == 0) {	
compound_type = COMPOUND_DIFFWTD	

} else {	
compound_type	S()
}	
}	
if (compound_type == COMPOUND_WEDGE) {	
read_wedge_mode()	
wedge_sign	L(1)
} else if (compound_type == COMPOUND_DIFFWTD) {	
mask_type	L(1)
}	
} else {	
if (inter_intra) {	
compound_type = wedge_interintra ? COMPOUND_WEDGE : COMPOUND_INTRA	
} else {	
compound_type = COMPOUND_AVERAGE	
}	
}	
}	

§ 5.20.7.17. Read refine mv syntax

read_refinemv(isCompound) {	Descriptor
use_refinemv = 0	
DecidedAgainstRefinemv = 0	
if (enable_refinemv &&	
isCompound &&	
(Block_Width[MiSize] >= 16 Block_Height[MiSize] >= 16) &&	
(Block_Width[MiSize] >= 8 && Block_Height[MiSize] >= 8) &&	
is_refinemv_allowed_mode() &&	
is_refinemv_allowed_reference(RefFrame)	
) {	
if (is_switchable_refinemv()) {	
use_refinemv	S()
} else {	
use_refinemv = 1	
}	
}	
}	

where the functions `is_refinemv_allowed_mode`, `is_switchable_refinemv`, `is_refinemv_allowed_reference` are specified as:

```
is_refinemv_allowed_mode() {
    if ( skip_mode || YMode == GLOBAL_GLOBALMV || motion_mode != SIMPLE ) {
        return 0
    }
    if ( opfl_refine_type == REFINESWITCHABLE &&
        has_newmv( YMode ) &&
        !use_optflow ) {
        return 0
    }
    return 1
}

is_switchable_refinemv() {
    if ( YMode == NEAR_NEARMV ||
        (YMode == JOINT_NEWMV && use_optflow &&
         opfl_refine_type == REFINESWITCHABLE) ) {
        return 0
    }
    return 1
}

is_refinemv_allowed_reference( refFrames ) {
    if ( FrameType == SWITCH_FRAME ||
        is_scaled( refFrames[ 0 ], 1 ) ||
        is_scaled( refFrames[ 1 ], 1 ) ) {
        return 0
    }
    d0 = get_relative_dist( OrderHint, OrderHints[ refFrames[ 0 ] ] )
    d1 = get_relative_dist( OrderHint, OrderHints[ refFrames[ 1 ] ] )
    return d0 != 0 && d0 == -d1
}
```

§ 5.20.7.18. Read wedge mode syntax

<code>read_wedge_mode() {</code>	Descriptor
<code>wedge_quad</code>	S()
<code>wedge_angle</code>	S()
<code>wedgeAngle = wedge_quad * 5 + wedge_angle</code>	
<code>if ((wedgeAngle >= H_WEDGE_ANGLES) </code>	
<code>(wedgeAngle == WEDGE_90) </code>	
<code>(wedgeAngle == WEDGE_0)) {</code>	
<code>wedge_dist2</code>	S()
<code>wedgeDist = wedge_dist2 + 1</code>	
<code>} else {</code>	
<code>wedge_dist1</code>	S()
<code>wedgeDist = wedge_dist1</code>	
<code>}</code>	
<code>WedgeIndex = Wedge_Angle_Dist_2_Index[wedgeAngle][wedgeDist]</code>	
<code>}</code>	

where the lookup table `Wedge_Angle_Dist_2_Index` is specified as:

```
Wedge_Angle_Dist_2_Index[ WEDGE_ANGLES ][ NUM_WEDGE_DIST ] = {
    { -1, 0, 1, 2 },
    { 3, 4, 5, 6 },
    { 7, 8, 9, 10 },
    { 11, 12, 13, 14 },
    { 15, 16, 17, 18 },
    { -1, 19, 20, 21 },
    { 22, 23, 24, 25 },
    { 26, 27, 28, 29 },
    { 30, 31, 32, 33 },
    { 34, 35, 36, 37 },
    { -1, 38, 39, 40 },
    { -1, 41, 42, 43 },
    { -1, 44, 45, 46 },
    { -1, 47, 48, 49 },
    { -1, 50, 51, 52 },
    { -1, 53, 54, 55 },
    { -1, 56, 57, 58 },
    { -1, 59, 60, 61 },
    { -1, 62, 63, 64 },
    { -1, 65, 66, 67 }
}
```

§ 5.20.7.19. Get mode function

```
get_mode( refList, baseList ) {
    if ( YMode == JOINT_NEWMV ) {
        if ( refList == baseList ) {
            compMode = NEWMV
        } else {
            compMode = NEARMV
        }
    } else if ( refList == 0 ) {
        if ( YMode == NEW_NEWMV || YMode == NEW_NEARMV ) {
            compMode = NEWMV
        } else if ( YMode < NEAR_NEARMV ) {
            compMode = YMode
        } else if ( YMode == NEAR_NEARMV || YMode == NEAR_NEWMV ) {
            compMode = NEARMV
        } else {
            compMode = GLOBALMV
        }
    } else {
        if ( YMode == NEW_NEWMV || YMode == NEAR_NEWMV ) {
            compMode = NEWMV
        } else if ( YMode == NEAR_NEARMV || YMode == NEW_NEARMV ) {
            compMode = NEARMV
        } else {
            compMode = GLOBALMV
        }
    }
    return compMode
}
```

§ 5.20.7.20. MV syntax

<code>read_mv()</code> {	Descriptor
<code>diffMv[0] = 0</code>	
<code>diffMv[1] = 0</code>	
<code>if (use_intrabc) {</code>	
<code> MvCtx = MV_INTRABC_CONTEXT</code>	

} else {	
MvCtx = 0	
}	
if (IsAdaptiveMvd) {	
mv_joint	S()
if (mv_joint == MV_JOINT_HZVNZ mv_joint == MV_JOINT_HNZVNZ) {	
diffMv[0] = read_mv_component(0)	
}	
if (mv_joint == MV_JOINT_HNZVZ mv_joint == MV_JOINT_HNZVNZ) {	
diffMv[1] = read_mv_component(1)	
}	
} else {	
shell_set	S()
shell_class	S()
shellClass = shell_class	
if (shell_set) {	
shellClass += (11 + MvPrecision) >> 1	
if (MvPrecision == MV_PRECISION_EIGHTH_PEL && shell_class == 7) {	
joint_shell_last_two_classes	S()
shellClass += joint_shell_last_two_classes	
}	
}	
shellClassOffset = 0	
if (shellClass < 2) {	
shell_offset_low_class	S()
shellClassOffset = shell_offset_low_class	
} else if (shellClass == 2) {	
for (i = 0; i < 3; i++) {	
if (i == 0) {	
shell_offset_class2	S()
shellClassOffset = shell_offset_class2	
} else {	
shell_offset_class2_high	L(1)
shellClassOffset = shell_offset_class2_high + i	
}	
if (shellClassOffset == i) {	
break	
}	
}	
} else {	
for (i = 0; i < shellClass; i++) {	
shell_offset_other_class	

	S()
shellClassOffset = shell_offset_other_class << i	
}	
}	
shellClassBaseIndex = (shellClass == 0) ? 0 : (1 << shellClass)	
shellIndex = shellClassBaseIndex + shellClassOffset	
if (shellIndex > 0) {	
col = 0	
maximumPairIndex = shellIndex >> 1	
if (maximumPairIndex > 0) {	
maxIdxBits = Min(maximumPairIndex, MAX_COL_TRUNCATED_UNARY_VAL)	
for (i = 0; i < maxIdxBits; i++) {	
col_mv_greater	S()
col = i + col_mv_greater	
if (col_mv_greater == 0) {	
break	
}	
}	
if (maximumPairIndex > MAX_COL_TRUNCATED_UNARY_VAL &&	
col == MAX_COL_TRUNCATED_UNARY_VAL) {	
n = maximumPairIndex - 1	
col_remainder	NS(n)
col = col_remainder + MAX_COL_TRUNCATED_UNARY_VAL	
}	
}	
skipCodingColBit = (col == maximumPairIndex) &&	
((shellIndex & 1) == 0)	
if (skipCodingColBit) {	
diffMv[1] = maximumPairIndex	
} else {	
col_mv_index	S()
if (col_mv_index == 0) {	
diffMv[1] = col	
} else {	
diffMv[1] = shellIndex - col	
}	
}	
diffMv[0] = shellIndex - diffMv[1]	
shift = MV_PRECISION_EIGHTH_PEL - MvPrecision	
diffMv[0] = diffMv[0] << shift	
diffMv[1] = diffMv[1] << shift	
}	

}	
return diffMv	
}	

§ 5.20.7.21. MV component syntax

read_mv_component(comp) {	Descriptor
amvd_index	S()
return Amvd_Index_To_Mvd[amvd_index]	
}	

where the constant table Amvd_Index_To_Mvd is defined as:

```
Amvd_Index_To_Mvd[ MAX_AMVD_INDEX ] = {
    2, 4, 6, 8, 16, 32, 64, 128
}
```

§ 5.20.7.22. Compute prediction syntax

compute_prediction() {	Descriptor
sbMask = Num_4x4_Blocks_Wide[SbSize] - 1	
for (plane = PlaneStart; plane < 1 + HasChroma * 2; plane++) {	
planeSz = get_plane_residual_size(plane > 0 ? ChromaMiSize : MiSize,	
plane)	
num4x4W = Num_4x4_Blocks_Wide[planeSz]	
num4x4H = Num_4x4_Blocks_High[planeSz]	
log2W = MI_SIZE_LOG2 + Mi_Width_Log2[planeSz]	
log2H = MI_SIZE_LOG2 + Mi_Height_Log2[planeSz]	
subX = (plane > 0) ? SubsamplingX : 0	
subY = (plane > 0) ? SubsamplingY : 0	
candRow = plane > 0 ? ChromaMiRow : MiRow	
candCol = plane > 0 ? ChromaMiCol : MiCol	
baseX = (candCol >> subX) * MI_SIZE	
baseY = (candRow >> subY) * MI_SIZE	
subBlockMiRow = candRow & sbMask	
subBlockMiCol = candCol & sbMask	
if (FrameIsIntra) {	
sub8x8Inter = 0	
} else {	
sub8x8Inter = (plane > 0 && MiSize != ChromaMiSize)	
}	
isInterIntra = is_inter && RefFrame[1] == INTRA_FRAME && !sub8x8Inter	
if (isInterIntra) {	
if (interintra_mode == II_DC_PRED) {	

mode = DC_PRED	
}	
else if (interintra_mode == II_V_PRED) mode = V_PRED	
else if (interintra_mode == II_H_PRED) mode = H_PRED	
else mode = SMOOTH_PRED	
predict_intra(plane, baseX, baseY,	
plane == 0 ? AvailL : AvailLChroma,	
plane == 0 ? AvailU : AvailUChroma,	
count_top_right_avail(plane,	
(subBlockMiCol >> subX),	
(subBlockMiRow >> subY),	
num4x4W),	
count_bottom_left_avail(plane,	
(subBlockMiCol >> subX),	
(subBlockMiRow >> subY),	
num4x4H),	
mode,	
log2W, log2H)	
for (i = 0; i < num4x4H * 4; i++) {	
for (j = 0; j < num4x4W * 4; j++) {	
IntraPred[i][j] =	
CurrFrame[plane][baseY + i][baseX + j]	
}	
}	
if (is_inter) {	
for (r = 0; r < num4x4H << subY ; r++) {	
for (c = 0; c < num4x4W << subX ; c++) {	
if (FrameIsIntra) {	
doBlock = r==0 && c==0	
predSize = plane > 0 ? ChromaMiSize : MiSize	
mvRow = MiRow	
mvCol = MiCol	
} else {	
mvRow = candRow + r	
mvCol = candCol + c	
doBlock = mvRow < MiRows && mvCol < MiCols &&	
MiRowBase[0][mvRow][mvCol] == mvRow &&	
MiColBase[0][mvRow][mvCol] == mvCol	
predSize = MiSizes[0][mvRow][mvCol]	
}	

if (doBlock) {	
predW = Block_Width[predSize] >> subX	
predH = Block_Height[predSize] >> subY	
x = (c * 4) >> subX	
y = (r * 4) >> subY	
predict_inter(plane, baseX + x, baseY + y,	
predW, predH,	
mvRow, mvCol, 0, sub8x8Inter)	
}	
}	
}	
if (isInterIntra) {	
h = num4x4H * 4	
w = num4x4W * 4	
if (compound_type == COMPOUND_WEDGE && plane == 0) {	
wedge_mask(w, h)	
} else if (compound_type == COMPOUND_INTRA) {	
intra_mode_variant_mask(w, h)	
}	
mask_blend(plane, baseX, baseY, w, h)	
}	
}	
}	
}	

[§](#) 5.20.7.23. Residual syntax

residual() {	Descriptor
widthChunks = Max(1, Block_Width[MiSize] >> 6)	
heightChunks = Max(1, Block_Height[MiSize] >> 6)	
miSizeChunk = (widthChunks > 1 heightChunks > 1) ? BLOCK_64X64 : MiSize	
doubleChromaW = SubsamplingX && widthChunks > 1 && !Lossless	
doubleChromaH = SubsamplingY && heightChunks > 1 && !Lossless	
for (startChunkY = 0; startChunkY < heightChunks; startChunkY += 2) {	
for (startChunkX = 0; startChunkX < widthChunks; startChunkX += 2) {	
for(chunkY = startChunkY;	
chunkY < Min(startChunkY + 2, heightChunks) ; chunkY++) {	
for (chunkX = startChunkX;	
chunkX < Min(startChunkX + 2, widthChunks); chunkX++) {	
miRowChunk = MiRow + (chunkY << 4)	
miColChunk = MiCol + (chunkX << 4)	
update_ibc_buffers(miRowChunk, miColChunk)	

isCfl = !is_inter && UVMode == UV_CFL_PRED	
atStart = (!doubleChromaW (chunkX&1) == 0) &&	
(!doubleChromaH (chunkY&1) == 0)	
atEnd = (!doubleChromaW (chunkX&1) == 1) &&	
(!doubleChromaH (chunkY&1) == 1)	
if (HasChroma && isCfl && (doubleChromaW doubleChromaH)) {	
doChromaParse = atStart	
doChromaRecon = 0	
doChromaReconAfter = atEnd	
} else {	
doChromaParse = HasChroma && atStart	
doChromaRecon = doChromaParse	
doChromaReconAfter = 0	
}	
for (plane = PlaneStart; plane < 1 + doChromaParse * 2; plane++) {	
if (plane > 0 && ChromaMiSize != MiSize) {	
planeSz = get_plane_residual_size(ChromaMiSize, plane)	
} else {	
planeSz = get_plane_residual_size(miSizeChunk, plane)	
}	
num4x4W = Num_4x4_Blocks_Wide[planeSz]	
num4x4H = Num_4x4_Blocks_High[planeSz]	
doRecon = plane == 0 doChromaRecon	
doPred = doRecon	
if (plane > 0 && doubleChromaW) {	
num4x4W = num4x4W << 1	
}	
if (plane > 0 && doubleChromaH) {	
num4x4H = num4x4H << 1	
}	
subX = (plane > 0) ? SubsamplingX : 0	
subY = (plane > 0) ? SubsamplingY : 0	
if (miRowChunk < MiRows && miColChunk < MiCols) {	
baseXBlock =	
(plane > 0 ? ChromaMiCol >> subX : MiCol) * MI_SIZE	
baseYBlock =	
(plane > 0 ? ChromaMiRow >> subY : MiRow) * MI_SIZE	
txSz = Lossless ? TX_4X4 : get_tx_size(plane, TX_4X4)	
stepX = Tx_Width[txSz] >> 2	
stepY = Tx_Height[txSz] >> 2	
allowCorners = 1	

if (plane == 0 &&	
LumaTxScanOrder[miRowChunk][miColChunk]) {	
for (x4 = 0; x4 < num4x4W; x4 += stepX) {	
col = miColChunk + x4	
for (y4 = 0; y4 < num4x4H; y4 += stepY) {	
row = miRowChunk + y4	
if (row >= MiRows col >= MiCols) {	
break	
}	
txSz = LumaTxSizes[row][col]	
allowCorners = !LumaTxMiddle[row][col]	
stepX = Tx_Width[txSz] >> 2	
stepY = Tx_Height[txSz] >> 2	
transform_block(plane, baseXBlock, baseYBlock,	
txSz,	
x4 + ((chunkX << 4) >> subX),	
y4 + ((chunkY << 4) >> subY),	
allowCorners, doParse = 1,	
doPred = 1, doRecon = 1,	
eob = 0)	
}	
if (col >= MiCols) {	
break	
}	
}	
} else {	
for (y4 = 0; y4 < num4x4H; y4 += stepY) {	
for (x4 = 0; x4 < num4x4W; x4 += stepX) {	
if (plane == 0) {	
row = miRowChunk + y4	
col = miColChunk + x4	
if (row >= MiRows col >= MiCols) {	
break	
}	
txSz = LumaTxSizes[row][col]	
allowCorners = !LumaTxMiddle[row][col]	
stepX = Tx_Width[txSz] >> 2	
stepY = Tx_Height[txSz] >> 2	
}	
eobs[plane] =	
transform_block(plane, baseXBlock,	

baseYBlock, txSz,	
x4 + ((chunkX << 4) >> subX),	
y4 + ((chunkY << 4) >> subY),	
allowCorners, doParse = 1,	
doPred, doRecon, eob = 0)	
}	
if (plane == 0 && row >= MiRows) {	
break	
}	
}	
}	
}	
if (doChromaReconAfter) {	
for (plane = 1; plane < 3; plane++) {	
miRowChunk = MiRow + ((chunkY - doubleChromaH) << 4)	
miColChunk = MiCol + ((chunkX - doubleChromaW) << 4)	
if (miRowChunk < MiRows && miColChunk < MiCols) {	
subX = SubsamplingX	
subY = SubsamplingY	
baseXBlock = (ChromaMiCol >> subX) * MI_SIZE	
baseYBlock = (ChromaMiRow >> subY) * MI_SIZE	
txSz = get_tx_size(plane, TX_4X4)	
transform_block(plane, baseXBlock, baseYBlock, txSz,	
(((chunkX - doubleChromaW) << 4) >> subX),	
(((chunkY - doubleChromaH) << 4) >> subY),	
allowCorners = 1, doParse = 0, doPred = 1,	
doRecon = 1, eobs[plane])	
}	
}	
}	
}	
}	
}	
}	

[§](#) 5.20.7.24. Transform block syntax

transform_block(plane, baseX, baseY, txSz, x, y, allowCorners, doParse, doPred, doRecon, eob) {	Descriptor
startX = baseX + 4 * x	
startY = baseY + 4 * y	

subX = (plane > 0) ? SubsamplingX : 0	
subY = (plane > 0) ? SubsamplingY : 0	
maxX = (MiCols * MI_SIZE) >> subX	
maxY = (MiRows * MI_SIZE) >> subY	
if (startX >= maxX startY >= maxY) {	
return 0	
}	
row = (startY << subY) >> MI_SIZE_LOG2	
col = (startX << subX) >> MI_SIZE_LOG2	
if (plane == 0 !is_cctx_allowed()) {	
if (doPred) {	
make_intra_prediction(plane, startX, startY, txSz, x, y, allowCorners)	
}	
if (!skip_flag) {	
if (doParse) {	
eob = coeffs(plane, startX, startY, txSz)	
}	
if (doParse && eob > 0) {	
dequant(plane, txSz)	
save_dequant(plane, txSz)	
}	
if (doRecon && eob > 0) {	
get_dequant(plane, txSz, CCTX_NONE)	
reconstruct(plane, startX, startY, txSz)	
}	
}	
store_tx_info(plane, row, col, txSz, eob, doParse, doPred)	
return eob	
} else if (plane == 1) {	
return 0	
} else {	
if (doParse && !skip_flag) {	
for (p = 1; p <= 2; p++) {	
eob = coeffs(p, startX, startY, txSz)	
CctxEobs[p] = eob	
if (eob > 0) {	
dequant(p, txSz)	
}	
save_dequant(p, txSz)	
}	
}	
}	

for (p = 1; p <= 2; p++) {	
if (doPred) {	
make_intra_prediction(p, startX, startY, txSz, x, y,	
allowCorners)	
}	
if (doRecon && !skip_flag) {	
planeEob = CctxEobs[p]	
if (planeEob > 0 cctx_type != CCTX_NONE) {	
get_dequant(p, txSz, cctx_type)	
reconstruct(p, startX, startY, txSz)	
}	
}	
store_tx_info(p, row, col, txSz, 0, doParse, doPred)	
}	
return 0	
}	
}	

The function store_tx_info is defined as:

```

store_tx_info(plane, row, col, txSz, eob, doParse, doPred) {
    subX = (plane > 0) ? SubsamplingX : 0
    subY = (plane > 0) ? SubsamplingY : 0
    sbMask = Num_4x4_Blocks_Wide[ SbSize ] - 1
    subBlockMiRow = row & sbMask
    subBlockMiCol = col & sbMask
    stepX = Tx_Width[ txSz ] >> MI_SIZE_LOG2
    stepY = Tx_Height[ txSz ] >> MI_SIZE_LOG2
    for ( i = 0; i < stepY; i++ ) {
        for ( j = 0; j < stepX; j++ ) {
            if ( doParse ) {
                if ( plane == 0 ) {
                    LrTxSkip[ row + i ][ col + j ] = skip_flag || (eob == 0)
                }
                DeblockingTxSizes[ plane ]
                    [ (row >> subY) + i ]
                    [ (col >> subX) + j ] = txSz
                TxColBase[ plane ]
                    [ (row >> subY) + i ]
                    [ (col >> subX) + j ] = col
                TxRowBase[ plane ]
                    [ (row >> subY) + i ]
                    [ (col >> subX) + j ] = row
            }
            if ( doPred ) {
                BlockDecoded[ plane ]
                    [ ( subBlockMiRow >> subY ) + i ]
                    [ ( subBlockMiCol >> subX ) + j ] = 1
            }
        }
    }
}

```

The function `make_intra_prediction` (which calls intra prediction processes) is defined as:

```
make_intra_prediction(plane, startX, startY, txSz, x, y, allowCorners) {
    if ( !is_inter ) {
        stepX = Tx_Width[ txSz ] >> MI_SIZE_LOG2
        stepY = Tx_Height[ txSz ] >> MI_SIZE_LOG2
        subX = (plane > 0) ? SubsamplingX : 0
        subY = (plane > 0) ? SubsamplingY : 0
        row = ( startY << subY ) >> MI_SIZE_LOG2
        col = ( startX << subX ) >> MI_SIZE_LOG2
        sbMask = Num_4x4_Blocks_Wide[ SbSize ] - 1
        subBlockMiRow = row & sbMask
        subBlockMiCol = col & sbMask
        if ( plane == 0 && PaletteSizeY ) {
            predict_palette( startX, startY, x, y, txSz )
        } else {
            isCfl = ( plane > 0 && UVMode == UV_CFL_PRED )
            if ( plane == 0 ) {
                mode = YMode
            } else {
                mode = ( isCfl ) ? DC_PRED : UVMode
            }
            log2W = Tx_Width_Log2[ txSz ]
            log2H = Tx_Height_Log2[ txSz ]
            predict_intra( plane, startX, startY,
                ( plane == 0 ? AvailL : AvailLChroma ) || x > 0,
                ( plane == 0 ? AvailU : AvailUChroma ) || y > 0,
                allowCorners ? count_top_right_avail( plane,
                    ( subBlockMiCol >> subX ),
                    ( subBlockMiRow >> subY ),
                    stepX ) : 0,
                allowCorners ? count_bottom_left_avail( plane,
                    ( subBlockMiCol >> subX ),
                    ( subBlockMiRow >> subY ),
                    stepY ) : 0,
                mode,
                log2W, log2H )
            if ( isCfl ) {
                predict_chroma_from_luma( plane, startX, startY, txSz )
            }
        }
    }
}
```

The functions `count_top_right_avail` and `count_bottom_left_avail` (which count how many samples have already been decoded in the corners) are defined as:

```
count_top_right_avail(plane, x4, y4, w4) {
    numTopRight = 0
    for ( i = 0; i < w4; i++ ) {
        if ( BlockDecoded[ plane ][ y4 - 1 ][ x4 + w4 + i ] ) {
            numTopRight = i + 1
        } else {
            break
        }
    }
    return numTopRight
}

count_bottom_left_avail(plane, x4, y4, h4) {
    numBottomLeft = 0
    for ( i = 0; i < h4; i++ ) {
        if ( BlockDecoded[ plane ][ y4 + h4 + i ][ x4 - 1 ] ) {
            numBottomLeft = i + 1
        } else {
            break
        }
    }
}
```

```

    }
    return numBottomLeft
}

```

§ 5.20.7.25. Get TX size function

```

get_tx_size( plane, txSz ) {
    if ( plane == 0 ) {
        return txSz
    }
    uvTx = Max_Tx_Size_Rect[ get_plane_residual_size( ChromaMiSize, plane ) ]
    return uvTx
}

```

§ 5.20.7.26. Get plane residual size function

The `get_plane_residual_size` function returns the size of a [residual block](#) for the specified plane. (The [residual block](#) will always have width and height at least equal to 4.)

```

get_plane_residual_size( subsize, plane ) {
    subx = plane > 0 ? SubsamplingX : 0
    suby = plane > 0 ? SubsamplingY : 0
    return Subsampled_Size[ subsize ][ subx ][ suby ]
}

```

The `Subsampled_Size` table is defined as:

```

Subsampled_Size[ BLOCK_SIZES ][ 2 ][ 2 ] = {
    { { BLOCK_4X4,    BLOCK_4X4 }, { BLOCK_4X4,    BLOCK_4X4 } },
    { { BLOCK_4X8,    BLOCK_4X4 }, { BLOCK_INVALID, BLOCK_4X4 } },
    { { BLOCK_8X4,    BLOCK_INVALID }, { BLOCK_4X4,    BLOCK_4X4 } },
    { { BLOCK_8X8,    BLOCK_8X4 }, { BLOCK_4X8,    BLOCK_4X4 } },
    { { BLOCK_8X16,   BLOCK_8X8 }, { BLOCK_4X16,   BLOCK_4X8 } },
    { { BLOCK_16X8,   BLOCK_16X4 }, { BLOCK_8X8,    BLOCK_8X4 } },
    { { BLOCK_16X16,  BLOCK_16X8 }, { BLOCK_8X16,   BLOCK_8X8 } },
    { { BLOCK_16X32,  BLOCK_16X16 }, { BLOCK_8X32,   BLOCK_8X16 } },
    { { BLOCK_32X16,  BLOCK_32X8 }, { BLOCK_16X16,   BLOCK_16X8 } },
    { { BLOCK_32X32,  BLOCK_32X16 }, { BLOCK_16X32,   BLOCK_16X16 } },
    { { BLOCK_32X64,  BLOCK_32X32 }, { BLOCK_16X64,   BLOCK_16X32 } },
    { { BLOCK_64X32,  BLOCK_64X16 }, { BLOCK_32X32,   BLOCK_32X16 } },
    { { BLOCK_64X64,  BLOCK_64X32 }, { BLOCK_32X64,   BLOCK_32X32 } },
    { { BLOCK_64X128, BLOCK_64X64 }, { BLOCK_INVALID, BLOCK_32X64 } },
    { { BLOCK_128X64, BLOCK_INVALID }, { BLOCK_64X64,   BLOCK_64X32 } },
    { { BLOCK_128X128, BLOCK_128X64 }, { BLOCK_64X128, BLOCK_64X64 } },
    { { BLOCK_128X256, BLOCK_128X128 }, { BLOCK_INVALID, BLOCK_64X128 } },
    { { BLOCK_256X128, BLOCK_INVALID }, { BLOCK_128X128, BLOCK_128X64 } },
    { { BLOCK_256X256, BLOCK_256X128 }, { BLOCK_128X256, BLOCK_128X128 } },
    { { BLOCK_4X16,   BLOCK_4X8 }, { BLOCK_INVALID, BLOCK_4X8 } },
    { { BLOCK_16X4,   BLOCK_INVALID }, { BLOCK_8X4,    BLOCK_8X4 } },
    { { BLOCK_8X32,   BLOCK_8X16 }, { BLOCK_4X32,   BLOCK_4X16 } },
    { { BLOCK_32X8,   BLOCK_32X4 }, { BLOCK_16X8,   BLOCK_16X4 } },
    { { BLOCK_16X64,  BLOCK_16X32 }, { BLOCK_8X64,   BLOCK_8X32 } },
    { { BLOCK_64X16,  BLOCK_64X8 }, { BLOCK_32X16,   BLOCK_32X8 } },
    { { BLOCK_4X32,   BLOCK_4X16 }, { BLOCK_INVALID, BLOCK_4X16 } },
    { { BLOCK_32X4,   BLOCK_INVALID }, { BLOCK_16X4,   BLOCK_16X4 } },
    { { BLOCK_8X64,   BLOCK_8X32 }, { BLOCK_INVALID, BLOCK_4X32 } },
    { { BLOCK_64X8,   BLOCK_INVALID }, { BLOCK_32X8,   BLOCK_32X4 } }
}

```

§ 5.20.7.27. Coefficients syntax

coeffs(plane, startX, startY, txSz) {	Descriptor
x4 = startX >> 2	
y4 = startY >> 2	
w4 = Tx_Width[txSz] >> 2	
h4 = Tx_Height[txSz] >> 2	
txSzCtx = (Tx_Size_Sqr[txSz] + Tx_Size_Sqr_Up[txSz] + 1) >> 1	
ptype = plane > 0	
segEob = Min(32, Tx_Width[txSz]) * Min(Tx_Height[txSz], 32)	
for (c = 0; c < segEob; c++) {	
Quant[c] = 0	
QuantSign[c] = 0	
}	
for (i = 0; i < Min(32, 4 * h4); i++) {	
for (j = 0; j < Min(32, 4 * w4); j++) {	
Dequant[i][j] = 0	
Level[i][j] = 0	
}	
}	
eob = 0	
culLevel = 0	
dcCategory = 0	
all_zero	S()
if (all_zero) {	
if (plane == 1) {	
EobU = 0	
cctx_type = 0	
}	
c = 0	
if (plane == 0) {	
for (i = 0; i < w4; i++) {	
for (j = 0; j < h4; j++) {	
TxTypes[y4 + j][x4 + i] = DCT_DCT	
}	
}	
}	
} else {	
eobMultisize = Min(Tx_Width_Log2[txSz], 5) +	
Min(Tx_Height_Log2[txSz], 5) - 4	
eobCtx = (plane > 0) ? 2 : is_inter	
if (eobMultisize == 0) {	

eob_pt_16	S()
eobPt = eob_pt_16 + 1	
} else if (eobMultisize == 1) {	
eob_pt_32	S()
eobPt = eob_pt_32 + 1	
} else if (eobMultisize == 2) {	
eob_pt_64	S()
eobPt = eob_pt_64 + 1	
} else if (eobMultisize == 3) {	
eob_pt_128	S()
eobPt = eob_pt_128 + 1	
} else if (eobMultisize == 4) {	
eob_pt_256	S()
if (eob_pt_256 == 7) {	
eob_pt_256_extra	L(1)
eobPt = 8 + eob_pt_256_extra	
} else {	
eobPt = eob_pt_256 + 1	
}	
} else if (eobMultisize == 5) {	
eob_pt_512	S()
if (eob_pt_512 == 7) {	
eob_pt_512_extra	L(2)
eobPt = 8 + eob_pt_512_extra	
} else {	
eobPt = eob_pt_512 + 1	
}	
} else {	
eob_pt_1024	S()
if (eob_pt_1024 == 7) {	
eob_pt_1024_extra	L(2)
eobPt = 8 + eob_pt_1024_extra	
} else {	
eobPt = eob_pt_1024 + 1	
}	
}	
eob = (eobPt < 2) ? eobPt : ((1 << (eobPt - 2)) + 1)	
if (eobPt >= 3) {	
eob_extra	S()
if (eob_extra) {	
eob += 1 << (eobPt - 3)	

}	
for (i = eobPt - 4; i >= 0; i--) {	
eob_extra_bit	L(1)
if (eob_extra_bit) {	
eob += 1 << i	
}	
}	
if (plane == 0) {	
transform_type(x4, y4, txSz, eob)	
} else if (plane == 1) {	
if ((is_inter eob != 1) && is_cctx_allowed()) {	
cctx_type	S()
} else {	
cctx_type = 0	
}	
}	
PlaneTxType = compute_tx_type(plane, txSz, x4, y4)	
txClass = get_tx_class(PlaneTxType)	
scan = get_scan(txSz, txClass)	
useFsc = enable_fsc && PlaneTxType == IDTX && plane == 0 &&	
(fsc_mode is_inter)	
if (plane == 1) {	
EobU = eob	
}	
parityHiding = allow_parity_hiding && !Lossless && plane == 0 &&	
PlaneTxType != IDTX	
numNz = 0	
sumAbs1 = 0	
isHidden = 0	
useTcq = allow_tcq && plane == 0 && !Lossless &&	
txClass == TX_CLASS_2D && !useFsc	
tcqState = 0	
hrLevelAvg = 0	
if (useFsc) {	
bob = segEob - eob	
eob = segEob	
for (c = bob; c < eob; c++) {	
pos = scan[c]	
(row, col) = get_tx_row_col(pos, txSz)	
if (c == bob) {	
coeff_base_bob	

	S()
level = coeff_base_bob + 1	
} else {	
coeff_base_idtx	S()
level = coeff_base_idtx	
}	
if (level > NUM_BASE_LEVELS) {	
coeff_br_idtx	S()
level += coeff_br_idtx	
}	
Level[row][col] = level	
}	
for (c = 0; c < eob; c += 1) {	
pos = scan[c]	
(row, col) = get_tx_row_col(pos, txSz)	
level = Level[row][col]	
if (level != 0) {	
idtx_sign	S()
sign = idtx_sign	
} else {	
sign = 0	
}	
(quant, hrLevelAvg) = read_quant(level, pos, 0,	
NUM_BASE_LEVELS + COEFF_BASE_RANGE + 1,	
hrLevelAvg, 0)	
if (pos == 0 && quant > 0) {	
dcCategory = sign ? 1 : 2	
}	
culLevel = Min(4, culLevel + quant)	
if (sign) {	
quant = -quant	
}	
Quant[pos] = quant	
if (level != 0) {	
QuantSign[pos] = sign ? -1 : 1	
}	
}	
} else {	
for (c = eob - 1; c >= 0; c--) {	
pos = scan[c]	
(row, col) = get_tx_row_col(pos, txSz)	
isLf = get_lf_limits(row, col, txClass, plane)	

if (c == eob - 1) {	
coeff_base_eob	S()
level = coeff_base_eob + 1	
} else {	
coeff_base	S()
level = coeff_base	
}	
baseLevels = isLf ? LF_NUM_BASE_LEVELS : NUM_BASE_LEVELS	
if (level > baseLevels && !(isLf && plane > 0)) {	
coeff_br	S()
level += coeff_br	
}	
if (useTcq) {	
tcqState = Tcq_Next_State[tcqState][level & 1]	
}	
if (parityHiding) {	
if (c > 0) {	
sumAbs1 ^= Min(level,	
NUM_BASE_LEVELS + COEFF_BASE_RANGE + 1) & 1	
if (level != 0) {	
numNz += 1	
isHidden = numNz >= PHTHRESH	
}	
}	
Level[row][col] = level	
}	
tcqState = 0	
for (c = eob - 1; c >= 0; c -= 1) {	
pos = scan[c]	
(row, col) = get_tx_row_col(pos, txSz)	
level = Level[row][col]	
if (level != 0 (isHidden && c == 0 && sumAbs1 > 0)) {	
if (row == 0 && col == 0 && plane == 0) {	
dc_sign	S()
sign = dc_sign	
} else if (txClass == TX_CLASS_HORIZ && col == 0 &&	
plane == 0) {	
dc_sign_horz_vert	S()
sign = dc_sign_horz_vert	
} else if (txClass == TX_CLASS_VERT && row == 0 &&	

plane == 0) {	
dc_sign_horz_vert	S()
sign = dc_sign_horz_vert	
} else {	
sign_bit	L(1)
sign = sign_bit	
}	
} else {	
sign = 0	
}	
if (get_lf_limits(row, col, txClass, plane)) {	
maxLevel = (plane == 0) ?	
(LF_NUM_BASE_LEVELS + COEFF_BASE_RANGE + 1) :	
(LF_NUM_BASE_LEVELS + 1)	
} else {	
maxLevel = NUM_BASE_LEVELS + COEFF_BASE_RANGE + 1	
}	
if (isHidden && c == 0) {	
maxLevel = NUM_BASE_LEVELS + 1	
}	
(quant,hrLevelAvg) = read_quant(level, pos, isHidden, maxLevel,	
hrLevelAvg, useTcq)	
if (c == 0 && isHidden) {	
quant = 2 * quant + sumAbs1	
}	
if (pos == 0 && quant > 0) {	
dcCategory = sign ? 1 : 2	
}	
culLevel = Min(4, culLevel + quant)	
if (!Lossless && useTcq) {	
q0 = ((tcqState >> 1) & 1)	
tcqState = Tcq_Next_State[tcqState][quant & 1]	
if (quant > 0) {	
quant = quant * 2 - q0	
}	
}	
if (sign) {	
quant = -quant	
}	
Quant[pos] = quant	
}	

}	
}	
for (i = 0; i < w4; i++) {	
AboveLevelContext[plane][x4 + i] = culLevel	
AboveDcContext[plane][x4 + i] = dcCategory	
}	
for (i = 0; i < h4; i++) {	
LeftLevelContext[plane][y4 + i] = culLevel	
LeftDcContext[plane][y4 + i] = dcCategory	
}	
return eob	
}	

where `get_tx_row_col` (which extracts the row and column for a position in raster order) is defined as:

```
get_tx_row_col(pos, txSz) {
  adjTxSz = Adjusted_Tx_Size[ txSz ]
  bwl = Tx_Width_Log2[ adjTxSz ]
  row = pos >> bwl
  col = pos - (row << bwl)
  return (row, col)
}
```

and `get_lf_limits` (which determines if this is a low frequency coefficient) is defined as:

```
get_lf_limits(row, col, txClass, plane) {
  if ( txClass == TX_CLASS_2D ) {
    return plane == 0 ? ((row + col) < 4) : ((row + col) < 1)
  } else if ( txClass == TX_CLASS_HORIZ ) {
    return plane == 0 ? (col < 2) : (col < 1)
  } else {
    return plane == 0 ? (row < 2) : (row < 1)
  }
}
```

and `is_cctx_allowed` is defined as:

<code>is_cctx_allowed()</code> {	Descriptor
is420 = SubsamplingX && SubsamplingY	
planeSz = get_plane_residual_size(ChromaMiSize, 1)	
return enable_cctx &&	
!Lossless &&	
(is420 Block_Width[planeSz] < 32 Block_Height[planeSz] < 32)	
}	

and `Tcq_Next_State` (which updates the [TCQ](#) state based on the current state and parity) is defined as:

```
Tcq_Next_State[ 8 ][ 2 ] = {
  {0, 4},
  {4, 0},

```

```

{1, 5},
{5, 1},
{6, 2},
{2, 6},
{7, 3},
{3, 7}
}

```

§ 5.20.7.28. Read quantized coefficient syntax

read_quant(level, pos, isHidden, maxLevel, hrLevelAvg, allowTcq) {	Descriptor
quant = level	
if (quant >= maxLevel - allowTcq) {	
lvlShift = (pos == 0 && isHidden) ? 1 : 0	
predLevel = hrLevelAvg >> lvlShift	
m = Clip3(1, 6, GetMsb(predLevel))	
k = m + 1	
cMax = Min(m + 4, 6)	
for (q = 0 ; q < cMax; q++) {	
q_length_bit	L(1)
if (q_length_bit) {	
break	
}	
}	
if (q == cMax) {	
length = -1	
do {	
length++	
golomb_length_bit	L(1)
} while (!golomb_length_bit)	
length += k	
xBase = (q << m) + (1 << length) - (1 << k)	
} else {	
length = m	
xBase = q << m	
}	
coeff_rem	L(length)
x = xBase + coeff_rem	
hrLevelAvg = ((x << lvlShift) + hrLevelAvg) >> 1	
quant += x << (allowTcq ? 1 : 0)	
}	
return (quant, hrLevelAvg)	
}	

§ 5.20.7.29. Compute transform type function

```

compute_tx_type( plane, txSz, blockX, blockY ) {
    if ( Lossless && plane == 0 && fsc_mode ) {
        return IDTX
    }
    if ( Lossless ) {
        if ( !is_inter ) {
            fscMode = PlaneStart == 0 ? fsc_mode :
                                   FscModes[ ChromaMiRow ][ ChromaMiCol ]

            if ( fscMode ) {
                return IDTX
            } else {
                return DCT_DCT
            }
        }
        if ( is_inter && txSz != TX_4X4 ) {
            return IDTX
        }
        if ( plane > 0 ) {
            x4 = Max( MiCol, blockX << SubsamplingX )
            y4 = Max( MiRow, blockY << SubsamplingY )
            if ( is_inter && LumaTxSizes[ y4 ][ x4 ] != TX_4X4 ) {
                return IDTX
            }
            if ( !FrameIsIntra &&
                MiRow == ChromaMiRow && MiCol == ChromaMiCol ) {
                return TxTypes[ y4 ][ x4 ]
            }
            return TxTypes[ MiRow ][ MiCol ]
        }
    }
    txSet = get_tx_set( txSz, plane )
    if ( plane == 0 ) {
        return TxTypes[ blockY ][ blockX ]
    }
    if ( enable_chroma_dctonly ) {
        return DCT_DCT
    }
    if ( is_inter ) {
        x4 = Max( MiCol, blockX << SubsamplingX )
        y4 = Max( MiRow, blockY << SubsamplingY )
        txType = TxTypes[ y4 ][ x4 ]
        if ( !is_tx_type_in_set( txSet, txType ) ) {
            return DCT_DCT
        }
        return txType
    }
    if ( is_directional_mode( UVMode ) ) {
        pAngle = Mode_To_Angle[ UVMode ] + AngleDeltaUV * ANGLE_STEP
        (mode, unusedAngle) = wide_angle_mapping( UVMode, Tx_Width[ txSz ],
                                                  Tx_Height[ txSz ], pAngle )

        txType = Mode_To_Txfrm[ mode ]
    } else {
        txType = Mode_To_Txfrm[ UVMode ]
    }
    if ( !is_tx_type_in_set( txSet, txType ) ) {
        return DCT_DCT
    }
    return txType
}

is_tx_type_in_set( txSet, txType ) {
    return is_inter ? Tx_Type_In_Set_Inter[ txSet ][ txType ] :
                   Tx_Type_In_Set_Intra[ txSet ][ txType ]
}

```


where the tables Tx_Type_In_Set_Inter and Tx_Type_In_Set_Intra are specified as follows:

```
Tx_Type_In_Set_Intra[ TX_SET_TYPES_INTRA ][ TX_TYPES ] = {
{
1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
},
{
1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0
},
{
1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0
},
{
1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0
},
{
1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1
},
{
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
},
{
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
}
}

Tx_Type_In_Set_Inter[ TX_SET_TYPES_INTER ][ TX_TYPES ] = {
{
1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
},
{
1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0
},
{
1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0
},
{
1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0
},
{
1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1
},
{
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
},
{
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0
},
{
1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0
},
{
1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0
}
}
```

The function wide_angle_mapping is defined as:

```
wide_angle_mapping(mode, w, h, pAngle) {
    if ((h == 2 * w && pAngle < WAIP_WH_RATIO_2_THRES) ||
        (h == 4 * w && pAngle < WAIP_WH_RATIO_4_THRES) ||
        (h == 8 * w && pAngle < WAIP_WH_RATIO_8_THRES) ||
        (h == 16 * w && pAngle < WAIP_WH_RATIO_16_THRES)) {
        return (D203_PRED, 180 + pAngle)
    } else if ((w == 2 * h && pAngle > 270 - WAIP_WH_RATIO_2_THRES) ||
        (w == 4 * h && pAngle > 270 - WAIP_WH_RATIO_4_THRES) ||
        (w == 8 * h && pAngle > 270 - WAIP_WH_RATIO_8_THRES) ||
```

```

        (w == 16 * h &&
         pAngle > 270 - WAIP_WH_RATIO_16_THRES)) {
            return (D45_PRED, pAngle - 180)
        }
        return (mode, pAngle)
    }
}

```

§ 5.20.7.30. Get scan function

```

get_scan( txSz, txClass ) {
    w = Min( Tx_Width[ txSz ], 32)
    h = Min( Tx_Height[ txSz ], 32)
    if ( txClass == TX_CLASS_VERT ) {
        c = 0
        for ( y = 0; y < h; y++ ) {
            for ( x = 0; x < w; x++ ) {
                out[ c ] = y * w + x
                c += 1
            }
        }
    } else if ( txClass == TX_CLASS_HORIZ ) {
        c = 0
        for ( x = 0; x < w; x++ ) {
            for ( y = 0; y < h; y++ ) {
                out[ c ] = y * w + x
                c += 1
            }
        }
    } else {
        x = 0
        y = 0
        for ( c = 0; c < w*h; c++ ) {
            out[ c ] = y * w + x
            x += 1
            y -= 1
            if ( y < 0 || x >= w ) {
                x += 1
                s = Min(x, h - 1 - y)
                x -= s
                y += s
            }
        }
    }
    return out
}

```

§ 5.20.7.31. Is directional mode function

```

is_directional_mode( mode ) {
    if ( ( mode >= V_PRED ) && ( mode <= D67_PRED ) ) {
        return 1
    }
    return 0
}

```

§ 5.20.7.32. Read CFL alphas syntax

read_cfl_alphas() {	Descriptor
if (!enable_cfl_intra) {	
cfl_mhccp = 1	
} else if (is_mhccp_allowed()) {	
cfl_mhccp	S()

} else {	
cfl_mhccp = 0	
}	
if (cfl_mhccp) {	
cfl_index = CFL_MULTI	
} else {	
cfl_index	S()
}	
if (cfl_index == CFL_MULTI) {	
cfl_mh_dir	S()
}	
if (cfl_index != CFL_EXPLICIT) {	
return	
}	
cfl_alpha_signs	S()
signU = (cfl_alpha_signs + 1) / 3	
signV = (cfl_alpha_signs + 1) % 3	
if (signU != CFL_SIGN_ZERO) {	
cfl_alpha_u	S()
CflAlphaU = 1 + cfl_alpha_u	
if (signU == CFL_SIGN_NEG) {	
CflAlphaU = -CflAlphaU	
}	
} else {	
CflAlphaU = 0	
}	
if (signV != CFL_SIGN_ZERO) {	
cfl_alpha_v	S()
CflAlphaV = 1 + cfl_alpha_v	
if (signV == CFL_SIGN_NEG) {	
CflAlphaV = -CflAlphaV	
}	
} else {	
CflAlphaV = 0	
}	
}	

§ 5.20.8. Coding tools structures

§ 5.20.8.1. Palette mode info syntax

palette_mode_info() {	Descriptor
if (PlaneStart == 0 && YMode == DC_PRED) {	

has_palette_y	S()
if (has_palette_y) {	
palette_size_y_minus_2	S()
PaletteSizeY = palette_size_y_minus_2 + 2	
cacheN = get_palette_cache()	
idx = 0	
for (i = 0; i < cacheN && idx < PaletteSizeY; i++) {	
use_palette_color_cache_y	L(1)
if (use_palette_color_cache_y) {	
palette_colors_y[idx] = PaletteCache[i]	
idx++	
}	
}	
if (idx < PaletteSizeY) {	
palette_colors_y[idx]	L(BitDepth)
idx++	
}	
if (idx < PaletteSizeY) {	
minBits = BitDepth - 3	
palette_num_extra_bits_y	L(2)
paletteBits = minBits + palette_num_extra_bits_y	
}	
while (idx < PaletteSizeY) {	
palette_delta_y	L(paletteBits)
palette_delta_y++	
palette_colors_y[idx] =	
Clip1(palette_colors_y[idx - 1] +	
palette_delta_y)	
range = (1 << BitDepth) - palette_colors_y[idx] - 1	
paletteBits = Min(paletteBits, CeilLog2(range))	
idx++	
}	
sort(palette_colors_y, 0, PaletteSizeY - 1)	
}	
}	

The function `sort( arr, i1, i2 )` sorts a subarray of the array `arr` in-place into ascending order. The subarray to be sorted is between indices `i1` and `i2` inclusive.

The function `get_palette_cache`, which merges the above and left palettes to form a cache, is specified as follows:

```
get_palette_cache( ) {
    r = MiRow
    c = MiCol
    aboveN = 0
    if ( ( r * MI_SIZE ) % 64 ) {
        aboveN = PaletteSizes[ r - 1 ][ c ]
    }
    leftN = 0
    if ( AvailL ) {
        leftN = PaletteSizes[ r ][ c - 1 ]
    }
    aboveIdx = 0
    leftIdx = 0
    n = 0
    while ( aboveIdx < aboveN || leftIdx < leftN ) {
        if ( aboveIdx < aboveN ) {
            val = PaletteColors[ r - 1 ][ c ][ aboveIdx ]
            aboveIdx++
            PaletteCache[ n ] = val
            n++
        }
        if ( leftIdx < leftN ) {
            val = PaletteColors[ r ][ c - 1 ][ leftIdx ]
            leftIdx++
            PaletteCache[ n ] = val
            n++
        }
    }
    return n
}
```

NOTE: `get_palette_cache` interleaves the available palette colors from above and left together.

§ 5.20.8.2. Transform type syntax

<code>transform_type(x4, y4, txSz, eob) {</code>	Descriptor
<code>set = get_tx_set(txSz, 0)</code>	
<code>if (fsc_mode) {</code>	
<code> TxType = IDTX</code>	
<code>} else if (!is_inter && eob==1) {</code>	
<code> TxType = DCT_DCT</code>	
<code>} else if (Lossless && is_inter) {</code>	
<code> if (txSz == TX_4X4) {</code>	
<code> lossless_inter_tx_type</code>	<code>S()</code>
<code> TxType = lossless_inter_tx_type ? IDTX : DCT_DCT</code>	
<code> } else {</code>	
<code> TxType = IDTX</code>	
<code> }</code>	
<code>} else if (set > 0 &&</code>	
<code> !Lossless &&</code>	
<code> !(reduced_tx_set == 2 && is_inter == 0)</code>	

) {	
if (set == TX_SET_WIDE_32 set == TX_SET_HIGH_32) {	
is_long_side_dct	S()
} else {	
is_long_side_dct = 1	
}	
if (is_inter) {	
inter_tx_type	S()
if (set == TX_SET_WIDE_64 set == TX_SET_WIDE_32) {	
TxType = Tx_Type_Inv_Long[is_long_side_dct][0]	
[inter_tx_type]	
} else if (set == TX_SET_HIGH_64 set == TX_SET_HIGH_32) {	
TxType = Tx_Type_Inv_Long[is_long_side_dct][1]	
[inter_tx_type]	
} else if (set == TX_SET_INTER_1) {	
inter_tx_type_offset	S()
TxType = Tx_Type_Inter_Inv_Set1[inter_tx_type * 8 +	
inter_tx_type_offset]	
} else if (set == TX_SET_INTER_2) {	
inter_tx_type_offset	S()
TxType = Tx_Type_Inter_Inv_Set2[inter_tx_type * 8 +	
inter_tx_type_offset]	
} else if (set == TX_SET_DCT_IDTX) {	
TxType = Tx_Type_Inter_Inv_Set3[inter_tx_type]	
} else {	
TxType = Tx_Type_Inter_Inv_Set4[inter_tx_type]	
}	
} else {	
intra_tx_type	S()
if (set == TX_SET_WIDE_64 set == TX_SET_WIDE_32) {	
TxType = Tx_Type_Inv_Long[is_long_side_dct][0]	
[intra_tx_type]	
} else if (set == TX_SET_HIGH_64 set == TX_SET_HIGH_32) {	
TxType = Tx_Type_Inv_Long[is_long_side_dct][1]	
[intra_tx_type]	
} else {	
sizeInfo = Size_Class[txSz]	
intraDir = YMode	
if (is_directional_mode(intraDir)) {	
pAngle = Mode_To_Angle[intraDir] +	
AngleDeltaY * ANGLE_STEP +	

Mrl_Index_To_Delta[MrlIndex]	
(intraDir,unusedAngle) = wide_angle_mapping(intraDir,	
Tx_Width[txSz],	
Tx_Height[txSz], pAngle)	
}	
TxType = Md_Idx_To_Type[sizeInfo][intraDir][intra_tx_type]	
}	
}	
} else {	
TxType = DCT_DCT	
}	
large = Tx_Width[txSz] >= 8 && Tx_Height[txSz] >= 8	
if (!large) {	
eobLim = IST_4X4_HEIGHT	
} else if (txSz == TX_8X8 TxType == ADST_ADST) {	
eobLim = IST_8X8_HEIGHT_RED	
} else {	
eobLim = IST_8X8_HEIGHT	
}	
if ((is_inter ?	
enable_inter_ist && eob > 3 && TxType == DCT_DCT &&	
Tx_Width[txSz] >= 16 && Tx_Height[txSz] >= 16 :	
enable_intra_ist && eob != 1) &&	
!Lossless &&	
(TxType == ADST_ADST TxType == DCT_DCT) &&	
YMode != PAETH_PRED &&	
eob <= eobLim) {	
sec_tx_type	S()
if (sec_tx_type != 0 && !is_inter) {	
most_probable_stx_set	S()
}	
} else {	
sec_tx_type = 0	
}	
for (i = 0; i < (Tx_Width[txSz] >> 2); i++) {	
for (j = 0; j < (Tx_Height[txSz] >> 2); j++) {	
TxTypes[y4 + j][x4 + i] = TxType	
}	
}	
}	

where the inversion tables used in the function are specified as follows:

```
Tx_Type_Inter_Inv_Set1[ 16 ] = {
    IDTX, V_DCT, H_DCT, V_ADST, H_ADST, V_FLIPADST, H_FLIPADST,
    DCT_DCT, ADST_DCT, DCT_ADST, FLIPADST_DCT, DCT_FLIPADST, ADST_ADST,
    FLIPADST_FLIPADST, ADST_FLIPADST, FLIPADST_ADST
}

Tx_Type_Inter_Inv_Set2[ 12 ] = {
    IDTX, V_DCT, H_DCT, DCT_DCT, ADST_DCT, DCT_ADST, FLIPADST_DCT,
    DCT_FLIPADST, ADST_ADST, FLIPADST_FLIPADST, ADST_FLIPADST,
    FLIPADST_ADST
}

Tx_Type_Inter_Inv_Set3[ 2 ] = { IDTX, DCT_DCT }

Tx_Type_Inter_Inv_Set4[ 4 ] = { DCT_DCT, V_DCT, H_DCT, IDTX }

Tx_Type_Inv_Long[ 2 ][ 2 ][ 4 ] = {
    {
        { V_DCT, V_ADST, V_FLIPADST, IDTX },
        { H_DCT, H_ADST, H_FLIPADST, IDTX },
    },
    {
        { DCT_DCT, ADST_DCT, FLIPADST_DCT, H_DCT },
        { DCT_DCT, DCT_ADST, DCT_FLIPADST, V_DCT },
    }
}
```

§ 5.20.8.3. Get transform set function

```
get_tx_set( txSz, plane ) {
    txSzSqr = Tx_Size_Sqr[ txSz ]
    txSzSqrUp = Tx_Size_Sqr_Up[ txSz ]
    if ( txSzSqrUp > TX_32X32 ) {
        if ( txSzSqr >= TX_32X32 ) {
            return TX_SET_DCTONLY
        }
        return (Tx_Width[ txSz ] > Tx_Height[ txSz ]) ? TX_SET_WIDE_64 :
            TX_SET_HIGH_64
    }
    if ( txSzSqrUp == TX_32X32 && txSzSqr != TX_32X32 ) {
        return (Tx_Width[ txSz ] > Tx_Height[ txSz ]) ? TX_SET_WIDE_32 :
            TX_SET_HIGH_32
    }
    if (!is_inter && txSzSqrUp == TX_32X32) {
        return TX_SET_DCTONLY
    }
    reducedTxSet = plane == 0 ? reduced_tx_set : enable_chroma_dctonly
    if ( txSzSqrUp == TX_32X32 || reducedTxSet == 1 ) {
        return is_inter ? TX_SET_DCT_IDTX : TX_SET_INTRA_2
    } else if ( reducedTxSet == 2 ) {
        return TX_SET_DCT_IDTX
    } else if ( reducedTxSet == 3 ) {
        return is_inter ? TX_SET_DCT_IDTX_IDDCT : TX_SET_INTRA_2
    }
    if ( is_inter ) {
        return ( txSzSqr == TX_16X16 ) ? TX_SET_INTER_2 : TX_SET_INTER_1
    }
    return TX_SET_INTRA_1
}
```

§ 5.20.8.4. Palette tokens syntax

```
palette_tokens( ) {
```

```
Descriptor
```


blockHeight = Block_Height[MiSize]	
blockWidth = Block_Width[MiSize]	
onscreenHeight = Min(blockHeight, (MiRows - MiRow) * MI_SIZE)	
onscreenWidth = Min(blockWidth, (MiCols - MiCol) * MI_SIZE)	
if (PlaneStart == 0 && PaletteSizeY) {	
palette_direction = 0	
if (blockWidth < 64 && blockHeight < 64) {	
palette_direction	L(1)
}	
prevIdentityRow = PALETTE_ROW_FLAG_CONTEXTS - 1	
if (palette_direction) {	
outerLim = onscreenWidth	
innerLim = onscreenHeight	
} else {	
innerLim = onscreenWidth	
outerLim = onscreenHeight	
}	
for (i = 0; i < outerLim; i++) {	
identity_row_y	S()
for (j = 0; j < innerLim; j++) {	
if (palette_direction) {	
c = i	
r = j	
} else {	
r = i	
c = j	
}	
if (identity_row_y == 2) {	
ColorMapY[r][c] = palette_direction ?	
ColorMapY[r][c - 1] : ColorMapY[r - 1][c]	
} else if (identity_row_y == 1 && j > 0) {	
ColorMapY[r][c] = palette_direction ?	
ColorMapY[r - 1][c] : ColorMapY[r][c - 1]	
} else if (r == 0 && c == 0) {	
color_index_map_y	NS(PaletteSizeY)
ColorMapY[0][0] = color_index_map_y	
} else {	
get_palette_color_context(ColorMapY, r, c, PaletteSizeY)	
palette_color_idx_y	S()
ColorMapY[r][c] = ColorOrder[palette_color_idx_y]	
}	

}	
prevIdentityRow = identity_row_y	
}	
for (i = 0; i < onscreenHeight; i++) {	
for (j = onscreenWidth; j < blockWidth; j++) {	
ColorMapY[i][j] = ColorMapY[i][onscreenWidth - 1]	
}	
}	
for (i = onscreenHeight; i < blockHeight; i++) {	
for (j = 0; j < blockWidth; j++) {	
ColorMapY[i][j] = ColorMapY[onscreenHeight - 1][j]	
}	
}	
}	
}	

§ 5.20.8.5. Palette color context function

```

get_palette_color_context( colorMap, r, c, n ) {
  for ( i = 0; i < PALETTE_COLORS; i++ ) {
    scores[ i ] = 0
    ColorOrder[ i ] = i
  }
  if ( c > 0 ) {
    neighbor = colorMap[ r ][ c - 1 ]
    scores[ neighbor ] += 2
  }
  if ( ( r > 0 ) && ( c > 0 ) ) {
    neighbor = colorMap[ r - 1 ][ c - 1 ]
    scores[ neighbor ] += 1
  }
  if ( r > 0 ) {
    neighbor = colorMap[ r - 1 ][ c ]
    scores[ neighbor ] += 2
  }
  for ( i = 0; i < PALETTE_NUM_NEIGHBORS; i++ ) {
    maxScore = scores[ i ]
    maxIdx = i
    for ( j = i + 1; j < n; j++ ) {
      if ( scores[ j ] > maxScore ) {
        maxScore = scores[ j ]
        maxIdx = j
      } else if ( scores[ j ] > 0 && scores[ j ] == maxScore &&
        c > 0 && j == colorMap[ r ][ c - 1 ] ) {
        maxScore = scores[ j ]
        maxIdx = j
      }
    }
    if ( maxIdx != i ) {
      maxScore = scores[ maxIdx ]
      maxColorOrder = ColorOrder[ maxIdx ]
      for ( k = maxIdx; k > i; k-- ) {
        scores[ k ] = scores[ k - 1 ]
        ColorOrder[ k ] = ColorOrder[ k - 1 ]
      }
      scores[ i ] = maxScore
      ColorOrder[ i ] = maxColorOrder
    }
  }
}

```

```

ColorContextHash = 0
for ( i = 0; i < PALETTE_NUM_NEIGHBORS; i++ ) {
    ColorContextHash += scores[ i ] * Palette_Color_Hash_Multipliers[ i ]
}
}

```

NOTE: The reference software has an alternative implementation that may be better suited for hardware implementations.

§ 5.20.9. Helper functions

§ 5.20.9.1. *Is inside function*

`is_inside` determines whether a candidate position is inside the current [tile](#).

```

is_inside( candidateR, candidateC ) {
    return ( candidateC >= MiColStart &&
            candidateC < MiColEnd &&
            candidateR >= MiRowStart &&
            candidateR < MiRowEnd )
}

```

§ 5.20.9.2. *Is inside frame function*

`is_inside_frame` determines whether a candidate position is inside the current [frame](#).

```

is_inside_frame( candidateR, candidateC ) {
    return ( candidateC >= 0 &&
            candidateC < MiCols &&
            candidateR >= 0 &&
            candidateR < MiRows )
}

```

§ 5.20.9.3. *Is inside filter region function*

`is_inside_filter_region` determines whether a candidate position is inside the region that is being used for [CDEF](#) and restoration filtering.

```

is_inside_filter_region( candidateR, candidateC ) {
    if ( disable_loopfilters_across_tiles ) {
        return is_inside( candidateR, candidateC )
    } else {
        return is_inside_frame( candidateR, candidateC )
    }
}

```

§ 5.20.9.4. *Clamp MV row function*

```

clamp_mv_row( mvec ) {
    bh4 = Num_4x4_Blocks_High[ MiSize ]
    low = -(MiRow + bh4) * MI_SIZE * 8 - MV_BORDER
    high = (MiRows - MiRow) * MI_SIZE * 8 + MV_BORDER
    return Clip3( low, high, mvec )
}

```

§ 5.20.9.5. Clamp MV col function

```
clamp_mv_col( mvec ) {
    bw4 = Num_4x4_Blocks_Wide[ MiSize ]
    low = -(MiCol + bw4) * MI_SIZE * 8 - MV_BORDER
    high = (MiCols - MiCol) * MI_SIZE * 8 + MV_BORDER
    return Clip3( low, high, mvec )
}
```

§ 5.20.9.6. Clear CDEF function

```
clear_cdef( r, c ) {
    cdef_idx[ r ][ c ] = -1
    num4x4 = Num_4x4_Blocks_Wide[ SbSize ]
    cdefSize4 = Num_4x4_Blocks_Wide[ BLOCK_64X64 ]
    num64x64 = num4x4 / cdefSize4
    for ( i = 0; i < num64x64; i++ ) {
        for ( j = 0; j < num64x64; j++ ) {
            cdef_idx[ r + i * cdefSize4 ][ c + j * cdefSize4 ] = -1
        }
    }
}
```

§ 5.20.10. Filtering structures

§ 5.20.10.1. Read CDEF syntax

read_cdef() {	Descriptor
if ((skip_flag && !cdef_on_skip_txfm_frame_enable)	
!cdef_frame_enable) {	
return	
}	
cdefSize4 = Num_4x4_Blocks_Wide[BLOCK_64X64]	
cdefMask4 = ~(cdefSize4 - 1)	
r = MiRow & cdefMask4	
c = MiCol & cdefMask4	
if (cdef_idx[r][c] == -1) {	
if (CdefStrengths == 1) {	
cdef_idx[r][c] = 0	
} else {	
cdef_index0	S()
if (cdef_index0) {	
cdef_idx[r][c] = 0	
} else if (CdefStrengths == 2) {	
cdef_idx[r][c] = 1	
} else {	
cdef_index_minus_1	S()
cdef_idx[r][c] = cdef_index_minus_1 + 1	
}	
}	

w4 = Num_4x4_Blocks_Wide[MiSize]	
h4 = Num_4x4_Blocks_High[MiSize]	
for (i = r; i < r + h4 ; i += cdefSize4) {	
for (j = c; j < c + w4 ; j += cdefSize4) {	
cdef_idx[i][j] = cdef_idx[r][c]	
}	
}	
}	
}	

§ 5.20.10.2. Read CCSO syntax

read_ccso() {	Descriptor
if (!enable_ccso) {	
return	
}	
shiftRow = CcsoLumaSizeLog2 - MI_SIZE_LOG2	
shiftCol = CcsoLumaSizeLog2 - MI_SIZE_LOG2	
blkH4 = 1 << shiftRow	
blkW4 = 1 << shiftCol	
if ((MiRow & (blkH4 - 1)) (MiCol & (blkW4 - 1))) {	
return	
}	
for (plane = 0; plane < NumPlanes; plane++) {	
if (ccso_planes[plane]) {	
if (!sb_reuse_ccso[plane]) {	
ccso_blk	S()
CcsoBlks[plane][MiRow >> shiftRow][MiCol >> shiftCol] =	
ccso_blk	
}	
}	
}	
}	

§ 5.20.10.3. Read GDF syntax

read_gdf() {	Descriptor
if (!gdf_frame_enable !gdf_per_block) {	
return	
}	
sbSize4 = Num_4x4_Blocks_Wide[SbSize]	
if (MiRow % sbSize4 != 0 MiCol % sbSize4 != 0) {	
return	

}	
sbRow = MiRow / sbSize4	
sbCol = MiCol / sbSize4	
sbPerGdf = GdfBlkSize / Block_Width[SbSize]	
if (sbCol % sbPerGdf != 0) {	
return	
}	
if (sbRow % sbPerGdf != 0) {	
return	
}	
use_gdf	S()
GdfBlks[sbRow / sbPerGdf][sbCol / sbPerGdf] = use_gdf	
}	

§ 5.20.10.4. Read loop restoration syntax

read_lr(row, col, bSize) {	Descriptor
w = Num_4x4_Blocks_Wide[bSize]	
h = Num_4x4_Blocks_High[bSize]	
for (plane = PlaneStart; plane < PlaneEnd; plane++) {	
if (FrameRestorationType[plane] != RESTORE_NONE) {	
subX = (plane == 0) ? 0 : SubsamplingX	
subY = (plane == 0) ? 0 : SubsamplingY	
unitSize = LoopRestorationSize[plane]	
miCols = MiColEnd - MiColStart	
miRows = MiRowEnd - MiRowStart	
lrRowOffset = (MiRowStart * MI_SIZE >> subY) / unitSize	
lrColOffset = (MiColStart * MI_SIZE >> subX) / unitSize	
c = col - MiColStart	
r = row - MiRowStart	
unitRows = count_units_in_frame(unitSize, miRows * MI_SIZE >> subY)	
unitCols = count_units_in_frame(unitSize, miCols * MI_SIZE >> subX)	
unitRowStart = (r * (MI_SIZE >> subY) +	
unitSize - 1) / unitSize	
unitRowEnd = Min(unitRows, ((r + h) * (MI_SIZE >> subY) +	
unitSize - 1) / unitSize)	
unitColStart = (c * (MI_SIZE >> subX) + unitSize - 1) / unitSize	
unitColEnd = Min(unitCols,	
((c + w) * (MI_SIZE >> subX) + unitSize - 1) / unitSize)	
for (unitRow = unitRowStart; unitRow < unitRowEnd; unitRow++) {	
for (unitCol = unitColStart; unitCol < unitColEnd; unitCol++) {	
read_lr_unit(plane, unitRow + lrRowOffset,	

unitCol + lrColOffset)	
}	
}	
}	
}	
}	

where count_units_in_frame is a function specified as:

```
count_units_in_frame(unitSize, frameSize) {
    return Max((frameSize + (unitSize >> 1)) / unitSize, 1)
}
```

§ 5.20.10.5. Read loop restoration unit syntax

read_lr_unit(plane, unitRow, unitCol) {	Descriptor
if (FrameRestorationType[plane] == RESTORE_WIENER_NONSEP) {	
use_wiener_ns	S()
restorationType = use_wiener_ns ? RESTORE_WIENER_NONSEP : RESTORE_NONE	
} else if (FrameRestorationType[plane] == RESTORE_PC_WIENER) {	
use_pc_wiener	S()
restorationType = use_pc_wiener ? RESTORE_PC_WIENER : RESTORE_NONE	
} else {	
restorationType = RESTORE_SWITCHABLE_TYPES - 1	
for (tool = 0; tool < RESTORE_SWITCHABLE_TYPES - 1; tool++) {	
flex_restoration_type	S()
if (flex_restoration_type) {	
restorationType = tool	
break	
}	
}	
LrType[plane][unitRow][unitCol] = restorationType	
if (restorationType == RESTORE_WIENER_NONSEP) {	
read_wienerns_filter(plane, unitRow, unitCol, 0)	
}	
}	

§ 5.20.10.6. Read Wiener NS syntax

read_wienerns_filter(plane, unitRow, unitCol, readFrameFilters) {	Descriptor
numClasses = 1	
if (frame_filters_on[plane]) {	
if (!readFrameFilters) {	

return	
}	
(numClasses, numRefFilters, _, _, _) = search_frame_filters(plane, -1)	
nopcw = lr_tools_disable[0][RESTORE_PC_WIENER]	
groupCounts[0] = numClasses	
groupCounts[1] = numRefFilters	
groupCounts[2] = (plane > 0 nopcw) ?	
0 : 64 - numClasses - numRefFilters	
for (i = 0; i < 3; i++) {	
groupHits[i] = 0	
}	
groupBase[0] = 0	
for (i = 1; i < 3; i++) {	
groupBase[i] = groupBase[i - 1] + groupCounts[i - 1]	
}	
for (c = 0 ; c < numClasses; c++) {	
groupCounts[0] = c + 1	
if (c == 0) {	
predGroup = (groupCounts[1] > 2) ?	
1 : predict_group(groupCounts)	
} else {	
predGroup = predict_group(groupHits)	
}	
numZeros = 0	
altGroup = 0	
for (i = 0; i < 3; i++) {	
if (i != predGroup) {	
if (groupCounts[i] == 0) {	
numZeros += 1	
} else {	
altGroup = i	
}	
}	
}	
if (numZeros == 2) {	
use_alt_group = 0	
} else {	
use_alt_group	f(1)
}	
if (use_alt_group) {	
if (numZeros == 1) {	

group = altGroup	
} else {	
group_bit	f(1)
group = predGroup <= group_bit ? group_bit + 1 : group_bit	
}	
} else {	
group = predGroup	
}	
n = groupCounts[group]	
ref = groupBase[group] + (n >> 1)	
if (n == 1) {	
matchIndices[c] = groupBase[group]	
} else {	
matchIndices[c] = decode_signed_subexp_with_ref(	
groupBase[group],	
groupBase[group] + n, ref, 4)	
}	
groupHits[group]++	
}	
}	
for (c = 0 ; c < numClasses ; c++) {	
if (readFrameFilters) {	
merged_param	f(1)
} else {	
merged_param	L(1)
}	
merged[c] = merged_param	
if (readFrameFilters) {	
refBank[c] = 0	
} else {	
for (k = 0; k < WienerNsBankSize[plane][c] - 1; k++) {	
use_bank	L(1)
if (use_bank) {	
break	
}	
}	
refBank[c] = (WienerNsPtr[plane][c] - k + LR_BANK_SIZE) %	
LR_BANK_SIZE	
}	
}	
for (c = 0 ; c < numClasses ; c++) {	

if (frame_filters_on[plane]) {	
fill_first_slot_of_bank_with_filter_match(c, plane,	
matchIndices[c])	
}	
nCoeffs = plane > 0 ? WIENER_NS_CHROMA_COEFFS :	
WIENER_NS_LUMA_COEFFS	
if (merged[c]) {	
if (WienerNsBankSize[plane][c] == 0) {	
WienerNsBankSize[plane][c] = 1	
}	
} else {	
if (WienerNsBankSize[plane][c] < LR_BANK_SIZE) {	
WienerNsPtr[plane][c] = WienerNsBankSize[plane][c]	
WienerNsBankSize[plane][c] += 1	
} else {	
WienerNsPtr[plane][c] = (WienerNsPtr[plane][c] + 1) %	
LR_BANK_SIZE	
}	
numSubsets = plane == 0 ? 4 : 3	
for (subset = 0; subset < numSubsets - 1; subset++) {	
if (readFrameFilters) {	
wiener_ns_length	f(1)
} else {	
wiener_ns_length	S()
}	
if (wiener_ns_length == 0) {	
break	
}	
}	
if (plane > 0 && subset > 0) {	
if (readFrameFilters) {	
wiener_ns_uv_sym	f(1)
} else {	
wiener_ns_uv_sym	S()
}	
} else {	
wiener_ns_uv_sym = 0	
}	
}	
for (j = 0; j < nCoeffs; j++) {	
min = Wiener_Ns_Taps_Min[plane!=0][j]	

k = Wiener_Ns_Taps_K[plane!=0][j]	
v = RefLrWienerNs[plane][c][refBank[c]][j]	
if (!merged[c]) {	
if (Wiener_Ns_Taps_Present[plane!=0][subset][j]) {	
if (readFrameFilters) {	
v = decode_signed_subexp_with_ref(min, min + (1 << k),	
v, k - 3)	
} else {	
v = decode_signed_4part(min, k, v)	
}	
} else {	
v = 0	
}	
}	
if (readFrameFilters) {	
FrameLrWienerNs[plane][c][j] = v	
if (!merged[c] && plane > 0 &&	
j >= WIENER_NS_SHORT_COEFFS && wiener_ns_uv_sym) {	
FrameLrWienerNs[plane][c][j + 1] = v	
j++	
}	
} else {	
LrWienerNs[plane][unitRow][unitCol][j] = v	
if (!merged[c]) {	
RefLrWienerNs[plane][c]	
[WienerNsPtr[plane][c]][j] = v	
}	
if (!merged[c] && plane > 0 &&	
j >= WIENER_NS_SHORT_COEFFS && wiener_ns_uv_sym) {	
LrWienerNs[plane][unitRow][unitCol][j + 1] = v	
RefLrWienerNs[plane][c]	
[WienerNsPtr[plane][c]][j + 1] = v	
j++	
}	
}	
}	
}	

where `decode_signed_4part` is a function defined as follows:

```

decode_signed_4part(low, k, r) {
    rOffset = r - low
    xOffset = decode_unsigned_4part(k, rOffset)
    x = xOffset + low
    return x
}

decode_unsigned_4part(k, r) {
    mx = 1 << k
    v = decode_4part( 6 - k )
    if ((r << 1) <= mx) {
        offset = inverse_recenter(r, v)
    } else {
        offset = mx - 1 - inverse_recenter(mx - 1 - r, v)
    }
    return offset
}

decode_4part(num) {
    S() wiener_ns_base;
    bits = 2 - num + Max(1, wiener_ns_base)
    offset = wiener_ns_base == 0 ? 0 : (1 << bits)
    L(bits) wiener_ns_rem;
    return offset + wiener_ns_rem
}

```

The function `fill_first_slot_of_bank_with_filter_match` is specified as:

```

fill_first_slot_of_bank_with_filter_match( c, plane, m ) {
    WienerNsPtr[ plane ][ c ] = 0
    WienerNsBankSize[ plane ][ c ] = 1
    (numClasses, numRefFilters, matchIdx, matchCls, matchPlane) =
        search_frame_filters( plane, m )
    for( j = 0; j < ( (plane > 0) ? WIENER_NS_CHROMA_COEFFS :
        WIENER_NS_LUMA_COEFFS ); j++ ) {
        if ( m == 0 ) {
            v = 0
        } else if ( m < numClasses ) {
            oldCls = m - 1
            v = FrameLrWienerNs[ plane ][ oldCls ][ j ]
        } else if ( m < numClasses + numRefFilters ) {
            v = RefFrameLrWienerNs[ matchIdx ][ matchPlane ][ matchCls ][ j ]
        } else {
            v = get_translated_pc_wiener(m - NumFilterClasses - numRefFilters, j)
        }
        RefLrWienerNs[ plane ][ c ][ 0 ][ j ] = v
    }
}

```

The function `search_frame_filters` is specified as:

```

search_frame_filters( plane, target ) {
    nopcw = lr_tools_disable[ 0 ][ RESTORE_PC_WIENER ]
    minPcWiener = (plane > 0 || nopcw) ? 0 : 16
    numClasses = (plane == 0) ? NumFilterClasses : 1
    maxRefFilters = (nopcw ? 16 : 64) - numClasses - minPcWiener
    numRefFilters = 0
    numCheckPlanes = plane > 0 ? 2 : 1
    matchIdx = 0
    matchCls = 0
    matchPlane = plane
    for ( ref = 0; ref < NumTotalRefs; ref++ ) {
        if ( FrameType != SWITCH_FRAME && OrderHints[ref] != RESTRICTED_OH ) {

```

```

idx = ref_frame_idx[ ref ]
for ( check = 0; check < numCheckPlanes; check++ ) {
    if ( check == 0 ) {
        checkPlane = plane
    } else {
        checkPlane = plane == 1 ? 2 : 1
    }
    if ( RefFrameFiltersOn[ idx ][ checkPlane ] ) {
        numRefClasses = (plane == 0) ?
            RefNumFilterClasses[ idx ] : 1
        for ( i = 0; i < numRefClasses; i++ ) {
            if ( numRefFilters < maxRefFilters ) {
                if ( numRefFilters + numClasses == target ) {
                    matchIdx = idx
                    matchCls = i
                    matchPlane = checkPlane
                }
                numRefFilters += 1
            }
        }
    }
}
return (numClasses, numRefFilters, matchIdx, matchCls, matchPlane)
}

```

The function `get_translated_pc_wiener` (which converts a pixel classified filter into a Wiener filter) is specified as:

```

get_translated_pc_wiener( m, j ) {
    if ( j >= 12 ) {
        return 0
    }
    filt = Shuffled_Index[ m ]
    coeff = Round2Signed( Pc_Wiener_Filters[ 0 ][ filt ][ j ],
        PC_WIENER_PREC_BITS - WIENER_NS_PREC_BITS )
    min = Wiener_Ns_Taps_Min[ 0 ][ j ]
    max = min + ( 1 << Wiener_Ns_Taps_K[ 0 ][ j ] ) - 1
    return Clip3(min, max, coeff)
}

```

where `Shuffled_Index` is defined as:

```

Shuffled_Index[ 64 ] = {
    16, 7, 58, 21, 12, 61, 26, 38, 18, 30, 50,
    45, 23, 49, 43, 62, 42, 54, 27, 36, 17, 44,
    32, 34, 4, 24, 52, 31, 37, 11, 33, 19, 35,
    6, 22, 53, 63, 25, 41, 47, 1, 59, 0, 28,
    40, 55, 48, 8, 5, 51, 9, 46, 56, 60, 15,
    2, 13, 14, 57, 29, 3, 20, 39, 10
}

```

The function `predict_group` (which finds which group has the highest count) is specified as:

```

predict_group( counts ) {
    pred = 0
    for ( i = 1; i <= 2; i++ ) {
        if ( counts[ i ] > counts[ pred ] ) {
            pred = i
        }
    }
}

```

```

    }
    return pred
}

```

The table Wiener_Ns_Taps_Present (which specifies which filter taps are present) is specified as:

```

Wiener_Ns_Taps_Present[ 2 ][ 4 ][ WIENER_NS_CHROMA_COEFFS ] = {
{
    { 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0 }
},
{
    { 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 }
}
}

```

[↑ Back to Table of Contents](#)

§ 6. Syntax structures semantics

§ 6.1. General

This section specifies the meaning of the syntax elements read in the syntax structures.

Important variables and function calls are also described.

§ 6.2. OBU semantics

§ 6.2.1. General OBU semantics

An ordered series of OBUs is presented to the decoding process. Each OBU is given to the decoding process as a string of bytes along with a variable `sz` that identifies the total number of bytes in the OBU.

Methods of framing the OBUs (i.e., of identifying the series of OBUs and their size and payload data) in a delivery or container format may be established in a manner outside the scope of this specification. One simple method is described in Annex B.

OBU data starts on the first (most significant) bit and ends on the last bit of the given bytes. The payload of an OBU lies between the first bit of the given bytes and the last bit before the first trailing bit. Trailing bits are always present, unless the OBU consists of only the header. Trailing bits achieve [byte alignment](#) when the payload of an OBU is not byte aligned. The trailing bits may also be used for additional byte padding, and if used are taken into account in the `sz` value. In all cases, the pattern used for the trailing bits guarantees that all OBUs (except header-only OBUs) end with the same pattern: one bit set to one, optionally followed by zeros.

NOTE: As a validity check for malformed encoded data and for operation in environments in which losses and errors can occur, decoders may detect an error if the end of the parsed data is not directly followed by the correct trailing bits pattern or if the parsing of the OBU header and payload leads to the consumption of bits within the trailing bits (except for [Tile Group](#) data which is allowed to read a small distance into the trailing bits as described in [§ 8.2.4 Exit process for symbol decoder](#)).

obu_extension_flag equal to 1 specifies that extension data is present in the OBU payload.

obu_extension_flag equal to 0 specifies that no extension data is present and only trailing bits follow the OBU payload.

It is a requirement of bitstream conformance that **obu_extension_flag** is equal to 0 in bitstreams conforming to this specification.

obu_extension_data_bit is a bit of extension data. The content of this data is not specified in this version of this specification and shall be ignored by conforming decoders.

NOTE: The extension data will end with trailing bits in the usual manner.

§ 6.2.2. OBU header semantics

OBUs are structured with a header and a payload. The header identifies the type of the payload using the **obu_type** header parameter.

obu_header_extension_flag equal to 1 indicates that the obu_header contains the obu_mlayer_id and obu_xlayer_id syntax elements to identify the embedded layer and extended layer of this OBU. obu_header_extension_flag equal to 0 indicates that obu_mlayer_id and obu_xlayer_id are not present and inferred.

NOTE: The inference is defined in [§ 5.2.2 OBU header syntax](#)

obu_type specifies the type of data structure contained in the OBU payload:

Table 6.1: OBU types and their layer-specific status

obu_type	Name of obu_type	Layer-specific
0	Reserved	-
1	OBU_SEQUENCE_HEADER	N
2	OBU_TEMPORAL_DELIMITER	N
3	OBU_MULTI_FRAME_HEADER	Y
4	OBU_CLOSED_LOOP_KEY	Y
5	OBU_OPEN_LOOP_KEY	Y
6	OBU_LEADING_TILE_GROUP	Y
7	OBU_REGULAR_TILE_GROUP	Y
8	OBU_METADATA_SHORT	See Table in § 6.16 Metadata OBU semantics
9	OBU_METADATA_GROUP	See Table in § 6.16 Metadata OBU semantics
10	OBU_SWITCH	Y
11	OBU_LEADING_SEF	Y
12	OBU_REGULAR_SEF	Y
13	OBU_LEADING_TIP	Y
14	OBU_REGULAR_TIP	Y
15	OBU_BUFFER_REMOVAL_TIMING	Y
16	OBU_LAYER_CONFIGURATION_RECORD	N
17	OBU_ATLAS_SEGMENT	N
18	OBU_OPERATING_POINT_SET	N
19	OBU_BRIDGE_FRAME	Y
20	OBU_MSDO	N
21	OBU_RAS_FRAME	Y
22	OBU_QUANTIZATION_MATRIX	Y
23	OBU_FILM_GRAIN	Y
24	OBU_CONTENT_INTERPRETATION	Y
25	OBU_PADDING	Either
26-31	Reserved	-

Reserved OBUs are for future use by AOMedia and shall be ignored by decoders conforming to this version of this specification.

The column “Layer-specific” indicates if the corresponding OBU type is considered to be associated with a specific layer (“Y”), or not (“N”).

Metadata OBU types may or may not be layer-specific, depending on the metadata type. The table in [§6.16 Metadata OBU semantics](#) specifies which types of metadata OBUs are layer-specific and which are not.

Padding OBUs may or may not be layer-specific.

obu_tlayer_id specifies the temporal level of the data contained in the OBU.

obu_mlayer_id specifies the embedded level of the data contained in the OBU.

obu_xlayer_id specifies the extended level of the data contained in the OBU.

If obu_xlayer_id is equal to GLOBAL_XLAYER_ID, it is a requirement of bitstream conformance that both obu_mlayer_id and obu_tlayer_id are equal to 0.

Tile group OBU data associated with obu_tlayer_id and obu_mlayer_id equal to 0 are referred to as the base layer, whereas tile group OBU data that are associated with obu_mlayer_id greater than 0 or obu_tlayer_id greater than 0 are referred to as enhancement layer(s).

It is a requirement of bitstream conformance that obu_tlayer_id is less than or equal to max_tlayer_id obtained from an activated sequence header.

It is a requirement of bitstream conformance that obu_mlayer_id is less than or equal to max_mlayer_id obtained from an activated sequence header.

NOTE: These constraints on obu_tlayer_id and obu_mlayer_id apply after a sequence header OBU is activated to specify max_tlayer_id and max_mlayer_id.

If obu_type is equal to OBU_MSDO or OBU_TEMPORAL_DELIMITER, it is a requirement of bitstream conformance that obu_xlayer_id is equal to GLOBAL_XLAYER_ID.

If obu_xlayer_id is equal to GLOBAL_XLAYER_ID, it is a requirement of bitstream conformance that obu_type is equal to one of OBU_TEMPORAL_DELIMITER, OBU_BUFFER_REMOVAL_TIMING, OBU_METADATA_SHORT, OBU_METADATA_GROUP, OBU_LAYER_CONFIGURATION_RECORD, OBU_ATLAS_SEGMENT, OBU_OPERATING_POINT_SET, OBU_MSDO, or OBU_PADDING.

If obu_type is equal to one of OBU_SEQUENCE_HEADER, OBU_TEMPORAL_DELIMITER, OBU_LAYER_CONFIGURATION_RECORD, OBU_OPERATING_POINT_SET, or OBU_ATLAS_SEGMENT, it is a requirement of bitstream conformance that all of the following are true:

- obu_tlayer_id is equal to 0.
- obu_mlayer_id is equal to 0.

If obu_type is equal to one of OBU_CLOSED_LOOP_KEY, OBU_OPEN_LOOP_KEY, OBU_SWITCH, or OBU_RAS_FRAME, it is a requirement of bitstream conformance that obu_tlayer_id is equal to 0.

§ 6.2.3. Trailing bits semantics

NOTE: Tile group OBUs and frame OBUs do end with trailing bits, but for these cases, the trailing bits are consumed by the `exit_symbol` process.

trailing_one_bit shall be equal to 1.

When the syntax element `trailing_one_bit` is read, it is a requirement that `nbBits` is greater than zero.

trailing_zero_bit shall be equal to 0 and is inserted into the bitstream to align the bit position to a multiple of 8 bits and add optional zero padding bytes to the OBU.

§ 6.2.4. Byte alignment semantics

zero_bit shall be equal to 0 and is inserted into the bitstream to align the bit position to a multiple of 8 bits.

§ 6.3. Reserved OBU semantics

The reserved OBU allows the extension of this specification with additional OBU types in a way that allows older decoders to ignore them.

§ 6.4. Sequence header OBU semantics

§ 6.4.1. General sequence header OBU semantics

seq_header_id specifies an identification number for the sequence header.

It is a requirement of bitstream conformance that `seq_header_id` is less than `MAX_SEQ_NUM`.

seq_profile_idc specifies the profile for the coded video sequence identified by the associated `obu_xlayer_id`. The profile constrains the coding capabilities that may be used, as specified in [Annex A.2 Profiles](#).

NOTE: The value space for `seq_profile_idc` is the same as for `multistream_profile_idc`.

single_picture_header_flag specifies that the syntax elements not needed by a still frame are omitted.

seq_level_idx specifies the level that the coded video sequence conforms to.

seq_tier equal to 0 specifies that the coded video sequence conforms to the main tier. `seq_tier` equal to 1 specifies that the coded video sequence conforms to the high tier.

monotonic_output_order_flag defines the output mode for a coded video sequence associated with this sequence header.

`monotonic_output_order_flag` equal to 1 specifies that the output order of coded output frame units is the same as their decoding order within the associated coded video sequence. `monotonic_output_order_flag` equal to 0 specifies that the output order of coded output frame units can differ from their decoding order within the associated coded video sequence.

NOTE: When `monotonic_output_order_flag` is equal to 1 for an associated coded video sequence, the output order for this coded video sequence is monotonic and the systems or application layer can determine that the presentation time is equal to the decoding time without parsing any frame headers. When `monotonic_output_order_flag` is equal to 0 for an associated coded video sequence, the output order can be non-monotonic for this coded video sequence and the systems or application layer will have to derive the presentation time from coded information associated with each frame.

When `single_picture_header_flag` is equal to 1, `monotonic_output_order_flag` is inferred to be equal to 1.

It is a requirement of bitstream conformance that in a [coded multistream video sequence](#), all extended layers shall be associated with the same value of `monotonic_output_order_flag`.

It is a requirement of bitstream conformance that in a [coded multistream video sequence](#), all extended layers within a temporal unit share the same output time and the coded extended layer units from different extended layers within a temporal unit shall appear in ascending order of `obu_xlayer_id`.

When `monotonic_output_order_flag` is equal to 0, additional display order hint constraints on the temporal unit apply as specified in [§ 7.3.7 Temporal unit](#).

chroma_format_idc specifies the chroma subsampling format.

Table 6.2: Chroma format indicator values

chroma_format_idc	Name of chroma_format_idc	SubsamplingX	SubsamplingY	Monochrome	Description
0	CHROMA_FORMAT_420	1	1	0	YUV 4:2:0
1	CHROMA_FORMAT_400	1	1	1	Monochrome 4:0:0
2	CHROMA_FORMAT_444	0	0	0	YUV 4:4:4
3	CHROMA_FORMAT_422	1	0	0	YUV 4:2:2

It is a requirement of bitstream conformance that `chroma_format_idc` is less than or equal to 3.

bit_depth_idc is used to determine the bit depth.

It is a requirement of bitstream conformance that `bit_depth_idc` is less than or equal to 1.

NOTE: Values of `bit_depth_idc` greater than 1 are reserved for future use by AOMedia.

The function `set_chroma_format_and_bit_depth( )` is defined as follows:

```

set_chroma_format_and_bit_depth( ) {
    if ( chroma_format_idc == CHROMA_FORMAT_420 ) {
        SubsamplingX = 1
        SubsamplingY = 1
    } else if ( chroma_format_idc == CHROMA_FORMAT_444 ) {
        SubsamplingX = 0
        SubsamplingY = 0
    } else if ( chroma_format_idc == CHROMA_FORMAT_422 ) {
        SubsamplingX = 1
        SubsamplingY = 0
    } else if ( chroma_format_idc == CHROMA_FORMAT_400 ) {
        SubsamplingX = 1
        SubsamplingY = 1
    }
}

```

```

BitDepth = lookup_bitdepth( bit_depth_idc )
MaxQ = lookup_maxq( bit_depth_idc )
Monochrome = chroma_format_idc == CHROMA_FORMAT_400
NumPlanes = Monochrome ? 1 : 3
}

```

where `lookup_bitdepth` and `lookup_maxq` are functions that indicate that the bit depth and maximum quantizer value are fetched based on the value of `bit_depth_idc` from the following table:

Table 6.3: Bit depth indicator values

bit_depth_idc	BitDepth	MaxQ
0	10	MAXQ_10_BITS
1	8	MAXQ_8_BITS
Greater than 1	Reserved	Reserved

Monochrome equal to 1 indicates that the video does not contain U and V color planes. Monochrome equal to 0 indicates that the video contains Y, U, and V color planes.

SubsamplingX, **SubsamplingY** specify the chroma subsampling format.

seq_lcr_id specifies the layer configuration record id that corresponds to this sequence header. If this sequence header is associated with a coded video sequence in an extended layer with `obu_xlayer_id` equal to `xLayerId` and if `seq_lcr_id` is not equal to 0, the following applies:

- if an OBU of `obu_type` equal to `OBU_LAYER_CONFIGURATION_RECORD` is associated with the extended layer id `xLayerId` (by having `lcr_local_id` equal to `seq_lcr_id`) and is either present prior to this sequence header in the same bitstream or is provided through external means, then this OBU is associated with this sequence header,
- otherwise, if an OBU of `obu_type` equal to `OBU_LAYER_CONFIGURATION_RECORD` is associated with an `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID` (by having `lcr_global_config_record_id` equal to `seq_lcr_id`) and is either present prior to this sequence header in the same bitstream or is provided through external means, then this OBU is associated with this sequence header,
- otherwise, no OBU of `obu_type` equal to `OBU_LAYER_CONFIGURATION_RECORD` is associated with this sequence header.

It is a requirement of bitstream conformance that when `seq_lcr_id` is not equal to 0 and the activated layer configuration record is a global layer configuration record, the extended layer with `obu_xlayer_id` equal to the `obu_xlayer_id` of the sequence header shall be included in the `lcr_xlayer_map` of the referenced global layer configuration record.

NOTE: See [§ 7.3.8.3 LCR availability](#) for the general availability requirements for layer configuration record OBUs.

still_picture equal to 1 specifies that the coded video sequence contains only one [coded frame](#). `still_picture` equal to 0 specifies that the coded video sequence contains one or more coded frames.

max_tlayer_id specifies the maximum value for `obu_tlayer_id` for the OBUs represented by this sequence header.

max_mlayer_id specifies the maximum value for obu_mlayer_id for the OBUs represented by this sequence header.

seq_max_mlayer_cnt_minus_1 plus 1 specifies the maximum number of embedded layers that can be included in the coded video sequence associated with this sequence header. It is a requirement of bitstream conformance that the value of seq_max_mlayer_cnt_minus_1 is less than or equal to max_mlayer_id. It is a requirement of bitstream conformance that the number of distinct values of obu_mlayer_id present in the coded video sequence associated with this sequence header is less than or equal to SeqMaxMlayerCnt.

NOTE: The counting applies to all OBUs, even if they are not layer-specific. This means that a sequence containing only embedded layer 1 will count as two layers as OBU_SEQUENCE_HEADER is forced to use an embedded layer of 0.

frame_width_bits_minus_1 specifies the number of bits minus 1 used for transmitting the frame width syntax elements.

frame_height_bits_minus_1 specifies the number of bits minus 1 used for transmitting the frame height syntax elements.

max_frame_width_minus_1 specifies the maximum frame width minus 1 for the frames represented by this sequence header.

max_frame_height_minus_1 specifies the maximum frame height minus 1 for the frames represented by this sequence header.

seq_cropping_window_present_flag equal to 1 specifies that the cropping window syntax elements seq_cropping_win_left_offset, seq_cropping_win_right_offset, seq_cropping_win_top_offset, and seq_cropping_win_bottom_offset are present in the sequence header to define a cropping rectangle. seq_cropping_window_present_flag equal to 0 specifies that the cropping window syntax elements are not present and all crop offset values are inferred to be equal to 0 (no cropping applied).

seq_cropping_win_left_offset is the amount to crop off the left of the frame.

It is a requirement of bitstream conformance that seq_cropping_win_left_offset is less than or equal to max_frame_width_minus_1.

seq_cropping_win_right_offset is the amount to crop off the right of the frame.

It is a requirement of bitstream conformance that seq_cropping_win_right_offset is less than or equal to max_frame_width_minus_1.

seq_cropping_win_top_offset is the amount to crop off the top of the frame.

It is a requirement of bitstream conformance that seq_cropping_win_top_offset is less than or equal to max_frame_height_minus_1.

seq_cropping_win_bottom_offset is the amount to crop off the bottom of the frame.

It is a requirement of bitstream conformance that seq_cropping_win_bottom_offset is less than or equal to max_frame_height_minus_1.

NOTE: The amounts are expressed in terms of pixels to crop for a frame of maximum size. Smaller frames will have proportionately fewer pixels cropped.

seq_initial_display_delay_present_flag equal to 1 specifies that the syntax element `seq_initial_display_delay_minus_1` is present to indicate the initial display delay for the xlayer or sequence that uses this sequence header. **seq_initial_display_delay_present_flag** equal to 0 specifies that `seq_initial_display_delay_minus_1` is not present and is inferred to be equal to `NumRefFrames + 1`.

seq_initial_display_delay_minus_1 plus 1 specifies the initial display delay for use in the decoder model when the video sequence or xlayer is to be decoded. When `seq_initial_display_delay_minus_1` is not present in the bitstream, it is inferred to be equal to `NumRefFrames + 1`.

decoder_model_info_present_flag equal to 1 specifies that decoder model information is present in the coded video sequence and the `decoder_model_info()` syntax structure shall be parsed to specify decoder buffering model parameters. **decoder_model_info_present_flag** equal to 0 specifies that decoder model information is not present and decoder buffering model parameters are not specified in the bitstream.

num_units_in_decoding_tick is the number of time units of a decoding clock operating at the frequency `time_scale` Hz that corresponds to one increment of a clock tick counter:

```
DecCT = num_units_in_decoding_tick ÷ time_scale
```

NOTE: The `÷` operator represents standard mathematical division (in contrast to the `/` operator which represents integer division).

`num_units_in_decoding_tick` shall be greater than 0. `DecCT` represents the expected time to decode a single frame or a common divisor of the expected times to decode frames of different sizes and dimensions present in the coded video sequence.

seq_decoder_model_info_present_flag equal to 1 specifies that the `seq_decoder_model_info()` syntax structure is present and contains decoder model parameters for the xlayer or sequence that uses this sequence header. **seq_decoder_model_info_present_flag** equal to 0 specifies that the `seq_decoder_model_info()` syntax structure is not present.

An operating point specifies which extended layers, embedded layers, and temporal layers should be decoded. Operating points are defined within Operating Point Set (OPS) OBUs (see [§5.10 Operating point set OBU syntax](#)).

For AV2, operating points are specified using:

- **ops_xlayer_map** (for global operating point sets): A 31-bit bitmask indicating which extended layers are included
- **ops_mlayer_map**: An 8-bit bitmask indicating which embedded layers are included for a given extended layer
- **ops_tlayer_map**: A 4-bit bitmask indicating which temporal layers are included for a given embedded layer

See [Annex F: Sub-bitstream extraction \(informative\)](#) for details on operating point selection and sub-bitstream extraction.

NOTE: Operating points are optional. A decoder may choose to decode the entire bitstream without selecting a specific operating point.

Operating point selection is an optional decoder capability. When an Operating Point Set (OPS) OBU is present, a decoder may:

1. Decode the entire bitstream without selecting a specific operating point
2. Select an operating point from a [global operating point set](#) (for [multistream](#) bitstreams)
3. Select an operating point from a [local operating point set](#) (for specific extended layers)

The selection process depends on:

- Decoder capabilities (profile, level, tier support)
- Application requirements (resolution, frame rate, bitrate constraints)
- Available operating points in the OPS OBU(s)

When an operating point is selected, the decoder should perform the [sub-bitstream extraction process](#) to obtain a sub-bitstream containing only the OBUs associated with that operating point. See [Annex F: Sub-bitstream extraction \(informative\)](#) for the extraction process.

NOTE: To help with conformance testing, decoders may allow the operating point to be explicitly signaled by external means.

NOTE: A decoder may need to change the operating point selection when a new coded video sequence begins or when different extended layers are encountered in a [multistream](#) bitstream.

It is a requirement of bitstream conformance that the display order hints computation for any frame (i.e., the value returned from `get_disp_order_hint`) is the same for all the operating points within the bitstream associated with this frame.

It is a requirement of bitstream conformance that if `explicit_ref_frame_map` is equal to 0 for a frame, the implicit reference mapping process results in the same reference mapping (i.e., they result in exactly the same reference frames to be associated with exactly the same reference indices) for all the operating points within the bitstream associated with the current frame.

NOTE: This means that the corresponding calls to the get ref frames process specified in [§7.7 Get ref frames process](#) result in exactly the same contents being written to the `ref_frame_idx` array, and that the corresponding reference frames are the same.

It is a requirement of bitstream conformance that if `explicit_ref_frame_map` is equal to 1 for a frame, any reference buffer index associated with a particular reference frame, indicated by the explicit reference mapping process, corresponds to the same frame for all operating points within the bitstream associated with the current frame.

NOTE: These requirements ensure that the references used by a frame are the same for all the operating points that are associated with the current frame.

mlayer_dependency_present_flag specifies whether mlayer_dependency_map syntax elements are present in the bitstream.

mlayer_dependency_map specifies the embedded layer dependencies.

If obu_type is equal to either OBU_SWITCH or OBU_RAS_FRAME, it is a requirement of bitstream conformance that, for any embedded layer ID m not equal to obu_mlayer_id, MLayerDependencyMap[obu_mlayer_id][m] shall be equal to 0.

tlayer_dependency_present_flag specifies whether tlayer_dependency_map syntax elements are present in the bitstream.

multi_tlayer_dependency_map_present_flag equal to 1 specifies that tlayer_dependency_map values are signaled for all embedded layers. multi_tlayer_dependency_map_present_flag equal to 0 specifies that tlayer_dependency_map is only signaled for embedded layer 0, and the same values are used for all embedded layers.

tlayer_dependency_map specifies the temporal layer dependencies.

film_grain_params_present equal to 1 specifies that film grain parameters are present in the coded video sequence and can be signaled in the [frame_header_info](#) to apply film grain synthesis.

film_grain_params_present equal to 0 specifies that film grain parameters are not present and film grain synthesis is disabled for the entire coded video sequence.

NOTE: Although some film grain parameters (such as apply_grain) are present when film_grain_params_present is equal to 1, this does not imply that OBUs with obu_type equal to OBU_FILM_GRAIN are definitely present.

save_sequence_header is a function call that indicates that all the syntax elements and variables read in sequence_header_obu are stored in an area of memory indexed by seq_header_id.

§ 6.4.2. Sequence tile config semantics

seq_tile_info_present_flag equal to 1 specifies that tile parameters are present in the coded video sequence and the tile_params() syntax structure shall be parsed to determine tile configuration at the sequence level. seq_tile_info_present_flag equal to 0 specifies that tile parameters are not present at the sequence level and can be signaled at the frame level when allow_tile_info_change is enabled, or default to a single tile covering the entire frame.

allow_tile_info_change equal to 1 specifies that tile configuration can be overridden on a per-frame basis in the [frame_header_info](#). allow_tile_info_change equal to 0 specifies that tile configuration cannot be changed in the [frame_header_info](#) and the sequence-level tile configuration applies to all frames.

§ 6.4.3. Sequence partition config semantics

use_256x256_superblock, when equal to 1, indicates that superblocks in inter frames contain 256x256 luma samples. When equal to 0, it indicates that use_128x128_superblock is read to determine the superblock size.

use_128x128_superblock, when equal to 1, indicates that superblocks contain 128x128 luma samples. When equal to 0, it indicates that superblocks contain 64x64 luma samples. (The number of contained chroma samples depends on SubsamplingX and SubsamplingY.)

enable_sdp equal to 1 specifies that [SDP](#) is enabled and chroma components can use different partitioning structures than the luma component within the coded video sequence. **enable_sdp** equal to 0 specifies that [SDP](#) is disabled and chroma components use the same partitioning structure as the luma component.

NOTE: When Monochrome is equal to 1, **enable_sdp** is inferred to be equal to 0. When enabled, [SDP](#) is triggered when TreeType is equal to SHARED_PART, block size is BLOCK_64X64, and FrameIsIntra is equal to 1.

enable_extended_sdp equal to 1 specifies that extended [SDP](#) is enabled and chroma components can use different partitioning structures than luma within inter-coded frames. **enable_extended_sdp** equal to 0 specifies that extended [SDP](#) is disabled for inter frames.

NOTE: **enable_extended_sdp** is only signaled when **enable_sdp** is equal to 1 and **single_picture_header_flag** is equal to 0. Otherwise, it is inferred to be equal to 0.

enable_ext_partitions equal to 1 specifies that an extended range of partition types beyond the basic set is allowed in the coded video sequence. **enable_ext_partitions** equal to 0 specifies that only the basic set of partition types is allowed.

NOTE: The actual usage of extended partitions (via `is_ext_partition_allowed()`) requires TreeType not equal to CHROMA_PART, or specific block size constraints for CHROMA_PART blocks.

enable_uneven_4way_partitions equal to 1 specifies that uneven four-way partitions are allowed in the coded video sequence. **enable_uneven_4way_partitions** equal to 0 specifies that uneven four-way partitions are not allowed.

NOTE: **enable_uneven_4way_partitions** is only signaled when **enable_ext_partitions** is equal to 1. Otherwise, it is inferred to be equal to 0.

reduce_pb_aspect_ratio equal to 1 specifies that a reduced aspect ratio of blocks is used in the coded video sequence. **reduce_pb_aspect_ratio** equal to 0 specifies that the full range of block aspect ratios is allowed.

max_pb_aspect_ratio_log2_minus_1 plus 1 specifies the base 2 logarithm of the maximum aspect ratio of blocks in the coded video sequence.

§ 6.4.4. Sequence segment config semantics

enable_ext_seg enables extra segment ids. **enable_ext_seg** equal to 0 specifies there are 8 segments available. **enable_ext_seg** equal to 1 specifies there are 16 segments available.

seq_seg_info_present_flag equal to 1 specifies that segment information is present in this sequence header and the `seg_info()` syntax structure shall be parsed to define sequence-level segmentation

parameters. `seq_seg_info_present_flag` equal to 0 specifies that segment information is not present at the sequence level and can be signaled at the frame level when `seq_allow_seg_info_change` is enabled.

`seq_allow_seg_info_change` equal to 1 specifies that segment information can be overridden on a per-frame basis in the [frame_header_info](#). `seq_allow_seg_info_change` equal to 0 specifies that segment information cannot be changed in the [frame_header_info](#) and the sequence-level segmentation parameters apply to all frames.

§ 6.4.5. Sequence intra config semantics

`enable_dip` equal to 1 specifies that the `use_dip` syntax element can be present. `enable_dip` equal to 0 specifies that the `use_dip` syntax element is not present.

`enable_intra_edge_filter` equal to 1 specifies that the intra edge filtering process is enabled for intra prediction reference samples in the coded video sequence. `enable_intra_edge_filter` equal to 0 specifies that intra edge filtering is disabled and shall not be applied.

`enable_mrls` equal to 1 specifies that multiple reference line selection (MRLS) for intra prediction is allowed in the coded video sequence. `enable_mrls` equal to 0 specifies that MRLS is not allowed and only the first reference line is used for intra prediction.

NOTE: When `enable_mrls` is equal to 1, MRLS is only used for directional intra prediction modes.

`enable_cfl_intra` equal to 1 specifies that chroma from luma (CfL) intra prediction is allowed in the coded video sequence. `enable_cfl_intra` equal to 0 specifies that CfL intra prediction is not allowed.

NOTE: When `enable_cfl_intra` is equal to 1, CfL prediction is subject to additional conditions including block size constraints, tree type restrictions, and lossless mode considerations as specified in the `cflAllowed` derivation.

`cfl_ds_filter_index` specifies the type of down-sampling applied to luma samples in CFL prediction process. It is also used to specify the type of down-sampling applied to luma samples in loop restoration filtering process.

NOTE: A value of 3 can be read for `cfl_ds_filter_index`, but behaves the same as a value of 0.

`enable_mhccp` equal to 1 specifies that [MHCCP](#) is allowed in the coded video sequence. `enable_mhccp` equal to 0 specifies that [MHCCP](#) is not allowed.

NOTE: When `enable_mhccp` is equal to 1, [MHCCP](#) is subject to additional conditions including block size constraints, tree type restrictions, and lossless mode considerations as specified in the `is_mhccp_allowed()` function.

`enable_ibp` equal to 1 specifies that [IBP](#) is enabled in the coded video sequence. `enable_ibp` equal to 0 specifies that [IBP](#) is disabled.

§ 6.4.6. Sequence inter config semantics

`seq_enabled_motion_modes` specifies which motion modes are enabled.

seq_frame_motion_modes_present_flag equal to 1 specifies that the `frame_enabled_motion_modes` syntax element can be present in the [frame_header_info](#) to override motion mode settings on a per-frame basis. `seq_frame_motion_modes_present_flag` equal to 0 specifies that `frame_enabled_motion_modes` is not present in frame headers and the sequence-level `seq_enabled_motion_modes` values apply to all frames.

enable_six_param_warp_delta equal to 1 specifies that six or four parameters are used for warp delta. `enable_six_param_warp_delta` equal to 0 specifies that four parameters are used for warp delta.

enable_masked_compound equal to 1 specifies that the [mode info](#) for inter blocks can contain the syntax element `compound_type`. `enable_masked_compound` equal to 0 specifies that the syntax element `compound_type` will not be present.

enable_ref_frame_mvs equal to 1 indicates that the `use_ref_frame_mvs` syntax element can be present. `enable_ref_frame_mvs` equal to 0 indicates that the `use_ref_frame_mvs` syntax element will not be present.

reduced_ref_frame_mvs_mode equal to 1 indicates that motion fields from at most one reference frame will be processed.

order_hint_bits_minus_1 is used to compute `OrderHintBits`.

OrderHintBits specifies the number of bits used for the `order_hint` syntax element.

enable_refmvbank equal to 1 specifies that banks of recently used motion vectors are used during motion vector prediction.

disable_drl_reorder and **constrain_drl_reorder** are used to set the value for `DrlReorder`:

Table 6.4: DrlReorder values and names

DrlReorder	Name of DrlReorder
0	DRL_REORDER_DISABLED
1	DRL_REORDER_CONSTRAINT
2	DRL_REORDER_ALWAYS

explicit_ref_frame_map equal to 1 specifies that the `ref_frame_idx` syntax elements will be present in the [frame_header_info](#).

explicit_num_ref_frames equal to 1 specifies that the `num_ref_frames_minus_1` syntax element is present. Otherwise, `num_ref_frames_minus_1` is not present and `NumRefFrames` is inferred equal to 8.

num_ref_frames_minus_1 plus 1 specifies the number of reference frame slots in the coded video sequence.

long_term_frame_id_bits specifies the number of bits used to specify long term ids.

It is a requirement of bitstream conformance that if `long_term_frame_id_bits` is equal to 0, no OBU with `obu_type` equal to `OBU_RAS_FRAME` shall be present in the coded video sequence.

seq_max_drl_bits_minus_1 controls the number of bits read for `drl_idx` for inter blocks.

allow_frame_max_drl_bits equal to 1 indicates that `change_drl` is present in the [frame_header_info](#).

seq_max_bvp_drl_bits_minus_1 controls the number of bits read for `drl_idx` for intra block copy.

allow_frame_max_bvp_drl_bits equal to 1 indicates that `change_bvp_drl` is present in the [frame_header_info](#).

num_same_ref_compound specifies the number of references that can be used for same reference compound prediction. This refers to a case when a block uses compound inter prediction, but both references are to the same reference frame.

enable_tip equal to 1 specifies that [TIP](#) is enabled in the coded video sequence. `enable_tip` equal to 0 specifies that [TIP](#) is disabled.

NOTE: When `enable_tip` is equal to 1, several TIP-related syntax elements and features become available: `disable_tip_output` and `EnableTipOutput` are determined, `enable_tip_refinemv` can be signaled (when `enable_opfl_refine` != 0 or `enable_refinemv` is 1), and TIP reference frame usage requires additional conditions including `use_ref_frame_mvs` equal to 1, `NumTotalRefs` >= 2, and `bru_inactive` equal to 0.

disable_tip_output equal to 1 prevents `TipFrameMode` from being set to `TIP_FRAME_AS_OUTPUT` in the coded video sequence.

enable_tip_hole_fill equal to 1 specifies that holes in the interpolated motion field are filled in with estimated motion vectors. `enable_tip_hole_fill` equal to 0 specifies that holes in the interpolated motion field are not filled.

enable_mv_traj equal to 1 specifies that motion vector trajectory analysis is enabled. `enable_mv_traj` equal to 0 specifies that motion vector trajectory analysis is disabled.

enable_bawp equal to 1 specifies that the `allow_bawp` syntax element can be present in frame headers for inter frames, and `morph_pred` can be used for intra frames when `allow_screen_content_tools` is enabled. Otherwise, `allow_bawp` is not present in frame headers, `morph_pred` is not used, and both are inferred to be equal to 0.

NOTE: The `allow_bawp` syntax element is only present when `FrameIsIntra` is equal to 0 (inter frames). For intra frames, `morph_pred` is only signaled when `FrameIsIntra` is equal to 1 and `allow_screen_content_tools` is equal to 1.

enable_cwp equal to 1 specifies that compound weighted prediction is enabled in the coded video sequence. `enable_cwp` equal to 0 specifies that compound weighted prediction is disabled.

enable_imp_msk_bld equal to 1 specifies that implicit mask blending is enabled in the coded video sequence. `enable_imp_msk_bld` equal to 0 specifies that implicit mask blending is disabled.

enable_df_sub_pu equal to 1 specifies that the `allow_df_sub_pu` syntax element is present in frame headers. `enable_df_sub_pu` equal to 0 specifies that the `allow_df_sub_pu` syntax element is not present in frame headers (and `allow_df_sub_pu` will be inferred to be equal to 0).

enable_tip_explicit_qp equal to 1 specifies that the quantization parameters for [TIP](#) are sent explicitly. `enable_tip_explicit_qp` equal to 0 specifies that the quantization parameters are inferred.

enable_opfl_refine specifies how optical flow is signaled:

Table 6.5: Optical flow signaling modes

enable_opfl_refine	Name of enable_opfl_refine
0	REFINE_NONE
1	REFINE_SWITCHABLE
2	REFINE_ALL
3	REFINE_AUTO

NOTE: REFINE_NONE means optical flow is not used in the coded video sequence. REFINE_SWITCHABLE means the syntax element use_optflow is present to signal the use per block. REFINE_ALL means that optical flow will be used where allowed without being signaled. REFINE_AUTO means that the [frame_header_info](#) contains the syntax element opfl_refine_type that allows the method to be varied per frame.

enable_refinemv equal to 1 specifies that motion vector refinement is enabled in the coded video sequence. enable_refinemv equal to 0 specifies that motion vector refinement is disabled.

enable_tip_refinemv equal to 1 specifies that motion vector refinement and optical flow can be used with [TIP](#) prediction in the coded video sequence. enable_tip_refinemv equal to 0 specifies that motion vector refinement and optical flow are not allowed with [TIP](#) prediction.

enable_bru equal to 1 specifies that the use_bru syntax element is present for inter frames in frame headers and backwards reference update is enabled. enable_bru equal to 0 specifies that use_bru is not present and backwards reference update is disabled.

enable_adaptive_mvd equal to 1 specifies that adaptive motion vector differences are enabled in the coded video sequence. enable_adaptive_mvd equal to 0 specifies that adaptive motion vector differences are not allowed.

enable_mvd_sign_derive equal to 1 specifies that the motion vector sign can be derived instead of being explicitly signaled in the coded video sequence. enable_mvd_sign_derive equal to 0 specifies that motion vector signs are explicitly signaled.

enable_flex_mvres equal to 1 specifies that the motion vector precision can be specified per block in the coded video sequence. enable_flex_mvres equal to 0 specifies that a fixed motion vector precision is used for all blocks.

enable_global_motion equal to 1 specifies that global motion is enabled in the coded video sequence. enable_global_motion equal to 0 specifies that global motion is disabled.

enable_short_refresh_frame_flags equal to 1 specifies that a compact refresh frame signaling mode is used where the has_refresh_frame_flags and frame_to_refresh syntax elements can be present to indicate a single reference frame slot to refresh. enable_short_refresh_frame_flags equal to 0 specifies that the full refresh_frame_flags bitmask is used to indicate which reference frame slots are refreshed.

§ 6.4.7. Sequence screen content config semantics

seq_choose_screen_content_tools equal to 0 indicates that the `seq_force_screen_content_tools` syntax element will be present. `seq_choose_screen_content_tools` equal to 1 indicates that `seq_force_screen_content_tools` is set to `SELECT_SCREEN_CONTENT_TOOLS`.

seq_force_screen_content_tools equal to `SELECT_SCREEN_CONTENT_TOOLS` indicates that the `allow_screen_content_tools` syntax element will be present in the [frame_header_info](#). Otherwise, `seq_force_screen_content_tools` contains the value for `allow_screen_content_tools`.

seq_choose_integer_mv equal to 0 indicates that the `seq_force_integer_mv` syntax element will be present. `seq_choose_integer_mv` equal to 1 indicates that `seq_force_integer_mv` is set to `SELECT_INTEGER_MV`.

seq_force_integer_mv equal to `SELECT_INTEGER_MV` indicates that the `force_integer_mv` syntax element will be present in the [frame_header_info](#) (providing `allow_screen_content_tools` is equal to 1). Otherwise, `seq_force_integer_mv` contains the value for `force_integer_mv`.

§ 6.4.8. Sequence transform quant entropy config semantics

enable_fsc equal to 1 specifies that forward skip coding (FSC) is enabled in the coded video sequence. `enable_fsc` equal to 0 specifies that FSC is disabled.

enable_idtx_intra equal to 1 specifies that the identity transform is allowed for intra blocks when `enable_fsc` is equal to 0. `enable_idtx_intra` equal to 0 specifies that the identity transform is not allowed for intra blocks when `enable_fsc` is equal to 0. When `enable_fsc` is equal to 1, `enable_idtx_intra` is inferred to be equal to 1.

NOTE: The actual usage of identity transform for intra blocks (via `allow_fsc_intra()`) is also subject to block size constraints where block width and height must be less than or equal to `FSC_MAX`.

enable_intra_ist equal to 1 specifies that the intra-inter secondary transform (IST) is allowed for intra blocks in the coded video sequence. `enable_intra_ist` equal to 0 specifies that IST is not allowed for intra blocks.

enable_inter_ist equal to 1 specifies that the intra-inter secondary transform (IST) is allowed for inter blocks in the coded video sequence. `enable_inter_ist` equal to 0 specifies that IST is not allowed for inter blocks.

enable_chroma_dctonly equal to 1 specifies that the chroma transform is forced to be only [DCT](#). `enable_chroma_dctonly` equal to 0 specifies that other transform types are allowed for chroma.

enable_inter_ddt equal to 1 specifies that [DDT](#) is allowed for inter blocks in the coded video sequence. `enable_inter_ddt` equal to 0 specifies that [DDT](#) is not allowed for inter blocks.

reduced_tx_part_set equal to 1 specifies that a reduced set of transform partitions is allowed in the coded video sequence. `reduced_tx_part_set` equal to 0 specifies that the full set of transform partitions is allowed.

enable_cctx equal to 1 specifies that [CCTX](#) is allowed in the coded video sequence. `enable_cctx` equal to 0 specifies that [CCTX](#) is not allowed.

enable_tcq equal to 1 specifies that [TCQ](#) is allowed in the coded video sequence. **enable_tcq** equal to 0 specifies that [TCQ](#) is not allowed in the coded video sequence.

choose_tcq_per_frame equal to 1 specifies that **allow_tcq** is specified in each frame header. **choose_tcq_per_frame** equal to 0 specifies that **allow_tcq** is inferred to be equal to **enable_tcq**.

enable_parity_hiding equal to 1 specifies that the **allow_parity_hiding** syntax elements are present in the coded video sequence and [Parity hiding](#) can be enabled. **enable_parity_hiding** equal to 0 specifies that **allow_parity_hiding** syntax elements are not present and [Parity hiding](#) is disabled.

NOTE: **enable_parity_hiding** is inferred to be equal to 0 when **enable_tcq** is equal to 1 and **choose_tcq_per_frame** is equal to 0. Additionally, **allow_parity_hiding** is set to 0 when **CodedLossless** is equal to 1 or **allow_tcq** is equal to 1.

enable_avg_cdf equal to 1 specifies that the CDFs will be based on an average across CDFs.

avg_cdf_type equal to 1 specifies that the CDFs will be averaged across tiles. **avg_cdf_type** equal to 0 specifies that the CDFs can be blended between the CDFs saved for different reference frames.

separate_uv_delta_q equal to 1 indicates that the U and V planes may have separate delta quantizer values. **separate_uv_delta_q** equal to 0 indicates that the U and V planes will share the same delta quantizer value.

equal_ac_dc_q specifies that the DC quantizers match the AC quantizers.

base_y_dc_delta_q specifies a quantizer offset for the DC coefficients in the Y plane.

base_uv_dc_delta_q specifies a quantizer offset for the DC coefficients in the U and V planes.

base_uv_ac_delta_q specifies a quantizer offset for the AC coefficients in the U and V planes.

y_dc_delta_q_enabled specifies that the [frame_header_info](#) has a quantizer offset for DC coefficients in the Y plane.

uv_dc_delta_q_enabled specifies that the [frame_header_info](#) has a quantizer offset for DC coefficients in the U and V planes.

uv_ac_delta_q_enabled specifies that the [frame_header_info](#) has a quantizer offset for AC coefficients in the U and V planes.

§ 6.4.9. Segment information semantics

feature_enabled equal to 0 indicates that the corresponding feature is unused and has value equal to 0. **feature_enabled** equal to 1 indicates that the feature value is coded.

feature_value specifies the feature data for a segment feature.

§ 6.4.10. Sequence filter config semantics

disable_loopfilters_across_tiles equal to 1 specifies that the loop filters do not access samples from a different tile.

enable_cdef equal to 1 specifies that cdef filtering can be enabled. **enable_cdef** equal to 0 specifies that cdef filtering is disabled.

NOTE: It is allowed to set **enable_cdef** equal to 1 even when cdef filtering is not used on any frame in the coded video sequence. CDEF filtering is automatically disabled when CodedLossless is equal to 1.

enable_gdf equal to 1 specifies that [GDF](#) filtering can be enabled. **enable_gdf** equal to 0 specifies that [GDF](#) filtering is disabled.

NOTE: GDF filtering is automatically disabled when CodedLossless is equal to 1.

gdf_unit_matches_sb_size equal to 1 specifies that the [GDF](#) size is taken from the superblock size. **gdf_unit_matches_sb_size** equal to 0 specifies that the [GDF](#) size is computed based on tile alignment.

enable_restoration equal to 1 specifies that loop restoration filtering can be enabled. **enable_restoration** equal to 0 specifies that loop restoration filtering is disabled.

NOTE: It is allowed to set **enable_restoration** equal to 1 even when loop restoration is not used on any frame in the coded video sequence.

lr_tools_disable[isChroma][i] equal to 1 specifies that loop restoration tool i is disabled.

lr_tools_disable[isChroma][i] equal to 0 specifies that loop restoration tool i is not disabled. **isChroma** equal to 0 selects luma; **isChroma** equal to 1 selects chroma.

lr_tools_uv_present equal to 1 specifies that the chroma **lr_tools_disable** syntax elements are present in the coded video sequence. **lr_tools_uv_present** equal to 0 specifies that the chroma **lr_tools_disable** syntax elements are not present.

NOTE: It is allowed to set **lr_tools_uv_present** equal to 1 even if the stream does not contain chroma.

enable_ccso equal to 1 specifies that [CCSO](#) filtering can be enabled. **enable_ccso** equal to 0 specifies that [CCSO](#) filtering is disabled.

ccso_unit_matches_sb_size equal to 1 specifies that the [CCSO](#) size is taken from the superblock size. **ccso_unit_matches_sb_size** equal to 0 specifies that the [CCSO](#) size is computed based on tile alignment.

cdef_on_skip_txfm_always_on equal to 1 specifies that CDEF will always be on for skipped transform blocks.

cdef_on_skip_txfm_disabled equal to 1 specifies that CDEF will always be off for skipped transform blocks. **cdef_on_skip_txfm_disabled** equal to 0 specifies that a frame level enable is used to specify how CDEF is applied for skipped transform blocks.

df_par_bits_minus_2 plus 2 specifies the number of bits used to read the **df_delta_q[i]** syntax element.

§ 6.4.11. User defined QM semantics

qm_copy_from_previous_plane equal to 1 specifies that the quantization matrices are copied from the previous plane.

qm_8x8_is_symmetric equal to 1 specifies that the quantization matrix for TX_8X8 is symmetric (so certain entries can be inferred instead of being present in the bitstream).

qm_4x8_is_transpose_of_8x4 equal to 1 specifies that the quantization matrix for TX_4X8 is equal to the transpose of the matrix for TX_8X4.

quant_delta specifies the adjustment between quantizer values.

It is a requirement of bitstream conformance that **quant_delta** is greater than or equal to -128, and less than or equal to 127.

It is a requirement of bitstream conformance that no value written into UserQm is equal to 0.

§ 6.4.12. Timing info semantics

num_units_in_display_tick is the number of time units of a clock operating at the frequency **time_scale** Hz that corresponds to one increment of a clock tick counter. A display clock tick, in seconds, is equal to **num_units_in_display_tick** divided by **time_scale**:

```
DispCT = num_units_in_display_tick ÷ time_scale
```

NOTE: The $\div$ operator represents standard mathematical division (in contrast to the $/$ operator which represents integer division).

It is a requirement of bitstream conformance that **num_units_in_display_tick** is greater than 0.

It is a requirement of bitstream conformance that within a coded video sequence, **num_units_in_display_tick**, when present, has the same value across all embedded layers.

time_scale is the number of time units that pass in one second.

It is a requirement of bitstream conformance that **time_scale** is greater than 0.

It is a requirement of bitstream conformance that within a coded video sequence, **time_scale**, when present, has the same value across all embedded layers.

equal_picture_interval equal to 1 indicates that pictures should be displayed according to their output order with the number of ticks between two consecutive pictures (without dropping frames) specified by **num_ticks_per_picture_minus_1** + 1. **equal_picture_interval** equal to 0 indicates that the interval between two consecutive pictures is not specified.

It is a requirement of bitstream conformance that within a coded video sequence, **equal_picture_interval**, when present, has the same value across all embedded layers.

num_ticks_per_picture_minus_1 plus 1 specifies the number of clock ticks corresponding to output time between two consecutive pictures in the output order.

It is a requirement of bitstream conformance that the value of `num_ticks_per_picture_minus_1` shall be in the range of 0 to $(1 \ll 32) - 2$, inclusive.

It is a requirement of bitstream conformance that within a coded video sequence, `num_ticks_per_picture_minus_1`, when present, has the same value across all embedded layers.

NOTE: The frame rate, when specified explicitly, applies to the top temporal layer of the bitstream. If bitstream is expected to be manipulated, e.g., by intermediate network elements, then the resulting frame rate may not match the specified one. In this case, an encoder is advised to use explicit time codes or some mechanisms that convey picture timing information outside the bitstream.

§ 6.4.13. Sequence decoder model info semantics

decoder_buffer_delay specifies the time interval between the arrival of the first bit in the smoothing buffer and the subsequent removal of the data that belongs to the first [coded frame](#), measured in units of 1/90000 seconds.

encoder_buffer_delay specifies, in combination with `decoder_buffer_delay` syntax element, the first bit arrival time of frames to be decoded to the smoothing buffer. `encoder_buffer_delay` is measured in units of 1/90000 seconds.

For a video sequence that includes one or more random access points the sum of `decoder_buffer_delay` and `encoder_buffer_delay` shall be kept constant.

low_delay_mode_flag equal to 1 indicates that the smoothing buffer operates in low-delay mode. In low-delay mode late decode times and buffer underflow are both permitted. `low_delay_mode_flag` equal to 0 indicates that the smoothing buffer operates in strict mode, where buffer underflow is not allowed.

The parameters `decoder_buffer_delay`, `encoder_buffer_delay`, and `low_delay_mode_flag` are applied to the xlayer or sub-bitstream that uses the sequence header containing these parameters.

§ 6.5. Temporal delimiter OBU semantics

SeenFrameHeader is a variable used to mark whether the [frame_header_info](#) for the current frame has been received. It is initialized to zero.

§ 6.6. Multi Stream Decoder Operation OBU semantics

It is a requirement of bitstream conformance that a Multi Stream Decoder Operation OBU has:

1. `obu_tlayer_id` equal to 0.
2. `obu_mlayer_id` equal to 0.
3. `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`.

num_streams_minus_2 plus 2 specifies the number of independent streams in the bitstream. It is a requirement of bitstream conformance that `num_streams_minus_2` is not greater than 2.

multistream_profile_idc specifies the coding features that can be used in a [coded multistream video sequence](#). The allowed values for `multistream_profile_idc` are the same as those for `seq_profile_idc` as defined in Table A.4.

It is a requirement of bitstream conformance that `multistream_profile_idc` is greater than or equal to `sub_stream_max_profile[i]` for all `i` in the range 0 to `num_streams_minus_2 + 1`, inclusive.

`multistream_level_idx` specifies the level to which the [coded multistream video sequence](#) conforms.

`multistream_tier` specifies the tier to which the [coded multistream video sequence](#) conforms.

`multistream_even_allocation_flag` specifies the resource allocation for the [multistream](#).

`multistream_large_picture_idc` specifies an index of the `sub_xlayer_id` array that has a larger resource allocation than the other independent sub-bitstreams.

`sub_xlayer_id[ i ]` specifies the value of `obu_xlayer_id` in the OBU header for the `i`-th independent sub-bitstream in the present bitstream.

`sub_stream_max_profile[ i ]` indicates the maximum value for `seq_profile_idc` that may appear in a sequence header activated by the `i`-th independent sub-bitstream.

It is a requirement of bitstream conformance that `seq_profile_idc` is less than or equal to `sub_stream_max_profile[i]` for each sequence header activated by the `i`-th independent sub-stream.

`sub_stream_max_level[ i ]` indicates the maximum value for `seq_level_idx` that may appear in a sequence header activated by the `i`-th independent sub-bitstream.

It is a requirement of bitstream conformance that `seq_level_idx` is less than or equal to `sub_stream_max_level[i]` for each sequence header activated by the `i`-th independent sub-stream.

`sub_stream_max_tier[ i ]` indicates the maximum value for `seq_tier` that may appear in a sequence header activated by the `i`-th independent sub-bitstream.

It is a requirement of bitstream conformance that `seq_tier` is less than or equal to `sub_stream_max_tier[i]` for each sequence header activated by the `i`-th independent sub-stream.

NOTE: The values of `sub_stream_max_profile[i]`, `sub_stream_max_level[i]`, and `sub_stream_max_tier[i]` are not used in determining the profile and level constraints in Annex A. There is no constraint that there exists a value of `seq_profile_idc`, `seq_level_idx` or `seq_tier` equal to the indicated maximum.

`multistream_doh_constraint_flag` equal to 1 specifies that additional display order hint (DOH) constraints on the temporal unit are enabled. `multistream_doh_constraint_flag` equal to 0 specifies that additional DOH constraints on the temporal unit are not enabled.

It is a requirement of bitstream conformance that when `monotonic_output_order_flag` is equal to 0 in any activated sequence header of the [coded multistream video sequence](#), `multistream_doh_constraint_flag` shall be equal to 1.

NOTE: The constraints enabled by the `multistream_doh_constraint_flag` appear in [§ 7.3.7 Temporal unit](#)

§ 6.7. Multi frame header OBU semantics

mfh_seq_header_id specifies a sequence header id.

It is a requirement of bitstream conformance that **mfh_seq_header_id** is less than **MAX_SEQ_NUM**.

mfh_id_minus_1 plus 1 identifies the multi-frame header for reference by a frame header or a coded frame.

It is a requirement of bitstream conformance that **mfh_id_minus_1** + 1 is less than **MAX_MFH_NUM**.

mfh_frame_size_present_flag equal to 1 specifies that the syntax elements **mfh_frame_width_minus_1** and **mfh_frame_height_minus_1** are present in the multi-frame header to override the sequence-level frame size. **mfh_frame_size_present_flag** equal to 0 specifies that these syntax elements are not present and the frame size from the sequence header applies to frames using this multi-frame header.

mfh_frame_width_bits_minus_1 plus one specifies the number of bits used to read **mfh_frame_width_minus_1**.

mfh_frame_height_bits_minus_1 plus one specifies the number of bits used to read **mfh_frame_height_minus_1**.

mfh_frame_width_minus_1 plus one specifies the width of the frame that references the multi-frame header in luma samples.

mfh_frame_height_minus_1 plus one specifies the height of the frame that references the multi-frame header in luma samples.

mfh_deblocking_filter_update equal to 1 specifies that the syntax elements **mfh_apply_deblocking_filter** are present in the multi-frame header. **mfh_deblocking_filter_update** equal to 0 specifies that **mfh_apply_deblocking_filter** syntax elements are not present.

mfh_apply_deblocking_filter is an array containing flags that specify if the deblocking filter is applied for a particular plane and direction. Different **mfh_apply_deblocking_filter** values from the array are used by a frame header or a coded frame that references the multi-frame header, depending on the image plane being filtered, and the edge direction (vertical or horizontal) being filtered.

mfh_seg_info_present_flag equal to 1 specifies that segment information is present in this multi-frame header and the **seg_info()** syntax structure shall be parsed. **mfh_seg_info_present_flag** equal to 0 specifies that segment information is not present in this multi-frame header.

mfh_ext_seg_flag equal to 1 specifies that the segment information uses an extended number of 16 segments. **mfh_ext_seg_flag** equal to 0 specifies that the segment information uses the standard 8 segments.

mfh_allow_seg_info_change equal to 1 specifies that the segment information in this multi-frame header can be overridden in the [frame_header_info](#). **mfh_allow_seg_info_change** equal to 0 specifies that segment information cannot be changed in the [frame_header_info](#).

§ 6.8. Layer config record OBU semantics

This OBU contains either global information or local layer information depending on the value of **obu_xlayer_id**.

§ 6.8.1. General

The Layer Configuration Record (LCR) provides comprehensive metadata about the structure, properties, and relationships of layers within an AV2 bitstream. The LCR serves multiple critical purposes:

Multi-view and multi-layer organization: The LCR enables complex content scenarios where multiple independent layers represent different aspects or views of the same scene. Each embedded layer within an extended layer can be annotated with metadata that describes its role in the overall composition.

Layer type and purpose identification: Through the combination of `lcr_layer_type` and `lcr_auxiliary_type`, the LCR distinguishes between primary texture content and auxiliary data. Texture layers (`lcr_layer_type == TEXTURE_LAYER`) carry the main visual content, while auxiliary layers (`lcr_layer_type == AUX_LAYER`) provide supplementary information such as alpha channels (transparency), depth maps for 3D representation, segmentation masks, or gain maps for HDR tone mapping.

View association and multi-view content: The `lcr_view_type` and `lcr_view_id` fields enable sophisticated multi-view scenarios. For stereoscopic content, different layers can be marked as `VIEW_LEFT` or `VIEW_RIGHT`, or assigned explicit view IDs through `VIEW_EXPLICIT` combined with `lcr_view_id`. This allows a single bitstream to carry multiple perspectives of the same scene, where each view can have its own texture layer plus associated auxiliary layers (alpha, depth, etc.). For example, a stereoscopic stream might have:

- Layer 0: Left view texture (`lcr_view_id = 0`, `lcr_layer_type = TEXTURE_LAYER`)
- Layer 1: Left view depth (`lcr_view_id = 0`, `lcr_layer_type = AUX_LAYER`, `lcr_auxiliary_type = DEPTH_AUX`)
- Layer 2: Right view texture (`lcr_view_id = 1`, `lcr_layer_type = TEXTURE_LAYER`)
- Layer 3: Right view depth (`lcr_view_id = 1`, `lcr_layer_type = AUX_LAYER`, `lcr_auxiliary_type = DEPTH_AUX`)

Atlas integration: The `lcr_layer_atlas_segment_id` field associates each layer with a specific atlas segment, enabling spatial composition and layout specification. The atlas defines how different layers should be positioned, scaled, or composed to form the final rendered output. This association is particularly powerful for:

- Subpicture and region-of-interest applications where different regions are encoded as separate layers
- Virtual screen composition where the atlas represents a larger virtual canvas and layers are placed at specific positions
- Multistream composition where layers from different extended layers are combined spatially

Layer dependencies: The `lcr_dependent_layer_map` indicates inter-prediction dependencies between layers, allowing decoders to understand which layers can be decoded independently and which require other layers as references.

The LCR can be specified at two scopes: global (`obu_xlayer_id == 31`) for [multistream](#) scenarios, or local (`obu_xlayer_id` in 0..30) for individual extended layers. Global LCRs provide cross-layer metadata and relationships, while local LCRs describe the structure within a single extended layer sub-bitstream.

For detailed usage examples including stereoscopic video, multi-property layers, and subpicture composition, see [Annex G: Layer composition and Atlas usage examples \(informative\)](#).

§ 6.8.2. LCR global info semantics

lcr_global_config_record_id provides an identifier for the global LCR for reference by other syntax elements.

It is a requirement of bitstream conformance that `lcr_global_config_record_id` is in the range of 1 to 7, inclusive.

lcr_xlayer_map is a bitmap indicating the extended layer sub-bitstreams that are associated with this global LCR and can be present in a CVS that refers to this global LCR. It is a requirement of bitstream conformance that `lcr_xlayer_map` is in the range of 1 to $(1 < 31) - 1$, inclusive.

It is a requirement of bitstream conformance that all extended layers present in the multistream shall reference the same activated global LCR (i.e., the same value of `lcr_global_config_record_id`).

lcr_aggregate_info_present_flag equal to 1 specifies that the `lcr_aggregate_info()` syntax structure is present in the current LCR to indicate the aggregate information of all sub-bitstreams that can be present in the CVS associated with this global LCR. `lcr_aggregate_info_present_flag` equal to 0 specifies that this information is not present but may be derived by examining the profile, tier, and level indicators, in addition to the maximum number of embedded layers that are indicated for each individual extended layer that is associated with this LCR.

lcr_seq_profile_tier_level_info_present_flag equal to 1 specifies that the `lcr_seq_profile_tier_level_info( i )` syntax structure is present in the current LCR for an extended layer with index `i` to indicate the sequence profile, tier, level, and maximum number of embedded layers that can be present in the extended layer sub-bitstream with `obu_xlayer_id` equal to `i` that is associated with this global LCR. `lcr_seq_profile_tier_level_info_present_flag` equal to 0 specifies that this information is not present but may be derived through other means.

lcr_global_payload_present_flag equal to 1 specifies that the payload `lcr_global_payload( i )` is present in this syntax structure for each individual extended layer `i` associated with this LCR.

`lcr_global_payload_present_flag` equal to 0 specifies that `lcr_global_payload( i )` for each individual extended layer `i` associated with this LCR is not present.

lcr_dependent_xlayers_flag equal to 1 specifies that the syntax element `lcr_num_dependent_xlayer_map[ j ]` for any extended layer with ID equal to `j` is present in the current LCR. `lcr_dependent_xlayers_flag` equal to 0 specifies that the `lcr_num_dependent_xlayer_map[ j ]` syntax element is not present in the current global LCR.

It is a requirement of bitstream conformance that the value of `lcr_dependent_xlayers_flag` is equal to 0. Decoders conforming to this version of this specification shall ignore non-zero values of `lcr_dependent_xlayers_flag`.

lcr_global_atlas_id_present_flag equal to 1 specifies that the `lcr_global_atlas_id` syntax element is present in the current global LCR. `lcr_global_atlas_id_present_flag` equal to 0 specifies that the `lcr_global_atlas_id` syntax element is not present in the current global LCR.

lcr_global_purpose_id specifies the application purpose for the layered bitstream associated with this global LCR by referencing its `lcr_global_config_record_id`, as follows:

Table 6.6: LCR global purpose identifier values

lcr_global_purpose_id	Application Purpose
0	Unspecified
1	Stereoscopic Viewports
2	Immersive Multiple Viewports
3	Immersive Multiple Viewports + Alpha
4	Immersive Multiple Viewports + Depth
5	Immersive Multiple Viewports + Alpha + Depth
6	Multiview Playback
7	Subregion Playback
8-127	Reserved

lcr_doh_constraint_flag equal to 1 specifies that additional display order hint (DOH) constraints on the temporal unit are enabled. **lcr_doh_constraint_flag** equal to 0 specifies that additional DOH constraints on the temporal unit are not enabled.

It is a requirement of bitstream conformance that when **monotonic_output_order_flag** is equal to 0 in any activated sequence header of the [coded multistream video sequence](#), **lcr_doh_constraint_flag** shall be equal to 1.

NOTE: The constraints enabled by the **lcr_doh_constraint_flag** appear in [§ 7.3.7 Temporal unit](#)

lcr_enforce_tile_alignment_flag equal to 1 specifies that all extended layer sub-bitstreams associated with this global LCR shall use the same tile structure. When **lcr_enforce_tile_alignment_flag** is set equal to 1, it is a requirement of bitstream conformance that all extended layers use the same values of **TileCols**, **TileRows**, and the same tile column and row start positions. **lcr_enforce_tile_alignment_flag** equal to 0 specifies that the extended layer sub-bitstreams are not required to use the same tile structure.

lcr_global_atlas_id specifies the value of the **atlas_segment_id**[31] associated with the current global LCR. When **lcr_global_atlas_id_present_flag** is equal to 0, the value of **lcr_global_atlas_id** is inferred to be equal to 0.

lcr_global_reserved_zero_3bits shall be equal to 0 in bitstreams conforming to this specification. Other values for **lcr_global_reserved_zero_3bits** are reserved for future use by AOMedia. Decoders shall ignore the value of **lcr_global_reserved_zero_3bits**.

lcr_global_reserved_zero_5bits shall be equal to 0 in bitstreams conforming to this specification. Other values for **lcr_global_reserved_zero_5bits** are reserved for future use by AOMedia. Decoders shall ignore the value of **lcr_global_reserved_zero_5bits**.

When both an OBU with **obu_type** equal to **OBU_MSDO** and an activated global layer configuration record OBU are present in the same coded multistream video sequence, it is a requirement of bitstream conformance that the following constraints hold:

1. The value of **num_streams_minus_2 + 2** is equal to **LcrMaxNumXLayerCount**.
2. For each **i** in the range of 0 to **num_streams_minus_2 + 1**, inclusive, there exists a **j** in the range of 0 to **LcrMaxNumXLayerCount - 1**, inclusive, such that **sub_xlayer_id**[**i**] is equal to **LcrXLayerID**[**j**].

3. When `lcr_aggregate_info_present_flag` is equal to 1 in the activated global LCR:

- `multistream_profile_idc` shall be consistent with the multi-sequence configuration indicated by `lcr_config_idc`, as specified in [Annex A.3 Multi-sequence configurations](#).
- The interoperability point associated with `multistream_profile_idc`, as specified in [Annex A.2 Profiles](#), shall be equal to `lcr_max_interop`.
- `multistream_level_idx` shall be equal to `lcr_aggregate_level_idx`.
- `multistream_tier` shall be equal to `lcr_max_tier_flag`.

1. When `lcr_seq_profile_tier_level_info_present_flag` is equal to 1 in the activated global LCR, for each `i` in the range of 0 to `num_streams_minus_2 + 1`, inclusive:

- `sub_stream_max_profile[ i ]` shall be equal to `lcr_seq_profile_idc[ sub_xlayer_id[ i ] ]`.
- `sub_stream_max_level[ i ]` shall be equal to `lcr_max_level_idx[ sub_xlayer_id[ i ] ]`.
- `sub_stream_max_tier[ i ]` shall be equal to `lcr_tier_flag[ sub_xlayer_id[ i ] ]`.

1. `multistream_doh_constraint_flag` shall be equal to `lcr_doh_constraint_flag`.

NOTE: The above constraints ensure that when both an MSDO OBU and a global LCR are present in the same coded multistream video sequence, the common information signaled in both structures is aligned.

`lcr_data_size[ i ]` indicates the number of bytes present in an indicated `lcr_global_payload()` module that is associated with the extended layer sub-bitstream with `obu_xlayer_id` equal to `i`.

NOTE: A decoder can use `lcr_data_size[ i ]` to skip over the `lcr_global_payload()` for extended layers that are not required for decoding.

§ 6.8.3. LCR local info semantics

`lcr_global_id[ i ]` specifies the value of the `lcr_global_config_record_id` associated with the local LCR that is indicated in an extended layer with `obu_xlayer_id` equal to `i`.

If `lcr_global_id` is equal to 0, no global LCR is associated with this local LCR.

`lcr_local_id[ i ]` provides an identifier for the local LCR indicated in an extended layer with ID equal to `i` for reference by other syntax elements.

It is a requirement of bitstream conformance that `lcr_local_id[ i ]` is not equal to 0.

`lcr_profile_tier_level_info_present_flag[ i ]` equal to 1 specifies that the `lcr_seq_profile_tier_level_info( i )` syntax structure is present in the current LCR for the extended layer with index `i`, indicating the sequence profile, tier, level, and maximum number of embedded layers that can be present in the extended layer sub-bitstream with `obu_xlayer_id` equal to `i`.
`lcr_profile_tier_level_info_present_flag[ i ]` equal to 0 specifies that this information is not present but may be derived through other means.

`lcr_local_atlas_id_present_flag[ i ]` equal to 1 specifies that the syntax element `lcr_local_atlas_id[ i ]` is present in the local LCR in the extended layer with `obu_xlayer_id` equal to `i`.

`lcr_local_atlas_id_present_flag[ i ]` equal to 0 specifies that the `lcr_local_atlas_id[ i ]` syntax element is not present.

`lcr_local_atlas_id[ i ]` provides an identifier for a local atlas with `atlas_segment_id` equal to `lcr_local_atlas_id[ i ]` that is associated with the extended layer with `obu_xlayer_id` equal to `i`. If this value is not present this information can be provided by a global atlas, if present, or is considered as unspecified.

`lcr_local_reserved_zero_3bits[ i ]` shall be equal to 0 in bitstreams conforming to this specification. Other values for `lcr_local_reserved_zero_3bits[ i ]` are reserved for future use by AOMedia. Decoders shall ignore the value of `lcr_local_reserved_zero_3bits[ i ]`.

`lcr_local_reserved_zero_5bits[ i ]` shall be equal to 0 in bitstreams conforming to this specification. Other values for `lcr_local_reserved_zero_5bits[ i ]` are reserved for future use by AOMedia. Decoders shall ignore the value of `lcr_local_reserved_zero_5bits[ i ]`.

§ 6.8.4. LCR aggregate info semantics

`lcr_config_idc` indicates a configuration to which the associated bitstream that has activated this global LCR conforms to Annex A. Bitstreams conforming to this specification shall not contain values of `lcr_config_idc` outside those specified in Annex A. Other values of `lcr_config_idc` are reserved for future extensions of this specification by AOMedia.

`lcr_aggregate_level_idx` indicates an aggregate level indicator to which the combination of all sub-bitstreams associated with a bitstream that has activated this LCR conforms to Annex A. Bitstreams conforming to this specification shall not contain values of `lcr_aggregate_level_idx` outside those specified in Annex A.

`lcr_max_tier_flag` indicates the maximum tier indicator to which all sub-bitstreams associated with a bitstream that has activated this LCR conform to according to Annex A.

`lcr_max_interop` indicates the maximum interoperability point that the associated bitstream that has activated this LCR conforms to Annex A. Bitstreams conforming to this specification shall not contain values of `lcr_max_interop` outside those specified in Annex A.

§ 6.8.5. LCR sequence profile tier level information semantics

`lcr_seq_profile_idc[ i ]` specifies the value of the `seq_profile_idc` associated with the local LCR that is indicated in an extended layer with `obu_xlayer_id` equal to `i`. Bitstreams conforming to this specification shall not contain values of `lcr_seq_profile_idc[ i ]` outside those specified in Annex A.

It is a requirement of bitstream conformance that, when `lcr_seq_profile_tier_level_info( i )` is present in an activated LCR, `seq_profile_idc` is less than or equal to `lcr_seq_profile_idc[ i ]` for each sequence header activated by the extended layer sub-bitstream with `obu_xlayer_id` equal to `i`.

`lcr_max_level_idx[ i ]` specifies the maximum level associated with the local LCR that is indicated in an extended layer with `obu_xlayer_id` equal to `i`. Bitstreams conforming to this specification shall not contain values of `lcr_max_level_idx[ i ]` outside those specified in Annex A.

It is a requirement of bitstream conformance that, when `lcr_seq_profile_tier_level_info( i )` is present in an activated LCR, `seq_level_idx` is less than or equal to `lcr_max_level_idx[ i ]` for each sequence header activated by the extended layer sub-bitstream with `obu_xlayer_id` equal to `i`.

lcr_tier_flag[i] specifies the tier indicator associated with the local LCR that is indicated in an extended layer with obu_xlayer_id equal to i. Bitstreams conforming to this specification shall not contain values of lcr_tier_flag[i] outside those specified in Annex A.

It is a requirement of bitstream conformance that, when lcr_seq_profile_tier_level_info(i) is present in an activated LCR, seq_tier is less than or equal to lcr_tier_flag[i] for each sequence header activated by the extended layer sub-bitstream with obu_xlayer_id equal to i.

NOTE: The values of lcr_seq_profile_idc[i], lcr_max_level_idx[i], and lcr_tier_flag[i] are not used in determining the profile and level constraints in Annex A. There is no constraint that there exists a value of seq_profile_idc, seq_level_idx or seq_tier equal to the indicated maximum.

lcr_max_mlayer_count[i] specifies the maximum number of embedded layers that can be associated with the local LCR that is indicated in an extended layer with obu_xlayer_id equal to i. Bitstreams conforming to this specification shall not contain values of lcr_max_mlayer_count[i] outside those specified in Annex A.

It is a requirement of bitstream conformance that, when lcr_seq_profile_tier_level_info(i) is present in an activated LCR, seq_max_mlayer_cnt_minus_1 plus 1 is less than or equal to lcr_max_mlayer_count[i] for each sequence header activated by the extended layer sub-bitstream with obu_xlayer_id equal to i.

lsptli_reserved_2bits shall be equal to 0 in bitstreams conforming to this specification. Other values for lsptli_reserved_2bits are reserved for future use by AOMedia. Decoders shall ignore the value of lsptli_reserved_2bits.

§ 6.8.6. LCR global payload semantics

lcr_num_dependent_xlayer_map[j] indicates the extended layers on which the extended layer with ID j can depend on in terms of inter-layer prediction. An extended layer with ID j can only depend on layers with an ID smaller than j. When lcr_dependent_xlayers_flag is equal to 0, or when j is equal to 0, the value of lcr_num_dependent_xlayer_map[j] is inferred to be equal to 0.

lcr_remaining_payload_bit can take any value but is reserved for future use by AOMedia. Decoders conforming to this specification shall ignore the value of lcr_remaining_payload_bit.

It is a requirement of bitstream conformance that any computed values for RemainingLcrPayloadBits shall not be less than 0.

§ 6.8.7. LCR xlayer info semantics

lcr_rep_info_present_flag[i][j] indicates the presence of the global, if i is equal to 1, or local, if i is equal to 0, lcr_rep_info(i, j) syntax in the extended layer information for extended layer id j. If lcr_rep_info_present_flag[i][j] is equal to 1, the corresponding lcr_rep_info(i, j) syntax is present, otherwise, this syntax is not present.

lcr_xlayer_purpose_present_flag[i][j] indicates the presence of the lcr_xlayer_purpose_id[i][j] syntax element in the current LCR. If lcr_xlayer_purpose_present_flag[i][j] is equal to 1, then lcr_xlayer_purpose_id[i][j] is present. Otherwise, if lcr_xlayer_purpose_present_flag[i][j] is equal to 0, then lcr_xlayer_purpose_id[i][j] is not present.

lcr_xlayer_color_info_present_flag[i][j] indicates the presence of the global, if i is equal to 1, or local, if i is equal to 0, lcr_xlayer_color_info(i, j) syntax in the extended layer information for extended layer id j. If lcr_xlayer_color_info_present_flag[i][j] is equal to 1, the corresponding lcr_xlayer_color_info(i, j) syntax is present, otherwise, this syntax is not present.

lcr_embedded_layer_info_present_flag[i][j] indicates the presence of the global, if i is equal to 1, or local, if i is equal to 0, lcr_embedded_layer_info(i, j) syntax in the extended layer information for extended layer id j. If lcr_embedded_layer_info_present_flag[i][j] is equal to 1, the corresponding lcr_embedded_layer_info(i, j) syntax is present, otherwise, this syntax is not present.

lcr_xlayer_purpose_id[i][j] specifies the application purpose for the extended layer with id j, in a global, if i is equal to 1, or a local, if i is equal to 0, LCR with the same semantics as for lcr_global_purpose_id. When the syntax elements lcr_xlayer_purpose_id[i][j] and lcr_global_purpose_id are not present then lcr_xlayer_purpose_id[i][j] is set to 0 (Unspecified).

lcr_xlayer_atlas_segment_id[j] indicates the corresponding atlas segment ID that the extended layer with index j in the global LCR is associated with. If lcr_xlayer_atlas_segment_id[j] is not present, such association can be provided in the embedded layer information, can be specified through external means, or can be unspecified.

lcr_xlayer_priority_order[j] indicates the priority order of an extended layer with index j when rendering it on an atlas compared to other extended layers. The lower the value of lcr_xlayer_priority_order[j] the higher the priority rendering order of that layer compared to other layers with a higher value. If this information is missing or two or more layers have the same priority value, then the priority between them is determined based on the extended layer ID of the layers (the lower ID value has a higher rendering priority than a higher ID value). Layers with a higher rendering priority value are rendered first compared to layers with a lower rendering priority value when placed on an atlas.

lcr_xlayer_rendering_method[j] indicates the rendering method applied to the extended layer j compared to previously rendered layers according to their priority order value. The interpretation of the value of lcr_xlayer_rendering_method[j] for rendering purposes is shown below:

Table 6.7: Extended layer rendering methods

lcr_xlayer_rendering_method	Interpretation
0	Overwrite
1	Blend 50%
2	Multiply
3	Darken
4	Lighten
5-255	Reserved

Values corresponding to a reserved interpretation are for future use by AOMedia. They shall be ignored by decoders conforming to this version of this specification.

§ 6.8.8. LCR rep info semantics

lcr_max_pic_width[i][j] specifies the maximum picture width for the decoded pictures associated with the extended layer j in either a global, when i is equal to 1, or a local, when i is equal to 0, LCR OBU. The value of lcr_max_pic_width[i][j] in an activated LCR OBU in an extended layer with index j shall equal max_frame_width_minus_1 + 1.

lcr_max_pic_height[i][j] specifies the maximum picture height for the decoded pictures associated with the extended layer j in either a global, when i is equal to 1, or a local, when i is equal to 0, LCR OBU. The value of lcr_max_pic_height[i][j] in an activated LCR OBU in an extended layer with index j shall equal max_frame_height_minus_1 + 1.

lcr_format_info_present_flag[i][j] specifies the presence of the lcr_bit_depth_idc[i][j] and lcr_chroma_format_idc[i][j] syntax elements that indicate the bitdepth and chroma format of the decoded pictures associated with the extended layer j in either a global, when i is equal to 1, or a local, when i is equal to 0, LCR OBU. If lcr_format_info_present_flag[i][j] is 1, then the syntax elements lcr_bit_depth_idc[i][j] and lcr_chroma_format_idc[i][j] are present in the LCR OBU. If lcr_format_info_present_flag[i][j] is 0, then the syntax elements lcr_bit_depth_idc[i][j] and lcr_chroma_format_idc[i][j] are not present in the LCR OBU.

lcr_cropping_window_present_flag[i][j] specifies the presence of a cropping window that should be applied to the decoded pictures associated with the extended layer j in either a global, when i is equal to 1, or a local, when i is equal to 0, LCR OBU, after upscaling such pictures to a width of lcr_max_pic_width[i][j] and to a height of lcr_max_pic_height[i][j]. The value of lcr_cropping_window_present_flag[i][j], when present in an activated LCR OBU in an extended layer with index j shall equal seq_cropping_window_present_flag.

lcr_bit_depth_idc[i][j] specifies the bit_depth for the decoded pictures associated with the extended layer j in either a global, when i is equal to 1, or a local, when i is equal to 0, LCR OBU. The value of lcr_bit_depth_idc[i][j] in an activated LCR OBU in an extended layer with index j shall equal bit_depth_idc.

lcr_chroma_format_idc[i][j] specifies the chroma format idc for the decoded pictures associated with the extended layer j in either a global, when i is equal to 1, or a local, when i is equal to 0, LCR OBU. The value of lcr_chroma_format_idc[i][j] in an activated LCR OBU in an extended layer with index j shall equal chroma_format_idc.

lcr_cropping_win_left_offset[i][j], lcr_cropping_win_right_offset[i][j],

lcr_cropping_win_top_offset[i][j], and lcr_cropping_win_bottom_offset[i][j] specify the cropping window that should be used to generate the output of the decoding process in combination with the lcr_max_pic_width[i][j] and lcr_max_pic_height[i][j] syntax elements, using the decoded pictures associated with the extended layer j in either a global, when i is equal to 1, or a local, when i is equal to 0, LCR OBU. The values of lcr_cropping_win_left_offset[i][j], lcr_cropping_win_right_offset[i][j], lcr_cropping_win_top_offset[i][j], and lcr_cropping_win_bottom_offset[i][j] in an activated LCR OBU in an extended layer with index j shall match the values of seq_cropping_win_left_offset, seq_cropping_win_right_offset, seq_cropping_win_top_offset, and seq_cropping_win_bottom_offset.

§ 6.8.9. LCR embedded layer info semantics

lcr_mlayer_map[isGlobal][xId] specifies a map that indicates which embedded layers are present in the extended layer with ID equal to xId.

lcr_tlayer_map[isGlobal][xId][j] specifies a map that indicates which temporal layers are present in the extended layer with ID equal to xId for the current embedded layer with ID equal to j.

It is a requirement of bitstream conformance that the indication of the dependency information for each extended layer with obu_xlayer_id equal to xId, in the activated LCR OBU, denoted by lcr_mlayer_map[isGlobal][xId] and lcr_tlayer_map[isGlobal][xId][cMId], if present, shall agree with the equivalent indication in the activated sequence header, denoted by MlayerDependencyMap[cMId][rMId] and TlayerDependencyMap[cMId][cTId][rTId], so that:

- If isGlobal is equal to 0, for any embedded layer with ID equal to cMId if MlayerDependencyMap[cMId][rMId] is equal to 1 and lcr_mlayer_map[0][xId] & (1 << cMId) is greater than 0, lcr_mlayer_map[0][xId] & (1 << rMId) shall not be equal to 0 for all non-negative rMId less than cMId.
- If isGlobal is equal to 1, for any embedded layer with ID equal to cMId if MlayerDependencyMap[cMId][rMId] is equal to 1 and lcr_mlayer_map[1][xId] & (1 << cMId) is greater than 0, lcr_mlayer_map[1][xId] & (1 << rMId) shall not be equal to 0 for all non-negative rMId less than cMId.
- If isGlobal is equal to 0, for any embedded layer with ID equal to cMId and temporal layer with ID equal to cTId, if TlayerDependencyMap[cMId][cTId][rTId] is equal to 1 and lcr_tlayer_map[0][xId][cMId] & (1 << cTId) is greater than 0, lcr_tlayer_map[0][xId][cMId] & (1 << rTId) shall not be equal to 0 for all non-negative rTId less than cTId.
- If isGlobal is equal to 1, for any embedded layer with ID equal to cMId and temporal layer with ID equal to cTId, if TlayerDependencyMap[cMId][cTId][rTId] is equal to 1 and lcr_tlayer_map[1][xId][cMId] & (1 << cTId) is greater than 0, lcr_tlayer_map[1][xId][cMId] & (1 << rTId) shall not be equal to 0 for all non-negative rTId less than cTId.

NOTE: Above bitstream constraints on lcr_mlayer_map (and similarly for lcr_tlayer_map based on TlayerDependencyMap) make sure that, if MlayerDependencyMap[cMId][rMId] is equal to 1, any embedded layer with ID rMId referenced from the existing embedded layer with ID cMId are indicated to be present in the activated LCR. Otherwise, if MlayerDependencyMap[cMId][rMId] is equal to 0, indicating that an embedded layer with ID cMId does not depend on an embedded layer with ID rMId, lcr_mlayer_map[isGlobal][xId] is allowed to indicate that the embedded layer with ID rMId may or may not be present.

lcr_layer_atlas_segment_id[isGlobal][xId][j] specifies the atlas segment ID with which the current embedded layer with obu_mlayer_id equal to j in the extended layer with obu_xlayer_id equal to xId is associated.

lcr_priority_order[isGlobal][xId][j] indicates the priority order of an embedded layer with ID j in an extended layer with ID xId when rendering it on an atlas compared to other embedded layers. The lower the value of lcr_priority_order[isGlobal][xId][j] the higher the priority rendering order of that layer compared to other layers with a higher value. If this information is missing or two or more layers have the same priority value, then the priority between them is determined based on the embedded layer ID followed by the extended layer ID of the layers (the lower ID value has a higher rendering priority than a higher ID value). Layers with a higher rendering priority value are rendered first compared to layers with a lower rendering priority value when placed on an atlas.

lcr_rendering_method[isGlobal][xId][j] indicates the rendering method applied to the embedded layer with ID j in the extended layer with ID xId compared to previously rendered layers according to their priority order value. The interpretation of the value of lcr_rendering_method is the same as for lcr_xlayer_rendering_method.

lcr_layer_type[isGlobal][xId][j] indicates the type of the embedded layer with ID j in the extended layer with ID xId as specified in [Table 6.8](#):

Table 6.8: Layer type values for LCR embedded layers

lcr_layer_type	Label	Interpretation
0	TEXTURE_LAYER	Texture
1	AUX_LAYER	Auxiliary
2-255	-	Reserved

Reserved values of lcr_layer_type[isGlobal][xId][j] are for future use by AOMedia. They shall be ignored by decoders conforming to this version of this specification.

lcr_auxiliary_type[isGlobal][xId][j] indicates the auxiliary type of the embedded layer with ID j in the extended layer with ID xId as specified in [Table 6.9](#):

Table 6.9: Auxiliary type values for LCR embedded layers

lcr_auxiliary_type	Label	Interpretation
0	ALPHA_AUX	Alpha auxiliary image
1	DEPTH_AUX	Depth auxiliary image
2	SEGMENTATION_AUX	Segmentation auxiliary image
3	GAIN_MAP_AUX	Gain map auxiliary image
4-127	-	Reserved
128-159	-	Unspecified
160-255	-	Reserved

NOTE: The interpretation of auxiliary layers with lcr_auxiliary_type in the range 128 to 159, inclusive, is specified through means external to the bitstream (e.g., container metadata or application-layer signaling).

lcr_auxiliary_type[isGlobal][xId][j] shall be in the range of 0 to 3, inclusive, or 128 to 159, inclusive, for bitstreams conforming to this specification. Decoders shall ignore auxiliary layers whose lcr_auxiliary_type[isGlobal][xId][j] value is reserved or whose interpretation is not known through external means.

lcr_view_type[isGlobal][xId][j] indicates the view type of the embedded layer with ID j in the extended layer with ID xId as specified in [Table 6.10](#):

Table 6.10: View type values for LCR embedded layers

lcr_view_type	Label	Interpretation
---------------	-------	----------------

0	VIEW_UNSPECIFIED	The view type is undefined or not specified
1	VIEW_CENTER	Central perspective view
2	VIEW_LEFT	View from the left perspective
3	VIEW_RIGHT	View from the right perspective
4	VIEW_EXPLICIT	Explicit view ID indication
5-255	-	Reserved

Reserved values of `lcr_view_type[ isGlobal ][ xId ][ j ]` are for future use by AOMedia. They shall be ignored by decoders conforming to this version of this specification.

`lcr_view_id[ isGlobal ][ xId ][ j ]` indicates the view id associated with the embedded layer with ID `j` in the extended layer with ID `xId`.

`lcr_dependent_layer_map[ isGlobal ][ xId ][ j ]` indicates with which embedded layers the current embedded layer with layer ID equal to `j`, in the extended layer `xId`, depends on in terms of inter prediction. If `lcr_dependent_layer_map[ isGlobal ][ xId ][ j ]` is equal to 0, then the current embedded layer can be independently decoded from other embedded layers.

`lcr_same_sh_max_resolution_flag[ isGlobal ][ xId ][ j ]` equal to 1, or not present, indicates that for the embedded layer with `obu_mlayer_id` equal to `j` in the extended layer with `obu_xlayer_id` equal to `xId` in an activated LCR OBU, the resolution limits for that layer are set equal to those in the activated sequence header, i.e., equal to `max_frame_width_minus_1 + 1` and `max_frame_height_minus_1 + 1` respectively. In that case the syntax elements `lcr_max_expected_width[ isGlobal ][ xId ][ j ]` and `lcr_max_expected_height[ isGlobal ][ xId ][ j ]` are not present.

`lcr_max_expected_width[ isGlobal ][ xId ][ j ]` in an activated LCR OBU specifies the maximum expected `FrameWidth` for all frames in embedded layer `j` of extended layer `xId`. It is a requirement of bitstream conformance that `FrameWidth` for all frames in embedded layer `j` of extended layer `xId` shall be less than or equal to `lcr_max_expected_width[ isGlobal ][ xId ][ j ]`. It is also a requirement of bitstream conformance that `lcr_max_expected_width[ isGlobal ][ xId ][ j ]` shall be less than or equal to `max_frame_width_minus_1 + 1` obtained from the activated sequence header.

`lcr_max_expected_height[ isGlobal ][ xId ][ j ]` in an activated LCR OBU specifies the maximum expected `FrameHeight` for all frames in embedded layer `j` of extended layer `xId`. It is a requirement of bitstream conformance that `FrameHeight` for all frames in embedded layer `j` of extended layer `xId` shall be less than or equal to `lcr_max_expected_height[ isGlobal ][ xId ][ j ]`. It is also a requirement of bitstream conformance that `lcr_max_expected_height[ isGlobal ][ xId ][ j ]` shall be less than or equal to `max_frame_height_minus_1 + 1` obtained from the activated sequence header.

§ 6.8.10. LCR xlayer color info semantics

`layer_color_description_idc`, `layer_color primaries`, `layer_matrix_coefficients`, `layer_transfer_characteristics`, `layer_full_range_flag` specify the color information for this layer with the same interpretation as `ops_color_description_idc`, `ops_color primaries`, `ops_matrix_coefficients`, `ops_transfer_characteristics` and `ops_full_range_flag`.

§ 6.9. Atlas segment info OBU semantics

§ 6.9.1. General

The Atlas Segment provides spatial layout and composition information for organizing multiple layers into a unified visual presentation. An atlas defines a virtual canvas or coordinate space onto which different video layers can be mapped, positioned, and composed. The atlas mechanism serves several key purposes:

Spatial composition and layout: An atlas specifies how multiple decoded video layers should be arranged in 2D space to form the final rendered output. Each atlas segment represents a rectangular region that can be populated by content from one or more video layers. The atlas defines:

- The nominal dimensions of the virtual canvas (signaled as `ats_nominal_width_minus_1 + 1` and `ats_nominal_height_minus_1 + 1`)
- How the canvas is divided into regions (column and row grid)
- Which input streams (layers) contribute to each region
- The position and size of each layer's content within the atlas space

Multi-layer composition modes: The atlas supports several composition modes through `ats_atlas_segment_mode_idc`:

- **Enhanced Atlas** (mode 0): Defines the atlas as a 2D grid of rectangular regions that are grouped into segments; stream-to-segment association is indicated in the LCR via `lcr_layer_atlas_segment_id`, enabling multiple layers to share a single segment
- **Region-based layout** (mode 1): Divides the atlas into a grid of regions that can be uniformly or non-uniformly spaced, enabling regular tiling patterns
- **Basic composition** (mode 2): Direct mapping of input streams to rectangular regions within the atlas
- **Multistream composition** (mode 3): Composes multiple independent video streams into a single atlas, with optional background filling
- **Multistream with alpha** (mode 4): Like mode 3 but with per-segment alpha channel support for transparency

Subpicture and region-of-interest support: The atlas is particularly powerful for subpicture applications where different regions of interest are encoded as separate layers. For example, in a video conferencing scenario, the atlas might define a 1920x1080 virtual screen where:

- Segment 0 maps to layer 0: main speaker at position (0, 0) with size 960x1080
- Segment 1 maps to layer 1: participant thumbnails at position (960, 0) with size 960x540
- Segment 2 maps to layer 2: shared content at position (960, 540) with size 960x540

Each segment can be independently decoded and positioned, enabling selective decoding and rendering based on viewport or bandwidth constraints.

Relationship with LCR and MSDO: The atlas works in conjunction with either the Layer Configuration Record (LCR) or the Multi Stream Decoder Operation (MSDO) OBU to define the complete layer structure. While the LCR describes the semantic properties of each layer (texture vs auxiliary, view

association, layer type), the atlas describes the geometric properties (position, size, spatial relationships). Layers are associated with atlas segments through `lcr_layer_atlas_segment_id` in the LCR, creating the link between semantic layer metadata and spatial layout information.

Alternatively, when using MSDO instead of LCR, the atlas provides spatial layout information for the extended layers defined in the MSDO OBU. In this case, each extended layer identified by `sub_xlayer_id[i]` in the MSDO corresponds to an input stream in the atlas segment description (via `ats_input_stream_id` or `ats_msi_input_stream_id`), and the atlas defines how these independently decodable extended layers are spatially composed. The MSDO approach provides a simpler layer identification mechanism suitable for applications where extended layers represent complete, independently decodable views or streams that are spatially composed using the atlas.

Virtual canvas rendering: The atlas can represent a virtual image larger than any individual layer, which is particularly useful for:

- Viewport-dependent streaming where different regions are encoded at different qualities
- Subpicture video where high-resolution content is split into independently decodable subpictures
- Multi-view displays where different viewing positions see different subsets of the atlas
- Scalable region-of-interest encoding where focus areas get higher quality encoding

For detailed usage examples including stereoscopic composition, subpicture layouts, and multi-view scenarios, see [Annex G: Layer composition and Atlas usage examples \(informative\)](#).

atlas_segment_id indicates the atlas segment id associated with the current atlas segment information OBU, which can be referred by other syntax structures in this specification.

ats_atlas_segment_mode_idc specifies the representation description and coding of the atlas segments as specified in [Table 6.11](#):

Table 6.11: Specifies the representation description and coding of the atlas segments

ats_atlas_segment_mode_idc	Label	Description
0	ENHANCED_ATLAS	Enhanced Atlas description
1	BASIC_ATLAS	Basic Atlas description
2	SINGLE_ATLAS	Single Atlas description
3	MULTISTREAM_ATLAS	Multistream Atlas description
4	MULTISTREAM_ALPHA_ATLAS	Multistream Alpha Atlas description

It is a requirement of bitstream conformance that `ats_atlas_segment_mode_idc` is less than or equal to 4.

It is a requirement of bitstream conformance that when `ats_atlas_segment_mode_idc[xAId]` is equal to `MULTISTREAM_ATLAS` or `MULTISTREAM_ALPHA_ATLAS`, `obu_xlayer_id` is equal to `GLOBAL_XLAYER_ID`.

ats_nominal_width_minus_1 plus 1 specifies the nominal width of the atlas.

ats_nominal_height_minus_1 plus 1 specifies the nominal height of the atlas.

§ 6.9.2. Atlas label segment info semantics

ats_signaled_atlas_segment_ids_flag indicates whether the atlas segments are assigned explicit IDs or these are set equal to their index. When **ats_signaled_atlas_segment_ids_flag** is equal to 1, then explicit IDs are assigned to each atlas segment. If **ats_signaled_atlas_segment_ids_flag** is equal to 0, then the ID of each atlas segment is equal to its index.

ats_atlas_segment_id[xlayerId][xAId][i] indicates the ID associated with the atlas segment with index *i*.

§ 6.9.3. Atlas enhanced atlas info semantics

The Enhanced Atlas (**ats_atlas_segment_mode_idc** == **ENHANCED_ATLAS**) describes the spatial layout of an atlas as a two-dimensional grid of rectangular regions. The **ats_enhanced_atlas_info** syntax structure is the top-level container for this description; it calls **ats_region_info** to define the grid geometry and **ats_region_to_segment_mapping** to group grid regions into named atlas segments.

Purpose and spatial layout: The atlas grid divides the virtual canvas into (**ats_num_region_columns_minus_1** + 1) columns and (**ats_num_region_rows_minus_1** + 1) rows. Each cell of the grid is a rectangular region. When **ats_uniform_spacing_flag** is equal to 1, all regions have the same width and height. When it is equal to 0, each column width and row height is signaled individually, enabling non-uniform layouts such as a large main area flanked by smaller participant windows. One or more adjacent rectangular groups of regions are then combined into atlas segments by **ats_region_to_segment_mapping**.

Association with the LCR: The segment IDs assigned by **ats_enhanced_atlas_info** (either implicitly as indices 0, 1, 2, ... or explicitly via **ats_label_segment_info** when **ats_signaled_atlas_segment_ids_flag** is equal to 1) are the values that decoders must match against **lcr_layer_atlas_segment_id[isGlobal][xId][j]** in the Layer Configuration Record. When **lcr_local_atlas_id_present_flag[xId]** is equal to 1, the local LCR for extended layer *xId* identifies its associated atlas via **lcr_local_atlas_id[xId]**, and each embedded layer *j* within that extended layer indicates which atlas segment it contributes to through **lcr_layer_atlas_segment_id**. This is the sole mechanism by which the Enhanced Atlas resolves which layer provides content for a given segment — no stream identifiers are present in the atlas itself.

Multiple layers per segment: Because the mapping is expressed in the LCR rather than in the atlas, multiple embedded layers from the same or different extended layers may reference the same segment ID. This supports co-located auxiliary data: for example, a texture layer, an alpha layer, and a depth layer for the same spatial region all carry the same **lcr_layer_atlas_segment_id**. The rendering order among layers sharing a segment is controlled by **lcr_priority_order**, and the compositing operation by **lcr_rendering_method**.

§ 6.9.3.1. Atlas region info semantics

ats_num_region_columns_minus_1[xAId] plus 1 specifies the number of column regions to which an atlas with ID equal to *xAId* needs to be segmented.

It is a requirement of bitstream conformance that **ats_num_region_columns_minus_1** is less than **MAX_ATLAS_COLS**.

ats_num_region_rows_minus_1[xAId] plus 1 specifies the number of row regions to which an atlas with ID equal to *xAId* needs to be segmented.

It is a requirement of bitstream conformance that `ats_num_region_rows_minus_1` is less than `MAX_ATLAS_ROWS`.

`ats_uniform_spacing_flag[ xAId ]` equal to 1 specifies that the regions to which an atlas is segmented are uniformly spaced. `ats_uniform_spacing_flag[ xAId ]` equal to 0 specifies that the atlas regions are not uniformly spaced and the region widths and heights are signaled individually.

`ats_column_width_minus_1[ xAId ][ i ]` plus 1 indicates the width of the regions in column `i` in the atlas with ID `xAId`.

`ats_row_height_minus_1[ xAId ][ i ]` plus 1 indicates the height of the regions in row `i` in the atlas with ID `xAId`.

`ats_region_width_minus_1[ xAId ]` plus 1 indicates the width of all regions in the atlas with ID `xAId`.

`ats_region_height_minus_1[ xAId ]` plus 1 indicates the height of all regions in the atlas with ID `xAId`.

§ 6.9.3.2. Atlas region to segment mapping semantics

`ats_single_region_per_atlas_segment_flag[ xAId ]` indicates whether there is one to one mapping of atlas regions with atlas segments.

If `ats_single_region_per_atlas_segment_flag[ xAId ]` is equal to 0, then the mapping of atlas regions with atlas segments is not one to one.

If `ats_single_region_per_atlas_segment_flag[ xAId ]` is equal to 1, then the mapping of atlas regions with atlas segments is one to one.

If `ats_single_region_per_atlas_segment_flag[ xAId ]` is equal to 1, it is a requirement of bitstream conformance that `NumRegionsInAtlas[ xAId ]` is less than or equal to `MAX_NUM_ATLAS_SEGMENTS`.

`ats_top_left_region_column[ xAId ][ i ]` indicates the column of the first region associated with the segment with index `i`.

`ats_top_left_region_row[ xAId ][ i ]` indicates the row of the first region associated with the segment with index `i`.

`ats_bottom_right_region_column_off[ xAId ][ i ]` indicates the offset for the column of the last region associated with the segment with index `i`. The column of the last region is derived as `ats_top_left_region_column[ xAId ][ i ] + ats_bottom_right_region_column_off[ xAId ][ i ]`.

`ats_bottom_right_region_row_off[ xAId ][ i ]` indicates the offset for the row of the last region associated with the segment with index `i`. The row of the last region is derived as `ats_top_left_region_row[ xAId ][ i ] + ats_bottom_right_region_row_off[ xAId ][ i ]`.

NOTE: The semantics of `ats_num_atlas_segments_minus_1` are provided in [§ 6.9.6 Atlas basic info semantics](#).

§ 6.9.4. Atlas multistream info semantics

NOTE: An informative composition process for MULTISTREAM_ATLAS and MULTISTREAM_ALPHA_ATLAS modes is described in [Annex D: Multistream composition process \(informative\)](#).

ats_msi_input_stream_id, **ats_msi_width**, **ats_msi_height**, **ats_msi_num_atlas_segments_minus_1**, **ats_msi_segment_top_left_pos_x**, **ats_msi_segment_top_left_pos_y**, **ats_msi_segment_width**, and **ats_msi_segment_height** have the same semantics as **ats_input_stream_id**, **ats_width**, **ats_height**, **ats_num_atlas_segments_minus_1**, **ats_segment_top_left_pos_x**, **ats_segment_top_left_pos_y**, **ats_segment_width**, and **ats_segment_height** in the Atlas basic info semantics [§ 6.9.6 Atlas basic info semantics](#).

ats_msi_background_info_present_flag equal to 1 specifies that the syntax elements **ats_msi_background_red_value**, **ats_msi_background_green_value**, and **ats_msi_background_blue_value** are present. **ats_msi_background_info_present_flag** equal to 0 specifies the syntax elements are not present.

ats_msi_background_red_value specifies the red component of the background color as the 8-bit quantized value (D'R) in Recommendation ITU-R BT.709. When **ats_msi_background_red_value** is not present, it is inferred to be equal to 16.

ats_msi_background_green_value specifies the green component of the background color as the 8-bit quantized value (D'G) in Recommendation ITU-R BT.709. When **ats_msi_background_green_value** is not present, it is inferred to be equal to 16.

ats_msi_background_blue_value specifies the blue component of the background color as the 8-bit quantized value (D'B) in Recommendation ITU-R BT.709. When **ats_msi_background_blue_value** is not present, it is inferred to be equal to 16.

§ 6.9.5. Atlas multistream with alpha info semantics

ats_msi_alpha_segments_present_flag equal to 1 specifies that the syntax element **ats_msi_alpha_segment_flag** is present in the bitstream. **ats_msi_alpha_segments_present_flag** equal to 0 specifies that the syntax element is not present.

ats_msi_alpha_segment_flag[xlayerId][xAId][i] specifies that the atlas segment with index *i* is an alpha frame. When not present, **ats_msi_alpha_segment_flag[xlayerId][xAId][i]** shall be inferred to be equal to 0.

NOTE: The semantics of **ats_msi_input_stream_id**, **ats_msi_width**, **ats_msi_height**, **ats_msi_num_atlas_segments_minus_1**, **ats_msi_segment_top_left_pos_x**, **ats_msi_segment_top_left_pos_y**, **ats_msi_segment_width**, **ats_msi_segment_height**, **ats_msi_background_info_present_flag**, **ats_msi_background_red_value**, **ats_msi_background_green_value**, and **ats_msi_background_blue_value** are provided in [§ 6.9.4 Atlas multistream info semantics](#).

§ 6.9.6. Atlas basic info semantics

ats_stream_id_present[xAId] indicates **ats_input_stream_id** is signaled.

ats_width[xAId] indicates the width of the atlas with ID xAId.

ats_height[xAId] indicates the height of the atlas with ID xAId.

ats_num_atlas_segments_minus_1[xAId] plus one indicates the number of atlas segments of the atlas with ID xAId.

It is a requirement of bitstream conformance that **ats_num_atlas_segments_minus_1** is less than **MAX_NUM_ATLAS_SEGMENTS**.

ats_input_stream_id[xAId][i] specifies the **obu_xlayer_id** value of the stream corresponding to the *i*-th composed region.

All values in **ats_input_stream_id[xAId]** shall be unique.

ats_segment_top_left_pos_x[xAId][i] indicates the horizontal coordinate of the top left position of the atlas segment with index *i*.

ats_segment_top_left_pos_y[xAId][i] indicates the vertical coordinate of the top left position of the atlas segment with index *i*.

ats_segment_width[xAId][i] indicates the width of the atlas segment with index *i*.

ats_segment_height[xAId][i] indicates the height of the atlas segment with index *i*.

§ 6.10. Operating point set OBU semantics

§ 6.10.1. General

The Operating Point Set (OPS) OBU indicates possible decoding operating points associated with the bitstream.

Each OPS OBU is associated with an extended layer via **obu_xlayer_id**:

- When **obu_xlayer_id** is equal to **GLOBAL_XLAYER_ID** (31), the OPS applies to the entire [multistream](#) (global OPS).
- When **obu_xlayer_id** is less than **GLOBAL_XLAYER_ID**, the OPS applies to that specific extended layer (local OPS).

OPS are identified by the pair (**obu_xlayer_id**, **ops_id**). Up to 16 OPS can be defined per extended layer (**ops_id** is a 4-bit value), each containing up to 7 operating points (**ops_cnt** is a 3-bit value with 0 reserved for reset). In a [multistream](#) with up to 31 extended layers and 16 OPS each, up to 496 total OPS are possible. [Singlestream](#) bitstreams support up to 16 OPS.

Each OPS groups operating points sharing a common **ops_intent** (e.g., scalability, stereo, gain map). Applications can:

1. First filter OPS by intent to find relevant sets.
2. Then examine individual operating points for detailed selection based on profile/level/tier, color info, decoder model info, and layer maps.
3. Consider multiple OPS simultaneously when needed.

The reset and update behavior of the OPS OBU is determined by the combination of `ops_reset_flag` and `ops_cnt`:

1. `ops_reset_flag` equal to 1 and `ops_cnt` equal to 0: All OPS for the associated extended layer (or all layers if global) are reset. No OPS remains active.
2. `ops_reset_flag` equal to 1, `ops_id` equal to `x`, and `ops_cnt` equal to `N` ($N > 0$): All OPS are reset, then OPS `x` is defined with `N` operating points.
3. `ops_reset_flag` equal to 0, `ops_id` equal to `x`, and `ops_cnt` equal to 0: Only OPS `x` is reset. Other OPS remain active.
4. `ops_reset_flag` equal to 0, `ops_id` equal to `x`, and `ops_cnt` equal to `N` ($N > 0$): OPS `x` is set or updated with `N` operating points. Other OPS are unchanged.

OPS information persists across coded video sequences. As informative guidance (not a normative requirement): a decoder that selects an operating point for the duration of a coded video sequence may only switch to an operating point that is a subset of the current one (downgrading is permitted; upgrading to decode additional layers is not, since the required data may not be available).

OPS processing is entirely optional. A decoder may ignore all OPS information and decode the entire bitstream.

§ 6.10.2. Operating point set OBU syntax elements

`ops_reset_flag[ obu_xlayer_id ]` equal to 1 specifies that all operating point sets associated with `obu_xlayer_id` are reset. `ops_reset_flag` equal to 0 specifies that the operating point sets associated with `obu_xlayer_id` are not reset. The specific behavior depends on the combination with `ops_cnt` as described in [§6.10.1 General](#).

`ops_id[ obu_xlayer_id ]` specifies the operating point set identifier within the extended layer given by `obu_xlayer_id`. The value of `ops_id` is in the range of 0 to 15, inclusive.

`ops_cnt[ obu_xlayer_id ][ opsID ]` specifies the number of operating points in the OPS identified by `opsID` within the extended layer given by `obu_xlayer_id`. When `ops_cnt` is equal to 0, the OPS is being reset or cleared as described in [§6.10.1 General](#). When `ops_cnt` is greater than 0, it specifies the number of operating points (1 to 7).

`ops_priority[ obu_xlayer_id ][ opsID ]` specifies the priority of the OPS identified by `opsID` within the extended layer given by `obu_xlayer_id`. Lower values indicate higher priority.

When `ops_priority[ obu_xlayer_id ][ opsID ]` is not present, `ops_priority [ obu_xlayer_id ][ opsID ]` shall be inferred to be equal to 0.

`ops_intent[ obu_xlayer_id ][ opsID ]` specifies the intent of the OPS at the `opsID` within the `obu_xlayer_id` as specified in [Table 6.12](#):

Table 6.12: *ops_intent* values and labels

<code>ops_intent</code>	Label
0	OPSI_UNSPECIFIED
1	OPSI_SCALABILITY
2	OPSI_STEREO

3	OPSI_TEXTURE_ALPHA
4	OPSI_TEXTURE_DEPTH
5	OPSI_GAIN_MAP
6	OPSI_MULTIVIEW
7-127	RESERVED

When `ops_intent[ obu_xlayer_id ][ opsID ]` is not present, `ops_intent[ obu_xlayer_id ][ opsID ]` shall be inferred to be equal to 0. Reserved values of `ops_intent[ obu_xlayer_id ][ opsID ]` are for future use by AOMedia. They shall be ignored by decoders conforming to this version of this specification.

ops_intent_present_flag[obu_xlayer_id][opsID] equal to 1 specifies that `ops_op_intent` is present in the current OPS. `ops_intent_present_flag[ obu_xlayer_id ][ opsID ]` equal to 0 specifies `ops_op_intent` is not present in the current OPS.

ops_ptl_present_flag[obu_xlayer_id][opsID] equal to 1 specifies that profile, tier, and level information is present for all the operating points within the OPS identified by `opsID`. When `obu_xlayer_id` is equal to `GLOBAL_XLAYER_ID`, this information is conveyed via the `ops_aggregate_info( )` and `ops_seq_profile_tier_level_info( )` syntax structures. When `obu_xlayer_id` is less than `GLOBAL_XLAYER_ID`, this information is conveyed via the `ops_seq_profile_tier_level_info( )` syntax structure. `ops_ptl_present_flag[ obu_xlayer_id ][ opsID ]` equal to 0 specifies that profile, tier, and level information is not present for the operating points within the OPS identified by `opsID`.

ops_color_info_present_flag[obu_xlayer_id][opsID] equal to 1 specifies that the `ops_color_info( opsID, i )` syntax is present in the current OPS. `ops_color_info_present_flag[ obu_xlayer_id ][ opsID ]` equal to 0 specifies that the `ops_color_info( opsID, i )` syntax is not present in the current OPS.

ops_mlayer_info_idc[opsID] is present only for global OPS (i.e., when `obu_xlayer_id == GLOBAL_XLAYER_ID`). `ops_mlayer_info_idc[ opsID ]` equal to 0 specifies that the `ops_mlayer_info` syntax structure is not present in the current OPS. `ops_mlayer_info_idc[ opsID ]` equal to 1 specifies that the `ops_mlayer_info` syntax is present in the current OPS for every extended layer in each operating point. `ops_mlayer_info_idc[ opsID ]` equal to 2 specifies that, for each extended layer in each operating point, the `ops_mlayer_info` syntax is either present in the current OPS or inherited from another operating point, as indicated by `ops_mlayer_explicit_info_flag`.

It is a requirement of bitstream conformance that `ops_mlayer_info_idc[ opsID ]` is not equal to 3.

ops_reserved_2bits must be set to 0. The value shall be ignored by a decoder.

ops_data_size[obu_xlayer_id][opsID][i] specifies the size in bytes of the *i*-th operating point payload data. This value enables a decoder to skip over or validate individual operating point payloads.

ops_op_intent[obu_xlayer_id][opsID][i] specifies the intent of the *i*-th operating point with the same semantics as `ops_intent`.

It is a requirement of bitstream conformance that when `ops_ptl_present_flag[ obu_xlayer_id ][ opsID ]` is equal to 1, the bitstream corresponding to the *i*-th operating point associated with `obu_xlayer_id` and `opsID` shall satisfy all bitstream constraints specified in [Annex A.4 Levels](#), by setting `seq_profile_idc`, `seq_tier`, and `seq_level_idx` to `ops_seq_profile_idc[ obu_xlayer_id ][ opsID ][ i ][ j ]`,

ops_tier_flag[obu_xlayer_id][opsID][i][j], and ops_level_idx[obu_xlayer_id][opsID][i][j], respectively, where j is the applicable layer index.

ops_decoder_model_info_for_this_op_present_flag[xId][opsID][i] equal to 1 specifies that the ops_decoder_model_info() syntax structure is present for the i-th operating point.

ops_decoder_model_info_for_this_op_present_flag[xId][opsID][i] equal to 0 specifies that the ops_decoder_model_info() syntax structure is not present.

ops_initial_display_delay_present_flag[xId][opsID][i] equal to 1 specifies that the ops_initial_display_delay_minus_1[xId][opsID][i] syntax element is present.

ops_initial_display_delay_present_flag[xId][opsID][i] equal to 0 specifies that ops_initial_display_delay_minus_1[xId][opsID][i] is not present.

ops_initial_display_delay_minus_1[xId][opsID][i] plus 1 specifies the number of decoded frames that should be present in the buffer pool before the first presentable frame is displayed. This will ensure that all presentable frames in the sequence can be decoded at or before the time that they are scheduled for display.

ops_xlayer_map[opsID][i] specifies a 31-bit bitmask for the i-th operating point. Bit j being set to 1 indicates that extended layer j is included in the operating point. ops_xlayer_map[opsID][i] is present and meaningful only for global OPS, i.e., when xId == GLOBAL_XLAYER_ID; for local OPS (xId != GLOBAL_XLAYER_ID) this syntax element is not present in the OPS OBU syntax.

ops_mlayer_explicit_info_flag[opsID][i][j] equal to 1 specifies that the ops_mlayer_info() syntax structure is explicitly present for the j-th extended layer. ops_mlayer_explicit_info_flag[opsID][i][j] equal to 0 specifies that the embedded layer and temporal layer information is inherited from the operating point set and operating point index referenced by ops_embedded_ops_id[opsID][i][j] and ops_embedded_op_index[opsID][i][j], respectively.

ops_embedded_ops_id[opsID][i][j] and **ops_embedded_op_index[opsID][i][j]** provide the operating point set identifier and operating point index, respectively, from which the j-th extended layer inherits its ops_mlayer_info configuration. This enables compact signaling when multiple operating points share embedded layer and temporal layer structure.

Let refID be equal to ops_embedded_ops_id[opsID][i][j].

It is a requirement of bitstream conformance that ops_embedded_op_index[opsID][i][j] is less than ops_cnt[obu_xlayer_id][refID].

If refID is equal to opsID, it is a requirement of bitstream conformance that ops_embedded_op_index[opsID][i][j] is less than j.

NOTE: These requirements ensure that the operating point is inherited from a previously received operating point.

opsBytes is a variable that contains the number of bytes read for the operating point.

It is a requirement of bitstream conformance that the computed value of opsBytes is equal to ops_data_size[obu_xlayer_id][opsID][i].

§ 6.10.3. Operating point set aggregate info semantics

The aggregate information applies to global OPS (`obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`) and describes the constraints for the combined multistream operating point.

ops_config_idc[opsID][i] indicates the aggregate profile identifier for the *i*-th operating point in the OPS identified by `opsID`. This profile applies to the combined multistream operating point.

ops_aggregate_level_idx[opsID][i] specifies the aggregate level indicator for the *i*-th operating point in the OPS identified by `opsID`. This level applies to the combined multistream operating point.

ops_max_tier_flag[opsID][i] specifies the maximum tier indicator for the *i*-th operating point in the OPS identified by `opsID`. This tier applies to the combined multistream operating point.

ops_max_interop[opsID][i] indicates the maximum interoperability point for the *i*-th operating point in the OPS identified by `opsID`.

§ 6.10.4. Operating point set sequence profile tier level information semantics

The sequence profile tier level information describes per-extended-layer profile, level, and tier constraints for each extended layer included in an operating point.

ops_seq_profile_idc[xId][opsID][i][j] specifies the profile indicator for the *j*-th extended layer in the *i*-th operating point of the OPS identified by `opsID`. This constrains the profile required to decode the *j*-th extended layer.

ops_level_idx[xId][opsID][i][j] specifies the level indicator for the *j*-th extended layer in the *i*-th operating point of the OPS identified by `opsID`. This constrains the level required to decode the *j*-th extended layer.

ops_tier_flag[xId][opsID][i][j] specifies the tier indicator for the *j*-th extended layer in the *i*-th operating point of the OPS identified by `opsID`. This constrains the tier required to decode the *j*-th extended layer.

ops_mlayer_count[xId][opsID][i][j] specifies the number of embedded layers for the *j*-th extended layer in the *i*-th operating point of the OPS identified by `opsID`.

ops_ptl_reserved_2bits must be set to 0. The value shall be ignored by a decoder.

§ 6.10.5. Operating point set decoder model info semantics

ops_decoder_buffer_delay[obu_xlayer_id][opsID][i] specifies the time interval between the arrival of the first bit in the smoothing buffer and the subsequent removal of the data that belongs to the first [coded frame](#) for operating point `op`, measured in units of 1/90000 seconds.

ops_encoder_buffer_delay[obu_xlayer_id][opsID][i] specifies, in combination with the `ops_decoder_buffer_delay` syntax element, the first bit arrival time of frames to be decoded to the smoothing buffer. `ops_encoder_buffer_delay` is measured in units of 1/90000 seconds.

For a video sequence that includes one or more random access points the sum of `ops_decoder_buffer_delay` and `ops_encoder_buffer_delay` shall be kept constant.

ops_low_delay_mode_flag[obu_xlayer_id][opsID][i] equal to 1 indicates that the smoothing buffer operates in low-delay mode for operating point op. In low-delay mode late decode times and buffer underflow are both permitted. ops_low_delay_mode_flag equal to 0 indicates that the smoothing buffer operates in strict mode, where buffer underflow is not allowed.

§ 6.10.6. Operating point set color info semantics

ops_color_description_idc[obu_xlayer_id][opsID][i] indicates the combination of color primaries, transfer characteristics, and matrix coefficients, within the i-th operating point index with an operating point id given by opsID, at the obu_xlayer_id as follows:

Table 6.13: ops_color_description_idc values and their interpretations

Value	Interpretation	ops_color_primaries	ops_transfer_characteristics	ops_matrix_coefficients
0	Explicitly signaled	Explicit	Explicit	Explicit
1	BT.709 SDR	1	1	1
2	BT.2100 PQ	9	16	9
3	BT.2100 HLG	9	18	9
4	sRGB	1	13	0
5	sYCC	1	13	5
6-127	Reserved	-	-	-

The value of ops_color_description_idc[obu_xlayer_id][opsID][i] shall be in the range of 0 to 127, inclusive. Values larger than 5 are reserved for future use by AOMedia and shall be ignored by decoders conforming to this version of this specification.

ops_color_primaries[obu_xlayer_id][opsID][i] specifies the color primaries at the i-th operating point index with an operating point id given by opsID at the obu_xlayer_id is an integer that is associated with the ColourPrimaries variable specified in ISO/IEC 23091-4/ITU-T H.273.

Table 6.14: ops_color_primaries values and names

ops_color_primaries	Name of color primaries	Description
1	CP_BT_709	[ITU-R-BT.709]
2	CP_UNSPECIFIED	Unspecified
4	CP_BT_470_M	BT.470 System M (historical)
5	CP_BT_470_B_G	BT.470 System B, G (historical)
6	CP_BT_601	[ITU-R-BT.601]
7	CP_SMPTE_240	SMPTE 240
8	CP_GENERIC_FILM	Generic film (color filters using illuminant C)
9	CP_BT_2020	BT.2020, BT.2100
10	CP_XYZ	SMPTE 428 (CIE 1931 XYZ)
11	CP_SMPTE_431	SMPTE RP 431-2
12	CP_SMPTE_432	SMPTE EG 432-1
22	CP_EBU_3213	EBU Tech. 3213-E

ops_transfer_characteristics[obu_xlayer_id][opsID][i] specifies the transfer characteristics at the i-th operating point index with an operating point id given by opsID at the obu_xlayer_id is an integer that is associated with the TransferCharacteristics variable specified in ISO/IEC 23091-4/ITU-T H.273.

ops_transfer_characteristics	Name of transfer characteristics	Description
0	TC_RESERVED_0	For future use
1	TC_BT_709	[ITU-R-BT.709]
2	TC_UNSPECIFIED	Unspecified
3	TC_RESERVED_3	For future use
4	TC_BT_470_M	BT.470 System M (historical)
5	TC_BT_470_B_G	BT.470 System B, G (historical)
6	TC_BT_601	[ITU-R-BT.601]
7	TC_SMPTE_240	SMPTE 240 M
8	TC_LINEAR	Linear
9	TC_LOG_100	Logarithmic (100 : 1 range)
10	TC_LOG_100_SQRT10	Logarithmic (100 * Sqrt(10) : 1 range)
11	TC_IEC_61966	IEC 61966-2-4
12	TC_BT_1361	BT.1361
13	TC_SRGB	sRGB or sYCC
14	TC_BT_2020_10_BIT	BT.2020 10-bit systems [Rec.2020]
15	Reserved	Reserved for AOMedia use
16	TC_SMPTE_2084	SMPTE ST 2084, ITU BT.2100 PQ
17	TC_SMPTE_428	SMPTE ST 428
18	TC_HLG	BT.2100 HLG, ARIB STD-B67

ops_matrix_coefficients[obu_xlayer_id][opsID][i] specifies the matrix coefficients at the i-th operating point index with an operating point id given by opsID at the obu_xlayer_id is an integer that is associated with the MatrixCoefficients variable specified in ISO/IEC 23091-4/ITU-T H.273.

Table 6.15: ops_matrix_coefficients values and names

ops_matrix_coefficients	Name of matrix coefficients	Description
0	MC_IDENTITY	Identity matrix
1	MC_BT_709	[ITU-R-BT.709]
2	MC_UNSPECIFIED	Unspecified
3	MC_RESERVED_3	For future use
4	MC_FCC	US FCC 73.628
5	MC_BT_470_B_G	BT.470 System B, G (historical)
6	MC_BT_601	[ITU-R-BT.601]
7	MC_SMPTE_240	SMPTE 240 M
8	MC_SMPTE_YCGCO	YCgCo
9	MC_BT_2020_NCL	BT.2020 non-constant luminance, BT.2100 YCbCr
10	MC_BT_2020_CL	BT.2020 constant luminance [Rec.2020]

11	MC_SMPTE_2085	SMPTE ST 2085 YDzDx
12	MC_CHROMAT_NCL	Chromaticity-derived non-constant luminance
13	MC_CHROMAT_CL	Chromaticity-derived constant luminance
14	MC_ICTCP	BT.2100 ICtCp
15	MC_IPT_C2	IPT-C2
16	MC_YCGCO_RE	YCgCo-Re
17	MC_YCGCO_RO	YCgCo-Ro

ops_full_range_flag[obu_xlayer_id][opsID][i] is a binary value that is associated with the VideoFullRangeFlag variable specified in ISO/IEC 23091-4/ITU-T H.273. **ops_full_range_flag** specifies the value of the full range flag at the i-th operating point index with an operating point id given by opsID at the obu_xlayer_id. **ops_full_range_flag** equal to 0 shall be referred to as the studio swing representation and **ops_full_range_flag** equal to 1 shall be referred to as the full swing representation for all intents relating to this specification.

§ 6.10.7. Operating point set mlayer info semantics

The mlayer info syntax structure describes the embedded layer and temporal layer configuration for each extended layer included in an operating point.

ops_mlayer_map[obuXLId][opsID][opIndex][xLId] specifies an 8-bit bitmask representing the embedded layers included for the xLId extended layer, within the operating point at index opIndex, in the OPS identified by opsID, at the obuXLId. Bit j being set to 1 indicates that embedded layer j is included.

ops_tlayer_map[obuXLId][opsID][opIndex][xLId][j] specifies a 4-bit bitmask representing the temporal layers included for embedded layer j of the xLId extended layer, within the operating point at index opIndex, in the OPS identified by opsID, at the obuXLId. Bit k being set to 1 indicates that temporal layer k is included.

It is a requirement of bitstream conformance that the indication of the dependency information for any operating point specified in an OPS OBU associated with this bitstream, denoted by **ops_mlayer_map[obuXLId][opsID][opIndex][xLId]** and **ops_tlayer_map[obuXLId][opsID][opIndex][xLId][cMId]**, if present, shall agree with the indication in the information in the activated sequence header, denoted by **MlayerDependencyMap[cMId][rMId]** and **TlayerDependencyMap[cMId][cTId][cTId]** so that:

- For any embedded layer with ID equal to cMId, if **MlayerDependencyMap[cMId][rMId]** is equal to 1 and **ops_mlayer_map[obuXLId][opsID][opIndex][xLId] & (1 << cMId)** is greater than 0, **ops_mlayer_map[obuXLId][opsID][opIndex][xLId] & (1 << rMId)** shall not be equal to 0 for all non-negative rMId less than cMId.
- For any embedded layer with ID equal to cMId and temporal layer with ID equal to cTId, if **TlayerDependencyMap[cMId][cTId][rTId]** is equal to 1 and **ops_tlayer_map[obuXLId][opsID][opIndex][xLId][cMId] & (1 << cTId)** is greater than 0, **ops_tlayer_map[obuXLId][opsID][opIndex][xLId][cMId] & (1 << rTId)** shall not be equal to 0 for all non-negative rTId less than cTId.

NOTE: Above bitstream constraints on `ops_mlayer_map` (and similarly for `ops_tlayer_map` based on `TLayerDependencyMap`) make sure that, if `MLayerDependencyMap[ cMId ][ rMId ]` is equal to 1, any embedded layer with ID `rMId` referenced from the existing embedded layer with ID `cMId` are indicated to be present in any operating point specified in an OPS OBU. Otherwise, if `MLayerDependencyMap[ cMId ][ rMId ]` is equal to 0, indicating that an embedded layer with ID `cMId` does not depend on an embedded layer with ID `rMId`, `lcr_mlayer_map[ isGlobal ][ xId ]` is allowed to indicate that the embedded layer with ID `rMId` may or may not be present in the operating point.

§ 6.11. Buffer removal timing OBU semantics

br_ops_dependent_flag equal to 1 specifies that the timing information associated with a specific operating point set is present in the `buffer_removal_timing_obu( )`. **br_ops_dependent_flag** equal to 0 specifies that timing information associated with an operating point set is not present in the `buffer_removal_timing_obu( )`.

br_ops_id specifies the operating point set id.

It is a requirement of bitstream conformance that **br_ops_id** is equal to an operating point set `ops_id[ obu_xlayer_id ]` that is present in the bitstream.

br_ops_cnt[br_ops_id] specifies the operating point count.

It is a requirement of bitstream conformance that **br_ops_cnt[br_ops_id]** is equal to `ops_cnt[ obu_xlayer_id ][ br_ops_id ]`.

NOTE: The conformance requirements on **br_ops_id** and **br_ops_cnt[br_ops_id]** ensure that the operating point index *i* in the `buffer_removal_timing_obu( )` loop has a one-to-one correspondence with the operating point index *i* in the `operating_point_set_obu( )` loop for the same operating point set. That is, the *i*-th operating point in the BRT OBU corresponds to the *i*-th operating point in the OPS OBU.

br_decoder_model_present_op_flag[br_ops_id][i] equal to 1 specifies that `br_buffer_removal_time` is present for operating point *i*. **br_decoder_model_present_op_flag[br_ops_id][i]** equal to 0 specifies that `br_buffer_removal_time` is not present.

br_time_op[br_ops_id][i] specifies the frame removal time in units of DecCT clock ticks counted from the removal time of the last random access point for operating point *i* of the specified operating point set **br_ops_id** when the current frame is not associated with a random access point and from the previous random access point when the current frame is associated with a random access point.

br_time specifies the frame removal time in units of DecCT clock ticks counted from the removal time of the last random access point when the current frame is not associated with a random access point and from the previous random access point when the current frame is associated with a random access point.

§ 6.12. Quantizer Matrix OBU semantics

qm_bit_map is a bitmask that specifies which quantizer matrices are present in the OBU.

When there are multiple quantizer matrices OBUs between coded frames, it is a requirement of bitstream conformance that only the first quantizer matrix can have `qm_bit_map` equal to 0.

When there are multiple quantizer matrices OBUs between coded frames, it is a requirement of bitstream conformance that the same level of quantizer matrix is not specified twice in those OBUs.

qm_chroma_info_present_flag equal to 1 specifies that the chroma quantizer matrices are present in this OBU. `qm_chroma_info_present_flag` equal to 0 specifies that chroma quantizer matrices are not present and default chroma quantizer matrices shall be used.

qm_is_default_flag equal to 1 specifies that the default quantizer matrix is used for the current quantizer level and `QmDataPresent` for this level is set to 0. `qm_is_default_flag` equal to 0 specifies that user-defined quantizer matrix data is present via the `user_defined_qm()` syntax structure.

QmDataPresent is an array specifying which quantizer matrix levels have data that can be used.

QmSeen is an array specifying which quantizer matrix levels have been seen since the last frame.

QmProtected is an array specifying which quantizer matrix levels are protected. Unprotected levels will be reset at the first OBU with `obu_type` equal to `OBU_CLOSED_LOOP_KEY` or `OBU_OPEN_LOOP_KEY` in a temporal layer.

Initialize every entry of `QmProtected`, `QmSeen`, and `QmDataPresent` to zero at the start of a bitstream.

§ 6.13. Film grain OBU semantics

fgm_update_flags specifies a bitmap of which film grain models are present in the OBU. If bit `i` of `fgm_update_flags` is equal to 1 (i.e., if `fgm_update_flags & (1 << i)` is non-zero), then a film grain model is present for slot `i`.

When there are multiple film grain OBUs present in the same coded frame unit, it is a requirement of bitstream conformance that bit `i` of `fgm_update_flags` is equal to 1 in at most one film grain OBU.

NOTE: The same film grain slot can be reused or updated by a film grain OBU in a subsequent coded frame unit.

It is a requirement of bitstream conformance that `fgm_update_flags` is not equal to 0.

fgm_chroma_idc is used to derive the subsampling format used by the film grain.

It is a requirement of bitstream conformance that `fgm_chroma_idc` is less than or equal to 3.

save_grain_model(i) is a function call that indicates that all the syntax elements read in `film_grain_model` should be saved into an area of memory indexed by `i`.

FilmGrainPresent is an array that records which film grain OBUs have been received. Initialize every entry of `FilmGrainPresent` to zero at the start of a bitstream.

NOTE: `FilmGrainPresent` is only used to specify a conformance constraint and does not affect the decoding process.

§ 6.14. Content interpretation OBU semantics

A content interpretation OBU can be present in any embedded layer. However, when present, all instances of a content interpretation OBU in an embedded layer within a coded video sequence shall contain the same information. No such constraint exists for content interpretation OBUs in different embedded layers except parameters in the `time_info()` structure which shall be the same across all embedded layers within a coded video sequence.

If no content interpretation OBU is present for embedded layer `m`, the content interpretation parameters are inherited from embedded layer `k`, where `k` is the highest embedded layer less than `m` for which `MLayerPresenceMap[m][k]` is equal to 1 and content interpretation parameters have been established.

The content interpretation parameters for each embedded layer are initialized and updated as specified in [§ 7.3.8.11 Content interpretation parameters initialization](#). When a content interpretation OBU is present in a temporal unit that does not contain a [CLK](#) or [OLK](#) for the same embedded layer, and does not contain a [CLK](#) or [OLK](#) for any embedded layer `k` where `MLayerPresenceMap[m][k]` is equal to 1, the contents shall be identical to the content interpretation parameters established at the most recent random access point.

ci_scan_type_idc indicates how to interpret the pictures within a CVS in terms of progressive or interlace samples, as follows:

ci_scan_type_idc	Interpretation of ci_scan_type_idc
0	Unspecified
1	Progressive frame picture samples
2	Interlace field picture samples
3	Interlace complementary field-pair picture samples

ci_color_description_present_flag equal to 1 specifies that the syntax element `ci_color_description_idc` and associated color description syntax elements are present to indicate color space information. `ci_color_description_present_flag` equal to 0 specifies that `ci_color_description_idc` and associated syntax elements are not present.

ci_chroma_sample_position_present_flag equal to 1 specifies that syntax elements describing the chroma sample positions are present. `ci_chroma_sample_position_present_flag` equal to 0 specifies that chroma sample position syntax elements are not present.

ci_aspect_ratio_info_present_flag equal to 1 specifies that the aspect ratio syntax elements are present to indicate the aspect ratio of the decoded frames. `ci_aspect_ratio_info_present_flag` equal to 0 specifies that aspect ratio syntax elements are not present.

ci_timing_info_present_flag equal to 1 specifies that timing information is present to indicate frame timing parameters. `ci_timing_info_present_flag` equal to 0 specifies that timing information is not present.

ci_reserved_2bit must be set to 0. The value shall be ignored by a decoder.

ci_color_description_idc, **ci_color primaries**, **ci_matrix_coefficients**, **ci_transfer_characteristics**, **ci_full_range_flag** specify the color information for this layer with the same interpretation as `ops_color_description_idc`, `ops_color primaries`, `ops_matrix_coefficients`, `ops_transfer_characteristics` and `ops_full_range_flag`.

ci_chroma_sample_position_top indicates the chroma sampling grid alignment for top video field or for a frame using the 4:2:0 (in which the two chroma arrays have half the width and half the height of the associated luma array) or 4:2:2 (in which the two chroma arrays have half the width of the associated luma array) color formats. For 4:2:0 formats, these interpretations match those of the Chroma420SampleLocType variable specified in ISO/IEC 23091-4/ITU-T H.273.

The chroma sample positions allowed are:

ci_chroma_sample_position_(top/ bottom)	Name of chroma sample position	Meaning for 4:2:2 (offsets from (0,0) luma sample)	Meaning for 4:2:0 (offsets from (0,0) luma sample)
0	CSP_LEFT	Horizontal offset 0	Horizontal offset 0, vertical offset 0.5
1	CSP_CENTER	Horizontal offset 0.5	Horizontal offset 0.5, vertical offset 0.5
2	CSP_TOPLEFT	N/A	Horizontal offset 0, vertical offset 0
3	CSP_TOP	N/A	Horizontal offset 0.5, vertical offset 0
4	CSP_BOTTOMLEFT	N/A	Horizontal offset 0, vertical offset 1
5	CSP_BOTTOM	N/A	Horizontal offset 0.5, vertical offset 1
6	CSP_UNSPECIFIED	Unknown or determined by the application	Unknown or determined by the application

If ci_chroma_sample_position_top is present in the bitstream, it is a requirement of bitstream conformance that the value is less than or equal to 5.

ci_chroma_sample_position_bottom indicates the chroma sampling grid alignment for bottom video field using the 4:2:0 (in which the two chroma arrays have half the width and half the height of the associated luma array) or 4:2:2 (in which the two chroma arrays have half the width of the associated luma array) color formats. For 4:2:0 formats, these interpretations match those of the Chroma420SampleLocType variable specified in ISO/IEC 23091-4/ITU-T H.273.

If ci_chroma_sample_position_bottom is present in the bitstream, it is a requirement of bitstream conformance that the value is less than or equal to 5.

ci_aspect_ratio_idc indicates the value of the sample aspect ratio of the coded luma samples. The sample aspect ratio is a quantity that describes how the width of a sample compares to its height.

When ci_aspect_ratio_idc is equal to 255, then the sample aspect ratio is explicitly indicated using the syntax elements ci_sar_width and ci_sar_height.

If ci_aspect_ratio_idc is not equal to 255, it is a requirement of bitstream conformance that ci_aspect_ratio_idc is less than or equal to 16.

ci_sar_width and **ci_sar_height** indicate the horizontal and vertical size of the sample aspect ratio (in the same arbitrary units).

When ci_sar_width is equal to 0 or ci_sar_height is equal to 0, the sample aspect ratio is unspecified in this specification but may be provided through external means.

§ 6.15. Padding OBU semantics

Multiple padding units can be present, each padding with an arbitrary number of bytes. Padding OBUs have no effect on the decoding process.

obu_padding_byte is a padding byte. Padding bytes may have arbitrary values and have no effect on the decoding process.

§ 6.16. Metadata OBU semantics

§ 6.16.1. Metadata unit semantics

Metadata units can be contained in either a metadata OBU or a metadata group OBU.

metadata_unit_remaining_bit can take any value but is reserved for future use by AOMedia.

Decoders conforming to this version of this specification shall ignore the value of **metadata_unit_remaining_bit**.

NOTE: Encoders are recommended to set **metadata_unit_remaining_bit** to zero and to ensure that **remainingMuPayloadBits** is less than 8 (i.e., encoders should only extend to reach byte alignment).

It is a requirement of bitstream conformance that any computed values for **remainingMuPayloadBits** shall not be less than 0.

§ 6.16.2. Metadata short OBU semantics

metadata_is_suffix has the same semantics as in metadata group OBU semantics [§6.16.3 Metadata group OBU semantics](#).

metadata_necessity_idc has the same semantics as in metadata group OBU semantics [§6.16.3 Metadata group OBU semantics](#).

metadata_application_id has the same semantics as in metadata group OBU semantics [§6.16.3 Metadata group OBU semantics](#).

muh_layer_idc has the same semantics as in metadata group OBU semantics [§6.16.3 Metadata group OBU semantics](#).

It is a requirement of bitstream conformance that **muh_layer_idc** is less than 3.

muh_cancel_flag has the same semantics as in metadata group OBU semantics [§6.16.3 Metadata group OBU semantics](#).

muh_persistence_idc has the same semantics as in metadata group OBU semantics [§6.16.3 Metadata group OBU semantics](#).

metadata_type has the same semantics as in metadata group OBU semantics [§6.16.3 Metadata group OBU semantics](#).

NOTE: **muh_priority** is not specified when this short form is used.

NOTE: For an OBU with `obu_type` equal to `OBU_METADATA_SHORT` and with `metadata_type` equal to `METADATA_TYPE_ICC_PROFILE`, `METADATA_TYPE_ITUT_T35`, or `METADATA_TYPE_USER_DATA_UNREGISTERED`, the value of the `metadataPayloadSize` ensures that the `trailing_bits` syntax contains exactly 8 bits. If an encoder wants to pad with additional bytes for these metadata types, it can add such bytes before the `trailing_bits` syntax. The added bytes do not need to be zero.

§ 6.16.3. Metadata group OBU semantics

metadata_is_suffix, when equal to 0 (prefix), indicates that the metadata appears before the frame data within coded frame units. Otherwise, `metadata_is_suffix` equal to 1 (suffix) indicates that the metadata appears after the frame data within coded frame units.

NOTE: Prefix metadata is suitable for signaling information that is known prior to encoding such as presentation time. Suffix metadata is suitable for information that is known after encoding such as a frame hash.

metadata_necessity_idc indicates the essentiality of the metadata OBU and the contained metadata units as follows:

metadata_necessity_idc	Name	Description
0	UNDEFINED	The necessity of the current metadata OBU is undefined.
1	NECESSARY	All metadata units within the metadata OBU are considered necessary for the receiving system.
2	ADVISORY	All metadata units within the metadata OBU are advisory for the receiving system.
3	MIXED	At least one metadata unit is considered necessary, and others may be advisory. The determination is made based on the semantics of each metadata type.

metadata_application_id indicates the application id associated with the current metadata OBU as specified in [Table 6.16](#):

Table 6.16: metadata_application_id values and descriptions

metadata_application_id	Name	Description
0	UNSPECIFIED	Application is undetermined.
1	MOBILE_OR_TV	Metadata is intended for a mobile device (e.g., smartphone) or a TV.
2	MOBILE	Metadata is intended for a mobile device (e.g., smartphone).
3	TV	Metadata is intended for a TV.
4	HMD	Metadata is intended for a Head Mounted Display.
5	WEARABLE	Metadata is intended for a wearable device (e.g., watch).
6-15	Reserved for AOMedia use	Reserved for AOMedia use.
16-31	Externally defined	Application can be determined through external signaling (e.g., within an mp4 file).

metadata_unit_cnt_minus_1 plus 1, specifies the total number of metadata units present in the current metadata_group_obu(). It is a requirement of bitstream conformance that the value of metadata_unit_cnt_minus_1 is less than 16383.

metadata_type indicates the type of metadata as specified in [Table 6.17](#):

Table 6.17: metadata_type values and layer-specific status

metadata_type	Name of metadata_type	Layer-specific
0	Reserved for AOMedia use	-
1	METADATA_TYPE_HDR_CLL	N
2	METADATA_TYPE_HDR_MDCV	N
3	METADATA_TYPE_ITUT_T35	payload-specific
4	METADATA_TYPE_TIMECODE	Y
5	METADATA_TYPE_DECODED_FRAME_HASH	Y
6	METADATA_TYPE_BANDING_HINTS	Y
7	METADATA_TYPE_ICC_PROFILE	N
8	METADATA_TYPE_SCAN_TYPE	N
9	METADATA_TYPE_TEMPORAL_POINT_INFO	Y
10	METADATA_TYPE_USER_DATA_UNREGISTERED	payload-specific
11 and greater	Reserved for AOMedia use	-

The semantics of the column “Layer-specific” and its values are defined in [§ 6.2.2 OBU header semantics](#).

muh_header_size specifies the number of bytes in the metadata unit header.

NOTE: muh_header_size includes muh_header_extension_byte syntax elements but excludes muh_cancel_flag.

muh_cancel_flag when set to 1, indicates that any previously signaled metadata information for a metadata with type equal to muh_metadata_type is cancelled for either the current extended layer if obu_xlayer_id is less than GLOBAL_XLAYER_ID, or for a set of extended layers if obu_xlayer_id is equal to GLOBAL_XLAYER_ID.

muh_layer_idc is used to signal a mode that specifies the layers to which the signaled metadata applies. This value can represent different modes, such as applying the metadata to all layers, applying the metadata to a continuous range of layer values, or applying the metadata to a set of specific layer values. The specific values for the layer_idc are defined as follows:

muh_layer_idc	Name	Description
0	LAYER_UNSPECIFIED	The current signaling does not specify to what layers the metadata applies to. This information can potentially be indicated or determined through external means.
1	LAYER_GLOBAL	The metadata applies to all layers if obu_xlayer_id is equal to GLOBAL_XLAYER_ID. If obu_xlayer_id is less than GLOBAL_XLAYER_ID, layers with matching obu_xlayer_id only.
2	LAYER_CURRENT	The metadata applies to the current layer only as indicated by the specific values for obu_xlayer_id and obu_mlayer_id in OBU header.

3	LAYER_VALUES	The metadata applies to a set of specific layer values, which are explicitly signaled.
4-7	Reserved	Reserved for AOMedia use.

muh_payload_size signals the size of the metadata payload in bytes.

NOTE: This includes the byte alignment bits if those are needed.

muh_persistence_idc is used to signal the mode in which the signaled metadata persists over time. This value can represent different modes, such as global persistence for the entire video sequence, persistence for a group of frames of a certain duration, or persistence for a single frame only.

The specific values for the **muh_persistence_idc** are defined as follows:

muh_persistence_idc	Name	Description
0	GLOBAL_PERSISTENCE	Global persistence for the entire video sequence. When this mode is signaled previously signaled global metadata of this type are overwritten. The cancel flag (muh_cancel_flag) does not do anything to it.
1	BASIC_PERSISTENCE	Persistence until a new metadata unit of the same type is encountered that applies to the layer or the cancel flag (muh_cancel_flag) is encountered.
2	NO_PERSISTENCE	Used only for the current frame.
3	ENHANCED_PERSISTENCE	This one is similar to basic but can allow updates of metadata without full replacement.
4-7	Reserved	Reserved for AOMedia use.

muh_priority is used to indicate the relative importance or urgency of a particular type of metadata. A lower value indicates a higher priority, while a higher value indicates a lower priority.

NOTE: This information can be used by decoders to prioritize the processing of different types of metadata, ensuring that critical or time-sensitive metadata is handled before less important metadata. Furthermore, it can also be beneficial on a system level. For example, in lossy channels, more important information can be protected or re-transmitted more frequently, ensuring that critical or time-sensitive metadata is less likely to be lost or corrupted during transmission.

muh_reserved_zero_2bits must be set to zero and shall be ignored by decoders.

muh_xlayer_map contains a bitmask. The metadata unit is intended for an extended layer *x* if bit *x* of **muh_xlayer_map** is equal to 1.

It is a requirement of bitstream conformance that bit 31 of **muh_xlayer_map** is equal to 0.

muh_mlayer_map contains a bitmask. The metadata unit is intended for an embedded layer *m* if bit *m* of **muh_mlayer_map** is equal to 1.

It is a requirement of bitstream conformance that bit *m* of **muh_mlayer_map** is equal to 0 for *m* less than **obu_mlayer_id**.

NOTE: It is possible that the layers indicated may have been removed because of a selection of an operating point. A decoder will only apply the metadata to the remaining layers according to the selected operating point.

When metadata is indicated as persistent and is specified at embedded layer K and temporal layer T , the metadata applies to other layers according to the following rules:

- **Temporal persistence:** Within embedded layer K , the metadata persists to temporal layer C if $T_{LayerDependencyMap}[K][C][T]$ is equal to 1. If $T_{LayerDependencyMap}[K][C][T]$ is equal to 0, the metadata does not apply to temporal layer C .
- **Multi-layer persistence:** The metadata persists from embedded layer K to embedded layer M (where $M > K$) if the metadata has explicit layer persistence indication and $M_{LayerDependencyMap}[M][K]$ is equal to 1.
- **Combined persistence:** When metadata persists from embedded layer K to embedded layer M , it applies to temporal layer C within embedded layer M if $T_{LayerDependencyMap}[M][C][T]$ is equal to 1.

NOTE: Metadata has explicit layer persistence indication when `muh_layer_idc` is equal to `LAYER_VALUES(3)` and `muh_mlayer_map` has bits set for embedded layers greater than `obu_mlayer_id`.

Decoders shall ignore metadata that does not apply to the current operating point based on these rules.

muh_header_extension_byte, if present, contains additional bytes. Decoders conforming to this version of this specification should ignore the contents.

§ 6.16.4. Metadata ITUT T35 semantics

itu_t_t35_country_code shall be a byte having a value specified as a country code by Annex A of Recommendation ITU-T T.35.

itu_t_t35_country_code_extension_byte shall be a byte having a value specified as a country code by Annex B of Recommendation ITU-T T.35.

itu_t_t35_payload_bytes shall be bytes containing data registered as specified in Recommendation ITU-T T.35.

The ITU-T T.35 terminal provider code and terminal provider oriented code shall be contained in the first one or more bytes of the `itu_t_t35_payload_bytes`, in the format specified by the Administration that issued the terminal provider code. Any remaining bytes in `itu_t_t35_payload_bytes` data shall be data having syntax and semantics as specified by the entity identified by the ITU-T T.35 country code and terminal provider code.

§ 6.16.5. Metadata high dynamic range content light level semantics

This metadata unit identifies upper bounds of the nominal target brightness light level of the associated content.

The values in this metadata unit are defined in relation to samples in a 4:4:4 representation of red, green, and blue color primary intensities in the linear light domain, in units of candelas per square meter. This metadata unit does not itself identify a conversion process from decoded sample values to that representation.

NOTE: Other syntax elements such as `BitDepth`, `color_primaries`, `transfer_characteristics`, and `matrix_coefficients`, when present, can assist in identifying such a conversion process.

Given the red, green, and blue linear-light intensities at a sample location, denoted E_R , E_G , and E_B , the maximum component intensity is computed as $E_{Max} = \text{Max}(E_R, \text{Max}(E_G, E_B))$. The light level at that location is the CIE 1931 luminance corresponding to equal amplitudes of E_{Max} for all three primaries, scaled so that peak white corresponds to the nominal maximum luminance (e.g., 10 000 cd/m² when `transfer_characteristics` corresponds to PQ).

NOTE: Because E_{Max} rather than a direct RGB-to-luminance conversion is used, the CIE 1931 luminance can be less than the indicated light level - for example when E_B is large and E_R , E_G are near zero.

The calculation method for `max_cll` and `max_fall` is defined in [\[CTA-861\]](#), Annex P (Calculation of MaxCLL and MaxFALL).

`metadata_hdr_cll` metadata associated with an embedded layer, when present, shall be indicated at the first coded picture of that embedded layer in the coded video sequence. Any additional `metadata_hdr_cll` metadata units associated with an embedded layer in a coded video sequence shall have the same content. When an embedded layer inherits color information from another layer, the inherited layer's `metadata_hdr_cll` applies unless overridden by a `metadata_hdr_cll` metadata unit present for the inheriting layer.

NOTE: These values are determined from the source content prior to encoding. The light levels of the reconstructed decoded pictures may differ due to quantization and any color space or transfer characteristic conversions applied during the encoding process.

max_cll, when not equal to 0, specifies an upper bound on the maximum light level among all individual samples, in a 4:4:4 representation of red, green, and blue color primary intensities in the linear light domain, across all pictures of the embedded layers of the coded video sequence, in units of cd/m² associated with this metadata unit. When equal to 0, no such upper bound is signaled.

max_fall, when not equal to 0, specifies an upper bound on the maximum frame-average light level across all pictures, in a 4:4:4 representation of red, green, and blue color primary intensities in the linear light domain, of the embedded layers of the coded video sequence, in units of cd/m² associated with this metadata unit. When equal to 0, no such upper bound is signaled.

NOTE: When the visually relevant region does not cover the entire decoded picture (e.g., letterbox content), the frame-average is expected to be computed only over the visually relevant region.

§ 6.16.6. Metadata high dynamic range mastering display color volume semantics

This metadata unit describes the color volume of the mastering display — the color primaries, white point, and luminance range of the display used when grading the associated video content.

NOTE: The semantics of this metadata unit differ from the equivalent metadata in AV1. AV2 uses integer units consistent with SMPTE ST 2086, making the binary encoding identical to other specifications and enabling mastering display metadata to be passed across container boundaries without conversion.

`metadata_hdr_mdcv` metadata associated with an embedded layer, when present, shall be indicated at the first coded picture of that embedded layer in the coded video sequence. Any additional `metadata_hdr_mdcv` metadata units associated with an embedded layer in a coded video sequence shall have the same content. When an embedded layer inherits color information from another layer, the inherited layer's `metadata_hdr_mdcv` applies unless overridden by a `metadata_hdr_mdcv` metadata unit present for the inheriting layer.

primary_chromaticity_x[i] specifies the normalized x chromaticity coordinate of color primary i of the mastering display, as defined by CIE 1931, in integer units of 0.00002. Valid values are in the range 5 to 37000, inclusive. Values outside this range indicate that the coordinate is unknown or unspecified.

primary_chromaticity_y[i] specifies the normalized y chromaticity coordinate of color primary i of the mastering display, as defined by CIE 1931, in integer units of 0.00002. Valid values are in the range 5 to 42000, inclusive. Values outside this range indicate that the coordinate is unknown or unspecified.

For mastering displays with red, green, and blue primaries, it is suggested that i = 0 corresponds to the green primary, i = 1 to the blue primary, and i = 2 to the red primary.

NOTE: SMPTE ST 2086 expresses chromaticity coordinates to four decimal places, which corresponds to multiples of 5 in this encoding. ANSI/CTA-861-G signals an unknown white point chromaticity using (x, y) = (0, 0).

white_point_chromaticity_x specifies the normalized x chromaticity coordinate of the mastering display white point, as defined by CIE 1931, in integer units of 0.00002. Valid values are in the range 5 to 37000, inclusive. Values outside this range indicate that the coordinate is unknown or unspecified.

white_point_chromaticity_y specifies the normalized y chromaticity coordinate of the mastering display white point, as defined by CIE 1931, in integer units of 0.00002. Valid values are in the range 5 to 42000, inclusive. Values outside this range indicate that the coordinate is unknown or unspecified.

luminance_max specifies the nominal maximum display luminance of the mastering display in units of 0.0001 cd/m². Valid values are in the range 50000 to 100000000, inclusive. Values outside this range indicate that the maximum luminance is unknown or unspecified.

NOTE: SMPTE ST 2086 expresses maximum luminance in whole cd/m², which corresponds to multiples of 10000 in this encoding. ANSI/CTA-861-G uses the value 0 to signal that the maximum display luminance is unknown.

luminance_min specifies the nominal minimum display luminance of the mastering display in units of 0.0001 cd/m². Valid values are in the range 1 to 50000, inclusive. Values outside this range indicate that the minimum luminance is unknown or unspecified. It is a requirement of bitstream conformance that when `luminance_max` is equal to 50000, `luminance_min` shall not be equal to 50000.

NOTE: SMPTE ST 2086 expresses minimum luminance in units of 0.0001 cd/m², consistent with this encoding. ANSI/CTA-861-G uses the value 0 to signal that the minimum display luminance is unknown.

At the minimum luminance level, the mastering display white point chromaticity applies.

§ 6.16.7. Metadata timecode semantics

counting_type specifies the method of dropping values of the `n_frames` syntax element as specified in the table below. `counting_type` should be the same for all pictures in the coded video sequence.

counting_type	Meaning
0	no dropping of <code>n_frames</code> count values and no use of <code>time_offset_value</code>
1	no dropping of <code>n_frames</code> count values
2	dropping of individual zero values of <code>n_frames</code> count
3	dropping of individual values of <code>n_frames</code> count equal to <code>maxFps - 1</code>
4	dropping of the two lowest (value 0 and 1) <code>n_frames</code> counts when <code>seconds_value</code> is equal to 0 and <code>minutes_value</code> is not an integer multiple of 10
5	dropping of unspecified individual <code>n_frames</code> count values
6	dropping of unspecified numbers of unspecified <code>n_frames</code> count values
7..31	reserved

full_timestamp_flag equal to 1 indicates that the `seconds_value`, `minutes_value`, `hours_value` syntax elements will be present. `full_timestamp_flag` equal to 0 indicates that there are flags to control the presence of these syntax elements.

When `ci_timing_info_present_flag` is equal to 1, the contents of the clock timestamp indicate a time of origin, capture, or ideal display. This indicated time is computed as follows:

```

if ( equal_picture_interval ) {
    TicksPerPicture = ( num_ticks_per_picture_minus_1 + 1 ) * num_units_in_display_tick
} else {
    TicksPerPicture = num_units_in_display_tick
}
ss = ( ( hours_value * 60 + minutes_value ) * 60 + seconds_value )
clockTimestamp = ss * time_scale +
    n_frames * TicksPerPicture + time_offset_value

```

`clockTimestamp` is in units of clock ticks of a clock with clock frequency equal to `time_scale` Hz, relative to some unspecified point in time for which `clockTimestamp` would be equal to 0.

discontinuity_flag equal to 0 indicates that the difference between the current value of `clockTimestamp` and the value of `clockTimestamp` computed from the previous set of timestamp syntax elements in output order can be interpreted as the time difference between the times of origin or capture of the associated frames or fields. `discontinuity_flag` equal to 1 indicates that the difference between the current value of `clockTimestamp` and the value of `clockTimestamp` computed from the previous set of clock timestamp syntax elements in output order should not be interpreted as the time difference between the times of origin or capture of the associated frames or fields.

When `ci_timing_info_present_flag` is equal to 1 and `discontinuity_flag` is equal to 0, the value of `clockTimestamp` shall be greater than or equal to the value of `clockTimestamp` for the previous set of clock timestamp syntax elements in output order.

cnt_dropped_flag specifies the skipping of one or more values of `n_frames` using the counting method specified by `counting_type`.

n_frames is used to compute `clockTimestamp`. When `ci_timing_info_present_flag` is equal to 1, `n_frames` shall be less than `maxPicPerSecond`, where `maxPicPerSecond` is specified by `maxPicPerSecond = ceil( time_scale / TicksPerPicture )`.

seconds_flag equal to 1 specifies that `seconds_value` and `minutes_flag` are present when `full_timestamp_flag` is equal to 0. `seconds_flag` equal to 0 specifies that `seconds_value` and `minutes_flag` are not present.

seconds_value is used to compute `clockTimestamp` and shall be in the range of 0 to 59. When `seconds_value` is not present, its value is inferred to be equal to the value of `seconds_value` for the previous set of clock timestamp syntax elements in decoding order, and it is required that such a previous `seconds_value` shall have been present.

minutes_flag equal to 1 specifies that `minutes_value` and `hours_flag` are present when `full_timestamp_flag` is equal to 0 and `seconds_flag` is equal to 1. `minutes_flag` equal to 0 specifies that `minutes_value` and `hours_flag` are not present.

minutes_value specifies the value of mm used to compute `clockTimestamp` and shall be in the range of 0 to 59, inclusive. When `minutes_value` is not present, its value is inferred to be equal to the value of `minutes_value` for the previous set of clock timestamp syntax elements in decoding order, and it is required that such a previous `minutes_value` shall have been present.

hours_flag equal to 1 specifies that `hours_value` is present when `full_timestamp_flag` is equal to 0 and `seconds_flag` is equal to 1 and `minutes_flag` is equal to 1.

hours_value is used to compute `clockTimestamp` and shall be in the range of 0 to 23, inclusive. When `hours_value` is not present, its value is inferred to be equal to the value of `hours_value` for the previous set of clock timestamp syntax elements in decoding order, and it is required that such a previous `hours_value` shall have been present.

time_offset_length greater than 0 specifies the length in bits of the `time_offset_value` syntax element. `time_offset_length` equal to 0 specifies that the `time_offset_value` syntax element is not present. `time_offset_length` should be the same for all frames in the coded video sequence.

time_offset_value is used to compute `clockTimestamp`. The number of bits used to represent `time_offset_value` is equal to `time_offset_length`. When `time_offset_value` is not present, its value is inferred to be equal to 0.

§ 6.16.8. Metadata banding hints semantics

When present, the banding metadata applies to a frame or multiple frames. It indicates hints about the presence of banding and its characteristics. A decoder may optionally choose to utilize this information and no normative debanding processing associated with this metadata is required for decoder conformance.

coding_banding_present_flag equal to 1 indicates banding due to compression is present in the current frame. **coding_banding_present_flag** equal to 0 indicates banding due to compression is not present in the current frame.

source_banding_present_flag equal to 1 indicates that source content that may be identified as banding by a debanding algorithm is present in the current frame. **source_banding_present_flag** equal to 0 indicates that no specific source content that may be identified as banding has been detected in the current frame.

NOTE: This parameter indicates that banding-like patterns are present in the source that might be detected as banding on the decoded output. The hint aims to reduce false positives and aid in better preserving source information. However, **source_banding_present_flag** equal to 0 does not guarantee the absence of content that an algorithm may mistakenly identify as banding.

banding_hints_flag equal to 1 indicates that additional information hints about the banding characteristic are present in this metadata message. **banding_hints_flag** equal to 0 indicates that additional information hints about the banding characteristic are not present in this metadata message.

three_color_components_flag equal to 1 indicates that the banding related additional information is signaled for three color components. **three_color_components_flag** equal to 0 indicates that the banding related additional information is signaled only for the color component 0.

banding_in_component_present_flag equal to 1 indicates banding in the color component plane is present. **banding_in_component_present_flag** equal to 0 indicates banding in the color component plane is not present.

max_band_width_minus_4 plus 4 specifies the typical maximum banding width in color component plane in the current frame in samples of component plane.

max_band_step_minus_1 plus 1 specifies the typical maximum difference between two consecutive bands in color component plane in the current frame.

band_units_information_present_flag equal to 1 indicates that additional information hints per band unit are present. **band_units_information_present_flag** equal to 0 indicates that no additional information on banding presence for band units is present.

num_band_units_rows_minus_1 plus 1 specifies the number of band units rows.

num_band_units_cols_minus_1 plus 1 specifies the number of band units columns.

varying_size_band_units_flag equal to 1 indicates that band units of varying size are used with unit sizes specified by syntax elements **vert_size_in_band_blocks_minus_1[r]** and **horz_size_in_band_blocks_minus_1[c]**. **varying_size_band_units_flag** equal to 0 indicates that band units of uniform size are used.

band_block_in_luma_samples specifies the horizontal and vertical size of the band block in samples of component 0 as $16 \ll \text{band_block_in_luma_samples}$.

vert_size_in_band_blocks_minus_1 plus 1 specifies the size of the r-th band unit row as $\text{bandBlockInSamples} * (\text{vert_size_in_band_blocks_minus_1}[r] + 1)$ in component 0 samples when **varying_size_band_units_flag** is equal to 1.

horz_size_in_band_blocks_minus_1 plus 1 specifies the size of the c-th band unit column as $\text{bandBlockInSamples} * (\text{horz_size_in_band_blocks_minus_1}[c] + 1)$ in component 0 samples when **varying_size_band_units_flag** is equal to 1.

Band units boundaries are aligned across components, taking into account possible component subsampling.

banding_in_band_unit_present_flag equal to 1 indicates banding is present in band unit in row r, column c. **banding_in_band_unit_present_flag[r][c]** equal to 0 indicates that banding is not present in band unit in row r, column c.

§ 6.16.9. Metadata ICC profile semantics

icc_profile_data_payload_bytes shall be bytes containing data corresponding to a profile from the International Color Consortium.

The variable **ICCmajorVer** is set equal to **icc_profile_data_payload_bytes[8]** and the variable **ICCminorVer** is set equal to **icc_profile_data_payload_bytes[9] >> 4**.

icc_profile_data_payload_bytes contains data with syntax and semantics specified according to the interpretation of **ICCmajorVer** and **ICCminorVer** as follows:

ICCmajorVer	ICCminorVer	Interpretation
4	2	Major profile 4 and minor profile 2 version as specified in ISO 15076-1
4	3	Major profile 4 and minor profile 3 version as specified in ISO 15076-1
4	4	Major profile 4 and minor profile 4 version as specified in ISO 15076-1
5	0	Major profile 5 and minor profile 0 version as specified in ISO 20677

Values of **ICCmajorVer** and **ICCminorVer** that are not listed are unspecified or specified by other means.

§ 6.16.10. Metadata scan type semantics

This metadata allows decoded frames to be interpreted as either progressive or interlaced content.

These values have no normative effect on the decoding process which is still frame based.

The prefix **mps** stands for metadata picture structure.

mps_pic_struct_type indicates whether a picture should be displayed as a frame or as one or more fields and, for the display of frames when **equal_picture_interval** is equal to 1, whether such frame should be repeated or not when output on certain devices.

The interpretation of **mps_pic_struct_type** is specified in [Table 6.18](#):

Table 6.18: mps_pic_struct_type values and picture output interpretations

Value	Indicated picture output	Elemental Units	Restrictions
0	Frame	1	ci_scan_type_idc shall be equal to 1
1	Top field	1	ci_scan_type_idc shall be equal to 2
2	Bottom field	1	ci_scan_type_idc shall be equal to 2

3	Top field, bottom field in that order	2	ci_scan_type_idc shall be equal to 3
4	Bottom field, top field in that order	2	ci_scan_type_idc shall be equal to 3
5	Top field, bottom field, top field repeated, in that order	3	ci_scan_type_idc shall be equal to 3
6	Bottom field, top field, bottom field repeated, in that order	3	ci_scan_type_idc shall be equal to 3
7	Frame doubling	2	ci_scan_type_idc shall be equal to 1 and equal_picture_interval shall be equal to 1
8	Frame tripling	3	ci_scan_type_idc shall be equal to 1 and equal_picture_interval shall be equal to 1
9	Top field paired with previous bottom field in output order	1	ci_scan_type_idc shall be equal to 2
10	Bottom field paired with previous top field in output order	1	ci_scan_type_idc shall be equal to 2
11	Top field paired with next bottom field in output order	1	ci_scan_type_idc shall be equal to 2
12	Bottom field paired with next top field in output order	1	ci_scan_type_idc shall be equal to 2

Values of mps_pic_struct_type above 12 are reserved for future use by AOMedia and shall not be present in bitstreams conforming to this specification.

Decoders shall ignore reserved values of mps_pic_struct_type.

It is a requirement of bitstream conformance that when mps_pic_struct_type is present that only one of the following conditions, for all pictures in the current CVS, is true: – The value of mps_pic_struct_type is equal to 0, 7 or 8. – The value of mps_pic_struct_type is equal to 1, 2, 9, 10, 11 or 12. – The value of mps_pic_struct_type is equal to 3, 4, 5 or 6.

mps_source_scan_type_idc specifies the scan type with the same semantics as for ci_scan_type_idc.

mps_duplicate_flag indicates whether the current picture should be indicated as a duplicate of a previous picture in output order. When mps_duplicate_flag is equal to 1 the current picture is indicated to be a duplicate of the previous picture. When mps_duplicate_flag is equal to 0 the current picture is not indicated to be a duplicate of the previous picture.

§ 6.16.11. Metadata temporal point info semantics

It is a requirement of bitstream conformance that metadata_type equal to METADATA_TYPE_TEMPORAL_POINT_INFO shall only appear in an OBU with obu_type equal to OBU_METADATA_SHORT.

NOTE: A metadata_type of METADATA_TYPE_TEMPORAL_POINT_INFO is only allowed in OBUs with obu_type equal to OBU_METADATA_SHORT to make parsing simpler for application layers.

frame_presentation_time specifies the presentation time of the frame in clock ticks DispCT counted from the presentation time of the previous random access point for the operating point that is being decoded if the current frame is a leading frame or is associated with a random access point. It specifies the presentation time of the frame in clock ticks DispCT counted from the presentation time of the most

recent random access point if the current frame is not a leading frame and is not associated with a random access point.

§ 6.16.12. Metadata user data unregistered semantics

uuid_iso_iec_11578 specifies a UUID value that conforms to the procedures in Annex A of ISO/IEC 11578:1996.

user_data_payload_byte specifies a byte of data whose structure and meaning are determined by the UUID. This standard does not specify or restrict the format or interpretation of the user_data_payload_byte payload bytes.

§ 6.16.13. Metadata decoded frame hash semantics

This metadata contains hash values that are calculated for the output frames. Generation of hash values should use the procedure below to ensure the correct interpretation of those values.

Output frames are prepared by the output process specified in [§ 7.21.1 Output process](#).

Let bitDepth, w, h, subX, subY be the values of the corresponding local variables at the end of the output process.

The hash is computed on the cropped frame dimensions as specified by w and h.

If has_grain is equal to 0, let decodedSamples[0]/decodedSamples[1]/decodedSamples[2] be the values of OutY/OutU/OutV generated by the intermediate output preparation process specified in [§ 7.21.2 Intermediate output preparation process](#).

If has_grain is equal to 1, let decodedSamples[0]/decodedSamples[1]/decodedSamples[2] be the values of OutY/OutU/OutV at the end of the output process.

NOTE: It is legal to set has_grain equal to 1 even if the sequence is not using film grain.

Prior to computing the hash, decoded sample values are converted to byte arrays as follows.

```
numPlanes = is_monochrome ? 1 : 3
for (planeIdx = 0; planeIdx < numPlanes; planeIdx++) {
    if (planeIdx == 0) {
        planeWidth = w
        planeHeight = h
    } else {
        planeWidth = (w + subX) >> subX
        planeHeight = (h + subY) >> subY
    }
    byteIdx = 0
    for (row = 0; row < planeHeight; row++) {
        for (col = 0; col < planeWidth; col++) {
            sample = decodedSamples[planeIdx][row][col]
            planeData[planeIdx][byteIdx++] = sample & 0xFF
            if ( bitDepth > 8 ) {
                planeData[planeIdx][byteIdx++] = sample >> 8
            }
        }
    }
    planeDataLength[planeIdx] = byteIdx
}
```

Samples are processed in raster scan order (left to right, top to bottom) within each plane. 8-bit samples (bitDepth equal to 8) are written as a single byte. Samples with bitDepth greater than 8 are written as two bytes in little-endian order (LSB first, then MSB). For monochrome frames (is_monochrome equal to 1), only the Y plane (planeIdx equal to 0) is processed.

hash_type specifies the hash algorithm used to compute the frame hash.

When hash_type equals 0, the hash is computed using MD5 as specified by [\[RFC1321\]](#). The MD5 computation is performed as follows:

When per_plane equals 1 (separate hash per plane):

```
for (planeIdx = 0; planeIdx < numPlanes; planeIdx++) {
    MD5Init(context)
    MD5Update(context, planeData[planeIdx], planeDataLength[planeIdx])
    MD5Final(plane_hash[planeIdx], context)
}
```

When per_plane equals 0 (single hash for all planes):

```
MD5Init(context)
for (planeIdx = 0; planeIdx < numPlanes; planeIdx++) {
    MD5Update(context, planeData[planeIdx], planeDataLength[planeIdx])
}
MD5Final(frame_hash, context)
```

where MD5Init, MD5Update, and MD5Final are the functions defined in [\[RFC1321\]](#).

All other values of hash_type are [reserved](#) for future use by AOMedia.

per_plane equal to 1 specifies that the hash is computed separately for each plane. When per_plane is equal to 0, a single hash is computed for all planes combined.

has_grain equal to 1 specifies that the hash is computed on the decoded frame after film grain synthesis has been applied according to the film grain synthesis process specified in [§ 7.21.7 Film grain synthesis process](#). When has_grain is equal to 0, the hash is computed on the raw decoded frame.

is_monochrome equal to 1 specifies that the frame has a single plane (monochrome). When is_monochrome is equal to 0, the frame has 3 planes. This field is only used when per_plane is equal to 1 to determine the number of plane_hash array elements to read.

reserved shall be set to 0 and ignored by decoders. This bit is [reserved](#) for future use by AOMedia.

plane_hash[planeIdx] is an array containing 16 bytes (128 bits) of hash data for each plane. Each plane_hash[planeIdx] element is computed over the corresponding plane's samples in raster scan order using the algorithm specified by hash_type. This array is present when per_plane is equal to 1. When is_monochrome is equal to 1, only plane_hash[0] (Y plane) is present. When is_monochrome is equal to 0, three elements are present: plane_hash[0] for Y, plane_hash[1] for U, and plane_hash[2] for V.

frame_hash contains 16 bytes (128 bits) of hash data for the entire frame. When multiple planes are present, the hash is computed over all planes' samples in plane order (Y, then U, then V) using the algorithm specified by hash_type. This syntax element is present when per_plane is equal to 0.

§ 6.17. Frame header OBU semantics

§ 6.17.1. General frame header semantics

It is a requirement of bitstream conformance that a sequence header OBU has been received before a frame header.

If `isFirst` is equal to 1, it is a requirement of bitstream conformance that `SeenFrameHeader` is equal to 0.

If `isFirst` is equal to 0, it is a requirement of bitstream conformance that `SeenFrameHeader` is equal to 1.

frame_header_copy is a syntax structure that contains an identical copy of the bits sent in the `frame_header` for the first tile group.

NOTE: When a frame header is present for the second tile group onwards, a decoder can choose to either read the syntax elements or to simply skip over the bits.

header_bit[i] contains a copy of a bit from the `frame_header` syntax structure sent with the first tile group in the frame.

It is a requirement of bitstream conformance that `header_bit[ i ]` is equal to the value of the bit at offset `i` from the start of the `frame_header` structure sent with the first tile group.

NOTE: The contents of `frame_header` are copied bit for bit but this does not include the bits sent before `frame_header`. This means that the duplicate copies have a different bit alignment within bytes when compared to the original version.

TileNum is a variable giving the index (zero-based) of the current tile.

decode_frame_wrapup is a function call that indicates that the decode frame wrapup process specified in § 7.2 [Decode frame wrapup process](#) is invoked.

§ 6.17.2. Frame header info semantics

bridge_frame_ref_idx specifies which reference frame is used in a [Bridge frame](#).

NOTE: The [Bridge frame](#) represents the same temporal instant as its reference frame at a different resolution. As such, it inherits the same order hint.

cur_mfh_id specifies which multi-frame header to use.

If `cur_mfh_id` is greater than 0, it is a requirement of bitstream conformance that a multi-frame header OBU with `mfh_id_minus_1` equal to `cur_mfh_id - 1` is present in the bitstream at some point before the syntax element `cur_mfh_id`, or is available through external means.

seq_header_id_in_frame_header specifies which sequence header is associated with this frame.

load_sequence_header(id) specifies that all the syntax elements and variables saved by a previous call to `save_sequence_header` are loaded from the area of memory indexed by `id`.

It is a requirement of bitstream conformance that `id` corresponds to an area of memory that was saved.

After the sequence header is loaded, if `cur_mfh_id` is greater than 0, it is a requirement of bitstream conformance that all the following are true:

- `mfh_frame_width_minus_1[ cur_mfh_id ]` is less than or equal to `max_frame_width_minus_1`.
- `mfh_frame_height_minus_1[ cur_mfh_id ]` is less than or equal to `max_frame_height_minus_1`.
- `MLayerDependencyMap[ obu_mlayer_id ][ MfhMLayerId[ cur_mfh_id ] ]` is equal to 1.
- `TLayerDependencyMap[ obu_mlayer_id ][ obu_tlayer_id ][ MfhTLayerId[ cur_mfh_id ] ]` is equal to 1.

FirstPictureInTU is a variable that specifies if this is the first frame unit in a coded extended layer unit in a temporal unit.

startCVS specifies if this is the start of a new coded video sequence.

activate_layer_configuration_record(id) specifies that the layer configuration records corresponding to the given id are activated.

A `lcr_local_info` syntax structure is activated if `lcr_local_id[ obu_xlayer_id ]` is equal to `id`. Otherwise (if there is no `lcr_local_info` syntax structure with `lcr_local_id[ obu_xlayer_id ]` equal to `id`), a `lcr_global_info` syntax structure is activated if the value of `lcr_global_config_record_id` is equal to `id`.

ShowExistingFrame equal to 1 indicates the frame indexed by `frame_to_show_map_idx` is to be output; **ShowExistingFrame** equal to 0 indicates that further processing is required.

frame_to_show_map_idx specifies the frame to be output. It is only available if **ShowExistingFrame** is 1.

derive_sef_order_hint specifies how the order hint for the show existing frame is derived.

`derive_sef_order_hint` equal to 1 specifies that the order hint is derived from the reference frame.

`derive_sef_order_hint` equal to 0 specifies that the order hint is explicitly signaled via the syntax element `sef_order_hint`.

If `derive_sef_order_hint` is equal to 1, it is a requirement of bitstream conformance that all of the following are true:

- the reference frame at slot `frame_to_show_map_idx` has not already been shown.
- `RefImplicitOutputFrame[ frame_to_show_map_idx ]` is equal to 0.
- `RefImmediateOutputFrame[ frame_to_show_map_idx ]` is equal to 0.

sef_order_hint is used to compute `OrderHint`.

FrameType specifies the type of the frame:

FrameType	Name of FrameType
0	KEY_FRAME
1	INTER_FRAME
2	INTRA_ONLY_FRAME
3	SWITCH_FRAME

restricted_prediction_switch equal to 1 specifies that all available reference frames will be marked as restricted.

NOTE: This allows future frames to use sample values from both the switch frame and other reference frames. However, the other reference frames are marked as restricted to indicate that only the sample values can be used, and not any of the other information associated with a reference frame. This is needed because switch frames switch between bitstreams so the other information is not consistent and cannot be used for parsing syntax elements.

frame_is_inter equal to 1 specifies that the frame is an inter frame and can use inter prediction. **frame_is_inter** equal to 0 specifies that the frame is an intra frame and shall use only intra prediction.

long_term_id_plus_1 minus 1 specifies a long term id number for the current frame.

num_key_ref_frames specifies the number of **ref_long_term_id** syntax elements to be read.

ref_long_term_id[i] specifies a value of long term id for a reference frame. It is a requirement of bitstream conformance that the value of **ref_long_term_id[i]** shall not be equal to $(1 \ll \text{long_term_frame_id_bits}) - 1$.

NOTE: For RAS frames, the **ref_long_term_id** is used to restrict the reference frames allowed to just the long term reference frames with matching long term ids. Not all long term reference frames need to be mentioned in this list, but only the mentioned ones can be used.

NOTE: It is legal for the RAS frame to use multiple long term reference frames that share the same value of long term id.

NOTE: It is recommended (but not a bitstream constraint), that the **ref_long_term_id** array does not contain duplicates. Duplicate entries have no effect on the decoding process - this note is included to ensure that decoders do not assume the values in **ref_long_term_id** are unique.

immediate_output_frame equal to 1 specifies that this frame shall be immediately queued for output once decoded. This frame may also be additionally output using [SEF](#) OBUs. **immediate_output_frame** equal to 0 specifies that this frame should not be immediately queued for output and that the output of this frame depends on additional syntax elements in the bitstream.

If **still_picture** is equal to 1, it is a requirement of bitstream conformance that **FrameType** is equal to **KEY_FRAME** and **immediate_output_frame** is equal to 1.

output_frame_buffers(i) is a function call that indicates that the output frame buffers process specified in [§ 7.21.6 Output frame buffers process](#) is invoked with **i** as input.

implicit_output_frame equal to 1 specifies that the frame will be output by the output frame buffers process specified in [§ 7.21.6 Output frame buffers process](#). This frame can also be additionally output using [SEF](#) OBUs. **implicit_output_frame** equal to 0 specifies that the frame is not output using the output frame buffers process but can be output using [SEF](#) OBUs. When not present, the value of **implicit_output_frame** is equal to 0.

NOTE: Due to the bitstream constraints in AV2, an OLK frame is required to be an implicit output frame by itself, or be present together with another output [Regular frame](#) in the same coded extended layer unit that only depends on the OLK frame. Consequently, when `monotonic_output_order_flag` is equal to 1, the temporal unit containing the OLK will result in a frame that is output before any leading frames. It is not legal to use an `obu_type` that marks this as a leading frame. This may result in the [Regular frame](#) being shown as the first frame before the OLK at an open random access point, potentially with skipped leading frames (and a gap in display time) between them.

frame_size_override_flag equal to 0 specifies that the frame size is equal to the size in the sequence header. `frame_size_override_flag` equal to 1 specifies that the frame size will either be specified as the size of one of the reference frames, or computed from the `frame_width_minus_1` and `frame_height_minus_1` syntax elements.

order_hint is used to compute `OrderHint`.

OrderHintLsbs specifies `OrderHintBits` least significant bits of the expected output order for this frame.

OrderHint specifies the expected output order for this frame.

NOTE: There is no requirement that `OrderHint` should reflect the true output order. As a guideline, the motion vector prediction is expected to be more accurate if the true output order is used for frames that will be shown later. If a frame is never to be shown (e.g., it has been constructed as an average of several frames for reference purposes), the encoder is free to choose whichever value of `OrderHint` will give the best compression.

signal_primary_ref_frame specifies that the `primary_ref_frame` syntax element is present.

disable_cross_frame_cdf_init equal to 1 specifies that the CDF values are set to default values instead of being taken from a reference frame. `disable_cross_frame_cdf_init` equal to 0 specifies that the CDF values can be taken from another reference frame (depending on the value of other syntax elements).

NOTE: The intention of setting `disable_cross_frame_cdf_init` equal to 1 is to allow frames to be arithmetically decoded in parallel.

primary_ref_frame specifies the reference frame which contains the CDF values and other state that are loaded at the start of the frame.

It is a requirement of bitstream conformance that when `primary_ref_frame` is present in the bitstream `primary_ref_frame` is either equal to `PRIMARY_REF_NONE`, or `primary_ref_frame` is less than `NumTotalRefs`.

NOTE: `NumTotalRefs` will be computed later in the decode process.

If `primary_ref_frame` is not equal to `PRIMARY_REF_NONE`, it is a requirement of bitstream conformance that `OrderHints[ primary_ref_frame ]` is not equal to `RESTRICTED_OH`.

change_drl equal to 1 indicates that `max_drl_bits_minus_1` is changed from the value in the sequence header.

max_drl_bits_minus_1 plus 1 specifies the maximum number of times the `drl_mode` syntax element is read within `read_drl_idx`.

flush_implicit_output_frames() is a function call that indicates that the flush implicit output frames process specified in [§ 7.21.5 Flush implicit output frames process](#) is invoked.

bridge_frame_overwrite_flag equal to 1 specifies that the syntax element `refresh_frame_flags` is present. `bridge_frame_overwrite_flag` equal to 0 specifies that `refresh_frame_flags` is not present and is inferred to be equal to $1 \ll \text{bridge_frame_ref_idx}$.

has_refresh_frame_flags equal to 1 specifies that the syntax element `frame_to_refresh` is present. `has_refresh_frame_flags` equal to 0 specifies that the syntax element `frame_to_refresh` is not present and that `refresh_frame_flags` is inferred equal to 0.

frame_to_refresh specifies which reference frame slot will be updated with the current frame after it is decoded.

It is a requirement of bitstream conformance that `frame_to_refresh` is less than `NumRefFrames`.

refresh_frame_flags contains a bitmask that specifies which reference frame slots will be updated with the current frame after it is decoded.

If `FrameType` is equal to `INTRA_ONLY_FRAME` and `NumRefFrames` is greater than 1, it is a requirement of bitstream conformance that `refresh_frame_flags` is not equal to $(1 \ll \text{NumRefFrames}) - 1$.

NOTE: This restriction encourages encoders to correctly label random access points (by forcing `FrameType` to be equal to `KEY_FRAME` when an [intra frame](#) is used to reset the decoding process).

If `IsRegular` is equal to 0 (i.e., this is a leading frame), it is a requirement of bitstream conformance that `refresh_frame_flags & 01kRefresh[ i ]` is equal to 0 for all $i = 0.. \text{MAX_NUM_MLAYERS} - 1$.

NOTE: This restriction forbids leading frames from overwriting frames that will be used by regular frames. This is needed to allow random access decoding to operate correctly.

See [§ 7.23 Reference frame update process](#) for details of the frame update process.

If `immediate_output_frame` is equal to 0, it is a requirement of bitstream conformance that the value of `refresh_frame_flags` is not equal to 0.

NOTE: This restriction also applies if the value of `refresh_frame_flags` is inferred from other syntax elements.

If `obu_type` is equal to `OBU_RAS_FRAME`, `refresh_frame_flags` must be set to refresh all short term frames that are present in the current embedded layer or any layer that depends on the current embedded layer (long term frames may or may not be refreshed).

frame_explicit_ref_frame_map equal to 1 specifies that `num_total_refs` is present in this frame to override the default number of reference frames. `frame_explicit_ref_frame_map` equal to 0 specifies that `num_total_refs` is not present and the default number of reference frames is used.

num_total_refs allows the number of references for this frame to be adjusted from the default values.

If num_total_refs is present, it is a requirement of bitstream conformance that num_total_refs is less than or equal to ActiveNumRefFrames.

use_bru equal to 1 specifies that this frame does a backwards reference update.

bru_ref specifies which reference is updated.

bru_inactive equal to 1 specifies that the whole frame is inactive.

If use_bru is equal to 1, it is a requirement of bitstream conformance that all the following are true:

- OrderHint is greater than or equal to RefOrderHint[i] for i in the range 0..NumRefFrames-1 where RefValid[i] is equal to 1,
- immediate_output_frame is equal to 1,
- bru_ref is less than NumTotalRefs,
- RefOrderHint[ref_frame_idx[bru_ref]] is not equal to RESTRICTED_OH,
- RefFrameWidth[ref_frame_idx[bru_ref]] is equal to FrameWidth,
- RefFrameHeight[ref_frame_idx[bru_ref]] is equal to FrameHeight,
- The value of refresh_frame_flags & (1 << ref_frame_idx[bru_ref]) must be non-zero.

get_ref_frames is a function call that indicates the conceptual point where the default ref_frame_idx values are prepared. When this function is called, the get ref frames process specified in [§ 7.7 Get ref frames process](#) is invoked.

get_past_future_cur_ref_lists is a function call that indicates the get past future cur ref lists process process specified in [§ 7.8 Get past future cur ref lists process](#) is invoked.

ref_frame_idx[i] specifies which reference frames are used by inter frames. It is a requirement of bitstream conformance that RefValid[ref_frame_idx[i]] is equal to 1, and that the selected reference frames match the current frame in bit depth, profile, chroma subsampling, and color space.

NOTE: Syntax elements indicate a reference (an integer between 0 and 6). These references are looked up in the ref_frame_idx array to find the reference frame which is to be used during inter prediction. There is no requirement that the values in ref_frame_idx are distinct.

If obu_type is equal to OBU_RAS_FRAME, it is a requirement of bitstream conformance that long_term_id_in_use(RefLongTermId[ref_frame_idx[i]]) is equal to 1.

It is a requirement of bitstream conformance that MLayerDependencyMap[obu_mlayer_id] [RefMLayerId[ref_frame_idx[i]]] is equal to 1.

It is a requirement of bitstream conformance that TLayerDependencyMap[obu_tlayer_id] [obu_tlayer_id] [RefTLayerId[ref_frame_idx[i]]] is equal to 1.

If use_bru is equal to 1, it is a requirement of bitstream conformance that the RefCounter[ref_frame_idx[bru_ref]] is not the same as RefCounter[ref_frame_idx[i]] for any value of i not equal to bru_ref in the range 0..NumTotalRefs-1.

NOTE: This constraint means that it is not legal to store a decoded frame into two reference frames via the refresh_frame_flags mechanism, and then only update one of the reference frames via a backwards reference update. This means an implementation of a decoder can keep a single copy of each decoded frame.

Once the frame size has been determined, it is a requirement of bitstream conformance that all the following conditions are satisfied for $i=0..NumTotalRefs-1$:

- $2 * FrameWidth \geq RefFrameWidth[ref_frame_idx[i]]$
- $2 * FrameHeight \geq RefFrameHeight[ref_frame_idx[i]]$
- $FrameWidth \leq 16 * RefFrameWidth[ref_frame_idx[i]]$
- $FrameHeight \leq 16 * RefFrameHeight[ref_frame_idx[i]]$

use_qtr_precision_mv equal to 1 specifies that motion vectors are specified to quarter pel precision.

allow_high_precision_mv equal to 0 specifies that motion vectors are specified to half pel precision; **allow_high_precision_mv** equal to 1 specifies that motion vectors are specified to eighth pel precision.

FrameMvPrecision specifies the default precision used for specifying motion vectors as specified in [Table 6.19](#):

Table 6.19: FrameMvPrecision values and names

FrameMvPrecision	Name of FrameMvPrecision
0	MV_PRECISION_EIGHT_PEL
1	MV_PRECISION_FOUR_PEL
2	MV_PRECISION_TWO_PEL
3	MV_PRECISION_ONE_PEL
4	MV_PRECISION_HALF_PEL
5	MV_PRECISION_QUARTER_PEL
6	MV_PRECISION_EIGHTH_PEL
7	NUM_MV_PRECISIONS

frame_enabled_motion_modes specifies which motion modes are allowed in this frame.

use_ref_frame_mvs equal to 1 specifies that motion vector information from a previous frame can be used when decoding the current frame. **use_ref_frame_mvs** equal to 0 specifies that this information will not be used.

tmvp_sample_step_minus_1 plus 1 specifies the step used during temporal motion vector prediction. A higher step means that motion vectors are projected at fewer locations and the motion field is interpolated at the locations that have been stepped over.

allow_df_sub_pu equal to 1 specifies that the deblocking filter filters subblock edges within prediction units. **allow_df_sub_pu** equal to 0 specifies that the deblocking filter does not filter subblock edges.

TipFrameMode specifies how [TIP](#) frames are generated and used as specified in [Table 6.20](#):

Table 6.20: TipFrameMode values and names

TipFrameMode	Name of TipFrameMode
0	TIP_FRAME_DISABLED
1	TIP_FRAME_AS_REF
2	TIP_FRAME_AS_OUTPUT

NOTE: TIP_FRAME_DISABLED means no [TIP](#) will be used. TIP_FRAME_AS_REF means individual blocks can be coded as [TIP](#) blocks. TIP_FRAME_AS_OUTPUT means that the whole frame is automatically generated from [TIP](#) blocks.

tip_frame_mode equal to 1 specifies that TipFrameMode is equal to TIP_FRAME_AS_REF.

tip_frame_mode equal to 0 specifies that TipFrameMode is equal to TIP_FRAME_DISABLED.

If `is_tip_frame()` is equal to 1, it is a requirement of bitstream conformance that the computed value for TipFrameMode is equal to TIP_FRAME_AS_OUTPUT.

allow_tip_hole_fill equal to 1 specifies that holes in the Temporally Interpolated Prediction ([TIP](#)) motion field are filled in using interpolation. **allow_tip_hole_fill** equal to 0 specifies that holes in the TIP motion field are not filled.

apply_deblocking_filter_tip specifies if the deblocking filter is applied after computing the [TIP](#) frame.

tip_global_wtd_index specifies an index that chooses the weighting factor of the two reference frames used in [TIP](#).

tip_mv_zero equal to 1 indicates that TipGlobalMv is equal to 0. **tip_mv_zero** equal to 0 indicates that additional syntax elements are read to compute TipGlobalMv.

TipGlobalMv is the [TIP](#) global motion vector (this provides an offset to the normal [TIP](#) motion vectors).

tip_mv_row and **tip_mv_col** give the absolute value of the [TIP](#) global motion vector.

tip_mv_row_sign and **tip_mv_col_sign** give the sign of the [TIP](#) global motion vector.

tip_sharp and **tip_regular** specify the type of interpolation used in the [TIP](#) process.

disable_cdf_update equal to 1 specifies that the CDF update in the symbol decoding process is disabled and CDFs shall not be modified during decoding of this frame. **disable_cdf_update** equal to 0 specifies that CDF updates are enabled and CDFs can be modified during decoding.

qm_index specifies which entry in the `qm_y`, `qm_u`, `qm_v` arrays gives the quantization matrix level for a particular segment.

It is a requirement of bitstream conformance that `qm_index` is less than or equal to `pic_qm_num_minus_1`.

allow_tcq equal to 1 specifies that Trellis Coded Quantization ([TCQ](#)) is enabled for this frame. **allow_tcq** equal to 0 specifies that TCQ is disabled for this frame.

motion_field_estimation is a function call which indicates that the motion field estimation process in [§7.9 Motion field estimation process](#) is invoked.

setup_tip_motion_field is a function call which indicates that the setup [TIP](#) motion field process in [§ 7.10 Setup TIP motion field process](#) is invoked.

fill_tpl_mvs_sample_gap is a function call which indicates that the fill temporal motion vectors sample gap process specified in [§ 7.10.5 Fill temporal motion vectors sample gap process](#) is invoked.

OrderHints specifies the expected output order for each reference frame.

CodedLossless is a variable that is equal to 1 when all segments use lossless encoding. In this case, the deblocking filter, CDEF filter, and loop restoration filters are disabled.

It is a requirement of bitstream conformance that `delta_q_present` is equal to 0 when `CodedLossless` is equal to 1.

NOTE: In a mixed lossy-lossless encode (when `CodedLossless` is false and `HasLosslessSegment` is true), to guarantee lossless reconstruction for chroma pixels belonging to a lossless segment and that are coded as part of a chroma block covering multiple luma blocks (with potentially different `segment_ids`), the co-located luma block from which the chroma block inherits its `segment_id` must also be coded in lossless mode. There are two scenarios where a chroma block may correspond to multiple luma blocks. These two scenarios must be handled as follows:

- In a chroma merge region, where luma blocks may be split but the chroma block remains unsplit, the luma block co-located with the bottom-right corner of the chroma block must be coded in lossless mode.
- In the case of SDP, where luma and chroma blocks may follow different partitioning structures, the luma block co-located with the top-left corner of the chroma block must be coded in lossless mode.

A simpler but arguably more restrictive way to achieve lossless chroma coding in a mixed lossy-lossless encode is to turn off SDP and restrict the minimum partition width and height to 8.

allow_parity_hiding equal to 1 specifies that this frame can hide the parity of some DC coefficients.

allow_bawp equal to 1 indicates that the syntax element `use_bawp` can be present. `allow_bawp` equal to 0 indicates that the syntax element `use_bawp` is not present. (this means that [BAWP](#) cannot be signaled if `allow_bawp` is equal to 0.)

allow_warpmv_mode equal to 1 indicates that the syntax element `warp_mv` can be present. `allow_warpmv_mode` equal to 0 indicates that the syntax element `warp_mv` is not present. (This means that `YMode` cannot be equal to `WARPMV` if `allow_warpmv_mode` is equal to 0.)

reduced_tx_set greater than 0 specifies that the frame is restricted to a reduced subset of the full set of transform types.

NOTE: `reduced_tx_set` can take values between 0 and 3. The value of `reduced_tx_set` (along with the size of the block and whether the block is inter or intra) is used in `get_tx_set` to determine a set of allowed transform types. The set is used in `transform_type` to read the luma transform type. The set is also used in `compute_tx_type` to work out the transform type for the current block.

setup_past_independence is a function call that indicates that this frame can be decoded without dependence on previous coded frames. When this function is invoked the following takes place:

- FeatureData[i][j] and FeatureEnabled[i][j] are set equal to 0 for i = 0..MAX_SEGMENTS-1 and j = 0..SEG_LVL_MAX-1.
- PrevSegmentIds[row][col] is set equal to 0 for row = 0..MiRows-1 and col = 0..MiCols-1.
- PrevGmParams[ref][i] is set equal to ((i % 3 == 2) ? 1 << WARPEDMODEL_PREC_BITS : 0) for ref = 0..REFS_PER_FRAME - 1, for i = 0..5.
- ccso_planes[plane] is set equal to 0 for plane = 0..2.

init_non_coeff_cdfs is a function call that initializes the CDF tables which are not used in the coeffs() syntax structure. When this function is invoked, the following steps apply:

- WarpMvCdf is set to a copy of Default_Warp_Mv_Cdf.
- TipPredModeCdf is set to a copy of Default_Tip_Pred_Mode_Cdf.
- WarpIdxCdf is set to a copy of Default_Warp_IdxCdf.
- WarpWithMvdCdf is set to a copy of Default_Warp_With_Mvd_Cdf.
- IsWarpCdf is set to a copy of Default_Is_Warp_Cdf.
- UseGdfCdf is set to a copy of Default_Use_Gdf_Cdf.
- BruModeCdf is set to a copy of Default_Bru_Mode_Cdf.
- CdefIndex0Cdf is set to a copy of Default_Cdef_Index0_Cdf.
- CdefIndexMinus1With3Cdf is set to a copy of Default_Cdef_Index_Minus1_With3_Cdf.
- CdefIndexMinus1With4Cdf is set to a copy of Default_Cdef_Index_Minus1_With4_Cdf.
- CdefIndexMinus1With5Cdf is set to a copy of Default_Cdef_Index_Minus1_With5_Cdf.
- CdefIndexMinus1With6Cdf is set to a copy of Default_Cdef_Index_Minus1_With6_Cdf.
- CdefIndexMinus1With7Cdf is set to a copy of Default_Cdef_Index_Minus1_With7_Cdf.
- CdefIndexMinus1With8Cdf is set to a copy of Default_Cdef_Index_Minus1_With8_Cdf.
- WarpDeltaPrecisionCdf is set to a copy of Default_Warp_Precision_Cdf.
- WarpDeltaParamLowCdf is set to a copy of Default_Warp_Delta_Param_Low_Cdf.
- WarpDeltaParamHighCdf is set to a copy of Default_Warp_Delta_Param_High_Cdf.
- WarpDeltaParamSignCdf is set to a copy of Default_Warp_Delta_Param_Sign_Cdf.
- YModeSetCdf is set to a copy of Default_Y_Mode_Set_Cdf.
- YModeIndexCdf is set to a copy of Default_Y_Mode_Index_Cdf.
- YModeOffsetCdf is set to a copy of Default_Y_Mode_Offset_Cdf.
- CwpIdxCdf is set to a copy of Default_Cwp_IdxCdf.
- FscModeCdf is set to a copy of Default_Fsc_Mode_Cdf.
- MrlIndexCdf is set to a copy of Default_Mrl_Index_Cdf.
- MrlSecIndexCdf is set to a copy of Default_Mrl_Sec_Index_Cdf.
- UseDpcmYCdf is set to a copy of Default_Use_Dpcm_Y_Cdf.

- DpcmModeYCdf is set to a copy of Default_Dpcm_Mode_Y_Cdf.
- UseDpcmUvCdf is set to a copy of Default_Use_Dpcm_UV_Cdf.
- DpcmModeUvCdf is set to a copy of Default_Dpcm_Mode_UV_Cdf.
- UVModeCflNotAllowedCdf is set to a copy of Default_Uv_Mode_Cfl_Not_Allowed_Cdf.
- IsCflCdf is set to a copy of Default_Is_Cfl_Cdf.
- IntrabcCdf is set to a copy of Default_Intrabc_Cdf.
- IntrabcPrecisionCdf is set to a copy of Default_Intrabc_Precision_Cdf.
- IntrabcModeCdf is set to a copy of Default_Intrabc_Mode_Cdf.
- MorphPredCdf is set to a copy of Default_Morph_Pred_Cdf.
- RegionTypeCdf is set to a copy of Default_Region_Type_Cdf.
- DipModeCdf is set to a copy of Default_Dip_Mode_Cdf.
- UseDipCdf is set to a copy of Default_Use_Dip_Cdf.
- DoSquareSplitCdf is set to a copy of Default_Do_Square_Split_Cdf.
- DoSplitCdf is set to a copy of Default_Do_Split_Cdf.
- RectTypeCdf is set to a copy of Default_Rect_Type_Cdf.
- DoExtPartitionCdf is set to a copy of Default_Do_Ext_Partition_Cdf.
- DoUneven4wayPartitionCdf is set to a copy of Default_Do_Uneven_4way_Partition_Cdf.
- SegIdExtFlagCdf is set to a copy of Default_Seg_Id_Ext_Flag_Cdf.
- SegmentIdCdf is set to a copy of Default_Segment_Id_Cdf.
- SegmentIdExtCdf is set to a copy of Default_Segment_Id_Ext_Cdf.
- SegmentIdPredictedCdf is set to a copy of Default_Segment_Id_Predicted_Cdf.
- If reduced_tx_part_set is equal to 0, TxPartitionTypeCdf is set to a copy of Default_Tx_Partition_Type_Cdf.
- If reduced_tx_part_set is equal to 1, TxPartitionTypeCdf is set to a copy of Default_Tx_Partition_Type_Reduced_Cdf.
- Tx2or3PartitionTypeCdf is set to a copy of Default_Tx_2or3_Partition_Type_Cdf.
- TxDoPartitionCdf is set to a copy of Default_Tx_Do_Partition_Cdf.
- LosslessTxSizeCdf is set to a copy of Default_Lossless_Tx_Size_Cdf.
- LosslessInterTxTypeCdf is set to a copy of Default_Lossless_Inter_Tx_Type_Cdf.
- SecTxTypeCdf is set to a copy of Default_Sec_Tx_Type_Cdf.
- CctxTypeCdf is set to a copy of Default_Cctx_Type_Cdf.
- MostProbableStxSetCdf is set to a copy of Default_Most_Probable_Stx_Set_Cdf.
- MostProbableStxSetAdstCdf is set to a copy of Default_Most_Probable_Stx_Set_Adst_Cdf.
- InterpFilterCdf is set to a copy of Default_Interp_Filter_Cdf.
- UseLocalWarpCdf is set to a copy of Default_Use_Local_Warp_Cdf.
- UseExtendWarpCdf is set to a copy of Default_Use_Extend_Warp_Cdf.

- SingleModeCdf is set to a copy of Default_Single_Mode_Cdf.
- UseBawpCdf is set to a copy of Default_Use_Bawp_Cdf.
- UseBawpChromaCdf is set to a copy of Default_Use_Bawp_Chroma_Cdf.
- ExplicitBawpCdf is set to a copy of Default_Explicit_Bawp_Cdf.
- ExplicitBawpScaleCdf is set to a copy of Default_Explicit_Bawp_Scale_Cdf.
- IsJointCdf is set to a copy of Default_Is_Joint_Cdf.
- CompoundModeNonJointCdf is set to a copy of Default_Compound_Mode_Non_Joint_Cdf.
- CompoundModeSameRefsCdf is set to a copy of Default_Compound_Mode_Same_Refs_Cdf.
- UseOptflowCdf is set to a copy of Default_Use_Optflow_Cdf.
- TipModeCdf is set to a copy of Default_Tip_Mode_Cdf.
- UseRefinemvCdf is set to a copy of Default_Use_Refinemv_Cdf.
- DrlModeCdf is set to a copy of Default_Drl_Mode_Cdf.
- SkipDrlModeCdf is set to a copy of Default_Skip_Drl_Mode_Cdf.
- TipDrlModeCdf is set to a copy of Default_Tip_Drl_Mode_Cdf.
- IsInterCdf is set to a copy of Default_Is_Inter_Cdf.
- CompModeCdf is set to a copy of Default_Comp_Mode_Cdf.
- SkipModeCdf is set to a copy of Default_Skip_Mode_Cdf.
- SkipCdf is set to a copy of Default_Skip_Cdf.
- CompRef0Cdf is set to a copy of Default_Comp_Ref0_Cdf.
- CompRef1Cdf is set to a copy of Default_Comp_Ref1_Cdf.
- SingleRefCdf is set to a copy of Default_Single_Ref_Cdf.
- UseMostProbablePrecisionCdf is set to a copy of Default_Use_Most_Probable_Precision_Cdf.
- PbMvPrecisionCdf is set to a copy of Default_Pb_Mv_Precision_Cdf.
- MvJointAdaptiveCdf is set to a copy of Default_Mv_Joint_Adaptive_Cdf.
- AmvdIndicesCdf is set to a copy of Default_Amvd_Indices_Cdf.
- JointShellSetCdf[i] is set to a copy of Default_Joint_Shell_Set_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell0Class0Cdf[i] is set to a copy of Default_Joint_Shell0_Class0_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell1Class0Cdf[i] is set to a copy of Default_Joint_Shell1_Class0_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell3Class0Cdf[i] is set to a copy of Default_Joint_Shell3_Class0_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell4Class0Cdf[i] is set to a copy of Default_Joint_Shell4_Class0_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell5Class0Cdf[i] is set to a copy of Default_Joint_Shell5_Class0_Cdf for i = 0..MV_CONTEXTS-1.

- JointShell6Class0Cdf[i] is set to a copy of Default_Joint_Shell6_Class0_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell0Class1Cdf[i] is set to a copy of Default_Joint_Shell0_Class1_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell1Class1Cdf[i] is set to a copy of Default_Joint_Shell1_Class1_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell3Class1Cdf[i] is set to a copy of Default_Joint_Shell3_Class1_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell4Class1Cdf[i] is set to a copy of Default_Joint_Shell4_Class1_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell5Class1Cdf[i] is set to a copy of Default_Joint_Shell5_Class1_Cdf for i = 0..MV_CONTEXTS-1.
- JointShell6Class1Cdf[i] is set to a copy of Default_Joint_Shell6_Class1_Cdf for i = 0..MV_CONTEXTS-1.
- JointShellLastTwoClassesCdf[i] is set to a copy of Default_Joint_Shell_Last_Two_Classes_Cdf for i = 0..MV_CONTEXTS-1.
- ShellOffsetLowClassCdf[i] is set to a copy of Default_Shell_Offset_Low_Class_Cdf for i = 0..MV_CONTEXTS-1.
- ShellOffsetClass2Cdf[i] is set to a copy of Default_Shell_Offset_Class2_Cdf for i = 0..MV_CONTEXTS-1.
- ShellOffsetOtherClassCdf[i] is set to a copy of Default_Shell_Offset_Other_Class_Cdf for i = 0..MV_CONTEXTS-1.
- ColMvGreaterCdf[i] is set to a copy of Default_Col_Mv_Greater_Cdf for i = 0..MV_CONTEXTS-1.
- ColMvIndexCdf[i] is set to a copy of Default_Col_Mv_Index_Cdf for i = 0..MV_CONTEXTS-1.
- JmvdScaleModeCdf is set to a copy of Default_Jmvd_Scale_Mode_Cdf.
- JmvdAdaptiveScaleModeCdf is set to a copy of Default_Jmvd_Adaptive_Scale_Mode_Cdf.
- PaletteYModeCdf is set to a copy of Default_Palette_Y_Mode_Cdf.
- IdentityRowYCdf is set to a copy of Default_Identity_Row_Y_Cdf.
- PaletteYSizeCdf is set to a copy of Default_Palette_Y_Size_Cdf.
- PaletteSize2YColorCdf is set to a copy of Default_Palette_Size_2_Y_Color_Cdf.
- PaletteSize3YColorCdf is set to a copy of Default_Palette_Size_3_Y_Color_Cdf.
- PaletteSize4YColorCdf is set to a copy of Default_Palette_Size_4_Y_Color_Cdf.
- PaletteSize5YColorCdf is set to a copy of Default_Palette_Size_5_Y_Color_Cdf.
- PaletteSize6YColorCdf is set to a copy of Default_Palette_Size_6_Y_Color_Cdf.
- PaletteSize7YColorCdf is set to a copy of Default_Palette_Size_7_Y_Color_Cdf.
- PaletteSize8YColorCdf is set to a copy of Default_Palette_Size_8_Y_Color_Cdf.
- DeltaQCdf is set to a copy of Default_Delta_Q_Cdf.
- IntraTxTypeLongCdf is set to a copy of Default_Intra_Tx_Type_Long_Cdf.
- InterTxTypeLongCdf is set to a copy of Default_Inter_Tx_Type_Long_Cdf.

- IsLongSideDctCdf is set to a copy of Default_Is_Long_Side_Dct_Cdf.
- IntraTxTypeSet1Cdf is set to a copy of Default_Intra_Tx_Type_Set1_Cdf.
- IntraTxTypeSet2Cdf is set to a copy of Default_Intra_Tx_Type_Set2_Cdf.
- InterTxTypeSet1Cdf is set to a copy of Default_Inter_Tx_Type_Set1_Cdf.
- InterTxTypeSet2Cdf is set to a copy of Default_Inter_Tx_Type_Set2_Cdf.
- InterTxTypeSet3Cdf is set to a copy of Default_Inter_Tx_Type_Set3_Cdf.
- InterTxTypeSet4Cdf is set to a copy of Default_Inter_Tx_Type_Set4_Cdf.
- InterTxTypeIndexSet1Cdf is set to a copy of Default_Inter_Tx_Type_Index_Set1_Cdf.
- InterTxTypeIndexSet2Cdf is set to a copy of Default_Inter_Tx_Type_Index_Set2_Cdf.
- InterTxTypeOffsetSet1Cdf is set to a copy of Default_Inter_Tx_Type_Offset_Set1_Cdf.
- InterTxTypeOffsetSet2Cdf is set to a copy of Default_Inter_Tx_Type_Offset_Set2_Cdf.
- InterIntraCdf is set to a copy of Default_Inter_Intra_Cdf.
- WarpInterIntraCdf is set to a copy of Default_Warp_Inter_Intra_Cdf.
- CflSignCdf is set to a copy of Default_Cfl_Sign_Cdf.
- WedgeInterIntraCdf is set to a copy of Default_Wedge_Inter_Intra_Cdf.
- CompGroupIdxCdf is set to a copy of Default_Comp_Group_Idx_Cdf.
- CompoundTypeCdf is set to a copy of Default_Compound_Type_Cdf.
- InterIntraModeCdf is set to a copy of Default_Inter_Intra_Mode_Cdf.
- WedgeQuadCdf is set to a copy of Default_Wedge_Quad_Cdf.
- WedgeAngleCdf is set to a copy of Default_Wedge_Angle_Cdf.
- WedgeDist1Cdf is set to a copy of Default_Wedge_Dist1_Cdf.
- WedgeDist2Cdf is set to a copy of Default_Wedge_Dist2_Cdf.
- CflAlphaCdf is set to a copy of Default_Cfl_Alpha_Cdf.
- CflIndexCdf is set to a copy of Default_Cfl_Index_Cdf.
- CflMhDirCdf is set to a copy of Default_Cfl_Mh_Dir_Cdf.
- CflMhccpCdf is set to a copy of Default_Cfl_Mhccp_Cdf.
- UseAmvdCdf is set to a copy of Default_Use_Amvd_Cdf.
- CcsoBlkCdf is set to a copy of Default_Ccso_Blkcdf.
- UseWienerNsCdf is set to a copy of Default_Use_Wiener_Ns_Cdf.
- WienerNsLengthCdf is set to a copy of Default_Wiener_Ns_Length_Cdf.
- WienerNsUvSymCdf is set to a copy of Default_Wiener_Ns_Uv_Sym_Cdf.
- WienerNsBaseCdf is set to a copy of Default_Wiener_Ns_Base_Cdf.
- UsePcWienerCdf is set to a copy of Default_Use_Pc_Wiener_Cdf.
- FlexRestorationTypeCdf is set to a copy of Default_Flex_Restoration_Type_Cdf.

init_coeff_cdfs() is a function call that initializes the CDF tables used in the coeffs() syntax structure. When this function is invoked, the following steps apply:

- The variable idx is derived as follows:
 - If base_q_idx is less than or equal to 90, idx is set equal to 0.
 - Otherwise, if base_q_idx is less than or equal to 140, idx is set equal to 1.
 - Otherwise, if base_q_idx is less than or equal to 190, idx is set equal to 2.
 - Otherwise, idx is set equal to 3.
- The cumulative distribution function arrays are reset to default values as follows:
 - TxbSkipCdf is set to a copy of Default_Txb_Skip_Cdf[idx].
 - EobPt16Cdf is set to a copy of Default_Eob_Pt_16_Cdf[idx].
 - EobPt32Cdf is set to a copy of Default_Eob_Pt_32_Cdf[idx].
 - EobPt64Cdf is set to a copy of Default_Eob_Pt_64_Cdf[idx].
 - EobPt128Cdf is set to a copy of Default_Eob_Pt_128_Cdf[idx].
 - EobPt256Cdf is set to a copy of Default_Eob_Pt_256_Cdf[idx].
 - EobPt512Cdf is set to a copy of Default_Eob_Pt_512_Cdf[idx].
 - EobPt1024Cdf is set to a copy of Default_Eob_Pt_1024_Cdf[idx].
 - EobExtraCdf is set to a copy of Default_Eob_Extra_Cdf[idx].
 - DcSignCdf is set to a copy of Default_Dc_Sign_Cdf[idx].
 - VTxbSkipCdf is set to a copy of Default_V_Txb_Skip_Cdf[idx].
 - CoeffBaseEobCdf is set to a copy of Default_Coeff_Base_Eob_Cdf[idx].
 - CoeffBaseLfEobCdf is set to a copy of Default_Coeff_Base_Lf_Eob_Cdf[idx].
 - CoeffBaseCdf is set to a copy of Default_Coeff_Base_Cdf[idx].
 - CoeffBaseLfCdf is set to a copy of Default_Coeff_Base_Lf_Cdf[idx].
 - CoeffBasePhCdf is set to a copy of Default_Coeff_Base_Ph_Cdf[idx].
 - CoeffBrCdf is set to a copy of Default_Coeff_Br_Cdf[idx].
 - CoeffBrLfCdf is set to a copy of Default_Coeff_Br_Lf_Cdf[idx].
 - CoeffBrUvCdf is set to a copy of Default_Coeff_Br_Uv_Cdf[idx].
 - CoeffBaseLfUvCdf is set to a copy of Default_Coeff_Base_Lf_Uv_Cdf[idx].
 - CoeffBaseLfEobUvCdf is set to a copy of Default_Coeff_Base_Lf_Eob_Uv_Cdf[idx].
 - CoeffBaseUvCdf is set to a copy of Default_Coeff_Base_Uv_Cdf[idx].
 - CoeffBaseEobUvCdf is set to a copy of Default_Coeff_Base_Eob_Uv_Cdf[idx].
 - CoeffBaseBobCdf is set to a copy of Default_Coeff_Base_Bob_Cdf[idx].
 - CoeffBrIdtxCdf is set to a copy of Default_Coeff_Br_Idtx_Cdf[idx].
 - CoeffBaseIdtxCdf is set to a copy of Default_Coeff_Base_Idtx_Cdf[idx].
 - IdtxSignCdf is set to a copy of Default_Idtx_Sign_Cdf[idx].

load_cdfs(ctx) is a function call that indicates that the CDF tables are loaded from [frame context](#) number ctx in the range 0 to (NUM_REF_FRAMES - 1). When this function is invoked, a copy of each CDF array mentioned in the semantics for init_coeff_cdfs and init_non_coeff_cdfs is loaded from an area of memory indexed by ctx. (The memory contents of these frame contexts have been initialized by previous calls to save_cdfs).

blend_cdfs(ctx) is a function call that indicates that the CDF tables are blended with the contents of frame context number ctx in the range 0 to (NUM_REF_FRAMES - 1). When this function is invoked, a blend is made of the CDF values for each of the CDF arrays mentioned in the semantics for init_coeff_cdfs and init_non_coeff_cdfs.

The blend works for each CDF of the cdf array in turn by calling the blend_cdf function with a reference to the CDF, a reference to the previously saved CDF for context ctx, and the length of each CDF as inputs.

The blend_cdf function (which updates the CDF with a small amount of the previously saved CDF) is specified as:

```
blend_cdf( cdf, savedCdf, sz ) {
    for( i = 0; i < sz - 2; i++ ) {
        cdf[ i ] = (1 << 15) -
            ( ( (1 << 15) - savedCdf[ i ] +
                7 * ((1 << 15) - cdf[ i ]) + 4) >> 3 )
    }
    i2 = sz - 1
    cdf[ i2 ] = (savedCdf[ i2 ] + 7 * cdf[ i2 ] + 4) >> 3
}
```

load_previous() is a function call that indicates that information from a previous frame (denoted by prevFrame) may be loaded for use in decoding the current frame. When this function is invoked the following ordered steps apply:

1. The variable prevFrame is set equal to ref_frame_idx[DerivedPrimaryRefFrame].
2. PrevGmParams is set equal to a copy of SavedGmParams[prevFrame].

load_previous_segment_ids() is a function call that indicates that a segmentation map from a previous frame (denoted by prevFrame) may be loaded for use in decoding the current frame. When this function is invoked the segmentation map contained in PrevSegmentIds is set as follows:

1. The variable prevFrame is set equal to ref_frame_idx[DerivedPrimaryRefFrame].
2. If segmentation_enabled is equal to 1, RefMiCols[prevFrame] is equal to MiCols, and RefMiRows[prevFrame] is equal to MiRows, PrevSegmentIds[row][col] is set equal to SavedSegmentIds[prevFrame][row][col] for row = 0..MiRows-1, for col = 0..MiCols-1.

Otherwise, PrevSegmentIds[row][col] is set equal to 0 for row = 0..MiRows-1, for col = 0..MiCols-1.

§ 6.17.3. Frame configuration structures

§ 6.17.3.1. Frame optical flow refine type semantics

opfl_refine_type specifies how optical flow refinement is signaled with the same semantics as enable_opfl_refine.

NOTE: It is not possible for `opfl_refine_type` to be set to `REFINE_AUTO`.

opfl_refine_all is used to set the value of `opfl_refine_type` when it does not fit in a single bit.

§ 6.17.3.2. Screen content params semantics

allow_screen_content_tools equal to 1 indicates that intra blocks may use palette encoding; `allow_screen_content_tools` equal to 0 indicates that palette encoding is never used.

force_integer_mv equal to 1 specifies that motion vectors will always be integers. `force_integer_mv` equal to 0 specifies that motion vectors can contain fractional bits.

§ 6.17.3.3. Intra block copy params semantics

allow_intrabc equal to 1 indicates that intra block copy can be used in this frame. `allow_intrabc` equal to 0 indicates that intra block copy is not allowed in this frame.

allow_local_intrabc equal to 1 indicates that intra block copy can use a block within the local area in this frame as reference. The local area consists of decoded samples, prior to any loop filtering operations, from the four most recently decoded 64x64 regions.

allow_global_intrabc equal to 1 indicates that intra block copy can use a block within the global area in this frame as reference. The global area consists of decoded samples, prior to any loop filtering operations, from the current and previous superblock rows, excluding the local area.

NOTE: The eligibility of a reference block in the local or global area for intra block copy is verified using `is_mv_valid`.

change_bvp_drl equal to 1 indicates that `max_bvp_drl_bits_minus_1` is changed from the value in the sequence header.

max_bvp_drl_bits_minus_1 plus 1 specifies the maximum number of times the `intrabc_drl_mode` syntax element is read within `read_intrabc_info` for blocks using intra block copy.

§ 6.17.4. Frame size structures

§ 6.17.4.1. Frame size semantics

frame_width_minus_1 plus one is the width of the frame in luma samples.

frame_height_minus_1 plus one is the height of the frame in luma samples.

It is a requirement of bitstream conformance that `frame_width_minus_1` is less than or equal to `max_frame_width_minus_1`.

It is a requirement of bitstream conformance that `frame_height_minus_1` is less than or equal to `max_frame_height_minus_1`.

If `FrameIsIntra` is equal to 0 (indicating that this frame may use inter prediction), the requirements described in the frame size with refs semantics of [section 6.8.6] must also be satisfied.

§ 6.17.4.2. Frame size with bridge semantics

bridge_frame_width_minus_1 plus 1 specifies the target width of the [Bridge frame](#).

bridge_frame_height_minus_1 plus 1 specifies the target height of the [Bridge frame](#).

NOTE: [Bridge frames](#) are used to make frames smaller. If the reference frame is already smaller than the target size then the frame dimensions are unchanged.

§ 6.17.4.3. Frame size with refs semantics

For inter frames, the frame size is either set equal to the size of a reference frame, or can be sent explicitly.

found_ref equal to 1 indicates that the frame dimensions can be inferred from reference frame *i* where *i* is the loop counter in the syntax parsing process for `frame_size_with_refs`. **found_ref** equal to 0 indicates that the frame dimensions are not inferred from reference frame *i*.

It is a requirement of bitstream conformance that `RefOrderHint[ ref_frame_idx[ i ] ]` is not equal to `RESTRICTED_OH`.

Once the `FrameWidth` and `FrameHeight` have been computed for an [inter frame](#), it is a requirement of bitstream conformance that for all values of *i* in the range `0..(REFS_PER_FRAME - 1)`, all the following conditions are true:

- `2 * FrameWidth >= RefFrameWidth[ ref_frame_idx[ i ] ]`
- `2 * FrameHeight >= RefFrameHeight[ ref_frame_idx[ i ] ]`
- `FrameWidth <= 16 * RefFrameWidth[ ref_frame_idx[ i ] ]`
- `FrameHeight <= 16 * RefFrameHeight[ ref_frame_idx[ i ] ]`

NOTE: This is a requirement even if all the blocks in an [inter frame](#) are coded using [intra prediction](#).

§ 6.17.4.4. Compute image size function semantics

MiCols is the number of 4x4 block columns in the frame.

MiRows is the number of 4x4 block rows in the frame.

CropLeft, **CropTop**, **CropWidth**, **CropHeight** express the size of the cropped window to output.

It is a requirement of bitstream conformance that:

- **CropWidth** is greater than 0.
- **CropHeight** is greater than 0.

If **Monochrome** is equal to 0, it is a requirement of bitstream conformance that:

- **CropLeft** is equal to `((CropLeft >> SubsamplingX) << SubsamplingX)`.
- **CropTop** is equal to `((CropTop >> SubsamplingY) << SubsamplingY)`.

§ 6.17.5. Filtering structures

§ 6.17.5.1. Interpolation filter semantics

is_filter_switchable equal to 1 indicates that the filter selection is signaled at the block level; **is_filter_switchable** equal to 0 indicates that the filter selection is signaled at the frame level.

interpolation_filter specifies the filter selection used for performing inter prediction:

interpolation_filter	Name of interpolation_filter
0	EIGHTTAP
1	EIGHTTAP_SMOOTH
2	EIGHTTAP_SHARP
3	BILINEAR
4	SWITCHABLE

§ 6.17.5.2. Deblocking filter params semantics

apply_deblocking_filter is an array containing flags that specify if the deblocking filter is applied for a particular plane and direction. Different values of **apply_deblocking_filter** from the array are used depending on the image plane being filtered, and the edge direction (vertical or horizontal) being filtered.

df_delta_q_present[i] equal to 1 means that **df_delta_q[i]** syntax element for the deblocking filter is present. **df_delta_q_present[i]** equal to 0 means that the **df_delta_q[i]** syntax element is not present.

df_delta_q[i] is used to adjust the deblocking filter strength by adding an offset to the quantizer-based index of the threshold tables used by the deblocking filter. The offsets can be set separately for horizontal and vertical boundaries of plane 0 (luma) and for boundaries of planes 1 and 2 (chroma).

The deblocking filter process is described in [§ 7.17 Deblocking filter process](#).

NOTE: The semantics of **allow_df_sub_pu** are provided in [§ 6.17.2 Frame header info semantics](#).

§ 6.17.6. Quantization structures

§ 6.17.6.1. Quantization params semantics

The residual is specified via decoded coefficients which are adjusted by one of four quantization parameters before the inverse transform is applied. The choice depends on the plane (Y or UV) and coefficient position (DC/AC coefficient). The [dequantization](#) process is specified in [§ 7.14 Reconstruction and dequantization](#).

base_q_idx indicates the base frame qindex. This is used for Y AC coefficients and as the base value for the other quantizers.

DeltaQYDc indicates the Y DC quantizer relative to **base_q_idx**.

diff_uv_delta equal to 1 indicates that the U and V delta quantizer values are coded separately. **diff_uv_delta** equal to 0 indicates that the U and V delta quantizer values share a common value.

DeltaQUdC indicates the U DC quantizer relative to **base_q_idx**.

DeltaQUAc indicates the U AC quantizer relative to base_q_idx.

DeltaQVDC indicates the V DC quantizer relative to base_q_idx.

DeltaQVAc indicates the V AC quantizer relative to base_q_idx.

[§](#) **6.17.6.2. Setup QM params semantics**

using_qmatrix specifies that the quantizer matrix will be used to compute quantizers.

pic_qm_num_minus_1 plus 1 specifies the number of qm_y syntax elements present.

qm_y specifies the level in the quantizer matrix that is to be used for luma plane decoding.

If $qm_y[i]$ is less than NUM_CUSTOM_QMS, it is a requirement of bitstream conformance that $QmNumPlanes[qm_y[i]]$ is equal to NumPlanes.

If $qm_y[i]$ is less than NUM_CUSTOM_QMS and $QmMLayerId[qm_y[i]]$ is greater than or equal to 0, it is a requirement of bitstream conformance that $MLayerDependencyMap[obu_mlayer_id][QmMLayerId[qm_y[i]]]$ is equal to 1.

If $qm_y[i]$ is less than NUM_CUSTOM_QMS and $QmMLayerId[qm_y[i]]$ is greater than or equal to 0, it is a requirement of bitstream conformance that $TLayerDependencyMap[obu_mlayer_id][obu_tlayer_id][QmTLayerId[qm_y[i]]]$ is equal to 1.

qm_uv_same_as_y specifies that qm_u and qm_v match qm_y.

qm_u specifies the level in the quantizer matrix that is to be used for chroma U plane decoding.

If $qm_u[i]$ is less than NUM_CUSTOM_QMS, it is a requirement of bitstream conformance that $QmNumPlanes[qm_u[i]]$ is equal to NumPlanes.

If $qm_u[i]$ is less than NUM_CUSTOM_QMS and $QmMLayerId[qm_u[i]]$ is greater than or equal to 0, it is a requirement of bitstream conformance that $MLayerDependencyMap[obu_mlayer_id][QmMLayerId[qm_u[i]]]$ is equal to 1.

If $qm_u[i]$ is less than NUM_CUSTOM_QMS and $QmMLayerId[qm_u[i]]$ is greater than or equal to 0, it is a requirement of bitstream conformance that $TLayerDependencyMap[obu_mlayer_id][obu_tlayer_id][QmTLayerId[qm_u[i]]]$ is equal to 1.

qm_v specifies the level in the quantizer matrix that is to be used for chroma V plane decoding.

If $qm_v[i]$ is less than NUM_CUSTOM_QMS, it is a requirement of bitstream conformance that $QmNumPlanes[qm_v[i]]$ is equal to NumPlanes.

If $qm_v[i]$ is less than NUM_CUSTOM_QMS and $QmMLayerId[qm_v[i]]$ is greater than or equal to 0, it is a requirement of bitstream conformance that $MLayerDependencyMap[obu_mlayer_id][QmMLayerId[qm_v[i]]]$ is equal to 1.

If $qm_v[i]$ is less than NUM_CUSTOM_QMS and $QmMLayerId[qm_v[i]]$ is greater than or equal to 0, it is a requirement of bitstream conformance that $TLayerDependencyMap[obu_mlayer_id][obu_tlayer_id][QmTLayerId[qm_v[i]]]$ is equal to 1.

§ 6.17.6.3. Delta quantizer semantics

delta_coded specifies that the delta_q syntax element is present.

delta_q specifies an offset (relative to base_q_idx) for a particular [quantization parameter](#).

§ 6.17.7. Segmentation and tiling structures

§ 6.17.7.1. Segmentation params semantics

AV2 provides a means of segmenting the image and then applying various adjustments at the segment level.

Up to 16 segments may be specified for any given frame. For each of these segments it is possible to specify:

1. A quantizer (absolute value or delta).
2. A block skip mode that implies both the use of a (0,0) motion vector and that no residual will be coded.
3. A forced use of global motion vector

Each of these data values for each segment may be individually updated at the frame level. Where a value is not updated in a given frame, the value from a previous frame, indicated by DerivedPrimaryRefFrame, persists. The exceptions to this are key frames, intra only frames or other frames where independence from past frame values is required (for example to enable error resilience). In such cases all values are reset as described in the semantics for setup_past_independence.

reuse_seg_info equal to 1 indicates that the segment data and enables are reused (from the sequence header or multi-frame header). reuse_seg_info equal to 0 indicates that the segment data and enables are present in the current syntax structure.

SegIdPreSkip equal to 1 indicates that the segment id will be read before the skip_flag syntax element. SegIdPreSkip equal to 0 indicates that the skip_flag syntax element will be read first.

LastActiveSegId indicates the highest numbered segment id that has some enabled feature. This is used when decoding the segment id to only decode choices corresponding to used segments.

segmentation_enabled equal to 1 indicates that this frame makes use of the segmentation tool; segmentation_enabled equal to 0 indicates that the frame does not use segmentation.

segmentation_update_map equal to 1 indicates that the segmentation map is updated during the decoding of this frame. segmentation_update_map equal to 0 means that the segmentation map from a previous frame, indicated by DerivedPrimaryRefFrame, is used.

segmentation_temporal_update equal to 1 indicates that the updates to the segmentation map are coded relative to the existing segmentation map. segmentation_temporal_update equal to 0 indicates that the new segmentation map is coded without reference to the existing segmentation map.

§ 6.17.7.2. Tile info semantics

reuse_tile_info equal to 1 specifies that the tile parameters are reused. reuse_tile_info equal to 0 specifies that the tile parameters are present.

TileColsLog2 specifies the base 2 logarithm of the desired number of tiles across the frame.

TileCols specifies the number of tiles across the frame. It is a requirement of bitstream conformance that TileCols is less than or equal to MAX_TILE_COLS.

TileRowsLog2 specifies the base 2 logarithm of the desired number of tiles down the frame.

NOTE: For small frame sizes the actual number of tiles in the frame may be smaller than the desired number because the tile size is rounded up to a multiple of the maximum superblock size.

TileRows specifies the number of tiles down the frame. It is a requirement of bitstream conformance that TileRows is less than or equal to MAX_TILE_ROWS.

MiColStarts is an array specifying the start column (in units of 4x4 luma samples) for each tile across the image.

MiRowStarts is an array specifying the start row (in units of 4x4 luma samples) for each tile down the image.

context_update_tile_id specifies which tile to use for the CDF update. It is a requirement of bitstream conformance that context_update_tile_id is less than TileCols * TileRows.

tile_size_bytes_minus_1 is used to compute TileSizeBytes.

TileSizeBytes specifies the number of bytes needed to code each tile size.

[§](#) 6.17.7.3. *Tile params semantics*

uniform_tile_spacing_flag equal to 1 means that the tiles are roughly uniformly spaced across the frame. (All tiles are roughly the same size except for the ones at the right and bottom edge which can be smaller.) uniform_tile_spacing_flag equal to 0 means that the tile sizes are coded.

increment_tile_cols_log2 is used to compute tileColsLog2.

increment_tile_rows_log2 is used to compute tileRowsLog2.

If uniform_tile_spacing_flag is equal to 0, it is a requirement of bitstream conformance that startSb is equal to sbCols when the loop writing sbColStarts exits.

If uniform_tile_spacing_flag is equal to 0, it is a requirement of bitstream conformance that startSb is equal to sbRows when the loop writing sbRowStarts exits.

NOTE: The requirements on startSb ensure that the sizes of each tile add up to the full size of the frame when measured in superblocks.

width_in_sbs_minus_1 specifies the width of a tile minus 1 in units of superblocks.

height_in_sbs_minus_1 specifies the height of a tile minus 1 in units of superblocks.

maxTileHeightSb specifies the maximum height (in units of superblocks) that can be used for a tile (to avoid making tiles with too much area).

§ 6.17.7.4. Quantizer index delta parameters semantics

delta_q_present equal to 1 specifies that quantizer index delta values are present in the frame. **delta_q_present** equal to 0 specifies that quantizer index delta values are not present.

delta_q_res specifies the left shift to be applied to decoded quantizer index delta values.

§ 6.17.7.5. GDF params semantics

gdf_frame_enable equal to 1 specifies that Guided Detail Filter ([GDF](#)) filtering is enabled in the frame. **gdf_frame_enable** equal to 0 specifies that GDF filtering is disabled for this frame.

gdf_per_block equal to 1 specifies that a block level enable flag is present for Guided Detail Filter ([GDF](#)) to control GDF on a per-block basis. **gdf_per_block** equal to 0 specifies that no block level enable flag is present and GDF is applied uniformly across the frame.

gdf_pic_qc_idx specifies an adjustment to the quantizer used in [GDF](#) filtering.

gdf_pic_scale_idx specifies a scaling for the predicted adjustment used in [GDF](#) filtering.

§ 6.17.7.6. CDEF params semantics

cdef_frame_enable equal to 1 specifies that Constrained Directional Enhancement Filter (CDEF) filtering is enabled in the frame. **cdef_frame_enable** equal to 0 specifies that CDEF filtering is disabled for this frame.

cdef_damping_minus_3 controls the amount of damping in the deringing filter.

cdef_strengths_minus_1 plus one specifies the number of strengths settings used for CDEF.

cdef_on_skip_txfm_frame_enable equal to 1 specifies that CDEF filtering is enabled on skipped transform blocks. **cdef_on_skip_txfm_frame_enable** equal to 0 specifies that CDEF filtering is disabled for skipped transform blocks.

cdef_y_pri_zero specifies that **cdef_y_pri_strength** is equal to 0.

cdef_uv_pri_zero specifies that **cdef_uv_pri_strength** is equal to 0.

cdef_y_pri_strength and **cdef_uv_pri_strength** specify the strength of the primary filter.

cdef_y_sec_strength and **cdef_uv_sec_strength** specify the strength of the secondary filter.

§ 6.17.7.7. Loop restoration params semantics

tool_index is used to compute **FrameRestorationType** by choosing one of the enabled tools.

FrameRestorationType specifies the type of restoration used for each plane as follows:

FrameRestorationType	Name of FrameRestorationType
0	RESTORE_NONE
1	RESTORE_PC_WIENER
2	RESTORE_WIENER_NONSEP
3	RESTORE_SWITCHABLE

UsesLr indicates if any plane uses loop restoration.

frame_filters_on specifies that the Wiener filters are specified at the frame level (instead of being specified in each loop restoration unit).

temporal_pred_flag specifies that the frame level Wiener filters are copied from a previous reference frame.

rst_ref_pic_idx specifies which reference to use for the frame level Wiener filters.

If `temporal_pred_flag[ plane ]` is equal to 1, it is a requirement of bitstream conformance that `rst_ref_pic_idx` is less than `numRefFrames`.

If `temporal_pred_flag[ plane ]` is equal to 1, it is a requirement of bitstream conformance that `RefFrameFiltersOn[ refIdx ][ refPlane ]` is equal to 1.

num_filter_classes_idx specifies an index into `Decode_Num_Filter_Classes` that gives the number of classes used in the frame level pixel classified Wiener filter.

lr_luma_use_half_size specifies that luma uses a restoration size of half the maximum size.

lr_luma_use_max_size specifies that luma uses a restoration size of the maximum size.

lr_luma_use_quarter_size specifies that luma uses a restoration size of quarter the maximum size.

lr_chroma_use_half_size specifies that chroma uses a restoration size of half the maximum size.

lr_chroma_use_max_size specifies that chroma uses a restoration size of the maximum size.

lr_chroma_use_quarter_size specifies that chroma uses a restoration size of quarter the maximum size.

LoopRestorationSize[plane] specifies the size of loop restoration units in units of samples in the current plane.

If `usesChromaLr` is equal to 1, it is a requirement of bitstream conformance that $64 \gg \text{SubsamplingY}$ is less than or equal to `LoopRestorationSize[ 1 ]`.

NOTE: This ensures that restoration units are not smaller than the restoration stripe height.

It is a requirement of bitstream conformance that `check_ru_size()` is equal to 1, where the function `check_ru_size` is defined as:

```
check_ru_size() {
    maxPlaneRuSize = Max( LoopRestorationSize[0],
        LoopRestorationSize[1] << Max(SubsamplingX, SubsamplingY) )
    for ( i = 0; i < TileCols - 1; i++ ) {
        tileWidth = (MiColStarts[ i + 1 ] - MiColStarts[ i ]) * MI_SIZE
        if ( tileWidth % maxPlaneRuSize != 0 ) return 0
    }
    for ( i = 0; i < TileRows - 1; i++ ) {
        tileHeight = (MiRowStarts[ i + 1 ] - MiRowStarts[ i ]) * MI_SIZE
        if ( tileHeight % maxPlaneRuSize != 0 ) return 0
    }
    return 1
}
```

NOTE: This check ensures that restoration units do not cross internal tile boundaries.

§ 6.17.7.8. CCSO params semantics

ccso_frame_flag equal to 1 specifies that [CCSO](#) can be used on this frame. **ccso_frame_flag** equal to 0 specifies that [CCSO](#) is not enabled for this frame.

ccso_planes[plane] equal to 1 specifies that Cross Component Sample Offset ([CCSO](#)) filtering is enabled for a particular plane. **ccso_planes[plane]** equal to 0 specifies that CCSO filtering is disabled for that plane.

reuse_ccso equal to 1 specifies that the Cross Component Sample Offset ([CCSO](#)) parameters are reused from a previous decoded frame. **reuse_ccso** equal to 0 specifies that CCSO parameters are signaled in the current frame and not reused from a previous frame.

sb_reuse_ccso equal to 1 specifies that the Cross Component Sample Offset ([CCSO](#)) block level enable flags are reused from a previous decoded frame. **sb_reuse_ccso** equal to 0 specifies that CCSO block level enable flags are signaled in the current frame and not reused.

ccso_ref_idx specifies which reference contains the parameters to reuse.

SavedCcsoPlanes[i][plane] is defined to be the value of **ccso_planes[plane]** when **save_ccso_params(i,plane)** was last called.

SavedCcsoLumaSizeLog2[i][plane] is defined to be the value of **CcsoLumaSizeLog2** when **save_ccso_params(i,plane)** was last called.

When **ccso_ref_idx** is present in the bitstream the following requirements apply:

- It is a requirement of bitstream conformance that **ccso_ref_idx[plane]** is less than **NumTotalRefs**.
- It is a requirement of bitstream conformance that **SavedCcsoPlanes[idx][plane]** is equal to 1.
- It is a requirement of bitstream conformance that **RefOrderHint[idx]** is not equal to **RESTRICTED_OH**.

When **ccso_ref_idx** is present in the bitstream and **sb_reuse_ccso[plane]** is equal to 1, the following requirements apply:

- It is a requirement of bitstream conformance that **RefMiRows[idx]** is equal to **MiRows**.
- It is a requirement of bitstream conformance that **RefMiCols[idx]** is equal to **MiCols**.
- It is a requirement of bitstream conformance that **SavedCcsoLumaSizeLog2[idx]** is equal to **CcsoLumaSizeLog2**.
- It is a requirement of bitstream conformance that **CcsoLumaSizeLog2** is equal to **CCSO_LUMA_SIZE_LOG2**.

load_ccso_params is a function call defined in [§ 7.23 Reference frame update process](#).

ccso_bo_only specifies that a smaller set of [CCSO](#) parameters is present.

ccso_quant_idx and **ccso_scale_idx** specify the quantization index and scaling for CCSO filtering.

ccso_ext_filter specifies the CCSO filter type.

It is a requirement of bitstream conformance that **ccso_ext_filter** is not equal to 7.

ccso_max_band_log2 specifies the base 2 logarithm of the maximum number of bands for CCSO filtering.

It is a requirement of bitstream conformance that $1 < \text{ccso_max_band_log2}$ is less than or equal to **CCSO_BAND_NUM**.

ccso_edge_clf is used to reduce the number of classes used within CCSO filtering.

ccso_offset_idx is used to compute the sample offset by providing an index into the **Ccso_Offset** table.

§ 6.17.8. Transform and coding mode structures

§ 6.17.8.1. TX mode semantics

tx_mode_select is used to compute TxMode.

TxMode specifies how the transform size is determined:

TxMode	Name of TxMode
0	ONLY_4X4
1	TX_MODE_LARGEST
2	TX_MODE_SELECT

For **tx_mode** equal to **TX_MODE_LARGEST**, the inverse transform will use the largest transform size that fits inside the block.

For **tx_mode** equal to **ONLY_4X4**, the inverse transform will use only 4x4 transforms.

For **tx_mode** equal to **TX_MODE_SELECT**, the choice of transform size is specified explicitly for each block.

§ 6.17.8.2. Skip mode params semantics

SkipModeFrame[list] specifies the initial frames to use for compound prediction when **skip_mode** is equal to 1. (These frames are used for motion vector prediction, but may change when an entry is selected from the motion vector stack.)

skip_mode_present equal to 1 specifies that the syntax element **skip_mode** will be present. **skip_mode_present** equal to 0 specifies that **skip_mode** will not be used for this frame.

§ 6.17.8.3. Frame reference mode semantics

reference_select equal to 1 specifies that the [mode info](#) for inter blocks contains the syntax element **comp_mode** that indicates whether to use single or compound reference prediction. **reference_select** equal to 0 specifies that all inter blocks will use single prediction.

§ 6.17.9. Global motion structures

§ 6.17.9.1. Global motion params semantics

use_global_motion equal to 1 specifies that global motion parameters are present for this frame.
use_global_motion equal to 0 specifies that no global motion parameters are present.

our_ref specifies a reference of the current frame. The base warp will be taken from one set of the parameters saved for this reference.

If our_ref is not equal to NumTotalRefs, it is a requirement of bitstream conformance that OrderHints[our_ref] is not equal to RESTRICTED_OH.

their_ref specifies a reference that was used by the our_ref reference. The base warp will be taken from the warp used by our_ref when it was predicting from their_ref.

It is a requirement of bitstream conformance that SavedOrderHints[refIdx][their_ref] is not equal to RESTRICTED_OH.

is_global equal to 1 specifies that global motion parameters are present for a particular reference frame.
is_global equal to 0 specifies that global motion parameters are not present for this reference frame.

is_rot_zoom equal to 1 specifies that a particular reference frame uses rotation and zoom global motion.
is_rot_zoom equal to 0 specifies that a more general affine global motion model is used.

§ 6.17.9.2. Global param semantics

precBits specifies the number of fractional bits used for representing gm_params[ref][idx]. All global motion parameters are stored in the model with WARPEDMODEL_PREC_BITS fractional bits, but the parameters are encoded with less precision.

§ 6.17.9.3. Decode signed subexp with ref semantics

NOTE: decode_signed_subexp_with_ref will return a value in the range low to high - 1 (inclusive).

§ 6.17.9.4. Decode unsigned subexp with ref semantics

NOTE: decode_unsigned_subexp_with_ref will return a value in the range 0 to mx - 1 (inclusive).

§ 6.17.9.5. Decode subexp semantics

subexp_final_bits provide the final bits that are read once the appropriate range has been determined.

subexp_more_bits equal to 0 specifies that the parameter is in the range mk to mk+a-1.
subexp_more_bits equal to 1 specifies that the parameter is greater than mk+a-1.

subexp_bits specifies the value of the parameter minus mk.

§ 6.17.10. Film grain structures

§ 6.17.10.1. Film grain config semantics

apply_grain equal to 1 specifies that film grain should be added to this frame. **apply_grain** equal to 0 specifies that film grain should not be added.

fgm_id specifies which film grain model to use.

It is a requirement of bitstream conformance that `FilmGrainPresent[ fgm_id ]` is equal to 1.

NOTE: The film grain model corresponding to `fgm_id` should be transmitted before it is used by the decoding process. See [§ 7.3.8.8 Film grain OBU availability](#) for the general availability requirements for film grain OBUs.

If **apply_grain** is equal to 1, it is a requirement of bitstream conformance that all of the following are true:

- `TLayerDependencyMap[obu_mlayer_id][obu_tlayer_id][FgmTLayerId[fgm_id]]` is equal to 1,
- `MLayerDependencyMap[obu_mlayer_id][FgmMLayerId[fgm_id]]` is equal to 1,
- `FgmChromaIdc[ fgm_id ]` is equal to `chroma_format_idc`.

grain_seed specifies the initialization value for the pseudo-random numbers generator used during film grain synthesis.

load_grain_model(idx) is a function call that indicates that all syntax elements read in `film_grain_model` should be set equal to the values stored in an area of memory indexed by `idx`.

§ 6.17.10.2. Film grain model semantics

chroma_scaling_from_luma specifies that the film grain model scaling for the chroma [component](#) is inferred from the film grain model scaling for the luma [component](#).

num_y_points specifies the number of points for the piece-wise linear scaling function of the luma [component](#).

It is a requirement of bitstream conformance that `num_y_points` is less than or equal to 14.

point_value_increment_bits_minus_1 plus 1 specifies the number of bits in the syntax element `point_y_value` (and corresponding chroma syntax elements `point_cb_value` and `point_cr_value`, depending on the context).

point_scaling_bits_minus_5 plus 5 specifies the number of bits in the syntax element `point_y_scaling` (and corresponding chroma syntax elements `point_cb_scaling` and `point_cr_scaling`, depending on the context).

point_y_value[i] represents the x (luma value) coordinate for the i-th point of the piecewise linear scaling function for luma [component](#). The values are signaled on the scale of 0..255. (In case of 10 bit video, these values correspond to luma values divided by 4.)

If *i* is greater than 0, it is a requirement of bitstream conformance that `point_y_value[ i ]` is greater than `point_y_value[ i - 1 ]` and less than 256. (this ensures the x coordinates are specified in increasing order).

NOTE: This conformance requirement refers to the final values of `point_y_value` after the addition of `point_y_value[ i - 1 ]`.

point_y_scaling[i] represents the scaling (output) value for the *i*-th point of the piecewise linear scaling function for luma [component](#).

num_cb_points specifies the number of points for the piece-wise linear scaling function of the cb [component](#).

It is a requirement of bitstream conformance that `num_cb_points` is less than or equal to 14.

point_cb_value[i] represents the x coordinate for the *i*-th point of the piece-wise linear scaling function for cb [component](#). The values are signaled on the scale of 0..255.

If *i* is greater than 0, it is a requirement of bitstream conformance that `point_cb_value[ i ]` is greater than `point_cb_value[ i - 1 ]` and less than 256.

point_cb_scaling[i] represents the scaling (output) value for the *i*-th point of the piecewise linear scaling function for cb [component](#).

num_cr_points specifies the number of points for the piece-wise linear scaling function of the cr [component](#).

It is a requirement of bitstream conformance that `num_cr_points` is less than or equal to 14.

If `subX` is equal to 1 and `subY` is equal to 1 and `num_cb_points` is equal to 0, it is a requirement of bitstream conformance that `num_cr_points` is equal to 0.

If `subX` is equal to 1 and `subY` is equal to 1 and `num_cb_points` is not equal to 0, it is a requirement of bitstream conformance that `num_cr_points` is not equal to 0.

NOTE: These requirements ensure that for 4:2:0 chroma subsampling, film grain noise will be applied to both chroma components, or to neither. There is no restriction for 4:2:2 or 4:4:4 chroma subsampling.

point_cr_value[i] represents the x coordinate for the *i*-th point of the piece-wise linear scaling function for cr [component](#). The values are signaled on the scale of 0..255.

If *i* is greater than 0, it is a requirement of bitstream conformance that `point_cr_value[ i ]` is greater than `point_cr_value[ i - 1 ]` and less than 256.

point_cr_scaling[i] represents the scaling (output) value for the *i*-th point of the piecewise linear scaling function for cr [component](#).

grain_scaling_minus_8 represents the shift - 8 applied to the grain values, which are obtained by a multiplication of the grain template value with the scaling function value. The `grain_scaling_minus_8` can take values of 0..3 and determines the range and quantization step of the film grain.

ar_coeff_lag specifies the number of auto-regressive coefficients for luma and chroma.

bits_per_ar_coeff_y_minus_5 plus 5 specifies the number of bits in the syntax element `ar_coeffs_y`.

bits_per_ar_coeff_cb_minus_5 plus 5 specifies the number of bits in the syntax element `ar_coeffs_cb`.

bits_per_ar_coeff_cr_minus_5 plus 5 specifies the number of bits in the syntax element `ar_coeffs_cr`.

ar_coeffs_y[i] specifies auto-regressive coefficients used for the Y plane.

ar_coeffs_cb[i] specifies auto-regressive coefficients used for the U plane.

ar_coeffs_cr[i] specifies auto-regressive coefficients used for the V plane.

ar_coeff_shift_minus_6 specifies the range of the auto-regressive coefficients. Values of 0, 1, 2, and 3 correspond to the ranges for auto-regressive coefficients of [-2, 2), [-1, 1), [-0.5, 0.5) and [-0.25, 0.25) respectively.

grain_scale_shift specifies how much the Gaussian random numbers are scaled down before the start of the grain template generation process.

cb_mult represents a multiplier for the cb component used in derivation of the input index to the cb component scaling function.

cb_luma_mult represents a multiplier for the average luma component used in derivation of the input index to the cb component scaling function.

cb_offset represents an offset used in derivation of the input index to the cb component scaling function.

cr_mult represents a multiplier for the cr component used in derivation of the input index to the cr component scaling function.

cr_luma_mult represents a multiplier for the average luma component used in derivation of the input index to the cr component scaling function.

cr_offset represents an offset used in derivation of the input index to the cr component scaling function.

overlap_flag equal to 1 indicates that the overlap between film grain blocks shall be applied. `overlap_flag` equal to 0 indicates that the overlap between film grain blocks shall not be applied.

clip_to_restricted_range equal to 1 indicates that clipping to the restricted (studio) range shall be applied to the [sample values](#) after adding the film grain. `clip_to_restricted_range` equal to 0 indicates that clipping to the full range shall be applied to the [sample values](#) after adding the film grain.

fg_mc_identity is used to adjust the clipping range for the video after adding the film grain. In particular, `fg_mc_identity` equal to 1 specifies that the chroma clipping range is equal to the luma clipping range when the `clip_to_restricted_range` is equal to 1.

film_grain_block_size equal to 0 indicates that when the film grain is applied to the reconstructed samples, a film grain block size of 16 by 16 is used. `film_grain_block_size` equal to 1 indicates that a film grain block size of 32 by 32 is used.

NOTE: The 16 by 16 and 32 by 32 numbers do not take into account the increase in the block size when the `overlap_flag` is equal to 1.

§ 6.18. Tile group OBU semantics

is_first_tile_group equal to 1 specifies that this is the first [Tile Group](#) for the current frame.
is_first_tile_group equal to 0 specifies that this is not the first [Tile Group](#) for the current frame.

It is a requirement of bitstream conformance that **SeenFrameHeader** is not equal to **is_first_tile_group**.

frame_header_present_flag equal to 1 specifies that the frame header is present.
frame_header_present_flag equal to 0 specifies that the frame header is not present.

NumTiles specifies the total number of tiles in the frame.

tile_start_and_end_present_flag equal to 1 specifies that the **tg_start** and **tg_end** syntax elements are present to indicate which tiles are contained in this [Tile Group](#). **tile_start_and_end_present_flag** equal to 0 specifies that **tg_start** and **tg_end** are not present and this [Tile Group](#) covers the entire frame (i.e., **tg_start** is inferred to be 0 and **tg_end** is inferred to be **NumTiles** - 1).

tg_start specifies the zero-based index of the first tile in the current [Tile Group](#).

It is a requirement of bitstream conformance that the value of **tg_start** is equal to the value of **TileNum** at the point that **tile_group_payload** is invoked.

tg_end specifies the zero-based index of the last tile in the current [Tile Group](#).

It is a requirement of bitstream conformance that the value of **tg_end** is greater than or equal to **tg_start**.

It is a requirement of bitstream conformance that the value of **tg_end** for the last tile group in each frame is equal to **NumTiles** - 1.

NOTE: These requirements ensure that conceptually all tile groups are present and received in order for the purposes of specifying the decode process.

bru_tile_active equal to 0 specifies that a whole tile is inactive. **bru_tile_active** equal to 1 specifies that the **bru_mode** syntax element is present for each superblock in a tile.

§ 6.19. Tile group payload semantics

§ 6.19.1. General tile group payload semantics

frame_end_update_cdf is a function call that indicates that the frame CDF arrays are set equal to the saved CDFs. This process is described in [§ 7.5 Frame end update CDF process](#).

tile_size_minus_1 is used to compute **tileSize**.

tileSize specifies the size in bytes of the next coded tile.

NOTE: This size includes any padding bytes if added by the exit process for the Symbol decoder. The size does not include the bytes used for **tile_size_minus_1** or syntax elements sent before **tile_size_minus_1**. For the last tile in the tile group, **tileSize** is computed instead of being read and includes the OBU trailing bits.

decode_frame_wrapup is a function call that invokes the decode frame wrapup process specified in [§7.2 Decode frame wrapup process](#).

§ 6.19.2. Tile-level structures

§ 6.19.2.1. Decode tile semantics

clear_left_context is a function call that indicates that some arrays are initialized. When this function is invoked the arrays WarpBankSize, WarpBankStart, RefMvBankSize, RefMvBankStart, LeftLevelContext, LeftDcContext, LeftMiSizes, and LeftSegPredContext are initialized as follows:

```
for (i = 0; i < MiRows; i++) {
    for (plane = 0; plane < 3; plane++) {
        LeftDcContext[ plane ][ i ] = 0
        LeftLevelContext[ plane ][ i ] = 0
    }
    LeftSegPredContext[ i ] = 0
}
sbSize4 = Num_4x4_Blocks_High[ SbSize ]
numSbs = (MiRows + sbSize4 - 1) / sbSize4
for (i = 0; i < numSbs * sbSize4; i++) {
    LeftMiSizes[ 0 ][ i ] = BLOCK_256X256
    LeftMiSizes[ 1 ][ i ] = BLOCK_256X256
}
for(ref = 0; ref < REFS_PER_FRAME; ref++) {
    WarpBankSize[ ref ] = 0
    WarpBankStart[ ref ] = 0
}
for(ref = 0; ref < BANK_REFS_PER_FRAME; ref++) {
    RefMvBankSize[ ref ] = 0
    RefMvBankStart[ ref ] = 0
}
```

clear_above_context is a function call that indicates that some arrays used to determine the probabilities are initialized. When this function is invoked the arrays AboveLevelContext, AboveDcContext, AboveMiSizes, and AboveSegPredContext are initialized as follows:

```
for (i = 0; i < MiCols; i++) {
    for (plane = 0; plane < 3; plane++) {
        AboveDcContext[ plane ][ i ] = 0
        AboveLevelContext[ plane ][ i ] = 0
    }
    AboveSegPredContext[ i ] = 0
}
sbSize4 = Num_4x4_Blocks_Wide[ SbSize ]
numSbs = (MiCols + sbSize4 - 1) / sbSize4
for (i = 0; i < numSbs * sbSize4; i++) {
    AboveMiSizes[ 0 ][ i ] = BLOCK_256X256
    AboveMiSizes[ 1 ][ i ] = BLOCK_256X256
}
```

TreeType specifies which syntax elements are present as follows:

TreeType	Name of TreeType
0	SHARED_PART
1	LUMA_PART
2	CHROMA_PART

When `TreeType` is equal to `LUMA_PART`, syntax elements related to the luma plane are present. When `TreeType` is equal to `CHROMA_PART`, syntax elements related to the chroma plane are present. Otherwise (`TreeType` is equal to `SHARED_PART`), both luma and chroma syntax elements can be present.

ReadDeltas specifies whether the current block may read delta values for the quantizer index. If the entire superblock is skipped the delta values are not read, otherwise delta values for the quantizer index are read on the first block of a superblock. If `delta_q_present` is equal to 0, no delta values are read for the quantizer index.

bru_mode specifies the type of superblock as specified in [Table 6.21](#):

Table 6.21: bru_mode values and interpretations

bru_mode	Name of bru_mode
0	BRU_INACTIVE
1	BRU_SUPPORT
2	BRU_ACTIVE

NOTE: `bru_mode` is also used outside [BRU](#) frames to determine if the syntax elements are parsed. In bridge frames, syntax is inferred, so `bru_mode` is `BRU_INACTIVE`. In normal frames, syntax is parsed, so `bru_mode` is `BRU_ACTIVE`.

§ 6.19.2.2. Reset reference motion vector bank function semantics

WarpBankHits counts how many times the `WarpBankParams` have been searched in the superblock.

RefMvBankHits counts how many times `update_ref_mv_bank` has been called in the superblock.

RefMvUnitHits counts how many times `update_ref_mv_bank` has been called since the last time the current block was aligned to a unit boundary. The unit size is defined relative to the superblock size such that a grid of 8 by 8 units fits within the superblock.

RefMvRemainHits defines how many calls to `update_ref_mv_bank` are allowed. This variable decreases when `update_ref_mv_bank` is called, but can be increased if a large block is processed that is aligned to a unit boundary.

§ 6.19.2.3. Clear block decoded flags function semantics

BlockDecoded is an array which stores one boolean value per 4x4 sample block per plane in the current superblock, plus a border of one 4x4 sample block on all sides of the superblock. Except for the borders, a value of 1 in `BlockDecoded` indicates that the corresponding 4x4 sample block has been decoded. The borders are used when computing above-right and below-left availability along the top and left edges of the superblock.

§ 6.19.3. Partition structures

§ 6.19.3.1. Decode partition semantics

The parameter **hasChroma** specifies that this partition contains one or more blocks with chroma mode information.

The parameter **chromaOffset** specifies whether the minimum size for chroma blocks has been reached. chromaOffset equal to 0 specifies that the minimum size has not been reached (in this case the chroma block will be the same size as the luma block). chromaOffset equal to 1 specifies that the minimum size has been reached (in this case the chroma block has stopped splitting so may be a different size to the luma block).

If chromaOffset is equal to 1 and hasChroma is equal to 1 and TreeType is not equal to LUMA_PART and NumPlanes is greater than 1, it is a requirement of bitstream conformance that r is less than MiRows or c is less than MiCols.

NOTE: This requirement ensures that chroma info is always present. To satisfy this requirement, only certain partition choices can be made near the edge.

If r is less than MiRows or c is less than MiCols, then if hasChroma is equal to 1 it is a requirement of bitstream conformance that `get_plane_residual_size( chromaOffset ? ChromaMiSize : subSize, 1 )` is not equal to BLOCK_INVALID.

NOTE: This requirement of bitstream conformance applies to the values of variables chromaOffset, ChromaMiSize, and subSize at the point just before the line `if ( partition == PARTITION_NONE ) {`.

ChromaMiRow is a variable holding the vertical location of the chroma block in units of 4x4 luma samples.

ChromaMiCol is a variable holding the horizontal location of the chroma block in units of 4x4 luma samples.

ChromaMiSize is a variable holding the size of the chroma block with values having the same interpretation for the variable subSize. The size corresponds to the amount of luma samples that are covered by the chroma block.

The variable **partition** specifies how a block is partitioned:

partition	Name of partition
0	PARTITION_NONE
1	PARTITION_HORZ
2	PARTITION_VERT
3	PARTITION_HORZ_3
4	PARTITION_VERT_3
5	PARTITION_HORZ_4A
6	PARTITION_HORZ_4B
7	PARTITION_VERT_4A
8	PARTITION_VERT_4B
9	PARTITION_SPLIT

NOTE: PARTITION_HORZ_3 and PARTITION_VERT_3 split into four parts by first splitting in a ratio 1:2:1, and then splitting the middle section in the perpendicular direction.

The variable **subSize** is computed from partition and indicates the size of the component blocks within this block as specified in [Table 6.22](#):

Table 6.22: subSize values for different partition types

subSize	Name of subSize
0	BLOCK_4X4
1	BLOCK_4X8
2	BLOCK_8X4
3	BLOCK_8X8
4	BLOCK_8X16
5	BLOCK_16X8
6	BLOCK_16X16
7	BLOCK_16X32
8	BLOCK_32X16
9	BLOCK_32X32
10	BLOCK_32X64
11	BLOCK_64X32
12	BLOCK_64X64
13	BLOCK_64X128
14	BLOCK_128X64
15	BLOCK_128X128
16	BLOCK_128X256
17	BLOCK_256X128
18	BLOCK_256X256
19	BLOCK_4X16
20	BLOCK_16X4
21	BLOCK_8X32
22	BLOCK_32X8
23	BLOCK_16X64
24	BLOCK_64X16
25	BLOCK_4X32
26	BLOCK_32X4
27	BLOCK_8X64
28	BLOCK_64X8

NOTE: When a partition splits into blocks of different sizes, the first and final blocks will be of size subSize.

The dimensions of these blocks are given in width, height order (e.g. BLOCK_8X16 corresponds to a block that is 8 samples wide, and 16 samples high).

ChromaFollowsLuma is a variable that is used to decide whether the chroma partitioning follows luma. The chroma partitioning follows luma if luma is split and none of the split partitions contains a block smaller than 32 by 32.

ChromaPartitionKnown is an array that records where the chroma partitioning is already known (as it is forced to follow the luma partitioning).

region_type equal to INTRA_REGION indicates that the luma partition tree is sent first, followed by information about a single chroma block. All blocks in this case will be intra blocks.

§ 6.19.3.2. Read partition semantics

do_split equal to 1 specifies that the block is to be split further. **do_split** equal to 0 specifies that no further splitting is performed.

do_square_split equal to 1 specifies that the block is split into 4 square parts. **do_square_split** equal to 0 specifies that the block is not split into 4 square parts.

rect_type specifies the direction in which the block is to be split. **rect_type** is equal to RECT_HORZ for a horizontal cut. **rect_type** is equal to RECT_VERT for a vertical cut.

do_ext_partition equal to 1 specifies that extended partitions are used and the block is split into four parts. **do_ext_partition** equal to 0 specifies that the block is split into two parts.

do_uneven_4way_partition equal to 1 specifies that an uneven partition is used when splitting the block into four parts. **do_uneven_4way_partition** equal to 0 specifies that the uneven 4-way partition is not used for the block.

uneven_4way_partition_type specifies the type of uneven partition.

Rect_Part_Table is a lookup table for finding the chosen partition.

§ 6.19.4. Block decoding structures

§ 6.19.4.1. Decode block semantics

MiRow is a variable holding the vertical location of the block in units of 4x4 luma samples.

MiCol is a variable holding the horizontal location of the block in units of 4x4 luma samples.

MiSize is a variable holding the size of the block with values having the same interpretation for the variable **subSize**.

HasChroma is a variable that specifies whether chroma information is coded for this block.

Variable **AvailU** is equal to 0 if the information from the block above cannot be used on the luma plane; **AvailU** is equal to 1 if the information from the block above can be used on the luma plane.

Variable **AvailL** is equal to 0 if the information from the block to the left cannot be used on the luma plane; **AvailL** is equal to 1 if the information from the block to the left can be used on the luma plane.

Variables **AvailUChroma** and **AvailLChroma** have the same significance as **AvailU** and **AvailL**, but on the chroma planes.

SubMvs contains motion vectors for each 4x4 subblock. SubMvs are initialized in decode block, but can get adjusted if the block is predicted with a warped prediction.

The function call to **motion_field_motion_vector_storage** indicates that the motion field motion vector storage process specified in [§ 7.22 Motion field motion vector storage process](#) is invoked.

After all the syntax elements have been read for the block, if `is_inter` is equal to 0, it is a requirement of bitstream conformance that `seg_feature_active(SEG_LVL_SKIP)` is equal to 0.

After the local variables `bw4` and `bh4` have been computed in the decode block syntax, it is a requirement of bitstream conformance that `bw4` is less than or equal to `bh4 * MaxPbAspectRatio`, and that `bh4` is less than or equal to `bw4 * MaxPbAspectRatio`.

§ 6.19.5. Mode information structures

§ 6.19.5.1. Mode info semantics

This switches between different ways of reading the mode info for different frame types.

§ 6.19.5.2. BRU mode info semantics

This syntax is used for inactive and support [BRU](#) blocks.

§ 6.19.5.3. Intra frame mode info semantics

This syntax is used when coding an intra block within an [intra frame](#).

use_intrabc equal to 1 specifies that intra block copy is used for this block. `use_intrabc` equal to 0 specifies that intra block copy is not used.

§ 6.19.5.4. Read intra block copy semantics

This syntax is used when coding a motion vector for intra block copy.

intrabc_mode equal to 1 indicates that there is no motion vector difference. `intrabc_mode` equal to 0 indicates that a motion vector difference is present.

intrabc_drl_mode is used to select a predicted motion vector from the stack.

intrabc_precision is used to decide the motion vector precision for intra block copy.

morph_pred equal to 1 specifies that morphological prediction (which tries to adjust the brightness of the samples to match the context) is used for this block. `morph_pred` equal to 0 specifies that morphological prediction is not used.

If `morph_pred` is equal to 1, it is a requirement of bitstream conformance that `is_offset_mv_valid(-1, -1)` is equal to 1.

The function `is_offset_mv_valid` is defined as:

```
is_offset_mv_valid( dx, dy ) {
    offsetMv[0] = BlockMvs[0][0] + dy * 8
    offsetMv[1] = BlockMvs[0][1] + dx * 8
    return is_mv_valid( offsetMv )
}
```

NOTE: This constraint ensures that the extra reference pixels fetched are also valid for intra block copy prediction.

§ 6.19.5.5. Read intra Y mode semantics

use_dpcm_y equal to 1 specifies that Differential Pulse Code Modulation (DPCM) is used for luma prediction. **use_dpcm_y** equal to 0 specifies that DPCM is not used for luma.

dpcm_mode_y is used to compute the direction for intra prediction when using DPCM.

y_mode_set equal to 0 specifies that **y_mode_index** is present. **y_mode_set** equal to 1 specifies that **y_second_mode** is present.

y_mode_index and **y_mode_offset** are used to send the first set of YMode choices.

y_second_mode is used to send the second set of YMode choices.

fsc_mode is used to control if the block uses forward skip coding of the coefficients and the type of transform.

mrl_index specifies the distance of the reference samples used for intra prediction.

mrl_sec_index equal to 1 specifies that the block uses a secondary intra prediction. **mrl_sec_index** equal to 0 specifies that only primary intra prediction is used.

YMode specifies the direction of intra prediction filtering:

YMode	Name of YMode
0	DC_PRED
1	V_PRED
2	H_PRED
3	D45_PRED
4	D135_PRED
5	D113_PRED
6	D157_PRED
7	D203_PRED
8	D67_PRED
9	SMOOTH_PRED
10	SMOOTH_V_PRED
11	SMOOTH_H_PRED
12	PAETH_PRED

AngleDeltaY is computed from `y_mode_index`, `y_mode_offset`, and `y_second_mode` to produce the final luma angle offset value, which may be positive or negative.

§ 6.19.5.6. Read intra UV mode semantics

use_dpcm_uv equal to 1 specifies that Differential Pulse Code Modulation (DPCM) is used for chroma prediction. `use_dpcm_uv` equal to 0 specifies that DPCM is not used for chroma.

dpcm_mode_uv is used to compute the direction for intra prediction when using DPCM.

is_cfl equal to 1 specifies that chroma from luma ([CFL](#)) prediction is used for chroma components. `is_cfl` equal to 0 specifies that CFL prediction is not used.

uv_mode and **uv_mode_idx** are used to compute the UVMode.

It is a requirement of bitstream conformance that `uv_mode_idx` is less than or equal to 5.

UVMode specifies the chrominance intra prediction mode using values with the same interpretation as in the semantics for YMode, with an additional mode UV_CFL_PRED.

UVMode	Name of UVMode
0	DC_PRED
1	V_PRED
2	H_PRED
3	D45_PRED
4	D135_PRED
5	D113_PRED
6	D157_PRED
7	D203_PRED
8	D67_PRED
9	SMOOTH_PRED
10	SMOOTH_V_PRED
11	SMOOTH_H_PRED
12	PAETH_PRED
13	UV_CFL_PRED

AngleDeltaUV is computed from `uv_mode` and may be positive or negative.

§ 6.19.5.7. Intra segment ID semantics

Lossless is a variable which, if equal to 1, indicates that the block is coded using a special reversible transform designed for encoding frames that are bit-identical with the original frames.

§ 6.19.5.8. Read segment ID semantics

seg_id_ext_flag and **segment_id** specify which segment is associated with the current intra block being decoded. It is first read from the stream, and then postprocessed based on the predicted segment id.

It is a requirement of bitstream conformance that the postprocessed value of `segment_id` (i.e., the value returned by `neg_deinterleave`) is in the range 0 to `LastActiveSegId` (inclusive of endpoints).

§ 6.19.5.9. Skip mode semantics

skip_mode equal to 1 indicates that this block will use some default settings (that correspond to [compound prediction](#)) and so most of the [mode info](#) is skipped. `skip_mode` equal to 0 indicates that the [mode info](#) is not skipped.

§ 6.19.5.10. Skip semantics

skip_flag equal to 0 indicates that there can be some [transform coefficients](#) to read for this block; `skip_flag` equal to 1 indicates that there are no transform coefficients.

§ 6.19.5.11. Quantizer index delta semantics

delta_q_abs specifies the absolute value of the quantizer index delta value being decoded. If `delta_q_abs` is equal to `DELTA_Q_SMALL`, the value is encoded using `delta_q_rem_bits` and `delta_q_abs_bits`.

delta_q_rem_bits and **delta_q_abs_bits** encode the absolute value of the quantizer index delta value being decoded, where the absolute value of the quantizer index delta value is of the form:

$$(1 \ll \text{delta_q_rem_bits}) + \text{delta_q_abs_bits} + 1$$

delta_q_sign_bit equal to 0 indicates that the quantizer index delta value is positive; `delta_q_sign_bit` equal to 1 indicates that the quantizer index delta value is negative.

§ 6.19.6. Transform and quantization structures

§ 6.19.6.1. TX size semantics

lossless_tx_size equal to 1 specifies that a 4x4 or larger transform size is used for a lossless block. `lossless_tx_size` equal to 0 specifies that the transform size is constrained for lossless coding.

TxSize specifies the transform size to be used for this block:

TxSize	Name of TxSize
0	TX_4X4
1	TX_8X8
2	TX_16X16
3	TX_32X32
4	TX_64X64
5	TX_4X8
6	TX_8X4
7	TX_8X16
8	TX_16X8
9	TX_16X32
10	TX_32X16
11	TX_32X64

12	TX_64X32
13	TX_4X16
14	TX_16X4
15	TX_8X32
16	TX_32X8
17	TX_16X64
18	TX_64X16
19	TX_4X32
20	TX_32X4
21	TX_8X64
22	TX_64X8
23	TX_4X64
24	TX_64X4
255	TX_INVALID

NOTE: TxSize is determined for skipped intra blocks because TxSize controls the granularity of the [intra prediction](#).

§ 6.19.6.2. Block TX size semantics

LumaTxSizes is an array that holds the luma transform sizes.

LumaTxMiddle is an array that records whether the transform block was from the middle of a transform partition. (This information is important for intra prediction as top-right and bottom-left values are marked unavailable for middle blocks.)

§ 6.19.6.3. Read TX partition semantics

tx_do_partition equal to 1 specifies that the block is split into smaller transform sizes. tx_do_partition equal to 0 specifies that the block is not split any more.

tx_partition_type and **tx_2or3_partition_type** are used to indicate the transform partition.

txPartition specifies the transform partition as specified in [Table 6.23](#):

Table 6.23: txPartition values and names

txPartition	Name of txPartition
0	TX_PARTITION_NONE
1	TX_PARTITION_SPLIT
2	TX_PARTITION_HORZ
3	TX_PARTITION_VERT
4	TX_PARTITION_HORZ4
5	TX_PARTITION_VERT4
6	TX_PARTITION_HORZ5

It is a requirement of bitstream conformance that the return value of the function `set_tx_size` is not equal to `TX_INVALID`.

§ 6.19.7. Motion vector and prediction structures

§ 6.19.7.1. Inter frame mode info semantics

This reads syntax elements for blocks within an inter frame.

§ 6.19.7.2. Inter segment ID semantics

seg_id_predicted equal to 1 specifies that the `segment_id` is taken from the segmentation map. `seg_id_predicted` equal to 0 specifies that the syntax element `segment_id` is parsed.

NOTE: It is allowed for `seg_id_predicted` to be equal to 0 even if the value coded for the `segment_id` is equal to `predictedSegmentId`.

§ 6.19.7.3. Is inter semantics

is_inter equal to 0 specifies that the block is an intra block; `is_inter` equal to 1 specifies that the block is an inter block.

NOTE: When intra block copy is used within an inter frame, the syntax element `is_inter` is read as 0, but then modified to equal 1 as the motion vector prediction uses the `IsInters` array to detect blocks with motion vectors and intra block copy includes motion vectors.

NOTE: The semantics of `use_intrabc` are provided in [§ 6.19.5.3 Intra frame mode info semantics](#).

§ 6.19.7.4. Intra block mode info semantics

This syntax is used when coding an intra block within an [inter frame](#).

§ 6.19.7.5. Inter block mode info semantics

This syntax is used when coding an inter block.

tip_pred_mode is used to compute the YMode when using [TIP](#).

is_warp specifies that the YMode is either `WARPMV` or `WARP_NEWMV`.

warp_mv specifies that the YMode is set to `WARPMV`.

use_amvd specifies that an asymmetric motion vector difference is used.

single_mode, **is_joint**, **compound_mode_non_joint**, and **compound_mode_same_refs** specify how the motion vector used by inter prediction is obtained. An offset is added to compute YMode as follows:

YMode	Name of YMode
14	NEARMV

15	GLOBALMV
16	NEWMV
17	WARPMV
18	WARP_NEWMV
19	NEAR_NEARMV
20	NEAR_NEWMV
21	NEW_NEARMV
22	GLOBAL_GLOBALMV
23	NEW_NEWMV
24	JOINT_NEWMV

NOTE: The intra modes take values 0 to 13 so these YMode values start at 14.

use_optflow specifies that optical flow is used for this block.

use_bawp equal to 1 specifies that [BAWP](#) is used for this block for luma samples.

explicit_bawp equal to 1 specifies that [BAWP](#) scaling factor is based on OrderHints.

explicit_bawp_scale specifies the sign for [BAWP](#) scaling factor delta based on OrderHints.

use_bawp_chroma equal to 1 specifies that [BAWP](#) is used for this block for chroma samples.

warp_idx equal to 0 specifies that a particular warp reference candidate is used to compute the warp parameters.

warpmv_with_mvd specifies that a motion vector difference is present which will be used to compute the warp parameters.

jmvd_scale_mode specifies a parameter used while scaling motion vectors in joint mode.

use_most_probable_precision equal to 1 specifies that the frame level precision is used for motion vectors. **use_most_probable_precision** equal to 0 specifies that the syntax element **pb_mv_precision** is read to determine the precision.

pb_mv_precision is used to compute the precision for motion vectors.

cwp_idx is used to compute the compound weighting factor.

interp_filter specifies the type of filter used in inter prediction. Values 0..3 are allowed with the same interpretation as for **interpolation_filter**.

NOTE: The syntax element **interpolation_filter** from the [frame header info](#) can specify the type of filter to be used for the whole frame. If it is set to SWITCHABLE then the **interp_filter** syntax element is read from the bitstream for every inter block.

When all the syntax elements have been read in the inter block mode info syntax, if use_bru is equal to 1, it is a requirement of bitstream conformance that:

- RefFrame[0] is not equal to bru_ref
- RefFrame[1] is not equal to bru_ref

When all the syntax elements have been read in the inter block mode info syntax, if use_bru is equal to 1 and RefFrame[0] is equal to TIP_FRAME, it is a requirement of bitstream conformance that:

- ClosestPast is not equal to bru_ref
- ClosestFuture is not equal to bru_ref

§ 6.19.7.6. Read warp delta semantics

warp_delta_precision equal to 1 specifies that high precision warp parameters are used for the block. warp_delta_precision equal to 0 specifies that standard precision warp parameters are used.

warp_delta_param_low, **warp_delta_param_high**, and **warp_delta_param_sign** are used to compute a warp parameter as an offset from the predicted value.

§ 6.19.7.7. Read drl idx semantics

RefMvIdx specifies which candidate in the RefStackMv is used.

RefMvIdx0 specifies which candidate in the RefStack0Mvs is used.

RefMvIdx1 specifies which candidate in the RefStack1Mvs is used.

drl_mode is a bit sent for candidates in the motion vector stack to indicate if they are used. drl_mode equal to 0 means to use the current value of idx. drl_mode equal to 1 says to continue searching. DRL stands for "Dynamic Reference List".

§ 6.19.7.8. DIP mode info semantics

use_dip is a bit specifying whether or not data driven intra prediction can be used.

dip_mode and **dip_transpose** are parameters used in the data driven intra prediction process.

§ 6.19.7.9. Ref frames semantics

tip_mode equal to 1 specifies that Temporally Interpolated Prediction ([TIP](#)) is used for the block. tip_mode equal to 0 specifies that TIP is not used and regular inter prediction is applied.

comp_mode equal to 1 specifies that compound prediction is used for the block, blending predictions from two reference frames. comp_mode equal to 0 specifies that single reference prediction is used.

comp_mode	Name of comp_mode
0	SINGLE_REFERENCE
1	COMPOUND_REFERENCE

SINGLE_REFERENCE indicates that the inter block uses only a single reference frame to generate motion compensated prediction.

COMPOUND_REFERENCE indicates that the inter block uses compound mode.

RefFrame[0] specifies which frame is used to compute the predicted samples for this block:

RefFrame[0]	Name of ref_frame
7	TIP_FRAME
8	INTRA_FRAME

NOTE: Values from 0 to 6 are also allowed, but do not have a name. These values correspond to using different inter frames for reference.

RefFrame[1] specifies which additional frame is used in compound prediction:

RefFrame[1]	Name of ref_frame
-1	NONE (this block uses single prediction)
8	INTRA_FRAME (this block uses inter intra prediction)

NOTE: Values from 0 to 6 are also allowed, but do not have a name. These values correspond to using different inter frames for reference.

§ 6.19.7.10. Read compound ref semantics

If `read_compound_ref` is called, it is a requirement of bitstream conformance that `NumTotalRefs` is greater than 0.

comp_ref equal to 1 means that reference `ref` is used for inter prediction by this block.

§ 6.19.7.11. Read single ref semantics

If `read_single_ref` is called, it is a requirement of bitstream conformance that `NumTotalRefs` is greater than 0.

single_ref equal to 1 means that reference `ref` is used for inter prediction by this block.

§ 6.19.7.12. Assign MV semantics

mv_sign equal to 0 means that the motion vector difference is positive; **mv_sign** equal to 1 means that the motion vector difference is negative.

It is a requirement of bitstream conformance that whenever `assign_mv` returns, `is_mv_valid( BlockMvs[0] )` is equal to 1, where `is_mv_valid` is defined as:

```
is_mv_valid( mv ) {
    if ( !use_intrabc ) {
        return 1
    }
    bw = Block_Width[ MiSize ]
    bh = Block_Height[ MiSize ]
    bottomBorder = (mv[ 0 ] & 7) != 0 ? 1 : 0
    rightBorder = (mv[ 1 ] & 7) != 0 ? 1 : 0
    deltaRow = mv[ 0 ] >> 3
    deltaCol = mv[ 1 ] >> 3
}
```

```

srcTopEdge = MiRow * MI_SIZE + deltaRow
srcLeftEdge = MiCol * MI_SIZE + deltaCol
srcBottomEdge = srcTopEdge + bh + bottomBorder
srcRightEdge = srcLeftEdge + bw + rightBorder
if (HasChroma) {
    srcLeftEdge = ChromaMiCol * MI_SIZE + deltaCol
    srcTopEdge = ChromaMiRow * MI_SIZE + deltaRow
}
if ( srcTopEdge < MiRowStart * MI_SIZE ||
    srcLeftEdge < MiColStart * MI_SIZE ||
    srcBottomEdge > MiRowEnd * MI_SIZE ||
    srcRightEdge > MiColEnd * MI_SIZE ) {
    return 0
}
if ( allow_local_intrabc ) {
    tmpCol = MiCol
    tmpRow = MiRow
    if ( (!enable_sdp || !FrameIsIntra) && HasChroma ) {
        bw = Block_Width[ ChromaMiSize ]
        tmpCol = ChromaMiCol
        bh = Block_Height[ ChromaMiSize ]
        tmpRow = ChromaMiRow
    }
    tmpTopEdge = tmpRow * MI_SIZE + deltaRow
    tmpLeftEdge = tmpCol * MI_SIZE + deltaCol
    tmpBottomEdge = tmpTopEdge + bh - 1 + bottomBorder
    tmpRightEdge = tmpLeftEdge + bw - 1 + rightBorder
    if (check_valid_local_ibc(tmpLeftEdge, tmpTopEdge) &&
        check_valid_local_ibc(tmpRightEdge, tmpBottomEdge)) {
        return 1
    }
}
if (!allow_global_intrabc) {
    return 0
}
sbH = Block_Height[ SbSize ]
activeSbRow = (MiRow * MI_SIZE) / sbH
activeSb64Col = (MiCol * MI_SIZE) >> 6
srcSbRow = (srcBottomEdge - 1) / sbH
srcSb64Col = (srcRightEdge - 1) >> 6
activeSb64Row = (MiRow * MI_SIZE) >> 6
isBottomLeft = (activeSb64Col & 1) == 0 && (activeSb64Row & 1) == 1
if (AllowExtraIBCRange && isBottomLeft) {
    sb64Residual = -1
} else {
    sb64Residual = 0
}
totalSb64PerRow = ((MiColEnd - MiColStart - 1) >> 4) + 1
activeSb64 = activeSbRow * totalSb64PerRow + activeSb64Col
srcSb64 = srcSbRow * totalSb64PerRow + srcSb64Col
if ( srcSb64 >= activeSb64 - INTRABC_DELAY_SB64 - sb64Residual ) {
    return 0
}
gradient = INTRABC_DELAY_SB64 + (Block_Width[ SbSize ] / 64)
wfOffset = gradient * (activeSbRow - srcSbRow)
if ( srcSbRow > activeSbRow ||
    srcSb64Col >= activeSb64Col - INTRABC_DELAY_SB64 +
        wfOffset - sb64Residual ) {
    return 0
}
return 1
}
}

```

NOTE: The purpose of this function is to constrain the motion vectors used for intra BC in order that the data is fetched from parts of the tile that have already been decoded.

NOTE: The constraints when `allow_local_intrabc` is equal to 1 are intended to allow an implementation that stores the four most recently decoded 64x64 regions of the image in a cache.

The function `check_valid_local_ibc` (which checks if a location is within the allowed intra block copy buffers) is specified as:

```
check_valid_local_ibc( x, y ) {
    if ( (!enable_sdp || !FrameIsIntra) && HasChroma ) {
        actCol = ChromaMiCol
        actRow = ChromaMiRow
    } else {
        actCol = MiCol
        actRow = MiRow
    }
    if ( x >= actCol * MI_SIZE && y >= actRow * MI_SIZE ) {
        return 0
    }
    if ( !IBCCoded[y >> MI_SIZE_LOG2][x >> MI_SIZE_LOG2] ) {
        return 0
    }
    bufCol = x >> IBC_BUFFER_SIZE_LOG2
    bufRow = y >> IBC_BUFFER_SIZE_LOG2
    bufIdx = ibc_buffer_index(bufRow, bufCol)
    inCurrent = bufCol == IBCBufferCurCol && bufRow == IBCBufferCurRow
    if (!inCurrent) {
        if ( !IBCBufferValid[bufIdx] ||
            bufCol != IBCBufferCol[bufIdx] ||
            bufRow != IBCBufferRow[bufIdx] ) {
            return 0
        }
    }
    if ( bufIdx == ibc_buffer_index(IBCBufferCurRow, IBCBufferCurCol) ) {
        if (!inCurrent) {
            colY = (y & (IBC_BUFFER_SIZE - 1)) |
                (IBCBufferCurRow << IBC_BUFFER_SIZE_LOG2)
            colX = (x & (IBC_BUFFER_SIZE - 1)) |
                (IBCBufferCurCol << IBC_BUFFER_SIZE_LOG2)
            if ( IBCCoded[ colY >> MI_SIZE_LOG2 ][ colX >> MI_SIZE_LOG2 ] ) {
                return 0
            }
        }
    }
    return 1
}
```

get_warp_motion_vector is a function call that indicates the get warp motion vector process specified in [§ 7.12.2.2 Get warp motion vector process](#) is invoked.

§ 6.19.7.13. Read motion mode semantics

use_extend_warp equal to 1 means that EXTENDWARP is used.

use_local_warp equal to 1 means that LOCALWARP is used.

§ 6.19.7.14. Read inter intra semantics

inter_intra equal to 1 specifies that an inter prediction is blended with an intra prediction.

warp_inter_intra equal to 1 specifies that an inter prediction is blended with an intra prediction for a WARPMV block.

interintra_mode specifies the type of [intra prediction](#) to be used:

Table 6.24: interintra_mode values and names

interintra_mode	Name of interintra_mode
0	II_DC_PRED
1	II_V_PRED
2	II_H_PRED
3	II_SMOOTH_PRED

wedge_interintra equal to 1 specifies that wedge blending is used. **wedge_interintra** equal to 0 specifies that intra blending is used.

§ **6.19.7.15. Read compound type semantics**

comp_group_idx equal to 0 indicates that the **compound_type** syntax element is not present and that an averaging scheme is used for blending. **comp_group_idx** equal to 1 indicates that the **compound_type** syntax element is present.

compound_type specifies how the two predictions are blended together:

compound_type	Name of compound_type
0	COMPOUND_WEDGE
1	COMPOUND_DIFFWTD
2	COMPOUND_AVERAGE
3	COMPOUND_INTRA

NOTE: COMPOUND_AVERAGE and COMPOUND_INTRA cannot be directly signaled with the **compound_type** syntax element but are inferred from other syntax elements.

wedge_sign specifies the sign of the wedge blend.

mask_type specifies the type of mask to be used during blending:

mask_type	Name of mask_type
0	UNIFORM_45
1	UNIFORM_45_INV

§ **6.19.7.16. Read refine mv semantics**

use_refinemv indicates that motion vector refinement is used for this block.

DecidedAgainstRefinemv indicates that **use_refinemv** was originally set to 1 in the bitstream, but later cleared due to incompatible compound weights. In this case the reference code does not apply motion vector refinement, but uses a different interpolation filter.

§ 6.19.7.17. Read wedge mode semantics

wedge_quad and **wedge_angle** are used to specify the wedge angle.

wedge_dist1 specifies the distance to the wedge for angles where a distance of 0 is allowed.

wedge_dist2 specifies the distance to the wedge for angles where a distance of 0 is not allowed.

wedgeAngle gives the angle of the wedge as specified in [Table 6.25](#):

Table 6.25: wedgeAngle values and names

wedgeAngle	Name of wedgeAngle
0	WEDGE_0
1	WEDGE_14
2	WEDGE_27
3	WEDGE_45
4	WEDGE_63
5	WEDGE_90
6	WEDGE_117
7	WEDGE_135
8	WEDGE_153
9	WEDGE_166
10	WEDGE_180
11	WEDGE_194
12	WEDGE_207
13	WEDGE_225
14	WEDGE_243
15	WEDGE_270
16	WEDGE_297
17	WEDGE_315
18	WEDGE_333
19	WEDGE_346

§ 6.19.7.18. MV semantics

MvCtx is used to determine which CDFs to use for the motion vector syntax elements.

mv_joint specifies which components of the motion vector difference are non-zero:

mv_joint	Name of mv_joint	Changes row	Changes col
0	MV_JOINT_ZERO	No	No
1	MV_JOINT_HNZVZ	No	Yes
2	MV_JOINT_HZVNZ	Yes	No
3	MV_JOINT_HNZVNZ	Yes	Yes

The motion vector difference is added to the PredMvs to compute the final motion vector in BlockMvs.

shell_set, **shell_class**, and **joint_shell_last_two_classes** are used to specify the class of the motion vector difference. A higher class means that the motion vector difference represents a larger update.

shell_offset_low_class is used to compute shellClassOffset when shell_class is equal to 0 or 1.

shell_offset_class2 and **shell_offset_class2_high** are used to compute shellClassOffset when shell_class is equal to 2.

shell_offset_other_class is used to compute shellClassOffset when shell_class is greater than 2.

col_mv_greater is used as part of a truncated unary coding for the variable col.

col_remainder is used to increment the variable col if the maximum unary value has been reached.

shellIndex is the sum of both motion vector components.

col_mv_index specifies which component of the motion vector will be computed based on the known sum. The other component will be set equal to the variable col.

[§](#) **6.19.7.19.** *MV component semantics*

amvd_index is used to compute the size of the motion vector difference via a table lookup.

[§](#) **6.19.7.20.** *Compute prediction semantics*

The prediction for inter and inter intra blocks is triggered within compute_prediction. However, intra prediction is done at the transform block granularity so predict_intra is also called from transform_block.

predW and **predH** are variables containing the smallest size that can be used for inter prediction. (This size may be increased for chroma blocks if not all blocks use inter prediction.)

predict_inter is a function call that indicates the conceptual point where inter prediction happens. When this function is called, the inter prediction process specified in [§ 7.13.3 Inter prediction process](#) is invoked.

predict_intra is a function call that indicates the conceptual point where [intra prediction](#) happens. When this function is called, the [intra prediction](#) process specified in [§ 7.13.2 Intra prediction process](#) is invoked.

wedge_mask is a function call that indicates the wedge mask process specified in [§ 7.13.3.27 Wedge mask process](#) is invoked.

intra_mode_variant_mask is a function call that indicates the intra mode variant mask process specified in [§ 7.13.3.29 Intra mode variant mask process](#) is invoked.

mask_blend is a function call that indicates the mask blend process specified in [§ 7.13.3.30 Mask blend process](#) is invoked.

NOTE: The `predict_inter`, `predict_intra`, `wedge_mask`, `intra_mode_variant_mask`, `mask_blend` functions do not affect the syntax decode process. `predict_inter` does affect the `SubMvs` array which is used by the motion vector prediction process, but motion vector prediction is not required for syntax decode.

NOTE: The chroma residual block size is always at least 4 in width and height. This means that no transform width or height smaller than 4 is required. As such, a chroma residual may actually cover several luma blocks.

§ 6.19.7.21. Residual semantics

The residual consists of a number of transform blocks.

If the block is wider or higher than 64 luma samples, then the residual is split into 64 by 64 chunks.

§ 6.19.7.22. Transform block semantics

reconstruct is a function call that indicates the conceptual point where [inverse transform](#) and [reconstruction](#) happens. When this function is called, the reconstruction process specified in [§ 7.14.3 Reconstruct process](#) is invoked.

predict_palette is a function call that indicates the conceptual point where palette prediction happens. When this function is called, the palette prediction process specified in [§ 7.13.4 Palette prediction process](#) is invoked.

predict_chroma_from_luma is a function call that indicates the conceptual point where predicting chroma from luma happens. When this function is called, the predict chroma from luma process specified in [§ 7.13.5 Predict chroma from luma process](#) is invoked.

DeblockingTxSizes is an array that stores the transform size for each plane and position for use in deblocking filtering. `DeblockingTxSizes[ plane ][ row ][ col ]` stores the transform size where row and col are in units of 4x4 samples.

NOTE: The transform size is always equal for planes 1 and 2.

§ 6.19.7.23. Coefficients semantics

TxTypes is an array which stores at a 4x4 luma sample granularity the transform type to be used.

NOTE: The transform type is only read for luma transform blocks, the chroma uses the transform type for a corresponding luma block. Chroma blocks will only use transform types that have been written for the current residual block.

Quant is an array storing the quantised coefficients for the current [transform block](#).

It is a requirement of bitstream conformance that the values written into `Quant` are greater than $-1 \ll 20$ and less than $1 \ll 20$.

QuantSign is an array storing the sign of the quantised coefficients for the current transform block, or zero for zero coefficients.

NOTE: It is possible for QuantSign[pos] to be not equal to zero when Quant[pos] is equal to zero as the quantised coefficients can wrap around.

all_zero equal to 1 specifies that all coefficients are zero.

eob_extra and **eob_extra_bit** specify the position of the last non-zero coefficient by being used to compute the variable eob.

cctx_type specifies the angle for the cross component transform:

cctx_type	Name of cctx_type
0	CCTX_NONE
1	CCTX_45
2	CCTX_30
3	CCTX_60
4	CCTX_MINUS45
5	CCTX_MINUS30
6	CCTX_MINUS60

eob_pt_16, eob_pt_32, eob_pt_64, eob_pt_128, eob_pt_256, eob_pt_512, eob_pt_1024, eob_pt_256_extra, eob_pt_512_extra, eob_pt_1024_extra: syntax elements used to compute eob.

It is a requirement of bitstream conformance that eob_pt_512_extra is not equal to 3.

eob is a variable that indicates the index of the end of block. This index is equal to one plus the index of the last non-zero coefficient.

coeff_base_eob is a syntax element used to compute the base level of the last non-zero coefficient.

NOTE: The base level is set to coeff_base_eob plus 1 because this coefficient is known to be non-zero.

coeff_base_bob is a syntax element used to compute the base level of the first non-zero coefficient.

coeff_base specifies the base level of a coefficient.

coeff_base_idtx specifies the base level of a coefficient when using forward skip coding.

idtx_sign specifies the sign of the coefficients when using forward skip coding.

dc_sign specifies the sign of the [DC coefficient](#).

dc_sign_horz_vert specifies the sign of the DC coefficients when using horizontal or vertical transform classes.

sign_bit specifies the sign of a non-zero [AC coefficient](#).

coeff_br specifies an increment to the coefficient.

coeff_br_idtx specifies an increment to the coefficient when using forward skip coding.

AboveLevelContext and **LeftLevelContext** are arrays that store at a 4 sample granularity the cumulative sum of coefficient levels.

AboveDcContext and **LeftDcContext** are arrays that store at a 4 sample granularity 2 bits signaling the sign of the [DC coefficient](#) (zero being counted as a separate sign).

§ 6.19.7.24. Read quantized coefficient semantics

q_length_bit is used to specify the prefix of the extra bits required to code the coefficient.

golomb_length_bit is used to compute the number of extra bits required to code the coefficient.

If length is equal to 20, it is a requirement of bitstream conformance that **golomb_length_bit** is equal to 1.

coeff_rem specifies the values of the extra bits.

§ 6.19.7.25. Read CFL alphas semantics

cfl_mhccp and **cfl_index** specify how the chroma from luma parameters are prepared:

Table 6.26: cfl_index values and names

cfl_index	Name of cfl_index
0	CFL_EXPLICIT
1	CFL_DERIVED_ALPHA
2	CFL_MULTI

cfl_mh_dir specifies a direction used by [MHCCP](#).

cfl_alpha_signs contains the sign of the alpha values for U and V packed together into a single syntax element with 8 possible values as specified in [Table 6.27](#): (The combination of two zero signs is prohibited as it is redundant with DC [intra prediction](#).)

Table 6.27: cfl_alpha_signs values and sign interpretations

cfl_alpha_signs	Name of signU	Name of signV
0	CFL_SIGN_ZERO	CFL_SIGN_NEG
1	CFL_SIGN_ZERO	CFL_SIGN_POS
2	CFL_SIGN_NEG	CFL_SIGN_ZERO
3	CFL_SIGN_NEG	CFL_SIGN_NEG
4	CFL_SIGN_NEG	CFL_SIGN_POS
5	CFL_SIGN_POS	CFL_SIGN_ZERO
6	CFL_SIGN_POS	CFL_SIGN_NEG
7	CFL_SIGN_POS	CFL_SIGN_POS

signU contains the sign of the alpha value for the U component:

signU	Name of signU
0	CFL_SIGN_ZERO

1	CFL_SIGN_NEG
2	CFL_SIGN_POS

signV contains the sign of the alpha value for the V component with the same interpretation as for signU.

cfl_alpha_u contains the absolute value of alpha minus one for the U component.

cfl_alpha_v contains the absolute value of alpha minus one for the V component.

CflAlphaU contains the signed value of the alpha component for the U component.

CflAlphaV contains the signed value of the alpha component for the V component.

§ 6.19.8. Coding tools structures

§ 6.19.8.1. Palette mode info semantics

has_palette_y is a boolean value specifying whether a palette is encoded for the Y plane.

palette_size_y_minus_2 is used to compute PaletteSizeY.

PaletteSizeY is a variable holding the Y plane palette size.

use_palette_color_cache_y, if equal to 1, indicates that for a particular palette entry in the luma palette, the cached entry is used.

palette_colors_y is an array holding the Y plane palette colors.

palette_num_extra_bits_y is used to calculate the number of bits used to store each palette delta value for the luma palette.

palette_delta_y is a delta value for the luma palette.

§ 6.19.8.2. Transform type semantics

set specifies the transform set.

is_inter	set	Name of transform set
Don't care	0	TX_SET_DCTONLY
Don't care	1	TX_SET_WIDE_64
Don't care	2	TX_SET_HIGH_64
Don't care	3	TX_SET_WIDE_32
Don't care	4	TX_SET_HIGH_32
0	5	TX_SET_INTRA_1
0	6	TX_SET_INTRA_2
1	5	TX_SET_INTER_1
1	6	TX_SET_INTER_2
1	7	TX_SET_DCT_IDTX
1	8	TX_SET_DCT_IDTX_IDDCT

lossless_inter_tx_type is used to specify the transform type for 4 by 4 lossless inter transform blocks.

is_long_side_dct equal to 1 specifies that the long side of a block uses Discrete Cosine Transform ([DCT](#)). **is_long_side_dct** equal to 0 specifies that the long side uses an alternative transform.

inter_tx_type and **inter_tx_type_offset** specify the transform type for inter blocks.

intra_tx_type is used in the computation of the transform type for intra blocks. The transform type depends on **intra_tx_type** and the intra direction for the block.

sec_tx_type specifies the secondary transform type.

most_probable_stx_set is used to compute the kernel used for the secondary transform.

§ 6.19.8.3. *Palette tokens semantics*

palette_direction equal to 0 specifies that the palette is read row by row. **palette_direction** equal to 1 specifies that the palette is read column by column.

identity_row_y equal to 0 specifies that each sample is coded individually. **identity_row_y** equal to 1 specifies that each line of luma samples in the block contains a constant color. **identity_row_y** equal to 2 specifies that each line is copied from the previous line.

It is a requirement of bitstream conformance that **i** is greater than 0 if **identity_row_y** is equal to 2.

NOTE: When **palette_direction** is equal to 0, the lines mentioned in **identity_row_y** refer to rows. When **direction** is equal to 1, the lines refer to columns.

color_index_map_y holds the index in **palette_colors_y** for the block's Y plane top left sample.

palette_color_idx_y holds the index in **ColorOrder** for a sample in the block's Y plane.

§ 6.19.8.4. *Palette color context function semantics*

ColorOrder is an array holding the mapping from an encoded index to the palette. **ColorOrder** is ranked in order of frequency of occurrence of each color in the neighborhood of the current block, weighted by closeness to the current block.

ColorContextHash is a variable derived from the distribution of colors in the neighborhood of the current block, which is used to determine the probability context used to decode **palette_color_idx_y** and **palette_color_idx_uv**.

§ 6.19.9. **Filtering structures**

§ 6.19.9.1. *Read CDEF semantics*

cdef_idx specifies which CDEF filtering parameters are used for a particular 64 by 64 block. A value of -1 means that CDEF is disabled for that block.

cdef_index0 specifies that **cdef_idx** is equal to 0.

cdef_index_minus_1 plus 1 specifies the value of **cdef_idx**.

§ 6.19.9.2. Read CCSO semantics

ccso_blk equal to 1 specifies that [CCSO](#) filtering is enabled for a particular plane and [CCSO](#) block. **ccso_blk** equal to 0 specifies that [CCSO](#) is disabled for that block.

§ 6.19.9.3. Read GDF semantics

use_gdf equal to 1 specifies that Guided Detail Filter ([GDF](#)) is enabled for a particular block. **use_gdf** equal to 0 specifies that GDF is disabled for that block.

§ 6.19.9.4. Read loop restoration semantics

This contains syntax for any new restoration units that are covered.

§ 6.19.9.5. Read loop restoration unit semantics

use_wiener_ns equal to 1 specifies that the non-separable Wiener filter is used for loop restoration. **use_wiener_ns** equal to 0 specifies that the non-separable Wiener filter is not used.

use_pc_wiener equal to 1 specifies that the pixel classified Wiener filter is used for loop restoration. **use_pc_wiener** equal to 0 specifies that the pixel classified filter is not used.

flex_restoration_type equal to 1 specifies that a particular enabled loop restoration tool is used for the restoration unit. **flex_restoration_type** equal to 0 specifies that the restoration tool is not used for this unit.

§ 6.19.9.6. Read Wiener NS semantics

matchIndices is used to determine the reference values for the Wiener coefficients.

use_alt_group equal to 0 specifies that the predicted group is used. **use_alt_group** equal to 1 specifies that a different group to the predicted group is used.

group_bit is used when there is more than one alternative group.

merged_param equal to 1 specifies that a previous set of parameters is used for loop restoration. **merged_param** equal to 0 specifies that new parameters are signaled for this restoration unit.

use_bank indicates that a particular bank of parameters is used for loop restoration.

wiener_ns_length is used to compute the number of coefficients to read.

wiener_ns_uv_sym equal to 1 specifies that the chroma filter is symmetric and fewer coefficients need to be signaled. **wiener_ns_uv_sym** equal to 0 specifies that the chroma filter is asymmetric and all coefficients are signaled.

wiener_ns_base is used to compute the base level of a coefficient.

wiener_ns_rem is used to provide an increment for a coefficient.

[↑ Back to Table of Contents](#)

§ 7. Decoding process

§ 7.1. General decoding process

When `film_grain_params_present` is equal to 0, decoders shall produce output frames that are identical in all respects and have the same output order as those produced by the decoding process specified herein.

When `film_grain_params_present` is equal to 1, a decoder shall implement a film grain synthesis process that modifies the output arrays OutY, OutU, OutV. The reference film grain synthesis process is described in [§ 7.21.7 Film grain synthesis process](#).

When `film_grain_params_present` is equal to 1, a conformant decoder shall satisfy at least one of the following two options:

1. A conformant decoder shall produce output frames that are identical in all respects and have the same output order as those produced by the decoding process specified herein including applying the exact film grain synthesis process as specified in [§ 7.21.7 Film grain synthesis process](#).
2. A conformant decoder shall produce intermediate frames that are identical in all respects and have the same order as the frames produced by the process specified in [§ 7.21.2 Intermediate output preparation process](#). In addition to that, a conformant decoder shall produce output frames that are in the same order and do not have perceptually significant differences with the frames produced by the reference film grain synthesis process specified in [§ 7.21.7 Film grain synthesis process](#) when applied to the input frames of the film grain synthesis process with the film grain parameters signaled for these frames. The decoder may also include optional processing steps which are applied to the intermediate frames produced by the process specified in [§ 7.21.2 Intermediate output preparation process](#) and before the film grain synthesis process, resulting in the input frames of the film grain synthesis process. Such optional processing steps are beyond the scope of this specification. Otherwise, the intermediate frames are the input frames of the film grain synthesis process. The definition of "perceptually significant differences" is beyond the scope of this specification and may be specified, for example, by a service provider as part of their accreditation program. The film grain synthesis process applied by a conformant decoder should be feature complete with regards to the reference film grain synthesis process of [§ 7.21.7 Film grain synthesis process](#) including scaling strength of the film grain as a function of intensity according to the signaled parameters, same maximum AR lag, and similar modeling of correlation between luma and chroma and smoothing of transitions between blocks of grain when applicable.

NOTE: To ensure conformance, decoder manufacturers are advised to implement the film grain synthesis process as specified in [§ 7.21.7 Film grain synthesis process](#). One reason to choose the second conformance option is implementation of optional processing steps between the output of [§ 7.21.2 Intermediate output preparation process](#) and the film grain synthesis process, in which case there may be minor differences in the output with the reference film grain synthesis process of [§ 7.21.7 Film grain synthesis process](#). Examples of these optional processing steps are algorithms improving output frame quality, such as de-banding filtering and coding artefacts removal.

NOTE: Some applications, such as transcoding from AV2 to AV2, may use intermediate output frames of [§ 7.21.2 Intermediate output preparation process](#) for transcoding. In such cases, the original film grain synthesis information may be adapted and inserted in the transcoded bitstream.

The input to this process is a sequence of open bitstream units (OBUs).

The output from this process is a sequence of decoded frames.

For each OBU in turn the syntax elements are extracted as specified in [§5.2 OBU syntax](#).

After all OBUs have been decoded, the flush implicit output frames process specified in [§7.21.5 Flush implicit output frames process](#) is invoked with 0 as input (this outputs any remaining frames).

The syntax tables include function calls indicating when the remaining decode processes are triggered.

A singestream can be decoded directly via this decoding process.

Each stream within a multistream can be decoded by decoding the corresponding extracted OBUs.

NOTE: Although the decoding process and semantics are defined for a single stream, a decoder implementation may choose to decode multiple extended layers at the same time as long as the output is equivalent.

The corresponding OBUs can be extracted from a multistream for stream *x* by concatenating all OBUs that satisfy either of the following conditions:

- obu_xlayer_id equal to GLOBAL_XLAYER_ID and obu_type is not equal to OBU_MSDO.
- OBUs with obu_xlayer_id corresponding to the chosen stream.

NOTE: In a coded video multistream sequence that contains an OBU with obu_type equal to OBU_MSDO, the obu_xlayer_id that corresponds to stream *x* is given by sub_xlayer_id[*x*]. Otherwise, a global LCR must be present and activated, and the obu_xlayer_id that corresponds to stream *x* is given by the *x*-th non-zero bit in lcr_xlayer_map. (For example, if lcr_xlayer_map is equal to 8, which is equal to $1 \ll 3$, then stream 0 would correspond to choosing OBUs with obu_xlayer_id equal to 3.)

§ 7.2. Decode frame wrapup process

This process is triggered by a call to decode_frame_wrapup from within the syntax tables.

At this stage, all the tile level decode has been done, and this process performs any frame level decode that is required.

The frame level filters are applied as follows:

- If TipFrameMode is equal to TIP_FRAME_AS_OUTPUT, the deblocking filter is applied by the following ordered steps:
 1. If apply_deblocking_filter_tip is equal to 1, the deblocking filter for [TIP](#) process specified in [§7.16 Deblocking filter for TIP process](#) is invoked.
 2. LrFrame is set equal to CurrFrame.
- Otherwise, if bru_inactive is equal to 1, the frame is updated by the following ordered steps:
 1. LrFrame is set equal to a copy of FrameStore[ref_frame_idx[bru_ref]].

2. `MfRefFrames[ y8 ][ x8 ][ list ]` is set equal to `bru_ref` for `y8 = 0..(MiRows>>1)-1`, `x8 = 0..(MiCols>>1)-1`, for `list=0..1`.
 3. `MfMvs[ y8 ][ x8 ][ list ][ comp ]` is set equal to 0 for `y8 = 0..(MiRows>>1)-1`, `x8 = 0..(MiCols>>1)-1`, for `list=0..1`, for `comp=0..1`.
- Otherwise, if `ShowExistingFrame` is equal to 0, the process first performs any post processing filtering by the following ordered steps:
 1. If `apply_deblocking_filter[ 0 ]` is not equal to 0 or `apply_deblocking_filter[ 1 ]` is not equal to 0, the deblocking filter process specified in [§ 7.17 Deblocking filter process](#) is invoked (this process modifies the contents of `CurrFrame`).
 2. The CDEF process specified in [§ 7.18 CDEF process](#) is invoked (this process takes `CurrFrame` and produces `CdefFrame`).
 3. The CCSO process specified in [§ 7.19 CCSO process](#) is invoked (this process takes `CurrFrame` and modifies `CdefFrame`).
 4. The loop restoration process specified in [§ 7.20 Loop restoration process](#) is invoked (this process takes `CurrFrame` and `CdefFrame` and produces `LrFrame`).
 5. If `segmentation_enabled` is equal to 1 and `segmentation_update_map` is equal to 0, `SegmentIds[ row ][ col ]` is set equal to `PrevSegmentIds[ row ][ col ]` for `row = 0..MiRows-1`, for `col = 0..MiCols-1`.
 6. If `use_bru` is equal to 1, it is a requirement of bitstream conformance that `bru_region_valid()` is equal to 1.

All the syntax elements that can be read in `film_grain_model` and `film_grain_config` should be saved into an area of memory indexed by `NUM_REF_FRAMES` (this is the same as calling the `save_grain_params` function specified in section [§ 7.23 Reference frame update process](#) with an input of `NUM_REF_FRAMES`). (This saving is needed because the reference frame update process can cause previous frames to be reloaded and film grain applied.)

The reference frame update process as specified in [§ 7.23 Reference frame update process](#) is invoked (this process saves the current frame state into the reference frames and can cause frames to be output).

The frames to output are decided as follows:

- If `ShowExistingFrame` is equal to 1, the output frame buffers process specified in [§ 7.21.6 Output frame buffers process](#) is invoked with `(derive_sef_order_hint ? frame_to_show_map_idx : -1)` as input.
- Otherwise, if `immediate_output_frame` is equal to 1, if the current frame has not already been output, the output frame buffers process specified in [§ 7.21.6 Output frame buffers process](#) is invoked with -1 as input.

NOTE: When `immediate_output_frame` is equal to 1, the current frame is stored into the frame buffers by the reference frame update process. However, this process can trigger the output of frames which can themselves trigger the output of the current frame.

The function `bru_region_valid` is used to check that `BruModes` has a valid pattern of blocks.

```
bru_region_valid() {
    sbSize4 = Num_4x4_Blocks_Wide[ SbSize ]
    num = 0
    sbRows = (MiRows + sbSize4 - 1) / sbSize4
    sbCols = (MiCols + sbSize4 - 1) / sbSize4
    for( r = 0; r < sbRows; r++ ) {
        for( c = 0; c < sbCols; c++ ) {
            if ( BruModes[ r * sbSize4 ][ c * sbSize4 ] == BRU_ACTIVE ) {
                left[num] = c - 1
                right[num] = c + 1
                top[num] = r - 1
                bottom[num] = r + 1
                active[num] = 1
                num = num + 1
            }
        }
    }
    changed = 1
    while( changed ) {
        changed = 0
        for( a = 0; a < num; a++ ) {
            for( b = a + 1; b < num; b++ ) {
                if ( active[a] && active[b] &&
                    !( right[a] < left[b] ||
                      right[b] < left[a] ||
                      bottom[a] < top[b] ||
                      bottom[b] < top[a] ) ) {
                    left[a] = Min( left[a], left[b] )
                    right[a] = Max( right[a], right[b] )
                    top[a] = Min( top[a], top[b] )
                    bottom[a] = Max( bottom[a], bottom[b] )
                    active[b] = 0
                    changed = 1
                }
            }
        }
    }
    for( a = 0; a < num; a++ ) {
        if ( active[a] ) {
            for( r = top[ a ]; r <= bottom[ a ]; r++ ) {
                for( c = left[ a ]; c <= right[ a ]; c++ ) {
                    row = r * sbSize4
                    col = c * sbSize4
                    if ( row >= 0 && row < MiRows && col >= 0 && col < MiCols ) {
                        if ( BruModes[ row ][ col ] == BRU_INACTIVE ) {
                            return 0
                        }
                        if ( r == top[ a ] ||
                            r == bottom[ a ] ||
                            c == left[ a ] ||
                            c == right[ a ] ) {
                            if ( BruModes[ row ][ col ] != BRU_SUPPORT ) {
                                return 0
                            }
                        }
                    }
                }
            }
        }
    }
    return 1
}
```

NOTE: `bru_region_valid` merges rectangles of `BRU_ACTIVE` blocks together if the rectangles (including a one block wide boundary) overlap, and then checks that there are no inactive blocks inside each merged rectangle and that the edge of each merged rectangle is either off-screen or marked as support.

§ 7.3. Ordering of OBUs

§ 7.3.1. General

A bitstream conforming to this specification consists of one or more coded video sequences.

A coded video sequence consists of one or more [temporal units](#). A temporal unit consists of at least one coded output frame unit belonging to one coded extended layer unit. The definition of a coded output frame unit, coded non-output frame unit and coded extended layer unit are provided in sub-sections [§ 7.3.3 Coded output frame unit](#), [§ 7.3.4 Coded non-output frame unit](#), and [§ 7.3.6 Coded extended layer unit](#), respectively. The temporal unit is further specified in sub-section [§ 7.3.7 Temporal unit](#).

A [coded multistream video sequence](#) is a set of coded video sequences across two or more extended layers that satisfies the following requirements:

1. The temporal units of the coded video sequences collectively contain OBUs with two or more distinct non-global values of `obu_xlayer_id`.
2. An OBU with `obu_type` equal to `OBU_MSDO` or an activated global layer configuration record OBU is present as specified in [Annex A.2 Profiles](#).
3. When an OBU with `obu_type` equal to `OBU_MSDO` is present, it is present in each temporal unit that contains a random access point.
4. For each OBU in a [coded multistream video sequence](#) with `obu_xlayer_id` not equal to `GLOBAL_XLAYER_ID`, `obu_xlayer_id` must be equal to some value of `sub_xlayer_id` in the preceding `OBU_MSDO` or to some value of `LcrXLayerID` in the activated global LCR.
5. All extended layers within a temporal unit share the same output time.
6. The coded extended layer units from different extended layers within a temporal unit shall appear in ascending order of `obu_xlayer_id`.
7. The extracted bitstream for each individual stream forms a valid bitstream.

NOTE: Not all extended layers are required to be present in every temporal unit. For example, in a multistream bitstream where extended layers operate at different frame rates, a temporal unit may contain coded extended layer units for only a subset of the extended layers. When multiple extended layers are present in a temporal unit, they are required to share the same output time. An encoder may use the show existing frame mechanism to satisfy this requirement when extended layers use different coding structures.

NOTE: The coded video sequences and random access points do not have to be aligned across different extended layers unless the OrderHint matching constraint is enabled via `multistream_doh_constraint_flag` or `lcr_doh_constraint_flag` (see [§ 7.3.7 Temporal unit](#) and [§ 7.4.6 Multistream Random Access](#)).

§ 7.3.2. Coded multistream video sequence boundaries

A [coded multistream video sequence](#) begins at a temporal unit that contains an OBU with `obu_type` equal to `OBU_CLOSED_LOOP_KEY` for at least one extended layer and satisfies one of the following conditions:

1. No coded multistream video sequence is currently active and an OBU with `obu_type` equal to `OBU_MSDO` is present.
2. A coded multistream video sequence is currently active, an OBU with `obu_type` equal to `OBU_MSDO` is present, and the value of `multistream_profile_idc`, `multistream_level_idx`, `multistream_tier`, `num_streams_minus_2`, `multistream_even_allocation_flag`, or `multistream_large_picture_idc` differs from the corresponding value in the previous `OBU_MSDO`.
3. No coded multistream video sequence is currently active and a global layer configuration record is activated.

A coded multistream video sequence ends at the earliest of:

1. A temporal unit that begins a new coded multistream video sequence as defined above.
2. A temporal unit that begins a new coded video sequence for at least one extended layer but does not contain an OBU with `obu_type` equal to `OBU_MSDO` and does not have an activated global layer configuration record.
3. The end of the bitstream.

At the end of a coded multistream video sequence, all remaining frames from all extended layers shall be output and all reference frame buffers for all extended layers shall be invalidated.

NOTE: The values of `sub_xlayer_id` may change at a random access point without starting a new coded multistream video sequence.

It is a requirement of bitstream conformance that, in a coded multistream video sequence in which both an OBU with `obu_type` equal to `OBU_MSDO` and an activated global layer configuration record are present, the set of coded multistream video sequence boundaries obtained by applying the rules of this section using both the `MSDO` and the activated global layer configuration record shall be identical to the set of boundaries obtained by applying those rules using the `MSDO` alone.

NOTE: In a bitstream conforming to interoperability point 0 or interoperability point 1, an OBU with `obu_type` equal to `OBU_MSDO` is required whenever a coded multistream video sequence is present (see [Annex A.2 Profiles](#), [Table A.4](#)). Together with the requirement above, this means that an implementation decoding such a bitstream may determine coded multistream video sequence boundaries from the `MSDO` alone, regardless of whether a global layer configuration record is also activated.

§ 7.3.3. Coded output frame unit

A coded output frame unit is a collection of consecutive OBUs in a bitstream, all having the same `obu_xlayer_id`, `obu_mlayer_id`, and `obu_tlayer_id`, according to the following rules and presence order:

- Zero or one OBU with `obu_type` equal to `OBU_CONTENT_INTERPRETATION`,
- Zero or more OBUs with `obu_type` equal to `OBU_MULTI_FRAME_HEADER`,

- Zero or more OBUs, which may be present in any order, with an obu_type equal to any of:
 - OBU_BUFFER_REMOVAL_TIMING
 - OBU_QUANTIZATION_MATRIX.
 - OBU_FILM_GRAIN.
 - OBU_METADATA_SHORT having metadata_is_suffix equal to 0
 - OBU_METADATA_GROUP having metadata_is_suffix equal to 0
- Either:
 - One or more OBUs that contain a single coded frame with immediate_output_frame equal to 1 or implicit_output_frame equal to 1, where the OBUs of the coded frame have the same obu_type and the obu_type can be equal to any of:
 - OBU_CLOSED_LOOP_KEY,
 - OBU_OPEN_LOOP_KEY,
 - OBU_LEADING_TILE_GROUP,
 - OBU_REGULAR_TILE_GROUP,
 - OBU_SWITCH,
 - OBU_LEADING_TIP,
 - OBU_REGULAR_TIP, and
 - OBU_RAS_FRAME.

If the OBUs of the coded frame have an obu_type equal to any of

- OBU_CLOSED_LOOP_KEY,
- OBU_OPEN_LOOP_KEY,
- OBU_LEADING_TILE_GROUP,
- OBU_REGULAR_TILE_GROUP,
- OBU_SWITCH, or
- OBU_RAS_FRAME,

then the first encountered OBU shall have is_first_tile_group equal to 1, and all remaining OBUs of the same type, if present, shall have is_first_tile_group equal to 0.

- Or:
 - One OBU of either type OBU_LEADING_SEF or OBU_REGULAR_SEF.

Such a frame is associated with a decoded display order hint value, OrderHint.

- Zero or more OBUs that may be present in any order, with different types also allowed to be interleaved, as follows:
 - Zero or more OBUs with obu_type equal to OBU_METADATA_SHORT having metadata_is_suffix equal to 1,

- Zero or more OBUs with obu_type equal to OBU_METADATA_GROUP having metadata_is_suffix equal to 1.

OBUs with obu_type equal to OBU_PADDING may appear at any position within a coded output frame unit.

§ 7.3.4. Coded non-output frame unit

A coded non-output frame unit is a collection of OBUs, all having the same obu_xlayer_id, obu_mlayer_id, and obu_tlayer_id, according to the following rules and presence order:

- Zero or one OBU with obu_type equal to OBU_CONTENT_INTERPRETATION,
- Zero or more OBUs with obu_type equal to OBU_MULTI_FRAME_HEADER,
- A sequence of different OBUs, which may be present in any order, with different types also allowed to be interleaved, as follows:
 - Zero or one OBU with obu_type equal to OBU_BUFFER_REMOVAL_TIMING
 - Zero or more OBUs with obu_type equal to OBU_QUANTIZATION_MATRIX.
 - Zero or more OBUs with obu_type equal to OBU_FILM_GRAIN.
 - Zero or more OBUs with obu_type equal to OBU_METADATA_SHORT having metadata_is_suffix equal to 0
 - Zero or more OBUs with obu_type equal to OBU_METADATA_GROUP having metadata_is_suffix equal to 0
- One or more OBUs that contain a single coded frame with immediate_output_frame equal to 0 and implicit_output_frame equal to 0, where the OBUs of the coded frame have the same obu_type and the obu_type can be equal to any of:
 - OBU_CLOSED_LOOP_KEY,
 - OBU_OPEN_LOOP_KEY,
 - OBU_LEADING_TILE_GROUP,
 - OBU_REGULAR_TILE_GROUP,
 - OBU_SWITCH,
 - OBU_LEADING_TIP,
 - OBU_REGULAR_TIP,
 - OBU_BRIDGE_FRAME, and
 - OBU_RAS_FRAME.

If the OBUs of the coded frame have an obu_type equal to any of the following values:

- OBU_CLOSED_LOOP_KEY,
- OBU_OPEN_LOOP_KEY,
- OBU_LEADING_TILE_GROUP,
- OBU_REGULAR_TILE_GROUP,
- OBU_SWITCH, or

- OBU_RAS_FRAME,

then the first encountered OBU shall have `is_first_tile_group` equal to 1, and all remaining OBUs of the same type, if present, shall have `is_first_tile_group` equal to 0.

- A sequence of different OBUs, that may be present in any order, with different types also allowed to be interleaved, as follows:
 - Zero or more OBUs with `obu_type` equal to `OBU_METADATA_SHORT` having `metadata_is_suffix` equal to 1,
 - Zero or more OBUs with `obu_type` equal to `OBU_METADATA_GROUP` having `metadata_is_suffix` equal to 1.

OBUs with `obu_type` equal to `OBU_PADDING` may appear at any position within a coded non-output frame unit.

§ 7.3.5. Coded frame unit

A coded frame unit is either a coded output frame unit or a coded non-output frame unit.

§ 7.3.6. Coded extended layer unit

A coded extended layer unit is a collection of OBUs that share the same `obu_xlayer_id` and are constrained to be present in the following order:

- Zero or more OBUs with `obu_type` equal to `OBU_LAYER_CONFIGURATION_RECORD`,
- Zero or more OBUs with `obu_type` equal to `OBU_OPERATING_POINT_SET`,
- Zero or more OBUs with `obu_type` equal to `OBU_ATLAS_SEGMENT`,
- Zero or more OBUs with `obu_type` equal to `OBU_SEQUENCE_HEADER`,
- For each embedded layer present in the bitstream, in ascending order of `obu_mlayer_id` the following can be present in the following order:
 - Zero or more coded non-output frame units in this layer,
 - Zero or one coded output frame unit in this layer

OBUs with `obu_type` equal to `OBU_PADDING` may appear at any position within a coded extended layer unit.

The following constraints apply to every coded extended layer unit:

- At least one coded output frame unit shall be present in the coded extended layer unit.
- If at least one coded non-output frame unit in a particular embedded layer is present, then one coded output frame unit shall also be present in this same embedded layer.
- All coded output frame units in this coded extended layer unit shall have the same value of `OrderHint`.
- If a coded extended layer unit contains a CLK OBU, then the following shall apply:
 - Only the first coded frame unit in each embedded layer of the coded extended layer unit can consist of CLK OBUs, while the first coded frame unit of the lowest embedded layer present in the coded extended layer unit shall be a CLK OBU.

- If a coded extended layer unit contains an OLK OBU, then the following shall apply:
 - Only the first coded frame unit in each embedded layer of the coded extended layer unit can consist of OLK OBUs, while the first coded frame unit of the lowest embedded layer present in the coded extended layer unit shall be an OLK OBU.
- A coded extended layer unit cannot contain both OLK and CLK OBUs.
- If a coded extended layer unit contains a [leading frame](#), then all coded frame units in that coded extended layer unit shall be leading frames.
- If an OBU with obu_type equal to OBU_CONTENT_INTERPRETATION is present in a coded extended layer unit, it shall only be present in the first frame unit of each embedded layer within this coded extended layer unit.
- If an OBU with obu_type equal to OBU_CONTENT_INTERPRETATION is present in any coded extended layer unit, this OBU shall also be present in the first coded extended layer unit of the sequence and shall contain the same contents in all its repetitions for a given embedded layer.

NOTE: When performing random access at an OBU_RAS_FRAME, OBU_CLOSED_LOOP_KEY or OBU_OPEN_LOOP_KEY OBUs that are required as long-term reference frames may appear in the same coded extended layer unit as the random access frame. See [§ 7.3.9 Availability of long-term reference frames](#) for the requirements on this case.

Each coded extended layer unit has an associated order hint that is given by the value of OrderHint in the coded output frame units.

NOTE: This is well defined because all coded output frame units are required to share the same value of OrderHint.

If monotonic_output_order_flag is equal to 0, it is a requirement of bitstream conformance that within a coded video sequence, for a given value of obu_xlayer_id and obu_mlayer_id, if a coded output frame unit X has an associated OrderHint value equal to ohX, there shall not be a coded output frame unit Y in the same extended layer and embedded layer that appears later than X in output order and has an associated OrderHint value less than or equal to ohX, unless a switch frame with restricted_prediction_switch equal to 1 appears between X and Y in coding order.

NOTE: The value of OrderHint is reset at the start of a new coded video sequence and at a switch frame with restricted_prediction_switch equal to 1. In both cases, the OrderHint counter is effectively restarted, allowing OrderHint values to be reused in subsequent coded output frame units.

For each coded extended layer unit that contains an OBU with obu_type equal to OBU_CLOSED_LOOP_KEY or OBU_OPEN_LOOP_KEY, the OBUs within the coded extended layer unit for each operating point satisfy two conditions:

- The OBUs contain one or more coded frame units.
- The first coded frame unit has obu_type equal to OBU_CLOSED_LOOP_KEY or OBU_OPEN_LOOP_KEY.

A new coded video sequence for an extended layer is defined to start at each temporal unit that contains an OBU with `obu_type` equal to `OBU_CLOSED_LOOP_KEY` in the coded extended layer unit corresponding to the extended layer.

Within a particular coded video sequence of an extended layer, it is allowed to send redundant copies of the activated `sequence_header_obu`, but the contents must be bit-identical each time the activated sequence header appears. A new coded video sequence is required if the activated sequence header parameters change.

Within each extended layer, only one sequence header shall remain active for the duration of a coded video sequence, i.e., until a `CLK` is encountered for that extended layer. Additional sequence header OBUs with a different `seq_header_id` can be present in the bitstream but are not activated and have no effect on the decoding process until referenced by a subsequent `CLK` frame header.

OBU types that are not defined in this specification can be ignored by a decoder.

§ 7.3.7. Temporal unit

A `temporal unit` consists of a series of OBUs constrained to be present in the following order:

- One OBU with `obu_type` equal to `OBU_TEMPORAL_DELIMITER` associated with `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`,
- Zero or one OBU with `obu_type` equal to `OBU_MSDO`,
- Zero or more OBUs with `obu_type` equal to `OBU_LAYER_CONFIGURATION_RECORD` associated with `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`,
- Zero or more OBUs with `obu_type` equal to `OBU_OPERATING_POINT_SET` associated with `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`,
- Zero or more OBUs with `obu_type` equal to `OBU_ATLAS_SEGMENT` associated with `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`,
- Zero or more OBUs with `obu_type` equal to `OBU_METADATA_SHORT` or `OBU_METADATA_GROUP` associated with `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID` and having `metadata_is_suffix` equal to 0,
- For each extended layer present in this temporal unit, in ascending order of `obu_xlayer_id`, a coded extended layer unit as defined in [§ 7.3.6 Coded extended layer unit](#).

Additionally, OBUs with `obu_type` equal to `OBU_PADDING` may also appear at any position within a temporal unit. When present outside of a coded extended layer unit, they shall have `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`.

Furthermore, it is a requirement of bitstream conformance that when `lcr_doh_constraint_flag` in the activated global LCR is equal to 1, or `multistream_doh_constraint_flag` in the preceding MSDO is equal to 1, the following conditions are additionally satisfied for each temporal unit in the coded multistream video sequence:

- All frame units within this temporal unit shall use the same value of `OrderHintBits`.
- Coded output frame units present in multiple coded extended layer units within this temporal unit shall have the same value of `OrderHint`.

§ 7.3.8. Availability of high level syntax OBUs

§ 7.3.8.1. General

High level syntax (HLS) OBUs carry configuration and parameter information that is referenced by other OBUs during the decoding process. Each HLS OBU shall be available to the decoding process prior to being referenced, by inclusion in the bitstream or by provision through external means.

This shall also be true if decoding process starts at any random access point and drops any temporal units containing leading frames.

NOTE: This means that HLS OBUs used at a random access point need to be resent in the same temporal unit (or be provided through external means). As a result, HLS OBUs such as sequence headers, multi-frame headers and film grain models that were only available from earlier positions in the bitstream cannot be assumed to be available at a random access point. When HLS OBUs are provided through external means, they remain available to the decoding process until superseded.

The semantics of syntax elements within an HLS OBU apply only when that OBU is activated for the current decoding context. An HLS OBU that is present in the bitstream but not activated has no effect on the decoding process.

The following subsections specify the availability requirements for each HLS OBU type.

§ 7.3.8.2. MSDO availability

When an OBU with obu_type equal to OBU_MSDO is present in a [multistream](#) bitstream, it shall be available to the decoding process at each random access point, by inclusion in the bitstream or by provision through external means. The requirements on the presence of MSDO OBUs depend on the [interoperability point](#), as specified in [Annex A.2 Profiles](#).

It is a requirement of bitstream conformance that an OBU with obu_type equal to OBU_MSDO that is not at a random access point shall be identical to the previous OBU_MSDO.

§ 7.3.8.3. LCR availability

A layer configuration record OBU with obu_xlayer_id equal to GLOBAL_XLAYER_ID and lcr_global_config_record_id equal to id shall be available to the decoding process prior to being referenced by a local layer configuration record OBU with lcr_global_id equal to id, or by a sequence header with seq_lcr_id equal to id, by inclusion in the bitstream or by provision through external means.

A layer configuration record OBU with obu_xlayer_id not equal to GLOBAL_XLAYER_ID shall be available to the decoding process prior to being referenced by a sequence header with seq_lcr_id that resolves to this local layer configuration record, by inclusion in the bitstream or by provision through external means.

§ 7.3.8.4. Atlas segment OBU availability

An atlas segment OBU with obu_xlayer_id equal to GLOBAL_XLAYER_ID and atlas_segment_id equal to id can be available to the decoding process prior to being referenced by a layer configuration record with lcr_global_atlas_id equal to id, by inclusion in the bitstream or by provision through external means.

An atlas segment OBU with obu_xlayer_id not equal to GLOBAL_XLAYER_ID and atlas_segment_id equal to id shall be available to the decoding process prior to being referenced by a layer configuration record with lcr_local_atlas_id equal to id, by inclusion in the bitstream or by provision through external means.

§ 7.3.8.5. OPS availability

An operating point set OBU with obu_xlayer_id equal to GLOBAL_XLAYER_ID and ops_id equal to id shall be available to the decoding process prior to being referenced, by inclusion in the bitstream or by provision through external means.

An operating point set OBU with obu_xlayer_id not equal to GLOBAL_XLAYER_ID and ops_id equal to id shall be available to the decoding process prior to being referenced, by inclusion in the bitstream or by provision through external means.

NOTE: The use of operating point set OBUs is optional for decoders.

§ 7.3.8.6. Sequence header availability

A sequence header OBU with seq_header_id equal to id shall be available to the decoding process prior to being referenced by a frame header with seq_header_id_in_frame_header equal to id, or by a multi-frame header OBU with mfh_seq_header_id equal to id, by inclusion in the bitstream or by provision through external means.

When seq_lcr_id is not equal to 0, the layer configuration record referenced by seq_lcr_id shall be available per [§ 7.3.8.3 LCR availability](#).

See [§ 7.3.6 Coded extended layer unit](#) for additional constraints on sequence header lifetime within a coded video sequence.

§ 7.3.8.7. Multi-frame header availability

A multi-frame header OBU with mfh_id_minus_1 equal to id minus 1 shall be available to the decoding process prior to being referenced by a frame header with cur_mfh_id equal to id, by inclusion in the bitstream or by provision through external means.

It is a requirement of bitstream conformance that the layer dependency constraints TLayerDependencyMap and MLayerDependencyMap are satisfied for the referenced multi-frame header OBU.

The sequence header referenced by mfh_seq_header_id shall be available per [§ 7.3.8.6 Sequence header availability](#).

§ 7.3.8.8. Film grain OBU availability

When apply_grain is equal to 1 in a frame header, a film grain OBU that has set FilmGrainPresent[fgm_id] equal to 1 for the referenced fgm_id shall be available to the decoding process, by inclusion in the bitstream or by provision through external means.

It is a requirement of bitstream conformance that the layer dependency constraints TLayerDependencyMap and MLayerDependencyMap are satisfied for the referenced film grain model, as specified in [§ 6.17.10.1 Film grain config semantics](#).

§ 7.3.8.9. Quantization matrix OBU availability

When using `qm_matrix` is equal to 1 in a frame header, the quantization matrix levels referenced by `qm_y`, `qm_u`, and `qm_v` shall be available to the decoding process, by inclusion of a quantization matrix OBU in the bitstream or by provision through external means.

Quantization matrix levels from previous temporal units are reset at the first OBU in a temporal unit with `obu_type` equal to `OBU_CLOSED_LOOP_KEY` or `OBU_OPEN_LOOP_KEY` or `OBU_SWITCH` or `OBU_RAS_FRAME` (the `QmProtected` array is used to avoid the reset of levels sent in the current temporal unit). When initiating decoding at a random access point, a decoder shall ensure that any required quantization matrix levels are available. If `obu_type` is equal to `OBU_SWITCH`, the reset only applies if `restricted_prediction_switch` is equal to 1.

It is a requirement of bitstream conformance that the layer dependency constraints `TLayerDependencyMap` and `MLayerDependencyMap` are satisfied for the referenced quantization matrix levels, as specified in [§ 6.17.6.2 Setup QM params semantics](#).

§ 7.3.8.10. Content interpretation OBU availability

When present, a content interpretation OBU shall be available to the decoding process from the first coded extended layer unit of the embedded layer in the coded video sequence in which it is present, by inclusion in the bitstream or by provision through external means.

All instances of a content interpretation OBU for a given embedded layer within a coded video sequence shall contain the same information, as specified in [§ 6.14 Content interpretation OBU semantics](#).

CI OBUs shall only appear in the first coded frame unit of each embedded layer within a temporal unit.

If a CI OBU is present in any temporal unit for a given embedded layer, a CI OBU shall also be present in the first temporal unit of the coded video sequence for that embedded layer and shall contain the same contents.

§ 7.3.8.11. Content interpretation parameters initialization

The content interpretation parameters for each embedded layer in an extended layer are initialized to default values at the start of the decoder and at each random access point of the extended layer (i.e., at each temporal unit containing an OBU in the extended layer with `obu_type` equal to `OBU_CLOSED_LOOP_KEY` or `OBU_OPEN_LOOP_KEY`).

The default values for the content interpretation parameters are:

- `ci_scan_type_idc` = 0 (unspecified)
- `ci_color_description_present_flag` = 0
- `ci_color_primaries` = `CP_UNSPECIFIED`
- `ci_transfer_characteristics` = `TC_UNSPECIFIED`
- `ci_matrix_coefficients` = `MC_UNSPECIFIED`
- `ci_full_range_flag` = 0
- `ci_chroma_sample_position_top` = `CSP_UNSPECIFIED`
- `ci_chroma_sample_position_bottom` = `CSP_UNSPECIFIED`

- ci_aspect_ratio_info_present_flag = 0
- ci_timing_info_present_flag = 0
- ci_extension_present_flag = 0

If the decoding process starts at a random access point, the content interpretation parameters for each embedded layer *m* are determined as follows:

1. The content interpretation parameters for embedded layer *m* are first reset to the default values listed above.
2. If a content interpretation OBU is present in the same temporal unit for embedded layer *m*, the content interpretation parameters are set to the values specified in that OBU.
3. Otherwise, if no content interpretation OBU is present for embedded layer *m* and there exists an embedded layer *k* such that MLayerPresenceMap[*m*][*k*] is equal to 1 and content interpretation parameters have been established for embedded layer *k*, the content interpretation parameters for embedded layer *m* are inherited from embedded layer *k*, where *k* is the highest such embedded layer less than *m*.

It is a requirement of bitstream conformance that when a content interpretation OBU is present in a temporal unit that does not contain a [CLK](#) or [OLK](#) for the same embedded layer, and does not contain a [CLK](#) or [OLK](#) for any embedded layer *k* where MLayerPresenceMap[*m*][*k*] is equal to 1, the contents of that content interpretation OBU shall be identical to the content interpretation parameters that were established at the most recent random access point for that embedded layer.

§ 7.3.9. Availability of long-term reference frames

§ 7.3.9.1. General

Long-term reference frames carry frame data that is referenced by other OBUs during the decoding process. Each long-term reference frame shall be available to the decoding process prior to being referenced, by inclusion in the bitstream or by provision through external means, and shall be held in the same reference frame buffer slot that it would occupy under sequential decoding.

When initiating decoding at a random access point containing an OBU_RAS_FRAME, or an OBU_OPEN_LOOP_KEY when long_term_frame_id_bits is not equal to 0, inclusion of long-term reference frames in the bitstream may result in coded extended layer units that do not follow the constraints in [§ 7.3.6 Coded extended layer unit](#). It is a requirement of bitstream conformance that in this case, any OBU_CLOSED_LOOP_KEY OBUs that are required as long-term reference frames appear as the first coded frame units in the coded extended layer unit containing the random access frame, followed by any OBU_OPEN_LOOP_KEY OBUs that are required as long-term reference frames. These long-term reference frame OBUs shall have immediate_output_frame equal to 0 and implicit_output_frame equal to 0.

NOTE: The definition of a coded extended layer unit requires that long-term reference frames with immediate_output_frame equal to 0 and implicit_output_frame equal to 0 are included in the same coded extended layer unit as the random access frame. Since the long-term reference frames are one or more OBU_CLOSED_LOOP_KEY and OBU_OPEN_LOOP_KEY OBUs, the above allows these frames to be in the same coded extended layer unit as the OBU_RAS_FRAME or OBU_OPEN_LOOP_KEY for the purpose of performing a random access operation.

§ 7.4. Random access decoding

This section specifies how decoding is initiated at a random access point. Three random access processes are described: a [closed random access](#) process (§ 7.4.3 Closed Random Access) for OBU_CLOSED_LOOP_KEY, an [open random access](#) process (§ 7.4.4 Open Random Access) for OBU_OPEN_LOOP_KEY, and a [random access switch](#) process (§ 7.4.5 Random Access Switch) for OBU_RAS_FRAME. These processes apply anytime decoding is initiated at one of these OBUs, which includes at the start of a new [coded video sequence](#) or at the start of a new [coded multistream video sequence](#), both of which always begin at a closed random access point. In a [multistream](#) bitstream, a temporal unit may be a random access point for some extended layers but not for others, which is also described in this section and specified in § 7.4.6 Multistream Random Access.

§ 7.4.1. General

A [temporal unit](#) containing one or more OBUs with obu_type equal to OBU_CLOSED_LOOP_KEY, OBU_OPEN_LOOP_KEY, or OBU_RAS_FRAME is defined to be a random access point. Decoding can be correctly initiated at such a temporal unit. The availability requirements for initiating decoding at a random access point are specified in § 7.3.8 Availability of high level syntax OBUs and § 7.3.9 Availability of long-term reference frames.

The process of initiating decoding at a random access point follows the ordered steps:

1. If the temporal unit contains one or more OBUs with an obu_type equal to OBU_CLOSED_LOOP_KEY, OBU_OPEN_LOOP_KEY or OBU_RAS_FRAME, the variable isRandomAccessPoint is set equal to 1. Otherwise, isRandomAccessPoint is set equal to 0.
2. If isRandomAccessPoint is equal to 1, the variable MultiStreamDecoderMode is determined as follows:
 1. If the temporal unit contains one or more OBUs with an obu_type equal to OBU_MSDO then MultiStreamDecoderMode is set equal to 1.
 2. Otherwise, MultiStreamDecoderMode is set equal to 0.
3. For each coded extended layer unit in the temporal unit, the random access process for that extended layer is determined by the OBU type present in the coded extended layer unit:
 1. If the first coded frame unit in a coded extended layer unit contains an OBU with obu_type equal to OBU_CLOSED_LOOP_KEY, then the closed loop key frame random access process in § 7.4.3 Closed Random Access applies to that extended layer.
 2. Otherwise, if the first coded frame unit in the coded extended layer unit contains an OBU with obu_type equal to OBU_OPEN_LOOP_KEY, then the open loop key frame random access process in § 7.4.4 Open Random Access applies to that extended layer.
 3. Otherwise, if the coded extended layer unit contains an OBU with obu_type equal to OBU_RAS_FRAME, then the random access switch process in § 7.4.5 Random Access Switch applies to that extended layer.

NOTE: The value for MultiStreamDecoderMode can only be updated at a random access point. The value for MultiStreamDecoderMode then persists for subsequent temporal units that are not random access points.

NOTE: MultiStreamDecoderMode is set to 1 only when an MSDO OBU is present. A [multistream](#) bitstream that does not contain an MSDO OBU will have MultiStreamDecoderMode equal to 0.

NOTE: For [multistream](#) bitstreams, additional random access requirements are specified in [§ 7.4.6 Multistream Random Access](#).

§ 7.4.2. Random access and use of long-term reference frames

§ 7.4.2.1. Random access with long-term reference frames

A coded video sequence may use random access with long-term reference frames when `long_term_frame_id_bits` is set to a value not equal to 0 in the sequence header associated with this coded video sequence. In such a coded video sequence, the random access described in [§ 7.4.4 Open Random Access](#) and [§ 7.4.5 Random Access Switch](#) may rely on previous `OBU_CLOSED_LOOP_KEY` and `OBU_OPEN_LOOP_KEY` frame data for the decoding of the video sequence. When the decoding starts with [§ 7.4.4 Open Random Access](#) and [§ 7.4.5 Random Access Switch](#), this frame data may need to be provided.

§ 7.4.2.2. Random access without long-term reference frames

A coded video sequence uses random access without long-term reference frames when `long_term_frame_id_bits` is set to 0 in the sequence header associated with this coded video sequence. In such a coded video sequence, random access described in [§ 7.4.4 Open Random Access](#) and [§ 7.4.5 Random Access Switch](#) does not use any previous `OBU_CLOSED_LOOP_KEY` and `OBU_OPEN_LOOP_KEY` frame data for the decoding of the video sequence.

§ 7.4.3. Closed Random Access

The closed random access process applies to an extended layer when the first coded frame unit in the coded extended layer unit has `obu_type` equal to `OBU_CLOSED_LOOP_KEY`. The process starts a new [coded video sequence](#) for the extended layer (see [§ 7.3.6 Coded extended layer unit](#)).

When the closed random access process is invoked for an extended layer, the following apply:

- All reference frame buffers for the extended layer are invalidated, and any pending implicit output frames are flushed. See [§ 5.18.2 Frame header info syntax](#) for the associated variable assignments.
- The sequence header referenced by the `CLK` frame header becomes the active sequence header and remains active for the remainder of the new coded video sequence, as specified in [§ 7.3.6 Coded extended layer unit](#).
- Content interpretation parameters for each embedded layer in the extended layer are re-established, as specified in [§ 7.3.8.11 Content interpretation parameters initialization](#).
- Quantization matrix levels from previous temporal units are reset, as specified in [§ 7.3.8.9 Quantization matrix OBU availability](#).

§ 7.4.4. Open Random Access

The open random access process applies to an extended layer when the first coded frame unit in the coded extended layer unit has `obu_type` equal to `OBU_OPEN_LOOP_KEY`. During sequential decoding, the process does not start a new [coded video sequence](#) for the extended layer. However, when a decoder

initiates decoding at the open random access point, the process is treated as if it were the start of a new coded video sequence for the extended layer (see [§ 7.3.6 Coded extended layer unit](#)). For the purposes of the decoding process, all reference frame buffers not refreshed by the OLK are invalidated except for the long term reference frames listed in `ref_long_term_id`, leading frames are discarded, and the sequence header referenced by the OLK frame header is activated.

NOTE: During sequential decoding, the OLK does not start a new coded video sequence. Leading frames that follow the OLK can be decoded using reference frames from the preceding frames.

Provided the following HLS OBUs are available to the decoding process, by inclusion in the bitstream or by provision through external means, and that the long term reference condition defined below is satisfied, decoding can be correctly initiated at such a temporal unit, and all subsequent non-leading frames in decoding order can be correctly decoded, without performing the decoding process of any frames that precede the temporal unit in decoding order (with exception of long term reference frames listed in `ref_long_term_id` of this OLK):

- A sequence header OBU with `seq_header_id` equal to the value referenced by the OLK frame header.
- If `seq_lcr_id` in the sequence header is not equal to 0, the layer configuration record referenced by `seq_lcr_id`.
- If the referenced layer configuration record references an atlas segment OBU via `lcr_global_atlas_id` or `lcr_local_atlas_id`, that atlas segment OBU.
- If `cur_mfh_id` is greater than 0, the multi-frame header OBU with `mfh_id_minus_1` equal to `cur_mfh_id` minus 1.
- If `apply_grain` is equal to 1, the film grain OBU for the referenced `fgm_id`.
- If `using_qmatrix` is equal to 1 and the referenced quantization matrix levels differ from the default levels established by the sequence header, the quantization matrix OBU providing those levels.
- In a [multistream](#) bitstream, an OBU with `obu_type` equal to `OBU_MSDO` or a global layer configuration record OBU, when present.

The long term reference condition is defined such that one or more of the following shall be satisfied:

1. `long_term_frame_id_bits` is equal to 0 for this sequence (where `ref_long_term_id` is inferred as empty), or
2. `num_key_ref_frames` is equal to 0 in this OLK frame header (where `ref_long_term_id` is inferred as empty), or
3. The decoded reference frames identified by the `ref_long_term_id` values signaled in the OLK frame header are available. These reference frames are retained from the previous coded video sequence and are required for reference in future inter frames.

It is a requirement of bitstream conformance that any regular frames (`IsRegular` equal to 1) after an OLK shall not reference any frames (or other information stored by the reference frame update process [§ 7.23 Reference frame update process](#)) that precede the OLK temporal unit, other than information made available through the reference frame buffers refreshed by the OLK temporal unit, or the long term references included in `ref_long_term_id`.

Regular frames that follow leading frames after the OLK temporal unit shall also not reference leading frames or HLS OBUs that are indicated in temporal units containing leading frames.

The constraint to not reference leading frames is enforced by the reference frame invalidation process in [§ 5.18.1 General frame header syntax](#), which sets RefValid[i] equal to 0 for reference frame slots not refreshed by the OLK when the first [Regular frame](#) is encountered.

See [§ 7.3.8 Availability of high level syntax OBUs](#) for the general availability requirements for each HLS OBU type.

See [§ 7.3.9 Availability of long-term reference frames](#) for the availability requirements for long-term reference frames.

A long term reference frame shall be included in the ref_long_term_id list of an OLK, if and only if:

1. when using sequential decoding, this long term reference frame is held in a reference frame buffer when the OLK is encountered, and
2. when using sequential decoding, this long term reference frame is held in a reference frame buffer when the first [Regular frame](#) (in a different temporal unit than the OLK) is encountered after the OLK, and
3. The long term reference frame is in the same embedded layer as the OLK, or is in an embedded layer that is dependent on the embedded layer of the OLK

NOTE: the constraints on the ref_long_term_id list above ensure that the reference frame buffers are the same whether randomly accessed from an OLK, or sequentially decoded. For example, consider the case when a leading frame updates a reference frame buffer that was originally taken by a long term reference. If randomly accessed, then the long term reference would still be available (given it is incorrectly included in the OLK ref_long_term_id list), but if sequentially decoded, the long term reference would not be held in a reference frame buffer. This is avoided by the constraints.

It is a requirement of bitstream conformance that if long_term_frame_id_bits is greater than 0, the OrderHint of an OLK shall be less than $(1 \ll \text{OrderHintBits})$.

NOTE: This constraint ensures that the OrderHint of an OLK is equal to the value of order_hint in the bitstream (i.e., no modular wrap-around has occurred) when long-term reference frames are in use. This guarantees that the relative distance and ordering between an OLK and its long-term reference frames are the same whether decoding is sequential or initiated at the OLK as a random access point. Encoders may select an appropriate value for order_hint_bits_minus_1 when addressing this constraint.

§ 7.4.5. Random Access Switch

The random access switch process applies to an extended layer when the coded extended layer unit contains an OBU with obu_type equal to OBU_RAS_FRAME. The process does not start a new [coded video sequence](#) for the extended layer (see [§ 7.3.6 Coded extended layer unit](#)).

NOTE: The RAS frame is an inter-predicted frame. Although it is inter-predicted, it may only reference long-term reference frames whose RefLongTermId appears in the ref_long_term_id list, as specified in [§ 6.17 Frame header OBU semantics](#). This restriction is what enables random access at an inter-predicted frame.

For decoding to be correctly initiated at a RAS frame, one of the following shall be satisfied:

1. `num_key_ref_frames` is equal to 0 in this RAS frame header (where `ref_long_term_id` is inferred as empty), or
2. The decoded reference frames identified by the `ref_long_term_id` values signaled in the RAS frame header are available, as specified in [§ 7.3.9 Availability of long-term reference frames](#).

When the random access switch process is invoked for an extended layer, the following apply:

- Reference frame buffers that do not hold long-term reference frames listed in `ref_long_term_id` are refreshed by the RAS frame, as specified in [§ 6.17 Frame header OBU semantics](#).
- The active sequence header for the extended layer remains in effect, as specified in [§ 7.3.6 Coded extended layer unit](#).
- Any active layer configuration record remains in effect as part of the active sequence header (see [§ 6.4 Sequence header OBU semantics](#) for LCR activation).
- Quantization matrix levels from previous temporal units are reset, as specified in [§ 7.3.8.9 Quantization matrix OBU availability](#).

NOTE: After the reference frame update process, only the first refreshed reference frame buffer (containing the decoded RAS frame) and the long-term reference frame buffers identified by `ref_long_term_id` are valid. See [§ 7.23 Reference frame update process](#).

The following bitstream conformance requirements apply to RAS frames:

It is a requirement of bitstream conformance that if a long term reference frame is included in the `ref_long_term_id` list of a RAS frame, then, when using sequential decoding, this long term reference frame is held in a reference frame buffer when the RAS frame is encountered.

NOTE: The constraint on the `ref_long_term_id` list above prevents the list from declaring long-term reference frames that are not present in a reference frame buffer under sequential decoding.

It is a requirement of bitstream conformance that if `long_term_frame_id_bits` is greater than 0, the `OrderHint` of a RAS frame with `restricted_prediction_switch` equal to 0 shall be less than $(1 \ll \text{OrderHintBits})$.

NOTE: This constraint ensures that the `OrderHint` of a RAS frame is equal to the value of `order_hint` in the bitstream (i.e., no modular wrap-around has occurred) when long-term reference frames are in use. This guarantees that the relative distance and ordering between a RAS frame and its long-term reference frames are the same whether decoding is sequential or initiated at the RAS frame as a random access point. Encoders may select an appropriate value for `order_hint_bits_minus_1` when addressing this constraint.

§ 7.4.6. Multistream Random Access

In a [multistream](#) bitstream, different coded extended layer units within the same temporal unit may contain different types of random access OBUs (e.g., `OBU_CLOSED_LOOP_KEY` in one extended layer and `OBU_OPEN_LOOP_KEY` in another). As specified in [§ 7.4.1 General](#), the corresponding random access process applies independently to each extended layer.

Random access points are not required to be aligned across extended layers, and a temporal unit may be a random access point for some extended layers but not for others. However, when MultiStreamDecoderMode is equal to 1 and multistream_doh_constraint_flag is equal to 1, or when a global layer configuration record is activated and lcr_doh_constraint_flag is equal to 1, all coded output frame units present together in a temporal unit are required to share the same OrderHintBits and OrderHint, as specified in [§ 7.3.7 Temporal unit](#).

When a decoder initiates decoding at a temporal unit that is a random access point for a subset of the extended layers in the multistream, the decoder shall not decode coded extended layer units for an extended layer until a random access point for that extended layer is encountered.

When an OBU with obu_type equal to OBU_MSDO is present, it is parsed before any coded extended layer units in the temporal unit, as specified in [§ 7.3.7 Temporal unit](#). The variable MultiStreamDecoderMode and the sub_xlayer_id array are therefore established before the per-extended-layer random access processes are invoked.

§ 7.5. Frame end update CDF process

This process is triggered when the function frame_end_update_cdf is called from the tile group syntax table.

The frame CDF arrays are set based on the saved CDF arrays as follows.

A copy is made of the saved CDF values for each of the CDF arrays mentioned in the semantics for init_coeff_cdfs and init_non_coeff_cdfs. The name of the destination for the copy is the name of the CDF array with no prefix. The name of the source for the copy is the name of the CDF array prefixed with "Saved".

Once the CDF arrays have been copied, the last entry in each destination array, representing the symbol count for that context, is set equal to $(3 * \text{count}) \gg 2$ where count is equal to the value of the last entry in each source array.

For example, the array IdentityRowYCdf will be created as follows:

```
for( i = 0; i < PALETTE_ROW_FLAG_CONTEXTS; i++ ) {
    for ( j = 0; j < 4; j++ ) {
        IdentityRowYCdf[ i ][ j ] = SavedIdentityRowYCdf[ i ][ j ]
    }
    IdentityRowYCdf[ i ][ 3 ] = ( 3 * SavedIdentityRowYCdf[ i ][ 3 ] ) >> 2
}
```

§ 7.6. Extended layer context management

The function save_xlayer_context is used to save information corresponding to the decoder state when obu_xlayer_id was last processed.

```
save_xlayer_context( obu_xlayer_id ) {
    if( obu_xlayer_id == GLOBAL_XLAYER_ID )
        return

    if( MultiStreamDecoderMode ) {
        for( i = 0; i < num_streams_minus_2 + 2; i++ ) {
            if( sub_xlayer_id[i] == obu_xlayer_id ) {
```

```

        streamID = i
        break
    }
} else {
    streamID = obu_xlayer_id
}

save_context( streamID )
}

```

where `save_context( streamID )` stores all decoder state information for the current `obu_xlayer_id` in a memory location denoted by the `streamID` value.

The function `load_xlayer_context` is used to load information corresponding to the decoder state when `obu_xlayer_id` was last processed.

```

load_xlayer_context( obu_xlayer_id ) {
    if( obu_xlayer_id == GLOBAL_XLAYER_ID )
        return

    if( MultiStreamDecoderMode ) {
        for( i = 0; i < num_streams_minus_2 + 2; i++ ) {
            if( sub_xlayer_id[i] == obu_xlayer_id ) {
                streamID = i
                break
            }
        }
    } else {
        streamID = obu_xlayer_id
    }

    load_context( streamID )
}

```

where `load_context( streamID )` loads all decoder state information for the current `obu_xlayer_id` from the memory location denoted by the `streamID` value.

NOTE: This specification defines decoding as the sequential processing of OBUs. The `load_xlayer_context()` and `save_xlayer_context()` realize the separate processing of extended layers in this context. Some implementations may use separate instances or other methods to separate the processing of individual streamIDs. These implementations may not need to implement the `load_xlayer_context()` and `save_xlayer_context()` functions.

NOTE: When `MultiStreamDecoderMode` is equal to 0, the `streamID` is set directly to `obu_xlayer_id`. This applies both to [singlestream](#) bitstreams and to [multistream](#) bitstreams that do not contain an MSDO OBU.

NOTE: When `MultiStreamDecoderMode` is equal to 1, the `sub_xlayer_id` lookup in `save_xlayer_context` and `load_xlayer_context` is guaranteed to find a match for any conformant bitstream, as a coded multistream video sequence requires that every `obu_xlayer_id` value (excluding `GLOBAL_XLAYER_ID`) corresponds to a value in `sub_xlayer_id`.

§ 7.7. Get ref frames process

This process is triggered if the function `get_ref_frames` is called while reading the [frame header info](#).

The input to this process is the variable `checkRes` specifying if the resolution of reference frames is used.

The syntax elements in the `ref_frame_idx` array are computed based on the quantizer and display order hints saved for the reference frames.

Variables indicating the quantizer and display order hint for distinct reference frames are prepared as follows:

```
maxDisp = 0
for ( i = 0; i < NumRefFrames; i++ ) {
    mapOrderHint[i] = -1
    if ( first_slot_with_ref(i) && RefOrderHint[i] != RESTRICTED_OH &&
        ( !IsBridge || i == bridge_frame_ref_idx ) &&
        ( AllowedFrames & (1 << i) ) &&
        TLayerDependencyMap[obu_mlayer_id][obu_tlayer_id][RefTLayerId[i]] &&
        MLayerDependencyMap[obu_mlayer_id][RefMLayerId[i]] ) {
        if ( valid_ref_frame_size( checkRes, i ) ) {
            mapOrderHint[i] = RefOrderHint[i]
        }
        mapBaseQIdx[i] = RefBaseQIdx[i]
        maxDisp = Max( maxDisp, RefOrderHint[i] )
    }
}
```

where `first_slot_with_ref` detects distinct reference frames as follows:

```
first_slot_with_ref( i ) {
    if ( !RefValid[i] ) {
        return 0
    }
    for ( j = 0; j < i; j++ ) {
        if ( RefValid[j] && RefCounter[j] == RefCounter[i] ) {
            return 0
        }
    }
    return 1
}
```

and `valid_ref_frame_size` checks resolution based validity as follows:

```
valid_ref_frame_size( checkRes, slot ) {
    if ( !checkRes )
        return 1
    return ( 2 * FrameWidth >= RefFrameWidth[ slot ] &&
        2 * FrameHeight >= RefFrameHeight[ slot ] &&
        FrameWidth <= 16 * RefFrameWidth[ slot ] &&
        FrameHeight <= 16 * RefFrameHeight[ slot ] )
}
```

The distinct reference frames are given a score as follows:

```
NRanked = 0
maxQ = 0
minQ = 0
for ( i = 0; i < NumRefFrames; i++ ) {
    d = mapOrderHint[i]
```

```

if (d != -1) {
    q = mapBaseQIdx[i]
    dispDiff = get_relative_dist( OrderHint, d )
    tDist = Abs(dispDiff) + obu_mlayer_id - RefMLayerId[i]
    if (maxDisp > OrderHint) {
        score = (tDist << DIST_WEIGHT_BITS) + q
    } else {
        score = Dist_Score_Lookup[Min(tDist, DECAY_DIST_CAP)] +
            Max(tDist - DECAY_DIST_CAP, 0) + q
    }
    refRatio = FloorLog2( RefFrameWidth[ i ] * RefFrameHeight[ i ] )
    score -= refRatio << 5
    if (new_score_or_dist(d,score,RefMLayerId[i])) {
        ScoresIndex[NRanked] = i
        ScoresScore[NRanked] = score
        ScoresOrderHint[NRanked] = d
        ScoresDistance[NRanked] = dispDiff
        ScoresBaseQIdx[NRanked] = q
        ScoresLayer[NRanked] = RefMLayerId[i]
        if (NRanked == 0) {
            minQ = q
            maxQ = q
        } else {
            minQ = Min(q,minQ)
            maxQ = Max(q,maxQ)
        }
        NRanked += 1
    }
}
}
}

```

where Dist_Score_Lookup is defined as:

```

Dist_Score_Lookup[ DECAY_DIST_CAP + 1 ] = {
    0, 64, 96, 112, 120, 124, 126,
}

```

and the function new_score_or_dist (which returns 1 if we have found a new score or a new display order hint) is given by:

```

new_score_or_dist(d,score,mLayer) {
    for ( i = 0; i < NRanked; i++ ) {
        if ( ScoresOrderHint[i] == d &&
            ScoresScore[i] == score &&
            mLayer == ScoresLayer[i] ) {
            return 0
        }
    }
    return 1
}

```

If too many references have been selected, a reference is dropped as follows:

```

if (NRanked > REFS_PER_FRAME) {
    qThresh = (maxQ + minQ + 1) / 2
    unmappedIdx = get_unmapped_ref(qThresh)
    if (unmappedIdx >= 0) {
        ScoresScore[unmappedIdx] = 0x7fffffff
    }
}

```

where `get_unmapped_ref` chooses the reference to drop as follows:

```
get_unmapped_ref(qThresh) {
    nPast = 0
    nFuture = 0
    maxPastDistance = 0
    maxFutureDistance = 0
    pastIdx = 0
    futureIdx = 0
    for ( i = 0; i < NRanked; i++ ) {
        if (ScoresBaseQIdx[i] >= qThresh) {
            d = ScoresDistance[i]
            if (d > 0) {
                if (d > maxPastDistance) {
                    maxPastDistance = d
                    pastIdx = i
                }
                nPast++
            } else if (d < 0) {
                if (-d > maxFutureDistance) {
                    maxFutureDistance = -d
                    futureIdx = i
                }
                nFuture++
            }
        }
    }
    if (nPast > nFuture) {
        return pastIdx
    }
    if (nPast < nFuture) {
        return futureIdx
    }
    if (nPast > 0) {
        return maxPastDistance >= maxFutureDistance ? pastIdx : futureIdx
    }
    return -1
}
```

The references are ranked and the values for `ref_frame_idx` are computed as follows:

```
bubble_sort_ref_scores()
NumTotalRefs = Min(NRanked,ActiveNumRefFrames)
for (i = 0; i < NumTotalRefs; i++) {
    ref_frame_idx[ i ] = ScoresIndex[ i ]
}
```

where the function `bubble_sort_ref_scores` (which sorts the references based on their score) is specified as:

```
bubble_sort_ref_scores( ) {
    for (i = NRanked - 1; i > 0; i--) {
        for (j = 0; j < i; j++) {
            if (ScoresScore[j] > ScoresScore[j + 1]) {
                index = ScoresIndex[j]
                score = ScoresScore[j]
                displayOrder = ScoresOrderHint[j]
                distance = ScoresDistance[j]
                baseQIdx = ScoresBaseQIdx[j]

                ScoresIndex[j] = ScoresIndex[j+1]
                ScoresScore[j] = ScoresScore[j+1]
                ScoresOrderHint[j] = ScoresOrderHint[j+1]
                ScoresDistance[j] = ScoresDistance[j+1]
                ScoresBaseQIdx[j] = ScoresBaseQIdx[j+1]
            }
        }
    }
}
```

```

        ScoresIndex[j+1] = index
        ScoresScore[j+1] = score
        ScoresOrderHint[j+1] = displayOrder
        ScoresDistance[j+1] = distance
        ScoresBaseQIdx[j+1] = baseQIdx
    }
}
}

```

Finally, any remaining restricted frames are added at the end as follows:

```

if ( checkRes && !IsBridge ) {
    for ( i = 0; i < NumRefFrames; i++ ) {
        if ( RefValid[ i ] && RefOrderHint[ i ] == RESTRICTED_OH &&
            TLayerDependencyMap[obu_mlayer_id][obu_tlayer_id][RefTLayerId[i]] &&
            MLayerDependencyMap[obu_mlayer_id][RefMLayerId[i]] &&
            (AllowedFrames & (1 << i)) &&
            NumTotalRefs < ActiveNumRefFrames ) {
            ref_frame_idx[ NumTotalRefs ] = i
            NumTotalRefs++
        }
    }
}

```

§ 7.8. Get past future cur ref lists process

This process is triggered by a call to `get_past_future_cur_ref_lists` while reading the [frame header info](#).

The process chooses references to be used as follows:

```

NumPastRefs = 0
NumFutureRefs = 0
numCurRefs = 0
FurthestFuture = NONE
ClosestPast = NONE
ClosestFuture = NONE
for ( i = 0; i < NumTotalRefs; i++ ) {
    if ( RefOrderHint[ref_frame_idx[i]] != RESTRICTED_OH ) {
        if ( ScoresDistance[i] > 0 ) {
            NumPastRefs++
            if ( ClosestPast == NONE ||
                ScoresDistance[i] < ScoresDistance[ClosestPast] ) {
                ClosestPast = i
            }
        } else if ( ScoresDistance[i] < 0 ) {
            NumFutureRefs++
            if ( FurthestFuture == NONE ||
                RefOrderHint[ref_frame_idx[FurthestFuture]] <
                RefOrderHint[ref_frame_idx[i]] ) {
                FurthestFuture = i
            }
            if ( ClosestFuture == NONE ||
                RefOrderHint[ref_frame_idx[i]] <
                RefOrderHint[ref_frame_idx[ClosestFuture]] ) {
                ClosestFuture = i
            }
        } else {
            curRefs[numCurRefs] = i
            numCurRefs++
        }
    }
}
SkipSegFrame = numCurRefs > 0 ? curRefs[0] : ClosestPast

```



```

if ( SkipSegFrame == NONE ) {
    SkipSegFrame = 0
}
OrigClosestFuture = ClosestFuture
OrigClosestPast = ClosestPast

```

§ 7.9. Motion field estimation process

§ 7.9.1. General

This process is triggered by a call to `motion_field_estimation` while reading the [frame header info](#).

A linear projection model is employed to create a motion field estimation that is able to capture high velocity temporal motion trajectories.

The motion field is estimated based on the saved motion vectors from the reference frames and the relative frame distances.

As the frame distances depend on the frame being referenced, a separate motion field is estimated for each reference frame used by the current frame.

A motion vector (for each reference frame type) is prepared at each location on an 8x8 luma sample grid.

The variable `w8` (representing the width of the motion field in units of 8x8 luma samples) is set equal to `MiCols >> 1`.

The variable `h8` (representing the height of the motion field in units of 8x8 luma samples) is set equal to `MiRows >> 1`.

As the linear projection can create a field with holes, the motion fields are initialized to an invalid motion vector as follows:

```

for ( y = 0; y < h8 ; y++ )
    for ( x = 0; x < w8; x++ ) {
        MotionFieldValid[ y ][ x ] = 0
        MotionFieldOffset[ y ][ x ] = 0
        for( src = 0; src < NumTotalRefs; src++ ) {
            TrajValid[ src ][ y ][ x ] = 0
            for(k=0;k<3;k++) {
                TrajPosValid[ k ][ src ][ y ][ x ] = 0
            }
        }
    }

```

An array `sortRef` that gives the reference frames in sorted order (sorted by order hint) is computed as follows:

```

for( i = 0 ; i < NumTotalRefs ; i++ ) {
    sortRef[i] = i
}
for( i = 0; i < NumTotalRefs ; i++ ) {
    for( j = i + 1 ; j < NumTotalRefs ; j++ ) {
        if ( get_relative_dist( OrderHints[ sortRef[ j ] ],
                               OrderHints[ sortRef[ i ] ] ) < 0 ) {
            tmp = sortRef[i]
            sortRef[i] = sortRef[j]
            sortRef[j] = tmp
        }
    }
}

```

```

    }
}

```

A variable `curIdx` that specifies the index of the reference just before the current order hint is computed as follows:

```

curIdx = -1
for( i = 0 ; i < NumTotalRefs ; i++ ) {
    if ( get_relative_dist( OrderHints[ sortRef[ i ] ], OrderHint ) < 0 ) {
        curIdx = i
    } else {
        break
    }
}

```

The references are topologically sorted as follows:

```

for ( rf = 0; rf < NumTotalRefs; rf++) {
    MotionFieldVisited[ rf ] = 0
    MotionFieldDepth[ rf ] = -1
    MotionFieldChecked[ rf ][ 0 ] = 0
    MotionFieldChecked[ rf ][ 1 ] = 0
}
MotionFieldStackCount = 0
for ( rf = 0; rf < NumTotalRefs; rf++) {
    if ( OrderHints[ rf ] != RESTRICTED_OH ) {
        topo_sort_refs( rf )
    }
}

```

Where `topo_sort_refs` is a recursive function specified as:

```

topo_sort_refs( rf ) {
    if ( MotionFieldVisited[ rf ] ) {
        return
    }
    MotionFieldVisited[ rf ] = 1
    refIdx = ref_frame_idx[ rf ]
    if (RefFrameType[ refIdx ] == INTER_FRAME) {
        for( i = 0; i < RefNumTotalRefs[ refIdx ]; i++ ) {
            if ( SavedOrderHints[ refIdx ][ i ] != RESTRICTED_OH ) {
                for( j = 0 ; j < NumTotalRefs ; j++) {
                    if ( OrderHints[ j ] == SavedOrderHints[ refIdx ][ i ] &&
                        !is_ref_overlay( j ) ) {
                        topo_sort_refs( j )
                        break
                    }
                }
            }
        }
    }
    MotionFieldDepth[ rf ] = MotionFieldStackCount
    MotionFieldStack[ MotionFieldStackCount ] = rf
    MotionFieldStackCount++
}

is_ref_overlay( ref ) {
    refIdx = ref_frame_idx[ ref ]
    for (i = 0; i < RefNumTotalRefs[ refIdx ]; i++) {
        if (SavedOrderHints[ refIdx ][ i ] == RefOrderHint[ refIdx ]) {
            return 1
        }
    }
}

```

```

    }
    return 0
}

```

If MotionFieldStackCount is less than 2, the process immediately terminates.

The variable processCount (representing how many motion fields have to be projected) is set equal to 0.

The projections to do are recorded as follows:

```

if ( enable_tip &&
    ( (NumFutureRefs > 0 && NumPastRefs > 0) || NumPastRefs >= 2 ) ) {
    past = sortRef[curIdx]
    if ( NumFutureRefs > 0 && NumPastRefs > 0 ) {
        future = sortRef[curIdx + 1]
    } else {
        future = sortRef[curIdx - 1]
    }
    if ( MotionFieldDepth[past] > MotionFieldDepth[future] ) {
        processCount = record_tip_projection( past, 1, future, processCount )
    } else {
        processCount = record_tip_projection( future, 0, past, processCount )
    }
    ClosestPast = past
    ClosestFuture = future
} else {
    ClosestPast = NONE
    ClosestFuture = NONE
}
for( groupId = 0; groupId < 2 ; groupId++ ) {
    pastIdx = curIdx >= groupId ? curIdx - groupId : -1
    if (pastIdx >= 0 && !has_future_ref( sortRef[ pastIdx ] ))
        pastIdx = -1
    pastDist = pastIdx >= 0 ?
        get_dist_to_closest_interp_ref(sortRef[pastIdx], 0) : -1
    futureIdx = curIdx < NumTotalRefs - groupId - 1 ?
        curIdx + 1 + groupId : -1
    if (futureIdx >= 0 && !has_past_ref( sortRef[ futureIdx ] ))
        futureIdx = -1
    futureDist = futureIdx >= 0 ?
        get_dist_to_closest_interp_ref(sortRef[futureIdx], 1) : -1
    if (futureDist < pastDist) {
        if (futureIdx >= 0) {
            processCount = record_projection( sortRef[futureIdx], 0,
                                             processCount )
        }
        if (pastIdx >= 0) {
            processCount = record_projection( sortRef[pastIdx], 1, processCount )
        }
    } else {
        if (pastIdx >= 0) {
            processCount = record_projection( sortRef[pastIdx], 1, processCount )
        }
        if (futureIdx >= 0) {
            processCount = record_projection( sortRef[futureIdx], 0,
                                             processCount )
        }
    }
}
}

if (curIdx >= 0) {
    processCount = record_projection( sortRef[curIdx], 0, processCount )
}
if (curIdx >= 1) {
    processCount = record_projection( sortRef[curIdx - 1], 0, processCount )
}
for ( ri = MotionFieldStackCount - 1; ri > 0 ; ri-- ) {
    ref = MotionFieldStack[ ri ]
}

```

```

isBwd = OrderHints[ ref ] < OrderHint
for( j = 0 ; j < 2 ; j++ ) {
    if ( !MotionFieldChecked[ ref ][ isBwd ] ) {
        processCount =
            record_projection_with_type( 0, ref, isBwd, -1, MFMV_STACK_SIZE,
                                         processCount )
    }
    isBwd = !isBwd;
}
}

```

where the functions `has_future_ref`, `has_past_ref`, `get_dist_to_closest_interp_ref`, `is_ref_motion_field_eligible`, `is_ref_motion_field_eligible_by_frame_size`, `is_ref_motion_field_eligible_by_frame_type`, `record_tip_projection`, `record_projection`, and `record_projection_with_type` are specified as:

```

has_future_ref( ref ) {
    if ( OrderHints[ ref ] == RESTRICTED_OH ) {
        return 0
    }
    refIdx = ref_frame_idx[ ref ]
    for ( i = 0; i < RefNumTotalRefs[ refIdx ]; i++ ) {
        if ( SavedOrderHints[ refIdx ][ i ] != RESTRICTED_OH &&
             SavedOrderHints[ refIdx ][ i ] > RefOrderHint[ refIdx ] ) {
            return 1
        }
    }
    return 0
}

has_past_ref( ref ) {
    if ( OrderHints[ ref ] == RESTRICTED_OH ) {
        return 0
    }
    refIdx = ref_frame_idx[ ref ]
    for ( i = 0; i < RefNumTotalRefs[ refIdx ]; i++ ) {
        if ( SavedOrderHints[ refIdx ][ i ] != RESTRICTED_OH &&
             SavedOrderHints[ refIdx ][ i ] < RefOrderHint[ refIdx ] ) {
            return 1
        }
    }
    return 0
}

get_dist_to_closest_interp_ref( startFrame, findForwardRef ) {
    absClosestRefOffset = 0x7fffffff
    startIdx = ref_frame_idx[ startFrame ]
    if ( !is_ref_motion_field_eligible( startIdx ) ) {
        return 0x7fffffff
    }
    for ( ref = 0; ref < RefNumTotalRefs[ startIdx ]; ref++ ) {
        if ( SavedOrderHints[ startIdx ][ ref ] != RESTRICTED_OH ) {
            refOffset = SavedOrderHints[ startIdx ][ ref ]
            startToRefOffset = get_relative_dist( OrderHints[startFrame],
                                                  refOffset )

            curToRefOffset = get_relative_dist( OrderHint, refOffset )
            absStartToRefOffset = Abs( startToRefOffset )
            isTwoSides = findForwardRef ?
                ( startToRefOffset > 0 && curToRefOffset > 0 ) :
                ( startToRefOffset < 0 && curToRefOffset < 0 )

            if ( isTwoSides ) {
                absClosestRefOffset = Min( absClosestRefOffset,
                                           absStartToRefOffset )
            }
        }
    }
    return absClosestRefOffset
}

```

```

}

is_ref_motion_field_eligible_by_frame_size( srcIdx ) {
    return RefFrameWidth[ srcIdx ] == FrameWidth &&
           RefFrameHeight[ srcIdx ] == FrameHeight
}

is_ref_motion_field_eligible_by_frame_type( srcIdx ) {
    return RefFrameType[ srcIdx ] != INTRA_ONLY_FRAME &&
           RefFrameType[ srcIdx ] != KEY_FRAME
}

is_ref_motion_field_eligible( srcIdx ) {
    return is_ref_motion_field_eligible_by_frame_type( srcIdx ) &&
           is_ref_motion_field_eligible_by_frame_size( srcIdx )
}

record_tip_projection( ref, isBwd, targetFrame, processCount ) {
    return record_projection_with_type( 1, ref, isBwd, targetFrame,
                                       TIP_MFMV_STACK_SIZE, processCount )
}

record_projection( ref, isBwd, processCount ) {
    return record_projection_with_type( 0, ref, isBwd, -1, TIP_MFMV_STACK_SIZE,
                                       processCount )
}

record_projection_with_type( doingTip, ref, isBwd, targetFrame, maxCheck,
                           processCount ) {
    refIdx = ref_frame_idx[ ref ]
    if ( !is_ref_motion_field_eligible( refIdx ) ) {
        return processCount
    }
    refToCur = get_relative_dist( OrderHints[ ref ], OrderHint )
    if ( Abs(refToCur) > MAX_FRAME_DISTANCE ) {
        return processCount
    }
    if ( use_bru ) {
        if ( bru_ref == ref || (doingTip && bru_ref == targetFrame) ) {
            return processCount
        }
    }
    if ( doingTip ) {
        isBwd = OrderHints[ref] < OrderHints[targetFrame]
    }
    if ( processCount >= maxCheck ||
        MotionFieldChecked[ ref ][ isBwd ] ) {
        return processCount
    }
    MotionFieldChecked[ ref ][ isBwd ] = 1
    c = processCount
    MotionFieldRef[ c ] = ref
    MotionFieldBwd[ c ] = isBwd
    MotionFieldTargetFrame[ c ] = targetFrame
    processCount++
    return processCount
}

```

The recorded projections are processed as follows:

```

if ( reduced_ref_frame_mvs_mode ) {
    processCount = Min( 1, processCount )
}
for( i = 0; i < processCount; i++ ) {
    ref = MotionFieldRef[ i ]
    isBwd = MotionFieldBwd[ i ]

```

```

    targetFrame = MotionFieldTargetFrame[ i ]
    projection(ref,isBwd ? -1 : 1, isBwd , targetFrame)
}

```

The function calls to projection indicate that the projection process specified in [§ 7.9.3 Projection process](#) is invoked.

If enable_mv_traj is equal to 1, the fill trajectory motion vector gap process specified in [§ 7.9.2 Fill trajectory motion vector gap process](#) is invoked.

§ 7.9.2. Fill trajectory motion vector gap process

If ProjStep is not equal to 2, this process immediately terminates.

Otherwise (ProjStep is equal to 2), this process fills in the gaps as follows:

```

w8 = MiCols >> 1
h8 = MiRows >> 1
for( rf = 0; rf < NumTotalRefs ; rf++ ) {
    for ( y8 = 0; y8 < h8 ; y8 += 2 ) {
        for ( x8 = 0; x8 < w8; x8 += 2 ) {
            fill_traj_mv(rf, y8, x8, 0, 1)
            fill_traj_mv(rf, y8, x8, 1, 0)
            fill_traj_mv(rf, y8, x8, 1, 1)
        }
    }
}

```

where the function fill_traj_mv (which fills a specific position) is defined as:

```

fill_traj_mv( rf, y8, x8, dy, dx ) {
    w8 = MiCols >> 1
    h8 = MiRows >> 1
    if ( !TrajValid[ rf ][ y8 ][ x8 ] || y8 + dy == h8 || x8 + dx == w8 ) {
        return
    }
    count = 1
    avgMv = TrajMv[ rf ][ y8 ][ x8 ]
    rAvail = dx > 0 && tmvp_avail(x8, x8 + 2, w8) &&
        TrajValid[ rf ][ y8 ][ x8 + 2 ]
    bAvail = dy > 0 && tmvp_avail(y8, y8 + 2, h8) &&
        TrajValid[ rf ][ y8 + 2 ][ x8 ]
    brAvail = dy > 0 && dx > 0 && tmvp_avail(x8, x8 + 2, w8) &&
        tmvp_avail(y8, y8 + 2, h8) && TrajValid[ rf ][ y8 + 2 ][ x8 + 2 ]
    if (rAvail) {
        count++
        for( c = 0 ; c < 2; c++ ) {
            avgMv[ c ] += TrajMv[ rf ][ y8 ][ x8 + 2 ][ c ]
        }
    }
    if (bAvail) {
        count++
        for( c = 0 ; c < 2; c++ ) {
            avgMv[ c ] += TrajMv[ rf ][ y8 + 2 ][ x8 ][ c ]
        }
    }
    if (brAvail) {
        count++
        for( c = 0 ; c < 2; c++ ) {
            avgMv[ c ] += TrajMv[ rf ][ y8 + 2 ][ x8 + 2 ][ c ]
        }
    }
    for( c = 0 ; c < 2; c++ ) {

```

```

    TrajMv[ rf ][ y8 + dy ][ x8 + dx ][ c ] = calc_avg(avgMv[ c ], count)
}
TrajValid[ rf ][ y8 + dy ][ x8 + dx ] = 1
}

```

The `get_tmvp_shift` function (which specifies the right shift required to convert from a position in terms of multiples of 8 pixels to a position in terms of TMVP units) is specified as:

```

get_tmvp_shift() {
    if ( SbSize == BLOCK_64X64 || ProjStep == 1 ) {
        return 3
    } else {
        return 4
    }
}

```

NOTE: TMVP units are either 64 by 64 (a shift of 3), or 128 by 128 pixels in size (a shift of 4).

The `get_tmvp_unit` function (which converts the position from a multiple of 8 pixels to the TMVP unit) is specified as:

```

get_tmvp_unit( x8 ) {
    return x8 >> get_tmvp_shift()
}

```

The `get_tmvp_phase` function (which specifies the phase of the given TMVP unit) is specified as:

```

get_tmvp_phase( x8 ) {
    return get_tmvp_unit( x8 ) % 3
}

```

NOTE: The TMVP is designed so that all the computation for a TMVP unit depends only on the TMVP unit and its left and right neighbors, and that the computation can happen in parallel. The phase is used to ensure that the computations are kept separate.

The `tmvp_avail` function (which checks that two positions are in the same TMVP unit) is specified as:

```

tmvp_avail( base8, loc8, max8 ) {
    if ( loc8 >= max8 ) {
        return 0
    }
    return get_tmvp_unit( base8 ) == get_tmvp_unit( loc8 )
}

```

§ 7.9.3. Projection process

The inputs to this process are:

- a variable `src` specifying which reference frame's motion vectors are projected,
- a variable `dstSign` specifying a negation multiplier for the motion vector direction,
- a variable `isBwd` specifying if the reference frame has a higher order hint,
- a variable `targetFrame` specifying the target frame (or -1 in some cases).

The process projects the motion vectors from a whole reference frame and stores the results in MotionFieldMvs.

The variable srcIdx (representing which reference frame is used) is set equal to ref_frame_idx[src].

The variable refToCur is set equal to get_relative_dist(OrderHints[src], OrderHint).

The array startRefMap (that will be used during the tracking of motion vector trajectories) is computed as follows:

```
for( k = 0 ; k < RefNumTotalRefs[ srcIdx ] ; k++ ) {
    startRefMap[ k ] = NONE
    for( rf = 0; rf < NumTotalRefs; rf++ ) {
        if ( SavedOrderHints[ srcIdx ][ k ] == OrderHints[ rf ] &&
            OrderHints[ rf ] != OrderHints[ src ] &&
            !( SavedOrderHints[ srcIdx ][ k ] == RESTRICTED_OH ||
              OrderHints[ rf ] == RESTRICTED_OH ) ) {
            startRefMap[ k ] = rf
            break
        }
    }
}
```

The variable w8 (representing the width of the motion field in units of 8x8 luma samples) is set equal to MiCols >> 1.

The variable h8 (representing the height of the motion field in units of 8x8 luma samples) is set equal to MiRows >> 1.

The process is specified as follows:

```
for ( y8 = 0; y8 < h8; y8 += ProjStep ) {
    for ( x8 = 0; x8 < w8; x8 += ProjStep ) {
        list = isBwd
        srcRef = SavedRefFrames[ srcIdx ][ y8 ][ x8 ][ list ]
        if ( is_inter_ref_frame( srcRef ) ) {
            if ( enable_mv_traj ) {
                mv2[ 0 ] = uncompression_mv(
                    SavedMvs[ srcIdx ][ y8 ][ x8 ][ list ][ 0 ] )
                mv2[ 1 ] = uncompression_mv(
                    SavedMvs[ srcIdx ][ y8 ][ x8 ][ list ][ 1 ] )
                check_traj_intersect( src, x8, y8, startRefMap[ srcRef ], mv2 )
            }
            refOffset = get_relative_dist( OrderHints[ src ],
                                           SavedOrderHints[ srcIdx ][ srcRef ] )
            if ( SavedOrderHints[ srcIdx ][ srcRef ] == RESTRICTED_OH ) {
                refOffset = 0
            }
            posValid = Abs( refOffset ) <= MAX_FRAME_DISTANCE
            if ( isBwd ) {
                refOffset = -refOffset
            }
            if ( posValid && refOffset >= 0 ) {
                mv = SavedMvs[ srcIdx ][ y8 ][ x8 ][ list ]
                mv[ 0 ] = uncompression_mv( mv[ 0 ] )
                mv[ 1 ] = uncompression_mv( mv[ 1 ] )
                projMv = get_mv_projection( mv, refToCur * dstSign, refOffset )
                if ( isBwd ) {
                    (posValid, posX8, posY8) = get_sampled_position( x8, y8, 1,
                                                                    projMv )
                } else {
                    (posValid, posX8, posY8) = get_sampled_position( x8, y8,
                                                                    dstSign,

```



```

projMv )
}
posValid = check_block_position(posValid, x8, y8, posX8, posY8)
if ( posValid && ( !MotionFieldValid[ posY8 ][ posX8 ] ||
    ( targetFrame != -1 &&
      targetFrame == startRefMap[srcRef] &&
      MotionFieldOffset[ posY8 ][ posX8 ] != refOffset )
    ) ) {
    if ( enable_mv_traj ) {
        k = get_tmvp_phase( posX8 )
        TrajPos[k][src][y8][x8][0] = posY8
        TrajPos[k][src][y8][x8][1] = posX8
        TrajPosValid[k][src][y8][x8] = 1
        for(c=0;c<2;c++) {
            TrajMv[src][posY8][posX8][c] =
                Clip3( -REFMVS_LIMIT, REFMVS_LIMIT, -projMv[c] )
        }
        TrajValid[src][posY8][posX8] = 1
        endFrame = startRefMap[srcRef]
        if (endFrame != NONE) {
            projMv = get_mv_projection( mv,
                refOffset - refToCur * dstSign,
                refOffset )
            for(c=0;c<2;c++) {
                TrajMv[endFrame][posY8][posX8][c] =
                    Clip3( -REFMVS_LIMIT, REFMVS_LIMIT,
                        projMv[c] )
            }
            TrajValid[endFrame][posY8][posX8] = 1
            (targetValid, targetX8, targetY8) =
                get_sampled_position( x8, y8, 1, mv)
            targetValid = check_block_position( targetValid,
                targetX8,
                targetY8,
                posX8, posY8 )

            if (targetValid) {
                TrajPos[k][endFrame][targetY8][targetX8][0] =
                    posY8
                TrajPos[k][endFrame][targetY8][targetX8][1] =
                    posX8
                TrajPosValid[k][endFrame][targetY8][targetX8]=1
            }
        }
    }
}
if (isBwd) {
    mv[ 0 ] = -mv[ 0 ]
    mv[ 1 ] = -mv[ 1 ]
}
MotionFieldValid[ posY8 ][ posX8 ] = 1
MotionFieldMvs[ posY8 ][ posX8 ] = mv
MotionFieldOffset[ posY8 ][ posX8 ] = refOffset
}
}
}
}
}
}
}

```

When the function `get_mv_projection` is called, the get mv projection process specified in [§7.9.4 Get MV projection process](#) is invoked and the output assigned to `projMv`.

When the function `get_sampled_position` is called, the get sampled position process specified in [§7.9.6 Get sampled position process](#) is invoked and the outputs are assigned to `posValid`, `posX8`, and `posY8`.

The function `check_traj_intersect` (which tries to extend a motion vector trajectory) is specified as:

```

check_traj_intersect(srcFrame, x8, y8, endFrame, mv) {
    if (endFrame == NONE) {
        return
    }
    for( k = 0; k < 3; k++ ) {
        if ( TrajPosValid[ k ][ srcFrame ][ y8 ][ x8 ] != 0 ) {
            trajY8 = TrajPos[ k ][ srcFrame ][ y8 ][ x8 ][ 0 ]
            trajX8 = TrajPos[ k ][ srcFrame ][ y8 ][ x8 ][ 1 ]
            if ( !TrajValid[ endFrame ][ trajY8 ][ trajX8 ] ) {
                for( c = 0; c < 2; c++ ) {
                    v = TrajMv[ srcFrame ][ trajY8 ][ trajX8 ][ c ] + mv[ c ]
                    TrajMv[ endFrame ][ trajY8 ][ trajX8 ][ c ] =
                        Clip3(-REFMVS_LIMIT, REFMVS_LIMIT, v)
                }
                TrajValid[ endFrame ][ trajY8 ][ trajX8 ] = 1
                (posValid,posX8,posY8) = get_sampled_position(
                    trajX8, trajY8, 1, TrajMv[ endFrame ][ trajY8 ][ trajX8 ] )
                posValid = check_block_position( posValid, posX8, posY8, trajX8,
                                                trajY8)

                if (posValid) {
                    TrajPos[ k ][ endFrame ][ posY8 ][ posX8 ][ 0 ] = trajY8
                    TrajPos[ k ][ endFrame ][ posY8 ][ posX8 ][ 1 ] = trajX8
                    TrajPosValid[ k ][ endFrame ][ posY8 ][ posX8 ] = 1
                }
            }
        }
    }
    (posValid,endX8,endY8) = get_sampled_position( x8, y8, 1, mv )
    if ( !posValid ) {
        return
    }
    for( k = 0; k < 3; k++ ) {
        if ( TrajPosValid[ k ][ endFrame ][ endY8 ][ endX8 ] != 0 ) {
            trajY8 = TrajPos[ k ][ endFrame ][ endY8 ][ endX8 ][ 0 ]
            trajX8 = TrajPos[ k ][ endFrame ][ endY8 ][ endX8 ][ 1 ]
            if ( check_block_position(1, x8, y8, trajX8, trajY8) &&
                !TrajValid[ srcFrame ][ trajY8 ][ trajX8 ] ) {
                for( c = 0; c < 2; c++ ) {
                    v = TrajMv[ endFrame ][ trajY8 ][ trajX8 ][ c ] - mv[ c ]
                    TrajMv[ srcFrame ][ trajY8 ][ trajX8 ][ c ] =
                        Clip3(-REFMVS_LIMIT, REFMVS_LIMIT, v)
                }
                TrajValid[ srcFrame ][ trajY8 ][ trajX8 ] = 1
                (posValid,posX8,posY8) = get_sampled_position(
                    trajX8, trajY8, 1, TrajMv[ srcFrame ][ trajY8 ][ trajX8 ] )
                posValid = check_block_position( posValid, posX8, posY8, trajX8,
                                                trajY8)

                if (posValid) {
                    TrajPos[ k ][ srcFrame ][ posY8 ][ posX8 ][ 0 ] = trajY8
                    TrajPos[ k ][ srcFrame ][ posY8 ][ posX8 ][ 1 ] = trajX8
                    TrajPosValid[ k ][ srcFrame ][ posY8 ][ posX8 ] = 1
                }
            }
        }
    }
}

```

The function calls to `check_block_position` indicate that the check block position process specified in [§7.9.8 Check block position process](#) is invoked.

§ 7.9.4. Get MV projection process

The inputs to this process are:

- a length 2 array mv specifying a motion vector,
- a variable numerator specifying the number of frames to be covered by the projected motion vector,
- a variable denominator specifying the number of frames covered by the original motion vector.

The outputs of this process are:

- a length 2 array projMv containing the projected motion vector.

This process starts with a motion vector mv. This motion vector gives the displacement expected when moving a certain number of frames (given by the variable denominator). In order to use the motion vector for predictions using a different reference frame, the length of the motion vector must be scaled.

The variable clippedDenominator is set equal to $\text{Min}(\text{MAX_FRAME_DISTANCE}, \text{denominator})$.

The variable clippedNumerator is set equal to $\text{Clip3}(-\text{MAX_FRAME_DISTANCE}, \text{MAX_FRAME_DISTANCE}, \text{numerator})$.

The projected motion vector is specified as follows:

```
for ( i = 0; i < 2; i++ ) {  
    scaled = Round2Signed( mv[ i ] * clippedNumerator *  
                           Div_Mult[ clippedDenominator ], 14 )  
    projMv[ i ] = Clip3( MV_LOW + 1, MV_UPP - 1, scaled )  
}
```

where Div_Mult is a constant lookup table specified as:

```
Div_Mult[32] = {  
    0,    16384, 8192, 5461, 4096, 3276, 2730, 2340, 2048, 1820, 1638,  
    1489, 1365, 1260, 1170, 1092, 1024, 963, 910, 862, 819, 780,  
    744, 712, 682, 655, 630, 606, 585, 564, 546, 528  
}
```

§ 7.9.5. Get MV projection clamp process

The inputs to this process are:

- a length 2 array mv specifying a motion vector,
- a variable numerator specifying the number of frames to be covered by the projected motion vector,
- a variable denominator specifying the number of frames covered by the original motion vector.

The outputs of this process are:

- a length 2 array projMv containing the projected motion vector.

The get mv projection process specified in [§ 7.9.4 Get MV projection process](#) is invoked with mv, numerator, and denominator as inputs, and the output is assigned to projMv.

The projected motion vector is clamped to a tighter range as follows:

```
for ( i = 0; i < 2; i++ ) {  
    projMv[ i ] = Clip3( -REFMVS_LIMIT, REFMVS_LIMIT, projMv[ i ] )  
}
```

§ 7.9.6. Get sampled position process

The inputs to this process are:

- variables x8 and y8 specifying a location in units of 8x8 luma samples,
- a variable dstSign specifying a negation multiplier for the motion vector direction,
- a length 2 array projMv specifying a projected motion vector.

The get block position no constraint process specified in [§ 7.9.7 Get block position no constraint process](#) is invoked with x8, y8, dstSign, and projMv as inputs, and the outputs are assigned to posValid, posX8, and posY8.

If ProjStep is equal to 2, the position is changed to an even location as follows:

```
posX8 -= posX8 & 1  
posY8 -= posY8 & 1
```

The outputs of this process are the variables posValid, posX8, and posY8.

§ 7.9.7. Get block position no constraint process

The inputs to this process are:

- variables x8 and y8 specifying a location in units of 8x8 luma samples,
- a variable dstSign specifying a negation multiplier for the motion vector direction,
- a length 2 array projMv specifying a projected motion vector.

The process returns a flag posValid that indicates if the position is to be used and variables posX8 and posY8 representing the projected location in units of 8x8 luma samples.

NOTE: This function does not check the constraints of being close to the current TMVP unit.

The variable posValid is set equal to 1.

The variable posY8 is set equal to project_no_constraint(y8, projMv[0], dstSign, MiRows >> 1).

The variable posX8 is set equal to project_no_constraint(x8, projMv[1], dstSign, MiCols >> 1).

where the function project_no_constraint is specified as follows:

```
project_no_constraint( v8, delta, dstSign, max8 ) {  
    if ( delta >= 0 ) {  
        offset8 = delta >> ( 3 + 1 + MI_SIZE_LOG2 )  
    } else {  
        offset8 = - ( ( -delta ) >> ( 3 + 1 + MI_SIZE_LOG2 ) )  
    }  
}
```

```

v8 += dstSign * offset8
if ( v8 < 0 ||
    v8 >= max8 ) {
    posValid = 0
}
return v8
}

```

The `project_no_constraint` function clears `posValid` if the resulting position is offset too far.

The outputs of this process are the variables `posValid`, `posX8`, and `posY8`.

§ 7.9.8. Check block position process

The inputs to this process are:

- a variable `posValid` specifying if the location is valid according to previous checks,
- variables `posX8` and `posY8` specifying a location to be checked in units of 8x8 luma samples,
- variables `baseX8` and `baseY8` specifying a base location in units of 8x8 luma samples.

The output of this process is the variable `posValid` that indicates if the checked position is sufficiently close to the base position.

If `posValid` is equal to 0, the process terminates immediately with 0 as output.

Otherwise, the position is checked as follows:

```

shift = get_tmvp_shift()
for (ord = 0; ord < 2; ord++) {
    v8 = ord ? baseX8 : baseY8
    sb0ff8 = 1 << shift
    if ( ProjStep > 1 ) {
        max0ff8 = ord ? sb0ff8 : 0
    } else {
        max0ff8 = ord ? sb0ff8 >> 1 : 0
    }
    base8 = (v8 >> shift) << shift
    pos8 = ord ? posX8 : posY8
    if (pos8 < base8 - max0ff8 ||
        pos8 >= base8 + sb0ff8 + max0ff8 ) {
        return 0
    }
}
return 1

```

§ 7.10. Setup [TIP](#) motion field process

§ 7.10.1. General

This process is triggered by a call to `setup_tip_motion_field` while reading the [frame header info](#).

The estimated motion field is temporally scaled based on the frames chosen for [TIP](#), and the [TIP](#) frame is constructed if `TipFrameMode` is equal to `TIP_FRAME_AS_OUTPUT`.

It is a requirement of bitstream conformance that all the following conditions are true whenever this process is triggered:

- The `FrameType` is not equal to `SWITCH_FRAME` (indicating that the frame is not a [switch frame](#)),

- ClosestPast is not equal to NONE,
- ClosestFuture is not equal to NONE,
- TipFrameMode is not equal to TIP_FRAME_DISABLED,
- use_ref_frame_mvs is equal to 1,
- HasBothRefs is equal to 1 or NumPastRefs is greater than or equal to 2,
- is_ref_motion_field_eligible_by_frame_type(ref_frame_idx[ClosestPast]) is true or is_ref_motion_field_eligible_by_frame_type(ref_frame_idx[ClosestFuture]) is true,
- is_ref_motion_field_eligible_by_frame_size(ref_frame_idx[ClosestPast]) is true and is_ref_motion_field_eligible_by_frame_size(ref_frame_idx[ClosestFuture]) is true.

The following ordered steps now apply:

1. The [TIP](#) temporal scale motion field process specified in [§ 7.10.2 TIP temporal scale motion field process](#) is invoked.
2. If allow_tip_hole_fill is equal to 1, the following ordered steps apply:
 1. The [TIP](#) fill motion field holes process specified in [§ 7.10.3 TIP fill motion field holes process](#) is invoked.
 2. The [TIP](#) block average filter motion vector process specified in [§ 7.10.4 TIP block average filter motion vector process](#) is invoked.
3. The fill temporal motion vectors sample gap process specified in [§ 7.10.5 Fill temporal motion vectors sample gap process](#) is invoked.
4. If TipFrameMode is equal to TIP_FRAME_AS_OUTPUT, the build [TIP](#) process specified in [§ 7.10.6 Build TIP process](#) is invoked.

§ 7.10.2. [TIP](#) temporal scale motion field process

The variable refOffset is set as follows:

```
(refOffset, _, _) = get_tip_offsets()
```

The variable w8 is set equal to MiCols >> 1.

The variable h8 is set equal to MiRows >> 1.

The motion field is scaled as follows:

```
for ( y8 = 0; y8 < h8 ; y8 += ProjStep ) {
    for ( x8 = 0; x8 < w8; x8 += ProjStep ) {
        mv = MotionFieldMvs[ y8 ][ x8 ]
        if ( MotionFieldValid[ y8 ][ x8 ] ) {
            startOffset = MotionFieldOffset[ y8 ][ x8 ]
            MotionFieldMvs[ y8 ][ x8 ] = get_mv_projection_clamp( mv, refOffset,
                                                                    startOffset )
            MotionFieldValid[ y8 ][ x8 ] = 1
        }
        MotionFieldOffset[ y8 ][ x8 ] = refOffset
    }
}
```

When the function `get_mv_projection_clamp` is called, the `get mv projection clamp` process specified in [§7.9.5 Get MV projection clamp process](#) is invoked.

§ 7.10.3. **TIP** fill motion field holes process

This process fills in holes in the motion field.

The filling is constrained to only look at locations within the same superblock (or within the same 128 by 128 block if superblocks are 256 by 256).

The motion vector filling is applied as follows:

```

step = ProjStep
sbSize8 = 1 << get_tmvp_shift()
w8 = MiCols >> 1
h8 = MiRows >> 1
for ( y8 = 0; y8 < h8 ; y8 += sbSize8 ) {
    for ( x8 = 0; x8 < w8; x8 += sbSize8 ) {
        endRow8 = Min(y8 + sbSize8, h8)
        endCol8 = Min(x8 + sbSize8, w8)
        for( row8 = y8; row8 < endRow8; row8 += step ) {
            for( col8 = x8; col8 < endCol8; col8 += step ) {
                for( dir = 0; dir < 4; dir++) {
                    dstRow8 = row8 + Tip_Dirs[ dir ][ 0 ] * step
                    dstCol8 = col8 + Tip_Dirs[ dir ][ 1 ] * step
                    if ( dstRow8 >= y8 && dstRow8 < endRow8 &&
                        dstCol8 >= x8 && dstCol8 < endCol8 &&
                        !MotionFieldValid[dstRow8][dstCol8] ) {
                        MotionFieldValid[ dstRow8 ][ dstCol8 ] =
                            MotionFieldValid[ row8 ][ col8 ]
                        for ( j = 0; j < 2; j++ )
                            MotionFieldMvs[ dstRow8 ][ dstCol8 ][ j ] =
                                MotionFieldMvs[ row8 ][ col8 ][ j ]
                        MotionFieldOffset[ dstRow8 ][ dstCol8 ] =
                            MotionFieldOffset[ row8 ][ col8 ]
                    }
                }
            }
        }
    }
}

```

where the constant table `Tip_Dirs` is specified as:

```

Tip_Dirs[ 5 ][ 2 ] = {
    { -1, 0 }, { 0, -1 }, { 1, 0 }, { 0, 1 }, { 0, 0 }
}

```

§ 7.10.4. **TIP** block average filter motion vector process

This process smooths the motion field by averaging motion vectors.

The averaging is constrained to only look at locations within the same superblock (or within the same 128 by 128 block if superblocks are 256 by 256).

The motion vectors are averaged and applied as follows:

```

step = ProjStep
sbSize8 = 1 << get_tmvp_shift()
w8 = MiCols >> 1

```

```

h8 = MiRows >> 1
for ( y8 = 0; y8 < h8 ; y8 += sbSize8 ) {
    for ( x8 = 0; x8 < w8; x8 += sbSize8 ) {
        endRow8 = Min(y8 + sbSize8, h8)
        endCol8 = Min(x8 + sbSize8, w8)
        for( row8 = y8; row8 < endRow8; row8 += step ) {
            for( col8 = x8; col8 < endCol8; col8 += step ) {
                mv[0] = 0
                mv[1] = 0
                count = 0
                for( dir = 0; dir < 5; dir++) {
                    dstRow8 = row8 + Tip_Dirs[ dir ][ 0 ] * step
                    dstCol8 = col8 + Tip_Dirs[ dir ][ 1 ] * step
                    if ( dstRow8 >= y8 && dstRow8 < endRow8 &&
                        dstCol8 >= x8 && dstCol8 < endCol8 &&
                        MotionFieldValid[dstRow8][dstCol8] ) {
                        for ( j = 0; j < 2; j++ )
                            mv[j] += MotionFieldMvs[ dstRow8 ][ dstCol8 ][ j ]
                        count += 1
                    }
                }
                if (count == 0) {
                    avgValid[ row8 ][ col8 ] = 0
                    avgMotionFieldMvs[ row8 ][ col8 ][ 0 ] = -(1<<15)
                    avgMotionFieldMvs[ row8 ][ col8 ][ 1 ] = -(1<<15)
                } else {
                    avgValid[ row8 ][ col8 ] = 1
                    for(j=0;j<2;j++) {
                        avgMotionFieldMvs[ row8 ][ col8 ][ j ] =
                            Round2Signed( mv[j] * Weight_Div_Mult[count], 16 )
                    }
                }
            }
        }
    }
}
for ( y8 = 0; y8 < h8 ; y8 += step ) {
    for ( x8 = 0; x8 < w8; x8 += step ) {
        for (comp = 0; comp < 2; comp++) {
            MotionFieldMvs[ y8 ][ x8 ][ comp ] =
                avgMotionFieldMvs[ y8 ][ x8 ][ comp ]
        }
        MotionFieldValid[ y8 ][ x8 ] = avgValid[ y8 ][ x8 ]
    }
}

```

where the constant table `Weight_Div_Mult` is specified as:

```

Weight_Div_Mult[6] = {
    0, 65536, 32768, 21845, 16384, 13107
}

```

NOTE: Multiplication by an entry in `Weight_Div_Mult` approximates a division by the value of the index.

§ 7.10.5. Fill temporal motion vectors sample gap process

At this stage the motion field is defined with a sampling step of `ProjStep 8x8s`.

This process fills in gaps so that the motion field is defined at every location.

If `ProjStep` is not equal to 2, this process terminates immediately.

Otherwise, the gaps are filled in as follows:

```
w8 = MiCols >> 1
h8 = MiRows >> 1
for ( y8 = 0; y8 < h8 ; y8 += 2 ) {
    for ( x8 = 0; x8 < w8; x8 += 2 ) {
        fill_tpl(y8, x8, 0, 1)
        fill_tpl(y8, x8, 1, 0)
        fill_tpl(y8, x8, 1, 1)
    }
}
```

where the fill_tpl function fills in a single gap as follows:

```
fill_tpl( y8, x8, dy, dx ) {
    w8 = MiCols >> 1
    h8 = MiRows >> 1
    if ( !MotionFieldValid[ y8 ][ x8 ] || y8 + dy == h8 || x8 + dx == w8 ) {
        return
    }
    curOffset = MotionFieldOffset[ y8 ][ x8 ]
    count = 0
    for ( c = 0; c < 2; c++ ) {
        avgMv[ c ] = 0
    }
    for ( i = 0; i < 4; i++ ) {
        candX = i & 1
        candY = i >> 1
        available = ( dy >= candY &&
                     dx >= candX &&
                     tmpv_avail( x8, x8 + 2 * candX, w8 ) &&
                     tmpv_avail( y8, y8 + 2 * candY, h8 ) &&
                     MotionFieldValid[ y8 + 2 * candY ][ x8 + 2 * candX ] )
        if (available) {
            count++
            if ( i == 0 ) {
                projMv = MotionFieldMvs[ y8 + 2 * candY ][ x8 + 2 * candX ]
            } else {
                projMv = get_mv_projection_clamp(
                    MotionFieldMvs[ y8 + 2 * candY ][ x8 + 2 * candX ],
                    curOffset,
                    MotionFieldOffset[ y8 + 2 * candY ][ x8 + 2 * candX ] )
            }
            for ( c = 0; c < 2; c++ ) {
                avgMv[ c ] += projMv[ c ]
            }
        }
    }
    MotionFieldOffset[ y8 + dy ][ x8 + dx ] = curOffset
    for ( c = 0 ; c < 2; c++ ) {
        MotionFieldMvs[ y8 + dy ][ x8 + dx ][ c ] = calc_avg (avgMv[ c ], count)
    }
    MotionFieldValid[ y8 + dy ][ x8 + dx ] = 1
}
```

The function calc_avg performs approximate division with rounding as follows:

```
calc_avg(n, d) {
    if ( d == 1 ) {
        return n
    } else if ( d == 2 ) {
        return Round2Signed( n, 1 )
    } else if ( d == 3 ) {
        return Round2Signed( n * 85, 8 )
    } else {

```

```

        return Round2Signed( n, 2 )
    }
}

```

When the function `get_mv_projection_clamp` is called, the `get mv projection clamp` process specified in [§ 7.9.5 Get MV projection clamp process](#) is invoked.

§ 7.10.6. Build **TIP** process

This process builds samples in the current frame out of 8 by 8 blocks coded in **TIP** mode as follows:

```

RefFrame[0] = ClosestPast
RefFrame[1] = ClosestFuture
motion_mode = SIMPLE
use_bawp = 0
compound_type = COMPOUND_AVERAGE
CwpIdx = Tip_Weighting_Factor[ tip_global_wtd_index ]
YMode = NEWMV
use_intrabc = 0
use_optflow = opfl_refine_type != REFINES_NONE &&
                TipInterpFilter == EIGHTTAP_SHARP && enable_tip_refinemv
DecidedAgainstRefinemv = 0
(refOffset, pastOffset, futureOffset) = get_tip_offsets()
tipSize = ( enable_tip_refinemv &&
            TipInterpFilter == EIGHTTAP_SHARP ) ? BLOCK_8X8 : BLOCK_16X16
storeRefinedMvs = store_refined_mvs()
use_refinemv = 0
for( row = 0; row < MiRows; row += Num_4x4_Blocks_High[tipSize] ) {
    for( col = 0; col < MiCols; col += Num_4x4_Blocks_Wide[tipSize] ) {
        for( i = 0; i < Num_4x4_Blocks_High[tipSize]; i++ ) {
            for( j = 0; j < Num_4x4_Blocks_Wide[tipSize]; j++ ) {
                RefFrames[row+i][col+j][0] = ClosestPast
                RefFrames[row+i][col+j][1] = ClosestFuture
            }
        }
        y8 = row >> 1
        x8 = col >> 1
        if ( !MotionFieldValid[ y8 ][ x8 ] ) {
            localMvs[ 0 ][ 0 ] = 0
            localMvs[ 0 ][ 1 ] = 0
            localMvs[ 1 ][ 0 ] = 0
            localMvs[ 1 ][ 1 ] = 0
        } else {
            localMvs[ 0 ] = get_mv_projection( MotionFieldMvs[ y8 ][ x8 ],
                                                pastOffset, refOffset )
            localMvs[ 1 ] = get_mv_projection( MotionFieldMvs[ y8 ][ x8 ],
                                                futureOffset, refOffset )
        }
        for( comp = 0; comp < 2; comp++ ) {
            BlockMvs[ 0 ][ comp ] = localMvs[ 0 ][ comp ] + TipGlobalMv[ comp ]
            BlockMvs[ 1 ][ comp ] = localMvs[ 1 ][ comp ] + TipGlobalMv[ comp ]
        }
        for( i = 0; i < Num_4x4_Blocks_High[tipSize]; i++ ) {
            for( j = 0; j < Num_4x4_Blocks_Wide[tipSize]; j++ ) {
                for( list = 0; list < 2; list++ ) {
                    Mvs[ row + i ][ col + j ][ list ] = BlockMvs[ list ]
                }
            }
        }
    }
}
for( plane = 0; plane < NumPlanes; plane++ ) {
    if( plane == 0 ) {
        subX = 0
        subY = 0
    } else {
        subX = SubsamplingX
        subY = SubsamplingY
    }
}

```

```

    bw = Block_Width[ tipSize ] >> subX
    bh = Block_Height[ tipSize ] >> subY
    x = (col * MI_SIZE) >> subX
    y = (row * MI_SIZE) >> subY
    predict_inter(plane, x, y, bw, bh, row, col, 1, 0)
    if ( plane == 0 ) {
        if ( storeRefinedMvs ) {
            motion_field_motion_vector_storage(row, col, tipSize,
                LumaUseOptflowRefinement ? 1 : 2)
        } else {
            motion_field_motion_vector_storage(row, col, tipSize, 0 )
        }
    }
}
}
}
}
}

```

The function call to `motion_field_motion_vector_storage` indicates that the motion field motion vector storage process specified in [§ 7.22 Motion field motion vector storage process](#) is invoked.

§ 7.11. Motion vector context processes

§ 7.11.1. General

The following sections define the processes used for getting the context needed for reading motion vectors.

The entry point to these processes is triggered by a function call to `find_mode_ctx`.

This function call invokes the find mode context process specified in [§ 7.11.2 Find mode context process](#).

§ 7.11.2. Find mode context process

This process is triggered by a function call to `find_mode_ctx`.

The input to this process is a variable `isCompound` containing 0 for single prediction, or 1 to signal compound prediction.

The variable `NewMvCount` is set equal to 0.

The variable `WarpMvCount` is set equal to 0.

The variable `WarpSampleFound[ 0 ]` is set equal to 0.

The variable `WarpSampleFound[ 1 ]` is set equal to 0.

Locations around the block are scanned as follows:

```

bw4 = Num_4x4_Blocks_Wide[ MiSize ]
bh4 = Num_4x4_Blocks_High[ MiSize ]
isSbBorder = ( MiRow & (Num_4x4_Blocks_High[ SbSize ] - 1) ) == 0 ? 1 : 0
scan_point_warp_ctx(bh4 - 1, -1)
leftA = scan_point_ctx( bh4 - 1, -1, isCompound )
scan_point_warp_ctx( -1, isSbBorder ? Max(0, bw4 - 2) : bw4 - 1 )
aboveA = scan_point_ctx( -1, bw4 - 1, isCompound )
scan_point_warp_ctx( 0, -1 )
leftB = scan_point_ctx( 0, -1, isCompound )
if ( bw4 >= (isSbBorder ? 4 : 2) ) {

```

```

    scan_point_warp_ctx( -1, 0)
}
aboveB = scan_point_ctx( -1, 0, isCompound )

```

where a call of `scan_point_ctx` indicates that the scan point context process specified in [§ 7.11.3 Scan point context process](#) is invoked and a call of `scan_point_warp_ctx` indicates that the scan point warp context process [§ 7.11.4 Scan point warp context process](#) is invoked.

The variable `NewMvContext` is set as follows:

```

nearestMatch = ((aboveA || aboveB) ? 1 : 0) + ((leftA || leftB) ? 1 : 0)
NewMvContext = nearestMatch + ((NewMvCount > 0) ? 2 : 0)

```

§ 7.11.3. Scan point context process

The inputs to this process are:

- a variable `deltaRow` specifying (in units of 4x4 luma samples) how far above to look for a motion vector,
- a variable `deltaCol` specifying (in units of 4x4 luma samples) how far left to look for a motion vector,
- a variable `isCompound` containing 0 for single prediction, or 1 to signal compound prediction.

This process updates the variable `NewMvCount`.

The variable `mvRow` is set equal to `MiRow + deltaRow`.

The variable `mvCol` is set equal to `MiCol + deltaCol`.

The variable `found` (specifying if a block with matching references has been found) is computed as follows:

```

found = 0
if ( is_inside( mvRow, mvCol ) ) {
    if ( IsInters[ mvRow ][ mvCol ] ) {
        candMode = YModes[ mvRow ][ mvCol ]
        if ( isCompound == 0 ) {
            for ( candList = 0; candList < 2 - use_intrabc; candList++ ) {
                if ( RefFrames[ mvRow ][ mvCol ][ candList ] == RefFrame[0] ) {
                    if ( has_newmv_for_list( candMode, candList ) ) {
                        NewMvCount = Min(3, NewMvCount + 1)
                    }
                    found = 1
                    break
                }
            }
        }
    } else {
        if ( RefFrames[ mvRow ][ mvCol ][ 0 ] == TIP_FRAME &&
            ClosestPast == RefFrame[ 0 ] &&
            ClosestFuture == RefFrame[ 1 ] ) {
            found = 1
        }
        if ( RefFrames[ mvRow ][ mvCol ][ 0 ] == RefFrame[ 0 ] &&
            RefFrames[ mvRow ][ mvCol ][ 1 ] == RefFrame[ 1 ] ) {
            found = 1
        }
    }
    if ( found > 0 ) {
        if ( has_newmv( candMode ) ) {
            NewMvCount = Min( 3, NewMvCount + found )
        }
    }
}

```

```

    }
  }
}

```

where `has_newmv_for_list` is specified as:

```

has_newmv_for_list( candMode, refList ) {
  if ( candMode == NEW_NEWMV || candMode == NEWMV ) {
    return 1
  }
  if ( refList == 0 ) {
    return candMode == NEW_NEARMV || candMode == JOINT_NEWMV
  } else {
    return candMode == NEAR_NEWMV
  }
}

```

If found is greater than 0, the output of this process is 1.

Otherwise, the output of this process is 0.

§ 7.11.4. Scan point warp context process

The inputs to this process are:

- a variable `deltaRow` specifying (in units of 4x4 luma samples) how far above to look for a motion vector,
- a variable `deltaCol` specifying (in units of 4x4 luma samples) how far left to look for a motion vector.

This process updates the variable `WarpMvCount` (counting the number of matching warp blocks) and the array `WarpSampleFound` (specifying if there are blocks with matching reference frames that may be used for warp).

`ExtendDeltaRow` and `ExtendDeltaCol` record the first place where a potential block for extended warp was found.

The position is adjusted to an aligned location on a superblock border as follows:

```

isSbBorder = ( MiRow & (Num_4x4_Blocks_High[ SbSize ] - 1) ) == 0
if ( deltaRow < 0 && isSbBorder ) {
  deltaCol -= MiCol & 1
}

```

NOTE: The intention is for the memory requirement for warp parameters to be reduced by only using even mode info locations.

The variable `mvRow` is set equal to `MiRow + deltaRow`.

The variable `mvCol` is set equal to `MiCol + deltaCol`.

If `is_inside( mvRow, mvCol )` is equal to 1 and `RefFrames[ mvRow ][ mvCol ][ 0 ]` has been written for this frame (this checks that the candidate location has been decoded) and `IsInters[ mvRow ][ mvCol ]` is equal to 1, the variables are updated as follows:

```

if ( RefFrames[ mvRow ][ mvCol ][ 0 ] == RefFrame[ 0 ] ||
    RefFrames[ mvRow ][ mvCol ][ 1 ] == RefFrame[ 0 ] ) {
    if ( !WarpSampleFound[ 0 ] ) {
        ExtendDeltaRow = deltaRow
        ExtendDeltaCol = deltaCol
    }
    WarpSampleFound[ 0 ] = 1
    if ( MotionModes[ mvRow ][ mvCol ] >= LOCALWARP ) {
        WarpMvCount++
    }
}
if ( RefFrames[ mvRow ][ mvCol ][ 0 ] == RefFrame[ 1 ] ||
    RefFrames[ mvRow ][ mvCol ][ 1 ] == RefFrame[ 1 ] ) {
    WarpSampleFound[ 1 ] = 1
}

```

§ 7.12. Motion vector prediction processes

§ 7.12.1. General

The following sections define the processes used for predicting the motion vectors.

The entry point to these processes is triggered by the function call to `find_mv_stack` in the inter block mode info syntax described in [§ 5.20.7.6 Inter block mode info syntax](#). This function call invokes the Find MV Stack Process specified in [§ 7.12.2 Find MV stack process](#).

§ 7.12.2. Find MV stack process

This process is triggered by a function call to `find_mv_stack`.

The input to this process is a variable `isCompound` containing 0 for single prediction, or 1 to signal [compound prediction](#).

This process constructs an array `RefStackMv` containing motion vector candidates.

If `DeriveWrl` is equal to 1, array `WarpParamStack` will also be constructed and `NumWarpFound` set to indicate the number of candidates in these arrays.

The process also prepares the value of the contexts used when decoding inter prediction syntax elements.

The array `RefStackMv` will be constructed during this process. `RefStackMv[ idx ][ list ][ comp ]` represents component `comp` (0 for y or 1 for x) of a motion vector for a particular `list` (0 or 1) at position `idx` (0 to `MAX_REF_MV_STACK_SIZE - 1`) in the stack.

The variable `SingleMvCount` is set equal to 0.

The variable `DerivedMvCount` is set equal to 0.

The variable `PruneCount` is set equal to 0.

The variable `SinglePruneCount` is set equal to 0.

The variable `DerivedPruneCount` is set equal to 0.

The variable NumWarpFound is set equal to 0.

The motion vector and warp parameter stacks are initialized as follows:

```
for( i = 0; i < MAX_REF_MV_STACK_SIZE; i++ ) {
    RefStackRowOffset[ i ] = 0
    RefStackColOffset[ i ] = 0
    for( list = 0; list < 2; list++ ) {
        for( comp = 0; comp < 2; comp++ ) {
            RefStackMv[ i ][ list ][ comp ] = 0
        }
    }
    RefStackCwp[ i ] = CWP_EQUAL
    if ( i < MAX_WARP_REF_CANDIDATES ) {
        for( j = 0; j < 6; j++ ) {
            WarpParamStack[ i ][ j ] = Default_Warp_Params[ j ]
        }
    }
}
```

The variable bw4 specifying the width of the block in 4x4 luma samples is set equal to Num_4x4_Blocks_Wide[MiSize].

The variable bh4 specifying the height of the block in 4x4 luma samples is set equal to Num_4x4_Blocks_High[MiSize].

The variables useTemporal (specifying if the temporal scan process is used) and useTemporalFirst (specifying if the temporal scan is done before other prediction steps) and isSbBorder (specifying if the block is at the top edge of a superblock) are specified as:

```
useTemporal = ( use_ref_frame_mvs == 1 && !use_intrabc &&
    RefFrame[ 0 ] != TIP_FRAME &&
    ( skip_mode || RefFrame[ 0 ] != RefFrame[ 1 ] ) )
useTemporalFirst = ( DrlReorder != DRL_REORDER_ALWAYS &&
    use_ref_frame_mvs &&
    RefFrame[ 1 ] == NONE &&
    is_inter_ref_frame( RefFrame[ 0 ] ) &&
    RefFrame[ 0 ] != TIP_FRAME &&
    (OrigClosestFuture == NONE || OrigClosestPast == NONE) &&
    Abs(get_relative_dist( OrderHint,
        OrderHints[ RefFrame[ 0 ] ] )) <= 2
    )
isSbBorder = ( MiRow & (Num_4x4_Blocks_High[ SbSize ] - 1) ) == 0 ? 1 : 0
```

The following ordered steps apply:

1. The variable NumMvFound (representing the number of motion vector candidates in RefStackMv) is set equal to 0.
2. The setup global mv process specified in [§ 7.12.2.1 Setup global MV process](#) is invoked with the input 0 and the output is assigned to GlobalMvs[0].
3. If isCompound is equal to 1, the setup global mv process specified in [§ 7.12.2.1 Setup global MV process](#) is invoked with the input 1 and the output is assigned to GlobalMvs[1].
4. If DeriveWrl is equal to 1, the generate points from corners process specified in [§ 7.12.2.3 Generate points from corners process](#) is invoked with the input 0.

5. If DeriveWrl is equal to 1 and NumWarpFound is equal to 0 and Num_4x4_Blocks_Wide[MiSize] is less than or equal to 16, the generate points from corners process specified in [§ 7.12.2.3 Generate points from corners process](#) is invoked with the input 1.
6. If useTemporal is equal to 1 and useTemporalFirst is equal to 1, the temporal scan process in [§ 7.12.2.7 Temporal scan process](#) is invoked with isCompound as input.
7. The scan point process in [§ 7.12.2.6 Scan point process](#) is invoked with deltaRow equal to bh4 - 1, deltaCol equal to -1, and isCompound as inputs.
8. The scan point process in [§ 7.12.2.6 Scan point process](#) is invoked with deltaRow equal to -1, deltaCol equal to Max(0, bw4 - 1 - isSbBorder), and isCompound as inputs.
9. If bh4 is greater than or equal to 2, the scan point process in [§ 7.12.2.6 Scan point process](#) is invoked with deltaRow equal to 0, deltaCol equal to -1, and isCompound as inputs.
10. If bw4 is greater than or equal to (isSbBorder ? 4 : 2), the scan point process in [§ 7.12.2.6 Scan point process](#) is invoked with deltaRow equal to -1, deltaCol equal to 0, and isCompound as inputs.
11. If bh4 is less than or equal to 16, the scan point process in [§ 7.12.2.6 Scan point process](#) is invoked with deltaRow equal to bh4, deltaCol equal to -1, and isCompound as inputs.
12. If bw4 is less than or equal to 16, the scan point process in [§ 7.12.2.6 Scan point process](#) is invoked with deltaRow equal to -1, deltaCol equal to isSbBorder ? Max(2, bw4) : bw4, and isCompound as inputs.
13. If useTemporal is equal to 1 and useTemporalFirst is equal to 0, the temporal scan process in [§ 7.12.2.7 Temporal scan process](#) is invoked with isCompound as input.
14. The scan point process in [§ 7.12.2.6 Scan point process](#) is invoked with deltaRow equal to -1, deltaCol equal to -1 - isSbBorder, and isCompound as inputs.
15. The variable numNearest (representing the number of motion vectors found in the immediate neighborhood) is set equal to NumMvFound.
16. The scan col process in [§ 7.12.2.5 Scan col process](#) is invoked with deltaCol equal to -3 and isCompound as inputs.
17. The variable useSort is set equal to DrlReorder == DRL_REORDER_ALWAYS || (DrlReorder == DRL_REORDER_CONSTRAINT && !useTemporalFirst && numNearest >= 4).
18. If useSort is equal to 1, the sorting process in [§ 7.12.2.19 Sorting process](#) is invoked with start equal to 0, end equal to numNearest, and isCompound as input.
19. If isCompound is equal to 1, the fill mvp from derived smvp process in [§ 7.12.2.22 Fill mvp from derived smvp process](#) is invoked with isCompound as input.
20. If enable_refmvpbank is equal to 1, the fill mvp from ref mv bank process in [§ 7.12.2.21 Fill mvp from ref mv bank process](#) is invoked with isCompound as input.
21. If isCompound is equal to 0, the fill mvp from derived smvp process in [§ 7.12.2.22 Fill mvp from derived smvp process](#) is invoked with isCompound as input.
22. The extra search process in [§ 7.12.2.20 Extra search process](#) is invoked with isCompound as input.
23. The clamping process in [§ 7.12.2.23 Clamping process](#) is invoked with isCompound as input.

[§ 7.12.2.1. Setup global MV process](#)

The input to this process is a variable refList specifying which set of motion vectors to predict.

The output of this process is the motion vector mv representing global motion for this block.

The motion vector mv is initialized to (0, 0).

The variable ref (specifying the reference frame) is set equal to RefFrame[refList].

If ref is not equal to INTRA_FRAME and ref is not equal to TIP_FRAME, the get warp motion vector process specified in § 7.12.2.2 [Get warp motion vector process](#) is invoked with gm_params[ref], FrameMvPrecision as inputs, and the output is assigned to mv.

§ 7.12.2.2. *Get warp motion vector process*

The inputs to this process are:

- an array params containing the warp parameters,
- a variable precision specifying the precision required for the motion vector.

The output of this process is the motion vector mv of the requested precision derived from the warp parameters.

The variable bw (representing the width of the block in units of luma samples) is set equal to Block_Width[MiSize].

The variable bh (representing the height of the block in units of luma samples) is set equal to Block_Height[MiSize].

The output motion vector mv is specified by projecting the central luma sample of the block as follows:

```
x = MiCol * MI_SIZE + bw / 2 - 1
y = MiRow * MI_SIZE + bh / 2 - 1
xc = (params[ 2 ] - (1 << WARPEDMODEL_PREC_BITS)) * x +
      params[ 3 ] * y +
      params[ 0 ]
yc = params[ 4 ] * x +
      (params[ 5 ] - (1 << WARPEDMODEL_PREC_BITS)) * y +
      params[ 1 ]
if ( precision == MV_PRECISION_EIGHTH_PEL ) {
    mv[ 0 ] = Round2Signed( yc, WARPEDMODEL_PREC_BITS - 3 )
    mv[ 1 ] = Round2Signed( xc, WARPEDMODEL_PREC_BITS - 3 )
} else {
    mv[ 0 ] = Round2Signed( yc, WARPEDMODEL_PREC_BITS - 2 ) * 2
    mv[ 1 ] = Round2Signed( xc, WARPEDMODEL_PREC_BITS - 2 ) * 2
}
mv[ 0 ] = Clip3(MV_LOW + 1, MV_UPP - 1, mv[ 0 ] )
mv[ 1 ] = Clip3(MV_LOW + 1, MV_UPP - 1, mv[ 1 ] )
mv[ 0 ] = clamp_mv_row( mv[ 0 ] )
mv[ 1 ] = clamp_mv_col( mv[ 1 ] )
if ( precision < MV_PRECISION_HALF_PEL ) {
    lower_mv_precision( precision, mv )
}
```

§ 7.12.2.3. *Generate points from corners process*

The input to this process is a variable iter specifying how many times the process has been invoked for the current block.

This process creates a warp model from motion vectors found around the current block.

The arrays CornerPts, CornerMvs and the variable CornersFound are created from the blocks at three of the corners of the current block as follows:

```
bw4 = Num_4x4_Blocks_Wide[ MiSize ]
bh4 = Num_4x4_Blocks_High[ MiSize ]
CornersFound = 0
warp_corner( -1, -1, iter )
warp_corner( -1, bw4 - 1, iter )
warp_corner( bh4 - 1, -1, 0 )
```

where the call to warp_corner invokes the warp corner process specified in section [§ 7.12.2.4 Warp corner process](#).

If CornersFound is not equal to 3, this process immediately terminates.

Otherwise, the motion vectors are examined to check they are not all the same as follows:

```
allMvsSame = 1
for (n = 0; n < CornersFound; n++) {
    for(c = 0; c < 2; c++) {
        refPts[n][c] = (CornerPts[n][c] << WARPEDMODEL_PREC_BITS) +
            (CornerMvs[n][c] << GM_TRANS_ONLY_PREC_DIFF)
        if (CornerMvs[n][c] != CornerMvs[0][c]) {
            allMvsSame = 0
        }
    }
}
```

If allMvsSame is equal to 1, the process immediately terminates.

If any of the values written into refPts are negative, the process immediately terminates.

The warp model is created and inserted into the candidate list as follows:

```
widthLog2 = Mi_Width_Log2[MiSize] + MI_SIZE_LOG2
heightLog2 = Mi_Height_Log2[MiSize] + MI_SIZE_LOG2
y0 = CornerPts[0][0]
x0 = CornerPts[0][1]
wmmat = zeros[6]
wmmat[ 2 ] = (refPts[ 1 ][ 1 ] - refPts[ 0 ][ 1 ]) >> widthLog2
wmmat[ 4 ] = (refPts[ 1 ][ 0 ] - refPts[ 0 ][ 0 ]) >> widthLog2
wmmat[ 3 ] = (refPts[ 2 ][ 1 ] - refPts[ 0 ][ 1 ]) >> heightLog2
wmmat[ 5 ] = (refPts[ 2 ][ 0 ] - refPts[ 0 ][ 0 ]) >> heightLog2
wmmat0 = refPts[ 0 ][ 1 ] - wmmat[ 2 ] * x0 - wmmat[ 3 ] * y0
wmmat1 = refPts[ 0 ][ 0 ] - wmmat[ 4 ] * x0 - wmmat[ 5 ] * y0
wmmat = reduce_warp_model( wmmat )
wmmat[ 0 ] = Clip3( -WARPEDMODEL_TRANS_CLAMP,
    WARPEDMODEL_TRANS_CLAMP - (1 << WARP_PARAM_REDUCE_BITS),
    wmmat0 )
wmmat[ 1 ] = Clip3( -WARPEDMODEL_TRANS_CLAMP,
    WARPEDMODEL_TRANS_CLAMP - (1 << WARP_PARAM_REDUCE_BITS),
    wmmat1 )
```

The insert warp candidate process in [§ 7.12.2.11 Insert warp candidate process](#) is invoked with wmmat as input.

§ 7.12.2.4. Warp corner process

The inputs to this process are:

- a variable `deltaRow` specifying (in units of 4x4 luma samples) how far above the base location to look for a motion vector,
- a variable `deltaCol` specifying (in units of 4x4 luma samples) how far left of the base location to look for a motion vector,
- a variable `adjustCol` specifying an adjustment to the `deltaCol` location.

The variables `isSbBorder` (specifying if the block is on a horizontal superblock boundary), `mvRow` and `mvCol` (specifying the corner location) and `mvCol2` (specifying the location containing the motion vector), are computed as follows:

```
mvRow = MiRow + deltaRow
mvCol = MiCol + deltaCol
isSbBorder = ( MiRow & (Num_4x4_Blocks_High[ SbSize ] - 1) ) == 0
deltaCol += adjustCol
if ( deltaRow < 0 && isSbBorder ) {
    mvCol2 = (MiCol - (MiCol & 1)) + (deltaCol - (deltaCol & 1))
} else {
    mvCol2 = MiCol + deltaCol
}
```

If `isSbBorder` is equal to 1 and `deltaCol` is equal to 0 and `Num_4x4_Blocks_Wide[ MiSize ]` is less than or equal to 2, this process terminates immediately.

For `ref = 0..1`, the following applies:

- If `is_inside( mvRow, mvCol2 )` is equal to 1 and `RefFrames[ mvRow ][ mvCol2 ][ ref ]` has been written for this frame and `IsInters[ mvRow ][ mvCol2 ]` is equal to 1 and `RefFrames[ mvRow ][ mvCol2 ][ ref ]` is equal to `RefFrame[ 0 ]`, the following applies:

```
CornerPts[ CornersFound ][ 0 ] = (mvRow + 1) * MI_SIZE
CornerPts[ CornersFound ][ 1 ] = (mvCol + 1) * MI_SIZE
if ( MotionModes[ mvRow ][ mvCol2 ] >= LOCALWARP ) {
    if ( ref > 0 ) {
        return
    }
    CornerMvs[ CornersFound ] = get_warp_motion_vector_xy_pos(
        WarpParams[ mvRow ][ mvCol2 ][ ref ],
        mvRow + 1, mvCol + 1 )
} else {
    CornerMvs[ CornersFound ] = SubMvs[ mvRow ][ mvCol2 ][ ref ]
}
CornersFound++
return
```

where `get_warp_motion_vector_xy_pos` (which returns a motion vector for a given location by taking into account any warp parameters for a block) as follows:

```
get_warp_motion_vector_xy_pos(mat, posRow, posCol) {
    y = posRow * MI_SIZE
    x = posCol * MI_SIZE
    xc = (mat[2] * x + mat[3] * y + mat[0]) - (x << WARPEDMODEL_PREC_BITS)
    yc = (mat[4] * x + mat[5] * y + mat[1]) - (y << WARPEDMODEL_PREC_BITS)
    mv[0] = Round2Signed( yc, WARPEDMODEL_PREC_BITS - 3 )
}
```

```

mv[1] = Round2Signed( xc, WARPEDMODEL_PREC_BITS - 3 )
mv[0] = Clip3(MV_LOW + 1, MV_UPP - 1, mv[0] )
mv[1] = Clip3(MV_LOW + 1, MV_UPP - 1, mv[1] )
mv[0] = clamp_mv_row( mv[0] )
mv[1] = clamp_mv_col( mv[1] )
return mv
}

```

§ 7.12.2.5. Scan col process

The inputs to this process are:

- a variable deltaCol specifying (in units of 4x4 luma samples) how far left to look for motion vectors,
- a variable isCompound containing 0 for single prediction, or 1 to signal [compound prediction](#).

The variable bh4 specifying the height of the block in 4x4 luma samples is set equal to Num_4x4_Blocks_High[MiSize].

If Num_4x4_Blocks_Wide[MiSize] is equal to 1, the offset is adjusted as follows:

```
deltaCol += MiCol & 1
```

A series of motion vector locations is scanned as follows:

```

scan_point_if_valid(bh4 - 1, deltaCol, isCompound)
if (bh4 > 1) {
    scan_point_if_valid(0, deltaCol, isCompound)
}

```

where the scan_point_if_valid function is specified as:

```

scan_point_if_valid( deltaRow, deltaCol, isCompound ) {
    mvRow = MiRow + deltaRow
    mvCol = MiCol + deltaCol
    mvOtherCol = MiCol - 1
    if ( is_inside( mvRow, mvCol ) && MiColBase[ 0 ][ mvRow ][ mvCol ] !=
        MiColBase[ 0 ][ mvRow ][ mvOtherCol ] ) {
        scan_point( deltaRow, deltaCol, isCompound )
    }
}

```

where the call to scan_point invokes the process in [§ 7.12.2.6 Scan point process](#).

§ 7.12.2.6. Scan point process

The inputs to this process are:

- a variable deltaRow specifying (in units of 4x4 luma samples) how far above to look for a motion vector,
- a variable deltaCol specifying (in units of 4x4 luma samples) how far left to look for a motion vector,
- a variable isCompound containing 0 for single prediction, or 1 to signal [compound prediction](#).

The variable mvRow is set equal to MiRow + deltaRow.

The variable mvCol is set equal to MiCol + deltaCol.

The position is adjusted to an aligned location on a superblock border as follows:

```
isSbBorder = ( MiRow & (Num_4x4_Blocks_High[ SbSize ] - 1) ) == 0
if ( deltaRow < 0 && isSbBorder ) {
    mvCol = (mvCol >> 1) << 1
    deltaCol = mvCol - MiCol
}
```

The variable weight is set as follows:

- If deltaRow is equal to -1 and deltaCol is equal to -1, weight is set equal to 0.
- Otherwise, if deltaCol is less than -1, weight is set equal to 0.
- Otherwise, weight is set equal to 1.

If is_inside(mvRow, mvCol) is equal to 1 and RefFrames[mvRow][mvCol][0] has been written for this frame (this checks that the candidate location has been decoded), the following applies:

- The add warp motion vector process in [§ 7.12.2.9 Add warp motion vector process](#) is invoked with mvRow and mvCol as inputs.
- If NumMvFound is greater than or equal to MAX_REF_MV_STACK_SIZE, this process immediately terminates.
- The add reference motion vector process in [§ 7.12.2.10 Add reference motion vector process](#) is invoked with mvRow, mvCol, isCompound, weight as inputs.

§ 7.12.2.7. Temporal scan process

The input to this process is a variable isCompound containing 0 for single prediction, or 1 to signal [compound prediction](#).

This process generates motion vector candidates from the motion vectors in MotionFieldMvs.

The variable bw4 specifying the width of the block in 4x4 luma samples is set equal to Num_4x4_Blocks_Wide[MiSize].

The variable bh4 specifying the height of the block in 4x4 luma samples is set equal to Num_4x4_Blocks_High[MiSize].

The variable stepW4 is set equal to (bw4 >= 16) ? 4 : 2.

The variable stepH4 is set equal to (bh4 >= 16) ? 4 : 2.

The process scans locations within the top 64x64 luma samples of the block as follows:

```
startMvFound = NumMvFound
rowEnd = Min( bh4, 16 )
colEnd = Min( bw4, 16 )
deltaRow = rowEnd - stepH4
deltaCol = colEnd - stepW4
if ( deltaRow >= 0 && deltaCol >= 0 ) {
    add_tpl_ref_mv( deltaRow, deltaCol, isCompound )
}
if ( ( rowEnd >= 3 * stepH4 || colEnd >= 3 * stepW4 ) &&
```

```

    startMvFound == NumMvFound) {
    add_tpl_ref_mv( rowEnd >> 1, colEnd >> 1, isCompound )
}

```

where the call to `add_tpl_ref_mv` invokes the temporal sample process in [§ 7.12.2.8 Temporal sample process](#).

§ 7.12.2.8. Temporal sample process

The inputs to this process are:

- variables `deltaRow` and `deltaCol` specifying (in units of 4x4 luma samples) the offset to the candidate location,
- a variable `isCompound` containing 0 for single prediction, or 1 to signal [compound prediction](#).

This process looks up a motion vector from the motion field and adds it into the stack.

If `NumMvFound` is greater than or equal to `MAX_REF_MV_STACK_SIZE`, this process immediately terminates.

The variable `mvRow` is set equal to `MiRow + deltaRow`.

The variable `mvCol` is set equal to `MiCol + deltaCol`.

If `is_inside( mvRow, mvCol )` is equal to 0, this process terminates immediately.

The variable `x8` is set equal to `mvCol >> 1`.

The variable `y8` is set equal to `mvRow >> 1`.

(`x8` and `y8` represent the position of the candidate in units of 8x8 luma samples.)

If `MotionFieldValid[ y8 ][ x8 ]` is equal to 0, this process terminates immediately.

The process is specified as follows:

```

if ( !isCompound ) {
    candMv = get_motion_field_mv( RefFrame[ 0 ], y8, x8 )
    if ( PruneCount >= MAX_PR_NUM ) {
        idx = NumMvFound
    } else {
        for ( idx = 0; idx < NumMvFound; idx++ ) {
            PruneCount++
            if ( candMv == RefStackMv[ idx ][ 0 ] )
                break
        }
    }
    weight = Abs( get_relative_dist( OrderHint,
                                    OrderHints[ RefFrame[ 0 ] ] ) ) <= 2 ? 2 : 1
    if ( idx < NumMvFound ) {
        WeightStack[ idx ] += weight
    } else {
        RefStackMv[ NumMvFound ][ 0 ] = candMv
        WeightStack[ NumMvFound ] = weight
        NumMvFound += 1
    }
} else {
    cand0Mv = get_motion_field_mv( RefFrame[ 0 ], y8, x8 )
    cand1Mv = get_motion_field_mv( RefFrame[ 1 ], y8, x8 )
    if ( PruneCount >= MAX_PR_NUM ) {

```

```

    idx = NumMvFound
} else {
    for ( idx = 0; idx < NumMvFound; idx++ ) {
        PruneCount++
        if ( cand0Mv == RefStackMv[ idx ][ 0 ] &&
            cand1Mv == RefStackMv[ idx ][ 1 ] ) {
            break
        }
    }
}
if ( idx < NumMvFound ) {
    WeightStack[ idx ] += 1
} else {
    RefStackMv[ NumMvFound ][ 0 ] = cand0Mv
    RefStackMv[ NumMvFound ][ 1 ] = cand1Mv
    WeightStack[ NumMvFound ] = 1
    RefStackCwp[ NumMvFound ] = CWP_EQUAL
    NumMvFound += 1
}
}
}

```

where the function `get_motion_field_mv` is defined as:

```

get_motion_field_mv(dst, y8, x8) {
    if ( TrajValid[ dst ][ y8 ][ x8 ] ) {
        return TrajMv[ dst ][ y8 ][ x8 ]
    }
    mv = MotionFieldMvs[ y8 ][ x8 ]
    refOffset = MotionFieldOffset[ y8 ][ x8 ]
    refToDst = get_relative_dist( OrderHint, OrderHints[ dst ] )
    return get_mv_projection( mv, refToDst, refOffset )
}

```

§ 7.12.2.9. Add warp motion vector process

The inputs to this process are:

- variables `mvRow` and `mvCol` specifying (in units of 4x4 luma samples) the candidate location.

This process examines the candidate to find suitable locations for use with warped prediction.

If `IsInters[ mvRow ][ mvCol ]` is equal to 1 and `DeriveWrl` is equal to 1 and `MotionModes[ mvRow ][ mvCol ]` is greater than or equal to `LOCALWARP` and `RefFrames[ mvRow ][ mvCol ][ 0 ]` is equal to `RefFrame[ 0 ]`, the insert warp candidate process in [§ 7.12.2.11 Insert warp candidate process](#) is invoked with `WarpParams[ mvRow ][ mvCol ][ 0 ]` as input.

§ 7.12.2.10. Add reference motion vector process

The inputs to this process are:

- variables `mvRow` and `mvCol` specifying (in units of 4x4 luma samples) the candidate location,
- a variable `isCompound` containing 0 for single prediction, or 1 to signal [compound prediction](#),
- a variable `weight` specifying the weight attached to this motion vector.

This process examines the candidate to find matching reference frames.

If `IsInters[ mvRow ][ mvCol ]` is equal to 0, this process terminates immediately.

If isCompound is equal to 0, the following applies for candList = 0..(1 - use_intrabc):

- If RefFrames[mvRow][mvCol][candList] is equal to RefFrame[0], the search stack process in [§ 7.12.2.12 Search stack process](#) is invoked with mvRow, mvCol, weight, and candList as inputs.
- Otherwise, if RefFrames[mvRow][mvCol][0] is equal to TIP_FRAME and RefFrame[0] is equal to (candList ? ClosestFuture : ClosestPast), the derive single ref mv candidate from TIP mode process specified in [§ 7.12.2.17 Derive single ref mv candidate from TIP mode process](#) is invoked with mvRow, mvCol, weight, and candList as inputs.
- Otherwise, if candList is equal to 0 and RefFrame[0] is equal to TIP_FRAME and ClosestPast is equal to RefFrames[mvRow][mvCol][0] and ClosestFuture is equal to RefFrames[mvRow][mvCol][1], the TIP add derived process specified in [§ 7.12.2.18 TIP add derived process](#) is invoked with mvRow and mvCol as inputs.
- Otherwise, if use_intrabc is equal to 0 and is_derivable_ref_frame(RefFrames[mvRow][mvCol], candList) is equal to 1 and RefFrame[0] is not equal to TIP_FRAME, the single add derived process specified in [§ 7.12.2.16 Single add derived process](#) is invoked with mvRow, mvCol, and candList as inputs.

Otherwise (isCompound is equal to 1), the following applies:

```
if ( RefFrames[ mvRow ][ mvCol ][ 0 ] == TIP_FRAME &&
    RefFrame[ 0 ] == ClosestPast && RefFrame[ 1 ] == ClosestFuture ) {
    derive_ref_mv_candidate_from_tip_mode( mvRow, mvCol, weight )
} else if ( RefFrames[ mvRow ][ mvCol ][ 0 ] == RefFrame[ 0 ] &&
    RefFrames[ mvRow ][ mvCol ][ 1 ] == RefFrame[ 1 ] ) {
    compound_search_stack( mvRow, mvCol, weight )
} else {
    compound_add_derived( mvRow, mvCol )
}
```

The function call of compound_search_stack indicates that the compound search stack process in [§ 7.12.2.13 Compound search stack process](#) is invoked with mvRow, mvCol, and weight as inputs.

The function call of compound_add_derived indicates that the compound add derived process in [§ 7.12.2.14 Compound add derived process](#) is invoked with mvRow and mvCol as inputs.

The function call of derive_ref_mv_candidate_from_tip_mode indicates that the derive ref mv candidate from tip mode process in [§ 7.12.2.15 Derive ref mv candidate from tip mode process](#) is invoked with mvRow, mvCol, and weight as inputs.

The function is_derivable_ref_frame is specified as:

```
is_derivable_ref_frame( candRefFrames, candList ) {
    return candRefFrames[ 0 ] == TIP_FRAME ||
        is_inter_ref_frame( candRefFrames[ candList ] )
}
```

§ 7.12.2.11. Insert warp candidate process

The input to this process is an array params specifying the candidate parameters.

If NumWarpFound is greater than or equal to MAX_WARP_REF_CANDIDATES, this process immediately terminates.

Otherwise, the parameters are saved into the warp parameter stack as follows:

```
for( i = 0; i < 6; i++ ) {  
    WarpParamStack[ NumWarpFound ][ i ] = params[ i ]  
}  
NumWarpFound++
```

§ 7.12.2.12. Search stack process

The inputs to this process are:

- variables mvRow and mvCol specifying (in units of 4x4 luma samples) the candidate location,
- a variable weight,
- a variable candList specifying which list in the candidate matches our reference frame.

This process searches the stack for an exact match with a candidate motion vector. If present, the weight of the candidate motion vector is added to the weight of its counterpart in the stack, otherwise the process adds a motion vector to the stack.

The motion vector candMv is set equal to `get_mv( mvRow, mvCol, 0, candList )`.

The process depends on whether the candidate motion vector is already in the stack as follows:

```
candMvFound = 0  
if ( PruneCount < MAX_PR_NUM ) {  
    for ( idx = 0; idx < NumMvFound; idx++ ) {  
        PruneCount++  
        if ( candMv == RefStackMv[ idx ][ 0 ] ) {  
            WeightStack[ idx ] += weight  
            candMvFound = 1  
            break  
        }  
    }  
}  
if ( !candMvFound && NumMvFound < MAX_REF_MV_STACK_SIZE ) {  
    RefStackMv[ NumMvFound ][ 0 ][ 0 ] = candMv[ 0 ]  
    RefStackMv[ NumMvFound ][ 0 ][ 1 ] = candMv[ 1 ]  
    RefStackRowOffset[ NumMvFound ] = mvRow - MiRow  
    RefStackColOffset[ NumMvFound ] = mvCol - MiCol  
    WeightStack[ NumMvFound ] = weight  
    NumMvFound++  
}
```

§ 7.12.2.13. Compound search stack process

The inputs to this process are:

- variables mvRow and mvCol specifying (in units of 4x4 luma samples) the candidate location,
- a variable weight.

This process searches the stack for an exact match with a candidate pair of motion vectors. If present, the weight of the candidate pair of motion vectors is added to the weight of its counterpart in the stack, otherwise the process adds the motion vectors to the stack.

The array candMvs (containing two motion vectors) is set equal to `SubMvs[ mvRow ][ mvCol ]`.

The variable candCwp is set equal to CwpIdxs[mvRow][mvCol].

The variable candMode is set equal to YModes[mvRow][mvCol].

The variable candSize is set equal to MiSizes[PlaneStart][mvRow][mvCol].

The variable large is set as follows:

- If Min(Block_Width[candSize], Block_Height[candSize]) is greater than or equal to 8, large is set equal to 1.
- Otherwise, large is set equal to 0.

If large is equal to 1 and candMode is equal to GLOBAL_GLOBALMV, for refList = 0..1 the following applies:

- If GmType[RefFrame[refList]] is greater than IDENTITY, candMvs[refList] is set equal to GlobalMvs[refList].

The process depends on whether the candidate motion vector pair is already in the stack as follows:

```
candMvFound = 0
if ( PruneCount < MAX_PR_NUM ) {
    for ( idx = 0; idx < NumMvFound; idx++ ) {
        PruneCount++
        if ( candMvs[ 0 ][ 0 ] == RefStackMv[ idx ][ 0 ][ 0 ] &&
            candMvs[ 0 ][ 1 ] == RefStackMv[ idx ][ 0 ][ 1 ] &&
            candMvs[ 1 ][ 0 ] == RefStackMv[ idx ][ 1 ][ 0 ] &&
            candMvs[ 1 ][ 1 ] == RefStackMv[ idx ][ 1 ][ 1 ] ) {
            WeightStack[ idx ] += weight
            candMvFound = 1
            break
        }
    }
}
if ( !candMvFound && NumMvFound < MAX_REF_MV_STACK_SIZE ) {
    for ( i = 0; i < 2; i++ ) {
        RefStackMv[ NumMvFound ][ i ][ 0 ] = candMvs[ i ][ 0 ]
        RefStackMv[ NumMvFound ][ i ][ 1 ] = candMvs[ i ][ 1 ]
    }
    RefStackCwp[ NumMvFound ] = candCwp
    WeightStack[ NumMvFound ] = weight
    RefStackRowOffset[ NumMvFound ] = mvRow - MiRow
    RefStackColOffset[ NumMvFound ] = mvCol - MiCol
    NumMvFound++
}
```

NOTE: NumMvFound will always be less than MAX_REF_MV_STACK_SIZE when this process is called.

§ 7.12.2.14. Compound add derived process

The inputs to this process are:

- variables mvRow and mvCol specifying (in units of 4x4 luma samples) the candidate location.

This process conditionally adds a candidate to the derived motion vector stack and the single motion vector stack as follows:

```

if ( enable_mv_traj && use_ref_frame_mvs && RefFrame[ 0 ] != RefFrame[ 1 ] ) {
    for (list = 0; list < 2; list++) {
        candRef = RefFrames[ mvRow ][ mvCol ][ list ]
        if ( is_inter_ref_frame( candRef ) && candRef != TIP_FRAME ) {
            candMv = get_mv(mvRow, mvCol, -1, list)
            trajY8 = MiRow >> 1
            trajX8 = MiCol >> 1
            trajCandValid = TrajValid[ candRef ][ trajY8 ][ trajX8 ]
            trajRef0Valid = TrajValid[ RefFrame[ 0 ] ][ trajY8 ][ trajX8 ]
            trajRef1Valid = TrajValid[ RefFrame[ 1 ] ][ trajY8 ][ trajX8 ]
            if ( trajCandValid && trajRef0Valid && trajRef1Valid ) {
                trajCandMv = TrajMv[ candRef ][ trajY8 ][ trajX8 ]
                trajRef0 = TrajMv[ RefFrame[ 0 ] ][ trajY8 ][ trajX8 ]
                trajRef1 = TrajMv[ RefFrame[ 1 ] ][ trajY8 ][ trajX8 ]
                for( c = 0; c < 2; c++ ) {
                    candMvs[ 0 ][ c ] = Clip3( MV_LOW + 1, MV_UPP - 1,
                                                candMv[ c ] + trajRef0[ c ] -
                                                trajCandMv[ c ] )
                    candMvs[ 1 ][ c ] = Clip3( MV_LOW + 1, MV_UPP - 1,
                                                candMv[ c ] + trajRef1[ c ] -
                                                trajCandMv[ c ] )
                }
                if ( DerivedMvCount < MAX_DR_STACK_SIZE &&
                    !comp_mv_in_stack( DerivedStackMv, DerivedMvCount,
                                       candMvs[ 0 ], candMvs[ 1 ] ) ) {
                    DerivedStackMv[DerivedMvCount][0][0] = candMvs[0][0]
                    DerivedStackMv[DerivedMvCount][0][1] = candMvs[0][1]
                    DerivedStackMv[DerivedMvCount][1][0] = candMvs[1][0]
                    DerivedStackMv[DerivedMvCount][1][1] = candMvs[1][1]
                    DerivedMvCount++
                }
            }
        }
    }
}

if (RefFrames[ mvRow ][ mvCol ][ 0 ] == RefFrame[ 0 ] ||
    RefFrames[ mvRow ][ mvCol ][ 1 ] == RefFrame[ 0 ]) {
    candRefIdx0 = 0
    candRefIdx1 = 1
} else if (RefFrames[ mvRow ][ mvCol ][ 0 ] == RefFrame[ 1 ] ||
           RefFrames[ mvRow ][ mvCol ][ 1 ] == RefFrame[ 1 ]) {
    candRefIdx0 = 1
    candRefIdx1 = 0
} else {
    return
}

candList = RefFrames[ mvRow ][ mvCol ][ 0 ] == RefFrame[ candRefIdx0 ] ? 0 : 1
candMv = get_mv(mvRow, mvCol, candRefIdx0, candList)
for( candIdx = 0; candIdx < SingleMvCount &&
    DerivedMvCount < MAX_DR_STACK_SIZE; candIdx++ ) {
    if (SingleRefFrame[candIdx] == RefFrame[candRefIdx1]) {
        l0Mv = candRefIdx0 == 0 ? candMv : SingleMv[candIdx]
        l1Mv = candRefIdx0 == 1 ? candMv : SingleMv[candIdx]
        if (!comp_mv_in_stack(DerivedStackMv, DerivedMvCount, l0Mv, l1Mv)) {
            DerivedStackMv[DerivedMvCount][0][0] = l0Mv[0]
            DerivedStackMv[DerivedMvCount][0][1] = l0Mv[1]
            DerivedStackMv[DerivedMvCount][1][0] = l1Mv[0]
            DerivedStackMv[DerivedMvCount][1][1] = l1Mv[1]
            DerivedMvCount++
        }
        break
    }
}

if ( SinglePruneCount < MAX_DR_PR_NUM ) {
    for( candIdx = 0; candIdx < SingleMvCount; candIdx++ ) {
        SinglePruneCount++
    }
}

```

```

        if (SingleRefFrame[candIdx] == RefFrame[candRefIdx0] &&
            SingleMv[candIdx][0] == candMv[0] &&
            SingleMv[candIdx][1] == candMv[1]) {
            return
        }
    }
}
if ( SingleMvCount < MAX_DR_STACK_SIZE ) {
    SingleRefFrame[SingleMvCount] = RefFrame[candRefIdx0]
    SingleMv[SingleMvCount][0] = candMv[0]
    SingleMv[SingleMvCount][1] = candMv[1]
    SingleMvCount++
}

```

The function `get_mv` (which gets a motion vector for a location) is specified as:

```

get_mv(mvRow, mvCol, refList, candList) {
    candMode = YModes[ mvRow ][ mvCol ]
    candSize = MiSizes[ PlaneStart ][ mvRow ][ mvCol ]
    candRefFrame = RefFrames[ mvRow ][ mvCol ][ candList ]
    large = ( Min( Block_Width[ candSize ], Block_Height[ candSize ] ) >= 8 )
    if ( refList >= 0 &&
        ( candMode == GLOBALMV || candMode == GLOBAL_GLOBALMV ) &&
        candRefFrame != TIP_FRAME &&
        GmType[ candRefFrame ] > IDENTITY &&
        large ) {
        return GlobalMvs[ refList ]
    } else {
        return SubMvs[ mvRow ][ mvCol ][ candList ]
    }
}

```

The function `comp_mv_in_stack` (which determines if the motion vector pair is already in the stack) is specified as:

```

comp_mv_in_stack(mvStack, count, list0Mv, list1Mv) {
    if ( DerivedPruneCount < MAX_DR_PR_NUM ) {
        for ( i = 0; i < count; i++ ) {
            DerivedPruneCount++
            if ( mvStack[i][0] == list0Mv &&
                mvStack[i][1] == list1Mv ) {
                return 1
            }
        }
    }
    return 0
}

```

[§](#) **7.12.2.15.** *Derive ref mv candidate from tip mode process*

The inputs to this process are:

- variables `candRow` and `candCol` specifying (in units of 4x4 luma samples) the candidate location,
- a variable `weight` specifying the weight attached to this motion vector.

The candidate is added to the stack of motion vectors as follows:

```

candMvs = get_tip_cand(candRow, candCol)
candMvFound = 0
if ( PruneCount < MAX_PR_NUM ) {
    for ( idx = 0; idx < NumMvFound; idx++ ) {

```

```

        PruneCount++
        match = candMvs[ 0 ] == RefStackMv[ idx ][ 0 ] &&
                candMvs[ 1 ] == RefStackMv[ idx ][ 1 ]
        if (match) {
            WeightStack[ idx ] += weight
            candMvFound = 1
            break
        }
    }
}
if (!candMvFound && NumMvFound < MAX_REF_MV_STACK_SIZE) {
    for (i = 0; i < 2; i++) {
        RefStackMv[ NumMvFound ][ i ] = candMvs[ i ]
    }
    RefStackCwp[ NumMvFound ] = CWP_EQUAL
    WeightStack[ NumMvFound ] = weight
    NumMvFound++
}

```

NOTE: NumMvFound will always be less than MAX_REF_MV_STACK_SIZE when this process is called.

§ 7.12.2.16. Single add derived process

The inputs to this process are:

- variables mvRow and mvCol specifying (in units of 4x4 luma samples) the candidate location,
- a variable candList specifying which slot of the reference list to examine.

The process conditionally adds a candidate to DerivedStackMv as follows:

```

if ( RefFrames[mvRow][mvCol][0] == TIP_FRAME ) {
    candMvs = get_tip_cand(mvRow,mvCol)
    candMv = candMvs[ candList ]
    candRef = candList ? ClosestFuture : ClosestPast
} else {
    candMv = get_mv(mvRow, mvCol, -1, candList)
    candRef = RefFrames[mvRow][mvCol][candList]
}
curDist = FrameDistance[ RefFrame[ 0 ] ]
candDist = FrameDistance[ candRef ]
haveProj = 0
if ( use_ref_frame_mvs && enable_mv_traj ) {
    trajY8 = MiRow >> 1
    trajX8 = MiCol >> 1
    trajCurValid = TrajValid[ RefFrame[ 0 ] ][ trajY8 ][ trajX8 ]
    trajCandValid = TrajValid[ candRef ][ trajY8 ][ trajX8 ]
    if ( trajCurValid && trajCandValid ) {
        trajCurMv = TrajMv[ RefFrame[ 0 ] ][ trajY8 ][ trajX8 ]
        trajCandMv = TrajMv[ candRef ][ trajY8 ][ trajX8 ]
        haveProj = 1
        for( c = 0; c < 2; c++ ) {
            projCandMv[ c ] = Clip3( MV_LOW + 1, MV_UPP - 1,
                                   candMv[c] + trajCurMv[c] - trajCandMv[c] )
        }
    }
}
if (!haveProj && ( (curDist > 0 && candDist > 0) ||
                  (curDist < 0 && candDist < 0) )) {
    projCandMv = get_mv_projection( candMv, Abs( curDist ), Abs( candDist ) )
    haveProj = 1
}
if (haveProj) {
    if ( DerivedPruneCount < MAX_DR_PR_NUM ) {

```

```

    for ( i = 0; i < DerivedMvCount; i++ ) {
        DerivedPruneCount++
        if ( DerivedStackMv[i][0] == projCandMv ) {
            return
        }
    }
}
if ( DerivedMvCount < MAX_DR_STACK_SIZE ) {
    DerivedStackMv[DerivedMvCount][0] = projCandMv
    DerivedMvCount++
}
}

```

§ 7.12.2.17. Derive single ref mv candidate from TIP mode process

The inputs to this process are:

- variables candRow and candCol specifying (in units of 4x4 luma samples) the candidate location,
- a variable weight specifying the weight attached to this motion vector,
- a variable candList specifying which slot of the reference list to examine.

The process conditionally adds a candidate to DerivedStackMv as follows:

```

candMvs = get_tip_cand(candRow, candCol)
candMvFound = 0
if ( PruneCount < MAX_PR_NUM ) {
    for ( idx = 0; idx < NumMvFound; idx++ ) {
        PruneCount++
        match = candMvs[ candList ][ 0 ] == RefStackMv[ idx ][ 0 ][ 0 ] &&
                candMvs[ candList ][ 1 ] == RefStackMv[ idx ][ 0 ][ 1 ]
        if ( match ) {
            WeightStack[ idx ] += weight
            candMvFound = 1
            break
        }
    }
}
if ( !candMvFound ) {
    if ( NumMvFound < MAX_REF_MV_STACK_SIZE ) {
        RefStackMv[ NumMvFound ][ 0 ][ 0 ] = candMvs[ candList ][ 0 ]
        RefStackMv[ NumMvFound ][ 0 ][ 1 ] = candMvs[ candList ][ 1 ]
        WeightStack[ NumMvFound ] = weight
        NumMvFound++
    }
}

```

§ 7.12.2.18. TIP add derived process

The inputs to this process are:

- variables mvRow and mvCol specifying (in units of 4x4 luma samples) the candidate location.

The process conditionally adds a candidate to DerivedStackMv as follows:

```

linearMv[0] = SubMvs[mvRow][mvCol][0][0] - SubMvs[mvRow][mvCol][1][0]
linearMv[1] = SubMvs[mvRow][mvCol][0][1] - SubMvs[mvRow][mvCol][1][1]
(refOffset, pastOffset, futureOffset) = get_tip_offsets()
projMv = get_mv_projection( linearMv, pastOffset, refOffset )
derivedMv[0] = Clip3( MV_LOW + 1, MV_UPP - 1,
                    SubMvs[mvRow][mvCol][0][0] - projMv[0] )
derivedMv[1] = Clip3( MV_LOW + 1, MV_UPP - 1,

```

```

        SubMvs[mvRow][mvCol][0][1] - projMv[1])
if ( DerivedPruneCount < MAX_DR_PR_NUM ) {
    for ( i = 0; i < DerivedMvCount; i++ ) {
        DerivedPruneCount++
        if ( DerivedStackMv[i][0][0] == derivedMv[0] &&
            DerivedStackMv[i][0][1] == derivedMv[1] ) {
            return
        }
    }
}
if ( DerivedMvCount < MAX_DR_STACK_SIZE ) {
    DerivedStackMv[DerivedMvCount][0][0] = derivedMv[0]
    DerivedStackMv[DerivedMvCount][0][1] = derivedMv[1]
    DerivedMvCount++
}

```

§ 7.12.2.19. Sorting process

The inputs to this process are:

- a variable start representing the first position to be sorted,
- a variable end representing the length of the array,
- a variable isCompound containing 0 for single prediction, or 1 to signal [compound prediction](#).

This process moves the highest weight entry in the stack to the start.

The process is specified as:

```

maxWeight = WeightStack[start]
maxWeightIdx = start
for ( idx = start + 1; idx < end; idx++ ) {
    if ( maxWeight < WeightStack[ idx ] ) {
        maxWeight = WeightStack[ idx ]
        maxWeightIdx = idx
    }
}
if ( maxWeightIdx != start ) {
    swap_stack( start, maxWeightIdx )
}

```

When the function swap_stack is invoked, the entries at locations i and j are swapped in WeightStack and RefStackMv as follows:

```

swap_stack( i, j ) {
    temp = WeightStack[ i ]
    WeightStack[ i ] = WeightStack[ j ]
    WeightStack[ j ] = temp
    temp = RefStackCwp[ i ]
    RefStackCwp[ i ] = RefStackCwp[ j ]
    RefStackCwp[ j ] = temp
    temp = RefStackRowOffset[ i ]
    RefStackRowOffset[ i ] = RefStackRowOffset[ j ]
    RefStackRowOffset[ j ] = temp
    temp = RefStackColOffset[ i ]
    RefStackColOffset[ i ] = RefStackColOffset[ j ]
    RefStackColOffset[ j ] = temp
    for ( list = 0; list < 1 + isCompound; list++ ) {
        for ( comp = 0; comp < 2; comp++ ) {
            temp = RefStackMv[ i ][ list ][ comp ]
            RefStackMv[ i ][ list ][ comp ] = RefStackMv[ j ][ list ][ comp ]
            RefStackMv[ j ][ list ][ comp ] = temp
        }
    }
}

```

```

    }
  }
}

```

§ 7.12.2.20. Extra search process

The input to this process is a variable `isCompound` containing 0 for single prediction, or 1 to signal [compound prediction](#).

This process clamps the stack and adds additional motion vectors to `RefStackMv`.

The candidates on the stack are clamped as follows:

```

for ( idx = 0; idx < NumMvFound ; idx++ ) {
  for ( list = 0; list < (isCompound ? 2 : 1); list++ ) {
    refMv[ 0 ] = RefStackMv[ idx ][ list ][ 0 ]
    refMv[ 1 ] = RefStackMv[ idx ][ list ][ 1 ]
    refMv[ 0 ] = clamp_mv_row( refMv[ 0 ] )
    refMv[ 1 ] = clamp_mv_col( refMv[ 1 ] )
    RefStackMv[ idx ][ list ][ 0 ] = refMv[ 0 ]
    RefStackMv[ idx ][ list ][ 1 ] = refMv[ 1 ]
  }
}

```

A global mv candidate is added if not already present as follows:

```

if ( NumMvFound < MAX_REF_MV_STACK_SIZE && !use_intrabc ) {
  found = 0
  if ( PruneCount < MAX_PR_NUM ) {
    for ( idx = 0; idx < NumMvFound; idx++ ) {
      PruneCount++
      if ( GlobalMvs[ 0 ] == RefStackMv[ idx ][ 0 ] ) {
        if ( !isCompound ||
              (GlobalMvs[ 1 ] == RefStackMv[ idx ][ 1 ] ) ) {
          found = 1
          break
        }
      }
    }
  }
  if (!found) {
    for ( list = 0; list < (isCompound ? 2 : 1); list++ ) {
      RefStackMv[ NumMvFound ][ list ][ 0 ] = GlobalMvs[ list ][ 0 ]
      RefStackMv[ NumMvFound ][ list ][ 1 ] = GlobalMvs[ list ][ 1 ]
    }
    RefStackCwp[ NumMvFound ] = CWP_EQUAL
    NumMvFound++
  }
}

```

If `Block_Width[ MiSize ]` is greater than 32 and `Block_Height[ MiSize ]` is greater than 32, extra candidates are added as follows:

```

num = NumMvFound
if ( num > 1 ) {
  insert_mvp_candidate( isCompound, 0, 1 )
  insert_mvp_candidate( isCompound, 1, 0 )
}
if ( num > 2 ) {
  insert_mvp_candidate( isCompound, 0, 2 )
  insert_mvp_candidate( isCompound, 2, 0 )
}

```



```

insert_mv_candidate( isCompound, 1, 2 )
insert_mv_candidate( isCompound, 2, 1 )
}

```

where insert_mv_candidate (which adds a candidate with a mixture of existing motion vectors) is specified as follows:

```

insert_mv_candidate( isCompound, yCand, xCand ) {
    candMvs[ 0 ][ 0 ] = RefStackMv[ yCand ][ 0 ][ 0 ]
    candMvs[ 0 ][ 1 ] = RefStackMv[ xCand ][ 0 ][ 1 ]
    candMvs[ 1 ][ 0 ] = RefStackMv[ yCand ][ 1 ][ 0 ]
    candMvs[ 1 ][ 1 ] = RefStackMv[ xCand ][ 1 ][ 1 ]
    if ( NumMvFound < MAX_REF_MV_STACK_SIZE ) {
        if ( PruneCount < MAX_PR_NUM ) {
            for ( idx = 0; idx < NumMvFound; idx++ ) {
                PruneCount++
                match = candMvs[ 0 ][ 0 ] == RefStackMv[ idx ][ 0 ][ 0 ] &&
                        candMvs[ 0 ][ 1 ] == RefStackMv[ idx ][ 0 ][ 1 ]
                if ( !isCompound && match )
                    return
                if ( isCompound && match &&
                    candMvs[ 1 ][ 0 ] == RefStackMv[ idx ][ 1 ][ 0 ] &&
                    candMvs[ 1 ][ 1 ] == RefStackMv[ idx ][ 1 ][ 1 ] )
                    return
            }
            RefStackMv[ NumMvFound ][ 0 ][ 0 ] = candMvs[ 0 ][ 0 ]
            RefStackMv[ NumMvFound ][ 0 ][ 1 ] = candMvs[ 0 ][ 1 ]
            RefStackMv[ NumMvFound ][ 1 ][ 0 ] = candMvs[ 1 ][ 0 ]
            RefStackMv[ NumMvFound ][ 1 ][ 1 ] = candMvs[ 1 ][ 1 ]
            NumMvFound++
        }
    }
}

```

If DeriveWrl is equal to 1, additional warp candidates are added as follows:

```

ref = RefFrame[ 0 ]
c = WarpBankSize[ ref ]
s = WarpBankStart[ ref ]
for( i = c - 1; i >= 0; i-- ) {
    idx = (s + i) % WARP_PARAM_BANK_SIZE
    insert_warp_candidate( WarpBankParams[ref][idx] )
}
insert_warp_candidate( gm_params[ref] )
for( i = 0; i < 2; i++ ) {
    insert_warp_candidate( Default_Warp_Params )
}

```

Where setup_shear invokes the setup shear process specified in [§7.13.3.21 Setup shear process](#), and insert_warp_candidate invokes the insert warp candidate process in [§7.12.2.11 Insert warp candidate process](#).

The table Default_Warp_Params is defined as:

```

Default_Warp_Params[6] = {
    0, 0, 1 << WARPEDMODEL_PREC_BITS, 0, 0, 1 << WARPEDMODEL_PREC_BITS
}

```

If use_intrabc is equal to 1, additional intra block copy candidates are added as follows:

```
add_to_ref_bv(0, -Block_Height[ SbSize ])
add_to_ref_bv(-Block_Width[ SbSize ] - INTRABC_DELAY_PIXELS, 0)
add_to_ref_bv(0, -Block_Height[ MiSize ])
add_to_ref_bv(-Block_Width[ MiSize ], 0)
```

where the function add_to_ref_bv is specified as:

```
add_to_ref_bv(dx, dy) {
    if ( NumMvFound < max_bvp_drl_bits_minus_1 + 2 ) {
        RefStackMv[ NumMvFound ][ 0 ][ 0 ] = dy << 3
        RefStackMv[ NumMvFound ][ 0 ][ 1 ] = dx << 3
        NumMvFound++
    }
}
```

§ 7.12.2.21. Fill mvp from ref mv bank process

The input to this process is a variable isCompound containing 0 for single prediction, or 1 to signal compound prediction.

This process adds additional motion vectors to RefStackMv from the bank of motion vectors.

The candidates are added as follows:

```
ref = get_rmb_list_index( RefFrame )
key = isCompound ? RefFrame[0] + (RefFrame[1] + 1) * BANK_REFS_PER_FRAME :
      RefFrame[0]
if ( use_intrabc ) {
    maxRefMvCount = max_bvp_drl_bits_minus_1 + 2
} else {
    maxRefMvCount = max_drl_bits_minus_1 + 2
}
c = RefMvBankSize[ ref ]
s = RefMvBankStart[ ref ]
for( i = c - 1; i >= 0 && NumMvFound < maxRefMvCount; i-- ) {
    idx = (s + i) % REF_MV_BANK_SIZE
    if ( RefMvBankParams[ ref ][ idx ][ 1 ] == key ) {
        for(list=0; list<2; list++) {
            for(comp=0; comp<2; comp++) {
                candMvs[list][comp] =
                    RefMvBankParams[ ref ][ idx ][ 2 + list * 2 + comp ]
            }
        }
        check_rmb_cand(candMvs, isCompound, RefMvBankParams[ ref ][ idx ][ 0 ])
    }
}
```

where the function check_rmb_cand (which checks if the motion vector is new and points inside the frame) is defined as:

```
check_rmb_cand(candMvs, isCompound, cwp) {
    bw = Block_Width[ MiSize ]
    bh = Block_Height[ MiSize ]
    if ( PruneCount < MAX_PR_NUM ) {
        for ( idx = 0; idx < NumMvFound; idx++ ) {
            PruneCount++
            if ( candMvs[ 0 ] == RefStackMv[ idx ][ 0 ] &&
                (!isCompound || candMvs[ 1 ] == RefStackMv[ idx ][ 1 ]) )
```

```

    ) {
        return
    }
}
}
for (i = 0; i < 1 + isCompound; i++) {
    refY = MiRow * MI_SIZE + (candMvs[i][0] / 8)
    refX = MiCol * MI_SIZE + (candMvs[i][1] / 8)
    if ( refX <= -bw || refY <= -bh ||
        refX >= MiCols * MI_SIZE ||
        refY >= MiRows * MI_SIZE ) {
        return
    }
}
for (i = 0; i < 1 + isCompound; i++) {
    RefStackMv[ NumMvFound ][ i ][ 0 ] = candMvs[ i ][ 0 ]
    RefStackMv[ NumMvFound ][ i ][ 1 ] = candMvs[ i ][ 1 ]
    RefStackCwp[ NumMvFound ] = cwp
}
NumMvFound++
}

```

and the function `get_rmb_list_index` which returns the bank to use for the current choice of reference frames is defined as:

```

get_rmb_list_index( refFrames ) {
    if ( !is_inter_ref_frame(refFrames[ 1 ]) && refFrames[ 0 ] <= 5 ) {
        return refFrames[ 0 ]
    } else if ( refFrames[ 0 ] == 0 && refFrames[ 1 ] == 0 ) {
        return 6
    } else if ( refFrames[ 0 ] == 0 && refFrames[ 1 ] == 1 ) {
        return 7
    } else {
        return 8
    }
}

```

§ 7.12.2.22. Fill mvp from derived smvp process

The input to this process is a variable `isCompound` containing 0 for single prediction, or 1 to signal compound prediction.

This process adds additional derived motion vectors to `RefStackMv`.

The candidates are added as follows:

```

if ( use_intrabc ) {
    maxRefMvCount = max_bvp_drl_bits_minus_1 + 2
} else {
    maxRefMvCount = max_drl_bits_minus_1 + 2
}
if ( NumMvFound >= maxRefMvCount ) {
    return
}
for( derivedIdx = 0; derivedIdx < DerivedMvCount; derivedIdx++) {
    found = 0
    if ( PruneCount < MAX_PR_NUM ) {
        for (idx = 0; idx < NumMvFound; idx++) {
            PruneCount++
            if ( stack_match(idx,derivedIdx,isCompound) ) {
                found = 1
                break
            }
        }
    }
}

```

```

    }
    if ( !found && NumMvFound < maxRefMvCount ) {
        for ( i = 0; i < 1 + isCompound; i++ ) {
            for ( comp = 0; comp < 2; comp++ ) {
                RefStackMv[ NumMvFound ][ i ][ comp ] =
                    DerivedStackMv[derivedIdx][ i ][ comp ]
            }
        }
        RefStackCwp[ NumMvFound ] = CWP_EQUAL
        NumMvFound++
    }
}

```

where `stack_match` (which returns true if a derived motion vector matches motion vectors already in `RefStackMv`) is specified as:

```

stack_match(idx,derivedIdx,isCompound) {
    for( lst = 0; lst <= isCompound; lst++ ) {
        for( comp = 0; comp < 2; comp++ ) {
            if ( DerivedStackMv[ derivedIdx ][ lst ][ comp ] !=
                RefStackMv[ idx ][ lst ][ comp ] ) {
                return 0
            }
        }
    }
    return 1
}

```

§ 7.12.2.23. Clamping process

The input to this process is a variable `isCompound` containing 0 for single prediction, or 1 to signal compound prediction.

This process clamps the candidates in `RefStackMv`.

The variable `numLists` specifying the number of reference frames used for this block is set equal to (`isCompound ? 2 : 1`).

If `use_intrabc` is equal to 0, the motion vectors are clamped as follows:

```

for ( list = 0; list < numLists; list++ ) {
    for ( idx = 0; idx < NumMvFound ; idx++ ) {
        refMv = RefStackMv[ idx ][ list ]
        refMv[ 0 ] = clamp_mv_row( refMv[ 0 ] )
        refMv[ 1 ] = clamp_mv_col( refMv[ 1 ] )
        RefStackMv[ idx ][ list ] = refMv
    }
}

```

§ 7.12.3. Find warp samples process

§ 7.12.3.1. General

The input to this process is a variable `ref` specifying which set of candidate motion vectors to prepare.

The process examines the neighboring inter predicted blocks and estimates a local warp transformation based on the motion vectors.

The process produces a variable NumSamples containing the number of valid candidates found, and an array CandList containing sorted candidates.

The variable NumSamples[ref] is set equal to 0.

The variable w4 specifying the width of the block in 4x4 luma samples is set equal to Num_4x4_Blocks_Wide[MiSize].

The variable h4 specifying the height of the block in 4x4 luma samples is set equal to Num_4x4_Blocks_High[MiSize].

The process is specified as:

```
doTopLeft = 1
doTopRight = 1
if (AvailU) {
    colOffset = MiColBase[ 0 ][ MiRow - 1 ][ MiCol ] - MiCol
    if (colOffset < 0)
        doTopLeft = 0
    for (i = colOffset; i < Min( w4, MiCols - MiCol ); i += srcW) {
        srcSize = MiSizes[ 0 ][ MiRow - 1 ][ MiCol + i ]
        srcW = Num_4x4_Blocks_Wide[ srcSize ]
        if ( above_sample_stored( i ) ) {
            add_sample( ref, -1, i )
        }
    }
    doTopRight = (i == w4) && i < (MiCols - MiCol)
}
if (AvailL) {
    rowOffset = MiRowBase[ 0 ][ MiRow ][ MiCol - 1 ] - MiRow
    if (rowOffset < 0)
        doTopLeft = 0
    for (i = rowOffset; i < Min( h4, MiRows - MiRow ); i += srcH) {
        srcSize = MiSizes[ 0 ][ MiRow + i ][ MiCol - 1 ]
        srcH = Num_4x4_Blocks_High[ srcSize ]
        add_sample( ref, i, -1 )
    }
}
if ( doTopLeft && above_sample_stored( -1 ) ) {
    add_sample( ref, -1, -1 )
}
if ( doTopRight && w4 <= 16 && above_sample_stored( w4 ) ) {
    add_sample( ref, -1, w4 )
}
```

where the call to add_sample specifies that the add sample process in [§7.12.3.2 Add sample process](#) is invoked.

The function above_sample_stored (which checks whether the warp parameters for a particular above location are available) is specified as follows:

```
above_sample_stored( deltaCol ) {
    if ( !is_inside( MiRow - 1, MiCol + deltaCol ) ) {
        return 0
    }
    isSbBorder = ( MiRow & (Num_4x4_Blocks_High[ SbSize ] - 1) ) == 0
    if (!isSbBorder) {
        return 1
    }
    if ((MiCol + deltaCol) % 2 == 0) {
        return 1
    }
}
```

```

srcW4 = Num_4x4_Blocks_Wide[ MiSizes[ 0 ][ MiRow - 1 ][ MiCol + deltaCol ] ]
if (srcW4 == 1) {
    return 0
}
return MiCol + deltaCol + 1 < MiCols
}

```

§ 7.12.3.2. Add sample process

The inputs to this process are:

- a variable ref specifying which set of candidate motion vectors to prepare,
- a variable deltaRow specifying (in units of 4x4 luma samples) how far above to look for a motion vector,
- a variable deltaCol specifying (in units of 4x4 luma samples) how far left to look for a motion vector.

The output of this process is to add a new sample to the list of candidates if it is a valid candidate and has not been seen before.

If NumSamples[ref] is greater than or equal to LEAST_SQUARES_SAMPLES_MAX, this process immediately terminates.

The variable mvRow is set equal to MiRow + deltaRow.

The variable mvCol is set equal to MiCol + deltaCol.

If RefFrames[mvRow][mvCol][0] has not been written for this frame, this process immediately terminates.

The candidates are added as follows:

```

for(list=0;list<2;list++) {
    if ( RefFrames[ mvRow ][ mvCol ][ list ] == RefFrame[ ref ] ) {
        candSz = MiSizes[ PlaneStart ][ mvRow ][ mvCol ]
        candW4 = Num_4x4_Blocks_Wide[ candSz ]
        candH4 = Num_4x4_Blocks_High[ candSz ]
        candRow = MiRowBase[ 0 ][ mvRow ][ mvCol ]
        candCol = MiColBase[ 0 ][ mvRow ][ mvCol ]
        midY = candRow * 4 + candH4 * 2 - 1
        midX = candCol * 4 + candW4 * 2 - 1
        cand[ 0 ] = midY * 8
        cand[ 1 ] = midX * 8
        cand[ 2 ] = midY * 8 + Mvs[ candRow ][ candCol ][ list ][ 0 ]
        cand[ 3 ] = midX * 8 + Mvs[ candRow ][ candCol ][ list ][ 1 ]
        for ( i = 0; i < 4; i++ )
            CandList[ ref ][ NumSamples[ ref ] ][ i ] = cand[ i ]
        NumSamples[ ref ]++
        if ( NumSamples[ ref ] >= LEAST_SQUARES_SAMPLES_MAX )
            return
    }
}

```

NOTE: candRow and candCol give the top-left position of the candidate block in units of 4x4 blocks. midX and midY give the central position of the candidate block in units of luma samples.

§ 7.13. Prediction processes

§ 7.13.1. General

The following sections define the processes used for predicting the sample values.

These processes are triggered at points defined by function calls to `predict_intra`, `predict_inter`, `predict_chroma_from_luma`, and `predict_palette` in the residual syntax table described in [§5.20.7.23 Residual syntax](#).

§ 7.13.2. Intra prediction process

§ 7.13.2.1. General

The [intra prediction](#) process is invoked for intra coded blocks to predict a part of the block corresponding to a [transform block](#). When the transform size is smaller than the block size, this process can be invoked multiple times within a single block for the same plane, and the invocations are in [raster scan](#) order within the block.

This process is triggered by a call to `predict_intra`.

The inputs to this process are:

- a variable `plane` specifying which plane is being predicted,
- variables `x` and `y` specifying the location of the top left sample in the `CurrFrame[ plane ]` array of the current transform block,
- a variable `haveLeft` that is equal to 1 if there are valid samples to the left of this transform block,
- a variable `haveAbove` that is equal to 1 if there are valid samples above this transform block,
- a variable `num4AboveRight` that specifies the number of valid samples (in units of 4x4 samples) above the transform block to the right of this transform block,
- a variable `num4BelowLeft` that specifies the number of valid samples (in units of 4x4 samples) to the left of the transform block below this transform block,
- a variable `mode` specifying the type of intra prediction to apply,
- a variable `log2W` specifying the base 2 logarithm of the width of the region to be predicted,
- a variable `log2H` specifying the base 2 logarithm of the height of the region to be predicted.

The process makes use of the already reconstructed samples in the current frame `CurrFrame` to form a prediction for the current block.

The outputs of this process are intra predicted samples in the current frame `CurrFrame`.

The variable `w` is set equal to $1 \ll \log2W$.

The variable `h` is set equal to $1 \ll \log2H$.

The variable `maxX` is set equal to $(\text{MiCols} * \text{MI_SIZE}) - 1$.

The variable `maxY` is set equal to $(\text{MiRows} * \text{MI_SIZE}) - 1$.

If `plane` is greater than 0 and `w` is greater than 32, the variable `num4AboveRight` is set equal to 0.

If plane is greater than 0 and h is greater than 32, the variable num4BelowLeft is set equal to 0.

The variable pxTopRight is set equal to 4 * num4AboveRight.

The variable pxBotLeft is set equal to 4 * num4BelowLeft.

If plane is greater than 0, then:

- maxX is set equal to ((MiCols * MI_SIZE) >> SubsamplingX) - 1.
- maxY is set equal to ((MiRows * MI_SIZE) >> SubsamplingY) - 1.

If is_inter is equal to 0 and plane is greater than 0 and UVMMode is equal to UV_CFL_PRED and cfl_index is equal to CFL_MULTI, the luma reference samples in the arrays CflRef are captured as follows:

```
CflAbove = haveAbove ? 2 : 0
CflLeft = haveLeft ? 2 : 0
subX = SubsamplingX
subY = SubsamplingY
lumaW = w << subX
lumaH = h << subY
if (lumaW <= 4 || !haveAbove) {
    pxTopRight = 0
}
if (lumaH <= 4 || !haveLeft) {
    pxBotLeft = 0
}
rightLumaX = Min( MiColEnd * MI_SIZE, (x + w + pxTopRight) << subX)
bottomLumaY = Min( MiRowEnd * MI_SIZE, (y + h + pxBotLeft) << subY)
sbRow = MiRow >> Mi_Height_Log2[ SbSize ]
sbChromaY = ( sbRow * Block_Height[ SbSize ] ) >> subY
CflRefWidth = Min((CflLeft << subX) + (rightLumaX - (x << subX)), 128)
CflRefHeight = Min((CflAbove << subY) + (bottomLumaY - (y << subY)), 128)
for( i = 0; i < h + CflAbove; i++ ) {
    for( j = 0; j < w + CflLeft ; j++ ) {
        CflRef[ 0 ][ i ][ j ] = 0
    }
}
for( i = 0; i < Round2( CflRefHeight, subY ); i++ ) {
    for( j = 0; j < Round2( CflRefWidth, subX ); j++ ) {
        chromaX = x + j - CflLeft
        chromaY = y + i - CflAbove
        if ( i < CflAbove || j < CflLeft ) {
            CflRef[ 1 ][ i ][ j ] =
                CurrFrame[ plane ][ Max(chromaY, sbChromaY - 1) ][ chromaX ]
        }
        if ( cfl_ref_luma_avail(i,j,w,h) ){
            CflRef[ 0 ][ i ][ j ] =
                get_cfl_luma_sample( chromaX, chromaY, j == 0, i == 0 )
        }
    }
}
```

where the function get_cfl_luma_sample (which gets an estimate of luma corresponding to the chroma location) is defined as:

```
get_cfl_luma_sample(chromaX,chromaY,clampX,clampY) {
    lumaX = chromaX << SubsamplingX
    lumaY = chromaY << SubsamplingY
    sbRow = MiRow >> Mi_Height_Log2[ SbSize ]
    limitLumaY = sbRow * Block_Height[ SbSize ] - 1
    filterIdx = cfl_ds_filter_index
    if (filterIdx == 3) {
        filterIdx = 0
    }
}
```



```

}
t = 0
subX = SubsamplingX
subY = SubsamplingY
for (dy = -subY; dy <= subY; dy++) {
    for (dx = -subX; dx <= subX; dx++) {
        v = CurrFrame[0]
            [Max( limitLumaY, lumaY + (clampY ? Max(0, dy) : dy) )]
            [lumaX + (clampX ? Max(0, dx) : dx)]
        if (subX && subY) {
            t += Cfl_Filters_420[filterIdx][dy + subY][dx + subX] * v
        } else if (subX) {
            t += Cfl_Filters_422[filterIdx][dx + subX] * v
        } else {
            t = 8 * v
        }
    }
}
r = t >> 3
return r
}

```

The variable MrlIndex and the arrays AboveRow and LeftCol are prepared as follows:

```

MrlIndex = (plane == 0) ? mrl_index : 0
sbHeight = Block_Height[ SbSize ]
sbBoundary = (y & (sbHeight - 1)) == 0
aboveMrlIndex = sbBoundary ? 0 : MrlIndex
useDip = plane == 0 && use_dip
if ( useDip ) {
    numAboveNeeded = w + (w >> 2)
    numLeftNeeded = h + (h >> 2)
} else {
    numAboveNeeded = w + h + (MrlIndex << 1)
    numLeftNeeded = w + h + (MrlIndex << 1)
}
for ( i = 0; i < numLeftNeeded; i++ ) {
    if ( haveLeft == 0 && haveAbove == 1 ) {
        LeftCol[ i ] = CurrFrame[ plane ][ y - 1 - aboveMrlIndex ][ x ]
        LeftSecCol[ i ] = CurrFrame[ plane ][ y - 1 ][ x ]
    } else if ( haveLeft == 0 ) {
        LeftCol[ i ] = ( 1 << ( BitDepth - 1 ) ) + 1
        LeftSecCol[ i ] = ( 1 << ( BitDepth - 1 ) ) + 1
    } else {
        leftLimit = Min( maxY, y + h + 4 * num4BelowLeft - 1 )
        LeftCol[ i ] =
            CurrFrame[ plane ][ Min(leftLimit, y+i) ][ x - 1 - MrlIndex ]
        LeftSecCol[ i ] = CurrFrame[ plane ][ Min(leftLimit, y+i) ][ x - 1 ]
    }
}
for ( i = 0; i < numAboveNeeded; i++ ) {
    if ( haveAbove == 0 && haveLeft == 1 ) {
        AboveRow[ i ] = CurrFrame[ plane ][ y ][ x - 1 - MrlIndex ]
        AboveSecRow[ i ] = CurrFrame[ plane ][ y ][ x - 1 ]
    } else if ( haveAbove == 0 ) {
        AboveRow[ i ] = ( 1 << ( BitDepth - 1 ) ) - 1
        AboveSecRow[ i ] = ( 1 << ( BitDepth - 1 ) ) - 1
    } else {
        aboveLimit = Min( maxX, x + w + 4 * num4AboveRight - 1 )
        AboveRow[ i ] =
            CurrFrame[ plane ][ y - 1 - aboveMrlIndex ][ Min(aboveLimit, x+i) ]
        AboveSecRow[ i ] = CurrFrame[ plane ][ y - 1 ][ Min(aboveLimit, x+i) ]
    }
}
for ( i = 1; i <= 1 + MrlIndex; i++ ) {
    if ( haveAbove == 1 && haveLeft == 1 ) {
        AboveRow[ -i ] = CurrFrame[ plane ][ y - 1 - aboveMrlIndex ][ x - i ]
        LeftCol[ -i ] = CurrFrame[ plane ][ y - Min(i, 1 + aboveMrlIndex) ]
            [ x - 1 - MrlIndex ]
    }
}

```

```

    AboveSecRow[ -i ] = CurrFrame[ plane ][ y - 1 ][ x - i ]
    LeftSecCol[ -i ] = CurrFrame[ plane ][ y - 1 ][ x - 1 ]
} else if ( haveAbove == 1 ) {
    AboveRow[ -i ] = CurrFrame[ plane ][ y - 1 - aboveMrlIndex ][ x ]
    LeftCol[ -i ] = AboveRow[ -i ]
    AboveSecRow[ -i ] = CurrFrame[ plane ][ y - 1 ][ x ]
    LeftSecCol[ -i ] = AboveSecRow[ -i ]
} else if ( haveLeft == 1 ) {
    AboveRow[ -i ] = CurrFrame[ plane ][ y ][ x - 1 - MrlIndex ]
    LeftCol[ -i ] = AboveRow[ -i ]
    AboveSecRow[ -i ] = CurrFrame[ plane ][ y ][ x - 1 ]
    LeftSecCol[ -i ] = AboveSecRow[ -i ]
} else {
    AboveRow[ -i ] = 1 << ( BitDepth - 1 )
    LeftCol[ -i ] = 1 << ( BitDepth - 1 )
    AboveSecRow[ -i ] = 1 << ( BitDepth - 1 )
    LeftSecCol[ -i ] = 1 << ( BitDepth - 1 )
}
}
}

```

The variable largeChroma is set as follows:

- If plane is equal to 0, largeChroma is set equal to 0.
- Otherwise, if w is greater than 32 or h is greater than 32, largeChroma is set equal to 1.
- Otherwise, largeChroma is set equal to 0.

A 2D array named pred containing the intra predicted samples is constructed as follows:

- If useDip is equal to 1, the data driven intra prediction process specified in [§ 7.13.2.3 Data driven intra prediction process](#) is invoked with w and h as inputs and the output is assigned to pred.
- Otherwise, if is_directional_mode(mode) is equal to 1, the directional intra prediction process specified in [§ 7.13.2.7 Directional intra prediction process](#) is invoked with plane, x, y, haveLeft, haveAbove, mode, w, h, maxX, maxY as inputs and the output is assigned to pred.
- Otherwise, if mode is equal to SMOOTH_PRED or SMOOTH_V_PRED or SMOOTH_H_PRED, the smooth intra prediction process specified in [§ 7.13.2.13 Smooth intra prediction process](#) is invoked with mode, log2W, log2H, w, and h as inputs, and the output is assigned to pred.
- Otherwise, if largeChroma is equal to 1 and mode is equal to DC_PRED and is_inter is equal to 0 and UVMODE is equal to UV_CFL_PRED, the DC intra prediction subsampled process specified in [§ 7.13.2.11 DC intra prediction subsampled process](#) is invoked with haveLeft, haveAbove, log2W, and log2H as inputs and the output is assigned to pred.
- Otherwise, if mode is equal to DC_PRED, the DC intra prediction process specified in [§ 7.13.2.10 DC intra prediction process](#) is invoked with haveLeft, haveAbove, log2W, and log2H as inputs and the output is assigned to pred.
- Otherwise (mode is equal to PAETH_PRED), the basic intra prediction process specified in [§ 7.13.2.2 Basic intra prediction process](#) is invoked with w, and h as inputs, and the output is assigned to pred.

If all of the following conditions are true, the IBP DC process (which modifies pred) specified in [§ 7.13.2.12 IBP DC process](#) is invoked with haveLeft, haveAbove, log2W, log2H, w, h, and pred as inputs:

- enable_ibp is equal to 1.
- useDip is equal to 0.
- mode is equal to DC_PRED.

- w is not equal to 4 or h is not equal to 4.
- plane is equal to 0 or UVMode is not equal to UV_CFL_PRED.

The current frame is updated as follows:

- CurrFrame[plane][y + i][x + j] is set equal to pred[i][j] for i = 0..h-1 and j = 0..w-1.

§ 7.13.2.2. Basic intra prediction process

The inputs to this process are:

- a variable w specifying the width of the region to be predicted,
- a variable h specifying the height of the region to be predicted.

The output of this process is a 2D array named pred containing the intra predicted samples.

The process generates filtered samples from the samples in LeftCol and AboveRow as follows:

- The following ordered steps apply for i = 0..h-1, for j = 0..w-1:
 1. The variable base is set equal to AboveRow[j] + LeftCol[i] - AboveRow[-1].
 2. The variable pLeft is set equal to Abs(base - LeftCol[i]).
 3. The variable pTop is set equal to Abs(base - AboveRow[j]).
 4. The variable pTopLeft is set equal to Abs(base - AboveRow[-1]).
 5. The predicted sample is computed as follows:
 - If pLeft is less than or equal to pTop and pLeft is less than or equal to pTopLeft, pred[i][j] is set equal to LeftCol[i].
 - Otherwise, if pTop is less than or equal to pTopLeft, pred[i][j] is set equal to AboveRow[j].
 - Otherwise, pred[i][j] is set equal to AboveRow[-1].

The output of the process is the array pred.

§ 7.13.2.3. Data driven intra prediction process

The inputs to this process are:

- a variable w specifying the width of the region to be predicted,
- a variable h specifying the height of the region to be predicted.

The output of this process is a 2D array named pred containing the intra predicted samples.

The following ordered steps apply:

1. The DIP features process specified in [§ 7.13.2.4 DIP features process](#) is invoked with w and h as inputs, and the output is assigned to f.
2. The DIP transform process specified in [§ 7.13.2.5 DIP transform process](#) is invoked with f as input, and the output is assigned to dipPred.

3. The DIP resample process specified in [§7.13.2.6 DIP resample process](#) is invoked with w, h, and dipPred as inputs, and the output is assigned to pred.

§ 7.13.2.4. DIP features process

The inputs to this process are:

- a variable w specifying the width of the region to be predicted,
- a variable h specifying the height of the region to be predicted.

The output of this process is a 1D array named f containing 11 features extracted from previously decoded samples in the current frame.

The features are prepared as follows:

```
f[ 0 ] = AboveRow[-1]
fAbove = dip_avg( 0, w )
fLeft = dip_avg( 1, h )
for(i = 0; i < 4; i++) {
    f[ i + 1 ] = dip_transpose ? fLeft[ i ] : fAbove[ i ]
    f[ i + 5 ] = dip_transpose ? fAbove[ i ] : fLeft[ i ]
}
f[ 9 ] = dip_transpose ? fLeft[ 4 ] : fAbove[ 4 ]
f[ 10 ] = dip_transpose ? fAbove[ 4 ] : fLeft[ 4 ]
```

where the function dip_avg downsamples the previously decoded samples as follows:

```
dip_avg( dir, n ) {
    down = n >> 2
    for( i = 0; i < 5; i++ ) {
        t = 0
        for ( j = 0; j < down; j++ ) {
            t += dir ? LeftCol[ i * down + j ] : AboveRow[ i * down + j ]
        }
        f[ i ] = ( t + (down >> 1) ) / down
    }
    return f
}
```

§ 7.13.2.5. DIP transform process

The input to this process is an array of 11 features named f.

The output of this process is an 8 by 8 2D array pred of the predicted samples.

The prediction is formed as follows:

```
for( i = 0; i < 8; i++ ) {
    for( j = 0; j < 8; j++ ) {
        c = 0
        for( k = 0; k < 11; k++ ) {
            c += Dip_Weights[ dip_mode ][ i * 8 + j ][ k ] * f[ k ]
        }
        v = Clip1( Round2( c, 10 ) )
        if ( dip_transpose ) {
            pred[ j ][ i ] = v
        } else {
            pred[ i ][ j ] = v
        }
    }
}
```

```

    }
}
}

```

§ 7.13.2.6. DIP resample process

The inputs to this process are:

- a variable w specifying the width of the region to be predicted,
- a variable h specifying the height of the region to be predicted,
- an 8 by 8 array dipPred of predicted samples.

The output of this process is the 2D array pred containing the predicted samples resampled to a size of w by h.

The samples are formed as follows:

```

upx = Max(1, w / 8)
upy = Max(1, h / 8)
downx = Max(1, 8 / w)
downy = Max(1, 8 / h)
for( i = 0; i < Min(h, 8); i++ ) {
    y = (i + 1) * upy - 1
    for( j = 0; j < Min(w, 8); j++ ) {
        p0 = j == 0 ? LeftCol[ y ] : dipPred[ i * downy ][ (j - 1) * downx ]
        p1 = dipPred[ i * downy ][ j * downx ]
        for( k = 0; k < upx; k++ ) {
            x = j * upx + k
            w1 = k + 1
            horzInterp[ i ][ x ] = ( (upx - w1) * p0 + w1 * p1 ) / upx
        }
    }
}
for( x = 0; x < w; x++ ) {
    for( i = 0; i < Min(h, 8); i++ ) {
        p0 = i == 0 ? AboveRow[ x ] : horzInterp[ i - 1 ][ x ]
        p1 = horzInterp[ i ][ x ]
        for( k = 0; k < upy; k++ ) {
            y = i * upy + k
            w1 = k + 1
            pred[ y ][ x ] = ( (upy - w1) * p0 + w1 * p1 ) / upy
        }
    }
}

```

§ 7.13.2.7. Directional intra prediction process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the current transform block,
- a variable haveLeft that is equal to 1 if there are valid samples to the left of this transform block,
- a variable haveAbove that is equal to 1 if there are valid samples above this transform block,
- a variable mode specifying the type of intra prediction to apply,
- a variable w specifying the width of the region to be predicted,

- a variable `h` specifying the height of the region to be predicted,
- a variable `maxX` specifying the largest valid `x` coordinate for the current plane,
- a variable `maxY` specifying the largest valid `y` coordinate for the current plane.

The output of this process is a 2D array containing the intra predicted samples.

The process uses a directional filter to generate filtered samples from the samples in `LeftCol` and `AboveRow`.

The variable `angleDelta` is derived as follows:

- If plane is equal to 0, `angleDelta` is set equal to `AngleDeltaY`.
- Otherwise (plane is not equal to 0), `angleDelta` is set equal to `AngleDeltaUV`.

The variable `pAngle` is derived by the following ordered steps:

1. The variable `pAngle` is set equal to (`Mode_To_Angle[ mode ] + angleDelta * ANGLE_STEP + Mrl_Index_To_Delta[ MrlIndex ]`).
2. If `is_inter` is equal to 0 (meaning we are not using inter intra prediction), the variable `pAngle` is modified as follows:

```
(unusedMode, pAngle) = wide_angle_mapping(mode, w, h, pAngle)
```

The variable `not4x4` is set equal to (`w!=4 || h!=4`).

The variable `applyIbp` is set equal to `enable_ibp && not4x4`.

The following ordered steps (which prepare filtered edge samples) apply:

1. If `enable_intra_edge_filter` is equal to 1 and `MrlIndex` is equal to 0, the following applies:

```
filterTypeAbove = 0
filterTypeLeft = 0
angleAbove = pAngle - 90
angleLeft = pAngle - 180
needRight = pAngle < 90
needBottom = pAngle > 180
if ( pAngle != 90 && pAngle != 180 ) {
    filterTypeAbove = get_filter_type_above( plane )
    filterTypeLeft = get_filter_type_left( plane )
    if ( applyIbp ) {
        needRight |= pAngle > 180
        needBottom |= pAngle < 90
        if ( angleAbove > 90 ) {
            angleAbove -= 180
        }
        if ( angleLeft < -90 ) {
            angleLeft += 180
        }
    }
    else {
        filterType = filterTypeAbove | filterTypeLeft
        filterTypeAbove = filterType
        filterTypeLeft = filterType
    }
}
if ( ( applyIbp || (pAngle > 90 && pAngle < 180) ) && ( w + h ) >= 24 ) {
    LeftCol[ -1 ] = filter_corner( )
    AboveRow[ -1 ] = LeftCol[ -1 ]
}
```

```

}
if ( haveAbove == 1 ) {
    strength = intra_edge_filter_strength_selection( w, h, filterTypeAbove, angleAbove )
    numPx = Min( w, ( maxX - x + 1 ) ) + ( needRight ? h : 0 ) + 1
    intra_edge_filter( numPx, strength, 0 )
}
if ( haveLeft == 1 ) {
    strength = intra_edge_filter_strength_selection( w, h, filterTypeLeft, angleLeft )
    numPx = Min( h, ( maxY - y + 1 ) ) + ( needBottom ? w : 0 ) + 1
    intra_edge_filter( numPx, strength, 1 )
}
}

```

The call of `get_filter_type_above` indicates that the intra filter type above process specified in [§ 7.13.2.15 Intra filter type above process](#) is invoked.

The call of `get_filter_type_left` indicates that the intra filter type left process specified in [§ 7.13.2.16 Intra filter type left process](#) is invoked.

The call of `intra_edge_filter_strength_selection` indicates that the intra edge filter strength selection process specified in [§ 7.13.2.17 Intra edge filter strength selection process](#) is invoked.

The call of `intra_edge_filter` indicates that the intra edge filter process specified in [§ 7.13.2.18 Intra edge filter process](#) is invoked.

2. The single directional prediction process specified in [§ 7.13.2.8 Single directional prediction process](#) is invoked with `pAngle`, `w`, `h`, `MrlIndex`, and `plane` as inputs, and the output is assigned to `pred`.
3. If `MrlIndex` is greater than 0 and `mrl_sec_index` is equal to 1 and `not4x4` is equal to 1, the following ordered steps apply:
 1. `LeftCol` is set equal to a copy of `LeftSecCol`.
 2. `AboveRow` is set equal to a copy of `AboveSecRow`.
 3. The single directional prediction process specified in [§ 7.13.2.8 Single directional prediction process](#) is invoked with `pAngle`, `w`, `h`, 0, and `plane` as inputs, and the output is assigned to `pred2`.
 4. Set `combinedPred[r][c]` equal to $(\text{pred}[r][c] + \text{pred2}[r][c] + 1) \gg 1$ for $r = 0..h-1$ and $c = 0..w-1$.
 5. The process terminates immediately with `combinedPred` as output.

The constant table `Mrl_Index_To_Delta` is defined as follows:

```

Mrl_Index_To_Delta[4] = {
    0, 1, -1, 0
}

```

The variable `useIBP` is set equal to 1 if all of the following conditions are true, otherwise, `useIBP` is set equal to 0:

- `applyIbp` is equal to 1.
- `angleDelta` is even.
- `plane` is equal to 0.
- `pAngle` is less than 90 or `pAngle` is greater than 180.
- `MrlIndex` is equal to 0.

If useIBP is equal to 0, this process immediately terminates with pred as output.

Otherwise, the weights and secondAngle are computed as follows:

```
if (pAngle < 90) {
    weights = ibp_weights(pAngle)
    secondAngle = pAngle + 180
} else {
    weights = ibp_weights(270 - pAngle)
    secondAngle = pAngle - 180
}
```

The call of `ibp_weights` indicates that the IBP weights process specified in [§ 7.13.2.9 IBP weights process](#) is invoked.

The single directional prediction process specified in [§ 7.13.2.8 Single directional prediction process](#) is invoked with `secondAngle`, `w`, `h`, `MrlIndex`, and `plane` as inputs, and the output is assigned to `secondPred`.

The combined prediction is formed as a weighted blend of the two predictions as follows:

```
cShift = w >> (IBP_WEIGHT_SIZE_LOG2 + 1)
rShift = h >> (IBP_WEIGHT_SIZE_LOG2 + 1)
for (r = 0; r < h; r++) {
    for (c = 0; c < w; c++) {
        s = pAngle < 90 ? weights[r >> rShift][c >> cShift] :
                        weights[c >> cShift][r >> rShift]
        combinedPred[r][c] = Round2( pred[r][c] * s +
                                     secondPred[r][c] * (IBP_WEIGHT_MAX - s),
                                     IBP_WEIGHT_SHIFT)
    }
}
```

The output of the process is the array `combinedPred`.

[§](#) **7.13.2.8.** *Single directional prediction process*

The inputs to this process are:

- a variable `pAngle` specifying the angle to use for directional prediction,
- a variable `w` specifying the width of the region to be predicted,
- a variable `h` specifying the height of the region to be predicted,
- a variable `mrlIndex` specifying the distance of the edge samples from the block,
- a variable `plane` specifying whether to use IDIF filtering.

The output of this process is a 2D array named `pred` containing the intra predicted samples.

The variable `enableIdif` is set equal to `plane == 0`.

If `enableIdif` is equal to 1, the following applies:

```
minBase = -(1 + mrlIndex)
maxBase = w + h - 1 + (mrlIndex << 1)
if ( pAngle > 90 && pAngle < 180 ) {
    LeftCol[h] = LeftCol[h - 1]
    AboveRow[w] = AboveRow[w - 1]
    LeftCol[h + 1] = LeftCol[h - 1]
```



```

    AboveRow[w + 1] = AboveRow[w - 1]
} else {
    LeftCol[maxBase + 1] = LeftCol[maxBase]
    AboveRow[maxBase + 1] = AboveRow[maxBase]
    LeftCol[maxBase + 2] = LeftCol[maxBase]
    AboveRow[maxBase + 2] = AboveRow[maxBase]
}
LeftCol[minBase - 1] = LeftCol[minBase]
AboveRow[minBase - 1] = AboveRow[minBase]

```

1. If pAngle is less than 90, the following steps apply for i = 0..h-1, for j = 0..w-1:

- The variable dx is set equal to Dr_Intra_Derivative[pAngle].
- The variable idx is set equal to (i + 1 + mrlIndex) * dx.
- The variable base is set equal to (idx >> 6) + j.
- The variable shift is set equal to (idx >> 1) & 0x1F.
- The variable maxBaseX is set equal to (w + h - 1 + (mrlIndex << 1)).
- If base is less than maxBaseX + enableIdif, the samples are filtered as follows:

```

if ( enableIdif ) {
    s = 0
    for(t = 0 ; t < 4; t++) {
        s += Dr_Interp_Filter[ shift ][ t ] * AboveRow[ base + t - 1 ]
    }
    pred[ i ][ j ] = Clip1( Round2( s, 7 ) )
} else {
    pred[ i ][ j ] = Round2( AboveRow[ base ] * ( 32 - shift ) + AboveRow[ base + 1 ] * shift, 5 )
}

```

- Otherwise (base is greater than or equal to maxBaseX + enableIdif), pred[i][j] is set equal to AboveRow[maxBaseX].

2. Otherwise, if pAngle is greater than 90 and pAngle is less than 180, the following steps apply for i = 0..h-1, for j = 0..w-1:

- The variable dx is set equal to Dr_Intra_Derivative[180 - pAngle].
- The variable dy is set equal to Dr_Intra_Derivative[pAngle - 90].
- The variable idx is set equal to (j << 6) - (i + 1 + mrlIndex) * dx.
- The variable base is set equal to idx >> 6 .
- If base is greater than or equal to -(1 + mrlIndex), the following steps apply:
 - The variable shift is set equal to (idx >> 1) & 0x1F.
 - The samples are filtered as follows:

```

if ( enableIdif ) {
    s = 0
    for(t = 0 ; t < 4; t++) {
        s += Dr_Interp_Filter[ shift ][ t ] * AboveRow[ base + t - 1 ]
    }
    pred[ i ][ j ] = Clip1( Round2( s, 7 ) )
} else {

```

```

    pred[ i ][ j ] = Round2( AboveRow[ base ] * ( 32 - shift ) + AboveRow[ base + 1 ] * shift,
5 )
}

```

◦ Otherwise, the following steps apply:

- The variable `idx` is set equal to $(i \ll 6) - (j + 1 + \text{mrlIndex}) * \text{dy}$.
- The variable `base` is set equal to $\text{idx} \gg 6$.
- The variable `shift` is set equal to $(\text{idx} \gg 1) \& 0x1F$.
- The samples are filtered as follows:

```

if ( enableIdif ) {
    s = 0
    for( t = 0 ; t < 4; t++ ) {
        s += Dr_Interp_Filter[ shift ][ t ] * LeftCol[ base + t - 1 ]
    }
    pred[ i ][ j ] = Clip1( Round2( s, 7 ) )
} else {
    pred[ i ][ j ] = Round2( LeftCol[ base ] * ( 32 - shift ) + LeftCol[ base + 1 ] * shift, 5 )
}

```

3. Otherwise, if `pAngle` is greater than 180, the following steps apply for $i = 0..h-1$, for $j = 0..w-1$:

- The variable `dy` is set equal to `Dr_Intra_Derivative[ 270 - pAngle ]`.
- The variable `idx` is set equal to $(j + 1 + \text{mrlIndex}) * \text{dy}$.
- The variable `base` is set equal to $(\text{idx} \gg 6) + i$.
- The variable `shift` is set equal to $(\text{idx} \gg 1) \& 0x1F$.
- The variable `maxBaseY` is set equal to $(w + h - 1 + (\text{mrlIndex} \ll 1))$.
- If `base` is less than `maxBaseY + enableIdif`, the samples are filtered as follows:

```

if ( enableIdif ) {
    s = 0
    for( t = 0 ; t < 4; t++ ) {
        s += Dr_Interp_Filter[ shift ][ t ] * LeftCol[ base + t - 1 ]
    }
    pred[ i ][ j ] = Clip1( Round2( s, 7 ) )
} else {
    pred[ i ][ j ] = Round2( LeftCol[ base ] * ( 32 - shift ) + LeftCol[ base + 1 ] * shift, 5 )
}

```

- Otherwise (base is greater than or equal to `maxBaseY + enableIdif`), `pred[ i ][ j ]` is set equal to `LeftCol[ maxBaseY ]`.

4. Otherwise, if `pAngle` is equal to 90, `pred[ i ][ j ]` is set equal to `AboveRow[ j ]` with $j = 0..w-1$ and $i = 0..h-1$ (each row of the block is filled with a copy of `AboveRow`).
5. Otherwise, if `pAngle` is equal to 180, `pred[ i ][ j ]` is set equal to `LeftCol[ i ]` with $j = 0..w-1$ and $i = 0..h-1$ (each column of the block is filled with a copy of `LeftCol`).

The output of the process is the array `pred`.

The filter taps in the constant table `Dr_Interp_Filter` (used when `enableIdif` is equal to 1) are defined as:

```
Dr_Interp_Filter[ 32 ][ 4 ] = {
    { 0, 128, 0, 0 }, { -2, 127, 4, -1 }, { -3, 125, 8, -2 },
    { -5, 123, 13, -3 }, { -6, 121, 17, -4 }, { -7, 118, 22, -5 },
    { -9, 116, 27, -6 }, { -9, 112, 32, -7 }, { -10, 109, 37, -8 },
    { -11, 106, 41, -8 }, { -11, 102, 46, -9 }, { -12, 98, 52, -10 },
    { -12, 94, 56, -10 }, { -12, 90, 61, -11 }, { -12, 85, 66, -11 },
    { -12, 81, 71, -12 }, { -12, 76, 76, -12 }, { -12, 71, 81, -12 },
    { -11, 66, 85, -12 }, { -11, 61, 90, -12 }, { -10, 56, 94, -12 },
    { -10, 52, 98, -12 }, { -9, 46, 102, -11 }, { -8, 41, 106, -11 },
    { -8, 37, 109, -10 }, { -7, 32, 112, -9 }, { -6, 27, 116, -9 },
    { -5, 22, 118, -7 }, { -4, 17, 121, -6 }, { -3, 13, 123, -5 },
    { -2, 8, 125, -3 }, { -1, 4, 127, -2 }
}
```

§ 7.13.2.9. IBP weights process

The input to this process is a variable `pAngle` specifying the angle to use for directional prediction.

The output of this process is a 2D array named `weights` containing the blending weights.

The array `weights` is computed as follows:

```
pAngle = Max( 39, pAngle )
dy = Dr_Intra_Derivative[90 - pAngle]
for (r = 0; r < IBP_WEIGHT_SIZE; r++) {
    y = dy
    for (c = 0; c < IBP_WEIGHT_SIZE; c++) {
        dist = ((r + 1) << 6) + y
        (shift, div) = resolve_divisor(dist)
        shift -= DIV_LUT_BITS
        weight0 = Round2(y * div, shift)
        weights[r][c] = weight0
        y += dy
    }
}
```

The output of the process is the array `weights`.

§ 7.13.2.10. DC intra prediction process

The inputs to this process are:

- a variable `haveLeft` that is equal to 1 if there are valid samples to the left of this transform block,
- a variable `haveAbove` that is equal to 1 if there are valid samples above this transform block,
- a variable `log2W` specifying the base 2 logarithm of the width of the region to be predicted,
- a variable `log2H` specifying the base 2 logarithm of the height of the region to be predicted.

The output of this process is a 2D array named `pred` containing the intra predicted samples.

The variable `w` is set equal to $1 \ll \log2W$.

The variable `h` is set equal to $1 \ll \log2H$.

The process averages the available edge samples in LeftCol and AboveRow to generate the prediction as follows:

- If haveLeft is equal to 1 and haveAbove is equal to 1, pred[i][j] is set equal to avg with i = 0..h-1 and j = 0..w-1. The variable avg (the average of the samples in union of AboveRow and LeftCol) is specified as follows:

```
sum = 0
for ( k = 0; k < h; k++ )
    sum += LeftCol[ k ]
for ( k = 0; k < w; k++ )
    sum += AboveRow[ k ]

avg = Clip1( approx_divide(sum, w + h) )
```

- Otherwise, if haveLeft is equal to 1 and haveAbove is equal to 0, pred[i][j] is set equal to leftAvg with i = 0..h-1 and j = 0..w-1. The variable leftAvg is specified as follows:

```
sum = 0
for ( k = 0; k < h; k++ ) {
    sum += LeftCol[ k ]
}
leftAvg = Round2( sum, log2H )
```

- Otherwise, if haveLeft is equal to 0 and haveAbove is equal to 1, pred[i][j] is set equal to aboveAvg with i = 0..h-1 and j = 0..w-1. The variable aboveAvg is specified as follows:

```
sum = 0
for ( k = 0; k < w; k++ ) {
    sum += AboveRow[ k ]
}
aboveAvg = Round2( sum, log2W )
```

- Otherwise (haveLeft is equal to 0 and haveAbove is equal to 0), pred[i][j] is set equal to 1 << (BitDepth - 1) with i = 0..h-1 and j = 0..w-1.

The output of the process is the array pred.

§ 7.13.2.11. DC intra prediction subsampled process

The inputs to this process are:

- a variable haveLeft that is equal to 1 if there are valid samples to the left of this transform block,
- a variable haveAbove that is equal to 1 if there are valid samples above this transform block,
- a variable log2W specifying the base 2 logarithm of the width of the region to be predicted,
- a variable log2H specifying the base 2 logarithm of the height of the region to be predicted.

The output of this process is a 2D array named pred containing the intra predicted samples.

The variable w is set equal to 1 << log2W.

The variable h is set equal to 1 << log2H.

The process averages the available edge samples in LeftCol and AboveRow to generate the prediction as follows:

```

sum = 0
count = 0
if ( haveLeft ) {
    stepH = h > 32 ? 2 : 1
    for ( k = 0; k < h; k += stepH ) {
        sum += LeftCol[ k ]
        count++
    }
}
if ( haveAbove ) {
    stepW = w > 32 ? 2 : 1
    for ( k = 0; k < w; k += stepW ) {
        sum += AboveRow[ k ]
        count++
    }
}
if ( count == 0 ) {
    avg = 1 << (BitDepth - 1)
} else {
    avg = Clip1( approx_divide(sum, count) )
}
for ( i = 0; i < h; i++ )
    for ( j = 0; j < w; j++ )
        pred[ i ][ j ] = avg

```

where approx_divide approximates the division of sum by count and is specified as:

```

approx_divide(num, den) norange {
    (shift, scale) = resolve_divisor(den)
    return Round2(num * scale, shift)
}

```

NOTE: The divide is only approximate so the average value computed by approx_divide needs to be clipped so that the predicted value fits within BitDepth bits.

§ 7.13.2.12. IBP DC process

The inputs to this process are:

- a variable haveLeft that is equal to 1 if there are valid samples to the left of this transform block,
- a variable haveAbove that is equal to 1 if there are valid samples above this transform block,
- a variable log2W specifying the base 2 logarithm of the width of the region to be predicted,
- a variable log2H specifying the base 2 logarithm of the height of the region to be predicted,
- a variable w specifying the width of the region to be predicted,
- a variable h specifying the height of the region to be predicted,
- an array pred containing the DC predicted samples.

This process modifies the intra predicted samples in the array pred as follows:

```

if (haveAbove) {
    for (r = 0; r < (h >> 2); r++) {
        for (c = (w < h && haveLeft) ? w >> 2 : 0; c < w; c++) {

```

```

        s = Ibp_Weights[log2H - 2][r]
        pred[ r ][ c ] = Round2( AboveRow[c] * (IBP_WEIGHT_MAX - s) +
                                pred[ r ][ c ] * s, IBP_WEIGHT_SHIFT )
    }
}
if (haveLeft) {
    for (r = (w >= h && haveAbove) ? h >> 2 : 0; r < h; r++) {
        for (c = 0; c < (w >> 2); c++) {
            s = Ibp_Weights[log2W - 2][c]
            pred[ r ][ c ] = Round2( LeftCol[r] * (IBP_WEIGHT_MAX - s) +
                                    pred[ r ][ c ] * s, IBP_WEIGHT_SHIFT )
        }
    }
}

```

where the constant table Ibp_Weights is defined as:

```

Ibp_Weights[ 5 ][ 16 ] = {
    { 96, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 86, 107, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 77, 90, 102, 115, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 71, 78, 86, 92, 100, 107, 114, 121, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 68, 72, 76, 79, 83, 87, 90, 94, 98, 102, 106, 109, 113, 117, 121, 124 }
}

```

§ 7.13.2.13. Smooth intra prediction process

The inputs to this process are:

- a variable mode specifying the type of intra prediction to apply,
- a variable log2W specifying the base 2 logarithm of the width of the region to be predicted,
- a variable log2H specifying the base 2 logarithm of the height of the region to be predicted,
- a variable w specifying the width of the region to be predicted,
- a variable h specifying the height of the region to be predicted.

The output of this process is a 2D array named pred containing the intra predicted samples.

The process uses linear interpolation to generate filtered samples from the samples in LeftCol and AboveRow.

The variable bl is set equal to LeftCol[h].

The variable tr is set equal to AboveRow[w].

The variable scale is set equal to Round2(log2W + log2H - 4, 2).

The array pred is derived as follows:

```

for ( i = 0; i < h; i++ ) {
    for ( j = 0; j < w; j++ ) {
        sTop = BLEND_WEIGHT_MAX >> Min(6, (i << 1) >> scale)
        sLeft = BLEND_WEIGHT_MAX >> Min(6, (j << 1) >> scale)
        top = AboveRow[ j ]
        left = LeftCol[ i ]
        predH = tr + Round2( (left - tr) * (w - 1 - j), log2W )
        predV = bl + Round2( (top - bl) * (h - 1 - i), log2H )
        predH2 = predH + Round2( (left - predH) * sLeft, 6 )
    }
}

```

```

    predV2 = predV + Round2( (top - predV) * sTop, 6 )
    if ( mode == SMOOTH_H_PRED ) {
        pred[ i ][ j ] = predH2
    } else if ( mode == SMOOTH_V_PRED ) {
        pred[ i ][ j ] = predV2
    } else {
        pred[ i ][ j ] = Round2( predV2 + predH2, 1 )
    }
}
}

```

The output of the process is the array pred.

§ 7.13.2.14. Filter corner process

This process uses a three tap filter to compute the value to be used for the top-left corner.

The variable s is set equal to LeftCol[0] * 5 + AboveRow[-1] * 6 + AboveRow[0] * 5.

The output of this process is Round2(s, 4).

§ 7.13.2.15. Intra filter type above process

The input to this process is a variable plane specifying the color plane being processed.

The output of this process is a variable that is set to 1 if the block above uses a smooth prediction mode.

The process is specified as follows:

```

get_filter_type_above( plane ) {
    aboveSmooth = 0
    if ( ( plane == 0 ) ? AvailU : AvailUChroma ) {
        if ( plane > 0 && TreeType == SHARED_PART ) {
            r = ChromaMiRow - 1
            c = ChromaMiCol
        } else {
            r = MiRow - 1
            c = MiCol
        }
        aboveSmooth = is_smooth( r, c, plane )
    }
    return aboveSmooth
}

```

where the function is_smooth indicates if a prediction mode is one of the smooth intra modes and is specified as:

```

is_smooth( row, col, plane ) {
    if ( plane == 0 ) {
        mode = YModes[ row ][ col ]
    } else {
        return UVSmooth[ row ][ col ]
    }
    return (mode == SMOOTH_PRED || mode == SMOOTH_V_PRED || mode == SMOOTH_H_PRED)
}

```

§ 7.13.2.16. Intra filter type left process

The input to this process is a variable plane specifying the color plane being processed.

The output of this process is a variable that is set to 1 if the block to the left uses a smooth prediction mode.

The process is specified as follows:

```
get_filter_type_left( plane ) {
    leftSmooth = 0
    if ( ( plane == 0 ) ? AvailL : AvailLChroma ) {
        if ( plane > 0 && TreeType == SHARED_PART ) {
            r = ChromaMiRow
            c = ChromaMiCol - 1
        } else {
            r = MiRow
            c = MiCol - 1
        }
        leftSmooth = is_smooth( r, c, plane )
    }
    return leftSmooth
}
```

[§](#) 7.13.2.17. Intra edge filter strength selection process

The inputs to this process are:

- a variable w containing the width of the transform in samples,
- a variable h containing the height of the transform in samples,
- a variable filterType that is 0 or 1 that controls the strength of filtering,
- a variable delta containing an angle difference in degrees.

The output is an intra edge filter strength from 0 to 3 inclusive.

The variable d is set equal to Abs(delta).

The variable blkWh (containing the sum of the dimensions) is set equal to w + h.

The output variable strength is specified as follows:

```
strength = 0
if ( filterType == 0 ) {
    if ( blkWh <= 8 ) {
        if ( d >= 56 ) strength = 1
    } else if ( blkWh <= 12 ) {
        if ( d >= 40 ) strength = 1
    } else if ( blkWh <= 16 ) {
        if ( d >= 40 ) strength = 1
    } else if ( blkWh <= 24 ) {
        if ( d >= 8 ) strength = 1
        if ( d >= 16 ) strength = 2
        if ( d >= 32 ) strength = 3
    } else if ( blkWh <= 32 ) {
        strength = 1
        if ( d >= 4 ) strength = 2
        if ( d >= 32 ) strength = 3
    } else {
        strength = 3
    }
} else {
    if ( blkWh <= 8 ) {
        if ( d >= 40 ) strength = 1
        if ( d >= 64 ) strength = 2
    }
}
```



```

    } else if ( blkWh <= 16 ) {
        if ( d >= 20 ) strength = 1
        if ( d >= 48 ) strength = 2
    } else if ( blkWh <= 24 ) {
        if ( d >= 4 ) strength = 3
    } else {
        strength = 3
    }
}

```

§ 7.13.2.18. Intra edge filter process

The inputs to this process are:

- a size sz (sz will always be less than or equal to 129),
- a filter strength between 0 and 3 inclusive,
- an edge direction left (when equal to 1, it specifies a vertical edge; when equal to 0, it specifies a horizontal edge).

The process filters the LeftCol (if left is equal to 1) or AboveRow (if left is equal to 0) arrays.

If strength is equal to 0, the process returns without doing anything.

The array edge is derived by setting edge[i] equal to (left ? LeftCol[i - 1] : AboveRow[i - 1]) for i = 0..sz-1.

Otherwise (strength is not equal to 0), the following ordered steps apply for i = 1..sz-1:

1. The variable s is set equal to 0.
2. The following steps now apply for j = 0..INTRA_EDGE_TAPS-1:
 1. The variable k is set equal to Clip3(0, sz - 1, i - 2 + j).
 2. The variable s is incremented by Intra_Edge_Kernel[strength - 1][j] * edge[k].
3. If left is equal to 1, LeftCol[i - 1] is set equal to (s + 8) >> 4.
4. If left is equal to 0, AboveRow[i - 1] is set equal to (s + 8) >> 4.

The array Intra_Edge_Kernel is specified as follows:

```

Intra_Edge_Kernel[INTRA_EDGE_KERNELS][INTRA_EDGE_TAPS] = {
    { 0, 4, 8, 4, 0 },
    { 0, 5, 6, 5, 0 },
    { 2, 4, 4, 4, 2 }
}

```

§ 7.13.3. Inter prediction process

§ 7.13.3.1. General

The inter prediction process is invoked for inter coded blocks and inter intra blocks.

The inputs to this process are:

- a variable plane specifying which plane is being predicted,

- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- variables w and h specifying the width and height of the region to be predicted,
- variables candRow and candCol specifying the location (in units of 4x4 blocks) of the motion vector information to be used,
- a variable fromBuildTip specifying if this was called from the build [TIP](#) process,
- a variable sub8x8Inter specifying whether to disable compound prediction.

The outputs of this process are predicted samples in the current frame CurrFrame.

This process is triggered by a function call to predict_inter.

The variable PuWidth is set equal to w.

The variable PuHeight is set equal to h.

The variable tipPred (indicating if the block has specified [TIP](#)) is set equal to RefFrames[candRow][candCol][0] == TIP_FRAME.

NOTE: tipPred is equal to 0 when called from the build [TIP](#) process.

The array refFrames is prepared as follows:

- If tipPred is equal to 1 and Tip_Weighting_Factor[tip_global_wtd_index] is equal to 16, refFrames[0] is set equal to ClosestPast and refFrames[1] is set equal to NONE.
- Otherwise, if tipPred is equal to 1, refFrames[0] is set equal to ClosestPast and refFrames[1] is set equal to ClosestFuture.
- Otherwise, if fromBuildTip is equal to 1 and CwpIdx is equal to 16, refFrames[0] is set equal to RefFrames[candRow][candCol][0] and refFrames[1] is set equal to NONE.
- Otherwise, refFrames[i] is set equal to RefFrames[candRow][candCol][i] for i = 0..1.

The constant table Tip_Weighting_Factor is defined as:

```
Tip_Weighting_Factor[ 8 ] = { 8, 12, 16, 18, 20, 4, 6, -4 }
```

The variable BlockInterp (giving the interpolation filter to be used by the predict subblock process) is set equal to InterpFilters[candRow][candCol].

The variable subX is set equal to (plane > 0) ? SubsamplingX : 0.

The variable subY is set equal to (plane > 0) ? SubsamplingY : 0.

The variable isCompound (equal to 1 if two inter predictions will be prepared, equal to 0 if only a single inter prediction will be prepared) is prepared as follows:

- If sub8x8Inter is equal to 1, isCompound is set equal to 0.
- Otherwise, if plane is greater than 0 and tipPred is equal to 0 and fromBuildTip is equal to 0 and is_thin_4xn_nx4_block() is equal to 1, isCompound is set equal to 0.

- Otherwise, isCompound is set equal to is_inter_ref_frame(refFrames[1]).

NOTE: Inter intra prediction only requires a single prediction so has isCompound equal to 0.

The variable LumaUseOptflowRefinement (specifying if the luma plane uses optical flow refinement) is set as follows:

```
if ( tipPred ) {
    LumaUseOptflowRefinement = opfl_refine_type != REFINE_NONE &&
        Tip_Weighting_Factor[ tip_global_wtd_index ] == CWP_EQUAL &&
        opfl_allowed_for_refs( refFrames ) && enable_tip_refinemv
    if ( enable_tip_refinemv ? ( w << subX ) == 256 && ( h << subY ) == 256 :
        ( w << subX ) >= 16 && ( h << subY ) >= 16 ) {
        tipSize = BLOCK_16X16
        LumaUseOptflowRefinement = 0
    } else {
        tipSize = BLOCK_8X8
    }
} else if ( isCompound && opfl_allowed_for_refs( RefFrame ) ) {
    LumaUseOptflowRefinement = use_optflow
} else {
    LumaUseOptflowRefinement = 0
}
```

The variable useOptflowRefinement (specifying if the current plane uses optical flow refinement) is set as follows:

```
if ( tipPred || fromBuildTip ) {
    useOptflowRefinement = ( plane == 0 ) && LumaUseOptflowRefinement
} else {
    useOptflowRefinement = LumaUseOptflowRefinement
}
```

The variable useRefinemv (specifying if the prediction uses motion vector refinement) is specified as follows:

```
if ( tipPred ) {
    useRefinemv = NumFutureRefs > 0 && NumPastRefs > 0 &&
        enable_refinemv && enable_tip_refinemv
} else if ( fromBuildTip ) {
    useRefinemv = 0
} else {
    useRefinemv = use_refinemv
}
```

NOTE: The variable useRefinemv means that the predict refinemv process will be invoked. However, this does not necessarily mean that the motion vector search is used. The search is only used if the input useSearch to the predict refinemv process is true.

If plane is equal to 0, the warp parameters are prepared as follows:

- If motion_mode is equal to LOCALWARP, the warp estimation process in [§ 7.13.3.23 Warp estimation process](#) is invoked with 0 as input.
- If motion_mode is equal to LOCALWARP and isCompound is equal to 1, the warp estimation process in [§ 7.13.3.23 Warp estimation process](#) is invoked with 1 as input.

- If motion_mode is equal to EXTENDWARP, the warp estimation process in [§ 7.13.3.24 Extend warp estimation process](#) is invoked with BlockMvs[0] as input.

The block is predicted in parts as follows:

```
if (tipPred) {
    sw = Block_Width[ tipSize ] >> subX
    sh = Block_Height[ tipSize ] >> subY
    for( i = 0; i < h; i += sh ) {
        for( j = 0; j < w; j += sw ) {
            predict_tip( plane, x + j, y + i, j, i, sw, sh, refFrames,
                        useRefinemv, useOptflowRefinement )
        }
    }
} else if (useRefinemv) {
    tw = Min(w, 16 >> subX)
    th = Min(h, 16 >> subY)
    for( i = 0; i < h; i += th ) {
        for( j = 0; j < w; j += tw ) {
            predict_refinemv( plane, x + j, y + i, j, i, tw, th,
                            Mvs[ candRow ][ candCol ], refFrames,
                            useOptflowRefinement, useSearch=1, tipPred=0 )
        }
    }
} else {
    mvs = Mvs[ candRow ][ candCol ]
    useRefArea = fromBuildTip && plane > 0 && enable_tip_refinemv &&
                NumFutureRefs > 0 && NumPastRefs > 0 &&
                (LumaUseOptflowRefinement || TipInterpFilter == EIGHTTAP_SHARP)
    if ( useRefArea ) {
        get_ref_area(plane,x,y,w,h,mvs,refFrames)
    }
    predict_block( plane, x, y, w, h, 0, 0, mvs, refFrames, isCompound,
                  useRefinemv=0, useOptflowRefinement, tipPred=0, fromBuildTip,
                  useRefArea )
}
```

The function call to predict_tip indicates that the predict [TIP](#) process specified in [§ 7.13.3.2 Predict TIP process](#) is invoked.

The function call to predict_refinemv indicates that the predict refine mv specified in [§ 7.13.3.3 Predict refine mv process](#) is invoked.

The function call to predict_block indicates that the predict block process specified in [§ 7.13.3.7 Predict block process](#) is invoked.

If use_bawp is equal to 1 and plane == 0 || use_bawp_chroma, the block adaptive weighted prediction process in [§ 7.13.3.25 Block adaptive weighted prediction process](#) is invoked with plane, x, y, w, h, BlockMvs[0], and 0 as inputs.

If plane is equal to 0 and use_intrabc is equal to 1 and morph_pred is equal to 1, the build morphological prediction process specified in [§ 7.13.3.26 Build morphological prediction process](#) is invoked with x, y, w, h, Mvs[candRow][candCol][0] as inputs.

§ 7.13.3.2. Predict [TIP](#) process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,

- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- a variable j specifying the x offset of the subblock within the inter predicted block,
- a variable i specifying the y offset of the subblock within the inter predicted block,
- variables w and h specifying the width and height of the region to be predicted,
- an array refFrames of the references to use for prediction,
- a variable useRefinemv specifying if refined motion vectors are being used,
- a variable useOptflowRefinement specifying if optical flow refinement has been used.

The [TIP](#) motion vector is prepared as follows:

```
subX = ( plane > 0 ) ? SubsamplingX : 0
subY = ( plane > 0 ) ? SubsamplingY : 0
lumaRow = y >> ( 2 - subY )
lumaCol = x >> ( 2 - subX )
candMvs = get_tip_cand( lumaRow , lumaCol )
```

Then the block is predicted as follows:

```
if ( useRefinemv ) {
    useSearch = enable_refinemv && is_refinemv_allowed_reference( refFrames )
    predict_refinemv( plane, x, y, j, i, w, h, candMvs, refFrames,
                    useOptflowRefinement, useSearch, tipPred = 1 )
} else {
    if ( plane == 0 ) {
        for ( i2 = i; i2 < i + h; i2 += MI_SIZE ) {
            for ( j2 = j; j2 < j + w; j2 += MI_SIZE ) {
                RefineMvs[ i2 >> 2 ][ j2 >> 2 ] = candMvs
            }
        }
    }
    predict_block( plane, x, y, w, h, j, i, candMvs, refFrames,
                  isCompound = refFrames[1] != NONE, useRefinemv = 0,
                  useOptflowRefinement, tipPred = 1, fromBuildTip = 0,
                  useRefArea = 0 )
}
```

The function call to predict_refinemv indicates that the predict refine mv specified in [§ 7.13.3.3 Predict refine mv process](#) is invoked.

The function call to predict_block indicates that the predict block process specified in [§ 7.13.3.7 Predict block process](#) is invoked.

§ 7.13.3.3. Predict refine mv process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- a variable j specifying the x offset of the subblock within the inter predicted block,
- a variable i specifying the y offset of the subblock within the inter predicted block,

- variables w and h specifying the width and height of the region to be predicted,
- an array candMvs of the starting motion vectors,
- an array refFrames of the references to use for prediction,
- a variable useOptflowRefinement specifying if optical flow refinement has been used,
- a variable useSearch specifying if a search for the best motion vector is done,
- a variable tipPred specifying if this block uses [TIP](#).

The variable useRefArea is set as follows:

```
if ( tipPred ) {
    if ( plane == 0 ) {
        useRefArea = useSearch
    } else {
        useRefArea = ( useSearch || LumaUseOptflowRefinement )
    }
} else {
    useRefArea = 1
}
```

If useRefArea is equal to 1, the get ref area process in [§ 7.13.3.4 Get ref area process](#) is invoked with plane, x, y, w, h, candMvs, refFrames as inputs.

The refined motion vectors offsetMvs are prepared as follows:

```
if (plane == 0) {
    if (useSearch) {
        (dx,dy) = search_refinemv(x, y, w, h, tipPred, candMvs, refFrames)
    } else {
        dx = 0
        dy = 0
    }
    offsetMvs = offset_refinemv(candMvs, dx, dy)
    for (i2 = i; i2 < i + h; i2 += MI_SIZE) {
        for (j2 = j; j2 < j + w; j2 += MI_SIZE) {
            RefineMvs[ i2 >> 2 ][ j2 >> 2 ] = offsetMvs
        }
    }
} else {
    offsetMvs = RefineMvs[i >> (2 - SubsamplingY)][j >> (2 - SubsamplingX)]
}
```

The function call to search_refinemv indicates that the search refine mv process in [§ 7.13.3.6 Search refine mv process](#) is invoked.

The function offset_refinemv adds the offset to a motion vector as follows:

```
offset_refinemv(srcMvs, dx, dy) {
    dstMvs[ 0 ][ 0 ] = srcMvs[ 0 ][ 0 ] + dy * 8
    dstMvs[ 0 ][ 1 ] = srcMvs[ 0 ][ 1 ] + dx * 8
    dstMvs[ 1 ][ 0 ] = srcMvs[ 1 ][ 0 ] - dy * 8
    dstMvs[ 1 ][ 1 ] = srcMvs[ 1 ][ 1 ] - dx * 8
    return dstMvs
}
```

Then the predict block process specified in [§ 7.13.3.7 Predict block process](#) is invoked with plane, x, y, w, h, j, i, offsetMvs, refFrames, isCompound equal to 1, useRefinemv equal to 1, useOptflowRefinement, tipPred, fromBuildTip equal to 0, and useRefArea as inputs.

§ 7.13.3.4. Get ref area process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- variables w and h specifying the width and height of the region to be predicted,
- an array candMvs of the starting motion vectors,
- an array refFrames of the references to use for prediction.

The get ref area single process specified in [§ 7.13.3.5 Get ref area single process](#) is invoked with plane, x, y, w, h, candMvs, refFrames, refList equal to 0 as inputs.

If is_inter_ref_frame(refFrames[1]) is equal to 1, the get ref area single process specified in [§ 7.13.3.5 Get ref area single process](#) is invoked with plane, x, y, w, h, candMvs, refFrames, refList equal to 1 as inputs.

§ 7.13.3.5. Get ref area single process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- variables w and h specifying the width and height of the region to be predicted,
- an array candMvs of the starting motion vectors,
- an array refFrames of the references to use for prediction,
- a variable refList specifying which reference list is being predicted.

Variables specifying the allowed reference area are prepared as follows:

```
subX = ( plane > 0 ) ? SubsamplingX : 0
subY = ( plane > 0 ) ? SubsamplingY : 0
refIdx = ref_frame_idx[ refFrames[refList] ]
(startX, startY, stepX, stepY) = motion_vector_scaling( plane, refIdx, x, y,
                                                         candMvs[ refList ], 0 )

lastX = ( (RefMiCols[ refIdx ] * MI_SIZE ) >> subX ) - 1
lastY = ( (RefMiRows[ refIdx ] * MI_SIZE ) >> subY ) - 1
if ( w == 4 ) {
    RefFirstX[refList] = Clip3( 0, lastX, (startX >> 10) - 1 )
    RefLastX[refList] = Clip3( 0, lastX,
                              ( (startX + stepX * (w - 1) ) >> 10 ) + 2 )
} else {
    RefFirstX[refList] = Clip3( 0, lastX, (startX >> 10) - 3 )
    RefLastX[refList] = Clip3( 0, lastX,
                              ( (startX + stepX * (w - 1) ) >> 10 ) + 4 )
}
if ( h == 4 ) {
    RefFirstY[refList] = Clip3( 0, lastY, (startY >> 10) - 1 )
```

```

    RefLastY[refList] = Clip3( 0, lastY,
                              ( (startY + stepY * (h - 1) ) >> 10 ) + 2 )
} else {
    RefFirstY[refList] = Clip3( 0, lastY, (startY >> 10) - 3 )
    RefLastY[refList] = Clip3( 0, lastY,
                              ( (startY + stepY * (h - 1) ) >> 10 ) + 4 )
}

```

The function call to `motion_vector_scaling` indicates that the motion vector scaling process in [§7.13.3.17 Motion vector scaling process](#) is invoked.

§ 7.13.3.6. Search refine mv process

The inputs to this process are:

- variables `x` and `y` specifying the location of the top left sample in the `CurrFrame[ plane ]` array of the region to be predicted,
- variables `w` and `h` specifying the width and height of the region to be predicted,
- a variable `tipPred` specifying if this block uses [TIP](#),
- an array `candMvs` of the starting motion vectors,
- an array `refFrames` of the references to use for prediction.

The process searches for an appropriate integer offset to apply to the motion vectors.

The output of the process is the chosen offset.

For `i = 0..1`, for `comp = 0..1`, the following ordered steps (which detect if applying the offsets to the motion vector would cause an overflow) apply:

1. The variable `t` is set equal to `candMvs[ i ][ comp ]`.
2. If `t - 4 * 8` is less than `MV_LOW + 1` or `t + 2 * 8` is greater than `MV_UPP - 1`, the process immediately terminates with outputs of 0 and 0.

The size of the region is expanded by 2 samples in all directions as follows:

```

x -= 2
y -= 2
w += 4
h += 4

```

The variable `allowCentre` (specifying if the central position corresponding to no offset is searched) is set equal to `tipPred || !is_switchable_refinermv()`.

The variables `bestDy`, `bestDx`, and `bestSad` are set equal to 0.

The variable `th` (specifying a threshold value) is set equal to `(w * h) << 1`.

If `allowCentre` is equal to 1, the following ordered steps apply:

1. The `sad_refinermv` function specified below is invoked with `x`, `y`, `w`, `h`, 0, 0, `candMvs`, `refFrames` as inputs, and the output is assigned to `bestSad`.
2. `bestSad` is set equal to `bestSad - (bestSad >> 3)`.

3. If bestSad is less than th, the process immediately terminates with outputs of 0 and 0.

The positions are searched as follows:

```
for( idx = 0; idx < 24; idx++) {
    tryDy = Refinemv_Neighbors[ idx ][ 0 ]
    tryDx = Refinemv_Neighbors[ idx ][ 1 ]
    sad = sad_refinemv(x, y, w, h, tryDx, tryDy, candMvs, refFrames)
    if ( (idx == 0 && !allowCentre) || sad < bestSad ) {
        bestDy = tryDy
        bestDx = tryDx
        bestSad = sad
    }
}
```

The outputs of this process are bestDx and bestDy.

The constant table Refinemv_Neighbors (containing the search locations) is specified as:

```
Refinemv_Neighbors[ 24 ][ 2 ] = {
    { -2, -2 }, { -2, -1 }, { -2, 0 }, { -2, 1 }, { -2, 2 }, { -1, -2 },
    { -1, -1 }, { -1, 0 }, { -1, 1 }, { -1, 2 }, { 0, -2 }, { 0, -1 },
    { 0, 1 }, { 0, 2 }, { 1, -2 }, { 1, -1 }, { 1, 0 }, { 1, 1 },
    { 1, 2 }, { 2, -2 }, { 2, -1 }, { 2, 0 }, { 2, 1 }, { 2, 2 }
}
```

The function get_sad (which computes the sum of absolute differences between two predictions with optional downsampling) is specified as:

```
get_sad(w, h, ds) {
    sad = 0
    for (i = 0; i < h; i += 1 + ds) {
        for (j = 0; j < w; j++) {
            sad += Abs(Clip1(Preds[0][i][j]) - Clip1(Preds[1][i][j]))
        }
    }
    return sad
}
```

The function sad_refinemv (which computes the sum of absolute values for a specific offset) is specified as:

```
sad_refinemv(x, y, w, h, dx, dy, candMvs, refFrames) {
    mvs = offset_refinemv(candMvs, dx, dy)
    make_inter_predictions(x, y, w, h, mvs, refFrames, 1)
    return get_sad(w, h, 1) >> (BitDepth - 8)
}
```

The function call to make_inter_predictions indicates that the make inter predictions process specified in [§7.13.3.13 Make inter predictions process](#) is invoked.

§ 7.13.3.7. Predict block process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,

- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- variables w and h specifying the width and height of the region to be predicted,
- a variable j specifying the x offset of the subblock within the inter predicted block,
- a variable i specifying the y offset of the subblock within the inter predicted block,
- an array mvs of the motion vectors to use for prediction,
- an array refFrames of the references to use for prediction,
- a variable isCompound specifying if two inter predictions are required,
- a variable useRefinemv specifying if refined motion vectors are being used,
- a variable useOptflowRefinement specifying if optical flow refinement has been used,
- a variable tipPred specifying if this block uses [TIP](#),
- a variable fromBuildTip specifying if the prediction process is called from the build [TIP](#) process,
- a variable useRefArea specifying if the prediction is to be clipped to sample only from within a reference area.

If plane is equal to 0 and useOptflowRefinement is equal to 1, the array OpflMvs is filled in with the original value of the motion vector and MvDeltas is cleared as follows:

```
for(i2=0;i2<h;i2+=4) {
    for(j2=0;j2<w;j2+=4) {
        for(list=0;list<2;list++) {
            for(comp=0;comp<2;comp++) {
                OpflMvs[(i2 + i)>>2][(j2 + j)>>2][list][comp] =
                    mvs[list][comp] * 2
                MvDeltas[(i2 + i)>>2][(j2 + j)>>2][list][comp] = 0
            }
        }
    }
}
```

The reference area for chroma blocks is prepared (when necessary) as follows:

```
if ( plane > 0 &&
    !useOptflowRefinement && LumaUseOptflowRefinement &&
    (tipPred || fromBuildTip) && !useRefArea ) {
    get_ref_area(plane,x,y,w,h,mvs,refFrames)
    useRefArea = 1
}
```

The block is predicted as follows:

- If useOptflowRefinement is equal to 1, the predict optflow block process specified in [§ 7.13.3.8 Predict optflow block process](#) is invoked with plane, x, y, w, h, j, i, mvs, refFrames, useRefinemv, tipPred, fromBuildTip, useRefArea as inputs.
- Otherwise, if plane is greater than 0 and LumaUseOptflowRefinement is equal to 1 and either tipPred is equal to 1 or fromBuildTip is equal to 1, the predict subblock process specified in [§ 7.13.3.14 Predict subblock process](#) is invoked with plane, x, y, w, h, OpflMvs[i >> (2 - SubsamplingY)][j >> (2 - SubsamplingX)], prescaled equal to 1, refFrames, isCompound, useRefinemv, useOptflowRefinement equal to 0, tipPred, fromBuildTip, useRefArea as inputs.

- Otherwise, the predict subblock process specified in [§ 7.13.3.14 Predict subblock process](#) is invoked with plane, x, y, w, h, mvsv, prescaled equal to 0, refFrames, isCompound, useRefinmv, useOptflowRefinement equal to 0, tipPred, fromBuildTip, useRefArea as inputs.

§ 7.13.3.8. Predict optflow block process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- variables w and h specifying the width and height of the region to be predicted,
- a variable j specifying the x offset of the subblock within the inter predicted block,
- a variable i specifying the y offset of the subblock within the inter predicted block,
- an array mvsv of the motion vectors to use for prediction,
- an array refFrames of the references to use for prediction,
- a variable useRefinmv specifying if refined motion vectors are being used,
- a variable tipPred specifying if this block uses [TIP](#),
- a variable fromBuildTip specifying if the prediction process is called from the build [TIP](#) process,
- a variable useRefArea specifying if the prediction is to be clipped to sample only from within a reference area.

If plane is equal to 0, the make inter predictions process specified in [§ 7.13.3.13 Make inter predictions process](#) is invoked with x, y, w, h, mvsv, refFrames, useRefArea as input.

If tipPred is equal to 1 or fromBuildTip is equal to 1 (in these cases plane will always be equal to 0), the following ordered steps apply:

1. the variable sad is set equal to `get_sad(w, h, 0) >> (BitDepth - 8)`.
2. the variable sadThresh is set equal to `TipFrameMode == TIP_FRAME_AS_OUTPUT ? 15 : 6`.
3. If sad is less than sadThresh, the following ordered steps apply:
 1. The predict subblock process specified in [§ 7.13.3.14 Predict subblock process](#) is invoked with plane, x, y, w, h, mvsv, prescaled equal to 0, refFrames, isCompound equal to 1, useRefinmv, useOptflowRefinement equal to 0, tipPred, fromBuildTip, useRefArea as inputs.
 2. This process immediately terminates.

The variables defining the size of the subblocks are prepared as follows:

```
subX = ( plane > 0 ) ? SubsamplingX : 0
subY = ( plane > 0 ) ? SubsamplingY : 0
use4x4 = (!tipPred && !fromBuildTip)
lumaN = ( (h << subY) <= 8 && (w << subX) <= 8 && use4x4 ) ? 4 : 8
sw = Max(4, lumaN >> subX)
sh = Max(4, lumaN >> subY)
```

If plane is equal to 0, the get optflow based mv process specified in [§ 7.13.3.9 Get optflow based mv process](#) is invoked with j, i, w, h, lumaN, mvs, and refFrames as inputs.

The block is then predicted out of subblocks of size sw by sh as follows:

```

if ( !useRefArea && tipPred && useRefinmv ) {
    get_ref_area( 0, x, y, w, h, mvs, refFrames )
    useRefArea = 1
}
if ( !useRefArea && fromBuildTip ) {
    get_ref_area( 0, x, y, w, h, mvs, refFrames )
    useRefArea = 1
}
setRefArea = !useRefArea && !fromBuildTip &&
    ( tipPred || plane > 0 || (sh == 8 && sw == 8) )
for ( i2 = 0; i2 < h; i2 += sh ) {
    for ( j2 = 0; j2 < w; j2 += sw ) {
        if ( setRefArea ) {
            get_ref_area( plane, x + j2, y + i2, sw, sh, mvs, refFrames )
        }
        for( refList = 0; refList < 2; refList++ ) {
            opflMvs[ refList ] = prepare_optflow_transl( plane, refList,
                                                         j + j2, i + i2 )
        }
        predict_subblock( plane, x + j2, y + i2, sw, sh, opflMvs,
                          prescaled=1, refFrames, isCompound=1, useRefinmv,
                          useOptflowRefinement=1, tipPred,
                          fromBuildTip, useRefArea || setRefArea )
    }
}

```

The function call to get_ref_area indicates that the get ref area process in [§ 7.13.3.4 Get ref area process](#) is invoked.

The function call to predict_subblock indicates that the predict subblock process specified in [§ 7.13.3.14 Predict subblock process](#) is invoked.

The prepare_optflow_transl function (which prepares the motion vector) is specified as:

```

prepare_optflow_transl(plane, refList, j, i) {
    subX = ( plane > 0 ) ? SubsamplingX : 0
    subY = ( plane > 0 ) ? SubsamplingY : 0
    r = i >> (2 - subY)
    c = j >> (2 - subX)
    return OpflMvs[ r ][ c ][ refList ]
}

```

§ 7.13.3.9. Get optflow based mv process

The inputs to this process are:

- a variable optX specifying the x offset of the subblock within the inter predicted block,
- a variable optY specifying the y offset of the subblock within the inter predicted block,
- variables w and h specifying the width and height of the subblock,
- a variable n specifying that the size of the optical flow blocks is n by n,
- an array mvs of the motion vectors to use for prediction,
- an array refFrames of the references to use for prediction.

The length 2 array dist is prepared as follows:

```
for( i = 0; i < 2; i++ ) {  
    dist[ i ] = get_relative_dist( OrderHint, OrderHints[ refFrames[ i ] ] )  
}
```

If dist[0] is equal to 0 or dist[1] is equal to 0, the process terminates immediately.

The distances are modified as follows (this reduces the size of the distances while preserving their ratio):

```
if ( Abs(dist[0]) == Abs(dist[1]) ) {  
    dist[0] = dist[0] < 0 ? -1 : 1  
    dist[1] = dist[1] < 0 ? -1 : 1  
} else if ( Abs(dist[0]) > Abs(dist[1]) ) {  
    dist[0] = dist[0] < 0 ? -2 : 2  
    dist[1] = dist[1] < 0 ? -1 : 1  
} else {  
    dist[0] = dist[0] < 0 ? -1 : 1  
    dist[1] = dist[1] < 0 ? -2 : 2  
}
```

The optflow difference process specified in [§ 7.13.3.10 Optflow difference process](#) is invoked with w, h, and dist as inputs, and the outputs are assigned to tmp and pDiff.

The compute gradient process specified in [§ 7.13.3.11 Compute gradient process](#) is invoked with w, h, and tmp as inputs, and the outputs are assigned to xGrad and yGrad.

The optical flow motion vectors are prepared as follows:

```
for( i = 0; i < h; i += n ) {  
    for( j = 0; j < w; j += n ) {  
        compute_opfl_mv(optX,optY,i,j,n,xGrad,yGrad,pDiff,dist,mvs)  
    }  
}
```

The function call to compute_opfl_mv indicates that the compute optflow motion vector process specified in [§ 7.13.3.12 Compute optflow motion vector process](#) is invoked.

§ 7.13.3.10. Optflow difference process

The inputs to this process are:

- variables w and h specifying the width and height of the subblock,
- an array dist containing the scaled order hint distances for each reference list.

The process clips and scales the predictions as follows:

```
for(i=0;i<h;i++) {  
    for(j=0;j<w;j++) {  
        src0 = Clip1(Preds[0][i][j])  
        src1 = Clip1(Preds[1][i][j])  
        tmp[i][j] = Round2Signed(dist[0] * src0 - dist[1] * src1, BitDepth - 8)  
        pDiff[i][j] = Round2Signed( src0 - src1, BitDepth - 8 )  
    }  
}
```

The outputs of this process are the 2D arrays tmp and pDiff.

§ 7.13.3.11. Compute gradient process

The inputs to this process are:

- variables w and h specifying the width and height of the subblock,
- an array tmp containing the scaled differences between the predicted samples from the two reference lists.

The arrays xGrad and yGrad (approximating the gradient of the values in tmp) are computed as follows:

```
for( i = 0; i < h; i++ ) {
    for( j = 0; j < w; j++ ) {
        jStart = (j >> OPFL_GRAD_UNIT_LOG2) << OPFL_GRAD_UNIT_LOG2
        jEnd = Min(jStart + OPFL_GRAD_UNIT,w) - 1
        iStart = (i >> OPFL_GRAD_UNIT_LOG2) << OPFL_GRAD_UNIT_LOG2
        iEnd = Min(iStart + OPFL_GRAD_UNIT,h) - 1
        jPrev = Max(j - 1, jStart)
        jPrev2 = Max(j - 2, jStart)
        jNext = Min(j + 1, jEnd)
        jNext2 = Min(j + 2, jEnd)
        temp = 42 * (tmp[i][jNext] - tmp[i][jPrev]) -
                5 * (tmp[i][jNext2] - tmp[i][jPrev2])
        if (j + 1 > jEnd || j - 1 < jStart) {
            temp = temp << 1
        }
        xGrad[i][j] = Round2Signed(temp,7)

        iPrev = Max(i - 1, iStart)
        iPrev2 = Max(i - 2, iStart)
        iNext = Min(i + 1, iEnd)
        iNext2 = Min(i + 2, iEnd)
        temp = 42 * (tmp[iNext][j] - tmp[iPrev][j]) -
                5 * (tmp[iNext2][j] - tmp[iPrev2][j])
        if (i + 1 > iEnd || i - 1 < iStart) {
            temp = temp << 1
        }
        yGrad[i][j] = Round2Signed(temp,7)
    }
}
```

The outputs of this process are xGrad and yGrad.

§ 7.13.3.12. Compute optflow motion vector process

The inputs to this process are:

- a variable optX specifying the x offset of the subblock within the inter predicted block,
- a variable optY specifying the y offset of the subblock within the inter predicted block,
- a variable iBase specifying the y offset of the optical flow block within the subblock,
- a variable jBase specifying the x offset of the optical flow block within the subblock,
- a variable n specifying that the size of the optical flow blocks is n by n,
- arrays xGrad and yGrad containing the estimated gradient vector at each sample within the subblock,
- an array pDiff containing the differences between predicted samples at each sample within the subblock,

- an array `dist` containing the scaled order hint distances for each reference list,
- an array `mvs` of the motion vectors to use for prediction.

The process prepares motion vectors in `OpflMvs` for a particular optical flow block of size `n` by `n` within the subblock. It also stores the delta from the original motion vector in `MvDeltas`.

Statistics about the correlations are gathered as follows:

```

su2 = 0
sv2 = 0
suv = 0
suw = 0
svw = 0
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        u = xGrad[iBase + i][jBase + j]
        v = yGrad[iBase + i][jBase + j]
        w = pDiff[iBase + i][jBase + j]
        su2 += u * u
        suv += u * v
        sv2 += v * v
        suw += u * w
        svw += v * w
    }
}
su2 += n * n
sv2 += n * n

```

The determinant of a matrix equation is computed as follows:

```

msbSu2 = 1 + GetMsb(su2)
msbSv2 = 1 + GetMsb(sv2)
msbSuv = 1 + GetMsb(Abs(suv))
msbSuw = 1 + GetMsb(Abs(suw))
msbSvw = 1 + GetMsb(Abs(svw))
maxMultMsb = Max(msbSu2 + msbSv2, Max( Max(msbSv2 + msbSuw, msbSuv + msbSvw),
                                         Max(msbSu2 + msbSvw, msbSuv + msbSuw) ))
redbit = Max(0, maxMultMsb - MAX_LS_BITS + 3) >> 1
su2 = Round2Signed(su2, redbit)
sv2 = Round2Signed(sv2, redbit)
suv = Round2Signed(suv, redbit)
suw = Round2Signed(suw, redbit)
svw = Round2Signed(svw, redbit)
det = su2 * sv2 - suv * suw

```

If the determinant `det` is less than or equal to 0, this process immediately terminates.

The matrix equation is solved and the results stored as follows:

```

bits = MV_REFINE_PREC_BITS - 1
sol[0] = sv2 * suw - suv * svw
sol[1] = su2 * svw - suv * suw
sol = divide_and_round_array(sol, det, bits)
vx0 = -sol[0]
vy0 = -sol[1]
vx1 = vx0 * dist[1]
vy1 = vy0 * dist[1]
vx0 = vx0 * dist[0]
vy0 = vy0 * dist[0]

mvDelta[0][0] = Clip3(-OPFL_MV_DELTA_LIMIT, OPFL_MV_DELTA_LIMIT, vy0)
mvDelta[0][1] = Clip3(-OPFL_MV_DELTA_LIMIT, OPFL_MV_DELTA_LIMIT, vx0)

```

```

mvDelta[1][0] = Clip3(-OPFL_MV_DELTA_LIMIT, OPFL_MV_DELTA_LIMIT, vy1)
mvDelta[1][1] = Clip3(-OPFL_MV_DELTA_LIMIT, OPFL_MV_DELTA_LIMIT, vx1)
for(list=0; list<2; list++) {
    for(comp=0; comp<2; comp++) {
        MvDeltas[(optY + iBase)>>2][(optX + jBase)>>2][list][comp] =
            mvDelta[list][comp]
        newComp = mvs[list][comp] * 2 + mvDelta[list][comp]
        OpflMvs[(optY + iBase)>>2][(optX + jBase)>>2][list][comp] =
            Clip3( -(1<<17), (1<<17) - 1, newComp )
    }
}

```

where `divide_and_round_array` is defined as:

```

divide_and_round_array(sol, den, shift) {
    if (den == 1) {
        invDen = 1
        denShift = 0
    } else {
        (denShift, invDen) = resolve_divisor(den)
    }
    invDenMsb = GetMsb(invDen)
    for (i = 0; i < 2; i++) {
        result[i] = 0
        if (sol[i] != 0) {
            sgn = sol[i] > 0
            tmp = sgn ? sol[i] : -sol[i]
            numRedBits = Max(0, GetMsb(tmp) + invDenMsb + 4 - MAX_LS_BITS)
            if (numRedBits > 0)
                tmp = Round2Signed(tmp, numRedBits)
            incBits = shift + numRedBits - denShift
            if (incBits <= -31) {
                tmp = Round2Signed(tmp, -incBits - 30)
                mult = tmp * invDen
                tmp = Round2Signed(mult, 30)
            } else {
                mult = tmp * invDen
                if (incBits >= 0)
                    tmp = mult << incBits
                else
                    tmp = Round2Signed(mult, -incBits)
            }
            result[i] = sgn ? tmp : -tmp
        }
    }
    return result
}

```

§ 7.13.3.13. Make inter predictions process

The inputs to this process are:

- variables `x` and `y` specifying the location of the top left sample in the `CurrFrame[ plane ]` array of the region to be predicted,
- variables `w` and `h` specifying the width and height of the region to be predicted,
- an array `mvs` of the motion vectors to use for prediction,
- an array `refFrames` of the references to use for prediction,
- a variable `useRefArea` specifying whether to only use samples within the reference area.

The rounding variables derivation process specified in [§ 7.13.3.16 Rounding variables derivation process](#) is invoked with the input variable `isCompound` set equal to 0.

The process forms two inter predictions as follows:

```
for ( refList = 0; refList < 2; refList++ ) {
    refFrame = refFrames[ refList ]
    refIdx = ref_frame_idx[ refFrame ]
    (startX, startY, stepX, stepY) = motion_vector_scaling( plane = 0, refIdx,
                                                            x, y,
                                                            mvs[ refList ], 0 )
    block_inter_prediction( plane = 0, refList, refIdx, startX, startY,
                           stepX, stepY, w, h, useRefArea, BILINEAR )
}
```

The function call to `motion_vector_scaling` indicates that the motion vector scaling process in [§ 7.13.3.17 Motion vector scaling process](#) is invoked.

The function call to `block_inter_prediction` indicates that the block inter prediction process specified in [§ 7.13.3.18 Block inter prediction process](#) is invoked.

§ 7.13.3.14. Predict subblock process

The inputs to this process are:

- a variable `plane` specifying which plane is being predicted,
- variables `x` and `y` specifying the location of the top left sample in the `CurrFrame[ plane ]` array of the region to be predicted,
- variables `w` and `h` specifying the width and height of the region to be predicted,
- an array `candMvs` of the motion vectors to use for prediction,
- a variable `prescaled` specifying the precision of the motion vectors in `candMvs` (`prescaled` is equal to 0 for 1/8 th precision, `prescaled` is equal to 1 for 1/16 th precision),
- an array `refFrames` of the references to use for prediction,
- a variable `isCompound` specifying if two inter predictions are required,
- a variable `useRefinemv` specifying if refined motion vectors are being used,
- a variable `useOptflowRefinement` specifying if optical flow refinement has been used,
- a variable `tipPred` specifying if this block uses [TIP](#),
- a variable `fromBuildTip` specifying if the prediction process is called from the build [TIP](#) process,
- a variable `useRefArea` specifying if the prediction is to be clipped to sample only from within a reference area.

The rounding variables derivation process specified in [§ 7.13.3.16 Rounding variables derivation process](#) is invoked with the variable `isCompound` as input.

The save subpu size process specified in [§ 7.13.3.15 Save subpu size process](#) is invoked with `plane`, `x`, `y`, `w`, and `h` as inputs.

The prediction arrays are formed as follows:

```
for ( refList = 0; refList < ( isCompound ? 2 : 1 ); refList++ ) {
    refFrame = refFrames[ refList ]
    mv = candMvs[ refList ]
    if ( useRefinemv || useOptflowRefinement ||
```

```

        tipPred || fromBuildTip ||
        force_integer_mv )
    useWarp = 0
else if ( motion_mode == LOCALWARP ||
        motion_mode == EXTENDWARP ||
        motion_mode == DELTAWARP )
    useWarp = 1
else if ( ( YMode == GLOBALMV || YMode == GLOBAL_GLOBALMV ) &&
        GmType[ refFrame ] > IDENTITY &&
        Min(Block_Height[MiSize], Block_Width[MiSize]) >= 8 )
    useWarp = 2
else
    useWarp = 0

if ( use_intrabc == 0 ) {
    refIdx = ref_frame_idx[ refFrame ]
} else {
    refIdx = -1
    RefFrameWidth[ -1 ] = FrameWidth
    RefFrameHeight[ -1 ] = FrameHeight
}

(startX, startY, stepX, stepY) = motion_vector_scaling( plane, refIdx, x, y,
                                                         mv, prescaled )

if ( useWarp != 0 ) {
    if (useWarp == 1) {
        params = LocalWarpParams[ refList ]
    } else {
        params = gm_params[ refFrame ]
    }
    (shearValid, _, _, _, _) = setup_shear( params )
    skipPred = !shearValid || w < 8 || h < 8 || is_scaled( refFrame, 0 )
    for ( y8 = 0; y8 <= ((h-1) >> 3); y8++ ) {
        for ( x8 = 0; x8 <= ((w-1) >> 3); x8++ ) {
            block_warp( useWarp, params, plane, refList, x, y, y8, x8,
                        skipPred )
        }
    }
    if (skipPred) {
        for ( y4 = 0; y4 < (h >> 2); y4++ ) {
            for ( x4 = 0; x4 < (w >> 2); x4++ ) {
                ext_block_warp( params, plane, refList, x, y, y4, x4, w, h )
            }
        }
    }
} else {
    if (motion_mode >= LOCALWARP) {
        for ( y8 = 0; y8 <= ((h-1) >> 3); y8++ ) {
            for ( x8 = 0; x8 <= ((w-1) >> 3); x8++ ) {
                block_warp( 1, LocalWarpParams[refList], plane, refList,
                            x, y, y8, x8, 1 )
            }
        }
    }
    if ( fromBuildTip ) {
        interp = TipInterpFilter
    } else if ( tipPred || useOptflowRefinement || useRefinemv ) {
        interp = EIGHTTAP_SHARP
    } else {
        interp = BlockInterp
    }
    block_inter_prediction( plane, refList, refIdx, startX, startY,
                            stepX, stepY, w, h, useRefArea=useRefArea,
                            interp )
}
RefStartX[ refList ] = startX >> SCALE_SUBPEL_BITS
RefStartY[ refList ] = startY >> SCALE_SUBPEL_BITS
}

```

The function call to `motion_vector_scaling` indicates that the motion vector scaling process in [§ 7.13.3.17 Motion vector scaling process](#) is invoked.

The function call to `block_warp` indicates that the block warp process specified in [§ 7.13.3.19 Block warp process](#) is invoked.

The function call to `ext_block_warp` indicates that the extended block warp process specified in [§ 7.13.3.20 Extended block warp process](#) is invoked.

The function call to `block_inter_prediction` indicates that the block inter prediction process specified in [§ 7.13.3.18 Block inter prediction process](#) is invoked.

An array named `Mask` is prepared as follows:

- If `isCompound` is equal to 1 and `compound_type` is equal to `COMPOUND_WEDGE` and `plane` is equal to 0, the wedge mask process in [§ 7.13.3.27 Wedge mask process](#) is invoked with `w`, `h` as inputs.
- Otherwise, if `isCompound` is equal to 1 and `compound_type` is equal to `COMPOUND_DIFFWTD` and `plane` is equal to 0, the difference weight mask process in [§ 7.13.3.28 Difference weight mask process](#) is invoked with `w`, `h` as inputs.
- Otherwise, no mask array is needed.

The variable `cwpWeight` is set as follows:

- If `tipPred` is equal to 1, the variable `cwpWeight` is set equal to `Tip_Weighting_Factor[ tip_global_wtd_index ]`.
- Otherwise (`tipPred` is equal to 0), the variable `cwpWeight` is set equal to `CwpIdx`.

The variable `compoundWarp` is set as follows:

- If `YMode` is equal to `NEW_NEWMV` and `motion_mode` is equal to `LOCALWARP`, `compoundWarp` is set equal to 1.
- Otherwise (`YMode` is not equal to `NEW_NEWMV` or `motion_mode` is not equal to `LOCALWARP`), `compoundWarp` is set equal to 0.

The inter predicted samples are then derived as follows:

- If `isCompound` is equal to 0, `CurrFrame[ plane ][ y + i ][ x + j ]` is set equal to `Clip1( Preds[ 0 ][ i ][ j ] )` for `i = 0..h-1` and `j = 0..w-1`.
- Otherwise, if `compound_type` is equal to `COMPOUND_AVERAGE` and `enable_imp_msk_bld` is equal to 1 and `cwpWeight` is equal to `CWP_EQUAL` and `YMode` is not equal to `GLOBAL_GLOBALMV` and `is_scaled( refFrames[ 0 ], 0 )` is equal to 0 and `is_scaled( refFrames[ 1 ], 0 )` is equal to 0 and `compoundWarp` is equal to 0, `CurrFrame[ plane ][ y + i ][ x + j ]` is set equal to `Clip1( Round2( get_mask(plane,i,j) * Preds[ 0 ][ i ][ j ] + ( 2 - get_mask(plane,i,j) ) * Preds[ 1 ][ i ][ j ], 1 + InterPostRound ) )` for `i = 0..h-1` and `j = 0..w-1`.
- Otherwise, if `compound_type` is equal to `COMPOUND_AVERAGE`, `CurrFrame[ plane ][ y + i ][ x + j ]` is set equal to `Clip1( Round2( cwpWeight * Preds[ 0 ][ i ][ j ] + (16 - cwpWeight) * Preds[ 1 ][ i ][ j ], 4 + InterPostRound ) )` for `i = 0..h-1` and `j = 0..w-1`.
- Otherwise, the mask blend process in [§ 7.13.3.30 Mask blend process](#) is invoked with `plane`, `x`, `y`, `w`, `h` as inputs.

The `get_mask` function is defined as:

```
get_mask(plane,i,j) {
    subX = (plane > 0) ? SubsamplingX : 0
    subY = (plane > 0) ? SubsamplingY : 0
    lastX = (MiCols * MI_SIZE >> subX) - 1
    lastY = (MiRows * MI_SIZE >> subY) - 1
    refY0 = RefStartY[0] + i
    refY1 = RefStartY[1] + i
    refX0 = RefStartX[0] + j
    refX1 = RefStartX[1] + j
    ref0Onscreen = refX0 >= 0 && refX0 <= lastX && refY0 >= 0 && refY0 <= lastY
    ref1Onscreen = refX1 >= 0 && refX1 <= lastX && refY1 >= 0 && refY1 <= lastY
    if ( ref0Onscreen && !ref1Onscreen ) {
        m = 2
    } else if ( ref1Onscreen && !ref0Onscreen ) {
        m = 0
    } else {
        m = 1
    }
    return m
}
```

§ 7.13.3.15. Save subpu size process

The inputs to this process are:

- a variable `plane` specifying which plane is being predicted,
- variables `x` and `y` specifying the location of the top left sample in the `CurrFrame[ plane ]` array of the region to be predicted,
- variables `w` and `h` specifying the width and height of the region to be predicted.

If `w` is equal to `PuWidth` and `h` is equal to `PuHeight`, this process terminates immediately.

Otherwise, the size of the sub prediction unit (for use in deblocking filtering) is saved as follows:

```
subX = ( plane > 0 ) ? SubsamplingX : 0
subY = ( plane > 0 ) ? SubsamplingY : 0
subPuSz = find_tx_size(w, h)
lumaRow = y >> (2 - subY)
lumaCol = x >> (2 - subX)
for ( r = 0; r < h >> (MI_SIZE_LOG2 - subY); r++ ) {
    for ( c = 0; c < w >> (MI_SIZE_LOG2 - subX); c++ ) {
        SubPuColBase[plane > 0][lumaRow + r][lumaCol + c] = lumaCol
        SubPuRowBase[plane > 0][lumaRow + r][lumaCol + c] = lumaRow
        SubPuSize[plane > 0][lumaRow + r][lumaCol + c] = subPuSz
    }
}
```

§ 7.13.3.16. Rounding variables derivation process

The input to this process is a variable `isCompound`.

The rounding variables `InterRound0`, `InterRound1`, and `InterPostRound` are derived as follows:

- `InterRound0` (representing the amount to round by after horizontal filtering) is set equal to 3.
- `InterRound1` (representing the amount to round by after vertical filtering) is set equal to $(isCompound ? 7 : 11)$.

- InterPostRound (representing the amount to round by at the end of the prediction process) is set equal to $2 * \text{FILTER_BITS} - (\text{InterRound0} + \text{InterRound1})$.

NOTE: The rounding is chosen to ensure that the output of the horizontal filter always fits within 16 bits.

§ 7.13.3.17. Motion vector scaling process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- a variable refIdx specifying which reference frame is being used,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- a variable mv specifying the clamped motion vector,
- a variable prescaled specifying the precision of mv. (If prescaled is equal to 0, mv is in units of 1/8 th of a luma sample, i.e., with 3 fractional bits. Otherwise, mv is in units of 1/16 th of a luma sample.)

The outputs of this process are the variables startX and startY giving the reference block location in units of 1/1024 th of a sample, and variables stepX and stepY giving the step size in units of 1/1024 th of a sample.

This process is responsible for computing the sampling locations in the reference frame based on the motion vector. The sampling locations are also adjusted to compensate for any difference in the size of the reference frame compared to the current frame.

NOTE: When intra block copy is being used, refIdx will be equal to -1 to signal prediction from the frame currently being decoded. The arrays RefFrameWidth and RefFrameHeight include values at index -1 giving the dimensions of the current frame.

The variable xScale is set equal to $((\text{RefFrameWidth}[\text{refIdx}] \ll \text{REF_SCALE_SHIFT}) + (\text{FrameWidth} / 2)) / \text{FrameWidth}$.

The variable yScale is set equal to

$((\text{RefFrameHeight}[\text{refIdx}] \ll \text{REF_SCALE_SHIFT}) + (\text{FrameHeight} / 2)) / \text{FrameHeight}$.

(xScale and yScale specify the size of the reference frame relative to the current frame in units where $(1 \ll 14)$ is equivalent to both frames having the same size.)

The variables subX and subY are set equal to the subsampling for the current plane as follows:

- If plane is equal to 0, subX is set equal to 0 and subY is set equal to 0.
- Otherwise, subX is set equal to SubsamplingX and subY is set equal to SubsamplingY.

The variable halfSample (representing half the size of a sample in units of 1/16 th of a sample) is set equal to $(1 \ll (\text{SUBPEL_BITS} - 1))$.

The variables origX and origY are set as follows:

```
if ( prescaled ) {
    origX = ( (x << SUBPEL_BITS) + Round2Signed( mv[1], subX ) + halfSample )
    origY = ( (y << SUBPEL_BITS) + Round2Signed( mv[0], subY ) + halfSample )
} else {
    origX = ( (x << SUBPEL_BITS) + ( ( 2 * mv[1] ) >> subX ) + halfSample )
    origY = ( (y << SUBPEL_BITS) + ( ( 2 * mv[0] ) >> subY ) + halfSample )
}
```

(origX and origY specify the location of the centre of the sample at the top-left corner of the reference block in the current frame's coordinate system in units of 1/16 th of a sample, i.e., with SUBPEL_BITS=4 fractional bits.)

The variable baseX is set equal to $(\text{origX} * \text{xScale} - (\text{halfSample} \ll \text{REF_SCALE_SHIFT}))$.

The variable baseY is set equal to $(\text{origY} * \text{yScale} - (\text{halfSample} \ll \text{REF_SCALE_SHIFT}))$.

(baseX and baseY specify the location of the top-left corner of the block in the reference frame in the reference frame's coordinate system with 18 fractional bits.)

The variable off (containing a rounding offset for the filter tap selection) is set equal to $((1 \ll (\text{SCALE_SUBPEL_BITS} - \text{SUBPEL_BITS})) / 2)$.

The output variable startX is set equal to $(\text{Round2Signed}(\text{baseX}, \text{REF_SCALE_SHIFT} + \text{SUBPEL_BITS} - \text{SCALE_SUBPEL_BITS}) + \text{off})$.

The output variable startY is set equal to $(\text{Round2Signed}(\text{baseY}, \text{REF_SCALE_SHIFT} + \text{SUBPEL_BITS} - \text{SCALE_SUBPEL_BITS}) + \text{off})$.

(startX and startY specify the location of the top-left corner of the block in the reference frame in the reference frame's coordinate system with SCALE_SUBPEL_BITS=10 fractional bits.)

The output variable stepX is set equal to $\text{Round2Signed}(\text{xScale}, \text{REF_SCALE_SHIFT} - \text{SCALE_SUBPEL_BITS})$.

The output variable stepY is set equal to $\text{Round2Signed}(\text{yScale}, \text{REF_SCALE_SHIFT} - \text{SCALE_SUBPEL_BITS})$.

(stepX and stepY are the size of one current frame sample in the reference frame's coordinate system with 10 fractional bits.)

§ 7.13.3.18. Block inter prediction process

The inputs to this process are:

- a variable plane,
- a variable refList specifying which reference list is being predicted,
- a variable refIdx specifying which reference frame is being used (or -1 for intra block copy),
- variables x and y giving the block location in units of 1/1024 th of a sample,
- variables xStep and yStep giving the step size in units of 1/1024 th of a sample,
- variables w and h giving the width and height of the block in units of samples,

- a variable useRefArea specifying if the prediction is to be clipped to sample only from within a reference area,
- a variable interp specifying the interpolation filter to use.

The output from this process are updated values in the Preds[refList] array.

The variable ref specifying the reference frame contents is set as follows:

- If refIdx is equal to -1, ref is set equal to CurrFrame.
- Otherwise (refIdx is greater than or equal to 0), ref is set equal to FrameStore[refIdx].

The variables subX and subY are set equal to the subsampling for the current plane as follows:

- If plane is equal to 0, subX is set equal to 0 and subY is set equal to 0.
- Otherwise, subX is set equal to SubsamplingX and subY is set equal to SubsamplingY.

The variables firstX, firstY, lastX, lastY (giving the clipping region) are set as follows:

```
if ( useRefArea ) {
    firstX = RefFirstX[refList]
    firstY = RefFirstY[refList]
    lastX = RefLastX[refList]
    lastY = RefLastY[refList]
} else if ( use_intrabc ) {
    lastX = (MiCols * MI_SIZE >> subX) - 1
    lastY = (MiRows * MI_SIZE >> subY) - 1
    firstX = 0
    firstY = 0
} else {
    lastX = ( (RefMiCols[ refIdx ] * MI_SIZE) >> subX) - 1
    lastY = ( (RefMiRows[ refIdx ] * MI_SIZE) >> subY) - 1
    firstX = 0
    firstY = 0
}
```

The variable intermediateHeight specifying the height required for the intermediate array is set equal to $((h - 1) * yStep + (1 \ll \text{SCALE_SUBPEL_BITS}) - 1) \gg \text{SCALE_SUBPEL_BITS} + 8$.

The sub-sample interpolation is effected via two one-dimensional convolutions. First a horizontal filter is used to build up a temporary array, and then this array is vertically filtered to obtain the final prediction. The fractional parts of the motion vectors determine the filtering process. If the fractional part is zero, then the filtering is equivalent to a straight sample copy.

The filtering is applied as follows:

- The array intermediate is specified as follows:

```
interpFilter = interp
if ( w <= 4 ) {
    if ( interpFilter == EIGHTTAP || interpFilter == EIGHTTAP_SHARP ) {
        interpFilter = 4
    } else if ( interpFilter == EIGHTTAP_SMOOTH ) {
        interpFilter = 5
    }
}
for ( r = 0; r < intermediateHeight; r++ ) {
    for ( c = 0; c < w; c++ ) {
```

```

    s = 0
    p = x + xStep * c
    for ( t = 0; t < 8; t++ )
        s += Subpel_Filters[ interpFilter ][ (p >> 6) & SUBPEL_MASK ][ t ] *
            ref[ plane ][ Clip3( firstY, lastY, (y >> 10) + r - 3 ) ]
                [ Clip3( firstX, lastX, (p >> 10) + t - 3 ) ]
        intermediate[ r ][ c ] = Round2(s, InterRound0)
    }
}

```

- The array Preds is updated as follows:

```

interpFilter = interp
if ( h <= 4 ) {
    if ( interpFilter == EIGHTTAP || interpFilter == EIGHTTAP_SHARP ) {
        interpFilter = 4
    } else if ( interpFilter == EIGHTTAP_SMOOTH ) {
        interpFilter = 5
    }
}
for ( r = 0; r < h; r++ ) {
    for ( c = 0; c < w; c++ ) {
        s = 0
        p = (y & 1023) + yStep * r
        for ( t = 0; t < 8; t++ )
            s += Subpel_Filters[ interpFilter ][ (p >> 6) & SUBPEL_MASK ][ t ] *
                intermediate[ (p >> 10) + t ][ c ]
        Preds[ refList ][ r ][ c ] = Round2(s, InterRound1)
    }
}

```

where the constant array Subpel_Filters is specified as:

```

Subpel_Filters[ 6 ][ 16 ][ 8 ] = {
{
    { 0, 0, 0, 128, 0, 0, 0, 0 },
    { 0, 2, -6, 126, 8, -2, 0, 0 },
    { 0, 2, -10, 122, 18, -4, 0, 0 },
    { 0, 2, -12, 116, 28, -8, 2, 0 },
    { 0, 2, -14, 110, 38, -10, 2, 0 },
    { 0, 2, -14, 102, 48, -12, 2, 0 },
    { 0, 2, -16, 94, 58, -12, 2, 0 },
    { 0, 2, -14, 84, 66, -12, 2, 0 },
    { 0, 2, -14, 76, 76, -14, 2, 0 },
    { 0, 2, -12, 66, 84, -14, 2, 0 },
    { 0, 2, -12, 58, 94, -16, 2, 0 },
    { 0, 2, -12, 48, 102, -14, 2, 0 },
    { 0, 2, -10, 38, 110, -14, 2, 0 },
    { 0, 2, -8, 28, 116, -12, 2, 0 },
    { 0, 0, -4, 18, 122, -10, 2, 0 },
    { 0, 0, -2, 8, 126, -6, 2, 0 }
},
{
    { 0, 0, 0, 128, 0, 0, 0, 0 },
    { 0, 2, 28, 62, 34, 2, 0, 0 },
    { 0, 0, 26, 62, 36, 4, 0, 0 },
    { 0, 0, 22, 62, 40, 4, 0, 0 },
    { 0, 0, 20, 60, 42, 6, 0, 0 },
    { 0, 0, 18, 58, 44, 8, 0, 0 },
    { 0, 0, 16, 56, 46, 10, 0, 0 },
    { 0, -2, 16, 54, 48, 12, 0, 0 },
    { 0, -2, 14, 52, 52, 14, -2, 0 },
    { 0, 0, 12, 48, 54, 16, -2, 0 },
    { 0, 0, 10, 46, 56, 16, 0, 0 },
    { 0, 0, 8, 44, 58, 18, 0, 0 },
    { 0, 0, 6, 42, 60, 20, 0, 0 },
    { 0, 0, 4, 40, 62, 22, 0, 0 },
    { 0, 0, 4, 40, 62, 22, 0, 0 },
}
}

```



```

{ 0, 0, 4, 36, 62, 26, 0, 0 },
{ 0, 0, 2, 34, 62, 28, 2, 0 }
},
{
{ 0, 0, 0, 128, 0, 0, 0, 0 },
{ -2, 2, -6, 126, 8, -2, 2, 0 },
{ -2, 6, -12, 124, 16, -6, 4, -2 },
{ -2, 8, -18, 120, 26, -10, 6, -2 },
{ -4, 10, -22, 116, 38, -14, 6, -2 },
{ -4, 10, -22, 108, 48, -18, 8, -2 },
{ -4, 10, -24, 100, 60, -20, 8, -2 },
{ -4, 10, -24, 90, 70, -22, 10, -2 },
{ -4, 12, -24, 80, 80, -24, 12, -4 },
{ -2, 10, -22, 70, 90, -24, 10, -4 },
{ -2, 8, -20, 60, 100, -24, 10, -4 },
{ -2, 8, -18, 48, 108, -22, 10, -4 },
{ -2, 6, -14, 38, 116, -22, 10, -4 },
{ -2, 6, -10, 26, 120, -18, 8, -2 },
{ -2, 4, -6, 16, 124, -12, 6, -2 },
{ 0, 2, -2, 8, 126, -6, 2, -2 }
},
{
{ 0, 0, 0, 128, 0, 0, 0, 0 },
{ 0, 0, 0, 120, 8, 0, 0, 0 },
{ 0, 0, 0, 112, 16, 0, 0, 0 },
{ 0, 0, 0, 104, 24, 0, 0, 0 },
{ 0, 0, 0, 96, 32, 0, 0, 0 },
{ 0, 0, 0, 88, 40, 0, 0, 0 },
{ 0, 0, 0, 80, 48, 0, 0, 0 },
{ 0, 0, 0, 72, 56, 0, 0, 0 },
{ 0, 0, 0, 64, 64, 0, 0, 0 },
{ 0, 0, 0, 56, 72, 0, 0, 0 },
{ 0, 0, 0, 48, 80, 0, 0, 0 },
{ 0, 0, 0, 40, 88, 0, 0, 0 },
{ 0, 0, 0, 32, 96, 0, 0, 0 },
{ 0, 0, 0, 24, 104, 0, 0, 0 },
{ 0, 0, 0, 16, 112, 0, 0, 0 },
{ 0, 0, 0, 8, 120, 0, 0, 0 }
},
{
{ 0, 0, 0, 128, 0, 0, 0, 0 },
{ 0, 0, -4, 126, 8, -2, 0, 0 },
{ 0, 0, -8, 122, 18, -4, 0, 0 },
{ 0, 0, -10, 116, 28, -6, 0, 0 },
{ 0, 0, -12, 110, 38, -8, 0, 0 },
{ 0, 0, -12, 102, 48, -10, 0, 0 },
{ 0, 0, -14, 94, 58, -10, 0, 0 },
{ 0, 0, -12, 84, 66, -10, 0, 0 },
{ 0, 0, -12, 76, 76, -12, 0, 0 },
{ 0, 0, -10, 66, 84, -12, 0, 0 },
{ 0, 0, -10, 58, 94, -14, 0, 0 },
{ 0, 0, -10, 48, 102, -12, 0, 0 },
{ 0, 0, -8, 38, 110, -12, 0, 0 },
{ 0, 0, -6, 28, 116, -10, 0, 0 },
{ 0, 0, -4, 18, 122, -8, 0, 0 },
{ 0, 0, -2, 8, 126, -4, 0, 0 }
},
{
{ 0, 0, 0, 128, 0, 0, 0, 0 },
{ 0, 0, 30, 62, 34, 2, 0, 0 },
{ 0, 0, 26, 62, 36, 4, 0, 0 },
{ 0, 0, 22, 62, 40, 4, 0, 0 },
{ 0, 0, 20, 60, 42, 6, 0, 0 },
{ 0, 0, 18, 58, 44, 8, 0, 0 },
{ 0, 0, 16, 56, 46, 10, 0, 0 },
{ 0, 0, 14, 54, 48, 12, 0, 0 },
{ 0, 0, 12, 52, 52, 12, 0, 0 },
{ 0, 0, 12, 48, 54, 14, 0, 0 },
{ 0, 0, 10, 46, 56, 16, 0, 0 },
{ 0, 0, 8, 44, 58, 18, 0, 0 },
{ 0, 0, 6, 42, 60, 20, 0, 0 },

```

```

    { 0, 0, 4, 40, 62, 22, 0, 0 },
    { 0, 0, 4, 36, 62, 26, 0, 0 },
    { 0, 0, 2, 34, 62, 30, 0, 0 }
}

```

NOTE: All the values in Subpel_Filters are even. The last two filter types are used for small blocks and only have four filter taps. The filter at index 4 has a four tap version of the EIGHTTAP filter. The filter at index 5 has a four tap version of the EIGHTTAP_SMOOTH filter.

§ 7.13.3.19. Block warp process

The inputs to this process are:

- a variable useWarp (equal to 1 for local warp, or 2 for global warp),
- an array warpParams specifying the warp parameters,
- a variable plane,
- a variable refList specifying that the RefFrame[refList] is to be used by the process for prediction,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- variables i8 and j8 specifying the offset (in units of 8 samples) relative to the top left sample,
- a variable skipPred specifying if the prediction part is to be skipped.

The process updates a section of the SubMvs array with warped motion vectors.

Also, if skipPred is equal to 0, this process updates the array Preds[refList] containing warped inter predicted samples.

The process only updates a section of the Preds array. The size of the updated section is 8x8 samples, clipped to the size of the block. Variables i8 and j8 give the location of the section to update.

The variable refIdx specifying which reference frame is being used is set equal to ref_frame_idx[RefFrame[refList]].

The variable ref specifying the reference frame contents is set equal to FrameStore[refIdx].

The variables subX and subY are set equal to the subsampling for the current plane as follows:

- If plane is equal to 0, subX is set equal to 0 and subY is set equal to 0.
- Otherwise, subX is set equal to SubsamplingX and subY is set equal to SubsamplingY.

The variable firstX is set equal to 0.

The variable firstY is set equal to 0.

The variable lastX is set equal to ((RefMiCols[refIdx] * MI_SIZE) >> subX) - 1.

The variable lastY is set equal to ((RefMiRows[refIdx] * MI_SIZE) >> subY) - 1.

(firstX and firstY specify the coordinates of the top left sample of the bounding box.)

(lastX and lastY specify the coordinates of the bottom right sample of the bounding box.)

The variable srcX is set equal to $(x + j8 * 8 + 4) \ll \text{subX}$.

The variable srcY is set equal to $(y + i8 * 8 + 4) \ll \text{subY}$.

(srcX and srcY specify a location in the luma plane that will be projected using the warp parameters.)

The variable dstX is set equal to $\text{warpParams}[2] * \text{srcX} + \text{warpParams}[3] * \text{srcY} + \text{warpParams}[0]$.

The variable dstY is set equal to $\text{warpParams}[4] * \text{srcX} + \text{warpParams}[5] * \text{srcY} + \text{warpParams}[1]$.

(dstX and dstY specify the destination location in the luma plane using WARPEDMODEL_PREC_BITS bits of precision).

If plane is equal to 0 and useWarp is equal to 1, the warped motion vectors are saved in the SubMvs array as follows:

```
mv[0] = Round2Signed( dstY - (srcY << WARPEDMODEL_PREC_BITS),
                     WARPEDMODEL_PREC_BITS - 3)
mv[1] = Round2Signed( dstX - (srcX << WARPEDMODEL_PREC_BITS),
                     WARPEDMODEL_PREC_BITS - 3)
mv[0] = Clip3(MV_LOW + 1, MV_UPP - 1, mv[0])
mv[1] = Clip3(MV_LOW + 1, MV_UPP - 1, mv[1])
row = y >> MI_SIZE_LOG2
col = x >> MI_SIZE_LOG2
for( i = 0; i < 2; i++ ) {
    for( j = 0; j < 2; j++ ) {
        SubMvs[row + i8 * 2 + i][col + j8 * 2 + j][ refList ] = mv
    }
}
```

If skipPred is equal to 1, the process immediately terminates.

The setup shear process specified in [§ 7.13.3.21 Setup shear process](#) is invoked with warpParams as input, and the outputs are assigned to warpValid, alpha, beta, gamma, and delta. (warpValid will always be equal to 1 at this point.)

The sub-sample interpolation is effected via two one-dimensional convolutions. First a horizontal filter is used to build up an intermediate array, and then this array is vertically filtered to obtain the final prediction.

The filtering is applied as follows:

- The array intermediate is specified as follows:

```
x4 = dstX >> subX
y4 = dstY >> subY
ix4 = x4 >> WARPEDMODEL_PREC_BITS
sx4 = x4 & ((1 << WARPEDMODEL_PREC_BITS) - 1)
iy4 = y4 >> WARPEDMODEL_PREC_BITS
sy4 = y4 & ((1 << WARPEDMODEL_PREC_BITS) - 1)

for ( i1 = -7; i1 < 8; i1++ ) {
    for ( i2 = -4; i2 < 4; i2++ ) {
        sx = sx4 + alpha * i2 + beta * i1
        offs = Round2(sx, WARPEDDIFF_PREC_BITS) + 3 * WARPEDPIXEL_PREC_SHIFTS
        s = 0
        for ( i3 = 0; i3 < 8; i3++ ) {
```

```

        s += Warped_Filters[ offs ][ i3 ] *
            ref[ plane ][ Clip3( firstY, lastY, iy4 + i1 ) ]
                        [ Clip3( firstX, lastX, ix4 + i2 - 3 + i3 ) ]
    }
    intermediate[(i1 + 7)][(i2 + 4)] = Round2(s, InterRound0)
}
}

```

- The array Preds is specified as follows:

```

for ( i1 = -4; i1 < 4; i1++ ) {
    for ( i2 = -4; i2 < 4; i2++ ) {
        sy = sy4 + gamma * i2 + delta * i1
        offs = Round2(sy, WARPEDDIFF_PREC_BITS) + 3 * WARPEDPIXEL_PREC_SHIFTS
        s = 0
        for ( i3 = 0; i3 < 8; i3++ ) {
            s += Warped_Filters[offs][i3] *
                intermediate[(i1 + i3 + 4)][(i2 + 4)]
        }
        Preds[ refList ][ i8 * 8 + i1 + 4 ][ j8 * 8 + i2 + 4 ] = Round2(s, InterRound1)
    }
}

```

§ 7.13.3.20. Extended block warp process

The inputs to this process are:

- an array warpParams specifying the warp parameters,
- a variable plane,
- a variable refList specifying that RefFrame[refList] is to be used by the process for prediction,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- variables i4 and j4 specifying the offset (in units of 4 samples) relative to the top left sample,
- variables w and h giving the width and height of the block in units of samples.

This process updates the Preds array containing extended warp inter predicted samples.

The process only updates a section of the Preds array. The size of the updated section is 4x4 samples. Variables i4 and j4 give the location of the section to update.

The variable refIdx specifying which reference frame is being used is set equal to ref_frame_idx[RefFrame[refList]].

The variables subX and subY are set equal to the subsampling for the current plane as follows:

- If plane is equal to 0, subX is set equal to 0 and subY is set equal to 0.
- Otherwise, subX is set equal to SubsamplingX and subY is set equal to SubsamplingY.

The variable firstX is set equal to 0.

The variable firstY is set equal to 0.

The variable lastX is set equal to ((RefMiCols[refIdx] * MI_SIZE) >> subX) - 1.

The variable lastY is set equal to ((RefMiRows[refIdx] * MI_SIZE) >> subY) - 1.

The variable scaled is set equal to is_scaled(RefFrame[refList], 0).

The bounding box is modified as follows:

```
i8 = i4 >> 1
j8 = j4 >> 1
bboxW = Min(w, 8)
bboxH = Min(h, 8)
mv = get_sub_block_warp_mv( warpParams, plane, x + j8 * 8, y + i8 * 8,
                           bboxW, bboxH, 0 )
mv[ 0 ] = clamp_mv_row( mv[ 0 ] )
mv[ 1 ] = clamp_mv_col( mv[ 1 ] )
(startX, startY, stepX, stepY) = motion_vector_scaling( plane, refIdx,
                                                         x + j8 * 8,
                                                         y + i8 * 8, mv, 0 )

firstX = Clip3( 0, lastX, (startX >> 10) - 3 )
firstY = Clip3( 0, lastY, (startY >> 10) - 3 )
lastX = Clip3( 0, lastX, ((startX + stepX * (bboxW - 1)) >> 10) + 4 )
lastY = Clip3( 0, lastY, ((startY + stepY * (bboxH - 1)) >> 10) + 4 )
```

(firstX and firstY specify the coordinates of the top left sample of the bounding box.)

(lastX and lastY specify the coordinates of the bottom right sample of the bounding box.)

The variable srcX is set equal to (x + j4 * 4 + 2) << subX.

The variable srcY is set equal to (y + i4 * 4 + 2) << subY.

(srcX and srcY specify a location in the luma plane that will be projected using the warp parameters.)

The variable dstX is set equal to warpParams[2] * srcX + warpParams[3] * srcY + warpParams[0].

The variable dstY is set equal to warpParams[4] * srcX + warpParams[5] * srcY + warpParams[1].

(dstX and dstY specify the destination location in the luma plane using WARPEDMODEL_PREC_BITS bits of precision).

The sub-sample interpolation is effected via two one-dimensional convolutions. First a horizontal filter is used to build up an intermediate array, and then this array is vertically filtered to obtain the final prediction as follows:

```
x4 = dstX >> subX
y4 = dstY >> subY
if ( scaled ) {
    xScale = ( ( RefFrameWidth[ refIdx ] << REF_SCALE_SHIFT ) +
               ( FrameWidth / 2 ) ) / FrameWidth
    yScale = ( ( RefFrameHeight[ refIdx ] << REF_SCALE_SHIFT ) +
               ( FrameHeight / 2 ) ) / FrameHeight
    x4 -= 2 << WARPEDMODEL_PREC_BITS
    y4 -= 2 << WARPEDMODEL_PREC_BITS
    x4 = Round2Signed( x4 * xScale, REF_SCALE_SHIFT )
    y4 = Round2Signed( y4 * yScale, REF_SCALE_SHIFT )
    stepX = Round2Signed( xScale, REF_SCALE_SHIFT - SCALE_SUBPEL_BITS ) <<
            (WARPEDMODEL_PREC_BITS - SCALE_SUBPEL_BITS)
    stepY = Round2Signed( yScale, REF_SCALE_SHIFT - SCALE_SUBPEL_BITS ) <<
            (WARPEDMODEL_PREC_BITS - SCALE_SUBPEL_BITS)

    iy4 = y4 >> WARPEDMODEL_PREC_BITS
    sy4 = y4 & ( (1 << WARPEDMODEL_PREC_BITS) - 1 )
```

```

intermediateHeight = ( (y4 + stepY * 3 ) >> WARPEDMODEL_PREC_BITS ) - iy4 +
EXT_WARP_TAPS

for (k = 0; k < intermediateHeight; k++) {
    for (l = 0; l < 4; l++) {
        ix4 = (x4 + stepX * l) >> WARPEDMODEL_PREC_BITS
        sx4 = (x4 + stepX * l) & ((1 << WARPEDMODEL_PREC_BITS) - 1)
        offsX = Round2(sx4, EXT_WARP_ROUND_BITS)
        intX = ix4
        intY = iy4 + k - 2
        s = 0
        for (m = 0; m < EXT_WARP_TAPS; m++) {
            s += Ext_Warped_Filters[ offsX ][ m ] *
                FrameStore[ refIdx ][ plane ]
                    [ Clip3( firstY, lastY, intY ) ]
                    [ Clip3( firstX, lastX, intX - 2 + m ) ]
        }
        intermediate[ k ][ l ] = Round2( s, InterRound0 )
    }
}

for (l = 0; l < 4; l++) {
    for (k = 0; k < 4; k++) {
        iy4off = ( (y4 + stepY * k ) >> WARPEDMODEL_PREC_BITS ) - iy4
        sy4 = (y4 + stepY * k ) & ((1 << WARPEDMODEL_PREC_BITS) - 1)
        offsY = Round2(sy4, EXT_WARP_ROUND_BITS)
        s = 0
        for (m = 0; m < EXT_WARP_TAPS; m++) {
            s += Ext_Warped_Filters[ offsY ][ m ] *
                intermediate[ iy4off + m ][ l ]
        }
        Preds[ refList ][ i4 * 4 + k ][ j4 * 4 + l ] =
            Round2( s, InterRound1 )
    }
}
} else {
    ix4 = x4 >> WARPEDMODEL_PREC_BITS
    sx4 = x4 & ((1 << WARPEDMODEL_PREC_BITS) - 1)
    iy4 = y4 >> WARPEDMODEL_PREC_BITS
    sy4 = y4 & ((1 << WARPEDMODEL_PREC_BITS) - 1)
    offsX = Round2(sx4, EXT_WARP_ROUND_BITS)

    for (k = -4; k < 5; k++) {
        for (l = -2; l < 2; l++) {
            s = 0
            for (m = 0; m < EXT_WARP_TAPS; m++) {
                s += Ext_Warped_Filters[ offsX ][ m ] *
                    FrameStore[ refIdx ][ plane ]
                        [ Clip3( firstY, lastY, iy4 + k ) ]
                        [ Clip3( firstX, lastX, ix4 + l - 2 + m ) ]
            }
            intermediate[ (k + 4) ][ (l + 2) ] = Round2( s, InterRound0 )
        }
    }

    offsY = Round2(sy4, EXT_WARP_ROUND_BITS)
    for (k = -2; k < 2; k++) {
        for (l = -2; l < 2; l++) {
            s = 0
            for (m = 0; m < EXT_WARP_TAPS; m++) {
                s += Ext_Warped_Filters[offsY][m] *
                    intermediate[ (k + m + 2) ][ (l + 2) ]
            }
            Preds[ refList ][ i4 * 4 + k + 2 ][ j4 * 4 + l + 2 ] =
                Round2( s, InterRound1 )
        }
    }
}
}

```

NOTE: The difference between this and the block warp process is that extended warp predicts 4x4 blocks with fixed phase, while the block warp predicts 8x8 blocks with variable phase. This means that extended warp is equivalent to a translation, while block warp approximates an affine transformation.

§ 7.13.3.21. Setup shear process

The input to this process is an array warpParams representing an affine transformation.

The outputs of this process are the variable warpValid and variables alpha, beta, gamma, delta representing two shearing operations that combine to make the full affine transformation.

The variable maxValue is set equal to $32767 - (1 \ll (\text{WARP_PARAM_REDUCE_BITS} - 1))$.

The variable alpha0 is set equal to $\text{Clip3}(-32768, \text{maxValue}, \text{warpParams}[2] - (1 \ll \text{WARPEDMODEL_PREC_BITS}))$.

The variable beta0 is set equal to $\text{Clip3}(-32768, \text{maxValue}, \text{warpParams}[3])$.

The resolve divisor process specified in § 7.13.3.22 Resolve divisor process is invoked with warpParams[2] as input, and the outputs are assigned to divShift and divFactor.

The variable v is set equal to $(\text{warpParams}[4] \ll \text{WARPEDMODEL_PREC_BITS})$.

The variable gamma0 is set equal to $\text{Clip3}(-32768, \text{maxValue}, \text{Round2Signed}(v * \text{divFactor}, \text{divShift}))$.

The variable w is set equal to $(\text{warpParams}[3] * \text{warpParams}[4])$.

The variable delta0 is set equal to $\text{Clip3}(-32768, \text{maxValue}, \text{warpParams}[5] - \text{Round2Signed}(w * \text{divFactor}, \text{divShift}) - (1 \ll \text{WARPEDMODEL_PREC_BITS}))$.

The output variables alpha, beta, gamma, delta are set as follows:

```
alpha = Round2Signed( alpha0, WARP_PARAM_REDUCE_BITS ) << WARP_PARAM_REDUCE_BITS
beta  = Round2Signed( beta0,  WARP_PARAM_REDUCE_BITS ) << WARP_PARAM_REDUCE_BITS
gamma = Round2Signed( gamma0, WARP_PARAM_REDUCE_BITS ) << WARP_PARAM_REDUCE_BITS
delta = Round2Signed( delta0, WARP_PARAM_REDUCE_BITS ) << WARP_PARAM_REDUCE_BITS
```

The output warpValid is set as follows:

- If $4 * \text{Abs}(\text{alpha}) + 7 * \text{Abs}(\text{beta})$ is greater than or equal to $(3 \ll \text{WARPEDMODEL_PREC_BITS})$, warpValid is set equal to 0.
- If $4 * \text{Abs}(\text{gamma}) + 4 * \text{Abs}(\text{delta})$ is greater than or equal to $(3 \ll \text{WARPEDMODEL_PREC_BITS})$, warpValid is set equal to 0.
- Otherwise, warpValid is set equal to 1.

§ 7.13.3.22. Resolve divisor process

The input to this process is a variable d.

The outputs of this process are variables divShift and divFactor that can be used to perform an approximate division by d via multiplying by divFactor and shifting right by divShift.

The variable n (representing the location of the most significant bit in $\text{Abs}(d)$) is set equal to $\text{FloorLog2}(\text{Abs}(d))$.

The variable e is set equal to $\text{Abs}(d) - (1 \ll n)$.

The variable f is set as follows:

- If n is greater than DIV_LUT_BITS , f is set equal to $\text{Round2}(e, n - \text{DIV_LUT_BITS})$.
- Otherwise, f is set equal to $e \ll (\text{DIV_LUT_BITS} - n)$.

The output variable divShift is set equal to $(n + \text{DIV_LUT_PREC_BITS})$.

The output variable divFactor is set as follows:

- If d is less than 0, divFactor is set equal to $-\text{Div_Lut}[f]$.
- Otherwise, divFactor is set equal to $\text{Div_Lut}[f]$.

The lookup table Div_Lut is specified as:

```
Div_Lut[ DIV_LUT_NUM ] = {
    512, 508, 504, 500, 496, 493, 489, 485, 482, 478, 475, 471, 468, 465, 462,
    458, 455, 452, 449, 446, 443, 440, 437, 434, 431, 428, 426, 423, 420, 417,
    415, 412, 410, 407, 405, 402, 400, 397, 395, 392, 390, 388, 386, 383, 381,
    379, 377, 374, 372, 370, 368, 366, 364, 362, 360, 358, 356, 354, 352, 350,
    349, 347, 345, 343, 341, 340, 338, 336, 334, 333, 331, 329, 328, 326, 324,
    323, 321, 320, 318, 317, 315, 314, 312, 311, 309, 308, 306, 305, 303, 302,
    301, 299, 298, 297, 295, 294, 293, 291, 290, 289, 287, 286, 285, 284, 282,
    281, 280, 279, 278, 277, 275, 274, 273, 272, 271, 270, 269, 267, 266, 265,
    264, 263, 262, 261, 260, 259, 258, 257, 256
}
```

The function call to $\text{resolve_divisor}()$ indicates that the process defined in this sub-section is invoked.

§ 7.13.3.23. Warp estimation process

The input to this process is a variable ref specifying which set of candidate motion vectors to prepare.

This process produces the array LocalWarpParams based on NumSamples candidates in CandList by performing a least squares fit.

The find warp samples process in [§ 7.12.3 Find warp samples process](#) is invoked with ref as input.

A 2×2 matrix A , and two length 2 arrays Bx and By are constructed as follows:

```
for ( i = 0; i < 2; i++ ) {
    for ( j = 0; j < 2; j++ ) {
        A[i][j] = 0
    }
    Bx[i] = 0
    By[i] = 0
}
w4 = Num_4x4_Blocks_Wide[MiSize]
h4 = Num_4x4_Blocks_High[MiSize]
midY = MiRow * 4 + h4 * 2 - 1
midX = MiCol * 4 + w4 * 2 - 1
suy = midY * 8
sux = midX * 8
duy = suy + BlockMvs[ref][0]
```



```

dux = sux + BlockMvs[ref][1]
for ( i = 0; i < NumSamples[ ref ]; i++ ) {
    sy = CandList[ ref ][ i ][ 0 ] - suy
    sx = CandList[ ref ][ i ][ 1 ] - sux
    dy = CandList[ ref ][ i ][ 2 ] - duy
    dx = CandList[ ref ][ i ][ 3 ] - dux
    if ( Abs(sx - dx) < LS_MV_MAX && Abs(sy - dy) < LS_MV_MAX ) {
        A[0][0] += ls_product(sx, sx) + 8
        A[0][1] += ls_product(sx, sy) + 4
        A[1][1] += ls_product(sy, sy) + 8
        Bx[0] += ls_product(sx, dx) + 8
        Bx[1] += ls_product(sy, dx) + 4
        By[0] += ls_product(sx, dy) + 4
        By[1] += ls_product(sy, dy) + 8
    }
}

```

where `ls_product` is specified as:

```

ls_product(a, b) {
    return ( (a * b) >> 2) + (a + b)
}

```

NOTE: The matrix A is symmetric so entry `A[1][0]` is omitted.

The variable `det` (containing the determinant of the matrix A) is set equal to `A[0][0] * A[1][1] - A[0][1] * A[0][1]`.

If `det` is equal to 0, the local warp parameters in `LocalWarpParams` are derived as follows:

```

if ( det == 0 ) {
    for ( i = 2; i < 6; i++ ) {
        LocalWarpParams[ ref ][ i ] = ( i == 2 || i == 5 ) ?
                                         1 << WARPEDMODEL_PREC_BITS : 0
    }
    (LocalWarpParams[ref][0], LocalWarpParams[ref][1]) =
        get_warp_translation(LocalWarpParams[ref], ref)
}

```

If `det` is equal to 0, this process terminates immediately.

The resolve divisor process specified in [§ 7.13.3.22 Resolve divisor process](#) is invoked with `det` as input, and the outputs are assigned to `divShift` and `divFactor`.

The local warp parameters in `LocalWarpParams` are derived as follows:

```

divShift -= WARPEDMODEL_PREC_BITS
if ( divShift < 0 ) {
    divFactor = divFactor << (-divShift)
    divShift = 0
}
LocalWarpParams[ ref ][ 2 ] = diag( A[1][1] * Bx[0] - A[0][1] * Bx[1] )
LocalWarpParams[ ref ][ 3 ] = diag( -A[0][1] * Bx[0] + A[0][0] * Bx[1] )
LocalWarpParams[ ref ][ 4 ] = diag( A[1][1] * By[0] - A[0][1] * By[1] )
LocalWarpParams[ ref ][ 5 ] = diag( -A[0][1] * By[0] + A[0][0] * By[1] )
LocalWarpParams[ ref ] = reduce_warp_model(LocalWarpParams[ ref ])
(LocalWarpParams[ ref ][ 0 ], LocalWarpParams[ ref ][ 1 ]) =
    get_warp_translation( LocalWarpParams[ ref ], ref )

```

where diag is specified to divide and clamp using divFactor and divShift as follows:

```
diag(v) {
    return Clip3( INT32MIN, INT32MAX, Round2Signed(v * divFactor, divShift) )
}
```

The function get_warp_translation (which works out the required translation for the block) is specified as:

```
get_warp_translation(params, refList) {
    w4 = Num_4x4_Blocks_Wide[ MiSize ]
    h4 = Num_4x4_Blocks_High[ MiSize ]
    midY = MiRow * 4 + h4 * 2 - 1
    midX = MiCol * 4 + w4 * 2 - 1
    mvx = BlockMvs[ refList ][ 1 ]
    mvy = BlockMvs[ refList ][ 0 ]
    vx = mvx * (1 << (WARPEDMODEL_PREC_BITS - 3)) -
        (midX * (params[2] - (1 << WARPEDMODEL_PREC_BITS)) + midY * params[3])
    vy = mvy * (1 << (WARPEDMODEL_PREC_BITS - 3)) -
        (midX * params[4] + midY * (params[5] - (1 << WARPEDMODEL_PREC_BITS)))
    cx = Clip3( -WARPEDMODEL_TRANS_CLAMP,
        WARPEDMODEL_TRANS_CLAMP - (1 << WARP_PARAM_REDUCE_BITS), vx )
    cy = Clip3( -WARPEDMODEL_TRANS_CLAMP,
        WARPEDMODEL_TRANS_CLAMP - (1 << WARP_PARAM_REDUCE_BITS), vy )
    return (cx, cy)
}
```

The function reduce_warp_model (which clamps and reduces the precision of a warp model to be ready for use in the warp filter) is specified as:

```
reduce_warp_model( params ) {
    maxValue = (1 << (WARPEDMODEL_PREC_BITS - 1)) -
        (1 << WARP_PARAM_REDUCE_BITS)
    minValue = -maxValue
    reducedParams[0] = params[0]
    reducedParams[1] = params[1]
    for (i = 2; i < 6; i++) {
        offset = (i == 2 || i == 5) ? (1 << WARPEDMODEL_PREC_BITS) : 0
        original = params[i] - offset
        clamped = Clip3(minValue, maxValue, original)
        rounded = Round2Signed(clamped, WARP_PARAM_REDUCE_BITS) <<
            WARP_PARAM_REDUCE_BITS
        reducedParams[ i ] = rounded + offset
    }
    return reducedParams
}
```

§ 7.13.3.24. Extend warp estimation process

This process produces the array LocalWarpParams based on extending the warp parameters from a neighboring block with the motion vector for the current block.

The input to this process is the motion vector mv for the current block.

The extended warp parameters are computed in LocalWarpParams as follows:

```
deltaRow = RefStackRowOffset[RefMvIdx]
deltaCol = RefStackColOffset[RefMvIdx]
if ( deltaRow != -1 && deltaCol != -1 ) {
    deltaRow = ExtendDeltaRow
    deltaCol = ExtendDeltaCol
}
```

```

}
mvRow = MiRow + deltaRow
mvCol = MiCol + deltaCol
ref = RefFrame[ 0 ]
neighborRef = RefFrames[ mvRow ][ mvCol ][ 0 ] == ref ? 0 : 1
if ( MotionModes[ mvRow ][ mvCol ] >= LOCALWARP ) {
    params = WarpParams[ mvRow ][ mvCol ][ 0 ]
} else if ( is_global_mv_block( mvRow, mvCol, neighborRef ) ) {
    params = gm_params[RefFrames[ mvRow ][ mvCol ][ neighborRef ]]
} else {
    for( i = 0; i < 6; i++) {
        params[ i ] = Default_Warp_Params[ i ]
    }
    params[0] = Mvs[ mvRow ][ mvCol ][ neighborRef ][ 1 ] <<
        (WARPEDMODEL_PREC_BITS - 3)
    params[1] = Mvs[ mvRow ][ mvCol ][ neighborRef ][ 0 ] <<
        (WARPEDMODEL_PREC_BITS - 3)
}
w4 = Num_4x4_Blocks_Wide[MiSize]
h4 = Num_4x4_Blocks_High[MiSize]
midY = MiRow * 4 + h4 * 2 - 1
midX = MiCol * 4 + w4 * 2 - 1
mvx = mv[ 1 ]
mvy = mv[ 0 ]
projMidX = (midX << WARPEDMODEL_PREC_BITS) +
    (mvx << (WARPEDMODEL_PREC_BITS - 3))
projMidY = (midY << WARPEDMODEL_PREC_BITS) +
    (mvy << (WARPEDMODEL_PREC_BITS - 3))

neighborIsAbove = deltaRow == -1 && deltaCol >= 0
extendWarpParams[0] = 0
extendWarpParams[1] = 0
if (neighborIsAbove) {
    extendWarpParams[ 2 ] = params[ 2 ]
    extendWarpParams[ 4 ] = params[ 4 ]
    aboveX = midX
    aboveY = MiRow * 4 - 1
    projAboveX = params[ 2 ] * aboveX + params[ 3 ] * aboveY + params[ 0 ]
    projAboveY = params[ 4 ] * aboveX + params[ 5 ] * aboveY + params[ 1 ]
    extendWarpParams[ 3 ] = Round2( projMidX - projAboveX,
        Mi_Height_Log2[MiSize] + MI_SIZE_LOG2 - 1)
    extendWarpParams[ 5 ] = Round2( projMidY - projAboveY,
        Mi_Height_Log2[MiSize] + MI_SIZE_LOG2 - 1)
} else {
    extendWarpParams[ 3 ] = params[ 3 ]
    extendWarpParams[ 5 ] = params[ 5 ]
    leftX = MiCol * 4 - 1
    leftY = midY
    projLeftX = params[ 2 ] * leftX + params[ 3 ] * leftY + params[ 0 ]
    projLeftY = params[ 4 ] * leftX + params[ 5 ] * leftY + params[ 1 ]
    extendWarpParams[2] = Round2( projMidX - projLeftX,
        Mi_Width_Log2[MiSize] + MI_SIZE_LOG2 - 1)
    extendWarpParams[4] = Round2( projMidY - projLeftY,
        Mi_Width_Log2[MiSize] + MI_SIZE_LOG2 - 1)
}
LocalWarpParams[ 0 ] = reduce_warp_model( extendWarpParams )
(LocalWarpParams[ 0 ][ 0 ], LocalWarpParams[ 0 ][ 1 ]) =
    get_warp_translation( LocalWarpParams[ 0 ], 0 )

```

The function `is_global_mv_block` (which works out if a block used global warp) is specified as:

```

is_global_mv_block(mvRow, mvCol, mvList) {
    candMode = YModes[ mvRow ][ mvCol ]
    candSize = MiSizes[ PlaneStart ][ mvRow ][ mvCol ]
    return is_global_mv_cand( candMode, candSize,
        RefFrames[ mvRow ][ mvCol ][ mvList ] )
}

```

The function `is_global_mv_cand` (which works out if a given candidate block used global warp) is specified as:

```
is_global_mv_cand( candMode, candSize, candRef ) {
    large = ( Min( Block_Width[ candSize ], Block_Height[ candSize ] ) >= 8 )
    return ( candMode == GLOBALMV || candMode == GLOBAL_GLOBALMV ) &&
           GmType[ candRef ] > IDENTITY &&
           large
}
```

§ 7.13.3.25. Block adaptive weighted prediction process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- variables x and y specifying the location of the top left sample in the CurrFrame[0] array of the region to be adapted,
- variables w and h specifying the width and height of the block,
- an array mv specifying the motion vector for the block,
- a variable morphPred specifying whether to use the current frame as the reference.

The outputs of this process are modified inter predicted samples in the current frame CurrFrame.

This process adjusts the inter predicted samples for the current block to try and match adjustments required for the surrounding samples.

Variables describing the location of the block (refX and refY) in the reference frame and the size of the block that is within planeWidth and planeHeight (bw and bh) are derived as:

```
if ( plane == 0 ) {
    plane = 0
    subX = 0
    subY = 0
} else {
    subX = SubsamplingX
    subY = SubsamplingY
}
planeWidth = MiCols * MI_SIZE >> subX
planeHeight = MiRows * MI_SIZE >> subY
bw = Min(planeWidth - x, w)
bh = Min(planeHeight - y, h)
dy = to_fullmv( mv[0] )
dx = to_fullmv( mv[1] )
refY = ( MiRow * MI_SIZE + dy ) >> subY
refX = ( MiCol * MI_SIZE + dx ) >> subX
```

The reference prevFrame (specifying which frame to use for the reference template) is set as follows:

```
if ( morphPred ) {
    prevFrame = CurrFrame
} else {
    refIdx = ref_frame_idx[ RefFrame[ 0 ] ]
    prevFrame = FrameStore[ refIdx ]
}
```

It is a requirement of bitstream conformance that all the following are true whenever this process is invoked:

- refX is greater than or equal to 1.
- refY is greater than or equal to 1.
- refX + bw is less than or equal to planeWidth.
- refY + bh is less than or equal to planeHeight.

NOTE: This ensures that the samples needed from the reference block are within the frame.

The adaptation parameters are set as follows:

```
shift = 8
alpha = 1 << 8
beta = -(1 << 7)
sumX = 0
sumY = 0
sumXX = 0
sumXY = 0
count = 0
if (plane == 0) {
    bw2 = Min(16,bw)
    bh2 = Min(16,bh)
} else {
    bw2 = Min(8,bw)
    bh2 = Min(8,bh)
}
width = bw2 == 12 ? 8 : bw2
height = bh2 == 12 ? 8 : bh2
numUp = 0
numLeft = 0
if (AvailU && AvailL) {
    if (width == 16 && height == 16) {
        numUp = 16
        numLeft = 16
    } else if (width > 4 && height > 4) {
        numUp = 8
        numLeft = 8
    } else if (width < 16 && height < 16) {
        numUp = 4
        numLeft = 4
    } else if (width == 16) {
        numUp = 16
    } else {
        numLeft = 16
    }
} else if (AvailU) {
    numUp = width
} else if (AvailL) {
    numLeft = height
}
if (numUp > 0) {
    upStep = width / numUp
    for( i = upStep >> 1; i < width; i += upStep ) {
        recon = CurrFrame[plane][y - 1][x + i]
        ref = prevFrame[ plane ][refY - 1][refX + i]
        sumX += ref
        sumY += recon
        sumXY += ref * recon
        sumXX += ref * ref
    }
    count += numUp
}
```

```

if (numLeft > 0) {
    leftStep = height / numLeft
    for( i = leftStep >> 1; i < height; i+= leftStep ) {
        recon = CurrFrame[plane][y + i][x - 1]
        ref = prevFrame[ plane ][refY + i][refX - 1]
        sumX += ref
        sumY += recon
        sumXY += ref * recon
        sumXX += ref * ref
    }
    count += numLeft
}
if ( plane > 0 ) {
    alpha = BawpAlpha
    if ( count == 0 ) {
        alpha = 1 << 8
    }
} else if ( explicit_bawp && !morphPred ) {
    firstRefDist = Abs( get_relative_dist( OrderHints[ RefFrame[ 0 ] ],
                                           OrderHint ) )

    listIndex = (YMode == NEARMV) ? 0 :
                ( (YMode == NEWMV && use_amvd) ? 1 : 2 )

    scale = listIndex + 1
    if (firstRefDist > 4) {
        scale += 1
    }
    if (!explicit_bawp_scale) {
        scale = -scale
    }
    alpha = 256 + 16 * scale
} else if ( count > 0 ) {
    nor = sumXY - sumX * sumY / count
    der = sumXX - sumX * sumX / count
    if ( der != 0 && nor != 0 ) {
        alpha = resolve_division(nor, der, shift)
        if (alpha == 0) {
            alpha = 1 << shift
        }
    } else {
        alpha = 1 << shift
    }
}
if ( count > 0 ) {
    beta = ( (sumY << shift) - sumX * alpha ) / count
}
if ( plane == 0 && !morphPred ) {
    BawpAlpha = alpha
}

```

where the function `resolve_division(N, D, shift)` approximates the division $(N \ll \text{shift}) / D$ and is defined as:

```

resolve_division(N, D, shift) {
    signN = N < 0
    N = Abs(N)
    shiftN = FloorLog2(N)
    shiftD = FloorLog2(D)
    eD = D - (1 << shiftD)
    if (shiftD > DIV_LUT_BITS)
        fD = Round2(eD, shiftD - DIV_LUT_BITS)
    else
        fD = eD << (DIV_LUT_BITS - shiftD)
    if (shiftN > DIV_LUT_BITS)
        fN = Round2(N, shiftN - DIV_LUT_BITS)
    else
        fN = N << (DIV_LUT_BITS - shiftN)
    shiftAdd = shiftD - shiftN - shift
    if (shiftAdd <= 1) {

```

```

    shift0 = (DIV_LUT_PREC_BITS + DIV_LUT_BITS + shiftAdd)
    if ( shift0 >= 0 ) {
        ret = (Div_Lut[fD] * fN) >> shift0
    } else {
        ret = (2 << shift) - 1
    }
} else {
    ret = 0
}
ret = Min( (2 << shift) - 1, ret)
if (signN) ret = -ret
return ret
}

```

Finally the samples in the block are adjusted as follows:

```

for( i = 0 ; i < h ; i++ ) {
    for( j = 0 ; j < w ; j++ ) {
        orig = CurrFrame[ plane ][ y + i ][ x + j ]
        CurrFrame[ plane ][ y + i ][ x + j ] =
            Clip1( (orig * alpha + beta) >> shift )
    }
}

```

NOTE: This adjusts all the samples in the block, not just the samples within planeWidth and planeHeight.

NOTE: The default parameters of alpha equal to 256 and beta equal to -128 (used if the current block is at the top-left of a tile) will subtract 1 off every sample value.

§ 7.13.3.26. Build morphological prediction process

The inputs to this process are:

- variables x and y specifying the location of the top left sample in the CurrFrame[0] array of the region to be adapted,
- variables w and h specifying the width and height of the block,
- an array mv specifying the motion vector used for intra block copy.

The block adaptive weighted prediction process specified in [§ 7.13.3.25 Block adaptive weighted prediction process](#) is invoked with plane set equal to 0, x, y, w, h, mv, and morphPred set equal to 1 as inputs.

§ 7.13.3.27. Wedge mask process

The input to this process is:

- variables w and h specifying the width and height of the region to be predicted.

This process sets up a mask array for the luma samples.

The mask is specified as:

```
for ( i = 0; i < h; i++ ) {
    for ( j = 0; j < w; j++ ) {
        Mask[ i ][ j ] =
            WedgeMasks[ MiSize ][ wedge_sign ][ WedgeIndex ][ i ][ j ]
    }
}
```

where WedgeMasks is a fixed lookup table that is generated by the following function:

```
initialise_wedge_mask_table( ) {
    w = MASK_MASTER_SIZE
    h = MASK_MASTER_SIZE
    for( boundary = 0; boundary < WEDGE_BOUNDARY_TYPES; boundary++ ) {
        for( angle = 0; angle < WEDGE_ANGLES; angle++ ) {
            for( n = 0; n < h; n++ ) {
                y = ((n << 1) - h + 1) * Wedge_Sin_Lut[ angle ]
                for( m = 0; m < w; m++ ) {
                    d = ((m << 1) - w + 1) * Wedge_Cos_Lut[ angle ] + y
                    if ( boundary == WEDGE_BOUNDARY_SHARP ) {
                        d = d * 2
                    }
                    clamp_d = Clip3( -31, 31, d )
                    MasterMask[ boundary ][ angle ][ n ][ m ] =
                        (clamp_d >= 0 ? Pos_Dist_2_Bld_Weight[ clamp_d ]
                         : Neg_Dist_2_Bld_Weight[ -clamp_d ]) << 2
                }
            }
        }
    }

    for ( bsize = BLOCK_8X8; bsize < BLOCK_SIZES; bsize++ ) {
        if ( Wedge_Bits[ bsize ] > 0 ) {
            w = Block_Width[ bsize ]
            h = Block_Height[ bsize ]
            boundary = bsize <= BLOCK_16X16 ? WEDGE_BOUNDARY_SHARP
                : WEDGE_BOUNDARY_SMOOTH

            for( wedge = 0; wedge < WEDGE_TYPES; wedge++ ) {
                dir = Wedge_Codebook[ wedge ][ 0 ]
                xoff = MASK_MASTER_SIZE / 2 -
                    ((Wedge_Codebook[ wedge ][ 1 ] * w) >> 3)
                yoff = MASK_MASTER_SIZE / 2 -
                    ((Wedge_Codebook[ wedge ][ 2 ] * h) >> 3)
                flipSign = 0
                for ( i = 0; i < h; i++ ) {
                    for ( j = 0; j < w; j++ ) {
                        WedgeMasks[ bsize ][ flipSign ][ wedge ][ i ][ j ] =
                            MasterMask[ boundary ][ dir ][ yoff+i ][ xoff+j ]
                        WedgeMasks[ bsize ][ !flipSign ][ wedge ][ i ][ j ] =
                            64 - MasterMask[ boundary ][ dir ][ yoff+i ][ xoff+j ]
                    }
                }
            }
        }
    }
}
```

The lookup tables are defined as:

```
Wedge_Cos_Lut[WEDGE_ANGLES] = {
    4, 4, 4, 2, 2,
    0, -2, -2, -4, -4,
    -4, -4, -4, -2, -2,
    0, 2, 2, 4, 4
}
```



```

}

Wedge_Sin_Lut[WEDGE_ANGLES] = {
    0, -1, -2, -2, -4,
    -4, -4, -2, -2, -1,
    0, 1, 2, 2, 4,
    4, 4, 2, 2, 1
}

Pos_Dist_2_Bld_Weight[WEDGE_BLD_LUT_SIZE] = {
    8, 8, 9, 9, 10, 10, 11, 11, 12, 12, 12, 13, 13, 13, 14, 14,
    14, 14, 14, 15, 15, 15, 15, 15, 15, 15, 15, 16, 16, 16, 16
}

Neg_Dist_2_Bld_Weight[WEDGE_BLD_LUT_SIZE] = {
    8, 8, 7, 7, 6, 6, 5, 5, 4, 4, 4, 3, 3, 3, 2, 2,
    2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0
}

```

The Wedge_Codebook (which gives the direction and offset to the wedge for each wedge index) is defined as:

```

Wedge_Codebook[WEDGE_TYPES][3] = {
    { WEDGE_0, 5, 4 }, { WEDGE_0, 6, 4 }, { WEDGE_0, 7, 4 },
    { WEDGE_14, 4, 4 }, { WEDGE_14, 5, 4 }, { WEDGE_14, 6, 4 },
    { WEDGE_14, 7, 4 }, { WEDGE_27, 4, 4 }, { WEDGE_27, 5, 4 },
    { WEDGE_27, 6, 4 }, { WEDGE_27, 7, 4 }, { WEDGE_45, 4, 4 },
    { WEDGE_45, 5, 4 }, { WEDGE_45, 6, 4 }, { WEDGE_45, 7, 4 },
    { WEDGE_63, 4, 4 }, { WEDGE_63, 4, 3 }, { WEDGE_63, 4, 2 },
    { WEDGE_63, 4, 1 }, { WEDGE_90, 4, 3 }, { WEDGE_90, 4, 2 },
    { WEDGE_90, 4, 1 }, { WEDGE_117, 4, 4 }, { WEDGE_117, 4, 3 },
    { WEDGE_117, 4, 2 }, { WEDGE_117, 4, 1 }, { WEDGE_135, 4, 4 },
    { WEDGE_135, 3, 4 }, { WEDGE_135, 2, 4 }, { WEDGE_135, 1, 4 },
    { WEDGE_153, 4, 4 }, { WEDGE_153, 3, 4 }, { WEDGE_153, 2, 4 },
    { WEDGE_153, 1, 4 }, { WEDGE_166, 4, 4 }, { WEDGE_166, 3, 4 },
    { WEDGE_166, 2, 4 }, { WEDGE_166, 1, 4 }, { WEDGE_180, 3, 4 },
    { WEDGE_180, 2, 4 }, { WEDGE_180, 1, 4 }, { WEDGE_194, 3, 4 },
    { WEDGE_194, 2, 4 }, { WEDGE_194, 1, 4 }, { WEDGE_207, 3, 4 },
    { WEDGE_207, 2, 4 }, { WEDGE_207, 1, 4 }, { WEDGE_225, 3, 4 },
    { WEDGE_225, 2, 4 }, { WEDGE_225, 1, 4 }, { WEDGE_243, 4, 5 },
    { WEDGE_243, 4, 6 }, { WEDGE_243, 4, 7 }, { WEDGE_270, 4, 5 },
    { WEDGE_270, 4, 6 }, { WEDGE_270, 4, 7 }, { WEDGE_297, 4, 5 },
    { WEDGE_297, 4, 6 }, { WEDGE_297, 4, 7 }, { WEDGE_315, 5, 4 },
    { WEDGE_315, 6, 4 }, { WEDGE_315, 7, 4 }, { WEDGE_333, 5, 4 },
    { WEDGE_333, 6, 4 }, { WEDGE_333, 7, 4 }, { WEDGE_346, 5, 4 },
    { WEDGE_346, 6, 4 }, { WEDGE_346, 7, 4 }
}

```

§ 7.13.3.28. Difference weight mask process

The inputs to this process are variables *w* and *h* specifying the width and height of the region to be predicted.

This process prepares an array *Mask* containing the blending weights for the luma samples.

The process sets the array based on the difference between the two predictions as follows:

```

for ( i = 0; i < h; i++ ) {
    for ( j = 0; j < w; j++ ) {
        diff = Abs(Preds[ 0 ][ i ][ j ] - Preds[ 1 ][ i ][ j ])
        diff = Round2(diff, (BitDepth - 8) + InterPostRound)
        m = Clip3(0, 64, 38 + diff / 16)
        if ( mask_type )
            Mask[ i ][ j ] = 64 - m
    }
}

```

```

        else
            Mask[ i ][ j ] = m
    }
}

```

§ 7.13.3.29. Intra mode variant mask process

The input to this process is:

- variables w and h specifying the width and height of the region to be predicted.

This process prepares an array Mask containing the blending weights for the luma samples.

The process sets the array based on the mode used for [intra prediction](#) as follows:

```

sizeScale = 128 / Max( h, w )
for ( i = 0; i < h; i++ ) {
    for ( j = 0; j < w; j++ ) {
        if ( interintra_mode == II_V_PRED ) {
            Mask[ i ][ j ] = Ii_Weights_1d[ i * sizeScale ]
        } else if ( interintra_mode == II_H_PRED ) {
            Mask[ i ][ j ] = Ii_Weights_1d[ j * sizeScale ]
        } else if ( interintra_mode == II_SMOOTH_PRED ) {
            Mask[ i ][ j ] = Ii_Weights_1d[ Min( i, j ) * sizeScale ]
        } else {
            Mask[ i ][ j ] = 32
        }
    }
}

```

where the table Ii_Weights_1d is defined as:

```

Ii_Weights_1d[ 128 ] = {
    60, 58, 56, 54, 52, 50, 48, 47, 45, 44, 42, 41, 39, 38, 37, 35, 34, 33, 32,
    31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 22, 21, 20, 19, 19, 18, 18, 17, 16,
    16, 15, 15, 14, 14, 13, 13, 12, 12, 12, 12, 11, 11, 10, 10, 10, 9, 9, 9, 8,
    8, 8, 8, 7, 7, 7, 7, 6, 6, 6, 6, 6, 5, 5, 5, 5, 5, 4, 4,
    4, 4, 4, 4, 4, 4, 3, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2,
    2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
}

```

§ 7.13.3.30. Mask blend process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- variables dstX and dstY specifying the location of the top left sample in the CurrFrame[plane] array of the region to be predicted,
- variables w and h specifying the width and height of the region to be predicted.

The process combines two predictions according to the mask. It makes use of an array Mask containing the blending weights to apply (the weights are defined for the current plane samples if compound_type is equal to COMPOUND_INTRA, or the luma plane otherwise).

The variables subX and subY describing the subsampling of the current plane are derived as follows:

- If plane is equal to 0, subX and subY are set equal to 0.
- Otherwise (plane is not equal to 0), subX is set equal to SubsamplingX and subY is set equal to SubsamplingY.

The process is specified as follows:

```
for ( y = 0; y < h; y++ ) {
  for ( x = 0; x < w; x++ ) {
    if ( ( !subX ) ||
          ( inter_intra && !wedge_interintra ) ) {
      m = Mask[ y ][ x ]
    } else if ( !subY ) {
      m = Round2( Mask[ y ][ 2*x ] + Mask[ y ][ 2*x+1 ], 1 )
    } else {
      m = Round2( Mask[ 2*y ][ 2*x ] + Mask[ 2*y ][ 2*x+1 ] +
                  Mask[ 2*y+1 ][ 2*x ] + Mask[ 2*y+1 ][ 2*x+1 ], 2 )
    }
    if ( inter_intra ) {
      pred0 = IntraPred[ y ][ x ]
      pred1 = CurrFrame[ plane ][ y + dstY ][ x + dstX ]
      CurrFrame[ plane ][ y + dstY ][ x + dstX ] =
        Round2( m * pred0 + (64 - m) * pred1, 6 )
    } else {
      pred0 = Preds[ 0 ][ y ][ x ]
      pred1 = Preds[ 1 ][ y ][ x ]
      CurrFrame[ plane ][ y + dstY ][ x + dstX ] =
        Clip1( Round2( m * pred0 + (64 - m) * pred1, 6 + InterPostRound ) )
    }
  }
}
```

§ 7.13.4. Palette prediction process

The palette prediction process is invoked for palette coded intra blocks to predict a part of the block using the limited palette.

The inputs to this process are:

- variables startX and startY specifying the location of the top left sample in the CurrFrame[plane] array of the current transform block,
- variables x and y specifying the location in 4x4 units relative to the top left sample of the current transform block,
- a variable txSz specifying the size of the current transform block.

The outputs of this process are palette predicted samples in the current frame CurrFrame.

The variable w specifying the width of the [transform block](#) is set equal to Tx_Width[txSz].

The variable h specifying the height of the [transform block](#) is set equal to Tx_Height[txSz].

The current frame is updated as follows:

- CurrFrame[0][startY + i][startX + j] is set equal to palette_colors_y[ColorMapY[y * 4 + i][x * 4 + j]] for i = 0..h-1 and j = 0..w-1.

§ 7.13.5. Predict chroma from luma process

The chroma from luma process uses reconstructed luma samples to form a prediction for the chroma samples. The high frequencies are taken from the reconstructed luma samples and combined with DC predicted chroma samples.

The inputs to this process are:

- a variable plane (greater than zero) specifying which plane is being predicted,
- variables startX and startY specifying the location of the top left sample in the CurrFrame[plane] array of the current transform block,
- a variable txSz specifying the size of the current transform block.

The outputs of this process are modified chroma predicted samples in the current frame CurrFrame.

If cfl_index is equal to CFL_MULTI, the mhccp process specified in § 7.13.6 MHCCP process is invoked with plane, startX, startY, and txSz as inputs, and then this process immediately terminates.

The variable w specifying the width of the transform block is set equal to Tx_Width[txSz].

The variable h specifying the height of the [transform block](#) is set equal to Tx_Height[txSz].

The variable subX is set equal to SubsamplingX.

The variable subY is set equal to SubsamplingY.

The variable lumaAvg (with an estimate of the average luma value) is prepared as follows:

```
stepW = w > 32 ? 2 : 1
stepH = h > 32 ? 2 : 1
x = startX << subX
y = startY << subY
filterIdx = cfl_ds_filter_index
if (filterIdx == 3) {
    filterIdx = 0
}
lumaSum = 0
lumaCount = 0
for (i = 0; i < w; i++) {
    lumaAbove[ i ] = 0
}
for (j = 0; j < h; j++) {
    lumaLeft[ j ] = 0
}
if ( AvailUChroma ) {
    prevSample = (1 << (BitDepth - 1))
    log2SbH = Mi_Height_Log2[ SbSize ]
    sbTop = (MiRow >> log2SbH) << (log2SbH + MI_SIZE_LOG2)
    for (i = 0; i < w; i++) {
        t = 0
        if (i >= (MiCols * MI_SIZE - x) >> subX) {
            t = prevSample
        } else {
            for (dy = -subY; dy <= subY; dy++) {
                for (dx = -subX; dx <= subX; dx++) {
                    v = CurrFrame[ 0 ]
                        [ Max(sbTop - 1, y - 1 - subY + dy) ]
                        [ x + Max(0, (i << subX) + dx) ]
                    if (subX && subY) {
                        t += Cfl_Filters_420[filterIdx][dy + subY][dx + subX]*v
                    } else if (subX) {

```

```

        t += Cfl_Filters_422[filterIdx][dx + subX] * v
    } else {
        t += 8 * v
    }
}

}

}

}

if ( i % stepW == 0 ) {
    lumaSum += t
    lumaAbove[ i ] = t >> 3
}

prevSample = t
}
lumaCount += w / stepW
}

if ( AvailLChroma ) {
    prevSample = (1 << (BitDepth - 1))
    for( j = 0; j < h; j++ ) {
        t = 0
        if ( j >= (MiRows * MI_SIZE - y) >> subY ) {
            t = prevSample
        } else {
            for( dy = -subY; dy <= subY; dy++ ) {
                for( dx = -subX; dx <= subX; dx++ ) {
                    v = CurrFrame[ 0 ]
                        [ y + Max(0, (j << subY) + dy) ]
                        [ x - 1 - subX + dx ]

                    if (subX && subY) {
                        t += Cfl_Filters_420[filterIdx][dy + subY][dx + subX]*v
                    } else if (subX) {
                        t += Cfl_Filters_422[filterIdx][dx + subX] * v
                    } else {
                        t += 8 * v
                    }
                }
            }
        }

        if ( j % stepH == 0 ) {
            lumaSum += t
            lumaLeft[ j ] = t >> 3
        }

        prevSample = t
    }
    lumaCount += h / stepH
}

lumaAvg = 8 << (BitDepth - 1)
if (lumaCount > 0) {
    lumaAvg = Min( (8 << BitDepth) - 1, approx_divide( lumaSum, lumaCount ) )
}

```

where the constant tables Cfl_Filters_420 and Cfl_Filters_422 are defined as follows:

```

Cfl_Filters_420[ 3 ][ 3 ][ 3 ] = {
    {{0, 0, 0},
     {0, 2, 2},
     {0, 2, 2}},
    {{0, 0, 0},
     {1, 2, 1},
     {1, 2, 1}},
    {{0, 1, 0},
     {1, 4, 1},
     {0, 1, 0}}
}

Cfl_Filters_422[ 3 ][ 3 ] = {
    {0, 4, 4},
}

```

```

{2, 4, 2},
{0, 8, 0}
}

```

The variable `implicitAlpha` is prepared based on correlations between luma and chroma as follows:

```

implicitAlpha = 0
if (cfl_index == CFL_DERIVED_ALPHA) {
    count = 0
    sumX = 0
    sumY = 0
    sumXY = 0
    sumXX = 0
    if ( AvailUChroma && AvailLChroma ) {
        if (w > h * 2) {
            numLeft = 0
            numAbove = NUM_REF_SAM_CFL
        } else if (h > w * 2) {
            numAbove = 0
            numLeft = NUM_REF_SAM_CFL
        } else {
            numAbove = NUM_REF_SAM_CFL >> 1
            numLeft = NUM_REF_SAM_CFL >> 1
        }
    } else {
        numAbove = AvailUChroma ? NUM_REF_SAM_CFL : 0
        numLeft = AvailLChroma ? NUM_REF_SAM_CFL : 0
    }
    numAbove = Min(numAbove, w)
    numLeft = Min(numLeft, h)
    if (numAbove > 0) {
        step = w / numAbove
        prevSample = 1 << (BitDepth - 1)
        for ( i = 0; i < w; i++ ) {
            sample = prevSample
            if (startX + i < (MiCols * MI_SIZE >> subX) ) {
                sample = CurrFrame[ plane ][ startY - 1 ][ startX + i ]
            }
            samples[ i ] = sample
            prevSample = sample
        }
        for ( i = step >> 1; i < w; i += step ) {
            sample = samples[ i ]
            sumX += lumaAbove[ i ]
            sumY += sample
            sumXY += lumaAbove[ i ] * sample
            sumXX += lumaAbove[ i ] * lumaAbove[ i ]
            count++
        }
    }
    if (numLeft > 0) {
        step = h / numLeft
        prevSample = 1 << (BitDepth - 1)
        for ( j = 0; j < h; j++ ) {
            sample = prevSample
            if (j + startY < (MiRows * MI_SIZE >> subY) ) {
                sample = CurrFrame[ plane ][ startY + j ][ startX - 1 ]
            }
            samples[ j ] = sample
            prevSample = sample
        }
        for ( j = step >> 1; j < h; j += step ) {
            sample = samples[ j ]
            sumX += lumaLeft[ j ]
            sumY += sample
            sumXY += lumaLeft[ j ] * sample
            sumXX += lumaLeft[ j ] * lumaLeft[ j ]
            count++
        }
    }
}

```

```

    }
    if (count > 0) {
        der = sumXX - (sumX * sumX) / count
        nor = sumXY - (sumX * sumY) / count
        shift = 8
        if ( der != 0 && nor != 0 ) {
            implicitAlpha = resolve_division(nor, der, shift)
        }
    }
}

```

An array L (containing subsampled reconstructed luma samples with 3 fractional bits of precision) and lumaAvg (representing the average reconstructed luma intensity with 3 fractional bits of precision) is specified as:

```

for ( i = 0; i < h; i++ ) {
    lumaY = (startY + i) << subY
    clampY = i == 0 || lumaY % 64 == 0
    for ( j = 0; j < w; j++ ) {
        lumaX = (startX + j) << subX
        clampX = j == 0 || lumaX % 64 == 0
        t = 0
        for (dy = -subY; dy <= subY; dy++) {
            for (dx = -subX; dx <= subX; dx++) {
                v = CurrFrame[ 0 ]
                    [ lumaY + (clampY ? Max(dy, 0) : dy) ]
                    [ lumaX + (clampX ? Max(dx, 0) : dx) ]
                if (subX && subY) {
                    t += Cfl_Filters_420[filterIdx][dy + subY][dx + subX] * v
                } else if (subX) {
                    t += Cfl_Filters_422[filterIdx][dx + subX] * v
                } else {
                    t = 8 * v
                }
            }
        }
        L[ i ][ j ] = t
    }
}

```

The variable alpha is prepared depending on cfl_index as follows:

```

if ( cfl_index == CFL_DERIVED_ALPHA ) alpha = implicitAlpha
else if ( plane == 1 ) alpha = CflAlphaU * 32
else alpha = CflAlphaV * 32

```

The predicted chroma samples are specified as:

```

for ( i = 0; i < h; i++ ) {
    for ( j = 0; j < w; j++ ) {
        dc = CurrFrame[ plane ][ startY + i ][ startX + j ]
        scaledLuma = Round2Signed( alpha * ( L[ i ][ j ] - lumaAvg ), 11 )
        CurrFrame[ plane ][ startY + i ][ startX + j ] = Clip1(dc + scaledLuma)
    }
}

```

§ 7.13.6. MHCCP process

The inputs to this process are:

- a variable plane (greater than zero) specifying which plane is being predicted,

- variables startX and startY specifying the location of the top left sample in the CurrFrame[plane] array of the current transform block,
- a variable txSz specifying the size of the current transform block.

The outputs of this process are modified chroma predicted samples in the current frame CurrFrame.

The variable w specifying the width of the transform block is set equal to Tx_Width[txSz].

The variable h specifying the height of the transform block is set equal to Tx_Height[txSz].

The derive multi param process specified in [§ 7.13.7 Derive multi param process](#) is invoked and the output is assigned to multiParams.

The samples are predicted as follows:

```
vec[2] = 1 << (BitDepth - 1)
for ( i = 0; i < h; i++ ) {
    for ( j = 0; j < w; j++ ) {
        a = CflRef[0][CflAbove + i][CflLeft + j]
        if ( cfl_mh_dir == 0 ) {
            vec[0] = a
        } else if ( cfl_mh_dir == 1 ) {
            vec[0] = CflRef[0][Max(0,CflAbove + i - 1)][CflLeft + j]
        } else {
            vec[0] = CflRef[0][CflAbove + i][Max(0,CflLeft + j - 1)]
        }
        vec[1] = Round2( a * a, BitDepth )
        t = 0
        for( k = 0; k < 3; k++ ) {
            t += mul_fixed32_adapt(multiParams[k], vec[k], MHCCP_BITS)
        }
        CurrFrame[ plane ][ startY + i ][ startX + j ] = Clip1( t )
    }
}
```

where the function mul_fixed32_adapt (which performs multiplication and right shift with adjustments made to ensure arithmetic can work with 32 bit signed integers) is specified as:

```
mul_fixed32_adapt(a, b, shift) {
    bitsA = GetMsb( Abs( a ) ) + 1
    bitsB = GetMsb( Abs( b ) ) + 1
    need = Max( 0, bitsA + bitsB - 29 )
    s1 = need >> 1
    s2 = need - s1
    adj = shift - (s1 + s2)
    prod = ( a >> s1 ) * ( b >> s2 )
    if ( adj <= 0 ) {
        return prod
    } else if ( adj > 29 ) {
        return 0
    } else {
        return Round2Signed( prod, adj )
    }
}
```

§ 7.13.7. Derive multi param process

This process works out the best (in a least squares sense) parameters to use to predict the chroma samples from luma samples.

All elements of a 1d array b of length 3 are set equal to 0.

All elements of a 2d array ata of size 3 by 3 are set equal to 0.

Statistics about the reference samples are collected as follows:

```
v[2] = 1 << (BitDepth - 1)
count = 0
for (i = 1; i < (CflRefHeight >> SubsamplingY) - 1; i++) {
    for (j = 1; j < (CflRefWidth >> SubsamplingX) - 1; j++) {
        if ( i < CflAbove || j < CflLeft ) {
            if (cfl_mh_dir == 0) {
                v[0] = CflRef[0][i][j]
            } else if (cfl_mh_dir == 1) {
                v[0] = CflRef[0][i - 1][j]
            } else {
                v[0] = CflRef[0][i][j - 1]
            }
            v[1] = Round2( CflRef[0][i][j] * CflRef[0][i][j], BitDepth )
            target = CflRef[1][i][j]
            for (i0 = 0; i0 < 3; i0++) {
                for (i1 = i0; i1 < 3; i1++) {
                    ata[i0][i1] += v[i0] * v[i1]
                }
                b[i0] += v[i0] * target
            }
            count++
        }
    }
}
```

where cfl_ref_luma_avail (which decides if a reference sample is available) is specified as:

```
cfl_ref_luma_avail(i, j, w, h) {
    return (i < CflAbove || j < CflLeft + w) &&
        (i < CflAbove + h || j < CflLeft)
}
```

The array newParams is initialized as follows:

```
for( i = 0; i < 2; i++ ) {
    newParams[ i ] = 0
}
newParams[ 2 ] = 1 << MHCCP_BITS
```

If count is equal to 0, the output of the process is the array newParams and the process immediately terminates.

Otherwise (count is greater than 0), the ata and b are normalized as follows:

```
matrixShift = MHCCP_BITS + 6 - 2 * BitDepth - CeilLog2(count)
if (matrixShift > 0) {
    leftShift = matrixShift
    rightShift = 0
} else {
    leftShift = 0
    rightShift = -matrixShift
}
for (i0 = 0; i0 < 3; i0++) {
    for (i1 = i0; i1 < 3; i1++) {
        ata[i0][i1] = (ata[i0][i1] << leftShift) >> rightShift
    }
}
```

```

    }
    b[i0] = (b[i0] << leftShift) >> rightShift
}

```

The Gaussian elimination process specified in [§ 7.13.8 Gaussian elimination process](#) is invoked with `ata` and `b` as inputs, and the output is assigned to `newParams`.

The output of this process is the array `newParams`.

§ 7.13.8. Gaussian elimination process

The inputs to this process are:

- a 3x3 array `ata`,
- a length 3 array `b`.

The output of this process is the array `params`.

This process solves a matrix equation via Gaussian elimination (without pivoting) as follows:

```

for ( i = 0; i < 3; i++) {
    for ( j = 0; j < 3; j++) {
        c[i][j] = j >= i ? ata[i][j] : ata[j][i];
    }
    c[i][i] += 2 << (BitDepth - 8)
    c[i][3] = b[i]
}
for ( i = 0; i < 3 ; i++) {
    diag = Max(1, Abs(c[i][i]))
    (scale, shift) = get_division_scale_shift( diag )
    for ( j = i + 1; j < 4; j++) {
        c[i][j] = mul_fixed32_adapt( c[i][j], scale, shift )
    }
    for ( j = i + 1; j < 3; j++) {
        scaleFactor = c[j][i];
        for ( k = i + 1; k < 4; k++) {
            c[j][k] -= mul_fixed32_adapt(scaleFactor, c[i][k], MHCCP_BITS)
        }
    }
}
for( i = 0; i < 2; i++ ) {
    params[ i ] = 0
}
params[ 2 ] = c[ 2 ][ 3 ]
for( i = 2 ; i >= 0 ; i-- ) {
    params[ i ] = c[ i ][ 3 ]
    for( j = i + 1 ; j < 3; j++ ) {
        params[ i ] -= mul_fixed32_adapt(c[ i ][ j ], params[ j ], MHCCP_BITS)
    }
}

```

where the function `get_division_scale_shift` (which returns a scale and shift that can be used to approximate division by the input) is defined as:

```

get_division_scale_shift( denom ) {
    shift = FloorLog2(denom)
    normDiff = Clip3( 1, 32767,
        Round2(denom << DIV_PREC_BITS, shift) ) & ((1 << DIV_PREC_BITS) - 1)
    index = normDiff >> (DIV_PREC_BITS - DIV_SLOT_BITS)
    normDiff2 = normDiff - Division_Pow2_0[index]
    scale = ((Division_Pow2_W[index] *

```

```

        ((normDiff2*normDiff2) >> DIV_PREC_BITS)) >> DIV_PREC_BITS_POW2) -
        (normDiff2 >> 1) + Division_Pow2_B[index]
scale = scale << (MHCCP_BITS - DIV_PREC_BITS)
return (scale, shift)
}

```

where the constant tables Division_Pow2_O, Division_Pow2_B, and Division_Pow2_W are defined as:

```

Division_Pow2_W[DIV_PREC_BITS_POW2] = {
    214, 153, 113, 86, 67, 53, 43, 35
}

Division_Pow2_O[DIV_PREC_BITS_POW2] = {
    4822, 5952, 6624, 6792, 6408, 5424, 3792, 1466
}

Division_Pow2_B[DIV_PREC_BITS_POW2] = {
    12784, 12054, 11670, 11583, 11764, 12195, 12870, 13782
}

```

§ 7.14. Reconstruction and dequantization

§ 7.14.1. General

This section details the process of reconstructing a block of coefficients using [dequantization](#) and inverse transforms.

§ 7.14.2. Dequantization functions

This section defines the functions get_dc_quant and get_ac_quant that are needed by the [dequantization](#) process.

The quantization parameters are derived from lookup tables.

The function qllookup(q) is specified as:

```

qllookup( q ) {
    if (q < 25) {
        return Ac_Qlookup[q]
    } else {
        return Ac_Qlookup[((q - 1) % 24) + 1] << ((q - 1) / 24)
    }
}

```

where Ac_Qlookup is defined as follows:

```

Ac_Qlookup[25] = {
    64, 40, 41, 43, 44, 45, 47, 48, 49, 51, 52,
    54, 55, 57, 59, 60, 62, 64, 66, 68, 70, 72,
    74, 76, 78
}

```

The function get_q(qindex, delta) is specified as:

```

get_q( qindex, delta ) {
    if ((qindex == 0) && (delta <= 0)) {
        return Ac_Qlookup[0]
    }
}

```

```

qClamped = Clip3(1, MaxQ, qindex + delta)
return qllookup(qClamped)
}

```

The function `get_qindex( ignoreDeltaQ, segmentId )` returns the quantizer index for the current block and is specified by the following:

- If `seg_feature_active_idx( segmentId, SEG_LVL_ALT_Q )` is equal to 1, the following ordered steps apply:
 1. Set the variable data equal to `FeatureData[ segmentId ][ SEG_LVL_ALT_Q ]`.
 2. Set `qindex` equal to `base_q_idx + data`.
 3. If `ignoreDeltaQ` is equal to 0 and `delta_q_present` is equal to 1, set `qindex` equal to `CurrentQIndex + data`.
 4. Return `Clip3( 0, MaxQ, qindex )`.
- Otherwise, if `ignoreDeltaQ` is equal to 0 and `delta_q_present` is equal to 1, return `CurrentQIndex`.
- Otherwise, return `base_q_idx`.

NOTE: When using both delta quantization and lossless segments, care should be taken that `get_qindex` returns 0 for the lossless segments. One approach is to set `FeatureData[ segmentId ][ SEG_LVL_ALT_Q ]` to -255 for the lossless segments.

The function `get_dc_quant( plane )` returns the quantizer value for the dc coefficient for a particular plane and is derived as follows:

- If `plane` is equal to 0, return `get_q( get_qindex( 0, segment_id ), DeltaQYDc + BaseYDcDeltaQ )`.
- Otherwise, if `plane` is equal to 1, return `get_q( get_qindex( 0, segment_id ), DeltaQUdc + BaseUVDcDeltaQ )`.
- Otherwise (`plane` is equal to 2), return `get_q( get_qindex( 0, segment_id ), DeltaQVDc + BaseUVDcDeltaQ )`.

The function `get_ac_quant( plane )` returns the quantizer value for the ac coefficient for a particular plane and is derived as follows:

- If `plane` is equal to 0, return `get_q( get_qindex( 0, segment_id ), 0 )`.
- Otherwise, if `plane` is equal to 1, return `get_q( get_qindex( 0, segment_id ), DeltaQUAc + BaseUVAcDeltaQ )`.
- Otherwise (`plane` is equal to 2), return `get_q( get_qindex( 0, segment_id ), DeltaQVAc + BaseUVAcDeltaQ )`.

§ 7.14.3. Reconstruct process

The reconstruct process is invoked to perform [dequantization](#), [inverse transform](#) and [reconstruction](#). This process is triggered at a point defined by a function call to reconstruct in the transform block syntax table described in [§ 5.20.7.24 Transform block syntax](#).

The inputs to this process are:

- a variable plane specifying which plane is being reconstructed,
- variables x and y specifying the location of the top left sample in the CurrFrame[plane] array of the current transform block,
- a variable txSz specifying the size of the transform block.

The outputs of this process are reconstructed samples in the current frame CurrFrame.

The variable log2W (specifying the base 2 logarithm of the width of the transform block) is set equal to Tx_Width_Log2[txSz].

The variable log2H (specifying the base 2 logarithm of the height of the [transform block](#)) is set equal to Tx_Height_Log2[txSz].

The variable w (specifying the width of the [transform block](#)) is set equal to $1 \ll \log_2 W$.

The variable h (specifying the height of the [transform block](#)) is set equal to $1 \ll \log_2 H$.

The following ordered steps apply:

1. If plane is equal to 0 and sec_tx_type is not equal to 0, the secondary transform process as specified in [§ 7.15.3 Secondary transform process](#) is invoked with the variable txSz as input. This modifies the values in Dequant.
2. The 2D inverse transform block process as specified in [§ 7.15.4 2D inverse transform process](#) is invoked with the variables plane and txSz as inputs. The inverse transform outputs are stored in the Residual buffer.
3. For $i = 0..(h-1)$, for $j = 0..(w-1)$, CurrFrame[plane][y + i][x + j] is set equal to Clip1(CurrFrame[plane][y + i][x + j] + Residual[i][j]).

If Lossless is equal to 1, it is a requirement of bitstream conformance that the values written into the Residual array in step 2 are representable by a signed integer with $1 + \text{BitDepth}$ bits.

NOTE: This requirement applies to the final values written to the Residual array, i.e., after any DPCM adjustment.

§ 7.14.4. Dequantization process

The dequantization process is triggered at a point defined by a function call to dequant in the transform block syntax table described in [§ 5.20.7.24 Transform block syntax](#).

The inputs to this process are:

- a variable plane specifying which plane is being reconstructed,
- a variable txSz specifying the size of the transform block.

The process dequantizes coefficients from the Quant array and places the results in the Dequant array.

The variable tw is set equal to $\text{Min}(32, \text{Tx_Width}[\text{txSz}])$.

The variable th is set equal to $\text{Min}(32, \text{Tx_Height}[\text{txSz}])$.

The variables dqDenom, shift, useQm, segLvl, useUserQm, and useFsc are derived as follows:

```
pels = Tx_Width[ txSz ] * Tx_Height[ txSz ]
shift = (pels > 256) + (pels > 1024)
useFsc = enable_fsc && PlaneTxType == IDTX && plane == 0 &&
        (fsc_mode || is_inter)
if ( allow_tcq && plane == 0 && !Lossless &&
    get_tx_class(PlaneTxType) == TX_CLASS_2D && !useFsc ) {
    shift += 1
}
dqDenom = 1 << shift

if ( tw > 8 || th > 8 ) {
    if ( plane == 0 ) {
        segLvl = qm_y[ 0 ]
    } else if ( plane == 1 ) {
        segLvl = qm_u[ 0 ]
    } else {
        segLvl = qm_v[ 0 ]
    }
} else {
    segLvl = SegQMLLevel[ plane ][ segment_id ]
}
useQm = using_qmatrix == 1 && PlaneTxType < IDTX && segLvl < NUM_CUSTOM_QMS
useUserQm = useQm && tw <= 8 && th <= 8 && QmDataPresent[ segLvl ]
```

For $i = 0..(th-1)$, for $j = 0..(tw-1)$, the following ordered steps apply:

1. The variable q is derived as follows:

- If i is equal to 0 and j is equal to 0, the variable q is set equal to `get_dc_quant( plane )`.
- Otherwise (i, j or both are not equal to 0), the variable q is set equal to `get_ac_quant( plane )`.

2. The variable $q2$ is derived as follows:

- If $useQm$ is equal to 1, $q2$ is set as follows:

```
if ( useUserQm ) {
    if ( tw < th ) {
        m = UserQm[ segLvl ][ 2 ][ plane ][ i ][ j ]
    } else if ( tw > th ) {
        m = UserQm[ segLvl ][ 1 ][ plane ][ i ][ j ]
    } else {
        qi = i * 8 / th
        qj = j * 8 / tw
        m = UserQm[ segLvl ][ 0 ][ plane ][ qi ][ qj ]
    }
} else {
    m = Quantizer_Matrix[ segLvl ][ plane > 0 ]
        [ Qm_Offset[ txSz ] + i * tw + j ]
}
q2 = Round2( q * m, 5 )
```

- Otherwise, $q2$ is set equal to q .

3. The variable qc is set equal to `Quant[  $i * tw + j$  ]`.

4. The variable $sign$ is set equal to `(  $qc < 0$  ) ? -1 : 1`.

5. The variable $dqHigh$ is set equal to `Abs( $qc$ ) *  $q2$` .

6. The variable dq is set equal to `Round2( $dqHigh$  & 0xFFFFFFFF, QUANT_TABLE_BITS)`.

7. The variable $dq2$ is set equal to `sign * (  $dq / dqDenom$  )`.

8. Dequant[i][j] is set equal to Clip3(- (1 << (7 + BitDepth)), (1 << (7 + BitDepth)) - 1, dq2).

§ 7.14.5. Save dequant process

The save dequant process is triggered at a point defined by a function call to save_dequant in the transform block syntax table described in [§ 5.20.7.24 Transform block syntax](#).

The inputs to this process are:

- a variable plane specifying which plane is being reconstructed,
- a variable txSz specifying the size of the transform block.

The process saves the dequantized coefficients as follows:

```
tw = Min(32, Tx_Width[ txSz ])
th = Min(32, Tx_Height[ txSz ])
for( i = 0; i < th; i++ ) {
    for( j = 0; j < tw; j++ ) {
        SaveDequant[ plane ][ i ][ j ] = Dequant[ i ][ j ]
    }
}
```

§ 7.14.6. Get dequant process

The get dequant process is triggered at a point defined by a function call to get_dequant in the transform block syntax table described in [§ 5.20.7.24 Transform block syntax](#).

The inputs to this process are:

- a variable plane specifying which plane is being reconstructed,
- a variable txSz specifying the size of the transform block,
- a variable cctxType specifying the type of cross component transform.

The process computes the dequantized coefficients as follows:

```
tw = Min( 32, Tx_Width[ txSz ] )
th = Min( 32, Tx_Height[ txSz ] )
for( i = 0; i < th; i++ ) {
    for( j = 0; j < tw; j++ ) {
        if (cctxType == CCTX_NONE) {
            v = SaveDequant[ plane ][ i ][ j ]
        } else {
            angle = cctxType - 1
            if (plane == 1) {
                cU = Cctx_Mtx[ angle ][ 0 ]
                cV = -Cctx_Mtx[ angle ][ 1 ]
            } else {
                cU = Cctx_Mtx[ angle ][ 1 ]
                cV = Cctx_Mtx[ angle ][ 0 ]
            }
            u = SaveDequant[ 1 ][ i ][ j ]
            v = SaveDequant[ 2 ][ i ][ j ]
            v = Round2Signed(u * cU + v * cV, CCTX_PREC_BITS)
            v = Clip3( -(1 << (BitDepth + 7)), (1 << (BitDepth + 7)) - 1, v )
        }
    }
}
```

```

    Dequant[i][j] = v
  }
}

```

where the constant table Cctx_Mtx (which stores the cosine and sine of the rotation angle shifted up by CCTX_PREC_BITS bits) is defined as follows:

```

Cctx_Mtx[CCTX_TYPES - 1][2] = {
  { 181, 181 },
  { 222, 128 },
  { 128, 222 },
  { 181, -181 },
  { 222, -128 },
  { 128, -222 }
}

```

§ 7.15. Inverse transform process

§ 7.15.1. General

This section details the inverse transforms used during the reconstruction processes detailed in [§7.14 Reconstruction and dequantization](#).

§ 7.15.2. 1D transforms

§ 7.15.2.1. 1d inverse transform process

The inputs to this process are:

- an array src with the input coefficients,
- a variable txType1D that specifies the type of transform to apply,
- a variable sz specifying the length of the src array,
- a variable shift specifying the amount of down-shifting to apply,
- a variable colTx specifying whether the current 1-D inverse transform is applied to the columns of the input coefficients.

The process transforms the input coefficients using a matrix multiplication as follows for $i = 0..(sz-1)$:

```

s = 0
if (sz == 4) {
  for (j = 0; j < 4; j++) {
    if (txType1D == DCT) {
      s += Dct_Kernel4[ j ][ i ] * src[ j ]
    } else if (txType1D == ADST) {
      s += Adst_Kernel4[ j ][ i ] * src[ j ]
    } else {
      s += Fdst_Kernel4[ j ][ i ] * src[ j ]
    }
  }
} else if (sz == 8) {
  for (j = 0; j < 8; j++) {
    if (txType1D == DCT) {
      s += Dct_Kernel8[ j ][ i ] * src[ j ]
    } else if (txType1D == ADST) {
      s += Adst_Kernel8[ j ][ i ] * src[ j ]
    } else if (txType1D == FDST) {
      s += Fdst_Kernel8[ j ][ i ] * src[ j ]
    }
  }
}

```



```

    } else if (txTypeID == DDTX) {
        s += Ddtx_Kernel8[ j ][ i ] * src[ j ]
    } else {
        s += Ddtx_Kernel8[ j ][ 7 - i ] * src[ j ]
    }
}
} else if (sz == 16) {
    for (j = 0; j < 16; j++) {
        if (txTypeID == DCT) {
            s += Dct_Kernel16[ j ][ i ] * src[ j ]
        } else if (txTypeID == ADST) {
            s += Adst_Kernel16[ j ][ i ] * src[ j ]
        } else if (txTypeID == FDST) {
            s += Fdst_Kernel16[ j ][ i ] * src[ j ]
        } else if (txTypeID == DDTX) {
            s += Ddtx_Kernel16[ j ][ i ] * src[ j ]
        } else {
            s += Ddtx_Kernel16[ j ][ 15 - i ] * src[ j ]
        }
    }
} else {
    for (j = 0; j < 32; j++) {
        s += Dct_Kernel32[ j ][ i ] * src[ j ]
    }
}
result[i] = Clip3( -( 1 << ( BitDepth + ( colTx ? 0 : 7 ) ) ),
                  ( 1 << ( BitDepth + ( colTx ? 0 : 7 ) ) ) - 1,
                  Round2(s, shift) )

```

The output of the process is the array result.

§ 7.15.2.2. Inverse Walsh-Hadamard transform process

The inputs to this process are:

- an array src (of length 4) with the input coefficients,
- a variable shift that specifies the amount of pre-scaling.

This process does an in-place transform of the array src as follows:

```

a = src[ 0 ] >> shift
c = src[ 1 ] >> shift
d = src[ 2 ] >> shift
b = src[ 3 ] >> shift
a += c
d -= b
e = (a - d) >> 1
b = e - b
c = e - c
a -= b
d += c
result[ 0 ] = a
result[ 1 ] = b
result[ 2 ] = c
result[ 3 ] = d

```

The output of this process is the array result.

§ 7.15.2.3. Inverse identity transform process

The inputs to this process are:

- an array src with the input coefficients,

- a variable scale that specifies the amount of scaling to apply,
- a variable sz specifying the length of the src array,
- a variable shift specifying the amount of down-shifting to apply,
- a variable colTx specifying whether the current 1-D inverse transform is applied to the columns of the input coefficients.

The process does a scaling of the array src by the following calculation for i = 0..(sz-1):

```
result[i] = Clip3( - ( 1 << ( BitDepth + ( colTx ? 0 : 7 ) ) ),
                  ( 1 << ( BitDepth + ( colTx ? 0 : 7 ) ) ) - 1,
                  Round2(src[i] * scale, shift) )
```

The output of the process is the array result.

NOTE: This section defines the inverse identity transform used for lossy segments. For lossless segments, the inverse identity transform is specially handled using a bit-shift operation as shown in [§ 7.15.4 2D inverse transform process](#).

§ 7.15.3. Secondary transform process

This process performs a matrix based transform for coefficients stored in the 2D array Dequant. The output is placed back into the array Dequant.

The input to this process is a variable txSz that specifies the transform size.

The variables w, h, bwl, large, and n (related to the size of the transform block) are derived as follows:

```
w = Min(32, Tx_Width[ txSz ])
h = Min(32, Tx_Height[ txSz ])
bwl = Min(5, Tx_Width_Log2[ txSz ])
large = w >= 8 && h >= 8
if ( !large ) {
    n = IST_4X4_HEIGHT
} else if ( txSz == TX_8X8 || PlaneTxType == ADST_ADST ) {
    n = IST_8X8_HEIGHT_RED
} else {
    n = IST_8X8_HEIGHT
}
```

The variables kernel and transpose (describing the type of transform to apply) are derived as follows:

```
mode = YMode
if ( is_directional_mode( mode ) ) {
    pAngle = Mode_To_Angle[ mode ] + AngleDeltaY * ANGLE_STEP +
             Mrl_Index_To_Delta[ MrlIndex ]
    (mode,unusedAngle) = wide_angle_mapping( mode, Tx_Width[txSz],
                                             Tx_Height[txSz], pAngle )
}
if ( is_inter ) {
    kernel = 0
} else if ( PlaneTxType == ADST_ADST && Tx_Width[ txSz ] >= 8 &&
            Tx_Height[ txSz ] >= 8 ) {
    kernel = Inv_Most_Probable_Stx_Mapping_Adst[ mode ][ most_probable_stx_set ]
} else {
    kernel = Inv_Most_Probable_Stx_Mapping[ mode ][ most_probable_stx_set ]
}
```

```

if (PlaneTxType == ADST_ADST) {
    kernel += 7
}
transpose = (mode == H_PRED || mode == D157_PRED ||
             mode == D67_PRED || mode == SMOOTH_H_PRED)

```

where the constant tables `Inv_Most_Probable_Stx_Mapping` and `Inv_Most_Probable_Stx_Mapping_Adst` are defined as:

```

Inv_Most_Probable_Stx_Mapping[ INTRA_MODES - 1 ][ IST_DIR_SIZE ] = {
    { 6, 1, 0, 5, 4, 3, 2 },
    { 1, 6, 0, 4, 2, 5, 3 },
    { 1, 6, 0, 4, 2, 5, 3 },
    { 2, 6, 0, 5, 1, 4, 3 },
    { 3, 4, 6, 1, 0, 2, 5 },
    { 4, 1, 3, 6, 0, 5, 2 },
    { 4, 1, 3, 6, 0, 5, 2 },
    { 5, 0, 6, 2, 1, 4, 3 },
    { 5, 0, 6, 2, 1, 4, 3 },
    { 6, 1, 0, 5, 4, 3, 2 },
    { 1, 6, 0, 4, 2, 5, 3 },
    { 1, 6, 0, 4, 2, 5, 3 }
}

Inv_Most_Probable_Stx_Mapping_Adst[INTRA_MODES - 1]
                                   [IST_REDUCE_SET_SIZE_ADST_ADST] = {
    { 3, 1, 0, 2 },
    { 1, 3, 0, 2 },
    { 1, 3, 0, 2 },
    { 1, 3, 0, 2 },
    { 0, 2, 3, 1 },
    { 2, 1, 0, 3 },
    { 2, 1, 0, 3 },
    { 1, 0, 3, 2 },
    { 1, 0, 3, 2 },
    { 3, 1, 0, 2 },
    { 1, 3, 0, 2 },
    { 1, 3, 0, 2 }
}

```

The coefficients are placed in scan order into the array `coefs` as follows:

```

scanIn = get_scan(txSz, TX_CLASS_2D)
for( i = 0 ; i < n ; i++) {
    pos = scanIn[ i ]
    x = pos & (w - 1)
    y = pos >> bwl
    coefs[ i ] = Dequant[ y ][ x ]
    Dequant[ y ][ x ] = 0
}

```

The coefficients are transformed by a matrix multiplication and placed back into `Dequant` as follows:

```

scanBwl = large ? 3 : 2
scanW = 1 << scanBwl
scanOut = large ? Stx_Scan_Order_8x8 : Stx_Scan_Order_4x4
if ( large ) {
    scanMap = Stx_Scan_Map[ kernel ][ sec_tx_type - 1 ]
}
n2 = large ? IST_8X8_WIDTH : IST_4X4_WIDTH
for( i = 0; i < n2; i++ ) {
    t = 0
    for( j = 0 ; j < n ; j++ ) {
        t += coefs[ j ] *

```

```

        (large ? Ist_8x8_Kernel[ kernel ][ sec_tx_type-1 ][ j ][ i ] :
             Ist_4x4_Kernel[ kernel ][ sec_tx_type-1 ][ j ][ i ] )
    }
    v = Round2Signed( t, 7 )
    v = Clip3( -(1 << (BitDepth + 7)), (1 << (BitDepth + 7)) - 1, v)
    if ( large ) {
        pos = scanOut[scanMap[i]]
    } else {
        pos = scanOut[i]
    }
    x = pos & (scanW - 1)
    y = pos >> scanBwl
    if (transpose) {
        Dequant[x][y] = v
    } else {
        Dequant[y][x] = v
    }
}

```

where constant tables Stx_Scan_Order_4x4 and Stx_Scan_Order_8x8 are defined as:

```

Stx_Scan_Order_4x4[IST_4X4_WIDTH] = {
    0, 1, 4, 8, 5, 2, 3, 6, 9, 12, 13, 10, 7, 11, 14, 15
}
Stx_Scan_Order_8x8[64] = {
    0, 1, 8, 16, 9, 2, 3, 10, 17, 24, 32, 25, 18, 11, 4, 5,
    12, 19, 26, 33, 40, 48, 41, 34, 27, 20, 13, 6, 7, 14, 21, 28,
    35, 42, 49, 56, 57, 50, 43, 36, 29, 22, 15, 23, 30, 37, 44, 51,
    58, 59, 52, 45, 38, 31, 39, 46, 53, 60, 61, 54, 47, 55, 62, 63
}

```

NOTE: The scanOut tables are not the inverse of the scanIn tables.

§ 7.15.4. 2D inverse transform process

This process performs a 2D inverse transform for an array of coefficients stored in the 2D array Dequant. The output is placed in the 2D array Residual.

The inputs to this process are:

- a variable plane specifying whether the process is filtering Y, U, or V samples,
- a variable txSz that specifies the transform size.

Set the variable adjTxSz equal to Adjusted_Tx_Size[txSz].

Set the variable log2W equal to Tx_Width_Log2[txSz].

Set the variable log2H equal to Tx_Height_Log2[txSz].

Set the variable adjLog2W equal to Tx_Width_Log2[adjTxSz].

Set the variable adjLog2H equal to Tx_Height_Log2[adjTxSz].

Set the variable w equal to 1 << adjLog2W.

Set the variable h equal to 1 << adjLog2H.

The variable pels is set equal to w * h.

The variable shift is set equal to (pels > 256) + (pels > 1024).

If Lossless is equal to 1 and PlaneTxType is equal to IDTX, set Residual[i][j] equal to Dequant[i][j] >> (3 - shift) for i = 0..h-1, for j = 0..w-1.

Otherwise, the 2d matrix transform process specified in [§ 7.15.4.1 2D matrix transform process](#) is invoked with adjTxSz and txSz as inputs.

The variable useDpcm is set equal to (plane == 0 ? use_dpcm_y : use_dpcm_uv).

The variable mode is set equal to (plane == 0 ? YMode : UVMode).

If useDpcm is equal to 1 a cumulative sum is applied to Residual as follows:

```
if ( mode == V_PRED ) {
    for ( j = 0; j < w; j++ ) {
        for ( i = 1; i < h; i++ ) {
            Residual[ i ][ j ] += Residual[ i - 1 ][ j ]
        }
    }
} else {
    for ( j = 1; j < w; j++ ) {
        for ( i = 0; i < h; i++ ) {
            Residual[ i ][ j ] += Residual[ i ][ j - 1 ]
        }
    }
}
```

If adjTxSz is not equal to txSz, the residual is expanded by sample duplication as follows:

```
w2 = Tx_Width[ txSz ]
if ( w != w2 ) {
    for( i = 0; i < h; i++ ) {
        for( j = w - 1; j >= 0; j-- ) {
            r = Residual[ i ][ j ]
            Residual[ i ][ 2 * j ] = r
            Residual[ i ][ 2 * j + 1 ] = r
        }
    }
}
h2 = Tx_Height[ txSz ]
if ( h != h2 ) {
    for( i = h - 1; i >= 0; i-- ) {
        for( j = 0; j < w2; j++ ) {
            r = Residual[ i ][ j ]
            Residual[ 2 * i ][ j ] = r
            Residual[ 2 * i + 1 ][ j ] = r
        }
    }
}
```

§ 7.15.4.1. 2D matrix transform process

This process performs a 2D matrix transform for an array of coefficients stored in the 2D array Dequant. The output is placed in the 2D array Residual.

The inputs to this process are:

- a variable adjTxSz that specifies the adjusted transform size,
- a variable txSz that specifies the transform size.

Set the variable log2W equal to Tx_Width_Log2[txSz].

Set the variable log2H equal to Tx_Height_Log2[txSz].

Set the variable adjLog2W equal to Tx_Width_Log2[adjTxSz].

Set the variable adjLog2H equal to Tx_Height_Log2[adjTxSz].

Set the variable w equal to 1 << adjLog2W.

Set the variable h equal to 1 << adjLog2H.

The constant table Transform_Shift is specified as:

```
Transform_Shift[ TX_SIZES_ALL ][ 2 ] = {
    { 7, 10 },
    { 7, 11 },
    { 6, 13 },
    { 6, 13 },
    { 6, 13 },
    { 7, 10 },
    { 7, 10 },
    { 7, 11 },
    { 7, 11 },
    { 6, 12 },
    { 6, 12 },
    { 6, 12 },
    { 6, 12 },
    { 6, 12 },
    { 6, 12 },
    { 6, 13 },
    { 6, 13 },
    { 6, 13 },
    { 6, 13 },
    { 7, 11 },
    { 7, 11 },
    { 6, 12 },
    { 6, 12 },
    { 6, 13 },
    { 6, 13 },
}
```

Set the variable rowShift equal to Transform_Shift[txSz][0].

Set the variable colShift equal to Transform_Shift[txSz][1].

The function get_transform_1d_type is specified as:

```
get_transform_1d_type( dir, sz ) {
    useDdt = enable_inter_ddt && !use_intrabc && is_inter
    t = Transform_1d_Type[ PlaneTxType ][ dir ]
    if ( useDdt && ( t == ADST || t == FDST ) && sz != 4 ) {
        return ( t == ADST ) ? DDTX : FDDT
    }
    return t
}
```

The 1d transform types returned from this function are specified as specified in [Table 7.1](#):

Table 7.1: 1D transform type values and names

Value of 1d transform type	Name of 1d transform type
0	DCT
1	IDT
2	ADST
3	FDST
4	DDTX
5	FDDT

where the constant table Transform_1d_Type is specified as:

```

Transform_1d_Type[ TX_TYPES ][ 2 ] = {
    { DCT, DCT },
    { DCT, ADST },
    { ADST, DCT },
    { ADST, ADST },
    { DCT, FDST },
    { FDST, DCT },
    { FDST, FDST },
    { FDST, ADST },
    { ADST, FDST },
    { IDT, IDT },
    { IDT, DCT },
    { DCT, IDT },
    { IDT, ADST },
    { ADST, IDT },
    { IDT, FDST },
    { FDST, IDT }
}

```

Set the variable rowType equal to get_transform_1d_type(0, w).

Set the variable colType equal to get_transform_1d_type(1, h).

txRowIn[j] is set equal to 0 for j = 0..w-1.

txColIn[i] is set equal to 0 for i = 0..h-1.

intermediate[i][j] is set equal to 0 for i = 0..Min(h,32)-1, for j = 0..w-1.

The following applies for i = 0..(Min(h,32)-1):

- txRowIn[j] is derived as follows for j = 0..(Min(w,32)-1):
 - If Abs(log2W - log2H) is odd, txRowIn[j] is set equal to Round2(Dequant[i][j] * 2896, 12).
 - Otherwise, txRowIn[j] is set equal to Dequant[i][j].
- The row transform is applied as follows:
 - If Lossless is equal to 1, the Inverse [WHT](#) process as specified in [§ 7.15.2.2 Inverse Walsh-Hadamard transform process](#) is invoked with txRowIn and the input variable shift equal to 3 as inputs, and the output is assigned to txRowOut.
 - Otherwise, if rowType is equal to IDT, the inverse identity transform process as specified in [§ 7.15.2.3 Inverse identity transform process](#) is invoked with txRowIn, get_identity_scale(log2W), w, rowShift, and 0 as inputs, and the output is assigned to txRowOut.

- Otherwise, the 1d matrix transform process specified in [§ 7.15.2.1 1d inverse transform process](#) is invoked with txRowIn, rowType, w, rowShift, and 0 as inputs, and the output is assigned to txRowOut.
- Set intermediate[i][j] equal to txRowOut[j] for j = 0..(w-1).

The following applies for j = 0..(w-1):

- Set txColIn[i] equal to intermediate[i][j] for i = 0..(Min(h,32)-1).
- The column transform is applied as follows:
 - If Lossless is equal to 1, the Inverse [WHT](#) process as specified in [§ 7.15.2.2 Inverse Walsh-Hadamard transform process](#) is invoked with txColIn and the input variable shift equal to 0 as inputs, and the output is assigned to txColOut.
 - Otherwise, if colType is equal to IDT, the inverse identity transform process as specified in [§ 7.15.2.3 Inverse identity transform process](#) is invoked with txColIn, get_identity_scale(log2H), h, colShift, and 1 as inputs, and the output is assigned to txColOut.
 - Otherwise, the 1d matrix transform process specified in [§ 7.15.2.1 1d inverse transform process](#) is invoked with txColIn, colType, h, colShift, and 1 as inputs, and the output is assigned to txColOut.
- Residual[i][j] is set equal to txColOut[i] for i = 0..(h-1).

where the function get_identity_scale is specified as:

```
get_identity_scale( log2Sz ) {
    if (log2Sz == 2) {
        return 128
    } else if (log2Sz == 3) {
        return 181
    } else if (log2Sz == 4) {
        return 256
    }
    return 362
}
```

§ 7.16. Deblocking filter for [TIP](#) process

Input to this process is the array CurrFrame of reconstructed samples.

Output from this process is a modified array CurrFrame containing deblocked samples.

The filtering is applied as follows:

```
tipSize = ( enable_tip_refinemv &&
             TipInterpFilter == EIGHTTAP_SHARP ) ? BLOCK_8X8 : BLOCK_16X16
for ( plane = 0; plane < NumPlanes; plane++ ) {
    baseFilterLevel = base_q_idx
    if (plane == 1) {
        baseFilterLevel += DeltaQUAc + BaseUVAcDeltaQ
    } else if (plane == 2) {
        baseFilterLevel += DeltaQVAc + BaseUVAcDeltaQ
    }
    qThr = Round2(get_q(baseFilterLevel,0),QUANT_TABLE_BITS) >> 6
    qInd = Clip3(0, MAX_SIDE_TABLE - 1, baseFilterLevel - 24 * (BitDepth - 8))
    side = Max( Side_Thresholds[qInd] + (1 << (12 - BitDepth)),
               0 ) >> ( 13 - BitDepth )
    subX = plane == 0 ? 0 : SubsamplingX
```



```

subY = plane == 0 ? 0 : SubsamplingY
sw = Block_Width[tipSize] >> subX
sh = Block_Height[tipSize] >> subY
h = (MiRows * MI_SIZE >> subY)
w = (MiCols * MI_SIZE >> subX)
for ( y = 0; y < h; y += 4 ) {
    for ( x = 0; x < w; x += sw ) {
        if ( x > 0 ) {
            vertTileEdge = is_vert_tile_edge( x, subX )
            (maxWidthPos, maxWidthNeg) = filter_maximum_width( plane,
                                                                filterSize = sw,
                                                                vertTileEdge )
            if ( !disable_loopfilters_across_tiles || !vertTileEdge ) {
                width = filter_choice( x, y, plane, qThr, side, dx=1, dy=0,
                                       maxWidthNeg, maxWidthPos, MI_SIZE )
                if (width > 0) {
                    for ( i = 0; i < 4; i++ ) {
                        sample_filtering( x, y + i, plane, qThr, dx=1, dy=0,
                                         Min(width,maxWidthNeg),
                                         Min(width,maxWidthPos), 0, 0 )
                    }
                }
            }
        }
    }
}
for ( x = 0; x < w; x += 4 ) {
    for ( y = 0; y < h; y += sh ) {
        if ( y > 0 ) {
            horzTileEdge = is_horz_tile_edge( y, subY )
            if ( !disable_loopfilters_across_tiles || !horzTileEdge ) {
                horz64Edge = ( (y << subY) % 64 ) == 0
                (maxWidthPos, maxWidthNeg) = filter_maximum_width( plane,
                                                                    filterSize = sh,
                                                                    horz64Edge )
                width = filter_choice( x, y, plane, qThr, side, dx=0, dy=1,
                                       maxWidthNeg, maxWidthPos, MI_SIZE )
                if (width > 0) {
                    for ( i = 0; i < MI_SIZE; i++ ) {
                        sample_filtering( x + i, y, plane, qThr, dx=0, dy=1,
                                         Min(width,maxWidthNeg),
                                         Min(width,maxWidthPos), 0, 0 )
                    }
                }
            }
        }
    }
}
}

```

The function call of `filter_maximum_width` indicates that the filter maximum width process specified in [§ 7.17.3 Filter maximum width process](#) is invoked.

The function call of `filter_choice` indicates that the filter choice process specified in [§ 7.17.7.2 Filter choice process](#) is invoked.

The function call of `sample_filtering` indicates that the sample filtering process specified in [§ 7.17.7 Sample filtering process](#) is invoked.

The function `is_vert_tile_edge` (which determines if the filter crosses a vertical tile edge) is specified as:

```

is_vert_tile_edge( x, subX ) {
    lumaX = x << subX
    col = lumaX >> MI_SIZE_LOG2
    for( t = 0; t < TileCols; t++ ) {
        if ( col == MiColStarts[ t ] )

```

```

        return 1
    }
    return 0
}

```

The function `is_horz_tile_edge` (which determines if the filter crosses a horizontal tile edge) is specified as:

```

is_horz_tile_edge( y, subY ) {
    lumaY = y << subY
    row = lumaY >> MI_SIZE_LOG2
    for( t = 0; t < TileRows; t++ ) {
        if ( row == MiRowStarts[ t ] )
            return 1
    }
    return 0
}

```

§ 7.17. Deblocking filter process

§ 7.17.1. General

Input to this process is the array `CurrFrame` of reconstructed samples.

Output from this process is a modified array `CurrFrame` containing deblocked samples.

The purpose of the deblocking filter is to eliminate (or at least reduce) visually objectionable artifacts associated with the semi-independence of the coding of super blocks and their constituent sub-blocks.

The deblocking filter is applied on all vertical boundaries followed by all horizontal boundaries as follows:

```

for ( plane = 0; plane < NumPlanes; plane++ ) {
    for ( pass = 0; pass < 2; pass++ ) {
        if ( apply_deblocking_filter[plane==0 ? pass : plane + 1] ) {
            rowStep = ( plane == 0 ) ? 1 : ( 1 << SubsamplingY )
            colStep = ( plane == 0 ) ? 1 : ( 1 << SubsamplingX )
            for ( row = 0; row < MiRows; row += rowStep )
                for ( col = 0; col < MiCols; col += colStep )
                    deblocking_filter_edge( plane, pass, row, col )
        }
    }
}

```

When the function `deblocking_filter_edge` is called, the edge deblocking filter process specified in [§7.17.2 Edge deblocking filter process](#) is invoked with the variables `plane`, `pass`, `row`, and `col` as inputs.

NOTE: The deblocking filter is an integral part of the decoding process, in that the results of deblocking filtering are used in the prediction of subsequent frames.

NOTE: The deblocking filtering is designed so that any order of filtering for the edges will give identical results, provided that the vertical boundaries are filtered before the horizontal boundaries.

§ 7.17.2. Edge deblocking filter process

The inputs to this process are:

- a variable plane specifying whether the process is filtering Y, U, or V samples,
- a variable pass specifying the direction of the edges. pass equal to 0 means the process is filtering vertical block boundaries, and pass equal to 1 means the process is filtering horizontal block boundaries,
- variables row and col specifying the location of the edge in units of 4x4 blocks in the luma plane.

The outputs of this process are modified values in the array CurrFrame.

The variable sbShift is set equal to `Mi_Width_Log2[SbSize]`.

The variable sbX (the superblock X position) is set equal to `(col >> sbShift)`.

The variable sbY (the superblock Y position) is set equal to `(row >> sbShift)`.

If use_bru is equal to 1 and `BruModes[sbY << sbShift][sbX << sbShift]` is not equal to `BRU_ACTIVE`, this process terminates immediately.

The variables subX and subY describing the subsampling of the current plane are derived as follows:

- If plane is equal to 0, subX and subY are set equal to 0.
- Otherwise (plane is not equal to 0), subX is set equal to `SubsamplingX` and subY is set equal to `SubsamplingY`.

The variables dx and dy are derived as follows:

- If pass is equal to 0, then dx is set equal to 1, dy is set equal to 0.
- Otherwise (pass is equal to 1), dy is set equal to 1, dx is set equal to 0.

dx and dy specify the offset between the samples to be filtered.

The variable x is set equal to `col * MI_SIZE`.

The variable y is set equal to `row * MI_SIZE`.

x and y contain the location in luma coordinates.

The variable sbEdge (equal to 1 if this is a horizontal edge on the 64x64 grid or a vertical tile edge) is computed as follows:

```
tileVertEdge = (pass == 0 && MiColStartGrid[ row ][ col ] == col)
tileHorzEdge = (pass == 1 && MiRowStartGrid[ row ][ col ] == row)
horz64Edge = (pass == 1 && ( y % 64 ) == 0)
sbEdge = horz64Edge || tileVertEdge
```

If `disable_loopfilters_across_tiles` is equal to 1 and `tileVertEdge` is equal to 1, then this process immediately returns and no filtering is applied to this edge.

If `disable_loopfilters_across_tiles` is equal to 1 and `tileHorzEdge` is equal to 1, then this process immediately returns and no filtering is applied to this edge.

The variable onScreen is derived as follows:

- If pass is equal to 0 and x is equal to 0, onScreen is set equal to 0.
- Otherwise, if pass is equal to 1 and y is equal to 0, onScreen is set equal to 0.
- Otherwise, onScreen is set equal to 1.

If onScreen is equal to 0, then this process immediately returns and no filtering is applied to this edge.

The variables xP and yP (containing the location in the current plane) are derived as follows:

- Set xP equal to $x \gg \text{subX}$.
- Set yP equal to $y \gg \text{subY}$.

The variables prevRow and prevCol (containing the location of the [mode info block](#) on the other side of the boundary) are derived as follows:

- Set prevRow equal to $\text{row} - (\text{dy} \ll \text{subY})$.
- Set prevCol equal to $\text{col} - (\text{dx} \ll \text{subX})$.

The variable isSubPuEdge (equal to 1 if the edge is treated as a subblock edge) is computed by comparing the locations of the subblock as follows:

```
subPuColBase = SubPuColBase[ plane > 0 ][ row ][ col ]
subPuRowBase = SubPuRowBase[ plane > 0 ][ row ][ col ]
prevSubPuColBase = SubPuColBase[ plane > 0 ][ prevRow ][ prevCol ]
prevSubPuRowBase = SubPuRowBase[ plane > 0 ][ prevRow ][ prevCol ]
isSubPuEdge = allow_df_sub_pu && ( subPuColBase != prevSubPuColBase ||
                                   subPuRowBase != prevSubPuRowBase )
```

Set the variable subPuSize (giving the size of the subblocks used in this block) equal to $\text{SubPuSize}[\text{plane} > 0][\text{row}][\text{col}]$.

Set the variable currLossless equal to $\text{LosslessArray}[(\text{plane} > 0) ? \text{ChromaSegmentIds}[\text{row}][\text{col}] : \text{SegmentIds}[\text{row}][\text{col}]$.

Set the variable prevLossless equal to

$\text{LosslessArray}[(\text{plane} > 0) ? \text{ChromaSegmentIds}[\text{prevRow}][\text{prevCol}] : \text{SegmentIds}[\text{prevRow}][\text{prevCol}]$.

Set the variable MiSize equal to $\text{MiSizes}[\text{plane} > 0][\text{row}][\text{col}]$.

Set the variable baseRow equal to $\text{MiRowBase}[\text{plane} > 0][\text{row}][\text{col}]$.

Set the variable baseCol equal to $\text{MiColBase}[\text{plane} > 0][\text{row}][\text{col}]$.

Set the variable baseY equal to $(\text{baseRow} * \text{MI_SIZE}) \gg \text{subY}$.

Set the variable baseX equal to $(\text{baseCol} * \text{MI_SIZE}) \gg \text{subX}$.

Set the variable txSz equal to $\text{DeblockingTxSizes}[\text{plane}][\text{row} \gg \text{subY}][\text{col} \gg \text{subX}]$.

Set the variable prevSubPuSize equal to $\text{SubPuSize}[\text{plane} > 0][\text{prevRow}][\text{prevCol}]$.

Set the variable prevTxSz equal to $\text{DeblockingTxSizes}[\text{plane}][\text{prevRow} \gg \text{subY}][\text{prevCol} \gg \text{subX}]$.

Set the variable txColBase equal to TxColBase[plane][row >> subY][col >> subX].

Set the variable txRowBase equal to TxRowBase[plane][row >> subY][col >> subX].

Set the variable prevTxColBase equal to TxColBase[plane][prevRow >> subY][prevCol >> subX].

Set the variable prevTxRowBase equal to TxRowBase[plane][prevRow >> subY][prevCol >> subX].

If plane is greater than 0, the chroma information is held in the bottom-right mode info so the variables are adjusted as follows:

- row is set equal to baseRow + Num_4x4_Blocks_High[MiSize] - 1.
- col is set equal to baseCol + Num_4x4_Blocks_Wide[MiSize] - 1.

Set the variable skip equal to Skips[row][col].

If plane is greater than 0, the variables are modified as follows:

```
if ( RegionTypes[ row ][ col ] == INTRA_REGION ||  
    (FrameIsIntra && enable_sdp) ) {  
    skip = 0  
}
```

The variable xR is set equal to xP - baseX.

The variable yR is set equal to yP - baseY.

The variable isBlockEdge (equal to 1 if the samples cross a prediction block edge) is derived as follows:

- If pass is equal to 0 and xR is equal to 0, isBlockEdge is set equal to 1.
- Otherwise, if pass is equal to 1 and yR is equal to 0, isBlockEdge is set equal to 1.
- Otherwise, isBlockEdge is set equal to 0.

The variable isTxEdge (equal to 1 if the samples cross a transform block edge) is derived as follows:

- If txColBase is not equal to prevTxColBase, isTxEdge is set equal to 1.
- Otherwise, if txRowBase is not equal to prevTxRowBase, isTxEdge is set equal to 1.
- Otherwise, isTxEdge is set equal to 0.

If isSubPuEdge is equal to 1, the variables txSz, prevTxSz, and isSubPuEdge are modified as follows:

```
(txSz, isSubPuEdge) = filt_max_size( pass, isTxEdge, txSz, subPuSize )  
(prevTxSz, _) = filt_max_size( pass, isTxEdge, prevTxSz, prevSubPuSize )  
if ( isBlockEdge ) {  
    isSubPuEdge = 0  
}
```

where the function filt_max_size is specified as:

```
filt_max_size( pass, isTxEdge, txSz, subPuSize ) {  
    isSubPuEdge = 1  
    if ( pass == 0 ) {  
        if ( Tx_Width[ txSz ] < Tx_Width[ subPuSize ] ) {
```

```

        isSubPuEdge = 0
    } else if ( !isTxEdge && Tx_Width[ txSz ] == 8 ) {
        txSz = TX_4X4
    } else if ( !isTxEdge && Tx_Width[ txSz ] == 16 &&
                Tx_Width[ subPuSize ] == 16 ) {
        txSz = TX_8X8
    } else {
        txSz = subPuSize
    }
}
if ( pass == 1 ) {
    if ( Tx_Height[ txSz ] < Tx_Height[ subPuSize ] ) {
        isSubPuEdge = 0
    } else if ( !isTxEdge && Tx_Height[ txSz ] == 8 ) {
        txSz = TX_4X4
    } else if ( !isTxEdge && Tx_Height[ txSz ] == 16 &&
                Tx_Height[ subPuSize ] == 16 ) {
        txSz = TX_8X8
    } else {
        txSz = subPuSize
    }
}
return (txSz, isSubPuEdge)
}

```

The adaptive filter strength process specified in [§ 7.17.5 Adaptive filter strength process](#) is invoked with the inputs row, col, plane, and pass, and the output assigned to the variables currQ and currSide.

The adaptive filter strength process specified in [§ 7.17.5 Adaptive filter strength process](#) is invoked with the inputs prevRow, prevCol, plane, and pass, and the output assigned to the variables prevQ and prevSide.

The variable applyFilter (equal to 1 if the samples are filtered) is derived as follows:

- If isTxEdge is equal to 0 and isSubPuEdge is equal to 0, applyFilter is set equal to 0.
- Otherwise, if (currQ != 0 && currSide != 0) is equal to 0 and (prevQ != 0 && prevSide != 0) is equal to 0, applyFilter is set equal to 0.
- Otherwise, if isBlockEdge is equal to 1 or skip is equal to 0 or isSubPuEdge is equal to 1, applyFilter is set equal to 1.
- Otherwise, applyFilter is set equal to 0.

If applyFilter is equal to 0, this process terminates immediately.

The filter size process specified in [§ 7.17.4 Filter size process](#) is invoked with the inputs txSz, prevTxSz, and pass, and the output assigned to the variable filterSize (containing the maximum filter size that can be used).

The variable filterSize is clipped at the edge of the screen as follows:

```

planeWidth = MiCols * MI_SIZE >> subX
planeHeight = MiRows * MI_SIZE >> subY
if ( plane == 0 ) {
    if ( xP + dx * 16 > planeWidth || yP + dy * 16 > planeHeight ) {
        filterSize = Min(filterSize, 16)
    }
} else {
    if ( xP + dx * 8 > planeWidth || yP + dy * 8 > planeHeight ) {

```

```

        filterSize = Min(filterSize, 8)
    }
}

```

The variables qThr and side are set as follows:

```

if ( currQ && prevQ ) {
    qThr = (currQ + prevQ + 1) >> 1
} else {
    qThr = Max( currQ, prevQ )
}
if ( currSide && prevSide ) {
    side = (currSide + prevSide + 1) >> 1
} else {
    side = Max( currSide, prevSide )
}
if ( isSubPuEdge && !isTxEdge ) {
    qThr = qThr >> 3
    side = side >> 3
}

```

If prevLossless is equal to 1 and currLossless is equal to 1, this process terminates immediately.

The filter maximum width process specified in [§7.17.3 Filter maximum width process](#) is invoked with plane, filterSize, and sbEdge as inputs, and the outputs are assigned to maxWidthPos and maxWidthNeg.

The filter choice process specified in [§7.17.7.2 Filter choice process](#) is invoked with xP, yP, plane, qThr, side, dx, dy, maxWidthNeg, maxWidthPos, MI_SIZE as inputs, and the output is assigned to width.

If width is equal to 0, this process terminates immediately.

For the variable i taking values from 0 to MI_SIZE - 1, the sample filtering process specified in [§7.17.7 Sample filtering process](#) is invoked with the input variable x set equal to xP + dy * i, the input variable y set equal to yP + dx * i, and the variables plane, qThr, dx, dy, Min(width,maxWidthNeg), Min(width,maxWidthPos), prevLossless, and currLossless as inputs.

NOTE: the vector (dx,dy) represents the direction of the filter, while (dy,dx) represents the direction of the boundary.

§ 7.17.3. Filter maximum width process

The inputs to this process are:

- a variable plane specifying whether the process is filtering Y, U or V samples,
- a variable filterSize specifying the maximum filter size that can be used,
- a variable sbEdge specifying if the edge is at a block boundary.

The variables maxWidthPos and maxWidthNeg are computed as follows:

```

if (filterSize <= 4) {
    maxWidthPos = 1
} else if (filterSize == 8) {
    maxWidthPos = 3
} else if (filterSize == 16) {
    maxWidthPos = plane != 0 ? 4 : 6
} else {

```

```

    maxWidthPos = plane != 0 ? 4 : 8
}
if ( sbEdge ) {
    maxWidthNeg = Min( maxWidthPos, plane != 0 ? 2 : 6)
} else {
    maxWidthNeg = maxWidthPos
}

```

The outputs of this process are the variables `maxWidthPos` and `maxWidthNeg`.

§ 7.17.4. Filter size process

The inputs to this process are:

- a variable `txSz` specifying the size of the transform block,
- a variable `prevTxSz` specifying the size of the transform block on the other side of the boundary,
- a variable `pass` specifying the direction of the edges.

The output of this process is the variable `filterSize` containing the maximum filter size that can be used in samples.

The output variable `filterSize` is derived as follows:

- If `pass` is equal to 0, `filterSize` is set equal to `Min( Tx_Width[ prevTxSz ], Tx_Width[ txSz ] )`.
- Otherwise (`pass` is equal to 1), `filterSize` is set equal to `Min( Tx_Height[ prevTxSz ], Tx_Height[ txSz ] )`.

§ 7.17.5. Adaptive filter strength process

The inputs to this process are:

- variables `row` and `col` specifying the luma location in units of 4x4 blocks,
- the variable `plane` specifying whether the process is filtering Y, U or V samples,
- the variable `pass` specifying the direction of the edge being filtered. `pass` equal to 0 means the process is filtering vertical block boundaries, and `pass` equal to 1 means the process is filtering horizontal block boundaries.

The outputs of this process are the variables `qThr` and `side`.

The variable `segment` is set as follows:

- If `plane` is equal to 0, `segment` is set equal to `SegmentIds[ row ][ col ]`.
- Otherwise (`plane` is greater than 0), `segment` is set equal to `ChromaSegmentIds[ row ][ col ]`.

The variable `qindex` is set as follows:

- If `plane` is equal to 0, `qindex` is set equal to `LumaQIndex[ row ][ col ]`.
- Otherwise (`plane` is greater than 0), `qindex` is set equal to `ChromaQIndex[ row ][ col ]`.

The adaptive filter strength selection process specified in [§ 7.17.6 Adaptive filter strength selection process](#) is invoked with `qindex`, `segment`, `plane`, and `pass` as inputs, and the output is assigned to `lvl`.

The output variables are derived as follows:

```
qThr = Round2( get_q( lvl , 0 ), QUANT_TABLE_BITS ) >> 6
qInd = Clip3( 0, MAX_SIDE_TABLE - 1, lvl - 24 * (BitDepth - 8) )
side = Max( Side_Thresholds[ qInd ] + (1 << (12 - BitDepth)), 0 ) >>
      ( 13 - BitDepth )
```

§ 7.17.6. Adaptive filter strength selection process

The inputs to this process are:

- the variable `qindex` specifying a value derived from the quantizer used for the block,
- the variable `segment` specifying the current segment id,
- the variable `plane` specifying whether the process is filtering Y, U or V samples,
- the variable `pass` specifying the direction of the edge being filtered. `pass` equal to 0 means the process is filtering vertical block boundaries, and `pass` equal to 1 means the process is filtering horizontal block boundaries.

The output of this process is the variable `lvlSeg` containing the filter strength level.

The variable `i` is set equal to `( plane == 0 ) ? pass : ( plane + 1 )`.

The variable `CurrentQIndex` is set equal to `qindex` (`CurrentQIndex` is used by the `get_qindex` function).

The variable `lvlSeg` is set as follows:

```
qindex2 = get_qindex( 0, segment )
if (plane == 1) {
    delta = DeltaQUAc + BaseUVAcDeltaQ
} else if (plane == 2) {
    delta = DeltaQVAc + BaseUVAcDeltaQ
} else {
    delta = 0
}
qc = q_clamped(qindex2, delta)
lvlSeg = qc + DF_DELTA_SCALE * DfDeltaQ[ i ]
```

Where the function `q_clamped` is defined as:

```
q_clamped( qindex, delta ) {
    if ( qindex == 0 && delta <= 0 ) {
        return 0
    }
    return Clip3(1, MaxQ, qindex + delta)
}
```

§ 7.17.7. Sample filtering process

§ 7.17.7.1. General

The inputs to this process are:

- variables `x` and `y` specifying the location within `CurrFrame[ plane ]`,
- a variable `plane` specifying whether the block is the Y, U or V plane,

- a variable qThr specifying a threshold used during the filtering operation,
- variables dx and dy specifying the direction perpendicular to the edge being filtered,
- a variable maxWidthNeg specifying the maximum number of samples allowed to be modified for negative offsets,
- a variable maxWidthPos specifying the maximum number of samples allowed to be modified for positive offsets,
- a variable prevLossless specifying if the previous samples are in a lossless segment,
- a variable currLossless specifying if the current samples are in a lossless segment.

The outputs of this process are modified values in the array CurrFrame.

The width is set equal to Max(maxWidthNeg, maxWidthPos).

The samples are filtered as follows:

```

q0 = CurrFrame[ plane ][ y ][ x ]
q1 = CurrFrame[ plane ][ y + dy ][ x + dx ]
p0 = CurrFrame[ plane ][ y - dy ][ x - dx ]
p1 = CurrFrame[ plane ][ y - dy * 2 ][ x - dx * 2 ]
qThrClamp = qThr * Q_Thresh_Mults[width - 1]
deltaM2 = p1 - q1 + 3 * (q0 - p0)
deltaM2 *= 4
deltaM2 = Clip3(-qThrClamp, qThrClamp, deltaM2)
deltaM2Neg = deltaM2 * W_Mult[maxWidthNeg - 1]
deltaM2Pos = deltaM2 * W_Mult[maxWidthPos - 1]
for (i = 0; i < width; i++) {
    diffNeg = Round2(deltaM2Neg * (maxWidthNeg - i), 3 + DF_SHIFT)
    diffPos = Round2(deltaM2Pos * (maxWidthPos - i), 3 + DF_SHIFT)
    qy = y + i * dy
    qx = x + i * dx
    if ( !currLossless ) {
        CurrFrame[ plane ][ qy ][ qx ] =
            Clip1( CurrFrame[ plane ][ qy ][ qx ] - diffPos )
    }
    if ( i < maxWidthNeg && !prevLossless ) {
        pi = -i - 1
        py = y + pi * dy
        px = x + pi * dx
        CurrFrame[ plane ][ py ][ px ] =
            Clip1( CurrFrame[ plane ][ py ][ px ] + diffNeg )
    }
}

```

§ 7.17.7.2. Filter choice process

The inputs to this process are:

- variables x and y specifying the location within CurrFrame[plane],
- a variable plane specifying whether the block is the Y, U or V plane,
- variables qThr and sideThr that specify thresholds used during the filtering operation,
- variables dx and dy specifying the direction perpendicular to the edge being filtered,
- a variable maxWidthNeg specifying the maximum number of samples allowed to be modified for negative offsets,

- a variable `maxWidthPos` specifying the maximum number of samples allowed to be modified for positive offsets,
- a variable `count` specifying the length of the edge.

The output from this process is the chosen filter width.

If `qThr` is equal to 0 or `sideThr` is equal to 0, the process terminates immediately with 0 as output.

The variable `maxSamplesPos` is set equal to `Clip3(3, 8, maxWidthPos + 1)`.

The variable `maxSamplesNeg` is set equal to `Clip3(3, 8, maxWidthNeg + 1)`.

Arrays `s` and `t` containing samples for indices from `-maxSamplesNeg` to `maxSamplesPos - 1` are prepared as follows:

```
x2 = x + (count - 1) * dy
y2 = y + (count - 1) * dx
for (dist = 0; dist < maxSamplesPos; dist++) {
    s[dist] = CurrFrame[ plane ][ y + dist * dy ][ x + dist * dx ]
    t[dist] = CurrFrame[ plane ][ y2 + dist * dy ][ x2 + dist * dx ]
}
for (dist = 0; dist < maxSamplesNeg; dist++) {
    s[-dist-1] = CurrFrame[ plane ]
                        [ y - (dist + 1) * dy ]
                        [ x - (dist + 1) * dx ]
    t[-dist-1] = CurrFrame[ plane ]
                        [ y2 - (dist + 1) * dy ]
                        [ x2 - (dist + 1) * dx ]
}
```

An array `secondDeriv` containing the estimated second derivative of the samples for indices from -2 to 1 is prepared as follows:

```
for (dist = -2; dist < 2; dist++) {
    p0 = s[dist - 1]
    q0 = s[dist]
    q1 = s[dist+1]
    derivS = Abs(p0 - (q0 << 1) + q1)
    p0 = t[dist - 1]
    q0 = t[dist]
    q1 = t[dist+1]
    derivT = Abs(p0 - (q0 << 1) + q1)
    secondDeriv[dist] = (derivS + derivT + 1) >> 1
}
```

The width to return is calculated as follows:

```
if (secondDeriv[-2] > sideThr || secondDeriv[1] > sideThr) return 0
if (maxWidthPos == 1) return 1

sideThr2 = sideThr >> 2
if (secondDeriv[-2] > sideThr2 || secondDeriv[1] > sideThr2) return 1
if (secondDeriv[-1] + secondDeriv[0] > qThr * 4) return 1

sideThr3 = sideThr >> 3
if (secondDeriv[-2] > sideThr3 || secondDeriv[1] > sideThr3) return 2
if (secondDeriv[-1] + secondDeriv[0] > qThr * 3) return 2

endThr = (sideThr * 3) >> 4
if (maxWidthNeg > 2) {
```

```

derivS = Abs(s[-1] - s[-4] - 3 * (s[-1] - s[-2]))
derivT = Abs(t[-1] - t[-4] - 3 * (t[-1] - t[-2]))
if ( ((derivS + derivT + 1) >> 1) > endThr ) return 2
}
derivS = Abs(s[0] - s[3] - 3 * (s[0] - s[1]))
derivT = Abs(t[0] - t[3] - 3 * (t[0] - t[1]))
if ( ((derivS + derivT + 1) >> 1) > endThr ) return 2
if (maxWidthPos == 3) return 3

transition = (secondDeriv[-1] + secondDeriv[0]) << 4
prevDist = 3
for (dist = 4; dist <= maxWidthPos; dist += 2) {
  qThr4 = qThr * Q_First[dist - 4]
  endThr4 = (sideThr * dist) >> 4
  if (transition > qThr4) return prevDist
  dist2 = Min(7,dist)
  if ( maxWidthNeg >= dist2 ) {
    derivS = Abs(s[-1] - s[-dist2 - 1] - dist2 * (s[-1] - s[-2]))
    derivT = Abs(t[-1] - t[-dist2 - 1] - dist2 * (t[-1] - t[-2]))
    if ( ((derivS + derivT + 1) >> 1) > endThr4) return prevDist
  }
  derivS = Abs(s[0] - s[dist2] - dist2 * (s[0] - s[1]))
  derivT = Abs(t[0] - t[dist2] - dist2 * (t[0] - t[1]))
  if ( ((derivS + derivT + 1) >> 1) > endThr4) return prevDist
  prevDist = dist
}
return maxWidthPos

```

§ 7.18. CDEF process

Input to this process is the array CurrFrame of reconstructed samples.

Output from this process is the array CdefFrame containing deringed samples.

The purpose of [CDEF](#) is to perform deringing based on the detected direction of blocks.

CDEF parameters are stored for each 64 by 64 block of luma samples.

The CDEF filter is applied on each 8 by 8 block as follows:

```

step4 = Num_4x4_Blocks_Wide[ BLOCK_8X8 ]
cdefSize4 = Num_4x4_Blocks_Wide[ BLOCK_64X64 ]
cdefMask4 = ~(cdefSize4 - 1)
for ( r = 0; r < MiRows; r += step4 ) {
  for ( c = 0; c < MiCols; c += step4 ) {
    baseR = r & cdefMask4
    baseC = c & cdefMask4
    idx = cdef_idx[ baseR ][ baseC ]
    cdef_block(r, c, idx)
  }
}

```

When the cdef_block function is called, the CDEF block process specified in [§7.18.1 CDEF block process](#) is invoked with r, c, and idx as inputs.

§ 7.18.1. CDEF block process

The inputs to this process are:

- variables r and c specifying the location of an 8x8 block in units of 4x4 blocks in the luma plane,

- a variable `idx` specifying which set of CDEF parameters to use, or -1 to signal that no filtering is to be applied.

The block is first copied to the `CdefFrame` as follows:

```

startY = r * MI_SIZE
endY = startY + MI_SIZE * 2
startX = c * MI_SIZE
endX = startX + MI_SIZE * 2
for ( y = startY; y < endY; y++ ) {
    for ( x = startX; x < endX; x++ ) {
        CdefFrame[ 0 ][ y ][ x ] = CurrFrame[ 0 ][ y ][ x ]
    }
}
if ( NumPlanes > 1 ) {
    startY >>= SubsamplingY
    endY >>= SubsamplingY
    startX >>= SubsamplingX
    endX >>= SubsamplingX
    for ( y = startY; y < endY; y++ ) {
        for ( x = startX; x < endX; x++ ) {
            CdefFrame[ 1 ][ y ][ x ] = CurrFrame[ 1 ][ y ][ x ]
            CdefFrame[ 2 ][ y ][ x ] = CurrFrame[ 2 ][ y ][ x ]
        }
    }
}

```

NOTE: If CDEF filtering turns out to be needed, then the contents of `CdefFrame` will be overwritten later in this process.

If `idx` is equal to -1, then the process returns immediately after performing this copy.

The variable `coeffShift` is set equal to `BitDepth - 8`.

The variable `skip` is set as follows:

- If `cdef_on_skip_txfm_frame_enable` is equal to 0, `skip` is set equal to `( Skips[ r ][ c ] && Skips[ r + 1 ][ c ] && Skips[ r ][ c + 1 ] && Skips[ r + 1 ][ c + 1 ] )`.
- Otherwise (`cdef_on_skip_txfm_frame_enable` is equal to 1), `skip` is set equal to 0.

The variables `skip` and `skipChroma` are updated as follows:

```

skipChroma = 0
for( i = 0; i < 2; i++ ) {
    for( j = 0; j < 2; j++ ) {
        s = SegmentIds[ r + i ][ c + j ]
        skip = skip | LosslessArray[ s ]
        if ( NumPlanes > 1 ) {
            s = ChromaSegmentIds[ r + i ][ c + j ]
            skipChroma = skipChroma | LosslessArray[ s ]
        }
    }
}

```

If `skip` is equal to 0, the CDEF direction process specified in [§ 7.18.2 CDEF direction process](#) is invoked with `r` and `c` as inputs, and the outputs assigned to variables `yDir` and `var`.

If skip is equal to 0, the following ordered steps apply:

1. The variable priStr is set equal to `cdef_y_pri_strength[ idx ] << coeffShift`.
2. The variable secStr is set equal to `cdef_y_sec_strength[ idx ] << coeffShift`.
3. The variable dir is set equal to `( priStr == 0 ) ? 0 : yDir`.
4. The variable varStr is set equal to `( var >> 6 ) ? Min( FloorLog2( var >> 6 ), 12 ) : 0`.
5. The variable priStr is set equal to `( var ? ( priStr * ( 4 + varStr ) + 8 ) >> 4 : 0 )`.
6. The variable damping is set equal to `CdefDamping + coeffShift`.
7. The CDEF filter process specified in [§ 7.18.3 CDEF filter process](#) is invoked with plane equal to 0, r, c, priStr, secStr, damping, and dir as input.
8. If NumPlanes is equal to 1 or skipChroma is equal to 1, the process terminates at this point (i.e., filtering is not done for the U and V planes).
9. The variable priStr is set equal to `cdef_uv_pri_strength[ idx ] << coeffShift`.
10. The variable secStr is set equal to `cdef_uv_sec_strength[ idx ] << coeffShift`.
11. The variable dir is set equal to `( priStr == 0 ) ? 0 : Cdef_Uv_Dir[ SubsamplingX ][ SubsamplingY ][ yDir ]`.
12. The variable damping is set equal to `CdefDamping + coeffShift - 1`.
13. The CDEF filter process specified in [§ 7.18.3 CDEF filter process](#) is invoked with plane equal to 1, r, c, priStr, secStr, damping, and dir as input.
14. The CDEF filter process specified in [§ 7.18.3 CDEF filter process](#) is invoked with plane equal to 2, r, c, priStr, secStr, damping, and dir as input.

Cdef_Uv_Dir is a constant lookup table defined as:

```
Cdef_Uv_Dir[ 2 ][ 2 ][ 8 ] = {  
  { {0, 1, 2, 3, 4, 5, 6, 7},  
    {1, 2, 2, 2, 3, 4, 6, 0} },  
  { {7, 0, 2, 4, 5, 6, 6, 6},  
    {0, 1, 2, 3, 4, 5, 6, 7} }  
}
```

§ 7.18.2. CDEF direction process

The inputs to this process are variables r and c specifying the location of an 8x8 block in units of 4x4 blocks in the luma plane.

The outputs of this process are:

- a variable yDir containing the direction of this block,
- a variable var containing the variance for this block.

This block uses luma samples to measure the direction and variance of a block.

The process is specified as:

```

for ( i = 0; i < 8; i++ ) {
    cost[i] = 0
    for ( j = 0; j < 15; j++ )
        partial[i][j] = 0
}
bestCost = 0
yDir = 0
x0 = c << MI_SIZE_LOG2
y0 = r << MI_SIZE_LOG2
for ( i = 0; i < 8; i++ ) {
    for ( j = 0; j < 8; j++ ) {
        x = (CurrFrame[ 0 ][y0 + i][x0 + j] >> (BitDepth - 8)) - 128
        partial[0][i + j] += x
        partial[1][i + j / 2] += x
        partial[2][i] += x
        partial[3][3 + i - j / 2] += x
        partial[4][7 + i - j] += x
        partial[5][3 - i / 2 + j] += x
        partial[6][j] += x
        partial[7][i / 2 + j] += x
    }
}
for ( i = 0; i < 8; i++ ) {
    cost[2] += partial[2][i] * partial[2][i]
    cost[6] += partial[6][i] * partial[6][i]
}
cost[2] *= Div_Table[8]
cost[6] *= Div_Table[8]
for ( i = 0; i < 7; i++ ) {
    cost[0] += (partial[0][i] * partial[0][i] +
                partial[0][14 - i] * partial[0][14 - i]) *
                Div_Table[i + 1]
    cost[4] += (partial[4][i] * partial[4][i] +
                partial[4][14 - i] * partial[4][14 - i]) *
                Div_Table[i + 1]
}
cost[0] += partial[0][7] * partial[0][7] * Div_Table[8]
cost[4] += partial[4][7] * partial[4][7] * Div_Table[8]
for ( i = 1; i < 8; i += 2 ) {
    for ( j = 0; j < 4 + 1; j++ ) {
        cost[i] += partial[i][3 + j] * partial[i][3 + j]
    }
    cost[i] *= Div_Table[8]
    for ( j = 0; j < 4 - 1; j++ ) {
        cost[i] += (partial[i][j] * partial[i][j] +
                    partial[i][10 - j] * partial[i][10 - j]) *
                    Div_Table[2 * j + 2]
    }
}
for ( i = 0; i < 8; i++ ) {
    if ( cost[i] > bestCost ) {
        bestCost = cost[i]
        yDir = i
    }
}
var = (bestCost - cost[(yDir + 4) & 7]) >> 10

```

where the Div_Table is a constant lookup table specified as:

```

Div_Table[9] = {
    0, 840, 420, 280, 210, 168, 140, 120, 105
}

```

§ 7.18.3. CDEF filter process

The inputs to this process are:

- a variable plane specifying which plane is being predicted,
- variables r and c specifying the location of an 8x8 block in units of 4x4 blocks in the luma plane,
- a variable priStr specifying the primary filter strength,
- a variable secStr specifying the secondary filter strength,
- a variable damping specifying a shift used for damping,
- a variable dir specifying the detected direction of the block.

The process modifies samples in CdefFrame based on filtering samples from CurrFrame.

The variable coeffShift is set equal to BitDepth - 8.

The filtering is applied as follows:

```
MiColStart = MiColStartGrid[ r ][ c ]
MiRowStart = MiRowStartGrid[ r ][ c ]
MiColEnd = MiColEndGrid[ r ][ c ]
MiRowEnd = MiRowEndGrid[ r ][ c ]
subX = (plane > 0) ? SubsamplingX : 0
subY = (plane > 0) ? SubsamplingY : 0
x0 = (c * MI_SIZE) >> subX
y0 = (r * MI_SIZE) >> subY
w = 8 >> subX
h = 8 >> subY
for ( i = 0; i < h; i++ ) {
    for ( j = 0; j < w; j++ ) {
        sum = 0
        x = CurrFrame[plane][y0 + i][x0 + j]
        max = x
        min = x
        for ( k = 0; k < 2; k++ ) {
            for ( sign = -1; sign <= 1; sign += 2 ) {
                p = cdef_get_at(plane, x0, y0, i, j, dir, k, sign, subX, subY)
                if ( CdefAvailable ) {
                    sum += Cdef_Pri_Taps[(priStr >> coeffShift) & 1][k] *
                        constrain(p - x, priStr, damping )
                    max = Max(p, max)
                    min = Min(p, min)
                }
                for ( dirOff = -2; dirOff <= 2; dirOff += 4 ) {
                    s = cdef_get_at( plane, x0, y0, i, j, (dir + dirOff) & 7, k,
                        sign, subX, subY)
                    if ( CdefAvailable ) {
                        sum += Cdef_Sec_Taps[(priStr >> coeffShift) & 1][k] *
                            constrain(s - x, secStr, damping )
                        max = Max(s, max)
                        min = Min(s, min)
                    }
                }
            }
        }
        CdefFrame[plane][y0 + i][x0 + j] =
            Clip3(min, max, x + ((8 + sum - (sum < 0)) >> 4) )
    }
}
```


where Cdef_Pri_Taps and Cdef_Sec_Taps are constant lookup tables specified as:

```
Cdef_Pri_Taps[2][2] = {
    { 4, 2 }, { 3, 3 }
}

Cdef_Sec_Taps[2][2] = {
    { 2, 1 }, { 2, 1 }
}
```

constrain is specified as:

```
constrain(diff, threshold, damping) {
    if ( !threshold )
        return 0
    dampingAdj = Max(0, damping - FloorLog2( threshold ) )
    sign = (diff < 0) ? -1 : 1
    return sign * Clip3(0, Abs(diff), threshold - (Abs(diff) >> dampingAdj) )
}
```

cdef_get_at fetches a sample from CurrFrame and sets CdefAvailable according to whether the sample is available. cdef_get_at is specified as:

```
cdef_get_at(plane, x0, y0, i, j, dir, k, sign, subX, subY) {
    y = y0 + i + sign * Cdef_Directions[dir][k][0]
    x = x0 + j + sign * Cdef_Directions[dir][k][1]
    candidateR = (y << subY) >> MI_SIZE_LOG2
    candidateC = (x << subX) >> MI_SIZE_LOG2
    if ( is_inside_filter_region( candidateR, candidateC ) ) {
        CdefAvailable = 1
        return CurrFrame[ plane ][ y ][ x ]
    } else {
        CdefAvailable = 0
        return 0
    }
}
```

where Cdef_Directions is a constant lookup table defined as:

```
Cdef_Directions[8][2][2] = {
    { { -1, 1 }, { -2, 2 } },
    { { 0, 1 }, { -1, 2 } },
    { { 0, 1 }, { 0, 2 } },
    { { 0, 1 }, { 1, 2 } },
    { { 1, 1 }, { 2, 2 } },
    { { 1, 0 }, { 2, 1 } },
    { { 1, 0 }, { 2, 0 } },
    { { 1, 0 }, { 2, -1 } }
}
```

§ 7.19. CCSO process

Input to this process is the array CurrFrame of reconstructed samples and the array CdefFrame of samples that have had [CDEF](#) applied.

This process modifies the samples in CdefFrame.

A CCSO enable bit is stored for each plane for each $(1 \ll \text{CcsolumaSizeLog2})$ by $(1 \ll \text{CcsolumaSizeLog2})$ block of luma samples.

The following applies for plane=0..NumPlanes-1:

- If `ccso_planes[plane]` is equal to 1, the apply CCSO filter process in [§ 7.19.1 Apply CCSO filter process](#) is invoked with plane as input.

§ 7.19.1. Apply CCSO filter process

The input to this process is a variable plane specifying which plane is being modified.

This process modifies the samples in `CdefFrame[plane]`.

Variables `subX` and `subY` are prepared as follows:

```
if ( plane == 0 ) {
    subX = 0
    subY = 0
} else {
    subX = SubsamplingX
    subY = SubsamplingY
}
```

Variables `blkW`, `blkH` representing the CCSO block size in units of samples in the current plane are derived as follows:

```
shiftY = CcsolumaSizeLog2 - subY
shiftX = CcsolumaSizeLog2 - subX
blkH = 1 << shiftY
blkW = 1 << shiftX
```

The filtering is applied as follows:

```
planeWidth = MiCols * MI_SIZE >> subX
planeHeight = MiRows * MI_SIZE >> subY
maxBandLog2 = ccso_max_band_log2[plane]
extFilter = ccso_ext_filter[plane]
quantStep = CCSO_Quant_Sz[ ccso_scale_idx[plane] ][ ccso_quant_idx[plane] ]
dy = Ccso_Pos[extFilter][0]
dx = Ccso_Pos[extFilter][1]
for(y = 0; y < planeHeight; y += blkH) {
    for(x = 0; x < planeWidth; x += blkW) {
        unitRow = y >> shiftY
        unitCol = x >> shiftX
        useCcs = CcsoBlks[ plane ][ unitRow ][ unitCol ]
        sbBlkW = Block_Width[ SbSize ] >> subX
        sbBlkH = Block_Height[ SbSize ] >> subY
        for (y2 = y; y2 < Min(planeHeight, y + blkH); y2 += sbBlkH) {
            for (x2 = x; x2 < Min(planeWidth, x + blkW); x2 += sbBlkW) {
                if ( useCcs && BruModes[y2 >> (MI_SIZE_LOG2 - subY)]
                    [x2 >> (MI_SIZE_LOG2 - subX)] ==
                    BRU_ACTIVE ) {
                    shift = BitDepth - maxBandLog2
                    for(y3 = y2; y3 < Min(planeHeight, y2 + sbBlkH); y3++) {
                        for(x3 = x2; x3 < Min(planeWidth, x2 + sbBlkW); x3++) {
                            yLuma = y3 << subY
                            xLuma = x3 << subX
                            row = yLuma >> MI_SIZE_LOG2
                            col = xLuma >> MI_SIZE_LOG2
                            s = ( plane > 0 ) ? ChromaSegmentIds[row][col] :
```


and CCSO_Quant_Sz is defined as:

```
CCSO_Quant_Sz[4][4] = {
    { 16, 8, 32, 0 },
    { 56, 40, 64, 128 },
    { 48, 24, 96, 192 },
    { 80, 112, 160, 256 }
}
```

NOTE: If edgeClassifier is 0, different classes are used for positive and negative significant differences. If edgeClassifier is 1, positive significant differences are treated the same as there being no difference.

The table Ccso_Pos is defined as:

```
Ccso_Pos[7][2] = {
    {-1, 0},
    {0, -1},
    {-1, -1},
    {-1, 1},
    {-1, -2},
    {1, -2},
    {0, 2}
}
```

§ 7.20. Loop restoration process

Input to this process are the arrays CurrFrame (of reconstructed samples) and CdefFrame (of deringed samples).

Output from this process is the array LrFrame of loop restored samples.

NOTE: Although this process loops over 4x4 blocks, loop restoration is designed to work in stripes 64 luma samples high without needing additional line buffers. Samples within the current stripe are fetched from CdefFrame. Samples outside the current stripe are fetched from CurrFrame (these samples will be deblocked, but will not have CDEF and CCSO filtering applied).

The array LrFrame is set equal to a copy of CdefFrame. (The contents of LrFrame will later be overwritten for blocks that require restoration filtering.)

If UsesLr is equal to 0 and gdf_frame_enable is equal to 0, then the process returns immediately after performing this copy.

Otherwise, loop restoration is applied as follows:

```
for ( plane = 0; plane < NumPlanes; plane++ ) {
    for ( y = 0; y < MiRows * MI_SIZE; y += MI_SIZE ) {
        for ( x = 0; x < MiCols * MI_SIZE; x += MI_SIZE ) {
            if ( FrameRestorationType[ plane ] != RESTORE_NONE ||
                ( plane==0 && gdf_frame_enable ) ) {
                row = y >> MI_SIZE_LOG2
                col = x >> MI_SIZE_LOG2
                loop_restore_block( plane, row, col )
            }
        }
    }
}
```

```

    }
  }
}

```

When `loop_restore_block` is called, the loop restore block process in [§ 7.20.1 Loop restore block process](#) is invoked with `plane`, `row`, and `col` as inputs.

§ 7.20.1. Loop restore block process

The inputs to this process are:

- a variable `plane` specifying whether the process is filtering Y, U, or V samples,
- variables `row` and `col` specifying the location of the block in units of 4x4 blocks in the upsampled luma plane.

The output of this process are samples in `LrFrame[ plane ]`.

The variable `unitSize` (specifying the size of restoration units in units of samples in the current plane) is set as follows:

- If `FrameRestorationType[ plane ]` is equal to `RESTORE_NONE`, `unitSize` is set equal to `RESTORATION_TILESIZE_MAX`.
- Otherwise (`FrameRestorationType[ plane ]` is not equal to `RESTORE_NONE`), `unitSize` is set equal to `LoopRestorationSize[ plane ]`.

The variables `subX` and `subY` are set equal to the subsampling for the current plane as follows:

- If `plane` is equal to 0, `subX` is set equal to 0 and `subY` is set equal to 0.
- Otherwise, `subX` is set equal to `SubsamplingX` and `subY` is set equal to `SubsamplingY`.

If `plane` is equal to 0 and `LosslessArray[SegmentIds[ row ][ col ]]` is equal to 1, this process terminates immediately.

If `plane` is greater than 0 and `LosslessArray[ChromaSegmentIds[ row ][ col ]]` is equal to 1, this process terminates immediately.

The variable `x` is set equal to `col * MI_SIZE >> subX`.

The variable `y` is set equal to `row * MI_SIZE >> subY`.

(Variables `x` and `y` represent the position of the block in samples relative to the top-left corner of the current plane.)

The variable `MiColStart` is set equal to `MiColStartGrid[ row ][ col ]`.

The variable `MiColEnd` is set equal to `MiColEndGrid[ row ][ col ]`.

The variable `MiRowStart` is set equal to `MiRowStartGrid[ row ][ col ]`.

The variable `MiRowEnd` is set equal to `MiRowEndGrid[ row ][ col ]`.

The variable `lrRowOffset` is set equal to $(\text{MiRowStart} * \text{MI_SIZE} \gg \text{subY}) / \text{unitSize}$.

The variable `lrColOffset` is set equal to $(\text{MiColStart} * \text{MI_SIZE} \gg \text{subX}) / \text{unitSize}$.

The variable `sbShift` is set equal to $\text{Mi_Width_Log2}[\text{SbSize}]$.

The variable `stripeRow` (specifying the row of the start of the stripe in units of 4x4 blocks) is set equal to $\text{Min}(\text{MiRowEnd} - 1, ((\text{row} + 2) \gg 4) \ll 4)$.

If `use_bru` is equal to 1 and $\text{BruModes}[(\text{stripeRow} \gg \text{sbShift}) \ll \text{sbShift}][(\text{col} \gg \text{sbShift}) \ll \text{sbShift}]$ is not equal to `BRU_ACTIVE`, this process terminates immediately.

The variable `col` is set equal to $\text{col} - \text{MiColStart}$.

The variable `row` is set equal to $\text{row} - \text{MiRowStart}$.

The variable `miCols` is set equal to $\text{MiColEnd} - \text{MiColStart}$.

The variable `miRows` is set equal to $\text{MiRowEnd} - \text{MiRowStart}$.

The variable `lumaY` is set equal to $\text{row} * \text{MI_SIZE}$.

The variable `stripeNum` (specifying the zero-based index of the current stripe) is set equal to $(\text{lumaY} + 8) / 64$.

NOTE: The stripes are offset upwards by 8 luma samples to make pipelined implementations more efficient. When a row of superblocks has been received, enough rows of deblocked output can be produced to allow loop restoration of the corresponding stripes.

The variable `unitRows` (specifying the number of restoration units down the frame) is set equal to $\text{count_units_in_frame}(\text{unitSize}, \text{miRows} * \text{MI_SIZE} \gg \text{subY})$.

The variable `unitCols` (specifying the number of restoration units across the frame) is set equal to $\text{count_units_in_frame}(\text{unitSize}, \text{miCols} * \text{MI_SIZE} \gg \text{subX})$.

NOTE: The number of restoration units in a frame can be different for chroma and luma.

The variable `unitRow` (specifying the vertical index of the current loop restoration unit) is set equal to $\text{lrRowOffset} + \text{Min}(\text{unitRows} - 1, ((\text{row} * \text{MI_SIZE} + 8) \gg \text{subY}) / \text{unitSize})$.

The variable `unitCol` (specifying the horizontal index of the current loop restoration unit) is set equal to $\text{lrColOffset} + \text{Min}(\text{unitCols} - 1, (\text{col} * \text{MI_SIZE} \gg \text{subX}) / \text{unitSize})$.

The horizontal extent of the space allowed for filtering is specified as follows:

The variable `w` is set equal to $\text{MI_SIZE} \gg \text{subX}$.

The variable `h` is set equal to $\text{MI_SIZE} \gg \text{subY}$.

(Variables `w` and `h` represent the size of the block in samples.)

NOTE: Although the filter is described as operating on small blocks, the output will be the same if larger blocks are used - provided all contained samples belong to the same loop restoration unit.

The variable `unclippedStripeStartY` is set equal to `MiRowStart * MI_SIZE + stripeNum * 64 - 8`.

The variable `unclippedStripeEndY` is set equal to `unclippedStripeStartY + 64`.

The variables representing which luma pixels are allowed to be accessed are set as follows:

```
if ( disable_loopfilters_across_tiles ) {
    LumaStartX = MiColStart * MI_SIZE
    LumaEndX = MiColEnd * MI_SIZE - 1
    LumaStartY = MiRowStart * MI_SIZE
    LumaEndY = MiRowEnd * MI_SIZE - 1
} else {
    LumaStartX = 0
    LumaEndX = MiCols * MI_SIZE - 1
    LumaStartY = 0
    LumaEndY = MiRows * MI_SIZE - 1
}
LumaStripeStartY = Max( LumaStartY, unclippedStripeStartY )
LumaStripeEndY = Min( LumaEndY, unclippedStripeEndY - 1 )
```

The variable `rType` (specifying the loop restoration type) is set as follows:

- If `FrameRestorationType[ plane ]` is equal to `RESTORE_NONE`, `rType` is set equal to `RESTORE_NONE`.
- Otherwise (`FrameRestorationType[ plane ]` is not equal to `RESTORE_NONE`), `rType` is set equal to `LrType[ plane ][ unitRow ][ unitCol ]`.

The filter to be used depends on `rType` as follows:

- If `rType` is equal to `RESTORE_WIENER_NONSEP`, the following ordered steps apply:
 1. If `frame_filters_on[ plane ]` is equal to 1 and `plane` is equal to 0 and `NumFilterClasses` is greater than 1, the pixel-classified Wiener filter process specified in [§ 7.20.4 Pixel classified Wiener filter process](#) is invoked with `x`, `y`, `w`, `h`, 1 as inputs.
 2. The non-separable Wiener filter process specified in [§ 7.20.3 Non-separable Wiener filter process](#) is invoked with `plane`, `unitRow`, `unitCol`, `x`, `y`, `w`, and `h` as inputs.
- Otherwise, if `rType` is equal to `RESTORE_PC_WIENER`, the pixel-classified Wiener filter process specified in [§ 7.20.4 Pixel classified Wiener filter process](#) is invoked with `x`, `y`, `w`, `h`, 0 as inputs.
- Otherwise (`rType` is equal to `RESTORE_NONE`), no filtering is applied.

The guided detail filter is conditionally applied on this block as follows:

```
if ( plane == 0 && gdf_frame_enable &&
    ( gdf_per_block == 0 ||
      GdfBlks[stripeRow * MI_SIZE / GdfBlkSize][x / GdfBlkSize] ) ) {
    qpBase = FrameIsIntra ? 85 : 110
    qpDiff = base_q_idx - qpBase - 24 * (BitDepth - 8)
    qpIdx = Clip3( 0, 2, (qpDiff - 37)/25 ) + gdf_pic_qc_idx
    if (FrameIsIntra) {
        refDstIdx = 0
    } else {
        maxDist = 0
        for(i = 0; i < Min( NumTotalRefs, 2); i++ ) {
            if ( OrderHints[ i ] != RESTRICTED_OH ) {
                maxDist = Max( Abs(FrameDistance[i]), maxDist )
            }
        }
    }
}
```

```

    if (maxDist == 0)
        refDstIdx = 5
    else if (maxDist < 2)
        refDstIdx = 1
    else if (maxDist < 3)
        refDstIdx = 2
    else if (maxDist < 6)
        refDstIdx = 3
    else if (maxDist < 11)
        refDstIdx = 4
    else
        refDstIdx = 5
}
apply_gdf_filter(x,y,qpIdx,refDstIdx,4,4,unclippedStripeEndY)
}

```

The function call to `apply_gdf_filter` indicates that the apply GDF filter process specified in [§ 7.20.5 Apply GDF filter process](#) is invoked.

§ 7.20.2. Get source sample process

The inputs to this process are:

- a variable plane specifying whether the process is filtering Y, U, or V samples,
- variables x and y specifying the location in the current plane in units of samples.

This process makes sure samples are taken from within the allowed extent for loop restoration filtering.

Samples within the current stripe are taken after CDEF and CCSO filtering has been applied, samples outside the current stripe are taken before CDEF and CCSO filtering.

The sample to return is specified as follows:

```

subX = (plane == 0) ? 0 : SubsamplingX
subY = (plane == 0) ? 0 : SubsamplingY
x = Clip3( LumaStartX >> subX, LumaEndX >> subX, x )
y = Clip3( LumaStartY >> subY, LumaEndY >> subY, y )
stripeStartY = LumaStripeStartY >> subY
stripeEndY = LumaStripeEndY >> subY
if (y < stripeStartY) {
    y = Max(stripeStartY - 2,y)
    return CurrFrame[ plane ][ y ][ x ]
} else if (y > stripeEndY) {
    y = Min(stripeEndY + 2,y)
    return CurrFrame[ plane ][ y ][ x ]
} else {
    return CdefFrame[ plane ][ y ][ x ]
}

```

NOTE: This process can be called for samples on the lines above and lines below the current stripe. However, the coordinates are cropped such that only two lines above and below the stripe need to be fetched. In other words, requests for the third line (above or below) are given a copy of the second line.

§ 7.20.3. Non-separable Wiener filter process

The inputs to this process are:

- a variable plane specifying whether the process is filtering Y, U, or V samples,
- variables unitRow and unitCol specifying the position of the loop restoration unit,
- variables x and y specifying the position of the block in samples relative to the top-left corner of the current plane,
- variables w and h specifying the size of the block in samples.

The output from this process are modified samples in LrFrame.

For luma this process applies a non-separable filter to the luma samples.

For chroma this process applies a non-separable filter to the chroma samples that includes taps from both chroma and luma samples.

The filtering is applied as follows:

```
if (plane==0) {
    nTaps = WIENER_NS_TAPS_Y
    config = Wiener_Ns_Config_Y
} else {
    nTaps = WIENER_NS_TAPS_UV
    config = Wiener_Ns_Config_Uv
}
for ( r = 0; r < h; r++ ) {
    for ( c = 0; c < w; c++ ) {
        m = get_source_sample(plane, x + c, y + r)
        s = m << WIENER_NS_PREC_BITS
        if ( plane == 0 && frame_filters_on[ plane ] && NumFilterClasses > 1 ) {
            cls = FilterClass[ (y + r) >> 2 ][ (x + c) >> 2 ]
            subcls = SubclassLookup[ cls ]
        } else {
            subcls = 0
        }
        for ( i = 0; i < nTaps; i++ ) {
            dy = config[ i ][ 0 ]
            dx = config[ i ][ 1 ]
            idx = config[ i ][ 2 ]
            diff = get_source_sample( plane, x + c + dx, y + r + dy ) - m
            if ( frame_filters_on[ plane ] ) {
                coeff = FrameLrWienerNs[ plane ][ subcls ][ idx ]
            } else {
                coeff = LrWienerNs[ plane ][ unitRow ][ unitCol ][ idx ]
            }
            s += diff * coeff
        }
        if (plane > 0) {
            mLuma = get_luma_sample(x + c, y + r)
            for ( i = 0; i < nTaps; i++ ) {
                if ( frame_filters_on[ plane ] ) {
                    coeff = FrameLrWienerNs[ plane ][ 0 ][ i + 6 ]
                } else {
                    coeff = LrWienerNs[ plane ][ unitRow ][ unitCol ][ i + 6 ]
                }
                if ( coeff != 0 ) {
                    dy = config[ i ][ 0 ]
                    dx = config[ i ][ 1 ]
                    lumaDiff = get_luma_sample( x + c + dx, y + r + dy ) - mLuma
                    s += lumaDiff * coeff
                }
            }
        }
    }
}
```

```

    }
    v = Round2( s, WIENER_NS_PREC_BITS )
    LrFrame[ plane ][ y + r ][ x + c ] = Clip1( v )
}
}

```

The function calls to `get_source_sample` indicate that the get source sample process specified in [§7.20.2 Get source sample process](#) is invoked.

The constant tables `Wiener_Ns_Config_Y` and `Wiener_Ns_Config_Uv` are defined as:

```

Wiener_Ns_Config_Y[WIENER_NS_TAPS_Y][3] = {
    { 1, 0, 0 }, { -1, 0, 0 }, { 0, 1, 1 }, { 0, -1, 1 }, { 2, 0, 2 },
    { -2, 0, 2 }, { 0, 2, 3 }, { 0, -2, 3 }, { 1, 1, 4 }, { -1, -1, 4 },
    { -1, 1, 5 }, { 1, -1, 5 }, { 2, 1, 6 }, { -2, -1, 6 }, { 2, -1, 7 },
    { -2, 1, 7 }, { 1, 2, 8 }, { -1, -2, 8 }, { 1, -2, 9 }, { -1, 2, 9 },
    { 3, 0, 10 }, { -3, 0, 10 }, { 0, 3, 11 }, { 0, -3, 11 },
    { 4, 0, 12 }, { -4, 0, 12 }, { 0, 4, 13 }, { 0, -4, 13 }, { 3, 3, 14 },
    { -3, -3, 14 }, { 3, -3, 15 }, { -3, 3, 15 }
}

Wiener_Ns_Config_Uv[WIENER_NS_TAPS_UV][3] = {
    { 1, 0, 0 }, { -1, 0, 0 }, { 0, 1, 1 }, { 0, -1, 1 },
    { 1, 1, 2 }, { -1, -1, 2 }, { -1, 1, 3 }, { 1, -1, 3 },
    { 2, 0, 4 }, { -2, 0, 4 }, { 0, 2, 5 }, { 0, -2, 5 }
}

```

The function `get_luma_sample` gets a filtered sample from luma as follows:

```

get_luma_sample(x, y) {
    subX = SubsamplingX
    subY = SubsamplingY
    lastY = MiRows * MI_SIZE - 1 - subY
    lastX = LumaEndX - subX
    x = x << subX
    y = y << subY
    y = Clip3( 0, lastY, y )
    x = Clip3( LumaStartX, lastX, x )
    filterIdx = cfl_ds_filter_index
    if (filterIdx == 3) {
        filterIdx = 0
    }
    if (subX && subY && filterIdx <= 1) {
        t = 0
        for (dy = 0; dy < 2; dy++) {
            for (dx = 0; dx < 2; dx++) {
                v = get_luma_source_sample(x + dx, y + dy)
                t += Wiener_Filters_420[filterIdx][dy][dx] * v
            }
        }
        return t >> 2
    } else {
        return get_luma_source_sample(x, y)
    }
}

```

The constant table `Wiener_Filters_420` is specified as:

```

Wiener_Filters_420[2][2][2] = {
    {
        {1, 1},
        {1, 1}
    },
}

```

```

{
    {2, 0},
    {2, 0}
}
}

```

The function `get_luma_source_sample` gets a sample from the luma stripe as follows:

```

get_luma_source_sample( x, y ) {
    return get_source_sample( 0, x, y )
}

```

§ 7.20.4. Pixel classified Wiener filter process

The inputs to this process are:

- variables `x` and `y` specifying the position of the block in samples relative to the top-left corner of the current plane,
- variables `w` and `h` specifying the size of the block in samples,
- a variable `skipFilter` specifying whether to only apply the pixel classification.

The output from this process are modified luma samples in `LrFrame`.

The variable `BlockStartX` (containing the start `x` location rounded to units of 64 by 64 luma samples) is set equal to `(x >> 6) << 6`.

The variable `BlockEndX` (containing the last on-screen `x` location in the current 64x64) is set equal to `Min(MiColEnd * MI_SIZE - 1, BlockStartX + 63)`.

The variable `qindex` is set equal to `base_q_idx`.

The variable `index` is set equal to `get_filter_set_index(qindex)`.

The variable `cls` (representing the pixel class) is computed as follows:

```

(f, tskip) = get_box_features( x, y )
lutInput = 0
for ( i = 0; i < PC_WIENER_NUM_FEATURES; i++ ) {
    qval = Round2Signed( f[i] + get_qval_given_tskip(qindex, tskip, i),
                        PC_WIENER_PREC_FEATURE )
    qval = Clip3(0,255,qval) >> 5
    lutInput += qval << (3 * (3-i))
}
cls = Pc_Wiener_Lut_To_Class[ lutInput ]

```

If `skipFilter` is equal to 1, the class is saved by setting `FilterClass[ y >> 2 ][ x >> 2 ]` equal to `cls`, and the process immediately terminates.

Otherwise (`skipFilter` is equal to 0), the filtering is applied as follows:

```

filt = Pc_Wiener_Sub_Classify[ index ][ cls ]
for ( r2 = 0; r2 < h; r2++ ) {
    for ( c2 = 0; c2 < w; c2++ ) {
        m = get_source_sample(0, x + c2, y + r2)
        s = m << PC_WIENER_PREC_BITS
        for ( i = 0; i < PC_WIENER_TAPS; i++ ) {

```

```

        coeff = Pc_Wiener_Filters[ index ][ filt ][ i >> 1 ]
        s += get_pc_wiener_sample( x + c2 , y + r2, i ) * coeff
    }
    v = Round2( s, PC_WIENER_PREC_BITS )
    LrFrame[ 0 ][ y + r2 ][ x + c2 ] = Clip1( v )
}
}

```

The functions `get_pc_wiener_sample`, `get_box_features`, and `get_qual_given_tskip` are defined as follows:

```

Pc_Wiener_Config[ PC_WIENER_TAPS ][ 2 ] = {
    { 1, 0 }, { -1, 0 }, { 0, 1 }, { 0, -1 }, { 2, 0 },
    { -2, 0 }, { 0, 2 }, { 0, -2 }, { 1, 1 }, { -1, -1 },
    { -1, 1 }, { 1, -1 }, { 2, 1 }, { -2, -1 }, { 2, -1 },
    { -2, 1 }, { 1, 2 }, { -1, -2 }, { 1, -2 }, { -1, 2 },
    { 3, 0 }, { -3, 0 }, { 0, 3 }, { 0, -3 }, { 0, 0 }
}

get_pc_wiener_sample( x, y, i ) {
    dy = Pc_Wiener_Config[i][0]
    dx = Pc_Wiener_Config[i][1]
    return get_source_sample(0, x + dx, y + dy)
}

Pc_Wiener_Normalizer[ PC_WIENER_NUM_FEATURES + 1 ] = {
    0, 3739, 3273, 3074, 7
}

get_box_features(x, y) {
    for( i = 0; i < PC_WIENER_NUM_FEATURES; i++) {
        f[i] = 0
    }
    s = 0
    for(dy = -PC_WIENER_LEAD; dy <= PC_WIENER_LAG; dy++) {
        for(dx = -PC_WIENER_LEAD; dx <= PC_WIENER_LAG; dx++) {
            (tf, skip) = get_features(x + dx, y + dy)
            for( i = 0; i < PC_WIENER_NUM_FEATURES; i++ ) {
                f[i] += tf[i]
            }
            s += skip
        }
    }
    for(i = 0; i < PC_WIENER_NUM_FEATURES; i++) {
        nf[i] = Round2( f[i] * Pc_Wiener_Normalizer[i], BitDepth - 8 )
    }
    ns = s * Pc_Wiener_Normalizer[ PC_WIENER_NUM_FEATURES ]
    return (nf, ns)
}

get_features(x, y) {
    x = Min(BlockEndX + 2, x)

    m = get_source_sample(0, x, y)

    up = get_source_sample(0, x, y - 1)
    down = get_source_sample(0, x, y + 1)
    vert = up - 2 * m + down

    upright = get_source_sample(0, x + 1, y - 1)
    downleft = get_source_sample(0, x - 1, y + 1)
    antiDiag = upright - 2 * m + downleft

    downright = get_source_sample(0, x + 1, y + 1)
    upleft = get_source_sample(0, x - 1, y - 1)
    diag = upleft - 2 * m + downright

    f[0] = 0
    f[1] = Abs(vert)
    f[2] = Abs(antiDiag)
}

```

```

    f[3] = Abs(diag)
    return (f, get_tx_skip(x, y))
}

get_tx_skip( x, y ) {
    x = Min( BlockEndX, x )
    x = Max( BlockStartX, x )
    y = Clip3( LumaStripeStartY, LumaStripeEndY, y )
    tileStartY = MiRowStart * MI_SIZE
    tileEndY = MiRowEnd * MI_SIZE - 1
    y = Clip3( tileStartY, tileEndY, y )
    return LrTxSkip[ y >> 2 ][ x >> 2 ]
}

Mode_Weights[ PC_WIENER_NUM_FEATURES ][ 3 ] = {
    { -527, 15325, 321 },
    { 26436, -17705, 17905 },
    { 366, -147, -194 },
    { 202, -267, -179 }
}

Mode_Offsets[ PC_WIENER_NUM_FEATURES ] = {
    -547, -21565, -573, -680
}

get_qual_given_tskip(qindex, tskip, i) {
    qstep = get_q(qindex, 0)
    qstepShift = QUANT_TABLE_BITS + 10
    qstep = Round2(qstep, BitDepth - 8)
    diffShift = qstepShift - 8
    prod = Round2(tskip * qstep, 8)
    qval = (Mode_Weights[ i ][ 0 ] * (tskip << diffShift)) +
           (Mode_Weights[ i ][ 1 ] * qstep) +
           (Mode_Weights[ i ][ 2 ] * prod)
    return 255 * ( Mode_Offsets[ i ] + Round2Signed(qval, qstepShift) )
}

```

NOTE: Pc_Wiener_Normalizer[0] is equal to 0, so the value of the first feature does not influence the decoding process.

§ 7.20.5. Apply GDF filter process

The inputs to this process are:

- variables x and y specifying the location of the top-left luma sample in a GDF unit,
- variables qpIdx and refDstIdx specifying which set of tables are active,
- variables w and h specifying the size of the GDF unit,
- a variable stripeEndY specifying the unclipped end of the current 64 pixel high stripe.

The curvature in different directions is estimated in the array grad as follows:

```

for( i = 0; i < h + 2; i++ ) {
    for( j = 0; j < w + 2; j++ ) {
        for( d = 0; d < 4; d++ ) {
            if (d == GDF_VER) {
                dx = 0
                dy = 1
            } else if (d == GDF_HOR) {
                dx = 1
                dy = 0
            } else if (d == GDF_DIAG0) {
                dx = 1
            }
        }
    }
}

```

```

        dy = 1
    } else {
        dx = 1
        dy = -1
    }
    a = get_gdf_sample( x - 1 + j - dx, y - 1 + i - dy )
    b = get_gdf_sample( x - 1 + j, y - 1 + i )
    c = get_gdf_sample( x - 1 + j + dx, y - 1 + i + dy )
    grad[d][i][j] = Abs( b * 2 - a - c )
}
}
}

```

where the function `get_gdf_sample` (which gets a sample from the current stripe with reflection at the stripe end) is specified as:

```

get_gdf_sample( x, y ) {
    return get_luma_source_sample(x,y)
}

```

The array `gdfCls` (containing the filter class for each sample) is derived as follows:

```

for ( i = (h >> 1) - 1; i >= 0; i--) {
    for( j = 0; j < w >> 1; j++) {
        for( d = 0; d < 4; d++) {
            str[ d ] = grad_sum(grad[d],i*2,j*2,4,4)
        }
        cls = str[GDF_VER] > str[GDF_HOR] ? 0 : 1
        cls |= str[GDF_DIAG0] > str[GDF_DIAG1] ? 0 : 2
        gdfCls[ i ][ j ] = cls
    }
}

```

The function `grad_sum` sums a rectangle of the values in an array as follows:

```

grad_sum(grad,i,j,down,across) {
    t = 0
    for( i2 = 0; i2 < down; i2++ ) {
        for( j2 = 0; j2 < across; j2++ ) {
            t += grad[i + i2][j + j2]
        }
    }
    return t
}

```

NOTE: The array `grad` contains values representable by an unsigned integer with `BitDepth + 1` bits. `grad_sum` sums 16 values within `grad`. This means `grad_sum` returns values representable by an unsigned integer with `BitDepth + 5` bits.

The scaling used for this unit is prepared as follows:

```

if ( refDstIdx == 0 ) {
    scale = 8
} else {
    scale = 5
}

```

The luma samples in LrFrame are modified as follows:

```

for( i = 0; i < h; i++ ) {
  y2 = i + y
  for( j = 0; j < w; j++ ) {
    x2 = x + j
    cls = gdfCls[i >> 1][j >> 1]
    for( idx = 0; idx < 3; idx++ ) {
      gdfIdx[ idx ] = 0
    }
    for( k = 0; k < 18 + 4; k++ ) {
      alpha = Gdf_Alpha[ refDstIdx ][ qpIdx ][ k ][ cls ]
      if ( k < 18 ) {
        dy = Gdf_Coords[k][0]
        dx = Gdf_Coords[k][1]
        x3 = x2 - dx
        y3 = y2 - dy
        x4 = x2 + dx
        y4 = y2 + dy
        sample2 = get_gdf_sample(x2,y2)
        sample3 = get_gdf_sample(x3,y3)
        sample4 = get_gdf_sample(x4,y4)
        above = Clip3( -alpha, alpha,
          ( sample3 - sample2 ) <<
            (10 - Min( 10, BitDepth ) ) )
        below = Clip3( -alpha, alpha,
          ( sample4 - sample2 ) <<
            (10 - Min( 10, BitDepth ) ) )
        comb = Clip3( -512, 511, above + below )
      } else {
        d = k - 18
        v = grad_sum(grad[d], (i>>1)<<1, (j>>1)<<1, 4, 4)
        if ( BitDepth == 8 ) {
          v = v >> 2
        } else {
          v = v >> 4
        }
        comb = Min( v, alpha )
      }
      for( idx = 0; idx < 3; idx++ ) {
        gdfIdx[ idx ] +=
          comb * Gdf_Weight[ refDstIdx ][ qpIdx ][ idx ][ k ][ cls ]
      }
    }
  }
  pos = 0
  for( idx = 0; idx < 3; idx++ ) {
    v = Round2Signed(
      ( gdfIdx[ idx ] + Gdf_Bias[ refDstIdx ][ qpIdx ][ idx ] ) *
        scale, 15 )
    pos = pos * scale * 2 + Clip3( -scale, scale - 1, v ) + scale
  }
  if ( refDstIdx == 0 ) {
    err = Gdf_Intra_Error[ qpIdx ][ pos ]
  } else {
    err = Gdf_Inter_Error[ refDstIdx - 1 ][ qpIdx ][ pos ]
  }
  res = Clip1( LrFrame[ 0 ][ y2 ][ x2 ] +
    Round2Signed( err * GdfPixScale, 12 - BitDepth ) )
  LrFrame[ 0 ][ y2 ][ x2 ] = res
}
}

```

where the constant table Gdf_Coords is specified as:

```

Gdf_Coords[18][2] = {
  { 6, 0 },
  { 5, 0 },

```

```

        { 4, 0},
        { 3, 0},
        { 2, 1}, { 2, 0}, { 2, -1},
        { 1, 2}, { 1, 1}, { 1, 0}, { 1, -1}, { 1, -2},
        { 0, 6}, { 0, 5}, { 0, 4}, { 0, 3}, { 0, 2}, { 0, 1}
    }

```

§ 7.21. Output processes

§ 7.21.1. Output process

The input to this process is a variable `frameToShowMapIdx` specifying which frame to output. If `frameToShowMapIdx` is equal to -1, the process will output the current frame. Otherwise, `frameToShowMapIdx` indicates which previously decoded frame to output.

This process is invoked to prepare output frames.

The variable `mixedOutput` is set equal to `frameToShowMapIdx == -1 && ShowExistingFrame`.

If `mixedOutput` is equal to 1, `frameToShowMapIdx` is set equal to `frame_to_show_map_idx`.

If scalability is being used (bitstream contains OBUs with different values of `obu_xlayer_id`, `obu_mlayer_id`, or `obu_tlayer_id`), an application-specific function is called to decide whether this frame will be output. If this function returns a value equal to 0, then this process terminates immediately.

Applications that are displaying the decoded video should determine which frames to display based on the layer properties specified in the [LCR](#) OBUs, when present. The decision should consider:

- **`lcr_layer_type`**: Distinguishes between texture layers (`TEXTURE_LAYER`) and auxiliary layers (`AUX_LAYER`)
- **`lcr_auxiliary_type`**: For auxiliary layers, specifies the type (e.g., `ALPHA_AUX`, `DEPTH_AUX`)
- **`lcr_global_purpose_id` and `lcr_xlayer_purpose_id`**: Indicate the application purpose for the layered bitstream (e.g., stereoscopic viewports, immersive multiple viewports)

Typically, applications displaying decoded video will output texture layers (`lcr_layer_type == TEXTURE_LAYER`) while using auxiliary layers (`lcr_layer_type == AUX_LAYER`) for purposes such as transparency (alpha) or depth information, according to the indicated purpose. Applications may set their own policy about which frames and layers are output based on their specific use case and the LCR layer properties.

The intermediate output preparation process specified in [§ 7.21.2 Intermediate output preparation process](#) is invoked with `mixedOutput` and `frameToShowMapIdx` as inputs, and the outputs are assigned to `bitDepth`, `w`, `h`, `subX`, `subY`, `filmGrainPresent`, and `numPlanes`.

If `filmGrainPresent` is equal to 1 and `apply_grain` is equal to 1, then the film grain synthesis process specified in [§ 7.21.7 Film grain synthesis process](#) is invoked with inputs of `w`, `h`, `subX`, `subY`, `bitDepth`, and `numPlanes`. (This process modifies the output arrays `OutY`, `OutU`, `OutV`).

Finally, the frame to be output is defined to be the arrays `OutY`, `OutU`, `OutV` where the bit depth for each sample is `bitDepth`.

This frame to be output is the overall output of the decoding process and further processing (such as color conversion) is outside the scope of this specification.

For example, a real implementation might use these arrays to display the frame to the user, or a test system might save the arrays so the output can be verified.

NOTE: If numPlanes is equal to 1, then the U and V planes are ignored.

§ 7.21.2. Intermediate output preparation process

The inputs to this process are:

- a variable mixedOutput specifying the source for the film grain parameters,
- a variable frameToShowMapIdx specifying which frame to output.

The outputs of this process are the variables bitDepth, w, h, subX, subY, filmGrainPresent, and numPlanes describing the format of the data in arrays OutY, OutU, and OutV.

If frameToShowMapIdx is greater than or equal to 0, then the decoder sets variables and copies OutY, OutU, and OutV from a previously decoded frame as follows:

- The variable w is set equal to RefCropWidth[frameToShowMapIdx].
- The variable h is set equal to RefCropHeight[frameToShowMapIdx].
- The variable left is set equal to RefCropLeft[frameToShowMapIdx].
- The variable top is set equal to RefCropTop[frameToShowMapIdx].
- The variable subX is set equal to RefSubsamplingX[frameToShowMapIdx].
- The variable subY is set equal to RefSubsamplingY[frameToShowMapIdx].
- The array OutY is w samples across by h samples down and the sample at location x samples across and y samples down is given by $OutY[y][x] = FrameStore[frameToShowMapIdx][0][y + top][x + left]$ with $x = 0..w - 1$ and $y = 0..h - 1$.
- The array OutU is $(w + subX) \gg subX$ samples across by $(h + subY) \gg subY$ samples down and the sample at location x samples across and y samples down is given by $OutU[y][x] = FrameStore[frameToShowMapIdx][1][y + (top \gg subY)][x + (left \gg subX)]$ with $x = 0..((w + subX) \gg subX) - 1$ and $y = 0..((h + subY) \gg subY) - 1$.
- The array OutV is $(w + subX) \gg subX$ samples across by $(h + subY) \gg subY$ samples down and the sample at location x samples across and y samples down is given by $OutV[y][x] = FrameStore[frameToShowMapIdx][2][y + (top \gg subY)][x + (left \gg subX)]$ with $x = 0..((w + subX) \gg subX) - 1$ and $y = 0..((h + subY) \gg subY) - 1$.
- The variable bitDepth is set equal to RefBitDepth[frameToShowMapIdx].
- The variable numPlanes is set equal to RefNumPlanes[frameToShowMapIdx].
- The variable filmGrainPresent is set equal to RefFilmGrainPresent[frameToShowMapIdx].
- If filmGrainPresent is equal to 1, the function load_grain_params is invoked with mixedOutput ? NUM_REF_FRAMES : frameToShowMapIdx as input.

Otherwise (frameToShowMapIdx is equal to -1), then the decoder sets variables and copies the current frame as follows:

- The variable w is set equal to CropWidth.

- The variable `h` is set equal to `CropHeight`.
- The variable `subX` is set equal to `SubsamplingX`.
- The variable `subY` is set equal to `SubsamplingY`.
- The array `OutY` is `w` samples across by `h` samples down and the sample at location `x` samples across and `y` samples down is given by `OutY[ y ][ x ] = LrFrame[ 0 ][ y + CropTop ][ x + CropLeft ]` with `x = 0..w - 1` and `y = 0..h - 1`.
- The array `OutU` is `(w + subX) >> subX` samples across by `(h + subY) >> subY` samples down and the sample at location `x` samples across and `y` samples down is given by `OutU[ y ][ x ] = LrFrame[ 1 ][ y + (CropTop >> subY) ][ x + (CropLeft >> subX) ]` with `x = 0..((w + subX) >> subX) - 1` and `y = 0..((h + subY) >> subY) - 1`.
- The array `OutV` is `(w + subX) >> subX` samples across by `(h + subY) >> subY` samples down and the sample at location `x` samples across and `y` samples down is given by `OutV[ y ][ x ] = LrFrame[ 2 ][ y + (CropTop >> subY) ][ x + (CropLeft >> subX) ]` with `x = 0..((w + subX) >> subX) - 1` and `y = 0..((h + subY) >> subY) - 1`.
- The variable `bitDepth` is set equal to `BitDepth`.
- The variable `numPlanes` is set equal to `NumPlanes`.
- The variable `filmGrainPresent` is set equal to `film_grain_params_present`.
- If `filmGrainPresent` is equal to 1, the function `load_grain_params` is invoked with `NUM_REF_FRAMES` as input.

The function `load_grain_params(idx)` indicates that all the syntax elements read in both `film_grain_model` and `film_grain_config` should be set equal to the values stored in an area of memory indexed by `idx`.

The output of this process are the variables `bitDepth`, `w`, `h`, `subX`, `subY`, `filmGrainPresent`, and `numPlanes`.

§ 7.21.3. Output successive frames process

The input to this process is a variable `orderHint` specifying the order hint (with additional bits for the embedded layer) for the current frame.

This process outputs additional frame buffers if they have successive order hints.

The variable `k` is set equal to 1.

While `k` is less than or equal to `NumRefFrames`, the following ordered steps apply:

1. The output implicit output frame process specified in [§ 7.21.4 Output implicit output frame process](#) is invoked with `orderHint + k` as input, and the output is assigned to the variable `madeOutput`.
2. If `madeOutput` is equal to 0, the process immediately terminates.
3. The variable `k` is incremented by 1.

§ 7.21.4. Output implicit output frame process

The input to this process is the variable `targetHint`.

The process examines the frames in the frame buffer and outputs any implicit output frames that match the target order hint as follows:

```
madeOutput = 0
for( i = 0; i < NumRefFrames; i++ ) {
    if ( output_ordering(i) == targetHint &&
        is_frame_eligible_for_output(i) ) {
        output_process( i )
        madeOutput = 1
    }
}
```

where `output_process( i )` denotes an invocation of the output process specified in [§ 7.21.1 Output process](#) with `frameToShowMapIdx` equal to `i`.

The function `is_frame_eligible_for_output(refIdx)` is specified as follows:

- `RefImplicitOutputFrame[ refIdx ]` has been written and is equal to 1 and `RefValid[ refIdx ]` is equal to 1 and the frame has not already been output by the output process specified in [§ 7.21.1 Output process](#) and `RefOrderHint[ refIdx ]` is not equal to `RESTRICTED_OH`, the function returns 1.
- Otherwise (`RefImplicitOutputFrame[ refIdx ]` is equal to 0, or `RefValid[ refIdx ]` is equal to 0, or the frame has already been output, or `RefOrderHint[ refIdx ]` is equal to `RESTRICTED_OH`), the function returns 0.

However, when considering whether a frame has been output by the output process, invocations of the output process with `frameToShowMapIdx` less than 0 and `ShowExistingFrame` equal to 1 are ignored.

NOTE: This requirement means that a frame can be shown with a specified order hint without affecting the normal output of that frame.

NOTE: The requirement that `RefImplicitOutputFrame[ refIdx ]` has been written prevents the use of uninitialized frame buffers when the first keyframe is decoded. This may also be implemented by initializing the array `RefImplicitOutputFrame` to 0 before decoding starts. However, note that later key frames in a video may trigger the output of frames.

NOTE: Even if a frame is stored into multiple reference frame buffers, it is still only eligible to be output once.

The output of this process is the variable `madeOutput` indicating if a matching frame was output.

§ 7.21.5. Flush implicit output frames process

The input to this process is a variable `olkLimit` (that limits the range of flushed frames).

This process is invoked after all other OBUs have been decoded and outputs all remaining eligible frames.

An eligible frame is found as follows:

```
outputHint = -1
outIdx = -1
outputLayer = -1
outputOrder = -1
for (i = 0; i < NUM_REF_FRAMES; i++) {
    if ( is_frame_eligible_for_output( i ) &&
        ( outIdx == -1 || RefOutputOrder[i] <= outputOrder ) &&
        !( olkLimit && RefOrderHint[ i ] >= OlkTUOrderHint ) ) {
        outIdx = i
        outputHint = RefOrderHint[i]
        outputLayer = RefMLayerId[i]
        outputOrder = RefOutputOrder[i]
    }
}
```

If outIdx is equal to -1, this process immediately terminates.

The output process specified in [§7.21.1 Output process](#) is invoked with outIdx as input.

This entire process is then repeated until the termination condition is reached.

§ 7.21.6. Output frame buffers process

The input to this process is a variable refIdx. If refIdx is greater than or equal to 0, refIdx specifies which reference frame buffer to output. If refIdx is equal to -1, it indicates that the current frame is output.

First any eligible frames with lower order hints are output as follows:

```
while(1) {
    outputHint = output_ordering( refIdx )
    outIdx = refIdx
    for (i = 0; i < NumRefFrames; i++) {
        if ( is_frame_eligible_for_output(i) &&
            output_ordering(i) < outputHint ) {
            outIdx = i
            outputHint = output_ordering(i)
        }
    }
    if (outIdx == refIdx) {
        break
    } else {
        output_process(outIdx)
    }
}
```

where output_process(outIdx) denotes an invocation of the output process specified in [§7.21.1 Output process](#) with frameToShowMapIdx equal to outIdx.

The function output_ordering (which returns an order hint with additional bits specifying the embedded layer) is specified as:

```
output_ordering( i ) {
    if ( i < 0 ) {
        return OrderHint * (max_mlayer_id + 1) + obu_mlayer_id
    }
    return RefOrderHint[i] * (max_mlayer_id + 1) + RefMLayerId[i]
}
```

The output process specified in [§ 7.21.1 Output process](#) is invoked with `refIdx` as input.

The output successive frames process specified in [§ 7.21.3 Output successive frames process](#) is invoked with `outputHint` as input.

§ 7.21.7. Film grain synthesis process

§ 7.21.7.1. General

The inputs to this process are:

- variables `w` and `h` specifying the width and height of the frame,
- variables `subX` and `subY` specifying the subsampling parameters of the frame,
- a variable `bitDepth` specifying the number of bits per sample,
- a variable `numPlanes` specifying the number of planes in the frame.

The process modifies the arrays `OutY`, `OutU`, `OutV` to add film grain noise by the following ordered steps:

1. The variable `RandomRegister` (used for generating pseudo-random numbers) is set equal to `grain_seed`.
2. The variable `GrainMin` is set equal to $-(1 \ll (\text{bitDepth} - 1))$.
3. The variable `GrainMax` is set equal to $(1 \ll (\text{bitDepth} - 1)) - 1$.
4. The generate grain process specified in [§ 7.21.7.3 Generate grain process](#) is invoked with `subX`, `subY`, and `bitDepth` as input.
5. The scaling lookup initialization process specified in [§ 7.21.7.4 Scaling lookup initialization process](#) is invoked with `numPlanes` as input.
6. The add noise process specified in [§ 7.21.7.5 Add noise synthesis process](#) is invoked with `w`, `h`, `subX`, `subY`, `bitDepth`, and `numPlanes` as inputs.

§ 7.21.7.2. Random number process

The input to this process is a variable `bits` specifying the number of random bits to return.

The output of this process is a pseudo-random number based on the state in `RandomRegister`.

The process is specified as follows:

```
get_random_number( bits ) {  
    r = RandomRegister  
    bit = ((r >> 0) ^ (r >> 1) ^ (r >> 3) ^ (r >> 12)) & 1  
    r = (r >> 1) | (bit << 15)  
    result = (r >> (16 - bits)) & ((1 << bits) - 1)  
    RandomRegister = r  
    return result  
}
```

The output of this process is the variable `result`.

§ 7.21.7.3. Generate grain process

The inputs to this process are:

- variables subX and subY specifying the subsampling parameters of the frame,
- a variable bitDepth specifying the number of bits per sample.

This process generates noise via an auto-regressive filter.

First an array LumaGrain 82 samples wide and 73 samples high of white noise is generated for luma as follows:

```
shift = 12 - bitDepth + grain_scale_shift
for ( y = 0; y < 73; y++ ) {
  for ( x = 0; x < 82; x++ ) {
    if ( num_y_points > 0 ) {
      g = Gaussian_Sequence[ get_random_number( 11 ) ]
    } else {
      g = 0
    }
    LumaGrain[ y ][ x ] = Round2( g, shift )
  }
}
```

where the function call get_random_number invokes the random number process specified in [§7.21.7.2 Random number process](#).

Then an auto-regressive filter is applied to the white noise as follows:

```
shift = ar_coeff_shift_minus_6 + 6
for ( y = 3; y < 73; y++ ) {
  for ( x = 3; x < 82 - 3; x++ ) {
    s = 0
    pos = 0
    for ( deltaRow = -ar_coeff_lag; deltaRow <= 0; deltaRow++ ) {
      for ( deltaCol = -ar_coeff_lag; deltaCol <= ar_coeff_lag; deltaCol++ ) {
        if ( deltaRow == 0 && deltaCol == 0 )
          break
        c = ar_coeffs_y[ pos ]
        s += LumaGrain[ y + deltaRow ][ x + deltaCol ] * c
        pos++
      }
    }
    LumaGrain[ y ][ x ] = Clip3( GrainMin, GrainMax,
                               LumaGrain[ y ][ x ] + Round2( s, shift ) )
  }
}
```

The variable chromaW (representing the width of the chroma noise array) is set equal to (subX ? 44 : 82).

The variable chromaH (representing the height of the chroma noise array) is set equal to (subY ? 38 : 73).

White noise arrays CbGrain and CrGrain chromaW samples wide and chromaH samples high are generated as follows:

```
shift = 12 - bitDepth + grain_scale_shift
RandomRegister = grain_seed ^ 0xb524
for ( y = 0; y < chromaH; y++ ) {
  for ( x = 0; x < chromaW; x++ ) {
```

```

    if ( num_cb_points > 0 || chroma_scaling_from_luma ) {
        g = Gaussian_Sequence[ get_random_number( 11 ) ]
    } else {
        g = 0
    }
    CbGrain[ y ][ x ] = Round2( g, shift )
}
}
RandomRegister = grain_seed ^ 0x49d8
for ( y = 0; y < chromaH; y++ ) {
    for ( x = 0; x < chromaW; x++ ) {
        if ( num_cr_points > 0 || chroma_scaling_from_luma ) {
            g = Gaussian_Sequence[ get_random_number( 11 ) ]
        } else {
            g = 0
        }
        CrGrain[ y ][ x ] = Round2( g, shift )
    }
}
}

```

Then the auto-regressive filter is applied as follows:

```

shift = ar_coeff_shift_minus_6 + 6
for ( y = 3; y < chromaH; y++ ) {
    for ( x = 3; x < chromaW - 3; x++ ) {
        s0 = 0
        s1 = 0
        pos = 0
        for ( deltaRow = -ar_coeff_lag; deltaRow <= 0; deltaRow++ ) {
            for ( deltaCol = -ar_coeff_lag; deltaCol <= ar_coeff_lag; deltaCol++ ) {
                c0 = ar_coeffs_cb[ pos ]
                c1 = ar_coeffs_cr[ pos ]
                if ( deltaRow == 0 && deltaCol == 0 ) {
                    if ( num_y_points > 0 ) {
                        luma = 0
                        lumaX = ( (x - 3) << subX ) + 3
                        lumaY = ( (y - 3) << subY ) + 3
                        for ( i = 0; i <= subY; i++ )
                            for ( j = 0; j <= subX; j++ )
                                luma += LumaGrain[ lumaY + i ][ lumaX + j ]
                        luma = Round2( luma, subX + subY )
                        s0 += luma * c0
                        s1 += luma * c1
                    }
                    break
                }
                s0 += CbGrain[ y + deltaRow ][ x + deltaCol ] * c0
                s1 += CrGrain[ y + deltaRow ][ x + deltaCol ] * c1
                pos++
            }
        }
        CbGrain[ y ][ x ] = Clip3( GrainMin, GrainMax,
                                CbGrain[ y ][ x ] + Round2( s0, shift ) )
        CrGrain[ y ][ x ] = Clip3( GrainMin, GrainMax,
                                CrGrain[ y ][ x ] + Round2( s1, shift ) )
    }
}
}

```

NOTE: When num_y_points is equal to 0, this process may use uninitialized values within ar_coeffs_y to compute LumaGrain. However, LumaGrain will never be read in this case so it does not matter what values are constructed. Similarly, when num_cr_points/num_cb_points are equal to 0 and chroma_scaling_from_luma is equal to 0, the CbGrain/CrGrain arrays will never be read.

§ 7.21.7.4. Scaling lookup initialization process

The input to this process is a variable numPlanes specifying the number of planes in the frame.

This process computes 3 lookup tables for the different color components.

Each lookup table ScalingLut[plane] contains 256 entries constructed by a piecewise linear interpolation of the given points as follows:

```
for ( plane = 0; plane < numPlanes; plane++ ) {
    if ( plane == 0 || chroma_scaling_from_luma )
        numPoints = num_y_points
    else if ( plane == 1 )
        numPoints = num_cb_points
    else
        numPoints = num_cr_points
    if ( numPoints == 0 ) {
        for ( x = 0; x < 256; x++ ) {
            ScalingLut[ plane ][ x ] = 0
        }
    } else {
        for ( x = 0; x < get_x( plane, 0 ); x++ ) {
            ScalingLut[ plane ][ x ] = get_y( plane, 0 )
        }
        for ( i = 0; i < numPoints - 1; i++ ) {
            deltaY = get_y( plane, i + 1 ) - get_y( plane, i )
            deltaX = get_x( plane, i + 1 ) - get_x( plane, i )
            delta = deltaY * ( ( 65536 + (deltaX >> 1) ) / deltaX )
            for ( x = 0; x < deltaX; x++ ) {
                v = get_y( plane, i ) + ( ( x * delta + 32768 ) >> 16 )
                ScalingLut[ plane ][ get_x( plane, i ) + x ] = v
            }
        }
        for ( x = get_x( plane, numPoints - 1 ); x < 256; x++ ) {
            ScalingLut[ plane ][ x ] = get_y( plane, numPoints - 1 )
        }
    }
}
```

where the functions get_x and get_y return the coordinates for a specific point and are specified as:

```
get_x( plane, i ) {
    if ( plane == 0 || chroma_scaling_from_luma )
        return point_y_value[ i ]
    else if ( plane == 1 )
        return point_cb_value[ i ]
    else
        return point_cr_value[ i ]
}

get_y( plane, i ) {
    if ( plane == 0 || chroma_scaling_from_luma )
        return point_y_scaling[ i ]
    else if ( plane == 1 )
        return point_cb_scaling[ i ]
    else
        return point_cr_scaling[ i ]
}
```


§ 7.21.7.5. Add noise synthesis process

The inputs to this process are:

- variables w and h specifying the width and height of the frame,
- variables subX and subY specifying the subsampling parameters of the frame,
- a variable bitDepth specifying the number of bits per sample,
- a variable numPlanes specifying the number of planes in the frame.

This process combines the film grain with the image data.

First an array of noise data noiseStripe is generated for each 32 luma sample high stripe of the image.

noiseStripe[lumaNum][0] is 34 samples high and w samples wide (a few additional samples across are actually written to the array, but these are never read) and contains noise for the luma [component](#).

noiseStripe[lumaNum][1] and noiseStripe[lumaNum][2] are (34 >> subY) samples high and Round2(w, subX) samples wide and contain noise for the chroma components.

noiseStripe represents the result of constructing square grain blocks and blending horizontally adjacent blocks together (although blending is only applied if overlap_flag is equal to 1) and is constructed as follows:

```
lumaSize = film_grain_block_size ? 32 : 16
lumaNum = 0
for ( y = 0; y < (h + 1)/2 ; y += (lumaSize >> 1) ) {
    RandomRegister = grain_seed
    lumaRand = y >> 3
    RandomRegister ^= ((lumaRand * 37 + 178) & 255) << 8
    RandomRegister ^= ((lumaRand * 173 + 105) & 255)
    for ( x = 0; x < (w + 1)/2 ; x += (lumaSize >> 1) ) {
        offsetY = get_random_number( 9 ) * (3 - film_grain_block_size) >> 6
        get_random_number( 1 )
        get_random_number( 1 )
        get_random_number( 1 )
        offsetX = get_random_number( 9 ) * (3 - film_grain_block_size) >> 6
        get_random_number( 1 )
        get_random_number( 1 )
        get_random_number( 1 )
        for ( plane = 0 ; plane < numPlanes; plane++ ) {
            planeSubX = ( plane > 0 ) ? subX : 0
            planeSubY = ( plane > 0 ) ? subY : 0
            planeOffsetX = planeSubX ? 6 + offsetX : 9 + offsetX * 2
            planeOffsetY = planeSubY ? 6 + offsetY : 9 + offsetY * 2
            for ( i = 0; i < (lumaSize + 2) >> planeSubY ; i++ ) {
                for ( j = 0; j < (lumaSize + 2) >> planeSubX ; j++ ) {
                    if ( plane == 0 )
                        g = LumaGrain[ planeOffsetY + i ][ planeOffsetX + j ]
                    else if ( plane == 1 )
                        g = CbGrain[ planeOffsetY + i ][ planeOffsetX + j ]
                    else
                        g = CrGrain[ planeOffsetY + i ][ planeOffsetX + j ]
                    if ( planeSubX == 0 ) {
                        if ( j < 2 && overlap_flag && x > 0 ) {
                            old = noiseStripe[ lumaNum ][ plane ][ i ][ x * 2 + j ]
                            if ( j == 0 ) {
                                g = old * 27 + g * 17
                            } else {
                                g = old * 17 + g * 27
                            }
                        }
                    }
                    g = Clip3( GrainMin, GrainMax, Round2(g, 5) )
                }
            }
        }
    }
}
```

```

    }
    noiseStripe[ lumaNum ][ plane ][ i ][ x * 2 + j ] = g
  } else {
    if ( j == 0 && overlap_flag && x > 0 ) {
      old = noiseStripe[ lumaNum ][ plane ][ i ][ x + j ]
      g = old * 23 + g * 22
      g = Clip3( GrainMin, GrainMax, Round2(g, 5) )
    }
    noiseStripe[ lumaNum ][ plane ][ i ][ x + j ] = g
  }
}
}
}
}
lumaNum++
}
}

```

Then the noise stripes are blended together to form a noise image noiseImage as follows:

```

for ( plane = 0; plane < numPlanes; plane++ ) {
  planeSubX = ( plane > 0 ) ? subX : 0
  planeSubY = ( plane > 0 ) ? subY : 0
  for ( y = 0; y < ( (h + planeSubY) >> planeSubY ) ; y++ ) {
    lumaNum = y >> ( 4 + film_grain_block_size - planeSubY )
    i = y - (lumaNum << ( 4 + film_grain_block_size - planeSubY ) )
    for ( x = 0; x < ( (w + planeSubX) >> planeSubX ) ; x++ ) {
      g = noiseStripe[ lumaNum ][ plane ][ i ][ x ]
      if ( planeSubY == 0 ) {
        if ( i < 2 && lumaNum > 0 && overlap_flag ) {
          old = noiseStripe[ lumaNum - 1 ][ plane ][ i + lumaSize ][ x ]
          if ( i == 0 ) {
            g = old * 27 + g * 17
          } else {
            g = old * 17 + g * 27
          }
        }
        g = Clip3( GrainMin, GrainMax, Round2(g, 5) )
      } else {
        if ( i < 1 && lumaNum > 0 && overlap_flag ) {
          old = noiseStripe[ lumaNum - 1 ][ plane ][ i + (lumaSize >> 1) ][ x ]
          g = old * 23 + g * 22
          g = Clip3( GrainMin, GrainMax, Round2(g, 5) )
        }
      }
      noiseImage[ plane ][ y ][ x ] = g
    }
  }
}
}

```

NOTE: Although this process is specified in terms of full size noiseStripe and noiseImage arrays, the reference code shows how it is possible to implement the grain synthesis with just 2 line buffers for luma, and 1 line buffer for each chroma [component](#).

Finally, the noise is blended with the original image data as follows:

```

if ( clip_to_restricted_range ) {
  minValue = 16 << (bitDepth - 8)
  maxLuma = 235 << (bitDepth - 8)
  if ( fg_mc_identity )
    maxChroma = maxLuma
  else
    maxChroma = 240 << (bitDepth - 8)
} else {
  minValue = 0

```

```

maxLuma = (256 << (bitDepth - 8)) - 1
maxChroma = maxLuma
}
ScalingShift = grain_scaling_minus_8 + 8
for ( y = 0; y < ( (h + subY) >> subY) ; y++ ) {
  for ( x = 0; x < ( (w + subX) >> subX) ; x++ ) {
    lumaX = x << subX
    lumaY = y << subY
    lumaNextX = Min( lumaX + 1, w - 1 )
    if ( subX )
      averageLuma =
        Round2( OutY[ lumaY ][ lumaX ] + OutY[ lumaY ][ lumaNextX ], 1 )
    else
      averageLuma = OutY[ lumaY ][ lumaX ]
    if ( num_cb_points > 0 || chroma_scaling_from_luma ) {
      orig = OutU[ y ][ x ]
      if ( chroma_scaling_from_luma ) {
        merged = averageLuma
      } else {
        combined = averageLuma * ( cb_luma_mult - 128 ) +
          orig * ( cb_mult - 128 )
        merged = Clip3( 0, (1 << bitDepth) - 1,
          ( combined >> 6 ) +
            ( (cb_offset - 256) << (bitDepth - 8) ) )
      }
      noise = noiseImage[ 1 ][ y ][ x ]
      noise = Round2( scale_lut( 1, merged, bitDepth ) * noise, ScalingShift )
      OutU[ y ][ x ] = Clip3( minValue, maxChroma, orig + noise )
    }

    if ( num_cr_points > 0 || chroma_scaling_from_luma ) {
      orig = OutV[ y ][ x ]
      if ( chroma_scaling_from_luma ) {
        merged = averageLuma
      } else {
        combined = averageLuma * ( cr_luma_mult - 128 ) +
          orig * ( cr_mult - 128 )
        merged = Clip3( 0, (1 << bitDepth) - 1, ( combined >> 6 ) +
          ( (cr_offset - 256) << (bitDepth - 8) ) )
      }
      noise = noiseImage[ 2 ][ y ][ x ]
      noise = Round2( scale_lut( 2, merged, bitDepth ) * noise, ScalingShift )
      OutV[ y ][ x ] = Clip3( minValue, maxChroma, orig + noise )
    }
  }
}
for ( y = 0; y < h ; y++ ) {
  for ( x = 0; x < w ; x++ ) {
    orig = OutY[ y ][ x ]
    noise = noiseImage[ 0 ][ y ][ x ]
    noise = Round2( scale_lut( 0, orig, bitDepth ) * noise, ScalingShift )
    if ( num_y_points > 0 ) {
      OutY[ y ][ x ] = Clip3( minValue, maxLuma, orig + noise )
    }
  }
}

```

where `scale_lut` is a function that performs a piecewise linear interpolation into the appropriate scaling table. The `scale_lut` function is specified as follows:

```

scale_lut( plane, index, bitDepth ) {
  shift = bitDepth - 8
  x = index >> shift
  rem = index - ( x << shift )
  if ( x == 255 ) {
    return ScalingLut[ plane ][ x ]
  } else {
    start = ScalingLut[ plane ][ x ]

```

```

    end = ScalingLut[ plane ][ x + 1 ]
    return start + Round2( (end - start) * rem, shift )
}
}

```

§ 7.22. Motion field motion vector storage process

The inputs to this process are:

- variables *r* and *c* specifying the location of the block in units of 4x4 blocks in the luma plane,
- a variable *bSize* specifying the size of the block,
- a variable *mvMethod* that affects how the motion vector to be stored is computed.

This process applies some filtering and reordering to the motion vectors to prepare them for storage as part of the reference frame update process.

If *enable_ref_frame_mvs* is equal to 0, this process immediately terminates.

The variables *bw4*, *bh4* (describing the size of the block in units of 4x4 blocks in the luma plane), and *n* (specifying the size of the optical flow blocks within the block) are computed as follows:

```

bw4 = Num_4x4_Blocks_Wide[ bSize ]
bh4 = Num_4x4_Blocks_High[ bSize ]
n = (bw4 <= 2 && bh4 <= 2 && TipFrameMode != TIP_FRAME_AS_OUTPUT) ? 4 : 8
bw4 = Min(MiCols - c, bw4)
bh4 = Min(MiRows - r, bh4)

```

The variables *isWedge* (specifying if the block uses a wedge compound mode of two inter frames), *refIdx0*, *refIdx1*, and *tipPred* are computed as follows:

```

refIdx0 = RefFrames[ r ][ c ][ 0 ]
refIdx1 = RefFrames[ r ][ c ][ 1 ]
isWedge = is_inter_ref_frame(refIdx0) && is_inter_ref_frame(refIdx1) &&
    refIdx0 != TIP_FRAME && compound_type == COMPOUND_WEDGE
tipPred = refIdx0 == TIP_FRAME
if (tipPred) {
    refIdx0 = ClosestPast
    refIdx1 = ClosestFuture
}
if ( (tipPred || TipFrameMode == TIP_FRAME_AS_OUTPUT) &&
    Tip_Weighting_Factor[ tip_global_wtd_index ] == 16 ) {
    refIdx1 = NONE
}

```

The following applies for *i8* = 0..*Round2*(*bh4*,1)-1, for *j8* = 0..*Round2*(*bw4*,1)-1:

```

allowList[ 0 ] = 1
allowList[ 1 ] = 1
if (isWedge) {
    count0 = 0
    count1 = 0
    for ( i = 0; i < 8; i++ ) {
        for( j = 0; j < 8; j++ ) {
            m = Mask[ i8 * 8 + i ][ j8 * 8 + j ]
            if ( m > 60 )
                count0++
            if ( m < 4 )
                count1++
        }
    }
}

```

```

    }
}
if (count0 >= 60) {
    allowList[ 1 ] = 0
} else if (count1 >= 60) {
    allowList[ 0 ] = 0
}
}

x8 = (c >> 1) + j8
y8 = (r >> 1) + i8
row = r + (i8 << 1)
col = c + (j8 << 1)
for( list = 0; list < 2; list++ ) {
    refs[ list ] = NONE
    for( comp = 0; comp < 2; comp++ ) {
        mfmvs[ list ][ comp ] = 0
    }
}
for ( list = 0; list < 2; list++ ) {
    refIdx = list == 0 ? refIdx0 : refIdx1
    if ( is_inter_ref_frame(refIdx) ) {
        if ( mvMethod > 0 ) {
            mvs = ( use_refinemv || tipPred ) ?
                RefineMvs[ i8 << 1 ][ j8 << 1 ] : Mvs[ row ][ col ]
            mvRow = mvs[list][0]
            mvCol = mvs[list][1]
            if ( mvMethod==1 ) {
                if ( n==4 && !tipPred ) {
                    totalRow = 0
                    totalCol = 0
                    for(a=0;a<2;a++) {
                        for(b=0;b<2;b++) {
                            totalRow += MvDeltas[ a ][ b ][ list ][ 0 ]
                            totalCol += MvDeltas[ a ][ b ][ list ][ 1 ]
                        }
                    }
                    mvRow += Round2Signed(totalRow, 1 + 2)
                    mvCol += Round2Signed(totalCol, 1 + 2)
                } else {
                    mvRow += Round2Signed(
                        MvDeltas[ i8 << 1 ][ j8 << 1 ][ list ][ 0 ], 1)
                    mvCol += Round2Signed(
                        MvDeltas[ i8 << 1 ][ j8 << 1 ][ list ][ 1 ], 1)
                }
            }
        } else {
            if ( tipPred ) {
                candMvs = get_tip_cand( row, col )
                mv = candMvs[ list ]
            } else if ( motion_mode >= LOCALWARP && !force_integer_mv ) {
                mv = get_sub_block_warp_mv( LocalWarpParams[ list ], 0,
                    col * MI_SIZE, row * MI_SIZE,
                    8, 8, 1 )
            } else if ( is_global_mv_cand( YMode, bSize, refIdx ) &&
                !force_integer_mv ) {
                mv = get_sub_block_warp_mv( gm_params[ refIdx ], 0,
                    col * MI_SIZE, row * MI_SIZE,
                    8, 8, 1 )
            } else {
                mv = Mvs[ row ][ col ][ list ]
            }
            mvRow = mv[ 0 ]
            mvCol = mv[ 1 ]
        }
    }

    if ( Abs( mvRow ) <= REFMVS_LIMIT && Abs( mvCol ) <= REFMVS_LIMIT ) {
        if ( allowList[list] ) {
            mfmvs[ list ][ 0 ] = mvRow
            mfmvs[ list ][ 1 ] = mvCol
            refs[ list ] = refIdx
        }
    }
}

```

```

    }
}
}
ref0 = refs[ 0 ]
mvRow0 = mfmvs[ 0 ][ 0 ]
mvCol0 = mfmvs[ 0 ][ 1 ]
ref1 = refs[ 1 ]
mvRow1 = mfmvs[ 1 ][ 0 ]
mvCol1 = mfmvs[ 1 ][ 1 ]
if ( ref0 != NONE && ref1 == NONE ) {
    refs[ 1 ] = ref0
    mfmvs[ 1 ][ 0 ] = mvRow0
    mfmvs[ 1 ][ 1 ] = mvCol0
} else if ( ref1 != NONE && ref0 == NONE ) {
    refs[ 0 ] = ref1
    mfmvs[ 0 ][ 0 ] = mvRow1
    mfmvs[ 0 ][ 1 ] = mvCol1
} else if ( ref0 != NONE && refs[ 1 ] != NONE ) {
    refOrder0 = OrderHints[ref0]
    refOrder1 = OrderHints[ref1]
    if ( get_relative_dist( refOrder0, OrderHint ) < 0 &&
        get_relative_dist( refOrder1, OrderHint ) < 0 ) {
        toSwitch = get_relative_dist( refOrder0, refOrder1 ) < 0
    } else if ( get_relative_dist( refOrder0, OrderHint ) > 0 &&
        get_relative_dist( refOrder1, OrderHint ) > 0 ) {
        toSwitch = get_relative_dist( refOrder0, refOrder1 ) < 0
    } else {
        toSwitch = get_relative_dist( refOrder0, OrderHint ) > 0 &&
            get_relative_dist( refOrder1, OrderHint ) < 0
    }
    if (toSwitch) {
        refs[ 0 ] = ref1
        mfmvs[ 0 ][ 0 ] = mvRow1
        mfmvs[ 0 ][ 1 ] = mvCol1
        refs[ 1 ] = ref0
        mfmvs[ 1 ][ 0 ] = mvRow0
        mfmvs[ 1 ][ 1 ] = mvCol0
    }
}

for ( list = 0; list < 2; list++ ) {
    MfRefFrames[ y8 ][ x8 ][ list ] = refs[ list ]
    for ( comp = 0; comp < 2; comp++ ) {
        MfMvs[ y8 ][ x8 ][ list ][ comp ] = compression_mv( mfmvs[list][comp] )
    }
}

```

The functions `get_tip_cand`, `get_tip_offsets`, `to_fullmv`, `get_sub_block_warp_mv` are defined as:

```

to_fullmv(mv) {
    return (mv + 3 + ((mv >= 0) ? 1 : 0) ) >> 3
}

get_tip_cand(candRow, candCol) {
    baseRow = MiRowBase[ 0 ][ candRow ][ candCol ]
    baseCol = MiColBase[ 0 ][ candRow ][ candCol ]
    shift = 1 + TipSizes16x16[ candRow ][ candCol ]
    candRow = baseRow + (((candRow - baseRow) >> shift) << shift)
    candCol = baseCol + (((candCol - baseCol) >> shift) << shift)
    x8 = candCol >> 1
    y8 = candRow >> 1
    candMvs[ 0 ][ 0 ] = 0
    candMvs[ 0 ][ 1 ] = 0
    candMvs[ 1 ][ 0 ] = 0
    candMvs[ 1 ][ 1 ] = 0
    refX8 = Clip3( 0, (MiCols >> 1) - 1, x8 )
    refY8 = Clip3( 0, (MiRows >> 1) - 1, y8 )
    if ( MotionFieldValid[ refY8 ][ refX8 ] ) {

```

```

    (refOffset, pastOffset, futureOffset) = get_tip_offsets()
    candMvs[ 0 ] = get_mv_projection( MotionFieldMvs[ refY8 ][ refX8 ],
                                     pastOffset, refOffset )
    candMvs[ 1 ] = get_mv_projection( MotionFieldMvs[ refY8 ][ refX8 ],
                                     futureOffset, refOffset )
}
for( list = 0; list < 2; list++ ) {
    for(comp=0;comp<2;comp++) {
        candMvs[ list ][ comp ] += Mvs[ candRow ][ candCol ][ 0 ][ comp ]
        candMvs[ list ][ comp ] =
            Clip3(MV_LOW + 1, MV_UPP - 1, candMvs[ list ][ comp ] )
    }
}
return candMvs
}

get_tip_offsets() {
    if ( NumFutureRefs > 0 && NumPastRefs > 0 ) {
        refOffset = get_relative_dist( OrderHints[ClosestFuture],
                                       OrderHints[ClosestPast])
    } else {
        refOffset = get_relative_dist( OrderHints[ClosestPast],
                                       OrderHints[ClosestFuture])
    }
    pastOffset = get_relative_dist( OrderHint,
                                    OrderHints[ClosestPast])
    futureOffset = get_relative_dist( OrderHint,
                                    OrderHints[ClosestFuture])
    refOffset = Min( refOffset, MAX_FRAME_DISTANCE )
    return (refOffset, pastOffset, futureOffset)
}

get_sub_block_warp_mv( warpParams, plane, x, y, w, h, rnd ) {
    if ( plane == 0 ) {
        subX = 0
        subY = 0
    } else {
        subX = SubsamplingX
        subY = SubsamplingY
    }
    srcX = (x + (w >> 1) ) << subX
    srcY = (y + (h >> 1) ) << subY
    dstX = warpParams[ 2 ] * srcX + warpParams[ 3 ] * srcY + warpParams[ 0 ]
    dstY = warpParams[ 4 ] * srcX + warpParams[ 5 ] * srcY + warpParams[ 1 ]
    if (rnd) {
        mv[ 0 ] = Round2Signed( dstY - (srcY << WARPEDMODEL_PREC_BITS),
                               WARPEDMODEL_PREC_BITS - 3)
        mv[ 1 ] = Round2Signed( dstX - (srcX << WARPEDMODEL_PREC_BITS),
                               WARPEDMODEL_PREC_BITS - 3)
    } else {
        mv[ 0 ] = (dstY - (srcY << WARPEDMODEL_PREC_BITS)) >>
            (WARPEDMODEL_PREC_BITS - 3)
        mv[ 1 ] = (dstX - (srcX << WARPEDMODEL_PREC_BITS)) >>
            (WARPEDMODEL_PREC_BITS - 3)
    }
    mv[ 0 ] = Clip3(MV_LOW + 1, MV_UPP - 1, mv[ 0 ])
    mv[ 1 ] = Clip3(MV_LOW + 1, MV_UPP - 1, mv[ 1 ])
    return mv
}

```

The function `compression_mv` (which compresses a motion vector component into fewer bits to reduce memory bandwidth) is specified as:

```

compression_mv( v ) {
    a = Abs( v )
    stepLog2 = Max( 0, GetMsb( a ) - 4 )
}

```

```

    c = ( a >> stepLog2 ) + ( stepLog2 << 4 )
    return v < 0 ? -c : c
}

```

The function `uncompression_mv` (which decompresses a motion vector component) is specified as:

```

uncompression_mv( v ) {
    c = Abs( v )
    stepLog2 = Max( 0, (c >> 4) - 1 )
    a = ( c - (stepLog2 << 4) ) << stepLog2
    return v < 0 ? -a : a
}

```

§ 7.23. Reference frame update process

This process is invoked as the final step in decoding a frame.

The inputs to this process are the decoded samples for the current frame `LrFrame[ plane ][ x ][ y ]`.

The output from this process is an updated set of reference frames and previous motion vectors.

If this is the first time this process is invoked, the variable `FrameCounter` (used to identify when a frame is stored in multiple reference frames) is set equal to 0. Otherwise, the variable `FrameCounter` is incremented by 1.

The variable `first` (indicating which is the first reference frame to be updated) is set equal to 1.

For each value of `i` from 0 to `NUM_REF_FRAMES - 1`, the following applies if bit `i` of `refresh_frame_flags` is equal to 1 (i.e., if `(refresh_frame_flags >> i) & 1` is equal to 1):

- If `is_frame_eligible_for_output(i)` is equal to 1, the output frame buffers process specified in [§7.21.6 Output frame buffers process](#) is invoked with `i` as input.
- `RefValid[ i ]` is set equal to `(FrameType == KEY_FRAME || FrameType == SWITCH_FRAME) ? first : 1`.
- `first` is set equal to 0.
- `RefFrameWidth[ i ]` is set equal to `FrameWidth`.
- `RefFrameHeight[ i ]` is set equal to `FrameHeight`.
- `RefCropWidth[ i ]` is set equal to `CropWidth`.
- `RefCropHeight[ i ]` is set equal to `CropHeight`.
- `RefCropLeft[ i ]` is set equal to `CropLeft`.
- `RefCropTop[ i ]` is set equal to `CropTop`.
- `RefMiCols[ i ]` is set equal to `MiCols`.
- `RefMiRows[ i ]` is set equal to `MiRows`.
- `RefFrameType[ i ]` is set equal to `FrameType`.
- `RefSubsamplingX[ i ]` is set equal to `SubsamplingX`.
- `RefSubsamplingY[ i ]` is set equal to `SubsamplingY`.
- `RefLongTermId[ i ]` is set equal to `LongTermId`.
- `RefOutputOrder[ i ]` is set equal to `output_ordering( -1 )`.

- RefBitDepth[i] is set equal to BitDepth.
- RefNumPlanes[i] is set equal to NumPlanes.
- RefFilmGrainPresent[i] is set equal to film_grain_params_present.
- RefImplicitOutputFrame[i] is set equal to implicit_output_frame.
- RefImmediateOutputFrame[i] is set equal to immediate_output_frame.
- RefOrderHint[i] is set equal to OrderHint.
- RefOrderHintLsbs[i] is set equal to OrderHintLsbs.
- RefBaseQIdx[i] is set equal to base_q_idx.
- RefDeltaQUAc[i] is set equal to DeltaQUAc.
- RefDeltaQVAc[i] is set equal to DeltaQVAc.
- RefFrameFiltersOn[i] is set equal to a copy of frame_filters_on.
- RefFrameLrWienerNs[i] is set equal to a copy of FrameLrWienerNs.
- RefNumFilterClasses[i] is set equal to NumFilterClasses.
- RefCounter[i] is set equal to FrameCounter.
- RefNumTotalRefs[i] is set equal to NumTotalRefs.
- RefTLayerId[i] is set equal to obu_tlayer_id.
- RefMLayerId[i] is set equal to obu_mlayer_id.
- SavedOrderHints[i][j] is set equal to OrderHints[j] for $j = 0..REFS_PER_FRAME-1$.
- FrameStore[i][0][y][x] is set equal to LrFrame[0][y][x] for $x = 0..(MiCols * MI_SIZE-1)$, for $y = 0..(MiRows * MI_SIZE-1)$.
- FrameStore[i][plane][y][x] is set equal to LrFrame[plane][y][x] for $plane = 1..2$, for $x = 0..(MiCols * MI_SIZE >> SubsamplingX) - 1$, for $y = 0..(MiRows * MI_SIZE >> SubsamplingY) - 1$.
- SavedRefFrames[i][y8][x8][list] is set equal to MfRefFrames[y8][x8][list] for $y8 = 0..(MiRows>>1)-1$, for $x8 = 0..(MiCols>>1)-1$, for $list = 0..1$.
- SavedMvs[i][y8][x8][list][comp] is set equal to MfMvs[y8][x8][list][comp] for $comp = 0..1$, for $y8 = 0..(MiRows>>1)-1$, for $x8 = 0..(MiCols>>1)-1$, for $list = 0..1$.
- SavedGmParams[i][ref][j] is set equal to gm_params[ref][j] for $ref = 0..REFS_PER_FRAME-1$, for $j = 0..5$.
- SavedSegmentIds[i][row][col] is set equal to SegmentIds[row][col] for $row = 0..MiRows-1$, for $col = 0..MiCols-1$.
- The function save_cdfs(i) is invoked (see below).
- If film_grain_params_present is equal to 1, the following ordered steps apply:
 1. The function load_grain_params is invoked with NUM_REF_FRAMES as input (see [§7.21.2 Intermediate output preparation process](#)).
 2. The function save_grain_params(i) is invoked (see below).
- The function save_ccso_params(i, plane) is invoked (see below) for $plane = 0..2$.

`save_cdfs( ctx )` is a function call that indicates that all the CDF arrays are saved into [frame context](#) number `ctx` in the range 0 to `(NUM_REF_FRAMES - 1)`. When this function is invoked the following takes place:

- A copy of each CDF array mentioned in the semantics for `init_coeff_cdfs` and `init_non_coeff_cdfs` is saved in an area of memory indexed by `ctx`.

`save_grain_params( i )` is a function call that indicates that all the syntax elements that can be read in both `film_grain_model` and `film_grain_config` should be saved into an area of memory indexed by `i`.

`save_ccso_params( i, plane )` is a function call that indicates that certain variables and arrays are saved into an area of memory indexed by `i` and `plane`:

- `CcsoLumaSizeLog2`
- `ccso_planes[plane]`
- `ccso_scale_idx[plane]`
- `ccso_bo_only[plane]`
- `ccso_quant_idx[plane]`
- `ccso_ext_filter[plane]`
- `ccso_max_band_log2[plane]`
- `ccso_edge_clf[plane]`
- `CcsoFilterOffset[plane]`
- `CcsoBlks[plane]`

`is_frame_eligible_for_output` is a function call that is specified in [§ 7.21.4 Output implicit output frame process](#).

The function `load_ccso_params` is used in other parts of the specification to reload the specified values.

`load_ccso_params( i, plane )` is a function call that indicates that the variables and arrays saved in `save_ccso_params` are to be reloaded from an area of memory indexed by `i` and `plane`.

[↑ Back to Table of Contents](#)

§ 8. Parsing process

§ 8.1. Parsing process for $f(n)$

This process is invoked when the descriptor of a syntax element in the syntax tables is equal to $f(n)$.

The next n bits are read from the bitstream.

This process is specified as follows:

```
x = 0
for ( i = 0; i < n; i++ ) {
    x = 2 * x + read_bit( )
}
```

`read_bit( )` reads the next bit from the bitstream and advances the bitstream position indicator by 1. If the bitstream is provided as a series of bytes, then the first bit is given by the most significant bit of the first byte.

The value for the syntax element is given by x .

§ 8.2. Parsing process for symbol decoder

§ 8.2.1. General

The entropy decoder is referred to as the "Symbol decoder" and the functions `init_symbol( sz )`, `exit_symbol( )`, `read_symbol( cdf )`, and `read_bool( )` are used in this specification to indicate the entropy decoding operation.

§ 8.2.2. Initialization process for symbol decoder

The input to this process is a variable sz specifying the number of bytes to be read by the Symbol decoder.

This process is invoked when the function `init_symbol( sz )` is called from the syntax structure.

NOTE: The bit position will always be byte aligned when `init_symbol` is invoked because the [frame header info](#) and the data partitions are always a whole number of bytes long.

The variable `numBits` is set equal to $\text{Min}(sz * 8, 15)$.

The variable `buf` is read using the $f(\text{numBits})$ parsing process.

The variable `paddedBuf` is set equal to $(\text{buf} \ll (15 - \text{numBits}))$.

The variable `SymbolValue` is set to $((1 \ll 15) - 1) \wedge \text{paddedBuf}$.

The variable `SymbolRange` is set to $1 \ll 15$.

The variable `SymbolMaxBits` is set to $8 * sz - 15$.

SymbolMaxBits (when non-negative) represents the number of bits still available to be read. It is allowed for this number to go negative (either here or during read_symbol or during read_bool). SymbolMaxBits (when negative) signifies that all available bits have been read, and that -SymbolMaxBits of padding zero bits have been used in the symbol decoding process. These padding zero bits are not present in the bitstream.

A copy is made of each of the CDF arrays mentioned in the semantics for init_coeff_cdfs and init_non_coeff_cdfs. The name of the destination for the copy is the name of the CDF array prefixed with "Tile". The name of the source for the copy is the name of the CDF array with no prefix. This copying produces the following arrays:

- TileWarpMvCdf
- TileTipPredModeCdf
- TileWarpIdxCdf
- TileWarpWithMvdCdf
- TileIsWarpCdf
- TileUseGdfCdf
- TileBruModeCdf
- TileCdefIndex0Cdf
- TileCdefIndexMinus1With3Cdf
- TileCdefIndexMinus1With4Cdf
- TileCdefIndexMinus1With5Cdf
- TileCdefIndexMinus1With6Cdf
- TileCdefIndexMinus1With7Cdf
- TileCdefIndexMinus1With8Cdf
- TileUseDeltaWarpCdf
- TileWarpDeltaPrecisionCdf
- TileWarpDeltaParamLowCdf
- TileWarpDeltaParamHighCdf
- TileWarpDeltaParamSignCdf
- TileYModeSetCdf
- TileYModeIndexCdf
- TileYModeOffsetCdf
- TileCwpIdxCdf
- TileFscModeCdf
- TileMrlIndexCdf
- TileMrlSecIndexCdf
- TileUseDpcmYCdf
- TileDpcmModeYCdf
- TileUseDpcmUvCdf

- TileDpcmModeUvCdf
- TileUVModeCflNotAllowedCdf
- TileIsCflCdf
- TileIntrabcCdf
- TileIntrabcPrecisionCdf
- TileIntrabcModeCdf
- TileMorphPredCdf
- TileRegionTypeCdf
- TileDoSquareSplitCdf
- TileDoSplitCdf
- TileRectTypeCdf
- TileDoExtPartitionCdf
- TileDoUneven4wayPartitionCdf
- TileSegIdExtFlagCdf
- TileSegmentIdCdf
- TileSegmentIdExtCdf
- TileSegmentIdPredictedCdf
- TileTxPartitionTypeCdf
- TileTx2or3PartitionTypeCdf
- TileTxDoPartitionCdf
- TileLosslessTxSizeCdf
- TileLosslessInterTxTypeCdf
- TileSecTxTypeCdf
- TileMostProbableStxSetCdf
- TileMostProbableStxSetAdstCdf
- TileInterpFilterCdf
- TileUseLocalWarpCdf
- TileUseExtendWarpCdf
- TileSingleModeCdf
- TileUseBawpCdf
- TileUseBawpChromaCdf
- TileExplicitBawpCdf
- TileExplicitBawpScaleCdf
- TileIsJointCdf
- TileCompoundModeNonJointCdf
- TileCompoundModeSameRefsCdf

- TileUseOptflowCdf
- TileTipModeCdf
- TileUseRefinemvCdf
- TileDrlModeCdf
- TileSkipDrlModeCdf
- TileTipDrlModeCdf
- TileIsInterCdf
- TileCompModeCdf
- TileSkipModeCdf
- TileSkipCdf
- TileCompRef0Cdf
- TileCompRef1Cdf
- TileSingleRefCdf
- TileUseMostProbablePrecisionCdf
- TilePbMvPrecisionCdf
- TileMvJointAdaptiveCdf
- TileAmvdIndicesCdf
- TileJointShellSetCdf
- TileJointShell0Class0Cdf
- TileJointShell1Class0Cdf
- TileJointShell3Class0Cdf
- TileJointShell4Class0Cdf
- TileJointShell5Class0Cdf
- TileJointShell6Class0Cdf
- TileJointShell0Class1Cdf
- TileJointShell1Class1Cdf
- TileJointShell3Class1Cdf
- TileJointShell4Class1Cdf
- TileJointShell5Class1Cdf
- TileJointShell6Class1Cdf
- TileJointShellLastTwoClassesCdf
- TileShellOffsetLowClassCdf
- TileShellOffsetOtherClassCdf
- TileColMvGreaterCdf
- TileColMvIndexCdf
- TileJmvdScaleModeCdf

- TileJmvdAdaptiveScaleModeCdf
- TilePaletteYModeCdf
- TileIdentityRowYCdf
- TilePaletteYSizeCdf
- TilePaletteSize2YColorCdf
- TilePaletteSize3YColorCdf
- TilePaletteSize4YColorCdf
- TilePaletteSize5YColorCdf
- TilePaletteSize6YColorCdf
- TilePaletteSize7YColorCdf
- TilePaletteSize8YColorCdf
- TileDeltaQCdf
- TileIntraTxTypeLongCdf
- TileInterTxTypeLongCdf
- TileIsLongSideDctCdf
- TileIntraTxTypeSet1Cdf
- TileIntraTxTypeSet2Cdf
- TileInterTxTypeSet1Cdf
- TileInterTxTypeSet2Cdf
- TileInterTxTypeSet3Cdf
- TileInterTxTypeSet4Cdf
- TileInterTxTypeIndexSet1Cdf
- TileInterTxTypeIndexSet2Cdf
- TileInterTxTypeOffsetSet1Cdf
- TileInterTxTypeOffsetSet2Cdf
- TileInterIntraCdf
- TileWarpInterIntraCdf
- TileCflSignCdf
- TileWedgeInterIntraCdf
- TileCompGroupIdxCdf
- TileCompoundTypeCdf
- TileInterIntraModeCdf
- TileWedgeAngleDirCdf
- TileWedgeAngle0Cdf
- TileWedgeAngle1Cdf
- TileWedgeDist1Cdf

- TileWedgeDist2Cdf
- TileCflAlphaCdf
- TileCflIndexCdf
- TileCflMhDirCdf
- TileCflMhccpCdf
- TileUseAmvdCdf
- TileCcs0BlkCdf
- TileUseWienerNsCdf
- TileWienerNsLengthCdf
- TileWienerNsUvSymCdf
- TileWienerNsBaseCdf
- TileUsePcWienerCdf
- TileFlexRestorationTypeCdf
- TileTxbSkipCdf
- TileCctxTypeCdf
- TileEobPt16Cdf
- TileEobPt32Cdf
- TileEobPt64Cdf
- TileEobPt128Cdf
- TileEobPt256Cdf
- TileEobPt512Cdf
- TileEobPt1024Cdf
- TileEobExtraCdf
- TileDcSignCdf
- TileVTxbSkipCdf
- TileCoeffBaseEobCdf
- TileCoeffBaseLfEobCdf
- TileCoeffBaseCdf
- TileCoeffBaseLfCdf
- TileCoeffBasePhCdf
- TileCoeffBrCdf
- TileCoeffBrLfCdf
- TileCoeffBrUvCdf
- TileCoeffBaseLfUvCdf
- TileCoeffBaseLfEobUvCdf
- TileCoeffBaseUvCdf

- TileCoeffBaseEobUvCdf
- TileCoeffBaseBobCdf
- TileCoeffBrIdtxCdf
- TileCoeffBaseIdtxCdf
- TileIdtxSignCdf
- TileUseDipCdf
- TileDipModeCdf

§ 8.2.3. Boolean decoding process

This process decodes a pseudo-raw bit assuming equal probability for decoding a 0 or a 1.

This process is invoked when the function `read_bool( )` is called from the `read_literal` function in [§8.2.5 Parsing process for read_literal](#).

The variables `cur` and `symbol` are calculated as follows:

```
cur = SymbolRange >> 1
symbol = SymbolValue < cur
```

If `symbol` is equal to 0, `SymbolValue` is set equal to `SymbolValue - cur`.

The range and value are renormalized by the following ordered steps:

1. The variable `numBits` is set equal to `Clip3(0, 1, SymbolMaxBits)`. This represents the number of new bits to read from the bitstream.
2. The variable `newData` is read using the `f(numBits)` parsing process.
3. The variable `SymbolValue` is set to `(SymbolValue << 1) | (newData ^ 1)`.
4. The variable `SymbolMaxBits` is set to `SymbolMaxBits - 1`.

The return value from the function is given by `symbol`.

§ 8.2.4. Exit process for symbol decoder

This process is invoked when the function `exit_symbol( )` is called from the syntax structure.

It is a requirement of bitstream conformance that `SymbolMaxBits` is greater than or equal to -14 whenever this process is invoked.

The variable `trailingBitPosition` is set equal to `get_position() - Min(15, SymbolMaxBits+15)`.

The bitstream position indicator is advanced by `Max(0, SymbolMaxBits)`. (This skips over any trailing bits that have not already been read during symbol decode.)

The variable `paddingEndPosition` is set equal to `get_position()`.

NOTE: `paddingEndPosition` will always be a multiple of 8 indicating that the bit position is byte aligned.

It is a requirement of bitstream conformance that the bit at position `trailingBitPosition` is equal to 1.

It is a requirement of bitstream conformance that the bit at position `x` is equal to 0 for values of `x` strictly between `trailingBitPosition` and `paddingEndPosition`.

NOTE: This exit process consumes the OBU trailing bits for a [Tile Group](#).

The variable `numLog2` (specifying the base 2 logarithm of the number of tiles used in CDF averaging) is set equal to `Min( 3, FloorLog2( TileCols * TileRows ) )`.

The variables `copyCdf` and `avgCdf` (specifying whether to copy or average the CDFs) are set as follows:

```
copyCdf = 0
avgCdf = 0
if ( enable_avg_cdf && avg_cdf_type ) {
    avgCdf = TileNum < 1 << numLog2
} else {
    copyCdf = ( TileNum == context_update_tile_id )
}
```

If `copyCdf` is equal to 1, a copy is made of the final CDF values for each of the CDF arrays mentioned in the semantics for `init_coeff_cdfs` and `init_non_coeff_cdfs`. The name of the destination for the copy is the name of the CDF array prefixed with "Saved". The name of the source for the copy is the name of the CDF array prefixed with "Tile". For example, an array `SavedIdentityRowYCdf` will be created with values equal to `TileIdentityRowYCdf`.

If `avgCdf` is equal to 1, a copy with averaging is made of the final CDF values for each of the CDF arrays mentioned in the semantics for `init_coeff_cdfs` and `init_non_coeff_cdfs`. The name of the destination is the name of the CDF array prefixed with "Saved". The name of the source is the name of the CDF array prefixed with "Tile". For example, an array `SavedIdentityRowYCdf` will be created based on values from `TileIdentityRowYCdf`.

The copy with averaging works for each CDF of the `cdf` array in turn by calling the `avg_cdf` function with a reference to the destination array, a reference to the source array, and the length of each CDF as inputs.

For example, the array `SavedIdentityRowYCdf` will be created as follows:

```
for( i = 0; i < PALETTE_ROW_FLAG_CONTEXTS; i++ ) {
    avg_cdf( SavedIdentityRowYCdf[ i ], IdentityRowYCdf[ i ], 4 )
}
```

The `avg_cdf` function (which updates the destination CDF) is specified as:

```
avg_cdf( cdf, tilecdf, sz ) {
    if ( TileNum == 0 ) {
        for( i = 0; i < sz - 2; i++ ) {
            cdf[i] = 1 << 15
        }
        cdf[ sz - 2 ] = tilecdf[ sz - 2 ]
        cdf[ sz - 1 ] = 0
    }
    for( i = 0; i < sz - 2; i++ ) {
```

```

    cdf[ i ] -= ( ( 1 << 15 ) - tilecdf[ i ] ) >> numLog2
}
cdf[ sz - 1 ] += tilecdf[ sz - 1 ] >> numLog2

```

NOTE: The `cdf[ sz - 2 ]` element contains the rate and is copied from the first tile. The `cdf[ sz - 1 ]` element contains the activation count and is averaged across the tiles. The other elements contain CDF values.

§ 8.2.5. Parsing process for `read_literal`

This process is invoked when the function `read_literal( n )` is invoked.

This process is specified as follows:

```

FrameSymbolCount += n
x = 0
for ( i = 0 ; i < n; i++ ) {
    x = 2 * x + read_bool( )
}

```

The return value for the function is given by `x`.

§ 8.2.6. Symbol decoding process

The input to this process is an array `cdf` of length `N + 1` which specifies the cumulative distribution for a symbol with `N` possible values.

The output of this process is the variable `symbol`, containing a decoded syntax element. The process also modifies the input array `cdf` to adapt the probabilities to the content of the stream.

This process is invoked when the function `read_symbol( cdf )` is called.

NOTE: When this process is invoked, `N` will be greater than 1. `cdf[ N-1 ]` contains a constant that defines the rate of adaption. `cdf[N]` contains a count of the number of times this `cdf` has been used (up to a maximum of 32).

The variables `cur`, `prev`, and `symbol` are calculated as follows:

```

FrameSymbolCount++
cur = SymbolRange
symbol = -1
do {
    symbol++
    prev = cur
    if (symbol == N - 1) {
        f = 0
    } else {
        f = ( 1 << 15 ) - cdf[ symbol ]
    }
    pp = ((f >> EC_PROB_SHIFT) << 4) + Prob_Inc[ N - 2 ][ symbol ]
    cur = ( ( (SymbolRange >> 8) * pp) >> 7 ) << 3
} while ( SymbolValue < cur )

```

NOTE: Implementations may prefer to store the inverse `cdf` to move the subtraction out of this loop.

The variable `newRange` is set equal to `prev - cur`.

The variable `newValue` is set equal to `SymbolValue - cur`.

The range and value are renormalized by the following ordered steps:

1. The variable `bits` is set to `15 - FloorLog2( newRange )`. This represents the number of new bits to be added to `SymbolValue`.
2. The variable `SymbolRange` is set equal to `newRange << bits`.
3. The variable `numBits` is set equal to `Clip3(0, bits, SymbolMaxBits)`. This represents the number of new bits to read from the bitstream.
4. The variable `newData` is read using the `f(numBits)` parsing process.
5. The variable `paddedData` is set equal to `newData << ( bits - numBits )`.
6. The variable `mask` is set equal to `(1 << bits) - 1`.
7. The variable `SymbolValue` is set to `(newValue << bits) | (paddedData ^ mask)`.
8. The variable `SymbolMaxBits` is set to `SymbolMaxBits - bits`.

NOTE: `bits` may be equal to 0, in which case these ordered steps have no effect.

If `disable_cdf_update` is equal to 0, the cumulative distribution is updated as follows:

```
timeInterval = cdf[ N ] > 31 ? 2 : cdf[ N ] > 15 ? 1 : 0
rate = 3 + timeInterval + Min( FloorLog2( N ), 2 ) +
      Para_Adjustment_List[cdf[N - 1]][timeInterval]
for ( i = 0; i < N - 1; i++ ) {
    if ( i < symbol ) {
        cdf[ i ] -= cdf[ i ] >> rate
    } else {
        cdf[ i ] += ( ( 1 << 15 ) - cdf[ i ] ) >> rate
    }
}
cdf[ N ] += ( cdf[ N ] < 32 )
```

NOTE: The last entry of the `cdf` array is used to keep a count of the number of times the symbol has been decoded (up to a maximum of 32). This allows the `cdf` adaption rate to depend on the number of times the symbol has been decoded.

NOTE: The penultimate entry of the `cdf` array holds the (constant) base adaption rate for the `cdf`.

The return value from the function is given by `symbol`.

§ 8.3. Parsing process for CDF encoded syntax elements

§ 8.3.1. General

This process is invoked when the descriptor of a syntax element in the syntax tables is equal to `S`.

The input to this process is the name of a syntax element.

[§8.3.2 Cdf selection process](#) specifies how a CDF array is chosen for the syntax element. The variable `cdf` is set equal to a reference to this CDF array.

NOTE: The array must be passed by reference because `read_symbol` will adjust the array contents.

The output of this process is the result of calling the function `read_symbol( cdf )`.

§ 8.3.2. Cdf selection process

The input to this process is the name of a syntax element.

The output of this process is a reference to a CDF array.

When the description in this section uses variables, these variables are taken to have the values defined by the syntax tables at the point that the syntax element is being decoded.

The probabilities depend on the syntax element as follows:

use_intrabc: The `cdf` for `use_intrabc` is given by `TileIntrabcCdf[ ctx ]` where `ctx` is computed as follows:

```
ctx = 0
for( n = 0; n < NNum; n++ ) {
    if ( RefFrames[NPos[n][0]][NPos[n][1]][0] == INTRA_FRAME &&
         IsInters[NPos[n][0]][NPos[n][1]] ) {
        ctx += 1
    }
}
```

intrabc_mode: The `cdf` for `intrabc_mode` is given by `TileIntrabcModeCdf`.

intrabc_precision: The `cdf` for `intrabc_precision` is given by `TileIntrabcPrecisionCdf`.

morph_pred: The `cdf` for `morph_pred` is given by `TileMorphPredCdf[ctx]` where `ctx` is computed as follows:

```
ctx = 0
for( n = 0; n < NNum; n++ ) {
    ctx += MorphPreds[ NPos[ n ][ 0 ] ][ NPos[ n ][ 1 ] ]
}
```

tip_pred_mode: The `cdf` is given by `TileTipPredModeCdf`.

is_warp: The `cdf` is given by `TileIsWarpCdf[WarpMvCount]`.

use_gdf: The `cdf` is given by `TileUseGdfCdf`.

bru_mode: The `cdf` is given by `TileBruModeCdf`.

warp_mv: The `cdf` is given by `TileWarpMvCdf`.

warp_idx: The `cdf` is given by `TileWarpIdxCdf[idx]`.

warpmv_with_mvd: The `cdf` is given by `TileWarpWithMvdCdf`.

y_mode_set: The `cdf` for `y_mode_set` is given by `TileYModeSetCdf`.

y_mode_index: The cdf for y_mode_index is given by TileYModeIndexCdf[ctx] where ctx is computed as follows:

```
ctx = (get_joint_mode(0) >= NON_DIRECTIONAL_MODES_COUNT) +
      (get_joint_mode(1) >= NON_DIRECTIONAL_MODES_COUNT)
```

y_mode_offset: y_mode_offset uses the same derivation for the variable ctx as for the syntax element y_mode_index.

The cdf for y_mode_offset is given by TileYModeOffsetCdf[ctx].

uv_mode: The variable ctx is set equal to is_directional_mode(YMode).

The cdf for uv_mode is given by TileUVModeCflNotAllowedCdf[ctx].

is_cfl: The cdf is given by TileIsCflCdf[ctx] where ctx is computed as follows:

```
ctx = 0
if ( AvailUChroma && UVCfls[ ChromaMiRow - 1 ][ ChromaMiCol ] )
    ctx += 1
if ( AvailLChroma && UVCfls[ ChromaMiRow ][ ChromaMiCol - 1 ] )
    ctx += 1
```

cwp_idx: The cdf is given by TileCwpIdxCdf[idx].

fsc_mode: The cdf is given by TileFscModeCdf[ctx][Fsc_Bsize_Groups[MiSize]] where ctx is computed as follows:

```
if ( FrameIsIntra || RegionType == INTRA_REGION ) {
    ctx = 0
    for( n = 0; n < NNum; n++ ) {
        ctx += FscModes[ NPos[ n ][ 0 ] ][ NPos[ n ][ 1 ] ]
    }
} else {
    ctx = 3
}
```

and the constant table Fsc_Bsize_Groups is defined as:

```
Fsc_Bsize_Groups[BLOCK_SIZES] = {
    0, 1, 1, 2, 3, 3, 4, 5, 5, 5, 6, 6, 6, 6, 6, 6,
    6, 6, 6, 3, 3, 4, 4, 6, 6, 4, 4, 6, 6
}
```

mrl_index: The cdf is given by TileMrlIndexCdf[ctx] where ctx is computed as follows:

```
ctx = 0
for( n = 0; n < NNum; n++ ) {
    ctx += UsesMrls[ NPos[ n ][ 0 ] ][ NPos[ n ][ 1 ] ] > 0
}
```

mrl_sec_index: The cdf is given by TileMrlSecIndexCdf[ctx] where ctx is computed as follows:

```
ctx = 0
for(n = 0; n < NNum; n++) {
    ctx += UsesMrls[ NPos[ n ][ 0 ] ][ NPos[ n ][ 1 ] ] == 2
}
```

use_dpcm_y: The cdf is given by TileUseDpcmYCdf.

dpcm_mode_y: The cdf is given by TileDpcmModeYCdf.

use_dpcm_uv: The cdf is given by TileUseDpcmUvCdf.

dpcm_mode_uv: The cdf is given by TileDpcmModeUvCdf.

region_type: The cdf is given by TileRegionTypeCdf[ctx] where ctx is computed as follows:

```
numSamples = (num4x4wide * 4) * (num4x4high * 4)
if (numSamples <= 128)
    ctx = 0
else if (numSamples <= 512)
    ctx = 1
else if (numSamples <= 1024)
    ctx = 2
else
    ctx = 3
```

cdef_index0: The cdf is given by TileCdefIndex0Cdf[ctx] where ctx is computed as follows:

```
ctx = 0
cnt = 0
leftCol = (MiCol - cdefSize4) & cdefMask4
leftRow = MiRow & cdefMask4
if ( leftCol >= MiColStart ) {
    ctx += cdef_idx[ leftRow ][leftCol ] == 0
    cnt += 1
}
aboveCol = MiCol & cdefMask4
aboveRow = (MiRow - cdefSize4) & cdefMask4
shift = Mi_Width_Log2[ SbSize ]
curSbRow = MiRow >> shift
aboveSbRow = aboveRow >> shift
if ( aboveRow >= MiRowStart && aboveSbRow == curSbRow ) {
    ctx += cdef_idx[ aboveRow ][aboveCol ] == 0
    cnt += 1
}
if ( ctx != 0 && cnt == ctx ) {
    ctx += 1
}
```

cdef_index_minus_1: The cdf is given as follows:

- If CdefStrengths is equal to 3, the cdf is given by TileCdefIndexMinus1With3Cdf.
- If CdefStrengths is equal to 4, the cdf is given by TileCdefIndexMinus1With4Cdf.
- If CdefStrengths is equal to 5, the cdf is given by TileCdefIndexMinus1With5Cdf.
- If CdefStrengths is equal to 6, the cdf is given by TileCdefIndexMinus1With6Cdf.
- If CdefStrengths is equal to 7, the cdf is given by TileCdefIndexMinus1With7Cdf.

- Otherwise (CdefStrengths is equal to 8), the cdf is given by TileCdefIndexMinus1With8Cdf.

do_split: The variable ctx is computed as follows:

```
bsw = Max(Mi_Width_Log2[ bSize ], 1)
bsh = Max(Mi_Height_Log2[ bSize ], 1)
ctx1 = Mi_Height_Log2[ LeftMiSizes[ PlaneStart ][ r ] ] < bsh
ctx2 = Mi_Width_Log2[ AboveMiSizes[ PlaneStart ][ c ] ] < bsw
ctx = Partition_Size_Adjust[ bSize ] * 4 + ctx1 * 2 + ctx2
```

where Partition_Size_Adjust is defined as:

```
Partition_Size_Adjust[ BLOCK_SIZES ] = {
    0, 0, 0, 0, 1, 1, 1, 2,
    2, 2, 3, 3, 3, 4, 5, 6,
    7, 8, 9, 10, 11, 12, 13, 14,
    15, 0, 0, 0, 0, 0
}
```

The cdf for do_split is given by TileDoSplitCdf[PlaneStart][ctx].

do_square_split: The cdf is given by TileDoSquareSplitCdf[PlaneStart][ctx] where ctx is computed as follows:

```
bsw = Mi_Width_Log2[ bSize ]
bsh = Mi_Height_Log2[ bSize ]
above = AvailU && ( Mi_Width_Log2[ MiSizes[ PlaneStart ][ r - 1 ][ c ] ] < bsw )
left = AvailL && ( Mi_Height_Log2[ MiSizes[ PlaneStart ][ r ][ c - 1 ] ] < bsh )
ctx = (bSize == BLOCK_256X256 ? 4 : 0) + left * 2 + above
```

NOTE: PlaneStart will always be equal to 0 for do_square_split as the chroma partition is forced for large block sizes.

rect_type: The variable ctx is computed as follows:

```
bsw = Max(Mi_Width_Log2[ bSize ], 1)
bsh = Max(Mi_Height_Log2[ bSize ], 1)
ctx1 = Mi_Height_Log2[ LeftMiSizes[ PlaneStart ][ r ] ] < bsh
ctx2 = Mi_Width_Log2[ AboveMiSizes[ PlaneStart ][ c ] ] < bsw
ctx = Partition_Size_Adjust_Rect_Type[ bSize ] * 4 + ctx1 * 2 + ctx2
```

where Partition_Size_Adjust_Rect_Type is defined as:

```
Partition_Size_Adjust_Rect_Type[ BLOCK_SIZES ] = {
    0, 0, 0, 0, 1, 2, 0, 1,
    2, 3, 4, 5, 6, 7, 8, 9,
    10, 11, 12, 13, 14, 13, 14, 13,
    14, 0, 0, 0, 0, 0
}
```

The cdf for rect_type is given by TileRectTypeCdf[PlaneStart][ctx].

do_ext_partition: The variable ctx is computed as follows:

```
if (rectType == RECT_HORZ) {
    bsh = Max(Mi_Height_Log2[ bSize ] - 1, 1)
    ctx1 = Mi_Height_Log2[ LeftMiSizes[ PlaneStart ][ r ] ] < bsh
    ctx2 = Mi_Height_Log2[
        LeftMiSizes[ PlaneStart ]
            [ r + (Num_4x4_Blocks_High[ bSize ] >> 1) ] ] < bsh
} else {
    bsw = Max(Mi_Width_Log2[ bSize ] - 1, 1)
    ctx1 = Mi_Width_Log2[ AboveMiSizes[ PlaneStart ][ c ] ] < bsw
    ctx2 = Mi_Width_Log2[
        AboveMiSizes[ PlaneStart ]
            [ c + (Num_4x4_Blocks_Wide[ bSize ] >> 1) ] ] < bsw
}
adjSize = Partition_Size_Adjust[ bSize ]
ctx = adjSize * 4 + ctx1 * 2 + ctx2
```

The cdf for do_ext_partition is given by TileDoExtPartitionCdf[PlaneStart][ctx].

do_uneven_4way_partition: do_uneven_4way_partition uses the same derivation for the variable ctx as for the syntax element do_ext_partition.

The cdf for do_uneven_4way_partition is given by TileDoUneven4wayPartitionCdf[PlaneStart][ctx].

tx_do_partition: the cdf is given by TileTxDoPartitionCdf[fsc_mode][is_inter]
[Size_To_Tx_Part_Group_Lookup[MiSize]].

tx_2or3_partition_type: the cdf is given by TileTx2or3PartitionTypeCdf[fsc_mode][is_inter]
[Size_To_Tx_Type_Group_Vert_Or_Horz[MiSize] - 1].

tx_partition_type: the cdf is given by TileTxPartitionTypeCdf[fsc_mode][is_inter]
[Size_To_Tx_Type_Group_Vert_And_Horz[MiSize]].

lossless_inter_tx_type: the cdf is given by TileLosslessInterTxTypeCdf.

lossless_tx_size: the cdf is given by TileLosslessTxSizeCdf[Size_Group[MiSize]][is_inter].

sec_tx_type: The cdf is given by TileSecTxTypeCdf[is_inter][Tx_Size_Sqr[txSz]].

most_probable_stx_set: The cdf is given as follows:

- If TxType is equal to ADST_ADST and Tx_Width[txSz] is greater than or equal to 8 and Tx_Height[txSz] is greater than or equal to 8, the cdf is given by TileMostProbableStxSetAdstCdf.
- Otherwise, the cdf is given by TileMostProbableStxSetCdf.

seg_id_ext_flag: The cdf is given by TileSegIdExtFlagCdf[ctx], where the variable ctx is computed by:

```
if ( prevUL < 0 )
    ctx = 0
else if ( (prevUL == prevU) && (prevUL == prevL) )
    ctx = 2
else if ( (prevUL == prevU) || (prevUL == prevL) || (prevU == prevL) )
    ctx = 1
else
    ctx = 0
```

segment_id: if seg_id_ext_flag is equal to 0, the cdf is given by TileSegmentIdCdf[ctx]. Otherwise, the cdf is given by TileSegmentIdExtCdf[ctx].

The variable ctx is computed by:

```
if ( prevUL < 0 )
    ctx = 0
else if ( (prevUL == prevU) && (prevUL == prevL) )
    ctx = 2
else if ( (prevUL == prevU) || (prevUL == prevL) || (prevU == prevL) )
    ctx = 1
else
    ctx = 0
```

seg_id_predicted: the cdf is given by TileSegmentIdPredictedCdf[ctx], where ctx is computed by:

```
ctx = LeftSegPredContext[ MiRow ] + AboveSegPredContext[ MiCol ]
```

single_mode: the cdf is given by TileSingleModeCdf[NewMvContext].

use_most_probable_precision: the cdf is given by TileUseMostProbablePrecisionCdf[ctx] where ctx is computed by:

```
ctx = 0
for( n = 0; n < NNum; n++ ) {
    if ( UseMostProbablePrecisions[ NPos[n][0] ][ NPos[n][1] ] ) {
        ctx += 1
    }
}
```

pb_mv_precision: the cdf is given by TilePbMvPrecisionCdf[ctx][FrameMvPrecision - MV_PRECISION_HALF_PEL] where ctx is computed by:

```
ctx = 0
for( n = 0; n < NNum; n++ ) {
    if ( MvPrecisions[ NPos[n][0] ][ NPos[n][1] ] < FrameMvPrecision ) {
        ctx = 1
    }
}
```

jmvd_scale_mode: if use_amvd is equal to 1, the cdf is given by TileJmvdAdaptiveScaleModeCdf. Otherwise, the cdf is given by TileJmvdScaleModeCdf.

use_bawp: the cdf is given by TileUseBawpCdf.

use_bawp_chroma: the cdf is given by TileUseBawpChromaCdf.

explicit_bawp: the cdf is given by TileExplicitBawpCdf[ctx] where ctx is computed by:

```
ctx = (YMode == NEARMV) ? 0 : (YMode == NEWMV && use_amvd ? 1 : 2)
```

explicit_bawp_scale: the cdf is given by TileExplicitBawpScaleCdf.

use_amvd: the cdf is given by TileUseAmvdCdf[index][ctx] where index and ctx are computed by:

```

if ( YMode == NEAR_NEWMV ) {
    index = use_optflow ? 2 : 0
} else if ( YMode == NEW_NEARMV ) {
    index = use_optflow ? 3 : 1
} else if ( YMode == NEWMV ) {
    index = 4
} else if ( YMode == JOINT_NEWMV ) {
    index = use_optflow ? 6 : 5
} else { // NEW_NEWMV
    index = use_optflow ? 8 : 7
}
ctx = 0
for( n = 0; n < NNumBuf; n++ ) {
    ctx += NRefFrame[n][0] == RefFrame[0] &&
        UsesAmvds[NPosBuf[n][0]][NPosBuf[n][1]]
}

```

drl_mode: If RefFrame[0] is equal to TIP_FRAME, the cdf is given by TileTipDrlModeCdf[Min(idx, 2)]. Otherwise, if skip_mode is equal to 1, the cdf is given by TileSkipDrlModeCdf[Min(idx, 2)]. Otherwise (skip_mode is equal to 0 and RefFrame[0] is not equal to TIP_FRAME), the cdf is given by TileDrlModeCdf[Min(idx, 2)][NewMvContext].

is_inter: the cdf is given by TileIsInterCdf[ctx] where ctx is computed by:

```

if ( NNumBuf == 2 )
    ctx = ( NIntra[ 0 ] && NIntra[ 1 ] ) ? 3 : NIntra[ 0 ] || NIntra[ 1 ]
else if ( NNumBuf == 1 )
    ctx = 2 * NIntra[ 0 ]
else
    ctx = 0

```

dip_mode: the cdf is given by TileDipModeCdf.

use_dip: the cdf is given by TileUseDipCdf[ctx] where ctx is computed as follows:

```

ctx = 0
for( n = 0; n < NNum; n++ ) {
    ctx += UseDip[ NPos[ n ][ 0 ] ][ NPos[ n ][ 1 ] ]
}

```

tip_mode: the cdf is given by TileTipModeCdf[ctx] where ctx is computed as follows:

```

ctx = 0
for( n = 0; n < NNumBuf; n++ ) {
    ctx += NRefFrame[ n ][ 0 ] == TIP_FRAME
}

```

comp_mode: the cdf is given by TileCompModeCdf[ctx] where ctx is computed by:

```

if ( NNumBuf == 2 ) {
    if ( NSingle[0] && NSingle[1] )
        ctx = check_backward( NRefFrame[ 0 ][ 0 ] ) ^
            check_backward( NRefFrame[ 1 ][ 0 ] )
    else if ( NSingle[0] )
        ctx = 2 + ( check_backward( NRefFrame[ 0 ][ 0 ] ) || NIntra[ 0 ] )
    else if ( NSingle[1] )

```

```

        ctx = 2 + ( check_backward( NRefFrame[ 1 ][ 0 ] ) || NIntra[ 1 ] )
    else
        ctx = 4
} else if ( NNumBuf == 1 ) {
    if ( NSingle[ 0 ] )
        ctx = check_backward( NRefFrame[ 0 ][ 0 ] )
    else
        ctx = 3
} else {
    ctx = 1
}

```

where `check_backward` is a function specified as follows:

```

check_backward(refFrame) {
    if ( refFrame == TIP_FRAME ) {
        return 1
    }
    return is_inter_ref_frame(refFrame) && FrameDistance[refFrame] < 0
}

```

skip_mode: the cdf is given by `TileSkipModeCdf[ ctx ]` where `ctx` is computed by:

```

ctx = 0
for( n = 0; n < NNumBuf; n++ ) {
    ctx += SkipModes[ NPosBuf[n][0] ][ NPosBuf[n][1] ]
}

```

skip_flag: the cdf is given by `TileSkipCdf[ ctx ]` where `ctx` is computed by:

```

ctx = 0
for( n = 0; n < NNumBuf; n++ ) {
    ctx += Skips[ NPosBuf[n][0] ][ NPosBuf[n][1] ]
}
if ( skip_mode ) {
    ctx += (SKIP_CONTEXTS >> 1)
}

```

comp_ref: if `nFound` is equal to 0, the cdf is given by `TileCompRef0Cdf[ ctx ][ ref ]`. Otherwise, the cdf is given by `TileCompRef1Cdf[ ctx ][ bitType ][ ref ]` where `bitType` is equal to $(\text{FrameDistance}[\text{RefFrame}[0]] \geq 0) \wedge (\text{FrameDistance}[\text{ref}] \geq 0)$. The variable `ctx` is computed by:

```

thisRefCount = count_refs(ref)
nextRefsCount = 0
for ( i = ref + 1; i < NumTotalRefs; i++ ) {
    nextRefsCount += count_refs(i)
}
if (thisRefCount == nextRefsCount) {
    ctx = 1
} else if (thisRefCount < nextRefsCount) {
    ctx = 0
} else {
    ctx = 2
}

```

where `count_refs` is defined as:

```
count_refs(frameType) {
    c = 0
    for( n = 0; n < NNumBuf; n++ ) {
        for( list = 0; list < 2; list++ ) {
            if ( NRefFrame[ n ][ list ] == frameType ) c++
        }
    }
    return c
}
```

single_ref: the cdf is given by `TileSingleRefCdf[ ctx ][ ref ]` where `ctx` is computed as in the CDF selection process for `comp_ref`.

is_joint: the cdf is given by `TileIsJointCdf[ctx]` where `ctx` is computed by:

```
firstDist = Abs(get_relative_dist( OrderHints[ RefFrame[ 0 ] ], OrderHint ))
secondDist = Abs(get_relative_dist( OrderHints[ RefFrame[ 1 ] ], OrderHint ))
ctx = is_same_side() || firstDist != secondDist ||
      (OrderHints[ RefFrame[ 0 ] ] == RESTRICTED_OH) !=
      (OrderHints[ RefFrame[ 1 ] ] == RESTRICTED_OH)
```

compound_mode_non_joint: the cdf is given by `TileCompoundModeNonJointCdf[ NewMvContext ]`.

compound_mode_same_refs: the cdf is given by `TileCompoundModeSameRefsCdf[ NewMvContext ]`.

use_optflow: the cdf is given by `TileUseOptflowCdf[ YMode != NEAR_NEARMV ]`.

use_refinemv: the cdf is given by `TileUseRefinemvCdf[ ctx ]` where `ctx` is computed as follows:

```
ctx = 1 + (YMode - NEAR_NEARMV) + 6 * use_optflow
if (use_optflow && YMode > GLOBAL_GLOBALMV) {
    ctx -= 1
}
```

interp_filter: the cdf is given by `TileInterpFilterCdf[ ctx ]` where `ctx` is computed by:

```
ctx = is_inter_ref_frame( RefFrame[ 1 ] ) * 4
leftType = 3
aboveType = 3

if ( NNum > 0 ) {
    if ( RefFrames[ NPos[0][0] ][ NPos[0][1] ][ 0 ] == RefFrame[ 0 ] ||
        RefFrames[ NPos[0][0] ][ NPos[0][1] ][ 1 ] == RefFrame[ 0 ] )
        leftType = InterpFilters[ NPos[0][0] ][ NPos[0][1] ]
}
if ( NNum > 1 ) {
    if ( RefFrames[ NPos[1][0] ][ NPos[1][1] ][ 0 ] == RefFrame[ 0 ] ||
        RefFrames[ NPos[1][0] ][ NPos[1][1] ][ 1 ] == RefFrame[ 0 ] )
        aboveType = InterpFilters[ NPos[1][0] ][ NPos[1][1] ]
}

if ( leftType == aboveType )
    ctx += leftType
else if ( leftType == 3 )
    ctx += aboveType
else if ( aboveType == 3 )
```

```

    ctx += leftType
else
    ctx += 3

```

use_local_warp: the cdf is given by `TileUseLocalWarpCdf[ ctx ]` where `ctx` is computed by:

```

ctx = 0
hasWarp = 0
for( n = 0; n < NNum; n++ ) {
    m = MotionModes[ NPos[n][0] ][ NPos[n][1] ]
    if ( m >= LOCALWARP ) {
        hasWarp = 1
    }
    if ( m == LOCALWARP ) {
        ctx += 1
    }
}
ctx += hasWarp

```

use_extend_warp: the cdf is given by `TileUseExtendWarpCdf[ ctx ]` where `ctx` is computed by:

```

ctx = 0
for( n = 0; n < NNum; n++ ) {
    if ( MotionModes[ NPos[n][0] ][ NPos[n][1] ] >= LOCALWARP ) {
        ctx += 1
    }
}

```

mv_joint: the cdf is given by `TileMvJointAdaptiveCdf`.

amvd_index: the cdf is given by `TileAmvdIndicesCdf[ comp ]`.

shell_set: the cdf is given by `TileJointShellSetCdf[ MvCtx ]`.

shell_class: the cdf is given by `TileJointShellPClassQCdf[ MvCtx ]`.

where `Q` is equal to the value of `shell_set` and `P` is equal to the value of `MvPrecision` (`P` will be between 0 and 6 inclusive, except 2 is not reachable).

joint_shell_last_two_classes: the cdf is given by `TileJointShellLastTwoClassesCdf[ MvCtx ]`.

shell_offset_low_class: the cdf is given by `TileShellOffsetLowClassCdf[ MvCtx ][ shellClass ]`.

shell_offset_class2: the cdf is given by `TileShellOffsetClass2Cdf[ MvCtx ]`.

shell_offset_other_class: the cdf is given by `TileShellOffsetOtherClassCdf[ MvCtx ][ i ]`.

col_mv_greater: the cdf is given by `TileColMvGreaterCdf[ MvCtx ][ i ]`.

col_mv_index: the cdf is given by `TileColMvIndexCdf[ MvCtx ][ Min(shellClass, NUM_CTX_COL_MV_INDEX - 1) ]`.

all_zero: the variable `ctx` is computed as follows:

```

maxX4 = MiCols
maxY4 = MiRows
if ( plane > 0 ) {
    maxX4 = maxX4 >> SubsamplingX
}

```

```

    maxY4 = maxY4 >> SubsamplingY
}

w = Tx_Width[txSz]
h = Tx_Height[txSz]

bsize = get_plane_residual_size( plane > 0 ? ChromaMiSize : MiSize, plane )
bw = Block_Width[ bsize ]
bh = Block_Height[ bsize ]

if ( plane == 0 ) {
    top = 0
    left = 0
    for ( k = 0; k < w4; k++ ) {
        if ( x4 + k < maxX4 )
            top |= AboveLevelContext[ plane ][ x4 + k ]
    }
    for ( k = 0; k < h4; k++ ) {
        if ( y4 + k < maxY4 )
            left |= LeftLevelContext[ plane ][ y4 + k ]
    }
    top = Min( top, 4 )
    left = Min( left, 4 )
    if ( fsc_mode && enable_fsc ) {
        ctx = TXB_SKIP_CONTEXTS - 1
    } else if ( bw == w && bh == h ) {
        ctx = 0
    } else {
        ctx = (top + left + 3) >> 1
    }
} else {
    above = 0
    left = 0
    for ( i = 0; i < w4; i++ ) {
        if ( x4 + i < maxX4 ) {
            above |= AboveLevelContext[ plane ][ x4 + i ]
            above |= AboveDcContext[ plane ][ x4 + i ]
        }
    }
    for ( i = 0; i < h4; i++ ) {
        if ( y4 + i < maxY4 ) {
            left |= LeftLevelContext[ plane ][ y4 + i ]
            left |= LeftDcContext[ plane ][ y4 + i ]
        }
    }
    ctx = ( above != 0 ) + ( left != 0 )
    if ( plane == 2 ) {
        if ( bw * bh > w * h )
            ctx += 3
        if ( EobU != 0 )
            ctx += 6
    } else {
        ctx += 6
    }
}
}

```

If plane is equal to 2, the cdf is given by TileVTxbSkipCdf[ctx].

Otherwise (plane is equal to 0 or 1), the cdf is given by TileTxbSkipCdf[is_inter || fsc_mode][txSzCtx][ctx].

cctx_type: the cdf is given by TileCctxTypeCdf.

eob_pt_16: the cdf is given by TileEobPt16Cdf[eobCtx].

eob_pt_32: the cdf is given by TileEobPt32Cdf[eobCtx].

eob_pt_64: the cdf is given by TileEobPt64Cdf[eobCtx].

eob_pt_128: the cdf is given by TileEobPt128Cdf[eobCtx].

eob_pt_256: the cdf is given by TileEobPt256Cdf[eobCtx].

eob_pt_512: the cdf is given by TileEobPt512Cdf[eobCtx].

eob_pt_1024: the cdf is given by TileEobPt1024Cdf[eobCtx].

eob_extra: the cdf is given by TileEobExtraCdf.

coeff_base: the variables ctx, lfCtx, hfCtx are computed as follows:

```
adjTxSz = Adjusted_Tx_Size[ txSz ]
width = Tx_Width[ adjTxSz ]
height = Tx_Height[ adjTxSz ]
mag = 0
num = SIG_REF_DIFF_OFFSET_NUM
if (plane > 0) {
    num = txClass == TX_CLASS_2D ? 3 : 2
}
for ( idx = 0; idx < num; idx++ ) {
    refRow = row + Sig_Ref_Diff_Offset[ txClass ][ idx ][ 0 ]
    refCol = col + Sig_Ref_Diff_Offset[ txClass ][ idx ][ 1 ]
    magLimit = ( isLf && (txClass == TX_CLASS_2D || idx < 2) &&
        !(isHidden && c == 0) ) ? 5 : 3
    if (refRow < height && refCol < width) {
        mag += Min( Level[ refRow ][ refCol ], magLimit )
    }
}
ctx = ( mag + 1 ) >> 1
if (plane > 0) {
    ctx2 = Min( ctx, 3 )
    if (txClass != TX_CLASS_2D) {
        uvCtx = ctx2 + LF_SIG_COEF_CONTEXTS_2D_UV
    } else {
        uvCtx = (plane == 1) ? ctx2 : ctx2 + 4
    }
} else if (isLf) {
    if (txClass == TX_CLASS_2D) {
        if (c == 0) {
            lfCtx = Min(ctx, 8)
        } else if (row + col < 2) {
            lfCtx = Min(ctx, 6) + 9
        } else {
            lfCtx = Min(ctx, 4) + 16
        }
    } else {
        idx = txClass == TX_CLASS_HORIZ ? col : row
        if (idx == 0) {
            lfCtx = LF_SIG_COEF_CONTEXTS_2D + Min(ctx, 6)
        } else {
            lfCtx = LF_SIG_COEF_CONTEXTS_2D + 7 + Min(ctx, 4)
        }
    }
} else {
    ctx2 = Min( ctx, 4 )
    if ( txClass == TX_CLASS_2D ) {
        if (row + col < 6) {
            hfCtx = ctx2
        } else if (row + col < 8) {
            hfCtx = ctx2 + 5
        } else {
            hfCtx = ctx2 + 10
        }
    }
}
```



```

    } else {
        hfCtx = ctx2 + 15
    }
}

```

If isHidden is equal to 1 and c is equal to 0, the cdf is given by TileCoeffBasePhCdf[Min(ctx,4)].

Otherwise, if plane is not equal to 0 and isLf is equal to 1, the cdf is given by TileCoeffBaseLfUvCdf[uvCtx].

Otherwise, if plane is not equal to 0, the cdf is given by TileCoeffBaseUvCdf[uvCtx].

Otherwise, if isLf is equal to 1, the cdf is given by TileCoeffBaseLfCdf[txSzCtx][lfCtx][(tcqState >> 1) & 1].

Otherwise, the cdf is given by TileCoeffBaseCdf[txSzCtx][hfCtx][(tcqState >> 1) & 1].

coeff_base_eob: the variable ctx is computed as follows:

```

adjTxSz = Adjusted_Tx_Size[ txSz ]
bwl = Tx_Width_Log2[ adjTxSz ]
height = Tx_Height[ adjTxSz ]
if (c == 0) {
    ctx = SIG_COEF_CONTEXTS_EOB - 4
} else if (c <= (height << bwl) / 8) {
    ctx = SIG_COEF_CONTEXTS_EOB - 3
} else if (c <= (height << bwl) / 4) {
    ctx = SIG_COEF_CONTEXTS_EOB - 2
} else {
    ctx = SIG_COEF_CONTEXTS_EOB - 1
}

```

If plane is not equal to 0 and isLf is equal to 1, the cdf is given by TileCoeffBaseLfEobUvCdf[ctx].

Otherwise, if plane is not equal to 0, the cdf is given by TileCoeffBaseEobUvCdf[ctx].

Otherwise, if isLf is equal to 1, the cdf is given by TileCoeffBaseLfEobCdf[txSzCtx][ctx].

Otherwise (plane is equal to 0 and isLf is equal to 0), the cdf is given by TileCoeffBaseEobCdf[txSzCtx][ctx].

coeff_base_bob: the cdf is given by TileCoeffBaseBobCdf[Min(TX_16X16,txSzCtx)][ctx] where ctx is computed as follows:

```

if ( bob <= (segEob>>3) ) {
    ctx = 0
} else if ( bob <= (segEob>>2) ) {
    ctx = 1
} else {
    ctx = 2
}

```

coeff_base_idtx: the cdf is given by `TileCoeffBaseIdtxCdf[ Min(TX_16X16,txSzCtx) ][ mag ]` where `mag` is computed as follows:

```
mag = 0
if (col > 0) mag += Min( 3, Level[ row ][ col - 1 ] )
if (row > 0) mag += Min( 3, Level[ row - 1 ][ col ] )
```

coeff_br_idtx: the cdf is given by `TileCoeffBrIdtxCdf[ Min(TX_16X16,txSzCtx) ][ mag ]` where `mag` is computed as follows:

```
mag = 0
if (col > 0) mag += Min( MAX_BASE_BR_RANGE - 1, Level[ row ][ col - 1 ] )
if (row > 0) mag += Min( MAX_BASE_BR_RANGE - 1, Level[ row - 1 ][ col ] )
mag = Min(mag, 6)
```

idtx_sign: the cdf is given by `TileIdtxSignCdf[ Min(TX_16X16,txSzCtx) ][ ctx ]` where `ctx` is computed as follows:

```
adjTxSz = Adjusted_Tx_Size[ txSz ]
txw = Tx_Width[ adjTxSz ]
signc = 0
if (col > 0) signc += QuantSign[ row * txw + col - 1 ]
if (row > 0) signc += QuantSign[ (row - 1) * txw + col ]
if (col > 0 && row > 0) signc += QuantSign[ (row - 1) * txw + col - 1 ]
if (signc > 2) ctx = 5
else if (signc < -2) ctx = 6
else if (signc > 0) ctx = 1
else if (signc < 0) ctx = 2
else ctx = 0
if ( Level[ row ][ col ] > COEFF_BASE_RANGE && ctx != 0 ) {
    ctx += 2
}
```

dc_sign: the variable `ctx` is computed as follows:

```
maxX4 = MiCols
maxY4 = MiRows
dcSign = 0
for ( k = 0; k < w4; k++ ) {
    if ( x4 + k < maxX4 ) {
        sign = AboveDcContext[ plane ][ x4 + k ]
        if ( sign == 1 ) {
            dcSign--
        } else if ( sign == 2 ) {
            dcSign++
        }
    }
}
for ( k = 0; k < h4; k++ ) {
    if ( y4 + k < maxY4 ) {
        sign = LeftDcContext[ plane ][ y4 + k ]
        if ( sign == 1 ) {
            dcSign--
        } else if ( sign == 2 ) {
            dcSign++
        }
    }
}
if ( dcSign < 0 ) {
    ctx = 1
} else if ( dcSign > 0 ) {
```

```

    ctx = 2
} else {
    ctx = 0
}

```

The cdf is given by `TileDcSignCdf[ ptype ][ isHidden ][ ctx ]`.

dc_sign_horz_vert: The cdf is given by `TileDcSignCdf[ ptype ][ isHidden ][ 0 ]`.

coeff_br: the variables `mag` and `ctx` are computed as follows:

```

adjTxSz = Adjusted_Tx_Size[ txSz ]
bwl = Tx_Width_Log2[ adjTxSz ]
txw = Tx_Width[ adjTxSz ]
txh = Tx_Height[ adjTxSz ]
row = pos >> bwl
col = pos - (row << bwl)

mag = 0

txType = compute_tx_type( plane, txSz, x4, y4 )
txClass = get_tx_class( txType )
num = 3
if ( txClass != TX_CLASS_2D && plane > 0 ) {
    num = 2
}
for ( idx = 0; idx < num; idx++ ) {
    refRow = row + Mag_Ref_Offset_With_Tx_Class[ txClass ][ idx ][ 0 ]
    refCol = col + Mag_Ref_Offset_With_Tx_Class[ txClass ][ idx ][ 1 ]
    if ( refRow < txh &&
        refCol < txw ) {
        mag += Min( Level[ refRow ][ refCol ], MAX_BASE_BR_RANGE - 1 )
    }
}

mag = Min( ( mag + 1 ) >> 1, 6 )
if ( plane > 0 ) {
    ctx = Min(mag, 3)
} else if ( pos == 0 ) {
    if (txClass != 0) {
        ctx = mag + 7
    } else {
        ctx = mag
    }
} else {
    if (isLf) {
        ctx = mag + 7
    } else {
        ctx = mag
    }
}

```

where `Mag_Ref_Offset_With_Tx_Class` is defined as:

```

Mag_Ref_Offset_With_Tx_Class[ 3 ][ 3 ][ 2 ] = {
    { { 0, 1 }, { 1, 0 }, { 1, 1 } },
    { { 0, 1 }, { 1, 0 }, { 0, 2 } },
    { { 0, 1 }, { 1, 0 }, { 2, 0 } }
}

```

and `get_tx_class` is defined as:

```
get_tx_class( txType ) {
    if ( ( txType == V_DCT ) ||
        ( txType == V_ADST ) ||
        ( txType == V_FLIPADST ) ) {
        return TX_CLASS_VERT
    } else if ( ( txType == H_DCT ) ||
        ( txType == H_ADST ) ||
        ( txType == H_FLIPADST ) ) {
        return TX_CLASS_HORIZ
    } else
        return TX_CLASS_2D
}
```

If plane is not equal to 0, the cdf is given by `TileCoeffBrUvCdf[ ctx ]`.

Otherwise, if `isLf` is equal to 1, the cdf is given by `TileCoeffBrLfCdf[ ctx ]`.

Otherwise, the cdf is given by `TileCoeffBrCdf[ ctx ]`.

has_palette_y: the cdf is given by `TilePaletteYModeCdf`.

palette_size_y_minus_2: the cdf is given by `TilePaletteYSizeCdf`.

palette_color_idx_y: the cdf depends on `PaletteSizeY`, as specified in [Table 8.1](#):

Table 8.1: Values for `palette_color_idx_y`

PaletteSizeY	cdf
2	<code>TilePaletteSize2YColorCdf[ctx]</code>
3	<code>TilePaletteSize3YColorCdf[ctx]</code>
4	<code>TilePaletteSize4YColorCdf[ctx]</code>
5	<code>TilePaletteSize5YColorCdf[ctx]</code>
6	<code>TilePaletteSize6YColorCdf[ctx]</code>
7	<code>TilePaletteSize7YColorCdf[ctx]</code>
8	<code>TilePaletteSize8YColorCdf[ctx]</code>

where `ctx` is computed as follows:

```
ctx = Palette_Color_Context[ ColorContextHash ]
```

identity_row_y: the cdf is given by `TileIdentityRowYCdf[ prevIdentityRow ]`.

delta_q_abs: the cdf is given by `TileDeltaQCdf`.

intra_tx_type: the cdf depends on the variable set, as specified in [Table 8.2](#):

Table 8.2: Values for `intra_tx_type`

set	cdf
<code>TX_SET_WIDE_64</code>	<code>TileIntraTxTypeLongCdf[Tx_Size_Sqr[txSz]]</code>

TX_SET_WIDE_32	TileIntraTxTypeLongCdf[Tx_Size_Sqr[txSz]]
TX_SET_HIGH_64	TileIntraTxTypeLongCdf[Tx_Size_Sqr[txSz]]
TX_SET_HIGH_32	TileIntraTxTypeLongCdf[Tx_Size_Sqr[txSz]]
TX_SET_INTRA_1	TileIntraTxTypeSet1Cdf[Tx_Size_Sqr[txSz]]
TX_SET_INTRA_2	TileIntraTxTypeSet2Cdf[Tx_Size_Sqr[txSz]]

is_long_side_dct: the cdf is given by TileIsLongSideDctCdf[is_inter].

inter_tx_type: the variables ctx and sqrSz are computed as follows:

```

bwl = Min( Tx_Width_Log2[ txSz ], 5)
eoby = (eob - 1) >> bwl
eobx = (eob - 1) - (eoby << bwl)
diag = eobx + eoby
ctx = 0
if (diag < 2) {
    ctx = 1
} else if (diag > (Min(Tx_Width[txSz], 32) + Min(Tx_Height[txSz], 32) - 4)) {
    ctx = 2
}
sqrSz = Tx_Size_Sqr[ txSz ]

```

the cdf depends on the variable set, as specified in [Table 8.3](#):

Table 8.3: CDF selection for inter_tx_type based on transform set

set	cdf
TX_SET_WIDE_64	TileInterTxTypeLongCdf[ctx][sqrSz]
TX_SET_WIDE_32	TileInterTxTypeLongCdf[ctx][sqrSz]
TX_SET_HIGH_64	TileInterTxTypeLongCdf[ctx][sqrSz]
TX_SET_HIGH_32	TileInterTxTypeLongCdf[ctx][sqrSz]
TX_SET_INTER_1	TileInterTxTypeSet1Cdf[ctx][sqrSz]
TX_SET_INTER_2	TileInterTxTypeSet2Cdf[ctx]
TX_SET_DCT_IDTX	TileInterTxTypeSet3Cdf[ctx][sqrSz]
TX_SET_DCT_IDTX_IDDCT	TileInterTxTypeSet4Cdf[ctx][sqrSz]

inter_tx_type_offset: the variable ctx is computed as follows:

```

bwl = Min( Tx_Width_Log2[ txSz ], 5)
eoby = (eob - 1) >> bwl
eobx = (eob - 1) - (eoby << bwl)
diag = eobx + eoby
ctx = 0
if (diag < 2) {
    ctx = 1
} else if (diag > (Tx_Width[txSz] + Tx_Height[txSz] - 4)) {
    ctx = 2
}

```

The cdf is given as follows:

- If set is equal to TX_SET_INTER_1 and inter_tx_type is equal to 0, the cdf is given by TileInterTxTypeIndexSet1Cdf[ctx].
- If set is equal to TX_SET_INTER_1 and inter_tx_type is equal to 1, the cdf is given by TileInterTxTypeOffsetSet1Cdf[ctx].
- If set is equal to TX_SET_INTER_2 and inter_tx_type is equal to 0, the cdf is given by TileInterTxTypeIndexSet2Cdf[ctx].
- If set is equal to TX_SET_INTER_2 and inter_tx_type is equal to 1, the cdf is given by TileInterTxTypeOffsetSet2Cdf[ctx].

comp_group_idx: The cdf is given by TileCompGroupIdxCdf[ctx], where ctx is computed as follows:

```
bckOrderHint = OrderHints[ RefFrame[ 0 ] ]
fwdOrderHint = OrderHints[ RefFrame[ 1 ] ]
bck = Abs(get_relative_dist( OrderHint, fwdOrderHint ))
fwd = Abs(get_relative_dist( bckOrderHint, OrderHint ))
offset = ( fwd == bck )
ctxs[ 0 ] = 0
ctxs[ 1 ] = 0
for( n = 0; n < NNumBuf; n++ ) {
    if ( !NSingle[n] )
        ctxs[ n ] = CompGroupIdxs[ NPosBuf[n][0] ][ NPosBuf[n][1] ]
    else if ( NRefFrame[ n ][ 0 ] == FurthestFuture )
        ctxs[ n ] = 2
}
ctx0 = ctxs[ 0 ]
ctx1 = ctxs[ 1 ]
ctx = ctx1 + ctx0 + ( Min(ctx1,ctx0) > 0 ) + offset * 6
```

compound_type: The cdf is given by TileCompoundTypeCdf.

inter_intra: The cdf is given by TileInterIntraCdf[ctx], where ctx is computed as follows:

```
ctx = Size_Group[ MiSize ]
```

warp_inter_intra: The cdf is given by TileWarpInterIntraCdf[ctx], where ctx is computed as follows:

```
ctx = Size_Group[ MiSize ]
```

interintra_mode: The cdf is given by TileInterIntraModeCdf[ctx], where ctx is computed as follows:

```
ctx = Size_Group[ MiSize ]
```

wedge_quad: The cdf is given by TileWedgeQuadCdf.

wedge_angle: The cdf is given by TileWedgeAngleCdf[wedge_quad].

wedge_dist1: The cdf is given by TileWedgeDist1Cdf.

wedge_dist2: The cdf is given by TileWedgeDist2Cdf.

wedge_interintra: The cdf is given by TileWedgeInterIntraCdf.

warp_delta_precision: The cdf is given by TileWarpDeltaPrecisionCdf[MiSize].

warp_delta_param_low: The cdf is given by TileWarpDeltaParamLowCdf[idx==3 || idx==4].

warp_delta_param_high: The cdf is given by TileWarpDeltaParamHighCdf[idx==3 || idx==4].

warp_delta_param_sign: The cdf is given by TileWarpDeltaParamSignCdf.

ccso_blk: The cdf is given by TileCcsoBlkCdf[plane][ctx], where ctx is computed as follows:

```
if ( MiCol - blkW4 >= MiColStart ) {  
    ctx = 2 * CcsoBlks[ plane ][ MiRow >> shiftRow ]  
                                     [ (MiCol - blkW4) >> shiftCol ]  
} else {  
    ctx = 0  
}
```

cfl_index: The cdf is given by TileCflIndexCdf.

cfl_alpha_signs: The cdf is given by TileCflSignCdf.

cfl_alpha_u: The cdf is given by TileCflAlphaCdf[ctx], where ctx is obtained from the following table:

Table 8.4: Context selection for cfl_alpha_u

cfl_alpha_signs	ctx
0	N/A
1	N/A
2	0
3	1
4	2
5	3
6	4
7	5

NOTE: N/A is used to indicate that no context is needed as the sign is zero and no value is decoded.

or computed as follows:

```
ctx = (signU - 1) * 3 + signV
```

NOTE: As shown in the previous table, the variable ctx produced by this calculation will be equal to cfl_alpha_signs - 2.

cfl_alpha_v: The cdf is given by TileCflAlphaCdf[ctx], where ctx is obtained from the following [Table 8.5](#):

Table 8.5: Context calculation for cfl_alpha_v based on cfl_alpha_signs

cfl_alpha_signs	ctx
0	0
1	3
2	N/A
3	1
4	4
5	N/A
6	2
7	5

NOTE: N/A is used to indicate that no context is needed as the sign is zero and no value is decoded.

or computed as follows:

```
ctx = (signV - 1) * 3 + signU
```

cfl_mhccp: The cdf is given by TileCflMhccpCdf.

cfl_mh_dir: The cdf is given by TileCflMhDirCdf[Size_Group[MiSize]].

use_wiener_ns: The cdf is given by TileUseWienerNsCdf.

use_pc_wiener: The cdf is given by TileUsePcWienerCdf.

flex_restoration_type: The cdf is given by TileFlexRestorationTypeCdf[tool][plane].

wiener_ns_base: The cdf is given by TileWienerNsBaseCdf.

wiener_ns_length: The cdf is given by TileWienerNsLengthCdf[Min(plane, 1)].

wiener_ns_uv_sym: The cdf is given by TileWienerNsUvSymCdf.

[↑ Back to Table of Contents](#)

§ 9. Additional tables

§ 9.1. General

This section contains tables that do not naturally fit in the main sections of the specification.

§ 9.2. Conversion tables

This section defines the constant lookup tables used to convert between different representations.

For a block size x (with values having the same interpretation as for the variable `subSize`), `Mi_Width_Log2[ x ]` gives the base 2 logarithm of the width of the block in units of 4 samples.

```
Mi_Width_Log2[ BLOCK_SIZES ] =  
{  
    0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3,  
    4, 4, 4, 5, 5, 5, 6, 6, 0, 2, 1, 3, 2, 4,  
    0, 3, 1, 4  
}
```

For a block size x , `Mi_Height_Log2[ x ]` gives the base 2 logarithm of the height of the block in units of 4 samples.

```
Mi_Height_Log2[ BLOCK_SIZES ] =  
{  
    0, 1, 0, 1, 2, 1, 2, 3, 2, 3, 4,  
    3, 4, 5, 4, 5, 6, 5, 6, 2, 0, 3, 1, 4, 2,  
    3, 0, 4, 1  
}
```

For a block size x , `Num_4x4_Blocks_Wide[ x ]` gives the width of the block in units of 4 samples.

```
Num_4x4_Blocks_Wide[ BLOCK_SIZES ] =  
{  
    1, 1, 2, 2, 2, 4, 4, 4, 8, 8, 8,  
    16, 16, 16, 32, 32, 32, 64, 64, 1, 4, 2, 8, 4, 16,  
    1, 8, 2, 16  
}
```

For a block size x , `Block_Width[ x ]` gives the width of the block in units of samples. `Block_Width[ x ]` is defined to be equal to $4 * \text{Num_4x4_Blocks_Wide}[x]$.

For a block size x , `Num_4x4_Blocks_High[ x ]` gives the height of the block in units of 4 samples.

```
Num_4x4_Blocks_High[ BLOCK_SIZES ] =  
{  
    1, 2, 1, 2, 4, 2, 4, 8, 4, 8, 16,  
    8, 16, 32, 16, 32, 64, 32, 64, 4, 1, 8, 2, 16, 4,  
    8, 1, 16, 2  
}
```

For a block size x , `Block_Height[ x ]` gives the height of the block in units of samples. `Block_Height[ x ]` is defined to be equal to $4 * \text{Num_4x4_Blocks_High}[x]$.

Size_Group is used to map a block size into a context for intra syntax elements.

```
Size_Group[ BLOCK_SIZES ] =
{
    0, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3,
    3, 3, 3, 3, 3, 3, 3, 3, 0, 0, 1, 1, 2, 2,
    1, 1, 2, 2
}
```

For a luma block size x, Max_Tx_Size_Rect[x] returns the largest transform size that can be used for blocks of size x (this can be either square or rectangular).

```
Max_Tx_Size_Rect[ BLOCK_SIZES ] =
{
    TX_4X4, TX_4X8, TX_8X4, TX_8X8,
    TX_8X16, TX_16X8, TX_16X16, TX_16X32,
    TX_32X16, TX_32X32, TX_32X64, TX_64X32,
    TX_64X64, TX_64X64, TX_64X64, TX_64X64,
    TX_64X64, TX_64X64, TX_64X64,
    TX_4X16, TX_16X4, TX_8X32, TX_32X8,
    TX_16X64, TX_64X16, TX_4X32, TX_32X4,
    TX_8X64, TX_64X8
}
```

For a block size x, and a partition type p, Partition_Subsize[p][x] returns the size of the sub-blocks used by this partition. (If the partition produces blocks of different sizes, then the table contains one of the sub-block sizes.)

```
Partition_Subsize[EXT_PARTITION_TYPES][BLOCK_SIZES] =
{
    { // PARTITION_NONE
        BLOCK_4X4, BLOCK_4X8, BLOCK_8X4, BLOCK_8X8, BLOCK_8X16, BLOCK_16X8,
        BLOCK_16X16, BLOCK_16X32, BLOCK_32X16, BLOCK_32X32, BLOCK_32X64,
        BLOCK_64X32, BLOCK_64X64, BLOCK_64X128, BLOCK_128X64, BLOCK_128X128,
        BLOCK_128X256, BLOCK_256X128, BLOCK_256X256,
        BLOCK_4X16, BLOCK_16X4, BLOCK_8X32, BLOCK_32X8, BLOCK_16X64, BLOCK_64X16,
        BLOCK_4X32, BLOCK_32X4,
        BLOCK_8X64, BLOCK_64X8,
    }, { // PARTITION_HORZ
        BLOCK_INVALID, BLOCK_4X4, BLOCK_INVALID, BLOCK_8X4, BLOCK_8X8, BLOCK_16X4,
        BLOCK_16X8, BLOCK_16X16, BLOCK_32X8, BLOCK_32X16, BLOCK_32X32, BLOCK_64X16,
        BLOCK_64X32, BLOCK_64X64, BLOCK_INVALID, BLOCK_128X64,
        BLOCK_128X128, BLOCK_INVALID, BLOCK_256X128,
        BLOCK_4X8, BLOCK_INVALID,
        BLOCK_8X16, BLOCK_32X4,
        BLOCK_16X32, BLOCK_64X8,
        BLOCK_INVALID, BLOCK_INVALID,
        BLOCK_INVALID, BLOCK_INVALID,
    }, { // PARTITION_VERT
        BLOCK_INVALID, BLOCK_INVALID, BLOCK_4X4, BLOCK_4X8, BLOCK_4X16, BLOCK_8X8,
        BLOCK_8X16, BLOCK_8X32, BLOCK_16X16, BLOCK_16X32, BLOCK_16X64, BLOCK_32X32,
        BLOCK_32X64, BLOCK_INVALID, BLOCK_64X64, BLOCK_64X128,
        BLOCK_INVALID, BLOCK_128X128, BLOCK_128X256,
        BLOCK_INVALID, BLOCK_8X4,
        BLOCK_4X32, BLOCK_16X8,
        BLOCK_8X64, BLOCK_32X16,
        BLOCK_INVALID, BLOCK_INVALID,
        BLOCK_INVALID, BLOCK_INVALID,
    }, { // PARTITION_HORZ_3
        BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID, BLOCK_8X4,
        BLOCK_INVALID, BLOCK_16X4,
        BLOCK_16X8, BLOCK_32X4, BLOCK_32X8,
        BLOCK_32X16, BLOCK_64X8, BLOCK_64X16,
        BLOCK_INVALID, BLOCK_INVALID,
        BLOCK_INVALID,
    }
```

```

BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_8X8, BLOCK_INVALID,
BLOCK_16X16, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
}, { // PARTITION_VERT_3
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_4X8, BLOCK_4X16,
BLOCK_4X32, BLOCK_8X16, BLOCK_8X32,
BLOCK_8X64, BLOCK_16X32, BLOCK_16X64,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID, BLOCK_8X8,
BLOCK_INVALID, BLOCK_16X16,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
},
{ // PARTITION_HORZ_4A
BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_16X4, BLOCK_INVALID, BLOCK_32X4,
BLOCK_32X8, BLOCK_INVALID, BLOCK_64X8,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_8X4, BLOCK_INVALID,
BLOCK_16X8, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
}, { // PARTITION_HORZ_4B
BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_16X4, BLOCK_INVALID, BLOCK_32X4,
BLOCK_32X8, BLOCK_INVALID, BLOCK_64X8,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_8X4, BLOCK_INVALID,
BLOCK_16X8, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
}, { // PARTITION_VERT_4A
BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_4X16, BLOCK_4X32,
BLOCK_INVALID, BLOCK_8X32, BLOCK_8X64,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_4X8,
BLOCK_INVALID, BLOCK_8X16,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
}, { // PARTITION_VERT_4B
BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_4X16, BLOCK_4X32,
BLOCK_INVALID, BLOCK_8X32, BLOCK_8X64,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_4X8,
BLOCK_INVALID, BLOCK_8X16,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
},
{ // PARTITION_SPLIT

```

```

BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_64X64,
BLOCK_INVALID, BLOCK_INVALID, BLOCK_128X128,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
BLOCK_INVALID, BLOCK_INVALID,
}
}

```

```

H_Partition_Midsize[ BLOCK_SIZES ] =
{
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_4X8,
    BLOCK_8X4,
    BLOCK_8X8,
    BLOCK_8X16,
    BLOCK_16X8,
    BLOCK_16X16,
    BLOCK_16X32,
    BLOCK_32X16,
    BLOCK_32X32,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_4X16,
    BLOCK_16X4,
    BLOCK_8X32,
    BLOCK_32X8,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID
}

```

```

Mode_To_Txfrm[ UV_INTRA_MODES_CFL_ALLOWED ] =
{
    DCT_DCT,    // DC_PRED
    ADST_DCT,   // V_PRED
    DCT_ADST,   // H_PRED
    DCT_DCT,    // D45_PRED
    ADST_ADST,  // D135_PRED
    ADST_DCT,   // D113_PRED
    DCT_ADST,   // D157_PRED
    DCT_ADST,   // D203_PRED
    ADST_DCT,   // D67_PRED
    ADST_ADST,  // SMOOTH_PRED
    ADST_DCT,   // SMOOTH_V_PRED
    DCT_ADST,   // SMOOTH_H_PRED
    ADST_ADST,  // PAETH_PRED
    DCT_DCT,    // UV_CFL_PRED
}

```

```

Palette_Color_Context[ PALETTE_MAX_COLOR_CONTEXT_HASH + 1 ] =
{ -1, -1, 0, -1, -1, 4, 3, 2, 1 }

```

NOTE: The negative numbers in the array `Palette_Color_Context` indicate values that will never be accessed.

```
Palette_Color_Hash_Multipliers[ PALETTE_NUM_NEIGHBORS ] = { 1, 2, 2 }
```

```
Mode_To_Angle[ INTRA_MODES ] =  
{  
    0, 90, 180, 45, 135, 113, 157, 203, 67, 0, 0, 0, 0  
}
```

```
Dr_Intra_Derivative[ 90 ] =  
{  
    0,      4096, 2048,  
    1365, 1024, 819,  
    682, 585, 512,  
    455, 409, 409, 409, 372,  
    341, 292, 273,  
    256, 227, 215,  
    204, 186, 178,  
    170, 157, 151,  
    146, 136, 132,  
    128, 117, 110,  
    107, 99, 97, 97,  
    93, 87, 83,  
    81, 77, 74,  
    73, 69, 66,  
    64, 62, 59,  
    56, 55, 53,  
    50, 49, 47,  
    44, 42, 42, 41,  
    38, 37, 35,  
    32, 31, 30,  
    28, 27, 26,  
    24, 23, 22,  
    20, 19, 18,  
    16, 15, 14,  
    12, 11, 10, 10, 10,  
    9, 8, 7,  
    6, 5, 4,  
    3, 2, 1  
}
```

```
Side_Thresholds[MAX_SIDE_TABLE] =  
{  
    -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16,  
    -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16,  
    -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16, -16,  
    -16, -16, -16, -16, -16, -14, -13, -11, -10, -9, -7, -6, -4,  
    -3, -2, 0, 0, 2, 3, 5, 6, 7, 9, 10, 12, 13,  
    15, 16, 17, 19, 20, 22, 23, 24, 26, 27, 29, 30, 32,  
    33, 34, 36, 37, 39, 40, 42, 43, 44, 46, 47, 49, 50,  
    51, 53, 54, 56, 57, 59, 60, 61, 63, 64, 66, 67, 69,  
    70, 71, 73, 74, 76, 77, 78, 80, 81, 83, 84, 86, 87,  
    88, 90, 91, 93, 94, 96, 101, 111, 120, 130, 140, 150, 160,  
    170, 180, 190, 200, 210, 220, 230, 240, 249, 259, 269, 279, 289,  
    299, 309, 319, 329, 339, 349, 359, 368, 378, 388, 398, 408, 418,  
    428, 438, 448, 458, 468, 478, 488, 497, 507, 517, 527, 537, 547,  
    557, 567, 577, 587, 597, 607, 616, 626, 636, 646, 656, 666, 676,  
    686, 696, 706, 716, 726, 736, 745, 755, 765, 775, 785, 795, 805,  
    815, 825, 835, 845, 855, 864, 874, 884, 894, 904, 914, 924, 934,  
    944, 954, 964, 974, 984, 993, 1003, 1013, 1023, 1033, 1043, 1053, 1063,  
    1073, 1083, 1093, 1103, 1112, 1122, 1132, 1142, 1152, 1162, 1172, 1182, 1192,  
    1202, 1212, 1222, 1232, 1241, 1251, 1261, 1271, 1281, 1291, 1301, 1311, 1321,  
    1331, 1341, 1351, 1360, 1370, 1380, 1390, 1400, 1410, 1420, 1430, 1440, 1450,  
}
```

```

    1460, 1470, 1480, 1489, 1499, 1509, 1519, 1529, 1539, 1549, 1559, 1569, 1579,
    1589, 1599, 1608, 1618, 1628, 1638, 1648, 1658, 1668, 1678
}

```

```

Q_First[DBL_REG_DECIS_LEN] = { 45, 43, 40, 35, 32, 32, 32, 32, 32 }

```

```

W_Mult[MAX_DBL_FLT_LEN] = { 85, 51, 37, 28, 23, 20, 17, 15 }

```

```

Q_Thresh_Mults[MAX_DBL_FLT_LEN] =
{
    32, 25, 19, 19, 18, 18, 17, 17
}

```

NOTE: Deblocking widths of 5 and 7 are not reachable, so entries 4 and 6 of `W_Mult` and `Q_Thresh_Mults` are not reachable.

For a transform size `t` (of width `w` and height `h`) (with the same interpretation as for the `TxSize` variable), `Tx_Size_Sqr[ t ]` returns a square tx size with side length `Min( w, h )`.

```

Tx_Size_Sqr[ TX_SIZES_ALL ] =
{
    TX_4X4,
    TX_8X8,
    TX_16X16,
    TX_32X32,
    TX_64X64,
    TX_4X4,
    TX_4X4,
    TX_8X8,
    TX_8X8,
    TX_16X16,
    TX_16X16,
    TX_32X32,
    TX_32X32,
    TX_4X4,
    TX_4X4,
    TX_8X8,
    TX_8X8,
    TX_16X16,
    TX_16X16,
    TX_4X4,
    TX_4X4,
    TX_8X8,
    TX_8X8,
    TX_4X4,
    TX_4X4,
}

```

For a transform size `t` (of width `w` and height `h`), `Tx_Size_Sqr_Up[ t ]` returns a square tx size with side length `Max( w, h )`.

```

Tx_Size_Sqr_Up[ TX_SIZES_ALL ] =
{
    TX_4X4,
    TX_8X8,
    TX_16X16,
    TX_32X32,
    TX_64X64,
    TX_8X8,
    TX_8X8,
    TX_16X16,
    TX_16X16,
}

```

```

TX_32X32,
TX_32X32,
TX_64X64,
TX_64X64,
TX_16X16,
TX_16X16,
TX_32X32,
TX_32X32,
TX_64X64,
TX_64X64,
TX_32X32,
TX_32X32,
TX_64X64,
TX_64X64,
TX_64X64,
TX_64X64,
}

```

For a transform size t (of width w and height h), Tx_Width[t] returns w.

```

Tx_Width[ TX_SIZES_ALL ] =
{
    4,  8, 16, 32, 64, 4,  8, 8,  16, 16, 32, 32, 64, 4, 16, 8, 32, 16, 64,
    4, 32, 8, 64, 4, 64,
}

```

For a transform size t (of width w and height h), Tx_Height[t] returns h.

```

Tx_Height[ TX_SIZES_ALL ] =
{
    4,  8, 16, 32, 64, 8,  4, 16, 8,  32, 16, 64, 32, 16, 4, 32, 8, 64, 16,
    32, 4, 64, 8, 64, 4,
}

```

For a transform size t (of width w and height h), Tx_Width_Log2[t] returns the base 2 logarithm of w.

```

Tx_Width_Log2[ TX_SIZES_ALL ] =
{
    2, 3, 4, 5, 6, 2, 3, 3, 4, 4, 5, 5, 6, 2, 4, 3, 5, 4, 6,
    2, 5, 3, 6, 2, 6,
}

```

For a transform size t (of width w and height h), Tx_Height_Log2[t] returns the base 2 logarithm of h.

```

Tx_Height_Log2[ TX_SIZES_ALL ] =
{
    2, 3, 4, 5, 6, 3, 2, 4, 3, 5, 4, 6, 5, 4, 2, 5, 3, 6, 4,
    5, 2, 6, 3, 6, 2,
}

```

```

Wedge_Bits[ BLOCK_SIZES ] =
{
    0, 0, 0, 4, 4, 4, 4, 4, 4, 4, 4, 4,
    4, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 4, 4, 4, 4,
    0, 0, 4, 4
}

```

```

Sig_Ref_Diff_Offset[ 3 ][ SIG_REF_DIFF_OFFSET_NUM ][ 2 ] =
{
    {
        { 0, 1 }, { 1, 0 }, { 1, 1 }, { 0, 2 }, { 2, 0 }
    }
}

```

```

},
{
    { 0, 1 }, { 1, 0 }, { 0, 2 }, { 0, 3 }, { 0, 4 }
},
{
    { 0, 1 }, { 1, 0 }, { 2, 0 }, { 3, 0 }, { 4, 0 }
}
}

```

```

Adjusted_Tx_Size[ TX_SIZES_ALL ] =
{
    TX_4X4,
    TX_8X8,
    TX_16X16,
    TX_32X32,
    TX_32X32,
    TX_4X8,
    TX_8X4,
    TX_8X16,
    TX_16X8,
    TX_16X32,
    TX_32X16,
    TX_32X32,
    TX_32X32,
    TX_4X16,
    TX_16X4,
    TX_8X32,
    TX_32X8,
    TX_16X32,
    TX_32X16,
    TX_4X32,
    TX_32X4,
    TX_8X32,
    TX_32X8,
    TX_4X32,
    TX_32X4
}

```

```

Size_To_Tx_Part_Group_Lookup[BLOCK_SIZES] =
{
    0, 0, 0, 1, 2, 2, 3, 4, 4, 5, 6, 6, 7, 7, 7, 7,
    7, 7, 7, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8
}

```

```

Size_To_Tx_Type_Group_Vert_And_Horz[BLOCK_SIZES] =
{
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    0,
    1,
    2,
    3,
    4,
    5,
    6,
    7,
    8,
    9,
    9,
    9,
    9,
    9,
    9,
    9,
    9,
    9,
    9,
    BLOCK_INVALID,
    BLOCK_INVALID,
    10,
}

```



```

11,
12,
13,
BLOCK_INVALID,
BLOCK_INVALID,
10,
11
}

```

```

Size_To_Tx_Type_Group_Vert_Or_Horz[BLOCK_SIZES] =
{
    0,
    0,
    0,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    1,
    2,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    BLOCK_INVALID,
    1,
    2,
    BLOCK_INVALID,
    BLOCK_INVALID
}

```

```

Size_Class[TX_SIZES_ALL] =
{
    0, 1, 2, 3, 3, 0, 0, 1, 1, 3, 3, 3, 3, 1, 1, 3, 3, 3, 3,
    3, 3, 3, 3, 3, 3
};

```

```

Md_Idx_To_Type[EXT_TX_SIZES][INTRA_MODES][INTRA_TX_TYPES] =
{
    {
        { 0, 3, 1, 2, 7, 8, 13 },
        { 0, 3, 1, 2, 7, 10, 12 },
        { 0, 3, 1, 2, 8, 11, 13 },
        { 0, 3, 1, 2, 6, 7, 8 },
        { 0, 3, 1, 2, 7, 8, 13 },
        { 0, 3, 1, 2, 7, 12, 14 },
        { 0, 3, 1, 2, 7, 8, 13 },
        { 0, 3, 1, 2, 8, 11, 13 },
        { 0, 3, 1, 2, 7, 10, 12 },
        { 0, 3, 1, 2, 6, 7, 8 },
        { 0, 3, 1, 2, 7, 8, 12 },
        { 0, 3, 1, 2, 7, 8, 13 },
        { 0, 3, 2, 10, 11, 12, 13 }
    },
    {
        { 0, 3, 1, 2, 4, 7, 8 },
        { 0, 3, 1, 2, 7, 6, 8 },
    }
}

```

```

    { 0, 3, 1, 2, 8, 4, 7 },
    { 0, 3, 1, 2, 6, 7, 8 },
    { 0, 3, 1, 2, 5, 7, 8 },
    { 0, 3, 1, 2, 5, 7, 8 },
    { 0, 3, 1, 2, 4, 7, 8 },
    { 0, 3, 1, 2, 4, 8, 7 },
    { 0, 3, 1, 2, 5, 6, 7 },
    { 0, 3, 1, 2, 6, 7, 8 },
    { 0, 3, 1, 2, 4, 7, 8 },
    { 0, 3, 1, 2, 5, 7, 8 },
    { 0, 3, 1, 2, 10, 11, 13 }
  },
  {
    { 0, 3, 1, 2, 4, 7, 8 },
    { 0, 3, 1, 2, 5, 7, 8 },
    { 0, 3, 1, 2, 4, 8, 7 },
    { 0, 3, 1, 2, 4, 7, 8 },
    { 0, 3, 1, 2, 4, 7, 8 },
    { 0, 3, 1, 2, 5, 7, 8 },
    { 0, 3, 1, 2, 4, 7, 8 },
    { 0, 3, 1, 2, 4, 6, 8 },
    { 0, 3, 1, 2, 5, 7, 8 },
    { 0, 3, 1, 2, 4, 7, 8 },
    { 0, 3, 1, 2, 4, 7, 8 },
    { 0, 3, 1, 2, 5, 7, 8 },
    { 0, 3, 1, 2, 10, 11, 12 }
  },
  {
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 },
    { 0, 3, 1, 2, 4, 5, 6 }
  }
}

```

The array `Gaussian_Sequence` contains random samples from a Gaussian distribution with zero mean and standard deviation of about 512 clipped to the range of `[-2048, 2047]` and rounded to the nearest multiple of 4.

```

Gaussian_Sequence[ 2048 ] =
{
  56,    568,   -180,   172,   124,   -84,   172,   -64,   -900,   24,   820,
  224,  1248,   996,   272,    -8,   -916,  -388,  -732,  -104,  -188,  800,
  112,  -652,  -320,  -376,   140,  -252,   492,  -168,   44,  -788,  588,
 -584,   500,  -228,    12,   680,   272,  -476,   972,  -100,   652,   368,
  432,  -196,  -720,  -192,  1000,  -332,   652,  -136,  -552,  -604,   -4,
  192,  -220,  -136,  1000,   -52,   372,   -96,  -624,   124,   -24,   396,
  540,   -12,  -104,   640,   464,   244,  -208,   -84,   368,  -528,  -740,
  248,  -968,  -848,   608,   376,   -60,  -292,   -40,  -156,   252,  -292,
  248,   224,  -280,   400,  -244,   244,   -60,    76,   -80,   212,   532,
  340,   128,   -36,   824,  -352,   -60,  -264,   -96,  -612,   416,  -704,
  220,  -204,   640,  -160,  1220,  -408,   900,   336,    20,  -336,   -96,
 -792,   304,    48,    -28,  -1232,  -1172,  -448,   104,  -292,  -520,   244,
   60,  -948,    0,   -708,   268,   108,   356,  -548,   488,  -344,  -136,
  488,  -196,  -224,   656,  -236,  -1128,    60,    4,   140,   276,  -676,
 -376,   168,  -108,   464,    8,   564,    64,   240,   308,  -300,  -400,
 -456,  -136,    56,   120,  -408,  -116,   436,   504,  -232,   328,   844,
 -164,   -84,   784,  -168,   -224,   348,  -376,   128,   568,    96,
-1244,  -288,   276,   848,   832,  -360,   656,   464,  -384,  -332,  -356,

```

728,	-388,	160,	-192,	468,	296,	224,	140,	-776,	-100,	280,
4,	196,	44,	-36,	-648,	932,	16,	1428,	28,	528,	808,
772,	20,	268,	88,	-332,	-284,	124,	-384,	-448,	208,	-228,
-1044,	-328,	660,	380,	-148,	-300,	588,	240,	540,	28,	136,
-88,	-436,	256,	296,	-1000,	1400,	0,	-48,	1056,	-136,	264,
-528,	-1108,	632,	-484,	-592,	-344,	796,	124,	-668,	-768,	388,
1296,	-232,	-188,	-200,	-288,	-4,	308,	100,	-168,	256,	-500,
204,	-508,	648,	-136,	372,	-272,	-120,	-1004,	-552,	-548,	-384,
548,	-296,	428,	-108,	-8,	-912,	-324,	-224,	-88,	-112,	-220,
-100,	996,	-796,	548,	360,	-216,	180,	428,	-200,	-212,	148,
96,	148,	284,	216,	-412,	-320,	120,	-300,	-384,	-604,	-572,
-332,	-8,	-180,	-176,	696,	116,	-88,	628,	76,	44,	-516,
240,	-208,	-40,	100,	-592,	344,	-308,	-452,	-228,	20,	916,
-1752,	-136,	-340,	-804,	140,	40,	512,	340,	248,	184,	-492,
896,	-156,	932,	-628,	328,	-688,	-448,	-616,	-752,	-100,	560,
-1020,	180,	-800,	-64,	76,	576,	1068,	396,	660,	552,	-108,
-28,	320,	-628,	312,	-92,	-92,	-472,	268,	16,	560,	516,
-672,	-52,	492,	-100,	260,	384,	284,	292,	304,	-148,	88,
-152,	1012,	1064,	-228,	164,	-376,	-684,	592,	-392,	156,	196,
-524,	-64,	-884,	160,	-176,	636,	648,	404,	-396,	-436,	864,
424,	-728,	988,	-604,	904,	-592,	296,	-224,	536,	-176,	-920,
436,	-48,	1176,	-884,	416,	-776,	-824,	-884,	524,	-548,	-564,
-68,	-164,	-96,	692,	364,	-692,	-1012,	-68,	260,	-480,	876,
-1116,	452,	-332,	-352,	892,	-1088,	1220,	-676,	12,	-292,	244,
496,	372,	-32,	280,	200,	112,	-440,	-96,	24,	-644,	-184,
56,	-432,	224,	-980,	272,	-260,	144,	-436,	420,	356,	364,
-528,	76,	172,	-744,	-368,	404,	-752,	-416,	684,	-688,	72,
540,	416,	92,	444,	480,	-72,	-1416,	164,	-1172,	-68,	24,
424,	264,	1040,	128,	-912,	-524,	-356,	64,	876,	-12,	4,
-88,	532,	272,	-524,	320,	276,	-508,	940,	24,	-400,	-120,
756,	60,	236,	-412,	100,	376,	-484,	400,	-100,	-740,	-108,
-260,	328,	-268,	224,	-200,	-416,	184,	-604,	-564,	-20,	296,
60,	892,	-888,	60,	164,	68,	-760,	216,	-296,	904,	-336,
-28,	404,	-356,	-568,	-208,	-1480,	-512,	296,	328,	-360,	-164,
-1560,	-776,	1156,	-428,	164,	-504,	-112,	120,	-216,	-148,	-264,
308,	32,	64,	-72,	72,	116,	176,	-64,	-272,	460,	-536,
-784,	-280,	348,	108,	-752,	-132,	524,	-540,	-776,	116,	-296,
-1196,	-288,	-560,	1040,	-472,	116,	-848,	-1116,	116,	636,	696,
284,	-176,	1016,	204,	-864,	-648,	-248,	356,	972,	-584,	-204,
264,	880,	528,	-24,	-184,	116,	448,	-144,	828,	524,	212,
-212,	52,	12,	200,	268,	-488,	-404,	-880,	824,	-672,	-40,
908,	-248,	500,	716,	-576,	492,	-576,	16,	720,	-108,	384,
124,	344,	280,	576,	-500,	252,	104,	-308,	196,	-188,	-8,
1268,	296,	1032,	-1196,	436,	316,	372,	-432,	-200,	-660,	704,
-224,	596,	-132,	268,	32,	-452,	884,	104,	-1008,	424,	-1348,
-280,	4,	-1168,	368,	476,	696,	300,	-8,	24,	180,	-592,
-196,	388,	304,	500,	724,	-160,	244,	-84,	272,	-256,	-420,
320,	208,	-144,	-156,	156,	364,	452,	28,	540,	316,	220,
-644,	-248,	464,	72,	360,	32,	-388,	496,	-680,	-48,	208,
-116,	-408,	60,	-604,	-392,	548,	-840,	784,	-460,	656,	-544,
-388,	-264,	908,	-800,	-628,	-612,	-568,	572,	-220,	164,	288,
-16,	-308,	308,	-112,	-636,	-760,	280,	-668,	432,	364,	240,
-196,	604,	340,	384,	196,	592,	-44,	-500,	432,	-580,	-132,
636,	-76,	392,	4,	-412,	540,	508,	328,	-356,	-36,	16,
-220,	-64,	-248,	-60,	24,	-192,	368,	1040,	92,	-24,	-1044,
-32,	40,	104,	148,	192,	-136,	-520,	56,	-816,	-224,	732,
392,	356,	212,	-80,	-424,	-1008,	-324,	588,	-1496,	576,	460,
-816,	-848,	56,	-580,	-92,	-1372,	-112,	-496,	200,	364,	52,
-140,	48,	-48,	-60,	84,	72,	40,	132,	-356,	-268,	-104,
-284,	-404,	732,	-520,	164,	-304,	-540,	120,	328,	-76,	-460,
756,	388,	588,	236,	-436,	-72,	-176,	-404,	-316,	-148,	716,
-604,	404,	-72,	-88,	-888,	-68,	944,	88,	-220,	-344,	960,
472,	460,	-232,	704,	120,	832,	-228,	692,	-508,	132,	-476,
844,	-748,	-364,	-44,	1116,	-1104,	-1056,	76,	428,	552,	-692,
60,	356,	96,	-384,	-188,	-612,	-576,	736,	508,	892,	352,
-1132,	504,	-24,	-352,	324,	332,	-600,	-312,	292,	508,	-144,
-8,	484,	48,	284,	-260,	-240,	256,	-100,	-292,	-204,	-44,
472,	-204,	908,	-188,	-1000,	-256,	92,	1164,	-392,	564,	356,
652,	-28,	-884,	256,	484,	-192,	760,	-176,	376,	-524,	-452,
-436,	860,	-736,	212,	124,	504,	-476,	468,	76,	-472,	552,
-692,	-944,	-620,	740,	-240,	400,	132,	20,	192,	-196,	264,

-668,	-1012,	-60,	296,	-316,	-828,	76,	-156,	284,	-768,	-448,
-832,	148,	248,	652,	616,	1236,	288,	-328,	-400,	-124,	588,
220,	520,	-696,	1032,	768,	-740,	-92,	-272,	296,	448,	-464,
412,	-200,	392,	440,	-200,	264,	-152,	-260,	320,	1032,	216,
320,	-8,	-64,	156,	-1016,	1084,	1172,	536,	484,	-432,	132,
372,	-52,	-256,	84,	116,	-352,	48,	116,	304,	-384,	412,
924,	-300,	528,	628,	180,	648,	44,	-980,	-220,	1320,	48,
332,	748,	524,	-268,	-720,	540,	-276,	564,	-344,	-208,	-196,
436,	896,	88,	-392,	132,	80,	-964,	-288,	568,	56,	-48,
-456,	888,	8,	552,	-156,	-292,	948,	288,	128,	-716,	-292,
1192,	-152,	876,	352,	-600,	-260,	-812,	-468,	-28,	-120,	-32,
-44,	1284,	496,	192,	464,	312,	-76,	-516,	-380,	-456,	-1012,
-48,	308,	-156,	36,	492,	-156,	-808,	188,	1652,	68,	-120,
-116,	316,	160,	-140,	352,	808,	-416,	592,	316,	-480,	56,
528,	-204,	-568,	372,	-232,	752,	-344,	744,	-4,	324,	-416,
-600,	768,	268,	-248,	-88,	-132,	-420,	-432,	80,	-288,	404,
-316,	-1216,	-588,	520,	-108,	92,	-320,	368,	-480,	-216,	-92,
1688,	-300,	180,	1020,	-176,	820,	-68,	-228,	-260,	436,	-904,
20,	40,	-508,	440,	-736,	312,	332,	204,	760,	-372,	728,
96,	-20,	-632,	-520,	-560,	336,	1076,	-64,	-532,	776,	584,
192,	396,	-728,	-520,	276,	-188,	80,	-52,	-612,	-252,	-48,
648,	212,	-688,	228,	-52,	-260,	428,	-412,	-272,	-404,	180,
816,	-796,	48,	152,	484,	-88,	-216,	988,	696,	188,	-528,
648,	-116,	-180,	316,	476,	12,	-564,	96,	476,	-252,	-364,
-376,	-392,	556,	-256,	-576,	260,	-352,	120,	-16,	-136,	-260,
-492,	72,	556,	660,	580,	616,	772,	436,	424,	-32,	-324,
-1268,	416,	-324,	-80,	920,	160,	228,	724,	32,	-516,	64,
384,	68,	-128,	136,	240,	248,	-204,	-68,	252,	-932,	-120,
-480,	-628,	-84,	192,	852,	-404,	-288,	-132,	204,	100,	168,
-68,	-196,	-868,	460,	1080,	380,	-80,	244,	0,	484,	-888,
64,	184,	352,	600,	460,	164,	604,	-196,	320,	-64,	588,
-184,	228,	12,	372,	48,	-848,	-344,	224,	208,	-200,	484,
128,	-20,	272,	-468,	-840,	384,	256,	-720,	-520,	-464,	-580,
112,	-120,	644,	-356,	-208,	-608,	-528,	704,	560,	-424,	392,
828,	40,	84,	200,	-152,	0,	-144,	584,	280,	-120,	80,
-556,	-972,	-196,	-472,	724,	80,	168,	-32,	88,	160,	-688,
0,	160,	356,	372,	-776,	740,	-128,	676,	-248,	-480,	4,
-364,	96,	544,	232,	-1032,	956,	236,	356,	20,	-40,	300,
24,	-676,	-596,	132,	1120,	-104,	532,	-1096,	568,	648,	444,
508,	380,	188,	-376,	-604,	1488,	424,	24,	756,	-220,	-192,
716,	120,	920,	688,	168,	44,	-460,	568,	284,	1144,	1160,
600,	424,	888,	656,	-356,	-320,	220,	316,	-176,	-724,	-188,
-816,	-628,	-348,	-228,	-380,	1012,	-452,	-660,	736,	928,	404,
-696,	-72,	-268,	-892,	128,	184,	-344,	-780,	360,	336,	400,
344,	428,	548,	-112,	136,	-228,	-216,	-820,	-516,	340,	92,
-136,	116,	-300,	376,	-244,	100,	-316,	-520,	-284,	-12,	824,
164,	-548,	-180,	-128,	116,	-924,	-828,	268,	-368,	-580,	620,
192,	160,	0,	-1676,	1068,	424,	-56,	-360,	468,	-156,	720,
288,	-528,	556,	-364,	548,	-148,	504,	316,	152,	-648,	-620,
-684,	-24,	-376,	-384,	-108,	-920,	-1032,	768,	180,	-264,	-508,
-1268,	-260,	-60,	300,	-240,	988,	724,	-376,	-576,	-212,	-736,
556,	192,	1092,	-620,	-880,	376,	-56,	-4,	-216,	-32,	836,
268,	396,	1332,	864,	-600,	100,	56,	-412,	-92,	356,	180,
884,	-468,	-436,	292,	-388,	-804,	-704,	-840,	368,	-348,	140,
-724,	1536,	940,	372,	112,	-372,	436,	-480,	1136,	296,	-32,
-228,	132,	-48,	-220,	868,	-1016,	-60,	-1044,	-464,	328,	916,
244,	12,	-736,	-296,	360,	468,	-376,	-108,	-92,	788,	368,
-56,	544,	400,	-672,	-420,	728,	16,	320,	44,	-284,	-380,
-796,	488,	132,	204,	-596,	-372,	88,	-152,	-908,	-636,	-572,
-624,	-116,	-692,	-200,	-56,	276,	-88,	484,	-324,	948,	864,
1000,	-456,	-184,	-276,	292,	-296,	156,	676,	320,	160,	908,
-84,	-1236,	-288,	-116,	260,	-372,	-644,	732,	-756,	-96,	84,
344,	-520,	348,	-688,	240,	-84,	216,	-1044,	-136,	-676,	-396,
-1500,	960,	-40,	176,	168,	1516,	420,	-504,	-344,	-364,	-360,
1216,	-940,	-380,	-212,	252,	-660,	-708,	484,	-444,	-152,	928,
-120,	1112,	476,	-260,	560,	-148,	-344,	108,	-196,	228,	-288,
504,	560,	-328,	-88,	288,	-1008,	460,	-228,	468,	-836,	-196,
76,	388,	232,	412,	-1168,	-716,	-644,	756,	-172,	-356,	-504,
116,	432,	528,	48,	476,	-168,	-608,	448,	160,	-532,	-272,
28,	-676,	-12,	828,	980,	456,	520,	104,	-104,	256,	-344,
-4,	-28,	-368,	-52,	-524,	-572,	-556,	-200,	768,	1124,	-208,

```

-512, 176, 232, 248, -148, -888, 604, -600, -304, 804, -156,
-212, 488, -192, -804, -256, 368, -360, -916, -328, 228, -240,
-448, -472, 856, -556, -364, 572, -12, -156, -368, -340, 432,
252, -752, -152, 288, 268, -580, -848, -592, 108, -76, 244,
312, -716, 592, -80, 436, 360, 4, -248, 160, 516, 584,
732, 44, -468, -280, -292, -156, -588, 28, 308, 912, 24,
124, 156, 180, -252, 944, -924, -772, -520, -428, -624, 300,
-212, -1144, 32, -724, 800, -1128, -212, -1288, -848, 180, -416,
440, 192, -576, -792, -76, -1080, 80, -532, -352, -132, 380,
-820, 148, 1112, 128, 164, 456, 700, -924, 144, -668, -384,
648, -832, 508, 552, -52, -100, -656, 208, -568, 748, -88,
680, 232, 300, 192, -408, -1012, -152, -252, -268, 272, -876,
-664, -648, -332, -136, 16, 12, 1152, -28, 332, -536, 320,
-672, -460, -316, 532, -260, 228, -40, 1052, -816, 180, 88,
-496, -556, -672, -368, 428, 92, 356, 404, -408, 252, 196,
-176, -556, 792, 268, 32, 372, 40, 96, -332, 328, 120,
372, -900, -40, 472, -264, -592, 952, 128, 656, 112, 664,
-232, 420, 4, -344, -464, 556, 244, -416, -32, 252, 0,
-412, 188, -696, 508, -476, 324, -1096, 656, -312, 560, 264,
-136, 304, 160, -64, -580, 248, 336, -720, 560, -348, -288,
-276, -196, -500, 852, -544, -236, -1128, -992, -776, 116, 56,
52, 860, 884, 212, -12, 168, 1020, 512, -552, 924, -148,
716, 188, 164, -340, -520, -184, 880, -152, -680, -208, -1156,
-300, -528, -472, 364, 100, -744, -1056, -32, 540, 280, 144,
-676, -32, -232, -280, -224, 96, 568, -76, 172, 148, 148,
104, 32, -296, -32, 788, -80, 32, -16, 280, 288, 944,
428, -484
}

```

```

Para_Adjustment_List[NUM_PARA_COMBINATIONS][NUM_PARA_INTERVALS] =
{
    { 0, 0, 0 }, { 0, 0, -1 }, { 0, 0, -2 }, { 0, 0, 1 },
    { 0, 0, 1 }, { 0, -1, 0 }, { 0, -1, -1 }, { 0, -1, -2 },
    { 0, -1, 1 }, { 0, -1, 1 }, { 0, -2, 0 }, { 0, -2, -1 },
    { 0, -2, -2 }, { 0, -2, 1 }, { 0, -2, 1 }, { 0, 1, 0 },
    { 0, 1, -1 }, { 0, 1, -2 }, { 0, 1, 1 }, { 0, 1, 1 },
    { 0, 1, 0 }, { 0, 1, -1 }, { 0, 1, -2 }, { 0, 1, 1 },
    { 0, 1, 1 }, { -1, 0, 0 }, { -1, 0, -1 }, { -1, 0, -2 },
    { -1, 0, 1 }, { -1, 0, 1 }, { -1, -1, 0 }, { -1, -1, -1 },
    { -1, -1, -2 }, { -1, -1, 1 }, { -1, -1, 1 }, { -1, -2, 0 },
    { -1, -2, -1 }, { -1, -2, -2 }, { -1, -2, 1 }, { -1, -2, 1 },
    { -1, 1, 0 }, { -1, 1, -1 }, { -1, 1, -2 }, { -1, 1, 1 },
    { -1, 1, 1 }, { -1, 1, 0 }, { -1, 1, -1 }, { -1, 1, -2 },
    { -1, 1, 1 }, { -1, 1, 1 }, { -2, 0, 0 }, { -2, 0, -1 },
    { -2, 0, -2 }, { -2, 0, 1 }, { -2, 0, 1 }, { -2, -1, 0 },
    { -2, -1, -1 }, { -2, -1, -2 }, { -2, -1, 1 }, { -2, -1, 1 },
    { -2, -2, 0 }, { -2, -2, -1 }, { -2, -2, -2 }, { -2, -2, 1 },
    { -2, -2, 1 }, { -2, 1, 0 }, { -2, 1, -1 }, { -2, 1, -2 },
    { -2, 1, 1 }, { -2, 1, 1 }, { -2, 1, 0 }, { -2, 1, -1 },
    { -2, 1, -2 }, { -2, 1, 1 }, { -2, 1, 1 }, { 1, 0, 0 },
    { 1, 0, -1 }, { 1, 0, -2 }, { 1, 0, 1 }, { 1, 0, 1 },
    { 1, -1, 0 }, { 1, -1, -1 }, { 1, -1, -2 }, { 1, -1, 1 },
    { 1, -1, 1 }, { 1, -2, 0 }, { 1, -2, -1 }, { 1, -2, -2 },
    { 1, -2, 1 }, { 1, -2, 1 }, { 1, 1, 0 }, { 1, 1, -1 },
    { 1, 1, -2 }, { 1, 1, 1 }, { 1, 1, 1 }, { 1, 1, 0 },
    { 1, 1, -1 }, { 1, 1, -2 }, { 1, 1, 1 }, { 1, 1, 1 },
    { 1, 0, 0 }, { 1, 0, -1 }, { 1, 0, -2 }, { 1, 0, 1 },
    { 1, 0, 1 }, { 1, -1, 0 }, { 1, -1, -1 }, { 1, -1, -2 },
    { 1, -1, 1 }, { 1, -1, 1 }, { 1, -2, 0 }, { 1, -2, -1 },
    { 1, -2, -2 }, { 1, -2, 1 }, { 1, -2, 1 }, { 1, 1, 0 },
    { 1, 1, -1 }, { 1, 1, -2 }, { 1, 1, 1 }, { 1, 1, 1 },
    { 1, 1, 0 }, { 1, 1, -1 }, { 1, 1, -2 }, { 1, 1, 1 },
    { 1, 1, 1 },
}

```

NOTE: Not all the rows of Para_Adjustment_List are reachable.

```
Prob_Inc[7][8] =
{
  { 8,  0,  0,  0,  0,  0,  0,  0 },
  { 10, 5,  0,  0,  0,  0,  0,  0 },
  { 12, 8,  4,  0,  0,  0,  0,  0 },
  { 12, 9,  6,  3,  0,  0,  0,  0 },
  { 13, 10, 8,  5,  2,  0,  0,  0 },
  { 13, 11, 9,  6,  4,  2,  0,  0 },
  { 14, 12, 10, 8,  6,  4,  2,  0 },
}
```

```
Tile_Width_Scaling_Factor[2][31] = {
  {4, 4, 4, 4, 4, 4,
  4, 4, 4, 4, 4, 4, 4,
  4, 4, 4, 4, 8, 8, 8, 8,
  reserved, reserved, reserved,
  reserved, reserved, reserved,
  reserved, reserved, reserved},
  {4, 4, 4, 4, 4, 4,
  4, 4, 4, 4, 4, 4, 4,
  8, 8, 8, 8, 16, 16, 16, 16,
  reserved, reserved, reserved,
  reserved, reserved, reserved,
  reserved, reserved, reserved}
}
```

```
Tile_Area_Scaling_Factor[2][31] = {
  {4, 4, 4, 4, 4, 4,
  4, 4, 4, 4, 4, 4, 4,
  8, 8, 8, 8, 16, 16, 16, 16,
  reserved, reserved, reserved,
  reserved, reserved, reserved,
  reserved, reserved, reserved},
  {4, 4, 4, 4, 4, 4,
  4, 4, 4, 4, 4, 4, 4,
  16, 16, 16, 16, 32, 32, 32, 32,
  reserved, reserved, reserved,
  reserved, reserved, reserved,
  reserved, reserved, reserved}
}
```

§ 9.3. Default CDF tables

This section contains the default values for the cumulative distributions.

```
Default_Mv_Joint_Adaptive_Cdf[ MV_JOINTS + 1 ] =  
{ 4, 17705, 32748, 1, 0}
```

```
Default_Amvd_Indices_Cdf[ 2 ][ MAX_AMVD_INDEX + 1 ] =  
{  
  {10549, 15298, 16241, 22533, 27449, 30520, 32080, 26, 0},  
  { 9414, 14965, 15966, 22465, 27468, 30628, 32144, 26, 0}  
}
```

```
Default_Shell_Offset_Low_Class_Cdf[ 2 ][ 3 ] =  
{  
  {14587, 36, 0},  
  {20966, 75, 0}  
}
```

```
Default_Shell_Offset_Class2_Cdf[ 3 ] =  
{13189, 0, 0}
```

```
Default_Shell_Offset_Other_Class_Cdf[ 16 ][ 3 ] =  
{  
  {17943, 93, 0},  
  {18934, 93, 0},  
  {18928, 93, 0},  
  {18696, 93, 0},  
  {19044, 93, 0},  
  {20362, 93, 0},  
  {20426, 93, 0},  
  {22563, 93, 0},  
  {22190, 93, 0},  
  {23458, 90, 0},  
  {26227, 2, 0},  
  {30765, 50, 0},  
  {16384, 0, 0},  
  {16384, 0, 0},  
  {16384, 0, 0},  
  {16384, 0, 0}  
}
```

```
Default_Col_Mv_Greater_Cdf[ NUM_CTX_COL_MV_GTX ][ 3 ] =  
{  
  { 5663, 25, 0},  
  { 4856, 90, 0}  
}
```

```
Default_Col_Mv_Index_Cdf[ NUM_CTX_COL_MV_INDEX ][ 3 ] =  
{  
  {13445, 1, 0},  
  {13541, 1, 0},  
}
```

```
{14045, 1, 0},  
{12888, 31, 0}  
}
```

```
Default_Joint_Shell_Set_Cdf[ 2 + 1 ] =  
{31579, 25, 0}
```

```
Default_Joint_Shell0_Class0_Cdf[ 5 + 1 ] =  
{ 4460, 12999, 22505, 30840, 38, 0}
```

```
Default_Joint_Shell1_Class0_Cdf[ 6 + 1 ] =  
{ 7519, 18907, 25563, 29875, 31983, 27, 0}
```

```
Default_Joint_Shell3_Class0_Cdf[ 7 + 1 ] =  
{ 8680, 13723, 18208, 22686, 26722, 30020, 5, 0}
```

```
Default_Joint_Shell4_Class0_Cdf[ 7 + 1 ] =  
{ 4324, 15300, 23690, 28697, 31282, 32359, 1, 0}
```

```
Default_Joint_Shell5_Class0_Cdf[ 8 + 1 ] =  
{ 7497, 17301, 23848, 27438, 29395, 30879, 32003, 31, 0}
```

```
Default_Joint_Shell6_Class0_Cdf[ 8 + 1 ] =  
{10667, 20239, 25883, 29670, 31400, 32153, 32579, 0, 0}
```

```
Default_Joint_Shell0_Class1_Cdf[ 6 + 1 ] =  
{21329, 30564, 32589, 32649, 32708, 50, 0}
```

```
Default_Joint_Shell1_Class1_Cdf[ 6 + 1 ] =  
{24250, 31806, 32676, 32722, 32732, 50, 0}
```

```
Default_Joint_Shell3_Class1_Cdf[ 7 + 1 ] =  
{19978, 30160, 32564, 32732, 32736, 32740, 1, 0}
```

```
Default_Joint_Shell4_Class1_Cdf[ 8 + 1 ] =  
{19707, 28414, 31240, 31648, 32692, 32717, 32721, 25, 0}
```

```
Default_Joint_Shell5_Class1_Cdf[ 8 + 1 ] =  
{18469, 27427, 31562, 32652, 32724, 32728, 32732, 31, 0}
```

```
Default_Joint_Shell6_Class1_Cdf[ 8 + 1 ] =  
{17810, 25196, 29372, 31953, 32564, 32720, 32724, 55, 0}
```

```
Default_Joint_Shell_Last_Two_Classes_Cdf[ 2 + 1 ] =  
{16384, 0, 0}
```

```
Default_Use_Bawp_Cdf[ 2 + 1 ] =  
{26456, 31, 0}
```

```
Default_Use_Bawp_Chroma_Cdf[ 2 + 1 ] =  
{ 5121, 31, 0}
```

```
Default_Do_Square_Split_Cdf  
[PARTITION_STRUCTURE_NUM][SQUARE_SPLIT_CONTEXTS][2 + 1] = {  
{  
{18000, 7, 0},
```


$\}$ $\{$

Defa

 $\{$ $\{2$

Defa
53

 $\{$

```

{16384, 0, 0},
{16384, 0, 0},
{16384, 0, 0}
}

```

```

Default_Cctx_Type_Cdf[ CCTX_TYPES + 1 ] =
{14350, 14836, 16054, 17075, 19408, 28530, 62, 0}

```

```

Default_Warp_Mv_Cdf[ 2 + 1 ] =
{15095, 1, 0}

```

```

Default_Is_Warp_Cdf[ WARPMV_MODE_CONTEXT ][ 2 + 1 ] =
{
{25999, 1, 0},
{14478, 7, 0},
{10868, 6, 0},
{ 5256, 31, 0},
{ 2722, 31, 0}
}

```

```

Default_Warp_With_Mvd_Cdf[ 2 + 1 ] =
{18452, 1, 0}

```

```

Default_Warp_Idx_Cdf[3][ 2 + 1 ] =
{
{18903, 0, 0},
{24500, 90, 0},
{25360, 75, 0}
}

```

```

Default_Jmvd_Scale_Mode_Cdf[ JOINT_NEWMV_SCALE_FACTOR_CNT + 1 ] =
{23180, 24894, 26548, 29872, 1, 0}

```

```

Default_Jmvd_Adaptive_Scale_Mode_Cdf[ JOINT_AMVD_SCALE_FACTOR_CNT + 1 ] =
{23178, 26812, 0, 0}

```

```

Default_Cwp_Idx_Cdf[ MAX_CWP_NUM - 1 ][2 + 1] =
{
{21704, 56, 0},
{15990, 31, 0},
{12544, 57, 0},
{25638, 62, 0}
}

```

```

Default_Fsc_Mode_Cdf[FSC_MODE_CONTEXTS][FSC_BSIZE_CONTEXTS][2 + 1] =
{
{
{30503, 3, 0},
{31244, 3, 0},
{32254, 78, 0},
{32324, 93, 0},
{32582, 93, 0},
{32691, 93, 0}
},
{
{27437, 0, 0},
{27242, 90, 0},
{28040, 76, 0},
{27589, 76, 0},
{27234, 7, 0},
{23583, 62, 0}
},
{

```

```

    {26068, 75, 0},
    {22635, 75, 0},
    {22069, 6, 0},
    {19218, 32, 0},
    {13701, 31, 0},
    { 4636, 38, 0}
},
{
    {16384, 0, 0},
    {16384, 0, 0},
    {32016, 15, 0},
    {32403, 93, 0},
    {32583, 15, 0},
    {32683, 76, 0}
}
}

```

```

Default_Y_Mode_Set_Cdf[INTRA_MODE_SETS + 1] =
{28863, 31022, 31724, 93, 0}

```

```

Default_Y_Mode_Index_Cdf[Y_MODE_CONTEXTS][MODE_INDEX_COUNT + 1] =
{
    {15175, 20075, 21728, 24098, 26405, 27655, 28860, 5, 0},
    {10114, 14957, 16815, 19127, 20147, 25583, 27169, 0, 0},
    { 5636, 9004, 10456, 12122, 12744, 20325, 25607, 0, 0}
}

```

```

Default_Y_Mode_Offset_Cdf[Y_MODE_CONTEXTS][MODE_OFFSET_COUNT + 1] =
{
    {12743, 18172, 20194, 23648, 26419, 6, 0},
    { 8976, 16084, 20827, 24595, 28496, 75, 0},
    { 8784, 14556, 19710, 24903, 28724, 75, 0}
}

```

```

Default_Uv_Mode_Cfl_Not_Allowed_Cdf
[ UV_MODE_CONTEXTS ][ CHROMA_MODE_COUNT + 1 ] = {
    { 9363, 20957, 22865, 24753, 26411, 27983, 30428, 31, 0},
    {21282, 23610, 28208, 29311, 30348, 31158, 31491, 30, 0}
}

```

```

Default_Is_Cfl_Cdf[CFL_CONTEXTS][ 2 + 1 ] =
{
    {20441, 30, 0},
    {11610, 6, 0},
    { 4643, 0, 0}
}

```

```

Default_Mrl_Index_Cdf[MRL_INDEX_CONTEXTS][4 + 1] =
{
    {29573, 31193, 32023, 78, 0},
    {21812, 27066, 30279, 75, 0},
    {16076, 23806, 28762, 1, 0}
}

```

```

Default_Ccso_Blk_Cdf[3][CCSO_CONTEXT][2 + 1] =
{
    {
        {18469, 62, 0},
        {16384, 0, 0},
        { 4949, 37, 0},
        {16384, 0, 0}
    },
    {
        {23470, 37, 0},
        {16384, 0, 0},
    }
}

```

```

    { 6666, 37, 0},
    {16384, 0, 0}
},
{
    {22914, 37, 0},
    {16384, 0, 0},
    { 6993, 37, 0},
    {16384, 0, 0}
}
}

```

```

Default_Intrabc_Mode_Cdf[ 2 + 1 ] =
    {29993, 6, 0}

```

```

Default_Intrabc_Precision_Cdf[ 2 + 1 ] =
    {19778, 31, 0}

```

```

Default_Intrabc_Cdf[INTRABC_CONTEXTS][ 2 + 1 ] =
{
    {32085, 5, 0},
    {15172, 30, 0},
    { 4503, 0, 0}
}

```

```

Default_Do_Split_Cdf[PARTITION_STRUCTURE_NUM][PARTITION_CONTEXTS][3] =
{
    {
        {28084, 3, 0},
        {23755, 93, 0},
        {23634, 93, 0},
        {19368, 3, 0},
        {24961, 0, 0},
        {14941, 1, 0},
        {16154, 1, 0},
        { 5905, 0, 0},
        {21934, 0, 0},
        {10440, 26, 0},
        {11984, 31, 0},
        { 3474, 0, 0},
        {20492, 16, 0},
        { 6963, 6, 0},
        { 8099, 26, 0},
        { 1529, 0, 0},
        {24117, 92, 0},
        { 7871, 10, 0},
        {23604, 2, 0},
        { 8429, 30, 0},
        {27356, 2, 0},
        {22441, 7, 0},
        { 8897, 31, 0},
        { 6811, 61, 0},
        {17592, 16, 0},
        { 5648, 32, 0},
        { 5339, 26, 0},
        { 1082, 26, 0},
        {26143, 77, 0},
        {11379, 85, 0},
        {20142, 93, 0},
        { 7401, 8, 0},
        {26235, 82, 0},
        {23674, 78, 0},
        {12441, 77, 0},
        {10482, 75, 0},
        {20663, 0, 0},
        { 4192, 27, 0},
        { 5274, 33, 0},
        { 713, 1, 0},
    }
}

```



```

    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {26227, 0, 0},
    {16384, 0, 0},
    {17669, 1, 0},
    {16384, 0, 0},
    {26179, 1, 0},
    {17889, 1, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {23500, 1, 0},
    {13115, 31, 0},
    {15253, 31, 0},
    { 6458, 55, 0},
    {22566, 31, 0},
    {11497, 32, 0},
    {10045, 31, 0},
    { 3750, 26, 0}
}
}

```

```
Default_Rect_Type_Cdf[PARTITION_STRUCTURE_NUM][PARTITION_CONTEXTS][3] =
```

```

{
{
    {14644, 0, 0},
    {10173, 75, 0},
    {18529, 0, 0},
    {16071, 90, 0},
    {20263, 1, 0},
    {12813, 1, 0},
    {26612, 0, 0},
    {23277, 1, 0},
    {10594, 76, 0},
    { 7000, 75, 0},
    {20002, 1, 0},
    {12889, 2, 0},
    {13854, 76, 0},
    {10750, 1, 0},
    {18380, 1, 0},
    {17505, 6, 0},
    {14430, 7, 0},
    {11554, 2, 0},
    {20078, 1, 0},
    {19097, 76, 0},
    {15278, 2, 0},
    {10137, 1, 0},
    {21921, 7, 0},
    {14621, 6, 0},
    {19330, 2, 0},
    {15921, 1, 0},
    {26218, 1, 0},
    {24318, 1, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16066, 78, 0},
    { 9225, 2, 0},
    {22849, 31, 0},
    {14817, 11, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
}
}

```



```

    {16384, 0, 0},
    {24384, 26, 0},
    {22113, 32, 0},
    {21798, 2, 0},
    {20067, 6, 0},
    {26220, 31, 0},
    {22997, 30, 0},
    {22249, 31, 0},
    {20091, 0, 0}
}
}

```

Default_Tx_Do_Partition_Cdf[FSC_MODES][2][TXFM_SPLIT_GROUP][2 + 1] =

```

{
    {
        {
            {26330, 30, 0},
            {29620, 0, 0},
            {20420, 30, 0},
            {21694, 31, 0},
            {13317, 31, 0},
            {15391, 31, 0},
            {15952, 31, 0},
            {14736, 31, 0},
            {13810, 5, 0}
        },
        {
            {31646, 55, 0},
            {32393, 25, 0},
            {30802, 26, 0},
            {30485, 31, 0},
            {20759, 32, 0},
            {22159, 1, 0},
            {26832, 31, 0},
            {27351, 1, 0},
            {24696, 1, 0}
        }
    },
    {
        {
            {29308, 0, 0},
            {32550, 93, 0},
            {27963, 1, 0},
            {27618, 31, 0},
            {22367, 7, 0},
            {23478, 35, 0},
            {16384, 0, 0},
            {16384, 0, 0},
            {27990, 0, 0}
        },
        {
            {16384, 0, 0},
            {16384, 0, 0},
            {16384, 0, 0},
            {16384, 0, 0},
            {16384, 0, 0},
            {16384, 0, 0},
            {16384, 0, 0},
            {16384, 0, 0},
            {16384, 0, 0}
        }
    }
}
}

```

Default_Tx_Partition_Type_Reduced_Cdf

[FSC_MODES][2][TX_PARTITION_TYPE_NUM_VERT_AND_HORZ]

[TX_PARTITION_TYPE_NUM + 1] = {

```

{
    {

```



```

    }
  }
}

```

Default Tx_2or3_Partition_Type_Cdf

```

[FSC_MODES][2][TX_PARTITION_TYPE_NUM_VERT_OR_HORZ - 1][2 + 1] = {
{
{
{25131, 30, 0},
{24514, 30, 0}
},
{
{19534, 6, 0},
{18637, 0, 0}
}
},
{
{
{30226, 50, 0},
{30703, 33, 0}
},
{
{16384, 0, 0},
{16384, 0, 0}
}
}
}
}

```

Default Tx_Partition_Type_Cdf

```

[FSC_MODES][2][TX_PARTITION_TYPE_NUM_VERT_AND_HORZ]
[TX_PARTITION_TYPE_NUM + 1] = {
{
{
{32744, 32748, 32752, 32756, 32760, 32764, 60, 0},
{ 3006, 22888, 27132, 29972, 29976, 32724, 31, 0},
{ 3673, 8849, 27652, 27656, 29944, 29948, 31, 0},
{ 5219, 19419, 28052, 28836, 29528, 31296, 31, 0},
{ 3055, 19649, 22157, 27038, 27466, 31646, 31, 0},
{ 6044, 11255, 26170, 26493, 28585, 29584, 31, 0},
{ 5896, 20361, 25685, 27552, 28695, 31097, 31, 0},
{ 2355, 17601, 21703, 26050, 27881, 31397, 32, 0},
{ 4701, 13502, 24958, 26413, 28166, 30129, 32, 0},
{ 3319, 16449, 21641, 27154, 29260, 31512, 32, 0},
{ 2526, 17088, 19643, 29378, 29382, 32724, 31, 0},
{ 2298, 4406, 23886, 23890, 30148, 30152, 31, 0},
{ 1553, 16160, 18679, 27983, 29592, 32139, 32, 0},
{ 2316, 4714, 22731, 23797, 29514, 30077, 37, 0}
},
{
{10790, 22602, 32736, 32740, 32744, 32748, 32, 0},
{14384, 19116, 26545, 28687, 28691, 32724, 31, 0},
{13910, 21690, 26343, 26347, 28432, 28436, 31, 0},
{15891, 21712, 28890, 29571, 30307, 31363, 32, 0},
{15946, 19447, 20270, 23423, 23858, 31148, 32, 0},
{16053, 16957, 20312, 20924, 24770, 25959, 32, 0},
{18479, 23053, 26582, 26966, 27567, 29836, 62, 0},
{ 9312, 11882, 14354, 17792, 19827, 29477, 62, 0},
{ 8490, 10668, 13295, 15353, 19952, 22575, 62, 0},
{ 6404, 12066, 16173, 20041, 24512, 28421, 62, 0},
{10019, 14455, 17658, 27012, 27016, 32724, 37, 0},
{ 9479, 14904, 19374, 19378, 28027, 28031, 37, 0},
{ 3717, 7198, 8103, 20546, 23558, 31447, 62, 0},
{ 4058, 5429, 8987, 13978, 25126, 26655, 62, 0}
}
},
{
{
{32434, 32490, 32545, 32601, 32657, 32712, 50, 0},
{ 1491, 14241, 29930, 32517, 32524, 32724, 57, 0},

```

```

    { 1719, 16525, 31000, 31004, 32575, 32579, 30, 0},
    { 1645, 16749, 29324, 30425, 32016, 32485, 62, 0},
    { 2908, 15802, 24689, 28470, 32122, 32542, 25, 0},
    { 3470, 17931, 25841, 29589, 31907, 32465, 43, 0},
    { 5638, 19594, 28693, 29977, 30703, 32154, 50, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 2734, 12129, 30342, 31805, 31844, 32724, 50, 0},
    { 3849, 21783, 31043, 31056, 32181, 32193, 65, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0}
},
{
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0},
    { 4681, 9362, 14043, 18725, 23406, 28087, 0, 0}
}
}
}

```

```

Default_Segment_Id_Cdf[ SEGMENT_ID_CONTEXTS ][ 8 + 1 ] =
{
    { 5622, 7893, 16093, 18233, 27809, 28373, 32533, 0, 0},
    {14274, 18230, 22557, 24935, 29980, 30851, 32344, 0, 0},
    {27527, 28487, 28723, 28890, 32397, 32647, 32679, 0, 0}
}

```

```

Default_Segment_Id_Ext_Cdf[ SEGMENT_ID_CONTEXTS ][ 8 + 1 ] =
{
    { 4096, 8192, 12288, 16384, 20480, 24576, 28672, 0, 0},
    { 4096, 8192, 12288, 16384, 20480, 24576, 28672, 0, 0},
    { 4096, 8192, 12288, 16384, 20480, 24576, 28672, 0, 0}
}

```

```

Default_Seg_Id_Ext_Flag_Cdf[ SEGMENT_ID_CONTEXTS ][ 2 + 1 ] =
{
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0}
}

```

```

Default_Segment_Id_Predicted_Cdf[ SEGMENT_ID_PREDICTED_CONTEXTS ][ 3 ] =
{
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0}
}

```

```

Default_Skip_Drl_Mode_Cdf[ 3 ][ 3 ] =
{
    {21634, 0, 0},

```

```

{17376, 0, 0},
{18432, 75, 0}
}

```

```

Default_Tip_Drl_Mode_Cdf[ 3 ][ 3 ] =
{
{30662, 0, 0},
{23823, 6, 0},
{21676, 6, 0}
}

```

```

Default_Single_Mode_Cdf[SINGLE_MODE_CONTEXTS][ 3 + 1 ] =
{
{10043, 11100, 6, 0},
{21561, 21758, 1, 0},
{25411, 25714, 0, 0},
{14117, 14341, 0, 0},
{18288, 18577, 0, 0}
}

```

```

Default_Drl_Mode_Cdf[ 3 ][ DRL_MODE_CONTEXTS ][ 3 ] =
{
{
{15721, 90, 0},
{21115, 0, 0},
{19567, 1, 0},
{17602, 93, 0},
{13319, 93, 0}
},
{
{18692, 93, 0},
{19343, 90, 0},
{18207, 90, 0},
{17908, 93, 0},
{18304, 93, 0}
},
{
{22157, 90, 0},
{23233, 90, 0},
{22782, 90, 0},
{22353, 93, 0},
{22457, 93, 0}
}
}
}

```

```

Default_Comp_Mode_Cdf[ COMP_INTER_CONTEXTS ][ 3 ] =
{
{26924, 15, 0},
{25000, 1, 0},
{17949, 1, 0},
{13581, 6, 0},
{ 7034, 0, 0}
}

```

```

Default_Is_Inter_Cdf[ IS_INTER_CONTEXTS ][ 3 ] =
{
{ 1522, 1, 0},
{14381, 0, 0},
{10455, 25, 0},
{27796, 0, 0}
}

```

```

Default_Skip_Mode_Cdf[ SKIP_MODE_CONTEXTS ][ 3 ] =
{
{30964, 93, 0},

```

```

{21769, 0, 0},
{12484, 15, 0}
}

```

```

Default_Skip_Cdf[ SKIP_CONTEXTS ][ 3 ] =
{
{25865, 25, 0},
{14316, 0, 0},
{ 4598, 0, 0},
{25612, 6, 0},
{12366, 1, 0},
{ 3320, 90, 0}
}

```

```

Default_Single_Ref_Cdf[REF_CONTEXTS][REFS_PER_FRAME - 1][ 3 ] =
{
{
{26469, 0, 0},
{28870, 30, 0},
{29662, 1, 0},
{29867, 6, 0},
{29772, 6, 0},
{29776, 26, 0}
},
{
{13631, 6, 0},
{18185, 37, 0},
{19992, 32, 0},
{18462, 62, 0},
{17451, 37, 0},
{11578, 62, 0}
},
{
{ 2599, 0, 0},
{ 5203, 31, 0},
{ 5185, 31, 0},
{ 3671, 31, 0},
{ 3954, 6, 0},
{ 1633, 5, 0}
}
}
}

```

```

Default_Comp_Ref0_Cdf[REF_CONTEXTS][REFS_PER_FRAME - 1][ 3 ] =
{
{
{ 9272, 32, 0},
{17175, 62, 0},
{16384, 0, 0},
{16384, 0, 0},
{16384, 0, 0},
{16384, 0, 0}
},
{
{ 1385, 0, 0},
{ 4439, 31, 0},
{16384, 0, 0},
{16384, 0, 0},
{16384, 0, 0},
{16384, 0, 0}
},
{
{ 521, 0, 0},
{ 1854, 25, 0},
{16384, 0, 0},
{16384, 0, 0},
{16384, 0, 0}
}
}
}

```

```

    {16384, 0, 0}
  }
}

```

Default_Comp_Ref1_Cdf[REF_CONTEXTS][2][REFS_PER_FRAME - 1][3] =

```

{
  {
    {
      {30729, 75, 0},
      {29403, 5, 0},
      {29037, 6, 0},
      {29355, 31, 0},
      {28573, 5, 0},
      {27396, 7, 0}
    },
    {
      {16384, 0, 0},
      {19315, 31, 0},
      {27821, 6, 0},
      {27892, 31, 0},
      {28695, 30, 0},
      {29637, 51, 0}
    }
  },
  {
    {
      {30432, 0, 0},
      {20290, 31, 0},
      {19855, 37, 0},
      {18567, 62, 0},
      {18331, 37, 0},
      {14241, 62, 0}
    },
    {
      {16384, 0, 0},
      { 5725, 31, 0},
      {13420, 31, 0},
      {12780, 32, 0},
      {10781, 62, 0},
      { 6424, 62, 0}
    }
  },
  {
    {
      {11634, 31, 0},
      {10093, 31, 0},
      { 6065, 31, 0},
      { 5408, 31, 0},
      { 6411, 31, 0},
      { 4075, 30, 0}
    },
    {
      {16384, 0, 0},
      { 898, 15, 0},
      { 3127, 1, 0},
      { 1775, 5, 0},
      { 1217, 6, 0},
      { 591, 5, 0}
    }
  }
}

```



```

    }
  }
}

```

```

Default_Is_Joint_Cdf[ 2 ][ 2 + 1 ] =
{
  {24720, 0, 0},
  {32764, 0, 0}
}

```

```

Default_Compound_Mode_Non_Joint_Cdf[ COMPOUND_MODE_CONTEXTS ][ 5 + 1 ] =
{
  {12177, 20001, 23193, 24448, 26, 0},
  {21192, 25117, 27806, 27948, 31, 0},
  {26779, 28724, 30192, 30249, 25, 0},
  {12506, 17871, 21295, 21389, 31, 0},
  {16948, 20335, 22582, 22617, 31, 0}
}

```

```

Default_Compound_Mode_Same_Refs_Cdf
[COMPOUND_MODE_CONTEXTS][COMPOUND_MODES - 2] = {
  { 6386, 23344, 23348, 31, 0},
  {10945, 24709, 24714, 31, 0},
  {11517, 25230, 25258, 31, 0},
  { 7563, 22176, 22180, 31, 0},
  { 6629, 20955, 20966, 6, 0}
}

```

```

Default_Use_Optflow_Cdf[2][3] =
{
  {11582, 31, 0},
  {24076, 26, 0}
}

```

```

Default_Interp_Filter_Cdf[ INTERP_FILTER_CONTEXTS ][ INTERP_FILTERS + 1 ] =
{
  {29975, 32748, 25, 0},
  { 2076, 32703, 75, 0},
  { 19, 1768, 15, 0},
  {17314, 27415, 31, 0},
  {31286, 31994, 0, 0},
  { 9581, 32608, 31, 0},
  { 535, 1036, 0, 0},
  {24819, 27722, 31, 0},
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0}
}

```

```

Default_Use_Most_Probable_Precision_Cdf[3][2 + 1] =
{
  {20650, 31, 0},
  {15758, 31, 0},
  { 4571, 31, 0}
}

```

```

Default_Pb_Mv_Precision_Cdf[2][3][3 + 1] =
{
  {
    {10923, 21845, 0, 0},

```

```

    {31340, 32505, 78, 0},
    {26039, 32175, 0, 0}
},
{
    {10923, 21845, 0, 0},
    {32197, 32676, 75, 0},
    {28679, 32626, 0, 0}
}
}

```

```

Default_Identity_Row_Y_Cdf[ PALETTE_ROW_FLAG_CONTEXTS ][ 4 ] =
{
    {22515, 25751, 25, 0},
    { 4014, 5233, 31, 0},
    { 3548, 4163, 33, 0},
    {12999, 32756, 56, 0}
}

```

```

Default_Palette_Y_Size_Cdf[ PALETTE_SIZES + 1 ] =
{ 8779, 15095, 20777, 24903, 27923, 30403, 32, 0}

```

```

Default_Palette_Y_Mode_Cdf[ 3 ] =
{30045, 62, 0}

```

```

Default_Palette_Size_2_Y_Color_Cdf[ PALETTE_COLOR_CONTEXTS ][ 3 ] =
{
    {28140, 90, 0},
    {16384, 0, 0},
    { 8582, 6, 0},
    {27413, 32, 0},
    {30429, 93, 0}
}

```

```

Default_Palette_Size_3_Y_Color_Cdf[ PALETTE_COLOR_CONTEXTS ][ 4 ] =
{
    {25350, 29026, 90, 0},
    {11363, 25273, 7, 0},
    { 6841, 28579, 1, 0},
    {21350, 26012, 6, 0},
    {30573, 31646, 93, 0}
}

```

```

Default_Palette_Size_4_Y_Color_Cdf[ PALETTE_COLOR_CONTEXTS ][ 5 ] =
{
    {23706, 26962, 29060, 0, 0},
    { 9976, 22516, 27382, 1, 0},
    { 6691, 25460, 29234, 6, 0},
    {18909, 23925, 28403, 31, 0},
    {30308, 31076, 31818, 93, 0}
}

```

```

Default_Palette_Size_5_Y_Color_Cdf[ PALETTE_COLOR_CONTEXTS ][ 6 ] =
{
    {24116, 26957, 28486, 29941, 0, 0},
    { 9568, 20472, 24294, 28942, 81, 0},
    { 5706, 25243, 28040, 30406, 76, 0},
    {20105, 22982, 27024, 28911, 31, 0},
    {30897, 31342, 31766, 32199, 93, 0}
}

```

```

Default_Palette_Size_6_Y_Color_Cdf[ PALETTE_COLOR_CONTEXTS ][ 7 ] =
{
    {20824, 24227, 25926, 27459, 29266, 75, 0},
    { 8141, 18989, 21599, 26182, 28576, 75, 0},

```

```

{ 5252, 24340, 26450, 28438, 30625, 75, 0},
{19519, 22695, 25587, 26972, 28423, 6, 0},
{30383, 30890, 31247, 31653, 32150, 78, 0}
}

```

```

Default_Palette_Size_7_Y_Color_Cdf[ PALETTE_COLOR_CONTEXTS ][ 8 ] =
{
{21628, 24512, 25873, 27054, 28131, 29539, 80, 0},
{ 8028, 18264, 20613, 25424, 27112, 28906, 90, 0},
{ 6489, 22242, 24461, 26394, 28350, 30510, 75, 0},
{22048, 24429, 26990, 27944, 28417, 29574, 76, 0},
{30801, 31205, 31472, 31728, 32005, 32305, 93, 0}
}

```

```

Default_Palette_Size_8_Y_Color_Cdf[ PALETTE_COLOR_CONTEXTS ][ 9 ] =
{
{22471, 25083, 25984, 26893, 27654, 28750, 29903, 93, 0},
{ 7542, 17057, 19151, 23550, 25459, 27066, 28804, 90, 0},
{ 7582, 20437, 22728, 24622, 26515, 28579, 30632, 90, 0},
{22102, 24144, 26916, 28151, 28846, 29212, 30153, 0, 0},
{30524, 30887, 31156, 31393, 31626, 31911, 32281, 93, 0}
}

```

```

Default_Delta_Q_Cdf[ DELTA_Q_SMALL + 2 ] =
{16594, 23325, 26424, 28225, 29358, 30099, 30613, 56, 0}

```

```

Default_Intra_Tx_Type_Set1_Cdf[ 3 ][ 8 ] =
{
{ 5026, 16816, 19974, 23404, 26845, 30499, 8, 0},
{14910, 25257, 26964, 29323, 30237, 31535, 0, 0},
{13759, 26108, 27688, 29793, 30265, 31576, 35, 0}
}

```

```

Default_Intra_Tx_Type_Set2_Cdf[ 3 ][ 3 ] =
{
{16384, 0, 0},
{16384, 0, 0},
{16384, 0, 0}
}

```

```

Default_Sec_Tx_Type_Cdf[ 2 ][ TX_SIZES ][ STX_TYPES + 1 ] =
{
{
{ 4486, 15589, 26440, 75, 0},
{ 2357, 9504, 16641, 5, 0},
{ 1364, 8034, 14431, 0, 0},
{ 2472, 8725, 13853, 76, 0},
{ 7523, 11681, 14783, 1, 0}
},
{
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0},
{10490, 15394, 22206, 31, 0},
{13967, 16953, 23109, 31, 0},
{20191, 21727, 25818, 32, 0}
}
}
}

```

```

Default_Inter_Tx_Type_Set1_Cdf[ 3 ][ 2 ][ 3 ] =
{
{
{14122, 25, 0},
{ 8962, 31, 0}
},
{

```

```

    {16423, 26, 0},
    {23446, 0, 0}
},
{
    {23352, 25, 0},
    {17069, 56, 0}
}
}

```

```

Default_Inter_Tx_Type_Index_Set1_Cdf[ 3 ][ 8 + 1 ] =
{
    { 8914, 10732, 12270, 14822, 17128, 19586, 21964, 25, 0},
    { 1160, 1555, 1972, 3414, 3962, 5151, 5908, 31, 0},
    {22819, 24338, 25592, 27001, 28395, 29648, 30990, 25, 0}
}

```

```

Default_Inter_Tx_Type_Index_Set2_Cdf[ 3 ][ 8 + 1 ] =
{
    { 1140, 1725, 2324, 14653, 19072, 23618, 28109, 31, 0},
    { 58, 261, 587, 32317, 32556, 32626, 32708, 2, 0},
    {17404, 17669, 18403, 24052, 26393, 28506, 30676, 57, 0}
}

```

```

Default_Inter_Tx_Type_Offset_Set1_Cdf[ 3 ][ 8 + 1 ] =
{
    { 3121, 6470, 9191, 12280, 17811, 22588, 27697, 93, 0},
    { 338, 377, 571, 743, 7932, 11860, 17524, 1, 0},
    { 3314, 7625, 10639, 14206, 19363, 23456, 28033, 93, 0}
}

```

```

Default_Inter_Tx_Type_Offset_Set2_Cdf[ 3 ][ 4 + 1 ] =
{
    { 8669, 16533, 24855, 93, 0},
    { 9441, 16413, 25276, 90, 0},
    { 8767, 17611, 24876, 10, 0}
}

```

```

Default_Inter_Tx_Type_Set2_Cdf[ 3 ][ 3 ] =
{
    {20835, 0, 0},
    {30720, 3, 0},
    {28857, 17, 0}
}

```

```

Default_Inter_Tx_Type_Set3_Cdf[ 3 ][ 4 ][ 3 ] =
{
    {
        {16384, 0, 0},
        {16384, 0, 0},
        {16384, 0, 0},
        {16384, 0, 0}
    },
    {
        {16384, 0, 0},
        {16384, 0, 0},
        {16384, 0, 0},
        {16384, 0, 0}
    },
    {
        {16384, 0, 0},
        {16384, 0, 0},
        {16384, 0, 0}
    }
}

```

```

    {16384, 0, 0}
}

```

```

Default_Inter_Tx_Type_Set4_Cdf[ 3 ][ 4 ][ 5 ] =
{
    {
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0}
    },
    {
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0}
    },
    {
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0},
        { 8192, 16384, 24576, 0, 0}
    }
}

```

```

Default_Comp_Group_Idx_Cdf[COMP_GROUP_IDX_CONTEXTS][ 3 ] =
{
    {17510, 26, 0},
    {10382, 0, 0},
    { 8336, 31, 0},
    { 6054, 1, 0},
    { 5764, 5, 0},
    { 7154, 10, 0},
    {15013, 26, 0},
    { 8426, 0, 0},
    { 8278, 1, 0},
    { 4856, 1, 0},
    { 3464, 0, 0},
    { 5295, 0, 0}
}

```

```

Default_Compound_Type_Cdf[ COMPOUND_TYPES + 1 ] =
{16894, 31, 0}

```

```

Default_Inter_Intra_Cdf[ BLOCK_SIZE_GROUPS ][ 3 ] =
{
    {20569, 1, 0},
    {17106, 1, 0},
    {20948, 1, 0},
    {25796, 32, 0}
}

```

```

Default_Warp_Inter_Intra_Cdf[ BLOCK_SIZE_GROUPS ][ 3 ] =
{
    {16384, 0, 0},
    {27980, 56, 0},
    {29163, 56, 0},
    {30658, 62, 0}
}

```

```

Default_Inter_Intra_Mode_Cdf[ BLOCK_SIZE_GROUPS ][ INTERINTRA_MODES + 1 ] =
{
    { 1819, 16131, 26802, 32, 0},
    { 1442, 15840, 28441, 1, 0},

```

```

{ 1995, 15814, 28221, 7, 0},
{ 3564, 15440, 28048, 32, 0}
}

```

```

Default_Wedge_Quad_Cdf[ 4 + 1 ] =
{ 6511, 18144, 27374, 1, 0}

```

```

Default_Wedge_Angle_Cdf[ 4 ][ 5 + 1 ] =
{
{10258, 15276, 19997, 26561, 6, 0},
{14039, 19183, 26143, 30047, 6, 0},
{19564, 22099, 25104, 29960, 1, 0},
{13808, 17950, 25715, 29008, 7, 0}
}

```

```

Default_Wedge_Dist1_Cdf[ NUM_WEDGE_DIST + 1 ] =
{ 8203, 16994, 21032, 75, 0}

```

```

Default_Wedge_Dist2_Cdf[ NUM_WEDGE_DIST ] =
{14463, 19115, 75, 0}

```

```

Default_Wedge_Inter_Intra_Cdf[ 3 ] =
{16758, 0, 0}

```

```

Default_Warp_Precision_Cdf[ BLOCK_SIZES ][ 2 + 1 ] =
{
{16384, 0, 0},
{16384, 0, 0},
{16384, 0, 0},
{31107, 56, 0},
{27357, 31, 0},
{26435, 26, 0},
{22585, 26, 0},
{20146, 27, 0},
{18671, 32, 0},
{18210, 32, 0},
{17968, 25, 0},
{17451, 40, 0},
{16736, 25, 0},
{16040, 25, 0},
{15633, 25, 0},
{13952, 25, 0},
{14893, 25, 0},
{13323, 25, 0},
{11830, 50, 0},
{16384, 0, 0},
{16384, 0, 0},
{21783, 28, 0},
{20345, 15, 0},
{19398, 25, 0},
{17823, 35, 0},
{16384, 0, 0},
{16384, 0, 0},

```

```

{20066, 25, 0},
{18893, 5, 0}
}

```

```

Default_Warp_Delta_Param_Low_Cdf[ 2 ][ WARP_DELTA_NUM_SYMBOLS_LOW + 1 ] =
{
{ 8995, 22970, 25406, 29673, 30295, 31670, 31863, 0, 0},
{13333, 24012, 26545, 30183, 30839, 31958, 32139, 0, 0}
}

```

```

Default_Warp_Delta_Param_High_Cdf[ 2 ][ WARP_DELTA_NUM_SYMBOLS_HIGH + 1 ] =
{
{ 8959, 14388, 19825, 21810, 25035, 28077, 29469, 15, 0},
{ 9199, 14146, 19484, 21591, 24614, 28015, 29538, 0, 0}
}

```

```

Default_Warp_Delta_Param_Sign_Cdf[ 2 + 1 ] =
{14285, 93, 0}

```

```

Default_Use_Local_Warp_Cdf[ WARP_CAUSAL_MODE_CTX ][ 2 + 1 ] =
{
{14877, 1, 0},
{12801, 0, 0},
{ 6885, 6, 0},
{ 2987, 30, 0}
}

```

```

Default_Use_Extend_Warp_Cdf[ 3 ][ 3 ] =
{
{20359, 0, 0},
{20310, 75, 0},
{21759, 75, 0}
}

```

```

Default_Cfl_Sign_Cdf[ CFL_JOINT_SIGNS + 1 ] =
{ 2421, 4332, 11256, 12766, 21386, 28725, 32087, 62, 0}

```

```

Default_Cfl_Alpha_Cdf[ CFL_ALPHA_CONTEXTS ][ CFL_ALPHABET_SIZE + 1 ] =
{
{21679, 25305, 30646, 31512, 32537, 32646, 32696, 62, 0},
{ 8262, 16302, 24082, 29422, 31398, 32286, 32525, 62, 0},
{17235, 26166, 30378, 31305, 32373, 32549, 32668, 62, 0},
{17618, 25732, 27865, 30338, 31125, 31522, 32238, 62, 0},

```

```

    {17542, 23066, 27907, 28728, 30702, 31165, 31435, 62, 0},
    {17675, 24802, 30468, 30783, 31841, 32264, 32422, 62, 0}
}

```

```

Default_Cfl_Mhccp_Cdf[ 2 + 1 ] =
{15499, 30, 0}

```

```

Default_Cfl_Index_Cdf[ 2 + 1 ] =
{12507, 30, 0}

```

```

Default_Use_Wiener_Ns_Cdf[ 2 + 1 ] =
{ 9051, 32, 0}

```

```

Default_Wiener_Ns_Length_Cdf[2][3] =
{
    { 4898, 56, 0},
    { 6088, 26, 0}
}

```

```

Default_Wiener_Ns_Uv_Sym_Cdf[3] =
{29286, 65, 0}

```

```

Default_Wiener_Ns_Base_Cdf[ 4 + 1 ] =
{16557, 24352, 29677, 6, 0}

```

```

Default_Use_Pc_Wiener_Cdf[ 2 + 1 ] =
{12799, 25, 0}

```

```

Default_Flex_Restoration_Type_Cdf[MAX_LR_FLEX_SWITCHABLE_BITS][3][3] =
{
    {
        {25542, 62, 0},
        {16384, 0, 0},
        {16384, 0, 0}
    },
    {
        {25550, 37, 0},
        {16384, 0, 0},
        {16384, 0, 0}
    },
    {
        {16384, 0, 0},
        {16384, 0, 0},
        {16384, 0, 0}
    }
}

```

```

Default_Txb_Skip_Cdf[ COEFF_CDF_Q_CTXS ][ 2 ][ TX_SIZES ]
[ TXB_SKIP_CONTEXTS ][ 3 ] = {
    {
        {
            {25759, 31, 0},
            { 1099, 93, 0},
            { 2762, 5, 0},
            { 7944, 26, 0},
            {16230, 6, 0},
            {29076, 31, 0},
            { 8898, 1, 0},
            {13655, 1, 0},
            {22348, 0, 0},
            {16384, 0, 0}
        },
    },
}

```



```

{
  {24799, 56, 0},
  { 3162, 75, 0},
  { 4813,  0, 0},
  {13562,  1, 0},
  {17989, 76, 0},
  {31799,  0, 0},
  { 5170, 75, 0},
  {12844,  1, 0},
  {22336,  6, 0},
  {16384,  0, 0}
},
{
  {18946, 62, 0},
  { 5651,  6, 0},
  { 6193,  7, 0},
  {15587,  7, 0},
  {18312, 91, 0},
  {31798, 80, 0},
  { 2961,  0, 0},
  {11605, 26, 0},
  {24657, 31, 0},
  {16384,  0, 0}
},
{
  { 7809, 62, 0},
  { 3864, 33, 0},
  {14130, 40, 0},
  {16018, 32, 0},
  {15599, 10, 0},
  {28161, 32, 0},
  { 1300, 36, 0},
  {11703, 57, 0},
  {24729, 33, 0},
  {16384,  0, 0}
},
{
  { 2800, 62, 0},
  {  159, 32, 0},
  { 1725, 50, 0},
  {11138, 50, 0},
  {16384,  0, 0},
  {32319,  0, 0},
  {  456, 63, 0},
  { 6416, 50, 0},
  {16384,  0, 0},
  {16384,  0, 0}
}
},
{
  {
    {
      {32764,  0, 0},
      {10751, 16, 0},
      {17579, 16, 0},
      {24060, 90, 0},
      {28085,  5, 0},
      {30928, 93, 0},
      { 5437, 37, 0},
      {17819, 26, 0},
      {28074,  8, 0},
      {30432, 80, 0}
    },
    {
      {32452, 62, 0},
      {14328, 93, 0},
      {18927, 93, 0},
      {22405, 93, 0},
      {25460, 93, 0},
      {30634, 75, 0},
      { 5152, 31, 0},
      {15362, 31, 0},
    }
  }
}

```

```

    {24814, 6, 0},
    {31865, 0, 0}
},
{
    {31891, 62, 0},
    {17014, 93, 0},
    {19836, 93, 0},
    {20450, 6, 0},
    {25131, 1, 0},
    {31538, 3, 0},
    { 5465, 25, 0},
    {17440, 31, 0},
    {28788, 1, 0},
    {29135, 37, 0}
},
{
    {31092, 62, 0},
    {11952, 28, 0},
    { 1928, 0, 0},
    {19906, 12, 0},
    {32150, 50, 0},
    {31347, 52, 0},
    { 9362, 10, 0},
    {29127, 50, 0},
    {32373, 50, 0},
    {27387, 60, 0}
},
{
    {26822, 63, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0}
}
}
},
{
    {
        {
            {30296, 5, 0},
            { 2871, 75, 0},
            { 5650, 76, 0},
            {13405, 26, 0},
            {19531, 1, 0},
            {30186, 5, 0},
            { 8778, 31, 0},
            {17506, 0, 0},
            {26467, 0, 0},
            {16384, 0, 0}
        },
        {
            {31530, 30, 0},
            { 2361, 78, 0},
            { 4301, 75, 0},
            {14660, 31, 0},
            {19138, 6, 0},
            {31475, 5, 0},
            { 5031, 0, 0},
            {16054, 31, 0},
            {27047, 5, 0},
            {16384, 0, 0}
        },
        {
            {31362, 25, 0},
            { 2991, 75, 0},
            { 6803, 1, 0},

```

```

    {19942, 32, 0},
    {22017, 6, 0},
    {32325, 5, 0},
    { 5502, 0, 0},
    {17059, 31, 0},
    {27657, 31, 0},
    {16384, 0, 0}
},
{
    {27230, 62, 0},
    { 604, 80, 0},
    { 4108, 0, 0},
    {15740, 62, 0},
    {18869, 27, 0},
    {32126, 32, 0},
    { 4208, 31, 0},
    {18962, 32, 0},
    {28315, 31, 0},
    {16384, 0, 0}
},
{
    {23062, 62, 0},
    { 740, 33, 0},
    { 9511, 42, 0},
    {22294, 62, 0},
    {21507, 93, 0},
    {31946, 57, 0},
    { 2406, 62, 0},
    {18145, 31, 0},
    {24883, 33, 0},
    {16384, 0, 0}
}
},
{
    {
        {32764, 0, 0},
        {10589, 91, 0},
        {17200, 7, 0},
        {20721, 1, 0},
        {23311, 6, 0},
        {28906, 5, 0},
        { 4037, 26, 0},
        {14946, 26, 0},
        {24129, 0, 0},
        {32128, 78, 0}
    },
    {
        {32577, 31, 0},
        {13622, 93, 0},
        {16203, 90, 0},
        {19412, 90, 0},
        {23766, 75, 0},
        {29589, 3, 0},
        { 2774, 26, 0},
        {14855, 26, 0},
        {24976, 0, 0},
        {31738, 75, 0}
    },
    {
        {32415, 62, 0},
        {15422, 93, 0},
        {18741, 91, 0},
        {20940, 75, 0},
        {25667, 76, 0},
        {31116, 0, 0},
        { 3281, 31, 0},
        {17757, 31, 0},
        {27707, 1, 0},
        {29599, 7, 0}
    },
    {

```

```

    {32182, 62, 0},
    {13890, 90, 0},
    {17539, 18, 0},
    {21145, 15, 0},
    {26833, 93, 0},
    {31582, 1, 0},
    { 4193, 27, 0},
    {21696, 32, 0},
    {29128, 6, 0},
    {26841, 62, 0}
},
{
    {31681, 62, 0},
    { 9846, 32, 0},
    {21469, 32, 0},
    {24254, 7, 0},
    {29068, 92, 0},
    {31736, 32, 0},
    { 8875, 62, 0},
    {25927, 60, 0},
    {29086, 60, 0},
    {16384, 0, 0}
}
},
{
    {
        {28021, 3, 0},
        { 4675, 75, 0},
        { 6407, 75, 0},
        {12870, 0, 0},
        {18069, 1, 0},
        {26728, 30, 0},
        { 4204, 6, 0},
        {11853, 75, 0},
        {19479, 0, 0},
        {16384, 0, 0}
    },
    {
        {30300, 0, 0},
        { 4727, 3, 0},
        { 5333, 75, 0},
        {13460, 6, 0},
        {17424, 6, 0},
        {28814, 30, 0},
        { 4777, 0, 0},
        {12522, 0, 0},
        {20439, 0, 0},
        {16384, 0, 0}
    },
    {
        {31298, 25, 0},
        { 6336, 0, 0},
        { 7488, 76, 0},
        {19193, 31, 0},
        {21405, 31, 0},
        {31491, 5, 0},
        { 5723, 0, 0},
        {15055, 1, 0},
        {23518, 1, 0},
        {16384, 0, 0}
    },
    {
        {31807, 56, 0},
        { 1393, 75, 0},
        { 3549, 2, 0},
        {18574, 32, 0},
        {19795, 32, 0},
        {31792, 36, 0},
        { 5941, 31, 0},
    }
}

```

```

    {16460, 32, 0},
    {25194, 31, 0},
    {16384, 0, 0}
},
{
    {32278, 61, 0},
    { 2582, 31, 0},
    {13992, 31, 0},
    {25993, 32, 0},
    {26283, 77, 0},
    {32056, 0, 0},
    { 6989, 32, 0},
    {17824, 32, 0},
    {24231, 32, 0},
    {16384, 0, 0}
}
},
{
    {
        {32764, 0, 0},
        { 9384, 91, 0},
        {12245, 7, 0},
        {14958, 75, 0},
        {18080, 30, 0},
        {23099, 30, 0},
        { 2191, 90, 0},
        { 9674, 1, 0},
        {17537, 1, 0},
        {32587, 90, 0}
    },
    {
        {32729, 18, 0},
        {12523, 93, 0},
        {13844, 93, 0},
        {16079, 93, 0},
        {21004, 90, 0},
        {25973, 0, 0},
        { 1860, 15, 0},
        { 9626, 1, 0},
        {18115, 1, 0},
        {31918, 25, 0}
    },
    {
        {32727, 43, 0},
        {14244, 93, 0},
        {15808, 91, 0},
        {18087, 75, 0},
        {23066, 90, 0},
        {27617, 0, 0},
        { 1932, 0, 0},
        {12274, 1, 0},
        {23957, 6, 0},
        {30007, 7, 0}
    },
    {
        {32605, 56, 0},
        {14434, 92, 0},
        {21625, 78, 0},
        {21326, 2, 0},
        {25757, 5, 0},
        {30059, 8, 0},
        { 2485, 26, 0},
        {15041, 6, 0},
        {26473, 5, 0},
        {25895, 63, 0}
    },
    {
        {32562, 56, 0},
        {15140, 7, 0},
        {23809, 7, 0},
        {26086, 31, 0},

```

```

        {29383, 2, 0},
        {31155, 16, 0},
        { 4393, 51, 0},
        {17678, 33, 0},
        {25607, 56, 0},
        {16384, 0, 0}
    }
}
},
{
    {
        {21443, 16, 0},
        { 4688, 0, 0},
        { 4478, 90, 0},
        { 7588, 0, 0},
        { 9776, 1, 0},
        {14682, 1, 0},
        { 421, 90, 0},
        { 2639, 76, 0},
        { 5035, 40, 0},
        {16384, 0, 0}
    },
    {
        {27522, 0, 0},
        { 5380, 78, 0},
        { 5185, 90, 0},
        {10193, 0, 0},
        {13111, 1, 0},
        {22246, 31, 0},
        { 1317, 0, 0},
        { 5966, 90, 0},
        {10326, 75, 0},
        {16384, 0, 0}
    },
    {
        {30197, 6, 0},
        { 8536, 0, 0},
        { 7061, 75, 0},
        {15977, 6, 0},
        {17416, 6, 0},
        {28033, 6, 0},
        { 2336, 0, 0},
        { 8267, 76, 0},
        {13804, 76, 0},
        {16384, 0, 0}
    },
    {
        {32558, 56, 0},
        { 3796, 0, 0},
        { 4485, 76, 0},
        {16972, 32, 0},
        {17777, 37, 0},
        {29879, 35, 0},
        { 3523, 1, 0},
        {10339, 2, 0},
        {17710, 7, 0},
        {16384, 0, 0}
    },
    {
        {32683, 93, 0},
        { 6378, 32, 0},
        {13628, 2, 0},
        {24546, 76, 0},
        {26525, 90, 0},
        {31671, 75, 0},
        { 5036, 31, 0},
        {11209, 32, 0},
        {17036, 7, 0},
        {16384, 0, 0}
    }
}

```

```

},
{
{
{32754, 50, 0},
{12951, 93, 0},
{15900, 93, 0},
{14312, 16, 0},
{17581, 15, 0},
{19079, 18, 0},
{ 507, 93, 0},
{ 3382, 28, 0},
{ 7041, 18, 0},
{32482, 81, 0}
},
{
{32755, 68, 0},
{11625, 93, 0},
{10365, 93, 0},
{11240, 93, 0},
{13860, 75, 0},
{18669, 0, 0},
{ 620, 93, 0},
{ 4574, 76, 0},
{11493, 83, 0},
{31754, 75, 0}
},
{
{32749, 93, 0},
{13494, 93, 0},
{13995, 90, 0},
{14723, 90, 0},
{19850, 0, 0},
{24076, 1, 0},
{ 816, 75, 0},
{ 8011, 6, 0},
{18757, 11, 0},
{32020, 3, 0}
},
{
{32697, 37, 0},
{12925, 90, 0},
{14605, 80, 0},
{15058, 77, 0},
{23741, 18, 0},
{25610, 6, 0},
{ 1293, 5, 0},
{11938, 1, 0},
{22991, 32, 0},
{26438, 65, 0}
},
{
{32644, 37, 0},
{16042, 82, 0},
{24056, 7, 0},
{24203, 7, 0},
{27963, 5, 0},
{29458, 5, 0},
{ 2804, 57, 0},
{19130, 43, 0},
{26311, 60, 0},
{16384, 0, 0}
}
}
}
}
}

```

```

Default_V_Txb_Skip_Cdf[COEFF_CDF_Q_CTXS][V_TXB_SKIP_CONTEXTS][3] =
{
{
{ 1439, 78, 0},

```

```

    { 6191, 0, 0},
    {14610, 1, 0},
    { 180, 50, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    { 7648, 90, 0},
    {16148, 90, 0},
    {24565, 90, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0}
},
{
    { 1341, 93, 0},
    { 7768, 75, 0},
    {16044, 1, 0},
    { 104, 60, 0},
    {16384, 0, 0},
    {16384, 0, 0},
    { 8426, 0, 0},
    {16919, 0, 0},
    {25248, 0, 0},
    { 8448, 50, 0},
    {27163, 50, 0},
    {27594, 50, 0}
},
{
    { 777, 93, 0},
    { 6871, 75, 0},
    {15620, 0, 0},
    { 1146, 35, 0},
    {14000, 25, 0},
    {24576, 50, 0},
    { 4536, 90, 0},
    {13035, 75, 0},
    {21755, 75, 0},
    { 5906, 35, 0},
    {18536, 25, 0},
    {26466, 50, 0}
},
{
    { 606, 90, 0},
    { 6309, 0, 0},
    {13697, 1, 0},
    { 801, 10, 0},
    {10893, 25, 0},
    {19895, 50, 0},
    { 6221, 76, 0},
    {13720, 75, 0},
    {20139, 75, 0},
    { 7333, 60, 0},
    {20045, 25, 0},
    {24485, 50, 0}
}
}

```

```

Default_Eob_Pt_16_Cdf[ COEFF_CDF_Q_CTXS ][ EOB_PLANE_CTXS ][ 6 ] =
{
    {
        { 1946, 3059, 6834, 15123, 6, 0},
        { 558, 835, 2739, 6534, 25, 0},
        { 8000, 10366, 14466, 19569, 31, 0}
    },
    {
        { 1839, 2995, 6796, 15126, 30, 0},
        { 1354, 1787, 4813, 9327, 30, 0},
        { 8298, 11869, 17297, 24569, 30, 0}
    },
    {
        { 3946, 6406, 13325, 26004, 30, 0},

```



```

    { 2616, 3402, 7857, 13406, 30, 0},
    {13678, 17362, 23556, 28557, 30, 0}
},
{
    { 5029, 12046, 26017, 32453, 31, 0},
    { 3860, 5738, 8894, 18233, 31, 0},
    {29060, 30345, 32212, 32705, 27, 0}
}
}

```

```

Default_Eob_Pt_32_Cdf[ COEFF_CDF_Q_CTXS ][ EOB_PLANE_CTXS ][ 7 ] =
{
    {
        { 1203, 1920, 4502, 10281, 17078, 6, 0},
        { 1078, 1340, 2464, 5152, 11416, 5, 0},
        {11670, 14271, 19287, 23919, 27084, 31, 0}
    },
    {
        { 1143, 1778, 3854, 8166, 13337, 31, 0},
        { 1109, 1433, 2804, 5652, 11224, 31, 0},
        { 8685, 11412, 16486, 22345, 27930, 31, 0}
    },
    {
        { 3277, 4780, 9255, 16544, 21853, 31, 0},
        { 1655, 2364, 4679, 10029, 18955, 26, 0},
        {15549, 19007, 24651, 29173, 31742, 6, 0}
    },
    {
        { 7189, 11732, 21773, 29775, 32060, 31, 0},
        { 3833, 7634, 15540, 24684, 30675, 31, 0},
        {26250, 28453, 31497, 32596, 32732, 32, 0}
    }
}
}

```

```

Default_Eob_Pt_64_Cdf[ COEFF_CDF_Q_CTXS ][ EOB_PLANE_CTXS ][ 8 ] =
{
    {
        { 1848, 2357, 4392, 7629, 13328, 21530, 32, 0},
        { 1476, 1703, 2435, 4245, 7520, 14818, 6, 0},
        {13125, 16136, 21482, 26341, 29681, 31034, 31, 0}
    },
    {
        { 939, 1381, 3142, 6536, 13617, 21799, 31, 0},
        { 1338, 1662, 2777, 5268, 9184, 15736, 31, 0},
        { 9398, 11810, 17045, 22537, 27448, 30795, 31, 0}
    },
    {
        { 2024, 3239, 6969, 13073, 21617, 27643, 31, 0},
        { 1514, 1937, 3810, 7727, 13471, 23183, 31, 0},
        {14959, 18244, 24321, 29177, 31773, 32497, 1, 0}
    },
    {
        { 3765, 6333, 13340, 20831, 28830, 32289, 31, 0},
        { 5045, 7407, 13926, 20909, 27015, 31664, 31, 0},
        {24202, 26714, 30623, 32371, 32728, 32732, 6, 0}
    }
}
}

```

```

Default_Eob_Pt_128_Cdf[ COEFF_CDF_Q_CTXS ][ EOB_PLANE_CTXS ][ 9 ] =
{
    {
        { 336, 988, 2984, 5382, 9800, 16995, 24488, 32, 0},
        { 1506, 2078, 2996, 4361, 7027, 12117, 21037, 6, 0},
        {10069, 12663, 18436, 24325, 28451, 30559, 31560, 31, 0}
    },
    {
        { 654, 1068, 2257, 5580, 12362, 21455, 26067, 32, 0},
        { 1852, 2243, 3554, 5549, 9085, 14604, 21559, 32, 0},
    }
}

```

```

    { 9713, 12146, 17484, 22818, 27372, 30388, 31811, 31, 0}
},
{
    { 1672, 2600, 5666, 11986, 21596, 28733, 30851, 31, 0},
    { 2101, 2605, 4469, 8056, 13125, 19491, 26379, 31, 0},
    {15041, 18279, 24430, 28935, 31545, 32439, 32672, 6, 0}
},
{
    { 4122, 6840, 13278, 20906, 28131, 31450, 32305, 31, 0},
    { 4575, 6063, 10938, 17191, 23126, 27885, 31584, 31, 0},
    {23490, 25861, 30005, 32096, 32701, 32720, 32724, 1, 0}
}
}

```

```

Default_Eob_Pt_256_Cdf[ COEFF_CDF_Q_CTXS ][ EOB_PLANE_CTXS ][ 9 ] =
{
    {
        { 321, 2376, 5427, 7836, 12726, 18552, 21084, 32, 0},
        { 1954, 2587, 3867, 5133, 7560, 10574, 14742, 37, 0},
        { 7984, 10878, 15991, 20185, 25357, 29358, 31262, 32, 0}
    },
    {
        { 790, 1184, 2218, 5022, 11432, 20289, 23689, 57, 0},
        { 2998, 3363, 4528, 6371, 9250, 13525, 18730, 32, 0},
        { 9611, 12008, 17284, 22532, 26744, 29711, 31730, 31, 0}
    },
    {
        { 1556, 2520, 5142, 10585, 19507, 28407, 30571, 32, 0},
        { 3608, 4081, 5802, 8998, 13007, 18326, 23738, 32, 0},
        {14007, 17061, 22822, 27233, 30196, 31899, 32583, 6, 0}
    },
    {
        { 3574, 5683, 11604, 19611, 27344, 30852, 31552, 31, 0},
        { 7197, 7949, 12414, 18002, 23115, 27424, 30391, 32, 0},
        {23383, 25747, 29846, 31947, 32613, 32720, 32724, 6, 0}
    }
}
}

```

```

Default_Eob_Pt_512_Cdf[ COEFF_CDF_Q_CTXS ][ EOB_PLANE_CTXS ][ 9 ] =
{
    {
        { 139, 5954, 13032, 16244, 20068, 24954, 25897, 32, 0},
        { 3412, 3896, 5734, 7524, 10117, 13773, 16441, 62, 0},
        {12810, 16127, 22415, 25016, 27768, 29850, 31372, 32, 0}
    },
    {
        { 1311, 2019, 3578, 6876, 13781, 23541, 24922, 62, 0},
        { 5415, 5926, 7647, 10113, 13081, 17084, 20509, 57, 0},
        { 9906, 12482, 17574, 22657, 27655, 30234, 31821, 32, 0}
    },
    {
        { 1654, 2720, 5317, 10565, 19306, 29790, 30875, 32, 0},
        { 7237, 7980, 10164, 13730, 18015, 23652, 27137, 32, 0},
        {14503, 17149, 22592, 27035, 30013, 31699, 32526, 7, 0}
    },
    {
        { 3753, 5634, 10147, 17539, 26473, 32201, 32398, 6, 0},
        {11196, 12269, 16359, 22348, 27230, 30729, 31805, 32, 0},
        {22240, 24551, 29042, 31545, 32498, 32720, 32724, 7, 0}
    }
}
}

```

```

Default_Eob_Pt_1024_Cdf[ COEFF_CDF_Q_CTXS ][ EOB_PLANE_CTXS ][ 9 ] =
{
    {
        { 576, 1953, 8086, 14995, 20006, 24682, 25870, 62, 0},
        { 2747, 3019, 3935, 4876, 6051, 8588, 10135, 57, 0},
        {11873, 15379, 25835, 27918, 29093, 30164, 30782, 32, 0}
    }
}

```

```

},
{
{ 634, 1777, 3873, 6293, 12653, 23825, 24712, 57, 0},
{ 4894, 5819, 8219, 11022, 14406, 18919, 21479, 57, 0},
{ 9216, 12549, 17630, 21723, 25862, 28760, 30440, 37, 0}
},
{
{ 2101, 3304, 6486, 12539, 21781, 30341, 30738, 32, 0},
{11349, 12231, 14967, 18532, 22406, 26663, 28302, 57, 0},
{15081, 17873, 23055, 27251, 30053, 31491, 32305, 32, 0}
},
{
{ 5838, 8921, 14454, 21162, 27888, 32275, 32424, 31, 0},
{15377, 16920, 20643, 25175, 28586, 31294, 32008, 32, 0},
{23067, 25626, 29608, 31655, 32465, 32687, 32716, 7, 0}
}
}
}

```

```

Default_Eob_Extra_Cdf[ COEFF_CDF_Q_CTXS ][ 3 ] =
{
{16391, 0, 0},
{18326, 0, 0},
{19673, 78, 0},
{20364, 3, 0}
}

```

```

Default Idtx Sign Cdf
[COEFF_CDF_Q_CTXS][ FSC_TX_SIZE_CONTEXTS ][IDTX_SIGN_CONTEXTS][2 + 1] = {
{
{
{15560, 93, 0},
{24775, 93, 0},
{ 7540, 93, 0},
{27844, 0, 0},
{ 3545, 75, 0},
{28880, 82, 0},
{ 4886, 80, 0},
{32178, 6, 0},
{ 1204, 75, 0}
},
{
{14810, 3, 0},
{26650, 75, 0},
{ 5063, 90, 0},
{29212, 15, 0},
{ 1994, 78, 0},
{29106, 90, 0},
{ 3267, 90, 0},
{32118, 0, 0},
{ 537, 93, 0}
},
{
{11533, 31, 0},
{28709, 76, 0},
{ 2095, 90, 0},
{31598, 90, 0},
{ 331, 93, 0},
{31760, 91, 0},
{ 319, 78, 0},
{32511, 78, 0},
{ 60, 43, 0}
}
},
{
{
{17010, 0, 0},
{25832, 0, 0},
{ 7718, 3, 0},
{29537, 1, 0},

```

```

    { 8925, 31, 0},
    {30595, 75, 0},
    { 4677, 90, 0},
    {32471, 90, 0},
    { 3023, 1, 0}
},
{
    {17528, 25, 0},
    {27808, 0, 0},
    { 6222, 0, 0},
    {29747, 1, 0},
    { 2848, 31, 0},
    {31635, 75, 0},
    { 2513, 75, 0},
    {32514, 75, 0},
    { 474, 18, 0}
},
{
    {19157, 31, 0},
    {30551, 31, 0},
    { 2788, 31, 0},
    {30752, 37, 0},
    { 701, 26, 0},
    {32565, 15, 0},
    { 359, 8, 0},
    {32515, 87, 0},
    { 223, 37, 0}
}
},
{
    {
        {20565, 0, 0},
        {28478, 25, 0},
        { 8011, 15, 0},
        {31569, 56, 0},
        { 9937, 31, 0},
        {32166, 78, 0},
        { 4360, 83, 0},
        {32715, 38, 0},
        { 2253, 43, 0}
    },
    {
        {21616, 26, 0},
        {29474, 25, 0},
        { 8169, 25, 0},
        {31873, 56, 0},
        { 6252, 31, 0},
        {32360, 18, 0},
        { 2458, 0, 0},
        {32719, 52, 0},
        { 682, 68, 0}
    },
    {
        {22816, 31, 0},
        {30531, 26, 0},
        { 5700, 31, 0},
        {31507, 62, 0},
        { 1539, 62, 0},
        {32572, 26, 0},
        { 1015, 32, 0},
        {32694, 35, 0},
        { 412, 37, 0}
    }
}
},
{
    {
        {18478, 31, 0},
        {30410, 26, 0},
        { 3274, 16, 0},
        {16384, 0, 0},
        {16384, 0, 0},

```

```

    {32138, 0, 0},
    { 697, 50, 0},
    {16384, 0, 0},
    {16384, 0, 0}
},
{
    {24519, 32, 0},
    {32572, 26, 0},
    { 653, 13, 0},
    {32404, 50, 0},
    {16384, 0, 0},
    {32753, 83, 0},
    { 78, 86, 0},
    {32612, 50, 0},
    {16384, 0, 0}
},
{
    {27765, 62, 0},
    {32723, 78, 0},
    { 368, 31, 0},
    {32138, 50, 0},
    { 2521, 50, 0},
    {32764, 35, 0},
    { 22, 53, 0},
    {32577, 50, 0},
    {16384, 0, 0}
}
}
}

```

```

Default_Coeff_Base_Idtx_Cdf[ COEFF_CDF_Q_CTXS ][ FSC_TX_SIZE_CONTEXTS ]
                             [ IDTX_SIG_COEF_CONTEXTS ][ NUM_BASE_LEVELS + 3 ] = {
{
{
    {28343, 29890, 30977, 93, 0},
    {20601, 26193, 28764, 93, 0},
    {19490, 23791, 27048, 75, 0},
    {16423, 19493, 22007, 78, 0},
    {12176, 17688, 21070, 1, 0},
    {11254, 15066, 18960, 1, 0},
    { 7135, 9594, 11748, 1, 0}
},
{
    {30897, 31794, 32178, 93, 0},
    {20986, 28222, 30421, 15, 0},
    {18659, 24176, 27934, 0, 0},
    {15633, 18666, 21224, 3, 0},
    { 9693, 15878, 20636, 1, 0},
    {10083, 14089, 18515, 6, 0},
    { 6265, 8352, 10523, 1, 0}
},
{
    {32379, 32564, 32633, 75, 0},
    {22177, 29474, 30765, 1, 0},
    {18538, 23452, 26801, 1, 0},
    {14706, 16590, 18239, 75, 0},
    { 6954, 12300, 15663, 7, 0},
    { 5210, 7282, 11087, 6, 0},
    { 2614, 3781, 4955, 75, 0}
}
},
{
{
    {28035, 30893, 31756, 78, 0},
    {18097, 26957, 29476, 75, 0},
    {15075, 22557, 26893, 0, 0},
    {11632, 16975, 20254, 0, 0},
    { 5659, 12007, 16445, 0, 0},
    { 3855, 7854, 12411, 1, 0},
    { 2360, 4192, 5868, 0, 0}
}
}
}
}
}

```

```

},
{
{31009, 32323, 32574, 78, 0},
{21342, 29678, 31451, 78, 0},
{16940, 25075, 29268, 0, 0},
{12426, 17800, 21533, 0, 0},
{ 6079, 13099, 18572, 1, 0},
{ 4249, 8763, 13561, 6, 0},
{ 2473, 4631, 6923, 5, 0}
},
{
{32079, 32614, 32706, 75, 0},
{22634, 30655, 31942, 0, 0},
{16704, 25079, 29917, 31, 0},
{12795, 17039, 20729, 6, 0},
{ 5362, 11212, 18340, 37, 0},
{ 3513, 6916, 12694, 31, 0},
{ 1883, 3540, 5856, 6, 0}
}
},
{
{
{29171, 31341, 32069, 3, 0},
{19097, 27499, 29866, 75, 0},
{16660, 23539, 28386, 0, 0},
{13020, 18986, 22564, 5, 0},
{ 6762, 13469, 18997, 31, 0},
{ 5230, 9452, 15176, 32, 0},
{ 3282, 5921, 7884, 31, 0}
},
{
{31506, 32472, 32667, 75, 0},
{20403, 29740, 31743, 3, 0},
{14938, 24923, 30084, 0, 0},
{10891, 19070, 24198, 30, 0},
{ 4812, 12877, 20442, 31, 0},
{ 3269, 8035, 14577, 31, 0},
{ 2373, 5157, 8269, 31, 0}
},
{
{32371, 32695, 32745, 75, 0},
{21598, 31049, 32391, 0, 0},
{14515, 26494, 31233, 31, 0},
{11239, 19165, 24574, 31, 0},
{ 4137, 12390, 21147, 32, 0},
{ 2658, 7002, 14001, 31, 0},
{ 2391, 5537, 8704, 26, 0}
}
},
{
{
{31026, 32443, 32748, 25, 0},
{22538, 31251, 32718, 5, 0},
{22228, 26294, 32627, 0, 0},
{20903, 23313, 25862, 38, 0},
{11627, 16067, 32345, 25, 0},
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{31817, 32581, 32735, 25, 0},
{19499, 31019, 32650, 0, 0},
{13910, 24210, 32424, 31, 0},
{12714, 20186, 29593, 62, 0},
{ 4822, 12648, 29509, 32, 0},
{ 2380, 7451, 16774, 75, 0},
{ 3072, 10791, 15675, 50, 0}
},
{
{32485, 32734, 32748, 75, 0},
{19059, 32009, 32725, 0, 0},

```

```

    { 9309, 28748, 32484, 32, 0},
    { 5722, 17292, 29536, 62, 0},
    { 2186, 8637, 26136, 57, 0},
    { 1951, 5879, 15659, 33, 0},
    { 2166, 5695, 11591, 36, 0}
  }
}
}

```

```

Default_Coeff_Br_Idtx_Cdf[ COEFF_CDF_Q_CTXS ][ FSC_TX_SIZE_CONTEXTS ]
[ IDTX_LEVEL_CONTEXTS ][ 4 + 1 ] = {

```

```

{
  {
    {10358, 16536, 21006, 15, 0},
    {10820, 18219, 22881, 91, 0},
    {10100, 15687, 20193, 75, 0},
    {10388, 15552, 19869, 75, 0},
    { 7467, 14671, 18379, 90, 0},
    { 5068, 8607, 12235, 15, 0},
    { 3545, 6569, 9269, 93, 0}
  },
  {
    {10846, 17226, 21443, 0, 0},
    {13398, 20355, 24290, 15, 0},
    {11910, 18796, 23075, 1, 0},
    {12559, 18882, 23077, 76, 0},
    { 8733, 17495, 21561, 90, 0},
    { 4080, 7646, 11249, 5, 0},
    { 3404, 6120, 8842, 75, 0}
  },
  {
    { 7328, 12456, 16707, 91, 0},
    { 8772, 14457, 18914, 2, 0},
    { 8195, 13709, 17968, 2, 0},
    {10966, 15804, 19497, 7, 0},
    { 5374, 13909, 17992, 7, 0},
    { 2314, 4570, 7480, 31, 0},
    { 1645, 3387, 5133, 76, 0}
  },
  {
    {
      {11825, 18784, 22974, 0, 0},
      {10844, 17662, 22290, 0, 0},
      {10616, 16959, 21303, 90, 0},
      {11817, 17622, 21664, 90, 0},
      { 8267, 15623, 19902, 90, 0},
      { 5350, 9801, 14462, 5, 0},
      { 3443, 6585, 9559, 75, 0}
    },
    {
      {13939, 20762, 24451, 31, 0},
      {16243, 23219, 26714, 31, 0},
      {15074, 22210, 25984, 6, 0},
      {16107, 22725, 26296, 6, 0},
      {10893, 20273, 24826, 6, 0},
      { 5645, 10671, 15100, 30, 0},
      { 4335, 8679, 12492, 31, 0}
    },
    {
      {14250, 21163, 24941, 37, 0},
      {17217, 23934, 27433, 32, 0},
      {17090, 23993, 27574, 37, 0},
      {19725, 25425, 28697, 37, 0},
      {12700, 23765, 28251, 37, 0},
      { 6542, 11855, 17108, 37, 0},
      { 5101, 10468, 15443, 37, 0}
    }
  },
  {

```

```

{
  {15586, 23809, 28480, 31, 0},
  {13137, 22338, 27776, 26, 0},
  {12960, 20989, 27007, 1, 0},
  {14487, 22004, 26872, 1, 0},
  { 8977, 19506, 25832, 1, 0},
  { 6689, 13456, 20960, 7, 0},
  { 3660, 8043, 13467, 0, 0}
},
{
  {19250, 25820, 29038, 31, 0},
  {20479, 27597, 30639, 31, 0},
  {19284, 26800, 30037, 26, 0},
  {19120, 26676, 30033, 31, 0},
  {14799, 24819, 29256, 31, 0},
  {10041, 18099, 24102, 31, 0},
  { 6659, 13728, 19983, 31, 0}
},
{
  {18981, 24347, 27397, 32, 0},
  {22083, 28198, 30532, 32, 0},
  {21589, 28346, 30716, 32, 0},
  {22576, 28726, 31113, 32, 0},
  {17332, 27607, 30702, 37, 0},
  {10788, 18280, 24206, 32, 0},
  { 7269, 15134, 21954, 62, 0}
}
},
{
  {
    {32317, 32467, 32618, 62, 0},
    { 8192, 16384, 24576, 0, 0},
    {26624, 28672, 30720, 50, 0},
    {32099, 32322, 32545, 50, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    {31284, 32482, 32636, 62, 0},
    {31915, 32524, 32727, 62, 0},
    {32202, 32681, 32724, 60, 0},
    {32241, 32717, 32748, 60, 0},
    {30064, 32611, 32737, 62, 0},
    {29352, 32394, 32715, 62, 0},
    {21876, 31279, 32303, 60, 0}
  },
  {
    {32083, 32670, 32719, 60, 0},
    {32154, 32631, 32700, 60, 0},
    {31950, 32686, 32727, 60, 0},
    {32416, 32734, 32748, 60, 0},
    {31835, 32703, 32746, 62, 0},
    {30941, 32582, 32737, 60, 0},
    {28688, 32549, 32724, 60, 0}
  }
}
}
}

```

```

Default_Coeff_Base_Bob_Cdf[ COEFF_CDF_Q_CTXS ][ FSC_TX_SIZE_CONTEXTS ]
[ SIG_COEF_CONTEXTS_BOB ][ NUM_BASE_LEVELS + 2 ] = {
{
  {
    { 9917, 17060, 90, 0},
    {13841, 21928, 91, 0},
    {11228, 19107, 90, 0}
  },
  {
    {11194, 18097, 15, 0},
    {13829, 21552, 91, 0},

```



```

    {15625, 22723, 1, 0}
  },
  {
    {10164, 15059, 1, 0},
    {11087, 15990, 0, 0},
    {12184, 18484, 31, 0}
  }
},
{
  {
    {16673, 23002, 75, 0},
    {19890, 25933, 90, 0},
    {18355, 25595, 75, 0}
  },
  {
    {19619, 25865, 0, 0},
    {22421, 27836, 0, 0},
    {22018, 27887, 0, 0}
  },
  {
    {22395, 27670, 31, 0},
    {24275, 28656, 31, 0},
    {22595, 28026, 1, 0}
  }
},
{
  {
    {17507, 24673, 90, 0},
    {19003, 25281, 90, 0},
    {18718, 27145, 0, 0}
  },
  {
    {22679, 29259, 15, 0},
    {24245, 30291, 0, 0},
    {25590, 31051, 0, 0}
  },
  {
    {26844, 31178, 6, 0},
    {28524, 31813, 1, 0},
    {29430, 32097, 1, 0}
  }
},
{
  {
    {20211, 31417, 5, 0},
    {22000, 30309, 5, 0},
    {24619, 32031, 32, 0}
  },
  {
    {25412, 31853, 32, 0},
    {26802, 31283, 32, 0},
    {29204, 32091, 32, 0}
  },
  {
    {30348, 32633, 32, 0},
    {31094, 32686, 40, 0},
    {31665, 32664, 7, 0}
  }
}
}

```

```

Default_Dc_Sign_Cdf[ COEFF_CDF_Q_CTXS ][ PLANE_TYPES ]
    [ DC_SIGN_GROUPS ][ DC_SIGN_CONTEXTS ][ 3 ] = {
  {
    {
      {15831, 93, 0},
      {13632, 75, 0},
      {19041, 75, 0}
    },
  },
}

```

```

{
  {16384, 0, 0},
  {16384, 0, 0},
  {16384, 0, 0}
}
},
{
  {
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0}
  },
  {
    {16384, 0, 0},
    {16384, 0, 0},
    {16384, 0, 0}
  }
}
},
{
  {
    {
      {16540, 93, 0},
      {14804, 93, 0},
      {18312, 93, 0}
    },
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    }
  },
  {
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    },
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    }
  }
}
},
{
  {
    {
      {17575, 75, 0},
      {14224, 78, 0},
      {19801, 93, 0}
    },
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    }
  },
  {
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    },
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    }
  }
}
},

```

```

{
  {
    {
      {20108, 1, 0},
      {14675, 0, 0},
      {20547, 93, 0}
    },
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    }
  },
  {
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    },
    {
      {16384, 0, 0},
      {16384, 0, 0},
      {16384, 0, 0}
    }
  }
}
}

```

Default_Coeff_Base_Ph_Cdf[COEFF_CDF_Q_CTXS][COEFF_BASE_PH_CONTEXTS][4 + 1] =

```

{
  {
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  }
}
}

```

Default_Do_Uneven_4way_Partition_Cdf

[PARTITION_STRUCTURE_NUM][PARTITION_CONTEXTS][2 + 1] = {

```

{
  {16384, 0, 0},
  {16384, 0, 0},
  {16384, 0, 0},
  {16384, 0, 0},
  {16384, 0, 0},
  {16384, 0, 0},

```

$$\left. \begin{array}{l} \} , \\ \{ \end{array} \right\}$$

}

}

{


```

    { 2632, 8256, 13389, 18349, 22057, 93, 0},
    { 4, 5539, 10753, 15541, 19671, 90, 0}
},
{
    { 1647, 4981, 8018, 10713, 12930, 78, 0},
    { 133, 2595, 5758, 9172, 12046, 78, 0}
},
{
    {17458, 29871, 32000, 32546, 32702, 93, 0},
    { 6925, 27088, 31750, 32368, 32706, 93, 0}
},
{
    {10512, 24503, 29646, 31529, 32218, 93, 0},
    { 862, 17944, 27174, 30714, 32074, 90, 0}
},
{
    { 6509, 17436, 24062, 28298, 30439, 93, 0},
    { 730, 11602, 20450, 26039, 29031, 90, 0}
},
{
    { 4334, 12843, 19639, 24807, 27809, 90, 0},
    { 1182, 8304, 16062, 22058, 26408, 93, 0}
},
{
    { 2763, 7942, 12551, 16873, 20575, 93, 0},
    { 588, 5113, 10255, 14725, 18439, 93, 0}
},
{
    {25261, 30274, 31401, 31859, 32108, 5, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {11341, 22787, 27330, 29329, 30402, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 7870, 16597, 22314, 26036, 27646, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 9102, 15212, 19596, 22630, 24714, 5, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 2627, 7584, 12537, 16148, 19228, 78, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 1773, 5159, 8384, 11582, 14326, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 1281, 3306, 5564, 7730, 10025, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {28275, 32087, 32554, 32679, 32732, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {13124, 24281, 29249, 31039, 31932, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 7252, 15503, 21990, 26021, 28329, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 6766, 13251, 18520, 22234, 24944, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},

```

```

{
  { 2022, 6282, 10559, 13831, 16913, 93, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {
    { 8794, 30831, 31512, 32035, 32245, 60, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 4155, 20109, 27969, 30536, 32143, 0, 0},
    { 4, 10300, 22300, 28132, 30839, 75, 0}
  },
  {
    {13302, 23761, 27348, 29952, 31476, 1, 0},
    { 4, 14590, 24086, 28071, 29718, 1, 0}
  },
  {
    { 6132, 14425, 20760, 25122, 27553, 76, 0},
    { 4, 7592, 16048, 21694, 25655, 81, 0}
  },
  {
    { 3376, 10282, 14415, 19084, 23156, 1, 0},
    { 4, 5986, 12540, 17467, 21046, 75, 0}
  },
  {
    { 2339, 7935, 12252, 15518, 18022, 75, 0},
    { 4, 4807, 8560, 13742, 17846, 93, 0}
  },
  {
    { 1008, 5061, 9267, 11938, 15655, 91, 0},
    { 2687, 2712, 6941, 10360, 13721, 90, 0}
  },
  {
    { 1067, 5070, 8639, 11287, 13789, 75, 0},
    { 4, 3968, 5983, 9072, 11954, 93, 0}
  },
  {
    { 513, 3188, 4940, 6642, 8326, 93, 0},
    { 1598, 1604, 3335, 5390, 7186, 93, 0}
  },
  {
    {18364, 30931, 32563, 32686, 32732, 78, 0},
    {20105, 28028, 32148, 32696, 32722, 76, 0}
  },
  {
    {14335, 27700, 31502, 32326, 32673, 18, 0},
    { 4, 23820, 30576, 32277, 32675, 75, 0}
  },
  {
    {13107, 25752, 29932, 31638, 32381, 75, 0},
    { 4, 19582, 27260, 30379, 31695, 90, 0}
  },
  {
    { 6458, 17966, 24490, 28327, 30455, 93, 0},
    { 4, 12996, 21614, 26979, 29871, 78, 0}
  },
  {
    { 4362, 12372, 18835, 23784, 27252, 93, 0},
    { 4, 8398, 15799, 21724, 25631, 90, 0}
  },
  {
    { 3430, 10462, 16282, 20919, 24340, 75, 0},
    { 30, 6110, 12299, 17850, 22504, 75, 0}
  },
  {
    { 1926, 5825, 9243, 12462, 15052, 78, 0},
    { 72, 3367, 7142, 10595, 13743, 78, 0}
  },
  {

```



```

    {24383, 32129, 32637, 32713, 32732, 93, 0},
    { 9137, 30561, 32521, 32699, 32732, 90, 0}
},
{
    {17270, 29740, 32008, 32483, 32655, 93, 0},
    { 8, 25977, 31428, 32465, 32657, 75, 0}
},
{
    {12268, 25877, 30422, 31961, 32485, 90, 0},
    { 12, 20569, 28767, 31457, 32359, 90, 0}
},
{
    { 7294, 18995, 26081, 29672, 31452, 93, 0},
    { 181, 14139, 23494, 28458, 30918, 93, 0}
},
{
    { 3440, 9808, 15386, 19904, 23390, 93, 0},
    { 139, 6455, 12703, 17797, 21832, 90, 0}
},
{
    {28752, 31869, 32320, 32478, 32572, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {15670, 27564, 30368, 31493, 31993, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 9799, 20178, 25558, 28413, 30046, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 9356, 16500, 21207, 24352, 26419, 6, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 2981, 9131, 14182, 18343, 21454, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 2154, 6775, 11531, 15084, 18264, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 1257, 3940, 6890, 9699, 12030, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {30668, 32581, 32693, 32736, 32740, 5, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {18504, 29402, 31702, 32283, 32519, 78, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {10766, 22532, 27978, 30115, 31262, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 8719, 18462, 24428, 27561, 29580, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 3879, 10481, 15972, 19633, 22761, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
    {
        {18644, 30427, 32255, 32544, 32664, 37, 0},
    }
}

```

```

    { 5461, 10923, 16384, 21845, 27307,    0, 0}
  },
  {
    { 4549, 25234, 29518, 31595, 31998,    1, 0},
    {    4, 12584, 25285, 29517, 31566,   76, 0}
  },
  {
    {12259, 24561, 28578, 30638, 31755,   80, 0},
    {    4, 18211, 26628, 30288, 31028,   25, 0}
  },
  {
    { 5548, 13583, 19991, 25062, 27790,   15, 0},
    {    4,  8981, 17065, 22847, 26409,   75, 0}
  },
  {
    { 2511,  8771, 13709, 18515, 21631,   90, 0},
    {    4,  6091, 12402, 16401, 21384,   75, 0}
  },
  {
    { 1456,  6940, 12018, 15507, 18576,   15, 0},
    { 3828,  4004,  7860, 12305, 16184,   91, 0}
  },
  {
    { 1456,  5488,  8386, 12251, 14583,   90, 0},
    {    5,  3667,  7540, 10600, 13938,   75, 0}
  },
  {
    { 2172,  4278,  6935, 10386, 13380,   93, 0},
    {    6,  2661,  6993,  9316, 12044,   90, 0}
  },
  {
    {  854,  2756,  4422,  6402,  8143,   75, 0},
    {    4,  1978,  3940,  6072,  7460,   93, 0}
  },
  {
    {27597, 32474, 32665, 32730, 32734,   16, 0},
    {24628, 29500, 31920, 32665, 32732,    2, 0}
  },
  {
    {16858, 28844, 31500, 32377, 32690,   90, 0},
    {    4, 27050, 31611, 32598, 32708,    1, 0}
  },
  {
    {17412, 28372, 31092, 32127, 32538,   75, 0},
    {    4, 23035, 29401, 31561, 32362,    1, 0}
  },
  {
    { 6754, 18144, 24912, 28603, 30596,   75, 0},
    {    4, 15859, 24392, 28503, 30570,    0, 0}
  },
  {
    { 4015, 12511, 19603, 24562, 27900,   80, 0},
    {    4,  9194, 17016, 22337, 26137,   75, 0}
  },
  {
    { 3458, 10380, 16144, 21113, 24683,   90, 0},
    {    5,  6767, 13223, 18657, 22519,   15, 0}
  },
  {
    { 1682,  5432,  8769, 12173, 14844,   93, 0},
    {    7,  3903,  7423, 10655, 13533,   93, 0}
  },
  {
    {28336, 32620, 32715, 32736, 32740,   80, 0},
    { 9058, 31414, 32677, 32736, 32740,    0, 0}
  },
  {
    {23033, 31648, 32521, 32700, 32732,   75, 0},
    {    75, 29218, 32255, 32646, 32732,    0, 0}
  },
  {

```

```

    {17992, 29745, 32048, 32579, 32724, 75, 0},
    { 95, 26324, 31408, 32426, 32663, 90, 0}
},
{
    { 9671, 22972, 28881, 31303, 32233, 93, 0},
    { 132, 18991, 27475, 30900, 32140, 75, 0}
},
{
    { 3245, 10146, 15682, 20055, 23306, 90, 0},
    { 715, 6956, 13155, 18158, 21881, 78, 0}
},
{
    {30282, 32463, 32643, 32689, 32720, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {18363, 29970, 31740, 32329, 32529, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {11103, 21929, 26883, 29338, 30615, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {10010, 15981, 20125, 22717, 24545, 26, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 2197, 6613, 12544, 16341, 19413, 32, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 977, 5129, 8874, 12334, 16079, 15, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 1326, 3745, 5773, 7906, 10325, 35, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {31687, 32702, 32738, 32742, 32746, 5, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {23101, 31425, 32294, 32485, 32560, 76, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {14043, 26486, 29937, 30917, 31636, 6, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {11590, 21553, 26089, 28623, 29781, 32, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 3345, 9687, 15060, 19237, 21946, 33, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
    {
        {17981, 30263, 31343, 31835, 32228, 52, 0},
        { 5461, 10923, 16384, 21845, 27307, 0, 0}
    },
    {
        { 541, 19909, 25694, 31571, 32707, 56, 0},
        { 5, 3008, 18509, 26661, 30500, 6, 0}
    },
    {
        { 6280, 16644, 22197, 27386, 30311, 31, 0},
    }
}

```

```

    { 3289, 9702, 18719, 20867, 24166, 42, 0}
},
{
    { 2433, 8354, 15330, 20115, 23238, 2, 0},
    { 21, 9300, 13157, 16064, 22107, 7, 0}
},
{
    { 3059, 5529, 10689, 14670, 16660, 87, 0},
    { 36, 4415, 9690, 12203, 15756, 10, 0}
},
{
    { 2528, 7188, 10559, 12046, 15715, 90, 0},
    { 228, 1917, 4701, 8762, 12870, 17, 0}
},
{
    { 1318, 4984, 6874, 11801, 12832, 85, 0},
    { 772, 1543, 4630, 8014, 10032, 90, 0}
},
{
    { 1741, 4692, 6205, 8476, 10822, 85, 0},
    { 287, 1649, 4446, 6812, 8891, 10, 0}
},
{
    { 819, 2068, 3940, 5617, 6164, 93, 0},
    { 20, 1444, 2749, 3441, 5300, 92, 0}
},
{
    {22627, 30107, 31954, 32695, 32732, 6, 0},
    { 9634, 23470, 31648, 32544, 32656, 40, 0}
},
{
    {14808, 27300, 31048, 32355, 32547, 31, 0},
    { 4, 26706, 31394, 32590, 32717, 16, 0}
},
{
    {14111, 25840, 29676, 31196, 32100, 2, 0},
    { 2240, 18421, 26730, 29569, 30965, 5, 0}
},
{
    { 3320, 16652, 23243, 27800, 29610, 6, 0},
    { 3241, 11944, 18304, 22700, 26752, 3, 0}
},
{
    { 2775, 10402, 16795, 21420, 24155, 82, 0},
    { 42, 7547, 13763, 18308, 22325, 83, 0}
},
{
    { 1526, 9483, 15296, 20232, 22733, 7, 0},
    { 3770, 6135, 11174, 17138, 20531, 93, 0}
},
{
    { 1566, 4208, 7493, 9981, 11307, 75, 0},
    { 10, 2707, 5347, 8016, 11209, 91, 0}
},
{
    {29051, 32611, 32732, 32736, 32740, 81, 0},
    {11740, 31606, 32673, 32736, 32740, 15, 0}
},
{
    {23082, 31405, 32517, 32736, 32740, 15, 0},
    { 1955, 29285, 32394, 32685, 32732, 15, 0}
},
{
    {16727, 28474, 31461, 32407, 32710, 75, 0},
    { 1043, 24584, 30321, 31899, 32456, 75, 0}
},
{
    { 7685, 20689, 27553, 30514, 31627, 90, 0},
    { 312, 15641, 24900, 29282, 31344, 78, 0}
},
{

```

[illegible]

```

{ 356, 534, 890, 1603, 7480, 50, 0}
},
{
{ 426, 2128, 4256, 5532, 6383, 50, 0},
{ 226, 452, 1130, 3390, 4520, 50, 0}
},
{
{ 936, 2185, 3433, 4057, 5305, 0, 0},
{ 213, 426, 851, 1702, 1915, 50, 0}
},
{
{ 1130, 6780, 11299, 13559, 15819, 0, 0},
{ 840, 1260, 1680, 2521, 3361, 50, 0}
},
{
{ 840, 2521, 2941, 3781, 6722, 75, 0},
{ 607, 910, 1820, 2427, 2731, 25, 0}
},
{
{ 2760, 9884, 19144, 26446, 28761, 75, 0},
{ 6378, 8797, 14295, 25731, 30349, 50, 0}
},
{
{ 1022, 5182, 10071, 14523, 20142, 10, 0},
{ 1625, 5578, 14671, 19283, 23851, 40, 0}
},
{
{ 1493, 14043, 16621, 19742, 20896, 40, 0},
{ 79, 5792, 10632, 14519, 17217, 75, 0}
},
{
{ 1896, 5958, 11645, 14353, 16249, 0, 0},
{ 973, 3244, 6164, 8111, 11031, 0, 0}
},
{
{ 2777, 4999, 8331, 10552, 13329, 75, 0},
{ 799, 2398, 5195, 7593, 11189, 25, 0}
},
{
{ 1771, 5314, 8856, 12399, 15056, 75, 0},
{ 2048, 6144, 14336, 16384, 18432, 50, 0}
},
{
{ 509, 3735, 7980, 10017, 11715, 75, 0},
{ 537, 2149, 5551, 6088, 8416, 75, 0}
},
{
{24857, 32089, 32690, 32716, 32732, 0, 0},
{10411, 28445, 32559, 32722, 32732, 18, 0}
},
{
{15845, 30289, 32552, 32660, 32732, 40, 0},
{ 7773, 29346, 31862, 32519, 32732, 40, 0}
},
{
{ 9766, 24287, 28720, 30969, 32382, 82, 0},
{ 585, 20320, 29044, 32023, 32449, 28, 0}
},
{
{ 7627, 18644, 24717, 29519, 31779, 0, 0},
{ 2718, 15100, 26426, 29748, 31560, 0, 0}
},
{
{ 2451, 8257, 11869, 16556, 19523, 12, 0},
{ 256, 4651, 10581, 13824, 16341, 92, 0}
},
{
{ 5461, 10923, 16384, 21845, 27307, 0, 0},
{ 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{

```



```

{
  { 1614, 5662, 8479, 11366, 13422, 18, 0},
  { 4, 1770, 4011, 7227, 9349, 17, 0}
},
{
  { 1307, 3944, 6304, 9227, 11645, 83, 0},
  { 4, 2665, 4779, 7323, 9602, 93, 0}
},
{
  { 1058, 2942, 4397, 5803, 6862, 78, 0},
  { 8, 1344, 2527, 4536, 6155, 78, 0}
},
{
  {13961, 27727, 31548, 32432, 32640, 30, 0},
  { 2708, 23637, 31303, 32335, 32585, 1, 0}
},
{
  { 8468, 21640, 27998, 30901, 32015, 90, 0},
  { 4, 16662, 26837, 30837, 32112, 90, 0}
},
{
  { 6388, 17049, 24419, 28383, 30593, 90, 0},
  { 4, 12537, 21264, 26680, 30075, 5, 0}
},
{
  { 4692, 13240, 20421, 25309, 28482, 78, 0},
  { 4, 9230, 16499, 22332, 26503, 0, 0}
},
{
  { 3442, 10509, 16259, 21231, 24617, 0, 0},
  { 4, 6258, 12848, 18162, 22620, 90, 0}
},
{
  { 2789, 8106, 13273, 17690, 21420, 78, 0},
  { 4, 5418, 10448, 15448, 19662, 83, 0}
},
{
  { 1801, 4801, 7786, 10991, 13154, 93, 0},
  { 25, 3162, 6409, 9255, 12072, 3, 0}
},
{
  {18092, 29844, 32040, 32571, 32719, 75, 0},
  { 9164, 26106, 31175, 32321, 32602, 78, 0}
},
{
  {11343, 24700, 29814, 31603, 32215, 93, 0},
  { 22, 18800, 27925, 31139, 32203, 90, 0}
},
{
  { 6970, 17708, 24782, 28661, 30778, 93, 0},
  { 38, 12260, 21656, 27173, 30241, 75, 0}
},
{
  { 4769, 13169, 19793, 24809, 28254, 78, 0},
  { 145, 8816, 16919, 22877, 26986, 78, 0}
},
{
  { 2805, 8155, 12943, 17359, 21110, 78, 0},
  { 566, 5051, 10431, 15290, 19445, 78, 0}
},
{
  {21153, 29831, 31499, 32049, 32276, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {11827, 24672, 29069, 30619, 31334, 93, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 6970, 16794, 23049, 26961, 28915, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}

```



```

},
{
  { 6107, 13557, 19154, 23110, 25619, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 2702, 7985, 12910, 17340, 20552, 78, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 1993, 5600, 9439, 13043, 16257, 90, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 1269, 3987, 6708, 9519, 11780, 93, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {26630, 32076, 32532, 32672, 32731, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {13939, 25333, 29988, 31536, 32142, 90, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 6772, 15540, 22058, 26174, 28801, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5996, 13722, 19150, 22923, 25872, 78, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 2372, 6802, 11003, 14855, 18017, 78, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
  {
    {17303, 31326, 31526, 32376, 32710, 11, 0},
    { 260, 16384, 22105, 27307, 28867, 68, 0}
  },
  {
    { 1845, 17089, 26116, 30050, 31620, 75, 0},
    { 4, 7809, 21186, 27970, 30787, 75, 0}
  },
  {
    { 9621, 20802, 26452, 29388, 31088, 0, 0},
    { 4, 15476, 24099, 28279, 30173, 1, 0}
  },
  {
    { 5542, 13743, 19899, 24736, 27946, 75, 0},
    { 4, 8467, 16948, 22772, 26471, 0, 0}
  },
  {
    { 3700, 10363, 15560, 20281, 23726, 75, 0},
    { 4, 6038, 11885, 17198, 21363, 15, 0}
  },
  {
    { 2418, 7197, 12060, 15605, 19277, 93, 0},
    { 4, 4658, 9217, 13468, 17379, 78, 0}
  },
  {
    { 1804, 6314, 9128, 12718, 15539, 91, 0},
    { 1372, 1648, 6025, 9656, 13223, 78, 0}
  },
  {
    { 1735, 4846, 7989, 10786, 13487, 78, 0},
    { 4, 3323, 6453, 9627, 12335, 93, 0}
  }
},

```

```

{
  { 1072, 3013, 4878, 6783, 8518, 93, 0},
  { 10, 1926, 3795, 5732, 7600, 93, 0}
},
{
  {18024, 30489, 32251, 32626, 32732, 90, 0},
  {19896, 24724, 31657, 32534, 32732, 5, 0}
},
{
  {11660, 26630, 30824, 32187, 32598, 75, 0},
  { 4, 21429, 29602, 31983, 32544, 0, 0}
},
{
  {12486, 24964, 29580, 31503, 32248, 15, 0},
  { 4, 19295, 27630, 30822, 31918, 1, 0}
},
{
  { 6927, 17619, 24441, 28334, 30439, 78, 0},
  { 4, 12994, 22001, 27172, 29927, 75, 0}
},
{
  { 4541, 12600, 19151, 24114, 27363, 93, 0},
  { 4, 8631, 16251, 21921, 25807, 0, 0}
},
{
  { 3464, 9852, 15685, 20588, 24106, 78, 0},
  { 4, 6580, 12812, 18079, 22278, 0, 0}
},
{
  { 2085, 5889, 9302, 12624, 15262, 93, 0},
  { 69, 3305, 6891, 10477, 13651, 78, 0}
},
{
  {23748, 31870, 32594, 32721, 32732, 90, 0},
  {10783, 29998, 32378, 32682, 32732, 78, 0}
},
{
  {17733, 30083, 32212, 32605, 32725, 75, 0},
  { 187, 25993, 31535, 32484, 32683, 75, 0}
},
{
  {12827, 26254, 30707, 32112, 32556, 75, 0},
  { 92, 21424, 29165, 31621, 32404, 75, 0}
},
{
  { 7651, 19635, 26664, 30009, 31616, 78, 0},
  { 27, 14981, 24532, 29277, 31387, 75, 0}
},
{
  { 3603, 10277, 15911, 20395, 23870, 78, 0},
  { 350, 6796, 13207, 18418, 22380, 3, 0}
},
{
  {28587, 32126, 32526, 32643, 32695, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {16637, 28727, 31322, 32097, 32381, 90, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 9765, 21318, 26832, 29587, 30810, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 6949, 15290, 21217, 24889, 27171, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 3604, 10158, 15795, 20231, 23348, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}

```

```

},
{
  { 2312, 7277, 11902, 16059, 19358, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 1616, 4713, 7922, 10951, 13675, 93, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {30272, 32591, 32703, 32736, 32740, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {19778, 30226, 32065, 32498, 32646, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {11082, 23345, 28543, 30633, 31593, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 7718, 18010, 24057, 27609, 29598, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 3560, 10218, 15775, 20026, 23024, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
  {
    {22472, 31406, 32415, 32617, 32686, 37, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 4579, 20789, 27200, 30357, 31565, 0, 0},
    { 4, 12804, 24534, 29241, 31305, 7, 0}
  },
  {
    {10553, 22962, 27982, 30214, 31394, 1, 0},
    { 4, 17230, 25117, 28842, 30403, 0, 0}
  },
  {
    { 5244, 13272, 19764, 24354, 27332, 15, 0},
    { 4, 8510, 16126, 21941, 25809, 0, 0}
  },
  {
    { 3263, 9944, 15196, 19855, 22889, 78, 0},
    { 4, 6173, 11683, 16946, 21240, 75, 0}
  },
  {
    { 1953, 7145, 11524, 15201, 18227, 5, 0},
    { 290, 4454, 9436, 13541, 17364, 75, 0}
  },
  {
    { 1722, 5876, 9313, 13099, 15693, 75, 0},
    { 4, 3730, 7620, 11131, 14444, 75, 0}
  },
  {
    { 1665, 4813, 7575, 10457, 12877, 0, 0},
    { 4, 3095, 6224, 9491, 12462, 90, 0}
  },
  {
    { 1050, 3459, 5369, 7424, 9155, 93, 0},
    { 125, 1583, 3405, 5421, 7274, 15, 0}
  },
  {
    {25676, 31984, 32605, 32712, 32732, 75, 0},
    {22804, 26009, 31701, 32573, 32732, 0, 0}
  }
},

```

```

{
  {13894, 27451, 31145, 32228, 32639, 15, 0},
  { 4, 23912, 30395, 32138, 32610, 75, 0}
},
{
  {14722, 26873, 30554, 31901, 32382, 81, 0},
  { 4, 19859, 27062, 30489, 31792, 6, 0}
},
{
  { 6399, 16967, 23871, 27811, 30028, 78, 0},
  { 4, 13114, 21748, 26707, 29694, 0, 0}
},
{
  { 4205, 12741, 19447, 23962, 26806, 76, 0},
  { 4, 8506, 15656, 21414, 25696, 78, 0}
},
{
  { 3566, 10323, 16256, 20824, 24096, 75, 0},
  { 4, 6984, 13002, 18282, 22495, 90, 0}
},
{
  { 1775, 5940, 9908, 13605, 16182, 0, 0},
  { 33, 4084, 7837, 11117, 14306, 80, 0}
},
{
  {26506, 32458, 32715, 32736, 32740, 90, 0},
  { 8374, 30740, 32600, 32729, 32733, 93, 0}
},
{
  {20516, 31118, 32508, 32699, 32732, 75, 0},
  { 9, 27963, 32073, 32637, 32732, 75, 0}
},
{
  {15612, 28327, 31610, 32454, 32687, 80, 0},
  { 26, 24066, 30443, 32138, 32600, 80, 0}
},
{
  { 8459, 21222, 27807, 30732, 31943, 78, 0},
  { 26, 16663, 26041, 30121, 31822, 75, 0}
},
{
  { 4168, 12043, 18257, 22879, 26034, 75, 0},
  { 44, 8409, 15622, 20993, 24796, 0, 0}
},
{
  {30650, 32519, 32673, 32723, 32732, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {17529, 29628, 31828, 32359, 32545, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {10542, 21115, 26318, 29161, 30805, 7, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {10247, 15768, 19711, 22296, 24028, 31, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 2397, 7449, 12493, 16590, 19724, 56, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 1869, 5577, 9489, 12835, 15631, 56, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 1228, 3968, 6037, 8766, 10801, 55, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}

```

```

},
{
  {31955, 32716, 32740, 32744, 32748, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {22693, 31709, 32467, 32593, 32659, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {14735, 26877, 30393, 31770, 32325, 1, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 9390, 20635, 25906, 28733, 30212, 6, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 4403, 11798, 18101, 22593, 25468, 5, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
  {
    {17498, 29424, 31225, 31893, 32132, 62, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 4465, 19403, 25248, 29392, 31099, 7, 0},
    {   4, 11650, 23783, 29177, 30996, 7, 0}
  },
  {
    { 9152, 21002, 26411, 29197, 30838, 6, 0},
    { 387, 15169, 23639, 27517, 29225, 7, 0}
  },
  {
    { 4263, 12796, 18125, 23216, 25968, 76, 0},
    {   4, 8811, 16017, 21391, 24903, 1, 0}
  },
  {
    { 2963, 7692, 13120, 17437, 20742, 75, 0},
    {   4, 5759, 11337, 16128, 19807, 78, 0}
  },
  {
    { 2381, 7283, 11060, 13175, 15657, 78, 0},
    { 30, 3641, 7204, 11473, 15621, 76, 0}
  },
  {
    { 1314, 5852, 7870, 11057, 13558, 75, 0},
    { 1846, 2126, 5191, 9398, 12426, 75, 0}
  },
  {
    { 1838, 5614, 8157, 11023, 13394, 93, 0},
    { 711, 875, 4561, 6006, 8631, 31, 0}
  },
  {
    { 1166, 3234, 5698, 8017, 9616, 93, 0},
    { 25, 1054, 2149, 3082, 4062, 0, 0}
  },
  {
    {26289, 31974, 32589, 32707, 32732, 6, 0},
    {15601, 27297, 32026, 32579, 32732, 30, 0}
  },
  {
    {14722, 28032, 31407, 32465, 32660, 92, 0},
    { 4, 25486, 30863, 32285, 32652, 81, 0}
  },
  {
    {14143, 25751, 29446, 31254, 32093, 75, 0},
    { 54, 19128, 26757, 29947, 31382, 1, 0}
  },
}

```



```

},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
  {
    {14193, 27782, 31321, 32004, 32607, 50, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 1210, 13319, 22982, 27380, 30780, 37, 0},
    {   4, 7290, 20938, 26753, 30532, 32, 0}
  },
  {
    { 6129, 19017, 24319, 27326, 29222, 32, 0},
    { 108, 12264, 17594, 22318, 25161, 57, 0}
  },
  {
    { 3672, 16362, 17673, 20077, 22838, 26, 0},
    { 1389, 4066, 11635, 17622, 22074, 7, 0}
  },
  {
    { 2395, 9381, 12019, 14390, 17169, 40, 0},
    { 123, 3545, 5521, 12207, 15924, 33, 0}
  },
  {
    { 2084, 5758, 9241, 11873, 14327, 93, 0},
    { 14, 4141, 8239, 10165, 13119, 93, 0}
  },
  {
    { 1510, 4925, 7765, 10264, 12295, 18, 0},
    { 424, 2326, 5212, 9049, 12105, 93, 0}
  },
  {
    { 1310, 4491, 6944, 9377, 11560, 17, 0},
    { 796, 2092, 4230, 6299, 9460, 18, 0}
  },
  {
    { 575, 2082, 3357, 4723, 5504, 93, 0},
    { 407, 1183, 2575, 3784, 5063, 90, 0}
  },
  {
    {21859, 29753, 31615, 32585, 32702, 31, 0},
    { 3511, 29157, 31832, 32567, 32701, 65, 0}
  },
  {
    {13469, 26644, 30586, 31697, 32163, 27, 0},
    { 43, 24491, 30511, 31649, 32156, 32, 0}
  },
  {
    {11517, 23927, 28667, 30593, 31651, 1, 0},
    { 25, 17446, 24435, 28271, 30325, 6, 0}
  },
  {
    { 6058, 16232, 22050, 26218, 27912, 1, 0},
    { 168, 8904, 17397, 23318, 27175, 75, 0}
  },
  {
    { 3216, 9575, 15671, 19813, 23338, 76, 0},
    { 65, 6797, 12411, 18393, 22224, 1, 0}
  },
},

```



```

    },
    {
      { 5461, 10923, 16384, 21845, 27307, 0, 0},
      { 5461, 10923, 16384, 21845, 27307, 0, 0}
    }
  },
  {
    {
      { 3361, 19325, 24366, 31088, 31928, 50, 0},
      { 5461, 10923, 16384, 21845, 27307, 0, 0}
    },
    {
      { 1632, 15337, 24150, 28503, 30563, 2, 0},
      { 4, 5295, 20829, 28126, 30850, 0, 0}
    },
    {
      { 6632, 16063, 23298, 27040, 29721, 3, 0},
      { 4, 11374, 19507, 24923, 28568, 25, 0}
    },
    {
      { 4909, 11090, 17838, 22385, 26279, 15, 0},
      { 35, 6017, 15862, 22480, 26459, 0, 0}
    },
    {
      { 3589, 9811, 14531, 19054, 22695, 3, 0},
      { 4, 6364, 9855, 16927, 21591, 0, 0}
    },
    {
      { 2743, 7444, 12418, 15058, 19040, 0, 0},
      { 6, 2870, 8495, 12527, 16693, 18, 0}
    },
    {
      { 1596, 6378, 10210, 14940, 17169, 93, 0},
      { 4, 1626, 3090, 7519, 9718, 18, 0}
    },
    {
      { 648, 4541, 7588, 11572, 14642, 5, 0},
      { 5, 3893, 6075, 9405, 12116, 18, 0}
    },
    {
      { 1519, 3450, 5055, 6173, 7170, 8, 0},
      { 5, 2567, 3706, 7533, 9113, 90, 0}
    },
    {
      { 14288, 28805, 31683, 32583, 32700, 78, 0},
      { 64, 24516, 31405, 32371, 32655, 6, 0}
    },
    {
      { 8353, 22447, 29049, 31579, 32315, 78, 0},
      { 4, 16982, 27998, 31526, 32421, 0, 0}
    },
    {
      { 7498, 18802, 25718, 29605, 31419, 75, 0},
      { 4, 13992, 23750, 28392, 31093, 0, 0}
    },
    {
      { 5931, 15139, 22182, 27418, 29974, 78, 0},
      { 4, 12206, 21157, 26583, 29002, 0, 0}
    },
    {
      { 4417, 12408, 19565, 24937, 27743, 0, 0},
      { 6, 8793, 17130, 22092, 26855, 1, 0}
    },
    {
      { 3697, 10293, 16641, 21345, 25242, 78, 0},
      { 4, 7551, 14182, 20667, 25647, 18, 0}
    },
    {
      { 3788, 10092, 15480, 20531, 23568, 76, 0},

```

```

    { 30, 4783, 10714, 16396, 20863, 0, 0}
  },
  {
    {18808, 30564, 32414, 32708, 32732, 93, 0},
    { 8341, 26149, 31856, 32628, 32726, 3, 0}
  },
  {
    {13189, 27414, 31625, 32526, 32712, 90, 0},
    { 95, 21665, 30252, 32221, 32614, 3, 0}
  },
  {
    { 9837, 22542, 28890, 31327, 32298, 78, 0},
    { 6, 16453, 26512, 30728, 32308, 75, 0}
  },
  {
    { 7017, 18407, 25212, 29652, 31622, 78, 0},
    { 31, 11962, 22617, 28211, 31109, 75, 0}
  },
  {
    { 5446, 14294, 21338, 26300, 29677, 78, 0},
    { 156, 8220, 17040, 23841, 28514, 78, 0}
  },
  {
    {24803, 31286, 32079, 32381, 32540, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    {12821, 26214, 30050, 31373, 31972, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 7896, 17264, 23904, 27850, 29752, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 7045, 14770, 19995, 23659, 26108, 78, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 2310, 6418, 12150, 16868, 20631, 78, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 1672, 5533, 10030, 13735, 17567, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 1219, 4524, 7028, 10416, 13388, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    {29688, 32425, 32651, 32731, 32735, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    {15856, 26584, 30786, 32053, 32570, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 8823, 17890, 24454, 28342, 30615, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 7001, 15331, 21333, 25485, 28372, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 3542, 10300, 15274, 19865, 23271, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  }
},

```

```

{
  {
    {20460, 32339, 32513, 32662, 32732, 13, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 283, 17174, 26580, 30375, 31638, 75, 0},
    { 4, 5576, 21745, 28213, 30821, 75, 0}
  },
  {
    {11048, 22170, 26485, 29197, 30921, 0, 0},
    { 4, 16199, 25093, 29139, 30504, 0, 0}
  },
  {
    { 6377, 13511, 19779, 25102, 28132, 0, 0},
    { 4, 7377, 18122, 23594, 27200, 78, 0}
  },
  {
    { 4500, 10287, 15895, 21180, 24672, 0, 0},
    { 16, 5691, 12262, 17802, 21695, 0, 0}
  },
  {
    { 2790, 7843, 12604, 16931, 20564, 93, 0},
    { 4, 4821, 9847, 13980, 18278, 91, 0}
  },
  {
    { 1381, 7439, 10964, 14802, 18073, 76, 0},
    { 1522, 2450, 6619, 10499, 14157, 16, 0}
  },
  {
    { 2025, 5643, 8935, 12150, 14843, 90, 0},
    { 224, 2917, 6661, 10512, 13337, 90, 0}
  },
  {
    { 1368, 3825, 6654, 9038, 11091, 90, 0},
    { 41, 2582, 5312, 7720, 10479, 90, 0}
  },
  {
    {20410, 31676, 32514, 32692, 32732, 80, 0},
    {25631, 26346, 32305, 32649, 32732, 6, 0}
  },
  {
    { 9017, 26371, 30820, 32288, 32624, 75, 0},
    { 4, 20642, 29893, 32138, 32626, 0, 0}
  },
  {
    {11957, 24371, 29434, 31605, 32329, 90, 0},
    { 4, 19715, 28037, 30945, 32081, 1, 0}
  },
  {
    { 6587, 17392, 24549, 28765, 30910, 75, 0},
    { 4, 13017, 22846, 28010, 30563, 75, 0}
  },
  {
    { 4820, 13076, 20235, 25274, 28367, 90, 0},
    { 4, 9517, 18022, 23590, 27505, 75, 0}
  },
  {
    { 3839, 11042, 17152, 22327, 25816, 93, 0},
    { 4, 7434, 14558, 20495, 24720, 90, 0}
  },
  {
    { 3178, 9048, 14018, 18246, 21456, 93, 0},
    { 41, 4587, 9592, 14488, 18984, 75, 0}
  },
  {
    {24602, 32389, 32700, 32736, 32740, 75, 0},
    { 7626, 30290, 32589, 32730, 32734, 75, 0}
  },
  {
    {17206, 30451, 32404, 32696, 32732, 90, 0},

```

```

    { 4, 26159, 31880, 32631, 32732, 75, 0}
},
{
    {12799, 26717, 31223, 32384, 32678, 75, 0},
    { 4, 21576, 29603, 32028, 32595, 0, 0}
},
{
    { 7923, 20676, 27760, 30909, 32092, 78, 0},
    { 10, 15320, 25351, 29888, 31811, 78, 0}
},
{
    { 4552, 13193, 20089, 24958, 28270, 78, 0},
    { 29, 9104, 17079, 23044, 27203, 78, 0}
},
{
    {29032, 32327, 32595, 32682, 32720, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {16965, 29286, 31651, 32307, 32527, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 9346, 21389, 27317, 29974, 31327, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 6585, 15649, 21693, 25674, 28044, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 3417, 10093, 16159, 21188, 24606, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 2304, 7456, 12293, 16563, 20267, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 1811, 5284, 8853, 12277, 15769, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {30975, 32679, 32738, 32742, 32746, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {20345, 30725, 32278, 32643, 32726, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {12243, 25011, 29788, 31583, 32260, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 8494, 19928, 26600, 29501, 31142, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 4992, 13924, 20358, 24637, 27524, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
    {
        {24594, 32389, 32641, 32718, 32732, 37, 0},
        { 5461, 10923, 16384, 21845, 27307, 0, 0}
    },
    {
        { 2358, 19217, 26997, 30649, 31793, 1, 0},
        { 4, 10990, 24500, 28869, 31218, 5, 0}
    }
}

```

```

},
{
  {12447, 24328, 28566, 30469, 31468, 1, 0},
  { 4, 18559, 25929, 29452, 30837, 1, 0}
},
{
  { 6059, 14350, 21101, 25538, 28284, 0, 0},
  { 4, 8626, 17947, 23810, 27621, 1, 0}
},
{
  { 3725, 10288, 16087, 20666, 24016, 75, 0},
  { 4, 6533, 12288, 17848, 22413, 80, 0}
},
{
  { 2171, 7502, 12172, 15887, 19156, 0, 0},
  { 7, 5094, 9919, 14712, 18887, 90, 0}
},
{
  { 2182, 6455, 10199, 13836, 17142, 75, 0},
  { 4, 4141, 8300, 12354, 15957, 75, 0}
},
{
  { 1639, 5232, 8372, 11445, 14349, 75, 0},
  { 10, 3077, 6545, 10446, 13509, 0, 0}
},
{
  { 1084, 4304, 6721, 9558, 11647, 75, 0},
  { 545, 1846, 4045, 6508, 8619, 15, 0}
},
{
  {24344, 31910, 32550, 32682, 32732, 86, 0},
  {26511, 26812, 32257, 32605, 32732, 6, 0}
},
{
  {11758, 27192, 31302, 32386, 32693, 76, 0},
  { 4, 22871, 30546, 32311, 32672, 76, 0}
},
{
  {14135, 26534, 30495, 32044, 32475, 1, 0},
  { 4, 20045, 27213, 30377, 31787, 5, 0}
},
{
  { 7076, 18263, 25043, 28687, 30681, 78, 0},
  { 4, 13464, 22336, 27466, 30144, 0, 0}
},
{
  { 4633, 13655, 20720, 25269, 27732, 90, 0},
  { 4, 8679, 16355, 22272, 26693, 90, 0}
},
{
  { 3748, 10999, 17182, 22075, 25415, 75, 0},
  { 4, 7508, 14197, 19726, 24108, 90, 0}
},
{
  { 2184, 7460, 12244, 16404, 19473, 75, 0},
  { 69, 5366, 10343, 14774, 18517, 76, 0}
},
{
  {25069, 32379, 32715, 32736, 32740, 90, 0},
  { 5129, 30522, 32636, 32736, 32740, 75, 0}
},
{
  {19064, 30841, 32497, 32710, 32732, 75, 0},
  { 4, 27337, 32042, 32655, 32732, 0, 0}
},
{
  {15344, 28138, 31636, 32491, 32707, 0, 0},
  { 10, 23734, 30387, 32192, 32638, 0, 0}
},
{
  { 8553, 21612, 28244, 31035, 32141, 93, 0},

```

```

    { 4, 16669, 26367, 30466, 32062, 75, 0}
  },
  {
    { 4707, 13756, 20729, 25449, 28537, 75, 0},
    { 9, 9798, 18016, 23799, 27538, 75, 0}
  },
  {
    {30472, 32430, 32636, 32704, 32730, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    {16740, 29432, 31724, 32306, 32530, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 9770, 21395, 26819, 29645, 31040, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 5883, 13836, 19460, 22882, 25424, 6, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 3255, 9742, 15113, 19977, 23209, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 2318, 6734, 11462, 15492, 18969, 91, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 1344, 4886, 8490, 11147, 13803, 1, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    {31970, 32735, 32740, 32744, 32748, 15, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    {21766, 31615, 32549, 32682, 32732, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    {14071, 27537, 31162, 32180, 32564, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 9394, 22451, 28320, 30624, 31619, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 6357, 16098, 22083, 26080, 28701, 76, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  }
},
{
  {
    {22670, 31040, 32062, 32396, 32494, 62, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 6204, 18887, 26106, 29994, 31527, 1, 0},
    { 4, 13216, 24238, 29218, 31178, 0, 0}
  },
  {
    {10730, 22881, 27929, 30084, 31344, 25, 0},
    { 4, 18763, 26180, 29027, 30353, 26, 0}
  },
  {
    { 4419, 14204, 19910, 24823, 27593, 6, 0},
    { 4, 9746, 17528, 23820, 27251, 0, 0}
  }
}

```

```

},
{
  { 3252, 8648, 14582, 18530, 21957, 31, 0},
  { 4, 6941, 13319, 18486, 23105, 75, 0}
},
{
  { 2193, 8919, 13231, 16176, 19402, 75, 0},
  { 957, 4227, 8679, 14052, 18540, 0, 0}
},
{
  { 1055, 5756, 9129, 12574, 14838, 26, 0},
  { 822, 4183, 8160, 12989, 16983, 75, 0}
},
{
  { 2072, 7295, 11937, 15990, 18808, 90, 0},
  { 1296, 1658, 4422, 5572, 8265, 26, 0}
},
{
  { 1733, 4681, 7680, 10654, 12528, 75, 0},
  { 4, 1678, 2994, 4855, 6537, 75, 0}
},
{
  {24889, 31894, 32576, 32692, 32732, 6, 0},
  { 7489, 28023, 32440, 32702, 32732, 12, 0}
},
{
  {13420, 28082, 31481, 32420, 32693, 2, 0},
  { 4, 24196, 30708, 32346, 32682, 5, 0}
},
{
  {14158, 26101, 30271, 31760, 32366, 5, 0},
  { 4, 20337, 27469, 30511, 31808, 5, 0}
},
{
  { 6545, 17433, 24600, 28570, 30488, 15, 0},
  { 4, 14301, 23067, 27600, 30048, 0, 0}
},
{
  { 4611, 12681, 19425, 24119, 27368, 75, 0},
  { 6, 8765, 16411, 22190, 26422, 90, 0}
},
{
  { 3293, 10520, 16126, 21429, 23703, 0, 0},
  { 13, 7300, 14277, 19168, 23448, 75, 0}
},
{
  { 2464, 7343, 11864, 15738, 18418, 90, 0},
  { 85, 4192, 8349, 12391, 15795, 0, 0}
},
{
  {26166, 32502, 32728, 32736, 32740, 0, 0},
  { 4561, 31126, 32673, 32736, 32740, 76, 0}
},
{
  {19676, 30981, 32490, 32712, 32732, 75, 0},
  { 5, 27917, 32248, 32677, 32732, 1, 0}
},
{
  {15573, 28642, 31918, 32572, 32718, 1, 0},
  { 33, 24002, 30459, 32142, 32635, 80, 0}
},
{
  { 8718, 22161, 28870, 31388, 32324, 76, 0},
  { 37, 17061, 26380, 30433, 31990, 75, 0}
},
{
  { 4574, 13326, 20007, 24664, 27510, 75, 0},
  { 75, 9240, 17009, 22503, 26129, 80, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},

```



```

},
{
  { 1976, 5938, 10048, 13248, 15220, 1, 0},
  { 74, 2872, 5881, 8719, 12203, 3, 0}
},
{
  { 1557, 5182, 7687, 10041, 12530, 18, 0},
  { 302, 3173, 6064, 9146, 12139, 16, 0}
},
{
  { 1041, 2835, 4612, 6619, 7701, 75, 0},
  { 284, 1622, 3979, 5622, 7317, 75, 0}
},
{
  {23459, 31826, 32523, 32706, 32732, 32, 0},
  { 4370, 28739, 32302, 32698, 32732, 30, 0}
},
{
  {14211, 27544, 31308, 32252, 32647, 0, 0},
  { 4, 24453, 30550, 32209, 32593, 1, 0}
},
{
  {13549, 26000, 29944, 31604, 32248, 1, 0},
  { 4, 19875, 26915, 30003, 31480, 6, 0}
},
{
  { 6688, 18459, 24591, 28101, 29825, 1, 0},
  { 956, 10175, 19300, 25085, 28308, 1, 0}
},
{
  { 4149, 12273, 18024, 22204, 25189, 0, 0},
  { 4, 7907, 14615, 20513, 24277, 0, 0}
},
{
  { 3084, 9362, 14515, 19060, 22086, 0, 0},
  { 881, 5812, 11992, 16891, 21229, 1, 0}
},
{
  { 2270, 6094, 9436, 13046, 15065, 90, 0},
  { 4, 2628, 6124, 9042, 11413, 1, 0}
},
{
  {26213, 32584, 32715, 32736, 32740, 31, 0},
  { 9595, 31175, 32663, 32736, 32740, 81, 0}
},
{
  {19100, 30551, 32351, 32675, 32732, 80, 0},
  { 42, 27427, 31958, 32640, 32732, 1, 0}
},
{
  {14907, 27403, 31007, 32163, 32585, 0, 0},
  { 74, 23383, 30040, 31948, 32492, 6, 0}
},
{
  { 8140, 21140, 27523, 30537, 31800, 1, 0},
  { 601, 15748, 25003, 29331, 31277, 1, 0}
},
{
  { 3878, 11230, 17295, 21736, 24654, 75, 0},
  { 267, 7839, 14608, 19822, 23499, 75, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
}

```



```

    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {18225, 31062, 32659, 32736, 32740, 31, 0},
    { 747, 25457, 32313, 32736, 32740, 43, 0}
},
{
    { 9428, 30477, 32339, 32668, 32696, 18, 0},
    { 4, 21160, 31076, 32661, 32732, 0, 0}
},
{
    {14256, 29803, 32043, 32616, 32732, 75, 0},
    { 10, 23629, 31540, 32549, 32730, 32, 0}
},
{
    { 8535, 25300, 30787, 31549, 32463, 85, 0},
    { 1351, 18749, 29052, 31248, 32599, 90, 0}
},
{
    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 3855, 9638, 17348, 19275, 23130, 50, 0}
},
{
    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {20884, 32507, 32740, 32744, 32748, 15, 0},
    {11439, 30717, 32718, 32736, 32740, 83, 0}
},
{
    {16651, 32234, 32676, 32736, 32740, 15, 0},
    { 2783, 29097, 32503, 32736, 32740, 80, 0}
},
{
    {14119, 29823, 32164, 32617, 32692, 12, 0},
    { 2682, 23298, 31008, 32517, 32684, 88, 0}
},
{
    {10503, 29407, 31508, 31928, 32348, 25, 0},
    { 886, 19484, 30111, 30997, 31882, 75, 0}
},
{
    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {29595, 31985, 32553, 32665, 32732, 93, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {16090, 27467, 31184, 32157, 32704, 1, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {11881, 22576, 26436, 31160, 32105, 27, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 4886, 21845, 24001, 27882, 28169, 60, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},

```

```

{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {29540, 32735, 32740, 32744, 32748, 6, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {21479, 29463, 32511, 32736, 32740, 93, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {15056, 26273, 29816, 32178, 32473, 25, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
  {
    {10519, 30400, 31391, 32327, 32713, 15, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 86, 20785, 28528, 31103, 32087, 0, 0},
    { 4, 6067, 24484, 29795, 31615, 0, 0}
  },
  {
    {10869, 20589, 26954, 30327, 31505, 0, 0},
    { 4, 15477, 26105, 29564, 31257, 1, 0}
  },
  {
    { 4914, 15045, 23040, 27717, 30490, 0, 0},
    { 4, 10307, 19393, 25816, 29383, 15, 0}
  },
  {
    { 2819, 10379, 17295, 23231, 27208, 75, 0},
    { 2939, 6238, 14472, 21322, 25358, 1, 0}
  },
  {
    { 2510, 8572, 13721, 19650, 24615, 91, 0},
    { 4, 6558, 12858, 16531, 22847, 1, 0}
  },
  {
    { 2098, 8373, 11493, 16534, 21082, 10, 0},
    { 371, 3300, 8699, 14469, 19163, 76, 0}
  },
  {
    { 1409, 6107, 9513, 13859, 18674, 40, 0},
    { 1882, 3210, 8081, 12842, 17048, 85, 0}
  },
  {
    { 780, 5461, 7802, 10923, 13263, 0, 0},
    { 537, 4835, 8058, 10744, 13967, 90, 0}
  }
},
{

```

```

    {21042, 31954, 32667, 32736, 32740, 18, 0},
    {28136, 28288, 32626, 32720, 32732, 57, 0}
},
{
    { 7851, 28094, 31782, 32677, 32732, 78, 0},
    {   4, 21935, 31329, 32631, 32732, 1, 0}
},
{
    {12762, 26468, 31109, 32414, 32693, 90, 0},
    {   4, 21556, 29551, 32049, 32643, 0, 0}
},
{
    {10020, 22866, 29258, 31456, 32395, 90, 0},
    {   4, 17576, 27446, 31017, 32403, 0, 0}
},
{
    { 7348, 19414, 26844, 30566, 31941, 90, 0},
    {   4, 12308, 22780, 28574, 31279, 75, 0}
},
{
    { 6457, 17740, 24197, 28585, 30902, 78, 0},
    {  769, 11802, 20680, 26000, 30085, 15, 0}
},
{
    { 4625, 14072, 20960, 25388, 28438, 40, 0},
    {  185,  8492, 16246, 20861, 25568, 90, 0}
},
{
    {24491, 32563, 32740, 32744, 32748, 90, 0},
    {10193, 31065, 32692, 32736, 32740, 75, 0}
},
{
    {18023, 31654, 32683, 32736, 32740, 78, 0},
    {   4, 28229, 32493, 32736, 32740, 0, 0}
},
{
    {15126, 29669, 32385, 32717, 32732, 3, 0},
    {   4, 25310, 31745, 32662, 32732, 0, 0}
},
{
    {11803, 26851, 31619, 32578, 32732, 90, 0},
    {   4, 21347, 30308, 32362, 32732, 75, 0}
},
{
    { 8413, 22123, 28902, 31454, 32484, 91, 0},
    {  533, 15815, 26833, 31091, 32338, 1, 0}
},
{
    {30851, 32572, 32702, 32736, 32740, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {15898, 29692, 32156, 32573, 32722, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 9275, 21031, 27592, 31095, 32259, 78, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 7586, 16864, 22782, 27672, 30277, 83, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 4779, 14336, 21504, 25259, 27307, 25, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},

```

```

{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {32033, 32744, 32748, 32752, 32756, 3, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {21905, 31682, 32689, 32736, 32740, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {16223, 28310, 31943, 32665, 32704, 2, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {10587, 25458, 29995, 32012, 32516, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
  {
    {20744, 31573, 32432, 32575, 32732, 37, 0},
    { 2048, 24576, 26624, 28672, 30720, 50, 0}
  },
  {
    { 1512, 20783, 28537, 31233, 32030, 0, 0},
    {   4,  9686, 25465, 29825, 31669, 0, 0}
  },
  {
    {11418, 22879, 28443, 30589, 31691, 0, 0},
    {   4, 18611, 26971, 30180, 31489, 1, 0}
  },
  {
    { 5931, 14893, 21314, 26301, 29130, 0, 0},
    {   4, 10485, 20060, 26727, 29964, 2, 0}
  },
  {
    { 4511, 11376, 17124, 22328, 26530, 90, 0},
    {  109,  4550, 13218, 19861, 24578,  6, 0}
  },
  {
    { 2402,  9192, 14384, 19305, 22990, 75, 0},
    {   4,  5323, 10546, 15982, 20848, 76, 0}
  },
  {
    { 3030,  8069, 12594, 17208, 20711,  3, 0},
    {   4,  4225,  9697, 14319, 18246, 75, 0}
  },
  {
    { 2223,  6369, 10286, 13595, 18323, 83, 0},
    {  175,  2961,  7374, 11755, 15610,  3, 0}
  },
  {
    { 1367,  6180,  9659, 13156, 15544, 90, 0},
    {  505,  2440,  6344, 10284, 13648, 92, 0}
  },
  {
    {22323, 32120, 32586, 32723, 32732,  6, 0},
    {28677, 28707, 32573, 32736, 32740, 31, 0}
  },
  {
    { 9067, 28461, 31803, 32646, 32732, 90, 0},
    {   4, 23713, 31500, 32628, 32732,  6, 0}
  },
  {

```

```

    {14440, 27462, 31270, 32400, 32671, 75, 0},
    { 4, 21690, 28829, 31630, 32453, 5, 0}
},
{
    { 8061, 20968, 27850, 30659, 31948, 93, 0},
    { 4, 16746, 25908, 29977, 31832, 5, 0}
},
{
    { 5820, 16451, 23783, 28293, 30655, 90, 0},
    { 4, 10996, 20608, 26697, 30042, 5, 0}
},
{
    { 4853, 14377, 20953, 25793, 29037, 75, 0},
    { 4, 10112, 18364, 24708, 28619, 75, 0}
},
{
    { 3528, 10929, 17584, 23009, 26629, 78, 0},
    { 5, 8177, 15921, 21527, 25499, 75, 0}
},
{
    {24757, 32619, 32740, 32744, 32748, 1, 0},
    { 8837, 31099, 32688, 32736, 32740, 78, 0}
},
{
    {19775, 31747, 32667, 32736, 32740, 75, 0},
    { 4, 28933, 32511, 32732, 32736, 0, 0}
},
{
    {17105, 29988, 32352, 32704, 32732, 75, 0},
    { 5, 26112, 31741, 32633, 32732, 0, 0}
},
{
    {10631, 24891, 30498, 32167, 32638, 93, 0},
    { 4, 20962, 29880, 32217, 32689, 76, 0}
},
{
    { 7259, 19563, 26854, 30459, 32035, 90, 0},
    { 161, 13687, 24042, 29325, 31661, 75, 0}
},
{
    {31243, 32617, 32724, 32736, 32740, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {15771, 29814, 32176, 32618, 32719, 90, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 8991, 20335, 27006, 30538, 31920, 1, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {11004, 16889, 22530, 26291, 28694, 32, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 2048, 4096, 16384, 20480, 28672, 50, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    { 5461, 10923, 16384, 21845, 27307, 0, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
    {32375, 32744, 32748, 32752, 32756, 75, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
},

```

```

{
  {21853, 32148, 32719, 32736, 32740, 90, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {14100, 28596, 31850, 32559, 32685, 85, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  {11320, 25619, 29193, 31576, 32172, 75, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
  {
    {17586, 29249, 30954, 31637, 31943, 60, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 6607, 19220, 27249, 31067, 32015, 15, 0},
    {   4, 12693, 25132, 30060, 31735, 0, 0}
  },
  {
    {11218, 23056, 28975, 31289, 32045, 5, 0},
    {   4, 16611, 26400, 28905, 30652, 1, 0}
  },
  {
    { 4973, 15646, 22529, 27020, 30234, 75, 0},
    {   4, 11786, 18820, 25595, 28661, 75, 0}
  },
  {
    { 4994, 11005, 17116, 23193, 25485, 0, 0},
    {  10, 5820, 14233, 19478, 24366, 75, 0}
  },
  {
    { 1327, 8818, 15358, 18969, 22525, 25, 0},
    { 1517, 5910, 11125, 16349, 21591, 0, 0}
  },
  {
    { 123, 5065, 11340, 15150, 17222, 16, 0},
    { 1786, 5857, 10807, 14291, 19911, 75, 0}
  },
  {
    { 2928, 7753, 12931, 17083, 20510, 75, 0},
    {  549, 2673, 5713, 7772, 12679, 3, 0}
  },
  {
    { 1125, 4606, 9305, 11931, 14884, 13, 0},
    {  12, 3709, 6058, 8777, 11932, 85, 0}
  },
  {
    {19675, 31285, 32524, 32698, 32732, 11, 0},
    {11455, 26740, 32279, 32717, 32732, 3, 0}
  },
  {
    {10345, 28211, 31805, 32644, 32732, 3, 0},
    {   4, 23277, 31209, 32545, 32732, 8, 0}
  },
  {
    {13559, 26348, 31030, 32344, 32653, 6, 0},
    {   4, 19849, 28463, 31321, 32304, 83, 0}
  },
  {
    { 7549, 20488, 27158, 30433, 31903, 75, 0},
    {   4, 15187, 24493, 29279, 31267, 0, 0}
  },
}
{

```



```

{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
},
{
  { 5461, 10923, 16384, 21845, 27307, 0, 0},
  { 5461, 10923, 16384, 21845, 27307, 0, 0}
}
},
{
  {
    {20160, 28303, 31010, 31935, 32259, 65, 0},
    { 5461, 10923, 16384, 21845, 27307, 0, 0}
  },
  {
    { 6387, 21131, 28651, 31386, 32236, 51, 0},
    { 4, 14343, 25805, 30386, 31978, 5, 0}
  },
  {
    {10794, 23651, 28414, 30514, 31623, 26, 0},
    { 4, 18792, 26345, 29842, 31433, 26, 0}
  },
  {
    { 7250, 18857, 23745, 26494, 29185, 31, 0},
    { 59, 5934, 16720, 25540, 28810, 51, 0}
  },
  {
    { 4853, 13443, 19345, 23384, 26041, 0, 0},
    { 4, 7130, 12348, 18932, 24173, 75, 0}
  },
  {
    { 2667, 9653, 13863, 18817, 21760, 1, 0},
    { 4, 5477, 10652, 16140, 19594, 1, 0}
  },
  {
    { 2595, 7918, 13026, 17820, 20007, 77, 0},
    { 508, 3110, 7370, 10565, 14704, 2, 0}
  },
  {
    { 1961, 6366, 9476, 12294, 15249, 90, 0},
    { 753, 2600, 6724, 10877, 14165, 15, 0}
  },
  {
    { 1420, 4356, 6620, 8937, 10997, 90, 0},
    { 120, 1977, 4498, 6986, 9746, 75, 0}
  },
  {
    {20741, 31938, 32628, 32728, 32732, 35, 0},
    { 9417, 28087, 32436, 32701, 32732, 3, 0}
  },
  {
    {12348, 28499, 31976, 32606, 32720, 0, 0},
    { 4, 24379, 31121, 32398, 32684, 0, 0}
  },
  {
    {13275, 26774, 30839, 32177, 32535, 16, 0},
    { 4, 21429, 28887, 31453, 32269, 26, 0}
  },
  {
    { 7379, 21206, 27561, 30526, 31713, 1, 0},
    { 703, 12333, 22878, 28540, 30860, 1, 0}
  },
  {
    { 5216, 15396, 22184, 26885, 29262, 75, 0},
    { 24, 9666, 18359, 24699, 28006, 0, 0}
  },
  {
    { 4332, 12591, 18916, 23721, 26816, 3, 0},
    { 499, 7000, 14338, 19317, 23450, 0, 0}
  },
  {
  }
}

```



```

    }
  }
}

```

Default_Coeff_Base_Lf_Uv_Cdf

```

[ COEFF_CDF_Q_CTXS ][ LF_SIG_COEF_CONTEXTS_UV ][ LF_BASE_SYMBOLS + 1 ] = {
{
{14076, 26464, 29938, 31308, 31828, 6, 0},
{ 7520, 21227, 27766, 30312, 31477, 75, 0},
{ 4377, 13290, 19811, 24220, 27064, 90, 0},
{ 1682, 5139, 8601, 11973, 15046, 90, 0},
{15235, 28605, 31367, 32151, 32451, 6, 0},
{10256, 24586, 29775, 31465, 32137, 93, 0},
{ 5918, 15629, 22317, 26602, 29101, 90, 0},
{ 2015, 5704, 9835, 13705, 17299, 76, 0},
{26420, 31955, 32312, 32430, 32526, 75, 0},
{16374, 29560, 31531, 32023, 32291, 30, 0},
{ 7197, 15954, 20986, 24934, 27737, 6, 0},
{ 4820, 9488, 11701, 14065, 16248, 11, 0}
},
{
{15362, 27650, 30927, 32014, 32397, 6, 0},
{ 9556, 22999, 28733, 31004, 31942, 75, 0},
{ 5238, 14626, 21567, 26015, 28727, 78, 0},
{ 2196, 6512, 10780, 14847, 18294, 93, 0},
{15569, 28727, 31706, 32423, 32650, 0, 0},
{11506, 25824, 30517, 31948, 32443, 75, 0},
{ 6211, 16772, 23701, 27732, 29961, 90, 0},
{ 2461, 7239, 11755, 15869, 19210, 90, 0},
{26951, 32136, 32556, 32654, 32696, 75, 0},
{17691, 30215, 31944, 32399, 32550, 75, 0},
{ 9349, 19307, 25385, 29027, 30821, 6, 0},
{ 8528, 15111, 18457, 21676, 23767, 37, 0}
},
{
{14853, 28192, 31400, 32275, 32557, 1, 0},
{10499, 24975, 30136, 31803, 32399, 75, 0},
{ 5543, 15727, 22759, 27080, 29613, 75, 0},
{ 2214, 6634, 11179, 15534, 19289, 90, 0},
{15032, 28602, 31421, 32234, 32548, 0, 0},
{10857, 24883, 29885, 31603, 32255, 0, 0},
{ 5068, 14275, 20979, 25484, 28338, 0, 0},
{ 1825, 5574, 9437, 13284, 16813, 93, 0},
{28668, 32450, 32696, 32736, 32740, 15, 0},
{17386, 30911, 32385, 32652, 32732, 90, 0},
{ 9471, 20994, 27290, 30512, 31784, 15, 0},
{ 7211, 13700, 19148, 23635, 26679, 0, 0}
},
{
{ 7986, 24484, 30470, 32123, 32478, 12, 0},
{ 6385, 22268, 29781, 31884, 32479, 6, 0},
{ 4563, 14337, 22377, 27527, 30631, 75, 0},
{ 1768, 5903, 10730, 16633, 21440, 90, 0},
{ 7554, 22427, 29258, 31475, 32386, 5, 0},
{ 6209, 20694, 28786, 31444, 32330, 90, 0},
{ 4185, 12361, 19887, 25739, 29491, 93, 0},
{ 1583, 5456, 9702, 15370, 20474, 90, 0},
{30183, 32316, 32574, 32639, 32703, 90, 0},
{15147, 31729, 32521, 32669, 32719, 35, 0},
{ 5461, 10923, 16384, 21845, 27307, 0, 0},
{ 5461, 10923, 16384, 21845, 27307, 0, 0}
}
}
}

```

Default_Coeff_Base_Cdf

```

[ COEFF_CDF_Q_CTXS ][ TX_SIZES ][ SIG_COEF_CONTEXTS ][ 2 ][ 5 ] = {
{
{

```

{ 12360, 26392, 29943, 26, 0},
{ 8590, 25353, 30895, 6, 0}
},
{
{ 7246, 19496, 26530, 75, 0},
{ 3468, 16232, 25621, 90, 0}
},
{
{ 4008, 12605, 18928, 93, 0},
{ 1999, 9165, 16685, 93, 0}
},
{
{ 3148, 9393, 14900, 93, 0},
{ 1110, 7696, 14122, 78, 0}
},
{
{ 2543, 7526, 12021, 93, 0},
{ 2561, 5445, 10546, 93, 0}
},
{
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 8192, 16384, 24576, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 28014, 31534, 32060, 5, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 13135, 23487, 28599, 0, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{ 7049, 15368, 20768, 93, 0},
{ 8192, 16384, 24576, 0, 0}
},

```

{
  { 3109, 8054, 12383, 75, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
}
},
{
  {
    {25298, 32426, 32713, 93, 0},
    {14651, 31551, 32591, 93, 0}
  },
  {
    {18466, 30483, 32267, 75, 0},
    { 306, 26643, 31787, 90, 0}
  },
  {
    {12399, 26150, 30691, 90, 0},
    { 1131, 21210, 29129, 75, 0}
  },
  {
    { 7487, 20115, 26881, 93, 0},
    { 656, 14639, 24430, 75, 0}
  },
  {
    { 3588, 11042, 17169, 78, 0},
    { 756, 7796, 14913, 78, 0}
  },
  {
    {26570, 32443, 32691, 78, 0},
    {17559, 31708, 32628, 78, 0}
  },
  {
    {18312, 30354, 32288, 90, 0},
    { 581, 26605, 31701, 90, 0}
  },
  {
    {11812, 25791, 30572, 75, 0},
    { 1073, 20736, 28973, 75, 0}
  },
  {
    { 7165, 19304, 26609, 93, 0},
    { 971, 14401, 24415, 93, 0}
  },
  {
    { 3978, 12366, 19160, 93, 0},
    { 935, 8823, 16611, 78, 0}
  },
  {
    {25670, 32230, 32678, 75, 0},
    {19556, 31721, 32677, 0, 0}
  },
  {
    {17121, 29735, 32054, 75, 0},
    { 1460, 26595, 31623, 90, 0}
  },
  {
    {11589, 25561, 30426, 90, 0},
    { 1652, 20606, 28939, 78, 0}
  },
  {
    { 7343, 20631, 27574, 90, 0},
    { 922, 15328, 25115, 75, 0}
  },
  {
    { 4778, 14146, 21151, 90, 0},
    { 902, 9679, 18420, 78, 0}
  },
  {

```

```

    {30964, 32538, 32693, 5, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    {18212, 29081, 31586, 93, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    {10647, 22153, 27538, 0, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    { 6396, 16045, 22849, 0, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    { 3574, 10559, 16078, 0, 0},
    { 8192, 16384, 24576, 0, 0}
}
},
{
    {
        {28697, 32692, 32748, 90, 0},
        {13717, 32316, 32740, 90, 0}
    },
    {
        {24783, 32374, 32712, 90, 0},
        { 1229, 30871, 32600, 75, 0}
    },
    {
        {19000, 30799, 32442, 75, 0},
        { 2583, 27380, 31887, 15, 0}
    },
    {
        {10656, 24633, 30053, 75, 0},
        { 882, 19759, 28348, 75, 0}
    },
    {
        { 4024, 11749, 17822, 75, 0},
        { 311, 8254, 15425, 75, 0}
    },
    {
        {29761, 32708, 32748, 75, 0},
        {18251, 32567, 32748, 0, 0}
    },
    {
        {24756, 32428, 32722, 75, 0},
        { 1802, 30951, 32542, 93, 0}
    },
    {
        {18650, 30593, 32394, 75, 0},
        { 2883, 27721, 31874, 78, 0}
    },
    {
        {10280, 24576, 29914, 75, 0},
        { 288, 19075, 28254, 1, 0}
    },
    {
        { 3976, 12847, 19261, 75, 0},
        { 754, 8850, 16554, 3, 0}
    },
    {
        {30346, 32725, 32748, 75, 0},
        {24555, 32672, 32748, 5, 0}
    },
    {
        {24866, 32414, 32726, 75, 0},
        { 857, 31274, 32665, 90, 0}
    },
    {
        {18664, 30690, 32404, 93, 0},

```

```

    { 2459, 27485, 31962, 93, 0}
  },
  {
    {10302, 24628, 30393, 93, 0},
    { 923, 19581, 28849, 78, 0}
  },
  {
    { 5108, 14969, 22262, 78, 0},
    { 637, 11172, 20006, 78, 0}
  },
  {
    {32114, 32706, 32748, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    {22647, 31591, 32522, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    {14439, 26903, 30661, 6, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 7410, 19635, 26647, 1, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 3267, 10417, 16526, 92, 0},
    { 8192, 16384, 24576, 0, 0}
  }
},
{
  {
    {29459, 32676, 32741, 26, 0},
    {12621, 32434, 32748, 83, 0}
  },
  {
    {23667, 32112, 32701, 75, 0},
    { 5563, 30354, 32581, 75, 0}
  },
  {
    {19383, 30660, 32564, 75, 0},
    { 3845, 27375, 31697, 90, 0}
  },
  {
    {10075, 24894, 29897, 90, 0},
    { 2073, 19727, 27880, 78, 0}
  },
  {
    { 3368, 9398, 13892, 81, 0},
    { 571, 6307, 12048, 81, 0}
  },
  {
    {30541, 32712, 32748, 18, 0},
    {17105, 32518, 32748, 90, 0}
  },
  {
    {26380, 32437, 32736, 78, 0},
    { 1892, 31537, 32682, 91, 0}
  },
  {
    {20557, 31281, 32473, 90, 0},
    { 3148, 28251, 32121, 0, 0}
  },
  {
    {10605, 25976, 30389, 76, 0},
    { 3627, 20150, 28517, 76, 0}
  },
  {
    { 3792, 11134, 15907, 81, 0},
    { 517, 7539, 13580, 81, 0}
  }
}

```



```

},
{
  {30408, 32748, 32752, 75, 0},
  {32291, 32705, 32748, 75, 0}
},
{
  {25730, 32585, 32739, 90, 0},
  { 5942, 31616, 32616, 93, 0}
},
{
  {20185, 31076, 32464, 78, 0},
  { 1004, 28344, 32094, 78, 0}
},
{
  {11707, 26199, 30689, 93, 0},
  { 1262, 21328, 29569, 93, 0}
},
{
  { 5035, 14930, 21229, 90, 0},
  { 746, 10699, 18756, 90, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
},
{
  {
    {28383, 32609, 32688, 10, 0},
    {15428, 32067, 32704, 35, 0}
  },
  {
    {15754, 30788, 32588, 90, 0},
    { 4880, 29631, 31606, 0, 0}
  },
  {
    {10550, 24328, 32147, 75, 0},
    { 8738, 22209, 29673, 75, 0}
  },
  {
    { 9695, 22879, 26951, 25, 0},
    { 1798, 15785, 27773, 75, 0}
  },
  {
    { 3427, 10282, 14127, 93, 0},
    { 1687, 7815, 14031, 78, 0}
  },
  {
    {31560, 32315, 32617, 65, 0},
    {12105, 31933, 32351, 75, 0}
  },
  {
    {20436, 29949, 32063, 0, 0},
    { 6712, 28623, 32571, 25, 0}
  },
},

```

```

{
  { 9717, 28135, 31186, 15, 0},
  { 8249, 24519, 30362, 40, 0}
},
{
  { 7123, 25288, 28494, 10, 0},
  {11556, 14880, 27228, 10, 0}
},
{
  { 4041, 10893, 15462, 5, 0},
  { 794, 10029, 17278, 18, 0}
},
{
  {28664, 32744, 32748, 0, 0},
  {31632, 32700, 32740, 15, 0}
},
{
  {26708, 32579, 32742, 78, 0},
  { 6672, 31742, 32705, 93, 0}
},
{
  {20833, 31645, 32639, 93, 0},
  { 1146, 29311, 32329, 93, 0}
},
{
  {13676, 28227, 31721, 93, 0},
  { 1479, 24229, 30975, 78, 0}
},
{
  { 5792, 15556, 21640, 91, 0},
  { 1677, 10836, 19468, 91, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
},
{
  {
    {
      {12507, 26077, 30060, 30, 0},
      { 7550, 25098, 30550, 30, 0}
    },
    {
      { 8009, 20715, 27321, 3, 0},
      { 3223, 17464, 26650, 3, 0}
    },
    {
      { 4305, 13150, 20144, 93, 0},
      { 1952, 10128, 18324, 3, 0}
    },
    {
      { 3098, 9586, 15580, 93, 0},
      { 1089, 7195, 13810, 93, 0}
    }
  }
}

```



```

{
  {18907, 30472, 32314, 90, 0},
  { 953, 26945, 31805, 75, 0}
},
{
  {13516, 26908, 30904, 75, 0},
  { 1549, 21936, 29542, 0, 0}
},
{
  { 8297, 20991, 27437, 78, 0},
  { 557, 15598, 25261, 0, 0}
},
{
  { 3980, 11626, 17598, 3, 0},
  { 918, 8073, 15400, 3, 0}
},
{
  {26391, 32035, 32595, 75, 0},
  {17672, 31342, 32493, 0, 0}
},
{
  {19580, 30651, 32319, 75, 0},
  { 589, 27593, 31900, 75, 0}
},
{
  {13734, 27193, 31043, 0, 0},
  { 1624, 22701, 29917, 0, 0}
},
{
  { 8311, 21239, 27850, 3, 0},
  { 510, 15971, 25802, 0, 0}
},
{
  { 4217, 12744, 19511, 3, 0},
  { 717, 9081, 16951, 3, 0}
},
{
  {25094, 32069, 32616, 5, 0},
  {17839, 31778, 32640, 5, 0}
},
{
  {19672, 30814, 32312, 75, 0},
  { 2025, 28482, 32043, 75, 0}
},
{
  {14800, 28188, 31442, 0, 0},
  { 1221, 24055, 30573, 0, 0}
},
{
  { 8995, 22820, 28992, 3, 0},
  { 1167, 17916, 27275, 3, 0}
},
{
  { 4844, 14798, 22123, 78, 0},
  { 839, 10591, 19419, 3, 0}
},
{
  {30845, 32589, 32713, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  {19409, 29802, 31864, 75, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  {11114, 22871, 27848, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 6092, 16202, 22429, 5, 0},
  { 8192, 16384, 24576, 0, 0}
}

```

```

},
{
  { 3247, 9679, 15027, 90, 0},
  { 8192, 16384, 24576, 0, 0}
}
},
{
  {
    {27264, 32622, 32748, 75, 0},
    {13182, 32042, 32728, 75, 0}
  },
  {
    {22849, 32019, 32682, 75, 0},
    { 491, 29944, 32540, 75, 0}
  },
  {
    {17618, 30067, 32261, 75, 0},
    { 1861, 25960, 31490, 75, 0}
  },
  {
    {10036, 24382, 29998, 75, 0},
    { 1347, 18931, 28166, 78, 0}
  },
  {
    { 5051, 14703, 21347, 75, 0},
    { 259, 10600, 19087, 0, 0}
  },
  {
    {29595, 32710, 32748, 75, 0},
    {19929, 32553, 32748, 0, 0}
  },
  {
    {24721, 32388, 32722, 75, 0},
    { 1482, 30913, 32625, 75, 0}
  },
  {
    {18729, 30720, 32485, 0, 0},
    { 1645, 27442, 31977, 0, 0}
  },
  {
    {10807, 25638, 30749, 75, 0},
    { 1150, 20387, 29227, 3, 0}
  },
  {
    { 5546, 16487, 23567, 0, 0},
    { 403, 12249, 21372, 0, 0}
  },
  {
    {30695, 32738, 32748, 80, 0},
    {25061, 32689, 32748, 0, 0}
  },
  {
    {26547, 32580, 32747, 75, 0},
    { 1334, 31785, 32708, 75, 0}
  },
  {
    {20996, 31697, 32662, 0, 0},
    { 3682, 29397, 32392, 78, 0}
  },
  {
    {13209, 28091, 31805, 3, 0},
    { 840, 23372, 30922, 3, 0}
  },
  {
    { 7389, 20059, 27005, 0, 0},
    { 332, 15296, 25093, 0, 0}
  },
  {
    {32282, 32726, 32748, 15, 0},
    { 8192, 16384, 24576, 0, 0}
  },
},

```

```

{
  {23211, 31638, 32504, 90, 0},
  { 8192, 16384, 24576,  0, 0}
},
{
  {13980, 26668, 30388,  1, 0},
  { 8192, 16384, 24576,  0, 0}
},
{
  { 7795, 19288, 25941,  6, 0},
  { 8192, 16384, 24576,  0, 0}
},
{
  { 3877, 11700, 17890,  1, 0},
  { 8192, 16384, 24576,  0, 0}
}
},
{
  {
    {28880, 32680, 32748,  0, 0},
    {12427, 32347, 32748, 90, 0}
  },
  {
    {24025, 32081, 32704, 75, 0},
    { 2065, 30727, 32657, 76, 0}
  },
  {
    {19872, 30769, 32395,  0, 0},
    { 1210, 27547, 31961, 76, 0}
  },
  {
    {10410, 24769, 29817,  1, 0},
    { 2247, 19228, 27805,  1, 0}
  },
  {
    { 3997, 11545, 16952, 75, 0},
    {  146,  7698, 14688,  1, 0}
  },
  {
    {30905, 32733, 32748, 75, 0},
    {18768, 32634, 32748, 76, 0}
  },
  {
    {26894, 32528, 32731, 76, 0},
    { 1978, 31675, 32684, 76, 0}
  },
  {
    {20750, 31222, 32412,  1, 0},
    { 3259, 28753, 32073,  1, 0}
  },
  {
    {10948, 25238, 30225,  0, 0},
    {  733, 20129, 28689,  1, 0}
  },
  {
    { 3995, 12602, 18654,  0, 0},
    {  997,  9216, 16560, 75, 0}
  },
  {
    {31712, 32750, 32754, 75, 0},
    {31152, 32718, 32748, 75, 0}
  },
  {
    {28091, 32642, 32748, 75, 0},
    { 5217, 32006, 32702, 78, 0}
  },
  {
    {22886, 31816, 32647,  0, 0},
    { 2358, 29840, 32394, 75, 0}
  },
  {

```

```

    {13616, 27730, 31451, 0, 0},
    { 1022, 23059, 30343, 0, 0}
},
{
    { 6026, 17154, 23829, 75, 0},
    { 837, 12628, 21443, 90, 0}
},
{
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
}
},
{
    {
        {27869, 32588, 32732, 1, 0},
        {11934, 31678, 32720, 78, 0}
    },
    {
        {20773, 31428, 32627, 75, 0},
        { 813, 29336, 32408, 0, 0}
    },
    {
        {16679, 28979, 31835, 6, 0},
        { 2825, 24549, 30651, 1, 0}
    },
    {
        { 8393, 21384, 27275, 1, 0},
        { 95, 15687, 24751, 6, 0}
    },
    {
        { 3024, 8983, 13215, 75, 0},
        { 476, 5931, 11675, 90, 0}
    },
    {
        {29793, 32715, 32748, 31, 0},
        {15137, 32136, 32735, 0, 0}
    },
    {
        {23697, 31828, 32635, 1, 0},
        { 1247, 29941, 32474, 31, 0}
    },
    {
        {15925, 28680, 31655, 6, 0},
        { 2639, 25018, 30693, 6, 0}
    },
    {
        { 8039, 21380, 27690, 80, 0},
        { 2054, 16597, 25993, 0, 0}
    },
    {
        { 3566, 10567, 15908, 90, 0},
        { 229, 7289, 14138, 1, 0}
    },
    {
        {31282, 32752, 32756, 75, 0},

```



```

    { 383, 10485, 19689, 78, 0}
  },
  {
    {27697, 32637, 32746, 90, 0},
    {19828, 32280, 32729, 0, 0}
  },
  {
    {20895, 31709, 32601, 93, 0},
    { 706, 29097, 32430, 75, 0}
  },
  {
    {14671, 28816, 32008, 3, 0},
    { 547, 23747, 31087, 75, 0}
  },
  {
    { 8923, 23097, 29710, 93, 0},
    { 217, 17350, 27724, 75, 0}
  },
  {
    { 5346, 16046, 24143, 78, 0},
    { 362, 12099, 22064, 78, 0}
  },
  {
    {26664, 32524, 32729, 0, 0},
    {19929, 32241, 32732, 5, 0}
  },
  {
    {20429, 31546, 32587, 78, 0},
    { 871, 29224, 32427, 75, 0}
  },
  {
    {15091, 29302, 32181, 93, 0},
    { 420, 24224, 31315, 75, 0}
  },
  {
    { 9650, 24609, 30553, 93, 0},
    { 716, 18899, 28798, 78, 0}
  },
  {
    { 6228, 18247, 26011, 93, 0},
    { 488, 12960, 23566, 93, 0}
  },
  {
    {31458, 32658, 32742, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    {20693, 30846, 32296, 90, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    {12771, 25313, 29851, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 7602, 19024, 25601, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 4692, 14076, 20351, 15, 0},
    { 8192, 16384, 24576, 0, 0}
  }
},
{
  {
    {26646, 32618, 32748, 75, 0},
    {12985, 32023, 32728, 75, 0}
  },
  {
    {21678, 31945, 32675, 93, 0},
    { 46, 29649, 32565, 0, 0}
  }
}

```

```

},
{
  {16813, 29938, 32304, 75, 0},
  { 734, 25488, 31518, 0, 0}
},
{
  {10047, 24573, 30320, 78, 0},
  { 680, 18741, 28408, 78, 0}
},
{
  { 5428, 16010, 23347, 75, 0},
  { 282, 11635, 21016, 78, 0}
},
{
  {29492, 32711, 32748, 90, 0},
  {22856, 32581, 32748, 90, 0}
},
{
  {23814, 32473, 32732, 75, 0},
  { 505, 30842, 32654, 75, 0}
},
{
  {17970, 30664, 32542, 0, 0},
  { 591, 26988, 31983, 0, 0}
},
{
  {10731, 25646, 30915, 75, 0},
  { 716, 19981, 29261, 78, 0}
},
{
  { 5897, 17573, 25080, 75, 0},
  { 178, 12954, 22725, 90, 0}
},
{
  {30940, 32740, 32748, 0, 0},
  {27493, 32693, 32748, 5, 0}
},
{
  {25557, 32571, 32738, 90, 0},
  { 1156, 31536, 32692, 90, 0}
},
{
  {19676, 31438, 32631, 75, 0},
  { 1853, 28395, 32266, 78, 0}
},
{
  {12168, 27322, 31648, 93, 0},
  { 374, 21862, 30426, 78, 0}
},
{
  { 6908, 19494, 27071, 78, 0},
  { 113, 14821, 25176, 90, 0}
},
{
  {32313, 32737, 32748, 25, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  {23711, 32006, 32620, 75, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  {16041, 28992, 31635, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 9990, 23637, 29213, 5, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 5762, 16053, 22500, 5, 0},

```

```

    { 8192, 16384, 24576, 0, 0}
  },
  {
    {
      {27184, 32672, 32748, 1, 0},
      {10613, 32252, 32738, 75, 0}
    },
    {
      {22470, 32091, 32716, 75, 0},
      { 949, 30238, 32643, 80, 0}
    },
    {
      {17935, 30422, 32413, 0, 0},
      { 674, 26736, 32008, 80, 0}
    },
    {
      {11011, 25612, 30643, 90, 0},
      { 997, 19957, 28943, 90, 0}
    },
    {
      { 4754, 13518, 19616, 75, 0},
      { 30, 10027, 18226, 76, 0}
    },
    {
      {29734, 32729, 32748, 0, 0},
      {17868, 32552, 32748, 0, 0}
    },
    {
      {25168, 32565, 32739, 76, 0},
      { 525, 31274, 32689, 76, 0}
    },
    {
      {19155, 30941, 32493, 75, 0},
      { 1986, 27740, 32084, 75, 0}
    },
    {
      {10959, 25516, 30465, 75, 0},
      { 233, 20155, 29122, 81, 0}
    },
    {
      { 4681, 14503, 21334, 0, 0},
      { 483, 10679, 18885, 75, 0}
    },
    {
      {31907, 32752, 32756, 0, 0},
      {31874, 32719, 32748, 75, 0}
    },
    {
      {26499, 32598, 32736, 90, 0},
      { 4327, 31612, 32678, 93, 0}
    },
    {
      {20964, 31523, 32599, 75, 0},
      { 1451, 28828, 32263, 90, 0}
    },
    {
      {12822, 27283, 31369, 78, 0},
      { 336, 21868, 30060, 75, 0}
    },
    {
      { 6252, 17633, 24699, 93, 0},
      { 465, 12979, 22414, 93, 0}
    },
    {
      { 8192, 16384, 24576, 0, 0},
      { 8192, 16384, 24576, 0, 0}
    },
    {
      { 8192, 16384, 24576, 0, 0},
      { 8192, 16384, 24576, 0, 0}
    }
  }

```

```

},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
}
},
{
  {
    {27513, 32649, 32748, 6, 0},
    {12533, 32080, 32727, 0, 0}
  },
  {
    {22267, 31858, 32685, 80, 0},
    { 475, 29885, 32524, 15, 0}
  },
  {
    {17673, 30194, 32223, 1, 0},
    { 1643, 25673, 31224, 1, 0}
  },
  {
    {10022, 24035, 29658, 75, 0},
    { 276, 18877, 27751, 6, 0}
  },
  {
    { 4529, 13192, 18915, 76, 0},
    { 285, 9212, 16975, 76, 0}
  },
  {
    {29967, 32707, 32748, 6, 0},
    {15326, 32376, 32746, 15, 0}
  },
  {
    {24351, 32244, 32717, 1, 0},
    { 1649, 31057, 32684, 0, 0}
  },
  {
    {17832, 30408, 32381, 6, 0},
    { 1876, 26558, 31696, 31, 0}
  },
  {
    {10499, 24843, 30081, 75, 0},
    { 1028, 19460, 28542, 1, 0}
  },
  {
    { 5050, 14590, 21227, 75, 0},
    { 92, 10933, 19446, 6, 0}
  },
  {
    {31604, 32752, 32756, 75, 0},
    {32163, 32722, 32748, 90, 0}
  },
  {
    {26537, 32585, 32739, 75, 0},
    { 5518, 31656, 32695, 90, 0}
  },
  {
    {20484, 31396, 32601, 75, 0},
    { 1028, 28479, 32227, 78, 0}
  },
  {
    {12807, 27291, 31420, 93, 0},
    { 469, 22186, 30263, 75, 0}
  },
},

```



```

},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  {31972, 32470, 32748, 28, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  {22231, 32012, 32721, 40, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 4681, 16384, 30427, 50, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
},
{
  { 8192, 16384, 24576, 0, 0},
  { 8192, 16384, 24576, 0, 0}
}
},
{
  {
    {26034, 32672, 32748, 93, 0},
    {12576, 32175, 32745, 90, 0}
  },
  {
    {22551, 32372, 32748, 93, 0},
    { 20, 30978, 32718, 15, 0}
  },
  {
    {17805, 31280, 32648, 93, 0},
    { 1557, 28114, 32425, 3, 0}
  },
  {
    {13292, 28676, 32147, 93, 0},
    { 555, 23489, 31233, 0, 0}
  },
  {
    {10339, 22768, 29435, 35, 0},
    { 758, 15761, 26377, 35, 0}
  },
  {
    {29142, 32726, 32748, 15, 0},
    {17698, 32623, 32748, 25, 0}
  },
  {
    {24984, 32622, 32748, 93, 0},
    { 600, 31789, 32745, 15, 0}
  },
  {
    {18241, 32005, 32740, 78, 0},
    { 3053, 28834, 32573, 93, 0}
  }
},

```

```

{
  {11582, 29556, 32181, 18, 0},
  { 5757, 25240, 31661, 33, 0}
},
{
  { 3072, 21504, 29184, 65, 0},
  {  643, 12208, 26343, 35, 0}
},
{
  {29915, 32732, 32748, 78, 0},
  {24589, 32741, 32748,  0, 0}
},
{
  {25072, 32653, 32747, 15, 0},
  { 2766, 32408, 32743, 78, 0}
},
{
  {19064, 32047, 32688, 90, 0},
  { 6208, 29694, 32582, 75, 0}
},
{
  {16118, 29137, 32325, 90, 0},
  { 3624, 26555, 32028,  6, 0}
},
{
  { 8192, 16384, 24576,  0, 0},
  { 8192, 16384, 24576,  0, 0}
},
{
  {32165, 32689, 32748, 75, 0},
  { 8192, 16384, 24576,  0, 0}
},
{
  {20616, 31528, 32613, 78, 0},
  { 8192, 16384, 24576,  0, 0}
},
{
  {17095, 29686, 32489, 18, 0},
  { 8192, 16384, 24576,  0, 0}
},
{
  {13280, 23529, 32046, 75, 0},
  { 8192, 16384, 24576,  0, 0}
},
{
  { 1725,  3449, 31043,  0, 0},
  { 8192, 16384, 24576,  0, 0}
}
},
{
  {
    {27252, 32687, 32748,  0, 0},
    {12250, 32394, 32747,  0, 0}
  },
  {
    {22953, 32345, 32732, 93, 0},
    {  100, 31081, 32711, 75, 0}
  },
  {
    {18289, 31130, 32616, 90, 0},
    { 1043, 27525, 32293, 15, 0}
  },
  {
    {12030, 27356, 31773, 75, 0},
    { 1328, 21750, 30500, 18, 0}
  },
  {
    { 8454, 22057, 29080, 90, 0},
    {  312, 16537, 27228, 76, 0}
  },
  {

```



```

    {29639, 32739, 32748, 0, 0},
    {24459, 32557, 32748, 78, 0}
},
{
    {24256, 32629, 32748, 75, 0},
    { 586, 31663, 32726, 5, 0}
},
{
    {18917, 31583, 32708, 0, 0},
    { 1616, 28584, 32470, 75, 0}
},
{
    {13655, 29545, 32407, 75, 0},
    { 1396, 23860, 31469, 90, 0}
},
{
    { 9270, 24668, 30796, 91, 0},
    { 1688, 18438, 28979, 91, 0}
},
{
    {31096, 32751, 32755, 0, 0},
    {28942, 32716, 32748, 5, 0}
},
{
    {25750, 32644, 32748, 75, 0},
    { 465, 32242, 32741, 90, 0}
},
{
    {20669, 32346, 32741, 90, 0},
    { 2885, 29990, 32664, 75, 0}
},
{
    {15748, 30594, 32632, 90, 0},
    { 372, 26420, 32273, 90, 0}
},
{
    {10002, 25783, 31458, 81, 0},
    { 869, 20294, 30019, 92, 0}
},
{
    {32513, 32752, 32756, 0, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    {24556, 32348, 32716, 90, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    {19561, 30597, 32461, 1, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    {11689, 27786, 31427, 0, 0},
    { 8192, 16384, 24576, 0, 0}
},
{
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
}
},
{
    {
        {26428, 32701, 32748, 5, 0},
        { 8484, 32368, 32747, 0, 0}
    },
    {
        {22687, 32407, 32740, 75, 0},
        { 390, 30961, 32716, 5, 0}
    },
    {
        {18394, 31398, 32670, 75, 0},

```



```

{
  {
    {27339, 32708, 32748, 81, 0},
    {11572, 32394, 32748, 8, 0}
  },
  {
    {23878, 32548, 32744, 5, 0},
    { 461, 31124, 32705, 0, 0}
  },
  {
    {19326, 31492, 32668, 75, 0},
    { 664, 28301, 32374, 75, 0}
  },
  {
    {13120, 27541, 31646, 76, 0},
    { 550, 22504, 30562, 76, 0}
  },
  {
    { 6365, 17893, 24908, 15, 0},
    { 608, 12570, 22302, 90, 0}
  },
  {
    {29384, 32732, 32748, 6, 0},
    {13817, 32574, 32748, 0, 0}
  },
  {
    {24520, 32525, 32747, 76, 0},
    { 1775, 31839, 32739, 6, 0}
  },
  {
    {19335, 31747, 32688, 6, 0},
    { 3566, 28880, 32362, 6, 0}
  },
  {
    {12397, 27427, 31587, 1, 0},
    { 509, 22594, 30553, 0, 0}
  },
  {
    { 7148, 19219, 26315, 75, 0},
    { 677, 13710, 23764, 76, 0}
  },
  {
    {31236, 32752, 32756, 80, 0},
    {32618, 32731, 32748, 90, 0}
  },
  {
    {27068, 32675, 32748, 75, 0},
    { 5945, 32111, 32744, 90, 0}
  },
  {
    {22852, 32207, 32731, 75, 0},
    { 2177, 30282, 32570, 75, 0}
  },
  {
    {14481, 29134, 32134, 0, 0},
    { 2589, 24920, 31424, 0, 0}
  },
  {
    { 8171, 21620, 28271, 75, 0},
    { 169, 16674, 26696, 75, 0}
  },
  {
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 8192, 16384, 24576, 0, 0},
  }
}

```

```

    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    { 8192, 16384, 24576, 0, 0},
    { 8192, 16384, 24576, 0, 0}
  }
}
}
}

```

Default_Coeff_Base_Uv_Cdf[COEFF_CDF_Q_CTXS][SIG_COEF_CONTEXTS_UV][5] =

```

{
  {
    {26904, 32102, 32598, 0, 0},
    {15749, 28898, 31610, 90, 0},
    { 9106, 21329, 26962, 90, 0},
    { 4828, 12923, 18983, 75, 0},
    {27779, 32406, 32689, 90, 0},
    {17414, 30077, 32025, 90, 0},
    { 9228, 22296, 27767, 81, 0},
    { 4564, 12734, 19144, 90, 0},
    {29238, 32489, 32693, 80, 0},
    {19819, 30853, 32222, 25, 0},
    { 9314, 19318, 25346, 6, 0},
    { 3060, 10265, 16088, 5, 0}
  },
  {
    {26264, 32282, 32667, 75, 0},
    {16468, 29532, 31912, 78, 0},
    { 8915, 21647, 27531, 0, 0},
    { 4341, 12415, 18782, 75, 0},
    {27171, 32473, 32714, 90, 0},
    {18547, 30679, 32297, 75, 0},
    { 9723, 23037, 28561, 0, 0},
    { 4310, 12399, 18745, 75, 0},
    {29747, 32607, 32733, 75, 0},
    {21347, 31575, 32509, 90, 0},
    {10548, 22298, 28335, 80, 0},
    { 2124, 9709, 17598, 90, 0}
  },
  {
    {26928, 32457, 32720, 75, 0},
    {16472, 30015, 32195, 75, 0},
    { 9599, 22905, 28625, 75, 0},
    { 4866, 13616, 20435, 75, 0},
    {27711, 32490, 32726, 75, 0},
    {16379, 30167, 32197, 75, 0},
    { 9085, 22146, 28034, 1, 0},
    { 3893, 11359, 17565, 75, 0},
    {30658, 32684, 32748, 90, 0},
    {22171, 31876, 32636, 90, 0},
    {10726, 23535, 29193, 17, 0},
    { 8192, 16384, 24576, 0, 0}
  },
  {
    {26130, 32392, 32726, 75, 0},
    {13730, 29626, 32253, 75, 0},
    { 9442, 23537, 29252, 6, 0},
    { 4255, 13986, 21188, 35, 0},
    {24285, 32033, 32659, 0, 0},
    {11954, 28100, 31763, 75, 0},
    { 7399, 19439, 26435, 5, 0},
    { 4096, 12081, 19339, 77, 0},
    {30583, 31894, 32331, 90, 0},
    {18905, 31508, 32138, 25, 0},
    { 8192, 16384, 24576, 0, 0},
  }
}

```

```

    { 8192, 16384, 24576, 0, 0}
}
}

```

Default_Coeff_Br_Cdf

```

[ COEFF_CDF_Q_CTXS ][ LEVEL_CONTEXTS ][ BR_CDF_SIZE + 1 ] = {
{
{22305, 28743, 30345, 6, 0},
{22663, 29948, 31320, 78, 0},
{19776, 28658, 30435, 78, 0},
{15436, 25313, 28181, 78, 0},
{11214, 20671, 24854, 78, 0},
{ 8548, 16982, 21766, 78, 0},
{ 5729, 11993, 17176, 78, 0}
},
{
{24190, 30560, 31576, 30, 0},
{24366, 30921, 31758, 78, 0},
{21555, 29858, 31096, 3, 0},
{16661, 26574, 28953, 78, 0},
{11988, 21738, 25479, 93, 0},
{ 8920, 17446, 22214, 93, 0},
{ 5802, 12294, 17499, 78, 0}
},
{
{29897, 32392, 32630, 30, 0},
{27484, 32113, 32486, 3, 0},
{23979, 31279, 32028, 3, 0},
{19126, 28997, 30760, 3, 0},
{14329, 25240, 28412, 0, 0},
{10861, 20773, 25861, 75, 0},
{ 7274, 15044, 21663, 0, 0}
},
{
{31501, 32621, 32738, 55, 0},
{29651, 32500, 32698, 6, 0},
{27204, 32153, 32586, 32, 0},
{22529, 30751, 32001, 31, 0},
{17048, 27341, 30583, 15, 0},
{12558, 24036, 28617, 6, 0},
{ 9501, 18960, 24998, 13, 0}
}
}
}

```

Default_Coeff_Br_Lf_Cdf

```

[ COEFF_CDF_Q_CTXS ][ LF_LEVEL_CONTEXTS ][ BR_CDF_SIZE + 1 ] = {
{
{ 7943, 14193, 20775, 32, 0},
{14297, 22400, 26238, 91, 0},
{10557, 18683, 22550, 90, 0},
{ 8289, 16068, 18454, 92, 0},
{ 5258, 10730, 13709, 90, 0},
{ 3933, 8166, 10680, 90, 0},
{ 2465, 5325, 6625, 90, 0},
{10865, 16430, 19691, 6, 0},
{14571, 22733, 26106, 15, 0},
{14072, 23021, 25971, 75, 0},
{11558, 20253, 23235, 78, 0},
{ 8603, 16200, 19466, 93, 0},
{ 6641, 13086, 16612, 78, 0},
{ 4240, 9043, 11946, 93, 0}
},
{
{11992, 17977, 23847, 37, 0},
{14229, 22329, 26075, 90, 0},
{11443, 20390, 22820, 76, 0},
{ 8397, 16223, 19264, 90, 0},
{ 5827, 11883, 14653, 90, 0},
{ 4416, 9159, 11664, 90, 0},

```

```

{ 2812, 6024, 7386, 93, 0},
{16755, 23759, 26700, 5, 0},
{17293, 25940, 28518, 93, 0},
{15547, 25297, 27304, 93, 0},
{12584, 22225, 24378, 93, 0},
{ 9185, 17517, 19977, 93, 0},
{ 7015, 13961, 16744, 93, 0},
{ 4509, 9609, 11858, 93, 0}
},
{
{13319, 21082, 26725, 37, 0},
{15058, 23563, 27312, 75, 0},
{12093, 21552, 24381, 91, 0},
{ 9847, 18914, 21611, 76, 0},
{ 6984, 14207, 17984, 75, 0},
{ 5580, 11627, 15411, 75, 0},
{ 3907, 8590, 10683, 81, 0},
{19858, 26499, 28738, 31, 0},
{19608, 28127, 30429, 0, 0},
{17478, 27390, 29446, 0, 0},
{14848, 25124, 27561, 0, 0},
{11376, 21082, 24410, 75, 0},
{ 9179, 17862, 21918, 90, 0},
{ 6435, 13300, 17660, 75, 0}
},
{
{13965, 20979, 26518, 62, 0},
{17417, 26343, 29434, 6, 0},
{14569, 24464, 27796, 6, 0},
{12532, 22823, 25671, 11, 0},
{ 8586, 17291, 21825, 6, 0},
{ 6326, 13118, 17907, 6, 0},
{ 4106, 9173, 12623, 15, 0},
{24175, 29267, 31202, 35, 0},
{22491, 30103, 31663, 1, 0},
{19961, 29271, 31158, 6, 0},
{17023, 27102, 29910, 6, 0},
{13897, 23531, 27451, 1, 0},
{11233, 19762, 24691, 7, 0},
{ 7378, 14545, 19934, 7, 0}
}
}

```

```

Default_Coeff_Base_Lf_Eob_Cdf
[ COEFF_CDF_Q_CTXS ][ TX_SIZES ][ SIG_COEF_CONTEXTS_EOB ]
[ LF_BASE_SYMBOLS ] = {
{
{
{27486, 31140, 31779, 32064, 7, 0},
{28263, 31142, 31813, 32057, 6, 0},
{27578, 30405, 31202, 31448, 6, 0},
{29800, 32145, 32589, 32665, 93, 0}
},
{
{27197, 31209, 31698, 31944, 32, 0},
{30415, 31987, 32302, 32387, 0, 0},
{30766, 32160, 32461, 32564, 75, 0},
{31446, 32510, 32712, 32740, 90, 0}
},
{
{ 9881, 28329, 30354, 31217, 62, 0},
{30815, 32135, 32360, 32446, 5, 0},
{31896, 32058, 32423, 32525, 37, 0},
{27097, 28987, 31508, 32138, 50, 0}
},
{
{ 76, 3661, 13759, 21509, 62, 0},
{31910, 32583, 32688, 32740, 93, 0},
{ 6554, 13107, 19661, 26214, 0, 0},
{ 6554, 13107, 19661, 26214, 0, 0}
}
}
}

```

```

},
{
{ 298, 596, 894, 1192, 50, 0},
{29008, 30611, 32490, 32683, 3, 0},
{ 6554, 13107, 19661, 26214, 0, 0},
{ 6554, 13107, 19661, 26214, 0, 0}
}
},
{
{
{28595, 31926, 32420, 32582, 91, 0},
{28506, 31820, 32509, 32686, 0, 0},
{28680, 31966, 32503, 32626, 0, 0},
{27591, 32110, 32578, 32712, 75, 0}
},
{
{28550, 32010, 32465, 32593, 1, 0},
{31254, 32502, 32660, 32713, 75, 0},
{31600, 32549, 32688, 32735, 75, 0},
{31747, 32600, 32712, 32740, 78, 0}
},
{
{27239, 31085, 31883, 32247, 7, 0},
{31836, 32524, 32640, 32686, 0, 0},
{32293, 32547, 32627, 32711, 43, 0},
{24966, 28087, 29127, 31988, 50, 0}
},
{
{22871, 30729, 31682, 32000, 37, 0},
{32054, 32674, 32735, 32740, 90, 0},
{ 6554, 13107, 19661, 26214, 0, 0},
{ 6554, 13107, 19661, 26214, 0, 0}
},
{
{ 7975, 22586, 27023, 29960, 62, 0},
{31368, 32346, 32628, 32738, 6, 0},
{ 6554, 13107, 19661, 26214, 0, 0},
{ 6554, 13107, 19661, 26214, 0, 0}
}
},
{
{
{29573, 32066, 32477, 32625, 76, 0},
{28827, 32168, 32643, 32735, 90, 0},
{29362, 32299, 32653, 32722, 75, 0},
{29765, 32457, 32711, 32740, 90, 0}
},
{
{29324, 32291, 32617, 32714, 0, 0},
{31938, 32652, 32728, 32740, 75, 0},
{31949, 32673, 32738, 32742, 75, 0},
{31802, 32666, 32744, 32748, 78, 0}
},
{
{28567, 31870, 32438, 32629, 6, 0},
{32100, 32663, 32732, 32740, 75, 0},
{32648, 32724, 32744, 32748, 83, 0},
{25565, 32000, 32473, 32709, 60, 0}
},
{
{26412, 30605, 31764, 32230, 37, 0},
{32167, 32698, 32744, 32748, 90, 0},
{ 6554, 13107, 19661, 26214, 0, 0},
{ 6554, 13107, 19661, 26214, 0, 0}
},
{
{24457, 29221, 30907, 31726, 62, 0},
{32052, 32690, 32737, 32741, 90, 0},
{ 6554, 13107, 19661, 26214, 0, 0},
{ 6554, 13107, 19661, 26214, 0, 0}
}
}
}

```

```

},
{
{
{30609, 32413, 32637, 32716, 5, 0},
{30333, 32610, 32744, 32748, 1, 0},
{31067, 32497, 32676, 32711, 1, 0},
{32062, 32748, 32752, 32756, 36, 0}
},
{
{31289, 32570, 32714, 32740, 1, 0},
{32248, 32709, 32744, 32748, 75, 0},
{32234, 32719, 32744, 32748, 75, 0},
{32449, 32740, 32744, 32748, 3, 0}
},
{
{31348, 32506, 32654, 32724, 6, 0},
{32301, 32703, 32744, 32748, 90, 0},
{32692, 32739, 32744, 32748, 7, 0},
{20480, 26624, 28672, 30720, 50, 0}
},
{
{30132, 32214, 32554, 32667, 7, 0},
{32297, 32711, 32744, 32748, 90, 0},
{ 6554, 13107, 19661, 26214, 0, 0},
{ 6554, 13107, 19661, 26214, 0, 0}
},
{
{28223, 31648, 32311, 32527, 37, 0},
{32184, 32695, 32744, 32748, 90, 0},
{ 6554, 13107, 19661, 26214, 0, 0},
{ 6554, 13107, 19661, 26214, 0, 0}
}
}
}
}

```

```

Default_Coeff_Base_Eob_Cdf[ COEFF_CDF_Q_CTXS ][ TX_SIZES ]
[ SIG_COEF_CONTEXTS_EOB ][ 4 ] = {

```

```

{
{
{10923, 21845, 0, 0},
{10923, 21845, 0, 0},
{10923, 21845, 0, 0},
{25475, 29789, 75, 0}
},
{
{10923, 21845, 0, 0},
{32132, 32609, 50, 0},
{32294, 32701, 75, 0},
{31618, 32500, 93, 0}
},
{
{10923, 21845, 0, 0},
{32658, 32751, 78, 0},
{32701, 32753, 93, 0},
{32675, 32745, 93, 0}
},
{
{10923, 21845, 0, 0},
{32658, 32719, 65, 0},
{32658, 32744, 67, 0},
{32647, 32756, 33, 0}
},
{
{10923, 21845, 0, 0},
{32538, 32711, 0, 0},
{32138, 32453, 0, 0},
{32356, 32562, 50, 0}
}
},
{

```



```

{
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0},
  {10923, 21845, 0, 0},
  {26028, 30728, 3, 0}
},
{
  {10923, 21845, 0, 0},
  {31077, 32105, 85, 0},
  {31889, 32651, 93, 0},
  {31276, 32538, 93, 0}
},
{
  {10923, 21845, 0, 0},
  {32624, 32752, 93, 0},
  {32688, 32756, 93, 0},
  {32695, 32756, 93, 0}
},
{
  {10923, 21845, 0, 0},
  {32677, 32743, 68, 0},
  {32051, 32739, 7, 0},
  {32291, 32748, 32, 0}
},
{
  {10923, 21845, 0, 0},
  {32710, 32756, 78, 0},
  {32700, 32756, 13, 0},
  {24989, 32282, 62, 0}
}
},
{
  {
    {10923, 21845, 0, 0},
    {10923, 21845, 0, 0},
    {10923, 21845, 0, 0},
    {28958, 32194, 93, 0}
  },
  {
    {10923, 21845, 0, 0},
    {32132, 32735, 25, 0},
    {32555, 32745, 93, 0},
    {32458, 32729, 93, 0}
  },
  {
    {10923, 21845, 0, 0},
    {32617, 32754, 93, 0},
    {32699, 32756, 93, 0},
    {32695, 32756, 93, 0}
  },
  {
    {10923, 21845, 0, 0},
    {32684, 32756, 93, 0},
    {32552, 32756, 10, 0},
    {32589, 32756, 75, 0}
  },
  {
    {10923, 21845, 0, 0},
    {32687, 32756, 93, 0},
    {32716, 32756, 91, 0},
    {31504, 32715, 56, 0}
  }
},
{
  {
    {10923, 21845, 0, 0},
    {10923, 21845, 0, 0},
    {10923, 21845, 0, 0},
    {31358, 32747, 18, 0}
  },
  {

```

```

    {10923, 21845, 0, 0},
    {32389, 32626, 75, 0},
    {32642, 32756, 93, 0},
    {32629, 32756, 18, 0}
  },
  {
    {10923, 21845, 0, 0},
    {32686, 32756, 93, 0},
    {32726, 32756, 93, 0},
    {32724, 32756, 15, 0}
  },
  {
    {10923, 21845, 0, 0},
    {32700, 32756, 93, 0},
    {32577, 32756, 40, 0},
    {32626, 32753, 88, 0}
  },
  {
    {10923, 21845, 0, 0},
    {32708, 32756, 93, 0},
    {32603, 32749, 91, 0},
    {32644, 32756, 42, 0}
  }
}
}

```

Default_Coeff_Br_Uv_Cdf

```

[ COEFF_CDF_Q_CTXS ][ LEVEL_CONTEXTS_UV ][ BR_CDF_SIZE + 1 ] = {
  {
    {20014, 26541, 29552, 7, 0},
    {20674, 27680, 30329, 78, 0},
    {16228, 24293, 28314, 75, 0},
    { 9580, 16283, 20959, 75, 0}
  },
  {
    {24442, 29827, 31535, 75, 0},
    {22490, 28954, 31256, 93, 0},
    {17882, 25670, 29377, 93, 0},
    {10664, 17735, 22437, 75, 0}
  },
  {
    {26645, 31153, 32251, 78, 0},
    {23320, 29627, 31652, 90, 0},
    {18265, 26040, 29663, 75, 0},
    {11140, 18504, 23213, 1, 0}
  },
  {
    {26977, 31598, 32364, 50, 0},
    {24721, 30575, 32124, 93, 0},
    {18803, 26659, 30610, 90, 0},
    {11757, 20385, 25284, 5, 0}
  }
}
}

```

Default_Coeff_Base_Lf_Eob_Uv_Cdf

```

[ COEFF_CDF_Q_CTXS ][ SIG_COEF_CONTEXTS_EOB ][ LF_BASE_SYMBOLS ] = {
  {
    {28950, 31443, 32009, 32257, 75, 0},
    {29916, 31919, 32224, 32441, 6, 0},
    {28902, 30805, 31579, 31816, 2, 0},
    { 6554, 13107, 19661, 26214, 0, 0}
  },
  {
    {29843, 32183, 32562, 32675, 75, 0},
    {29975, 31731, 32100, 32264, 12, 0},
    {30096, 31949, 32460, 32611, 27, 0},
    { 6554, 13107, 19661, 26214, 0, 0}
  },
  {

```

```

    {30964, 32463, 32673, 32733, 75, 0},
    {31731, 32623, 32730, 32740, 1, 0},
    {30879, 32453, 32716, 32740, 16, 0},
    { 6554, 13107, 19661, 26214, 0, 0}
},
{
    {31345, 32511, 32676, 32725, 76, 0},
    {31940, 32413, 32531, 32650, 50, 0},
    { 6554, 13107, 19661, 26214, 0, 0},
    { 6554, 13107, 19661, 26214, 0, 0}
}
}

```

```

Default_Coeff_Base_Eob_Uv_Cdf
[ COEFF_CDF_Q_CTXS ][ SIG_COEF_CONTEXTS_EOB ][ 4 ] = {
{
    {10923, 21845, 0, 0},
    {31214, 32437, 93, 0},
    {31888, 32447, 78, 0},
    {30612, 32073, 93, 0}
},
{
    {10923, 21845, 0, 0},
    {31595, 32587, 93, 0},
    {32266, 32710, 93, 0},
    {32287, 32698, 93, 0}
},
{
    {10923, 21845, 0, 0},
    {32086, 32691, 93, 0},
    {32538, 32743, 93, 0},
    {32512, 32730, 93, 0}
},
{
    {10923, 21845, 0, 0},
    {31987, 32697, 93, 0},
    {32662, 32756, 90, 0},
    {32487, 32698, 60, 0}
}
}
}

```

```

Default_Use_Dpcm_Y_Cdf[ 2 + 1 ] =
{16384, 0, 0}

```

```

Default_Dpcm_Mode_Y_Cdf[ 2 + 1 ] =
{16384, 0, 0}

```

```

Default_Use_Dpcm_UV_Cdf[ 2 + 1 ] =
{16384, 0, 0}

```

```

Default_Dpcm_Mode_UV_Cdf[ 2 + 1 ] =
{16384, 0, 0}

```

```

Default_Morph_Pred_Cdf[ 3 ][ 2 + 1 ] =
{
    {31715, 55, 0},
    {19667, 1, 0},
    {10555, 91, 0}
}

```

```

Default_Region_Type_Cdf[INTER_SDP_BSIZE_GROUP][2 + 1] =
{
    { 2635, 25, 0},
    { 883, 25, 0},

```

```

{ 503, 50, 0},
{ 279, 55, 0}
}

```

```

Default_Tip_Pred_Mode_Cdf[ 2 + 1 ] =
{22129, 31, 0}

```

```

Default_Intra_Tx_Type_Long_Cdf[EXT_TX_SIZES][4 + 1] =
{
{10692, 26586, 29231, 31, 0},
{26700, 32160, 32748, 6, 0},
{26915, 32411, 32748, 6, 0},
{ 8192, 16384, 24576, 0, 0}
}

```

```

Default_Inter_Tx_Type_Long_Cdf[3][EXT_TX_SIZES][4 + 1] =
{
{
{ 8347, 20254, 24536, 31, 0},
{15994, 26294, 32748, 1, 0},
{21212, 27810, 32748, 1, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{21815, 25084, 26230, 62, 0},
{29354, 31747, 32748, 37, 0},
{31614, 32529, 32748, 36, 0},
{ 8192, 16384, 24576, 0, 0}
},
{
{10501, 20590, 24181, 31, 0},
{17596, 26388, 32748, 37, 0},
{15407, 26475, 32732, 60, 0},
{ 8192, 16384, 24576, 0, 0}
}
}
}

```

```

Default_Is_Long_Side_Dct_Cdf[2][2 + 1] =
{
{ 36, 0, 0},
{ 128, 31, 0}
}

```

```

Default_Cdef_Index0_Cdf[CDEF_STRENGTH_INDEX0_CTX][2 + 1] =
{
{29034, 32, 0},
{16472, 32, 0},

```

```

{ 5751, 32, 0},
{ 3115, 31, 0}
}

```

```

Default_Cdef_Index_Minus1_With3_Cdf[ 3 ] =
{17708, 32, 0}

```

```

Default_Cdef_Index_Minus1_With4_Cdf[ 4 ] =
{13413, 24899, 32, 0}

```

```

Default_Cdef_Index_Minus1_With5_Cdf[ 5 ] =
{10588, 19866, 26664, 37, 0}

```

```

Default_Cdef_Index_Minus1_With6_Cdf[ 6 ] =
{10131, 17874, 23876, 28766, 37, 0}

```

```

Default_Cdef_Index_Minus1_With7_Cdf[ 7 ] =
{ 8363, 15451, 20811, 25453, 29393, 32, 0}

```

```

Default_Cdef_Index_Minus1_With8_Cdf[ 8 ] =
{ 7372, 13867, 18969, 23278, 26977, 30156, 32, 0}

```

```

Default_Use_Amvd_Cdf[9][3][2 + 1] =
{

```

```

{
    { 5980, 15, 0},
    { 6091, 15, 0},
    { 6237, 93, 0}
},
{
    { 861, 30, 0},
    { 847, 50, 0},
    { 1198, 66, 0}
},
{
    { 456, 53, 0},
    { 431, 68, 0},
    { 849, 68, 0}
},
{
    { 409, 68, 0},
    { 385, 68, 0},
    { 581, 68, 0}
},
{
    {16246, 0, 0},
    { 9696, 0, 0},
    { 8791, 0, 0}
},
{
    {13199, 0, 0},
    {10624, 0, 0},
    { 8586, 1, 0}
},
{
    { 5112, 1, 0},
    { 3920, 3, 0},
    { 3668, 3, 0}
},
{
    {12017, 76, 0},
    {10177, 75, 0},
    { 9184, 75, 0}
},
{

```

```

    {12111, 6, 0},
    { 8056, 5, 0},
    { 6641, 27, 0}
}

```

```

Default_Mrl_Sec_Index_Cdf[MRL_INDEX_CONTEXTS][2 + 1] =
{
    {19678, 6, 0},
    {12287, 6, 0},
    { 9574, 6, 0}
}

```

```

Default_Lossless_Tx_Size_Cdf[BLOCK_SIZE_GROUPS][2][2 + 1] =
{
    {
        {16384, 1, 0},
        {16384, 1, 0}
    },
    {
        {16384, 75, 0},
        {16384, 75, 0}
    },
    {
        {16384, 75, 0},
        {16384, 75, 0}
    },
    {
        {16384, 75, 0},
        {16384, 75, 0}
    }
}

```

```

Default_Lossless_Inter_Tx_Type_Cdf[2 + 1] =
{16384, 0, 0}

```

```

Default_Use_Gdf_Cdf[ 2 + 1 ] =
{14593, 32, 0}

```

```

Default_Bru_Mode_Cdf[ 3 + 1 ] =
{ 4124, 16615, 0, 0}

```

§ 9.4. Quantizer matrix tables

§ 9.4.1. General

The default quantizer matrices are defined via the tables `Qm_Offset` and `Quantizer_Matrix` in [§9.4.3 Tables](#).

There is a set of matrices defined for 15 different levels, and for each of luma and chroma.

For a level given by the variable `lvl`, the luma matrices are defined in the array `Quantizer_Matrix[lvl][0]` and the chroma matrices are defined in the array `Quantizer_Matrix[lvl][1]`.

All the matrices for different sizes are packed together in raster order into this array. The table `Qm_Offset` gives the offset for a given transform size. (Note that certain transform sizes share the same offset as they share the same quantizer matrix.)

All matrix sizes, including derived ones, are defined in the table `Quantizer_Matrix` in [§9.4.3 Tables](#). For informative purposes the subsampling process is defined in [§9.4.2 Derivation process \(Informative\)](#).

Quantizer matrices for transform sizes of 8 by 8 or smaller can also be explicitly signaled with quantizer matrix OBUs.

§ 9.4.2. Derivation process (Informative)

NOTE: This subsection is provided for information only regarding the derivation of Quantizer_Matrix, and is not required to correctly decode AV2 bitstreams (and therefore not invoked by this specification). All required tables are defined in §9.4.3 Tables.

The input to this process is a transform size txSz.

The output is an array derivedMatrix of size Tx_Width[txSz] * Tx_Height[txSz], containing the derived matrix.

There are three fundamental quantizer matrix sizes: 32x32, 32x16 and 16x32. One set of these three sizes is defined for each plane type (luma or chroma). All other quantizer matrix sizes are subsampled from these.

- The array fundamentalMatrix is derived as follows.
 - If Tx_Width[txSz] is equal to Tx_Height[txSz], fundamentalMatrix is set equal to the 32x32 fundamental matrix for the quantizer level and plane type in question.
 - If Tx_Width[txSz] is greater than Tx_Height[txSz], fundamentalMatrix is set equal to the 32x16 fundamental matrix for the quantizer level and plane type in question.
 - Otherwise (Tx_Width[txSz] is less than Tx_Height[txSz]), fundamentalMatrix is set equal to the 16x32 fundamental matrix for the quantizer level and plane type in question.
- The variable fW is set equal to the width of fundamentalMatrix.
- The variable fH is set equal to the height of fundamentalMatrix.
- The variable ratioW is set equal to $fW / Tx_Width[txSz]$.
- The variable ratioH is set equal to $fH / Tx_Height[txSz]$.
- The variable phaseW is set equal to $(ratioW + 1) / 2 - 1$.
- The variable phaseH is set equal to $(ratioH + 1) / 2 - 1$.
- The array element derivedMatrix[i * Tx_Width[txSz] + j] is set equal to fundamentalMatrix[(ratioH * i + phaseH) * fW + ratioW * j + phaseW] for i=0..Tx_Height[txSz] - 1, for j=0..Tx_Width[txSz] - 1.

§ 9.4.3. Tables

```
Qm_Offset[ TX_SIZES_ALL ] =
{
    0, 16, 80, 336, 336, 1360, 1392, 1424,
    1552, 1680, 2192, 336, 336, 2704, 2768, 2832,
    3088, 1680, 2192, 3344, 3472, 2832, 3088, 3344,
    3472,
}

Quantizer_Matrix[ 15 ][ 2 ][ 3600 ] =
{
    {
        { // Luma
          // Size 4x4
```

```

32, 43, 73, 97, 43, 67, 94, 110, 73, 94, 137, 150, 97, 110, 150, 200,
// Size 8x8
32, 32, 38, 51, 68, 84, 95, 109, 32, 35, 40, 49, 63, 76, 89, 102, 38,
40, 54, 65, 78, 91, 98, 106, 51, 49, 65, 82, 97, 111, 113, 121, 68, 63,
78, 97, 117, 134, 138, 142, 84, 76, 91, 111, 134, 152, 159, 168, 95, 89,
98, 113, 138, 159, 183, 199, 109, 102, 106, 121, 142, 168, 199, 220,
// Size 16x16
32, 31, 31, 34, 36, 44, 48, 59, 65, 80, 83, 91, 97, 104, 111, 119, 31,
32, 32, 33, 34, 41, 44, 54, 59, 72, 75, 83, 90, 97, 104, 112, 31, 32,
33, 35, 36, 42, 45, 54, 59, 71, 74, 81, 86, 93, 100, 107, 34, 33, 35,
39, 42, 47, 51, 58, 63, 74, 76, 81, 84, 90, 97, 105, 36, 34, 36, 42, 48,
54, 57, 64, 68, 79, 81, 88, 91, 96, 102, 105, 44, 41, 42, 47, 54, 63,
67, 75, 79, 90, 92, 95, 100, 102, 109, 112, 48, 44, 45, 51, 57, 67, 71,
80, 85, 96, 99, 107, 108, 111, 117, 120, 59, 54, 54, 58, 64, 75, 80, 92,
98, 110, 113, 115, 116, 122, 125, 130, 65, 59, 59, 63, 68, 79, 85, 98,
105, 118, 121, 127, 130, 134, 135, 140, 80, 72, 71, 74, 79, 90, 96, 110,
118, 134, 137, 140, 143, 144, 146, 152, 83, 75, 74, 76, 81, 92, 99, 113,
121, 137, 140, 151, 152, 155, 158, 165, 91, 83, 81, 81, 88, 95, 107,
115, 127, 140, 151, 159, 166, 169, 173, 179, 97, 90, 86, 84, 91, 100,
108, 116, 130, 143, 152, 166, 174, 182, 189, 193, 104, 97, 93, 90, 96,
102, 111, 122, 134, 144, 155, 169, 182, 191, 200, 210, 111, 104, 100,
97, 102, 109, 117, 125, 135, 146, 158, 173, 189, 200, 210, 220, 119,
112, 107, 105, 105, 112, 120, 130, 140, 152, 165, 179, 193, 210, 220,
231,
// Size 32x32
32, 31, 31, 31, 31, 32, 34, 35, 36, 39, 44, 46, 48, 54, 59, 62, 65, 71,
80, 81, 83, 88, 91, 94, 97, 101, 104, 107, 111, 115, 119, 123, 31, 32,
32, 32, 32, 32, 34, 34, 35, 38, 42, 44, 46, 51, 56, 59, 62, 68, 76, 77,
78, 84, 86, 89, 92, 95, 99, 102, 105, 109, 113, 116, 31, 32, 32, 32, 32,
32, 33, 34, 34, 37, 41, 42, 44, 49, 54, 56, 59, 65, 72, 73, 75, 80, 83,
86, 90, 93, 97, 101, 104, 108, 112, 116, 31, 32, 32, 32, 33, 33, 34, 35,
35, 38, 41, 43, 45, 49, 54, 56, 59, 64, 72, 73, 74, 79, 82, 85, 88, 91,
94, 97, 101, 104, 107, 111, 31, 32, 32, 33, 33, 34, 35, 36, 36, 39, 42,
44, 45, 50, 54, 56, 59, 64, 71, 72, 74, 78, 81, 84, 86, 89, 93, 96, 100,
104, 107, 111, 32, 32, 32, 33, 34, 35, 37, 37, 38, 40, 42, 44, 46, 49,
53, 55, 58, 63, 69, 70, 72, 76, 79, 82, 85, 89, 93, 96, 99, 102, 106,
109, 34, 34, 33, 34, 35, 37, 39, 41, 42, 45, 47, 49, 51, 54, 58, 60, 63,
68, 74, 75, 76, 80, 81, 82, 84, 87, 90, 93, 97, 101, 105, 110, 35, 34,
34, 35, 36, 37, 41, 43, 45, 47, 50, 52, 53, 57, 61, 63, 65, 70, 76, 77,
79, 82, 84, 86, 89, 91, 92, 93, 96, 100, 103, 107, 36, 35, 34, 35, 36,
38, 42, 45, 48, 50, 54, 55, 57, 60, 64, 66, 68, 73, 79, 80, 81, 85, 88,
90, 91, 93, 96, 99, 102, 103, 105, 107, 39, 38, 37, 38, 39, 40, 45, 47,
50, 54, 58, 59, 61, 65, 69, 71, 73, 78, 84, 85, 86, 91, 92, 92, 95, 98,
100, 101, 103, 106, 110, 114, 44, 42, 41, 41, 42, 42, 47, 50, 54, 58,
63, 65, 67, 71, 75, 77, 79, 84, 90, 91, 92, 95, 95, 97, 100, 101, 102,
105, 109, 111, 112, 114, 46, 44, 42, 43, 44, 44, 49, 52, 55, 59, 65, 67,
69, 74, 78, 80, 82, 87, 93, 94, 95, 98, 100, 103, 102, 105, 108, 110,
111, 113, 117, 121, 48, 46, 44, 45, 45, 46, 51, 53, 57, 61, 67, 69, 71,
76, 80, 83, 85, 90, 96, 97, 99, 103, 107, 105, 108, 111, 111, 113, 117,
119, 120, 122, 54, 51, 49, 49, 50, 49, 54, 57, 60, 65, 71, 74, 76, 82,
87, 89, 92, 97, 104, 105, 106, 111, 110, 111, 114, 113, 116, 120, 120,
121, 125, 130, 59, 56, 54, 54, 54, 53, 58, 61, 64, 69, 75, 78, 80, 87,
92, 95, 98, 103, 110, 111, 113, 115, 115, 119, 116, 120, 122, 122, 125,
129, 130, 130, 62, 59, 56, 56, 56, 55, 60, 63, 66, 71, 77, 80, 83, 89,
95, 98, 101, 107, 114, 115, 117, 119, 123, 121, 125, 126, 125, 129, 131,
131, 135, 140, 65, 62, 59, 59, 59, 58, 63, 65, 68, 73, 79, 82, 85, 92,
98, 101, 105, 111, 118, 119, 121, 126, 127, 128, 130, 130, 134, 133,
135, 140, 140, 140, 71, 68, 65, 64, 64, 63, 68, 70, 73, 78, 84, 87, 90,
97, 103, 107, 111, 117, 125, 126, 128, 134, 132, 136, 133, 138, 137,
140, 143, 142, 145, 150, 80, 76, 72, 72, 71, 69, 74, 76, 79, 84, 90, 93,
96, 104, 110, 114, 118, 125, 134, 135, 137, 139, 140, 139, 143, 142,
144, 146, 146, 151, 152, 151, 81, 77, 73, 73, 72, 70, 75, 77, 80, 85,
91, 94, 97, 105, 111, 115, 119, 126, 135, 137, 138, 144, 147, 146, 148,
149, 151, 150, 156, 155, 157, 163, 83, 78, 75, 74, 74, 72, 76, 79, 81,
86, 92, 95, 99, 106, 113, 117, 121, 128, 137, 138, 140, 147, 151, 156,
152, 157, 155, 161, 158, 162, 165, 164, 88, 84, 80, 79, 78, 76, 80, 82,
85, 91, 95, 98, 103, 111, 115, 119, 126, 134, 139, 144, 147, 152, 154,
158, 163, 159, 165, 163, 168, 168, 169, 176, 91, 86, 83, 82, 81, 79, 81,
84, 88, 92, 95, 100, 107, 110, 115, 123, 127, 132, 140, 147, 151, 154,
159, 161, 166, 171, 169, 173, 173, 176, 179, 177, 94, 89, 86, 85, 84,
82, 82, 86, 90, 92, 97, 103, 105, 111, 119, 121, 128, 136, 139, 146,

```


156, 158, 161, 166, 168, 174, 179, 178, 180, 183, 183, 190, 97, 92, 90,
 88, 86, 85, 84, 89, 91, 95, 100, 102, 108, 114, 116, 125, 130, 133, 143,
 148, 152, 163, 166, 168, 174, 176, 182, 187, 189, 188, 193, 191, 101,
 95, 93, 91, 89, 89, 87, 91, 93, 98, 101, 105, 111, 113, 120, 126, 130,
 138, 142, 149, 157, 159, 171, 174, 176, 183, 184, 191, 195, 199, 197,
 204, 104, 99, 97, 94, 93, 93, 90, 92, 96, 100, 102, 108, 111, 116, 122,
 125, 134, 137, 144, 151, 155, 165, 169, 179, 182, 184, 191, 193, 200,
 204, 210, 206, 107, 102, 101, 97, 96, 96, 93, 93, 99, 101, 105, 110,
 113, 120, 122, 129, 133, 140, 146, 150, 161, 163, 173, 178, 187, 191,
 193, 200, 202, 210, 214, 222, 111, 105, 104, 101, 100, 99, 97, 96, 102,
 103, 109, 111, 117, 120, 125, 131, 135, 143, 146, 156, 158, 168, 173,
 180, 189, 195, 200, 202, 210, 212, 220, 224, 115, 109, 108, 104, 104,
 102, 101, 100, 103, 106, 111, 113, 119, 121, 129, 131, 140, 142, 151,
 155, 162, 168, 176, 183, 188, 199, 204, 210, 212, 220, 222, 230, 119,
 113, 112, 107, 107, 106, 105, 103, 105, 110, 112, 117, 120, 125, 130,
 135, 140, 145, 152, 157, 165, 169, 179, 183, 193, 197, 210, 214, 220,
 222, 231, 232, 123, 116, 116, 111, 111, 109, 110, 107, 107, 114, 114,
 121, 122, 130, 130, 140, 140, 150, 151, 163, 164, 176, 177, 190, 191,
 204, 206, 222, 224, 230, 232, 242,
 // Size 4x8
 32, 42, 75, 91, 33, 42, 69, 86, 37, 58, 84, 91, 49, 71, 103, 110, 65,
 84, 125, 128, 80, 97, 142, 152, 91, 100, 145, 178, 104, 112, 146, 190,
 // Size 8x4
 32, 33, 37, 49, 65, 80, 91, 104, 42, 42, 58, 71, 84, 97, 100, 112, 75,
 69, 84, 103, 125, 142, 145, 146, 91, 86, 91, 110, 128, 152, 178, 190,
 // Size 8x16
 32, 32, 36, 53, 65, 87, 93, 99, 31, 33, 34, 49, 59, 78, 86, 93, 32, 34,
 36, 50, 59, 77, 82, 89, 34, 37, 42, 54, 63, 79, 80, 88, 36, 38, 48, 60,
 68, 84, 86, 90, 44, 43, 53, 71, 79, 95, 94, 97, 48, 46, 56, 76, 85, 102,
 105, 105, 58, 54, 63, 87, 98, 116, 112, 115, 65, 58, 68, 92, 105, 124,
 122, 124, 79, 70, 79, 104, 118, 141, 135, 135, 82, 72, 81, 106, 121,
 144, 149, 146, 91, 80, 88, 106, 130, 148, 162, 159, 97, 86, 94, 107,
 128, 157, 167, 171, 103, 93, 98, 114, 131, 150, 174, 186, 110, 100, 101,
 117, 138, 161, 183, 193, 118, 107, 105, 118, 136, 157, 182, 203,
 // Size 16x8
 32, 31, 32, 34, 36, 44, 48, 58, 65, 79, 82, 91, 97, 103, 110, 118, 32,
 33, 34, 37, 38, 43, 46, 54, 58, 70, 72, 80, 86, 93, 100, 107, 36, 34,
 36, 42, 48, 53, 56, 63, 68, 79, 81, 88, 94, 98, 101, 105, 53, 49, 50,
 54, 60, 71, 76, 87, 92, 104, 106, 106, 107, 114, 117, 118, 65, 59, 59,
 63, 68, 79, 85, 98, 105, 118, 121, 130, 128, 131, 138, 136, 87, 78, 77,
 79, 84, 95, 102, 116, 124, 141, 144, 148, 157, 150, 161, 157, 93, 86,
 82, 80, 86, 94, 105, 112, 122, 135, 149, 162, 167, 174, 183, 182, 99,
 93, 89, 88, 90, 97, 105, 115, 124, 135, 146, 159, 171, 186, 193, 203,
 // Size 16x32
 32, 31, 32, 34, 36, 44, 53, 59, 65, 79, 87, 90, 93, 96, 99, 102, 31, 32,
 32, 34, 35, 42, 51, 56, 62, 75, 82, 85, 88, 91, 94, 97, 31, 32, 33, 33,
 34, 41, 49, 54, 59, 72, 78, 82, 86, 90, 93, 97, 31, 32, 33, 34, 35, 41,
 49, 54, 59, 71, 78, 81, 84, 87, 90, 93, 32, 32, 34, 35, 36, 42, 50, 54,
 59, 71, 77, 80, 82, 86, 89, 93, 32, 33, 35, 37, 38, 42, 49, 53, 58, 69,
 75, 78, 82, 86, 89, 92, 34, 34, 37, 39, 42, 48, 54, 58, 63, 73, 79, 78,
 80, 83, 88, 92, 35, 34, 37, 41, 45, 50, 57, 61, 65, 76, 82, 83, 84, 84,
 87, 90, 36, 34, 38, 43, 48, 54, 60, 64, 68, 78, 84, 87, 86, 89, 90, 90,
 39, 37, 40, 45, 50, 58, 65, 69, 73, 84, 89, 89, 91, 91, 93, 96, 44, 41,
 43, 48, 53, 63, 71, 75, 79, 90, 95, 93, 94, 95, 97, 97, 46, 43, 44, 49,
 55, 65, 73, 78, 82, 93, 98, 100, 98, 100, 99, 103, 48, 45, 46, 51, 56,
 67, 76, 80, 85, 96, 102, 102, 105, 102, 105, 104, 53, 49, 50, 54, 60,
 71, 82, 87, 92, 103, 109, 107, 107, 110, 107, 111, 58, 54, 54, 58, 63,
 75, 87, 92, 98, 110, 116, 115, 112, 111, 115, 112, 61, 57, 56, 60, 66,
 77, 89, 95, 101, 114, 120, 118, 119, 118, 116, 120, 65, 60, 58, 63, 68,
 79, 92, 98, 105, 118, 124, 123, 122, 123, 124, 121, 71, 65, 63, 68, 73,
 84, 97, 103, 111, 125, 132, 132, 130, 128, 127, 130, 79, 72, 70, 74, 79,
 90, 104, 110, 118, 133, 141, 136, 135, 135, 135, 131, 81, 74, 71, 75,
 80, 91, 105, 112, 119, 135, 142, 140, 140, 138, 139, 142, 82, 75, 72,
 76, 81, 92, 106, 113, 121, 136, 144, 151, 149, 149, 146, 143, 88, 80,
 77, 80, 85, 97, 108, 115, 126, 142, 149, 153, 153, 152, 152, 154, 91,
 83, 80, 81, 88, 100, 106, 114, 130, 142, 148, 155, 162, 160, 159, 155,
 94, 85, 83, 82, 91, 100, 105, 118, 131, 137, 153, 160, 165, 167, 166,
 168, 97, 88, 86, 85, 94, 100, 107, 123, 128, 140, 157, 161, 167, 173,
 171, 169, 100, 91, 89, 87, 97, 100, 111, 121, 127, 145, 152, 164, 173,
 178, 182, 181, 103, 94, 93, 90, 98, 101, 114, 120, 131, 144, 150, 170,
 174, 180, 186, 183, 107, 97, 96, 93, 100, 104, 117, 119, 136, 142, 155,

```

168, 177, 187, 191, 198, 110, 101, 100, 97, 101, 108, 117, 123, 138,
141, 161, 165, 183, 188, 193, 200, 114, 104, 104, 100, 103, 112, 117,
127, 137, 146, 159, 167, 185, 190, 201, 206, 118, 108, 107, 103, 105,
115, 118, 131, 136, 151, 157, 172, 182, 197, 203, 208, 122, 111, 111,
107, 107, 119, 119, 136, 136, 156, 156, 178, 179, 203, 204, 217,
// Size 32x16
32, 31, 31, 31, 32, 32, 34, 35, 36, 39, 44, 46, 48, 53, 58, 61, 65, 71,
79, 81, 82, 88, 91, 94, 97, 100, 103, 107, 110, 114, 118, 122, 31, 32,
32, 32, 32, 33, 34, 34, 34, 37, 41, 43, 45, 49, 54, 57, 60, 65, 72, 74,
75, 80, 83, 85, 88, 91, 94, 97, 101, 104, 108, 111, 32, 32, 33, 33, 34,
35, 37, 37, 38, 40, 43, 44, 46, 50, 54, 56, 58, 63, 70, 71, 72, 77, 80,
83, 86, 89, 93, 96, 100, 104, 107, 111, 34, 34, 33, 34, 35, 37, 39, 41,
43, 45, 48, 49, 51, 54, 58, 60, 63, 68, 74, 75, 76, 80, 81, 82, 85, 87,
90, 93, 97, 100, 103, 107, 36, 35, 34, 35, 36, 38, 42, 45, 48, 50, 53,
55, 56, 60, 63, 66, 68, 73, 79, 80, 81, 85, 88, 91, 94, 97, 98, 100,
101, 103, 105, 107, 44, 42, 41, 41, 42, 42, 48, 50, 54, 58, 63, 65, 67,
71, 75, 77, 79, 84, 90, 91, 92, 97, 100, 100, 100, 100, 101, 104, 108,
112, 115, 119, 53, 51, 49, 49, 50, 49, 54, 57, 60, 65, 71, 73, 76, 82,
87, 89, 92, 97, 104, 105, 106, 108, 106, 105, 107, 111, 114, 117, 117,
117, 118, 119, 59, 56, 54, 54, 54, 53, 58, 61, 64, 69, 75, 78, 80, 87,
92, 95, 98, 103, 110, 112, 113, 115, 114, 118, 123, 121, 120, 119, 123,
127, 131, 136, 65, 62, 59, 59, 59, 58, 63, 65, 68, 73, 79, 82, 85, 92,
98, 101, 105, 111, 118, 119, 121, 126, 130, 131, 128, 127, 131, 136,
138, 137, 136, 136, 79, 75, 72, 71, 71, 69, 73, 76, 78, 84, 90, 93, 96,
103, 110, 114, 118, 125, 133, 135, 136, 142, 142, 137, 140, 145, 144,
142, 141, 146, 151, 156, 87, 82, 78, 78, 77, 75, 79, 82, 84, 89, 95, 98,
102, 109, 116, 120, 124, 132, 141, 142, 144, 149, 148, 153, 157, 152,
150, 155, 161, 159, 157, 156, 90, 85, 82, 81, 80, 78, 78, 83, 87, 89,
93, 100, 102, 107, 115, 118, 123, 132, 136, 140, 151, 153, 155, 160,
161, 164, 170, 168, 165, 167, 172, 178, 93, 88, 86, 84, 82, 82, 80, 84,
86, 91, 94, 98, 105, 107, 112, 119, 122, 130, 135, 140, 149, 153, 162,
165, 167, 173, 174, 177, 183, 185, 182, 179, 96, 91, 90, 87, 86, 86, 83,
84, 89, 91, 95, 100, 102, 110, 111, 118, 123, 128, 135, 138, 149, 152,
160, 167, 173, 178, 180, 187, 188, 190, 197, 203, 99, 94, 93, 90, 89,
89, 88, 87, 90, 93, 97, 99, 105, 107, 115, 116, 124, 127, 135, 139, 146,
152, 159, 166, 171, 182, 186, 191, 193, 201, 203, 204, 102, 97, 97, 93,
93, 92, 92, 90, 90, 96, 97, 103, 104, 111, 112, 120, 121, 130, 131, 142,
143, 154, 155, 168, 169, 181, 183, 198, 200, 206, 208, 217,
// Size 4x16
31, 44, 79, 96, 32, 41, 72, 90, 32, 42, 71, 86, 34, 48, 73, 83, 34, 54,
78, 89, 41, 63, 90, 95, 45, 67, 96, 102, 54, 75, 110, 111, 60, 79, 118,
123, 72, 90, 133, 135, 75, 92, 136, 149, 83, 100, 142, 160, 88, 100,
140, 173, 94, 101, 144, 180, 101, 108, 141, 188, 108, 115, 151, 197,
// Size 16x4
31, 32, 32, 34, 34, 41, 45, 54, 60, 72, 75, 83, 88, 94, 101, 108, 44,
41, 42, 48, 54, 63, 67, 75, 79, 90, 92, 100, 100, 101, 108, 115, 79, 72,
71, 73, 78, 90, 96, 110, 118, 133, 136, 142, 140, 144, 141, 151, 96, 90,
86, 83, 89, 95, 102, 111, 123, 135, 149, 160, 173, 180, 188, 197,
// Size 8x32
32, 32, 36, 53, 65, 87, 93, 99, 31, 32, 35, 51, 62, 82, 88, 94, 31, 33,
34, 49, 59, 78, 86, 93, 31, 33, 35, 49, 59, 78, 84, 90, 32, 34, 36, 50,
59, 77, 82, 89, 32, 35, 38, 49, 58, 75, 82, 89, 34, 37, 42, 54, 63, 79,
80, 88, 35, 37, 45, 57, 65, 82, 84, 87, 36, 38, 48, 60, 68, 84, 86, 90,
39, 40, 50, 65, 73, 89, 91, 93, 44, 43, 53, 71, 79, 95, 94, 97, 46, 44,
55, 73, 82, 98, 98, 99, 48, 46, 56, 76, 85, 102, 105, 105, 53, 50, 60,
82, 92, 109, 107, 107, 58, 54, 63, 87, 98, 116, 112, 115, 61, 56, 66,
89, 101, 120, 119, 116, 65, 58, 68, 92, 105, 124, 122, 124, 71, 63, 73,
97, 111, 132, 130, 127, 79, 70, 79, 104, 118, 141, 135, 135, 81, 71, 80,
105, 119, 142, 140, 139, 82, 72, 81, 106, 121, 144, 149, 146, 88, 77,
85, 108, 126, 149, 153, 152, 91, 80, 88, 106, 130, 148, 162, 159, 94,
83, 91, 105, 131, 153, 165, 166, 97, 86, 94, 107, 128, 157, 167, 171,
100, 89, 97, 111, 127, 152, 173, 182, 103, 93, 98, 114, 131, 150, 174,
186, 107, 96, 100, 117, 136, 155, 177, 191, 110, 100, 101, 117, 138,
161, 183, 193, 114, 104, 103, 117, 137, 159, 185, 201, 118, 107, 105,
118, 136, 157, 182, 203, 122, 111, 107, 119, 136, 156, 179, 204,
// Size 32x8
32, 31, 31, 31, 32, 32, 34, 35, 36, 39, 44, 46, 48, 53, 58, 61, 65, 71,
79, 81, 82, 88, 91, 94, 97, 100, 103, 107, 110, 114, 118, 122, 32, 32,
33, 33, 34, 35, 37, 37, 38, 40, 43, 44, 46, 50, 54, 56, 58, 63, 70, 71,
72, 77, 80, 83, 86, 89, 93, 96, 100, 104, 107, 111, 36, 35, 34, 35, 36,
38, 42, 45, 48, 50, 53, 55, 56, 60, 63, 66, 68, 73, 79, 80, 81, 85, 88,

```

```

91, 94, 97, 98, 100, 101, 103, 105, 107, 53, 51, 49, 49, 50, 49, 54, 57,
60, 65, 71, 73, 76, 82, 87, 89, 92, 97, 104, 105, 106, 108, 106, 105,
107, 111, 114, 117, 117, 117, 118, 119, 65, 62, 59, 59, 59, 58, 63, 65,
68, 73, 79, 82, 85, 92, 98, 101, 105, 111, 118, 119, 121, 126, 130, 131,
128, 127, 131, 136, 138, 137, 136, 136, 87, 82, 78, 78, 77, 75, 79, 82,
84, 89, 95, 98, 102, 109, 116, 120, 124, 132, 141, 142, 144, 149, 148,
153, 157, 152, 150, 155, 161, 159, 157, 156, 93, 88, 86, 84, 82, 82, 80,
84, 86, 91, 94, 98, 105, 107, 112, 119, 122, 130, 135, 140, 149, 153,
162, 165, 167, 173, 174, 177, 183, 185, 182, 179, 99, 94, 93, 90, 89,
89, 88, 87, 90, 93, 97, 99, 105, 107, 115, 116, 124, 127, 135, 139, 146,
152, 159, 166, 171, 182, 186, 191, 193, 201, 203, 204,
// Size 4x32
31, 44, 79, 96, 32, 42, 75, 91, 32, 41, 72, 90, 32, 41, 71, 87, 32, 42,
71, 86, 33, 42, 69, 86, 34, 48, 73, 83, 34, 50, 76, 84, 34, 54, 78, 89,
37, 58, 84, 91, 41, 63, 90, 95, 43, 65, 93, 100, 45, 67, 96, 102, 49,
71, 103, 110, 54, 75, 110, 111, 57, 77, 114, 118, 60, 79, 118, 123, 65,
84, 125, 128, 72, 90, 133, 135, 74, 91, 135, 138, 75, 92, 136, 149, 80,
97, 142, 152, 83, 100, 142, 160, 85, 100, 137, 167, 88, 100, 140, 173,
91, 100, 145, 178, 94, 101, 144, 180, 97, 104, 142, 187, 101, 108, 141,
188, 104, 112, 146, 190, 108, 115, 151, 197, 111, 119, 156, 203,
// Size 32x4
31, 32, 32, 32, 32, 33, 34, 34, 34, 37, 41, 43, 45, 49, 54, 57, 60, 65,
72, 74, 75, 80, 83, 85, 88, 91, 94, 97, 101, 104, 108, 111, 44, 42, 41,
41, 42, 42, 48, 50, 54, 58, 63, 65, 67, 71, 75, 77, 79, 84, 90, 91, 92,
97, 100, 100, 100, 100, 101, 104, 108, 112, 115, 119, 79, 75, 72, 71,
71, 69, 73, 76, 78, 84, 90, 93, 96, 103, 110, 114, 118, 125, 133, 135,
136, 142, 142, 137, 140, 145, 144, 142, 141, 146, 151, 156, 96, 91, 90,
87, 86, 86, 83, 84, 89, 91, 95, 100, 102, 110, 111, 118, 123, 128, 135,
138, 149, 152, 160, 167, 173, 178, 180, 187, 188, 190, 197, 203,
},
{ // Chroma
// Size 4x4
35, 46, 57, 66, 46, 60, 69, 71, 57, 69, 90, 90, 66, 71, 90, 109,
// Size 8x8
31, 38, 47, 50, 57, 63, 67, 71, 38, 47, 46, 47, 52, 57, 62, 67, 47, 46,
54, 57, 61, 66, 67, 68, 50, 47, 57, 66, 72, 77, 75, 75, 57, 52, 61, 72,
82, 88, 86, 84, 63, 57, 66, 77, 88, 96, 95, 95, 67, 62, 67, 75, 86, 95,
104, 107, 71, 67, 68, 75, 84, 95, 107, 113,
// Size 16x16
32, 30, 33, 41, 49, 49, 50, 54, 57, 63, 65, 68, 70, 72, 74, 76, 30, 32,
35, 42, 46, 45, 46, 49, 52, 57, 58, 62, 64, 67, 70, 72, 33, 35, 39, 45,
47, 45, 46, 49, 51, 56, 57, 60, 62, 64, 66, 69, 41, 42, 45, 48, 50, 49,
50, 52, 53, 57, 58, 59, 60, 61, 64, 67, 49, 46, 47, 50, 53, 53, 54, 55,
56, 60, 61, 64, 64, 65, 66, 66, 49, 45, 45, 49, 53, 58, 60, 62, 63, 67,
68, 67, 69, 68, 70, 70, 50, 46, 46, 50, 54, 60, 61, 65, 67, 71, 71, 74,
73, 73, 74, 74, 54, 49, 49, 52, 55, 62, 65, 71, 73, 78, 79, 78, 77, 78,
78, 78, 57, 52, 51, 53, 56, 63, 67, 73, 76, 82, 83, 84, 84, 84, 82, 83,
63, 57, 56, 57, 60, 67, 71, 78, 82, 89, 90, 90, 89, 88, 87, 88, 65, 58,
57, 58, 61, 68, 71, 79, 83, 90, 91, 94, 93, 93, 92, 93, 68, 62, 60, 59,
64, 67, 74, 78, 84, 90, 94, 98, 99, 98, 98, 98, 70, 64, 62, 60, 64, 69,
73, 77, 84, 89, 93, 99, 102, 103, 104, 104, 72, 67, 64, 61, 65, 68, 73,
78, 84, 88, 93, 98, 103, 106, 108, 109, 74, 70, 66, 64, 66, 70, 74, 78,
82, 87, 92, 98, 104, 108, 111, 112, 76, 72, 69, 67, 66, 70, 74, 78, 83,
88, 93, 98, 104, 109, 112, 116,
// Size 32x32
32, 31, 30, 32, 33, 36, 41, 45, 49, 48, 49, 50, 50, 52, 54, 56, 57, 60,
63, 64, 65, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 78, 31, 31, 31, 33,
34, 38, 42, 45, 47, 47, 47, 47, 48, 50, 52, 53, 54, 57, 60, 61, 61, 63,
64, 65, 66, 67, 68, 69, 70, 71, 72, 74, 30, 31, 32, 33, 35, 40, 42, 44,
46, 45, 45, 45, 46, 47, 49, 51, 52, 54, 57, 58, 58, 61, 62, 63, 64, 66,
67, 68, 70, 71, 72, 74, 32, 33, 33, 35, 37, 41, 43, 45, 47, 46, 45, 46,
46, 47, 49, 50, 51, 54, 57, 57, 58, 60, 61, 62, 63, 64, 65, 66, 67, 68,
69, 70, 33, 34, 35, 37, 39, 43, 45, 46, 47, 46, 45, 46, 46, 47, 49, 50,
51, 53, 56, 57, 57, 59, 60, 61, 62, 63, 64, 65, 66, 68, 69, 70, 36, 38,
40, 41, 43, 47, 47, 47, 48, 46, 45, 46, 46, 47, 48, 49, 50, 52, 54, 55,
55, 57, 58, 59, 61, 62, 64, 65, 66, 67, 68, 69, 41, 42, 42, 43, 45, 47,
48, 49, 50, 49, 49, 49, 50, 52, 52, 53, 55, 57, 58, 58, 60, 59, 59,
60, 61, 61, 63, 64, 66, 67, 69, 45, 45, 44, 45, 46, 47, 49, 50, 51, 51,
51, 51, 52, 52, 53, 54, 55, 57, 59, 59, 60, 61, 61, 62, 63, 63, 63, 63,
63, 64, 65, 66, 49, 47, 46, 47, 47, 48, 50, 51, 53, 53, 53, 54, 54, 54,
55, 56, 56, 58, 60, 61, 61, 63, 64, 64, 64, 64, 65, 66, 66, 66, 66, 66,

```

48, 47, 45, 46, 46, 46, 49, 51, 53, 54, 55, 56, 56, 57, 58, 59, 60, 61,
 63, 64, 64, 66, 66, 65, 66, 67, 67, 67, 67, 68, 69, 70, 49, 47, 45, 45,
 45, 45, 49, 51, 53, 55, 58, 59, 60, 61, 62, 63, 63, 65, 67, 67, 68, 69,
 67, 68, 69, 68, 68, 69, 70, 70, 70, 70, 50, 47, 45, 46, 46, 46, 49, 51,
 54, 56, 59, 60, 60, 62, 64, 64, 65, 67, 69, 69, 70, 70, 71, 71, 70, 70,
 71, 71, 71, 71, 72, 74, 50, 48, 46, 46, 46, 46, 50, 52, 54, 56, 60, 60,
 61, 63, 65, 66, 67, 68, 71, 71, 71, 73, 74, 72, 73, 74, 73, 73, 74, 74,
 74, 74, 52, 50, 47, 47, 47, 47, 50, 52, 54, 57, 61, 62, 63, 66, 68, 69,
 70, 72, 75, 75, 75, 77, 75, 75, 76, 75, 75, 76, 75, 75, 76, 77, 54, 52,
 49, 49, 49, 48, 52, 53, 55, 58, 62, 64, 65, 68, 71, 72, 73, 75, 78, 78,
 79, 79, 78, 79, 77, 78, 78, 77, 78, 79, 78, 78, 56, 53, 51, 50, 50, 49,
 52, 54, 56, 59, 63, 64, 66, 69, 72, 73, 75, 77, 80, 80, 81, 81, 82, 80,
 81, 81, 79, 81, 80, 79, 81, 82, 57, 54, 52, 51, 51, 50, 53, 55, 56, 60,
 63, 65, 67, 70, 73, 75, 76, 79, 82, 82, 83, 85, 84, 83, 84, 83, 84, 82,
 82, 84, 83, 82, 60, 57, 54, 54, 53, 52, 55, 57, 58, 61, 65, 67, 68, 72,
 75, 77, 79, 82, 85, 85, 86, 88, 86, 87, 85, 86, 85, 85, 86, 84, 85, 86,
 63, 60, 57, 57, 56, 54, 57, 59, 60, 63, 67, 69, 71, 75, 78, 80, 82, 85,
 89, 89, 90, 90, 90, 89, 89, 88, 88, 88, 87, 88, 88, 87, 64, 61, 58, 57,
 57, 55, 58, 59, 61, 64, 67, 69, 71, 75, 78, 80, 82, 85, 89, 90, 91, 92,
 93, 92, 92, 91, 91, 90, 91, 90, 90, 92, 65, 61, 58, 58, 57, 55, 58, 60,
 61, 64, 68, 70, 71, 75, 79, 81, 83, 86, 90, 91, 91, 94, 94, 96, 93, 94,
 93, 94, 92, 93, 93, 92, 67, 63, 61, 60, 59, 57, 60, 61, 63, 66, 69, 70,
 73, 77, 79, 81, 85, 88, 90, 92, 94, 96, 96, 97, 98, 95, 97, 95, 96, 95,
 95, 96, 68, 64, 62, 61, 60, 58, 59, 61, 64, 66, 67, 71, 74, 75, 78, 82,
 84, 86, 90, 93, 94, 96, 98, 98, 99, 100, 98, 99, 98, 98, 98, 97, 69, 65,
 63, 62, 61, 59, 59, 62, 64, 65, 68, 71, 72, 75, 79, 80, 83, 87, 89, 92,
 96, 97, 98, 100, 100, 101, 102, 101, 101, 101, 100, 102, 70, 66, 64, 63,
 62, 61, 60, 63, 64, 66, 69, 70, 73, 76, 77, 81, 84, 85, 89, 92, 93, 98,
 99, 100, 102, 102, 103, 104, 104, 103, 104, 102, 71, 67, 66, 64, 63, 62,
 61, 63, 64, 67, 68, 70, 74, 75, 78, 81, 83, 86, 88, 91, 94, 95, 100,
 101, 102, 104, 104, 105, 106, 107, 105, 107, 72, 68, 67, 65, 64, 64, 61,
 63, 65, 67, 68, 71, 73, 75, 78, 79, 84, 85, 88, 91, 93, 97, 98, 102,
 103, 104, 106, 106, 108, 108, 109, 107, 73, 69, 68, 66, 65, 65, 63, 63,
 66, 67, 69, 71, 73, 76, 77, 81, 82, 85, 88, 90, 94, 95, 99, 101, 104,
 105, 106, 109, 108, 110, 111, 112, 74, 70, 70, 67, 66, 66, 64, 63, 66,
 67, 70, 71, 74, 75, 78, 80, 82, 86, 87, 91, 92, 96, 98, 101, 104, 106,
 108, 108, 111, 111, 112, 113, 75, 71, 71, 68, 68, 67, 66, 64, 66, 68,
 70, 71, 74, 75, 79, 79, 84, 84, 88, 90, 93, 95, 98, 101, 103, 107, 108,
 110, 111, 113, 113, 115, 76, 72, 72, 69, 69, 68, 67, 65, 66, 69, 70, 72,
 74, 76, 78, 81, 83, 85, 88, 90, 93, 95, 98, 100, 104, 105, 109, 111,
 112, 113, 116, 115, 78, 74, 74, 70, 70, 69, 66, 66, 70, 70, 74, 74,
 77, 78, 82, 82, 86, 87, 92, 92, 96, 97, 102, 102, 107, 107, 112, 113,
 115, 115, 118,
 // Size 4x8
 31, 47, 60, 66, 40, 45, 54, 61, 46, 56, 64, 64, 48, 61, 75, 73, 54, 65,
 85, 82, 61, 69, 92, 92, 64, 68, 90, 102, 68, 71, 87, 105,
 // Size 8x4
 31, 40, 46, 48, 54, 61, 64, 68, 47, 45, 56, 61, 65, 69, 68, 71, 60, 54,
 64, 75, 85, 92, 90, 87, 66, 61, 64, 73, 82, 92, 102, 105,
 // Size 8x16
 32, 37, 48, 52, 57, 66, 68, 71, 30, 40, 46, 48, 52, 60, 63, 66, 33, 43,
 47, 47, 51, 59, 60, 63, 42, 47, 50, 50, 53, 60, 59, 62, 49, 48, 53, 54,
 57, 62, 62, 62, 49, 46, 53, 61, 64, 69, 66, 66, 50, 46, 54, 64, 67, 73,
 72, 70, 54, 49, 55, 68, 73, 80, 76, 75, 57, 50, 56, 70, 76, 84, 80, 79,
 63, 55, 60, 75, 82, 92, 87, 84, 64, 56, 61, 75, 83, 93, 93, 89, 68, 59,
 64, 74, 86, 94, 98, 94, 70, 62, 66, 73, 83, 96, 99, 98, 72, 64, 66, 75,
 83, 92, 101, 104, 74, 67, 66, 74, 84, 94, 103, 106, 76, 69, 67, 73, 82,
 91, 101, 109,
 // Size 16x8
 32, 30, 33, 42, 49, 49, 50, 54, 57, 63, 64, 68, 70, 72, 74, 76, 37, 40,
 43, 47, 48, 46, 46, 49, 50, 55, 56, 59, 62, 64, 67, 69, 48, 46, 47, 50,
 53, 53, 54, 55, 56, 60, 61, 64, 66, 66, 66, 67, 52, 48, 47, 50, 54, 61,
 64, 68, 70, 75, 75, 74, 73, 75, 74, 73, 57, 52, 51, 53, 57, 64, 67, 73,
 76, 82, 83, 86, 83, 83, 84, 82, 66, 60, 59, 60, 62, 69, 73, 80, 84, 92,
 93, 94, 96, 92, 94, 91, 68, 63, 60, 59, 62, 66, 72, 76, 80, 87, 93, 98,
 99, 101, 103, 101, 71, 66, 63, 62, 62, 66, 70, 75, 79, 84, 89, 94, 98,
 104, 106, 109,
 // Size 16x32
 32, 31, 37, 42, 48, 49, 52, 54, 57, 63, 66, 67, 68, 69, 71, 72, 31, 31,
 38, 42, 47, 47, 50, 52, 54, 60, 63, 64, 65, 66, 67, 68, 30, 32, 40, 42,
 46, 45, 48, 50, 52, 57, 60, 62, 63, 65, 66, 68, 32, 34, 41, 44, 46, 45,

```

48, 49, 51, 57, 59, 61, 62, 63, 64, 65, 33, 36, 43, 45, 47, 46, 47, 49,
51, 56, 59, 60, 60, 62, 63, 65, 37, 40, 47, 47, 47, 45, 47, 48, 50, 54,
57, 58, 60, 61, 62, 63, 42, 43, 47, 48, 50, 49, 50, 52, 53, 57, 60, 58,
59, 60, 62, 63, 45, 44, 47, 49, 51, 51, 52, 54, 55, 59, 61, 61, 61, 60,
61, 61, 49, 46, 48, 50, 53, 53, 54, 55, 57, 60, 62, 63, 62, 63, 62, 62,
48, 46, 47, 50, 53, 56, 57, 59, 60, 64, 66, 65, 65, 64, 64, 65, 49, 45,
46, 49, 53, 58, 61, 62, 64, 67, 69, 67, 66, 66, 66, 65, 49, 46, 46, 49,
53, 59, 62, 64, 65, 69, 71, 70, 68, 68, 67, 68, 50, 46, 46, 50, 54, 59,
64, 65, 67, 71, 73, 72, 72, 70, 70, 69, 52, 48, 47, 50, 54, 61, 66, 68,
71, 75, 77, 74, 73, 73, 71, 72, 54, 50, 49, 52, 55, 62, 68, 71, 73, 78,
80, 78, 76, 74, 75, 73, 55, 51, 49, 52, 56, 63, 69, 72, 75, 80, 82, 80,
79, 78, 76, 77, 57, 52, 50, 53, 56, 64, 70, 73, 76, 82, 84, 82, 80, 80,
79, 77, 60, 54, 52, 55, 58, 65, 72, 75, 79, 85, 88, 86, 84, 82, 81, 81,
63, 57, 55, 58, 60, 67, 75, 78, 82, 89, 92, 88, 87, 85, 84, 81, 64, 58,
55, 58, 61, 68, 75, 78, 82, 89, 92, 90, 89, 87, 86, 86, 64, 59, 56, 58,
61, 68, 75, 79, 83, 90, 93, 95, 93, 91, 89, 87, 67, 61, 58, 60, 63, 69,
76, 79, 85, 92, 95, 96, 94, 92, 91, 91, 68, 62, 59, 60, 64, 71, 74, 78,
86, 91, 94, 96, 98, 96, 94, 91, 69, 62, 60, 60, 65, 70, 72, 79, 85, 88,
95, 98, 99, 98, 97, 96, 70, 63, 62, 60, 66, 69, 73, 81, 83, 89, 96, 97,
99, 101, 98, 97, 71, 64, 63, 61, 67, 68, 74, 79, 82, 90, 93, 98, 102,
102, 102, 101, 72, 65, 64, 62, 66, 68, 75, 78, 83, 89, 92, 100, 101,
103, 104, 102, 73, 66, 65, 63, 66, 69, 75, 76, 84, 87, 93, 98, 102, 105,
106, 107, 74, 67, 67, 64, 66, 70, 74, 77, 84, 86, 94, 96, 103, 105, 106,
107, 75, 68, 68, 65, 66, 71, 74, 78, 83, 87, 93, 96, 103, 105, 109, 109,
76, 69, 69, 66, 67, 72, 73, 80, 82, 88, 91, 97, 101, 107, 109, 110, 77,
70, 70, 67, 67, 73, 73, 81, 81, 90, 90, 99, 99, 108, 108, 113,
// Size 32x16
32, 31, 30, 32, 33, 37, 42, 45, 49, 48, 49, 49, 50, 52, 54, 55, 57, 60,
63, 64, 64, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 31, 31, 32, 34,
36, 40, 43, 44, 46, 46, 45, 46, 46, 48, 50, 51, 52, 54, 57, 58, 59, 61,
62, 62, 63, 64, 65, 66, 67, 68, 69, 70, 37, 38, 40, 41, 43, 47, 47, 47,
48, 47, 46, 46, 46, 47, 49, 49, 50, 52, 55, 55, 56, 58, 59, 60, 62, 63,
64, 65, 67, 68, 69, 70, 42, 42, 42, 44, 45, 47, 48, 49, 50, 50, 49, 49,
50, 50, 52, 52, 53, 55, 58, 58, 58, 60, 60, 60, 60, 61, 62, 63, 64, 65,
66, 67, 48, 47, 46, 46, 47, 47, 50, 51, 53, 53, 53, 53, 54, 54, 55, 56,
56, 58, 60, 61, 61, 63, 64, 65, 66, 67, 66, 66, 66, 67, 67, 49, 47,
45, 45, 46, 45, 49, 51, 53, 56, 58, 59, 59, 61, 62, 63, 64, 65, 67, 68,
68, 69, 71, 70, 69, 68, 68, 69, 70, 71, 72, 73, 52, 50, 48, 48, 47, 47,
50, 52, 54, 57, 61, 62, 64, 66, 68, 69, 70, 72, 75, 75, 75, 76, 74, 72,
73, 74, 75, 75, 74, 74, 73, 73, 54, 52, 50, 49, 49, 48, 52, 54, 55, 59,
62, 64, 65, 68, 71, 72, 73, 75, 78, 78, 79, 79, 78, 79, 81, 79, 78, 76,
77, 78, 80, 81, 57, 54, 52, 51, 51, 50, 53, 55, 57, 60, 64, 65, 67, 71,
73, 75, 76, 79, 82, 82, 83, 85, 86, 85, 83, 82, 83, 84, 84, 83, 82, 81,
63, 60, 57, 57, 56, 54, 57, 59, 60, 64, 67, 69, 71, 75, 78, 80, 82, 85,
89, 89, 90, 92, 91, 88, 89, 90, 89, 87, 86, 87, 88, 90, 66, 63, 60, 59,
59, 57, 60, 61, 62, 66, 69, 71, 73, 77, 80, 82, 84, 88, 92, 92, 93, 95,
94, 95, 96, 93, 92, 93, 94, 93, 91, 90, 67, 64, 62, 61, 60, 58, 58, 61,
63, 65, 67, 70, 72, 74, 78, 80, 82, 86, 88, 90, 95, 96, 96, 98, 97, 98,
100, 98, 96, 96, 97, 99, 68, 65, 63, 62, 60, 60, 59, 61, 62, 65, 66, 68,
72, 73, 76, 79, 80, 84, 87, 89, 93, 94, 98, 99, 99, 102, 101, 102, 103,
103, 101, 99, 69, 66, 65, 63, 62, 61, 60, 60, 63, 64, 66, 68, 70, 73,
74, 78, 80, 82, 85, 87, 91, 92, 96, 98, 101, 102, 103, 105, 105, 105,
107, 108, 71, 67, 66, 64, 63, 62, 62, 61, 62, 64, 66, 67, 70, 71, 75,
76, 79, 81, 84, 86, 89, 91, 94, 97, 98, 102, 104, 106, 106, 109, 109,
108, 72, 68, 68, 65, 65, 63, 63, 61, 62, 65, 65, 68, 69, 72, 73, 77, 77,
81, 81, 86, 87, 91, 91, 96, 97, 101, 102, 107, 107, 109, 110, 113,
// Size 4x16
31, 49, 63, 69, 32, 45, 57, 65, 36, 46, 56, 62, 43, 49, 57, 60, 46, 53,
60, 63, 45, 58, 67, 66, 46, 59, 71, 70, 50, 62, 78, 74, 52, 64, 82, 80,
57, 67, 89, 85, 59, 68, 90, 91, 62, 71, 91, 96, 63, 69, 89, 101, 65, 68,
89, 103, 67, 70, 86, 105, 69, 72, 88, 107,
// Size 16x4
31, 32, 36, 43, 46, 45, 46, 50, 52, 57, 59, 62, 63, 65, 67, 69, 49, 45,
46, 49, 53, 58, 59, 62, 64, 67, 68, 71, 69, 68, 70, 72, 63, 57, 56, 57,
60, 67, 71, 78, 82, 89, 90, 91, 89, 89, 86, 88, 69, 65, 62, 60, 63, 66,
70, 74, 80, 85, 91, 96, 101, 103, 105, 107,
// Size 8x32
32, 37, 48, 52, 57, 66, 68, 71, 31, 38, 47, 50, 54, 63, 65, 67, 30, 40,
46, 48, 52, 60, 63, 66, 32, 41, 46, 48, 51, 59, 62, 64, 33, 43, 47, 47,
51, 59, 60, 63, 37, 47, 47, 47, 50, 57, 60, 62, 42, 47, 50, 50, 53, 60,
59, 62, 45, 47, 51, 52, 55, 61, 61, 61, 49, 48, 53, 54, 57, 62, 62, 62,

```

```

48, 47, 53, 57, 60, 66, 65, 64, 49, 46, 53, 61, 64, 69, 66, 66, 49, 46,
53, 62, 65, 71, 68, 67, 50, 46, 54, 64, 67, 73, 72, 70, 52, 47, 54, 66,
71, 77, 73, 71, 54, 49, 55, 68, 73, 80, 76, 75, 55, 49, 56, 69, 75, 82,
79, 76, 57, 50, 56, 70, 76, 84, 80, 79, 60, 52, 58, 72, 79, 88, 84, 81,
63, 55, 60, 75, 82, 92, 87, 84, 64, 55, 61, 75, 82, 92, 89, 86, 64, 56,
61, 75, 83, 93, 93, 89, 67, 58, 63, 76, 85, 95, 94, 91, 68, 59, 64, 74,
86, 94, 98, 94, 69, 60, 65, 72, 85, 95, 99, 97, 70, 62, 66, 73, 83, 96,
99, 98, 71, 63, 67, 74, 82, 93, 102, 102, 72, 64, 66, 75, 83, 92, 101,
104, 73, 65, 66, 75, 84, 93, 102, 106, 74, 67, 66, 74, 84, 94, 103, 106,
75, 68, 66, 74, 83, 93, 103, 109, 76, 69, 67, 73, 82, 91, 101, 109, 77,
70, 67, 73, 81, 90, 99, 108,
// Size 32x8
32, 31, 30, 32, 33, 37, 42, 45, 49, 48, 49, 49, 50, 52, 54, 55, 57, 60,
63, 64, 64, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 37, 38, 40, 41,
43, 47, 47, 47, 48, 47, 46, 46, 46, 47, 49, 49, 50, 52, 55, 55, 56, 58,
59, 60, 62, 63, 64, 65, 67, 68, 69, 70, 48, 47, 46, 46, 47, 47, 50, 51,
53, 53, 53, 53, 54, 54, 55, 56, 56, 58, 60, 61, 61, 63, 64, 65, 66, 67,
66, 66, 66, 66, 67, 67, 52, 50, 48, 48, 47, 47, 50, 52, 54, 57, 61, 62,
64, 66, 68, 69, 70, 72, 75, 75, 75, 76, 74, 72, 73, 74, 75, 75, 74, 74,
73, 73, 57, 54, 52, 51, 51, 50, 53, 55, 57, 60, 64, 65, 67, 71, 73, 75,
76, 79, 82, 82, 83, 85, 86, 85, 83, 82, 83, 84, 84, 83, 82, 81, 66, 63,
60, 59, 59, 57, 60, 61, 62, 66, 69, 71, 73, 77, 80, 82, 84, 88, 92, 92,
93, 95, 94, 95, 96, 93, 92, 93, 94, 93, 91, 90, 68, 65, 63, 62, 60, 60,
59, 61, 62, 65, 66, 68, 72, 73, 76, 79, 80, 84, 87, 89, 93, 94, 98, 99,
99, 102, 101, 102, 103, 103, 101, 99, 71, 67, 66, 64, 63, 62, 62, 61,
62, 64, 66, 67, 70, 71, 75, 76, 79, 81, 84, 86, 89, 91, 94, 97, 98, 102,
104, 106, 106, 109, 109, 108,
// Size 4x32
31, 49, 63, 69, 31, 47, 60, 66, 32, 45, 57, 65, 34, 45, 57, 63, 36, 46,
56, 62, 40, 45, 54, 61, 43, 49, 57, 60, 44, 51, 59, 60, 46, 53, 60, 63,
46, 56, 64, 64, 45, 58, 67, 66, 46, 59, 69, 68, 46, 59, 71, 70, 48, 61,
75, 73, 50, 62, 78, 74, 51, 63, 80, 78, 52, 64, 82, 80, 54, 65, 85, 82,
57, 67, 89, 85, 58, 68, 89, 87, 59, 68, 90, 91, 61, 69, 92, 92, 62, 71,
91, 96, 62, 70, 88, 98, 63, 69, 89, 101, 64, 68, 90, 102, 65, 68, 89,
103, 66, 69, 87, 105, 67, 70, 86, 105, 68, 71, 87, 105, 69, 72, 88, 107,
70, 73, 90, 108,
// Size 32x4
31, 31, 32, 34, 36, 40, 43, 44, 46, 46, 45, 46, 46, 48, 50, 51, 52, 54,
57, 58, 59, 61, 62, 62, 63, 64, 65, 66, 67, 68, 69, 70, 49, 47, 45, 45,
46, 45, 49, 51, 53, 56, 58, 59, 59, 61, 62, 63, 64, 65, 67, 68, 68, 69,
71, 70, 69, 68, 68, 69, 70, 71, 72, 73, 63, 60, 57, 57, 56, 54, 57, 59,
60, 64, 67, 69, 71, 75, 78, 80, 82, 85, 89, 89, 90, 92, 91, 88, 89, 90,
89, 87, 86, 87, 88, 90, 69, 66, 65, 63, 62, 61, 60, 60, 63, 64, 66, 68,
70, 73, 74, 78, 80, 82, 85, 87, 91, 92, 96, 98, 101, 102, 103, 105, 105,
105, 107, 108,

```

```

},

```

```

{

```

```

{ // Luma
// Size 4x4
32, 41, 69, 92, 41, 63, 88, 103, 69, 88, 127, 140, 92, 103, 140, 184,
// Size 8x8
32, 32, 37, 47, 62, 78, 90, 102, 32, 35, 39, 46, 58, 72, 84, 96, 37, 39,
51, 60, 71, 84, 93, 100, 47, 46, 60, 73, 87, 100, 106, 113, 62, 58, 71,
87, 105, 121, 129, 132, 78, 72, 84, 100, 121, 140, 148, 155, 90, 84, 93,
106, 129, 148, 169, 183, 102, 96, 100, 113, 132, 155, 183, 201,
// Size 16x16
32, 31, 31, 32, 36, 39, 47, 54, 61, 71, 80, 86, 92, 98, 104, 111, 31,
32, 32, 33, 34, 37, 44, 50, 56, 65, 73, 79, 85, 91, 98, 105, 31, 32, 33,
34, 36, 39, 45, 50, 56, 64, 71, 77, 82, 88, 94, 100, 32, 33, 34, 36, 40,
42, 47, 51, 57, 65, 71, 76, 80, 85, 91, 98, 36, 34, 36, 40, 48, 50, 56,
60, 65, 73, 79, 84, 86, 90, 95, 98, 39, 37, 39, 42, 50, 54, 60, 65, 70,
78, 84, 89, 95, 96, 102, 105, 47, 44, 45, 47, 56, 60, 69, 75, 81, 89,
95, 100, 102, 104, 109, 112, 54, 50, 50, 51, 60, 65, 75, 82, 89, 97,
104, 109, 110, 114, 117, 121, 61, 56, 56, 57, 65, 70, 81, 89, 97, 106,
113, 119, 122, 126, 125, 130, 71, 65, 64, 65, 73, 78, 89, 97, 106, 117,
125, 131, 134, 134, 136, 141, 80, 73, 71, 71, 79, 84, 95, 104, 113, 125,
134, 140, 142, 145, 146, 152, 86, 79, 77, 76, 84, 89, 100, 109, 119,
131, 140, 147, 154, 157, 160, 165, 92, 85, 82, 80, 86, 95, 102, 110,
122, 134, 142, 154, 162, 168, 174, 178, 98, 91, 88, 85, 90, 96, 104,
114, 126, 134, 145, 157, 168, 176, 184, 193, 104, 98, 94, 91, 95, 102,

```

```

109, 117, 125, 136, 146, 160, 174, 184, 193, 201, 111, 105, 100, 98, 98,
105, 112, 121, 130, 141, 152, 165, 178, 193, 201, 210,
// Size 32x32
32, 31, 31, 31, 31, 32, 32, 34, 36, 38, 39, 44, 47, 49, 54, 59, 61, 65,
71, 76, 80, 83, 86, 89, 92, 95, 98, 101, 104, 108, 111, 114, 31, 32, 32,
32, 32, 32, 33, 34, 35, 37, 38, 42, 45, 47, 51, 56, 58, 62, 68, 72, 76,
78, 82, 85, 88, 90, 93, 96, 99, 102, 105, 109, 31, 32, 32, 32, 32, 32,
33, 33, 34, 36, 37, 41, 44, 46, 50, 54, 56, 60, 65, 70, 73, 76, 79, 82,
85, 88, 91, 95, 98, 101, 105, 109, 31, 32, 32, 32, 32, 33, 33, 34, 35,
36, 38, 41, 44, 45, 49, 54, 56, 59, 65, 69, 72, 75, 78, 81, 84, 86, 89,
92, 95, 98, 101, 104, 31, 32, 32, 32, 33, 34, 34, 35, 36, 38, 39, 42,
45, 46, 50, 54, 56, 59, 64, 68, 71, 74, 77, 79, 82, 85, 88, 91, 94, 97,
100, 104, 32, 32, 32, 33, 34, 35, 36, 37, 38, 39, 40, 42, 45, 46, 49,
53, 55, 58, 63, 66, 69, 72, 74, 78, 81, 84, 87, 90, 93, 96, 99, 102, 32,
33, 33, 33, 34, 36, 36, 38, 40, 41, 42, 44, 47, 48, 51, 55, 57, 60, 65,
68, 71, 73, 76, 78, 80, 82, 85, 88, 91, 95, 98, 102, 34, 34, 33, 34, 35,
37, 38, 39, 42, 44, 45, 47, 50, 51, 54, 58, 60, 63, 68, 71, 74, 76, 79,
82, 85, 86, 87, 88, 90, 93, 96, 99, 36, 35, 34, 35, 36, 38, 40, 42, 48,
50, 50, 54, 56, 57, 60, 64, 65, 68, 73, 76, 79, 81, 84, 86, 86, 88, 90,
93, 95, 97, 98, 100, 38, 37, 36, 36, 38, 39, 41, 44, 50, 51, 52, 56, 58,
60, 63, 67, 68, 71, 76, 79, 82, 84, 87, 87, 90, 93, 94, 95, 96, 100,
103, 106, 39, 38, 37, 38, 39, 40, 42, 45, 50, 52, 54, 58, 60, 62, 65,
69, 70, 73, 78, 81, 84, 86, 89, 92, 95, 95, 96, 99, 102, 104, 105, 106,
44, 42, 41, 41, 42, 42, 44, 47, 54, 56, 58, 63, 66, 68, 71, 75, 77, 79,
84, 88, 90, 92, 95, 97, 97, 99, 102, 103, 103, 106, 109, 113, 47, 45,
44, 44, 45, 45, 47, 50, 56, 58, 60, 66, 69, 71, 75, 79, 81, 84, 89, 92,
95, 97, 100, 100, 102, 105, 104, 106, 109, 111, 112, 113, 49, 47, 46,
45, 46, 46, 48, 51, 57, 60, 62, 68, 71, 73, 77, 81, 83, 87, 92, 95, 98,
100, 103, 105, 107, 106, 109, 112, 112, 113, 117, 120, 54, 51, 50, 49,
50, 49, 51, 54, 60, 63, 65, 71, 75, 77, 82, 87, 89, 92, 97, 101, 104,
106, 109, 112, 110, 113, 114, 114, 117, 121, 121, 121, 59, 56, 54, 54,
54, 53, 55, 58, 64, 67, 69, 75, 79, 81, 87, 92, 94, 98, 103, 107, 110,
113, 116, 114, 117, 118, 117, 121, 122, 122, 125, 129, 61, 58, 56, 56,
56, 55, 57, 60, 65, 68, 70, 77, 81, 83, 89, 94, 97, 101, 106, 110, 113,
116, 119, 120, 122, 121, 126, 124, 125, 130, 130, 130, 65, 62, 60, 59,
59, 58, 60, 63, 68, 71, 73, 79, 84, 87, 92, 98, 101, 105, 111, 115, 118,
121, 124, 128, 125, 129, 128, 131, 133, 132, 135, 139, 71, 68, 65, 65,
64, 63, 65, 68, 73, 76, 78, 84, 89, 92, 97, 103, 106, 111, 117, 122,
125, 128, 131, 131, 134, 132, 134, 136, 136, 140, 141, 140, 76, 72, 70,
69, 68, 66, 68, 71, 76, 79, 81, 88, 92, 95, 101, 107, 110, 115, 122,
127, 130, 133, 136, 136, 138, 139, 141, 140, 145, 143, 146, 151, 80, 76,
73, 72, 71, 69, 71, 74, 79, 82, 84, 90, 95, 98, 104, 110, 113, 118, 125,
130, 134, 137, 140, 146, 142, 146, 145, 149, 146, 150, 152, 151, 83, 78,
76, 75, 74, 72, 73, 76, 81, 84, 86, 92, 97, 100, 106, 113, 116, 121,
128, 133, 137, 140, 144, 147, 152, 148, 154, 151, 156, 155, 156, 162,
86, 82, 79, 78, 77, 74, 76, 79, 84, 87, 89, 95, 100, 103, 109, 116, 119,
124, 131, 136, 140, 144, 147, 150, 154, 159, 157, 160, 160, 162, 165,
162, 89, 85, 82, 81, 79, 78, 78, 82, 86, 87, 92, 97, 100, 105, 112, 114,
120, 128, 131, 136, 146, 147, 150, 155, 156, 161, 166, 165, 167, 169,
169, 175, 92, 88, 85, 84, 82, 81, 80, 85, 86, 90, 95, 97, 102, 107, 110,
117, 122, 125, 134, 138, 142, 152, 154, 156, 162, 163, 168, 173, 174,
174, 178, 176, 95, 90, 88, 86, 85, 84, 82, 86, 88, 93, 95, 99, 105, 106,
113, 118, 121, 129, 132, 139, 146, 148, 159, 161, 163, 169, 170, 176,
180, 183, 181, 187, 98, 93, 91, 89, 88, 87, 85, 87, 90, 94, 96, 102,
104, 109, 114, 117, 126, 128, 134, 141, 145, 154, 157, 166, 168, 170,
176, 178, 184, 188, 193, 188, 101, 96, 95, 92, 91, 90, 88, 88, 93, 95,
99, 103, 106, 112, 114, 121, 124, 131, 136, 140, 149, 151, 160, 165,
173, 176, 178, 184, 186, 192, 196, 203, 104, 99, 98, 95, 94, 93, 91, 90,
95, 96, 102, 103, 109, 112, 117, 122, 125, 133, 136, 145, 146, 156, 160,
167, 174, 180, 184, 186, 193, 194, 201, 204, 108, 102, 101, 98, 97, 96,
95, 93, 97, 100, 104, 106, 111, 113, 121, 122, 130, 132, 140, 143, 150,
155, 162, 169, 174, 183, 188, 192, 194, 201, 202, 210, 111, 105, 105,
101, 100, 99, 98, 96, 98, 103, 105, 109, 112, 117, 121, 125, 130, 135,
141, 146, 152, 156, 165, 169, 178, 181, 193, 196, 201, 202, 210, 211,
114, 109, 109, 104, 104, 102, 102, 99, 100, 106, 106, 113, 113, 120,
121, 129, 130, 139, 140, 151, 151, 162, 162, 175, 176, 187, 188, 203,
204, 210, 211, 219,
// Size 4x8
32, 42, 69, 88, 33, 42, 64, 83, 36, 56, 77, 88, 46, 67, 93, 105, 60, 79,
112, 122, 75, 92, 130, 144, 86, 95, 136, 167, 98, 105, 136, 177,
// Size 8x4

```

```

32, 33, 36, 46, 60, 75, 86, 98, 42, 42, 56, 67, 79, 92, 95, 105, 69, 64,
77, 93, 112, 130, 136, 136, 88, 83, 88, 105, 122, 144, 167, 177,
// Size 8x16
32, 32, 36, 47, 65, 79, 90, 96, 31, 32, 35, 44, 60, 72, 84, 90, 32, 34,
36, 45, 59, 71, 80, 87, 32, 35, 40, 47, 60, 71, 78, 85, 36, 37, 48, 56,
68, 78, 83, 87, 39, 40, 50, 60, 73, 84, 91, 94, 47, 45, 56, 69, 84, 95,
101, 101, 53, 50, 60, 75, 92, 103, 108, 110, 61, 56, 65, 81, 100, 113,
116, 118, 71, 64, 73, 89, 111, 125, 129, 129, 79, 70, 79, 95, 118, 133,
142, 138, 86, 76, 84, 100, 124, 140, 153, 150, 92, 82, 89, 101, 121,
148, 157, 161, 98, 88, 93, 108, 124, 141, 163, 174, 104, 94, 95, 110,
129, 151, 171, 181, 110, 100, 98, 111, 127, 147, 169, 188,
// Size 16x8
32, 31, 32, 32, 36, 39, 47, 53, 61, 71, 79, 86, 92, 98, 104, 110, 32,
32, 34, 35, 37, 40, 45, 50, 56, 64, 70, 76, 82, 88, 94, 100, 36, 35, 36,
40, 48, 50, 56, 60, 65, 73, 79, 84, 89, 93, 95, 98, 47, 44, 45, 47, 56,
60, 69, 75, 81, 89, 95, 100, 101, 108, 110, 111, 65, 60, 59, 60, 68, 73,
84, 92, 100, 111, 118, 124, 121, 124, 129, 127, 79, 72, 71, 71, 78, 84,
95, 103, 113, 125, 133, 140, 148, 141, 151, 147, 90, 84, 80, 78, 83, 91,
101, 108, 116, 129, 142, 153, 157, 163, 171, 169, 96, 90, 87, 85, 87,
94, 101, 110, 118, 129, 138, 150, 161, 174, 181, 188,
// Size 16x32
32, 31, 32, 32, 36, 44, 47, 53, 65, 73, 79, 87, 90, 93, 96, 99, 31, 32,
32, 33, 35, 42, 45, 51, 62, 69, 75, 83, 86, 88, 91, 94, 31, 32, 32, 33,
35, 41, 44, 49, 60, 67, 72, 80, 84, 87, 90, 94, 31, 32, 33, 33, 35, 41,
44, 49, 59, 66, 71, 79, 82, 84, 87, 90, 32, 32, 34, 34, 36, 42, 45, 50,
59, 65, 71, 78, 80, 83, 87, 90, 32, 33, 35, 36, 38, 42, 45, 49, 58, 64,
69, 76, 80, 83, 86, 88, 32, 33, 35, 36, 40, 44, 47, 51, 60, 66, 71, 76,
78, 81, 85, 89, 34, 34, 36, 38, 42, 48, 50, 54, 63, 69, 73, 80, 82, 81,
84, 86, 36, 34, 37, 40, 48, 54, 56, 60, 68, 74, 78, 84, 83, 86, 87, 87,
38, 36, 39, 41, 49, 56, 58, 63, 71, 77, 81, 86, 88, 88, 90, 93, 39, 37,
40, 42, 50, 58, 60, 65, 73, 79, 84, 90, 91, 92, 94, 93, 44, 41, 42, 45,
53, 63, 66, 71, 79, 85, 90, 96, 94, 96, 96, 99, 47, 44, 45, 47, 56, 66,
69, 75, 84, 90, 95, 99, 101, 98, 101, 99, 49, 46, 47, 48, 57, 67, 71,
77, 86, 93, 97, 103, 103, 105, 102, 106, 53, 49, 50, 51, 60, 71, 75, 82,
92, 99, 103, 111, 108, 107, 110, 107, 58, 54, 54, 55, 63, 75, 79, 87,
98, 105, 110, 114, 114, 113, 111, 115, 61, 56, 56, 57, 65, 77, 81, 89,
100, 107, 113, 118, 116, 117, 118, 116, 65, 60, 59, 60, 68, 79, 84, 92,
105, 112, 118, 126, 124, 122, 121, 124, 71, 65, 64, 65, 73, 84, 89, 97,
111, 119, 125, 130, 129, 129, 129, 125, 76, 69, 68, 69, 76, 88, 92, 101,
115, 123, 130, 134, 134, 131, 132, 135, 79, 72, 70, 71, 79, 90, 95, 104,
118, 127, 133, 143, 142, 141, 138, 136, 82, 75, 73, 74, 81, 92, 97, 106,
121, 130, 136, 146, 145, 144, 144, 145, 86, 78, 76, 77, 84, 95, 100,
109, 124, 133, 140, 147, 153, 151, 150, 146, 89, 81, 79, 78, 87, 95, 99,
112, 124, 130, 145, 152, 156, 157, 156, 158, 92, 84, 82, 80, 89, 95,
101, 116, 121, 132, 148, 151, 157, 163, 161, 159, 95, 86, 85, 83, 92,
95, 105, 114, 120, 136, 143, 155, 163, 167, 171, 170, 98, 89, 88, 85,
93, 95, 108, 113, 124, 136, 141, 160, 163, 169, 174, 171, 101, 92, 91,
88, 94, 98, 110, 112, 128, 133, 146, 158, 166, 175, 179, 185, 104, 95,
94, 91, 95, 101, 110, 115, 129, 132, 151, 154, 171, 175, 181, 186, 107,
98, 97, 94, 96, 105, 110, 119, 128, 136, 149, 156, 173, 177, 188, 192,
110, 101, 100, 97, 98, 108, 111, 123, 127, 141, 147, 161, 169, 183, 188,
193, 114, 104, 104, 100, 100, 111, 111, 126, 127, 145, 145, 166, 166,
189, 190, 201,
// Size 32x16
32, 31, 31, 31, 32, 32, 32, 34, 36, 38, 39, 44, 47, 49, 53, 58, 61, 65,
71, 76, 79, 82, 86, 89, 92, 95, 98, 101, 104, 107, 110, 114, 31, 32, 32,
32, 32, 33, 33, 34, 34, 36, 37, 41, 44, 46, 49, 54, 56, 60, 65, 69, 72,
75, 78, 81, 84, 86, 89, 92, 95, 98, 101, 104, 32, 32, 32, 33, 34, 35,
35, 36, 37, 39, 40, 42, 45, 47, 50, 54, 56, 59, 64, 68, 70, 73, 76, 79,
82, 85, 88, 91, 94, 97, 100, 104, 32, 33, 33, 33, 34, 36, 36, 38, 40,
41, 42, 45, 47, 48, 51, 55, 57, 60, 65, 69, 71, 74, 77, 78, 80, 83, 85,
88, 91, 94, 97, 100, 36, 35, 35, 35, 36, 38, 40, 42, 48, 49, 50, 53, 56,
57, 60, 63, 65, 68, 73, 76, 79, 81, 84, 87, 89, 92, 93, 94, 95, 96, 98,
100, 44, 42, 41, 41, 42, 42, 44, 48, 54, 56, 58, 63, 66, 67, 71, 75, 77,
79, 84, 88, 90, 92, 95, 95, 95, 95, 95, 98, 101, 105, 108, 111, 47, 45,
44, 44, 45, 45, 47, 50, 56, 58, 60, 66, 69, 71, 75, 79, 81, 84, 89, 92,
95, 97, 100, 99, 101, 105, 108, 110, 110, 110, 111, 111, 53, 51, 49, 49,
50, 49, 51, 54, 60, 63, 65, 71, 75, 77, 82, 87, 89, 92, 97, 101, 104,
106, 109, 112, 116, 114, 113, 112, 115, 119, 123, 126, 65, 62, 60, 59,
59, 58, 60, 63, 68, 71, 73, 79, 84, 86, 92, 98, 100, 105, 111, 115, 118,
121, 124, 124, 121, 120, 124, 128, 129, 128, 127, 127, 73, 69, 67, 66,

```



```

65, 64, 66, 69, 74, 77, 79, 85, 90, 93, 99, 105, 107, 112, 119, 123,
127, 130, 133, 130, 132, 136, 136, 133, 132, 136, 141, 145, 79, 75, 72,
71, 71, 69, 71, 73, 78, 81, 84, 90, 95, 97, 103, 110, 113, 118, 125,
130, 133, 136, 140, 145, 148, 143, 141, 146, 151, 149, 147, 145, 87, 83,
80, 79, 78, 76, 76, 80, 84, 86, 90, 96, 99, 103, 111, 114, 118, 126,
130, 134, 143, 146, 147, 152, 151, 155, 160, 158, 154, 156, 161, 166,
90, 86, 84, 82, 80, 80, 78, 82, 83, 88, 91, 94, 101, 103, 108, 114, 116,
124, 129, 134, 142, 145, 153, 156, 157, 163, 163, 166, 171, 173, 169,
166, 93, 88, 87, 84, 83, 83, 81, 81, 86, 88, 92, 96, 98, 105, 107, 113,
117, 122, 129, 131, 141, 144, 151, 157, 163, 167, 169, 175, 175, 177,
183, 189, 96, 91, 90, 87, 87, 86, 85, 84, 87, 90, 94, 96, 101, 102, 110,
111, 118, 121, 129, 132, 138, 144, 150, 156, 161, 171, 174, 179, 181,
188, 188, 190, 99, 94, 94, 90, 90, 88, 89, 86, 87, 93, 93, 99, 99, 106,
107, 115, 116, 124, 125, 135, 136, 145, 146, 158, 159, 170, 171, 185,
186, 192, 193, 201,
// Size 4x16
31, 44, 73, 93, 32, 41, 67, 87, 32, 42, 65, 83, 33, 44, 66, 81, 34, 54,
74, 86, 37, 58, 79, 92, 44, 66, 90, 98, 49, 71, 99, 107, 56, 77, 107,
117, 65, 84, 119, 129, 72, 90, 127, 141, 78, 95, 133, 151, 84, 95, 132,
163, 89, 95, 136, 169, 95, 101, 132, 175, 101, 108, 141, 183,
// Size 16x4
31, 32, 32, 33, 34, 37, 44, 49, 56, 65, 72, 78, 84, 89, 95, 101, 44, 41,
42, 44, 54, 58, 66, 71, 77, 84, 90, 95, 95, 95, 101, 108, 73, 67, 65,
66, 74, 79, 90, 99, 107, 119, 127, 133, 132, 136, 132, 141, 93, 87, 83,
81, 86, 92, 98, 107, 117, 129, 141, 151, 163, 169, 175, 183,
// Size 8x32
32, 32, 36, 47, 65, 79, 90, 96, 31, 32, 35, 45, 62, 75, 86, 91, 31, 32,
35, 44, 60, 72, 84, 90, 31, 33, 35, 44, 59, 71, 82, 87, 32, 34, 36, 45,
59, 71, 80, 87, 32, 35, 38, 45, 58, 69, 80, 86, 32, 35, 40, 47, 60, 71,
78, 85, 34, 36, 42, 50, 63, 73, 82, 84, 36, 37, 48, 56, 68, 78, 83, 87,
38, 39, 49, 58, 71, 81, 88, 90, 39, 40, 50, 60, 73, 84, 91, 94, 44, 42,
53, 66, 79, 90, 94, 96, 47, 45, 56, 69, 84, 95, 101, 101, 49, 47, 57,
71, 86, 97, 103, 102, 53, 50, 60, 75, 92, 103, 108, 110, 58, 54, 63, 79,
98, 110, 114, 111, 61, 56, 65, 81, 100, 113, 116, 118, 65, 59, 68, 84,
105, 118, 124, 121, 71, 64, 73, 89, 111, 125, 129, 129, 76, 68, 76, 92,
115, 130, 134, 132, 79, 70, 79, 95, 118, 133, 142, 138, 82, 73, 81, 97,
121, 136, 145, 144, 86, 76, 84, 100, 124, 140, 153, 150, 89, 79, 87, 99,
124, 145, 156, 156, 92, 82, 89, 101, 121, 148, 157, 161, 95, 85, 92,
105, 120, 143, 163, 171, 98, 88, 93, 108, 124, 141, 163, 174, 101, 91,
94, 110, 128, 146, 166, 179, 104, 94, 95, 110, 129, 151, 171, 181, 107,
97, 96, 110, 128, 149, 173, 188, 110, 100, 98, 111, 127, 147, 169, 188,
114, 104, 100, 111, 127, 145, 166, 190,
// Size 32x8
32, 31, 31, 31, 32, 32, 32, 34, 36, 38, 39, 44, 47, 49, 53, 58, 61, 65,
71, 76, 79, 82, 86, 89, 92, 95, 98, 101, 104, 107, 110, 114, 32, 32, 32,
33, 34, 35, 35, 36, 37, 39, 40, 42, 45, 47, 50, 54, 56, 59, 64, 68, 70,
73, 76, 79, 82, 85, 88, 91, 94, 97, 100, 104, 36, 35, 35, 35, 36, 38,
40, 42, 48, 49, 50, 53, 56, 57, 60, 63, 65, 68, 73, 76, 79, 81, 84, 87,
89, 92, 93, 94, 95, 96, 98, 100, 47, 45, 44, 44, 45, 45, 47, 50, 56, 58,
60, 66, 69, 71, 75, 79, 81, 84, 89, 92, 95, 97, 100, 99, 101, 105, 108,
110, 110, 110, 111, 111, 65, 62, 60, 59, 59, 58, 60, 63, 68, 71, 73, 79,
84, 86, 92, 98, 100, 105, 111, 115, 118, 121, 124, 124, 121, 120, 124,
128, 129, 128, 127, 127, 79, 75, 72, 71, 71, 69, 71, 73, 78, 81, 84, 90,
95, 97, 103, 110, 113, 118, 125, 130, 133, 136, 140, 145, 148, 143, 141,
146, 151, 149, 147, 145, 90, 86, 84, 82, 80, 80, 78, 82, 83, 88, 91, 94,
101, 103, 108, 114, 116, 124, 129, 134, 142, 145, 153, 156, 157, 163,
163, 166, 171, 173, 169, 166, 96, 91, 90, 87, 87, 86, 85, 84, 87, 90,
94, 96, 101, 102, 110, 111, 118, 121, 129, 132, 138, 144, 150, 156, 161,
171, 174, 179, 181, 188, 188, 190,
// Size 4x32
31, 44, 73, 93, 32, 42, 69, 88, 32, 41, 67, 87, 32, 41, 66, 84, 32, 42,
65, 83, 33, 42, 64, 83, 33, 44, 66, 81, 34, 48, 69, 81, 34, 54, 74, 86,
36, 56, 77, 88, 37, 58, 79, 92, 41, 63, 85, 96, 44, 66, 90, 98, 46, 67,
93, 105, 49, 71, 99, 107, 54, 75, 105, 113, 56, 77, 107, 117, 60, 79,
112, 122, 65, 84, 119, 129, 69, 88, 123, 131, 72, 90, 127, 141, 75, 92,
130, 144, 78, 95, 133, 151, 81, 95, 130, 157, 84, 95, 132, 163, 86, 95,
136, 167, 89, 95, 136, 169, 92, 98, 133, 175, 95, 101, 132, 175, 98,
105, 136, 177, 101, 108, 141, 183, 104, 111, 145, 189,
// Size 32x4
31, 32, 32, 32, 32, 33, 33, 34, 34, 36, 37, 41, 44, 46, 49, 54, 56, 60,
65, 69, 72, 75, 78, 81, 84, 86, 89, 92, 95, 98, 101, 104, 44, 42, 41,

```

```

41, 42, 42, 44, 48, 54, 56, 58, 63, 66, 67, 71, 75, 77, 79, 84, 88, 90,
92, 95, 95, 95, 95, 95, 98, 101, 105, 108, 111, 73, 69, 67, 66, 65, 64,
66, 69, 74, 77, 79, 85, 90, 93, 99, 105, 107, 112, 119, 123, 127, 130,
133, 130, 132, 136, 136, 133, 132, 136, 141, 145, 93, 88, 87, 84, 83,
83, 81, 81, 86, 88, 92, 96, 98, 105, 107, 113, 117, 122, 129, 131, 141,
144, 151, 157, 163, 167, 169, 175, 175, 177, 183, 189,
},
{ // Chroma
  // Size 4x4
  33, 45, 56, 64, 45, 58, 66, 69, 56, 66, 86, 87, 64, 69, 87, 105,
  // Size 8x8
  31, 38, 47, 48, 54, 61, 66, 69, 38, 47, 47, 46, 50, 55, 61, 65, 47, 47,
  53, 55, 58, 63, 65, 66, 48, 46, 55, 62, 67, 72, 73, 73, 54, 50, 58, 67,
  76, 83, 84, 82, 61, 55, 63, 72, 83, 91, 92, 92, 66, 61, 65, 73, 84, 92,
  101, 103, 69, 65, 66, 73, 82, 92, 103, 109,
  // Size 16x16
  32, 30, 33, 38, 49, 48, 50, 52, 55, 60, 63, 66, 68, 70, 72, 74, 30, 31,
  35, 41, 46, 46, 46, 48, 51, 55, 58, 60, 63, 65, 68, 70, 33, 35, 39, 44,
  47, 46, 46, 47, 50, 53, 56, 58, 60, 62, 65, 67, 38, 41, 44, 47, 49, 48,
  47, 48, 50, 53, 55, 58, 58, 60, 62, 65, 49, 46, 47, 49, 53, 53, 54, 54,
  56, 58, 60, 62, 63, 64, 64, 48, 46, 46, 48, 53, 54, 56, 57, 59, 61,
  63, 65, 67, 66, 68, 68, 50, 46, 46, 47, 54, 56, 61, 63, 65, 68, 70, 72,
  71, 71, 72, 72, 52, 48, 47, 48, 54, 57, 63, 66, 69, 72, 75, 76, 75, 76,
  76, 76, 55, 51, 50, 50, 56, 59, 65, 69, 73, 77, 79, 81, 81, 81, 80, 80,
  60, 55, 53, 53, 58, 61, 68, 72, 77, 82, 85, 87, 87, 85, 84, 85, 63, 58,
  56, 55, 60, 63, 70, 75, 79, 85, 89, 91, 91, 90, 89, 90, 66, 60, 58, 58,
  62, 65, 72, 76, 81, 87, 91, 94, 96, 95, 95, 95, 68, 63, 60, 58, 62, 67,
  71, 75, 81, 87, 91, 96, 99, 100, 100, 100, 70, 65, 62, 60, 63, 66, 71,
  76, 81, 85, 90, 95, 100, 103, 104, 105, 72, 68, 65, 62, 64, 68, 72, 76,
  80, 84, 89, 95, 100, 104, 107, 108, 74, 70, 67, 65, 64, 68, 72, 76, 80,
  85, 90, 95, 100, 105, 108, 111,
  // Size 32x32
  32, 31, 30, 31, 33, 36, 38, 41, 49, 49, 48, 49, 50, 51, 52, 54, 55, 57,
  60, 62, 63, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 31, 31, 31, 32,
  34, 38, 40, 42, 47, 47, 47, 47, 48, 48, 50, 52, 53, 54, 57, 59, 60, 61,
  63, 64, 65, 66, 67, 67, 68, 69, 70, 71, 30, 31, 31, 32, 35, 39, 41, 42,
  46, 46, 46, 45, 46, 47, 48, 50, 51, 52, 55, 57, 58, 59, 60, 62, 63, 64,
  65, 67, 68, 69, 70, 71, 31, 32, 32, 33, 36, 40, 41, 43, 46, 46, 45, 45,
  46, 46, 47, 49, 50, 51, 54, 56, 57, 58, 59, 61, 62, 63, 63, 64, 65, 66,
  67, 68, 33, 34, 35, 36, 39, 43, 44, 45, 47, 46, 46, 45, 46, 47, 47, 49,
  50, 51, 53, 55, 56, 57, 58, 59, 60, 61, 62, 63, 65, 66, 67, 68, 36, 38,
  39, 40, 43, 47, 47, 47, 48, 47, 46, 45, 46, 46, 47, 48, 49, 50, 52, 53,
  54, 55, 56, 58, 59, 61, 62, 63, 64, 65, 66, 66, 38, 40, 41, 41, 44, 47,
  47, 48, 49, 48, 48, 47, 47, 47, 48, 49, 50, 51, 53, 54, 55, 56, 58, 58,
  58, 59, 60, 61, 62, 64, 65, 66, 41, 42, 42, 43, 45, 47, 48, 48, 50, 50,
  49, 49, 50, 50, 50, 52, 52, 53, 55, 56, 57, 58, 59, 60, 61, 61, 61, 61,
  62, 63, 63, 64, 49, 47, 46, 46, 47, 48, 49, 50, 53, 53, 53, 53, 54, 54,
  54, 55, 56, 56, 58, 59, 60, 61, 62, 63, 62, 62, 63, 64, 64, 64, 64, 64,
  49, 47, 46, 46, 46, 47, 48, 50, 53, 53, 54, 55, 55, 55, 56, 57, 58, 58,
  60, 61, 62, 63, 64, 64, 64, 65, 65, 65, 65, 66, 67, 68, 48, 47, 46, 45,
  46, 46, 48, 49, 53, 54, 54, 55, 56, 56, 57, 58, 59, 60, 61, 63, 63, 64,
  65, 66, 67, 66, 66, 67, 68, 68, 68, 49, 47, 45, 45, 45, 45, 47, 49,
  53, 55, 55, 58, 59, 60, 61, 62, 63, 63, 65, 66, 67, 68, 69, 69, 68, 68,
  69, 69, 69, 69, 70, 71, 50, 48, 46, 46, 46, 46, 47, 50, 54, 55, 56, 59,
  61, 61, 63, 64, 65, 66, 68, 69, 70, 71, 72, 71, 71, 72, 71, 71, 72, 72,
  72, 71, 51, 48, 47, 46, 47, 46, 47, 50, 54, 55, 56, 60, 61, 62, 64, 66,
  66, 67, 69, 70, 71, 72, 73, 73, 74, 73, 74, 73, 74, 73, 74, 75, 52, 50,
  48, 47, 47, 47, 48, 50, 54, 56, 57, 61, 63, 64, 66, 68, 69, 70, 72, 74,
  75, 75, 76, 77, 75, 76, 76, 75, 76, 77, 76, 75, 54, 52, 50, 49, 49, 48,
  49, 52, 55, 57, 58, 62, 64, 66, 68, 71, 72, 73, 75, 77, 78, 79, 80, 78,
  79, 78, 77, 78, 78, 77, 78, 79, 55, 53, 51, 50, 50, 49, 50, 52, 56, 58,
  59, 63, 65, 66, 69, 72, 73, 74, 77, 78, 79, 80, 81, 81, 81, 80, 81, 80,
  80, 81, 80, 79, 57, 54, 52, 51, 51, 50, 51, 53, 56, 58, 60, 63, 66, 67,
  70, 73, 74, 76, 79, 80, 82, 83, 84, 85, 83, 84, 83, 83, 83, 82, 82, 83,
  60, 57, 55, 54, 53, 52, 53, 55, 58, 60, 61, 65, 68, 69, 72, 75, 77, 79,
  82, 84, 85, 86, 87, 86, 87, 85, 85, 84, 86, 85, 84, 62, 59, 57, 56,
  55, 53, 54, 56, 59, 61, 63, 66, 69, 70, 74, 77, 78, 80, 84, 86, 87, 88,
  90, 89, 89, 88, 88, 87, 88, 87, 87, 88, 63, 60, 58, 57, 56, 54, 55, 57,
  60, 62, 63, 67, 70, 71, 75, 78, 79, 82, 85, 87, 89, 90, 91, 93, 91, 91,
  90, 91, 89, 90, 90, 89, 65, 61, 59, 58, 57, 55, 56, 58, 61, 63, 64, 68,
  71, 72, 75, 79, 80, 83, 86, 88, 90, 91, 93, 94, 95, 92, 94, 92, 93, 92,

```

91, 93, 66, 63, 60, 59, 58, 56, 58, 59, 62, 64, 65, 69, 72, 73, 76, 80,
 81, 84, 87, 90, 91, 93, 94, 95, 96, 97, 95, 95, 95, 95, 93, 67, 64,
 62, 61, 59, 58, 58, 60, 63, 64, 66, 69, 71, 73, 77, 78, 81, 85, 86, 89,
 93, 94, 95, 97, 97, 98, 99, 97, 97, 97, 96, 98, 68, 65, 63, 62, 60, 59,
 58, 61, 62, 64, 67, 68, 71, 74, 75, 79, 81, 83, 87, 89, 91, 95, 96, 97,
 99, 98, 100, 100, 100, 99, 100, 98, 69, 66, 64, 63, 61, 61, 59, 61, 62,
 65, 66, 68, 72, 73, 76, 78, 80, 84, 85, 88, 91, 92, 97, 98, 98, 101,
 100, 102, 102, 103, 101, 102, 70, 67, 65, 63, 62, 62, 60, 61, 63, 65,
 66, 69, 71, 73, 76, 77, 81, 83, 85, 88, 90, 94, 95, 99, 100, 100, 103,
 102, 104, 104, 105, 103, 71, 67, 67, 64, 63, 63, 61, 61, 64, 65, 67, 69,
 71, 74, 75, 78, 80, 83, 85, 87, 91, 92, 95, 97, 100, 102, 102, 105, 104,
 106, 106, 108, 72, 68, 68, 65, 65, 64, 62, 62, 64, 65, 68, 69, 72, 73,
 76, 78, 80, 83, 84, 88, 89, 93, 95, 97, 100, 102, 104, 104, 107, 106,
 108, 108, 73, 69, 69, 66, 66, 65, 64, 63, 64, 66, 68, 69, 72, 73, 77,
 77, 81, 82, 86, 87, 90, 92, 95, 97, 99, 103, 104, 106, 106, 109, 108,
 110, 74, 70, 70, 67, 67, 66, 65, 63, 64, 67, 68, 70, 72, 74, 76, 78, 80,
 82, 85, 87, 90, 91, 95, 96, 100, 101, 105, 106, 108, 108, 111, 110, 75,
 71, 71, 68, 68, 66, 66, 64, 64, 68, 68, 71, 71, 75, 75, 79, 79, 83, 84,
 88, 89, 93, 93, 98, 98, 102, 103, 108, 108, 110, 110, 113,
 // Size 4x8
 31, 47, 57, 65, 40, 45, 52, 61, 46, 55, 61, 63, 47, 60, 70, 72, 52, 64,
 79, 81, 59, 68, 87, 90, 63, 66, 88, 99, 66, 69, 85, 102,
 // Size 8x4
 31, 40, 46, 47, 52, 59, 63, 66, 47, 45, 55, 60, 64, 68, 66, 69, 57, 52,
 61, 70, 79, 87, 88, 85, 65, 61, 63, 72, 81, 90, 99, 102,
 // Size 8x16
 32, 35, 48, 50, 57, 63, 68, 70, 30, 38, 46, 46, 52, 58, 63, 65, 33, 41,
 47, 46, 51, 56, 60, 63, 39, 46, 48, 47, 51, 55, 58, 61, 49, 48, 53, 54,
 57, 60, 61, 61, 48, 46, 53, 56, 60, 64, 65, 65, 50, 46, 54, 61, 66, 70,
 71, 69, 52, 47, 54, 63, 71, 75, 75, 74, 55, 49, 56, 65, 74, 79, 79, 78,
 60, 53, 58, 68, 79, 85, 85, 82, 63, 55, 60, 70, 82, 89, 91, 87, 66, 58,
 62, 72, 84, 91, 95, 91, 68, 60, 64, 71, 81, 94, 97, 96, 70, 62, 65, 73,
 81, 89, 98, 101, 72, 65, 65, 72, 82, 92, 100, 103, 74, 67, 65, 71, 79,
 89, 98, 105,
 // Size 16x8
 32, 30, 33, 39, 49, 48, 50, 52, 55, 60, 63, 66, 68, 70, 72, 74, 35, 38,
 41, 46, 48, 46, 46, 47, 49, 53, 55, 58, 60, 62, 65, 67, 48, 46, 47, 48,
 53, 53, 54, 54, 56, 58, 60, 62, 64, 65, 65, 65, 50, 46, 46, 47, 54, 56,
 61, 63, 65, 68, 70, 72, 71, 73, 72, 71, 57, 52, 51, 51, 57, 60, 66, 71,
 74, 79, 82, 84, 81, 81, 82, 79, 63, 58, 56, 55, 60, 64, 70, 75, 79, 85,
 89, 91, 94, 89, 92, 89, 68, 63, 60, 58, 61, 65, 71, 75, 79, 85, 91, 95,
 97, 98, 100, 98, 70, 65, 63, 61, 61, 65, 69, 74, 78, 82, 87, 91, 96,
 101, 103, 105,
 // Size 16x32
 32, 31, 35, 38, 48, 49, 50, 52, 57, 61, 63, 67, 68, 69, 70, 71, 31, 31,
 37, 40, 47, 47, 48, 50, 54, 57, 60, 63, 64, 65, 66, 67, 30, 32, 38, 40,
 46, 45, 46, 48, 52, 55, 58, 61, 63, 64, 65, 67, 31, 33, 38, 41, 46, 45,
 46, 48, 52, 55, 57, 60, 61, 62, 63, 64, 33, 36, 41, 44, 47, 46, 46, 47,
 51, 54, 56, 59, 60, 61, 63, 64, 37, 40, 45, 47, 47, 45, 46, 47, 50, 52,
 54, 57, 59, 61, 62, 62, 39, 41, 46, 47, 48, 47, 47, 48, 51, 54, 55, 57,
 58, 59, 61, 62, 42, 43, 46, 48, 50, 49, 50, 53, 56, 57, 60, 60, 59,
 60, 60, 49, 46, 48, 49, 53, 53, 54, 54, 57, 59, 60, 63, 61, 62, 61, 61,
 48, 46, 47, 48, 53, 55, 55, 56, 58, 61, 62, 64, 64, 63, 63, 64, 48, 46,
 46, 48, 53, 56, 56, 57, 60, 62, 64, 66, 65, 65, 65, 64, 49, 45, 45, 47,
 53, 58, 59, 61, 64, 66, 67, 69, 67, 67, 66, 67, 50, 46, 46, 48, 54, 59,
 61, 63, 66, 68, 70, 71, 71, 68, 69, 67, 51, 47, 47, 48, 54, 60, 61, 64,
 68, 70, 71, 73, 72, 72, 70, 71, 52, 48, 47, 48, 54, 61, 63, 66, 71, 73,
 75, 77, 75, 73, 74, 71, 54, 50, 49, 50, 55, 62, 65, 68, 73, 76, 78, 79,
 78, 76, 74, 75, 55, 51, 49, 50, 56, 63, 65, 69, 74, 77, 79, 81, 79, 78,
 78, 75, 57, 52, 50, 51, 56, 64, 66, 70, 76, 79, 82, 85, 83, 81, 79, 79,
 60, 54, 53, 53, 58, 65, 68, 72, 79, 82, 85, 87, 85, 84, 82, 80, 62, 56,
 54, 55, 60, 66, 69, 74, 81, 84, 87, 88, 87, 85, 84, 84, 63, 57, 55, 56,
 60, 67, 70, 75, 82, 86, 89, 92, 91, 89, 87, 84, 64, 59, 56, 57, 61, 68,
 71, 75, 83, 87, 90, 93, 92, 90, 89, 89, 66, 60, 58, 58, 62, 69, 72, 76,
 84, 88, 91, 94, 95, 93, 91, 89, 67, 61, 59, 58, 63, 68, 71, 78, 83, 86,
 93, 96, 96, 96, 94, 94, 68, 62, 60, 59, 64, 67, 71, 79, 81, 86, 94, 95,
 97, 98, 96, 94, 69, 63, 61, 60, 65, 66, 72, 77, 80, 88, 91, 96, 99, 99,
 100, 98, 70, 64, 62, 60, 65, 66, 73, 76, 81, 87, 89, 97, 98, 100, 101,
 99, 71, 65, 64, 61, 65, 67, 73, 74, 82, 85, 90, 95, 99, 102, 103, 104,
 72, 65, 65, 62, 65, 68, 72, 75, 82, 83, 92, 93, 100, 102, 103, 104, 73,
 66, 66, 63, 65, 69, 72, 76, 81, 85, 90, 93, 100, 102, 105, 106, 74, 67,

```

67, 64, 65, 70, 71, 77, 79, 86, 89, 94, 98, 103, 105, 106, 75, 68, 68,
65, 65, 71, 71, 78, 78, 87, 87, 96, 96, 105, 105, 109,
// Size 32x16
32, 31, 30, 31, 33, 37, 39, 42, 49, 48, 48, 49, 50, 51, 52, 54, 55, 57,
60, 62, 63, 64, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 31, 31, 32, 33,
36, 40, 41, 43, 46, 46, 46, 45, 46, 47, 48, 50, 51, 52, 54, 56, 57, 59,
60, 61, 62, 63, 64, 65, 65, 66, 67, 68, 35, 37, 38, 38, 41, 45, 46, 46,
48, 47, 46, 45, 46, 47, 47, 49, 49, 50, 53, 54, 55, 56, 58, 59, 60, 61,
62, 64, 65, 66, 67, 68, 38, 40, 40, 41, 44, 47, 47, 48, 49, 48, 48, 47,
48, 48, 48, 50, 50, 51, 53, 55, 56, 57, 58, 58, 59, 60, 60, 61, 62, 63,
64, 65, 48, 47, 46, 46, 47, 47, 48, 50, 53, 53, 53, 54, 54, 54, 55,
56, 56, 58, 60, 60, 61, 62, 63, 64, 65, 65, 65, 65, 65, 65, 65, 49, 47,
45, 45, 46, 45, 47, 49, 53, 55, 56, 58, 59, 60, 61, 62, 63, 64, 65, 66,
67, 68, 69, 68, 67, 66, 66, 67, 68, 69, 70, 71, 50, 48, 46, 46, 46, 46,
47, 50, 54, 55, 56, 59, 61, 61, 63, 65, 65, 66, 68, 69, 70, 71, 72, 71,
71, 72, 73, 73, 72, 72, 71, 71, 52, 50, 48, 48, 47, 47, 48, 50, 54, 56,
57, 61, 63, 64, 66, 68, 69, 70, 72, 74, 75, 75, 76, 78, 79, 77, 76, 74,
75, 76, 77, 78, 57, 54, 52, 52, 51, 50, 51, 53, 57, 58, 60, 64, 66, 68,
71, 73, 74, 76, 79, 81, 82, 83, 84, 83, 81, 80, 81, 82, 82, 81, 79, 78,
61, 57, 55, 55, 54, 52, 54, 56, 59, 61, 62, 66, 68, 70, 73, 76, 77, 79,
82, 84, 86, 87, 88, 86, 86, 88, 87, 85, 83, 85, 86, 87, 63, 60, 58, 57,
56, 54, 55, 57, 60, 62, 64, 67, 70, 71, 75, 78, 79, 82, 85, 87, 89, 90,
91, 93, 94, 91, 89, 90, 92, 90, 89, 87, 67, 63, 61, 60, 59, 57, 57, 60,
63, 64, 66, 69, 71, 73, 77, 79, 81, 85, 87, 88, 92, 93, 94, 96, 95, 96,
97, 95, 93, 93, 94, 96, 68, 64, 63, 61, 60, 59, 58, 60, 61, 64, 65, 67,
71, 72, 75, 78, 79, 83, 85, 87, 91, 92, 95, 96, 97, 99, 98, 99, 100,
100, 98, 96, 69, 65, 64, 62, 61, 61, 59, 59, 62, 63, 65, 67, 68, 72, 73,
76, 78, 81, 84, 85, 89, 90, 93, 96, 98, 99, 100, 102, 102, 102, 103,
105, 70, 66, 65, 63, 63, 62, 61, 60, 61, 63, 65, 66, 69, 70, 74, 74, 78,
79, 82, 84, 87, 89, 91, 94, 96, 100, 101, 103, 103, 105, 105, 105, 71,
67, 67, 64, 64, 62, 62, 60, 61, 64, 64, 67, 71, 71, 75, 75, 79, 80,
84, 84, 89, 89, 94, 94, 98, 99, 104, 104, 106, 106, 109,
// Size 4x16
31, 49, 61, 69, 32, 45, 55, 64, 36, 46, 54, 61, 41, 47, 54, 59, 46, 53,
59, 62, 46, 56, 62, 65, 46, 59, 68, 68, 48, 61, 73, 73, 51, 63, 77, 78,
54, 65, 82, 84, 57, 67, 86, 89, 60, 69, 88, 93, 62, 67, 86, 98, 64, 66,
87, 100, 65, 68, 83, 102, 67, 70, 86, 103,
// Size 16x4
31, 32, 36, 41, 46, 46, 46, 48, 51, 54, 57, 60, 62, 64, 65, 67, 49, 45,
46, 47, 53, 56, 59, 61, 63, 65, 67, 69, 67, 66, 68, 70, 61, 55, 54, 54,
59, 62, 68, 73, 77, 82, 86, 88, 86, 87, 83, 86, 69, 64, 61, 59, 62, 65,
68, 73, 78, 84, 89, 93, 98, 100, 102, 103,
// Size 8x32
32, 35, 48, 50, 57, 63, 68, 70, 31, 37, 47, 48, 54, 60, 64, 66, 30, 38,
46, 46, 52, 58, 63, 65, 31, 38, 46, 46, 52, 57, 61, 63, 33, 41, 47, 46,
51, 56, 60, 63, 37, 45, 47, 46, 50, 54, 59, 62, 39, 46, 48, 47, 51, 55,
58, 61, 42, 46, 50, 50, 53, 57, 60, 60, 49, 48, 53, 54, 57, 60, 61, 61,
48, 47, 53, 55, 58, 62, 64, 63, 48, 46, 53, 56, 60, 64, 65, 65, 49, 45,
53, 59, 64, 67, 67, 66, 50, 46, 54, 61, 66, 70, 71, 69, 51, 47, 54, 61,
68, 71, 72, 70, 52, 47, 54, 63, 71, 75, 75, 74, 54, 49, 55, 65, 73, 78,
78, 74, 55, 49, 56, 65, 74, 79, 79, 78, 57, 50, 56, 66, 76, 82, 83, 79,
60, 53, 58, 68, 79, 85, 85, 82, 62, 54, 60, 69, 81, 87, 87, 84, 63, 55,
60, 70, 82, 89, 91, 87, 64, 56, 61, 71, 83, 90, 92, 89, 66, 58, 62, 72,
84, 91, 95, 91, 67, 59, 63, 71, 83, 93, 96, 94, 68, 60, 64, 71, 81, 94,
97, 96, 69, 61, 65, 72, 80, 91, 99, 100, 70, 62, 65, 73, 81, 89, 98,
101, 71, 64, 65, 73, 82, 90, 99, 103, 72, 65, 65, 72, 82, 92, 100, 103,
73, 66, 65, 72, 81, 90, 100, 105, 74, 67, 65, 71, 79, 89, 98, 105, 75,
68, 65, 71, 78, 87, 96, 105,
// Size 32x8
32, 31, 30, 31, 33, 37, 39, 42, 49, 48, 48, 49, 50, 51, 52, 54, 55, 57,
60, 62, 63, 64, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 35, 37, 38, 38,
41, 45, 46, 46, 48, 47, 46, 45, 46, 47, 47, 49, 49, 50, 53, 54, 55, 56,
58, 59, 60, 61, 62, 64, 65, 66, 67, 68, 48, 47, 46, 46, 47, 47, 48, 50,
53, 53, 53, 53, 54, 54, 54, 55, 56, 56, 58, 60, 60, 61, 62, 63, 64, 65,
65, 65, 65, 65, 65, 50, 48, 46, 46, 46, 46, 47, 50, 54, 55, 56, 59,
61, 61, 63, 65, 65, 66, 68, 69, 70, 71, 72, 71, 71, 72, 73, 73, 72, 72,
71, 71, 57, 54, 52, 52, 51, 50, 51, 53, 57, 58, 60, 64, 66, 68, 71, 73,
74, 76, 79, 81, 82, 83, 84, 83, 81, 80, 81, 82, 82, 81, 79, 78, 63, 60,
58, 57, 56, 54, 55, 57, 60, 62, 64, 67, 70, 71, 75, 78, 79, 82, 85, 87,
89, 90, 91, 93, 94, 91, 89, 90, 92, 90, 89, 87, 68, 64, 63, 61, 60, 59,
58, 60, 61, 64, 65, 67, 71, 72, 75, 78, 79, 83, 85, 87, 91, 92, 95, 96,

```

```

97, 99, 98, 99, 100, 100, 98, 96, 70, 66, 65, 63, 63, 62, 61, 60, 61,
63, 65, 66, 69, 70, 74, 74, 78, 79, 82, 84, 87, 89, 91, 94, 96, 100,
101, 103, 103, 105, 105, 105,
// Size 4x32
31, 49, 61, 69, 31, 47, 57, 65, 32, 45, 55, 64, 33, 45, 55, 62, 36, 46,
54, 61, 40, 45, 52, 61, 41, 47, 54, 59, 43, 49, 56, 59, 46, 53, 59, 62,
46, 55, 61, 63, 46, 56, 62, 65, 45, 58, 66, 67, 46, 59, 68, 68, 47, 60,
70, 72, 48, 61, 73, 73, 50, 62, 76, 76, 51, 63, 77, 78, 52, 64, 79, 81,
54, 65, 82, 84, 56, 66, 84, 85, 57, 67, 86, 89, 59, 68, 87, 90, 60, 69,
88, 93, 61, 68, 86, 96, 62, 67, 86, 98, 63, 66, 88, 99, 64, 66, 87, 100,
65, 67, 85, 102, 65, 68, 83, 102, 66, 69, 85, 102, 67, 70, 86, 103, 68,
71, 87, 105,
// Size 32x4
31, 31, 32, 33, 36, 40, 41, 43, 46, 46, 46, 45, 46, 47, 48, 50, 51, 52,
54, 56, 57, 59, 60, 61, 62, 63, 64, 65, 65, 66, 67, 68, 49, 47, 45, 45,
46, 45, 47, 49, 53, 55, 56, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
69, 68, 67, 66, 66, 67, 68, 69, 70, 71, 61, 57, 55, 55, 54, 52, 54, 56,
59, 61, 62, 66, 68, 70, 73, 76, 77, 79, 82, 84, 86, 87, 88, 86, 86, 88,
87, 85, 83, 85, 86, 87, 69, 65, 64, 62, 61, 61, 59, 59, 62, 63, 65, 67,
68, 72, 73, 76, 78, 81, 84, 85, 89, 90, 93, 96, 98, 99, 100, 102, 102,
102, 103, 105
},
{
{ // Luma
// Size 4x4
32, 38, 63, 86, 38, 56, 78, 97, 63, 78, 113, 130, 86, 97, 130, 169,
// Size 8x8
32, 32, 35, 46, 57, 76, 85, 96, 32, 34, 37, 45, 54, 70, 79, 90, 35, 37,
48, 56, 64, 79, 87, 93, 46, 45, 56, 70, 80, 96, 100, 105, 57, 54, 64,
80, 93, 111, 121, 122, 76, 70, 79, 96, 111, 134, 138, 144, 85, 79, 87,
100, 121, 138, 156, 168, 96, 90, 93, 105, 122, 144, 168, 184,
// Size 16x16
32, 31, 31, 32, 34, 39, 44, 49, 58, 65, 71, 81, 87, 93, 98, 104, 31, 32,
32, 32, 34, 38, 41, 46, 54, 60, 66, 75, 81, 86, 92, 98, 31, 32, 33, 34,
36, 39, 42, 46, 53, 59, 64, 73, 78, 83, 88, 94, 32, 32, 34, 35, 37, 40,
42, 46, 52, 58, 63, 71, 75, 80, 86, 92, 34, 34, 36, 37, 42, 47, 50, 53,
59, 65, 70, 77, 82, 85, 89, 92, 39, 38, 39, 40, 47, 54, 58, 62, 68, 73,
78, 85, 90, 90, 96, 98, 44, 41, 42, 42, 50, 58, 63, 68, 74, 79, 84, 91,
96, 98, 102, 104, 49, 46, 46, 46, 53, 62, 68, 73, 81, 87, 92, 99, 103,
107, 109, 112, 58, 54, 53, 52, 59, 68, 74, 81, 90, 97, 102, 110, 114,
118, 117, 121, 65, 60, 59, 58, 65, 73, 79, 87, 97, 105, 111, 120, 125,
125, 126, 130, 71, 66, 64, 63, 70, 78, 84, 92, 102, 111, 117, 127, 133,
134, 136, 141, 81, 75, 73, 71, 77, 85, 91, 99, 110, 120, 127, 137, 143,
145, 148, 152, 87, 81, 78, 75, 82, 90, 96, 103, 114, 125, 133, 143, 150,
156, 160, 163, 93, 86, 83, 80, 85, 90, 98, 107, 118, 125, 134, 145, 156,
163, 169, 177, 98, 92, 88, 86, 89, 96, 102, 109, 117, 126, 136, 148,
160, 169, 176, 184, 104, 98, 94, 92, 92, 98, 104, 112, 121, 130, 141,
152, 163, 177, 184, 191,
// Size 32x32
32, 31, 31, 31, 31, 32, 32, 34, 34, 36, 39, 41, 44, 48, 49, 54, 58, 59,
65, 69, 71, 80, 81, 83, 87, 90, 93, 95, 98, 101, 104, 107, 31, 32, 32,
32, 32, 32, 32, 34, 34, 35, 38, 39, 42, 46, 47, 51, 55, 57, 62, 66, 68,
76, 77, 78, 83, 85, 88, 90, 93, 96, 99, 101, 31, 32, 32, 32, 32, 32, 32,
33, 34, 34, 38, 39, 41, 45, 46, 50, 54, 55, 60, 64, 66, 73, 75, 76, 81,
83, 86, 89, 92, 95, 98, 101, 31, 32, 32, 32, 32, 32, 32, 33, 34, 34, 37,
38, 41, 44, 45, 49, 53, 54, 59, 63, 65, 72, 74, 75, 79, 81, 84, 86, 89,
91, 94, 97, 31, 32, 32, 32, 33, 33, 34, 35, 36, 36, 39, 40, 42, 45, 46,
50, 53, 54, 59, 63, 64, 71, 73, 74, 78, 80, 83, 85, 88, 91, 94, 97, 32,
32, 32, 32, 33, 34, 34, 36, 36, 37, 40, 40, 42, 45, 46, 49, 53, 54, 58,
62, 63, 70, 72, 73, 77, 79, 82, 85, 87, 90, 92, 95, 32, 32, 32, 32, 34,
34, 35, 37, 37, 38, 40, 41, 42, 45, 46, 49, 52, 54, 58, 61, 63, 69, 71,
72, 75, 78, 80, 83, 86, 89, 92, 95, 34, 34, 33, 33, 35, 36, 37, 39, 41,
42, 45, 46, 47, 50, 51, 54, 57, 59, 63, 66, 68, 74, 75, 76, 80, 81, 82,
83, 85, 87, 90, 93, 34, 34, 34, 34, 36, 36, 37, 41, 42, 45, 47, 48, 50,
53, 53, 56, 59, 61, 65, 68, 70, 76, 77, 78, 82, 83, 85, 88, 89, 90, 92,
93, 36, 35, 34, 34, 36, 37, 38, 42, 45, 48, 50, 51, 54, 56, 57, 60, 63,
64, 68, 71, 73, 79, 80, 81, 85, 87, 89, 89, 90, 93, 96, 99, 39, 38, 38,
37, 39, 40, 40, 45, 47, 50, 54, 55, 58, 61, 62, 65, 68, 69, 73, 76, 78,
84, 85, 86, 90, 89, 90, 93, 96, 97, 98, 99, 41, 39, 39, 38, 40, 40, 41,
46, 48, 51, 55, 56, 59, 62, 63, 67, 70, 71, 75, 78, 80, 86, 87, 88, 91,

```

93, 96, 97, 97, 99, 102, 105, 44, 42, 41, 41, 42, 42, 42, 47, 50, 54,
 58, 59, 63, 66, 68, 71, 74, 75, 79, 83, 84, 90, 91, 92, 96, 98, 98, 99,
 102, 104, 104, 105, 48, 46, 45, 44, 45, 45, 45, 50, 53, 56, 61, 62, 66,
 70, 71, 76, 79, 80, 85, 88, 90, 96, 97, 98, 101, 100, 102, 105, 105,
 105, 109, 112, 49, 47, 46, 45, 46, 46, 46, 51, 53, 57, 62, 63, 68, 71,
 73, 77, 81, 82, 87, 90, 92, 98, 99, 100, 103, 106, 107, 106, 109, 112,
 112, 112, 54, 51, 50, 49, 50, 49, 49, 54, 56, 60, 65, 67, 71, 76, 77,
 82, 86, 87, 92, 96, 97, 104, 105, 106, 110, 110, 109, 113, 114, 113,
 116, 120, 58, 55, 54, 53, 53, 53, 52, 57, 59, 63, 68, 70, 74, 79, 81,
 86, 90, 91, 97, 100, 102, 109, 110, 111, 114, 114, 118, 116, 117, 121,
 121, 120, 59, 57, 55, 54, 54, 54, 54, 59, 61, 64, 69, 71, 75, 80, 82,
 87, 91, 93, 99, 102, 104, 111, 112, 113, 117, 121, 120, 122, 124, 122,
 125, 129, 65, 62, 60, 59, 59, 58, 58, 63, 65, 68, 73, 75, 79, 85, 87,
 92, 97, 99, 105, 109, 111, 118, 120, 121, 125, 124, 125, 127, 126, 130,
 130, 129, 69, 66, 64, 63, 63, 62, 61, 66, 68, 71, 76, 78, 83, 88, 90,
 96, 100, 102, 109, 113, 115, 123, 125, 126, 129, 130, 131, 130, 134,
 133, 135, 139, 71, 68, 66, 65, 64, 63, 63, 68, 70, 73, 78, 80, 84, 90,
 92, 97, 102, 104, 111, 115, 117, 125, 127, 128, 133, 136, 134, 139, 136,
 139, 141, 140, 80, 76, 73, 72, 71, 70, 69, 74, 76, 79, 84, 86, 90, 96,
 98, 104, 109, 111, 118, 123, 125, 134, 136, 137, 142, 138, 143, 140,
 144, 144, 144, 149, 81, 77, 75, 74, 73, 72, 71, 75, 77, 80, 85, 87, 91,
 97, 99, 105, 110, 112, 120, 125, 127, 136, 137, 139, 143, 148, 145, 148,
 148, 150, 152, 149, 83, 78, 76, 75, 74, 73, 72, 76, 78, 81, 86, 88, 92,
 98, 100, 106, 111, 113, 121, 126, 128, 137, 139, 140, 145, 149, 153,
 153, 154, 155, 155, 161, 87, 83, 81, 79, 78, 77, 75, 80, 82, 85, 90, 91,
 96, 101, 103, 110, 114, 117, 125, 129, 133, 142, 143, 145, 150, 151,
 156, 159, 160, 160, 163, 161, 90, 85, 83, 81, 80, 79, 78, 81, 83, 87,
 89, 93, 98, 100, 106, 110, 114, 121, 124, 130, 136, 138, 148, 149, 151,
 156, 157, 162, 166, 168, 166, 172, 93, 88, 86, 84, 83, 82, 80, 82, 85,
 89, 90, 96, 98, 102, 107, 109, 118, 120, 125, 131, 134, 143, 145, 153,
 156, 157, 163, 164, 169, 172, 177, 172, 95, 90, 89, 86, 85, 85, 83, 83,
 88, 89, 93, 97, 99, 105, 106, 113, 116, 122, 127, 130, 139, 140, 148,
 153, 159, 162, 164, 169, 170, 176, 179, 185, 98, 93, 92, 89, 88, 87, 86,
 85, 89, 90, 96, 97, 102, 105, 109, 114, 117, 124, 126, 134, 136, 144,
 148, 154, 160, 166, 169, 170, 176, 177, 184, 186, 101, 96, 95, 91, 91,
 90, 89, 87, 90, 93, 97, 99, 104, 105, 112, 113, 121, 122, 130, 133, 139,
 144, 150, 155, 160, 168, 172, 176, 177, 184, 185, 191, 104, 99, 98, 94,
 94, 92, 92, 90, 92, 96, 98, 102, 104, 109, 112, 116, 121, 125, 130, 135,
 141, 144, 152, 155, 163, 166, 177, 179, 184, 185, 191, 192, 107, 101,
 101, 97, 97, 95, 95, 93, 93, 99, 99, 105, 105, 112, 112, 120, 120, 129,
 129, 139, 140, 149, 149, 161, 161, 172, 172, 185, 186, 191, 192, 199,
 // Size 4x8
 32, 38, 62, 86, 32, 40, 58, 80, 34, 51, 68, 85, 44, 61, 85, 101, 54, 69,
 98, 117, 72, 84, 118, 136, 82, 89, 129, 157, 92, 98, 127, 165,
 // Size 8x4
 32, 32, 34, 44, 54, 72, 82, 92, 38, 40, 51, 61, 69, 84, 89, 98, 62, 58,
 68, 85, 98, 118, 129, 127, 86, 80, 85, 101, 117, 136, 157, 165,
 // Size 8x16
 32, 32, 36, 44, 58, 79, 88, 93, 31, 32, 35, 41, 54, 73, 81, 88, 32, 33,
 36, 42, 53, 71, 78, 84, 32, 34, 38, 42, 52, 69, 76, 82, 34, 36, 44, 50,
 59, 75, 81, 84, 39, 39, 50, 58, 68, 84, 88, 90, 44, 42, 53, 63, 74, 90,
 97, 97, 49, 46, 57, 67, 81, 97, 104, 105, 57, 53, 63, 74, 90, 108, 111,
 113, 65, 59, 68, 79, 97, 118, 123, 122, 71, 64, 73, 84, 102, 125, 135,
 131, 81, 72, 80, 91, 110, 135, 145, 141, 87, 77, 85, 96, 114, 140, 148,
 151, 92, 83, 88, 102, 117, 133, 153, 163, 98, 88, 89, 103, 121, 141,
 160, 169, 103, 94, 92, 103, 119, 137, 158, 175,
 // Size 16x8
 32, 31, 32, 32, 34, 39, 44, 49, 57, 65, 71, 81, 87, 92, 98, 103, 32, 32,
 33, 34, 36, 39, 42, 46, 53, 59, 64, 72, 77, 83, 88, 94, 36, 35, 36, 38,
 44, 50, 53, 57, 63, 68, 73, 80, 85, 88, 89, 92, 44, 41, 42, 42, 50, 58,
 63, 67, 74, 79, 84, 91, 96, 102, 103, 103, 58, 54, 53, 52, 59, 68, 74,
 81, 90, 97, 102, 110, 114, 117, 121, 119, 79, 73, 71, 69, 75, 84, 90,
 97, 108, 118, 125, 135, 140, 133, 141, 137, 88, 81, 78, 76, 81, 88, 97,
 104, 111, 123, 135, 145, 148, 153, 160, 158, 93, 88, 84, 82, 84, 90, 97,
 105, 113, 122, 131, 141, 151, 163, 169, 175,
 // Size 16x32
 32, 31, 32, 32, 36, 39, 44, 53, 58, 65, 79, 81, 88, 90, 93, 96, 31, 32,
 32, 32, 35, 38, 42, 51, 55, 62, 75, 77, 83, 86, 88, 91, 31, 32, 32, 32,
 35, 38, 41, 50, 54, 60, 73, 75, 81, 84, 88, 91, 31, 32, 32, 33, 34, 37,
 41, 49, 53, 59, 72, 74, 79, 82, 84, 87, 32, 32, 33, 34, 36, 39, 42, 50,
 53, 59, 71, 72, 78, 81, 84, 87, 32, 32, 34, 34, 37, 40, 42, 49, 53, 58,

```

70, 71, 77, 80, 83, 85, 32, 33, 34, 35, 38, 40, 42, 49, 52, 58, 69, 70,
76, 78, 82, 86, 34, 34, 35, 37, 42, 45, 48, 54, 57, 63, 73, 75, 79, 79,
81, 83, 34, 34, 36, 37, 44, 47, 50, 56, 59, 65, 75, 77, 81, 83, 84, 84,
36, 34, 37, 38, 48, 51, 54, 60, 63, 68, 78, 80, 85, 85, 86, 89, 39, 37,
39, 40, 50, 54, 58, 65, 68, 73, 84, 85, 88, 89, 90, 89, 40, 38, 40, 41,
51, 55, 59, 67, 70, 75, 85, 87, 91, 92, 92, 95, 44, 41, 42, 43, 53, 58,
63, 71, 74, 79, 90, 91, 97, 94, 97, 95, 47, 44, 45, 46, 56, 61, 66, 75,
79, 85, 95, 97, 99, 101, 98, 102, 49, 46, 46, 47, 57, 62, 67, 77, 81,
86, 97, 99, 104, 102, 105, 102, 53, 49, 50, 50, 60, 65, 71, 82, 86, 92,
103, 105, 109, 108, 106, 110, 57, 53, 53, 53, 63, 68, 74, 86, 90, 97,
108, 110, 111, 112, 113, 110, 59, 54, 54, 64, 69, 75, 87, 91, 98,
111, 112, 119, 117, 115, 118, 65, 60, 59, 58, 68, 73, 79, 92, 97, 105,
118, 119, 123, 123, 122, 119, 69, 63, 62, 62, 71, 76, 83, 96, 100, 109,
122, 124, 127, 125, 125, 128, 71, 65, 64, 63, 73, 78, 84, 97, 102, 111,
125, 127, 135, 134, 131, 129, 79, 72, 71, 70, 79, 84, 90, 104, 109, 118,
133, 135, 137, 136, 136, 137, 81, 74, 72, 71, 80, 85, 91, 105, 110, 120,
135, 137, 145, 143, 141, 138, 82, 75, 73, 72, 81, 86, 92, 106, 111, 121,
136, 139, 147, 148, 147, 149, 87, 79, 77, 76, 85, 90, 96, 110, 114, 125,
140, 143, 148, 154, 151, 149, 90, 82, 80, 78, 87, 89, 99, 108, 113, 129,
135, 146, 153, 157, 160, 159, 92, 84, 83, 81, 88, 90, 102, 106, 117,
128, 133, 150, 153, 158, 163, 160, 95, 87, 85, 83, 88, 92, 103, 105,
120, 125, 137, 148, 155, 164, 168, 173, 98, 89, 88, 85, 89, 95, 103,
108, 121, 124, 141, 144, 160, 164, 169, 174, 100, 92, 91, 88, 90, 98,
103, 111, 120, 127, 139, 146, 161, 165, 175, 179, 103, 94, 94, 90, 92,
101, 103, 114, 119, 131, 137, 150, 158, 170, 175, 180, 106, 97, 97, 93,
93, 104, 104, 118, 118, 135, 135, 154, 155, 175, 176, 187,
// Size 32x16
32, 31, 31, 31, 32, 32, 32, 34, 34, 36, 39, 40, 44, 47, 49, 53, 57, 59,
65, 69, 71, 79, 81, 82, 87, 90, 92, 95, 98, 100, 103, 106, 31, 32, 32,
32, 32, 32, 33, 34, 34, 34, 37, 38, 41, 44, 46, 49, 53, 54, 60, 63, 65,
72, 74, 75, 79, 82, 84, 87, 89, 92, 94, 97, 32, 32, 32, 32, 33, 34, 34,
35, 36, 37, 39, 40, 42, 45, 46, 50, 53, 54, 59, 62, 64, 71, 72, 73, 77,
80, 83, 85, 88, 91, 94, 97, 32, 32, 32, 33, 34, 34, 35, 37, 37, 38, 40,
41, 43, 46, 47, 50, 53, 54, 58, 62, 63, 70, 71, 72, 76, 78, 81, 83, 85,
88, 90, 93, 36, 35, 35, 34, 36, 37, 38, 42, 44, 48, 50, 51, 53, 56, 57,
60, 63, 64, 68, 71, 73, 79, 80, 81, 85, 87, 88, 88, 89, 90, 92, 93, 39,
38, 38, 37, 39, 40, 40, 45, 47, 51, 54, 55, 58, 61, 62, 65, 68, 69, 73,
76, 78, 84, 85, 86, 90, 89, 90, 92, 95, 98, 101, 104, 44, 42, 41, 41,
42, 42, 42, 48, 50, 54, 58, 59, 63, 66, 67, 71, 74, 75, 79, 83, 84, 90,
91, 92, 96, 99, 102, 103, 103, 103, 103, 104, 53, 51, 50, 49, 50, 49,
49, 54, 56, 60, 65, 67, 71, 75, 77, 82, 86, 87, 92, 96, 97, 104, 105,
106, 110, 108, 106, 105, 108, 111, 114, 118, 58, 55, 54, 53, 53, 53, 52,
57, 59, 63, 68, 70, 74, 79, 81, 86, 90, 91, 97, 100, 102, 109, 110, 111,
114, 113, 117, 120, 121, 120, 119, 118, 65, 62, 60, 59, 59, 58, 58, 63,
65, 68, 73, 75, 79, 85, 86, 92, 97, 98, 105, 109, 111, 118, 120, 121,
125, 129, 128, 125, 124, 127, 131, 135, 79, 75, 73, 72, 71, 70, 69, 73,
75, 78, 84, 85, 90, 95, 97, 103, 108, 111, 118, 122, 125, 133, 135, 136,
140, 135, 133, 137, 141, 139, 137, 135, 81, 77, 75, 74, 72, 71, 70, 75,
77, 80, 85, 87, 91, 97, 99, 105, 110, 112, 119, 124, 127, 135, 137, 139,
143, 146, 150, 148, 144, 146, 150, 154, 88, 83, 81, 79, 78, 77, 76, 79,
81, 85, 88, 91, 97, 99, 104, 109, 111, 119, 123, 127, 135, 137, 145,
147, 148, 153, 153, 155, 160, 161, 158, 155, 90, 86, 84, 82, 81, 80, 78,
79, 83, 85, 89, 92, 94, 101, 102, 108, 112, 117, 123, 125, 134, 136,
143, 148, 154, 157, 158, 164, 164, 165, 170, 175, 93, 88, 88, 84, 84,
83, 82, 81, 84, 86, 90, 92, 97, 98, 105, 106, 113, 115, 122, 125, 131,
136, 141, 147, 151, 160, 163, 168, 169, 175, 175, 176, 96, 91, 91, 87,
87, 85, 86, 83, 84, 89, 89, 95, 95, 102, 102, 110, 110, 118, 119, 128,
129, 137, 138, 149, 149, 159, 160, 173, 174, 179, 180, 187,
// Size 4x16
31, 39, 65, 90, 32, 38, 60, 84, 32, 39, 59, 81, 33, 40, 58, 78, 34, 47,
65, 83, 37, 54, 73, 89, 41, 58, 79, 94, 46, 62, 86, 102, 53, 68, 97,
112, 60, 73, 105, 123, 65, 78, 111, 134, 74, 85, 120, 143, 79, 90, 125,
154, 84, 90, 128, 158, 89, 95, 124, 164, 94, 101, 131, 170,
// Size 16x4
31, 32, 32, 33, 34, 37, 41, 46, 53, 60, 65, 74, 79, 84, 89, 94, 39, 38,
39, 40, 47, 54, 58, 62, 68, 73, 78, 85, 90, 90, 95, 101, 65, 60, 59, 58,
65, 73, 79, 86, 97, 105, 111, 120, 125, 128, 124, 131, 90, 84, 81, 78,
83, 89, 94, 102, 112, 123, 134, 143, 154, 158, 164, 170,
// Size 8x32
32, 32, 36, 44, 58, 79, 88, 93, 31, 32, 35, 42, 55, 75, 83, 88, 31, 32,
35, 41, 54, 73, 81, 88, 31, 32, 34, 41, 53, 72, 79, 84, 32, 33, 36, 42,

```

```

53, 71, 78, 84, 32, 34, 37, 42, 53, 70, 77, 83, 32, 34, 38, 42, 52, 69,
76, 82, 34, 35, 42, 48, 57, 73, 79, 81, 34, 36, 44, 50, 59, 75, 81, 84,
36, 37, 48, 54, 63, 78, 85, 86, 39, 39, 50, 58, 68, 84, 88, 90, 40, 40,
51, 59, 70, 85, 91, 92, 44, 42, 53, 63, 74, 90, 97, 97, 47, 45, 56, 66,
79, 95, 99, 98, 49, 46, 57, 67, 81, 97, 104, 105, 53, 50, 60, 71, 86,
103, 109, 106, 57, 53, 63, 74, 90, 108, 111, 113, 59, 54, 64, 75, 91,
111, 119, 115, 65, 59, 68, 79, 97, 118, 123, 122, 69, 62, 71, 83, 100,
122, 127, 125, 71, 64, 73, 84, 102, 125, 135, 131, 79, 71, 79, 90, 109,
133, 137, 136, 81, 72, 80, 91, 110, 135, 145, 141, 82, 73, 81, 92, 111,
136, 147, 147, 87, 77, 85, 96, 114, 140, 148, 151, 90, 80, 87, 99, 113,
135, 153, 160, 92, 83, 88, 102, 117, 133, 153, 163, 95, 85, 88, 103,
120, 137, 155, 168, 98, 88, 89, 103, 121, 141, 160, 169, 100, 91, 90,
103, 120, 139, 161, 175, 103, 94, 92, 103, 119, 137, 158, 175, 106, 97,
93, 104, 118, 135, 155, 176,
// Size 32x8
32, 31, 31, 31, 32, 32, 32, 34, 34, 36, 39, 40, 44, 47, 49, 53, 57, 59,
65, 69, 71, 79, 81, 82, 87, 90, 92, 95, 98, 100, 103, 106, 32, 32, 32,
32, 33, 34, 34, 35, 36, 37, 39, 40, 42, 45, 46, 50, 53, 54, 59, 62, 64,
71, 72, 73, 77, 80, 83, 85, 88, 91, 94, 97, 36, 35, 35, 34, 36, 37, 38,
42, 44, 48, 50, 51, 53, 56, 57, 60, 63, 64, 68, 71, 73, 79, 80, 81, 85,
87, 88, 88, 89, 90, 92, 93, 44, 42, 41, 41, 42, 42, 42, 48, 50, 54, 58,
59, 63, 66, 67, 71, 74, 75, 79, 83, 84, 90, 91, 92, 96, 99, 102, 103,
103, 103, 103, 104, 58, 55, 54, 53, 53, 53, 52, 57, 59, 63, 68, 70, 74,
79, 81, 86, 90, 91, 97, 100, 102, 109, 110, 111, 114, 113, 117, 120,
121, 120, 119, 118, 79, 75, 73, 72, 71, 70, 69, 73, 75, 78, 84, 85, 90,
95, 97, 103, 108, 111, 118, 122, 125, 133, 135, 136, 140, 135, 133, 137,
141, 139, 137, 135, 88, 83, 81, 79, 78, 77, 76, 79, 81, 85, 88, 91, 97,
99, 104, 109, 111, 119, 123, 127, 135, 137, 145, 147, 148, 153, 153,
155, 160, 161, 158, 155, 93, 88, 88, 84, 84, 83, 82, 81, 84, 86, 90, 92,
97, 98, 105, 106, 113, 115, 122, 125, 131, 136, 141, 147, 151, 160, 163,
168, 169, 175, 175, 176,
// Size 4x32
31, 39, 65, 90, 32, 38, 62, 86, 32, 38, 60, 84, 32, 37, 59, 82, 32, 39,
59, 81, 32, 40, 58, 80, 33, 40, 58, 78, 34, 45, 63, 79, 34, 47, 65, 83,
34, 51, 68, 85, 37, 54, 73, 89, 38, 55, 75, 92, 41, 58, 79, 94, 44, 61,
85, 101, 46, 62, 86, 102, 49, 65, 92, 108, 53, 68, 97, 112, 54, 69, 98,
117, 60, 73, 105, 123, 63, 76, 109, 125, 65, 78, 111, 134, 72, 84, 118,
136, 74, 85, 120, 143, 75, 86, 121, 148, 79, 90, 125, 154, 82, 89, 129,
157, 84, 90, 128, 158, 87, 92, 125, 164, 89, 95, 124, 164, 92, 98, 127,
165, 94, 101, 131, 170, 97, 104, 135, 175,
// Size 32x4
31, 32, 32, 32, 32, 32, 33, 34, 34, 34, 37, 38, 41, 44, 46, 49, 53, 54,
60, 63, 65, 72, 74, 75, 79, 82, 84, 87, 89, 92, 94, 97, 39, 38, 38, 37,
39, 40, 40, 45, 47, 51, 54, 55, 58, 61, 62, 65, 68, 69, 73, 76, 78, 84,
85, 86, 90, 89, 90, 92, 95, 98, 101, 104, 65, 62, 60, 59, 59, 58, 58,
63, 65, 68, 73, 75, 79, 85, 86, 92, 97, 98, 105, 109, 111, 118, 120,
121, 125, 129, 128, 125, 124, 127, 131, 135, 90, 86, 84, 82, 81, 80, 78,
79, 83, 85, 89, 92, 94, 101, 102, 108, 112, 117, 123, 125, 134, 136,
143, 148, 154, 157, 158, 164, 164, 165, 170, 175,
},
{
// Chroma
// Size 4x4
32, 45, 53, 63, 45, 55, 62, 67, 53, 62, 80, 84, 63, 67, 84, 101,
// Size 8x8
31, 36, 47, 48, 52, 60, 64, 67, 36, 43, 47, 46, 49, 55, 59, 63, 47, 47,
53, 54, 55, 60, 63, 64, 48, 46, 54, 61, 65, 70, 71, 71, 52, 49, 55, 65,
71, 78, 81, 79, 60, 55, 60, 70, 78, 89, 89, 89, 64, 59, 63, 71, 81, 89,
97, 99, 67, 63, 64, 71, 79, 89, 99, 104,
// Size 16x16
32, 30, 33, 36, 44, 48, 49, 51, 54, 57, 60, 64, 67, 68, 70, 72, 30, 31,
35, 39, 44, 46, 46, 47, 50, 53, 55, 59, 61, 64, 66, 68, 33, 35, 39, 43,
46, 46, 45, 47, 49, 51, 53, 57, 59, 61, 63, 65, 36, 39, 43, 47, 47, 46,
45, 46, 48, 50, 52, 55, 57, 58, 61, 63, 44, 44, 46, 47, 50, 51, 51, 51,
53, 54, 56, 59, 61, 61, 63, 62, 48, 46, 46, 46, 51, 54, 55, 56, 58, 60,
61, 64, 65, 64, 66, 66, 49, 46, 45, 45, 51, 55, 58, 60, 62, 63, 65, 68,
69, 69, 69, 69, 51, 47, 47, 46, 51, 56, 60, 62, 65, 67, 69, 72, 73, 74,
73, 73, 54, 50, 49, 48, 53, 58, 62, 65, 70, 73, 75, 78, 79, 79, 77, 77,
57, 53, 51, 50, 54, 60, 63, 67, 73, 76, 79, 82, 84, 83, 82, 82, 60, 55,
53, 52, 56, 61, 65, 69, 75, 79, 82, 86, 88, 87, 86, 87, 64, 59, 57, 55,
59, 64, 68, 72, 78, 82, 86, 90, 93, 92, 91, 92, 67, 61, 59, 57, 61, 65,
69, 73, 79, 84, 88, 93, 95, 96, 96, 96, 68, 64, 61, 58, 61, 64, 69, 74,

```



```

79, 83, 87, 92, 96, 99, 100, 101, 70, 66, 63, 61, 63, 66, 69, 73, 77,
82, 86, 91, 96, 100, 103, 104, 72, 68, 65, 63, 62, 66, 69, 73, 77, 82,
87, 92, 96, 101, 104, 106,
// Size 32x32
32, 31, 30, 30, 33, 35, 36, 41, 44, 49, 48, 48, 49, 50, 51, 52, 54, 55,
57, 59, 60, 63, 64, 65, 67, 68, 68, 69, 70, 71, 72, 73, 31, 31, 31, 31,
34, 36, 38, 42, 44, 47, 47, 47, 47, 48, 48, 50, 51, 52, 54, 56, 57, 60,
61, 61, 63, 64, 65, 66, 67, 67, 68, 69, 30, 31, 31, 31, 35, 37, 39, 42,
44, 47, 46, 46, 46, 47, 47, 48, 50, 51, 53, 54, 55, 58, 59, 60, 61, 63,
64, 65, 66, 67, 68, 69, 30, 31, 31, 32, 35, 37, 40, 42, 44, 46, 45, 45,
45, 46, 46, 47, 49, 50, 52, 53, 54, 57, 58, 58, 60, 61, 62, 63, 63, 64,
65, 66, 33, 34, 35, 35, 39, 41, 43, 45, 46, 47, 46, 46, 45, 46, 47, 47,
49, 49, 51, 53, 53, 56, 57, 57, 59, 60, 61, 62, 63, 64, 65, 66, 35, 36,
37, 37, 41, 43, 45, 46, 46, 47, 46, 46, 45, 46, 46, 47, 48, 49, 50, 52,
53, 55, 56, 58, 59, 60, 61, 62, 63, 64, 64, 36, 38, 39, 40, 43, 45,
47, 47, 47, 48, 46, 46, 45, 46, 46, 47, 48, 48, 50, 51, 52, 54, 55, 55,
57, 58, 58, 59, 61, 62, 63, 64, 41, 42, 42, 42, 45, 46, 47, 48, 49, 50,
49, 49, 49, 50, 50, 50, 51, 52, 53, 54, 55, 57, 58, 58, 60, 60, 59, 59,
60, 61, 61, 62, 44, 44, 44, 44, 46, 46, 47, 49, 50, 51, 51, 51, 51, 51,
51, 52, 53, 53, 54, 56, 56, 59, 59, 59, 61, 61, 61, 62, 63, 62, 62, 62,
49, 47, 47, 46, 47, 47, 48, 50, 51, 53, 53, 53, 53, 54, 54, 54, 55, 55,
56, 58, 58, 60, 61, 61, 63, 63, 64, 63, 63, 64, 65, 66, 48, 47, 46, 45,
46, 46, 46, 49, 51, 53, 54, 54, 55, 56, 56, 57, 58, 59, 60, 61, 61, 63,
64, 64, 65, 65, 64, 65, 66, 66, 66, 66, 48, 47, 46, 45, 46, 46, 46, 49,
51, 53, 54, 55, 56, 57, 57, 58, 59, 60, 61, 62, 63, 65, 65, 65, 66, 67,
68, 67, 67, 68, 69, 49, 47, 46, 45, 45, 45, 45, 49, 51, 53, 55, 56,
58, 59, 60, 61, 62, 62, 63, 65, 65, 67, 68, 68, 69, 70, 69, 69, 69, 70,
69, 69, 50, 48, 47, 46, 46, 46, 46, 50, 51, 54, 56, 57, 59, 61, 62, 63,
64, 65, 66, 68, 68, 70, 71, 71, 72, 71, 71, 72, 71, 71, 71, 72, 51, 48,
47, 46, 47, 46, 46, 50, 51, 54, 56, 57, 60, 62, 62, 64, 65, 66, 67, 69,
69, 71, 72, 72, 73, 74, 74, 72, 73, 74, 73, 73, 52, 50, 48, 47, 47, 47,
47, 50, 52, 54, 57, 58, 61, 63, 64, 66, 68, 68, 70, 72, 72, 75, 75, 75,
77, 76, 75, 76, 76, 74, 75, 76, 54, 51, 50, 49, 49, 48, 48, 51, 53, 55,
58, 59, 62, 64, 65, 68, 70, 70, 73, 74, 75, 77, 78, 78, 79, 78, 79, 78,
77, 78, 77, 77, 55, 52, 51, 50, 49, 49, 48, 52, 53, 55, 59, 60, 62, 65,
66, 68, 70, 71, 73, 75, 76, 78, 79, 79, 80, 81, 80, 80, 81, 79, 79, 81,
57, 54, 53, 52, 51, 50, 50, 53, 54, 56, 60, 61, 63, 66, 67, 70, 73, 73,
76, 78, 79, 82, 82, 83, 84, 83, 83, 83, 82, 83, 82, 81, 59, 56, 54, 53,
53, 52, 51, 54, 56, 58, 61, 62, 65, 68, 69, 72, 74, 75, 78, 80, 81, 84,
85, 85, 86, 86, 86, 84, 85, 84, 84, 85, 60, 57, 55, 54, 53, 53, 52, 55,
56, 58, 61, 63, 65, 68, 69, 72, 75, 76, 79, 81, 82, 85, 86, 86, 88, 88,
87, 88, 86, 87, 87, 85, 63, 60, 58, 57, 56, 55, 54, 57, 59, 60, 63, 65,
67, 70, 71, 75, 77, 78, 82, 84, 85, 89, 89, 90, 92, 89, 91, 89, 90, 89,
88, 89, 64, 61, 59, 58, 57, 56, 55, 58, 59, 61, 64, 65, 68, 71, 72, 75,
78, 79, 82, 85, 86, 89, 90, 91, 93, 94, 92, 92, 91, 91, 92, 90, 65, 61,
60, 58, 57, 56, 55, 58, 59, 61, 64, 65, 68, 71, 72, 75, 78, 79, 83, 85,
86, 90, 91, 91, 93, 94, 95, 94, 94, 94, 93, 94, 67, 63, 61, 60, 59, 58,
57, 60, 61, 63, 65, 66, 69, 72, 73, 77, 79, 80, 84, 86, 88, 92, 93, 93,
95, 95, 96, 97, 96, 95, 96, 94, 68, 64, 63, 61, 60, 59, 58, 60, 61, 63,
65, 67, 70, 71, 74, 76, 78, 81, 83, 86, 88, 89, 94, 94, 95, 97, 97, 98,
99, 99, 97, 99, 68, 65, 64, 62, 61, 60, 58, 59, 61, 64, 64, 68, 69, 71,
74, 75, 79, 80, 83, 86, 87, 91, 92, 95, 96, 97, 99, 99, 100, 100, 101,
99, 69, 66, 65, 63, 62, 61, 59, 59, 62, 63, 65, 67, 69, 72, 72, 76, 78,
80, 83, 84, 88, 89, 92, 94, 97, 98, 99, 101, 100, 102, 102, 104, 70, 67,
66, 63, 63, 62, 61, 60, 63, 63, 66, 67, 69, 71, 73, 76, 77, 81, 82, 85,
86, 90, 91, 94, 96, 99, 100, 103, 102, 104, 104, 71, 67, 67, 64,
64, 63, 62, 61, 62, 64, 66, 67, 70, 71, 74, 74, 78, 79, 83, 84, 87, 89,
91, 94, 95, 99, 100, 102, 102, 104, 104, 106, 72, 68, 68, 65, 65, 64,
63, 61, 62, 65, 66, 68, 69, 71, 73, 75, 77, 79, 82, 84, 87, 88, 92, 93,
96, 97, 101, 102, 104, 104, 106, 106, 73, 69, 69, 66, 66, 64, 64, 62,
62, 66, 66, 69, 72, 73, 76, 77, 81, 81, 85, 85, 89, 90, 94, 94, 99,
99, 104, 104, 106, 106, 108,
// Size 4x8
31, 47, 54, 64, 38, 46, 50, 60, 46, 53, 57, 62, 46, 56, 66, 71, 50, 59,
74, 79, 57, 64, 82, 88, 61, 65, 85, 97, 65, 67, 82, 99,
// Size 8x4
31, 38, 46, 46, 50, 57, 61, 65, 47, 46, 53, 56, 59, 64, 65, 67, 54, 50,
57, 66, 74, 82, 85, 82, 64, 60, 62, 71, 79, 88, 97, 99,
// Size 8x16
32, 34, 48, 49, 54, 63, 67, 69, 31, 36, 46, 46, 50, 58, 62, 65, 33, 40,
47, 46, 49, 56, 59, 62, 37, 44, 47, 45, 48, 54, 57, 60, 44, 46, 51, 51,

```

```

53, 59, 60, 61, 48, 46, 53, 56, 58, 64, 64, 64, 49, 45, 53, 58, 62, 67,
70, 68, 51, 47, 54, 60, 65, 71, 73, 72, 54, 49, 55, 62, 70, 77, 77, 76,
57, 51, 56, 64, 73, 82, 83, 81, 60, 53, 58, 65, 75, 85, 89, 85, 64, 57,
61, 68, 78, 89, 93, 89, 66, 59, 63, 69, 79, 91, 94, 93, 68, 61, 63, 71,
79, 87, 96, 98, 70, 63, 63, 70, 80, 89, 97, 100, 72, 65, 63, 69, 77, 86,
95, 102,
// Size 16x8
32, 31, 33, 37, 44, 48, 49, 51, 54, 57, 60, 64, 66, 68, 70, 72, 34, 36,
40, 44, 46, 46, 45, 47, 49, 51, 53, 57, 59, 61, 63, 65, 48, 46, 47, 47,
51, 53, 53, 54, 55, 56, 58, 61, 63, 63, 63, 63, 49, 46, 46, 45, 51, 56,
58, 60, 62, 64, 65, 68, 69, 71, 70, 69, 54, 50, 49, 48, 53, 58, 62, 65,
70, 73, 75, 78, 79, 79, 80, 77, 63, 58, 56, 54, 59, 64, 67, 71, 77, 82,
85, 89, 91, 87, 89, 86, 67, 62, 59, 57, 60, 64, 70, 73, 77, 83, 89, 93,
94, 96, 97, 95, 69, 65, 62, 60, 61, 64, 68, 72, 76, 81, 85, 89, 93, 98,
100, 102,
// Size 16x32
32, 31, 34, 37, 48, 48, 49, 52, 54, 57, 63, 64, 67, 68, 69, 69, 31, 31,
35, 38, 47, 47, 47, 50, 51, 54, 60, 61, 63, 64, 65, 66, 31, 32, 36, 39,
46, 46, 46, 48, 50, 53, 58, 59, 62, 63, 65, 66, 30, 32, 36, 40, 46, 45,
45, 48, 49, 52, 57, 58, 60, 61, 62, 63, 33, 36, 40, 43, 47, 46, 46, 47,
49, 51, 56, 57, 59, 60, 62, 63, 35, 38, 42, 45, 47, 46, 45, 47, 48, 50,
55, 56, 58, 60, 61, 61, 37, 40, 44, 47, 47, 46, 45, 47, 48, 50, 54, 55,
57, 58, 60, 61, 42, 43, 45, 47, 50, 50, 49, 50, 51, 53, 57, 58, 59, 58,
59, 59, 44, 44, 46, 47, 51, 51, 51, 52, 53, 54, 59, 59, 60, 61, 61, 60,
49, 46, 47, 48, 53, 53, 53, 54, 55, 57, 60, 61, 63, 62, 62, 63, 48, 46,
46, 47, 53, 54, 56, 57, 58, 60, 64, 64, 64, 64, 63, 48, 45, 46, 46,
53, 55, 56, 58, 59, 61, 65, 65, 66, 66, 65, 66, 49, 45, 45, 46, 53, 56,
58, 61, 62, 64, 67, 68, 70, 67, 68, 66, 50, 46, 46, 46, 54, 56, 59, 63,
65, 66, 70, 71, 70, 71, 68, 70, 51, 47, 47, 47, 54, 57, 60, 64, 65, 68,
71, 72, 73, 71, 72, 70, 52, 48, 47, 47, 54, 57, 61, 66, 68, 71, 75, 75,
76, 75, 73, 73, 54, 49, 49, 48, 55, 58, 62, 68, 70, 73, 77, 78, 77, 77,
76, 74, 54, 50, 49, 49, 55, 59, 62, 68, 70, 74, 78, 79, 81, 79, 77, 78,
57, 52, 51, 50, 56, 60, 64, 70, 73, 76, 82, 82, 83, 82, 81, 78, 59, 54,
52, 52, 58, 61, 65, 72, 74, 78, 84, 85, 85, 83, 82, 82, 60, 54, 53, 52,
58, 62, 65, 72, 75, 79, 85, 86, 89, 87, 85, 82, 63, 57, 56, 55, 60, 64,
67, 75, 77, 82, 89, 90, 90, 88, 87, 86, 64, 58, 57, 55, 61, 64, 68, 75,
78, 82, 89, 90, 93, 91, 89, 87, 64, 59, 57, 56, 61, 65, 68, 75, 78, 83,
90, 91, 94, 93, 92, 91, 66, 60, 59, 57, 63, 66, 69, 77, 79, 84, 91, 93,
94, 95, 93, 91, 67, 61, 60, 58, 63, 65, 70, 75, 78, 85, 88, 93, 96, 97,
97, 95, 68, 62, 61, 59, 63, 64, 71, 74, 79, 84, 87, 94, 96, 97, 98, 96,
69, 63, 62, 60, 63, 65, 71, 72, 80, 82, 88, 93, 96, 99, 100, 101, 70,
64, 63, 60, 63, 66, 70, 73, 80, 81, 89, 90, 97, 99, 100, 101, 71, 65,
64, 61, 63, 67, 70, 74, 78, 82, 88, 90, 97, 99, 102, 103, 72, 65, 65,
62, 63, 68, 69, 75, 77, 83, 86, 92, 95, 100, 102, 103, 73, 66, 66, 63,
63, 69, 69, 76, 76, 84, 84, 93, 93, 101, 101, 105,
// Size 32x16
32, 31, 31, 30, 33, 35, 37, 42, 44, 49, 48, 48, 49, 50, 51, 52, 54, 54,
57, 59, 60, 63, 64, 64, 66, 67, 68, 69, 70, 71, 72, 73, 31, 31, 32, 32,
36, 38, 40, 43, 44, 46, 46, 45, 45, 46, 47, 48, 49, 50, 52, 54, 54, 57,
58, 59, 60, 61, 62, 63, 64, 65, 65, 66, 34, 35, 36, 36, 40, 42, 44, 45,
46, 47, 46, 46, 45, 46, 47, 47, 49, 49, 51, 52, 53, 56, 57, 57, 59, 60,
61, 62, 63, 64, 65, 66, 37, 38, 39, 40, 43, 45, 47, 47, 47, 48, 47, 46,
46, 46, 47, 47, 48, 49, 50, 52, 52, 55, 55, 56, 57, 58, 59, 60, 60, 61,
62, 63, 48, 47, 46, 46, 47, 47, 47, 50, 51, 53, 53, 53, 53, 54, 54, 54,
55, 55, 56, 58, 58, 60, 61, 61, 63, 63, 63, 63, 63, 63, 63, 48, 47,
46, 45, 46, 46, 46, 50, 51, 53, 54, 55, 56, 57, 57, 58, 59, 60, 61,
62, 64, 64, 65, 66, 65, 64, 65, 66, 67, 68, 69, 49, 47, 46, 45, 46, 45,
45, 49, 51, 53, 56, 56, 58, 59, 60, 61, 62, 62, 64, 65, 65, 67, 68, 68,
69, 70, 71, 71, 70, 70, 69, 69, 52, 50, 48, 48, 47, 47, 47, 50, 52, 54,
57, 58, 61, 63, 64, 66, 68, 68, 70, 72, 72, 75, 75, 75, 77, 75, 74, 72,
73, 74, 75, 76, 54, 51, 50, 49, 49, 48, 48, 51, 53, 55, 58, 59, 62, 65,
65, 68, 70, 70, 73, 74, 75, 77, 78, 78, 79, 78, 79, 80, 80, 78, 77, 76,
57, 54, 53, 52, 51, 50, 50, 53, 54, 57, 60, 61, 64, 66, 68, 71, 73, 74,
76, 78, 79, 82, 82, 83, 84, 85, 84, 82, 81, 82, 83, 84, 63, 60, 58, 57,
56, 55, 54, 57, 59, 60, 64, 65, 67, 70, 71, 75, 77, 78, 82, 84, 85, 89,
89, 90, 91, 88, 87, 88, 89, 88, 86, 84, 64, 61, 59, 58, 57, 56, 55, 58,
59, 61, 64, 65, 68, 71, 72, 75, 78, 79, 82, 85, 86, 90, 90, 91, 93, 93,
94, 93, 90, 90, 92, 93, 67, 63, 62, 60, 59, 58, 57, 59, 60, 63, 64, 66,
70, 70, 73, 76, 77, 81, 83, 85, 89, 90, 93, 94, 94, 96, 96, 96, 97, 97,
95, 93, 68, 64, 63, 61, 60, 60, 58, 58, 61, 62, 64, 66, 67, 71, 71, 75,
77, 79, 82, 83, 87, 88, 91, 93, 95, 97, 97, 99, 99, 99, 100, 101, 69,

```

```

65, 65, 62, 62, 61, 60, 59, 61, 62, 64, 65, 68, 68, 72, 73, 76, 77, 81,
82, 85, 87, 89, 92, 93, 97, 98, 100, 100, 102, 102, 101, 69, 66, 66, 63,
63, 61, 61, 59, 60, 63, 63, 66, 66, 70, 70, 73, 74, 78, 78, 82, 82, 86,
87, 91, 91, 95, 96, 101, 101, 103, 103, 105,
// Size 4x16
31, 48, 57, 68, 32, 46, 53, 63, 36, 46, 51, 60, 40, 46, 50, 58, 44, 51,
54, 61, 46, 54, 60, 64, 45, 56, 64, 67, 47, 57, 68, 71, 49, 58, 73, 77,
52, 60, 76, 82, 54, 62, 79, 87, 58, 64, 82, 91, 60, 66, 84, 95, 62, 64,
84, 97, 64, 66, 81, 99, 65, 68, 83, 100,
// Size 16x4
31, 32, 36, 40, 44, 46, 45, 47, 49, 52, 54, 58, 60, 62, 64, 65, 48, 46,
46, 46, 51, 54, 56, 57, 58, 60, 62, 64, 66, 64, 66, 68, 57, 53, 51, 50,
54, 60, 64, 68, 73, 76, 79, 82, 84, 84, 81, 83, 68, 63, 60, 58, 61, 64,
67, 71, 77, 82, 87, 91, 95, 97, 99, 100,
// Size 8x32
32, 34, 48, 49, 54, 63, 67, 69, 31, 35, 47, 47, 51, 60, 63, 65, 31, 36,
46, 46, 50, 58, 62, 65, 30, 36, 46, 45, 49, 57, 60, 62, 33, 40, 47, 46,
49, 56, 59, 62, 35, 42, 47, 45, 48, 55, 58, 61, 37, 44, 47, 45, 48, 54,
57, 60, 42, 45, 50, 49, 51, 57, 59, 59, 44, 46, 51, 51, 53, 59, 60, 61,
49, 47, 53, 53, 55, 60, 63, 62, 48, 46, 53, 56, 58, 64, 64, 64, 48, 46,
53, 56, 59, 65, 66, 65, 49, 45, 53, 58, 62, 67, 70, 68, 50, 46, 54, 59,
65, 70, 70, 68, 51, 47, 54, 60, 65, 71, 73, 72, 52, 47, 54, 61, 68, 75,
76, 73, 54, 49, 55, 62, 70, 77, 77, 76, 54, 49, 55, 62, 70, 78, 81, 77,
57, 51, 56, 64, 73, 82, 83, 81, 59, 52, 58, 65, 74, 84, 85, 82, 60, 53,
58, 65, 75, 85, 89, 85, 63, 56, 60, 67, 77, 89, 90, 87, 64, 57, 61, 68,
78, 89, 93, 89, 64, 57, 61, 68, 78, 90, 94, 92, 66, 59, 63, 69, 79, 91,
94, 93, 67, 60, 63, 70, 78, 88, 96, 97, 68, 61, 63, 71, 79, 87, 96, 98,
69, 62, 63, 71, 80, 88, 96, 100, 70, 63, 63, 70, 80, 89, 97, 100, 71,
64, 63, 70, 78, 88, 97, 102, 72, 65, 63, 69, 77, 86, 95, 102, 73, 66,
63, 69, 76, 84, 93, 101,
// Size 32x8
32, 31, 31, 30, 33, 35, 37, 42, 44, 49, 48, 48, 49, 50, 51, 52, 54, 54,
57, 59, 60, 63, 64, 64, 66, 67, 68, 69, 70, 71, 72, 73, 34, 35, 36, 36,
40, 42, 44, 45, 46, 47, 46, 46, 45, 46, 47, 47, 49, 49, 51, 52, 53, 56,
57, 57, 59, 60, 61, 62, 63, 64, 65, 66, 48, 47, 46, 46, 47, 47, 47, 50,
51, 53, 53, 53, 53, 54, 54, 54, 55, 55, 56, 58, 58, 60, 61, 61, 63, 63,
63, 63, 63, 63, 63, 63, 49, 47, 46, 45, 46, 45, 45, 49, 51, 53, 56, 56,
58, 59, 60, 61, 62, 62, 64, 65, 65, 67, 68, 68, 69, 70, 71, 71, 70, 70,
69, 69, 54, 51, 50, 49, 49, 48, 48, 51, 53, 55, 58, 59, 62, 65, 65, 68,
70, 70, 73, 74, 75, 77, 78, 78, 79, 78, 79, 80, 80, 78, 77, 76, 63, 60,
58, 57, 56, 55, 54, 57, 59, 60, 64, 65, 67, 70, 71, 75, 77, 78, 82, 84,
85, 89, 89, 90, 91, 88, 87, 88, 89, 88, 86, 84, 67, 63, 62, 60, 59, 58,
57, 59, 60, 63, 64, 66, 70, 70, 73, 76, 77, 81, 83, 85, 89, 90, 93, 94,
94, 96, 96, 96, 97, 97, 95, 93, 69, 65, 65, 62, 62, 61, 60, 59, 61, 62,
64, 65, 68, 68, 72, 73, 76, 77, 81, 82, 85, 87, 89, 92, 93, 97, 98, 100,
100, 102, 102, 102, 101,
// Size 4x32
31, 48, 57, 68, 31, 47, 54, 64, 32, 46, 53, 63, 32, 45, 52, 61, 36, 46,
51, 60, 38, 46, 50, 60, 40, 46, 50, 58, 43, 50, 53, 58, 44, 51, 54, 61,
46, 53, 57, 62, 46, 54, 60, 64, 45, 55, 61, 66, 45, 56, 64, 67, 46, 56,
66, 71, 47, 57, 68, 71, 48, 57, 71, 75, 49, 58, 73, 77, 50, 59, 74, 79,
52, 60, 76, 82, 54, 61, 78, 83, 54, 62, 79, 87, 57, 64, 82, 88, 58, 64,
82, 91, 59, 65, 83, 93, 60, 66, 84, 95, 61, 65, 85, 97, 62, 64, 84, 97,
63, 65, 82, 99, 64, 66, 81, 99, 65, 67, 82, 99, 65, 68, 83, 100, 66, 69,
84, 101,
// Size 32x4
31, 31, 32, 32, 36, 38, 40, 43, 44, 46, 46, 45, 45, 46, 47, 48, 49, 50,
52, 54, 54, 57, 58, 59, 60, 61, 62, 63, 64, 65, 65, 66, 48, 47, 46, 45,
46, 46, 46, 50, 51, 53, 54, 55, 56, 56, 57, 57, 58, 59, 60, 61, 62, 64,
64, 65, 66, 65, 64, 65, 66, 67, 68, 69, 57, 54, 53, 52, 51, 50, 50, 53,
54, 57, 60, 61, 64, 66, 68, 71, 73, 74, 76, 78, 79, 82, 82, 83, 84, 85,
84, 82, 81, 82, 83, 84, 68, 64, 63, 61, 60, 60, 58, 58, 61, 62, 64, 66,
67, 71, 71, 75, 77, 79, 82, 83, 87, 88, 91, 93, 95, 97, 97, 99, 99, 99,
100, 101
},
{
{ // Luma
// Size 4x4
32, 37, 58, 81, 37, 54, 72, 91, 58, 72, 102, 121, 81, 91, 121, 156,
// Size 8x8

```

```

32, 32, 35, 42, 53, 68, 78, 90, 32, 33, 36, 42, 51, 64, 74, 84, 35, 36,
46, 52, 60, 72, 80, 87, 42, 42, 52, 63, 73, 84, 92, 98, 53, 51, 60, 73,
86, 100, 109, 114, 68, 64, 72, 84, 100, 117, 128, 133, 78, 74, 80, 92,
109, 128, 140, 155, 90, 84, 87, 98, 114, 133, 155, 168,
// Size 16x16
32, 31, 31, 32, 34, 36, 41, 47, 54, 59, 65, 74, 82, 87, 92, 97, 31, 32,
32, 32, 34, 35, 39, 45, 50, 55, 61, 69, 76, 81, 87, 92, 31, 32, 33, 33,
35, 36, 40, 44, 49, 54, 59, 67, 73, 78, 83, 88, 32, 32, 33, 35, 37, 38,
41, 45, 49, 53, 58, 65, 71, 75, 80, 86, 34, 34, 35, 37, 39, 42, 46, 50,
54, 58, 63, 70, 76, 80, 84, 85, 36, 35, 36, 38, 42, 48, 52, 56, 60, 64,
68, 75, 80, 85, 90, 91, 41, 39, 40, 41, 46, 52, 57, 62, 67, 71, 75, 83,
88, 92, 95, 97, 47, 45, 44, 45, 50, 56, 62, 69, 75, 79, 84, 91, 97, 100,
102, 104, 54, 50, 49, 49, 54, 60, 67, 75, 82, 87, 92, 100, 106, 110,
109, 112, 59, 55, 54, 53, 58, 64, 71, 79, 87, 92, 98, 106, 112, 117,
117, 121, 65, 61, 59, 58, 63, 68, 75, 84, 92, 98, 105, 114, 120, 125,
126, 130, 74, 69, 67, 65, 70, 75, 83, 91, 100, 106, 114, 123, 131, 135,
137, 140, 82, 76, 73, 71, 76, 80, 88, 97, 106, 112, 120, 131, 139, 144,
148, 150, 87, 81, 78, 75, 80, 85, 92, 100, 110, 117, 125, 135, 144, 150,
155, 162, 92, 87, 83, 80, 84, 90, 95, 102, 109, 117, 126, 137, 148, 155,
162, 168, 97, 92, 88, 86, 85, 91, 97, 104, 112, 121, 130, 140, 150, 162,
168, 174,
// Size 32x32
32, 31, 31, 31, 31, 31, 32, 32, 34, 35, 36, 39, 41, 44, 47, 48, 54, 56,
59, 64, 65, 71, 74, 80, 82, 83, 87, 90, 92, 95, 97, 100, 31, 32, 32, 32,
32, 32, 32, 33, 34, 35, 35, 38, 40, 42, 45, 46, 51, 53, 56, 61, 62, 68,
71, 76, 78, 83, 85, 88, 90, 92, 95, 31, 32, 32, 32, 32, 32, 33, 33,
34, 34, 35, 38, 39, 42, 45, 45, 50, 52, 55, 60, 61, 67, 69, 74, 76, 77,
81, 84, 87, 89, 92, 95, 31, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 37,
38, 41, 44, 44, 49, 51, 54, 58, 59, 65, 68, 72, 74, 75, 79, 81, 84, 86,
88, 90, 31, 32, 32, 32, 33, 33, 33, 34, 35, 36, 36, 39, 40, 42, 44, 45,
49, 51, 54, 58, 59, 64, 67, 71, 73, 74, 78, 80, 83, 85, 88, 90, 31, 32,
32, 32, 33, 33, 34, 34, 35, 36, 36, 39, 40, 42, 45, 45, 50, 51, 54, 58,
59, 64, 67, 71, 73, 74, 78, 80, 82, 84, 86, 89, 32, 32, 32, 32, 33, 34,
35, 36, 37, 38, 38, 40, 41, 42, 45, 46, 49, 51, 53, 57, 58, 63, 65, 69,
71, 72, 75, 78, 80, 83, 86, 89, 32, 33, 33, 33, 34, 34, 36, 36, 38, 39,
40, 42, 43, 44, 47, 47, 51, 53, 55, 59, 60, 65, 67, 71, 73, 73, 77, 78,
80, 82, 84, 86, 34, 34, 34, 33, 35, 35, 37, 38, 39, 42, 42, 45, 46, 47,
50, 51, 54, 56, 58, 62, 63, 68, 70, 74, 76, 76, 80, 82, 84, 85, 85, 86,
35, 35, 34, 34, 36, 36, 38, 39, 42, 46, 47, 49, 50, 52, 55, 55, 59, 60,
62, 66, 67, 72, 74, 78, 79, 80, 83, 84, 85, 87, 90, 92, 36, 35, 35, 34,
36, 36, 38, 40, 42, 47, 48, 50, 52, 54, 56, 57, 60, 61, 64, 67, 68, 73,
75, 79, 80, 81, 85, 87, 90, 91, 91, 92, 39, 38, 38, 37, 39, 39, 40, 42,
45, 49, 50, 54, 55, 58, 60, 61, 65, 66, 69, 72, 73, 78, 80, 84, 86, 86,
90, 91, 91, 92, 95, 97, 41, 40, 39, 38, 40, 40, 41, 43, 46, 50, 52, 55,
57, 60, 62, 63, 67, 69, 71, 75, 75, 80, 83, 86, 88, 89, 92, 93, 95, 97,
97, 98, 44, 42, 42, 41, 42, 42, 42, 44, 47, 52, 54, 58, 60, 63, 66, 67,
71, 73, 75, 79, 79, 84, 86, 90, 92, 92, 96, 98, 98, 98, 101, 104, 47,
45, 45, 44, 44, 45, 45, 47, 50, 55, 56, 60, 62, 66, 69, 70, 75, 77, 79,
83, 84, 89, 91, 95, 97, 97, 100, 99, 102, 105, 104, 104, 48, 46, 45, 44,
45, 45, 46, 47, 51, 55, 57, 61, 63, 67, 70, 71, 76, 78, 80, 84, 85, 90,
93, 96, 98, 99, 102, 106, 106, 105, 108, 111, 54, 51, 50, 49, 49, 50,
49, 51, 54, 59, 60, 65, 67, 71, 75, 76, 82, 84, 87, 91, 92, 97, 100,
104, 106, 106, 110, 108, 109, 112, 112, 111, 56, 53, 52, 51, 51, 51, 51,
53, 56, 60, 61, 66, 69, 73, 77, 78, 84, 86, 89, 93, 94, 100, 102, 106,
108, 109, 112, 113, 115, 114, 116, 119, 59, 56, 55, 54, 54, 54, 53, 55,
58, 62, 64, 69, 71, 75, 79, 80, 87, 89, 92, 97, 98, 103, 106, 110, 112,
113, 117, 118, 117, 121, 121, 119, 64, 61, 60, 58, 58, 58, 57, 59, 62,
66, 67, 72, 75, 79, 83, 84, 91, 93, 97, 102, 103, 109, 112, 116, 118,
119, 122, 121, 125, 123, 125, 128, 65, 62, 61, 59, 59, 59, 58, 60, 63,
67, 68, 73, 75, 79, 84, 85, 92, 94, 98, 103, 105, 111, 114, 118, 120,
121, 125, 129, 126, 129, 130, 129, 71, 68, 67, 65, 64, 64, 63, 65, 68,
72, 73, 78, 80, 84, 89, 90, 97, 100, 103, 109, 111, 117, 120, 125, 127,
128, 133, 130, 134, 133, 133, 137, 74, 71, 69, 68, 67, 67, 65, 67, 70,
74, 75, 80, 83, 86, 91, 93, 100, 102, 106, 112, 114, 120, 123, 128, 131,
131, 135, 137, 137, 138, 140, 137, 80, 76, 74, 72, 71, 71, 69, 71, 74,
78, 79, 84, 86, 90, 95, 96, 104, 106, 110, 116, 118, 125, 128, 134, 136,
137, 142, 141, 142, 143, 143, 147, 82, 78, 76, 74, 73, 73, 71, 73, 76,
79, 80, 86, 88, 92, 97, 98, 106, 108, 112, 118, 120, 127, 131, 136, 139,
139, 144, 147, 148, 147, 150, 148, 83, 78, 77, 75, 74, 74, 72, 73, 76,
80, 81, 86, 89, 92, 97, 99, 106, 109, 113, 119, 121, 128, 131, 137, 139,
140, 145, 150, 152, 155, 152, 157, 87, 83, 81, 79, 78, 78, 75, 77, 80,

```

```

83, 85, 90, 92, 96, 100, 102, 110, 112, 117, 122, 125, 133, 135, 142,
144, 145, 150, 151, 155, 158, 162, 158, 90, 85, 84, 81, 80, 80, 78, 78,
82, 84, 87, 91, 93, 98, 99, 106, 108, 113, 118, 121, 129, 130, 137, 141,
147, 150, 151, 156, 156, 161, 164, 169, 92, 88, 87, 84, 83, 82, 80, 80,
84, 85, 90, 91, 95, 98, 102, 106, 109, 115, 117, 125, 126, 134, 137,
142, 148, 152, 155, 156, 162, 162, 168, 170, 95, 90, 89, 86, 85, 84, 83,
82, 85, 87, 91, 92, 97, 98, 105, 105, 112, 114, 121, 123, 129, 133, 138,
143, 147, 155, 158, 161, 162, 168, 168, 174, 97, 92, 92, 88, 88, 86, 86,
84, 85, 90, 91, 95, 97, 101, 104, 108, 112, 116, 121, 125, 130, 133,
140, 143, 150, 152, 162, 164, 168, 168, 174, 175, 100, 95, 95, 90, 90,
89, 89, 86, 86, 92, 92, 97, 98, 104, 104, 111, 111, 119, 119, 128, 129,
137, 137, 147, 148, 157, 158, 169, 170, 174, 175, 181,
// Size 4x8
32, 35, 59, 83, 32, 36, 57, 78, 34, 47, 65, 82, 41, 53, 78, 97, 51, 61,
92, 111, 65, 73, 108, 129, 75, 81, 117, 148, 86, 92, 119, 154,
// Size 8x4
32, 32, 34, 41, 51, 65, 75, 86, 35, 36, 47, 53, 61, 73, 81, 92, 59, 57,
65, 78, 92, 108, 117, 119, 83, 78, 82, 97, 111, 129, 148, 154,
// Size 8x16
32, 31, 35, 44, 53, 65, 82, 90, 31, 32, 34, 41, 50, 61, 76, 85, 31, 33,
35, 42, 49, 59, 73, 81, 32, 34, 37, 42, 49, 58, 71, 79, 34, 35, 41, 48,
54, 63, 76, 81, 36, 36, 46, 54, 60, 68, 80, 87, 41, 40, 49, 60, 67, 76,
88, 93, 47, 44, 53, 66, 75, 84, 97, 101, 53, 50, 57, 71, 82, 92, 106,
108, 58, 54, 61, 75, 87, 98, 112, 116, 65, 59, 66, 79, 92, 105, 120,
124, 74, 67, 73, 86, 100, 113, 131, 134, 82, 73, 79, 92, 105, 120, 139,
142, 87, 78, 83, 96, 110, 125, 144, 153, 92, 83, 84, 97, 114, 132, 150,
157, 97, 88, 86, 97, 111, 128, 147, 163,
// Size 16x8
32, 31, 31, 32, 34, 36, 41, 47, 53, 58, 65, 74, 82, 87, 92, 97, 31, 32,
33, 34, 35, 36, 40, 44, 50, 54, 59, 67, 73, 78, 83, 88, 35, 34, 35, 37,
41, 46, 49, 53, 57, 61, 66, 73, 79, 83, 84, 86, 44, 41, 42, 42, 48, 54,
60, 66, 71, 75, 79, 86, 92, 96, 97, 97, 53, 50, 49, 49, 54, 60, 67, 75,
82, 87, 92, 100, 105, 110, 114, 111, 65, 61, 59, 58, 63, 68, 76, 84, 92,
98, 105, 113, 120, 125, 132, 128, 82, 76, 73, 71, 76, 80, 88, 97, 106,
112, 120, 131, 139, 144, 150, 147, 90, 85, 81, 79, 81, 87, 93, 101, 108,
116, 124, 134, 142, 153, 157, 163,
// Size 16x32
32, 31, 31, 32, 35, 36, 44, 47, 53, 62, 65, 79, 82, 88, 90, 93, 31, 32,
32, 32, 35, 35, 42, 45, 51, 59, 62, 75, 78, 83, 86, 88, 31, 32, 32, 32,
34, 35, 41, 45, 50, 58, 61, 74, 76, 82, 85, 88, 31, 32, 32, 33, 34, 34,
41, 44, 49, 57, 59, 72, 74, 79, 82, 84, 31, 32, 33, 34, 35, 36, 42, 44,
49, 57, 59, 71, 73, 79, 81, 84, 32, 32, 33, 34, 36, 36, 42, 45, 50, 57,
59, 71, 73, 78, 80, 82, 32, 33, 34, 35, 37, 38, 42, 45, 49, 56, 58, 69,
71, 76, 79, 83, 32, 33, 34, 36, 39, 40, 44, 47, 51, 58, 60, 71, 73, 76,
78, 80, 34, 34, 35, 37, 41, 42, 48, 50, 54, 61, 63, 73, 76, 81, 81, 80,
35, 34, 36, 38, 45, 47, 52, 55, 59, 65, 67, 77, 79, 82, 83, 86, 36, 34,
36, 38, 46, 48, 54, 56, 60, 66, 68, 78, 80, 85, 87, 86, 39, 37, 39, 40,
48, 50, 58, 60, 65, 71, 73, 84, 86, 89, 88, 91, 41, 39, 40, 41, 49, 51,
60, 62, 67, 74, 76, 86, 88, 91, 93, 91, 44, 41, 42, 43, 51, 53, 63, 66,
71, 78, 79, 90, 92, 97, 94, 97, 47, 44, 44, 45, 53, 56, 66, 69, 75, 82,
84, 95, 97, 98, 101, 98, 48, 45, 45, 46, 54, 56, 67, 70, 76, 83, 85, 96,
98, 104, 101, 105, 53, 49, 50, 50, 57, 60, 71, 75, 82, 90, 92, 103, 106,
107, 108, 105, 55, 51, 51, 51, 59, 61, 72, 77, 84, 92, 94, 106, 108,
111, 110, 112, 58, 54, 54, 54, 61, 63, 75, 79, 87, 95, 98, 110, 112,
117, 116, 113, 63, 58, 58, 57, 65, 67, 78, 83, 91, 100, 103, 116, 118,
119, 119, 121, 65, 60, 59, 58, 66, 68, 79, 84, 92, 102, 105, 118, 120,
127, 124, 122, 71, 65, 64, 63, 71, 73, 84, 89, 97, 108, 111, 125, 127,
129, 129, 130, 74, 68, 67, 66, 73, 75, 86, 91, 100, 110, 113, 128, 131,
135, 134, 130, 79, 72, 71, 70, 77, 79, 90, 95, 104, 115, 118, 133, 136,
140, 139, 140, 82, 75, 73, 72, 79, 81, 92, 97, 105, 117, 120, 136, 139,
145, 142, 140, 82, 75, 74, 72, 79, 81, 92, 97, 106, 117, 121, 136, 139,
148, 150, 149, 87, 79, 78, 76, 83, 85, 96, 100, 110, 120, 125, 141, 144,
148, 153, 150, 89, 82, 81, 78, 83, 87, 97, 99, 113, 118, 128, 139, 145,
153, 157, 161, 92, 84, 83, 80, 84, 89, 97, 101, 114, 116, 132, 135, 150,
153, 157, 162, 94, 86, 85, 82, 85, 92, 97, 104, 112, 119, 130, 136, 151,
154, 163, 166, 97, 88, 88, 85, 86, 94, 97, 107, 111, 123, 128, 140, 147,
159, 163, 167, 99, 91, 91, 87, 87, 97, 97, 110, 110, 126, 126, 144, 144,
163, 163, 173,
// Size 32x16
32, 31, 31, 31, 31, 32, 32, 32, 34, 35, 36, 39, 41, 44, 47, 48, 53, 55,
58, 63, 65, 71, 74, 79, 82, 82, 87, 89, 92, 94, 97, 99, 31, 32, 32, 32,

```

```

32, 32, 33, 33, 34, 34, 34, 37, 39, 41, 44, 45, 49, 51, 54, 58, 60, 65,
68, 72, 75, 75, 79, 82, 84, 86, 88, 91, 31, 32, 32, 32, 33, 33, 34, 34,
35, 36, 36, 39, 40, 42, 44, 45, 50, 51, 54, 58, 59, 64, 67, 71, 73, 74,
78, 81, 83, 85, 88, 91, 32, 32, 32, 33, 34, 34, 35, 36, 37, 38, 38, 40,
41, 43, 45, 46, 50, 51, 54, 57, 58, 63, 66, 70, 72, 72, 76, 78, 80, 82,
85, 87, 35, 35, 34, 34, 35, 36, 37, 39, 41, 45, 46, 48, 49, 51, 53, 54,
57, 59, 61, 65, 66, 71, 73, 77, 79, 79, 83, 83, 84, 85, 86, 87, 36, 35,
35, 34, 36, 36, 38, 40, 42, 47, 48, 50, 51, 53, 56, 56, 60, 61, 63, 67,
68, 73, 75, 79, 81, 81, 85, 87, 89, 92, 94, 97, 44, 42, 41, 41, 42, 42,
42, 44, 48, 52, 54, 58, 60, 63, 66, 67, 71, 72, 75, 78, 79, 84, 86, 90,
92, 92, 96, 97, 97, 97, 97, 97, 47, 45, 45, 44, 44, 45, 45, 47, 50, 55,
56, 60, 62, 66, 69, 70, 75, 77, 79, 83, 84, 89, 91, 95, 97, 97, 100, 99,
101, 104, 107, 110, 53, 51, 50, 49, 49, 50, 49, 51, 54, 59, 60, 65, 67,
71, 75, 76, 82, 84, 87, 91, 92, 97, 100, 104, 105, 106, 110, 113, 114,
112, 111, 110, 62, 59, 58, 57, 57, 57, 56, 58, 61, 65, 66, 71, 74, 78,
82, 83, 90, 92, 95, 100, 102, 108, 110, 115, 117, 117, 120, 118, 116,
119, 123, 126, 65, 62, 61, 59, 59, 59, 58, 60, 63, 67, 68, 73, 76, 79,
84, 85, 92, 94, 98, 103, 105, 111, 113, 118, 120, 121, 125, 128, 132,
130, 128, 126, 79, 75, 74, 72, 71, 71, 69, 71, 73, 77, 78, 84, 86, 90,
95, 96, 103, 106, 110, 116, 118, 125, 128, 133, 136, 136, 141, 139, 135,
136, 140, 144, 82, 78, 76, 74, 73, 73, 71, 73, 76, 79, 80, 86, 88, 92,
97, 98, 106, 108, 112, 118, 120, 127, 131, 136, 139, 139, 144, 145, 150,
151, 147, 144, 88, 83, 82, 79, 79, 78, 76, 76, 81, 82, 85, 89, 91, 97,
98, 104, 107, 111, 117, 119, 127, 129, 135, 140, 145, 148, 148, 153,
153, 154, 159, 163, 90, 86, 85, 82, 81, 80, 79, 78, 81, 83, 87, 88, 93,
94, 101, 101, 108, 110, 116, 119, 124, 129, 134, 139, 142, 150, 153,
157, 157, 163, 163, 163, 93, 88, 88, 84, 84, 82, 83, 80, 80, 86, 86, 91,
91, 97, 98, 105, 105, 112, 113, 121, 122, 130, 130, 140, 140, 149, 150,
161, 162, 166, 167, 173,
// Size 4x16
31, 36, 62, 88, 32, 35, 58, 82, 32, 36, 57, 79, 33, 38, 56, 76, 34, 42,
61, 81, 34, 48, 66, 85, 39, 51, 74, 91, 44, 56, 82, 98, 49, 60, 90, 107,
54, 63, 95, 117, 60, 68, 102, 127, 68, 75, 110, 135, 75, 81, 117, 145,
79, 85, 120, 148, 84, 89, 116, 153, 88, 94, 123, 159,
// Size 16x4
31, 32, 32, 33, 34, 34, 39, 44, 49, 54, 60, 68, 75, 79, 84, 88, 36, 35,
36, 38, 42, 48, 51, 56, 60, 63, 68, 75, 81, 85, 89, 94, 62, 58, 57, 56,
61, 66, 74, 82, 90, 95, 102, 110, 117, 120, 116, 123, 88, 82, 79, 76,
81, 85, 91, 98, 107, 117, 127, 135, 145, 148, 153, 159,
// Size 8x32
32, 31, 35, 44, 53, 65, 82, 90, 31, 32, 35, 42, 51, 62, 78, 86, 31, 32,
34, 41, 50, 61, 76, 85, 31, 32, 34, 41, 49, 59, 74, 82, 31, 33, 35, 42,
49, 59, 73, 81, 32, 33, 36, 42, 50, 59, 73, 80, 32, 34, 37, 42, 49, 58,
71, 79, 32, 34, 39, 44, 51, 60, 73, 78, 34, 35, 41, 48, 54, 63, 76, 81,
35, 36, 45, 52, 59, 67, 79, 83, 36, 36, 46, 54, 60, 68, 80, 87, 39, 39,
48, 58, 65, 73, 86, 88, 41, 40, 49, 60, 67, 76, 88, 93, 44, 42, 51, 63,
71, 79, 92, 94, 47, 44, 53, 66, 75, 84, 97, 101, 48, 45, 54, 67, 76, 85,
98, 101, 53, 50, 57, 71, 82, 92, 106, 108, 55, 51, 59, 72, 84, 94, 108,
110, 58, 54, 61, 75, 87, 98, 112, 116, 63, 58, 65, 78, 91, 103, 118,
119, 65, 59, 66, 79, 92, 105, 120, 124, 71, 64, 71, 84, 97, 111, 127,
129, 74, 67, 73, 86, 100, 113, 131, 134, 79, 71, 77, 90, 104, 118, 136,
139, 82, 73, 79, 92, 105, 120, 139, 142, 82, 74, 79, 92, 106, 121, 139,
150, 87, 78, 83, 96, 110, 125, 144, 153, 89, 81, 83, 97, 113, 128, 145,
157, 92, 83, 84, 97, 114, 132, 150, 157, 94, 85, 85, 97, 112, 130, 151,
163, 97, 88, 86, 97, 111, 128, 147, 163, 99, 91, 87, 97, 110, 126, 144,
163,
// Size 32x8
32, 31, 31, 31, 31, 32, 32, 32, 34, 35, 36, 39, 41, 44, 47, 48, 53, 55,
58, 63, 65, 71, 74, 79, 82, 82, 87, 89, 92, 94, 97, 99, 31, 32, 32, 32,
33, 33, 34, 34, 35, 36, 36, 39, 40, 42, 44, 45, 50, 51, 54, 58, 59, 64,
67, 71, 73, 74, 78, 81, 83, 85, 88, 91, 35, 35, 34, 34, 35, 36, 37, 39,
41, 45, 46, 48, 49, 51, 53, 54, 57, 59, 61, 65, 66, 71, 73, 77, 79, 79,
83, 83, 84, 85, 86, 87, 44, 42, 41, 41, 42, 42, 42, 44, 48, 52, 54, 58,
60, 63, 66, 67, 71, 72, 75, 78, 79, 84, 86, 90, 92, 92, 96, 97, 97, 97,
97, 97, 53, 51, 50, 49, 49, 50, 49, 51, 54, 59, 60, 65, 67, 71, 75, 76,
82, 84, 87, 91, 92, 97, 100, 104, 105, 106, 110, 113, 114, 112, 111,
110, 65, 62, 61, 59, 59, 59, 58, 60, 63, 67, 68, 73, 76, 79, 84, 85, 92,
94, 98, 103, 105, 111, 113, 118, 120, 121, 125, 128, 132, 130, 128, 126,
82, 78, 76, 74, 73, 73, 71, 73, 76, 79, 80, 86, 88, 92, 97, 98, 106,
108, 112, 118, 120, 127, 131, 136, 139, 139, 144, 145, 150, 151, 147,
144, 90, 86, 85, 82, 81, 80, 79, 78, 81, 83, 87, 88, 93, 94, 101, 101,

```

```

108, 110, 116, 119, 124, 129, 134, 139, 142, 150, 153, 157, 157, 163,
163, 163,
// Size 4x32
31, 36, 62, 88, 32, 35, 59, 83, 32, 35, 58, 82, 32, 34, 57, 79, 32, 36,
57, 79, 32, 36, 57, 78, 33, 38, 56, 76, 33, 40, 58, 76, 34, 42, 61, 81,
34, 47, 65, 82, 34, 48, 66, 85, 37, 50, 71, 89, 39, 51, 74, 91, 41, 53,
78, 97, 44, 56, 82, 98, 45, 56, 83, 104, 49, 60, 90, 107, 51, 61, 92,
111, 54, 63, 95, 117, 58, 67, 100, 119, 60, 68, 102, 127, 65, 73, 108,
129, 68, 75, 110, 135, 72, 79, 115, 140, 75, 81, 117, 145, 75, 81, 117,
148, 79, 85, 120, 148, 82, 87, 118, 153, 84, 89, 116, 153, 86, 92, 119,
154, 88, 94, 123, 159, 91, 97, 126, 163,
// Size 32x4
31, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34, 37, 39, 41, 44, 45, 49, 51,
54, 58, 60, 65, 68, 72, 75, 75, 79, 82, 84, 86, 88, 91, 36, 35, 35, 34,
36, 36, 38, 40, 42, 47, 48, 50, 51, 53, 56, 56, 60, 61, 63, 67, 68, 73,
75, 79, 81, 81, 85, 87, 89, 92, 94, 97, 62, 59, 58, 57, 57, 57, 56, 58,
61, 65, 66, 71, 74, 78, 82, 83, 90, 92, 95, 100, 102, 108, 110, 115,
117, 117, 120, 118, 116, 119, 123, 126, 88, 83, 82, 79, 79, 78, 76, 76,
81, 82, 85, 89, 91, 97, 98, 104, 107, 111, 117, 119, 127, 129, 135, 140,
145, 148, 148, 153, 153, 154, 159, 163,
},
{ // Chroma
// Size 4x4
32, 45, 51, 61, 45, 54, 59, 65, 51, 59, 75, 81, 61, 65, 81, 97,
// Size 8x8
31, 34, 46, 47, 50, 57, 61, 65, 34, 39, 47, 45, 48, 53, 57, 61, 46, 47,
52, 52, 54, 58, 61, 62, 47, 45, 52, 58, 62, 65, 68, 68, 50, 48, 54, 62,
68, 73, 77, 76, 57, 53, 58, 65, 73, 82, 86, 86, 61, 57, 61, 68, 77, 86,
91, 95, 65, 61, 62, 68, 76, 86, 95, 100,
// Size 16x16
32, 31, 33, 36, 41, 49, 49, 50, 52, 54, 57, 61, 64, 67, 68, 70, 31, 31,
34, 39, 42, 47, 46, 47, 49, 51, 53, 57, 60, 62, 64, 66, 33, 34, 37, 42,
44, 47, 46, 46, 47, 49, 51, 55, 57, 59, 61, 63, 36, 39, 42, 47, 47, 48,
46, 46, 47, 48, 50, 53, 55, 57, 59, 61, 41, 42, 44, 47, 48, 50, 49, 50,
50, 52, 53, 56, 58, 60, 61, 60, 49, 47, 47, 48, 50, 53, 53, 54, 54, 55,
56, 59, 61, 63, 64, 64, 49, 46, 46, 49, 53, 55, 57, 59, 60, 61, 64,
66, 67, 67, 67, 50, 47, 46, 46, 50, 54, 57, 61, 63, 64, 66, 69, 70, 72,
71, 71, 52, 49, 47, 47, 50, 54, 59, 63, 66, 68, 70, 73, 75, 77, 75, 75,
54, 51, 49, 48, 52, 55, 60, 64, 68, 71, 73, 76, 79, 80, 79, 79, 57, 53,
51, 50, 53, 56, 61, 66, 70, 73, 76, 80, 82, 84, 83, 84, 61, 57, 55, 53,
56, 59, 64, 69, 73, 76, 80, 84, 87, 89, 88, 88, 64, 60, 57, 55, 58, 61,
66, 70, 75, 79, 82, 87, 91, 93, 93, 93, 67, 62, 59, 57, 60, 63, 67, 72,
77, 80, 84, 89, 93, 95, 96, 97, 68, 64, 61, 59, 61, 64, 67, 71, 75, 79,
83, 88, 93, 96, 99, 100, 70, 66, 63, 61, 60, 64, 67, 71, 75, 79, 84, 88,
93, 97, 100, 102,
// Size 32x32
32, 31, 31, 30, 33, 33, 36, 38, 41, 47, 49, 48, 49, 49, 50, 50, 52, 53,
54, 56, 57, 60, 61, 63, 64, 65, 67, 67, 68, 69, 70, 71, 31, 31, 31, 31,
34, 34, 38, 40, 42, 46, 47, 47, 47, 47, 48, 48, 50, 50, 52, 54, 54, 57,
58, 60, 61, 61, 63, 64, 65, 65, 66, 67, 31, 31, 31, 31, 34, 35, 39, 40,
42, 46, 47, 46, 46, 46, 47, 47, 49, 50, 51, 53, 53, 56, 57, 59, 60, 60,
62, 63, 64, 65, 66, 67, 30, 31, 31, 32, 34, 35, 40, 41, 42, 45, 46, 45,
45, 45, 46, 46, 47, 48, 49, 51, 52, 54, 55, 57, 58, 58, 60, 61, 62, 62,
63, 64, 33, 34, 34, 34, 37, 38, 42, 43, 44, 46, 47, 46, 46, 45, 46, 46,
47, 48, 49, 51, 51, 53, 55, 56, 57, 57, 59, 60, 61, 62, 63, 64, 33, 34,
35, 35, 38, 39, 43, 44, 45, 47, 46, 46, 45, 46, 46, 47, 48, 49, 51,
51, 53, 54, 56, 57, 57, 59, 60, 60, 61, 62, 62, 36, 38, 39, 40, 42, 43,
47, 47, 47, 47, 48, 46, 46, 45, 46, 46, 47, 47, 48, 49, 50, 52, 53, 54,
55, 55, 57, 58, 59, 60, 61, 62, 38, 40, 40, 41, 43, 44, 47, 47, 48, 48,
49, 48, 47, 47, 47, 47, 48, 49, 49, 51, 51, 53, 54, 55, 56, 56, 58, 58,
58, 59, 60, 60, 41, 42, 42, 42, 44, 45, 47, 48, 48, 50, 50, 49, 49, 49,
50, 50, 50, 51, 52, 53, 53, 55, 56, 57, 58, 58, 60, 61, 61, 61, 60, 60,
47, 46, 46, 45, 46, 47, 47, 48, 50, 52, 52, 52, 52, 52, 53, 53, 53, 54,
55, 55, 56, 58, 58, 60, 60, 61, 62, 61, 61, 62, 63, 64, 49, 47, 47, 46,
47, 47, 48, 49, 50, 52, 53, 53, 53, 53, 54, 54, 54, 55, 55, 56, 56, 58,
59, 60, 61, 61, 63, 63, 64, 64, 64, 48, 47, 46, 45, 46, 46, 46, 48,
49, 52, 53, 54, 55, 55, 56, 56, 57, 58, 58, 59, 60, 61, 62, 63, 64, 64,
66, 65, 65, 65, 66, 67, 49, 47, 46, 45, 46, 46, 46, 46, 47, 49, 52, 53, 55,
55, 57, 57, 58, 59, 59, 60, 61, 61, 63, 64, 65, 66, 66, 67, 67, 67, 68,
67, 67, 49, 47, 46, 45, 45, 45, 45, 47, 49, 52, 53, 55, 57, 58, 59, 60,
61, 62, 62, 63, 63, 65, 66, 67, 68, 68, 69, 70, 69, 68, 69, 70, 50, 48,

```

47, 46, 46, 46, 46, 47, 50, 53, 54, 56, 57, 59, 61, 61, 63, 64, 64, 66,
 66, 68, 69, 70, 70, 71, 72, 70, 71, 72, 71, 70, 50, 48, 47, 46, 46, 46,
 46, 47, 50, 53, 54, 56, 58, 60, 61, 61, 63, 64, 65, 66, 67, 68, 69, 71,
 71, 71, 73, 74, 73, 72, 73, 74, 52, 50, 49, 47, 47, 47, 47, 48, 50, 53,
 54, 57, 59, 61, 63, 63, 66, 67, 68, 70, 70, 72, 73, 75, 75, 75, 77, 75,
 75, 76, 75, 74, 53, 50, 50, 48, 48, 48, 47, 49, 51, 54, 55, 58, 59, 62,
 64, 64, 67, 68, 69, 71, 71, 73, 74, 76, 77, 77, 78, 78, 78, 76, 77, 78,
 54, 52, 51, 49, 49, 49, 48, 49, 52, 55, 55, 58, 60, 62, 64, 65, 68, 69,
 71, 73, 73, 75, 76, 78, 79, 79, 80, 80, 79, 80, 79, 78, 56, 54, 53, 51,
 51, 51, 49, 51, 53, 55, 56, 59, 61, 63, 66, 66, 70, 71, 73, 75, 76, 78,
 79, 81, 82, 82, 83, 81, 83, 81, 81, 82, 57, 54, 53, 52, 51, 51, 50, 51,
 53, 56, 56, 60, 61, 63, 66, 67, 70, 71, 73, 76, 76, 79, 80, 82, 82, 83,
 84, 85, 83, 84, 84, 82, 60, 57, 56, 54, 53, 53, 52, 53, 55, 58, 58, 61,
 63, 65, 68, 68, 72, 73, 75, 78, 79, 82, 83, 85, 86, 86, 88, 86, 87, 86,
 85, 86, 61, 58, 57, 55, 55, 54, 53, 54, 56, 58, 59, 62, 64, 66, 69, 69,
 73, 74, 76, 79, 80, 83, 84, 86, 87, 88, 89, 89, 88, 88, 88, 86, 63, 60,
 59, 57, 56, 56, 54, 55, 57, 60, 60, 63, 65, 67, 70, 71, 75, 76, 78, 81,
 82, 85, 86, 89, 90, 90, 92, 91, 91, 90, 89, 91, 64, 61, 60, 58, 57, 57,
 55, 56, 58, 60, 61, 64, 66, 68, 70, 71, 75, 77, 79, 82, 82, 86, 87, 90,
 91, 91, 93, 93, 93, 92, 93, 91, 65, 61, 60, 58, 57, 57, 55, 56, 58, 61,
 61, 64, 66, 68, 71, 71, 75, 77, 79, 82, 83, 86, 88, 90, 91, 91, 93, 94,
 95, 95, 93, 95, 67, 63, 62, 60, 59, 59, 57, 58, 60, 62, 63, 66, 67, 69,
 72, 73, 77, 78, 80, 83, 84, 88, 89, 92, 93, 93, 95, 95, 96, 96, 97, 95,
 67, 64, 63, 61, 60, 60, 58, 58, 61, 61, 63, 65, 67, 70, 70, 74, 75, 78,
 80, 81, 85, 86, 89, 91, 93, 94, 95, 97, 97, 98, 98, 100, 68, 65, 64, 62,
 61, 60, 59, 58, 61, 61, 64, 65, 67, 69, 71, 73, 75, 78, 79, 83, 83, 87,
 88, 91, 93, 95, 96, 97, 99, 98, 100, 100, 69, 65, 65, 62, 62, 61, 60,
 59, 61, 62, 64, 65, 68, 68, 72, 72, 76, 76, 80, 81, 84, 86, 88, 90, 92,
 95, 96, 98, 98, 100, 100, 101, 70, 66, 66, 63, 63, 62, 61, 60, 60, 63,
 64, 66, 67, 69, 71, 73, 75, 77, 79, 81, 84, 85, 88, 89, 93, 93, 97, 98,
 100, 100, 102, 101, 71, 67, 67, 64, 64, 62, 62, 60, 60, 64, 64, 67, 67,
 70, 70, 74, 74, 78, 78, 82, 82, 86, 86, 91, 91, 95, 95, 100, 100, 101,
 101, 104,
 // Size 4x8
 31, 47, 53, 63, 36, 47, 50, 59, 46, 52, 55, 61, 45, 53, 63, 70, 49, 55,
 71, 77, 54, 58, 77, 86, 59, 61, 81, 94, 63, 65, 80, 95,
 // Size 8x4
 31, 36, 46, 45, 49, 54, 59, 63, 47, 47, 52, 53, 55, 58, 61, 65, 53, 50,
 55, 63, 71, 77, 81, 80, 63, 59, 61, 70, 77, 86, 94, 95,
 // Size 8x16
 32, 33, 45, 49, 52, 57, 64, 68, 31, 34, 45, 46, 49, 53, 60, 64, 33, 37,
 46, 45, 47, 51, 57, 61, 37, 43, 47, 45, 47, 50, 55, 59, 42, 44, 49, 49,
 50, 53, 58, 60, 49, 47, 52, 53, 54, 57, 61, 63, 48, 46, 51, 57, 59, 61,
 66, 67, 50, 46, 52, 59, 63, 66, 71, 71, 52, 47, 53, 61, 66, 71, 75, 74,
 54, 49, 54, 62, 68, 73, 79, 79, 57, 51, 55, 64, 70, 76, 83, 83, 61, 55,
 58, 66, 73, 80, 87, 87, 64, 57, 60, 68, 75, 83, 91, 91, 66, 59, 61, 69,
 77, 84, 93, 95, 68, 61, 61, 68, 77, 86, 94, 97, 70, 63, 61, 67, 75, 83,
 92, 98,
 // Size 16x8
 32, 31, 33, 37, 42, 49, 48, 50, 52, 54, 57, 61, 64, 66, 68, 70, 33, 34,
 37, 43, 44, 47, 46, 46, 47, 49, 51, 55, 57, 59, 61, 63, 45, 45, 46, 47,
 49, 52, 51, 52, 53, 54, 55, 58, 60, 61, 61, 61, 49, 46, 45, 45, 49, 53,
 57, 59, 61, 62, 64, 66, 68, 69, 68, 67, 52, 49, 47, 47, 50, 54, 59, 63,
 66, 68, 70, 73, 75, 77, 77, 75, 57, 53, 51, 50, 53, 57, 61, 66, 71, 73,
 76, 80, 83, 84, 86, 83, 64, 60, 57, 55, 58, 61, 66, 71, 75, 79, 83, 87,
 91, 93, 94, 92, 68, 64, 61, 59, 60, 63, 67, 71, 74, 79, 83, 87, 91, 95,
 97, 98,
 // Size 16x32
 32, 31, 33, 37, 45, 48, 49, 50, 52, 56, 57, 63, 64, 67, 68, 68, 31, 31,
 34, 38, 45, 47, 47, 48, 50, 53, 54, 60, 61, 63, 64, 65, 31, 32, 34, 39,
 45, 46, 46, 47, 49, 52, 53, 59, 60, 62, 64, 65, 30, 32, 35, 40, 44, 46,
 45, 46, 48, 51, 52, 57, 58, 60, 61, 62, 33, 35, 37, 42, 46, 47, 45, 46,
 47, 50, 51, 56, 57, 60, 61, 62, 33, 36, 38, 43, 46, 47, 46, 46, 47, 50,
 51, 56, 57, 59, 60, 60, 37, 40, 43, 47, 47, 47, 45, 46, 47, 49, 50, 54,
 55, 57, 59, 61, 39, 41, 43, 47, 48, 48, 47, 47, 48, 50, 51, 55, 56, 57,
 58, 59, 42, 43, 44, 47, 49, 50, 49, 50, 50, 53, 53, 57, 58, 60, 60, 59,
 47, 46, 46, 48, 51, 52, 53, 53, 53, 55, 56, 60, 61, 61, 61, 62, 49, 46,
 47, 48, 52, 53, 53, 54, 54, 56, 57, 60, 61, 63, 63, 62, 48, 46, 46, 47,
 51, 53, 56, 56, 57, 59, 60, 64, 64, 65, 64, 65, 48, 45, 46, 46, 51, 53,
 57, 57, 59, 61, 61, 65, 66, 66, 67, 65, 49, 45, 45, 46, 51, 53, 58, 59,
 61, 63, 64, 67, 68, 70, 67, 68, 50, 46, 46, 46, 52, 54, 59, 61, 63, 65,


```

66, 70, 71, 70, 71, 68, 50, 46, 46, 46, 52, 54, 59, 61, 64, 66, 67, 71,
71, 73, 71, 72, 52, 48, 47, 47, 53, 54, 61, 63, 66, 70, 71, 75, 75, 75,
74, 72, 53, 49, 48, 48, 53, 55, 61, 64, 67, 71, 72, 76, 77, 77, 75, 76,
54, 50, 49, 49, 54, 55, 62, 65, 68, 72, 73, 78, 79, 80, 79, 76, 56, 51,
51, 50, 55, 56, 63, 66, 70, 74, 76, 81, 82, 81, 80, 80, 57, 52, 51, 50,
55, 56, 64, 66, 70, 75, 76, 82, 83, 85, 83, 80, 60, 54, 54, 52, 57, 58,
65, 68, 72, 77, 79, 85, 86, 86, 85, 84, 61, 56, 55, 53, 58, 59, 66, 69,
73, 79, 80, 86, 87, 89, 87, 84, 63, 57, 56, 55, 59, 60, 67, 70, 75, 80,
82, 89, 90, 91, 89, 89, 64, 58, 57, 56, 60, 61, 68, 71, 75, 81, 83, 90,
91, 93, 91, 89, 64, 59, 58, 56, 60, 61, 68, 71, 75, 81, 83, 90, 91, 94,
94, 93, 66, 60, 59, 57, 61, 63, 69, 72, 77, 82, 84, 92, 93, 94, 95, 93,
67, 61, 60, 58, 61, 63, 69, 70, 78, 80, 85, 90, 93, 96, 97, 97, 68, 62,
61, 59, 61, 64, 68, 71, 77, 79, 86, 88, 94, 96, 97, 98, 69, 63, 62, 59,
61, 65, 68, 72, 76, 80, 85, 88, 94, 95, 99, 99, 70, 63, 63, 60, 61, 66,
67, 73, 75, 81, 83, 89, 92, 97, 98, 99, 70, 64, 64, 61, 61, 67, 67, 74,
74, 82, 82, 90, 98, 98, 102,
// Size 32x16
32, 31, 31, 30, 33, 33, 37, 39, 42, 47, 49, 48, 48, 49, 50, 50, 52, 53,
54, 56, 57, 60, 61, 63, 64, 64, 66, 67, 68, 69, 70, 70, 31, 31, 32, 32,
35, 36, 40, 41, 43, 46, 46, 46, 45, 45, 46, 46, 48, 49, 50, 51, 52, 54,
56, 57, 58, 59, 60, 61, 62, 63, 64, 33, 34, 34, 35, 37, 38, 43, 43,
44, 46, 47, 46, 46, 45, 46, 46, 47, 48, 49, 51, 51, 54, 55, 56, 57, 58,
59, 60, 61, 62, 63, 64, 37, 38, 39, 40, 42, 43, 47, 47, 47, 48, 48, 47,
46, 46, 46, 46, 47, 48, 49, 50, 50, 52, 53, 55, 56, 56, 57, 58, 59, 59,
60, 61, 45, 45, 45, 44, 46, 46, 47, 48, 49, 51, 52, 51, 51, 51, 52, 52,
53, 53, 54, 55, 55, 57, 58, 59, 60, 61, 61, 61, 61, 61, 48, 47,
46, 46, 47, 47, 47, 48, 50, 52, 53, 53, 53, 53, 54, 54, 54, 55, 55, 56,
56, 58, 59, 60, 61, 61, 63, 63, 64, 65, 66, 67, 49, 47, 46, 45, 45, 46,
45, 47, 49, 53, 53, 56, 57, 58, 59, 59, 61, 61, 62, 63, 64, 65, 66, 67,
68, 68, 69, 69, 68, 68, 67, 67, 50, 48, 47, 46, 46, 46, 46, 47, 50, 53,
54, 56, 57, 59, 61, 61, 63, 64, 65, 66, 66, 68, 69, 70, 71, 71, 72, 70,
71, 72, 73, 74, 52, 50, 49, 48, 47, 47, 47, 48, 50, 53, 54, 57, 59, 61,
63, 64, 66, 67, 68, 70, 70, 72, 73, 75, 75, 75, 77, 78, 77, 76, 75, 74,
56, 53, 52, 51, 50, 50, 49, 50, 53, 55, 56, 59, 61, 63, 65, 66, 70, 71,
72, 74, 75, 77, 79, 80, 81, 81, 82, 80, 79, 80, 81, 82, 57, 54, 53, 52,
51, 51, 50, 51, 53, 56, 57, 60, 61, 64, 66, 67, 71, 72, 73, 76, 76, 79,
80, 82, 83, 83, 84, 85, 86, 85, 83, 82, 63, 60, 59, 57, 56, 56, 54, 55,
57, 60, 60, 64, 65, 67, 70, 71, 75, 76, 78, 81, 82, 85, 86, 89, 90, 90,
92, 90, 88, 88, 89, 90, 64, 61, 60, 58, 57, 57, 55, 56, 58, 61, 61, 64,
66, 68, 71, 71, 75, 77, 79, 82, 83, 86, 87, 90, 91, 91, 93, 93, 94, 94,
92, 90, 67, 63, 62, 60, 60, 59, 57, 57, 60, 61, 63, 65, 66, 70, 70, 73,
75, 77, 80, 81, 85, 86, 89, 91, 93, 94, 94, 96, 96, 95, 97, 98, 68, 64,
64, 61, 61, 60, 59, 58, 60, 61, 63, 64, 67, 67, 71, 71, 74, 75, 79, 80,
83, 85, 87, 89, 91, 94, 95, 97, 97, 99, 98, 98, 68, 65, 65, 62, 62, 60,
61, 59, 59, 62, 62, 65, 65, 68, 68, 72, 72, 76, 76, 80, 80, 84, 84, 89,
89, 93, 93, 97, 98, 99, 99, 102,
// Size 4x16
31, 48, 56, 67, 32, 46, 52, 62, 35, 47, 50, 60, 40, 47, 49, 57, 43, 50,
53, 60, 46, 53, 56, 63, 45, 53, 61, 66, 46, 54, 65, 70, 48, 54, 70, 75,
50, 55, 72, 80, 52, 56, 75, 85, 56, 59, 79, 89, 58, 61, 81, 93, 60, 63,
82, 94, 62, 64, 79, 96, 63, 66, 81, 97,
// Size 16x4
31, 32, 35, 40, 43, 46, 45, 46, 48, 50, 52, 56, 58, 60, 62, 63, 48, 46,
47, 47, 50, 53, 53, 54, 54, 55, 56, 59, 61, 63, 64, 66, 56, 52, 50, 49,
53, 56, 61, 65, 70, 72, 75, 79, 81, 82, 79, 81, 67, 62, 60, 57, 60, 63,
66, 70, 75, 80, 85, 89, 93, 94, 96, 97,
// Size 8x32
32, 33, 45, 49, 52, 57, 64, 68, 31, 34, 45, 47, 50, 54, 61, 64, 31, 34,
45, 46, 49, 53, 60, 64, 30, 35, 44, 45, 48, 52, 58, 61, 33, 37, 46, 45,
47, 51, 57, 61, 33, 38, 46, 46, 47, 51, 57, 60, 37, 43, 47, 45, 47, 50,
55, 59, 39, 43, 48, 47, 48, 51, 56, 58, 42, 44, 49, 49, 50, 53, 58, 60,
47, 46, 51, 53, 53, 56, 61, 61, 49, 47, 52, 53, 54, 57, 61, 63, 48, 46,
51, 56, 57, 60, 64, 64, 48, 46, 51, 57, 59, 61, 66, 67, 49, 45, 51, 58,
61, 64, 68, 67, 50, 46, 52, 59, 63, 66, 71, 71, 50, 46, 52, 59, 64, 67,
71, 71, 52, 47, 53, 61, 66, 71, 75, 74, 53, 48, 53, 61, 67, 72, 77, 75,
54, 49, 54, 62, 68, 73, 79, 79, 56, 51, 55, 63, 70, 76, 82, 80, 57, 51,
55, 64, 70, 76, 83, 83, 60, 54, 57, 65, 72, 79, 86, 85, 61, 55, 58, 66,
73, 80, 87, 87, 63, 56, 59, 67, 75, 82, 90, 89, 64, 57, 60, 68, 75, 83,
91, 91, 64, 58, 60, 68, 75, 83, 91, 94, 66, 59, 61, 69, 77, 84, 93, 95,
67, 60, 61, 69, 78, 85, 93, 97, 68, 61, 61, 68, 77, 86, 94, 97, 69, 62,
61, 68, 76, 85, 94, 99, 70, 63, 61, 67, 75, 83, 92, 98, 70, 64, 61, 67,

```

```

74, 82, 90, 98,
// Size 32x8
32, 31, 31, 30, 33, 33, 37, 39, 42, 47, 49, 48, 48, 49, 50, 50, 52, 53,
54, 56, 57, 60, 61, 63, 64, 64, 66, 67, 68, 69, 70, 70, 33, 34, 34, 35,
37, 38, 43, 43, 44, 46, 47, 46, 46, 45, 46, 46, 47, 48, 49, 51, 51, 54,
55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 45, 45, 45, 44, 46, 46, 47, 48,
49, 51, 52, 51, 51, 51, 52, 52, 53, 53, 54, 55, 55, 57, 58, 59, 60, 60,
61, 61, 61, 61, 61, 61, 49, 47, 46, 45, 45, 46, 45, 47, 49, 53, 53, 56,
57, 58, 59, 59, 61, 61, 62, 63, 64, 65, 66, 67, 68, 68, 69, 69, 68, 68,
67, 67, 52, 50, 49, 48, 47, 47, 47, 48, 50, 53, 54, 57, 59, 61, 63, 64,
66, 67, 68, 70, 70, 72, 73, 75, 75, 75, 77, 78, 77, 76, 75, 74, 57, 54,
53, 52, 51, 51, 50, 51, 53, 56, 57, 60, 61, 64, 66, 67, 71, 72, 73, 76,
76, 79, 80, 82, 83, 83, 84, 85, 86, 85, 83, 82, 64, 61, 60, 58, 57, 57,
55, 56, 58, 61, 61, 64, 66, 68, 71, 71, 75, 77, 79, 82, 83, 86, 87, 90,
91, 91, 93, 93, 94, 94, 92, 90, 68, 64, 64, 61, 61, 60, 59, 58, 60, 61,
63, 64, 67, 67, 71, 71, 74, 75, 79, 80, 83, 85, 87, 89, 91, 94, 95, 97,
97, 99, 98, 98,
// Size 4x32
31, 48, 56, 67, 31, 47, 53, 63, 32, 46, 52, 62, 32, 46, 51, 60, 35, 47,
50, 60, 36, 47, 50, 59, 40, 47, 49, 57, 41, 48, 50, 57, 43, 50, 53, 60,
46, 52, 55, 61, 46, 53, 56, 63, 46, 53, 59, 65, 45, 53, 61, 66, 45, 53,
63, 70, 46, 54, 65, 70, 46, 54, 66, 73, 48, 54, 70, 75, 49, 55, 71, 77,
50, 55, 72, 80, 51, 56, 74, 81, 52, 56, 75, 85, 54, 58, 77, 86, 56, 59,
79, 89, 57, 60, 80, 91, 58, 61, 81, 93, 59, 61, 81, 94, 60, 63, 82, 94,
61, 63, 80, 96, 62, 64, 79, 96, 63, 65, 80, 95, 63, 66, 81, 97, 64, 67,
82, 98,
// Size 32x4
31, 31, 32, 32, 35, 36, 40, 41, 43, 46, 46, 46, 45, 45, 46, 46, 48, 49,
50, 51, 52, 54, 56, 57, 58, 59, 60, 61, 62, 63, 63, 64, 48, 47, 46, 46,
47, 47, 47, 48, 50, 52, 53, 53, 53, 54, 54, 54, 55, 55, 56, 56, 58,
59, 60, 61, 61, 63, 63, 64, 65, 66, 67, 56, 53, 52, 51, 50, 50, 49, 50,
53, 55, 56, 59, 61, 63, 65, 66, 70, 71, 72, 74, 75, 77, 79, 80, 81, 81,
82, 80, 79, 80, 81, 82, 67, 63, 62, 60, 60, 59, 57, 57, 60, 61, 63, 65,
66, 70, 70, 73, 75, 77, 80, 81, 85, 86, 89, 91, 93, 94, 94, 96, 96, 95,
97, 98,
},
{
{ // Luma
// Size 4x4
32, 34, 53, 75, 34, 49, 64, 81, 53, 64, 91, 112, 75, 81, 112, 140,
// Size 8x8
32, 32, 34, 39, 50, 62, 76, 84, 32, 33, 35, 40, 48, 59, 71, 79, 34, 35,
39, 46, 53, 63, 74, 81, 39, 40, 46, 56, 65, 75, 86, 92, 50, 48, 53, 65,
78, 90, 101, 106, 62, 59, 63, 75, 90, 105, 118, 123, 76, 71, 74, 86,
101, 118, 134, 142, 84, 79, 81, 92, 106, 123, 142, 153,
// Size 16x16
32, 31, 31, 32, 33, 36, 39, 44, 48, 54, 59, 66, 74, 81, 86, 91, 31, 32,
32, 32, 33, 35, 38, 42, 46, 51, 56, 63, 70, 77, 81, 86, 31, 32, 32, 33,
34, 35, 38, 41, 45, 49, 54, 60, 67, 73, 77, 82, 32, 32, 33, 34, 36, 37,
40, 42, 45, 49, 53, 59, 66, 71, 75, 80, 33, 33, 34, 36, 38, 42, 44, 46,
50, 53, 57, 63, 69, 74, 78, 80, 36, 35, 35, 37, 42, 48, 50, 54, 57, 60,
64, 69, 75, 80, 84, 85, 39, 38, 38, 40, 44, 50, 54, 58, 61, 65, 69, 74,
80, 85, 89, 91, 44, 42, 41, 42, 46, 54, 58, 63, 67, 71, 75, 80, 86, 91,
95, 97, 48, 46, 45, 45, 50, 57, 61, 67, 71, 76, 80, 86, 93, 98, 101,
104, 54, 51, 49, 49, 53, 60, 65, 71, 76, 82, 87, 93, 100, 105, 109, 112,
59, 56, 54, 53, 57, 64, 69, 75, 80, 87, 92, 99, 106, 112, 116, 120, 66,
63, 60, 59, 63, 69, 74, 80, 86, 93, 99, 107, 115, 121, 125, 129, 74, 70,
67, 66, 69, 75, 80, 86, 93, 100, 106, 115, 123, 130, 135, 138, 81, 77,
73, 71, 74, 80, 85, 91, 98, 105, 112, 121, 130, 137, 142, 148, 86, 81,
77, 75, 78, 84, 89, 95, 101, 109, 116, 125, 135, 142, 147, 153, 91, 86,
82, 80, 80, 85, 91, 97, 104, 112, 120, 129, 138, 148, 153, 159,
// Size 32x32
32, 31, 31, 31, 31, 31, 32, 32, 33, 34, 36, 36, 39, 41, 44, 46, 48, 52,
54, 58, 59, 65, 66, 71, 74, 80, 81, 83, 86, 89, 91, 93, 31, 32, 32, 32,
32, 32, 32, 32, 33, 34, 35, 35, 38, 39, 42, 44, 46, 50, 51, 56, 56, 62,
63, 68, 71, 76, 77, 78, 82, 84, 86, 88, 31, 32, 32, 32, 32, 32, 32, 32,
33, 34, 35, 35, 38, 39, 42, 44, 46, 49, 51, 55, 56, 61, 63, 67, 70, 75,
77, 78, 81, 84, 86, 88, 31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34,
37, 38, 41, 42, 44, 48, 49, 53, 54, 59, 60, 65, 68, 72, 74, 75, 78, 80,
82, 84, 31, 32, 32, 32, 32, 33, 33, 33, 34, 34, 35, 35, 38, 39, 41, 43,

```

45, 48, 49, 53, 54, 59, 60, 65, 67, 72, 73, 74, 77, 80, 82, 84, 31, 32,
 32, 32, 33, 33, 33, 34, 35, 35, 36, 36, 39, 40, 42, 44, 45, 48, 50, 53,
 54, 59, 60, 64, 67, 71, 73, 74, 77, 79, 81, 83, 32, 32, 32, 32, 33, 33,
 34, 35, 36, 36, 37, 38, 40, 40, 42, 44, 45, 48, 49, 53, 53, 58, 59, 63,
 66, 70, 71, 72, 75, 78, 80, 83, 32, 32, 32, 32, 33, 34, 35, 35, 36, 37,
 38, 38, 40, 41, 42, 44, 46, 48, 49, 53, 53, 58, 59, 63, 65, 69, 71, 72,
 74, 77, 79, 80, 33, 33, 33, 33, 34, 35, 36, 36, 38, 39, 42, 42, 44, 45,
 46, 48, 50, 52, 53, 57, 57, 62, 63, 67, 69, 73, 74, 75, 78, 79, 80, 81,
 34, 34, 34, 33, 34, 35, 36, 37, 39, 39, 42, 43, 45, 46, 47, 49, 51, 53,
 54, 58, 58, 63, 64, 68, 70, 74, 75, 76, 79, 81, 84, 86, 36, 35, 35, 34,
 35, 36, 37, 38, 42, 42, 48, 48, 50, 51, 54, 55, 57, 59, 60, 63, 64, 68,
 69, 73, 75, 79, 80, 81, 84, 85, 85, 86, 36, 35, 35, 34, 35, 36, 38, 38,
 42, 43, 48, 49, 51, 52, 54, 55, 57, 59, 60, 64, 64, 68, 69, 73, 75, 79,
 80, 81, 84, 86, 88, 91, 39, 38, 38, 37, 38, 39, 40, 40, 44, 45, 50, 51,
 54, 55, 58, 59, 61, 64, 65, 68, 69, 73, 74, 78, 80, 84, 85, 86, 89, 91,
 91, 91, 41, 39, 39, 38, 39, 40, 40, 41, 45, 46, 51, 52, 55, 56, 59, 61,
 63, 65, 67, 70, 70, 75, 76, 80, 82, 86, 87, 88, 91, 92, 94, 96, 44, 42,
 42, 41, 41, 42, 42, 42, 46, 47, 54, 54, 58, 59, 63, 65, 67, 70, 71, 75,
 75, 79, 80, 84, 86, 90, 91, 92, 95, 97, 97, 97, 46, 44, 44, 42, 43, 44,
 44, 44, 48, 49, 55, 55, 59, 61, 65, 67, 69, 72, 74, 77, 78, 82, 83, 87,
 89, 93, 94, 95, 98, 98, 100, 103, 48, 46, 46, 44, 45, 45, 45, 46, 50,
 51, 57, 57, 61, 63, 67, 69, 71, 74, 76, 80, 80, 85, 86, 90, 93, 96, 98,
 99, 101, 104, 104, 103, 52, 50, 49, 48, 48, 48, 48, 48, 52, 53, 59, 59,
 64, 65, 70, 72, 74, 78, 80, 84, 85, 90, 91, 95, 97, 101, 103, 104, 106,
 106, 107, 110, 54, 51, 51, 49, 49, 50, 49, 49, 53, 54, 60, 60, 65, 67,
 71, 74, 76, 80, 82, 86, 87, 92, 93, 97, 100, 104, 105, 106, 109, 112,
 112, 110, 58, 56, 55, 53, 53, 53, 53, 57, 58, 63, 64, 68, 70, 75,
 77, 80, 84, 86, 91, 91, 97, 98, 103, 105, 110, 111, 112, 115, 114, 115,
 118, 59, 56, 56, 54, 54, 54, 53, 53, 57, 58, 64, 64, 69, 70, 75, 78, 80,
 85, 87, 91, 92, 98, 99, 103, 106, 110, 112, 113, 116, 119, 120, 119, 65,
 62, 61, 59, 59, 58, 58, 62, 63, 68, 68, 73, 75, 79, 82, 85, 90, 92,
 97, 98, 105, 106, 111, 114, 118, 120, 121, 124, 123, 123, 126, 66, 63,
 63, 60, 60, 60, 59, 59, 63, 64, 69, 69, 74, 76, 80, 83, 86, 91, 93, 98,
 99, 106, 107, 112, 115, 119, 121, 122, 125, 128, 129, 126, 71, 68, 67,
 65, 65, 64, 63, 63, 67, 68, 73, 73, 78, 80, 84, 87, 90, 95, 97, 103,
 103, 111, 112, 117, 120, 125, 127, 128, 131, 132, 132, 135, 74, 71, 70,
 68, 67, 67, 66, 65, 69, 70, 75, 75, 80, 82, 86, 89, 93, 97, 100, 105,
 106, 114, 115, 120, 123, 128, 130, 131, 135, 135, 138, 136, 80, 76, 75,
 72, 72, 71, 70, 69, 73, 74, 79, 79, 84, 86, 90, 93, 96, 101, 104, 110,
 110, 118, 119, 125, 128, 134, 136, 137, 140, 142, 140, 144, 81, 77, 77,
 74, 73, 73, 71, 71, 74, 75, 80, 80, 85, 87, 91, 94, 98, 103, 105, 111,
 112, 120, 121, 127, 130, 136, 137, 139, 142, 145, 148, 144, 83, 78, 78,
 75, 74, 74, 72, 72, 75, 76, 81, 81, 86, 88, 92, 95, 99, 104, 106, 112,
 113, 121, 122, 128, 131, 137, 139, 140, 144, 148, 150, 155, 86, 82, 81,
 78, 77, 77, 75, 74, 78, 79, 84, 84, 89, 91, 95, 98, 101, 106, 109, 115,
 116, 124, 125, 131, 135, 140, 142, 144, 147, 149, 153, 155, 89, 84, 84,
 80, 80, 79, 78, 77, 79, 81, 85, 86, 91, 92, 97, 98, 104, 106, 112, 114,
 119, 123, 128, 132, 135, 142, 145, 148, 149, 153, 154, 159, 91, 86, 86,
 82, 82, 81, 80, 79, 80, 84, 85, 88, 91, 94, 97, 100, 104, 107, 112, 115,
 120, 123, 129, 132, 138, 140, 148, 150, 153, 154, 159, 159, 93, 88, 88,
 84, 84, 83, 83, 80, 81, 86, 86, 91, 91, 96, 97, 103, 103, 110, 110, 118,
 119, 126, 126, 135, 136, 144, 144, 155, 155, 159, 159, 164,
 // Size 4x8
 32, 35, 51, 77, 32, 36, 50, 72, 34, 42, 54, 75, 38, 51, 67, 87, 48, 59,
 80, 103, 60, 68, 92, 119, 72, 79, 104, 135, 81, 86, 112, 144,
 // Size 8x4
 32, 32, 34, 38, 48, 60, 72, 81, 35, 36, 42, 51, 59, 68, 79, 86, 51, 50,
 54, 67, 80, 92, 104, 112, 77, 72, 75, 87, 103, 119, 135, 144,
 // Size 8x16
 32, 31, 33, 40, 51, 65, 79, 87, 31, 32, 33, 39, 49, 61, 74, 82, 31, 32,
 34, 38, 47, 59, 71, 79, 32, 33, 36, 40, 48, 58, 69, 77, 33, 34, 38, 44,
 52, 62, 72, 78, 36, 35, 42, 51, 58, 68, 78, 84, 39, 38, 44, 54, 63, 73,
 84, 89, 44, 41, 46, 59, 69, 79, 90, 96, 48, 45, 50, 62, 74, 85, 96, 103,
 53, 49, 53, 66, 79, 92, 103, 111, 58, 54, 57, 70, 84, 98, 110, 118, 66,
 60, 63, 75, 90, 106, 119, 126, 74, 67, 69, 81, 97, 113, 128, 134, 81,
 73, 75, 86, 102, 120, 135, 143, 86, 78, 78, 90, 106, 124, 140, 147, 91,
 82, 80, 90, 103, 119, 137, 151,
 // Size 16x8
 32, 31, 31, 32, 33, 36, 39, 44, 48, 53, 58, 66, 74, 81, 86, 91, 31, 32,
 32, 33, 34, 35, 38, 41, 45, 49, 54, 60, 67, 73, 78, 82, 33, 33, 34, 36,
 38, 42, 44, 46, 50, 53, 57, 63, 69, 75, 78, 80, 40, 39, 38, 40, 44, 51,

54, 59, 62, 66, 70, 75, 81, 86, 90, 90, 51, 49, 47, 48, 52, 58, 63, 69,
 74, 79, 84, 90, 97, 102, 106, 103, 65, 61, 59, 58, 62, 68, 73, 79, 85,
 92, 98, 106, 113, 120, 124, 119, 79, 74, 71, 69, 72, 78, 84, 90, 96,
 103, 110, 119, 128, 135, 140, 137, 87, 82, 79, 77, 78, 84, 89, 96, 103,
 111, 118, 126, 134, 143, 147, 151,
 // Size 16x32
 32, 31, 31, 32, 33, 36, 40, 44, 51, 53, 65, 66, 79, 81, 87, 90, 31, 32,
 32, 32, 33, 35, 39, 42, 49, 51, 62, 63, 75, 77, 83, 85, 31, 32, 32, 32,
 33, 35, 39, 42, 49, 51, 61, 62, 74, 76, 82, 85, 31, 32, 32, 33, 33, 34,
 38, 41, 47, 49, 59, 60, 72, 74, 79, 81, 31, 32, 32, 33, 34, 35, 38, 41,
 47, 49, 59, 60, 71, 73, 79, 81, 32, 32, 33, 34, 35, 36, 39, 42, 48, 50,
 59, 60, 71, 72, 78, 80, 32, 32, 33, 35, 36, 37, 40, 42, 48, 49, 58, 59,
 69, 71, 77, 80, 32, 33, 33, 35, 36, 38, 41, 42, 48, 49, 58, 59, 69, 70,
 75, 77, 33, 33, 34, 36, 38, 41, 44, 46, 52, 53, 62, 63, 72, 74, 78, 78,
 34, 34, 34, 37, 39, 42, 45, 48, 53, 54, 63, 64, 73, 75, 80, 83, 36, 34,
 35, 38, 42, 48, 51, 54, 58, 60, 68, 69, 78, 80, 84, 83, 36, 35, 35, 38,
 42, 48, 51, 54, 59, 60, 68, 69, 79, 80, 85, 87, 39, 37, 38, 40, 44, 50,
 54, 58, 63, 65, 73, 74, 84, 85, 89, 88, 40, 38, 39, 41, 45, 51, 56, 59,
 65, 67, 75, 76, 85, 87, 90, 93, 44, 41, 41, 43, 46, 53, 59, 63, 69, 71,
 79, 80, 90, 91, 96, 93, 46, 43, 43, 44, 48, 55, 60, 65, 72, 73, 82, 83,
 93, 94, 97, 100, 48, 45, 45, 46, 50, 56, 62, 67, 74, 76, 85, 86, 96, 98,
 103, 100, 52, 48, 48, 49, 52, 59, 65, 70, 78, 80, 90, 91, 101, 103, 105,
 107, 53, 49, 49, 50, 53, 60, 66, 71, 79, 82, 92, 93, 103, 105, 111, 107,
 58, 53, 53, 53, 57, 63, 69, 74, 83, 86, 97, 98, 109, 111, 113, 115, 58,
 54, 54, 54, 57, 63, 70, 75, 84, 87, 98, 99, 110, 112, 118, 115, 65, 60,
 59, 58, 62, 68, 74, 79, 89, 92, 105, 106, 118, 119, 122, 123, 66, 61,
 60, 59, 63, 69, 75, 80, 90, 93, 106, 107, 119, 121, 126, 123, 71, 65,
 65, 63, 67, 73, 79, 84, 94, 97, 111, 112, 125, 127, 131, 132, 74, 68,
 67, 66, 69, 75, 81, 86, 97, 100, 113, 115, 128, 130, 134, 132, 79, 72,
 72, 70, 73, 79, 85, 90, 101, 104, 118, 119, 133, 135, 141, 140, 81, 74,
 73, 71, 75, 80, 86, 91, 102, 105, 120, 121, 135, 137, 143, 140, 82, 75,
 74, 72, 75, 81, 87, 92, 103, 106, 121, 122, 136, 139, 147, 151, 86, 78,
 78, 75, 78, 84, 90, 95, 106, 109, 124, 125, 140, 142, 147, 151, 88, 81,
 80, 77, 80, 86, 90, 98, 105, 112, 122, 127, 140, 144, 152, 155, 91, 83,
 82, 79, 80, 88, 90, 100, 103, 114, 119, 130, 137, 148, 151, 155, 93, 85,
 85, 81, 81, 90, 90, 102, 103, 117, 117, 134, 134, 151, 152, 160,
 // Size 32x16
 32, 31, 31, 31, 31, 32, 32, 32, 33, 34, 36, 36, 39, 40, 44, 46, 48, 52,
 53, 58, 58, 65, 66, 71, 74, 79, 81, 82, 86, 88, 91, 93, 31, 32, 32, 32,
 32, 32, 32, 33, 33, 34, 34, 35, 37, 38, 41, 43, 45, 48, 49, 53, 54, 60,
 61, 65, 68, 72, 74, 75, 78, 81, 83, 85, 31, 32, 32, 32, 33, 33, 33,
 34, 34, 35, 35, 38, 39, 41, 43, 45, 48, 49, 53, 54, 59, 60, 65, 67, 72,
 73, 74, 78, 80, 82, 85, 32, 32, 32, 33, 33, 34, 35, 35, 36, 37, 38, 38,
 40, 41, 43, 44, 46, 49, 50, 53, 54, 58, 59, 63, 66, 70, 71, 72, 75, 77,
 79, 81, 33, 33, 33, 33, 34, 35, 36, 36, 38, 39, 42, 42, 44, 45, 46, 48,
 50, 52, 53, 57, 57, 62, 63, 67, 69, 73, 75, 75, 78, 80, 80, 81, 36, 35,
 35, 34, 35, 36, 37, 38, 41, 42, 48, 48, 50, 51, 53, 55, 56, 59, 60, 63,
 63, 68, 69, 73, 75, 79, 80, 81, 84, 86, 88, 90, 40, 39, 39, 38, 38, 39,
 40, 41, 44, 45, 51, 51, 54, 56, 59, 60, 62, 65, 66, 69, 70, 74, 75, 79,
 81, 85, 86, 87, 90, 90, 90, 90, 44, 42, 42, 41, 41, 42, 42, 42, 46, 48,
 54, 54, 58, 59, 63, 65, 67, 70, 71, 74, 75, 79, 80, 84, 86, 90, 91, 92,
 95, 98, 100, 102, 51, 49, 49, 47, 47, 48, 48, 48, 52, 53, 58, 59, 63,
 65, 69, 72, 74, 78, 79, 83, 84, 89, 90, 94, 97, 101, 102, 103, 106, 105,
 103, 103, 53, 51, 51, 49, 49, 50, 49, 49, 53, 54, 60, 60, 65, 67, 71,
 73, 76, 80, 82, 86, 87, 92, 93, 97, 100, 104, 105, 106, 109, 112, 114,
 117, 65, 62, 61, 59, 59, 58, 62, 63, 68, 68, 73, 75, 79, 82, 85,
 90, 92, 97, 98, 105, 106, 111, 113, 118, 120, 121, 124, 122, 119, 117,
 66, 63, 62, 60, 60, 60, 59, 59, 63, 64, 69, 69, 74, 76, 80, 83, 86, 91,
 93, 98, 99, 106, 107, 112, 115, 119, 121, 122, 125, 127, 130, 134, 79,
 75, 74, 72, 71, 71, 69, 69, 72, 73, 78, 79, 84, 85, 90, 93, 96, 101,
 103, 109, 110, 118, 119, 125, 128, 133, 135, 136, 140, 140, 137, 134,
 81, 77, 76, 74, 73, 72, 71, 70, 74, 75, 80, 80, 85, 87, 91, 94, 98, 103,
 105, 111, 112, 119, 121, 127, 130, 135, 137, 139, 142, 144, 148, 151,
 87, 83, 82, 79, 79, 78, 77, 75, 78, 80, 84, 85, 89, 90, 96, 97, 103,
 105, 111, 113, 118, 122, 126, 131, 134, 141, 143, 147, 147, 152, 151,
 152, 90, 85, 85, 81, 81, 80, 80, 77, 78, 83, 83, 87, 88, 93, 93, 100,
 100, 107, 107, 115, 115, 123, 123, 132, 132, 140, 140, 151, 151, 155,
 155, 160,
 // Size 4x16
 31, 36, 53, 81, 32, 35, 51, 76, 32, 35, 49, 73, 32, 37, 49, 71, 33, 41,
 53, 74, 34, 48, 60, 80, 37, 50, 65, 85, 41, 53, 71, 91, 45, 56, 76, 98,

```

49, 60, 82, 105, 54, 63, 87, 112, 61, 69, 93, 121, 68, 75, 100, 130, 74,
80, 105, 137, 78, 84, 109, 142, 83, 88, 114, 148,
// Size 16x4
31, 32, 32, 32, 33, 34, 37, 41, 45, 49, 54, 61, 68, 74, 78, 83, 36, 35,
35, 37, 41, 48, 50, 53, 56, 60, 63, 69, 75, 80, 84, 88, 53, 51, 49, 49,
53, 60, 65, 71, 76, 82, 87, 93, 100, 105, 109, 114, 81, 76, 73, 71, 74,
80, 85, 91, 98, 105, 112, 121, 130, 137, 142, 148,
// Size 8x32
32, 31, 33, 40, 51, 65, 79, 87, 31, 32, 33, 39, 49, 62, 75, 83, 31, 32,
33, 39, 49, 61, 74, 82, 31, 32, 33, 38, 47, 59, 72, 79, 31, 32, 34, 38,
47, 59, 71, 79, 32, 33, 35, 39, 48, 59, 71, 78, 32, 33, 36, 40, 48, 58,
69, 77, 32, 33, 36, 41, 48, 58, 69, 75, 33, 34, 38, 44, 52, 62, 72, 78,
34, 34, 39, 45, 53, 63, 73, 80, 36, 35, 42, 51, 58, 68, 78, 84, 36, 35,
42, 51, 59, 68, 79, 85, 39, 38, 44, 54, 63, 73, 84, 89, 40, 39, 45, 56,
65, 75, 85, 90, 44, 41, 46, 59, 69, 79, 90, 96, 46, 43, 48, 60, 72, 82,
93, 97, 48, 45, 50, 62, 74, 85, 96, 103, 52, 48, 52, 65, 78, 90, 101,
105, 53, 49, 53, 66, 79, 92, 103, 111, 58, 53, 57, 69, 83, 97, 109, 113,
58, 54, 57, 70, 84, 98, 110, 118, 65, 59, 62, 74, 89, 105, 118, 122, 66,
60, 63, 75, 90, 106, 119, 126, 71, 65, 67, 79, 94, 111, 125, 131, 74,
67, 69, 81, 97, 113, 128, 134, 79, 72, 73, 85, 101, 118, 133, 141, 81,
73, 75, 86, 102, 120, 135, 143, 82, 74, 75, 87, 103, 121, 136, 147, 86,
78, 78, 90, 106, 124, 140, 147, 88, 80, 80, 90, 105, 122, 140, 152, 91,
82, 80, 90, 103, 119, 137, 151, 93, 85, 81, 90, 103, 117, 134, 152,
// Size 32x8
32, 31, 31, 31, 31, 32, 32, 32, 33, 34, 36, 36, 39, 40, 44, 46, 48, 52,
53, 58, 58, 65, 66, 71, 74, 79, 81, 82, 86, 88, 91, 93, 31, 32, 32, 32,
32, 33, 33, 33, 34, 34, 35, 35, 38, 39, 41, 43, 45, 48, 49, 53, 54, 59,
60, 65, 67, 72, 73, 74, 78, 80, 82, 85, 33, 33, 33, 33, 34, 35, 36, 36,
38, 39, 42, 42, 44, 45, 46, 48, 50, 52, 53, 57, 57, 62, 63, 67, 69, 73,
75, 75, 78, 80, 80, 81, 40, 39, 39, 38, 38, 39, 40, 41, 44, 45, 51, 51,
54, 56, 59, 60, 62, 65, 66, 69, 70, 74, 75, 79, 81, 85, 86, 87, 90, 90,
90, 90, 51, 49, 49, 47, 47, 48, 48, 48, 52, 53, 58, 59, 63, 65, 69, 72,
74, 78, 79, 83, 84, 89, 90, 94, 97, 101, 102, 103, 106, 105, 103, 103,
65, 62, 61, 59, 59, 59, 58, 58, 62, 63, 68, 68, 73, 75, 79, 82, 85, 90,
92, 97, 98, 105, 106, 111, 113, 118, 120, 121, 124, 122, 119, 117, 79,
75, 74, 72, 71, 71, 69, 69, 72, 73, 78, 79, 84, 85, 90, 93, 96, 101,
103, 109, 110, 118, 119, 125, 128, 133, 135, 136, 140, 140, 137, 134,
87, 83, 82, 79, 79, 78, 77, 75, 78, 80, 84, 85, 89, 90, 96, 97, 103,
105, 111, 113, 118, 122, 126, 131, 134, 141, 143, 147, 147, 152, 151,
152,
// Size 4x32
31, 36, 53, 81, 32, 35, 51, 77, 32, 35, 51, 76, 32, 34, 49, 74, 32, 35,
49, 73, 32, 36, 50, 72, 32, 37, 49, 71, 33, 38, 49, 70, 33, 41, 53, 74,
34, 42, 54, 75, 34, 48, 60, 80, 35, 48, 60, 80, 37, 50, 65, 85, 38, 51,
67, 87, 41, 53, 71, 91, 43, 55, 73, 94, 45, 56, 76, 98, 48, 59, 80, 103,
49, 60, 82, 105, 53, 63, 86, 111, 54, 63, 87, 112, 60, 68, 92, 119, 61,
69, 93, 121, 65, 73, 97, 127, 68, 75, 100, 130, 72, 79, 104, 135, 74,
80, 105, 137, 75, 81, 106, 139, 78, 84, 109, 142, 81, 86, 112, 144, 83,
88, 114, 148, 85, 90, 117, 151,
// Size 32x4
31, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 35, 37, 38, 41, 43, 45, 48,
49, 53, 54, 60, 61, 65, 68, 72, 74, 75, 78, 81, 83, 85, 36, 35, 35, 34,
35, 36, 37, 38, 41, 42, 48, 48, 50, 51, 53, 55, 56, 59, 60, 63, 63, 68,
69, 73, 75, 79, 80, 81, 84, 86, 88, 90, 53, 51, 51, 49, 49, 50, 49, 49,
53, 54, 60, 60, 65, 67, 71, 73, 76, 80, 82, 86, 87, 92, 93, 97, 100,
104, 105, 106, 109, 112, 114, 117, 81, 77, 76, 74, 73, 72, 71, 70, 74,
75, 80, 80, 85, 87, 91, 94, 98, 103, 105, 111, 112, 119, 121, 127, 130,
135, 137, 139, 142, 144, 148, 151,
},
{ // Chroma
// Size 4x4
32, 46, 49, 58, 46, 53, 55, 62, 49, 55, 70, 78, 58, 62, 78, 91,
// Size 8x8
31, 34, 42, 47, 49, 54, 60, 64, 34, 39, 45, 46, 47, 51, 56, 59, 42, 45,
48, 49, 50, 53, 57, 60, 47, 46, 49, 55, 58, 61, 65, 66, 49, 47, 50, 58,
65, 69, 73, 74, 54, 51, 53, 61, 69, 76, 82, 83, 60, 56, 57, 65, 73, 82,
89, 92, 64, 59, 60, 66, 74, 83, 92, 96,
// Size 16x16
32, 31, 31, 35, 40, 49, 48, 49, 50, 52, 54, 57, 61, 64, 66, 68, 31, 31,
32, 37, 41, 47, 47, 46, 48, 49, 51, 54, 57, 60, 62, 64, 31, 32, 34, 39,
43, 46, 46, 45, 46, 47, 49, 52, 55, 57, 59, 61, 35, 37, 39, 44, 46, 47,

```

```

46, 45, 46, 47, 48, 51, 53, 56, 57, 59, 40, 41, 43, 46, 48, 50, 49, 48,
49, 49, 51, 53, 55, 57, 59, 59, 49, 47, 46, 47, 50, 53, 53, 53, 54, 54,
55, 57, 59, 61, 62, 62, 48, 47, 46, 46, 49, 53, 54, 55, 56, 57, 58, 60,
62, 64, 65, 65, 49, 46, 45, 45, 48, 53, 55, 58, 60, 61, 62, 64, 66, 68,
69, 69, 50, 48, 46, 46, 49, 54, 56, 60, 61, 63, 65, 67, 69, 71, 72, 72,
52, 49, 47, 47, 49, 54, 57, 61, 63, 66, 68, 71, 73, 75, 76, 77, 54, 51,
49, 48, 51, 55, 58, 62, 65, 68, 71, 74, 76, 78, 80, 81, 57, 54, 52, 51,
53, 57, 60, 64, 67, 71, 74, 77, 80, 83, 84, 85, 61, 57, 55, 53, 55, 59,
62, 66, 69, 73, 76, 80, 84, 87, 89, 89, 64, 60, 57, 56, 57, 61, 64, 68,
71, 75, 78, 83, 87, 90, 92, 94, 66, 62, 59, 57, 59, 62, 65, 69, 72, 76,
80, 84, 89, 92, 94, 96, 68, 64, 61, 59, 59, 62, 65, 69, 72, 77, 81, 85,
89, 94, 96, 98,
// Size 32x32
32, 31, 31, 30, 31, 33, 35, 36, 40, 41, 49, 49, 48, 48, 49, 50, 50, 52,
52, 54, 54, 57, 57, 60, 61, 63, 64, 65, 66, 67, 68, 69, 31, 31, 31, 31,
32, 34, 37, 38, 41, 42, 47, 47, 47, 47, 48, 49, 50, 52, 52, 54,
55, 57, 58, 60, 61, 61, 63, 64, 64, 65, 31, 31, 31, 31, 32, 35, 37, 39,
41, 42, 47, 47, 47, 46, 46, 47, 48, 49, 49, 51, 51, 54, 54, 56, 57, 59,
60, 61, 62, 63, 64, 65, 30, 31, 31, 32, 33, 35, 38, 40, 42, 42, 46, 46,
45, 45, 45, 45, 46, 47, 47, 49, 49, 52, 52, 54, 55, 57, 58, 58, 60, 61,
61, 62, 31, 32, 32, 33, 34, 37, 39, 41, 43, 43, 46, 46, 46, 45, 45, 46,
46, 47, 47, 49, 49, 51, 52, 54, 55, 57, 57, 58, 59, 60, 61, 62, 33, 34,
35, 35, 37, 39, 41, 43, 44, 45, 47, 47, 46, 46, 45, 46, 46, 47, 47, 49,
49, 51, 51, 53, 54, 56, 57, 57, 58, 59, 60, 61, 35, 37, 37, 38, 39, 41,
44, 46, 46, 46, 47, 47, 46, 46, 45, 46, 46, 47, 47, 48, 48, 50, 51, 52,
53, 55, 56, 56, 57, 58, 59, 61, 36, 38, 39, 40, 41, 43, 46, 47, 47, 47,
48, 47, 46, 46, 45, 46, 46, 46, 47, 48, 48, 50, 50, 52, 53, 54, 55, 55,
56, 57, 58, 58, 40, 41, 41, 42, 43, 44, 46, 47, 48, 48, 50, 49, 49, 49,
48, 49, 49, 49, 49, 51, 51, 52, 53, 54, 55, 57, 57, 58, 59, 59, 59, 59,
41, 42, 42, 42, 43, 45, 46, 47, 48, 48, 50, 50, 49, 49, 49, 49, 50, 50,
50, 52, 52, 53, 53, 55, 56, 57, 58, 58, 59, 60, 61, 62, 49, 47, 47, 46,
46, 47, 47, 48, 50, 50, 53, 53, 53, 53, 53, 54, 54, 54, 55, 55, 56,
57, 58, 59, 60, 61, 61, 62, 62, 62, 62, 49, 47, 47, 46, 46, 47, 47, 47,
49, 50, 53, 53, 53, 53, 54, 54, 54, 54, 54, 55, 56, 57, 57, 59, 59, 61,
61, 62, 63, 63, 64, 65, 48, 47, 47, 45, 46, 46, 46, 46, 49, 49, 53, 53,
54, 54, 55, 56, 56, 57, 57, 58, 58, 60, 60, 61, 62, 63, 64, 64, 65, 66,
65, 65, 48, 47, 46, 45, 45, 46, 46, 46, 49, 49, 53, 53, 54, 55, 56, 57,
57, 58, 58, 59, 60, 61, 61, 63, 63, 65, 65, 65, 66, 66, 67, 68, 49, 47,
46, 45, 45, 45, 45, 45, 48, 49, 53, 54, 55, 56, 58, 59, 60, 61, 61, 62,
62, 63, 64, 65, 66, 67, 68, 68, 69, 70, 69, 68, 50, 47, 47, 45, 46, 46,
46, 46, 49, 49, 54, 54, 56, 57, 59, 60, 60, 62, 62, 63, 64, 65, 65, 67,
68, 69, 69, 70, 70, 70, 71, 71, 50, 48, 48, 46, 46, 46, 46, 46, 49, 50,
54, 54, 56, 57, 60, 60, 61, 63, 63, 65, 65, 67, 67, 68, 69, 71, 71, 71,
72, 73, 72, 71, 52, 49, 49, 47, 47, 47, 47, 46, 49, 50, 54, 54, 57, 58,
61, 62, 63, 65, 65, 67, 67, 69, 70, 71, 72, 73, 74, 74, 75, 74, 74, 75,
52, 50, 49, 47, 47, 47, 47, 47, 49, 50, 54, 54, 57, 58, 61, 62, 63, 65,
66, 68, 68, 70, 71, 72, 73, 75, 75, 75, 76, 77, 77, 75, 54, 52, 51, 49,
49, 49, 48, 48, 51, 52, 55, 55, 58, 59, 62, 63, 65, 67, 68, 70, 70, 73,
73, 75, 76, 78, 78, 78, 79, 78, 78, 79, 54, 52, 51, 49, 49, 49, 48, 48,
51, 52, 55, 56, 58, 60, 62, 64, 65, 67, 68, 70, 71, 73, 74, 75, 76, 78,
78, 79, 80, 81, 81, 79, 57, 54, 54, 52, 51, 51, 50, 50, 52, 53, 56, 57,
60, 61, 63, 65, 67, 69, 70, 73, 73, 76, 77, 79, 80, 82, 82, 83, 84, 83,
82, 83, 57, 55, 54, 52, 52, 51, 51, 50, 53, 53, 57, 57, 60, 61, 64, 65,
67, 70, 71, 73, 74, 77, 77, 79, 80, 82, 83, 83, 84, 85, 85, 83, 60, 57,
56, 54, 54, 53, 52, 52, 54, 55, 58, 59, 61, 63, 65, 67, 68, 71, 72, 75,
75, 79, 79, 82, 83, 85, 86, 86, 87, 87, 86, 87, 61, 58, 57, 55, 55, 54,
53, 53, 55, 56, 59, 59, 62, 63, 66, 68, 69, 72, 73, 76, 76, 80, 80, 83,
84, 86, 87, 88, 89, 89, 89, 87, 63, 60, 59, 57, 57, 56, 55, 54, 57, 57,
60, 61, 63, 65, 67, 69, 71, 73, 75, 78, 78, 82, 82, 85, 86, 89, 89, 90,
91, 92, 90, 91, 64, 61, 60, 58, 57, 57, 56, 55, 57, 58, 61, 61, 64, 65,
68, 69, 71, 74, 75, 78, 78, 82, 83, 86, 87, 89, 90, 91, 92, 93, 94, 91,
65, 61, 61, 58, 58, 57, 56, 55, 58, 58, 61, 62, 64, 65, 68, 70, 71, 74,
75, 78, 79, 83, 83, 86, 88, 90, 91, 91, 93, 94, 94, 96, 66, 63, 62, 60,
59, 58, 57, 56, 59, 59, 62, 63, 65, 66, 69, 70, 72, 75, 76, 79, 80, 84,
84, 87, 89, 91, 92, 93, 94, 94, 96, 96, 67, 64, 63, 61, 60, 59, 58, 57,
59, 60, 62, 63, 66, 66, 70, 70, 73, 74, 77, 78, 81, 83, 85, 87, 89, 92,
93, 94, 94, 96, 96, 97, 68, 64, 64, 61, 61, 60, 59, 58, 59, 61, 62, 64,
65, 67, 69, 71, 72, 74, 77, 78, 81, 82, 85, 86, 89, 90, 94, 94, 96, 96,
98, 97, 69, 65, 65, 62, 62, 61, 61, 58, 59, 62, 62, 65, 65, 68, 68, 71,
71, 75, 75, 79, 79, 83, 83, 87, 87, 91, 91, 96, 96, 97, 97, 99,
// Size 4x8

```

```

31, 47, 50, 61, 36, 47, 47, 57, 43, 50, 50, 58, 45, 53, 58, 65, 47, 54,
66, 74, 52, 56, 70, 82, 57, 60, 75, 90, 61, 63, 77, 93,
// Size 8x4
31, 36, 43, 45, 47, 52, 57, 61, 47, 47, 50, 53, 54, 56, 60, 63, 50, 47,
50, 58, 66, 70, 75, 77, 61, 57, 58, 65, 74, 82, 90, 93,
// Size 8x16
32, 32, 40, 49, 51, 57, 63, 67, 31, 33, 41, 47, 49, 54, 59, 63, 31, 35,
43, 46, 47, 51, 57, 60, 35, 39, 46, 46, 47, 50, 55, 58, 41, 43, 48, 49,
49, 52, 57, 59, 49, 47, 50, 53, 54, 57, 60, 62, 48, 46, 49, 54, 57, 60,
64, 65, 49, 45, 48, 56, 61, 64, 67, 69, 50, 46, 49, 57, 63, 67, 71, 73,
52, 48, 50, 58, 65, 71, 75, 77, 54, 50, 51, 59, 67, 73, 78, 81, 57, 52,
53, 61, 69, 77, 82, 85, 61, 55, 56, 63, 72, 80, 86, 88, 64, 58, 58, 65,
73, 82, 89, 92, 66, 59, 59, 66, 75, 84, 91, 94, 68, 61, 59, 65, 72, 81,
89, 95,
// Size 16x8
32, 31, 31, 35, 41, 49, 48, 49, 50, 52, 54, 57, 61, 64, 66, 68, 32, 33,
35, 39, 43, 47, 46, 45, 46, 48, 50, 52, 55, 58, 59, 61, 40, 41, 43, 46,
48, 50, 49, 48, 49, 50, 51, 53, 56, 58, 59, 59, 49, 47, 46, 46, 49, 53,
54, 56, 57, 58, 59, 61, 63, 65, 66, 65, 51, 49, 47, 47, 49, 54, 57, 61,
63, 65, 67, 69, 72, 73, 75, 72, 57, 54, 51, 50, 52, 57, 60, 64, 67, 71,
73, 77, 80, 82, 84, 81, 63, 59, 57, 55, 57, 60, 64, 67, 71, 75, 78, 82,
86, 89, 91, 89, 67, 63, 60, 58, 59, 62, 65, 69, 73, 77, 81, 85, 88, 92,
94, 95,
// Size 16x32
32, 31, 32, 37, 40, 48, 49, 49, 51, 52, 57, 58, 63, 64, 67, 67, 31, 31,
33, 38, 41, 47, 47, 47, 49, 50, 54, 55, 60, 61, 63, 64, 31, 31, 33, 38,
41, 47, 47, 47, 49, 49, 54, 54, 59, 60, 63, 64, 30, 32, 33, 40, 42, 46,
45, 45, 47, 48, 52, 52, 57, 58, 60, 61, 31, 33, 35, 41, 43, 46, 46, 45,
47, 48, 51, 52, 57, 57, 60, 61, 33, 36, 37, 43, 44, 47, 46, 46, 47, 47,
51, 52, 56, 57, 59, 60, 35, 38, 39, 45, 46, 47, 46, 45, 47, 47, 50, 51,
55, 56, 58, 60, 37, 40, 41, 47, 47, 46, 45, 46, 47, 50, 50, 54, 55,
57, 58, 41, 42, 43, 47, 48, 49, 49, 48, 49, 50, 52, 53, 57, 57, 59, 58,
42, 43, 43, 47, 48, 50, 49, 49, 50, 50, 53, 54, 57, 58, 60, 61, 49, 46,
47, 48, 50, 53, 53, 53, 54, 54, 57, 57, 60, 61, 62, 61, 49, 46, 47, 48,
50, 53, 53, 54, 54, 55, 57, 57, 61, 61, 63, 64, 48, 46, 46, 47, 49, 53,
54, 56, 57, 57, 60, 60, 64, 64, 65, 64, 48, 45, 46, 46, 49, 53, 55, 56,
58, 58, 61, 61, 65, 65, 66, 67, 49, 45, 45, 46, 48, 53, 56, 58, 61, 61,
64, 64, 67, 68, 69, 67, 49, 46, 46, 46, 49, 53, 57, 59, 62, 62, 65, 66,
69, 69, 70, 70, 50, 46, 46, 46, 49, 54, 57, 59, 63, 64, 67, 67, 71, 71,
73, 71, 51, 47, 47, 47, 49, 54, 58, 61, 64, 66, 69, 70, 73, 74, 74, 74,
52, 48, 48, 47, 50, 54, 58, 61, 65, 66, 71, 71, 75, 75, 77, 74, 54, 50,
49, 48, 51, 55, 59, 62, 67, 68, 73, 73, 77, 78, 78, 78, 54, 50, 50, 49,
51, 55, 59, 62, 67, 68, 73, 74, 78, 78, 81, 78, 57, 52, 52, 50, 52, 56,
60, 64, 69, 70, 76, 77, 82, 82, 83, 82, 57, 52, 52, 51, 53, 57, 61, 64,
69, 71, 77, 77, 82, 83, 85, 82, 60, 54, 54, 52, 55, 58, 62, 65, 71, 72,
79, 79, 85, 86, 87, 86, 61, 56, 55, 53, 56, 59, 63, 66, 72, 73, 80, 81,
86, 87, 88, 86, 63, 57, 57, 55, 57, 60, 64, 67, 73, 75, 82, 82, 89, 90,
92, 90, 64, 58, 58, 55, 58, 61, 65, 68, 73, 75, 82, 83, 89, 90, 92, 90,
64, 59, 58, 56, 58, 61, 65, 68, 74, 75, 83, 83, 90, 91, 94, 95, 66, 60,
59, 57, 59, 62, 66, 69, 75, 76, 84, 85, 91, 92, 94, 95, 67, 61, 60, 58,
59, 63, 66, 70, 74, 77, 82, 85, 91, 93, 96, 96, 68, 62, 61, 58, 59, 64,
65, 71, 72, 78, 81, 86, 89, 94, 95, 96, 68, 62, 59, 59, 65, 65, 71,
71, 79, 79, 87, 87, 95, 95, 98,
// Size 32x16
32, 31, 31, 30, 31, 33, 35, 37, 41, 42, 49, 49, 48, 48, 49, 49, 50, 51,
52, 54, 54, 57, 57, 60, 61, 63, 64, 66, 67, 68, 68, 31, 31, 31, 32,
33, 36, 38, 40, 42, 43, 46, 46, 46, 45, 45, 46, 46, 47, 48, 50, 50, 52,
52, 54, 56, 57, 58, 59, 60, 61, 62, 62, 32, 33, 33, 33, 35, 37, 39, 41,
43, 43, 47, 47, 46, 46, 45, 46, 46, 47, 48, 49, 50, 52, 52, 54, 55, 57,
58, 58, 59, 60, 61, 62, 37, 38, 38, 40, 41, 43, 45, 47, 47, 47, 48, 48,
47, 46, 46, 46, 46, 47, 48, 49, 50, 51, 52, 53, 55, 55, 56, 57, 58,
58, 59, 40, 41, 41, 42, 43, 44, 46, 47, 48, 48, 50, 50, 49, 49, 48, 49,
49, 49, 50, 51, 51, 52, 53, 55, 56, 57, 58, 58, 59, 59, 59, 59, 48, 47,
47, 46, 46, 47, 47, 47, 49, 50, 53, 53, 53, 53, 53, 53, 54, 54, 54, 55,
55, 56, 57, 58, 59, 60, 61, 61, 62, 63, 64, 65, 49, 47, 47, 45, 46, 46,
46, 46, 49, 49, 53, 53, 54, 55, 56, 57, 57, 58, 58, 59, 60, 61, 62,
63, 64, 65, 65, 66, 66, 65, 49, 47, 47, 45, 45, 46, 45, 45, 48, 49,
53, 54, 56, 56, 58, 59, 59, 61, 61, 62, 62, 64, 64, 65, 66, 67, 68, 68,
69, 70, 71, 71, 51, 49, 49, 47, 47, 47, 47, 46, 49, 50, 54, 54, 57, 58,
61, 62, 63, 64, 65, 67, 67, 69, 69, 71, 72, 73, 73, 74, 75, 74, 72, 71,
52, 50, 49, 48, 48, 47, 47, 47, 50, 50, 54, 55, 57, 58, 61, 62, 64, 66,

```

```

66, 68, 68, 70, 71, 72, 73, 75, 75, 75, 76, 77, 78, 79, 57, 54, 54, 52,
51, 51, 50, 50, 52, 53, 57, 57, 60, 61, 64, 65, 67, 69, 71, 73, 73, 76,
77, 79, 80, 82, 82, 83, 84, 82, 81, 79, 58, 55, 54, 52, 52, 52, 51, 50,
53, 54, 57, 57, 60, 61, 64, 66, 67, 70, 71, 73, 74, 77, 77, 79, 81, 82,
83, 83, 85, 85, 86, 87, 63, 60, 59, 57, 57, 56, 55, 54, 57, 57, 60, 61,
64, 65, 67, 69, 71, 73, 75, 77, 78, 82, 82, 85, 86, 89, 89, 90, 91, 91,
89, 87, 64, 61, 60, 58, 57, 57, 56, 55, 57, 58, 61, 61, 64, 65, 68, 69,
71, 74, 75, 78, 78, 82, 83, 86, 87, 90, 90, 91, 92, 93, 94, 95, 67, 63,
63, 60, 60, 59, 58, 57, 59, 60, 62, 63, 65, 66, 69, 70, 73, 74, 77, 78,
81, 83, 85, 87, 88, 92, 92, 94, 94, 96, 95, 95, 67, 64, 64, 61, 61, 60,
60, 58, 58, 61, 61, 64, 64, 67, 67, 70, 71, 74, 74, 78, 78, 82, 82, 86,
86, 90, 90, 95, 95, 96, 96, 98,
// Size 4x16
31, 48, 52, 64, 31, 47, 49, 60, 33, 46, 48, 57, 38, 47, 47, 56, 42, 49,
50, 57, 46, 53, 54, 61, 46, 53, 57, 64, 45, 53, 61, 68, 46, 54, 64, 71,
48, 54, 66, 75, 50, 55, 68, 78, 52, 57, 71, 83, 56, 59, 73, 87, 58, 61,
75, 90, 60, 62, 76, 92, 62, 64, 78, 94,
// Size 16x4
31, 31, 33, 38, 42, 46, 46, 45, 46, 48, 50, 52, 56, 58, 60, 62, 48, 47,
46, 47, 49, 53, 53, 53, 54, 54, 55, 57, 59, 61, 62, 64, 52, 49, 48, 47,
50, 54, 57, 61, 64, 66, 68, 71, 73, 75, 76, 78, 64, 60, 57, 56, 57, 61,
64, 68, 71, 75, 78, 83, 87, 90, 92, 94,
// Size 8x32
32, 32, 40, 49, 51, 57, 63, 67, 31, 33, 41, 47, 49, 54, 60, 63, 31, 33,
41, 47, 49, 54, 59, 63, 30, 33, 42, 45, 47, 52, 57, 60, 31, 35, 43, 46,
47, 51, 57, 60, 33, 37, 44, 46, 47, 51, 56, 59, 35, 39, 46, 46, 47, 50,
55, 58, 37, 41, 47, 46, 46, 50, 54, 57, 41, 43, 48, 49, 49, 52, 57, 59,
42, 43, 48, 49, 50, 53, 57, 60, 49, 47, 50, 53, 54, 57, 60, 62, 49, 47,
50, 53, 54, 57, 61, 63, 48, 46, 49, 54, 57, 60, 64, 65, 48, 46, 49, 55,
58, 61, 65, 66, 49, 45, 48, 56, 61, 64, 67, 69, 49, 46, 49, 57, 62, 65,
69, 70, 50, 46, 49, 57, 63, 67, 71, 73, 51, 47, 49, 58, 64, 69, 73, 74,
52, 48, 50, 58, 65, 71, 75, 77, 54, 49, 51, 59, 67, 73, 77, 78, 54, 50,
51, 59, 67, 73, 78, 81, 57, 52, 52, 60, 69, 76, 82, 83, 57, 52, 53, 61,
69, 77, 82, 85, 60, 54, 55, 62, 71, 79, 85, 87, 61, 55, 56, 63, 72, 80,
86, 88, 63, 57, 57, 64, 73, 82, 89, 92, 64, 58, 58, 65, 73, 82, 89, 92,
64, 58, 58, 65, 74, 83, 90, 94, 66, 59, 59, 66, 75, 84, 91, 94, 67, 60,
59, 66, 74, 82, 91, 96, 68, 61, 59, 65, 72, 81, 89, 95, 68, 62, 59, 65,
71, 79, 87, 95,
// Size 32x8
32, 31, 31, 30, 31, 33, 35, 37, 41, 42, 49, 49, 48, 48, 49, 49, 50, 51,
52, 54, 54, 57, 57, 60, 61, 63, 64, 66, 67, 68, 68, 32, 33, 33, 33,
35, 37, 39, 41, 43, 43, 47, 47, 46, 46, 45, 46, 46, 47, 48, 49, 50, 52,
52, 54, 55, 57, 58, 58, 59, 60, 61, 62, 40, 41, 41, 42, 43, 44, 46, 47,
48, 48, 50, 50, 49, 49, 48, 49, 49, 49, 50, 51, 51, 52, 53, 55, 56, 57,
58, 58, 59, 59, 59, 49, 47, 47, 45, 46, 46, 46, 46, 49, 49, 53, 53,
54, 55, 56, 57, 57, 58, 58, 59, 59, 60, 61, 62, 63, 64, 65, 65, 66, 66,
65, 65, 51, 49, 49, 47, 47, 47, 46, 49, 50, 54, 54, 57, 58, 61, 62,
63, 64, 65, 67, 67, 69, 69, 71, 72, 73, 73, 74, 75, 74, 72, 71, 57, 54,
54, 52, 51, 51, 50, 50, 52, 53, 57, 57, 60, 61, 64, 65, 67, 69, 71, 73,
73, 76, 77, 79, 80, 82, 82, 83, 84, 82, 81, 79, 63, 60, 59, 57, 57, 56,
55, 54, 57, 57, 60, 61, 64, 65, 67, 69, 71, 73, 75, 77, 78, 82, 82, 85,
86, 89, 89, 90, 91, 91, 89, 87, 67, 63, 63, 60, 60, 59, 58, 57, 59, 60,
62, 63, 65, 66, 69, 70, 73, 74, 77, 78, 81, 83, 85, 87, 88, 92, 92, 94,
94, 96, 95, 95,
// Size 4x32
31, 48, 52, 64, 31, 47, 50, 61, 31, 47, 49, 60, 32, 46, 48, 58, 33, 46,
48, 57, 36, 47, 47, 57, 38, 47, 47, 56, 40, 47, 47, 55, 42, 49, 50, 57,
43, 50, 50, 58, 46, 53, 54, 61, 46, 53, 55, 61, 46, 53, 57, 64, 45, 53,
58, 65, 45, 53, 61, 68, 46, 53, 62, 69, 46, 54, 64, 71, 47, 54, 66, 74,
48, 54, 66, 75, 50, 55, 68, 78, 50, 55, 68, 78, 52, 56, 70, 82, 52, 57,
71, 83, 54, 58, 72, 86, 56, 59, 73, 87, 57, 60, 75, 90, 58, 61, 75, 90,
59, 61, 75, 91, 60, 62, 76, 92, 61, 63, 77, 93, 62, 64, 78, 94, 62, 65,
79, 95,
// Size 32x4
31, 31, 31, 32, 33, 36, 38, 40, 42, 43, 46, 46, 46, 45, 45, 46, 46, 47,
48, 50, 50, 52, 52, 54, 56, 57, 58, 59, 60, 61, 62, 62, 48, 47, 47, 46,
46, 47, 47, 49, 50, 53, 53, 53, 53, 53, 54, 54, 54, 55, 55, 56,
57, 58, 59, 60, 61, 61, 62, 63, 64, 65, 52, 50, 49, 48, 48, 47, 47, 47,
50, 50, 54, 55, 57, 58, 61, 62, 64, 66, 66, 68, 68, 70, 71, 72, 73, 75,
75, 75, 76, 77, 78, 79, 64, 61, 60, 58, 57, 57, 56, 55, 57, 58, 61, 61,
64, 65, 68, 69, 71, 74, 75, 78, 78, 82, 83, 86, 87, 90, 90, 91, 92, 93,

```



```

    94, 95,
},
{
{ // Luma
// Size 4x4
32, 34, 49, 72, 34, 48, 60, 79, 49, 60, 82, 104, 72, 79, 104, 134,
// Size 8x8
32, 32, 34, 38, 46, 56, 68, 78, 32, 33, 35, 39, 45, 54, 64, 74, 34, 35,
39, 45, 51, 58, 68, 76, 38, 39, 45, 54, 61, 69, 78, 86, 46, 45, 51, 61,
71, 80, 90, 99, 56, 54, 58, 69, 80, 92, 103, 113, 68, 64, 68, 78, 90,
103, 117, 128, 78, 74, 76, 86, 99, 113, 128, 140,
// Size 16x16
32, 31, 31, 31, 32, 34, 36, 39, 44, 48, 54, 59, 65, 71, 80, 83, 31, 32,
32, 32, 32, 34, 35, 38, 42, 46, 51, 56, 62, 68, 76, 78, 31, 32, 32, 32,
32, 33, 34, 37, 41, 44, 49, 54, 59, 65, 72, 75, 31, 32, 32, 33, 34, 35,
36, 39, 42, 45, 50, 54, 59, 64, 71, 74, 32, 32, 32, 34, 35, 37, 38, 40,
42, 46, 49, 53, 58, 63, 69, 72, 34, 34, 33, 35, 37, 39, 42, 45, 47, 51,
54, 58, 63, 68, 74, 76, 36, 35, 34, 36, 38, 42, 48, 50, 54, 57, 60, 64,
68, 73, 79, 81, 39, 38, 37, 39, 40, 45, 50, 54, 58, 61, 65, 69, 73, 78,
84, 86, 44, 42, 41, 42, 42, 47, 54, 58, 63, 67, 71, 75, 79, 84, 90, 92,
48, 46, 44, 45, 46, 51, 57, 61, 67, 71, 76, 80, 85, 90, 96, 99, 54, 51,
49, 50, 49, 54, 60, 65, 71, 76, 82, 87, 92, 97, 104, 106, 59, 56, 54,
54, 53, 58, 64, 69, 75, 80, 87, 92, 98, 103, 110, 113, 65, 62, 59, 59,
58, 63, 68, 73, 79, 85, 92, 98, 105, 111, 118, 121, 71, 68, 65, 64, 63,
68, 73, 78, 84, 90, 97, 103, 111, 117, 125, 128, 80, 76, 72, 71, 69, 74,
79, 84, 90, 96, 104, 110, 118, 125, 134, 137, 83, 78, 75, 74, 72, 76,
81, 86, 92, 99, 106, 113, 121, 128, 137, 140,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 32, 32, 34, 34, 36, 39, 39, 44, 44, 48,
48, 54, 54, 59, 59, 65, 65, 71, 71, 80, 80, 83, 83, 87, 31, 32, 32, 32,
32, 32, 32, 32, 32, 34, 34, 35, 35, 38, 38, 42, 42, 46, 46, 51, 51, 56,
56, 62, 62, 68, 68, 76, 76, 78, 78, 83, 31, 32, 32, 32, 32, 32, 32,
32, 34, 34, 35, 35, 38, 38, 42, 42, 46, 46, 51, 51, 56, 56, 62, 62, 68,
68, 76, 76, 78, 78, 83, 31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34,
34, 37, 37, 41, 41, 44, 44, 49, 49, 54, 54, 59, 59, 65, 65, 72, 72, 75,
75, 79, 31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 37, 37, 41,
41, 44, 44, 49, 49, 54, 54, 59, 59, 65, 65, 72, 72, 75, 75, 79, 31, 32,
32, 32, 32, 33, 33, 34, 34, 35, 35, 36, 36, 39, 39, 42, 42, 45, 45, 50,
50, 54, 54, 59, 59, 64, 64, 71, 71, 74, 74, 77, 31, 32, 32, 32, 32, 33,
33, 34, 34, 35, 35, 36, 36, 39, 39, 42, 42, 45, 45, 50, 50, 54, 54, 59,
59, 64, 64, 71, 71, 74, 74, 77, 32, 32, 32, 32, 32, 34, 34, 35, 35, 37,
37, 38, 38, 40, 40, 42, 42, 46, 46, 49, 49, 53, 53, 58, 58, 63, 63, 69,
69, 72, 72, 75, 32, 32, 32, 32, 32, 34, 34, 35, 35, 37, 37, 38, 38, 40,
40, 42, 42, 46, 46, 49, 49, 53, 53, 58, 58, 63, 63, 69, 69, 72, 72, 75,
34, 34, 34, 33, 33, 35, 35, 37, 37, 39, 39, 42, 42, 45, 45, 47, 47, 51,
51, 54, 54, 58, 58, 63, 63, 68, 68, 74, 74, 76, 76, 80, 34, 34, 34, 33,
33, 35, 35, 37, 37, 39, 39, 42, 42, 45, 45, 47, 47, 51, 51, 54, 54, 58,
58, 63, 63, 68, 68, 74, 74, 76, 76, 80, 36, 35, 35, 34, 34, 36, 36, 38,
38, 42, 42, 48, 48, 50, 50, 54, 54, 57, 57, 60, 60, 64, 64, 68, 68, 73,
73, 79, 79, 81, 81, 84, 36, 35, 35, 34, 34, 36, 36, 38, 38, 42, 42, 48,
48, 50, 50, 54, 54, 57, 57, 60, 60, 64, 64, 68, 68, 73, 73, 79, 79, 81,
81, 84, 39, 38, 38, 37, 37, 39, 39, 40, 40, 45, 45, 50, 50, 54, 54, 58,
58, 61, 61, 65, 65, 69, 69, 73, 73, 78, 78, 84, 84, 86, 86, 90, 39, 38,
38, 37, 37, 39, 39, 40, 40, 45, 45, 50, 50, 54, 54, 58, 58, 61, 61, 65,
65, 69, 69, 73, 73, 78, 78, 84, 84, 86, 86, 90, 44, 42, 42, 41, 41, 42,
42, 42, 42, 47, 47, 54, 54, 58, 58, 63, 63, 67, 67, 71, 71, 75, 75, 79,
79, 84, 84, 90, 90, 92, 92, 96, 44, 42, 42, 41, 41, 42, 42, 42, 42, 47,
47, 54, 54, 58, 58, 63, 63, 67, 67, 71, 71, 75, 75, 79, 79, 84, 84, 90,
90, 92, 92, 96, 48, 46, 46, 44, 44, 45, 45, 46, 46, 51, 51, 57, 57, 61,
61, 67, 67, 71, 71, 76, 76, 80, 80, 85, 85, 90, 96, 96, 99, 99, 102,
48, 46, 46, 44, 44, 45, 45, 46, 46, 51, 51, 57, 57, 61, 61, 67, 67, 71,
71, 76, 76, 80, 80, 85, 85, 90, 90, 96, 96, 99, 99, 102, 54, 51, 51, 49,
49, 50, 50, 49, 49, 54, 54, 60, 60, 65, 65, 71, 71, 76, 76, 82, 82, 87,
87, 92, 92, 97, 97, 104, 104, 106, 106, 109, 54, 51, 51, 49, 49, 50, 50,
49, 49, 54, 54, 60, 60, 65, 65, 71, 71, 76, 76, 82, 82, 87, 87, 92, 92,
97, 97, 104, 104, 106, 106, 109, 59, 56, 56, 54, 54, 54, 54, 53, 53, 58,
58, 64, 64, 69, 69, 75, 75, 80, 80, 87, 87, 92, 92, 98, 98, 103, 103,
110, 110, 113, 113, 116, 59, 56, 56, 54, 54, 54, 54, 53, 53, 58, 58, 64,
64, 69, 69, 75, 75, 80, 80, 87, 87, 92, 92, 98, 98, 103, 103, 110, 110,
113, 113, 116, 65, 62, 62, 59, 59, 59, 59, 58, 58, 63, 63, 68, 68, 73,

```

```

73, 79, 79, 85, 85, 92, 92, 98, 98, 105, 105, 111, 111, 118, 118, 121,
121, 124, 65, 62, 62, 59, 59, 59, 59, 58, 58, 63, 63, 68, 68, 73, 73,
79, 79, 85, 85, 92, 92, 98, 98, 105, 105, 111, 111, 118, 118, 121, 121,
124, 71, 68, 68, 65, 65, 64, 64, 63, 63, 68, 68, 73, 73, 78, 78, 84, 84,
90, 90, 97, 97, 103, 103, 111, 111, 117, 117, 125, 125, 128, 128, 132,
71, 68, 68, 65, 65, 64, 64, 63, 63, 68, 68, 73, 73, 78, 78, 84, 84, 90,
90, 97, 97, 103, 103, 111, 111, 117, 117, 125, 125, 128, 128, 132, 80,
76, 76, 72, 72, 71, 71, 69, 69, 74, 74, 79, 79, 84, 84, 90, 90, 96, 96,
104, 104, 110, 110, 118, 118, 125, 125, 134, 134, 137, 137, 141, 80, 76,
76, 72, 72, 71, 71, 69, 69, 74, 74, 79, 79, 84, 84, 90, 90, 96, 96, 104,
104, 110, 110, 118, 118, 125, 125, 134, 134, 137, 137, 141, 83, 78, 78,
75, 75, 74, 74, 72, 72, 76, 76, 81, 81, 86, 86, 92, 92, 99, 99, 106,
106, 113, 113, 121, 121, 128, 128, 137, 137, 140, 140, 144, 83, 78, 78,
75, 75, 74, 74, 72, 72, 76, 76, 81, 81, 86, 86, 92, 92, 99, 99, 106,
106, 113, 113, 121, 121, 128, 128, 137, 137, 140, 140, 144, 87, 83, 83,
79, 79, 77, 77, 75, 75, 80, 80, 84, 84, 90, 90, 96, 96, 102, 102, 109,
109, 116, 116, 124, 124, 132, 132, 141, 141, 144, 144, 149,
// Size 4x8
32, 35, 51, 75, 32, 36, 50, 71, 34, 42, 54, 73, 37, 50, 65, 84, 45, 56,
76, 96, 54, 63, 87, 110, 65, 73, 97, 125, 75, 81, 106, 136,
// Size 8x4
32, 32, 34, 37, 45, 54, 65, 75, 35, 36, 42, 50, 56, 63, 73, 81, 51, 50,
54, 65, 76, 87, 97, 106, 75, 71, 73, 84, 96, 110, 125, 136,
// Size 8x16
32, 31, 32, 36, 44, 53, 65, 79, 31, 32, 32, 35, 42, 51, 62, 75, 31, 32,
33, 34, 41, 49, 59, 72, 32, 32, 34, 36, 42, 50, 59, 71, 32, 33, 35, 38,
42, 49, 58, 69, 34, 34, 37, 42, 48, 54, 63, 73, 36, 34, 38, 48, 54, 60,
68, 78, 39, 37, 40, 50, 58, 65, 73, 84, 44, 41, 43, 53, 63, 71, 79, 90,
48, 45, 46, 56, 67, 76, 85, 96, 53, 49, 50, 60, 71, 82, 92, 103, 58, 54,
54, 63, 75, 87, 98, 110, 65, 60, 58, 68, 79, 92, 105, 118, 71, 65, 63,
73, 84, 97, 111, 125, 79, 72, 70, 79, 90, 104, 118, 133, 82, 75, 72, 81,
92, 106, 121, 136,
// Size 16x8
32, 31, 31, 32, 32, 34, 36, 39, 44, 48, 53, 58, 65, 71, 79, 82, 31, 32,
32, 32, 33, 34, 34, 37, 41, 45, 49, 54, 60, 65, 72, 75, 32, 32, 33, 34,
35, 37, 38, 40, 43, 46, 50, 54, 58, 63, 70, 72, 36, 35, 34, 36, 38, 42,
48, 50, 53, 56, 60, 63, 68, 73, 79, 81, 44, 42, 41, 42, 42, 48, 54, 58,
63, 67, 71, 75, 79, 84, 90, 92, 53, 51, 49, 50, 49, 54, 60, 65, 71, 76,
82, 87, 92, 97, 104, 106, 65, 62, 59, 59, 58, 63, 68, 73, 79, 85, 92,
98, 105, 111, 118, 121, 79, 75, 72, 71, 69, 73, 78, 84, 90, 96, 103,
110, 118, 125, 133, 136,
// Size 16x32
32, 31, 31, 32, 32, 36, 36, 44, 44, 53, 53, 65, 65, 79, 79, 87, 31, 32,
32, 32, 32, 35, 35, 42, 42, 51, 51, 62, 62, 75, 75, 82, 31, 32, 32, 32, 32,
32, 35, 35, 42, 42, 51, 51, 62, 62, 75, 75, 82, 31, 32, 32, 33, 33, 34,
34, 41, 41, 49, 49, 59, 59, 72, 72, 78, 31, 32, 32, 33, 33, 34, 34, 41,
41, 49, 49, 59, 59, 72, 72, 78, 32, 32, 32, 34, 34, 36, 36, 42, 42, 50,
50, 59, 59, 71, 71, 77, 32, 32, 32, 34, 34, 36, 36, 42, 42, 50, 50, 59,
59, 71, 71, 77, 32, 33, 33, 35, 35, 38, 38, 42, 42, 49, 49, 58, 58, 69,
69, 75, 32, 33, 33, 35, 35, 38, 38, 42, 42, 49, 49, 58, 58, 69, 69, 75,
34, 34, 34, 37, 37, 42, 42, 48, 48, 54, 54, 63, 63, 73, 73, 79, 34, 34,
34, 37, 37, 42, 42, 48, 48, 54, 54, 63, 63, 73, 73, 79, 36, 34, 34, 38,
38, 48, 48, 54, 54, 60, 60, 68, 68, 78, 78, 84, 36, 34, 34, 38, 38, 48,
48, 54, 54, 60, 60, 68, 68, 78, 78, 84, 39, 37, 37, 40, 40, 50, 50, 58,
58, 65, 65, 73, 73, 84, 84, 89, 39, 37, 37, 40, 40, 50, 50, 58, 58, 65,
65, 73, 73, 84, 84, 89, 44, 41, 41, 43, 43, 53, 53, 63, 63, 71, 71, 79,
79, 90, 90, 95, 44, 41, 41, 43, 43, 53, 53, 63, 63, 71, 71, 79, 79, 90,
90, 95, 48, 45, 45, 46, 46, 56, 56, 67, 67, 76, 76, 85, 85, 96, 96, 102,
48, 45, 45, 46, 46, 56, 56, 67, 67, 76, 76, 85, 85, 96, 96, 102, 53, 49,
49, 50, 50, 60, 60, 71, 71, 82, 82, 92, 92, 103, 103, 109, 53, 49, 49,
50, 50, 60, 60, 71, 71, 82, 82, 92, 92, 103, 103, 109, 58, 54, 54, 54,
54, 63, 63, 75, 75, 87, 87, 98, 98, 110, 110, 116, 58, 54, 54, 54, 54,
63, 63, 75, 75, 87, 87, 98, 98, 110, 110, 116, 65, 60, 60, 58, 58, 68,
68, 79, 79, 92, 92, 105, 105, 118, 118, 124, 65, 60, 60, 58, 58, 68, 68,
79, 79, 92, 92, 105, 105, 118, 118, 124, 71, 65, 65, 63, 63, 73, 73, 84,
84, 97, 97, 111, 111, 125, 125, 132, 71, 65, 65, 63, 63, 73, 73, 84, 84,
97, 97, 111, 111, 125, 125, 132, 79, 72, 72, 70, 70, 79, 79, 90, 90,
104, 104, 118, 118, 133, 133, 141, 79, 72, 72, 70, 70, 79, 79, 90, 90,
104, 104, 118, 118, 133, 133, 141, 82, 75, 75, 72, 72, 81, 81, 92, 92,
106, 106, 121, 121, 136, 136, 144, 82, 75, 75, 72, 72, 81, 81, 92, 92,
106, 106, 121, 121, 136, 136, 144, 87, 79, 79, 76, 76, 84, 84, 96, 96,

```

```

109, 109, 124, 124, 141, 141, 149,
// Size 32x16
32, 31, 31, 31, 31, 32, 32, 32, 32, 34, 34, 36, 36, 39, 39, 44, 44, 48,
48, 53, 53, 58, 58, 65, 65, 71, 71, 79, 79, 82, 82, 87, 31, 32, 32, 32,
32, 32, 32, 33, 33, 34, 34, 34, 34, 37, 37, 41, 41, 45, 45, 49, 49, 54,
54, 60, 60, 65, 65, 72, 72, 75, 75, 79, 31, 32, 32, 32, 32, 32, 32, 33,
33, 34, 34, 34, 34, 37, 37, 41, 41, 45, 45, 49, 49, 54, 54, 60, 60, 65,
65, 72, 72, 75, 75, 79, 32, 32, 32, 33, 33, 34, 34, 35, 35, 37, 37, 38,
38, 40, 40, 43, 43, 46, 46, 50, 50, 54, 54, 58, 58, 63, 63, 70, 70, 72,
72, 76, 32, 32, 32, 33, 33, 34, 34, 35, 35, 37, 37, 38, 38, 40, 40, 43,
43, 46, 46, 50, 50, 54, 54, 58, 58, 63, 63, 70, 70, 72, 72, 76, 36, 35,
35, 34, 34, 36, 36, 38, 38, 42, 42, 48, 48, 50, 50, 53, 53, 56, 56, 60,
60, 63, 63, 68, 68, 73, 73, 79, 79, 81, 81, 84, 36, 35, 35, 34, 34, 36,
36, 38, 38, 42, 42, 48, 48, 50, 50, 53, 53, 56, 56, 60, 60, 63, 63, 68,
68, 73, 73, 79, 79, 81, 81, 84, 44, 42, 42, 41, 41, 42, 42, 42, 42, 48,
48, 54, 54, 58, 58, 63, 63, 67, 67, 71, 71, 75, 75, 79, 79, 84, 84, 90,
90, 92, 92, 96, 44, 42, 42, 41, 41, 42, 42, 42, 42, 48, 48, 54, 54, 58,
58, 63, 63, 67, 67, 71, 71, 75, 75, 79, 79, 84, 84, 90, 90, 92, 92, 96,
53, 51, 51, 49, 49, 50, 50, 49, 49, 54, 54, 60, 60, 65, 65, 71, 71, 76,
76, 82, 82, 87, 87, 92, 92, 97, 97, 104, 104, 106, 106, 109, 53, 51, 51,
49, 49, 50, 50, 49, 49, 54, 54, 60, 60, 65, 65, 71, 71, 76, 76, 82, 82,
87, 87, 92, 92, 97, 97, 104, 104, 106, 106, 109, 65, 62, 62, 59, 59, 59,
59, 58, 58, 63, 63, 68, 68, 73, 73, 79, 79, 85, 85, 92, 92, 98, 98, 105,
105, 111, 111, 118, 118, 121, 121, 124, 65, 62, 62, 59, 59, 59, 59, 58,
58, 63, 63, 68, 68, 73, 73, 79, 79, 85, 85, 92, 92, 98, 98, 105, 105,
111, 111, 118, 118, 121, 121, 124, 79, 75, 75, 72, 72, 71, 71, 69, 69,
73, 73, 78, 78, 84, 84, 90, 90, 96, 96, 103, 103, 110, 110, 118, 118,
125, 125, 133, 133, 136, 136, 141, 79, 75, 75, 72, 72, 71, 71, 69, 69,
73, 73, 78, 78, 84, 84, 90, 90, 96, 96, 103, 103, 110, 110, 118, 118,
125, 125, 133, 133, 136, 136, 141, 87, 82, 82, 78, 78, 77, 77, 75, 75,
79, 79, 84, 84, 89, 89, 95, 95, 102, 102, 109, 109, 116, 116, 124, 124,
132, 132, 141, 141, 144, 144, 149,
// Size 4x16
31, 36, 53, 79, 32, 35, 51, 75, 32, 34, 49, 72, 32, 36, 50, 71, 33, 38,
49, 69, 34, 42, 54, 73, 34, 48, 60, 78, 37, 50, 65, 84, 41, 53, 71, 90,
45, 56, 76, 96, 49, 60, 82, 103, 54, 63, 87, 110, 60, 68, 92, 118, 65,
73, 97, 125, 72, 79, 104, 133, 75, 81, 106, 136,
// Size 16x4
31, 32, 32, 32, 33, 34, 34, 37, 41, 45, 49, 54, 60, 65, 72, 75, 36, 35,
34, 36, 38, 42, 48, 50, 53, 56, 60, 63, 68, 73, 79, 81, 53, 51, 49, 50,
49, 54, 60, 65, 71, 76, 82, 87, 92, 97, 104, 106, 79, 75, 72, 71, 69,
73, 78, 84, 90, 96, 103, 110, 118, 125, 133, 136,
// Size 8x32
32, 31, 32, 36, 44, 53, 65, 79, 31, 32, 32, 35, 42, 51, 62, 75, 31, 32,
32, 35, 42, 51, 62, 75, 31, 32, 33, 34, 41, 49, 59, 72, 31, 32, 33, 34,
41, 49, 59, 72, 32, 32, 34, 36, 42, 50, 59, 71, 32, 32, 34, 36, 42, 50,
59, 71, 32, 33, 35, 38, 42, 49, 58, 69, 32, 33, 35, 38, 42, 49, 58, 69,
34, 34, 37, 42, 48, 54, 63, 73, 34, 34, 37, 42, 48, 54, 63, 73, 36, 34,
38, 48, 54, 60, 68, 78, 36, 34, 38, 48, 54, 60, 68, 78, 39, 37, 40, 50,
58, 65, 73, 84, 39, 37, 40, 50, 58, 65, 73, 84, 44, 41, 43, 53, 63, 71,
79, 90, 44, 41, 43, 53, 63, 71, 79, 90, 48, 45, 46, 56, 67, 76, 85, 96,
48, 45, 46, 56, 67, 76, 85, 96, 53, 49, 50, 60, 71, 82, 92, 103, 53, 49,
50, 60, 71, 82, 92, 103, 58, 54, 54, 63, 75, 87, 98, 110, 58, 54, 54,
63, 75, 87, 98, 110, 65, 60, 58, 68, 79, 92, 105, 118, 65, 60, 58, 68,
79, 92, 105, 118, 71, 65, 63, 73, 84, 97, 111, 125, 71, 65, 63, 73, 84,
97, 111, 125, 79, 72, 70, 79, 90, 104, 118, 133, 79, 72, 70, 79, 90,
104, 118, 133, 82, 75, 72, 81, 92, 106, 121, 136, 82, 75, 72, 81, 92,
106, 121, 136, 87, 79, 76, 84, 96, 109, 124, 141,
// Size 32x8
32, 31, 31, 31, 31, 32, 32, 32, 32, 34, 34, 36, 36, 39, 39, 44, 44, 48,
48, 53, 53, 58, 58, 65, 65, 71, 71, 79, 79, 82, 82, 87, 31, 32, 32, 32,
32, 32, 32, 33, 33, 34, 34, 34, 34, 37, 37, 41, 41, 45, 45, 49, 49, 54,
54, 60, 60, 65, 65, 72, 72, 75, 75, 79, 32, 32, 32, 33, 33, 34, 34, 35,
35, 37, 37, 38, 38, 40, 40, 43, 43, 46, 46, 50, 50, 54, 54, 58, 58, 63,
63, 70, 70, 72, 72, 76, 36, 35, 35, 34, 34, 36, 36, 38, 38, 42, 42, 48,
48, 50, 50, 53, 53, 56, 56, 60, 60, 63, 63, 68, 68, 73, 73, 79, 79, 81,
81, 84, 44, 42, 42, 41, 41, 42, 42, 42, 42, 48, 48, 54, 54, 58, 58, 63,
63, 67, 67, 71, 71, 75, 75, 79, 79, 84, 84, 90, 90, 92, 92, 96, 53, 51,
51, 49, 49, 50, 50, 49, 49, 54, 54, 60, 60, 65, 65, 71, 71, 76, 76, 82,
82, 87, 87, 92, 92, 97, 97, 104, 104, 106, 106, 109, 65, 62, 62, 59, 59,
59, 59, 58, 58, 63, 63, 68, 68, 73, 73, 79, 79, 85, 85, 92, 92, 98, 98,

```

```

105, 105, 111, 111, 118, 118, 121, 121, 124, 79, 75, 75, 72, 72, 71, 71,
69, 69, 73, 73, 78, 78, 84, 84, 90, 90, 96, 96, 103, 103, 110, 110, 118,
118, 125, 125, 133, 133, 136, 136, 141,
// Size 4x32
31, 36, 53, 79, 32, 35, 51, 75, 32, 35, 51, 75, 32, 34, 49, 72, 32, 34,
49, 72, 32, 36, 50, 71, 32, 36, 50, 71, 33, 38, 49, 69, 33, 38, 49, 69,
34, 42, 54, 73, 34, 42, 54, 73, 34, 48, 60, 78, 34, 48, 60, 78, 37, 50,
65, 84, 37, 50, 65, 84, 41, 53, 71, 90, 41, 53, 71, 90, 45, 56, 76, 96,
45, 56, 76, 96, 49, 60, 82, 103, 49, 60, 82, 103, 54, 63, 87, 110, 54,
63, 87, 110, 60, 68, 92, 118, 60, 68, 92, 118, 65, 73, 97, 125, 65, 73,
97, 125, 72, 79, 104, 133, 72, 79, 104, 133, 75, 81, 106, 136, 75, 81,
106, 136, 79, 84, 109, 141,
// Size 32x4
31, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34, 34, 37, 37, 41, 41, 45,
45, 49, 49, 54, 54, 60, 60, 65, 65, 72, 72, 75, 75, 79, 36, 35, 35, 34,
34, 36, 36, 38, 38, 42, 42, 48, 48, 50, 50, 53, 53, 56, 56, 60, 60, 63,
63, 68, 68, 73, 73, 79, 79, 81, 81, 84, 53, 51, 51, 49, 49, 50, 50, 49,
49, 54, 54, 60, 60, 65, 65, 71, 71, 76, 76, 82, 82, 87, 87, 92, 92, 97,
97, 104, 104, 106, 106, 109, 79, 75, 75, 72, 72, 71, 71, 69, 69, 73, 73,
78, 78, 84, 84, 90, 90, 96, 96, 103, 103, 110, 110, 118, 118, 125, 125,
133, 133, 136, 136, 141,
},
{ // Chroma
// Size 4x4
32, 46, 47, 57, 46, 53, 54, 60, 47, 54, 66, 75, 57, 60, 75, 89,
// Size 8x8
31, 34, 42, 47, 48, 52, 57, 61, 34, 39, 45, 46, 46, 49, 53, 57, 42, 45,
48, 49, 50, 52, 55, 58, 47, 46, 49, 54, 56, 58, 61, 64, 48, 46, 50, 56,
61, 65, 68, 71, 52, 49, 52, 58, 65, 71, 75, 79, 57, 53, 55, 61, 68, 75,
82, 86, 61, 57, 58, 64, 71, 79, 86, 91,
// Size 16x16
32, 31, 30, 33, 36, 41, 49, 48, 49, 50, 52, 54, 57, 60, 63, 65, 31, 31,
31, 34, 38, 42, 47, 47, 47, 48, 50, 52, 54, 57, 60, 61, 30, 31, 32, 35,
40, 42, 46, 45, 45, 46, 47, 49, 52, 54, 57, 58, 33, 34, 35, 39, 43, 45,
47, 46, 45, 46, 47, 49, 51, 53, 56, 57, 36, 38, 40, 43, 47, 47, 48, 46,
45, 46, 47, 48, 50, 52, 54, 55, 41, 42, 42, 45, 47, 48, 50, 49, 49, 50,
50, 52, 53, 55, 57, 58, 49, 47, 46, 47, 48, 50, 53, 53, 53, 54, 54, 55,
56, 58, 60, 61, 48, 47, 45, 46, 46, 49, 53, 54, 55, 56, 57, 58, 60, 61,
63, 64, 49, 47, 45, 45, 45, 49, 53, 55, 58, 60, 61, 62, 63, 65, 67, 68,
50, 48, 46, 46, 46, 50, 54, 56, 60, 61, 63, 65, 67, 68, 71, 71, 52, 50,
47, 47, 47, 50, 54, 57, 61, 63, 66, 68, 70, 72, 75, 75, 54, 52, 49, 49,
48, 52, 55, 58, 62, 65, 68, 71, 73, 75, 78, 79, 57, 54, 52, 51, 50, 53,
56, 60, 63, 67, 70, 73, 76, 79, 82, 83, 60, 57, 54, 53, 52, 55, 58, 61,
65, 68, 72, 75, 79, 82, 85, 86, 63, 60, 57, 56, 54, 57, 60, 63, 67, 71,
75, 78, 82, 85, 89, 90, 65, 61, 58, 57, 55, 58, 61, 64, 68, 71, 75, 79,
83, 86, 90, 91,
// Size 32x32
32, 31, 31, 30, 30, 33, 33, 36, 36, 41, 41, 49, 49, 48, 48, 49, 49, 50,
50, 52, 52, 54, 54, 57, 57, 60, 60, 63, 63, 65, 65, 67, 31, 31, 31, 31,
31, 34, 34, 38, 38, 42, 42, 47, 47, 47, 47, 47, 47, 48, 48, 50, 50, 52,
52, 54, 54, 57, 57, 60, 60, 61, 61, 63, 31, 31, 31, 31, 34, 34, 38,
38, 42, 42, 47, 47, 47, 47, 47, 47, 48, 48, 50, 50, 52, 52, 54, 54, 57,
57, 60, 60, 61, 61, 63, 30, 31, 31, 32, 32, 35, 35, 40, 40, 42, 42, 46,
46, 45, 45, 45, 45, 46, 46, 47, 47, 49, 49, 52, 52, 54, 54, 57, 57, 58,
58, 60, 30, 31, 31, 32, 32, 35, 35, 40, 40, 42, 42, 46, 46, 45, 45, 45,
45, 46, 46, 47, 47, 49, 49, 52, 52, 54, 54, 57, 57, 58, 60, 33, 34,
34, 35, 35, 39, 39, 43, 43, 45, 45, 47, 47, 46, 46, 45, 45, 46, 46, 47,
47, 49, 49, 51, 51, 53, 53, 56, 56, 57, 57, 59, 33, 34, 34, 35, 35, 39,
39, 43, 43, 45, 45, 47, 47, 46, 46, 45, 45, 46, 46, 47, 47, 49, 49, 51,
51, 53, 53, 56, 56, 57, 57, 59, 36, 38, 38, 40, 40, 43, 43, 47, 47, 47,
47, 48, 48, 46, 46, 45, 45, 46, 46, 47, 47, 48, 48, 50, 50, 52, 52, 54,
54, 55, 55, 57, 36, 38, 38, 40, 40, 43, 43, 47, 47, 47, 47, 48, 48, 46,
46, 45, 45, 46, 46, 47, 47, 48, 48, 50, 50, 52, 52, 54, 54, 55, 55, 57,
41, 42, 42, 42, 42, 45, 45, 47, 47, 48, 48, 50, 50, 49, 49, 49, 49, 50,
50, 50, 50, 52, 52, 53, 53, 55, 55, 57, 57, 58, 58, 60, 41, 42, 42, 42,
42, 45, 45, 47, 47, 48, 48, 50, 50, 49, 49, 49, 49, 50, 50, 50, 52,
52, 53, 53, 55, 55, 57, 57, 58, 58, 60, 49, 47, 47, 46, 46, 47, 47, 48,
48, 50, 50, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 55, 55, 56, 56, 58,
58, 60, 60, 61, 61, 62, 49, 47, 47, 46, 46, 47, 47, 48, 48, 50, 50, 53,
53, 53, 53, 53, 54, 54, 54, 54, 55, 55, 56, 56, 58, 58, 60, 60, 61,
61, 62, 48, 47, 47, 45, 45, 46, 46, 46, 46, 49, 49, 53, 53, 54, 54, 55,

```

55, 56, 56, 57, 57, 58, 58, 60, 60, 61, 61, 63, 63, 64, 64, 66, 48, 47,
47, 45, 45, 46, 46, 46, 46, 49, 49, 53, 53, 54, 54, 55, 55, 56, 56, 57,
57, 58, 58, 60, 60, 61, 61, 63, 63, 64, 64, 66, 49, 47, 47, 45, 45, 45,
45, 45, 45, 49, 49, 53, 53, 55, 55, 58, 58, 60, 60, 61, 61, 62, 62, 63,
63, 65, 65, 67, 67, 68, 68, 69, 49, 47, 47, 45, 45, 45, 45, 45, 49,
49, 53, 53, 55, 55, 58, 58, 60, 60, 61, 61, 62, 62, 63, 63, 65, 65, 67,
67, 68, 68, 69, 50, 48, 48, 46, 46, 46, 46, 46, 46, 50, 50, 54, 54, 56,
56, 60, 60, 61, 61, 63, 63, 65, 65, 67, 67, 68, 68, 71, 71, 71, 71, 72,
50, 48, 48, 46, 46, 46, 46, 46, 46, 50, 50, 54, 54, 56, 56, 60, 60, 61,
61, 63, 63, 65, 65, 67, 67, 68, 68, 71, 71, 71, 71, 72, 52, 50, 50, 47,
47, 47, 47, 47, 47, 50, 50, 54, 54, 57, 57, 61, 61, 63, 63, 66, 66, 68,
68, 70, 70, 72, 72, 75, 75, 75, 75, 76, 52, 50, 50, 47, 47, 47, 47, 47,
47, 50, 50, 54, 54, 57, 57, 61, 61, 63, 63, 66, 66, 68, 68, 70, 70, 72,
72, 75, 75, 75, 75, 76, 54, 52, 52, 49, 49, 49, 49, 48, 48, 52, 52, 55,
55, 58, 58, 62, 62, 65, 65, 68, 68, 71, 71, 73, 73, 75, 75, 78, 78, 79,
79, 80, 54, 52, 52, 49, 49, 49, 49, 48, 48, 52, 52, 55, 55, 58, 58, 62,
62, 65, 65, 68, 68, 71, 71, 73, 73, 75, 75, 78, 78, 79, 79, 80, 57, 54,
54, 52, 52, 51, 51, 50, 50, 53, 53, 56, 56, 60, 60, 63, 63, 67, 67, 70,
70, 73, 73, 76, 76, 79, 79, 82, 82, 83, 83, 84, 57, 54, 54, 52, 52, 51,
51, 50, 50, 53, 53, 56, 56, 60, 60, 63, 63, 67, 67, 70, 70, 73, 73, 76,
76, 79, 79, 82, 82, 83, 83, 84, 60, 57, 57, 54, 54, 53, 53, 52, 52, 55,
55, 58, 58, 61, 61, 65, 65, 68, 68, 72, 72, 75, 75, 79, 79, 82, 82, 85,
85, 86, 86, 88, 60, 57, 57, 54, 54, 53, 53, 52, 52, 55, 55, 58, 58, 61,
61, 65, 65, 68, 68, 72, 72, 75, 75, 79, 79, 82, 82, 85, 85, 86, 86, 88,
63, 60, 60, 57, 57, 56, 56, 54, 54, 57, 57, 60, 60, 63, 63, 67, 67, 71,
71, 75, 75, 78, 78, 82, 82, 85, 85, 89, 89, 90, 90, 92, 63, 60, 60, 57,
57, 56, 56, 54, 54, 57, 57, 60, 60, 63, 63, 67, 67, 71, 71, 75, 75, 78,
78, 82, 82, 85, 85, 89, 89, 90, 90, 92, 65, 61, 61, 58, 58, 57, 57, 55,
55, 58, 58, 61, 61, 64, 64, 68, 68, 71, 71, 75, 75, 79, 79, 83, 83, 86,
86, 90, 90, 91, 91, 93, 65, 61, 61, 58, 58, 57, 57, 55, 55, 58, 58, 61,
61, 64, 64, 68, 68, 71, 71, 75, 75, 79, 79, 83, 83, 86, 86, 90, 90, 91,
91, 93, 67, 63, 63, 60, 60, 59, 59, 57, 57, 60, 60, 62, 62, 66, 66, 69,
69, 72, 72, 76, 76, 80, 80, 84, 84, 88, 88, 92, 92, 93, 93, 95,
// Size 4x8
31, 47, 50, 60, 36, 47, 47, 56, 43, 50, 50, 57, 46, 53, 57, 64, 46, 54,
64, 71, 50, 55, 68, 78, 54, 58, 72, 85, 59, 61, 75, 90,
// Size 8x4
31, 36, 43, 46, 46, 50, 54, 59, 47, 47, 50, 53, 54, 55, 58, 61, 50, 47,
50, 57, 64, 68, 72, 75, 60, 56, 57, 64, 71, 78, 85, 90,
// Size 8x16
32, 31, 37, 48, 49, 52, 57, 63, 31, 31, 38, 47, 47, 50, 54, 60, 30, 32,
40, 46, 45, 48, 52, 57, 33, 36, 43, 47, 46, 47, 51, 56, 37, 40, 47, 47,
45, 47, 50, 54, 42, 43, 47, 50, 49, 50, 53, 57, 49, 46, 48, 53, 53, 54,
57, 60, 48, 46, 47, 53, 56, 57, 60, 64, 49, 45, 46, 53, 58, 61, 64, 67,
50, 46, 46, 54, 59, 64, 67, 71, 52, 48, 47, 54, 61, 66, 71, 75, 54, 50,
49, 55, 62, 68, 73, 78, 57, 52, 50, 56, 64, 70, 76, 82, 60, 54, 52, 58,
65, 72, 79, 85, 63, 57, 55, 60, 67, 75, 82, 89, 64, 59, 56, 61, 68, 75,
83, 90,
// Size 16x8
32, 31, 30, 33, 37, 42, 49, 48, 49, 50, 52, 54, 57, 60, 63, 64, 31, 31,
32, 36, 40, 43, 46, 46, 45, 46, 48, 50, 52, 54, 57, 59, 37, 38, 40, 43,
47, 47, 48, 47, 46, 46, 47, 49, 50, 52, 55, 56, 48, 47, 46, 47, 47, 50,
53, 53, 53, 54, 54, 55, 56, 58, 60, 61, 49, 47, 45, 46, 45, 49, 53, 56,
58, 59, 61, 62, 64, 65, 67, 68, 52, 50, 48, 47, 47, 50, 54, 57, 61, 64,
66, 68, 70, 72, 75, 75, 57, 54, 52, 51, 50, 53, 57, 60, 64, 67, 71, 73,
76, 79, 82, 83, 63, 60, 57, 56, 54, 57, 60, 64, 67, 71, 75, 78, 82, 85,
89, 90,
// Size 16x32
32, 31, 31, 37, 37, 48, 48, 49, 49, 52, 52, 57, 57, 63, 63, 66, 31, 31,
31, 38, 38, 47, 47, 47, 47, 50, 50, 54, 54, 60, 60, 63, 31, 31, 31, 38,
38, 47, 47, 47, 47, 50, 50, 54, 54, 60, 60, 63, 30, 32, 32, 40, 40, 46,
46, 45, 45, 48, 48, 52, 52, 57, 57, 60, 30, 32, 32, 40, 40, 46, 46, 45,
45, 48, 48, 52, 52, 57, 57, 60, 33, 36, 36, 43, 43, 47, 47, 46, 46, 47,
47, 51, 51, 56, 56, 59, 33, 36, 36, 43, 43, 47, 47, 46, 46, 47, 47, 51,
51, 56, 56, 59, 37, 40, 40, 47, 47, 47, 47, 45, 45, 47, 47, 50, 50, 54,
54, 57, 37, 40, 40, 47, 47, 47, 45, 45, 47, 47, 50, 50, 54, 54, 57,
42, 43, 43, 47, 47, 50, 50, 49, 49, 50, 50, 53, 53, 57, 57, 60, 42, 43,
43, 47, 47, 50, 50, 49, 49, 50, 50, 53, 53, 57, 57, 60, 49, 46, 46, 48,
48, 53, 53, 53, 53, 54, 54, 57, 57, 60, 60, 62, 49, 46, 46, 48, 48, 53,
53, 53, 53, 54, 54, 57, 57, 60, 60, 62, 48, 46, 46, 47, 47, 53, 53, 56,
56, 57, 57, 60, 60, 64, 64, 66, 48, 46, 46, 47, 47, 53, 53, 56, 56, 57,

```

57, 60, 60, 64, 64, 66, 49, 45, 45, 46, 46, 53, 53, 58, 58, 61, 61, 64,
64, 67, 67, 69, 49, 45, 45, 46, 46, 53, 53, 58, 58, 61, 61, 64, 64, 67,
67, 69, 50, 46, 46, 46, 46, 54, 54, 59, 59, 64, 64, 67, 67, 71, 71, 73,
50, 46, 46, 46, 46, 54, 54, 59, 59, 64, 64, 67, 67, 71, 71, 73, 52, 48,
48, 47, 47, 54, 54, 61, 61, 66, 66, 71, 71, 75, 75, 77, 52, 48, 48, 47,
47, 54, 54, 61, 61, 66, 66, 71, 71, 75, 75, 77, 54, 50, 50, 49, 49, 55,
55, 62, 62, 68, 68, 73, 73, 78, 78, 80, 54, 50, 50, 49, 49, 55, 55, 62,
62, 68, 68, 73, 73, 78, 78, 80, 57, 52, 52, 50, 50, 56, 56, 64, 64, 70,
70, 76, 76, 82, 82, 84, 57, 52, 52, 50, 50, 56, 56, 64, 64, 70, 70, 76,
76, 82, 82, 84, 60, 54, 54, 52, 52, 58, 58, 65, 65, 72, 72, 79, 79, 85,
85, 88, 60, 54, 54, 52, 52, 58, 58, 65, 65, 72, 72, 79, 79, 85, 85, 88,
63, 57, 57, 55, 55, 60, 60, 67, 67, 75, 75, 82, 82, 89, 89, 92, 63, 57,
57, 55, 55, 60, 60, 67, 67, 75, 75, 82, 82, 89, 89, 92, 64, 59, 59, 56,
56, 61, 61, 68, 68, 75, 75, 83, 83, 90, 90, 93, 64, 59, 59, 56, 56, 61,
61, 68, 68, 75, 75, 83, 83, 90, 90, 93, 66, 60, 60, 57, 57, 63, 63, 69,
69, 77, 77, 84, 84, 92, 92, 95,
// Size 32x16
32, 31, 31, 30, 30, 33, 33, 37, 37, 42, 42, 49, 49, 48, 48, 49, 49, 50,
50, 52, 52, 54, 54, 57, 57, 60, 60, 63, 63, 64, 64, 66, 31, 31, 31, 32,
32, 36, 36, 40, 40, 43, 43, 46, 46, 46, 46, 45, 45, 46, 46, 48, 48, 50,
50, 52, 52, 54, 54, 57, 57, 59, 59, 60, 31, 31, 31, 32, 32, 36, 36, 40,
40, 43, 43, 46, 46, 46, 46, 45, 45, 46, 46, 48, 48, 50, 50, 52, 52, 54,
54, 57, 57, 59, 59, 60, 37, 38, 38, 40, 40, 43, 43, 47, 47, 47, 47, 48,
48, 47, 47, 46, 46, 46, 46, 47, 47, 49, 49, 50, 50, 52, 52, 55, 55, 56,
56, 57, 37, 38, 38, 40, 40, 43, 43, 47, 47, 47, 47, 48, 48, 47, 47, 46,
46, 46, 46, 47, 47, 49, 49, 50, 50, 52, 52, 55, 55, 56, 56, 57, 48, 47,
47, 46, 46, 47, 47, 47, 47, 50, 50, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 55, 54, 54,
54, 55, 55, 56, 56, 58, 58, 60, 60, 61, 61, 63, 48, 47, 47, 46, 46, 47,
47, 47, 47, 50, 50, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 55, 55, 56,
56, 58, 58, 60, 60, 61, 61, 63, 49, 47, 47, 45, 45, 46, 46, 45, 45, 49,
49, 53, 53, 56, 56, 58, 58, 59, 59, 61, 61, 62, 62, 64, 64, 65, 65, 67,
67, 68, 68, 69, 49, 47, 47, 45, 45, 46, 46, 45, 45, 49, 49, 53, 53, 56,
56, 58, 58, 59, 59, 61, 61, 62, 62, 64, 64, 65, 65, 67, 67, 68, 68, 69,
52, 50, 50, 48, 48, 47, 47, 47, 47, 50, 50, 54, 54, 57, 57, 61, 61, 64,
64, 66, 66, 68, 68, 70, 70, 72, 72, 75, 75, 75, 75, 77, 52, 50, 50, 48,
48, 47, 47, 47, 47, 50, 50, 54, 54, 57, 57, 61, 61, 64, 64, 66, 66, 68,
68, 70, 70, 72, 72, 75, 75, 75, 75, 77, 57, 54, 54, 52, 52, 51, 51, 50,
50, 53, 53, 57, 57, 60, 60, 64, 64, 67, 67, 71, 71, 73, 73, 76, 76, 79,
79, 82, 82, 83, 83, 84, 57, 54, 54, 52, 52, 51, 51, 50, 50, 53, 53, 57,
57, 60, 60, 64, 64, 67, 67, 71, 71, 73, 73, 76, 76, 79, 79, 82, 82, 83,
83, 84, 63, 60, 60, 57, 57, 56, 56, 54, 54, 57, 57, 60, 60, 64, 64, 67,
67, 71, 71, 75, 75, 78, 78, 82, 82, 85, 85, 89, 89, 90, 90, 92, 63, 60,
60, 57, 57, 56, 56, 54, 54, 57, 57, 60, 60, 64, 64, 67, 67, 71, 71, 75,
75, 78, 78, 82, 82, 85, 85, 89, 89, 90, 90, 92, 66, 63, 63, 60, 60, 59,
59, 57, 57, 60, 60, 62, 62, 66, 66, 69, 69, 73, 73, 77, 77, 80, 80, 84,
84, 88, 88, 92, 92, 93, 93, 95,
// Size 4x16
31, 48, 52, 63, 31, 47, 50, 60, 32, 46, 48, 57, 36, 47, 47, 56, 40, 47,
47, 54, 43, 50, 50, 57, 46, 53, 54, 60, 46, 53, 57, 64, 45, 53, 61, 67,
46, 54, 64, 71, 48, 54, 66, 75, 50, 55, 68, 78, 52, 56, 70, 82, 54, 58,
72, 85, 57, 60, 75, 89, 59, 61, 75, 90,
// Size 16x4
31, 31, 32, 36, 40, 43, 46, 46, 45, 46, 48, 50, 52, 54, 57, 59, 48, 47,
46, 47, 47, 50, 53, 53, 53, 54, 54, 55, 56, 58, 60, 61, 52, 50, 48, 47,
47, 50, 54, 57, 61, 64, 66, 68, 70, 72, 75, 75, 63, 60, 57, 56, 54, 57,
60, 64, 67, 71, 75, 78, 82, 85, 89, 90,
// Size 8x32
32, 31, 37, 48, 49, 52, 57, 63, 31, 31, 38, 47, 47, 50, 54, 60, 31, 31,
38, 47, 47, 50, 54, 60, 30, 32, 40, 46, 45, 48, 52, 57, 30, 32, 40, 46,
45, 48, 52, 57, 33, 36, 43, 47, 46, 47, 51, 56, 33, 36, 43, 47, 46, 47,
51, 56, 37, 40, 47, 47, 45, 47, 50, 54, 37, 40, 47, 47, 45, 47, 50, 54,
42, 43, 47, 50, 49, 50, 53, 57, 42, 43, 47, 50, 49, 50, 53, 57, 49, 46,
48, 53, 53, 54, 57, 60, 49, 46, 48, 53, 53, 54, 57, 60, 48, 46, 47, 53,
56, 57, 60, 64, 48, 46, 47, 53, 56, 57, 60, 64, 49, 45, 46, 53, 58, 61,
64, 67, 49, 45, 46, 53, 58, 61, 64, 67, 50, 46, 46, 54, 59, 64, 67, 71,
50, 46, 46, 54, 59, 64, 67, 71, 52, 48, 47, 54, 61, 66, 71, 75, 52, 48,
47, 54, 61, 66, 71, 75, 54, 50, 49, 55, 62, 68, 73, 78, 54, 50, 49, 55,
62, 68, 73, 78, 57, 52, 50, 56, 64, 70, 76, 82, 57, 52, 50, 56, 64, 70,
76, 82, 60, 54, 52, 58, 65, 72, 79, 85, 60, 54, 52, 58, 65, 72, 79, 85,
63, 57, 55, 60, 67, 75, 82, 89, 63, 57, 55, 60, 67, 75, 82, 89, 64, 59,
56, 61, 68, 75, 83, 90, 64, 59, 56, 61, 68, 75, 83, 90, 66, 60, 57, 63,

```

```

69, 77, 84, 92,
// Size 32x8
32, 31, 31, 30, 30, 33, 33, 37, 37, 42, 42, 49, 49, 48, 48, 49, 49, 50,
50, 52, 52, 54, 54, 57, 57, 60, 60, 63, 63, 64, 64, 66, 31, 31, 31, 32,
32, 36, 36, 40, 40, 43, 43, 46, 46, 46, 46, 45, 45, 46, 46, 48, 48, 50,
50, 52, 52, 54, 54, 57, 57, 59, 59, 60, 37, 38, 38, 40, 40, 43, 43, 47,
47, 47, 47, 48, 48, 47, 47, 46, 46, 46, 46, 47, 47, 49, 49, 50, 50, 52,
52, 55, 55, 56, 56, 57, 48, 47, 47, 46, 46, 47, 47, 47, 47, 50, 50, 53,
53, 53, 53, 53, 53, 54, 54, 54, 54, 55, 55, 56, 56, 58, 58, 60, 60, 61,
61, 63, 49, 47, 47, 45, 45, 46, 46, 45, 45, 49, 49, 53, 53, 56, 56, 58,
58, 59, 59, 61, 61, 62, 62, 64, 64, 65, 65, 67, 67, 68, 68, 69, 52, 50,
50, 48, 48, 47, 47, 47, 47, 50, 50, 54, 54, 57, 57, 61, 61, 64, 64, 66,
66, 68, 68, 70, 70, 72, 72, 75, 75, 75, 75, 77, 57, 54, 54, 52, 52, 51,
51, 50, 50, 53, 53, 57, 57, 60, 60, 64, 64, 67, 67, 71, 71, 73, 73, 76,
76, 79, 79, 82, 82, 83, 83, 84, 63, 60, 60, 57, 57, 56, 56, 54, 54, 57,
57, 60, 60, 64, 64, 67, 67, 71, 71, 75, 75, 78, 78, 82, 82, 85, 85, 89,
89, 90, 90, 92,
// Size 4x32
31, 48, 52, 63, 31, 47, 50, 60, 31, 47, 50, 60, 32, 46, 48, 57, 32, 46,
48, 57, 36, 47, 47, 56, 36, 47, 47, 56, 40, 47, 47, 54, 40, 47, 47, 54,
43, 50, 50, 57, 43, 50, 50, 57, 46, 53, 54, 60, 46, 53, 54, 60, 46, 53,
57, 64, 46, 53, 57, 64, 45, 53, 61, 67, 45, 53, 61, 67, 46, 54, 64, 71,
46, 54, 64, 71, 48, 54, 66, 75, 48, 54, 66, 75, 50, 55, 68, 78, 50, 55,
68, 78, 52, 56, 70, 82, 52, 56, 70, 82, 54, 58, 72, 85, 54, 58, 72, 85,
57, 60, 75, 89, 57, 60, 75, 89, 59, 61, 75, 90, 59, 61, 75, 90, 60, 63,
77, 92,
// Size 32x4
31, 31, 31, 32, 32, 36, 36, 40, 40, 43, 43, 46, 46, 46, 46, 45, 45, 46,
46, 48, 48, 50, 50, 52, 52, 54, 54, 57, 57, 59, 59, 60, 48, 47, 47, 46,
46, 47, 47, 47, 47, 50, 50, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 55,
55, 56, 56, 58, 58, 60, 60, 61, 61, 63, 52, 50, 50, 48, 48, 47, 47, 47,
47, 50, 50, 54, 54, 57, 57, 61, 61, 64, 64, 66, 66, 68, 68, 70, 70, 72,
72, 75, 75, 75, 75, 77, 63, 60, 60, 57, 57, 56, 56, 54, 54, 57, 57, 60,
60, 64, 64, 67, 67, 71, 71, 75, 75, 78, 78, 82, 82, 85, 85, 89, 89, 90,
90, 92,

```

```

},

```

```

{

```

```

{ // Luma
// Size 4x4
32, 33, 45, 62, 33, 39, 51, 64, 45, 51, 71, 87, 62, 64, 87, 108,
// Size 8x8
31, 32, 32, 35, 42, 51, 59, 69, 32, 32, 33, 35, 41, 49, 56, 65, 32, 33,
35, 38, 43, 49, 56, 64, 35, 35, 38, 48, 54, 59, 66, 73, 42, 41, 43, 54,
63, 71, 77, 85, 51, 49, 49, 59, 71, 81, 89, 97, 59, 56, 56, 66, 77, 89,
98, 108, 69, 65, 64, 73, 85, 97, 108, 119,
// Size 16x16
32, 31, 31, 31, 32, 34, 35, 38, 41, 45, 48, 54, 59, 65, 71, 80, 31, 32,
32, 32, 32, 34, 35, 37, 40, 43, 46, 51, 56, 62, 68, 76, 31, 32, 32, 32,
32, 33, 34, 36, 38, 41, 44, 49, 54, 59, 65, 72, 31, 32, 32, 33, 34, 35,
36, 38, 40, 42, 45, 50, 54, 59, 64, 71, 32, 32, 32, 34, 35, 37, 38, 39,
41, 43, 46, 49, 53, 58, 63, 69, 34, 34, 33, 35, 37, 39, 42, 44, 46, 48,
51, 54, 58, 63, 68, 74, 35, 35, 34, 36, 38, 42, 46, 48, 50, 53, 55, 59,
62, 67, 72, 78, 38, 37, 36, 38, 39, 44, 48, 51, 54, 57, 59, 63, 67, 71,
76, 82, 41, 40, 38, 40, 41, 46, 50, 54, 57, 60, 63, 67, 71, 75, 80, 86,
45, 43, 41, 42, 43, 48, 53, 57, 60, 65, 68, 72, 76, 81, 85, 91, 48, 46,
44, 45, 46, 51, 55, 59, 63, 68, 71, 76, 80, 85, 90, 96, 54, 51, 49, 50,
49, 54, 59, 63, 67, 72, 76, 82, 87, 92, 97, 104, 59, 56, 54, 54, 53, 58,
62, 67, 71, 76, 80, 87, 92, 98, 103, 110, 65, 62, 59, 59, 58, 63, 67,
71, 75, 81, 85, 92, 98, 105, 111, 118, 71, 68, 65, 64, 63, 68, 72, 76,
80, 85, 90, 97, 103, 111, 117, 125, 80, 76, 72, 71, 69, 74, 78, 82, 86,
91, 96, 104, 110, 118, 125, 134,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 32, 32, 32, 34, 34, 35, 36, 38, 39, 41, 44,
45, 48, 48, 53, 54, 57, 59, 62, 65, 67, 71, 72, 80, 80, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 34, 34, 35, 35, 37, 38, 40, 42, 43, 46, 46, 51,
52, 55, 56, 59, 62, 64, 68, 69, 76, 76, 31, 32, 32, 32, 32, 32, 32, 32,
32, 32, 34, 34, 35, 35, 37, 38, 40, 42, 43, 46, 46, 51, 51, 55, 56, 59,
62, 64, 68, 69, 76, 76, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33,
34, 34, 36, 38, 39, 41, 42, 45, 45, 49, 50, 53, 54, 57, 60, 62, 66, 66,
73, 73, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 36, 37,

```

38, 41, 41, 44, 44, 49, 49, 52, 54, 56, 59, 61, 65, 65, 72, 72, 31, 32,
 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 35, 35, 37, 38, 39, 41, 42, 45,
 45, 49, 49, 52, 54, 56, 59, 61, 64, 65, 72, 72, 31, 32, 32, 32, 32, 33,
 33, 33, 34, 34, 35, 35, 36, 36, 38, 39, 40, 42, 42, 45, 45, 49, 50, 52,
 54, 56, 59, 60, 64, 65, 71, 71, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34,
 35, 35, 36, 37, 38, 39, 40, 42, 43, 45, 45, 49, 49, 52, 54, 56, 59, 60,
 64, 64, 70, 70, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 37, 37, 38, 38,
 39, 40, 41, 42, 43, 46, 46, 49, 49, 52, 53, 55, 58, 59, 63, 63, 69, 69,
 32, 32, 32, 32, 33, 33, 34, 34, 35, 35, 37, 37, 38, 38, 40, 41, 41, 43,
 43, 46, 46, 49, 50, 52, 54, 56, 58, 60, 63, 64, 70, 70, 34, 34, 34, 33,
 33, 34, 35, 35, 37, 37, 39, 39, 42, 42, 44, 45, 46, 47, 48, 51, 51, 54,
 54, 57, 58, 60, 63, 64, 68, 68, 74, 74, 34, 34, 34, 33, 33, 34, 35, 35,
 37, 37, 39, 39, 42, 42, 44, 45, 46, 47, 48, 51, 51, 54, 54, 57, 58, 60,
 63, 64, 68, 68, 74, 74, 35, 35, 35, 34, 34, 35, 36, 36, 38, 38, 42, 42,
 46, 47, 48, 49, 50, 52, 53, 55, 55, 58, 59, 61, 62, 64, 67, 68, 72, 72,
 78, 78, 36, 35, 35, 34, 34, 35, 36, 37, 38, 38, 42, 42, 47, 48, 50, 50,
 52, 54, 54, 57, 57, 59, 60, 62, 64, 66, 68, 69, 73, 73, 79, 79, 38, 37,
 37, 36, 36, 37, 38, 38, 39, 40, 44, 44, 48, 50, 51, 52, 54, 56, 57, 59,
 59, 62, 63, 65, 67, 69, 71, 72, 76, 76, 82, 82, 39, 38, 38, 38, 37, 38,
 39, 39, 40, 41, 45, 45, 49, 50, 52, 54, 55, 58, 58, 61, 61, 64, 65, 67,
 69, 71, 73, 74, 78, 78, 84, 84, 41, 40, 40, 39, 38, 39, 40, 40, 41, 41,
 46, 46, 50, 52, 54, 55, 57, 60, 60, 63, 63, 67, 67, 70, 71, 73, 75, 77,
 80, 81, 86, 86, 44, 42, 42, 41, 41, 41, 42, 42, 42, 43, 47, 47, 52, 54,
 56, 58, 60, 63, 64, 67, 67, 71, 71, 74, 75, 77, 79, 81, 84, 85, 90, 90,
 45, 43, 43, 42, 41, 42, 42, 43, 43, 43, 48, 48, 53, 54, 57, 58, 60, 64,
 65, 68, 68, 72, 72, 75, 76, 78, 81, 82, 85, 86, 91, 91, 48, 46, 46, 45,
 44, 45, 45, 45, 46, 46, 51, 51, 55, 57, 59, 61, 63, 67, 68, 71, 71, 75,
 76, 79, 80, 83, 85, 87, 90, 91, 96, 96, 48, 46, 46, 45, 44, 45, 45, 45,
 46, 46, 51, 51, 55, 57, 59, 61, 63, 67, 68, 71, 71, 75, 76, 79, 80, 83,
 85, 87, 90, 91, 96, 96, 53, 51, 51, 49, 49, 49, 49, 49, 49, 54, 54,
 58, 59, 62, 64, 67, 71, 72, 75, 75, 81, 81, 85, 86, 89, 91, 93, 97, 97,
 103, 103, 54, 52, 51, 50, 49, 49, 50, 49, 49, 50, 54, 54, 59, 60, 63,
 65, 67, 71, 72, 76, 76, 81, 82, 85, 87, 89, 92, 94, 97, 98, 104, 104,
 57, 55, 55, 53, 52, 52, 52, 52, 52, 52, 57, 57, 61, 62, 65, 67, 70, 74,
 75, 79, 79, 85, 85, 89, 90, 93, 96, 98, 102, 102, 108, 108, 59, 56, 56,
 54, 54, 54, 54, 54, 53, 54, 58, 58, 62, 64, 67, 69, 71, 75, 76, 80, 80,
 86, 87, 90, 92, 95, 98, 99, 103, 104, 110, 110, 62, 59, 59, 57, 56, 56,
 56, 56, 55, 56, 60, 60, 64, 66, 69, 71, 73, 77, 78, 83, 83, 89, 89, 93,
 95, 98, 101, 103, 107, 108, 114, 114, 65, 62, 62, 60, 59, 59, 59, 59,
 58, 58, 63, 63, 67, 68, 71, 73, 75, 79, 81, 85, 85, 91, 92, 96, 98, 101,
 105, 106, 111, 111, 118, 118, 67, 64, 64, 62, 61, 61, 60, 60, 59, 60,
 64, 64, 68, 69, 72, 74, 77, 81, 82, 87, 87, 93, 94, 98, 99, 103, 106,
 108, 113, 113, 120, 120, 71, 68, 68, 66, 65, 64, 64, 64, 63, 63, 68, 68,
 72, 73, 76, 78, 80, 84, 85, 90, 90, 97, 97, 102, 103, 107, 111, 113,
 117, 118, 125, 125, 72, 69, 69, 66, 65, 65, 65, 64, 63, 64, 68, 68, 72,
 73, 76, 78, 81, 85, 86, 91, 91, 97, 98, 102, 104, 108, 111, 113, 118,
 119, 126, 126, 80, 76, 76, 73, 72, 72, 71, 70, 69, 70, 74, 74, 78, 79,
 82, 84, 86, 90, 91, 96, 96, 103, 104, 108, 110, 114, 118, 120, 125, 126,
 134, 134, 80, 76, 76, 73, 72, 72, 71, 70, 69, 70, 74, 74, 78, 79, 82,
 84, 86, 90, 91, 96, 96, 103, 104, 108, 110, 114, 118, 120, 125, 126,
 134, 134,
 // Size 4x8
 32, 34, 43, 62, 32, 34, 42, 59, 33, 37, 44, 58, 35, 43, 54, 68, 41, 48,
 64, 79, 49, 54, 71, 91, 57, 60, 78, 101, 66, 68, 86, 111,
 // Size 8x4
 32, 32, 33, 35, 41, 49, 57, 66, 34, 34, 37, 43, 48, 54, 60, 68, 43, 42,
 44, 54, 64, 71, 78, 86, 62, 59, 58, 68, 79, 91, 101, 111,
 // Size 8x16
 32, 31, 32, 36, 44, 53, 62, 73, 31, 32, 32, 35, 42, 51, 59, 69, 31, 32,
 33, 34, 41, 49, 57, 66, 32, 32, 34, 36, 42, 50, 57, 65, 32, 33, 35, 38,
 42, 49, 56, 64, 34, 34, 37, 42, 48, 54, 61, 69, 35, 34, 38, 47, 52, 59,
 65, 73, 38, 36, 40, 49, 56, 63, 69, 77, 41, 39, 41, 51, 60, 67, 74, 81,
 44, 42, 43, 54, 64, 72, 79, 86, 48, 45, 46, 56, 67, 76, 83, 91, 53, 49,
 50, 60, 71, 82, 90, 99, 58, 54, 54, 63, 75, 87, 95, 105, 65, 60, 58, 68,
 79, 92, 102, 112, 71, 65, 63, 73, 84, 97, 108, 119, 79, 72, 70, 79, 90,
 104, 115, 127,
 // Size 16x8
 32, 31, 31, 32, 32, 34, 35, 38, 41, 44, 48, 53, 58, 65, 71, 79, 31, 32,
 32, 32, 33, 34, 34, 36, 39, 42, 45, 49, 54, 60, 65, 72, 32, 32, 33, 34,
 35, 37, 38, 40, 41, 43, 46, 50, 54, 58, 63, 70, 36, 35, 34, 36, 38, 42,
 47, 49, 51, 54, 56, 60, 63, 68, 73, 79, 44, 42, 41, 42, 42, 48, 52, 56,


```

60, 64, 67, 71, 75, 79, 84, 90, 53, 51, 49, 50, 49, 54, 59, 63, 67, 72,
76, 82, 87, 92, 97, 104, 62, 59, 57, 57, 56, 61, 65, 69, 74, 79, 83, 90,
95, 102, 108, 115, 73, 69, 66, 65, 64, 69, 73, 77, 81, 86, 91, 99, 105,
112, 119, 127,
// Size 16x32
32, 31, 31, 32, 32, 34, 36, 38, 44, 44, 53, 53, 62, 65, 73, 79, 31, 32,
32, 32, 32, 34, 35, 37, 42, 43, 51, 51, 60, 62, 70, 75, 31, 32, 32, 32,
32, 34, 35, 37, 42, 43, 51, 51, 59, 62, 69, 75, 31, 32, 32, 32, 32, 33,
35, 36, 41, 42, 50, 50, 58, 60, 67, 73, 31, 32, 32, 32, 33, 33, 34, 36,
41, 41, 49, 49, 57, 59, 66, 72, 31, 32, 32, 33, 33, 34, 35, 37, 41, 42,
49, 49, 57, 59, 66, 71, 32, 32, 32, 34, 34, 35, 37, 38, 42, 43, 50, 50,
57, 59, 65, 71, 32, 32, 32, 34, 34, 35, 37, 38, 42, 43, 49, 49, 56, 59,
65, 70, 32, 32, 33, 34, 35, 37, 38, 39, 42, 43, 49, 49, 56, 58, 64, 69,
32, 33, 33, 34, 35, 37, 39, 40, 43, 44, 50, 50, 56, 58, 64, 69, 34, 34,
34, 36, 37, 39, 42, 44, 48, 48, 54, 54, 61, 63, 69, 73, 34, 34, 34, 36,
37, 39, 42, 44, 48, 48, 54, 54, 61, 63, 69, 73, 35, 34, 34, 37, 38, 42,
47, 48, 52, 53, 59, 59, 65, 67, 73, 77, 36, 35, 34, 37, 38, 43, 48, 49,
54, 54, 60, 60, 66, 68, 74, 78, 38, 36, 36, 38, 40, 44, 49, 51, 56, 57,
63, 63, 69, 71, 77, 81, 39, 38, 37, 40, 40, 45, 50, 52, 58, 58, 65, 65,
71, 73, 79, 84, 41, 39, 39, 41, 41, 46, 51, 54, 60, 60, 67, 67, 74, 76,
81, 86, 44, 41, 41, 42, 43, 48, 53, 56, 63, 64, 71, 71, 78, 79, 85, 90,
44, 42, 42, 43, 43, 48, 54, 56, 64, 64, 72, 72, 79, 81, 86, 91, 48, 45,
45, 46, 46, 51, 56, 59, 67, 67, 76, 76, 83, 85, 91, 96, 48, 45, 45, 46,
46, 51, 56, 59, 67, 67, 76, 76, 83, 85, 91, 96, 53, 49, 49, 49, 49, 54,
59, 62, 71, 71, 81, 81, 89, 91, 98, 103, 53, 50, 49, 50, 50, 54, 60, 63,
71, 72, 82, 82, 90, 92, 99, 103, 57, 53, 52, 52, 52, 57, 62, 65, 74, 75,
85, 85, 94, 96, 103, 108, 58, 54, 54, 54, 54, 58, 63, 67, 75, 76, 87,
87, 95, 98, 105, 110, 61, 57, 57, 56, 56, 60, 66, 69, 77, 78, 89, 89,
98, 101, 108, 114, 65, 60, 60, 59, 58, 63, 68, 71, 79, 80, 92, 92, 102,
105, 112, 118, 67, 62, 61, 60, 60, 64, 69, 72, 81, 82, 94, 94, 103, 106,
114, 120, 71, 66, 65, 64, 63, 68, 73, 76, 84, 85, 97, 108, 111, 119,
125, 72, 66, 66, 64, 64, 68, 73, 76, 85, 86, 98, 98, 108, 111, 119, 125,
79, 73, 72, 71, 70, 74, 79, 82, 90, 91, 104, 104, 115, 118, 127, 133,
79, 73, 72, 71, 70, 74, 79, 82, 90, 91, 104, 104, 115, 118, 127, 133,
// Size 32x16
32, 31, 31, 31, 31, 32, 32, 32, 32, 34, 34, 35, 36, 38, 39, 41, 44,
44, 48, 48, 53, 53, 57, 58, 61, 65, 67, 71, 72, 79, 79, 31, 32, 32, 32,
32, 32, 32, 32, 32, 33, 34, 34, 34, 35, 36, 38, 39, 41, 42, 45, 45, 49,
50, 53, 54, 57, 60, 62, 66, 66, 73, 73, 31, 32, 32, 32, 32, 32, 32, 32,
33, 33, 34, 34, 34, 36, 37, 39, 41, 42, 45, 45, 49, 49, 52, 54, 57,
60, 61, 65, 66, 72, 72, 32, 32, 32, 32, 33, 33, 34, 34, 34, 36, 36,
37, 37, 38, 40, 41, 42, 43, 46, 46, 49, 50, 52, 54, 56, 59, 60, 64, 64,
71, 71, 32, 32, 32, 32, 33, 33, 34, 34, 35, 35, 37, 37, 38, 38, 40, 40,
41, 43, 43, 46, 46, 49, 50, 52, 54, 56, 58, 60, 63, 64, 70, 70, 34, 34,
34, 33, 33, 34, 35, 35, 37, 37, 39, 39, 42, 43, 44, 45, 46, 48, 48, 51,
51, 54, 54, 57, 58, 60, 63, 64, 68, 68, 74, 74, 36, 35, 35, 35, 34, 35,
36, 37, 38, 39, 42, 42, 47, 48, 49, 50, 51, 53, 54, 56, 56, 59, 60, 62,
63, 66, 68, 69, 73, 73, 79, 79, 38, 37, 37, 36, 36, 37, 38, 38, 39, 40,
44, 44, 48, 49, 51, 52, 54, 56, 56, 59, 59, 62, 63, 65, 67, 69, 71, 72,
76, 76, 82, 82, 44, 42, 42, 41, 41, 41, 42, 42, 42, 43, 48, 48, 52, 54,
56, 58, 60, 63, 64, 67, 67, 71, 71, 74, 75, 77, 79, 81, 84, 85, 90, 90,
44, 43, 43, 42, 41, 42, 43, 43, 43, 44, 48, 48, 53, 54, 57, 58, 60, 64,
64, 67, 67, 71, 72, 75, 76, 78, 80, 82, 85, 86, 91, 91, 53, 51, 51, 50,
49, 49, 50, 49, 49, 50, 54, 54, 59, 60, 63, 65, 67, 71, 72, 76, 76, 81,
82, 85, 87, 89, 92, 94, 97, 98, 104, 104, 53, 51, 51, 50, 49, 49, 50,
49, 49, 50, 54, 54, 59, 60, 63, 65, 67, 71, 72, 76, 76, 81, 82, 85, 87,
89, 92, 94, 97, 98, 104, 104, 62, 60, 59, 58, 57, 57, 57, 56, 56, 56,
61, 61, 65, 66, 69, 71, 74, 78, 79, 83, 83, 89, 90, 94, 95, 98, 102,
103, 108, 108, 115, 115, 65, 62, 62, 60, 59, 59, 59, 59, 58, 58, 63, 63,
67, 68, 71, 73, 76, 79, 81, 85, 85, 91, 92, 96, 98, 101, 105, 106, 111,
111, 118, 118, 73, 70, 69, 67, 66, 65, 65, 64, 64, 69, 69, 73, 74,
77, 79, 81, 85, 86, 91, 91, 98, 99, 103, 105, 108, 112, 114, 119, 119,
127, 127, 79, 75, 75, 73, 72, 71, 71, 70, 69, 69, 73, 73, 77, 78, 81,
84, 86, 90, 91, 96, 96, 103, 103, 108, 110, 114, 118, 120, 125, 125,
133, 133,
// Size 4x16
31, 34, 44, 65, 32, 34, 43, 62, 32, 33, 41, 59, 32, 35, 43, 59, 32, 37,
43, 58, 34, 39, 48, 63, 34, 42, 53, 67, 36, 44, 57, 71, 39, 46, 60, 76,
42, 48, 64, 81, 45, 51, 67, 85, 50, 54, 72, 92, 54, 58, 76, 98, 60, 63,
80, 105, 66, 68, 85, 111, 73, 74, 91, 118,
// Size 16x4

```

```

31, 32, 32, 32, 32, 34, 34, 36, 39, 42, 45, 50, 54, 60, 66, 73, 34, 34,
33, 35, 37, 39, 42, 44, 46, 48, 51, 54, 58, 63, 68, 74, 44, 43, 41, 43,
43, 48, 53, 57, 60, 64, 67, 72, 76, 80, 85, 91, 65, 62, 59, 59, 58, 63,
67, 71, 76, 81, 85, 92, 98, 105, 111, 118,
// Size 8x32
32, 31, 32, 36, 44, 53, 62, 73, 31, 32, 32, 35, 42, 51, 60, 70, 31, 32,
32, 35, 42, 51, 59, 69, 31, 32, 32, 35, 41, 50, 58, 67, 31, 32, 33, 34,
41, 49, 57, 66, 31, 32, 33, 35, 41, 49, 57, 66, 32, 32, 34, 36, 42, 50,
57, 65, 32, 32, 34, 37, 42, 49, 56, 65, 32, 33, 35, 38, 42, 49, 56, 64,
32, 33, 35, 39, 43, 50, 56, 64, 34, 34, 37, 42, 48, 54, 61, 69, 34, 34,
37, 42, 48, 54, 61, 69, 35, 34, 38, 47, 52, 59, 65, 73, 36, 34, 38, 48,
54, 60, 66, 74, 38, 36, 40, 49, 56, 63, 69, 77, 39, 37, 40, 50, 58, 65,
71, 79, 41, 39, 41, 51, 60, 67, 74, 81, 44, 41, 43, 53, 63, 71, 78, 85,
44, 42, 43, 54, 64, 72, 79, 86, 48, 45, 46, 56, 67, 76, 83, 91, 48, 45,
46, 56, 67, 76, 83, 91, 53, 49, 49, 59, 71, 81, 89, 98, 53, 49, 50, 60,
71, 82, 90, 99, 57, 52, 52, 62, 74, 85, 94, 103, 58, 54, 54, 63, 75, 87,
95, 105, 61, 57, 56, 66, 77, 89, 98, 108, 65, 60, 58, 68, 79, 92, 102,
112, 67, 61, 60, 69, 81, 94, 103, 114, 71, 65, 63, 73, 84, 97, 108, 119,
72, 66, 64, 73, 85, 98, 108, 119, 79, 72, 70, 79, 90, 104, 115, 127, 79,
72, 70, 79, 90, 104, 115, 127,
// Size 32x8
32, 31, 31, 31, 31, 31, 32, 32, 32, 32, 34, 34, 35, 36, 38, 39, 41, 44,
44, 48, 48, 53, 53, 57, 58, 61, 65, 67, 71, 72, 79, 79, 31, 32, 32, 32,
32, 32, 32, 32, 33, 33, 34, 34, 34, 34, 36, 37, 39, 41, 42, 45, 45, 49,
49, 52, 54, 57, 60, 61, 65, 66, 72, 72, 32, 32, 32, 32, 33, 33, 34, 34,
35, 35, 37, 37, 38, 38, 40, 40, 41, 43, 43, 46, 46, 49, 50, 52, 54, 56,
58, 60, 63, 64, 70, 70, 36, 35, 35, 35, 34, 35, 36, 37, 38, 39, 42, 42,
47, 48, 49, 50, 51, 53, 54, 56, 56, 59, 60, 62, 63, 66, 68, 69, 73, 73,
79, 79, 44, 42, 42, 41, 41, 41, 42, 42, 42, 43, 48, 48, 52, 54, 56, 58,
60, 63, 64, 67, 67, 71, 71, 74, 75, 77, 79, 81, 84, 85, 90, 90, 53, 51,
51, 50, 49, 49, 50, 49, 49, 50, 54, 54, 59, 60, 63, 65, 67, 71, 72, 76,
76, 81, 82, 85, 87, 89, 92, 94, 97, 98, 104, 104, 62, 60, 59, 58, 57,
57, 57, 56, 56, 56, 61, 61, 65, 66, 69, 71, 74, 78, 79, 83, 83, 89, 90,
94, 95, 98, 102, 103, 108, 108, 115, 115, 73, 70, 69, 67, 66, 66, 65,
65, 64, 64, 69, 69, 73, 74, 77, 79, 81, 85, 86, 91, 91, 98, 99, 103,
105, 108, 112, 114, 119, 119, 127, 127,
// Size 4x32
31, 34, 44, 65, 32, 34, 43, 62, 32, 34, 43, 62, 32, 33, 42, 60, 32, 33,
41, 59, 32, 34, 42, 59, 32, 35, 43, 59, 32, 35, 43, 59, 32, 37, 43, 58,
33, 37, 44, 58, 34, 39, 48, 63, 34, 39, 48, 63, 34, 42, 53, 67, 35, 43,
54, 68, 36, 44, 57, 71, 38, 45, 58, 73, 39, 46, 60, 76, 41, 48, 64, 79,
42, 48, 64, 81, 45, 51, 67, 85, 45, 51, 67, 85, 49, 54, 71, 91, 50, 54,
72, 92, 53, 57, 75, 96, 54, 58, 76, 98, 57, 60, 78, 101, 60, 63, 80,
105, 62, 64, 82, 106, 66, 68, 85, 111, 66, 68, 86, 111, 73, 74, 91, 118,
73, 74, 91, 118,
// Size 32x4
31, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 35, 36, 38, 39, 41,
42, 45, 45, 49, 50, 53, 54, 57, 60, 62, 66, 66, 73, 73, 34, 34, 34, 33,
33, 34, 35, 35, 37, 37, 39, 39, 42, 43, 44, 45, 46, 48, 48, 51, 51, 54,
54, 57, 58, 60, 63, 64, 68, 68, 74, 74, 44, 43, 43, 42, 41, 42, 43, 43,
43, 44, 48, 48, 53, 54, 57, 58, 60, 64, 64, 67, 71, 72, 75, 76, 78,
80, 82, 85, 86, 91, 91, 65, 62, 62, 60, 59, 59, 59, 59, 58, 58, 63, 63,
67, 68, 71, 73, 76, 79, 81, 85, 85, 91, 92, 96, 98, 101, 105, 106, 111,
111, 118, 118 ,
},
{ // Chroma
// Size 4x4
31, 42, 47, 53, 42, 48, 50, 54, 47, 50, 61, 67, 53, 54, 67, 78,
// Size 8x8
31, 32, 38, 48, 47, 50, 53, 57, 32, 35, 42, 47, 45, 47, 50, 54, 38, 42,
47, 48, 45, 47, 49, 52, 48, 47, 48, 53, 53, 54, 56, 58, 47, 45, 45, 53,
58, 61, 63, 65, 50, 47, 47, 54, 61, 66, 69, 72, 53, 50, 49, 56, 63, 69,
73, 77, 57, 54, 52, 58, 65, 72, 77, 82,
// Size 16x16
32, 31, 30, 33, 36, 41, 47, 49, 49, 49, 50, 52, 54, 57, 60, 63, 31, 31,
31, 34, 38, 42, 46, 47, 47, 47, 48, 50, 52, 54, 57, 60, 30, 31, 32, 35,
40, 42, 45, 46, 45, 45, 46, 47, 49, 52, 54, 57, 33, 34, 35, 39, 43, 45,
47, 46, 46, 45, 46, 47, 49, 51, 53, 56, 36, 38, 40, 43, 47, 47, 47, 47,
46, 45, 46, 47, 48, 50, 52, 54, 41, 42, 42, 45, 47, 48, 50, 50, 49, 49,
50, 50, 52, 53, 55, 57, 47, 46, 45, 47, 47, 50, 52, 52, 52, 52, 53, 53,
55, 56, 58, 60, 49, 47, 46, 46, 47, 50, 52, 53, 54, 55, 55, 56, 57, 58,

```

```

60, 62, 49, 47, 45, 46, 46, 49, 52, 54, 55, 57, 58, 59, 60, 61, 63, 65,
49, 47, 45, 45, 45, 49, 52, 55, 57, 59, 60, 61, 63, 64, 66, 68, 50, 48,
46, 46, 46, 50, 53, 55, 58, 60, 61, 63, 65, 67, 68, 71, 52, 50, 47, 47,
47, 50, 53, 56, 59, 61, 63, 66, 68, 70, 72, 75, 54, 52, 49, 49, 48, 52,
55, 57, 60, 63, 65, 68, 71, 73, 75, 78, 57, 54, 52, 51, 50, 53, 56, 58,
61, 64, 67, 70, 73, 76, 79, 82, 60, 57, 54, 53, 52, 55, 58, 60, 63, 66,
68, 72, 75, 79, 82, 85, 63, 60, 57, 56, 54, 57, 60, 62, 65, 68, 71, 75,
78, 82, 85, 89,
// Size 32x32
32, 31, 31, 30, 30, 32, 33, 34, 36, 37, 41, 41, 47, 49, 49, 48, 49, 49,
49, 50, 50, 52, 52, 54, 54, 56, 57, 58, 60, 60, 63, 63, 31, 31, 31, 31,
31, 32, 34, 35, 38, 38, 42, 42, 46, 48, 47, 47, 47, 47, 48, 48, 50,
50, 51, 52, 53, 54, 55, 57, 57, 60, 60, 31, 31, 31, 31, 31, 33, 34, 35,
38, 39, 42, 42, 46, 47, 47, 47, 47, 47, 47, 48, 48, 49, 50, 51, 52, 53,
54, 55, 57, 57, 60, 60, 30, 31, 31, 31, 31, 33, 35, 36, 39, 40, 42, 42,
46, 47, 46, 46, 46, 45, 46, 47, 47, 48, 48, 50, 50, 51, 52, 53, 55, 55,
58, 58, 30, 31, 31, 31, 32, 33, 35, 36, 40, 40, 42, 42, 45, 46, 46, 45,
45, 45, 45, 46, 46, 47, 47, 49, 49, 51, 52, 52, 54, 54, 57, 57, 32, 32,
33, 33, 33, 35, 37, 38, 41, 42, 43, 43, 46, 47, 46, 46, 45, 45, 45, 46,
46, 47, 47, 49, 49, 50, 51, 52, 54, 54, 57, 57, 33, 34, 34, 35, 35, 37,
39, 40, 43, 43, 45, 45, 47, 47, 46, 46, 46, 45, 45, 46, 46, 47, 47, 49,
49, 50, 51, 52, 53, 54, 56, 56, 34, 35, 35, 36, 36, 38, 40, 41, 44, 44,
45, 45, 47, 47, 47, 46, 46, 45, 45, 46, 46, 47, 47, 48, 49, 50, 51, 51,
53, 53, 55, 55, 36, 38, 38, 39, 40, 41, 43, 44, 47, 47, 47, 47, 47, 48,
47, 46, 46, 45, 45, 46, 46, 46, 47, 48, 48, 49, 50, 50, 52, 52, 54, 54,
37, 38, 39, 40, 40, 42, 43, 44, 47, 47, 47, 47, 48, 48, 47, 47, 46, 45,
46, 46, 46, 47, 47, 48, 48, 49, 50, 51, 52, 52, 55, 55, 41, 42, 42, 42,
42, 43, 45, 45, 47, 47, 48, 48, 50, 50, 50, 49, 49, 49, 49, 50, 50, 50,
50, 51, 52, 52, 53, 54, 55, 55, 57, 57, 41, 42, 42, 42, 42, 43, 45, 45,
47, 47, 48, 48, 50, 50, 50, 49, 49, 49, 49, 50, 50, 50, 51, 52, 52,
53, 54, 55, 55, 57, 57, 47, 46, 46, 45, 46, 47, 47, 47, 48, 50, 50,
52, 52, 52, 52, 52, 52, 52, 53, 53, 53, 53, 54, 55, 55, 56, 56, 58, 58,
60, 60, 49, 48, 47, 47, 46, 47, 47, 47, 48, 48, 50, 50, 52, 53, 53, 53,
53, 53, 53, 54, 54, 54, 54, 55, 55, 56, 56, 57, 58, 58, 60, 60, 49, 47,
47, 46, 46, 46, 47, 47, 47, 50, 50, 52, 53, 53, 54, 54, 55, 55, 55,
55, 56, 56, 57, 57, 58, 58, 59, 60, 60, 62, 62, 48, 47, 47, 46, 45, 46,
46, 46, 46, 47, 49, 49, 52, 53, 54, 54, 55, 55, 56, 56, 56, 57, 57, 58,
58, 59, 60, 60, 61, 62, 63, 63, 49, 47, 47, 46, 45, 45, 46, 46, 46, 46,
49, 49, 52, 53, 54, 55, 55, 57, 57, 58, 58, 59, 59, 60, 60, 61, 61, 62,
63, 63, 65, 65, 49, 47, 47, 45, 45, 45, 45, 45, 45, 49, 49, 52, 53,
55, 55, 57, 58, 59, 60, 60, 61, 61, 62, 62, 63, 63, 64, 65, 65, 67, 67,
49, 47, 47, 46, 45, 45, 45, 45, 45, 46, 49, 49, 52, 53, 55, 56, 57, 59,
59, 60, 60, 61, 61, 62, 63, 63, 64, 65, 66, 66, 68, 68, 50, 48, 48, 47,
46, 46, 46, 46, 46, 46, 50, 50, 53, 54, 55, 56, 58, 60, 60, 61, 61, 63,
63, 65, 65, 66, 67, 67, 68, 69, 71, 71, 50, 48, 48, 47, 46, 46, 46, 46,
46, 46, 50, 50, 53, 54, 55, 56, 58, 60, 60, 61, 61, 63, 63, 65, 65, 66,
67, 67, 68, 69, 71, 71, 52, 50, 49, 48, 47, 47, 47, 47, 46, 47, 50, 50,
53, 54, 56, 57, 59, 61, 61, 63, 63, 66, 66, 67, 68, 69, 70, 71, 72, 72,
74, 74, 52, 50, 50, 48, 47, 47, 47, 47, 47, 47, 50, 50, 53, 54, 56, 57,
59, 61, 61, 63, 63, 66, 66, 68, 68, 69, 70, 71, 72, 73, 75, 75, 54, 51,
51, 50, 49, 49, 49, 48, 48, 48, 51, 51, 54, 55, 57, 58, 60, 62, 62, 65,
65, 67, 68, 69, 70, 71, 72, 73, 74, 75, 77, 77, 54, 52, 52, 50, 49, 49,
49, 49, 48, 48, 52, 52, 55, 55, 57, 58, 60, 62, 63, 65, 65, 68, 68, 70,
71, 72, 73, 74, 75, 76, 78, 78, 56, 53, 53, 51, 51, 50, 50, 50, 49, 49,
52, 52, 55, 56, 58, 59, 61, 63, 63, 66, 66, 69, 69, 71, 72, 73, 75, 75,
77, 77, 80, 80, 57, 54, 54, 52, 52, 51, 51, 51, 50, 53, 53, 56, 56,
58, 60, 61, 63, 64, 67, 67, 70, 70, 72, 73, 75, 76, 77, 79, 79, 82, 82,
58, 55, 55, 53, 52, 52, 52, 51, 50, 51, 54, 54, 56, 57, 59, 60, 62, 64,
65, 67, 67, 71, 71, 73, 74, 75, 77, 78, 80, 80, 83, 83, 60, 57, 57, 55,
54, 54, 53, 52, 52, 55, 55, 58, 58, 60, 61, 63, 65, 66, 68, 68, 72,
72, 74, 75, 77, 79, 80, 82, 82, 85, 85, 60, 57, 57, 55, 54, 54, 53,
52, 52, 55, 55, 58, 58, 60, 62, 63, 65, 66, 69, 69, 72, 73, 75, 76, 77,
79, 80, 82, 82, 85, 85, 63, 60, 60, 58, 57, 57, 56, 55, 54, 55, 57, 57,
60, 60, 62, 63, 65, 67, 68, 71, 71, 74, 75, 77, 78, 80, 82, 83, 85, 85,
89, 89, 63, 60, 60, 58, 57, 57, 56, 55, 54, 55, 57, 57, 60, 60, 62, 63,
65, 67, 68, 71, 71, 74, 75, 77, 78, 80, 82, 83, 85, 85, 89, 89,
// Size 4x8
31, 42, 47, 54, 33, 44, 45, 51, 40, 47, 46, 50, 47, 50, 54, 57, 45, 49,
59, 64, 48, 50, 61, 70, 51, 52, 63, 75, 55, 55, 66, 79,
// Size 8x4
31, 33, 40, 47, 45, 48, 51, 55, 42, 44, 47, 50, 49, 50, 52, 55, 47, 45,

```

```

46, 54, 59, 61, 63, 66, 54, 51, 50, 57, 64, 70, 75, 79,
// Size 8x16
32, 31, 37, 48, 49, 52, 56, 61, 31, 31, 38, 47, 47, 50, 53, 57, 30, 32,
40, 46, 45, 48, 51, 55, 33, 36, 43, 47, 46, 47, 50, 54, 37, 40, 47, 47,
45, 47, 49, 52, 42, 43, 47, 50, 49, 50, 53, 56, 47, 46, 48, 52, 53, 53,
55, 58, 48, 46, 47, 53, 55, 56, 58, 61, 48, 45, 46, 53, 57, 59, 61, 63,
49, 45, 46, 53, 58, 62, 64, 66, 50, 46, 46, 54, 59, 64, 66, 69, 52, 48,
47, 54, 61, 66, 70, 73, 54, 50, 49, 55, 62, 68, 72, 76, 57, 52, 50, 56,
64, 70, 75, 79, 60, 54, 52, 58, 65, 72, 77, 82, 63, 57, 55, 60, 67, 75,
80, 86,
// Size 16x8
32, 31, 30, 33, 37, 42, 47, 48, 48, 49, 50, 52, 54, 57, 60, 63, 31, 31,
32, 36, 40, 43, 46, 46, 45, 45, 46, 48, 50, 52, 54, 57, 37, 38, 40, 43,
47, 47, 48, 47, 46, 46, 46, 47, 49, 50, 52, 55, 48, 47, 46, 47, 47, 50,
52, 53, 53, 53, 54, 54, 55, 56, 58, 60, 49, 47, 45, 46, 45, 49, 53, 55,
57, 58, 59, 61, 62, 64, 65, 67, 52, 50, 48, 47, 47, 50, 53, 56, 59, 62,
64, 66, 68, 70, 72, 75, 56, 53, 51, 50, 49, 53, 55, 58, 61, 64, 66, 70,
72, 75, 77, 80, 61, 57, 55, 54, 52, 56, 58, 61, 63, 66, 69, 73, 76, 79,
82, 86,
// Size 16x32
32, 31, 31, 35, 37, 42, 48, 48, 49, 49, 52, 52, 56, 57, 61, 63, 31, 31,
31, 36, 38, 42, 47, 47, 47, 47, 50, 50, 54, 54, 58, 60, 31, 31, 31, 36,
38, 42, 47, 47, 47, 47, 50, 50, 53, 54, 57, 60, 30, 32, 32, 37, 39, 42,
46, 46, 46, 46, 48, 48, 52, 52, 56, 58, 30, 32, 32, 37, 40, 42, 46, 46,
45, 45, 48, 48, 51, 52, 55, 57, 32, 33, 34, 39, 41, 44, 46, 46, 45, 45,
48, 48, 51, 51, 54, 57, 33, 35, 36, 40, 43, 45, 47, 46, 46, 46, 47, 47,
50, 51, 54, 56, 34, 37, 37, 42, 44, 45, 47, 47, 45, 46, 47, 47, 50, 51,
53, 55, 37, 40, 40, 45, 47, 47, 47, 47, 45, 46, 47, 47, 49, 50, 52, 54,
37, 40, 40, 45, 47, 47, 48, 47, 46, 46, 47, 47, 49, 50, 53, 55, 42, 43,
43, 46, 47, 48, 50, 50, 49, 49, 49, 50, 50, 53, 53, 56, 57, 42, 43, 43, 46,
47, 48, 50, 50, 49, 49, 50, 50, 53, 53, 56, 57, 47, 46, 46, 47, 48, 50,
52, 52, 53, 53, 53, 53, 55, 56, 58, 60, 49, 47, 46, 47, 48, 50, 53, 53,
53, 54, 54, 54, 56, 57, 59, 60, 48, 46, 46, 47, 47, 50, 53, 53, 55, 55,
56, 56, 58, 58, 61, 62, 48, 46, 46, 46, 47, 50, 53, 54, 56, 56, 57, 57,
59, 60, 62, 64, 48, 46, 45, 46, 46, 49, 53, 54, 57, 57, 59, 59, 61, 61,
63, 65, 49, 45, 45, 45, 46, 49, 53, 55, 58, 59, 61, 61, 63, 64, 66, 67,
49, 46, 45, 46, 46, 49, 53, 55, 58, 59, 62, 62, 64, 64, 66, 68, 50, 47,
46, 46, 46, 50, 54, 55, 59, 60, 64, 64, 66, 67, 69, 71, 50, 47, 46, 46,
46, 50, 54, 55, 59, 60, 64, 64, 66, 67, 69, 71, 52, 48, 48, 47, 47, 50,
54, 56, 61, 61, 66, 66, 69, 70, 72, 74, 52, 48, 48, 47, 47, 50, 54, 56,
61, 61, 66, 66, 70, 71, 73, 75, 53, 50, 49, 48, 48, 51, 55, 57, 62, 62,
68, 68, 71, 72, 75, 77, 54, 50, 49, 49, 52, 55, 57, 62, 63, 68, 68,
72, 73, 76, 78, 55, 51, 51, 50, 49, 52, 56, 58, 63, 63, 69, 69, 74, 75,
78, 80, 57, 52, 52, 51, 50, 53, 56, 58, 64, 64, 70, 70, 75, 76, 79, 82,
58, 53, 53, 51, 51, 54, 57, 59, 64, 65, 71, 71, 76, 77, 80, 83, 60, 55,
54, 53, 52, 55, 58, 60, 65, 66, 72, 72, 77, 79, 82, 85, 60, 55, 55, 53,
53, 55, 59, 60, 65, 66, 73, 73, 78, 79, 83, 85, 63, 58, 57, 56, 55, 58,
60, 62, 67, 68, 75, 75, 80, 82, 86, 89, 63, 58, 57, 56, 55, 58, 60, 62,
67, 68, 75, 75, 80, 82, 86, 89,
// Size 32x16
32, 31, 31, 30, 30, 32, 33, 34, 37, 37, 42, 42, 47, 49, 48, 48, 48, 49,
49, 50, 50, 52, 52, 53, 54, 55, 57, 58, 60, 60, 63, 63, 31, 31, 31, 32,
32, 33, 35, 37, 40, 40, 43, 43, 46, 47, 46, 46, 46, 45, 46, 47, 47, 48,
48, 50, 50, 51, 52, 53, 55, 55, 58, 58, 31, 31, 31, 32, 32, 34, 36, 37,
40, 40, 43, 43, 46, 46, 46, 46, 45, 45, 45, 46, 46, 48, 48, 49, 50, 51,
52, 53, 54, 55, 57, 57, 35, 36, 36, 37, 37, 39, 40, 42, 45, 45, 46, 46,
47, 47, 47, 46, 46, 45, 46, 46, 46, 47, 47, 48, 49, 50, 51, 51, 53, 53,
56, 56, 37, 38, 38, 39, 40, 41, 43, 44, 47, 47, 47, 47, 48, 48, 47, 47,
46, 46, 46, 46, 46, 47, 47, 48, 49, 49, 50, 51, 52, 53, 55, 55, 42, 42,
42, 42, 42, 44, 45, 45, 47, 47, 48, 48, 50, 50, 50, 50, 49, 49, 49, 50,
50, 50, 50, 51, 52, 52, 53, 54, 55, 55, 58, 58, 48, 47, 47, 46, 46, 46,
47, 47, 47, 48, 50, 50, 52, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 55,
55, 56, 56, 57, 58, 59, 60, 60, 48, 47, 47, 46, 46, 46, 46, 47, 47, 47,
50, 50, 52, 53, 53, 54, 54, 55, 55, 55, 55, 56, 56, 57, 57, 58, 58, 59,
60, 60, 62, 62, 49, 47, 47, 46, 45, 45, 46, 45, 45, 46, 49, 49, 53, 53,
55, 56, 57, 58, 58, 59, 59, 61, 61, 62, 62, 63, 64, 64, 65, 65, 67, 67,
49, 47, 47, 46, 45, 45, 46, 46, 46, 46, 49, 49, 53, 54, 55, 56, 57, 59,
59, 60, 60, 61, 61, 62, 63, 63, 64, 65, 66, 66, 68, 68, 52, 50, 50, 48,
48, 48, 47, 47, 47, 47, 50, 50, 53, 54, 56, 57, 59, 61, 62, 64, 64, 66,
66, 68, 68, 69, 70, 71, 72, 73, 75, 75, 52, 50, 50, 48, 48, 48, 47, 47,
47, 47, 50, 50, 53, 54, 56, 57, 59, 61, 62, 64, 64, 66, 66, 68, 68, 69,

```

```

70, 71, 72, 73, 75, 75, 56, 54, 53, 52, 51, 51, 50, 50, 49, 49, 53, 53,
55, 56, 58, 59, 61, 63, 64, 66, 66, 69, 70, 71, 72, 74, 75, 76, 77, 78,
80, 80, 57, 54, 54, 52, 52, 51, 51, 51, 50, 50, 53, 53, 56, 57, 58, 60,
61, 64, 64, 67, 67, 70, 71, 72, 73, 75, 76, 77, 79, 79, 82, 82, 61, 58,
57, 56, 55, 54, 54, 53, 52, 53, 56, 56, 58, 59, 61, 62, 63, 66, 66, 69,
69, 72, 73, 75, 76, 78, 79, 80, 82, 83, 86, 86, 63, 60, 60, 58, 57, 57,
56, 55, 54, 55, 57, 57, 60, 60, 62, 64, 65, 67, 68, 71, 71, 74, 75, 77,
78, 80, 82, 83, 85, 85, 89, 89,
// Size 4x16
31, 42, 49, 57, 31, 42, 47, 54, 32, 42, 45, 52, 35, 45, 46, 51, 40, 47,
46, 50, 43, 48, 49, 53, 46, 50, 53, 56, 46, 50, 55, 58, 46, 49, 57, 61,
46, 49, 59, 64, 47, 50, 60, 67, 48, 50, 61, 71, 50, 52, 63, 73, 52, 53,
64, 76, 55, 55, 66, 79, 58, 58, 68, 82,
// Size 16x4
31, 31, 32, 35, 40, 43, 46, 46, 46, 46, 47, 48, 50, 52, 55, 58, 42, 42,
42, 45, 47, 48, 50, 50, 49, 49, 50, 50, 52, 53, 55, 58, 49, 47, 45, 46,
46, 49, 53, 55, 57, 59, 60, 61, 63, 64, 66, 68, 57, 54, 52, 51, 50, 53,
56, 58, 61, 64, 67, 71, 73, 76, 79, 82,
// Size 8x32
32, 31, 37, 48, 49, 52, 56, 61, 31, 31, 38, 47, 47, 50, 54, 58, 31, 31,
38, 47, 47, 50, 53, 57, 30, 32, 39, 46, 46, 48, 52, 56, 30, 32, 40, 46,
45, 48, 51, 55, 32, 34, 41, 46, 45, 48, 51, 54, 33, 36, 43, 47, 46, 47,
50, 54, 34, 37, 44, 47, 45, 47, 50, 53, 37, 40, 47, 47, 45, 47, 49, 52,
37, 40, 47, 48, 46, 47, 49, 53, 42, 43, 47, 50, 49, 50, 53, 56, 42, 43,
47, 50, 49, 50, 53, 56, 47, 46, 48, 52, 53, 53, 55, 58, 49, 46, 48, 53,
53, 54, 56, 59, 48, 46, 47, 53, 55, 56, 58, 61, 48, 46, 47, 53, 56, 57,
59, 62, 48, 45, 46, 53, 57, 59, 61, 63, 49, 45, 46, 53, 58, 61, 63, 66,
49, 45, 46, 53, 58, 62, 64, 66, 50, 46, 46, 54, 59, 64, 66, 69, 50, 46,
46, 54, 59, 64, 66, 69, 52, 48, 47, 54, 61, 66, 69, 72, 52, 48, 47, 54,
61, 66, 70, 73, 53, 49, 48, 55, 62, 68, 71, 75, 54, 50, 49, 55, 62, 68,
72, 76, 55, 51, 49, 56, 63, 69, 74, 78, 57, 52, 50, 56, 64, 70, 75, 79,
58, 53, 51, 57, 64, 71, 76, 80, 60, 54, 52, 58, 65, 72, 77, 82, 60, 55,
53, 59, 65, 73, 78, 83, 63, 57, 55, 60, 67, 75, 80, 86, 63, 57, 55, 60,
67, 75, 80, 86,
// Size 32x8
32, 31, 31, 30, 30, 32, 33, 34, 37, 37, 42, 42, 47, 49, 48, 48, 48, 49,
49, 50, 50, 52, 52, 53, 54, 55, 57, 58, 60, 60, 63, 63, 31, 31, 31, 32,
32, 34, 36, 37, 40, 40, 43, 43, 46, 46, 46, 46, 45, 45, 45, 46, 46, 48,
48, 49, 50, 51, 52, 53, 54, 55, 57, 57, 37, 38, 38, 39, 40, 41, 43, 44,
47, 47, 47, 47, 48, 48, 47, 47, 46, 46, 46, 46, 46, 47, 47, 48, 49, 49,
50, 51, 52, 53, 55, 55, 48, 47, 47, 46, 46, 46, 47, 47, 47, 48, 50, 50,
52, 53, 53, 53, 53, 53, 54, 54, 54, 54, 55, 55, 56, 56, 57, 58, 59,
60, 60, 49, 47, 47, 46, 45, 45, 46, 45, 45, 46, 49, 49, 53, 53, 55, 56,
57, 58, 58, 59, 59, 61, 61, 62, 62, 63, 64, 64, 65, 65, 67, 67, 52, 50,
50, 48, 48, 48, 47, 47, 47, 47, 50, 50, 53, 54, 56, 57, 59, 61, 62, 64,
64, 66, 66, 68, 68, 69, 70, 71, 72, 73, 75, 75, 56, 54, 53, 52, 51, 51,
50, 50, 49, 49, 53, 53, 55, 56, 58, 59, 61, 63, 64, 66, 66, 69, 70, 71,
72, 74, 75, 76, 77, 78, 80, 80, 61, 58, 57, 56, 55, 54, 54, 53, 52, 53,
56, 56, 58, 59, 61, 62, 63, 66, 66, 69, 69, 72, 73, 75, 76, 78, 79, 80,
82, 83, 86, 86,
// Size 4x32
31, 42, 49, 57, 31, 42, 47, 54, 31, 42, 47, 54, 32, 42, 46, 52, 32, 42,
45, 52, 33, 44, 45, 51, 35, 45, 46, 51, 37, 45, 46, 51, 40, 47, 46, 50,
40, 47, 46, 50, 43, 48, 49, 53, 43, 48, 49, 53, 46, 50, 53, 56, 47, 50,
54, 57, 46, 50, 55, 58, 46, 50, 56, 60, 46, 49, 57, 61, 45, 49, 59, 64,
46, 49, 59, 64, 47, 50, 60, 67, 47, 50, 60, 67, 48, 50, 61, 70, 48, 50,
61, 71, 50, 51, 62, 72, 50, 52, 63, 73, 51, 52, 63, 75, 52, 53, 64, 76,
53, 54, 65, 77, 55, 55, 66, 79, 55, 55, 66, 79, 58, 58, 68, 82, 58, 58,
68, 82,
// Size 32x4
31, 31, 31, 32, 32, 33, 35, 37, 40, 40, 43, 43, 46, 47, 46, 46, 46, 45,
46, 47, 47, 48, 48, 50, 50, 51, 52, 53, 55, 55, 58, 58, 42, 42, 42, 42,
42, 44, 45, 45, 47, 47, 48, 48, 50, 50, 50, 50, 49, 49, 49, 50, 50, 50,
50, 51, 52, 52, 53, 54, 55, 55, 58, 58, 49, 47, 47, 46, 45, 45, 46, 46,
46, 46, 49, 49, 53, 54, 55, 56, 57, 59, 59, 60, 60, 61, 61, 62, 63, 63,
64, 65, 66, 66, 68, 68, 57, 54, 54, 52, 52, 51, 51, 51, 50, 50, 53, 53,
56, 57, 58, 60, 61, 64, 64, 67, 67, 70, 71, 72, 73, 75, 76, 77, 79, 79,
82, 82,

```

```

},

```

```

{

```

```

{ // Luma
  // Size 4x4
  32, 33, 42, 55, 33, 38, 46, 57, 42, 46, 63, 75, 55, 57, 75, 92,
  // Size 8x8
  31, 32, 32, 34, 38, 46, 52, 63, 32, 32, 32, 34, 37, 44, 49, 59, 32, 32,
  35, 37, 40, 45, 49, 58, 34, 37, 42, 47, 52, 56, 65, 38, 37, 40, 47,
  54, 60, 65, 73, 46, 44, 45, 52, 60, 69, 75, 84, 52, 49, 49, 56, 65, 75,
  82, 92, 63, 59, 58, 65, 73, 84, 92, 105,
  // Size 16x16
  32, 31, 31, 31, 32, 32, 34, 36, 38, 41, 44, 48, 54, 58, 61, 65, 31, 32,
  32, 32, 32, 32, 34, 35, 38, 40, 42, 46, 51, 55, 58, 62, 31, 32, 32, 32,
  32, 32, 33, 34, 37, 38, 41, 44, 49, 53, 56, 59, 31, 32, 32, 33, 33, 33,
  35, 36, 38, 40, 42, 45, 49, 53, 56, 59, 32, 32, 32, 33, 34, 34, 36, 37,
  39, 40, 42, 45, 49, 53, 55, 59, 32, 32, 32, 33, 34, 35, 37, 38, 40, 41,
  42, 46, 49, 52, 55, 58, 34, 34, 33, 35, 36, 37, 39, 42, 44, 46, 47, 51,
  54, 57, 60, 63, 36, 35, 34, 36, 37, 38, 42, 48, 50, 52, 54, 57, 60, 63,
  65, 68, 38, 38, 37, 38, 39, 40, 44, 50, 52, 54, 57, 60, 64, 67, 69, 72,
  41, 40, 38, 40, 40, 41, 46, 52, 54, 57, 60, 63, 67, 70, 73, 75, 44, 42,
  41, 42, 42, 42, 47, 54, 57, 60, 63, 67, 71, 74, 77, 79, 48, 46, 44, 45,
  45, 46, 51, 57, 60, 63, 67, 71, 76, 79, 82, 85, 54, 51, 49, 49, 49, 49,
  54, 60, 64, 67, 71, 76, 82, 86, 89, 92, 58, 55, 53, 53, 53, 52, 57, 63,
  67, 70, 74, 79, 86, 90, 93, 97, 61, 58, 56, 56, 55, 55, 60, 65, 69, 73,
  77, 82, 89, 93, 97, 101, 65, 62, 59, 59, 59, 58, 63, 68, 72, 75, 79, 85,
  92, 97, 101, 105,
  // Size 32x32
  32, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 33, 34, 34, 36, 36, 38, 39,
  41, 44, 44, 47, 48, 50, 54, 54, 58, 59, 61, 65, 65, 70, 31, 31, 31, 32,
  32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 38, 38, 40, 42, 42, 46,
  47, 49, 52, 52, 56, 57, 59, 63, 63, 67, 31, 31, 32, 32, 32, 32, 32, 32,
  32, 32, 32, 33, 34, 34, 35, 35, 38, 38, 40, 42, 42, 45, 46, 48, 51, 51,
  55, 56, 58, 62, 62, 67, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33,
  34, 34, 35, 35, 37, 38, 39, 42, 42, 45, 45, 47, 50, 50, 54, 55, 57, 61,
  61, 65, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34,
  37, 37, 38, 41, 41, 44, 44, 46, 49, 49, 53, 54, 56, 59, 59, 64, 31, 32,
  32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34, 37, 37, 38, 41,
  41, 44, 44, 46, 49, 49, 53, 54, 56, 59, 59, 64, 31, 32, 32, 32, 32, 32,
  33, 33, 33, 33, 33, 34, 35, 35, 36, 36, 38, 39, 40, 42, 42, 44, 45, 47,
  49, 49, 53, 54, 56, 59, 59, 63, 31, 32, 32, 32, 32, 32, 33, 33, 33, 34,
  34, 35, 35, 36, 36, 36, 38, 39, 40, 42, 42, 45, 45, 47, 50, 50, 53, 54,
  56, 59, 59, 63, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34, 35, 36, 36,
  37, 37, 39, 39, 40, 42, 42, 45, 45, 47, 49, 49, 53, 54, 55, 59, 59, 63,
  32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 36, 37, 37, 38, 38, 40, 40,
  41, 42, 42, 45, 46, 47, 49, 49, 52, 53, 55, 58, 58, 62, 32, 32, 32, 32,
  32, 32, 33, 34, 34, 35, 35, 36, 37, 37, 38, 38, 40, 40, 41, 42, 42, 45,
  46, 47, 49, 49, 52, 53, 55, 58, 58, 62, 33, 33, 33, 33, 33, 34, 35,
  35, 36, 36, 38, 39, 40, 42, 42, 43, 44, 45, 46, 46, 49, 50, 51, 53, 53,
  56, 57, 59, 62, 62, 66, 34, 34, 34, 34, 33, 33, 35, 35, 36, 37, 37, 39,
  39, 41, 42, 42, 44, 45, 46, 47, 47, 50, 51, 52, 54, 54, 57, 58, 60, 63,
  63, 67, 34, 34, 34, 34, 34, 34, 35, 36, 36, 37, 37, 40, 41, 42, 45, 45,
  46, 47, 48, 50, 50, 52, 53, 54, 56, 56, 59, 60, 62, 65, 65, 69, 36, 35,
  35, 35, 34, 34, 36, 36, 37, 38, 38, 42, 42, 45, 48, 48, 50, 50, 52, 54,
  54, 56, 57, 58, 60, 60, 63, 64, 65, 68, 68, 72, 36, 35, 35, 35, 34, 34,
  36, 36, 37, 38, 38, 42, 42, 45, 48, 48, 50, 50, 52, 54, 54, 56, 57, 58,
  60, 60, 63, 64, 65, 68, 68, 72, 38, 38, 38, 37, 37, 37, 38, 38, 39, 40,
  40, 43, 44, 46, 50, 50, 52, 53, 54, 57, 57, 59, 60, 61, 64, 64, 67, 68,
  69, 72, 72, 76, 39, 38, 38, 38, 37, 37, 39, 39, 39, 40, 40, 44, 45, 47,
  50, 50, 53, 54, 55, 58, 58, 60, 61, 62, 65, 65, 68, 69, 70, 73, 73, 77,
  41, 40, 40, 39, 38, 38, 40, 40, 40, 41, 41, 45, 46, 48, 52, 52, 54, 55,
  57, 60, 60, 62, 63, 65, 67, 67, 70, 71, 73, 75, 75, 79, 44, 42, 42, 42,
  41, 41, 42, 42, 42, 42, 42, 46, 47, 50, 54, 54, 57, 58, 60, 63, 63, 66,
  67, 68, 71, 71, 74, 75, 77, 79, 79, 83, 44, 42, 42, 42, 41, 41, 42, 42,
  42, 42, 42, 46, 47, 50, 54, 54, 57, 58, 60, 63, 63, 66, 67, 68, 71, 71,
  74, 75, 77, 79, 79, 83, 47, 46, 45, 45, 44, 44, 44, 45, 45, 45, 45, 49,
  50, 52, 56, 56, 59, 60, 62, 66, 66, 69, 70, 72, 75, 75, 78, 79, 81, 84,
  84, 88, 48, 47, 46, 45, 44, 44, 45, 45, 45, 46, 46, 50, 51, 53, 57, 57,
  60, 61, 63, 67, 67, 70, 71, 73, 76, 76, 79, 80, 82, 85, 85, 89, 50, 49,
  48, 47, 46, 46, 47, 47, 47, 47, 47, 51, 52, 54, 58, 58, 61, 62, 65, 68,
  68, 72, 73, 75, 78, 78, 82, 83, 85, 88, 88, 92, 54, 52, 51, 50, 49, 49,
  49, 50, 49, 49, 49, 53, 54, 56, 60, 60, 64, 65, 67, 71, 71, 75, 76, 78,
  82, 82, 86, 87, 89, 92, 92, 96, 54, 52, 51, 50, 49, 49, 49, 50, 49, 49,
  49, 53, 54, 56, 60, 60, 64, 65, 67, 71, 71, 75, 76, 78, 82, 82, 86, 87,

```

```

89, 92, 92, 96, 58, 56, 55, 54, 53, 53, 53, 53, 53, 52, 52, 56, 57, 59,
63, 63, 67, 68, 70, 74, 74, 78, 79, 82, 86, 86, 90, 91, 93, 97, 97, 101,
59, 57, 56, 55, 54, 54, 54, 54, 54, 53, 53, 57, 58, 60, 64, 64, 68, 69,
71, 75, 75, 79, 80, 83, 87, 87, 91, 92, 94, 98, 98, 102, 61, 59, 58, 57,
56, 56, 56, 56, 55, 55, 55, 59, 60, 62, 65, 65, 69, 70, 73, 77, 77, 81,
82, 85, 89, 89, 93, 94, 97, 101, 101, 105, 65, 63, 62, 61, 59, 59, 59,
59, 59, 58, 58, 62, 63, 65, 68, 68, 72, 73, 75, 79, 79, 84, 85, 88, 92,
92, 97, 98, 101, 105, 105, 109, 65, 63, 62, 61, 59, 59, 59, 59, 59, 58,
58, 62, 63, 65, 68, 68, 72, 73, 75, 79, 79, 84, 85, 88, 92, 92, 97, 98,
101, 105, 105, 109, 70, 67, 67, 65, 64, 64, 63, 63, 63, 62, 62, 66, 67,
69, 72, 72, 76, 77, 79, 83, 83, 88, 89, 92, 96, 96, 101, 102, 105, 109,
109, 114,
// Size 4x8
32, 32, 42, 56, 32, 33, 41, 53, 32, 35, 42, 52, 34, 37, 50, 59, 38, 40,
58, 68, 44, 45, 66, 78, 50, 50, 71, 86, 61, 58, 79, 97,
// Size 8x4
32, 32, 32, 34, 38, 44, 50, 61, 32, 33, 35, 37, 40, 45, 50, 58, 42, 41,
42, 50, 58, 66, 71, 79, 56, 53, 52, 59, 68, 78, 86, 97,
// Size 8x16
32, 31, 32, 35, 39, 44, 53, 65, 31, 32, 32, 35, 38, 42, 51, 62, 31, 32,
33, 34, 37, 41, 49, 59, 31, 32, 34, 35, 38, 42, 49, 59, 32, 32, 34, 36,
39, 42, 49, 58, 32, 33, 35, 37, 40, 42, 49, 58, 34, 34, 37, 41, 44, 48,
54, 63, 36, 34, 38, 46, 50, 54, 60, 68, 38, 37, 40, 47, 52, 57, 64, 72,
41, 39, 41, 49, 54, 60, 67, 76, 44, 41, 43, 51, 57, 63, 71, 79, 48, 45,
46, 54, 60, 67, 76, 85, 53, 49, 50, 57, 64, 71, 82, 92, 57, 53, 53, 60,
67, 74, 86, 97, 61, 56, 56, 63, 69, 77, 89, 100, 65, 60, 58, 66, 72, 79,
92, 105,
// Size 16x8
32, 31, 31, 31, 32, 32, 34, 36, 38, 41, 44, 48, 53, 57, 61, 65, 31, 32,
32, 32, 32, 33, 34, 34, 37, 39, 41, 45, 49, 53, 56, 60, 32, 32, 33, 34,
34, 35, 37, 38, 40, 41, 43, 46, 50, 53, 56, 58, 35, 35, 34, 35, 36, 37,
41, 46, 47, 49, 51, 54, 57, 60, 63, 66, 39, 38, 37, 38, 39, 40, 44, 50,
52, 54, 57, 60, 64, 67, 69, 72, 44, 42, 41, 42, 42, 42, 48, 54, 57, 60,
63, 67, 71, 74, 77, 79, 53, 51, 49, 49, 49, 49, 54, 60, 64, 67, 71, 76,
82, 86, 89, 92, 65, 62, 59, 59, 58, 58, 63, 68, 72, 76, 79, 85, 92, 97,
100, 105,
// Size 16x32
32, 31, 31, 31, 32, 32, 35, 36, 39, 44, 44, 51, 53, 58, 65, 65, 31, 32,
32, 32, 32, 32, 35, 35, 38, 42, 42, 49, 52, 56, 63, 63, 31, 32, 32, 32,
32, 32, 32, 35, 35, 38, 42, 42, 49, 51, 55, 62, 62, 31, 32, 32, 32, 32, 32,
34, 35, 37, 41, 41, 48, 50, 54, 61, 61, 31, 32, 32, 32, 33, 33, 34, 34,
37, 41, 41, 47, 49, 53, 59, 59, 31, 32, 32, 33, 34, 34, 35, 36, 38, 42, 42, 48,
41, 47, 49, 53, 59, 59, 31, 32, 32, 33, 34, 34, 35, 36, 38, 42, 42, 48,
49, 53, 59, 59, 32, 32, 32, 33, 34, 34, 36, 36, 38, 42, 42, 48, 50, 53,
59, 59, 32, 32, 32, 33, 34, 34, 36, 37, 39, 42, 42, 48, 49, 53, 58, 58,
32, 32, 32, 33, 34, 35, 35, 37, 38, 40, 42, 42, 48, 49, 52, 58, 58, 32, 32,
33, 34, 35, 35, 37, 38, 40, 42, 42, 48, 49, 52, 58, 58, 33, 33, 33, 35,
36, 36, 40, 41, 43, 46, 46, 52, 53, 56, 62, 62, 34, 34, 34, 35, 37, 37,
41, 42, 44, 48, 48, 53, 54, 57, 63, 63, 34, 34, 34, 35, 37, 37, 43, 44,
46, 50, 50, 55, 56, 59, 65, 65, 36, 35, 34, 36, 38, 38, 46, 48, 50, 54,
54, 58, 60, 63, 68, 68, 36, 35, 34, 36, 38, 46, 48, 50, 54, 54, 58,
60, 63, 68, 68, 38, 37, 37, 38, 40, 40, 47, 50, 52, 57, 57, 62, 64, 67,
72, 72, 39, 38, 37, 39, 40, 40, 48, 50, 53, 58, 58, 63, 65, 68, 73, 73,
41, 39, 39, 40, 41, 41, 49, 51, 54, 60, 60, 66, 67, 70, 76, 76, 44, 41,
41, 42, 43, 43, 51, 53, 57, 63, 63, 69, 71, 74, 79, 79, 44, 41, 41, 42,
43, 43, 51, 53, 57, 63, 63, 69, 71, 74, 79, 47, 44, 44, 44, 45, 45,
53, 56, 59, 66, 66, 73, 75, 78, 84, 84, 48, 45, 45, 45, 46, 46, 54, 56,
60, 67, 67, 74, 76, 79, 85, 85, 50, 47, 46, 47, 47, 47, 55, 58, 61, 68,
68, 76, 78, 82, 88, 88, 53, 50, 49, 50, 50, 50, 57, 60, 64, 71, 71, 79,
82, 86, 92, 92, 53, 50, 49, 50, 50, 50, 57, 60, 64, 71, 71, 79, 82, 86,
92, 92, 57, 54, 53, 53, 53, 53, 60, 63, 67, 74, 74, 83, 86, 90, 97, 97,
58, 55, 54, 54, 54, 54, 61, 63, 68, 75, 75, 84, 87, 91, 98, 98, 61, 57,
56, 56, 56, 56, 63, 65, 69, 77, 77, 86, 89, 93, 100, 100, 65, 61, 60,
59, 58, 58, 66, 68, 72, 79, 79, 89, 92, 97, 105, 105, 65, 61, 60, 59,
58, 58, 66, 68, 72, 79, 79, 89, 92, 97, 105, 105, 70, 65, 64, 63, 62,
62, 70, 72, 76, 83, 83, 93, 96, 101, 109, 109,
// Size 32x16
32, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 33, 34, 34, 36, 36, 38, 39,
41, 44, 44, 47, 48, 50, 53, 53, 57, 58, 61, 65, 65, 70, 31, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 37, 38, 39, 41, 41, 44,
45, 47, 50, 50, 54, 55, 57, 61, 61, 65, 31, 32, 32, 32, 32, 32, 32, 32,

```

```

32, 33, 33, 33, 34, 34, 34, 34, 37, 37, 39, 41, 41, 44, 45, 46, 49, 49,
53, 54, 56, 60, 60, 64, 31, 32, 32, 32, 32, 33, 33, 33, 34, 34, 35,
35, 35, 36, 36, 38, 39, 40, 42, 42, 44, 45, 47, 50, 50, 53, 54, 56, 59,
59, 63, 32, 32, 32, 32, 33, 33, 34, 34, 34, 35, 35, 36, 37, 37, 38, 38,
40, 40, 41, 43, 43, 45, 46, 47, 50, 50, 53, 54, 56, 58, 58, 62, 32, 32,
32, 32, 33, 33, 34, 34, 34, 35, 35, 36, 37, 37, 38, 38, 40, 40, 41, 43,
43, 45, 46, 47, 50, 50, 53, 54, 56, 58, 58, 62, 35, 35, 35, 34, 34, 34,
35, 36, 36, 37, 37, 40, 41, 43, 46, 46, 47, 48, 49, 51, 51, 53, 54, 55,
57, 57, 60, 61, 63, 66, 66, 70, 36, 35, 35, 35, 34, 34, 36, 36, 37, 38,
38, 41, 42, 44, 48, 48, 50, 50, 51, 53, 53, 56, 56, 58, 60, 60, 63, 63,
65, 68, 68, 72, 39, 38, 38, 37, 37, 38, 38, 39, 40, 40, 43, 44, 46,
50, 50, 52, 53, 54, 57, 57, 59, 60, 61, 64, 64, 67, 68, 69, 72, 72, 76,
44, 42, 42, 41, 41, 41, 42, 42, 42, 42, 42, 46, 48, 50, 54, 54, 57, 58,
60, 63, 63, 66, 67, 68, 71, 71, 74, 75, 77, 79, 79, 83, 44, 42, 42, 41,
41, 41, 42, 42, 42, 42, 42, 46, 48, 50, 54, 54, 57, 58, 60, 63, 63, 66,
67, 68, 71, 71, 74, 75, 77, 79, 79, 83, 51, 49, 49, 48, 47, 47, 48, 48,
48, 48, 48, 52, 53, 55, 58, 58, 62, 63, 66, 69, 69, 73, 74, 76, 79, 79,
83, 84, 86, 89, 89, 93, 53, 52, 51, 50, 49, 49, 49, 50, 49, 49, 49, 53,
54, 56, 60, 60, 64, 65, 67, 71, 71, 75, 76, 78, 82, 82, 86, 87, 89, 92,
92, 96, 58, 56, 55, 54, 53, 53, 53, 53, 52, 52, 56, 57, 59, 63, 63,
67, 68, 70, 74, 74, 78, 79, 82, 86, 86, 90, 91, 93, 97, 97, 101, 65, 63,
62, 61, 59, 59, 59, 58, 58, 58, 58, 62, 63, 65, 68, 68, 72, 73, 76, 79,
79, 84, 85, 88, 92, 92, 97, 98, 100, 105, 105, 109, 65, 63, 62, 61, 59,
59, 59, 59, 58, 58, 58, 62, 63, 65, 68, 68, 72, 73, 76, 79, 79, 84, 85,
88, 92, 92, 97, 98, 100, 105, 105, 109,
// Size 4x16
31, 32, 44, 58, 32, 32, 42, 55, 32, 33, 41, 53, 32, 34, 42, 53, 32, 34,
42, 53, 32, 35, 42, 52, 34, 37, 48, 57, 35, 38, 54, 63, 37, 40, 57, 67,
39, 41, 60, 70, 41, 43, 63, 74, 45, 46, 67, 79, 50, 50, 71, 86, 54, 53,
74, 90, 57, 56, 77, 93, 61, 58, 79, 97,
// Size 16x4
31, 32, 32, 32, 32, 32, 34, 35, 37, 39, 41, 45, 50, 54, 57, 61, 32, 32,
33, 34, 34, 35, 37, 38, 40, 41, 43, 46, 50, 53, 56, 58, 44, 42, 41, 42,
42, 42, 48, 54, 57, 60, 63, 67, 71, 74, 77, 79, 58, 55, 53, 53, 53, 52,
57, 63, 67, 70, 74, 79, 86, 90, 93, 97,
// Size 8x32
32, 31, 32, 35, 39, 44, 53, 65, 31, 32, 32, 35, 38, 42, 52, 63, 31, 32,
32, 35, 38, 42, 51, 62, 31, 32, 32, 34, 37, 41, 50, 61, 31, 32, 33, 34,
37, 41, 49, 59, 31, 32, 33, 34, 37, 41, 49, 59, 31, 32, 34, 35, 38, 42,
49, 59, 32, 32, 34, 36, 38, 42, 50, 59, 32, 32, 34, 36, 39, 42, 49, 58,
32, 33, 35, 37, 40, 42, 49, 58, 32, 33, 35, 37, 40, 42, 49, 58, 33, 33,
36, 40, 43, 46, 53, 62, 34, 34, 37, 41, 44, 48, 54, 63, 34, 34, 37, 43,
46, 50, 56, 65, 36, 34, 38, 46, 50, 54, 60, 68, 36, 34, 38, 46, 50, 54,
60, 68, 38, 37, 40, 47, 52, 57, 64, 72, 39, 37, 40, 48, 53, 58, 65, 73,
41, 39, 41, 49, 54, 60, 67, 76, 44, 41, 43, 51, 57, 63, 71, 79, 44, 41,
43, 51, 57, 63, 71, 79, 47, 44, 45, 53, 59, 66, 75, 84, 48, 45, 46, 54,
60, 67, 76, 85, 50, 46, 47, 55, 61, 68, 78, 88, 53, 49, 50, 57, 64, 71,
82, 92, 53, 49, 50, 57, 64, 71, 82, 92, 57, 53, 53, 60, 67, 74, 86, 97,
58, 54, 54, 61, 68, 75, 87, 98, 61, 56, 56, 63, 69, 77, 89, 100, 65, 60,
58, 66, 72, 79, 92, 105, 65, 60, 58, 66, 72, 79, 92, 105, 70, 64, 62,
70, 76, 83, 96, 109,
// Size 32x8
32, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 33, 34, 34, 36, 36, 38, 39,
41, 44, 44, 47, 48, 50, 53, 53, 57, 58, 61, 65, 65, 70, 31, 32, 32, 32,
32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 34, 37, 37, 39, 41, 41, 44,
45, 46, 49, 49, 53, 54, 56, 60, 60, 64, 32, 32, 32, 33, 33, 34, 34,
34, 35, 35, 36, 37, 37, 38, 38, 40, 40, 41, 43, 43, 45, 46, 47, 50, 50,
53, 54, 56, 58, 58, 62, 35, 35, 35, 34, 34, 34, 35, 36, 36, 37, 37, 40,
41, 43, 46, 46, 47, 48, 49, 51, 51, 53, 54, 55, 57, 57, 60, 61, 63, 66,
66, 70, 39, 38, 38, 37, 37, 37, 38, 38, 39, 40, 40, 43, 44, 46, 50, 50,
52, 53, 54, 57, 57, 59, 60, 61, 64, 64, 67, 68, 69, 72, 72, 76, 44, 42,
42, 41, 41, 41, 42, 42, 42, 42, 42, 46, 48, 50, 54, 54, 57, 58, 60, 63,
63, 66, 67, 68, 71, 71, 74, 75, 77, 79, 79, 83, 53, 52, 51, 50, 49, 49,
49, 50, 49, 49, 49, 53, 54, 56, 60, 60, 64, 65, 67, 71, 71, 75, 76, 78,
82, 82, 86, 87, 89, 92, 92, 96, 65, 63, 62, 61, 59, 59, 59, 59, 58, 58,
58, 62, 63, 65, 68, 68, 72, 73, 76, 79, 79, 84, 85, 88, 92, 92, 97, 98,
100, 105, 105, 109,
// Size 4x32
31, 32, 44, 58, 32, 32, 42, 56, 32, 32, 42, 55, 32, 32, 41, 54, 32, 33,
41, 53, 32, 33, 41, 53, 32, 34, 42, 53, 32, 34, 42, 53,
32, 35, 42, 52, 32, 35, 42, 52, 33, 36, 46, 56, 34, 37, 48, 57, 34, 37,

```



```

50, 59, 35, 38, 54, 63, 35, 38, 54, 63, 37, 40, 57, 67, 38, 40, 58, 68,
39, 41, 60, 70, 41, 43, 63, 74, 41, 43, 63, 74, 44, 45, 66, 78, 45, 46,
67, 79, 47, 47, 68, 82, 50, 50, 71, 86, 50, 50, 71, 86, 54, 53, 74, 90,
55, 54, 75, 91, 57, 56, 77, 93, 61, 58, 79, 97, 61, 58, 79, 97, 65, 62,
83, 101,
// Size 32x4
31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 37, 38,
39, 41, 41, 44, 45, 47, 50, 50, 54, 55, 57, 61, 61, 65, 32, 32, 32, 32,
33, 33, 34, 34, 34, 35, 35, 36, 37, 37, 38, 38, 40, 40, 41, 43, 43, 45,
46, 47, 50, 50, 53, 54, 56, 58, 58, 62, 44, 42, 42, 41, 41, 41, 42, 42,
42, 42, 42, 46, 48, 50, 54, 54, 57, 58, 60, 63, 63, 66, 67, 68, 71, 71,
74, 75, 77, 79, 79, 83, 58, 56, 55, 54, 53, 53, 53, 53, 53, 52, 52, 56,
57, 59, 63, 63, 67, 68, 70, 74, 74, 78, 79, 82, 86, 86, 90, 91, 93, 97,
97, 101,
},
{
// Chroma
// Size 4x4
31, 41, 46, 51, 41, 48, 48, 51, 46, 48, 58, 62, 51, 51, 62, 71,
// Size 8x8
31, 31, 38, 44, 47, 48, 50, 55, 31, 32, 40, 44, 45, 46, 47, 52, 38, 40,
47, 47, 46, 46, 47, 50, 44, 44, 47, 50, 51, 51, 52, 54, 47, 45, 46, 51,
54, 56, 57, 60, 48, 46, 46, 51, 56, 61, 63, 66, 50, 47, 47, 52, 57, 63,
66, 70, 55, 52, 50, 54, 60, 66, 70, 76,
// Size 16x16
32, 31, 30, 33, 34, 36, 41, 49, 48, 49, 49, 50, 52, 54, 55, 57, 31, 31,
31, 34, 36, 38, 42, 47, 47, 47, 47, 48, 50, 51, 53, 54, 30, 31, 32, 34,
37, 40, 42, 46, 45, 45, 45, 46, 47, 49, 50, 52, 33, 34, 34, 37, 40, 42,
44, 47, 46, 46, 45, 46, 47, 49, 50, 51, 34, 36, 37, 40, 42, 45, 46, 47,
46, 46, 45, 46, 47, 48, 49, 50, 36, 38, 40, 42, 45, 47, 47, 48, 47, 46,
45, 46, 47, 48, 49, 50, 41, 42, 42, 44, 46, 47, 48, 50, 50, 49, 49, 50,
50, 51, 52, 53, 49, 47, 46, 47, 47, 48, 50, 53, 53, 53, 54, 54, 55,
56, 56, 48, 47, 45, 46, 46, 47, 50, 53, 54, 54, 55, 56, 57, 58, 58, 59,
49, 47, 45, 46, 46, 46, 49, 53, 54, 55, 57, 58, 59, 60, 60, 61, 49, 47,
45, 45, 45, 45, 49, 53, 55, 57, 58, 60, 61, 62, 63, 63, 50, 48, 46, 46,
46, 46, 50, 54, 56, 58, 60, 61, 63, 65, 66, 67, 52, 50, 47, 47, 47, 47,
50, 54, 57, 59, 61, 63, 66, 68, 69, 70, 54, 51, 49, 49, 48, 48, 51, 55,
58, 60, 62, 65, 68, 70, 71, 73, 55, 53, 50, 50, 49, 49, 52, 56, 58, 60,
63, 66, 69, 71, 73, 74, 57, 54, 52, 51, 50, 50, 53, 56, 59, 61, 63, 67,
70, 73, 74, 76,
// Size 32x32
32, 31, 31, 31, 30, 30, 33, 33, 34, 36, 36, 40, 41, 44, 49, 49, 48, 48,
49, 49, 49, 50, 50, 51, 52, 52, 54, 54, 55, 57, 57, 59, 31, 31, 31, 31,
31, 31, 33, 34, 36, 38, 38, 41, 42, 44, 48, 48, 47, 47, 47, 47, 47, 48,
49, 49, 50, 50, 52, 52, 53, 55, 55, 57, 31, 31, 31, 31, 31, 31, 34, 34,
36, 38, 38, 41, 42, 44, 47, 47, 47, 47, 47, 47, 48, 48, 49, 50, 50,
51, 52, 53, 54, 54, 56, 31, 31, 31, 31, 31, 34, 35, 36, 39, 39, 41,
42, 44, 47, 47, 46, 46, 46, 46, 46, 47, 47, 48, 49, 49, 50, 51, 52, 53,
53, 55, 30, 31, 31, 31, 32, 32, 34, 35, 37, 40, 40, 42, 42, 44, 46, 46,
45, 45, 45, 45, 45, 46, 46, 47, 47, 47, 49, 49, 50, 52, 52, 54, 30, 31,
31, 31, 32, 32, 34, 35, 37, 40, 40, 42, 42, 44, 46, 46, 45, 45, 45, 45,
45, 46, 46, 47, 47, 47, 49, 49, 50, 52, 52, 54, 33, 33, 34, 34, 34, 34,
37, 38, 40, 42, 42, 44, 44, 45, 47, 47, 46, 46, 46, 45, 45, 46, 46, 47,
47, 47, 49, 49, 50, 51, 51, 53, 33, 34, 34, 35, 35, 35, 38, 39, 40, 43,
43, 44, 45, 46, 47, 47, 46, 46, 46, 45, 45, 46, 46, 47, 47, 47, 49, 49,
50, 51, 51, 53, 34, 36, 36, 37, 37, 40, 40, 42, 45, 45, 45, 46, 46,
47, 47, 46, 46, 46, 45, 45, 46, 46, 47, 47, 47, 48, 49, 49, 50, 50, 52,
36, 38, 38, 39, 40, 40, 42, 43, 45, 47, 47, 47, 47, 47, 47, 48, 48, 47, 46,
46, 45, 45, 46, 46, 46, 47, 47, 48, 48, 49, 50, 50, 51, 36, 38, 38, 39,
40, 40, 42, 43, 45, 47, 47, 47, 47, 47, 48, 48, 47, 46, 46, 45, 45, 46,
46, 46, 47, 47, 48, 48, 49, 50, 50, 51, 40, 41, 41, 41, 42, 42, 44, 44,
45, 47, 47, 48, 48, 49, 50, 50, 49, 49, 48, 48, 49, 49, 49, 49, 49,
51, 51, 51, 52, 52, 54, 41, 42, 42, 42, 42, 44, 45, 46, 47, 47, 48,
48, 49, 50, 50, 50, 49, 49, 49, 49, 50, 50, 50, 50, 50, 51, 52, 52, 53,
53, 55, 44, 44, 44, 44, 44, 44, 45, 46, 46, 47, 47, 49, 49, 50, 51, 51,
51, 51, 51, 51, 51, 51, 52, 52, 53, 53, 54, 54, 54, 56, 49, 48,
47, 47, 46, 46, 47, 47, 47, 48, 48, 50, 50, 51, 53, 53, 53, 53, 53, 53,
53, 54, 54, 54, 54, 54, 55, 55, 56, 56, 56, 58, 49, 48, 47, 47, 46, 46,
47, 47, 49, 50, 51, 53, 53, 54, 54, 54, 55, 55, 56, 56, 56, 57, 57, 58, 58,
58, 59, 59, 60, 48, 47, 47, 46, 45, 45, 46, 46, 46, 46, 46, 46, 49, 49, 51,

```

53, 53, 54, 54, 55, 55, 55, 56, 56, 57, 57, 57, 58, 58, 59, 60, 60, 61,
 49, 47, 47, 46, 45, 45, 46, 46, 46, 46, 49, 49, 51, 53, 53, 54, 55,
 55, 57, 57, 57, 58, 58, 59, 59, 60, 60, 60, 61, 61, 63, 49, 47, 47, 46,
 45, 45, 45, 45, 45, 45, 45, 48, 49, 51, 53, 53, 55, 55, 57, 58, 58, 59,
 60, 60, 61, 61, 62, 62, 63, 63, 63, 65, 49, 47, 47, 46, 45, 45, 45, 45,
 45, 45, 45, 48, 49, 51, 53, 53, 55, 55, 57, 58, 58, 59, 60, 60, 61, 61,
 62, 62, 63, 63, 63, 65, 50, 48, 48, 47, 46, 46, 46, 46, 46, 46, 49,
 50, 51, 54, 54, 56, 56, 57, 59, 59, 61, 61, 62, 63, 63, 64, 64, 65, 66,
 66, 67, 50, 49, 48, 47, 46, 46, 46, 46, 46, 46, 49, 50, 51, 54, 54,
 56, 56, 58, 60, 60, 61, 61, 62, 63, 63, 65, 65, 66, 67, 67, 68, 51, 49,
 49, 48, 47, 47, 47, 47, 47, 46, 46, 49, 50, 51, 54, 54, 56, 57, 58, 60,
 60, 62, 62, 63, 65, 65, 66, 66, 67, 68, 68, 70, 52, 50, 50, 49, 47, 47,
 47, 47, 47, 47, 47, 49, 50, 52, 54, 54, 57, 57, 59, 61, 61, 63, 63, 65,
 66, 66, 68, 68, 69, 70, 70, 72, 52, 50, 50, 49, 47, 47, 47, 47, 47, 47,
 47, 49, 50, 52, 54, 54, 57, 57, 59, 61, 61, 63, 63, 65, 66, 66, 68, 68,
 69, 70, 70, 72, 54, 52, 51, 50, 49, 49, 49, 48, 48, 48, 51, 51, 53,
 55, 55, 58, 58, 60, 62, 62, 64, 65, 66, 68, 68, 70, 70, 71, 73, 73, 74,
 54, 52, 52, 51, 49, 49, 49, 49, 49, 48, 48, 51, 52, 53, 55, 55, 58, 58,
 60, 62, 62, 64, 65, 66, 68, 68, 70, 71, 72, 73, 73, 75, 55, 53, 53, 52,
 50, 50, 50, 50, 49, 49, 49, 51, 52, 54, 56, 56, 58, 59, 60, 63, 63, 65,
 66, 67, 69, 69, 71, 72, 73, 74, 74, 76, 57, 55, 54, 53, 52, 52, 51, 51,
 50, 50, 50, 52, 53, 54, 56, 56, 59, 60, 61, 63, 63, 66, 67, 68, 70, 70,
 73, 73, 74, 76, 76, 78, 57, 55, 54, 53, 52, 52, 51, 51, 50, 50, 50, 52,
 53, 54, 56, 56, 59, 60, 61, 63, 63, 66, 67, 68, 70, 70, 73, 73, 74, 76,
 76, 78, 59, 57, 56, 55, 54, 54, 53, 53, 52, 51, 51, 54, 55, 56, 58, 58,
 60, 61, 63, 65, 65, 67, 68, 70, 72, 72, 74, 75, 76, 78, 78, 80,
 // Size 4x8
 31, 38, 47, 52, 32, 40, 45, 49, 39, 47, 45, 48, 44, 47, 51, 53, 46, 47,
 56, 58, 47, 46, 59, 64, 48, 47, 61, 68, 53, 50, 64, 73,
 // Size 8x4
 31, 32, 39, 44, 46, 47, 48, 53, 38, 40, 47, 47, 47, 46, 47, 50, 47, 45,
 45, 51, 56, 59, 61, 64, 52, 49, 48, 53, 58, 64, 68, 73,
 // Size 8x16
 32, 31, 37, 45, 48, 49, 52, 57, 31, 31, 38, 45, 47, 47, 50, 54, 30, 32,
 40, 44, 45, 45, 48, 52, 33, 35, 42, 46, 46, 45, 47, 51, 35, 37, 44, 46,
 46, 45, 47, 51, 37, 40, 47, 47, 47, 45, 47, 50, 42, 43, 47, 49, 50, 49,
 50, 53, 49, 46, 48, 52, 53, 53, 54, 57, 48, 46, 47, 51, 54, 55, 57, 59,
 48, 45, 46, 51, 54, 57, 59, 61, 49, 45, 46, 51, 55, 58, 61, 64, 50, 46,
 46, 52, 56, 59, 64, 67, 52, 48, 47, 53, 57, 61, 66, 71, 54, 49, 48, 54,
 58, 62, 68, 73, 55, 51, 49, 54, 58, 63, 69, 74, 57, 52, 50, 55, 59, 64,
 70, 76,
 // Size 16x8
 32, 31, 30, 33, 35, 37, 42, 49, 48, 48, 49, 50, 52, 54, 55, 57, 31, 31,
 32, 35, 37, 40, 43, 46, 46, 45, 45, 46, 48, 49, 51, 52, 37, 38, 40, 42,
 44, 47, 47, 48, 47, 46, 46, 46, 47, 48, 49, 50, 45, 45, 44, 46, 46, 47,
 49, 52, 51, 51, 52, 53, 54, 54, 55, 48, 47, 45, 46, 46, 47, 50, 53,
 54, 54, 55, 56, 57, 58, 58, 59, 49, 47, 45, 45, 45, 45, 49, 53, 55, 57,
 58, 59, 61, 62, 63, 64, 52, 50, 48, 47, 47, 47, 50, 54, 57, 59, 61, 64,
 66, 68, 69, 70, 57, 54, 52, 51, 51, 50, 53, 57, 59, 61, 64, 67, 71, 73,
 74, 76,
 // Size 16x32
 32, 31, 31, 33, 37, 37, 45, 48, 48, 49, 49, 51, 52, 54, 57, 57, 31, 31,
 31, 34, 38, 38, 45, 47, 47, 47, 47, 50, 50, 52, 55, 55, 31, 31, 31, 34,
 38, 38, 45, 47, 47, 47, 47, 49, 50, 51, 54, 54, 31, 31, 32, 34, 39, 39,
 45, 46, 46, 46, 46, 48, 49, 51, 53, 53, 30, 32, 32, 35, 40, 40, 44, 46,
 45, 45, 45, 47, 48, 49, 52, 52, 30, 32, 32, 35, 40, 40, 44, 46, 45, 45,
 45, 47, 48, 49, 52, 52, 33, 34, 35, 37, 42, 42, 46, 47, 46, 45, 45, 47,
 47, 49, 51, 51, 33, 35, 36, 38, 43, 43, 46, 47, 46, 46, 46, 47, 47, 49,
 51, 51, 35, 37, 37, 40, 44, 44, 46, 47, 46, 45, 45, 47, 47, 48, 51, 51,
 37, 39, 40, 43, 47, 47, 47, 47, 47, 45, 45, 46, 47, 48, 50, 50, 37, 39,
 40, 43, 47, 47, 47, 47, 45, 45, 46, 47, 48, 50, 50, 41, 42, 42, 44,
 47, 47, 49, 49, 49, 48, 48, 49, 50, 51, 52, 52, 42, 42, 43, 44, 47, 47,
 49, 50, 50, 49, 49, 50, 50, 51, 53, 53, 44, 44, 44, 45, 47, 47, 50, 51,
 51, 51, 51, 52, 52, 53, 54, 54, 49, 47, 46, 47, 48, 48, 52, 53, 53, 53,
 53, 54, 54, 55, 57, 57, 49, 47, 46, 47, 48, 48, 52, 53, 53, 53, 54,
 54, 55, 57, 57, 48, 46, 46, 46, 47, 47, 51, 53, 54, 55, 56, 57, 58,
 59, 59, 48, 46, 46, 46, 47, 47, 51, 53, 54, 56, 56, 57, 57, 58, 60, 60,
 48, 46, 45, 46, 46, 46, 51, 53, 54, 57, 57, 58, 59, 60, 61, 61, 49, 46,
 45, 45, 46, 46, 51, 53, 55, 58, 58, 61, 61, 62, 64, 64, 49, 46, 45, 45,
 46, 46, 51, 53, 55, 58, 58, 61, 61, 62, 64, 64, 50, 47, 46, 46, 46, 46,
 52, 54, 56, 59, 59, 62, 63, 64, 66, 66, 50, 47, 46, 46, 46, 52, 54,

56, 59, 59, 63, 64, 65, 67, 67, 51, 48, 47, 47, 47, 47, 52, 54, 56, 60,
60, 64, 65, 66, 68, 68, 52, 48, 48, 47, 47, 47, 53, 54, 57, 61, 61, 65,
66, 68, 71, 71, 52, 48, 48, 47, 47, 47, 53, 54, 57, 61, 61, 65, 66, 68,
71, 71, 54, 50, 49, 49, 48, 48, 54, 55, 58, 62, 62, 67, 68, 70, 73, 73,
54, 51, 50, 49, 49, 49, 54, 55, 58, 62, 62, 67, 68, 70, 73, 73, 55, 51,
51, 50, 49, 49, 54, 56, 58, 63, 63, 68, 69, 71, 74, 74, 57, 53, 52, 51,
50, 50, 55, 56, 59, 64, 64, 69, 70, 73, 76, 76, 57, 53, 52, 51, 50, 50,
55, 56, 59, 64, 64, 69, 70, 73, 76, 76, 59, 55, 54, 53, 52, 52, 57, 58,
61, 65, 65, 70, 72, 74, 78, 78,
// Size 32x16
32, 31, 31, 31, 30, 30, 33, 33, 35, 37, 37, 41, 42, 44, 49, 49, 48, 48,
48, 49, 49, 50, 50, 51, 52, 52, 54, 54, 55, 57, 57, 59, 31, 31, 31, 31,
32, 32, 34, 35, 37, 39, 39, 42, 42, 44, 47, 47, 46, 46, 46, 46, 46, 47,
47, 48, 48, 48, 50, 51, 51, 53, 53, 55, 31, 31, 31, 32, 32, 32, 35, 36,
37, 40, 40, 42, 43, 44, 46, 46, 46, 46, 45, 45, 45, 46, 46, 47, 48, 48,
49, 50, 51, 52, 52, 54, 33, 34, 34, 34, 35, 35, 37, 38, 40, 43, 43, 44,
44, 45, 47, 47, 46, 46, 46, 45, 45, 46, 46, 47, 47, 47, 49, 49, 50, 51,
51, 53, 37, 38, 38, 39, 40, 40, 42, 43, 44, 47, 47, 47, 47, 47, 48, 48,
47, 47, 46, 46, 46, 46, 46, 47, 47, 47, 48, 49, 49, 50, 50, 52, 37, 38,
38, 39, 40, 40, 42, 43, 44, 47, 47, 47, 47, 48, 48, 47, 47, 46, 46,
46, 46, 46, 47, 47, 47, 48, 49, 49, 50, 50, 52, 45, 45, 45, 44, 44,
46, 46, 46, 47, 47, 49, 49, 50, 52, 52, 51, 51, 51, 51, 52, 52, 52,
53, 53, 54, 54, 54, 55, 55, 57, 48, 47, 47, 46, 46, 46, 47, 47, 47, 47,
47, 49, 50, 51, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 55, 55,
56, 56, 56, 58, 48, 47, 47, 46, 45, 45, 46, 46, 46, 47, 47, 49, 50, 51,
53, 53, 54, 54, 54, 55, 55, 56, 56, 56, 57, 57, 58, 58, 59, 59, 61,
49, 47, 47, 46, 45, 45, 45, 46, 45, 45, 45, 48, 49, 51, 53, 53, 55, 56,
57, 58, 58, 59, 59, 60, 61, 61, 62, 62, 63, 64, 64, 65, 49, 47, 47, 46,
45, 45, 45, 46, 45, 45, 45, 48, 49, 51, 53, 53, 55, 56, 57, 58, 58, 59,
59, 60, 61, 61, 62, 62, 63, 64, 64, 65, 51, 50, 49, 48, 47, 47, 47, 47,
47, 46, 46, 49, 50, 52, 54, 54, 56, 57, 58, 61, 61, 62, 63, 64, 65, 65,
67, 67, 68, 69, 69, 70, 52, 50, 50, 49, 48, 48, 47, 47, 47, 47, 50,
50, 52, 54, 54, 57, 57, 59, 61, 61, 63, 64, 65, 66, 66, 68, 68, 69, 70,
70, 72, 54, 52, 51, 51, 49, 49, 49, 49, 48, 48, 48, 51, 51, 53, 55, 55,
58, 58, 60, 62, 62, 64, 65, 66, 68, 68, 70, 70, 71, 73, 73, 74, 57, 55,
54, 53, 52, 52, 51, 51, 51, 50, 52, 53, 54, 57, 57, 59, 60, 61, 64,
64, 66, 67, 68, 71, 71, 73, 73, 74, 76, 76, 78, 57, 55, 54, 53, 52, 52,
51, 51, 51, 50, 50, 52, 53, 54, 57, 57, 59, 60, 61, 64, 64, 66, 67, 68,
71, 71, 73, 73, 74, 76, 76, 78,
// Size 4x16
31, 37, 49, 54, 31, 38, 47, 51, 32, 40, 45, 49, 34, 42, 45, 49, 37, 44,
45, 48, 39, 47, 45, 48, 42, 47, 49, 51, 47, 48, 53, 55, 46, 47, 55, 58,
46, 46, 57, 60, 46, 46, 58, 62, 47, 46, 59, 65, 48, 47, 61, 68, 50, 48,
62, 70, 51, 49, 63, 71, 53, 50, 64, 73,
// Size 16x4
31, 31, 32, 34, 37, 39, 42, 47, 46, 46, 46, 47, 48, 50, 51, 53, 37, 38,
40, 42, 44, 47, 47, 48, 47, 46, 46, 46, 47, 48, 49, 50, 49, 47, 45, 45,
45, 45, 49, 53, 55, 57, 58, 59, 61, 62, 63, 64, 54, 51, 49, 49, 48, 48,
51, 55, 58, 60, 62, 65, 68, 70, 71, 73,
// Size 8x32
32, 31, 37, 45, 48, 49, 52, 57, 31, 31, 38, 45, 47, 47, 50, 55, 31, 31,
38, 45, 47, 47, 50, 54, 31, 32, 39, 45, 46, 46, 49, 53, 30, 32, 40, 44,
45, 45, 48, 52, 30, 32, 40, 44, 45, 45, 48, 52, 33, 35, 42, 46, 46, 45,
47, 51, 33, 36, 43, 46, 46, 46, 47, 51, 35, 37, 44, 46, 46, 45, 47, 51,
37, 40, 47, 47, 47, 45, 47, 50, 37, 40, 47, 47, 47, 45, 47, 50, 41, 42,
47, 49, 49, 48, 50, 52, 42, 43, 47, 49, 50, 49, 50, 53, 44, 44, 47, 50,
51, 51, 52, 54, 49, 46, 48, 52, 53, 53, 54, 57, 49, 46, 48, 52, 53, 53,
54, 57, 48, 46, 47, 51, 54, 55, 57, 59, 48, 46, 47, 51, 54, 56, 57, 60,
48, 45, 46, 51, 54, 57, 59, 61, 49, 45, 46, 51, 55, 58, 61, 64, 49, 45,
46, 51, 55, 58, 61, 64, 50, 46, 46, 52, 56, 59, 63, 66, 50, 46, 46, 52,
56, 59, 64, 67, 51, 47, 47, 52, 56, 60, 65, 68, 52, 48, 47, 53, 57, 61,
66, 71, 52, 48, 47, 53, 57, 61, 66, 71, 54, 49, 48, 54, 58, 62, 68, 73,
54, 50, 49, 54, 58, 62, 68, 73, 55, 51, 49, 54, 58, 63, 69, 74, 57, 52,
50, 55, 59, 64, 70, 76, 57, 52, 50, 55, 59, 64, 70, 76, 59, 54, 52, 57,
61, 65, 72, 78,
// Size 32x8
32, 31, 31, 31, 30, 30, 33, 33, 35, 37, 37, 41, 42, 44, 49, 49, 48, 48,
48, 49, 49, 50, 50, 51, 52, 52, 54, 54, 55, 57, 57, 59, 31, 31, 31, 32,
32, 32, 35, 36, 37, 40, 40, 42, 43, 44, 46, 46, 46, 46, 45, 45, 45, 46,
46, 47, 48, 48, 49, 50, 51, 52, 52, 54, 37, 38, 38, 39, 40, 40, 42, 43,
44, 47, 47, 47, 47, 47, 48, 48, 47, 47, 46, 46, 46, 46, 46, 47, 47,

```

48, 49, 49, 50, 50, 52, 45, 45, 45, 45, 44, 44, 46, 46, 46, 47, 47, 49,
49, 50, 52, 52, 51, 51, 51, 51, 51, 52, 52, 52, 53, 53, 54, 54, 54, 55,
55, 57, 48, 47, 47, 46, 45, 45, 46, 46, 46, 47, 47, 49, 50, 51, 53, 53,
54, 54, 54, 55, 55, 56, 56, 56, 57, 57, 58, 58, 58, 59, 59, 61, 49, 47,
47, 46, 45, 45, 45, 46, 45, 45, 45, 48, 49, 51, 53, 53, 55, 56, 57, 58,
58, 59, 59, 60, 61, 61, 62, 62, 63, 64, 64, 65, 52, 50, 50, 49, 48, 48,
47, 47, 47, 47, 47, 50, 50, 52, 54, 54, 57, 57, 59, 61, 61, 63, 64, 65,
66, 66, 68, 68, 69, 70, 70, 72, 57, 55, 54, 53, 52, 52, 51, 51, 51, 50,
50, 52, 53, 54, 57, 57, 59, 60, 61, 64, 64, 66, 67, 68, 71, 71, 73, 73,
74, 76, 76, 78,
// Size 4x32
31, 37, 49, 54, 31, 38, 47, 52, 31, 38, 47, 51, 31, 39, 46, 51, 32, 40,
45, 49, 32, 40, 45, 49, 34, 42, 45, 49, 35, 43, 46, 49, 37, 44, 45, 48,
39, 47, 45, 48, 39, 47, 45, 48, 42, 47, 48, 51, 42, 47, 49, 51, 44, 47,
51, 53, 47, 48, 53, 55, 47, 48, 53, 55, 46, 47, 55, 58, 46, 47, 56, 58,
46, 46, 57, 60, 46, 46, 58, 62, 46, 46, 58, 62, 47, 46, 59, 64, 47, 46,
59, 65, 48, 47, 60, 66, 48, 47, 61, 68, 48, 47, 61, 68, 50, 48, 62, 70,
51, 49, 62, 70, 51, 49, 63, 71, 53, 50, 64, 73, 53, 50, 64, 73, 55, 52,
65, 74,
// Size 32x4
31, 31, 31, 31, 32, 32, 34, 35, 37, 39, 39, 42, 42, 44, 47, 47, 46, 46,
46, 46, 46, 47, 47, 48, 48, 48, 50, 51, 51, 53, 53, 55, 37, 38, 38, 39,
40, 40, 42, 43, 44, 47, 47, 47, 47, 47, 48, 48, 47, 47, 46, 46, 46, 46,
46, 47, 47, 47, 48, 49, 49, 50, 50, 52, 49, 47, 47, 46, 45, 45, 45, 46,
45, 45, 45, 48, 49, 51, 53, 53, 55, 56, 57, 58, 58, 59, 59, 60, 61, 61,
62, 62, 63, 64, 64, 65, 54, 52, 51, 51, 49, 49, 49, 49, 48, 48, 48, 51,
51, 53, 55, 55, 58, 58, 60, 62, 62, 64, 65, 66, 68, 68, 70, 70, 71, 73,
73, 74,
},
{
{ // Luma
// Size 4x4
32, 32, 38, 51, 32, 35, 40, 49, 38, 40, 54, 64, 51, 49, 64, 81,
// Size 8x8
31, 32, 32, 34, 35, 41, 47, 53, 32, 32, 32, 33, 34, 40, 44, 50, 32, 32,
34, 35, 37, 41, 45, 51, 34, 33, 35, 39, 42, 47, 51, 55, 35, 34, 37, 42,
48, 53, 57, 61, 41, 40, 41, 47, 53, 60, 65, 70, 47, 44, 45, 51, 57, 65,
71, 77, 53, 50, 51, 55, 61, 70, 77, 85,
// Size 16x16
32, 31, 31, 31, 31, 32, 32, 34, 36, 38, 39, 44, 47, 49, 54, 59, 31, 32,
32, 32, 32, 32, 33, 34, 35, 37, 38, 42, 45, 47, 51, 56, 31, 32, 32, 32,
32, 32, 33, 33, 34, 36, 37, 41, 44, 46, 50, 54, 31, 32, 32, 32, 32, 33,
33, 34, 35, 36, 38, 41, 44, 45, 49, 54, 31, 32, 32, 32, 33, 34, 34, 35,
36, 38, 39, 42, 45, 46, 50, 54, 32, 32, 32, 33, 34, 35, 36, 37, 38, 39,
40, 42, 45, 46, 49, 53, 32, 33, 33, 34, 36, 36, 38, 40, 41, 42, 44,
47, 48, 51, 55, 34, 34, 33, 34, 35, 37, 38, 39, 42, 44, 45, 47, 50, 51,
54, 58, 36, 35, 34, 35, 36, 38, 40, 42, 48, 50, 50, 54, 56, 57, 60, 64,
38, 37, 36, 36, 38, 39, 41, 44, 50, 51, 52, 56, 58, 60, 63, 67, 39, 38,
37, 38, 39, 40, 42, 45, 50, 52, 54, 58, 60, 62, 65, 69, 44, 42, 41, 41,
42, 42, 44, 47, 54, 56, 58, 63, 66, 68, 71, 75, 47, 45, 44, 44, 45, 45,
47, 50, 56, 58, 60, 66, 69, 71, 75, 79, 49, 47, 46, 45, 46, 46, 48, 51,
57, 60, 62, 68, 71, 73, 77, 81, 54, 51, 50, 49, 50, 49, 51, 54, 60, 63,
65, 71, 75, 77, 82, 87, 59, 56, 54, 54, 54, 53, 55, 58, 64, 67, 69, 75,
79, 81, 87, 92,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 34, 34, 35, 36, 36,
38, 39, 39, 42, 44, 44, 47, 48, 49, 53, 54, 55, 59, 59, 31, 31, 31, 31,
32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 35, 35, 37, 39, 39, 41,
43, 43, 46, 47, 48, 51, 52, 53, 57, 57, 31, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 33, 34, 34, 34, 35, 35, 37, 38, 38, 41, 42, 43, 45, 46,
47, 51, 51, 53, 56, 56, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
33, 34, 34, 34, 35, 35, 37, 38, 38, 41, 42, 42, 45, 46, 47, 51, 51, 52,
56, 56, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34,
34, 34, 36, 37, 37, 40, 41, 41, 44, 45, 46, 49, 50, 51, 54, 54, 31, 32,
32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34, 36, 37,
37, 40, 41, 41, 44, 44, 45, 49, 49, 50, 54, 54, 31, 32, 32, 32, 32, 32,
32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 35, 35, 36, 38, 38, 40, 41, 41,
44, 45, 45, 49, 49, 50, 54, 54, 31, 32, 32, 32, 32, 32, 32, 33, 33, 33,
34, 34, 34, 35, 35, 35, 36, 36, 38, 39, 39, 41, 42, 42, 44, 45, 46, 49,
50, 51, 54, 54, 31, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 35,

```

```

35, 36, 36, 36, 38, 39, 39, 41, 42, 42, 45, 45, 46, 49, 50, 51, 54, 54,
32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 35, 35, 35, 36, 37, 37,
38, 39, 39, 41, 42, 42, 45, 45, 46, 49, 49, 51, 54, 54, 32, 32, 32, 32,
32, 32, 33, 34, 34, 34, 35, 35, 36, 37, 37, 37, 38, 38, 39, 40, 40, 42,
42, 43, 45, 46, 46, 49, 49, 50, 53, 53, 32, 32, 32, 32, 32, 33, 34,
34, 34, 35, 35, 36, 37, 37, 37, 38, 38, 39, 40, 40, 42, 42, 43, 45, 46,
46, 49, 49, 50, 53, 53, 32, 33, 33, 33, 33, 33, 34, 34, 35, 36, 36,
36, 38, 38, 39, 40, 40, 41, 42, 42, 44, 44, 45, 47, 47, 48, 51, 51, 52,
55, 55, 34, 34, 34, 34, 33, 33, 34, 35, 35, 35, 37, 37, 38, 39, 39, 41,
42, 42, 44, 45, 45, 47, 47, 48, 50, 51, 51, 54, 54, 55, 58, 58, 34, 34,
34, 34, 33, 33, 34, 35, 35, 35, 37, 37, 38, 39, 39, 41, 42, 42, 44, 45,
45, 47, 47, 48, 50, 51, 51, 54, 54, 55, 58, 58, 35, 34, 34, 34, 34, 34,
34, 35, 36, 36, 37, 37, 39, 41, 41, 43, 45, 45, 47, 47, 47, 49, 50, 51,
53, 53, 54, 57, 57, 58, 61, 61, 36, 35, 35, 35, 34, 34, 35, 36, 36, 37,
38, 38, 40, 42, 42, 45, 48, 48, 50, 50, 50, 53, 54, 54, 56, 57, 57, 59,
60, 61, 64, 64, 36, 35, 35, 35, 34, 34, 35, 36, 36, 37, 38, 38, 40, 42,
42, 45, 48, 48, 50, 50, 53, 54, 54, 56, 57, 57, 59, 60, 61, 64, 64,
38, 37, 37, 37, 36, 36, 36, 38, 38, 38, 39, 39, 41, 44, 44, 47, 50, 50,
51, 52, 52, 55, 56, 56, 58, 59, 60, 62, 63, 64, 67, 67, 39, 39, 38, 38,
37, 37, 38, 39, 39, 39, 40, 40, 42, 45, 45, 47, 50, 50, 52, 54, 54, 56,
58, 58, 60, 61, 62, 64, 65, 66, 69, 69, 39, 38, 38, 37, 37, 38, 39,
39, 39, 40, 40, 42, 45, 45, 47, 50, 50, 52, 54, 54, 56, 58, 58, 60, 61,
62, 64, 65, 66, 69, 69, 42, 41, 41, 41, 40, 40, 40, 41, 41, 41, 42, 42,
44, 47, 47, 49, 53, 53, 55, 56, 56, 60, 61, 62, 64, 65, 66, 69, 69, 70,
73, 73, 44, 43, 42, 42, 41, 41, 41, 42, 42, 42, 42, 42, 44, 47, 47, 50,
54, 54, 56, 58, 58, 61, 63, 64, 66, 67, 68, 71, 71, 72, 75, 75, 44, 43,
43, 42, 41, 41, 41, 42, 42, 42, 43, 43, 45, 48, 48, 51, 54, 54, 56, 58,
58, 62, 64, 64, 66, 67, 68, 71, 72, 73, 76, 76, 47, 46, 45, 45, 44, 44,
44, 44, 45, 45, 45, 45, 47, 50, 50, 53, 56, 56, 58, 60, 60, 64, 66, 66,
69, 70, 71, 74, 75, 76, 79, 79, 48, 47, 46, 46, 45, 44, 45, 45, 45, 45,
46, 46, 47, 51, 51, 53, 57, 57, 59, 61, 61, 65, 67, 67, 70, 71, 72, 75,
76, 77, 80, 80, 49, 48, 47, 47, 46, 45, 45, 46, 46, 46, 46, 46, 48, 51,
51, 54, 57, 57, 60, 62, 62, 66, 68, 68, 71, 72, 73, 77, 77, 78, 81, 81,
53, 51, 51, 51, 49, 49, 49, 49, 49, 49, 49, 49, 51, 54, 54, 57, 59, 59,
62, 64, 64, 69, 71, 71, 74, 75, 77, 81, 81, 83, 86, 86, 54, 52, 51, 51,
50, 49, 49, 50, 50, 49, 49, 49, 51, 54, 54, 57, 60, 60, 63, 65, 65, 69,
71, 72, 75, 76, 77, 81, 82, 83, 87, 87, 55, 53, 53, 52, 51, 50, 50, 51,
51, 51, 50, 50, 52, 55, 55, 58, 61, 61, 64, 66, 66, 70, 72, 73, 76, 77,
78, 83, 83, 85, 88, 88, 59, 57, 56, 56, 54, 54, 54, 54, 54, 54, 53, 53,
55, 58, 58, 61, 64, 64, 67, 69, 69, 73, 75, 76, 79, 80, 81, 86, 87, 88,
92, 92, 59, 57, 56, 56, 54, 54, 54, 54, 54, 53, 53, 55, 58, 58, 61,
64, 64, 67, 69, 69, 73, 75, 76, 79, 80, 81, 86, 87, 88, 92, 92,
// Size 4x8
32, 32, 37, 52, 32, 33, 36, 49, 32, 34, 38, 49, 34, 37, 44, 54, 35, 38,
49, 60, 40, 42, 55, 69, 46, 46, 59, 76, 52, 51, 64, 83,
// Size 8x4
32, 32, 32, 34, 35, 40, 46, 52, 32, 33, 34, 37, 38, 42, 46, 51, 37, 36,
38, 44, 49, 55, 59, 64, 52, 49, 49, 54, 60, 69, 76, 83,
// Size 8x16
32, 31, 32, 32, 36, 44, 47, 53, 31, 32, 32, 33, 35, 42, 45, 51, 31, 32,
32, 33, 35, 41, 44, 49, 31, 32, 33, 33, 35, 41, 44, 49, 32, 32, 34, 34,
36, 42, 45, 50, 32, 33, 35, 36, 38, 42, 45, 49, 32, 33, 35, 36, 40, 44,
47, 51, 34, 34, 36, 38, 42, 48, 50, 54, 36, 34, 37, 40, 48, 54, 56, 60,
38, 36, 39, 41, 49, 56, 58, 63, 39, 37, 40, 42, 50, 58, 60, 65, 44, 41,
42, 45, 53, 63, 66, 71, 47, 44, 45, 47, 56, 66, 69, 75, 49, 46, 47, 48,
57, 67, 71, 77, 53, 49, 50, 51, 60, 71, 75, 82, 58, 54, 54, 55, 63, 75,
79, 87,
// Size 16x8
32, 31, 31, 31, 32, 32, 32, 34, 36, 38, 39, 44, 47, 49, 53, 58, 31, 32,
32, 32, 32, 33, 33, 34, 34, 36, 37, 41, 44, 46, 49, 54, 32, 32, 32, 33,
34, 35, 35, 36, 37, 39, 40, 42, 45, 47, 50, 54, 32, 33, 33, 34, 36,
36, 38, 40, 41, 42, 45, 47, 48, 51, 55, 36, 35, 35, 35, 36, 38, 40, 42,
48, 49, 50, 53, 56, 57, 60, 63, 44, 42, 41, 41, 42, 42, 44, 48, 54, 56,
58, 63, 66, 67, 71, 75, 47, 45, 44, 44, 45, 45, 47, 50, 56, 58, 60, 66,
69, 71, 75, 79, 53, 51, 49, 49, 50, 49, 51, 54, 60, 63, 65, 71, 75, 77,
82, 87,
// Size 16x32
32, 31, 31, 31, 32, 32, 32, 35, 36, 38, 44, 44, 47, 53, 53, 59, 31, 32,
32, 32, 32, 32, 33, 35, 35, 37, 43, 43, 46, 52, 52, 57, 31, 32, 32, 32,
32, 32, 33, 35, 35, 37, 42, 42, 45, 51, 51, 56, 31, 32, 32, 32, 32, 32,
33, 35, 35, 37, 42, 42, 45, 51, 51, 56, 31, 32, 32, 32, 32, 33, 34,

```

```

35, 36, 41, 41, 44, 49, 49, 54, 31, 32, 32, 32, 32, 33, 33, 34, 34, 36,
41, 41, 44, 49, 49, 54, 31, 32, 32, 32, 33, 33, 33, 35, 35, 36, 41, 41,
44, 49, 49, 54, 32, 32, 32, 32, 33, 34, 34, 36, 36, 38, 42, 42, 45, 49,
49, 54, 32, 32, 32, 33, 34, 34, 34, 36, 36, 38, 42, 42, 45, 50, 50, 54,
32, 32, 32, 33, 34, 34, 35, 37, 37, 38, 42, 42, 45, 49, 49, 54, 32, 32,
33, 33, 35, 35, 36, 38, 38, 39, 42, 42, 45, 49, 49, 53, 32, 32, 33, 33,
35, 35, 36, 38, 38, 39, 42, 42, 45, 49, 49, 53, 32, 33, 33, 33, 35, 36,
36, 39, 40, 41, 44, 44, 47, 51, 51, 55, 34, 34, 34, 34, 36, 37, 38, 42,
42, 44, 48, 48, 50, 54, 54, 58, 34, 34, 34, 34, 36, 37, 38, 42, 42, 44,
48, 48, 50, 54, 54, 58, 35, 34, 34, 34, 37, 37, 39, 44, 45, 46, 50, 50,
53, 57, 57, 61, 36, 35, 34, 35, 37, 38, 40, 47, 48, 49, 54, 54, 56, 60,
60, 64, 36, 35, 34, 35, 37, 38, 40, 47, 48, 49, 54, 54, 56, 60, 60, 64,
38, 37, 36, 37, 39, 40, 41, 48, 49, 51, 56, 56, 58, 63, 63, 67, 39, 38,
37, 38, 40, 40, 42, 49, 50, 52, 58, 58, 60, 65, 65, 69, 39, 38, 37, 38,
40, 40, 42, 49, 50, 52, 58, 58, 60, 65, 65, 69, 42, 40, 40, 40, 42, 42,
44, 51, 52, 55, 61, 61, 64, 69, 69, 73, 44, 42, 41, 41, 42, 43, 45, 52,
53, 56, 63, 63, 66, 71, 71, 75, 44, 42, 41, 41, 43, 43, 45, 52, 54, 56,
63, 63, 66, 72, 72, 76, 47, 45, 44, 44, 45, 45, 47, 54, 56, 58, 66, 66,
69, 75, 75, 79, 48, 46, 45, 45, 46, 46, 48, 55, 56, 59, 67, 67, 70, 76,
76, 80, 49, 47, 46, 46, 47, 47, 48, 56, 57, 60, 67, 67, 71, 77, 77, 81,
53, 50, 49, 49, 49, 51, 58, 59, 62, 71, 71, 74, 81, 81, 86, 53, 51,
49, 49, 50, 50, 51, 59, 60, 63, 71, 71, 75, 82, 82, 87, 55, 52, 51, 51,
51, 51, 53, 60, 61, 64, 72, 72, 76, 83, 83, 88, 58, 55, 54, 54, 54, 54,
55, 62, 63, 67, 75, 75, 79, 87, 87, 92, 58, 55, 54, 54, 54, 55, 62,
63, 67, 75, 79, 87, 87, 92,
// Size 32x16
32, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 34, 34, 35, 36, 36,
38, 39, 39, 42, 44, 44, 47, 48, 49, 53, 53, 55, 58, 58, 31, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 35, 35, 37, 38, 38, 40,
42, 42, 45, 46, 47, 50, 51, 52, 55, 55, 31, 32, 32, 32, 32, 32, 32,
32, 32, 33, 33, 34, 34, 34, 34, 34, 36, 37, 37, 40, 41, 41, 44, 45,
46, 49, 49, 51, 54, 54, 31, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33,
33, 34, 34, 34, 35, 35, 37, 38, 38, 40, 41, 41, 44, 45, 46, 49, 49, 51,
54, 54, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 35, 35, 35, 36, 36, 37,
37, 37, 39, 40, 40, 42, 42, 43, 45, 46, 47, 49, 50, 51, 54, 54, 32, 32,
32, 32, 32, 33, 33, 34, 34, 34, 35, 35, 36, 37, 37, 38, 38, 40, 40,
40, 42, 43, 43, 45, 46, 47, 49, 50, 51, 54, 54, 32, 33, 33, 33, 33, 33,
33, 34, 34, 35, 36, 36, 36, 38, 38, 39, 40, 40, 41, 42, 42, 44, 45, 45,
47, 48, 48, 51, 51, 53, 55, 55, 35, 35, 35, 35, 34, 34, 35, 36, 36, 37,
38, 38, 39, 42, 42, 44, 47, 47, 48, 49, 49, 51, 52, 52, 54, 55, 56, 58,
59, 60, 62, 62, 36, 35, 35, 35, 35, 36, 36, 37, 38, 38, 40, 42,
42, 45, 48, 48, 49, 50, 50, 52, 53, 54, 56, 56, 57, 59, 60, 61, 63, 63,
38, 37, 37, 37, 36, 36, 36, 38, 38, 38, 39, 39, 41, 44, 44, 46, 49, 49,
51, 52, 52, 55, 56, 56, 58, 59, 60, 62, 63, 64, 67, 67, 44, 43, 42, 42,
41, 41, 41, 42, 42, 42, 42, 42, 44, 48, 48, 50, 54, 54, 56, 58, 58, 61,
63, 63, 66, 67, 71, 71, 72, 75, 75, 44, 43, 42, 42, 41, 41, 41, 42,
42, 42, 42, 44, 48, 48, 50, 54, 54, 56, 58, 58, 61, 63, 63, 66, 67,
67, 71, 71, 72, 75, 75, 47, 46, 45, 45, 44, 44, 44, 45, 45, 45, 45,
47, 50, 50, 53, 56, 56, 58, 60, 60, 64, 66, 66, 69, 70, 71, 74, 75, 76,
79, 79, 53, 52, 51, 51, 49, 49, 49, 49, 50, 49, 49, 49, 51, 54, 54, 57,
60, 60, 63, 65, 65, 69, 71, 72, 75, 76, 77, 81, 82, 83, 87, 87, 53, 52,
51, 51, 49, 49, 49, 49, 50, 49, 49, 49, 51, 54, 54, 57, 60, 60, 63, 65,
65, 69, 71, 72, 75, 76, 77, 81, 82, 83, 87, 87, 59, 57, 56, 56, 54, 54,
54, 54, 54, 54, 53, 53, 55, 58, 58, 61, 64, 64, 67, 69, 69, 73, 75, 76,
79, 80, 81, 86, 87, 88, 92, 92,
// Size 4x16
31, 32, 38, 53, 32, 32, 37, 51, 32, 32, 36, 49, 32, 33, 36, 49, 32, 34,
38, 50, 32, 35, 39, 49, 33, 36, 41, 51, 34, 37, 44, 54, 35, 38, 49, 60,
37, 40, 51, 63, 38, 40, 52, 65, 42, 43, 56, 71, 45, 45, 58, 75, 47, 47,
60, 77, 51, 50, 63, 82, 55, 54, 67, 87,
// Size 16x4
31, 32, 32, 32, 32, 32, 33, 34, 35, 37, 38, 42, 45, 47, 51, 55, 32, 32,
32, 33, 34, 35, 36, 37, 38, 40, 40, 43, 45, 47, 50, 54, 38, 37, 36, 36,
38, 39, 41, 44, 49, 51, 52, 56, 58, 60, 63, 67, 53, 51, 49, 49, 50, 49,
51, 54, 60, 63, 65, 71, 75, 77, 82, 87,
// Size 8x32
32, 31, 32, 32, 36, 44, 47, 53, 31, 32, 32, 33, 35, 43, 46, 52, 31, 32,
32, 33, 35, 42, 45, 51, 31, 32, 32, 33, 35, 42, 45, 51, 31, 32, 32, 33,
35, 41, 44, 49, 31, 32, 32, 33, 34, 41, 44, 49, 31, 32, 33, 33, 35, 41,
44, 49, 32, 32, 33, 34, 36, 42, 45, 49, 32, 32, 34, 34, 36, 42, 45, 50,
32, 32, 34, 35, 37, 42, 45, 49, 32, 33, 35, 36, 38, 42, 45, 49, 32, 33,

```

```

35, 36, 38, 42, 45, 49, 32, 33, 35, 36, 40, 44, 47, 51, 34, 34, 36, 38,
42, 48, 50, 54, 34, 34, 36, 38, 42, 48, 50, 54, 35, 34, 37, 39, 45, 50,
53, 57, 36, 34, 37, 40, 48, 54, 56, 60, 36, 34, 37, 40, 48, 54, 56, 60,
38, 36, 39, 41, 49, 56, 58, 63, 39, 37, 40, 42, 50, 58, 60, 65, 39, 37,
40, 42, 50, 58, 60, 65, 42, 40, 42, 44, 52, 61, 64, 69, 44, 41, 42, 45,
53, 63, 66, 71, 44, 41, 43, 45, 54, 63, 66, 72, 47, 44, 45, 47, 56, 66,
69, 75, 48, 45, 46, 48, 56, 67, 70, 76, 49, 46, 47, 48, 57, 67, 71, 77,
53, 49, 49, 51, 59, 71, 74, 81, 53, 49, 50, 51, 60, 71, 75, 82, 55, 51,
51, 53, 61, 72, 76, 83, 58, 54, 54, 55, 63, 75, 79, 87, 58, 54, 54, 55,
63, 75, 79, 87,
// Size 32x8
32, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 34, 34, 35, 36, 36,
38, 39, 39, 42, 44, 44, 47, 48, 49, 53, 53, 55, 58, 58, 31, 32, 32, 32,
32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 34, 34, 36, 37, 37, 40,
41, 41, 44, 45, 46, 49, 49, 51, 54, 54, 32, 32, 32, 32, 32, 32, 33, 33,
34, 34, 35, 35, 35, 36, 36, 37, 37, 37, 39, 40, 40, 42, 42, 43, 45, 46,
47, 49, 50, 51, 54, 54, 32, 33, 33, 33, 33, 33, 34, 34, 35, 36, 36,
36, 38, 38, 39, 40, 40, 41, 42, 42, 44, 45, 45, 47, 48, 48, 51, 51, 53,
55, 55, 36, 35, 35, 35, 35, 34, 35, 36, 36, 37, 38, 38, 40, 42, 42, 45,
48, 48, 49, 50, 50, 52, 53, 54, 56, 56, 57, 59, 60, 61, 63, 63, 44, 43,
42, 42, 41, 41, 41, 42, 42, 42, 42, 42, 44, 48, 48, 50, 54, 54, 56, 58,
58, 61, 63, 63, 66, 67, 67, 71, 71, 72, 75, 75, 47, 46, 45, 45, 44, 44,
44, 45, 45, 45, 45, 45, 47, 50, 50, 53, 56, 56, 58, 60, 60, 64, 66, 66,
69, 70, 71, 74, 75, 76, 79, 79, 53, 52, 51, 51, 49, 49, 49, 49, 50, 49,
49, 49, 51, 54, 54, 57, 60, 60, 63, 65, 65, 69, 71, 72, 75, 76, 77, 81,
82, 83, 87, 87,
// Size 4x32
31, 32, 38, 53, 32, 32, 37, 52, 32, 32, 37, 51, 32, 32, 37, 51, 32, 32,
36, 49, 32, 33, 36, 49, 32, 33, 36, 49, 32, 34, 38, 49, 32, 34, 38, 50,
32, 34, 38, 49, 32, 35, 39, 49, 32, 35, 39, 49, 33, 36, 41, 51, 34, 37,
44, 54, 34, 37, 44, 54, 34, 37, 46, 57, 35, 38, 49, 60, 35, 38, 49, 60,
37, 40, 51, 63, 38, 40, 52, 65, 38, 40, 52, 65, 40, 42, 55, 69, 42, 43,
56, 71, 42, 43, 56, 72, 45, 45, 58, 75, 46, 46, 59, 76, 47, 47, 60, 77,
50, 49, 62, 81, 51, 50, 63, 82, 52, 51, 64, 83, 55, 54, 67, 87, 55, 54,
67, 87,
// Size 32x4
31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 35, 35,
37, 38, 38, 40, 42, 42, 45, 46, 47, 50, 51, 52, 55, 55, 32, 32, 32, 32,
32, 33, 33, 34, 34, 34, 35, 35, 36, 37, 37, 37, 38, 38, 40, 40, 40, 42,
43, 43, 45, 46, 47, 49, 50, 51, 54, 54, 38, 37, 37, 37, 36, 36, 36, 38,
38, 38, 39, 39, 41, 44, 44, 46, 49, 49, 51, 52, 52, 55, 56, 56, 58, 59,
60, 62, 63, 64, 67, 67, 53, 52, 51, 51, 49, 49, 49, 50, 49, 49, 49,
51, 54, 54, 57, 60, 60, 63, 65, 65, 69, 71, 72, 75, 76, 77, 81, 82, 83,
87, 87,
},
{ // Chroma
// Size 4x4
31, 38, 47, 49, 38, 47, 46, 46, 47, 46, 54, 57, 49, 46, 57, 66,
// Size 8x8
31, 31, 35, 42, 48, 47, 49, 51, 31, 32, 36, 42, 46, 45, 46, 48, 35, 36,
41, 45, 47, 45, 46, 48, 42, 42, 45, 48, 50, 49, 50, 51, 48, 46, 47, 50,
53, 53, 54, 54, 47, 45, 45, 49, 53, 57, 59, 60, 49, 46, 46, 50, 54, 59,
61, 64, 51, 48, 48, 51, 54, 60, 64, 68,
// Size 16x16
32, 31, 30, 31, 33, 36, 38, 41, 49, 49, 48, 49, 50, 51, 52, 54, 31, 31,
31, 32, 34, 38, 40, 42, 47, 47, 47, 47, 48, 48, 50, 52, 30, 31, 31, 32,
35, 39, 41, 42, 46, 46, 46, 45, 46, 47, 48, 50, 31, 32, 32, 33, 36, 40,
41, 43, 46, 46, 45, 45, 46, 46, 47, 49, 33, 34, 35, 36, 39, 43, 44, 45,
47, 46, 46, 45, 46, 47, 47, 49, 36, 38, 39, 40, 43, 47, 47, 47, 48, 47,
46, 45, 46, 46, 47, 48, 38, 40, 41, 41, 44, 47, 47, 48, 49, 48, 48, 47,
47, 47, 48, 49, 41, 42, 42, 43, 45, 47, 48, 48, 50, 50, 49, 49, 50, 50,
50, 52, 49, 47, 46, 46, 47, 48, 49, 50, 53, 53, 53, 53, 54, 54, 54, 55,
49, 47, 46, 46, 46, 47, 48, 50, 53, 53, 54, 55, 55, 55, 56, 57, 48, 47,
46, 45, 46, 46, 48, 49, 53, 54, 54, 55, 56, 56, 57, 58, 49, 47, 45, 45,
45, 45, 47, 49, 53, 55, 55, 58, 59, 60, 61, 62, 50, 48, 46, 46, 46, 46,
47, 50, 54, 55, 56, 59, 61, 61, 63, 64, 51, 48, 47, 46, 47, 46, 47, 50,
54, 55, 56, 60, 61, 62, 64, 66, 52, 50, 48, 47, 47, 47, 48, 50, 54, 56,
57, 61, 63, 64, 66, 68, 54, 52, 50, 49, 49, 48, 49, 52, 55, 57, 58, 62,
64, 66, 68, 71,
// Size 32x32
32, 31, 31, 31, 30, 30, 31, 33, 33, 34, 36, 36, 38, 41, 41, 45, 49, 49,

```

49, 48, 48, 49, 49, 49, 50, 50, 51, 52, 52, 53, 54, 54, 31, 31, 31, 31,
 31, 31, 31, 34, 34, 35, 38, 38, 39, 42, 42, 45, 48, 48, 47, 47, 47, 47,
 47, 47, 49, 49, 49, 50, 50, 51, 53, 53, 31, 31, 31, 31, 31, 31, 32, 34,
 34, 35, 38, 38, 40, 42, 42, 45, 47, 47, 47, 47, 47, 47, 47, 48, 48,
 48, 49, 50, 50, 52, 52, 31, 31, 31, 31, 31, 32, 34, 34, 36, 38, 38,
 40, 42, 42, 45, 47, 47, 47, 47, 47, 47, 46, 47, 48, 48, 48, 49, 49, 50,
 52, 52, 30, 31, 31, 31, 31, 31, 32, 35, 35, 36, 39, 39, 41, 42, 42, 44,
 46, 46, 46, 46, 46, 45, 45, 45, 46, 47, 47, 48, 48, 48, 50, 50, 30, 31,
 31, 31, 31, 32, 32, 35, 35, 36, 40, 40, 41, 42, 42, 44, 46, 46, 46, 45,
 45, 45, 45, 46, 46, 46, 47, 47, 48, 49, 49, 31, 31, 32, 32, 32, 32,
 33, 35, 36, 37, 40, 40, 41, 43, 43, 44, 46, 46, 46, 45, 45, 45, 45,
 46, 46, 46, 47, 47, 48, 49, 49, 33, 34, 34, 34, 35, 35, 35, 38, 38, 40,
 43, 43, 43, 44, 44, 46, 47, 47, 46, 46, 46, 45, 45, 45, 46, 46, 47, 47,
 47, 48, 49, 49, 33, 34, 34, 34, 35, 35, 36, 38, 39, 40, 43, 43, 44, 45,
 45, 46, 47, 47, 46, 46, 46, 45, 45, 45, 46, 46, 47, 47, 47, 48, 49, 49,
 34, 35, 35, 36, 36, 36, 37, 40, 40, 41, 44, 44, 45, 45, 45, 46, 47, 47,
 47, 46, 46, 45, 45, 45, 46, 46, 47, 47, 48, 49, 49, 36, 38, 38, 38,
 39, 40, 40, 43, 43, 44, 47, 47, 47, 47, 47, 47, 48, 48, 47, 46, 46, 45,
 45, 45, 46, 46, 46, 46, 47, 47, 48, 48, 36, 38, 38, 38, 39, 40, 40, 43,
 43, 44, 47, 47, 47, 47, 47, 47, 48, 48, 47, 46, 46, 45, 45, 45, 46, 46,
 46, 46, 47, 47, 48, 48, 38, 39, 40, 40, 41, 41, 41, 43, 44, 45, 47, 47,
 47, 48, 48, 48, 49, 49, 48, 48, 48, 47, 47, 47, 47, 47, 48, 48, 48,
 49, 49, 41, 42, 42, 42, 42, 42, 43, 44, 45, 45, 47, 47, 48, 48, 48, 49,
 50, 50, 50, 49, 49, 49, 49, 49, 50, 50, 50, 50, 50, 51, 52, 52, 41, 42,
 42, 42, 42, 43, 44, 45, 45, 47, 47, 48, 48, 48, 49, 50, 50, 50, 49,
 49, 49, 49, 50, 50, 50, 51, 52, 52, 45, 45, 45, 45, 44, 44,
 44, 46, 46, 46, 47, 47, 48, 49, 49, 50, 51, 51, 51, 51, 51, 51,
 52, 52, 52, 52, 52, 52, 53, 53, 49, 48, 47, 47, 46, 46, 46, 47, 47, 47,
 48, 48, 49, 50, 50, 51, 53, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54,
 54, 54, 55, 55, 49, 48, 47, 47, 46, 46, 46, 47, 47, 47, 48, 48, 49, 50,
 50, 51, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 54, 55, 55,
 49, 47, 47, 47, 46, 46, 46, 46, 46, 47, 47, 47, 48, 50, 50, 51, 53, 53,
 53, 54, 54, 54, 55, 55, 55, 55, 55, 56, 56, 56, 57, 57, 48, 47, 47, 47,
 46, 45, 45, 46, 46, 46, 46, 46, 48, 49, 49, 51, 53, 53, 54, 54, 55,
 55, 56, 56, 56, 57, 57, 58, 58, 58, 48, 47, 47, 47, 46, 45, 45, 46,
 46, 46, 46, 46, 48, 49, 49, 51, 53, 53, 54, 54, 55, 55, 56, 56, 56,
 56, 57, 57, 58, 58, 58, 49, 47, 47, 47, 45, 45, 45, 45, 45, 45,
 47, 49, 49, 51, 53, 53, 54, 55, 55, 57, 57, 58, 58, 59, 59, 60, 60, 60,
 61, 61, 49, 47, 47, 46, 45, 45, 45, 45, 45, 45, 45, 45, 47, 49, 49, 51,
 53, 53, 55, 55, 55, 57, 58, 58, 59, 60, 60, 61, 61, 61, 62, 62, 49, 47,
 47, 47, 45, 45, 45, 45, 45, 45, 47, 49, 49, 51, 53, 53, 55, 56,
 56, 58, 58, 59, 59, 60, 60, 61, 61, 62, 63, 63, 50, 49, 48, 48, 46, 46,
 46, 46, 46, 46, 46, 46, 47, 50, 50, 52, 54, 54, 55, 56, 56, 58, 59, 59,
 61, 61, 61, 63, 63, 63, 64, 64, 50, 49, 48, 48, 47, 46, 46, 46, 46, 46,
 46, 46, 47, 50, 50, 52, 54, 54, 55, 56, 56, 59, 60, 60, 61, 61, 62, 63,
 63, 64, 65, 65, 51, 49, 48, 48, 47, 46, 46, 47, 47, 46, 46, 46, 47, 50,
 50, 52, 54, 54, 55, 56, 56, 59, 60, 60, 61, 62, 62, 64, 64, 64, 66, 66,
 52, 50, 49, 49, 48, 47, 47, 47, 47, 47, 46, 46, 48, 50, 50, 52, 54, 54,
 56, 57, 57, 60, 61, 61, 63, 63, 64, 66, 66, 67, 68, 68, 52, 50, 50, 49,
 48, 47, 47, 47, 47, 47, 47, 47, 48, 50, 50, 52, 54, 54, 56, 57, 57, 60,
 61, 61, 63, 63, 64, 66, 66, 67, 68, 68, 53, 51, 50, 50, 48, 48, 48, 48,
 48, 48, 47, 47, 48, 51, 51, 52, 54, 54, 56, 58, 58, 60, 61, 62, 63, 64,
 64, 67, 67, 68, 69, 69, 54, 53, 52, 52, 50, 49, 49, 49, 49, 49, 48, 48,
 49, 52, 52, 53, 55, 55, 57, 58, 58, 61, 62, 63, 64, 65, 66, 68, 68, 69,
 71, 71, 54, 53, 52, 52, 50, 49, 49, 49, 49, 49, 48, 48, 49, 52, 52, 53,
 55, 55, 57, 58, 58, 61, 62, 63, 64, 65, 66, 68, 68, 69, 71, 71,
 // Size 4x8
 31, 38, 47, 50, 31, 40, 46, 48, 36, 44, 47, 47, 42, 47, 50, 50, 47, 48,
 53, 54, 46, 46, 54, 60, 48, 46, 55, 64, 50, 48, 56, 67,
 // Size 8x4
 31, 31, 36, 42, 47, 46, 48, 50, 38, 40, 44, 47, 48, 46, 46, 48, 47, 46,
 47, 50, 53, 54, 55, 56, 50, 48, 47, 50, 54, 60, 64, 67,
 // Size 8x16
 32, 31, 35, 38, 48, 49, 50, 52, 31, 31, 37, 40, 47, 47, 48, 50, 30, 32,
 38, 40, 46, 45, 46, 48, 31, 33, 38, 41, 46, 45, 46, 48, 33, 36, 41, 44,
 47, 46, 46, 47, 37, 40, 45, 47, 47, 45, 46, 47, 39, 41, 46, 47, 48, 47,
 47, 48, 42, 43, 46, 48, 50, 49, 50, 50, 49, 46, 48, 49, 53, 53, 54, 54,
 48, 46, 47, 48, 53, 55, 55, 56, 48, 46, 46, 48, 53, 56, 56, 57, 49, 45,
 45, 47, 53, 58, 59, 61, 50, 46, 46, 48, 54, 59, 61, 63, 51, 47, 47, 48,
 54, 60, 61, 64, 52, 48, 47, 48, 54, 61, 63, 66, 54, 50, 49, 50, 55, 62,
 65, 68,


```

// Size 16x8
32, 31, 30, 31, 33, 37, 39, 42, 49, 48, 48, 49, 50, 51, 52, 54, 31, 31,
32, 33, 36, 40, 41, 43, 46, 46, 46, 45, 46, 47, 48, 50, 35, 37, 38, 38,
41, 45, 46, 46, 48, 47, 46, 45, 46, 47, 47, 49, 38, 40, 40, 41, 44, 47,
47, 48, 49, 48, 48, 47, 48, 48, 48, 50, 48, 47, 46, 46, 47, 47, 48, 50,
53, 53, 53, 53, 54, 54, 54, 55, 49, 47, 45, 45, 46, 45, 47, 49, 53, 55,
56, 58, 59, 60, 61, 62, 50, 48, 46, 46, 46, 46, 47, 50, 54, 55, 56, 59,
61, 61, 63, 65, 52, 50, 48, 48, 47, 47, 48, 50, 54, 56, 57, 61, 63, 64,
66, 68,
// Size 16x32
32, 31, 31, 31, 35, 37, 38, 47, 48, 48, 49, 49, 50, 52, 52, 54, 31, 31,
31, 32, 36, 38, 39, 46, 47, 47, 48, 48, 49, 50, 50, 53, 31, 31, 31, 32,
37, 38, 40, 46, 47, 47, 47, 47, 48, 50, 50, 52, 31, 31, 31, 32, 37, 38,
40, 46, 47, 47, 47, 47, 48, 50, 50, 52, 30, 31, 32, 32, 38, 39, 40, 45,
46, 46, 45, 45, 46, 48, 48, 50, 30, 31, 32, 33, 38, 40, 41, 45, 46, 46,
45, 45, 46, 48, 48, 50, 31, 32, 33, 33, 38, 40, 41, 45, 46, 46, 45, 45,
46, 48, 48, 50, 33, 35, 35, 36, 41, 43, 43, 46, 47, 46, 45, 45, 46, 47,
47, 49, 33, 35, 36, 36, 41, 43, 44, 46, 47, 46, 46, 46, 46, 47, 47, 49,
34, 36, 37, 37, 42, 44, 45, 47, 47, 47, 45, 45, 46, 47, 47, 49, 37, 39,
40, 41, 45, 47, 47, 47, 47, 47, 45, 45, 46, 47, 47, 48, 37, 39, 40, 41,
45, 47, 47, 47, 47, 47, 45, 46, 47, 47, 48, 39, 40, 41, 42, 46, 47,
47, 48, 48, 48, 47, 47, 47, 48, 48, 50, 42, 42, 43, 43, 46, 47, 48, 50,
50, 50, 49, 49, 50, 50, 50, 52, 42, 42, 43, 43, 46, 47, 48, 50, 50, 50,
49, 49, 50, 50, 50, 52, 45, 45, 44, 45, 47, 47, 48, 51, 51, 51, 51, 51,
52, 52, 52, 54, 49, 47, 46, 47, 48, 48, 49, 52, 53, 53, 53, 53, 54, 54,
54, 55, 49, 47, 46, 47, 48, 48, 49, 52, 53, 53, 53, 54, 54, 54, 55,
48, 47, 46, 46, 47, 47, 48, 52, 53, 53, 55, 55, 55, 56, 56, 57, 48, 46,
46, 46, 46, 47, 48, 52, 53, 54, 56, 56, 56, 57, 57, 59, 48, 46, 46, 46,
46, 47, 48, 52, 53, 54, 56, 56, 56, 57, 57, 59, 49, 46, 45, 45, 46, 46,
47, 52, 53, 54, 57, 57, 58, 60, 60, 61, 49, 46, 45, 45, 45, 46, 47, 52,
53, 55, 58, 58, 59, 61, 61, 62, 49, 46, 45, 45, 46, 46, 47, 52, 53, 55,
58, 58, 60, 61, 61, 63, 50, 47, 46, 46, 46, 46, 48, 53, 54, 55, 59, 59,
61, 63, 63, 65, 50, 48, 46, 46, 46, 46, 48, 53, 54, 55, 59, 59, 61, 64,
64, 65, 51, 48, 47, 47, 47, 47, 48, 53, 54, 55, 60, 60, 61, 64, 64, 66,
52, 49, 48, 48, 47, 47, 48, 53, 54, 56, 61, 61, 63, 66, 66, 68, 52, 49,
48, 48, 47, 47, 48, 53, 54, 56, 61, 61, 63, 66, 66, 68, 53, 50, 48, 48,
48, 48, 49, 54, 54, 56, 61, 61, 63, 67, 67, 69, 54, 51, 50, 50, 49, 49,
50, 55, 55, 57, 62, 62, 65, 68, 68, 71, 54, 51, 50, 50, 49, 49, 50, 55,
55, 57, 62, 62, 65, 68, 68, 71,
// Size 32x16
32, 31, 31, 31, 30, 30, 31, 33, 33, 34, 37, 37, 39, 42, 42, 45, 49, 49,
48, 48, 48, 49, 49, 49, 50, 50, 51, 52, 52, 53, 54, 54, 31, 31, 31, 31,
31, 31, 32, 35, 35, 36, 39, 39, 40, 42, 42, 45, 47, 47, 47, 46, 46, 46,
46, 46, 47, 48, 48, 49, 49, 50, 51, 51, 31, 31, 31, 31, 32, 32, 33, 35,
36, 37, 40, 40, 41, 43, 43, 44, 46, 46, 46, 46, 46, 45, 45, 45, 46, 46,
47, 48, 48, 48, 50, 31, 32, 32, 32, 33, 33, 36, 36, 37, 41, 41,
42, 43, 43, 45, 47, 47, 46, 46, 46, 45, 45, 45, 46, 46, 47, 48, 48, 48,
50, 50, 35, 36, 37, 37, 38, 38, 38, 41, 41, 42, 45, 45, 46, 46, 46, 47,
48, 48, 47, 46, 46, 46, 45, 46, 46, 46, 47, 47, 47, 48, 49, 49, 37, 38,
38, 38, 39, 40, 40, 43, 43, 44, 47, 47, 47, 47, 47, 47, 48, 48, 47, 47,
47, 46, 46, 46, 46, 47, 47, 47, 48, 49, 49, 38, 39, 40, 40, 40, 41,
41, 43, 44, 45, 47, 47, 47, 48, 48, 48, 49, 49, 48, 48, 48, 47, 47, 47,
48, 48, 48, 48, 48, 49, 50, 50, 47, 46, 46, 46, 45, 45, 45, 46, 46, 47,
47, 47, 48, 50, 50, 51, 52, 52, 52, 52, 52, 52, 52, 53, 53, 53, 53,
53, 54, 55, 55, 48, 47, 47, 47, 46, 46, 46, 47, 47, 47, 47, 47, 48, 50,
50, 51, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 54, 55, 55,
48, 47, 47, 47, 46, 46, 46, 46, 46, 47, 47, 47, 48, 50, 50, 51, 53, 53,
53, 54, 54, 54, 55, 55, 55, 55, 55, 56, 56, 56, 57, 57, 49, 48, 47, 47,
45, 45, 45, 45, 46, 45, 45, 45, 47, 49, 49, 51, 53, 53, 55, 56, 56, 57,
58, 58, 59, 59, 60, 61, 61, 61, 62, 62, 49, 48, 47, 47, 45, 45, 45, 45,
46, 45, 45, 45, 47, 49, 51, 53, 53, 55, 56, 56, 57, 58, 58, 59, 59,
60, 61, 61, 61, 62, 62, 50, 49, 48, 48, 46, 46, 46, 46, 46, 46, 46, 46,
47, 50, 50, 52, 54, 54, 55, 56, 56, 58, 59, 60, 61, 61, 61, 63, 63, 63,
65, 65, 52, 50, 50, 50, 48, 48, 48, 47, 47, 47, 47, 47, 48, 50, 50, 52,
54, 54, 56, 57, 57, 60, 61, 61, 63, 64, 64, 66, 66, 67, 68, 68, 52, 50,
50, 50, 48, 48, 47, 47, 47, 47, 47, 48, 50, 52, 54, 54, 56, 57,
57, 60, 61, 61, 63, 64, 64, 66, 66, 67, 68, 68, 54, 53, 52, 52, 50, 50,
50, 49, 49, 49, 48, 48, 50, 52, 52, 54, 55, 55, 57, 59, 59, 61, 62, 63,
65, 65, 66, 68, 68, 69, 71, 71,
// Size 4x16
31, 37, 48, 52, 31, 38, 47, 50, 31, 39, 46, 48, 32, 40, 46, 48, 35, 43,

```

```

46, 47, 39, 47, 47, 47, 40, 47, 48, 48, 42, 47, 50, 50, 47, 48, 53, 54,
47, 47, 53, 56, 46, 47, 54, 57, 46, 46, 55, 61, 47, 46, 55, 63, 48, 47,
55, 64, 49, 47, 56, 66, 51, 49, 57, 68,
// Size 16x4
31, 31, 31, 32, 35, 39, 40, 42, 47, 47, 46, 46, 47, 48, 49, 51, 37, 38,
39, 40, 43, 47, 47, 47, 48, 47, 47, 46, 46, 47, 47, 49, 48, 47, 46, 46,
46, 47, 48, 50, 53, 53, 54, 55, 55, 55, 56, 57, 52, 50, 48, 48, 47, 47,
48, 50, 54, 56, 57, 61, 63, 64, 66, 68,
// Size 8x32
32, 31, 35, 38, 48, 49, 50, 52, 31, 31, 36, 39, 47, 48, 49, 50, 31, 31,
37, 40, 47, 47, 48, 50, 31, 31, 37, 40, 47, 47, 48, 50, 30, 32, 38, 40,
46, 45, 46, 48, 30, 32, 38, 41, 46, 45, 46, 48, 31, 33, 38, 41, 46, 45,
46, 48, 33, 35, 41, 43, 47, 45, 46, 47, 33, 36, 41, 44, 47, 46, 46, 47,
34, 37, 42, 45, 47, 45, 46, 47, 37, 40, 45, 47, 47, 45, 46, 47, 37, 40,
45, 47, 47, 45, 46, 47, 39, 41, 46, 47, 48, 47, 47, 48, 42, 43, 46, 48,
50, 49, 50, 50, 42, 43, 46, 48, 50, 49, 50, 50, 45, 44, 47, 48, 51, 51,
52, 52, 49, 46, 48, 49, 53, 53, 54, 54, 49, 46, 48, 49, 53, 53, 54, 54,
48, 46, 47, 48, 53, 55, 55, 56, 48, 46, 46, 48, 53, 56, 56, 57, 48, 46,
46, 48, 53, 56, 56, 57, 49, 45, 46, 47, 53, 57, 58, 60, 49, 45, 45, 47,
53, 58, 59, 61, 49, 45, 46, 47, 53, 58, 60, 61, 50, 46, 46, 48, 54, 59,
61, 63, 50, 46, 46, 48, 54, 59, 61, 64, 51, 47, 47, 48, 54, 60, 61, 64,
52, 48, 47, 48, 54, 61, 63, 66, 52, 48, 47, 48, 54, 61, 63, 66, 53, 48,
48, 49, 54, 61, 63, 67, 54, 50, 49, 50, 55, 62, 65, 68, 54, 50, 49, 50,
55, 62, 65, 68,
// Size 32x8
32, 31, 31, 31, 30, 30, 31, 33, 33, 34, 37, 37, 39, 42, 42, 45, 49, 49,
48, 48, 48, 49, 49, 49, 50, 50, 51, 52, 52, 53, 54, 54, 31, 31, 31, 31,
32, 32, 33, 35, 36, 37, 40, 40, 41, 43, 43, 44, 46, 46, 46, 46, 46, 45,
45, 45, 46, 46, 47, 48, 48, 48, 50, 50, 35, 36, 37, 37, 38, 38, 38, 41,
41, 42, 45, 45, 46, 46, 46, 47, 48, 48, 47, 46, 46, 46, 45, 46, 46, 46,
47, 47, 47, 48, 49, 49, 38, 39, 40, 40, 41, 41, 43, 44, 45, 47, 47,
47, 48, 48, 48, 49, 49, 48, 48, 48, 47, 47, 47, 48, 48, 48, 48, 48, 49,
50, 50, 48, 47, 47, 47, 46, 46, 46, 47, 47, 47, 47, 47, 47, 48, 50, 50, 51,
53, 53, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 54, 54, 55, 55, 49, 48,
47, 47, 45, 45, 45, 45, 46, 45, 45, 45, 47, 49, 49, 51, 53, 53, 55, 56,
56, 57, 58, 58, 59, 59, 60, 61, 61, 61, 62, 62, 50, 49, 48, 48, 46, 46,
46, 46, 46, 46, 46, 46, 47, 50, 50, 52, 54, 54, 55, 56, 56, 58, 59, 60,
61, 61, 61, 63, 63, 63, 65, 65, 52, 50, 50, 50, 48, 48, 48, 47, 47, 47,
47, 47, 48, 50, 50, 52, 54, 54, 56, 57, 57, 60, 61, 61, 63, 64, 64, 66,
66, 67, 68, 68,
// Size 4x32
31, 37, 48, 52, 31, 38, 47, 50, 31, 38, 47, 50, 31, 38, 47, 50, 31, 39,
46, 48, 31, 40, 46, 48, 32, 40, 46, 48, 35, 43, 46, 47, 35, 43, 46, 47,
36, 44, 47, 47, 39, 47, 47, 47, 39, 47, 47, 47, 40, 47, 48, 48, 42, 47,
50, 50, 42, 47, 50, 50, 45, 47, 51, 52, 47, 48, 53, 54, 47, 48, 53, 54,
47, 47, 53, 56, 46, 47, 54, 57, 46, 47, 54, 57, 46, 46, 54, 60, 46, 46,
55, 61, 46, 46, 55, 61, 47, 46, 55, 63, 48, 46, 55, 64, 48, 47, 55, 64,
49, 47, 56, 66, 49, 47, 56, 66, 50, 48, 56, 67, 51, 49, 57, 68, 51, 49,
57, 68,
// Size 32x4
31, 31, 31, 31, 31, 31, 32, 35, 35, 36, 39, 39, 40, 42, 42, 45, 47, 47,
47, 46, 46, 46, 46, 46, 47, 48, 48, 49, 49, 50, 51, 51, 37, 38, 38, 38,
39, 40, 40, 43, 43, 44, 47, 47, 47, 47, 47, 47, 48, 48, 47, 47, 47, 46,
46, 46, 46, 47, 47, 47, 48, 49, 49, 48, 47, 47, 47, 46, 46, 46, 46,
46, 47, 47, 47, 48, 50, 50, 51, 53, 53, 53, 54, 54, 54, 55, 55, 55, 55,
55, 56, 56, 56, 57, 57, 52, 50, 50, 48, 48, 48, 47, 47, 47, 47,
48, 50, 50, 52, 54, 54, 56, 57, 57, 60, 61, 61, 63, 64, 64, 66, 66, 67,
68, 68 ,
},

```

```

},
{
{ // Luma
// Size 4x4
32, 32, 35, 43, 32, 34, 37, 43, 35, 37, 48, 54, 43, 43, 54, 65,
// Size 8x8
31, 31, 32, 32, 34, 37, 43, 47, 31, 32, 32, 32, 34, 36, 41, 44, 32, 32,
33, 34, 35, 38, 42, 45, 32, 32, 34, 35, 37, 39, 42, 46, 34, 34, 35, 37,
41, 45, 49, 52, 37, 36, 38, 39, 45, 51, 56, 59, 43, 41, 42, 42, 49, 56,
63, 67, 47, 44, 45, 46, 52, 59, 67, 71,
// Size 16x16
32, 31, 31, 31, 31, 31, 32, 32, 34, 35, 36, 39, 41, 44, 47, 48, 31, 32,

```

32, 32, 32, 32, 32, 33, 34, 35, 35, 38, 40, 42, 45, 46, 31, 32, 32, 32,
32, 32, 32, 33, 34, 34, 35, 38, 39, 42, 45, 45, 31, 32, 32, 32, 32, 32,
32, 33, 33, 34, 34, 37, 38, 41, 44, 44, 31, 32, 32, 32, 33, 33, 34, 34, 35, 36,
35, 36, 36, 39, 40, 42, 44, 45, 31, 32, 32, 32, 33, 33, 34, 34, 35, 36,
36, 39, 40, 42, 45, 45, 32, 32, 32, 32, 33, 34, 35, 36, 37, 38, 38, 40,
41, 42, 45, 46, 32, 33, 33, 33, 34, 34, 36, 36, 38, 39, 40, 42, 43, 44,
47, 47, 34, 34, 34, 33, 35, 35, 37, 38, 39, 42, 42, 45, 46, 47, 50, 51,
35, 35, 34, 34, 36, 36, 38, 39, 42, 46, 47, 49, 50, 52, 55, 55, 36, 35,
35, 34, 36, 36, 38, 40, 42, 47, 48, 50, 52, 54, 56, 57, 39, 38, 38, 37,
39, 39, 40, 42, 45, 49, 50, 54, 55, 58, 60, 61, 41, 40, 39, 38, 40, 40,
41, 43, 46, 50, 52, 55, 57, 60, 62, 63, 44, 42, 42, 41, 42, 42, 42, 44,
47, 52, 54, 58, 60, 63, 66, 67, 47, 45, 45, 44, 44, 45, 45, 47, 50, 55,
56, 60, 62, 66, 69, 70, 48, 46, 45, 44, 45, 45, 46, 47, 51, 55, 57, 61,
63, 67, 70, 71,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 34, 34, 34,
35, 36, 36, 38, 39, 39, 41, 44, 44, 45, 47, 48, 48, 51, 31, 31, 31, 31,
31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 35, 35, 35, 37,
39, 39, 40, 43, 43, 44, 46, 47, 47, 50, 31, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 35, 35, 35, 37, 38, 38, 40, 42,
42, 43, 45, 46, 46, 49, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 33, 34, 34, 34, 35, 35, 35, 37, 38, 38, 40, 42, 42, 43, 45, 46,
46, 49, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34,
34, 34, 34, 35, 35, 36, 38, 38, 39, 42, 42, 42, 45, 45, 45, 48, 31, 31,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34,
34, 36, 37, 37, 38, 41, 41, 41, 44, 44, 44, 47, 31, 31, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 34, 36, 37, 37,
38, 41, 41, 41, 44, 44, 44, 47, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 36, 38, 38, 39, 41, 41, 42,
44, 45, 45, 47, 31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33,
34, 35, 35, 35, 36, 36, 37, 39, 39, 40, 42, 42, 42, 44, 45, 45, 48,
31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 35, 35, 35,
36, 36, 36, 38, 39, 39, 40, 42, 42, 42, 45, 45, 45, 48, 31, 32, 32, 32,
32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 35, 35, 35, 36, 36, 36, 38,
39, 39, 40, 42, 42, 42, 45, 45, 45, 48, 32, 32, 32, 32, 32, 32, 33,
33, 33, 33, 34, 34, 35, 35, 36, 36, 36, 37, 37, 39, 40, 40, 41, 42,
42, 43, 45, 45, 45, 48, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 35,
35, 35, 36, 37, 37, 37, 38, 38, 38, 39, 40, 40, 41, 42, 42, 43, 45, 46,
46, 48, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 35, 35, 35, 36, 37,
37, 37, 38, 38, 39, 40, 40, 41, 42, 42, 43, 45, 46, 46, 48, 32, 33,
33, 33, 33, 33, 33, 34, 34, 34, 35, 36, 36, 38, 38, 38, 39, 40,
40, 41, 42, 42, 43, 44, 44, 45, 47, 47, 47, 50, 34, 34, 34, 34, 34, 33,
33, 34, 35, 35, 35, 36, 37, 37, 38, 39, 39, 40, 42, 42, 42, 44, 45, 45,
46, 47, 47, 48, 50, 51, 51, 53, 34, 34, 34, 34, 34, 33, 33, 34, 35, 35,
35, 36, 37, 37, 38, 39, 39, 40, 42, 42, 42, 44, 45, 45, 46, 47, 47, 48,
50, 51, 51, 53, 34, 34, 34, 34, 34, 34, 35, 35, 35, 36, 37, 37,
38, 40, 40, 41, 43, 44, 44, 45, 46, 46, 47, 49, 49, 49, 51, 52, 52, 54,
35, 35, 35, 35, 34, 34, 34, 34, 36, 36, 36, 37, 38, 38, 39, 42, 42, 43,
46, 47, 47, 48, 49, 49, 50, 52, 52, 53, 55, 55, 55, 57, 36, 35, 35, 35,
35, 34, 34, 35, 36, 36, 36, 37, 38, 38, 40, 42, 42, 44, 47, 48, 48, 50,
50, 50, 52, 54, 54, 54, 56, 57, 57, 58, 36, 35, 35, 35, 34, 34, 35,
36, 36, 36, 37, 38, 38, 40, 42, 42, 44, 47, 48, 48, 50, 50, 50, 52, 54,
54, 54, 56, 57, 57, 58, 38, 37, 37, 37, 36, 36, 36, 36, 37, 38, 38, 39,
39, 39, 41, 44, 44, 45, 48, 50, 50, 51, 52, 52, 54, 56, 56, 57, 58, 59,
59, 61, 39, 39, 38, 38, 38, 37, 37, 38, 39, 39, 39, 40, 40, 40, 42, 45,
45, 46, 49, 50, 50, 52, 54, 54, 55, 58, 58, 60, 61, 61, 63, 39, 39,
38, 38, 38, 37, 37, 38, 39, 39, 39, 40, 40, 40, 42, 45, 45, 46, 49, 50,
50, 52, 54, 54, 55, 58, 58, 58, 60, 61, 61, 63, 41, 40, 40, 40, 39, 38,
38, 39, 40, 40, 40, 41, 41, 41, 43, 46, 46, 47, 50, 52, 52, 54, 55, 55,
57, 60, 60, 60, 62, 63, 63, 66, 44, 43, 42, 42, 42, 41, 41, 41, 42, 42,
42, 42, 42, 44, 47, 47, 49, 52, 54, 54, 56, 58, 58, 60, 63, 63, 64,
66, 67, 67, 69, 44, 43, 42, 42, 42, 41, 41, 41, 42, 42, 42, 42, 42,
44, 47, 47, 49, 52, 54, 54, 56, 58, 58, 60, 63, 63, 64, 66, 67, 67, 69,
45, 44, 43, 43, 42, 41, 41, 42, 42, 42, 42, 43, 43, 43, 45, 48, 48, 49,
53, 54, 54, 57, 58, 58, 60, 64, 64, 65, 67, 68, 68, 70, 47, 46, 45, 45,
45, 44, 44, 44, 45, 45, 45, 45, 45, 47, 50, 50, 51, 55, 56, 56, 58,
60, 60, 62, 66, 66, 67, 69, 70, 70, 73, 48, 47, 46, 46, 45, 44, 44, 45,
45, 45, 45, 45, 46, 46, 47, 51, 51, 52, 55, 57, 57, 59, 61, 61, 63, 67,
67, 68, 70, 71, 71, 74, 48, 47, 46, 46, 45, 44, 44, 45, 45, 45, 45,
46, 46, 47, 51, 51, 52, 55, 57, 57, 59, 61, 61, 63, 67, 67, 68, 70, 71,
71, 74, 51, 50, 49, 49, 48, 47, 47, 47, 48, 48, 48, 48, 48, 50, 53,

```

53, 54, 57, 58, 58, 61, 63, 63, 66, 69, 69, 70, 73, 74, 74, 77,
// Size 4x8
31, 32, 35, 43, 32, 33, 34, 41, 32, 34, 36, 42, 32, 35, 38, 42, 34, 37,
43, 49, 37, 40, 49, 56, 42, 43, 53, 63, 46, 46, 56, 67,
// Size 8x4
31, 32, 32, 32, 34, 37, 42, 46, 32, 33, 34, 35, 37, 40, 43, 46, 35, 34,
36, 38, 43, 49, 53, 56, 43, 41, 42, 42, 49, 56, 63, 67,
// Size 8x16
32, 31, 31, 32, 35, 36, 44, 47, 31, 32, 32, 32, 35, 35, 42, 45, 31, 32,
32, 32, 34, 35, 41, 45, 31, 32, 32, 33, 34, 34, 41, 44, 31, 32, 33, 34,
35, 36, 42, 44, 32, 32, 33, 34, 36, 36, 42, 45, 32, 33, 34, 35, 37, 38,
42, 45, 32, 33, 34, 36, 39, 40, 44, 47, 34, 34, 35, 37, 41, 42, 48, 50,
35, 34, 36, 38, 45, 47, 52, 55, 36, 34, 36, 38, 46, 48, 54, 56, 39, 37,
39, 40, 48, 50, 58, 60, 41, 39, 40, 41, 49, 51, 60, 62, 44, 41, 42, 43,
51, 53, 63, 66, 47, 44, 44, 45, 53, 56, 66, 69, 48, 45, 45, 46, 54, 56,
67, 70,
// Size 16x8
32, 31, 31, 31, 31, 32, 32, 32, 34, 35, 36, 39, 41, 44, 47, 48, 31, 32,
32, 32, 32, 32, 33, 33, 34, 34, 34, 37, 39, 41, 44, 45, 31, 32, 32, 32,
33, 33, 34, 34, 35, 36, 36, 39, 40, 42, 44, 45, 32, 32, 32, 33, 34, 34,
35, 36, 37, 38, 38, 40, 41, 43, 45, 46, 35, 35, 34, 34, 35, 36, 37, 39,
41, 45, 46, 48, 49, 51, 53, 54, 36, 35, 35, 34, 36, 36, 38, 40, 42, 47,
48, 50, 51, 53, 56, 56, 44, 42, 41, 41, 42, 42, 42, 44, 48, 52, 54, 58,
60, 63, 66, 67, 47, 45, 45, 44, 44, 45, 45, 47, 50, 55, 56, 60, 62, 66,
69, 70,
// Size 16x32
32, 31, 31, 31, 31, 32, 32, 32, 35, 36, 36, 40, 44, 44, 47, 53, 31, 31,
32, 32, 32, 32, 32, 33, 35, 35, 35, 39, 43, 43, 46, 52, 31, 32, 32, 32,
32, 32, 32, 33, 35, 35, 35, 39, 42, 42, 45, 51, 31, 32, 32, 32, 32, 32,
32, 33, 35, 35, 35, 39, 42, 45, 51, 31, 32, 32, 32, 32, 32, 32, 33,
34, 35, 35, 39, 41, 41, 45, 50, 31, 32, 32, 32, 32, 33, 33, 33, 34, 34,
34, 38, 41, 41, 44, 49, 31, 32, 32, 32, 32, 33, 33, 33, 34, 35, 35, 38, 41, 41,
41, 41, 44, 49, 31, 32, 32, 32, 32, 33, 33, 33, 34, 35, 35, 38, 41, 41,
44, 49, 31, 32, 32, 32, 33, 34, 34, 34, 35, 36, 36, 39, 42, 42, 44, 49,
32, 32, 32, 32, 33, 34, 34, 34, 36, 36, 36, 39, 42, 42, 45, 50, 32, 32,
32, 32, 33, 34, 34, 34, 36, 36, 36, 39, 42, 42, 45, 50, 32, 32, 32, 32,
33, 35, 35, 35, 37, 37, 37, 40, 42, 42, 45, 49, 32, 32, 33, 33, 34, 35,
35, 36, 37, 38, 38, 41, 42, 42, 45, 49, 32, 32, 33, 33, 34, 35, 35, 36,
37, 38, 38, 41, 42, 42, 45, 49, 32, 33, 33, 33, 34, 36, 36, 36, 39, 40,
40, 42, 44, 44, 47, 51, 34, 34, 34, 34, 35, 37, 37, 38, 41, 42, 42, 45,
48, 48, 50, 54, 34, 34, 34, 34, 35, 37, 37, 38, 41, 42, 42, 45, 48, 48,
50, 54, 34, 34, 34, 34, 35, 37, 37, 38, 42, 43, 43, 46, 49, 49, 51, 55,
35, 35, 34, 34, 36, 38, 38, 39, 45, 47, 47, 50, 52, 52, 55, 59, 36, 35,
34, 34, 36, 38, 38, 40, 46, 48, 48, 51, 54, 54, 56, 60, 36, 35, 34, 34,
36, 38, 38, 40, 46, 48, 48, 51, 54, 54, 56, 60, 38, 37, 36, 36, 37, 40,
40, 41, 47, 49, 49, 53, 56, 56, 58, 63, 39, 38, 37, 37, 39, 40, 40, 42,
48, 50, 50, 54, 58, 58, 60, 65, 39, 38, 37, 37, 39, 40, 40, 42, 48, 50,
50, 54, 58, 58, 60, 65, 41, 40, 39, 39, 40, 41, 41, 43, 49, 51, 51, 56,
60, 60, 62, 67, 44, 42, 41, 41, 42, 43, 43, 45, 51, 53, 53, 59, 63, 63,
66, 71, 44, 42, 41, 41, 42, 43, 43, 45, 51, 53, 53, 59, 63, 63, 66, 71,
44, 43, 42, 42, 42, 43, 43, 45, 51, 54, 54, 59, 64, 64, 67, 72, 47, 45,
44, 44, 44, 45, 45, 47, 53, 56, 56, 61, 66, 66, 69, 75, 48, 46, 45, 45,
45, 46, 46, 48, 54, 56, 56, 62, 67, 67, 70, 76, 48, 46, 45, 45, 45, 46,
46, 48, 54, 56, 56, 62, 67, 67, 70, 76, 51, 49, 47, 47, 48, 48, 48, 50,
56, 58, 58, 64, 69, 69, 73, 79,
// Size 32x16
32, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 34, 34, 34,
35, 36, 36, 38, 39, 39, 41, 44, 44, 44, 47, 48, 48, 51, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 35, 35, 35, 37,
38, 38, 40, 42, 42, 43, 45, 46, 46, 49, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 34, 36, 37, 37, 39, 41,
41, 42, 44, 45, 45, 47, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
33, 33, 33, 34, 34, 34, 34, 34, 34, 36, 37, 37, 39, 41, 41, 42, 44, 45,
45, 47, 31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 35,
35, 35, 36, 36, 36, 37, 39, 39, 40, 42, 42, 42, 44, 45, 45, 48, 32, 32,
32, 32, 32, 33, 33, 33, 34, 34, 34, 35, 35, 36, 37, 37, 37, 38, 38,
38, 40, 40, 40, 41, 43, 43, 43, 45, 46, 46, 48, 32, 32, 32, 32, 32, 33,
33, 33, 34, 34, 34, 35, 35, 35, 36, 37, 37, 37, 38, 38, 38, 40, 40, 40,
41, 43, 43, 43, 45, 46, 46, 48, 32, 33, 33, 33, 33, 33, 33, 33, 34, 34,
34, 35, 36, 36, 36, 38, 38, 38, 39, 40, 40, 41, 42, 42, 43, 45, 45, 45,
47, 48, 48, 50, 35, 35, 35, 35, 34, 34, 34, 34, 35, 36, 36, 37, 37, 37,

```

39, 41, 41, 42, 45, 46, 46, 47, 48, 48, 49, 51, 51, 51, 53, 54, 54, 56,
 36, 35, 35, 35, 35, 34, 34, 35, 36, 36, 36, 37, 38, 38, 40, 42, 42, 43,
 47, 48, 48, 49, 50, 50, 51, 53, 53, 54, 56, 56, 56, 58, 36, 35, 35, 35,
 35, 34, 34, 35, 36, 36, 36, 37, 38, 38, 40, 42, 42, 43, 47, 48, 48, 49,
 50, 50, 51, 53, 53, 54, 56, 56, 56, 58, 40, 39, 39, 39, 39, 38, 38, 38,
 39, 39, 39, 40, 41, 41, 42, 45, 45, 46, 50, 51, 51, 53, 54, 54, 56, 59,
 59, 59, 61, 62, 62, 64, 44, 43, 42, 42, 41, 41, 41, 41, 42, 42, 42, 42,
 42, 42, 44, 48, 48, 49, 52, 54, 54, 56, 58, 58, 60, 63, 63, 64, 66, 67,
 67, 69, 44, 43, 42, 42, 41, 41, 41, 41, 42, 42, 42, 42, 42, 44, 48,
 48, 49, 52, 54, 54, 56, 58, 58, 60, 63, 63, 64, 66, 67, 67, 69, 47, 46,
 45, 45, 45, 44, 44, 44, 44, 45, 45, 45, 45, 47, 50, 50, 51, 55, 56,
 56, 58, 60, 60, 62, 66, 66, 67, 69, 70, 70, 73, 53, 52, 51, 51, 50, 49,
 49, 49, 49, 50, 50, 49, 49, 49, 51, 54, 54, 55, 59, 60, 60, 63, 65, 65,
 67, 71, 71, 72, 75, 76, 76, 79,
 // Size 4x16
 31, 32, 36, 44, 32, 32, 35, 42, 32, 32, 35, 41, 32, 33, 34, 41, 32, 34,
 36, 42, 32, 34, 36, 42, 32, 35, 38, 42, 33, 36, 40, 44, 34, 37, 42, 48,
 35, 38, 47, 52, 35, 38, 48, 54, 38, 40, 50, 58, 40, 41, 51, 60, 42, 43,
 53, 63, 45, 45, 56, 66, 46, 46, 56, 67,
 // Size 16x4
 31, 32, 32, 32, 32, 32, 32, 33, 34, 35, 35, 38, 40, 42, 45, 46, 32, 32,
 32, 33, 34, 34, 35, 36, 37, 38, 38, 40, 41, 43, 45, 46, 36, 35, 35, 34,
 36, 36, 38, 40, 42, 47, 48, 50, 51, 53, 56, 56, 44, 42, 41, 41, 42, 42,
 42, 44, 48, 52, 54, 58, 60, 63, 66, 67,
 // Size 8x32
 32, 31, 31, 32, 35, 36, 44, 47, 31, 32, 32, 32, 35, 35, 43, 46, 31, 32,
 32, 32, 35, 35, 42, 45, 31, 32, 32, 32, 35, 35, 42, 45, 31, 32, 32, 32,
 34, 35, 41, 45, 31, 32, 32, 33, 34, 34, 41, 44, 31, 32, 32, 33, 34, 34,
 41, 44, 31, 32, 32, 33, 34, 35, 41, 44, 31, 32, 33, 34, 35, 36, 42, 44,
 32, 32, 33, 34, 36, 36, 42, 45, 32, 32, 33, 34, 36, 36, 42, 45, 32, 32,
 33, 35, 37, 37, 42, 45, 32, 33, 34, 35, 37, 38, 42, 45, 32, 33, 34, 35,
 37, 38, 42, 45, 32, 33, 34, 36, 39, 40, 44, 47, 34, 34, 35, 37, 41, 42,
 48, 50, 34, 34, 35, 37, 41, 42, 48, 50, 34, 34, 35, 37, 42, 43, 49, 51,
 35, 34, 36, 38, 45, 47, 52, 55, 36, 34, 36, 38, 46, 48, 54, 56, 36, 34,
 36, 38, 46, 48, 54, 56, 38, 36, 37, 40, 47, 49, 56, 58, 39, 37, 39, 40,
 48, 50, 58, 60, 39, 37, 39, 40, 48, 50, 58, 60, 41, 39, 40, 41, 49, 51,
 60, 62, 44, 41, 42, 43, 51, 53, 63, 66, 44, 41, 42, 43, 51, 53, 63, 66,
 44, 42, 42, 43, 51, 54, 64, 67, 47, 44, 44, 45, 53, 56, 66, 69, 48, 45,
 45, 46, 54, 56, 67, 70, 48, 45, 45, 46, 54, 56, 67, 70, 51, 47, 48, 48,
 56, 58, 69, 73,
 // Size 32x8
 32, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 34, 34, 34,
 35, 36, 36, 38, 39, 39, 41, 44, 44, 44, 47, 48, 48, 51, 31, 32, 32, 32,
 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 34, 34, 36,
 37, 37, 39, 41, 41, 42, 44, 45, 45, 47, 31, 32, 32, 32, 32, 32, 32,
 33, 33, 33, 33, 34, 34, 35, 35, 35, 36, 36, 37, 39, 39, 40, 42,
 42, 42, 44, 45, 45, 48, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 35,
 35, 35, 36, 37, 37, 37, 38, 38, 38, 40, 40, 40, 41, 43, 43, 43, 45, 46,
 46, 48, 35, 35, 35, 35, 34, 34, 34, 34, 35, 36, 36, 37, 37, 37, 39, 41,
 41, 42, 45, 46, 46, 47, 48, 48, 49, 51, 51, 51, 53, 54, 54, 56, 36, 35,
 35, 35, 35, 34, 34, 35, 36, 36, 37, 38, 38, 40, 42, 42, 43, 47, 48,
 48, 49, 50, 50, 51, 53, 53, 54, 56, 56, 56, 58, 44, 43, 42, 42, 41, 41,
 41, 41, 42, 42, 42, 42, 42, 42, 44, 48, 48, 49, 52, 54, 54, 56, 58, 58,
 60, 63, 63, 64, 66, 67, 67, 69, 47, 46, 45, 45, 45, 44, 44, 44, 44, 45,
 45, 45, 45, 45, 47, 50, 50, 51, 55, 56, 56, 58, 60, 60, 62, 66, 66, 67,
 69, 70, 70, 73,
 // Size 4x32
 31, 32, 36, 44, 31, 32, 35, 43, 32, 32, 35, 42, 32, 32, 35, 42, 32, 32,
 35, 41, 32, 33, 34, 41, 32, 33, 34, 41, 32, 33, 35, 41, 32, 34, 36, 42,
 32, 34, 36, 42, 32, 34, 36, 42, 32, 35, 37, 42, 32, 35, 38, 42, 32, 35,
 38, 42, 33, 36, 40, 44, 34, 37, 42, 48, 34, 37, 42, 48, 34, 37, 43, 49,
 35, 38, 47, 52, 35, 38, 48, 54, 35, 38, 48, 54, 37, 40, 49, 56, 38, 40,
 50, 58, 38, 40, 50, 58, 40, 41, 51, 60, 42, 43, 53, 63, 42, 43, 53, 63,
 43, 43, 54, 64, 45, 45, 56, 66, 46, 46, 56, 67, 46, 46, 56, 67, 49, 48,
 58, 69,
 // Size 32x4
 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34,
 35, 35, 35, 37, 38, 38, 40, 42, 42, 43, 45, 46, 46, 49, 32, 32, 32, 32,
 32, 33, 33, 33, 34, 34, 34, 35, 35, 35, 36, 37, 37, 37, 38, 38, 38, 40,
 40, 40, 41, 43, 43, 43, 45, 46, 46, 48, 36, 35, 35, 35, 35, 34, 34, 35,
 36, 36, 36, 37, 38, 38, 40, 42, 42, 43, 47, 48, 48, 49, 50, 50, 51, 53,

```

53, 54, 56, 56, 56, 58, 44, 43, 42, 42, 41, 41, 41, 41, 42, 42, 42, 42,
42, 42, 44, 48, 48, 49, 52, 54, 54, 56, 58, 58, 60, 63, 63, 64, 66, 67,
67, 69,
},
{ // Chroma
  // Size 4x4
  31, 37, 47, 47, 37, 44, 47, 45, 47, 47, 53, 53, 47, 45, 53, 59,
  // Size 8x8
  31, 31, 34, 37, 43, 48, 47, 49, 31, 32, 35, 40, 43, 46, 45, 46, 34, 35,
  39, 43, 45, 46, 45, 46, 37, 40, 43, 47, 47, 47, 45, 46, 43, 43, 45, 47,
  49, 50, 50, 50, 48, 46, 46, 47, 50, 53, 55, 55, 47, 45, 45, 45, 50, 55,
  58, 60, 49, 46, 46, 46, 50, 55, 60, 61,
  // Size 16x16
  32, 31, 31, 30, 33, 33, 36, 38, 41, 47, 49, 48, 49, 49, 50, 50, 31, 31,
  31, 31, 34, 34, 38, 40, 42, 46, 47, 47, 47, 47, 48, 48, 31, 31, 31, 31,
  34, 35, 39, 40, 42, 46, 47, 46, 46, 46, 47, 47, 30, 31, 31, 32, 34, 35,
  40, 41, 42, 45, 46, 45, 45, 45, 46, 46, 33, 34, 34, 34, 37, 38, 42, 43,
  44, 46, 47, 46, 46, 45, 46, 46, 33, 34, 35, 35, 38, 39, 43, 44, 45, 47,
  47, 46, 46, 45, 46, 46, 36, 38, 39, 40, 42, 43, 47, 47, 47, 47, 48, 46,
  46, 45, 46, 46, 38, 40, 40, 41, 43, 44, 47, 47, 48, 48, 49, 48, 47, 47,
  47, 47, 41, 42, 42, 42, 44, 45, 47, 48, 48, 50, 50, 49, 49, 49, 50, 50,
  47, 46, 46, 45, 46, 47, 47, 48, 50, 52, 52, 52, 52, 52, 53, 53, 49, 47,
  47, 46, 47, 47, 48, 49, 50, 52, 53, 53, 53, 53, 54, 54, 48, 47, 46, 45,
  46, 46, 46, 48, 49, 52, 53, 54, 55, 55, 56, 56, 49, 47, 46, 45, 46, 46,
  46, 47, 49, 52, 53, 55, 55, 57, 57, 58, 49, 47, 46, 45, 45, 45, 45, 47,
  49, 52, 53, 55, 57, 58, 59, 60, 50, 48, 47, 46, 46, 46, 46, 47, 50, 53,
  54, 56, 57, 59, 61, 61, 50, 48, 47, 46, 46, 46, 46, 47, 50, 53, 54, 56,
  58, 60, 61, 61,
  // Size 32x32
  32, 31, 31, 31, 31, 30, 30, 31, 33, 33, 33, 35, 36, 36, 38, 41, 41, 43,
  47, 49, 49, 49, 48, 48, 49, 49, 49, 50, 50, 51, 31, 31, 31, 31,
  31, 31, 31, 31, 33, 34, 34, 36, 37, 37, 39, 42, 42, 43, 47, 48, 48, 48,
  47, 47, 47, 47, 47, 48, 49, 49, 49, 50, 31, 31, 31, 31, 31, 31, 32,
  34, 34, 34, 37, 38, 38, 40, 42, 42, 43, 46, 47, 47, 47, 47, 47, 47,
  47, 47, 48, 48, 48, 49, 31, 31, 31, 31, 31, 31, 32, 34, 34, 34, 37,
  38, 38, 40, 42, 42, 43, 46, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48,
  48, 49, 31, 31, 31, 31, 31, 31, 31, 32, 34, 35, 35, 37, 39, 39, 40, 42,
  42, 43, 46, 47, 47, 46, 46, 46, 46, 46, 46, 47, 47, 47, 48, 30, 31,
  31, 31, 31, 32, 32, 32, 34, 35, 35, 38, 40, 40, 41, 42, 42, 43, 45, 46,
  46, 46, 45, 45, 45, 45, 45, 46, 46, 46, 47, 30, 31, 31, 31, 31, 32,
  32, 32, 34, 35, 35, 38, 40, 40, 41, 42, 42, 43, 45, 46, 46, 46, 45, 45,
  45, 45, 45, 45, 46, 46, 46, 47, 31, 31, 32, 32, 32, 32, 32, 33, 35, 36,
  36, 38, 40, 40, 41, 43, 43, 43, 46, 46, 46, 46, 45, 45, 45, 45, 45, 45,
  46, 46, 46, 47, 33, 33, 34, 34, 34, 34, 34, 35, 37, 38, 38, 41, 42, 42,
  43, 44, 44, 45, 46, 47, 47, 46, 46, 46, 46, 45, 45, 45, 46, 46, 47,
  33, 34, 34, 34, 35, 35, 35, 36, 38, 39, 39, 41, 43, 43, 44, 45, 45, 45,
  47, 47, 47, 46, 46, 46, 46, 45, 45, 45, 46, 46, 47, 33, 34, 34, 34,
  35, 35, 35, 36, 38, 39, 39, 41, 43, 43, 44, 45, 45, 45, 47, 47, 47, 46,
  46, 46, 46, 45, 45, 45, 46, 46, 46, 47, 35, 36, 37, 37, 37, 38, 38, 38,
  41, 41, 41, 44, 46, 46, 46, 46, 46, 46, 47, 47, 47, 47, 46, 46, 46, 45,
  45, 45, 46, 46, 47, 36, 37, 38, 38, 39, 40, 40, 40, 42, 43, 43, 46,
  47, 47, 47, 47, 47, 47, 47, 48, 48, 47, 46, 46, 46, 45, 45, 45, 46, 46,
  46, 46, 36, 37, 38, 38, 39, 40, 40, 40, 42, 43, 43, 46, 47, 47, 47, 47,
  47, 47, 47, 48, 48, 47, 46, 46, 46, 45, 45, 45, 46, 46, 38, 39,
  40, 40, 40, 41, 41, 41, 43, 44, 44, 46, 47, 47, 47, 48, 48, 48, 48, 49,
  49, 48, 48, 48, 47, 47, 47, 47, 47, 47, 48, 41, 42, 42, 42, 42, 42,
  42, 43, 44, 45, 45, 46, 47, 47, 48, 48, 48, 49, 50, 50, 50, 49, 49,
  49, 49, 49, 49, 50, 50, 50, 50, 41, 42, 42, 42, 42, 42, 42, 43, 44, 45,
  45, 46, 47, 47, 48, 48, 48, 49, 50, 50, 50, 50, 49, 49, 49, 49, 49, 49,
  50, 50, 50, 50, 43, 43, 43, 43, 43, 43, 43, 43, 45, 45, 45, 46, 47, 47,
  48, 49, 49, 49, 50, 51, 51, 50, 50, 50, 50, 50, 50, 50, 50, 50, 51,
  47, 47, 46, 46, 46, 45, 45, 46, 46, 47, 47, 47, 47, 47, 48, 50, 50, 50,
  52, 52, 52, 52, 52, 52, 52, 52, 52, 52, 53, 53, 53, 53, 49, 48, 47, 47,
  47, 46, 46, 46, 47, 47, 47, 47, 48, 48, 49, 50, 50, 51, 52, 53, 53, 53,
  53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 49, 48, 47, 47, 46, 46, 46,
  47, 47, 47, 47, 48, 48, 49, 50, 50, 51, 52, 53, 53, 53, 53, 53, 53,
  53, 53, 54, 54, 54, 54, 49, 48, 47, 47, 46, 46, 46, 46, 46, 46, 47,
  47, 47, 48, 50, 50, 50, 52, 53, 53, 53, 54, 54, 54, 55, 55, 55, 55, 55,
  55, 56, 48, 47, 47, 47, 46, 45, 45, 45, 46, 46, 46, 46, 46, 46, 48, 49,
  49, 50, 52, 53, 53, 54, 54, 54, 55, 55, 55, 56, 56, 56, 56, 57, 48, 47,
  47, 47, 46, 45, 45, 45, 46, 46, 46, 46, 46, 46, 48, 49, 49, 50, 52, 53,

```

```

53, 54, 54, 54, 55, 55, 55, 56, 56, 56, 56, 57, 49, 47, 47, 47, 46, 45,
45, 45, 46, 46, 46, 46, 46, 46, 47, 49, 49, 50, 52, 53, 53, 54, 55, 55,
55, 57, 57, 57, 57, 58, 58, 58, 49, 47, 47, 47, 46, 45, 45, 45, 45, 45,
45, 45, 45, 45, 47, 49, 49, 50, 52, 53, 53, 55, 55, 55, 57, 58, 58, 59,
59, 60, 60, 60, 49, 47, 47, 47, 46, 45, 45, 45, 45, 45, 45, 45, 45, 45,
47, 49, 49, 50, 52, 53, 53, 55, 55, 55, 57, 58, 58, 59, 59, 60, 60, 60,
49, 48, 47, 47, 46, 45, 45, 45, 45, 45, 45, 45, 47, 49, 49, 50,
52, 53, 53, 55, 56, 56, 57, 59, 59, 59, 60, 60, 60, 61, 50, 49, 48, 48,
47, 46, 46, 46, 46, 46, 46, 46, 46, 46, 47, 50, 50, 50, 53, 54, 54, 55,
56, 56, 57, 59, 59, 60, 61, 61, 61, 62, 50, 49, 48, 48, 47, 46, 46, 46,
46, 46, 46, 46, 46, 46, 47, 50, 50, 50, 53, 54, 54, 55, 56, 56, 58, 60,
60, 60, 61, 61, 61, 63, 50, 49, 48, 48, 47, 46, 46, 46, 46, 46, 46,
46, 46, 47, 50, 50, 50, 53, 54, 54, 55, 56, 56, 58, 60, 60, 60, 61, 61,
61, 63, 51, 50, 49, 49, 48, 47, 47, 47, 47, 47, 47, 47, 46, 46, 48, 50,
50, 51, 53, 54, 54, 56, 57, 57, 58, 60, 60, 61, 62, 63, 63, 64,
// Size 4x8
31, 38, 47, 48, 31, 40, 46, 45, 35, 43, 47, 46, 39, 47, 47, 45, 43, 47,
50, 50, 47, 47, 53, 55, 46, 46, 53, 58, 48, 46, 54, 59,
// Size 8x4
31, 31, 35, 39, 43, 47, 46, 48, 38, 40, 43, 47, 47, 47, 46, 46, 47, 46,
47, 47, 50, 53, 53, 54, 48, 45, 46, 45, 50, 55, 58, 59,
// Size 8x16
32, 31, 33, 37, 45, 48, 49, 50, 31, 31, 34, 38, 45, 47, 47, 48, 31, 32,
34, 39, 45, 46, 46, 47, 30, 32, 35, 40, 44, 46, 45, 46, 33, 35, 37, 42,
46, 47, 45, 46, 33, 36, 38, 43, 46, 47, 46, 46, 37, 40, 43, 47, 47, 47,
45, 46, 39, 41, 43, 47, 48, 48, 47, 47, 42, 43, 44, 47, 49, 50, 49, 50,
47, 46, 46, 48, 51, 52, 53, 53, 49, 46, 47, 48, 52, 53, 53, 54, 48, 46,
46, 47, 51, 53, 56, 56, 48, 45, 46, 46, 51, 53, 57, 57, 49, 45, 45, 46,
51, 53, 58, 59, 50, 46, 46, 46, 52, 54, 59, 61, 50, 46, 46, 46, 52, 54,
59, 61,
// Size 16x8
32, 31, 31, 30, 33, 33, 37, 39, 42, 47, 49, 48, 48, 49, 50, 50, 31, 31,
32, 32, 35, 36, 40, 41, 43, 46, 46, 46, 45, 45, 46, 46, 33, 34, 34, 35,
37, 38, 43, 43, 44, 46, 47, 46, 46, 45, 46, 46, 37, 38, 39, 40, 42, 43,
47, 47, 47, 48, 48, 47, 46, 46, 46, 46, 45, 45, 45, 44, 46, 46, 47, 48,
49, 51, 52, 51, 51, 51, 52, 52, 48, 47, 46, 46, 47, 47, 47, 48, 50, 52,
53, 53, 53, 53, 54, 54, 49, 47, 46, 45, 45, 46, 45, 47, 49, 53, 53, 56,
57, 58, 59, 59, 50, 48, 47, 46, 46, 46, 46, 47, 50, 53, 54, 56, 57, 59,
61, 61,
// Size 16x32
32, 31, 31, 31, 33, 37, 37, 38, 45, 48, 48, 49, 49, 49, 50, 52, 31, 31,
31, 31, 33, 38, 38, 39, 45, 47, 47, 48, 48, 48, 49, 51, 31, 31, 31, 31,
34, 38, 38, 40, 45, 47, 47, 47, 47, 47, 48, 50, 31, 31, 31, 31, 34, 38,
38, 40, 45, 47, 47, 47, 47, 47, 48, 50, 31, 31, 32, 32, 34, 39, 39, 40,
45, 46, 46, 46, 46, 46, 47, 49, 30, 31, 32, 32, 35, 40, 40, 41, 44, 46,
46, 45, 45, 45, 46, 48, 30, 31, 32, 32, 35, 40, 40, 41, 44, 46, 46, 45,
45, 45, 46, 48, 31, 32, 33, 33, 35, 40, 40, 41, 45, 46, 46, 45, 45, 45,
46, 48, 33, 34, 35, 35, 37, 42, 42, 43, 46, 47, 47, 46, 45, 45, 46, 47,
33, 35, 36, 36, 38, 43, 43, 44, 46, 47, 47, 46, 46, 46, 46, 47, 33, 35,
36, 36, 38, 43, 43, 44, 46, 47, 47, 46, 46, 46, 46, 47, 35, 37, 38, 38,
41, 45, 45, 46, 47, 47, 47, 46, 45, 45, 46, 47, 37, 39, 40, 40, 43, 47,
47, 47, 47, 47, 47, 46, 45, 45, 46, 47, 39, 40, 41, 41, 43, 47, 47, 47, 48, 48,
47, 47, 47, 47, 47, 48, 42, 42, 43, 43, 44, 47, 47, 48, 49, 50, 50, 49,
49, 49, 50, 50, 42, 42, 43, 43, 44, 47, 47, 48, 49, 50, 50, 49, 49, 49,
50, 50, 43, 43, 43, 43, 45, 47, 47, 48, 50, 50, 50, 50, 50, 50, 51,
47, 46, 46, 46, 46, 48, 48, 48, 48, 51, 52, 52, 52, 53, 53, 53, 53, 49, 47,
46, 46, 47, 48, 48, 49, 52, 53, 53, 53, 53, 53, 54, 54, 48, 47, 46, 46, 46, 47,
47, 48, 48, 49, 52, 53, 53, 53, 53, 53, 54, 54, 48, 47, 46, 46, 46, 47,
47, 48, 52, 53, 53, 54, 55, 55, 55, 56, 48, 47, 46, 46, 46, 47, 47, 48,
51, 53, 53, 54, 56, 56, 56, 57, 48, 47, 46, 46, 46, 47, 47, 48, 51, 53,
53, 54, 56, 56, 56, 57, 48, 47, 45, 45, 46, 46, 46, 47, 51, 53, 53, 55,
57, 57, 57, 59, 49, 46, 45, 45, 45, 46, 46, 47, 51, 53, 53, 56, 58, 58,
59, 61, 49, 46, 45, 45, 45, 46, 46, 47, 51, 53, 53, 56, 58, 58, 59, 61,
49, 47, 45, 45, 45, 46, 46, 47, 52, 53, 53, 56, 58, 58, 60, 62, 50, 48,
46, 46, 46, 46, 46, 48, 52, 54, 57, 59, 61, 63, 50, 48, 46, 46,
46, 46, 46, 48, 52, 54, 54, 57, 59, 59, 61, 64, 50, 48, 46, 46, 46, 46,
46, 48, 52, 54, 54, 57, 59, 59, 61, 64, 51, 49, 47, 47, 47, 47, 47, 48,
52, 54, 54, 58, 60, 60, 62, 65,
// Size 32x16
32, 31, 31, 31, 31, 30, 30, 31, 33, 33, 33, 35, 37, 37, 39, 42, 42, 43,

```

47, 49, 49, 48, 48, 48, 48, 49, 49, 49, 50, 50, 50, 51, 31, 31, 31, 31,
 31, 31, 31, 32, 34, 35, 35, 37, 39, 39, 40, 42, 42, 43, 46, 47, 47, 47,
 47, 47, 47, 46, 46, 47, 48, 48, 48, 49, 31, 31, 31, 31, 32, 32, 32, 33,
 35, 36, 36, 38, 40, 40, 41, 43, 43, 43, 46, 46, 46, 46, 46, 46, 45, 45,
 45, 45, 46, 46, 46, 47, 31, 31, 31, 31, 32, 32, 32, 33, 35, 36, 36, 38,
 40, 40, 41, 43, 43, 43, 46, 46, 46, 46, 46, 45, 45, 45, 45, 46, 46,
 46, 47, 33, 33, 34, 34, 34, 35, 35, 35, 37, 38, 38, 41, 43, 43, 43, 44,
 44, 45, 46, 47, 47, 46, 46, 46, 46, 45, 45, 45, 46, 46, 46, 47, 37, 38,
 38, 38, 39, 40, 40, 40, 42, 43, 43, 45, 47, 47, 47, 47, 47, 47, 48, 48,
 48, 47, 47, 47, 46, 46, 46, 46, 46, 46, 46, 47, 37, 38, 38, 38, 39, 40,
 40, 40, 42, 43, 43, 45, 47, 47, 47, 47, 47, 48, 48, 48, 47, 47, 47,
 46, 46, 46, 46, 46, 46, 46, 47, 38, 39, 40, 40, 40, 41, 41, 41, 43, 44,
 44, 46, 47, 47, 47, 48, 48, 48, 48, 49, 49, 48, 48, 48, 47, 47, 47, 47,
 48, 48, 48, 48, 45, 45, 45, 45, 45, 44, 44, 45, 46, 46, 46, 47, 47, 47,
 48, 49, 49, 50, 51, 52, 52, 52, 51, 51, 51, 51, 51, 52, 52, 52, 52, 52,
 48, 47, 47, 47, 46, 46, 46, 46, 47, 47, 47, 47, 47, 48, 50, 50, 50,
 52, 53, 53, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 48, 47, 47, 47,
 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 48, 50, 50, 50, 52, 53, 53, 53,
 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 49, 48, 47, 47, 46, 45, 45, 45,
 46, 46, 46, 46, 46, 46, 47, 49, 49, 50, 52, 53, 53, 54, 54, 54, 55, 56,
 56, 56, 57, 57, 57, 58, 49, 48, 47, 47, 46, 45, 45, 45, 45, 46, 46, 45,
 45, 45, 47, 49, 49, 50, 53, 53, 53, 55, 56, 56, 57, 58, 58, 58, 59, 59,
 59, 60, 49, 48, 47, 47, 46, 45, 45, 45, 45, 46, 46, 45, 45, 45, 47, 49,
 49, 50, 53, 53, 53, 55, 56, 56, 57, 58, 58, 58, 59, 59, 59, 60, 50, 49,
 48, 48, 47, 46, 46, 46, 46, 46, 46, 46, 46, 47, 50, 50, 50, 53, 54,
 54, 55, 56, 56, 57, 59, 59, 60, 61, 61, 62, 52, 51, 50, 50, 49, 48,
 48, 48, 47, 47, 47, 47, 47, 47, 48, 50, 50, 51, 53, 54, 54, 56, 57, 57,
 59, 61, 61, 62, 63, 64, 64, 65,
 // Size 4x16
 31, 37, 48, 49, 31, 38, 47, 47, 31, 39, 46, 46, 31, 40, 46, 45, 34, 42,
 47, 45, 35, 43, 47, 46, 39, 47, 47, 45, 40, 47, 48, 47, 42, 47, 50, 49,
 46, 48, 52, 53, 47, 48, 53, 53, 47, 47, 53, 56, 47, 46, 53, 57, 46, 46,
 53, 58, 48, 46, 54, 59, 48, 46, 54, 59,
 // Size 16x4
 31, 31, 31, 31, 34, 35, 39, 40, 42, 46, 47, 47, 47, 46, 48, 48, 37, 38,
 39, 40, 42, 43, 47, 47, 47, 48, 48, 47, 46, 46, 46, 46, 47, 46, 46,
 47, 47, 47, 48, 50, 52, 53, 53, 53, 53, 54, 54, 49, 47, 46, 45, 45, 46,
 45, 47, 49, 53, 53, 56, 57, 58, 59, 59,
 // Size 8x32
 32, 31, 33, 37, 45, 48, 49, 50, 31, 31, 33, 38, 45, 47, 48, 49, 31, 31,
 34, 38, 45, 47, 47, 48, 31, 31, 34, 38, 45, 47, 47, 48, 31, 32, 34, 39,
 45, 46, 46, 47, 30, 32, 35, 40, 44, 46, 45, 46, 30, 32, 35, 40, 44, 46,
 45, 46, 31, 33, 35, 40, 45, 46, 45, 46, 33, 35, 37, 42, 46, 47, 45, 46,
 33, 36, 38, 43, 46, 47, 46, 46, 33, 36, 38, 43, 46, 47, 46, 46, 35, 38,
 41, 45, 47, 47, 45, 46, 37, 40, 43, 47, 47, 47, 45, 46, 37, 40, 43, 47,
 47, 47, 45, 46, 39, 41, 43, 47, 48, 48, 47, 47, 42, 43, 44, 47, 49, 50,
 49, 50, 42, 43, 44, 47, 49, 50, 49, 50, 43, 43, 45, 47, 50, 50, 50, 50,
 47, 46, 46, 48, 51, 52, 53, 53, 49, 46, 47, 48, 52, 53, 53, 54, 49, 46,
 47, 48, 52, 53, 53, 54, 48, 46, 46, 47, 52, 53, 55, 55, 48, 46, 46, 47,
 51, 53, 56, 56, 48, 46, 46, 47, 51, 53, 56, 56, 48, 45, 46, 46, 51, 53,
 57, 57, 49, 45, 45, 46, 51, 53, 58, 59, 49, 45, 45, 46, 51, 53, 58, 59,
 49, 45, 45, 46, 52, 53, 58, 60, 50, 46, 46, 46, 52, 54, 59, 61, 50, 46,
 46, 46, 52, 54, 59, 61, 50, 46, 46, 46, 52, 54, 59, 61, 51, 47, 47, 47,
 52, 54, 60, 62,
 // Size 32x8
 32, 31, 31, 31, 31, 30, 30, 31, 33, 33, 33, 35, 37, 37, 39, 42, 42, 43,
 47, 49, 49, 48, 48, 48, 48, 49, 49, 49, 50, 50, 50, 51, 31, 31, 31, 31,
 32, 32, 32, 33, 35, 36, 36, 38, 40, 40, 41, 43, 43, 43, 46, 46, 46, 46,
 46, 46, 45, 45, 45, 45, 46, 46, 46, 47, 33, 33, 34, 34, 34, 35, 35, 35,
 37, 38, 38, 41, 43, 43, 43, 44, 44, 45, 46, 47, 47, 46, 46, 46, 46, 45,
 45, 45, 46, 46, 46, 47, 37, 38, 38, 38, 39, 40, 40, 40, 42, 43, 43, 45,
 47, 47, 47, 47, 47, 47, 48, 48, 48, 47, 47, 47, 46, 46, 46, 46, 46,
 46, 47, 45, 45, 45, 45, 45, 44, 44, 45, 46, 46, 46, 47, 47, 47, 48, 49,
 49, 50, 51, 52, 52, 52, 51, 51, 51, 51, 51, 52, 52, 52, 52, 52, 48, 47,
 47, 47, 46, 46, 46, 46, 47, 47, 47, 47, 47, 48, 50, 50, 50, 52, 53,
 53, 53, 53, 53, 53, 53, 54, 54, 54, 49, 48, 47, 47, 46, 45,
 45, 45, 45, 46, 46, 45, 45, 45, 47, 49, 49, 50, 53, 53, 53, 55, 56, 56,
 57, 58, 58, 58, 59, 59, 59, 60, 50, 49, 48, 48, 47, 46, 46, 46, 46, 46,
 46, 46, 46, 46, 47, 50, 50, 50, 53, 54, 54, 55, 56, 56, 57, 59, 59, 60,
 61, 61, 61, 62,
 // Size 4x32


```

31, 37, 48, 49, 31, 38, 47, 48, 31, 38, 47, 47, 31, 38, 47, 47, 31, 39,
46, 46, 31, 40, 46, 45, 31, 40, 46, 45, 32, 40, 46, 45, 34, 42, 47, 45,
35, 43, 47, 46, 35, 43, 47, 46, 37, 45, 47, 45, 39, 47, 47, 45, 39, 47,
47, 45, 40, 47, 48, 47, 42, 47, 50, 49, 42, 47, 50, 49, 43, 47, 50, 50,
46, 48, 52, 53, 47, 48, 53, 53, 47, 48, 53, 53, 47, 47, 53, 55, 47, 47,
53, 56, 47, 47, 53, 56, 47, 46, 53, 57, 46, 46, 53, 58, 46, 46, 53, 58,
47, 46, 53, 58, 48, 46, 54, 59, 48, 46, 54, 59, 48, 46, 54, 59, 49, 47,
54, 60,
// Size 32x4
31, 31, 31, 31, 31, 31, 31, 31, 32, 34, 35, 35, 37, 39, 39, 40, 42, 42, 43,
46, 47, 47, 47, 47, 47, 47, 46, 46, 47, 48, 48, 48, 49, 37, 38, 38, 38,
39, 40, 40, 40, 42, 43, 43, 45, 47, 47, 47, 47, 47, 47, 48, 48, 48, 47,
47, 47, 46, 46, 46, 46, 46, 46, 46, 47, 48, 47, 47, 47, 46, 46, 46, 46,
47, 47, 47, 47, 47, 47, 48, 50, 50, 50, 52, 53, 53, 53, 53, 53, 53,
53, 53, 54, 54, 54, 54, 49, 48, 47, 47, 46, 45, 45, 45, 45, 46, 46, 45,
45, 45, 47, 49, 49, 50, 53, 53, 53, 55, 56, 56, 57, 58, 58, 58, 59, 59,
59, 60,
},
{
{ // Luma
// Size 4x4
32, 32, 34, 38, 32, 33, 35, 39, 34, 35, 39, 45, 38, 39, 45, 54,
// Size 8x8
31, 31, 32, 32, 33, 34, 37, 41, 31, 32, 32, 32, 33, 34, 36, 39, 32, 32,
32, 33, 34, 35, 37, 40, 32, 32, 33, 34, 35, 36, 38, 41, 33, 33, 34, 35,
37, 39, 41, 44, 34, 34, 35, 36, 39, 43, 46, 49, 37, 36, 37, 38, 41, 46,
51, 54, 41, 39, 40, 41, 44, 49, 54, 58,
// Size 16x16
32, 31, 31, 31, 31, 31, 31, 32, 32, 34, 34, 36, 36, 39, 39, 44, 31, 32,
32, 32, 32, 32, 32, 32, 34, 34, 35, 35, 38, 38, 42, 31, 32, 32, 32, 32,
32, 32, 32, 32, 32, 34, 34, 35, 35, 38, 38, 42, 31, 32, 32, 32, 32, 32,
32, 32, 32, 33, 33, 34, 34, 37, 37, 41, 31, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 34, 34, 37, 37, 41, 31, 32, 32, 32, 32, 33, 33, 34, 34, 35,
35, 36, 36, 39, 39, 42, 31, 32, 32, 32, 32, 33, 33, 34, 34, 35, 35, 36,
36, 39, 39, 42, 32, 32, 32, 32, 34, 34, 35, 35, 37, 37, 38, 38, 40,
40, 42, 32, 32, 32, 32, 34, 34, 35, 35, 37, 37, 38, 38, 40, 40, 42,
34, 34, 34, 33, 33, 35, 35, 37, 37, 39, 39, 42, 42, 45, 45, 47, 34, 34,
34, 33, 33, 35, 35, 37, 37, 39, 39, 42, 42, 45, 45, 47, 36, 35, 35, 34,
34, 36, 36, 38, 38, 42, 42, 48, 48, 50, 50, 54, 36, 35, 35, 34, 34, 36,
36, 38, 38, 42, 42, 48, 48, 50, 50, 54, 39, 38, 38, 37, 37, 39, 39, 40,
40, 45, 45, 50, 50, 54, 54, 58, 39, 38, 38, 37, 37, 39, 39, 40, 40, 45,
45, 50, 50, 54, 54, 58, 44, 42, 42, 41, 41, 42, 42, 42, 42, 47, 47, 54,
54, 58, 58, 63,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 33,
34, 34, 34, 35, 36, 36, 36, 37, 39, 39, 39, 41, 44, 44, 31, 31, 31, 31,
31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34,
35, 35, 35, 37, 39, 39, 39, 41, 43, 43, 31, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 34, 35, 35, 35, 37,
38, 38, 38, 40, 42, 42, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 33, 34, 34, 34, 34, 35, 35, 35, 37, 38, 38, 38, 40,
42, 42, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 34, 34, 34, 34, 35, 35, 35, 37, 38, 38, 38, 40, 42, 42, 31, 31,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34,
34, 34, 35, 35, 35, 36, 38, 38, 39, 41, 41, 31, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 36,
37, 37,
37, 39, 41, 41, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 33, 33, 33, 34, 34, 34, 34, 36, 37, 37, 39, 41, 41,
31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 34,
34, 34, 34, 35, 35, 35, 35, 37, 38, 38, 38, 40, 41, 41, 31, 32, 32, 32,
32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 36,
36, 36, 36, 38, 39, 39, 39, 40, 42, 42, 31, 32, 32, 32, 32, 32, 32,
32, 33, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 36, 36, 36, 36, 38,
39, 39, 39, 40, 42, 42, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33,
33, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 36, 36, 36, 36, 38, 39,
39, 39, 40,
42, 42, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34,
34, 35, 36, 36, 36, 36, 37, 37, 37, 38, 40, 40, 40, 41, 42, 42, 32, 32,
32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 34, 35, 35, 36, 37, 37,

```

```

37, 37, 38, 38, 38, 39, 40, 40, 40, 41, 42, 42, 32, 32, 32, 32, 32, 32,
32, 32, 32, 33, 34, 34, 34, 34, 35, 35, 35, 36, 37, 37, 37, 37, 38, 38,
38, 39, 40, 40, 40, 41, 42, 42, 32, 32, 32, 32, 32, 32, 32, 32, 33,
34, 34, 34, 34, 35, 35, 35, 36, 37, 37, 37, 37, 38, 38, 38, 39, 40, 40,
40, 41, 42, 42, 33, 33, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 35,
36, 36, 36, 37, 38, 38, 38, 39, 40, 40, 40, 41, 42, 42, 42, 44, 45, 45,
34, 34, 34, 34, 34, 34, 33, 33, 33, 34, 35, 35, 35, 36, 37, 37, 37, 38,
39, 39, 39, 41, 42, 42, 42, 44, 45, 45, 45, 46, 47, 47, 34, 34, 34, 34,
34, 34, 33, 33, 33, 34, 35, 35, 35, 36, 37, 37, 37, 38, 39, 39, 39, 41,
42, 42, 42, 44, 45, 45, 45, 46, 47, 47, 34, 34, 34, 34, 34, 34, 33, 33,
33, 34, 35, 35, 35, 36, 37, 37, 37, 38, 39, 39, 39, 41, 42, 42, 42, 44,
45, 45, 45, 46, 47, 47, 35, 34, 34, 34, 34, 34, 34, 34, 35, 36, 36,
36, 36, 37, 37, 37, 39, 41, 41, 41, 43, 45, 45, 45, 46, 47, 47, 47, 49,
50, 50, 36, 35, 35, 35, 35, 35, 34, 34, 34, 35, 36, 36, 36, 37, 38, 38,
38, 40, 42, 42, 42, 45, 48, 48, 48, 49, 50, 50, 50, 52, 54, 54, 36, 35,
35, 35, 35, 35, 34, 34, 34, 35, 36, 36, 36, 37, 38, 38, 38, 40, 42, 42,
42, 45, 48, 48, 48, 49, 50, 50, 50, 52, 54, 54, 36, 35, 35, 35, 35, 35,
34, 34, 34, 35, 36, 36, 36, 37, 38, 38, 38, 40, 42, 42, 42, 45, 48, 48,
48, 49, 50, 50, 50, 52, 54, 54, 37, 37, 37, 37, 37, 36, 36, 36, 36, 37,
38, 38, 38, 38, 39, 39, 39, 41, 44, 44, 44, 46, 49, 49, 49, 51, 52, 52,
52, 54, 56, 56, 39, 39, 38, 38, 38, 37, 37, 37, 38, 39, 39, 39, 40,
40, 40, 40, 42, 45, 45, 45, 47, 50, 50, 50, 52, 54, 54, 54, 56, 58, 58,
39, 39, 38, 38, 38, 38, 37, 37, 37, 38, 39, 39, 39, 40, 40, 40, 40, 42,
45, 45, 45, 47, 50, 50, 50, 52, 54, 54, 54, 56, 58, 58, 39, 39, 38, 38,
38, 38, 37, 37, 37, 38, 39, 39, 39, 40, 40, 40, 40, 42, 45, 45, 45, 47,
50, 50, 50, 52, 54, 54, 54, 56, 58, 58, 41, 41, 40, 40, 40, 39, 39, 39,
39, 40, 40, 40, 41, 41, 41, 41, 44, 46, 46, 46, 49, 52, 52, 52, 54,
56, 56, 56, 58, 60, 60, 44, 43, 42, 42, 42, 41, 41, 41, 41, 41, 42, 42,
42, 42, 42, 42, 42, 45, 47, 47, 47, 50, 54, 54, 54, 56, 58, 58, 58, 60,
63, 63, 44, 43, 42, 42, 42, 41, 41, 41, 41, 41, 42, 42, 42, 42, 42, 42,
42, 45, 47, 47, 47, 50, 54, 54, 54, 56, 58, 58, 58, 60, 63, 63,
// Size 4x8
31, 32, 34, 39, 32, 32, 34, 38, 32, 33, 34, 38, 32, 33, 36, 40, 33, 34,
38, 42, 34, 36, 41, 47, 37, 38, 44, 52, 40, 40, 46, 56,
// Size 8x4
31, 32, 32, 32, 33, 34, 37, 40, 32, 32, 33, 33, 34, 36, 38, 40, 34, 34,
34, 36, 38, 41, 44, 46, 39, 38, 38, 40, 42, 47, 52, 56,
// Size 8x16
32, 31, 31, 32, 32, 36, 36, 44, 31, 32, 32, 32, 32, 35, 35, 42, 31, 32,
32, 32, 32, 35, 35, 42, 31, 32, 32, 33, 33, 34, 34, 41, 31, 32, 32, 33,
33, 34, 34, 41, 32, 32, 32, 34, 34, 36, 36, 42, 32, 32, 32, 34, 34, 36,
36, 42, 32, 33, 33, 35, 35, 38, 38, 42, 32, 33, 33, 35, 35, 38, 38, 42,
34, 34, 34, 37, 37, 42, 42, 48, 34, 34, 34, 37, 37, 42, 42, 48, 36, 34,
34, 38, 38, 48, 48, 54, 36, 34, 34, 38, 38, 48, 48, 54, 39, 37, 37, 40,
40, 50, 50, 58, 39, 37, 37, 40, 40, 50, 50, 58, 44, 41, 41, 43, 43, 53,
53, 63,
// Size 16x8
32, 31, 31, 31, 31, 32, 32, 32, 32, 34, 34, 36, 36, 39, 39, 44, 31, 32,
32, 32, 32, 32, 32, 33, 33, 34, 34, 34, 34, 37, 37, 41, 31, 32, 32, 32,
32, 32, 32, 33, 33, 34, 34, 34, 34, 37, 37, 41, 32, 32, 32, 33, 33, 34,
34, 35, 35, 37, 37, 38, 38, 40, 40, 43, 32, 32, 33, 33, 34, 34, 35,
35, 37, 37, 38, 38, 40, 40, 43, 36, 35, 35, 34, 34, 36, 36, 38, 38, 42, 42,
42, 48, 48, 50, 50, 53, 36, 35, 35, 34, 34, 36, 36, 38, 38, 42, 42, 48,
48, 50, 50, 53, 44, 42, 42, 41, 41, 42, 42, 42, 42, 48, 48, 54, 54, 58,
58, 63,
// Size 16x32
32, 31, 31, 31, 31, 32, 32, 32, 32, 34, 36, 36, 36, 39, 44, 44, 31, 31,
31, 31, 31, 32, 32, 32, 32, 34, 35, 35, 35, 39, 43, 43, 31, 32, 32, 32,
32, 32, 32, 32, 32, 34, 35, 35, 35, 38, 42, 42, 31, 32, 32, 32, 32, 32,
32, 34, 35, 35, 35, 38, 42, 42, 31, 32, 32, 32, 32, 32, 32, 32, 34,
35, 35, 35, 38, 41, 41, 31, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34,
34, 37, 41, 41, 31, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 37,
41, 41, 31, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 37, 41, 41,
31, 32, 32, 32, 32, 33, 33, 33, 33, 34, 35, 35, 35, 38, 41, 41, 32, 32,
32, 32, 32, 33, 34, 34, 34, 35, 36, 36, 36, 39, 42, 42, 32, 32, 32, 32,
32, 33, 34, 34, 34, 35, 36, 36, 36, 39, 42, 42, 32, 32, 32, 32, 33,
34, 34, 34, 35, 36, 36, 36, 39, 42, 42, 32, 32, 32, 32, 32, 33, 34, 34,
34, 36, 37, 37, 37, 40, 42, 42, 32, 32, 33, 33, 33, 34, 35, 35, 35, 37,
38, 38, 38, 40, 42, 42, 32, 32, 33, 33, 33, 34, 35, 35, 35, 37, 38, 38,
38, 40, 42, 42, 32, 32, 33, 33, 33, 34, 35, 35, 35, 37, 38, 38, 40,

```

```

42, 42, 33, 33, 33, 33, 33, 34, 36, 36, 36, 38, 40, 40, 40, 42, 45, 45,
34, 34, 34, 34, 34, 35, 37, 37, 37, 39, 42, 42, 42, 45, 48, 48, 34, 34,
34, 34, 34, 35, 37, 37, 37, 39, 42, 42, 42, 45, 48, 48, 34, 34, 34, 34,
34, 35, 37, 37, 37, 39, 42, 42, 42, 45, 48, 48, 35, 34, 34, 34, 34, 36,
37, 37, 37, 41, 45, 45, 45, 47, 50, 50, 36, 35, 34, 34, 34, 36, 38, 38,
38, 43, 48, 48, 48, 51, 54, 54, 36, 35, 34, 34, 34, 36, 38, 38, 38, 38, 43,
48, 48, 48, 51, 54, 54, 36, 35, 34, 34, 34, 36, 38, 38, 38, 43, 48, 48,
48, 51, 54, 54, 37, 37, 36, 36, 36, 38, 39, 39, 39, 44, 49, 49, 49, 52,
56, 56, 39, 38, 37, 37, 37, 39, 40, 40, 40, 45, 50, 50, 50, 54, 58, 58,
39, 38, 37, 37, 37, 39, 40, 40, 40, 45, 50, 50, 50, 54, 58, 58, 39, 38,
37, 37, 37, 39, 40, 40, 40, 45, 50, 50, 50, 54, 58, 58, 41, 40, 39, 39,
39, 40, 42, 42, 42, 46, 52, 52, 52, 56, 60, 60, 44, 42, 41, 41, 41, 42,
43, 43, 43, 48, 53, 53, 53, 58, 63, 63, 44, 42, 41, 41, 41, 42, 43, 43,
43, 48, 53, 53, 53, 58, 63, 63,
// Size 32x16
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 33,
34, 34, 34, 35, 36, 36, 36, 37, 39, 39, 39, 41, 44, 44, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 34,
35, 35, 35, 37, 38, 38, 38, 40, 42, 42, 31, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 36,
37, 37, 37, 39, 41, 41, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 36, 37, 37, 37, 39,
41, 41, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33,
33, 33, 34, 34, 34, 34, 34, 34, 34, 36, 37, 37, 37, 39, 41, 41, 32, 32,
32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35,
35, 36, 36, 36, 36, 38, 39, 39, 39, 40, 42, 42, 32, 32, 32, 32, 32, 32,
33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 35, 36, 37, 37, 37, 38, 38,
38, 39, 40, 40, 40, 42, 43, 43, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33,
34, 34, 34, 34, 35, 35, 35, 36, 37, 37, 37, 37, 38, 38, 38, 39, 40, 40,
40, 42, 43, 43, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 34,
35, 35, 35, 36, 37, 37, 37, 37, 38, 38, 38, 39, 40, 40, 40, 42, 43, 43,
34, 34, 34, 34, 34, 34, 33, 33, 33, 34, 35, 35, 35, 36, 37, 37, 37, 38,
39, 39, 39, 41, 43, 43, 43, 44, 45, 45, 45, 46, 48, 48, 36, 35, 35, 35,
35, 35, 34, 34, 34, 35, 36, 36, 36, 37, 38, 38, 38, 40, 42, 42, 42, 45,
48, 48, 48, 49, 50, 50, 50, 52, 53, 53, 36, 35, 35, 35, 35, 35, 34, 34,
34, 35, 36, 36, 36, 37, 38, 38, 38, 40, 42, 42, 42, 45, 48, 48, 48, 49,
50, 50, 50, 52, 53, 53, 36, 35, 35, 35, 35, 34, 34, 34, 35, 36, 36,
36, 37, 38, 38, 38, 40, 42, 42, 42, 45, 48, 48, 48, 49, 50, 50, 50, 52,
53, 53, 39, 39, 38, 38, 38, 38, 37, 37, 37, 38, 39, 39, 39, 40, 40, 40,
40, 42, 45, 45, 45, 47, 51, 51, 51, 52, 54, 54, 54, 56, 58, 58, 44, 43,
42, 42, 42, 41, 41, 41, 41, 41, 42, 42, 42, 42, 42, 42, 45, 48, 48,
48, 50, 54, 54, 54, 56, 58, 58, 58, 60, 63, 63, 44, 43, 42, 42, 42, 41,
41, 41, 41, 41, 42, 42, 42, 42, 42, 42, 42, 45, 48, 48, 48, 50, 54, 54,
54, 56, 58, 58, 58, 60, 63, 63,
// Size 4x16
31, 32, 34, 39, 32, 32, 34, 38, 32, 32, 34, 38, 32, 32, 33, 37, 32, 32,
33, 37, 32, 33, 35, 39, 32, 33, 35, 39, 32, 34, 37, 40, 32, 34, 37, 40,
34, 35, 39, 45, 34, 35, 39, 45, 35, 36, 43, 51, 35, 36, 43, 51, 38, 39,
45, 54, 38, 39, 45, 54, 42, 42, 48, 58,
// Size 16x4
31, 32, 32, 32, 32, 32, 32, 32, 34, 34, 35, 35, 38, 38, 42, 32, 32,
32, 32, 32, 33, 33, 34, 34, 35, 35, 36, 36, 39, 39, 42, 34, 34, 34, 33,
33, 35, 35, 37, 37, 39, 39, 43, 43, 45, 45, 48, 39, 38, 38, 37, 37, 39,
39, 40, 40, 45, 45, 51, 51, 54, 54, 58,
// Size 8x32
32, 31, 31, 32, 32, 36, 36, 44, 31, 31, 31, 32, 32, 35, 35, 43, 31, 32,
32, 32, 32, 35, 35, 42, 31, 32, 32, 32, 32, 35, 35, 42, 31, 32, 32, 32,
32, 35, 35, 42, 31, 32, 32, 32, 32, 35, 35, 41, 31, 32, 32, 33, 33, 34,
34, 41, 31, 32, 32, 33, 33, 34, 34, 41, 31, 32, 32, 33, 33, 34, 34, 41,
31, 32, 32, 33, 33, 35, 35, 41, 32, 32, 32, 34, 34, 36, 36, 42, 32, 32,
32, 34, 34, 36, 36, 42, 32, 32, 34, 34, 36, 36, 42, 32, 32, 32, 34,
34, 37, 37, 42, 32, 33, 33, 35, 35, 38, 38, 42, 32, 33, 33, 35, 35, 38,
38, 42, 32, 33, 33, 35, 35, 38, 38, 42, 33, 33, 33, 36, 36, 40, 40, 45,
34, 34, 34, 37, 37, 42, 42, 48, 34, 34, 34, 37, 37, 42, 42, 48, 34, 34,
34, 37, 37, 42, 42, 48, 35, 34, 34, 37, 37, 45, 45, 50, 36, 34, 34, 38,
38, 48, 48, 54, 36, 34, 34, 38, 38, 48, 48, 54, 36, 34, 34, 38, 38, 48,
48, 54, 37, 36, 36, 39, 39, 49, 49, 56, 39, 37, 37, 40, 40, 50, 50, 58, 58,
39, 37, 37, 40, 40, 50, 50, 58, 39, 37, 37, 40, 40, 50, 50, 58, 41, 39,
39, 42, 42, 52, 52, 60, 44, 41, 41, 43, 43, 53, 53, 63, 44, 41, 41, 43,
43, 53, 53, 63,
// Size 32x8

```

```

32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 33,
34, 34, 34, 35, 36, 36, 36, 37, 39, 39, 39, 41, 44, 44, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34,
34, 34, 34, 36, 37, 37, 37, 39, 41, 41, 31, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 36,
37, 37, 37, 39, 41, 41, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34,
34, 34, 35, 35, 35, 36, 37, 37, 37, 37, 38, 38, 38, 39, 40, 40, 40, 42,
43, 43, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35,
35, 36, 37, 37, 37, 37, 38, 38, 38, 39, 40, 40, 40, 42, 43, 43, 36, 35,
35, 35, 35, 35, 34, 34, 34, 35, 36, 36, 36, 37, 38, 38, 38, 40, 42, 42,
42, 45, 48, 48, 48, 49, 50, 50, 52, 53, 53, 36, 35, 35, 35, 35, 35,
34, 34, 34, 35, 36, 36, 36, 37, 38, 38, 38, 40, 42, 42, 42, 45, 48, 48,
48, 49, 50, 50, 50, 52, 53, 53, 44, 43, 42, 42, 42, 41, 41, 41, 41, 41,
42, 42, 42, 42, 42, 42, 42, 45, 48, 48, 48, 50, 54, 54, 54, 56, 58, 58,
58, 60, 63, 63,
// Size 4x32
31, 32, 34, 39, 31, 32, 34, 39, 32, 32, 34, 38, 32, 32, 34, 38, 32, 32,
34, 38, 32, 32, 34, 38, 32, 32, 33, 37, 32, 32, 33, 37, 32, 32, 33, 37,
32, 33, 34, 38, 32, 33, 35, 39, 32, 33, 35, 39, 32, 33, 35, 39, 32, 33,
36, 40, 32, 34, 37, 40, 32, 34, 37, 40, 32, 34, 37, 40, 33, 34, 38, 42,
34, 35, 39, 45, 34, 35, 39, 45, 34, 35, 39, 45, 34, 36, 41, 47, 35, 36,
43, 51, 35, 36, 43, 51, 35, 36, 43, 51, 37, 38, 44, 52, 38, 39, 45, 54,
38, 39, 45, 54, 38, 39, 45, 54, 40, 40, 46, 56, 42, 42, 48, 58, 42, 42,
48, 58,
// Size 32x4
31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33,
34, 34, 34, 34, 35, 35, 35, 37, 38, 38, 38, 40, 42, 42, 32, 32, 32, 32,
32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 35, 36,
36, 36, 36, 38, 39, 39, 39, 40, 42, 42, 34, 34, 34, 34, 34, 34, 33, 33,
33, 34, 35, 35, 35, 36, 37, 37, 37, 38, 39, 39, 39, 41, 43, 43, 43, 44,
45, 45, 45, 46, 48, 48, 39, 39, 38, 38, 38, 38, 37, 37, 37, 38, 39, 39,
39, 40, 40, 40, 40, 42, 45, 45, 45, 47, 51, 51, 51, 52, 54, 54, 54, 56,
58, 58,
},
{ // Chroma
// Size 4x4
31, 34, 42, 47, 34, 39, 45, 46, 42, 45, 48, 49, 47, 46, 49, 54,
// Size 8x8
31, 31, 32, 35, 39, 45, 48, 48, 31, 31, 33, 37, 41, 44, 46, 46, 32, 33,
35, 39, 42, 45, 46, 45, 35, 37, 39, 43, 45, 47, 47, 46, 39, 41, 42, 45,
47, 48, 48, 47, 45, 44, 45, 47, 48, 50, 51, 51, 48, 46, 46, 47, 48, 51,
53, 54, 48, 46, 45, 46, 47, 51, 54, 56,
// Size 16x16
32, 31, 31, 30, 33, 33, 36, 36, 41, 41, 49, 49, 48, 48, 49, 31, 31,
31, 31, 31, 34, 34, 38, 38, 42, 42, 47, 47, 47, 47, 47, 31, 31, 31, 31,
31, 34, 34, 38, 38, 42, 42, 47, 47, 47, 47, 47, 30, 31, 31, 32, 32, 35,
35, 40, 40, 42, 42, 46, 46, 45, 45, 45, 30, 31, 31, 32, 32, 35, 35, 40,
40, 42, 42, 46, 46, 45, 45, 45, 33, 34, 34, 35, 35, 39, 39, 43, 43, 45,
45, 47, 47, 46, 46, 45, 33, 34, 34, 35, 35, 39, 39, 43, 43, 45, 45, 47,
47, 46, 46, 45, 36, 38, 38, 40, 40, 43, 43, 47, 47, 47, 47, 48, 48, 46,
46, 45, 36, 38, 38, 40, 40, 43, 43, 47, 47, 47, 47, 48, 48, 46, 46, 45,
41, 42, 42, 42, 42, 45, 45, 47, 47, 48, 48, 50, 50, 49, 49, 49, 41, 42,
42, 42, 42, 45, 45, 47, 47, 48, 48, 50, 50, 49, 49, 49, 47, 47, 46,
46, 47, 47, 48, 48, 50, 50, 53, 53, 53, 53, 53, 49, 47, 47, 46, 46, 47,
47, 48, 48, 50, 50, 53, 53, 53, 53, 48, 47, 47, 45, 45, 46, 46, 46,
46, 49, 49, 53, 53, 54, 54, 55, 48, 47, 47, 45, 45, 46, 46, 46, 46, 49,
49, 53, 53, 54, 54, 55, 49, 47, 47, 45, 45, 45, 45, 45, 45, 49, 49, 53,
53, 55, 55, 58,
// Size 32x32
32, 31, 31, 31, 31, 31, 30, 30, 30, 32, 33, 33, 33, 35, 36, 36, 36, 39,
41, 41, 41, 45, 49, 49, 49, 49, 48, 48, 48, 49, 49, 49, 31, 31, 31, 31,
31, 31, 31, 31, 31, 32, 34, 34, 34, 35, 37, 37, 37, 39, 42, 42, 42, 45,
48, 48, 48, 48, 48, 48, 48, 48, 48, 48, 31, 31, 31, 31, 31, 31, 31, 31,
31, 33, 34, 34, 34, 36, 38, 38, 38, 40, 42, 42, 42, 45, 47, 47, 47, 47,
47, 47, 47, 47, 47, 47, 31, 31, 31, 31, 31, 31, 31, 31, 31, 33, 34, 34,
34, 36, 38, 38, 38, 40, 42, 42, 45, 47, 47, 47, 47, 47, 47, 47, 47, 47,
47, 47, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 33, 34, 34, 34, 36, 38, 38,
38, 40, 42, 42, 42, 45, 47, 47, 47, 47, 47, 47, 47, 47, 47, 31, 31,
31, 31, 31, 31, 31, 31, 33, 35, 35, 35, 37, 39, 39, 39, 41, 42, 42,
42, 44, 47, 47, 47, 46, 46, 46, 46, 46, 46, 46, 46, 30, 31, 31, 31, 31, 31,

```

32, 32, 32, 33, 35, 35, 35, 37, 40, 40, 40, 41, 42, 42, 42, 44, 46, 46,
 46, 46, 45, 45, 45, 45, 45, 45, 30, 31, 31, 31, 31, 31, 32, 32, 32, 33,
 35, 35, 35, 37, 40, 40, 40, 41, 42, 42, 42, 44, 46, 46, 46, 46, 45, 45,
 45, 45, 45, 45, 30, 31, 31, 31, 31, 31, 32, 32, 32, 33, 35, 35, 35, 37,
 40, 40, 40, 41, 42, 42, 42, 44, 46, 46, 46, 46, 45, 45, 45, 45, 45, 45,
 32, 32, 33, 33, 33, 33, 33, 33, 33, 35, 37, 37, 39, 41, 41, 41, 42,
 43, 43, 43, 45, 47, 47, 47, 46, 46, 46, 46, 45, 45, 45, 33, 34, 34, 34,
 34, 35, 35, 35, 35, 37, 39, 39, 39, 41, 43, 43, 43, 44, 45, 45, 45, 46,
 47, 47, 47, 47, 46, 46, 46, 46, 45, 45, 33, 34, 34, 34, 34, 35, 35, 35,
 35, 37, 39, 39, 39, 41, 43, 43, 43, 44, 45, 45, 45, 46, 47, 47, 47, 47,
 46, 46, 46, 46, 45, 45, 33, 34, 34, 34, 34, 35, 35, 35, 37, 39, 39,
 39, 41, 43, 43, 43, 44, 45, 45, 45, 46, 47, 47, 47, 47, 46, 46, 46, 46,
 45, 45, 35, 35, 36, 36, 36, 37, 37, 37, 37, 39, 41, 41, 41, 43, 45, 45,
 45, 45, 46, 46, 46, 47, 47, 47, 47, 47, 46, 46, 46, 46, 45, 45, 36, 37,
 38, 38, 38, 39, 40, 40, 40, 41, 43, 43, 43, 45, 47, 47, 47, 47, 47, 47,
 47, 47, 48, 48, 48, 47, 46, 46, 46, 45, 45, 36, 37, 38, 38, 38, 39,
 40, 40, 40, 41, 43, 43, 43, 45, 47, 47, 47, 47, 47, 47, 47, 48, 48,
 48, 47, 46, 46, 46, 46, 45, 45, 36, 37, 38, 38, 38, 39, 40, 40, 40, 41,
 43, 43, 43, 45, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 47, 46, 46,
 46, 46, 45, 45, 39, 39, 40, 40, 40, 41, 41, 41, 41, 42, 44, 44, 44, 45,
 47, 47, 47, 47, 48, 48, 48, 48, 48, 49, 49, 49, 48, 48, 48, 47, 47, 47,
 41, 42, 42, 42, 42, 42, 42, 42, 42, 43, 45, 45, 45, 46, 47, 47, 47, 48,
 48, 48, 48, 49, 50, 50, 50, 50, 49, 49, 49, 49, 49, 49, 41, 42, 42, 42,
 42, 42, 42, 42, 42, 43, 45, 45, 45, 46, 47, 47, 47, 48, 48, 48, 48, 49,
 50, 50, 50, 50, 49, 49, 49, 49, 49, 49, 41, 42, 42, 42, 42, 42, 42,
 42, 43, 45, 45, 46, 47, 47, 47, 48, 48, 48, 48, 49, 50, 50, 50, 50,
 49, 49, 49, 49, 49, 49, 45, 45, 45, 45, 44, 44, 44, 45, 46, 46,
 46, 47, 47, 47, 47, 48, 49, 49, 49, 50, 51, 51, 51, 51, 51, 51, 51,
 51, 51, 49, 48, 47, 47, 47, 47, 46, 46, 46, 47, 47, 47, 47, 48, 48,
 48, 49, 50, 50, 51, 53, 53, 53, 53, 53, 53, 53, 53, 53, 53, 49, 48,
 47, 47, 47, 47,
 50, 51, 53, 53, 53, 53, 53, 53, 53, 53, 53, 49, 48, 47, 47, 47,
 46, 46, 46, 47, 47, 47, 47, 47, 48, 48, 48, 49, 50, 50, 50, 51, 53, 53,
 53, 53, 53, 53, 53, 53, 53, 53, 49, 48, 47, 47, 47, 46, 46, 46, 46,
 47, 47, 47, 47, 47, 47, 47, 48, 50, 50, 50, 51, 53, 53, 53, 53, 53,
 53, 54, 54, 54, 48, 48, 47, 47, 46, 45, 45, 45, 46, 46, 46, 46, 46,
 46, 46, 46, 48, 49, 49, 49, 51, 53, 53, 53, 53, 54, 54, 54, 55, 55,
 48, 48, 47, 47, 47, 46, 45, 45, 45, 46, 46, 46, 46, 46, 46, 46, 48,
 49, 49, 49, 51, 53, 53, 53, 53, 54, 54, 54, 55, 55, 55, 48, 48, 47, 47,
 47, 46, 45, 45, 45, 46, 46, 46, 46, 46, 46, 46, 48, 49, 49, 49, 51,
 53, 53, 53, 53, 54, 54, 54, 55, 55, 49, 48, 47, 47, 46, 45, 45,
 45, 45, 46, 46, 46, 46, 46, 46, 46, 47, 49, 49, 49, 51, 53, 53, 53, 54, 55, 55, 57,
 58, 58, 49, 48, 47, 47, 46, 45, 45, 45, 45, 45, 45, 45, 45, 45,
 45, 47, 49, 49, 51, 53, 53, 53, 54, 55, 55, 55, 57, 58, 58,
 // Size 4x8
 31, 34, 42, 48, 31, 35, 42, 46, 33, 37, 44, 46, 36, 41, 46, 46, 40, 44,
 48, 48, 45, 46, 49, 51, 47, 47, 50, 54, 47, 46, 49, 55,
 // Size 8x4
 31, 31, 33, 36, 40, 45, 47, 47, 34, 35, 37, 41, 44, 46, 47, 46, 42, 42,
 44, 46, 48, 49, 50, 49, 48, 46, 46, 46, 48, 51, 54, 55,
 // Size 8x16
 32, 31, 31, 37, 37, 48, 48, 49, 31, 31, 31, 38, 38, 47, 47, 47, 31, 31,
 31, 38, 38, 47, 47, 47, 30, 32, 32, 40, 40, 46, 46, 45, 30, 32, 32, 40,
 40, 46, 46, 45, 33, 36, 36, 43, 43, 47, 47, 46, 33, 36, 36, 43, 43, 47,
 47, 46, 37, 40, 40, 47, 47, 47, 47, 45, 37, 40, 40, 47, 47, 47, 45,
 42, 43, 43, 47, 47, 50, 50, 49, 42, 43, 43, 47, 47, 50, 50, 49, 49, 46,
 46, 48, 48, 53, 53, 53, 49, 46, 46, 48, 48, 53, 53, 53, 48, 46, 46, 47,
 47, 53, 53, 56, 48, 46, 46, 47, 47, 53, 53, 56, 49, 45, 45, 46, 46, 53,
 53, 58,
 // Size 16x8
 32, 31, 31, 30, 30, 33, 33, 37, 37, 42, 42, 49, 49, 48, 48, 49, 31, 31,
 31, 32, 32, 36, 36, 40, 40, 43, 43, 46, 46, 46, 46, 45, 31, 31, 31, 32,
 32, 36, 36, 40, 40, 43, 43, 46, 46, 46, 46, 45, 37, 38, 38, 40, 40, 43,
 43, 47, 47, 47, 48, 48, 47, 47, 46, 47, 46, 37, 38, 40, 43, 43, 47,
 47, 47, 47, 48, 48, 47, 47, 46, 48, 47, 47, 46, 47, 47, 47, 50,
 50, 53, 53, 53, 53, 53, 48, 47, 47, 46, 46, 47, 47, 47, 47, 50, 50, 53,
 53, 53, 53, 53, 49, 47, 47, 45, 45, 46, 46, 45, 45, 49, 49, 53, 53, 56,
 56, 58,
 // Size 16x32

```

32, 31, 31, 31, 31, 33, 37, 37, 37, 42, 48, 48, 48, 48, 49, 49, 31, 31,
31, 31, 31, 34, 37, 37, 37, 42, 47, 47, 47, 48, 48, 48, 31, 31, 31, 31,
31, 34, 38, 38, 38, 42, 47, 47, 47, 47, 47, 47, 31, 31, 31, 31, 31, 34,
38, 38, 38, 42, 47, 47, 47, 47, 47, 47, 31, 31, 31, 31, 31, 34, 38, 38,
38, 42, 47, 47, 47, 47, 47, 31, 31, 32, 32, 32, 35, 39, 39, 39, 42,
46, 46, 46, 46, 46, 46, 30, 31, 32, 32, 32, 35, 40, 40, 40, 42, 46, 46,
46, 45, 45, 45, 45, 40, 31, 32, 32, 32, 35, 40, 40, 40, 42, 46, 46, 46, 45,
45, 45, 30, 31, 32, 32, 32, 35, 40, 40, 40, 42, 46, 46, 46, 45, 45, 45,
32, 33, 34, 34, 34, 37, 41, 41, 41, 44, 46, 46, 46, 46, 45, 45, 33, 34,
36, 36, 36, 39, 43, 43, 43, 45, 47, 47, 47, 46, 46, 46, 33, 34, 36, 36,
36, 39, 43, 43, 43, 45, 47, 47, 47, 46, 46, 46, 33, 34, 36, 36, 36, 39,
43, 43, 43, 45, 47, 47, 47, 46, 46, 46, 35, 36, 38, 38, 38, 41, 45, 45,
45, 46, 47, 47, 47, 46, 45, 45, 37, 38, 40, 40, 40, 43, 47, 47, 47, 47,
47, 47, 47, 46, 45, 45, 37, 38, 40, 40, 40, 43, 47, 47, 47, 47, 47, 47,
47, 46, 45, 45, 37, 38, 40, 40, 40, 43, 47, 47, 47, 47, 47, 47, 46,
45, 45, 39, 40, 41, 41, 41, 44, 47, 47, 47, 48, 49, 49, 49, 48, 47, 47,
42, 42, 43, 43, 43, 45, 47, 47, 47, 48, 50, 50, 50, 50, 49, 49, 42, 42,
43, 43, 43, 45, 47, 47, 47, 48, 50, 50, 50, 50, 49, 49, 42, 42, 43, 43,
43, 45, 47, 47, 47, 48, 50, 50, 50, 50, 49, 49, 45, 45, 44, 44, 44, 46,
47, 47, 47, 49, 51, 51, 51, 51, 51, 49, 48, 46, 46, 46, 47, 48, 48, 48,
48, 50, 53, 53, 53, 53, 53, 49, 48, 46, 46, 46, 47, 48, 48, 48, 50, 53,
53, 53, 53, 53, 53, 53, 49, 48, 46, 46, 46, 47, 47, 47, 47, 50, 53, 53,
53, 54, 54, 48, 47, 46, 46, 46, 46, 47, 47, 47, 50, 53, 53, 53, 54, 56,
48, 47, 46, 46, 46, 46, 47, 47, 47, 50, 53, 53, 53, 54, 56, 56, 48, 47,
46, 46, 46, 46, 47, 47, 47, 50, 53, 53, 53, 54, 56, 56, 48, 47, 45,
45, 46, 46, 46, 46, 49, 53, 53, 53, 55, 57, 57, 49, 47, 45, 45, 45, 45,
46, 46, 46, 49, 53, 53, 53, 56, 58, 58, 49, 47, 45, 45, 45, 45, 46, 46,
46, 49, 53, 53, 53, 56, 58, 58,
// Size 32x16
32, 31, 31, 31, 31, 30, 30, 30, 32, 33, 33, 33, 35, 37, 37, 37, 39,
42, 42, 42, 45, 49, 49, 49, 48, 48, 48, 48, 48, 49, 49, 31, 31, 31, 31,
31, 31, 31, 31, 31, 33, 34, 34, 34, 36, 38, 38, 38, 40, 42, 42, 42, 45,
48, 48, 48, 47, 47, 47, 47, 47, 47, 47, 31, 31, 31, 31, 31, 32, 32, 32,
32, 34, 36, 36, 36, 38, 40, 40, 40, 41, 43, 43, 43, 44, 46, 46, 46, 46,
46, 46, 46, 45, 45, 45, 31, 31, 31, 31, 32, 32, 32, 32, 34, 36, 36,
36, 38, 40, 40, 40, 41, 43, 43, 44, 46, 46, 46, 46, 46, 46, 46, 45,
45, 45, 31, 31, 31, 31, 31, 32, 32, 32, 32, 34, 36, 36, 36, 38, 40, 40,
40, 41, 43, 43, 43, 44, 46, 46, 46, 46, 46, 46, 46, 45, 45, 45, 33, 34,
34, 34, 34, 35, 35, 35, 35, 37, 39, 39, 39, 41, 43, 43, 43, 44, 45, 45,
45, 46, 47, 47, 47, 47, 46, 46, 46, 45, 45, 37, 37, 38, 38, 38, 39,
40, 40, 40, 41, 43, 43, 43, 45, 47, 47, 47, 47, 47, 47, 47, 48, 48,
48, 47, 47, 47, 47, 46, 46, 46, 37, 37, 38, 38, 38, 39, 40, 40, 40, 41,
43, 43, 43, 45, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 47, 47, 47,
47, 46, 46, 46, 37, 37, 38, 38, 38, 39, 40, 40, 40, 41, 43, 43, 43, 45,
47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 47, 47, 47, 46, 46, 46,
42, 42, 42, 42, 42, 42, 42, 42, 44, 45, 45, 45, 46, 47, 47, 47, 48,
48, 48, 48, 49, 50, 50, 50, 50, 50, 50, 50, 49, 49, 49, 48, 47, 47, 47,
47, 46, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 49, 50, 50, 50, 51,
53, 53, 53, 53, 53, 53, 53, 53, 53, 48, 47, 47, 47, 47, 46, 46, 46,
46, 46, 47, 47, 47, 47, 47, 47, 49, 50, 50, 50, 51, 53, 53, 53, 53,
53, 53, 53, 53, 53, 53, 48, 47, 47, 47, 46, 46, 46, 46, 46, 47, 47,
47, 47, 47, 47, 47, 49, 50, 50, 50, 51, 53, 53, 53, 53, 53, 53, 53,
53, 53, 48, 48, 47, 47, 47, 46, 45, 45, 45, 46, 46, 46, 46, 46, 46,
46, 48, 50, 50, 50, 51, 53, 53, 53, 54, 54, 54, 54, 55, 56, 56, 49, 48,
47, 47, 47, 46, 45, 45, 45, 46, 46, 46, 45, 45, 45, 45, 47, 49, 49,
49, 51, 53, 53, 53, 54, 56, 56, 56, 57, 58, 58, 49, 48, 47, 47, 47, 46,
45, 45, 45, 45, 46, 46, 46, 45, 45, 45, 45, 47, 49, 49, 49, 51, 53, 53,
53, 54, 56, 56, 56, 57, 58, 58,
// Size 4x16
31, 33, 42, 48, 31, 34, 42, 47, 31, 34, 42, 47, 31, 35, 42, 45, 31, 35,
42, 45, 34, 39, 45, 46, 34, 39, 45, 46, 38, 43, 47, 46, 38, 43, 47, 46,
42, 45, 48, 50, 42, 45, 48, 50, 48, 47, 50, 53, 48, 47, 50, 53, 47, 46,
50, 54, 47, 46, 50, 54, 47, 45, 49, 56,
// Size 16x4
31, 31, 31, 31, 31, 34, 34, 38, 38, 42, 42, 48, 48, 47, 47, 47, 33, 34,
34, 35, 35, 39, 39, 43, 43, 45, 45, 47, 47, 46, 46, 45, 42, 42, 42, 42,
42, 45, 45, 47, 47, 48, 48, 50, 50, 50, 50, 49, 48, 47, 47, 45, 45, 46,
46, 46, 46, 50, 50, 53, 53, 54, 54, 56,
// Size 8x32
32, 31, 31, 37, 37, 48, 48, 49, 31, 31, 31, 37, 37, 47, 47, 48, 31, 31,

```

```

31, 38, 38, 47, 47, 47, 31, 31, 31, 38, 38, 47, 47, 47, 31, 31, 31, 38,
38, 47, 47, 47, 31, 32, 32, 39, 39, 46, 46, 46, 30, 32, 32, 40, 40, 46,
46, 45, 30, 32, 32, 40, 40, 46, 46, 45, 30, 32, 32, 40, 40, 46, 46, 45,
32, 34, 34, 41, 41, 46, 46, 45, 33, 36, 36, 43, 43, 47, 47, 46, 33, 36,
36, 43, 43, 47, 47, 46, 33, 36, 36, 43, 43, 47, 47, 46, 35, 38, 38, 45,
45, 47, 47, 45, 37, 40, 40, 47, 47, 47, 47, 45, 37, 40, 40, 47, 47, 47,
47, 45, 37, 40, 40, 47, 47, 47, 47, 45, 39, 41, 41, 47, 47, 49, 49, 47,
42, 43, 43, 47, 47, 50, 50, 49, 42, 43, 43, 47, 47, 50, 50, 49, 42, 43,
43, 47, 47, 50, 50, 49, 45, 44, 44, 47, 47, 51, 51, 51, 49, 46, 46, 48,
48, 53, 53, 53, 49, 46, 46, 48, 48, 53, 53, 53, 49, 46, 46, 48, 48, 53,
53, 53, 48, 46, 47, 47, 53, 53, 54, 48, 46, 46, 47, 47, 53, 53, 56,
48, 46, 46, 47, 47, 53, 53, 56, 48, 46, 46, 47, 47, 53, 53, 56, 48, 45,
45, 46, 46, 53, 53, 57, 49, 45, 45, 46, 46, 53, 53, 58, 49, 45, 45, 46,
46, 53, 53, 58,
// Size 32x8
32, 31, 31, 31, 31, 31, 30, 30, 30, 32, 33, 33, 33, 35, 37, 37, 37, 39,
42, 42, 42, 45, 49, 49, 49, 48, 48, 48, 48, 48, 49, 49, 31, 31, 31, 31,
31, 32, 32, 32, 32, 34, 36, 36, 36, 38, 40, 40, 40, 41, 43, 43, 43, 44,
46, 46, 46, 46, 46, 46, 46, 45, 45, 45, 31, 31, 31, 31, 31, 32, 32, 32,
32, 34, 36, 36, 36, 38, 40, 40, 40, 41, 43, 43, 43, 44, 46, 46, 46, 46,
46, 46, 46, 45, 45, 45, 37, 37, 38, 38, 38, 39, 40, 40, 40, 41, 43, 43,
43, 45, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 47, 47, 47, 47, 46,
46, 46, 37, 37, 38, 38, 38, 39, 40, 40, 40, 41, 43, 43, 43, 45, 47, 47,
47, 47, 47, 47, 47, 47, 48, 48, 48, 47, 47, 47, 47, 46, 46, 46, 48, 47,
47, 47, 47, 46, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 47, 49, 50, 50,
50, 51, 53, 53, 53, 53, 53, 53, 53, 53, 53, 53, 48, 47, 47, 47, 47, 46,
46, 46, 46, 46, 47, 47, 47, 47, 47, 49, 50, 50, 50, 51, 53, 53,
53, 53, 53, 53, 53, 53, 53, 53, 49, 48, 47, 47, 47, 46, 45, 45, 45, 45,
46, 46, 46, 45, 45, 45, 45, 47, 49, 49, 49, 51, 53, 53, 53, 54, 56, 56,
56, 57, 58, 58,
// Size 4x32
31, 33, 42, 48, 31, 34, 42, 48, 31, 34, 42, 47, 31, 34, 42, 47, 31, 34,
42, 47, 31, 35, 42, 46, 31, 35, 42, 45, 31, 35, 42, 45, 31, 35, 42, 45,
33, 37, 44, 46, 34, 39, 45, 46, 34, 39, 45, 46, 34, 39, 45, 46, 36, 41,
46, 46, 38, 43, 47, 46, 38, 43, 47, 46, 38, 43, 47, 46, 40, 44, 48, 48,
42, 45, 48, 50, 42, 45, 48, 50, 42, 45, 48, 50, 45, 46, 49, 51, 48, 47,
50, 53, 48, 47, 50, 53, 48, 47, 50, 53, 47, 47, 50, 54, 47, 46, 50, 54,
47, 46, 50, 54, 47, 46, 50, 54, 47, 46, 49, 55, 47, 45, 49, 56, 47, 45,
49, 56,
// Size 32x4
31, 31, 31, 31, 31, 31, 31, 31, 31, 33, 34, 34, 34, 36, 38, 38, 38, 40,
42, 42, 42, 45, 48, 48, 48, 47, 47, 47, 47, 47, 47, 47, 33, 34, 34, 34,
34, 35, 35, 35, 35, 37, 39, 39, 39, 41, 43, 43, 43, 44, 45, 45, 45, 46,
47, 47, 47, 47, 46, 46, 46, 46, 45, 45, 42, 42, 42, 42, 42, 42, 42, 42,
42, 44, 45, 45, 45, 46, 47, 47, 47, 48, 48, 48, 48, 49, 50, 50, 50, 50,
50, 50, 50, 49, 49, 49, 48, 48, 47, 47, 46, 45, 45, 45, 46, 46, 46,
46, 46, 46, 46, 46, 48, 50, 50, 50, 51, 53, 53, 53, 54, 54, 54, 54, 55,
56, 56,
},
{
{ // Luma
// Size 4x4
32, 32, 32, 35, 32, 32, 33, 35, 32, 33, 35, 38, 35, 35, 38, 46,
// Size 8x8
31, 31, 31, 32, 32, 32, 34, 35, 31, 32, 32, 32, 32, 33, 34, 35, 31, 32,
32, 32, 32, 33, 33, 34, 32, 32, 32, 33, 34, 34, 35, 36, 32, 32, 32, 34,
35, 35, 36, 38, 32, 33, 33, 34, 35, 36, 38, 40, 34, 34, 33, 35, 36, 38,
39, 42, 35, 35, 34, 36, 38, 40, 42, 48,
// Size 16x16
32, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 33, 34, 34, 36, 36, 31, 31,
31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 33, 33, 34, 34, 34, 31, 32, 32, 32, 32, 33, 33, 33, 33, 34,
32, 33, 33, 34, 34, 34, 31, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34,
35, 35, 36, 36, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34,
35, 36, 36, 37, 37,
36, 36, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34, 35, 36, 36, 37, 37,
32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 36, 37, 37, 38, 38, 32, 32,
32, 32, 32, 32, 33, 34, 34, 35, 35, 36, 37, 37, 38, 38, 33, 33, 33, 33,
33, 33, 34, 35, 35, 36, 36, 38, 39, 40, 42, 42, 34, 34, 34, 34, 33, 33,

```



```

33, 33, 34, 34, 31, 32, 32, 32, 33, 33, 34, 34, 31, 32, 32, 33, 34, 34,
35, 36, 32, 32, 32, 33, 34, 34, 36, 36, 32, 32, 32, 33, 34, 34, 36, 37,
32, 32, 33, 34, 35, 35, 37, 38, 32, 32, 33, 34, 35, 35, 37, 38, 33, 33,
33, 35, 36, 36, 40, 41, 34, 34, 34, 35, 37, 37, 41, 42, 34, 34, 34, 35,
37, 37, 43, 44, 36, 35, 34, 36, 38, 38, 46, 48, 36, 35, 34, 36, 38, 38,
46, 48,
// Size 16x8
32, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 33, 34, 34, 36, 36, 31, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 31, 32, 32, 32,
32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 31, 32, 32, 32, 32, 32,
33, 33, 33, 34, 34, 35, 35, 35, 35, 36, 36, 32, 32, 32, 33, 33, 34, 34,
34, 35, 35, 36, 37, 37, 38, 38, 32, 32, 32, 32, 33, 33, 34, 34, 34, 35,
35, 36, 37, 37, 38, 38, 35, 35, 35, 34, 34, 34, 35, 36, 36, 37, 37, 40,
41, 43, 46, 46, 36, 35, 35, 35, 34, 34, 36, 36, 37, 38, 38, 41, 42, 44,
48, 48,
// Size 16x32
32, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 33, 35, 36, 36, 36, 31, 31,
31, 31, 31, 31, 32, 32, 32, 32, 32, 33, 35, 35, 35, 35, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 33, 35, 35, 35, 35, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 33, 35, 35, 35, 35, 31, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 35, 35, 35, 35, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33,
34, 35, 35, 35, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 35,
35, 35, 31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34,
31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 31, 32,
32, 32, 32, 32, 33, 33, 33, 33, 34, 35, 35, 35, 35, 31, 32, 32, 32, 32, 32,
32, 32, 33, 33, 33, 33, 33, 33, 34, 35, 36, 36, 36, 32, 32, 32, 32, 32, 33,
34, 34, 34, 35, 36, 36, 36, 36, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34,
34, 35, 36, 36, 36, 36, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 34, 35,
36, 36, 36, 36, 32, 32, 32, 32, 32, 32, 33, 34, 34, 34, 34, 35, 36, 37,
37, 37, 32, 32, 32, 33, 33, 33, 33, 33, 33, 34, 35, 35, 35, 35, 36, 37, 38,
38, 38, 32, 32, 32, 32, 33, 33, 33, 34, 35, 35, 35, 35, 36, 37, 38, 38, 38,
32, 32, 32, 33, 33, 33, 34, 35, 35, 35, 35, 36, 37, 38, 38, 38, 32, 32, 32,
33, 33, 33, 34, 35, 35, 35, 35, 36, 37, 38, 38, 38, 32, 33, 33, 33, 33, 33,
34, 35, 36, 36, 36, 37, 39, 40, 40, 40, 33, 33, 33, 33, 33, 35, 36,
36, 36, 36, 38, 40, 41, 41, 41, 41, 34, 34, 34, 34, 34, 34, 35, 36, 37, 37,
37, 39, 41, 42, 42, 42, 42, 34, 34, 34, 34, 34, 34, 35, 36, 37, 37, 37, 39,
41, 42, 42, 42, 34, 34, 34, 34, 34, 34, 35, 37, 37, 37, 37, 40, 43, 44, 44,
44, 35, 35, 34, 34, 34, 34, 36, 37, 38, 38, 38, 41, 45, 47, 47, 47, 36, 35,
35, 34, 34, 34, 36, 37, 38, 38, 38, 42, 46, 48, 48, 48, 36, 35, 35, 34, 34,
34, 36, 37, 38, 38, 38, 42, 46, 48, 48, 48, 37, 36, 36, 36, 36, 37, 38,
39, 39, 39, 42, 46, 49, 49, 49,
// Size 32x16
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 33, 34, 34, 34, 34, 35, 36, 36, 36, 37, 31, 31, 31, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33,
33, 34, 34, 34, 34, 35, 35, 35, 35, 36, 31, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 34,
34, 34, 35, 35, 35, 36, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34,
34, 36, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 34, 36, 31, 31,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33,
33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 36, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35,
35, 35, 35, 36, 36, 36, 36, 37, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 34, 34, 34, 34, 34, 35, 35, 35, 35, 36, 36, 36, 36, 37, 37,
37, 37, 37, 38, 38, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34,
34, 34, 34, 34, 35, 35, 35, 35, 36, 36, 37, 37, 37, 37, 38, 38, 38, 39,
32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 35,
35, 35, 35, 36, 36, 37, 37, 37, 37, 38, 38, 38, 38, 39, 32, 32, 32, 32,
32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 35, 35, 36,
36, 37, 37, 37, 37, 38, 38, 38, 38, 39, 33, 33, 33, 33, 33, 33,
33, 33, 33, 34, 34, 35, 35, 35, 35, 36, 36, 36, 36, 37, 38, 39, 39, 39,
40, 41, 42, 42, 42, 42, 35, 35, 35, 35, 35, 35, 34, 34, 34, 34, 34, 35,
35, 36, 36, 36, 36, 37, 37, 37, 37, 39, 40, 41, 41, 41, 43, 45, 46, 46,
46, 46, 36, 35, 35, 35, 35, 35, 35, 35, 34, 34, 34, 35, 36, 36, 36, 36,
37, 38, 38, 38, 38, 40, 41, 42, 42, 42, 44, 47, 48, 48, 48, 49, 36, 35,

```

```

35, 35, 35, 35, 35, 35, 34, 34, 34, 35, 36, 36, 36, 36, 37, 38, 38, 38,
38, 40, 41, 42, 42, 42, 44, 47, 48, 48, 48, 49, 36, 35, 35, 35, 35, 35,
35, 35, 34, 34, 34, 35, 36, 36, 36, 36, 37, 38, 38, 38, 38, 40, 41, 42,
42, 42, 44, 47, 48, 48, 48, 49,
// Size 4x16
31, 31, 32, 36, 31, 32, 32, 35, 32, 32, 32, 35, 32, 32, 32, 35, 32, 32,
33, 34, 32, 32, 33, 34, 32, 32, 34, 36, 32, 32, 34, 36, 32, 32, 34, 37,
32, 33, 35, 38, 32, 33, 35, 38, 33, 33, 36, 41, 34, 34, 37, 42, 34, 34,
37, 44, 35, 34, 38, 48, 35, 34, 38, 48,
// Size 16x4
31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 35, 35, 31, 32,
32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 34, 34, 34, 34, 32, 32, 32, 32,
33, 33, 34, 34, 34, 35, 35, 36, 37, 37, 38, 38, 36, 35, 35, 35, 34, 34,
36, 36, 37, 38, 38, 41, 42, 44, 48, 48,
// Size 8x32
32, 31, 31, 31, 32, 32, 35, 36, 31, 31, 31, 32, 32, 32, 35, 35, 31, 32,
32, 32, 32, 32, 35, 35, 31, 32, 32, 32, 32, 35, 35, 31, 32, 32, 32,
32, 32, 35, 35, 31, 32, 32, 32, 32, 32, 35, 35, 31, 32, 32, 32, 32, 32,
34, 35, 31, 32, 32, 32, 32, 32, 34, 35, 31, 32, 32, 32, 33, 33, 34, 34,
31, 32, 32, 32, 33, 33, 34, 34, 31, 32, 32, 32, 33, 33, 34, 34, 31, 32,
32, 33, 33, 33, 35, 35, 31, 32, 32, 33, 34, 34, 35, 36, 32, 32, 32, 33,
34, 34, 36, 36, 32, 32, 32, 33, 34, 34, 36, 36, 32, 32, 32, 33, 34, 34,
36, 36, 32, 32, 32, 33, 34, 34, 36, 37, 32, 32, 33, 33, 35, 35, 37, 38,
32, 32, 33, 34, 35, 35, 37, 38, 32, 32, 33, 34, 35, 35, 37, 38, 32, 32,
33, 34, 35, 35, 37, 38, 32, 33, 33, 34, 36, 36, 39, 40, 33, 33, 33, 35,
36, 36, 40, 41, 34, 34, 34, 35, 37, 37, 41, 42, 34, 34, 34, 35, 37, 37,
41, 42, 34, 34, 34, 34, 35, 37, 37, 41, 42, 34, 34, 34, 35, 37, 43, 44,
35, 34, 34, 36, 38, 38, 45, 47, 36, 35, 34, 36, 38, 38, 46, 48, 36, 35,
34, 36, 38, 38, 46, 48, 36, 35, 34, 36, 38, 38, 46, 48, 37, 36, 36, 37,
39, 39, 46, 49,
// Size 32x8
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 33, 34, 34, 34, 34, 35, 36, 36, 36, 37, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33,
33, 34, 34, 34, 34, 34, 35, 35, 35, 36, 31, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 34, 34, 34,
34, 34, 34, 34, 34, 36, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33,
33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 35, 35, 35, 36, 36, 36,
36, 37, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34,
34, 35, 35, 35, 35, 36, 36, 37, 37, 37, 37, 38, 38, 38, 38, 39, 32, 32,
32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 35,
35, 36, 36, 37, 37, 37, 37, 38, 38, 38, 38, 39, 35, 35, 35, 35, 35, 35,
34, 34, 34, 34, 34, 35, 35, 36, 36, 36, 36, 37, 37, 37, 37, 39, 40, 41,
41, 41, 43, 45, 46, 46, 46, 46, 36, 35, 35, 35, 35, 35, 35, 35, 34, 34,
34, 35, 36, 36, 36, 36, 37, 38, 38, 38, 38, 40, 41, 42, 42, 42, 44, 47,
48, 48, 48, 49,
// Size 4x32
31, 31, 32, 36, 31, 31, 32, 35, 31, 32, 32, 35, 32, 32, 32, 35, 32, 32,
32, 35, 32, 32, 32, 35, 32, 32, 32, 35, 32, 32, 35, 32, 32, 33, 34,
32, 32, 33, 34, 32, 32, 33, 34, 32, 32, 33, 35, 32, 32, 34, 36, 32, 32,
34, 36, 32, 32, 34, 36, 32, 32, 34, 36, 32, 32, 34, 37, 32, 33, 35, 38,
32, 33, 35, 38, 32, 33, 35, 38, 32, 33, 35, 38, 33, 33, 36, 40, 33, 33,
36, 41, 34, 34, 37, 42, 34, 34, 37, 42, 34, 34, 37, 42, 34, 34, 37, 44,
35, 34, 38, 47, 35, 34, 38, 48, 35, 34, 38, 48, 35, 34, 38, 48, 36, 36,
39, 49,
// Size 32x4
31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 33, 33, 34, 34, 34, 34, 35, 35, 35, 35, 36, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33,
33, 34, 34, 34, 34, 34, 34, 34, 34, 36, 32, 32, 32, 32, 32, 32, 32,
33, 33, 33, 33, 34, 34, 34, 34, 35, 35, 35, 35, 36, 36, 37, 37, 37,
37, 38, 38, 38, 38, 39, 36, 35, 35, 35, 35, 35, 35, 35, 34, 34, 35,
36, 36, 36, 36, 37, 38, 38, 38, 38, 40, 41, 42, 42, 42, 44, 47, 48, 48,
48, 49,
},
{ // Chroma
// Size 4x4
31, 32, 38, 46, 32, 34, 41, 46, 38, 41, 47, 47, 46, 46, 47, 52,
// Size 8x8
31, 31, 30, 34, 36, 39, 42, 48, 31, 31, 31, 34, 37, 40, 42, 47, 30, 31,
32, 35, 39, 41, 42, 46, 34, 34, 35, 39, 42, 44, 45, 47, 36, 37, 39, 42,

```

```

46, 47, 47, 47, 39, 40, 41, 44, 47, 47, 48, 49, 42, 42, 42, 45, 47, 48,
48, 50, 48, 47, 46, 47, 47, 49, 50, 53,
// Size 16x16
32, 31, 31, 31, 30, 30, 33, 33, 34, 36, 36, 40, 41, 44, 49, 49, 31, 31,
31, 31, 31, 31, 33, 34, 36, 38, 38, 41, 42, 44, 48, 48, 31, 31, 31, 31,
31, 31, 34, 34, 36, 38, 38, 41, 42, 44, 47, 47, 31, 31, 31, 31, 31, 31,
34, 35, 36, 39, 39, 41, 42, 44, 47, 47, 30, 31, 31, 31, 32, 32, 34, 35,
37, 40, 40, 42, 42, 44, 46, 46, 30, 31, 31, 31, 32, 32, 34, 35, 37, 40,
40, 42, 42, 44, 46, 46, 33, 33, 34, 34, 34, 34, 37, 38, 40, 42, 42, 44,
44, 45, 47, 47, 33, 34, 34, 35, 35, 35, 38, 39, 40, 43, 43, 44, 45, 46,
47, 47, 34, 36, 36, 36, 37, 37, 40, 40, 42, 45, 45, 45, 46, 46, 47, 47,
36, 38, 38, 39, 40, 40, 42, 43, 45, 47, 47, 47, 47, 47, 47, 48, 48, 36, 38,
38, 39, 40, 40, 42, 43, 45, 47, 47, 47, 47, 47, 48, 48, 40, 41, 41, 41,
42, 42, 44, 44, 45, 47, 47, 48, 48, 49, 50, 50, 41, 42, 42, 42, 42, 42,
44, 45, 46, 47, 47, 48, 48, 49, 50, 50, 44, 44, 44, 44, 44, 45, 46,
46, 47, 47, 49, 49, 50, 51, 51, 49, 48, 47, 47, 46, 46, 47, 47, 47, 48, 48, 50,
48, 50, 50, 51, 53, 53, 49, 48, 47, 47, 46, 46, 47, 47, 47, 48, 48, 50,
50, 51, 53, 53,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 30, 30, 30, 30, 31, 33, 33, 33, 33, 34, 36,
36, 36, 36, 38, 40, 41, 41, 41, 44, 47, 49, 49, 49, 49, 31, 31, 31, 31,
31, 31, 31, 31, 30, 30, 30, 32, 33, 34, 34, 34, 35, 36, 37, 37, 37, 39,
41, 42, 42, 42, 44, 47, 48, 48, 48, 48, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 32, 33, 34, 34, 34, 36, 37, 38, 38, 38, 39, 41, 42, 42, 42,
44, 46, 48, 48, 48, 47, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32,
34, 34, 34, 34, 36, 37, 38, 38, 38, 40, 41, 42, 42, 42, 44, 46, 47, 47,
47, 47, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 34, 34, 34, 34,
36, 37, 38, 38, 38, 40, 41, 42, 42, 42, 44, 46, 47, 47, 47, 47, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 34, 34, 34, 34, 36, 37, 38, 38,
38, 40, 41, 42, 42, 42, 44, 46, 47, 47, 47, 47, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 33, 34, 35, 35, 35, 36, 38, 39, 39, 39, 40, 41, 42,
42, 42, 44, 46, 47, 47, 47, 30, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 33, 34, 35, 35, 35, 37, 38, 39, 39, 39, 41, 42, 42, 42, 42, 44, 46,
46, 46, 46, 46, 30, 30, 31, 31, 31, 31, 31, 31, 32, 32, 32, 33, 34, 35,
35, 35, 37, 39, 40, 40, 40, 41, 42, 42, 42, 42, 44, 45, 46, 46, 46, 46,
30, 30, 31, 31, 31, 31, 31, 31, 32, 32, 32, 33, 34, 35, 35, 35, 37, 39,
40, 40, 40, 41, 42, 42, 42, 42, 44, 45, 46, 46, 46, 46, 30, 30, 31, 31,
31, 31, 31, 31, 32, 32, 32, 33, 34, 35, 35, 35, 37, 39, 40, 40, 40, 41,
42, 42, 42, 42, 44, 45, 46, 46, 46, 46, 31, 32, 32, 32, 32, 32, 33, 33,
33, 33, 33, 34, 36, 37, 37, 37, 38, 40, 41, 41, 41, 42, 43, 43, 43, 43,
44, 46, 46, 46, 46, 46, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 36,
37, 38, 38, 38, 40, 41, 42, 42, 42, 43, 44, 44, 44, 44, 45, 46, 47, 47,
47, 46, 33, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 37, 38, 39, 39, 39,
40, 42, 43, 43, 43, 44, 44, 45, 45, 45, 46, 47, 47, 47, 47, 33, 34,
34, 34, 34, 34, 35, 35, 35, 35, 35, 37, 38, 39, 39, 39, 40, 42, 43, 43,
43, 44, 44, 45, 45, 45, 46, 47, 47, 47, 47, 33, 34, 34, 34, 34, 34,
35, 35, 35, 35, 35, 37, 38, 39, 39, 40, 42, 43, 43, 43, 44, 44, 45,
45, 45, 46, 47, 47, 47, 47, 47, 34, 35, 36, 36, 36, 36, 36, 37, 37, 37,
37, 38, 40, 40, 40, 40, 42, 44, 45, 45, 45, 45, 45, 46, 46, 46, 46, 47,
47, 47, 47, 47, 36, 36, 37, 37, 37, 37, 38, 38, 39, 39, 39, 40, 41, 42,
42, 42, 44, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47,
36, 37, 38, 38, 38, 38, 39, 39, 40, 40, 40, 41, 42, 43, 43, 43, 45, 46,
47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 47, 36, 37, 38, 38,
38, 38, 39, 39, 40, 40, 40, 41, 42, 43, 43, 43, 45, 46, 47, 47, 47, 47,
47, 47, 47, 47, 47, 47, 48, 48, 48, 47, 36, 37, 38, 38, 38, 39, 39,
40, 40, 41, 42, 43, 43, 43, 45, 46, 47, 47, 47, 47, 47, 47, 47, 47,
47, 47, 48, 48, 48, 47, 38, 39, 39, 40, 40, 40, 40, 41, 41, 41, 41, 42,
43, 44, 44, 44, 45, 47, 47, 47, 47, 47, 48, 48, 48, 48, 48, 48, 49, 49,
49, 48, 40, 41, 41, 41, 41, 41, 41, 42, 42, 42, 42, 43, 44, 44, 44, 44,
45, 47, 47, 47, 47, 48, 48, 48, 48, 48, 49, 49, 50, 50, 50, 49, 41, 42,
42, 42, 42, 42, 42, 42, 42, 42, 42, 43, 44, 45, 45, 45, 46, 47, 47,
47, 48, 48, 48, 48, 48, 49, 50, 50, 50, 50, 41, 42, 42, 42, 42, 42, 42,
42, 42, 42, 42, 42, 43, 44, 45, 45, 45, 46, 47, 47, 47, 47, 48, 48, 48,
48, 48, 49, 50, 50, 50, 50, 50, 41, 42, 42, 42, 42, 42, 42, 42, 42,
42, 43, 44, 45, 45, 45, 46, 47, 47, 47, 47, 48, 48, 48, 48, 49, 50,
50, 50, 50, 50, 44, 44, 44, 44, 44, 44, 44, 44, 45, 46,
46, 46, 46, 47, 47, 47, 47, 48, 49, 49, 49, 49, 50, 51, 51, 51, 51,
47, 47, 46, 46, 46, 46, 46, 46, 45, 45, 45, 46, 46, 47, 47, 47, 47, 47,
47, 47, 47, 48, 49, 50, 50, 50, 51, 52, 52, 52, 52, 52, 49, 48, 48, 47,
47, 47, 47, 46, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 48, 48, 48, 49,
50, 50, 50, 50, 51, 52, 53, 53, 53, 53, 49, 48, 48, 47, 47, 47, 47, 46,

```

```

46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 48, 48, 48, 49, 50, 50, 50, 50,
51, 52, 53, 53, 53, 53, 49, 48, 48, 47, 47, 47, 47, 46, 46, 46, 46, 46,
47, 47, 47, 47, 47, 47, 48, 48, 48, 49, 50, 50, 50, 50, 51, 52, 53, 53,
53, 53, 49, 48, 47, 47, 47, 47, 47, 47, 46, 46, 46, 46, 46, 46, 47, 47, 47,
47, 47, 47, 47, 47, 48, 49, 50, 50, 50, 51, 52, 53, 53, 53, 53,
// Size 4x8
31, 31, 37, 48, 31, 31, 38, 47, 31, 32, 40, 46, 34, 36, 43, 47, 37, 39,
46, 47, 39, 41, 47, 48, 42, 43, 47, 50, 48, 46, 48, 53,
// Size 8x4
31, 31, 31, 34, 37, 39, 42, 48, 31, 31, 32, 36, 39, 41, 43, 46, 37, 38,
40, 43, 46, 47, 47, 48, 48, 47, 46, 47, 47, 48, 50, 53,
// Size 8x16
32, 31, 31, 33, 37, 37, 45, 48, 31, 31, 31, 34, 38, 38, 45, 47, 31, 31,
31, 34, 38, 38, 45, 47, 31, 31, 32, 34, 39, 39, 45, 46, 30, 32, 32, 35,
40, 40, 44, 46, 30, 32, 32, 35, 40, 40, 44, 46, 33, 34, 35, 37, 42, 42,
46, 47, 33, 35, 36, 38, 43, 43, 46, 47, 35, 37, 37, 40, 44, 44, 46, 47,
37, 39, 40, 43, 47, 47, 47, 47, 37, 39, 40, 43, 47, 47, 47, 47, 41, 42,
42, 44, 47, 47, 49, 49, 42, 42, 43, 44, 47, 47, 49, 50, 44, 44, 44, 45,
47, 47, 50, 51, 49, 47, 46, 47, 48, 48, 52, 53, 49, 47, 46, 47, 48, 48,
52, 53,
// Size 16x8
32, 31, 31, 31, 30, 30, 33, 33, 35, 37, 37, 41, 42, 44, 49, 49, 31, 31,
31, 31, 32, 32, 34, 35, 37, 39, 39, 42, 42, 44, 47, 47, 31, 31, 31, 32,
32, 32, 35, 36, 37, 40, 40, 42, 43, 44, 46, 46, 33, 34, 34, 34, 35, 35,
37, 38, 40, 43, 43, 44, 44, 45, 47, 47, 37, 38, 38, 39, 40, 40, 42, 43,
44, 47, 47, 47, 47, 47, 48, 48, 37, 38, 38, 39, 40, 40, 42, 43, 44, 47,
47, 47, 47, 47, 48, 48, 45, 45, 45, 45, 44, 44, 46, 46, 46, 47, 47, 49,
49, 50, 52, 52, 48, 47, 47, 46, 46, 46, 47, 47, 47, 47, 47, 49, 50, 51,
53, 53,
// Size 16x32
32, 31, 31, 31, 31, 31, 33, 35, 37, 37, 37, 40, 45, 48, 48, 48, 31, 31,
31, 31, 31, 31, 33, 36, 37, 37, 37, 41, 45, 48, 48, 48, 31, 31, 31, 31,
31, 31, 34, 36, 38, 38, 38, 41, 45, 47, 47, 47, 31, 31, 31, 31, 31, 31,
34, 37, 38, 38, 38, 41, 45, 47, 47, 47, 31, 31, 31, 31, 31, 31, 34, 37,
38, 38, 38, 41, 45, 47, 47, 47, 31, 31, 31, 31, 31, 31, 34, 37, 38, 38,
38, 41, 45, 47, 47, 47, 31, 31, 32, 32, 32, 34, 37, 39, 39, 39, 41,
45, 46, 46, 46, 30, 31, 31, 32, 32, 32, 34, 38, 39, 39, 39, 42, 44, 46,
46, 46, 30, 31, 32, 32, 32, 32, 35, 38, 40, 40, 40, 42, 44, 46, 46, 46,
30, 31, 32, 32, 32, 32, 35, 38, 40, 40, 40, 42, 44, 46, 46, 46, 30, 31,
32, 32, 32, 32, 35, 38, 40, 40, 40, 42, 44, 46, 46, 46, 31, 32, 33, 33,
33, 33, 36, 39, 41, 41, 41, 43, 45, 46, 46, 46, 33, 34, 34, 35, 35, 35,
37, 40, 42, 42, 42, 44, 46, 47, 47, 33, 34, 35, 36, 36, 36, 38, 41,
43, 43, 43, 44, 46, 47, 47, 47, 33, 34, 35, 36, 36, 36, 38, 41, 43, 43,
43, 44, 46, 47, 47, 47, 33, 34, 35, 36, 36, 36, 38, 41, 43, 43, 43, 44,
46, 47, 47, 47, 35, 36, 37, 37, 37, 40, 43, 44, 44, 44, 45, 46, 47,
47, 47, 36, 37, 38, 39, 39, 39, 42, 44, 46, 46, 46, 47, 47, 47, 47, 47,
37, 38, 39, 40, 40, 40, 43, 45, 47, 47, 47, 47, 47, 47, 47, 47, 37, 38,
39, 40, 40, 40, 43, 45, 47, 47, 47, 47, 47, 47, 47, 47, 47, 37, 38, 39, 40,
40, 40, 43, 45, 47, 47, 47, 47, 47, 47, 47, 47, 39, 39, 40, 41, 41, 41,
43, 46, 47, 47, 47, 48, 48, 48, 48, 48, 48, 41, 41, 42, 42, 42, 42, 44, 46,
47, 47, 47, 48, 49, 49, 49, 49, 42, 42, 42, 43, 43, 43, 44, 46, 47, 47,
47, 48, 49, 50, 50, 50, 42, 42, 42, 43, 43, 43, 44, 46, 47, 47, 47, 48,
49, 50, 50, 50, 42, 42, 42, 43, 43, 43, 44, 46, 47, 47, 47, 48, 49, 50,
50, 50, 44, 44, 44, 44, 44, 44, 45, 47, 47, 47, 47, 49, 50, 51, 51, 51,
47, 46, 46, 46, 46, 46, 46, 47, 48, 48, 48, 49, 51, 52, 52, 52, 49, 48,
47, 46, 46, 46, 47, 48, 48, 48, 48, 50, 52, 53, 53, 53, 53, 49, 48, 47, 46,
46, 46, 46, 47, 48, 48, 48, 48, 50, 52, 53, 53, 53, 49, 48, 47, 46, 46, 46,
47, 47, 47, 49, 52, 53, 53, 53,
// Size 32x16
32, 31, 31, 31, 31, 31, 30, 30, 30, 30, 31, 33, 33, 33, 33, 35, 36,
37, 37, 37, 39, 41, 42, 42, 42, 44, 47, 49, 49, 49, 49, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 32, 34, 34, 34, 34, 36, 37, 38, 38, 38, 39,
41, 42, 42, 42, 44, 46, 48, 48, 48, 48, 31, 31, 31, 31, 31, 31, 31, 31,
32, 32, 32, 33, 34, 35, 35, 35, 37, 38, 39, 39, 39, 40, 42, 42, 42, 42,
44, 46, 47, 47, 47, 47, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 33,
35, 36, 36, 36, 37, 39, 40, 40, 40, 41, 42, 43, 43, 43, 44, 46, 46, 46,
46, 46, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 33, 35, 36, 36, 36,
37, 39, 40, 40, 40, 41, 42, 43, 43, 43, 44, 46, 46, 46, 46, 46, 31, 31,
31, 31, 31, 31, 32, 32, 32, 32, 32, 33, 35, 36, 36, 36, 37, 39, 40, 40,
40, 41, 42, 43, 43, 43, 44, 46, 46, 46, 46, 46, 33, 33, 34, 34, 34, 34,

```

```

34, 34, 35, 35, 35, 36, 37, 38, 38, 38, 40, 42, 43, 43, 43, 43, 44, 44,
44, 44, 45, 46, 47, 47, 47, 47, 35, 36, 36, 37, 37, 37, 37, 38, 38, 38,
38, 39, 40, 41, 41, 41, 43, 44, 45, 45, 45, 46, 46, 46, 46, 47, 47,
48, 48, 48, 47, 37, 37, 38, 38, 38, 38, 39, 39, 40, 40, 40, 41, 42, 43,
43, 43, 44, 46, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 48, 47,
37, 37, 38, 38, 38, 38, 39, 39, 40, 40, 40, 41, 42, 43, 43, 43, 44, 46,
47, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 48, 47, 37, 37, 38, 38,
38, 38, 39, 39, 40, 40, 40, 41, 42, 43, 43, 43, 44, 46, 47, 47, 47, 47,
47, 47, 47, 47, 47, 48, 48, 48, 48, 47, 40, 41, 41, 41, 41, 41, 41, 42,
42, 42, 42, 43, 44, 44, 44, 44, 45, 47, 47, 47, 47, 48, 48, 48, 48, 48,
49, 49, 50, 50, 50, 49, 45, 45, 45, 45, 45, 45, 44, 44, 44, 44, 45,
46, 46, 46, 46, 46, 47, 47, 47, 47, 48, 49, 49, 49, 49, 50, 51, 52, 52,
52, 52, 48, 48, 47, 47, 47, 47, 46, 46, 46, 46, 46, 46, 47, 47, 47, 47,
47, 47, 47, 47, 47, 48, 49, 50, 50, 50, 51, 52, 53, 53, 53, 53, 48, 48,
47, 47, 47, 47, 46, 46, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 47,
47, 48, 49, 50, 50, 50, 51, 52, 53, 53, 53, 53, 48, 48, 47, 47, 47,
46, 46, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 47, 47, 48, 49, 50,
50, 50, 51, 52, 53, 53, 53, 53,
// Size 4x16
31, 31, 37, 48, 31, 31, 38, 47, 31, 31, 38, 47, 31, 32, 39, 46, 31, 32,
40, 46, 31, 32, 40, 46, 34, 35, 42, 47, 34, 36, 43, 47, 36, 37, 44, 47,
38, 40, 47, 47, 38, 40, 47, 47, 41, 42, 47, 49, 42, 43, 47, 50, 44, 44,
47, 51, 48, 46, 48, 53, 48, 46, 48, 53,
// Size 16x4
31, 31, 31, 31, 31, 31, 34, 34, 36, 38, 38, 41, 42, 44, 48, 48, 31, 31,
31, 32, 32, 32, 35, 36, 37, 40, 40, 42, 43, 44, 46, 46, 37, 38, 38, 39,
40, 40, 42, 43, 44, 47, 47, 47, 47, 48, 48, 48, 47, 47, 46, 46,
47, 47, 47, 47, 47, 49, 50, 51, 53, 53,
// Size 8x32
32, 31, 31, 33, 37, 37, 45, 48, 31, 31, 31, 33, 37, 37, 45, 48, 31, 31,
31, 34, 38, 38, 45, 47, 31, 31, 31, 34, 38, 38, 45, 47, 31, 31, 31, 34,
38, 38, 45, 47, 31, 31, 31, 34, 38, 38, 45, 47, 31, 31, 32, 34, 39, 39,
45, 46, 30, 31, 32, 34, 39, 39, 44, 46, 30, 32, 32, 35, 40, 40, 44, 46,
30, 32, 32, 35, 40, 40, 44, 46, 30, 32, 32, 35, 40, 40, 44, 46, 31, 33,
33, 36, 41, 41, 45, 46, 33, 34, 35, 37, 42, 42, 46, 47, 33, 35, 36, 38,
43, 43, 46, 47, 33, 35, 36, 38, 43, 43, 46, 47, 33, 35, 36, 38, 43, 43,
46, 47, 35, 37, 37, 40, 44, 44, 46, 47, 36, 38, 39, 42, 46, 46, 47, 47,
37, 39, 40, 43, 47, 47, 47, 47, 37, 39, 40, 43, 47, 47, 47, 47, 37, 39,
40, 43, 47, 47, 47, 47, 39, 40, 41, 43, 47, 47, 48, 48, 41, 42, 42, 44,
47, 47, 49, 49, 42, 42, 43, 44, 47, 47, 49, 50, 42, 42, 43, 44, 47, 47,
49, 50, 42, 42, 43, 44, 47, 47, 49, 50, 44, 44, 44, 45, 47, 47, 50, 51,
47, 46, 46, 46, 48, 48, 51, 52, 49, 47, 46, 47, 48, 48, 52, 53, 49, 47,
46, 47, 48, 48, 52, 53, 49, 47, 46, 47, 48, 48, 52, 53, 49, 47, 46, 47,
47, 47, 52, 53,
// Size 32x8
32, 31, 31, 31, 31, 31, 31, 30, 30, 30, 30, 31, 33, 33, 33, 33, 35, 36,
37, 37, 37, 39, 41, 42, 42, 42, 44, 47, 49, 49, 49, 49, 31, 31, 31, 31,
31, 31, 31, 31, 32, 32, 32, 33, 34, 35, 35, 35, 37, 38, 39, 39, 39, 40,
42, 42, 42, 42, 44, 46, 47, 47, 47, 47, 31, 31, 31, 31, 31, 31, 32, 32,
32, 32, 32, 33, 35, 36, 36, 36, 37, 39, 40, 40, 40, 41, 42, 43, 43, 43,
44, 46, 46, 46, 46, 46, 33, 33, 34, 34, 34, 34, 34, 35, 35, 35, 36,
37, 38, 38, 38, 40, 42, 43, 43, 43, 43, 44, 44, 44, 44, 45, 46, 47, 47,
47, 47, 37, 37, 38, 38, 38, 38, 39, 39, 40, 40, 40, 41, 42, 43, 43, 43,
44, 46, 47, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 48, 47, 37, 37,
38, 38, 38, 38, 39, 39, 40, 40, 40, 41, 42, 43, 43, 43, 44, 46, 47, 47,
47, 47, 47, 47, 47, 47, 48, 48, 48, 48, 47, 45, 45, 45, 45, 45, 45,
45, 44, 44, 44, 44, 45, 46, 46, 46, 46, 46, 47, 47, 47, 47, 48, 49, 49,
49, 49, 50, 51, 52, 52, 52, 52, 48, 48, 47, 47, 47, 47, 46, 46, 46, 46,
46, 46, 47, 47, 47, 47, 47, 47, 47, 47, 47, 48, 49, 50, 50, 50, 51, 52,
53, 53, 53, 53,
// Size 4x32
31, 31, 37, 48, 31, 31, 37, 48, 31, 31, 38, 47, 31, 31, 38, 47, 31, 31,
38, 47, 31, 31, 38, 47, 31, 32, 39, 46, 31, 32, 39, 46, 31, 32, 40, 46,
31, 32, 40, 46, 31, 32, 40, 46, 32, 33, 41, 46, 34, 35, 42, 47, 34, 36,
43, 47, 34, 36, 43, 47, 34, 36, 43, 47, 36, 37, 44, 47, 37, 39, 46, 47,
38, 40, 47, 47, 38, 40, 47, 47, 38, 40, 47, 47, 39, 41, 47, 48, 41, 42,
47, 49, 42, 43, 47, 50, 42, 43, 47, 50, 42, 43, 47, 50, 44, 44, 47, 51,
46, 46, 48, 52, 48, 46, 48, 53, 48, 46, 48, 53, 48, 46, 48, 53, 48, 46,
47, 53,
// Size 32x4
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 34, 34, 34, 34, 36, 37,

```

```

38, 38, 38, 39, 41, 42, 42, 42, 44, 46, 48, 48, 48, 48, 31, 31, 31, 31,
31, 31, 32, 32, 32, 32, 32, 33, 35, 36, 36, 36, 37, 39, 40, 40, 40, 41,
42, 43, 43, 43, 44, 46, 46, 46, 46, 46, 37, 37, 38, 38, 38, 38, 39, 39,
40, 40, 40, 41, 42, 43, 43, 43, 44, 46, 47, 47, 47, 47, 47, 47, 47, 47,
47, 48, 48, 48, 48, 47, 48, 48, 47, 47, 47, 47, 46, 46, 46, 46, 46, 46,
47, 47, 47, 47, 47, 47, 47, 47, 47, 48, 49, 50, 50, 50, 51, 52, 53, 53,
53, 53,
},
{
{ // Luma
// Size 4x4
31, 32, 32, 32, 32, 32, 32, 33, 32, 32, 33, 34, 32, 33, 34, 35,
// Size 8x8
31, 31, 31, 31, 32, 32, 32, 33, 31, 32, 32, 32, 32, 32, 32, 33, 31, 32,
32, 32, 32, 32, 32, 33, 31, 32, 32, 32, 32, 33, 32, 32, 32, 32, 32,
33, 33, 34, 35, 32, 32, 32, 32, 33, 34, 34, 35, 32, 32, 32, 32, 34, 34,
35, 36, 33, 33, 33, 33, 35, 35, 36, 38,
// Size 16x16
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 34, 31, 31,
31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 33, 34, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 31, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 33, 34, 31, 31, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 33, 33, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 33, 33, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33,
33, 34, 31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 34, 35,
31, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 35, 31, 32,
32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 34, 34, 34, 35, 32, 32, 32, 32,
32, 32, 32, 33, 33, 33, 33, 34, 35, 35, 35, 36, 32, 32, 32, 32, 32, 32,
32, 33, 33, 34, 34, 35, 35, 35, 36, 37, 32, 32, 32, 32, 32, 32, 32, 33,
33, 34, 34, 35, 35, 35, 36, 37, 32, 33, 33, 33, 33, 33, 33, 34, 34,
34, 35, 36, 36, 36, 38, 34, 34, 34, 34, 33, 33, 34, 35, 35, 35, 36,
37, 37, 38, 39,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 33, 33, 34, 34, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33,
34, 34, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 31, 31,
31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 31, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 33, 33, 33, 33, 31, 31, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 33, 33, 33, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
33, 33, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 31, 31,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 33, 33, 33, 33, 33, 33, 34, 34, 31, 31, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 33,
33, 33, 33, 33, 34, 34, 34, 34, 31, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 33, 33, 33, 33, 34,
34, 34, 35, 35, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 35, 35, 35,
31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33,
33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 35, 35, 35, 31, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33,
33, 34, 34, 34, 34, 34, 35, 35, 35, 31, 32, 32, 32, 32, 32, 32, 32,

```


AV2 Specification


```

34, 34, 35, 35, 35, 35, 35, 35, 36, 36, 37, 37, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 34, 35, 35,
35, 35, 35, 35, 36, 36, 37, 37, 32, 33, 33, 33, 33, 33, 33, 33, 33, 33,
33, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 35, 35, 36, 36, 36, 36, 36,
36, 37, 38, 38,
// Size 4x32
31, 31, 32, 32, 31, 31, 32, 32, 31, 32, 32, 32, 31, 32, 32, 32, 31, 32,
32, 32, 31, 32, 32, 32, 31, 32, 32, 32, 31, 32, 32, 32, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 33, 32, 32, 32, 33, 32, 32, 32, 33, 32, 32,
32, 33, 32, 32, 33, 33, 32, 32, 33, 33, 32, 32, 33, 34, 32, 32, 33, 34,
32, 32, 33, 34, 32, 32, 33, 34, 32, 32, 33, 34, 32, 32, 34, 34, 32, 32,
34, 35, 32, 33, 34, 35, 32, 33, 34, 35, 32, 33, 34, 35, 32, 33, 34, 35,
32, 33, 34, 35, 33, 33, 35, 36, 33, 33, 35, 36, 34, 34, 36, 37, 34, 34,
36, 37,
// Size 32x4
31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 34, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 33, 33, 33, 33, 33, 34, 34, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 34,
34, 34, 35, 35, 36, 36, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33,
33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 35, 36, 36,
37, 37,
},
{ // Chroma
// Size 4x4
31, 31, 34, 38, 31, 32, 35, 40, 34, 35, 39, 43, 38, 40, 43, 47,
// Size 8x8
31, 31, 31, 30, 34, 35, 37, 40, 31, 31, 31, 31, 34, 35, 38, 41, 31, 31,
31, 31, 35, 36, 39, 41, 30, 31, 31, 32, 35, 36, 40, 42, 34, 34, 35, 35,
39, 40, 43, 44, 35, 35, 36, 36, 40, 41, 44, 45, 37, 38, 39, 40, 43, 44,
47, 47, 40, 41, 41, 42, 44, 45, 47, 48,
// Size 16x16
32, 31, 31, 31, 31, 30, 30, 31, 33, 33, 33, 35, 36, 36, 38, 41, 31, 31,
31, 31, 31, 31, 31, 31, 33, 34, 34, 36, 37, 37, 39, 42, 31, 31, 31, 31,
31, 31, 31, 32, 34, 34, 34, 37, 38, 38, 40, 42, 31, 31, 31, 31, 31, 31,
31, 32, 34, 34, 34, 37, 38, 38, 40, 42, 31, 31, 31, 31, 31, 31, 32,
34, 35, 35, 37, 39, 39, 40, 42, 30, 31, 31, 31, 31, 32, 32, 32, 34, 35,
35, 38, 40, 40, 41, 42, 30, 31, 31, 31, 31, 32, 32, 32, 34, 35, 35, 38,
40, 40, 41, 42, 31, 31, 32, 32, 32, 32, 32, 33, 35, 36, 36, 38, 40, 40,
41, 43, 33, 33, 34, 34, 34, 34, 34, 35, 37, 38, 38, 41, 42, 43, 44,
33, 34, 34, 34, 35, 35, 35, 36, 38, 39, 39, 41, 43, 43, 44, 45, 33, 34,
34, 34, 35, 35, 35, 36, 38, 39, 39, 41, 43, 43, 44, 45, 35, 36, 37, 37,
37, 38, 38, 38, 41, 41, 41, 44, 46, 46, 46, 46, 36, 37, 38, 38, 39, 40,
40, 40, 42, 43, 43, 46, 47, 47, 47, 47, 36, 37, 38, 38, 39, 40, 40, 40,
42, 43, 43, 46, 47, 47, 47, 47, 38, 39, 40, 40, 40, 41, 41, 41, 43, 44,
44, 46, 47, 47, 47, 48, 41, 42, 42, 42, 42, 42, 43, 44, 45, 45, 46,
47, 47, 48, 48,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 31, 30, 30, 30, 30, 30, 31, 32, 33, 33,
33, 33, 33, 34, 35, 36, 36, 36, 36, 37, 38, 40, 41, 41, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 30, 30, 30, 30, 31, 32, 33, 34, 34, 34, 34, 35,
36, 37, 37, 37, 37, 37, 39, 40, 42, 42, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 32, 33, 34, 34, 34, 34, 35, 36, 37, 37, 37,
37, 38, 39, 40, 42, 42, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 32, 32, 34, 34, 34, 34, 34, 35, 36, 38, 38, 38, 38, 40, 41,
42, 42, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 33,
34, 34, 34, 34, 34, 35, 37, 38, 38, 38, 38, 39, 40, 41, 42, 42, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 33, 34, 34, 34,
34, 35, 37, 38, 38, 38, 38, 39, 40, 41, 42, 42, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 33, 34, 35, 37, 38,
38, 38, 38, 39, 40, 41, 42, 42, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 32, 33, 34, 34, 34, 34, 36, 37, 38, 38, 38, 38, 39,
40, 41, 42, 42, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
32, 33, 34, 35, 35, 35, 35, 36, 37, 38, 39, 39, 39, 39, 40, 41, 42, 42,
30, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 33, 34, 35,
35, 35, 35, 36, 37, 39, 39, 39, 39, 40, 40, 41, 42, 42, 30, 30, 31, 31,
31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 33, 34, 35, 35, 35, 36,
38, 39, 40, 40, 40, 40, 41, 42, 42, 42, 30, 30, 31, 31, 31, 31, 31, 31,
31, 31, 32, 32, 32, 32, 32, 33, 34, 35, 35, 35, 35, 36, 38, 39, 40, 40,
40, 40, 41, 42, 42, 42, 30, 30, 31, 31, 31, 31, 31, 31, 31, 32, 32,

```

```

32, 32, 32, 33, 34, 35, 35, 35, 35, 36, 38, 39, 40, 40, 40, 40, 41, 42,
42, 42, 30, 30, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 33,
34, 35, 35, 35, 35, 36, 38, 39, 40, 40, 40, 40, 41, 42, 42, 42, 31, 31,
31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 35, 36, 36, 36,
36, 37, 38, 40, 40, 40, 40, 41, 41, 42, 43, 43, 32, 32, 32, 32, 33, 33,
33, 33, 33, 33, 33, 33, 33, 33, 34, 35, 36, 37, 37, 37, 37, 38, 39, 41,
41, 41, 41, 42, 42, 43, 43, 43, 33, 33, 33, 34, 34, 34, 34, 34, 34,
34, 34, 34, 34, 35, 36, 37, 38, 38, 38, 38, 39, 41, 42, 42, 42, 42, 43,
43, 44, 44, 44, 33, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 35,
36, 37, 38, 39, 39, 39, 39, 40, 41, 43, 43, 43, 43, 43, 44, 44, 45, 45,
33, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 35, 36, 37, 38, 39,
39, 39, 39, 40, 41, 43, 43, 43, 43, 43, 44, 44, 45, 45, 33, 34, 34, 34,
34, 34, 34, 34, 35, 35, 35, 35, 35, 35, 36, 37, 38, 39, 39, 39, 39, 40,
41, 43, 43, 43, 43, 43, 44, 44, 45, 45, 33, 34, 34, 34, 34, 34, 34, 34,
35, 35, 35, 35, 35, 35, 36, 37, 38, 39, 39, 39, 39, 40, 41, 43, 43, 43,
43, 43, 44, 44, 45, 45, 34, 35, 35, 35, 35, 35, 36, 36, 36, 36, 36,
36, 36, 37, 38, 39, 40, 40, 40, 40, 41, 42, 44, 44, 44, 44, 45, 45,
45, 45, 35, 36, 36, 36, 37, 37, 37, 37, 37, 37, 38, 38, 38, 38, 38, 39,
41, 41, 41, 41, 41, 42, 44, 45, 46, 46, 46, 46, 46, 46, 46, 46, 36, 37,
37, 38, 38, 38, 38, 38, 38, 39, 39, 39, 39, 39, 40, 41, 42, 43, 43, 43,
43, 44, 45, 46, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 38, 38,
38, 38, 39, 39, 40, 40, 40, 40, 40, 41, 42, 43, 43, 43, 43, 44, 46, 47,
47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47,
40, 40, 40, 40, 40, 41, 42, 43, 43, 43, 43, 44, 46, 47, 47, 47, 47, 47,
47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47,
40, 41, 42, 43, 43, 43, 44, 46, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47,
37, 37, 38, 38, 39, 39, 39, 39, 39, 39, 40, 40, 40, 40, 41, 42, 43, 43,
43, 43, 43, 44, 46, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47, 47,
40, 40, 40, 40, 40, 40, 41, 41, 41, 41, 41, 42, 43, 44, 44, 44, 44, 45,
46, 47, 47, 47, 47, 47, 47, 48, 48, 48, 40, 40, 40, 41, 41, 41, 41, 41,
41, 41, 42, 42, 42, 42, 42, 43, 44, 44, 44, 44, 45, 46, 47, 47, 47,
47, 47, 48, 48, 48, 48, 41, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42,
42, 42, 43, 43, 44, 45, 45, 45, 45, 45, 46, 47, 47, 47, 47, 47, 48, 48,
48, 48, 41, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 43, 43,
44, 45, 45, 45, 45, 46, 47, 47, 47, 47, 47, 48, 48, 48, 48,
// Size 4x8
31, 31, 35, 37, 31, 31, 36, 38, 31, 32, 37, 39, 31, 32, 37, 40, 34, 36,
40, 43, 35, 37, 42, 44, 38, 40, 45, 47, 41, 42, 45, 47,
// Size 8x4
31, 31, 31, 31, 34, 35, 38, 41, 31, 31, 32, 32, 36, 37, 40, 42, 35, 36,
37, 37, 40, 42, 45, 45, 37, 38, 39, 40, 43, 44, 47, 47,
// Size 8x16
32, 31, 31, 31, 33, 37, 37, 38, 31, 31, 31, 31, 33, 38, 38, 39, 31, 31,
31, 31, 34, 38, 38, 40, 31, 31, 31, 31, 34, 38, 38, 40, 31, 31, 32, 32,
34, 39, 39, 40, 30, 31, 32, 32, 35, 40, 40, 41, 30, 31, 32, 32, 35, 40,
40, 41, 31, 32, 33, 33, 35, 40, 41, 33, 34, 35, 35, 37, 42, 42, 43,
33, 35, 36, 36, 38, 43, 43, 44, 33, 35, 36, 38, 43, 43, 44, 35, 37,
38, 38, 41, 45, 45, 46, 37, 39, 40, 40, 43, 47, 47, 47, 37, 39, 40, 40,
43, 47, 47, 47, 39, 40, 41, 41, 43, 47, 47, 47, 42, 42, 43, 43, 44, 47,
47, 48,
// Size 16x8
32, 31, 31, 31, 31, 30, 30, 31, 33, 33, 33, 35, 37, 37, 39, 42, 31, 31,
31, 31, 31, 31, 31, 32, 34, 35, 35, 37, 39, 39, 40, 42, 31, 31, 31, 31,
32, 32, 32, 33, 35, 36, 36, 38, 40, 40, 41, 43, 31, 31, 31, 31, 32, 32,
32, 33, 35, 36, 36, 38, 40, 40, 41, 43, 33, 33, 34, 34, 34, 35, 35, 35,
37, 38, 38, 41, 43, 43, 43, 44, 37, 38, 38, 39, 40, 40, 40, 42, 43,
43, 45, 47, 47, 47, 47, 47, 47, 38, 38, 38, 39, 40, 40, 40, 42, 43, 43, 45,
47, 47, 47, 47, 38, 39, 40, 40, 40, 41, 41, 41, 43, 44, 44, 46, 47, 47,
47, 48,
// Size 16x32
32, 31, 31, 31, 31, 31, 31, 31, 33, 35, 37, 37, 37, 37, 38, 42, 31, 31,
31, 31, 31, 31, 31, 31, 33, 35, 37, 37, 37, 37, 39, 42, 31, 31, 31, 31,
31, 31, 31, 32, 33, 35, 38, 38, 38, 38, 39, 42, 31, 31, 31, 31, 31, 31,
31, 32, 34, 36, 38, 38, 38, 38, 40, 42, 31, 31, 31, 31, 31, 31, 31, 32,
34, 36, 38, 38, 38, 38, 40, 42, 31, 31, 31, 31, 31, 31, 31, 31, 32, 34, 36,
38, 38, 38, 38, 40, 42, 31, 31, 31, 31, 31, 31, 31, 31, 32, 34, 36, 38, 38,
38, 38, 40, 42, 31, 31, 31, 31, 31, 31, 31, 31, 32, 34, 36, 38, 38, 38,
40, 42, 31, 31, 31, 31, 32, 32, 32, 32, 34, 36, 39, 39, 39, 39, 40, 42,
30, 31, 31, 32, 32, 32, 32, 32, 34, 37, 39, 39, 39, 39, 40, 42, 30, 31,
31, 32, 32, 32, 32, 33, 35, 37, 40, 40, 40, 40, 41, 42, 30, 31, 31, 32,
32, 32, 32, 33, 35, 37, 40, 40, 40, 40, 41, 42, 30, 31, 31, 32, 32,

```

```

32, 33, 35, 37, 40, 40, 40, 40, 41, 42, 30, 31, 31, 32, 32, 32, 32, 33,
35, 37, 40, 40, 40, 40, 41, 42, 31, 31, 32, 32, 33, 33, 33, 33, 35, 38,
40, 40, 40, 40, 41, 43, 32, 32, 33, 33, 34, 34, 34, 34, 36, 39, 41, 41,
41, 41, 42, 44, 33, 33, 34, 35, 35, 35, 35, 35, 37, 40, 42, 42, 42, 42,
43, 44, 33, 34, 35, 35, 36, 36, 36, 36, 36, 38, 40, 43, 43, 43, 43, 44, 45,
33, 34, 35, 35, 36, 36, 36, 36, 38, 40, 43, 43, 43, 43, 44, 45, 33, 34,
35, 35, 36, 36, 36, 36, 38, 40, 43, 43, 43, 43, 44, 45, 33, 34, 35, 35,
36, 36, 36, 36, 38, 40, 43, 43, 43, 43, 44, 45, 34, 35, 36, 37, 37, 37,
37, 37, 39, 42, 44, 44, 44, 44, 45, 45, 35, 36, 37, 38, 38, 38, 38, 39,
41, 43, 45, 45, 45, 45, 46, 46, 36, 37, 38, 39, 39, 39, 39, 40, 42, 44,
47, 47, 47, 47, 47, 47, 37, 38, 39, 40, 40, 40, 41, 43, 45, 47, 47,
47, 47, 47, 47, 37, 38, 39, 40, 40, 40, 41, 43, 45, 47, 47, 47, 47,
47, 47, 37, 38, 39, 40, 40, 40, 40, 41, 43, 45, 47, 47, 47, 47, 47, 47,
37, 38, 39, 40, 40, 40, 40, 41, 43, 45, 47, 47, 47, 47, 47, 39, 39,
40, 41, 41, 41, 41, 42, 43, 45, 47, 47, 47, 47, 47, 48, 40, 41, 41, 42,
42, 42, 42, 42, 44, 45, 47, 47, 47, 47, 48, 42, 42, 42, 43, 43, 43,
43, 43, 44, 46, 47, 47, 47, 48, 48, 42, 42, 42, 43, 43, 43, 43, 43,
44, 46, 47, 47, 47, 47, 48, 48,
// Size 32x16
32, 31, 31, 31, 31, 31, 31, 31, 30, 30, 30, 30, 30, 31, 32, 33, 33,
33, 33, 33, 34, 35, 36, 37, 37, 37, 39, 40, 42, 42, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 33, 34, 34, 34, 34, 35,
36, 37, 38, 38, 38, 38, 39, 41, 42, 42, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 32, 33, 34, 35, 35, 35, 35, 36, 37, 38, 39, 39,
39, 39, 40, 41, 42, 42, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32,
32, 32, 32, 33, 35, 35, 35, 35, 35, 37, 38, 39, 40, 40, 40, 40, 41, 42,
43, 43, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 33, 34,
35, 36, 36, 36, 36, 37, 38, 39, 40, 40, 40, 40, 41, 42, 43, 43, 31, 31,
31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 33, 34, 35, 36, 36, 36,
36, 37, 38, 39, 40, 40, 40, 40, 41, 42, 43, 43, 31, 31, 31, 31, 31, 31,
31, 31, 32, 32, 32, 32, 32, 32, 33, 34, 35, 36, 36, 36, 36, 37, 38, 39,
40, 40, 40, 40, 41, 42, 43, 43, 43, 43, 43, 31, 31, 32, 32, 32, 32,
33, 33, 33, 33, 33, 34, 35, 36, 36, 36, 36, 37, 39, 40, 41, 41, 41, 41,
42, 42, 43, 43, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35,
35, 36, 37, 38, 38, 38, 38, 39, 41, 42, 43, 43, 43, 43, 43, 44, 44, 44,
35, 35, 35, 36, 36, 36, 36, 36, 36, 37, 37, 37, 37, 38, 39, 40, 40,
40, 40, 40, 42, 43, 44, 45, 45, 45, 45, 45, 46, 46, 37, 37, 38, 38,
38, 38, 38, 38, 39, 39, 40, 40, 40, 40, 40, 41, 42, 43, 43, 43, 43, 44,
45, 47, 47, 47, 47, 47, 47, 47, 47, 47, 37, 37, 38, 38, 38, 38, 38, 38,
39, 39, 40, 40, 40, 40, 40, 41, 42, 43, 43, 43, 43, 44, 45, 47, 47, 47,
47, 47, 47, 47, 47, 47, 37, 38, 38, 38, 38, 38, 38, 39, 39, 40, 40,
40, 40, 40, 41, 42, 43, 43, 43, 43, 44, 45, 47, 47, 47, 47, 47, 47,
47, 47, 37, 37, 38, 38, 38, 38, 38, 38, 39, 39, 40, 40, 40, 41,
42, 43, 43, 43, 43, 44, 45, 47, 47, 47, 47, 47, 47, 47, 47, 38, 39,
39, 40, 40, 40, 40, 40, 40, 41, 41, 41, 41, 41, 42, 43, 44, 44, 44,
44, 45, 46, 47, 47, 47, 47, 47, 47, 47, 48, 48, 42, 42, 42, 42, 42,
42, 42, 42, 42, 42, 42, 42, 42, 43, 44, 44, 45, 45, 45, 45, 46, 47,
47, 47, 47, 47, 48, 48, 48, 48,
// Size 4x16
31, 31, 35, 37, 31, 31, 35, 38, 31, 31, 36, 38, 31, 31, 36, 38, 31, 32,
36, 39, 31, 32, 37, 40, 31, 32, 37, 40, 31, 33, 38, 40, 33, 35, 40, 42,
34, 36, 40, 43, 34, 36, 40, 43, 36, 38, 43, 45, 38, 40, 45, 47, 38, 40,
45, 47, 39, 41, 45, 47, 42, 43, 46, 47,
// Size 16x4
31, 31, 31, 31, 31, 31, 31, 31, 33, 34, 34, 36, 38, 38, 39, 42, 31, 31,
31, 31, 32, 32, 32, 33, 35, 36, 38, 40, 40, 41, 43, 35, 35, 36, 36,
36, 37, 37, 38, 40, 40, 40, 43, 45, 45, 45, 46, 37, 38, 38, 38, 39, 40,
40, 40, 42, 43, 43, 45, 47, 47, 47, 47,
// Size 8x32
32, 31, 31, 31, 33, 37, 37, 38, 31, 31, 31, 31, 33, 37, 37, 39, 31, 31,
31, 31, 33, 38, 38, 39, 31, 31, 31, 34, 38, 38, 40, 31, 31, 31, 31, 31,
34, 38, 38, 40, 31, 31, 31, 31, 34, 38, 38, 40, 31, 31, 32, 32, 34, 39, 39, 40,
38, 40, 31, 31, 31, 31, 34, 38, 38, 40, 31, 31, 32, 32, 34, 39, 39, 40,
30, 31, 32, 32, 34, 39, 39, 40, 30, 31, 32, 32, 35, 40, 40, 41, 30, 31,
32, 32, 35, 40, 40, 41, 30, 31, 32, 32, 35, 40, 40, 41, 30, 31, 32, 32,
35, 40, 40, 41, 31, 32, 33, 33, 35, 40, 40, 41, 32, 33, 34, 34, 36, 41,
41, 42, 33, 34, 35, 35, 37, 42, 42, 43, 33, 35, 36, 36, 38, 43, 43, 44,
33, 35, 36, 36, 38, 43, 43, 44, 33, 35, 36, 36, 38, 43, 43, 44, 33, 35,
36, 36, 38, 43, 43, 44, 34, 36, 37, 37, 39, 44, 44, 45, 35, 37, 38, 38,
41, 45, 45, 46, 36, 38, 39, 39, 42, 47, 47, 47, 37, 39, 40, 40, 43, 47,
47, 47, 37, 39, 40, 40, 43, 47, 47, 47, 37, 39, 40, 40, 43, 47, 47,

```

```

37, 39, 40, 40, 43, 47, 47, 47, 39, 40, 41, 41, 43, 47, 47, 47, 40, 41,
42, 42, 44, 47, 47, 47, 42, 42, 43, 43, 44, 47, 47, 48, 42, 42, 43, 43,
44, 47, 47, 48,
// Size 32x8
32, 31, 31, 31, 31, 31, 31, 31, 31, 30, 30, 30, 30, 30, 31, 32, 33, 33,
33, 33, 33, 34, 35, 36, 37, 37, 37, 37, 39, 40, 42, 42, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 33, 34, 35, 35, 35, 35, 36,
37, 38, 39, 39, 39, 39, 40, 41, 42, 42, 31, 31, 31, 31, 31, 31, 31,
32, 32, 32, 32, 32, 32, 33, 34, 35, 36, 36, 36, 36, 37, 38, 39, 40, 40,
40, 40, 41, 42, 43, 43, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32,
32, 32, 33, 34, 35, 36, 36, 36, 37, 38, 39, 40, 40, 40, 40, 41, 42,
43, 43, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 36,
37, 38, 38, 38, 38, 39, 41, 42, 43, 43, 43, 43, 43, 44, 44, 44, 37, 37,
38, 38, 38, 38, 38, 38, 39, 39, 40, 40, 40, 40, 40, 41, 42, 43, 43, 43,
43, 44, 45, 47, 47, 47, 47, 47, 47, 47, 47, 47, 37, 37, 38, 38, 38, 38,
38, 38, 39, 39, 40, 40, 40, 40, 40, 41, 42, 43, 43, 43, 44, 45, 47,
47, 47, 47, 47, 47, 47, 47, 47, 38, 39, 39, 40, 40, 40, 40, 40, 40,
41, 41, 41, 41, 41, 42, 43, 44, 44, 44, 44, 45, 46, 47, 47, 47, 47, 47,
47, 47, 48, 48,
// Size 4x32
31, 31, 35, 37, 31, 31, 35, 37, 31, 31, 35, 38, 31, 31, 36, 38, 31, 31,
36, 38, 31, 31, 36, 38, 31, 31, 36, 38, 31, 31, 36, 38, 31, 32, 36, 39,
31, 32, 37, 39, 31, 32, 37, 40, 31, 32, 37, 40, 31, 32, 37, 40, 31, 32,
37, 40, 31, 33, 38, 40, 32, 34, 39, 41, 33, 35, 40, 42, 34, 36, 40, 43,
34, 36, 40, 43, 34, 36, 40, 43, 34, 36, 40, 43, 35, 37, 42, 44, 36, 38,
43, 45, 37, 39, 44, 47, 38, 40, 45, 47, 38, 40, 45, 47, 38, 40, 45, 47,
38, 40, 45, 47, 39, 41, 45, 47, 41, 42, 45, 47, 42, 43, 46, 47, 42, 43,
46, 47,
// Size 32x4
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 33, 34,
34, 34, 34, 35, 36, 37, 38, 38, 38, 38, 39, 41, 42, 42, 31, 31, 31, 31,
31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 34, 35, 36, 36, 36, 36, 37,
38, 39, 40, 40, 40, 40, 41, 42, 43, 43, 35, 35, 35, 36, 36, 36, 36, 36,
36, 37, 37, 37, 37, 37, 38, 39, 40, 40, 40, 40, 40, 42, 43, 44, 45, 45,
45, 45, 45, 45, 46, 46, 37, 37, 38, 38, 38, 38, 38, 38, 39, 39, 40, 40,
40, 40, 40, 41, 42, 43, 43, 43, 43, 44, 45, 47, 47, 47, 47, 47, 47,
47, 47,
},
{
{ // Luma
// Size 4x4
31, 31, 31, 32, 31, 32, 32, 32, 31, 32, 32, 32, 32, 32, 32, 33,
// Size 8x8
31, 31, 31, 31, 31, 31, 32, 32, 31, 32, 32, 32, 32, 32, 32, 31, 32,
32, 32, 32, 32, 32, 31, 32, 32, 32, 32, 32, 32, 31, 32, 32, 32,
32, 32, 32, 32, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
33, 33, 32, 32, 32, 32, 32, 32, 33, 33,
// Size 16x16
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 31, 31, 31, 31,
31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 31, 31, 31, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 31, 31, 31, 32, 32, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 31, 31, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
32, 32, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32,
31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 31, 31,
32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 32, 31, 31, 32, 32,
32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 31, 32, 32, 32, 32,
32, 32, 32, 32, 33, 33, 33, 33, 31, 32, 32, 32, 32, 32, 32, 32,
32, 32, 33, 33, 33, 33, 33, 33, 31, 32, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 33, 33, 33, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33,
33, 33, 33, 33,
// Size 32x32
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
32, 32, 32, 32, 32, 32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,

```

Page 829 of 1169

Page 833 of 1169

```

34, 34, 34, 35, 35, 35, 35, 35, 35, 35, 36, 37, 37, 38, 38, 38, 38, 38,
38, 39, 34, 34, 34, 35, 35, 35, 35, 35, 35, 35, 35, 35, 36, 36, 36, 36,
36, 36, 36, 36, 36, 37, 37, 38, 39, 40, 40, 40, 40, 40, 40, 40, 35, 35,
36, 36, 36, 37, 37, 37, 37, 37, 37, 37, 37, 37, 38, 38, 38, 38, 38, 38,
38, 38, 39, 40, 40, 41, 41, 41, 41, 41, 41, 42, 37, 37, 37, 38, 38, 38,
38, 38, 38, 38, 38, 38, 39, 39, 39, 40, 40, 40, 40, 40, 40, 40, 41, 41,
42, 43, 43, 43, 43, 43, 43, 44,
// Size 4x16
31, 31, 31, 34, 31, 31, 31, 34, 31, 31, 31, 35, 31, 31, 31, 35, 31, 31,
31, 35, 31, 31, 31, 35, 31, 32, 32, 36, 31, 32, 32, 36, 31, 32, 32, 36,
31, 32, 32, 36, 31, 32, 32, 36, 32, 33, 33, 37, 33, 35, 35, 39, 34, 36,
36, 40, 34, 36, 36, 40, 34, 36, 36, 40,
// Size 16x4
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 33, 34, 34, 34, 31, 31,
31, 31, 31, 31, 32, 32, 32, 32, 32, 33, 35, 36, 36, 36, 31, 31, 31, 31,
31, 31, 32, 32, 32, 32, 32, 33, 35, 36, 36, 34, 34, 35, 35, 35, 35,
36, 36, 36, 36, 36, 37, 39, 40, 40, 40,
// Size 8x32
32, 31, 31, 31, 31, 31, 33, 35, 31, 31, 31, 31, 31, 31, 33, 35, 31, 31,
31, 31, 31, 31, 33, 36, 31, 31, 31, 31, 31, 33, 36, 31, 31, 31, 31,
31, 31, 34, 36, 31, 31, 31, 31, 31, 31, 34, 37, 31, 31, 31, 31, 31, 31,
34, 37, 31, 31, 31, 31, 31, 31, 34, 37, 31, 31, 31, 31, 31, 31, 34, 37,
31, 31, 31, 31, 31, 31, 34, 37, 31, 31, 31, 31, 31, 31, 34, 37, 31, 31,
31, 31, 31, 31, 34, 37, 31, 31, 31, 32, 32, 32, 34, 37, 31, 31, 31, 32,
32, 32, 34, 37, 30, 31, 31, 32, 32, 32, 34, 38, 30, 31, 32, 32, 32, 32,
35, 38, 30, 31, 32, 32, 32, 32, 35, 38, 30, 31, 32, 32, 32, 32, 35, 38,
30, 31, 32, 32, 32, 32, 35, 38, 30, 31, 32, 32, 32, 32, 35, 38, 30, 31,
32, 32, 32, 32, 35, 38, 31, 31, 32, 33, 33, 33, 35, 38, 31, 32, 33, 33,
33, 33, 36, 39, 32, 33, 34, 34, 34, 34, 37, 40, 33, 34, 34, 35, 35, 35,
37, 40, 33, 34, 35, 36, 36, 36, 38, 41, 33, 34, 35, 36, 36, 36, 38, 41,
33, 34, 35, 36, 36, 36, 38, 41, 33, 34, 35, 36, 36, 36, 38, 41, 33, 34,
35, 36, 36, 36, 38, 41, 33, 34, 35, 36, 36, 36, 38, 41, 34, 35, 36, 36,
36, 36, 39, 42,
// Size 32x8
32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 30, 30, 30, 30,
30, 30, 30, 31, 31, 32, 33, 33, 33, 33, 33, 34, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
32, 33, 34, 34, 34, 34, 34, 34, 34, 35, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 33, 34, 34, 35,
35, 35, 35, 35, 35, 36, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 35, 36, 36, 36, 36,
36, 36, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32,
32, 32, 32, 32, 32, 33, 33, 34, 35, 36, 36, 36, 36, 36, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32,
32, 33, 33, 34, 35, 36, 36, 36, 36, 36, 36, 36, 33, 33, 33, 33, 34, 34,
34, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 35, 35, 36, 37,
37, 38, 38, 38, 38, 38, 38, 39, 35, 35, 36, 36, 36, 37, 37, 37, 37,
37, 37, 37, 37, 38, 38, 38, 38, 38, 38, 38, 38, 39, 40, 40, 41, 41, 41,
41, 41, 41, 42,
// Size 4x32
31, 31, 31, 34, 31, 31, 31, 34, 31, 31, 31, 34, 31, 31, 31, 35, 31, 31,
31, 35, 31, 31, 31, 35, 31, 31, 31, 35, 31, 31, 35, 31, 31, 31, 35,
31, 31, 31, 35, 31, 31, 31, 35, 31, 31, 31, 35, 31, 32, 32, 36, 31, 32,
32, 36, 31, 32, 32, 36, 31, 32, 32, 36, 31, 32, 32, 36, 31, 32, 32, 36,
31, 32, 32, 36, 31, 32, 32, 36, 31, 32, 32, 36, 31, 33, 33, 37, 32, 33,
33, 37, 32, 34, 34, 38, 33, 35, 35, 39, 34, 36, 36, 40, 34, 36, 36, 40,
34, 36, 36, 40, 34, 36, 36, 40, 34, 36, 36, 40, 34, 36, 36, 40, 34, 36,
36, 40,
// Size 32x4
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 32, 32, 33, 34, 34, 34, 34, 34, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33,
33, 34, 35, 36, 36, 36, 36, 36, 36, 36, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 34, 35, 36,
36, 36, 36, 36, 36, 36, 34, 34, 34, 35, 35, 35, 35, 35, 35, 35, 35, 35,
36, 36, 36, 36, 36, 36, 36, 36, 37, 37, 38, 39, 40, 40, 40, 40, 40,
40, 40,

```

```

},

```

```

},
{

```

```

{ // Luma

```


Page 837 of 1169

AV2 Specification


```

31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 32, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32,
32, 32, 32, 32, 32, 32, 32, 32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32,
32, 32, 32, 32,
// Size 4x32
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 32, 31, 31, 31, 32, 31, 31, 32, 32, 31, 31, 32, 32,
31, 31, 32, 32, 31, 31, 32, 32, 30, 31, 32, 32, 30, 31, 32, 32, 30, 31,
32, 32,
// Size 32x4
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 30, 30, 30, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32,
32, 32, 32, 32, 32, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32,
32, 32,
},
},
}

```

§ 9.5. Warp filter tables

```

Warped_Filters[WARPEDPIXEL_PREC_SHIFTS * 7 + 1][ 8 ] =
{
{ 127, 1, 0, 0, 0, 0, 0, 0 }, { 126, 2, 0, 0, 0, 0, 0, 0 },
{ 124, 4, 0, 0, 0, 0, 0, 0 }, { 122, 6, 0, 0, 0, 0, 0, 0 },
{ 120, 8, 0, 0, 0, 0, 0, 0 }, { 118, 10, 0, 0, 0, 0, 0, 0 },
{ 116, 12, 0, 0, 0, 0, 0, 0 }, { 114, 14, 0, 0, 0, 0, 0, 0 },
{ 112, 16, 0, 0, 0, 0, 0, 0 }, { 110, 18, 0, 0, 0, 0, 0, 0 },
{ 108, 20, 0, 0, 0, 0, 0, 0 }, { 106, 22, 0, 0, 0, 0, 0, 0 },
{ 104, 24, 0, 0, 0, 0, 0, 0 }, { 102, 26, 0, 0, 0, 0, 0, 0 },
{ 100, 28, 0, 0, 0, 0, 0, 0 }, { 98, 30, 0, 0, 0, 0, 0, 0 },
{ 96, 32, 0, 0, 0, 0, 0, 0 }, { 94, 34, 0, 0, 0, 0, 0, 0 },
{ 92, 36, 0, 0, 0, 0, 0, 0 }, { 90, 38, 0, 0, 0, 0, 0, 0 },
{ 88, 40, 0, 0, 0, 0, 0, 0 }, { 86, 42, 0, 0, 0, 0, 0, 0 },
{ 84, 44, 0, 0, 0, 0, 0, 0 }, { 82, 46, 0, 0, 0, 0, 0, 0 },
{ 80, 48, 0, 0, 0, 0, 0, 0 }, { 78, 50, 0, 0, 0, 0, 0, 0 },
{ 76, 52, 0, 0, 0, 0, 0, 0 }, { 74, 54, 0, 0, 0, 0, 0, 0 },
{ 72, 56, 0, 0, 0, 0, 0, 0 }, { 70, 58, 0, 0, 0, 0, 0, 0 },
{ 68, 60, 0, 0, 0, 0, 0, 0 }, { 66, 62, 0, 0, 0, 0, 0, 0 },
{ 64, 64, 0, 0, 0, 0, 0, 0 }, { 62, 66, 0, 0, 0, 0, 0, 0 },
{ 60, 68, 0, 0, 0, 0, 0, 0 }, { 58, 70, 0, 0, 0, 0, 0, 0 },
{ 56, 72, 0, 0, 0, 0, 0, 0 }, { 54, 74, 0, 0, 0, 0, 0, 0 },
{ 52, 76, 0, 0, 0, 0, 0, 0 }, { 50, 78, 0, 0, 0, 0, 0, 0 },
{ 48, 80, 0, 0, 0, 0, 0, 0 }, { 46, 82, 0, 0, 0, 0, 0, 0 },
{ 44, 84, 0, 0, 0, 0, 0, 0 }, { 42, 86, 0, 0, 0, 0, 0, 0 },
{ 40, 88, 0, 0, 0, 0, 0, 0 }, { 38, 90, 0, 0, 0, 0, 0, 0 },
{ 36, 92, 0, 0, 0, 0, 0, 0 }, { 34, 94, 0, 0, 0, 0, 0, 0 },
{ 32, 96, 0, 0, 0, 0, 0, 0 }, { 30, 98, 0, 0, 0, 0, 0, 0 },
{ 28, 100, 0, 0, 0, 0, 0, 0 }, { 26, 102, 0, 0, 0, 0, 0, 0 },
{ 24, 104, 0, 0, 0, 0, 0, 0 }, { 22, 106, 0, 0, 0, 0, 0, 0 },
{ 20, 108, 0, 0, 0, 0, 0, 0 }, { 18, 110, 0, 0, 0, 0, 0, 0 },
{ 16, 112, 0, 0, 0, 0, 0, 0 }, { 14, 114, 0, 0, 0, 0, 0, 0 },
{ 12, 116, 0, 0, 0, 0, 0, 0 }, { 10, 118, 0, 0, 0, 0, 0, 0 },
{ 8, 120, 0, 0, 0, 0, 0, 0 }, { 6, 122, 0, 0, 0, 0, 0, 0 },
{ 4, 124, 0, 0, 0, 0, 0, 0 }, { 2, 126, 0, 0, 0, 0, 0, 0 },

{ 0, 127, 1, 0, 0, 0, 0, 0 }, { -1, 127, 2, 0, 0, 0, 0, 0 },
{ -2, 127, 4, -1, 0, 0, 0, 0 }, { -3, 126, 6, -1, 0, 0, 0, 0 },

```

```

{ - 3, 125, 8, -2, 0, 0, 0, 0 }, { - 4, 124, 11, - 3, 0, 0, 0, 0 },
{ - 5, 123, 13, -3, 0, 0, 0, 0 }, { - 5, 121, 15, - 3, 0, 0, 0, 0 },
{ - 6, 120, 18, -4, 0, 0, 0, 0 }, { - 7, 119, 20, - 4, 0, 0, 0, 0 },
{ - 7, 118, 22, -5, 0, 0, 0, 0 }, { - 8, 116, 25, - 5, 0, 0, 0, 0 },
{ - 8, 115, 27, -6, 0, 0, 0, 0 }, { - 9, 113, 30, - 6, 0, 0, 0, 0 },
{ - 9, 112, 32, -7, 0, 0, 0, 0 }, { - 9, 110, 34, - 7, 0, 0, 0, 0 },
{ -10, 108, 37, -7, 0, 0, 0, 0 }, { -10, 107, 39, - 8, 0, 0, 0, 0 },
{ -10, 105, 41, -8, 0, 0, 0, 0 }, { -11, 103, 44, - 8, 0, 0, 0, 0 },
{ -11, 101, 47, -9, 0, 0, 0, 0 }, { -11, 99, 49, - 9, 0, 0, 0, 0 },
{ -11, 97, 51, -9, 0, 0, 0, 0 }, { -11, 95, 54, -10, 0, 0, 0, 0 },
{ -11, 93, 56, -10, 0, 0, 0, 0 }, { -12, 91, 59, -10, 0, 0, 0, 0 },
{ -12, 89, 61, -10, 0, 0, 0, 0 }, { -12, 87, 64, -11, 0, 0, 0, 0 },
{ -12, 85, 66, -11, 0, 0, 0, 0 }, { -12, 82, 69, -11, 0, 0, 0, 0 },
{ -12, 80, 71, -11, 0, 0, 0, 0 }, { -12, 78, 73, -11, 0, 0, 0, 0 },
{ -11, 75, 75, -11, 0, 0, 0, 0 }, { -11, 73, 78, -12, 0, 0, 0, 0 },
{ -11, 71, 80, -12, 0, 0, 0, 0 }, { -11, 69, 82, -12, 0, 0, 0, 0 },
{ -11, 66, 85, -12, 0, 0, 0, 0 }, { -11, 64, 87, -12, 0, 0, 0, 0 },
{ -10, 61, 89, -12, 0, 0, 0, 0 }, { -10, 59, 91, -12, 0, 0, 0, 0 },
{ -10, 56, 93, -11, 0, 0, 0, 0 }, { -10, 54, 95, -11, 0, 0, 0, 0 },
{ - 9, 51, 97, -11, 0, 0, 0, 0 }, { - 9, 49, 99, -11, 0, 0, 0, 0 },
{ - 9, 47, 101, -11, 0, 0, 0, 0 }, { - 8, 44, 103, -11, 0, 0, 0, 0 },
{ - 8, 41, 105, -10, 0, 0, 0, 0 }, { - 8, 39, 107, -10, 0, 0, 0, 0 },
{ - 7, 37, 108, -10, 0, 0, 0, 0 }, { - 7, 34, 110, - 9, 0, 0, 0, 0 },
{ - 7, 32, 112, - 9, 0, 0, 0, 0 }, { - 6, 30, 113, - 9, 0, 0, 0, 0 },
{ - 6, 27, 115, - 8, 0, 0, 0, 0 }, { - 5, 25, 116, - 8, 0, 0, 0, 0 },
{ - 5, 22, 118, - 7, 0, 0, 0, 0 }, { - 4, 20, 119, - 7, 0, 0, 0, 0 },
{ - 4, 18, 120, - 6, 0, 0, 0, 0 }, { - 3, 15, 121, - 5, 0, 0, 0, 0 },
{ - 3, 13, 123, - 5, 0, 0, 0, 0 }, { - 3, 11, 124, - 4, 0, 0, 0, 0 },
{ - 2, 8, 125, - 3, 0, 0, 0, 0 }, { - 1, 6, 126, - 3, 0, 0, 0, 0 },
{ - 1, 4, 127, - 2, 0, 0, 0, 0 }, { 0, 2, 127, - 1, 0, 0, 0, 0 },

```

```

{ 0, 0, 127, 1, 0, 0, 0, 0 }, { 0, - 1, 127, 2, 0, 0, 0, 0 },
{ 1, - 3, 127, 4, - 1, 0, 0, 0 }, { 1, - 4, 126, 6, - 2, 1, 0, 0 },
{ 1, - 5, 126, 8, - 3, 1, 0, 0 }, { 1, - 6, 125, 11, - 4, 1, 0, 0 },
{ 1, - 7, 124, 13, - 4, 1, 0, 0 }, { 2, - 8, 123, 15, - 5, 1, 0, 0 },
{ 2, - 9, 122, 18, - 6, 1, 0, 0 }, { 2, -10, 121, 20, - 6, 1, 0, 0 },
{ 2, -11, 120, 22, - 7, 2, 0, 0 }, { 2, -12, 119, 25, - 8, 2, 0, 0 },
{ 3, -13, 117, 27, - 8, 2, 0, 0 }, { 3, -13, 116, 29, - 9, 2, 0, 0 },
{ 3, -14, 114, 32, -10, 3, 0, 0 }, { 3, -15, 113, 35, -10, 2, 0, 0 },
{ 3, -15, 111, 37, -11, 3, 0, 0 }, { 3, -16, 109, 40, -11, 3, 0, 0 },
{ 3, -16, 108, 42, -12, 3, 0, 0 }, { 4, -17, 106, 45, -13, 3, 0, 0 },
{ 4, -17, 104, 47, -13, 3, 0, 0 }, { 4, -17, 102, 50, -14, 3, 0, 0 },
{ 4, -17, 100, 52, -14, 3, 0, 0 }, { 4, -18, 98, 55, -15, 4, 0, 0 },
{ 4, -18, 96, 58, -15, 3, 0, 0 }, { 4, -18, 94, 60, -16, 4, 0, 0 },
{ 4, -18, 91, 63, -16, 4, 0, 0 }, { 4, -18, 89, 65, -16, 4, 0, 0 },
{ 4, -18, 87, 68, -17, 4, 0, 0 }, { 4, -18, 85, 70, -17, 4, 0, 0 },
{ 4, -18, 82, 73, -17, 4, 0, 0 }, { 4, -18, 80, 75, -17, 4, 0, 0 },
{ 4, -18, 78, 78, -18, 4, 0, 0 }, { 4, -17, 75, 80, -18, 4, 0, 0 },
{ 4, -17, 73, 82, -18, 4, 0, 0 }, { 4, -17, 70, 85, -18, 4, 0, 0 },
{ 4, -17, 68, 87, -18, 4, 0, 0 }, { 4, -16, 65, 89, -18, 4, 0, 0 },
{ 4, -16, 63, 91, -18, 4, 0, 0 }, { 4, -16, 60, 94, -18, 4, 0, 0 },
{ 3, -15, 58, 96, -18, 4, 0, 0 }, { 4, -15, 55, 98, -18, 4, 0, 0 },
{ 3, -14, 52, 100, -17, 4, 0, 0 }, { 3, -14, 50, 102, -17, 4, 0, 0 },
{ 3, -13, 47, 104, -17, 4, 0, 0 }, { 3, -13, 45, 106, -17, 4, 0, 0 },
{ 3, -12, 42, 108, -16, 3, 0, 0 }, { 3, -11, 40, 109, -16, 3, 0, 0 },
{ 3, -11, 37, 111, -15, 3, 0, 0 }, { 2, -10, 35, 113, -15, 3, 0, 0 },
{ 3, -10, 32, 114, -14, 3, 0, 0 }, { 2, - 9, 29, 116, -13, 3, 0, 0 },
{ 2, - 8, 27, 117, -13, 3, 0, 0 }, { 2, - 8, 25, 119, -12, 2, 0, 0 },
{ 2, - 7, 22, 120, -11, 2, 0, 0 }, { 1, - 6, 20, 121, -10, 2, 0, 0 },
{ 1, - 6, 18, 122, - 9, 2, 0, 0 }, { 1, - 5, 15, 123, - 8, 2, 0, 0 },
{ 1, - 4, 13, 124, - 7, 1, 0, 0 }, { 1, - 4, 11, 125, - 6, 1, 0, 0 },
{ 1, - 3, 8, 126, - 5, 1, 0, 0 }, { 1, - 2, 6, 126, - 4, 1, 0, 0 },
{ 0, - 1, 4, 127, - 3, 1, 0, 0 }, { 0, 0, 2, 127, - 1, 0, 0, 0 },

```

```

{ 0, 0, 0, 127, 1, 0, 0, 0 }, { 0, 0, -1, 127, 2, 0, 0, 0 },
{ 0, 1, -3, 127, 4, -2, 1, 0 }, { 0, 1, -5, 127, 6, -2, 1, 0 },
{ 0, 2, -6, 126, 8, -3, 1, 0 }, { -1, 2, -7, 126, 11, -4, 2, -1 },
{ -1, 3, -8, 125, 13, -5, 2, -1 }, { -1, 3, -10, 124, 16, -6, 3, -1 },
{ -1, 4, -11, 123, 18, -7, 3, -1 }, { -1, 4, -12, 122, 20, -7, 3, -1 },
{ -1, 4, -13, 121, 23, -8, 3, -1 }, { -2, 5, -14, 120, 25, -9, 4, -1 },
{ -1, 5, -15, 119, 27, -10, 4, -1 }, { -1, 5, -16, 118, 30, -11, 4, -1 },

```

```

{-2, 6, -17, 116, 33, -12, 5, -1}, {-2, 6, -17, 114, 35, -12, 5, -1},
{-2, 6, -18, 113, 38, -13, 5, -1}, {-2, 7, -19, 111, 41, -14, 6, -2},
{-2, 7, -19, 110, 43, -15, 6, -2}, {-2, 7, -20, 108, 46, -15, 6, -2},
{-2, 7, -20, 106, 49, -16, 6, -2}, {-2, 7, -21, 104, 51, -16, 7, -2},
{-2, 7, -21, 102, 54, -17, 7, -2}, {-2, 8, -21, 100, 56, -18, 7, -2},
{-2, 8, -22, 98, 59, -18, 7, -2}, {-2, 8, -22, 96, 62, -19, 7, -2},
{-2, 8, -22, 94, 64, -19, 7, -2}, {-2, 8, -22, 91, 67, -20, 8, -2},
{-2, 8, -22, 89, 69, -20, 8, -2}, {-2, 8, -22, 87, 72, -21, 8, -2},
{-2, 8, -21, 84, 74, -21, 8, -2}, {-2, 8, -22, 82, 77, -21, 8, -2},
{-2, 8, -21, 79, 79, -21, 8, -2}, {-2, 8, -21, 77, 82, -22, 8, -2},
{-2, 8, -21, 74, 84, -21, 8, -2}, {-2, 8, -21, 72, 87, -22, 8, -2},
{-2, 8, -20, 69, 89, -22, 8, -2}, {-2, 8, -20, 67, 91, -22, 8, -2},
{-2, 7, -19, 64, 94, -22, 8, -2}, {-2, 7, -19, 62, 96, -22, 8, -2},
{-2, 7, -18, 59, 98, -22, 8, -2}, {-2, 7, -18, 56, 100, -21, 8, -2},
{-2, 7, -17, 54, 102, -21, 7, -2}, {-2, 7, -16, 51, 104, -21, 7, -2},
{-2, 6, -16, 49, 106, -20, 7, -2}, {-2, 6, -15, 46, 108, -20, 7, -2},
{-2, 6, -15, 43, 110, -19, 7, -2}, {-2, 6, -14, 41, 111, -19, 7, -2},
{-1, 5, -13, 38, 113, -18, 6, -2}, {-1, 5, -12, 35, 114, -17, 6, -2},
{-1, 5, -12, 33, 116, -17, 6, -2}, {-1, 4, -11, 30, 118, -16, 5, -1},
{-1, 4, -10, 27, 119, -15, 5, -1}, {-1, 4, -9, 25, 120, -14, 5, -2},
{-1, 3, -8, 23, 121, -13, 4, -1}, {-1, 3, -7, 20, 122, -12, 4, -1},
{-1, 3, -7, 18, 123, -11, 4, -1}, {-1, 3, -6, 16, 124, -10, 3, -1},
{-1, 2, -5, 13, 125, -8, 3, -1}, {-1, 2, -4, 11, 126, -7, 2, -1},
{ 0, 1, -3, 8, 126, -6, 2, 0}, { 0, 1, -2, 6, 127, -5, 1, 0},
{ 0, 1, -2, 4, 127, -3, 1, 0}, { 0, 0, 0, 2, 127, -1, 0, 0},

```

```

{ 0, 0, 0, 1, 127, 0, 0, 0 }, { 0, 0, 0, -1, 127, 2, 0, 0 },
{ 0, 0, 1, -3, 127, 4, -1, 0 }, { 0, 0, 1, -4, 126, 6, -2, 1 },
{ 0, 0, 1, -5, 126, 8, -3, 1 }, { 0, 0, 1, -6, 125, 11, -4, 1 },
{ 0, 0, 1, -7, 124, 13, -4, 1 }, { 0, 0, 2, -8, 123, 15, -5, 1 },
{ 0, 0, 2, -9, 122, 18, -6, 1 }, { 0, 0, 2, -10, 121, 20, -6, 1 },
{ 0, 0, 2, -11, 120, 22, -7, 2 }, { 0, 0, 2, -12, 119, 25, -8, 2 },
{ 0, 0, 3, -13, 117, 27, -8, 2 }, { 0, 0, 3, -13, 116, 29, -9, 2 },
{ 0, 0, 3, -14, 114, 32, -10, 3 }, { 0, 0, 3, -15, 113, 35, -10, 2 },
{ 0, 0, 3, -15, 111, 37, -11, 3 }, { 0, 0, 3, -16, 109, 40, -11, 3 },
{ 0, 0, 3, -16, 108, 42, -12, 3 }, { 0, 0, 4, -17, 106, 45, -13, 3 },
{ 0, 0, 4, -17, 104, 47, -13, 3 }, { 0, 0, 4, -17, 102, 50, -14, 3 },
{ 0, 0, 4, -17, 100, 52, -14, 3 }, { 0, 0, 4, -18, 98, 55, -15, 4 },
{ 0, 0, 4, -18, 96, 58, -15, 3 }, { 0, 0, 4, -18, 94, 60, -16, 4 },
{ 0, 0, 4, -18, 91, 63, -16, 4 }, { 0, 0, 4, -18, 89, 65, -16, 4 },
{ 0, 0, 4, -18, 87, 68, -17, 4 }, { 0, 0, 4, -18, 85, 70, -17, 4 },
{ 0, 0, 4, -18, 82, 73, -17, 4 }, { 0, 0, 4, -18, 80, 75, -17, 4 },
{ 0, 0, 4, -18, 78, 78, -18, 4 }, { 0, 0, 4, -17, 75, 80, -18, 4 },
{ 0, 0, 4, -17, 73, 82, -18, 4 }, { 0, 0, 4, -17, 70, 85, -18, 4 },
{ 0, 0, 4, -17, 68, 87, -18, 4 }, { 0, 0, 4, -16, 65, 89, -18, 4 },
{ 0, 0, 4, -16, 63, 91, -18, 4 }, { 0, 0, 4, -16, 60, 94, -18, 4 },
{ 0, 0, 3, -15, 58, 96, -18, 4 }, { 0, 0, 4, -15, 55, 98, -18, 4 },
{ 0, 0, 3, -14, 52, 100, -17, 4 }, { 0, 0, 3, -14, 50, 102, -17, 4 },
{ 0, 0, 3, -13, 47, 104, -17, 4 }, { 0, 0, 3, -13, 45, 106, -17, 4 },
{ 0, 0, 3, -12, 42, 108, -16, 3 }, { 0, 0, 3, -11, 40, 109, -16, 3 },
{ 0, 0, 3, -11, 37, 111, -15, 3 }, { 0, 0, 2, -10, 35, 113, -15, 3 },
{ 0, 0, 3, -10, 32, 114, -14, 3 }, { 0, 0, 2, -9, 29, 116, -13, 3 },
{ 0, 0, 2, -8, 27, 117, -13, 3 }, { 0, 0, 2, -8, 25, 119, -12, 2 },
{ 0, 0, 2, -7, 22, 120, -11, 2 }, { 0, 0, 1, -6, 20, 121, -10, 2 },
{ 0, 0, 1, -6, 18, 122, -9, 2 }, { 0, 0, 1, -5, 15, 123, -8, 2 },
{ 0, 0, 1, -4, 13, 124, -7, 1 }, { 0, 0, 1, -4, 11, 125, -6, 1 },
{ 0, 0, 1, -3, 8, 126, -5, 1 }, { 0, 0, 1, -2, 6, 126, -4, 1 },
{ 0, 0, 0, -1, 4, 127, -3, 1 }, { 0, 0, 0, 0, 2, 127, -1, 0 },

```

```

{ 0, 0, 0, 0, 0, 127, 1, 0 }, { 0, 0, 0, 0, -1, 127, 2, 0 },
{ 0, 0, 0, 0, -2, 127, 4, -1 }, { 0, 0, 0, 0, -3, 126, 6, -1 },
{ 0, 0, 0, 0, -3, 125, 8, -2 }, { 0, 0, 0, 0, -4, 124, 11, -3 },
{ 0, 0, 0, 0, -5, 123, 13, -3 }, { 0, 0, 0, 0, -5, 121, 15, -3 },
{ 0, 0, 0, 0, -6, 120, 18, -4 }, { 0, 0, 0, 0, -7, 119, 20, -4 },
{ 0, 0, 0, 0, -7, 118, 22, -5 }, { 0, 0, 0, 0, -8, 116, 25, -5 },
{ 0, 0, 0, 0, -8, 115, 27, -6 }, { 0, 0, 0, 0, -9, 113, 30, -6 },
{ 0, 0, 0, 0, -9, 112, 32, -7 }, { 0, 0, 0, 0, -9, 110, 34, -7 },
{ 0, 0, 0, 0, -10, 108, 37, -7 }, { 0, 0, 0, 0, -10, 107, 39, -8 },
{ 0, 0, 0, 0, -10, 105, 41, -8 }, { 0, 0, 0, 0, -11, 103, 44, -8 },
{ 0, 0, 0, 0, -11, 101, 47, -9 }, { 0, 0, 0, 0, -11, 99, 49, -9 },
{ 0, 0, 0, 0, -11, 97, 51, -9 }, { 0, 0, 0, 0, -11, 95, 54, -10 },

```

```

{ 0, 0, 0, 0, -11, 93, 56, -10 }, { 0, 0, 0, 0, -12, 91, 59, -10 },
{ 0, 0, 0, 0, -12, 89, 61, -10 }, { 0, 0, 0, 0, -12, 87, 64, -11 },
{ 0, 0, 0, 0, -12, 85, 66, -11 }, { 0, 0, 0, 0, -12, 82, 69, -11 },
{ 0, 0, 0, 0, -12, 80, 71, -11 }, { 0, 0, 0, 0, -12, 78, 73, -11 },
{ 0, 0, 0, 0, -11, 75, 75, -11 }, { 0, 0, 0, 0, -11, 73, 78, -12 },
{ 0, 0, 0, 0, -11, 71, 80, -12 }, { 0, 0, 0, 0, -11, 69, 82, -12 },
{ 0, 0, 0, 0, -11, 66, 85, -12 }, { 0, 0, 0, 0, -11, 64, 87, -12 },
{ 0, 0, 0, 0, -10, 61, 89, -12 }, { 0, 0, 0, 0, -10, 59, 91, -12 },
{ 0, 0, 0, 0, -10, 56, 93, -11 }, { 0, 0, 0, 0, -10, 54, 95, -11 },
{ 0, 0, 0, 0, -9, 51, 97, -11 }, { 0, 0, 0, 0, -9, 49, 99, -11 },
{ 0, 0, 0, 0, -9, 47, 101, -11 }, { 0, 0, 0, 0, -8, 44, 103, -11 },
{ 0, 0, 0, 0, -8, 41, 105, -10 }, { 0, 0, 0, 0, -8, 39, 107, -10 },
{ 0, 0, 0, 0, -7, 37, 108, -10 }, { 0, 0, 0, 0, -7, 34, 110, -9 },
{ 0, 0, 0, 0, -7, 32, 112, -9 }, { 0, 0, 0, 0, -6, 30, 113, -9 },
{ 0, 0, 0, 0, -6, 27, 115, -8 }, { 0, 0, 0, 0, -5, 25, 116, -8 },
{ 0, 0, 0, 0, -5, 22, 118, -7 }, { 0, 0, 0, 0, -4, 20, 119, -7 },
{ 0, 0, 0, 0, -4, 18, 120, -6 }, { 0, 0, 0, 0, -3, 15, 121, -5 },
{ 0, 0, 0, 0, -3, 13, 123, -5 }, { 0, 0, 0, 0, -3, 11, 124, -4 },
{ 0, 0, 0, 0, -2, 8, 125, -3 }, { 0, 0, 0, 0, -1, 6, 126, -3 },
{ 0, 0, 0, 0, -1, 4, 127, -2 }, { 0, 0, 0, 0, 0, 2, 127, -1 },

```

```

{0, 0, 0, 0, 0, 0, 127, 1 }, {0, 0, 0, 0, 0, 0, 126, 2 },
{0, 0, 0, 0, 0, 0, 124, 4 }, {0, 0, 0, 0, 0, 0, 122, 6 },
{0, 0, 0, 0, 0, 0, 120, 8 }, {0, 0, 0, 0, 0, 0, 118, 10 },
{0, 0, 0, 0, 0, 0, 116, 12 }, {0, 0, 0, 0, 0, 0, 114, 14 },
{0, 0, 0, 0, 0, 0, 112, 16 }, {0, 0, 0, 0, 0, 0, 110, 18 },
{0, 0, 0, 0, 0, 0, 108, 20 }, {0, 0, 0, 0, 0, 0, 106, 22 },
{0, 0, 0, 0, 0, 0, 104, 24 }, {0, 0, 0, 0, 0, 0, 102, 26 },
{0, 0, 0, 0, 0, 0, 100, 28 }, {0, 0, 0, 0, 0, 0, 98, 30 },
{0, 0, 0, 0, 0, 0, 96, 32 }, {0, 0, 0, 0, 0, 0, 94, 34 },
{0, 0, 0, 0, 0, 0, 92, 36 }, {0, 0, 0, 0, 0, 0, 90, 38 },
{0, 0, 0, 0, 0, 0, 88, 40 }, {0, 0, 0, 0, 0, 0, 86, 42 },
{0, 0, 0, 0, 0, 0, 84, 44 }, {0, 0, 0, 0, 0, 0, 82, 46 },
{0, 0, 0, 0, 0, 0, 80, 48 }, {0, 0, 0, 0, 0, 0, 78, 50 },
{0, 0, 0, 0, 0, 0, 76, 52 }, {0, 0, 0, 0, 0, 0, 74, 54 },
{0, 0, 0, 0, 0, 0, 72, 56 }, {0, 0, 0, 0, 0, 0, 70, 58 },
{0, 0, 0, 0, 0, 0, 68, 60 }, {0, 0, 0, 0, 0, 0, 66, 62 },
{0, 0, 0, 0, 0, 0, 64, 64 }, {0, 0, 0, 0, 0, 0, 62, 66 },
{0, 0, 0, 0, 0, 0, 60, 68 }, {0, 0, 0, 0, 0, 0, 58, 70 },
{0, 0, 0, 0, 0, 0, 56, 72 }, {0, 0, 0, 0, 0, 0, 54, 74 },
{0, 0, 0, 0, 0, 0, 52, 76 }, {0, 0, 0, 0, 0, 0, 50, 78 },
{0, 0, 0, 0, 0, 0, 48, 80 }, {0, 0, 0, 0, 0, 0, 46, 82 },
{0, 0, 0, 0, 0, 0, 44, 84 }, {0, 0, 0, 0, 0, 0, 42, 86 },
{0, 0, 0, 0, 0, 0, 40, 88 }, {0, 0, 0, 0, 0, 0, 38, 90 },
{0, 0, 0, 0, 0, 0, 36, 92 }, {0, 0, 0, 0, 0, 0, 34, 94 },
{0, 0, 0, 0, 0, 0, 32, 96 }, {0, 0, 0, 0, 0, 0, 30, 98 },
{0, 0, 0, 0, 0, 0, 28, 100 }, {0, 0, 0, 0, 0, 0, 26, 102 },
{0, 0, 0, 0, 0, 0, 24, 104 }, {0, 0, 0, 0, 0, 0, 22, 106 },
{0, 0, 0, 0, 0, 0, 20, 108 }, {0, 0, 0, 0, 0, 0, 18, 110 },
{0, 0, 0, 0, 0, 0, 16, 112 }, {0, 0, 0, 0, 0, 0, 14, 114 },
{0, 0, 0, 0, 0, 0, 12, 116 }, {0, 0, 0, 0, 0, 0, 10, 118 },
{0, 0, 0, 0, 0, 0, 8, 120 }, {0, 0, 0, 0, 0, 0, 6, 122 },
{0, 0, 0, 0, 0, 0, 4, 124 }, {0, 0, 0, 0, 0, 0, 2, 126 },

```

```

{ 0, 0, 0, 0, 0, 0, 2, 126 }

```

```

Ext_Warped_Filters[EXT_WARP_PHASES + 1][EXT_WARP_TAPS] =

```

```

{
{ 0, 0, 128, 0, 0, 0 },
{ 0, -1, 127, 2, 0, 0 },
{ 0, -2, 127, 4, -1, 0 },
{ 0, -3, 126, 6, -1, 0 },
{ 1, -4, 125, 8, -2, 0 },
{ 1, -5, 124, 11, -3, 0 },
{ 1, -6, 123, 13, -3, 0 },
{ 1, -6, 122, 15, -4, 0 },
{ 1, -7, 120, 17, -4, 1 },
{ 1, -8, 119, 20, -5, 1 },
{ 1, -9, 118, 22, -5, 1 },

```

```

{ 1, -9, 117, 24, -6, 1 },
{ 1, -10, 115, 27, -6, 1 },
{ 1, -10, 114, 29, -7, 1 },
{ 1, -11, 112, 32, -7, 1 },
{ 1, -11, 111, 34, -8, 1 },
{ 1, -11, 109, 36, -8, 1 },
{ 2, -12, 107, 39, -9, 1 },
{ 2, -12, 105, 41, -9, 1 },
{ 2, -12, 103, 44, -10, 1 },
{ 2, -13, 102, 46, -10, 1 },
{ 2, -13, 99, 49, -10, 1 },
{ 2, -13, 98, 51, -11, 1 },
{ 2, -13, 95, 54, -11, 1 },
{ 2, -14, 93, 56, -11, 2 },
{ 2, -14, 91, 59, -12, 2 },
{ 2, -14, 89, 61, -12, 2 },
{ 2, -14, 87, 63, -12, 2 },
{ 2, -14, 85, 66, -13, 2 },
{ 2, -14, 83, 68, -13, 2 },
{ 2, -14, 80, 71, -13, 2 },
{ 2, -14, 78, 73, -13, 2 },
{ 2, -13, 75, 75, -13, 2 },
{ 2, -13, 73, 78, -14, 2 },
{ 2, -13, 71, 80, -14, 2 },
{ 2, -13, 68, 83, -14, 2 },
{ 2, -13, 66, 85, -14, 2 },
{ 2, -12, 63, 87, -14, 2 },
{ 2, -12, 61, 89, -14, 2 },
{ 2, -12, 59, 91, -14, 2 },
{ 2, -11, 56, 93, -14, 2 },
{ 1, -11, 54, 95, -13, 2 },
{ 1, -11, 51, 98, -13, 2 },
{ 1, -10, 49, 99, -13, 2 },
{ 1, -10, 46, 102, -13, 2 },
{ 1, -10, 44, 103, -12, 2 },
{ 1, -9, 41, 105, -12, 2 },
{ 1, -9, 39, 107, -12, 2 },
{ 1, -8, 36, 109, -11, 1 },
{ 1, -8, 34, 111, -11, 1 },
{ 1, -7, 32, 112, -11, 1 },
{ 1, -7, 29, 114, -10, 1 },
{ 1, -6, 27, 115, -10, 1 },
{ 1, -6, 24, 117, -9, 1 },
{ 1, -5, 22, 118, -9, 1 },
{ 1, -5, 20, 119, -8, 1 },
{ 1, -4, 17, 120, -7, 1 },
{ 0, -4, 15, 122, -6, 1 },
{ 0, -3, 13, 123, -6, 1 },
{ 0, -3, 11, 124, -5, 1 },
{ 0, -2, 8, 125, -4, 1 },
{ 0, -1, 6, 126, -3, 0 },
{ 0, -1, 4, 127, -2, 0 },
{ 0, 0, 2, 127, -1, 0 },
{ 0, 0, 0, 128, 0, 0 },
}

```

§ 9.6. 1d transform tables

```

Dct_Kernel4[4][4] =
{
    { 64, 64, 64, 64 },
    { 83, 35, -35, -83 },
    { 64, -64, -64, 64 },
    { 35, -83, 83, -35 },
}

Dct_Kernel8[8][8] =
{
    { 64, 64, 64, 64, 64, 64, 64, 64 },

```

```

{ 89, 75, 50, 18, -18, -50, -75, -89 },
{ 83, 35, -35, -83, -83, -35, 35, 83 },
{ 75, -18, -89, -50, 50, 89, 18, -75 },
{ 64, -64, -64, 64, 64, -64, -64, 64 },
{ 50, -89, 18, 75, -75, -18, 89, -50 },
{ 35, -83, 83, -35, -35, 83, -83, 35 },
{ 18, -50, 75, -89, 89, -75, 50, -18 },
}

```

Dct_Kernel16[16][16] =

```

{
{ 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64 },
{ 90, 87, 80, 70, 57, 43, 26, 9, -9, -26, -43, -57, -70, -80, -87, -90 },
{ 89, 75, 50, 18, -18, -50, -75, -89, -89, -75, -50, -18, 18, 50, 75, 89 },
{ 87, 57, 9, -43, -80, -90, -70, -26, 26, 70, 90, 80, 43, -9, -57, -87 },
{ 83, 35, -35, -83, -83, -35, 35, 83, 83, 35, -35, -83, -83, -35, 35, 83 },
{ 80, 9, -70, -87, -26, 57, 90, 43, -43, -90, -57, 26, 87, 70, -9, -80 },
{ 75, -18, -89, -50, 50, 89, 18, -75, -75, 18, 89, 50, -50, -89, -18, 75 },
{ 70, -43, -87, 9, 90, 26, -80, -57, 57, 80, -26, -90, -9, 87, 43, -70 },
{ 64, -64, -64, 64, 64, -64, -64, 64, 64, -64, -64, 64, 64, -64, -64, 64 },
{ 57, -80, -26, 90, -9, -87, 43, 70, -70, -43, 87, 9, -90, 26, 80, -57 },
{ 50, -89, 18, 75, -75, -18, 89, -50, -50, 89, -18, -75, 75, 18, -89, 50 },
{ 43, -90, 57, 26, -87, 70, 9, -80, 80, -9, -70, 87, -26, -57, 90, -43 },
{ 35, -83, 83, -35, -35, 83, -83, 35, 35, -83, 83, -35, -35, 83, -83, 35 },
{ 26, -70, 90, -80, 43, 9, -57, 87, -87, 57, -9, -43, 80, -90, 70, -26 },
{ 18, -50, 75, -89, 89, -75, 50, -18, -18, 50, -75, 89, -89, 75, -50, 18 },
{ 9, -26, 43, -57, 70, -80, 87, -90, 90, -87, 80, -70, 57, -43, 26, -9 },
}

```

Dct_Kernel32[32][32] =

```

{
{
{ 64, 64, 64, 64, 64, 64, 64, 64,
64, 64, 64, 64, 64, 64, 64, 64,
64, 64, 64, 64, 64, 64, 64, 64,
64, 64, 64, 64, 64, 64, 64, 64 },
{
90, 90, 88, 85, 82, 78, 73, 67,
61, 54, 47, 39, 30, 22, 13, 4,
-4, -13, -22, -30, -39, -47, -54, -61,
-67, -73, -78, -82, -85, -88, -90, -90 },
{
90, 87, 80, 70, 57, 43, 26, 9,
-9, -26, -43, -57, -70, -80, -87, -90,
-90, -87, -80, -70, -57, -43, -26, -9,
9, 26, 43, 57, 70, 80, 87, 90 },
{
90, 82, 67, 47, 22, -4, -30, -54,
-73, -85, -90, -88, -78, -61, -39, -13,
13, 39, 61, 78, 88, 90, 85, 73,
54, 30, 4, -22, -47, -67, -82, -90 },
{
89, 75, 50, 18, -18, -50, -75, -89,
-89, -75, -50, -18, 18, 50, 75, 89,
89, 75, 50, 18, -18, -50, -75, -89,
-89, -75, -50, -18, 18, 50, 75, 89 },
{
88, 67, 30, -13, -54, -82, -90, -78,
-47, -4, 39, 73, 90, 85, 61, 22,
-22, -61, -85, -90, -73, -39, 4, 47,
78, 90, 82, 54, 13, -30, -67, -88 },
{
87, 57, 9, -43, -80, -90, -70, -26,
26, 70, 90, 80, 43, -9, -57, -87,
-87, -57, -9, 43, 80, 90, 70, 26,
-26, -70, -90, -80, -43, 9, 57, 87 },
{
85, 47, -13, -67, -90, -73, -22, 39,
82, 88, 54, -4, -61, -90, -78, -30,
30, 78, 90, 61, 4, -54, -88, -82,
-39, 22, 73, 90, 67, 13, -47, -85 },
{
83, 35, -35, -83, -83, -35, 35, 83,
83, 35, -35, -83, -83, -35, 35, 83,
83, 35, -35, -83, -83, -35, 35, 83,
83, 35, -35, -83, -83, -35, 35, 83 },
{
82, 22, -54, -90, -61, 13, 78, 85,
30, -47, -90, -67, 4, 73, 88, 39,
-39, -88, -73, -4, 67, 90, 47, -30,
-85, -78, -13, 61, 90, 54, -22, -82 },
}
}

```

```

{ 80, 9, -70, -87, -26, 57, 90, 43,
-43, -90, -57, 26, 87, 70, -9, -80,
-80, -9, 70, 87, 26, -57, -90, -43,
43, 90, 57, -26, -87, -70, 9, 80 },
{ 78, -4, -82, -73, 13, 85, 67, -22,
-88, -61, 30, 90, 54, -39, -90, -47,
47, 90, 39, -54, -90, -30, 61, 88,
22, -67, -85, -13, 73, 82, 4, -78 },
{ 75, -18, -89, -50, 50, 89, 18, -75,
-75, 18, 89, 50, -50, -89, -18, 75,
75, -18, -89, -50, 50, 89, 18, -75,
-75, 18, 89, 50, -50, -89, -18, 75 },
{ 73, -30, -90, -22, 78, 67, -39, -90,
-13, 82, 61, -47, -88, -4, 85, 54,
-54, -85, 4, 88, 47, -61, -82, 13,
90, 39, -67, -78, 22, 90, 30, -73 },
{ 70, -43, -87, 9, 90, 26, -80, -57,
57, 80, -26, -90, -9, 87, 43, -70,
-70, 43, 87, -9, -90, -26, 80, 57,
-57, -80, 26, 90, 9, -87, -43, 70 },
{ 67, -54, -78, 39, 85, -22, -90, 4,
90, 13, -88, -30, 82, 47, -73, -61,
61, 73, -47, -82, 30, 88, -13, -90,
-4, 90, 22, -85, -39, 78, 54, -67 },
{ 64, -64, -64, 64, 64, -64, -64, 64,
64, -64, -64, 64, 64, -64, -64, 64,
64, -64, -64, 64, 64, -64, -64, 64,
64, -64, -64, 64, 64, -64, -64, 64 },
{ 61, -73, -47, 82, 30, -88, -13, 90,
-4, -90, 22, 85, -39, -78, 54, 67,
-67, -54, 78, 39, -85, -22, 90, 4,
-90, 13, 88, -30, -82, 47, 73, -61 },
{ 57, -80, -26, 90, -9, -87, 43, 70,
-70, -43, 87, 9, -90, 26, 80, -57,
-57, 80, 26, -90, 9, 87, -43, -70,
70, 43, -87, -9, 90, -26, -80, 57 },
{ 54, -85, -4, 88, -47, -61, 82, 13,
-90, 39, 67, -78, -22, 90, -30, -73,
73, 30, -90, 22, 78, -67, -39, 90,
-13, -82, 61, 47, -88, 4, 85, -54 },
{ 50, -89, 18, 75, -75, -18, 89, -50,
-50, 89, -18, -75, 75, 18, -89, 50,
50, -89, 18, 75, -75, -18, 89, -50,
-50, 89, -18, -75, 75, 18, -89, 50 },
{ 47, -90, 39, 54, -90, 30, 61, -88,
22, 67, -85, 13, 73, -82, 4, 78,
-78, -4, 82, -73, -13, 85, -67, -22,
88, -61, -30, 90, -54, -39, 90, -47 },
{ 43, -90, 57, 26, -87, 70, 9, -80,
80, -9, -70, 87, -26, -57, 90, -43,
-43, 90, -57, -26, 87, -70, -9, 80,
-80, 9, 70, -87, 26, 57, -90, 43 },
{ 39, -88, 73, -4, -67, 90, -47, -30,
85, -78, 13, 61, -90, 54, 22, -82,
82, -22, -54, 90, -61, -13, 78, -85,
30, 47, -90, 67, 4, -73, 88, -39 },
{ 35, -83, 83, -35, -35, 83, -83, 35,
35, -83, 83, -35, -35, 83, -83, 35,
35, -83, 83, -35, -35, 83, -83, 35,
35, -83, 83, -35, -35, 83, -83, 35 },
{ 30, -78, 90, -61, 4, 54, -88, 82,
-39, -22, 73, -90, 67, -13, -47, 85,
-85, 47, 13, -67, 90, -73, 22, 39,
-82, 88, -54, -4, 61, -90, 78, -30 },
{ 26, -70, 90, -80, 43, 9, -57, 87,
-87, 57, -9, -43, 80, -90, 70, -26,
-26, 70, -90, 80, -43, -9, 57, -87,
87, -57, 9, 43, -80, 90, -70, 26 },
{ 22, -61, 85, -90, 73, -39, -4, 47,
-78, 90, -82, 54, -13, -30, 67, -88,
88, -67, 30, 13, -54, 82, -90, 78,

```

```

    -47,  4, 39, -73, 90, -85, 61, -22 },
{ 18, -50, 75, -89, 89, -75, 50, -18,
  -18, 50, -75, 89, -89, 75, -50, 18,
  18, -50, 75, -89, 89, -75, 50, -18,
  -18, 50, -75, 89, -89, 75, -50, 18 },
{ 13, -39, 61, -78, 88, -90, 85, -73,
  54, -30,  4, 22, -47, 67, -82, 90,
  -90, 82, -67, 47, -22, -4, 30, -54,
  73, -85, 90, -88, 78, -61, 39, -13 },
{  9, -26, 43, -57, 70, -80, 87, -90,
  90, -87, 80, -70, 57, -43, 26, -9,
  -9, 26, -43, 57, -70, 80, -87, 90,
  -90, 87, -80, 70, -57, 43, -26,  9 },
{  4, -13, 22, -30, 39, -47, 54, -61,
  67, -73, 78, -82, 85, -88, 90, -90,
  90, -90, 88, -85, 82, -78, 73, -67,
  61, -54, 47, -39, 30, -22, 13, -4 }
}

Adst_Kernel4[4][4] =
{
    { 18, 50, 75, 89 },
    { 50, 89, 18, -75 },
    { 75, 18, -89, 50 },
    { 89, -75, 50, -18 },
}

Adst_Kernel8[8][8] =
{
    { 11, 28, 44, 58, 70, 79, 86, 89 },
    { 34, 74, 89, 76, 39, -12, -58, -86 },
    { 54, 89, 48, -34, -87, -66, 12, 79 },
    { 71, 68, -41, -86, 1, 87, 38, -70 },
    { 84, 17, -89, 10, 86, -35, -75, 58 },
    { 88, -44, -44, 88, -44, -44, 88, -44 },
    { 79, -83, 50, 6, -59, 86, -74, 29 },
    { 50, -69, 81, -84, 78, -62, 40, -14 },
}

Adst_Kernel16[16][16] =
{
    { 8, 17, 25, 33, 41, 48, 55, 62, 67, 73, 77, 81, 84, 87, 88, 89 },
    { 25, 48, 67, 81, 88, 88, 81, 67, 48, 25, 0, -25, -48, -67, -81, -88 },
    { 41, 73, 88, 84, 62, 25, -17, -55, -81, -89, -77, -48, -8, 33, 67, 87 },
    { 55, 87, 81, 41, -17, -67, -89, -73, -25, 33, 77, 88, 62, 8, -48, -84 },
    { 67, 88, 48, -25, -81, -81, -25, 48, 88, 67, 0, -67, -88, -48, 25, 81 },
    { 77, 77, 0, -77, -77, 0, 77, 77, 0, -77, -77, 0, 77, 77, 0, -77 },
    { 84, 55, -48, -87, -8, 81, 62, -41, -88, -17, 77, 67, -33, -89, -25, 73 },
    { 88, 25, -81, -48, 67, 67, -48, -81, 25, 88, 0, -88, -25, 81, 48, -67 },
    { 89, -8, -88, 17, 87, -25, -84, 33, 81, -41, -77, 48, 73, -55, -67, 62 },
    { 87, -41, -67, 73, 33, -88, 8, 84, -48, -62, 77, 25, -89, 17, 81, -55 },
    { 81, -67, -25, 88, -48, -88, 88, -25, -67, 81, 0, -81, 67, 25, -88, 48 },
    { 73, -84, 25, 55, -89, 48, 33, -87, 67, 8, -77, 81, -17, -62, 88, -41 },
    { 62, -89, 67, -8, -55, 88, -73, 17, 48, -87, 77, -25, -41, 84, -81, 33 },
    { 48, -81, 88, -67, 25, 25, -67, 88, -81, 48, 0, -48, 81, -88, 67, -25 },
    { 33, -62, 81, -89, 84, -67, 41, -8, -25, 55, -77, 88, -87, 73, -48, 17 },
    { 17, -33, 48, -62, 73, -81, 87, -89, 88, -84, 77, -67, 55, -41, 25, -8 },
}

Fdst_Kernel4[4][4] =
{
    { 89, 75, 50, 18 },
    { 75, -18, -89, -50 },
    { 50, -89, 18, 75 },
    { 18, -50, 75, -89 },
}

Fdst_Kernel8[8][8] =
{
    { 89, 86, 79, 70, 58, 44, 28, 11 },
    { -86, -58, -12, 39, 76, 89, 74, 34 },
}

```



```

{ 79, 12, -66, -87, -34, 48, 89, 54 },
{ -70, 38, 87, 1, -86, -41, 68, 71 },
{ 58, -75, -35, 86, 10, -89, 17, 84 },
{ -44, 88, -44, -44, 88, -44, -44, 88 },
{ 29, -74, 86, -59, 6, 50, -83, 79 },
{ -14, 40, -62, 78, -84, 81, -69, 50 },
}

Fdst_Kernel16[16][16] =
{
{ 89, 88, 87, 84, 81, 77, 73, 67, 62, 55, 48, 41, 33, 25, 17, 8 },
{ 88, 81, 67, 48, 25, 0, -25, -48, -67, -81, -88, -88, -81, -67, -48, -25 },
{ 87, 67, 33, -8, -48, -77, -89, -81, -55, -17, 25, 62, 84, 88, 73, 41 },
{ 84, 48, -8, -62, -88, -77, -33, 25, 73, 89, 67, 17, -41, -81, -87, -55 },
{ 81, 25, -48, -88, -67, 0, 67, 88, 48, -25, -81, -81, -25, 48, 88, 67 },
{ 77, 0, -77, -77, 0, 77, 77, 0, -77, -77, 0, 77, 77, 0, -77, -77 },
{ 73, -25, -89, -33, 67, 77, -17, -88, -41, 62, 81, -8, -87, -48, 55, 84 },
{ 67, -48, -81, 25, 88, 0, -88, -25, 81, 48, -67, -67, 48, 81, -25, -88 },
{ 62, -67, -55, 73, 48, -77, -41, 81, 33, -84, -25, 87, 17, -88, -8, 89 },
{ 55, -81, -17, 89, -25, -77, 62, 48, -84, -8, 88, -33, -73, 67, 41, -87 },
{ 48, -88, 25, 67, -81, 0, 81, -67, -25, 88, -48, -48, 88, -25, -67, 81 },
{ 41, -88, 62, 17, -81, 77, -8, -67, 87, -33, -48, 89, -55, -25, 84, -73 },
{ 33, -81, 84, -41, -25, 77, -87, 48, 17, -73, 88, -55, -8, 67, -89, 62 },
{ 25, -67, 88, -81, 48, 0, -48, 81, -88, 67, -25, -25, 67, -88, 81, -48 },
{ 17, -48, 73, -87, 88, -77, 55, -25, -8, 41, -67, 84, -89, 81, -62, 33 },
{ 8, -25, 41, -55, 67, -77, 84, -88, 89, -87, 81, -73, 62, -48, 33, -17 },
}

Ddtx_Kernel8[8][8] =
{
{ 4, 7, 15, 33, 65, 98, 106, 80 },
{ 6, 14, 36, 77, 100, 45, -57, -98 },
{ 22, 48, 85, 88, 0, -86, -23, 82 },
{ 57, 94, 76, -26, -73, 34, 54, -66 },
{ 96, 73, -43, -69, 55, 20, -71, 53 },
{ 103, -17, -80, 56, 15, -66, 75, -41 },
{ 78, -79, 7, 56, -82, 79, -56, 26 },
{ 56, -96, 98, -77, 54, -33, 19, -6 },
}

Ddtx_Kernel16[16][16] =
{
{ 12, 15, 19, 23, 30, 39, 51, 66, 78, 88, 94, 97, 93, 83, 68, 50 },
{ 17, 23, 30, 38, 48, 61, 76, 87, 83, 59, 19, -30, -73, -99, -99, -76 },
{ 37, 49, 60, 69, 75, 75, 61, 29, -18, -67, -96, -83, -28, 40, 84, 83 },
{ 45, 60, 69, 73, 66, 40, -8, -65, -91, -57, 21, 86, 81, 8, -69, -90 },
{ 47, 60, 61, 49, 19, -29, -77, -83, -16, 75, 93, 3, -92, -74, 32, 97 },
{ 60, 74, 64, 28, -31, -87, -82, 4, 88, 54, -55, -77, 29, 88, 3, -86 },
{ 64, 70, 40, -19, -79, -78, 11, 92, 28, -85, -41, 82, 39, -83, -37, 83 },
{ 82, 73, 3, -80, -91, 10, 94, 18, -84, -5, 80, -17, -70, 47, 55, -68 },
{ 89, 48, -53, -96, -5, 89, 16, -83, 12, 75, -51, -43, 81, -14, -75, 67 },
{ 100, 9, -99, -45, 84, 36, -81, 4, 73, -60, -17, 76, -55, -21, 81, -56 },
{ 92, -35, -91, 42, 71, -69, -22, 85, -60, -17, 77, -70, 11, 56, -83, 49 },
{ 84, -71, -46, 88, -16, -67, 79, -22, -46, 84, -68, 15, 46, -83, 82, -40 },
{ 69, -83, 2, 75, -78, 18, 50, -85, 81, -43, -6, 53, -81, 88, -69, 32 },
{ 50, -79, 47, 14, -60, 67, -37, -6, 49, -80, 98, -99, 90, -71, 48, -19 },
{ 51, -89, 73, -17, -45, 89, -103, 97, -83, 71, -56, 44, -31, 22, -11, 5 },
{ 44, -95, 124, -126, 108, -81, 54, -30, 16, -6, 1, 3, -4, 5, -3, 2 },
}

```

§ 9.7. Secondary transform tables

```

Ist_4x4_Kernel[IST_SET_SIZE_4X4][STX_TYPES - 1][IST_4X4_HEIGHT][IST_4X4_WIDTH] =
{
{
// 0
{
{ 102, -45, -53, -5, -13, -3, -3, 19, 22, -2, 3, 8, 2, -5, -8, -4 },
{ 34, 39, 55, -33, -81, -18, 0, -10, -14, 0, 19, 42, 8, 2, 3, -13 },

```

```

    { -53, -8, -21, -8, -43, -3, 2, 56, 72, 8, 9, 21, 4, -29, -37, -8 },
    { -35, -49, -40, 20, -21, 23, 5, -24, -36, 9, 29, 61, 13, 25, 30,
      -40 },
    { 0, -80, 74, 39, -9, -16, 8, 33, -11, -11, -29, -3, 13, -11, 5, 4 },
    { -22, -24, -30, -28, -49, -54, 11, -8, -6, 25, -8, -48, 6, 33, 38,
      50 },
    { 5, 20, -7, 69, -3, -58, -3, -54, 40, 3, -27, 15, 30, 21, -26, -14 },
    { -9, -29, -6, 5, -45, 33, -30, -63, -18, -12, -6, -29, -30, -39, -58,
      16 },
  },
  {
    { 115, -36, -36, -10, -16, -11, -4, 4, 4, -3, 4, 7, 2, 1, 1, -1 },
    { 51, 67, 73, -2, 28, -6, -9, -30, -30, -9, -16, -26, -12, 2, 1, 10 },
    { -17, 0, -17, -50, -68, -23, 2, -47, -66, -6, 1, -8, 4, 21, 26, 19 },
    { 6, 52, -47, -58, 21, 68, 13, 46, -24, -15, 3, 0, 1, -8, 4, -1 },
    { -12, -45, -19, -16, 31, -12, -13, -3, -27, -30, -48, -68, -22, -37,
      -44, -2 },
    { 2, -50, 62, -7, -5, 17, 31, 53, -37, -24, -34, 23, 44, 17, 9, 3 },
    { -10, 2, -9, -46, 55, -60, -30, 3, 24, -51, -7, 41, 7, 31, 21, 15 },
    { 1, -28, -15, 12, 62, 6, 25, -10, -20, 40, 34, -43, 19, 30, 41, 52 },
  },
  {
    { 114, 13, -42, -18, 23, -22, -4, -3, -3, -3, -4, 9, -2, -3, 4, 1 },
    { -39, 75, -64, 13, 17, -14, -5, 36, -43, 9, -9, 28, -15, 3, -6, 11 },
    { 19, -13, -37, 7, -98, 29, -4, 37, -22, -8, 17, -27, 12, 24, -17,
      4 },
    { 33, 66, 74, 40, 11, 22, -4, 38, -4, -4, 16, -27, 0, 9, -25, -3 },
    { -4, 35, -5, 38, -39, -70, -6, -68, 2, 3, 18, -40, -8, -17, 1, -15 },
    { 12, -49, 34, 36, -6, -45, -12, 7, -61, 8, -3, 44, -29, 11, -31,
      37 },
    { -9, 2, 16, -70, 17, -16, 16, -8, -74, -17, -15, -56, 17, 8, -25,
      -15 },
    { 11, 33, 24, -24, -41, 51, -7, -46, -27, -7, -42, 29, -41, -39, 23,
      21 },
  },
},
{
  // 1
  {
    { 70, 64, -63, 12, -52, 6, -12, 0, 4, -1, 2, -10, 13, -7, 4, 1 },
    { -47, -26, -29, 64, -45, 9, 11, -29, 66, -17, -13, 18, -7, -9, 5,
      10 },
    { 32, -48, -23, 27, 22, -78, -30, 55, 21, -13, -10, 1, 20, 4, -4,
      -4 },
    { 77, -28, 21, -8, 21, 15, 18, -45, 27, -19, -25, 51, -32, 22, -18,
      1 },
    { -23, 77, 42, 15, 17, -6, -30, 22, 23, -38, -48, 29, 15, 17, -25,
      -2 },
    { 22, -10, 1, 35, 40, 59, 40, 37, 2, -38, -27, -53, 13, -35, 5, 8 },
    { -1, -32, -15, 2, 7, 68, -98, 1, -3, 0, 3, -3, 8, 26, -4, -6 },
    { 22, 5, 55, 8, -5, -23, -42, -36, 12, -15, 13, -25, -1, -44, 62,
      47 },
  },
  {
    { 80, -82, -39, -1, 26, 22, 1, 13, 10, -3, 1, -11, -11, 0, -1, 2 },
    { 53, 21, -6, -6, -47, -86, 24, 42, 10, -2, 0, 16, 18, -20, -3, -1 },
    { 61, 55, -25, -1, -5, 9, -57, -63, -12, -1, -1, 12, 24, 27, 4, -6 },
    { -39, -27, -52, 28, 54, -61, -1, -24, -6, 2, -6, 10, 56, 10, 7, -7 },
    { 13, 63, -39, 8, 59, -10, 33, 20, -5, 1, -10, -51, -52, 6, 6, 14 },
    { -27, 8, -52, 15, -26, 13, -76, 55, 42, 1, -8, -21, 5, -17, -15,
      14 },
    { 21, 25, 29, 7, 50, 32, -4, 32, -26, -10, -4, 7, 60, -72, -15, -4 },
    { -8, 19, -64, 7, -39, 55, 72, -10, 17, 4, 1, 16, 34, -8, -1, -11 },
  },
},
{
  { -39, -94, 30, -3, 1, 29, 21, -55, 24, 1, 1, 0, 4, -18, 5, 1 },
  { -85, 42, 20, 8, -55, 45, -9, 16, 0, 1, 1, -28, 25, 8, -1, -6 },
  { -80, 6, 5, 13, 35, -63, 4, 7, -11, 4, -2, 40, -52, 0, -2, 11 },
  { -5, 51, -46, 6, 7, 26, 55, -62, -18, 6, -5, 20, 3, -55, 5, 9 },
  { 9, 38, 69, -6, 63, 50, -25, -4, 24, 0, -13, 41, 5, 0, 8, 18 },
  { -25, -31, -54, 0, 62, 44, 5, 44, -42, 9, -5, 6, 39, 19, -18, -3 },

```

```

    { 17, -13, -10, 20, -42, 56, 13, 15, -13, -7, 7, 40, -72, 31, -26,
      38 },
    { -7, -11, -39, 0, -8, 14, -103, -13, -3, 7, -1, 5, -17, -53, 14,
      16 },
  },
{
  // 2
  {
    { 103, -54, -44, -5, -9, -3, -3, 20, 14, -3, 3, 9, 2, -6, -2, -5 },
    { 55, 73, 25, -23, -47, -25, -7, -44, -28, 0, 5, 23, 8, 24, 13, -3 },
    { 44, 18, 35, -9, 74, 24, -7, -10, -30, -9, -14, -61, -33, -10, 0,
      23 },
    { -13, -75, 36, -22, 1, -61, -6, -25, -42, -2, 4, -9, 14, 28, 31,
      25 },
    { 16, 3, 78, 6, 11, -33, -3, 56, 10, -14, -15, 26, 32, -42, -25,
      -32 },
    { -7, -10, 9, -56, -45, 52, 30, 42, -58, 11, 22, -9, -5, -37, 17,
      -1 },
    { 15, -22, 44, 41, 17, 66, 40, -13, -1, 9, 6, 44, 12, 50, 29, -15 },
    { -4, 31, -38, -40, 65, -13, 2, 41, -13, 11, 26, 21, 57, 33, 24, 0 },
  },
  {
    { 87, -43, -77, 12, 22, -3, -3, 13, 15, 2, -6, -10, 1, -1, -2, 2 },
    { 50, 29, 19, -60, -73, -20, -3, 16, 40, 16, 12, 22, 5, -9, -22, 0 },
    { 3, -76, 50, -69, 48, -3, 6, 17, -5, 20, 11, -6, -9, 1, -5, 1 },
    { -51, -13, -16, 8, -2, -13, 3, 47, 79, 14, -42, -41, 1, -17, -18,
      21 },
    { 2, 15, 5, 13, 10, -83, -15, 67, -49, -6, 16, -11, 33, -20, 5, -2 },
    { -16, 52, -47, -47, 32, -1, -1, -11, -21, 83, -2, -19, 0, 15, 3, 2 },
    { 48, 42, 45, -6, 16, 6, 7, -3, -4, -22, -42, -72, -23, 3, 33, 28 },
    { -17, -11, -27, -12, -42, 44, 19, 37, -23, -10, 69, -48, -25, -17,
      26, 19 },
  },
},
{
  { 109, -45, -42, -7, -12, -14, -1, 13, 13, -2, 2, 6, 5, -1, -1, -2 },
  { 31, 73, 37, -21, -67, -53, 3, -14, -15, 0, 2, 21, 13, 4, 5, 0 },
  { -46, -30, -15, 6, -45, -43, 17, 66, 52, 7, 5, 25, 20, -23, -14,
    -9 },
  { -22, 27, -91, 32, 0, -13, -7, -44, -6, 5, 10, 40, 28, 30, 10, -14 },
  { 15, 77, -41, 17, 28, -4, 5, 41, 27, -2, -19, -43, -27, -34, -24,
    13 },
  { 0, 19, -20, -34, -58, 94, -8, 10, 21, 11, 23, 21, -24, -2, -1, -5 },
  { 23, 20, 35, 41, 31, 37, 1, 17, 1, -17, -3, 54, 57, -30, -27, -42 },
  { -7, 9, -25, -64, 34, -5, 33, 46, -66, -3, 31, 21, 15, -22, 26, 0 },
},
{
  // 3
  {
    { -86, 25, 7, 20, -64, 11, 2, 25, -16, -4, 14, -42, 7, 17, -16, -18 },
    { 16, -37, 39, -24, -29, 4, -3, -41, 62, -2, 5, -58, 17, -13, 28,
      -42 },
    { 66, -28, 20, 2, -27, 3, 9, 33, -19, -21, 30, -28, 7, 55, -56, 16 },
    { 22, 28, -54, -18, 28, -38, 6, 15, -24, 4, -6, -40, -7, 11, -5,
      -84 },
    { -35, 26, -17, -29, 50, -19, 23, -39, 44, -23, 25, -21, 26, 40, -36,
      36 },
    { 28, 80, 35, -3, -3, -14, -10, 53, 41, -2, 7, -4, -4, 18, 50, 16 },
    { -30, -17, 69, -15, 35, 0, 13, 14, 8, -4, -31, 37, -39, 45, -19,
      -48 },
    { 6, 21, 38, 51, 6, -53, -3, -35, -8, -17, 56, 0, -41, -43, -26,
      -11 },
  },
  {
    { -95, -25, 57, 2, -18, 22, 2, -35, 34, 0, -6, 5, 0, -12, 10, 5 },
    { 14, -45, 27, -21, 79, -6, 10, -15, -11, -1, -18, 64, -22, 16, -26,
      20 },
    { 47, 11, -18, 14, -15, 25, -8, -53, 49, 0, -11, 37, 8, -44, 33, 52 },
    { -16, 37, -28, 34, -39, 27, 22, -8, 6, -26, -24, 45, -28, 51, -58,
      0 },
  },
}

```

```

    { 44, -27, 15, -22, -12, 3, 14, -44, 44, -10, -25, -23, -13, 13, -8,
      -84 },
    { -19, -71, -68, 48, 0, -12, 15, -32, -16, 7, 36, -8, -25, -14, 0,
      -9 },
    { 12, -14, 24, 46, -3, -54, 2, 44, 60, -18, 11, 4, -32, 41, 39, 16 },
    { -41, 53, -43, -2, 44, -30, -5, -4, 14, 11, -35, 26, -24, -24, 41,
      -46 },
  },
  {
    { 110, -43, 10, -22, 35, -13, 0, -8, 10, -6, -3, 15, -5, 2, 0, 3 },
    { 15, 60, -76, -16, 42, -27, -3, 40, -43, 2, -11, 12, -5, 11, -9,
      -6 },
    { -47, -10, -5, -22, 72, -4, 9, -46, 29, -1, -24, 59, -17, -18, 15,
      20 },
    { -15, -70, -86, -41, -25, 15, 5, -14, 5, -2, 5, -24, 13, -12, 3,
      -10 },
    { 21, 61, -14, -18, 1, -4, -14, -30, 62, -4, 12, -41, 31, -37, 50,
      -8 },
    { -14, 10, 36, -90, 7, 34, -6, 40, 16, -35, -21, -8, 18, 34, -15,
      -11 },
    { 25, 20, -14, -2, -48, 65, -14, 6, -15, 6, -36, 57, 12, -29, 19,
      48 },
    { 0, -11, -27, 44, 24, 41, 2, 59, 67, -8, 37, 13, -11, 36, 14, 14 },
  },
},
{
  // 4
  {
    { -103, -31, 38, 7, -26, 28, 8, -36, 19, 3, 0, -3, 1, -12, 4, 0 },
    { -16, 75, -16, 3, -60, 33, -16, 31, -10, 0, 4, -48, 44, -4, 4, -12 },
    { -51, 31, 28, 6, 27, -34, -8, 70, -17, -3, -1, 22, -25, 59, -16,
      -3 },
    { 21, -54, 49, -30, 20, 18, -10, 26, 7, 0, -2, -41, 64, 35, -8, -33 },
    { -29, -49, -55, 9, -11, -68, -8, 20, -30, 4, 15, -50, 1, -24, 4,
      -23 },
    { 19, -40, -28, 32, -67, 20, 16, 13, 32, -7, 12, 1, -30, 71, -6, 7 },
    { -25, -8, -67, 6, 45, 58, 42, -1, -42, 2, -8, 17, 28, 21, -26, -9 },
    { 13, -30, 30, -19, -29, 42, -5, 39, -70, 1, 9, -2, -44, -38, -31,
      26 },
  },
  {
    { -88, 68, 2, 20, -29, -2, -5, 40, -24, 1, 2, 1, -10, 14, -9, 5 },
    { 20, 37, -38, 12, 65, -47, 5, -1, -29, 5, -8, 59, -35, -22, 3, 19 },
    { 56, -13, 16, 10, -18, 8, -9, 72, -24, -14, 18, 14, -24, 54, -33,
      36 },
    { -28, -54, 21, 27, -26, 43, -13, -11, -21, 1, 2, 50, -38, -58, -2,
      32 },
    { 31, 28, 63, -4, -36, -49, -24, 12, 21, 2, 0, -7, -49, -38, 27,
      -34 },
    { -34, -12, 59, -8, 25, -37, 27, -37, 46, -10, 15, 16, -16, 43, -15,
      49 },
    { 7, -13, -10, 34, -34, -51, 44, -32, -47, -17, 38, -33, 6, -22, -51,
      -5 },
    { -39, -66, 1, -4, 34, -26, 7, 32, -19, 3, -14, -1, -34, 24, -5,
      -72 },
  },
},
{
  { 104, -59, 35, -4, 11, -7, 1, -22, 14, 1, 1, 3, -1, -5, 3, 1 },
  { 33, 48, -45, -10, 80, -44, 2, 11, -14, -1, -6, 39, -23, 2, -3, 9 },
  { 45, 30, 11, 12, -37, 11, -15, 86, -38, -3, 3, -11, 0, 45, -21, -4 },
  { 30, 30, -53, -27, -8, -11, 9, -20, 19, -9, 9, -74, 53, 3, 2, -40 },
  { -6, -76, -81, -2, -3, 18, 16, 7, -47, -1, 0, 7, 0, 11, -29, 9 },
  { 34, 32, -40, 39, -52, 19, -18, -16, -16, 7, 7, 15, -14, -72, 29,
    26 },
  { -10, -13, 5, 25, -37, -98, 15, -10, -19, -5, 23, -25, -46, -3, -15,
    -16 },
  { 8, 13, -23, 13, -22, 8, 19, -15, 66, -40, 37, 10, -17, 49, -23,
    61 },
},
},
{

```

```

// 5
{
    { -94, 10, -63, 11, 30, 16, 2, 18, 37, 16, 7, -2, 0, -5, -14, -3 },
    { 50, -53, -81, -38, -26, 7, 1, 25, 20, -5, 22, 26, 5, 1, 8, -5 },
    { 64, 8, -23, 49, 64, -4, -8, -5, 42, 16, -23, -36, -10, -10, -34,
      1 },
    { -17, -75, 2, 77, -43, -4, 11, 4, -12, 29, -35, -4, 14, 7, -1, 4 },
    { 7, -5, 54, 23, -8, 16, -3, 30, 60, 33, 65, 42, -6, -16, -5, -27 },
    { -8, 4, -5, -26, -51, -57, -7, -60, 37, 33, 20, -31, -1, 6, -46, 6 },
    { 12, 33, -33, 38, 12, -21, 3, -37, -24, 47, 42, 20, 9, 28, 69, 9 },
    { -19, -76, 21, -11, 58, -20, -6, -35, 7, -36, 44, -24, -8, 6, 25,
      9 },
},
{
    { 84, -42, -75, 12, 17, -1, -3, 15, 27, -1, -12, -12, 0, -3, -7, 5 },
    { 44, 8, 28, -57, -75, -7, -2, 17, 39, 3, 27, 34, 2, -11, -28, -6 },
    { -52, 35, -29, -4, 2, -9, -1, 21, 81, 25, -31, -32, -2, -9, -39,
      17 },
    { 37, 12, 45, 12, -35, -31, 4, 8, -9, 8, -79, -46, 1, 9, 34, 27 },
    { -28, -44, -8, -54, 3, -2, 9, 56, 10, 17, 24, -27, -11, 8, 75, 4 },
    { 17, -35, 62, -30, 56, 6, -5, -3, 8, -19, 18, -49, -12, -13, -50,
      30 },
    { -40, -77, 5, 32, -42, 13, -2, 33, -5, -40, -33, 20, 11, -26, -27,
      9 },
    { 20, 8, 51, 64, 27, 15, 2, 50, 44, 16, 12, 20, -15, -15, 21, -46 },
},
{
    { -114, 32, -5, 31, 26, 14, 2, 5, 5, 11, -10, -10, -3, -4, -8, 0 },
    { 14, 17, -104, -24, 50, -14, 2, -5, 40, 6, 7, -1, 5, 0, -3, -3 },
    { 9, 69, 40, -26, 38, -52, 10, -60, -15, -21, -7, -20, 10, 9, 9, 5 },
    { -34, 24, 4, -95, -32, 13, -6, 23, 0, -21, 40, 40, -6, -4, 15, -7 },
    { 12, -7, 42, -20, 43, 39, -24, 14, 60, -29, 18, -49, -27, -14, -41,
      4 },
    { -24, -42, 16, 1, -25, -40, 41, -32, 59, 12, 48, -5, 47, 16, -17,
      -11 },
    { 26, 67, 17, 23, -24, -13, -6, 21, 36, 64, 10, 32, -13, -31, -33,
      -23 },
    { 2, -15, 20, 33, 51, -22, 12, 23, 17, -49, 8, 64, 0, -36, 37, -37 },
},
{
// 6
{
    { -104, 25, 12, 32, -49, 21, 1, 6, -1, 0, 11, -24, 8, 0, 1, -5 },
    { 7, 55, -83, 18, 5, -16, -4, 45, -55, 11, 5, 1, -2, 14, -18, -1 },
    { 11, 94, 70, -23, 6, -24, -15, 17, 1, -15, -12, 14, -8, 7, 4, 5 },
    { -56, -6, -2, 19, 79, -7, 5, -16, -10, -1, -23, 65, -23, -6, -5,
      29 },
    { 17, 8, -3, 83, -2, -67, -4, -18, 35, -6, 33, -8, -26, -18, 29, -2 },
    { -27, -13, -29, -62, -54, -75, 4, -9, 11, 3, -17, 35, -13, -7, 13,
      13 },
    { 1, -45, 51, 17, -17, -37, 7, 20, -61, -3, 23, 3, -26, 37, -55, 0 },
    { 5, 23, 3, -9, -3, -3, 3, -92, -61, 23, 1, -22, -4, -47, -12, -6 },
},
{
    { 104, -39, -59, -9, 3, -15, -2, 8, 13, 3, 0, 6, 6, 0, -2, -1 },
    { 53, 28, 82, -47, -13, -34, -3, -29, -19, -26, 8, 15, 3, 8, 16, -1 },
    { -11, -90, 43, 1, -57, 12, 27, 7, 35, -10, 19, -6, 16, -9, -6, -8 },
    { -26, 22, -44, -13, -69, -54, 21, -13, -15, 27, 22, 48, 29, 5, 3,
      -15 },
    { 7, -7, 11, 94, 11, -62, 12, -48, 4, -16, -12, -21, 11, -1, 8, -2 },
    { -37, -41, -6, -50, 63, -54, -4, -21, 42, 4, -19, 31, -8, -10, 13,
      2 },
    { 11, 42, 20, 26, 3, -15, 10, 68, 64, -23, -10, 46, 8, -27, -26,
      -13 },
    { -9, -30, -3, 5, 24, -4, 34, 45, -76, -51, -27, 33, 34, 6, 17, 1 },
},
{
    { 115, -20, -32, -22, 20, -27, -1, -2, 0, 0, -1, 8, 0, -1, 1, 0 },
    { 12, 3, 99, -22, -7, -49, 3, -43, 17, -27, 2, 17, -6, -7, 16, 0 },
    { -4, -102, -1, 25, -31, 10, 28, -15, 46, -3, 13, -21, 16, -11, 5,

```

```

        -7 },
    { -26, 2, -36, 62, 58, -65, 3, -32, 22, 6, -6, 19, -20, -20, 15, 4 },
    { 18, -16, 36, 32, 58, 56, -58, 1, 18, -15, -15, -33, -36, 16, 4,
      12 },
    { -33, -43, -7, -74, 41, 4, -6, 27, 24, 12, -29, 59, -18, -4, 3, 12 },
    { 8, 26, 13, 0, 55, 38, 81, 6, 17, -25, -22, -10, 45, -29, 7, -8 },
    { -11, 16, -25, -53, 9, 18, -16, -84, 19, 41, 0, -45, 7, -9, 19,
      -16 },
    },
},
{
    // 7
    {
        { 105, -52, -47, 6, 12, 10, 0, 8, 8, 0, -4, -11, -5, 3, 2, 0 },
        { 42, 33, 42, -38, -65, -44, 14, 45, 37, 9, -4, -9, -7, -11, -9, 10 },
        { -4, -64, 66, -59, -3, 61, -14, -14, 15, 11, 3, -1, -3, 6, -4, 0 },
        { -48, -29, -42, -26, 15, -16, 20, 38, 43, 14, -22, -62, -31, 27, 20,
          -1 },
        { -11, -52, 33, 43, -17, -18, 33, 45, -41, -49, 33, 1, -39, 8, -6,
          -1 },
        { 16, 3, 28, -32, 65, -43, 65, -18, -31, 45, -2, 18, -17, 6, 1, -3 },
        { -27, -48, -48, -45, -17, -41, -2, -3, -1, -5, 18, 48, 21, -44, -39,
          20 },
        { 8, 43, -11, -29, 46, 35, 16, 16, 26, -49, 60, -12, -16, -3, -45,
          22 },
    },
    {
        { 121, -17, -14, -4, -31, -6, -2, 10, 9, -1, 0, 10, 1, -5, -4, -2 },
        { 15, 71, 69, 5, -30, 3, 0, -43, -42, 1, 3, 32, 5, 16, 15, -17 },
        { -6, -73, 71, 24, -6, -29, 2, 42, -39, -1, -9, 6, 14, -23, 21, -3 },
        { -36, -6, -22, -27, -80, -27, -7, 15, 22, -10, 17, 61, 20, -20, -23,
          -22 },
        { 11, 36, 35, -54, 44, -57, -14, 23, 22, -12, 31, -19, 31, -28, -29,
          17 },
        { -1, -32, -37, -46, 6, -54, -20, -51, -53, -16, 5, -8, 13, 37, 36,
          -3 },
        { 1, -15, 12, -71, 1, 63, 27, 23, -30, -30, 45, 5, -41, -9, 9, -5 },
        { -9, 15, 4, -13, -67, -11, -1, -2, -20, -7, -31, -65, -30, -25, -12,
          64 },
    },
    {
        { 122, -2, -28, -12, -12, -14, -2, 0, 7, 1, 1, 6, 4, -1, -2, -1 },
        { 20, 53, 97, -18, -24, 2, -7, -31, -25, -27, 11, 14, 2, 10, 8, -6 },
        { -9, 102, -60, 10, 9, 5, -9, -27, -19, 24, -6, 14, -7, 9, 0, -1 },
        { -13, 8, -16, -16, -97, 50, -15, 22, 41, 1, 24, 12, -4, 4, -23, -4 },
        { 24, -2, 14, 40, 25, 96, 5, -6, -2, 5, -12, -45, -37, 19, 9, 8 },
        { -6, -10, -10, -112, 36, 32, 1, -11, -7, 17, 16, 3, -20, 9, 6, -2 },
        { 4, 12, 4, 0, 7, -10, -56, 86, -57, 5, 37, -24, -13, 7, -9, 5 },
        { -2, -42, -21, 18, -36, 8, 30, -30, -86, 9, 24, 39, -23, 20, 12,
          -13 },
    },
    },
},
{
    // 8
    {
        { 72, 100, -15, -1, -4, 20, -8, 15, -8, -1, 0, -6, 4, 3, 0, -1 },
        { -52, 19, -30, 4, -66, 76, 29, -22, -1, -2, -1, -21, 17, -14, 2,
          -2 },
        { 40, -25, -59, 21, -75, -59, -13, -24, 16, -2, 2, -7, -9, -5, 2,
          -2 },
        { 74, -44, 31, -2, 5, 43, 56, -51, 1, 4, -2, 3, -14, -15, -1, 4 },
        { 25, -38, -35, 20, 7, 14, 42, 87, 3, -4, -3, -12, 53, 9, -1, -3 },
        { -5, 13, 9, 50, 4, 15, -2, -1, 98, -10, -6, 55, 3, 6, 18, 17 },
        { -24, 28, -46, 5, 33, -31, 83, 0, 0, -10, 1, 5, -62, -4, 1, 5 },
        { 4, -13, -32, -42, -15, 23, -4, 0, -24, 3, -16, 78, -10, 72, -19,
          10 },
    },
    {
        { 109, -60, -19, 0, 2, 16, 0, 9, 4, -1, 0, -4, -6, 1, 1, 0 },
        { 34, 27, 76, -23, -63, -50, 23, 30, 9, 3, 1, 9, -2, -10, -4, 2 },
        { -32, -58, 65, -20, -25, 75, -32, 0, 4, -1, 2, 3, -1, 0, -1, -1 },
    }
}

```

```

    { 32, 27, 45, 35, 25, 8, -14, -68, -50, -5, 9, 41, 39, -7, 0, -6 },
    { 13, 18, 7, -90, 38, 1, 0, -50, 51, 20, -3, -15, 23, 3, -7, 5 },
    { -32, -84, 6, -9, 11, -62, 48, -31, -17, -1, -1, 18, 15, -10, -4,
      0 },
    { 4, 11, 39, 26, 45, 37, 89, 11, 5, 18, -13, -29, -17, 23, 20, 0 },
    { -1, 14, -22, -43, 11, 37, 43, 31, -23, -17, 31, 72, -11, -44, -23,
      8 },
  },
  {
    { 122, -14, 13, 2, 12, -26, 6, -13, 3, 2, 2, 0, -1, -1, -2, 0 },
    { 8, 113, -24, -3, 28, -35, -20, 15, -1, -2, 3, 10, -15, -2, 3, -1 },
    { -34, -32, -7, -3, 26, -105, 25, -37, 1, 1, 1, 6, -18, -11, -4, -3 },
    { -13, 32, 115, 7, -14, -9, -6, -34, 13, 6, 5, -14, 11, -1, -4, -1 },
    { 2, 33, -22, -2, -15, 13, 98, -37, 1, -1, -2, -10, 58, -3, -3, 6 },
    { -5, -4, 3, -15, 93, 53, 7, -48, 25, 2, 4, 15, -25, -27, 6, -6 },
    { 1, 2, -39, -10, -18, -4, -65, -64, 39, -8, 8, -53, 39, 1, -6, -11 },
    { 1, -3, 11, -94, -4, -6, -21, -10, -26, -17, -27, 54, 41, 6, 16,
      13 },
  },
},
{
  // 9
  {
    { 119, -16, -11, -7, -34, -7, -1, 16, 13, -1, 0, 7, 0, -6, -5, 0 },
    { 0, 71, 66, -7, -59, -10, -2, -22, -19, 0, 7, 47, 11, -2, -2, -15 },
    { -30, 6, 7, -39, -35, -43, 0, 62, 57, -2, 4, -20, 4, -37, -31, 31 },
    { -10, -32, -26, 18, 4, 8, -35, -8, -13, -27, 33, 64, 46, -54, -42,
      4 },
    { -8, -76, 68, 19, -11, -23, 10, 41, -41, 2, -8, 5, -1, -11, 30,
      -14 },
    { 26, 21, 59, 14, 49, 36, 4, -1, -16, -7, 9, -35, -11, -39, -36, 57 },
    { -10, -37, 18, -54, -19, 63, 21, -5, 16, -22, 41, 19, -48, 27, -29,
      -15 },
    { -8, -30, -11, -22, -47, -25, -26, -55, -37, -15, -8, -19, -9, 19, 2,
      77 },
  },
  {
    { 110, -35, -49, 5, -1, 0, 0, 13, 16, 1, -6, -11, -4, 1, 1, 2 },
    { 34, 58, 43, -38, -75, -24, 1, 16, 32, 4, 4, 19, 4, -14, -20, 7 },
    { 37, 22, 17, 26, 0, 39, -9, -60, -55, -19, 27, 62, 13, 2, -11, -16 },
    { 29, 31, 49, 6, 11, 5, 32, -5, -21, 23, -26, -40, -32, 50, 57, -30 },
    { -8, 81, -67, 55, -3, -29, -3, -9, 5, 5, -25, -4, 15, -6, 14, 0 },
    { 24, 6, 54, 44, 40, -36, -4, 36, -33, -11, 4, -23, 23, -54, -9, 37 },
    { -11, -40, -10, 19, -51, -72, 8, 21, -47, -29, 13, 24, -4, 17, 31,
      -30 },
    { 1, -26, 8, -20, -28, -13, -56, -53, -22, 33, -56, -14, 35, -14, 33,
      31 },
  },
},
{
  { 119, -26, -28, -6, -16, -7, 1, 13, 13, -2, 0, 1, 0, -4, -4, 1 },
  { 23, 96, 34, -12, -49, -34, -4, -21, -7, 0, 5, 30, 16, -3, -4, -6 },
  { 9, -55, 102, -37, -17, 2, 10, 10, -19, -8, 16, 14, -3, -5, -1, -2 },
  { 28, 18, 17, 8, 23, 86, -15, -65, -25, 2, -2, -2, -20, 36, 11, -9 },
  { -21, -1, -15, -23, -77, 68, -23, 24, 54, -7, 7, 7, -10, -9, -19,
    5 },
  { 0, -20, 17, 106, -45, 2, -9, 3, -16, -30, -18, 9, 16, -6, 22, -9 },
  { 14, 40, 36, 26, 10, 15, 53, 41, 23, 4, -16, -61, -51, -10, -3, 25 },
  { 10, 14, 29, 2, 26, -2, -97, 25, 11, 0, -1, -52, 36, -8, 2, 15 },
},
},
{
  // 10
  {
    { -93, 41, -73, -8, 12, -9, 2, 10, -15, -5, -2, 9, -4, 0, -1, 1 },
    { 57, 16, -49, -49, 76, -16, 5, -24, 25, -10, -4, 19, 0, -11, 15,
      -1 },
    { 51, 31, -26, 40, 1, -2, 2, 51, -69, 23, -16, 40, -12, 10, -16, 19 },
    { -16, 53, 47, 67, 49, -43, 10, -38, 10, 19, 15, 3, -3, -13, 0, -2 },
    { 23, 64, 2, -14, -13, -9, 4, 35, 19, -42, 48, -55, 8, 38, -34, 0 },
    { -17, 32, 57, -33, 3, 1, -9, 53, 18, -36, -16, 56, -25, -7, 47, 7 },
    { 24, 53, -19, 5, -53, 10, 8, -15, -1, 15, -22, -25, 10, -44, 59,

```

```

        -51 },
    { 0, 5, 21, -35, -25, -56, 27, -47, -61, -50, -34, 7, 15, 4, -20, 1 },
},
{
    { 109, 29, -54, 11, 10, -10, 1, 14, -17, 0, 4, 0, 0, 3, -4, 0 },
    { -36, 63, -47, 40, -73, 7, -3, 9, 12, -14, 12, -34, 3, -6, 12, -5 },
    { 0, -93, -55, 53, -12, -3, 6, -26, 22, -16, 2, 0, -10, -6, 8, 3 },
    { -42, -15, -39, 0, 28, -35, 1, 44, -80, 6, 25, -19, -4, 17, -29,
      -20 },
    { 2, 6, 26, 26, 0, -99, -14, 39, 44, -20, -8, 12, -20, 24, 8, 5 },
    { -32, 35, -48, 4, 28, 7, 10, -20, 4, -14, -34, 83, -25, 17, -23,
      27 },
    { 5, -12, -20, -65, -64, -47, -2, -39, -9, 48, 5, 16, -27, -9, -13,
      2 },
    { -5, -20, -35, -43, -11, 11, -24, 51, 34, 3, -70, -9, 51, 12, -17,
      -19 },
},
{
    { 118, -25, 4, -16, 34, -4, 2, -12, 9, 4, -5, 12, -1, -3, 4, 3 },
    { -14, -87, 52, -7, -34, 29, -5, -42, 32, 1, 2, -25, 11, -11, 6,
      -10 },
    { -12, 46, 97, -19, 30, -37, 5, -7, 33, -6, -2, 23, -9, -4, 14, 7 },
    { 43, 35, 36, 44, -63, -3, 0, 30, -4, -12, 35, -54, 8, 17, -15, -17 },
    { -6, -18, -13, 72, 18, -65, 27, -53, -5, -10, 32, 3, -22, -29, -5,
      -9 },
    { 6, 25, -30, -70, -50, -51, 20, -35, 11, 10, -14, -31, -2, -31, 12,
      -25 },
    { -1, -44, 27, -23, -26, -50, 17, 27, -60, 10, -17, 22, -24, 29, -51,
      20 },
    { 7, 35, 23, 5, -7, 40, 8, -60, -67, 4, -18, 16, 44, -34, -32, 4 },
},
},
{
    // 11
    {
        { -80, -85, 37, -12, 25, -18, -4, -11, 2, 3, -3, 13, -7, 3, -3, 1 },
        { -60, 22, -11, 11, -69, 57, 17, -47, 30, -5, -6, -2, -5, -19, 10,
          7 },
        { -55, 55, 27, -31, 30, 36, -6, 46, -45, 7, 9, -30, 35, -3, -2, -9 },
        { 40, -2, 53, -24, 45, 66, 43, -39, 0, 9, -5, 34, -12, -2, -7, 11 },
        { -32, 52, -2, 8, 17, -17, -5, 15, 29, -10, -22, 69, -34, 62, -22,
          2 },
        { 17, 5, 65, 11, -13, 1, -56, 15, 66, -3, -22, 6, 55, -12, 11, 8 },
        { 0, 38, 36, -43, -14, -44, -45, -51, -19, 27, 31, -1, -45, -15, 24,
          12 },
        { 17, -20, 29, -14, -57, 16, -19, -7, -32, 10, -12, -11, 0, 56, -57,
          -54 },
    },
    {
        { 102, -58, 44, -7, -13, 17, -1, -9, 7, 1, -1, -3, 5, -1, 1, -1 },
        { 47, 15, -69, -27, 74, -30, 14, -38, 11, 3, -2, 13, 3, -12, 4, -1 },
        { 9, 58, 77, 20, 10, -67, 24, -37, 6, -2, 7, -6, 10, -2, -1, -2 },
        { 37, 21, -18, 71, 0, -12, 1, 39, -68, 20, -5, 41, -23, -1, -12, 13 },
        { -23, -44, -13, 73, 18, 13, 9, -64, -12, 20, 3, -36, 38, 12, -15,
          1 },
        { 33, 54, -12, 23, 15, 34, -25, 30, 32, -15, 36, -62, 4, 42, -25, 5 },
        { -10, 11, 37, 19, 57, 40, -51, -4, 27, 48, -27, 13, -36, -24, 30,
          -11 },
        { 12, 41, -5, -27, -25, 25, 16, -2, -17, 62, -57, -7, 60, 28, 19, 9 },
    },
    {
        { 113, -32, -35, 3, 29, -14, 8, -6, -7, 1, 0, 9, -6, -2, -1, 1 },
        { 28, 12, 96, -56, 10, -25, 1, -24, 31, 19, -11, 16, -4, -12, 5, 3 },
        { -14, -102, 17, 1, -10, 51, -4, -45, 14, 0, -5, -13, 16, -13, 5,
          -6 },
        { -9, 23, 2, -2, 86, 64, -37, -14, -21, -3, -1, 46, 5, 3, 2, 11 },
        { 43, 29, 12, -17, -45, 75, -20, 43, 12, 6, -9, -30, 40, 24, 6, -1 },
        { -19, -22, -45, -89, -14, -2, 9, 20, -26, 41, -29, 33, 3, 4, -22,
          16 },
        { 1, 44, -52, -21, -7, 10, 7, -62, 44, 0, -18, -12, 20, -56, 31,
          -17 },
    },
},

```



```

    { -13, -24, -16, -9, 36, -3, -4, 58, 77, -6, -25, -7, -33, 16, 51,
      4 },
  },
{
  // 12
  {
    { 122, -4, -16, -7, -30, 0, 0, 2, 10, -1, 2, 11, -1, 1, -4, -3 },
    { 11, 31, 107, -5, -25, -2, 2, -17, -47, -8, 8, 19, 2, 2, 15, -5 },
    { -28, 29, -17, -82, -59, -4, -1, 2, 30, -8, 39, 35, 0, -3, -22, -7 },
    { 12, 104, -20, 0, 42, 3, 5, -48, 11, 13, -3, -17, 0, 18, -7, 3 },
    { -10, 21, 24, 63, -39, -15, -3, 19, 58, 52, -23, 24, 10, -11, -43,
      -7 },
    { 15, -13, 54, -50, 49, 2, 4, 29, 53, 10, 3, -50, -3, -15, -33, 23 },
    { -6, 7, -9, -33, -50, -24, 1, 0, -18, 46, -65, -49, -11, 4, 34, 41 },
    { -2, -4, 0, 3, -10, 94, 22, 11, -8, 59, 31, -3, -43, 19, 9, 2 },
  },
  {
    { 106, -24, -62, 17, 2, 2, -1, 6, 14, 0, -9, -7, -2, 1, 1, 0 },
    { 42, 50, 42, -56, -68, -9, 3, 8, 38, 20, -3, 13, -3, -4, -18, 4 },
    { 50, -9, 65, 5, 10, 24, 2, -29, -68, -29, 42, 30, 0, 2, 1, -7 },
    { -12, 72, -47, 52, -32, 8, -3, -34, -7, -31, 10, 47, 7, -1, -22,
      -8 },
    { 20, 47, 46, 67, 5, -26, 8, 22, -10, 1, -50, -43, 1, -1, 38, 3 },
    { -7, -13, -18, -12, -46, -60, 1, 46, -26, -68, 38, -19, -4, -12, 24,
      8 },
    { 6, -7, 28, 38, 34, -19, -15, 43, 41, -8, 32, 15, 32, -43, -66, 18 },
    { -9, -63, 17, 23, -60, 3, -26, -20, -10, -16, -70, 18, 9, -21, -26,
      9 },
  },
},
{
  { 124, -6, -14, -10, -25, -5, -1, 3, 7, -2, 2, 7, 0, 0, -2, -2 },
  { 7, 79, 83, -10, -27, -12, 6, -27, -31, -8, 5, 16, 3, 5, 9, -3 },
  { -5, 86, -84, -9, 12, -23, 5, -25, 16, 13, 0, 5, 5, 5, -5, -1 },
  { -26, -8, -4, -19, -81, -52, -2, 43, 35, 7, 14, 40, 16, -20, -21,
    -7 },
  { 9, 12, 7, 105, -5, -34, 2, 20, -6, 36, -44, -8, 9, -5, -2, 3 },
  { -9, 16, -20, 27, -61, 95, -15, 2, -6, 18, 4, 22, -27, 14, -2, -8 },
  { 6, 37, 29, -10, 14, 27, 4, 48, 64, 6, 2, -57, -21, -21, -44, 24 },
  { -4, 17, -28, 3, -25, -6, 13, 51, -50, -75, -14, -33, -24, -23, 31,
    18 },
},
},
{
  // 13
  {
    { 120, -24, -33, -10, -1, -12, 1, 8, 6, 3, 0, -1, -1, -1, -1, 0 },
    { 31, 81, 71, -36, -9, -36, -4, -12, -18, 1, 10, 21, 9, -2, -4, -2 },
    { -5, 84, -83, 38, -9, -17, -13, 14, -9, 8, -3, -3, 5, 1, -1, 0 },
    { -18, -20, -9, -20, -100, -50, 10, 25, 37, 2, 5, 9, 9, -16, -13, 3 },
    { 23, 7, 47, 98, -30, 10, -3, -1, 24, -41, -1, -23, 7, 0, 13, -2 },
    { -11, -27, -2, 34, 46, -102, 27, -19, -11, -8, -10, 21, -7, -4, 3,
      0 },
    { 1, -16, -21, 2, -43, 6, -27, -101, -43, -15, 3, 19, 8, 8, 13, -13 },
    { -1, -19, 5, 8, -21, -4, -4, 59, -98, -28, 31, -3, -18, 16, -4, -7 },
  },
  {
    { 126, -4, 8, -4, -17, -10, -2, -1, -2, -2, 0, 4, 1, 0, 1, -1 },
    { 7, 97, 55, 2, 44, 19, -6, -27, -21, 0, -2, -15, -9, 5, 3, 2 },
    { -1, -64, 82, 34, 53, -34, 1, 6, 2, 6, -10, -15, 1, -3, -3, 3 },
    { -16, 4, 28, 14, -68, -57, -17, -71, -23, 9, -1, 19, 11, 25, 9, -2 },
    { 5, 18, -10, 90, -18, 20, 7, -15, 74, 30, -2, -11, -4, -5, -19, 0 },
    { -4, -15, 44, 20, -56, 73, 43, 24, -50, 13, -6, -4, 10, -6, 12, 1 },
    { 6, -30, -31, -4, 9, 19, 12, -64, -26, 0, -15, -77, -50, -3, -4,
      19 },
    { -1, -21, 33, -42, 1, 35, 22, -42, 52, -4, 61, 37, -39, 5, 15, -6 },
  },
},
{
  { 122, 10, 17, -14, 24, -16, -1, -3, -5, -1, -2, 4, -2, 0, -1, 0 },
  { 0, -94, 74, 7, -4, 2, 0, -32, 33, 2, 1, 2, 0, -3, 3, 1 },
  { 31, -67, -85, 8, -56, 13, 8, 5, -4, 5, 6, -8, 8, 0, 0, -1 },

```

```

    { 17, 29, 43, 20, -52, 79, 8, 25, -3, 2, 10, -45, 33, -4, -1, -9 },
    { -1, 26, 20, -29, -82, -53, -1, -61, -31, -3, 2, -12, -7, -6, 3, 1 },
    { 9, 25, -9, 96, -6, -32, -8, -21, 48, 16, 40, -14, -20, -6, 11, -1 },
    { -3, -23, 26, 38, -7, -43, -14, 59, -69, 22, 5, -11, -5, 34, -41,
      -5 },
    { 2, 11, 15, -19, -45, -33, 6, 62, 58, -11, 10, 58, 29, 26, 14, 5 },
  },
},
}

```

```

Ist_8x8_Kernel[IST_SET_SIZE_8X8][STX_TYPES - 1][IST_8X8_HEIGHT][IST_8X8_WIDTH] =
{
  {
    // 0
    {
      { 103, -44, -48, -9, -12, -6, -4, 19, 24, -2, -2, 4,
        7, 2, -2, -1, 3, -7, -10, 2, 0, -1, 1, -2,
        -4, -1, 1, -2, 2, 3, -1, 1, 0, 0, 0, 1,
        2, 0, 0, -1, 1, 0, 0, 0, -1, 0, 0, 0 },
      { 28, 31, 52, -35, -70, -18, 0, -14, -23, -3, -7, 28,
        50, 12, -5, 0, -1, 7, 14, 3, -2, -2, 5, -12,
        -24, -1, 4, 1, -5, -9, -2, 1, -1, 1, -2, 3,
        8, -2, 2, 5, 1, 0, 0, -1, -2, -2, 0, 0 },
      { -47, -3, -12, -10, -37, 3, 1, 43, 62, 8, -1, 18,
        26, 4, 0, 2, 3, -35, -52, -9, 5, 1, 1, -14,
        -18, -2, 1, -5, 14, 26, 2, -3, 2, 0, 0, 9,
        10, 0, -2, -8, 0, 0, 0, -5, -5, 1, -1, 2 },
      { -29, -52, -33, 18, 12, 26, 6, 2, -19, 5, -3, 25,
        38, 3, 2, 2, -2, 16, 35, 8, -2, 2, 3, -34,
        -49, -10, 3, 3, -15, -31, -8, 4, 0, 0, -4, 18,
        27, 1, 8, 19, 5, -2, 2, -6, -9, -8, -2, 0 },
      { -25, -25, -13, -16, -43, -17, 11, 32, 20, 16, 9, -7,
        -5, 14, -1, 4, -2, 10, 29, 5, -1, 0, -3, 32,
        34, 2, 1, 0, -26, -47, -13, 5, 0, 2, 0, -33,
        -34, -2, 11, 28, 5, -4, -1, 19, 19, -9, 0, -7 },
      { 4, -78, 85, -4, 5, 18, 5, 25, -6, -17, -6, -7,
        -14, -12, 4, 5, -6, -24, -2, 10, -5, -3, 2, 2,
        6, 3, -4, 2, 17, 3, -5, 4, -2, 1, -1, 1,
        1, 1, -8, 0, 2, -2, 2, -1, -2, -2, -1, 0 },
      { -18, -40, -19, 15, -18, 1, 1, -32, -29, -16, 10, 15,
        33, 30, 1, 0, 8, 12, -4, 12, 2, 2, -1, 17,
        7, -9, 1, -2, 16, 43, 12, -3, 1, 1, -2, -33,
        -33, -5, -19, -44, -15, 4, -2, 19, 19, 23, 7, -6 },
      { 7, 4, 14, 80, -16, -36, 7, 8, 6, -3, -10, -56,
        15, 30, -4, 2, -4, -8, -4, -6, -2, -4, 9, 26,
        -26, -29, 5, -1, -3, -2, 4, 1, -2, 3, -3, -9,
        24, 17, 7, 6, -2, -1, 1, 4, -11, -4, 0, -2 },
      { -16, -26, -8, -21, -31, -38, 21, -8, 11, 15, -9, -26,
        -30, -9, 9, 1, 10, 40, 24, 12, 4, -1, 12, 1,
        9, 13, 0, -6, -11, 14, 9, -2, 2, -1, 4, 37,
        31, 2, -13, -38, -19, 2, -2, -32, -28, 19, 7, 13 },
      { 7, -25, 33, 4, 25, -12, 9, -42, 43, 26, -4, 19,
        7, 12, 3, -1, 1, 37, -27, -46, 6, 0, -8, -12,
        -1, -9, -3, -9, -43, 2, 28, -4, -1, -2, 7, 0,
        -15, -6, 30, 8, -11, 4, -4, 1, 14, -4, 3, 0 },
      { -12, -25, -19, 1, -26, -6, -26, -44, -27, -6, 6, 0,
        -1, 10, 9, 5, 7, -18, -26, -26, 1, 0, 15, 22,
        30, 21, 4, 9, 19, 6, 12, 8, -3, 2, 1, 15,
        8, -7, 14, 44, 23, 0, -5, -30, -27, -37, -16, 11 },
      { 11, 8, 1, 37, -53, 88, 4, -12, 8, 26, -13, -14,
        -10, -39, -4, -2, 2, 17, 1, -12, 2, -7, 12, 8,
        3, 16, 2, 3, -9, 2, 9, 0, -1, 4, -8, -9,
        -1, -8, -1, -7, -8, 1, 2, 4, 0, 4, 3, -1 },
      { -7, -3, -1, -59, 19, 28, -22, -20, 24, -33, 2, -37,
        21, -6, -4, -4, 11, 18, -7, 11, 11, -4, 18, 45,
        -22, -18, 3, -6, -12, 2, -1, -4, 3, 4, -18, -26,
        27, 23, 14, 3, 2, 1, 8, 13, -18, -2, 1, -5 },
      { 1, -6, -9, -39, 4, 6, 28, 15, -44, 64, 4, -14,
        -10, 7, -13, 6, -26, -18, -2, -39, 5, 0, -3, 9,
        -14, -25, 3, 18, 16, 12, 33, 3, -3, 0, 4, -11,
        16, 27, -3, 0, -16, 0, -1, 16, 1, -8, 5, -11 },
    }
  }
}

```

```

{ -8, -13, -3, 0, -18, -33, -23, -28, 11, 9, -12, -16,
  -21, -47, -10, 5, 27, 9, 4, 12, 4, 1, -18, -28,
  -26, 0, 9, 5, 25, 13, 2, 5, -1, 1, 10, -2,
  5, 8, -11, 13, 19, 0, 12, 46, 35, -24, -18, -33 },
{ 6, -1, 16, 2, 16, 7, -43, -12, 19, 78, 10, 5,
  10, 26, 3, -1, 28, 5, 11, 13, -14, -8, -3, 15,
  -1, -12, -4, -5, 23, -8, -35, 0, 0, -2, -5, -9,
  -1, -1, -36, -11, 22, -1, -4, -15, -16, 4, -12, 14 },
{ -4, 1, 1, 3, 20, -21, -56, 53, -24, 10, -34, -8,
  34, -27, 0, 4, 24, 18, 9, -27, 2, -3, 21, 6,
  1, 29, -2, -7, -8, 9, 30, 9, -4, 2, -15, -7,
  -11, -24, 1, -4, -22, -9, 3, -1, 1, -3, 8, 1 },
{ 3, 4, -1, 4, 26, -20, 61, -25, 31, 13, -17, 8,
  39, -30, 8, 5, -16, -18, 21, 10, 11, -7, 25, 19,
  2, 32, 2, 18, 28, -1, 5, -1, -2, 2, -24, -20,
  -3, -22, -18, 5, -3, -6, 11, 2, -9, -17, -7, 3 },
{ 3, 12, 0, -14, 3, 28, 30, 18, 6, -11, -38, -24,
  -3, 43, 4, -1, 33, 24, 15, 12, 15, 4, 6, -16,
  -1, -23, -16, -4, 18, 23, 18, 14, -2, -3, 8, 11,
  -25, -13, -7, 12, 6, -8, -13, -17, 19, -44, -32, 11 },
{ 7, 5, 10, 13, 2, -2, 22, 29, -9, 1, 89, 0,
  15, -21, -3, 1, 24, 33, -6, 10, 13, -1, -31, 1,
  -1, 13, 4, -8, -5, 20, 14, -3, 0, -1, -4, -10,
  4, -1, 2, 3, 1, 0, 2, -3, -25, -32, -19, 2 },
{ 0, 1, 4, 0, -2, 11, -37, -10, 42, -22, 27, -15,
  17, 5, -20, 16, -30, -5, 53, -22, 5, 9, -26, -8,
  11, -7, 11, 32, 28, -7, 32, 2, -4, -5, 19, 16,
  -7, 4, -18, -1, -27, -6, -13, -11, 1, -6, 13, 7 },
{ -2, -8, -3, -16, 7, 4, 12, -5, -15, 12, 5, -71,
  45, -6, -25, -2, -22, 3, -29, 1, -5, 0, -20, -11,
  -3, 29, 8, 1, -8, -6, -26, -6, 6, 4, 24, 24,
  -21, -27, -1, -2, 23, 9, -11, -11, 23, 20, -2, 5 },
{ -5, -5, 0, 6, 0, -11, -3, 6, 0, -2, -20, 24,
  -16, -13, -94, 5, -22, 28, -15, 22, -7, -8, 4, 5,
  -3, -13, 12, 22, 2, 12, -2, 3, 3, -2, -13, -18,
  -11, 6, 9, 5, 9, 3, -4, -19, -9, -5, -8, 37 },
{ 1, -1, 0, 0, -8, -6, -26, -8, -6, 25, 10, -16,
  -5, -9, 33, 5, -23, -12, -11, 58, 43, -12, 4, -19,
  -10, -21, -24, 24, -4, 6, 8, -16, 6, -6, -20, -3,
  -22, -12, 36, 10, -24, -1, 5, -13, 9, -10, 23, 24 },
{ 3, 3, 7, 9, 12, 11, -13, 11, 2, 17, -17, 17,
  6, 35, -7, -5, -23, 18, -9, 41, 31, 9, 9, 15,
  24, 37, 39, 12, -5, 11, -4, -18, 5, 1, 27, 21,
  26, 9, 8, -1, -14, -1, -7, 18, 0, -23, -2, -48 },
{ 1, 0, -2, 9, 0, -4, -1, 7, -6, 2, -20, 12,
  36, -55, 24, 2, -18, 8, 6, -7, 15, 16, -6, 17,
  40, -53, -1, -8, -9, 14, -19, -16, 0, -8, 19, 15,
  -6, 41, 0, -11, 19, 3, -19, -11, 4, -9, -24, 5 },
{ 0, -3, 1, 1, 4, 3, 9, -16, -5, 9, 2, 3,
  1, -8, -41, 11, 52, -62, 25, 7, 37, 0, -2, 11,
  -2, -1, 29, -46, -21, 0, 2, -17, -4, -4, 11, 10,
  -5, 1, 16, -4, -17, -4, -16, -9, 11, -2, 12, 12 },
{ 0, -3, 3, 2, -2, -4, -8, -2, 0, -2, 40, -8,
  -5, 4, -20, -34, -18, 2, -3, -18, 21, 18, 78, -23,
  7, -8, 1, -14, -5, -12, -11, -11, -14, -8, -30, 18,
  -6, 7, -28, -10, -2, -13, 12, 2, 32, -11, -19, -3 },
{ 0, -4, 1, 2, -4, -3, -2, 11, -5, -14, -14, 4,
  -16, 13, 2, 2, 2, 13, 9, -47, 78, -7, -20, -3,
  -15, 11, -1, 16, 12, -5, -26, -47, 12, 4, -10, -15,
  -4, -9, -13, -3, 25, 27, 9, 2, -16, 12, -2, 7 },
{ -5, -5, -5, -5, -5, -2, -15, -6, -1, 0, -17, -4,
  -8, 3, 10, -64, -42, -18, 14, 0, -9, -20, -41, -2,
  -8, 3, 22, -35, -20, 10, 6, -8, -6, 16, -4, -15,
  -1, -16, -8, -16, -11, -16, 18, 3, -15, -31, -34, 20 },
{ -1, -2, -2, 3, -3, -6, -2, -4, -5, -6, -1, 16,
  -9, -17, -6, -26, -5, 7, -29, 7, 23, 17, -7, 43,
  -40, 6, -47, -9, 9, -41, 11, 6, -1, -4, 37, 0,
  -3, -9, -34, -2, -22, -16, -40, -11, 5, -9, 2, -6 },
{ -1, -3, 3, 5, -1, -2, -2, 1, 8, 10, 5, 0,
  9, 2, -28, -57, 2, -18, 37, -1, 6, 16, -2, 1,
  10, 11, -66, 10, -1, 34, 1, 15, 10, -9, 0, 3,

```

```

    9, -3, 40, 4, 23, 14, -3, 1, -9, 21, -4, -29 },
},
{
{ 109, 7, -40, -24, 31, -26, -4, -1, -3, -1, -2, -11,
  17, -5, -7, -1, 2, -6, 8, -4, 0, -1, -1, -1,
  3, 2, -1, -2, -1, 1, 0, -2, 4, -1, 0, 0,
  0, 0, 1, -1, 1, 1, 0, 0, 0, 1, 0, 0 },
{ 38, -61, 73, -14, -13, 5, 4, -36, 46, -18, -5, 14,
  -22, 9, 2, 1, -4, -8, 15, -3, -4, -2, -1, 12,
  -16, 5, 1, 0, 1, -2, -3, 3, -1, 3, 0, -1,
  0, 0, 4, -5, 0, 1, 1, 0, 2, -2, 1, 0 },
{ -35, 44, 31, -21, 64, -35, 3, -34, 25, -4, -1, -20,
  28, -18, 2, 1, 4, -30, 36, -14, -1, 0, -7, 8,
  -4, 4, -4, 1, 0, 0, -2, -10, 15, -2, -3, 0,
  -2, -1, 8, -9, 5, 0, -1, -2, 1, 1, 2, -3 },
{ -31, -70, -63, -36, 27, 32, 4, -10, 26, 18, 19, -25,
  5, 14, 4, 3, -1, -10, 18, -6, 7, 3, 5, -9,
  -7, 10, 2, 2, 1, 0, -4, -1, 9, -9, 4, 2,
  0, -2, 2, -7, 3, 2, 0, -2, 4, 1, -3, -3 },
{ -9, -9, -44, 41, -11, -61, 24, -50, -3, 9, -1, 27,
  -28, 11, 7, 2, 22, -24, -2, 15, 1, 0, 1, 18,
  -31, 17, -1, 1, 1, 3, 1, 1, -19, 16, -1, 0,
  0, 6, 0, -11, 6, -1, 1, -3, 6, -12, 5, 0 },
{ -3, 56, -24, -58, -32, 15, 4, 7, 29, -28, 9, 9,
  -38, 23, -4, -3, -1, 12, 13, -27, 6, 2, -5, 24,
  -31, 14, 1, -3, -1, 0, -6, 22, -14, -2, -1, 3,
  0, -4, 9, -7, -3, 3, 1, 0, 12, -13, 3, 5 },
{ 12, 19, -12, 61, 56, 52, -39, -1, 36, 15, -4, 18,
  -36, -4, 2, -4, -17, 3, 1, -1, -8, -5, -10, 14,
  -15, -8, 3, 0, -1, -2, -6, 7, -6, 10, -6, -3,
  -4, 0, 8, -3, -6, 2, 0, 0, 2, -4, 7, 1 },
{ -2, -10, -27, 18, -47, -6, -29, 27, 30, -22, -2, 28,
  24, -41, 4, 4, 4, -26, 41, -11, 2, 2, -7, 23,
  6, -25, 9, 2, 1, 3, 10, -33, 27, -4, -2, 2,
  0, -9, 23, -16, 1, -3, 3, 3, -11, 3, 8, -11 },
{ 18, 32, 9, 17, -45, 32, -7, -34, -12, 46, 16, -42,
  -17, 10, 7, -5, 7, -28, 23, 0, -8, -3, 23, -25,
  -10, 11, 5, -4, -2, 0, 1, -18, 30, -16, 3, -2,
  1, 5, 2, -24, 15, 0, 2, -5, 3, 9, -11, -13 },
{ -6, -4, 7, -37, -6, -39, -60, -7, 13, 52, 33, -2,
  -28, -43, 16, 3, -3, 22, -24, 7, 3, -4, 15, 1,
  -1, -21, 13, 6, 1, 0, 11, 1, -13, 5, 5, 0,
  0, 11, -3, 2, -13, 5, 3, 8, -5, -6, 2, 1 },
{ 4, 9, 5, 22, -8, -30, 44, 41, 66, 46, 31, 12,
  12, 17, -34, 3, -14, 5, 0, -7, -4, -8, 10, -20,
  3, -2, -12, -1, 1, 3, -16, 6, 4, -16, -1, -5,
  -3, 3, -5, 9, -10, -2, -1, -4, 0, 10, -10, 2 },
{ -5, 13, -25, 11, -18, -12, -17, -58, 33, -31, -17, -12,
  -4, 21, 2, -1, -18, 30, 9, 1, -2, 5, 0, -22,
  44, -20, 9, 2, 1, 0, -8, 5, 14, 0, -3, 2,
  1, 5, -24, 36, -20, 3, 3, 6, -19, 28, -15, 4 },
{ 3, 22, -5, -36, -3, 42, 21, -20, 19, 9, 1, 46,
  12, -18, -14, 5, 5, -45, -26, 47, 8, 1, 0, -17,
  12, -3, 1, -2, -1, 4, 4, -25, -14, 31, -9, 0,
  -3, 13, -24, 8, 2, 0, 0, 2, -19, 6, 7, -2 },
{ -7, -1, 3, -18, -8, -10, -29, -1, -21, 35, -34, 19,
  30, 40, -31, 3, -49, 18, 35, 42, -12, 2, 0, 20,
  -18, 5, 7, 4, 3, 2, -21, 4, 8, 17, -11, 2,
  -3, 6, 8, -15, -9, 10, 1, -7, 0, 1, 10, -9 },
{ 1, -4, 0, 8, 1, 7, 10, -35, -38, -10, 49, 14,
  -8, -21, -58, 22, -49, -14, -5, -29, 29, -1, -1, 14,
  5, -7, 9, -2, 2, 10, -10, 6, 6, -14, 18, -1,
  4, -14, 15, 6, -13, 12, 0, 4, -8, -1, 4, 1 },
{ -8, -7, -5, -8, 9, -20, 2, 32, 4, -22, -44, -31,
  -63, -22, -42, 24, -17, -25, -16, 12, -21, 11, 15, -11,
  -9, 8, 4, -1, 3, 8, -3, -11, 23, 4, -5, 7,
  1, 16, -4, -7, 7, 11, -2, 4, -8, 18, -11, -17 },
{ 0, -2, -3, 18, -29, 3, -30, -6, 26, -8, -15, -46,
  42, -28, -27, 12, -8, -21, -15, -2, 8, 3, -10, -16,
  -7, 28, 2, -1, 2, 6, -14, 35, -36, 18, 0, 2,
  4, 1, -10, 7, 11, 3, -2, -7, 28, -30, 14, 19 },

```

```

{ 3, 0, -5, -3, -12, 16, 56, -12, 2, 37, -30, -41,
  -7, -28, -1, 6, 0, 18, 11, -11, -22, -7, -8, 49,
  20, -37, -1, -1, 1, 3, 2, -3, -14, 30, -4, 0,
  0, -3, 16, 10, -14, 1, -1, 8, -10, -13, 18, 8 },
{ -3, -6, -1, -15, -8, -1, -19, -8, -8, 39, -30, 30,
  -15, -11, -12, -8, 21, -9, -12, -42, -7, -11, -44, 5,
  32, 46, -29, 1, 2, 5, -17, 6, 30, -3, -9, -3,
  0, -13, 5, 22, 8, -6, -3, -18, 5, 21, 1, 11 },
{ 0, 2, 3, -1, -3, -11, -10, 3, 8, 10, -9, -16,
  12, 57, 11, 22, -14, -25, -52, -40, 4, -10, -39, -5,
  -13, -18, 34, 3, -2, 0, 15, -36, -4, 11, -5, -2,
  3, -12, 9, -4, 3, 9, -4, 14, -18, 0, 20, -12 },
{ 1, 5, 4, 9, 4, 1, -9, -5, 21, -4, 16, -18,
  -3, 12, -33, 20, 32, 50, -12, 28, 38, 13, -6, 27,
  33, 34, -8, -18, 5, -2, 11, -31, 7, 13, 6, -2,
  5, -13, 12, -29, 27, -4, -5, -3, -14, -9, 5, -18 },
{ 4, 7, -1, -1, 3, 6, 24, -12, 6, 19, -40, 26,
  7, -28, 20, 55, 5, 29, 1, 0, 22, 18, -10, -32,
  -10, 6, 48, -12, -2, -1, 2, 21, 3, -36, -6, 1,
  4, -14, 12, -19, -6, 11, -2, 2, 11, -4, -14, -10 },
{ 2, 0, 6, 0, -1, -8, 0, 12, -1, 19, -27, -13,
  -26, 7, 1, -31, -6, -19, 21, 15, 84, 35, -18, -12,
  -8, -30, -23, 8, -3, -2, -3, -6, -8, -9, 10, 6,
  19, -24, -7, 17, -4, -4, 1, -4, 0, -4, 10, 8 },
{ 2, 0, 10, 4, 6, -3, -10, 29, -12, 7, 15, 0,
  -19, 24, 11, 48, 25, -30, 49, 2, 8, -4, 8, 12,
  33, 19, 41, -17, 2, -4, -5, 8, -4, 22, 9, 1,
  -1, 4, -24, 35, -16, 14, -9, 0, -6, -12, 8, 21 },
{ -5, -4, 1, -2, 7, 5, -17, -10, -4, 13, -32, 22,
  4, 13, -53, -8, 29, 8, 19, -41, -16, 5, 37, -13,
  -1, -18, 0, -7, 2, 6, 41, -25, -41, -21, 14, 8,
  -1, -2, -20, 3, 9, -7, -3, 8, 1, -22, -8, 2 },
{ -1, -5, 0, -1, -2, -5, 7, 2, -11, -5, 29, -5,
  -21, -11, -20, -2, -4, 20, 43, 13, -20, -13, -70, -45,
  -7, -5, 4, 6, -3, 4, 36, -2, -14, 4, -39, -16,
  -5, -3, -14, -4, 19, -2, -1, 5, -1, -2, 18, 3 },
{ 0, -1, -3, -2, 2, -7, 11, 3, -3, -2, -8, 17,
  -13, -4, 27, 0, -45, 10, 11, -38, 16, -10, 15, -27,
  39, -1, -10, -6, 7, -7, -20, -17, -4, 37, 10, -8,
  3, 26, -25, -41, 28, -3, 3, -16, 23, -27, 15, -16 },
{ -3, 0, -1, -7, 4, -6, -4, -6, 2, -7, 0, 6,
  -15, 22, -17, -11, 22, -19, -12, 22, -12, -4, 11, -30,
  46, -31, 7, 0, 1, -5, -5, 22, 19, -10, -14, 4,
  -9, -5, 63, -6, -10, -4, 4, -4, 30, -42, 34, 0 },
{ -2, 1, -4, -1, -1, -6, 10, 1, 9, 9, -5, 3,
  -1, -3, 3, -32, -45, -13, -6, 9, -3, 9, 6, 20,
  18, 24, 31, -30, -11, -48, 63, 23, 13, -4, 9, 7,
  0, -12, 2, 15, 33, -19, 0, 13, 10, 4, -2, 9 },
{ 0, 6, -4, -7, -1, -3, -5, -2, 5, -2, 1, -2,
  -11, 6, 18, 36, -26, -11, -3, 33, -33, -13, -14, 17,
  20, -13, -22, 9, 10, 4, -15, -28, -28, -51, 25, 9,
  6, -42, -11, 0, 31, -12, -4, -17, 20, -9, -11, 22 },
{ 3, 4, 2, 3, 3, 2, 8, 5, 7, 2, -3, 2,
  -2, 4, 8, -8, -9, -2, 3, 2, 3, 1, 11, 6,
  24, 26, 18, 90, -23, 56, 31, 20, 1, 0, 13, 4,
  3, -9, -1, -6, 9, 6, -4, -2, -13, -9, 3, -14 },
{ 4, -1, 3, 4, -1, -1, -2, 0, -4, 6, 21, 2,
  -1, 1, 10, -1, -9, -10, -1, -16, -34, 96, -3, 1,
  20, 16, -9, 1, -1, -10, -13, -2, -24, -3, -35, 30,
  2, -11, -4, -15, -18, -3, -1, 5, -16, -4, 6, -14 },
},
{
{ 113, -35, -40, -16, 3, -16, -3, 8, 6, -1, -4, 1,
  3, 2, -4, -1, 2, 0, 0, 2, -1, -1, 0, 1,
  0, 1, 0, -2, 0, 1, 0, 0, 0, 0, 1, 0,
  0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 25, 36, 75, -67, -52, -28, 0, -6, 5, 10, -9, 8,
  21, 4, -5, 0, -1, 0, -5, -3, 0, -2, 4, 3,
  -1, 4, 1, -2, 0, -1, 0, 0, 1, 0, -1, 0,
  1, 1, -1, 1, 0, 1, 0, 0, 0, 0, 0, 0 },
{ -15, -100, 43, -34, 9, 42, 11, -6, 32, 6, -1, -1,

```

```

-2, -5, 8, 4, -3, -3, -3, 5, -1, 0, 1, -2,
0, 0, -2, 2, 1, -1, -1, 0, -1, 0, 2, -1,
0, -1, 0, -1, 0, 1, -1, 0, 0, 0, 0, 0 },
{ -30, -3, -52, -18, -49, -18, 13, 56, 62, 40, -5, -6,
1, 4, 1, 3, 5, -15, -20, 0, 9, 0, 2, 6,
3, 0, 1, 1, 1, 1, -5, -4, -3, -7, -2, 3,
1, 1, -1, 0, 1, -1, 0, -1, 0, 0, 1, -1 },
{ 1, -33, 29, 60, -48, -48, 27, 27, 3, -62, 15, 14,
2, 7, 2, 7, -3, -11, 9, 7, -10, 0, 1, -3,
-2, -4, 2, 0, 2, -2, -3, 2, -2, 2, 4, -1,
0, 1, 0, 0, -1, -1, 1, 2, 0, 1, -2, 0 },
{ -4, -8, -34, -2, -62, 52, -27, -31, -17, -12, 20, 47,
40, 36, 4, -6, 7, 4, 0, -7, -2, 3, 8, -13,
-18, -8, 6, 1, -1, 3, 1, 2, 7, 6, 1, 0,
1, -6, -6, -1, -4, -2, 2, 0, 0, -1, 1, 0 },
{ -27, -29, -21, -25, 36, -67, 27, -13, -38, 20, 4, 26,
62, 12, 9, 8, -3, 1, -5, -2, 3, 3, -1, -3,
-13, -1, -1, 4, 2, -1, 1, 0, 2, 3, 2, 1,
-1, -2, -2, -1, -3, -1, -1, 0, 0, 0, -1, 0 },
{ 1, 14, -2, -37, 21, 29, 19, 70, -26, -30, 37, 49,
-8, -34, -9, -3, -11, -11, -17, -15, -11, 7, -1, -18,
-3, 11, -3, -3, -1, -3, 1, 3, 4, 1, 1, -1,
-4, -5, 2, -2, -3, 4, -1, 2, 3, 2, 0, 0 },
{ -9, -23, -8, -10, -40, 1, 6, 25, -69, 26, -28, 3,
-37, -14, 1, 1, 7, 45, 36, 32, 16, -5, 2, 8,
5, 10, 2, 0, 1, 2, 8, -12, -16, -8, 5, 1,
3, 5, 5, 2, -1, 2, 1, -4, -3, -2, -5, 0 },
{ 15, -6, 32, 50, 5, 18, -17, 37, -28, 44, -48, 20,
20, 13, 3, -2, -14, -18, -53, -17, 10, -12, 5, 12,
-10, -7, -2, 1, -1, -4, 2, 5, 13, -9, -3, -1,
2, 1, -4, 2, -2, -1, 2, 3, 2, 4, 0 },
{ -13, -17, 7, -10, 12, -38, -102, 26, 8, -10, 19, 11,
-17, 6, 26, 0, 4, 32, -5, -12, -8, 5, 0, 4,
1, 1, -1, 10, 4, 0, 2, -2, 7, 0, -1, -2,
-2, 2, 0, -4, -2, -2, -2, -1, 1, 1, 4, 0 },
{ 1, 13, 6, -11, 20, 20, -14, 47, 6, -27, -17, -35,
48, 35, 5, -7, -9, 5, 30, 49, 16, -2, -16, -24,
-30, -25, -10, 1, -3, -4, 2, 1, -18, -15, 4, 4,
-1, 0, 7, 10, 7, -4, -3, 2, -2, -1, -4, 2 },
{ -9, -8, -13, -26, -8, 5, -8, 4, -41, -30, 8, -48,
-1, 34, 7, -2, -20, -45, -27, 13, -3, 7, 15, 43,
48, 18, 6, 1, -1, -5, 6, 18, 3, -9, 1, 0,
4, 3, -12, -16, -1, 4, 0, 3, 0, 6, 6, 0 },
{ 8, 6, 12, -1, 12, -1, 27, 12, -2, 34, 29, 8,
-47, 83, 23, -7, -21, -3, 29, -21, -27, 8, 8, -4,
-19, -9, -2, 2, -2, -6, 3, -7, -5, 3, -3, -7,
-2, 0, -9, -4, -3, 1, -3, -2, -1, 2, -2, -1 },
{ -7, 5, -9, -17, 14, -10, -3, -18, 19, -34, -52, 61,
-37, 22, 11, 0, -13, -32, 12, 11, 52, -12, -13, 7,
-4, 15, 0, 1, -1, -1, 0, 6, 3, -18, -3, 8,
-2, 9, 2, -1, -7, 5, 0, 2, 3, 1, 2, 1 },
{ 3, 10, -4, -7, 7, 12, 31, 6, 10, -34, -21, 2,
13, -3, 53, -16, 20, 50, -9, -36, 9, 2, 33, 49,
5, -31, -5, 3, -5, -1, -7, -17, -13, -14, -9, 2,
2, -4, -16, -8, 3, -5, -5, 1, 3, -1, 1, -2 },
{ -9, -8, -9, -22, -6, -7, 9, 3, -18, -25, -22, -33,
-31, 16, -19, 16, 30, 8, -26, -29, 11, -9, -1, -16,
-42, -28, -11, -5, 1, 0, 6, 26, 52, 35, 9, 9,
0, 2, 0, 17, 11, 2, -3, 4, -5, -16, -2, 0 },
{ -6, -5, 4, 2, 16, 3, -5, 10, 10, -12, -3, 8,
9, 35, -93, 17, 22, 27, 4, -14, 1, 3, 16, 35,
-4, 29, 0, -8, 5, -2, -12, -22, -16, -14, -5, -1,
1, -6, -15, -2, -2, 1, -1, -2, 2, 4, 2, -1 },
{ 7, 7, 8, 5, 4, 3, 18, -9, 3, 3, 39, 12,
-13, 26, 7, 4, 24, 39, -66, 50, 18, -15, -50, 8,
8, 0, -4, -1, -1, -2, -8, 2, -7, -4, 13, 7,
-6, 13, 7, -15, -6, -1, -2, 4, -6, -6, -6, 4 },
{ -4, -2, -10, -1, -9, -3, 7, -10, 10, -14, -4, -17,
0, -3, -11, -18, -94, 50, -25, -14, 6, -7, -3, -5,
-25, 27, 24, 2, -3, 1, 8, 6, -3, 3, 1, 3,
4, 6, 8, 11, -5, -5, 5, 5, 6, 8, -5, -4 },

```

```

{ -2, -1, -3, -4, -2, -8, -6, -15, 0, 5, 16, 17,
  -20, -21, -20, 8, -19, -10, -16, 37, -16, 23, 24, 40,
  -41, -57, -13, -3, 1, 5, 21, 28, -18, -32, -18, -11,
  4, -1, 3, 17, 18, 5, 2, -3, -13, 7, 11, -3 },
{ -5, 1, -9, -10, -2, -1, -1, -10, -6, -17, -45, 4,
  -12, 13, 10, 13, 10, -5, -37, 7, -55, 39, -8, -22,
  2, 8, -2, 5, 0, -7, -22, -35, -41, 11, 2, -12,
  1, 3, 33, 32, 11, 4, -2, 5, 14, 14, 2, -6 },
{ -1, -10, 1, -4, -8, -5, -7, -8, -18, -1, 34, -18,
  -6, -1, -6, -7, 5, -22, -7, -45, 38, -41, -9, -17,
  -3, -22, -7, -3, -2, 2, -3, -35, -38, -37, -3, 11,
  5, 25, 20, 26, 20, 8, 2, 2, 22, 22, 12, -3 },
{ -3, -7, 5, 1, 5, -9, 4, -1, -2, -2, 5, -8,
  -7, 0, -5, -94, 39, -14, -13, 18, 3, 15, 17, -2,
  -32, 26, 32, 9, -6, 2, 18, -17, 14, -11, -13, -8,
  2, -1, 7, 11, -12, -5, 1, 3, 2, -9, 0, -7 },
{ 6, 5, 2, 3, 0, 5, 4, -3, -4, 0, 15, -17,
  -2, -5, 32, 56, 12, 0, -7, 13, 10, 1, 34, -18,
  -43, 52, 10, 0, 0, -11, -29, -2, 16, -38, -40, -5,
  11, 7, -3, -3, -3, -10, 0, 5, -7, -1, 15, -4 },
{ 1, 2, -2, -2, -2, 6, -8, 0, -3, 1, 8, -5,
  2, -16, 16, 8, -4, -26, 14, 3, -15, -17, -28, 63,
  -56, 16, 14, -1, -1, -2, -24, -34, 8, 16, 52, 14,
  -16, -11, -1, 1, 10, 4, 5, 8, 8, 4, -12, -3 },
{ -2, -2, -4, -3, -5, 5, 5, 2, -1, -4, -17, -8,
  8, -3, 1, 4, 21, 2, 12, -36, -37, 6, -57, 11,
  -19, 12, 9, 5, 0, 8, 32, 36, -8, -44, -1, -6,
  7, 38, 18, -30, -24, -14, -4, -10, -12, 14, 8, 14 },
{ -2, -4, -5, 2, -11, -3, 1, -7, -2, -2, -6, -4,
  0, -6, -2, -30, -15, 3, -2, 5, -12, 7, -14, -5,
  -13, 21, -94, -9, -1, -1, -6, -25, 27, -11, -14, -8,
  2, 4, -20, -31, 23, 19, -3, 4, 8, 11, 11, 23 },
{ -3, -2, -2, -7, -2, -8, -3, -5, -6, -4, -14, -9,
  -16, -10, -8, 1, 0, -4, -13, 10, -9, -11, -8, -30,
  -15, -31, 30, 2, 0, -3, -9, -14, -18, -3, 1, 8,
  -4, -37, -62, -41, -35, -6, 1, 12, 20, 34, 32, 19 },
{ -1, 3, -1, 2, 4, -3, -1, -2, 8, -3, -4, 1,
  -1, 4, 14, 3, 16, 0, -13, 7, -19, -44, 44, -11,
  -13, 36, -31, -28, -6, 16, 52, 21, -40, 12, 32, 30,
  -12, -14, 2, 1, -9, 3, -5, -12, -6, 16, 1, -6 },
{ -6, 1, -7, -8, 6, 0, 0, 1, 2, -5, -11, 14,
  2, 1, -7, -9, -6, 2, 4, 22, -61, -74, 10, 0,
  15, -15, 17, 3, -1, -8, -13, -16, 26, -6, -26, 22,
  18, 41, -11, 9, 11, 1, 0, 2, -12, -17, -1, 2 },
{ 3, 4, 0, -2, -1, 5, 3, 1, 0, -5, 4, -1,
  -1, -2, -6, 19, -3, -12, -4, 6, 10, -14, 0, 12,
  -9, 3, -19, 86, 7, 14, 40, -31, -12, 41, -27, -9,
  8, 4, 5, -11, -7, -40, -11, -7, 3, -2, 2, 2 },
},
},
{
  // 1
  {
    { 98, 51, -42, -8, -12, -21, -30, 18, -11, -2, -2, 0,
      -3, 12, -14, -5, 2, 4, 0, -2, -1, 0, 0, -1,
      5, 0, -2, -1, 1, 2, -1, 1, 0, 0, 1, 0,
      1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { -52, 92, 1, 8, -42, 46, -11, -3, -8, 3, 2, 0,
      -14, 18, -15, -9, 10, -5, 1, -2, 1, 0, -2, -1,
      3, 2, -2, -2, 0, 4, -2, 0, 0, 0, 1, 0,
      1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 35, -35, -42, 10, -27, 73, 50, -34, 13, 1, 1, 6,
      -15, 8, -3, -13, 25, -23, 5, 2, 0, 1, -3, 2,
      -9, 13, -8, -3, 2, 3, -4, -3, 1, 0, 0, -4,
      4, 0, 1, -1, 1, -1, -1, 0, 0, 0, 0, 0 },
    { 33, 19, 67, -48, 54, 50, 16, -14, -30, -1, -6, -6,
      2, -15, -18, -17, -3, 9, -8, -3, -1, -3, 1, -2,
      9, -3, -8, -3, -5, 8, 2, -1, -2, 0, 0, 2,
      4, -3, 1, 2, 1, 0, 0, 0, 1, 0, 0, 0 },
    { -11, -44, -12, 5, 20, 36, -49, 44, -7, 1, 2, 3,

```

```

-27, 54, -46, -1, 3, 6, -4, 2, 0, 1, -2, -11,
29, -24, 9, 5, -16, 15, -7, 2, -2, -3, 3, 3,
-1, -4, -3, 5, -1, -1, 1, -1, 2, 0, 1, 0 },
{ 12, -18, 34, -8, -39, -3, -61, -69, 34, 1, -5, -1,
30, -2, -43, 2, 13, 4, 6, 2, 0, 1, 2, 10,
-2, 2, 12, 6, 0, -2, 2, 1, 3, 2, 2, -3,
-3, 3, -4, -1, 1, -1, 0, 0, -1, 0, 0, 0 },
{ 7, 13, -17, 59, 38, 43, -21, 4, 36, -30, 3, -23,
14, -40, -6, 7, -43, 17, 0, -5, -4, -3, -8, 17,
-6, -10, 2, -2, 8, -23, 21, -3, 1, 3, 0, 11,
-14, 5, -2, -1, 2, 3, 1, 3, -3, 2, -1, 0 },
{ -6, -14, -31, -43, -22, 33, -28, -15, -42, 28, 3, 20,
-5, -26, 30, 44, -38, 33, -6, 4, 5, 4, 5, -10,
10, -2, 14, -1, 20, -19, 1, 4, -2, 1, -4, 4,
-13, 11, 0, -3, -5, 3, 1, -1, -4, 0, -1, -1 },
{ 29, 1, 56, 2, -29, 20, -2, 21, 34, -14, -6, -9,
-14, 48, 58, 46, -18, -20, 1, 1, -3, -1, -5, -3,
-7, -18, 4, -7, -1, -11, -2, -2, 0, -2, -2, 4,
-5, 4, 0, 3, -3, 0, 1, 1, 0, 1, 0, 0 },
{ -1, -2, -6, -21, -36, 0, 45, 19, 26, 2, 1, -18,
57, 16, -16, -15, -14, 49, -31, 4, 3, 0, -2, -13,
35, -22, -5, 0, -10, 0, 16, -13, -3, 0, -11, 23,
-16, 0, -11, 11, -2, 2, 2, 4, 1, 3, 0, 0 },
{ 11, -11, 28, 17, -46, -15, 17, 6, -5, -8, 2, 21,
-56, -39, -23, -31, -53, -3, 27, 4, -3, 3, 2, 2,
32, -21, -9, 3, -1, 15, -3, 7, 1, 0, 3, 6,
-5, 6, -4, 3, -2, 4, 4, -1, -2, 0, 1, -1 },
{ 0, -19, 14, -7, -23, 18, -39, 43, 14, 9, -5, 5,
8, -12, 26, -53, -1, 23, -8, 0, 1, 0, -2, 2,
-19, 50, -53, -13, 14, -4, 4, 1, -3, 1, -2, -8,
21, -10, 15, -10, 4, 0, -3, 6, -1, 0, -2, 1 },
{ 0, 4, 5, -22, 1, 4, 21, 41, 28, 50, -22, 38,
13, -1, -44, 8, -17, -23, 17, -17, 5, -1, -5, 16,
-35, -3, 19, 8, 13, -36, 10, 5, -6, -3, 17, -12,
-11, 3, 1, -19, 14, -1, -1, 6, -11, -1, 0, 2 },
{ -6, -13, 1, -37, -27, -2, 6, 29, -19, -53, 21, -55,
-19, -13, -32, 16, 4, -16, -13, 17, 1, 4, -6, 26,
-19, 27, 24, 13, 18, -15, 23, -13, 4, 3, -3, 0,
4, 1, 6, -8, 2, -2, -6, -1, -1, -2, -3, 0 },
{ 11, -1, 29, 48, -2, -5, 20, 5, -16, -4, -7, 18,
-10, 8, -27, 33, 29, 41, -9, -8, -5, 2, 3, -32,
24, 51, 6, -6, 41, -26, -11, 1, 0, -2, -5, -1,
-13, 8, 9, -8, -5, 1, -2, 3, -7, 2, -4, -1 },
{ -11, 0, -13, -15, 19, -18, 6, -34, -5, -7, 6, -10,
-7, 53, -17, -2, -34, -12, 17, 5, 2, 0, -2, 11,
12, -7, -60, -38, 43, -25, 18, -5, 4, -4, 2, 7,
-2, 10, 16, -11, 4, 0, 0, 9, -12, 5, -3, -1 },
{ 3, -9, 10, 14, -11, 3, 0, 0, -49, -24, 6, 28,
15, 20, 20, -21, 24, 34, 46, -10, -4, 6, -23, 41,
-12, -15, 9, 4, -25, -23, 30, -13, 2, -7, 4, 8,
-24, -4, -10, -8, 20, -7, -2, 14, -12, 4, 2, 2 },
{ 7, -8, 13, 35, -12, -4, -4, -12, -44, 16, 0, -6,
2, 13, -6, -29, -18, -21, -58, 16, -2, 0, 29, -30,
-52, -23, 3, 6, -11, -31, 16, 6, -2, 6, -5, 16,
-9, 1, 6, -1, 3, 8, 2, 2, -3, -2, -1, -1 },
{ -4, 5, -7, -22, 20, -7, -8, -16, 31, 6, -11, -14,
-41, 16, 28, -53, 6, 1, -2, -8, 2, -2, -2, -16,
17, 21, 35, 41, 21, -26, 9, 2, 1, -1, -5, 22,
-42, 10, -19, 0, 1, 0, -3, 10, -11, 5, -4, -1 },
{ -4, -8, -6, 2, -12, -5, -4, -5, -16, -5, -76, -23,
-13, -12, -1, 10, 7, -9, -25, -71, 24, 18, -23, 4,
3, -8, -12, -12, -1, 9, 13, 1, -1, -8, 11, 18,
6, 1, 0, 14, 10, -11, -6, 11, 5, 9, -4, -1 },
{ 0, 4, -6, 3, 3, -3, 13, -25, -1, -10, -11, 0,
-5, 32, -3, 4, -53, 26, 4, -12, 6, 6, -8, -8,
-4, 32, 10, 47, -29, -11, 0, 2, 3, -4, 11, -23,
42, -53, 12, -10, 7, 2, -4, -1, 17, -11, 2, 6 },
{ 2, 5, 3, -2, 5, -8, 0, -5, 22, 41, 12, -2,
-51, -16, -10, 33, 27, 18, -8, 18, -1, -4, 0, 12,
-3, 0, -33, -2, -41, -20, 39, -22, 7, 7, -22, 30,
4, -33, -2, 11, 0, -3, -7, 14, 1, 7, -3, 2 },

```



```

{ 8, -2, 10, 30, 1, 1, -1, -2, -17, 53, 2, -10,
  10, 16, 10, -2, -17, -14, -16, 8, -4, -4, -13, 45,
  24, 17, 22, 20, 29, 49, 34, -36, 2, 4, -16, -2,
  18, 16, -2, -12, 2, -6, -9, 1, -4, 0, -4, -1 },
{ -1, -1, 1, 0, 1, 4, -8, 8, -17, -15, 11, 30,
  33, -2, -9, 15, -16, -50, 9, -5, 0, 2, -19, -16,
  3, 33, -36, 41, 2, 3, -5, -12, 3, 6, -16, 33,
  -24, -18, -33, 37, -17, 1, -6, 8, 6, 17, -10, -4 },
{ -4, 4, -5, -14, 6, -2, -3, -4, 17, -34, -3, 58,
  -3, 5, -1, -12, -9, -5, -8, -12, -8, -1, 4, -17,
  -10, 8, 46, -43, 4, 15, 37, -24, 2, 7, -32, 27,
  29, -3, 28, 13, -13, 7, -8, 0, 3, 6, -8, 0 },
{ -2, 0, -4, 1, -1, 4, 3, 4, -4, -9, -33, 3,
  7, 9, 4, 8, 0, -8, 5, 12, -12, -23, 71, 39,
  22, 28, 2, -5, -9, 6, 31, 52, -31, -9, 11, 18,
  -8, -15, -10, 9, -10, 10, 9, -7, 1, -1, 4, 3 },
{ -1, 3, -3, -4, -2, 3, -6, 2, 7, -28, -5, 30,
  8, -17, 10, 9, 27, -20, -23, 1, -6, -4, 11, -8,
  36, -41, -22, 52, 22, -4, 19, 0, -4, 1, -2, -3,
  14, -19, 32, -47, 19, -6, -1, 15, -22, -13, 3, 9 },
{ -3, 4, -3, -8, 7, -3, 7, -5, 12, -24, -9, 26,
  -22, 14, -8, 14, 0, 40, -16, -2, 1, -4, 14, 7,
  -49, -7, -32, 41, 7, 34, 14, -2, -7, 2, -2, -15,
  10, 57, -35, 8, -7, 7, 1, -6, -7, 0, 1, -4 },
{ 4, 2, 6, 8, 4, 5, 2, 7, 2, 8, 8, -7,
  9, 2, 1, -3, 10, 5, 33, 9, 70, 66, 29, 2,
  -6, -12, 2, 20, 15, -3, 6, 18, 8, -10, -21, 24,
  19, 5, 20, 5, -29, -7, -7, -13, 3, -6, -11, -6 },
{ 1, 5, 1, -1, 7, -1, -2, -4, 10, -16, 19, 29,
  -3, 0, 3, 3, -12, -15, -46, 10, 42, 44, 13, 12,
  21, 28, 1, -28, -33, 6, 4, -5, 15, -7, 21, -29,
  -28, 6, -26, -16, 25, -20, -12, 11, -17, 0, 7, 4 },
{ 4, 0, 5, 0, -2, -1, 2, -4, 6, 12, 63, 0,
  0, 0, 2, 6, 2, 3, -6, -26, -10, 0, -20, -17,
  -6, 7, 4, 4, 9, 18, 26, 47, -13, -27, 53, 40,
  21, 9, 9, 16, 24, -19, -2, 4, -5, 2, 6, 4 },
{ 1, 1, 3, 0, 1, -2, 1, -1, 4, -3, 8, 25,
  -11, 2, 5, -10, 12, 12, -34, 2, 1, 8, -4, 39,
  -5, -26, 2, -2, 54, -9, -5, -4, 16, 1, 29, -11,
  -13, -50, -12, 36, -10, -5, 10, -30, 45, -7, -2, -5 },
},
{
{ 89, -76, -38, -6, 17, 2, 13, 18, 12, -4, -1, 3,
  -4, -8, -1, 0, -3, -6, -2, 3, -1, 1, -1, 1,
  3, 1, -1, 0, 1, 1, 1, 0, 0, 0, 0, -1,
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 40, 44, -4, -6, -37, -93, 15, 22, 1, -2, -2, 0,
  20, 28, 14, -5, -16, -13, 0, 0, 0, 0, 3, -4,
  -8, -2, -1, 0, 2, 7, 4, -2, 0, -1, 0, 1,
  0, 1, -1, -2, 0, 1, 0, 0, 0, 0, 0, 0, 0 },
{ -55, -41, 15, 5, -4, -18, 79, 46, 11, 2, 1, 1,
  -4, -19, -10, -7, -31, -26, -3, 3, 0, 0, -1, 2,
  14, 13, 2, 2, 3, 8, 5, -3, -1, 1, 1, -5,
  -5, -2, -1, -2, 0, 1, -1, 1, 2, 0, 0, 0, 0 },
{ 37, 50, 7, -11, -22, 17, 18, 15, -1, -2, -3, 0,
  -15, -54, -67, 7, 13, 0, -1, 0, -1, 0, -1, 9,
  37, 33, 4, -2, -13, -16, -4, 0, 0, 2, 2, -9,
  -9, -1, 5, 7, 0, -1, -2, 1, 2, -1, 1, 0, 0 },
{ -22, 35, -90, 32, 50, -17, -4, 26, -3, 9, 9, -13,
  -26, -18, 11, -5, -1, 3, 0, -5, 3, -4, 1, 12,
  10, 2, 1, -1, 4, 0, -4, 0, 1, 1, -4, -5,
  -4, -2, 0, 1, 3, 0, 0, 1, 2, -1, -1, 0, 0 },
{ 28, 28, 16, -3, 48, 15, 42, -51, -28, -2, -2, -3,
  4, 0, 7, -51, -45, 0, 10, -4, -1, -1, 2, -6,
  0, 11, 4, 0, 24, 30, 4, -3, 3, -1, 1, -3,
  -12, -8, -5, -7, 3, 2, -1, -1, 4, 1, -2, 1 },
{ -8, -19, 10, -1, 41, -57, 37, -46, -15, -2, 1, -2,
  5, -13, -19, 18, 48, 36, 16, -2, 0, -1, 4, -5,
  -7, -2, 4, -1, -20, -32, -21, -3, 4, -1, 3, 10,
  9, 2, 8, 11, 4, -3, 1, -5, -5, -2, 0, 1 },
{ 14, 10, 53, -23, 29, -13, -14, 31, 4, -3, -4, -2,

```

```

-34, -54, 65, -6, 4, 5, -2, 0, -1, -3, -1, 19,
14, -24, -22, 5, 3, -9, -9, 2, 0, 4, -7, -3,
13, 7, -3, 4, 4, -1, 1, 3, -3, -2, -1, -1 },
{ -9, -31, 17, 20, 0, -35, -52, -12, -12, -8, -1, 1,
3, -6, 14, -9, -17, 7, 4, 2, -2, 2, 2, 10,
38, 62, 27, 1, -3, -3, -1, 2, 1, 1, -5, -28,
-43, -20, 12, 10, 3, 0, -4, 4, 13, -5, 0, 0 },
{ 8, 9, 29, -3, 65, -6, -18, 22, -17, -3, -3, -6,
4, 25, -31, 66, -29, -34, -8, -4, -2, -3, 0, -5,
-1, -8, 2, -14, -25, 10, 25, 3, -2, -1, 1, -3,
-4, 1, 8, -7, -11, 3, 0, 2, 3, 4, 2, -1 },
{ -18, 4, -21, -83, 24, -12, -10, -24, 58, 16, 4, 8,
31, -11, -2, -8, 13, -25, -26, -5, 6, -3, -1, -1,
11, 9, 8, 2, 6, 4, 13, 11, -1, 1, -2, -7,
-10, -6, -2, -3, -5, -4, 1, 3, 3, 2, 1, -1 },
{ 1, 9, -14, -9, 4, 27, 15, 25, -49, -2, 0, 19,
66, -15, 43, -8, 12, -15, -6, 0, 0, 4, -7, -24,
-9, 22, -1, -7, -39, -25, 2, 5, -4, 1, 4, -1,
-4, 3, 27, 21, -2, -2, -2, -2, -6, -5, 4, 0 },
{ -11, -15, -7, -14, -3, -19, -31, 19, -41, 10, -2, 8,
37, -54, -28, 6, -11, 34, 11, 0, 1, 5, -2, -22,
-16, -4, -13, 12, 37, 35, 8, 3, -1, 1, 12, 15,
6, 4, -26, -27, -9, 1, -2, -2, 3, 8, -1, 0 },
{ 1, 4, 19, -18, 24, 4, -14, 55, 9, 5, -3, -7,
-10, 21, -22, -46, 6, 26, -1, -1, -1, -2, 0, -11,
-35, 3, 63, 0, -3, -12, -17, -3, -3, -4, 8, 29,
-3, -29, 5, 5, 7, 0, -1, -15, -4, 0, 0, 5 },
{ -7, 0, -21, -65, -14, 10, 12, 4, -20, 19, -1, 5,
-27, 27, 14, 38, -47, 50, 23, 0, 5, 2, 1, 1,
9, 12, -13, -10, -3, -11, -27, -15, 2, -2, 3, -3,
-9, -4, 10, 9, 8, 5, -4, -1, 3, -4, -3, 2 },
{ 12, 15, 13, 20, 8, 17, 26, 16, 37, 8, -1, -23,
16, 4, 28, 39, 34, 27, 5, -3, -1, -3, -3, -21,
-27, 31, -18, 9, 20, 10, -2, -5, 1, -3, 3, -12,
-44, -32, -14, -20, -7, 1, -4, 7, 30, 3, 0, 2 },
{ -2, 10, -1, 17, 16, -1, -13, -1, 55, -58, 4, 33,
10, -7, -5, -1, -43, 16, 15, 3, 1, -3, -13, -35,
-6, 24, -32, 10, -4, -24, -7, -2, -2, 3, 18, 18,
18, 10, 0, 11, 5, 2, -7, -12, -16, -1, 1, 4 },
{ -4, -2, 6, -10, 21, 2, -5, 27, -20, -8, 4, 2,
18, 47, -30, -28, 22, -1, -1, 2, -1, 0, -1, 2,
32, -12, -54, 53, 25, -18, -16, -2, -2, -4, -9, -25,
-8, -1, -10, 18, 13, -2, 3, 10, 11, -14, -4, -1 },
{ -6, 5, -11, -21, -3, 0, 13, 4, -21, -74, 5, 53,
-27, 5, 8, 10, 31, -2, 12, 11, 1, -1, -19, -14,
9, -25, 31, -9, 15, 17, 16, 5, -1, 7, -1, -16,
-17, -19, -6, -9, -9, -3, 1, 10, 17, 2, 0, -3 },
{ 6, 4, 6, 22, 4, -1, 13, -2, -3, 21, -12, 49,
3, 7, 0, 12, -17, 42, -86, -6, 2, 2, -4, 4,
7, -6, 10, -8, 25, -27, 8, 26, 0, -2, -11, -8,
6, -5, -12, 4, -2, -7, 5, 3, -3, 2, 0, -1 },
{ -4, -6, -4, -13, -6, -12, -2, -8, -15, 4, -10, 1,
-55, -1, -7, -23, 4, 4, -28, 10, 2, -2, -1, -16,
-43, 26, -41, 33, -38, 9, 43, 19, 1, 1, 9, 5,
-6, -9, 23, 1, -21, -7, 0, 8, 14, -3, 11, -3 },
{ -2, 0, -2, -16, 5, 4, 5, 5, -19, -42, 14, -4,
2, 13, 7, 10, 12, -5, -24, -1, -1, 0, 18, 58,
-25, 61, -1, 10, -2, 9, -13, -8, 1, -5, -14, 5,
32, 8, -42, -20, 4, 3, 2, -13, -18, 15, -3, 2 },
{ -2, -2, 7, 1, 11, -1, 0, 20, 16, 3, 5, 7,
5, 23, -6, -34, 19, 43, 7, -3, -3, 2, -4, -5,
19, 11, -17, -59, -33, 32, 6, 4, 0, -2, -9, -23,
2, 54, -3, -24, -15, -5, 2, 18, 2, 13, 5, -9 },
{ -3, -2, -4, -4, -7, 1, 1, -1, 10, -45, 13, -34,
31, -21, -17, -5, -27, 31, -16, 0, -2, -5, 6, 44,
-36, -37, -7, -11, -13, 5, 5, 4, 4, -7, -20, -20,
-19, -10, 28, 22, 12, 2, 12, 20, 22, -14, -5, -5 },
{ 3, -6, 8, 3, -5, -5, -6, 0, -8, 12, 76, 23,
-10, -1, -8, -6, 3, -21, -3, -52, 0, -7, -19, 4,
-10, 2, -31, -34, 2, 7, -30, 23, 15, 3, 1, 20,
-10, -31, 7, 0, 9, -6, -1, -10, 11, 6, -2, 8 },

```

```

{ 3, 5, 3, 12, 8, -1, -3, 0, 15, 34, -14, 75,
  2, -9, -9, 6, 2, -9, 20, 11, -1, 0, 3, 28,
  -37, -1, 5, 25, -23, 39, -30, -21, -5, -4, -13, -24,
  -7, 7, 6, 12, 19, 11, 3, 7, 3, -11, -8, 0 },
{ 6, 3, 2, 2, -5, 6, 9, -1, 7, 5, 50, -4,
  5, 1, 11, 9, -14, 24, 13, -41, -4, -4, -10, -4,
  15, -11, 41, 73, -32, 2, 17, 16, 11, -1, -10, -7,
  2, 26, -12, -10, -16, -5, 7, 6, -3, 3, 8, -8 },
{ -2, 0, -5, 5, -2, 1, 0, -5, 3, -11, -18, 10,
  14, 0, 4, -9, -5, 9, -24, -4, -4, 6, 27, 13,
  41, -29, -14, 13, -53, 14, -22, -30, -3, 0, 21, 37,
  -15, -38, -8, -35, -8, 9, -13, -13, 21, 26, 12, 5 },
{ -2, -2, -2, -5, -3, -3, -1, -4, -8, -19, -21, -22,
  -13, -3, 1, 7, 2, -14, -42, -16, 0, -4, -19, -24,
  -9, -4, 11, 8, -5, 26, -55, 11, 8, 7, 25, 10,
  -45, 59, -17, 20, 18, 0, -10, -2, 4, -7, -5, 3 },
{ -2, -3, -2, -2, -3, 2, -2, 6, 1, -15, -29, 11,
  -10, -5, 0, 0, 5, -6, 7, -70, 14, 2, 81, -29,
  -4, 4, -2, -4, 10, -2, 9, 4, 4, -25, -29, 1,
  -4, 8, 10, 0, 15, 7, 8, 3, 2, -4, -7, -2 },
{ -1, -2, -3, -3, -2, -2, -7, -7, -6, 8, -4, 5,
  -5, -8, -5, -15, -21, -23, 14, -4, 1, 2, 0, -4,
  -30, -7, 4, -14, -15, -65, -9, -11, -6, 2, 5, -20,
  -8, 10, -61, -17, -6, -2, 10, 33, 49, 19, 5, -13 },
{ -2, -1, -4, 2, 0, 1, 3, 2, 3, -5, -66, -2,
  10, 7, -2, 0, -5, -1, 33, -39, 11, 11, -39, 37,
  -1, 2, -6, 3, -4, 2, -16, 61, 2, 6, -3, 7,
  4, -20, 0, -4, -23, -24, 5, 5, -10, 8, 11, -2 },
},
{
{ 105, -48, -38, -10, 18, -31, 11, 7, 4, -4, -2, 2,
  4, -3, 0, 0, 1, -1, 1, 1, -1, -1, 0, 1,
  0, 0, 0, -2, 0, 1, 0, 0, 0, 0, 0, -1,
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 24, 93, -5, -3, -23, -74, -20, 20, -12, -1, -1, -3,
  10, 0, 9, -4, 0, 4, 2, -3, 0, 0, -1, 2,
  1, -2, 1, 1, -2, 0, 1, 0, 1, 1, -1, 0,
  0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0 },
{ -43, -34, 17, 1, 10, -76, 76, 9, 4, 1, 1, 0,
  14, -19, 23, -8, 2, -11, 1, 1, 1, 0, 0, 3,
  -1, -5, 0, 1, 2, -1, 2, -1, -3, 0, 1, 0,
  0, 1, 0, -1, 0, 0, -1, 0, 0, 0, -1, 0 },
{ -31, -32, -41, 25, 24, -22, -69, 22, -4, 2, 6, 2,
  -1, -1, 71, 9, -14, 16, -2, 0, 1, 2, 0, -1,
  2, -13, 4, -3, 3, -1, -2, 0, 4, -1, 0, 0,
  0, 0, 1, -3, -1, 1, -1, 0, -1, 0, 2, 0 },
{ 21, -25, 87, -42, -24, -16, -38, -36, 16, -5, -9, 2,
  14, 16, 38, 7, -6, 4, 4, 3, -2, -3, 1, 4,
  -1, -6, 3, -5, 1, -3, -3, 0, 1, 1, 1, 0,
  0, 1, 0, -2, 0, 3, 0, 0, -1, 0, 1, 0 },
{ -20, -28, 4, 2, -1, -43, -34, 12, 7, 1, 1, 0,
  3, 13, -75, 72, -2, 6, 1, 1, 0, 1, 0, 2,
  -1, 18, -16, 17, -1, 4, -13, 1, 1, 0, 1, 0,
  0, 1, -1, 3, 0, -4, 0, -1, -3, 0, 0, 0 },
{ -1, 19, 5, 6, 83, -11, -13, -73, -30, -3, 1, -8,
  24, -21, -8, 5, 31, 0, 7, -6, -1, 0, -3, 4,
  4, -3, -10, 0, -1, 0, 4, -4, 2, 3, -2, 0,
  -1, 1, 2, -1, 0, -1, -3, 0, 0, -2, 1, 0 },
{ 10, 7, 45, -23, 47, 14, -7, 63, -14, 0, -4, -5,
  -9, -70, -11, 2, -32, 18, -11, -3, 0, -1, -2, -2,
  7, 1, 12, 2, 0, -3, -1, 5, 3, -3, -1, 0,
  -1, -2, 3, 1, -2, 2, 0, 1, -1, 2, 1, -1 },
{ -9, -22, 5, 1, -5, -18, -35, 5, 4, -1, 1, 1,
  4, -3, -32, -86, 16, 3, 0, 1, 0, 0, 0, 1,
  1, 6, 0, 67, 10, -13, 22, -2, 1, -1, 0, 0,
  0, 0, 0, 1, 0, -14, 6, 1, 4, -1, 0, 1 },
{ 4, 9, 19, 9, 48, -4, 12, 12, -17, -6, -1, 0,
  -12, 78, 2, -20, -66, -7, -1, -3, -2, 0, -1, 0,
  -13, 20, -30, 7, -1, 12, 0, 8, -3, -1, -1, -1,
  -1, 0, -4, 1, 6, -3, -2, -3, 2, 3, -1, 2 },
{ -20, 7, -37, -88, 16, 9, -5, 11, 27, 34, 2, -14,

```

```

54, 13, -2, -5, -7, -25, -11, -1, 10, 1, -3, -3,
3, -4, -8, -3, 6, 7, 2, -1, 0, -4, -1, 3,
-2, -1, 1, 2, 3, 4, -2, -2, 1, 1, 0, 0 },
{ 9, 7, 9, -9, 5, 19, 16, 26, -14, 5, -3, 4,
0, 11, 38, 37, 42, -7, -6, 0, 1, -1, 1, -1,
0, -9, -43, 70, -45, -14, 2, -9, -2, -1, 0, 0,
0, -1, 0, -3, 5, -20, 13, 1, 0, -3, -1, 1 },
{ -1, 0, 18, -1, 22, -5, -12, 38, 26, -2, 0, -11,
-32, 18, -2, -15, 66, -11, 1, 0, 0, -1, -2, -2,
0, 12, -47, -55, 28, -18, 11, -12, 0, 0, 0, 0,
-1, 0, -1, -2, 8, 9, 0, 3, 1, -4, 0, 3 },
{ 0, 17, -9, -5, 19, -9, -2, -26, 77, 6, 0, -19,
-62, -22, 15, 6, -16, -27, 16, -3, 1, 0, -5, 3,
3, 34, 11, 28, -14, 7, -12, -2, 3, 6, -2, 1,
-2, 2, 2, -1, 0, -9, 2, -2, -2, -1, 1, -1 },
{ -10, -7, -19, -54, -13, -10, 8, -21, -45, 24, -2, 9,
-55, -6, 7, -8, 7, 64, 6, 3, 7, 0, 2, 4,
-12, 32, -21, -2, 6, 14, -12, 2, -3, 1, 1, 3,
1, 2, -2, -2, 4, 2, 1, -4, 2, -1, -2, 1 },
{ 5, -3, 0, 21, -26, 3, -8, -13, 7, 1, -2, -4,
20, -46, 4, -9, -28, -10, -4, 4, 0, -1, 0, 0,
9, 3, -86, -9, 6, 54, 7, 3, 1, 0, 1, 0,
-1, 1, 3, 0, 16, -13, 16, -8, 2, 1, 1, 4 },
{ -1, -10, -1, -12, -15, -1, -12, 1, -59, 0, -2, 7,
-14, -11, 12, 13, -3, -82, 6, 4, 0, 1, 3, 1,
12, 51, 16, -1, 22, -2, 30, -14, 5, 0, 2, 0,
2, 0, 4, -1, 1, -12, -6, 2, -4, -1, 2, -1 },
{ -12, 4, -16, -35, 3, 4, 0, 6, 6, -102, 16, 33,
4, -7, 0, 1, -1, -1, 46, -6, -6, 11, 2, 0,
-20, -3, -8, 1, -2, 7, 2, -2, 3, 3, -3, 1,
-1, -2, -2, -3, 1, 3, 3, 4, 0, 1, 1, 1 },
{ 7, 12, -2, 4, 2, 10, 16, -4, 9, -2, -5, -2,
5, 0, 14, 26, -7, 12, 5, 2, 2, -2, 0, 1,
-4, -12, -7, 39, 111, -18, -12, 1, -3, 0, -1, 1,
0, -1, 0, 0, 1, -9, 4, -1, 0, 1, -1, 0 },
{ 3, 1, 5, 17, 0, 10, 3, 13, 9, -12, -5, -5,
52, 1, 16, -9, 21, 23, -6, -1, -2, -1, 0, 1,
-4, 85, 17, -3, 0, 13, -51, -6, -4, -2, -1, -1,
0, -1, -1, -7, 9, -27, -12, -7, 2, -4, -1, 1 },
{ 2, -2, 11, 0, 6, 2, -1, 19, -7, 3, -6, -22,
-11, 14, 0, -1, 30, -5, 25, 0, 0, -1, -1, -1,
3, -30, 24, 18, 12, 93, -4, -11, 0, 2, 1, 1,
0, -1, -2, 2, -3, 4, -48, -17, -1, -2, -1, -2 },
{ -3, -7, -6, -2, -12, 3, 5, 2, -16, -20, -10, -97,
7, 6, 0, 1, -14, 18, 48, 17, 2, -1, -4, -8,
36, 2, -4, -2, -8, -23, 8, -17, -4, 3, 7, 1,
0, -4, 2, 3, 6, -1, 11, 7, -3, -1, -2, 0 },
{ -4, 0, -7, -12, -2, -4, 3, -10, 1, -53, 4, -27,
-15, 7, 4, 5, 9, 7, -89, 8, -3, 3, -3, 8,
41, 5, 1, 6, 5, 13, 16, 34, -13, 0, 1, 0,
-1, 4, 0, 3, -15, -3, -13, 3, -3, 0, -3, 0 },
{ -4, 5, -7, 2, 7, 2, 4, 2, 20, 0, -78, 25,
8, 5, 1, 11, -5, 27, 9, 14, 8, -7, -6, 23,
2, 9, 7, -11, -3, 6, 68, -14, -10, 0, -1, 5,
-3, 2, 0, 2, -8, -39, -10, 9, -9, 0, -2, 0 },
{ 6, 2, 9, 1, 1, 1, 8, -1, 12, 12, 76, -7,
7, 7, 4, 14, 0, 29, -1, -26, -5, 4, 5, -29,
-6, 15, 15, -6, 6, 17, 62, -14, 21, -2, -4, -6,
1, -3, -4, -3, -2, -39, 15, 5, -9, 1, 0, -1 },
{ 3, 2, -2, 3, -8, 2, 3, -6, 8, 6, 20, 17,
5, -8, 2, 0, -15, 11, 10, -8, 0, 3, -1, -13,
29, 8, -31, 7, -6, -33, 7, -14, 2, -1, -4, -2,
-1, -1, -1, 4, -6, 12, -105, 13, -1, 0, -1, 2 },
{ 1, -1, -2, 4, -7, 1, -4, -3, -2, -4, -1, -41,
4, -14, 2, 7, 1, -5, -15, 23, 0, 1, -5, 1,
-96, 4, 0, 4, -4, -12, 26, 30, -2, 1, 4, 2,
-1, 0, 0, -4, 34, 6, -35, -7, 2, 0, 2, 1 },
{ 0, -5, 0, -1, -6, 3, 0, 5, -5, 8, -8, -11,
2, 1, 1, 1, 8, -6, 32, -63, 4, -1, 0, 26,
10, 0, -6, -7, -1, -11, 4, 86, 13, -4, -3, 2,
4, 0, 0, -16, -24, -27, -9, -23, 13, -1, 2, -3 },

```

```

{ -2, -5, -4, -3, -8, 0, 1, -3, -5, -11, -16, -21,
  -6, -1, -5, -2, -10, -1, -30, -74, -4, 0, 5, 49,
  -15, -14, 0, -1, 0, -3, -12, -59, 33, -4, -4, 0,
  5, 5, -2, -15, 24, -7, -7, 17, 2, 4, 1, -1 },
{ -2, -1, -1, -11, 2, -5, -4, -3, -8, -3, 7, 2,
  -17, 0, -9, -9, -4, -13, -5, 24, 3, 2, -3, -17,
  -1, -38, 2, -22, -3, -12, -39, 0, -4, 0, 1, 2,
  -2, -1, 2, 9, 19, -101, -17, 7, 1, 1, 1, 2 },
{ -3, -2, -4, -2, -2, -5, 2, -6, -1, -9, -51, 0,
  -2, 0, 1, 0, 2, 0, -12, -38, 1, -4, 4, -96,
  -3, 3, 1, 2, 1, 0, -3, 8, 37, 13, -1, 5,
  2, -3, -7, 26, 12, 7, 2, -4, -6, -5, 4, -1 },
{ 1, -1, 0, 1, -3, 1, 0, 0, -4, -1, -2, -17,
  3, -4, 1, -5, -1, -6, -4, 9, 17, -6, -13, -4,
  -37, 3, -15, -2, 1, 6, -15, -12, 27, 3, 4, 2,
  -1, 2, -2, 8, -103, -8, 0, 41, 1, -3, 1, 12 },
},
},
{
  // 2
  {
    { 83, -72, -48, -1, 23, 9, 2, 24, 16, -4, 1, 0,
      -16, -12, 2, 0, 1, -5, -3, 2, -1, 0, -1, 7,
      6, -1, 0, 0, 1, -1, 0, 0, -1, 1, -2, -2,
      0, -1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0 },
    { 43, 24, 7, -13, -70, -55, 5, 40, 18, -2, -3, 5,
      32, 27, -2, 3, -10, -36, -16, 3, 0, 1, 2, -4,
      -2, 2, -1, 1, -2, 4, 17, 4, -4, -1, -2, -4,
      0, 2, 0, -1, -5, 0, 1, 3, 0, 0, 1, -2 },
    { 49, 21, -25, 6, -1, -5, -23, -62, -24, 0, 0, 1,
      37, 37, -5, -4, 17, 38, 15, -3, 0, 1, 0, -34,
      -37, 0, 1, -1, 2, -1, -9, 0, 3, -4, 14, 18,
      0, 0, 0, -2, 0, -1, -3, -6, 0, 1, 0, 1 },
    { -33, -33, -14, 26, 28, 14, -7, 8, -12, -2, 2, -1,
      26, 40, 5, 8, -18, -46, -9, 2, -1, 1, -2, -26,
      -34, -7, 3, 0, -6, 28, 54, 13, -3, -2, 4, 6,
      -1, -1, 3, -15, -29, -2, 2, 4, 2, 4, 8, -3 },
    { 37, 19, 25, -5, 10, 19, -22, -37, -43, -9, -5, 13,
      8, -8, 11, 1, -16, -20, 1, 1, -2, 3, -4, 23,
      43, 0, -4, -1, 0, 20, 31, 11, 1, 2, -31, -51,
      -9, 4, 3, -2, -7, 0, 13, 27, 4, -3, -1, -8 },
    { -18, 50, -86, 33, 24, -29, -12, 4, 11, 3, 7, -8,
      -11, 17, 12, -4, -5, -9, -13, -4, 2, -4, 6, 25,
      13, 0, 1, -2, 5, -1, -7, 3, 1, -3, -21, -19,
      -5, 0, -1, 9, 16, 3, 7, 7, 1, -5, -9, -1 },
    { 27, 50, 2, -13, 24, -16, 0, 7, -1, 9, 1, -26,
      -53, -29, -15, 8, 5, -2, 14, -3, -1, -5, 8, 9,
      -15, -7, 0, 0, -6, 24, 34, 2, 0, 2, 14, 30,
      11, -3, 1, -34, -43, -7, -3, -10, 0, 14, 19, 0 },
    { -17, 2, -35, -10, -48, 46, -28, 2, 39, 15, -3, 3,
      9, -21, -11, 8, 47, 22, -4, 3, 3, 4, -9, -13,
      5, -5, -2, 0, -1, -2, 17, 11, -3, 3, -5, -23,
      -1, 4, -3, -19, -33, -13, 13, 29, 7, 6, 12, -14 },
    { 15, 31, 4, -6, 2, 56, -39, 0, 12, -13, 2, 8,
      0, -9, -9, -10, 4, -40, -50, -3, 0, -2, 2, 3,
      -2, 11, 1, 0, 1, -21, 7, 25, -3, -2, 13, 31,
      7, -4, 3, 28, 17, -1, -16, -36, -13, -9, -6, 15 },
    { -10, -27, 24, -30, 6, -28, -45, -18, 21, -3, 3, 6,
      -20, 43, 27, 6, 32, 8, -15, -2, 0, -2, 16, 46,
      -15, -27, 11, 0, -3, -9, 2, 9, 0, -8, -35, 9,
      22, -5, -2, -6, -6, -4, 15, -8, -13, 7, 4, 2 },
    { 18, 21, 33, 1, 35, 5, 4, 19, 26, -1, -10, -31,
      -13, 6, -3, 3, 11, -1, -15, 1, 0, -1, -10, -33,
      -41, -34, -5, -1, 5, -9, -6, 12, -1, 4, -6, -19,
      -4, 1, -5, 20, 24, -1, 27, 49, 20, -21, -24, -19 },
    { -12, -23, -6, 17, -24, -35, -14, -8, -32, -9, -2, -20,
      -20, -25, -14, 10, 32, 3, 21, 29, -2, 1, 3, -2,
      0, 5, -2, 5, 11, -3, 40, 35, 1, 1, 6, 6,
      -2, 1, 8, 42, 6, -15, -1, -6, 0, -46, -27, 8 },
    { 9, 1, 24, 62, 20, -19, -6, 18, -14, -23, -2, -4,

```

```

13, -2, -3, -6, 39, -3, -10, -4, -7, 1, -1, -2,
16, 15, 2, 5, -26, -58, -11, -1, -4, 3, 2, -7,
-2, 1, 8, 3, -33, -15, 1, 6, -3, 18, 39, -9 },
{ -11, 4, -14, -66, 52, -38, 23, -4, 4, 10, -2, 36,
35, -21, -3, -4, 17, 0, -18, -2, 5, -2, -23, -21,
12, 9, 0, 2, -4, -16, 11, 27, 1, 5, 2, -6,
-6, -3, 3, 10, -13, -19, 0, -4, -3, -6, 5, 5 },
{ 1, -19, 15, 23, -3, -21, 25, -64, 54, 6, -4, -27,
-5, -2, -19, -7, -10, -3, -40, -18, 0, 0, 22, 5,
14, 29, -3, -2, 9, 12, 8, 24, 9, -2, 11, -4,
-24, 4, -5, -8, -2, -7, -10, 3, 16, 6, 1, -3 },
{ -13, -13, -20, -30, -13, -16, -22, -16, -30, -16, 12, -7,
-11, -17, 6, -8, -19, -22, -19, -8, 1, 12, 19, -2,
2, 0, 13, 0, -8, -15, -13, -4, 2, 17, 41, 22,
26, 11, 5, 7, 9, 10, 6, 45, 19, 14, 13, -55 },
{ 3, 5, -14, 13, -20, 18, 74, -21, -17, -20, -1, 22,
-4, -3, -27, -4, 0, -4, -15, 2, 2, 3, -2, 18,
-21, -22, 8, 0, -6, -12, 4, 22, 6, -9, -33, 17,
42, 0, -1, 1, -1, -11, 35, 5, -20, 6, -1, -9 },
{ 0, 3, -8, -42, 2, 30, 29, 6, -16, 21, 11, -36,
0, 43, -19, -10, 9, -10, 16, 4, -1, 3, 46, 23,
-2, 24, -5, 2, -14, -28, 4, -13, -10, -19, -8, -6,
-27, -2, 13, 23, -19, -9, -3, 5, 21, -20, 10, -4 },
{ 10, 12, 8, 19, 1, 13, 23, -3, 23, 14, 10, 22,
2, 12, 68, -26, -8, 10, 14, 4, -10, 8, 5, 11,
15, -11, 7, -9, 13, -4, 11, 4, -14, 15, 29, 15,
18, 5, -9, 8, -27, -35, -17, 4, -1, -42, -11, -26 },
{ -3, -1, -12, -15, -26, 5, 16, -11, -11, -20, -14, -25,
-33, 1, 56, -25, -14, 2, -10, 5, 5, -2, -20, -21,
-16, -24, -14, 6, 3, -13, 1, 20, 14, 9, -1, -20,
-27, -12, 22, 18, -4, -7, -8, -10, 2, 27, 40, 30 },
{ -4, 7, -2, -13, 3, 6, 7, 17, -2, -90, 17, 24,
-17, 27, -8, 2, 12, 21, -5, 18, 1, -2, 3, 4,
3, 15, -7, -9, 13, 10, -1, 14, -1, -1, 11, -2,
-24, 2, -28, -27, -2, -11, -15, 3, 32, 2, -15, -2 },
{ 4, -1, 1, 4, -1, 7, 14, 32, -44, 25, 1, -23,
27, -11, 32, 20, 26, 25, -38, -25, 7, 6, 34, 9,
4, -14, -11, -1, 2, 14, -3, 36, 31, -17, 6, 1,
-1, 5, -16, -21, 14, 0, -22, -4, 3, 14, -17, -6 },
{ 3, 8, -5, -3, 4, 6, 29, -26, 22, -16, 4, 4,
-4, 12, 17, 68, 35, -25, 17, 6, 5, -2, 0, -5,
17, -23, -15, 10, -32, -6, 14, -15, 3, 7, 20, -10,
8, 6, 20, 8, 17, 33, -34, -5, -8, 6, -5, -12 },
{ 3, -4, 1, 11, -15, -1, -3, 2, -22, 40, 10, 53,
-53, 28, -12, 25, -8, 7, -23, -40, -8, -3, -23, -5,
-7, 8, -8, 1, -30, -11, -4, 7, 11, 18, 10, -3,
-16, 4, 7, -2, -6, -10, 0, 7, 28, -16, -11, 15 },
{ -5, -7, -11, -10, -7, -4, 5, -20, -16, -13, -12, -34,
7, -11, 8, 3, 12, -31, -16, -19, -20, 11, -7, -6,
6, -19, -11, -21, -11, -17, -16, -33, -28, 18, 4, -3,
7, 10, -42, -32, -19, -10, 12, -3, 2, -27, -28, 48 },
{ 0, 2, -1, -12, 3, 10, 3, 12, -3, 0, -39, -36,
-15, 46, -6, 9, -6, 19, -2, 7, 2, 19, -21, -11,
34, 37, 29, -6, -13, 3, 2, 21, 10, 38, 16, -5,
20, -10, 1, -8, 7, -10, 15, -2, -45, 10, -17, -8 },
{ 0, 4, -2, -2, 2, 3, -14, 4, 20, -24, 2, -9,
22, -24, 18, 50, -58, 34, 1, 6, -1, 13, 22, 6,
-12, 10, -1, 0, -49, -11, -2, 12, 0, -10, -8, 9,
2, 15, 24, 6, -11, -25, 21, 5, 7, -10, -7, 23 },
{ -4, -3, -1, -4, 6, -1, -11, 10, 6, -16, 8, -9,
7, 21, -48, 0, -24, 18, 7, -38, -7, 10, 9, -2,
28, -55, -26, 10, 26, 1, 1, 6, 20, 8, 13, -16,
28, 19, 0, 22, -12, -15, -25, 0, -14, -16, 29, 13 },
{ 1, -1, 2, 6, 6, -1, -5, -6, -6, 38, -2, 3,
0, 13, -19, 2, -15, -5, -7, 84, 17, 5, -4, 11,
12, -26, -27, -6, -5, -14, -15, 21, -20, 8, 12, 2,
10, 25, -15, -11, 8, -19, -13, 2, 22, 32, 1, 13 },
{ 3, -3, 5, -5, -6, 2, 0, -4, -3, 6, 94, -37,
3, 3, 5, 14, -11, 0, 0, 0, -5, -46, -39, -9,
6, 12, -4, 5, 2, -11, -2, 11, -1, 2, -12, -3,
5, 1, -15, -4, 0, -12, 4, -6, -20, 6, -5, -14 },

```

```

{ 4, 4, 7, 5, 7, 6, 9, 6, 9, 1, 39, -2,
  18, 5, 9, -14, 26, -4, 17, -6, 11, 28, 1, 18,
  8, 11, 10, 10, 10, 13, 12, 6, 21, 45, 13, 20,
  9, 21, 16, 6, 15, 21, 40, 17, 31, 14, 8, 56 },
{ 2, 2, -1, 0, 2, 4, 8, 0, -12, 3, -20, -6,
  10, 2, 7, 59, -5, 16, -34, 11, -6, -6, -22, 28,
  -11, 19, -8, 23, 61, 1, -1, -21, -20, 22, 2, 15,
  -5, -6, -11, 17, -12, 9, 13, 2, 9, -16, 39, -7 },
},
{
{ 83, -61, -61, 3, 25, 4, -1, 21, 21, -1, 1, -7,
  -20, -7, 1, -1, 2, -2, -3, 2, -1, -1, 2, 8,
  2, -1, 0, 0, 1, -1, -2, -1, -1, 0, -2, 0,
  0, -1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0 },
{ 33, 18, 17, -29, -75, -31, -2, 42, 41, -2, -3, 10,
  21, 10, -3, 1, 0, -36, -35, 1, 1, 0, 5, 10,
  6, 0, -1, 0, 0, -3, 13, 13, -3, -4, -11, -4,
  2, 1, -1, 2, -2, -2, 5, 1, -1, -1, -1, 0 },
{ -39, 5, 8, -14, 13, -12, 12, 37, 36, 11, 5, -21,
  -66, -24, 6, 2, -6, 1, 1, -6, 2, -5, 19, 54,
  21, -5, 0, 0, 1, -2, -26, -25, -2, -1, -16, -2,
  4, -1, -1, -1, 17, 15, -1, -2, 1, 2, -4, 1 },
{ -29, -12, -13, 18, 36, 18, -19, -7, -6, -18, 6, 19,
  4, 20, 7, 1, -3, -44, -44, -2, 1, -5, 5, 33,
  5, -6, 3, -1, -2, 5, 35, 35, 5, -18, -51, -20,
  3, -1, 0, 5, 0, 0, 24, 7, -4, -2, -7, 1 },
{ -45, -16, -13, 9, 7, 9, 9, 38, 38, 8, -7, -21,
  -15, -21, -7, -1, 19, -6, -8, 17, 0, 1, -16, -35,
  -18, 0, 0, 1, 0, -7, 32, 32, -6, 14, 27, 15,
  0, -1, 0, -3, -41, -40, 7, 6, 3, -3, 14, -4 },
{ 9, 78, -61, 34, 4, -42, -11, -10, 3, 8, 3, -36,
  -12, 24, -7, -2, 5, 2, 2, 1, 0, -4, 18, 5,
  -19, 1, 0, 0, 0, 4, 10, 2, -1, -8, -3, 9,
  -1, 0, 2, -3, -8, -2, 4, -1, 2, 0, 3, 0 },
{ 39, 15, 38, -29, 2, -12, 6, -19, -21, 3, -12, -9,
  -37, -22, -9, 2, 18, 27, 29, 19, 0, -6, -11, 16,
  -3, -11, 1, 1, 0, 5, 26, 26, 5, -4, -37, -13,
  7, -2, 2, -5, -27, -26, 40, 18, -2, -4, -1, -2 },
{ -33, -29, -34, 9, -29, -15, 18, 20, 11, 17, -9, 8,
  25, 20, -6, 11, 8, 26, 39, 14, 6, -7, -21, -15,
  -21, -11, 2, 2, 1, 13, -7, -18, 4, -13, -42, -26,
  -1, 0, 0, -6, 15, 18, 36, 22, -1, -3, -20, 1 },
{ 5, -11, 21, 41, 9, -43, 14, 46, -50, -19, -5, -8,
  -3, 5, 9, 5, -29, -43, 31, 19, -9, 5, -9, 2,
  18, 2, 0, 1, -6, 9, 21, -22, -14, 21, 9, -13,
  -1, 1, 3, 0, -12, 3, -4, 8, 0, -1, 8, -4 },
{ -20, -34, -17, -11, -29, -9, -8, -19, -10, 7, 17, 12,
  8, 33, 13, -8, 8, 15, 21, 0, -10, 8, 36, 42,
  15, 12, 4, 1, -3, -15, 1, -15, -19, -7, -5, 10,
  3, 0, 2, -9, -52, -31, -6, -12, -4, 6, 41, -5 },
{ 10, -23, 46, -8, 16, 10, -18, -3, 18, 19, 3, -42,
  -3, 40, -9, 10, -1, -20, 7, -9, -2, -1, 31, -11,
  -53, -9, 2, 1, -5, 21, 24, -26, -6, -20, 0, 20,
  0, -2, 4, -15, -10, 33, 5, 2, 5, 5, 4, -3 },
{ -15, -36, -4, -2, -29, -32, 7, -18, -19, -7, 4, -23,
  -15, -4, -2, 15, 12, 9, 6, 16, 13, -8, 20, 7,
  4, 2, -7, 2, 9, 24, 33, 43, 28, -16, 13, 4,
  -11, 2, 1, 10, 23, 10, -46, -28, 3, -11, -28, 18 },
{ 5, -13, 30, 26, 29, -50, 41, 17, 2, -21, 11, 17,
  13, 4, -19, -8, 8, 21, -19, -17, -1, -14, 7, -5,
  -27, 10, -1, 2, 0, -29, -31, 14, 16, -33, -15, 28,
  -1, -4, -1, 7, 2, -14, 12, -22, -3, 7, 14, 9 },
{ -9, 1, -17, -71, 50, -51, 25, -10, 1, 43, 3, 15,
  21, 7, 9, 4, -25, -11, -5, -20, 1, 6, -6, -12,
  10, 2, -1, 1, 0, 17, 14, 5, 6, 10, 10, -8,
  -4, 0, 3, -5, -5, -3, -4, 6, 2, -1, -1, -6 },
{ 13, 29, 5, 2, 19, 34, 16, 32, 11, 23, 2, 15,
  18, -3, 1, 2, 5, 5, 24, 11, 2, -21, -5, -1,
  6, -13, -15, 1, 5, 8, 17, -5, -3, -36, -25, -32,
  -23, -2, 2, 0, -18, 0, -54, -34, -2, 1, 25, 27 },
{ -11, 12, -28, -42, -5, 4, -21, 31, -35, -37, 21, 44,

```

```

-24, -12, -15, 7, 17, -6, 10, -3, 5, -14, -17, -3,
-29, -4, 3, 3, -6, 9, 17, -11, 10, -5, 5, 37,
14, -4, 7, -12, -5, 33, 3, -17, -5, 17, 14, -2 },
{ 1, 0, -19, 8, -25, 30, 53, -1, -35, 29, 3, -21,
13, -50, -36, 4, -12, -19, -13, -30, -3, 12, 31, -1,
12, 16, -4, 4, -6, 2, 13, -4, 7, -26, -5, 7,
5, 2, 3, -4, -9, 17, 21, 10, 3, 8, 11, -8 },
{ 9, 19, 2, -1, 9, 35, 40, 28, 2, -10, 25, 21,
7, 29, 16, 7, -11, 14, 16, 14, 1, 13, 30, 17,
-3, 7, 7, -9, -16, 14, 4, 13, 2, 0, 16, 25,
11, 1, -11, -26, -3, -24, 3, 5, 4, -19, -64, -13 },
{ -6, 3, -14, -46, 2, 23, 10, 10, -24, -9, -22, -35,
3, 21, -5, -20, -29, -20, 8, 44, 3, -1, 4, -3,
-15, 17, -3, 0, 11, -24, -18, 3, -19, -8, -11, 16,
-9, -4, 4, 44, 25, -18, 19, -22, -3, -13, 5, 20 },
{ -2, 1, 5, -3, 28, -17, -33, 12, -6, 23, -2, 1,
40, -27, -33, 9, 61, -4, 4, 28, -9, 16, 17, 13,
19, 13, 2, 1, -6, -25, 6, -12, -34, 1, 0, 11,
11, 3, -6, 6, 4, 15, 2, -8, -6, -17, -33, 3 },
{ 2, -2, 2, 18, -5, -3, 20, -49, 43, 6, 7, 23,
-9, -5, -33, -1, -26, -28, 18, 22, 3, -27, -33, 10,
12, 34, 3, 0, -7, 14, 14, -15, -15, 10, -3, 28,
2, -2, 3, -10, -7, 7, 14, -33, -14, -1, -12, 17 },
{ 5, -3, 9, 23, -9, 5, -39, 29, -25, 78, -19, 13,
-8, 5, 27, -10, -18, 12, -6, -7, 12, -7, -14, 7,
1, 20, 9, 2, -6, 2, -1, 17, 26, 0, 0, 28,
9, -4, 9, 1, 6, -4, 17, -28, -21, 7, -7, 1 },
{ -5, -8, -3, -3, -10, -13, -43, 5, -11, -1, 21, 6,
-11, -30, -10, -19, -40, 4, -7, -24, -27, -10, 5, -21,
-11, -2, -10, -11, -2, 12, -8, -14, -27, -30, -7, -7,
-17, 11, -14, -15, -14, -31, 3, 2, 25, -29, -34, 38 },
{ -2, 4, 6, 0, 13, -5, -34, 13, 30, -26, 15, -1,
25, -24, -1, -29, -31, 24, 26, 21, 14, 7, 28, -9,
23, 40, 3, 2, 16, 27, 12, 12, 19, -25, 4, -1,
-12, -5, 12, 9, 7, 11, 17, 13, -1, 11, 19, -36 },
{ -4, -1, -7, -2, 3, -11, 9, -12, 1, -21, -57, -10,
28, -41, 50, -7, -12, 8, -24, 28, 11, -5, 10, 9,
-14, -22, 8, -7, -10, 18, -4, -13, -18, -13, -8, 18,
12, -3, -6, -39, -15, 6, -4, -16, -8, 23, -7, 5 },
{ -4, -2, -2, -19, 6, 7, -16, 12, -5, -25, -21, -43,
10, 24, -27, 0, 5, -1, 5, -20, 9, -21, -17, -3,
33, 32, -2, -4, -43, -17, -15, 3, 34, 1, -3, -17,
-11, 4, 0, -44, -19, -4, -3, -7, -5, 28, -22, 11 },
{ 3, 2, 1, -5, -6, -3, 10, -12, 4, 1, 53, -22,
2, -20, 74, -15, 22, -20, 17, -4, -6, -20, -14, -20,
-1, 18, 15, -3, -26, -41, 10, -2, 2, -18, -5, -4,
-2, 3, 11, 13, -5, 21, 8, 3, -3, -2, -14, 3 },
{ 1, 5, 1, -1, 2, 5, 16, 11, 1, -18, -27, 3,
-5, 10, 25, -9, 25, 29, -14, -31, -22, 22, -1, 5,
2, 48, -5, 0, 22, 18, 29, -9, -9, 26, -1, 1,
-42, -10, -3, 3, -8, 29, 27, -15, 11, -18, 1, 37 },
{ 2, -4, 5, -2, -2, 1, 13, 5, -26, 20, 32, -9,
-2, 11, -10, -39, 17, 14, -50, 43, 10, -44, -18, 14,
-7, 21, 3, -3, 18, 18, -9, -8, -16, 8, 9, -11,
-21, -3, -13, -23, -2, 10, -9, 22, 24, 10, 3, -21 },
{ 1, 1, 2, -3, 2, 7, -7, 10, 0, -6, 47, -47,
45, -11, 7, 35, -7, 13, -1, -6, -33, -13, -27, 26,
9, -22, -13, -1, 22, 30, -6, 7, 9, 26, -5, 21,
12, 4, -11, -1, 10, -12, 22, -18, -9, 5, 18, 15 },
{ -3, -3, -2, -4, -6, -1, 6, 2, -3, -6, 0, -13,
-11, 5, -10, -18, -6, 15, -11, -15, -56, 7, -17, -11,
-3, -2, 5, 5, -3, 6, 9, 1, -26, -8, -15, -12,
-3, -38, 8, 2, 4, -2, -20, -27, -57, 1, -22, -60 },
{ -3, -2, -2, 0, 0, 2, 6, 2, 7, -2, -17, -7,
23, -5, -15, -82, 6, -8, 23, -24, -18, -12, -9, 33,
-15, -22, 23, -14, 4, 2, 16, -3, 34, 18, 2, 9,
27, 11, -3, 15, -2, 5, -19, 11, 7, -22, -4, 8 },
},
{
{ 106, -44, -50, -9, 0, -14, -3, 14, 14, -2, -2, 2,
4, 3, -3, -1, 3, -2, -2, 3, 0, -1, 0, -1,

```



```

-2, -1, 1, -1, 0, 1, -1, 0, 0, -1, 1, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 38, 69, 38, -34, -65, -47, -8, -11, -7, -1, -5, 7,
24, 10, -7, -1, 4, 8, 4, -1, 0, -2, 2, 4,
-1, 4, 4, -2, -1, 1, 0, -3, -2, 0, 0, 0,
1, 1, -1, 0, -1, 0, 2, 0, 1, 0, 0, 0 },
{ -39, -34, -9, -5, -57, -24, 24, 60, 54, 12, 1, 9,
25, 20, 3, 6, 0, -24, -21, 2, 3, 0, 2, -3,
-13, -11, 0, 1, 2, -2, -7, -1, -2, -5, 1, 1,
0, -1, 0, 3, 2, -2, 0, 2, 1, 1, 0, 0 },
{ -24, 18, -78, 39, -12, -20, -12, -33, -20, 8, 6, 15,
39, 36, 8, 0, 16, 30, 18, 0, 2, 2, 1, -5,
-19, -11, 6, 2, -1, 3, -5, -15, -7, 2, 0, 1,
0, -1, -3, -2, -6, -6, 2, -1, 3, 0, -1, 1 },
{ 4, 80, -50, 26, 12, -2, -3, 43, 20, 7, 0, -26,
-34, -17, -10, -4, -9, -29, -25, -8, 2, 0, -4, 4,
11, 2, -1, -3, -2, -1, 1, 10, 9, -1, -3, 1,
-2, 1, 4, 5, 6, 2, 0, 2, 0, -1, 1, -2 },
{ -21, -16, -24, -19, -21, -41, 10, -3, 11, 18, 4, -11,
-50, -36, 4, 6, 23, 39, 31, 10, 3, 0, -2, 16,
48, 37, 5, 1, 2, 3, -2, -13, -10, 1, 3, 2,
0, 4, 0, -17, -10, 3, 1, -7, -8, -7, -4, 2 },
{ -1, 15, -30, -46, -39, 90, -5, -7, 12, 25, 4, 30,
4, -25, -12, -6, 5, 12, -2, -3, 4, 1, 2, -14,
-7, 13, 4, -5, -3, 5, 5, -7, -4, -3, -2, 2,
0, -4, 2, 2, -6, 1, 2, -5, 1, 1, 1, -1 },
{ 14, 21, 17, 3, 25, 9, 50, 37, 10, 12, -6, -12,
-4, 1, -7, -6, 1, 43, 37, 0, -2, -2, -7, -15,
-27, -25, -16, -7, -1, -4, -20, -52, -40, -6, 0, 0,
-2, -3, 0, 3, 2, -2, -4, 2, 16, 9, -5, 7 },
{ -3, 0, -21, -11, -13, -1, 54, 32, -77, -17, 14, 25,
1, -2, -4, 0, -35, -29, 29, 22, -1, 2, 3, -3,
8, 9, 0, -2, 0, -3, 20, 21, -9, -4, 4, 0,
1, -3, -4, -7, 0, 7, 1, -1, -8, 2, -2, 0 },
{ 15, -13, 18, 86, -65, 25, 9, -10, -2, -28, -14, -13,
-23, -25, -16, -1, 2, 4, 12, 7, -6, -2, 4, -2,
-6, 3, 10, -4, 0, 4, 2, -2, -2, 4, 4, -2,
1, 0, 6, 11, 2, -1, 4, 1, 4, 4, -1, -2 },
{ -15, -11, -12, -25, 2, -32, -15, -3, -13, 8, -8, -17,
-7, -47, -37, 7, 4, 11, 12, 11, 6, 1, 0, -21,
-52, -27, 6, 2, 5, 11, 18, 19, 12, 5, 3, 2,
0, -1, 22, 52, 34, 5, 0, 1, 0, 2, 3, -9 },
{ 3, -2, -10, -22, -26, 20, 33, -34, -5, 19, -5, -48,
-38, 48, 25, -3, 0, -4, 1, 11, 7, -2, 9, 32,
1, -46, -19, 0, -2, -2, 2, 11, 16, 8, 0, 1,
3, 3, -7, 9, 27, 9, -4, 9, 0, -9, -4, -1 },
{ -8, -3, 0, -15, -11, 21, -73, 56, -24, -43, 8, -22,
-10, 26, -1, 1, 25, 14, 21, 32, 4, 2, 1, 13,
6, -11, -4, 1, 1, -4, -15, -5, -8, -12, 4, 2,
2, -2, -8, -2, 7, -1, -3, 8, 0, 0, 1, 0 },
{ 8, 12, 13, 12, 14, 14, 24, 18, 13, 12, -9, -11,
23, 9, -10, 16, 44, 22, 20, 26, 5, -4, -4, -5,
-10, 4, 5, -5, -3, -3, 16, 47, 33, 1, -3, 1,
-1, -7, -9, -10, -19, -17, -5, -25, -52, -37, -6, -5 },
{ -3, 4, -9, -15, 2, -2, 41, -3, -18, -56, 18, 14,
-5, -17, 0, 6, 55, 8, -44, -9, 4, 3, 6, 1,
-7, 1, 0, -7, 0, -9, -44, -18, 36, 17, -2, 1,
2, -2, 3, 7, 0, -2, -2, 19, 12, -15, -3, -2 },
{ 9, 6, 10, 13, -8, -9, -18, 6, -11, 27, 1, 53,
-49, -2, 64, 16, 26, -14, 5, 3, -7, -1, -16, -43,
-18, -19, -22, 3, 0, -14, -9, 10, -1, -5, 1, -1,
-3, 3, 18, 0, 0, 4, -7, 1, -13, -1, 6, 6 },
{ -6, 15, -15, -15, 3, -3, 8, -28, 58, -76, 9, 0,
7, -16, 33, -11, -21, -5, 32, 25, 4, 6, -9, -18,
-5, -11, -21, -5, -2, 5, 15, 16, -9, -7, 0, 2,
-3, -2, 5, -7, -3, 7, 1, -4, -6, 0, 3, 5 },
{ -2, -2, 4, 9, 1, -9, -5, -14, 16, 2, 31, 45,
-25, 11, -77, -8, 8, -3, 0, 7, -9, 0, -17, 4,
10, -37, -31, -6, 4, 6, 0, 7, 2, -5, -3, -2,
-3, -3, -27, -30, 13, 21, 3, 2, -6, -6, 2, 19 },
{ 6, 9, 8, 7, 15, 4, 11, -11, 23, 2, 44, 28,

```

```

-9, 23, -1, 2, 15, -16, 15, 24, -11, 3, 7, 25,
8, 17, 52, 10, -2, -2, -10, -6, -22, -26, -9, -4,
-1, 0, 11, 49, 34, 0, 4, -3, -10, -3, 5, -38 },
{ 1, 2, -3, -3, 2, 0, 2, -28, -3, 20, -5, -26,
17, -31, -10, 35, 23, -58, 9, 48, 12, -3, -4, 6,
-11, -8, 4, -2, 6, -17, -35, -5, -19, -27, 1, 4,
-2, -4, -16, -20, -23, -15, 2, 16, 12, 17, 12, 15 },
{ 5, 2, 2, 17, -1, 4, -9, 13, -2, 16, 8, 13,
48, -57, 34, 7, -5, 9, 11, -4, -8, -4, 15, 42,
28, -30, -29, 1, -1, -5, -9, -4, 11, -2, -7, -1,
3, -11, -21, 5, 39, 25, 5, 7, -8, -18, -9, -9 },
{ -4, -5, 1, 4, 2, -6, -5, -4, -1, 19, -27, 5,
5, 4, -5, -80, -16, -9, 9, 40, 6, -2, -5, -14,
2, 22, -12, -15, -3, -9, -40, -7, 23, -7, -7, 0,
-1, 1, 17, 10, -4, 11, 10, 30, -6, -37, -7, -8 },
{ -1, 10, -2, -2, 14, 4, 3, 7, 17, -8, -72, 47,
-10, 11, -7, 25, 2, -6, 35, 7, 21, 4, 15, 19,
14, -8, 22, 25, 10, 18, 3, 4, 24, 23, 3, 4,
0, -2, -4, 1, 4, -3, -4, 28, 35, 5, -8, 2 },
{ -7, -3, -12, -9, 0, 3, 3, -9, -2, -18, -53, -2,
7, 10, -23, 5, 24, -25, 19, -51, -21, 8, 10, 2,
14, 7, -26, -3, -2, -38, -20, 5, -31, -17, 6, -1,
3, 14, 21, 12, 17, 16, 2, -19, -39, -2, 9, -2 },
{ 1, 1, 5, 7, 8, 0, 13, 5, -5, 6, -10, -5,
16, 3, 11, -37, 67, -11, -12, 12, 9, -2, 7, -2,
9, 16, -19, -9, 0, 41, 47, 6, -13, -15, 7, 1,
1, 1, 4, -2, 14, 36, 11, 3, 25, 33, 22, 3 },
{ 4, 2, 2, 6, -2, 11, 4, 0, 7, 10, 24, -20,
24, 19, -11, 43, -6, 14, 14, 5, -1, 0, -33, -42,
32, 32, -19, 4, 12, -17, -15, 23, 17, 18, 12, 1,
-3, 15, 23, 7, 27, 30, 2, 33, 25, 12, 0, 0 },
{ 1, 3, -1, 8, 4, -4, 0, -5, 1, -4, -17, 18,
-10, 10, -7, 35, -23, 34, -46, 39, 41, -7, 37, 12,
-12, 29, -45, -4, 0, -22, 5, 3, -20, -21, -14, 0,
0, -14, 3, 15, -1, 16, 1, -11, -3, 3, 2, -9 },
{ 1, 1, -2, 0, -2, 3, 8, -2, -9, -9, -25, -7,
5, -5, 6, 1, -8, 15, -35, 15, 14, -2, -30, -24,
9, -11, 44, 57, -2, 10, -5, -14, -10, -30, -7, 5,
3, 10, -5, -21, 35, 31, 3, -9, -25, -19, 3, 36 },
{ -4, -2, -7, -4, -2, 0, 7, -1, -8, -6, -35, 10,
4, -2, 1, 11, -10, 26, -29, 17, -38, -9, -56, 7,
28, -23, -1, -38, -7, 18, -1, 12, -4, -32, 0, 9,
1, 9, 1, 9, -2, -33, -12, 8, 7, 11, 23, -39 },
{ -3, -1, -1, -3, 4, 0, -9, -1, 1, -5, -16, 2,
-5, 7, 9, 26, -1, -21, 0, 9, -2, -15, -40, 12,
-27, 20, 18, -60, -8, 3, 10, -33, -2, 34, 3, 5,
3, 3, -23, -2, 12, 45, 30, -24, -4, -17, -26, -3 },
{ -2, -1, 0, 0, 1, -4, 2, 2, 8, 0, -6, 5,
-7, -5, 9, -25, 2, 25, 0, -16, 11, 4, -2, 4,
-19, 10, 39, -29, -16, -65, 6, 41, -7, -6, 4, -1,
0, -14, -45, -15, 25, 5, -11, 33, 8, 26, 14, 11 },
{ -2, 5, -5, -5, 3, 1, 2, 3, 6, 2, -8, -3,
-7, 3, 6, 11, -8, 14, -6, 17, -94, 8, 27, 18,
-40, 28, -4, 18, 11, 19, -12, 16, 4, -4, 11, -4,
3, -1, -9, -10, -6, 23, 9, 21, 0, -1, 9, 21 },
},
{
// 3
{
{ 102, -31, -14, -32, 47, -12, -2, -13, 11, 0, -5, -16,
24, -2, -3, -1, -2, -7, 10, -1, -1, -1, -3, -5,
9, 2, -1, -1, -2, 5, 0, 0, 0, -1, 0, -2,
3, 1, 0, 2, 0, 0, 0, 0, 0, 1, 0, 0 },
{ -4, -22, 69, -15, -19, 7, -4, -33, 70, -11, 2, 3,
-21, 5, 2, -1, -6, -20, 47, -10, -5, 0, 2, 9,
-18, 2, 2, -4, -6, 22, -3, -2, -2, 0, 1, 7,
-10, -1, 0, 7, 0, 0, 0, 4, -4, 2, 0, 1 },
{ -47, 14, 13, -14, 32, 15, -1, -12, 6, 12, -5, -50,
57, -3, 1, 0, -2, -17, 20, 3, 1, 1, -4, -43,

```

```

45, 0, -2, -3, -11, 20, -3, 0, 0, 0, -3, -21,
20, 4, -4, 11, -2, 0, -1, -7, 6, 4, -1, -2 },
{ 1, -39, 48, -9, -11, 14, 4, -23, 2, -16, -9, 3,
4, 7, 2, 2, 2, 12, -51, 15, 5, -2, -8, -15,
23, -1, 1, 4, 18, -58, 23, 7, 1, 0, -1, -22,
26, -1, 6, -32, 12, 5, 0, -15, 13, -11, 4, -6 },
{ -33, 2, 0, -24, 50, -4, 2, -22, 2, 3, 14, -17,
27, -18, -3, 3, 3, -9, -8, -8, 5, 0, 2, 37,
-25, -2, -3, 1, 11, -33, 6, 5, 3, -4, -4, 49,
-37, 1, 11, -34, 12, 3, 0, 26, -17, -17, 7, 7 },
{ 0, -38, 50, -6, 3, 11, 11, 16, -34, 7, -3, 1,
18, -13, -2, -1, -4, 35, -35, 24, -9, 0, 7, 5,
5, -10, -3, 1, -2, 28, -19, -3, -2, -2, 2, 26,
-20, 0, -12, 44, -28, -3, -2, 28, -20, 18, -10, 14 },
{ 25, 82, 58, 1, 15, -29, -6, 28, 0, -31, -5, 15,
17, -15, -7, -3, -1, 20, 0, -21, -4, -4, -9, 14,
13, -7, -5, 1, 14, -6, -5, -5, -1, -2, -4, 3,
14, -4, 6, -4, -1, -2, -1, -2, 9, 0, -1, -1 },
{ -27, -6, 4, -44, 47, -24, 12, -3, -6, 11, 1, 7,
-24, -6, 9, 0, -2, 18, -10, 2, 4, 2, -17, 20,
-45, 23, 2, -2, 9, 6, -2, -4, -1, 2, 9, -37,
10, 2, -5, 23, -11, -4, 10, -46, 25, 21, -12, -18 },
{ 8, 17, 1, -65, -40, 10, -4, 32, -3, -1, 16, -54,
-16, 12, 2, -2, 0, 24, -3, -15, -3, 7, 22, -35,
-18, 8, 5, 3, 19, -7, -16, 2, 1, 5, 7, -8,
-25, 7, 17, -14, -5, 0, 3, 0, -15, -11, -1, -1 },
{ -6, 19, -34, -5, 2, 3, 15, -30, 42, -25, 5, -2,
-6, 18, -5, -1, 15, 14, -12, -2, 4, 5, 1, -4,
15, -5, -3, -6, 42, -29, 11, -17, -1, -3, 5, 4,
12, -11, 1, 35, -36, 0, -5, 27, -13, 39, -33, 21 },
{ 14, 44, 16, -10, -10, -29, -7, -29, -11, 26, -8, -11,
-13, 4, -3, -2, -3, -26, -16, 58, 8, 4, 8, -16,
-14, 5, 2, -3, -25, -7, 51, -4, -2, 2, 11, -7,
-21, 4, -24, 16, 18, 2, 3, 8, -25, 12, 4, 6 },
{ -12, 3, 0, -49, 23, 4, -5, 13, -8, 16, -5, 32,
-32, 41, -1, -1, -14, 5, 4, -3, 6, 1, -2, 3,
21, 8, -10, 0, -16, 11, 0, 3, 5, -6, 3, -2,
44, -26, -8, -9, 13, 2, -21, 47, -6, -20, 19, 36 },
{ 10, 19, -11, -28, -17, 69, -19, 18, 15, 7, 13, -4,
13, 11, 0, -8, -9, 8, -6, 22, 5, -11, -28, 37,
2, -2, 4, -7, -6, 1, 25, 1, 0, -2, -34, 31,
4, 1, -18, 18, 14, -2, -1, -18, 29, 17, 8, -20 },
{ -5, -5, -12, -47, -60, -53, 16, -32, -22, -9, 12, 5,
28, -12, 2, 8, 14, -24, -5, -6, 0, -4, -9, 24,
16, -4, 1, 5, -18, 4, -18, 7, 4, -2, -18, 16,
18, -4, -8, 4, -16, 7, -8, -1, 24, -3, -5, -2 },
{ 17, 19, 25, 9, 3, 31, -8, -28, -40, 55, -10, 1,
-11, 28, 1, 5, 23, -30, 9, -7, -10, -8, -3, 5,
-6, 19, 0, 9, 13, -8, -22, -1, -3, -1, -11, 8,
6, 3, 37, -15, -29, -1, -8, 1, 13, 14, -38, 0 },
{ -8, -32, 9, -13, -12, -18, -20, 55, -6, 18, 3, 8,
13, -19, 3, 2, 18, -22, 31, -13, 20, 2, 3, 5,
8, -8, -6, 7, 8, -19, 38, -11, -6, 5, 6, -1,
8, -10, 15, -5, 27, -19, 9, 6, -4, 53, -29, 15 },
{ 1, 5, -7, -1, -12, -22, 8, 3, 46, 85, 8, 13,
6, -26, -8, -2, -14, 29, -6, 22, -19, -6, 9, 3,
16, -30, 1, -11, 25, 1, -8, -2, -2, -1, 5, 2,
8, -15, 19, -10, -4, 1, 5, -5, 5, -15, 7, -9 },
{ 0, 15, -11, -35, -2, 46, -24, -17, -5, -9, -23, 39,
-1, -45, -6, 1, 1, -19, -7, -9, -1, 14, -1, 3,
8, -41, 4, 11, -18, 1, -11, 0, 3, 6, 36, -22,
-10, -13, -1, -7, -16, 4, 32, -13, -27, -7, -16, 1 },
{ -12, -6, -3, -19, 15, -20, -1, -1, -6, -3, -33, 23,
-22, 27, -20, 4, 0, 5, 6, -13, 9, -18, 3, -25,
27, -32, 13, 5, 2, -1, 4, -2, 1, -1, -33, 21,
-28, 5, 3, 2, 3, -2, -19, -8, -24, 13, -4, -72 },
{ -3, 1, -11, -9, -22, -2, 19, 10, 16, 8, -79, 14,
31, 4, 15, -1, 5, 9, 5, 5, 3, 5, -54, 6,
-11, 31, 2, -4, 14, 3, -3, -5, -1, 0, -8, -11,
-17, 28, 14, -4, 4, -7, -4, 7, -26, -9, 8, 19 },
{ -10, -6, -7, -2, 13, -17, -38, 40, 37, -18, -1, 1,

```

```

-3, 3, 11, 11, 2, -22, -24, 41, -34, -6, -2, -3,
1, 13, 1, 26, -32, -3, -4, -25, 1, -7, -9, 5,
-2, 12, 11, -17, -36, 9, -15, 4, 10, -20, -37, -4 },
{ 5, 10, 5, -12, 7, 22, 88, 28, 16, 1, 17, 34,
18, 16, -11, 4, 35, -28, -8, 7, -4, -5, 30, -7,
1, 11, -3, 13, -14, 2, 4, -7, 0, -7, 18, 0,
-9, 13, 1, -3, 6, -4, 8, -5, 6, -2, 5, -11 },
{ -6, -2, -8, -12, 0, 3, 6, -21, -31, -38, 2, 8,
1, 3, -3, 5, -4, 18, 38, 56, -36, 7, 7, 1,
3, -11, 7, 1, 26, 24, 15, -29, -9, -1, 3, 7,
3, -5, 49, -10, 19, -27, 10, -2, 15, -6, 4, 3 },
{ -1, 7, -6, 2, -2, -9, -7, -19, 16, 0, 12, 18,
24, 39, -10, 20, -47, 27, -20, -7, 14, -3, 9, -1,
5, 5, 16, 24, -28, 12, -18, 26, -16, 9, 9, 3,
-13, 21, 12, -21, 25, -24, 25, -8, 6, 31, -32, 22 },
{ 0, 7, -7, -12, 3, 12, -15, -18, 3, 1, -7, 31,
-5, -50, 42, -4, 2, 11, -4, -10, 4, -25, 43, -28,
2, 21, 22, 1, 7, 3, 4, 4, 3, -16, -5, 5,
-4, 42, 3, 6, 14, -2, -41, 15, 30, 15, 10, 11 },
{ -5, -9, 3, -6, -6, -8, -40, 4, -11, 2, 12, 41,
44, 40, -12, -16, -12, -3, 19, -1, 9, -9, 28, -3,
-3, 20, -3, -36, 25, -15, 9, -15, 13, -2, 22, -2,
-14, 20, -12, 15, -27, 22, 22, -10, 4, -25, 8, -4 },
{ -3, -3, 0, -2, 4, -4, 20, 11, -4, 3, -60, -28,
-26, 0, -20, 2, -22, -8, 5, -12, -6, -4, 31, 9,
10, -16, 25, -8, -3, -15, 10, -18, 8, 13, 2, 30,
0, 10, -21, 3, -9, 4, 30, -11, 50, -10, -9, 35 },
{ 5, -1, 4, 1, -4, 7, 29, -5, -16, 11, 33, 7,
3, -9, 22, 7, -53, 0, 6, -33, -15, 2, -28, -12,
-1, -5, 22, -13, -12, -13, 24, -61, -1, -3, -13, -9,
-6, 5, -17, -4, -6, -18, -16, 3, -16, -21, -29, 0 },
{ 1, 1, -5, 2, -15, 11, 11, 0, -1, -8, -35, -15,
18, -15, -19, 6, -49, 2, 5, 5, 9, -5, 33, 15,
-24, 24, -37, -9, -6, -3, 1, 0, 3, -34, 27, -3,
21, -29, 2, -11, 1, -1, -26, 17, 1, 9, -25, -44 },
{ 1, 1, 2, -3, 6, 3, 24, 18, -4, -14, 17, 13,
-14, -23, -18, -4, -43, -31, 13, 39, 50, 2, -18, -19,
0, -5, 7, -26, 20, 0, -13, 37, 7, 13, -27, -2,
-7, 7, 22, -7, -18, 25, -13, -3, 3, 0, -17, 20 },
{ 4, 5, 4, 6, 3, -2, 13, 6, -1, 0, -10, -6,
16, 27, 81, -5, 7, 20, 26, 21, 39, 0, 10, 8,
-15, -40, 30, 13, -11, -10, -3, 10, 7, -11, 13, 1,
2, -25, -4, -6, -14, 16, 2, 2, 5, -9, -13, -9 },
{ 2, 0, 2, 4, -2, 3, -9, -13, 7, 7, 3, -1,
-3, -14, -29, 5, 27, 28, 2, 7, 82, 9, 3, -7,
-3, 10, -16, 26, -13, 10, -3, -50, 6, 12, -9, 6,
2, 2, 5, 0, -8, -29, -3, -1, 13, -34, -5, 0 },
},
{
{ -89, 1, 28, 8, -54, 31, -2, -20, 20, 3, 3, 3,
-35, 23, 6, 1, 0, -18, 13, 3, 1, 2, -1, -17,
10, 3, 2, 0, 1, 1, -11, 6, 3, -1, -6, 3,
1, 1, 0, 1, -5, 2, -2, 0, 1, 0, -2, 0 },
{ 7, -45, 15, -3, 26, 9, 1, -63, 28, 5, -3, -7,
35, -6, -5, 5, 8, -55, 24, 6, 1, -3, -5, 33,
-14, -4, -1, 1, 3, 5, -33, 12, 5, -1, 20, -11,
-2, -1, 1, 1, -15, 3, 8, -5, -1, 0, -5, -1 },
{ -55, 5, 20, 14, 7, 6, 13, 6, -2, -3, 2, 3,
40, -32, -7, -1, 5, 30, -18, -4, -1, -1, -1, 46,
-41, -5, 1, 0, -3, -5, 38, -21, -4, 3, 28, -26,
-3, 0, -2, -5, 26, -11, 10, -10, -2, -2, 11, -2 },
{ 6, -49, 36, -16, 18, 1, 16, -30, 33, -1, -4, -15,
4, -6, 8, -4, -3, 23, 0, -1, -2, -1, -1, -27,
16, 10, 3, 0, -5, -18, 47, -17, -2, 3, -39, 32,
2, 0, -3, -10, 31, -9, -25, 23, 0, -1, 11, 8 },
{ 48, -11, 0, -7, -37, 11, -2, -11, 27, 0, 4, 16,
-43, 31, -9, -1, 10, -2, 18, -4, -4, 4, 7, 2,
-12, -6, -1, -2, -6, -2, 30, -8, -2, -1, 32, -40,
2, 2, -4, -11, 41, -17, 23, -28, 0, -7, 24, -10 },
{ 3, -52, 43, -18, 7, -2, 6, 11, -2, -11, 4, 5,
-17, 0, 5, 3, -24, 40, -34, 2, 1, 3, 15, -18,

```

```

6, -9, -2, 2, 4, 6, -11, -7, 0, 3, 16, -22,
-4, 1, 2, 25, -44, 14, 27, -33, 3, 12, -27, -18 },
{ -2, 53, 63, -24, 37, -14, -22, 20, 24, 6, -2, -18,
10, 5, 2, -4, -23, 3, 35, 1, -2, -5, 0, -7,
22, -11, -1, -2, -8, -8, -5, 35, -3, -6, 12, -4,
-1, -1, -4, -13, 13, 11, 18, -15, 2, -12, 18, -6 },
{ 37, 13, 35, 8, -33, 38, -15, 13, 21, -13, -9, 7,
7, 22, -5, -9, -19, 21, -4, -1, 0, -7, -24, 50,
-10, 11, -1, 1, 1, -12, 3, -3, 3, -14, 17, 23,
-3, -2, 4, 5, -24, 12, -14, 42, -9, 9, -24, 22 },
{ -11, 9, -12, -14, 31, 32, 1, -7, -17, -2, -4, -22,
15, 56, 4, 7, 15, -6, -19, -11, -1, -7, -13, 8,
54, -16, -6, 1, 6, 14, 12, -36, -3, -14, 26, 18,
-7, -4, 3, 16, 9, -32, 26, -3, 0, 15, -3, -1 },
{ 8, 55, -11, 2, -7, -7, 8, -33, 39, -8, 4, -17,
9, -15, -4, -10, 3, -4, 13, -20, 0, 5, 2, -13,
-8, 1, -5, 2, 10, -22, 30, -30, 5, 15, -21, -5,
-5, 3, 2, 19, -27, -1, 6, -27, 4, 29, -47, -26 },
{ -22, -25, -54, 24, 14, -19, 4, 2, 24, 10, 5, 13,
-3, 9, 15, -12, -32, 18, 27, -1, 2, -5, 6, 5,
24, -2, -3, 2, -1, -47, 22, 17, 3, -15, 33, 12,
-8, -4, -3, -19, -8, 25, 31, 8, -9, -2, -17, 11 },
{ 13, -18, -16, 21, -33, 45, -12, 13, -2, 1, -6, -32,
36, -11, 9, -3, -12, 11, -1, 14, 2, 10, -23, 7,
12, -2, -10, -2, -1, -7, -5, 19, 0, 17, -28, 24,
-22, 3, -4, -9, 7, 11, 21, -40, 10, -14, 22, -45 },
{ -10, 11, -33, -57, 23, 49, 8, -16, -8, 9, 0, -26,
-6, 23, -18, -9, -15, 15, -14, 16, 9, -5, -3, -2,
-23, 11, 6, -2, -7, -21, 5, 8, 9, 2, -6, -34,
25, 4, -4, -16, -13, 32, -24, -3, 6, -20, 1, 5 },
{ 11, 37, -3, 18, -5, -1, 33, -48, 21, -26, 11, 13,
3, -7, 4, 9, -3, 19, -39, 19, 5, -4, 23, -8,
14, 0, -5, -7, 2, 15, -11, -3, 12, 1, 12, 15,
-8, -1, -12, 11, -12, 26, 22, 9, -5, -36, 48, 15 },
{ -15, -11, -31, 8, 19, -15, -1, 22, 59, -21, -9, -13,
-10, 14, -7, 21, -13, 23, 14, -16, -2, -5, -29, 12,
-10, -1, 4, -3, 5, 36, -9, 3, -2, -16, -6, -10,
7, 3, -11, 46, -7, -5, -32, -2, 9, 3, 32, -18 },
{ 6, 2, 5, 59, 52, 68, 8, 15, 8, -2, 6, 30,
-10, 3, -18, -5, 11, 4, 17, -10, -8, 1, 33, -18,
-4, -10, -4, -1, -3, 8, 1, 15, -11, 24, -13, -7,
0, -1, 2, 5, 3, 6, -6, 0, 1, 10, -8, 8 },
{ -6, 7, 14, 19, 30, -24, 0, -22, -18, 1, -22, 41,
-29, 34, -24, -2, -6, -3, -18, 9, 4, 3, -23, 22,
-9, 2, -12, 2, -3, -16, 2, -8, 11, -5, -15, 16,
-19, -5, 3, -21, -1, 9, -19, -7, -12, -15, -1, -65 },
{ -2, -3, -5, 30, 4, -7, -84, -17, 6, -12, 10, -7,
7, 3, -12, 7, -44, -3, -23, 11, 3, 2, 11, -11,
4, -20, 7, 3, -4, -12, -7, -33, 18, 15, -6, -17,
7, 4, -3, -11, 0, -27, -14, -4, 9, -15, 1, 23 },
{ 7, -5, 19, 55, -4, -21, 32, 11, -5, -25, -2, -38,
7, 21, 2, -12, 29, -10, -8, 1, 2, -4, -28, -8,
16, -8, 15, 0, -14, -8, 4, -4, 9, -21, 2, -26,
35, 3, 1, -24, -7, 21, -12, -16, 32, -20, -17, 14 },
{ -1, -5, 2, -2, -28, -26, 8, -2, 3, 47, 3, 10,
35, 32, -53, 5, 11, 13, 1, 23, -1, 2, 16, 8,
21, -49, 1, -3, -2, 4, 14, 13, 0, 24, -8, -4,
-9, 1, -8, 21, 1, 26, -23, -2, 6, 21, 2, 12 },
{ -12, -16, -4, -28, 5, -10, -20, 39, 26, -36, 17, -1,
4, 15, -20, -14, 46, -17, 3, -17, 13, 18, 13, 9,
-7, 0, -7, -8, -18, 11, 6, -21, 25, 35, -9, 19,
-15, -8, -6, -14, -13, 24, 21, 7, -14, -34, -7, 6 },
{ 5, 0, 9, 15, 11, 8, -22, 18, 12, 75, -15, 3,
-10, -13, 29, -13, 26, -13, -7, 15, 16, -14, 1, -2,
-3, 20, 5, -12, 7, 1, 17, -32, 41, -13, 5, 0,
15, -2, -15, 27, -16, 18, 7, 0, 7, -7, 19, -7 },
{ 1, -9, 11, 2, -2, 6, -19, -20, -57, -28, 5, -5,
-13, -16, -14, -3, 4, 0, 41, -40, 18, 7, -13, -1,
6, -20, 2, -10, 31, -19, 28, -2, 8, -12, 4, -3,
-8, 4, -17, 33, -15, 30, -7, 1, -2, -4, 33, 8 },
{ 3, -1, -7, -17, -15, 22, 10, 17, 16, -10, -10, 14,

```

```

9, -37, 5, -8, 4, -12, -2, -7, 8, -34, 20, -13,
25, -57, 3, -5, -7, -4, -6, -13, 9, -37, 17, -12,
-19, -7, -3, -17, -4, -8, -44, 8, -14, -22, -11, -32 },
{ 2, 1, 6, -11, -3, 1, 46, 26, 3, 10, -24, 31,
12, 16, 15, 10, -41, -29, 6, -31, 11, 11, 2, 13,
0, -6, -4, -6, 18, -28, -10, -24, 3, 27, -15, 2,
-1, 18, -8, 0, -14, -23, -4, -19, 32, -27, 14, 27 },
{ 3, -2, 8, 14, 8, 1, 35, 28, 4, 6, 28, -54,
-33, -10, -29, 7, -21, -28, -6, 33, 1, 9, -6, 0,
-12, -14, -15, 4, 25, -25, -4, -10, 22, 6, 6, -1,
-23, -14, 10, 5, -1, -16, 1, 13, -38, 13, 16, 4 },
{ 2, -5, 7, 11, -14, -9, -3, 9, -8, -10, 11, -8,
45, 46, 24, 10, -3, -8, 3, -17, -4, -26, 43, -17,
-12, 28, -2, 7, 28, -18, 2, 1, -6, 13, 4, -18,
23, -27, 13, 3, -7, 6, -15, 17, -32, 1, 22, -37 },
{ -3, 3, -3, -15, 10, 2, -6, 10, 2, -48, 1, 28,
1, 6, 52, -4, 13, -13, 1, 60, -5, 3, -3, 13,
3, -16, 3, 6, 21, -14, 13, 6, 29, 12, -3, -6,
-5, 10, 16, 8, 14, 16, -18, -14, 17, 44, 11, 7 },
{ -5, -4, -4, -5, -2, 0, -17, 12, 19, -13, -60, -1,
-5, -16, -50, 10, 22, -14, -29, -3, -9, -28, 12, -10,
3, 12, 5, 13, 42, -26, 4, 11, -17, -13, 2, 12,
6, 9, 33, -7, -2, 9, 21, -9, 14, 9, 17, 12 },
{ -2, -4, 1, -7, -7, 6, 14, 11, -10, -23, 17, 30,
23, -13, -55, -22, -24, -8, 27, 31, 5, -9, 1, -12,
18, 37, 5, -1, -16, 3, -1, -13, 30, -16, 5, 10,
39, -12, -14, 13, 2, -22, 16, 4, 22, 12, 4, -21 },
{ 2, 1, 2, -7, 4, 8, -8, 4, 18, 20, 69, 37,
23, 2, -8, 13, 26, 14, -10, -10, -3, 21, -45, -28,
-5, -1, -5, 4, 35, -22, -15, -1, -5, -31, -2, -1,
-1, 7, 27, -14, -8, -6, -2, -7, 6, -8, 9, 4 },
{ 3, 3, 2, -1, -2, 9, 4, 2, -8, 0, -9, 0,
-2, -8, -2, 108, 7, 16, 31, 14, 1, -3, 2, 2,
3, -1, 9, 7, 3, 6, 11, -9, 31, 4, -1, 4,
10, -5, 3, -16, 1, 8, 11, 6, 2, -15, -27, -5 },
},
{
{ 110, -27, -39, -16, 32, -23, -3, 8, -7, -1, -3, -1,
9, -3, -3, -1, -1, 4, -3, 0, 0, -1, 0, 1,
0, 2, -1, -1, 0, 0, 0, 1, 0, -1, 0, 0,
0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0 },
{ 4, -66, 71, -34, 27, 17, 11, -47, 37, -5, -4, -14,
24, -9, 3, 2, -4, -8, 4, 5, -1, -2, -3, -2,
9, -3, -1, 1, 1, -1, -3, 3, -3, 2, 2, -1,
-1, -1, 1, 1, 1, -1, 0, 0, 1, -1, -1, 0 },
{ -29, 30, 9, -24, 67, -28, 8, 16, -46, 13, 0, -21,
52, -32, 1, 1, -5, 24, -29, 3, 3, 0, -5, 2,
10, -1, -6, 0, 1, -1, 1, 8, -8, -3, 1, 1,
-2, 0, 4, -2, 4, -1, -2, 1, 0, 0, -2, 0 },
{ -41, -77, -66, 44, 32, 16, 21, -7, -11, 8, 5, 8,
1, -10, 8, 4, 3, -3, -9, 3, 3, 1, 1, 3,
0, -5, 1, 2, 2, 2, -1, -1, -2, -2, 1, 1,
0, 1, 2, -2, 0, -1, 0, 0, -1, 0, -1, 0 },
{ 0, -40, 33, -26, -15, 9, -1, 38, -39, 9, -2, 30,
-42, 7, 9, 1, -16, 45, -39, 11, 0, -1, 2, 22,
-40, 17, 0, 2, 0, -4, 5, 3, 0, -6, 3, 0,
1, 5, 2, -13, 4, 2, 0, 3, -6, 7, -3, -1 },
{ -12, 13, -35, -69, 15, 30, -4, -36, -18, 49, 1, -6,
-12, 33, -6, 0, 11, -35, 8, 8, 7, 2, -6, 10,
-24, 23, -1, -1, 0, 1, 7, -24, 10, 2, 2, 3,
-2, 0, 2, -4, 0, 7, 0, -2, -5, 1, 2, 1 },
{ -13, -1, 17, 14, 41, -70, 8, 0, 14, -5, -1, 23,
-28, -7, 8, 7, -9, -21, 45, -17, -2, 0, -2, 23,
-37, 19, -7, 2, 2, -1, 1, -25, 37, -8, -4, -1,
0, 5, -2, -4, 1, 2, -3, -1, -8, 11, 2, 4 },
{ 7, -3, 6, 3, -50, -49, 32, -42, -26, 32, -15, 41,
19, -25, -5, 2, 25, -23, -23, 19, 2, -3, -4, 25,
6, -26, 6, -1, 1, 6, 3, -5, -12, 4, 3, 0,
0, 1, 11, 1, -11, 0, 1, -4, 0, -2, -3, -1 },
{ -16, 2, -35, -31, -20, -40, 22, -12, 54, -10, 20, -28,
-2, -1, 7, 0, -6, 26, -12, 2, 3, 5, 1, 10,

```

```

-34, 11, 7, 0, 1, 3, -18, 39, -30, 12, -3, 0,
-1, 2, 13, -34, 15, 0, 1, 5, 4, -1, -4, -11 },
{ 18, 38, 12, 39, 42, 37, 18, -55, -14, -35, 1, 21,
-22, 7, -14, -6, 13, 5, -17, -4, -3, 0, -5, 27,
-29, 7, 2, -5, -2, 2, -1, 20, -25, 6, -2, -1,
-2, 2, 13, -18, 5, 1, 0, 3, 4, -4, -5, -6 },
{ -13, -2, -3, -21, 25, -25, 15, 0, 9, 7, -18, 34,
-35, 39, -14, 4, 1, 12, -2, -24, 7, -1, 2, -16,
27, -9, -6, 0, 2, -4, -1, 33, -35, 6, -1, 2,
-2, 9, -32, 48, -26, 5, -2, -2, 25, -27, 4, 5 },
{ 1, 17, -17, -22, 6, 39, -3, 7, 34, 27, -17, 45,
4, -49, 7, -9, -14, 26, 23, -28, 4, -2, -10, 29,
1, -36, 13, -2, -3, 1, -17, 20, 23, -23, -3, 1,
-2, 1, 10, -1, -11, -2, 2, -6, 10, 13, -9, -1 },
{ -2, -2, -7, -43, -11, 13, 62, 15, -28, -61, 27, -5,
-17, -32, -1, -3, 30, -13, 19, -21, 0, 8, 14, -13,
0, -17, 8, -2, -1, 3, 15, -9, 8, -9, 0, 0,
3, 6, -8, 2, -13, 7, 1, 2, -3, 1, 1, -1 },
{ 1, -13, 17, 5, 3, 3, -24, 30, -13, 24, -13, -10,
-6, -17, 5, -7, 39, -34, 23, -32, 9, -1, 13, -3,
-16, -1, 0, 5, 1, -4, 5, 20, -29, 2, 4, 0,
1, 1, 21, -38, 11, 3, -2, -13, 41, -44, 16, -27 },
{ -13, -4, -8, -19, 17, -6, -55, 21, 27, -37, 22, 18,
-5, -4, -13, 6, 19, -20, -24, 22, 0, 3, -16, 37,
4, -33, -2, 8, 3, -5, 26, -21, -20, 20, -2, 1,
-3, -8, 29, 5, -32, 8, 0, 8, -11, -6, 4, 2 },
{ 8, 9, 12, 16, 13, 14, 65, 54, 46, 30, -24, -16,
0, 21, -26, -3, 29, -17, -23, 14, -4, -5, -14, 14,
-1, 3, -7, -4, 1, -3, 15, -8, 2, -10, -2, -1,
-2, -7, 11, -3, -4, 6, -4, 5, -6, 13, -10, 1 },
{ -9, -13, -9, -8, -20, 1, -8, 3, -1, -33, -8, 35,
67, 36, -29, 5, 21, 9, 5, -40, 2, 0, -9, 17,
-9, 34, -30, 1, 1, -5, 11, 5, 9, -30, 3, 3,
-1, 0, 5, -1, 15, -9, -4, 5, 9, -1, -11, 6 },
{ -1, 12, -10, -7, 1, 9, 0, 1, 25, -15, -56, 22,
4, -29, 44, 8, 1, -10, -31, -16, 27, 0, 5, -19,
-7, 22, -1, 1, -4, 6, 23, -30, -18, 18, 2, 2,
-4, 14, -27, -2, 25, -11, 3, 23, -28, -15, 16, 2 },
{ -7, -5, -1, -4, 8, -14, -12, -13, -2, -1, -30, 1,
-9, 22, -47, 7, 1, 18, -4, -22, 20, -6, 33, -32,
12, -36, 23, 1, 3, -3, 11, -12, 12, -3, -2, 3,
3, 7, 14, -41, 1, 11, 2, 14, -30, 29, -9, -46 },
{ 1, 3, 4, -5, 6, 12, 17, 16, 9, 20, 44, 56,
13, -16, -39, 2, -21, -2, 21, 16, -21, -2, 17, -30,
7, 18, -18, -7, 1, 5, -5, -8, -18, 43, -19, -1,
5, -3, 8, -12, 22, -16, 1, 10, -16, -9, 22, -12 },
{ 5, 2, 2, 15, -4, 0, -18, -22, 21, 34, 37, -19,
-6, -37, -30, 2, 37, 33, -20, -18, -1, 3, 27, -8,
-27, 5, -24, 8, 1, -9, 33, -11, 11, -15, 3, -5,
3, 22, -33, 17, -19, 2, 0, 14, 2, -9, 2, 11 },
{ -6, 2, -1, -6, 15, -6, -17, 4, 10, -7, 2, 32,
10, 9, 23, -34, 56, 3, -3, 43, -33, 12, -10, -35,
-9, -4, 21, 7, -1, -7, -4, 15, 18, -1, -5, -7,
-3, 16, -38, -11, 18, 7, -5, -19, 11, 24, -15, -16 },
{ -9, -7, -10, -9, -7, 0, -14, -1, -5, -22, -48, -17,
-28, -46, -63, -10, 8, -1, 10, 38, 16, 4, -13, 8,
12, 29, -29, -2, 4, 1, -24, 12, 6, 8, 2, 4,
-5, 7, -9, 8, 14, -8, -4, -17, 8, 10, 0, 2 },
{ 1, 7, -4, 2, -2, 9, 11, 6, 9, -16, -8, 4,
26, 6, -19, 19, -49, -19, -19, 28, -6, 11, 18, 4,
-32, -16, 9, 1, 9, -8, -5, -10, 21, -16, 17, -3,
-8, 38, -26, -6, -20, 16, 1, -26, 30, -34, 29, -28 },
{ 1, -5, 9, 6, 0, 5, -7, 17, -4, 0, 7, -1,
25, 8, -5, -38, 3, -31, -11, -17, 41, -12, 39, 7,
-32, -4, 4, 2, -2, 14, -47, 19, -7, 28, -18, 6,
3, 18, -14, 15, -24, 13, -7, -10, -26, 32, 3, 24 },
{ 0, 7, 1, -9, 9, -4, -7, -1, 13, -3, 28, 13,
-27, -2, 11, 12, -4, -39, -51, -5, 8, -12, 38, -4,
36, 3, -27, -4, 2, 10, -17, 6, 15, -41, 3, 5,
-3, 12, 13, -12, 29, -33, 4, -20, 16, 9, -22, 5 },
{ 0, -6, 5, 2, -2, -1, -15, 8, -4, 9, 12, -3,

```

```

-7, -15, -27, 13, -8, -34, -21, -39, -38, 32, -49, -6,
-2, 6, 28, -7, 8, 29, -31, 7, -18, -21, 23, -3,
3, 9, -26, -1, 6, 10, -3, 18, -33, 1, -8, -8 },
{ 3, 3, -2, -2, -3, 9, -2, 1, -8, 0, -24, -4,
3, -4, 25, 56, 3, -12, 7, 3, -38, 8, 11, -13,
-14, -3, -62, 27, 0, 16, -5, 27, -4, 14, -20, -2,
4, 5, 3, -5, -40, 8, 4, 8, -6, 27, -22, -20 },
{ 5, 3, 5, 6, 0, 3, 3, 6, -4, 7, 28, 7,
2, 5, 12, -3, -7, -5, 36, 34, 58, -9, -24, 7,
10, -5, -24, 14, -4, 0, 10, 15, -28, -30, 32, 2,
-13, 36, -21, -11, -3, -1, -1, 30, -20, 6, -18, -28 },
{ -3, -2, -6, -1, -7, -5, 4, -8, 0, 1, -6, -5,
-4, -5, 10, -81, -32, -11, -11, -20, -35, 16, 10, 18,
29, 24, -28, 12, -8, -23, 22, -4, -7, -2, -2, 0,
0, 15, 9, -1, -18, 19, -7, 5, -5, 8, 0, -36 },
{ -4, -4, -1, -5, 2, 1, -5, 6, -8, 2, -15, -19,
1, 18, -1, 0, 9, -1, 22, 9, -37, 10, 36, 56,
11, -32, 4, -22, -6, 10, 10, 17, -3, 1, -18, -5,
-2, 35, -27, -1, 30, -42, 9, 29, -19, -8, 15, 11 },
{ -3, -1, -3, 0, -1, 1, -10, 3, 0, -3, -8, 6,
5, -20, -9, 13, -17, -20, 11, 21, -17, -39, 31, -3,
0, 38, 49, -13, 0, -4, 30, 19, -22, -29, -27, 18,
-2, 1, -1, 3, -11, 31, -8, 24, 9, -1, -40, 18 },
},
},
{
// 4
{
{ -99, 31, 29, 8, -44, 39, -10, 7, 4, 2, 3, 0,
-21, 19, 3, -1, -7, 6, 0, 1, 1, 0, -2, -6,
6, 1, 1, 0, -1, -3, 2, 1, 0, -1, -1, 2,
0, 1, 0, -1, 1, 1, 0, 0, 0, 0, 0, 0 },
{ 11, 70, -13, 5, -2, -36, -25, 69, -21, -3, 1, 3,
2, -25, 5, -5, -24, 44, -9, -5, -1, 1, 1, 3,
-15, 6, 1, -1, -2, -10, 18, 1, -3, 1, 1, -7,
2, 0, 0, -3, 5, 2, 0, -2, 1, 0, 1, 0 },
{ -47, -1, 6, 12, 12, -28, 24, -14, -8, 0, 2, 3,
36, -66, 5, 3, 29, -16, -6, -1, 0, 0, 4, 26,
-54, 11, 5, 0, -1, 22, -13, -3, -1, 6, 7, -26,
6, 2, -1, 11, -6, -2, -1, -8, 2, 3, -1, -2 },
{ 2, 59, -28, 9, 2, -10, 0, 4, -25, -1, 0, 0,
4, 20, -7, 6, 42, -42, 0, -3, 0, -1, -2, -1,
32, -21, -6, -3, 1, 44, -42, 2, 1, -1, -2, 25,
-18, -3, 0, 23, -18, -5, 2, 10, -7, 7, -3, 2 },
{ -38, -3, 12, 8, 49, -37, 12, -10, -7, 4, -1, -7,
36, -14, 10, -2, -17, 1, 2, 2, -3, 0, -6,
43, -21, -6, 4, 6, -34, 11, 4, 1, -1, -12, 49,
-25, -6, 6, -29, 10, 4, 0, 23, -13, -13, 4, 6 },
{ 0, 43, -49, 7, -7, -1, 17, -35, -5, 9, 1, -7,
-4, 10, -10, -14, 15, -36, 19, 1, 2, -1, -2, -5,
-7, 10, 5, 3, 8, -43, 18, 2, 4, 0, -1, -22,
25, 2, 11, -49, 27, -1, -1, -18, 19, -21, 7, -7 },
{ -6, -36, -68, 26, -23, 48, 14, 28, -28, -2, 4, 15,
7, -13, 0, -5, 11, 13, -21, 2, 1, 1, -3, 23,
-14, 2, -7, 2, 2, -4, 5, -5, -3, 1, 1, 25,
-26, -1, 9, -16, 7, 0, -2, 26, -23, -12, 4, 8 },
{ 37, 25, 48, -1, -55, -5, -14, -21, 23, -6, -3, 12,
-1, -25, 16, -10, 7, -20, 11, 4, -2, 0, -2, 19,
-21, 19, -7, 2, -2, -8, -10, 7, 4, 1, -6, 30,
-23, -1, 10, -24, 5, 2, -8, 36, -27, -17, 5, 12 },
{ -7, -20, -11, 4, -39, -57, 56, 9, 5, -3, 4, 6,
-43, -4, 26, -10, 29, 14, 0, -2, -1, 4, 5, -41,
15, 14, 1, -3, -13, 14, 19, -5, -1, -3, -21, 11,
4, 0, -10, 8, 12, -3, -5, 4, 1, 5, 6, 1 },
{ 2, 35, -28, 3, -1, 7, 28, -43, 38, 1, 5, -4,
8, 1, -12, 5, -19, 21, 10, -8, -1, -2, 5, 10,
-9, 2, -10, -6, 14, -16, 40, -17, 1, 1, 8, 6,
-18, -3, -21, 47, -13, 1, 2, 17, -27, 41, -16, 11 },
{ -16, -17, -46, 6, -4, -20, -45, -9, 7, 6, 2, -4,
-14, -12, 40, 18, -42, -23, 16, 2, 2, -1, -1, -15,

```



```

-3, 42, 0, 3, 18, -11, -33, 14, 5, -2, -15, 9,
19, -4, 2, 12, -31, 5, -9, 13, 3, 10, -13, 7 },
{ 17, 1, 0, 28, -37, 26, -5, -3, 8, -1, -6, -20,
42, -37, 26, -5, 1, 2, 4, 5, -4, 1, -15, -6,
27, -18, 8, -5, 4, -4, 7, 5, -2, 5, -29, 24,
-4, 14, -11, 14, 3, -1, 7, -41, 48, 20, -2, -27 },
{ -1, -22, 1, 28, -44, -47, -3, 0, -6, -6, 5, 21,
-11, 6, -28, 17, -20, -11, -1, 2, -1, 6, 5, 13,
2, -51, 4, 7, 34, -25, -11, -1, 0, 1, 25, -20,
-28, 8, 21, -6, -16, 0, 15, -24, -7, 9, -16, -10 },
{ 15, 11, 32, 1, -8, 13, 54, -1, -49, -5, -7, 1,
10, 8, -5, 8, -11, 5, -34, 4, 1, -1, -5, 1,
6, 29, -3, 2, 41, -23, -14, -7, -2, -4, -8, 8,
35, -13, 22, 3, -32, 7, -9, 13, 20, 11, -21, 11 },
{ -10, -13, -12, -89, -33, -12, 8, 29, -6, 19, -4, -32,
35, 15, 16, -12, 10, -12, 16, 2, 3, -2, -14, 22,
3, -3, 0, -2, 8, -13, 1, 8, -5, -5, 15, -3,
-5, -2, 3, -1, 3, 0, 8, 0, -8, 23, -11, 3 },
{ 4, -11, 24, 21, 21, 14, -27, 21, -30, 22, -1, 12,
-21, -13, 5, -13, 35, -29, 30, -6, -5, 4, 7, -16,
-5, 2, 1, -2, 25, -21, 28, -1, -6, 5, -7, -3,
0, -8, 8, 6, 26, -17, -1, 6, -15, 61, -19, 9 },
{ -2, -15, 5, 43, -23, -33, -27, -12, -19, 1, -5, -2,
36, 47, 2, 7, 6, 4, -2, 8, -1, -3, -17, 39,
6, 14, 3, -4, -23, 5, 20, -7, 1, -10, 18, 2,
34, -19, -28, 7, 24, -7, -6, 21, 13, 11, 12, 17 },
{ 0, -7, 2, 28, 2, 4, 36, 54, 47, 0, 0, -5,
12, 0, -49, -16, -13, -20, 46, 2, -2, -1, -8, 4,
-2, -2, 11, -3, -8, -5, -19, 33, 1, 1, -7, 4,
21, -10, -7, 1, -8, 14, -15, 16, 10, 4, -3, 16 },
{ -4, -5, -6, -33, -23, -7, -21, -16, -23, 21, -3, -4,
0, -29, -75, 39, -8, 0, -9, 9, 3, 2, 2, -22,
-3, -6, 16, -7, 4, 6, 8, -1, 0, 7, -36, 15,
6, 4, -16, 7, 19, -6, -27, 15, 5, 1, 21, 6 },
{ 6, -1, -3, 7, -20, 12, -13, -3, 9, -14, 21, -33,
23, -24, 10, -1, 12, 6, -11, -3, -4, 1, 28, -38,
25, -28, 17, -1, 6, 3, -1, -8, -2, 8, 14, -32,
29, -33, 8, -5, -5, -2, 15, 10, -24, -16, -4, 61 },
{ -9, -2, -8, -10, 9, -8, -18, 20, 58, -37, -1, 6,
2, 5, -6, 32, 35, 1, -16, -15, -5, -5, -7, 15,
3, 8, 5, 3, 40, 6, 16, -34, -1, -3, -4, 20,
12, -6, 45, -6, 10, -21, -8, 9, 29, 1, -3, -5 },
{ -7, -6, -8, -4, 0, -15, -29, -17, 5, 0, 0, 0,
-1, 7, -24, -82, 16, 11, -28, 8, 4, -1, -2, -2,
-9, -10, 27, 12, -20, -5, 1, -18, 9, 1, -19, 9,
2, 14, 13, -3, -33, 13, -19, 10, 13, 10, -38, 4 },
{ 5, 1, 6, 33, 5, -7, 4, 6, 19, 59, -9, -76,
-15, 5, -6, 10, 5, 7, -27, 25, -1, 0, -33, 1,
-7, 13, -4, 9, 8, 11, -4, -7, 5, -7, -5, -7,
-11, 7, 28, -4, 4, -6, 0, -7, -29, 0, 0, -7 },
{ 4, 7, -1, -8, 4, 6, 9, 10, 23, 68, -10, 41,
-21, -24, 21, -4, -10, -21, -28, 16, 10, -7, 5, 20,
19, -22, 25, -12, 4, -4, 1, -22, 11, -15, 30, 3,
12, 0, -17, 6, 0, -8, 2, 12, 29, -8, 8, 20 },
{ 1, -9, 10, 7, -9, -3, -6, 29, 1, 18, 2, -1,
27, 14, -11, -3, 1, -39, -2, -25, 8, 3, 18, -17,
-4, 23, -24, -6, -1, -17, 22, -52, 13, 19, -15, 0,
-5, -2, -29, 18, -27, -11, 8, -12, -15, -50, -14, -20 },
{ 0, -5, 5, 3, 6, 1, -7, -5, -22, -4, -8, -28,
-28, -7, 11, 9, 30, 19, 40, -30, 4, 0, -33, 17,
-6, -22, 32, -25, 27, -13, 16, 1, 4, -18, 6, 2,
-1, 15, -16, 19, -29, 23, -13, 15, 9, -45, 1, 7 },
{ -3, -6, -2, -6, -4, -2, -14, -8, -3, -2, 5, -8,
-30, -45, -40, -10, 14, 9, 4, -8, -3, 5, -16, 21,
42, 32, -35, 6, -15, -4, 3, 5, -4, -14, 33, 15,
12, -39, -9, -4, -11, 3, 42, -10, 7, -4, -19, -15 },
{ -1, 0, -4, 3, -6, -3, -10, -12, 12, 60, -14, 32,
23, 17, 1, 13, 37, 51, 23, -5, -1, 3, 24, -8,
-9, -6, -33, 1, 8, 3, -6, 27, -8, 16, -13, 2,
5, -24, 23, -3, -15, 23, -1, -7, 12, -15, -11, 0 },
{ -2, -1, 0, -2, 7, -2, -13, 10, 18, -23, -3, 2,

```

```

-3, 4, 2, 2, 37, -5, -39, 49, 11, -10, 4, -5,
-2, -1, -14, -40, 22, -36, -6, 21, 18, -2, -5, -8,
2, -17, -38, 0, -9, 42, -5, -7, -19, 13, 25, -18 },
{ 0, -4, 1, 3, -2, -5, -5, -8, -2, 14, 43, 6,
15, 7, -11, -52, -18, 4, 0, -12, -5, 5, 4, -8,
4, 23, 20, -37, 55, 29, -2, 11, -14, -6, 12, 5,
-2, -9, 17, 17, 15, -1, 14, 5, -5, -11, 43, -28 },
{ -5, -3, -7, 1, -5, -1, 1, -3, -7, -17, -95, -9,
-8, -8, -2, -33, -18, 4, 19, 6, 14, -19, 18, 8,
5, -10, -13, 0, 23, 18, -2, -17, 12, 10, 0, -4,
9, -26, 10, 9, 5, -15, -10, -8, -12, -4, 26, -4 },
{ 2, -1, 1, 10, -6, 2, 0, 3, -2, 3, -16, -17,
8, -1, -9, 17, 15, 8, 4, -5, 2, -44, 68, -1,
23, 34, 39, -3, -14, -11, -3, 11, -1, -3, 33, -1,
-6, 42, 8, -2, -3, 6, 3, 23, -1, 8, -8, -32 },
},
{
{ 96, -64, -6, -21, 31, -6, 6, -30, 19, -2, -4, -3,
9, 2, -2, 1, 2, -14, 8, 2, -1, -1, 0, 1,
3, 0, -1, 0, 0, 0, -5, 2, 1, 0, 0, 1,
0, 0, 0, 0, -1, 0, 0, 0, 0, 0, 0, 0 },
{ 32, 49, -54, 9, 45, -43, -3, 34, -38, 3, 1, -5,
39, -31, -4, -1, 4, 8, -10, -4, 1, -2, -1, 19,
-14, -3, -1, 0, 1, 3, -1, 0, -2, 2, 5, -4,
-1, -1, 1, 1, -1, 0, 1, -1, 0, 0, 0, 0 },
{ -56, -6, 3, -1, 45, -11, 15, -56, 11, 11, 0, -18,
41, -20, 2, 5, 17, -53, 23, 2, 2, -4, -3, 13,
-1, -3, 0, 1, 4, 7, -28, 11, 3, 2, 0, 4,
-1, -1, 0, 2, -10, 2, -1, 2, 0, 1, -3, 1 },
{ -6, 13, -81, 1, 10, 11, 6, -13, -4, 12, 6, -10,
-32, 45, -1, -1, 6, -20, 16, -4, 3, 0, 2, -42,
46, -3, -3, 1, 2, -7, 3, 1, 2, -1, -22, 22,
1, -1, -1, -6, 8, -3, -6, 7, 1, -2, 3, 1 },
{ -29, -69, -49, 26, 10, 58, 3, 13, -30, 7, 1, 7,
12, -2, -4, 3, -1, 15, -30, 8, 3, 1, -8, 28,
-19, 0, 1, 1, 2, 5, -1, -8, -3, -1, 15, -10,
-2, 0, 1, 4, -5, 0, 4, -2, -1, 1, -2, 0 },
{ 28, 11, -11, 29, -61, 10, -16, 9, -16, -4, -4, 13,
-6, -2, 0, 4, 22, -48, 9, 8, -1, 4, -10, 15,
-8, 4, -1, 0, 4, 29, -61, 23, 1, -1, -2, 9,
-5, 1, 5, 11, -31, 8, -7, 10, -2, 2, -9, 4 },
{ -8, -28, -3, 37, -19, -61, 53, -21, -20, -13, 2, 35,
-21, -17, 6, 4, 33, -1, -27, 0, 0, 3, 22, -22,
-2, -2, 0, 1, 1, 13, 10, -26, 2, 4, -7, 0,
-4, 0, 1, 6, 5, -12, 1, -1, -2, 2, 1, 0 },
{ 4, 28, -39, 1, -14, 1, 5, -33, 46, -9, 8, 16,
-22, 14, -16, 1, 0, -8, 15, -10, 1, -2, 10, 22,
-41, 5, 1, 0, 4, -5, 12, -14, 6, -4, 43, -54,
11, 3, -2, 9, -5, -3, 18, -24, 1, 7, -8, -5 },
{ -18, -33, -35, -26, -19, -19, -14, 34, 37, 7, 10, -7,
-10, -56, 28, -1, -4, 1, 31, -9, 0, 5, 9, -23,
-31, 26, 7, -1, -4, 1, -1, 20, -6, 13, -31, -6,
9, 4, -2, 0, 6, 2, -16, -3, 4, 1, 4, -3 },
{ 17, 10, 8, 84, 10, 25, -40, -27, 6, -5, 0, 12,
2, -28, 10, -5, -4, -9, 21, -3, -5, 2, 7, -5,
-10, 4, 0, 0, -1, -22, 24, 1, 1, 13, -18, 7,
-5, 3, 1, -26, 34, -11, -3, -4, 4, -10, 15, -5 },
{ -8, 3, 10, 10, 59, 1, -39, -6, -1, -20, 1, 36,
-39, -1, 3, 13, -29, 14, -13, -4, 3, -5, 28, -27,
-5, 10, 1, 1, 3, 15, -28, 1, 5, 3, -7, -6,
7, -2, -2, 28, -44, 11, -11, 11, -4, 11, -19, 9 },
{ 8, 22, 6, -36, 12, 57, -2, 9, 3, -24, 7, 8,
5, -23, 0, -3, 55, -16, -10, -22, 1, -2, 25, -12,
-1, -4, -2, -5, -5, 39, 5, -33, -3, 18, -11, 5,
-9, 0, 0, 16, 19, -33, 5, -3, -5, 8, 7, -2 },
{ -5, -1, -10, 20, 4, 7, 42, 29, 52, -54, -1, 26,
24, 3, -34, 1, -11, 12, 22, -18, -2, -4, 3, 16,
8, -17, -5, -1, 0, -5, 2, 10, -4, 2, -5, 31,
-21, -3, -3, 3, -9, 14, -13, 31, -14, -4, 0, 12 },
{ 0, 15, -23, -14, -18, -5, -23, -39, 29, -2, -11, -2,
8, -10, 37, -1, -3, 17, -39, 21, 5, -8, -21, 25,

```

```

-7, 24, -1, -1, -2, 3, 12, -34, 18, -17, -3, 38,
-8, -1, -4, 7, -5, -3, -18, 45, -13, -5, 3, 18 },
{ -16, -25, -9, -8, -3, -31, -56, 21, 8, -29, -1, 28,
15, 10, 37, -5, 4, -16, 9, -7, 4, -8, 10, 7,
37, -28, -1, 3, -4, -3, -5, -9, 6, -13, 39, 6,
-20, -1, 5, -13, 8, -14, 40, -12, -3, -5, -3, -5 },
{ -2, 1, -10, -37, -34, 7, -6, -32, -47, -42, -1, 26,
45, 3, -10, 10, -34, -10, 1, -10, 7, -3, 26, -12,
-10, 12, 1, 5, 20, -36, 6, -4, -1, 19, -20, -3,
12, 1, 6, -15, -5, 1, -11, -2, 8, -2, -10, -1 },
{ 8, -1, -3, 31, -23, 4, -1, -5, 17, -2, 19, -55,
31, -12, 14, 2, -11, 16, -5, -18, -2, 3, 5, -18,
26, -12, 1, 0, 14, -4, 10, -23, -2, 13, -8, 4,
-3, 1, -2, 37, -38, -3, 22, -22, 10, 34, -41, -2 },
{ -3, -4, 2, 11, 9, -7, 3, -13, -26, -60, 0, -26,
0, 39, 45, -11, 20, 25, 26, -29, -6, 2, -25, 12,
-12, 32, -3, -3, -18, 25, 6, 20, -18, -6, -2, -8,
20, -1, -8, 14, 8, 6, -7, 0, 7, 5, 8, -2 },
{ -8, -18, 4, 10, -7, -31, -37, 11, 3, 14, 12, -14,
14, 35, -50, 40, -18, -14, 4, -21, -4, 10, -3, 9,
-11, 16, -6, 3, 1, 26, 3, -25, -5, 11, -8, 1,
10, -10, -7, 21, 24, -43, -14, 17, -11, 12, 18, 5 },
{ 7, -3, 12, 0, -1, 6, 11, 6, -25, 50, 19, 15,
-2, 7, 26, -21, 5, -6, 36, -45, 8, -6, 27, 12,
-9, 11, -3, -1, 2, -19, 18, -5, -6, 0, 23, 6,
-5, -9, -4, 2, -6, -1, 2, 43, -29, 14, -25, 39 },
{ 3, 2, 11, 18, 14, 9, 25, 47, 18, -3, -26, 14,
25, 20, 25, -7, -8, -45, -5, 22, 7, -15, -5, -15,
4, 43, 0, -4, 3, -15, 2, -30, 24, -14, -1, -12,
45, -9, -3, 1, -1, -13, -4, -2, 22, 10, -9, 3 },
{ -5, -10, 3, -6, 5, -2, -16, 6, -18, -33, -12, -23,
-40, -40, -33, 14, 28, -17, 12, -13, -3, 5, -35, 7,
14, -4, 14, 1, 33, -27, 23, -13, 10, -35, 12, -2,
19, -4, 14, -8, -3, 3, -14, 15, 7, 15, -23, 19 },
{ -7, -10, -4, 3, -7, -12, -29, -1, -3, 6, -45, -1,
18, 22, -23, -47, 35, 18, 24, 32, 8, -21, 16, -6,
-14, -35, 22, -5, -13, 6, 19, -4, 19, 13, -18, -18,
2, 4, -9, 16, -5, 10, -30, 5, 0, 18, -12, 10 },
{ -2, -4, 6, 1, 10, -10, -13, 9, 14, -7, 67, 0,
5, 26, -1, -51, 12, -24, -40, -17, -1, 31, -19, -1,
-21, -16, 13, 4, 1, -11, -4, -13, 3, 6, -19, -8,
2, 8, 17, -24, -9, 14, -26, 0, 4, -12, -12, 3 },
{ 3, -3, -1, 12, -11, 4, 0, -1, -2, -21, 29, -47,
7, -13, -13, -5, -7, 3, -8, 15, 8, -34, 39, -28,
7, 1, 16, 1, -16, 15, -16, 1, 12, -11, 31, -21,
24, -6, -1, -16, 5, 7, 7, 24, -11, -39, 25, 41 },
{ 3, 6, -4, -1, -7, 9, -5, -13, 7, 14, 20, 43,
32, -12, 9, 5, -2, 4, -7, 7, -23, 22, -20, -25,
22, -25, 26, -7, 14, 14, 12, 25, -34, -32, 14, -17,
29, -7, -17, 29, 17, 9, -13, 7, 13, 22, 21, 27 },
{ 1, -1, 1, -5, 0, 4, 8, -3, -27, -12, 23, 7,
-1, -18, -18, -33, -42, -21, 29, 20, 1, 9, -35, -27,
-14, 8, 3, 2, -33, 23, 18, -15, 29, -29, 5, 11,
-27, 20, -24, 21, 3, -4, 18, -4, -22, 6, -6, -14 },
{ 0, -4, 2, 3, -9, -2, -15, -7, 5, 19, 5, 21,
12, -12, -29, 4, 21, 1, -18, -51, 46, -32, -10, 3,
28, 27, -6, -4, -36, 5, 17, 9, 10, -16, -4, 4,
13, 5, -25, 5, -13, 36, -4, -14, 21, -19, 0, -22 },
{ 1, 1, -3, 1, -3, 0, 16, -6, -2, 4, -33, -7,
-19, -10, 12, -33, -49, -11, -12, -51, 12, -27, -5, 17,
4, -30, 48, 0, 8, 14, -20, -6, -6, -3, -1, 1,
2, 18, 5, 6, 6, -35, -19, -2, 21, -2, 19, -3 },
{ -6, -1, -7, -3, -1, -6, -7, -9, 8, -6, -25, 0,
-7, -14, -33, -47, -9, -8, -25, 6, -28, 22, 14, 14,
25, 41, 0, 8, -39, 2, -3, 16, -42, 18, 7, 11,
15, -2, 2, -12, 10, -11, 26, 2, 5, 18, -29, 20 },
{ 0, 1, -2, 3, -1, 3, 0, -9, 8, 19, -15, 11,
26, 7, -14, 1, 10, 41, 18, -22, -18, 1, -8, -39,
-25, 9, 1, 16, 2, 27, -26, -14, 5, -46, 3, 15,
1, 6, 46, -37, 1, -13, 10, -3, 19, -11, -31, 2 },
{ -2, -2, -4, -3, -5, 4, 1, 0, -7, 6, -18, -4,

```

```

2, -7, 6, 1, -3, 2, 6, -29, -68, 47, 26, 16,
17, -11, -12, 0, -24, -6, -7, -20, 51, -11, -4, -10,
11, -9, -27, 5, -14, 13, -17, -5, 6, -29, 13, -3 },
},
{
{ 110, -29, -41, -11, 27, -27, 0, 6, -2, -3, -2, 0,
6, -3, -3, 0, 1, 0, 0, 0, -1, 0, 1, 0,
1, 0, -2, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 8, 100, -24, 8, -13, -40, -23, 47, -27, 0, 2, -1,
1, -5, -2, -5, -2, 14, -4, -5, 0, 0, -1, 1,
0, -1, -1, -2, 0, 2, 1, 1, -1, 0, 0, 0,
0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0 },
{ -32, -11, 13, 1, 46, -79, 38, -9, -10, 3, 0, -6,
42, -53, 15, 4, 7, -4, -1, -1, 1, -2, 5, 9,
-14, -1, 2, 2, 1, 0, -1, -1, 0, 2, -1, -1,
-2, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ -26, 0, -92, 43, 0, 13, 42, -16, -15, 8, 7, 3,
-6, 8, -10, -4, 35, -33, 6, -1, 2, 1, -1, 0,
0, -1, -2, 0, 5, 7, -10, -1, 1, 0, 0, 0,
0, 0, 2, -1, 0, -2, 0, 0, 0, 0, 0, 0 },
{ -28, -48, -40, 26, 21, 15, -39, 54, -18, 3, 4, 7,
-6, -22, 25, 6, -38, 45, -18, 1, 1, 2, -1, 2,
-13, 8, 4, 1, -1, -10, 12, 2, -4, 0, 0, -4,
0, 1, -1, 0, 0, 3, 0, -1, 0, 0, 0, 0 },
{ 12, -22, 1, 20, -91, -33, 13, 11, 28, -9, 0, 18,
-17, -27, 31, -5, 8, -4, 5, 4, -2, 5, 0, 7,
-37, 26, 2, -1, 0, 1, 1, 0, 2, 1, 0, -11,
2, 5, 0, 0, 0, 0, -2, -1, -1, 0, 0, -1 },
{ -17, -26, 0, 1, -14, -58, 30, 28, 4, -2, 3, 10,
-37, 50, -19, 16, -23, 8, 2, -1, 0, 2, 6, -36,
52, -25, 2, 3, 2, -10, 6, 0, 0, -5, -5, 14,
-3, -2, -1, 1, 0, 1, 2, 0, 1, 1, 0, 0 },
{ 11, -11, 30, -11, -3, 30, 58, 54, -45, -4, -4, 14,
-9, -15, -17, -24, 38, 18, -33, 3, -1, 2, 1, -2,
-3, -6, -2, -2, -15, 34, -5, -6, -5, 0, 0, -1,
-1, -1, 2, 9, -1, -1, 0, 0, -1, 0, 1, 0 },
{ -11, -22, 5, -10, 8, -20, -57, 17, -9, 5, 0, 2,
-10, 12, 40, -13, 41, -39, 13, 2, 1, 1, 0, -8,
14, -13, 10, 1, -18, 50, -48, 14, 0, 0, -4, 7,
-5, 1, 1, 10, -8, -5, 0, 0, 0, 0, 1, 0 },
{ 15, -7, 32, 89, -1, -2, -23, -3, -6, -35, -5, 16,
14, -8, -41, 13, 3, -5, 2, -1, -8, 1, -4, 22,
5, -35, 6, 3, 3, 5, -8, 2, 1, 2, 7, 13,
-24, 1, 0, 3, -4, 1, 3, 4, -6, 0, -1, 0 },
{ -13, -18, -25, -44, -41, 3, -11, 32, 22, 21, 1, -22,
56, -12, -47, 20, 2, -9, -5, 8, 3, 1, -13, 30,
7, -33, 7, 2, 4, 0, -9, 4, -3, 3, 6, 11,
-20, 4, 0, 1, -2, -1, 1, 3, -4, 0, 1, 0 },
{ 6, 16, -5, 15, 12, 22, 23, 23, 63, -16, 2, -27,
6, -24, 42, -37, 3, 7, 12, -8, -2, -2, -6, -13,
31, -21, 3, -7, -9, -1, 13, -2, 0, 2, -21, 40,
-30, 6, -4, 2, 5, 1, 0, 9, -3, 1, 1, -1 },
{ -10, 3, 2, 5, 37, -5, 5, 42, 62, -15, 3, -12,
-26, 6, -45, -2, 3, -12, 26, -12, -2, -3, -6, 13,
-30, 18, -3, 3, -20, 17, -4, 6, -2, -2, 17, -37,
29, -10, -9, 11, -1, 1, -1, -7, 6, 2, 2, 1 },
{ 7, 4, 13, -10, 9, 19, 5, 28, -11, 16, -3, 16,
-39, -53, -7, 24, -9, -44, 35, -7, 0, 2, 3, -10,
8, -3, 0, -3, 42, -33, -21, 25, -4, 2, -4, 6,
-5, 2, 25, -30, 4, 1, 0, 2, -3, -5, -2, -1 },
{ -10, 1, -18, -27, 3, -9, -29, -34, 16, -5, 7, 31,
-40, -44, -37, -14, 34, 35, -3, -6, 3, 1, 27, -23,
-4, -29, 26, 1, -8, 5, 21, -12, 2, 6, -3, -5,
-19, 13, -5, 1, 11, -4, 2, -5, -9, 2, 2, -1 },
{ 4, 14, -10, -4, -3, 12, 13, -17, 12, -12, 6, -2,
-11, -32, 0, 64, -25, 12, -16, 5, 1, -3, 11, -20,
11, 14, -21, 4, -5, 38, -36, 6, 0, 4, -12, 11,
-3, 1, -19, 47, -40, 9, -2, 4, -2, 8, -4, 0 },
{ -6, -11, -2, -3, -9, 2, -22, 11, -5, -60, 17, -23,
23, -10, -11, 24, 46, 0, -4, -6, -1, 1, 4, -30,

```

```

21, 27, -21, 1, 15, -13, 17, -13, -1, 9, -31, 7,
38, -25, 15, -14, 12, -8, -7, -2, 20, -1, 0, 2 },
{ -9, 6, -12, -36, 11, 4, 14, 6, -1, -76, 10, 22,
-6, 25, 34, 24, 3, 0, 0, -14, 2, -1, -5, 32,
-20, -26, 4, -8, 28, -3, -4, -12, 3, -9, 27, -9,
-24, 13, 15, 2, -6, -3, 9, 2, -18, 0, -3, 0 },
{ 4, 8, 10, 15, 19, -3, -7, 14, 42, 49, -17, 21,
-7, 19, 17, 44, 43, -1, -51, 6, 6, -7, 14, 10,
-10, -3, -21, -9, 23, -6, 12, -26, 5, 3, 1, 8,
-5, -8, 16, -5, 6, -12, -3, 12, -8, 1, -1, 2 },
{ 8, 2, 5, 13, -5, -1, 4, -8, -3, 5, 11, -75,
-32, -12, 14, 12, 7, 7, -28, 24, -5, 0, -13, 2,
5, -34, 41, -3, 22, 0, -7, -10, 6, -4, 15, -31,
16, 7, 12, -3, -8, -2, 11, -35, 22, -2, -3, -5 },
{ 4, 8, -3, 5, -4, 16, 5, 10, 16, -2, -8, 50,
44, -3, 17, -5, -17, -23, -10, -5, 5, -5, 41, -28,
14, -32, 21, -9, 2, -4, 0, 3, -3, 10, 8, -37,
29, -14, -10, 10, -3, 0, 18, -39, 31, 4, -2, -4 },
{ -7, 2, -8, -4, 6, -10, -5, -5, 23, -23, -1, 8,
1, 4, -23, -50, -24, -9, -54, 36, 2, -6, 14, -7,
-3, 20, -1, -1, 21, 2, -38, 7, 10, 7, -10, 6,
-1, 1, 40, -27, -27, 20, -2, 6, -9, -24, -4, 1 },
{ -2, 2, -7, -8, -3, -1, -8, -5, -3, -15, -5, 9,
-25, -33, 6, -6, -22, -39, -29, 23, 2, -4, -9, 36,
43, -5, -35, 27, -31, 3, 17, -26, 16, -12, 26, -2,
10, -19, -21, -7, 26, -18, 13, 3, 2, -10, 12, 6 },
{ 4, 6, 3, 3, 4, 7, 7, 11, -5, -9, 13, -14,
8, 12, 0, 35, -20, -34, -12, 11, -7, -4, 25, -27,
-30, 7, 29, 17, -43, 12, 10, -10, 2, 19, -24, -4,
-23, 40, 1, -27, 33, -14, -7, 0, -15, -26, 24, -4 },
{ 2, 6, -5, 3, -4, 4, 11, -11, 12, 2, -5, 6,
7, 3, 15, 19, 17, 53, 14, 4, -4, -1, 2, 3,
2, -25, -28, 32, -29, 2, -34, 42, -10, -8, 14, -8,
10, -15, 16, -47, 16, 11, 8, -5, 8, -37, 22, -2 },
{ 1, -7, 7, 5, -4, -1, -7, 6, -8, 9, 44, -28,
2, 4, 0, -21, 0, -6, 10, 8, -3, -4, 39, -19,
-20, -25, -78, 7, 12, -8, 5, -2, 1, 20, 11, -17,
-20, 6, -6, 10, -5, 0, 24, -18, -30, 10, -7, -6 },
{ 6, 1, 6, 4, -3, 0, 2, -9, 8, 21, 90, 15,
4, -4, 1, -3, -15, -3, -26, -55, -23, 16, -16, 8,
14, 4, 14, -2, 5, 22, -2, 4, -9, -5, 4, -6,
3, -20, 19, -7, 9, -7, -6, 11, -4, -5, 4, 10 },
{ -1, 1, -1, -7, 3, -3, -2, -3, 0, 2, -7, 12,
-12, 4, -1, -8, -13, -12, -14, -26, 8, 29, -37, -1,
-41, -41, -25, 46, 3, 0, 0, -13, 8, 11, -42, 24,
13, 8, -1, 1, -11, 1, -19, -23, 40, -2, -10, -26 },
{ -3, -6, -5, 4, -10, -3, 0, -8, -2, 1, -52, -25,
0, -4, -6, -6, -26, 6, 5, -58, 20, -1, 23, 1,
11, 9, -16, -6, 20, 53, 4, -17, -13, 3, 8, -9,
-12, 0, 43, -8, 18, -26, 6, -11, 1, -6, 5, -9 },
{ -1, 0, 3, 3, 7, -2, -5, 3, 8, 2, 4, 16,
10, -1, -6, 4, 20, 3, -14, -13, -5, 11, -30, 0,
35, 46, -15, 1, -7, -15, -3, 9, 3, -13, 7, -16,
-30, 62, 0, -1, -5, 5, 26, -49, 10, 0, 2, -38 },
{ -1, 3, -1, 0, 6, 1, 1, -1, 4, 7, 16, 25,
11, -3, -4, 3, -16, 15, 38, 57, -15, 10, -39, -21,
-7, -5, -17, -41, 21, 42, 1, -33, 21, -6, -1, -1,
-5, -3, 18, -4, 21, -19, -4, -10, 17, -6, 18, -9 },
{ -4, -3, -4, -4, -1, -7, -5, -9, -2, -12, -16, -10,
-12, -2, -3, 0, -4, -17, -36, -11, -13, 18, -10, -2,
-14, -20, -26, -79, -4, -1, 10, 49, -22, -5, -11, 1,
3, 4, -24, 4, 25, 17, -6, 0, 6, -8, 30, -1 },
},
},
{
// 5
{
{ 108, -42, -38, -25, -12, -7, -4, 9, 9, -7, -2, 12,
10, 2, -2, -1, 2, 0, 3, 7, 0, -1, 2, -3,
-3, 1, 1, 0, 0, -3, -3, 1, 0, 0, -1, 0,

```

```

0, -1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0 },
{ 32, 13, 92, -14, -32, -14, 1, -20, -48, -28, -12, -8,
7, 1, -3, 0, -3, 8, 26, 14, -5, -3, 10, 15,
5, 3, 0, 2, 1, -6, -1, 3, -1, 2, -3, -8,
-2, 0, -1, 0, -2, -1, 1, 2, 0, 0, 1, 0 },
{ -41, -9, -6, -63, -46, 12, 4, 21, 5, -5, 12, 54,
41, 3, 2, 1, 2, -1, 17, 29, 6, 2, 1, -22,
-17, 2, 1, 0, -3, -19, -23, -3, 3, -1, -8, -1,
0, -3, -2, 6, 8, -1, 4, 4, 1, 0, -1, -1 },
{ 26, 73, 21, -46, 27, -16, -12, -19, 26, 6, 8, 27,
-11, -13, -4, -4, 0, -4, -31, -16, 4, 3, -24, -35,
-2, 6, -2, 0, 3, 12, -4, -11, 2, -4, 13, 24,
5, -2, 1, 1, 12, 8, 0, -7, 1, -1, -7, 0 },
{ 4, -47, 59, 38, -31, 9, 5, 17, 37, 33, -6, 15,
11, 0, 2, 2, -1, -16, -33, -29, -11, -5, -15, -32,
-21, -5, 0, -3, -1, 4, 4, 2, -2, 3, 13, 21,
11, 0, 4, 8, 8, 4, -3, -4, 0, -4, -5, -1 },
{ -12, 1, -17, -38, -62, -28, 11, -15, 1, 27, 3, -14,
1, 19, 2, 2, 7, 19, -14, -38, 6, 3, 0, 23,
23, 4, 3, 1, -4, 31, 48, 10, 0, 4, 20, 5,
-7, 0, 2, -16, -23, -1, -18, -13, -2, 1, 2, 4 },
{ 16, 32, -19, 45, -24, -25, 6, -22, -29, 61, 32, 16,
15, 17, -3, 0, 0, 19, 29, -5, -6, -9, -26, -11,
-7, -4, 3, 1, -16, -31, -18, -10, -3, 2, 4, -11,
-11, -4, 1, 11, 10, 8, 4, 13, 8, 2, 1, -4 },
{ -17, -63, 1, -6, 23, -15, 5, -49, -31, -17, 28, 20,
-29, 8, 12, 2, 7, 21, 6, 13, 6, 5, -29, -24,
20, 8, -1, 1, 4, 13, -7, -12, 2, -1, 25, 27,
-6, -3, -4, -5, 18, 17, -6, -12, 2, 0, -12, 1 },
{ 4, -22, 37, -22, 27, 6, 3, -3, 45, 40, 40, 12,
-18, -2, 1, 0, -3, 8, 3, 11, 16, -6, -4, 28,
24, -2, -3, -2, -11, -4, -5, -18, -4, -6, -22, -43,
-28, -1, -4, -20, -24, -6, 5, 13, 5, 14, 20, 5 },
{ -12, -4, -2, 20, 4, -74, 6, -38, 61, -28, -42, 18,
16, 19, 7, 1, 10, 16, 8, 10, -8, 3, 18, 4,
-4, -4, 0, -8, -10, -9, -7, 3, 3, -4, -12, -6,
-3, -5, 2, 2, 1, -4, 7, 3, 2, 1, 0, 0 },
{ 1, -6, 9, -11, 56, 17, 15, 7, -8, -15, -7, 18,
46, 22, -6, -2, -15, 20, 29, -41, -9, 10, -17, -7,
-17, -17, -4, -2, -24, -19, 35, 18, -1, 6, 24, -4,
-11, -2, 10, 3, -25, -9, -26, -2, 11, 1, 6, 2 },
{ -13, -25, -1, -51, 10, -11, -8, -16, 1, 23, -22, -45,
-16, -16, 1, 4, -2, -3, 6, -37, -25, -9, -13, 7,
-8, 1, 3, 7, -3, -33, -12, 9, 2, 13, 0, -11,
9, 8, 8, 37, 29, 8, 25, 35, 11, -6, -1, -18 },
{ 0, 3, 8, -6, -1, -32, 31, 42, 10, -29, 33, -56,
10, 40, -8, 1, -10, -1, -11, 5, 40, 10, -22, 2,
-12, -25, -1, -4, -11, -4, -18, -29, 5, -5, 11, 4,
3, 9, 9, 2, 15, 20, 0, 1, 4, 1, -6, -1 },
{ -6, -18, 4, -1, 21, -47, -33, -12, -24, 11, 44, -2,
32, -23, 0, 7, 5, -22, -8, -12, 36, 13, 25, -3,
-34, 5, 10, 6, 14, 5, 18, 6, -9, -11, -23, 3,
30, 6, 5, 12, -18, -22, 5, 0, -14, -13, 11, -2 },
{ 2, 0, 13, -16, 31, -20, 23, 30, 4, 60, -12, -10,
-4, 10, 3, -2, 6, 7, 31, 39, -5, 5, 27, 11,
7, 6, -7, -1, 9, -3, -2, 24, 3, 1, 13, 29,
15, -3, -12, 4, 13, -7, -18, -32, -18, -21, -34, -1 },
{ -6, -2, 2, 11, -4, -28, -57, 39, 19, -14, 7, -14,
26, -49, -4, 4, 13, 0, 37, 1, -10, -8, -41, 9,
27, 19, 7, -1, 9, 3, -2, -4, 0, 11, 31, 2,
-16, -6, -12, -11, 0, 9, -16, -3, 3, 5, -4, 1 },
{ -5, -1, -5, -7, 7, -46, 28, 25, -21, -6, 7, 25,
-31, 4, -5, 2, -11, -46, -14, 25, -50, -35, -24, 5,
-16, -12, -2, 2, 12, 3, 12, 9, -10, 16, 14, -12,
5, 10, 3, -5, -24, -8, -8, 10, 2, 8, 22, 3 },
{ 3, 5, -5, 6, -26, 18, 12, -37, 38, -9, 10, -3,
-35, -30, -11, 3, -25, -26, 42, 20, 31, 14, -12, 10,
-17, -19, 5, 12, -2, -24, 18, 5, -1, -1, 27, 10,
18, 17, 12, 20, -14, -10, -29, -2, 3, -13, -1, -5 },
{ -4, -7, 0, -4, -7, -25, -6, 31, -23, 14, -43, -8,
-43, -20, -7, 1, -2, 18, 2, 0, 30, 27, -4, -51,

```

```

-3, -1, -4, 3, -22, -18, -9, 4, 6, -25, -11, 6,
-17, -6, 5, -11, -28, -23, -7, -1, 19, 18, 22, 17 },
{ -5, -11, -6, -10, 8, 2, 0, -26, -9, 26, -43, -10,
33, -14, -32, 4, -41, -9, 11, 6, -2, -19, -12, 8,
-14, -17, 5, 11, 7, 21, -13, -51, -2, -13, -9, 8,
15, 11, -1, -28, -6, 27, -1, -23, -11, 19, 9, 26 },
{ 3, 6, 0, -8, -1, 18, 16, -17, 12, -2, 2, -42,
7, -2, 48, 0, 52, 9, 15, 15, -4, -44, -17, -25,
-34, 17, -9, -16, -6, -8, 7, -14, -18, 3, -4, 13,
16, -16, -6, -13, -28, -9, 1, -8, -11, 4, 20, 11 },
{ 5, 3, 0, 6, 2, -7, 60, 12, -5, -3, -16, 21,
2, -16, 28, 6, 15, -36, 39, -45, 20, 28, -24, 7,
12, 20, -6, 3, 33, 14, 0, -13, 9, -15, -16, -7,
2, -2, -17, -8, 5, 4, 24, 6, -10, -3, 0, -7 },
{ 1, -1, 0, -6, 2, 2, -28, -13, -17, 14, -18, 4,
5, 13, 48, -6, 29, -58, -19, 1, 2, 33, 4, 34,
-12, -15, -10, -16, -21, -18, -20, -18, 10, -6, 24, -8,
-19, -2, 7, -3, 5, 6, -34, -13, 13, 12, -4, 10 },
{ 0, -2, -4, -4, -7, 1, 17, -28, 9, -7, 26, -36,
20, 3, -14, -6, -25, -24, -10, -13, -30, 22, -3, -27,
18, 13, -2, -1, 23, -21, -43, 32, 9, 6, -11, -6,
-15, -10, -26, -15, -8, -35, -22, -23, -8, 12, -1, 22 },
{ -3, 0, 2, -2, -5, 4, -34, 17, 7, -12, 3, 15,
-33, 53, -7, 0, -3, 6, 16, -32, -21, 3, -6, 20,
7, 27, 14, 9, 9, -33, -11, -37, -20, -25, -9, 16,
32, 4, -8, 15, -14, -14, -5, -22, -26, -9, 13, 12 },
{ -1, -1, -5, -2, -8, -10, 19, 8, -3, -16, 28, 3,
-7, -46, 10, -1, -10, -3, 9, -27, -30, -22, 24, 6,
10, -25, -15, -18, -46, -4, 3, -14, -26, -34, -27, 16,
-17, -7, 15, 8, 16, -1, 1, -32, 3, -9, -28, 16 },
{ 0, 1, 2, -4, -7, 5, -35, -1, 6, 4, 10, -4,
-13, 20, 12, -14, 6, 2, 34, 2, -24, 21, -36, -1,
-14, -56, -14, -12, -5, 41, -1, 26, 34, -4, -34, 8,
15, 17, 4, -17, 3, -20, 26, -2, -8, 2, -7, 3 },
{ -1, -3, 2, 0, -2, 0, 9, 1, 3, -6, 14, -1,
-20, -15, -39, -7, 31, 21, -6, 1, -26, 28, -2, 15,
-68, 25, 0, 5, 8, -11, 24, -1, 28, -17, -7, -27,
-8, -11, -13, -15, 14, 31, -11, -13, -4, 14, -20, 15 },
{ -5, -6, -4, 0, -5, -7, -11, -2, -7, -2, -13, 8,
-7, -2, -14, -63, -13, 6, -6, -13, 32, -25, -16, 7,
-9, 2, -73, -29, 20, -6, -7, -3, -12, 14, -3, -24,
4, -13, -9, 7, -4, -7, -11, -5, -10, -21, -18, -5 },
{ 1, 2, 0, 3, -2, -8, -2, 9, 4, -3, 20, 9,
-15, -13, 23, -15, -23, 8, 17, -23, 0, 12, 47, -3,
-5, 1, -19, -8, -12, -16, -12, 5, 19, 55, 19, 15,
28, 7, -4, -27, -5, 40, 19, -4, -14, 34, 30, 12 },
{ 1, 3, 1, -2, 1, -3, -4, 6, -4, -7, -8, 13,
-18, -4, 21, 41, 11, 19, -2, -28, 25, -36, 1, -9,
-11, -37, 27, 18, 15, 1, -30, 17, 3, 25, -12, -42,
1, 11, -10, -3, 4, 10, -39, -19, -24, -11, -22, 16 },
{ 2, 2, -1, 4, -2, -1, 19, 6, -2, 1, 3, 0,
18, -34, 32, -16, 2, 49, -26, 13, -31, 30, -10, 5,
19, -6, -15, 10, 17, 1, -15, -19, 15, -18, 0, -17,
35, 38, 22, 24, -17, 1, -32, 5, -6, -8, 16, -4 },
},
{
{ 76, -45, -77, 23, 32, 0, -3, 11, 13, 0, 1, -18,
-18, 0, 0, -1, 2, 0, 5, 6, -1, 0, -2, 5,
5, -2, 0, 0, 0, -4, -4, 1, 0, -1, 1, -1,
-1, 1, -1, 1, 2, 0, 0, 0, 0, 0, -1, 0 },
{ 37, 11, 2, -51, -59, -14, -3, 23, 65, 22, -1, -2,
8, 3, -3, 0, 5, -14, -41, -19, 2, -3, 7, 23,
12, 1, 1, -3, -1, 8, -1, -2, 1, 2, -2, -12,
-5, 2, 1, 1, 4, 0, 0, 3, 0, -1, -2, 0 },
{ -52, 25, -12, 26, 11, -9, 1, 15, 31, 23, -5, -65,
-44, 1, 0, 1, 0, -2, 8, 7, -2, -1, 8, 42,
28, -2, -1, 0, -4, -24, -23, -1, 1, 0, 3, -7,
-3, 5, -1, 11, 11, 0, -4, -3, -2, -1, -2, 1 },
{ 38, 0, 33, -22, -34, -19, 2, 19, -12, -1, -17, -14,
-10, -3, -1, 1, -1, 6, 55, 45, -3, 2, -2, -17,
-2, 3, -1, 2, -5, -43, -35, 0, 0, -2, 17, 35,

```

```

13, -1, -5, 9, 5, -4, -7, -18, -3, 1, 3, 4 },
{ -38, 2, -33, -12, 8, -3, 6, 28, 45, 20, -10, 11,
-18, -13, 3, 1, 2, 2, -4, 6, 13, 6, -21, -57,
-17, 3, -2, -1, -4, 13, 15, -4, 0, -2, 21, 49,
19, -2, 0, -22, -29, -6, -1, -13, 0, 8, 14, 0 },
{ 31, 7, 39, 3, 0, -19, 13, -4, -9, 7, -12, -43,
-36, -15, -3, 1, -2, 24, 16, -4, 15, -8, -11, 7,
2, -1, -1, 2, -3, 31, 50, 3, -3, 4, -5, -14,
2, 4, -1, -34, -46, -7, 18, 26, 6, 7, 11, -10 },
{ 30, 11, 38, 13, 30, -5, -12, -27, -4, -4, 1, -8,
-21, 9, 2, -1, -5, -16, -41, -36, -9, 11, 4, 2,
28, 5, -2, 0, 1, 8, 1, -10, -2, -5, 32, 48,
5, 0, 5, 3, 0, 4, -29, -47, -10, 6, 13, 11 },
{ -15, -72, 35, -56, 40, 27, 6, 6, 12, 25, 10, 9,
-24, -18, 7, 2, -3, 11, 11, -21, -7, -1, 9, 7,
12, 6, -5, 3, -13, -22, 3, 4, -2, -1, -13, -6,
-1, 1, 3, 13, 2, 1, 6, -1, -4, -3, 0, 2 },
{ 23, 29, 26, 10, 32, -2, 1, -8, 23, 18, -14, -6,
-19, -28, -6, -2, 9, 5, -13, -3, -5, -10, -18, -40,
-36, -8, -3, -1, 12, 11, -13, 8, 4, -4, -17, -4,
9, 1, -3, 23, 52, 12, 16, 17, 7, -22, -42, 4 },
{ -9, -31, 29, 24, -7, -14, 4, 48, 12, -58, 22, 13,
-26, -2, 1, 3, 2, -31, -16, 23, 4, -1, -38, -10,
30, 3, -3, -4, 15, 12, 5, 12, -1, -2, 13, -9,
-27, -1, 1, 10, 5, -6, -1, 9, 13, -14, -11, 1 },
{ 6, 53, -33, -43, 17, -36, 3, 3, -26, 3, 34, 27,
-23, 1, -4, 1, -1, 12, 22, -21, -2, 1, -24, -5,
31, 5, -2, 4, -8, -17, 25, 21, -1, -6, -1, -6,
-23, -2, 8, 25, -1, -6, 0, -6, 3, -16, -6, 11 },
{ -19, -29, -18, -21, -30, -2, 15, -17, -15, -6, -14, -24,
-6, 1, -4, 2, 19, 33, 7, 10, 9, 2, 6, 8,
3, -7, -1, 2, 21, 47, 27, 19, 9, -9, 10, 10,
-5, 0, -6, 0, 31, 16, -32, -35, -4, -20, -37, 25 },
{ 4, 22, -6, -52, 30, 32, -12, -36, 9, -23, 7, -3,
-9, -3, -1, -5, 5, 8, -6, 49, 15, 8, -18, 8,
17, -1, 0, -2, 8, 20, -34, -26, 3, 7, 25, -10,
-17, 3, -11, -28, 4, 6, -9, 19, 16, 13, 2, -16 },
{ -13, -14, -20, -17, -20, -27, -4, -18, -25, -27, -4, -7,
-23, -5, 18, -1, -6, -5, -19, -16, -14, 4, 13, 0,
18, 33, 6, -3, -4, 4, -9, -3, -1, 17, 5, 34,
40, 6, -1, -1, 18, 7, 49, 32, -6, 0, -22, -38 },
{ 4, 24, -2, -17, 28, -14, 16, 42, -7, -38, -11, 11,
-10, -28, -1, 1, -4, -8, -8, 18, 2, -4, 52, 10,
-1, 11, -7, -1, 4, 18, -7, -9, 9, -16, -51, -5,
11, -6, -1, -21, 2, 17, 5, -27, -22, 18, 7, 19 },
{ -9, -19, 2, -23, 35, -60, 0, 18, -33, 45, -36, -9,
25, 29, -1, 5, -5, -38, -13, 12, 3, 2, -1, -2,
-2, -6, -1, 5, 32, 15, -9, -9, -1, 3, 8, -4,
-18, -6, -9, -1, 2, 1, 1, 10, 17, -2, 7, -5 },
{ 3, 17, -7, -24, 5, 54, -37, 6, -10, -17, -37, -26,
1, 8, 3, -2, 0, -38, -4, 10, 14, -7, 10, -1,
0, 1, -4, 4, 23, -4, 24, 31, -9, 6, -7, 11,
6, -5, 6, 29, -21, -38, 21, -6, -5, -30, -1, 6 },
{ 10, 14, 14, 11, 29, 8, 17, 25, 15, 17, 15, 16,
30, 28, 11, -10, -15, 7, 14, 14, 3, 13, 19, 21,
16, 13, 5, -16, -16, 7, 8, 13, -8, 10, 16, 17,
12, 1, -21, -31, -8, -20, 3, -1, -6, -25, -68, -11 },
{ -8, -9, 8, -20, 22, -40, -20, -23, 36, -37, 50, -32,
20, 37, -3, 2, 6, -3, 14, 2, -18, -7, -3, 1,
-34, -27, 1, 2, 12, -2, 5, 20, -1, -6, -14, 4,
27, 11, -9, -3, -5, -10, 8, -7, -14, 5, 7, 4 },
{ 1, 12, -5, -20, 9, 20, 25, 49, -26, -19, 11, -31,
18, -30, -16, 0, -2, -6, -4, -33, -31, 19, -3, 13,
-33, -14, 6, -3, 2, -1, 3, 6, -3, 7, 38, -1,
19, 19, -3, 4, 5, 4, -36, 19, 16, 2, 5, -19 },
{ 5, 7, 2, 7, -21, 31, -33, 19, -18, 38, 41, 9,
-35, 16, 4, 0, 14, -15, 18, 0, -46, 12, 16, -12,
7, -10, -7, 6, 40, 23, -2, 9, 1, -14, -14, 7,
-5, -1, -29, -24, 6, 13, 9, -1, 5, 13, 5, -14 },
{ -7, -11, -9, -4, -14, -14, -14, -23, -15, 4, 4, -2,
-15, -35, -7, -14, -14, -23, -16, -3, -22, 3, 2, -5,

```



```

-9, 0, 19, -19, -11, -11, -21, -7, -19, -14, -9, -9,
0, 22, -24, -30, -26, -37, -11, 2, 27, -15, -26, 58 },
{ -4, 0, 4, 7, 9, -22, -32, -1, 28, -30, -50, 35,
-16, 6, -8, -1, 32, 30, 19, -18, -20, 34, 43, 5,
-1, -18, -1, -5, 3, -4, 5, 0, -9, -2, 10, -1,
-16, 6, -10, 6, -7, -8, -11, 14, 32, -16, 6, -17 },
{ -2, 3, -2, -13, -3, 18, -18, 23, -5, -23, -30, -18,
-10, 58, 11, -8, -36, 5, 5, -29, -12, -7, -17, -21,
-3, -18, -2, 1, -35, -8, 2, -15, 10, -6, -8, -4,
-15, -3, 10, -22, 7, 35, 8, 6, 16, 22, -28, 28 },
{ -1, -3, 4, 1, -1, -7, -53, 33, -28, 11, 23, 17,
-23, 8, -1, 0, 18, 20, -15, -1, 61, -6, -2, 21,
-23, -27, 2, -3, -6, 0, -6, -33, -5, 16, 8, -2,
31, 25, 4, 6, 0, 6, -3, 1, -14, -16, -7, 10 },
{ -4, -4, 1, 2, 4, -18, -70, 4, 0, 9, -16, 6,
10, -35, 16, -1, -21, 2, 0, 14, -15, -5, -22, 13,
-4, 21, -2, 5, -19, 5, 8, 43, 15, -12, 6, -16,
5, -10, 14, -13, 8, -1, -36, -3, -22, 32, -16, -24 },
{ 1, 0, -1, 1, -7, -6, -11, -16, 18, -3, 25, -9,
-5, -14, 19, 6, -49, -36, 34, -4, 31, 30, 33, -11,
-14, 15, 7, 25, 14, 2, 22, -34, 0, -10, 1, -4,
-3, -10, 17, 14, -3, 26, -27, 14, 16, -20, -14, -7 },
{ 0, 1, 5, 3, 7, -2, -36, 22, -1, 4, 20, -39,
50, -20, 19, 0, 27, 37, -9, 12, -22, 20, 1, -19,
23, 33, 8, -8, -10, 8, 9, -26, -12, 0, -6, 9,
-15, -6, 9, 13, -4, 19, 22, 2, 16, -18, 13, 27 },
{ 0, -4, 1, -3, -6, 0, 10, -5, -17, 11, 11, -11,
-12, 16, -21, 7, -7, 9, -46, 42, -6, 54, 14, -15,
-6, -28, -16, -4, -36, -8, 8, 8, -5, -42, -18, 0,
-4, 5, 34, 14, -10, -10, 0, 0, 0, -1, -12, -30 },
{ 3, 2, 5, 4, 7, 8, 4, -3, 5, 10, -6, 13,
0, 20, -11, -11, 11, -16, 5, 17, 15, 15, 0, 8,
14, 24, 52, -17, -8, 0, 18, 34, 34, -24, -3, 6,
33, 46, 2, 8, 13, 28, -4, 26, 24, 10, 27, 26 },
{ 4, 7, 1, -1, -1, 0, 20, 5, -2, 2, -2, 11,
-19, 17, 91, -3, -25, 34, -24, 16, -15, 8, -1, 12,
-25, 2, 2, 15, 17, 6, -8, 14, -7, -1, 5, -9,
9, 3, 1, 12, 4, -20, -10, 10, 6, -9, 26, 20 },
{ -2, 1, -2, 1, 2, 1, -1, -3, 11, 7, -15, 25,
-12, 6, -37, 10, -25, -4, 6, 35, -59, 7, -19, 25,
-4, 2, 6, 19, 2, 17, 25, -29, -26, 34, 11, -9,
23, 7, 21, 15, -3, 18, 10, -1, -27, -13, 1, 25 },
},
{
{ 82, -51, -74, 14, 33, 2, -3, 8, 7, 1, 1, -7, -9, 1, 0, -1,
1, 0, 2, 2, 0, 0, -1, 1, 1, -1, 0, 0, 0, -1, 0, 1,
0, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 64, -10, 41, -78, -50, -14, -1, 11, 27, 8, -2, 17,
17, 0, -3, 0, 1, -3, -12, -4, 1, -1, 3, 0,
-1, 3, 0, -1, -1, 1, 0, -1, 0, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ -41, 15, -52, -26, -8, -2, 2, 26, 65, 64, -3, -20,
-13, -1, 1, 1, 4, -6, -27, -20, 4, 0, 4, 12,
7, -1, 0, -2, -4, 2, 1, -2, 2, 0, 0, -2,
0, 1, 1, 0, 0, -1, 0, 0, 0, 0, 0, 0 },
{ -39, -44, -19, -44, 27, 39, -1, -11, -4, -9, 41, 64,
25, -2, 6, 0, 0, 0, -12, -16, -2, 6, -11, -29,
-12, 2, 0, 0, 0, 9, 11, 1, -1, -2, -3, 3,
-2, -3, 1, -1, -2, 1, 1, 0, 1, 0, 0, 0 },
{ 12, 72, -58, 3, -26, -44, -2, 13, -26, -16, 20, 38,
34, 13, -5, -1, 1, -7, -5, 1, 4, 1, -13, -19,
-10, 1, 3, -1, 3, 5, 3, 0, 2, -1, 3, 4,
-1, -3, 0, -2, -1, -1, -1, 0, 1, 0, 0, 0 },
{ -31, -37, -21, 3, -51, 5, 11, 49, 22, -46, -4, 3,
-7, -2, 3, 3, 4, -6, 25, 59, 20, 1, -4, -7,
2, 1, 0, -1, 1, -17, -28, -4, 3, 1, 5, 6,
1, 0, -3, 0, 3, -3, -1, -2, 0, 0, -1, 0 },
{ -7, 23, -21, -71, 36, -1, -2, -25, -30, 17, -12, -28,
0, -3, -1, -1, 2, 25, 50, 42, 16, 3, 2, 2,
-3, 1, -1, 5, -4, -22, -21, -6, 4, 1, 1, 5,
3, 0, -4, 1, 4, 1, 0, -2, 0, 0, -1, 0 },

```

```

{ 12, 30, 24, -9, 46, -18, 2, 18, 42, -30, 42, -2,
  -53, -25, -2, 0, -2, -2, 2, 3, 5, -6, -47, -31,
  11, 3, -5, -2, -2, -2, 1, 0, -1, -3, 18, 19,
  -1, 2, 0, -1, -4, -3, 0, -1, 4, 0, 2, 0 },
{ -6, 14, -17, -30, 8, 3, -8, -8, 6, -72, 24, -16,
  -11, 5, 1, -1, 0, -8, -16, -19, -3, 10, 36, 59,
  36, 6, 0, -1, 5, 5, 0, -3, 2, -1, -18, -31,
  -14, 4, 1, 3, 7, 3, 1, 2, -3, 0, -1, 0 },
{ 18, 36, 3, 15, 14, 57, -37, -25, 39, -9, -27, 20,
  15, 7, -2, -6, 2, -14, -27, 24, 27, 3, 4, -8,
  -13, -7, 0, -2, 6, 1, -41, -35, -1, 0, -1, 1,
  2, -1, -2, 1, 21, 11, -2, -1, 0, 3, -3, 0 },
{ -11, -17, 19, -3, 59, -60, 35, 28, 1, 2, -27, 14,
  36, -6, 0, 5, -13, -35, -27, 14, 13, 7, 15, 7,
  0, 4, -2, -3, 8, 2, -23, -20, -2, -3, -10, -6,
  -3, -3, -1, 1, 11, 7, 2, 1, 1, 2, -1, 0 },
{ 7, -5, 12, 20, -7, -7, -3, -2, -4, 38, 73, 12,
  3, 16, 3, 1, -5, -4, 11, 4, -28, 2, 5, 17,
  16, 1, 1, -1, -12, -47, -51, -21, -5, -1, 0, -8,
  -9, 0, 0, 21, 28, 11, -6, -5, -1, 1, -3, 1 },
{ -14, -29, -6, -1, -12, -39, -4, -66, 20, -13, 14, -36,
  23, 44, 8, 2, 0, -11, -14, 6, 16, -11, -35, 0,
  5, -8, 5, 1, 13, 15, 0, -7, 0, 14, 39, 14,
  -3, 1, -2, -3, 3, 4, -16, -5, 3, 0, 0, -1 },
{ 1, -5, -4, -13, -12, 18, -6, 20, -66, 18, -8, 4,
  -33, -15, 2, -1, -3, -14, -48, 3, 28, -10, -24, 13,
  42, 19, 0, 3, 4, 12, -10, -19, -3, 17, 37, 12,
  -11, -1, 3, 0, 7, 9, -15, -6, 3, 0, 0, -1 },
{ -6, -11, -10, 9, -16, -40, 14, -47, 13, -2, -28, 56,
  -46, -19, 2, 4, 12, 41, 8, -19, 27, 12, 12, 5,
  12, 8, 4, -2, -21, -10, -5, -24, -3, -9, -5, 3,
  3, -1, 6, 3, 10, 17, 3, -3, -2, 1, -2, 0 },
{ 8, 0, 10, 14, -1, 4, 13, -4, 2, 23, 35, 5,
  7, -2, -6, 1, 5, 15, 3, 27, 70, -34, -17, 18,
  3, -6, -2, 0, 2, 19, 17, 1, 3, -3, -31, -51,
  -21, -2, -3, -14, -16, -6, 18, 26, 4, -3, -2, -2 },
{ -1, -7, -5, -5, -21, -7, 12, -12, -18, 0, 32, -40,
  -18, -34, -9, 0, 14, 0, -24, 8, -9, 14, 19, -35,
  -31, -4, 2, 2, 23, 38, -10, -29, -1, -18, -29, 5,
  19, 7, -8, -12, 28, 35, 18, 7, -1, 4, -13, 1 },
{ -9, -12, 6, -6, 5, -34, -82, 7, -9, 14, -5, 17,
  -38, 34, 32, 0, -6, -10, -8, 30, -10, 11, 6, -9,
  1, -5, -6, 8, 17, 12, 8, 14, 2, -20, -25, -8,
  -8, -3, -3, 2, -2, -6, 18, 4, 1, -3, 0, 2 },
{ 4, 9, -10, 1, -9, 11, 37, -46, 10, 13, -12, 18,
  -12, -17, -19, 3, 1, -40, -9, 53, -48, 10, -12, -4,
  31, 24, 7, -2, 13, 4, 10, 27, 9, -13, -10, -11,
  -23, -9, 0, 4, -1, -13, 5, 5, 7, -4, -2, 0 },
{ 8, 6, 7, 8, 1, 8, 15, 0, 4, 14, 26, 2,
  -5, 21, 3, -4, -12, -5, 7, 21, 18, 63, 39, 0,
  13, 7, 3, -1, -1, 17, 17, -28, -23, -5, 10, 22,
  4, 0, -5, -34, -49, -14, -16, -21, -6, 12, 23, -1 },
{ -5, -7, 1, 12, 2, -19, -48, -4, 9, 2, 12, -6,
  46, -74, 6, -1, 34, 33, -19, 25, -14, 4, 8, 11,
  22, 7, 1, -11, -14, 2, 0, 6, 1, 6, 13, 6,
  -1, 2, -2, -11, -13, -17, -10, -5, -2, 3, 6, -1 },
{ 0, -4, -7, 0, -11, -3, -11, -27, -6, -3, 10, -14,
  0, -28, 16, -4, -25, -53, -1, -21, 36, -26, 21, -17,
  1, 28, 10, -1, -12, -35, -13, 12, -3, -7, -18, 22,
  27, 2, 9, 10, -20, -31, 33, 1, -11, 3, 19, -1 },
{ 0, 3, -1, -4, 6, -6, 16, -9, -2, 1, 13, 28,
  -32, 2, -4, -2, 2, 6, -25, 26, -19, -49, 32, 35,
  -44, -25, -1, 4, 14, 5, -17, 14, 10, 28, 20, 11,
  33, 14, -14, -17, -15, -26, -10, -2, -10, 5, 5, -1 },
{ -5, -2, -7, -6, -4, -3, -7, 0, 1, -6, -15, 1,
  -2, -3, 13, -5, -10, -6, 17, -6, -46, -28, -19, -2,
  2, 8, 5, -3, -8, 3, -5, -63, -34, 27, 0, -22,
  -9, 0, -5, -29, -30, 6, 29, 37, 12, 21, 26, -13 },
{ 0, -1, 0, 4, 0, -5, -20, 0, 11, 12, 5, 17,
  -1, -27, -5, 5, 4, -39, 68, -22, 10, -20, 27, 3,
  3, 18, 1, 13, 41, 40, -2, -1, 8, 22, 20, -2,

```

```

-16, -16, -13, -9, 16, 13, -24, -11, 1, -4, -11, 2 },
{ 5, 11, 1, 3, 5, 8, 22, 5, 9, -1, 3, -2,
12, 17, 63, -13, -19, 28, -4, 19, -11, -51, 33, -25,
38, 21, -1, -2, -13, 15, 17, 0, 17, -3, -5, 18,
1, -10, 4, -9, 11, 28, 3, -18, -13, 4, -16, 3 },
{ 5, 6, -4, 2, -11, 2, 14, -7, 3, 2, 0, 2,
2, -41, 69, -29, -56, 3, 2, -2, 2, 38, -22, 21,
-19, -32, 7, 19, 19, -2, -4, 7, 6, 7, 9, -13,
-6, 0, -3, 3, 5, -3, -6, 11, 7, -5, -11, -5 },
{ -5, -5, -5, -1, -4, -5, -23, 0, 6, -1, 3, 0,
2, -17, -45, 4, -72, 7, -4, 24, -5, -21, -5, 15,
-19, 7, 11, 8, -9, -6, 41, -16, -19, -11, -8, -4,
8, -5, 19, 19, 2, 32, -29, -24, -13, 2, 8, 15 },
{ -1, -2, 4, 6, 5, -3, -9, 7, 8, 1, 7, -1,
0, 15, -22, -8, -31, 30, -6, 5, 3, 27, 8, 0,
-11, 67, 33, -12, 1, 11, -4, 11, 17, 46, 14, 1,
15, -2, -3, 3, 14, -10, 28, 33, 5, -14, -25, -24 },
{ 0, 2, -2, -2, -2, -6, 5, -5, 0, -6, 1, -13,
-12, -4, 0, 4, -1, 13, -21, 3, 6, -3, 48, -57,
-9, -12, -11, 11, 5, -17, 3, 1, -31, 39, 16, -35,
-41, -20, 16, 33, -1, -23, -22, 5, 18, -15, 8, 5 },
{ -2, -6, -1, 0, -4, -3, -3, -1, -3, -4, -4, -8,
-1, 2, -23, -6, -36, 28, -8, -23, -6, -9, 2, -18,
-1, 8, 3, 4, 9, 6, -46, -3, 62, -41, 3, -12,
-32, -17, -11, -26, -20, -23, -15, -9, 13, 22, 21, 8 },
{ -1, 1, -2, 1, 0, 3, 8, 2, 6, -2, -5, 2,
9, 3, -5, 1, -8, 38, -6, -10, -1, -12, -15, 8,
15, 15, 6, 21, 54, 6, -31, 22, -76, -18, -17, 18,
1, -17, -25, 1, 1, -16, 15, -18, -7, -10, 1, 9 },
},

```

```

},
{

```

```

// 6
{
{ 109, -39, -29, -27, 27, -20, 1, -3, 6, -2, -2, -4,
12, -4, -3, 0, 1, -2, 2, 0, -1, -1, 0, 0,
3, 0, 0, -1, 0, 0, 0, -1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 6, 68, -81, 21, 11, -18, -15, 40, -40, 20, 2, -4,
1, -5, -3, -3, 1, 14, -15, 1, 3, 1, 0, 0,
2, -2, 0, 0, -1, 1, 1, 2, -2, -1, -1, 1,
0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ -24, -73, -65, 35, 13, 58, 7, 3, 8, 19, 3, 4,
-26, 9, 1, 2, 4, -1, -7, 5, 2, 1, 1, -1,
-4, 1, 0, 1, 1, 1, -1, -1, -2, 0, 1, 1,
0, -1, 0, -1, 0, -1, 0, 0, 0, 0, 0, 0 },
{ -40, -5, -3, -23, 91, -7, 4, -23, -4, 14, 7, -39,
46, -27, 4, 2, 1, -7, 3, -3, 1, 2, -3, -11,
21, -8, -5, 1, 1, 0, 1, -4, 3, 0, 0, 1,
-2, 0, -2, 5, 0, 0, -2, 0, -1, 0, 0, 1 },
{ 22, 26, 24, -31, 23, 86, -59, 6, -13, -6, 0, -15,
-20, 7, -11, -3, -16, 29, -6, -3, -1, 0, 0, -5,
3, 7, -3, -2, -2, -1, -3, 7, 0, -2, 0, 0,
-1, -2, -1, 1, 3, 2, -1, 0, 2, 0, 0, 0 },
{ -20, -8, -21, -92, -44, 7, 16, 30, 17, 40, 26, -10,
19, 11, 0, 0, 1, -2, -19, 4, 3, 2, 6, -14,
1, 6, 2, 0, 0, 0, 0, 0, -3, 0, 1, 2,
1, 0, -4, -1, 0, 2, 1, -1, 0, -1, 0, 0 },
{ -3, -44, 36, 10, 0, -10, 7, 53, -52, -6, -7, 13,
22, -26, 4, 3, -13, 41, -49, 22, -2, -1, 1, -5,
16, -15, 2, 1, 2, -3, 0, 17, -19, 3, 3, -1,
1, 0, -1, 7, -6, -1, 0, 1, 2, -4, 0, 1 },
{ 5, 3, 1, -14, -5, 9, 30, -66, -80, 18, -5, 6,
3, 40, -7, -3, 19, -1, -10, 28, 0, 0, 1, 2,
-15, 17, -5, -3, -1, 1, 10, -4, -14, 12, 2, 0,
1, 1, -2, -3, 5, 0, 0, 1, -3, -4, 0, -1 },
{ -13, -20, -28, -8, -30, -6, -62, -29, -19, -40, 17, 19,
46, 9, 45, 2, 1, 16, 27, -1, 12, 3, 3, -13,
5, -29, 11, 4, 1, 3, 1, -3, 6, -5, 2, 2,
1, -1, -1, 4, -7, 1, 1, -1, -2, 0, 0, 1 },

```

```

{ 3, -7, 13, 9, -9, -27, -53, -30, 19, 85, -24, -9,
  -20, -12, 28, 7, -2, 16, -17, 20, -15, -3, -3, 15,
  3, -12, 8, 4, 2, 2, -5, -4, -4, 14, -3, -2,
  0, 2, 3, -2, -3, 2, 0, -3, -2, -1, 4, -1 },
{ -22, -13, -8, -38, 36, -40, -6, 1, -4, -23, 19, 19,
  -55, 4, 18, 3, -7, 40, 6, -10, 8, 4, -6, 35,
  -44, 20, -2, 3, 2, -1, -7, 17, -11, -3, 0, 0,
  -1, 2, 11, -21, 5, 4, -1, -2, 7, -5, -1, -4 },
{ 1, 6, -3, -27, -3, 25, 3, -5, -19, 3, -5, 54,
  -16, -80, 3, -4, 34, -28, 1, -1, 5, 1, -12, 27,
  5, -16, 11, -1, -2, 2, 19, -25, 11, -3, 0, 0,
  -1, -3, 17, -5, -11, 5, 2, 4, -8, 3, 1, -2 },
{ 0, 10, 4, 3, 28, 11, -8, 4, 21, 22, -5, 79,
  51, 34, -8, -3, -28, -18, -24, -11, -6, -3, -25, 8,
  -34, -2, -10, -1, 0, 1, -7, 12, 3, 5, -6, 0,
  -4, -5, 12, 0, -3, -4, -2, -4, 4, 0, 3, 3 },
{ 12, 9, 18, 5, 18, 13, 19, 29, -4, 5, 10, -12,
  -11, 27, 88, -45, 15, -21, -7, -3, -9, -4, 1, -8,
  -19, -24, 2, -5, -4, -12, 28, -14, 6, -1, 0, -1,
  0, 2, -3, -7, 3, 3, -3, 6, -3, 1, 0, -1 },
{ -10, -22, 6, -11, 0, -8, -21, 44, -32, 11, -58, -15,
  11, 17, -4, 10, -16, -27, 47, 17, 11, 9, -8, 19,
  -17, 22, -8, 0, 4, -1, 4, -28, 30, -21, 2, 1,
  2, -9, 9, -11, 13, -1, -2, -3, -10, 13, -1, -4 },
{ 4, -15, 19, 18, 5, -15, -29, 9, -29, 21, 79, 10,
  -11, -7, -18, 13, -8, -29, 2, -12, -22, -17, 21, -29,
  -12, 26, -1, 3, 2, 1, 4, -19, 33, -1, 3, -4,
  -2, 11, -4, -13, 6, -2, 0, 0, -8, 10, 0, -4 },
{ -7, -2, -6, -2, 3, -10, -41, 6, 17, -41, -8, -11,
  0, 3, 4, 11, 32, -49, -57, 32, 11, 6, -8, -1,
  2, 39, -12, -1, 0, -2, 12, -6, -21, 29, -5, 1,
  0, 3, 0, -12, 18, -6, -2, 8, -4, -17, 12, -1 },
{ -4, 3, -10, -16, 6, -7, 4, -9, -12, -16, -2, 0,
  -35, 5, 3, 5, -78, -50, -10, 28, 5, 4, 2, -3,
  13, -26, 37, 0, 3, 4, -1, 17, 6, 15, -2, 1,
  -1, 4, -10, 20, -30, 16, 1, 2, -11, 6, 6, 4 },
{ 4, -5, -4, 10, -24, 13, 3, -18, -15, -1, -7, -39,
  28, -27, 14, -8, -25, -19, -31, -49, 1, 1, 12, 30,
  -25, 19, 26, -1, 0, 3, -8, 38, 15, -5, 15, 0,
  3, -7, 24, -37, 9, 2, 1, -5, 15, -5, -3, -12 },
{ 6, 9, -1, 11, -5, 15, 9, 4, 14, -1, 24, -14,
  22, -35, -4, -11, -17, 5, 38, 73, -8, -1, 0, -10,
  -52, -7, 6, 2, 0, 3, -3, 9, -30, 13, -15, -2,
  -1, -3, 21, -23, 14, -3, 2, -5, 1, -7, 13, -9 },
{ -5, -6, -7, -4, -6, -7, -5, -9, -3, 4, -13, 21,
  -7, -23, 9, -80, -24, -2, 15, 6, -3, 3, -1, -21,
  31, 48, -30, 26, -1, 4, -1, 25, 9, 8, -1, 2,
  -1, 5, -16, 6, 16, -21, 7, -14, 15, 0, -1, 7 },
{ -6, -10, -1, 1, -4, -12, -17, -5, 7, -5, 19, -21,
  2, 16, -51, -54, -8, 7, -18, 3, 2, 5, 2, 42,
  -4, -40, -15, 25, 3, -8, 53, -16, 4, -4, 8, -1,
  2, -11, 21, 1, -3, -9, 7, 14, -15, 8, -3, -3 },
{ -6, -2, -10, -7, -2, 6, 4, 9, -5, -18, 3, -1,
  10, 10, 6, 6, 0, 4, 13, -2, -99, 10, 2, 44,
  28, 6, -1, -2, 0, 2, -9, -7, 4, 43, -18, 7,
  0, -8, -1, -9, -4, 0, 0, -3, -2, 5, 7, 1 },
{ 8, 2, 11, 12, -1, 7, 0, 9, -5, 15, 48, -2,
  6, 16, 16, 0, -5, -18, 18, 15, 40, -9, -22, 59,
  54, 13, -11, -7, -2, 0, -28, 1, -28, -13, 11, -8,
  -1, -21, 10, -9, -10, 1, -2, -8, 14, -12, 4, 7 },
{ 3, 11, -6, 5, 0, 5, 22, -17, 20, -13, 6, 6,
  8, 0, 18, 4, -26, 42, -31, 33, 12, -5, -17, 3,
  22, 28, 15, -8, 0, 1, -3, -39, 46, -8, 2, -1,
  -1, 0, 7, -29, 14, -5, 0, 10, -33, 36, -19, -12 },
{ 3, 2, -4, -6, -12, 2, 7, -9, -8, -9, 6, -13,
  -15, -12, 14, 18, -16, -12, -18, -1, -5, -1, -17, -1,
  -9, -43, -84, -11, 9, 33, -26, -2, 15, -1, 4, -3,
  4, 2, 16, 13, 38, -5, 1, -20, 1, 11, -2, 5 },
{ 1, 7, 1, -1, 10, 6, 2, 0, 9, 0, -10, 20,
  9, -2, 6, 2, -12, -2, -5, 5, 18, 5, 105, 29,
  2, -8, -20, 0, 0, 11, -9, -20, -15, 1, -2, 3,

```

```

-10, 11, -31, -16, 9, -2, 1, -2, -2, 4, 1, -14 },
{ -1, 2, 2, -1, 7, -2, -7, 1, 5, -5, 3, 0,
-1, 10, -8, -13, 46, -10, -13, 48, -14, 11, 19, 9,
-3, -9, -3, -2, -3, -2, -29, 53, 52, -38, -2, 2,
-4, 1, 4, -3, -32, 13, -1, -15, 10, 12, -26, 1 },
{ 0, -4, 5, 0, 3, -1, 0, 7, -8, -6, -3, -9,
-2, 7, -13, -32, 15, 1, -10, -10, 20, 4, -8, -9,
-15, -16, 16, 42, -13, 10, -74, -47, 11, 43, 0, 0,
7, 4, 3, -12, -19, 16, 2, -12, 14, 5, 12, -4 },
{ -7, -6, -6, -2, -3, -4, -8, 2, 1, -4, -7, -3,
7, -5, -14, -31, 1, 7, 8, 3, 9, -57, 2, 12,
-7, 1, -7, -77, 21, -26, -7, -3, 16, 36, 16, -1,
-8, 25, -3, 9, 0, 36, -6, 4, 6, -8, -3, 8 },
{ -2, -4, -1, 3, -2, -9, -10, -10, -2, -1, 5, 3,
-4, -6, -8, -29, -9, -6, -14, -6, -20, 61, -1, -16,
-3, 6, 4, -50, 14, -18, -24, -32, -35, -44, -8, 0,
10, -25, -1, 5, 9, 30, 0, 6, 14, 21, 5, -2 },
{ -4, 1, -7, -5, -2, 1, -5, -4, 1, -13, -11, -7,
4, -4, 4, -6, -4, -11, -12, 5, -34, -69, -7, 9,
-12, 12, 6, 28, -2, 12, -15, -23, -44, -56, 14, 10,
-9, 27, -4, 15, -10, 2, 4, 1, 6, 24, -16, 8 },
},
{
{ 109, -42, -42, -19, 1, -18, -3, 7, 9, -2, -2, 5,
6, 5, -2, -1, 2, 0, 0, 2, -1, -1, 0, 0,
-1, 0, 1, -1, 0, 1, 0, 0, 0, 0, 1, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 44, 56, 76, -28, -14, -33, -16, -34, -29, -24, -9, -3,
6, -3, -5, -3, 1, 13, 15, 2, -4, -2, 1, 6,
5, 5, 1, -1, -1, 0, 2, 0, -1, 3, 0, -1,
0, 1, -1, -1, -1, 1, 0, 0, 0, 0, 0, 0 },
{ -5, -91, 76, -2, -4, 13, 26, -10, 20, -21, -11, 5,
-2, 0, 10, 5, -8, -8, 3, 8, -4, -1, 3, 2,
-2, -5, -3, 2, 1, -2, -1, 2, -1, 0, 2, -1,
0, 0, -1, 1, 1, 0, -1, 1, 0, 0, -1, 0 },
{ -23, -9, -13, -26, -91, -35, 18, 0, 1, 17, 8, 34,
38, 34, 7, 4, 11, 10, 11, 11, 3, 1, 3, -5,
-14, -4, 4, 1, 1, 2, -3, -9, -10, -3, 1, 1,
2, -1, -3, -2, -3, -1, 2, -1, 1, 0, -1, 1 },
{ 1, 4, 6, -83, -7, 80, -8, 29, -19, 3, 17, 13,
3, -11, -16, -7, -9, 4, -6, 9, 7, 3, 0, -15,
0, 11, -1, -3, -2, -1, 7, 3, -3, -8, -1, 1,
0, -3, 1, 0, 0, 4, 0, 0, -1, 1, 1, 0 },
{ -31, -17, -10, -65, 66, -56, 4, -13, 3, 5, 21, -9,
37, -13, 20, 8, 2, 7, 12, -2, 10, 4, -7, -1,
1, 1, -5, 4, 3, -1, -4, -4, -2, -3, -2, 2,
-2, 0, 1, -4, -1, 0, -1, -1, 0, 0, 0, 1 },
{ -8, -32, -26, 6, -3, 20, -10, -69, -56, 9, 9, 6,
-33, -6, 6, 3, 20, 37, 33, 18, 0, -1, 6, 15,
21, 19, 8, 1, 1, 4, 4, -8, -8, 4, 3, 0,
1, 0, -7, -13, -9, 0, 2, -4, -4, -3, -3, 1 },
{ 1, -2, -5, 10, 12, -11, 57, 47, -77, -30, 2, -28,
3, 33, 1, -6, -17, 1, 7, 20, 4, 2, 12, 9,
-2, -9, -14, -3, -2, -5, 5, 3, -10, -5, 5, 0,
1, -4, -9, -1, 9, 4, -2, 4, 1, 0, -4, 0 },
{ -14, -23, -6, 15, -21, -10, -76, 44, -10, -53, -4, -20,
27, -26, 2, 18, 3, 26, 8, 10, 10, 6, 12, 1,
21, 2, 11, 9, 4, 0, -5, 3, 5, -8, 4, 2,
2, -5, -3, -2, -3, -7, 0, 1, -2, -2, 1, -1 },
{ 7, -19, 31, 20, 13, -15, -29, 29, -45, 82, 6, 14,
12, -6, -1, 9, -7, 26, -35, -1, -24, -14, -5, -18,
-9, -1, 0, 4, 3, -3, 0, -6, -1, 3, 3, -4,
2, 5, 6, 3, -2, -2, -1, -1, 1, 4, 2, 0 },
{ -10, -8, -2, -31, -32, -42, 6, 19, -6, 11, 8, -17,
-67, -27, 4, 7, -1, -28, -36, 3, 3, -1, 5, 22,
28, 27, 7, 2, 1, 0, 13, 28, 27, 12, 3, 1,
3, 6, 2, -3, 2, 4, 1, -3, -11, -10, -2, -4 },
{ 4, 3, -7, 21, -6, -5, 28, 1, -20, 5, -5, 40,
31, -82, -11, 1, -28, -42, 37, 11, -4, -3, -4, -7,
9, 2, 13, -2, 0, 10, 17, 10, 0, -10, -2, -1,
0, -3, 8, 3, -3, 2, 5, -9, -6, 5, 4, -1 },

```

```

{ -2, 7, 6, 5, 24, -17, -21, 27, 0, -30, 23, 81,
  -48, 14, 22, 0, 14, -9, 9, 32, -2, 0, -17, -5,
  -23, 4, -10, 1, 0, -8, -1, -6, -14, -13, -14, 0,
  -4, 1, 2, -7, -5, 4, -4, 3, -1, 1, 8, 2 },
{ 12, 9, 1, 0, -15, 21, 0, -3, -18, 2, 12, -18,
  9, -24, 95, 2, 40, -29, -10, -1, -3, -5, 4, -9,
  -1, -38, -8, -12, -5, -9, -9, -13, 12, 1, 1, -2,
  1, 0, 9, -1, 7, 3, 0, 7, 4, -3, 1, -2 },
{ -3, 7, -1, -19, 9, 5, 10, 38, 9, 21, -85, 19,
  -9, -7, 25, 1, 14, 20, 32, -17, -14, 5, 5, 43,
  14, 6, -4, -2, 0, -3, -6, -20, -2, 5, 8, 7,
  0, -6, -12, -13, -12, -3, -1, -3, 0, -1, -8, 5 },
{ 15, 14, 19, 15, -3, 3, 25, 29, 42, 24, 55, -23,
  -3, -9, 8, -3, 5, 28, 28, 26, 3, -12, -2, 2,
  48, 9, 2, -6, -2, -2, -7, -12, -22, -16, -9, -7,
  -1, -4, -20, -32, -14, -4, 0, -6, -8, -4, -4, 6 },
{ -2, -7, 7, -8, 0, 2, -44, 5, -3, 34, 13, -27,
  -9, 31, -18, 8, -8, -64, 60, 12, -7, -4, 3, 34,
  -12, -27, -1, 6, 4, -1, 7, -13, 9, -12, 3, -4,
  1, -8, -8, 1, 10, 5, 0, -1, 0, 2, 3, 0 },
{ -2, -6, 8, 8, 8, 0, 6, 10, -5, -3, -1, -14,
  26, -6, -30, -30, 84, -30, -8, 16, -7, -4, 3, 5,
  -26, 57, 8, -11, 3, -5, 6, -15, 3, 4, 1, 0,
  0, -2, 5, -10, -11, -9, -3, -3, -3, -9, 2, -2 },
{ 0, -12, 7, 4, -7, -5, 4, 18, -17, -7, 24, 5,
  -14, 12, -3, -15, 14, -3, 42, -91, 2, 8, -27, -36,
  21, 14, -6, -3, 2, 0, -1, -7, 22, -4, 10, -5,
  6, 12, 15, 1, -7, 2, -2, -6, 9, 0, 2, 3 },
{ 6, 10, 2, -2, 11, 13, 21, 2, -8, -6, 3, 6,
  5, 20, -10, 96, 19, -13, -1, -4, -11, -1, -8, -4,
  7, 0, 51, 6, -2, -9, -25, 8, 12, 5, 0, 0,
  1, 1, 3, 2, -26, -15, -10, -3, -16, -6, -1, -6 },
{ 1, 1, 2, 10, 14, 13, -11, -7, -3, -1, 11, 40,
  44, 37, 10, -23, -4, -12, -15, 1, -6, -2, 2, 30,
  52, 1, -14, -5, -2, -1, 10, 38, 47, 17, -3, -3,
  -1, -4, -13, -13, -3, 2, -2, -5, -20, -29, -13, -4 },
{ -6, 1, -13, -9, -7, 4, 4, 4, 6, -26, 23, -8,
  4, -10, -3, -5, -12, 10, 5, 4, -97, -22, -27, 20,
  -8, -2, -11, 15, 5, 5, -9, -11, 1, 42, 7, 4,
  10, 15, 15, -6, 4, -3, 4, 5, 3, 7, -2, 4 },
{ 0, -3, 4, -2, 0, -4, 12, 5, -7, -6, 23, 21,
  -6, -27, -44, -12, 20, 12, -20, -8, 27, 7, 1, 39,
  7, -59, 10, 2, 2, 4, -27, -51, 30, 16, -1, -3,
  -4, -7, -3, 4, -2, 4, 0, 18, 17, -3, -11, 1 },
{ -8, -1, -6, -13, 6, -11, -1, -5, -1, -1, -31, 2,
  -12, 9, -25, -15, 27, -21, -3, 30, -31, -9, 19, -63,
  62, -33, -11, 9, 7, 2, -20, -3, -6, -12, 7, 2,
  -3, 9, -10, 18, -7, 1, 0, 4, 8, 7, 3, 3 },
{ 5, 3, -2, 5, -8, 5, 1, -7, -5, 6, 5, 6,
  4, -14, -9, 28, -6, -16, 1, -6, 14, -2, 18, 5,
  -3, 32, -79, 28, 11, -18, -65, 8, -2, 1, 1, -2,
  -1, -4, -3, -13, 8, -16, 2, 13, 15, -9, -3, 13 },
{ 1, 5, 4, -1, 11, -1, -1, 9, 7, -1, 24, 18,
  -7, -4, 6, -1, 4, 0, 15, -24, -8, -9, 93, -13,
  -18, 0, 17, 13, -1, 12, 9, 7, -18, 44, 21, -8,
  -3, -1, -25, 10, 5, 0, -1, -9, 6, -12, -18, -12 },
{ 1, 3, 2, 1, 1, 1, 12, 11, 12, 13, 1, -4,
  -8, -6, -2, -4, 17, 41, 43, 31, 3, -6, 4, -5,
  -17, -3, -10, -15, -1, -9, -18, 36, 52, 2, -1, -1,
  -4, 1, 33, 57, 18, -4, -1, -1, -5, -6, -3, -29 },
{ -4, -3, -4, 1, -1, -6, -8, -4, 5, -2, 0, 4,
  1, -5, -16, 24, -10, -1, 3, 0, -10, 12, 13, -11,
  6, 4, -15, -92, -13, -54, 9, -13, -4, 19, 13, 1,
  -1, 6, -3, -14, 11, 32, 0, 26, -1, -2, -2, 0 },
{ -1, -4, 2, -2, -2, 0, -2, 2, -4, 4, 9, 5,
  1, -13, -14, -1, 40, 4, -3, -25, -13, 11, -25, 30,
  2, -28, 4, -10, 3, -11, -8, 68, -60, -13, 3, 1,
  6, -7, -30, 19, 28, 1, -1, 2, 11, -6, 4, 1 },
{ 0, 0, 0, 0, 2, -4, -3, -7, -3, 1, 15, 0,
  -3, 1, 14, -29, -24, -7, -10, 0, -31, 44, 16, 12,
  -10, 17, 26, -26, -10, 1, -52, 4, 13, -43, 30, 8,

```

```

    5, -24, -14, 19, -38, -10, 5, -6, 9, 20, -10, 8 },
{ -7, -4, -7, -7, 0, -2, -2, -4, -5, -12, -6, 3,
  0, 0, 0, -7, -9, 1, -6, -22, -1, -97, 16, 15,
  -8, 6, 11, -27, 1, -5, -17, 2, 9, -39, -33, -23,
  8, 12, -20, 9, -12, 2, 6, 4, 8, 3, 13, 1 },
{ 2, 5, -1, 2, 1, 2, 12, -7, 7, -4, 10, 10,
  3, -3, -1, 29, 16, 14, -13, -11, -32, 9, 20, 8,
  9, 16, -14, 25, 5, -9, 47, -24, 28, -59, 5, 3,
  2, -29, -16, 12, 47, 13, -5, 11, 10, 21, -3, -7 },
},
{
{ 104, -43, -51, -12, 22, -15, -2, 8, 4, 1, -3, -1,
  2, 0, -2, 0, 2, 1, -1, 1, 0, -1, 0, 1,
  0, 1, 0, -1, 0, 1, 0, 0, 0, 0, 1, 0,
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 43, 30, 77, -68, -32, -37, 0, -6, 9, 1, -7, 0,
  22, 1, -6, 0, -2, 0, -2, 1, -1, -2, 0, 2,
  -1, 4, 0, -2, 0, -1, 0, 0, 0, 0, 0, 0,
  0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0 },
{ -11, -92, 52, -20, 24, 45, 12, -31, 27, -8, 0, -5,
  -1, -6, 6, 3, -3, -3, 3, 5, -2, 0, -1, -3,
  3, 0, -1, 1, 1, -1, -1, 0, -1, 1, 2, -1,
  0, -1, 0, 0, 0, 0, -1, 0, 0, 0, 0, 0 },
{ -18, -30, -28, -17, -81, -1, 16, 42, 61, 32, -4, 4,
  -13, 9, 1, 4, 5, -14, -12, 3, 5, 0, 3, 2,
  0, 0, 4, 0, 1, 1, -3, -2, 0, -2, 0, 2,
  1, 1, -1, 0, -1, 0, 2, -1, 0, 0, 0, 0 },
{ 37, -1, 30, 66, -52, 10, -10, -8, 10, -66, 2, 22,
  -31, 16, -4, -2, 1, -3, 22, -3, -7, -1, 5, -3,
  -4, -3, 3, -1, -1, -1, -1, 2, 2, 1, -1, -2,
  2, 0, -1, 1, -1, -1, 1, 1, 0, 0, 1, 0 },
{ -19, -31, 14, 36, 18, -92, 52, 20, 6, -20, 2, -6,
  18, -19, 6, 10, -6, -13, 1, 3, -3, -1, 0, 4,
  4, -2, -2, 2, 3, -1, -3, 3, -3, 0, 1, -1,
  0, 2, 1, -1, 0, -1, -1, 1, 0, 1, -1, 0 },
{ 12, 27, 25, 10, 37, 12, -8, 51, 45, 1, -18, -52,
  -59, -38, -6, -2, -5, 7, 1, 2, 4, -4, -4, 8,
  19, 5, -6, -2, -1, -1, -1, 0, -2, -5, -1, 0,
  -1, 2, 2, 1, 4, 2, -2, 0, 0, 0, 0, 0 },
{ 0, 28, -25, 5, 9, -11, 4, -71, 68, -3, 7, -43,
  18, 17, -6, -1, 12, -31, 29, -10, 1, 1, 0, -8,
  12, -6, 2, -1, -1, 4, -5, 0, 1, 7, -3, 1,
  0, -1, -2, 2, 2, -1, 1, -1, 1, -2, 1, -1 },
{ 17, 13, 8, 16, -6, 18, 79, -31, -25, 44, -39, 12,
  -31, -6, -39, 0, 12, -19, 5, 2, 6, -7, 0, 14,
  -4, 12, 3, -9, -1, 2, 2, -8, -3, -1, 2, 1,
  -1, 2, 1, 4, -4, 3, 1, -2, 0, 0, -2, 0 },
{ -5, 15, -22, -45, 3, 27, 52, 24, -4, -59, 57, 7,
  -3, -23, -28, -3, 5, -16, 17, -9, -6, 9, 2, -22,
  5, 12, 1, -6, -1, 0, 1, 1, -1, 2, -2, 0,
  1, -2, 1, -1, -1, 4, 1, 1, 0, 0, 0, 0 },
{ 18, 14, 13, 34, -1, 42, 14, 25, 8, -3, -11, -12,
  73, -7, -7, -7, -10, -37, -59, -1, -6, -3, 3, -8,
  22, -14, 3, -5, -4, -1, 0, 15, 0, 3, -1, -1,
  1, -6, -1, 0, 0, -4, 0, 2, 1, 0, 2, -1 },
{ -2, 12, 9, -5, 46, -4, 0, 16, 34, 2, 1, 57,
  -18, 49, 1, -2, -23, -38, -19, -48, -1, 1, -1, 16,
  -30, 14, -6, 0, 0, -5, 0, 3, 21, -3, -6, -1,
  -2, 2, -6, -4, -4, 3, -3, 2, 2, 3, 1, 0 },
{ 0, 7, -7, 2, 0, 1, -9, -17, 27, 0, -19, 62,
  14, -84, 15, -5, 22, 17, 5, -41, 15, -4, 2, 3,
  10, -17, 5, 2, -2, 6, -6, -5, 9, -4, -3, 2,
  -2, 1, 5, 3, -5, -3, 2, -1, 6, -3, 1, -1 },
{ -13, 0, -15, -21, 6, 9, 4, 19, 0, -48, -75, -3,
  27, 21, -6, 3, -2, 0, 32, 13, 60, -7, -10, 9,
  -13, -3, -5, 0, 1, -1, 0, 2, 0, -23, 3, 9,
  -1, 2, 2, 6, 2, -2, -2, 2, -2, -3, -2, 0 },
{ 8, 11, -3, -13, -6, 8, 28, -3, -17, -3, -5, -5,
  -20, 0, 99, -35, -22, -35, 13, 4, 0, 1, 5, 6,
  16, -27, 10, 1, -8, 2, 13, 4, -2, -4, -2, 0,
  2, 0, -2, -4, -4, 2, 2, -3, 3, 2, 1, 0 },

```

```

{ -11, -8, -17, -18, -13, -13, -18, -33, -15, -38, -8, -10,
  -26, -14, -8, 8, 0, -25, -59, -2, 12, 4, 7, 61,
  37, 32, 6, 2, 1, 1, -2, 9, -1, -4, -1, 3,
  2, 2, -4, -17, -1, 3, 1, 4, 2, 3, 1, -2 },
{ 3, 17, -10, -1, 3, 5, 40, -21, 31, -18, 4, 5,
  -4, 12, 14, -7, -19, 76, -46, 16, -1, 1, -12, 15,
  -33, -17, 5, -4, -4, 3, -9, 27, -28, 7, -1, 1,
  -2, 0, 8, -6, 4, -3, 1, 5, -1, 0, -8, 0 },
{ -1, 9, 2, -1, 17, -6, -13, -1, 16, -5, 5, 40,
  -10, -4, 1, -8, 43, -34, -14, 82, -4, 4, -52, -3,
  -3, -10, -10, 3, 1, -1, 11, -9, -12, 9, 2, -2,
  -8, -2, 16, 2, -3, 1, -2, 2, -7, -5, -1, 5 },
{ 0, -4, 8, -5, 4, -2, 13, 9, -10, -13, 1, -21,
  -10, 26, 12, 0, 91, 11, -29, -34, 5, -2, 17, 3,
  -4, -43, -28, 0, -2, 0, 12, -14, 15, 5, 2, 2,
  1, 1, -12, 7, 8, 3, -6, 1, 0, -9, -2, 2 },
{ -3, 1, 2, 2, 11, 3, 7, 15, 12, 0, -10, 21,
  20, 31, -3, -37, 18, 28, 22, -3, -34, 22, 11, 32,
  58, 23, 2, 7, -1, -5, -4, -34, -48, -11, -9, -7,
  0, -8, -6, 0, -6, -2, 0, -3, 11, 6, 3, -2 },
{ -3, 0, -8, -4, -6, 8, -2, 5, 1, -12, -31, -19,
  16, -25, 7, -7, 12, -10, 14, 4, -67, 39, 5, 38,
  -62, 15, -1, 2, 0, 3, 2, -6, 25, 12, -1, -10,
  2, -17, 9, -13, 13, -1, 1, 1, -6, 6, -1, 1 },
{ 10, 1, 12, 19, -4, 11, 4, 4, 4, 15, 53, -13,
  24, -7, 10, -25, 17, 1, 11, 5, 42, -26, -10, 48,
  -31, 26, -1, 3, -3, 0, -1, -22, 9, -50, 4, -1,
  -1, 10, 10, -25, 8, -9, 0, 1, -3, 10, -5, 1 },
{ 7, 11, 1, -1, 4, 15, 6, 6, 1, 6, 14, 12,
  7, 4, 16, 93, -3, -4, 21, 11, -15, 3, 3, 39,
  16, -34, -17, -24, 5, 8, -28, 3, -11, -25, -3, -4,
  2, -3, -2, -7, 3, 9, -2, -6, -5, 6, 2, 0 },
{ -6, -8, 0, -7, 0, -6, -10, 3, -2, 5, 13, 0,
  -3, -2, -49, -33, -18, -5, 17, 1, -9, 1, 2, 45,
  3, -78, 18, 6, 2, -6, 36, 31, 6, -17, 0, 0,
  1, 0, -1, 10, -18, 2, 0, -3, -2, 1, -1, 2 },
{ 0, 4, -2, 0, 5, -2, -3, -7, 9, 2, 1, 14,
  -1, -12, -4, -7, -6, -7, -3, 37, 8, -4, 81, -8,
  -16, 4, -49, 8, -1, 7, 23, 9, -17, -13, -49, 3,
  6, 8, -23, 0, 16, -6, 0, 0, -9, 5, 17, -2 },
{ -1, 2, 4, -2, 13, -1, -6, 5, 1, -5, 0, -3,
  -4, 1, 1, 11, 20, -10, -11, 17, 3, -10, 41, -5,
  -26, -9, 93, 0, -1, -20, -32, -24, -14, -6, -22, 1,
  1, 7, 0, 16, -18, 14, 4, 6, 2, 0, 8, -13 },
{ 0, -1, -6, -2, -7, -3, -2, -14, -2, -11, 0, -8,
  -5, -8, -7, 2, -36, -7, -26, -12, -6, 13, -12, -8,
  -11, -25, -21, 6, -4, -2, 6, -79, -15, -29, 4, -3,
  2, 14, 22, 52, 22, 6, 5, 6, -9, 21, -3, -2 },
{ -1, -6, 3, 0, -4, -2, -15, 3, -11, 5, 1, -8,
  -4, -5, -2, -5, 13, -30, 5, -33, 4, 4, -3, -2,
  -29, -1, -12, -3, 2, 4, -12, 37, -85, -3, 4, -1,
  2, 1, 51, 1, 13, 5, -2, -7, 23, -13, 9, -10 },
{ 8, 5, 5, 3, -1, 4, 9, 0, -3, 8, 7, -1,
  -5, 2, 0, 21, 2, -1, 1, -2, 25, 51, -3, -1,
  4, -4, 4, 94, -16, -19, -13, 13, 5, -7, -8, -25,
  4, -20, 4, 5, 2, -34, 4, 8, -3, -9, -1, 0 },
{ 0, 0, 1, 3, 0, -4, 1, -9, 5, 2, -7, -4,
  -3, 2, 8, 9, 18, 8, -12, -4, -12, 47, -9, -35,
  3, 21, 15, -25, 6, 2, 39, 31, 7, -79, -4, -16,
  4, 24, 4, 3, -27, 5, -1, -1, -6, 9, -1, 1 },
{ 4, -3, 7, 10, -1, -3, -3, 2, 1, 9, 15, -1,
  2, -2, 0, -3, -9, 3, 0, 0, 56, 66, 5, 11,
  -5, -3, 4, -54, 7, -1, 17, -20, -2, 34, -15, -22,
  3, -42, -1, 2, -6, 19, -2, -2, 2, -15, 5, 0 },
{ 2, 4, -1, 1, 1, 6, 2, 3, 8, -4, -1, -3,
  7, -2, 10, 42, -1, 3, 4, -15, -1, -27, 0, 9,
  -7, 14, 17, 16, 2, -7, 91, -20, -31, 27, 4, 7,
  1, -7, 10, -9, -29, -22, 1, 10, -18, -1, 2, -1 },
},
},

```



```

{
// 7
{
{ 108, -34, -51, 5, -2, -1, -1, 14, 18, 2, 1, -11,
-12, -4, 1, -1, 1, 1, 1, 3, -1, 0, -1, 4,
5, 1, -1, 0, -2, -4, -2, 1, 0, -1, 1, 0,
0, 1, 0, 1, 0, 0, 0, -1, -1, 0, 0, 0 },
{ 27, 47, 40, -47, -65, -25, 0, 19, 37, 14, 1, 3,
12, 6, -1, 2, -1, -21, -32, -11, 2, -2, 4, 20,
18, 6, 0, -2, 0, 0, -4, -1, 1, 1, 1, -9,
-9, -1, 4, 7, 5, -1, -2, -1, 0, -3, -1, 2 },
{ 42, 14, 22, 15, 1, 31, -11, -41, -39, -26, 12, 40,
51, 15, 0, 0, 1, -7, -23, -11, 2, 1, -3, -11,
-20, -8, 1, 2, 20, 34, 16, 0, -1, 1, -6, -20,
-16, -6, -1, -3, 3, 2, 0, 10, 9, -6, -5, 0 },
{ 13, -72, 71, -51, 16, 24, 12, -7, 8, 10, 20,
-15, -23, 2, -1, 2, 12, 4, -15, -2, 0, 2, -6,
2, 7, 0, -4, -12, -2, 9, 1, 1, -1, -6, 1,
6, 3, 2, -2, -3, 2, 4, 2, -4, 2, -1, -1 },
{ -36, -51, -43, -12, 4, 5, -25, 1, 12, -15, 19, 19,
13, 18, 2, -2, -3, -29, -41, -27, -9, -4, 7, 30,
31, 5, -2, 2, 2, -3, 0, 3, -1, 0, -8, -22,
-25, -8, 15, 29, 17, 2, -5, -11, -8, -7, -1, 8 },
{ 27, 31, 40, 33, 47, 3, -6, -5, -23, 5, 17, -2,
-12, 23, 18, 5, -16, -19, 5, -11, -8, 5, 13, 24,
34, 13, 0, 4, -15, -41, -19, 0, 3, -1, 1, 12,
2, -5, 15, 25, 10, 2, -8, -27, -22, 10, 11, 3 },
{ 0, -18, 14, 31, -15, -65, 16, 47, -27, -50, 9, 25,
-7, 0, 5, 1, -14, -26, 5, 12, 9, -2, -23, -28,
14, 18, 1, -1, 0, 5, 18, 6, -2, -1, 2, 5,
-12, -11, 10, 0, -14, -5, 8, 5, 2, -4, 1, -1 },
{ -5, -19, 10, 24, -69, 47, -36, -14, 8, -49, 14, -12,
4, -4, 8, -5, -2, 3, 9, 14, -6, 3, -9, -1,
4, 7, -3, -4, -19, -22, -15, -3, -2, -1, 14, 24,
21, 5, 2, -7, -10, -4, -4, -12, -13, 18, 13, -9 },
{ 0, -10, 41, 34, 10, 24, -27, 17, 37, -2, -2, -33,
-26, 14, -11, 8, 26, -7, 5, 30, 19, -2, -22, -1,
8, -27, -8, 8, 30, 11, 1, 8, 2, 1, 4, -20,
-36, -6, -1, 18, 7, -7, 0, 6, 20, -27, -10, 15 },
{ -2, 42, -13, 14, 9, 33, 36, 16, 25, -36, 43, 19,
-28, -39, -7, 1, 14, 30, -9, -16, -1, -10, -11, 6,
7, 6, 3, -10, -23, 3, 24, 18, 2, -3, -19, -22,
4, 12, 7, 10, 9, 3, 6, 5, -10, -4, -12, 7 },
{ 1, 32, -20, -58, 30, 23, -43, 16, 15, -22, 13, 22,
-24, 32, 35, 5, -12, -14, 19, 11, -5, -1, -9, -29,
4, 0, -6, 10, 8, 1, 16, 5, -5, 4, 6, 17,
-1, -7, 0, -11, -24, -9, 13, 8, 7, 0, 5, -7 },
{ -3, -6, 11, 32, 36, 6, 2, 31, 51, 2, 7, 21,
31, 14, -1, -16, 6, -9, -35, 7, 0, 17, 16, 11,
0, 15, 26, -18, -9, 12, -16, -13, -2, 10, 20, 13,
20, 17, -17, -36, -17, -5, 0, 20, 15, 11, 3, -22 },
{ 3, -6, 13, 7, -1, -56, -48, -15, 10, -6, 55, -4,
-18, 15, -14, -11, 7, 21, 12, -31, -35, 4, 12, 6,
-23, -27, 2, 3, 7, -2, 4, 12, 3, -9, -18, -4,
18, 19, -23, -11, 6, 5, 2, 6, 7, 3, -6, -1 },
{ -5, -12, 5, 30, -14, -4, -4, -10, 43, 48, 18, 8,
22, 10, 43, 2, -39, 22, 19, 1, 14, -3, -2, -17,
-13, 27, -6, 23, -9, 13, 26, 15, 8, -1, -18, -11,
1, -27, 24, 4, -1, 1, 11, 2, -20, 4, -7, 1 },
{ 1, -2, 4, 1, 4, -7, -1, -61, -3, 5, 4, -36,
-41, -10, 10, -20, -34, -25, -34, -10, 11, 1, -27, -3,
20, 16, 14, -22, -20, 5, 8, -5, -8, 5, 6, -8,
-1, 7, 1, -13, -19, -16, 19, 34, 14, -14, -16, -6 },
{ -4, 6, -4, 26, -24, 24, -27, 55, -43, 26, -18, 0,
-32, -8, 49, -7, -5, 6, -14, -43, 2, 4, 16, 12,
-5, -9, -2, -2, 2, 15, 4, -19, 0, -1, -4, -10,
0, 5, -10, -12, 4, 9, 1, 16, 13, -4, -8, -4 },
{ -4, -8, -2, -9, -14, 23, 61, 13, -11, -6, 35, -47,
16, 45, 19, 3, -10, -15, 29, -3, -44, 11, 3, 13,
6, 6, 20, -2, 18, 9, 5, 7, -1, -9, -2, -2,
-6, -2, -14, -8, 5, 4, 4, 11, 18, -8, -10, 0 },

```

```

{ 2, 1, -5, -6, -4, 2, -26, 14, -23, 14, -6, 7,
  37, 23, -20, -22, 11, 19, 42, 5, 27, -16, -15, 28,
  29, 12, 22, -15, -25, -23, 25, 13, -6, 14, 5, -15,
  12, 31, -6, 9, -9, -16, 20, 18, 0, -22, -23, 14 },
{ -4, 0, -11, 15, -17, 21, 8, 6, -11, 53, 31, 11,
  -11, 24, -53, 25, 23, -33, -7, -28, 9, 6, -18, -38,
  14, -14, -8, -2, -6, -7, 19, 1, 5, -6, -3, 18,
  2, 1, 3, -8, -23, -4, 9, -3, -8, 15, 18, -12 },
{ -2, -4, -6, 12, -22, -4, 20, -35, 7, 12, 13, 58,
  -22, -10, 14, 10, 7, 19, 25, 6, 2, 24, 28, 11,
  44, 3, -13, 13, 25, -10, -14, -14, -5, 12, 16, 13,
  -20, 8, -19, 1, -12, -6, 3, 9, 33, -25, 1, 2 },
{ 0, -10, 4, -1, 0, -14, 27, -15, 2, -20, -16, -6,
  -14, 51, 27, 17, 25, 42, -26, -23, 37, -28, -20, 26,
  16, -21, -2, 22, 9, 17, 14, 6, 3, 5, 16, 17,
  23, 5, 0, -7, -6, -7, 4, -8, -14, 17, 12, -11 },
{ 1, -6, 1, 6, -17, 8, 17, -13, 17, -7, -29, 37,
  -43, 48, -3, -17, -5, -26, 16, 3, 16, -20, 27, -23,
  -24, -5, 38, -29, -7, -5, -15, 0, 9, 3, -27, -12,
  10, 9, -7, 4, 26, 9, -3, -8, -9, 0, -13, 15 },
{ 1, 3, 2, -1, -1, -3, -22, -10, -7, 0, -14, -10,
  -19, 3, -16, 6, 23, 0, 13, 27, -13, 23, 27, 8,
  45, 37, 5, -11, 6, 63, 33, 1, 3, -8, -13, 0,
  11, 11, 23, -15, 14, 29, -8, -5, -18, 17, 3, -13 },
{ 2, 2, 7, 7, 4, 5, -3, 3, 21, -32, -37, -12,
  -2, 21, -48, 28, -29, 23, 17, -51, -15, -5, 24, -6,
  0, 22, -13, 16, -20, -5, 10, -33, -13, 12, 3, -7,
  -14, -23, 9, -4, -14, 6, 3, 17, 13, -10, -2, -2 },
{ 0, -5, 3, 10, 3, -9, 0, -13, 13, -6, -27, -6,
  18, -8, 40, 49, 53, -33, -6, -10, -24, 6, 9, -19,
  -5, 9, -22, -6, -10, -22, 16, 6, -14, 3, -19, 4,
  36, 18, 0, 14, 1, -2, 18, 18, 3, -19, -19, 14 },
{ -1, -4, 1, 2, 2, 4, 4, -1, 0, -8, 11, 5,
  -5, -18, -11, 33, -28, -51, 28, 10, 30, -26, 22, 54,
  -13, -16, -37, 6, 10, 5, 5, 12, 15, 9, 0, -26,
  25, 2, -7, -25, -10, 0, 2, 16, -5, 14, -5, -18 },
{ -1, -3, -6, -13, -5, -3, -5, 11, -23, 2, 40, -6,
  -12, 25, -10, 51, -12, 23, -24, 32, 29, 31, 10, -1,
  -30, 17, -4, -4, -21, 14, -29, -25, -1, 15, 9, -1,
  22, 9, 17, 18, 18, -9, 1, 10, 0, -26, -13, 11 },
{ 0, 2, -3, -7, 1, -12, 0, -12, 6, -5, 19, -9,
  -4, -9, 21, -3, 50, -17, 37, -24, 27, 14, -2, 11,
  -34, 16, 35, -9, 1, -6, 7, -40, -4, 12, 22, -30,
  -20, -11, 27, 11, -16, 0, -20, 5, -4, 12, 27, -18 },
{ 0, 2, -1, 3, 2, 6, -8, 3, 8, 0, 11, -9,
  3, -25, -11, -3, -12, -17, 10, -36, 13, -33, 5, -7,
  -8, 33, 22, 4, 49, 27, 2, 2, 12, 10, 27, 51,
  32, 13, 0, 35, 14, -4, 2, -13, 9, -22, -3, 16 },
{ 1, 0, 2, 4, -6, 3, 12, 2, -1, 0, -5, 6,
  -14, 3, -11, -40, 1, -10, -11, 13, -19, 30, 29, 26,
  -22, -42, 1, 10, 0, 2, 38, 5, -26, 23, 15, 22,
  23, -29, 41, 19, -24, -13, 17, -1, -16, -19, -8, 13 },
{ 1, 2, -1, -2, -3, 4, -2, -1, -18, 2, 9, 0,
  -15, 8, -14, -31, 46, -1, -16, 13, -2, -29, 13, 7,
  -8, 51, -7, 43, 5, -17, -28, 15, 18, -2, -16, 1,
  11, -50, 4, -14, 0, -1, 15, 32, 11, 0, -11, -7 },
{ 0, -2, 1, 6, -7, 3, -10, 0, -7, 11, -21, 17,
  -18, -1, -8, 45, -14, 19, -17, 16, -32, -7, -7, 28,
  -28, 21, 43, -19, 28, -21, 11, 43, -6, -9, 4, -2,
  -15, 5, 10, 24, -18, -9, 9, 14, 12, 19, 19, -23 },
},
{
{ 121, -2, -4, -6, -35, -3, -1, 1, 2, -3, -1, 2,
  17, 1, -1, 0, 1, 0, -1, 1, 0, 0, 0, -1,
  -8, 0, 0, 0, 0, 0, 0, 0, 1, -1, 0, 0,
  0, 0, 0, 4, 0, 0, 0, 0, 0, 0, 0, -2 },
{ 6, 61, 65, 12, 0, 12, 7, -47, -51, 5, 1, -5,
  2, -5, 2, 1, -5, 31, 33, -3, 1, 1, 1, 2,
  -4, 3, 0, 1, 1, -1, 3, -19, -20, 1, -1, 0,
  0, 0, 0, 4, -1, 0, 0, -1, 9, 10, -1, -3 },
{ -36, -17, -13, -19, -70, -15, -4, -9, -13, -6, -9, 14,

```

```

62, 8, -5, -2, -2, 9, 13, 1, -5, -4, 4, -14,
-49, -8, 2, -3, -2, 0, 1, -8, -9, 0, 0, -2,
0, -2, 10, 32, 5, -1, 0, -1, 4, 5, 0, -18 },
{ 1, -61, 62, 50, -7, -32, -7, 29, -21, 15, 5, -32,
4, 25, -5, 0, 1, -19, 12, -4, 0, 1, -2, 21,
-3, -17, 4, -1, 0, -1, 0, 14, -8, 1, 2, 1,
0, 1, -13, 2, 10, -2, 1, 0, -8, 6, 0, -1 },
{ -1, -48, -42, -17, -26, -21, -9, -30, -31, -5, 0, -13,
-28, -8, -1, -2, -5, 29, 34, -5, -2, -1, -9, 15,
39, 8, -6, -1, -1, -3, 4, -25, -29, 3, -4, -1,
-3, 5, -12, -34, -8, 4, -3, -2, 17, 20, -2, 22 },
{ 2, 12, -11, 21, 15, -46, -20, -40, 40, 11, 10, -19,
5, 35, -17, -11, 19, 42, -29, -15, 9, 5, -6, 13,
-9, -22, 11, -7, -4, 7, -14, -35, 20, 13, -4, 3,
-2, 3, -10, 6, 13, -6, 3, 7, 23, -12, -9, -4 },
{ -7, 2, -24, 58, -38, 52, 32, -2, 17, 39, 16, -29,
11, -27, 14, 6, -18, 5, -11, -30, 7, 1, -6, 18,
-2, 15, -5, 2, 0, -3, 12, -6, 6, 22, -4, 0,
0, 3, -11, 1, -7, 2, -1, -7, 6, -2, -14, 0 },
{ 12, -14, -21, 17, 57, 20, 8, 33, 21, 5, 6, 9,
18, 6, 7, 3, 3, 19, 35, 8, 1, 0, 7, -3,
-25, -1, 4, 2, 0, 4, 4, -33, -50, -6, 8, 0,
3, -2, 2, 22, 2, 1, 2, -3, 29, 44, 4, -13 },
{ 0, -48, 53, 8, -2, -2, -4, -20, 38, 4, -3, 34,
3, -33, 2, -1, -3, 25, -21, 5, 6, 6, 4, -39,
4, 40, -6, -1, -1, -3, 5, -19, 7, -2, 0, 4,
0, -2, 29, -9, -29, 6, -1, -2, 14, -1, 0, 8 },
{ 1, 0, -11, 25, 7, 14, -52, -4, -13, -29, -23, -37,
4, -51, -29, -26, 25, -3, -4, 13, -17, -9, 14, 20,
-14, 34, 12, -15, -8, 9, -17, 0, 5, -14, 5, -4,
3, -12, -10, 16, -22, -10, 5, 8, -1, -5, 9, -13 },
{ -1, 11, 0, -17, -2, 8, -49, 37, -24, 58, 42, 29,
10, -9, -26, -15, 29, -4, 12, -22, 21, 8, -24, -12,
5, 14, 15, -6, -2, 8, -12, -3, 3, 17, -7, 2,
-3, 18, 3, -9, -5, -5, 2, 8, 4, -1, -8, 11 },
{ 11, -40, -17, -6, 30, 15, 2, -34, -45, 14, 15, -6,
-40, -12, 1, -1, -14, -3, -12, -17, 5, 2, -14, -28,
-23, -17, -7, -2, -2, -4, -3, 4, 16, 6, -8, -2,
-4, 1, 29, 45, 17, -7, -3, 3, -3, -12, -2, -38 },
{ 0, 28, -27, 55, -8, -49, -23, -27, 14, 25, 9, 22,
-15, -18, -3, 2, -17, -30, 9, 4, -1, 3, 3, -30,
-4, -4, -12, 1, 0, -11, 3, 26, -29, -25, 4, 3,
-1, -8, 13, -3, 2, 2, -5, -4, -19, 23, 16, 3 },
{ 1, -18, -20, 45, -1, 38, 7, -18, -12, 4, -4, 44,
9, 40, -6, -9, 23, 33, 30, 31, -9, 1, 13, -3,
26, -3, 18, -3, -2, 13, -8, 20, 24, -11, 10, 1,
4, -1, 10, -6, 9, -1, 6, 12, -24, -25, 14, 0 },
{ -6, 6, 10, 9, -45, 35, 2, 16, 0, 3, -9, 0,
-48, 12, -10, -3, 14, -14, -33, 19, -3, 1, -4, -21,
-1, -35, 6, 0, 2, 4, -36, -36, -14, -33, -5, 2,
-4, -7, 3, -4, 8, -13, 1, 19, 33, 18, 19, 9 },
{ 1, 12, 1, -2, -12, -9, 8, 27, 0, 19, -43, 12,
-34, 22, -53, -27, -36, 16, -5, -24, -21, -13, 20, -1,
16, 14, 18, -17, -9, 11, 35, 1, -1, 18, 13, -7,
4, -6, 17, 17, -4, -3, 6, -21, 4, 10, -6, -14 },
{ -2, 14, -6, 0, -18, -36, 13, 23, -6, 20, -8, -12,
-22, 6, 48, 24, 48, 12, 7, -5, -7, -9, 13, 8,
22, 37, -5, 17, 7, 5, -20, 5, 12, 6, 10, -4,
7, -3, 33, 29, -12, 12, 2, 19, 13, 8, 9, -32 },
{ -4, -5, 18, -32, -20, 4, 6, -17, 42, 20, 34, 5,
-19, -21, -13, -11, -9, -1, 20, 41, 25, 11, 0, 46,
19, 3, 9, -8, -5, 6, 10, 16, -3, -18, 1, 8,
4, -2, -18, 28, 24, -8, 3, 0, 1, 7, 13, -40 },
{ 1, -13, 13, -12, 1, 22, -22, -31, 27, 14, -42, 31,
-7, -13, 9, 5, 24, -23, 16, -41, -16, -9, 28, 12,
1, -21, -8, 6, 5, -7, -25, 13, -15, 39, 16, -5,
8, -8, -21, 0, 27, -3, -7, 20, -1, 13, -21, -2 },
{ 2, -4, -4, -12, 11, -14, -1, 13, -18, 58, -2, 15,
1, -12, 7, 10, -22, 14, -10, 29, -23, -25, 26, 14,
-17, -13, -27, 4, 2, -14, -14, -31, 7, -19, 15, -5,
11, -32, -35, -12, -20, 0, -10, -3, -10, -38, -6, -8 },

```

```

{ 0, 9, -4, 27, -2, -15, -11, 27, -16, -29, -1, 29,
  6, -31, 1, -4, -37, -7, 15, 21, 11, 13, 15, 17,
  19, -18, -26, -4, -4, -10, -5, -14, 36, 28, 3, 9,
  -3, 13, 16, 10, 27, 8, -8, 25, 50, -10, -13, 4 },
{ -1, -2, 1, -4, 1, 25, -52, -11, -17, 6, 20, -1,
  14, 25, 6, 20, -9, 13, -53, 16, -14, -17, 13, 15,
  10, 9, -13, 18, 11, -9, 35, 19, 8, 4, 9, -6,
  12, -4, 2, 4, 20, 27, -9, -2, 15, 45, 19, 8 },
{ -1, 5, 5, -10, -4, 11, -23, 28, 23, 11, -23, -46,
  8, -22, -14, 6, -4, 52, 18, -5, 4, 6, -1, -26,
  15, -22, -27, 12, 8, -18, -1, 6, -9, -12, -1, 2,
  -7, 8, 27, -8, 43, 12, -15, 7, -30, -18, 15, -3 },
{ 2, -5, 1, -18, 17, -8, 43, -18, -3, 29, 10, -24,
  41, -12, -11, -39, -6, -19, -6, -4, -25, -21, 6, -4,
  20, -3, -10, -23, -12, 15, -22, 7, 7, -5, 22, -7,
  9, 8, 24, -4, 11, 3, 5, 32, 13, 19, 33, 28 },
{ 1, -5, -8, -7, 6, -11, 19, -6, -15, 23, -17, -17,
  -1, -25, -21, 37, 19, 1, 2, 19, 40, 17, 45, -10,
  -20, -9, 43, 15, 17, 10, 20, 14, 17, 10, 6, 11,
  -6, -24, 3, -1, 10, -12, 4, -3, 16, 6, 4, 43 },
{ 0, -1, 2, 8, 2, -17, 11, 11, -6, -17, 16, 20,
  12, -36, -9, 42, -22, 24, -23, -16, -35, -16, -26, -6,
  22, -3, 53, 25, 12, 13, -12, 4, -14, -6, 15, -11,
  7, 14, -21, 3, 21, -28, 9, 20, -5, -3, 2, -13 },
{ 0, 2, 3, -2, -2, 3, -13, -10, 21, 20, -14, -12,
  -18, -7, 2, 21, 9, -12, 29, 45, -48, -21, -45, 6,
  -26, -15, 12, 0, 2, 1, 19, -5, 20, 16, 9, -7,
  6, 46, 23, 6, -8, 2, 5, -15, 9, -3, -13, 27 },
{ 1, -3, 0, -3, 9, 1, -17, -3, -3, 21, -47, 14,
  17, -6, 32, 6, -8, -10, 10, -38, -6, 23, -28, 15,
  10, -5, 10, -7, -1, 11, 23, -7, 28, -48, -15, 13,
  -18, -5, -15, 6, 6, 4, 14, -14, 28, -8, 46, 0 },
{ 0, 1, 1, 7, -5, -13, 21, 13, -10, -4, 0, 17,
  -21, -25, 16, -33, 37, 23, -15, 6, -9, -8, -16, -13,
  -28, 11, -17, -14, -12, 29, 34, 14, -11, -5, -6, -6,
  3, -7, -35, -1, 51, 36, 11, 6, 3, -6, -16, 17 },
{ 0, 2, 3, -2, -6, 5, -26, -3, 7, -3, 27, -22,
  11, -3, 38, -14, -1, -26, 13, 0, -11, -23, 33, -31,
  33, -18, 33, -2, -10, 24, 28, -25, -4, -10, -3, -10,
  3, -25, 19, -12, 18, -1, 17, -19, 10, -23, -34, -15 },
{ 0, -2, 2, -2, -5, 2, 1, -4, 11, -17, 34, 4,
  -21, 12, -24, -7, -5, -20, 26, -36, -15, 6, 15, 19,
  -30, 32, -3, 51, 20, 1, 6, -30, 9, -23, 24, -14,
  5, -5, 14, 3, 22, 18, -4, 8, -1, -30, 20, 26 },
{ -1, 1, 4, -1, -9, -2, -4, 15, 0, -20, 28, 13,
  -18, -19, 28, 1, 17, 24, -1, -26, -1, 1, 15, 16,
  -16, -30, 6, -41, -28, -29, 9, -8, 16, 9, 38, -4,
  17, -2, 14, -3, -3, -30, 3, -29, -23, 13, 37, 18 },
},
{
{ 122, -24, 0, -15, -15, -18, 1, 6, 0, -2, 0, 4,
  8, 3, 3, 1, -1, -3, -1, 1, 1, -1, 0, -1,
  -3, -1, -1, 0, 0, 0, 0, 1, 0, -1, 0, 0,
  0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 4, 52, 95, 8, -26, -21, -10, -26, -35, -14, -5, 1,
  21, 16, 2, 3, 2, 8, 14, 6, -1, 1, 1, -1,
  -10, -7, -1, 0, 0, -2, -1, -1, -5, -2, -1, 0,
  0, 0, 0, 4, 2, 0, 0, 1, 0, 2, 0, -1 },
{ 16, 98, -61, -12, 10, -27, -32, -8, 1, 14, 9, 4,
  0, 9, 4, 7, 7, 7, -5, -6, 3, -1, -1, -1,
  -1, -4, -3, 2, 0, -1, -2, -3, 4, 1, -1, 0,
  0, 1, -1, -1, 2, 1, 0, 0, 1, -1, 0, 0 },
{ 25, 22, -1, 90, -22, 57, -16, 21, -4, 15, -16, -17,
  -33, -6, -15, 4, 1, 6, 7, -5, -5, 2, 6, 12,
  8, 9, 3, 6, 1, -1, 0, -7, -4, 2, 1, 1,
  0, -3, -6, -1, -2, -3, 0, 1, 1, 3, 0, 1 },
{ -17, 5, -9, -24, -94, 15, -24, 51, 12, -7, -3, 28,
  21, 8, -7, 4, -8, -26, -19, 1, 0, 0, -1, -13,
  -11, 3, -2, 1, 0, -1, 3, 12, 11, 1, -3, 0,
  0, 2, 3, 3, -2, 0, 1, -1, -5, -4, 0, 0 },
{ 15, 23, 24, -61, 27, 86, -11, -7, 7, -10, 13, 18,

```

```

-9, -26, -25, 4, 14, 6, -3, 5, 6, 0, -4, -10,
6, 14, 7, 9, 3, -1, -8, -6, 1, 0, 0, 0,
0, 1, 4, -1, -4, -1, -2, 4, 3, -2, 0, 1 },
{ 5, 22, 9, 7, -10, -4, 38, -29, 83, -53, -7, 3,
-34, 37, -8, -6, -11, -18, -3, -1, 9, 5, 0, 2,
10, 8, -1, 2, 3, 2, 6, 4, 6, -1, -2, -1,
1, -1, 1, -6, -5, -1, -1, 0, 0, -4, 1, 3 },
{ -5, -32, -38, 0, -26, 12, -45, -58, -15, -49, -15, -4,
2, 21, -24, 10, 21, 27, 32, 14, 0, 1, 1, -4,
-18, -14, -4, 7, -3, -9, -7, -13, -12, -6, -3, -1,
-1, -3, 2, 11, 7, 1, -2, 4, 3, 2, 5, -2 },
{ 2, 3, -13, -7, -32, 31, 50, -55, -1, 62, 18, -21,
33, 26, -22, -12, 2, -14, 3, -24, -7, -8, -4, 5,
-9, -16, -2, 5, 5, -1, 9, 0, 1, 10, 4, 0,
1, 1, -2, 5, 6, 0, -1, -5, -1, 1, -5, -3 },
{ -5, -23, 30, -19, -13, -20, -61, -15, 41, 48, 5, -13,
-35, 13, -3, 14, 14, 21, -25, -31, -3, -2, -2, 24,
22, 0, -6, 1, -6, -6, -9, -15, 5, 11, 3, 1,
0, -1, -13, -12, 5, 3, 1, 4, 6, 2, -3, 3 },
{ 4, -11, 10, 29, 48, 21, -26, 5, 19, 17, -2, 43,
26, 58, 4, 16, 10, -8, -20, 8, 0, 6, -2, -19,
-36, -25, -12, -4, -3, 2, 9, 20, 20, 1, 3, 3,
2, 2, 11, 12, 13, 11, 5, -7, -14, -11, 1, -3 },
{ -2, 2, -7, -17, -2, -7, 29, -6, -29, 21, -70, 58,
-36, 17, -28, -4, -26, 33, -21, -4, -11, 8, 8, 19,
-12, 8, 9, 3, 2, 3, -2, -2, -4, -3, 1, 4,
-2, -7, -1, -1, 0, -6, -2, 3, 2, 3, 0, 1 },
{ 2, 2, -4, 20, -20, 9, 8, -42, 16, -2, -28, 28,
20, -54, 53, -15, 34, 19, -46, 9, -5, 5, 0, -14,
18, -21, -8, -16, 3, -2, -11, 4, 12, 5, -2, 3,
-1, 3, 3, -5, -2, 9, 1, 2, 0, -8, -4, 2 },
{ 0, 10, 1, -30, 7, 21, 1, 22, 24, 7, -60, -38,
21, 14, 38, 10, 12, -11, 22, 17, -21, -2, 35, 34,
2, -21, -10, -2, 1, 2, -1, 0, -27, -15, 12, 4,
-1, -16, -11, 7, 11, 7, 4, -3, -1, 17, 10, -6 },
{ 0, 0, -5, -13, -17, 30, 14, 6, -19, -8, 32, 10,
-39, 28, 75, 11, -27, 36, 19, -20, 14, -2, -19, 4,
-8, -19, -10, -22, -2, 11, 2, -4, -5, 9, -6, -2,
2, 5, 0, -1, 17, 5, 6, -6, 1, 4, -3, 1 },
{ -2, 4, 4, 10, -7, -13, 25, 36, 43, 12, 25, 22,
22, -3, -20, -11, 23, 65, 21, 16, 5, -1, 1, -6,
-7, -15, 19, 6, 6, -4, -20, -30, -29, -2, 5, -1,
5, 5, 9, 20, 16, 7, -2, 9, 17, 13, 3, -14 },
{ 0, 0, 1, -9, -10, -2, 6, -2, -13, 21, 9, -30,
-38, 20, -8, 6, -10, 3, -33, 71, -12, 5, 3, -51,
35, -25, -3, -2, 2, -2, 10, -4, -10, -23, -10, 6,
-10, 1, 17, 13, 3, 4, 2, -5, -4, 0, 13, -5 },
{ 0, 3, -4, -2, 7, 6, 19, 30, -30, -42, 2, -17,
15, 36, -18, -2, 35, 17, -43, -14, -16, -8, -15, 22,
38, -36, -3, 18, 3, -7, 1, -4, 10, 31, 3, -5,
1, 6, -19, -22, 0, 8, 1, -1, 10, -7, -11, 12 },
{ -2, -1, 2, -5, -4, -10, 18, 11, -17, 12, -7, 23,
-41, 14, 14, -15, 77, -39, 34, -10, -11, -6, 0, -25,
1, 17, -30, 13, 4, -8, -6, -29, 13, -2, 5, 4,
-3, 3, 8, 8, 3, 11, 6, 14, 4, -1, -1, -1 },
{ -2, -1, -5, 7, -4, -1, 27, -6, 1, 7, -4, 8,
23, 4, 6, 101, -1, 3, 9, 7, -3, 6, -6, -13,
24, 34, -6, 13, -21, -10, -26, -1, 1, 6, -2, 2,
1, 3, 3, -19, 0, -4, 7, 20, 7, -2, -1, 12 },
{ 1, 0, -1, 3, 1, 1, -9, -8, 7, -8, 23, 23,
4, -1, 5, -10, -27, 9, 12, 14, -92, -29, 19, -12,
11, 19, -16, -3, -3, 7, 16, -8, 3, 30, 36, 1,
12, -7, -19, 3, -3, -3, 0, 0, 1, 5, -5, 4 },
{ 1, 6, 5, -3, -1, -9, 17, 5, 5, -15, 1, -20,
-35, -41, -18, 40, 1, -6, -17, -32, -28, -26, 5, -13,
-50, -43, -2, 5, -10, -16, -18, 15, -1, 1, 9, -5,
4, -3, 7, 24, 22, 12, -3, -9, -17, -6, -7, -15 },
{ 0, 2, 3, -8, 4, 1, -4, 13, 11, 4, -46, -30,
14, 0, -7, -8, -21, 32, 34, -25, 5, -17, 0, -51,
14, -7, -2, -6, 6, 11, 7, -24, 53, 6, -3, -7,
2, 23, 21, -7, 0, -3, 3, -4, -16, -28, -16, 11 },

```

```

{ 0, 0, 1, 5, 5, -2, -4, -2, 0, -1, 3, 43,
  4, -24, -18, 14, -29, -31, 24, -13, 24, 11, 20, 4,
  47, -59, -23, 1, -9, -18, 29, -36, -5, 0, -3, 5,
  -2, 0, -20, 4, 18, 5, 5, -9, 2, 10, 2, -2 },
{ 0, 4, 1, -6, -1, -6, -5, 3, 10, 12, -20, 2,
  -10, -13, -14, 6, 7, -11, 34, 28, -13, 26, -65, 15,
  6, -12, 1, -29, 2, -1, 9, 23, -28, 57, 1, -1,
  0, 14, 14, 10, -8, 15, 7, -17, -2, -23, -19, 4 },
{ 0, -3, -1, 0, 4, 1, -8, -4, 0, 0, -5, 6,
  -3, 13, 23, 9, 1, -29, 4, -17, -21, 14, -5, -21,
  6, -26, 96, -3, 14, -3, -14, -22, 4, 14, -2, 5,
  -6, -8, -11, 16, -19, -29, -11, 16, 8, 7, 3, -2 },
{ -1, -1, -1, -1, -8, -3, 3, -2, -4, 11, 13, 19,
  -16, 3, -7, 5, 19, 8, 35, 21, 13, -42, 38, 20,
  24, -17, 37, -21, 4, -16, -8, 60, 27, -15, 9, -8,
  15, 12, -7, -16, -12, 18, 2, -3, -12, -13, -3, 13 },
{ -1, -2, 1, -3, -2, -3, -2, -1, 5, 5, 0, 2,
  -6, -9, 24, -1, 0, 22, 13, -3, 3, 16, -5, 2,
  12, -4, 2, 95, 9, -21, 41, 39, 5, 10, 7, 3,
  2, -4, 11, 24, -3, -20, -13, 2, -17, -5, 6, 1 },
{ 0, 1, 0, -7, -7, -3, 12, 5, 0, -1, 22, -11,
  -6, -9, -9, 6, 13, 16, 11, 15, -14, 74, 36, 16,
  -20, -3, -1, -13, 2, 8, -2, -6, 50, 16, -7, 30,
  -21, -25, -18, -8, 14, -7, 3, -9, -19, -24, 6, 9 },
{ 0, 0, 1, 2, -1, 1, -6, -2, 2, 7, 2, 6,
  -5, -3, 5, -21, -25, -24, 13, 31, -10, 0, -18, 20,
  -9, -33, -10, 43, -4, 22, -69, -7, 19, -8, 10, -3,
  7, 15, 16, -41, 21, -16, -19, 4, 14, -8, -9, 3 },
{ 1, 0, 0, 2, 3, 2, -5, 3, -4, -4, 5, 5,
  8, 10, -9, -17, -11, 6, 14, -51, -27, 33, 6, -20,
  39, -7, -17, -8, 1, 1, -42, 44, -20, -14, -8, 7,
  -14, -27, 35, -1, -18, 25, -5, 3, 6, -3, 27, -15 },
{ 0, 0, 0, -2, 5, 3, 0, 2, -3, 3, -5, -11,
  -3, 10, 7, -20, -26, -3, -6, 12, 29, -1, 34, -14,
  -10, 3, -20, 8, -6, -40, -45, 3, -4, 55, -12, 12,
  8, 13, -34, 38, -27, 17, 1, 32, 7, -11, -8, 4 },
},
},
{
  // 8
  {
    { 90, 67, 6, -3, 13, -52, -22, -3, -5, 1, 0, 2,
      4, -9, 15, 2, 1, 4, -2, 0, 1, 0, 1, 0,
      -1, -2, 3, -2, 0, -1, -1, 1, 0, -1, 0, 0,
      0, 0, 0, 0, 0, 1, 0, -1, 0, 0, 0, 0 },
    { -52, 75, 39, -9, 49, 42, -39, -10, -1, -1, -1, 3,
      4, -16, -12, 10, 1, 3, 2, 1, 0, 0, 2, 0,
      -1, 1, 4, 3, 0, 1, -2, 0, 0, 1, 1, 0,
      1, 0, -1, 1, 0, 0, 0, 0, 0, -1, 0, 0 },
    { 12, -53, 80, -3, 48, -23, 14, -56, 4, 4, 2, -3,
      5, -17, 7, -2, 19, -9, -2, 1, 2, 2, -1, -2,
      2, -2, 4, -2, -2, -4, 4, -1, -1, -1, 1, 1,
      0, -1, 1, 1, -1, -1, -1, 1, 0, -1, 0, 0 },
    { 69, -6, 20, 16, 8, 87, 25, 14, 8, 1, 2, -3,
      -5, 9, -44, -1, -7, -2, 7, -1, 2, 1, -1, 1,
      3, 6, -6, 14, 3, 3, 1, -3, 2, 2, 0, 1,
      0, 1, 1, 1, 0, -2, 3, 1, 0, -1, 0, 0 },
    { -9, 15, -8, 79, 8, -13, 7, -5, 72, -11, 1, 1,
      -47, -13, 6, -7, 6, -34, 8, 1, 1, 2, 3, 0,
      -4, 15, 4, -1, 1, -3, 9, -3, 0, -2, 2, 0,
      2, -1, -2, 0, 2, -4, -1, 0, 0, 1, -1, 0 },
    { 3, -33, -18, 4, 65, -17, -9, 69, 4, -1, -2, -1,
      9, -56, -10, 6, -41, 7, -1, 2, -1, -1, 0, 5,
      1, -9, 17, 2, -3, 11, -6, 4, -3, 0, 1, 0,
      0, 2, 1, -4, -1, 2, -2, -2, 2, 0, -2, -1 },
    { -14, 36, 4, -14, 30, -6, 95, 24, 6, 1, -1, 4,
      23, 16, 28, -44, 3, -3, 0, 2, -1, -1, 2, 4,
      8, -9, -10, 1, 11, -1, 7, 3, -1, 2, 1, 0,
      1, 1, 3, 0, -5, 3, 4, 2, 2, 1, 0, -1 },
    { 7, 5, -54, -12, 8, 24, 23, -37, -20, -35, 4, -23,

```

```

-2, -64, 5, 5, 48, -1, 20, 4, -6, -2, -4, -8,
10, -5, 31, 2, -1, -17, 7, -8, -5, 2, 0, -1,
0, -1, 0, 6, -4, 3, -5, 5, 3, -1, -3, -1 },
{ -12, 9, -10, 2, 16, -39, 20, -17, -37, -7, -2, -2,
-47, 4, -75, -24, -3, 29, -7, -3, -1, 0, -2, 1,
0, 35, -15, 33, 0, 4, -7, 4, 1, -5, -1, -2,
-1, -1, 3, -1, 0, -14, 2, 0, 1, 0, 1, -3 },
{ 3, -2, 4, 19, 7, 23, 14, 14, -39, 41, 0, 34,
-51, -11, 48, 13, 21, 30, -32, -1, 4, 2, 1, 3,
-25, 28, 20, -19, 1, -8, -8, 23, -6, -4, -1, 2,
1, -2, 0, -4, 10, -2, -4, -8, 7, 4, -1, -1 },
{ -1, -2, 31, 22, -1, 2, -3, 22, -27, -60, 6, -59,
-26, 22, 38, 4, -15, 35, 32, -6, -4, 0, -7, -1,
25, 12, 0, -18, 2, 5, -13, -5, 8, 2, -4, -1,
-2, 2, 2, 0, -5, 2, 0, -2, -1, 2, 4, -1 },
{ 8, -18, -36, -6, 61, 1, -24, 11, 13, -17, -4, 16,
9, 52, 7, 12, 56, 13, 17, 3, -4, -1, 3, 0,
-12, 17, -41, -8, -4, -16, -2, -7, 6, 1, 2, -2,
1, 1, 1, 2, 11, -8, 15, 5, -4, 2, 1, 0 },
{ -6, 7, 2, 44, -6, -11, 25, -11, 11, 4, 34, 24,
34, 9, -13, 44, 5, 62, 16, 22, -2, 5, 0, -15,
-4, -32, 16, 19, -20, 7, -31, -3, -4, 6, 1, 1,
1, 1, 6, 6, 2, 13, -6, 2, 3, -3, 2, -2 },
{ 1, 2, -21, 11, 14, 6, 8, -27, 22, 13, -52, -30,
13, 7, 5, 25, -2, 20, -42, -45, 5, -4, -1, 29,
41, -6, -3, 4, -13, 11, -19, 33, 23, -3, -3, -2,
-3, -1, 6, -6, -15, 4, -4, -12, -6, 2, 5, -3 },
{ -4, -2, 25, 33, -23, -8, -19, 33, -10, 11, -28, 10,
23, -16, -24, -44, 48, 23, 20, -31, 0, 0, -4, 31,
4, -15, 16, -9, 11, -41, 6, -8, 16, 6, -2, 1,
-1, 4, 4, -6, -9, 11, 1, 6, 1, -2, 0, -2 },
{ 5, -7, -21, 25, 8, 18, -23, -37, -4, -5, 17, 4,
5, -19, 12, -62, -27, 27, -15, 16, 2, 2, 2, -4,
-6, -30, -49, -26, 29, -2, -26, 16, -10, -1, 0, 2,
0, -1, -1, -3, -7, 11, 11, -10, 2, -4, 0, -4 },
{ 2, 0, -5, -4, 0, 3, 15, -28, -10, -17, -38, 46,
-9, -11, 14, 11, -37, 3, 52, -33, -4, -3, 0, 33,
-37, -7, -12, -2, -1, 21, -5, -28, 23, 5, -2, -3,
0, 8, 2, -23, 19, 1, 1, 3, -14, -2, -2, -3 },
{ -5, 5, 1, 37, 10, -4, 10, 10, -51, -4, 4, -21,
-9, 5, -5, 30, 8, -41, -29, -24, -4, 2, -9, -12,
-27, -63, -33, 11, -9, -10, 18, -11, 1, -5, -6, -2,
-5, -2, -6, -4, 5, 12, 19, 2, -4, -6, -5, 1 },
{ 1, -4, -23, 14, 30, 1, -13, -16, -29, 32, 12, 13,
-20, 41, -3, -18, -6, -15, 23, 5, 16, 7, 7, 3,
40, -25, 41, -17, 6, 30, 22, -21, 18, -5, 5, 5,
3, -1, 1, 6, -26, 23, -27, 9, -6, 1, 1, 2 },
{ 2, -3, -4, 13, 8, 2, -9, -10, -2, -37, 0, -2,
25, 27, -6, -23, -1, 3, -21, -8, -31, 1, -17, 0,
-45, 4, 48, 9, 15, 28, 36, 43, 7, 13, 1, -7,
-2, 0, 4, 0, 33, 14, -15, -21, 6, -2, -3, 0 },
{ 0, 6, 1, -3, 1, -5, 14, -4, -4, -20, -9, 0,
-6, 12, -33, 17, -7, -12, -3, 37, 5, -2, 13, 50,
0, -12, 4, -70, -31, -25, 2, 18, -33, 4, 6, -4,
3, 3, 5, -24, 8, 6, -13, -14, 23, -7, 0, -2 },
{ -1, -4, 0, 15, -2, 2, -13, 0, -15, 6, -12, -12,
6, 1, 24, 1, 16, -4, 11, 44, 18, 0, 11, 58,
11, -5, -4, 65, 19, 29, 10, 7, -35, -1, 5, -2,
0, 8, -6, -28, 6, -12, 18, 3, 20, -4, -5, -1 },
{ -4, 4, -10, 40, 2, -1, 4, -13, -30, 27, 21, -4,
50, -12, 3, 12, -22, -8, 19, -6, -6, 4, -14, 4,
21, 46, -23, -21, -6, -7, 44, 9, 5, 16, -8, 2,
-4, 2, 12, 3, 9, -46, 1, -3, 1, 2, 3, 1 },
{ 0, 0, -1, 11, -1, 1, 3, -4, -3, -19, -10, -6,
24, -2, -4, -2, -6, 20, -8, -15, 67, -1, 60, -16,
-17, 14, 2, -7, -5, 3, 30, -3, -6, -47, -4, 6,
5, -2, -31, 6, 15, 8, -4, 17, 6, 21, -5, 13 },
{ 0, 2, -12, 7, 14, 2, 1, -12, 7, 29, -13, -45,
14, 20, 1, -8, -14, 10, -15, -8, 2, -5, -24, -1,
-35, 9, 24, -6, 17, -18, -25, -65, -36, -1, -4, 1,
-3, -1, -2, -21, 10, -23, -27, 13, 10, -7, -5, 1 },

```

```

{ 1, -2, 1, 13, 3, 3, -3, 2, -21, -32, 10, 28,
  19, 13, 6, -8, 0, -36, -4, -11, 3, 3, 21, -1,
  -6, 0, 20, 18, -6, -17, -49, 13, 0, -15, -3, 1,
  2, -1, -12, -12, -45, -58, -28, -28, -10, 9, 2, -5 },
{ 0, -2, -2, -11, 1, 3, -1, -5, 12, -19, 43, 2,
  -6, 0, 8, 8, -17, 21, -45, 12, -3, 11, 17, 35,
  -2, 2, 6, 16, 11, -41, 34, -27, 50, 9, 8, 5,
  6, 10, 10, -35, -15, -2, -2, 19, -20, -1, 5, -6 },
{ 1, -3, 1, 2, 4, -1, -1, -6, 1, -19, -37, 37,
  -7, 14, 3, 24, -29, 14, 1, -2, -15, -16, -11, -28,
  27, -8, 6, 13, 47, -47, 35, 9, -41, -12, -9, -7,
  -2, -5, 6, 3, -25, 1, 2, 11, 18, -8, 4, 8 },
{ 2, 0, 5, 8, -6, 6, -6, 5, 0, -20, -15, 20,
  8, -9, 9, -19, 12, 15, -27, 4, -20, 1, -21, -8,
  -2, 16, -17, -3, -60, 42, 30, -29, -31, -10, -2, -4,
  -3, -10, 8, -22, -52, 9, -11, 14, -8, -4, -1, -6 },
{ 2, -3, -3, -2, 6, 7, -10, -5, -1, 11, -11, -9,
  -13, 6, 22, -28, -22, 4, 15, 6, 0, -15, 11, -4,
  -4, -14, -12, 42, -67, -45, 11, 16, 3, 21, -2, -3,
  1, 4, 0, 23, 22, 4, -48, 8, 11, -6, 8, 5 },
{ 0, 1, 0, -7, 0, 2, -1, 1, 7, 3, 7, -3,
  -2, -1, 1, -2, -1, 1, 15, 19, 20, 0, -61, 10,
  -5, -10, -4, 7, -5, -15, 8, 10, 16, -81, -11, 1,
  -10, -8, 34, -7, 11, 0, -12, -18, -9, 39, 2, -17 },
{ 0, 0, -3, -20, 3, 1, -3, 0, 19, 13, 40, -1,
  -16, 5, -1, -9, 4, 14, 25, -52, 7, 16, -3, -17,
  13, -17, 7, 4, -18, -2, 16, 21, -26, 3, -17, 7,
  -6, -4, -17, -72, 8, -10, 16, -18, 14, -9, 2, 6 },
},
{
{ 94, -70, -33, 5, 12, 33, -15, 0, 1, -3, 2, -1,
  -5, -6, 5, -2, 5, 4, 2, 0, -1, 0, -1, -1,
  -2, -2, -3, 0, 0, 2, 0, 0, 1, 1, 0, 0,
  0, 0, -1, 0, 0, 0, -1, 0, 0, 0, 0, 0 },
{ 59, 23, 62, -31, -54, -26, 24, 38, 26, 12, -1, -6,
  -12, -20, -12, 8, 7, 0, -1, 0, 3, 0, 0, 3,
  6, 7, 0, -2, 2, -2, -7, -6, -4, -2, 0, 2,
  0, 1, 2, 3, 4, 5, 1, -2, -1, -2, -1, 0 },
{ 49, 59, -52, 34, 28, -53, 44, -12, -16, -7, 2, 3,
  7, 6, -20, 11, -6, -2, 1, 2, 0, 1, 0, -2,
  -1, 1, 4, -2, 1, -3, 1, 0, 1, 0, 1, 0,
  0, -1, 1, 0, -1, -1, 1, 1, 0, -1, 0, 0 },
{ 40, 57, 49, 9, 15, 39, -39, -33, -16, 1, 5, 11,
  20, 35, 40, -16, -18, -10, -6, -3, 2, 2, 3, 1,
  -1, -1, 6, 11, -1, 0, 7, 7, 3, 0, 0, 1,
  1, 0, -2, -5, -7, -6, -1, 5, 4, 3, 1, -2 },
{ -3, -16, 16, 72, -45, -7, -5, 34, -51, -42, 13, 23,
  29, -24, 6, -8, 10, -18, -10, -6, -5, 4, -1, 0,
  0, 8, -5, 4, -3, 0, -3, 4, 2, 2, -2, 0,
  -2, 1, -3, -3, -5, 0, 1, 5, 3, 2, 0, -1 },
{ 1, -51, 52, -2, 23, -21, 32, -40, -23, 9, -2, 7,
  30, 34, -43, 27, -21, -25, -11, -3, 4, 1, 0, 1,
  6, 16, 15, -17, 3, -8, -7, -1, 2, 0, -1, 1,
  0, -1, -1, -4, -2, 2, 5, 3, 3, 2, 1, -1 },
{ 2, -14, -41, -32, -53, -43, -20, -13, 11, -13, 4, 2,
  11, 34, 27, -29, -46, -26, -18, -9, -7, -2, 0, 8,
  14, 21, 24, 10, -7, -12, -7, -5, -3, -4, -2, -3,
  -1, 1, 0, -4, -4, -1, 1, 6, 7, 2, 1, -4 },
{ -1, 13, -5, -32, 26, 30, 16, 9, 23, -60, 33, 57,
  -5, -7, -11, 14, 8, -5, -40, -24, -14, 2, 12, 16,
  28, 13, 1, -3, 3, -1, -12, -15, -3, -3, 2, -2,
  2, -3, -2, -1, 7, 9, 3, -1, 6, 4, 3, -4 },
{ 2, -25, 37, 26, 31, -62, -15, -10, 28, -23, 29, 15,
  -44, 5, 16, -30, -4, 39, -2, -10, -6, 7, 5, 0,
  -6, -27, 5, 11, -10, 0, 17, 5, 1, -2, 1, 0,
  1, -2, 0, 0, -2, -7, -1, 0, 1, 0, 1, -1 },
{ -4, -7, 7, 14, 60, -10, 5, 63, 17, 33, -10, -3,
  18, 0, 28, -30, 0, -31, 1, 12, 6, 1, 5, 9,
  20, 36, 22, 20, -8, -2, -19, -13, -12, -3, 2, 2,
  3, 5, 6, 9, 6, 7, 6, 4, 0, -1, -3, -4 },
{ 7, -1, 1, -40, 18, -44, -34, -27, -32, 14, -14, 9,

```



```

27, -40, 5, -16, 45, -16, -26, -8, 3, -5, 1, 11,
12, -3, -45, 3, -8, 22, 3, -7, -11, -6, -4, 1,
-1, 1, -1, 5, 7, -2, -14, -5, -3, 1, -1, 0 },
{ -3, 4, -4, 48, -18, 9, -15, -32, 20, 44, 45, -22,
-10, -8, -8, 11, -1, -2, -9, -33, -10, 12, 12, 31,
24, 13, -9, -10, 5, 0, -13, -33, -28, -14, -5, 3,
-1, 2, 7, 20, 24, 12, -3, -15, -6, 1, 1, -6 },
{ 1, 5, -2, 34, -4, -15, -49, -5, 58, -12, -31, 20,
6, 11, -15, 22, 6, -10, -19, 44, 23, -5, -12, -23,
11, 19, -16, -36, 14, 13, -5, -8, 14, 6, 11, 3,
2, -3, -4, -8, 1, 9, -7, -2, 2, -1, 1, 1 },
{ -2, -2, 8, 14, -8, 17, 33, -42, -15, -10, -47, 7,
-44, -32, 14, -22, -15, 2, -29, 25, 19, -7, -13, -21,
24, 4, 8, 29, -10, -17, -17, -31, -7, -5, 1, 2,
-1, -3, 9, 11, 15, 6, 0, -4, -6, -9, -6, 3 },
{ 3, 12, 7, -13, 22, -8, -45, -9, -16, -23, 15, -23,
-29, -45, -23, -1, -18, -27, 30, -4, -16, 2, -4, -17,
-34, 29, 18, -14, 0, -28, -41, 1, 6, 6, 0, -6,
-2, -6, -11, -3, 2, 22, 10, 1, 1, 1, -4, -2 },
{ -3, -5, 1, 10, -7, 7, 42, -42, 54, 8, 23, 3,
26, -39, 20, -14, 8, -26, 10, 5, -6, 7, 5, -9,
-26, 22, -20, 27, -7, 6, -3, 29, 14, 6, 3, -1,
3, -4, -12, -15, -27, 0, -10, 16, 11, 10, 0, -2 },
{ 0, -5, 10, 1, 3, -6, 22, -8, 1, -53, -13, -46,
-20, 41, 39, 13, 32, -28, 18, 3, -28, 3, 2, 0,
-8, 16, -22, -7, 17, 35, -15, -9, -15, -12, -2, -6,
-4, 3, 9, 16, 6, 2, -14, -8, -13, -3, -1, 0 },
{ 2, 1, 4, -2, -6, 1, -6, -18, 20, -30, -19, 5,
62, -1, -19, -18, 9, 46, 39, 17, -15, 2, 16, 18,
-6, 8, 13, 18, -16, -16, -6, -6, -13, -3, 1, -5,
5, 11, 15, 27, 24, 22, 14, -16, -23, -16, -5, 8 },
{ -1, -2, -1, -11, -5, -7, 10, -7, -20, 5, 37, 3,
5, 18, 23, -4, 23, 37, 26, -3, 32, -9, -37, -15,
21, 33, -3, 4, 3, 11, -21, -13, 32, 42, 18, 7,
-7, -12, -17, -11, 29, 23, -1, -14, 9, 15, 10, -1 },
{ -1, -8, -1, -9, 1, -15, 7, -7, -14, 16, 2, 41,
-32, 66, 43, -26, 11, 26, 22, -11, 7, 28, 2,
5, -1, 9, -35, 41, -4, 0, 6, 0, -9, 0, -2,
8, 7, 12, 6, 10, 6, 0, 0, -7, -5, -4, 2 },
{ 1, 1, -6, 0, -15, 1, -5, -2, -18, 34, 1, 33,
-29, 33, -18, -13, 37, 10, -4, 18, -36, 25, 41, -30,
2, 25, 0, 12, -12, 8, -27, 23, 26, -27, -14, -6,
9, 2, 16, -5, -5, 19, 6, 6, 1, -2, 4, 3 },
{ 0, 0, 6, 1, 11, 0, 3, 11, 3, -4, -8, -45,
29, -16, 6, 4, -28, 21, -26, 0, -55, 19, 13, -31,
25, -10, -13, 1, 0, -1, 8, -28, 45, 4, -15, -14,
2, 0, -5, -32, 17, 0, -10, -16, 19, 6, 12, 1 },
{ 0, 1, 3, -4, -1, 7, -1, 3, -10, -8, 14, 13,
-18, 0, -32, -21, -40, -22, 55, 38, -5, 5, 16, 22,
42, -3, -41, 4, -15, 23, 6, -5, 9, 13, 11, 1,
4, 6, 4, 3, 10, -26, -31, 5, 9, 2, -4, -3 },
{ 0, 2, 3, 8, 3, -2, -7, -11, -6, -12, -33, -30,
-13, -15, -1, 6, 11, 34, 1, -9, -4, -14, 9, 50,
47, 36, 13, -8, 8, -3, 5, 37, 8, 3, 4, -2,
3, 8, 5, -4, -33, 6, 6, 35, 28, 21, 9, -17 },
{ 0, 1, 2, 0, 7, 5, 9, 16, -6, 4, -14, 10,
5, 5, -5, -22, -51, 46, -13, -32, 7, -7, -10, -12,
-30, 35, -49, -17, 10, 18, -27, 6, -21, -11, -6, -1,
1, 6, 10, 5, -22, 14, -35, 3, -9, -3, -5, -4 },
{ -1, 1, -1, -13, 6, 3, -4, 0, -6, -14, 52, -31,
13, -11, -3, -1, 1, 10, -16, 29, 26, 23, -28, -29,
30, 17, 7, -6, 5, -2, 16, 22, -1, -29, -15, 2,
-4, 11, 38, 18, -17, -12, -2, 18, -10, -37, -11, 22 },
{ 0, -1, -1, -1, 1, 1, 10, 3, -3, -6, 13, -23,
4, 5, 4, -24, -6, -4, -34, 28, 33, 4, 29, 48,
-16, -42, -11, -14, 17, -5, -50, 17, 45, 3, 3, 9,
3, 0, 1, 10, -1, 25, 1, -13, -3, -11, 1, 1 },
{ -1, -1, -1, 4, 3, -6, -8, 1, 10, -6, -21, 10,
-9, 4, 7, 34, -6, -13, 17, -36, 33, 41, -17, 14,
5, -6, -17, 29, -16, -6, -8, 12, 22, 15, -56, -11,
8, 46, 15, -13, 13, 16, 0, -1, 4, -7, -19, 14 },

```

```

{ -1, -3, 1, -2, 3, -7, -12, 11, -9, -1, 10, -10,
  -10, 6, 9, 60, -21, 7, -25, 23, 3, -15, 13, 16,
  -24, 31, -33, 51, -37, -18, 22, 15, 8, -12, 27, 10,
  -5, -16, -4, 18, 15, 8, 2, -8, -2, 0, 11, 2 },
{ 0, -1, 1, 7, 3, -4, -6, 2, 11, 4, -2, 4,
  9, 2, 5, 17, -7, -12, 19, -29, -20, -49, -19, -21,
  44, -41, -19, 16, -7, -10, -45, 30, 5, -13, 22, 7,
  -11, -17, 27, -8, -15, 10, 12, -1, -24, -14, 10, 11 },
{ 0, 0, 1, 4, -1, 2, 3, -4, 2, -3, -5, 1,
  -7, -6, -2, -16, 6, -12, 10, -35, 14, -44, 25, 8,
  -22, 27, 11, -15, 1, 6, 32, -19, 45, -15, 15, 20,
  -1, 4, 50, -26, 18, -8, -6, -8, 14, -34, -4, 28 },
{ 1, 1, 0, 3, 1, 4, 7, 1, 7, 2, -14, 4,
  -5, -4, -2, -23, -6, -16, -14, -23, -10, 7, -5, -14,
  8, 15, -4, -26, 4, -15, 19, 58, 20, 27, -4, -4,
  -2, -8, 5, 59, 42, -20, 1, -22, -20, 26, 20, 15 },
},
{
{ 113, -33, -41, 19, 6, -14, 8, 5, 0, -3, 0, -4,
  -4, -3, -1, 1, -1, 0, 2, 3, 1, 0, -2, -1,
  1, 2, 0, -1, 0, 0, 0, -1, -1, 0, 1, 0,
  -1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 53, 52, 89, -32, -9, -24, 5, -21, 8, 18, 0, 5,
  9, 11, 4, 1, -1, -6, -8, -3, 1, 2, 5, 3,
  1, 1, 0, 1, 1, 0, -1, 2, 1, -3, -1, 1,
  1, 0, -1, -1, 0, 1, 0, 0, 1, 2, 1, 0 },
{ -3, -97, 55, -11, -53, 19, 7, 8, 9, 4, 6, -9,
  -9, 6, -12, 1, -7, 4, 1, -3, -2, 2, -2, -1,
  0, 1, 3, -1, -1, -1, -2, -1, 1, 0, -2, 0,
  -1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0 },
{ 24, 37, 17, 32, -16, 100, -25, 32, -4, -18, 9, 2,
  -10, 1, 2, 9, -4, 16, 13, 0, -2, 1, 0, -3,
  -6, -7, 4, 2, 3, -1, 0, 2, 4, 4, 1, 1,
  0, 0, 0, -1, -1, 1, 1, 1, -1, -1, 0, 1 },
{ -8, 17, 10, 78, -58, -51, -2, -10, -13, -44, 22, 7,
  -16, 20, 12, -5, -6, 6, -2, -5, -1, 0, -5, -2,
  -1, -2, -2, -1, -2, 2, 4, 4, 1, -2, 2, 3,
  -3, -1, 1, -1, -4, -1, 0, 0, 2, 0, -1, 0 },
{ -2, -31, 37, 40, 76, 14, -5, -64, -16, -16, 12, 14,
  -9, 3, 1, -5, 15, 14, 8, 6, -1, 1, -2, -15,
  -17, -5, -5, 2, -1, 0, -2, 4, 8, 6, 5, 1,
  -1, -6, -2, 2, 3, 1, 0, -1, -2, -2, -1, 1 },
{ -3, 14, -14, 2, -6, 31, 101, -23, 4, -5, 0, -5,
  0, 55, -24, -4, -6, -6, 1, 3, -1, -1, -1, -4,
  11, 11, -6, 10, 2, 4, -4, -2, 3, 2, 1, 0,
  0, 0, 4, 1, 0, -1, 0, 0, -1, 0, 2, -2 },
{ 6, 6, -35, -38, -42, 19, -45, -81, 21, -16, 7, -23,
  -6, 23, 4, 5, 8, -7, 9, 1, -13, 1, -2, -12,
  -14, 5, -12, -1, -2, -3, -3, 4, 8, 2, -2, -2,
  -1, -2, -7, -2, 1, 2, -3, -3, -1, 3, 0, 0 },
{ -6, 7, 8, 21, 17, -8, 0, 11, 97, 4, 4, -15,
  -60, -10, -10, 2, 5, -15, 12, 21, -3, 3, 4, -4,
  1, 24, -2, -1, 0, 1, -7, -5, -11, -7, 3, -1,
  4, 3, 4, 4, 1, 1, -1, 0, 1, 1, -5, -2 },
{ 3, -4, 5, -36, 19, -9, 3, 27, 19, -79, 50, 26,
  31, 0, -14, 1, -10, -26, 20, -13, -31, 1, 5, -1,
  -11, 10, 7, -2, -1, 0, -1, 2, 8, 12, 6, -2,
  -3, 0, -7, -8, -1, 0, 1, 1, 2, 2, -1, -1 },
{ 0, 9, -1, 7, 2, -10, -25, -10, -14, 6, 1, 0,
  -5, -5, -107, 19, -50, 10, -11, 2, 3, 2, -2, -4,
  2, 2, 7, 1, -7, -2, -11, -12, 1, -1, 0, 1,
  0, -3, 0, 0, 3, 3, -7, 0, -1, -4, 1, -1 },
{ 1, 10, 2, 6, -28, 11, 37, -30, -10, -11, 11, -5,
  6, -99, -16, 9, 36, -21, 4, -2, 0, -2, -3, -5,
  2, 5, 5, 2, 3, -12, -2, -13, -12, 3, 0, 2,
  -3, -1, 1, 8, 10, -3, -1, 0, -3, -3, -6, 0 },
{ -2, -9, -4, 43, -4, 18, -16, -12, 36, 36, 20, 24,
  68, 7, -5, -2, 3, -51, -41, -1, 3, 9, 7, 13,
  11, 7, -2, 3, -3, 1, 13, 5, 1, 0, 1, 2,
  0, -3, -12, -10, -6, -2, -1, 1, 5, 8, 6, 1 },
{ 2, -3, -14, -19, -8, 10, 7, -9, 0, 1, 8, 79,

```

```

-53, -8, 5, 5, 2, 14, -60, -29, -6, -2, 25, 8,
-6, 7, 4, -1, 2, -3, -11, 19, -2, -13, -1, 0,
4, -8, -5, 1, -9, 2, 0, 3, 3, 5, 4, 0 },
{ 0, -5, 1, 4, -8, 1, 9, -19, 39, -36, -69, 33,
36, -10, 6, 19, -25, 26, 9, 21, 24, -21, -3, 27,
-20, -18, 17, -5, 3, -2, 1, 9, -14, 0, 6, -2,
3, 8, 5, 0, -1, 9, -2, 0, 0, -6, -5, 3 },
{ 3, -1, 5, 1, -4, -4, -19, 18, -5, -21, -31, 8,
10, 27, -44, -7, 90, 7, -4, -6, 8, -10, 0, 20,
2, 34, -27, -2, -3, 1, -6, 3, -10, -3, -1, 1,
-1, 5, 5, 16, 4, 4, 9, -6, -5, 0, 4, 2 },
{ 2, -3, 9, 0, 3, 10, -10, 1, -33, -18, -45, 3,
-27, -4, 6, -27, -33, -81, 2, 3, 18, -16, -8, 1,
-16, 33, -19, 18, -1, 11, -2, 8, -12, -1, -1, 2,
1, 0, -8, -6, -6, -15, -3, -1, 7, 6, -6, -2 },
{ -2, -4, 7, 3, 17, 3, -3, -4, -11, -29, -13, -39,
-11, 13, 17, 77, 7, -24, -38, -14, -2, -6, -10, 15,
37, 12, 45, -16, -1, -9, -4, -5, 1, -11, -2, -3,
-5, -4, 0, -4, 3, 4, 7, -3, 5, 10, 10, 0 },
{ 3, 0, 6, -6, 11, 10, -4, -7, 15, -40, -10, -36,
2, -17, -4, -59, -13, 20, -53, -6, -16, -1, -8, 16,
47, -28, -28, 14, 2, -3, 14, -3, -5, -15, -2, -2,
-4, -1, 12, -2, -20, -3, 0, 5, 4, 2, 6, -5 },
{ 0, 2, 1, -11, -6, -4, 1, 19, -2, -11, -3, 5,
2, 5, -2, 15, 16, -11, -46, 71, 1, -1, -24, -67,
-15, -28, -7, 2, 0, 3, -3, 11, 3, 5, 26, 1,
-1, -20, -4, 13, -10, 0, 6, 1, -13, -1, -2, 4 },
{ 1, 0, -3, -12, -4, 3, -12, -10, -5, -1, 22, 15,
7, -2, 10, -25, -3, 26, 4, 30, 33, -13, -20, -6,
43, 61, 52, 31, -15, 13, 3, 0, -3, 10, 22, 10,
-3, -1, 15, 12, -6, -2, -7, 16, 11, 2, -3, -7 },
{ -2, 1, 3, 8, 2, -2, 10, 8, 5, 1, 1, -36,
24, -21, 14, 16, -18, 36, -29, -11, -8, -1, -5, -2,
-41, 59, -32, 18, -4, 18, -39, 35, 23, 0, -4, -3,
-2, -3, -8, -3, -5, 9, -10, -12, -3, 9, 3, 0 },
{ 0, -4, 4, 2, -3, 0, -5, -5, -10, -1, -11, 29,
6, -8, 15, 34, -21, 4, 24, 28, -27, 12, 23, -7,
65, 22, -64, -17, 3, 11, -12, -11, -5, 8, 1, -11,
9, -1, 8, 15, 1, 1, 2, 4, -10, 7, 9, -5 },
{ 0, -3, -2, -12, 4, 1, -1, 0, 1, 0, 46, 1,
-7, 1, 3, 35, 2, -7, 4, 1, 49, -33, -16, 33,
6, -34, -44, 30, -9, 35, -8, 5, -14, -26, 13, 24,
-9, 10, 14, -9, 0, 5, -5, -9, -5, -12, -14, 2 },
{ 0, -1, 0, 2, -3, -2, -1, -2, -16, 0, -3, -8,
-3, 1, -4, 8, 13, -4, 3, 38, -49, 32, 36, 33,
-13, -16, 27, 60, -7, 40, -3, 13, -32, -10, -5, -15,
10, 9, -5, 3, -2, 8, -8, 5, 12, 6, 1, -6 },
{ 0, 2, -1, -4, 1, 4, 4, -4, -19, 5, 29, -12,
-5, 1, -4, -16, -6, -3, -1, 51, -1, 2, 0, 49,
-8, 8, 0, -62, 7, -41, -18, 46, -22, -7, 15, 3,
2, 16, 0, 0, -15, 1, 3, -7, 12, 5, -6, -3 },
{ 0, 0, 0, 2, -3, -6, 3, 2, 3, 2, -11, -3,
-13, -13, -15, 5, 11, -11, 21, 8, 6, -19, 23, 6,
27, -19, 2, 21, -9, -11, 18, 66, 79, 18, -1, 6,
9, 3, -1, -9, -8, 1, 1, 10, 1, 12, 13, 11 },
{ 0, 2, 0, 4, 1, -1, -2, -2, 8, 14, -8, 4,
-1, 1, -5, -2, 2, -8, 17, -40, -24, -2, -48, -14,
25, -20, 9, -4, -8, 16, -29, 56, -46, 40, 1, -3,
-18, -11, -2, 16, -22, 18, -9, -1, -3, -5, -3, 5 },
{ 0, -1, -1, 5, -2, -1, -6, 0, 1, -2, -2, 4,
7, 5, 7, -20, 6, -17, 4, 2, 12, -2, 12, -2,
11, -24, 15, 16, 6, -22, -105, -26, 25, -6, -4, -3,
8, 9, 5, -1, -5, 19, 8, 9, -5, -1, 5, -19 },
{ 0, 1, 1, 0, -2, -3, 2, 2, 0, 11, -2, 6,
-17, 2, 7, 6, 1, 4, -15, 14, -28, -12, -36, 53,
-9, -2, -15, 21, -12, -20, -4, -40, 17, 69, 29, -1,
-10, -6, -26, -8, 13, -13, -2, 10, 2, -13, 6, 11 },
{ 1, 2, 1, 2, -3, 5, 3, -3, -2, 3, -3, -3,
-6, -7, -2, -20, 5, -9, -6, 6, -15, -30, 9, 13,
-2, -6, 18, -59, -11, 81, -3, -23, 33, 3, 13, 6,
-1, 1, -10, 10, -9, 29, -25, -8, -7, 4, 3, -4 },

```

```

        { 0, 0, -2, -6, -1, 0, -2, -2, 2, -12, 4, 2,
          -7, -4, -3, -9, -4, -5, -11, 5, 37, 58, -16, 16,
          6, -3, 0, -6, 59, 36, -3, 13, 14, 31, -12, -10,
          1, -9, 9, 4, 46, 17, 27, -15, 17, -7, 11, 10 },
    },
},
{
    // 11
    {
        { 65, 93, -25, 4, -15, 31, -1, 22, -7, -2, 1, 3,
          -16, 23, -2, 1, 8, -4, 5, -2, -1, 1, -2, -3,
          7, 2, 1, 1, 2, 4, -4, 2, 1, -1, 1, 1,
          1, 1, 1, 1, -1, 0, 1, 0, 0, 1, 0, 0 },
        { -67, 19, -22, 5, -59, 56, 23, -41, 12, -4, 0, -4,
          -12, 3, 3, 1, 16, -30, 9, 0, -2, -2, -5, 9,
          -17, 6, 3, 1, 2, -1, -5, -3, 2, 0, 4, -9,
          1, 1, 0, -3, 1, -2, 0, -2, 0, -2, 1, 0 },
        { -80, 29, -5, -12, 21, -4, -29, 59, -22, -2, -3, 0,
          -14, 36, -9, -1, 2, 19, -5, -3, -2, 0, 2, -16,
          31, -6, -1, -2, -1, 15, -7, 4, -1, -2, -1, 7,
          3, 0, 1, 7, -4, 1, 2, 1, 2, 3, -1, 0 },
        { 11, -20, 58, -20, 47, 59, 34, 4, -18, 11, -1, 1,
          -5, 34, -8, -6, 43, -18, -8, 3, 3, -1, 3, -1,
          -5, 16, 1, 0, 3, 17, -10, -2, -1, 2, 3, -13,
          14, 2, 4, -2, 2, -1, 0, -4, 4, -3, 2, 0 },
        { -21, 40, 14, -6, 33, 32, 40, 0, 14, -2, -1, -9,
          51, -34, 24, 4, -20, 45, -14, 3, 0, -1, 2, 13,
          1, 2, 2, 2, 4, -19, 37, -11, 0, 6, -12, 24,
          -10, 0, -2, 3, 5, 3, -3, 11, -2, 5, -2, 2 },
        { -16, 23, -23, -17, 32, -35, 20, 15, -41, 4, -5, 20,
          -22, -29, 13, -9, 13, -26, 7, -4, 0, 3, 6, 6,
          -55, 31, -3, 2, 15, -33, 5, 4, -2, 0, 4, -22,
          1, 5, 7, -30, 12, -1, -5, 4, -12, -7, 1, 3 },
        { -11, 30, 17, 11, 23, -15, -44, -10, 43, -1, 0, -1,
          40, 8, -52, -22, 21, -15, 21, 2, 0, 2, -8, 43,
          -22, -4, -4, -3, -11, 5, 13, -6, 6, 2, 18, -17,
          9, -4, 2, -11, 22, -10, -3, 2, 1, -5, 7, 4 },
        { 2, -16, -67, 22, 15, -23, 45, 15, 27, -21, 3, -11,
          30, 14, 10, -1, 48, 12, -5, 1, -6, 1, 1, 0,
          10, 23, 3, 2, 3, 34, -4, -3, 1, 1, -2, -9,
          29, -2, 10, 6, -2, 0, 0, -7, 13, -4, 2, -1 },
        { -6, 32, -9, -28, 24, -12, 1, -51, -21, 3, -3, -7,
          12, -39, -29, -4, -5, -9, -34, 10, -1, 0, 13, -32,
          5, -24, 6, 2, -21, 29, -33, -5, -2, 4, -13, -19,
          15, -9, -2, 10, -22, 4, 2, -24, 16, -10, -1, -6 },
        { 12, -26, -31, -23, -27, 23, -18, 1, -39, -2, -7, -17,
          7, 0, -50, -44, 16, 29, -32, -7, -2, -5, -1, -3,
          -6, 15, -14, -3, -6, -12, 33, -22, -3, 8, -21, 13,
          6, -6, 5, -12, 12, -1, -16, 17, -7, 1, -7, 6 },
        { -5, 18, 54, 33, -39, -18, -11, 19, 18, -4, 12, -31,
          2, -11, 16, -13, 5, 19, -35, 8, 2, -3, 2, -31,
          -15, 36, -1, 4, 9, -2, -14, -5, 1, 5, -16, -25,
          29, 0, 22, -25, -6, 4, -4, -8, -2, -15, -2, 3 },
        { 3, -17, -8, 10, -20, 34, 1, 39, -23, 9, 3, 19,
          18, -32, 0, -1, -19, 30, 18, -4, 3, -5, 20, 14,
          -9, -23, 19, 6, -32, 17, -5, 28, -6, 0, 28, -38,
          21, -11, -1, -17, 20, 3, 13, -16, 10, -24, 22, 9 },
        { -7, 14, 26, 25, -17, -22, 29, -12, -6, 30, 2, 60,
          -19, -21, -14, -44, 19, 18, 28, -14, 8, 7, 1, 0,
          19, 13, -18, -3, -11, 21, 14, 10, -8, -3, -11, 24,
          13, -12, 4, 23, -9, 7, -2, 6, 12, 11, -7, -3 },
        { 1, 9, 21, -17, -32, -27, -8, 5, -43, 14, -11, -13,
          18, -5, 23, 40, 28, -4, -3, -22, 4, -7, -25, 33,
          -12, -1, 30, 5, -4, 31, 19, -34, 0, -2, -3, 4,
          9, 9, -9, 26, 11, -21, -11, -3, 25, 5, 2, -3 },
        { 0, 11, 4, -40, -7, -27, 14, -46, 12, 10, -11, 16,
          -18, 40, 6, 20, -5, 34, -12, 7, -3, -3, 4, -4,
          30, 6, 14, 13, -5, -2, 20, 11, -15, -6, 23, -10,
          22, -7, 14, -32, 36, -2, 1, 16, -10, -15, 19, 22 },
        { 4, -6, -12, -44, -7, 20, -46, -8, 13, 21, -26, 38,

```

```

40, -4, 22, 7, 1, 12, 19, -15, 5, 3, 4, -5,
4, 40, -15, -9, 39, -1, -18, 6, 2, 0, -8, -6,
15, 14, 19, 2, -24, 8, 6, -16, 6, -4, -6, -9 },
{ 1, 8, 11, -13, -28, -18, 3, -1, -21, 4, 9, -19,
34, -4, 0, -2, 36, -7, -7, 35, -3, -16, 52, 6,
5, 0, -17, -14, 19, 8, -11, 40, -6, 8, 28, 17,
-30, 7, -7, 12, 10, 16, 25, 15, -19, 23, 11, 2 },
{ -5, -3, -14, 61, 13, 11, -22, -13, -11, 40, 7, 39,
15, 7, -18, 42, 23, 1, -27, 6, 12, 6, 21, -23,
-4, -5, 37, 16, 14, -10, 7, -5, -5, 12, -15, 4,
-15, 14, 0, -12, 6, 5, -9, 10, -9, 0, -4, 6 },
{ -1, 2, -6, 34, 18, 12, -10, -16, -28, 13, -3, -23,
14, -27, 16, -8, -2, -15, 4, -31, 0, -16, -31, -6,
42, -2, -25, -10, 19, 14, -24, -4, -14, -32, 22, 3,
6, -7, 27, -22, 21, -26, -1, 23, -24, 10, 5, 15 },
{ 2, -5, -3, -20, 8, 16, -34, 11, 22, -17, 10, 6,
-13, -35, 28, -1, 14, -32, 16, 33, -9, -11, 27, -21,
3, 8, 26, 19, -24, 16, 14, -2, -2, -9, -14, 25,
29, -19, 12, 5, 14, -9, -22, 38, 4, 17, -9, 19 },
{ 2, 1, 6, -24, -20, -15, 25, 25, 16, 28, -24, -5,
39, 11, 4, -15, 21, -28, 15, -33, 12, 6, -2, -27,
11, -33, 17, 24, -27, -23, -18, -4, 11, 0, -17, 5,
-19, -24, 0, -28, -10, 8, -7, 8, -25, -9, -11, 4 },
{ -2, -1, -10, 9, 1, -4, 0, -4, -24, 40, 21, -9,
10, 25, 27, -3, -23, -29, 6, 38, 13, 10, 18, 28,
16, -15, -34, -15, -3, -2, 18, -21, 29, 31, -16, 7,
28, -24, 26, -15, -7, 8, -2, -8, 14, -19, -1, 6 },
{ 0, 2, 11, 11, 8, -2, -8, -25, -34, -42, 9, -29,
-2, -8, -13, 4, 18, 10, 59, -15, -1, 9, 3, -4,
31, 11, 35, 17, 14, -20, 10, 13, 31, 19, -8, 12,
6, 6, 9, -15, -8, 29, 15, -10, 11, -18, 1, 2 },
{ -1, 2, -4, 9, 6, -3, 5, -12, -17, 17, 8, -24,
15, 32, -14, 10, -31, 13, 13, -5, -5, -27, 7, 27,
-27, 25, 11, 33, -15, -10, -28, 38, -26, -13, -6, 9,
22, -25, 8, 9, -27, -3, -17, 18, 4, 23, -34, -9 },
{ 1, -1, 12, -14, -17, -5, 22, 11, -12, -41, 27, 40,
13, -3, -27, 20, -8, 12, 14, 27, -15, -2, -4, 20,
7, -20, 1, 0, 22, 18, -25, -25, 8, 1, -16, -13,
-8, 6, 36, -19, -14, -22, -23, 11, -35, 6, -19, -2 },
{ 0, 0, 0, -6, -4, -3, 15, 15, 14, 26, 6, -17,
-5, -9, -26, 6, 3, -14, 26, 12, -16, -53, 22, -14,
23, -14, 3, -9, 28, -25, 33, -14, -41, -29, -5, -8,
6, 20, 4, -6, -4, -15, -10, -33, 24, -14, -9, -16 },
{ 0, 2, 3, 13, 8, 2, -27, -24, -18, -31, 11, 25,
8, 20, 54, -23, 36, 14, -9, 2, -16, -30, -1, 7,
2, -23, -6, -4, -27, -16, 17, -7, -22, -11, 8, -21,
-14, -20, -9, -10, -22, 9, -3, -21, -11, -4, -25, -21 },
{ 0, -2, -14, -20, 5, 1, -6, -3, 6, 48, 62, -12,
-7, -11, 8, -23, 10, 26, 16, 30, 5, -5, -32, 1,
17, 27, 21, 14, -9, -1, -21, -14, 23, -2, 11, -15,
-21, 14, -36, -1, -3, -11, -3, -3, -15, 0, -2, -3 },
{ 1, -3, -5, -4, 4, 4, -7, 12, 17, 17, -24, -9,
-34, -29, -5, -2, 16, -3, -31, 15, 2, -4, -20, 53,
11, -3, 22, 25, 5, 17, 4, 10, -15, 3, 8, 27,
-14, -5, 41, -16, -29, 25, 25, -15, -19, -19, -9, -10 },
{ 0, 1, -3, 2, 9, 2, -2, -7, 15, 23, -30, -32,
-34, -11, 7, 11, 14, 32, 21, -16, 7, 0, 31, 8,
-2, -23, -2, -32, -4, 17, 7, -2, 18, 24, -4, -39,
12, 17, 1, -5, -4, 10, -25, 13, -31, 35, -31, -13 },
{ 0, 2, 0, 6, 7, -2, -2, -6, 1, 1, -13, 0,
0, 8, 12, -10, -9, -2, 15, -3, -26, -40, -3, 15,
-6, 8, -1, -21, -17, 31, -23, 8, -24, 28, -55, 18,
-17, 37, -24, -21, 5, 12, -12, 11, -6, -36, 24, 27 },
{ -1, 0, -1, -17, 2, 2, -4, 6, 14, 4, 54, 1,
-9, -13, -18, 59, 23, 6, 3, -34, 5, -25, -20, -12,
-17, 0, -35, -27, -20, -4, 8, 4, -5, 5, -11, 11,
9, -34, 7, -3, -8, 29, 23, 4, -12, -2, 11, 8 },
},
{
{ 116, -32, 27, 7, 12, -16, 12, -21, 5, 5, 2, 3,
-3, 1, -2, 0, 5, -4, -4, 2, 2, 1, -2, 0,

```

```

0, 1, -1, 1, -1, 1, 0, -1, -1, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 30, 102, -23, 2, 34, -40, -9, 28, 0, -3, 0, 6,
12, -24, 9, -1, 1, 2, 4, 1, -1, 2, 4, -1,
-6, 2, 1, 0, 1, 0, 0, 1, 2, -1, -1, -1,
0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ -32, -33, -20, -4, 32, -81, 37, -43, 3, -3, -1, 4,
9, -38, 6, -7, 26, -21, -5, 1, 0, 2, 2, -4,
-5, -3, 0, -1, -3, 7, -2, -6, -2, -2, -1, 0,
-2, -1, -1, 1, 1, -1, 0, 0, 0, 0, 0, 0 },
{ 4, 33, -30, -7, 18, 35, 56, -37, 7, -3, -1, 0,
-7, 51, -38, 3, 38, -22, -1, 1, -2, 0, -2, 0,
33, -29, 5, 1, 4, 7, 0, -5, 0, 0, 6, 7,
-9, 2, 1, 1, 1, 1, 2, 0, -2, 0, 1, 0 },
{ 0, 18, 31, 15, -38, 30, 41, 11, -13, 4, 5, -4,
5, -33, 30, -25, 61, -15, -3, -4, 1, -2, -1, 5,
-27, 28, 7, 0, -20, 40, -8, -3, 0, 1, -4, -1,
3, 5, -7, 11, 5, -3, 0, 1, -1, 1, 3, 0 },
{ -23, 16, 90, 27, 8, -32, -19, 8, 6, 17, 7, 6,
-4, 12, -27, -6, 14, -1, -3, 2, 5, 2, -1, -3,
29, -42, 7, -5, -7, 13, -3, 0, -1, -1, 3, 13,
-24, 6, 1, 1, 1, -1, 3, -1, -5, 0, 0, -1 },
{ -18, 32, 50, 3, 14, 5, 26, -50, 34, 8, 1, 10,
-26, 16, 29, 11, -25, 3, -5, 8, 1, 4, -5, -5,
-15, 44, -10, 4, 18, -31, 10, -2, -3, -2, -2, -14,
33, -8, 4, -7, -2, 2, -4, 0, 10, 0, -1, 2 },
{ 6, 13, -3, 29, -55, -4, 30, -13, -24, 4, 5, -9,
7, -31, 19, 32, -16, -9, -7, -5, 0, -3, -3, 2,
5, -32, 8, -4, 44, -42, 3, 0, -2, 0, -2, 14,
-33, 10, 26, -24, -2, 2, 2, -1, -7, -3, -5, -1 },
{ -5, -14, 19, 1, 67, 56, -10, 5, 14, 3, -3, 4,
53, -40, 8, 15, 4, -36, 20, 4, 2, 1, 12, 2,
-1, -6, 2, 0, 22, -10, -9, -4, 9, -1, -4, 8,
-9, 3, 13, -11, 0, -3, 1, 0, 0, -1, -3, 1 },
{ 6, 13, -11, -23, -21, 12, -52, -47, 35, -11, -6, 6,
-32, -17, 41, 5, -1, -32, 4, 8, -4, -1, -2, -26,
20, -7, 5, 2, -21, 17, -6, -10, 3, -9, -5, 25,
-32, 14, -15, 6, 4, -3, 7, 3, -16, -2, 2, -2 },
{ -2, -1, 8, -18, 11, 11, 41, 3, 31, -19, -7, 21,
0, -17, -8, 17, -37, 47, -10, 9, -7, 8, 2, 22,
-36, -20, 27, 2, -22, 14, -1, 8, -3, 6, 0, -2,
-40, 32, -20, 20, -7, 2, 0, 0, -22, -2, 3, -5 },
{ 0, 15, -9, 31, -18, 7, -28, -43, -5, 18, 9, -3,
2, -22, -69, 18, -20, -29, -5, -1, 7, -2, -2, 10,
-47, -1, 1, 3, -4, 16, -30, -2, -1, 3, -6, -22,
14, -7, -11, 15, -9, -10, -5, -3, 3, 3, 2, 0 },
{ -5, -9, -30, 72, 43, 17, -13, -7, -19, 29, 22, 13,
-29, 4, 28, 4, 5, 21, -39, 7, 11, 7, -21, 15,
2, 11, -5, 3, -7, 12, 5, 6, -15, 1, 4, 16,
-19, 9, 0, -2, 4, 3, 4, 4, -8, -4, 2, -2 },
{ 2, -3, -19, 50, -19, 6, 20, 6, 48, 16, 19, 5,
22, -21, 5, -17, -21, 9, 26, 12, 7, 4, 13, -25,
35, -21, 19, 10, -31, -10, 22, -5, 5, -6, 1, -8,
27, -19, -27, 9, 3, 4, 0, -8, 22, 3, 2, 3 },
{ -1, 13, 14, -20, 12, 4, -1, -42, -50, 19, 0, -54,
40, 4, 26, 10, -21, 19, 0, -23, 6, -22, 0, 13,
17, -10, 4, 14, -32, 3, 10, 14, -5, 9, 4, 6,
4, -3, -34, 14, 2, 4, 3, -4, 7, 3, 3, 2 },
{ 2, 4, -1, -10, 8, 16, -22, -20, -3, 2, -3, -6,
-15, -17, -9, -70, 18, 5, -10, -7, -1, -4, -9, 9,
-26, -27, 23, -6, -13, -45, 36, -3, -5, 2, 13, -35,
-7, 17, 6, -33, 6, 8, -6, -18, 4, -5, -6, -3 },
{ -1, -3, 7, -25, 23, 12, 23, 31, -30, 19, 11, -14,
-51, -33, -21, 18, -16, -16, -23, -12, 6, -5, -8, -58,
11, 4, 22, 1, -4, -9, 7, -25, -10, -17, -17, -2,
7, 8, -20, 7, 3, -6, -2, -4, 7, 7, 2, 2 },
{ 1, -1, 1, -10, -5, 3, -24, -6, 3, -11, -13, 18,
7, -10, 19, 48, 36, 19, -37, 8, -6, 4, -9, 8,
14, -26, 39, -6, 21, 17, 12, -11, -13, -4, 24, -38,
32, -7, 10, 23, -14, 8, -5, -15, 31, 10, 1, 3 },
{ 1, 7, 7, -19, -7, 16, 5, -14, -2, -19, -12, 19,

```

```

8, -43, -28, -14, -11, 28, -46, 11, -7, 8, -11, 4,
25, 0, -31, 17, -20, 11, -4, 3, -8, -1, -6, 35,
8, -45, 25, -30, 13, 1, 4, 7, 21, -18, 3, 4 },
{ -1, 0, 12, 39, 10, 8, -2, -11, -45, -67, -29, 23,
-15, -9, -6, 2, 0, -2, 15, -2, -26, 1, 25, -6,
2, -1, -23, -3, -2, -14, 30, -17, 18, -6, 13, -5,
-1, -14, -33, 22, -4, 6, 3, 9, -12, 8, 0, 1 },
{ 0, -2, 2, 28, 13, -2, 8, 5, -7, -47, -25, -18,
-9, 19, 23, -24, -25, -18, -8, -3, -19, -6, -17, -7,
0, -18, 42, 12, -8, 7, -59, 18, -3, -3, -28, -2,
17, 4, 11, -16, -16, -8, -12, -8, 24, -7, -3, 0 },
{ 0, 1, 1, -27, -6, 1, 4, -3, -50, 31, 5, 70,
3, 13, 22, -13, -9, -19, 11, 43, 20, 31, 8, 4,
-8, -24, 4, 9, -12, -16, -20, 8, 6, 3, -9, 1,
5, -8, -14, 5, -15, 0, -5, 1, 13, 1, 1, 0 },
{ -1, 0, 0, 9, -7, -9, -8, -8, -18, -2, -8, 24,
19, 15, -20, 34, 10, 3, 14, 5, -15, -2, 18, -17,
-1, 42, 34, 35, -30, 4, 22, -2, 5, -1, -5, 8,
-4, 44, 14, -42, 30, -4, 5, -25, 20, -14, -5, -8 },
{ -1, -2, 4, 0, -6, -9, 5, 32, 19, -7, -14, -8,
9, 11, 2, 28, 2, -54, -22, 12, -4, 4, -49, 32,
-7, 2, -21, 17, -31, -27, 9, -4, -18, -11, 22, -4,
-11, -11, -44, -17, 2, 14, 3, -13, 6, -10, -3, -6 },
{ 0, 2, -1, 6, -11, -4, -5, -4, 2, 6, -19, 10,
62, 25, -4, -31, -12, -5, -56, 12, -10, -4, -12, -29,
-4, 16, 16, -16, 17, -8, 9, -37, -5, -13, -11, 15,
5, 20, -15, 17, 3, -8, -9, 27, -25, 4, 10, 3 },
{ 0, -1, -1, -2, -2, -3, -4, -1, -1, 9, -23, 4,
-18, -19, -21, -11, -1, -13, 17, 6, -4, 0, -15, 42,
21, 29, 58, -21, 27, -8, 15, 40, 2, 5, 3, 43,
15, -1, -22, 12, 15, 19, 0, 27, 8, 4, 15, 2 },
{ 0, 2, 4, -6, 2, -1, -2, -6, -12, -45, 57, -13,
7, 9, 3, -7, -18, -3, 23, 24, 15, 21, -34, 11,
-19, -9, 6, -19, 16, 21, 22, -30, -18, -10, -3, 20,
2, 1, 4, 6, 39, -21, 15, -11, 29, 12, 5, -1 },
{ 0, -2, -4, 4, 0, 0, -1, 1, 14, 16, -24, 36,
1, 10, 10, 8, 10, 6, 1, -66, -11, -8, -10, -2,
-24, -22, -16, -24, 7, -4, 10, -1, -3, 22, -46, 22,
-4, -11, -23, 0, 23, -22, 3, -19, 38, 12, 11, -5 },
{ 0, -1, -2, 6, 4, -1, 0, 5, 3, 16, -34, -49,
-12, 8, 5, 17, 12, 2, -16, 66, -13, 18, 35, -14,
-30, -20, -5, -27, -8, -2, 13, 14, 16, 15, -22, 3,
-1, -15, -1, -6, 30, -6, 6, 1, 10, 0, 18, 0 },
{ 0, 1, 1, 2, -4, 4, -6, -11, -3, -22, 8, -3,
19, -10, -14, 7, 25, 25, 12, 14, 9, 34, -32, -39,
21, 21, -16, -34, 4, -17, -19, 42, -30, 21, -28, -18,
-8, 16, -30, -9, -13, 17, -6, -19, -10, 8, -8, -6 },
{ -1, -2, 2, -2, 3, -1, -11, 0, 4, -19, 30, 3,
1, 10, 2, 22, 36, 17, 11, -3, 15, -19, -5, -14,
-29, -16, 31, 30, -2, -23, 2, 4, -12, -1, -22, 9,
8, -44, -15, -13, 7, -8, -20, 45, -34, -37, 13, 8 },
{ 0, 0, -2, 6, 2, 2, 2, -5, -6, 26, -44, -5,
-3, -8, -4, 2, 10, 27, 53, 25, -17, -19, -62, -14,
-16, -16, -21, 17, -2, 5, 8, -16, -8, -32, -4, 12,
7, 17, 22, 15, -15, 7, -11, 16, 9, 1, -5, 7 },
},
{
{ 120, 1, 2, -4, 27, -30, 7, -5, -2, 2, 0, 1,
10, -11, 1, 0, 3, -1, -1, 0, 1, 0, 1, 2,
-3, 1, -1, 0, 0, 1, 0, -1, 0, 0, 0, -1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 6, -106, 42, -4, -13, 15, 12, -46, 16, 4, -1, -4,
-7, 11, -5, 1, 5, -16, 1, 2, 2, -2, -3, -1,
4, 0, -1, 0, -1, 2, -5, -1, 0, 0, 0, 1,
0, 0, -1, 0, -2, -1, 0, 0, 0, 0, 0, 0 },
{ -41, 11, 42, -9, 56, -73, 11, -32, 16, 3, -2, 2,
24, -39, 7, -3, 10, -16, 0, 4, 1, 2, 2, 6,
-14, 4, -1, -1, -1, 5, -4, -3, 0, 0, 1, -4,
1, -1, 0, 1, -1, -1, 0, -1, 0, 0, 0, 0 },
{ 13, 54, 96, -4, 1, 42, -18, -5, 29, 7, 1, 0,
-11, 25, -3, 2, -8, -4, 11, 3, 2, 1, -2, -3,

```

```

11, -1, 1, 1, 0, -3, -2, 5, 2, 0, -1, 4,
0, 1, 0, -1, -1, 1, 0, 1, 0, 0, 0, 0 },
{ 3, 40, -25, 5, -29, -14, 56, -58, 9, -3, 2, 4,
-33, 28, -6, 0, 40, -40, 5, 1, 0, 1, -5, -13,
23, -9, 2, 1, -2, 16, -13, -2, -2, -3, -1, 8,
-5, -1, -2, 5, -3, -1, 0, 2, -1, 1, 0, 0 },
{ -5, -1, -31, -14, 79, 66, 15, -8, 8, 2, -5, -5,
39, 14, -27, 4, 18, -21, 12, 4, 1, -2, 1, 17,
6, -13, 0, 1, 2, 11, -11, 2, 4, 2, 7, 2,
-3, 1, 1, 4, -3, -1, 1, 0, 0, 1, 0, -1 },
{ -5, 0, 27, -63, -17, -15, 61, 32, -38, 11, -5, -21,
28, 35, -14, -9, 12, 27, -15, -4, 0, -4, -5, 22,
6, -5, 2, 0, -2, 1, 12, 0, -2, 4, 9, -2,
-1, 3, 0, 0, 4, 1, 2, -2, 1, 0, 1, 0 },
{ 8, 14, 4, -20, -46, 37, -1, -12, -11, 1, -2, -15,
30, -52, 27, -7, 26, -27, 7, 0, 0, -3, -1, 23,
-52, 26, 4, 1, -10, 23, -18, -3, 4, 3, 6, -22,
12, 1, -4, 11, -5, -4, 1, -7, 5, 3, 0, -2 },
{ 0, -7, 21, 80, 4, 12, 65, 41, 15, -4, 11, 24,
-1, -15, -13, -13, 21, 18, 2, -2, 2, 4, 11, 8,
-20, 4, 5, -1, -6, 11, 5, 2, -1, 7, -1, -10,
3, 3, -1, 5, 4, 1, -1, -3, 2, 2, 2, 0 },
{ 2, 8, -20, -16, -24, -6, 33, -14, 59, -8, 2, 0,
15, -3, -32, 11, -62, 4, 29, 2, -5, 2, 0, 18,
-27, -6, 2, 2, 11, -42, 2, 14, 4, 4, 6, -20,
6, -1, 3, -12, -4, 4, -2, -7, 3, -3, -2, -2 },
{ -1, 14, 1, 8, 21, 30, 19, -59, -37, 0, 0, 6,
-22, -11, 8, 14, -16, 34, -47, 1, 3, -1, -7, 2,
-21, 13, 1, 4, 7, -28, 42, -24, -7, 0, 5, -17,
11, -3, 1, -15, 16, -5, 1, -8, 4, -7, 5, -2 },
{ 1, 6, 14, 29, -20, -10, -24, -12, -47, 10, 5, -1,
12, -13, -81, 21, -11, -17, -19, 1, 5, -1, -3, 16,
-8, -42, 6, -6, 8, 7, -22, -6, 0, -1, 11, -7,
-18, 4, 3, 6, -11, -7, 3, -6, -6, 3, -4, -3 },
{ 0, 4, 0, 42, -3, -14, -21, -33, -12, -48, 5, -33,
33, 44, 24, -14, 8, 31, 23, -16, -5, -5, -17, 42,
0, -4, -9, -1, -6, 4, 1, 21, -1, 2, 25, -10,
2, -8, 0, 1, -6, 10, 7, -4, 1, 1, -6, -1 },
{ 1, -4, 2, -34, 17, -3, -14, 8, 4, -68, -2, 2,
-53, -2, -48, -23, 17, 10, 8, -17, -14, 3, -2, -13,
-23, 8, 8, -6, -18, 20, 3, 2, -5, -1, 5, -31,
18, 3, -9, 12, 2, -1, -2, -13, 4, 4, 3, -3 },
{ -3, -2, -5, 6, 15, -17, -9, 10, -10, 40, -3, 23,
-21, 60, 2, 14, -7, -16, -7, 15, 7, 3, -2, 15,
-36, 43, -16, -3, 19, 8, -30, 4, 2, 4, 10, -33,
37, -5, 5, 11, -20, -3, -1, -9, 12, 6, -5, -1 },
{ 2, 2, -15, -6, -13, -6, -30, -11, 45, 43, 2, 28,
11, 12, -11, -46, 23, 21, -23, 26, 4, 5, -1, 20,
-1, -27, 2, -5, -33, 14, 30, -20, 4, 1, 25, -17,
-5, 1, -11, 2, 17, -5, 9, -15, 1, 1, 5, -5 },
{ 0, 0, 5, -6, 0, 3, -1, -16, -46, -3, -5, 55,
16, 19, 13, -52, -10, -13, 34, -4, -2, 1, 41, -22,
-23, -27, 3, 1, -10, -23, -4, 15, 11, 16, -23, -7,
-8, 1, 3, -16, 0, 0, -13, 0, 2, -7, 1, -1 },
{ 0, -2, -1, 12, 18, 0, 3, 15, -13, 37, 1, -57,
-47, -11, 4, -48, 4, -30, 17, -3, 4, -6, -28, 14,
-7, -11, 17, -1, -4, -36, -1, 16, 2, -9, 15, -12,
-8, 11, 4, -33, 4, 9, -1, -8, 0, -12, -2, 0 },
{ 0, -3, 5, 2, -13, -7, -17, 15, 1, -15, -2, 15,
16, 3, -17, 25, 50, -31, 20, -3, -5, -2, 15, 4,
13, 19, -14, -5, 32, -27, 36, -12, 6, 0, 20, -10,
13, -8, 31, -45, 34, -7, 10, -12, 11, -21, 9, -5 },
{ 0, 5, -3, 18, -6, -3, -3, -10, -3, 13, 15, -42,
36, 8, -39, -33, -16, -4, 0, 0, 5, -1, -4, -43,
9, 32, -18, 7, -22, 11, 5, -13, -1, -1, -35, 8,
48, -19, -6, -9, 18, -11, -14, 5, 22, -9, 5, 3 },
{ 1, -2, 5, 2, 1, 1, 8, 23, 6, -48, 36, 4,
18, 19, 25, -26, -25, -50, -47, 22, -5, 15, -24, -10,
-9, -19, 0, 7, 10, -8, -4, -38, 3, -26, 10, -8,
-6, 2, 9, -1, -8, -11, 6, 0, 1, -3, -4, 0 },
{ -1, -3, -2, -11, 3, -1, -3, -7, -16, 13, 90, 8,

```



```

3, -2, 2, 18, 7, 16, 38, 25, 0, 24, -21, -15,
-1, -2, 40, -4, 15, 11, 14, 21, -2, -8, -1, -1,
7, 16, -1, 11, 12, 5, 2, -9, 11, 9, 5, -2 },
{ 0, 0, 1, -12, 1, 6, 4, 2, -14, -2, 31, 30,
-16, -15, -14, -3, 0, -4, 14, 4, 6, 22, -35, 31,
-5, 15, -76, 5, -35, -11, -1, 10, -2, -9, 4, 30,
-12, -28, -8, -8, 2, 5, 2, 23, -17, -10, -3, 6 },
{ 1, 2, -1, -7, -6, 6, -3, -10, 6, -12, -5, -1,
-1, -19, -12, -32, 28, 40, -15, 55, 8, 13, -3, -3,
12, -1, -2, -14, 41, -12, -32, 10, 4, 3, -7, 0,
17, -12, 43, -14, -32, 20, -17, 23, -4, -7, -13, 14 },
{ 0, -1, 0, 7, 0, -7, -1, 0, -8, -13, -31, -11,
-9, 16, -8, -1, -1, 7, 44, 48, 20, -6, -15, 12,
-35, 13, 15, -8, 9, -2, 16, -47, 27, -11, -14, 33,
-19, 14, -6, 17, 16, -32, -1, 25, -10, 6, 8, 6 },
{ 0, 0, 2, 4, -3, 1, 4, -4, -20, -10, -16, 25,
-5, -19, -8, -27, -40, -28, 1, 24, -9, -1, 10, 38,
51, 38, 36, 5, 0, 18, 18, 18, 9, 3, 33, 18,
23, -6, -5, 13, 4, 15, 6, 6, 14, 6, 7, 6 },
{ 0, 1, -5, -2, -3, -1, -12, -6, 16, 7, 3, 24,
12, 14, -13, -23, 5, -6, -24, -55, -7, 2, -32, 20,
-25, 26, 36, -8, 29, 9, 3, 12, -30, 0, -23, 31,
-10, 20, 16, -10, 13, 2, -15, 32, -17, -10, 8, 14 },
{ 0, -1, 2, -1, -1, 5, 6, 2, -6, 14, -26, 25,
6, -16, 5, -17, -3, 4, 35, -34, 13, -9, -52, -20,
10, -29, -13, -8, 36, 0, 20, -22, -18, -25, 16, -4,
29, -22, 6, 30, -19, -1, 7, -7, 20, 22, -14, -3 },
{ 0, 1, -1, -5, 2, 1, -2, -2, 6, 6, 42, -28,
-21, -3, 3, -23, -5, -10, -3, -23, -1, -4, 53, 30,
0, -12, -25, -20, 44, 1, 22, -17, 9, 30, -6, 4,
1, -27, 2, 34, 4, -17, -7, 15, -7, 27, 2, 10 },
{ 0, -1, 1, 5, -4, -3, -11, 6, 1, -6, -6, -1,
12, 6, -8, 14, 35, -19, -15, 3, 8, 22, 0, 12,
4, 4, 19, 1, -17, -63, 29, 6, -2, -8, -24, -7,
11, -3, -45, 30, -28, 24, -35, 14, 13, 23, -10, 11 },
{ 0, -1, -2, -5, 0, -1, 0, 5, 2, -18, -2, -1,
-7, 1, 10, 8, -10, -10, 0, 1, 101, 18, 7, 19,
5, -16, 12, -16, -14, 12, -1, 4, -44, 18, -12, -1,
8, -11, 13, -9, 9, -6, -8, -9, 17, -14, 11, -6 },
{ 0, -1, 1, -3, 1, -2, -2, 0, 1, 0, 9, 14,
-13, -8, 8, 4, 6, 17, 8, -18, -23, 7, -7, 52,
29, -18, 13, 31, -12, -16, -43, -34, 14, -3, -37, 1,
28, 9, 10, -6, 4, -42, -13, -12, 35, -8, 6, -3 },
},
},
{
// 13
{
{ 114, -37, -35, -3, -15, -2, 0, 15, 14, 0, 1, -4,
-3, -5, 0, 0, 1, -4, -3, 1, -1, 0, -1, 2,
3, 3, -1, 0, 0, 0, -1, 0, 0, 0, 0, 0,
-1, 0, 0, -1, -1, 0, 0, 0, 0, 0, 0, 0 },
{ 49, 82, 68, -15, -25, -17, 2, -16, -14, 1, 0, 14,
23, 15, 0, 2, -4, -7, -7, -2, 2, 0, 1, -2,
-6, -3, 1, 0, 1, -1, 1, 6, 5, 0, -1, 1,
1, 0, -1, -1, -1, 0, 1, 0, -2, -2, 0, 1 },
{ -31, -11, -31, -37, -94, -36, -2, 15, 16, -1, -3, 9,
33, 9, -4, 0, -7, -22, -21, -6, 0, -2, 0, 4,
0, 4, 0, -2, -1, -2, 0, 8, 8, 0, -2, -1,
0, 1, -4, -5, -4, 0, -1, 1, 0, -1, 0, 2 },
{ 1, 78, -90, 7, 14, -7, -13, -25, 15, 15, -1, -12,
-3, 9, 4, -2, 4, 12, -1, -3, -1, -2, 2, 7,
-1, -10, -4, 2, 0, 1, 3, -4, -1, -2, 0, 0,
0, 0, -2, 2, 4, 0, -1, -3, 0, 1, 2, 0 },
{ -4, 11, 18, 24, 33, -66, 12, 50, 55, 12, 2, -17,
-21, 19, 5, -1, -9, -29, -21, 4, 3, 1, 2, 18,
28, 10, 5, -1, 0, 3, 1, 2, -3, -7, 2, 1,
1, 2, -4, -13, -6, -2, 2, 2, 5, 7, 3, 1 },
{ -2, 27, -2, -71, 18, 64, 10, 49, 27, 3, 2, 20,
-12, -27, 3, 5, 8, -13, -18, -8, -1, -3, 2, 2,

```

```

19, 18, 0, 3, 2, 2, -10, -5, 1, 5, 2, -1,
2, -1, -3, -8, -3, 3, 0, 4, 6, 3, -1, 0 },
{ 2, -23, -8, -83, 53, -60, 4, -29, -20, -1, 8, 21,
0, 16, 1, -4, -5, 5, 4, -6, -3, -2, -2, -10,
-5, -10, -3, -1, -1, 2, 3, 3, 3, 6, 2, -2,
1, -2, 2, 1, 4, -1, -1, -3, -3, -2, -2, 1 },
{ 0, 20, -19, -6, -14, -17, 41, 39, -77, -21, -9, -24,
-33, -11, 11, 2, -22, -18, 36, 13, -3, -2, -6, 4,
16, 12, 5, 2, 2, -1, 0, -5, -21, -6, -2, -2,
-3, 4, 8, 0, -6, -6, 0, 5, 6, 7, -1, -4 },
{ 0, -2, 20, -20, -35, -14, -3, -52, 19, 21, 3, -27,
-64, -39, -4, -3, 1, 26, -7, -13, 2, 0, 1, 16,
31, 11, -6, -2, -1, -3, -11, -31, -21, -7, -1, 1,
-1, -3, -1, 2, 5, 2, -4, 0, 13, 11, 3, -9 },
{ 0, -8, -3, 11, -5, 11, 70, -3, -2, 89, 11, 25,
14, 14, 3, 5, -23, 12, 10, -25, 0, 5, 9, -5,
2, 0, 11, 5, 3, 0, 7, -5, -5, 8, 2, 3,
-1, -6, 2, 4, 2, -6, 0, 0, 0, -1, -2, -2 },
{ -1, 4, -9, 12, -1, 2, 67, -43, 45, -61, -5, 33,
-4, -11, 9, 2, -28, -6, 2, 25, 4, -1, -2, -22,
-1, 19, -1, 3, 2, -1, 1, 2, 3, -1, 2, -1,
1, 1, 11, -1, -9, 0, -5, 6, 4, -3, -6, 0 },
{ 1, -10, -2, -4, -2, 34, 17, -31, -19, -9, 5, -31,
-38, 52, 10, 10, -13, -40, -49, -22, -1, -1, -9, 10,
10, -28, -2, 8, 1, 1, 8, 26, 23, 0, -5, 0,
-4, -5, -13, -22, -8, -5, -1, -5, -3, 0, 6, 12 },
{ -1, -2, 10, -29, -5, 16, 35, 10, 31, -24, -2, -60,
23, 48, -14, 6, 11, 38, 27, 10, -7, -3, 0, 33,
-12, -18, 14, 0, 1, 5, 2, -17, -14, -5, -1, -1,
0, 4, 3, 23, 16, 2, 2, -7, -2, 1, 5, -10 },
{ 2, -1, 0, -2, 23, -7, 21, -17, -6, 6, -16, -62,
49, -64, -6, -4, -10, -27, -30, -20, -6, -5, 4, 10,
-31, 19, 6, -3, 2, -8, -5, 2, 9, 3, -6, -2,
-3, -5, -9, 3, -12, -6, 0, 3, -2, -5, 0, 0 },
{ 0, -9, 6, -8, 2, 10, -13, -21, 11, 10, -6, -5,
31, -7, 78, 6, -3, -21, 41, 14, -1, 4, 13, 36,
24, 0, -36, 7, 0, 6, 22, 22, -20, -6, 1, 1,
2, 3, -9, -17, 0, 4, -2, -5, 1, 20, 8, 5 },
{ -1, 2, 6, -10, -7, -3, 4, 8, 22, 21, -46, 2,
-39, -15, 10, 7, 4, -35, 10, 17, -4, 1, 18, -21,
-53, -48, -8, 3, 3, -3, -7, 16, -17, -22, -5, 2,
-1, -3, 8, 28, 17, -4, -2, -14, -30, -13, -2, -1 },
{ 2, -1, -1, -8, 7, 17, -20, -21, 9, 13, -16, 7,
5, 6, -79, -6, -40, -34, 38, 20, -3, -1, 5, 23,
17, 6, 15, -14, -7, -17, 3, 24, -19, -15, 4, -2,
2, 2, -13, -14, -10, -9, -2, 9, -2, 11, 8, 4 },
{ -2, 4, 0, -4, -5, 3, 0, -4, 7, 9, 105, -16,
-1, -16, -1, 2, 1, -34, 11, 31, 14, 4, -27, -10,
-18, -14, 2, 0, -1, -9, -6, 5, -8, -9, 13, 2,
2, 5, 5, 12, 11, -5, 0, -4, -5, 0, -6, -6 },
{ 0, 3, 4, 3, 0, 5, -24, 15, 19, -24, 10, -2,
-2, 3, 9, -14, -62, -4, 20, -72, -4, -4, -13, -18,
-21, -18, -13, -6, -1, -7, 18, -22, -32, 23, 3, -3,
-5, -6, -1, -2, -2, -10, -4, -12, 6, 10, -14, -11 },
{ 0, 1, -2, 11, -2, -9, 15, -15, 7, -14, 1, 12,
-5, 0, -7, -1, 64, -45, 45, -51, -9, -8, 2, 13,
7, -1, 18, 5, 2, 4, -36, 9, 2, 40, 4, -7,
-2, 5, -12, 3, 16, 0, 8, 12, 4, 1, -10, -1 },
{ -1, 6, 4, -5, -6, -7, 7, 11, 13, 3, -1, -28,
-10, -23, -17, 9, -3, 28, 33, -15, 21, 1, -18, -19,
27, -3, -24, -3, -1, -9, 28, 50, 41, 22, -6, -1,
-4, 8, 19, -14, 10, 16, -4, -13, -31, -28, -11, 29 },
{ 0, 0, 2, -9, -2, 10, -8, -11, 9, 19, -9, -30,
-1, 34, 6, -36, 26, -16, 10, -3, -18, 0, -17, -61,
-4, 56, -6, -5, -9, 17, 7, 12, -19, -15, -3, -4,
-4, 7, 34, 6, -30, -2, 1, 3, 3, 9, -2, 0 },
{ 0, -2, -2, -3, 4, -4, -17, -6, 5, 0, -4, -10,
7, 8, 6, 103, 1, 1, 5, -11, 17, 6, 1, -31,
-1, 16, 30, 11, 14, -23, -9, -3, -21, -9, -9, 2,
-2, 0, 12, -14, -8, 3, 16, 7, 2, 13, 0, 4 },
{ 0, 1, 0, -5, -5, 0, -16, 0, 2, 0, 11, 5,

```

```

-35, 8, 17, 8, -16, 9, 4, -1, 2, 4, 24, 42,
-45, 56, 30, -6, 3, 0, 25, 37, 20, 30, 12, 1,
4, -5, 2, 40, -8, -9, 7, 1, 3, -2, -6, -3 },
{ 0, 3, -1, 9, 0, -6, 18, 7, -8, -10, 17, 12,
-3, -1, -27, 26, 19, 12, -19, -11, -60, -5, 3, 5,
-12, 21, -53, 0, -2, -15, 15, 24, -33, -9, 21, -6,
-4, -17, -23, 3, 7, 31, -3, -20, -19, 23, 7, 2 },
{ 0, -2, -3, 1, 5, 2, 9, 1, -8, 2, -6, 3,
-1, 20, -21, 5, 8, -18, -1, -9, 76, 8, 7, 16,
-26, 25, -67, -7, -6, -11, -11, -16, 0, -3, -21, 8,
13, 12, 7, 11, 7, 23, -10, -2, 24, 8, -5, -3 },
{ 0, -1, -2, 4, 5, 3, -7, -2, -6, -11, 7, -2,
16, 0, -1, -2, -19, -13, -16, -20, -7, 7, 37, -10,
51, -5, -1, -3, -2, -10, -7, 20, 11, -12, -5, 0,
2, -4, 48, 61, 39, 9, 0, 10, -5, 13, 22, -33 },
{ 0, 1, -3, -1, -2, -2, 12, 4, -4, -19, 17, 1,
1, -10, -5, -27, 26, 11, -5, -21, 50, 16, 50, -16,
1, -3, 35, -8, -3, 13, 32, 21, -34, -28, -12, 11,
-3, -29, -22, -18, 2, -3, 2, -14, -14, 17, 12, 14 },
{ 0, -1, 0, 4, 5, -1, 3, 0, 0, -1, -13, 24,
2, -10, 7, -5, 4, 11, -11, -25, 15, -10, -83, 27,
-5, 7, 23, 13, -5, 3, 8, 31, -19, -42, -29, -7,
-6, 24, -4, 20, 17, 0, 0, 1, -5, 5, 17, -10 },
{ 0, 0, 0, 1, 0, 1, 2, -6, -4, 5, -19, 1,
5, -11, -17, 6, 18, -21, -25, 23, 7, -2, -14, -4,
10, -15, 17, 7, -1, 11, 67, -13, -23, 51, 15, -5,
7, 21, 22, 10, 25, 6, 3, -26, 27, 30, -20, 7 },
{ -1, 1, 0, -2, 0, -2, -1, 0, 3, -6, 1, 5,
-2, -8, -14, 10, 6, -18, 19, -18, -21, 81, -5, 9,
-7, 5, -2, 28, -4, 21, 29, -32, 36, -34, 11, 4,
-23, -2, 7, 2, 2, -4, -6, -15, 19, -11, 21, 11 },
{ 0, 0, 0, -2, 0, -1, -8, 3, 0, 0, -3, -10,
3, 6, -4, -19, -16, 14, -12, 11, 5, 35, 3, -6,
-3, 11, -1, 86, 7, -5, -32, 24, -30, 28, 5, 5,
-3, 0, -7, -5, 27, -3, -32, 23, -5, 5, -13, 17 },
},
{
{ 124, 1, 12, 2, -25, -11, -4, -5, -6, 2, 0, -3,
7, 3, -2, 0, 1, 3, 2, -2, 0, 0, -1, 1,
-1, 0, 1, 0, 0, 0, 0, -1, -1, 1, -1, 0,
0, 1, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ -8, -79, -63, -17, -60, -23, 2, 19, 11, -5, -1, 5,
27, 15, 2, 0, 0, -4, 0, 2, -1, -1, 0, -1,
-10, -9, -2, -1, -1, 0, -1, 0, -1, 0, 0, -1,
0, 1, 0, 3, 3, 1, 0, 0, 0, 1, 0, 0 },
{ 24, -18, -14, 6, 63, 34, 15, 79, 44, -3, -1, 7,
4, 3, 3, 1, -8, -33, -11, 7, -1, 0, 3, -1,
-7, -6, -1, 2, 1, 2, 4, 12, 1, -1, 3, 0,
2, 0, -1, 3, 4, 2, 1, -1, -4, 0, -1, -1 },
{ 6, 66, -64, -46, -27, 58, 20, 6, -20, -12, -5, 14,
9, -11, -2, -3, -10, -2, 10, 3, -3, -2, 3, -2,
-5, 3, -2, 0, 0, 0, 3, -2, -5, 0, 0, -1,
1, -1, 0, 1, -2, 0, -1, -2, 1, 2, -1, 0 },
{ -7, -30, 52, -18, -19, 33, 35, 38, -73, -10, -1, -21,
-8, 21, 9, 2, -16, -13, 25, -2, -1, -1, -4, 15,
6, -17, -7, 2, 1, -1, 2, 1, -6, 4, -3, -1,
-2, 3, -5, -5, 7, 2, 1, 1, 0, 0, -2, 2 },
{ 13, -37, -35, -23, 25, 0, -15, -21, -30, -15, -2, -19,
-75, -46, -2, 0, -2, -23, -12, 1, -2, -2, -2, 2,
29, 20, -1, 0, 0, -2, 3, 19, 10, -3, -1, -1,
-1, -2, 1, -7, -3, -2, -2, -8, -3, 2, 1 },
{ 2, 15, -6, -58, 45, -54, -39, 23, -24, -43, -18, -6,
19, 27, -10, 0, 21, 14, 19, 9, -10, -6, 3, 13,
-3, -6, 7, 2, 0, 2, -9, -11, -6, -1, 2, -3,
2, 0, -6, -2, 0, -1, 0, 5, 5, 1, -1, 1 },
{ -7, -13, 42, -24, -17, 5, -41, 9, 6, -6, -7, 28,
34, -69, -33, -9, -29, -22, 17, 9, -3, -2, 6, -7,
-19, 22, 7, -4, -2, 1, 20, 13, -15, -2, 2, -1,
1, -2, -3, 8, -2, 1, 0, -8, -6, 6, -2, -3 },
{ 7, -37, -3, -3, 28, 31, 14, -14, 6, 9, 4, 41,
11, -7, 17, 5, 22, 55, 55, 15, 1, 0, 3, 1,

```

```

31, 33, 7, 3, 1, 3, -2, -22, -20, 0, 3, 1,
1, 0, -2, -20, -18, 0, 1, 0, 5, 5, 0, 9 },
{ -2, -1, -26, 46, 1, 23, -37, 26, -16, 2, 6, -57,
19, -8, -48, -23, -14, 38, 7, -17, 1, 1, -8, 24,
14, 12, 9, -6, -1, 9, 10, -17, 8, 13, -4, 0,
-2, 5, -4, -9, -6, 1, 2, -2, 5, -8, -2, 2 },
{ 1, -10, 12, -2, 2, 15, 30, -31, 37, -65, -47, -48,
25, -25, 29, 11, -6, 7, -4, -21, -21, -10, 1, 14,
-5, 2, -9, 0, 0, -3, -2, -6, 1, 13, 2, -3,
1, 1, -3, -1, -3, 2, -2, 1, 3, -1, -5, 0 },
{ -3, -8, 23, -19, -22, 37, -19, 20, -2, -3, -2, -13,
-1, 8, 6, 7, 71, 9, -45, -5, 2, 0, -3, -15,
-18, 41, 35, 4, 4, 6, -13, 2, 18, -4, -4, 0,
-1, -4, 12, 14, -24, -16, 3, -1, -4, 1, 3, -7 },
{ 1, -7, -16, 37, 6, 23, -31, -17, -17, -4, 5, -4,
-7, -19, 12, 17, 32, -35, 23, 18, -3, 1, 8, 17,
-47, -37, 12, 10, 3, -6, -29, -28, -33, -2, 7, -2,
4, -1, -20, 12, 9, -15, -6, 9, 19, 11, -7, 0 },
{ -6, 26, -4, 33, -30, -18, -7, 23, 9, -9, -14, 9,
1, 1, 19, 19, 30, -32, 41, 8, -10, -5, 4, 43,
42, 22, 18, 8, 4, 1, 7, 46, 14, 11, 6, -3,
2, 8, -12, -8, -1, -5, 0, -3, -19, -5, -3, 0 },
{ 0, -8, 11, 9, -14, 21, 7, -19, 21, -36, -19, 21,
-35, 41, -49, -52, 5, -2, -4, 46, -13, -6, 24, 20,
-4, 0, -2, -21, -6, 10, -2, 3, 2, -4, 15, -4,
7, -5, -14, 4, -1, 2, 7, 2, 1, -2, 1, -2 },
{ -1, -3, 4, -6, -5, 11, -17, 5, 11, 24, -50, 29,
-21, 9, -21, -16, 14, -17, 17, -72, -32, -17, -27, -18,
12, -20, -5, -7, -2, -4, -20, -14, -4, 19, -1, -8,
-1, 14, 7, -11, 0, -3, -2, 11, 5, -4, -3, 7 },
{ -3, 8, 18, -25, -24, 4, -14, 19, 31, 2, 22, -5,
-5, -21, -1, 6, -9, -3, -16, 23, 11, 5, 4, 21,
32, -26, -22, 2, 1, -17, -53, -32, 12, -2, -3, 2,
-2, 1, -14, -45, -29, -12, -7, 16, -4, -23, -3, 14 },
{ 2, -12, 4, -7, 7, 39, -44, -19, -1, -8, 10, 10,
21, 6, 8, 7, 5, 21, 5, 7, -1, 2, -2, 0,
24, -34, -29, 5, -1, -13, -13, 38, 44, 2, 3, 0,
1, 2, 20, 32, 43, 14, -8, 21, -4, -11, 9, -19 },
{ -2, 9, -4, 37, -10, -4, -21, 25, -18, -29, -10, 1,
-2, 0, 25, -20, 8, 8, -1, 6, -6, 0, -7, -42,
-2, 21, -65, -13, -7, -25, -16, 12, -30, -33, 0, -4,
-2, -8, 4, -10, -18, 10, 0, 3, -21, 16, 12, 2 },
{ -1, 2, 2, 18, -13, 7, -9, 6, 7, -36, -8, 5,
-15, 11, -22, 43, -27, 15, -1, 7, -13, -1, -11, -47,
26, -26, 46, 41, 14, 18, 4, 0, -12, -40, 1, -2,
-2, -14, 5, -13, 7, -10, -6, 1, -2, 4, 11, 7 },
{ 0, -1, 1, 3, -8, 6, -32, 12, -9, 22, -33, 18,
-22, 12, 21, 33, -27, 28, -35, 12, -22, -19, 32, 22,
-19, 15, -26, 39, 12, 3, 22, -12, -1, 28, 16, -9,
11, 4, -10, -5, -3, 21, -3, -6, 1, -1, -19, 9 },
{ -1, 3, 1, -2, -8, -6, 7, 14, 4, 6, -26, -18,
-7, -8, -16, 4, 10, 4, 7, -16, 45, 26, 77, -24,
25, -2, -1, -2, 1, -10, -13, 5, -19, 21, 31, 16,
13, -8, 20, 9, 17, -5, -2, 14, 15, 23, -2, 3 },
{ 1, -4, -9, 33, 15, -13, 23, 2, -38, -20, -14, 40,
17, -11, -14, 10, -21, 1, -6, 1, -4, 3, 17, -4,
-12, -3, 7, -1, 0, -9, -34, -9, 42, 24, 4, -1,
-1, 4, 30, 14, -42, -35, -6, -5, -17, -20, 0, -20 },
{ 1, -7, -2, 1, 10, 19, -28, -25, -9, -20, 8, 11,
21, 24, 18, -11, 6, -25, -8, -26, 31, 20, 24, -17,
0, -22, 6, -2, -5, 19, 28, 3, -6, 15, 5, 9,
0, -5, -14, -47, -16, 10, 7, -31, -38, -35, -11, 6 },
{ 1, -7, -5, -11, 17, 7, 15, -24, -6, 38, -11, -24,
28, 15, -33, 8, -3, 0, -8, 1, -12, -15, 5, -2,
2, 10, -12, 22, 19, -11, -35, 45, -31, -13, 3, -5,
7, -13, -38, -3, -4, -30, -1, -2, -37, -7, -18, -16 },
{ 0, -1, 3, -8, -3, 9, -10, -6, 20, -27, 29, 4,
-21, 33, -22, 25, -13, 1, 15, -21, 14, 23, -17, 11,
-22, 27, -11, 8, 1, -63, 14, 6, -9, 21, -1, 10,
-4, 17, 4, -1, 1, -30, -41, -19, 1, 11, -1, 3 },
{ 0, 2, 2, -7, -3, -1, -1, -6, 15, -11, 29, -12,

```

```

-9, 5, -9, -11, -2, -10, 24, -3, 5, 7, -8, -1,
-12, 4, -28, 50, 34, 54, -27, 6, -10, 30, 3, -3,
-7, 25, 34, 11, -24, 15, 27, -4, -15, 23, 10, -3 },
{ 0, 2, 2, -14, -3, 5, -13, 1, 18, 34, -22, -35,
-15, 5, 13, 4, -3, 7, 31, 27, -9, -14, 8, -3,
-5, -31, 4, -19, -17, -9, 17, -15, 2, -10, 10, -4,
7, 6, 36, 8, -21, -19, -7, -50, -40, 9, 27, -24 },
{ 0, 0, 1, -4, -4, 2, -15, 5, 13, 2, -26, 5,
-19, 12, 16, -4, -23, 1, 27, -27, 38, 22, -4, 19,
-4, 11, -11, 13, 3, 17, -10, -13, 20, -41, -21, 21,
-4, -45, -28, 42, -22, -5, 6, -6, 9, -10, -16, -26 },
{ -1, 5, 3, 0, -12, -14, 16, 27, 13, -15, 23, 2,
-38, -29, -2, 1, 17, 49, 19, -10, -1, -12, 5, -20,
-29, -18, 2, -8, -4, 8, -3, 11, -12, 24, -4, -7,
-5, -2, -22, 4, 25, 2, 1, -1, -30, -43, -32, -33 },
{ 0, 1, 2, -4, -3, 3, -21, 3, 3, -7, 33, 2,
-3, 11, 42, -17, -38, 11, -11, -17, -29, 0, 5, -6,
21, -1, 22, -27, -20, 16, -23, 22, -19, 11, 33, -2,
15, 18, -20, 24, -15, -38, 15, -14, 25, 16, -15, 0 },
{ 0, 0, 0, 3, 0, 2, -10, 0, 5, -6, 10, 5,
1, 6, 16, -25, -7, -2, -6, -2, 7, -47, 32, 5,
15, -7, 9, 7, 12, -9, 3, 5, -18, 23, -76, -30,
-30, -35, 20, 0, 4, -24, -9, -5, 9, 18, -3, -1 },
},
{
{ 122, 25, 5, -20, 18, -7, -1, 2, -10, 1, 1, -2, 1, 1, -1, 1,
0, -1, 0, 0, 1, 0, 0, 0, -1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ -14, 22, 113, 8, 47, -4, -2, -6, 11, -17, -1, -8,
6, 1, -1, -1, -1, 3, 1, -2, 0, 2, 0, 0,
0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0, 1, 0, 0, -1, 0, 0 },
{ 30, -102, 26, 27, -25, -21, 5, -20, 45, -2, -2, 2,
4, 0, 0, 1, -2, -2, 7, -4, 1, 1, -1, 0,
1, 0, 1, -2, -1, -1, 0, 0, 0, -2, 0, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
{ 16, 30, -8, 99, 1, 45, 2, 24, 36, 8, -17, 16,
-27, 7, 0, 0, 3, 0, 0, -7, 1, 2, -1, 4,
-1, -1, 1, 2, 1, 1, 1, 0, 0, 0, 0, 2,
2, -1, 0, 2, -1, -1, 1, 0, 0, -1, 0, 0 },
{ -7, -23, -44, 8, 97, -22, -7, 33, 30, -3, -4, -25,
31, -9, -1, -1, 1, 9, -6, -1, -1, 0, 1, -4,
3, 3, -1, -1, -1, 2, 1, 1, -3, 1, 2, -1,
0, 2, 0, 0, 1, 2, 0, -1, 0, 0, 0, 0 },
{ 4, -48, 14, -33, 15, 83, 17, 52, -24, -6, 7, -17,
-30, 23, -3, 2, 1, -3, -14, 7, 0, -1, 0, 2,
-7, 3, 3, 2, 1, 1, -1, -2, -2, 3, 0, 0,
0, 0, 1, 0, -2, 0, 2, 0, 0, 0, 0, 0 },
{ 1, -21, 15, 40, 1, -47, -24, 33, -72, 38, 1, 7,
-17, -1, -4, 0, -3, 26, -42, 24, -2, 3, -4, 5,
-10, 2, 0, -2, -2, 0, 3, -2, -6, 4, -1, 1,
-2, 0, 1, -1, 0, -1, 0, -1, -4, 1, 1, 1 },
{ -2, 12, 16, -27, -24, -6, 9, 58, 44, 55, 10, 46,
52, 29, 0, 0, 2, 11, -10, 1, -7, -1, -3, -3,
1, 7, 3, 0, 0, 0, 3, 3, -3, 2, -1, 1,
2, 2, 1, -2, 1, 3, 2, 2, -1, -2, 1, 0 },
{ 4, 9, 2, 7, -46, -16, -31, 51, 4, -75, -12, -42,
24, 13, -5, -2, -4, 29, -10, -25, 8, 0, 3, -10,
-2, 8, 2, -1, -1, -1, 4, 5, -6, -3, 0, -2,
-1, 1, -1, -2, -1, 3, 2, 1, -1, 0, 0, 0 },
{ -3, 9, -2, 8, -2, -44, 84, 11, -3, -10, -12, -25,
-13, 47, 23, 5, 26, -40, -24, -11, 1, 1, 3, 5,
-12, -9, 7, 2, 2, 6, -3, -9, 1, 0, 0, 2,
0, -1, 2, -1, -4, -3, 3, -3, 1, 1, 2, 1 },
{ 5, 4, 5, 24, -16, 31, 31, -20, -16, 30, 21, -57,
56, -25, 16, 4, 1, 15, -21, 9, 1, 1, 9, -39,
36, -12, 2, 3, 2, 1, -1, 8, -13, 7, -4, 1,
-4, 0, -2, -2, 4, -1, -1, 0, -2, -1, 0, 0 },
{ -3, 13, 3, -22, -12, -8, -24, -4, 47, 40, -1, -50,
-46, -19, -23, -1, -26, -21, -60, -21, -7, 1, 10, -3,
-12, 7, -7, -1, 0, -3, -5, -1, 0, 4, -1, 1,

```

```

-2, -1, 1, -4, -1, 0, -2, -4, -1, 2, 0, -1 },
{ 1, 8, -9, 9, 4, 0, -16, -29, 21, -10, 71, -12,
-9, 57, -11, -4, 22, 21, -7, 51, 8, -7, 19, -9,
-26, 25, -1, -4, -1, 0, 12, -5, -9, 4, -2, -2,
0, 1, 0, -8, 2, 6, 1, 0, -2, -2, 1, -2 },
{ -1, -6, -4, -3, 11, 17, -11, -44, -16, 25, -57, -8,
17, 55, -21, -1, 20, 34, -6, -48, -11, 4, -19, -16,
-20, 27, -1, -1, 0, -1, 8, -4, -8, -4, 1, 0,
-1, -5, 1, -6, 5, 4, -2, 1, -3, -3, -1, -1 },
{ 0, 3, -3, 7, 0, -3, 54, -3, -6, -19, -11, 11,
11, -5, -81, -16, -52, 5, -6, 24, -4, 1, 0, 10,
10, 35, -22, -5, -2, -6, 6, 11, -9, 7, -2, -1,
0, -2, 2, -3, 7, -1, -6, 5, -4, 1, 1, -1 },
{ -2, -2, 5, 12, -2, -12, -18, 22, -18, 31, 24, -31,
-1, 24, -48, -14, 8, -42, 59, -24, 7, 1, 8, -16,
31, 3, 4, -7, -4, 0, 4, -19, 31, -19, -3, 1,
-5, 6, -9, 11, -1, 0, 1, 0, 4, 4, -2, 2 },
{ 2, 0, 2, 6, -11, 16, -28, -4, -2, -1, -33, -21,
49, 0, -12, -14, 21, -52, -7, 47, -15, 9, -5, 46,
-30, -14, -5, -5, -1, -3, 6, -25, 6, 30, -12, 1,
1, -5, 7, 2, -7, 2, -1, -6, -6, 0, 0, 1 },
{ 0, 5, 2, -7, -8, -4, 11, 5, 9, 27, -19, -41,
-13, 2, 30, 4, -5, 44, 37, 19, 18, 14, 15, 71,
20, 35, -2, 7, 4, -2, 7, 8, 18, -5, 0, 4,
-4, -5, 11, -5, 12, 4, 0, 3, -1, 10, -1, 1 },
{ -1, 2, 0, -12, -1, -9, -14, -3, 14, -7, -39, -2,
-25, 29, -12, 14, 13, 11, -4, 48, -40, -3, 3, -17,
66, -21, 21, 1, 4, 8, -15, 21, -7, 8, -13, -6,
-2, -2, -21, 31, -9, -3, 4, -3, 9, -2, 6, 3 },
{ -1, -2, -3, 3, 7, 3, -14, -11, -6, 5, -5, -1,
14, 51, 10, 8, -65, -2, 1, 0, 39, 15, 23, 5,
-9, -53, -24, 1, 1, -11, -35, 26, 15, 7, 9, 1,
5, 4, 8, 13, 0, -13, -4, -5, 7, -6, 4, 1 },
{ 1, 3, -2, 8, -1, -2, 4, 0, 0, -2, 35, -20,
0, 13, 11, 5, -31, 9, 12, -6, -64, -8, -80, 14,
-12, -10, -4, 1, 1, -2, -10, 15, 23, -5, -25, 8,
3, -6, 16, 3, -1, 0, -4, -3, 8, 6, -1, 2 },
{ 1, -1, -3, -4, 3, 6, 1, -11, 4, -11, 15, 13,
1, 5, -3, -55, 25, 14, -48, -15, 25, 8, -20, 24,
41, -21, -13, -22, -7, -13, 27, 3, 41, -14, 7, -3,
-1, -18, 14, 27, -15, 12, -5, 4, 5, 7, -2, 10 },
{ 1, -3, 1, 2, 0, -1, -6, 4, -6, -3, -11, 4,
-5, 9, 48, -42, -15, -21, 3, 4, -41, -2, 27, -29,
1, 29, -53, 0, -5, -16, 31, 19, 20, 14, 4, -1,
1, 16, -26, 8, 26, 5, -5, 12, -10, 7, -6, -3 },
{ 0, 4, 2, -8, -4, -6, 14, 10, 12, 13, -17, -17,
-19, -5, 3, -56, -5, 28, 33, 24, -3, 13, -14, -22,
-21, -44, -15, -26, -7, -16, 0, -32, -48, -27, 13, 4,
4, -1, 1, -6, -16, 6, -6, -2, -5, -2, -1, -7 },
{ 0, -2, 1, 0, 1, -4, -8, 4, -4, 6, 12, -3,
-9, 4, -2, -31, 24, -16, 13, -24, 13, -16, -12, 24,
17, 0, -9, -20, -14, -2, -13, 55, -56, 48, -18, 3,
-3, 1, 10, -18, 19, -8, 2, 3, 14, -30, 19, -5 },
{ -1, 1, 0, -1, -2, 1, -12, 3, 2, 0, -8, -5,
4, 2, 9, 20, 6, -35, -8, 28, 39, 23, -48, -10,
4, 24, -24, 8, 3, -11, 3, 26, -22, -62, -1, -5,
4, -29, -23, 5, 24, -10, 2, 7, 3, -13, -14, -8 },
{ 0, -1, 1, 1, 3, 3, -2, -5, -2, 5, -15, 1,
5, 8, 15, -22, -33, -6, -3, 5, 21, -74, 9, 5,
-4, -5, 30, -1, -2, 10, 25, 3, 0, -28, -59, -17,
-21, -16, -19, -18, -16, 9, 3, -2, -8, -5, -6, 3 },
{ 0, 0, 2, -1, -1, -1, 4, 4, 4, -1, -1, 1,
-2, -4, -7, 22, 30, 20, -3, -4, 6, -48, 1, 6,
10, -6, -75, 11, -1, -27, -40, -42, 7, 1, -31, -5,
-6, -1, -5, 3, 5, -30, -3, 8, -2, 2, 3, -4 },
{ 0, -1, -3, 1, 4, -1, -14, -4, -1, -8, 7, 7,
-3, 22, 11, 10, -25, -13, -10, -6, -12, 15, -8, 11,
54, -12, -7, 17, 7, -1, 26, -42, -14, 4, 15, 0,
5, -10, 10, -70, -3, -8, 7, 19, -33, -18, 14, -16 },
{ 1, -2, -1, 6, 1, 6, 0, -4, -8, -4, 17, 4,
14, 2, 0, -11, 4, 1, -11, -17, -42, 11, 34, 41,

```

```

        2, 6, 1, -17, -13, -1, -41, 7, -10, -38, 18, -2,
        -5, -18, -59, -24, -23, -9, -7, -23, 14, -10, -21, -4 },
    { 0, 0, 0, 1, 1, 1, -7, -4, -1, 0, 3, 2,
      3, 7, 20, -32, -17, -8, -2, -2, 19, 8, -22, -2,
      14, 44, -8, 46, 0, 27, -18, -19, -26, 12, -3, 13,
      6, 23, -6, 22, -64, -21, 0, -22, 8, 14, 10, 22 },
    { 1, -1, 1, -1, -3, -1, 0, 0, 2, 2, -3, -5,
      1, -4, -21, -36, 19, 7, 0, 7, -13, -18, 7, 5,
      -11, -27, -9, 68, 22, 39, 4, 29, 12, -24, 27, -7,
      -9, 12, 21, -16, 18, -32, 28, -2, 7, -8, -18, -15 },
    },
},
}

```

```

Stx_Scan_Map[ IST_SET_SIZE_8X8 ][ STX_TYPES - 1 ][ IST_8X8_WIDTH ] =
{
    {
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 30, 31, 32, 33, 34,
          35, 36, 37, 38, 39, 40, 45, 46, 47, 48, 49, 50, 51, 56, 57, 58 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
          32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 44, 45, 46, 47, 51 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
          32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 44, 45, 46, 47, 51 }
    },
    {
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32,
          33, 38, 39, 40, 41, 42, 43, 44, 45, 46, 51, 52, 53, 54, 55, 60 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32,
          33, 38, 39, 40, 41, 42, 43, 44, 45, 46, 51, 52, 53, 54, 55, 60 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
          32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 44, 45, 46, 47, 51 }
    },
    {
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32,
          33, 38, 39, 40, 41, 42, 43, 44, 45, 46, 51, 52, 53, 54, 55, 60 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32,
          33, 38, 39, 40, 41, 42, 43, 44, 45, 46, 51, 52, 53, 54, 55, 60 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
          32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 44, 45, 46, 47, 51 }
    },
    {
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 30, 31, 32, 33, 34,
          35, 36, 37, 38, 39, 40, 45, 46, 47, 48, 49, 50, 51, 56, 57, 58 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32,
          33, 38, 39, 40, 41, 42, 43, 44, 45, 46, 51, 52, 53, 54, 55, 60 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
          32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 44, 45, 46, 47, 51 },
    },
    {
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32,
          33, 38, 39, 40, 41, 42, 43, 44, 45, 46, 51, 52, 53, 54, 55, 60 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32,
          33, 38, 39, 40, 41, 42, 43, 44, 45, 46, 51, 52, 53, 54, 55, 60 },
        { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
          16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
          32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 44, 45, 46, 47, 51 },
    }
}

```


§ 9.8. Loop restoration tables

```
Gdf_Alpha[ 6 ][ 6 ][ 22 ][ 4 ] =
{
  {
    {
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 386, 511, 386},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {43, 511, 43, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {10, 111, 10, 111},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {386, 511, 386, 511},
      {511, 511, 511, 511},
      {511, 43, 511, 43},
      {111, 10, 111, 10},
      {25, 25, 25, 25},
      {28, 28, 28, 28},
      {2, 2, 2, 2},
      {2, 2, 2, 2},
    },
    {
      {478, 120, 478, 120},
      {511, 511, 511, 511},
      {232, 109, 232, 109},
      {511, 127, 511, 127},
      {511, 511, 511, 511},
      {375, 15, 375, 15},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {120, 478, 120, 478},
      {511, 511, 511, 511},
      {109, 232, 109, 232},
      {127, 511, 127, 511},
      {15, 375, 15, 375},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {8, 8, 8, 8},
      {10, 10, 10, 10},
      {511, 511, 511, 511},
    },
    {
      {511, 103, 511, 103},
      {511, 114, 511, 114},
      {511, 4, 511, 4},
      {511, 13, 511, 13},
      {511, 204, 511, 11},
      {511, 511, 511, 511},
      {511, 11, 511, 204},
      {204, 511, 11, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {511, 511, 511, 511},
      {11, 511, 204, 511},
      {103, 511, 103, 511},
      {114, 511, 114, 511},
      {4, 511, 4, 511},
      {13, 511, 13, 511},
    }
  }
}
```

```

{511, 511, 511, 511},
{511, 511, 511, 511},
{66, 66, 66, 66},
{22, 22, 22, 22},
{10, 10, 10, 10},
{147, 147, 147, 147},
},
{
{511, 511, 511, 511},
{511, 5, 511, 5},
{511, 6, 511, 6},
{511, 511, 511, 511},
{511, 511, 39, 16},
{18, 36, 18, 36},
{39, 16, 511, 511},
{511, 511, 16, 39},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{16, 39, 511, 511},
{511, 511, 511, 511},
{5, 511, 5, 511},
{6, 511, 6, 511},
{511, 511, 511, 511},
{36, 18, 36, 18},
{511, 511, 511, 511},
{52, 52, 52, 52},
{72, 72, 72, 72},
{69, 69, 69, 69},
{78, 78, 78, 78},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 12, 511, 12},
{511, 286, 511, 286},
{511, 511, 511, 25},
{16, 64, 16, 64},
{511, 25, 511, 511},
{511, 511, 25, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{25, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{12, 511, 12, 511},
{286, 511, 286, 511},
{64, 16, 64, 16},
{511, 511, 511, 511},
{201, 201, 201, 201},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{17, 25, 17, 25},
{511, 511, 511, 511},
{511, 511, 52, 35},
{511, 64, 511, 64},
{52, 35, 511, 511},
{511, 511, 35, 52},
{511, 511, 26, 26},
{511, 178, 511, 178},
{26, 26, 511, 511},
{35, 52, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{25, 17, 25, 17},

```

```

    {511, 511, 511, 511},
    {64, 511, 64, 511},
    {178, 511, 178, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {3, 3, 3, 3},
  },
},
{
  {
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {197, 145, 197, 145},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 17, 17},
    {511, 40, 511, 40},
    {17, 17, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {145, 197, 145, 197},
    {511, 511, 511, 511},
    {40, 511, 40, 511},
    {511, 511, 511, 511},
    {335, 335, 335, 335},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
  },
  {
    {511, 511, 511, 511},
    {98, 511, 98, 511},
    {511, 511, 511, 511},
    {118, 145, 118, 145},
    {511, 511, 511, 12},
    {511, 511, 511, 511},
    {511, 12, 511, 511},
    {511, 511, 12, 511},
    {511, 511, 511, 511},
    {26, 33, 26, 33},
    {511, 511, 511, 511},
    {12, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 98, 511, 98},
    {511, 511, 511, 511},
    {145, 118, 145, 118},
    {511, 511, 511, 511},
    {33, 26, 33, 26},
    {14, 14, 14, 14},
    {12, 12, 12, 12},
    {171, 171, 171, 171},
    {48, 48, 48, 48},
  },
  {
    {6, 93, 6, 93},
    {511, 6, 511, 6},
    {511, 7, 511, 7},
    {85, 130, 85, 130},
    {511, 511, 6, 511},
    {511, 39, 511, 39},
    {6, 511, 511, 511},
    {511, 511, 511, 6},
    {511, 511, 26, 26},
    {511, 511, 511, 511},
    {26, 26, 511, 511},
    {511, 6, 511, 511},
  }
}

```

```

{93, 6, 93, 6},
{6, 511, 6, 511},
{7, 511, 7, 511},
{130, 85, 130, 85},
{39, 511, 39, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{9, 9, 9, 9},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 9, 511, 9},
{511, 511, 511, 511},
{511, 1, 511, 17},
{2, 511, 2, 511},
{511, 17, 511, 1},
{1, 511, 17, 511},
{511, 511, 511, 511},
{12, 47, 12, 47},
{511, 511, 511, 511},
{17, 511, 1, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{9, 511, 9, 511},
{511, 511, 511, 511},
{511, 2, 511, 2},
{47, 12, 47, 12},
{511, 511, 511, 511},
{183, 183, 183, 183},
{92, 92, 92, 92},
{511, 511, 511, 511},
},
{
{511, 6, 511, 6},
{511, 4, 511, 4},
{511, 7, 511, 7},
{511, 9, 511, 9},
{511, 511, 511, 26},
{511, 131, 511, 131},
{511, 26, 511, 511},
{511, 511, 26, 511},
{511, 511, 9, 9},
{511, 511, 511, 511},
{9, 9, 511, 511},
{26, 511, 511, 511},
{6, 511, 6, 511},
{4, 511, 4, 511},
{7, 511, 7, 511},
{9, 511, 9, 511},
{131, 511, 131, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 12, 511, 12},
{511, 23, 511, 23},
{14, 511, 14, 511},
{511, 511, 511, 511},
{511, 511, 13, 511},
{511, 511, 511, 511},
{13, 511, 511, 511},
{511, 511, 511, 13},
{511, 511, 511, 511},
{511, 20, 511, 20},
{511, 511, 511, 511},

```

```

{511, 13, 511, 511},
{12, 511, 12, 511},
{23, 511, 23, 511},
{511, 14, 511, 14},
{511, 511, 511, 511},
{511, 511, 511, 511},
{20, 511, 20, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
},
{
{
{511, 511, 511, 511},
{138, 511, 138, 511},
{511, 511, 511, 511},
{238, 511, 238, 511},
{511, 511, 511, 511},
{59, 511, 59, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{24, 47, 24, 47},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 138, 511, 138},
{511, 511, 511, 511},
{511, 238, 511, 238},
{511, 59, 511, 59},
{47, 24, 47, 24},
{511, 511, 511, 511},
{511, 511, 511, 511},
{242, 242, 242, 242},
{74, 74, 74, 74},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 158, 511, 158},
{133, 511, 133, 511},
{511, 4, 511, 14},
{511, 11, 511, 11},
{511, 14, 511, 4},
{4, 511, 14, 511},
{511, 511, 511, 511},
{511, 52, 511, 52},
{511, 511, 511, 511},
{14, 511, 4, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{158, 511, 158, 511},
{511, 133, 511, 133},
{11, 511, 11, 511},
{52, 511, 52, 511},
{467, 467, 467, 467},
{511, 511, 511, 511},
{28, 28, 28, 28},
{33, 33, 33, 33},
},
{
{511, 214, 511, 214},
{511, 511, 511, 511},
{79, 194, 79, 194},
{511, 289, 511, 289},
{511, 72, 37, 24},
{43, 21, 43, 21},
{37, 24, 511, 72},
{72, 511, 24, 37},

```

```

{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{24, 37, 72, 511},
{214, 511, 214, 511},
{511, 511, 511, 511},
{194, 79, 194, 79},
{289, 511, 289, 511},
{21, 43, 21, 43},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{115, 115, 115, 115},
{23, 23, 23, 23},
},
{
{511, 211, 511, 211},
{511, 511, 511, 511},
{132, 511, 132, 511},
{511, 511, 511, 511},
{511, 511, 32, 25},
{56, 8, 56, 8},
{32, 25, 511, 511},
{511, 511, 25, 32},
{511, 511, 14, 14},
{511, 67, 511, 67},
{14, 14, 511, 511},
{25, 32, 511, 511},
{211, 511, 211, 511},
{511, 511, 511, 511},
{511, 132, 511, 132},
{511, 511, 511, 511},
{8, 56, 8, 56},
{67, 511, 67, 511},
{89, 89, 89, 89},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 6, 511, 6},
{511, 10, 511, 10},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 24},
{511, 511, 511, 511},
{511, 24, 511, 511},
{511, 511, 24, 511},
{511, 511, 6, 6},
{511, 511, 511, 511},
{6, 6, 511, 511},
{24, 511, 511, 511},
{6, 511, 6, 511},
{10, 511, 10, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{26, 511, 26, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},

```

```

{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 26, 511, 26},
{511, 511, 511, 511},
{511, 511, 511, 511},
{6, 6, 6, 6},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
},
{
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{115, 511, 115, 511},
{511, 511, 511, 511},
{511, 173, 511, 173},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 14, 14},
{26, 51, 26, 51},
{14, 14, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 115, 511, 115},
{173, 511, 173, 511},
{51, 26, 51, 26},
{1, 1, 1, 1},
{2, 2, 2, 2},
{511, 511, 511, 511},
{384, 384, 384, 384},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 153, 511, 153},
{511, 511, 67, 12},
{511, 511, 511, 511},
{67, 12, 511, 511},
{511, 511, 12, 67},
{511, 511, 511, 511},
{30, 37, 30, 37},
{511, 511, 511, 511},
{12, 67, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{153, 511, 153, 511},
{511, 511, 511, 511},
{37, 30, 37, 30},
{511, 511, 511, 511},
{12, 12, 12, 12},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{106, 11, 106, 11},

```

```

{511, 511, 5, 34},
{511, 511, 511, 511},
{5, 34, 511, 511},
{511, 511, 34, 5},
{511, 511, 9, 9},
{511, 44, 511, 44},
{9, 9, 511, 511},
{34, 5, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{11, 106, 11, 106},
{511, 511, 511, 511},
{44, 511, 44, 511},
{3, 3, 3, 3},
{7, 7, 7, 7},
{4, 4, 4, 4},
{511, 511, 511, 511},
},
{
{511, 142, 511, 142},
{511, 9, 511, 9},
{511, 5, 511, 5},
{511, 511, 511, 511},
{511, 511, 511, 10},
{511, 68, 511, 68},
{511, 10, 511, 511},
{511, 511, 10, 511},
{511, 511, 46, 46},
{511, 28, 511, 28},
{46, 46, 511, 511},
{10, 511, 511, 511},
{142, 511, 142, 511},
{9, 511, 9, 511},
{5, 511, 5, 511},
{511, 511, 511, 511},
{68, 511, 68, 511},
{28, 511, 28, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 511, 511, 511},
{511, 13, 511, 13},
{511, 511, 511, 511},
{511, 37, 511, 37},
{511, 511, 50, 8},
{511, 511, 511, 511},
{50, 8, 511, 511},
{511, 511, 8, 50},
{511, 511, 2, 2},
{511, 71, 511, 71},
{2, 2, 511, 511},
{8, 50, 511, 511},
{511, 511, 511, 511},
{13, 511, 13, 511},
{511, 511, 511, 511},
{37, 511, 37, 511},
{511, 511, 511, 511},
{71, 511, 71, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},

```



```

{24, 511, 24, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 35, 35},
{511, 511, 511, 511},
{35, 35, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 24, 511, 24},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{13, 13, 13, 13},
{12, 12, 12, 12},
},
},
{
{
{511, 511, 511, 511},
{511, 120, 511, 120},
{511, 511, 511, 511},
{323, 511, 323, 511},
{511, 511, 49, 511},
{511, 511, 511, 511},
{49, 511, 511, 511},
{511, 511, 511, 49},
{511, 511, 17, 17},
{511, 45, 511, 45},
{17, 17, 511, 511},
{511, 49, 511, 511},
{511, 511, 511, 511},
{120, 511, 120, 511},
{511, 511, 511, 511},
{511, 323, 511, 323},
{511, 511, 511, 511},
{45, 511, 45, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{53, 53, 53, 53},
{47, 47, 47, 47},
},
{
{511, 511, 511, 511},
{511, 231, 511, 231},
{105, 511, 105, 511},
{511, 511, 511, 511},
{511, 511, 511, 14},
{511, 511, 511, 511},
{511, 14, 511, 511},
{511, 511, 14, 511},
{511, 511, 511, 511},
{511, 53, 511, 53},
{511, 511, 511, 511},
{14, 511, 511, 511},
{511, 511, 511, 511},
{231, 511, 231, 511},
{511, 105, 511, 105},
{511, 511, 511, 511},
{511, 511, 511, 511},
{53, 511, 53, 511},
{9, 9, 9, 9},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{

```

```

{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 238, 511, 238},
{119, 511, 119, 511},
{511, 4, 10, 7},
{511, 511, 511, 511},
{10, 7, 511, 4},
{4, 511, 7, 10},
{511, 511, 29, 29},
{511, 42, 511, 42},
{29, 29, 511, 511},
{7, 10, 4, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{238, 511, 238, 511},
{511, 119, 511, 119},
{511, 511, 511, 511},
{42, 511, 42, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{12, 12, 12, 12},
{9, 9, 9, 9},
},
{
{511, 5, 511, 5},
{511, 5, 511, 5},
{511, 5, 511, 5},
{511, 210, 511, 210},
{511, 511, 20, 12},
{511, 67, 511, 67},
{20, 12, 511, 511},
{511, 511, 12, 20},
{511, 511, 47, 47},
{511, 511, 511, 511},
{47, 47, 511, 511},
{12, 20, 511, 511},
{5, 511, 5, 511},
{5, 511, 5, 511},
{5, 511, 5, 511},
{210, 511, 210, 511},
{67, 511, 67, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{71, 71, 71, 71},
{511, 511, 511, 511},
},
{
{511, 4, 511, 4},
{511, 5, 511, 5},
{511, 10, 511, 10},
{511, 511, 511, 511},
{511, 1, 511, 23},
{511, 511, 511, 511},
{511, 23, 511, 1},
{1, 511, 23, 511},
{511, 511, 7, 7},
{511, 40, 511, 40},
{7, 7, 511, 511},
{23, 511, 1, 511},
{4, 511, 4, 511},
{5, 511, 5, 511},
{10, 511, 10, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{40, 511, 40, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},

```

```

{
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {25, 511, 25, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {511, 25, 511, 25},
  {511, 511, 511, 511},
  {511, 511, 511, 511},
  {11, 11, 11, 11},
  {14, 14, 14, 14},
  {16, 16, 16, 16},
  {511, 511, 511, 511},
},
{
  {
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 163, 511, 163},
    {342, 511, 342, 511},
    {511, 511, 511, 511},
    {511, 10, 511, 10},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {163, 511, 163, 511},
    {511, 342, 511, 342},
    {10, 511, 10, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {34, 34, 34, 34},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
  },
  {
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 123, 511, 123},
    {40, 511, 511, 14},
    {511, 511, 511, 511},
    {511, 14, 40, 511},
    {511, 40, 14, 511},
    {511, 511, 511, 511},
    {511, 49, 511, 49},
    {511, 511, 511, 511},
    {14, 511, 511, 40},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {123, 511, 123, 511},
    {511, 511, 511, 511},
    {49, 511, 49, 511},
    {511, 511, 511, 511},
    {429, 429, 429, 429},
  },
},

```

```

{62, 62, 62, 62},
{511, 511, 511, 511},
},
{
{511, 511, 511, 511},
{511, 511, 511, 511},
{114, 511, 114, 511},
{511, 511, 511, 511},
{511, 57, 511, 511},
{511, 9, 511, 9},
{511, 511, 511, 57},
{57, 511, 511, 511},
{14, 14, 46, 46},
{511, 511, 511, 511},
{46, 46, 14, 14},
{511, 511, 57, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 114, 511, 114},
{511, 511, 511, 511},
{9, 511, 9, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 4, 511, 4},
{16, 7, 16, 7},
{151, 511, 151, 511},
{511, 511, 511, 511},
{511, 511, 11, 511},
{40, 18, 40, 18},
{11, 511, 511, 511},
{511, 511, 511, 11},
{511, 511, 20, 20},
{511, 511, 511, 511},
{20, 20, 511, 511},
{511, 11, 511, 511},
{4, 511, 4, 511},
{7, 16, 7, 16},
{511, 151, 511, 151},
{511, 511, 511, 511},
{18, 40, 18, 40},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
},
{
{511, 7, 511, 7},
{511, 511, 511, 511},
{511, 12, 511, 12},
{511, 511, 511, 511},
{511, 511, 24, 23},
{5, 511, 5, 511},
{24, 23, 511, 511},
{511, 511, 23, 24},
{511, 511, 511, 511},
{511, 511, 511, 511},
{511, 511, 511, 511},
{23, 24, 511, 511},
{7, 511, 7, 511},
{511, 511, 511, 511},
{12, 511, 12, 511},
{511, 511, 511, 511},
{511, 5, 511, 5},
{511, 511, 511, 511},
{511, 511, 511, 511},

```

```

    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
  },
  {
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 4},
    {30, 511, 30, 511},
    {511, 4, 511, 511},
    {511, 511, 4, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {4, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {511, 30, 511, 30},
    {511, 511, 511, 511},
    {511, 511, 511, 511},
    {75, 75, 75, 75},
    {21, 21, 21, 21},
    {511, 511, 511, 511},
  },
},
}

```

```

Gdf_Weight[ 6 ][ 6 ][ 3 ][ 22 ][ 4 ] =
{
  {
    {
      {
        {-5, 0, -5, 0},
        {13, 0, 13, 0},
        {-19, 0, -19, 0},
        {15, 0, 15, 0},
        {19, 0, -24, 0},
        {-17, -36, -17, -36},
        {-24, 0, 19, 0},
        {0, 19, 0, -24},
        {-39, -39, 39, 39},
        {103, 327, 103, 327},
        {39, 39, -39, -39},
        {0, -24, 0, 19},
        {0, -5, 0, -5},
        {0, 13, 0, 13},
        {0, -19, 0, -19},
        {0, 15, 0, 15},
        {-36, -17, -36, -17},
        {327, 103, 327, 103},
        {0, 0, 0, 0},
        {0, 0, 0, 0},
        {0, 0, 0, 0},
        {0, 0, 0, 0},
      },
      {
        {0, -11, 0, -11},
        {0, 33, 0, 33},
        {0, -60, 0, -60},
        {0, 91, 0, 91},
        {0, 11, 0, -12},
        {-29, -138, -29, -138},
        {0, -12, 0, 11},
        {11, 0, -12, 0},
        {-22, -22, 17, 17},
        {36, 136, 36, 136},
      }
    }
  }
}

```

```

{17, 17, -22, -22},
{-12, 0, 11, 0},
{-11, 0, -11, 0},
{33, 0, 33, 0},
{-60, 0, -60, 0},
{91, 0, 91, 0},
{-138, -29, -138, -29},
{136, 36, 136, 36},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-15, -31, -15, -31},
{0, 0, 0, 0},
{13, 46, 13, 46},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-49, -12, -49, -12},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-31, -15, -31, -15},
{46, 13, 46, 13},
{-12, -49, -12, -49},
{59, 59, 59, 59},
{57, 57, 57, 57},
{35, 35, 35, 35},
{22, 22, 22, 22},
},
},
{
{
{-4, 0, -4, 0},
{10, 0, 10, 0},
{-14, 0, -14, 0},
{-2, 0, -2, 0},
{7, 0, -44, 0},
{23, 80, 23, 80},
{-44, 0, 7, 0},
{0, 7, 0, -44},
{-41, -41, 126, 126},
{-110, 135, -110, 135},
{126, 126, -41, -41},
{0, -44, 0, 7},
{0, -4, 0, -4},
{0, 10, 0, 10},
{0, -14, 0, -14},
{0, -2, 0, -2},
{80, 23, 80, 23},
{135, -110, 135, -110},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -9, 0, -9},
{0, 9, 0, 9},
{0, -8, 0, -8},
{0, -22, 0, -22},
{0, 2, 0, -42},
{34, 244, 34, 244},
{0, -42, 0, 2},

```

```

{2, 0, -42, 0},
{1, 1, 122, 122},
{-53, 139, -53, 139},
{122, 122, 1, 1},
{-42, 0, 2, 0},
{-9, 0, -9, 0},
{9, 0, 9, 0},
{-8, 0, -8, 0},
{-22, 0, -22, 0},
{244, 34, 244, 34},
{139, -53, 139, -53},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{55, 49, 55, 49},
{0, 0, 0, 0},
{1, 56, 1, 56},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-105, -71, -105, -71},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{49, 55, 49, 55},
{56, 1, 56, 1},
{-71, -105, -71, -105},
{-12, -12, -12, -12},
{61, 61, 61, 61},
{51, 51, 51, 51},
{-19, -19, -19, -19},
},
},
{
{
{-20, 0, -20, 0},
{8, 0, 8, 0},
{-16, 0, -16, 0},
{-10, 0, -10, 0},
{-16, 0, -24, 0},
{14, -36, 14, -36},
{-24, 0, -16, 0},
{0, -16, 0, -24},
{-27, -27, 52, 52},
{-21, 106, -21, 106},
{52, 52, -27, -27},
{0, -24, 0, -16},
{0, -20, 0, -20},
{0, 8, 0, 8},
{0, -16, 0, -16},
{0, -10, 0, -10},
{-36, 14, -36, 14},
{106, -21, 106, -21},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -14, 0, -14},
{0, -24, 0, -24},
{0, 462, 0, 462},
{0, 172, 0, 172},

```

```

{0, -15, 0, 472},
{33, -32, 33, -32},
{0, 472, 0, -15},
{-15, 0, 472, 0},
{19, 19, 143, 143},
{-126, 125, -126, 125},
{143, 143, 19, 19},
{472, 0, -15, 0},
{-14, 0, -14, 0},
{-24, 0, -24, 0},
{462, 0, 462, 0},
{172, 0, 172, 0},
{-32, 33, -32, 33},
{125, -126, 125, -126},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-5, -265, -5, -265},
{0, 0, 0, 0},
{6, -2, 6, -2},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{87, -194, 87, -194},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-265, -5, -265, -5},
{-2, 6, -2, 6},
{-194, 87, -194, 87},
{-5, -5, -5, -5},
{-4, -4, -4, -4},
{0, 0, 0, 0},
{-4, -4, -4, -4},
},
},
{
{
{-19, 0, -19, 0},
{3, 0, 3, 0},
{-11, 0, -11, 0},
{11, 0, 11, 0},
{-27, 0, -79, 0},
{-99, 29, -99, 29},
{-79, 0, -27, 0},
{0, -27, 0, -79},
{-22, -22, 18, 18},
{-4, 90, -4, 90},
{18, 18, -22, -22},
{0, -79, 0, -27},
{0, -19, 0, -19},
{0, 3, 0, 3},
{0, -11, 0, -11},
{0, 11, 0, 11},
{29, -99, 29, -99},
{90, -4, 90, -4},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -15, 0, -15},

```



```

{0, 511, 0, 511},
{0, 372, 0, 372},
{0, -23, 0, -23},
{0, -2, 0, 422},
{173, 243, 173, 243},
{0, 422, 0, -2},
{-2, 0, 422, 0},
{-21, -21, 84, 84},
{-114, 110, -114, 110},
{84, 84, -21, -21},
{422, 0, -2, 0},
{-15, 0, -15, 0},
{511, 0, 511, 0},
{372, 0, 372, 0},
{-23, 0, -23, 0},
{243, 173, 243, 173},
{110, -114, 110, -114},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{39, 87, 39, 87},
{0, 0, 0, 0},
{-394, -13, -394, -13},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-37, 51, -37, 51},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{87, 39, 87, 39},
{-13, -394, -13, -394},
{51, -37, 51, -37},
{41, 41, 41, 41},
{33, 33, 33, 33},
{-24, -24, -24, -24},
{-20, -20, -20, -20},
},
},
{
{
{-46, 0, -46, 0},
{15, 0, 15, 0},
{-25, 0, -25, 0},
{10, 0, 10, 0},
{-39, 0, -23, 0},
{144, 76, 144, 76},
{-23, 0, -39, 0},
{0, -39, 0, -23},
{1, 1, 50, 50},
{75, 36, 75, 36},
{50, 50, 1, 1},
{0, -23, 0, -39},
{0, -46, 0, -46},
{0, 15, 0, 15},
{0, -25, 0, -25},
{0, 10, 0, 10},
{76, 144, 76, 144},
{36, 75, 36, 75},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},

```

```

},
{
  {0, -8, 0, -8},
  {0, -6, 0, -6},
  {0, 511, 0, 511},
  {0, -28, 0, -28},
  {0, -14, 0, 289},
  {509, 237, 509, 237},
  {0, 289, 0, -14},
  {-14, 0, 289, 0},
  {1, 1, 58, 58},
  {-94, -16, -94, -16},
  {58, 58, 1, 1},
  {289, 0, -14, 0},
  {-8, 0, -8, 0},
  {-6, 0, -6, 0},
  {511, 0, 511, 0},
  {-28, 0, -28, 0},
  {237, 509, 237, 509},
  {-16, -94, -16, -94},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
},
{
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-21, -152, -21, -152},
  {0, 0, 0, 0},
  {-94, -46, -94, -46},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-95, 55, -95, 55},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-152, -21, -152, -21},
  {-46, -94, -46, -94},
  {55, -95, 55, -95},
  {2, 2, 2, 2},
  {5, 5, 5, 5},
  {-6, -6, -6, -6},
  {-2, -2, -2, -2},
},
},
{
  {
    {-35, 0, -35, 0},
    {-3, 0, -3, 0},
    {511, 0, 511, 0},
    {-47, 0, -47, 0},
    {-40, 0, 318, 0},
    {-57, -88, -57, -88},
    {318, 0, -40, 0},
    {0, -40, 0, 318},
    {30, 30, -280, -280},
    {135, -76, 135, -76},
    {-280, -280, 30, 30},
    {0, 318, 0, -40},
    {0, -35, 0, -35},
    {0, -3, 0, -3},
    {0, 511, 0, 511},
    {0, -47, 0, -47},
    {-88, -57, -88, -57},
    {-76, 135, -76, 135},
    {0, 0, 0, 0},
  }
}

```

```

    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
},
{
    {0, -32, 0, -32},
    {0, -11, 0, -11},
    {0, 511, 0, 511},
    {0, -31, 0, -31},
    {0, 5, 0, 320},
    {16, 239, 16, 239},
    {0, 320, 0, 5},
    {5, 0, 320, 0},
    {-45, -45, 66, 66},
    {-154, -69, -154, -69},
    {66, 66, -45, -45},
    {320, 0, 5, 0},
    {-32, 0, -32, 0},
    {-11, 0, -11, 0},
    {511, 0, 511, 0},
    {-31, 0, -31, 0},
    {239, 16, 239, 16},
    {-69, -154, -69, -154},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
},
{
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {91, 17, 91, 17},
    {0, 0, 0, 0},
    {-36, 87, -36, 87},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-6, -49, -6, -49},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {17, 91, 17, 91},
    {87, -36, 87, -36},
    {-49, -6, -49, -6},
    {1, 1, 1, 1},
    {2, 2, 2, 2},
    {1, 1, 1, 1},
    {12, 12, 12, 12},
},
},
{
    {
        {
            {-3, 0, -3, 0},
            {6, 0, 6, 0},
            {-11, 0, -11, 0},
            {26, 0, 26, 0},
            {-5, 0, 23, 0},
            {-21, 37, -21, 37},
            {23, 0, -5, 0},
            {0, -5, 0, 23},
            {13, 13, -246, -246},
            {17, -315, 17, -315},
            {-246, -246, 13, 13},
            {0, 23, 0, -5},
            {0, -3, 0, -3},
            {0, 6, 0, 6},

```

```

{0, -11, 0, -11},
{0, 26, 0, 26},
{37, -21, 37, -21},
{-315, 17, -315, 17},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -16, 0, -16},
{0, 9, 0, 9},
{0, -25, 0, -25},
{0, 28, 0, 28},
{0, -25, 0, 13},
{-25, 26, -25, 26},
{0, 13, 0, -25},
{-25, 0, 13, 0},
{6, 6, -112, -112},
{29, -23, 29, -23},
{-112, -112, 6, 6},
{13, 0, -25, 0},
{-16, 0, -16, 0},
{9, 0, 9, 0},
{-25, 0, -25, 0},
{28, 0, 28, 0},
{26, -25, 26, -25},
{-23, 29, -23, 29},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{17, 20, 17, 20},
{0, 0, 0, 0},
{7, 30, 7, 30},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-12, 32, -12, 32},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{20, 17, 20, 17},
{30, 7, 30, 7},
{32, -12, 32, -12},
{-3, -3, -3, -3},
{24, 24, 24, 24},
{-2, -2, -2, -2},
{4, 4, 4, 4},
},
},
{
{
{-12, 0, -12, 0},
{19, 0, 19, 0},
{-54, 0, -54, 0},
{79, 0, 79, 0},
{-64, 0, 0, 0},
{-4, 23, -4, 23},
{0, 0, -64, 0},
{0, -64, 0, 0},
{114, 114, 61, 61},
{-12, -126, -12, -126},
{61, 61, 114, 114},

```

```

{0, 0, 0, -64},
{0, -12, 0, -12},
{0, 19, 0, 19},
{0, -54, 0, -54},
{0, 79, 0, 79},
{23, -4, 23, -4},
{-126, -12, -126, -12},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 3, 0, 3},
{0, -5, 0, -5},
{0, 18, 0, 18},
{0, -36, 0, -36},
{0, 20, 0, 263},
{6, -15, 6, -15},
{0, 263, 0, 20},
{20, 0, 263, 0},
{-30, -30, -9, -9},
{266, 329, 266, 329},
{-9, -9, -30, -30},
{263, 0, 20, 0},
{3, 0, 3, 0},
{-5, 0, -5, 0},
{18, 0, 18, 0},
{-36, 0, -36, 0},
{-15, 6, -15, 6},
{329, 266, 329, 266},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-52, -39, -52, -39},
{0, 0, 0, 0},
{-57, -74, -57, -74},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{156, 336, 156, 336},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-39, -52, -39, -52},
{-74, -57, -74, -57},
{336, 156, 336, 156},
{-16, -16, -16, -16},
{-13, -13, -13, -13},
{6, 6, 6, 6},
{1, 1, 1, 1},
},
},
{
{
{85, 0, 85, 0},
{25, 0, 25, 0},
{10, 0, 10, 0},
{-83, 0, -83, 0},
{58, 0, 83, 0},
{45, -14, 45, -14},
{83, 0, 58, 0},
{0, 58, 0, 83},

```

```

{-16, -16, 18, 18},
{-146, -123, -146, -123},
{18, 18, -16, -16},
{0, 83, 0, 58},
{0, 85, 0, 85},
{0, 25, 0, 25},
{0, 10, 0, 10},
{0, -83, 0, -83},
{-14, 45, -14, 45},
{-123, -146, -123, -146},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 33, 0, 33},
{0, -336, 0, -336},
{0, -294, 0, -294},
{0, 61, 0, 61},
{0, 14, 0, 17},
{-1, -248, -1, -248},
{0, 17, 0, 14},
{14, 0, 17, 0},
{12, 12, -411, -411},
{30, -203, 30, -203},
{-411, -411, 12, 12},
{17, 0, 14, 0},
{33, 0, 33, 0},
{-336, 0, -336, 0},
{-294, 0, -294, 0},
{61, 0, 61, 0},
{-248, -1, -248, -1},
{-203, 30, -203, 30},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{69, 61, 69, 61},
{0, 0, 0, 0},
{-11, 75, -11, 75},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{168, 29, 168, 29},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{61, 69, 61, 69},
{75, -11, 75, -11},
{29, 168, 29, 168},
{-25, -25, -25, -25},
{123, 123, 123, 123},
{-1, -1, -1, -1},
{9, 9, 9, 9},
},
},
{
{
{21, 0, 21, 0},
{8, 0, 8, 0},
{2, 0, 2, 0},
{8, 0, 8, 0},
{61, 0, -11, 0},

```

```

{-167, -81, -167, -81},
{-11, 0, 61, 0},
{0, 61, 0, -11},
{-1, -1, 0, 0},
{123, -38, 123, -38},
{0, 0, -1, -1},
{0, -11, 0, 61},
{0, 21, 0, 21},
{0, 8, 0, 8},
{0, 2, 0, 2},
{0, 8, 0, 8},
{-81, -167, -81, -167},
{-38, 123, -38, 123},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
  {0, 11, 0, 11},
  {0, -3, 0, -3},
  {0, -512, 0, -512},
  {0, -1, 0, -1},
  {0, -307, 0, -468},
  {-291, -6, -291, -6},
  {0, -468, 0, -307},
  {-307, 0, -468, 0},
  {79, 79, 7, 7},
  {12, -512, 12, -512},
  {7, 7, 79, 79},
  {-468, 0, -307, 0},
  {11, 0, 11, 0},
  {-3, 0, -3, 0},
  {-512, 0, -512, 0},
  {-1, 0, -1, 0},
  {-6, -291, -6, -291},
  {-512, 12, -512, 12},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
},
{
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {41, 192, 41, 192},
  {0, 0, 0, 0},
  {-220, 146, -220, 146},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-15, -178, -15, -178},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {192, 41, 192, 41},
  {146, -220, 146, -220},
  {-178, -15, -178, -15},
  {-8, -8, -8, -8},
  {34, 34, 34, 34},
  {-35, -35, -35, -35},
  {-1, -1, -1, -1},
},
{
  {
    {78, 0, 78, 0},
    {-31, 0, -31, 0},

```

```

{22, 0, 22, 0},
{31, 0, 31, 0},
{71, 0, -73, 0},
{-48, -169, -48, -169},
{-73, 0, 71, 0},
{0, 71, 0, -73},
{37, 37, 3, 3},
{-31, -94, -31, -94},
{3, 3, 37, 37},
{0, -73, 0, 71},
{0, 78, 0, 78},
{0, -31, 0, -31},
{0, 22, 0, 22},
{0, 31, 0, 31},
{-169, -48, -169, -48},
{-94, -31, -94, -31},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 511, 0, 511},
{0, 511, 0, 511},
{0, 511, 0, 511},
{0, 469, 0, 469},
{0, -123, 0, 511},
{19, 51, 19, 51},
{0, 511, 0, -123},
{-123, 0, 511, 0},
{-4, -4, 511, 511},
{-58, 74, -58, 74},
{511, 511, -4, -4},
{511, 0, -123, 0},
{511, 0, 511, 0},
{511, 0, 511, 0},
{511, 0, 511, 0},
{469, 0, 469, 0},
{51, 19, 51, 19},
{74, -58, 74, -58},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-21, 15, -21, 15},
{0, 0, 0, 0},
{-63, -246, -63, -246},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-33, 87, -33, 87},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{15, -21, 15, -21},
{-246, -63, -246, -63},
{87, -33, 87, -33},
{38, 38, 38, 38},
{-53, -53, -53, -53},
{8, 8, 8, 8},
{0, 0, 0, 0},
},
},
{

```



```

{
    {216, 0, 216, 0},
    {-28, 0, -28, 0},
    {-512, 0, -512, 0},
    {-124, 0, -124, 0},
    {34, 0, -512, 0},
    {-53, 26, -53, 26},
    {-512, 0, 34, 0},
    {0, 34, 0, -512},
    {51, 51, -50, -50},
    {10, 144, 10, 144},
    {-50, -50, 51, 51},
    {0, -512, 0, 34},
    {0, 216, 0, 216},
    {0, -28, 0, -28},
    {0, -512, 0, -512},
    {0, -124, 0, -124},
    {26, -53, 26, -53},
    {144, 10, 144, 10},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
},
{
    {0, 511, 0, 511},
    {0, 511, 0, 511},
    {0, -174, 0, -174},
    {0, 92, 0, 92},
    {0, -189, 0, 291},
    {-42, 186, -42, 186},
    {0, 291, 0, -189},
    {-189, 0, 291, 0},
    {-147, -147, 67, 67},
    {-88, 411, -88, 411},
    {67, 67, -147, -147},
    {291, 0, -189, 0},
    {511, 0, 511, 0},
    {511, 0, 511, 0},
    {-174, 0, -174, 0},
    {92, 0, 92, 0},
    {186, -42, 186, -42},
    {411, -88, 411, -88},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
},
{
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {93, 71, 93, 71},
    {0, 0, 0, 0},
    {-57, -89, -57, -89},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {22, 371, 22, 371},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {71, 93, 71, 93},
    {-89, -57, -89, -57},
    {371, 22, 371, 22},
    {-3, -3, -3, -3},
    {2, 2, 2, 2},
    {2, 2, 2, 2},
    {10, 10, 10, 10},
}

```

```

    },
  },
{
  {
    {
      {6, 0, 6, 0},
      {-9, 0, -9, 0},
      {16, 0, 16, 0},
      {-1, 0, -1, 0},
      {38, 0, 4, 0},
      {-73, -23, -73, -23},
      {4, 0, 38, 0},
      {0, 38, 0, 4},
      {-74, -74, -35, -35},
      {195, 157, 195, 157},
      {-35, -35, -74, -74},
      {0, 4, 0, 38},
      {0, 6, 0, 6},
      {0, -9, 0, -9},
      {0, 16, 0, 16},
      {0, -1, 0, -1},
      {-23, -73, -23, -73},
      {157, 195, 157, 195},
      {0, 0, 0, 0},
      {0, 0, 0, 0},
      {0, 0, 0, 0},
      {0, 0, 0, 0},
    },
  },
  {
    {0, -10, 0, -10},
    {0, 5, 0, 5},
    {0, -9, 0, -9},
    {0, -2, 0, -2},
    {0, -19, 0, -8},
    {12, 16, 12, 16},
    {0, -8, 0, -19},
    {-19, 0, -8, 0},
    {-8, -8, -4, -4},
    {-119, -139, -119, -139},
    {-4, -4, -8, -8},
    {-8, 0, -19, 0},
    {-10, 0, -10, 0},
    {5, 0, 5, 0},
    {-9, 0, -9, 0},
    {-2, 0, -2, 0},
    {16, 12, 16, 12},
    {-139, -119, -139, -119},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {29, 5, 29, 5},
    {0, 0, 0, 0},
    {11, 36, 11, 36},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-203, -344, -203, -344},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {5, 29, 5, 29},
    {36, 11, 36, 11},
  },

```

```

    {-344, -203, -344, -203},
    {-5, -5, -5, -5},
    {-9, -9, -9, -9},
    {5, 5, 5, 5},
    {8, 8, 8, 8},
  },
},
{
  {
    {-8, 0, -8, 0},
    {1, 0, 1, 0},
    {-13, 0, -13, 0},
    {17, 0, 17, 0},
    {-29, 0, 1, 0},
    {-19, -140, -19, -140},
    {1, 0, -29, 0},
    {0, -29, 0, 1},
    {7, 7, 7, 7},
    {-20, -6, -20, -6},
    {7, 7, 7, 7},
    {0, 1, 0, -29},
    {0, -8, 0, -8},
    {0, 1, 0, 1},
    {0, -13, 0, -13},
    {0, 17, 0, 17},
    {-140, -19, -140, -19},
    {-6, -20, -6, -20},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, -8, 0, -8},
    {0, 29, 0, 29},
    {0, -39, 0, -39},
    {0, -12, 0, -12},
    {0, 126, 0, 362},
    {49, -41, 49, -41},
    {0, 362, 0, 126},
    {126, 0, 362, 0},
    {-67, -67, -22, -22},
    {-110, 276, -110, 276},
    {-22, -22, -67, -67},
    {362, 0, 126, 0},
    {-8, 0, -8, 0},
    {29, 0, 29, 0},
    {-39, 0, -39, 0},
    {-12, 0, -12, 0},
    {-41, 49, -41, 49},
    {276, -110, 276, -110},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {32, 22, 32, 22},
    {0, 0, 0, 0},
    {14, -293, 14, -293},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-63, -386, -63, -386},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  }
}

```

```

    {0, 0, 0, 0},
    {22, 32, 22, 32},
    {-293, 14, -293, 14},
    {-386, -63, -386, -63},
    {-9, -9, -9, -9},
    {-13, -13, -13, -13},
    {15, 15, 15, 15},
    {15, 15, 15, 15},
  },
},
{
  {
    {6, 0, 6, 0},
    {17, 0, 17, 0},
    {-34, 0, -34, 0},
    {15, 0, 15, 0},
    {52, 0, 96, 0},
    {-16, 87, -16, 87},
    {96, 0, 52, 0},
    {0, 52, 0, 96},
    {-22, -22, -37, -37},
    {-55, -74, -55, -74},
    {-37, -37, -22, -22},
    {0, 96, 0, 52},
    {0, 6, 0, 6},
    {0, 17, 0, 17},
    {0, -34, 0, -34},
    {0, 15, 0, 15},
    {87, -16, 87, -16},
    {-74, -55, -74, -55},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, -15, 0, -15},
    {0, 21, 0, 21},
    {0, -7, 0, -7},
    {0, -40, 0, -40},
    {0, -27, 0, 365},
    {-51, 256, -51, 256},
    {0, 365, 0, -27},
    {-27, 0, 365, 0},
    {-20, -20, 58, 58},
    {-98, 197, -98, 197},
    {58, 58, -20, -20},
    {365, 0, -27, 0},
    {-15, 0, -15, 0},
    {21, 0, 21, 0},
    {-7, 0, -7, 0},
    {-40, 0, -40, 0},
    {256, -51, 256, -51},
    {197, -98, 197, -98},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-2, -13, -2, -13},
    {0, 0, 0, 0},
    {-32, -134, -32, -134},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-135, -64, -135, -64},
    {0, 0, 0, 0},
  },
}

```

```

{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-13, -2, -13, -2},
{-134, -32, -134, -32},
{-64, -135, -64, -135},
{-2, -2, -2, -2},
{-14, -14, -14, -14},
{11, 11, 11, 11},
{-14, -14, -14, -14},
},
},
{
{
{-6, 0, -6, 0},
{-20, 0, -20, 0},
{28, 0, 28, 0},
{-1, 0, -1, 0},
{-38, 0, -70, 0},
{-99, 32, -99, 32},
{-70, 0, -38, 0},
{0, -38, 0, -70},
{9, 9, -138, -138},
{2, 146, 2, 146},
{-138, -138, 9, 9},
{0, -70, 0, -38},
{0, -6, 0, -6},
{0, -20, 0, -20},
{0, 28, 0, 28},
{0, -1, 0, -1},
{32, -99, 32, -99},
{146, 2, 146, 2},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -34, 0, -34},
{0, 26, 0, 26},
{0, 14, 0, 14},
{0, -61, 0, -61},
{0, -1, 0, 414},
{-83, 511, -83, 511},
{0, 414, 0, -1},
{-1, 0, 414, 0},
{-52, -52, 302, 302},
{-72, 480, -72, 480},
{302, 302, -52, -52},
{414, 0, -1, 0},
{-34, 0, -34, 0},
{26, 0, 26, 0},
{14, 0, 14, 0},
{-61, 0, -61, 0},
{511, -83, 511, -83},
{480, -72, 480, -72},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{57, 90, 57, 90},
{0, 0, 0, 0},
{-189, 286, -189, 286},
{0, 0, 0, 0},
{0, 0, 0, 0},

```

```

{0, 0, 0, 0},
{16, 100, 16, 100},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{90, 57, 90, 57},
{286, -189, 286, -189},
{100, 16, 100, 16},
{-25, -25, -25, -25},
{3, 3, 3, 3},
{-3, -3, -3, -3},
{2, 2, 2, 2},
},
},
{
{
{-69, 0, -69, 0},
{23, 0, 23, 0},
{-18, 0, -18, 0},
{-51, 0, -51, 0},
{-45, 0, 77, 0},
{47, 105, 47, 105},
{77, 0, -45, 0},
{0, -45, 0, 77},
{-40, -40, 94, 94},
{-4, 34, -4, 34},
{94, 94, -40, -40},
{0, 77, 0, -45},
{0, -69, 0, -69},
{0, 23, 0, 23},
{0, -18, 0, -18},
{0, -51, 0, -51},
{105, 47, 105, 47},
{34, -4, 34, -4},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 511, 0, 511},
{0, 511, 0, 511},
{0, -61, 0, -61},
{0, 72, 0, 72},
{0, -13, 0, 511},
{28, -58, 28, -58},
{0, 511, 0, -13},
{-13, 0, 511, 0},
{-32, -32, 511, 511},
{-58, 131, -58, 131},
{511, 511, -32, -32},
{511, 0, -13, 0},
{511, 0, 511, 0},
{511, 0, 511, 0},
{-61, 0, -61, 0},
{72, 0, 72, 0},
{-58, 28, -58, 28},
{131, -58, 131, -58},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-10, -222, -10, -222},
{0, 0, 0, 0},

```

```

{-163, -28, -163, -28},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-97, 129, -97, 129},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-222, -10, -222, -10},
{-28, -163, -28, -163},
{129, -97, 129, -97},
{2, 2, 2, 2},
{-88, -88, -88, -88},
{2, 2, 2, 2},
{-6, -6, -6, -6},
},
{
{
{132, 0, 132, 0},
{-72, 0, -72, 0},
{89, 0, 89, 0},
{-285, 0, -285, 0},
{152, 0, -7, 0},
{31, 10, 31, 10},
{-7, 0, 152, 0},
{0, 152, 0, -7},
{65, 65, -139, -139},
{-61, -83, -61, -83},
{-139, -139, 65, 65},
{0, -7, 0, 152},
{0, 132, 0, 132},
{0, -72, 0, -72},
{0, 89, 0, 89},
{0, -285, 0, -285},
{10, 31, 10, 31},
{-83, -61, -83, -61},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -512, 0, -512},
{0, -512, 0, -512},
{0, -402, 0, -402},
{0, -194, 0, -194},
{0, 142, 0, -174},
{20, -22, 20, -22},
{0, -174, 0, 142},
{142, 0, -174, 0},
{94, 94, -55, -55},
{32, 157, 32, 157},
{-55, -55, 94, 94},
{-174, 0, 142, 0},
{-512, 0, -512, 0},
{-512, 0, -512, 0},
{-402, 0, -402, 0},
{-194, 0, -194, 0},
{-22, 20, -22, 20},
{157, 32, 157, 32},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},

```

```

{0, 0, 0, 0},
{-512, 29, -512, 29},
{0, 0, 0, 0},
{10, -23, 10, -23},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-13, -64, -13, -64},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{29, -512, 29, -512},
{-23, 10, -23, 10},
{-64, -13, -64, -13},
{108, 108, 108, 108},
{-2, -2, -2, -2},
{-2, -2, -2, -2},
{-4, -4, -4, -4},
},
},
{
{
{
{4, 0, 4, 0},
{-2, 0, -2, 0},
{6, 0, 6, 0},
{-29, 0, -29, 0},
{17, 0, -23, 0},
{6, -51, 6, -51},
{-23, 0, 17, 0},
{0, 17, 0, -23},
{-37, -37, 206, 206},
{90, 323, 90, 323},
{206, 206, -37, -37},
{0, -23, 0, 17},
{0, 4, 0, 4},
{0, -2, 0, -2},
{0, 6, 0, 6},
{0, -29, 0, -29},
{-51, 6, -51, 6},
{323, 90, 323, 90},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -19, 0, -19},
{0, 0, 0, 0},
{0, -8, 0, -8},
{0, -16, 0, -16},
{0, -33, 0, 19},
{-22, 58, -22, 58},
{0, 19, 0, -33},
{-33, 0, 19, 0},
{20, 20, -60, -60},
{-15, 8, -15, 8},
{-60, -60, 20, 20},
{19, 0, -33, 0},
{-19, 0, -19, 0},
{0, 0, 0, 0},
{-8, 0, -8, 0},
{-16, 0, -16, 0},
{58, -22, 58, -22},
{8, -15, 8, -15},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},

```



```

    {0, 0, 0, 0},
  },
  {
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-12, -12, -12, -12},
    {0, 0, 0, 0},
    {-21, -35, -21, -35},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {17, -59, 17, -59},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-12, -12, -12, -12},
    {-35, -21, -35, -21},
    {-59, 17, -59, 17},
    {-5, -5, -5, -5},
    {-15, -15, -15, -15},
    {-10, -10, -10, -10},
    {-14, -14, -14, -14},
  },
},
{
  {
    {3, 0, 3, 0},
    {24, 0, 24, 0},
    {-12, 0, -12, 0},
    {7, 0, 7, 0},
    {50, 0, -87, 0},
    {15, -1, 15, -1},
    {-87, 0, 50, 0},
    {0, 50, 0, -87},
    {-88, -88, -48, -48},
    {-12, 37, -12, 37},
    {-48, -48, -88, -88},
    {0, -87, 0, 50},
    {0, 3, 0, 3},
    {0, 24, 0, 24},
    {0, -12, 0, -12},
    {0, 7, 0, 7},
    {-1, 15, -1, 15},
    {37, -12, 37, -12},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, -1, 0, -1},
    {0, 1, 0, 1},
    {0, -21, 0, -21},
    {0, 64, 0, 64},
    {0, -24, 0, -357},
    {-3, 30, -3, 30},
    {0, -357, 0, -24},
    {-24, 0, -357, 0},
    {53, 53, 7, 7},
    {-237, -435, -237, -435},
    {7, 7, 53, 53},
    {-357, 0, -24, 0},
    {-1, 0, -1, 0},
    {1, 0, 1, 0},
    {-21, 0, -21, 0},
    {64, 0, 64, 0},
    {30, -3, 30, -3},
    {-435, -237, -435, -237},
  }
}

```

```

{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{44, 32, 44, 32},
{0, 0, 0, 0},
{18, 24, 18, 24},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{50, -112, 50, -112},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{32, 44, 32, 44},
{24, 18, 24, 18},
{-112, 50, -112, 50},
{-25, -25, -25, -25},
{93, 93, 93, 93},
{3, 3, 3, 3},
{9, 9, 9, 9},
},
},
{
{
{3, 0, 3, 0},
{22, 0, 22, 0},
{5, 0, 5, 0},
{-70, 0, -70, 0},
{62, 0, 122, 0},
{41, -77, 41, -77},
{122, 0, 62, 0},
{0, 62, 0, 122},
{-47, -47, 325, 325},
{-123, 230, -123, 230},
{325, 325, -47, -47},
{0, 122, 0, 62},
{0, 3, 0, 3},
{0, 22, 0, 22},
{0, 5, 0, 5},
{0, -70, 0, -70},
{-77, 41, -77, 41},
{230, -123, 230, -123},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -7, 0, -7},
{0, 12, 0, 12},
{0, -19, 0, -19},
{0, 449, 0, 449},
{0, 2, 0, 182},
{-6, 4, -6, 4},
{0, 182, 0, 2},
{2, 0, 182, 0},
{-3, -3, 124, 124},
{-14, 405, -14, 405},
{124, 124, -3, -3},
{182, 0, 2, 0},
{-7, 0, -7, 0},
{12, 0, 12, 0},
{-19, 0, -19, 0},

```

```

{449, 0, 449, 0},
{4, -6, 4, -6},
{405, -14, 405, -14},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-62, -32, -62, -32},
{0, 0, 0, 0},
{-12, -134, -12, -134},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-117, 330, -117, 330},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-32, -62, -32, -62},
{-134, -12, -134, -12},
{330, -117, 330, -117},
{-32, -32, -32, -32},
{-97, -97, -97, -97},
{-62, -62, -62, -62},
{20, 20, 20, 20},
},
},
{
{
{-21, 0, -21, 0},
{-25, 0, -25, 0},
{16, 0, 16, 0},
{-31, 0, -31, 0},
{-58, 0, 59, 0},
{-19, 161, -19, 161},
{59, 0, -58, 0},
{0, -58, 0, 59},
{47, 47, -70, -70},
{-8, 4, -8, 4},
{-70, -70, 47, 47},
{0, 59, 0, -58},
{0, -21, 0, -21},
{0, -25, 0, -25},
{0, 16, 0, 16},
{0, -31, 0, -31},
{161, -19, 161, -19},
{4, -8, 4, -8},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 47, 0, 47},
{0, -512, 0, -512},
{0, -512, 0, -512},
{0, 26, 0, 26},
{0, 0, 0, -473},
{-10, -174, -10, -174},
{0, -473, 0, 0},
{0, 0, -473, 0},
{50, 50, -136, -136},
{43, -512, 43, -512},
{-136, -136, 50, 50},
{-473, 0, 0, 0},

```

```

{47, 0, 47, 0},
{-512, 0, -512, 0},
{-512, 0, -512, 0},
{26, 0, 26, 0},
{-174, -10, -174, -10},
{-512, 43, -512, 43},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{52, 226, 52, 226},
{0, 0, 0, 0},
{112, 178, 112, 178},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{94, -224, 94, -224},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{226, 52, 226, 52},
{178, 112, 178, 112},
{-224, 94, -224, 94},
{-16, -16, -16, -16},
{62, 62, 62, 62},
{-11, -11, -11, -11},
{4, 4, 4, 4},
},
},
{
{
{45, 0, 45, 0},
{-15, 0, -15, 0},
{15, 0, 15, 0},
{-11, 0, -11, 0},
{75, 0, 91, 0},
{-14, -51, -14, -51},
{91, 0, 75, 0},
{0, 75, 0, 91},
{-63, -63, 89, 89},
{-5, 44, -5, 44},
{89, 89, -63, -63},
{0, 91, 0, 75},
{0, 45, 0, 45},
{0, -15, 0, -15},
{0, 15, 0, 15},
{0, -11, 0, -11},
{-51, -14, -51, -14},
{44, -5, 44, -5},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -34, 0, -34},
{0, 511, 0, 511},
{0, -67, 0, -67},
{0, 180, 0, 180},
{0, -94, 0, 495},
{-12, -108, -12, -108},
{0, 495, 0, -94},
{-94, 0, 495, 0},
{-20, -20, 486, 486},

```

```

{-11, 503, -11, 503},
{486, 486, -20, -20},
{495, 0, -94, 0},
{-34, 0, -34, 0},
{511, 0, 511, 0},
{-67, 0, -67, 0},
{180, 0, 180, 0},
{-108, -12, -108, -12},
{503, -11, 503, -11},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{26, 511, 26, 511},
{0, 0, 0, 0},
{-22, -33, -22, -33},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{33, 243, 33, 243},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{511, 26, 511, 26},
{-33, -22, -33, -22},
{243, 33, 243, 33},
{-13, -13, -13, -13},
{-1, -1, -1, -1},
{6, 6, 6, 6},
{5, 5, 5, 5},
},
},
{
{
{-120, 0, -120, 0},
{62, 0, 62, 0},
{-87, 0, -87, 0},
{426, 0, 426, 0},
{-153, 0, -80, 0},
{-16, -28, -16, -28},
{-80, 0, -153, 0},
{0, -153, 0, -80},
{-48, -48, 482, 482},
{72, 95, 72, 95},
{482, 482, -48, -48},
{0, -80, 0, -153},
{0, -120, 0, -120},
{0, 62, 0, 62},
{0, -87, 0, -87},
{0, 426, 0, 426},
{-28, -16, -28, -16},
{95, 72, 95, 72},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 511, 0, 511},
{0, 466, 0, 466},
{0, 238, 0, 238},
{0, 109, 0, 109},
{0, -87, 0, 148},
{-23, -12, -23, -12},

```

```

{0, 148, 0, -87},
{-87, 0, 148, 0},
{-63, -63, 53, 53},
{-21, -113, -21, -113},
{53, 53, -63, -63},
{148, 0, -87, 0},
{511, 0, 511, 0},
{466, 0, 466, 0},
{238, 0, 238, 0},
{109, 0, 109, 0},
{-12, -23, -12, -23},
{-113, -21, -113, -21},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-512, 20, -512, 20},
{0, 0, 0, 0},
{9, -8, 9, -8},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-18, -37, -18, -37},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{20, -512, 20, -512},
{-8, 9, -8, 9},
{-37, -18, -37, -18},
{-5, -5, -5, -5},
{-8, -8, -8, -8},
{54, 54, 54, 54},
{63, 63, 63, 63},
},
},
{
{
{
{6, 0, 6, 0},
{1, 0, 1, 0},
{10, 0, 10, 0},
{-38, 0, -38, 0},
{27, 0, -72, 0},
{58, -53, 58, -53},
{-72, 0, 27, 0},
{0, 27, 0, -72},
{-47, -47, 293, 293},
{-114, 366, -114, 366},
{293, 293, -47, -47},
{0, -72, 0, 27},
{0, 6, 0, 6},
{0, 1, 0, 1},
{0, 10, 0, 10},
{0, -38, 0, -38},
{-53, 58, -53, 58},
{366, -114, 366, -114},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 18, 0, 18},

```

```

{0, -18, 0, -18},
{0, 9, 0, 9},
{0, 9, 0, 9},
{0, 20, 0, -18},
{27, -20, 27, -20},
{0, -18, 0, 20},
{20, 0, -18, 0},
{9, 9, 140, 140},
{-22, 35, -22, 35},
{140, 140, 9, 9},
{-18, 0, 20, 0},
{18, 0, 18, 0},
{-18, 0, -18, 0},
{9, 0, 9, 0},
{9, 0, 9, 0},
{-20, 27, -20, 27},
{35, -22, 35, -22},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-2, -16, -2, -16},
{0, 0, 0, 0},
{-11, 45, -11, 45},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{24, 154, 24, 154},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-16, -2, -16, -2},
{45, -11, 45, -11},
{154, 24, 154, 24},
{6, 6, 6, 6},
{16, 16, 16, 16},
{-11, -11, -11, -11},
{-9, -9, -9, -9},
},
},
{
{
{-3, 0, -3, 0},
{30, 0, 30, 0},
{-32, 0, -32, 0},
{-19, 0, -19, 0},
{46, 0, -26, 0},
{48, -55, 48, -55},
{-26, 0, 46, 0},
{0, 46, 0, -26},
{-105, -105, 6, 6},
{-88, 345, -88, 345},
{6, 6, -105, -105},
{0, -26, 0, 46},
{0, -3, 0, -3},
{0, 30, 0, 30},
{0, -32, 0, -32},
{0, -19, 0, -19},
{-55, 48, -55, 48},
{345, -88, 345, -88},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},

```

```

},
{
  {0, 7, 0, 7},
  {0, -10, 0, -10},
  {0, 2, 0, 2},
  {0, -8, 0, -8},
  {0, 19, 0, 256},
  {11, -18, 11, -18},
  {0, 256, 0, 19},
  {19, 0, 256, 0},
  {3, 3, 4, 4},
  {-9, 299, -9, 299},
  {4, 4, 3, 3},
  {256, 0, 19, 0},
  {7, 0, 7, 0},
  {-10, 0, -10, 0},
  {2, 0, 2, 0},
  {-8, 0, -8, 0},
  {-18, 11, -18, 11},
  {299, -9, 299, -9},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
},
{
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-31, -41, -31, -41},
  {0, 0, 0, 0},
  {-26, 22, -26, 22},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-12, 279, -12, 279},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-41, -31, -41, -31},
  {22, -26, 22, -26},
  {279, -12, 279, -12},
  {-66, -66, -66, -66},
  {-10, -10, -10, -10},
  {6, 6, 6, 6},
  {8, 8, 8, 8},
},
},
{
  {
    {2, 0, 2, 0},
    {11, 0, 11, 0},
    {0, 0, 0, 0},
    {-59, 0, -59, 0},
    {35, 0, 100, 0},
    {1, -7, 1, -7},
    {100, 0, 35, 0},
    {0, 35, 0, 100},
    {-13, -13, 185, 185},
    {-49, 324, -49, 324},
    {185, 185, -13, -13},
    {0, 100, 0, 35},
    {0, 2, 0, 2},
    {0, 11, 0, 11},
    {0, 0, 0, 0},
    {0, -59, 0, -59},
    {-7, 1, -7, 1},
    {324, -49, 324, -49},
    {0, 0, 0, 0},
  }
}

```



```

    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
},
{
    {0, -21, 0, -21},
    {0, 33, 0, 33},
    {0, -36, 0, -36},
    {0, -1, 0, -1},
    {0, 232, 0, 440},
    {17, -81, 17, -81},
    {0, 440, 0, 232},
    {232, 0, 440, 0},
    {-79, -79, 150, 150},
    {-112, 361, -112, 361},
    {150, 150, -79, -79},
    {440, 0, 232, 0},
    {-21, 0, -21, 0},
    {33, 0, 33, 0},
    {-36, 0, -36, 0},
    {-1, 0, -1, 0},
    {-81, 17, -81, 17},
    {361, -112, 361, -112},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
},
{
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-47, -43, -47, -43},
    {0, 0, 0, 0},
    {-61, 62, -61, 62},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {84, 154, 84, 154},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-43, -47, -43, -47},
    {62, -61, 62, -61},
    {154, 84, 154, 84},
    {-4, -4, -4, -4},
    {0, 0, 0, 0},
    {-68, -68, -68, -68},
    {-44, -44, -44, -44},
},
{
    {
        {-23, 0, -23, 0},
        {-9, 0, -9, 0},
        {0, 0, 0, 0},
        {-4, 0, -4, 0},
        {-65, 0, -19, 0},
        {5, 108, 5, 108},
        {-19, 0, -65, 0},
        {0, -65, 0, -19},
        {-18, -18, -15, -15},
        {47, 118, 47, 118},
        {-15, -15, -18, -18},
        {0, -19, 0, -65},
        {0, -23, 0, -23},
        {0, -9, 0, -9},
        {0, 0, 0, 0},
        {0, -4, 0, -4},
    }
}

```

```

{108, 5, 108, 5},
{118, 47, 118, 47},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -512, 0, -512},
{0, -512, 0, -512},
{0, -512, 0, -512},
{0, 97, 0, 97},
{0, 15, 0, -448},
{-18, -347, -18, -347},
{0, -448, 0, 15},
{15, 0, -448, 0},
{49, 49, -158, -158},
{76, -108, 76, -108},
{-158, -158, 49, 49},
{-448, 0, 15, 0},
{-512, 0, -512, 0},
{-512, 0, -512, 0},
{-512, 0, -512, 0},
{97, 0, 97, 0},
{-347, -18, -347, -18},
{-108, 76, -108, 76},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{40, 166, 40, 166},
{0, 0, 0, 0},
{-10, 92, -10, 92},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{104, 28, 104, 28},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{166, 40, 166, 40},
{92, -10, 92, -10},
{28, 104, 28, 104},
{-1, -1, -1, -1},
{57, 57, 57, 57},
{-33, -33, -33, -33},
{-5, -5, -5, -5},
},
},
{
{
{-55, 0, -55, 0},
{20, 0, 20, 0},
{-11, 0, -11, 0},
{-38, 0, -38, 0},
{-44, 0, 12, 0},
{25, 53, 25, 53},
{12, 0, -44, 0},
{0, -44, 0, 12},
{-5, -5, 70, 70},
{34, 72, 34, 72},
{70, 70, -5, -5},
{0, 12, 0, -44},
{0, -55, 0, -55},

```

```

{0, 20, 0, 20},
{0, -11, 0, -11},
{0, -38, 0, -38},
{53, 25, 53, 25},
{72, 34, 72, 34},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 511, 0, 511},
{0, 511, 0, 511},
{0, 511, 0, 511},
{0, -45, 0, -45},
{0, 352, 0, 443},
{8, -18, 8, -18},
{0, 443, 0, 352},
{352, 0, 443, 0},
{-50, -50, 374, 374},
{-8, 468, -8, 468},
{374, 374, -50, -50},
{443, 0, 352, 0},
{511, 0, 511, 0},
{511, 0, 511, 0},
{511, 0, 511, 0},
{-45, 0, -45, 0},
{-18, 8, -18, 8},
{468, -8, 468, -8},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-10, 318, -10, 318},
{0, 0, 0, 0},
{105, 18, 105, 18},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{206, -378, 206, -378},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{318, -10, 318, -10},
{18, 105, 18, 105},
{-378, 206, -378, 206},
{8, 8, 8, 8},
{60, 60, 60, 60},
{-20, -20, -20, -20},
{2, 2, 2, 2},
},
},
{
{
{-104, 0, -104, 0},
{62, 0, 62, 0},
{-87, 0, -87, 0},
{392, 0, 392, 0},
{-154, 0, -80, 0},
{-2, -67, -2, -67},
{-80, 0, -154, 0},
{0, -154, 0, -80},
{-47, -47, 156, 156},
{89, 136, 89, 136},

```

```

    {156, 156, -47, -47},
    {0, -80, 0, -154},
    {0, -104, 0, -104},
    {0, 62, 0, 62},
    {0, -87, 0, -87},
    {0, 392, 0, 392},
    {-67, -2, -67, -2},
    {136, 89, 136, 89},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
},
{
    {0, -512, 0, -512},
    {0, -512, 0, -512},
    {0, -282, 0, -282},
    {0, -103, 0, -103},
    {0, 149, 0, -190},
    {23, -9, 23, -9},
    {0, -190, 0, 149},
    {149, 0, -190, 0},
    {87, 87, -62, -62},
    {28, 94, 28, 94},
    {-62, -62, 87, 87},
    {-190, 0, 149, 0},
    {-512, 0, -512, 0},
    {-512, 0, -512, 0},
    {-282, 0, -282, 0},
    {-103, 0, -103, 0},
    {-9, 23, -9, 23},
    {94, 28, 94, 28},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
},
{
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-512, 22, -512, 22},
    {0, 0, 0, 0},
    {12, -18, 12, -18},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-21, -16, -21, -16},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {22, -512, 22, -512},
    {-18, 12, -18, 12},
    {-16, -21, -16, -21},
    {41, 41, 41, 41},
    {34, 34, 34, 34},
    {48, 48, 48, 48},
    {-3, -3, -3, -3},
},
},
{
    {
        {
            {16, 0, 16, 0},
            {-14, 0, -14, 0},
            {24, 0, 24, 0},
            {-55, 0, -55, 0},
            {35, 0, 7, 0},

```

```

{59, 46, 59, 46},
{7, 0, 35, 0},
{0, 35, 0, 7},
{-55, -55, -14, -14},
{-145, 55, -145, 55},
{-14, -14, -55, -55},
{0, 7, 0, 35},
{0, 16, 0, 16},
{0, -14, 0, -14},
{0, 24, 0, 24},
{0, -55, 0, -55},
{46, 59, 46, 59},
{55, -145, 55, -145},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -11, 0, -11},
{0, -2, 0, -2},
{0, 7, 0, 7},
{0, 7, 0, 7},
{0, -19, 0, -2},
{-33, -60, -33, -60},
{0, -2, 0, -19},
{-19, 0, -2, 0},
{-2, -2, 8, 8},
{11, 89, 11, 89},
{8, 8, -2, -2},
{-2, 0, -19, 0},
{-11, 0, -11, 0},
{-2, 0, -2, 0},
{7, 0, 7, 0},
{7, 0, 7, 0},
{-60, -33, -60, -33},
{89, 11, 89, 11},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-5, 4, -5, 4},
{0, 0, 0, 0},
{-14, -78, -14, -78},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-20, -133, -20, -133},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{4, -5, 4, -5},
{-78, -14, -78, -14},
{-133, -20, -133, -20},
{3, 3, 3, 3},
{35, 35, 35, 35},
{-4, -4, -4, -4},
{-5, -5, -5, -5},
},
{
{
{2, 0, 2, 0},
{-17, 0, -17, 0},

```

```

{15, 0, 15, 0},
{1, 0, 1, 0},
{63, 0, 54, 0},
{-55, 25, -55, 25},
{54, 0, 63, 0},
{0, 63, 0, 54},
{44, 44, -101, -101},
{65, -512, 65, -512},
{-101, -101, 44, 44},
{0, 54, 0, 63},
{0, 2, 0, 2},
{0, -17, 0, -17},
{0, 15, 0, 15},
{0, 1, 0, 1},
{25, -55, 25, -55},
{-512, 65, -512, 65},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -7, 0, -7},
{0, -9, 0, -9},
{0, -10, 0, -10},
{0, 65, 0, 65},
{0, -44, 0, -289},
{-65, 53, -65, 53},
{0, -289, 0, -44},
{-44, 0, -289, 0},
{43, 43, 41, 41},
{61, -107, 61, -107},
{41, 41, 43, 43},
{-289, 0, -44, 0},
{-7, 0, -7, 0},
{-9, 0, -9, 0},
{-10, 0, -10, 0},
{65, 0, 65, 0},
{53, -65, 53, -65},
{-107, 61, -107, 61},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-27, -37, -27, -37},
{0, 0, 0, 0},
{10, -33, 10, -33},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-73, -32, -73, -32},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-37, -27, -37, -27},
{-33, 10, -33, 10},
{-32, -73, -32, -73},
{3, 3, 3, 3},
{-6, -6, -6, -6},
{27, 27, 27, 27},
{-8, -8, -8, -8},
},
},
{

```

```

{
  {3, 0, 3, 0},
  {-21, 0, -21, 0},
  {48, 0, 48, 0},
  {-9, 0, -9, 0},
  {-4, 0, 29, 0},
  {-38, -368, -38, -368},
  {29, 0, -4, 0},
  {0, -4, 0, 29},
  {-65, -65, -308, -308},
  {124, -93, 124, -93},
  {-308, -308, -65, -65},
  {0, 29, 0, -4},
  {0, 3, 0, 3},
  {0, -21, 0, -21},
  {0, 48, 0, 48},
  {0, -9, 0, -9},
  {-368, -38, -368, -38},
  {-93, 124, -93, 124},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
},
{
  {0, -12, 0, -12},
  {0, -14, 0, -14},
  {0, 10, 0, 10},
  {0, -17, 0, -17},
  {0, 52, 0, 47},
  {-56, 50, -56, 50},
  {0, 47, 0, 52},
  {52, 0, 47, 0},
  {-195, -195, 33, 33},
  {45, 114, 45, 114},
  {33, 33, -195, -195},
  {47, 0, 52, 0},
  {-12, 0, -12, 0},
  {-14, 0, -14, 0},
  {10, 0, 10, 0},
  {-17, 0, -17, 0},
  {50, -56, 50, -56},
  {114, 45, 114, 45},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
},
{
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-56, -83, -56, -83},
  {0, 0, 0, 0},
  {33, 47, 33, 47},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-116, 27, -116, 27},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {0, 0, 0, 0},
  {-83, -56, -83, -56},
  {47, 33, 47, 33},
  {27, -116, 27, -116},
  {9, 9, 9, 9},
  {-6, -6, -6, -6},
  {-4, -4, -4, -4},
  {-13, -13, -13, -13},
}

```

```

},
{
{
{9, 0, 9, 0},
{107, 0, 107, 0},
{-36, 0, -36, 0},
{14, 0, 14, 0},
{8, 0, 101, 0},
{90, 142, 90, 142},
{101, 0, 8, 0},
{0, 8, 0, 101},
{16, 16, 222, 222},
{-42, -62, -42, -62},
{222, 222, 16, 16},
{0, 101, 0, 8},
{0, 9, 0, 9},
{0, 107, 0, 107},
{0, -36, 0, -36},
{0, 14, 0, 14},
{142, 90, 142, 90},
{-62, -42, -62, -42},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, -512, 0, -512},
{0, -512, 0, -512},
{0, 23, 0, 23},
{0, 8, 0, 8},
{0, 7, 0, -142},
{135, -358, 135, -358},
{0, -142, 0, 7},
{7, 0, -142, 0},
{39, 39, -254, -254},
{125, -365, 125, -365},
{-254, -254, 39, 39},
{-142, 0, 7, 0},
{-512, 0, -512, 0},
{-512, 0, -512, 0},
{23, 0, 23, 0},
{8, 0, 8, 0},
{-358, 135, -358, 135},
{-365, 125, -365, 125},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
},
{
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-79, -102, -79, -102},
{0, 0, 0, 0},
{152, 89, 152, 89},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-41, -4, -41, -4},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{0, 0, 0, 0},
{-102, -79, -102, -79},
{89, 152, 89, 152},
{-4, -41, -4, -41},
{1, 1, 1, 1},

```



```

    {-18, -18, -18, -18},
    {4, 4, 4, 4},
    {4, 4, 4, 4},
  },
},
{
  {
    {46, 0, 46, 0},
    {-12, 0, -12, 0},
    {14, 0, 14, 0},
    {-9, 0, -9, 0},
    {60, 0, 71, 0},
    {-115, -91, -115, -91},
    {71, 0, 60, 0},
    {0, 60, 0, 71},
    {23, 23, -4, -4},
    {-51, -52, -51, -52},
    {-4, -4, 23, 23},
    {0, 71, 0, 60},
    {0, 46, 0, 46},
    {0, -12, 0, -12},
    {0, 14, 0, 14},
    {0, -9, 0, -9},
    {-91, -115, -91, -115},
    {-52, -51, -52, -51},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, -512, 0, -512},
    {0, 41, 0, 41},
    {0, -512, 0, -512},
    {0, -39, 0, -39},
    {0, 17, 0, -462},
    {-512, 0, -512, 0},
    {0, -462, 0, 17},
    {17, 0, -462, 0},
    {41, 41, -97, -97},
    {106, -174, 106, -174},
    {-97, -97, 41, 41},
    {-462, 0, 17, 0},
    {-512, 0, -512, 0},
    {41, 0, 41, 0},
    {-512, 0, -512, 0},
    {-39, 0, -39, 0},
    {0, -512, 0, -512},
    {-174, 106, -174, 106},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-48, -194, -48, -194},
    {0, 0, 0, 0},
    {356, -34, 356, -34},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-14, 129, -14, 129},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-194, -48, -194, -48},
  }
}

```

```

    {-34, 356, -34, 356},
    {129, -14, 129, -14},
    {7, 7, 7, 7},
    {-31, -31, -31, -31},
    {7, 7, 7, 7},
    {2, 2, 2, 2},
  },
},
{
  {
    {-89, 0, -89, 0},
    {38, 0, 38, 0},
    {-34, 0, -34, 0},
    {6, 0, 6, 0},
    {-105, 0, -87, 0},
    {263, -35, 263, -35},
    {-87, 0, -105, 0},
    {0, -105, 0, -87},
    {-75, -75, 106, 106},
    {50, 113, 50, 113},
    {106, 106, -75, -75},
    {0, -87, 0, -105},
    {0, -89, 0, -89},
    {0, 38, 0, 38},
    {0, -34, 0, -34},
    {0, 6, 0, 6},
    {-35, 263, -35, 263},
    {113, 50, 113, 50},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, 511, 0, 511},
    {0, 511, 0, 511},
    {0, 281, 0, 281},
    {0, 104, 0, 104},
    {0, -189, 0, 511},
    {274, -22, 274, -22},
    {0, 511, 0, -189},
    {-189, 0, 511, 0},
    {-110, -110, 148, 148},
    {-8, -180, -8, -180},
    {148, 148, -110, -110},
    {511, 0, -189, 0},
    {511, 0, 511, 0},
    {511, 0, 511, 0},
    {281, 0, 281, 0},
    {104, 0, 104, 0},
    {-22, 274, -22, 274},
    {-180, -8, -180, -8},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  },
  {
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-13, -33, -13, -33},
    {0, 0, 0, 0},
    {511, 29, 511, 29},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {11, 18, 11, 18},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {0, 0, 0, 0},
  }
}

```

```

    {0, 0, 0, 0},
    {0, 0, 0, 0},
    {-33, -13, -33, -13},
    {29, 511, 29, 511},
    {18, 11, 18, 11},
    {12, 12, 12, 12},
    {12, 12, 12, 12},
    {-69, -69, -69, -69},
    {-10, -10, -10, -10},
  },
},
},
}

```

```

Gdf_Bias[6][6][3] =
{
  {
    {1925, -13583, 11215},
    {-3674, -1502, 8347},
    {-3652, -2643, 363},
    {-2178, -2054, -3486},
    {-1328, -2508, 2097},
    {-2124, -2024, -3121},
  },
  {
    {-1067, -432, -4},
    {9262, -3515, -3754},
    {-7059, 6034, -7748},
    {-6571, 91, 3451},
    {-5335, -1251, 12069},
    {-4763, 1391, -4988},
  },
  {
    {1762, -4143, 417},
    {-5345, 1826, -255},
    {-208, -3312, 9374},
    {-2768, -284, 264},
    {-1907, -6576, -1333},
    {-2787, -2222, 1078},
  },
  {
    {17278, 1136, -976},
    {-3596, 335, -11414},
    {-10642, -8146, -42},
    {-1286, -626, 1415},
    {-3309, -2694, -3776},
    {-2461, -3889, -2859},
  },
  {
    {10217, -10258, -835},
    {3431, -3551, -6685},
    {-1138, -2845, 12088},
    {-847, -1156, 295},
    {-2479, -3801, -737},
    {-3460, -951, -3323},
  },
  {
    {-12046, 10279, -209},
    {9790, -177, -6841},
    {-3332, 12423, -2475},
    {-2864, -4263, -7763},
    {-3309, -2744, -5688},
    {908, -3477, -929},
  },
}
}

```

```

Gdf_Intra_Error[ 6 ][ 4096 ] =
{
  { 0, -1, -1, -1, -1, -1, -1, -1,

```

```

-1, -1, -1, 0, 0, 0, 0, 0,
0, -1, -1, -1, -1, -1, -1, -1,
-1, -1, 0, 0, 0, 0, 0, 0,
0, -1, -1, -1, -1, -1, -1, -1,
-1, -1, 0, 0, 0, 0, 0, 0,
0, -1, -1, -1, -1, -1, -1, -1,
-1, -1, 0, 0, 0, 0, 0, 0,
0, -1, -1, -1, -1, -1, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -2, -1, -1, -1,
-1, 0, 0, 0, 0, 0, 0, -1,
-1, -1, -2, -2, -2, -1, -1, -1,
-1, 0, 0, 0, 0, 0, 0, -1,
-2, -2, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, -1, -1,
-2, -2, -2, -2, -2, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -2, -2, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -2, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -1, -1, -1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 1,
-3, -2, -2, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-3, -2, -2, -2, -1, -1, -1, 0,
0, 0, 0, 1, 0, 0, 0, 0,
-3, -2, -2, -2, -1, -1, -1, 0,
0, 0, 1, 0, 0, 0, 0, 0,
1, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, 0, 0, 0, 0, 0,
1, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, 0, 0, 0, 0, 0,
1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, 0, 0, 0, 0, 0,
0, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, 0, 0, 0, 0, -1,
0, -1, -1, -1, -1, -1, -1, -1,
-1, -1, 0, 0, 0, 0, 0, -1,
-1, -1, 0, 0, 0, 0, -1, -1,
-1, -2, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, -1, -1,
-1, -2, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, -1, -1,
-2, -2, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, 0, -1,
-2, -2, -2, -2, -2, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -2, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -1, -1, -1, 0, 0,
0, 0, 0, 1, 0, 0, 0, 0,
-3, -3, -2, -2, -1, -1, -1, 0,
0, 0, 1, 0, 0, 0, 0, 0,
1, 1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, 0, 0, 0, 0,
1, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, 0, 0, 0, -1,
1, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, 0, 0, -1,
1, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, 0, 0, -1, -1,

```

```

1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, 0, 0, -1, -1, -1,
0, -1, -2, -2, -2, -2, -1, -1,
-1, -1, 0, 0, 0, -1, -1, -1,
0, -2, -2, -2, -2, -2, -2, -1,
-1, -1, 0, 0, 0, 0, -1, -1,
-1, -2, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, -1, -1,
-1, -2, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, 0, -1,
-1, -2, -2, -2, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -2, -2, -2, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -2, -2, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -2, -1, -1, -1, 0,
0, 0, 0, 1, 0, 0, 0, 0,
-2, -2, -2, -2, -1, -1, -1, 0,
0, 0, 1, 0, 0, 0, 0, 0,
1, 1, 0, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, 0, 0, -1,
1, 1, 0, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, 0, -1,
1, 1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, -1, -2, -2, -2, -2, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, 0, 0, 0, -1, -1, -1,
0, -1, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, -1, -1,
0, -1, -2, -2, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, -1,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, -1,
0, -1, -2, -2, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -2, -2, -1, -1, -1, -1, 0,
0, 0, 0, 1, 0, 0, 0, 0,
-2, -2, -2, -2, -1, -1, -1, 0,
0, 0, 1, 0, 0, 0, 0, 0,
1, 1, 1, 0, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
1, 1, 0, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, -1, -2, -2, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -2, -2, -1, -1, -1,
-1, -1, -1, 0, 0, -1, -1, -1,
-1, -1, -1, -2, -2, -1, -1, -1,
-1, -1, 0, 0, 0, -1, -1, -1,
0, 0, -1, -1, -1, -1, -1,

```

```

-1, 0, 0, 0, 0, 0, -1, -1,
0, 0, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, -1,
0, 0, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, 0, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -2, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -2, -1, -1, -1, -1, 0,
0, 0, 0, 1, 0, 0, 0, 0,
-2, -2, -2, -2, -1, -1, -1, 0,
0, 0, 1, 0, 0, 0, 0, 0,
1, 1, 0, 0, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, -1, -2, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, -1, -1, -1, -1, -1,
-1, -1, -1, 0, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, 0, 0, 0, -1, -1, -1,
0, 0, 0, -1, -1, -1, -1, -1,
-1, 0, 0, 0, 0, 0, -1, -1,
0, 0, 0, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, 0, 0, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 0, 0, 1, 0, 0, 0, 0,
-2, -1, -2, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, -1, -1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,

```

[illegible]

```

0, 0, 0, 0, 1, 1, 1, 1,
-1, -1, -1, 0, 0, 0, 0, 0,
0, 0, 0, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
0, 0, -1, -1, -1, -1, -1, -1,
1, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, -1,
1, 0, 0, 0, 0, 0, 0, 0,
0, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, -1, -1, -1,
0, 0, 0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 1, 1, 1, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 1, 1, 1, 1, 1, 1, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 1, 1, 1, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
-1, -1, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, -1, 0, 0, 0,
1, 1, 1, 1, 0, 0, 0, 0,
0, -1, -1, -1, -1, -1, 0, 0,
1, 1, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, -1, -1, 0,
0, 0, 0, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 0, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 1, 1, 0, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
-1, -1, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, -1, -1, -1, 0, 0,
1, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, -1, -1, 0, 0,

```



```

1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 1,
1, 1, 1, 1, 1, 1, 2, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 2, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 2, 2, 2, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 2, 2, 2, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 1, 1, 0, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 2, 2, 2,
2, 1, 1, 1, 0, 0, 0, 1,
1, 1, 1, 1, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 2, 2, 2, 2, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 1, 1, 0,
0, 1, 1, 1, 1, 1, 1, 1,
-1, 0, 0, 0, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 0,
1, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 2, 2, 2, 2,
1, 1, 1, 0, 0, 0, 0, 0,
1, 1, 1, 2, 2, 2, 2, 2,
2, 1, 1, 1, 0, 0, 0, 1,
1, 1, 2, 2, 2, 2, 2,

```

```

2, 1, 1, 1, 1, 1, 1, 1,
1, 2, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 2, 2, 2, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 1, 1, 0,
0, 1, 1, 1, 1, 1, 1, 1,
-1, 0, 0, 0, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 0,
1, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 2, 2, 2, 2,
2, 1, 1, 0, 0, 0, 0, 0,
1, 1, 1, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 1,
1, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
0, 1, 1, 1, 1, 1, 1, 1,
1, 0, 1, 1, 1, 1, 1, 1,
0, 0, 0, 1, 1, 1, 1, 1,
0, 0, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 1, 1, 1,
0, 1, 1, 1, 1, 1, 1, 1,
-1, 0, 0, 0, 0, 0, 0, 1,
0, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 2, 1,
1, 1, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 2, 2, 2, 2,
2, 1, 1, 0, 0, 0, 0, 0,
1, 1, 1, 2, 2, 2, 2, 2,
2, 1, 1, 0, 0, 0, 0, 0,
1, 2, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 0, 0, 0, 1,
2, 2, 2, 2, 2, 2, 2, 1,
1, 1, 1, 1, 1, 0, 1, 1,
2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 0, 0, 1, 1,
2, 2, 2, 2, 2, 1, 1, 1,
1, 1, 1, 1, 0, 1, 1, 1,

```

```

2, 2, 2, 2, 2, 1, 1, 1,
1, 1, 1, 0, 0, 1, 1, 1,
1, 1, 2, 2, 1, 1, 1, 1,
1, 1, 0, 0, 0, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 1, 1, 1, 1,
0, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 1, 1, 1, 1,
0, 0, 0, 1, 1, 1, 1, 1,
1, 0, 0, 0, 1, 1, 1, 1,
0, 0, 0, 0, 0, 1, 1, 1,
1, 0, 0, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 1,
0, 0, 1, 1, 1, 1, 1, 1 },
{ -1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 1, 1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-2, -2, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-2, -2, -2, -2, -1, -1, -1, -1,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -3, -3, -3, -3, -3, -2, -2,
-2, -2, -2, -2, -2, -3, -3, -3,
-3, -3, -3, -3, -3, -3, -3, -3,
-2, -2, -3, -3, -3, -3, -3, -3,
-3, -3, -3, -4, -4, -3, -3, -3,
-2, -2, -3, -3, -3, -3, -3, -3,
-3, -4, -4, -4, -4, -4, -3, -3,
-2, -3, -3, -3, -3, -3, -3, -3,
-4, -4, -4, -4, -3, -3, -3, -3,
-2, -3, -3, -3, -3, -3, -3, -3,
-3, -3, -3, -3, -3, -3, -2, -3,
-2, -2, -2, -2, -2, -3, -3, -3,
-3, -3, -3, -2, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1,
-2, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-2, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -2, -2, -2, -2, -2,
-2, -2, -2, -1, -1, -1, -1, -1,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -3, -3, -3, -2, -2,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -3, -3, -3, -3, -3, -3, -3,
-2, -2, -2, -2, -2, -2, -3, -3,
-3, -3, -3, -3, -3, -3, -3, -3,
-2, -2, -2, -2, -2, -3, -3, -3,
-3, -3, -3, -4, -3, -3, -3, -3,
-2, -2, -2, -2, -2, -3, -3, -3,
-3, -3, -3, -3, -3, -3, -2, -3,
-2, -2, -2, -2, -2, -2, -2, -3,
-3, -3, -3, -2, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -2, -2,

```

```

-2, -2, -2, -2, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0,
0, 1, 1, 1, 1, 1, 2, 2,
-2, -2, -2, -2, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -1, -1, -1, -1, -1,
-1, 0, 0, 0, 0, -1, -1, 0,
-2, -2, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -2, -2, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-2, -2, -2, -2, -2, -2, -2, -2,
-1, -1, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -3, -3, -3, -3, -3,
-1, -2, -2, -2, -2, -2, -2, -2,
-2, -3, -3, -3, -3, -3, -3, -3,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -3, -3, -3, -3, -3, -2, -2,
-2, -2, -2, -2, -2, -2, -2, -2,
-2, -3, -3, -3, -2, -2, -2, -2,
-1, -1, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -2, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, -1,
-1, -1, 0, 0, 0, 1, 1, 1,
1, 0, 0, 1, 1, 0, 0, 0,
0, 0, 0, 1, 1, 1, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 2, 2, 2, 3,
-2, -2, -2, -2, -2, -2, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -2, -2, -1, -1, -1,
-1, 0, 0, 0, 0, -1, -1, -1,
-2, -2, -2, -2, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-2, -2, -2, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -2, -2, -2,
-2, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -2, -2, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -2, -2, -2, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -2,
-2, -2, -2, -2, -3, -3, -3, -2,
-1, -1, -2, -2, -2, -2, -2, -2,
-2, -2, -3, -3, -2, -2, -2, -2,
-1, -1, -1, -1, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, 0, 0, 0,
0, 0, 0, 0, 0, 0, -1, -1,
-1, -1, 0, 0, 0, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 2, 2, 2, 3, 3,

```

```

-3, -3, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-3, -2, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, -1, -1, -1,
-2, -2, -2, -2, -2, -2, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -2,
-2, -2, -2, -2, -2, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -2, -2,
-2, -2, -2, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -2, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -2, -2, -2, -2, -2,
-1, -1, -2, -2, -2, -2, -2, -2,
0, -1, -1, -1, -1, -1, -1, -1,
-1, -2, -2, -2, -2, -1, -1, -1,
0, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, 0, 0, 0, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 2, 3, 3,
1, 1, 2, 2, 2, 2, 2, 1,
2, 2, 2, 2, 2, 3, 4, 4,
-3, -3, -3, -3, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-3, -3, -3, -3, -2, -2, -2, -1,
-1, 0, 0, 0, 0, -1, -1, -1,
-3, -3, -3, -3, -2, -2, -2, -1,
-1, -1, 0, 0, -1, -1, -1, -2,
-3, -3, -3, -2, -2, -2, -1, -1,
-1, -1, -1, -1, -1, -1, -2, -2,
-2, -2, -2, -2, -2, -1, -1, -1,
-1, -1, -1, -1, -1, -2, -2, -2,
-2, -2, -2, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -2, -2, -2, -3,
-1, -1, -1, -1, -1, 0, 0, 0,
0, -1, -1, -1, -2, -2, -2, -3,
-1, -1, -1, -1, -1, 0, 0, 0,
-1, -1, -1, -1, -2, -2, -2, -2,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, -1, -1,
-1, -1, -1, -1, -1, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, 0, 0, 0, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 2,
1, 1, 1, 0, 0, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 2, 2, 3, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 3, 3, 4,
1, 1, 1, 2, 1, 1, 1, 1,
1, 2, 2, 2, 2, 3, 4, 4,
-4, -3, -3, -3, -3, -2, -2, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-3, -3, -3, -3, -3, -2, -2, -1,
-1, -1, 0, 0, 0, -1, -1, -1,
-3, -3, -3, -3, -3, -2, -2, -1,
-1, -1, 0, 0, -1, -1, -1, -1,
-3, -3, -3, -3, -2, -2, -2, -1,

```

```

-1, -1, -1, -1, -1, -1, -2, -2,
-3, -3, -2, -2, -2, -2, -1, -1,
-1, -1, -1, -1, -1, -2, -2, -2,
-2, -2, -2, -2, -2, -1, -1, -1,
0, 0, -1, -1, -2, -2, -2, -2,
-1, -1, -2, -1, -1, -1, 0, 0,
0, 0, -1, -1, -2, -2, -3, -3,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, -1, -1, -1, -2, -2, -2,
1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 0, 0, -1, -1,
1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 0, 1, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 1, 1, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 3, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 2, 2, 3, 4,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 3, 3, 4,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 2, 2, 2, 3, 4, 4,
-4, -4, -4, -3, -3, -3, -2, -1,
-1, -1, 0, 0, 0, 0, 0, 0,
-4, -4, -4, -3, -3, -3, -2, -1,
-1, -1, 0, 0, -1, -1, -1, -1,
-3, -3, -3, -3, -3, -3, -2, -2,
-1, -1, 0, -1, -1, -1, -1, -1,
-3, -3, -3, -3, -3, -2, -2, -1,
-1, -1, 0, -1, -1, -1, -1, -1,
-3, -3, -3, -3, -2, -2, -2, -1,
-1, 0, 0, -1, -1, -1, -1, -2,
-2, -2, -2, -2, -2, -2, -1, -1,
0, 0, 0, -1, -1, -2, -2, -2,
-1, -1, -2, -2, -1, -1, -1, -1,
0, 0, 0, -1, -1, -2, -2, -2,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, -1, -1, -2, -2, -2,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 2, 2, 3, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 2, 2, 3, 4,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 3, 3, 4,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 3, 3, 4,
-4, -4, -4, -3, -3, -3, -2, -2,
-1, -1, -1, 0, 0, -1, -1, -1,
-4, -4, -4, -3, -3, -3, -2, -2,
-1, -1, 0, 0, -1, -1, -1, -1,
-4, -4, -3, -3, -3, -3, -2, -2,
-1, -1, 0, -1, -1, -1, -1, -1,
-3, -3, -3, -3, -3, -2, -2, -1,
-1, -1, 0, -1, -1, -1, -1, -1,
-3, -3, -3, -3, -2, -2, -2, -1,
-1, 0, 0, 0, -1, -1, -1, -1,
-2, -2, -2, -2, -2, -2, -2, -1,
-1, 0, 0, 0, 0, -1, -1, -1,
0, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, -1, -1, -1,

```

```

1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, -1, -1, -1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, -1,
2, 2, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 1,
2, 2, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 2, 2, 3, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 2, 2, 3, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 2, 3, 3,
-4, -4, -4, -3, -3, -3, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-4, -4, -3, -3, -3, -3, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-3, -3, -3, -3, -3, -2, -2, -1,
-1, -1, 0, 0, -1, -1, -1, -1,
-3, -3, -3, -3, -3, -2, -2, -1,
-1, 0, 0, 0, 0, 0, -1, -1,
-2, -2, -2, -2, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
2, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
2, 2, 2, 2, 2, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
2, 2, 2, 2, 2, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 1,
2, 2, 2, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 3,
1, 1, 1, 1, 2, 2, 2, 3,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 2, 2, 3, 3,
-4, -4, -3, -3, -3, -3, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-4, -3, -3, -3, -3, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-3, -3, -3, -3, -3, -2, -2, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-2, -2, -2, -2, -2, -2, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 1, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 0,
1, 1, 0, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
2, 1, 1, 1, 1, 1, 1, 1,
2, 2, 1, 1, 1, 1, 1, 0,
2, 2, 1, 2, 2, 2, 2, 1,
1, 1, 1, 1, 0, 0, 0, 0,
2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
2, 2, 2, 2, 1, 1, 1, 1,

```

```

1, 1, 1, 0, 0, 0, 0, 0,
2, 2, 2, 1, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 1,
2, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 2, 3,
-4, -3, -3, -3, -3, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-3, -3, -3, -3, -2, -2, -2, -1,
-1, -1, -1, 0, 0, 0, -1, -1,
-2, -2, -2, -2, -2, -2, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 1, 1, 1, 1,
2, 2, 2, 2, 2, 1, 1, 1,
1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 2, 2, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 2,
2, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
2, 2, 2, 2, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
2, 2, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 2,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 2, 2,
-3, -3, -3, -3, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, -1,
-2, -2, -2, -2, -2, -1, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 0, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 1,
1, 1, 1, 1, 1, 1, 2, 2,
2, 3, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 1,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 1, 1, 1, 0,
2, 2, 2, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
2, 2, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0,

```



```

1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
-3, -2, -2, -2, -2, -2, -1, -1,
-1, -1, 0, 0, 0, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 2, 2,
2, 3, 3, 3, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 3,
3, 3, 3, 3, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 3, 3,
3, 3, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 1,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 1, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 2,
1, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
2, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1,
-2, -2, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, 0,
0, 0, 0, 0, 0, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 1,
1, 1, 1, 1, 1, 1, 2, 2,
2, 3, 3, 3, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 3, 3,
3, 3, 3, 3, 3, 3, 3, 2,
2, 2, 2, 3, 3, 3, 3, 3,
3, 3, 3, 3, 2, 2, 2, 2,
3, 3, 3, 2, 3, 3, 3, 3,
3, 3, 3, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 3, 3,
3, 3, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 1, 1, 1, 1,
2, 2, 2, 2, 2, 2, 2, 2,
2, 1, 1, 1, 1, 1, 1, 1,
2, 2, 2, 1, 1, 1, 1, 1,
1, 1, 1, 1, 0, 1, 1, 0,
2, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, -1, -1,
1, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-2, -1, -1, -1, 0, 0, 0, 0,
0, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 0, 1, 1, 1,

```

```

1, 2, 2, 2, 2, 2, 1, 1,
1, 1, 1, 1, 1, 1, 2, 2,
2, 2, 3, 3, 3, 2, 2, 2,
2, 2, 2, 2, 2, 2, 3, 3,
3, 3, 3, 3, 3, 3, 3, 3,
2, 2, 2, 3, 3, 3, 3, 3,
3, 4, 4, 3, 3, 3, 3, 3,
3, 3, 3, 3, 3, 3, 3, 3,
3, 3, 3, 3, 3, 2, 2, 2,
3, 3, 3, 3, 3, 3, 3, 3,
3, 3, 3, 3, 2, 2, 2, 2,
2, 2, 2, 2, 3, 3, 3, 3,
3, 3, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 1, 1, 2, 1,
2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 1, 1,
2, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 1, 0,
1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, -1, -1, -1, -1, -1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, -1 },
{ -8, -8, -8, -8, -8, -8, -9, -9,
-9, -9, -8, -8, -7, -7, -6, -3,
-9, -9, -8, -8, -9, -9, -10, -10,
-9, -9, -9, -8, -8, -8, -7, -5,
-9, -9, -9, -9, -9, -10, -10, -10,
-9, -9, -9, -8, -8, -8, -8, -7,
-9, -9, -9, -9, -9, -9, -9, -9,
-8, -8, -8, -8, -8, -8, -8, -8,
-9, -8, -9, -9, -8, -8, -8, -7,
-7, -7, -6, -7, -7, -8, -8, -8,
-8, -8, -8, -8, -7, -7, -6, -6,
-5, -5, -6, -6, -7, -8, -8, -8,
-8, -7, -7, -7, -6, -5, -4, -4,
-3, -4, -5, -6, -6, -7, -8, -8,
-7, -6, -6, -5, -4, -3, -2, -2,
-1, -2, -4, -5, -6, -7, -7, -8,
-6, -5, -4, -3, -2, -1, 0, 1,
0, -1, -2, -4, -5, -6, -7, -7,
-4, -3, -2, -1, 0, 1, 3, 3,
2, 1, -1, -3, -4, -5, -6, -7,
-3, -2, 0, 1, 2, 4, 4, 4,
3, 2, 0, -1, -3, -4, -5, -6,
-1, 0, 2, 3, 5, 5, 5, 5,
5, 3, 1, 0, -2, -3, -4, -5,
0, 2, 4, 6, 6, 6, 6, 6,
5, 4, 2, 0, -1, -2, -3, -3,
2, 4, 5, 7, 7, 7, 7, 7,
6, 5, 3, 1, 0, -1, -2, -3,
3, 5, 6, 7, 7, 7, 7, 7,
6, 6, 4, 2, 1, 0, -1, -2,
4, 6, 7, 7, 7, 7, 7, 7,
6, 6, 5, 4, 3, 2, 1, 1,
-6, -6, -6, -6, -6, -6, -6, -7,
-7, -7, -7, -6, -6, -5, -4, -2,
-7, -6, -6, -6, -6, -7, -8, -8,
-8, -8, -7, -7, -7, -6, -5, -4,
-7, -6, -7, -7, -7, -8, -8, -8,
-8, -8, -7, -7, -7, -7, -6, -6,
-7, -6, -7, -7, -7, -7, -7, -7,
-7, -7, -6, -6, -6, -7, -7, -7,
-7, -6, -7, -7, -7, -7, -6, -6,
-6, -6, -6, -6, -6, -6, -7, -7,

```

-7, -6, -6, -6, -6, -6, -5, -5,
 -5, -5, -5, -5, -6, -6, -7, -7,
 -7, -6, -5, -5, -5, -5, -4, -3,
 -3, -3, -4, -5, -6, -6, -7, -7,
 -6, -6, -5, -4, -4, -3, -2, -1,
 -1, -2, -3, -5, -6, -6, -7, -7,
 -5, -4, -4, -3, -2, -1, 0, 1,
 0, -1, -2, -4, -5, -6, -6, -6,
 -4, -3, -2, -2, 0, 1, 2, 2,
 2, 0, -1, -3, -4, -5, -5, -6,
 -3, -2, -1, 0, 1, 3, 4, 4,
 3, 1, 0, -2, -3, -4, -4, -5,
 -2, -1, 1, 2, 3, 4, 5, 5,
 4, 2, 0, -1, -2, -3, -3, -4,
 -1, 0, 2, 4, 5, 5, 6, 5,
 5, 3, 1, -1, -2, -2, -2, -3,
 0, 2, 4, 5, 6, 6, 6, 6,
 5, 4, 2, 0, -1, -2, -2, -2,
 2, 4, 5, 6, 7, 6, 6, 6,
 5, 5, 3, 1, 0, -1, -1, -2,
 3, 5, 6, 6, 7, 6, 6, 6,
 5, 5, 4, 3, 2, 1, 0, 0,
 -4, -4, -4, -5, -4, -5, -5, -5,
 -5, -6, -6, -5, -5, -4, -3, -1,
 -5, -5, -5, -5, -5, -6, -6, -6,
 -6, -7, -6, -6, -6, -5, -4, -3,
 -5, -5, -5, -6, -6, -6, -7, -7,
 -7, -7, -6, -6, -6, -5, -5, -5,
 -5, -5, -5, -5, -6, -6, -6, -7,
 -6, -6, -5, -5, -5, -5, -5, -5,
 -5, -5, -5, -5, -6, -6, -6, -6,
 -6, -5, -5, -5, -5, -5, -6, -6,
 -5, -5, -5, -5, -5, -5, -5, -4,
 -4, -4, -4, -5, -5, -6, -6, -6,
 -5, -5, -5, -4, -4, -4, -3, -3,
 -3, -3, -4, -5, -5, -6, -6, -6,
 -5, -5, -4, -4, -4, -3, -2, -1,
 -1, -2, -3, -4, -5, -6, -6, -6,
 -5, -4, -3, -3, -2, -2, -1, 0,
 0, -1, -2, -4, -5, -5, -5, -5,
 -4, -3, -3, -2, -1, 0, 2, 2,
 2, 0, -1, -3, -4, -5, -5, -5,
 -3, -2, -1, 0, 0, 2, 3, 3,
 3, 1, -1, -2, -3, -3, -4, -4,
 -2, -1, 0, 1, 2, 3, 4, 4,
 3, 1, 0, -2, -2, -2, -3, -3,
 -1, 0, 1, 3, 4, 5, 5, 4,
 4, 2, 0, -1, -2, -2, -2, -3,
 0, 1, 3, 4, 5, 5, 5, 5,
 4, 3, 1, -1, -2, -2, -2, -2,
 1, 3, 4, 5, 5, 6, 5, 5,
 4, 3, 2, 1, -1, -1, -1, -2,
 2, 4, 5, 5, 6, 5, 5, 4,
 4, 4, 3, 2, 1, 0, 0, -1,
 -3, -3, -3, -4, -3, -4, -4, -4,
 -4, -4, -5, -4, -4, -3, -2, -1,
 -4, -3, -4, -4, -4, -5, -5, -5,
 -5, -5, -5, -5, -4, -4, -3, -2,
 -4, -4, -4, -4, -5, -5, -6, -6,
 -6, -6, -6, -5, -4, -4, -4, -4,
 -4, -4, -4, -4, -5, -5, -6, -6,
 -6, -5, -5, -4, -4, -4, -5, -5,
 -4, -4, -4, -4, -4, -5, -5, -5,
 -5, -5, -4, -4, -4, -5, -5, -5,
 -4, -4, -4, -4, -4, -4, -4, -4,
 -4, -4, -4, -4, -4, -5, -5, -5,
 -4, -4, -4, -4, -4, -4, -3, -3,
 -3, -3, -4, -4, -5, -5, -5, -5,
 -5, -4, -4, -3, -3, -3, -2, -1,
 -1, -2, -3, -4, -5, -5, -5, -5,
 -4, -4, -3, -3, -2, -2, -1, 0,

```

0, -1, -3, -4, -4, -5, -5, -5,
-4, -3, -2, -2, -1, 0, 1, 2,
1, 0, -2, -3, -4, -4, -4, -4,
-3, -2, -2, -1, 0, 1, 2, 3,
2, 0, -1, -2, -3, -3, -3, -4,
-2, -1, -1, 0, 1, 3, 3, 4,
3, 1, -1, -2, -2, -2, -3, -3,
-2, -1, 0, 2, 3, 4, 4, 4,
3, 2, 0, -2, -2, -2, -2, -3,
-1, 0, 2, 3, 4, 5, 5, 4,
3, 2, 0, -1, -2, -2, -2, -2,
0, 2, 3, 4, 5, 5, 5, 4,
3, 2, 1, 0, -1, -1, -1, -2,
1, 3, 4, 5, 5, 5, 4, 4,
3, 2, 1, 0, 0, 0, -1, -1,
-2, -2, -3, -3, -3, -3, -3, -3,
-3, -3, -4, -4, -3, -2, -2, -1,
-2, -3, -3, -3, -4, -4, -4, -4,
-5, -5, -5, -4, -3, -3, -2, -1,
-3, -3, -3, -3, -4, -4, -5, -5,
-5, -5, -5, -4, -3, -3, -3, -3,
-3, -3, -3, -3, -4, -4, -5, -5,
-5, -5, -4, -3, -3, -4, -4, -4,
-3, -3, -3, -3, -4, -4, -4, -5,
-5, -4, -3, -3, -4, -4, -4, -4,
-3, -3, -3, -3, -3, -4, -4, -4,
-4, -3, -3, -3, -4, -4, -4, -4,
-4, -3, -3, -3, -3, -3, -3, -3,
-2, -2, -3, -3, -4, -4, -4, -4,
-4, -4, -3, -3, -3, -2, -2, -1,
-1, -2, -3, -4, -4, -4, -4, -4,
-4, -3, -3, -2, -2, -2, -1, 0,
0, -1, -3, -3, -4, -4, -4, -4,
-3, -3, -2, -2, -1, -1, 0, 1,
1, 0, -2, -3, -3, -4, -4, -4,
-3, -2, -2, -1, 0, 0, 2, 2,
2, 0, -1, -2, -2, -3, -3, -3,
-2, -2, -1, 0, 1, 2, 3, 3,
2, 0, -1, -2, -2, -2, -3, -3,
-2, -1, 0, 1, 2, 3, 4, 3,
2, 1, -1, -2, -2, -2, -2, -3,
-1, 0, 1, 2, 4, 4, 4, 4,
3, 1, 0, -1, -1, -2, -2, -2,
0, 1, 2, 4, 4, 4, 4, 3,
2, 1, 0, 0, -1, -1, -2, -2,
1, 2, 3, 4, 4, 4, 4, 3,
2, 1, 0, 0, 0, -1, -1, -1,
-1, -2, -2, -3, -2, -2, -2, -3,
-3, -3, -3, -3, -2, -2, -1, 0,
-1, -2, -2, -3, -3, -3, -3, -4,
-4, -4, -4, -3, -2, -2, -2, -1,
-2, -2, -2, -2, -3, -4, -4, -4,
-5, -4, -4, -3, -2, -2, -3, -2,
-2, -2, -2, -2, -3, -3, -4, -5,
-5, -4, -3, -3, -3, -3, -3, -3,
-2, -2, -2, -2, -3, -3, -3, -4,
-4, -4, -3, -2, -3, -3, -3, -3,
-3, -2, -2, -2, -2, -3, -3, -3,
-4, -3, -2, -3, -3, -3, -3, -3,
-3, -3, -2, -2, -2, -3, -3, -3,
-2, -2, -2, -3, -3, -3, -3, -3,
-3, -3, -3, -2, -2, -2, -2, -1,
-1, -2, -2, -3, -3, -3, -4, -4,
-3, -3, -2, -2, -2, -1, -1, 0,
0, -1, -2, -3, -3, -3, -3, -3,
-3, -2, -2, -1, -1, -1, 0, 1,
1, -1, -2, -2, -3, -3, -3, -3,
-2, -2, -1, -1, 0, 0, 1, 2,
1, 0, -1, -2, -2, -3, -3, -3,
-2, -1, -1, 0, 0, 1, 2, 2,
1, 0, -1, -2, -2, -2, -2, -3,

```

```

-2, -1, 0, 0, 2, 3, 3, 3,
2, 0, -1, -2, -2, -2, -2, -2,
-2, -1, 0, 2, 3, 3, 3, 3,
2, 0, 0, -1, -1, -2, -2, -2,
-1, 0, 2, 3, 3, 3, 3, 2,
1, 1, 0, -1, -1, -1, -2, -2,
0, 2, 3, 3, 3, 3, 3, 2,
1, 1, 0, -1, -1, -1, -1, -2,
0, -1, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -1, -1, 0,
0, -1, -1, -2, -2, -3, -2, -3,
-3, -3, -3, -2, -2, -2, -1, -1,
-1, -1, -1, -1, -2, -3, -3, -4,
-4, -3, -3, -2, -2, -2, -2, -2,
-1, -1, -1, -1, -2, -2, -3, -4,
-4, -3, -2, -2, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -2, -2, -3,
-4, -3, -2, -2, -2, -2, -2, -2,
-2, -1, -1, -1, -1, -2, -2, -3,
-3, -2, -2, -2, -2, -2, -2, -2,
-2, -2, -2, -1, -1, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -3, -3,
-3, -2, -2, -2, -2, -1, -1, -1,
-1, -1, -2, -2, -2, -3, -3, -3,
-3, -2, -2, -1, -1, -1, -1, 0,
0, -1, -2, -2, -2, -3, -3, -3,
-3, -2, -2, -1, -1, 0, 0, 1,
1, -1, -1, -2, -2, -2, -3, -3,
-2, -2, -1, -1, 0, 0, 1, 1,
1, 0, -1, -2, -2, -2, -3, -3,
-2, -1, -1, 0, 0, 1, 1, 1,
1, 0, -1, -2, -2, -2, -2, -3,
-2, -1, -1, 0, 1, 2, 2, 2,
1, 0, -1, -1, -2, -2, -2, -2,
-2, -1, 0, 1, 2, 2, 2, 2,
1, 0, 0, -1, -1, -2, -2, -2,
-1, 0, 1, 2, 3, 2, 2, 1,
1, 0, 0, -1, -1, -1, -2, -2,
0, 1, 2, 3, 3, 3, 2, 1,
1, 0, 0, -1, -1, -1, -2, -2,
0, -1, -2, -2, -2, -1, -1, -2,
-2, -2, -2, -1, -1, -1, 0, 0,
0, 0, -1, -1, -2, -2, -2, -2,
-2, -2, -2, -1, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -2, -2,
-3, -2, -2, -1, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -1, -2,
-2, -2, -1, -1, -1, -2, -2, -2,
-1, 0, 0, 0, 0, -1, -1, -2,
-2, -1, -1, -1, -1, -2, -2, -2,
-1, 0, 0, 0, 0, -1, -1, -1,
-1, -1, -1, -1, -1, -2, -2, -2,
-1, -1, -1, 0, 0, -1, -1, -1,
-1, -1, 0, -1, -1, -2, -2, -2,
-2, -1, -1, -1, 0, 0, -1, 0,
0, 0, 0, -1, -1, -2, -2, -2,
-2, -1, -1, -1, 0, 0, 0, 0,
0, 0, 0, -1, -1, -2, -2, -2,
-2, -1, -1, -1, 0, 0, 0, 1,
1, 0, 0, -1, -2, -2, -2, -2,
-2, -1, -1, 0, 0, 0, 0, 1,
1, 0, 0, -1, -1, -2, -2, -2,
-2, -1, -1, 0, 0, 0, 1, 1,
1, 1, 0, -1, -1, -2, -2, -2,
-2, -1, 0, 0, 0, 1, 1, 1,
1, 1, 0, -1, -1, -1, -2, -2,
-2, -1, 0, 0, 1, 1, 1, 1,
1, 1, 0, -1, -1, -1, -2, -2,
-1, -1, 1, 1, 2, 2, 2, 1,
1, 0, 0, -1, -1, -2, -2, -2,
-1, 0, 1, 2, 2, 2, 1, 1,

```

```

0, 0, -1, -1, -1, -1, -2, -2,
0, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, 0, 0,
0, 0, 0, -1, -1, -1, -1, -2,
-2, -2, -1, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 1, 0, 0, 0, 0, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
0, 0, 1, 1, 1, 0, 0, -1,
-1, -1, -1, 0, 0, -1, -1, -1,
0, 0, 0, 1, 1, 0, 0, 0,
-1, -1, 0, 0, 0, -1, -1, -1,
-1, 0, 0, 0, 0, -1, -1, -1,
-1, 0, 0, 0, 0, 0, 1, 0,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, 0, 0, 0, 1, 1, 1,
1, 1, 1, 0, 0, -1, -1, -1,
-1, -1, 0, 0, 0, 1, 1, 1,
2, 2, 1, 0, 0, -1, -1, -2,
-1, -1, 0, 0, 0, 1, 1, 2,
2, 2, 1, 0, 0, -1, -1, -2,
-1, -1, 0, 0, 0, 1, 1, 2,
2, 2, 1, 0, 0, -1, -1, -2,
-2, -1, 0, 0, 1, 1, 2, 2,
2, 2, 1, 0, -1, -1, -1, -2,
-2, -1, 0, 1, 1, 1, 2, 1,
1, 0, 0, -1, -1, -1, -2, -2,
-1, 0, 1, 1, 2, 2, 1, 0,
0, 0, -1, -1, -1, -1, -2, -2,
0, 0, -1, -1, -2, -2, -2, -2,
-1, -1, -1, -1, -1, -1, -1, 0,
1, 1, 0, 0, -1, -1, -2, -2,
-2, -1, -2, -2, -1, -1, -1, 0,
1, 1, 1, 1, 1, 0, -1, -1,
-1, -2, -2, -1, -1, -1, -1, 0,
1, 1, 1, 1, 1, 1, 0, -1,
-1, -2, -1, -1, 0, 0, 0, -1,
0, 1, 1, 1, 1, 1, 1, 0,
-1, -1, -1, 0, 0, 0, 0, -1,
0, 1, 1, 1, 1, 1, 1, 0,
-1, -1, 0, 0, 0, 0, 0, -1,
0, 0, 1, 1, 1, 1, 1, 0,
0, 0, 1, 1, 0, 0, 0, -1,
0, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, -1,
0, 0, 1, 1, 1, 1, 2, 2,
2, 2, 2, 1, 1, 0, -1, -1,
0, 0, 1, 1, 1, 2, 2, 2,
3, 3, 2, 1, 0, 0, -1, -1,
-1, 0, 1, 1, 1, 2, 2, 3,
3, 3, 2, 1, 0, 0, -1, -1,
-1, 0, 1, 1, 1, 2, 3, 3,
3, 3, 2, 1, 0, -1, -1, -1,
-1, 0, 0, 1, 1, 2, 3, 3,
2, 2, 2, 0, -1, -1, -1, -1,
-2, -1, 0, 1, 1, 2, 2, 2,
1, 1, 0, 0, -1, -1, -1, -2,
-1, 0, 1, 1, 1, 1, 1, 0,
0, 0, -1, -1, -1, -1, -2, -2,
0, 0, -1, -1, -2, -2, -2, -2,
-2, -1, -2, -2, -1, -1, -1, 0,
1, 1, 1, 0, -1, -1, -2, -2,
-2, -2, -2, -2, -2, -1, -1, 0,
1, 1, 1, 1, 1, 0, -1, -2,
-2, -3, -2, -2, -2, -1, -1, 0,

```

```

1, 1, 1, 1, 1, 1, 0, -1,
-2, -2, -2, -2, -1, -1, 0, 0,
1, 1, 1, 1, 1, 1, 0, -1,
-2, -2, -2, -1, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 1, -1,
-2, -2, -1, -1, 0, 0, 0, 0,
1, 1, 1, 1, 1, 2, 1, 0,
-1, -1, -1, 0, 0, 0, 0, 0,
1, 1, 1, 1, 2, 2, 2, 0,
0, 0, 0, 1, 0, 0, 0, 0,
0, 1, 1, 2, 2, 2, 2, 1,
1, 1, 2, 1, 1, 0, 0, 0,
0, 1, 1, 2, 2, 2, 2, 2,
2, 2, 2, 2, 1, 1, 0, 0,
0, 1, 1, 2, 2, 2, 3, 3,
3, 3, 2, 2, 1, 1, 0, -1,
0, 0, 1, 2, 2, 2, 3, 4,
4, 4, 3, 2, 1, 0, 0, 0,
-1, 0, 1, 2, 2, 3, 3, 4,
4, 3, 3, 1, 0, 0, 0, 0,
-1, 0, 1, 1, 2, 3, 3, 3,
3, 3, 2, 1, 0, 0, 0, -1,
-1, 0, 1, 1, 2, 2, 2, 2,
2, 2, 1, 0, 0, -1, -1, -1,
-1, 0, 1, 1, 1, 1, 1, 1,
0, 0, 0, -1, -1, -1, -1, -1,
1, 0, 0, -1, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -1, -1, 0,
1, 1, 1, 1, 0, -1, -2, -2,
-3, -3, -3, -3, -2, -2, -1, 0,
1, 1, 1, 1, 1, 0, -1, -2,
-3, -3, -3, -3, -3, -2, -1, 0,
2, 2, 2, 1, 1, 1, 0, -2,
-3, -3, -3, -3, -2, -1, 0, 0,
1, 1, 2, 1, 2, 1, 0, -2,
-3, -3, -3, -2, -1, 0, 0, 0,
2, 2, 2, 2, 2, 1, 1, -1,
-2, -3, -2, -1, 0, 0, 0, 0,
1, 1, 2, 2, 2, 2, 1, 0,
-2, -2, -1, 0, 0, 0, 0, 0,
1, 1, 2, 2, 3, 3, 2, 0,
-1, -1, 0, 1, 1, 0, 0, 0,
1, 1, 2, 2, 3, 3, 3, 2,
1, 1, 2, 2, 2, 1, 1, 0,
1, 1, 2, 2, 3, 3, 3, 3,
2, 3, 3, 2, 2, 2, 1, 0,
1, 1, 2, 3, 3, 3, 3, 3,
3, 3, 3, 2, 2, 1, 1, 0,
0, 1, 2, 3, 3, 3, 3, 4,
4, 4, 3, 2, 1, 1, 1, 0,
0, 1, 2, 2, 3, 3, 4, 5,
5, 4, 3, 2, 1, 1, 0, 0,
-1, 0, 1, 2, 3, 3, 4, 4,
4, 3, 3, 2, 1, 0, 0, 0,
-1, 0, 1, 2, 2, 3, 3, 3,
3, 2, 2, 1, 0, 0, 0, -1,
-1, 0, 1, 1, 1, 1, 1, 1,
1, 0, 0, 0, -1, -1, -1, -1,
1, 0, 0, -1, -2, -2, -3, -3,
-3, -3, -3, -3, -2, -2, -1, -1,
1, 1, 1, 1, 0, -1, -2, -3,
-3, -4, -4, -3, -3, -2, -1, -1,
2, 2, 2, 1, 1, 0, -1, -3,
-4, -4, -4, -4, -3, -2, -1, 0,
2, 2, 2, 2, 2, 1, -1, -2,
-4, -4, -4, -4, -2, -1, 0, 0,
2, 2, 2, 2, 2, 1, 0, -2,
-3, -4, -4, -3, -1, 0, 0, 0,
2, 2, 3, 3, 2, 1, 0, -1,
-3, -3, -3, -2, 0, 0, 1, 1,
2, 2, 3, 3, 3, 3, 1, -1,

```

```

-2, -2, -1, 0, 1, 1, 1, 1,
2, 2, 3, 3, 3, 3, 2, 0,
-1, -1, 0, 1, 1, 1, 1, 1,
2, 2, 3, 3, 3, 4, 3, 2,
1, 1, 2, 2, 2, 2, 1, 1,
2, 2, 3, 3, 3, 4, 4, 3,
2, 3, 3, 3, 2, 2, 2, 1,
2, 2, 3, 3, 4, 4, 4, 4,
4, 4, 4, 3, 2, 2, 2, 1,
1, 2, 3, 3, 4, 4, 4, 5,
5, 5, 4, 3, 2, 2, 1, 1,
0, 2, 3, 3, 3, 4, 5, 5,
5, 5, 4, 3, 2, 1, 1, 0,
0, 1, 2, 3, 3, 4, 4, 5,
5, 4, 4, 2, 1, 1, 0, 0,
-1, 1, 2, 2, 3, 3, 3, 3,
3, 3, 2, 2, 1, 0, 0, 0,
-1, 0, 1, 1, 2, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0,
1, 0, 0, -1, -2, -3, -3, -3,
-3, -3, -4, -4, -3, -2, -2, -1,
2, 1, 1, 1, 0, -1, -3, -4,
-4, -4, -4, -4, -4, -3, -2, -1,
2, 2, 2, 2, 1, 0, -2, -3,
-5, -5, -5, -5, -4, -3, -2, -1,
3, 2, 2, 2, 2, 1, -1, -3,
-4, -5, -5, -4, -3, -2, -1, 0,
3, 3, 3, 2, 2, 1, -1, -2,
-4, -5, -4, -3, -2, -1, 0, 1,
3, 3, 3, 3, 3, 1, 0, -2,
-3, -4, -3, -2, -1, 0, 1, 1,
3, 3, 3, 4, 4, 3, 1, -1,
-2, -3, -2, -1, 1, 1, 1, 1,
3, 3, 4, 4, 4, 3, 2, 0,
-1, -1, -1, 0, 1, 1, 1, 1,
3, 3, 4, 4, 4, 4, 3, 2,
1, 1, 2, 2, 3, 2, 2, 1,
3, 3, 4, 4, 4, 4, 4, 3,
3, 3, 3, 3, 3, 3, 3, 2,
2, 3, 4, 4, 4, 5, 4, 4,
4, 4, 4, 4, 3, 3, 2, 2,
2, 3, 4, 4, 5, 5, 5, 5,
5, 5, 5, 4, 3, 2, 2, 2,
1, 2, 3, 4, 4, 5, 5, 6,
6, 5, 5, 4, 3, 2, 2, 1,
0, 2, 3, 3, 4, 5, 5, 5,
5, 5, 4, 3, 2, 2, 1, 1,
0, 1, 2, 3, 3, 3, 4, 4,
4, 3, 3, 2, 2, 1, 1, 0,
-1, 0, 1, 1, 2, 2, 2, 2,
1, 1, 1, 0, 0, 0, 0, 0,
1, 0, -1, -1, -2, -3, -4, -4,
-4, -4, -5, -5, -4, -3, -3, -2,
2, 2, 1, 0, -1, -2, -3, -5,
-5, -5, -5, -5, -4, -4, -3, -1,
3, 3, 2, 2, 1, -1, -2, -4,
-5, -6, -6, -6, -5, -4, -3, -1,
4, 3, 3, 2, 2, 0, -2, -3,
-5, -6, -6, -5, -4, -3, -1, 0,
5, 4, 4, 3, 2, 1, -1, -3,
-4, -5, -5, -4, -3, -1, 0, 1,
5, 4, 4, 4, 3, 1, 0, -2,
-3, -4, -4, -3, -1, 0, 1, 2,
4, 4, 4, 4, 4, 2, 1, -1,
-2, -3, -2, -1, 0, 1, 2, 2,
4, 4, 5, 5, 4, 4, 2, 0,
0, -1, -1, 0, 1, 2, 2, 2,
4, 4, 5, 5, 5, 4, 3, 2,
1, 1, 2, 3, 3, 4, 3, 2,
4, 4, 5, 5, 5, 5, 4, 3,
3, 3, 4, 4, 4, 3, 3,

```



```

4, 4, 5, 5, 5, 5, 5, 4,
4, 5, 5, 4, 4, 4, 3, 3,
3, 4, 5, 5, 6, 6, 5, 5,
5, 6, 5, 5, 4, 3, 3, 3,
2, 3, 4, 5, 5, 5, 6, 6,
6, 6, 5, 5, 4, 3, 3, 3,
1, 3, 4, 4, 5, 5, 6, 6,
6, 6, 5, 4, 4, 3, 3, 2,
0, 2, 3, 3, 4, 4, 4, 4,
4, 4, 4, 3, 3, 3, 2, 2,
-1, 0, 1, 2, 2, 2, 2, 2,
2, 2, 1, 1, 1, 1, 1, 1,
1, 0, -1, -2, -3, -4, -5, -5,
-6, -6, -6, -6, -5, -4, -4, -3,
2, 2, 1, 0, -1, -3, -4, -5,
-6, -6, -6, -6, -6, -5, -4, -3,
3, 3, 3, 2, 1, -1, -3, -5,
-6, -7, -7, -7, -6, -5, -4, -2,
5, 4, 3, 3, 2, 0, -2, -4,
-5, -6, -7, -6, -5, -4, -2, -1,
6, 5, 4, 3, 2, 1, -1, -3,
-4, -5, -6, -5, -4, -2, -1, 0,
6, 6, 5, 4, 3, 2, 0, -2,
-3, -4, -4, -3, -2, 0, 1, 2,
6, 6, 6, 5, 5, 2, 1, -1,
-2, -3, -3, -2, 0, 1, 2, 3,
6, 6, 6, 6, 5, 4, 2, 0,
0, -1, -1, 0, 1, 2, 3, 3,
6, 6, 6, 6, 6, 5, 4, 2,
1, 0, 1, 3, 3, 4, 4, 3,
5, 6, 6, 6, 6, 6, 5, 4,
4, 3, 4, 5, 5, 5, 5,
5, 6, 6, 6, 7, 6, 6, 5,
5, 5, 6, 5, 5, 5, 5, 5,
4, 5, 6, 6, 7, 7, 6, 6,
6, 7, 6, 6, 5, 5, 5, 5,
4, 5, 6, 6, 6, 6, 7, 7,
7, 7, 7, 6, 6, 5, 5, 5,
3, 4, 5, 5, 6, 6, 7, 7,
7, 7, 6, 6, 5, 5, 5, 4,
1, 3, 4, 4, 5, 5, 5, 6,
5, 5, 4, 4, 4, 4, 4, 4,
-1, 1, 2, 3, 3, 3, 4, 4,
3, 3, 3, 2, 2, 2, 2, 2 },
{ -7, -12, -16, -17, -18, -18, -18, -18,
-18, -18, -18, -18, -18, -17, -13, -10,
-6, -10, -13, -14, -14, -14, -14, -13,
-13, -14, -13, -13, -13, -11, -8, -5,
-5, -8, -10, -11, -11, -10, -10, -10,
-9, -9, -9, -9, -9, -7, -4, -2,
-4, -7, -8, -8, -8, -8, -7, -7,
-7, -7, -7, -7, -7, -6, -3, 0,
-3, -6, -7, -7, -6, -6, -6, -6,
-6, -6, -6, -5, -5, -4, -1, 1,
-3, -4, -5, -5, -5, -5, -5, -5,
-5, -4, -4, -3, -3, -2, 1, 3,
-3, -3, -3, -4, -4, -5, -5, -4,
-3, -2, -2, -1, -1, 0, 2, 5,
-2, -2, -2, -3, -3, -4, -3, -3,
-2, 0, 0, 1, 1, 2, 4, 7,
-1, -1, -2, -2, -2, -2, -2, -1,
0, 2, 2, 3, 3, 4, 7, 9,
0, -1, -1, -1, -1, -1, 0, 1,
3, 4, 5, 5, 5, 6, 9, 11,
0, 0, 0, 1, 1, 0, 1, 3,
5, 6, 7, 7, 8, 9, 11, 13,
1, 1, 1, 2, 2, 3, 4, 6,
8, 9, 9, 10, 10, 12, 13, 16,
2, 2, 3, 3, 4, 6, 7, 9,
10, 12, 12, 12, 13, 14, 16, 18,
3, 3, 4, 5, 6, 8, 10, 12,

```

13, 14, 15, 15, 16, 17, 19, 20,
 4, 4, 5, 7, 9, 11, 13, 15,
 16, 17, 18, 18, 19, 20, 21, 22,
 6, 6, 7, 9, 11, 13, 15, 17,
 19, 20, 20, 21, 21, 23, 24, 24,
 -3, -7, -12, -14, -15, -15, -15, -16,
 -17, -17, -17, -17, -17, -17, -15, -12,
 -2, -6, -10, -11, -11, -12, -12, -12,
 -12, -13, -13, -13, -13, -12, -10, -7,
 -3, -6, -8, -9, -9, -9, -9, -9,
 -9, -9, -10, -10, -9, -8, -6, -4,
 -3, -6, -7, -8, -8, -8, -7, -7,
 -7, -7, -8, -8, -7, -6, -5, -2,
 -3, -5, -6, -7, -7, -6, -6, -6,
 -6, -6, -6, -6, -6, -5, -3, -1,
 -3, -5, -6, -6, -6, -5, -5, -5,
 -4, -4, -4, -4, -4, -3, -1, 1,
 -2, -4, -5, -5, -5, -4, -4, -4,
 -3, -3, -2, -2, -2, -1, 0, 2,
 -2, -3, -3, -4, -3, -4, -3, -3,
 -1, 0, 0, 0, 0, 1, 2, 3,
 -2, -2, -2, -3, -3, -3, -2, -1,
 1, 2, 2, 2, 2, 2, 3, 5,
 -1, -1, -1, -2, -2, -1, 0, 1,
 3, 4, 4, 4, 3, 3, 5, 7,
 0, -1, -1, -1, -1, 0, 1, 3,
 5, 6, 6, 5, 5, 6, 7, 9,
 0, 0, 0, 1, 1, 2, 3, 5,
 7, 7, 7, 7, 7, 8, 9, 11,
 1, 1, 1, 2, 3, 4, 5, 7,
 9, 9, 9, 9, 9, 10, 11, 13,
 1, 1, 2, 3, 5, 6, 8, 9,
 11, 11, 11, 11, 12, 13, 14, 16,
 2, 2, 4, 5, 7, 9, 11, 12,
 13, 14, 14, 14, 15, 16, 17, 19,
 4, 4, 5, 7, 9, 11, 13, 15,
 16, 16, 16, 17, 18, 19, 20, 22,
 -2, -4, -7, -10, -11, -12, -13, -14,
 -15, -16, -17, -17, -17, -17, -16, -13,
 -1, -4, -7, -8, -9, -10, -10, -11,
 -11, -12, -13, -13, -13, -12, -11, -9,
 -1, -4, -7, -8, -8, -8, -8, -9,
 -9, -10, -10, -11, -10, -9, -8, -6,
 -1, -4, -6, -7, -7, -7, -7, -7,
 -8, -8, -9, -9, -8, -7, -6, -4,
 -1, -4, -6, -6, -7, -7, -6, -6,
 -6, -7, -7, -7, -6, -5, -4, -2,
 -1, -4, -5, -6, -6, -6, -5, -5,
 -5, -5, -5, -4, -4, -3, -2, -1,
 -1, -4, -5, -5, -5, -5, -5, -4,
 -3, -3, -3, -3, -2, -2, -1, 0,
 -1, -3, -4, -4, -4, -4, -3, -3,
 -1, -1, 0, 0, -1, -1, 0, 1,
 -1, -2, -3, -3, -3, -3, -2, -1,
 1, 1, 1, 1, 1, 0, 1, 2,
 -1, -2, -2, -2, -2, -1, 0, 1,
 3, 3, 3, 3, 2, 2, 2, 4,
 0, -1, -1, -2, -2, 0, 1, 3,
 4, 5, 4, 4, 3, 3, 4, 5,
 1, 0, 0, -1, -1, 1, 2, 4,
 5, 6, 6, 5, 4, 5, 6, 7,
 2, 0, 0, 0, 1, 2, 4, 6,
 7, 7, 6, 6, 6, 7, 8, 10,
 3, 1, 1, 2, 3, 4, 6, 8,
 8, 8, 8, 8, 8, 9, 11, 12,
 4, 1, 3, 4, 4, 6, 8, 10,
 10, 10, 11, 11, 11, 12, 13, 15,
 5, 3, 4, 5, 7, 9, 11, 12,
 13, 13, 13, 14, 14, 15, 16, 18,
 -1, -3, -6, -7, -9, -10, -12, -13,
 -15, -16, -17, -17, -17, -17, -16, -16,

```

-1, -3, -5, -7, -8, -9, -10, -11,
-12, -13, -13, -14, -14, -13, -12, -12,
0, -3, -5, -6, -7, -8, -8, -9,
-10, -11, -11, -12, -11, -11, -10, -9,
0, -2, -4, -6, -6, -7, -7, -8,
-8, -9, -10, -10, -9, -9, -7, -7,
0, -2, -4, -5, -6, -6, -6, -6,
-7, -8, -8, -8, -7, -6, -5, -5,
1, -2, -4, -5, -5, -5, -5, -5,
-5, -6, -6, -5, -5, -4, -4, -4,
1, -1, -4, -5, -5, -4, -4, -4,
-4, -4, -3, -3, -3, -3, -3, -3,
2, -1, -3, -4, -4, -3, -3, -2,
-2, -1, -1, -1, -2, -2, -2, -2,
2, -2, -2, -3, -2, -2, -2, -1,
0, 1, 1, 0, 0, -1, -1, -1,
2, -1, -2, -2, -2, -1, 0, 1,
2, 2, 2, 1, 1, 0, 0, 1,
4, 0, -1, -2, -1, 0, 1, 3,
3, 3, 3, 2, 1, 1, 2, 2,
5, 1, -1, -1, -1, 1, 2, 4,
4, 4, 4, 3, 2, 2, 3, 4,
6, 2, 0, -1, 0, 2, 3, 5,
5, 5, 4, 4, 4, 4, 5, 6,
6, 2, 1, 1, 2, 3, 5, 6,
6, 6, 5, 5, 5, 6, 8, 9,
7, 3, 2, 2, 3, 5, 6, 7,
7, 7, 7, 7, 8, 9, 10, 11,
7, 5, 3, 4, 5, 7, 8, 9,
9, 10, 10, 10, 10, 12, 13, 14,
1, -2, -5, -7, -8, -9, -11, -13,
-14, -16, -17, -18, -18, -17, -16, -16,
1, -1, -4, -6, -7, -8, -10, -11,
-13, -14, -15, -15, -15, -14, -14, -14,
1, -1, -4, -5, -6, -7, -8, -9,
-11, -12, -13, -13, -12, -12, -12, -12,
1, -1, -3, -4, -5, -6, -7, -8,
-9, -10, -11, -11, -11, -10, -10, -10,
2, 0, -2, -4, -5, -5, -5, -6,
-8, -9, -9, -9, -9, -8, -8, -8,
2, 0, -2, -3, -4, -4, -4, -5,
-6, -7, -7, -7, -6, -6, -6, -7,
3, 1, -2, -3, -4, -4, -4, -4,
-4, -5, -4, -4, -4, -4, -5, -6,
3, 1, -1, -3, -3, -3, -2, -2,
-2, -2, -2, -3, -3, -3, -4, -5,
4, 2, -1, -2, -2, -2, -1, 0,
0, 0, -1, -2, -2, -2, -3, -3,
5, 2, -1, -2, -2, -1, 1, 1,
1, 1, 0, -1, -1, -1, -2, -2,
6, 3, 0, -1, -1, 0, 2, 2,
2, 2, 1, 0, 0, 0, -1, -1,
7, 4, 0, -1, -1, 1, 2, 3,
3, 2, 2, 1, 1, 1, 1, 1,
8, 5, 1, 0, 0, 2, 3, 3,
3, 3, 2, 2, 2, 2, 2, 3,
9, 6, 3, 0, 1, 3, 4, 4,
4, 4, 3, 3, 3, 4, 4, 5,
9, 7, 4, 1, 3, 4, 5, 5,
5, 5, 5, 5, 5, 6, 7, 8,
9, 8, 5, 3, 4, 5, 6, 6,
6, 6, 7, 7, 8, 8, 10, 10,
2, 0, -3, -4, -6, -8, -10, -12,
-14, -15, -16, -17, -17, -16, -16, -16,
2, 0, -2, -3, -5, -7, -9, -11,
-12, -14, -15, -15, -15, -15, -14, -14,
3, 0, -2, -3, -4, -5, -8, -9,
-11, -12, -13, -13, -13, -13, -13, -14,
3, 1, -1, -2, -3, -5, -7, -8,
-10, -11, -12, -12, -12, -12, -12, -13,
3, 1, -1, -2, -3, -4, -5, -7,

```

```

-8, -9, -10, -10, -10, -10, -11, -11,
3, 2, 0, -2, -3, -4, -4, -5,
-7, -8, -8, -8, -8, -8, -9, -9,
4, 2, 0, -1, -2, -3, -3, -4,
-5, -6, -6, -6, -6, -7, -7, -8,
4, 3, 1, 0, -2, -3, -2, -3,
-3, -3, -3, -4, -5, -5, -6, -7,
5, 4, 2, 0, -1, -1, -1, -1,
-1, -2, -2, -3, -4, -4, -5, -6,
6, 4, 3, 1, -1, -1, 0, 0,
0, -1, -2, -2, -3, -3, -4, -5,
7, 6, 3, 1, -1, 0, 1, 1,
1, 0, -1, -1, -2, -2, -3, -4,
9, 7, 4, 1, 0, 1, 2, 2,
1, 0, 0, 0, -1, -1, -2, -2,
10, 7, 4, 2, 1, 2, 2, 2,
2, 1, 1, 0, 0, 0, 0, 0,
11, 8, 5, 3, 2, 2, 3, 3,
2, 2, 2, 2, 2, 2, 1, 2,
11, 9, 7, 4, 2, 3, 4, 3,
3, 3, 3, 3, 3, 3, 3, 4,
11, 9, 7, 5, 3, 4, 4, 4,
4, 4, 4, 5, 5, 5, 6, 7,
2, 0, -1, -2, -3, -6, -8, -10,
-12, -14, -15, -15, -16, -15, -15, -15,
2, 0, -1, -1, -2, -4, -7, -9,
-12, -13, -14, -15, -15, -14, -14, -14,
3, 1, 0, -1, -2, -3, -6, -8,
-11, -12, -13, -13, -13, -13, -13, -14,
3, 2, 1, 0, -1, -2, -5, -7,
-9, -11, -12, -12, -12, -12, -12, -13,
3, 2, 1, 0, -1, -2, -4, -6,
-8, -9, -10, -10, -10, -11, -11, -11,
4, 2, 1, 0, -1, -2, -3, -5,
-7, -8, -8, -8, -9, -9, -10, -10,
4, 2, 1, 1, 0, -2, -3, -4,
-5, -6, -6, -7, -7, -8, -8, -9,
5, 3, 2, 1, 0, -1, -2, -3,
-3, -4, -5, -5, -6, -7, -7, -8,
6, 5, 4, 3, 1, 0, -1, -1,
-2, -3, -4, -4, -5, -6, -6, -7,
7, 6, 5, 4, 2, 0, 0, 0,
-1, -2, -3, -4, -4, -5, -5, -6,
8, 7, 6, 5, 2, 1, 1, 1,
-1, -2, -2, -3, -3, -4, -5, -5,
10, 8, 7, 5, 3, 2, 2, 1,
0, -1, -1, -2, -3, -3, -4, -4,
11, 9, 8, 6, 4, 2, 2, 1,
0, 0, -1, -1, -1, -2, -3, -4,
12, 10, 9, 7, 4, 3, 3, 2,
1, 0, 0, 0, 0, -1, -1, -2,
12, 11, 10, 8, 5, 4, 3, 2,
2, 1, 1, 1, 1, 1, 0, 0,
13, 11, 10, 8, 6, 4, 4, 3,
2, 2, 2, 2, 3, 2, 2, 3,
2, 0, 0, 0, -1, -2, -5, -8,
-11, -12, -13, -13, -14, -14, -14, -14,
2, 1, 1, 0, 0, -1, -4, -7,
-10, -12, -13, -13, -14, -14, -14, -14,
3, 2, 1, 1, 0, -1, -2, -5,
-9, -11, -12, -13, -13, -13, -13, -13,
4, 2, 2, 1, 1, 0, -1, -4,
-8, -10, -11, -11, -11, -12, -12, -13,
4, 3, 2, 2, 1, 0, -1, -2,
-7, -9, -9, -10, -10, -11, -11, -11,
4, 3, 3, 2, 2, 1, 0, -2,
-5, -7, -8, -8, -9, -9, -10, -10,
5, 3, 3, 2, 2, 1, 0, -1,
-4, -6, -6, -7, -7, -8, -9, -9,
5, 4, 3, 3, 3, 2, 0, 0,
-2, -4, -4, -5, -6, -7, -7, -8,

```

```

7, 6, 5, 4, 4, 3, 1, 0,
-1, -3, -4, -4, -5, -6, -6, -7,
8, 7, 6, 5, 5, 3, 2, 1,
-1, -2, -3, -4, -4, -5, -6, -6,
9, 8, 7, 7, 6, 5, 2, 1,
0, -2, -2, -3, -4, -4, -5, -5,
10, 9, 8, 8, 7, 5, 3, 2,
0, -2, -2, -3, -3, -4, -4, -5,
11, 10, 10, 9, 8, 6, 4, 2,
0, -1, -2, -2, -2, -3, -4, -4,
12, 11, 11, 10, 8, 6, 4, 2,
0, -1, -1, -1, -1, -2, -3, -3,
13, 12, 12, 10, 9, 7, 5, 2,
1, 0, 0, 0, 0, -1, -1, -1,
13, 12, 11, 10, 8, 7, 5, 3,
1, 1, 1, 1, 1, 0, 0, 0,
1, 0, 0, 0, -1, -1, -2, -5,
-7, -9, -10, -11, -12, -13, -13, -14,
2, 1, 1, 1, 0, -1, -2, -3,
-6, -9, -11, -12, -13, -13, -13, -14,
3, 2, 2, 1, 1, 0, -1, -2,
-4, -8, -10, -11, -12, -13, -13, -14,
4, 3, 2, 2, 1, 0, -1, -2,
-4, -6, -9, -10, -11, -12, -12, -13,
4, 3, 2, 2, 1, 1, 0, -2,
-3, -6, -7, -9, -10, -11, -11, -11,
5, 3, 3, 2, 2, 1, 0, -1,
-3, -4, -6, -7, -9, -10, -10, -10,
5, 4, 3, 3, 2, 1, 0, -1,
-2, -3, -5, -6, -8, -9, -9, -9,
6, 5, 4, 4, 3, 2, 1, -1,
-1, -2, -4, -5, -6, -7, -8, -8,
7, 6, 5, 5, 4, 3, 3, 0,
0, -2, -3, -4, -5, -6, -7, -7,
8, 7, 7, 6, 5, 5, 4, 1,
0, -1, -3, -3, -4, -5, -6, -6,
9, 8, 8, 7, 7, 6, 5, 2,
0, -1, -2, -3, -4, -4, -5, -6,
11, 10, 9, 9, 8, 7, 6, 2,
0, -1, -2, -2, -3, -4, -4, -5,
12, 11, 11, 10, 9, 8, 6, 3,
0, -1, -1, -2, -3, -3, -4, -4,
13, 12, 12, 11, 10, 9, 7, 4,
1, 0, -1, -1, -2, -3, -3, -3,
14, 13, 13, 12, 10, 9, 7, 4,
1, 0, 0, 0, -1, -2, -2, -3,
13, 13, 12, 12, 10, 9, 7, 5,
2, 1, 0, 0, -1, -1, -1, -2,
0, -1, -1, -1, -2, -2, -2, -3,
-4, -5, -7, -8, -10, -12, -13, -14,
1, 0, 0, -1, -1, -2, -2, -2,
-3, -4, -6, -9, -11, -13, -13, -14,
2, 1, 1, 0, 0, -1, -2, -2,
-2, -3, -5, -8, -10, -12, -13, -14,
3, 2, 1, 1, 0, -1, -1, -2,
-2, -3, -4, -6, -9, -11, -12, -13,
4, 3, 2, 1, 1, 0, -1, -1,
-2, -2, -3, -5, -8, -10, -11, -12,
4, 3, 2, 2, 1, 0, 0, -1,
-1, -1, -1, -5, -8, -10, -10, -11,
5, 4, 3, 2, 2, 1, 0, -1,
-1, 0, -1, -4, -7, -9, -9, -10,
6, 5, 4, 3, 2, 2, 1, 0,
0, 0, -1, -4, -6, -8, -8, -9,
7, 6, 5, 4, 4, 3, 2, 2,
1, 1, -1, -3, -5, -7, -7, -8,
8, 7, 7, 6, 5, 5, 4, 3,
2, 1, 0, -2, -4, -6, -6, -7,
10, 8, 8, 8, 7, 6, 5, 4,
3, 1, -1, -2, -4, -5, -5, -6,
11, 10, 10, 9, 9, 8, 6, 5,

```

```

3, 1, 0, -2, -3, -4, -4, -6,
12, 12, 11, 11, 10, 9, 7, 6,
3, 1, 0, -1, -3, -3, -4, -5,
13, 13, 13, 12, 11, 10, 8, 6,
4, 2, 0, -1, -2, -3, -3, -5,
14, 14, 13, 13, 12, 11, 9, 7,
5, 2, 1, -1, -2, -2, -3, -4,
14, 14, 14, 13, 12, 11, 9, 7,
5, 3, 1, 0, -1, -2, -3, -3,
-2, -3, -3, -3, -3, -3, -3, -3,
-4, -4, -5, -7, -9, -11, -13, -14,
-1, -2, -2, -2, -3, -3, -3, -3,
-3, -3, -3, -5, -8, -11, -12, -14,
0, -1, -1, -1, -2, -2, -3, -3,
-3, -3, -2, -3, -7, -10, -12, -13,
1, 0, 0, -1, -1, -2, -3, -3,
-3, -2, -1, -2, -6, -9, -11, -13,
2, 1, 0, 0, -1, -2, -2, -3,
-2, -2, -1, -1, -5, -8, -10, -12,
3, 2, 1, 0, 0, -1, -2, -2,
-2, -1, 0, -1, -5, -7, -10, -11,
4, 3, 2, 1, 0, 0, -1, -2,
-1, 0, 1, -1, -4, -7, -9, -11,
5, 4, 3, 2, 1, 0, 0, 0,
0, 1, 1, -1, -4, -6, -8, -10,
6, 5, 4, 3, 2, 2, 1, 1,
2, 2, 1, -1, -3, -5, -7, -9,
7, 6, 5, 5, 4, 4, 4, 3,
3, 2, 1, 0, -2, -4, -6, -8,
9, 8, 7, 7, 7, 6, 5, 5,
4, 3, 2, 0, -2, -4, -5, -7,
10, 10, 9, 9, 9, 8, 6, 6,
4, 3, 2, 0, -1, -3, -5, -6,
12, 11, 11, 11, 10, 9, 8, 7,
5, 3, 2, 1, -1, -2, -4, -6,
13, 13, 13, 12, 11, 10, 9, 7,
6, 4, 2, 1, -1, -2, -4, -5,
14, 14, 14, 13, 12, 11, 10, 8,
6, 4, 2, 1, -1, -2, -3, -5,
16, 15, 15, 14, 13, 12, 10, 9,
7, 5, 3, 1, 0, -2, -3, -4,
-5, -5, -5, -5, -5, -5, -5, -5,
-5, -5, -5, -6, -7, -9, -10, -12,
-3, -4, -4, -4, -4, -4, -5, -5,
-4, -4, -4, -4, -6, -9, -11, -12,
-2, -2, -3, -3, -4, -4, -4, -4,
-4, -3, -2, -2, -4, -8, -11, -12,
-1, -1, -2, -2, -3, -4, -4, -4,
-4, -3, -1, -1, -2, -7, -9, -12,
0, 0, -1, -2, -3, -3, -4, -4,
-3, -2, 0, 0, -1, -6, -9, -11,
1, 0, -1, -1, -2, -3, -3, -3,
-2, -1, 0, 1, -1, -5, -8, -11,
2, 1, 0, 0, -1, -2, -2, -2,
-1, 0, 1, 1, -1, -4, -7, -10,
3, 2, 1, 1, 0, -1, -1, 0,
1, 1, 2, 1, -1, -4, -6, -9,
4, 3, 3, 2, 1, 1, 1, 1,
2, 3, 2, 1, -1, -3, -6, -9,
6, 5, 4, 3, 3, 3, 3, 3,
3, 3, 3, 1, 0, -2, -5, -8,
7, 6, 6, 6, 6, 6, 5, 5,
4, 4, 3, 2, 0, -2, -4, -7,
8, 8, 8, 8, 8, 7, 7, 6,
5, 4, 3, 2, 0, -1, -4, -6,
10, 10, 10, 10, 10, 9, 8, 7,
6, 5, 3, 2, 1, -1, -4, -6,
12, 12, 12, 12, 11, 10, 9, 8,
7, 5, 3, 2, 1, -1, -3, -5,
14, 14, 14, 13, 12, 11, 10, 9,
7, 5, 3, 2, 1, -1, -3, -5,

```

16,	16,	16,	15,	14,	13,	11,	10,
8,	6,	4,	2,	0,	-1,	-3,	-5,
-7,	-7,	-7,	-7,	-7,	-7,	-8,	-7,
-7,	-7,	-7,	-7,	-6,	-7,	-7,	-8,
-6,	-6,	-6,	-6,	-6,	-6,	-6,	-6,
-6,	-6,	-5,	-5,	-6,	-7,	-8,	-10,
-5,	-5,	-5,	-5,	-5,	-6,	-6,	-6,
-5,	-4,	-3,	-3,	-4,	-6,	-8,	-10,
-3,	-4,	-4,	-4,	-5,	-5,	-5,	-5,
-4,	-3,	-2,	-2,	-2,	-5,	-8,	-10,
-2,	-2,	-3,	-3,	-4,	-5,	-5,	-4,
-3,	-2,	0,	0,	-1,	-3,	-6,	-9,
-1,	-1,	-2,	-3,	-3,	-4,	-4,	-3,
-2,	-1,	1,	1,	0,	-2,	-6,	-9,
1,	0,	-1,	-2,	-3,	-3,	-3,	-2,
-1,	0,	1,	1,	1,	-1,	-5,	-9,
2,	1,	0,	0,	-1,	-1,	-1,	0,
1,	1,	2,	2,	1,	-1,	-5,	-8,
3,	2,	2,	1,	1,	1,	1,	2,
2,	3,	3,	2,	1,	-1,	-4,	-7,
4,	3,	3,	3,	3,	3,	3,	3,
4,	4,	3,	3,	1,	0,	-3,	-7,
5,	5,	5,	5,	5,	5,	5,	5,
4,	4,	4,	3,	1,	0,	-3,	-7,
6,	6,	7,	7,	7,	7,	7,	6,
5,	5,	4,	3,	2,	0,	-3,	-6,
8,	8,	9,	9,	9,	9,	8,	7,
6,	5,	4,	3,	2,	0,	-2,	-6,
10,	10,	11,	11,	11,	10,	9,	8,
7,	6,	4,	3,	2,	0,	-2,	-5,
13,	13,	13,	13,	12,	11,	10,	9,
8,	6,	4,	3,	2,	0,	-2,	-5,
16,	16,	16,	15,	14,	13,	12,	11,
9,	7,	5,	3,	2,	0,	-2,	-5,
-10,	-10,	-10,	-11,	-12,	-12,	-12,	-12,
-11,	-11,	-10,	-8,	-7,	-6,	-6,	-6,
-9,	-8,	-8,	-9,	-9,	-10,	-10,	-9,
-8,	-8,	-8,	-8,	-6,	-6,	-7,	-7,
-7,	-7,	-7,	-7,	-8,	-8,	-8,	-7,
-6,	-5,	-5,	-5,	-5,	-5,	-6,	-8,
-6,	-6,	-6,	-6,	-6,	-7,	-7,	-6,
-5,	-3,	-3,	-3,	-3,	-4,	-6,	-8,
-5,	-5,	-4,	-5,	-5,	-5,	-6,	-5,
-3,	-2,	-1,	-1,	-1,	-2,	-5,	-8,
-3,	-3,	-3,	-4,	-4,	-4,	-4,	-3,
-2,	-1,	0,	0,	0,	-1,	-4,	-7,
-2,	-2,	-2,	-3,	-3,	-3,	-3,	-2,
0,	1,	2,	2,	2,	0,	-3,	-6,
0,	0,	-1,	-1,	-2,	-2,	-1,	0,
1,	2,	2,	2,	2,	1,	-2,	-6,
1,	1,	1,	0,	0,	1,	1,	2,
3,	3,	3,	3,	2,	1,	-2,	-6,
2,	2,	2,	2,	2,	3,	3,	3,
4,	4,	3,	3,	3,	1,	-2,	-6,
4,	3,	4,	4,	5,	5,	5,	5,
4,	4,	4,	4,	3,	2,	-2,	-6,
5,	5,	6,	6,	6,	7,	7,	6,
5,	5,	5,	4,	3,	2,	-2,	-6,
6,	7,	8,	8,	8,	8,	8,	7,
6,	6,	5,	4,	3,	2,	-2,	-6,
8,	9,	10,	10,	10,	10,	9,	8,
7,	7,	6,	4,	3,	2,	-1,	-5,
11,	12,	13,	12,	12,	11,	10,	9,
8,	7,	6,	5,	3,	2,	-2,	-6,
15,	16,	16,	15,	14,	13,	12,	11,
10,	8,	6,	4,	3,	1,	-2,	-6,
-15,	-15,	-15,	-15,	-16,	-16,	-16,	-15,
-15,	-13,	-11,	-9,	-7,	-6,	-5,	-6,
-13,	-13,	-13,	-13,	-13,	-14,	-14,	-13,
-12,	-11,	-10,	-8,	-7,	-5,	-6,	-6,
-12,	-11,	-11,	-11,	-11,	-12,	-11,	-10,

```

-8, -7, -7, -7, -6, -4, -5, -6,
-11, -10, -9, -9, -9, -9, -9, -7,
-5, -4, -4, -5, -4, -3, -4, -5,
-9, -8, -7, -7, -7, -7, -7, -5,
-3, -2, -2, -2, -3, -2, -3, -5,
-8, -7, -6, -6, -6, -5, -5, -3,
-2, 0, -1, -1, -1, -1, -3, -5,
-6, -4, -4, -4, -4, -4, -3, -2,
0, 1, 1, 1, 1, 0, -1, -4,
-3, -3, -2, -2, -2, -2, -1, 0,
2, 3, 2, 3, 3, 2, 0, -3,
-2, -1, 0, 0, 0, 0, 1, 2,
3, 3, 3, 4, 4, 2, 1, -3,
0, 1, 1, 2, 2, 3, 3, 4,
4, 4, 4, 4, 4, 3, 1, -4,
0, 2, 3, 3, 4, 4, 5, 5,
5, 5, 5, 5, 4, 3, 1, -4,
2, 4, 5, 5, 6, 6, 7, 6,
6, 6, 6, 5, 5, 4, 1, -5,
4, 5, 7, 7, 8, 8, 8, 7,
7, 7, 6, 6, 5, 4, 1, -5,
5, 7, 8, 9, 9, 10, 9, 9,
8, 7, 7, 6, 5, 4, 1, -5,
9, 10, 11, 12, 12, 11, 11, 10,
9, 8, 7, 6, 5, 4, 0, -5,
13, 14, 15, 15, 14, 14, 13, 12,
11, 9, 8, 6, 5, 3, -1, -5,
-20, -20, -19, -19, -19, -18, -18, -17,
-16, -14, -12, -10, -7, -5, -4, -4,
-18, -18, -17, -17, -17, -16, -16, -15,
-13, -12, -10, -8, -7, -5, -4, -5,
-17, -16, -15, -15, -15, -15, -14, -12,
-10, -10, -9, -7, -6, -4, -3, -5,
-15, -14, -13, -13, -13, -13, -12, -9,
-7, -7, -6, -6, -5, -3, -2, -4,
-14, -13, -12, -11, -11, -10, -9, -6,
-4, -4, -4, -4, -3, -2, -1, -3,
-12, -11, -10, -9, -8, -7, -6, -3,
-2, -1, -2, -2, -1, -1, -1, -3,
-10, -9, -8, -7, -6, -4, -3, -1,
0, 0, 0, 0, 0, 1, 0, -2,
-8, -6, -5, -4, -3, -2, -1, 1,
2, 2, 2, 2, 2, 2, 1, -1,
-6, -4, -2, -1, 0, 0, 1, 2,
4, 4, 3, 3, 3, 3, 2, 0,
-4, -2, -1, 1, 2, 2, 3, 4,
5, 5, 5, 5, 5, 4, 3, 1,
-1, 0, 1, 3, 3, 4, 4, 5,
6, 6, 6, 6, 6, 4, 4, 0,
0, 1, 3, 4, 5, 5, 6, 6,
7, 7, 7, 7, 6, 5, 4, -1,
1, 3, 5, 6, 7, 7, 8, 8,
7, 7, 7, 7, 7, 6, 4, -2,
3, 5, 6, 8, 9, 9, 9, 9,
8, 8, 8, 7, 7, 6, 4, -3,
6, 8, 10, 11, 11, 12, 11, 10,
10, 9, 8, 7, 7, 6, 3, -4,
11, 12, 13, 14, 15, 14, 13, 13,
11, 10, 9, 8, 7, 6, 1, -4 },
{ -26, -29, -31, -33, -34, -32, -30, -31,
-30, -28, -27, -28, -30, -30, -26, -22,
-21, -26, -29, -30, -29, -28, -26, -24,
-24, -23, -22, -24, -27, -25, -22, -17,
-16, -20, -26, -26, -25, -24, -22, -20,
-19, -18, -20, -22, -24, -21, -17, -12,
-13, -16, -21, -22, -21, -20, -18, -16,
-15, -16, -18, -20, -21, -18, -13, -8,
-10, -13, -17, -18, -17, -16, -15, -13,
-12, -14, -16, -19, -18, -15, -10, -5,
-8, -10, -12, -14, -13, -12, -11, -10,
-10, -12, -15, -17, -16, -12, -7, -2,

```


-6, -7, -9, -9, -9, -8, -7, -7,
 -8, -11, -14, -15, -13, -10, -5, -1,
 -4, -5, -6, -5, -5, -4, -3, -4,
 -6, -8, -11, -12, -11, -7, -3, 1,
 -1, -3, -3, -2, -1, 0, 0, -1,
 -4, -6, -9, -9, -7, -4, 0, 2,
 2, 1, 1, 2, 2, 3, 2, 0,
 -3, -6, -8, -6, -4, -1, 0, 2,
 5, 4, 4, 5, 5, 5, 4, 2,
 -1, -5, -6, -4, -2, 1, 2, 3,
 8, 7, 7, 8, 8, 8, 6, 4,
 0, -5, -6, -2, 0, 2, 4, 5,
 11, 10, 10, 11, 10, 10, 8, 5,
 1, -3, -5, -1, 1, 3, 4, 7,
 15, 14, 14, 13, 13, 11, 9, 6,
 2, -2, -3, 0, 2, 4, 6, 9,
 20, 18, 18, 17, 15, 13, 10, 7,
 1, -1, -1, 3, 6, 7, 10, 12,
 25, 23, 22, 20, 17, 15, 11, 6,
 0, -2, -1, 4, 7, 9, 12, 15,
 -27, -30, -33, -34, -35, -33, -31, -30,
 -28, -26, -24, -25, -27, -26, -23, -18,
 -23, -27, -30, -31, -31, -30, -27, -25,
 -23, -22, -21, -23, -25, -23, -19, -13,
 -19, -23, -27, -28, -27, -26, -24, -21,
 -20, -18, -19, -21, -22, -19, -16, -9,
 -15, -19, -24, -25, -24, -23, -21, -18,
 -16, -15, -17, -19, -19, -16, -12, -5,
 -11, -15, -20, -20, -20, -19, -18, -15,
 -13, -12, -15, -17, -16, -13, -8, -1,
 -9, -12, -15, -16, -16, -15, -14, -12,
 -10, -10, -13, -15, -13, -10, -5, 1,
 -7, -9, -11, -10, -10, -10, -9, -8,
 -7, -8, -12, -13, -11, -7, -2, 2,
 -5, -7, -7, -7, -6, -5, -5, -4,
 -4, -6, -10, -10, -8, -5, 0, 3,
 -3, -4, -4, -4, -3, -2, -1, 0,
 -2, -4, -8, -7, -5, -2, 2, 4,
 -1, -1, -1, -1, 0, 1, 2, 2,
 -1, -4, -6, -4, -2, 0, 2, 4,
 2, 1, 1, 2, 3, 4, 4, 3,
 0, -3, -4, -1, 1, 3, 4, 5,
 4, 4, 4, 5, 6, 6, 6, 5,
 2, -2, -3, 0, 2, 4, 6, 6,
 7, 7, 7, 8, 9, 9, 8, 7,
 3, -1, -2, 2, 3, 5, 7, 8,
 11, 10, 11, 11, 11, 11, 10, 8,
 4, 1, 0, 2, 4, 6, 8, 10,
 16, 15, 15, 15, 14, 13, 11, 8,
 4, 1, 2, 5, 7, 8, 10, 13,
 22, 21, 20, 19, 18, 15, 12, 7,
 2, 0, 3, 6, 8, 9, 13, 15,
 -29, -31, -34, -36, -35, -33, -30, -28,
 -26, -24, -22, -24, -25, -24, -20, -12,
 -25, -29, -31, -32, -32, -30, -27, -25,
 -23, -21, -20, -22, -22, -21, -16, -9,
 -22, -26, -28, -29, -29, -29, -25, -22,
 -20, -17, -18, -20, -20, -17, -12, -6,
 -18, -22, -25, -26, -26, -25, -24, -20,
 -17, -14, -16, -17, -17, -14, -8, -2,
 -14, -18, -21, -22, -22, -22, -21, -17,
 -14, -12, -14, -15, -14, -11, -5, 0,
 -11, -14, -17, -18, -18, -18, -17, -13,
 -11, -10, -12, -13, -12, -8, -2, 2,
 -9, -11, -11, -12, -12, -12, -11, -10,
 -8, -7, -10, -11, -9, -5, 1, 3,
 -7, -8, -8, -8, -8, -7, -6, -6,
 -4, -5, -8, -8, -6, -3, 3, 4,
 -5, -5, -5, -5, -4, -3, -3, -2,
 -1, -3, -5, -5, -3, 0, 3, 5,
 -3, -3, -3, -2, -2, -1, 0, 1,

0, -3, -5, -1, 0, 3, 4, 6,
 -1, -1, -1, 0, 1, 2, 3, 3,
 1, -2, -3, 1, 3, 5, 5, 6,
 1, 1, 2, 2, 3, 4, 5, 5,
 3, -1, -2, 2, 4, 7, 7, 7,
 4, 4, 4, 5, 6, 7, 7, 7,
 4, 0, 0, 3, 5, 7, 8, 9,
 7, 7, 8, 8, 9, 9, 9, 8,
 5, 1, 2, 4, 6, 8, 9, 10,
 12, 12, 12, 12, 12, 12, 11, 9,
 5, 2, 4, 6, 8, 9, 11, 13,
 18, 17, 17, 17, 16, 14, 13, 9,
 4, 2, 4, 8, 9, 11, 13, 15,
 -30, -32, -34, -37, -34, -32, -29, -28,
 -25, -23, -20, -22, -22, -20, -15, -8,
 -27, -30, -32, -33, -33, -30, -27, -25,
 -22, -19, -18, -20, -19, -17, -12, -5,
 -24, -28, -29, -30, -30, -28, -25, -22,
 -20, -17, -16, -18, -17, -14, -9, -3,
 -20, -24, -26, -27, -27, -26, -24, -20,
 -17, -15, -15, -16, -15, -11, -6, -1,
 -17, -20, -22, -23, -23, -23, -22, -18,
 -15, -12, -14, -14, -12, -8, -3, 1,
 -13, -16, -18, -18, -19, -19, -18, -15,
 -12, -9, -12, -11, -9, -5, 0, 2,
 -11, -12, -13, -13, -14, -14, -13, -12,
 -9, -7, -10, -10, -7, -3, 2, 4,
 -9, -9, -9, -9, -9, -9, -8, -7,
 -5, -5, -8, -7, -4, 0, 4, 5,
 -7, -7, -6, -6, -5, -5, -4, -3,
 -1, -3, -4, -4, -1, 3, 4, 6,
 -4, -5, -4, -3, -3, -2, -1, 0,
 1, -2, -3, 0, 3, 4, 5, 7,
 -2, -3, -2, -2, -1, 0, 1, 2,
 2, -1, -2, 2, 6, 6, 7, 7,
 0, -1, 0, 0, 1, 2, 3, 4,
 3, 0, 0, 4, 7, 8, 8, 8,
 2, 2, 2, 3, 4, 5, 6, 6,
 4, 1, 1, 5, 8, 9, 10, 10,
 5, 4, 5, 6, 6, 7, 8, 8,
 5, 2, 3, 6, 8, 10, 11, 12,
 9, 8, 9, 9, 10, 10, 10, 9,
 6, 3, 4, 8, 9, 11, 12, 14,
 15, 14, 14, 14, 14, 13, 12, 9,
 5, 2, 5, 8, 10, 12, 14, 16,
 -30, -33, -34, -36, -34, -32, -29, -27,
 -23, -21, -18, -19, -17, -15, -10, -6,
 -28, -31, -32, -33, -32, -30, -27, -25,
 -21, -18, -16, -17, -15, -12, -7, -4,
 -26, -29, -30, -30, -30, -27, -25, -23,
 -19, -16, -15, -15, -13, -10, -4, -2,
 -22, -25, -27, -28, -27, -26, -23, -20,
 -17, -14, -14, -13, -11, -7, -2, 0,
 -18, -21, -23, -24, -24, -23, -21, -18,
 -15, -12, -12, -11, -9, -4, 0, 1,
 -15, -17, -19, -19, -20, -19, -19, -16,
 -12, -9, -11, -9, -7, -2, 1, 3,
 -12, -13, -14, -15, -15, -15, -14, -13,
 -9, -7, -9, -7, -4, 0, 3, 4,
 -10, -11, -10, -10, -10, -10, -9, -8,
 -6, -4, -7, -4, -2, 2, 4, 6,
 -8, -8, -7, -7, -6, -6, -5, -4,
 -2, -3, -2, -1, 1, 4, 5, 7,
 -6, -6, -5, -5, -4, -3, -3, -1,
 0, -2, -1, 2, 5, 6, 6, 7,
 -4, -4, -4, -3, -2, -1, 0, 1,
 1, -1, 0, 4, 7, 8, 8, 8,
 -2, -2, -2, -1, 0, 1, 2, 3,
 2, 0, 2, 6, 9, 9, 10, 10,
 0, 0, 1, 1, 2, 3, 4, 5,
 4, 1, 3, 8, 10, 11, 11, 11,

3, 3, 3, 3, 4, 5, 6, 7,
 5, 3, 5, 9, 11, 12, 13, 13,
 7, 6, 6, 7, 7, 8, 9, 8,
 6, 3, 6, 9, 12, 13, 15, 15,
 12, 12, 11, 11, 11, 11, 11, 8,
 5, 3, 7, 9, 12, 14, 15, 17,
 -28, -32, -35, -34, -34, -31, -29, -26,
 -22, -19, -15, -15, -13, -10, -7, -5,
 -27, -30, -33, -32, -31, -29, -27, -24,
 -20, -17, -14, -13, -11, -7, -4, -3,
 -25, -28, -30, -30, -29, -27, -25, -22,
 -18, -15, -13, -12, -9, -5, -2, -1,
 -22, -25, -27, -26, -26, -25, -23, -20,
 -16, -13, -12, -10, -7, -3, 0, 1,
 -18, -21, -23, -23, -23, -22, -21, -18,
 -14, -11, -10, -8, -5, -1, 1, 2,
 -15, -18, -19, -19, -19, -19, -18, -16,
 -12, -9, -9, -7, -3, 0, 2, 4,
 -13, -14, -15, -15, -15, -15, -14, -13,
 -10, -7, -7, -5, -1, 2, 3, 5,
 -11, -12, -12, -12, -11, -10, -9, -8,
 -6, -4, -4, -2, 1, 3, 5, 6,
 -8, -10, -9, -8, -7, -6, -6, -5,
 -3, -2, 0, 2, 4, 5, 6, 7,
 -7, -7, -6, -6, -5, -4, -3, -2,
 0, -1, 2, 5, 7, 7, 7, 8,
 -5, -5, -5, -4, -3, -2, -2, 0,
 1, 0, 4, 7, 9, 9, 9, 10,
 -3, -3, -3, -2, -2, -1, 0, 2,
 2, 1, 5, 9, 11, 11, 11, 12,
 -1, -1, -1, 0, 0, 1, 2, 4,
 3, 2, 7, 11, 12, 13, 13, 13,
 1, 1, 1, 2, 2, 3, 4, 6,
 4, 4, 8, 12, 14, 14, 15, 15,
 5, 5, 5, 5, 5, 6, 7, 8,
 5, 5, 8, 12, 14, 16, 17, 18,
 10, 9, 9, 9, 9, 9, 10, 8,
 5, 5, 8, 11, 13, 16, 18, 19,
 -26, -31, -33, -33, -32, -30, -28, -25,
 -20, -16, -11, -10, -9, -7, -6, -4,
 -25, -29, -31, -30, -30, -28, -26, -23,
 -18, -14, -11, -9, -7, -5, -3, -2,
 -22, -26, -28, -28, -27, -26, -24, -21,
 -17, -12, -9, -8, -5, -3, -1, 1,
 -19, -23, -25, -25, -24, -23, -22, -19,
 -15, -10, -8, -7, -3, -1, 1, 2,
 -16, -20, -21, -21, -21, -20, -19, -18,
 -13, -9, -7, -6, -2, 0, 2, 4,
 -14, -18, -18, -18, -18, -17, -17, -15,
 -11, -8, -7, -4, 0, 2, 3, 5,
 -13, -14, -15, -14, -15, -14, -13, -12,
 -9, -7, -5, -2, 1, 3, 4, 6,
 -11, -12, -12, -11, -10, -10, -8, -8,
 -5, -3, -2, 0, 3, 4, 6, 7,
 -9, -10, -10, -9, -7, -6, -6, -5,
 -3, -1, 2, 4, 6, 6, 7, 8,
 -7, -8, -7, -6, -5, -4, -4, -3,
 -1, 0, 5, 7, 9, 9, 9, 9,
 -6, -6, -6, -5, -4, -3, -2, -1,
 1, 2, 7, 9, 10, 11, 11, 11,
 -4, -4, -4, -3, -3, -2, -1, 1,
 2, 3, 8, 11, 12, 12, 13, 13,
 -2, -2, -2, -2, -1, 0, 1, 3,
 3, 4, 10, 13, 14, 14, 15, 15,
 0, 0, 0, 1, 1, 2, 3, 4,
 4, 5, 11, 14, 16, 16, 17, 17,
 4, 3, 3, 4, 4, 4, 5, 6,
 5, 7, 11, 15, 17, 19, 19, 19,
 8, 8, 7, 7, 7, 7, 8, 7,
 5, 7, 11, 14, 17, 19, 20, 20,
 -26, -29, -31, -31, -30, -29, -26, -23,

-19, -13, -9, -8, -7, -6, -5, -4,
 -23, -27, -29, -28, -28, -27, -25, -21,
 -16, -11, -8, -7, -5, -4, -2, -1,
 -21, -24, -26, -25, -25, -24, -23, -19,
 -14, -8, -6, -5, -3, -2, 0, 1,
 -17, -20, -22, -22, -22, -21, -20, -18,
 -12, -6, -5, -4, -2, 0, 1, 3,
 -15, -18, -19, -19, -19, -18, -18, -16,
 -10, -5, -4, -3, -1, 1, 3, 4,
 -13, -16, -16, -16, -16, -15, -15, -13,
 -7, -4, -3, -1, 1, 2, 4, 6,
 -12, -13, -13, -13, -13, -12, -12, -10,
 -5, -3, -2, 0, 2, 4, 5, 7,
 -11, -12, -11, -10, -10, -9, -7, -6,
 -2, 0, 0, 2, 4, 5, 7, 8,
 -9, -10, -9, -8, -7, -6, -5, -3,
 0, 1, 4, 6, 7, 8, 8, 9,
 -8, -8, -7, -6, -5, -4, -3, -2,
 1, 2, 6, 9, 10, 10, 10, 10,
 -6, -6, -6, -5, -4, -3, -2, 0,
 2, 4, 8, 11, 12, 12, 13, 13,
 -4, -5, -4, -4, -3, -2, -1, 1,
 3, 6, 10, 12, 13, 14, 14, 14,
 -3, -3, -2, -2, -1, 0, 1, 3,
 4, 7, 11, 14, 15, 16, 16, 16,
 -1, -1, 0, 0, 1, 2, 3, 5,
 5, 9, 13, 16, 17, 18, 18, 18,
 3, 2, 2, 3, 3, 4, 5, 6,
 6, 9, 14, 17, 19, 20, 21, 20,
 7, 7, 6, 6, 6, 7, 7, 7,
 7, 10, 14, 17, 19, 21, 21, 21,
 -24, -28, -30, -30, -29, -28, -26, -22,
 -17, -12, -8, -6, -6, -5, -5, -4,
 -22, -25, -27, -27, -27, -26, -24, -20,
 -15, -9, -6, -4, -4, -3, -2, -1,
 -19, -22, -24, -24, -23, -23, -22, -18,
 -13, -6, -5, -3, -2, -1, 0, 1,
 -16, -19, -21, -21, -20, -20, -19, -16,
 -11, -5, -4, -2, -1, 0, 2, 3,
 -15, -17, -19, -18, -18, -17, -16, -14,
 -9, -4, -3, -1, 0, 1, 3, 5,
 -13, -15, -16, -16, -16, -15, -14, -11,
 -7, -4, -2, 0, 1, 2, 4, 6,
 -11, -13, -13, -13, -13, -12, -11, -9,
 -4, -2, -1, 1, 2, 4, 6, 7,
 -10, -12, -11, -10, -9, -8, -7, -4,
 0, 0, 2, 4, 4, 5, 7, 8,
 -9, -10, -9, -8, -7, -6, -4, -2,
 1, 1, 5, 7, 8, 9, 9, 9,
 -8, -8, -7, -6, -5, -4, -2, 0,
 2, 2, 7, 10, 11, 12, 11, 11,
 -6, -6, -6, -5, -4, -3, -1, 1,
 2, 4, 9, 12, 13, 14, 14, 13,
 -5, -5, -4, -4, -3, -2, 0, 2,
 3, 6, 10, 14, 15, 15, 15, 16,
 -3, -3, -3, -2, -1, 0, 1, 3,
 4, 7, 12, 15, 17, 17, 17, 17,
 -1, -1, -1, 0, 1, 2, 3, 5,
 5, 9, 13, 17, 19, 19, 19, 19,
 2, 2, 2, 3, 4, 4, 6, 6,
 6, 10, 15, 19, 21, 22, 22, 22,
 6, 6, 6, 6, 6, 7, 7, 8,
 7, 11, 16, 19, 22, 23, 23, 23,
 -23, -26, -28, -28, -28, -27, -24, -20,
 -16, -11, -6, -6, -6, -5, -4, -3,
 -20, -24, -25, -26, -25, -24, -22, -18,
 -13, -8, -5, -5, -4, -3, -2, -1,
 -17, -21, -23, -23, -22, -21, -20, -16,
 -12, -5, -3, -3, -2, -1, 0, 1,
 -15, -19, -21, -21, -20, -19, -17, -14,
 -10, -4, -2, -1, 0, 1, 2, 3,

-14, -17, -18, -18, -18, -16, -15, -12,
 -8, -3, -2, 0, 1, 2, 4, 5,
 -12, -15, -16, -16, -15, -13, -12, -10,
 -5, -2, 0, 1, 2, 3, 5, 7,
 -11, -12, -13, -14, -13, -11, -10, -7,
 -2, 0, 1, 2, 3, 5, 6, 8,
 -10, -11, -11, -10, -9, -8, -6, -3,
 1, 2, 3, 4, 5, 6, 8, 9,
 -8, -9, -9, -8, -7, -5, -3, 0,
 3, 4, 7, 8, 9, 9, 10, 10,
 -7, -8, -7, -6, -5, -3, -1, 2,
 4, 6, 10, 12, 13, 13, 13, 12,
 -6, -6, -6, -5, -4, -2, 0, 3,
 5, 7, 11, 14, 16, 17, 16, 15,
 -5, -5, -4, -4, -3, -1, 1, 3,
 5, 8, 12, 16, 18, 18, 18, 17,
 -3, -3, -3, -2, -1, 0, 2, 4,
 5, 9, 13, 17, 19, 19, 19, 20,
 -2, -1, -1, 0, 0, 2, 3, 5,
 6, 10, 14, 18, 20, 21, 21, 21,
 1, 1, 2, 2, 3, 4, 5, 6,
 7, 11, 16, 20, 23, 24, 23, 23,
 5, 5, 5, 6, 7, 7, 7, 7,
 8, 11, 17, 20, 23, 24, 25, 24,
 -21, -24, -26, -26, -26, -24, -21, -18,
 -15, -11, -8, -8, -7, -6, -5, -4,
 -19, -22, -23, -24, -23, -22, -19, -16,
 -13, -8, -7, -6, -5, -4, -3, -1,
 -16, -20, -21, -22, -21, -20, -17, -14,
 -11, -6, -5, -4, -3, -2, 0, 1,
 -15, -18, -19, -20, -19, -17, -15, -12,
 -9, -5, -3, -2, -1, 1, 2, 3,
 -13, -16, -17, -17, -16, -15, -13, -10,
 -6, -3, -2, 0, 1, 2, 4, 5,
 -12, -14, -15, -15, -14, -12, -10, -8,
 -4, -2, 0, 1, 3, 4, 5, 7,
 -10, -11, -12, -13, -11, -10, -8, -5,
 -1, 0, 2, 3, 4, 5, 7, 8,
 -9, -10, -10, -9, -8, -7, -4, -1,
 2, 2, 4, 4, 6, 7, 8, 10,
 -8, -9, -9, -7, -5, -3, 0, 3,
 4, 5, 7, 9, 9, 10, 10, 11,
 -7, -7, -7, -5, -3, -1, 1, 5,
 6, 8, 11, 13, 13, 14, 14, 14,
 -6, -6, -6, -4, -2, 0, 3, 6,
 7, 10, 13, 16, 18, 18, 18, 17,
 -5, -5, -4, -3, -1, 1, 4, 7,
 8, 10, 15, 18, 20, 21, 20, 20,
 -3, -3, -2, -1, 0, 3, 5, 8,
 9, 11, 16, 19, 21, 22, 23, 22,
 -1, -1, -1, 0, 2, 4, 7, 8,
 9, 12, 16, 20, 22, 23, 24, 24,
 1, 1, 2, 3, 4, 6, 9, 9,
 10, 14, 18, 22, 24, 26, 26, 26,
 5, 5, 5, 6, 7, 8, 9, 10,
 11, 14, 19, 23, 25, 26, 27, 26,
 -19, -22, -24, -24, -24, -22, -18, -16,
 -13, -10, -10, -11, -10, -9, -8, -7,
 -17, -20, -21, -22, -22, -20, -17, -14,
 -12, -8, -9, -8, -7, -6, -4, -3,
 -15, -19, -20, -20, -19, -18, -15, -12,
 -10, -7, -7, -6, -5, -3, -1, 1,
 -13, -17, -18, -18, -17, -16, -13, -11,
 -8, -6, -5, -4, -2, -1, 1, 3,
 -12, -15, -16, -16, -15, -13, -10, -8,
 -5, -4, -3, -1, 0, 2, 4, 5,
 -10, -13, -14, -13, -13, -11, -8, -6,
 -2, -3, -1, 1, 2, 4, 6, 7,
 -8, -10, -11, -11, -10, -8, -6, -3,
 0, -1, 1, 2, 4, 5, 7, 9,
 -7, -8, -8, -8, -7, -5, -2, 1,

1, 1, 2, 4, 5, 7, 8, 10,
 -7, -7, -7, -6, -4, -1, 2, 4,
 4, 4, 7, 8, 9, 10, 10, 11,
 -6, -6, -5, -4, -2, 1, 4, 7,
 6, 8, 11, 12, 13, 14, 14, 14,
 -5, -5, -4, -2, 0, 3, 6, 8,
 8, 11, 14, 16, 17, 18, 18, 18,
 -4, -4, -3, -1, 1, 4, 8, 9,
 10, 13, 16, 19, 21, 22, 22, 21,
 -3, -2, -1, 0, 3, 6, 9, 10,
 11, 14, 17, 20, 23, 25, 25, 24,
 -1, 0, 1, 3, 5, 8, 11, 11,
 12, 14, 18, 22, 24, 26, 27, 26,
 1, 2, 3, 5, 8, 10, 12, 12,
 14, 16, 20, 24, 27, 28, 28, 28,
 5, 6, 7, 8, 10, 12, 13, 14,
 15, 17, 21, 25, 27, 28, 29, 29,
 -17, -20, -21, -22, -21, -19, -16, -13,
 -11, -10, -11, -12, -13, -13, -11, -9,
 -15, -18, -19, -20, -19, -17, -15, -12,
 -10, -8, -10, -11, -10, -9, -7, -5,
 -14, -17, -17, -18, -17, -15, -13, -11,
 -8, -7, -9, -8, -7, -5, -3, -1,
 -12, -15, -15, -15, -15, -13, -11, -9,
 -6, -6, -7, -6, -4, -3, 0, 2,
 -10, -13, -13, -13, -13, -11, -9, -7,
 -4, -5, -5, -3, -2, 0, 3, 4,
 -9, -11, -12, -11, -10, -8, -6, -4,
 -2, -3, -2, -1, 1, 3, 5, 7,
 -7, -9, -9, -9, -8, -6, -4, -2,
 -1, -2, 0, 1, 3, 5, 7, 9,
 -6, -6, -6, -6, -5, -2, 0, 1,
 1, 0, 1, 3, 5, 7, 9, 10,
 -6, -6, -5, -4, -2, 1, 3, 4,
 3, 3, 6, 7, 8, 9, 10, 12,
 -6, -5, -3, -2, 0, 4, 6, 7,
 5, 7, 10, 12, 13, 13, 14, 13,
 -4, -4, -3, -1, 2, 6, 8, 9,
 8, 10, 14, 16, 17, 17, 18, 17,
 -3, -2, -1, 0, 4, 8, 10, 10,
 10, 13, 16, 19, 21, 22, 22, 21,
 -2, -1, 0, 2, 6, 10, 11, 12,
 12, 16, 18, 22, 23, 25, 26, 25,
 0, 1, 2, 5, 8, 12, 13, 13,
 14, 17, 20, 23, 26, 27, 28, 27,
 3, 4, 5, 8, 12, 14, 15, 15,
 16, 19, 22, 26, 28, 30, 30, 29,
 6, 7, 9, 12, 15, 16, 16, 17,
 18, 20, 23, 27, 29, 31, 31, 29,
 -15, -18, -19, -19, -18, -17, -15, -12,
 -10, -9, -11, -14, -15, -16, -15, -13,
 -13, -16, -17, -17, -16, -15, -13, -11,
 -9, -8, -10, -12, -13, -12, -10, -8,
 -12, -14, -15, -15, -14, -13, -11, -9,
 -7, -7, -9, -10, -9, -8, -6, -3,
 -10, -13, -13, -13, -12, -11, -9, -7,
 -6, -6, -8, -7, -6, -5, -2, 0,
 -9, -11, -11, -11, -10, -8, -7, -5,
 -4, -5, -6, -5, -4, -2, 1, 3,
 -7, -9, -9, -8, -7, -6, -5, -4,
 -2, -4, -4, -3, -1, 2, 4, 6,
 -6, -7, -7, -6, -4, -3, -2, -1,
 -1, -2, -1, 0, 2, 4, 6, 8,
 -5, -4, -4, -4, -2, 0, 1, 2,
 1, 0, 0, 2, 4, 6, 8, 10,
 -4, -4, -3, -1, 1, 3, 4, 5,
 3, 2, 4, 6, 7, 8, 10, 12,
 -4, -3, -2, 0, 3, 6, 7, 7,
 5, 6, 9, 11, 12, 13, 13, 13,
 -3, -2, -1, 1, 5, 8, 9, 8,
 7, 9, 13, 15, 17, 17, 17, 15,

```

-2, -1, 0, 3, 7, 10, 10, 10,
10, 12, 16, 19, 20, 21, 21, 19,
0, 1, 2, 6, 10, 12, 13, 12,
12, 16, 19, 21, 24, 25, 24, 23,
1, 2, 5, 8, 12, 14, 15, 14,
15, 18, 21, 24, 26, 27, 28, 26,
3, 5, 8, 12, 15, 17, 18, 17,
19, 22, 25, 27, 30, 31, 31, 29,
7, 9, 13, 16, 19, 19, 19, 19,
20, 23, 26, 28, 31, 33, 33, 30,
-14, -16, -16, -17, -16, -14, -13, -11,
-10, -9, -12, -15, -17, -19, -19, -17,
-11, -14, -14, -14, -14, -13, -11, -9,
-8, -8, -10, -13, -15, -15, -14, -13,
-10, -12, -12, -12, -11, -11, -9, -8,
-7, -7, -9, -11, -12, -11, -9, -8,
-8, -10, -10, -10, -9, -8, -7, -7,
-6, -6, -8, -9, -8, -7, -4, -3,
-7, -9, -9, -8, -7, -6, -5, -5,
-4, -5, -7, -7, -6, -3, -1, 0,
-5, -7, -6, -6, -5, -4, -3, -3,
-3, -4, -5, -4, -3, 0, 2, 3,
-4, -5, -4, -4, -2, -1, -1, 0,
-1, -2, -2, -2, 0, 3, 5, 6,
-3, -2, -2, -1, 0, 1, 2, 3,
1, 0, 0, 1, 3, 6, 7, 9,
-2, -1, -1, 1, 3, 4, 5, 5,
3, 2, 3, 4, 6, 8, 9, 11,
-2, -1, 0, 3, 5, 7, 8, 7,
5, 5, 8, 10, 11, 11, 12, 13,
-2, 0, 1, 5, 8, 9, 10, 9,
7, 8, 12, 14, 16, 16, 15, 14,
-1, 1, 4, 7, 10, 11, 11, 11,
10, 11, 15, 18, 20, 20, 19, 17,
1, 2, 6, 9, 12, 13, 14, 13,
12, 15, 18, 21, 24, 23, 22, 20,
2, 5, 9, 12, 15, 16, 16, 15,
15, 18, 21, 24, 26, 27, 25, 23,
5, 9, 14, 16, 18, 20, 19, 19,
19, 22, 26, 28, 30, 32, 30, 27,
10, 14, 18, 21, 22, 22, 22, 21,
22, 25, 28, 30, 33, 34, 33, 30,
-12, -14, -14, -15, -14, -12, -11, -10,
-10, -10, -12, -16, -19, -21, -22, -23,
-10, -11, -12, -12, -11, -11, -9, -8,
-8, -9, -11, -13, -16, -18, -19, -18,
-8, -9, -10, -10, -9, -9, -8, -7,
-6, -7, -9, -11, -14, -15, -14, -12,
-6, -8, -8, -8, -7, -6, -6, -5,
-5, -6, -7, -9, -11, -10, -9, -7,
-5, -6, -6, -6, -5, -4, -4, -4,
-4, -5, -6, -8, -7, -6, -4, -3,
-3, -4, -4, -4, -3, -2, -2, -1,
-2, -4, -5, -5, -5, -3, 0, 1,
-2, -2, -2, -2, -1, 0, 0, 1,
0, -2, -3, -3, -2, 1, 3, 5,
-1, 0, 0, 1, 2, 2, 3, 4,
2, 0, 0, 0, 2, 4, 7, 8,
-1, 0, 2, 3, 4, 5, 7, 6,
4, 3, 2, 3, 5, 7, 10, 11,
0, 1, 3, 5, 7, 9, 10, 8,
6, 5, 7, 8, 9, 10, 12, 13,
0, 1, 4, 7, 9, 11, 12, 10,
8, 8, 10, 13, 14, 15, 14, 15,
1, 4, 7, 10, 12, 13, 13, 13,
11, 11, 14, 17, 19, 20, 17, 16,
2, 6, 10, 13, 14, 15, 15, 14,
14, 14, 17, 20, 23, 23, 20, 18,
5, 10, 13, 15, 17, 17, 18, 17,
16, 18, 21, 24, 26, 26, 24, 21,
9, 14, 17, 20, 21, 21, 21, 20,

```

```

20, 22, 26, 29, 30, 31, 29, 26,
14, 19, 23, 24, 25, 25, 24, 23,
22, 25, 28, 31, 33, 34, 32, 29 },
{ -53, -51, -49, -45, -41, -37, -32, -32,
-34, -35, -38, -41, -41, -37, -30, -14,
-50, -48, -46, -43, -39, -35, -31, -29,
-31, -33, -35, -39, -39, -34, -26, -12,
-46, -45, -43, -40, -37, -33, -29, -26,
-28, -30, -33, -36, -36, -30, -22, -9,
-43, -41, -39, -36, -33, -31, -27, -24,
-26, -28, -31, -34, -32, -27, -19, -6,
-39, -37, -34, -31, -29, -28, -26, -23,
-25, -26, -28, -31, -29, -24, -17, -4,
-35, -32, -29, -26, -24, -23, -23, -22,
-23, -24, -26, -29, -26, -20, -14, -3,
-30, -28, -24, -19, -18, -20, -20, -19,
-20, -21, -24, -27, -24, -17, -12, -1,
-25, -23, -20, -15, -14, -17, -17, -16,
-16, -18, -21, -23, -19, -14, -8, 1,
-20, -19, -17, -14, -13, -15, -15, -14,
-12, -13, -16, -18, -15, -10, -4, 4,
-15, -14, -14, -12, -12, -14, -15, -13,
-11, -11, -12, -15, -12, -8, -1, 7,
-8, -9, -10, -10, -10, -13, -14, -12,
-10, -9, -10, -13, -10, -6, 4, 10,
-1, -3, -5, -6, -8, -11, -12, -10,
-8, -7, -8, -10, -9, -5, 6, 13,
4, 2, 0, -2, -5, -8, -10, -9,
-7, -5, -5, -7, -7, -3, 8, 14,
7, 6, 5, 3, 0, -5, -7, -7,
-5, -3, -1, -3, -3, -1, 8, 15,
11, 11, 9, 8, 5, 0, -3, -2,
-1, 1, 4, 3, 2, 3, 8, 16,
15, 14, 12, 11, 9, 4, 2, 2,
4, 5, 8, 8, 6, 6, 10, 16,
-47, -45, -43, -40, -35, -31, -27, -26,
-30, -34, -38, -41, -39, -34, -26, -15,
-44, -42, -40, -37, -33, -29, -25, -23,
-28, -32, -36, -39, -36, -30, -22, -13,
-41, -39, -36, -33, -30, -27, -24, -21,
-27, -30, -34, -37, -33, -26, -19, -11,
-37, -35, -32, -29, -27, -25, -22, -19,
-25, -28, -32, -34, -29, -23, -16, -8,
-33, -30, -28, -24, -21, -21, -21, -19,
-23, -26, -30, -31, -26, -19, -13, -6,
-29, -26, -22, -14, -11, -17, -18, -18,
-20, -23, -28, -28, -22, -16, -11, -4,
-25, -21, -14, -6, -5, -12, -16, -16,
-18, -21, -25, -25, -18, -12, -8, -2,
-19, -17, -10, -4, -4, -10, -13, -13,
-14, -17, -21, -20, -14, -9, -4, 0,
-15, -13, -10, -5, -5, -10, -12, -12,
-11, -13, -17, -15, -10, -5, 0, 3,
-10, -10, -8, -6, -6, -10, -12, -11,
-10, -10, -14, -12, -8, -3, 4, 7,
-4, -6, -6, -6, -6, -9, -11, -10,
-9, -9, -11, -11, -7, -1, 7, 10,
1, 0, -3, -4, -4, -7, -9, -9,
-8, -7, -9, -10, -6, 1, 10, 13,
5, 4, 1, -1, -2, -5, -8, -8,
-6, -6, -7, -8, -4, 2, 12, 14,
9, 8, 5, 2, 1, -2, -6, -6,
-5, -4, -4, -5, -2, 4, 14, 16,
12, 11, 9, 7, 4, 2, -2, -3,
-2, 0, 1, 0, 1, 5, 15, 16,
16, 15, 13, 11, 9, 5, 1, 1,
2, 4, 6, 5, 5, 7, 15, 16,
-41, -40, -38, -34, -30, -27, -23, -23,
-30, -36, -39, -39, -36, -29, -20, -15,
-38, -36, -34, -31, -28, -25, -21, -21,
-28, -33, -37, -38, -32, -25, -18, -14,

```


-35, -32, -29, -27, -25, -23, -20, -18,
 -26, -31, -35, -35, -29, -22, -16, -12,
 -31, -28, -25, -20, -20, -21, -19, -17,
 -24, -29, -33, -32, -25, -19, -14, -10,
 -27, -23, -17, -11, -12, -17, -18, -17,
 -22, -27, -31, -29, -21, -15, -11, -7,
 -22, -17, -7, -4, -6, -12, -16, -16,
 -19, -24, -28, -25, -17, -12, -9, -5,
 -18, -10, -4, 0, -2, -7, -14, -14,
 -17, -22, -25, -21, -14, -9, -5, -3,
 -13, -7, -2, 1, 0, -5, -11, -11,
 -13, -18, -21, -16, -10, -5, -2, 0,
 -9, -6, -3, 0, 0, -5, -10, -10,
 -10, -13, -16, -11, -6, -1, 1, 3,
 -6, -5, -3, -1, -1, -5, -9, -9,
 -9, -10, -13, -9, -3, 2, 5, 6,
 -3, -4, -3, -2, -1, -4, -8, -9,
 -8, -8, -11, -7, -1, 4, 9, 9,
 2, -1, -1, -1, 0, -3, -7, -8,
 -7, -7, -9, -6, 0, 6, 11, 12,
 7, 3, 1, 1, 1, -1, -6, -7,
 -6, -6, -8, -5, 2, 8, 14, 14,
 9, 7, 4, 3, 3, 1, -4, -6,
 -4, -4, -6, -3, 3, 9, 15, 15,
 13, 11, 8, 6, 6, 4, -1, -3,
 -2, 0, -1, 0, 5, 11, 17, 16,
 16, 15, 12, 10, 9, 7, 2, -1,
 1, 3, 4, 4, 7, 12, 17, 16,
 -36, -34, -31, -28, -25, -23, -20, -21,
 -33, -37, -37, -36, -31, -25, -17, -14,
 -32, -30, -26, -24, -23, -21, -19, -18,
 -30, -35, -36, -34, -28, -23, -15, -14,
 -29, -25, -21, -19, -20, -20, -18, -17,
 -28, -33, -35, -32, -25, -20, -14, -13,
 -25, -20, -15, -15, -16, -18, -17, -16,
 -25, -31, -33, -28, -22, -17, -12, -12,
 -20, -13, -10, -8, -11, -15, -16, -15,
 -23, -29, -31, -24, -19, -14, -11, -9,
 -15, -9, -4, -3, -6, -11, -14, -14,
 -20, -26, -27, -21, -15, -11, -8, -6,
 -10, -5, -2, 0, -2, -7, -12, -12,
 -17, -23, -23, -17, -11, -7, -5, -4,
 -7, -4, 0, 1, 0, -3, -9, -9,
 -12, -19, -18, -13, -7, -4, -2, 0,
 -5, -3, 0, 3, 2, -1, -6, -7,
 -8, -14, -13, -8, -3, 0, 2, 3,
 -3, -1, 0, 3, 3, 1, -4, -6,
 -7, -11, -10, -5, 0, 3, 5, 5,
 -1, 0, 1, 3, 4, 1, -3, -6,
 -6, -9, -8, -3, 2, 6, 9, 9,
 1, 1, 2, 4, 4, 2, -2, -5,
 -5, -7, -7, -1, 4, 8, 11, 11,
 6, 4, 4, 4, 5, 3, -1, -5,
 -4, -6, -6, 0, 7, 11, 13, 13,
 9, 7, 5, 6, 6, 4, -1, -4,
 -3, -4, -5, 2, 8, 13, 15, 14,
 13, 10, 8, 8, 8, 6, 2, -2,
 -1, 0, -2, 3, 10, 15, 16, 15,
 16, 14, 12, 11, 11, 9, 5, 1,
 2, 3, 3, 6, 12, 16, 17, 16,
 -30, -28, -25, -24, -21, -20, -18, -20,
 -33, -36, -35, -31, -26, -21, -16, -14,
 -26, -23, -21, -20, -19, -18, -16, -16,
 -31, -34, -33, -29, -25, -20, -15, -13,
 -22, -18, -16, -16, -17, -16, -14, -14,
 -29, -33, -31, -27, -23, -19, -14, -12,
 -17, -13, -12, -13, -14, -15, -14, -13,
 -27, -31, -29, -24, -20, -16, -12, -12,
 -13, -10, -9, -7, -10, -12, -13, -13,
 -25, -29, -26, -21, -17, -13, -11, -11,
 -10, -7, -5, -3, -5, -8, -11, -11,

-22, -26, -23, -18, -13, -10, -9, -8,
 -8, -4, -2, -1, -2, -4, -8, -9,
 -18, -23, -19, -14, -9, -7, -6, -4,
 -6, -3, 0, 1, 0, -1, -4, -7,
 -13, -18, -15, -10, -6, -3, -2, -1,
 -3, -1, 1, 3, 3, 2, -1, -5,
 -9, -13, -10, -5, -2, 0, 2, 2,
 -1, 1, 4, 5, 5, 5, 1, -3,
 -7, -11, -7, -2, 1, 4, 5, 5,
 1, 3, 5, 7, 6, 5, 2, -3,
 -6, -9, -5, 0, 4, 6, 8, 8,
 3, 5, 6, 7, 7, 6, 2, -3,
 -5, -7, -3, 3, 7, 9, 10, 10,
 7, 7, 7, 8, 8, 7, 3, -2,
 -3, -6, -1, 5, 9, 12, 12, 12,
 10, 8, 8, 9, 9, 7, 3, -2,
 -2, -4, 0, 7, 12, 14, 14, 14,
 12, 11, 10, 10, 10, 9, 5, 0,
 0, -1, 1, 8, 14, 16, 16, 14,
 16, 14, 13, 13, 12, 11, 7, 3,
 3, 3, 4, 10, 15, 17, 17, 15,
 -24, -23, -22, -21, -19, -17, -16, -19,
 -32, -32, -30, -26, -23, -18, -14, -13,
 -20, -19, -18, -17, -16, -15, -14, -15,
 -30, -31, -28, -25, -22, -18, -13, -13,
 -17, -15, -15, -14, -13, -13, -12, -13,
 -27, -29, -27, -24, -21, -17, -13, -12,
 -13, -12, -11, -10, -11, -11, -11, -12,
 -26, -27, -25, -22, -19, -15, -12, -11,
 -11, -9, -7, -7, -8, -9, -10, -11,
 -24, -25, -22, -19, -15, -13, -11, -10,
 -10, -8, -5, -3, -5, -6, -9, -10,
 -22, -23, -19, -15, -12, -10, -9, -9,
 -8, -5, -3, -1, -2, -4, -7, -8,
 -19, -19, -16, -11, -8, -7, -7, -6,
 -5, -2, 0, 1, 0, 0, -3, -6,
 -14, -15, -11, -7, -5, -4, -3, -2,
 -1, 1, 3, 4, 4, 4, 1, -4,
 -9, -11, -7, -3, -1, 0, 1, 2,
 1, 3, 5, 6, 6, 6, 3, -3,
 -7, -8, -4, 0, 2, 4, 4, 5,
 4, 6, 7, 8, 8, 8, 5, -2,
 -5, -6, -1, 3, 5, 6, 7, 7,
 6, 8, 9, 9, 9, 8, 5, -2,
 -4, -4, 1, 5, 8, 9, 9, 10,
 8, 9, 10, 10, 10, 9, 6, -1,
 -2, -3, 4, 8, 10, 11, 11, 11,
 11, 10, 11, 11, 11, 10, 6, 0,
 -2, -1, 5, 10, 13, 13, 13, 13,
 13, 12, 12, 12, 11, 11, 8, 2,
 1, 1, 7, 12, 15, 15, 15, 14,
 15, 15, 14, 14, 13, 12, 10, 4,
 3, 4, 9, 14, 17, 17, 16, 15,
 -20, -20, -19, -18, -17, -16, -14, -18,
 -27, -27, -25, -23, -20, -16, -14, -13,
 -17, -17, -16, -15, -15, -14, -12, -14,
 -25, -25, -24, -22, -19, -16, -13, -13,
 -15, -13, -12, -12, -12, -11, -10, -12,
 -23, -23, -23, -22, -19, -15, -13, -12,
 -13, -10, -9, -9, -9, -10, -9, -10,
 -21, -22, -21, -20, -17, -14, -12, -11,
 -11, -9, -7, -6, -7, -8, -9, -9,
 -20, -20, -19, -17, -14, -13, -10, -10,
 -9, -7, -5, -4, -5, -7, -8, -8,
 -19, -18, -16, -13, -11, -10, -9, -9,
 -7, -5, -3, -2, -3, -4, -5, -7,
 -17, -15, -13, -10, -8, -7, -7, -6,
 -4, -2, 0, 1, 0, -1, -2, -5,
 -13, -11, -8, -5, -5, -5, -4, -2,
 0, 2, 4, 4, 4, 3, 1, -3,
 -8, -7, -4, -2, -1, -1, 0, 1,

3, 5, 7, 7, 7, 6, 4, -2,
 -6, -4, -1, 1, 3, 3, 3, 3,
 5, 8, 9, 9, 9, 8, 5, -1,
 -4, -2, 2, 4, 5, 6, 6, 6,
 7, 9, 10, 10, 10, 10, 6, 0,
 -2, 0, 5, 7, 8, 8, 8, 8,
 9, 11, 12, 12, 11, 10, 7, 1,
 0, 2, 7, 9, 10, 10, 11, 11,
 11, 12, 13, 13, 12, 11, 8, 2,
 1, 3, 9, 11, 12, 13, 13, 12,
 13, 13, 14, 14, 13, 13, 10, 4,
 3, 5, 10, 14, 14, 15, 14, 13,
 16, 15, 15, 15, 15, 14, 12, 7,
 6, 8, 12, 16, 16, 16, 15, 15,
 -18, -18, -18, -18, -17, -15, -13, -14,
 -21, -21, -22, -20, -18, -15, -13, -13,
 -15, -14, -14, -14, -14, -13, -11, -11,
 -18, -20, -21, -20, -18, -15, -13, -12,
 -13, -11, -11, -12, -12, -11, -8, -9,
 -16, -18, -20, -19, -17, -15, -12, -12,
 -12, -10, -9, -9, -10, -9, -7, -8,
 -15, -16, -18, -18, -16, -14, -11, -11,
 -10, -8, -7, -7, -8, -8, -7, -7,
 -13, -15, -16, -15, -14, -13, -10, -10,
 -9, -7, -5, -5, -6, -6, -5, -6,
 -13, -14, -13, -12, -11, -10, -8, -8,
 -7, -5, -4, -4, -3, -3, -4, -5,
 -11, -11, -10, -8, -7, -7, -7, -6,
 -3, -2, 0, 0, 0, 0, 0, -2,
 -8, -7, -6, -5, -5, -5, -4, -3,
 1, 3, 4, 4, 4, 3, 3, -1,
 -5, -3, -2, -1, -2, -2, -1, 1,
 4, 6, 7, 7, 7, 7, 5, 0,
 -2, 0, 1, 2, 1, 1, 1, 3,
 6, 8, 9, 9, 9, 9, 6, 1,
 1, 2, 3, 4, 4, 4, 4, 5,
 8, 10, 11, 12, 11, 10, 8, 2,
 2, 4, 6, 7, 7, 7, 6, 7,
 10, 12, 13, 13, 13, 12, 9, 3,
 3, 5, 8, 9, 9, 9, 9, 9,
 12, 13, 14, 14, 14, 13, 10, 5,
 5, 7, 10, 11, 11, 12, 12, 11,
 14, 14, 15, 15, 15, 14, 12, 7,
 7, 9, 13, 14, 14, 14, 13, 13,
 16, 16, 16, 17, 16, 15, 14, 10,
 9, 11, 14, 16, 16, 15, 14, 14,
 -16, -16, -17, -17, -16, -13, -10, -10,
 -17, -18, -20, -18, -15, -14, -12, -13,
 -14, -14, -14, -14, -13, -11, -8, -7,
 -14, -17, -19, -18, -16, -14, -12, -12,
 -12, -12, -11, -12, -11, -9, -5, -6,
 -11, -15, -18, -18, -16, -14, -11, -11,
 -11, -10, -9, -10, -10, -8, -4, -5,
 -9, -14, -16, -16, -15, -14, -11, -10,
 -10, -8, -8, -8, -8, -7, -4, -4,
 -9, -12, -14, -14, -13, -13, -10, -9,
 -9, -7, -7, -6, -6, -5, -3, -3,
 -7, -11, -11, -12, -11, -11, -8, -8,
 -6, -5, -5, -4, -3, -2, -1, -2,
 -6, -8, -9, -9, -8, -8, -6, -6,
 -3, -2, -1, -1, 0, 1, 2, 0,
 -3, -4, -5, -5, -6, -5, -5, -3,
 2, 3, 4, 4, 4, 4, 6, 1,
 0, -1, -1, -2, -3, -3, -1, 0,
 5, 6, 7, 7, 7, 8, 9, 3,
 2, 2, 2, 1, 0, -1, 0, 3,
 8, 9, 9, 9, 9, 10, 10, 4,
 3, 4, 4, 4, 3, 1, 2, 5,
 10, 11, 12, 12, 12, 12, 11, 6,
 5, 6, 6, 6, 6, 5, 4, 7,
 12, 13, 14, 14, 14, 14, 12, 9,

6, 8, 9, 9, 8, 8, 7, 9,
 13, 14, 15, 15, 15, 15, 13, 10,
 7, 9, 11, 11, 11, 10, 10, 11,
 15, 16, 16, 16, 16, 16, 15, 12,
 9, 11, 14, 14, 13, 13, 12, 12,
 16, 17, 18, 18, 18, 17, 17, 15,
 12, 14, 16, 16, 16, 15, 14, 14,
 -15, -16, -16, -16, -13, -9, -6, -7,
 -14, -15, -17, -15, -14, -12, -11, -12,
 -14, -13, -14, -14, -11, -6, -3, -5,
 -12, -15, -16, -15, -14, -13, -11, -11,
 -13, -12, -12, -12, -9, -5, -1, -4,
 -10, -13, -15, -15, -14, -13, -11, -10,
 -12, -11, -10, -10, -8, -4, 0, -2,
 -8, -12, -15, -14, -14, -12, -10, -9,
 -10, -9, -9, -9, -6, -3, 0, -1,
 -6, -11, -13, -13, -13, -12, -9, -8,
 -9, -8, -7, -6, -4, -1, 1, -1,
 -5, -9, -10, -11, -11, -10, -8, -7,
 -7, -6, -5, -4, -2, 1, 3, 0,
 -4, -7, -8, -9, -9, -8, -6, -5,
 -3, -2, -2, -1, 1, 4, 6, 2,
 0, -4, -5, -6, -6, -6, -4, -3,
 2, 2, 3, 4, 5, 7, 10, 3,
 2, 0, -2, -3, -4, -3, -2, 0,
 5, 6, 7, 7, 8, 11, 14, 6,
 4, 3, 1, -1, -2, -1, 0, 3,
 8, 9, 9, 9, 11, 13, 16, 10,
 5, 5, 4, 2, 0, 0, 2, 5,
 10, 11, 12, 12, 14, 15, 17, 13,
 6, 7, 6, 5, 4, 2, 4, 7,
 12, 13, 14, 15, 16, 16, 18, 14,
 7, 9, 9, 9, 7, 5, 6, 9,
 14, 15, 16, 17, 18, 18, 18, 16,
 9, 11, 11, 11, 10, 9, 9, 11,
 15, 17, 18, 18, 19, 19, 20, 17,
 11, 12, 14, 14, 13, 13, 12, 12,
 17, 18, 19, 20, 20, 21, 22, 20,
 15, 15, 17, 17, 17, 16, 15, 15,
 -15, -16, -16, -15, -9, -5, -3, -6,
 -11, -13, -15, -14, -12, -11, -10, -10,
 -14, -14, -14, -12, -7, -2, 0, -4,
 -9, -12, -13, -14, -13, -12, -10, -10,
 -13, -12, -12, -10, -6, 0, 2, -2,
 -8, -11, -12, -13, -13, -12, -10, -9,
 -12, -11, -10, -9, -4, 1, 3, -2,
 -7, -10, -12, -12, -13, -12, -10, -8,
 -10, -10, -9, -7, -3, 2, 4, -1,
 -6, -10, -10, -11, -11, -11, -9, -7,
 -9, -8, -7, -5, -1, 2, 5, 0,
 -4, -8, -9, -10, -10, -10, -7, -5,
 -7, -6, -5, -3, 1, 4, 7, 1,
 -3, -7, -8, -9, -9, -8, -6, -4,
 -3, -2, -2, 0, 3, 6, 9, 3,
 0, -4, -6, -7, -7, -6, -5, -3,
 2, 2, 3, 5, 7, 10, 14, 4,
 4, 1, -3, -5, -5, -4, -2, 1,
 5, 6, 7, 8, 10, 14, 18, 8,
 5, 4, 0, -2, -2, -2, 0, 3,
 8, 9, 10, 10, 13, 18, 20, 11,
 7, 6, 3, 0, -1, 0, 2, 5,
 11, 11, 12, 13, 16, 20, 22, 15,
 8, 9, 7, 4, 2, 2, 4, 7,
 13, 14, 15, 17, 18, 21, 23, 18,
 9, 11, 10, 7, 6, 4, 6, 9,
 14, 16, 18, 19, 20, 22, 24, 20,
 10, 12, 13, 11, 10, 8, 9, 11,
 15, 18, 19, 20, 22, 23, 25, 22,
 12, 14, 15, 14, 14, 13, 13, 13,
 17, 19, 20, 22, 23, 25, 28, 25,
 16, 16, 18, 18, 17, 16, 15, 16,

```

-16, -16, -15, -10, -5, -1, -2, -3,
-8, -12, -13, -12, -11, -10, -9, -9,
-14, -14, -13, -8, -3, 2, 1, -2,
-6, -10, -12, -12, -11, -10, -9, -9,
-13, -12, -11, -7, -1, 4, 3, -1,
-5, -8, -11, -12, -11, -11, -9, -8,
-12, -11, -10, -5, 0, 5, 4, 0,
-4, -7, -10, -11, -12, -11, -9, -7,
-10, -9, -8, -4, 1, 6, 6, 0,
-3, -6, -8, -10, -11, -10, -8, -6,
-9, -8, -6, -2, 2, 7, 7, 1,
-3, -5, -7, -9, -10, -9, -7, -5,
-7, -6, -4, 0, 4, 8, 9, 2,
-2, -5, -6, -7, -8, -8, -6, -3,
-3, -2, -1, 2, 6, 10, 11, 4,
0, -3, -5, -6, -7, -6, -4, -2,
2, 3, 4, 7, 10, 14, 15, 6,
4, 1, -3, -5, -6, -5, -2, 2,
5, 6, 7, 10, 13, 18, 19, 10,
7, 5, -1, -3, -4, -2, 1, 5,
8, 9, 11, 13, 17, 22, 22, 13,
8, 8, 4, -1, -1, 0, 3, 6,
11, 12, 13, 15, 20, 25, 25, 16,
10, 11, 8, 3, 1, 2, 5, 8,
13, 14, 16, 18, 22, 26, 27, 19,
11, 13, 12, 8, 5, 5, 7, 10,
15, 16, 18, 21, 23, 27, 28, 21,
12, 14, 14, 12, 10, 9, 11, 13,
16, 18, 20, 22, 24, 27, 29, 23,
14, 16, 17, 16, 14, 13, 14, 15,
17, 20, 21, 23, 25, 29, 31, 26,
16, 17, 19, 19, 18, 17, 17, 17,
-18, -16, -12, -6, 0, 1, -1, -2,
-5, -9, -11, -10, -9, -9, -8, -9,
-17, -14, -10, -4, 2, 4, 1, -1,
-3, -8, -10, -10, -10, -9, -9, -8,
-15, -13, -8, -2, 4, 6, 4, 0,
-2, -6, -10, -11, -10, -10, -9, -7,
-13, -11, -6, -1, 5, 7, 5, 1,
-1, -4, -8, -10, -11, -10, -8, -6,
-11, -9, -6, 0, 6, 9, 6, 2,
1, -3, -7, -9, -10, -9, -8, -5,
-10, -8, -4, 1, 6, 10, 7, 3,
1, -2, -6, -8, -9, -8, -6, -3,
-8, -6, -2, 3, 8, 11, 8, 4,
2, -1, -5, -7, -8, -7, -4, -2,
-3, -2, 1, 5, 10, 13, 11, 6,
4, 0, -4, -6, -6, -6, -3, 0,
2, 3, 6, 9, 14, 18, 15, 8,
7, 3, -3, -5, -6, -4, -1, 4,
5, 6, 9, 12, 18, 22, 19, 12,
9, 7, 0, -4, -4, -2, 2, 7,
8, 9, 12, 15, 21, 26, 22, 15,
10, 11, 5, -1, -2, 0, 5, 9,
10, 11, 15, 18, 25, 29, 25, 17,
12, 13, 10, 4, 2, 4, 8, 11,
13, 14, 17, 21, 27, 31, 27, 19,
13, 15, 14, 10, 6, 8, 11, 14,
15, 15, 19, 23, 28, 32, 29, 21,
14, 16, 17, 14, 11, 11, 15, 17,
16, 18, 22, 25, 28, 32, 30, 23,
15, 17, 19, 18, 16, 16, 18, 19,
18, 20, 23, 26, 28, 32, 32, 25,
16, 18, 20, 21, 20, 20, 21, 22,
-18, -15, -8, -1, 2, 1, -1, 0,
-2, -6, -8, -8, -8, -8, -8, -8,
-17, -13, -6, 2, 6, 4, 2, 1,
-1, -4, -8, -8, -9, -9, -8, -7,
-16, -12, -5, 3, 8, 7, 4, 2,
1, -3, -7, -9, -10, -9, -8, -6,
-14, -9, -3, 4, 8, 9, 6, 3,

```

2, -1, -6, -9, -10, -9, -7, -4,
 -12, -8, -2, 4, 9, 10, 7, 4,
 4, 1, -4, -8, -9, -8, -5, -2,
 -10, -6, 0, 5, 10, 11, 8, 5,
 5, 2, -3, -7, -8, -6, -2, 1,
 -8, -4, 2, 6, 12, 12, 9, 7,
 6, 3, -2, -6, -7, -3, 0, 3,
 -3, -1, 4, 8, 13, 14, 11, 8,
 7, 4, -2, -5, -5, -1, 2, 6,
 1, 4, 8, 13, 17, 19, 14, 10,
 10, 6, -1, -4, -4, 0, 5, 10,
 4, 8, 12, 16, 22, 23, 18, 13,
 12, 9, 2, -2, -2, 3, 8, 13,
 6, 10, 14, 19, 25, 26, 22, 17,
 14, 13, 6, 1, 0, 6, 11, 16,
 9, 12, 16, 22, 28, 29, 24, 19,
 14, 15, 12, 6, 5, 10, 15, 19,
 11, 14, 19, 25, 31, 31, 26, 19,
 15, 17, 16, 12, 11, 14, 19, 22,
 13, 17, 22, 27, 33, 33, 27, 21,
 15, 18, 18, 17, 14, 17, 22, 24,
 15, 19, 24, 29, 35, 34, 28, 22,
 15, 18, 21, 20, 19, 20, 25, 27,
 18, 22, 26, 29, 34, 35, 30, 24,
 17, 20, 23, 24, 24, 24, 27, 27,
 -18, -10, -2, 3, 3, 1, 0, 1,
 1, -3, -6, -6, -7, -7, -7, -6,
 -16, -8, 1, 6, 7, 4, 2, 2,
 2, -1, -6, -7, -7, -8, -7, -5,
 -15, -6, 2, 8, 9, 7, 5, 4,
 4, 1, -4, -7, -8, -8, -6, -2,
 -13, -4, 3, 9, 11, 9, 7, 5,
 5, 2, -2, -7, -8, -6, -2, 1,
 -11, -3, 4, 9, 12, 11, 8, 6,
 6, 4, -1, -6, -6, -2, 1, 4,
 -9, -1, 5, 10, 13, 12, 9, 7,
 8, 6, 0, -5, -3, 1, 4, 7,
 -7, 1, 6, 11, 13, 12, 10, 8,
 9, 7, 2, -2, 0, 4, 7, 9,
 -2, 4, 8, 12, 15, 14, 11, 9,
 10, 9, 4, -1, 2, 6, 9, 12,
 2, 7, 13, 17, 20, 19, 15, 12,
 13, 11, 3, 1, 5, 9, 14, 16,
 5, 10, 16, 21, 24, 23, 19, 15,
 16, 14, 5, 3, 8, 14, 18, 20,
 7, 12, 18, 24, 27, 25, 21, 18,
 17, 15, 10, 6, 11, 18, 22, 23,
 9, 14, 21, 27, 30, 28, 24, 20,
 16, 18, 15, 12, 15, 22, 26, 26,
 11, 16, 23, 30, 32, 30, 25, 21,
 16, 20, 19, 17, 19, 25, 29, 29,
 13, 19, 26, 33, 35, 32, 26, 22,
 18, 21, 21, 20, 21, 27, 31, 32,
 15, 22, 30, 36, 38, 34, 27, 23,
 19, 22, 24, 23, 25, 29, 33, 34,
 17, 24, 33, 39, 39, 36, 29, 25,
 21, 23, 27, 27, 29, 31, 34, 34,
 -17, -3, 3, 4, 3, 0, 2, 3,
 3, 0, -3, -4, -5, -6, -5, -3,
 -15, -1, 6, 8, 6, 4, 3, 4,
 5, 2, -2, -4, -5, -5, -2, 1,
 -14, 1, 8, 10, 9, 7, 6, 5,
 6, 4, 0, -5, -5, -2, 2, 5,
 -12, 3, 9, 12, 12, 10, 8, 6,
 8, 6, 2, -4, -2, 2, 6, 9,
 -10, 5, 10, 13, 13, 11, 9, 7,
 9, 9, 4, -1, 2, 7, 9, 11,
 -8, 6, 11, 14, 14, 12, 10, 8,
 11, 11, 6, 3, 6, 10, 11, 13,
 -6, 8, 12, 15, 15, 13, 11, 9,
 12, 13, 9, 8, 10, 12, 13, 15,

```

-2,  9, 13, 16, 17, 15, 13, 10,
14, 15, 11, 12, 14, 16, 18, 20,
 3, 13, 18, 20, 21, 19, 15, 14,
16, 17, 14, 16, 17, 20, 22, 25,
 5, 16, 21, 24, 24, 22, 19, 16,
18, 19, 17, 19, 21, 23, 27, 29,
 7, 19, 23, 27, 27, 24, 21, 18,
19, 20, 20, 21, 25, 27, 31, 33,
 9, 21, 26, 30, 29, 27, 24, 20,
20, 22, 22, 23, 28, 30, 34, 35,
11, 23, 29, 33, 32, 30, 25, 22,
22, 24, 24, 25, 30, 34, 36, 36,
14, 26, 33, 36, 36, 31, 26, 23,
23, 25, 27, 27, 31, 35, 38, 38,
16, 29, 37, 39, 38, 33, 27, 24,
25, 26, 30, 31, 33, 37, 40, 40,
18, 33, 40, 42, 40, 35, 29, 26,
27, 28, 31, 34, 36, 38, 42, 41 }
}

```

```

Gdf_Inter_Error[ 5 ][ 6 ][ 1000 ] =
{
  {
    {
      6,  5,  4,  3,  2,  3,  4,  5,
      5,  5,  6,  5,  4,  3,  2,  3,
      3,  3,  3,  3,  6,  4,  3,  2,
      2,  2,  3,  2,  2,  3,  4,  3,
      2,  1,  1,  1,  1,  1,  2,  2,
      3,  2,  1,  1,  1,  0,  0,  1,
      2,  2,  2,  2,  2,  1,  1,  0,
      0,  1,  2,  2,  2,  2,  2,  1,
      1,  0,  0,  0,  1,  1,  2,  1,
      1,  1,  1,  0, -1,  0,  1,  1,
      1,  1,  1,  1,  1,  0,  0,  0,
      0,  1,  0,  1,  1,  1,  1,  0,
      0, -1,  0,  0,  6,  5,  4,  3,
      2,  2,  4,  4,  4,  5,  5,  5,
      3,  2,  1,  2,  3,  3,  4,  4,
      4,  4,  3,  2,  1,  2,  2,  2,
      2,  3,  3,  2,  1,  1,  1,  1,
      1,  2,  2,  2,  3,  2,  1,  1,
      1,  1,  0,  1,  2,  2,  2,  2,
      2,  1,  1,  0,  0,  0,  1,  1,
      2,  2,  2,  1,  1,  0,  0, -1,
      0,  0,  2,  1,  1,  1,  1,  1,
      0, -1, -1,  0,  1,  1,  1,  1,
      1,  1,  0, -1, -1,  0,  0,  0,
      1,  1,  1,  1,  0, -1, -1,  0,
      5,  5,  4,  3,  2,  1,  3,  4,
      5,  4,  3,  3,  3,  1,  1,  2,
      2,  4,  4,  4,  3,  2,  2,  1,
      2,  2,  2,  3,  3,  3,  3,  2,
      2,  2,  1,  1,  1,  2,  3,  3,
      3,  2,  2,  1,  1,  0,  0,  1,
      1,  2,  3,  2,  2,  1,  1,  0,
      0,  0,  0,  1,  2,  2,  1,  1,
      1,  1,  0,  0,  0,  0,  2,  2,
      1,  1,  1,  0,  0, -1, -1, -1,
      1,  0,  1,  1,  1,  1,  0, -1,
      -1, -1, -1,  0,  1,  1,  1,  1,
      0, -1, -2, -2,  4,  3,  3,  2,
      1,  1,  3,  3,  5,  4,  4,  3,
      2,  2,  1,  2,  3,  3,  4,  4,
      3,  2,  2,  1,  1,  2,  2,  3,
      3,  3,  2,  2,  1,  1,  1,  1,
      1,  2,  2,  3,  2,  2,  1,  1,
      0,  0,  0,  1,  1,  2,  2,  2,
      1,  1,  1,  0,  0,  0,  0,  0,
      2,  2,  1,  1,  1,  1,  0,  0,
      0,  0,  2,  1,  1,  1,  1,  1,
    }
  }
}

```

```

0, 0, -1, 0, 0, 0, 0, 1,
1, 0, 0, -1, -1, -1, -1, -1,
0, 0, 1, 1, 0, -1, -1, -2,
4, 3, 2, 2, 1, 1, 3, 3,
3, 4, 3, 3, 2, 1, 1, 2,
2, 2, 2, 3, 3, 2, 1, 0,
1, 1, 2, 2, 2, 2, 2, 2,
1, 0, 0, 0, 1, 1, 1, 1,
2, 2, 1, 0, 0, 0, 0, 0,
1, 1, 2, 1, 1, 0, 0, 0,
0, 0, 0, 0, 2, 1, 1, 1,
0, 0, 0, 0, 0, 0, 1, 1,
0, 1, 1, 1, 0, 0, -1, -1,
0, 0, 0, 0, 1, 1, 0, 0,
0, -1, -1, -1, -1, 0, 0, 0,
0, 0, -1, -1, 4, 3, 2, 1,
0, 0, 1, 1, 1, 2, 3, 3,
1, 1, 0, 0, 0, 1, 1, 1,
3, 2, 1, 0, 0, 0, 0, 0,
1, 1, 3, 2, 1, 0, 0, 0,
0, 0, 0, 0, 2, 2, 1, 0,
0, 0, 0, 0, 0, 0, 2, 1,
1, 0, 0, 0, 0, 0, 0, 0,
2, 1, 1, 0, 0, 0, 0, 0,
0, 0, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, -1,
-1, 0, 0, 0, 0, -1, -2, -2,
-1, -1, -1, 0, 0, 0, 0, -1,
4, 3, 2, 1, 0, -1, -1, 0,
0, 1, 3, 2, 1, 1, 0, -1,
-1, -1, 0, 1, 3, 2, 1, 1,
0, -1, -1, -1, 0, 0, 3, 2,
1, 1, 0, -1, -1, -1, 0, 0,
2, 2, 1, 0, 0, -1, -1, -1,
-1, -1, 1, 2, 1, 0, 0, 0,
-1, -1, 0, -1, 1, 1, 1, 0,
0, 0, 0, -1, 0, -1, 0, 0,
0, -1, -1, -1, 0, -1, -1, -1,
-2, -1, -1, -2, -1, -1, -1, 0,
0, -1, -3, -3, -2, -2, -2, -1,
-1, 0, -1, -1, 5, 4, 2, 2,
1, -1, -2, -1, -1, 0, 3, 2,
2, 2, 0, -1, -2, -2, -1, 0,
3, 3, 2, 1, 0, -1, -2, -2,
-1, -1, 3, 2, 1, 1, 0, -1,
-2, -2, -1, -1, 2, 2, 1, 1,
0, -1, -2, -2, -1, -1, 1, 2,
1, 1, 0, 0, -1, -1, -1, -1,
1, 1, 1, 0, 0, 0, -1, -1,
-1, -1, -1, 0, 0, -1, -1, -1,
-1, -1, -1, -1, -3, -2, -2, -2,
-2, -1, -1, -1, -1, -1, -4, -4,
-4, -3, -2, -1, -1, 0, -1, -1,
10, 9, 7, 5, 2, -1, -2, -2,
-2, -1, 7, 5, 2, 1, 0, -1,
-2, -2, -2, -1, 3, 2, 1, 1,
0, -1, -2, -2, -2, -1, 2, 1,
1, 1, 0, -1, -2, -2, -2, -2,
1, 1, 1, 1, 0, -1, -2, -2,
-2, -1, 1, 2, 2, 1, 0, 0,
-1, -2, -2, -1, 1, 1, 1, 1,
0, 0, -1, -2, -2, -1, 0, 0,
0, 0, 0, 0, -1, -1, -2, -1,
-2, -2, -2, -1, -1, -1, -1, -1,
-1, -1, -2, -2, -2, -2, -1, -1,
-1, 0, -1, -1, 15, 14, 13, 11,
9, 6, 3, -1, -3, -3, 12, 11,
9, 7, 4, 1, -2, -2, -2, -2,
8, 7, 4, 2, 1, -1, -2, -3,
-3, -2, 4, 2, 1, 1, 0, -1,
-2, -3, -2, -2, 2, 1, 1, 1,

```



```

0, -1, -2, -2, -2, -1, 1, 1,
0, 0, 0, 0, -1, -2, -2, -2,
0, 0, 0, 0, 0, 0, -1, -2,
-2, -2, -1, -1, -1, 0, 0, 0,
-1, -1, -2, -2, -2, -1, -2, -1,
-1, -1, -1, -1, -1, -2, 1, 1,
0, 0, 1, 1, 1, 1, 0, 0 },
{ -5, -5, -4, -3, -2, -1, -1, 0,
0, 0, -5, -5, -4, -3, -2, -1,
0, 0, 0, 0, -4, -4, -3, -3,
-2, -1, 0, 1, 1, 1, -2, -2,
-1, -1, -2, -1, 0, 1, 1, 1,
-1, 0, 0, 0, 0, 0, 0, 1,
1, 1, -1, 0, 1, 1, 1, 0,
0, 1, 1, 1, -1, 0, 2, 2,
1, 1, 1, 1, 1, 1, -1, 1,
3, 3, 3, 2, 2, 1, 1, 1,
2, 4, 5, 4, 4, 3, 3, 2,
2, 2, 5, 6, 6, 6, 5, 4,
4, 3, 3, 2, -4, -4, -3, -2,
-1, -1, 0, 0, 0, 0, -4, -4,
-3, -2, -1, 0, 0, 0, 0, 1,
-4, -4, -3, -3, -1, 0, 0, 1,
1, 1, -3, -2, -2, -2, -1, 0,
1, 1, 1, 1, -2, -1, 0, 0,
-1, 0, 1, 1, 1, 1, -1, 0,
1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 2, 2, 2, 1, 1, 1,
1, 1, 0, 1, 3, 3, 3, 2,
2, 1, 1, 1, 3, 4, 5, 4,
4, 3, 3, 2, 2, 2, 6, 6,
6, 6, 5, 4, 4, 3, 3, 2,
-4, -3, -2, -1, 0, 0, 0, 0,
0, -1, -4, -3, -3, -2, -1, 0,
0, 0, 1, 0, -3, -3, -3, -2,
-1, 0, 0, 1, 1, 1, -3, -3,
-2, -2, -1, 0, 1, 1, 1, 1,
-2, -1, -1, -1, -1, 0, 1, 1,
1, 1, -1, 0, 0, 1, 1, 1,
1, 1, 1, 1, 0, 1, 2, 2,
2, 2, 1, 1, 1, 1, 1, 2,
3, 3, 3, 2, 2, 2, 1, 1,
4, 4, 5, 4, 4, 4, 3, 3,
2, 1, 5, 6, 6, 5, 5, 4,
4, 4, 3, 2, -3, -2, -1, 0,
0, 0, 0, 0, 0, -1, -3, -3,
-2, -1, 0, 0, 0, 0, 0, 0,
-3, -3, -3, -2, -1, 0, 0, 1,
1, 1, -3, -2, -2, -2, -1, 0,
1, 1, 1, 1, -2, -1, -1, -1,
-1, 1, 1, 1, 1, 1, -1, 0,
0, 1, 1, 1, 1, 1, 1, 1,
0, 1, 1, 2, 2, 2, 2, 1,
1, 1, 1, 2, 3, 3, 3, 3,
2, 1, 1, 1, 3, 4, 4, 4,
4, 3, 3, 2, 1, 1, 5, 5,
5, 4, 4, 4, 4, 3, 2, 1,
-3, -2, -1, 1, 1, 1, 0, 0,
0, -1, -3, -2, -1, 0, 0, 0,
0, 0, 0, 0, -3, -2, -2, -1,
0, 0, 0, 0, 1, 1, -2, -2,
-2, -1, 0, 0, 0, 1, 1, 1,
-2, -1, -1, -1, 0, 0, 1, 1,
1, 1, -1, 0, 0, 0, 1, 1,
1, 1, 1, 1, 0, 1, 1, 2,
2, 2, 1, 1, 1, 1, 1, 2,
2, 3, 3, 2, 2, 1, 1, 1,
3, 3, 3, 3, 3, 3, 2, 1,
1, 1, 5, 4, 4, 4, 3, 3,
3, 2, 1, 1, -3, -1, 0, 0,
1, 1, 0, 0, -1, -1, -3, -2,

```

```

-1, 0, 1, 0, 0, 0, 0, 0,
-2, -2, -1, 0, 0, 0, 0, 0,
0, 0, -2, -2, -2, -1, 0, 0,
0, 0, 1, 1, -2, -1, -1, -1,
0, 0, 1, 1, 1, 1, -1, 0,
0, 0, 1, 1, 1, 1, 1, 1,
0, 0, 1, 1, 2, 1, 1, 1,
1, 1, 1, 1, 2, 2, 2, 1,
1, 1, 1, 1, 2, 2, 3, 3,
2, 1, 1, 0, 0, 1, 4, 4,
4, 3, 2, 2, 1, 1, 1, 1,
-2, -1, 0, 0, 0, 0, -1, -1,
-2, -2, -2, -1, 0, 0, 0, 0,
-1, -1, -1, -1, -2, -2, -1, 0,
0, 0, 0, 0, 0, 0, -2, -2,
-1, -1, 0, 0, 0, 0, 0, 1,
-2, -1, -1, 0, 0, 0, 0, 0,
1, 1, -1, -1, 0, 0, 0, 0,
1, 1, 1, 1, 0, 0, 0, 1,
0, 0, 1, 1, 1, 1, 0, 0,
1, 1, 1, 0, 0, 0, 1, 1,
2, 2, 2, 2, 1, 0, 0, 0,
0, 1, 4, 3, 3, 2, 1, 0,
0, 0, 0, 1, -2, -1, -1, 0,
0, -1, -1, -2, -3, -3, -2, -1,
-1, 0, 0, -1, -2, -2, -2, -2,
-2, -2, -1, -1, 0, -1, -1, -1,
-1, 0, -2, -2, -1, -1, -1, -1,
-1, 0, 0, 0, -2, -2, -1, -1,
0, 0, 0, 0, 1, 1, -1, -1,
-1, 0, 0, 0, 0, 1, 1, 2,
-1, -1, 0, 0, 0, 0, 0, 1,
1, 2, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 0,
0, 0, 0, 0, 0, 1, 3, 3,
2, 1, 0, 0, 0, 0, 0, 1,
-2, -1, -1, -1, -1, -1, -2, -3,
-4, -4, -2, -2, -1, -1, -1, -2,
-2, -2, -3, -2, -2, -2, -1, -1,
-1, -2, -2, -1, -1, -1, -2, -2,
-2, -2, -2, -1, -1, -1, 0, 0,
-1, -1, -1, -1, -1, 0, 0, 0,
0, 1, -1, -1, -1, -1, 0, 0,
0, 1, 1, 1, -1, -1, -1, -1,
0, 0, 0, 1, 2, 2, 0, -1,
-1, 0, 0, 0, 0, 1, 2, 2,
1, 1, 0, 0, 0, 0, 0, 0,
1, 1, 3, 2, 1, 1, 0, 0,
-1, 0, 0, 1, -2, -1, -1, -2,
-2, -2, -3, -4, -4, -5, -2, -2,
-2, -2, -2, -2, -2, -3, -3, -3,
-2, -2, -2, -2, -2, -2, -2, -2,
-1, -1, -1, -2, -2, -2, -2, -2,
-1, -1, 0, 0, -1, -1, -1, -1,
-1, 0, 0, 0, 0, 1, -1, -1,
-1, -1, 0, 0, 0, 1, 1, 1,
-1, -1, -1, -1, -1, 0, 1, 2,
2, 2, -1, -1, -1, -1, 0, 0,
1, 2, 2, 3, 1, 0, 0, 0,
0, 0, 0, 1, 2, 3, 2, 1,
1, 0, 0, 0, 0, 0, 1, 2 },
{
0, -1, -1, -1, -1, -1, -1, -1,
-1, 1, 1, 0, -1, 0, 1, 1,
1, 0, 0, 0, 2, 1, 1, 1,
2, 2, 2, 0, -1, -1, 2, 2,
1, 2, 2, 2, 1, 0, -1, -1,
1, 1, 1, 1, 1, 2, 1, 0,
-1, -1, 0, -1, 0, 0, 0, 0,
0, -1, -1, -1, -1, -1, -1, 0,
0, -1, -2, -2, -1, -1, -3, -2,
-1, -1, -1, -2, -3, -3, -2, -1,

```

```

-5, -3, -2, -1, -2, -2, -3, -3,
-3, -2, -6, -5, -3, -3, -3, -4,
-4, -4, -4, -3, 0, -1, -1, -2,
-2, -2, -2, -2, -1, 1, 1, 0,
0, -1, 0, 0, -1, -1, 0, 0,
2, 1, 0, 1, 1, 1, 0, -1,
-1, 0, 3, 2, 1, 2, 2, 2,
0, -1, -1, -1, 2, 2, 2, 2,
2, 2, 0, -1, -1, -1, 0, 0,
0, 1, 1, 1, -1, -1, -1, -1,
-1, -1, -1, -1, 0, -1, -1, -2,
-1, -1, -2, -2, -2, -2, -2, -2,
-3, -2, -2, -1, -3, -3, -3, -2,
-2, -3, -3, -4, -2, 0, -5, -4,
-3, -2, -3, -4, -4, -4, -2, -1,
0, 0, -1, -1, -1, -2, -2, -1,
0, 1, 1, 0, -1, -1, -1, -1,
-1, -1, 0, 1, 2, 1, 0, 0,
0, 0, -1, -1, -1, 0, 3, 2,
1, 1, 1, 1, -1, -1, -1, -1,
2, 2, 2, 2, 2, 0, -1, -1,
-1, -1, 1, 1, 1, 1, 1, 0,
-1, -1, -1, -1, 0, 0, 0, 0,
0, -1, -2, -2, -1, -1, -1, -1,
-2, -2, -2, -2, -3, -3, -2, -1,
-2, -2, -2, -2, -3, -3, -4, -4,
-2, -1, -3, -3, -2, -2, -3, -4,
-4, -4, -2, -1, 1, 0, -1, -1,
-1, -1, -1, 0, 1, 2, 2, 0,
-1, -1, -1, -1, -1, -1, 1, 2,
2, 1, 0, 0, 0, -1, -1, 0,
0, 1, 3, 1, 1, 0, 0, 0,
-1, -1, 0, 0, 2, 2, 1, 1,
1, 0, -1, -1, -1, -1, 1, 1,
1, 1, 0, 0, -1, -1, -1, -1,
1, 0, 0, 0, 0, -1, -2, -1,
-2, -2, 0, 0, -1, -1, -1, -2,
-3, -3, -2, -2, -1, -1, -2, -1,
-2, -3, -4, -4, -4, -3, -2, -2,
-2, -1, -2, -3, -4, -5, -5, -4,
0, 0, -1, -1, -1, -1, -1, 1,
2, 3, 1, 0, 0, 0, -1, -1,
0, 1, 2, 3, 2, 1, 0, 0,
0, 0, 0, 1, 2, 2, 2, 1,
1, 0, 0, 0, 1, 1, 1, 1,
2, 1, 1, 1, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 0, 0,
-1, -1, -1, -1, 1, 1, 0, 0,
0, -1, -1, -1, -1, -2, 1, 0,
0, 0, -1, -2, -2, -2, -2, -2,
0, 0, 0, 0, -2, -3, -3, -4,
-4, -4, 0, 0, 0, -1, -2, -3,
-4, -4, -5, -5, -1, -1, -1, -1,
-1, -1, 0, 1, 3, 5, 0, 0,
0, 0, 0, 0, 0, 2, 4, 5,
0, 1, 0, 0, 0, 0, 1, 3,
3, 3, 1, 1, 1, 0, 0, 1,
2, 2, 2, 2, 1, 1, 1, 1,
1, 1, 2, 1, 1, 1, 1, 1,
2, 1, 0, 1, 1, 0, 0, 0,
1, 1, 1, 0, 0, 0, -1, -1,
-1, -1, 1, 1, 0, 0, -1, -2,
-2, -2, -2, -2, 1, 0, 0, -1,
-2, -3, -3, -3, -3, -3, 0, 0,
0, -1, -2, -3, -3, -4, -4, -5,
-2, -1, -1, -1, -1, -1, 0, 2,
4, 7, -1, -1, 0, -1, -1, 0,
1, 3, 5, 6, -1, 0, 0, 0,
0, 1, 3, 4, 5, 5, 0, 0,
1, 0, 1, 2, 3, 4, 3, 4,
0, 1, 2, 1, 1, 2, 3, 2,

```

```

2, 2, 1, 2, 2, 1, 1, 2,
1, 1, 1, 1, 1, 1, 1, 0,
0, 0, 0, 0, 0, 0, 1, 1,
0, -1, -2, -2, -2, -2, -1, -1,
1, 0, -1, -2, -3, -3, -3, -3,
-3, -3, 0, 0, -1, -2, -3, -3,
-4, -4, -4, -5, -2, -1, -1, -2,
-2, -1, 1, 3, 5, 8, -1, -1,
-1, -1, -1, 0, 2, 4, 7, 8,
-1, 0, 0, 0, 0, 1, 3, 6,
6, 7, 0, 0, 1, -1, 0, 2,
4, 5, 5, 5, 0, 1, 1, 0,
1, 3, 4, 3, 3, 3, 1, 1,
1, 0, 0, 2, 2, 2, 2, 2,
1, 1, 0, -1, 0, 0, 0, 1,
1, 1, 1, 0, -1, -2, -1, -2,
-2, -1, -1, -1, 1, 0, -2, -3,
-3, -3, -3, -3, -3, -3, 0, -1,
-2, -3, -3, -4, -4, -4, -4, -4,
-1, -1, -2, -2, -2, 0, 1, 4,
6, 9, -1, -1, -1, -2, -1, 1,
3, 5, 8, 9, 0, 0, -1, -2,
-1, 2, 4, 7, 8, 8, 0, 0,
-1, -2, -1, 3, 5, 6, 6, 7,
0, 0, -1, -1, 0, 2, 4, 4,
5, 5, 0, 0, 0, -1, -1, 1,
3, 3, 3, 3, 0, 0, -1, 0,
0, 1, 1, 1, 2, 2, 0, 0,
-1, -1, -1, -1, -1, -1, 0, 0,
1, -1, -2, -2, -2, -2, -3, -2,
-2, -2, 0, -2, -3, -3, -4, -4,
-4, -4, -4, -4, -1, -2, -2, -3,
-2, 0, 2, 5, 7, 10, 0, -2,
-2, -3, -2, 1, 3, 6, 9, 11,
0, -1, -2, -3, -2, 2, 5, 7,
9, 10, 0, -1, -2, -3, -1, 2,
6, 7, 8, 8, 0, -1, -2, -2,
-2, 1, 5, 6, 6, 6, 0, -1,
-1, -2, -2, 1, 3, 4, 4, 4,
0, -1, -1, -1, 0, 1, 2, 2,
3, 2, 0, -1, -1, 0, 0, 0,
0, 0, 1, 1, 0, -1, -2, -2,
-2, -2, -2, -2, -1, -1, 1, -1,
-3, -3, -3, -3, -3, -3, -3, -3 },
{
5, 2, 2, 3, 3, 4, 3, 1,
-1, -2, 6, 3, 3, 3, 3, 4,
3, 1, 0, -2, 4, 4, 4, 4,
3, 3, 3, 1, 0, -2, 3, 2,
2, 2, 2, 3, 3, 1, -1, -2,
1, 0, 0, 0, 1, 2, 2, 0,
-2, -3, -1, -2, -1, -1, -1, 0,
1, -1, -2, -4, -3, -3, -3, -3,
-2, -1, -1, -2, -4, -5, -5, -5,
-5, -5, -4, -3, -2, -3, -4, -5,
-6, -6, -6, -6, -5, -4, -3, -4,
-5, -6, -7, -7, -7, -6, -5, -5,
-4, -4, -5, -6, 7, 4, 2, 3,
4, 4, 3, 2, 0, -1, 7, 5,
3, 4, 4, 4, 3, 1, 0, -1,
6, 5, 5, 5, 4, 3, 2, 1,
0, -1, 4, 4, 3, 3, 3, 3,
2, 1, 0, -2, 2, 2, 2, 1,
1, 1, 2, 0, -1, -3, 1, 0,
0, -1, -1, 0, 1, -1, -2, -4,
0, -2, -2, -2, -2, -1, -1, -2,
-3, -5, -2, -3, -3, -3, -3, -3,
-2, -3, -4, -5, -3, -4, -4, -4,
-4, -4, -3, -4, -4, -5, -4, -4,
-5, -4, -4, -4, -3, -3, -4, -6,
7, 4, 2, 3, 3, 3, 2, 2,
0, -1, 6, 6, 4, 4, 4, 4,

```

2, 1, 0, -1, 6, 5, 5, 5,
 4, 3, 2, 1, 0, -1, 5, 5,
 4, 4, 3, 3, 2, 1, 0, -1,
 3, 3, 3, 2, 2, 2, 2, 0,
 -1, -2, 2, 2, 1, 1, 0, 0,
 0, -1, -2, -3, 1, 1, 0, -1,
 -1, -1, 0, -2, -3, -4, 0, -1,
 -1, -1, -2, -1, -1, -3, -4, -5,
 -1, -2, -2, -2, -2, -2, -2, -3,
 -4, -5, -2, -2, -3, -3, -3, -2,
 -2, -3, -4, -6, 6, 5, 3, 1,
 2, 2, 2, 1, 0, 0, 6, 5,
 4, 3, 3, 3, 2, 1, 0, -1,
 6, 5, 4, 4, 4, 3, 2, 0,
 -1, -1, 5, 5, 4, 4, 3, 2,
 2, 0, -1, -2, 4, 4, 3, 3,
 3, 2, 2, 0, -1, -2, 3, 2,
 2, 2, 1, 1, 1, -1, -2, -2,
 2, 1, 1, 1, 0, 0, 0, -1,
 -3, -3, 1, 1, 0, 0, 0, -1,
 -1, -2, -3, -5, 0, 0, 0, -1,
 -1, -1, -1, -3, -4, -5, 0, -1,
 -1, -1, -1, -1, -1, -3, -4, -6,
 6, 5, 3, 1, 1, 2, 1, 1,
 1, 1, 5, 4, 4, 2, 1, 2,
 2, 0, 0, 0, 5, 4, 4, 3,
 3, 2, 1, 0, -1, -1, 5, 4,
 3, 3, 2, 2, 0, 0, -1, -2,
 4, 3, 3, 2, 2, 1, 0, -1,
 -2, -2, 3, 3, 2, 2, 1, 0,
 0, -1, -2, -2, 2, 2, 1, 1,
 1, -1, -1, -3, -3, -4, 1, 1,
 1, 0, 0, -1, -2, -3, -4, -5,
 1, 0, 0, 0, -1, -1, -2, -3,
 -4, -5, 0, 0, -1, -1, -1, -2,
 -2, -4, -4, -6, 5, 4, 4, 2,
 0, 0, 1, 2, 3, 2, 5, 4,
 3, 2, 1, 1, 2, 2, 2, 1,
 5, 4, 3, 2, 1, 1, 1, 1,
 1, 0, 4, 4, 3, 2, 1, 1,
 1, 0, 0, -1, 3, 3, 2, 2,
 1, 0, 0, 0, -1, -2, 3, 2,
 2, 1, 1, -1, -1, -1, -2, -3,
 2, 1, 1, 0, 0, -2, -2, -4,
 -4, -4, 1, 1, 0, 0, -1, -2,
 -3, -5, -5, -5, 1, 0, -1, -1,
 -2, -3, -4, -5, -6, -6, 0, -1,
 -1, -2, -3, -3, -4, -6, -5, -7,
 5, 4, 4, 2, 0, -1, 1, 4,
 5, 5, 5, 4, 3, 2, 1, -1,
 2, 4, 3, 3, 4, 4, 3, 2,
 1, 1, 2, 3, 2, 2, 4, 3,
 3, 2, 1, 1, 2, 2, 1, 1,
 3, 3, 2, 2, 1, 1, 1, 1,
 0, -1, 3, 2, 1, 1, 0, 0,
 0, 0, -1, -1, 2, 1, 0, 0,
 -2, -2, -2, -3, -3, -4, 1, 0,
 0, -1, -2, -3, -4, -5, -6, -6,
 1, 0, -1, -2, -3, -4, -5, -6,
 -6, -6, 0, -1, -2, -3, -4, -5,
 -6, -6, -6, -7, 5, 4, 4, 2,
 -1, -1, 1, 5, 7, 7, 5, 4,
 4, 2, 1, 1, 2, 5, 6, 6,
 5, 4, 3, 2, 2, 2, 3, 4,
 4, 4, 4, 3, 3, 2, 2, 2,
 3, 3, 3, 2, 3, 3, 2, 1,
 1, 1, 2, 2, 2, 1, 3, 2,
 1, 1, 0, 0, 0, 1, 1, 0,
 2, 1, 0, -1, -2, -2, -2, -1,
 -2, -3, 1, 0, -1, -2, -3, -4,
 -4, -4, -4, -5, 0, -1, -2, -3,

```

-4, -5, -5, -6, -6, -6, 0, -1,
-3, -4, -5, -6, -6, -7, -6, -7,
6, 4, 3, 2, 2, 1, 2, 6,
10, 10, 5, 4, 3, 3, 3, 3,
4, 6, 8, 8, 5, 4, 3, 3,
3, 4, 4, 6, 7, 6, 4, 4,
3, 3, 3, 3, 3, 4, 5, 4,
4, 3, 2, 2, 1, 1, 1, 2,
4, 3, 3, 2, 1, 1, 0, 0,
0, 1, 2, 1, 2, 1, 0, -1,
-2, -2, -2, -1, 0, -1, 1, 0,
-1, -3, -3, -4, -4, -3, -3, -3,
0, -1, -2, -4, -5, -5, -6, -5,
-5, -5, 0, -2, -3, -4, -5, -6,
-7, -6, -5, -7, 6, 4, 4, 4,
4, 4, 5, 7, 10, 13, 5, 4,
4, 4, 5, 5, 6, 7, 10, 11,
5, 4, 4, 4, 5, 5, 6, 6,
8, 10, 4, 4, 3, 3, 4, 3,
3, 3, 5, 8, 4, 3, 2, 2,
2, 1, 1, 1, 3, 5, 3, 2,
2, 1, 1, 0, 0, 0, 2, 3,
2, 1, 1, 0, -1, -2, -2, -1,
1, 1, 1, 0, -1, -2, -3, -3,
-3, -2, -1, -1, 0, -1, -3, -4,
-5, -5, -5, -4, -3, -2, 0, -2,
-4, -5, -6, -6, -6, -4, -3, -5 },
{
-8, -9, -9, -10, -8, -6, -4, -5,
-7, -8, -6, -8, -8, -7, -6, -5,
-2, -4, -6, -7, -5, -7, -7, -6,
-5, -4, -2, -3, -4, -5, -5, -7,
-7, -6, -4, -3, -1, -2, -3, -3,
-5, -6, -6, -5, -3, -2, 0, -1,
-2, -2, -5, -6, -6, -5, -3, -2,
0, 0, 0, -1, -4, -6, -6, -4,
-3, -2, 1, 2, 2, 1, -4, -5,
-5, -5, -3, -1, 2, 3, 3, 3,
-4, -5, -5, -5, -3, 0, 3, 4,
5, 5, -4, -5, -5, -4, -3, 0,
3, 5, 6, 7, -7, -9, -10, -11,
-11, -7, -5, -4, -4, -4, -6, -8,
-8, -9, -9, -6, -4, -3, -3, -3,
-4, -7, -7, -7, -7, -5, -3, -2,
-2, -1, -4, -6, -7, -7, -6, -4,
-2, -2, -2, 0, -4, -6, -6, -6,
-5, -3, -1, -1, 0, 1, -4, -6,
-6, -6, -4, -2, 0, 1, 1, 2,
-3, -5, -6, -6, -4, -2, 1, 2,
3, 4, -2, -5, -5, -5, -3, -1,
2, 3, 4, 5, -2, -5, -5, -5,
-3, -1, 2, 4, 6, 7, -1, -4,
-4, -4, -4, -1, 2, 4, 6, 8,
-7, -9, -11, -11, -11, -9, -6, -2,
-2, -2, -6, -8, -8, -9, -10, -8,
-5, -2, -1, 0, -4, -6, -7, -7,
-8, -7, -4, -2, -1, 0, -4, -6,
-7, -6, -7, -6, -4, -1, 0, 1,
-3, -5, -6, -6, -6, -4, -2, -1,
1, 2, -3, -5, -6, -6, -5, -3,
-1, 1, 3, 3, -3, -4, -5, -5,
-5, -2, 0, 3, 4, 5, -2, -4,
-5, -5, -4, -2, 1, 4, 5, 6,
-1, -4, -4, -4, -3, -2, 1, 4,
6, 8, 1, -2, -3, -3, -3, -2,
1, 4, 5, 8, -8, -10, -11, -11,
-11, -10, -7, -1, -1, 0, -6, -8,
-9, -9, -10, -9, -6, -1, 0, 0,
-4, -6, -7, -8, -8, -8, -5, -1,
1, 1, -3, -5, -6, -6, -7, -7,
-5, -1, 1, 2, -3, -5, -5, -6,
-6, -5, -3, 0, 2, 3, -2, -4,

```

```

-5, -5, -5, -5, -2, 2, 3, 4,
-2, -4, -5, -5, -5, -3, 0, 3,
5, 5, -1, -3, -4, -4, -4, -3,
0, 3, 5, 7, 0, -3, -3, -3,
-3, -2, 0, 3, 5, 8, 2, -1,
-2, -2, -2, -2, 0, 2, 4, 9,
-8, -10, -11, -12, -11, -9, -6, -2,
-1, 0, -6, -8, -9, -9, -10, -9,
-5, -2, 0, 0, -4, -6, -7, -8,
-8, -8, -5, -2, 0, 1, -3, -5,
-6, -6, -7, -7, -4, -2, 1, 2,
-2, -4, -5, -5, -5, -5, -4, -1,
2, 3, -1, -3, -4, -5, -5, -4,
-2, 0, 3, 4, -1, -3, -4, -4,
-4, -3, -1, 1, 4, 5, 0, -2,
-4, -4, -3, -3, -1, 1, 4, 7,
1, -1, -2, -2, -2, -2, -1, 1,
4, 7, 3, 1, 0, -1, -1, -1,
0, 1, 4, 8, -7, -10, -11, -12,
-11, -9, -6, -3, -2, 0, -6, -8,
-9, -10, -10, -8, -5, -2, -1, 1,
-4, -6, -7, -8, -8, -8, -5, -2,
0, 2, -2, -5, -5, -6, -7, -7,
-5, -3, 0, 3, -2, -4, -4, -5,
-5, -6, -5, -2, 1, 3, -1, -3,
-4, -4, -4, -4, -3, -1, 3, 4,
0, -2, -3, -4, -4, -3, -2, 0,
4, 5, 1, -2, -3, -3, -3, -2,
-1, 0, 4, 6, 3, 0, -1, -2,
-2, -2, 0, 1, 4, 7, 5, 2,
1, 0, 0, 0, 1, 1, 4, 8,
-7, -10, -12, -13, -13, -10, -7, -5,
-3, 0, -5, -8, -10, -11, -11, -10,
-6, -5, -3, 1, -3, -6, -7, -8,
-9, -9, -6, -4, -2, 2, -2, -4,
-5, -7, -8, -8, -6, -4, -1, 3,
-1, -3, -4, -5, -5, -6, -5, -3,
0, 3, 0, -3, -4, -4, -4, -4,
-3, -1, 2, 4, 1, -2, -3, -4,
-3, -3, -1, 0, 2, 5, 2, -1,
-3, -3, -2, -2, 0, 1, 3, 6,
4, 1, -1, -1, -1, -1, 1, 1,
3, 6, 7, 4, 2, 1, 1, 1,
2, 2, 4, 7, -6, -11, -12, -13,
-13, -11, -8, -7, -5, 0, -4, -8,
-10, -11, -11, -10, -7, -6, -4, 0,
-3, -5, -7, -8, -9, -9, -6, -5,
-4, 1, -1, -4, -5, -6, -7, -8,
-6, -4, -2, 2, 0, -3, -4, -4,
-5, -6, -4, -2, -1, 3, 1, -2,
-3, -3, -3, -3, -2, 0, 1, 3,
1, -2, -3, -3, -2, -2, 0, 1,
1, 4, 3, -1, -2, -2, -1, -1,
1, 1, 2, 5, 6, 2, 0, 0,
0, 0, 2, 2, 3, 6, 9, 5,
3, 2, 2, 2, 3, 2, 3, 7,
-5, -10, -12, -13, -13, -11, -10, -9,
-6, -1, -3, -7, -9, -10, -11, -10,
-8, -7, -5, -1, -1, -4, -6, -8,
-8, -8, -6, -5, -4, 1, 0, -2,
-4, -5, -6, -6, -5, -4, -3, 1,
2, -1, -2, -3, -4, -5, -3, -2,
-2, 2, 3, 0, -1, -1, -2, -2,
0, 0, 0, 3, 4, 0, -1, -1,
0, 0, 2, 1, 0, 3, 6, 1,
0, 0, 1, 1, 2, 1, 1, 4,
8, 4, 2, 2, 3, 3, 3, 2,
2, 5, 11, 7, 5, 5, 5, 5,
4, 2, 3, 6, -5, -9, -11, -12,
-12, -12, -11, -10, -8, -2, -2, -6,
-8, -9, -10, -9, -8, -8, -7, -2,

```

```

1, -3, -5, -6, -7, -7, -6, -5,
-5, 0, 2, -1, -3, -4, -4, -4,
-4, -4, -3, 1, 4, 1, 0, -1,
-1, -2, -1, -1, -2, 1, 5, 3,
2, 1, 1, 1, 1, 0, -1, 2,
7, 4, 2, 3, 3, 3, 3, 1,
-1, 3, 9, 4, 3, 4, 4, 4,
4, 2, 0, 4, 11, 7, 5, 5,
6, 5, 5, 3, 1, 5, 14, 10,
8, 8, 8, 7, 6, 3, 3, 6 },
{
1, 1, 1, 1, -1, -2, -10, -14, -15,
-11, -3, 0, 1, 0, -1, -1, -8,
-12, -12, -8, -1, 0, 0, 0, -1,
-1, -6, -9, -8, -4, 1, -1, -1,
-1, -1, -1, -3, -6, -4, 1, 3,
-1, -1, -1, -1, 0, 0, -2, 1,
5, 6, 0, -1, -1, 0, 1, 1,
1, 3, 6, 7, 0, -1, 1, 4,
5, 4, 4, 6, 7, 8, 1, 2,
5, 8, 10, 8, 6, 7, 6, 6,
2, 6, 10, 12, 13, 11, 7, 7,
6, 6, 4, 10, 13, 15, 15, 12,
9, 7, 6, 7, -2, -1, 0, 0,
-2, -10, -14, -14, -9, -1, -2, -1,
0, 0, -1, -8, -11, -11, -6, 0,
-2, -1, 0, 0, 0, -6, -9, -8,
-2, 2, -2, -1, 0, 0, 0, -4,
-6, -4, 1, 4, -1, 0, 0, 0,
1, -1, -2, 0, 5, 6, 0, 0,
1, 1, 2, 2, 1, 3, 6, 7,
-1, 1, 3, 5, 7, 5, 3, 5,
6, 7, -1, 4, 6, 8, 11, 8,
5, 6, 6, 7, 2, 8, 11, 12,
12, 10, 7, 7, 6, 7, 3, 12,
14, 13, 12, 11, 9, 7, 7, 8,
-6, -4, -2, -2, -2, -10, -13, -13,
-9, -1, -5, -3, -2, -1, -1, -8,
-11, -11, -6, 1, -6, -3, -1, 0,
-1, -6, -9, -5, -1, 3, -7, -2,
-1, 0, 0, -4, -4, -1, 2, 4,
-6, -2, -1, 0, 1, -2, -2, 1,
4, 6, -4, -1, 1, 2, 3, 1,
1, 3, 6, 7, -5, 0, 2, 5,
7, 4, 3, 4, 6, 7, -7, 3,
6, 8, 9, 6, 4, 5, 6, 8,
-6, 6, 10, 11, 10, 8, 6, 6,
7, 9, -1, 7, 12, 12, 10, 9,
8, 7, 8, 8, -12, -8, -6, -4,
-3, -9, -13, -12, -8, 0, -11, -6,
-4, -3, -2, -8, -11, -8, -3, 2,
-10, -5, -4, -2, -2, -6, -7, -3,
3, 4, -10, -4, -3, -2, -1, -4,
-3, 1, 5, 6, -9, -4, -2, -1,
0, -3, 0, 3, 5, 6, -9, -2,
-1, 1, 2, 0, 1, 3, 5, 7,
-12, -2, 1, 4, 4, 2, 1, 4,
6, 8, -15, -2, 5, 6, 5, 4,
3, 4, 6, 9, -13, -1, 6, 8,
6, 6, 5, 6, 7, 9, -1, 0,
7, 10, 6, 8, 7, 7, 8, 9,
-15, -14, -11, -8, -6, -9, -12, -11,
-5, 2, -13, -12, -8, -6, -5, -9,
-10, -5, 1, 5, -11, -9, -6, -5,
-4, -8, -7, 0, 5, 8, -10, -7,
-5, -5, -4, -6, -4, 3, 6, 8,
-8, -6, -4, -3, -3, -5, -2, 3,
5, 7, -10, -5, -2, -1, 0, -1,
0, 2, 5, 7, -16, -8, -1, 1,
0, 0, 0, 3, 5, 8, -17, -8,
0, 3, 1, 2, 3, 4, 6, 9,
-12, -7, 0, 4, 2, 3, 6, 6,

```



```

7, 9, 2, 0, 2, 5, 2, 4,
7, 7, 8, 9, -13, -17, -16, -13,
-9, -13, -14, -8, -3, 4, -10, -13,
-13, -11, -9, -12, -12, -5, 2, 8,
-8, -10, -10, -9, -8, -11, -10, -3,
4, 10, -6, -7, -8, -7, -7, -10,
-6, -1, 6, 10, -5, -4, -6, -5,
-5, -7, -3, 1, 5, 8, -6, -4,
-4, -3, -3, -4, -1, 1, 4, 8,
-11, -7, -2, -1, -2, -2, 0, 2,
5, 10, -10, -7, -1, 0, -2, -1,
2, 3, 5, 10, -5, -5, -1, 1,
-1, 1, 4, 5, 7, 11, 3, 1,
2, 1, -1, 2, 4, 6, 8, 11,
-11, -15, -19, -16, -14, -15, -20, -13,
-2, 7, -8, -11, -15, -15, -13, -14,
-19, -11, 0, 10, -5, -8, -12, -13,
-12, -13, -16, -9, 2, 12, -3, -5,
-8, -11, -11, -12, -13, -7, 4, 11,
-1, -4, -6, -9, -10, -10, -7, -2,
3, 8, -3, -4, -5, -6, -6, -5,
-3, -1, 3, 8, -5, -3, -2, -3,
-4, -3, 0, 1, 3, 10, -5, -3,
-1, -1, -3, -1, 1, 3, 5, 10,
0, -2, 1, -1, -2, 0, 2, 4,
7, 10, 5, 3, 4, 2, -2, 0,
2, 4, 6, 10, -9, -13, -17, -18,
-15, -16, -22, -23, -10, 7, -6, -9,
-13, -16, -15, -15, -19, -20, -6, 10,
-3, -6, -10, -15, -14, -14, -16, -16,
-3, 11, -1, -4, -7, -12, -13, -13,
-12, -11, -1, 10, 0, -3, -6, -9,
-12, -11, -6, -4, 1, 7, -2, -3,
-5, -6, -6, -6, -2, -1, 1, 2,
-3, -2, -2, -2, -4, -3, 0, 1,
3, 7, -2, -1, 0, 0, -2, -1,
1, 2, 5, 10, 2, 1, 1, 1,
-2, -1, 1, 3, 5, 9, 6, 5,
4, 1, -2, 0, 1, 3, 5, 8,
-6, -11, -15, -18, -16, -16, -21, -21,
-16, 3, -3, -7, -11, -16, -16, -16,
-17, -17, -12, 6, -1, -4, -8, -14,
-15, -16, -14, -13, -7, 7, 0, -2,
-6, -11, -13, -14, -10, -7, -3, 7,
0, -3, -5, -9, -11, -12, -6, -1,
1, 6, -1, -3, -4, -4, -4, -6,
-2, 0, 1, -5, -3, -2, -2, -2,
-2, -2, 0, 2, 4, 1, 0, 0,
1, 0, -1, 0, 1, 3, 6, 10,
3, 2, 2, 1, -1, 0, 2, 4,
6, 9, 7, 6, 5, 1, -1, 0,
1, 4, 5, 9, -4, -9, -14, -18,
-19, -19, -18, -14, -8, 2, -1, -6,
-10, -15, -17, -17, -15, -10, -4, 4,
0, -3, -7, -13, -14, -16, -12, -6,
-1, 6, 1, -2, -6, -11, -12, -14,
-9, -3, 2, 6, 0, -2, -6, -9,
-9, -11, -5, 1, 4, 6, -1, -3,
-4, -4, -2, -2, 0, 3, 2, -3,
-2, -2, -1, -2, -1, 0, 2, 4,
6, -1, -1, 0, 1, 1, 0, 1,
4, 6, 8, 11, 4, 3, 4, 2,
0, 2, 4, 7, 8, 11, 7, 5,
5, 2, 0, 2, 5, 7, 7, 10 },

```

```

},
{

```

```

{ 1, 1, 0, 0, 1, 1, -1, -1,
0, 0, 1, 1, 0, 0, 0, 0,
-1, -1, -1, -1, 1, 1, 1, 0,
0, 0, -1, -1, -1, -1, 1, 1,
1, 0, 0, 0, -1, -1, -1, -1,

```

```

2, 1, 1, 1, 0, 0, -1, -1,
-1, -1, 3, 2, 1, 1, 1, 1,
0, -1, -1, -1, 3, 2, 1, 0,
0, 0, 0, 0, 0, 0, 2, 2,
1, 0, -1, -2, -2, 0, 0, 0,
2, 1, 1, 0, -1, -3, -4, -3,
-1, 0, 2, 1, 0, 0, -2, -3,
-5, -5, -3, -1, 1, 1, 1, 1,
1, 0, 0, 0, -1, -1, 0, 0,
0, 0, 1, 0, 0, -1, -1, -1,
0, 0, 0, 0, 0, 0, -1, -1,
-1, -1, 1, 1, 1, 0, 0, 0,
-1, -1, -1, -1, 2, 2, 1, 0,
0, 0, -1, -1, -1, -1, 2, 2,
1, 1, 0, 0, 0, -1, -1, -1,
2, 2, 1, 0, 0, -1, 0, -1,
-1, -1, 2, 1, 1, 0, -1, -2,
-2, -1, 0, -1, 2, 1, 0, 0,
-1, -3, -4, -3, -2, 0, 1, 0,
-1, -1, -2, -4, -4, -4, -3, -2,
1, 1, 1, 1, 0, 0, 0, -1,
-1, -1, 0, 0, 1, 0, 0, 0,
0, -1, -1, -1, 1, 0, 1, 0,
0, 0, 0, -1, -1, -1, 1, 1,
1, 0, 0, 0, -1, -1, -1, -1,
2, 1, 1, 1, 0, -1, -1, -1,
-1, -1, 2, 2, 1, 1, 0, -1,
-1, -1, -1, -1, 2, 2, 1, 1,
0, -1, -1, -1, -1, -1, 2, 1,
1, 0, -1, -2, -3, -2, -1, -1,
2, 1, 0, 0, -2, -3, -3, -3,
-2, -1, 0, 0, -1, -1, -2, -4,
-4, -4, -3, -2, 1, 1, 1, 0,
0, 0, 0, -1, 0, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, -1,
1, 0, 0, 0, 0, 0, -1, -1,
-1, -1, 1, 1, 1, 0, 0, 0,
-1, -1, -1, -1, 1, 1, 1, 0,
0, -1, -1, -1, -1, -1, 2, 1,
1, 0, 0, -1, -1, -1, -1, -1,
2, 1, 1, 0, 0, -1, -2, -1,
-1, -1, 2, 1, 1, 0, -1, -2,
-2, -2, -1, 0, 1, 1, 0, -1,
-1, -2, -3, -2, -2, -1, 1, 0,
-1, -2, -2, -3, -3, -3, -2, -2,
1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, 0, 0, 0, 0, 0,
0, 0, -1, -1, -1, -1, 0, 0,
0, 0, 0, 0, -1, -1, -1, -1,
1, 1, 1, 0, 0, 0, -1, -1,
-1, -1, 2, 1, 1, 1, 0, -1,
-1, 0, -1, -1, 2, 1, 1, 1,
0, -1, -1, 0, 0, -1, 1, 1,
0, 0, 0, -1, -1, -1, 0, 0,
1, 0, 0, 0, -1, -2, -2, -1,
-1, 0, 0, 0, -1, -1, -2, -2,
-3, -2, -2, -1, 0, 0, 1, 1,
2, 1, 1, 0, 1, 0, 0, 0,
0, 0, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, -1,
-1, -1, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, 1, 1, 1, 0,
0, 0, 0, -1, -1, -1, 1, 1,
1, 1, 0, 0, 0, 0, -1, -1,
1, 1, 1, 1, 0, 0, 0, 0,
0, -1, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, -1, -1, 0, 0, 0, 0, -1,
0, -1, -1, -2, -2, -1, 0, 0,
0, 1, 2, 2, 2, 2, 1, 1,

```

```

1, 0, 0, 0, 1, 1, 2, 1,
1, 0, 0, 0, 0, 0, 0, 0,
1, 1, 0, 0, -1, -1, 0, 0,
0, 0, 1, 1, 0, -1, -1, -1,
1, 1, 0, 0, 0, 0, 0, -1,
-1, -1, 1, 1, 1, 1, 1, 0,
0, 0, -1, -1, 1, 1, 1, 1,
1, 0, 0, 0, 0, -1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0,
1, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 2, 2, 3,
4, 3, 2, 1, 1, 0, 0, 1,
2, 3, 3, 3, 1, 1, 0, 0,
0, 0, 1, 1, 2, 2, 1, 0,
0, -1, 0, 0, 0, 1, 2, 2,
1, -1, -1, -1, 0, 0, 0, 0,
1, 1, 0, -1, -1, -2, 1, 1,
1, 1, 1, 1, 0, 0, -1, -1,
1, 1, 1, 1, 1, 0, 0, 0,
-1, -1, 1, 1, 1, 1, 1, 0,
0, 0, 0, 0, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 1, 0,
0, 0, 0, 0, 0, 0, 0, 1,
1, 2, 3, 4, 4, 4, 3, 2,
1, 1, 0, 1, 2, 3, 4, 3,
2, 1, 1, -1, 0, 1, 1, 2,
3, 3, 1, 0, 0, -1, 0, 0,
0, 1, 2, 2, 1, 0, -1, -2,
0, 0, 0, 1, 1, 1, 0, 0,
-1, -1, 1, 1, 0, 1, 1, 0,
0, 0, 0, -1, 1, 1, 1, 1,
1, 0, 0, 1, 0, 0, 1, 1,
1, 1, 1, 0, 1, 1, 1, 0,
1, 1, 1, 1, 0, 0, 1, 1,
0, 0, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 3, 4, 4, 4,
4, 4, 3, 2, 1, 1, 1, 2,
3, 3, 4, 4, 2, 1, 1, 0,
0, 1, 2, 2, 3, 3, 2, 0,
-1, -1, 0, 0, 1, 1, 2, 1,
0, 0, -1, -2, 0, 0, 0, 1,
1, 0, 0, 0, -1, -1, 1, 0,
0, 1, 1, 0, 0, 0, 0, -1,
1, 1, 1, 1, 0, 0, 0, 0,
0, -1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0 },
{
1, 0, 0, -1, -2, -3, -3, -3,
-3, -3, 1, 0, 0, -1, -1, -2,
-3, -3, -3, -3, 1, 1, 1, 1,
-1, -2, -2, -2, -3, -3, 1, 1,
2, 2, 1, -1, -2, -2, -2, -3,
2, 2, 2, 2, 2, 1, -1, -2,
-2, -3, 2, 3, 3, 3, 3, 2,
1, -1, -2, -3, 2, 3, 3, 4,
4, 4, 2, 0, -2, -3, 2, 3,
4, 4, 5, 4, 3, 0, -2, -4,
2, 3, 4, 4, 4, 4, 1, -1,
-2, -4, 2, 3, 3, 3, 3, 2,
0, -2, -2, -3, 0, 0, 0, -1,
-2, -3, -3, -3, -3, -2, 0, 0,
0, -1, -1, -2, -3, -3, -2, -2,
0, 0, 0, 0, -1, -2, -2, -2,
-2, -2, 0, 0, 0, 0, -1, -1,
-2, -2, -2, -3, 0, 1, 1, 1,
1, 0, -1, -2, -2, -3, 0, 1,
1, 2, 2, 2, 0, -1, -2, -3,
1, 1, 2, 3, 3, 3, 1, -1,
-2, -3, 0, 1, 2, 3, 4, 3,

```

```

2, 0, -2, -4, 1, 2, 3, 3,
3, 3, 1, -1, -2, -4, 0, 1,
1, 2, 2, 1, 0, -1, -2, -3,
0, 0, -1, -1, -1, -2, -3, -2,
-2, -2, 0, 0, -1, -1, -2, -2,
-3, -2, -2, -2, 0, 0, 0, -1,
-1, -2, -2, -2, -2, -2, 0, 0,
0, 0, -1, -2, -2, -2, -2, -2,
0, 0, 0, 0, 0, -1, -1, -2,
-2, -2, 0, 0, 0, 0, 1, 1,
0, -2, -2, -3, 0, 0, 1, 1,
2, 2, 1, -1, -2, -3, 0, 0,
1, 2, 2, 2, 1, -1, -3, -4,
0, 0, 1, 1, 1, 1, 0, -1,
-2, -3, 0, 0, 0, 0, 1, 1,
-1, -1, -2, -3, 0, -1, -1, -1,
-2, -2, -2, -1, -1, -1, 0, -1,
-1, -1, -2, -2, -2, -1, -1, -1,
0, 0, -1, -1, -2, -2, -2, -1,
-1, -1, 0, 0, -1, -1, -1, -1,
-2, -1, -1, -1, 0, -1, -1, -1,
-1, -1, -1, -1, -1, -2, 0, 0,
0, 0, 0, 0, -1, -1, -2, -2,
0, 0, 0, 0, 0, 0, 0, -1,
-2, -2, -1, 0, 0, 0, 1, 1,
0, -1, -2, -3, -1, 0, 0, 0,
0, 0, 0, -1, -2, -2, -1, -1,
-1, 0, 0, 0, 0, 0, -1, -2,
0, -1, -1, -2, -2, -2, -1, -1,
0, 0, 0, -1, -1, -2, -2, -1,
-1, -1, 0, 0, 0, 0, -1, -2,
-2, -1, -1, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, 0, 0, 0,
0, 0, 0, 0, 0, 0, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, -1, -1, 0, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, -1, -1,
-1, -1, 0, 0, 0, 0, 0, 0,
0, -1, -1, -1, -1, 0, 1, 0,
-1, -1, -2, -2, -1, 0, 0, 1,
0, 0, 0, -1, -2, -1, -1, 0,
0, 1, 0, 0, 0, 0, -1, -1,
0, 0, 0, 0, 0, 0, 1, 0,
0, -1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1,
0, 0, 0, 0, -1, 0, 0, 0,
1, 1, 1, 0, 0, 0, -1, -1,
0, 0, 1, 1, 1, 1, 0, 0,
0, 1, 1, 0, 0, -1, -1, -2,
-1, -1, 0, 1, 1, 1, 0, -1,
-1, -1, -1, -1, 0, 1, 1, 1,
1, 0, -1, -1, -1, 0, 0, 1,
1, 1, 1, 0, -1, -1, -1, 0,
0, 1, 1, 1, 1, 1, 0, -1,
-1, 0, 0, 1, 0, 0, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0,
0, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 1, 1, 2, 1, 0,
0, 0, 0, 0, 0, 1, 2, 2,
1, 0, 0, 0, 1, 1, 0, 0,
0, -1, -1, 0, 0, 0, 1, 1,
0, 0, -1, -1, -1, -1, 0, 0,
1, 1, 0, 0, -1, -2, -2, -2,
-1, 0, 2, 1, 1, 0, -1, -2,
-2, -2, -1, 0, 2, 1, 1, 0,

```

```

-1, -2, -2, -1, -1, 1, 1, 1,
1, 0, -1, -2, -2, -1, 0, 1,
1, 1, 1, 1, 0, -1, -1, -1,
0, 1, 0, 1, 1, 1, 1, 1,
0, 0, 0, 1, 0, 1, 1, 1,
2, 2, 1, 0, 0, 0, 0, 0,
1, 2, 2, 2, 1, 0, 0, 0,
1, 1, 1, 1, 0, 0, -1, -1,
-1, -1, 1, 1, 1, 1, 0, -1,
-2, -2, -2, -1, 2, 1, 0, 0,
-1, -2, -3, -2, -2, -1, 2, 1,
0, 0, -2, -3, -3, -3, -2, -1,
2, 1, 0, -1, -2, -3, -3, -3,
-2, -1, 1, 1, 0, 0, -1, -3,
-3, -3, -1, 0, 1, 1, 1, 0,
0, -2, -2, -2, -1, 1, 1, 1,
1, 1, 1, 0, -1, -1, 0, 1,
0, 1, 1, 1, 2, 1, 1, 0,
0, 1, 0, 1, 1, 2, 2, 2,
1, 0, 0, 0, 2, 2, 1, 1,
1, 0, -1, -2, -3, -2, 2, 1,
1, 1, 0, -1, -3, -3, -3, -2,
2, 1, 1, 0, -1, -2, -3, -3,
-3, -3, 2, 1, 0, 0, -2, -3,
-4, -4, -4, -2, 1, 1, 0, -1,
-3, -4, -4, -5, -3, -2, 1, 1,
0, -1, -2, -3, -4, -4, -2, -2,
1, 1, 0, 0, -1, -2, -3, -3,
-2, -1, 1, 1, 1, 1, 0, -1,
-2, -2, -1, 0, 1, 1, 1, 1,
1, 1, 0, -1, 0, 0, 1, 1,
1, 2, 2, 2, 2, 0, 0, 1 },
{ 0, -1, -2, -1, -1, 0, 0, 0,
-2, -5, 0, -1, -2, -1, 0, 0,
1, 0, -2, -6, 0, -1, -2, -2,
-1, 1, 1, 0, -1, -6, 0, -1,
-2, -1, 0, 1, 2, 1, -1, -6,
1, 0, -1, 0, 1, 1, 0, 1,
-1, -6, 1, 0, 1, 2, 2, 2,
0, -1, -2, -4, 0, 0, 1, 3,
4, 3, 1, -1, -2, -3, -1, 0,
1, 3, 4, 4, 2, 1, 0, -1,
-1, -1, 0, 1, 1, 1, 1, 1,
1, 0, -1, -1, -2, -2, -1, -1,
0, -1, -1, 0, 0, -1, -2, -2,
-1, 0, 0, 0, -1, -4, 0, -1,
-2, -2, -1, 0, 1, 0, -1, -5,
1, -1, -2, -2, -1, 0, 0, 0,
-2, -5, 1, -1, -2, -2, -1, 0,
1, 0, -2, -5, 1, 0, -1, -1,
0, 1, 0, -1, -2, -4, 1, 0,
0, 1, 2, 2, 0, -2, -2, -3,
0, -1, 0, 2, 3, 3, 1, 0,
-1, -2, -1, -1, 0, 1, 3, 3,
2, 1, 0, 0, 0, -1, -1, 0,
1, 1, 1, 1, 1, 1, -1, -1,
-2, -2, -2, -1, -1, -1, 0, 1,
0, -1, -2, -2, -2, -1, 0, 0,
0, -3, 1, -1, -2, -3, -2, -1,
0, 0, -1, -3, 1, 0, -2, -2,
-2, -1, 0, 0, -2, -3, 1, 0,
-1, -2, -1, -1, 0, -1, -3, -3,
2, 0, 0, -1, -1, 0, 0, -1,
-3, -3, 1, 0, 0, 0, 1, 2,
0, -1, -2, -2, 0, 0, 0, 0,
2, 3, 1, 0, 0, 0, 0, -1,
-1, 0, 2, 2, 1, 1, 1, 2,
0, 0, -1, 0, 1, 0, 0, 0,
2, 2, 0, -1, -1, -2, -2, -2,
-2, 0, 1, 2, 0, -1, -2, -3,
-2, -1, -1, 0, 0, -1, 1, -1,

```

```

-2, -3, -2, -1, 0, 0, -1, -1,
1, 0, -1, -2, -2, -1, 0, -1,
-1, -1, 2, 1, -1, -1, -2, -1,
0, -1, -2, -1, 2, 1, 0, 0,
-1, 0, 0, -1, -2, -1, 1, 0,
0, 0, 0, 1, 1, -1, -1, -1,
0, 0, 0, 0, 1, 2, 2, 1,
1, 1, 0, -1, -1, 0, 1, 1,
0, 1, 2, 3, 0, 0, -1, -1,
0, -1, -1, 0, 2, 3, 0, 0,
-1, -2, -2, -2, -1, 0, 1, 3,
-1, -2, -3, -3, -3, -2, -1, 0,
0, 0, 0, -1, -2, -3, -2, -2,
-1, 0, 0, 0, 1, 0, -2, -2,
-2, -2, -1, 0, 0, 0, 2, 1,
-1, -1, -2, -2, -1, 0, 0, 0,
1, 1, 0, 0, -1, -1, 0, 0,
0, 0, 1, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 0, 0, 0,
0, 0, 1, 1, 2, 2, 0, 0,
-1, -1, -1, 0, 0, 1, 2, 3,
1, 0, -1, -1, -1, -1, -1, 0,
2, 3, 1, 1, 0, -1, -2, -2,
-1, 0, 2, 3, -3, -4, -4, -3,
-3, -2, -1, 0, 0, 1, -1, -3,
-3, -3, -3, -2, -1, 0, 0, 1,
0, -2, -2, -3, -3, -2, -1, 0,
0, 1, 0, -1, -1, -1, -2, -2,
-1, 0, 0, 1, 1, 0, 0, 0,
-1, -1, 0, 0, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 1, 1,
1, 1, 0, 0, 0, 0, 0, 1,
2, 2, 1, 1, 0, 0, 0, 0,
0, 1, 2, 3, 2, 1, 0, 0,
0, -1, -1, 0, 2, 3, 2, 2,
1, 0, -1, -1, -1, 0, 1, 3,
-4, -5, -5, -4, -4, -3, -2, -1,
0, 1, -2, -3, -3, -4, -3, -3,
-2, -1, 0, 1, -1, -1, -2, -3,
-3, -3, -2, -1, 0, 1, 0, 0,
-1, -1, -2, -2, -2, -1, -1, 1,
1, 1, 0, 0, -1, -1, 0, -1,
0, 0, 2, 2, 1, 1, 0, 0,
0, 0, 1, 0, 2, 2, 1, 1,
0, 0, 0, 0, 1, 1, 3, 2,
1, 1, 1, 0, -1, 0, 1, 1,
3, 2, 2, 1, 0, 0, -1, 0,
1, 1, 3, 3, 2, 1, 0, -1,
-1, 0, 0, 1, -5, -6, -5, -5,
-4, -4, -2, -1, -1, 1, -2, -3,
-4, -4, -3, -3, -2, -2, -1, 0,
0, -1, -2, -2, -2, -2, -2, -2,
-1, 0, 1, 0, -1, -1, -1, -1,
-1, -2, -2, 0, 3, 2, 1, 0,
0, 0, 1, -1, -1, 0, 4, 3,
2, 2, 1, 1, 1, -1, 0, 0,
4, 4, 3, 2, 2, 1, 0, 0,
0, 1, 5, 4, 3, 3, 2, 1,
-1, -1, 1, 1, 5, 5, 4, 3,
2, 0, -1, -1, 0, 1, 5, 5,
3, 3, 1, -1, -1, -1, 0, 0,
-4, -5, -5, -5, -4, -3, -3, -2,
-1, 0, -2, -3, -3, -3, -3, -3,
-2, -3, -2, 0, 0, 0, -1, -1,
-2, -2, -2, -3, -2, -1, 2, 2,
1, 0, 0, 0, -1, -2, -2, -1,
4, 4, 3, 2, 2, 1, 0, -1,
-2, -1, 6, 5, 5, 4, 3, 2,
0, -2, -1, 0, 6, 6, 5, 5,
4, 2, -1, -1, 0, 0, 7, 6,
6, 5, 4, 1, -1, -1, 0, 1,

```

```

7, 7, 6, 5, 3, 0, -1, -1,
0, 1, 6, 6, 5, 4, 2, -1,
-2, -1, -1, 1, -3, -4, -4, -4,
-4, -3, -3, -3, -2, 0, -1, -2,
-2, -2, -3, -2, -3, -3, -3, -1,
2, 1, 0, 0, -1, -1, -1, -3,
-3, -2, 4, 3, 3, 2, 1, 1,
0, -2, -3, -2, 6, 5, 5, 4,
4, 2, 0, -1, -2, -1, 8, 7,
6, 6, 5, 2, -1, -2, -1, -1,
8, 8, 8, 7, 5, 2, -2, -1,
-1, 0, 8, 8, 8, 7, 5, 1,
-2, -1, 0, 0, 8, 8, 7, 7,
3, 0, -2, -1, 0, 0, 8, 8,
7, 4, 2, -1, -2, -1, 0, 1 },
{
-4, -5, -5, -6, -6, -5, -5, -4,
-3, -2, -3, -4, -5, -5, -5, -4,
-3, -2, -1, 0, -2, -4, -4, -4,
-4, -2, -1, 0, 1, 2, -2, -3,
-4, -4, -2, -1, 0, 2, 3, 4,
-1, -3, -3, -3, -2, 0, 1, 3,
5, 6, 0, -2, -3, -2, -1, 0,
2, 3, 6, 7, 0, -1, -2, -2,
-1, 1, 3, 3, 6, 9, 1, 0,
-1, -1, 0, 2, 4, 5, 8, 10,
2, 1, 0, 0, 1, 3, 5, 7,
9, 11, 3, 2, 1, 1, 2, 3,
5, 7, 10, 12, -3, -4, -4, -5,
-6, -5, -5, -4, -4, -6, -2, -3,
-4, -4, -4, -4, -3, -2, -3, -4,
-2, -3, -3, -3, -3, -2, -1, -1,
-2, -2, -1, -3, -3, -3, -2, -1,
0, 1, 0, 0, 0, -2, -3, -3,
-2, 0, 2, 3, 3, 2, 0, -1,
-3, -3, -1, 1, 3, 4, 4, 4,
1, -1, -2, -2, -1, 2, 3, 4,
5, 5, 1, -1, -2, -2, 0, 2,
4, 5, 6, 6, 0, -1, -2, -2,
-1, 1, 3, 4, 6, 7, 1, -1,
-1, -1, 0, 1, 3, 4, 7, 8,
-2, -2, -3, -4, -5, -6, -6, -7,
-8, -8, -1, -2, -3, -3, -4, -5,
-5, -5, -6, -6, 0, -2, -2, -3,
-2, -3, -3, -4, -4, -5, 0, -1,
-2, -2, -2, -1, -1, -2, -2, -3,
1, -1, -2, -2, -2, 0, 0, 0,
0, -1, 1, 0, -1, -2, -1, 1,
2, 2, 1, 1, 2, 0, -1, -2,
-1, 2, 3, 3, 2, 2, 2, 1,
-1, -1, -1, 1, 3, 3, 3, 3,
1, 0, -1, -1, -1, 0, 2, 3,
3, 4, 1, 0, -1, -1, -1, 0,
1, 2, 4, 4, 0, -1, -2, -3,
-4, -6, -8, -9, -9, -9, 0, -1,
-1, -2, -4, -5, -7, -7, -7, -7,
1, -1, -1, -2, -3, -4, -5, -5,
-6, -6, 1, 0, -1, -2, -2, -3,
-3, -4, -4, -5, 2, 0, -1, -2,
-2, -1, -1, -2, -3, -3, 2, 1,
0, -1, -1, 0, 0, 0, -1, -2,
3, 1, 0, 0, -1, 1, 1, 1,
0, 0, 3, 2, 1, 0, -1, 0,
1, 1, 1, 0, 2, 1, 0, 0,
-1, 0, 0, 1, 1, 1, 2, 1,
0, 0, -1, -1, 0, 1, 1, 2,
1, 0, 0, -1, -3, -5, -7, -8,
-8, -8, 1, 0, 0, -1, -2, -4,
-6, -8, -8, -8, 2, 0, 0, -1,
-2, -4, -5, -6, -6, -7, 2, 1,
0, -1, -2, -3, -4, -5, -5, -6,
3, 2, 1, 0, -1, -2, -3, -3,

```

```

-4, -4, 3, 2, 2, 1, 0, 0,
-1, -2, -3, -3, 3, 2, 2, 2,
1, 1, 0, -1, -2, -2, 3, 3,
3, 2, 1, 0, 0, -1, -1, -2,
4, 3, 3, 2, 1, 0, -1, -1,
-1, -1, 3, 3, 2, 2, 1, -1,
-1, -1, -1, 0, 1, 0, 0, 0,
-2, -3, -5, -6, -7, -8, 1, 0,
0, 0, -1, -3, -4, -6, -7, -8,
2, 0, 0, 0, -1, -2, -4, -5,
-6, -7, 2, 1, 0, 0, 0, -2,
-3, -4, -5, -6, 3, 2, 1, 0,
0, -1, -2, -3, -4, -5, 3, 2,
2, 1, 0, 0, 0, -2, -3, -4,
3, 3, 3, 2, 1, 1, 0, -2,
-3, -4, 3, 3, 3, 3, 3, 1,
0, -1, -2, -3, 4, 3, 4, 4,
3, 1, 0, -1, -2, -3, 4, 4,
4, 3, 2, 1, 0, -1, -2, -2,
0, 0, 0, 0, 0, -1, -3, -5,
-6, -7, 1, 0, 0, 0, 0, -1,
-2, -4, -6, -7, 1, 0, 0, 0,
0, 0, -2, -4, -5, -7, 2, 0,
0, -1, 0, 0, -1, -3, -5, -6,
2, 1, 0, -1, -1, 0, 0, -2,
-4, -5, 3, 1, 1, 0, -1, 0,
1, -1, -3, -5, 3, 2, 1, 1,
1, 1, 1, -1, -3, -4, 3, 3,
3, 2, 2, 2, 1, -1, -3, -4,
4, 4, 4, 4, 3, 2, 1, -1,
-2, -3, 4, 4, 5, 4, 3, 2,
1, -1, -2, -3, -1, 0, 0, 0,
0, -1, -1, -3, -5, -7, -1, -1,
-1, -1, -1, 0, -1, -2, -5, -7,
-1, -1, -1, -1, -1, 0, 0, -1,
-4, -6, 0, -1, -2, -2, -2, -1,
0, 0, -3, -5, 1, -1, -2, -2,
-2, -1, 1, 1, -2, -5, 1, -1,
-1, -2, -2, -1, 1, 1, -2, -4,
2, 0, -1, -1, -1, 0, 1, 1,
-2, -4, 2, 1, 1, 0, 1, 1,
1, 0, -2, -4, 3, 3, 3, 2,
2, 2, 1, 0, -2, -4, 5, 5,
4, 3, 3, 2, 1, 0, -2, -3,
-3, -2, -2, -1, 0, 0, 0, -2,
-6, -9, -3, -2, -2, -2, -1, -1,
1, 0, -4, -7, -3, -3, -3, -3,
-2, -1, 1, 1, -2, -7, -2, -3,
-3, -3, -3, -2, 0, 2, 0, -5,
-2, -2, -3, -3, -3, -1, 1, 3,
1, -3, -1, -2, -3, -3, -2, -1,
1, 3, 1, -2, 0, -2, -2, -2,
-1, 0, 1, 2, 0, -3, 1, -1,
-1, 0, 0, 1, 1, 1, 0, -3,
2, 1, 1, 1, 1, 1, 1, 1,
-1, -3, 4, 4, 3, 3, 3, 2,
1, 1, -1, -3, -7, -5, -4, -3,
-2, -2, -2, -3, -6, -9, -6, -5,
-5, -4, -3, -2, -1, -1, -5, -8,
-6, -5, -5, -4, -4, -2, 0, 1,
-2, -7, -5, -5, -5, -4, -4, -2,
0, 1, 0, -5, -5, -4, -4, -4,
-3, -1, 1, 2, 1, -3, -4, -4,
-4, -3, -2, 0, 2, 3, 2, -2,
-3, -3, -3, -2, -1, 1, 2, 3,
1, -2, -2, -2, -1, -1, 0, 1,
2, 2, 1, -3, 0, 0, 0, 1,
1, 1, 2, 1, 0, -3, 3, 2,
3, 3, 3, 2, 2, 1, 0, -3 },
{ -10, -10, -10, -10, -10, -9, -9, -9,
-6, 1, -8, -9, -8, -8, -8, -8,

```



```

-8, -7, -5, 2, -7, -6, -6, -6,
-5, -6, -6, -6, -3, 3, -5, -4,
-4, -3, -3, -3, -4, -4, -1, 4,
-2, -2, -1, 0, -1, -1, -2, -3,
0, 5, 1, 1, 2, 2, 1, 0,
-2, -1, 1, 6, 3, 3, 3, 3,
2, 1, -1, 0, 2, 9, 5, 5,
5, 5, 3, 0, -2, 0, 3, 9,
7, 7, 7, 5, 3, -1, -2, 0,
3, 10, 10, 9, 8, 5, 1, -2,
-3, 1, 4, 10, -11, -10, -9, -9,
-8, -8, -8, -7, -5, 1, -9, -8,
-8, -7, -7, -7, -7, -6, -4, 2,
-8, -7, -7, -6, -5, -5, -5, -4,
-3, 3, -6, -6, -5, -4, -3, -3,
-3, -3, 0, 4, -3, -4, -3, -1,
0, -1, -1, -1, 1, 5, -2, -1,
0, 2, 2, 1, 0, 0, 2, 6,
0, 0, 2, 3, 3, 2, -1, 1,
4, 8, 2, 2, 3, 4, 3, 1,
-1, 1, 4, 9, 4, 4, 5, 5,
3, 0, -1, 2, 5, 9, 7, 6,
6, 4, 1, -1, -1, 3, 6, 10,
-12, -10, -8, -8, -7, -6, -6, -5,
-3, 1, -11, -10, -8, -7, -6, -5,
-5, -4, -2, 2, -9, -9, -8, -7,
-6, -4, -4, -3, -1, 3, -8, -7,
-6, -5, -4, -3, -3, -2, 1, 4,
-4, -4, -4, -3, -1, -1, -1, 0,
2, 5, -3, -2, -2, 0, 1, 1,
0, 2, 4, 6, -2, -1, 0, 1,
2, 1, 0, 3, 5, 8, -1, 0,
1, 3, 2, 1, 0, 3, 5, 9,
1, 2, 3, 4, 2, 0, 0, 3,
6, 9, 4, 4, 4, 4, 1, 0,
1, 4, 7, 10, -11, -10, -8, -7,
-6, -5, -4, -2, -1, 1, -11, -9,
-8, -7, -6, -5, -4, -2, 0, 2,
-9, -9, -8, -7, -5, -4, -4, -1,
1, 3, -7, -8, -7, -6, -5, -3,
-2, 0, 2, 4, -5, -4, -4, -4,
-2, -1, 0, 2, 3, 5, -3, -3,
-3, -2, 0, 1, 1, 3, 5, 6,
-2, -2, -2, -1, 1, 1, 1, 4,
6, 7, -1, -1, 0, 1, 1, 0,
1, 4, 6, 8, 0, 1, 1, 2,
1, 0, 2, 5, 7, 8, 2, 2,
2, 2, 1, 1, 2, 6, 8, 9,
-11, -9, -7, -6, -5, -3, -3, -1,
0, 1, -10, -8, -7, -5, -4, -2,
-2, 0, 1, 2, -9, -8, -7, -5,
-4, -3, -1, 1, 2, 3, -7, -7,
-6, -5, -4, -3, 0, 2, 3, 4,
-4, -4, -3, -3, -2, -1, 2, 3,
4, 4, -3, -3, -3, -2, -1, 0,
3, 4, 5, 6, -2, -2, -2, -1,
0, 0, 2, 4, 6, 7, -1, -1,
-1, -1, 0, 0, 2, 4, 6, 7,
0, 0, 0, 0, 0, 0, 2, 4,
6, 8, 1, 1, 0, 1, 1, 1,
3, 5, 7, 8, -11, -9, -7, -6,
-5, -2, -1, 0, 0, 0, -10, -8,
-6, -5, -3, -1, 0, 1, 1, 2,
-8, -7, -6, -5, -3, -1, 0, 2,
2, 2, -6, -6, -6, -4, -3, -1,
1, 2, 3, 3, -4, -4, -3, -2,
-1, 0, 2, 3, 4, 4, -3, -3,
-3, -2, 1, 1, 3, 4, 5, 6,
-2, -2, -2, -1, 1, 1, 3, 4,
6, 7, -1, -2, -1, -1, 0, 0,
2, 4, 5, 7, 0, -1, -1, -1,

```

```

0, 0, 2, 4, 5, 7, 1, -1,
-1, 0, 0, 0, 3, 4, 6, 8,
-12, -10, -7, -5, -3, -1, -1, 0,
0, 0, -10, -8, -6, -4, -2, -1,
0, 0, 1, 1, -9, -7, -6, -4,
-2, -1, 1, 1, 2, 2, -7, -6,
-5, -3, -1, 0, 1, 2, 2, 3,
-5, -4, -3, -2, 1, 2, 3, 4,
4, 4, -3, -3, -2, -1, 2, 3,
5, 6, 6, 7, -2, -2, -1, 0,
3, 3, 4, 5, 7, 9, -2, -1,
-1, 0, 2, 2, 4, 5, 7, 9,
0, -1, -1, 0, 1, 1, 3, 4,
6, 9, 1, -1, -1, 0, 1, 1,
3, 4, 6, 9, -12, -9, -6, -3,
-1, -2, -2, -1, -1, 0, -9, -7,
-5, -3, 0, -1, -2, -1, 0, 1,
-9, -7, -4, -3, -1, -1, -1, 0,
1, 2, -7, -5, -4, -2, -1, -1,
0, 2, 2, 3, -4, -3, -2, -1,
1, 1, 3, 3, 4, 4, -3, -2,
-1, 1, 3, 3, 5, 6, 6, 7,
-2, -1, 0, 2, 4, 4, 5, 6,
7, 9, -1, 0, 1, 2, 4, 4,
5, 6, 7, 10, 0, 0, 1, 2,
3, 3, 4, 5, 7, 10, 1, 0,
1, 2, 2, 2, 4, 5, 7, 11,
-12, -9, -5, -2, -1, -2, -4, -3,
-3, -2, -10, -7, -4, -2, 0, -2,
-3, -3, -1, 0, -8, -6, -3, -2,
0, -2, -2, -1, 0, 2, -5, -4,
-3, -1, 0, -1, -1, 0, 1, 3,
-3, -2, -1, 1, 1, 1, 2, 2,
3, 4, -2, -1, 0, 2, 3, 3,
4, 5, 6, 6, -2, 0, 2, 3,
4, 4, 5, 6, 7, 8, -1, 1,
2, 3, 4, 5, 5, 6, 8, 10,
0, 1, 2, 3, 4, 4, 5, 6,
8, 11, 0, 1, 2, 3, 3, 3,
4, 5, 7, 11, -13, -9, -5, -2,
-1, -3, -4, -5, -6, -6, -10, -7,
-4, -1, -1, -3, -4, -4, -4, -3,
-8, -6, -3, 0, -1, -2, -3, -3,
-2, -1, -5, -3, -1, 0, 0, -1,
-1, -1, -1, 0, -3, -1, 0, 2,
1, 1, 1, 1, 1, 2, -2, 1,
2, 3, 3, 2, 2, 3, 3, 4,
-1, 1, 3, 4, 4, 4, 4, 4,
5, 6, 0, 1, 3, 5, 5, 5,
5, 5, 6, 7, 0, 1, 3, 5,
5, 5, 5, 5, 7, 7, -1, 0,
1, 3, 3, 3, 3, 4, 5, 8 },
{ 13, 14, 12, 9, 11, 13, 6, 4,
5, 6, 12, 14, 11, 9, 10, 13,
7, 3, 2, 7, 11, 13, 11, 9,
10, 13, 7, 1, 0, 5, 10, 12,
11, 9, 9, 11, 5, 0, -3, 4,
10, 10, 10, 6, 4, 6, 3, -2,
-6, 1, 10, 9, 8, 4, 2, 2,
1, -3, -7, -1, 11, 9, 7, 4,
2, 4, 1, -3, -6, -2, 11, 9,
6, 5, 4, 5, 1, -3, -4, -1,
13, 9, 7, 8, 8, 7, 2, -3,
-2, 0, 15, 10, 9, 10, 12, 10,
3, -2, -1, 1, 13, 14, 12, 8,
9, 7, 4, 2, 2, 1, 13, 14,
12, 9, 9, 10, 7, 5, 5, 6,
11, 13, 12, 10, 9, 10, 8, 5,
5, 8, 11, 11, 11, 9, 8, 10,
7, 5, 4, 7, 10, 9, 10, 7,
5, 7, 5, 2, 1, 4, 10, 8,

```

```

8, 5, 3, 3, 2, -1, -2, 2,
9, 8, 7, 5, 3, 5, 2, -1,
-2, 1, 10, 8, 6, 5, 4, 5,
3, -1, -2, 0, 12, 8, 6, 7,
6, 6, 3, -1, -2, 0, 13, 7,
7, 7, 8, 7, 2, -1, -2, 0,
13, 13, 10, 9, 7, 5, 2, 2,
0, -2, 13, 12, 11, 10, 8, 8,
6, 4, 4, 3, 12, 12, 11, 10,
9, 9, 7, 5, 5, 6, 11, 10,
10, 10, 8, 9, 7, 5, 5, 7,
9, 8, 9, 8, 5, 6, 5, 3,
2, 5, 8, 5, 6, 6, 2, 3,
2, 1, 1, 3, 7, 5, 5, 5,
2, 3, 2, 0, 0, 2, 7, 5,
5, 6, 3, 4, 2, 0, -1, 0,
9, 6, 5, 6, 5, 5, 2, -1,
-1, -1, 10, 7, 5, 6, 6, 5,
2, -1, -1, -2, 12, 10, 7, 6,
5, 3, 1, 0, -2, -5, 12, 10,
8, 8, 7, 6, 4, 3, 2, 1,
11, 10, 8, 8, 8, 7, 5, 4,
4, 4, 10, 8, 8, 9, 7, 7,
5, 4, 4, 6, 7, 6, 6, 7,
5, 5, 5, 3, 3, 5, 5, 3,
4, 5, 1, 2, 2, 1, 1, 3,
4, 3, 3, 4, 2, 2, 1, 0,
-1, 1, 5, 3, 3, 4, 3, 2,
0, -1, -1, 0, 6, 4, 4, 4,
3, 3, 0, -1, -2, -2, 7, 5,
5, 4, 4, 3, 0, -2, -3, -4,
10, 8, 6, 5, 2, -1, -4, -5,
-7, -8, 11, 8, 7, 6, 4, 0,
-1, -2, -3, -4, 10, 8, 7, 6,
4, 1, 0, 0, -1, 0, 8, 7,
6, 5, 4, 1, 0, 1, 0, 3,
5, 5, 4, 3, 3, 0, 0, 1,
1, 3, 3, 2, 1, 2, 1, -1,
-2, -1, -1, 1, 3, 1, 1, 1,
0, -3, -3, -3, -3, -2, 4, 1,
1, 1, 0, -2, -3, -4, -5, -4,
4, 2, 2, 2, 0, -2, -3, -5,
-6, -6, 5, 4, 3, 3, 1, -1,
-3, -5, -7, -8, 9, 6, 5, 3,
1, -1, -6, -8, -9, -9, 9, 6,
5, 3, 2, 0, -4, -5, -6, -4,
8, 6, 4, 3, 2, 0, -3, -4,
-3, 1, 6, 5, 3, 2, 1, 1,
-2, -3, -1, 3, 4, 3, 1, 0,
0, 1, -2, -3, -1, 2, 1, -1,
-2, -1, -2, 0, -3, -5, -3, 0,
1, -2, -3, -3, -4, -3, -5, -7,
-5, -2, 0, -2, -3, -3, -4, -3,
-5, -8, -7, -5, 1, -1, -1, -2,
-3, -2, -6, -8, -9, -7, 3, 1,
0, 0, -1, -1, -5, -8, -10, -9,
8, 3, 2, 2, -1, -4, -7, -9,
-10, -12, 7, 3, 1, 1, -1, -4,
-5, -6, -7, -6, 6, 2, 0, 1,
-1, -3, -4, -5, -3, 0, 5, 1,
-1, 0, -1, -3, -4, -4, -3, 1,
2, -2, -1, 0, -2, -2, -3, -5,
-3, 1, -1, -4, -4, -2, -4, -4,
-4, -6, -4, 0, -3, -6, -6, -5,
-7, -8, -6, -8, -7, -2, -4, -6,
-7, -6, -8, -9, -8, -9, -8, -4,
-2, -4, -6, -6, -7, -8, -8, -9,
-10, -7, -1, -3, -4, -5, -6, -7,
-8, -9, -11, -9, 6, 1, 1, 1,
-1, -6, -9, -10, -12, -15, 5, 1,
0, 1, -2, -5, -7, -8, -9, -10,

```

```

4, -1, -1, 1, -2, -5, -5, -6,
-6, -4, 2, -2, -1, 1, -1, -5,
-4, -6, -6, -3, 0, -3, -1, 1,
-2, -3, -3, -7, -6, -3, -3, -5,
-4, -1, -3, -5, -5, -8, -7, -4,
-5, -7, -6, -4, -7, -10, -9, -9,
-9, -7, -6, -8, -7, -6, -8, -11,
-10, -10, -10, -9, -5, -7, -8, -7,
-9, -11, -11, -10, -11, -11, -5, -6,
-7, -7, -8, -10, -10, -11, -12, -12,
7, 1, 1, 2, -1, -8, -10, -11,
-13, -17, 5, -1, 1, 2, -1, -7,
-8, -9, -10, -13, 4, -2, 1, 2,
-1, -6, -6, -7, -7, -8, 1, -3,
1, 2, -1, -5, -5, -6, -7, -8,
0, -3, 1, 2, -1, -4, -4, -7,
-8, -8, -3, -5, -1, 0, -3, -4,
-5, -8, -10, -10, -5, -7, -4, -3,
-6, -11, -8, -9, -11, -12, -6, -8,
-6, -4, -8, -13, -10, -10, -12, -12,
-7, -9, -8, -6, -9, -13, -12, -11,
-12, -13, -7, -8, -8, -7, -9, -12,
-13, -12, -12, -13, 7, 1, 3, 4,
1, -9, -11, -11, -15, -18, 5, 0,
3, 5, 2, -7, -9, -8, -11, -14,
3, 0, 4, 5, 2, -7, -6, -5,
-8, -10, 2, 0, 4, 5, 1, -5,
-4, -5, -8, -10, 1, 0, 4, 4,
1, -4, -3, -5, -8, -10, -1, -2,
3, 2, -1, -4, -4, -7, -10, -11,
-3, -3, 1, 1, -3, -12, -9, -8,
-11, -12, -5, -5, 0, 0, -5, -15,
-14, -9, -12, -13, -8, -8, -4, -1,
-5, -16, -16, -11, -12, -13, -12, -11,
-9, -3, -7, -17, -19, -12, -13, -13 },

```

```

},
{

```

```

{ -2, -2, -1, 0, 1, 2, 4, 7,
11, 15, -2, -1, 0, 0, 1, 2,
4, 7, 10, 14, -2, -1, 0, 1,
1, 2, 4, 7, 10, 13, -2, -1,
0, 1, 2, 2, 4, 6, 9, 12,
-2, -1, 0, 1, 2, 3, 4, 6,
9, 12, -2, -1, 0, 1, 2, 3,
4, 6, 8, 11, -2, -1, 0, 1,
2, 3, 4, 5, 8, 11, -2, -1,
0, 1, 2, 4, 4, 5, 8, 10,
-2, -1, 0, 1, 2, 4, 5, 5,
7, 10, -1, -2, -1, 1, 2, 3,
5, 5, 6, 9, -2, -3, -2, -1,
0, 0, 2, 4, 7, 11, -2, -2,
-2, -1, 0, 1, 2, 4, 7, 10,
-1, -2, -2, -1, 0, 1, 2, 4,
6, 9, -1, -2, -2, -1, 0, 1,
2, 4, 6, 9, -1, -1, -2, -1,
0, 1, 2, 4, 5, 8, -1, -1,
-2, -1, 0, 1, 2, 3, 5, 8,
-1, -1, -2, -1, 0, 2, 2, 3,
5, 7, -1, -1, -1, -1, 0, 2,
3, 3, 5, 7, 0, -1, -1, -1,
0, 1, 3, 3, 4, 6, 0, 0,
-1, -2, 0, 1, 2, 3, 3, 5,
-2, -2, -2, -2, -2, -1, 0, 2,
4, 7, -1, -2, -2, -1, -1, -1,
0, 2, 3, 6, -1, -1, -2, -1,
-1, -1, 0, 2, 3, 6, -1, -1,
-1, -1, -1, -1, 0, 1, 3, 5,
-1, -1, -1, -1, -1, -1, 0, 1,
3, 5, 0, -1, -1, -1, -1, -1,
0, 1, 3, 4, 0, -1, -1, -1,
-1, -1, 0, 1, 2, 4, 0, -1,

```

```

-1, -1, -1, -1, 0, 1, 2, 3,
0, 0, -1, -1, 0, -1, 0, 0,
1, 3, 0, 0, 0, -1, 0, -1,
-1, 0, 0, 2, -1, -1, -2, -1,
-1, -1, -1, -1, 1, 3, -1, -1,
-2, -1, -1, -1, -1, -1, 1, 3,
-1, -1, -1, -1, -1, -1, -1, -1,
1, 2, 0, -1, -1, -1, -1, -1,
0, -1, 0, 2, 0, -1, -1, -1,
-1, 0, 0, -1, 0, 2, 0, -1,
-1, -1, -1, 0, 0, -1, 0, 1,
0, 0, -1, -1, -1, 0, 0, 0,
-1, 1, 0, 0, -1, -1, -1, 0,
0, 0, -1, 1, 0, 0, 0, -1,
0, 0, 0, -1, -1, 0, 0, 0,
0, 0, 0, 0, 0, -1, -3, -2,
-1, -1, -2, -1, -1, -1, -1, -1,
0, 0, -1, -1, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -1, -1,
-1, -1, 0, 0, 0, 0, 0, -1,
-1, -1, -1, -1, 0, 0, 0, 0,
0, 0, -1, -1, -1, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
0, -1, -1, 0, 0, 0, 0, -1,
0, 0, 0, -1, 0, 0, -1, -1,
-1, -1, 0, 0, 0, 0, 0, -1,
-1, -1, -2, -2, -1, -1, -1, -1,
-1, -1, 0, -1, 0, 0, 0, -1,
-1, -1, -1, -1, 0, 0, 0, 1,
0, -1, -1, -1, -1, -1, 0, 0,
0, 1, 0, -1, -1, -1, -1, 0,
0, 0, 0, 1, 0, 0, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, 0, 0, 0, 1, 0,
0, 0, -1, -1, -1, 0, 0, 1,
0, 0, -1, 0, 0, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, -1, -1, -2, -1,
0, 0, 0, -1, -2, -2, -3, -2,
0, -1, -1, -1, -1, -1, -1, -1,
-1, 0, 0, -1, -1, -1, -1, 0,
0, 0, 0, 0, 0, 0, -1, -1,
-1, -1, 0, 0, 0, 0, 0, 0,
-1, -1, -1, 0, 0, 1, 0, 0,
0, 0, -1, -1, -1, 0, 0, 1,
0, 0, -1, 0, 0, -1, -1, 0,
0, 1, 0, 0, 0, 0, 0, -1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, -1, -1, -1, -1, -1, 0,
-2, -1, 0, 0, 0, -1, -2, -2,
-2, -1, -2, -2, -1, 0, 0, -1,
-2, -2, -2, -2, 1, 0, 0, -1,
-1, -1, -1, -1, -1, 0, 0, 0,
0, -1, -1, -1, 0, 0, -1, 0,
0, 0, 0, -1, -1, -1, 0, 0,
0, 0, 0, 0, 0, -1, -1, 0,
1, 0, 0, 0, 0, 0, 0, 0,
-1, 0, 0, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, -1, -1, 0, 0, 0, 0,
0, 0, 0, 0, -2, -2, -1, 0,
0, 0, -1, -1, -1, -1, -2, -2,
-1, -1, -1, 0, -1, -2, -2, -2,
2, 2, 2, 1, 0, 0, 0, 0,
0, 0, 2, 2, 2, 1, 0, 0,
0, 0, -1, 0, 1, 1, 1, 1,
1, 0, 0, 0, -1, -1, 1, 1,
1, 1, 0, 0, 0, 0, 0, -1,

```

```

0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, -1, -1, 0, 0,
0, 0, 0, 0, 0, 0, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, 0, 1, 0, 0,
0, 0, -1, -1, -1, -1, 0, 0,
1, 0, -1, -1, 4, 3, 3, 2,
1, 0, 0, 0, 0, 0, 3, 3,
3, 2, 1, 0, 0, 0, 0, 0,
2, 3, 3, 2, 1, 1, 0, 0,
-1, -1, 2, 2, 2, 2, 1, 1,
0, 0, -1, -1, 1, 1, 1, 1,
1, 0, 0, 0, 0, -1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, -1, -1, 0, 0, 1,
1, 1, 1, 0, -1, -1, -1, -1,
0, 1, 1, 1, 0, 0, -1, -1,
-1, -1, 0, 1, 1, 1, 0, 0 },
{
3, 2, 1, 0, -1, 0, 0, 1,
2, 2, 3, 2, 1, 0, -1, 0,
0, 1, 1, 2, 2, 2, 1, 0,
-1, -1, 0, 0, 0, 1, 2, 1,
1, 0, -1, -1, -1, -1, 0, 0,
1, 0, 0, 0, -1, -1, -1, -1,
0, 0, 0, 0, -1, -1, -2, -1,
-1, -1, -1, -1, 0, -1, -1, -2,
-2, -2, -1, -1, 0, -2, 0, -1,
-2, -2, -2, -2, -1, 0, -1, -2,
-1, -2, -2, -3, -3, -2, -1, -1,
-1, -2, -3, -3, -3, -3, -3, -2,
-2, -1, -2, -3, 2, 2, 1, 0,
0, 0, 1, 1, 2, 3, 3, 2,
1, 0, -1, 0, 0, 1, 1, 2,
2, 2, 1, 0, -1, -1, 0, 0,
0, 1, 1, 2, 1, 0, -1, -1,
-1, -1, -1, 0, 1, 1, 0, 0,
-1, -1, -1, -1, -1, -1, 0, 0,
0, 0, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -2, -2, -2, -1, -1,
-1, -2, -1, -1, -2, -2, -2, -2,
-1, -1, -1, -2, -2, -2, -2, -2,
-2, -2, -2, -1, -2, -2, -4, -3,
-3, -3, -3, -2, -2, -2, -3, -3,
2, 1, 0, 0, 0, 0, 1, 1,
2, 4, 2, 1, 1, 0, 0, 0,
0, 1, 1, 2, 2, 2, 1, 1,
-1, -1, -1, 0, 0, 1, 1, 2,
1, 1, -1, -1, -1, -1, -1, 0,
1, 1, 1, 0, -1, -1, -1, -1,
-1, -1, 0, 0, 0, 0, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1,
-2, -2, -1, -1, -2, -2, -2, -2,
-1, -2, -2, -2, -2, -2, -2, -2,
-3, -2, -2, -2, -2, -3, -2, -2,
-2, -2, -4, -3, -2, -2, -2, -2,
-2, -2, -3, -3, 1, 0, 0, 0,
0, 1, 1, 2, 3, 4, 1, 1,
0, 0, 0, 0, 1, 1, 2, 3,
2, 1, 1, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 0, 0, -1,
-1, -1, 0, 0, 1, 1, 1, 0,
0, -1, -1, -1, -1, -1, 0, 0,
0, 0, -1, -2, -1, -1, -1, -1,
-1, -1, -1, -1, -2, -2, -2, -2,
-2, -1, -2, -2, -1, -2, -2, -2,
-2, -2, -2, -2, -3, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -4, -3,
-2, -2, -2, -2, -2, -2, -3, -3,
1, 0, 0, 0, 0, 1, 2, 3,

```

```

4, 4, 1, 1, 0, 0, 0, 1,
2, 3, 3, 3, 2, 1, 0, 0,
0, 1, 1, 1, 2, 2, 2, 1,
1, 0, 0, 0, 0, 0, 0, 1,
1, 1, 1, 0, 0, 0, -1, -1,
-1, -1, 1, 1, 0, 0, 0, -1,
-1, -1, -1, -1, 0, -1, 0, 0,
-1, -2, -2, -2, -2, -1, -1, -1,
-1, -1, -1, -2, -2, -2, -2, -2,
-2, -2, -1, -1, -1, -2, -3, -2,
-2, -2, -3, -2, -1, -1, -1, -2,
-2, -2, -3, -3, 1, 0, 0, 0,
0, 2, 3, 4, 4, 5, 1, 1,
0, 0, 1, 1, 2, 3, 4, 4,
1, 1, 1, 0, 1, 1, 2, 2,
3, 3, 1, 1, 1, 0, 0, 1,
1, 1, 1, 1, 1, 1, 1, 0,
0, 0, 0, 0, 0, 0, 1, 1,
1, 0, 0, -1, -1, -1, -1, -1,
1, 0, 0, 0, 0, -1, -2, -2,
-2, -1, 0, 0, 0, 0, -1, -2,
-2, -2, -2, -2, -1, -1, 0, 0,
-1, -2, -2, -2, -2, -2, -2, -1,
0, 0, -1, -2, -2, -2, -2, -3,
1, 0, 0, -1, 0, 1, 2, 4,
5, 5, 1, 1, 0, 0, 0, 1,
2, 3, 4, 4, 1, 1, 1, 0,
1, 1, 2, 3, 3, 3, 1, 1,
1, 1, 1, 1, 2, 2, 2, 2,
1, 1, 1, 1, 0, 1, 1, 1,
1, 1, 1, 1, 1, 0, 0, 0,
0, 0, -1, -1, 1, 1, 1, 0,
0, -1, -1, -1, -1, -1, 1, 1,
0, 0, -1, -2, -2, -2, -2, -2,
0, 0, 0, 0, -1, -2, -2, -2,
-2, -3, -1, 0, 0, 0, 0, -2,
-2, -2, -2, -3, 0, 0, -1, -1,
0, 1, 2, 3, 4, 5, 1, 1,
0, 0, 0, 0, 2, 3, 4, 4,
1, 1, 1, 1, 1, 1, 1, 2,
3, 3, 1, 1, 1, 1, 1, 1,
1, 2, 2, 2, 1, 1, 1, 1,
0, 0, 0, 1, 1, 1, 1, 1,
1, 0, 0, -1, 0, 0, 0, -1,
1, 1, 1, 0, 0, -1, -1, -1,
-1, -2, 1, 1, 1, 0, -1, -2,
-2, -2, -2, -2, 1, 1, 0, 0,
-1, -2, -2, -2, -2, -3, 0, 1,
0, 0, -1, -2, -2, -2, -3, -3,
0, 0, -1, -1, -1, 1, 2, 3,
4, 5, 1, 0, 0, 0, 0, 0,
1, 2, 3, 4, 1, 1, 1, 1,
0, 0, 0, 1, 2, 3, 1, 1,
1, 1, 1, 0, 0, 0, 1, 2,
1, 1, 1, 1, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 0, -1,
-1, 0, 0, 0, 1, 1, 1, 0,
-1, -1, -1, -1, -1, -1, 1, 1,
1, 0, -1, -2, -2, -2, -2, -2,
1, 1, 1, 0, -1, -1, -2, -2,
-3, -3, 1, 0, 0, 0, -1, -1,
-2, -3, -3, -3, 0, -1, -1, -1,
-1, 0, 1, 2, 3, 4, 1, 0,
-1, -1, -1, -1, 0, 1, 2, 3,
1, 0, 0, 0, 0, -1, 0, 0,
0, 2, 1, 1, 1, 1, 0, 0,
-1, -1, -1, 1, 1, 1, 1, 1,
1, 0, -1, -1, 0, 1, 1, 1,
1, 1, 0, -1, -1, 0, 0, 0,
1, 1, 1, 0, -1, -1, -1, 0,
0, -1, 1, 1, 1, 0, -1, -1,

```

```

-1, -1, -2, -2, 1, 1, 0, 0,
-1, -2, -2, -2, -3, -3, 1, 0,
0, 0, -1, -1, -2, -3, -3, -4 },
{ -5, -4, -3, -2, -2, -2, -3, -4,
-5, -6, -5, -4, -3, -2, -1, -2,
-3, -4, -5, -5, -4, -3, -3, -3,
-2, -2, -2, -4, -4, -3, -2, -3,
-3, -3, -2, -1, -2, -2, -2, -1,
-2, -2, -3, -2, -1, 0, -1, -1,
0, 0, -2, -2, -2, -1, 0, 1,
0, 0, 1, 2, -1, -2, -1, -1,
1, 1, 1, 2, 2, 2, -1, -1,
-1, -1, 0, 1, 1, 1, 1, 2,
0, -1, -2, -1, 0, 0, 0, 0,
0, 1, 0, -1, -2, -2, -1, -1,
0, -1, 0, 0, -6, -4, -3, -2,
-1, -1, -1, -2, -3, -3, -6, -4,
-3, -2, -2, -1, -1, -2, -3, -3,
-4, -4, -4, -3, -3, -1, -2, -3,
-2, -1, -2, -3, -3, -3, -2, -1,
-2, -2, -1, 0, -1, -2, -2, -2,
-1, 0, 0, 0, 0, 1, -1, -1,
-2, -1, 0, 1, 1, 1, 2, 2,
-1, -1, -1, -1, 0, 1, 1, 1,
2, 2, -1, -1, -1, -1, 0, 0,
0, 0, 1, 2, 0, -1, -1, -1,
-1, -1, -1, 0, 0, 1, 0, -1,
-2, -2, -2, -2, -2, -1, 0, 0,
-6, -5, -3, -2, -1, 0, 0, 0,
-1, -2, -6, -5, -3, -2, -1, -1,
0, -1, -1, -1, -5, -4, -4, -3,
-2, -1, -1, -1, -1, 0, -3, -3,
-3, -3, -2, 0, -1, -1, 0, 1,
-2, -2, -2, -2, -1, 0, 0, 0,
1, 2, -1, -1, -2, -2, -1, 0,
1, 1, 2, 3, -1, -1, -1, -1,
-1, 0, 0, 1, 2, 3, 0, -1,
-1, -1, -1, -1, -1, 0, 1, 2,
0, 0, -1, -2, -1, -1, -1, 0,
0, 1, 1, 0, -1, -2, -2, -2,
-2, -1, 0, 1, -6, -5, -4, -2,
-1, 0, 1, 0, 0, 0, -6, -5,
-4, -2, -1, 0, 0, 0, 0, 0,
-5, -5, -3, -3, -2, 0, 0, 0,
0, 1, -3, -3, -3, -2, -1, 0,
0, 0, 1, 2, -2, -2, -2, -1,
0, 0, 0, 1, 2, 2, -1, -1,
-1, -1, 0, 0, 1, 1, 2, 3,
-1, -1, -1, -1, 0, 0, 0, 1,
2, 3, 0, 0, 0, -1, -1, -1,
-1, 0, 1, 2, 1, 0, 0, -1,
-1, -1, -1, 0, 1, 1, 2, 0,
0, -1, -1, -2, -2, -1, 0, 1,
-6, -5, -4, -2, -1, 0, 0, 1,
1, 0, -6, -5, -4, -3, -2, 0,
0, 1, 1, 1, -6, -5, -3, -3,
-2, 0, 0, 1, 1, 1, -4, -3,
-3, -2, -1, 0, 0, 1, 1, 2,
-2, -2, -2, -1, 0, 0, 1, 2,
2, 3, -1, -1, -1, 0, 0, 0,
1, 2, 3, 3, 0, 0, 0, 0,
0, 0, 1, 1, 2, 3, 1, 1,
0, 0, 0, 0, 0, 1, 2, 3,
2, 1, 1, 0, 0, -1, -1, 0,
1, 2, 2, 1, 1, 0, -1, -1,
-1, 0, 0, 0, -7, -5, -4, -3,
-2, -1, 0, 1, 1, 1, -6, -5,
-4, -3, -2, -1, 0, 1, 1, 1,
-5, -4, -4, -3, -2, -1, 0, 1,
1, 1, -3, -3, -3, -2, -1, -1,
0, 1, 1, 1, -2, -2, -1, 0,

```



```

0, 0, 1, 2, 2, 2, -1, 0,
1, 1, 1, 0, 1, 2, 2, 3,
1, 1, 1, 2, 1, 0, 1, 1,
2, 3, 2, 2, 2, 2, 1, 0,
0, 1, 2, 3, 3, 3, 2, 1,
0, 0, 0, 0, 1, 1, 3, 3,
2, 1, 0, -1, -1, -1, -1, 0,
-6, -5, -4, -3, -3, -2, -1, 0,
1, 1, -6, -5, -4, -3, -3, -2,
-1, 0, 1, 1, -5, -4, -3, -3,
-2, -2, -1, 0, 1, 1, -3, -3,
-2, -2, -1, -1, 0, 1, 1, 1,
-1, -1, 0, 1, 0, 0, 1, 1,
2, 2, 0, 1, 2, 2, 1, 1,
1, 2, 2, 2, 2, 3, 3, 3,
1, 0, 1, 1, 1, 3, 4, 4,
4, 3, 1, 0, 0, 1, 1, 2,
4, 4, 4, 2, 1, 0, 0, 0,
0, 1, 5, 4, 3, 2, 1, -1,
-2, -1, -1, 0, -6, -5, -4, -3,
-3, -2, -2, -1, 1, 1, -6, -5,
-4, -3, -3, -3, -2, -1, 1, 1,
-4, -3, -3, -3, -2, -3, -2, -1,
1, 1, -2, -2, -2, -1, -1, -1,
-1, 0, 1, 1, -1, 0, 0, 1,
1, 0, 1, 1, 2, 2, 1, 2,
3, 3, 2, 1, 1, 1, 2, 3,
3, 4, 5, 4, 2, 1, 1, 1,
1, 2, 5, 5, 5, 4, 2, 0,
0, 0, 1, 2, 6, 6, 5, 3,
2, 0, -1, -1, 0, 1, 6, 6,
5, 3, 1, -1, -2, -1, -1, -1,
-6, -5, -4, -4, -4, -3, -2, -1,
0, 1, -6, -5, -4, -3, -3, -3,
-2, -1, 0, 1, -4, -3, -2, -2,
-2, -2, -2, -1, 0, 1, -2, -1,
-1, 0, -1, -1, -1, 0, 1, 1,
0, 0, 1, 1, 1, 1, 1, 1,
1, 2, 2, 3, 4, 2, 2, 2,
1, 1, 2, 2, 4, 5, 5, 4,
3, 2, 1, 0, 1, 2, 6, 6,
6, 4, 2, 1, 0, 0, 0, 1,
7, 7, 6, 4, 2, 0, -1, -1,
0, 0, 7, 7, 6, 4, 1, -1,
-2, -1, -1, -1, -6, -5, -4, -4,
-4, -4, -3, -1, 0, 1, -5, -4,
-4, -3, -3, -3, -3, -1, 0, 1,
-4, -3, -2, -2, -1, -2, -2, -1,
0, 1, -2, -1, 0, -1, 0, 0,
0, 0, 1, 1, 0, 1, 1, -1,
1, 1, 1, 1, 1, 2, 3, 4,
3, 1, 2, 2, 2, 1, 1, 2,
5, 6, 6, 4, 3, 3, 1, 0,
1, 2, 7, 8, 7, 5, 3, 1,
0, 0, 0, 1, 8, 8, 7, 4,
2, 1, 0, -1, 0, 0, 9, 8,
6, 4, 1, 0, -1, -1, -1, -1 },
{
4, 4, 5, 4, 4, 3, 3, 5,
9, 10, 4, 4, 4, 4, 3, 2,
3, 4, 7, 8, 4, 4, 4, 3,
2, 1, 1, 3, 5, 6, 4, 4,
4, 3, 2, 1, 1, 2, 3, 4,
3, 3, 3, 2, 1, 1, 1, 1,
2, 2, 3, 2, 2, 1, -1, -1,
0, 0, 1, 1, 2, 1, 1, -1,
-2, -2, -3, -2, -1, -1, 1, 1,
0, -2, -3, -3, -4, -3, -3, -3,
1, 0, -1, -3, -4, -4, -4, -4,
-5, -5, 0, -1, -2, -4, -4, -5,
-6, -6, -7, -7, 4, 4, 4, 4,
3, 2, 2, 4, 7, 8, 4, 4,

```

4, 4, 2, 1, 1, 3, 5, 6,
 4, 4, 4, 3, 2, 1, 1, 2,
 4, 4, 4, 4, 3, 3, 2, 1,
 1, 2, 3, 3, 3, 3, 3, 2,
 1, 1, 1, 1, 2, 1, 3, 2,
 2, 1, 0, 0, 0, 0, 0, 0,
 2, 1, 1, -1, -2, -2, -2, -2,
 -2, -2, 2, 1, 0, -2, -3, -3,
 -3, -3, -3, -4, 1, 0, -1, -3,
 -3, -3, -4, -4, -4, -5, 0, -1,
 -2, -3, -4, -5, -5, -5, -6, -6,
 3, 3, 3, 3, 2, 1, 1, 2,
 5, 6, 4, 4, 3, 3, 2, 1,
 1, 2, 4, 4, 4, 4, 4, 3,
 2, 2, 2, 2, 3, 3, 4, 4,
 4, 3, 2, 1, 1, 2, 2, 1,
 3, 3, 3, 2, 1, 1, 1, 1,
 1, 0, 2, 2, 1, 1, 0, 0,
 0, 0, -1, -1, 2, 1, 1, 0,
 -2, -2, -2, -2, -3, -3, 2, 1,
 0, -1, -2, -3, -3, -3, -4, -4,
 1, 0, -1, -2, -3, -3, -3, -4,
 -5, -5, 1, 0, -1, -2, -3, -4,
 -4, -5, -6, -7, 4, 4, 3, 2,
 1, 0, 0, 1, 3, 4, 5, 4,
 4, 3, 2, 1, 1, 2, 2, 2,
 4, 4, 4, 4, 3, 1, 2, 2,
 1, 1, 4, 4, 3, 3, 2, 1,
 1, 1, 1, 0, 4, 3, 3, 3,
 1, 1, 1, 0, 0, -1, 2, 2,
 2, 2, 0, 0, 0, -1, -1, -2,
 2, 1, 1, 0, -1, -2, -2, -3,
 -3, -3, 2, 1, 0, 0, -1, -2,
 -3, -3, -4, -5, 1, 0, 0, -1,
 -2, -3, -3, -4, -5, -6, 0, 0,
 -1, -1, -2, -3, -4, -5, -7, -8,
 5, 4, 3, 2, 1, 0, 0, 0,
 2, 2, 5, 4, 4, 3, 3, 1,
 1, 1, 1, 1, 5, 4, 4, 3,
 3, 1, 1, 1, 0, 0, 4, 4,
 3, 3, 2, 0, 0, 0, 0, -1,
 3, 3, 3, 2, 2, 0, 0, -1,
 -1, -1, 2, 2, 2, 2, 1, 0,
 -1, -2, -2, -2, 2, 1, 1, 0,
 -1, -1, -2, -3, -3, -4, 1, 1,
 0, 0, -1, -1, -3, -3, -4, -5,
 1, 0, 0, -1, -1, -2, -3, -4,
 -5, -6, 0, 0, -1, -1, -2, -2,
 -4, -5, -6, -8, 5, 4, 3, 2,
 1, 0, 0, 0, 1, 1, 5, 4,
 4, 3, 2, 1, 0, 0, 0, 0,
 5, 4, 4, 3, 2, 0, 0, 0,
 0, -1, 4, 4, 3, 3, 2, 1,
 0, -1, -1, -1, 3, 3, 3, 2,
 2, 0, 0, -1, -1, -2, 2, 2,
 2, 1, 1, 0, 0, -1, -2, -2,
 2, 1, 1, 0, 0, -1, -2, -3,
 -3, -4, 1, 1, 0, 0, 0, -1,
 -2, -3, -4, -5, 0, 0, 0, 0,
 -1, -1, -3, -4, -5, -6, 0, -1,
 -1, -1, -1, -2, -3, -5, -6, -7,
 5, 4, 3, 3, 1, 1, 0, 0,
 0, 0, 5, 4, 4, 3, 2, 1,
 0, 0, 0, -1, 5, 4, 4, 3,
 2, 1, 1, 0, -1, -1, 4, 4,
 4, 3, 2, 1, 1, 0, -1, -2,
 3, 3, 3, 2, 2, 1, 1, 0,
 -1, -2, 2, 1, 1, 1, 0, 0,
 0, -1, -2, -2, 1, 0, 0, 0,
 -1, -1, -2, -3, -4, -4, 0, 0,
 0, -1, -1, -1, -2, -4, -4, -5,

```

-1, -1, -1, -1, -1, -1, -2, -4,
-5, -6, -2, -2, -2, -2, -2, -2,
-3, -5, -6, -8, 5, 4, 3, 2,
1, 1, 0, 0, 0, -1, 5, 4,
3, 3, 2, 1, 1, 0, 0, -1,
5, 4, 3, 3, 2, 2, 2, 0,
0, -1, 4, 3, 3, 2, 2, 2,
2, 0, -1, -1, 2, 2, 2, 2,
1, 1, 1, 0, -1, -2, 1, 0,
0, 0, 0, 0, 0, -1, -2, -2,
0, -1, -1, -2, -2, -1, -2, -3,
-3, -4, -1, -1, -2, -2, -2, -2,
-2, -3, -4, -5, -2, -2, -3, -3,
-3, -3, -3, -4, -5, -6, -3, -4,
-4, -4, -3, -3, -4, -5, -6, -8,
4, 4, 3, 1, 1, 1, 1, 0,
-1, -1, 5, 4, 3, 2, 2, 2,
2, 0, 0, -1, 4, 4, 3, 3,
2, 2, 2, 1, 0, -1, 3, 3,
2, 2, 2, 2, 2, 1, 0, -2,
1, 1, 1, 1, 1, 1, 1, 0,
-1, -2, 0, -1, -1, -1, -1, 0,
0, -1, -2, -3, -1, -2, -2, -2,
-2, -2, -2, -3, -4, -4, -2, -3,
-3, -3, -3, -3, -3, -3, -4, -5,
-4, -4, -5, -4, -4, -3, -3, -4,
-5, -6, -6, -6, -5, -5, -5, -5,
-5, -6, -6, -8, 4, 3, 2, 1,
1, 1, 1, 0, -1, -2, 4, 4,
3, 2, 2, 2, 2, 1, -1, -2,
3, 3, 3, 3, 2, 2, 2, 1,
-1, -2, 2, 2, 2, 2, 2, 2,
2, 1, 0, -2, 0, 0, 0, 1,
1, 1, 1, 0, -1, -2, -1, -1,
-1, -1, -1, 0, 0, -1, -2, -3,
-2, -3, -2, -2, -2, -2, -2, -3,
-3, -4, -4, -4, -4, -4, -3, -3,
-3, -4, -4, -5, -6, -6, -6, -5,
-5, -4, -5, -5, -5, -6, -8, -8,
-7, -7, -6, -6, -6, -7, -7, -8 },
{
-5, -7, -13, -15, -13, -9, -7, -7,
-8, -11, -5, -6, -7, -8, -7, -6,
-4, -4, -5, -8, -6, -6, -7, -6,
-5, -3, -2, -2, -3, -7, -6, -6,
-7, -7, -5, -2, 0, 0, -1, -5,
-6, -6, -6, -6, -4, 0, 2, 2,
1, -2, -5, -5, -5, -4, -2, 1,
3, 4, 3, 0, -5, -4, -4, -2,
0, 3, 4, 5, 4, 2, -4, -4,
-2, 0, 2, 4, 5, 6, 6, 4,
-3, -3, -1, 1, 4, 5, 6, 7,
7, 5, -3, -2, 1, 4, 7, 7,
7, 9, 8, 7, -3, -4, -8, -12,
-13, -10, -7, -7, -7, -9, -2, -3,
-3, -5, -7, -6, -5, -4, -5, -8,
-2, -3, -4, -5, -4, -4, -3, -3,
-4, -6, -2, -2, -3, -4, -4, -1,
0, -1, -2, -5, -2, -2, -2, -3,
-3, 0, 2, 1, 0, -3, -1, -1,
-1, -1, -1, 1, 4, 3, 2, -1,
0, -1, 0, 1, 1, 3, 5, 5,
3, 2, 0, 0, 1, 2, 3, 4,
5, 6, 5, 3, 1, 0, 2, 3,
5, 5, 6, 7, 6, 5, 2, 0,
1, 4, 7, 8, 8, 8, 8, 6,
-4, -3, -5, -12, -13, -10, -7, -6,
-6, -8, -2, -1, -1, -3, -8, -7,
-5, -5, -5, -7, -1, -1, -2, -2,
-3, -5, -4, -3, -4, -6, -1, -1,
-2, -2, -3, -2, -2, -2, -3, -5,
0, -1, -1, -1, -2, 0, 1, 0,

```

```

-1, -3, 1, 0, 0, 0, 0, 2,
3, 2, 1, -1, 1, 0, 1, 2,
3, 4, 4, 4, 2, 0, 2, 1,
2, 4, 4, 5, 5, 5, 4, 2,
3, 2, 3, 4, 6, 7, 6, 6,
5, 4, 4, 2, 3, 5, 8, 9,
8, 7, 6, 5, -6, -5, -6, -10,
-12, -11, -7, -6, -6, -7, -3, -2,
-2, -3, -8, -8, -6, -5, -5, -7,
-1, 0, -1, -1, -3, -5, -5, -4,
-5, -6, 0, 0, 0, -1, -1, -3,
-3, -3, -4, -5, 1, 0, 0, 0,
-1, 0, -1, -1, -2, -3, 2, 1,
1, 2, 2, 2, 2, 1, -1, -2,
3, 2, 2, 4, 4, 4, 3, 2,
1, -1, 3, 2, 3, 5, 5, 5,
4, 4, 3, 1, 4, 3, 4, 5,
6, 7, 5, 5, 4, 3, 5, 4,
4, 6, 9, 10, 7, 6, 5, 5,
-8, -7, -8, -11, -12, -10, -7, -5,
-5, -6, -5, -4, -4, -5, -8, -8,
-6, -4, -5, -6, -2, -1, -2, -2,
-4, -6, -5, -4, -4, -5, 0, 0,
-1, -1, -2, -4, -4, -3, -4, -5,
1, 1, 0, 0, -1, -1, -3, -3,
-3, -4, 2, 2, 2, 2, 0, 0,
-1, -1, -2, -3, 3, 2, 2, 4,
3, 1, 1, 0, -1, -2, 3, 2,
4, 5, 5, 3, 2, 1, 1, 0,
4, 3, 4, 5, 6, 5, 3, 2,
2, 1, 5, 4, 4, 6, 8, 9,
6, 4, 3, 3, -9, -9, -9, -11,
-12, -11, -7, -4, -4, -5, -5, -6,
-5, -6, -9, -8, -6, -4, -3, -5,
-3, -3, -3, -3, -5, -6, -5, -4,
-4, -5, -1, -2, -2, -3, -3, -5,
-5, -4, -4, -5, 1, 0, -1, -1,
-1, -2, -3, -3, -3, -4, 2, 1,
1, 0, 0, 0, -1, -2, -2, -3,
2, 2, 2, 2, 2, 1, 0, -1,
-1, -2, 2, 2, 3, 4, 3, 1,
1, 1, 0, -1, 3, 3, 3, 5,
5, 4, 2, 1, 1, 1, 5, 4,
4, 5, 7, 8, 5, 3, 2, 2,
-11, -11, -10, -11, -13, -11, -8, -4,
-2, -3, -8, -8, -8, -8, -10, -9,
-6, -4, -2, -3, -6, -6, -6, -6,
-7, -7, -5, -3, -2, -3, -4, -5,
-5, -6, -6, -5, -4, -3, -2, -3,
-3, -3, -4, -5, -3, -3, -3, -2,
-2, -2, -1, -2, -2, -2, -1, 0,
-1, -1, 0, -1, 0, -1, 0, 0,
0, 1, 1, 0, 1, 0, 1, 1,
1, 2, 2, 2, 2, 2, 2, 1,
3, 3, 2, 3, 4, 3, 3, 2,
3, 3, 4, 4, 3, 5, 6, 7,
5, 4, 3, 3, -12, -12, -10, -11,
-12, -10, -5, -2, -1, -1, -9, -10,
-9, -8, -9, -8, -4, -2, -1, -1,
-7, -8, -8, -7, -7, -6, -3, -2,
0, -1, -6, -6, -7, -6, -5, -4,
-3, -1, 0, -1, -5, -5, -5, -5,
-2, -1, -1, 0, 0, 0, -3, -4,
-3, -2, -1, 1, 1, 1, 1, 1,
-2, -2, -1, 0, 0, 2, 3, 2,
2, 2, 1, 0, 0, 1, 1, 3,
4, 3, 3, 3, 4, 3, 2, 2,
3, 4, 4, 4, 4, 4, 6, 5,
4, 4, 6, 7, 6, 5, 4, 4,
-14, -12, -11, -10, -11, -9, -5, 0,
2, 2, -11, -11, -9, -8, -8, -6,

```

```

-3, 0, 1, 2, -9, -9, -8, -6,
-5, -5, -2, 0, 1, 2, -8, -7,
-7, -6, -4, -3, -1, 1, 2, 2,
-6, -6, -6, -5, -2, 0, 1, 2,
3, 3, -5, -5, -4, -4, -1, 3,
4, 3, 4, 4, -4, -4, -3, -2,
0, 4, 5, 5, 5, 5, -1, -2,
-1, 0, 0, 4, 6, 6, 6, 6,
3, 2, 1, 2, 3, 4, 6, 6,
6, 6, 6, 6, 4, 4, 6, 7,
8, 6, 5, 6, -15, -13, -11, -9,
-9, -8, -4, 0, 3, 6, -13, -11,
-9, -7, -6, -5, -2, 2, 4, 6,
-11, -10, -8, -6, -4, -3, 1, 3,
5, 7, -9, -8, -7, -6, -3, -1,
2, 4, 5, 6, -8, -7, -7, -5,
-3, 1, 4, 5, 6, 7, -7, -6,
-5, -5, -2, 3, 7, 7, 7, 8,
-5, -5, -4, -4, -1, 4, 8, 8,
8, 9, -3, -3, -3, -2, -1, 3,
8, 9, 10, 10, 1, 0, 0, 1,
1, 2, 6, 8, 8, 9, 5, 5,
3, 4, 4, 5, 6, 6, 6, 7 },
{ -11, -14, -9, -8, -12, -17, -14, -12,
-14, -18, -9, -10, -4, -4, -11, -17,
-12, -10, -13, -16, -5, -7, 0, -3,
-10, -16, -10, -9, -10, -12, -3, -4,
3, -1, -9, -14, -7, -7, -8, -9,
-1, 0, 5, 0, -3, -7, -3, -4,
-6, -6, 0, 2, 8, 2, -1, -6,
0, -2, -6, -8, 0, 2, 7, 4,
-2, -7, -2, -4, -7, -9, 0, 1,
6, 3, -2, -9, -4, -6, -10, -13,
0, 1, 5, 3, -2, -10, -8, -9,
-12, -17, 0, 1, 3, 2, -3, -12,
-12, -13, -16, -21, -7, -10, -7, -7,
-9, -13, -13, -10, -11, -13, -8, -9,
-6, -7, -10, -13, -11, -9, -10, -11,
-6, -8, -3, -5, -9, -13, -10, -8,
-9, -7, -5, -6, -2, -4, -9, -12,
-7, -7, -5, -4, -3, -2, 2, -2,
-4, -6, -4, -5, -3, -2, -1, 0,
4, 0, -3, -6, -2, -2, -4, -2,
0, 1, 5, 1, -3, -7, -3, -4,
-5, -4, 0, 1, 4, 1, -4, -7,
-4, -6, -8, -8, -1, 1, 4, 0,
-4, -9, -8, -9, -11, -14, -1, 0,
3, 0, -5, -11, -11, -12, -14, -18,
-5, -8, -6, -6, -8, -9, -9, -10,
-9, -8, -6, -9, -7, -8, -9, -10,
-9, -9, -9, -7, -8, -9, -6, -7,
-10, -10, -8, -8, -7, -4, -7, -7,
-4, -6, -9, -10, -6, -8, -4, -1,
-4, -4, 0, -2, -4, -5, -3, -6,
-1, 2, -2, -1, 2, 0, -3, -4,
-2, -5, -2, 3, -2, -1, 2, 0,
-3, -6, -3, -6, -4, 2, -2, 0,
2, 0, -4, -6, -4, -8, -6, -3,
-2, 0, 2, 0, -4, -8, -6, -10,
-8, -9, -1, 0, 2, -1, -5, -9,
-9, -12, -13, -15, -4, -6, -5, -5,
-6, -6, -7, -8, -9, -6, -6, -8,
-7, -7, -8, -8, -8, -8, -8, -5,
-7, -9, -7, -8, -9, -8, -7, -7,
-6, -2, -7, -8, -6, -6, -8, -8,
-6, -6, -3, 0, -5, -5, -2, -3,
-4, -5, -3, -3, -1, 3, -3, -2,
1, 0, -3, -4, -2, -2, 0, 4,
-2, -1, 1, 0, -4, -4, -3, -3,
-2, 3, -2, -1, 1, 0, -4, -5,
-3, -5, -3, -1, -2, 0, 1, 0,

```

```

-4, -6, -6, -8, -7, -7, -1, 0,
1, 0, -4, -7, -8, -10, -12, -13,
-3, -3, -3, -3, -2, -2, -5, -7,
-7, -5, -4, -6, -5, -5, -4, -3,
-6, -6, -6, -4, -6, -7, -6, -6,
-5, -3, -6, -5, -4, -2, -6, -7,
-6, -6, -4, -3, -5, -4, -2, 0,
-6, -5, -3, -3, -1, -1, -3, -2,
0, 2, -3, -2, 0, -1, 0, 0,
-1, 0, 2, 3, -1, -1, 0, 0,
0, -1, -2, -1, 1, 2, -1, 0,
1, 0, -1, -1, -3, -1, 0, 0,
0, 1, 1, 1, -1, -3, -5, -4,
-4, -6, 1, 1, 1, 0, -2, -4,
-7, -8, -9, -12, 0, 1, 1, 1,
1, -1, -2, -3, -3, -3, -1, -1,
-1, -1, 0, -1, -2, -3, -2, -2,
-3, -3, -2, -2, -1, -2, -2, -2,
-1, 0, -4, -4, -2, -3, -2, -3,
-2, -1, 0, 2, -4, -3, -2, -2,
0, 0, 0, 0, 2, 4, -2, 0,
2, 1, 1, 0, 2, 2, 4, 5,
1, 2, 3, 2, 1, 0, 2, 3,
4, 5, 2, 3, 4, 2, 1, 0,
1, 3, 3, 3, 3, 4, 4, 3,
1, -1, -1, 1, 0, -2, 4, 3,
3, 1, 0, -3, -4, -4, -5, -8,
4, 4, 5, 4, 5, 4, 0, -1,
-1, -1, 3, 3, 4, 3, 3, 3,
0, -1, 0, 0, 1, 2, 3, 2,
2, 2, 0, -1, 0, 1, 0, 1,
2, 1, 2, 3, 0, 0, 1, 3,
-1, 1, 2, 2, 1, 2, 1, 0,
2, 4, 2, 3, 4, 4, 4, 5,
4, 2, 4, 6, 4, 6, 6, 5,
6, 6, 6, 4, 6, 7, 5, 7,
6, 5, 6, 6, 6, 4, 6, 6,
6, 7, 6, 5, 5, 4, 4, 3,
3, 0, 6, 6, 4, 3, 2, 0,
0, 0, -2, -5, 10, 8, 7, 6,
6, 5, 1, -1, -1, 0, 9, 7,
6, 5, 5, 4, 1, -1, 0, 1,
7, 6, 5, 4, 4, 3, 1, -1,
0, 2, 5, 4, 4, 3, 3, 3,
1, -1, 0, 3, 4, 4, 4, 3,
1, 2, 1, -1, 1, 3, 6, 6,
6, 5, 5, 6, 4, 1, 3, 6,
8, 8, 8, 7, 8, 9, 7, 4,
6, 9, 8, 8, 8, 7, 8, 8,
7, 5, 6, 8, 8, 7, 8, 7,
7, 7, 6, 4, 5, 2, 7, 6,
6, 5, 4, 3, 2, 1, 1, -3,
12, 10, 9, 9, 9, 7, 2, -1,
0, 0, 11, 9, 8, 7, 8, 6,
1, -2, 0, 1, 10, 8, 6, 4,
5, 5, 1, -3, -1, 1, 8, 6,
4, 3, 3, 4, 0, -3, -1, 1,
7, 5, 4, 2, 1, 3, 1, -3,
-1, 2, 9, 8, 6, 4, 4, 6,
3, -1, 1, 5, 10, 10, 8, 7,
8, 9, 6, 2, 4, 9, 10, 10,
8, 8, 9, 10, 6, 4, 6, 10,
10, 10, 9, 9, 9, 9, 6, 4,
7, 5, 10, 9, 8, 7, 8, 6,
4, 3, 3, 0, 14, 13, 12, 12,
14, 9, 1, -2, 1, 1, 13, 11,
10, 8, 11, 7, 0, -3, 0, 0,
11, 10, 8, 5, 5, 5, 0, -4,
-2, -1, 10, 9, 6, 3, 3, 4,
0, -5, -3, -1, 9, 7, 5, 2,
1, 3, 0, -5, -4, 0, 11, 10,

```

```

    7,  4,  4,  5,  1, -4, -3,  2,
    12, 11,  9,  7,  8, 10,  4, -2,
    1,  7, 12, 11, 10,  9, 10, 11,
    5,  0,  4,  9, 12, 12, 11, 10,
    12, 12,  6,  3,  7,  7, 13, 12,
    11, 10, 12, 11,  6,  5,  5,  3 },
},
{
{
    0,  0, -1, -1, -1, -1, -1, -1,
    -1,  0,  1,  0,  0, -1, -1, -1,
    -1, -1, -1, -1,  1,  0,  0, -1,
    -1, -1, -1, -1, -2, -1,  1,  0,
    0, -1, -1, -1, -1, -2, -2, -2,
    1,  0, -1, -1, -1, -1, -1, -2,
    -2, -2,  0,  0, -1, -1, -2, -1,
    -1, -1, -2, -2,  0, -1, -1, -2,
    -2, -2, -1, -2, -2, -2,  0, -1,
    -1, -2, -2, -2, -2, -2, -1, -2,
    0,  0, -1, -2, -2, -2, -2, -1,
    -1, -1,  0,  0, -1, -2, -2, -2,
    -2, -1, -1, -1,  0,  0, -1, -1,
    -1, -1, -1,  0,  0,  0,  1,  1,
    0, -1, -1, -1,  0,  0, -1,  0,
    1,  0,  0, -1, -1, -1, -1, -1,
    -1, -1,  1,  0,  0, -1, -1, -1,
    -1, -1, -1, -2,  1,  0, -1, -1,
    -1, -1, -1, -2, -2, -2,  0,  0,
    -1, -1, -2, -1, -1, -2, -2, -2,
    0,  0, -1, -2, -2, -1, -1, -2,
    -1, -1,  0,  0, -1, -2, -2, -2,
    -2, -1, -1, -1,  0,  0, -1, -2,
    -2, -2, -1, -1, -1, -1,  0,  0,
    0, -1, -2, -1, -1,  0,  0,  0,
    0, -1, -1, -2, -2, -2, -1,  0,
    0,  0,  1,  1,  0,  0, -1, -1,
    0,  0,  0,  0,  1,  1,  0,  0,
    -1, -1, -1, -1, -1, -1,  1,  0,
    0, -1, -1, -1, -1, -1, -1, -2,
    1,  0,  0, -1, -1, -1, -1, -1,
    -2, -2,  0,  0, -1, -1, -2, -1,
    -1, -1, -2, -2,  0,  0, -1, -1,
    -2, -2, -2, -1, -1, -1,  0,  0,
    -1, -2, -2, -2, -1, -1, -1, -1,
    0,  0, -1, -1, -2, -1, -1,  0,
    0,  0,  0,  0,  0, -1, -2, -1,
    -1,  0,  1,  1,  0, -1, -1, -2,
    -3, -2, -1,  0,  0, -2,  1,  1,
    0,  0, -1, -1,  0,  0,  0,  0,
    1,  1,  0,  0,  0, -1, -1, -1,
    -1, -1,  1,  1,  0,  0, -1, -1,
    -1, -1, -1, -2,  1,  0,  0, -1,
    -1, -1, -1, -1, -2, -2,  0,  0,
    0, -1, -2, -1, -1, -1, -1, -1,
    0,  0,  0, -1, -2, -2, -1, -1,
    -1, -1,  0,  0, -1, -1, -2, -1,
    -1,  0,  0,  0,  0,  0,  0, -1,
    -2, -1,  0,  0,  0,  0,  1,  0,
    0, -1, -1, -1,  0,  1,  1,  1,
    0, -1, -1, -2, -3, -3, -1,  0,
    -1, -1,  1,  1,  0, -1, -1, -2,
    0,  0,  0,  0,  1,  1,  0,  0,
    0, -1, -1,  0, -1, -1,  1,  1,
    0,  0,  0, -1, -1, -1, -1, -1,
    1,  0,  0,  0, -1, -1, -1, -1,
    -1, -1,  1,  0,  0, -1, -1, -1,
    -1, -1, -1, -1,  0,  0,  0, -1,
    -2, -1, -1, -1, -1, -1,  0,  0,
    0, -1, -1, -1,  0,  0,  0,  0,
    0,  0,  0, -1, -1, -1,  0,  1,
    1,  1,  1,  0,  0, -1, -1,  0,
    1,  1,  1,  1,  0,  0, -1, -2,

```

```

-3, -2, -1, 0, -1, -1, 1, 1,
0, 0, -1, -2, -1, 0, 0, -2,
1, 1, 0, 0, 0, -1, 0, 0,
-1, -1, 1, 1, 0, 0, 0, -1,
-1, -1, -1, -1, 1, 0, 0, 0,
-1, -1, -1, -1, -1, -1, 1, 0,
0, -1, -1, -1, -1, -1, -1, -1,
0, 0, 0, -1, -1, -1, 0, 0,
0, -1, 0, 0, 0, -1, -1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
-1, 0, 1, 1, 1, 1, 0, 0,
0, 0, 0, 1, 1, 2, 2, 1,
0, 0, -1, -1, -2, -2, -1, 0,
-1, -1, 1, 1, 1, 0, -1, -1,
0, 0, -1, -2, 1, 1, 1, 0,
0, 0, 0, 0, -1, -1, 1, 1,
1, 0, 0, 0, 0, 0, -1, -1,
1, 1, 0, 0, 0, 0, 0, 0,
-1, -1, 1, 0, 0, 0, 0, 0,
0, 0, -1, -1, 1, 0, 0, 0,
-1, 0, 0, 0, 0, -1, 1, 0,
0, 0, 0, 0, 1, 1, 0, 0,
1, 0, 0, 0, 0, 1, 2, 2,
1, 1, 1, 0, 0, 0, 1, 1,
2, 2, 2, 2, 1, 0, 0, -1,
-1, -1, 0, 0, -1, -1, 1, 1,
1, 0, 0, 0, 0, 0, -1, -2,
1, 1, 1, 1, 0, 0, 0, 0,
-1, -2, 1, 1, 1, 1, 0, 0,
0, 0, -1, -1, 1, 1, 1, 0,
0, 0, 0, 0, -1, -1, 0, 0,
0, 0, 0, 0, 0, 0, 0, -1,
0, 0, 0, 0, 1, 1, 1, 0,
0, 0, 1, 0, 0, 0, 1, 2,
2, 1, 1, 0, 1, 0, 0, 0,
1, 2, 2, 2, 2, 1, 1, 1,
0, 0, 1, 2, 3, 3, 3, 2,
1, 1, 0, 0, 0, 0, 0, 0,
-1, -1, 1, 1, 1, 1, 1, 1,
0, 0, -1, -2, 1, 1, 1, 1,
1, 1, 0, 0, -1, -1, 1, 1,
1, 1, 1, 0, 0, 0, -1, -1,
1, 1, 0, 0, 0, 0, 0, 0,
0, -1, 0, 0, 0, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, -1,
1, 2, 1, 1, 0, 0, 1, 0,
0, 0, 1, 3, 2, 2, 1, 1,
1, 1, 0, 0, 2, 3, 3, 3,
2, 1, 1, 1, 0, 1, 2, 3,
4, 3, 3, 2, 1, 1, 1, 1,
1, 1, 0, 0, -1, -1, 1, 1,
1, 1, 1, 1, 1, 0, -1, -2,
1, 1, 1, 1, 1, 1, 1, 0,
-1, -1, 1, 1, 1, 1, 1, 1,
0, 0, -1, -2, 1, 0, 0, 0,
0, 0, 0, 0, 0, -1, 0, 0,
0, 0, 0, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 2, 2, 1,
1, 0, 1, 0, 0, 0, 2, 3,
2, 2, 1, 1, 1, 1, 0, 1,
2, 4, 3, 3, 2, 2, 1, 1,
1, 1, 3, 4, 4, 3, 3, 2 },
{ -1, -1, -1, -1, -2, -2, -3, -3,
-4, -4, -1, -1, -1, -1, -1, -2,
-2, -3, -3, -3, -3, -3, -2, -1,
-1, -1, -2, -2, -1, -1, -3, -3,
-3, -2, -2, -2, -1, 0, 0, 1,
-3, -3, -2, -2, -2, -2, -1, 0,
1, 2, -2, -2, -2, -2, -1, -1,
0, 1, 2, 2, -2, -1, -1, -1,
-1, 0, 0, 1, 1, 2, -1, -1,

```



```

-1, 0, 0, 0, 0, 0, 1, 1,
0, 0, 0, 1, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 1, -1, 0, -1, -2,
-2, -2, -3, -4, -4, -4, -2, -2,
-1, -1, -2, -2, -3, -3, -3, -3,
-2, -2, -2, -2, -2, -1, -2, -2,
-1, -1, -3, -3, -2, -2, -2, -2,
-2, -1, 0, 2, -3, -3, -2, -2,
-2, -2, -1, 0, 1, 1, -3, -2,
-2, -2, -1, 0, 0, 1, 2, 2,
-2, -1, -1, -1, 0, 0, 1, 1,
1, 2, -1, -1, 0, 0, 0, 0,
0, 1, 1, 1, 0, 0, 1, 1,
0, 0, 0, 0, 1, 1, 2, 2,
1, 1, 1, 1, 0, 0, 1, 1,
-1, 0, -1, -2, -2, -3, -3, -4,
-4, -5, -1, -1, -1, -2, -2, -3,
-3, -4, -4, -4, -2, -2, -2, -2,
-2, -2, -2, -2, -2, -2, -3, -2,
-2, -2, -2, -2, -2, -1, 0, 0,
-3, -3, -3, -2, -2, -1, -1, 0,
1, 1, -3, -2, -2, -2, -1, 0,
1, 2, 2, 2, -1, -2, -1, -1,
0, 0, 1, 1, 2, 2, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 1,
1, 1, 4, 2, 1, 1, 1, 1,
0, 1, 0, 1, 0, 0, -1, -2,
-2, -3, -4, -4, -5, -6, -1, -1,
-1, -1, -2, -2, -4, -4, -5, -4,
-2, -2, -1, -1, -2, -2, -2, -3,
-3, -3, -2, -2, -2, -1, -2, -2,
-2, -1, -1, -1, -3, -3, -2, -2,
-1, -1, 0, 0, 0, 0, -2, -2,
-2, -1, -1, 0, 1, 1, 1, 2,
-1, -1, -1, -1, 0, 0, 1, 1,
2, 2, 1, 0, 0, 0, 0, 0,
1, 1, 1, 1, 3, 2, 1, 1,
0, 0, 1, 1, 1, 1, 4, 3,
2, 1, 1, 1, 1, 0, 0, 1,
0, 0, 0, 0, -1, -3, -4, -5,
-6, -6, -1, -1, 0, -1, -1, -2,
-3, -4, -5, -5, -2, -1, -1, -1,
-1, -2, -3, -3, -3, -3, -2, -2,
-2, -1, -1, -1, -2, -2, -2, -1,
-2, -2, -2, -1, 0, 0, -1, 0,
0, 0, -1, -1, -2, -1, 0, 0,
1, 1, 1, 2, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 2, 1,
1, 0, 0, 0, 1, 1, 1, 1,
4, 3, 2, 1, 1, 0, 0, 1,
1, 1, 5, 4, 3, 2, 1, 1,
0, 0, 0, 1, 0, 0, 0, 0,
-1, -2, -3, -5, -6, -6, -1, 0,
0, 0, 0, -2, -3, -4, -4, -4,
-2, -1, -1, 0, 0, -1, -2, -3,
-3, -3, -2, -2, -1, -1, 0, -1,
-2, -2, -2, -1, -2, -2, -1, -1,
0, 0, -1, 0, 0, 1, -1, 0,
-1, 0, 0, 0, 0, 1, 1, 2,
1, 1, 1, 0, 0, 0, 0, 1,
1, 1, 2, 2, 2, 1, 0, 0,
0, 1, 1, 1, 4, 4, 3, 2,
1, 0, 0, 0, 1, 1, 5, 5,
3, 3, 2, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, -1, -2, -3,
-4, -6, -1, 0, 0, 0, 0, -1,
-2, -3, -3, -4, -2, -1, -1, 0,
0, -1, -1, -2, -2, -2, -2, -2,
-1, 0, 0, 0, -1, -1, -1, -1,

```

```

-1, -1, -1, 0, 0, 0, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0,
1, 1, 1, 2, 2, 2, 2, 1,
0, 0, 0, 1, 1, 1, 3, 3,
2, 1, 1, 0, 0, 0, 1, 1,
5, 4, 3, 2, 1, 0, 0, 0,
0, 0, 6, 5, 4, 3, 2, 1,
0, 0, 0, 0, -1, 0, 1, 1,
0, 0, -1, -2, -3, -3, -1, 0,
0, 0, 0, 0, -1, -2, -2, -2,
-2, -1, -1, 0, 0, 0, 0, -1,
-1, -1, -2, -1, -1, 0, 0, 1,
1, 0, 0, 0, -1, 0, -1, 0,
0, 1, 1, 1, 2, 2, 0, 1,
1, 0, 0, 1, 1, 1, 1, 1,
2, 2, 2, 1, 1, 0, 1, 1,
1, 1, 4, 4, 3, 2, 1, 0,
0, 0, 1, 1, 5, 5, 4, 3,
2, 1, 0, 0, 0, 0, 7, 6,
5, 4, 2, 1, 0, 0, 0, 0,
-1, 0, 1, 1, 1, 1, 0, -1,
-2, -2, -2, -1, 0, 0, 0, 0,
0, 0, -1, -1, -2, -1, -1, 0,
1, 1, 1, 1, 0, 0, -2, -1,
-1, -1, 1, 1, 1, 1, 2, 2,
-1, -1, -1, 0, 1, 1, 1, 2,
2, 2, 0, 0, 0, 0, 1, 1,
1, 2, 2, 2, 1, 2, 1, 1,
1, 1, 1, 1, 1, 1, 3, 3,
3, 2, 1, 0, 0, 0, 0, 1,
4, 5, 4, 2, 1, 1, 0, 0,
0, 0, 6, 6, 5, 3, 2, 1,
0, 0, 0, 0, 1, 0, 0, 0,
1, 1, 1, 0, 0, -1, 0, 0,
0, 0, 0, 1, 1, 1, 0, 0,
-1, -1, -1, 0, 0, 1, 1, 1,
1, 2, -1, -2, -1, 0, 1, 1,
1, 2, 2, 2, -1, -1, 0, 0,
1, 1, 2, 2, 2, 2, -1, 0,
0, 1, 1, 1, 1, 2, 2, 2,
0, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 2, 2, 2, 1, 1,
0, 0, 1, 1, 3, 4, 4, 2,
1, 0, 0, 0, 0, 0, 5, 6,
4, 4, 2, 1, 0, 0, 0, 0 },
{
-5, -4, -3, -2, -1, -1, -1, -1,
-1, -1, -4, -3, -3, -1, 0, 0,
0, -1, -1, -1, -4, -3, -2, 0,
0, 1, 0, -1, -2, -2, -3, -2,
-2, 0, 0, 1, 0, -2, -3, -3,
-3, -2, -1, 0, 0, 0, 0, -2,
-4, -6, -3, -1, -1, 0, 0, 0,
-1, -3, -5, -7, -3, -2, -2, 0,
0, 0, -1, -3, -6, -8, -3, -2,
-2, 0, 0, 0, -1, -3, -6, -8,
-3, -2, -1, 1, 1, 0, -1, -3,
-5, -8, -2, -1, -1, 1, 1, 0,
-1, -3, -5, -7, -5, -4, -4, -3,
-2, -2, -1, -1, -1, -1, -4, -4,
-3, -3, -1, 0, 0, -1, -1, -1,
-4, -4, -2, -2, -1, 0, 0, 0,
-1, -1, -3, -3, -2, -1, 0, 0,
1, -1, -2, -2, -3, -3, -2, -1,
0, 0, 0, -1, -3, -4, -3, -3,
-2, -1, 0, 0, 0, -2, -4, -5,
-3, -2, -2, -1, 0, 0, 0, -2,
-4, -6, -2, -2, -2, 0, 0, 0,
0, -2, -4, -6, -1, -1, -1, 1,
0, 0, 0, -1, -3, -6, -1, 0,
0, 1, 1, 1, 1, -1, -3, -5,
-5, -5, -4, -4, -3, -2, -2, -2,

```

```

-1, -1, -5, -5, -4, -3, -3, -2,
-1, -1, -1, 0, -5, -4, -3, -3,
-2, -1, 0, 0, -1, -1, -4, -4,
-3, -2, -1, -1, 0, 0, -1, -1,
-4, -4, -2, -2, -1, 0, 0, 0,
-2, -3, -4, -4, -2, -2, -1, 0,
0, -1, -3, -4, -3, -3, -1, -1,
0, 0, 0, -1, -3, -5, -2, -2,
0, 0, 0, 0, 0, -1, -3, -5,
-1, -1, 1, 1, 1, 0, 0, 0,
-2, -4, 0, 0, 1, 1, 1, 1,
1, 1, -1, -3, -6, -4, -4, -4,
-4, -3, -2, -1, -2, -1, -5, -4,
-4, -3, -3, -3, -2, -2, -1, -1,
-5, -4, -3, -3, -3, -2, -1, -1,
-1, -1, -5, -4, -3, -2, -2, -1,
-1, 0, -1, -1, -5, -5, -3, -2,
-2, -1, 0, 0, -1, -2, -4, -4,
-2, -2, -1, 0, 0, 0, -1, -3,
-4, -4, -2, -1, 0, 0, 0, 0,
-2, -3, -3, -3, -1, 0, 0, 0,
0, 0, -1, -2, -2, -2, 0, 1,
1, 1, 1, 1, 0, -1, -1, 0,
1, 1, 1, 1, 1, 1, 1, 0,
-5, -4, -3, -3, -3, -3, -2, -1,
-1, -2, -5, -3, -3, -3, -3, -2,
-2, -2, -1, -2, -6, -3, -3, -2,
-2, -2, -2, -1, -1, -1, -6, -3,
-2, -2, -2, -2, -1, 0, -1, -1,
-5, -3, -2, -2, -1, -1, -1, 0,
-1, -1, -5, -2, -1, -1, 0, 0,
0, 0, -1, -1, -5, -2, -2, -1,
0, 0, 0, 1, 0, -1, -4, -2,
-1, 0, 0, 1, 1, 1, 0, 0,
-3, -1, 0, 0, 1, 1, 1, 1,
1, 1, -1, 0, 1, 1, 1, 1,
1, 2, 2, 2, -3, -2, -2, -2,
-2, -2, -2, -1, -1, -1, -4, -2,
-2, -2, -1, -2, -2, -2, -1, -1,
-4, -2, -2, -1, -1, -1, -2, -1,
-1, -1, -4, -2, -1, -1, -1, -1,
-1, -1, -1, -1, -4, -1, -1, 0,
0, 0, -1, 0, 0, -1, -4, -1,
-1, 0, 1, 1, 0, 0, 0, 0,
-4, -1, -1, 0, 1, 1, 1, 1,
1, 1, -4, -1, -1, -1, 0, 1,
1, 1, 1, 1, -3, -1, 0, 0,
1, 1, 1, 2, 2, 2, -2, 0,
1, 1, 1, 1, 1, 2, 2, 3,
-1, 0, 0, 0, 0, 0, -1, -1,
-1, -2, -1, 0, 0, 0, 0, 0,
-1, -1, -1, -1, 0, 0, 1, 1,
1, 0, -1, -1, -1, -1, 0, 0,
1, 1, 1, 1, 0, -1, -1, 0,
-1, 0, 1, 1, 2, 1, 0, 0,
0, 0, -2, -1, 0, 1, 2, 2,
0, 0, 1, 1, -2, -2, -1, 0,
1, 2, 0, 0, 1, 1, -2, -2,
-1, -1, 1, 1, 0, 1, 1, 2,
-2, -1, 0, 0, 1, 1, 1, 1,
2, 2, -1, 0, 1, 1, 1, 1,
1, 2, 2, 3, 2, 2, 2, 2,
2, 2, 1, 0, 0, -2, 2, 2,
3, 3, 3, 2, 1, 0, 0, -2,
2, 3, 3, 4, 3, 3, 1, 0,
0, -2, 2, 3, 3, 4, 4, 3,
1, 0, 0, 0, 2, 3, 3, 4,
4, 3, 1, 0, 0, 0, -1, 0,
1, 3, 4, 3, 1, 0, 0, 0,
-3, -2, -1, 0, 3, 2, 1, 0,
0, 1, -3, -2, -1, 0, 2, 2,

```

```

0, 0, 1, 1, -2, -1, -1, 0,
0, 1, 0, 0, 1, 2, -1, 0,
0, 1, 1, 1, 1, 1, 2, 2,
4, 5, 5, 5, 5, 4, 3, 2,
0, -2, 4, 5, 6, 6, 5, 4,
3, 2, -1, -2, 4, 5, 6, 6,
6, 5, 3, 2, 0, -2, 4, 5,
6, 7, 6, 5, 3, 2, 1, -2,
3, 4, 6, 7, 6, 5, 3, 2,
1, -1, 0, 1, 3, 6, 6, 4,
2, 1, 0, -1, -2, -1, 0, 3,
5, 4, 2, 1, 0, 0, -3, -2,
-1, 1, 3, 3, 1, 0, 0, 0,
-2, -1, 0, 1, 2, 2, 1, 0,
0, 1, -1, -1, 0, 1, 1, 1,
0, 0, 1, 2, 7, 8, 8, 8,
7, 6, 5, 2, 1, -1, 7, 8,
9, 8, 8, 7, 5, 2, 0, -1,
6, 8, 9, 9, 8, 7, 5, 3,
0, -1, 6, 8, 9, 9, 8, 7,
5, 3, 0, -2, 5, 7, 9, 9,
8, 6, 4, 3, 0, -2, 2, 3,
6, 9, 8, 6, 4, 2, 1, -2,
0, 0, 3, 6, 7, 5, 3, 1,
0, -2, -1, -1, 0, 3, 5, 4,
2, 1, 0, -2, -1, -1, 0, 1,
3, 2, 1, 0, 0, 0, -1, -1,
1, 1, 2, 1, 1, 0, 0, 1 },
{
1, 1, 1, 0, 0, -1, -1, 2,
7, 12, 1, 1, 1, 0, 0, -1,
0, 3, 6, 10, 1, 1, 1, 1,
0, -1, 0, 1, 4, 7, 1, 1,
1, 0, -1, -2, -1, 0, 3, 5,
1, 1, 0, -1, -2, -2, -1, 2,
3, 3, 1, 0, -1, -2, -2, -2,
0, 1, 1, 2, 2, 0, -2, -2,
-3, -3, -2, -1, -1, 0, 1, -1,
-2, -3, -4, -4, -4, -3, -3, -2,
0, -2, -4, -4, -5, -6, -6, -5,
-5, -3, -1, -3, -5, -6, -6, -7,
-7, -8, -6, -5, 2, 2, 1, 0,
-1, -2, 0, 3, 6, 9, 2, 2,
2, 1, 0, -1, 2, 4, 6, 7,
3, 2, 2, 1, 0, 0, 2, 3,
5, 5, 3, 2, 1, 1, -1, -1,
0, 3, 3, 3, 2, 2, 1, 0,
-1, 0, 0, 2, 2, 1, 2, 1,
0, -1, -1, -1, 0, 0, 0, 0,
2, 1, -1, -2, -3, -3, -2, -2,
-1, -2, 1, 0, -2, -3, -4, -4,
-4, -3, -3, -3, 0, -1, -3, -4,
-4, -5, -5, -5, -5, -4, -1, -2,
-4, -5, -6, -6, -7, -8, -7, -6,
3, 3, 2, 0, -1, -2, 1, 3,
5, 7, 3, 3, 2, 1, 0, 0,
3, 4, 4, 4, 3, 3, 2, 2,
1, 2, 3, 3, 3, 2, 3, 2,
2, 2, 0, 1, 2, 2, 1, 1,
2, 2, 2, 1, 0, 0, 1, 1,
0, 0, 2, 1, 1, -1, -1, -1,
0, 0, -1, -1, 2, 1, 0, -2,
-2, -2, -2, -2, -2, -2, 1, 0,
-1, -2, -3, -4, -4, -4, -4, -4,
1, -1, -2, -3, -4, -4, -5, -6,
-6, -5, 0, -1, -3, -4, -5, -5,
-6, -7, -8, -7, 4, 3, 1, 1,
1, -1, 1, 2, 3, 4, 4, 3,
2, 2, 1, 1, 2, 3, 2, 2,
3, 3, 3, 2, 1, 2, 2, 2,
1, 1, 3, 2, 2, 2, 1, 1,
1, 1, 0, 0, 2, 2, 2, 1,

```

```

0, 0, 0, 0, -1, -1, 2, 1,
1, 0, -1, 0, -1, -1, -2, -2,
2, 1, 0, -1, -2, -2, -3, -3,
-3, -3, 1, 1, 0, -2, -3, -3,
-4, -5, -5, -4, 1, 0, -1, -2,
-3, -4, -5, -6, -7, -6, 0, -1,
-2, -3, -4, -5, -6, -7, -8, -8,
3, 3, 2, 2, 1, -1, 0, 1,
1, 2, 3, 3, 2, 2, 1, 0,
1, 1, 1, 1, 3, 3, 3, 2,
1, 1, 1, 0, 0, -1, 3, 3,
3, 2, 1, 0, 0, -1, -1, -1,
2, 2, 2, 1, 1, 0, 0, -1,
-2, -2, 2, 2, 1, 1, -1, -1,
-1, -2, -2, -2, 2, 1, 1, 0,
-1, -2, -3, -4, -4, -3, 1, 1,
0, 0, -2, -2, -3, -4, -5, -4,
1, 0, 0, -1, -2, -2, -4, -5,
-6, -6, 0, -1, -1, -1, -2, -3,
-5, -6, -7, -8, 4, 3, 3, 2,
1, 0, 0, 0, 0, 1, 4, 3,
3, 2, 2, 0, 0, 0, 0, -1,
4, 3, 3, 2, 2, 0, 0, -1,
-1, -1, 3, 3, 3, 3, 2, 0,
0, -1, -2, -2, 2, 2, 2, 2,
1, 1, 0, -1, -2, -2, 2, 2,
1, 2, 1, 0, -1, -2, -2, -2,
2, 1, 1, 1, 0, -1, -2, -3,
-3, -3, 1, 1, 1, 1, 0, -1,
-3, -4, -4, -4, 1, 0, 0, 0,
-1, -1, -4, -5, -5, -6, 0, 0,
0, 0, -1, -2, -5, -6, -6, -7,
4, 4, 3, 2, 2, 1, 0, -1,
-1, -1, 4, 4, 3, 3, 2, 1,
0, -1, -1, -1, 4, 4, 4, 3,
2, 1, 0, -1, -1, -1, 3, 3,
3, 3, 2, 2, 1, -1, -2, -2,
2, 2, 2, 2, 2, 2, 1, -1,
-2, -2, 2, 1, 1, 1, 1, 1,
0, -2, -2, -3, 1, 1, 1, 0,
0, -1, -1, -3, -3, -3, 1, 1,
0, 0, -1, -1, -2, -4, -4, -4,
0, 0, 0, -1, -1, -2, -4, -5,
-5, -6, -2, -1, -1, -1, -2, -2,
-5, -6, -6, -7, 4, 4, 3, 3,
2, 2, 1, 0, -1, -1, 4, 4,
4, 3, 2, 2, 1, 0, -1, -1,
4, 4, 4, 3, 3, 3, 2, 0,
-1, -2, 3, 3, 3, 3, 3, 2,
1, 0, -1, -2, 2, 2, 2, 2,
2, 2, 1, 0, -2, -2, 2, 1,
1, 0, 0, 0, 0, -1, -2, -3,
1, 1, 0, -1, -1, -1, -1, -2,
-3, -3, 0, 0, -1, -1, -2, -2,
-2, -4, -4, -4, -1, -1, -2, -2,
-3, -3, -3, -5, -5, -6, -3, -3,
-3, -3, -3, -3, -5, -6, -7, -7,
4, 4, 4, 4, 3, 3, 2, 0,
-1, -1, 4, 5, 5, 4, 3, 3,
2, 1, -1, -1, 3, 4, 4, 4,
3, 3, 2, 0, -1, -1, 3, 3,
3, 3, 3, 3, 2, 0, -1, -2,
2, 2, 1, 1, 2, 2, 2, 0,
-1, -2, 1, 0, 0, 0, 0, 0,
0, -1, -2, -2, 0, -1, -1, -2,
-1, -1, -1, -2, -2, -3, -1, -2,
-2, -3, -2, -2, -2, -3, -3, -4,
-3, -3, -3, -4, -3, -3, -2, -4,
-5, -5, -5, -5, -5, -4, -4, -4,
-4, -5, -6, -7, 4, 4, 4, 4,
3, 3, 2, 0, -1, -1, 5, 5,

```

```

4, 4, 4, 3, 2, 1, 0, -1,
4, 4, 4, 4, 4, 4, 3, 1,
0, -1, 2, 3, 3, 3, 3, 3,
3, 1, 0, -1, 1, 1, 1, 1,
2, 2, 2, 1, 0, -1, 0, -1,
-1, 0, 0, 0, 1, 0, -1, -2,
-3, -2, -2, -2, -1, -1, 0, -1,
-2, -2, -4, -3, -3, -3, -2, -2,
-1, -2, -3, -3, -7, -5, -4, -4,
-3, -3, -2, -3, -4, -4, -9, -7,
-6, -5, -4, -4, -4, -5, -5, -6 },
{ -6, -10, -10, -10, -9, -9, -10, -11,
-12, -12, -5, -8, -9, -8, -7, -6,
-6, -7, -8, -8, -3, -6, -7, -7,
-5, -3, -4, -5, -6, -7, -2, -5,
-6, -5, -4, -1, -2, -3, -5, -5,
0, -3, -4, -4, -2, 0, 0, 0,
-1, -2, 1, -2, -3, -3, -1, 2,
3, 2, 1, 0, 4, -1, -3, -2,
0, 3, 4, 4, 2, 1, 5, 2,
-2, -2, 0, 3, 5, 5, 4, 2,
7, 4, 1, 0, 0, 3, 6, 7,
6, 5, 9, 7, 4, 1, 1, 3,
6, 8, 9, 11, -4, -8, -8, -8,
-8, -9, -10, -11, -12, -13, -2, -6,
-7, -7, -6, -6, -7, -9, -10, -11,
-1, -5, -6, -6, -4, -4, -5, -7,
-9, -9, 0, -3, -5, -5, -3, -2,
-4, -6, -7, -7, 1, -2, -3, -3,
-1, 0, -1, -3, -4, -5, 3, -1,
-2, -2, 0, 1, 2, 0, -1, -2,
4, 0, -1, -1, 0, 3, 3, 2,
0, -2, 6, 2, -1, -1, 0, 3,
4, 3, 1, -1, 7, 5, 2, -1,
1, 3, 5, 5, 3, 2, 9, 6,
4, 0, 1, 3, 5, 7, 6, 5,
-1, -4, -6, -6, -7, -9, -10, -11,
-12, -13, 0, -4, -5, -5, -5, -6,
-8, -9, -10, -11, 1, -3, -5, -4,
-4, -5, -7, -8, -10, -11, 2, -2,
-4, -4, -3, -3, -5, -7, -8, -9,
2, 0, -2, -2, -1, -1, -3, -5,
-5, -5, 3, 1, 0, 0, 0, 1,
0, -2, -3, -4, 5, 2, 0, 0,
0, 2, 2, 0, -2, -3, 6, 4,
0, 0, 0, 2, 2, 1, -1, -2,
8, 5, 2, 0, 0, 3, 4, 2,
1, -1, 9, 7, 4, 1, -1, 2,
5, 5, 4, 3, 0, -1, -3, -4,
-5, -7, -8, -9, -11, -12, 1, -1,
-3, -4, -5, -6, -7, -8, -10, -11,
2, 0, -2, -3, -4, -5, -7, -8,
-9, -10, 2, 1, -2, -3, -3, -4,
-6, -7, -8, -9, 3, 2, 0, -1,
-1, -2, -4, -5, -6, -6, 4, 3,
2, 1, 0, 0, -2, -3, -4, -4,
6, 4, 2, 1, 0, 1, 0, -2,
-3, -4, 7, 5, 2, 1, 0, 1,
1, -1, -2, -3, 8, 6, 3, 0,
0, 2, 1, 0, -1, -2, 10, 7,
4, 2, -1, 1, 3, 2, 2, 1,
1, 0, -1, -2, -3, -4, -7, -8,
-9, -11, 2, 1, 0, -1, -2, -3,
-6, -7, -9, -10, 3, 2, 1, 0,
-2, -3, -6, -7, -8, -9, 3, 2,
1, 1, -2, -2, -5, -6, -7, -7,
3, 3, 2, 2, -1, -1, -4, -4,
-5, -5, 4, 4, 4, 3, 1, 0,
-2, -3, -4, -5, 6, 5, 4, 3,
1, 1, -1, -2, -3, -4, 8, 6,
4, 2, 1, 1, 0, -2, -3, -3,

```

```

8, 6, 4, 2, 1, 1, 0, -1,
-2, -2, 10, 7, 5, 2, 0, 1,
1, 1, 0, 0, 0, 0, 0, -1,
-2, -3, -5, -7, -9, -10, 2, 2,
1, 0, -1, -2, -5, -6, -8, -9,
3, 3, 2, 1, -1, -2, -4, -6,
-7, -8, 3, 3, 3, 2, -1, -1,
-4, -5, -6, -6, 4, 3, 3, 3,
0, 0, -2, -4, -4, -4, 5, 5,
5, 5, 2, 1, -1, -3, -3, -4,
7, 6, 6, 5, 2, 1, 0, -2,
-3, -4, 8, 7, 6, 4, 2, 1,
0, -1, -2, -3, 8, 7, 5, 3,
2, 1, 0, -1, -2, -3, 9, 7,
6, 3, 2, 1, 0, 0, -1, -1,
0, 0, -1, -1, -2, -3, -4, -7,
-9, -11, 2, 1, 1, 0, -1, -2,
-3, -6, -8, -9, 2, 2, 1, 1,
-1, -2, -3, -5, -7, -8, 3, 2,
2, 2, 0, -1, -2, -4, -6, -7,
3, 3, 3, 3, 2, 1, -1, -3,
-4, -4, 5, 6, 6, 5, 4, 2,
1, -1, -3, -4, 8, 8, 7, 6,
5, 4, 3, 0, -2, -3, 9, 8,
7, 6, 5, 4, 3, 1, -1, -3,
10, 8, 7, 6, 5, 4, 3, 1,
-1, -2, 10, 9, 7, 5, 4, 3,
2, 1, 0, -1, -1, -1, -3, -4,
-4, -3, -3, -6, -10, -11, 1, 0,
-1, -2, -3, -3, -3, -5, -8, -10,
2, 1, 1, -1, -2, -2, -2, -4,
-7, -9, 3, 2, 2, 0, -1, -2,
-1, -3, -6, -7, 3, 3, 2, 2,
0, 0, 0, -1, -3, -5, 5, 5,
5, 4, 3, 2, 2, 1, -2, -4,
8, 8, 7, 6, 5, 4, 4, 2,
0, -3, 10, 9, 8, 7, 6, 4,
4, 3, 1, -3, 11, 9, 8, 7,
6, 5, 5, 4, 1, -2, 12, 10,
8, 7, 6, 5, 5, 4, 2, -1,
-3, -5, -6, -6, -4, -4, -4, -6,
-9, -12, -1, -2, -4, -4, -4, -3,
-2, -4, -7, -11, 1, 0, -2, -3,
-3, -2, -2, -2, -5, -9, 2, 1,
0, -2, -2, -2, -1, -1, -4, -7,
3, 2, 1, 0, 0, 0, 0, 0,
-2, -4, 4, 4, 3, 3, 1, 1,
2, 2, 0, -3, 7, 7, 6, 5,
3, 3, 3, 3, 2, -2, 9, 9,
8, 7, 5, 4, 5, 4, 3, -2,
10, 9, 9, 8, 6, 5, 5, 5,
4, -1, 12, 10, 9, 8, 7, 6,
5, 5, 4, -1, -8, -8, -7, -6,
-5, -4, -4, -5, -8, -12, -5, -6,
-6, -6, -4, -3, -2, -3, -6, -10,
-1, -3, -4, -5, -4, -3, -2, -1,
-4, -8, 0, -1, -3, -3, -3, -2,
-1, 0, -3, -6, 2, 1, -1, -2,
-1, -1, 0, 0, 0, -3, 4, 3,
2, 1, 0, 0, 1, 3, 1, -2,
6, 6, 4, 3, 2, 2, 3, 4,
3, -1, 8, 8, 6, 5, 4, 3,
4, 5, 5, 0, 9, 9, 8, 7,
5, 5, 5, 6, 6, 1, 11, 9,
9, 8, 7, 6, 6, 6, 6, 0 },
{
2, 1, 3, 1, -3, -11, -12, -12,
-15, -17, 2, 1, 4, 2, -3, -10,
-8, -9, -12, -15, 2, 2, 5, 2,
-3, -8, -3, -6, -9, -12, 2, 2,
5, 2, -2, -7, -1, -4, -7, -10,
2, 1, 5, 2, -2, -6, -1, -3,

```

```

-6, -9, 0, 0, 3, 0, -3, -5,
-2, -5, -8, -9, -4, -3, 3, -2,
-8, -11, -5, -7, -11, -12, -6, -4,
2, -2, -9, -14, -7, -9, -12, -14,
-9, -7, -1, -3, -10, -14, -8, -11,
-13, -15, -10, -11, -6, -7, -12, -15,
-9, -11, -14, -16, 2, 1, 2, 0,
-5, -9, -10, -10, -12, -15, 2, 1,
2, 0, -5, -8, -7, -8, -10, -13,
1, 1, 3, 1, -4, -6, -3, -6,
-7, -10, 0, 1, 3, 1, -4, -5,
-1, -4, -5, -7, -1, 0, 4, 1,
-3, -5, -1, -3, -3, -5, -4, -2,
1, -1, -4, -5, -2, -4, -4, -5,
-8, -5, -2, -4, -9, -10, -6, -6,
-7, -8, -9, -7, -3, -5, -10, -12,
-7, -8, -9, -10, -9, -8, -4, -5,
-10, -11, -7, -9, -11, -12, -7, -8,
-6, -6, -9, -11, -8, -9, -11, -13,
1, 0, 1, -1, -5, -7, -8, -9,
-10, -13, 0, 0, 1, -1, -5, -6,
-5, -7, -8, -11, 0, 0, 1, -1,
-4, -5, -3, -5, -5, -8, -1, 0,
2, 0, -4, -4, -2, -3, -3, -5,
-2, -1, 2, 0, -3, -4, -2, -2,
-1, -3, -6, -4, 0, -2, -4, -4,
-3, -3, -2, -2, -9, -7, -4, -6,
-8, -9, -6, -6, -5, -4, -10, -9,
-6, -7, -9, -10, -8, -7, -6, -6,
-9, -8, -6, -6, -8, -9, -7, -7,
-8, -8, -6, -7, -6, -6, -7, -8,
-7, -7, -8, -10, 1, 0, 1, -1,
-4, -6, -7, -7, -9, -11, 0, 0,
1, -1, -3, -5, -5, -6, -7, -9,
0, 0, 1, -1, -3, -3, -3, -4,
-4, -5, -2, -1, 1, -1, -3, -3,
-2, -2, -2, -3, -3, -2, 0, -1,
-3, -3, -2, -1, -1, -2, -7, -5,
-2, -3, -3, -3, -3, -3, -1, -1,
-10, -8, -5, -6, -7, -6, -6, -6,
-4, -4, -10, -9, -6, -6, -8, -7,
-7, -7, -6, -5, -8, -8, -6, -6,
-7, -7, -6, -7, -7, -6, -6, -6,
-5, -5, -6, -6, -6, -6, -7, -7,
1, 1, 1, 1, -1, -3, -5, -6,
-8, -10, 0, 0, 1, 1, 0, -2,
-4, -4, -4, -5, -1, 0, 0, 0,
0, -1, -2, -3, -2, -2, -2, -1,
0, 0, 1, 0, -1, -2, -1, -1,
-3, -3, -1, 0, 1, 0, 0, -1,
0, 0, -7, -5, -3, -2, 0, -1,
-2, -2, 0, 0, -9, -7, -5, -4,
-2, -3, -4, -5, -3, -1, -8, -7,
-5, -5, -2, -3, -5, -5, -4, -2,
-6, -6, -4, -4, -2, -3, -5, -5,
-5, -4, -4, -3, -2, -2, 0, -2,
-4, -5, -5, -5, 2, 2, 2, 2,
0, -2, -3, -3, -4, -7, 1, 2,
3, 2, 1, 0, 1, 1, 0, -2,
1, 1, 2, 2, 1, 1, 3, 3,
3, 1, 0, 0, 2, 3, 2, 1,
3, 3, 4, 2, -2, -1, 1, 2,
1, 1, 3, 3, 4, 3, -5, -3,
-1, 0, 0, 0, 1, 0, 1, 1,
-6, -5, -3, -2, -1, -2, -1, -1,
0, 1, -5, -4, -3, -2, -1, -2,
-1, -1, -1, 0, -3, -2, -2, 0,
0, -1, -1, -1, -1, -1, 0, 1,
1, 2, 2, 0, -1, -1, -1, -2,
4, 3, 4, 3, 2, 0, 0, -1,
-1, -5, 3, 4, 5, 6, 6, 5,

```



```

4, 3, 3, 0, 3, 4, 5, 6,
6, 6, 5, 4, 4, 4, 2, 4,
6, 6, 7, 6, 5, 4, 5, 5,
1, 2, 4, 5, 6, 5, 4, 2,
4, 6, -2, -1, 2, 2, 2, 2,
1, -1, 0, 1, -3, -1, 0, 1,
2, 1, 0, -2, -1, 1, -2, -1,
0, 1, 2, 2, 0, -2, -1, 0,
1, 1, 1, 2, 3, 2, 1, -2,
-1, -1, 4, 4, 4, 4, 5, 3,
1, -1, -1, -1, 6, 6, 6, 6,
4, 2, 2, 1, 0, -3, 6, 6,
7, 8, 8, 7, 6, 4, 4, 3,
6, 7, 7, 7, 8, 8, 6, 4,
6, 6, 5, 6, 7, 7, 8, 8,
6, 3, 4, 6, 3, 5, 7, 6,
6, 6, 4, 1, 3, 5, 0, 1,
4, 2, 2, 2, 1, -2, -1, 1,
0, 1, 2, 2, 2, 2, 0, -3,
-2, 0, 1, 2, 2, 2, 3, 2,
0, -3, -2, 0, 3, 3, 3, 3,
4, 3, 0, -2, -2, -1, 7, 7,
6, 5, 6, 4, 1, -1, -1, 0,
9, 8, 9, 9, 7, 5, 3, 2,
2, -1, 10, 9, 9, 9, 10, 9,
6, 4, 5, 4, 10, 9, 9, 8,
10, 9, 6, 3, 5, 7, 9, 9,
8, 7, 9, 9, 5, 1, 3, 6,
7, 9, 8, 5, 6, 7, 3, 0,
2, 4, 3, 4, 5, 2, 2, 2,
0, -3, -2, 0, 3, 4, 4, 2,
2, 2, -1, -3, -2, -1, 3, 4,
3, 2, 3, 3, -1, -3, -3, -1,
5, 6, 4, 3, 4, 3, -1, -3,
-2, -1, 9, 9, 7, 7, 8, 5,
1, -1, -1, 0, 12, 10, 9, 10,
12, 9, 5, 4, 5, 4, 13, 11,
10, 9, 12, 11, 5, 2, 5, 6,
13, 11, 9, 8, 11, 10, 3, -2,
1, 6, 12, 11, 9, 7, 9, 8,
2, -4, -2, 4, 12, 11, 8, 5,
5, 6, 1, -4, -3, 2, 6, 9,
6, 1, 1, 2, -1, -5, -5, -1,
6, 7, 5, 1, 2, 2, -2, -5,
-5, -2, 7, 7, 4, 1, 4, 3,
-2, -5, -4, -2, 9, 9, 4, 3,
7, 5, -1, -4, -2, -2, 13, 11,
9, 11, 12, 8, 1, -2, 0, 0 },

```

```

},
{

```

```

{ 0, 0, -1, -1, 0, 0, 1, 1,
2, 2, 0, 0, -1, -1, 0, 0,
1, 1, 2, 2, 0, 0, -1, -1,
-1, 0, 0, 1, 2, 2, -1, 0,
-1, -1, -1, -1, 0, 1, 1, 1,
-1, -1, -1, -1, -1, -1, 0, 0,
1, 1, -1, -1, -1, -1, -1, -1,
-1, 0, 0, 0, -2, -2, -1, -1,
-1, -1, -1, 0, 0, 0, -2, -2,
-2, -2, -1, -1, -1, -1, -1, 0,
-2, -2, -2, -2, -1, -1, -1, -1,
-1, -1, -2, -2, -2, -1, -1, 0,
-1, -1, -2, -2, 1, 1, 0, -1,
0, 0, 1, 1, 2, 2, 1, 1,
0, -1, 0, 0, 1, 1, 2, 2,
1, 1, 0, -1, -1, 0, 1, 1,
2, 2, 0, 0, 0, -1, -1, 0,
0, 1, 1, 2, 0, 0, 0, 0,
-1, -1, 0, 1, 1, 1, -1, -1,
0, 0, -1, -1, 0, 0, 1, 1,
-1, -1, -1, -1, -1, -1, -1, 0,

```

```

0, 0, -1, -1, -1, -1, 0, -1,
-1, -1, -1, 0, -2, -2, -1, -1,
0, 0, -1, -1, -1, -1, -2, -2,
-1, -1, -1, -1, -1, -2, -2, -2,
1, 1, 0, 0, 0, 0, 1, 1,
2, 2, 1, 1, 1, 0, 0, 0,
1, 1, 2, 2, 1, 1, 1, 0,
0, 0, 1, 1, 2, 2, 1, 1,
1, 0, -1, 0, 0, 1, 2, 2,
1, 0, 1, 0, -1, -1, 0, 1,
1, 2, 0, 0, 0, 0, -1, -1,
0, 0, 1, 1, 0, -1, 0, 0,
0, 0, 0, 0, 0, 1, -1, -1,
-1, 0, 0, 0, 0, 0, 0, 0,
-1, -1, -1, -1, 0, 0, -1, -1,
-1, -1, -2, -2, -1, -1, -1, 0,
-1, -2, -2, -2, 2, 1, 0, 0,
0, 1, 1, 2, 2, 3, 2, 2,
1, 0, 0, 0, 1, 1, 2, 3,
2, 2, 1, 0, 0, 0, 1, 1,
2, 2, 2, 2, 2, 1, 0, 0,
1, 1, 2, 2, 1, 1, 1, 1,
0, 0, 0, 1, 2, 2, 0, 0,
1, 1, 0, 0, 0, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, -1, -1, -1, 0, 0, 0,
0, 0, 0, 0, -1, -1, -1, -1,
0, 0, 0, -1, -1, -1, -2, -1,
-1, -1, -1, 0, -1, -1, -2, -1,
2, 2, 1, 0, 0, 1, 1, 2,
2, 3, 2, 2, 1, 0, 0, 1,
1, 2, 2, 3, 2, 2, 2, 1,
0, 0, 1, 2, 2, 3, 2, 2,
2, 1, 0, 0, 1, 1, 2, 3,
1, 1, 1, 1, 1, 0, 0, 1,
2, 2, 1, 1, 1, 1, 0, 0,
0, 1, 1, 2, 0, 0, 0, 0,
0, 0, 0, 0, 1, 1, 0, -1,
-1, 0, 0, 0, 0, 0, 0, 0, 1,
-1, -1, -1, -1, 0, 0, 0, 0,
0, 0, -2, -1, -1, -1, -1, 0,
0, -1, -1, -1, 3, 2, 1, 0,
0, 1, 1, 2, 2, 3, 2, 2,
1, 1, 0, 1, 1, 2, 2, 3,
2, 2, 2, 1, 0, 0, 1, 2,
2, 3, 2, 2, 2, 1, 0, 0,
0, 1, 2, 3, 1, 1, 1, 1,
1, 0, 0, 1, 1, 2, 1, 1,
1, 1, 0, 0, 0, 1, 1, 2,
0, 0, 0, 0, 0, 0, 1, 1,
1, 2, 0, 0, 0, 0, 0, 1,
1, 1, 1, 1, -1, -1, -1, 0,
0, 1, 1, 0, 0, 0, -1, -1,
-1, -1, 0, 0, 1, 0, -1, -1,
3, 2, 1, 1, 0, 1, 1, 2,
3, 3, 2, 2, 2, 1, 0, 0,
1, 2, 2, 3, 2, 2, 2, 1,
0, 0, 1, 1, 2, 3, 1, 2,
2, 1, 1, 0, 0, 1, 2, 3,
1, 1, 1, 1, 0, 0, 0, 1,
2, 2, 1, 1, 1, 1, 0, 0,
1, 1, 2, 2, 0, 0, 0, 0,
0, 1, 1, 1, 2, 2, 0, 0,
0, 0, 1, 1, 1, 1, 1, 2,
-1, -1, 0, 0, 1, 1, 1, 1,
1, 1, -1, -1, -1, 0, 0, 1,
1, 1, 0, 0, 3, 2, 1, 1,
0, 0, 1, 2, 3, 4, 3, 3,
2, 1, 0, 0, 1, 2, 3, 3,
2, 2, 2, 1, 0, 0, 1, 1,
2, 3, 2, 2, 2, 1, 0, 0,

```

```

0, 1, 2, 3, 1, 1, 1, 0,
0, 0, 1, 1, 2, 2, 1, 1,
1, 0, 0, 1, 1, 1, 2, 2,
0, 0, 0, 1, 1, 1, 1, 1,
2, 2, 0, 0, 0, 1, 1, 1,
1, 1, 1, 2, -1, -1, 0, 0,
1, 1, 1, 1, 1, 1, -1, -1,
0, 0, 1, 1, 1, 1, 1, 1,
3, 2, 2, 1, 0, 1, 1, 2,
3, 4, 3, 3, 2, 1, 0, 0,
1, 2, 3, 3, 2, 2, 1, 1,
0, 0, 1, 1, 2, 3, 2, 1,
1, 0, 0, 0, 1, 1, 2, 3,
1, 1, 0, 0, 0, 0, 1, 1,
2, 2, 1, 1, 1, 0, 1, 1,
1, 2, 2, 2, 0, 0, 0, 1,
1, 1, 1, 1, 2, 2, 0, 0,
0, 1, 1, 1, 1, 1, 1, 2,
-1, 0, 0, 1, 1, 1, 1, 1,
1, 2, -1, 0, 0, 1, 1, 1,
1, 1, 1, 1, 3, 2, 1, 0,
0, 1, 2, 2, 3, 4, 3, 2,
1, 0, 0, 0, 1, 2, 3, 3,
2, 2, 1, 0, 0, 0, 1, 2,
2, 3, 2, 1, 0, 0, 0, 0,
1, 1, 2, 3, 1, 1, 0, 0,
0, 1, 1, 2, 2, 3, 1, 1,
1, 1, 1, 1, 1, 2, 2, 3,
0, 0, 1, 1, 1, 2, 1, 1,
2, 2, 0, 0, 1, 1, 2, 2,
1, 1, 2, 2, 0, 0, 0, 1,
2, 1, 1, 1, 1, 2, 0, 0,
0, 1, 1, 1, 1, 1, 1, 1 },
{ 3, 2, 1, 1, 1, 1, 2, 2,
2, 2, 3, 2, 1, 1, 1, 1,
2, 2, 1, 2, 2, 1, 1, 0,
0, 1, 1, 1, 2, 3, 2, 1,
0, 0, 0, 0, 1, 2, 3, 3,
1, 0, 0, -1, 0, 0, 1, 2,
3, 3, 0, 0, -1, -1, 0, 0,
1, 2, 3, 3, 0, -1, -1, -1,
-1, 0, 1, 2, 2, 2, -1, -1,
-1, -2, -1, 0, 1, 1, 2, 2,
-1, -2, -2, -2, -1, 0, 0, 1,
1, 1, -1, -2, -2, -2, -1, -1,
0, 0, 1, 1, 4, 3, 2, 2,
1, 2, 2, 2, 2, 2, 4, 3,
2, 1, 1, 2, 2, 2, 2, 2,
3, 2, 1, 1, 1, 1, 1, 2,
2, 3, 2, 1, 1, 0, 0, 1,
1, 2, 3, 3, 2, 1, 0, 0,
0, 1, 2, 3, 3, 3, 1, 0,
0, 0, 0, 1, 2, 3, 3, 3,
0, 0, -1, -1, 0, 1, 2, 2,
3, 3, -1, -1, -1, -1, 0, 1,
2, 2, 3, 3, 0, -1, -1, -1,
0, 1, 1, 2, 2, 2, -1, -2,
-1, -1, 0, 0, 1, 1, 1, 1,
4, 3, 2, 2, 2, 2, 2, 2,
2, 2, 4, 3, 2, 2, 1, 2,
2, 2, 2, 2, 4, 3, 2, 1,
1, 1, 2, 2, 2, 2, 3, 2,
1, 1, 1, 1, 2, 2, 2, 3,
2, 1, 1, 0, 1, 2, 2, 2,
3, 3, 1, 0, 0, 0, 0, 2,
2, 3, 3, 3, 0, 0, 0, 0,
1, 1, 2, 3, 3, 3, 0, -1,
-1, 0, 0, 1, 2, 3, 3, 3,
0, -1, -1, 0, 0, 1, 2, 2,
2, 2, -1, -1, -1, -1, 0, 1,
1, 2, 1, 1, 5, 4, 2, 2,

```

2, 2, 2, 2, 2, 2, 5, 4,
 2, 2, 1, 2, 2, 2, 2, 2,
 4, 3, 2, 1, 1, 1, 2, 2,
 2, 2, 3, 2, 2, 1, 1, 1,
 2, 2, 2, 2, 2, 2, 1, 0,
 1, 2, 2, 2, 2, 3, 0, 0,
 0, 0, 1, 2, 2, 2, 3, 3,
 0, 0, 0, 0, 1, 2, 2, 3,
 3, 3, 0, -1, -1, 0, 1, 1,
 2, 2, 3, 3, -1, -1, -1, 0,
 1, 1, 2, 2, 2, 2, -1, -1,
 -1, 0, 1, 1, 1, 2, 1, 1,
 5, 4, 3, 2, 2, 2, 2, 2,
 3, 2, 5, 4, 2, 2, 2, 2,
 2, 2, 2, 2, 4, 3, 2, 1,
 1, 2, 2, 2, 2, 2, 3, 2,
 2, 1, 1, 1, 2, 2, 2, 2,
 1, 1, 1, 0, 1, 1, 2, 2,
 2, 2, 0, 0, 0, 0, 1, 1,
 2, 2, 3, 3, 0, 0, 0, 0,
 0, 1, 2, 2, 3, 3, 0, 0,
 0, 0, 0, 1, 2, 2, 2, 2,
 -1, -1, -1, 0, 1, 1, 1, 1,
 1, 1, -1, -1, -1, -1, 1, 1,
 1, 1, 0, 0, 5, 4, 2, 1,
 1, 2, 2, 2, 3, 2, 4, 4,
 2, 2, 2, 2, 2, 2, 2, 2,
 3, 3, 2, 2, 1, 2, 2, 2,
 1, 2, 2, 2, 2, 1, 1, 1,
 2, 2, 1, 1, 1, 1, 1, 0,
 1, 2, 2, 2, 2, 1, 1, 0,
 0, 0, 0, 1, 1, 2, 2, 2,
 0, 0, 0, 0, 0, 1, 1, 2,
 1, 1, 0, -1, 0, 0, 0, 1,
 1, 1, 1, 1, -1, -1, -1, 0,
 0, 0, 0, 0, 0, 0, 0, 0,
 -1, -1, 0, 0, 0, -1, -1, -1,
 4, 3, 1, 0, 0, 1, 1, 2,
 2, 2, 4, 3, 2, 1, 1, 1,
 1, 2, 2, 2, 3, 3, 2, 1,
 1, 1, 1, 1, 1, 1, 1, 2,
 1, 1, 0, 1, 1, 1, 1, 1,
 1, 0, 0, 0, 0, 1, 1, 1,
 0, 1, 0, 0, 0, 0, 0, 0,
 0, 1, 1, 1, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 1, 0, 0,
 -1, 0, 0, -1, -1, -1, -1, 0,
 0, 0, 0, -1, 0, -1, -1, -2,
 -2, -1, 0, -1, 0, 0, -1, 0,
 -1, -2, -3, -3, 3, 2, 0, -1,
 0, 1, 1, 2, 3, 2, 3, 3,
 1, 0, 0, 0, 1, 2, 2, 2,
 2, 2, 2, 1, 0, 1, 1, 1,
 1, 2, 1, 1, 1, 0, 0, 0,
 1, 1, 1, 1, 0, 0, 0, 0,
 0, 0, 0, 1, 1, 1, -1, -1,
 -1, -1, 0, 0, 0, 0, 0, 1,
 0, -1, -1, -1, 0, 0, -1, -1,
 -1, -1, 0, 0, 0, -1, -1, -1,
 -2, -2, -2, -2, 0, 0, -1, -1,
 -1, -1, -2, -3, -3, -3, -1, -1,
 -1, -1, -1, -1, -2, -3, -4, -4,
 1, 1, 0, -1, 0, 0, 1, 2,
 3, 2, 1, 1, 0, 0, -1, 0,
 1, 2, 2, 2, 0, 0, 0, -1,
 -1, 0, 1, 1, 2, 2, -1, -1,
 -1, -1, -1, 0, 0, 1, 1, 2,
 -1, -1, -2, -1, -1, 0, 0, 1,
 1, 1, -1, -2, -2, -2, -1, 0,
 0, 0, 1, 1, -1, -2, -2, -2,
 -1, 0, -1, -1, -1, -1, -1, -1,

```

-1, -1, -1, -1, -1, -2, -2, -2,
-1, -1, -1, -1, -1, -2, -2, -3,
-3, -4, -1, -1, -1, -1, -2, -2,
-2, -3, -4, -4, 0, -1, -1, -1,
-1, 0, 1, 2, 3, 2, -1, -1,
-1, -1, -1, 0, 1, 2, 2, 2,
-1, -2, -2, -1, -1, 0, 1, 1,
2, 2, -2, -2, -2, -2, -1, 0,
1, 1, 2, 2, -2, -2, -2, -2,
-1, -1, 0, 1, 1, 2, -2, -3,
-2, -2, -1, -1, 0, 0, 1, 1,
-2, -2, -2, -2, -1, 0, 0, -1,
0, 0, -2, -2, -2, -2, -1, -1,
-1, -1, -2, -2, -2, -2, -2, -1,
-1, -1, -1, -2, -3, -4, -2, -2,
-2, -2, -2, -2, -2, -2, -4, -5 },
{
5, 5, 3, 2, 0, 0, 0, 0,
0, 4, 6, 6, 5, 3, 1, -1,
-1, -1, -1, 1, 7, 7, 4, 4,
2, 0, -1, -1, -1, -1, 8, 6,
2, 2, 3, 1, 0, 0, -1, -1,
8, 7, 5, 4, 3, 2, 1, 0,
0, 0, 7, 6, 6, 5, 3, 2,
2, 1, 0, 0, 6, 5, 4, 3,
2, 2, 2, 2, 2, 2, 4, 3,
2, 1, 1, 1, 2, 2, 3, 3,
3, 2, 1, 0, 0, 1, 0, 1,
2, 3, 1, 0, 0, -1, -2, -1,
0, 0, 1, 2, 2, 2, 2, 1,
0, 0, 0, 1, 1, 3, 4, 3,
3, 2, 0, -1, -1, 0, 0, 1,
5, 5, 2, 2, 1, 0, -1, -1,
-1, -1, 5, 4, 0, 1, 1, 0,
0, -1, -1, -1, 5, 6, 3, 2,
2, 1, 1, 0, 0, 0, 5, 5,
5, 4, 3, 2, 1, 1, 0, 0,
4, 4, 3, 3, 2, 2, 2, 2,
1, 1, 2, 2, 2, 1, 1, 1,
2, 2, 3, 3, 1, 1, 0, 0,
0, 1, 2, 2, 3, 3, 0, 0,
-1, -1, -1, 0, 1, 1, 2, 3,
0, 0, 0, 1, 0, 1, 1, 1,
1, 3, 1, 1, 1, 1, 0, 0,
0, 0, 0, 1, 2, 2, 3, 2,
1, -1, -1, 0, 0, 0, 3, 3,
1, 0, 1, 0, -1, 0, 0, 0,
2, 3, 2, 1, 1, 1, 0, 0,
0, 0, 2, 3, 4, 3, 2, 1,
1, 1, 0, 0, 2, 2, 2, 2,
1, 2, 2, 1, 1, 1, 1, 1,
1, 0, 0, 1, 2, 2, 2, 2,
0, 0, 0, 0, 0, 1, 2, 3,
3, 3, 0, 0, 0, -1, 0, 0,
1, 2, 3, 3, -2, -2, -1, 0,
1, 1, 1, 1, 2, 3, -1, -1,
0, 0, 0, 0, 0, 1, 1, 1,
0, 0, 1, 1, 0, 0, 0, 0,
1, 1, 0, 1, 1, 1, 0, -1,
0, 0, 1, 1, 0, 1, 2, 0,
1, 0, 0, 1, 1, 1, 0, 0,
1, 2, 1, 1, 1, 1, 1, 0,
0, 0, 1, 1, 0, 1, 2, 1,
1, 1, 0, 0, 0, 0, 0, 1,
2, 2, 2, 2, -1, -1, 0, 0,
0, 1, 2, 3, 3, 3, -1, -1,
0, 0, 0, 0, 1, 2, 3, 3,
-3, -3, -2, 0, 1, 2, 2, 2,
2, 3, -3, -2, -2, -1, 0, 1,
1, 1, 2, 2, -2, -2, -1, -1,
0, 0, 0, 1, 1, 1, -2, -2,
-1, 0, 0, 0, 0, 1, 1, 1,

```

```

-2, -2, -1, 0, 0, -1, 0, 1,
1, 1, -2, -2, -1, 0, 0, 0,
1, 1, 1, 1, -1, -1, -1, 0,
0, 0, 1, 1, 1, 1, -1, -1,
-1, 0, 0, 0, 1, 1, 2, 1,
-1, -1, -1, 0, 0, 0, 1, 2,
2, 2, 0, -1, -1, 0, 0, 0,
1, 2, 2, 3, -4, -3, -2, -1,
0, 1, 2, 2, 2, 3, -4, -3,
-3, -1, 0, 1, 1, 2, 2, 2,
-4, -3, -3, -2, -1, 0, 1, 1,
2, 2, -4, -3, -3, -2, -1, 0,
0, 1, 2, 2, -4, -3, -3, -2,
-1, 0, 0, 1, 2, 1, -3, -3,
-3, -2, -1, 0, 0, 1, 2, 2,
-2, -2, -2, -2, -1, 0, 0, 1,
1, 2, -1, -2, -2, -1, 0, 0,
0, 1, 1, 1, -1, -1, -2, -1,
0, 0, 0, 1, 1, 1, -1, -1,
-1, -1, 0, 0, 0, 1, 1, 1,
-4, -3, -2, -1, 0, 1, 2, 3,
3, 3, -5, -4, -3, -2, 0, 1,
1, 2, 2, 3, -5, -4, -3, -2,
-1, 0, 1, 1, 2, 2, -5, -4,
-4, -2, -1, 0, 1, 1, 2, 2,
-4, -4, -4, -3, -1, 0, 0, 1,
2, 2, -4, -4, -4, -3, -2, -1,
0, 1, 2, 2, -3, -3, -3, -3,
-2, -1, 0, 0, 1, 2, -2, -2,
-2, -2, -1, 0, 0, 0, 0, 1,
-1, -2, -2, -2, -1, 0, -1, -1,
0, 0, 0, -1, -1, -1, -1, 0,
-1, -1, -1, 0, -4, -3, -2, -1,
0, 1, 2, 3, 3, 3, -5, -4,
-3, -2, -1, 0, 1, 2, 3, 3,
-5, -4, -3, -2, -1, 0, 1, 2,
2, 3, -6, -5, -4, -2, -1, 0,
0, 1, 2, 2, -6, -5, -4, -3,
-2, -1, 0, 1, 2, 2, -5, -5,
-4, -3, -2, -1, 0, 1, 1, 2,
-4, -4, -4, -3, -2, -1, 0, 0,
1, 1, -2, -3, -3, -3, -2, -1,
0, 0, 0, 0, -2, -2, -2, -2,
-2, -1, -1, -2, -2, -1, 0, -1,
-1, -2, -2, -1, -1, -2, -2, -2,
-4, -3, -2, -1, 0, 1, 1, 2,
3, 3, -5, -4, -3, -2, -1, 0,
1, 2, 2, 3, -6, -4, -3, -2,
-1, 0, 1, 1, 2, 3, -6, -5,
-4, -3, -1, -1, 0, 1, 2, 2,
-6, -5, -4, -3, -2, -1, 0, 0,
1, 2, -5, -5, -4, -3, -2, -1,
0, 0, 1, 1, -4, -5, -4, -3,
-2, -1, 0, 0, 0, 1, -3, -3,
-3, -3, -3, -1, -1, -1, -1, -1,
-2, -2, -3, -3, -3, -2, -2, -2,
-3, -3, -1, -1, -1, -2, -2, -2,
-2, -3, -4, -4, -5, -3, -2, -1,
-1, 0, 1, 1, 2, 3, -6, -4,
-3, -2, -1, 0, 1, 1, 2, 3,
-6, -5, -3, -2, -1, 0, 0, 1,
1, 2, -6, -5, -4, -3, -2, -1,
0, 0, 1, 2, -6, -5, -4, -3,
-2, -1, 0, 0, 1, 1, -5, -5,
-4, -3, -2, -1, 0, 0, 0, 1,
-4, -4, -4, -3, -3, -2, -1, 0,
0, 0, -3, -3, -3, -3, -3, -2,
-1, -1, -2, -2, -2, -2, -2, -3,
-3, -3, -2, -3, -4, -5, -1, -1,
-2, -2, -2, -3, -3, -3, -5, -7 },
{ -4, -1, 0, 1, 1, 1, 0, 1,

```

```

1, 0, -4, -2, 0, 1, 2, 2,
1, 0, -1, -2, -4, -2, 0, 1,
2, 1, 0, -1, -3, -5, -4, -2,
0, 1, 1, -1, -2, -3, -4, -6,
-5, -2, 0, 0, -1, -3, -4, -5,
-6, -8, -5, -3, -1, 0, -2, -4,
-5, -6, -8, -10, -6, -3, -1, -1,
-2, -4, -6, -8, -9, -10, -5, -4,
-2, -2, -2, -4, -7, -8, -10, -11,
-6, -4, -2, -2, -2, -4, -6, -8,
-10, -11, -7, -4, -2, -2, -2, -3,
-5, -7, -9, -11, -5, -3, -1, 1,
1, 2, 2, 3, 4, 4, -5, -3,
0, 1, 2, 3, 3, 2, 2, 2,
-5, -3, 0, 2, 3, 3, 2, 1,
0, 0, -5, -3, -1, 2, 2, 1,
0, -1, -2, -2, -6, -4, -1, 1,
1, -1, -2, -3, -4, -4, -6, -4,
-1, 0, 0, -2, -3, -4, -5, -5,
-7, -5, -2, 0, -1, -2, -4, -5,
-5, -5, -7, -6, -2, -1, -1, -2,
-4, -5, -5, -5, -7, -5, -3, -1,
-1, -1, -3, -4, -4, -5, -8, -5,
-2, -1, -1, -1, -2, -3, -4, -5,
-5, -3, -1, 1, 1, 2, 2, 3,
3, 4, -5, -3, -1, 1, 2, 2,
2, 2, 2, 2, -5, -3, -1, 2,
2, 2, 2, 1, 1, 0, -5, -4,
-1, 2, 2, 1, 0, 0, -1, -1,
-6, -4, -1, 1, 1, 0, -1, -2,
-2, -2, -7, -5, -2, 0, 0, -1,
-2, -3, -3, -2, -7, -5, -3, -1,
0, -1, -2, -3, -3, -2, -7, -6,
-4, -1, 0, -1, -2, -2, -2, -1,
-8, -6, -4, -1, -1, -1, -1, -1,
-1, -1, -9, -6, -4, -2, -1, -1,
-1, -1, -1, -1, -5, -3, -1, 0,
1, 2, 2, 3, 4, 5, -5, -3,
-1, 1, 2, 2, 2, 2, 3, 3,
-5, -3, -1, 1, 2, 2, 2, 1,
1, 1, -5, -4, -2, 1, 1, 1,
1, 0, 0, 0, -5, -4, -2, 0,
0, 0, -1, -1, -1, 0, -7, -5,
-3, -1, 0, -1, -1, -1, -1, 0,
-7, -6, -4, -1, -1, -1, -1, -1,
0, 0, -7, -7, -5, -2, 0, 0,
-1, 0, 0, 1, -8, -6, -6, -3,
-1, 0, 0, 0, 1, 2, -9, -7,
-5, -4, -1, 0, 0, 0, 1, 2,
-4, -3, -1, 0, 1, 2, 3, 4,
5, 6, -4, -3, -1, 0, 1, 2,
3, 4, 4, 5, -4, -3, -1, 0,
1, 2, 3, 3, 3, 3, -4, -3,
-1, 0, 1, 2, 2, 2, 2, 2,
-4, -3, -2, 0, 0, 1, 1, 1,
1, 2, -5, -4, -3, -2, -1, 0,
0, 1, 2, 2, -5, -6, -4, -2,
-1, 0, 0, 1, 2, 3, -6, -6,
-5, -3, -1, 0, 1, 1, 2, 3,
-7, -6, -6, -4, -1, 0, 1, 1,
2, 3, -9, -8, -6, -5, -2, 0,
0, 1, 2, 4, -3, -2, -1, 0,
1, 3, 4, 5, 6, 7, -3, -1,
-1, 0, 1, 3, 4, 5, 6, 6,
-3, -1, -1, 0, 1, 3, 4, 5,
5, 5, -2, -1, -1, 0, 1, 3,
4, 4, 4, 4, -3, -1, -1, 0,
0, 1, 2, 2, 3, 4, -3, -3,
-2, -1, -1, 1, 1, 2, 3, 4,
-4, -4, -3, -2, -1, 0, 1, 2,
3, 4, -5, -4, -4, -3, -1, 0,

```

```

1, 2, 3, 4, -6, -5, -5, -4,
-2, 0, 0, 1, 3, 4, -9, -7,
-6, -5, -3, -1, 0, 1, 2, 4,
0, 0, 0, 0, 1, 2, 4, 5,
6, 7, 0, 0, 0, 1, 1, 3,
4, 5, 6, 7, 0, 1, 1, 1,
1, 3, 5, 5, 6, 6, 0, 1,
1, 1, 1, 3, 4, 4, 5, 5,
0, 1, 1, 0, 0, 1, 2, 3,
4, 5, -1, 0, 0, -1, -1, 0,
1, 3, 4, 5, -1, -1, -1, -1,
-1, 0, 1, 2, 4, 5, -2, -2,
-3, -3, -2, -1, 0, 1, 3, 5,
-4, -4, -4, -3, -3, -1, 0, 1,
3, 4, -7, -6, -5, -4, -3, -2,
-1, 0, 2, 4, 3, 3, 2, 2,
1, 1, 3, 4, 5, 7, 3, 3,
3, 2, 1, 1, 3, 5, 6, 7,
3, 3, 3, 2, 1, 2, 3, 5,
6, 6, 3, 3, 3, 2, 1, 1,
3, 4, 5, 6, 2, 2, 2, 1,
-1, 0, 1, 4, 5, 6, 1, 0,
0, -1, -1, -1, 1, 3, 5, 6,
0, -1, -2, -2, -2, -1, 0, 2,
4, 6, -1, -2, -3, -3, -3, -2,
-1, 1, 3, 5, -3, -4, -4, -4,
-3, -2, -1, 1, 2, 4, -6, -6,
-5, -4, -4, -3, -1, 0, 2, 4,
7, 6, 4, 2, 1, 1, 2, 3,
4, 6, 7, 6, 4, 2, 1, 1,
2, 3, 5, 6, 7, 6, 4, 2,
1, 0, 2, 4, 5, 6, 6, 5,
3, 1, 1, 0, 2, 4, 5, 7,
4, 3, 2, 0, -1, 0, 1, 3,
6, 7, 2, 1, 0, -1, -2, -1,
1, 3, 5, 7, 1, -1, -2, -2,
-2, -2, 0, 2, 4, 6, 0, -2,
-3, -4, -3, -2, -1, 1, 3, 5,
-3, -4, -4, -4, -3, -3, -1, 1,
3, 5, -5, -6, -6, -5, -4, -3,
-2, 0, 3, 5, 13, 10, 7, 4,
2, 1, 1, 2, 3, 5, 12, 10,
7, 4, 2, 1, 1, 2, 4, 5,
12, 10, 7, 4, 2, 1, 1, 3,
4, 6, 11, 9, 6, 3, 1, 0,
1, 3, 5, 6, 9, 8, 5, 2,
0, 0, 1, 3, 5, 7, 7, 5,
3, 1, -1, -1, 0, 3, 5, 6,
5, 3, 1, -1, -2, -1, 0, 2,
5, 7, 3, 1, -1, -3, -3, -2,
-1, 2, 4, 6, 0, -1, -3, -4,
-4, -3, -1, 1, 4, 6, -3, -4,
-4, -5, -4, -3, -2, 1, 3, 5 },
{ -3, 0, 3, 6, 7, 8, 9, 10,
10, 9, -4, 0, 3, 5, 6, 8,
8, 8, 7, 6, -5, -2, 2, 5,
5, 6, 6, 5, 4, 4, -6, -3,
1, 4, 4, 4, 4, 3, 2, 2,
-7, -4, 1, 2, 2, 2, 1, 0,
0, 0, -8, -5, -1, 0, 0, -1,
-1, -2, -2, -2, -9, -7, -4, -2,
-2, -3, -4, -4, -3, -4, -11, -8,
-6, -4, -4, -5, -5, -6, -6, -7,
-13, -11, -8, -6, -6, -6, -7, -8,
-9, -9, -16, -13, -11, -8, -7, -8,
-9, -10, -11, -12, -1, 2, 4, 6,
7, 8, 9, 10, 11, 12, -2, 1,
3, 5, 6, 7, 8, 9, 9, 9,
-4, 0, 2, 4, 5, 6, 6, 6,
6, 6, -5, -2, 1, 4, 4, 4,
4, 3, 3, 4, -7, -4, 0, 2,

```



```

2, 2, 1, 1, 1, 3, -8, -5,
-2, 0, 0, -1, -1, -1, 0, 2,
-9, -6, -4, -2, -2, -2, -2, -2,
-1, 0, -10, -8, -5, -4, -4, -4,
-3, -3, -2, -2, -12, -10, -7, -5,
-5, -5, -5, -5, -4, -4, -14, -12,
-9, -6, -6, -6, -6, -6, -6, -6,
0, 1, 3, 5, 5, 7, 8, 10,
12, 13, -1, 1, 3, 4, 5, 7,
8, 9, 10, 11, -3, 1, 2, 4,
5, 6, 7, 8, 8, 8, -4, -1,
1, 3, 4, 5, 5, 5, 5, 6,
-6, -4, -1, 2, 3, 2, 2, 2,
3, 4, -7, -5, -2, 0, 0, 0,
0, 1, 2, 3, -8, -6, -4, -1,
-1, -1, -1, 0, 2, 2, -9, -7,
-5, -3, -3, -2, -1, -1, 0, 1,
-11, -8, -6, -4, -3, -3, -3, -2,
-1, -1, -13, -11, -8, -5, -4, -4,
-4, -3, -3, -2, 0, 0, 1, 3,
4, 6, 8, 9, 11, 12, -1, 0,
1, 3, 4, 6, 8, 9, 10, 11,
-2, -1, 0, 3, 4, 5, 7, 8,
9, 9, -4, -2, 0, 2, 4, 5,
6, 6, 7, 7, -5, -4, -1, 1,
3, 3, 3, 4, 5, 5, -7, -5,
-3, -1, 0, 1, 1, 2, 3, 4,
-8, -6, -4, -2, -1, 0, 0, 2,
3, 3, -9, -7, -5, -2, -2, -1,
0, 1, 2, 2, -11, -9, -6, -3,
-2, -1, 0, 0, 0, 1, -14, -11,
-8, -4, -3, -2, -2, -1, -1, -1,
0, 0, 0, 1, 2, 4, 7, 9,
10, 12, -1, -1, 0, 0, 1, 4,
6, 8, 9, 10, -3, -2, -1, 0,
1, 3, 6, 7, 8, 9, -3, -2,
-1, 0, 1, 3, 5, 6, 7, 7,
-4, -3, -2, -1, 0, 2, 3, 4,
5, 6, -5, -4, -4, -2, 0, 0,
2, 3, 4, 5, -7, -6, -5, -4,
-1, 0, 1, 2, 3, 4, -9, -8,
-6, -4, -2, 0, 0, 1, 2, 3,
-11, -9, -7, -5, -2, -1, 0, 0,
1, 1, -14, -11, -9, -7, -3, -2,
-1, -1, -1, 0, 2, 1, 0, 1,
2, 3, 6, 8, 10, 12, 0, -1,
-1, 0, 1, 3, 6, 7, 9, 10,
-2, -2, -1, 0, 1, 2, 5, 7,
8, 9, -3, -2, -1, 0, 1, 2,
4, 6, 6, 7, -4, -3, -2, -1,
0, 1, 3, 4, 5, 5, -5, -4,
-3, -3, 0, 0, 1, 2, 3, 4,
-6, -6, -5, -5, -2, -1, 0, 2,
3, 3, -8, -7, -7, -6, -2, -2,
-1, 1, 2, 3, -11, -9, -8, -6,
-3, -2, -1, 0, 1, 2, -14, -12,
-10, -8, -4, -3, -2, -1, -1, 1,
3, 3, 2, 2, 2, 2, 4, 7,
9, 10, 1, 1, 1, 1, 2, 2,
4, 6, 8, 10, -1, -1, 0, 0,
1, 1, 3, 5, 7, 8, -2, -1,
-1, 0, 0, 0, 2, 4, 6, 7,
-3, -2, -1, -1, -1, 0, 1, 3,
4, 5, -5, -4, -3, -3, -2, -2,
-1, 1, 3, 4, -6, -6, -6, -5,
-4, -3, -2, -1, 2, 3, -8, -7,
-8, -7, -5, -4, -3, -2, 1, 3,
-10, -9, -9, -7, -6, -5, -4, -3,
0, 1, -14, -12, -10, -9, -7, -6,
-5, -4, -1, 0, 5, 5, 4, 4,
4, 3, 2, 4, 6, 9, 3, 3,

```

```

3, 3, 3, 2, 2, 3, 6, 8,
0, 1, 1, 2, 2, 1, 1, 3,
5, 7, -1, 0, 0, 1, 1, 1,
0, 1, 4, 6, -2, -1, 0, 0,
0, -1, -2, 0, 3, 5, -4, -3,
-2, -2, -2, -2, -2, -1, 3, 4,
-5, -5, -6, -5, -4, -4, -3, -2,
1, 3, -7, -7, -8, -7, -6, -5,
-5, -3, 0, 3, -9, -10, -10, -8,
-7, -6, -5, -4, -1, 2, -13, -12,
-11, -10, -9, -8, -6, -5, -3, 1,
8, 7, 7, 6, 4, 2, 1, 1,
3, 5, 5, 5, 5, 5, 4, 2,
1, 2, 4, 6, 3, 3, 3, 3,
3, 1, 1, 1, 3, 6, 1, 1,
2, 2, 2, 1, 0, -1, 2, 5,
-1, 0, 1, 1, 0, -1, -2, -2,
1, 4, -2, -1, -1, -1, -1, -3,
-4, -2, 1, 3, -4, -3, -4, -4,
-5, -5, -5, -4, 0, 3, -5, -5,
-6, -6, -6, -6, -6, -5, -2, 2,
-8, -8, -9, -8, -8, -8, -8, -6,
-3, 1, -12, -12, -11, -10, -10, -10,
-9, -7, -4, 0, 13, 11, 9, 7,
4, 2, 0, 0, 0, 0, 11, 9,
7, 6, 3, 1, 0, 1, 1, 2,
8, 6, 4, 4, 2, 0, 0, -1,
1, 3, 5, 3, 2, 2, 1, -1,
-2, -2, 0, 4, 4, 2, 1, 0,
-1, -3, -3, -3, 0, 3, 2, 0,
-2, -2, -3, -5, -5, -3, 0, 3,
0, -2, -4, -6, -6, -6, -6, -5,
-2, 2, -2, -4, -7, -8, -8, -8,
-7, -6, -3, 1, -4, -7, -9, -9,
-10, -10, -9, -7, -5, 0, -8, -10,
-12, -12, -13, -13, -12, -9, -6, -2 },
{
-16, -13, -11, -11, -17, -14, -5, -4,
-11, -13, -13, -11, -9, -8, -13, -11,
-5, -4, -7, -11, -10, -8, -6, -5,
-10, -9, -4, -3, -5, -9, -10, -7,
-4, -2, -7, -6, -3, -2, -4, -8,
-9, -7, -4, -1, -3, -2, -2, -1,
-2, -7, -10, -7, -4, -1, -3, 0,
0, 1, 0, -6, -10, -7, -4, -2,
-4, 0, 1, 3, 2, -3, -13, -9,
-5, -4, -5, -1, 3, 5, 4, -2,
-17, -13, -8, -7, -7, -2, 3, 5,
5, -1, -20, -16, -12, -10, -10, -4,
2, 5, 4, 0, -15, -12, -11, -8,
-13, -12, -7, -6, -10, -10, -12, -11,
-9, -7, -11, -11, -7, -6, -9, -10,
-8, -8, -6, -5, -10, -10, -7, -6,
-8, -10, -9, -7, -4, -3, -7, -8,
-7, -5, -6, -9, -9, -7, -4, -1,
-3, -3, -4, -4, -5, -8, -8, -6,
-4, -1, -3, 0, -1, -1, -3, -8,
-9, -7, -4, -2, -4, 0, 1, 2,
-1, -6, -11, -7, -5, -3, -5, -1,
2, 3, 1, -4, -15, -12, -8, -6,
-6, -3, 2, 3, 2, -3, -20, -15,
-11, -10, -9, -5, 1, 3, 2, -2,
-12, -11, -9, -7, -10, -10, -8, -8,
-8, -7, -9, -10, -8, -6, -9, -10,
-9, -8, -9, -9, -5, -7, -6, -5,
-9, -10, -8, -8, -9, -9, -5, -6,
-4, -3, -7, -8, -8, -7, -8, -10,
-6, -6, -4, -2, -3, -4, -5, -5,
-7, -9, -6, -5, -3, -1, -3, 0,
-2, -2, -6, -8, -7, -5, -3, -1,
-3, -1, 1, 0, -3, -7, -8, -6,
-4, -3, -4, -2, 1, 1, -2, -6,

```

```

-14, -10, -8, -5, -6, -4, 1, 1,
-1, -4, -19, -14, -11, -9, -9, -5,
0, 1, 0, -3, -8, -9, -8, -6,
-8, -8, -7, -7, -6, -4, -5, -8,
-7, -5, -8, -9, -8, -7, -7, -7,
-1, -5, -5, -5, -7, -8, -8, -8,
-8, -8, -1, -3, -4, -3, -6, -8,
-7, -7, -8, -9, -1, -3, -3, -2,
-3, -4, -5, -6, -7, -9, -3, -4,
-2, -1, -2, -1, -2, -4, -7, -8,
-4, -4, -2, -2, -2, -1, -1, -2,
-5, -8, -5, -5, -3, -3, -3, -2,
0, -1, -4, -6, -11, -9, -7, -5,
-5, -3, 0, 0, -2, -5, -17, -14,
-10, -9, -8, -5, 0, 0, -1, -3,
-3, -5, -6, -4, -5, -5, -4, -3,
-2, -1, 0, -3, -5, -5, -6, -6,
-6, -5, -4, -3, 3, 0, -3, -4,
-6, -6, -6, -6, -6, -6, 3, 1,
-1, -3, -5, -6, -6, -6, -7, -8,
2, 0, -1, -2, -3, -3, -5, -6,
-7, -8, 0, -1, -1, -1, -1, -1,
-3, -4, -6, -8, -2, -2, -2, -2,
-2, -1, -1, -3, -5, -7, -4, -3,
-3, -3, -2, -1, -1, -1, -4, -6,
-9, -7, -7, -5, -4, -2, 0, 0,
-2, -4, -15, -12, -10, -8, -6, -3,
0, 0, -1, -3, -1, -2, -3, -3,
-2, 0, 0, 1, 2, 1, 4, 1,
-2, -3, -3, -1, -1, 0, 0, 0,
6, 4, 0, -2, -3, -2, -2, -2,
-1, -2, 6, 4, 2, -1, -3, -2,
-3, -3, -3, -4, 5, 3, 2, 1,
-1, 0, -3, -4, -4, -5, 3, 2,
1, 0, 0, 0, -1, -3, -4, -6,
1, 0, -1, -1, 0, 1, 1, -1,
-3, -5, -1, -1, -2, -2, -1, 1,
1, 0, -2, -4, -5, -5, -5, -4,
-3, 0, 1, 1, 0, -3, -14, -10,
-9, -7, -4, -1, 0, 1, 0, -2,
1, 0, -1, -1, -1, 1, 2, 3,
3, 2, 4, 2, 0, -1, -1, 1,
1, 1, 2, 1, 6, 4, 1, 0,
-1, 0, 0, 0, 0, -1, 7, 5,
2, 1, -1, 0, -1, -2, -2, -2,
6, 5, 3, 3, 1, 0, -1, -2,
-3, -2, 5, 3, 3, 4, 3, 2,
1, -2, -3, -2, 3, 2, 1, 2,
3, 3, 3, 0, -2, -2, 1, 0,
0, 1, 2, 3, 3, 1, -1, -1,
-4, -3, -2, -1, 2, 3, 3, 2,
1, 0, -11, -8, -7, -5, -1, 0,
1, 1, 1, 0, 1, 1, 0, 0,
1, 5, 4, 5, 5, 5, 3, 2,
1, 0, 1, 4, 4, 4, 4, 4,
5, 3, 1, 0, 0, 3, 4, 3,
3, 3, 6, 4, 2, 1, 0, 2,
2, 2, 3, 2, 7, 5, 3, 3,
1, 1, 1, 2, 2, 2, 7, 5,
5, 4, 3, 3, 3, 2, 2, 2,
6, 4, 4, 5, 6, 6, 5, 4,
2, 2, 4, 3, 3, 4, 5, 6,
6, 5, 4, 3, 0, 1, 2, 3,
4, 6, 6, 5, 5, 2, -7, -4,
-2, 1, 2, 4, 6, 5, 5, 3,
2, 1, 0, -1, 2, 7, 8, 8,
8, 8, 3, 0, -1, -2, 1, 6,
8, 8, 7, 7, 4, 0, -2, -3,
0, 3, 5, 6, 6, 6, 5, 2,
-2, -2, -1, 2, 2, 3, 5, 5,
7, 3, 1, 0, 0, 1, 1, 2,

```

}

{

{

```

0, 0, 0, 0, 1, 1, 0, 0,
1, 1, 1, 1, 0, 1, 1, 1,
0, 1, 1, 1, 0, 0, 1, 1,
1, 0, 0, 0, 1, 0, 1, 0,
0, 0, 0, 1, 0, 0, 1, 0,
1, 1, 0, 1, 0, 0, 0, 1,
0, 1, 1, 1, 0, 1, 0, 0,
1, 0, 0, 0, 0, 1, 1, 0,
1, 1, 0, 1, 1, 1, 0, 1,
1, 1, 0, 0, 1, 1, 1, 1,
0, 1, 0, 1, 1, 0, 0, 1,
0, 0, 0, 0, 0, 0, 0, 1,
1, 0, 0, 1, 1, 0, 0, 1,
0, 1, 1, 1, 1, 1, 0, 0,
0, 1, 0, 1, 0, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 0, 1, 0, 1, 1, 0,
0, 0, 0, 1, 0, 0, 0, 0 },
{
2, 1, 0, 2, 2, 2, 2, 2,
2, 2, 2, 2, 0, 0, 2, 2,
2, 0, 0, 2, 0, 2, 0, 2,
0, 2, 2, 2, 0, 0, 0, 2,
0, 0, 2, 2, 0, 2, 1, 2,
2, 0, 0, 2, 2, 2, 2, 2,
2, 2, 0, 2, 2, 2, 0, 2,
0, 2, 2, 0, 2, 2, 0, 0,
0, 2, 0, 2, 0, 0, 2, 2,
0, 2, 1, 1, 2, 2, 0, 0,
0, 0, 0, 1, 2, 2, 2, 0,
0, 0, 1, 1, 2, 0, 1, 2,
0, 2, 2, 2, 0, 0, 0, 2,
0, 0, 0, 0, 2, 2, 0, 0,
2, 2, 2, 2, 0, 2, 2, 2,
0, 2, 1, 1, 0, 0, 2, 2,
2, 0, 0, 0, 2, 0, 2, 0,
0, 0, 0, 1, 0, 0, 2, 0,
1, 2, 0, 2, 0, 0, 0, 1,
0, 2, 2, 2, 0, 2, 0, 0,
2, 0, 0, 0, 0, 2, 2, 0,
2, 2, 0, 2, 2, 2, 0, 2,
2, 2, 0, 0, 2, 2, 2, 2,
0, 2, 0, 2, 1, 0, 0, 1,
0, 0, 0, 0, 0, 0, 0, 2,
2, 0, 0, 2, 2, 0, 0, 2,
0, 2, 2, 1, 2, 2, 0, 0,
0, 2, 0, 2, 0, 0, 2, 2,
0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0,
2, 2, 0, 1, 0, 2, 2, 0,
0, 0, 0, 2, 0, 0, 0, 0 },
{
1, 2, 3, 1, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 1, 1,
1, 3, 0, 1, 0, 1, 0, 1,
0, 1, 1, 1, 0, 0, 0, 1,
0, 0, 1, 1, 3, 1, 2, 1,
1, 0, 0, 1, 1, 1, 1, 1,
1, 1, 0, 1, 1, 1, 3, 1,
0, 1, 1, 3, 1, 1, 0, 0,
0, 1, 0, 1, 3, 0, 1, 1,
3, 1, 2, 2, 1, 1, 0, 3,
0, 0, 0, 2, 1, 1, 1, 0,
3, 0, 2, 2, 1, 3, 2, 1,
0, 1, 1, 1, 0, 0, 0, 1,
3, 3, 0, 3, 1, 1, 0, 0,
1, 1, 1, 1, 0, 1, 1, 1,
3, 1, 2, 2, 0, 0, 1, 1,
1, 0, 0, 0, 1, 0, 1, 0,
0, 0, 0, 2, 0, 0, 1, 0,
2, 1, 3, 1, 0, 0, 0, 2,
0, 1, 1, 1, 3, 1, 0, 0,

```

```

1, 0, 3, 0, 3, 1, 1, 0,
1, 1, 3, 1, 1, 1, 0, 1,
1, 1, 0, 3, 1, 1, 1, 1,
3, 1, 0, 1, 2, 0, 0, 2,
0, 3, 0, 0, 0, 0, 0, 1,
1, 3, 0, 1, 1, 3, 0, 1,
3, 1, 1, 2, 1, 1, 3, 0,
0, 1, 0, 1, 0, 3, 1, 1,
3, 0, 0, 0, 0, 0, 0, 0,
3, 3, 3, 0, 0, 0, 3, 0,
1, 1, 3, 2, 0, 1, 1, 0,
0, 3, 0, 1, 0, 0, 0, 0 },
{ 4, 2, 3, 1, 4, 4, 1, 4,
1, 4, 1, 1, 5, 5, 1, 1,
1, 3, 5, 4, 5, 4, 0, 4,
0, 1, 4, 4, 5, 0, 0, 4,
5, 5, 1, 4, 3, 1, 2, 4,
1, 5, 5, 1, 4, 1, 1, 1,
4, 4, 0, 4, 1, 1, 3, 4,
0, 1, 1, 3, 4, 1, 5, 5,
0, 4, 5, 4, 3, 0, 1, 1,
3, 1, 2, 2, 1, 4, 0, 3,
0, 5, 5, 2, 4, 1, 1, 0,
3, 0, 2, 2, 1, 3, 2, 1,
5, 1, 1, 4, 5, 5, 5, 1,
3, 3, 5, 3, 1, 4, 0, 0,
4, 4, 4, 4, 0, 1, 1, 4,
3, 4, 2, 2, 0, 0, 1, 1,
4, 0, 0, 5, 1, 5, 1, 0,
0, 0, 0, 2, 0, 5, 4, 0,
2, 1, 3, 1, 5, 5, 5, 2,
0, 4, 1, 1, 3, 1, 0, 0,
4, 0, 3, 5, 3, 4, 1, 0,
1, 1, 3, 1, 4, 4, 0, 1,
1, 1, 0, 3, 1, 1, 1, 4,
3, 4, 0, 4, 2, 5, 5, 2,
0, 3, 0, 5, 0, 5, 0, 1,
1, 3, 5, 4, 4, 3, 0, 1,
3, 1, 1, 2, 1, 1, 3, 0,
0, 1, 0, 1, 0, 3, 1, 1,
3, 5, 5, 5, 5, 0, 5, 0,
3, 3, 3, 0, 0, 0, 3, 0,
4, 4, 3, 2, 0, 1, 1, 5,
5, 3, 0, 4, 0, 0, 0, 0 },
{ 0, 2, 6, 5, 0, 0, 7, 0,
5, 0, 5, 5, 1, 1, 5, 5,
7, 4, 1, 0, 1, 0, 3, 0,
3, 5, 0, 0, 1, 3, 3, 0,
1, 1, 5, 0, 4, 5, 2, 0,
5, 1, 1, 5, 0, 7, 5, 5,
0, 0, 3, 0, 7, 5, 4, 0,
3, 5, 5, 4, 0, 5, 1, 1,
3, 0, 1, 0, 6, 3, 7, 7,
4, 5, 2, 2, 5, 0, 3, 6,
3, 1, 1, 2, 0, 7, 7, 3,
6, 3, 2, 2, 7, 4, 2, 7,
1, 5, 5, 0, 1, 1, 1, 7,
6, 4, 1, 4, 7, 0, 3, 3,
0, 0, 0, 0, 3, 7, 7, 0,
4, 0, 2, 2, 3, 3, 7, 7,
0, 3, 3, 1, 7, 1, 7, 3,
3, 3, 3, 2, 3, 1, 0, 3,
2, 5, 4, 7, 1, 1, 1, 2,
3, 0, 7, 7, 6, 7, 3, 3,
0, 3, 4, 1, 4, 0, 7, 3,
5, 5, 6, 5, 0, 0, 3, 5,
5, 7, 3, 6, 7, 5, 7, 0,
4, 0, 3, 0, 2, 1, 1, 2,
3, 4, 3, 1, 3, 1, 3, 5,
7, 4, 1, 0, 0, 4, 3, 7,
6, 7, 7, 2, 7, 7, 6, 3,

```

```

3, 7, 3, 7, 3, 4, 7, 7,
6, 1, 1, 1, 1, 3, 1, 3,
6, 4, 6, 3, 3, 3, 6, 3,
0, 0, 4, 2, 3, 7, 7, 1,
1, 4, 3, 0, 3, 3, 3, 3 },
{ 9, 2, 6, 5, 9, 9, 11, 9,
5, 9, 5, 5, 10, 1, 5, 5,
7, 4, 1, 9, 10, 9, 8, 9,
3, 5, 9, 9, 10, 3, 3, 9,
10, 10, 5, 9, 4, 5, 2, 9,
5, 10, 10, 5, 9, 7, 5, 5,
0, 9, 3, 9, 7, 5, 4, 9,
8, 5, 5, 4, 0, 5, 1, 1,
3, 9, 1, 9, 6, 3, 7, 7,
4, 5, 2, 2, 5, 9, 8, 6,
3, 1, 10, 2, 9, 7, 7, 8,
6, 8, 2, 2, 11, 4, 2, 11,
1, 5, 5, 9, 1, 1, 1, 7,
6, 4, 10, 4, 11, 9, 8, 8,
0, 0, 0, 9, 8, 7, 7, 9,
4, 0, 2, 2, 3, 3, 11, 7,
9, 8, 8, 10, 11, 10, 7, 8,
8, 8, 8, 2, 8, 10, 0, 8,
2, 5, 4, 11, 1, 10, 1, 2,
8, 0, 11, 7, 6, 7, 8, 3,
9, 8, 4, 10, 4, 9, 7, 3,
5, 5, 6, 5, 9, 9, 8, 5,
5, 11, 8, 6, 7, 5, 11, 9,
4, 0, 3, 0, 2, 10, 10, 2,
8, 4, 8, 1, 8, 1, 8, 5,
7, 4, 1, 0, 0, 4, 8, 7,
6, 7, 11, 2, 11, 11, 6, 3,
3, 11, 3, 11, 3, 4, 11, 11,
6, 1, 10, 1, 1, 8, 1, 8,
6, 4, 6, 3, 8, 8, 6, 3,
0, 0, 4, 2, 3, 11, 7, 1,
1, 4, 3, 9, 3, 3, 3, 3 },
{ 14, 1, 11, 12, 14, 14, 0, 14,
12, 14, 12, 12, 10, 3, 12, 12,
7, 13, 3, 14, 10, 14, 5, 14,
2, 12, 14, 14, 10, 2, 2, 14,
10, 10, 12, 14, 13, 12, 1, 14,
12, 10, 10, 12, 14, 7, 12, 12,
15, 14, 2, 14, 7, 12, 13, 14,
5, 12, 12, 13, 15, 12, 3, 4,
2, 14, 4, 14, 11, 2, 7, 7,
13, 12, 1, 6, 12, 14, 5, 11,
2, 3, 10, 1, 14, 7, 7, 5,
9, 5, 6, 6, 0, 8, 6, 0,
3, 12, 12, 14, 3, 4, 4, 7,
11, 13, 10, 13, 0, 14, 5, 5,
15, 15, 15, 14, 5, 7, 7, 14,
13, 15, 1, 1, 2, 2, 0, 7,
14, 5, 5, 10, 0, 10, 7, 5,
5, 5, 5, 1, 5, 10, 15, 5,
6, 12, 8, 0, 3, 10, 3, 6,
5, 15, 0, 7, 11, 7, 5, 2,
14, 5, 13, 10, 8, 14, 7, 2,
12, 12, 11, 12, 14, 14, 5, 12,
12, 0, 5, 9, 7, 12, 0, 14,
8, 15, 2, 15, 6, 10, 10, 6,
5, 8, 5, 4, 5, 3, 5, 12,
7, 13, 3, 15, 15, 13, 5, 7,
11, 7, 0, 1, 0, 0, 11, 2,
2, 0, 2, 0, 2, 8, 0, 0,
11, 4, 10, 4, 3, 5, 3, 5,
11, 13, 11, 2, 5, 5, 9, 2,
15, 15, 13, 1, 2, 0, 7, 4,
3, 13, 2, 14, 2, 2, 2, 2 },
},
{

```

[illegible]


```

0, 1, 1, 2, 1, 1, 0, 1,
0, 1, 0, 1, 2, 0, 0, 1,
0, 1, 1, 1, 1, 1, 0, 2,
0, 0, 0, 1, 1, 0, 1, 0,
2, 0, 1, 1, 0, 0, 1, 1,
0, 1, 1, 1, 0, 0, 1, 0,
2, 2, 0, 2, 1, 1, 0, 0,
1, 0, 1, 1, 0, 1, 1, 1,
0, 1, 1, 1, 0, 0, 1, 0,
1, 0, 0, 1, 1, 1, 1, 0,
0, 0, 0, 0, 0, 0, 1, 0,
1, 1, 2, 0, 0, 1, 0, 1,
0, 1, 1, 1, 2, 0, 0, 0,
1, 0, 0, 0, 0, 1, 0, 0,
1, 1, 2, 1, 1, 1, 0, 1,
1, 1, 0, 2, 0, 1, 0, 1,
2, 1, 0, 1, 1, 1, 0, 0,
0, 0, 0, 1, 0, 0, 0, 1,
0, 0, 0, 2, 1, 0, 0, 0,
0, 1, 0, 1, 0, 0, 0, 0,
0, 1, 1, 1, 0, 1, 1, 0,
2, 0, 0, 0, 0, 0, 0, 0,
2, 0, 2, 0, 0, 0, 0, 0,
0, 1, 0, 1, 0, 1, 0, 0,
0, 0, 1, 1, 0, 0, 0, 0,
1, 2, 2, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 0, 1,
0, 1, 1, 1, 0, 1, 1, 1,
0, 1, 1, 1, 0, 1, 1, 1,
1, 1, 0, 1, 1, 1, 2, 1,
0, 1, 1, 2, 1, 1, 0, 1,
0, 1, 0, 1, 2, 0, 0, 1,
0, 1, 1, 1, 1, 0, 0, 1,
0, 1, 1, 1, 0, 1, 1, 1,
1, 1, 2, 0, 0, 1, 0, 3,
0, 1, 1, 3, 2, 0, 0, 0,
1, 0, 0, 0, 0, 1, 0, 0,
1, 1, 2, 1, 1, 1, 0, 1,
1, 1, 0, 2, 0, 1, 0, 1,
2, 1, 0, 1, 3, 1, 0, 0,
0, 0, 0, 1, 0, 0, 0, 1,
0, 0, 0, 2, 1, 0, 0, 0,
0, 1, 0, 3, 0, 0, 0, 0,
0, 1, 1, 1, 0, 1, 1, 0,
2, 0, 0, 0, 0, 0, 0, 0,
2, 0, 2, 0, 0, 0, 0, 0,
0, 1, 0, 1, 0, 1, 0, 0,
0, 0, 1, 1, 0, 0, 0, 0 },
{
5, 2, 2, 5, 1, 5, 1, 1,
5, 5, 5, 5, 2, 5, 5, 5,
1, 4, 2, 1, 5, 5, 0, 1,
0, 5, 5, 1, 5, 0, 4, 1,
0, 5, 5, 1, 4, 5, 1, 5,
0, 5, 0, 5, 1, 1, 5, 5,
5, 5, 0, 1, 1, 5, 2, 1,
4, 5, 5, 2, 5, 5, 0, 5,
0, 1, 4, 1, 2, 0, 4, 1,
4, 5, 3, 1, 5, 5, 4, 2,
0, 0, 0, 3, 5, 4, 1, 4,
2, 4, 3, 3, 0, 0, 1, 1,
0, 5, 5, 5, 4, 0, 5, 4,
2, 2, 0, 2, 1, 5, 4, 0,

```

```

5, 4, 1, 5, 0, 1, 1, 5,
4, 5, 3, 1, 4, 0, 1, 4,
1, 0, 0, 5, 1, 5, 1, 4,
0, 0, 0, 0, 0, 0, 5, 0,
1, 5, 2, 0, 0, 5, 0, 3,
0, 5, 1, 3, 2, 4, 4, 0,
1, 4, 4, 0, 0, 5, 4, 4,
5, 5, 2, 5, 1, 5, 4, 5,
5, 1, 4, 2, 4, 5, 0, 5,
2, 5, 0, 5, 3, 5, 0, 4,
0, 0, 0, 1, 0, 4, 0, 5,
4, 4, 0, 2, 5, 4, 0, 4,
4, 1, 0, 3, 4, 0, 4, 0,
0, 1, 1, 1, 0, 1, 1, 4,
2, 4, 0, 0, 0, 0, 0, 4,
2, 4, 2, 0, 4, 4, 4, 4,
4, 1, 4, 1, 0, 1, 4, 4,
0, 4, 1, 1, 0, 4, 0, 0 },
{ 0, 2, 2, 0, 1, 0, 5, 1,
0, 0, 0, 0, 2, 0, 0, 0,
5, 3, 2, 1, 0, 0, 6, 1,
7, 0, 0, 1, 0, 7, 3, 1,
6, 0, 0, 1, 3, 0, 5, 0,
6, 0, 6, 0, 1, 1, 0, 0,
0, 0, 7, 1, 5, 0, 2, 1,
3, 0, 0, 2, 0, 0, 6, 0,
7, 1, 3, 1, 2, 7, 3, 5,
3, 0, 4, 5, 0, 0, 3, 2,
7, 6, 6, 4, 0, 3, 5, 3,
2, 3, 4, 4, 7, 7, 5, 5,
6, 0, 0, 0, 3, 6, 0, 3,
2, 2, 6, 2, 5, 0, 3, 7,
0, 3, 5, 0, 6, 5, 5, 0,
3, 0, 4, 5, 3, 7, 5, 3,
1, 6, 7, 0, 5, 0, 5, 3,
7, 7, 7, 7, 6, 6, 0, 6,
1, 0, 2, 7, 6, 0, 6, 4,
7, 0, 5, 4, 2, 3, 3, 7,
1, 3, 3, 6, 6, 0, 3, 3,
0, 0, 2, 0, 1, 0, 3, 0,
0, 5, 3, 2, 3, 0, 7, 0,
2, 0, 7, 0, 4, 0, 6, 3,
6, 6, 7, 5, 6, 3, 6, 0,
3, 3, 6, 2, 0, 3, 7, 3,
3, 5, 7, 4, 3, 7, 3, 7,
7, 5, 1, 5, 7, 5, 5, 3,
2, 3, 6, 6, 6, 6, 6, 3,
2, 3, 2, 7, 3, 3, 3, 3,
3, 5, 3, 5, 7, 5, 3, 3,
7, 3, 5, 5, 7, 3, 7, 7 },
{ 8, 2, 2, 9, 1, 11, 5, 1,
11, 11, 9, 9, 2, 11, 9, 9,
5, 10, 2, 1, 11, 11, 6, 1,
7, 11, 8, 1, 11, 7, 3, 1,
6, 11, 9, 1, 10, 11, 5, 11,
6, 11, 6, 9, 1, 1, 11, 11,
0, 11, 7, 1, 5, 11, 2, 1,
10, 9, 0, 2, 8, 0, 6, 9,
7, 1, 10, 1, 2, 7, 3, 5,
10, 9, 4, 5, 9, 8, 3, 2,
7, 6, 6, 4, 11, 3, 5, 3,
2, 3, 4, 4, 7, 7, 5, 5,
6, 9, 9, 8, 10, 6, 9, 3,
2, 2, 6, 2, 5, 0, 3, 7,
0, 3, 5, 8, 6, 5, 5, 8,
3, 8, 4, 5, 3, 7, 5, 3,
1, 6, 7, 11, 5, 11, 5, 10,
7, 7, 7, 7, 6, 6, 0, 6,
1, 9, 2, 7, 6, 0, 6, 4,
7, 8, 5, 4, 2, 3, 3, 7,
1, 3, 3, 6, 6, 11, 3, 3,

```


[illegible]

```

2, 2, 2, 2, 3, 2, 2, 2,
1, 3, 3, 1, 2, 2, 3, 2,
3, 2, 1, 1, 2, 3, 1, 2,
3, 3, 1, 1, 3, 2, 1, 2,
3, 3, 3, 1, 1, 2, 2, 3,
1, 2, 3, 1, 1, 2, 3, 1,
1, 1, 2, 2, 1, 2, 3, 1,
1, 1, 3, 2, 3, 3, 1, 1,
3, 3, 1, 1, 1, 1, 1, 3,
1, 3, 3, 1, 2, 1, 1, 3,
3, 1, 1, 0, 3, 3, 1, 1,
3, 1, 3, 1, 3, 3, 3, 1,
3, 3, 3, 3, 1, 2, 1, 3,
1, 3, 1, 1, 3, 1, 1, 1,
3, 1, 0, 1, 1, 1, 1, 1,
1, 2, 2, 2, 1, 2, 1, 1,
3, 2, 3, 1, 3, 2, 1, 3,
2, 1, 3, 3, 3, 3, 3, 2,
3, 1, 1, 1, 3, 1, 3, 1,
1, 1, 1, 3, 3, 2, 1, 1,
2, 2, 3, 2, 2, 2, 1, 2,
2, 1, 1, 3, 3, 2, 3, 1,
3, 2, 3, 2, 0, 3, 3, 0,
3, 3, 3, 1, 3, 3, 3, 2,
1, 1, 3, 1, 0, 1, 3, 1,
1, 1, 3, 1, 3, 3, 1, 1,
3, 1, 2, 1, 3, 1, 1, 1,
3, 3, 3, 1, 3, 3, 3, 3,
3, 3, 3, 3, 3, 3, 3, 3,
3, 1, 3, 0, 3, 1, 1, 3,
3, 1, 1, 1, 3, 3, 1, 3 },
{
2, 2, 4, 5, 2, 2, 1, 2,
2, 2, 2, 2, 4, 2, 2, 2,
1, 4, 4, 5, 2, 2, 4, 2,
3, 2, 5, 5, 2, 4, 1, 2,
4, 4, 1, 5, 4, 2, 1, 2,
4, 4, 4, 1, 5, 2, 2, 4,
1, 2, 3, 5, 1, 2, 4, 5,
1, 1, 2, 2, 5, 2, 4, 5,
1, 5, 4, 2, 4, 3, 1, 1,
4, 4, 5, 5, 1, 5, 1, 4,
1, 4, 4, 5, 2, 1, 1, 4,
4, 1, 5, 0, 3, 4, 5, 1,
4, 1, 4, 1, 4, 4, 4, 1,
4, 4, 4, 4, 1, 2, 1, 3,
1, 4, 1, 5, 4, 5, 1, 1,
4, 5, 0, 5, 1, 1, 5, 1,
5, 2, 2, 2, 1, 2, 1, 1,
3, 2, 3, 5, 4, 2, 1, 4,
2, 1, 4, 3, 4, 4, 4, 2,
3, 5, 1, 5, 4, 1, 4, 1,
5, 1, 1, 4, 4, 2, 1, 1,
2, 2, 4, 2, 2, 2, 1, 2,
2, 5, 1, 4, 4, 2, 3, 1,
4, 2, 3, 2, 0, 4, 4, 0,
4, 4, 4, 1, 4, 4, 4, 2,
1, 1, 4, 1, 0, 1, 4, 1,
1, 5, 4, 5, 4, 3, 1, 1,
3, 1, 2, 1, 3, 5, 1, 1,
4, 4, 4, 1, 4, 4, 4, 4,
4, 4, 4, 3, 4, 4, 4, 4,
4, 1, 4, 0, 4, 1, 1, 4,
3, 1, 1, 5, 3, 4, 1, 4 },
{
3, 3, 6, 7, 3, 3, 2, 3,
3, 3, 3, 3, 6, 3, 3, 3,
1, 6, 6, 7, 3, 3, 6, 3,
4, 3, 7, 7, 3, 6, 2, 3,
6, 6, 1, 7, 6, 3, 2, 3,
6, 6, 6, 1, 7, 3, 3, 6,
0, 3, 4, 7, 1, 3, 6, 7,
2, 1, 3, 3, 7, 3, 6, 7,

```

```

2, 7, 6, 3, 6, 4, 2, 1,
6, 6, 7, 7, 1, 7, 1, 6,
2, 6, 6, 7, 3, 2, 1, 6,
6, 2, 7, 5, 4, 6, 7, 2,
6, 1, 6, 1, 6, 6, 6, 1,
6, 6, 6, 6, 2, 3, 2, 4,
0, 6, 1, 7, 6, 7, 1, 1,
6, 7, 5, 7, 2, 2, 7, 2,
7, 3, 3, 3, 2, 3, 2, 2,
4, 3, 4, 7, 6, 3, 0, 6,
3, 1, 6, 4, 6, 6, 6, 3,
4, 7, 2, 7, 6, 2, 6, 2,
7, 1, 2, 6, 6, 3, 2, 2,
3, 3, 6, 3, 3, 3, 1, 3,
3, 7, 2, 6, 6, 3, 4, 1,
6, 3, 4, 3, 5, 6, 6, 5,
6, 6, 6, 1, 6, 6, 6, 3,
1, 2, 6, 0, 5, 2, 6, 2,
2, 7, 6, 7, 6, 4, 2, 2,
4, 0, 3, 0, 4, 7, 2, 2,
6, 6, 6, 1, 6, 6, 6, 6,
6, 6, 6, 4, 6, 6, 6, 6,
6, 1, 6, 5, 6, 2, 2, 6,
4, 0, 2, 7, 4, 6, 2, 6 },
{ 10, 3, 8, 7, 3, 3, 11, 3,
3, 3, 10, 10, 8, 3, 10, 10,
1, 9, 9, 7, 3, 3, 8, 3,
4, 3, 7, 7, 3, 6, 11, 3,
6, 6, 1, 7, 9, 3, 2, 10,
9, 6, 8, 1, 7, 3, 3, 6,
0, 3, 4, 7, 1, 3, 9, 7,
11, 1, 10, 10, 7, 10, 6, 7,
11, 7, 6, 3, 8, 4, 2, 1,
9, 9, 7, 7, 1, 7, 1, 9,
11, 6, 6, 7, 3, 11, 1, 6,
8, 2, 7, 5, 4, 6, 7, 11,
8, 1, 9, 1, 6, 6, 9, 1,
8, 8, 9, 9, 2, 3, 11, 4,
0, 9, 1, 7, 6, 7, 1, 1,
9, 7, 5, 7, 2, 11, 7, 11,
7, 10, 10, 10, 11, 3, 11, 11,
4, 3, 4, 7, 6, 10, 0, 6,
10, 1, 8, 4, 6, 8, 6, 3,
4, 7, 11, 7, 8, 11, 8, 11,
7, 1, 11, 6, 8, 3, 2, 11,
10, 3, 8, 10, 3, 3, 1, 3,
10, 7, 11, 8, 6, 3, 4, 1,
8, 10, 4, 10, 5, 9, 6, 5,
6, 8, 6, 1, 6, 6, 6, 10,
1, 11, 6, 0, 5, 11, 6, 11,
11, 7, 6, 7, 6, 4, 11, 11,
4, 0, 3, 0, 4, 7, 11, 11,
8, 6, 6, 1, 8, 6, 8, 8,
8, 9, 8, 4, 8, 8, 8, 8,
9, 1, 9, 5, 6, 11, 11, 6,
4, 0, 11, 7, 4, 8, 11, 6 },
{ 6, 0, 9, 1, 0, 7, 10, 0,
7, 7, 6, 6, 9, 7, 6, 6,
12, 14, 14, 1, 7, 7, 9, 0,
15, 0, 1, 1, 7, 13, 10, 0,
13, 13, 12, 1, 14, 7, 8, 6,
14, 13, 9, 12, 1, 7, 7, 13,
5, 0, 15, 1, 12, 0, 14, 1,
10, 12, 6, 6, 1, 6, 13, 1,
10, 1, 13, 7, 9, 15, 8, 12,
14, 14, 11, 11, 12, 1, 12, 14,
10, 13, 13, 11, 0, 10, 12, 13,
9, 8, 11, 4, 15, 13, 11, 10,
9, 12, 14, 12, 13, 13, 14, 12,
9, 9, 14, 14, 8, 0, 10, 2,
5, 14, 12, 1, 13, 1, 12, 12,

```


[illegible]


```

1, 0, 3, 0, 1, 0, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 1, 1, 1, 1,
1, 1, 1, 3, 1, 1, 1, 1,
1, 1, 1, 3, 1, 1, 1, 1 },
{ 4, 5, 1, 1, 3, 1, 4, 4,
1, 2, 1, 1, 1, 5, 1, 1,
4, 1, 1, 4, 1, 5, 1, 4,
1, 5, 4, 4, 1, 1, 1, 4,
1, 1, 4, 4, 1, 1, 0, 4,
1, 1, 1, 1, 4, 1, 4, 1,
5, 5, 0, 4, 4, 1, 1, 4,
4, 4, 1, 1, 1, 1, 1, 1,
1, 4, 1, 5, 1, 1, 0, 4,
1, 4, 3, 3, 4, 4, 1, 1,
1, 1, 1, 3, 1, 4, 4, 1,
1, 4, 3, 3, 3, 1, 0, 1,
1, 1, 1, 4, 1, 1, 1, 4,
1, 1, 1, 1, 0, 5, 4, 2,
4, 1, 4, 4, 1, 1, 4, 4,
1, 4, 3, 3, 0, 1, 0, 1,
4, 1, 4, 1, 4, 1, 3, 1,
2, 2, 2, 0, 1, 1, 4, 1,
0, 4, 1, 1, 1, 1, 1, 3,
2, 5, 1, 4, 1, 1, 4, 1,
4, 1, 1, 1, 1, 5, 0, 4,
1, 1, 1, 1, 4, 4, 1, 4,
1, 4, 1, 1, 4, 5, 1, 4,
1, 1, 3, 4, 0, 4, 1, 0,
1, 1, 1, 1, 1, 1, 1, 1,
4, 1, 1, 0, 5, 1, 1, 1,
1, 3, 1, 3, 1, 1, 1, 1,
1, 0, 3, 0, 1, 0, 1, 4,
1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 1, 1, 1, 1,
4, 1, 1, 3, 1, 4, 1, 1,
1, 4, 1, 3, 1, 1, 1, 1 },
{ 1, 5, 7, 7, 6, 7, 1, 1,
7, 2, 7, 7, 7, 5, 7, 7,
1, 7, 7, 1, 7, 5, 7, 1,
7, 5, 1, 1, 7, 7, 7, 1,
7, 7, 1, 1, 7, 7, 3, 1,
7, 7, 7, 7, 1, 7, 1, 7,
5, 5, 3, 1, 1, 7, 7, 1,
1, 1, 7, 7, 7, 7, 7, 7,
7, 1, 7, 5, 7, 7, 3, 1,
7, 1, 6, 4, 1, 1, 7, 7,
7, 7, 7, 6, 7, 1, 1, 7,
7, 1, 6, 6, 6, 7, 3, 7,
7, 7, 7, 1, 7, 7, 7, 1,
7, 7, 7, 7, 3, 5, 1, 2,
1, 7, 1, 1, 7, 7, 1, 1,
7, 1, 4, 6, 3, 7, 3, 7,
1, 7, 1, 7, 1, 7, 6, 7,
2, 2, 2, 3, 7, 7, 1, 7,
0, 1, 7, 7, 7, 7, 7, 6,
2, 5, 7, 1, 7, 7, 1, 7,
1, 7, 7, 7, 7, 5, 3, 1,
7, 7, 7, 7, 1, 1, 7, 1,
7, 1, 7, 7, 1, 5, 7, 1,
7, 7, 6, 1, 0, 1, 7, 3,
7, 7, 7, 7, 7, 7, 7, 7,
1, 7, 7, 0, 5, 7, 7, 7,
7, 6, 7, 6, 7, 7, 7, 7,
7, 3, 6, 0, 7, 0, 7, 1,
7, 7, 7, 7, 7, 7, 7, 7,
7, 7, 7, 3, 7, 7, 7, 7,
1, 7, 7, 6, 7, 1, 7, 7,
7, 1, 7, 6, 7, 7, 7, 7 },
{ 10, 5, 8, 9, 6, 9, 1, 10,
9, 2, 9, 9, 9, 5, 9, 9,

```

```

10, 11, 9, 1, 9, 5, 8, 1,
7, 5, 10, 1, 9, 7, 7, 1,
9, 9, 10, 10, 11, 9, 3, 10,
9, 9, 9, 9, 10, 9, 10, 9,
5, 5, 3, 10, 10, 9, 9, 10,
1, 10, 9, 9, 9, 9, 9, 9,
7, 10, 11, 5, 11, 7, 3, 10,
11, 10, 6, 4, 10, 10, 7, 11,
7, 9, 8, 6, 9, 1, 10, 8,
8, 1, 6, 6, 6, 8, 3, 7,
8, 11, 8, 10, 8, 8, 9, 10,
8, 8, 8, 9, 3, 5, 1, 2,
1, 9, 10, 10, 9, 7, 10, 10,
8, 10, 4, 6, 3, 7, 3, 7,
10, 7, 10, 9, 10, 9, 6, 11,
2, 2, 2, 3, 8, 7, 1, 7,
0, 10, 8, 7, 11, 9, 8, 6,
2, 5, 7, 10, 8, 8, 1, 7,
10, 9, 11, 9, 8, 5, 3, 1,
11, 9, 9, 9, 1, 1, 7, 1,
8, 10, 11, 8, 10, 5, 7, 10,
8, 9, 6, 1, 0, 10, 9, 3,
9, 8, 7, 11, 8, 11, 8, 9,
10, 11, 8, 0, 5, 11, 7, 11,
8, 6, 7, 6, 7, 7, 7, 7,
7, 3, 6, 0, 7, 0, 7, 1,
8, 8, 8, 11, 8, 8, 8, 8,
8, 8, 8, 3, 8, 8, 8, 8,
10, 9, 8, 6, 7, 10, 7, 8,
7, 1, 7, 6, 7, 8, 7, 7 },
{
9, 2, 13, 8, 1, 8, 4, 9,
8, 6, 8, 8, 8, 2, 8, 8,
9, 14, 8, 10, 8, 2, 13, 10,
7, 12, 9, 10, 8, 7, 7, 10,
8, 8, 9, 9, 14, 8, 11, 9,
8, 8, 8, 8, 9, 8, 9, 8,
12, 12, 5, 9, 9, 8, 8, 9,
4, 9, 8, 8, 8, 8, 8, 8,
7, 9, 14, 2, 14, 7, 5, 9,
14, 9, 1, 3, 9, 9, 7, 14,
7, 8, 13, 1, 8, 4, 9, 13,
13, 4, 1, 1, 0, 13, 11, 7,
13, 14, 13, 9, 13, 13, 8, 9,
13, 13, 13, 8, 11, 12, 4, 6,
10, 8, 9, 9, 8, 7, 9, 9,
13, 9, 3, 1, 5, 7, 11, 7,
9, 7, 9, 8, 9, 8, 1, 14,
6, 6, 6, 11, 13, 7, 10, 7,
15, 9, 13, 7, 14, 8, 13, 1,
6, 12, 7, 9, 13, 13, 4, 7,
9, 8, 14, 8, 13, 2, 5, 4,
14, 8, 8, 8, 10, 10, 7, 10,
13, 9, 14, 13, 9, 12, 7, 9,
13, 8, 0, 10, 15, 9, 8, 11,
8, 13, 7, 14, 13, 14, 13, 8,
9, 14, 13, 15, 12, 14, 7, 14,
13, 1, 7, 1, 7, 7, 7, 7,
7, 11, 1, 15, 7, 15, 7, 4,
13, 13, 13, 14, 13, 13, 13, 13,
13, 13, 13, 5, 13, 13, 13, 13,
9, 8, 13, 1, 7, 9, 7, 13,
7, 4, 7, 1, 7, 13, 7, 7 },
}
}

```

```

Pc_Wiener_Lut_To_Class[4096] =
{
83, 154, 254, 125, 125, 125, 253, 253, 77, 200, 207, 30, 30, 239, 239,
239, 0, 98, 101, 229, 229, 231, 231, 231, 0, 34, 101, 100, 100, 229,
229, 231, 15, 34, 98, 100, 100, 100, 164, 164, 15, 34, 98, 106, 100,

```

96, 164, 164, 15, 14, 43, 106, 96, 96, 96, 164, 15, 14, 43, 106,
106, 96, 96, 184, 83, 154, 64, 110, 125, 239, 253, 253, 119, 225, 87,
231, 64, 64, 239, 239, 120, 101, 100, 229, 236, 231, 202, 231, 15, 98,
228, 100, 150, 202, 236, 231, 15, 40, 18, 100, 228, 96, 228, 88, 15,
43, 106, 68, 96, 193, 224, 88, 199, 43, 34, 106, 104, 184, 224, 184,
11, 14, 43, 96, 96, 184, 96, 184, 243, 212, 244, 22, 239, 239, 253,
253, 99, 192, 198, 230, 22, 22, 231, 239, 8, 163, 140, 230, 230, 230,
230, 231, 47, 32, 226, 82, 226, 230, 179, 164, 41, 32, 226, 226, 82,
193, 179, 224, 106, 42, 42, 226, 193, 82, 193, 184, 15, 42, 42, 193,
193, 193, 193, 184, 11, 43, 42, 106, 193, 193, 193, 184, 241, 157, 64,
255, 255, 255, 255, 255, 240, 66, 228, 228, 22, 22, 22, 22, 10, 150,
228, 228, 230, 146, 230, 232, 199, 150, 150, 228, 146, 146, 146, 184, 199,
42, 62, 146, 146, 193, 193, 184, 47, 42, 62, 193, 146, 193, 193, 184,
199, 42, 42, 146, 146, 146, 193, 179, 199, 42, 42, 146, 193, 146, 193,
184, 138, 237, 158, 237, 239, 253, 255, 255, 156, 17, 236, 88, 88, 231,
231, 22, 171, 105, 224, 224, 232, 146, 232, 22, 12, 104, 146, 146, 146,
146, 224, 232, 12, 146, 146, 146, 146, 146, 232, 199, 106, 234, 146,
146, 146, 146, 146, 199, 42, 234, 146, 146, 146, 146, 146, 199, 42, 146,
146, 146, 146, 146, 184, 145, 179, 236, 158, 158, 110, 253, 29, 11, 179,
238, 237, 238, 238, 238, 88, 41, 156, 242, 88, 88, 88, 232, 2, 12,
107, 104, 232, 232, 232, 232, 232, 12, 170, 234, 234, 2, 232, 2, 179,
12, 156, 234, 234, 2, 2, 234, 179, 12, 156, 234, 234, 234, 2, 2,
179, 199, 156, 234, 234, 234, 234, 146, 184, 133, 211, 179, 179, 110, 110,
110, 253, 160, 166, 179, 238, 238, 238, 238, 238, 129, 35, 179, 236, 88,
88, 88, 179, 137, 170, 104, 242, 88, 88, 2, 2, 171, 170, 156, 104,
232, 2, 2, 232, 12, 170, 156, 234, 2, 2, 2, 189, 170, 156,
234, 2, 2, 2, 12, 156, 234, 179, 234, 2, 2, 179, 186, 219,
179, 179, 237, 110, 110, 253, 155, 179, 60, 231, 238, 238, 238, 238, 28,
58, 188, 179, 88, 238, 238, 238, 226, 238, 234, 179, 104, 88, 2, 88,
14, 179, 170, 242, 104, 232, 2, 2, 39, 2, 170, 156, 232, 232, 232,
2, 179, 170, 170, 234, 232, 232, 2, 2, 170, 170, 156, 234, 234, 234,
2, 179, 90, 132, 95, 254, 159, 159, 159, 253, 44, 71, 154, 222, 254,
254, 125, 159, 55, 183, 52, 86, 250, 250, 254, 125, 39, 26, 145, 103,
180, 246, 250, 215, 39, 26, 183, 73, 103, 180, 180, 30, 39, 39, 183,
145, 73, 247, 247, 87, 76, 39, 26, 145, 73, 102, 247, 87, 25, 76,
76, 57, 145, 73, 102, 202, 83, 132, 254, 125, 159, 159, 254, 159, 44,
209, 154, 64, 125, 125, 125, 159, 26, 121, 241, 86, 64, 207, 254, 125,
84, 0, 103, 103, 207, 246, 215, 215, 39, 0, 57, 227, 52, 52, 196,
30, 39, 15, 57, 145, 52, 102, 207, 87, 76, 53, 10, 77, 103, 97,
197, 87, 25, 15, 15, 0, 34, 73, 18, 100, 74, 182, 252, 252, 215,
125, 71, 69, 99, 78, 206, 244, 244, 244, 125, 159, 8, 133, 116, 198,
198, 244, 244, 215, 46, 33, 163, 190, 116, 198, 198, 215, 47, 33, 163,
116, 190, 140, 190, 230, 47, 46, 33, 163, 190, 190, 190, 230, 156, 47,
33, 33, 163, 190, 190, 82, 25, 40, 41, 41, 163, 62, 226, 193, 177,
127, 64, 255, 255, 255, 255, 255, 121, 225, 87, 196, 22, 22, 22, 22,
10, 102, 148, 230, 198, 198, 22, 22, 199, 40, 62, 226, 230, 198, 198,
22, 47, 40, 62, 62, 226, 226, 198, 230, 47, 41, 32, 62, 226, 226,
140, 230, 47, 47, 41, 62, 62, 226, 226, 230, 47, 47, 41, 42, 32,
32, 226, 193, 49, 125, 158, 158, 110, 29, 29, 255, 58, 162, 178, 231,
231, 22, 22, 22, 58, 120, 66, 228, 228, 196, 196, 22, 171, 18, 150,
228, 228, 228, 230, 22, 149, 18, 150, 150, 228, 228, 62, 230, 11, 42,
150, 62, 228, 146, 226, 230, 11, 42, 42, 62, 62, 193, 193, 230, 169,
47, 42, 42, 42, 193, 62, 193, 85, 238, 179, 158, 158, 110, 253, 29,
179, 179, 158, 158, 237, 237, 238, 22, 226, 59, 17, 236, 236, 164, 164,
22, 189, 59, 105, 224, 224, 228, 228, 230, 189, 107, 104, 146, 146, 224,
146, 228, 189, 107, 150, 146, 146, 146, 146, 228, 149, 43, 146, 146, 146,
146, 146, 228, 11, 199, 42, 146, 96, 146, 146, 193, 62, 92, 88, 214,
238, 89, 89, 253, 42, 228, 110, 158, 158, 158, 158, 239, 85, 137, 179,
237, 237, 237, 238, 231, 156, 170, 233, 17, 236, 88, 88, 228, 109, 171,
107, 105, 224, 232, 224, 232, 171, 171, 170, 232, 232, 232, 232, 189,
12, 156, 104, 146, 232, 2, 232, 149, 14, 156, 146, 146, 146, 146,
158, 205, 179, 179, 179, 179, 110, 253, 179, 254, 122, 110, 110, 158, 158,
110, 81, 122, 179, 238, 2, 158, 158, 238, 10, 179, 42, 179, 237, 238,
238, 238, 238, 58, 179, 233, 242, 88, 88, 88, 204, 179, 107, 233, 104,
232, 88, 88, 179, 12, 107, 156, 104, 232, 232, 2, 149, 12, 170, 156,
156, 234, 234, 179, 90, 132, 95, 254, 159, 159, 159, 19, 155, 222,
222, 254, 254, 125, 159, 55, 115, 71, 245, 222, 222, 250, 254, 39, 55,
115, 71, 86, 250, 250, 215, 39, 55, 115, 119, 52, 86, 246, 250, 25,
39, 26, 183, 119, 103, 180, 246, 39, 39, 26, 26, 119, 103, 103, 180,
76, 39, 39, 26, 183, 145, 73, 247, 108, 132, 95, 254, 89, 50, 69,
239, 128, 209, 245, 154, 254, 134, 102, 80, 39, 60, 241, 161, 127, 157,

250, 125, 0, 177, 243, 87, 116, 127, 223, 254, 10, 240, 115, 71, 86,
 71, 245, 127, 23, 169, 115, 115, 115, 209, 118, 86, 114, 23, 199, 26,
 99, 52, 118, 246, 25, 35, 145, 199, 180, 119, 103, 197, 211, 182, 252,
 252, 244, 213, 95, 17, 65, 78, 210, 244, 244, 244, 206, 80, 5, 133,
 116, 194, 244, 244, 244, 215, 5, 20, 131, 116, 194, 143, 244, 215, 5,
 46, 131, 131, 116, 194, 194, 8, 46, 46, 131, 190, 194, 190, 215,
 45, 46, 37, 33, 190, 190, 116, 198, 175, 47, 33, 33, 33, 163, 163,
 82, 177, 223, 80, 255, 24, 24, 24, 24, 121, 241, 180, 212, 244, 244,
 252, 244, 84, 73, 192, 192, 198, 206, 244, 244, 53, 145, 192, 140, 198,
 143, 206, 244, 169, 41, 32, 140, 140, 198, 198, 198, 169, 8, 32, 32,
 140, 140, 198, 230, 47, 47, 41, 32, 32, 226, 140, 198, 47, 47, 33,
 33, 163, 163, 163, 82, 203, 223, 158, 158, 253, 253, 29, 24, 105, 162,
 157, 157, 64, 239, 255, 255, 185, 120, 66, 197, 87, 196, 22, 244, 189,
 240, 148, 148, 100, 198, 198, 22, 149, 43, 62, 148, 148, 148, 198, 198,
 11, 199, 40, 62, 62, 226, 62, 230, 11, 41, 40, 32, 62, 193, 140,
 198, 181, 47, 41, 32, 32, 42, 32, 226, 155, 210, 85, 238, 158, 89,
 253, 50, 1, 179, 85, 158, 158, 158, 239, 255, 0, 58, 162, 135, 229,
 231, 196, 22, 171, 58, 233, 224, 228, 236, 196, 196, 189, 58, 105, 224,
 228, 164, 228, 230, 189, 12, 18, 150, 224, 146, 228, 228, 189, 199, 40,
 150, 104, 146, 150, 62, 53, 199, 42, 42, 62, 62, 193, 226, 166, 168,
 144, 222, 179, 239, 89, 50, 117, 179, 179, 88, 158, 158, 158, 239, 78,
 233, 189, 56, 237, 237, 237, 239, 28, 10, 59, 17, 236, 236, 164, 231,
 64, 171, 59, 233, 242, 224, 236, 224, 171, 189, 107, 104, 224, 104, 148,
 228, 189, 171, 18, 105, 150, 224, 179, 228, 149, 199, 43, 150, 96, 232,
 146, 193, 219, 233, 220, 238, 158, 29, 17, 253, 70, 110, 179, 158, 110,
 158, 110, 110, 137, 50, 55, 2, 158, 158, 158, 237, 166, 178, 209, 224,
 237, 237, 237, 238, 29, 179, 97, 233, 17, 236, 236, 238, 221, 171, 58,
 107, 242, 242, 88, 88, 61, 189, 171, 107, 233, 104, 232, 88, 149, 12,
 12, 170, 156, 104, 232, 2, 91, 94, 95, 95, 254, 254, 125, 125, 19,
 155, 222, 222, 222, 222, 254, 125, 23, 44, 155, 245, 222, 222, 222, 254,
 23, 55, 115, 16, 245, 86, 222, 254, 160, 7, 44, 115, 71, 86, 86,
 250, 160, 7, 55, 115, 119, 71, 86, 86, 160, 7, 55, 35, 115, 119,
 52, 86, 160, 160, 160, 35, 35, 115, 119, 103, 166, 124, 254, 167, 17,
 50, 235, 97, 49, 211, 83, 239, 95, 80, 69, 253, 23, 0, 209, 155,
 134, 125, 238, 110, 114, 177, 211, 209, 167, 222, 147, 213, 153, 172, 44,
 60, 243, 245, 215, 250, 47, 7, 55, 35, 243, 99, 118, 86, 5, 25,
 55, 83, 83, 243, 51, 127, 23, 160, 7, 7, 76, 51, 119, 52, 211,
 182, 252, 252, 252, 254, 71, 69, 65, 174, 210, 216, 244, 244, 252, 213,
 5, 133, 130, 194, 244, 216, 244, 22, 5, 20, 133, 130, 130, 194, 244,
 244, 46, 46, 20, 131, 130, 194, 194, 46, 46, 33, 131, 131, 116,
 194, 194, 233, 46, 37, 131, 131, 131, 194, 194, 25, 46, 46, 33, 33,
 33, 131, 190, 177, 223, 80, 255, 24, 24, 24, 24, 121, 241, 246, 212,
 244, 244, 252, 244, 84, 73, 192, 192, 143, 206, 244, 244, 53, 145, 192,
 140, 198, 143, 206, 244, 169, 41, 163, 140, 140, 140, 143, 244, 169, 8,
 41, 163, 140, 140, 140, 143, 169, 47, 8, 32, 140, 140, 140, 143, 181,
 47, 8, 33, 33, 163, 163, 140, 2, 205, 158, 158, 253, 29, 29, 24,
 185, 162, 157, 127, 64, 22, 255, 255, 185, 120, 225, 197, 87, 212, 22,
 244, 189, 240, 66, 148, 148, 198, 212, 143, 149, 10, 102, 148, 148, 192,
 192, 143, 11, 10, 40, 62, 148, 226, 226, 198, 11, 41, 40, 40, 81,
 226, 230, 198, 181, 47, 41, 41, 32, 32, 32, 226, 232, 209, 241, 89,
 89, 89, 253, 29, 116, 9, 178, 158, 158, 64, 64, 255, 149, 58, 162,
 178, 197, 231, 196, 22, 171, 58, 120, 66, 197, 197, 196, 196, 189, 58,
 18, 66, 228, 228, 100, 230, 189, 10, 18, 150, 150, 193, 228, 230, 189,
 10, 18, 18, 148, 146, 228, 230, 11, 199, 42, 40, 62, 62, 62, 226,
 28, 177, 230, 22, 110, 56, 89, 253, 43, 179, 108, 238, 158, 158, 110,
 239, 181, 89, 146, 56, 237, 237, 239, 230, 170, 59, 17, 17, 197,
 231, 196, 73, 58, 59, 233, 242, 236, 164, 236, 58, 58, 107, 105, 66,
 224, 224, 228, 189, 171, 107, 18, 150, 224, 193, 228, 149, 199, 43, 18,
 96, 146, 193, 193, 145, 88, 142, 238, 238, 179, 253, 253, 194, 87, 241,
 110, 158, 158, 238, 110, 147, 90, 179, 237, 238, 158, 158, 237, 50, 29,
 162, 238, 237, 237, 237, 45, 91, 96, 233, 17, 242, 236, 238, 126,
 40, 58, 107, 233, 242, 242, 236, 149, 171, 58, 107, 233, 242, 224, 88,
 149, 12, 12, 107, 156, 104, 104, 232, 91, 94, 95, 95, 159, 254, 254,
 69, 128, 74, 132, 132, 132, 95, 95, 159, 172, 19, 74, 6, 132, 132,
 95, 95, 172, 128, 51, 155, 6, 132, 132, 95, 23, 55, 44, 51, 155,
 245, 6, 222, 39, 172, 55, 44, 16, 16, 245, 222, 160, 172, 55, 44,
 51, 16, 71, 245, 160, 7, 55, 55, 44, 115, 115, 52, 108, 75, 245,
 69, 162, 244, 69, 125, 31, 152, 219, 167, 108, 57, 235, 92, 128, 209,
 177, 6, 95, 82, 143, 159, 23, 172, 51, 132, 245, 129, 223, 254, 23,
 232, 128, 83, 210, 132, 86, 147, 169, 4, 133, 245, 51, 132, 28, 246,
 19, 149, 31, 60, 245, 251, 60, 6, 4, 53, 51, 53, 7, 7, 16,

52, 90, 213, 186, 216, 244, 126, 186, 69, 65, 174, 152, 216, 210, 147,
147, 235, 5, 20, 130, 130, 194, 136, 216, 92, 5, 13, 131, 130, 194,
194, 148, 208, 5, 46, 131, 131, 194, 194, 194, 194, 5, 46, 131, 131,
131, 130, 194, 194, 3, 46, 33, 20, 37, 131, 194, 143, 84, 172, 41,
33, 33, 33, 131, 190, 217, 223, 124, 50, 235, 235, 186, 213, 60, 177,
134, 182, 220, 252, 216, 244, 84, 241, 52, 206, 206, 147, 252, 252, 25,
45, 161, 78, 206, 147, 93, 244, 181, 45, 145, 78, 81, 194, 246, 206,
181, 47, 8, 161, 161, 116, 190, 194, 181, 46, 46, 33, 161, 230, 227,
194, 181, 46, 46, 8, 8, 33, 33, 32, 11, 245, 166, 89, 124, 50,
50, 235, 153, 249, 85, 167, 125, 255, 252, 252, 185, 113, 195, 246, 246,
212, 182, 252, 185, 121, 227, 227, 196, 86, 206, 147, 109, 84, 97, 192,
140, 135, 78, 101, 149, 47, 14, 140, 81, 140, 197, 206, 181, 169, 41,
32, 52, 227, 226, 206, 181, 169, 169, 41, 33, 32, 163, 81, 175, 119,
248, 167, 89, 124, 124, 69, 74, 212, 166, 85, 167, 167, 80, 29, 218,
112, 201, 201, 157, 30, 215, 255, 177, 185, 36, 72, 135, 180, 143, 244,
189, 185, 36, 72, 148, 82, 214, 215, 189, 171, 233, 17, 100, 32, 241,
246, 109, 149, 10, 40, 227, 227, 82, 79, 175, 53, 47, 163, 163, 32,
81, 81, 253, 36, 201, 189, 49, 190, 70, 69, 88, 140, 38, 179, 166,
166, 167, 80, 178, 206, 112, 201, 85, 56, 215, 80, 3, 148, 142, 201,
205, 207, 127, 215, 179, 185, 142, 36, 17, 197, 247, 250, 105, 185, 58,
107, 79, 82, 197, 192, 176, 185, 107, 73, 197, 79, 197, 207, 176, 149,
199, 18, 40, 79, 148, 81, 192, 214, 111, 191, 1, 158, 50, 89, 227,
86, 158, 158, 238, 158, 70, 89, 250, 167, 232, 4, 170, 166, 166, 110,
157, 156, 90, 166, 201, 85, 56, 56, 70, 90, 53, 142, 162, 162, 56,
56, 216, 189, 185, 142, 17, 17, 162, 178, 120, 112, 185, 58, 233, 17,
242, 178, 149, 149, 171, 59, 107, 54, 54, 105, 91, 94, 94, 95, 95,
92, 95, 215, 31, 211, 94, 95, 95, 95, 95, 23, 19, 74, 94,
132, 132, 95, 95, 23, 128, 19, 74, 6, 132, 222, 95, 23, 172, 19,
51, 155, 245, 222, 222, 23, 172, 55, 44, 51, 245, 245, 245, 23, 172,
44, 19, 51, 118, 118, 245, 160, 7, 7, 35, 44, 115, 118, 118, 109,
75, 75, 210, 95, 38, 86, 208, 31, 161, 201, 143, 208, 6, 126, 95,
151, 251, 155, 79, 251, 69, 78, 110, 172, 44, 39, 114, 95, 76, 205,
154, 4, 128, 16, 84, 132, 136, 94, 159, 175, 4, 67, 35, 118, 74,
212, 132, 23, 55, 181, 205, 19, 6, 174, 245, 7, 4, 4, 160, 49,
44, 76, 118, 90, 213, 252, 216, 248, 186, 117, 158, 65, 174, 152, 216,
216, 206, 216, 254, 5, 20, 130, 194, 194, 216, 244, 206, 5, 20, 130,
130, 194, 194, 216, 194, 46, 46, 20, 131, 130, 130, 194, 194, 20, 46,
131, 131, 131, 194, 194, 194, 12, 46, 37, 13, 131, 130, 194, 194, 23,
224, 46, 33, 131, 131, 131, 190, 217, 223, 80, 29, 24, 24, 216, 235,
60, 241, 214, 147, 244, 252, 252, 216, 84, 97, 52, 212, 147, 210, 206,
216, 53, 57, 52, 78, 174, 78, 93, 244, 181, 45, 163, 161, 78, 206,
206, 206, 165, 5, 8, 133, 78, 81, 143, 206, 165, 46, 8, 20, 161,
116, 116, 206, 5, 46, 37, 33, 8, 33, 133, 116, 239, 217, 166, 89,
124, 29, 29, 24, 58, 249, 162, 127, 64, 125, 22, 252, 185, 113, 195,
225, 246, 212, 206, 244, 185, 63, 66, 227, 52, 212, 212, 206, 11, 199,
97, 227, 192, 78, 140, 206, 25, 199, 40, 227, 192, 140, 180, 206, 25,
45, 41, 41, 163, 81, 78, 143, 181, 169, 8, 41, 163, 133, 140, 140,
143, 104, 223, 166, 89, 167, 89, 50, 178, 131, 249, 166, 158, 167, 80,
255, 171, 142, 201, 205, 178, 64, 214, 213, 189, 142, 36, 66, 17, 197,
180, 212, 187, 58, 54, 66, 66, 100, 246, 198, 189, 171, 107, 148, 105,
193, 197, 78, 109, 199, 57, 150, 148, 148, 192, 78, 25, 53, 57, 32,
62, 81, 81, 140, 162, 196, 28, 131, 69, 70, 223, 253, 198, 135, 221,
76, 166, 166, 167, 80, 118, 179, 61, 85, 56, 157, 167, 64, 232, 171,
142, 162, 17, 157, 157, 215, 130, 185, 142, 72, 66, 135, 82, 196, 12,
187, 58, 54, 66, 197, 82, 197, 189, 189, 107, 105, 225, 148, 224, 197,
149, 15, 57, 98, 79, 148, 148, 113, 82, 172, 50, 174, 2, 205,
89, 171, 130, 16, 179, 158, 158, 70, 110, 133, 234, 14, 100, 120, 158,
158, 158, 203, 158, 179, 201, 56, 157, 157, 237, 174, 112, 63, 36, 17,
178, 178, 178, 221, 187, 142, 59, 17, 17, 135, 178, 179, 189, 58, 59,
54, 242, 242, 236, 176, 171, 171, 107, 54, 54, 79, 224, 91, 126, 75,
95, 95, 208, 252, 95, 128, 251, 94, 126, 95, 254, 159, 254, 23, 19,
155, 155, 132, 95, 95, 95, 23, 128, 19, 155, 245, 52, 125, 95, 23,
128, 19, 51, 51, 245, 16, 132, 23, 7, 128, 19, 51, 51, 76, 222,
23, 7, 128, 19, 44, 51, 114, 64, 160, 7, 7, 7, 35, 44, 44,
16, 219, 218, 235, 126, 247, 28, 108, 179, 138, 245, 173, 99, 220, 176,
204, 239, 203, 144, 111, 208, 188, 5, 152, 167, 144, 188, 38, 244, 195,
52, 171, 29, 151, 6, 100, 152, 131, 44, 141, 157, 186, 203, 90, 144,
20, 144, 235, 238, 124, 13, 126, 29, 175, 251, 86, 14, 48, 31, 169,
179, 46, 126, 55, 223, 74, 182, 93, 216, 65, 137, 111, 37, 51, 174,
78, 93, 143, 65, 235, 50, 45, 161, 129, 174, 93, 224, 104, 75, 23,
99, 129, 78, 78, 82, 155, 87, 181, 5, 20, 131, 136, 148, 206, 116,

```

165, 21, 225, 65, 81, 130, 226, 208, 21, 21, 119, 9, 137, 136, 141,
136, 31, 21, 9, 9, 9, 163, 78, 116, 249, 223, 127, 80, 235, 24,
235, 248, 241, 52, 118, 250, 246, 220, 135, 186, 7, 145, 192, 202, 117,
174, 66, 162, 160, 57, 78, 192, 81, 179, 206, 227, 160, 8, 73, 103,
78, 140, 130, 198, 181, 73, 66, 247, 140, 225, 230, 132, 165, 99, 79,
202, 32, 161, 20, 250, 21, 21, 20, 20, 115, 131, 99, 78, 171, 115,
167, 223, 69, 69, 69, 235, 90, 201, 205, 85, 118, 114, 208, 24, 185,
121, 225, 103, 87, 202, 246, 252, 187, 240, 192, 87, 118, 202, 135, 87,
61, 84, 79, 66, 192, 135, 86, 135, 160, 99, 73, 192, 150, 190, 236,
147, 25, 65, 163, 102, 42, 148, 62, 205, 181, 46, 240, 41, 115, 81,
226, 116, 178, 148, 10, 190, 89, 208, 69, 208, 175, 64, 83, 85, 223,
85, 127, 29, 232, 60, 201, 162, 114, 157, 208, 135, 176, 61, 59, 54,
157, 205, 88, 206, 185, 58, 107, 105, 102, 140, 247, 232, 187, 42, 240,
120, 150, 135, 104, 224, 176, 0, 73, 236, 79, 200, 117, 163, 39, 57,
65, 17, 16, 225, 193, 62, 195, 50, 190, 232, 202, 141, 166, 69, 249,
110, 167, 176, 166, 205, 167, 80, 222, 179, 117, 156, 63, 224, 167, 157,
92, 179, 142, 59, 54, 17, 116, 135, 118, 189, 58, 59, 225, 202, 97,
208, 185, 189, 58, 54, 236, 2, 157, 208, 187, 171, 105, 10, 79, 183,
59, 105, 160, 0, 200, 240, 225, 150, 119, 102, 210, 131, 40, 11, 179,
222, 222, 208, 127, 161, 239, 233, 131, 218, 238, 208, 86, 28, 39, 238,
179, 217, 114, 64, 52, 138, 219, 36, 59, 60, 162, 106, 128, 0, 204,
249, 59, 162, 56, 118, 128, 103, 142, 61, 59, 233, 36, 195, 78, 170,
48, 170, 241, 34, 54, 54, 55, 55, 35, 121, 121, 107, 54, 54, 188,
75, 75, 75, 126, 245, 89, 214, 151, 123, 94, 126, 126, 95, 95, 159,
4, 123, 251, 94, 126, 126, 126, 159, 4, 31, 251, 251, 132, 126, 126,
254, 4, 31, 27, 251, 251, 6, 6, 222, 4, 31, 31, 27, 251, 251,
245, 114, 23, 31, 27, 27, 27, 251, 51, 245, 23, 23, 31, 27, 27,
27, 19, 114, 203, 218, 204, 204, 126, 222, 179, 162, 1, 139, 218, 218,
38, 95, 98, 179, 1, 139, 221, 38, 38, 95, 69, 158, 123, 144, 221,
204, 191, 191, 69, 198, 208, 144, 144, 218, 218, 191, 221, 95, 109, 208,
144, 168, 6, 218, 218, 89, 130, 46, 172, 112, 185, 152, 218, 244, 4,
173, 172, 181, 187, 185, 37, 208, 122, 92, 186, 186, 248, 248, 216, 69,
67, 111, 152, 216, 248, 248, 208, 208, 21, 137, 138, 152, 152, 248, 248,
248, 21, 9, 137, 138, 138, 136, 136, 248, 21, 13, 9, 138, 136, 136,
136, 136, 165, 13, 13, 9, 130, 136, 136, 136, 165, 13, 13, 13, 28,
136, 136, 136, 46, 46, 46, 28, 28, 28, 28, 141, 219, 108, 124, 50,
235, 235, 235, 235, 153, 209, 134, 213, 220, 220, 220, 216, 49, 3, 117,
117, 147, 147, 220, 216, 25, 3, 161, 161, 174, 174, 210, 216, 181, 45,
133, 161, 161, 174, 210, 93, 165, 5, 20, 133, 161, 161, 174, 93, 165,
5, 20, 133, 161, 161, 116, 93, 165, 37, 37, 37, 20, 133, 133, 129,
38, 38, 70, 70, 124, 50, 50, 235, 48, 217, 85, 167, 80, 213, 213,
24, 48, 60, 195, 195, 214, 214, 147, 252, 187, 63, 97, 227, 241, 246,
212, 210, 109, 199, 97, 227, 227, 78, 143, 206, 109, 15, 43, 81, 227,
227, 192, 206, 175, 169, 41, 81, 81, 81, 81, 78, 173, 169, 8, 33,
163, 81, 81, 81, 37, 1, 191, 70, 70, 124, 124, 69, 188, 38, 217,
166, 167, 167, 80, 29, 48, 112, 249, 201, 127, 64, 214, 255, 48, 142,
113, 72, 17, 207, 246, 212, 187, 185, 63, 66, 227, 197, 192, 212, 187,
84, 63, 72, 227, 227, 227, 212, 109, 15, 10, 102, 148, 227, 227, 78,
175, 53, 41, 32, 32, 81, 62, 81, 110, 104, 70, 191, 189, 70, 70,
69, 29, 159, 158, 89, 70, 166, 167, 80, 213, 38, 217, 249, 85, 85,
223, 80, 151, 48, 112, 201, 162, 178, 246, 212, 93, 112, 142, 36, 72,
17, 197, 207, 48, 187, 58, 72, 66, 66, 242, 197, 187, 187, 10, 105,
79, 66, 148, 197, 176, 11, 14, 18, 79, 79, 148, 148, 50, 226, 254,
110, 110, 50, 170, 89, 3, 61, 165, 89, 110, 166, 70, 89, 228, 37,
208, 89, 166, 166, 166, 110, 73, 179, 56, 249, 249, 85, 56, 56, 5,
179, 112, 249, 201, 205, 205, 178, 20, 187, 112, 142, 36, 205, 205, 135,
110, 187, 185, 113, 36, 72, 72, 135, 176, 168, 61, 59, 54, 79, 79,
68
}

```

```

Pc_Wiener_Sub_Classify[ 4 ][ NUM_PC_WIENER_LUT_CLASSES ] =
{
{
7, 54, 38, 45, 57, 51, 11, 57, 52, 51, 16, 52, 60, 6, 45, 32, 63, 27,
29, 23, 22, 23, 55, 57, 58, 32, 25, 23, 22, 31, 35, 57, 9, 22, 16, 23,
41, 53, 44, 7, 53, 9, 9, 45, 23, 13, 52, 52, 8, 7, 31, 23, 50, 32,
41, 23, 10, 16, 21, 41, 42, 21, 29, 20, 35, 23, 34, 23, 43, 30, 18, 63,
27, 53, 36, 33, 32, 25, 15, 17, 4, 29, 49, 44, 7, 18, 2, 14, 1, 10,
61, 33, 37, 12, 61, 11, 6, 39, 39, 51, 24, 20, 20, 50, 17, 27, 5, 41,
11, 7, 10, 3, 8, 42, 56, 25, 15, 63, 63, 25, 41, 42, 54, 36, 4, 4,

```

```

11, 18, 23, 15, 3, 22, 11, 22, 63, 46, 3, 3, 3, 54, 15, 9, 42, 15,
33, 53, 62, 37, 28, 60, 29, 33, 3, 8, 11, 63, 47, 18, 10, 4, 57, 15,
26, 22, 12, 51, 18, 4, 21, 52, 47, 21, 57, 51, 3, 16, 21, 0, 14, 19,
50, 39, 37, 25, 59, 8, 58, 8, 33, 60, 49, 33, 15, 62, 55, 20, 14, 28,
55, 52, 13, 26, 40, 8, 56, 27, 55, 2, 47, 13, 37, 54, 37, 37, 43, 35,
58, 0, 30, 0, 58, 62, 11, 11, 43, 34, 49, 34, 28, 14, 28, 48, 43, 27,
38, 58, 46, 10, 1, 30, 42, 56, 27, 36, 58, 11, 50, 34, 6, 26, 35, 23,
58, 31, 4, 30
}, {
21, 62, 54, 46, 23, 8, 0, 7, 27, 60, 46, 18, 31, 60, 18, 18, 56, 34,
3, 29, 60, 8, 4, 23, 44, 45, 59, 29, 60, 44, 58, 29, 61, 27, 46, 12,
34, 27, 28, 45, 61, 27, 61, 46, 12, 12, 27, 27, 16, 45, 44, 12, 33, 45,
3, 12, 52, 46, 37, 31, 21, 37, 61, 57, 24, 12, 20, 12, 62, 44, 63, 56,
34, 57, 51, 49, 18, 21, 48, 10, 24, 30, 2, 5, 45, 63, 17, 58, 13, 25,
41, 38, 36, 22, 49, 1, 39, 57, 57, 59, 20, 50, 57, 55, 10, 10, 39, 31,
1, 43, 25, 40, 16, 47, 56, 59, 6, 33, 56, 59, 47, 21, 19, 49, 53, 24,
49, 63, 29, 61, 9, 60, 1, 60, 17, 11, 9, 40, 40, 9, 30, 6, 16, 4,
23, 46, 54, 22, 26, 43, 39, 41, 9, 21, 0, 51, 3, 63, 25, 24, 7, 48,
32, 61, 2, 8, 63, 63, 37, 27, 31, 37, 7, 27, 48, 45, 43, 28, 11, 42,
58, 45, 36, 59, 54, 16, 15, 16, 38, 43, 6, 34, 26, 2, 9, 35, 4, 20,
4, 18, 14, 32, 50, 54, 27, 34, 22, 58, 58, 35, 22, 5, 58, 36, 58, 24,
15, 28, 7, 28, 22, 35, 1, 63, 13, 14, 30, 26, 2, 50, 4, 11, 54, 34,
42, 15, 11, 52, 52, 52, 47, 35, 34, 28, 22, 0, 58, 14, 15, 32, 17, 49,
15, 53, 24, 44
}, {
59, 6, 47, 15, 6, 36, 31, 58, 36, 40, 59, 59, 49, 40, 59, 59, 10, 1,
63, 11, 18, 61, 9, 58, 51, 2, 11, 54, 40, 32, 57, 6, 50, 50, 41, 11,
45, 36, 28, 59, 63, 50, 9, 41, 11, 36, 36, 50, 42, 58, 52, 11, 27, 2,
14, 11, 19, 41, 46, 46, 23, 46, 3, 53, 57, 11, 16, 5, 25, 52, 28, 10,
1, 63, 43, 22, 41, 53, 13, 4, 31, 3, 56, 43, 58, 19, 10, 17, 48, 35,
43, 29, 0, 32, 22, 31, 9, 27, 63, 41, 16, 16, 63, 27, 49, 48, 4, 14,
28, 58, 19, 8, 42, 45, 10, 11, 3, 15, 10, 41, 45, 23, 24, 21, 35, 57,
22, 57, 54, 26, 26, 26, 31, 18, 31, 7, 44, 5, 44, 21, 56, 30, 42, 17,
37, 41, 9, 51, 56, 49, 16, 6, 8, 54, 57, 15, 49, 7, 33, 31, 11, 20,
55, 50, 9, 60, 28, 31, 62, 36, 49, 46, 58, 60, 20, 2, 12, 23, 7, 47,
17, 60, 51, 41, 48, 62, 0, 37, 29, 14, 56, 24, 17, 9, 32, 10, 17, 17,
17, 59, 27, 55, 16, 34, 24, 55, 39, 57, 57, 15, 39, 43, 39, 51, 57, 57,
51, 34, 5, 38, 51, 54, 31, 31, 48, 17, 56, 13, 9, 17, 9, 33, 48, 4,
47, 52, 48, 33, 33, 33, 63, 10, 1, 24, 32, 57, 57, 17, 51, 34, 57, 21,
51, 33, 31, 32
}, {
33, 38, 54, 36, 18, 39, 6, 59, 39, 34, 25, 50, 50, 20, 1, 39, 44, 15,
1, 58, 39, 60, 22, 32, 17, 11, 59, 32, 21, 49, 7, 32, 1, 1, 33, 59,
15, 39, 24, 59, 1, 1, 1, 50, 59, 39, 0, 1, 45, 12, 27, 59, 44, 39,
50, 59, 5, 33, 25, 25, 36, 25, 1, 25, 7, 59, 61, 38, 63, 49, 14, 44,
2, 33, 16, 40, 33, 33, 48, 15, 17, 57, 22, 16, 25, 6, 44, 41, 55, 5,
16, 16, 42, 53, 51, 7, 47, 13, 41, 59, 22, 41, 1, 44, 55, 55, 47, 50,
51, 11, 5, 8, 29, 25, 0, 59, 57, 37, 44, 33, 22, 33, 40, 26, 52, 7,
51, 7, 59, 48, 28, 57, 28, 21, 9, 4, 62, 8, 62, 46, 22, 37, 58, 48,
35, 33, 47, 31, 61, 50, 47, 18, 62, 12, 7, 59, 47, 41, 5, 17, 59, 21,
10, 1, 22, 60, 14, 6, 15, 39, 50, 50, 32, 32, 48, 19, 30, 33, 10, 54,
44, 11, 7, 33, 47, 25, 23, 19, 35, 33, 57, 51, 57, 22, 48, 10, 22, 61,
22, 50, 44, 10, 41, 35, 12, 2, 48, 10, 53, 9, 48, 16, 48, 31, 31, 7,
31, 46, 18, 3, 31, 3, 7, 5, 55, 41, 22, 13, 22, 41, 22, 22, 54, 55,
54, 27, 55, 55, 54, 55, 33, 57, 55, 16, 48, 28, 7, 41, 31, 56, 7, 43,
48, 54, 17, 7
}
}

```

```

Pc_Wiener_Filters[ 4 ][ NUM_PC_WIENER_FILTERS ][ PC_WIENER_COEFFS ] =
{
{
{ 73, 127, -20, -30, -38, -29, 10, 7,
-1, -3, 1, 7, -208 },
{ -1, 3, 1, -1, -2, -1, 0, 0,
1, 0, 0, 0, 0 },
{ 12, 14, -5, -6, -6, 2, 1, -1,
2, -1, 1, 1, -28 },
{ 43, 23, -7, -6, -12, -7, -2, -3,
3, 2, 2, 1, -74 },

```

```

{ 5, 12, -1, -2, -4, 19, 1, -5,
  -1, -7, 1, 2, -40 },
{ 8, 8, -3, -3, -2, -4, 0, 2,
  0, 0, 0, 1, -14 },
{ 5, 7, -2, -3, -2, -3, 1, 1,
  0, 0, 0, 1, -10 },
{ 16, 16, -7, -6, 11, -6, -3, 1,
  -3, 2, 2, 1, -48 },
{ 18, 50, -3, -14, 3, -18, -2, 5,
  -4, 1, 0, 4, -80 },
{ 16, 4, -6, -1, -1, -6, 0, 1,
  0, 2, 1, 0, -20 },
{ -7, 11, 7, -3, -2, 11, 0, -5,
  0, -3, -1, 1, -18 },
{ 21, 26, -6, -8, -10, 7, 1, -4,
  1, -3, 2, 2, -58 },
{ 5, 7, -2, -4, -3, -1, 1, 0,
  1, 0, 0, 1, -10 },
{ 14, 15, -6, -5, -4, -4, 1, 1,
  0, 0, 1, 1, -28 },
{ 9, 10, -4, -4, -6, 0, 1, 0,
  2, -1, 1, 1, -18 },
{ 25, 17, -6, -4, -12, -9, 1, 0,
  3, 2, 1, 0, -36 },
{ 13, 13, -4, -5, 3, -7, -2, 1,
  -1, 2, 1, 1, -30 },
{ 4, 7, -1, -3, -5, -4, 1, 1,
  2, 1, 0, 0, -6 },
{ 8, 33, 0, -13, -10, 1, 2, -2,
  2, -1, 0, 3, -46 },
{ -2, -1, 1, 0, 0, 0, 0, 0,
  0, 0, 0, 0, 4 },
{ 11, 11, -5, -5, -4, -5, 1, 1,
  1, 1, 1, 1, -18 },
{ -2, 17, 4, -3, 7, -6, -3, 2,
  -3, 0, -1, 1, -26 },
{ 30, 15, -7, 0, -5, -13, 0, 1,
  -1, 2, 1, 0, -46 },
{ 33, 26, -8, -4, 7, -12, -4, 0,
  -5, 0, 2, 2, -74 },
{ 8, 10, -4, -5, -4, -4, 1, 1,
  1, 1, 1, 1, -14 },
{ 23, 24, -8, -8, -1, -9, -1, 1,
  -1, 1, 2, 2, -50 },
{ 18, 39, -5, -10, -11, -9, 5, 3,
  -2, -1, -1, 3, -58 },
{ 7, 15, -3, -6, -5, -5, 2, 2,
  0, 0, 0, 2, -18 },
{ 12, 9, -5, -3, -8, -5, 2, 1,
  2, 1, 1, 0, -14 },
{ 11, 8, -4, -1, -5, -6, 1, 1,
  0, 1, 1, 0, -14 },
{ 3, 3, 0, 1, -2, 13, 0, -4,
  -1, -4, 1, 1, -22 },
{ 4, 3, 2, 2, -4, 26, 0, -6,
  1, -6, 0, 0, -44 },
{ 13, 16, -7, -5, 15, -3, -3, 2,
  -6, 1, 1, 1, -50 },
{ 32, 20, 1, 9, 16, 15, -7, -7,
  -7, -7, -2, -2, -122 },
{ 16, 18, -6, -7, -9, -9, 2, 2,
  2, 2, 1, 1, -26 },
{ 10, 11, -3, -3, -7, 8, 1, -3,
  2, -4, 1, 1, -28 },
{ 36, -1, -12, 4, 17, 19, -10, -11,
  -5, -6, 8, 1, -80 },
{ 36, 12, -12, -3, -11, 5, 1, -1,
  3, -3, 2, 1, -60 },
{ 1, 1, 0, -1, -2, -3, 0, 1,
  1, 1, 0, 0, 2 },
{ 11, 11, -5, -5, -1, -4, 0, 1,

```



```

    0, 1, 1, 1, -22 },
{ 5, 7, -3, -4, -3, -1, 1, 0,
  1, 0, 1, 1, -10 },
{ 8, 14, -3, -4, -4, -8, 1, 3,
  0, 1, 0, 1, -18 },
{ 28, 42, -9, -13, -13, -19, 3, 6,
  1, 2, 0, 3, -62 },
{ 1, 1, 0, -1, -3, -2, 1, 0,
  1, 1, 0, 0, 2 },
{ 17, 17, -5, -1, 2, 5, -1, -2,
  -3, -4, 1, 1, -54 },
{ 8, 12, -1, -4, 3, -5, -2, -1,
  -2, 2, 1, 1, -24 },
{ 4, 8, -2, -2, -5, -3, 2, 1,
  1, 0, 0, 0, -8 },
{ -3, 6, 2, -2, 1, -2, 0, 1,
  -1, 0, -1, 1, -4 },
{ 5, 6, -1, -2, -4, 4, 0, -2,
  1, -2, 1, 1, -14 },
{ 14, 12, -4, -5, -8, -7, 1, 0,
  3, 3, 1, 0, -20 },
{ 17, 18, -7, -8, -8, -3, 1, 0,
  2, 0, 2, 2, -32 },
{ 41, 18, -14, -3, -5, -14, 1, 1,
  -1, 4, 3, 0, -62 },
{ 17, 0, -4, 2, 15, -5, -5, 0,
  -5, 2, 1, 0, -36 },
{ 16, 14, -6, -5, -2, -8, 0, 2,
  -1, 1, 1, 2, -28 },
{ 39, 2, -10, 8, 17, 20, -10, -9,
  -7, -9, 6, 1, -96 },
{ 18, 7, -4, 1, -10, 2, 1, -2,
  2, -2, 1, 0, -28 },
{ 49, 64, -13, -20, -31, -27, 6, 5,
  6, 4, 1, 3, -94 },
{ 24, 22, -7, -5, 24, -6, -5, -2,
  -6, 0, 2, 2, -86 },
{ 28, 0, -9, 9, -10, 14, 2, -1,
  -1, -8, 2, 0, -52 },
{ 2, 2, -1, -1, -2, -1, 1, 0,
  1, 0, 0, 0, -2 },
{ -5, 11, 6, -3, 16, -3, -6, 1,
  -3, 0, -1, 1, -28 },
{ 41, 32, -9, 3, -6, 12, -1, -5,
  -6, -9, 2, 1, -110 },
{ 5, 4, -2, -1, -4, -4, 1, 1,
  1, 1, 0, 0, -4 },
{ 25, 25, -8, -8, -7, -5, 1, 0,
  0, 0, 1, 2, -52 },
},
{
{ 21, 23, -6, -7, -7, 6, 0, -4,
  0, -3, 2, 2, -54 },
{ 22, 23, -5, -5, -12, 15, 2, -6,
  1, -5, 1, 1, -64 },
{ 7, 6, -3, -2, -5, -4, 2, 1,
  1, 1, 0, 0, -8 },
{ 4, 7, -1, -3, -4, -5, 1, 1,
  1, 2, 0, 0, -6 },
{ 16, 5, -6, 0, -8, 0, 3, -1,
  1, -1, 1, 0, -20 },
{ 34, 5, -8, 8, 13, 17, -8, -8,
  -7, -9, 5, 1, -86 },
{ 24, 20, -3, -4, -11, -9, -1, -1,
  3, 2, 1, 0, -42 },
{ 24, 20, -8, -5, 19, -6, -4, -1,
  -5, 1, 2, 1, -76 },
{ 37, 5, -12, 3, 12, -7, 2, 1,
  -8, 1, 1, 1, -72 },
{ 32, 17, -3, -3, -9, -1, -4, -4,
  3, -1, 2, 1, -60 },

```

```

{ 3, 6, 0, -2, -4, -5, 1, 2,
  1, 1, -1, 0, -4 },
{ 4, 11, -1, -4, -6, -2, 2, 0,
  1, 0, 0, 1, -12 },
{ 33, 26, -8, -7, 5, -13, -4, 0,
  -3, 2, 2, 2, -70 },
{ 2, 3, -1, -1, -3, -2, 1, 1,
  1, 0, 0, 0, -2 },
{ 14, 14, -6, -5, -7, -4, 2, 1,
  1, 0, 1, 1, -24 },
{ 30, 2, -9, 9, -11, 14, 1, 1,
  0, -9, 2, 0, -60 },
{ 5, 28, 1, -5, 7, -10, -4, 5,
  -3, -1, -1, 2, -48 },
{ 17, 17, -7, -6, -7, 4, 1, -2,
  1, -3, 2, 2, -38 },
{ 13, 12, -6, -4, 11, -4, -3, 2,
  -4, 1, 1, 1, -40 },
{ 41, 0, -10, 5, 18, 23, -9, -10,
  -7, -8, 5, 2, -100 },
{ 10, 12, -3, -4, -7, -6, 2, 1,
  1, 1, 0, 1, -16 },
{ 31, 41, -11, -10, -9, -18, 3, 5,
  -1, 1, 1, 2, -70 },
{ 28, 6, -9, 2, -10, 6, 1, -2,
  2, -4, 2, 0, -44 },
{ 24, 22, -10, -7, 24, -2, -2, -4,
  -2, -1, 4, 2, -96 },
{ 11, 8, -2, -1, -7, 15, 1, -5,
  1, -5, 1, 1, -36 },
{ -8, 16, 7, -3, -4, 15, 0, -5,
  1, -5, -1, 1, -28 },
{ 16, 13, -5, -4, -8, -8, 1, 1,
  2, 2, 1, 0, -22 },
{ 29, 2, -10, 4, 5, -10, -2, 3,
  -3, 0, 2, 0, -40 },
{ 21, 13, -5, 10, 3, 9, -1, -4,
  -9, -10, 3, 1, -62 },
{ 30, 26, -5, -3, 13, -6, -5, -3,
  -5, -2, 1, 1, -84 },
{ 16, 11, -6, -2, -8, -6, 2, 1,
  1, 1, 1, 0, -22 },
{ 0, 11, 2, -3, 2, -5, -1, 2,
  -1, 0, -1, 1, -14 },
{ 19, 39, -7, -11, -12, -8, 5, 3,
  -1, -1, 0, 3, -58 },
{ 21, 23, -7, -8, -9, -5, 2, 0,
  1, 0, 1, 2, -42 },
{ 10, 16, -4, -4, -8, -7, 4, 3,
  0, 0, -1, 1, -20 },
{ 29, 33, -11, -10, -15, -12, 5, 3,
  1, 1, 1, 2, -54 },
{ 37, 16, -14, -4, -10, 11, 1, -2,
  2, -6, 3, 2, -72 },
{ 0, 18, 4, -4, 8, -8, -3, 2,
  -3, 0, -1, 2, -30 },
{ 32, 18, 4, 11, 18, 18, -8, -8,
  -8, -8, -3, -2, -128 },
{ 5, 6, -3, -3, -2, -3, 1, 2,
  0, 0, 0, 1, -8 },
{ 49, 25, -5, -9, -8, -8, -5, -5,
  4, 4, 2, 1, -90 },
{ 43, 16, -7, 13, 13, 18, -7, -8,
  -11, -13, 3, 1, -122 },
{ -2, -1, 1, 0, 0, 0, 0, 0,
  0, 0, 0, 0, 4 },
{ -2, 13, 5, -4, 19, -4, -6, 1,
  -3, 0, -1, 2, -40 },
{ 12, 1, -5, 4, -2, 25, 0, -5,
  0, -8, 2, 0, -48 },
{ 19, 17, -8, -4, 15, -7, -4, 2,

```

```

    -5, 1, 2, 1, -58 },
{ 16, 15, -7, -6, 2, -6, -1, 1,
  -2, 1, 2, 2, -34 },
{ 14, 21, -5, -5, -7, -10, 2, 4,
  0, 0, 0, 1, -30 },
{ 35, 22, -9, -7, -10, -10, -1, 0,
  3, 2, 2, 1, -56 },
{ 26, 26, -1, -3, -1, 10, -3, -7,
  -3, -5, 0, 1, -80 },
{ 10, 9, -5, -4, -4, -3, 1, 1,
  1, 0, 1, 1, -16 },
{ 25, 10, -4, 7, 7, 11, -5, -6,
  -7, -8, 3, 1, -68 },
{ -3, 7, 3, -2, -3, 4, 1, -1,
  0, -2, -1, 1, -8 },
{ 3, 12, 4, -2, -4, 31, 0, -10,
  1, -7, 0, 1, -58 },
{ 2, 2, 0, -1, -2, -2, 0, 0,
  1, 1, 0, 0, -2 },
{ 16, 19, -6, -9, -7, -4, 1, 0,
  2, 1, 1, 2, -32 },
{ 22, 25, -7, -8, -7, -5, 1, 0,
  0, 0, 1, 2, -48 },
{ 12, 14, -5, -6, -2, -6, 0, 1,
  0, 2, 1, 1, -24 },
{ 12, 13, -5, -5, -7, -3, 1, 1,
  2, 0, 1, 1, -22 },
{ 25, 22, -8, -7, 1, -11, -2, 1,
  -2, 2, 2, 2, -50 },
{ 33, 15, -7, -1, -2, -12, -2, -1,
  -1, 3, 2, 0, -54 },
{ 19, 8, -9, -1, -4, -7, 1, 2,
  0, 1, 2, 0, -24 },
{ -1, -2, 1, 1, -2, -1, 1, 0,
  1, 1, -1, -1, 6 },
{ 10, 32, 0, -12, -11, 3, 2, -3,
  2, -2, 0, 3, -48 },
},
{
{ 38, 18, -10, -5, -10, 20, -1, 2,
  4, -9, 1, 2, -100 },
{ 11, 15, -4, -5, -8, -7, 3, 2,
  1, 1, 0, 1, -20 },
{ 20, 9, -10, -3, 18, -4, -2, 1,
  -4, 1, 3, 1, -60 },
{ 18, 12, -7, -4, -7, -8, 1, 2,
  2, 2, 1, 0, -24 },
{ 10, 15, -4, -6, -6, -9, 2, 3,
  1, 2, 0, 1, -18 },
{ 61, 30, -8, -10, 5, -13, -8, -6,
  0, 5, 4, 2, -124 },
{ 26, 27, -5, -5, 22, -4, -3, -4,
  -2, -3, 1, 1, -102 },
{ 8, 19, -3, -8, -8, -3, 3, 1,
  1, 0, 0, 2, -24 },
{ 38, 16, 3, -3, -3, 10, -9, -7,
  3, -5, 1, 2, -92 },
{ 8, 7, -3, -2, -5, -5, 1, 1,
  1, 1, 1, 0, -10 },
{ 22, 25, -8, -8, -9, -5, 2, 1,
  1, -1, 1, 2, -46 },
{ 28, 24, -8, -6, 6, -10, -3, 0,
  -4, 1, 2, 2, -64 },
{ 5, 7, 2, -1, 24, -7, -3, 3,
  1, -4, 0, 4, -62 },
{ 30, 20, -11, -8, -11, -9, 1, 0,
  3, 3, 3, 1, -44 },
{ 2, 13, 1, -5, 0, -5, -1, 1,
  0, 1, 0, 1, -16 },
{ 25, 21, -8, -5, -1, -1, -1, -1,
  -2, -2, 2, 2, -58 },

```

```

{ 8, 8, -3, -5, -3, -4, 0, 0,
  1, 2, 1, 1, -12 },
{ 15, 16, -6, -6, -8, -5, 2, 1,
  1, 0, 1, 2, -26 },
{ 33, 14, -7, 0, -1, -12, -2, 0,
  -2, 2, 2, 0, -54 },
{ -2, 16, 3, -3, -4, 13, 1, -4,
  -1, -5, 0, 2, -32 },
{ 31, 21, -6, -9, -5, -6, -3, -2,
  2, 2, 2, 2, -58 },
{ 24, 22, 5, 4, 4, 5, -5, -7,
  -4, -7, -1, 0, -80 },
{ 23, 22, 1, 1, -5, 19, -3, -7,
  -4, -6, 0, 1, -84 },
{ 76, 103, -23, -32, -40, -48, 9, 12,
  5, 7, 1, 6, -152 },
{ 29, 2, -2, 8, 15, 18, -8, -7,
  -5, -7, 1, -1, -86 },
{ 9, 10, -4, -5, -8, -7, 3, 2,
  2, 2, 0, 1, -10 },
{ 33, 19, -5, -3, -9, -9, -2, -2,
  2, 2, 2, 0, -56 },
{ 19, 19, -7, -7, -7, -5, 1, 1,
  1, 0, 1, 2, -36 },
{ 8, 33, 4, -8, -10, 21, 1, -11,
  0, -2, 1, 2, -78 },
{ 27, 14, 3, 10, 16, 15, -6, -5,
  -5, -6, -4, -3, -112 },
{ 37, 33, 2, 3, -15, -18, -1, 1,
  -3, 0, 1, 0, -80 },
{ 18, 20, -4, -4, -10, 16, 0, -6,
  2, -6, 2, 1, -58 },
{ 21, -1, -5, 5, -8, 12, 0, -5,
  0, -4, 3, 0, -36 },
{ -3, 8, 3, -3, -2, 4, 0, -2,
  0, -1, 0, 1, -10 },
{ 21, 38, -7, -2, -5, -3, 4, 1,
  -8, -5, 1, 4, -78 },
{ 2, 13, 6, -6, -4, 32, -1, -10,
  2, -2, 0, 1, -66 },
{ 31, 5, -10, 3, 9, -9, -1, 2,
  -5, 1, 1, 0, -54 },
{ 7, 17, 3, -2, 19, -6, -10, 0,
  3, -1, 2, 0, -64 },
{ 53, 80, -16, -4, -13, -2, 4, 0,
  -10, -11, 3, 5, -178 },
{ 25, 11, -9, -2, -9, -1, 1, 0,
  2, -2, 2, 1, -38 },
{ 40, 9, 3, 7, 7, -10, -4, -3,
  -7, 1, -1, 0, -84 },
{ 22, 20, -8, -7, 0, -9, -1, 1,
  -2, 2, 2, 2, -44 },
{ 16, 32, -4, -1, 3, -10, -3, 5,
  -3, -5, 1, 3, -68 },
{ 36, 7, -8, 10, 12, 17, -6, -8,
  -9, -10, 4, 1, -92 },
{ 38, 23, 5, -6, -7, 1, -10, -7,
  6, -1, 2, 1, -90 },
{ 15, 21, -6, -5, -8, -10, 3, 4,
  0, 0, 0, 1, -30 },
{ 1, 18, 2, -4, 6, -8, -2, 3,
  -3, 0, -1, 2, -28 },
{ -2, -2, 1, 1, 1, 0, 0, 0,
  -1, 0, 0, 0, 4 },
{ -1, 3, 0, -2, -1, -1, 0, 1,
  1, 0, 0, 0, 0 },
{ 0, 3, 0, -2, 0, -2, 0, 1,
  0, 0, 0, 1, -2 },
{ 24, 7, -10, 0, -1, -9, 0, 3,
  -1, 1, 2, 0, -32 },
{ 35, 7, -13, 2, -9, 13, 1, -2,

```

```

    1, -7, 3, 1, -64 },
{ 8, 5, -2, -5, -3, 39, 0, -5,
  3, -4, 1, 1, -76 },
{ 16, 12, -8, -2, 0, -7, 0, 4,
  -1, -1, 1, 1, -30 },
{ 31, 31, -10, -4, 8, -9, -3, 2,
  -5, -3, 2, 2, -84 },
{ 18, 31, -7, -8, -10, -8, 5, 3,
  -2, -1, 0, 3, -48 },
{ 16, 13, -5, -4, -8, -9, 1, 1,
  2, 3, 1, 0, -22 },
{ 15, 15, -6, -6, -6, 7, 1, -3,
  1, -3, 2, 2, -38 },
{ 22, 21, -9, -7, 17, -6, -4, 0,
  -3, 1, 3, 1, -72 },
{ 15, 13, -7, -3, 12, -5, -3, 1,
  -5, 1, 2, 1, -44 },
{ 24, 10, -13, -3, 22, -5, 5, 3,
  -5, 0, 2, 2, -84 },
{ 36, 10, -5, 0, 15, -5, 6, 0,
  -10, 1, -2, 2, -96 },
{ 5, 14, 2, -2, 11, -7, -5, 2,
  -2, -1, 0, 2, -38 },
{ 14, 14, -7, -6, -4, -6, 1, 1,
  0, 2, 2, 1, -24 },
},
{
{ 21, 15, -11, 0, -6, -5, 5, 3,
  -5, -3, 2, 3, -38 },
{ 18, 12, -7, -4, 0, -10, 0, 3,
  -1, 2, 1, 1, -30 },
{ 16, 18, -7, -5, -10, -7, 4, 3,
  0, -1, 1, 2, -28 },
{ 23, 29, 2, -5, 5, 10, -4, -7,
  -3, -5, -2, 9, -104 },
{ 10, 17, -6, -7, -9, -4, 4, 2,
  1, 0, 1, 2, -22 },
{ -3, 18, 2, -6, -3, 13, 0, -3,
  0, -4, 1, 3, -36 },
{ 13, 23, -2, -5, -9, 6, 2, -4,
  0, -1, 1, 1, -50 },
{ 18, 17, -8, -8, -6, 5, 2, -2,
  1, -1, 2, 2, -44 },
{ 30, 13, 6, 4, 6, 14, -10, -14,
  -4, -8, 11, 2, -100 },
{ 23, 18, -9, -4, -5, 1, 1, 0,
  0, -2, 2, 1, -52 },
{ 15, 18, -6, -4, -8, -2, 3, 1,
  -1, -2, 1, 2, -34 },
{ 20, 14, -10, -8, 20, -7, 0, 2,
  2, 2, 3, 1, -78 },
{ 21, 16, -6, -2, 3, -1, -1, 0,
  2, -4, 1, 2, -62 },
{ 19, 17, -6, -7, -8, -9, 1, 1,
  2, 3, 1, 1, -30 },
{ 2, 21, 5, -6, -4, 12, 1, -6,
  -6, 10, 0, 5, -68 },
{ 10, 13, -5, -3, -3, -7, 2, 4,
  -2, -1, 0, 2, -20 },
{ 36, 8, -6, 10, 13, 17, -7, -8,
  -9, -10, 3, 1, -96 },
{ 13, 9, -9, -2, -3, 24, 1, -3,
  1, -7, 3, 1, -56 },
{ 21, 23, -1, -2, 22, -1, -1, -4,
  2, -2, -1, -2, -108 },
{ 11, 17, -3, -4, 17, -9, -2, 3,
  0, -5, 1, 7, -66 },
{ 39, 6, 1, 13, 9, -11, 3, -3,
  -12, -2, 1, 1, -90 },
{ 28, 17, -5, -4, -5, -9, -2, -1,
  1, 3, 2, 0, -50 },

```

```

{ 60, 1, -47, 11, -7, 38, 16, 18,
-16, -22, 9, 0, -122 },
{ 23, 6, -3, -5, 2, 11, -4, 23,
3, -7, -3, 3, -98 },
{ 10, 26, 3, -5, 5, 11, -7, -8,
-12, 5, 4, 18, -100 },
{ 11, 13, -3, -2, 1, -8, -1, 2,
-1, 0, 1, 1, -28 },
{ 19, 21, 14, 13, 19, 14, -10, -9,
-10, -11, -3, -2, -110 },
{ 14, 6, -10, -9, -2, 29, 0, 8,
5, 2, 1, 0, -88 },
{ 20, 17, 4, -3, -6, 5, -6, -8,
0, 0, 3, 1, -54 },
{ 18, 27, -2, -3, 4, -3, -4, -1,
-6, -8, 3, 10, -70 },
{ 6, 5, 1, -3, 24, -6, -1, 4,
3, -8, 0, 8, -66 },
{ 31, 9, -12, 1, -9, 12, 0, -1,
2, -7, 3, 1, -60 },
{ 23, 18, -8, -1, 16, -6, -3, 1,
-4, -3, 1, 2, -72 },
{ -1, 8, -8, -9, -2, 12, 23, -3,
-9, 28, -13, 16, -84 },
{ 38, 12, -3, 6, 8, -7, -2, -8,
-10, 5, 7, -2, -88 },
{ 9, 4, 7, 6, 14, 10, 5, 6,
6, 3, -13, -9, -96 },
{ 20, 17, -7, -2, 1, -9, -1, 2,
-3, 0, 2, 1, -42 },
{ 25, 22, -2, -7, -8, -3, -4, -3,
4, 0, 2, 1, -54 },
{ 34, 11, 5, 7, 15, 6, -6, -7,
-7, -13, 6, 3, -108 },
{ 29, 9, -9, -1, 11, -12, -3, 3,
-3, 1, 2, 1, -56 },
{ 14, 0, 9, 6, 9, 10, 2, 0,
1, -1, -10, -6, -68 },
{ -26, 24, -15, 12, -20, 25, -12, 38,
28, 4, 0, -15, -86 },
{ 30, 6, -2, 4, 1, 15, -7, 7,
-2, -4, 1, 0, -98 },
{ 20, 19, 5, 5, 5, 10, -6, -8,
-5, -8, 1, 0, -76 },
{ 20, 21, -7, -8, -8, -3, 1, 0,
2, -1, 1, 2, -40 },
{ 16, 25, 2, -4, 7, 1, -7, -7,
3, -9, 4, 9, -80 },
{ 19, 26, -7, -8, -1, 1, -1, -1,
-6, -1, 3, 10, -68 },
{ 5, 3, -1, -1, -3, -4, 0, 1,
1, 1, 0, 0, -4 },
{ 27, 12, -11, -4, -8, -2, 1, 1,
2, -1, 3, 1, -42 },
{ 2, 0, 0, -3, 0, 35, -4, -7,
3, 0, 3, 1, -60 },
{ 10, 16, -4, -5, 0, -8, 1, 3,
-1, 0, 0, 2, -28 },
{ 19, 17, 0, 1, 0, 13, -4, -4,
-6, -1, 1, 1, -74 },
{ 18, 7, -10, -9, 0, 24, 0, -1,
1, 10, 3, -1, -84 },
{ 23, 16, -4, 2, -12, -3, -1, 4,
3, -8, 1, 1, -44 },
{ 12, 23, -1, 5, 13, 24, -19, 9,
-8, -24, -1, 14, -94 },
{ 14, 15, -7, -6, -6, -6, 1, 2,
1, 0, 2, 2, -24 },
{ 15, 22, -5, -2, -1, -1, 0, 2,
-7, -6, 2, 7, -52 },
{ 19, 14, -8, -5, -6, -9, 1, 3,

```

```

    1, 2, 1, 1, -28 },
{ 14, 19, -4, -1, 1, -6, -2, 4,
  -3, -5, 1, 3, -42 },
{ 17, 17, -1, 2, -3, 4, 0, -4,
  -5, -4, 2, 0, -50 },
{ 30, 5, -6, -5, 17, -5, 10, 0,
  -4, 1, 2, 3, -96 },
{ 15, 12, -7, -5, -5, -6, 1, 1,
  0, 1, 2, 2, -22 },
{ 34, 11, 5, 2, -3, 12, -11, -9,
  3, -7, 6, 1, -88 },
{ 8, 6, -4, 0, -5, -3, 3, 2,
  -1, -2, 0, 1, -10 },
}
}

```

```

Dip_Weights[ 6 ][ 64 ][ 11 ] =
{
{
{ -248, 690, 53, -26, 19, 413, 133, 3, 3, -13, -1 },
{ -179, 495, 574, -132, 64, -36, 195, 55, 15, -24, -2 },
{ -16, -35, 838, 100, 6, -47, 36, 100, 52, -8, -2 },
{ 13, -109, 383, 702, -98, 5, -51, 66, 83, 30, 1 },
{ -17, 30, -96, 878, 162, 8, -46, 16, 78, 6, 4 },
{ -14, 50, -125, 385, 754, -3, -17, -22, 67, -51, -1 },
{ -3, 13, 0, -97, 1060, -1, -5, -36, 51, 40, 4 },
{ 0, -2, 41, -206, 838, 5, -6, -33, 37, 343, 6 },
{ -202, 259, 227, -57, 25, 218, 542, -5, 36, -16, -6 },
{ -91, 131, 538, 10, 25, -80, 251, 197, 62, -21, 0 },
{ -21, -38, 455, 371, -6, -53, -22, 194, 138, -1, 3 },
{ -18, 8, 53, 702, 103, -10, -89, 69, 180, 16, 9 },
{ -20, 60, -132, 507, 522, -2, -56, -29, 159, 2, 12 },
{ -8, 39, -65, 62, 930, -2, -22, -67, 118, 29, 13 },
{ -1, 17, 11, -185, 998, 1, -10, -67, 82, 163, 16 },
{ 1, 8, 24, -193, 782, 11, -11, -47, 46, 381, 21 },
{ -39, 26, 245, 3, 3, -98, 634, 223, 43, -16, 1 },
{ -35, -23, 357, 189, 0, -71, 104, 359, 153, -17, 9 },
{ -26, -8, 151, 491, 74, -28, -92, 177, 263, 3, 17 },
{ -25, 56, -91, 526, 357, -12, -83, -17, 271, 20, 24 },
{ -14, 62, -110, 188, 770, -4, -41, -95, 204, 38, 26 },
{ -5, 37, -33, -116, 994, 4, -20, -102, 137, 104, 26 },
{ -2, 22, 10, -218, 954, 11, -17, -81, 87, 226, 30 },
{ -1, 12, 15, -181, 766, 25, -23, -44, 38, 379, 36 },
{ 7, -60, 172, 88, -14, -86, 230, 622, 62, -17, 22 },
{ -19, -66, 160, 305, 25, -7, -65, 355, 321, -12, 26 },
{ -24, 14, -35, 471, 215, -9, -86, 26, 397, 13, 44 },
{ -21, 64, -131, 303, 590, -10, -42, -127, 315, 35, 49 },
{ -9, 51, -76, -24, 922, -1, -15, -141, 199, 77, 44 },
{ -4, 34, -15, -203, 1018, 11, -16, -112, 119, 152, 41 },
{ -3, 23, 10, -224, 930, 23, -23, -76, 65, 251, 44 },
{ -1, 13, 15, -171, 762, 33, -27, -37, 21, 365, 49 },
{ -10, -80, 84, 149, -10, 42, -117, 658, 302, -11, 18 },
{ -13, -61, 1, 341, 95, 32, -93, 126, 542, 3, 54 },
{ -20, 26, -132, 361, 387, -3, -26, -140, 459, 30, 83 },
{ -13, 47, -120, 106, 774, -9, 6, -183, 283, 56, 78 },
{ -5, 32, -42, -144, 1002, 5, 4, -142, 150, 103, 63 },
{ -2, 22, 3, -235, 1018, 19, -9, -99, 75, 177, 58 },
{ 1, 15, 18, -216, 910, 28, -18, -67, 36, 261, 57 },
{ 4, 6, 22, -160, 758, 38, -27, -32, 7, 353, 56 },
{ -7, -73, 7, 171, 23, 58, -110, 222, 722, 1, 7 },
{ -4, -50, -99, 303, 205, 37, -28, -119, 646, 22, 116 },
{ -11, 18, -165, 221, 558, 5, 18, -199, 401, 44, 136 },
{ -8, 26, -98, -26, 890, -2, 27, -164, 197, 72, 108 },
{ -1, 14, -18, -196, 1028, 10, 13, -108, 78, 125, 84 },
{ -1, 7, 18, -232, 994, 25, -5, -72, 24, 194, 73 },
{ 1, 3, 27, -197, 878, 36, -17, -45, 2, 269, 67 },
{ 6, -2, 27, -146, 742, 42, -24, -21, -12, 346, 63 },
{ 5, -57, -57, 163, 84, 30, 8, -161, 890, 17, 102 },
{ 4, -49, -151, 218, 342, 42, 23, -203, 546, 39, 216 },
{ -4, 1, -164, 88, 694, 23, 30, -162, 269, 60, 191 },
}
}
}

```

```

{ -2, 7, -78, -102, 938, 12, 23, -100, 93, 95, 139 },
{ 1, -4, -4, -204, 1002, 19, 9, -59, 10, 148, 105 },
{ 2, -7, 29, -210, 946, 31, -7, -37, -25, 209, 90 },
{ 4, -5, 34, -173, 842, 38, -19, -17, -34, 275, 80 },
{ 9, -5, 29, -130, 722, 45, -28, 2, -37, 341, 72 },
{ 8, -47, -98, 128, 174, 28, 41, -203, 614, 26, 355 },
{ 12, -56, -167, 118, 469, 54, 30, -146, 321, 55, 335 },
{ 7, -27, -148, -13, 762, 45, 24, -85, 122, 84, 249 },
{ 2, -13, -66, -136, 926, 34, 10, -39, 10, 126, 172 },
{ 3, -17, 3, -184, 940, 34, -6, -15, -40, 178, 127 },
{ 6, -17, 33, -177, 879, 34, -12, -8, -55, 231, 108 },
{ 9, -10, 34, -146, 790, 38, -21, 5, -59, 283, 96 },
{ 11, -4, 27, -111, 690, 45, -29, 19, -58, 342, 86 },
},
{
{ -225, 513, 29, -9, 3, 730, -27, 21, -3, -5, -4 },
{ -261, 485, 426, -119, 37, 377, 81, 13, 3, -12, -5 },
{ -102, 10, 877, -101, 36, 189, 99, 31, 0, -12, -4 },
{ -6, -195, 697, 365, -66, 96, 79, 51, -4, 10, -2 },
{ -31, -30, 105, 838, -15, 36, 56, 68, -10, 3, 2 },
{ -45, 69, -128, 633, 451, 5, 31, 78, -11, -62, 4 },
{ -22, 17, -8, 74, 921, 0, 7, 79, -5, -45, 7 },
{ -10, -21, 76, -168, 845, 0, -5, 73, 8, 221, 5 },
{ -255, 123, 116, -25, 12, 722, 387, -64, 24, -6, -8 },
{ -199, 87, 374, -80, 37, 457, 339, 22, 6, -11, -6 },
{ -107, -84, 602, -33, 40, 257, 270, 94, -6, -9, -2 },
{ -53, -126, 473, 265, 12, 122, 197, 144, -11, 0, 2 },
{ -49, -13, 126, 526, 107, 33, 132, 172, -11, -6, 7 },
{ -47, 49, -22, 387, 433, -10, 73, 184, -6, -28, 11 },
{ -28, 24, 37, 43, 730, -23, 27, 178, 8, 16, 13 },
{ -15, -5, 84, -110, 685, -25, 5, 162, 24, 203, 16 },
{ -80, 14, 91, -12, 17, 142, 866, -20, 15, -8, -5 },
{ -94, -44, 245, -38, 40, 189, 606, 138, -11, -10, -1 },
{ -58, -132, 368, -3, 57, 151, 424, 238, -14, -8, 2 },
{ -33, -119, 293, 162, 66, 73, 293, 293, -9, -1, 5 },
{ -35, -18, 110, 291, 155, 2, 193, 315, 0, 0, 11 },
{ -34, 32, 41, 197, 367, -36, 112, 311, 17, 1, 16 },
{ -24, 17, 83, -5, 550, -49, 56, 285, 39, 49, 19 },
{ -13, -11, 114, -91, 526, -48, 22, 257, 55, 187, 26 },
{ 8, 7, 44, -2, 22, -121, 685, 459, -85, -9, 15 },
{ -29, -38, 120, -12, 47, -5, 503, 483, -44, -11, 10 },
{ -22, -103, 187, 3, 69, 39, 361, 501, -12, -7, 9 },
{ -12, -110, 166, 77, 90, 34, 250, 505, 14, -1, 10 },
{ -15, -49, 86, 135, 152, 0, 164, 491, 36, 7, 15 },
{ -16, -1, 65, 81, 274, -33, 98, 457, 61, 18, 20 },
{ -14, 3, 104, -33, 381, -52, 50, 407, 87, 64, 26 },
{ -8, -18, 136, -85, 373, -56, 18, 357, 100, 168, 37 },
{ -10, 6, 15, 1, 26, -19, 113, 942, -55, -9, 16 },
{ -20, -13, 39, -2, 50, -2, 145, 796, 33, -12, 8 },
{ -14, -55, 73, 2, 70, 19, 129, 726, 72, -8, 12 },
{ -5, -86, 88, 22, 89, 30, 96, 682, 95, -2, 16 },
{ -6, -74, 72, 42, 118, 29, 58, 638, 114, 10, 20 },
{ -9, -42, 72, 20, 171, 11, 25, 586, 135, 30, 25 },
{ -9, -20, 101, -40, 228, -16, 1, 518, 152, 71, 34 },
{ -4, -24, 134, -76, 239, -38, -15, 450, 161, 150, 45 },
{ -22, 4, 1, 1, 26, 55, -146, 790, 367, -7, -40 },
{ -21, -2, 1, 0, 45, 41, -88, 742, 325, -10, -10 },
{ -12, -30, 18, -3, 61, 39, -62, 722, 289, -8, 11 },
{ -4, -70, 50, -7, 72, 47, -52, 702, 265, -2, 22 },
{ -1, -87, 67, -6, 77, 57, -51, 670, 255, 12, 29 },
{ -4, -76, 75, -9, 82, 55, -52, 626, 252, 38, 35 },
{ -7, -47, 90, -35, 106, 32, -53, 566, 250, 74, 45 },
{ -4, -28, 114, -63, 129, -2, -54, 495, 245, 135, 57 },
{ -8, -2, -2, -4, 22, 28, -75, 254, 858, -7, -39 },
{ -11, -5, -9, -5, 37, 45, -115, 425, 654, -8, 17 },
{ -8, -27, 2, -6, 44, 54, -130, 526, 530, -5, 43 },
{ -2, -62, 35, -15, 45, 61, -126, 574, 458, 1, 53 },
{ 3, -88, 64, -21, 32, 71, -115, 586, 416, 18, 56 },
{ 3, -87, 75, -19, 19, 71, -99, 570, 385, 43, 60 },
{ 1, -66, 79, -25, 22, 56, -86, 534, 359, 77, 69 },
{ 2, -41, 93, -47, 46, 24, -77, 482, 331, 125, 82 },

```



```

{ -2, -7, 1, -10, 21, 8, -10, -28, 870, -7, 186 },
{ -4, -14, -4, -11, 30, 34, -85, 191, 723, -6, 167 },
{ -2, -25, -2, -7, 27, 47, -122, 337, 618, -1, 153 },
{ 2, -53, 26, -15, 19, 59, -135, 427, 546, 9, 140 },
{ 8, -81, 61, -27, 3, 65, -126, 469, 497, 25, 129 },
{ 13, -90, 75, -24, -18, 63, -105, 479, 457, 49, 124 },
{ 12, -75, 71, -16, -27, 53, -90, 471, 417, 80, 126 },
{ 13, -57, 79, -31, -7, 33, -82, 444, 376, 119, 133 },
},
{
{ -253, 974, -92, 21, -5, 392, -10, 2, -5, 1, 0 },
{ -281, 934, 442, -152, 51, -8, 57, -13, -3, -7, 4 },
{ -54, 119, 1092, -154, 44, -35, 33, -8, -9, -7, 4 },
{ 43, -234, 834, 479, -145, 22, 2, 1, -10, 31, 2 },
{ -11, -26, 51, 1092, -121, 15, 2, 5, -7, 24, -1 },
{ -31, 82, -216, 794, 479, -2, 6, 5, -8, -88, 1 },
{ -7, 16, -37, 68, 1068, -4, 6, 5, -9, -80, 2 },
{ 1, -11, 61, -230, 962, -9, 11, 0, -9, 247, 1 },
{ -325, 890, -46, 4, 0, 270, 291, -71, 11, 0, 3 },
{ -263, 878, 473, -157, 52, -109, 187, -36, -4, -7, 9 },
{ -54, 135, 1060, -126, 35, -89, 80, -11, -16, -3, 11 },
{ 32, -193, 786, 503, -140, -8, 22, 2, -17, 31, 6 },
{ -14, -14, 45, 1060, -78, 1, 13, 7, -16, 18, 2 },
{ -30, 82, -203, 750, 509, -13, 17, 4, -13, -80, 2 },
{ -6, 20, -41, 68, 1044, -15, 16, 3, -15, -56, 4 },
{ 1, -5, 52, -218, 958, -19, 20, 0, -17, 247, 4 },
{ -207, 814, -7, -11, 4, -197, 696, -79, 5, 0, 6 },
{ -205, 814, 507, -162, 53, -301, 341, -13, -20, -6, 16 },
{ -37, 138, 1044, -98, 25, -169, 126, 9, -26, 1, 16 },
{ 33, -157, 738, 526, -135, -50, 39, 9, -23, 32, 10 },
{ -8, -1, 36, 1036, -45, -26, 26, 4, -18, 16, 5 },
{ -24, 82, -195, 714, 530, -30, 29, -1, -17, -72, 5 },
{ -4, 24, -43, 62, 1036, -27, 28, -4, -17, -38, 7 },
{ -1, 0, 47, -212, 954, -26, 30, -2, -23, 250, 9 },
{ -141, 734, 32, -26, 9, -413, 630, 263, -81, 0, 19 },
{ -164, 750, 534, -162, 52, -419, 333, 138, -57, -5, 24 },
{ -21, 139, 1014, -64, 13, -238, 131, 64, -41, 5, 23 },
{ 35, -125, 694, 554, -132, -91, 48, 21, -25, 34, 14 },
{ 0, 9, 29, 1010, -11, -53, 38, -1, -15, 14, 8 },
{ -15, 83, -186, 678, 554, -49, 39, -9, -16, -63, 9 },
{ 1, 29, -45, 54, 1028, -46, 44, -13, -21, -21, 12 },
{ 1, 2, 44, -209, 950, -40, 44, -8, -29, 255, 15 },
{ -159, 658, 67, -38, 11, -329, 184, 702, -93, 2, 22 },
{ -153, 682, 554, -157, 48, -421, 179, 311, -44, -3, 25 },
{ -18, 145, 974, -23, 0, -271, 103, 105, -22, 10, 24 },
{ 35, -94, 646, 574, -126, -126, 55, 22, -16, 35, 17 },
{ 7, 17, 25, 974, 25, -77, 48, -8, -12, 12, 11 },
{ -7, 82, -176, 642, 570, -66, 50, -15, -17, -53, 13 },
{ 5, 33, -45, 48, 1018, -63, 55, -13, -27, -4, 17 },
{ 5, 5, 41, -204, 938, -61, 58, -7, -35, 261, 20 },
{ -168, 579, 97, -47, 12, -259, -30, 626, 232, 3, -20 },
{ -152, 614, 566, -146, 44, -404, 87, 293, 107, -1, 16 },
{ -19, 152, 926, 17, -10, -295, 85, 91, 38, 13, 26 },
{ 34, -64, 602, 586, -112, -162, 70, 5, 8, 35, 21 },
{ 11, 23, 29, 934, 62, -101, 63, -15, -10, 12, 18 },
{ -3, 80, -161, 606, 587, -84, 62, -14, -23, -44, 19 },
{ 7, 36, -43, 44, 1002, -78, 61, -6, -34, 13, 23 },
{ 6, 8, 40, -196, 926, -73, 59, 0, -41, 268, 25 },
{ -151, 503, 123, -53, 12, -283, 61, 134, 706, 4, -32 },
{ -149, 542, 566, -129, 37, -412, 117, 92, 334, 3, 25 },
{ -25, 156, 870, 53, -14, -315, 94, 27, 118, 17, 40 },
{ 31, -34, 561, 582, -88, -191, 76, -8, 27, 36, 35 },
{ 15, 29, 38, 878, 103, -124, 71, -16, -9, 12, 28 },
{ -2, 76, -142, 566, 598, -99, 69, -7, -32, -32, 28 },
{ 6, 38, -39, 46, 978, -93, 65, 5, -45, 32, 30 },
{ 5, 12, 37, -180, 902, -84, 60, 8, -47, 276, 33 },
{ -142, 431, 146, -55, 12, -289, 141, -154, 794, 5, 137 },
{ -149, 467, 546, -107, 31, -413, 154, -62, 435, 8, 113 },
{ -35, 151, 806, 81, -11, -326, 110, -30, 161, 23, 95 },
{ 26, -11, 523, 550, -52, -208, 74, -18, 26, 38, 74 },
{ 19, 34, 56, 805, 143, -145, 67, -4, -27, 18, 58 },

```

```

    { 3, 74, -117, 526, 594, -119, 67, 4, -45, -14, 48 },
    { 7, 41, -33, 54, 938, -108, 66, 11, -55, 54, 48 },
    { 6, 19, 34, -157, 866, -99, 64, 10, -55, 286, 49 },
},
{
    { -252, 542, 11, 43, -3, 622, 69, 0, 1, -6, -4 },
    { -287, 389, 351, 59, 5, 341, 187, -6, 2, -12, -4 },
    { -152, -144, 686, 183, 1, 248, 207, 12, -4, -11, -2 },
    { -73, -417, 535, 578, -42, 214, 204, 26, -6, 6, -1 },
    { -84, -381, 105, 894, 63, 198, 194, 37, -5, 4, -1 },
    { -87, -334, -95, 658, 518, 167, 188, 43, 2, -35, -2 },
    { -72, -331, -39, 68, 1010, 138, 177, 47, 7, 15, 2 },
    { -63, -294, -7, -227, 964, 108, 162, 46, 6, 319, 13 },
    { -277, 160, 72, 84, -7, 530, 530, -68, 14, -7, -4 },
    { -248, -10, 242, 216, -12, 301, 560, -11, -1, -12, 0 },
    { -160, -373, 369, 413, -4, 218, 546, 36, -15, -9, 5 },
    { -104, -622, 286, 650, 45, 191, 522, 67, -20, 0, 9 },
    { -97, -686, 75, 710, 263, 177, 499, 86, -13, 0, 8 },
    { -84, -686, -39, 407, 694, 145, 479, 99, 1, 4, 6 },
    { -76, -642, -71, -2, 1006, 109, 449, 107, 10, 120, 12 },
    { -72, -566, -110, -182, 938, 79, 405, 107, 10, 383, 31 },
    { -117, -13, 57, 123, -6, 38, 934, 15, -4, -7, 3 },
    { -132, -225, 147, 294, 0, 25, 818, 121, -23, -11, 11 },
    { -86, -538, 195, 483, 43, 17, 763, 166, -29, -9, 19 },
    { -53, -766, 126, 618, 163, 17, 738, 185, -24, 0, 23 },
    { -44, -858, -7, 530, 453, 16, 714, 196, -9, 11, 23 },
    { -32, -882, -88, 214, 834, -1, 682, 207, 4, 57, 26 },
    { -31, -830, -148, -80, 1044, -20, 634, 212, 14, 193, 37 },
    { -40, -738, -209, -193, 974, -33, 570, 209, 16, 412, 59 },
    { -34, -94, 37, 137, 4, -148, 711, 483, -88, -7, 24 },
    { -62, -323, 82, 303, 34, -103, 648, 479, -52, -9, 28 },
    { -37, -602, 88, 451, 118, -90, 639, 450, -25, -6, 37 },
    { -13, -814, 14, 503, 303, -87, 651, 425, -5, 5, 44 },
    { 1, -910, -102, 331, 638, -95, 654, 411, 9, 31, 52 },
    { 8, -928, -189, 50, 962, -106, 638, 407, 21, 102, 59 },
    { 3, -890, -257, -160, 1100, -110, 592, 397, 33, 241, 73 },
    { -12, -814, -315, -216, 1022, -111, 534, 375, 46, 425, 91 },
    { -34, -130, 20, 128, 22, -56, 169, 914, -24, -7, 21 },
    { -40, -343, 25, 269, 81, -81, 265, 766, 54, -6, 33 },
    { -20, -578, -4, 362, 212, -96, 336, 670, 88, 2, 54 },
    { 5, -774, -77, 313, 471, -107, 389, 614, 100, 16, 73 },
    { 21, -870, -184, 105, 822, -119, 415, 586, 108, 52, 85 },
    { 24, -889, -285, -112, 1076, -127, 415, 566, 116, 142, 96 },
    { 16, -858, -364, -233, 1156, -130, 392, 534, 130, 275, 109 },
    { 3, -810, -413, -235, 1060, -132, 365, 483, 154, 433, 121 },
    { -32, -134, 0, 109, 43, -8, -72, 743, 398, -5, -18 },
    { -29, -314, -29, 205, 137, -59, 48, 634, 401, 0, 32 },
    { -14, -505, -85, 224, 323, -85, 129, 562, 385, 13, 76 },
    { 12, -674, -155, 81, 650, -104, 184, 526, 376, 29, 100 },
    { 27, -770, -252, -126, 978, -113, 215, 505, 369, 81, 113 },
    { 26, -802, -357, -261, 1163, -120, 226, 481, 373, 177, 121 },
    { 22, -797, -439, -299, 1180, -125, 225, 444, 379, 303, 132 },
    { 12, -775, -471, -261, 1068, -128, 221, 389, 391, 438, 141 },
    { -19, -116, -26, 79, 68, -33, 1, 150, 918, 0, 3 },
    { -18, -269, -75, 117, 203, -66, 48, 174, 806, 11, 91 },
    { -7, -413, -147, 46, 453, -93, 97, 171, 754, 27, 135 },
    { 12, -551, -221, -150, 806, -109, 134, 168, 734, 52, 150 },
    { 26, -651, -309, -319, 1076, -115, 153, 166, 726, 118, 152 },
    { 28, -706, -403, -381, 1196, -118, 160, 160, 718, 216, 156 },
    { 24, -726, -475, -355, 1172, -121, 161, 143, 710, 329, 159 },
    { 15, -728, -495, -291, 1060, -125, 162, 122, 694, 442, 165 },
    { -11, -103, -50, 43, 95, -50, 84, -225, 982, 8, 251 },
    { -10, -228, -118, 16, 279, -73, 96, -174, 930, 26, 279 },
    { 0, -339, -199, -129, 574, -98, 121, -156, 927, 51, 272 },
    { 14, -453, -270, -327, 892, -115, 143, -148, 939, 93, 255 },
    { 27, -558, -345, -440, 1100, -123, 158, -142, 947, 168, 235 },
    { 33, -632, -421, -447, 1171, -128, 165, -135, 940, 259, 226 },
    { 27, -670, -477, -391, 1132, -126, 158, -123, 921, 355, 220 },
    { 16, -689, -489, -316, 1036, -127, 151, -103, 880, 448, 220 },
},
{

```

```

{ -84, 730, -84, 11, -3, 522, -64, 3, -3, 0, -2 },
{ -218, 951, 202, -66, 14, 153, 4, -12, 2, -6, -4 },
{ -94, 407, 795, -120, 21, 24, 3, -1, 0, -9, -3 },
{ 38, -64, 910, 210, -71, 1, -10, 9, -4, 9, -3 },
{ 18, -47, 389, 786, -133, 2, -11, 14, -7, 16, -2 },
{ -19, 72, -45, 917, 142, -8, -5, 14, -8, -36, -1 },
{ -16, 58, -45, 395, 713, -9, 1, 10, -9, -74, 1 },
{ 6, 6, 83, -123, 938, -20, 12, 3, -10, 129, 2 },
{ -146, 431, -45, -2, 2, 702, 137, -55, 5, -2, -4 },
{ -146, 802, 78, -32, 4, 300, 53, -27, 0, -4, -4 },
{ -111, 622, 519, -87, 9, 77, 9, -5, -1, -7, -3 },
{ -13, 173, 842, 60, -37, 21, -27, 11, -7, -1, -1 },
{ 16, -4, 627, 500, -110, 10, -28, 13, -12, 9, 1 },
{ -10, 42, 208, 817, -16, 4, -15, 10, -14, -4, 2 },
{ -18, 72, 20, 626, 363, -4, -3, 5, -17, -24, 5 },
{ -3, 41, 58, 157, 714, -9, 5, 0, -21, 75, 7 },
{ -75, 240, -30, -6, 5, 421, 570, -99, 6, -3, -7 },
{ -100, 582, 26, -24, 5, 379, 200, -30, -8, -4, -3 },
{ -103, 662, 323, -69, 5, 178, 42, -4, -6, -5, 0 },
{ -44, 368, 702, -17, -22, 65, -23, 3, -7, -3, 4 },
{ 0, 104, 730, 285, -91, 28, -37, 2, -12, 9, 7 },
{ -3, 45, 435, 646, -92, 18, -25, -8, -14, 14, 7 },
{ -11, 62, 166, 698, 116, 10, -10, -12, -19, 13, 10 },
{ -6, 59, 79, 411, 439, 3, 1, -14, -27, 64, 15 },
{ 4, 131, -26, -8, 8, 105, 746, 121, -56, -3, 3 },
{ -60, 385, 0, -22, 6, 316, 411, 16, -26, -2, -2 },
{ -99, 598, 187, -53, -1, 252, 157, -10, -9, -1, 2 },
{ -65, 477, 543, -55, -15, 122, 38, -24, -1, -1, 6 },
{ -12, 216, 738, 132, -75, 49, -8, -30, -7, 9, 12 },
{ 4, 82, 593, 467, -121, 25, -16, -35, -13, 23, 15 },
{ 1, 63, 323, 662, -24, 12, -6, -35, -21, 32, 18 },
{ 2, 59, 160, 550, 220, 4, 7, -34, -31, 63, 22 },
{ 7, 72, -23, -8, 8, 40, 475, 554, -106, -3, 7 },
{ -28, 248, -20, -18, 3, 188, 526, 170, -43, 1, -1 },
{ -81, 481, 94, -39, -6, 232, 341, 7, -12, 3, 3 },
{ -84, 501, 395, -64, -17, 162, 166, -48, -1, 3, 11 },
{ -35, 305, 670, 36, -60, 83, 60, -59, -6, 10, 19 },
{ 3, 134, 666, 311, -118, 36, 16, -61, -15, 26, 23 },
{ 15, 73, 461, 558, -86, 10, 10, -56, -27, 39, 27 },
{ 13, 62, 267, 582, 77, -1, 14, -46, -38, 63, 30 },
{ -8, 40, -19, -9, 4, 69, 150, 766, 59, -1, -25 },
{ -16, 158, -30, -14, -4, 110, 451, 388, -19, 4, -4 },
{ -57, 361, 36, -28, -13, 168, 454, 106, -17, 7, 7 },
{ -82, 459, 271, -61, -19, 156, 308, -24, -10, 7, 16 },
{ -56, 351, 566, -18, -48, 105, 160, -60, -16, 12, 25 },
{ -12, 190, 670, 185, -101, 54, 69, -64, -26, 27, 31 },
{ 13, 99, 550, 433, -103, 19, 28, -56, -40, 41, 36 },
{ 22, 75, 375, 534, 0, -5, 15, -45, -48, 62, 38 },
{ -8, 29, -16, -11, -2, 59, 43, 522, 457, 1, -53 },
{ -12, 104, -31, -13, -11, 81, 287, 520, 88, 5, 1 },
{ -35, 247, 9, -22, -19, 120, 427, 273, -9, 10, 21 },
{ -61, 377, 176, -47, -25, 124, 387, 78, -27, 12, 28 },
{ -63, 358, 443, -37, -41, 96, 261, -12, -33, 16, 34 },
{ -36, 238, 614, 96, -80, 61, 141, -36, -44, 27, 39 },
{ -5, 139, 586, 313, -97, 28, 62, -32, -58, 41, 45 },
{ 14, 102, 455, 452, -40, 0, 17, -25, -61, 61, 46 },
{ 3, 30, -16, -10, -9, 22, 71, 127, 726, 3, 76 },
{ 1, 72, -28, -8, -21, 61, 158, 463, 257, 9, 57 },
{ -8, 158, -2, -13, -27, 89, 321, 401, 30, 14, 59 },
{ -30, 279, 109, -30, -31, 94, 375, 217, -34, 16, 55 },
{ -54, 334, 323, -37, -37, 74, 309, 87, -52, 19, 54 },
{ -50, 270, 526, 37, -61, 46, 205, 27, -64, 28, 55 },
{ -29, 179, 577, 211, -88, 17, 116, 8, -71, 42, 57 },
{ -4, 129, 501, 365, -64, -7, 51, -3, -71, 63, 57 },
},
{
{ -199, 662, 105, 76, 15, 269, 68, 9, 31, -4, -4 },
{ -105, 332, 602, 60, 47, 2, 39, 14, 48, -7, -6 },
{ -16, -11, 802, 148, 49, 23, -13, 3, 50, -4, -6 },
{ 4, -71, 574, 455, 8, 29, -22, -6, 52, 9, -7 },
{ -13, -8, 214, 730, 57, 30, -25, -5, 54, 0, -9 },

```

```

{ -16, 4, 84, 617, 339, 26, -19, -3, 54, -53, -9 },
{ -11, -12, 126, 301, 621, 25, -12, -5, 51, -54, -8 },
{ -9, -10, 148, 125, 562, 21, 0, -13, 47, 160, -6 },
{ -160, 301, 291, 132, 32, 158, 227, -1, 60, -4, -12 },
{ -54, 109, 566, 227, 55, 10, 20, 32, 77, -7, -12 },
{ -16, -25, 593, 325, 74, 36, -34, 5, 86, -7, -12 },
{ -12, -34, 472, 437, 102, 41, -42, -9, 89, -8, -12 },
{ -18, -9, 325, 518, 164, 41, -42, -10, 87, -18, -12 },
{ -18, -10, 256, 502, 257, 38, -32, -15, 87, -29, -12 },
{ -17, -16, 240, 449, 291, 39, -21, -19, 87, 4, -10 },
{ -17, -3, 210, 412, 220, 34, -2, -22, 75, 121, -6 },
{ -57, 147, 341, 178, 48, -27, 297, 47, 66, -6, -12 },
{ -29, 36, 522, 301, 75, 15, -17, 46, 95, -9, -11 },
{ -13, -32, 518, 372, 106, 38, -54, 5, 106, -12, -11 },
{ -13, -38, 455, 417, 142, 46, -53, -13, 108, -16, -11 },
{ -18, -21, 372, 466, 165, 48, -49, -19, 106, -17, -10 },
{ -20, -12, 317, 501, 168, 44, -37, -28, 106, -8, -8 },
{ -19, -13, 297, 505, 141, 43, -25, -31, 101, 29, -5 },
{ -20, 2, 256, 501, 78, 42, -5, -31, 90, 113, -1 },
{ -22, 85, 343, 203, 65, -56, 171, 203, 44, -8, -4 },
{ -20, 10, 499, 325, 97, 26, -50, 43, 113, -11, -8 },
{ -8, -40, 505, 379, 119, 40, -64, -6, 117, -13, -5 },
{ -9, -54, 472, 407, 143, 47, -55, -23, 117, -17, -4 },
{ -15, -34, 397, 467, 137, 49, -44, -35, 118, -11, -3 },
{ -18, -16, 331, 530, 105, 42, -29, -45, 117, 6, -1 },
{ -17, -11, 308, 542, 71, 42, -19, -45, 110, 42, 2 },
{ -20, 4, 275, 523, 32, 44, -4, -42, 99, 106, 5 },
{ -34, 55, 337, 212, 77, 7, -1, 311, 72, -9, -6 },
{ -20, 0, 487, 337, 102, 37, -46, 7, 136, -11, -4 },
{ -11, -42, 514, 378, 113, 46, -58, -29, 123, -12, 3 },
{ -8, -65, 498, 401, 127, 49, -51, -35, 119, -15, 4 },
{ -12, -50, 421, 477, 104, 46, -31, -52, 123, -5, 3 },
{ -17, -23, 335, 565, 57, 40, -15, -62, 122, 17, 5 },
{ -18, -5, 293, 582, 31, 39, -6, -62, 116, 49, 7 },
{ -21, 17, 271, 534, 16, 41, 5, -54, 108, 102, 9 },
{ -34, 35, 329, 220, 78, 23, -29, 204, 237, -7, -32 },
{ -22, -1, 481, 343, 93, 39, -19, -36, 150, -9, 4 },
{ -15, -31, 514, 377, 99, 45, -39, -51, 118, -10, 16 },
{ -8, -58, 514, 399, 103, 43, -37, -48, 113, -11, 16 },
{ -10, -53, 441, 483, 70, 38, -17, -60, 117, 1, 13 },
{ -14, -28, 341, 582, 24, 36, -1, -71, 120, 24, 12 },
{ -18, 2, 279, 606, 3, 36, 6, -70, 119, 53, 12 },
{ -19, 26, 253, 550, 2, 33, 16, -62, 113, 99, 13 },
{ -27, 25, 323, 221, 68, 10, 6, 47, 384, -5, -27 },
{ -25, 8, 470, 345, 81, 39, -2, -37, 124, -7, 28 },
{ -18, -13, 505, 379, 82, 41, -22, -50, 95, -7, 34 },
{ -10, -40, 509, 403, 77, 34, -21, -52, 100, -6, 29 },
{ -7, -47, 451, 483, 42, 30, -8, -58, 106, 8, 24 },
{ -12, -23, 350, 578, 4, 30, 6, -65, 107, 29, 22 },
{ -18, 11, 271, 606, -11, 32, 13, -65, 107, 58, 21 },
{ -18, 30, 241, 554, -3, 33, 19, -57, 105, 98, 22 },
{ -27, 28, 293, 218, 53, 5, 3, -1, 362, -2, 94 },
{ -26, 20, 440, 343, 68, 31, -1, 2, 78, -3, 73 },
{ -21, 7, 479, 381, 63, 35, -8, -33, 62, -1, 61 },
{ -12, -15, 487, 409, 48, 28, -10, -40, 77, 5, 48 },
{ -6, -26, 449, 465, 23, 24, -4, -44, 84, 18, 41 },
{ -10, -12, 364, 542, -6, 28, 3, -49, 89, 37, 37 },
{ -16, 19, 281, 574, -13, 28, 14, -49, 91, 62, 34 },
{ -11, 30, 244, 537, -5, 27, 17, -41, 89, 99, 36 },
}
}

```

[↑ Back to Table of Contents](#)

§ Annex A: Profiles, levels, and tiers

§ A.1.General

This annex specifies profiles, levels, and tiers that collectively define the conformance requirements for AV2 bitstreams and decoders.

A [profile](#) specifies the allowed coding tools, chroma formats, and bit depths that a conforming [coded video sequence](#) or [coded multistream video sequence](#) shall satisfy. Further information is provided in [Annex A.2 Profiles](#).

A [level](#) and [tier](#) combination defines constraints on picture size, display rate, decoding rate, and bitrate that a conforming [coded video sequence](#) or [coded multistream video sequence](#) shall not exceed. Further information is provided in [Annex A.4 Levels](#).

§ A.2.Profiles

The AV2 profiles supported in this version of this specification are defined in [Table A.1](#). A profile specifies the allowed coding tools, chroma formats, bit depths, and interoperability point that a conforming [coded video sequence](#) or [coded multistream video sequence](#) shall satisfy. An interoperability point indicates the layering capabilities of the bitstream, and it is explicitly determined by the profile identifier for all profiles except the Configurable profile. The Configurable profile indicates that a bitstream does not conform to any of the other defined profiles, and additional information is needed to determine its constraints.

Decoders are required to support one or more profiles to claim conformance with the AV2 video coding standard.

NOTE: This version of this specification specifies one toolset, the Main toolset. This includes all coding tools defined in this specification. Future versions of this specification may define additional toolsets using the extensibility mechanisms of AV2.

A [coded video sequence](#) signals its profile via `seq_profile_idc` in the associated sequence header. A [coded multistream video sequence](#) may signal its aggregate profile via `multistream_profile_idc` in the MSDO OBU. Both use the same value space, as specified in [Table A.1](#).

Table A.1: AV2 profile definitions

Profile label	seq_profile_idc or multistream_profile_idc	chroma_format_idc	bit_depth_idc	Interoperability point
Main_420_10_IP0	0	CHROMA_FORMAT_400, CHROMA_FORMAT_420	0 or 1	0
Main_420_10_IP1	1	CHROMA_FORMAT_400, CHROMA_FORMAT_420	0 or 1	1
Main_420_10_IP2	2	CHROMA_FORMAT_400, CHROMA_FORMAT_420	0 or 1	2
Main_422_10_IP1	3	CHROMA_FORMAT_400, CHROMA_FORMAT_420, CHROMA_FORMAT_422	0 or 1	1

Main_444_10_IP1	4	CHROMA_FORMAT_400, CHROMA_FORMAT_420, CHROMA_FORMAT_444	0 or 1	1
Reserved	5-30			
Configurable	31	CHROMA_FORMAT_400, CHROMA_FORMAT_420, CHROMA_FORMAT_422, CHROMA_FORMAT_444	-	-

For example, if seq_profile_idc is equal to 3, the [coded video sequence](#) conforms to the "Main_422_10_IP1" profile at Interoperability Point 1, and may use chroma formats 4:0:0, 4:2:0, or 4:2:2 at 8 or 10 bit depth. Similarly, if multistream_profile_idc is equal to 3, the [coded multistream video sequence](#) conforms to the same profile and interoperability point.

For the Configurable profile, the constraints are determined from the chroma_format_idc, bit_depth_idc, and SeqMaxMlayerCnt syntax elements in the sequence header. Additionally, the multi-sequence configuration signaling described in [Annex A.3 Multi-sequence configurations](#) may be used to convey the aggregate constraints of a bitstream using the Configurable profile.

The variables ProfileScalingFactor, PicSizeProfileFactor, and BitrateProfileFactor are derived from the profile as defined in [Table A.2](#) and are used in the level and tier constraints specified in [Annex A.4 Levels](#) and [Annex E: Decoder model](#). For the Configurable profile, ProfileScalingFactor and the related variables need to be determined based on the characteristics of the chosen configuration.

Table A.2: Definition of ProfileScalingFactor, PicSizeProfileFactor, and BitrateProfileFactor

seq_profile_idc or multistream_profile_idc	ProfileScalingFactor	PicSizeProfileFactor	BitrateProfileFactor
0, 1, 2	0	15	1.0
3	1	20	1.667
4	2	30	2.5
31	-	-	-

Interoperability points are defined in [Table A.3](#). An interoperability point specifies the number of extended and embedded layers a decoder is capable of decoding simultaneously.

Table A.3: AV2 interoperability points

Interoperability Point	Number of Extended Layers	Number of Embedded Layers	Combination of Extended and Embedded Layers	Number of Layers
0	1-4	1	0	1-4
1	1-4	1-2	0	1-4
2	1-4	1-3	0 or 1	1-8
3-14	Reserved			
15 (max)	1-31	1-8	0 or 1	1-248

where the columns in the table are defined as follows:

- **Number of Extended Layers** denotes the number of singlestreams in a coded video sequence or coded multistream video sequence. For a coded video sequence, this value is equal to 1. For a coded multistream video sequence, when MultiStreamDecoderMode is equal to 1, this value is equal to num_streams_minus_2 plus 2. When a global layer configuration record is activated, this value is equal to LcrMaxNumXLayerCount. Otherwise, this value is equal to the number of distinct values of obu_xlayer_id (excluding GLOBAL_XLAYER_ID) present in the coded multistream video sequence.
- **Number of Embedded Layers** denotes the maximum value for seq_max_mlayer_cnt_minus_1 plus 1 for the coded video sequence or coded multistream video sequence.
- **Combination of Extended and Embedded Layers** denotes if a coded video sequence or coded multistream video sequence contains more than one extended layer and more than one embedded layer. This value is equal to 1 when Number of Extended Layers and Number of Embedded Layers are both greater than one. Otherwise, the value is equal to 0. For a coded video sequence, this value is equal to 0.
- **Number of Layers** denotes the sum of seq_max_mlayer_cnt_minus_1 plus 1 across all singlestreams in a coded multistream video sequence. For a coded video sequence, this value is equal to seq_max_mlayer_cnt_minus_1 plus 1.

NOTE: A coded multistream video sequence that contains two extended layers, where the first extended layer contains two embedded layers and the second extended layer contains three embedded layers, will have "Number of Extended Layers" equal to 2, "Number of Embedded Layers" equal to 3, "Combination of Extended and Embedded Layers" equal to 1, and "Number of Layers" equal to 5. A coded video sequence that contains two embedded layers will have "Number of Extended Layers" equal to 1, "Number of Embedded Layers" equal to 2, "Combination of Extended and Embedded Layers" equal to 0, and "Number of Layers" equal to 2.

For interoperability points 0 through 2, requirements on the presence of OBUs with obu_type equal to OBU_MSDO (MSDO) and obu_type equal to OBU_LAYER_CONFIGURATION_RECORD (LCR) are given in the [Table A.4](#). The OBU with obu_type equal to OBU_OPERATING_POINT_SET is optional in all of these cases.

Table A.4: OBU requirements for interoperability points

IOP	Number of Extended Layers > 1	Number of Embedded Layers > 1	MSDO	LCR
0	N	N/A	Prohibited	Optional
0	Y	N/A	Required	Optional
1	N	N	Prohibited	Optional
1	Y	N	Required	Optional
1	N	Y	Prohibited	Required (Local)
2	N	N	Prohibited	Optional
2	Y	N	One (or both) of (a) MSDO or (b) Global LCR is required	
2	N	Y	Prohibited	Required (Global or Local)
2	Y	Y	One (or both) of (a) MSDO plus Local LCR or (b) Global LCR is required	

§ A.3. Multi-sequence configurations

A **multi-sequence configuration** specifies the collective minimum requirements for coding tools, chroma formats, and bit depths needed to decode all [coded video sequences](#) within an AV2 bitstream. Multi-sequence configurations are particularly relevant for bitstreams using the Configurable profile (see [Annex A.2 Profiles](#)), where they provide a mechanism to convey the aggregate constraints that are not otherwise determined by the profile identifier.

This specification defines three multi-sequence configurations: "C_Main_420_10", "C_Main_422_10", and "C_Main_444_10", as listed in [Table A.5](#). A bitstream can explicitly identify its multi-sequence configuration through the `lcr_config_idc` syntax elements in a LCR OBU, if one is present. Alternatively, this information may be implicitly determined from syntax elements within the bitstream, such as the `chroma_format_idc` and `bit_depth_idc` of each individual [coded video sequence](#).

Table A.5: AV2 multi-sequence configurations

ConfigurationID	Multi-sequence configuration label	Toolset	BitDepth		Chroma Format			
			8	10	4:0:0	4:2:0	4:2:2	4:4:4
0	C_Main_420_10	Main	x	x	x	x		
1	C_Main_422_10	Main	x	x	x	x	x	
2	C_Main_444_10	Main	x	x	x	x		x
3-63	Reserved							

Table A.6: Allowed syntax element values for multi-sequence configurations

Multi-sequence configuration label	seq_profile_idc	chroma_format_idc	bit_depth_idc
C_Main_420_10	0..2, 31	CHROMA_FORMAT_400, CHROMA_FORMAT_420	0..1
C_Main_422_10	0..3, 31	CHROMA_FORMAT_400, CHROMA_FORMAT_420, CHROMA_FORMAT_422	0..1
C_Main_444_10	0..2, 4, 31	CHROMA_FORMAT_400, CHROMA_FORMAT_420, CHROMA_FORMAT_444	0..1

§ A.4. Levels

Each operating point contains a [syntax element](#) `seq_level_idx`.

The following table defines the mapping from the [syntax element](#) (which takes integer values) to the defined levels:

Table A.7: Values for level

Value of seq_level_idx	Level
0	2.0
1	2.1
2	3.0
3	3.1
4	4.0

5	4.1
6	5.0
7	5.1
8	5.2
9	5.3
10	6.0
11	6.1
12	6.2
13	6.3
14	7.0
15	7.1
16	7.2
17	7.3
18	8.0
19	8.1
20	8.2
21	8.3
22-30	Reserved
31	Maximum parameters

The [level](#) defines variables as specified in the following tables:

Table A.8: Values for level

LevelIdx	Level	MaxPicSize	MaxHSize/MaxVSize	MaxDisplayRate	MaxDecodeRate
		(Samples)	(Samples)	(Samples/sec)	(Samples/sec)
0	2.0	147456	640	4423680	5529600
1	2.1	278784	880	8363520	10454400
2	3.0	665856	1360	19975680	24969600
3	3.1	1065024	1720	31950720	39938400
4	4.0	2359296	2560	70778880	77856768
5	4.1	2359296	2560	141557760	155713536
6	5.0	8912896	4975	267386880	273715200
7	5.1	8912896	4975	534773760	547430400
8	5.2	8912896	4975	1069547520	1094860800
9	5.3	8912896	4975	1069547520	1176502272
10	6.0	35651584	9951	1069547520	1176502272
11	6.1	35651584	9951	2139095040	2189721600
12	6.2	35651584	9951	4278190080	4379443200
13	6.3	35651584	9951	4278190080	4706009088
14	7.0	142606336	19902	4278190080	4706009088
15	7.1	142606336	19902	8556380160	8758886400

16	7.2	142606336	19902	17112760320	17517772800
17	7.3	142606336	19902	17112760320	18824036352
18	8.0	530841600	38400	17112760320	18824036352
19	8.1	530841600	38400	34225520640	34910031052
20	8.2	530841600	38400	68451041280	69820062105
21	8.3	530841600	38400	68451041280	75296145408

Table A.9: Level bitrate and tile constraints

LevelIdx	Level	MaxHeaderRate	MainMbps	HighMbps	MainCR	HighCR	MaxTiles	MaxTileCols	Example
		(/sec)	(Mbits/sec)	(Mbits/sec)					
0	2.0	150	1.5	-	2	-	8	4	426x240@30fps
1	2.1	150	3.0	-	2	-	8	4	640x360@30fps
2	3.0	150	6.0	-	2	-	16	6	854x480@30fps
3	3.1	150	10.0	-	2	-	16	6	1280x720@30fps
4	4.0	300	12.0	30.0	4	4	32	8	1920x1080@30fps
5	4.1	300	20.0	50.0	4	4	32	8	1920x1080@60fps
6	5.0	300	30.0	100.0	6	4	64	8	3840x2160@30fps
7	5.1	300	40.0	160.0	8	4	64	8	3840x2160@60fps
8	5.2	300	60.0	240.0	8	4	64	8	3840x2160@120fps
9	5.3	300	60.0	240.0	8	4	64	8	3840x2160@120fps
10	6.0	300	60.0	240.0	8	4	128	16	7680x4320@30fps
11	6.1	300	100.0	480.0	8	4	128	16	7680x4320@60fps
12	6.2	300	160.0	800.0	8	4	128	16	7680x4320@120fps
13	6.3	300	160.0	800.0	8	4	128	16	7680x4320@120fps
14	7.0	960	160.0	800.0	8	4	256	32	15360x8640@30fps
15	7.1	960	200.0	960.0	8	4	256	32	15360x8640@60fps
16	7.2	960	320.0	1600.0	8	4	256	32	15360x8640@120fps
17	7.3	960	320.0	1600.0	8	4	256	32	15360x8640@120fps
18	8.0	960	320.0	1600.0	8	4	512	64	30720x17280@30fps
19	8.1	960	400.0	1920.0	8	4	512	64	30720x17280@60fps
20	8.2	960	640.0	3200.0	8	4	512	64	30720x17280@120fps
21	8.3	960	640.0	3200.0	8	4	512	64	30720x17280@120fps

NOTE: HighMbps and HighCR values are not defined for levels below [level](#) 4.0. seq_tier equal to 1 can only be signaled for [level](#) 4.0 and above.

Bitstream constraints shall be applied at the bitstream level and shall correspond to the tier ID seq_tier and level ID seq_level_idx signaled in the sequence_header_obu().

A bitstream may contain one or more operating points. It can also represent a sub-bitstream extracted from a source bitstream containing multiple operating points, based on the operating point indication. In

the latter case, the sub-bitstream may signal different values of the tier ID `seq_tier` and level ID `seq_level_idx` in the `sequence_header_obu()`, which may be derived from the corresponding `ops_tier_flag` and `ops_level_idx` values signaled in the `operating_point_set_obu()`. Bitstream constraints shall be applied to the sub-bitstream according to its own `seq_tier` and `seq_level_idx` values.

If `MultiStreamDecoderMode` is equal to 0, bitstream constraints shall be applied to each substream in the bitstream according to the `seq_tier` and `seq_level_idx` values associated with that substream.

Otherwise, if `MultiStreamDecoderMode` is equal to 1, the syntax elements `multistream_even_allocation_flag`, `multistream_large_picture_idc`, `multistream_level_idx`, `multistream_tier`, `num_streams_minus_2`, and `sub_xlayer_id[ i ]` refer to the values from the most recently parsed Multi Stream Decoder Operation OBU. The substream level variables `MaxPicSizeX`, `MaxMbpsX`, `MaxDisplayRateX`, `MaxDecodeRateX`, `MaxHeaderRateX`, `MaxTilesX`, `MaxTileColsX` and `MinCompBasisX` for the bitstream associated with `obu_xlayer_id` are derived by using the following ordered steps:

1. The variable `ScaleFactorX` is derived by:
 - If `multistream_even_allocation_flag` is equal to 1, `ScaleFactorX` is set to 4.
 - Otherwise, if `multistream_even_allocation_flag` is equal to 0 and the `obu_xlayer_id` value associated with the current subbitstream is equal to `sub_xlayer_id[ multistream_large_picture_idc ]`, then the `ScaleFactorX` for that subbitstream is set to 1.5.
 - Otherwise (`multistream_even_allocation_flag` is equal to 0 and the `obu_xlayer_id` value associated with the current subbitstream is not equal to `sub_xlayer_id[ multistream_large_picture_idc ]`), `ScaleFactorX` is set to 9.
2. Let `MaxPicSize`, `MaxDisplayRate` and `MaxDecodeRate`, `MaxHeaderRate`, `MainMbps`, `HighMbps`, `MainCR`, `HighCR`, `MaxTiles` and `MaxTileCols` be level variables in the table associated with `multistream_level_idx`. The values for the substream-level variables, `MaxVSizeX`, `MaxHSizeX`, `MaxTileColsX`, and `MaxHeaderRateX`, are determined by looking up the table below, using `MaxPicSize` and `ScaleFactorX`.

MaxPicSize	ScaleFactorX	MaxVSizeX	MaxHSizeX	MaxTileColsX	MaxHeaderRateX
2359296	1.5	1600	896	7	132
2359296	4	960	576	4	132
2359296	9	640	384	3	132
8912896	1.5	2560	1472	7	132
8912896	4	1920	1088	4	132
8912896	9	1280	768	3	132
35651584	1.5	5120	2280	13	132
35651584	4	3840	2176	8	132
35651584	9	2560	1472	5	132
142606336	1.5	10240	5760	26	132
142606336	4	7680	4320	16	132
142606336	9	5120	2880	11	132
530841600	1.5	20480	11520	52	132
530841600	4	15360	8640	32	132

530841600	9	10240	5760	21	132
-----------	---	-------	------	----	-----

1. The values for the remaining substream level variables MaxPicSizeX, MaxMbpsX, MaxDisplayRateX, MaxDecodeRateX, MaxTilesX, MaxTileColsX, and MinCompBasisX are set as follows:

- $\text{MaxPicSizeX} = \text{MaxVSizeX} * \text{MaxHSizeX}$
- $\text{MaxMbpsX} = \text{multistream_tier} == 0 ? (\text{MainMbps} / \text{ScaleFactorX}) : (\text{HighMbps} / \text{ScaleFactorX})$
- $\text{MaxDisplayRateX} = \text{MaxDisplayRate} / \text{ScaleFactorX}$
- $\text{MaxDecodeRateX} = \text{MaxDecodeRate} / \text{ScaleFactorX}$
- $\text{MaxTilesX} = \text{MaxTiles} / \text{ScaleFactorX}$
- $\text{MinCompBasisX} = \text{multistream_tier} == 0 ? \text{MainCR} : \text{HighCR}.$

Let MaxPicSize, MaxDisplayRate, MaxDecodeRate, MaxHeaderRate, MainMbps, HighMbps, MainCR, HighCR, MaxTiles and MaxTileCols be level variables in the table associated with seq_level_idx, the additional variables are derived as follows:

- TileWidth is defined as $(\text{MiColEnd} - \text{MiColStart}) * \text{MI_SIZE}$
- TileHeight is defined as $(\text{MiRowEnd} - \text{MiRowStart}) * \text{MI_SIZE}$
- RightMostTile is defined as $\text{MiColEnd} == \text{MiCols}$
- MaxTileSizeInLumaSamples is defined as the largest product of TileWidth * TileHeight for all tiles within the coded video [sequence](#)
- InloopFilteringEnabled for a particular Frame is set equal to 1 if $\text{apply_deblocking_filter}[0] \neq 0 \parallel \text{apply_deblocking_filter}[1] \neq 0 \parallel \text{cdef_frame_enable} \neq 0 \parallel \text{ccso_frame_flag} \neq 0 \parallel \text{ccso_planes}[0] \neq 0 \parallel \text{ccso_planes}[1] \neq 0 \parallel \text{ccso_planes}[2] \neq 0 \parallel \text{FrameRestorationType}[0] \neq \text{RESTORE_NONE} \parallel \text{FrameRestorationType}[1] \neq \text{RESTORE_NONE} \parallel \text{FrameRestorationType}[2] \neq \text{RESTORE_NONE} \parallel \text{gdf_frame_enable} \neq 0$. Otherwise, it is set equal to 0.
- DecodeCount for a particular Frame is set equal to 2 if both allow_global_intrabc is equal to 1 and InloopFilteringEnabled is equal to 1. Otherwise, it is set equal to 1.
- LumaSampleCount for a particular Frame is determined as follows:
 - If (FrameIsIntra) LumaSampleCount is set equal to $\text{FrameWidth} * \text{FrameHeight}$.
 - Otherwise, LumaSampleCount is set equal to $(\text{max_frame_width_minus_1} + 1) * (\text{max_frame_height_minus_1} + 1)$.
- The output time of a [temporal unit](#) is defined as the time indicated through either the timing information OBU, if present, or the timing information that may be indicated through external means. The output duration of a [temporal unit](#) is defined as the difference between the output time of the next [temporal unit](#) and the output time of the current [temporal unit](#) in display order. For the last [temporal unit](#) in the bitstream, the output duration from the previous [temporal unit](#) is used.
- TotalDisplayLumaSampleRate is defined as the sum of the LumaSampleCount of all frames with immediate_output_frame equal to 1 or implicit_output_frame equal to 1 or ShowExistingFrame equal to 1 that belong to the [temporal unit](#), divided by the output duration of the [temporal unit](#).
- FrameParsingTime for a Frame belonging to Decodable Frame Group (DFG) i and with ShowExistingFrame equal to 0 is defined as $(\text{Removal}[i+1] - \text{Removal}[i]) \div \text{DecodeCount}$ if Removal[i+1] is present. For the Frame belonging to the last DFG in the bitstream,

FrameParsingTime shall be set equal to that of the previous Frame with ShowExistingFrame equal to 0. The DFG is defined in Section [Annex E.3 Decoder model definitions](#), and the *i*th DFG removal time Removal[*i*] is defined in Section [Annex E.5.4 Removal times in decoding schedule mode](#).

- MaxNumFrameHeadersPerSec is set equal to $\text{MaxHeaderRate} * (1 + (\text{seq_tier} \ll 1))$
- NumFrameHeadersPerSec is defined as the number of OBUs received per second that contain a frame_header() and for which the variable CountFrameHeaderForLevelConstraint is equal to 1.
- CompressedSize is defined for each [frame](#) as the total bytes in the OBUs, with obu_type equal to OBU_CLOSED_LOOP_KEY, OBU_OPEN_LOOP_KEY, OBU_LEADING_TILE_GROUP, OBU_REGULAR_TILE_GROUP, OBU_METADATA_SHORT, OBU_METADATA_GROUP, OBU_SWITCH, OBU_LEADING_SEF, OBU_REGULAR_SEF, OBU_LEADING_TIP, OBU_REGULAR_TIP, OBU_BRIDGE_FRAME or OBU_RAS_FRAME, related to this [frame](#), minus 128 (to allow for overhead of metadata and header data).
- FrameSymbolCount is defined for each [frame](#) as the total number of symbols in the OBUs related to this Frame. It is initialized to 0 in the syntax table frame_header(), and accumulated for the frame in the parsing process as defined in read_literal(*n*) and read_symbol(cdf).
- If seq_tier is equal to 0, MaxMbps is set equal to MainMbps, otherwise MaxMbps is set equal to HighMbps.
- If seq_tier is equal to 0, MinCompBasis is set equal to MainCR, otherwise MinCompBasis is set equal to HighCR.

When MultiStreamDecoderMode is equal to 1, the level variables are adjusted as follows:

- $\text{MaxPicSize} = \text{Min}(\text{MaxPicSize}, \text{MaxPicSizeX})$
- $\text{MaxMbps} = \text{Min}(\text{MaxMbps}, \text{MaxMbpsX})$
- $\text{MaxDisplayRate} = \text{Min}(\text{MaxDisplayRate}, \text{MaxDisplayRateX})$
- $\text{MaxDecodeRate} = \text{Min}(\text{MaxDecodeRate}, \text{MaxDecodeRateX})$
- $\text{MaxVSize} = \text{Min}(\text{MaxVSize}, \text{MaxVSizeX})$
- $\text{MaxHSize} = \text{Min}(\text{MaxHSize}, \text{MaxHSizeX})$
- $\text{MaxHeaderRate} = \text{Min}(\text{MaxHeaderRate}, \text{MaxHeaderRateX})$
- $\text{MaxTiles} = \text{Min}(\text{MaxTiles}, \text{MaxTilesX})$
- $\text{MaxTileCols} = \text{Min}(\text{MaxTileCols}, \text{MaxTileColsX})$
- $\text{MinCompBasis} = \text{Max}(\text{MinCompBasis}, \text{MinCompBasisX})$.

The additional variable MaxLevelRefFrames is derived as follows:

- If the bitstream contains any frame with DecodeCount equal to 2 and satisfies one of the following conditions, MaxLevelRefFrames is set to $\text{Min}((8 * \text{MaxPicSize}) / ((\text{max_frame_width_minus_1} + 1) * (\text{max_frame_height_minus_1} + 1)) - 1, (8 \ll \text{explicit_num_ref_frames}))$:
 - max_mlayer_id is not equal to 0,
 - at least one of such frames is not coded using OBUs with obu_type equal to OBU_CLOSED_LOOP_KEY.
- Otherwise, MaxLevelRefFrames is set to $\text{Min}((8 * \text{MaxPicSize}) / ((\text{max_frame_width_minus_1} + 1) * (\text{max_frame_height_minus_1} + 1)), (8 \ll \text{explicit_num_ref_frames}))$.

NOTE: MaxLevelRefFrames in the case of DecodeCount equal to 2, e.g., a frame is encoded with both InloopFilteringEnabled and allow_global_intrabc equal to 1, is lowered by 1 to reserve memory space in a reference frame buffer that may be used for the reconstruction of the intermediate decoded frame associated with this coded frame and prior to the application of any loop filtering operations.

When the mapped [level](#) ID, LevelIdx is contained in the tables above, it is a requirement of bitstream conformance that the following constraints hold:

- FrameWidth * FrameHeight is less than or equal to MaxPicSize
- FrameWidth is less than or equal to MaxHSize
- FrameHeight is less than or equal to MaxVSize
- NumTiles is less than or equal to MaxTiles
- TileCols is less than or equal to MaxTileCols
- TileWidth is less than or equal to Tile_Width_Scaling_Factor[seq_tier][LevelIdx] * MAX_TILE_WIDTH / 4 for each [tile](#)
- For each [tile](#), if RightMostTile is equal to 0, then TileWidth is greater than or equal to 64
- TileWidth * TileHeight is less than or equal to Tile_Area_Scaling_Factor[seq_tier][LevelIdx] * 4096 * 2304 / 4 for each [tile](#)
- FrameWidth is greater than or equal to 16
- FrameHeight is greater than or equal to 16.

When the mapped [level](#) ID, LevelIdx is contained in the tables above, it is a requirement of video bitstream conformance (i.e., still_picture is equal to 0) that the following constraints hold:

- TotalDisplayLumaSampleRate is less than or equal to MaxDisplayRate
- NumFrameHeadersPerSec is less than or equal to MaxNumFrameHeadersPerSec
- NumRefFrames is less than or equal to MaxLevelRefFrames
- For a particular Frame with ShowExistingFrame equal to 0
 - LumaSampleCount is less than or equal to FrameParsingTime*MaxDecodeRate.
 - NumTiles is less than or equal to Min(MaxTiles, Max(1, MaxTiles * 120 * FrameParsingTime))
 - CompressedSize is less than or equal to Min((LumaSampleCount * PicSizeProfileFactor >> 3) * 1.25, (FrameParsingTime* MaxDecodeRate * PicSizeProfileFactor >> 3) ÷ MinCompBasis)
 - FrameSymbolCount is less than or equal to FrameParsingTime * MaxDecodeRate * PicSizeProfileFactor * (8 ÷ (9 * MinCompBasis) + 1 ÷ 48)
- MaxTileSizeInLumaSamples * NumFrameHeadersPerSec is less than or equal to (Tile_Area_Scaling_Factor[seq_tier][LevelIdx] * 547,430,400) / 4. (The number of 547,430,400 corresponds to the decode luma sample rate of 3840x2160 * 60fps * 1.1)

NOTE: The purpose of this constraint is to ensure that for decode luma sample rates above 4K60 there is sufficient parallelism for decoder implementations. Parallelism can be chosen by the [encoder](#) as either tile level parallelism or temporal layer parallelism or a combination provided the above constraint holds. The constraint has no effect on levels 5.1 and below.

If `seq_level_idx` is equal to 31 (indicating the maximum parameters [level](#)), then there are no [level](#)-based constraints on the [bitstream](#).

NOTE: The maximum parameters [level](#) should only be set for bitstreams that do not conform to any other [level](#). Typically this would be used for large resolution still images.

The buffer model is used to define additional conformance requirements.

These requirements depend on the following [level](#), [tier](#), and [profile](#) dependent variables:

- `MaxBitrate` is equal to `MaxMbps` multiplied by 1,000,000
- `MaxBufferSize` is equal to `MaxBitrate` multiplied by 1 second

§ A.5.Decoder Conformance

A [level](#) X.Y conformant [decoder](#) shall be capable of decoding all bitstreams (that can be decoded by the general [decoding process](#)) that conform to that [level](#).

In doing so, the [decoder](#) shall display output frames according to the display schedule, if indicated by the [bitstream](#).

NOTE: If the [level](#) of a [bitstream](#) is equal to 31 (indicating the maximum parameters [level](#)), the [decoder](#) should examine the properties of the [bitstream](#) in order to determine if it can be decoded.

[↑ Back to Table of Contents](#)

§ Annex B: Length delimited bitstream format

§ B.1.Overview

[§5 Syntax structures](#) define the syntax for [OBUs](#). This annex defines a length-delimited format for packing OBUs into a [bitstream](#).

In derived specifications, such as container formats enabling storage of AV2 videos together with audio or subtitles, other methods of packing OBUs into a bitstream format are also allowed.

§ B.2.Length delimited bitstream syntax

```
bitstream( ) {  
    while ( more_data_in_bitstream() ) {  
        leb128() num_bytes_in_obu;  
        open_bitstream_unit( num_bytes_in_obu )  
    }  
}
```

§ B.3.Length delimited bitstream semantics

more_data_in_bitstream() is a system-dependent method of determining when the system reaches the end of the [bitstream](#). The method returns 1 when there is more data to read, or 0 when at the end of the bitstream.

num_bytes_in_obu specifies the length in bytes of the next [OBU](#).

[↑ Back to Table of Contents](#)

§ Annex C: Error resilience behavior (informative)

§ C.1.General

This annex defines additional starting points for decoding.

It is recommended that decoders should support these starting points. (This annex is marked as informative because it is not mandatory for a conformant decoder to support these starting points.)

The intention is to allow decoders to start even when the decoded output may be corrupted.

A property of a bitstream is defined in [Annex C.2 Definition of processable frames](#).

The recommendations are expressed in [Annex C.3 Recommendation for processable frames](#).

The consequences for encoders are specified in [Annex C.4 Encoder consequences of processable frames](#).

The consequences for decoders are specified in [Annex C.5 Decoder consequences of processable frames](#).

§ C.2.Definition of processable frames

This section defines a property of frames that is called being "processable".

Informally, a frame is processable if it is certain (based on the current state and information in the [frame_header_info](#)) that everything other than the [sample values](#) can be decoded correctly.

In particular, a frame that is processable will have correct values for:

- All syntax elements
- The size, bitdepth, subsampling structure of any output frames
- All values written in the reference frame update process specified in [§ 7.23 Reference frame update process](#), except for the contents of FrameStore (which may or may not be correct).

In most codecs, this concept is unnecessary because it is trivial to determine if frames are processable (either because all frames are automatically processable, or because the conditions are straightforward). However, AV2 makes greater use of state in the reference frames and so the condition for being processable is more complicated.

Formally, the property of being processable is defined as follows.

A frame with ShowExistingFrame equal to 0 is defined to be processable if the following conditions are met:

- Either `primary_ref_frame` is equal to `PRIMARY_REF_NONE` or `ref_frame_idx[ primary_ref_frame ]` indicates a frame that has been processed
- Either `use_ref_frame_mvs` is equal to 0, or `ref_frame_idx[ i ]` indicates a frame that has been processed for all `i = 0..REFS_PER_FRAME-1`
- The decoding process for the frame does not use values in `RefOrderHint` before they have been written (written either by the decoding process for the current frame, or written when a previous frame was processed)

- If the syntax element `found_ref` is equal to 1, `ref_frame_idx[ i ]` indicates a frame that has been processed (this is necessary because the frame dimensions are only correct for processed frames)

A frame with `ShowExistingFrame` equal to 1 is processable if the following condition is met:

- `frame_to_show_map_idx` indicates a frame that has been processed

(A frame being "processed" means that the frame was processable and has been decoded.)

§ C.3.Recommendation for processable frames

It is recommended that decoders should support decoding bitstreams if the first temporal unit contains a sequence header and all frames contained in the bitstream are processable according to the definition above.

As the inter prediction may depend on missing reference frames, there is not a requirement that exactly the same output samples as the reference code are produced.

In certain cases (e.g., when the first frame only contains [intra coding](#)), it is possible that correct output is produced, but, in general, error concealment techniques may be required.

§ C.4.Encoder consequences of processable frames

If an application chooses to use a non-key frame starting point, then the encoder needs to be careful that the resulting bitstream is processable.

There are some features of the bitstream specification that make this easier to achieve:

- `primary_ref_frame` and `use_ref_frame_mvs` can be controlled at a frame level to satisfy the corresponding conditions
- `found_ref` can be cleared to allow the frame resolution to be sent explicitly

§ C.5.Decoder consequences of processable frames

For the decoding process to handle this mode of operation, the following modifications should be used:

- `RefValid[ i ]` should be set equal to 0 for $i = 0..NUM_REF_FRAMES-1$ before the decoding process begins
- The requirement for bitstream conformance described in the semantics for `ref_frame_idx[ i ]` (that uses `RefValid` to check that the reference frames are available) should be ignored
- When using the inter [prediction process](#), if `RefValid[ refIdx ]` is equal to 0, then the motion vector scaling and block inter prediction processes are not followed. Instead, `Preds[ refList ]` should be generated using an alternative approach. For conformance testing, it may help to define the predicted samples in a standard way. The suggested approach is to fill `Preds[ refList ]` with neutral gray samples, i.e., all values equal to $1 \ll (BitDepth - 1)$.

[↑ Back to Table of Contents](#)

§ Annex D: Multistream composition process (informative)

§ D.1.General

This annex describes the composition process for combining two or more decoded frames into a single output frame using the spatial layout specified by the `ats_multistream_info` or `ats_multistream_with_alpha_info` syntax structure. This process applies when `ats_atlas_segment_mode_idc[ xAId ]` is equal to `MULTISTREAM_ATLAS` or `MULTISTREAM_ALPHA_ATLAS`.

It is recommended that decoders support the process when the multistream atlas information syntax is present in the bitstream. However, this annex is marked as informative because supporting the composition process or implementing it according to the description is not mandatory for a conformant decoder.

Throughout this annex, let `xlayerId` be equal to `GLOBAL_XLAYER_ID` and let `xAId` be equal to `atlas_segment_id[ xlayerId ]`.

The input to this process is:

- The representation description of the atlas segment (i.e., `ats_atlas_segment_mode_idc`),
- Two or more decoded frames that are associated with the same time instance,
- The extended layer identifier (i.e., `obu_xlayer_id`) for each of the decoded frames,
- The chroma subsampling format for each of the decoded frames,
- The multistream atlas information syntax structure that is associated with the decoded frames (i.e., `ats_multistream_info( xlayerId, xAId )` or `ats_multistream_with_alpha_info( xlayerId, xAId )`).

The output of this process is the composited frame.

The process consists of the following ordered steps:

1. The chroma format determination process specified in [Annex D.2 Chroma format determination process](#) is invoked. The chroma subsampling format for the decoded frames is provided as input. The outputs are the variables `subX` and `subY`.
2. The array initialization process specified in [Annex D.3 Array initialization process](#) is invoked. `ats_msi_width[ xlayerId ][ xAId ]`, `ats_msi_height[ xlayerId ][ xAId ]`, `subX` and `subY` are provided as the width, height, `subX` and `subY` inputs, respectively. The outputs are the arrays `compositeFrameY`, `compositeFrameU`, and `compositeFrameV`.
3. For each value of `i` in the range of `0 ... ats_msi_num_atlas_segments_minus_1[ xlayerId ][ xAId ]`, the following ordered steps are performed:
 1. The variable `segXLayerId` is set equal to `ats_msi_input_stream_id[ xlayerId ][ xAId ][ i ]`
 2. If `ats_atlas_segment_mode_idc[ xAId ]` equals `MULTISTREAM_ATLAS` or `ats_msi_alpha_segment_flag[ xlayerId ][ xAId ][ i ]` equals 0, the spatial mapping process specified in [Annex D.4 Spatial mapping process](#) is invoked. The decoded frame associated with the extended layer identifier `segXLayerId`, `compositeFrameY`, `compositeFrameU`, `compositeFrameV`, `ats_msi_width[ xlayerId ][ xAId ]`, `ats_msi_height[ xlayerId ][ xAId ]`, `i`, `subX`, `subY` are provided as

input. The outputs are modified arrays of compositeFrameY, compositeFrameU, and compositeFrameV values.

3. Otherwise, the following ordered steps apply:

1. The variable iAlpha is set equal to i
2. The variable segXLayerIdAlpha is set equal to `ats_msi_input_stream_id[ xlayerId ][ xAid ][ iAlpha ]`
3. The variable i is incremented by 1
4. The variable segXLayerId is set equal to `ats_msi_input_stream_id[ xlayerId ][ xAid ][ i ]`
5. The spatial mapping process specified in [Annex D.5 Spatial mapping with alpha process](#) is invoked. The decoded frame associated with the extended layer identifier segXLayerId, the decoded alpha frame associated with the extended layer identifier segXLayerIdAlpha, the value of BitDepth for the decoded alpha frame associated with the extended layer identifier segXLayerIdAlpha, compositeFrameY, compositeFrameU, compositeFrameV, `ats_msi_width[ xlayerId ][ xAid ]`, `ats_msi_height[ xlayerId ][ xAid ]`, i, iAlpha, subX, subY are provided as input. The outputs are modified arrays of compositeFrameY, compositeFrameU, and compositeFrameV values.

NOTE: The normative syntax constrains `ats_msi_alpha_segment_flag` to 0 for the last segment (i equal to `ats_msi_num_atlas_segments_minus_1`), ensuring that an alpha segment is always followed by its paired texture segment.

NOTE: All decoded frames should be converted to the same rendering format prior to being input to this process. The conversion process is outside the scope of this annex. But the (non-alpha) input frames should be represented using the same dynamic range, color format, color subsampling and bit-depth.

§ D.2.Chroma format determination process

This section defines the process of determining the chroma subsampling factors.

The input to this process is the chroma subsampling format for the decoded frames.

The outputs of this process are the variables subX and subY.

The process consists of the following ordered steps:

1. If the chroma subsampling format corresponds to a 4:2:0 subsampling format, then the variable subX is set equal to 1 and the variable subY is set equal to 1
2. Otherwise, if the chroma subsampling format corresponds to a 4:2:2 subsampling format, then the variable subX is set equal to 1 and the variable subY is set equal to 0
3. Otherwise, if the chroma subsampling format corresponds to a 4:4:4 subsampling format, then the variable subX is set equal to 0 and the variable subY is set equal to 0.
4. Otherwise (the chroma subsampling format does not correspond to a 4:2:0, 4:2:2 or 4:4:4 subsampling format), the variable subX is set equal to 0 and the variable subY is set equal to 0.

§ D.3.Array initialization process

This section defines the process of initializing a frame array.

The input to this process is:

- The variables width and height that indicate the dimensions of the array to be initialized,
- The variables subX and subY that indicate the chroma subsampling format of the initialized array.

The outputs of this process are the arrays initializedFrameY, initializedFrameU and initializedFrameV.

The process consists of the following ordered steps:

1. The background color determination process specified in [Annex D.3.1 Background color determination process](#) is invoked. `ats_msi_background_red_value[ xlayerId ][ xAId ]`, `ats_msi_background_green_value[ xlayerId ][ xAId ]` and `ats_msi_background_blue_value[ xlayerId ][ xAId ]` are provided as the redValue, greenValue, and blueValue inputs. The outputs are the variables backgroundValueY, backgroundValueU, and backgroundValueV
2. The array initializedFrameY is width samples across by height samples down. The sample at location x samples across and y samples down is given by `initializedFrameY[ y ][ x ] = backgroundValueY`.
3. The array initializedFrameU is 'width >> subX' samples across by 'height >> subY' samples down. The sample at location x samples across and y samples down is given by `initializedFrameU[ y ][ x ] = backgroundValueU`.
4. The array initializedFrameV is 'width >> subX' samples across by 'height >> subY' samples down. The sample at location x samples across and y samples down is given by `initializedFrameV[ y ][ x ] = backgroundValueV`.

§ D.3.1.Background color determination process

This section defines the process of determining the background color for the composited frame.

The inputs to this process are the variables redValue, greenValue, and blueValue.

The outputs of this process are the variables backgroundValueY, backgroundValueU, and backgroundValueV.

The process consists of the following ordered steps:

1. The values Y, U and V are determined that correspond to red, green and blue values specified by redValue, greenValue and blueValue, respectively.
2. The variable backgroundValueY is set equal to Y
3. The variable backgroundValueU is set equal to U
4. The variable backgroundValueV is set equal to V

NOTE: The determination of the background color depends on the dynamic range, color space, bit-depth, and/or other characteristics used by the implementation of the composite frame format.

§ D.4. Spatial mapping process

This section defines the spatial mapping process.

The inputs to this process are:

- A decoded frame that is stored in arrays inputY, inputU, and inputV,
- A decoded frame width and decoded frame height that are stored in the variables inputWidth and inputHeight, respectively,
- A composite frame that is stored in arrays compositeFrameY, compositeFrameU, and compositeFrameV,
- A composite frame width and composite frame height that are stored in the variables compositeFrameWidth and compositeFrameHeight, respectively,
- A segment index that is stored in the variable segIdx,
- A chroma subsampling format that is stored in the variables subX and subY,

The outputs of this process are the modified arrays compositeFrameY, compositeFrameU, and compositeFrameV. The process consists of the following ordered steps:

1. The array initialization process specified in [Annex D.3 Array initialization process](#) is invoked. The ats_msi_segment_width[xlayerId][xAId][segIdx], ats_msi_segment_height[xlayerId][xAId][segIdx], subX, and subY are provided as the width, height, and chroma subsampling format inputs, respectively. The outputs are the arrays resampledFrameY, resampledFrameU, and resampledFrameV.
2. The resampling process specified in [Annex D.5.1 Frame resampling process](#) is invoked. The arrays inputY, inputU, and inputV, and the variables inputWidth, inputHeight, resampledFrameY, resampledFrameU, resampledFrameV, ats_msi_segment_width[xlayerId][xAId][segIdx], ats_msi_segment_height[xlayerId][xAId][segIdx], subX, and subY are provided as input. The outputs are the modified arrays resampledFrameY, resampledFrameU, and resampledFrameV.
3. The arrays compositeFrameY, compositeFrameU, and compositeFrameV are then updated as follows:

```
topLeftPosX = ats_msi_segment_top_left_pos_x[ xlayerId ][ xAId ][ segIdx ]
topLeftPosY = ats_msi_segment_top_left_pos_y[ xlayerId ][ xAId ][ segIdx ]
width = min( ats_msi_segment_width[ xlayerId ][ xAId ][ segIdx ], compositeFrameWidth - topLeftPosX )
height = min( ats_msi_segment_height[ xlayerId ][ xAId ][ segIdx ], compositeFrameHeight - topLeftPosY )

for( x = 0; x < width; x++ ) {
    for( y = 0; y < height; y++ ) {
        compositeFrameY[ y + topLeftPosY ][ x + topLeftPosX ] = resampledFrameY[ y ][ x ]
    }
}

topLeftPosX = topLeftPosX >> subX
topLeftPosY = topLeftPosY >> subY
width = width >> subX
height = height >> subY

for( x=0; x < width; x++ ) {
    for( y = 0; y < height; y++ ) {
        compositeFrameU[ y + topLeftPosY ][ x + topLeftPosX ] = resampledFrameU[ y ][ x ]
        compositeFrameV[ y + topLeftPosY ][ x + topLeftPosX ] = resampledFrameV[ y ][ x ]
    }
}
```

§ D.5.Spatial mapping with alpha process

This section defines the spatial mapping process with an alpha frame.

The inputs to this process are:

- A decoded frame that is stored in arrays inputY, inputU, and inputV,
- A decoded frame width and decoded frame height that are stored in the variables inputWidth and inputHeight, respectively,
- A decoded alpha frame that is stored in array alphaY,
- A decoded alpha frame width and decoded alpha frame height that are stored in the variables alphaWidth and alphaHeight, respectively,
- A decoded alpha frame bitdepth that is stored in the variable bitdepthAlpha,
- A composite frame that is stored in arrays compositeFrameY, compositeFrameU, and compositeFrameV,
- A composite frame width and composite frame height that are stored in the variables compositeFrameWidth and compositeFrameHeight, respectively,
- A segment index that is stored in the variable segIdx,
- An alpha segment index that is stored in the variable segIdxAlpha,
- A chroma subsampling format that is stored in the variables subX and subY,

The outputs of this process are the modified arrays compositeFrameY, compositeFrameU, and compositeFrameV. The process consists of the following ordered steps:

1. The array initialization process specified in [Annex D.3 Array initialization process](#) is invoked. The `ats_msi_segment_width[ xlayerId ][ xAId ][ segIdx ]`, `ats_msi_segment_height[ xlayerId ][ xAId ][ segIdx ]`, `subX`, and `subY` are provided as the width, height, and chroma subsampling format inputs, respectively. The outputs are the arrays `resampledFrameY`, `resampledFrameU`, and `resampledFrameV`.
2. The resampling process specified in [Annex D.5.1 Frame resampling process](#) is invoked. The arrays `inputY`, `inputU`, and `inputV`, and the variables `inputWidth`, `inputHeight`, `resampledFrameY`, `resampledFrameU`, `resampledFrameV`, `ats_msi_segment_width[ xlayerId ][ xAId ][ segIdx ]`, `ats_msi_segment_height[ xlayerId ][ xAId ][ segIdx ]`, `subX`, and `subY` are provided as input. The outputs are the modified arrays `resampledFrameY`, `resampledFrameU`, and `resampledFrameV`.
3. The array `resampleAlphaFrameY` is `ats_msi_segment_width[ xlayerId ][ xAId ][ segIdxAlpha ]` samples across by `ats_msi_segment_height[ xlayerId ][ xAId ][ segIdxAlpha ]` samples down. The sample at location `x` samples across and `y` samples down is given by `resampleAlphaFrameY[ y ][ x ] = 1`.
4. The resampling process specified in [Annex D.5.2 Monochrome frame resampling process](#) is invoked. The array `alphaY` and the variables `alphaWidth`, `alphaHeight`, `resampleAlphaFrameY`, `ats_msi_segment_width[ xlayerId ][ xAId ][ segIdxAlpha ]`, `ats_msi_segment_height[ xlayerId ][ xAId ][ segIdxAlpha ]` are provided as input. The outputs are the modified array `resampleAlphaFrameY`.
5. The arrays `compositeFrameY`, `compositeFrameU`, and `compositeFrameV` are then updated as follows:

```
topLeftPosX = ats_msi_segment_top_left_pos_x[ xlayerId ][ xAId ][ segIdx ]
topLeftPosY = ats_msi_segment_top_left_pos_y[ xlayerId ][ xAId ][ segIdx ]
width = min( ats_msi_segment_width[ xlayerId ][ xAId ][ segIdx ], compositeFrameWidth - topLeftPosX )
```

```

height = min( ats_msi_segment_height[ xlayerId ][ xAid ][ segIdx ], compositeFrameHeight - topLeftPosY )

alphaTopLeftPosX = ats_msi_segment_top_left_pos_x[ xlayerId ][ xAid ][ segIdxAlpha ]
alphaTopLeftPosY = ats_msi_segment_top_left_pos_y[ xlayerId ][ xAid ][ segIdxAlpha ]
alphaWidth = ats_msi_segment_width[ xlayerId ][ xAid ][ segIdxAlpha ]
alphaHeight = ats_msi_segment_height[ xlayerId ][ xAid ][ segIdxAlpha ]
alphaMax = 1 << bitdepthAlpha

for( x = 0; x < width; x++ ) {
    for( y = 0; y < height; y++ ) {
        ax = x + topLeftPosX - alphaTopLeftPosX
        ay = y + topLeftPosY - alphaTopLeftPosY
        alpha = ( ax >= 0 && ax < alphaWidth && ay >= 0 && ay < alphaHeight ) ?
                resampleAlphaFrameY[ ay ][ ax ] : alphaMax
        temp = ( alphaMax - alpha ) * compositeFrameY[ y + topLeftPosY ][ x + topLeftPosX ] + alpha *
resampledFrameY[ y ][ x ]
        compositeFrameY[ y + topLeftPosY ][ x + topLeftPosX ] = Round2(temp, bitdepthAlpha)
    }
}

uvTopLeftPosX = topLeftPosX >> subX
uvTopLeftPosY = topLeftPosY >> subY
width = width >> subX
height = height >> subY

for( x = 0; x < width; x++ ) {
    for( y = 0; y < height; y++ ) {
        ax = (x << subX) + topLeftPosX - alphaTopLeftPosX
        ay = (y << subY) + topLeftPosY - alphaTopLeftPosY
        alpha = ( ax >= 0 && ax < alphaWidth && ay >= 0 && ay < alphaHeight ) ?
                resampleAlphaFrameY[ ay ][ ax ] : alphaMax
        temp = ( alphaMax - alpha ) * compositeFrameU[ y + uvTopLeftPosY ][ x + uvTopLeftPosX ] + alpha *
resampledFrameU[ y ][ x ]
        compositeFrameU[ y + uvTopLeftPosY ][ x + uvTopLeftPosX ] = Round2(temp, bitdepthAlpha)
        temp = ( alphaMax - alpha ) * compositeFrameV[ y + uvTopLeftPosY ][ x + uvTopLeftPosX ] + alpha *
resampledFrameV[ y ][ x ]
        compositeFrameV[ y + uvTopLeftPosY ][ x + uvTopLeftPosX ] = Round2(temp, bitdepthAlpha)
    }
}

```

§ D.5.1. Frame resampling process

This section is a placeholder for the frame resampling process. The actual resampling process is outside the scope of this annex.

The input to this process is:

- An input frame that is stored in the arrays inputY, inputU, and inputV,
- An input frame width and input frame height that are stored in the variables inputWidth and inputHeight, respectively,
- An output frame that is stored in the variables resampledFrameY, resampledFrameU, and resampledFrameV,
- An output frame width and output frame height that is stored in the variables outputWidth and outputHeight, respectively,
- A chroma subsampling format that is stored in the variables subX and subY.

The outputs of this process are arrays of modified resampledFrameY, resampledFrameU, and resampledFrameV values.

§ D.5.2. Monochrome frame resampling process

This section is a placeholder for the monochrome frame resampling process. The actual resampling process is outside the scope of this annex.

The input to this process is:

- An input frame that is stored in the array `inputY`,
- An input frame width and input frame height that are stored in the variables `inputWidth` and `inputHeight`, respectively,
- An output frame that is stored in the variables `resampledFrameY`,
- An output frame width and output frame height that are stored in the variables `outputWidth` and `outputHeight`, respectively,

The output of this process is an array of modified `resampledFrameY` values.

[↑ Back to Table of Contents](#)

§ Annex E: Decoder model

§ E.1.General

The decoder model is used to verify that a bitstream, sub-bitstream or an operating point can be decoded within the constraints imposed by one of the coding levels defined in [Annex A.4 Levels](#). The decoder model is also used to verify conformance for a decoder that claims conformance to a certain coding level.

A set of decoder model parameters may be optionally specified for extended layers or for zero or more operating points. If the new Sequence Header OBU does not signal decoder model parameters for an extended layer, the previous set of decoder model parameters does not persist. If the new Operating Point Set OBU does not signal decoder model parameters for a given operating point, the previous set of decoder model parameters does not persist.

The decoder model constraints are checked for each extended layer independently. When a bitstream includes multiple operating points, the decoder model constraints are verified for each operating point and extended layer independently against its own decoder model parameters (BitRate, BufferSize, DecoderBufferDelay, EncoderBufferDelay) as signaled in the seq_decoder_model_info() or ops_decoder_model_info() and updated, if necessary, according to section [Annex A.4 Levels](#). If the decoder model is verified for a certain operating point or a certain extended layer, the corresponding profile, level and tier are used to set the decoding model parameters.

NOTE: The variables MaxDisplayRate, MaxDecodeRate, and BitRate depend on the value of variable MultiStreamDecoderMode, which is set in [§ 7.4.1 General](#) and used to adjust level variables in [Annex A.4 Levels](#).

The decoder model describes the smoothing buffer, decoding process, operation of the frame buffers and the frame output process.

The decoder model can be applied to an extended layer. The decoder model parameters for an extended layer take into account all embedded layers within that extended layer that are necessary for decoding the extended layer.

The decoder model can be applied to an operating point. An operating point can specify the decoder model that allows establishing conformance to the level signaled for this operating point.

The decoder model defines two modes of operation. A conformant bitstream shall satisfy constraints imposed by one of these two modes of the decoder model depending on which mode is applicable.

[Annex E.2 Operating point selection](#) describes how the operating point is selected for the decoder model.

[Annex E.3 Decoder model definitions](#) defines additional concepts used by the decoder model.

[Annex E.4 Operating modes](#) defines the operating modes.

[Annex E.5 Frame timing definitions](#) specifies how the frame timings can be computed in the different operating modes.

[Annex E.6 Decoder model](#) specifies the decoder model process.

[Annex E.7 Bitstream conformance](#) specifies the conformance requirements.

§ E.2. Operating point selection

The decoder model process is performed for an extended layer or for a certain operating point. The decoder model is applied to each extended layer independently. If an operating point includes more than one extended layer, the decoder model is checked for each extended layer independently. When an extended layer conformance is checked by the decoder model, the OBUs related to this extended layer are taken into account by the decoder model, whereas the OBUs not related to this extended layer are not taken into account by the decoder model.

The operating point is selected by choosing an operating points set `ops_id` and an operating point `op` within the operating point set. When the operating point `op` conformance is checked by the decoder model for a certain extended layer with id `xId`, the OBUs related to the operating point set `ops`, the operating point `op`, and this extended layer `xId` are taken into account by the decoder model, whereas the OBUs not related to the operating point `op` in the operating point set `ops` and the extended layer `xId` are not taken into account by the decoder model. When the decoder model is applied to the entire extended layer `xId`, the entire extended layer is treated as an operating point, whereas the decoder model parameters may be conveyed in the sequence header associated with this extended layer, in an operating point OBU or delivered by external means.

The decoder model parameters are defined as follows.

When the decoder model is applied to the whole extended layer `xId`, the parameters `DecoderBufferDelay`, `EncoderBufferDelay`, and `LowDelayMode` are defined as follows:

- `DecoderBufferDelay` is assigned the value of `decoder_buffer_delay`.
- `EncoderBufferDelay` is assigned the value of `encoder_buffer_delay`.
- `LowDelayMode` is assigned the value of `low_delay_mode_flag`.

Otherwise, when the parameters for the operating point `op` in the operating point set `ops` and xlayer `xId` are present, and the operating point `op` is selected, parameters `DecoderBufferDelay`, `EncoderBufferDelay`, and `LowDelayMode` are defined as follows:

- `DecoderBufferDelay` is assigned the value of `ops_decoder_buffer_delay[ xId ][ ops ][ op ]`.
- `EncoderBufferDelay` is assigned the value of `ops_encoder_buffer_delay[ xId ][ ops ][ op ]`.
- `LowDelayMode` is assigned the value of `ops_low_delay_mode_flag[ xId ][ ops ][ op ]`.

§ E.3. Decoder model definitions

The decoder model uses the following elements to verify bitstream conformance that are not part of the decoding process specified in [§ 7 Decoding process](#).

NOTE: The elements defined in this section do not have to be present in a conformant decoder implementation. These elements may be considered examples of elements of a conformant decoder, although the actual decoder implementation may differ. The elements are defined for the extended layer, which is used by the selected operating point.

BufferPool is a storage area for a set of frame buffers. Buffer pool area allocated for storing separate frames is defined as BufferPool[i], where i takes values from 0 to NumRefFrames + 1. When a frame buffer is used for storing a [decoded frame](#), it is indicated by a VBI slot that points to this frame buffer.

VBI (virtual buffer index) is an array of indices of the frame areas in the BufferPool. VBI elements which do not point to any slot in the BufferPool are set to -1. VBI array size is equal to NumRefFrames, with the indices taking values from 0 to NumRefFrames - 1.

Cfbi (current frame buffer index) is the variable that contains the index to the area in the BufferPool that contains the current frame.

DecoderRefCount[i] is a variable associated with a frame buffer i. DecoderRefCount[i] is initialized to 0, and incremented by 1 each time the decoder adds the buffer i to a VBI index slot. It is decremented by 1 each time the decoder removes the buffer from a VBI index slot i. The decoder may update multiple VBI index slots with the same frame buffer, as specified by refresh_frame_flags, so the counter may be incremented several times. When the counter is 0 the pixel data becomes permanently invalid and shall not be used by the decode process.

PlayerRefCount[i] is a variable associated with a frame buffer i. PlayerRefCount[i] is initialized to 0, incremented by 1 each time the decoder determines that the frame is a presentation frame. It is reset to 0 after the last time the frame is presented.

PresentationTimes[i] is an array corresponding to the BufferPool [i] that holds the last presentation time for the [decoded frame](#) that is kept in the BufferPool [i].

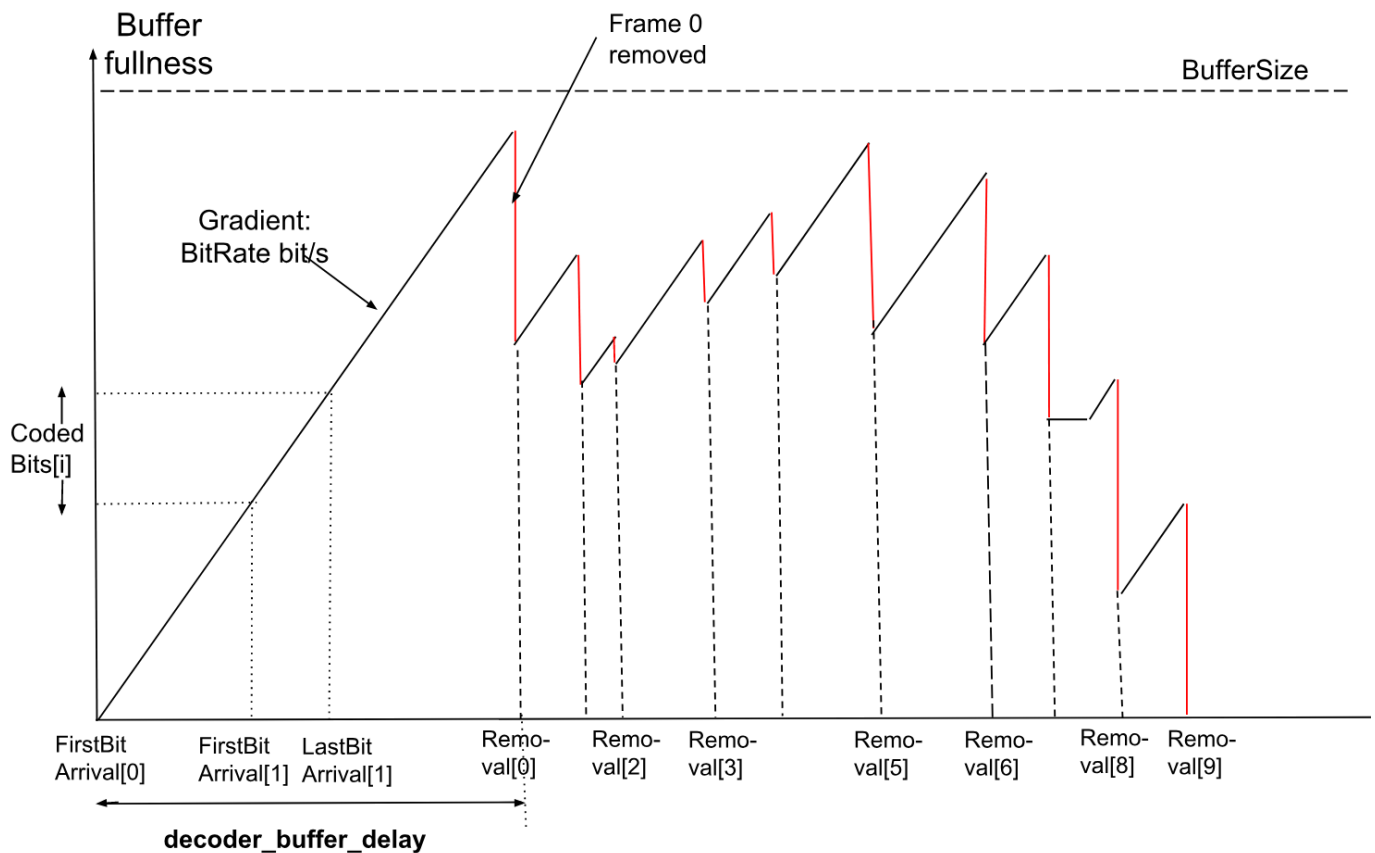


Figure E.1: Example of how the **coded frame** buffer fullness varies as data arrives from the stream, and is subsequently removed for decoding. Relevant timing points and values are indicated.

Coded frames arrive at the decoder smoothing buffer of the size **BufferSize** at a rate defined by **BitRate**. The following variables are used in this section and below:

BitRate is set to a value equal to $\text{MaxBitrate} * \text{BitrateProfileFactor}$ specified for the level signaled for the operating point or an extended layer that is being decoded.

BufferSize is set to a value equal to $\text{MaxBufferSize} * \text{BitrateProfileFactor}$ value specified for the level signaled for the operating point that is being decoded.

Decodable Frame Group i (DFG i) consists of all OBUs, including headers, between the end of the last OBU associated with the previous frame with **ShowExistingFrame** flag equal to 0 (frame k), and the end of the last OBU associated with the current frame with **ShowExistingFrame** flag equal to 0 (frame p). This comprises the OBUs that make up frame p, plus any additional OBUs present in the bitstream that belong to frame p (such as the metadata OBU), and OBUs that belong to frames with **ShowExistingFrame** flag equal to 1 which are located between frame k and frame p. The decoder model assumes that the decoding time for processing a frame with **ShowExistingFrame** flag equal to 1, a header, or a metadata OBU is 0, hence the smoothing buffer operates in the units of DFG. The decoder model used to verify the constraints for an extended layer xId only takes into account OBUs related to the extended layer xId. The decoder model used to verify the constraints for an operating point op in the operating point set ops and the extended layer xId only takes into account OBUs related to the operating point op in the operating point set ops and the extended layer xId. The OBUs not related to the operating point op in the operating

point set ops and the extended layer xId should be omitted by the decoder model and not increase the value of the DFG index i.

CodedBits[i] is the amount of data, in bits, that belongs to DFG i. Note that the index i of the DFG only increases with frames with ShowExistingFrame flag equal to 0, i.e., frames that need to be decoded by the decoding process.

FirstBitArrival[i] is the time when the first bit of the i-th DFG starts entering the decoder smoothing buffer. For the first coded DFG in the sequence, DFG 0 (or after updating decoder model parameters at a random access point), FirstBitArrival[0] = 0.

LastBitArrival[i] is the time when the last bit of DFG i finishes entering the smoothing buffer.

Each **output frame j** has a scheduled presentation time, **PresentationTime[j]**, defined to be a multiple of the display clock tick DispCT. The index j counts all output frames related to the operating point and/or extended layer in output order, including immediate output frames, frames with ShowExistingFrame equal to 1, and implicit output frames. These output frames may belong to one or more embedded layers.

DispCT represents the expected time interval between displaying two consecutive frames, or a common divisor of the expected times between displaying two consecutive frames if the encoded bitstream has a variable display frame rate.

§ E.4. Operating modes

§ E.4.1. Resource availability mode

In this mode the model simulates the operation of the decoder under the assumption that the complete [coded frame](#) is available in the smoothing buffer when decoding of that frame begins. In addition, it is assumed that the decoder will begin to decode a frame immediately after it finishes decoding the previous frame or when a frame buffer becomes available, whichever is later. This model uses the generated time moments when the decoding of a frame begins as times when the data is removed from the smoothing buffer to check the conformance of a bitstream to the bitrate specified for a level signaled for the operating point or an extended layer of a bitstream.

To verify that a bitstream can be decoded by a decoder under the constraints of a particular level it is assumed that the decoder performs the decoding operations at maximum speed (the minimum time interval) specified for that level in [Annex A.4 Levels](#).

To use Resource Availability mode, the following parameters should be set in the encoded video bitstream:

- ci_timing_info_present_flag equal to 1
- ops_decoder_model_info_for_this_op_present_flag[xId][ops][op] equal to 0 and seq_decoder_model_info_present_flag equal to 0
- equal_picture_interval equal to 1,

where xId is the extended layer id for which conformance needs to be established, ops is the operating point set id and op is the selected operating point, and parameter seq_decoder_model_info_present_flag is signaled in the sequence header that is associated with the extended layer xId for which conformance is checked, and equal_picture_interval is signaled in the Content Interpretation OBU.

If the parameters listed above are not specified by the bitstream, the parameters necessary to input into this model can be signaled by the application or some other means. If the parameters necessary to run this model are not signaled, it is not possible to check the conformance of the stream or an operating point to the claimed level.

In this mode of operation, the decoder model parameters below take the following (default) values:

- EncoderBufferDelay = 20 000
- DecoderBufferDelay = 70 000
- LowDelayMode = 0

The decoder writes the [decoded frame](#) into one of the available frame buffers. Decoding must be delayed until a frame buffer becomes available.

§ E.4.2. Decoding schedule mode

This mode imposes additional constraints relating to the operation of the smoothing buffer and the timing points, specified for each frame, defining exactly when the decoder should start decoding a frame and when that frame should be presented.

To use Decoding Schedule Mode, the following parameters should be signaled by the encoded video bitstream:

- ci_timing_info_present_flag equal to 1 in the content interpretation OBU associated with this extended layer
- decoder_model_info_present_flag equal to 1
- seq_decoder_model_info_present_flag equal to 1 or
ops_decoder_model_info_for_this_op_present_flag[xId][op],

where xId is the extended layer, for which conformance needs to be established, ops is the selected operating point set and op is the selected operating point, and parameter seq_decoder_model_info_present_flag is signaled in the sequence header that is associated with the extended layer xId for which conformance needs to be established.

When these flags are signaled, the bitstream should provide the associated information specified in seq_decoder_model_info() or ops_decoder_model_info(), depending on if the parameters are signaled for the extended layer or an operating point.

In addition, for each frame and each operating point op, the following parameters must be specified:

- BufferRemovalTime
- frame_presentation_time for each frame or equal_picture_interval set to 1 in the Content Interpretation OBU

BufferRemovalTime is defined equal to

- br_time when the decoder model is applied to the extended layer xId
- or br_time_op[ops][op] when the decoding model is applied to an operating point set ops and operating point op.

NOTE: The two cases above are mutually exclusive. When `br_ops_dependent_flag` is equal to 0 in the `buffer_removal_timing_obu()`, only `br_time` is present and the decoder model is applied to the extended layer as a whole. When `br_ops_dependent_flag` is equal to 1, only `br_time_op` is present and the decoder model is applied per operating point within the specified operating point set.

If the parameters listed above are not specified by the bitstream, the parameters necessary to input into this model can be signaled by the application or some other means. If the parameters necessary to input into this model are not signaled, it is not possible to check conformance of the stream to the claimed level with this model.

§ E.4.3. Establishing bitstream conformance

When the parameters necessary for the decoding schedule mode are specified by the bitstream, extended layer or an operating point or signaled to the decoder by the application or some other means, the decoder schedule mode shall be used for establishing the bitstream conformance.

When the parameters necessary for the decoding schedule mode are not available and the parameters necessary for the resource availability mode are specified by the bitstream, extended layer or an operating point or signaled to the decoder by the application or some other means, the resource availability mode shall be used for establishing the bitstream conformance.

§ E.4.4. When timing information is not present in the bitstream

When the parameters necessary as the input to at least one of the operating modes specified in [Annex E.4 Operating modes](#), i.e., resource availability mode or decoding schedule mode, are not present in the bitstream, it is impossible to verify whether the bitstream satisfies the level constraints according to either of the decoder models. In order to enable verification of the bitstream conformance, the equivalent information necessary to verify the conformance can be provided by external means. Otherwise, conformance cannot be established.

§ E.5. Frame timing definitions

§ E.5.1. Start of DFG bits arrival

The bits arrive in the smoothing buffer at a constant bitrate `BitRate` or the bitrate equal to 0. Hence, the average bitrate can be lower than the bitrate `BitRate` specified in the level definition, which, in this case, represents a peak bitrate. The first bit of DFG `i` is expected to arrive by the latest time that would guarantee timely reception of the entire DFG by the time when the decodable frame in the DFG `i` is due to be decoded:

```
FirstBitArrival[ i ] = max ( LastBitArrival[ i - 1 ], LatestArrivalTime[ i ] ),
```

where `LatestArrivalTime[ i ]` is the latest time when the first bit of DFG `i` must arrive in the smoothing buffer to ensure that the complete DFG is available at the scheduled removal time, `ScheduledRemoval`

[i], in units of seconds, unless the new set of decoding model parameters is received. In its turn, the latest time the DFG data should start being received is determined as follows:

$$\text{LatestArrivalTime}[i] = \text{ScheduledRemoval}[i] - \frac{(\text{EncoderBufferDelay} + \text{DecoderBufferDelay})}{90\,000}$$

§ E.5.2.End of DFG bits arrival

For the bits that belong to the DFG i, the time of arrival of the last bit of the DFG i is determined as follows:

$$\text{LastBitArrival}[i] = \text{FirstBitArrival}[i] + \text{CodedBits}[i] \div \text{BitRate}$$

§ E.5.3.Scheduled removal times

The decoder starts to decode a frame exactly at the moment when the data corresponding to the DFG of that frame is removed from the smoothing buffer. Each DFG has a scheduled removal time and an actual removal time. Under certain circumstances these times may be different.

The ScheduledRemoval[i] time is determined differently in the resource availability and the decoding schedule mode.

When the decoder model operates in the decoding schedule mode

$$\text{ScheduledRemoval}[i] = \text{ScheduledRemovalTiming}[i]$$

When the decoder model operates in the resource availability mode

$$\text{ScheduledRemoval}[i] = \text{ScheduledRemovalResource}[i]$$

Derivation of ScheduledRemovalTiming[i] in the decoding schedule mode is described in [Annex E.5.4 Removal times in decoding schedule mode](#), and derivation of ScheduledRemovalResource[i] in the resource availability mode is described in [Annex E.5.5 Removal times in resource availability mode](#).

§ E.5.4.Removal times in decoding schedule mode

DFG i is scheduled for removal from the smoothing buffer at time ScheduledRemovalTiming [i] which is defined as an offset, BufferRemovalTime[i], signaled for the frame of the DFG with ShowExistingFrame equal to 0, relative to the moment of time when the first DFG is removed from the smoothing buffer, DecoderBufferDelay:

$$\begin{aligned} \text{ScheduledRemovalTiming}[0] &= \text{DecoderBufferDelay} \div 90\,000 \\ \text{ScheduledRemovalTiming}[i] &= \text{ScheduledRemovalTiming}[\text{PrevRap}] + \text{BufferRemovalTime}[i] * \text{DecCT} \end{aligned}$$

When i is not equal to 0 and frame i is associated with a random access point, PrevRap is the index associated with the previous random access point. Otherwise, if frame i is not associated with the random access point, PrevRap corresponds to the index associated with the most recent random access point.

DFG i is removed from the smoothing buffer at time $\text{Removal}[i]$.

There are two modes of operation of a decoder which determine whether the actual DFG removal time $\text{Removal}[i]$ may be different from the scheduled DFG removal timing $\text{ScheduledRemovalTiming}[i]$. As mentioned earlier, the decoder starts decoding a frame when the data that belongs to its DFG is removed from the smoothing buffer.

In this mode, frame decoding start times / DFG removal times are determined by the $\text{BufferRemovalTime}[i]$ for the chosen operating point, op or extended layer.

If LowDelayMode is equal to 0, the decoder operates in Strict Arrival Mode, and DFG is removed from the smoothing buffer at the scheduled time, that is:

```
Removal[ i ] = ScheduledRemovalTiming[ i ]
```

Otherwise, LowDelayMode is equal to 1 and the decoder operates in Low-Delay Mode, where the DFG data may not be available in the smoothing buffer at the scheduled removal time, i.e., $\text{ScheduledRemovalTiming}[i] < \text{LastBitArrival}[i]$. In that case, the removal of the DFG is deferred until the first decode clock tick after the complete DFG is present in the smoothing buffer, that is:

```
Removal[ i ] = ceil ( LastBitArrival[ i ] ÷ DecCT ) * DecCT
```

If the entire DFG is available in the smoothing buffer at the scheduled removal time, i.e., $\text{ScheduledRemovalTiming}[i] \geq \text{LastBitArrival}[i]$, then it is removed at the scheduled time, that is:

```
Removal[ i ] = ScheduledRemovalTiming[ i ]
```

§ E.5.5. Removal times in resource availability mode

In the resource availability mode, $\text{BufferRemovalTime}[i]$ are not signaled for the chosen operating point. In this mode, timing of the decoder model is driven by the availability of the resources in the decoder, in particular, by times when the decoding of the previous frame with ShowExistingFrame flag equal to 0 has been completed and a free frame buffer is available.

In particular, $\text{ScheduledRemovalResource}[i]$ times are generated as the earliest time that a non-assigned frame buffer becomes available for decoding of the frame i . In this mode, the decoder starts to decode a frame as fast as it can after completing decoding of the previous frame and a free frame buffer is available. A frame buffer is defined as being available if it is no longer being used and its content can be overwritten.

Removal times in the resource availability mode are produced by [Annex E.6.2 Decoder model functions](#).

The following function, `time_next_buffer_is_free`, is used by the decode process to determine the $\text{Removal}[i]$ time for the next DFG and generate the value of $\text{ScheduledRemovalResource}[i]$.

```
time_next_buffer_is_free ( i, time ) {  
    if ( i == 0 ) {  
        time = DecoderBufferDelay ÷ 90000  
    }  
    foundBuffer = 0  
    for ( k = 0; k < NumRefFrames + 2; k++ ) {  
        if ( DecoderRefCount[ k ] == 0 ) {
```

```

    if ( PlayerRefCount[ k ] == 0 ) {
        ScheduledRemovalResource[ i ] = time
        return time
    }
    if ( !foundBuffer || PresentationTimes[ k ] < bufFreeTime ) {
        bufFreeTime = PresentationTimes[ k ]
        foundBuffer = 1
    }
}
}
ScheduledRemovalResource[ i ] = bufFreeTime
return bufFreeTime
}

```

§ E.5.6.Frame decode timing

The time required to decode a frame (i.e., to process the decodable frame's DFG), `TimeToDecode [ i ]`, is calculated based on the frame type, a maximum number of luma pixels for the frame, and the throughput of the decoder as specified in the definition of the level assigned to the operating point or extended layer that the frame belongs to.

The time that it takes the decoder to decode a frame according to the decoder model is estimated by using the function `time_to_decode_frame( )` as follows.

```

time_to_decode_frame( ) {
    if ( ShowExistingFrame == 1 ) {
        lumaSamples = 0
    } else if ( FrameIsIntra ) {
        if ( allow_global_intrabc == 1 && InloopFilteringEnabled == 1 )
            lumaSamples = 2 * FrameWidth * FrameHeight
        else
            lumaSamples = FrameWidth * FrameHeight
    } else {
        lumaSamples = ( max_frame_width_minus_1 + 1 ) *
                      ( max_frame_height_minus_1 + 1 )
    }
    return lumaSamples ÷ MaxDecodeRate
}

```

§ E.5.7.Frame presentation timing

When the decoder model is applied to the whole extended layer, `InitialDisplayDelay` is set to `seq_initial_display_delay_minus_1 + 1`.

When the decoder model is applied to a chosen operating point, `InitialDisplayDelay` is set equal to `ops_initial_display_delay_minus_1[ xId ][ ops ][ op ] + 1` if the `ops_initial_display_delay_present_flag[ xId ][ ops ][ op ]` is equal to 1 for the current operating point and to `seq_initial_display_delay_minus_1 + 1` when `ops_initial_display_delay_present_flag[ xId ][ ops ][ op ]` is equal to 0 or is not specified for the current operating point.

Initial presentation delay is determined as follows:

```

InitialPresentationDelay = Removal [ InitialDisplayDelay - 1 ] +
                          TimeToDecode [ InitialDisplayDelay - 1 ]

```

When `equal_picture_interval` is equal to 0, the decoder operates in variable frame rate mode, the frame presentation time is defined as follows:

```
PresentationTime[ 0 ] = InitialPresentationDelay  
  
PresentationTime[ j ] = PresentationTime[ PrevPresent ] +  
                        frame_presentation_time[ j ] * DispCT
```

When `j` is not equal to 0 and frame `j` is associated with a leading frame or a random access point, `PrevPresent` corresponds to the index associated with the previous random access point. Otherwise, `PrevPresent` corresponds to the index associated with the last random access point.

When `equal_picture_interval` is equal to 1, the decoder operates in the constant frame rate mode, and the frame presentation time is defined as follows:

```
PresentationTime[ 0 ] = InitialPresentationDelay
```

If frame `j` and frame `j - 1` belong to the same temporal unit

```
PresentationTime[ j ] = PresentationTime[ j - 1 ]
```

Otherwise, if frame `j` and frame `j - 1` belong to different temporal units

```
PresentationTime[ j ] = PresentationTime[ j - 1 ] +  
                        ( num_ticks_per_picture_minus_1 + 1 ) * DispCT ;
```

where `PresentationTime[ j - 1 ]` refers to the previous frame in the output order, and `j` counts all output frames.

The presentation interval, i.e., the time interval between the display of consecutive frames `j` and `j + 1` in presentation order and when frames `j` and `j + 1` belong to different temporal units is defined as follows:

```
PresentationInterval[ j ] = PresentationTime[ j + 1 ] - PresentationTime[ j ]
```

§ E.6.Decoder model

§ E.6.1.Decoder model structure

The decoder model simulates the values of selected timing points as successive frames are decoded. This includes the time that the decoder has to wait for a free frame buffer, the time required to decode the frame and various basic checks to make sure that buffer slots are occupied when they are supposed to be. Non-conformance is signaled by a call to the function `bitstream_non_conformant`; the various error codes are tabulated in [Annex E.6.3 Decoder model error codes](#).

To align the decoder model with the general decoding process and output frame management, the decoder model in AV2 is defined as running in parallel to the decoding process and relies on the decoding process functions for the reference frames management and frame output.

In particular, the decoder model defines functions that are invoked at specified points of the corresponding functions and processes of [§ 7 Decoding process](#). This allows the decoder model to rely on variables and processes defined in the [§ 7 Decoding process](#) and other parts of the specification.

The proposed approach is used for convenience of the decoder model description and to avoid duplication of definitions of certain functions and processes. Other implementations of the decoder model may use a standalone approach that derives values of variables used by the decoder model without the use of the complete decode process.

§ E.6.2.Decoder model functions

This section defines the buffer management functions invoked by the decoder model process.

The `free_buffer` function clears the variables for a particular index in the BufferPool.

```
free_buffer( idx ) {  
    DecoderRefCount[ idx ] = 0  
    PlayerRefCount[ idx ] = 0  
    PresentationTimes[ idx ] = -1  
}
```

The `initialize_buffer_pool` function resets the BufferPool and the VBI.

```
initialize_buffer_pool( ) {  
    for ( i = 0; i < NumRefFrames + 2; i++ )  
        free_buffer( i )  
    for ( i = 0; i < NumRefFrames; i++ )  
        VBI[ i ] = -1  
}
```

The `initialize_decoder_model` function initializes the BufferPool related arrays and sets the decoder model variables to initial values. This function is called before the start of decoding an extended layer or an operating point. This function is also called during random access before the start of the decoding process.

```
initialize_decoder_model( ) {  
    initialize_buffer_pool( )  
    Time = 0  
    FrameNum = -1  
    DfgNum = -1  
    ShownFrameNum = -1  
    Cfbi = -1  
    InitialPresentationDelay = 0  
}
```

The `get_free_buffer` function searches for an un-assigned frame in the BufferPool. The decoder needs an un-assigned frame buffer from the BufferPool for each frame that it decodes.

```
get_free_buffer( ) {  
    for ( i = 0; i < NumRefFrames + 2; i++ ) {  
        if ( DecoderRefCount[ i ] == 0 &&  
            PlayerRefCount[ i ] == 0 )  
            return i  
    }
```

```

    }
    return -1
}

```

In the decoding schedule mode, the decoder only starts to decode a frame at the time designated by a removal time associated with that frame, and expects a free frame buffer to be immediately available.

In the resource availability mode, the decoder may start to decode the next frame as soon as a free reference buffer is available. If a free frame buffer is not available immediately, the `PresentationTimes[ i ]` may be used to compute the time when such a buffer will become available.

The function `start_decode_at_removal_time` returns buffers to the `BufferPool` when they are no longer required for decode or display.

```

start_decode_at_removal_time( removal ) {
    for ( i = 0; i < NumRefFrames + 2; i++ ) {
        if ( PlayerRefCount[ i ] > 0 ) {
            if ( PresentationTimes[ i ] <= removal ) {
                PlayerRefCount[ i ] = 0
                if ( DecoderRefCount[ i ] == 0 )
                    free_buffer( i )
            }
        }
    }
    return removal
}

```

Function `start_frame_decode` is invoked at the start of the [§ 7.2 Decode frame wrapup process](#) function in the decoding process. Function `start_frame_decode` does not change the flow or the results of the [§ 7.2 Decode frame wrapup process](#). It uses the variables available to the decoding process at the start of the [§ 7.2 Decode frame wrapup process](#). In `start_frame_decode`, `UsingResourceAvailabilityMode` is a variable that is set to 1 when using resource availability mode, or 0 when using decoding schedule mode.

```

start_frame_decode( ){
    FrameNum++
    if ( !ShowExistingFrame ) {
        DfgNum++
        if ( UsingResourceAvailabilityMode )
            Removal [ DfgNum ] = time_next_buffer_is_free( DfgNum, Time )
        Time = start_decode_at_removal_time( Removal[ DfgNum ] )
        Cfbi = get_free_buffer( )
        if ( Cfbi == -1 )
            bitstream_non_conformant( DECODE_FRAME_BUF_UNAVAILABLE )
        Time += time_to_decode_frame( )
    }
}

```

Once decoded, frames may update one or more of the VBI index slots, as defined by `refresh_frame_flags`. Each time a VBI index slot is updated, the decoder reference count is incremented by 1 for the corresponding frame buffer. If the VBI index slot being updated is currently occupied, the decoder reference count for the frame buffer being displaced must be decremented by 1.

The `update_ref_buffers` function is called at the end of [§ 7.2.3 Reference frame update process](#). The function `update_ref_buffers` updates the VBI and reference counts when the reference frames are updated, according to the results of [§ 7.2.3 Reference frame update process](#). This function mirrors the results of the frame update process in [§ 7.2.3 Reference frame update process](#) with respect to the decoder

model variables. This function uses `refresh_frame_flags` of the current frame and the `RefValid` array updated by [§ 7.23 Reference frame update process](#).

```
update_ref_buffers ( ) {
    for ( i = 0; i < NumRefFrames; i++ ) {
        if ( (refresh_frame_flags >> i) & 1 ) {
            if ( VBI[ i ] != -1 )
                DecoderRefCount[ VBI[ i ] ] --
            if( RefValid[ i ] ){
                VBI[ i ] = Cfbi
                DecoderRefCount[ Cfbi ] ++
            } else
                VBI[ i ] = -1
        }
    }
}
```

The decoder needs to know the number of decoded frames in the `BufferPool` in order to determine the presentation delay for the first frame. A buffer is un-assigned if both `DecoderRefCount[ i ]` is equal to 0, and `PlayerRefCount[ i ]` is equal to 0.

The function `frames_in_buffer_pool` returns the number of assigned frames in the `BufferPool`.

```
frames_in_buffer_pool( ) {
    framesInPool = 0
    for ( i = 0; i < NumRefFrames + 2; i++ )
        if ( DecoderRefCount[ i ] != 0 || PlayerRefCount[ i ] != 0 )
            framesInPool++
    return framesInPool
}
```

The function `set_initial_presentation_delay` is invoked during [§ 7.2 Decode frame wrapup process](#) immediately after the function [§ 7.23 Reference frame update process](#) is invoked and has returned. The function `set_initial_presentation_delay` initializes the `InitialPresentationDelay`.

```
set_initial_presentation_delay( ){
    if ( !ShowExistingFrame ) {
        if ( InitialPresentationDelay == 0 &&
            ( frames_in_buffer_pool( ) >=
              InitialDisplayDelay ) )
            InitialPresentationDelay = Time
    }
}
```

Function `check_output_frame` is invoked at the end of [§ 7.21.1 Output process](#). The function checks the availability of the frames to be output, increases the output frame number and checks if the frames can be output at their presentation time.

```
check_output_frame( ){
    if ( frameToShowMapIdx == -1 ) {
        bufIdx = Cfbi
    } else {
        if ( !RefValid[ frameToShowMapIdx ] || VBI[ frameToShowMapIdx ] == -1 )
            bitstream_non_conformant( DECODE_EXISTING_FRAME_BUF_EMPTY )
        bufIdx = VBI[ frameToShowMapIdx ]
    }
    ShownFrameNum++
    PresentationTimes[ bufIdx ] = PresentationTime[ ShownFrameNum ]
    PlayerRefCount[ bufIdx ]++
}
```

```

if ( InitialPresentationDelay != 0 ) {
    if ( Time > PresentationTime[ ShownFrameNum ] )
        bitstream_non_conformant( DISPLAY_FRAME_LATE )
}
}

```

NOTE: PresentationTime[ShownFrameNum] includes the InitialPresentationDelay in its calculation. However, InitialPresentationDelay may be unknown until the number of frames in the buffer pool reaches InitialDisplayDelay. Depending on the implementation, PresentationTime of output frames may need to be updated when the InitialPresentationDelay is known.

§ E.6.3.Decoder model error codes

The various non-conformant error codes are as specified in [Table E.1](#):

Table E.1: Error codes produced by bitstream_non_conformant().

Error Codes	Description
DECODE_FRAME_BUF_UNAVAILABLE	All the frame buffers were in use.
DECODE_EXISTING_FRAME_BUF_EMPTY	The buffer of the frame designated for display was empty.
DISPLAY_FRAME_LATE	The frame was decoded too late for timely display, i.e., by the PresentationTime[i] time associated with the frame.

§ E.7.Bitstream conformance

§ E.7.1.General

A conformant coded bitstream shall satisfy the following set of constraints.

For the decoder model, a DFG shall be available in the smoothing buffer at the scheduled removal time, i.e., $\text{ScheduledRemoval}[i] \geq \text{LastBitArrival}[i]$.

It is a requirement of the bitstream conformance that after each random access point, the $\text{PresentationTime}[j]$, where j corresponds to the frame output order (counting all output frames, including implicit output frames) is non-decreasing until the next random access point or the end of the coded video sequence, i.e., $\text{PresentationTime}[j+1] \geq \text{PresentationTime}[j]$.

When $\text{BufferRemovalTime}[i]$ is not specified in the bitstream, a bitstream is conformant if the decoder model in resource availability mode can decode frames successfully before they are scheduled for presentation.

If $\text{BufferRemovalTime}[i]$ is signaled, it shall have a value greater than or equal to the equivalent value that would have been assigned if the decoder model was decoding frames in the resource availability mode.

It is a requirement of a bitstream conformance that a conformant bitstream is decodable according to the decoder model if the decoding starts from any of its random access points. This means that for a conformant bitstream, a bitstream produced from the conformant bitstream by removing the part of the bitstream preceding a random access point associated with an `OBU_CLOSED_LOOP_KEY` shall also be a conformant bitstream according to the decoder model.

For a conformant bitstream, a bitstream produced from the conformant bitstream by: 1) removing the part of the bitstream preceding a random access point associated with an OBU_OPEN_LOOP_KEY 2) removing the part of the bitstream corresponding to the leading frames following the OBU_OPEN_LOOP_KEY shall also be a conformant bitstream according to the decoder model.

For a random access point associated with an OBU_RAS_FRAME, the bitstream shall also be a conformant bitstream according to the decoder model, provided that the long-term key frames are available at the specified frame buffer slots.

Conformance requirements based on a decoder model are not applicable to a bitstream with `seq_level_idx` equal to 31.

In addition to these, a conformant bitstream shall satisfy the constraints specified in the following sections.

§ E.7.2.Decoder buffer delay consistency across random access points (applies to decoding schedule mode)

For frame i , where $i > 0$, `TimeDelta[ i ]` is defined as follows:

```
TimeDelta[ i ] = ( ScheduledRemoval[ i ] - LastBitArrival[ i - 1 ] ) * 90 000
```

For the video sequence that includes one or more random access points, for each random access point, where the `DecoderBufferDelay` is signaled, the following expression shall hold.

```
DecoderBufferDelay <= ceil( TimeDelta[ i ] )
```

§ E.7.3.Smoothing buffer overflow

Smoothing buffer overflow is defined as the state where the total number of bits in the smoothing buffer exceeds the size of the smoothing buffer `BufferSize`. The smoothing buffer shall never overflow.

§ E.7.4.Smoothing buffer underflow

Smoothing buffer underflow is defined as the state where a complete DFG is not present in the smoothing buffer at the scheduled removal time, `ScheduledRemoval[ i ]`:

```
ScheduledRemoval[ i ] < LastBitArrival[ i ]
```

When the `LowDelayMode` is equal to 0, the smoothing buffer shall never underflow.

§ E.7.5.Minimum decode time (applies to decoding schedule mode)

There must be enough time between a DFG being removed from the smoothing buffer, `Removal[ i ]`, and the scheduled removal of the next DFG, `ScheduledRemoval[ i + 1 ]`:

```
ScheduledRemoval[ i + 1 ] - Removal[ i ] >= Max( TimeToDecode[ i ],  
1 ÷ MaxNumFrameHeadersPerSec ),
```

where `MaxNumFrameHeadersPerSec` is defined in the level constraints.

§ E.7.6. Minimum presentation interval

Variable `numOutputFramesInTU[ j ]` is equal to the number of output frames with the `PresentationTime[ j ]`, in the temporal unit associated with the presentation time `PresentationTime[ j ]`, that belong to the selected operating point `op` in the operating point set `ops` and / or extended layer `xId`, which may include frames that belong to different embedded layers.

The difference between presentation times for consecutive shown frames or groups of shown frames that belong to different temporal units, shall satisfy the following constraint:

```
MinFrameTime = MaxDecodeRate ÷ ( MaxNumFrameHeadersPerSec * MaxDisplayRate )

PresentationInterval[ j ] >= Max( ( max_frame_width_minus_1 + 1 ) * ( max_frame_height_minus_1 + 1 )
    * numOutputFramesInTU[ j ] ÷ MaxDisplayRate, MinFrameTime )
```

Where `MaxNumFrameHeadersPerSec` is defined in the level constraints.

§ E.7.7. Decode deadline

It is a requirement of the bitstream conformance that each frame shall be fully decoded at, or before, the time that it is scheduled for presentation:

```
Removal[ i ] + TimeToDecode[ i ] <= PresentationTime[ i ]
```

§ E.7.8. Level imposed constraints

When operating in the decoding schedule mode, `DecoderBufferDelay` shall not be equal to 0 and shall not exceed $90000 * (\text{BufferSize} \div \text{BitRate})$.

NOTE: It is common to choose $((\text{EncoderBufferDelay} + \text{DecoderBufferDelay}) \div 90000) * \text{BitRate}$ equal to a constant within a coded video sequence, and for this constant to be equal to `BufferSize`, but these are not strict requirements for bitstream conformance.

§ E.7.9. Decode Process constraints

It is a requirement of bitstream conformance that the decoder model process can be invoked with the bitstream data for any signaled operating point or an extended layer without triggering a call to the `bitstream_non_conformant` function.

[↑ Back to Table of Contents](#)

§ Annex F: Sub-bitstream extraction (informative)

§ F.1.General

This annex specifies processes for extracting sub-bitstreams from AV2 bitstreams based on operating point selection. The [sub-bitstream extraction process](#) allows decoders to selectively decode portions of a bitstream that match their capabilities or application requirements.

An AV2 bitstream may contain one or more operating points, defined within [OPS](#) OBUs, that describe different combinations of extended layers, embedded layers, and temporal layers. A decoder can select an appropriate operating point and extract a [sub-bitstream](#) containing only the OBUs associated with that operating point.

The extraction process differs depending on whether the bitstream is a [multistream](#) bitstream or a [singlestream](#) bitstream:

- For [multistream](#) bitstreams, extraction involves selecting extended layers, embedded layers, and temporal layers from [global operating point sets](#), and optionally refining individual extended layers using [local operating point sets](#).
- For [singlestream](#) bitstreams, extraction involves selecting embedded and temporal layers from local operating point sets.

The processes defined in this annex are informative and represent one conformant approach to sub-bitstream extraction. Decoders may use alternative methods provided they produce equivalent results.

§ F.2.Operating point usage

§ F.2.1.General decoder operation

When decoding an AV2 bitstream, a decoder can select to decode the entire bitstream or can examine whether it contains operating points, defined within one or more [OPS](#) OBUs, which may be more appropriate given the decoder's capabilities or the intended application.

The decoder operation depends on whether the bitstream is a [multistream](#) bitstream or a [singlestream](#) bitstream.

NOTE: The decoder modes described below allow selection of operating points that may retain a subset of the extended layers and embedded layers present in the bitstream. These processes are valuable for applications that require partial decoding of a bitstream. However, operating point selection does not change the conformance requirements defined in [Annex A.2 Profiles](#). Without direction from application-level requirements external to this specification, a conformant decoder is expected to decode all extended layers and embedded layers present in the bitstream.

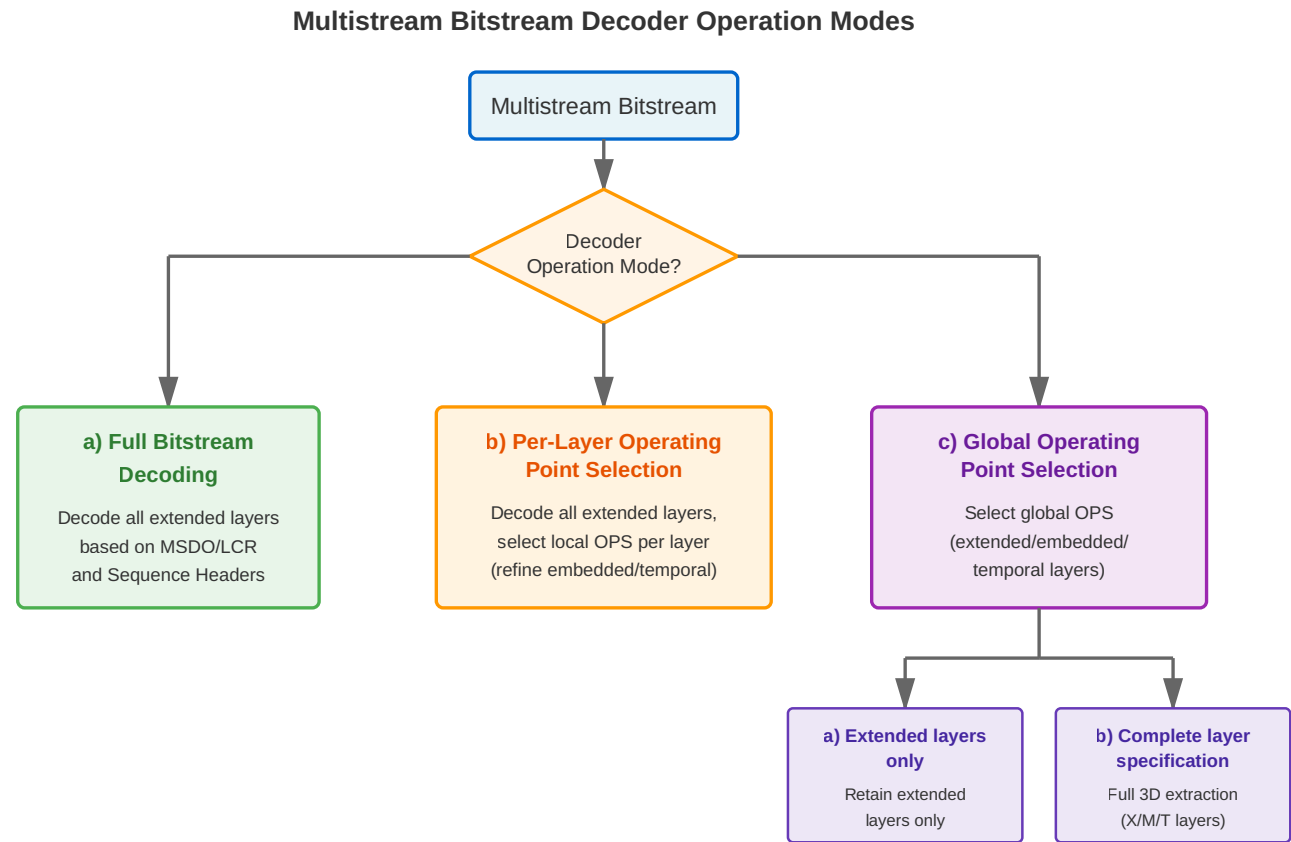
§ F.2.2.Multistream bitstream decoder operation

When decoding an AV2 bitstream, a decoder first invokes the operating point selection and analysis process ([Annex F.3.1 Operating point selection and analysis process](#)). This process examines the bitstream structure and determines whether it is a [multistream](#) or [singlestream](#) bitstream (see Step 2 of the process).

If the process determines the bitstream is a [multistream](#), the bitstream contains several extended layer sub-bitstreams. The bitstream may include an MSDO OBU and/or one or more [LCR](#) OBUs that describe the structure and properties of the bitstream and each associated extended layer sub-bitstream. The bitstream may also contain one or more [global operating point sets](#) providing operating points that span multiple extended layers.

Each extended layer sub-bitstream has its own OBUs, including Sequence Header OBUs, MFH OBUs, video coding layer OBUs ([CLK](#), [OLK](#), [TG](#), [SEE](#), [TIP](#), etc.), and other OBU types. Extended layers may also contain [local operating point sets](#).

For [multistream](#) bitstreams, a decoder may operate in one of the following modes (illustrated in the figure below):



Legend:
X = Extended layer, M = Embedded layer, T = Temporal layer

Figure F.1: Multistream bitstream decoder operation modes showing the three decoding approaches: full bitstream decoding, per-layer operating point selection, and global operating point selection with its two sub-modes.

§ **F.2.2.1. Full bitstream decoding**

Decode the entire bitstream including all extended layers based on the information provided in the MSDO or global LCR OBUs, when present, and the associated Sequence Headers of each extended layer.

§ F.2.2.2. Per-layer operating point selection

Decode all extended layers associated with the bitstream, but for each extended layer examine if there are any [local operating point sets](#) that may be preferable for operation.

The decoder invokes the operating point selection and analysis process defined in [Annex F.3.1 Operating point selection and analysis process](#) with input `inputBitstream` (the entire input bitstream). In this decoder mode, the abstract function `global_operating_point_selection()` returns an indication to decode all extended layers (no global operating point constraints), and the abstract function `local_operating_point_selection(xLayerId)` is called for each extended layer to potentially select a local operating point for embedded/temporal layer refinement.

The process outputs the arrays `OpRetentionMap`, `OpXLayerIsSelected`, `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt`.

The decoder then invokes the sub-bitstream extraction process defined in [Annex F.3.2 Sub-bitstream extraction process](#) with inputs: `inputBitstream` (the entire input bitstream) and `OpRetentionMap`. The process outputs `subBitstream`.

The decoder then decodes `subBitstream` and uses the arrays `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt` for conformance verification of each independent extended layer that is still present in the `subBitstream`. Extended layers with `OpXLayerIsSelected[xLayerId] == 0` are not selected and their corresponding entries in `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt` will have INVALID values.

§ F.2.2.3. Global operating point selection

Examine if one or more [global operating point sets](#) (`obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`) are present. If yes, examine if there is a preferred operating point in one of these operating point sets based on application needs or device capabilities, and use its information to select which layers to decode.

The decoder invokes the operating point selection and analysis process defined in [Annex F.3.1 Operating point selection and analysis process](#) with input `inputBitstream` (the entire input bitstream).

A global operating point may specify extended layers only, or it may specify complete information about extended layers, embedded layers, and temporal layers. Depending on the level of detail provided in the selected operating point, the abstract function `global_operating_point_selection()` and the abstract function `local_operating_point_selection(xLayerId)` behave differently:

a) Extended layers only

If the operating point contains information about which extended layers to retain (via `ops_xlayer_map`), but does not provide complete details about their associated embedded and temporal layers (i.e., `ops_mlayer_map` and `ops_tlayer_map` are not fully specified for all indicated extended layers), the decoder may choose between two approaches:

- The abstract function `global_operating_point_selection()` returns the selected global operating point (`globalOpsId`, `globalOpIdx`), which determines the extended layers to retain. The abstract function `local_operating_point_selection(xLayerId)` returns an indication to decode all embedded and temporal layers for each selected extended layer (no further refinement). This results in an `OpRetentionMap` where only the selected extended layers have non-zero entries, and for each such extended layer, all embedded and temporal layer entries are set to 1.

- Alternatively, the abstract function `global_operating_point_selection()` returns the selected global operating point (`globalOpsId`, `globalOpIdx`) to determine extended layers, and the abstract function `local_operating_point_selection(xLayerId)` examines local OPS information (if available) for each selected extended layer to refine embedded and temporal layers. This results in an `OpRetentionMap` with selective extended layers and refined embedded/temporal layers based on local operating points.

The operating point selection and analysis process outputs the arrays `OpRetentionMap`, `OpXLayerIsSelected`, `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt`.

The decoder then invokes the sub-bitstream extraction process defined in [Annex F.3.2 Sub-bitstream extraction process](#) with inputs: `inputBitstream` (the entire input bitstream) and `OpRetentionMap`. The process outputs `subBitstream`.

The decoder then decodes `subBitstream` and uses the arrays `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt` for conformance verification of each independent extended layer that is still present in the `subBitstream`. Extended layers with `OpXLayerIsSelected[xLayerId] == 0` are not selected and their corresponding entries in `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt` will have `INVALID` values.

b) Complete layer specification

If the operating point contains complete information about the extended layers (via `ops_xlayer_map`), embedded layers (via `ops_mlayer_map`), and temporal layers (via `ops_tlayer_map`) that should be retained, the abstract function `global_operating_point_selection()` returns the selected global operating point (`globalOpsId`, `globalOpIdx`), and the operating point selection and analysis process uses the complete layer information from the global OPS to build the `OpRetentionMap` (Step 4 may use global OPS embedded/temporal layer information instead of calling `local_operating_point_selection`).

The operating point selection and analysis process outputs the arrays `OpRetentionMap`, `OpXLayerIsSelected`, `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt`.

The decoder then invokes the sub-bitstream extraction process defined in [Annex F.3.2 Sub-bitstream extraction process](#) with inputs: `inputBitstream` (the entire input bitstream) and `OpRetentionMap`. The process outputs `subBitstream`.

The decoder then decodes `subBitstream` and uses the arrays `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt` for conformance verification of each independent extended layer that is still present in the `subBitstream`. Extended layers with `OpXLayerIsSelected[xLayerId] == 0` are not selected and their corresponding entries in `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt` will have `INVALID` values.

§ F.2.3. Singlestream bitstream decoder operation

As described in [Annex F.2.2 Multistream bitstream decoder operation](#), when decoding an AV2 bitstream, a decoder first invokes the operating point selection and analysis process ([Annex F.3.1 Operating point selection and analysis process](#)). This process examines the bitstream structure and determines whether it is a [multistream](#) or [singlestream](#) bitstream (see Step 2 of the process).

If the process determines the bitstream is [singlestream](#) (only a single distinct extended layer identifier is present), the bitstream contains only a single extended layer sub-bitstream. It may contain global level (`obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`) OBU types such as temporal delimiters. The bitstream

includes Sequence Header OBUs, MFH OBUs, video coding layer OBUs ([CLK](#), [OLK](#), [TG](#), [SEF](#), [TIP](#), etc.), and other OBU types. It may also contain [local operating point sets](#).

For [singlestream](#) bitstreams, a decoder may operate in one of the following modes:

§ **F.2.3.1. Full bitstream decoding**

Decode the entire bitstream based on its Sequence Header information. No extraction is performed. The output is identical to the input bitstream, and the profile, tier, and level information are as indicated in the sequence header.

§ **F.2.3.2. Local operating point selection**

Examine if local OPS information exists, and if so, select a local operating point based on the application and capabilities of the device. In this case, retain only the embedded and temporal layers in the bitstream that correspond to the selected local operating point, and discard the others.

The decoder invokes the operating point selection and analysis process defined in [Annex F.3.1 Operating point selection and analysis process](#) with input `inputBitstream` (the entire input bitstream). In this decoder mode, since the bitstream is a [singlestream](#) bitstream (single extended layer), the abstract function `global_operating_point_selection()` returns an indication to decode all extended layers (which is effectively the single extended layer present), and the abstract function `local_operating_point_selection(xLayerId)` is called for the single extended layer to select a local operating point for embedded/temporal layer refinement.

The process outputs `OpRetentionMap` (with non-zero entries only for the single extended layer), `OpXLayerIsSelected` (with only one entry set to 1), `OpProfileIdc`, `OpLevelIdc`, `OpTierIdc`, and `OpMlayerCnt`.

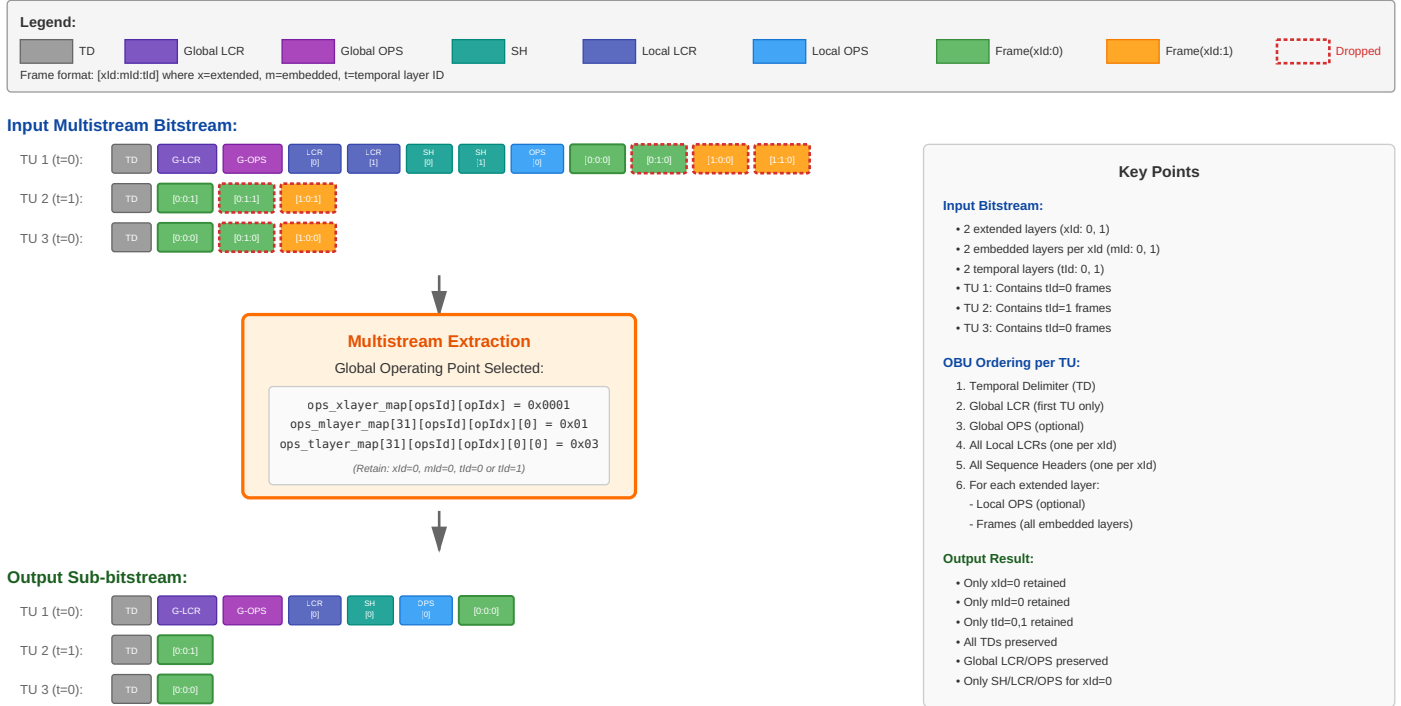
The decoder then invokes the sub-bitstream extraction process defined in [Annex F.3.2 Sub-bitstream extraction process](#) with inputs: `inputBitstream` (the entire input bitstream) and `OpRetentionMap`. The process outputs `subBitstream`.

The decoder then decodes `subBitstream` and uses the values `OpProfileIdc[xLayerId]`, `OpLevelIdc[xLayerId]`, `OpTierIdc[xLayerId]`, and `OpMlayerCnt[xLayerId]` (where `xLayerId` is the single extended layer identifier, typically 0) for conformance verification of the extended layer that is still present in the `subBitstream`.

§ **F.3. Sub-bitstream extraction processes**

The following examples illustrate the sub-bitstream extraction process for both [multistream](#) and [singlestream](#) scenarios, showing how OBUs are filtered based on the selected operating point.

Multistream Sub-bitstream Extraction Example



Note: All Local LCRs and all Sequence Headers are grouped together after Global OBUs, before per-layer Local OPS and frames.
Frames in same temporal unit from same extended/embedded layer have the same temporal layer ID.

Figure F.2: Multistream sub-bitstream extraction example showing three temporal units (TUs) with OBUs from two extended layers. The input bitstream contains properly ordered OBUs (Temporal Delimiter, Global LCR, Global OPS, followed by per-layer Local LCR, Local OPS, Sequence Header, and frames). Frames within the same temporal unit from the same extended/embedded layer have the same temporal layer ID. The extraction process retains only OBUs matching the selected operating point (xId=0, mId=0, tId=0 or 1).

Singlestream Sub-bitstream Extraction Example

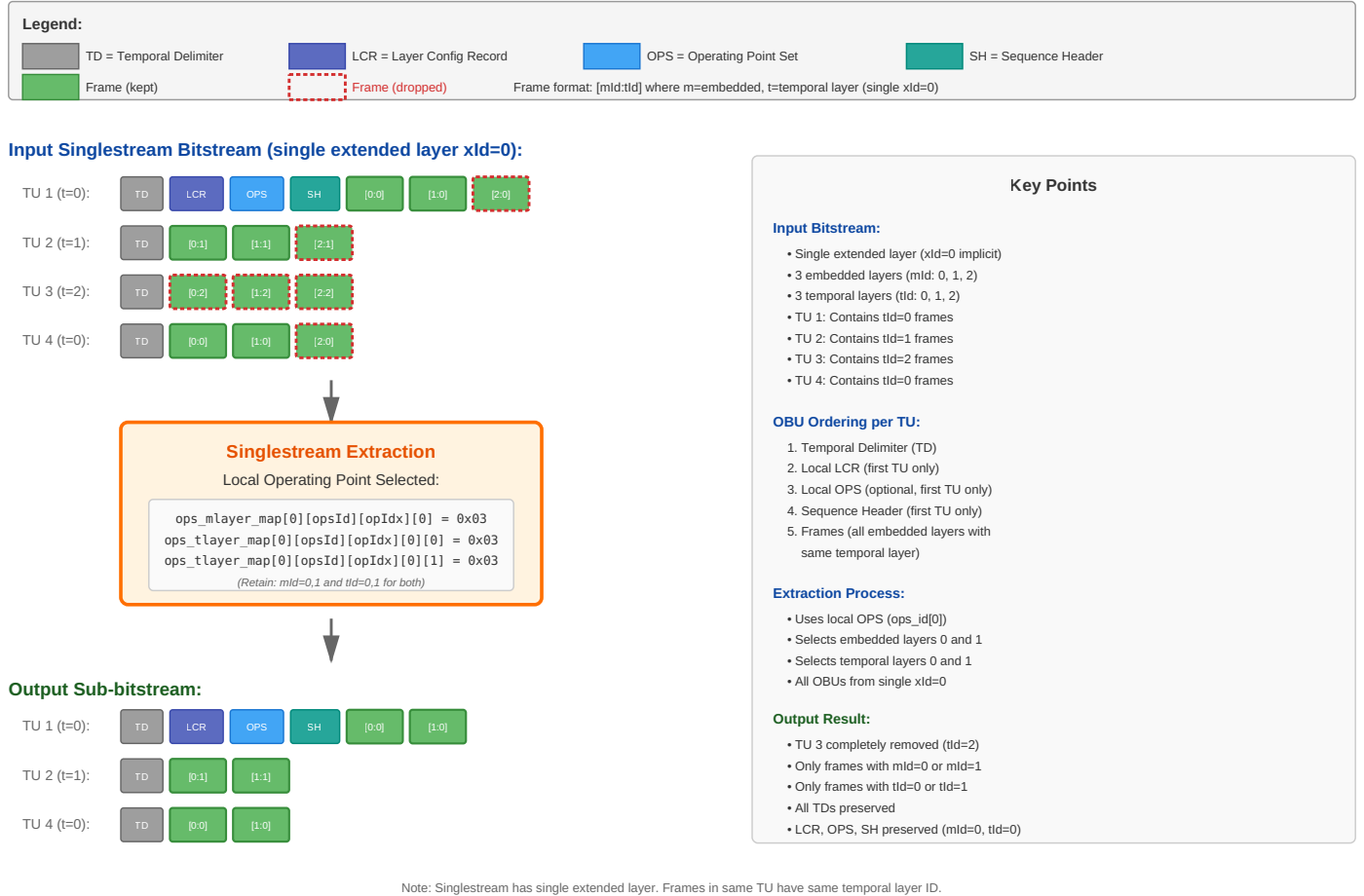


Figure F.3: Singlestream sub-bitstream extraction example showing four temporal units (TUs) with a single extended layer (xId=0 implicit). The input contains Local LCR, Local OPS, and Sequence Header in the first temporal unit, followed by frames. Each temporal unit contains frames with the same temporal layer ID. The extraction process retains only frames matching the selected embedded layers (mId=0, 1) and temporal layers (tId=0, 1), completely removing TU 3 which contains only tId=2 frames.

§ F.3.1. Operating point selection and analysis process

This process analyzes an AV2 bitstream to determine which extended layers, embedded layers, and temporal layers should be retained based on operating point selection. The process builds a 3D layer retention map and extracts profile/level/tier information for conformance verification.

The operating point selection and analysis process has the following input:

- **inputBitstream:** The bitstream to analyze

The process produces the following outputs:

- **retentionMap[32][8][4]:** Three-dimensional retention map indicating which layers to retain (1) or discard (0)
- **xLayerIsSelected[32]:** Array indicating which extended layers are selected (1) or not selected (0)

- **profileIdc[32]**: Profile identifier for each extended layer
- **levelIdc[32]**: Level identifier for each extended layer
- **tierIdc[32]**: Tier identifier for each extended layer
- **mLayerCnt[32]**: Maximum embedded layer count for each extended layer

The process for determining the retention map and profile information is as follows:

Step 1: Initialize outputs

Initialize `retentionMap[xLayerId][mLayerId][tLayerId]` with all values set to 0 for `xLayerId` from 0 to 31, `mLayerId` from 0 to 7, and `tLayerId` from 0 to 3.

Initialize `xLayerIsSelected[xLayerId]` with all values set to 0 for `xLayerId` from 0 to 31.

Initialize `profileIdc[xLayerId]`, `levelIdc[xLayerId]`, `tierIdc[xLayerId]`, and `mLayerCnt[xLayerId]` with all values set to `INVALID` for `xLayerId` from 0 to 31, where `INVALID` is a sentinel value (such as -1 for signed integer representations) that indicates the value has not been set.

Step 2: Determine bitstream type and extended layers

Examine the bitstream structure to determine whether it is a [multistream](#) or [singlestream](#) bitstream:

- If the bitstream contains two or more distinct `obu_xlayer_id` values (excluding `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`), the bitstream is [multistream](#). Determine which extended layers are present using one of the following:
 - If `MultiStreamDecoderMode` is equal to 1, use the extended layer information from `num_streams_minus_2` and `sub_xlayer_id[i]`.
 - Else if global [LCR](#) OBUs are present (`obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`), use the extended layer information from `lcr_xlayer_map` in the `lcr_global_info()`.
 - Otherwise, scan the bitstream to identify the distinct extended layer identifiers present.
- Otherwise, the bitstream is [singlestream](#). Mark only the single extended layer identifier present in the bitstream as the extended layer to process. This is typically `obu_xlayer_id = 0` for most [singlestream](#) bitstreams.

NOTE: A [multistream](#) bitstream that contains an MSDO OBU or global LCR OBUs provides structural metadata that enables the extraction process to enumerate extended layers without scanning the entire bitstream. When neither is present, the extraction process requires scanning the bitstream to identify distinct extended layer identifiers.

Additionally, examine the bitstream to determine:

- Whether global OPS OBUs are present (`obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`)
- For each extended layer, whether local OPS OBUs are present

Set the global OBU retention status in `retentionMap[GLOBAL_XLAYER_ID]`:

- For [multistream](#) bitstreams where global OBUs are present (MSDO OBU, global [LCR](#) OBUs, or global OPS OBUs), set `retentionMap[GLOBAL_XLAYER_ID][0][0] = 1` to indicate that global OBUs (`obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`) should be retained in the extracted sub-bitstream.
- Otherwise, `retentionMap[GLOBAL_XLAYER_ID][0][0]` remains 0 (initialized value).

Step 3: Global operating point selection

If one or more [global operating point sets](#) are present in the bitstream (OPS OBUs with `obu_xlayer_id` equal to `GLOBAL_XLAYER_ID`):

- Call the abstract function **`global_operating_point_selection()`**

This function represents device-specific or application-specific logic that selects a preferred global operating point based on decoder capabilities and requirements. The function returns either:

- A selected operating point identified by `globalOpsId` and `globalOpIdx`, or
- An indication to decode all extended layers without global operating point constraints

If a global operating point is selected:

- Set `xLayerMap = ops_xlayer_map[globalOpsId][globalOpIdx]`
- For each extended layer identifier `i` from 0 to 30:
 - If bit `i` is set in `xLayerMap` (i.e., $(xLayerMap \& (1 \ll i)) \neq 0$), set `xLayerIsSelected[i] = 1`
 - If bit `i` is not set, `xLayerIsSelected[i]` remains 0

If no global operating point is selected:

- If the bitstream is a [multistream](#) (as determined in Step 2), set `xLayerIsSelected[i] = 1` only for the extended layer identifiers `i` identified in Step 2
- If the bitstream is a [singlestream](#) (as determined in Step 2), set `xLayerIsSelected[i] = 1` only for the single extended layer identifier `i` identified in Step 2

Step 4: Local operating point selection and retention map construction

For each extended layer identifier `xLayerId` where `xLayerIsSelected[xLayerId] == 1`:

- Call the abstract function **`local_operating_point_selection(xLayerId)`**

This function represents device-specific or application-specific logic that determines whether to refine the embedded and temporal layers for this extended layer using a [local operating point set](#). The function returns either:

- A selected local operating point identified by `localOpsId` and `localOpIdx`, or
- An indication to decode all embedded and temporal layers for this extended layer

If a local operating point is selected for `xLayerId`:

- Set `mLayerMap = ops_mlayer_map[xLayerId][localOpsId][localOpIdx][xLayerId]`

- For each embedded layer identifier j from 0 to 7:
 - If bit j is set in $mLayerMap$ (i.e., $(mLayerMap \& (1 \ll j)) \neq 0$), then:
 - Set $tLayerMap = ops_tlayer_map[xLayerId][localOpsId][localOpIdx][xLayerId][j]$
 - For each temporal layer identifier k from 0 to 3:
 - If bit k is set in $tLayerMap$ (i.e., $(tLayerMap \& (1 \ll k)) \neq 0$), then:
 - Set $retentionMap[xLayerId][j][k] = 1$

If no local operating point is selected for $xLayerId$, set $retentionMap[xLayerId][j][k] = 1$ for all j from 0 to 7 and all k from 0 to 3 (retain all embedded and temporal layers encountered).

If a global operating point was selected in Step 3 and provides embedded/temporal layer information for $xLayerId$ (via ops_mlayer_map and ops_tlayer_map), this information may be used instead of or in combination with local operating point information, based on decoder policy.

Step 5: Extract profile, level, tier, and embedded layer count

For each extended layer identifier $xLayerId$ where $xLayerIsSelected[xLayerId] == 1$:

Determine the profile, level, tier, and maximum embedded layer count for this extended layer from the selected operating point (global or local) or from the bitstream metadata:

- If a global operating point was selected and provides operational parameters for $xLayerId$:
 - Set $profileIdx[xLayerId] = ops_seq_profile_idx[GLOBAL_XLAYER_ID][globalOpsId][globalOpIdx][xLayerId]$
 - Set $levelIdx[xLayerId] = ops_level_idx[GLOBAL_XLAYER_ID][globalOpsId][globalOpIdx][xLayerId]$
 - Set $tierIdx[xLayerId] = ops_tier_flag[GLOBAL_XLAYER_ID][globalOpsId][globalOpIdx][xLayerId]$
 - Set $mlayerCnt[xLayerId] = ops_mlayer_count[GLOBAL_XLAYER_ID][globalOpsId][globalOpIdx][xLayerId]$
- Else if a local operating point was selected for $xLayerId$:
 - Set $profileIdx[xLayerId] = ops_seq_profile_idx[xLayerId][localOpsId][localOpIdx][xLayerId]$
 - Set $levelIdx[xLayerId] = ops_level_idx[xLayerId][localOpsId][localOpIdx][xLayerId]$
 - Set $tierIdx[xLayerId] = ops_tier_flag[xLayerId][localOpsId][localOpIdx][xLayerId]$
 - Set $mlayerCnt[xLayerId] = ops_mlayer_count[xLayerId][localOpsId][localOpIdx][xLayerId]$
- Otherwise, extract the profile, level, and tier from the [LCR](#) OBU or Sequence Header OBU associated with this extended layer:
 - Set $profileIdx[xLayerId]$ from $lcr_seq_profile_idx$ (LCR OBU) or $seq_profile_idx$ (Sequence Header OBU)
 - Set $levelIdx[xLayerId]$ from $lcr_max_level_idx$ (LCR OBU) or seq_level_idx (Sequence Header OBU)
 - Set $tierIdx[xLayerId]$ from lcr_tier_flag (LCR OBU) or seq_tier (Sequence Header OBU)

- Set `mLayerCnt[xLayerId]` from `lcr_max_mlayer_count` (LCR OBU) or `seq_max_mlayer_cnt_minus_1 + 1` (Sequence Header OBU)

Step 6: Return outputs

Return `retentionMap`, `xLayerIsSelected`, `profileIdc`, `levelIdc`, `tierIdc`, and `mLayerCnt`.

§ F.3.2.Sub-bitstream extraction process

This process extracts a sub-bitstream from an AV2 bitstream by filtering OBUs based on a 3D layer retention map. The process is purely mechanical and does not make selection decisions.

The sub-bitstream extraction process has the following inputs:

- **inputBitstream**: The bitstream from which to extract a sub-bitstream
- **retentionMap[32][8][4]**: Three-dimensional retention map indicating which layers to retain

The process produces the following output:

- **subBitstream**: The extracted sub-bitstream containing only OBUs from retained layers

The process for deriving the output sub-bitstream is as follows:

Step 1: Initialize output

Set the sub-bitstream `subBitstream` to be initially identical to the input bitstream `inputBitstream`.

Step 2: Filter OBUs based on retention map

For each OBU in `subBitstream` with `obu_xlayer_id` equal to `xId`, `obu_mlayer_id` equal to `mId`, and `obu_tlayer_id` equal to `tId`:

- If the OBU type is `OBU_TEMPORAL_DELIMITER`:
 - Retain the OBU in `subBitstream` (temporal delimiters are always retained regardless of layer selection)
- Otherwise, determine if extended layer `xId` is selected by checking if there exists at least one pair (`j`, `k`) where `retentionMap[xId][j][k] == 1`. Set `isXLayerSelected` to true if such a pair exists, false otherwise.
- If `isXLayerSelected` is false:
 - Remove the OBU from `subBitstream`
- Else if `retentionMap[xId][mId][tId] == 0`:
 - If `mId == 0` and `tId == 0`:
 - Remove the OBU from `subBitstream`, **except** if the OBU type is `OBU_SEQUENCE_HEADER`, `OBU_LAYER_CONFIGURATION_RECORD`, `OBU_ATLAS_SEGMENT`, or `OBU_OPERATING_POINT_SET`
 - Otherwise:
 - Remove the OBU from `subBitstream`

Step 3: Return output

Return subBitstream.

§ F.3.3.Preserved OBU types

NOTE: The extraction processes preserve certain OBU types that contain essential configuration and metadata, even when their embedded layer identifier (`obu_mlayer_id`) or temporal layer identifier (`obu_tlayer_id`) would normally cause them to be removed.

OBU_TEMPORAL_DELIMITER OBUs are unconditionally retained in the sub-bitstream regardless of which layers are selected. Temporal delimiters mark the boundaries of temporal units and must be preserved to maintain the temporal structure of the extracted sub-bitstream.

The following OBU types are preserved when `obu_mlayer_id` is 0 and `obu_tlayer_id` is 0, **provided that their extended layer (`obu_xlayer_id`) is included in the selected operating point:**

- OBU_SEQUENCE_HEADER: Contains sequence-level parameters needed for decoding
- OBU_LAYER_CONFIGURATION_RECORD: Describes layer structure
- OBU_ATLAS_SEGMENT: Contains atlas information
- OBU_OPERATING_POINT_SET: Defines operating points

If an extended layer is not part of the selected operating point (i.e., not included in the sub-bitstream), then all OBUs with that extended layer identifier are removed, including the above OBU types. The preservation rule only applies within extended layers that are retained in the sub-bitstream.

[↑ Back to Table of Contents](#)

§ Annex G: Layer composition and Atlas usage examples (informative)

§ G.1.General

This annex provides detailed examples demonstrating how the Layer Configuration Record (LCR) works with Atlas Segments to enable complex multi-layer and multi-view content scenarios. The examples illustrate practical use cases including viewport-dependent 360-degree video streaming, subpicture composition with resampling and cropping, and region-of-interest scalability.

The **Layer Configuration Record (LCR)** provides detailed semantic metadata about each layer including its type (texture or auxiliary), purpose (alpha, depth, gain map, etc.), view association, and atlas segment mapping.

The **Atlas** provides geometric metadata: where each layer should be positioned in the final rendered output, how layers are composed spatially, and the dimensions of the virtual canvas.

Together, LCR and Atlas enable decoders and renderers to understand both *what* each layer represents semantically and *where* it should be placed geometrically. When the coded layer resolution differs from the atlas segment dimensions, resampling is required. When only a portion of the decoded layer should be used, cropping is applied before spatial mapping.

§ G.2.360-degree viewport-dependent streaming with subpictures

This example demonstrates a 360-degree video streaming application using subpicture-based viewport-dependent delivery. The equirectangular projection is divided into spatial subpictures, with the viewport region encoded at high quality and peripheral regions at lower quality. This example extends the approach to include alpha and depth auxiliary layers for each subpicture, enabling advanced rendering techniques.

Configuration:

- 9 extended layers representing different spatial subpictures in a 3×3 grid
- Subpictures arranged to completely cover 3840×1920 equirectangular projection:
 - Extended layer 0: Top-left subpicture (1280×640, low quality)
 - Extended layer 1: Top-center subpicture (1280×640, medium quality)
 - Extended layer 2: Top-right subpicture (1280×640, low quality)
 - Extended layer 3: Middle-left subpicture (1280×640, medium quality)
 - Extended layer 4: **Center viewport subpicture (1280×640, HIGH quality)**
 - Extended layer 5: Middle-right subpicture (1280×640, medium quality)
 - Extended layer 6: Bottom-left subpicture (1280×640, low quality)
 - Extended layer 7: Bottom-center subpicture (1280×640, medium quality)
 - Extended layer 8: Bottom-right subpicture (1280×640, low quality - back-facing)
- Each subpicture includes texture, alpha, and depth using embedded layers within each extended layer

- Atlas mode 0 (Enhanced Atlas) composes subpictures into complete equirectangular layout with no gaps
- The 3×3 grid provides symmetrical coverage with the viewport at the natural center position

§ G.2.1.Layer structure

Each extended layer contains three embedded layers:

- Embedded layer 0: Texture (video content)
- Embedded layer 1: Alpha channel (for subpicture blending at boundaries)
- Embedded layer 2: Depth map (for 3D-aware viewport adaptation)

Total structure:

```
Extended layer 0 (top-left subpicture):
- Embedded layer 0: Texture (1280×640, low quality)
- Embedded layer 1: Alpha (for smooth subpicture blending)
- Embedded layer 2: Depth (for parallax-aware rendering)

Extended layer 1 (top-center subpicture):
- Embedded layer 0: Texture (1280×640, medium quality)
- Embedded layer 1: Alpha
- Embedded layer 2: Depth

Extended layer 2 (top-right subpicture):
- Embedded layer 0: Texture (1280×640, low quality)
- Embedded layer 1: Alpha
- Embedded layer 2: Depth

Extended layer 3 (middle-left subpicture):
- Embedded layer 0: Texture (1280×640, medium quality)
- Embedded layer 1: Alpha
- Embedded layer 2: Depth

Extended layer 4 (center viewport subpicture):
- Embedded layer 0: Texture (1280×640, HIGH quality)
- Embedded layer 1: Alpha
- Embedded layer 2: Depth

Extended layer 5 (middle-right subpicture):
- Embedded layer 0: Texture (1280×640, medium quality)
- Embedded layer 1: Alpha
- Embedded layer 2: Depth

Extended layer 6 (bottom-left subpicture):
- Embedded layer 0: Texture (1280×640, low quality)
- Embedded layer 1: Alpha
- Embedded layer 2: Depth

Extended layer 7 (bottom-center subpicture):
- Embedded layer 0: Texture (1280×640, medium quality)
- Embedded layer 1: Alpha
- Embedded layer 2: Depth

Extended layer 8 (bottom-right subpicture - back-facing):
- Embedded layer 0: Texture (1280×640, low quality)
- Embedded layer 1: Alpha
- Embedded layer 2: Depth
```


§ G.2.2.LCR configuration

In this example, a global LCR is used, carried in the global layer context

(`obu_xlayer_id = GLOBAL_XLAYER_ID = 31`), consistent with the atlas also being signaled in the global layer context.

The LCR specifies properties for each embedded layer within each extended layer.

`lcr_local_atlas_id_present_flag` enables atlas segment assignment, and `lcr_layer_atlas_segment_id` maps each embedded layer to its target atlas segment — this is how the Enhanced Atlas knows which layer fills each segment. Multiple embedded layers within the same extended layer (texture, alpha, depth) all reference the same atlas segment, with `lcr_priority_order` controlling rendering order.

For extended layer 4 (center viewport subpicture):

```
// Extended layer 4 - Local LCR
lcr_local_atlas_id_present_flag[4] = 1 // Enable atlas segment assignment
lcr_local_atlas_id[4] = 0 // References atlas with atlas_segment_id = 0

// Extended layer 4, embedded layer 0 (center viewport texture)
lcr_layer_type[0][4][0] = TEXTURE_LAYER (0)
lcr_view_type[0][4][0] = VIEW_CENTER (1)
lcr_view_id[0][4][0] = 0
lcr_layer_atlas_segment_id[0][4][0] = 4 // Maps to atlas segment 4 (center cell)
lcr_priority_order[0][4][0] = 0 // Rendered first (base content)
lcr_rendering_method[0][4][0] = 0 // Overwrite

// Extended layer 4, embedded layer 1 (center viewport alpha)
lcr_layer_type[0][4][1] = AUX_LAYER (1)
lcr_auxiliary_type[0][4][1] = ALPHA_AUX (0)
lcr_view_type[0][4][1] = VIEW_CENTER (1)
lcr_view_id[0][4][1] = 0
lcr_layer_atlas_segment_id[0][4][1] = 4 // Same segment – auxiliary layer for boundary blending
lcr_priority_order[0][4][1] = 1
lcr_rendering_method[0][4][1] = 0 // Overwrite

// Extended layer 4, embedded layer 2 (center viewport depth)
lcr_layer_type[0][4][2] = AUX_LAYER (1)
lcr_auxiliary_type[0][4][2] = DEPTH_AUX (1)
lcr_view_type[0][4][2] = VIEW_CENTER (1)
lcr_view_id[0][4][2] = 0
lcr_layer_atlas_segment_id[0][4][2] = 4 // Same segment – auxiliary depth for 3D rendering
lcr_priority_order[0][4][2] = 2
lcr_rendering_method[0][4][2] = 0 // Overwrite

// Similar LCR configuration for extended layers 0-3, 5-8 (other subpictures)
// Each subpicture's embedded layers map to their respective atlas segments
```

For extended layer 0 (top-left subpicture):

```
// Extended layer 0 - Local LCR
lcr_local_atlas_id_present_flag[0] = 1
lcr_local_atlas_id[0] = 0

// Extended layer 0, embedded layer 0 (top-left subpicture texture)
lcr_layer_type[0][0][0] = TEXTURE_LAYER (0)
lcr_view_type[0][0][0] = VIEW_CENTER (1)
lcr_view_id[0][0][0] = 0 // Same view, different spatial region
lcr_layer_atlas_segment_id[0][0][0] = 0 // Maps to atlas segment 0 (top-left cell)
lcr_priority_order[0][0][0] = 0
lcr_rendering_method[0][0][0] = 0 // Overwrite

// Extended layer 0, embedded layer 1 (top-left subpicture alpha)
lcr_layer_type[0][0][1] = AUX_LAYER (1)
lcr_auxiliary_type[0][0][1] = ALPHA_AUX (0)
```

```

lcr_view_type[0][0][1] = VIEW_CENTER (1)
lcr_view_id[0][0][1] = 0
lcr_layer_atlas_segment_id[0][0][1] = 0 // Same segment as texture
lcr_priority_order[0][0][1] = 1
lcr_rendering_method[0][0][1] = 0 // Overwrite

// Extended layer 0, embedded layer 2 (top-left subpicture depth)
lcr_layer_type[0][0][2] = AUX_LAYER (1)
lcr_auxiliary_type[0][0][2] = DEPTH_AUX (1)
lcr_view_type[0][0][2] = VIEW_CENTER (1)
lcr_view_id[0][0][2] = 0
lcr_layer_atlas_segment_id[0][0][2] = 0 // Same segment as texture
lcr_priority_order[0][0][2] = 2
lcr_rendering_method[0][0][2] = 0 // Overwrite

// Configurations for extended layers 1-3, 5-8 follow same pattern

```

Key observations:

- All subpictures share the same `lcr_view_id = 0` (single 360-degree view)
- Extended layers differentiate spatial subpictures
- Embedded layers within each extended layer differentiate content types (texture, alpha, depth)
- Alpha channels enable smooth blending at subpicture boundaries
- Depth maps enable motion-parallax and 3D-aware rendering

§ G.2.3. Atlas configuration

The atlas uses mode 0 (enhanced atlas) with a 3×3 uniform grid that completely covers the 3840×1920 equirectangular projection with no gaps. All 9 cells are the same size (1280×640), so `ats_uniform_spacing_flag = 1` applies. With `ats_single_region_per_atlas_segment_flag = 1`, each of the 9 grid regions maps one-to-one to a segment (segments 0-8 in row-major order). The LCR's `lcr_layer_atlas_segment_id` for each embedded layer references these segment IDs —no stream IDs appear in the atlas itself. Assuming the atlas is in the global layer context (`xlayerId = GLOBAL_XLAYER_ID = 31`) and `atlas_segment_id = 0`:

```

// atlas_segment_info_obu() - OBU with obu_xlayer_id = GLOBAL_XLAYER_ID (31)
atlas_segment_id[31] = 0 // xAId = 0
ats_atlas_segment_mode_idc[0] = 0 // ENHANCED_ATLAS

// ats_enhanced_atlas_info(xAId=0) [wrapper defined in companion normative PR] calls ats_region_info then
// ats_region_to_segment_mapping:

// ats_region_info(xAId=0): 3×3 uniform grid
ats_num_region_columns_minus_1[0] = 2 // 3 columns
ats_num_region_rows_minus_1[0] = 2 // 3 rows
ats_uniform_spacing_flag[0] = 1 // Uniform spacing (all cells equal size)
ats_region_width_minus_1[0] = 1279 // Each region: 1280 pixels wide
ats_region_height_minus_1[0] = 639 // Each region: 640 pixels tall
// AtlasWidth = 1280 × 3 = 3840, AtlasHeight = 640 × 3 = 1920

// ats_region_to_segment_mapping(xAId=0): 1-to-1 mapping (each region = one segment)
ats_single_region_per_atlas_segment_flag[0] = 1
// ats_num_atlas_segments_minus_1[0] = 8 (inferred: NumRegionsInAtlas - 1 = 9 - 1 = 8)

// Segment IDs are assigned implicitly in row-major order (left-right, top-bottom):
// Segment 0: region (col=0, row=0) → top-left canvas position (0, 0), 1280×640
// Segment 1: region (col=1, row=0) → top-center canvas position (1280, 0), 1280×640
// Segment 2: region (col=2, row=0) → top-right canvas position (2560, 0), 1280×640
// Segment 3: region (col=0, row=1) → middle-left canvas position (0, 640), 1280×640
// Segment 4: region (col=1, row=1) → CENTER canvas position (1280, 640), 1280×640
// Segment 5: region (col=2, row=1) → middle-right canvas position (2560, 640), 1280×640

```

```
// Segment 6: region (col=0, row=2) → bottom-left canvas position (0, 1280), 1280×640
// Segment 7: region (col=1, row=2) → bottom-center canvas position (1280, 1280), 1280×640
// Segment 8: region (col=2, row=2) → bottom-right canvas position (2560, 1280), 1280×640
```

The center viewport subpicture (extended layer 4) maps to segment 4 via `lcr_layer_atlas_segment_id = 4`. Note that this example uses 9 extended layers, which requires LCR (not MSDO) since MSDO is limited to a maximum of 4 independent streams.

§ G.2.4. Viewport-dependent streaming process

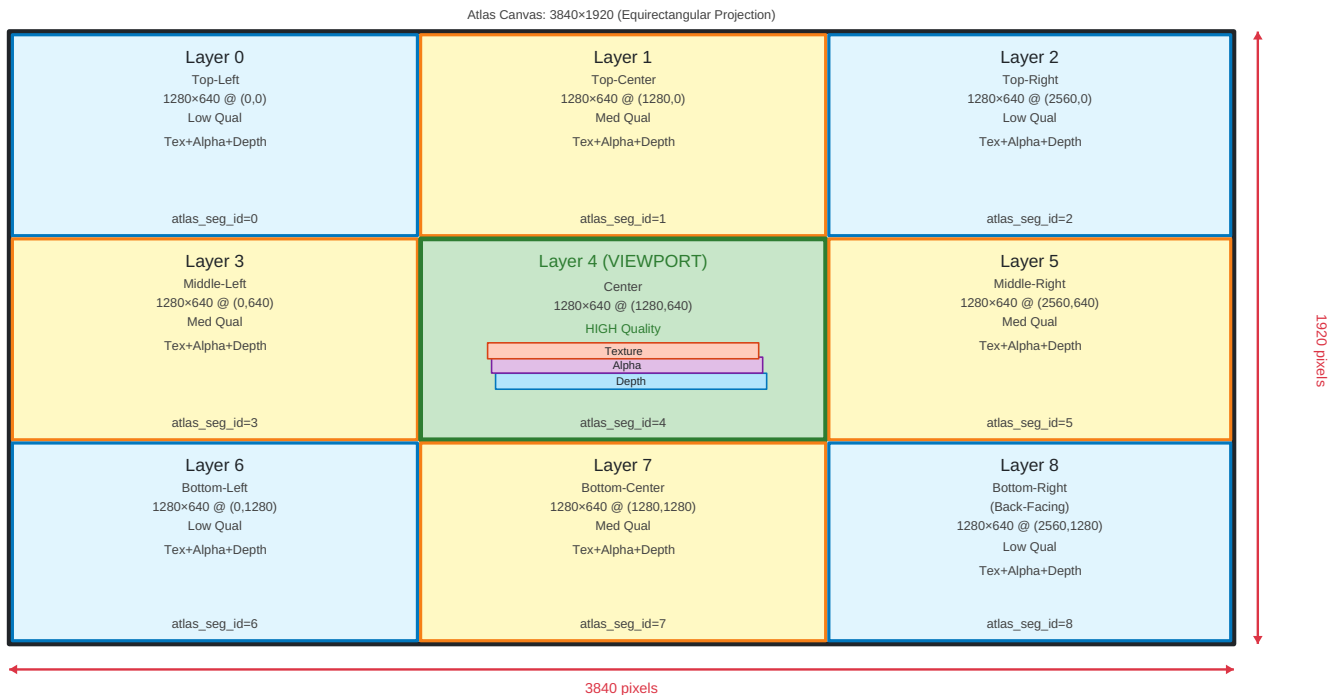
1. **Initial state:** Client detects user's head orientation/gaze direction
2. **Viewport determination:** Based on orientation, client determines which subpictures are visible:
 - Front-facing (0°): Extended layer 4 (center viewport) at high priority
 - Immediately adjacent subpictures: Extended layers 1, 3, 5, 7 at medium priority
 - Corner and back-facing subpictures: Extended layers 0, 2, 6, 8 at lower priority
3. **Adaptive fetching:**
 - **High bandwidth:** Fetch all 9 extended layers (complete sphere coverage)
 - Center viewport subpicture (extended layer 4): high quality
 - Adjacent subpictures (extended layers 1, 3, 5, 7): medium quality
 - Corner/back subpictures (extended layers 0, 2, 6, 8): low quality
 - **Medium bandwidth:** Fetch center + immediately adjacent visible subpictures
 - Skip corner and back-facing subpictures until user rotates toward them
 - **Low bandwidth:** Fetch center viewport subpicture only
 - Decoder synthesizes peripheral regions from viewport using depth map
4. **Rendering with alpha and depth:**
 - **Texture layers:** Provide base video content for each subpicture
 - **Alpha channels:** Enable smooth blending at subpicture boundaries
 - Prevents visible seams between subpictures in the 3×3 grid
 - Allows feathering for quality transitions
 - **Depth maps:** Enable advanced rendering:
 - Motion parallax compensation for head translation
 - View synthesis for missing subpictures (depth-image-based rendering)
 - Foveated rendering (higher quality in gaze direction)
 - Occlusion-aware composition for overlaid UI elements
5. **Head motion tracking:**
 - When user rotates head, client dynamically switches which extended layers are fetched
 - Smooth transition enabled by alpha blending between subpictures

- Depth maps allow temporal interpolation during subpicture switches
- Complete 3×3 grid coverage with center viewport ensures content available for any viewing direction

§ G.2.5. Benefits for 360-degree streaming

- **Bandwidth efficiency:** Only fetch subpictures in or near viewport, reducing bandwidth by 50-80% compared to full-sphere streaming
- **Quality adaptation:** Viewport receives high quality, periphery receives lower quality
- **Smooth transitions:** Alpha channels eliminate subpicture boundary artifacts
- **3D-aware rendering:** Depth maps enable parallax, view synthesis, and occlusion handling
- **Scalable:** Can adjust number of subpictures (extended layers) based on content complexity
- **Low latency:** Viewport subpicture can be prioritized for fast initial display
- **Subpicture independence:** Each extended layer is independently decodable without dependencies on other subpictures

360° Viewport-Dependent Streaming: 3×3 Grid with Center Viewport



Symmetrical 3×3 Grid with Center Viewport Layout: 9 subpictures in 3 columns × 3 rows (1280×640 each) completely cover 3840×1920 canvas Center viewport: Layer 4 at position (1,1) encoded at HIGH quality for front-facing view Adjacent subpictures: Layers 1, 3, 5, 7 (top/left/right/bottom of center) at medium quality Corner subpictures: Layers 0, 2, 6, 8 at low quality (peripheral and back-facing regions) Perfect symmetry: Equal granularity horizontally and vertically; natural center positioning Complete coverage: No gaps; content available for any viewing direction	Layer Configuration (LCR Required - 9 Layers) Each extended layer contains 3 embedded layers: - Embedded layer 0: Texture (TEXTURE_LAYER) - video content - Embedded layer 1: Alpha (AUX_LAYER, ALPHA_AUX) - smooth blending - Embedded layer 2: Depth (AUX_LAYER, DEPTH_AUX) - parallax rendering All subpictures share lcr_view_id = 0 (single 360° view) MSDO cannot be used (limited to 4 streams); requires LCR for 9 layers
Key Benefits of 3×3 Grid for 360° Streaming - Symmetrical coverage: 3×3 grid provides equal horizontal and vertical granularity (better than 3×2 or 2×3) - Natural center viewport: Center subpicture (1,1) is the obvious front-facing position surrounded by 8 adjacent subpictures - Complete corner coverage: All 4 corners explicitly covered (0=top-left, 2=top-right, 6=bottom-left, 8=bottom-right) - Bandwidth efficiency: Fetch high-quality center + visible adjacent subpictures only (60-80% reduction vs full sphere) - Quality adaptation: Center at high (1280×640), adjacent at medium, corners at low - matches viewport importance - Smooth transitions: Alpha channels eliminate visible seams; enables quality feathering between subpictures 3D-aware rendering: Depth maps enable motion parallax, view synthesis for missing subpictures, occlusion handling - Dynamic streaming: As user rotates, fetch different subpictures with priority based on viewing direction - Better aspect ratio: 1280×640 subpictures (~2:1) more natural for equirectangular content than taller subpictures - Independent decoding: Each extended layer independently decodable; no inter-subpicture dependencies - Scalable quality: Can adjust per-subpicture encoding parameters based on importance without changing structure	

Figure G.1: 360-degree viewport-dependent streaming using subpictures arranged in a 3×3 grid. Nine extended layers completely cover the 3840×1920 equirectangular projection with perfect symmetry and no gaps. The center viewport subpicture (extended layer 4, position 1,1) is encoded at high quality (1280×640) for the front-facing view. Immediately adjacent subpictures (layers 1, 3, 5, 7) use medium quality, while corner and back-facing subpictures (layers 0, 2, 6, 8) use low quality. Each subpicture contains three embedded layers: texture, alpha (for smooth blending), and depth (for parallax and view synthesis). The symmetrical 3×3 grid layout ensures complete sphere coverage with natural center viewport positioning, so content is available regardless of viewing direction. Alpha channels eliminate subpicture boundary artifacts, while depth maps enable 3D-aware rendering and motion parallax compensation.

§ G.3.Subpicture composition example

This example demonstrates a video conferencing application where multiple video sources (participants) are composed into a single virtual canvas. The atlas acts as a virtual screen layout manager, positioning different layers at different locations to create a multi-party conferencing view. This scenario uses three extended layers representing three participants, with one participant requiring resampling:

- Extended layer 0: Main speaker (high resolution, 1280×1080)
- Extended layer 1: Participant 2 (encoded at 480×360, upsampled to 640×540)
- Extended layer 2: Participant 3 (medium resolution, 640×540)

§ G.3.1.LCR configuration

Each extended layer has its own local LCR. The `lcr_local_atlas_id_present_flag` enables atlas segment assignment, and `lcr_layer_atlas_segment_id` explicitly maps each embedded layer to its target atlas segment. This is the mechanism by which the Enhanced Atlas knows which layer provides content for each segment — there are no stream IDs in the atlas itself.

```
// Extended layer 0 (main speaker) - Local LCR
lcr_local_atlas_id_present_flag[0] = 1 // Enable atlas segment assignment for this layer
lcr_local_atlas_id[0] = 0 // References atlas with atlas_segment_id = 0
lcr_layer_type[0][0][0] = TEXTURE_LAYER (0)
lcr_view_type[0][0][0] = VIEW_CENTER (1)
lcr_layer_atlas_segment_id[0][0][0] = 0 // Maps to atlas segment 0 (full left column)
lcr_priority_order[0][0][0] = 0 // Single layer per segment; priority order not critical
lcr_rendering_method[0][0][0] = 0 // Overwrite

// Extended layer 1 (participant 2) - Local LCR
lcr_local_atlas_id_present_flag[1] = 1
lcr_local_atlas_id[1] = 0
lcr_layer_type[0][1][0] = TEXTURE_LAYER (0)
lcr_view_type[0][1][0] = VIEW_CENTER (1)
lcr_layer_atlas_segment_id[0][1][0] = 1 // Maps to atlas segment 1 (top-right cell)
lcr_priority_order[0][1][0] = 0
lcr_rendering_method[0][1][0] = 0 // Overwrite

// Extended layer 2 (participant 3) - Local LCR
lcr_local_atlas_id_present_flag[2] = 1
lcr_local_atlas_id[2] = 0
lcr_layer_type[0][2][0] = TEXTURE_LAYER (0)
lcr_view_type[0][2][0] = VIEW_CENTER (1)
lcr_layer_atlas_segment_id[0][2][0] = 2 // Maps to atlas segment 2 (bottom-right cell)
lcr_priority_order[0][2][0] = 0
lcr_rendering_method[0][2][0] = 0 // Overwrite
```

NOTE: each segment here has exactly one layer assigned to it. However, the Enhanced Atlas allows multiple layers to reference the same segment. For example, if participant 2 also had an alpha channel layer (for chroma-key compositing), that layer would set `lcr_layer_atlas_segment_id = 1` as well, with `lcr_rendering_method` controlling how it composites with the texture layer already mapped to segment 1.

A global LCR describes the overall structure:

```
// Global LCR (obu_xlayer_id = 31)
lcr_global_config_record_id = 1
lcr_xlayer_map = 0x07 // Extended layers 0, 1, 2 present (bits 0-2 set)
```

```
lcr_global_purpose_id = 6 // Multiview Playback
lcr_global_atlas_id_present_flag = 1
lcr_global_atlas_id = 0 // References the global atlas
```

§ G.3.2. Atlas configuration

The global atlas (`obu_xlayer_id = 31`) uses mode 0 (enhanced atlas) to define the layout as a 2-column × 2-row non-uniform grid. Unlike the multistream atlas, no stream IDs appear in the atlas itself — stream-to-segment assignment is handled entirely by `lcr_layer_atlas_segment_id` in each layer's LCR. The three participants map naturally to three grid-derived segments: the main speaker occupies the full left column (both rows merged into one segment), while each participant occupies one right-column cell.

```
// atlas_segment_info_obu() - OBU with obu_xlayer_id = GLOBAL_XLAYER_ID (31)
atlas_segment_id[31] = 0 // xAId = 0
ats_atlas_segment_mode_idc[0] = 0 // ENHANCED_ATLAS

// ats_enhanced_atlas_info(xAId=0) [wrapper defined in companion normative PR] calls ats_region_info then
ats_region_to_segment_mapping:

// ats_region_info(xAId=0): 2x2 non-uniform grid
ats_num_region_columns_minus_1[0] = 1 // 2 columns
ats_num_region_rows_minus_1[0] = 1 // 2 rows
ats_uniform_spacing_flag[0] = 0 // Non-uniform spacing
ats_column_width_minus_1[0][0] = 1279 // Column 0: 1280 pixels (main speaker)
ats_column_width_minus_1[0][1] = 639 // Column 1: 640 pixels (participants)
ats_row_height_minus_1[0][0] = 539 // Row 0: 540 pixels
ats_row_height_minus_1[0][1] = 539 // Row 1: 540 pixels
// AtlasWidth = 1280 + 640 = 1920, AtlasHeight = 540 + 540 = 1080

// ats_region_to_segment_mapping(xAId=0)
ats_single_region_per_atlas_segment_flag[0] = 0 // Not 1-to-1: main speaker spans 2 rows
ats_num_atlas_segments_minus_1[0] = 2 // 3 segments

// Segment 0: Main speaker (left column, spans both rows → 1280×1080)
// Canvas top-left: (0, 0)
ats_top_left_region_column[0][0] = 0
ats_top_left_region_row[0][0] = 0
ats_bottom_right_region_column_off[0][0] = 0 // Remains in column 0
ats_bottom_right_region_row_off[0][0] = 1 // Extends to row 1

// Segment 1: Participant 2 (top-right cell → 640×540)
// Canvas top-left: (1280, 0)
ats_top_left_region_column[0][1] = 1
ats_top_left_region_row[0][1] = 0
ats_bottom_right_region_column_off[0][1] = 0
ats_bottom_right_region_row_off[0][1] = 0

// Segment 2: Participant 3 (bottom-right cell → 640×540)
// Canvas top-left: (1280, 540)
ats_top_left_region_column[0][2] = 1
ats_top_left_region_row[0][2] = 1
ats_bottom_right_region_column_off[0][2] = 0
ats_bottom_right_region_row_off[0][2] = 0
```

Segment dimensions and canvas positions are derived from the cumulative column widths and row heights. The segment IDs (0, 1, 2) are assigned implicitly by index since `ats_signaled_atlas_segment_ids_flag` is not set. These IDs are what each layer's `lcr_layer_atlas_segment_id` references.

§ G.3.3. Rendering and adaptive streaming

The renderer composes the final view by:

1. Creating a 1920×1080 canvas filled with the background color (if specified)

2. Decoding each extended layer independently:

- Extended layer 0 → decoded to 1280×1080 (matches atlas segment)
- Extended layer 1 → decoded to 480×360 (requires resampling)
- Extended layer 2 → decoded to 640×540 (matches atlas segment)

3. Resampling for resolution mismatch (Extended layer 1):

- Decoded resolution: 480×360
 - Target atlas segment: 640×540
 - Resampling required: upscale by factor of 4/3 horizontally and 3/2 vertically
 - The resampling process is implementation-dependent and outside the scope of this specification. One example approach:
 1. Initialize resampled frame buffers (640×540 for Y plane, with appropriate chroma dimensions based on subsampling format)
 2. For each output sample position (x, y) in the resampled frame:
 - Calculate corresponding input position: $\text{inputX} = x \times (\text{inputWidth} / \text{outputWidth})$, $\text{inputY} = y \times (\text{inputHeight} / \text{outputHeight})$
 - Apply interpolation filter (e.g., bilinear, bicubic, or Lanczos) using neighboring input samples
 - Store result in resampled frame buffer
 3. Repeat for U and V chroma planes with subsampling-aware calculations
 - **Note:** This is one possible implementation. Decoders may use different resampling algorithms (nearest-neighbor, bilinear, bicubic, Lanczos, learned upsampling, etc.) based on quality-performance tradeoffs
- ## 4. Positioning decoded (and resampled) content according to atlas layout:
- Layer 0 at position (0, 0) with size 1280×1080 (cumulative: col 0 start, rows 0-1 span)
 - Layer 1 (after resampling to 640×540) at position (1280, 0) (cumulative: col 0 width=1280, row 0 start)
 - Layer 2 at position (1280, 540) with size 640×540 (cumulative: col 0 width=1280, row 0 height=540)
- ## 5. Compositing all layers onto the canvas to produce the final 1920×1080 output

Adaptive streaming benefits: This structure enables intelligent bandwidth adaptation:

- On bandwidth constraints, selectively decode layers (e.g., only main speaker if needed)
- Different layers can have different quality/resolution/framerate
- Layout can be reconfigured dynamically by sending new atlas OBUs
- Participants can join/leave by adding/removing extended layers

Selective decoding: A mobile client with limited screen space might:

- Only decode extended layer 0 (main speaker) and display it full-screen
- Skip decoding layers 1 and 2 to save bandwidth and power
- The decoder knows this is valid because each extended layer is independently decodable

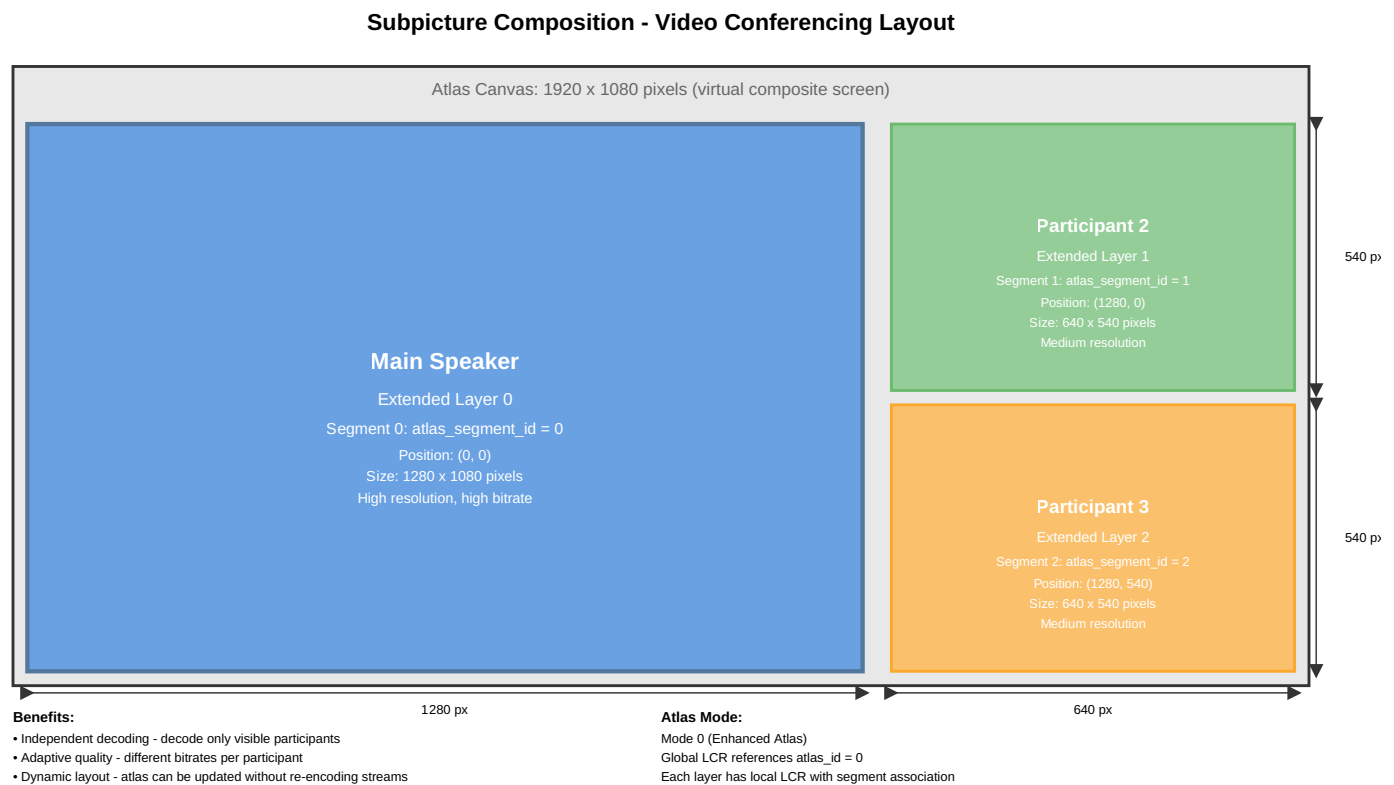


Figure G.2: Subpicture composition for video conferencing. The atlas defines a 1920x1080 virtual canvas with three segments: main speaker (1280x1080) at left, and two participants (640x540 each) positioned on the right. Each segment maps to an independent extended layer that can be selectively decoded.

§ G.4. Region-of-interest scalability example with encoder padding

This example demonstrates a stadium sports broadcast where a high-resolution field-of-play region is encoded separately from lower-resolution audience/stadium context. Additionally, this example shows how encoder padding and normative cropping work when the encoder needs to operate on dimensions that differ from the display resolution for hardware or algorithmic reasons.

The content uses two extended layers:

- Extended layer 0: Full stadium view at base quality (1920x1080 display, coded as 1920x1088 with padding)
- Extended layer 1: Field-of-play region at high quality (1280x720)

§ G.4.1.LCR configuration

```
// Extended layer 0 (base layer - full stadium with padding and cropping)
lcr_local_atlas_id_present_flag[0] = 1 // Enable atlas segment assignment
lcr_local_atlas_id[0] = 0 // References atlas with atlas_segment_id = 0
lcr_layer_type[0][0][0] = TEXTURE_LAYER (0)
lcr_view_type[0][0][0] = VIEW_CENTER (1)
lcr_layer_atlas_segment_id[0][0][0] = 0 // Maps to atlas segment 0 (full 1920×1080 canvas)
lcr_priority_order[0][0][0] = 0 // Rendered first (background)
lcr_rendering_method[0][0][0] = 0 // Overwrite

// Encoder padding and cropping for extended layer 0:
// - Original video: 1920×1080
// - Encoder operates on: 1920×1088 (padded to align with 64×64 superblocks)
// - Cropping window removes padding to produce 1920×1080 output
lcr_max_pic_width[0][0] = 1920
lcr_max_pic_height[0][0] = 1088 // Coded height (with padding)
lcr_cropping_window_present_flag[0][0] = 1
lcr_cropping_win_left_offset[0][0] = 0
lcr_cropping_win_right_offset[0][0] = 0
lcr_cropping_win_top_offset[0][0] = 0
lcr_cropping_win_bottom_offset[0][0] = 8 // Remove 8 pixels of bottom padding

// After cropping: 1920×1080 (matches atlas segment 0 dimensions)

// Extended layer 1 (enhancement - field detail, no padding needed)
lcr_local_atlas_id_present_flag[1] = 1
lcr_local_atlas_id[1] = 0
lcr_layer_type[0][1][0] = TEXTURE_LAYER (0)
lcr_view_type[0][1][0] = VIEW_CENTER (1)
lcr_layer_atlas_segment_id[0][1][0] = 1 // Maps to atlas segment 1 (center cell, 1280×720)
lcr_priority_order[0][1][0] = 1 // Rendered second (overlays base in center region)
lcr_rendering_method[0][1][0] = 0 // Overwrite (replaces base layer data in center)
lcr_max_pic_width[0][1] = 1280
lcr_max_pic_height[0][1] = 720
lcr_cropping_window_present_flag[0][1] = 0 // No cropping needed
```

Note on cropping semantics:

- Cropping is normative and must be applied to generate the conformant output
- The cropped dimensions determine what maps to the atlas segment
- Cropping happens after any upscaling specified by `lcr_max_pic_width/height`
- The purpose is encoder implementation convenience (e.g., superblock alignment), not bandwidth savings
- All coded samples (including padding) must be signaled and decoded

§ G.4.2.Atlas configuration

The atlas uses mode 0 (enhanced atlas) with a 3-column × 3-row non-uniform grid sized so the center cell exactly matches the field-of-play region (1280×720 at position 320,180). Two segments are defined: segment 0 spans all 9 grid regions (full 1920×1080 canvas) and segment 1 spans only the center cell. Segments 0 and 1 overlap on the center region; the LCR's `lcr_priority_order` values (base=0, enhancement=1) control rendering order so the field enhancement overwrites the base layer in that region.

```
// atlas_segment_info_obu() - OBU with obu_xlayer_id = GLOBAL_XLAYER_ID (31)
atlas_segment_id[31] = 0 // xAId = 0
ats_atlas_segment_mode_idc[0] = 0 // ENHANCED_ATLAS

// ats_enhanced_atlas_info(xAId=0) [wrapper defined in companion normative PR] calls ats_region_info then
```

```

ats_region_to_segment_mapping:

// ats_region_info(xAid=0): 3x3 non-uniform grid
// Columns: 320 + 1280 + 320 = 1920, Rows: 180 + 720 + 180 = 1080
ats_num_region_columns_minus_1[0] = 2    // 3 columns
ats_num_region_rows_minus_1[0] = 2       // 3 rows
ats_uniform_spacing_flag[0] = 0           // Non-uniform spacing
ats_column_width_minus_1[0][0] = 319     // Column 0: 320 px (left border)
ats_column_width_minus_1[0][1] = 1279    // Column 1: 1280 px (field width)
ats_column_width_minus_1[0][2] = 319     // Column 2: 320 px (right border)
ats_row_height_minus_1[0][0] = 179       // Row 0: 180 px (top border)
ats_row_height_minus_1[0][1] = 719       // Row 1: 720 px (field height)
ats_row_height_minus_1[0][2] = 179       // Row 2: 180 px (bottom border)
// AtlasWidth = 320+1280+320 = 1920, AtlasHeight = 180+720+180 = 1080

// ats_region_to_segment_mapping(xAid=0)
ats_single_region_per_atlas_segment_flag[0] = 0
ats_num_atlas_segments_minus_1[0] = 1    // 2 segments

// Segment 0: Full frame base layer (all 9 regions → 1920×1080)
ats_top_left_region_column[0][0] = 0
ats_top_left_region_row[0][0] = 0
ats_bottom_right_region_column_off[0][0] = 2    // Spans to column 2
ats_bottom_right_region_row_off[0][0] = 2       // Spans to row 2

// Segment 1: Field-of-play enhancement (center cell only → 1280×720)
// Canvas top-left: x=320 (col 0 width), y=180 (row 0 height)
ats_top_left_region_column[0][1] = 1
ats_top_left_region_row[0][1] = 1
ats_bottom_right_region_column_off[0][1] = 0
ats_bottom_right_region_row_off[0][1] = 0

```

§ G.4.3. Rendering scenarios

Decoding and cropping process for extended layer 0:

1. Decode extended layer 0 to produce a 1920×1088 frame (coded dimensions)
2. **Apply normative cropping** as specified in LCR:
 - Input frame: 1920×1088
 - Cropping window: left=0, right=0, top=0, bottom=8
 - Output frame calculation:

```

cropWidth = lcr_max_pic_width - (lcr_cropping_win_left_offset + lcr_cropping_win_right_offset)
           = 1920 - (0 + 0) = 1920
cropHeight = lcr_max_pic_height - (lcr_cropping_win_top_offset + lcr_cropping_win_bottom_offset)
            = 1088 - (0 + 8) = 1080

```

- Cropped output: 1920×1080
3. The cropped frame (1920×1080) is what maps to atlas segment 0

Full quality rendering (high bandwidth):

1. Decode extended layer 0 (produces 1920×1088, normatively cropped to 1920×1080)
2. Decode extended layer 1 (produces 1280×720, no cropping)
3. Render layer 0 as background (1920×1080)

4. Place the field enhancement (segment 1) at position (320, 180) — derived from the atlas grid: x = column 0 width = 320 px, y = row 0 height = 180 px. The field enhancement overwrites the base layer in this region (`lcr_rendering_method = 0`, `lcr_priority_order = 1`)
5. Result: Full 1920×1080 output with high-quality field region

Why use padding and cropping:

- **Hardware alignment:** Many hardware encoders require dimensions aligned to specific boundaries (e.g., 64×64 superblocks, 32×32 transform units)
- **Algorithmic efficiency:** Some encoding algorithms work more efficiently on certain dimension multiples
- **Example:** 1920×1080 → pad to 1920×1088 (17 rows of 64-pixel superblocks), then crop back
- **Not for bandwidth savings:** All padded samples must still be coded and transmitted

Bandwidth-constrained rendering:

- Option A: Decode only layer 0 (with cropping applied) → full stadium at 1920×1080 base quality
- Option B: Decode only layer 1 → high-quality field-of-play (1280×720) without stadium context

Atlas mapping considerations:

- Atlas segment dimensions reference the **cropped** output dimensions, not the coded dimensions
- In this example: atlas segment 0 is 1920×1080 (after cropping), not 1920×1088
- Cropping must be applied before spatial mapping to the atlas canvas

Region-of-Interest Scalable Encoding - Sports Broadcast



Figure G.3: Region-of-interest scalable encoding for sports broadcast. Extended layer 0 provides full stadium view at base quality (1920×1080 after normative cropping from coded 1920×1088 dimensions). Extended layer 1 provides high-quality 1280×720 field-of-play that overlays the center region. This example demonstrates encoder padding (8 pixels for superblock alignment) with LCR cropping window to produce conformant output. Decoders can selectively decode layers based on viewport and bandwidth.

§ G.5.Implementation considerations

§ G.5.1.Decoder requirements

Decoders implementing LCR and atlas support should:

1. Parse and validate LCR metadata:

- Verify layer type and auxiliary type combinations are valid
- Check view ID consistency across layers belonging to same view
- Validate atlas segment ID references

2. Parse and interpret atlas layout:

- Support all required atlas modes
- Calculate final canvas dimensions and segment positions
- Handle segment overlays correctly (later segments may overlay earlier ones)

3. Selective decoding:

- Use Operating Point Set (OPS) information in combination with LCR to determine which layers are required for a given operating point
- Support independent decoding of extended layers
- Implement bandwidth-adaptive layer selection based on LCR metadata

4. Multi-view rendering:

- Group layers by `lcr_view_id` for multi-view display
- Associate auxiliary data (alpha, depth, gain map) with correct texture layers
- Support stereoscopic display modes when `VIEW_LEFT`/`VIEW_RIGHT` layers are present

§ G.5.2.Encoder recommendations

Encoders should:

1. Choose appropriate layer structure:

- Use extended layers for independently decodable streams (different views, different regions)
- Use embedded layers for scalability within a single view (quality/temporal scalability)
- Balance granularity vs. overhead (more layers = more flexibility but more metadata)

2. Populate LCR metadata accurately:

- Set `lcr_layer_type` and `lcr_auxiliary_type` to reflect actual content
- Use consistent `lcr_view_id` values for layers belonging to same view
- Associate layers with appropriate atlas segments via `lcr_layer_atlas_segment_id`

3. Design atlas layouts efficiently:

- Choose atlas mode appropriate for use case (mode 0/1 for regular grids, mode 2/3 for flexible layouts)
- Minimize canvas size to reduce padding and memory requirements
- Consider decoder memory constraints when designing segment layouts

4. Provide Operating Point Sets:

- Define OPS entries for common playback scenarios (mono vs. stereo, with/without depth, different quality levels)
- Include profile/tier/level information in OPS for conformance checking
- Reference atlas segments in OPS where applicable

§ G.5.3.Interoperability

For maximum interoperability:

1. Legacy decoder fallback:

- Ensure extended layer 0, embedded layer 0 contains playable base content

- Decoders that ignore LCR/atlas should still get reasonable output
- Use sequence header to signal when advanced features are required

2. Progressive enhancement:

- Structure layers so additional data enhances rather than replaces base content
- Design atlas layouts that degrade gracefully if not all segments are decoded

3. Signaling and discovery:

- Use content interpretation metadata (CIMD) to signal presence of stereo/depth/HDR
- Include sufficient LCR information for clients to discover available views and auxiliary data types
- Document expected rendering behavior in supplementary information

§ Index

§ Terms defined by this specification

[AC coefficient](#), in § 2

[ADST](#), in § 2

[AOMedia](#), in § 2

[Atlas](#), in § 2

[Base layer](#), in § 2

[BAWP](#), in § 2

[Bitstream](#), in § 2

[Bit string](#), in § 2

[blocks](#), in § 2

[Bridge frame](#), in § 2

[BRU](#), in § 2

[Byte](#), in § 2

[Byte alignment](#), in § 2

[CCSO](#), in § 2

[CCTX](#), in § 2

[CDEF](#), in § 2

[CDE](#), in § 2

[CFL](#), in § 2

[Chroma](#), in § 2

[CLK](#), in § 2

[Closed random access](#), in § 2

[Coded frame](#), in § 2

[Coded multistream video sequence](#), in § 2

[coded video sequences](#), in § 2

[Component](#), in § 2

[Compound prediction](#), in § 2

[DC coefficient](#), in § 2

[DCT](#), in § 2

[DDT](#), in § 2

[Deblocking filter](#), in § 2

[Decoded frame](#), in § 2

[Decoder](#), in § 2

[Decoding process](#), in § 2

[Dequantization](#), in § 2

[Embedded layer](#), in § 2

[Encoder](#), in § 2

[Encoding process](#), in § 2

[Enhancement layer](#), in § 2

[EOB](#), in § 2

[extended layers](#), in § 2

[Frame](#), in § 2

[Frame context](#), in § 2

[Frame header info](#), in § 2

[frame_header_info](#), in § 2

[FSC](#), in § 2

[GDF](#), in § 2

[Global operating point set](#), in § 2

[IBP](#), in § 2

[Inter coding](#), in § 2

[Inter frame](#), in § 2

[Interoperability point](#), in § A.2

[Inter prediction](#), in § 2

[Intra coding](#), in § 2

[Intra frame](#), in § 2

[Intra prediction](#), in § 2

[Inverse transform](#), in § 2

[IST](#), in § 2

[Key frame](#), in § 2

[Layer](#), in § 2

[LCR](#), in § 2

[Leading frame](#), in § 2

[Level](#), in § 2

[LF](#), in § 2

[Local operating point set](#), in § 2

Long-term reference frame , in § 2	Prediction , in § 2	Sequence , in § 2
LSB , in § 2	Prediction process , in § 2	Singlestream , in § 2
Luma , in § 2	Prediction value , in § 2	Sub-bitstream , in § 2
MHCCP , in § 2	Profile , in § 2	Sub-bitstream extraction process , in § 2
Mode info , in § 2	Quantization parameter , in § 2	Superblock , in § 2
Mode info block , in § 2	Quantized coefficient , in § 2	Switch Frame , in § 2
Motion vector , in § 2	Random access switch , in § 2	Syntax element , in § 2
MRL , in § 2	RAS , in § 2	TCQ , in § 2
MSB , in § 2	Raster scan , in § 2	Temporal delimiter OBU , in § 2
MSDO , in § 2	Reconstruction , in § 2	Temporal layer , in § 2
multi-sequence configuration , in § A.3	Reference , in § 2	temporal units , in § 2
Multistream , in § 2	Reference frame , in § 2	TG , in § 2
OBU , in § 2	Regular frame , in § 2	Tier , in § 2
OLK , in § 2	Reserved , in § 2	Tile , in § 2
Open random access , in § 2	Residual , in § 2	Tile Group , in § 2
OPS , in § 2	Sample , in § 2	TIP , in § 2
Parity hiding , in § 2	Sample value , in § 2	Transform block , in § 2
Parse , in § 2	SDP , in § 2	Transform coefficient , in § 2
Picture , in § 2	SEF , in § 2	WAIP , in § 2
	Segmentation map , in § 2	WHT , in § 2

§ References

§ Normative References

[CTA-861]

A DTV Profile for Uncompressed High Speed Digital Interfaces (ANSI/CTA-861-J). standard. URL: <https://www.cta.tech/standards/a-dtv-profile-for-uncompressed-high-speed-digital-interfaces/>

[RFC1321]

R. Rivest. *The MD5 Message-Digest Algorithm*. April 1992. Informational. URL: <https://www.rfc-editor.org/rfc/rfc1321>

[RFC2119]

S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. March 1997. Best Current Practice. URL: <https://datatracker.ietf.org/doc/html/rfc2119>

§ Informative References

[ITU-R-BT.601]

Recommendation ITU-R BT.601-7 (03/2011), Studio encoding parameters of digital television for standard 4:3 and wide screen 16:9 aspect ratios. 8 March 2011. Recommendation. URL: <https://www.itu.int/rec/R-REC-BT.601/>

[ITU-R-BT.709]

Recommendation ITU-R BT.709-6 (06/2015), Parameter values for the HDTV standards for production and international programme exchange. 17 June 2015. Recommendation. URL: <https://www.itu.int/rec/R-REC-BT.709/>

[Rec.2020]

Recommendation ITU-R BT.2020-2: Parameter values for ultra-high definition television systems for production and international programme exchange. October 2015. URL: <http://www.itu.int/rec/R-REC-BT.2020/en>